
i.MX 8M Dual/8M QuadLite/8M Quad Applications Processors Reference Manual

Document Number: IMX8MDQLQRM
Rev. 3.1, 06/2021





Contents

Section number	Title	Page
Chapter 1		
Introduction		
1.1	Product Overview	7
1.2	Target Applications.....	7
1.3	Acronyms and Abbreviations.....	7
1.4	Features.....	10
1.5	Architectural Overview.....	16
1.6	Primary Boot Options.....	17
1.7	Endianness Support.....	18
Chapter 2		
Memory Map		
2.1	Memory.....	19
Chapter 3		
Security		
3.1	System Security.....	33
3.2	Resource Domain Controller (RDC).....	35
Chapter 4		
ARM Platform and Debug		
4.1	ARM Cortex A53 Platform (A53).....	79
4.2	ARM Cortex M4 Platform (M4).....	85
4.3	Messaging Unit (MU).....	117
4.4	Semaphore (SEMA4).....	156
4.5	On-Chip RAM Memory Controller (OCRAM).....	189
4.6	Network Interconnect Bus System (NOC).....	192
4.7	AHB to IP Bridge (AIPSTZ).....	193
4.8	Shared Peripheral Bus Arbiter (SPBA).....	214
4.9	ROM Controller with Patch (ROMCP).....	228
4.10	Watchdog Timer (WDOG).....	241

Section number	Title	Page
4.11	System Counter (SYS_CTR).....	258
4.12	TrustZone Address Space Controller (TZASC).....	296
4.13	System Debug.....	298
4.14	System JTAG Controller (SJC).....	302

Chapter 5 Clocks and Power Management

5.1	Clock Control Module (CCM).....	333
5.2	General Power Controller (GPC).....	657
5.3	Crystal Oscillator (XTALOSC).....	796
5.4	Thermal Management Unit (TMU).....	800

Chapter 6 SNVS, Reset, Fuse, and Boot

6.1	System Boot.....	831
6.2	Fusemap.....	900
6.3	On-Chip OTP Controller (OCOTP_CTRL).....	912
6.4	Secure Non-Volatile Storage (SNVS).....	940
6.5	System Reset Controller (SRC).....	974

Chapter 7 Interrupts and DMA

7.1	Interrupts and DMA Events.....	1041
7.2	Smart Direct Memory Access Controller (SDMA).....	1056

Chapter 8 Chip IO and Pinmux

8.1	External Signals and Pin Multiplexing.....	1301
8.2	IOMUX Controller (IOMUXC).....	1344
8.3	General Purpose Input/Output (GPIO).....	1847

Chapter 9 External Memory

9.1	External Memory Overview.....	1867
9.2	DRC Performance Monitor (DRC_PERF_MON).....	1869

Section number	Title	Page
9.3	DDR Controller (DDRC).....	1883
9.4	DDR PHY (DDR_PHY).....	2082
9.5	AHB-to-APBH Bridge with DMA (APBH-Bridge-DMA).....	2388
9.6	62BIT Correcting ECC Accelerator (BCH).....	2426
9.7	General Purpose Media Interface (GPMI).....	2483

Chapter 10 Mass Storage

10.1	Enhanced Configurable SPI (ECSPI).....	2535
10.2	Quad Serial Peripheral Interface (QuadSPI).....	2564
10.3	Ultra Secured Digital Host Controller (uSDHC).....	2669

Chapter 11 Connectivity

11.1	Universal Serial Bus Controller (USB).....	2801
11.2	Universal Serial Bus PHY (USB_PHY).....	3091
11.3	PCI Express (PCIe).....	3095
11.4	PCI Express PHY (PCIe_PHY).....	3477
11.5	Ethernet MAC (ENET).....	3670

Chapter 12 Timers

12.1	General Purpose Timer (GPT).....	3825
12.2	Pulse Width Modulation (PWM).....	3849

Chapter 13 Multimedia

13.1	Multimedia Overview.....	3863
13.2	Enhanced LCD Interface (eLCDIF).....	3870
13.3	Graphics Processing Unit (GPU).....	3935
13.4	HD Display Transmitter Controller (HDMI TX).....	3943
13.5	HDMI / DisplayPort Transmit PHY (HDMI PHY).....	5036
13.6	MIPI DSI Host Controller (MIPI_DSI).....	5051
13.7	CSI Bridge (CSI).....	5131

Section number	Title	Page
13.8	MIPI CSI Host Controller (MIPI_CSI).....	5168
13.9	Sony/Philips Digital Interface (SPDIF).....	5213
13.10	Synchronous Audio Interface (SAI).....	5245

Chapter 14 Video Processing Unit (VPU)

14.1	VPU G1 (VPU_G1).....	5293
14.2	VPU G2 (VPU_G2).....	5510

Chapter 15 Display Controller Subsystem (DCSS)

15.1	Display Controller Subsystem.....	5763
15.2	DCSS Block Control (BLK_CTL).....	5775
15.3	Display Timing Generator (DTG).....	5796
15.4	Context Load (CTX_LD).....	5847
15.5	Graphics Decompression (DEC400D).....	5870
15.6	Decompression and Tile to Raster Conversion (DTRC).....	5905
15.7	Display, Prefetch and Resolve (DPR).....	5949
15.8	Scaler (MED_DC_SCALE).....	6069
15.9	Look Up Table Load (LUT_LD).....	6102
15.10	HDR10 Image Processing (MED_HDR10).....	6110
15.11	Color Sub-Sampler (SUBSAM).....	6351
15.12	Write Scale (WR_SCL).....	6409
15.13	Read Surface (RD_SRC).....	6423
15.14	Interrupt Request Steering (IRQ_STEER).....	6437

Chapter 16 Low Speed Communication and Interconnects

16.1	I2C Controller (I2C).....	6449
16.2	Universal Asynchronous Receiver/Transmitter(UART).....	6472

Chapter 1

Introduction

1.1 Product Overview

This chapter introduces the architecture of the i.MX 8M Dual/8M QuadLite/8M Quad Applications Processors.

The i.MX 8M Dual/8M QuadLite/8M Quad is a family of products focused on delivering an excellent 4K video and audio experience, combining media-specific features with high-performance processing optimized for low-power consumption.

1.2 Target Applications

The i.MX 8M Dual/8M QuadLite/8M Quad Applications Processors achieve both high performance and low power consumption and rely on a powerful, fully-coherent core complex based on a quad Cortex-A53 cluster, with graphics processing GPU supporting the latest graphic APIs.

The i.MX 8M Family provides additional computing resources and peripherals:

- Advanced security modules for secure boot, cipher acceleration and DRM support
- General purpose Cortex-M4 processor for low power processing
- A wide range of audio interfaces including I2S, AC97, TDM and S/PDIF
- Large set of peripherals that are commonly used in consumer/industrial markets including USB 3.0, PCIe and Ethernet

1.3 Acronyms and Abbreviations

The table below contains acronyms and abbreviations used in this document.

Acronyms and Abbreviated Terms

Acronyms and Abbreviations

Term	Meaning
ADC	Analog-to-Digital Converter
AHB	Advanced High-performance Bus
AIPS	Arm IP Bus
ALU	Arithmetic Logic Unit
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
ASRC	Asynchronous Sample Rate Converter
AXI	Advanced eXtensible Interface
BIST	Built-In Self Test
CA/CM	Arm Cortex-A/Cortex-M
CAAM	Cryptographic Acceleration and Assurance Module
CA53	ARM Cortex A53 Core
CAN	Controller Area Network
CCM	Clock Controller Module
CPU	Central Processing Unit
CSI	CMOS Sensor Interface
CSU	Central Security Unit
CTI	Cross Trigger Interface
DAP	Debug Access Port
DDR	Double data rate
DMA	Direct memory access
DPLL	Digital phase-locked loop
DRAM	Dynamic random access memory
ECC	Error correcting codes
ECSPI	Enhanced Configurable SPI
LPSPi	Low-power SPI
EDMA	Enhanced Direct Memory Access
EIM	External Interface Module
ENET	Ethernet
EPIT	Enhanced Periodic Interrupt Timer
EPROM	Erasable Programmable Read-Only Memory
ETF	Embedded Trace FIFO
ETM	Embedded Trace Macrocell
FIFO	First-In-First-Out
GIC	General Interrupt Controller
GPC	General Power Controller
GPIO	General-Purpose I/O
GPR	General-Purpose Register
GPS	Global Positioning System
GPT	General-Purpose Timer
GPU	Graphics Processing Unit

Table continues on the next page...

Term	Meaning
GPV	Global Programmers View
HAB	High-Assurance Boot
I-cache	Instruction cache
I2C or I ² C	Inter-Integrated Circuit
IC	Integrated Circuit
IEEE	Institute of Electrical and Electronics Engineers
IOMUX	Input-Output Multiplexer
IP	Intellectual Property
IrDA	Infrared Data Association
JTAG	Joint Test Action Group (a serial bus protocol usually used for test purposes)
LCDIF	Liquid Crystal Display Interface
LDO	Low-Dropout
LIFO	Last-In-First-Out
LRU	Least-Recently Used
LSB	Least-Significant Byte
LUT	Look-Up Table
LVDS	Low Voltage Differential Signaling
MAC	Medium Access Control
MCM	Miscellaneous control Module
MMC	Multimedia Card
MSB	Most-Significant Byte
MT/s	Mega Transfers per second
OCRAM	On-Chip Random-Access Memory
OCOTP	On-Chip One-Time Programmable Controller
PCI	Peripheral Component Interconnect
PCIe	PCI express
PCMCIA	Personal Computer Memory Card International Association
PGC	Power Gating Controller
PIC	Programmable Interrupt Controller
PMU	Power Management Unit
POR	Power-On Reset
PSRAM	Pseudo-Static Random Access Memory
PWM	Pulse Width Modulation
PXP	Pixel Pipeline
QoS	Quality of Service
R2D	Radians to Degrees
RISC	Reduced Instruction Set Computing
ROM	Read-Only Memory
ROMCP	ROM Controller with Patch
RTOS	Real-Time Operating System
Rx	Receive

Table continues on the next page...

Features

Term	Meaning
SAI	Synchronous Audio Interface
SCU	Snoop Control Unit
SD	Secure Digital
SDIO	Secure Digital Input/Output
SDLC	Synchronous Data Link Control
SDMA	Smart DMA
SIM	Subscriber Identification Module
SNVS	Secure Non-Volatile Storage
SoC	System-on-Chip
SPBA	Shared Peripheral Bus Arbiter
SPDIF	Sony Phillips Digital Interface
SPI	Serial Peripheral Interface
SRAM	Static Random-Access Memory
SRC	System Reset Controller
TFT	Thin-Film Transistor
TPIU	Trace Port Interface
TSGEN	Time Stamp Generator
Tx	Transmit
TZASC	TrustZone Address Space Controller
UART	Universal Asynchronous Receiver/Transmitter
USB	Universal Serial Bus
USDHC	Ultra Secured Digital Host Controller
WDOG	Watchdog
WLAN	Wireless Local Area Network
WXGA	Wide Extended Graphics Array

1.4 Features

1.4.1 Arm Cortex-A53 MPCore™ Platform

The i.MX 8M Family Applications Processors are based on the Arm Cortex-A53 MPCore™ Platform, which has the following features:

- Quad symmetric Cortex-A53 processors, including:
 - 32 KB L1 Instruction Cache
 - 32 KB L1 Data Cache

- Media Processing Engine (MPE) with NEON technology supporting the Advanced Single Instruction Multiple Data architecture
- Floating Point Unit (FPU) with support of the VFPv4-D16 architecture
- Support of 64-bit Armv8-A architecture
- 1 MB unified L2 cache
- Target frequency of 1.4GHz

1.4.2 Arm Cortex-M4 Platform

Cortex-M4 Core Platform include the following:

- Low power microcontroller available for customer application:
 - Low power standby mode
 - IoT features including Weave
 - Manage IR or wireless remote
- Arm Cortex M4 CPU Processor, including:
 - 16 KB L1 Instruction Cache
 - 16 KB L1 Data Cache
 - 256 KB TCM
 - Target frequency of 266MHz

1.4.3 System Bus and Interconnect

System bus and interconnect include the following:

- Network interconnect (NoC) AXI arbiter
- Quality of service controller (QoSC) to configure priorities and limits of AXI transactions
- Performance monitor (PERFMON) to monitor AXI bus activity
- Debug monitor (DBGMON) to record AXI transactions preceding a system reset

1.4.4 Clocking and Resets

Clocking and resets include:

- Clock control module (CCM) provides centralized clock generation and control
 - Simplified clock tree structure

- Unified clock programming model for each clock root
- Multicore awareness for resource domains
- System reset controller (SRC) provides reset generation and distribution

1.4.5 Interrupts and DMA

Interrupts and DMA include:

- 128 shared peripheral interrupts routed to Cortex-A53 Global Interrupt Controller (GIC) and Cortex-M4 nested vector interrupt controller (NVIC) for flexible interrupt handling
- Two Smart direct memory access (SDMA) engines. Although these two engines are identical to each other, they are integrated into the processor to serve different peripherals. Each SDMA controller supports 48 DMA requests

1.4.6 On-Chip Memory

The on-chip memory system consists of the following:

- Boot ROM (96KB)
- On-chip RAM (128KB + 32KB)

1.4.7 External Memory Interface

The external memory interfaces supported on this chip include:

- 16/32-bit DRAM Interface:
 - PoP LPDDR4-3200
 - Non-PoP LPDDR4-3200, DDR4-2400, DDR3L-1600
- 8-bit NAND FLASH, including support for Raw MLC/SLC devices, BCH ECC up to 62-bit, and ONFi3.2 compliance (clock rates up to 100 MHz and data rates up to 200 MB/sec)
- eMMC 5.1 FLASH (2 interfaces)
- SPI NOR FLASH (3 interfaces)
- FlexSPI FLASH with support for XIP (for M4 in low-power mode) and parallel read mode of two identical FLASH devices

1.4.8 Timers

The timers on this chip include:

- One local generic timer integrated into each Cortex-A53 CPU
- Global system counter with timer bus interface to Cortex-A53 MPCore generic timers
- One local system timer (SysTick) integrated into the Cortex-M4 CPU
- Six general purpose timer (GPT) modules
- Three watchdog timer (WDOG) modules
- Four pulse width modulation (PWM) modules

1.4.9 Graphics Processing Unit (GPU)

The chip incorporates the following Graphics Processing Unit (GPU) features:

- 4 shader
- Supports OpenGL ES 3.1, 3.0, 2.0, 1.0, OpenCL
- Target frequency of 800 MHz
- Frame Buffer Compression – Lossless compression of buffers
- TrustZone support using a local MMU to manage secure regions

1.4.10 Video Processing Unit (VPU)

The chip incorporates the following Video Processing Unit (VPU) features:

- VP9 Profile 0, 2 (10 bit) decoder (VPU G2)
- HEVC/H.265 Main, Main10 decoder (VPU G2)
- AVC/H.264 Baseline, Main, High decoder (VPU G1)
- VP8 decoder (VPU G1)
- Frame Buffer Compression – Lossless compression of buffers
- TrustZone support

1.4.11 Display Interfaces

The chip has the following display support:

- LCD Interface with MIPI-DSI Output:
 - MIPI-DSI 4 channels supporting one display. Supports resolution up to 1920x1080p60, 1800x1200p60

- LCDIF display controller
- Output can be LCDIF output or Display Controller output (turns off HDMI)
- Two MIPI-CSI2 Display Interfaces:
 - Each MIPI-CSI2 is 4 channels supporting one camera input
 - Supports 4Kp30, 1080p60/30, 5MP at 15 fps, 720p60, VGA at 60 fps
 - Maximum bit rate of 1.5 Gbps

1.4.12 Audio

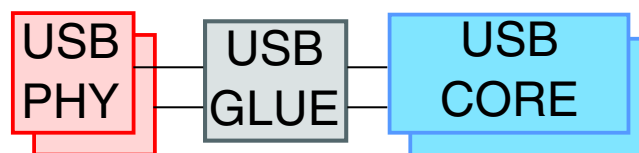
Audio include the following:

- S/PDIF Input and Output, including a Raw Capture input mode
- Five SAI (synchronous audio interface) modules supporting I2S, AC97, TDM, and codec/DSP interfaces, including one SAI with 8 TX and 8 RX lanes, one SAI with 4 TX and 4 RX lanes, and three SAI with 1 TX and 1 RX lanes. Supports over 20 channels of audio
- One internal SAI port with 4 I2S lanes to drive HDMI audio output
- One internal S/PDIF interface to capture HDMI ARC at up to 192kHz Fs

1.4.13 General Connectivity Interfaces

The chip contains a rich set of general connectivity interfaces, including:

- Two PCI Express (PCIe):
 - Single lane supporting PCIe Gen 2
 - Dual mode operation to function as root complex or endpoint
 - Integrated PHY interface
 - Supports L1 substate
- Two USB 3.0/2.0 OTG controllers with integrated PHY interface



- USB host mode, support SS/HS/FS/LS
- USB device mode, support SS/HS/FS
- USB2.0 OTG mode, ADP/SRP/HNP
- Battery charger
- Spread spectrum clock support

- Two Ultra Secure Digital Host Controller (uSDHC) interfaces
 - MMC 5.1 compliance with HS400 DDR signaling to support up to 400 MB/sec
 - SD/SDIO 3.01 compliance with 200 MHZ SDR signaling to support up to 100 MB/sec
 - Support for SDXC (extended capacity)
- One Gigabit Ethernet controller with support for IEEE, Ethernet AVB and IEEE1588
- Four universal asynchronous receiver/transmitter (UART) modules
- Four I2C modules
- Three SPI modules

1.4.14 Security

Security functions are enabled and accelerated by the following hardware:

- RDC – Resource Domain Controller:
 - Supports 4 domains and up to 8 regions
- Arm TrustZone including the TZ architecture:
 - ARM Cortex-A53 MPCore TrustZone support
- On-chip RAM (OCRAM) secure region protection using OCRAM controller
- High Assurance Boot (HAB)
- Cryptographic Acceleration and Assurance Module (CAAM)
 - Support Widevine and PlayReady content protection
 - Public Key Cryptography (PKHA) with RSA and Elliptic Curve (ECC) algorithms
 - Real-time integrity checker (RTIC)
 - DRM support for RSA, AES, 3DES, DES
 - Side channel attack resistance
 - True random number generation (RNG)
 - Manufacturing protection support
- Secure Non-Volatile Storage (SNVS), including
 - Secure Real Time Clock (SRTC)
- Secure JTAG Controller (SJC)

1.4.15 Multicore Support

Multicore support contains:

- Resource domain controller (RDC) to support isolation and safe sharing of system resources

- Messaging unit (MU)
- Hardware Semaphore (SEMA42)
- Shared bus topology

1.4.16 GPIO and Pin Multiplexing

- General-purpose input/output (GPIO) modules with interrupt capability
- Input/output multiplexing controller (IOMUXC) to provide centralized pad control

1.4.17 Power Management

The power management unit consists of:

- Temperature sensor with programmable trip points
- Flexible power domain partitioning with internal power switches to support efficient power management

1.4.18 System Debug

The system debug features are:

- ARM CoreSight debug and trace architecture
- Trace Port Interface Unit (TPIU) to support off-chip real-time trace
- Embedded Trace FIFO (ETF) with 4 KB internal storage to provide trace buffering
- Unified trace capability for Quad Cortex-A53 and Cortex-M4 CPUs
- Cross Triggering Interface (CTI)
- Support for 5-pin (JTAG) debug interfaces

1.5 Architectural Overview

This section contains the i.MX 8M Dual/8M QuadLite/8M Quad architectural details.

1.5.1 Block Diagram

The high-level block diagram is shown in the figure below.

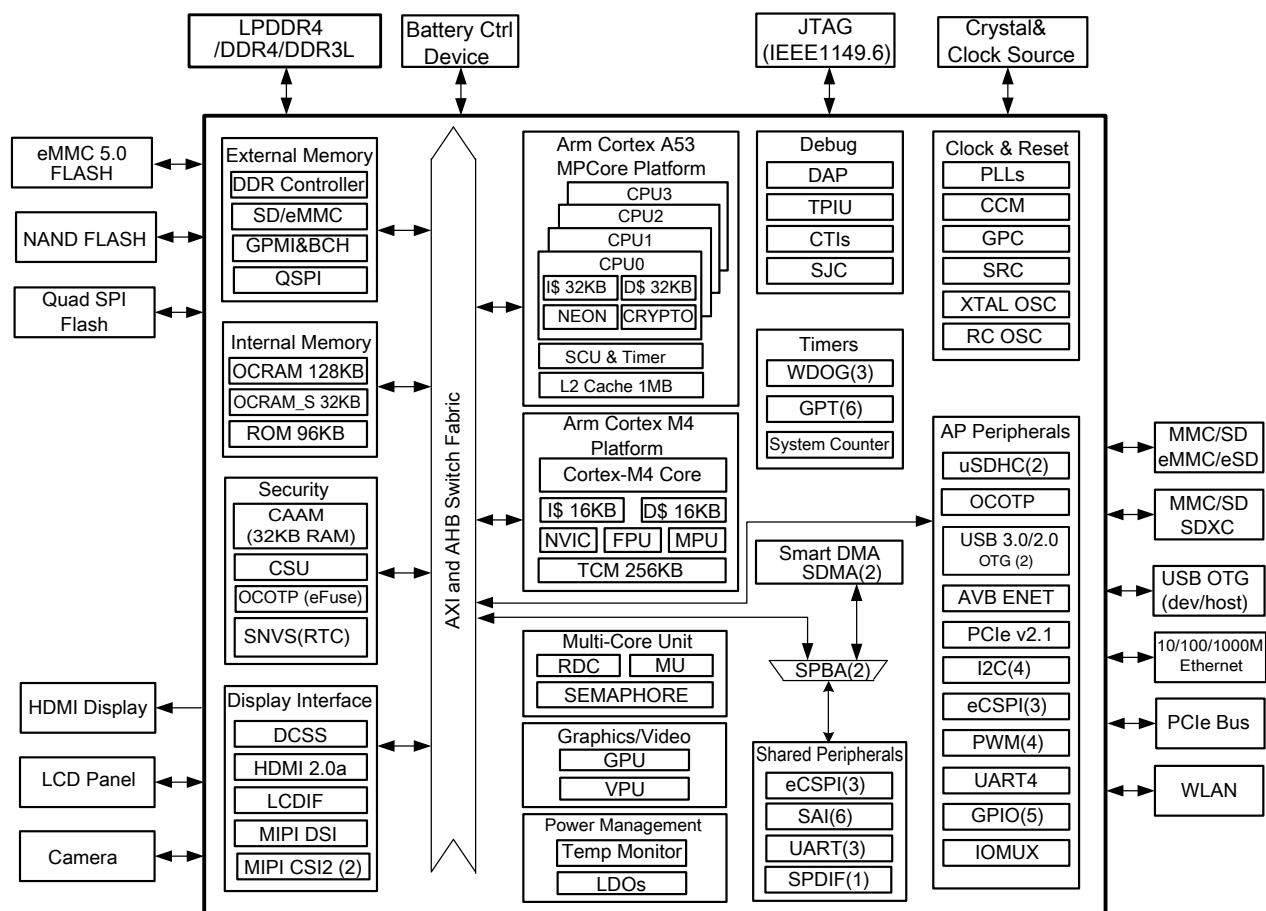


Figure 1-1. Block Diagram

1.6 Primary Boot Options

The i.MX 8M Dual/8M QuadLite/8M Quad supports the following boot devices:

- NAND FLASH (including SLC and MLC)
- SDIO / MMC / SDXC
- eSD 3.0/eMMC 5.1 (fast boot)
- SPI (serial FLASH)
- USB

- QSPI
- Ethernet (via plug-in mode)

The Quad-A53 core on i.MX 8M Dual/8M QuadLite/8M Quad is enabled during boot as the primary core to handle the entire secure boot flow. The chip will always boot from the A53 core first, the M4 core will be held in reset during the A53 boot and won't run until it is enabled by the A53 core. The image for the M4 core will be loaded into memory and authenticated by the A53 core.

1.7 Endianness Support

This chip supports Little Endian mode only.

Chapter 2

Memory Map

2.1 Memory

This chapter introduces the memory architecture of the chip. The system memory high-level partition is defined below:

2.1.1 Memory system overview

2.1.1.1 On-chip L1, L2 caches, TCM

Cortex-A53 MPcore Platform

- Level 1 Cache (4x per Cortex-A53 Core)
 - Instruction (32 KB)
 - Data (32 KB)
- Level 2 Cache, shared by the four Cortex-A53 cores:
 - Unified instruction and data (1 MB)

Cortex M4 Platform

- Cache
 - Instruction (16 KB)
 - Data (16 KB)
- Tightly-Coupled-Memory
 - TCML on Code Bus (128KB)
 - TCMH on System Bus (128KB)

2.1.1.2 On-chip memories

- Boot ROM (128 KB)

- On-Chip RAM - OCRAM (128 KB)
- On-Chip RAM for State Retention - OCRAM_S (32 KB)

2.1.1.3 External L3 memories

The chip supports external memories, via the following memory interfaces / controllers:

- 16/32-bit DRAM Interface:
 - PoP LPDDR4-3200, Non-PoP LPDDR4-3200, DDR4-2400, DDR3L-1600
- 8-bit NAND FLASH, including support for Raw MLC/SLC devices, BCH ECC up to 62-bit, and ONFI 2.x compliance (clock rates up to 100 MHz and data rates up to 200 MB/sec)
- eMMC 5.0 FLASH
- SPI NOR FLASH
- Quad SPI FLASH with support for XIP (for M4 in low-power mode) and parallel read mode of two identical FLASH devices

2.1.2 Cortex-A53 Memory Map

Start Address	End Address	Region	Size	Description
1_0000_0000	1_3FFF_FFFF	DDR Address	1024MB	DDR Memory (Quad-A53 only)
4000_0000	FFFF_FFFF	DDR Address	3072MB	DDR Memory (All modules)
3E00_0000	3FFF_FFFF	Reserved	32MB	Reserved
3DC0_0000	3DFF_FFFF	DDRC	4MB	Reserved
3D80_0000	3DBF_FFFF		4MB	DDR PERF_MON
3D40_0000	3D7F_FFFF		4MB	DDR CTL
3C00_0000	3D3F_FFFF		20MB	DDR PHY
3890_0000	3BFF_FFFF	Reserved	55MB	Reserved
3880_0000	388F_FFFF	GIC	1MB	GIC REG
3870_0000	387F_FFFF	Reserved	1MB	Reserved
3830_0000	384F_FFFF	VPU	2MB	VPU_Decoder
3820_0000	382F_FFFF	USB	1MB	USB2 REG
3810_0000	381F_FFFF		1MB	USB1 REG
3801_0000	380F_FFFF	Reserved	960KB	Reserved
3800_0000	3800_FFFF	GPU REG	64KB	GPU REG
3600_0000	37FF_FFFF	QSPI RX Buffers	32MB	Reserved
3400_0000	35FF_FFFF		32MB	QSPI1 RX Buffer
3390_0000	33FF_FFFF	Reserved	7.9MB	Reserved
3381_4000	3381_FFFF	PCIe REG	48KB	Reserved

Table continues on the next page...

Start Address	End Address	Region	Size	Description
3381_0000	3381_3FFF		4MB	PCIe2 REG
3380_4000	3380_FFFF		48KB	Reserved
3380_0000	3380_3FFF		4MB	PCIe1 REG
3310_0000	337F_FFFF	MMAP Peipherals	7960KB	Reserved
3300_8000	3300_FFFF		32KB	Reserved (APBH DMA)
3300_0000	3300_7FFF		32KB	APBH DMA
32C0_0000	32FF_FFFF	Periph (AIPS)	4MB	AIPS4
3280_0000	32BF_FFFF	Reserved	4MB	Reserved
3270_0000	327F_FFFF	GPV_7	1MB	NoC configuration
3260_0000	326F_FFFF	GPV_6	1MB	"m4" configuration port
3250_0000	325F_FFFF	GPV_5	1MB	"display" configuration port
3240_0000	324F_FFFF	GPV_4	1MB	"enet" configuration port
3230_0000	323F_FFFF	GPV_3	1MB	"per_m" configuration port
3220_0000	322F_FFFF	GPV_2	1MB	"per_s" configuration port
3210_0000	321F_FFFF	GPV_1	1MB	"wakeup" configuration port
3200_0000	320F_FFFF	GPV_0	1MB	"main" configuration port
3140_0000	31FF_FFFF	Reserved	12MB	Reserved
3100_0000	313F_FFFF	ARM Peripherals	4MB	Reserved
30C0_0000	30FF_FFFF	Periph (AIPS)	4MB	Reserved
3080_0000	30BF_FFFF		4MB	AIPS-3, See IP listing on separate map.
3040_0000	307F_FFFF		4MB	AIPS-2, See IP listing on separate map.
3000_0000	303F_FFFF		4MB	AIPS-1, See IP listing on separate map.
2900_0000	2FFF_FFFF	Reserved	112MB	Reserved
2800_0000	28FF_FFFF	A53 / DAP	16MB	A53 / DAP
2000_0000	27FF_FFFF	PCIe-2	128MB	PCIe-2
1800_0000	1FFF_FFFF	PCIe-1	128MB	PCIe-1
0800_0000	17FF_FFFF	QSPI	256MB	QSPI
00B0_0000	07FF_FFFF	Reserved	117MB	Reserved
0092_0000	00AF_FFFF	OCRAM	1.9MB	Reserved
0090_0000	0091_FFFF		128KB	OCRAM 128KB
0084_0000	008F_FFFF	Reserved	768KB	Reserved
0082_0000	0083_FFFF	TCM	128KB	Reserved
0080_0000	0081_FFFF		128KB	TCMU
007E_0000	007F_FFFF		128KB	TCML
007C_0000	007D_FFFF		128KB	Reserved
0019_0000	007B_FFFF	Reserved	1221MB	Reserved
0018_8000	0018_FFFF	OCRAM_S	32KB	Reserved
0018_0000	0018_7FFF		32KB	OCRAM_S
0011_0000	0017_FFFF	Reserved	448KB	Reserved
0010_8000	0010_FFFF	CAAM	32KB	Reserved
0010_0000	0010_7FFF		32KB	CAAM (32K secure RAM)

Table continues on the next page...

Memory

Start Address	End Address	Region	Size	Description
0002_0000	000F_FFFF	Reserved	896KB	Reserved
0001_8000	0001_FFFF	Boot ROM	32KB	Reserved
0001_E800	0001_FFFF		4KB	Boot ROM - Protected 4KB area
0000_0000	0001_E7FF		128KB	Boot ROM

2.1.3 Cortex-M4 Memory Map

Start Address	End Address	Region	Size	Description
E010_0000	FFFF_FFFF	Reserved	511MB	Reserved
E000_0000	E00F_FFFF	CM4 PPB	1MB	CM4 PPB
D800_0000	DFFF_FFFF	PCIe-2	128MB	PCIe-2
D000_0000	D7FF_FFFF	PCIe-1	128MB	PCIe-1
C000_0000	CFFF_FFFF	FLASH	256MB	QSPI
4000_0000	BFFF_FFFF	DDR Address	2048MB	DDR Memory
3E00_0000	3FFF_FFFF	Reserved	32MB	Reserved
3DC0_0000	3DFF_FFFF	DDRC	4MB	Reserved
3D80_0000	3DBF_FFFF		4MB	DDR PERF_MON
3D40_0000	3D7F_FFFF		4MB	DDR CTL
3C00_0000	3D3F_FFFF		20MB	DDR PHY
3890_0000	3BFF_FFFF	Reserved	55MB	Reserved
3880_0000	388F_FFFF	GIC	1MB	GIC400
3870_0000	387F_FFFF	Reserved	1MB	Reserved
3830_0000	384F_FFFF	VPU	2MB	VPU_Decoder
3820_0000	382F_FFFF	USB	1MB	USB2 REG
3810_0000	381F_FFFF		1MB	USB1 REG
3801_0000	380F_FFFF	Reserved	960KB	Reserved
3800_0000	3800_FFFF	GPU REG	64KB	GPU REG
3600_0000	37FF_FFFF	QSPI RX Buffers	32MB	Reserved
3400_0000	35FF_FFFF		32MB	QSPI1 RX Buffer
3382_0000	33FF_FFFF	Reserved	7.9MB	Reserved
3381_4000	3381_FFFF	PCIe REG	48KB	Reserved
3381_0000	3381_3FFF		4MB	PCIe2 REG
3380_4000	3380_FFFF		48KB	Reserved
3380_0000	3380_3FFF		4MB	PCIe1 REG
3310_0000	337F_FFFF	MMAP Peripherals	7MB	Reserved
3301_0000	330F_FFFF		960KB	Reserved
3300_8000	3300_FFFF		32KB	Reserved (APBH DMA)
3300_0000	3300_7FFF		32KB	APBH DMA

Table continues on the next page...

Start Address	End Address	Region	Size	Description
32C0_0000	32FF_FFFF	Periph (AIPS)	4MB	AIPS4
3280_0000	32BF_FFFF	Reserved	4MB	Reserved
3270_0000	327F_FFFF	GPV_7	1MB	NoC configuration
3260_0000	326F_FFFF	GPV_6	1MB	"m4" configuration port
3250_0000	325F_FFFF	GPV_5	1MB	"display" configuration port
3240_0000	324F_FFFF	GPV_4	1MB	"enet" configuration port
3230_0000	323F_FFFF	GPV_3	1MB	"per_m" configuration port
3220_0000	322F_FFFF	GPV_2	1MB	"per_s" configuration port
3210_0000	321F_FFFF	GPV_1	1MB	"wakeup" configuration port
3200_0000	320F_FFFF	GPV_0	1MB	"main" configuration port
3100_0000	31FF_FFFF	Reserved	16MB	Reserved
30C0_0000	30FF_FFFF	Pheriph (AIPS)	4MB	Reserved
3080_0000	30BF_FFFF		4MB	AIPS-3, See IP listing on separate map.
3040_0000	307F_FFFF		4MB	AIPS-2, See IP listing on separate map.
3000_0000	303F_FFFF		4MB	AIPS-1, See IP listing on separate map.
2900_0000	2FFF_FFFF	Reserved	112MB	Reserved
2800_0000	28FF_FFFF	A53/ DAP	16MB	A53/ DAP
2040_0000	27FF_FFFF	Reserved	124MB	Reserved
2022_0000	203F_FFFF	OCRAM	1.9MB	Reserved
2020_0000	2021_FFFF		128KB	OCRAM_128KB
2019_0000	201F_FFFF	Reserved	448KB	Reserved
2018_8000	2018_FFFF	OCRAM	32KB	Reserved
2018_0000	2018_7FFF		32KB	OCRAM_S
2011_0000	2017_FFFF	Reserved	448KB	Reserved
2010_8000	2010_FFFF	CAAM	32KB	Reserved
2010_0000	2010_7FFF		32KB	CAAM (32K secure RAM)
2004_0000	200F_FFFF	Reserved	768KB	Reserved
2003_8000	2003_FFFF	ROMCP	32KB	Reserved
2002_0000	2003_7FFF		128KB	Boot ROM (ROMCP)
2000_0000	2001_FFFF	TCM	128KB	TCMU
1FFE_0000	1FFF_FFFF		128KB	TCML
1000_0000	1FFD_FFFF	CM4 ALIAS CODE	262016KB	DDR Code alias
0800_0000	0FFF_FFFF		128MB	QSPI Code alias
0400_0000	07FF_FFFF		64MB	Reserved
00B0_0000	03FF_FFFF	Reserved	53MB	Reserved
0092_0000	00AF_FFFF	OCRAM	1896KB	Reserved
0090_0000	0091_FFFF		128KB	OCRAM_128KB
0019_0000	008F_FFFF	Reserved	7.5MB	Reserved
0018_8000	0018_FFFF	OCRAM_S	32KB	Reserved
0018_0000	0018_7FFF		32KB	OCRAM_S
0011_0000	0017_FFFF	Reserved	448KB	Reserved

Table continues on the next page...

Memory

Start Address	End Address	Region	Size	Description
0010_8000	0010_FFFF	CAAM	32KB	Reserved
0010_0000	0010_7FFF		32KB	CAAM (32K secure RAM)
0002_0000	000F_FFFF	Reserved	896KB	Reserved
0000_0000	0001_FFFF	TCML	128KB	TCML alias

2.1.4 DMA memory maps

The Smart DMA memory maps are defined in the following tables.

Table 2-1. SDMA1 Peripheral Memory Map

Address	Size	Peripheral
0xF000	4KB	SPBA
0xE000	4KB	Reserved for SDMA internal registers
0xD000	4KB	Reserved
0xC000	4KB	SAI3
0xB000	4KB	SAI2
0xA000	4KB	SPDIF2
0x9000	4KB	UART2
0x8000	4KB	UART3
0x7000	4KB	Reserved for SDMA internal registers
0x6000	4KB	UART1
0x5000	4KB	Reserved
0x4000	4KB	eCSPI3
0x3000	4KB	eCSPI2
0x2000	4KB	eCSPI1
0x1000	4KB	SPDIF1
0x0000	4KB	Reserved for SDMA internal memory

Table 2-2. SDMA2 Peripheral Memory Map

Address	Size	Peripheral
0xF000	4KB	SPBA
0xE000	4KB	Reserved for SDMA internal registers
0xD000	4KB	Reserved
0xC000	4KB	Reserved
0xB000	4KB	Reserved
0xA000	4KB	Reserved
0x9000	4KB	Reserved

Table continues on the next page...

Table 2-2. SDMA2 Peripheral Memory Map (continued)

Address	Size	Peripheral
0x8000	4KB	Reserved
0x7000	4KB	Reserved for SDMA internal registers
0x6000	4KB	Reserved
0x5000	4KB	SAI4
0x4000	4KB	SAI5
0x3000	4KB	SAI6
0x2000	4KB	Reserved
0x1000	4KB	SAI1
0x0000	4KB	Reserved for SDMA internal memory

2.1.5 AIPS Memory Maps

Table 2-3. AIPS1 Memory Map

Start Address	End Address	Region	NIC Port	Size
303F_0000	303F_FFFF	AIPS-1 (s_b_0)	Reserved	64KB
303E_0000	303E_FFFF		CSU	64KB
303D_0000	303D_FFFF		RDC	64KB
303C_0000	303C_FFFF		SEMAPHORE2	64KB
303B_0000	303B_FFFF		SEMAPHORE1	64KB
303A_0000	303A_FFFF		GPC	64KB
3039_0000	3039_FFFF		SRC	64KB
3038_0000	3038_FFFF		CCM	64KB
3037_0000	3037_FFFF		SNVS_HP	64KB
3036_0000	3036_FFFF		ANA_PLL	64KB
3035_0000	3035_FFFF		OCOTP_CTRL	64KB
3034_0000	3034_FFFF		IOMUXC_GPR	64KB
3033_0000	3033_FFFF		IOMUXC	64KB
3032_0000	3032_FFFF		LCDIF	64KB
3031_0000	3031_FFFF		ROMCP	64KB
3030_0000	3030_FFFF		Reserved	64KB
302F_0000	302F_FFFF		GPT3	64KB
302E_0000	302E_FFFF		GPT2	64KB
302D_0000	302D_FFFF		GPT1	64KB
302C_0000	302C_FFFF		SDMA2	64KB
302B_0000	302B_FFFF		Reserved	64KB
302A_0000	302A_FFFF		WDOG3	64KB

Table continues on the next page...

Table 2-3. AIPS1 Memory Map (continued)

Start Address	End Address	Region	NIC Port	Size
3029_0000	3029_FFFF		WDOG2	64KB
3028_0000	3028_FFFF		WDOG1	64KB
3027_0000	3027_FFFF		ANA_OSC	64KB
3026_0000	3026_FFFF		TMU (ANA_TSENSOR)	64KB
3025_0000	3025_FFFF		Reserved	64KB
3024_0000	3024_FFFF		GPIO5	64KB
3023_0000	3023_FFFF		GPIO4	64KB
3022_0000	3022_FFFF		GPIO3	64KB
3021_0000	3021_FFFF		GPIO2	64KB
3020_0000	3020_FFFF		GPIO1	64KB
301F_0000	301F_FFFF		AIPS1_Configuration	64KB
3014_0000	301E_FFFF	AIPS-1 Glob. Module Enable	Reserved	704KB
3010_0000	3013_FFFF		Reserved	256KB
300F_0000	300F_FFFF	AIPS-1 (s_b_1, via SPBA) Glob. Module Enable	SPBA2	64KB
300E_0000	300E_FFFF		Reserved for SDMA2 internal memory	64KB
300D_0000	300D_FFFF		Reserved	64KB
300C_0000	300C_FFFF		Reserved	64KB
300B_0000	300B_FFFF		Reserved	64KB
300A_0000	300A_FFFF		Reserved	64KB
3009_0000	3009_FFFF		Reserved	64KB
3008_0000	3008_FFFF		Reserved	64KB
3007_0000	3007_FFFF		Reserved for SDMA2 internal memory	64KB
3006_0000	3006_FFFF		Reserved	64KB
3005_0000	3005_FFFF		SAI4	64KB
3004_0000	3004_FFFF		SAI5	64KB
3003_0000	3003_FFFF		SAI6	64KB
3002_0000	3002_FFFF		Reserved	64KB
3001_0000	3001_FFFF		SAI1	64KB
3000_0000	3000_FFFF		Reserved for SDMA2 internal memory	64KB

Table 2-4. AIPS2 Memory Map

Start Address	End Address	Region	NIC Port	Size
307F_0000	307F_FFFF	AIPS-2 (s_b_1)	QoS	64KB
307E_0000	307E_FFFF		Reserved	64KB
307D_0000	307D_FFFF		PERFMON2	64KB
307C_0000	307C_FFFF		PERFMON1	64KB

Table continues on the next page...

Table 2-4. AIPS2 Memory Map (continued)

Start Address	End Address	Region	NIC Port	Size
307B_0000	307B_FFFF		Reserved	64KB
307A_0000	307A_FFFF		Reserved	64KB
3079_0000	3079_FFFF		Reserved	64KB
3078_0000	3078_FFFF		Reserved	64KB
3077_0000	3077_FFFF		Reserved	64KB
3076_0000	3076_FFFF		Reserved	64KB
3075_0000	3075_FFFF		Reserved	64KB
3074_0000	3074_FFFF		Reserved	64KB
3073_0000	3073_FFFF		Reserved	64KB
3072_0000	3072_FFFF		Reserved	64KB
3071_0000	3071_FFFF		Reserved	64KB
3070_0000	3070_FFFF		GPT4	64KB
306F_0000	306F_FFFF		GPT5	64KB
306E_0000	306E_FFFF		GPT6	64KB
306D_0000	306D_FFFF		Reserved	64KB
306C_0000	306C_FFFF		System_Counter_CTRL	64KB
306B_0000	306B_FFFF		System_Counter_CMP	64KB
306A_0000	306A_FFFF		System_Counter_RD	64KB
3069_0000	3069_FFFF		PWM4	64KB
3068_0000	3068_FFFF		PWM3	64KB
3067_0000	3067_FFFF		PWM2	64KB
3066_0000	3066_FFFF		PWM1	64KB
3065_0000	3065_FFFF		Reserved	64KB
3064_0000	3064_FFFF		Reserved	64KB
3063_0000	3063_FFFF		Reserved	64KB
3062_0000	3062_FFFF		Reserved	64KB
3061_0000	3061_FFFF		Reserved	64KB
3060_0000	3060_FFFF		Reserved	64KB
305F_0000	305F_FFFF		AIPS2_configuration	64KB
3050_0000	305E_FFFF	AIPS-2 Glob. Module Enable	Reserved	960KB
3040_0000	304F_FFFF		Reserved	1024KB

Table 2-5. AIPS3 Memory Map

Start Address	End Address	Region	NIC Port	Size
30BF_0000	30BF_FFFF	AIPS-3 (s_b_2)	Reserved	64KB
30BE_0000	30BE_FFFF		ENET1	64KB
30BD_0000	30BD_FFFF		SDMA1	64KB
30BC_0000	30BC_FFFF		Reserved	64KB

Table continues on the next page...

Table 2-5. AIPS3 Memory Map (continued)

Start Address	End Address	Region	NIC Port	Size
30BB_0000	30BB_FFFF		QSPI	64KB
30BA_0000	30BA_FFFF		Reserved	64KB
30B9_0000	30B9_FFFF		Reserved	64KB
30B8_0000	30B8_FFFF		CSI2 (CSI Bridge)	64KB
30B7_0000	30B7_FFFF		MIPI_CSI2_PHY	64KB
30B6_0000	30B6_FFFF		MIPI_CSI2	64KB
30B5_0000	30B5_FFFF		uSDHC2	64KB
30B4_0000	30B4_FFFF		uSDHC1	64KB
30B3_0000	30B3_FFFF		Reserved	64KB
30B2_0000	30B2_FFFF		Reserved	64KB
30B1_0000	30B1_FFFF		Reserved	64KB
30B0_0000	30B0_FFFF		Reserved	64KB
30AF_0000	30AF_FFFF		Reserved	64KB
30AE_0000	30AE_FFFF		Reserved	64KB
30AD_0000	30AD_FFFF		Reserved	64KB
30AC_0000	30AC_FFFF		SEMAPHORE_HS	64KB
30AB_0000	30AB_FFFF		MU_B	64KB
30AA_0000	30AA_FFFF		MU_A	64KB
30A9_0000	30A9_FFFF		CSI1 (CSI Bridge)	64KB
30A8_0000	30A8_FFFF		MIPI_CSI1_PHY	64KB
30A7_0000	30A7_FFFF		MIPI_CSI1	64KB
30A6_0000	30A6_FFFF		UART4	64KB
30A5_0000	30A5_FFFF		I2C4	64KB
30A4_0000	30A4_FFFF		I2C3	64KB
30A3_0000	30A3_FFFF		I2C2	64KB
30A2_0000	30A2_FFFF		I2C1	64KB
30A0_0300	30A1_FFFF		MIPI_PHY (DPHY)	63KB
30A0_0000	30A0_02FF		MIPI_DSI	256B
309F_0000	309F_FFFF		AIPS3_Configuration	64KB
3094_0000	309E_FFFF	AIPS-3 Glob. Module Enable	Reserved	704KB
3090_0000	3093_FFFF		CAAM	256KB
308F_0000	308F_FFFF	AIPS-3 (s_b_2, via SPBA) Glob. Module Enable	SPBA1	64KB
308E_0000	308E_FFFF		Reserved for SDMA internal registers	64KB
308D_0000	308D_FFFF		Reserved	64KB
308C_0000	308C_FFFF		SAI3	64KB
308B_0000	308B_FFFF		SAI2	64KB
308A_0000	308A_FFFF		SPDIF2	64KB
3089_0000	3089_FFFF		UART2	64KB
3088_0000	3088_FFFF		UART3	64KB

Table continues on the next page...

Table 2-5. AIPS3 Memory Map (continued)

Start Address	End Address	Region	NIC Port	Size
3087_0000	3087_FFFF		Reserved for SDMA internal registers	64KB
3086_0000	3086_FFFF		UART1	64KB
3085_0000	3085_FFFF		Reserved	64KB
3084_0000	3084_FFFF		eCSPI3	64KB
3083_0000	3083_FFFF		eCSPI2	64KB
3082_0000	3082_FFFF		eCSPI1	64KB
3081_0000	3081_FFFF		SPDIF1	64KB
3080_0000	3080_FFFF		Reserved for SDMA internal memory	64KB

Table 2-6. AIPS4 Memory Map

Start Address	End Address	Region	NIC Port	Size
32FF_0000	32FF_FFFF	AIPS-4 (s_h_10)	Reserved	64KB
32FE_0000	32FE_FFFF		PLATFORM_CTRL	64KB
32FD_0000	32FD_FFFF		Reserved	64KB
32FC_0000	32FC_FFFF		Reserved	64KB
32FB_0000	32FB_FFFF		MTR	64KB
32FA_0000	32FA_FFFF		Reserved	64KB
32F9_0000	32F9_FFFF		Reserved	64KB
32F8_0000	32F8_FFFF		TZASC	64KB
32F7_0000	32F7_FFFF		Reserved	64KB
32F6_0000	32F6_FFFF		Reserved	64KB
32F5_0000	32F5_FFFF		Reserved	64KB
32F4_0000	32F4_FFFF		Reserved	64KB
32F3_0000	32F3_FFFF		Reserved	64KB
32F2_0000	32F2_FFFF		Reserved	64KB
32F1_0000	32F1_FFFF		Reserved	64KB
32F0_0000	32F0_FFFF		Reserved	64KB
32EF_0000	32EF_FFFF		Reserved	64KB
32EE_0000	32EE_FFFF		Reserved	64KB
32ED_0000	32ED_FFFF		Reserved	64KB
32EC_0000	32EC_FFFF		Reserved	64KB
32EB_0000	32EB_FFFF		Reserved	64KB
32EA_0000	32EA_FFFF		Reserved	64KB
32E9_0000	32E9_FFFF		Reserved	64KB
32E8_0000	32E8_FFFF		Reserved	64KB
32E7_0000	32E7_FFFF		Reserved	64KB
32E6_0000	32E6_FFFF		Reserved	64KB

Table continues on the next page...

Table 2-6. AIPS4 Memory Map (continued)

Start Address	End Address	Region	NIC Port	Size
32E5_0000	32E5_FFFF	DCSS	Reserved	64KB
32E4_0000	32E4_FFFF		HDMI_SEC	64KB
32E2_F000	32E2_FFFF		BLK_CTL	4KB
32E2_E000	32E2_EFFF		LPCG	4KB
32E2_D000	32E2_DFFF		IRQ_STEER	4KB
32E2_C000	32E2_CFFF		HDMI_PHY Registers	4KB
32E2_5000	32E2_BFFF		Reserved	28KB
32E2_4000	32E2_4FFF		LUT_LD	4KB
32E2_3000	32E2_3FFF		CTX_LD	4KB
32E2_2000	32E2_2FFF		RD_SRC	4KB
32E2_1000	32E2_1FFF		WR_SCL	4KB
32E2_0000	32E2_0FFF		DTG	4KB
32E1_D000	32E1_FFFF		Reserved	12KB
32E1_C000	32E1_CFFF		Scaler (all Channels 3 in 1)	4KB
32E1_B000	32E1_BFFF		SUBSAM	4KB
32E1_A000	32E1_AFFF		DPR-chan3	4KB
32E1_9000	32E1_9FFF		DPR-chan2	4KB
32E1_8000	32E1_8FFF		DPR-chan1	4KB
32E1_7000	32E1_7FFF		DTRC-chan3	4KB
32E1_6000	32E1_7FFF		DTRC-chan2	4KB
32E1_5000	32E1_7FFF		DEC400D-chan1	4KB
32E1_3000	32E1_4FFF		Reserved	8KB
32E1_2000	32E1_2FFF		Dolby Vision Output (Core3)	4KB
32E1_1000	32E1_1FFF		Dolby Vision Graphics (Core 2)	4KB
32E1_0000	32E1_0FFF		Dolby Vision Video (Core1)	4KB
32E0_C000	32E0_FFFF		HDR10 Tabela and Registers	16KB
32E0_8000	32E0_BFFF		HDR10 Chan3 LUT	16KB
32E0_4000	32E0_7FFF		HDR10 Chan2 LUT	16KB
32E0_0000	32E0_3FFF		HDR10 Chan1 LUT	16KB
32DF_0000	32DF_FFFF	AIPS-4 (s_h_10)	AIPS4_configuration	64KB
32D0_0000	32DE_FFFF	AIPS-4 Glob. Module Enable	Reserved	960KB
32C0_0000	32CF_FFFF		HDMI_CTRL	1024KB

2.1.6 DAP Memory Map

Table 2-7. DAP Memory Map Table

Start Address	End Address	Size	Allocation
28C0_A000	28FF_FFFF	4056KB	Reserved
28C0_9000	28C0_9FFF	4KB	HUGO_CXCTI1
28C0_8000	28C0_8FFF	4KB	HUGO_CXCTI0
28C0_7000	28C0_7FFF	4KB	CXTPIU
28C0_6000	28C0_6FFF	4KB	CXTMC_ETR
28C0_5000	28C0_5FFF	4KB	ATB_REPLICATOR
28C0_4000	28C0_4FFF	4KB	CXTMC_ETB
28C0_3000	28C0_3FFF	4KB	HUGO_ATB_FUNNEL
28C0_2000	28C0_2FFF	4KB	CXTSGEN_READ
28C0_1000	28C0_1FFF	4KB	CXTSGEN_CTRL
28C0_0000	28C0_0FFF	4KB	HUGO ROM Table
2875_0000	28BF_FFFF	4800KB	Reserved
2874_0000	2874_FFFF	64KB	MP4-CPU3 Trace
2873_0000	2873_FFFF	64KB	MP4-CPU3 PMU
2872_0000	2872_FFFF	64KB	MP4-CPU3 CTI
2871_0000	2871_FFFF	64KB	MP4-CPU3 Debug
2865_0000	2870_FFFF	768KB	Reserved
2864_0000	2864_FFFF	64KB	MP4-CPU2 Trace
2863_0000	2863_FFFF	64KB	MP4-CPU2 PMU
2862_0000	2862_FFFF	64KB	MP4-CPU2 CTI
2861_0000	2861_FFFF	64KB	MP4-CPU2 Debug
2855_0000	2860_FFFF	768KB	Reserved
2854_0000	2854_FFFF	64KB	MP4-CPU1 Trace
2853_0000	2853_FFFF	64KB	MP4-CPU1 PMU
2852_0000	2852_FFFF	64KB	MP4-CPU1 CTI
2851_0000	2851_FFFF	64KB	MP4-CPU1 Debug
2845_0000	2850_FFFF	768KB	Reserved
2844_0000	2844_FFFF	64KB	MP4-CPU0 Trace
2843_0000	2843_FFFF	64KB	MP4-CPU0 PMU
2842_0000	2842_FFFF	64KB	MP4-CPU0 CTI
2841_0000	2841_FFFF	64KB	MP4-CPU0 Debug
2840_0000	2840_FFFF	64KB	MP4 ROM Table
2801_0000	283F_FFFF	4032KB	Reserved
2800_0000	2800_FFFF	64KB	DAP ROM Table

Chapter 3 Security

3.1 System Security

3.1.1 Overview

The security system modules are described in the sections below.

3.1.2 Central Security Unit (CSU)

The chip uses the CSU to manage security for all masters/slaves that don't directly support security like the CPU core.

3.1.3 Cryptographic Acceleration and Assurance Module (CAAM)

CAAM is the chip's cryptographic acceleration and assurance module, which serves as NXP's latest cryptographic acceleration and offloading hardware. It combines functions previously implemented in separate modules to create a modular and scalable acceleration and assurance engine. It also implements block encryption algorithms, stream cipher algorithms, hashing algorithms, public key algorithms, run-time integrity checking, a secure memory controller, and a hardware random number generator.

CAAM supports the following key features:

- PKHA block to support Public Key Cryptography with RSA 4096 and ECC algorithms
- 3 Job Rings
- RTIC (real time integrity checking)
- AES, DES, 3DES with Side channel attack resistance
- Widevine cipher text stealing (AES-CBC-CTS mode)
- PlayReady content protection

In order to provide better video content protection, CAAM also supports the domain based resource protection.

3.1.4 Secure Non-Volatile Storage (SNVS)

Secure Non-Volatile Storage (SNVS) is a companion logic block to the Cryptographic Acceleration and Assurance Module (CAAM). This block is the chip's central reporting point for security-relevant events, such as the success or failure of boot software validation and the detection of potential security compromises. This security event information allows the SNVS to determine whether the chip is in a trustworthy state.

3.1.5 On-Chip OTP Controller (OCOTP_CTRL)

The OCOPT_CTRL is used to control the 8K bit OTP fuse in the chip. It supports:

- Load fuse into shadow registers after power-on reset;
- Register interface to allow SW to read, override or program the fuse
- Flexible permission control to the fuse, including read-protect, override-protect, program-protect, lock and etc
- Programming sequence to allow single bit to be programmed individually, and also allow the non-programmed bit to be programmed later

For the 8K bits fuse, following bits are reserved for various IP requirements:

- Fuse control to disable each of the four Cortex A53 cores.
- Fuse control to disable individual IP modules such as MIPI, HDMI, ENET, USB, PCIe and etc.
- Fuse control to disable the Dolby Vision logic inside the display controller block.
- Fuse for ROM patch, with a dedicated fuse lock bit for security purpose.
- Fuse for HDMI firmware SRK and HDCP key

3.1.6 TrustZone

TrustZone security architecture is supported in the chip. For on-chip RAM, both OCRAM/OCRAM_S have TrustZone access control support through its TZ wrapper logic. One region with configurable address range of the OCRAM/OCRAM_S can be set to TZ access only. DRAM has a dedicated TZASC block that can support up to 16 configurable memory regions.

3.1.7 Resource Domain Controller (RDC)

The chip uses domain based resource control architecture for the memory/peripheral resource sharing and isolation between the Quad-core Cortex A53 platform, Cortex M4 core, and other bus masters. RDC supports flexible configuration on IP's access permission, each individual IP can, for example, be configured as A-core only or M-core only. RDC also supports multiple regions with flexible access permission control for each memory space. The memory region access control is defined in the table below. The security and exclusive support for DRAM will be done by the TZASC and the DRAM controller itself.

NOTE

The RDC and TrustZone features are completely independent of each other but will be used together.

Table 3-1. Memory Region Access Control

Memory Space	Regions Supported	Granularity	Security Support	Exclusive Support
DRAM	8 regions	4K Bytes	Disable	Disable
QSPI	8 regions	4K Bytes	Enable	Enable
PCIe	8 regions	4K Bytes	Enable	Enable
OCRAM	5 regions	128 Bytes	Disable	Enable
OCRAM_S	5 regions	128 Bytes	Disable	Enable
TCM	5 regions	128 Bytes	Enable	Enable
GIC	4 regions	4K Bytes	Enable	Disable
USBMIX	4 regions	4K Bytes	Enable	Disable
GPUMIX	4 regions	4K Bytes	Enable	Disable
VPUMIX(Decoder)	4 regions	4K Bytes	Enable	Disable
VPUMIX(Peripheral)	4 regions	4K Bytes	Enable	Disable
DRAM Register	4 regions	4K Bytes	Enable	Disable
DEBUG	4 regions	4K Bytes	Enable	Disable

3.2 Resource Domain Controller (RDC)

3.2.1 Overview

The Resource Domain Controller (RDC) provides robust support for the isolation of destination memory mapped locations such as peripherals and memory to a single core, a bus master, or set of cores and bus masters.

Many of today's processors have multiple cores for increased performance and flexibility. In some cases, the cores serve different functions (e.g. user level applications versus real time machine control) and in such cases the software for each core may be developed by different providers.

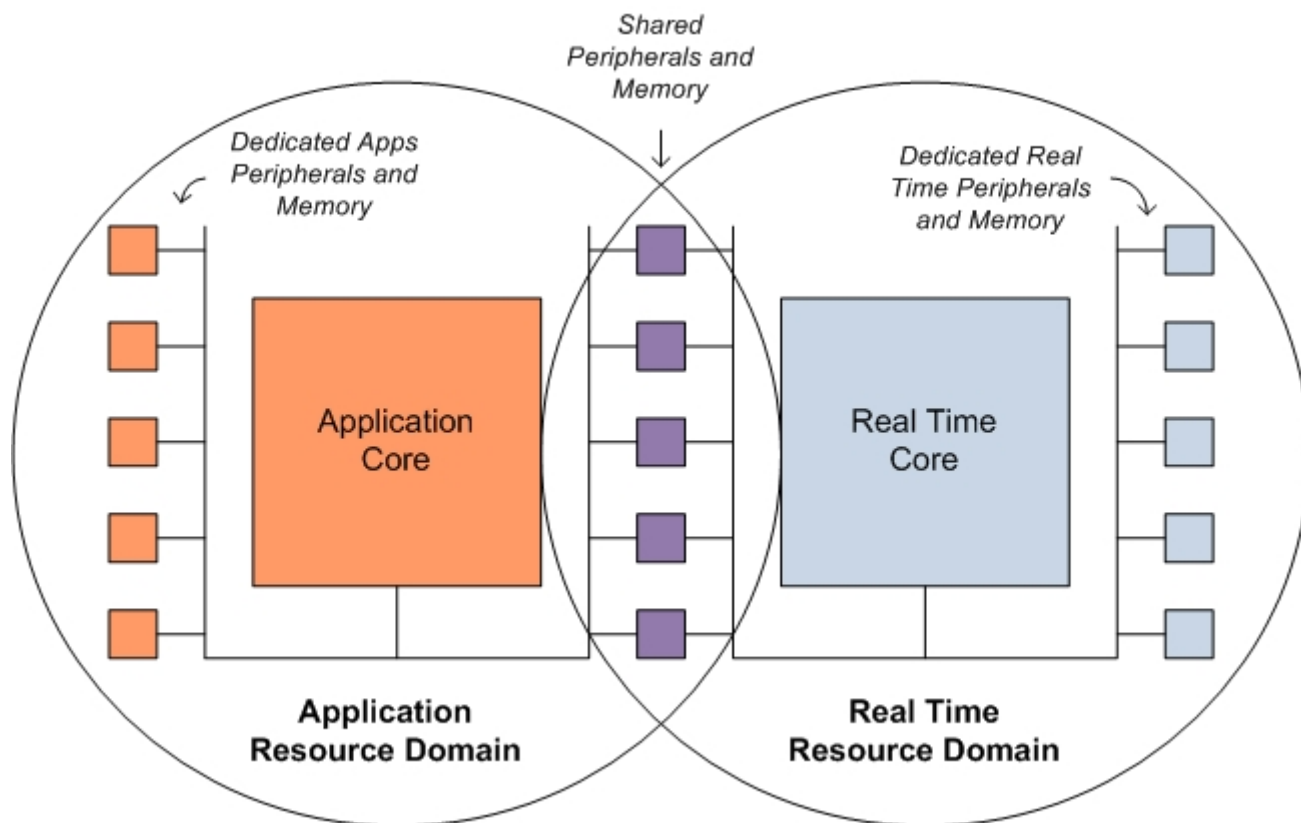


Figure 3-1. Dedicated and Shared Peripherals

For efficiency reasons the code on the cores may share chip resources such as peripherals and memory. The sharing of chip resources between the somewhat independent processing domains allows for the opportunity of data collisions where information stored in peripherals or memory by a process on one core is overwritten by software running on another core. Without careful collaboration between the two operating systems inadvertent malfunction or degradation in performance may result.

The RDC provides a mechanism to allow boot time configuration code to establish resource domains by assigning cores, bus masters, peripherals and memory regions to domain identifiers. Once configured, bus transactions are monitored to restrict accesses initiated by cores and bus masters to their respective peripherals and memory.

For shared peripherals, the RDC provides a semaphore-based locking mechanism to provide for temporary exclusivity while the domain software uses the peripheral. Once the software of one domain has finished the task and finished with the peripheral then it may release the semaphore making the peripheral available to the other domain.

3.2.1.1 Features

Resource domain subsystem has the following features:

- Assignment of cores, bus masters, peripherals, and memory regions to a resource domain
- Fixed memory resolution of 128 Bytes for small address spaces and 4 KB for large address spaces
- Four resource domain identifiers
- Memory read/write access controls for each resource domain and region
- Optional semaphore-based, hardware-enforced exclusive access of shared peripherals to a resource domain
- Prioritized access permissions for overlapping memory regions
- Automatic restoration of resource domain access permissions to memory regions in the power-down domain

3.2.2 Functional Description

The RDC is the central location for creation of isolated resource domains and for the enablement of semaphore-based access also known as “safe sharing”. Configuration software assigns one of four resource domain identifiers to each core and bus master, and allocates each memory region and peripheral to one or more resource domains.

Memory Read or Write access privileges for each resource domain are declared for each memory region. In addition, the software configuration determines which shared peripherals (those peripherals allocated to more than one domain) require safe sharing by setting the semaphore-required configuration for each peripheral.

The RDC configuration information is sent to the fabric ports, memories gaskets, semaphore controller, and peripherals to control access based on domain assignments. The fabric uses the domain identifier associated with each port to include this information along with the bus transaction. When the slave gasket encounters a bus transaction it makes a comparison of the transaction domain ID to the RDC-provided list of allowed domains. If the transaction domain ID is on the list then access may be permitted.

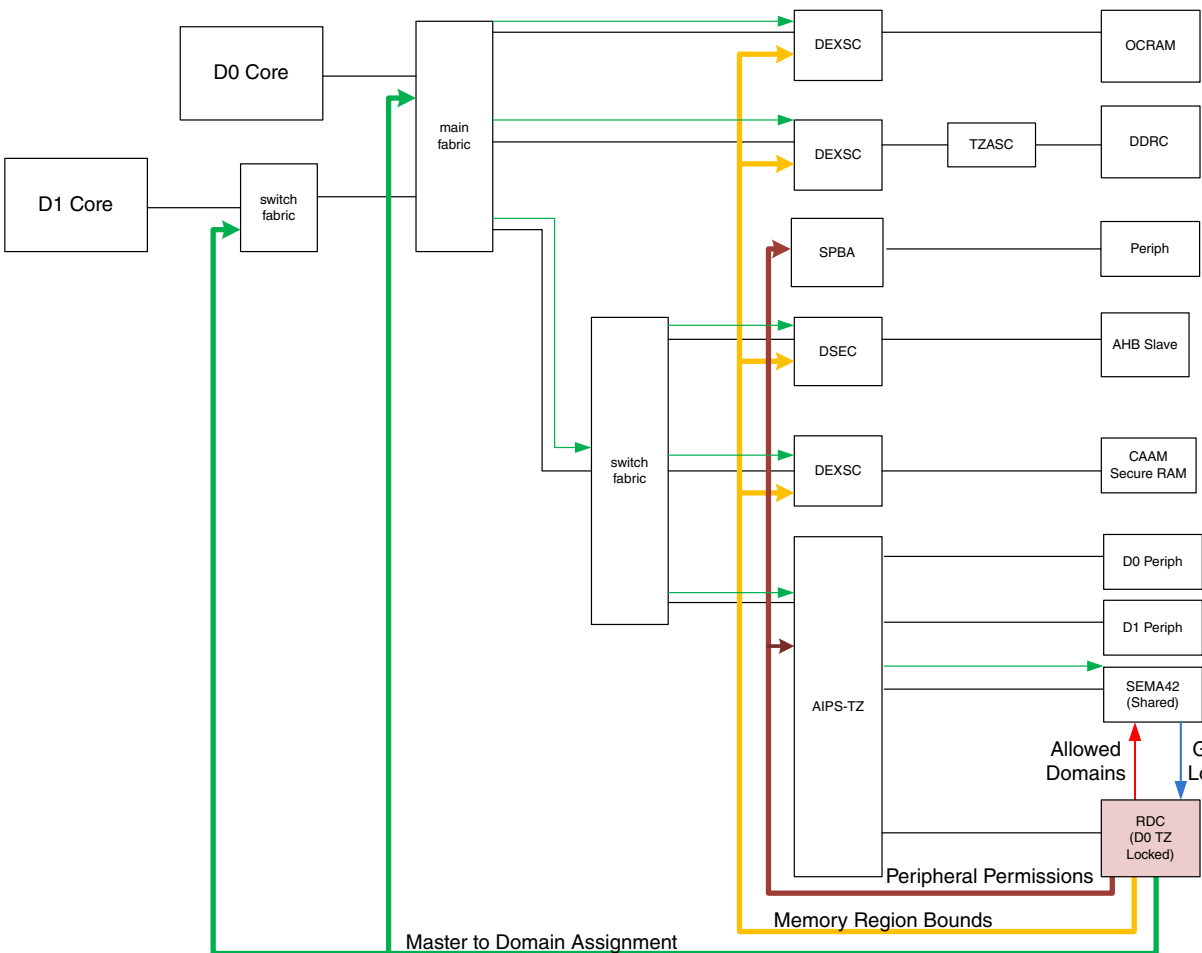


Figure 3-2. Example RDC Connections

For shared peripherals, RDC permits more than one domain access to a single peripheral. RDC also provides three ways to control synchronized use of shared peripherals. These methods include hardware-enforced synchronization, software-based semaphores, or no synchronization. The latter may be suitable for well-tuned multi-core operating systems that handle synchronization in the core platform, for instance.

For hardware-enforced synchronization, also known as "safe sharing", ownership of the peripheral must be claimed in the semaphore controller before access is allowed to the shared peripheral. The "semaphore required" bit (SREQ) is set in the PDAP register corresponding to the shared peripheral which causes the RDC to require that a semaphore is obtained by a domain before access by that domain to the shared peripheral is allowed. During the time that the domain has the semaphore in possession its bus masters have exclusive access to the peripheral.

When the semaphore is released then no domain masters have access until the semaphore is obtained again. When the SREQ is set, RDC does not allow masters to obtain semaphores of peripherals to which it is not allocated; the master must have designated access in the D-registers of the corresponding PDAP register (e.g. D3R bit set for Domain 3 access of the shared peripheral). There is a one-to-one mapping between the semaphore controller gate and the resource domain controller peripheral. The mapping of PDAP registers and peripherals can be found in the Peripheral Map section of the RDC chapter.

3.2.2.1 Domain ID

The RDC provides for an isolation of domain resources by use of a identifier called the Domain ID (DID). A core and its resources including memory, bus masters, and peripherals are all associated with a single DID. When software or a DMA attempts to access a peripheral or memory, the corresponding bus transaction includes the DID along with the other bus control information such as Read, Write, and privilege mode.

3.2.2.2 Resource Assignment

The RDC allows assignment of peripherals and memories to one or more domains while each bus master or core is placed in one of four domains. The masters are assigned a domain in the MDA register. A peripheral is given R/W access permissions to each domain in the PDAP register. Memory regions are bound by address space in start and end registers, the MRSA and MREA. Each memory region is assigned one or more allowed domains and R/W permissions in the MRC control register. Memory regions must be enabled before the permissions are active. Otherwise the permissions are not restricted.

The RDC itself should be isolated to ensure that only a trustworthy resource manager can configure the RDC registers. This process may either be present initially, during secure boot, or during the runtime in the secure world, for example. If the operating system does not support a runtime trusted execution then during the secure boot process the RDC configuration can be locked to prevent further modification after the operating systems are running.

NOTE

The CCM supports multicore awareness based on resource domain assignments programmed into the RDC. Refer to the CCM chapter regarding the relationship between core resource domains and their respective CCM resources. Failing to follow the proper sequence when updating the resource domain

assignments of the core can result in clocks being inadvertently gated.

3.2.2.3 Safe Sharing

For shared peripherals, the RDC can be configured to require a domain to obtain a semaphore lock before access to the peripheral is allowed. This feature helps prevent collisions from processes on separate cores that may want to use the same peripheral at the same time. The RDC sends a list of eligible domains to the semaphore module for each gate/peripheral. The eligible domains are those that are set in the peripheral domain access permissions (PDAP) registers. There is a one-to-one correspondence between semaphore gates and peripherals so each gate in the semaphore block represents a peripheral. The RDC receives semaphore locks from the hardware semaphore module (SEMA42). A semaphore lock is acquired when a core or bus master from a given domain requests a lock for a particular gate. The semaphore module compares the request's domain ID against the list of eligible domain IDs. If the domain ID is on the list and the lock is available then the lock is set and a signal is sent back to the RDC module indicating a lock has been acquired for a particular gate and to which domain ID the lock belongs. The RDC then restricts access to the corresponding peripheral to only transactions originating from the domain that has the lock. Another domain, though on the shared list to access the peripheral, must then wait until the lock is released before acquiring the lock and gaining access to the peripheral. To enable this feature of hardware enforcement for the semaphore locks, the SREQ bit is set in the RDC resource register.

If the SREQ is set, then when a process determines it needs a shared peripheral, it must first lock the resource in the semaphore module. Once the resource is locked, the semaphore module sends a signal to the RDC indicating the domain has access to the resource. The RDC will then set the access permissions to allow that domain access to the peripheral.

For a domain to acquire a lock on a peripheral, the domain must have been assigned to the peripheral in the RDC Peripheral Domain Access Permissions register (PDAP). The semaphore module only allows safe-sharing locks for those domains that are assigned to the peripheral. The semaphore module does not consider the access type (Read or Write) when allowing domains to acquire locks.

The SEMA42 module implements hardware-enforced semaphores as an IPS-mapped slave peripheral device. The feature set includes:

- Module definition supporting 64 hardware-enforced gates in a multi-processor configuration, where up to 15 processors can be supported; cpX is meant to represent core processor X
 - Gates appear as an n -entry byte-size array with read and write accesses ($n = 16, 32, 64$).
 - Processors lock gates by writing "Master_index" to the appropriate gate and must read back the gate value to verify the lock operation was successful. The Master_index value for the processors can be found in the Master Index Allocation table, which can be found in the AIPSTZ block.. Also note that after locking, the gate register contains the master_id value of the locking processor (in bits [3:0]), and also the value of the locking domain (in bits [5:4]).
 - Once locked, the gate is unlocked by a write of zeroes from the locking processor.
 - The number of implemented gates is specified by a hardware configuration define.
 - Each hardware gate appears as a 16-state, 4-bit state machine.
 - 16-state implementation
 - if gate = 0x0, then state = unlocked
 - if gate = 0x1, then state = locked by processor (master_index) 0
 - if gate = 0x2, then state = locked by processor (master_index) 1
 - ...
 - if gate = 0xF, then state = locked by processor (master_index) 14
 - Uses the logical bus master number (master_index) as a reference attribute plus the specified data patterns to validate all write operations.
 - Once locked, the gate can (and must) be unlocked by a write of zeroes from the locking processor.
 - Secure reset mechanisms are supported to clear the contents of individual gates, as well as a clear_all capability.
- Memory-mapped IPS slave peripheral platform module
 - Interface to the IPS bus for programming-model accesses

3.2.2.4 Resource Domain Control and Security Considerations

Conceptually, the RDC configuration is independent of the processor privilege mode and security domain. It is intended to allow for isolation between core processing environments to prevent collisions and increase reliability. Access between resource domains is mutually exclusive and each domain should be in control of its own privilege modes and access rights.

However, it is important to realize multi-core processors may have a multiple resource domains but only one overarching security domain. Chip security controls reside in one resource domain. In this configuration, a domain can affect at least one level of access privileges in the other domain. This may be acceptable but clarity and care is needed to ensure expected functionality.

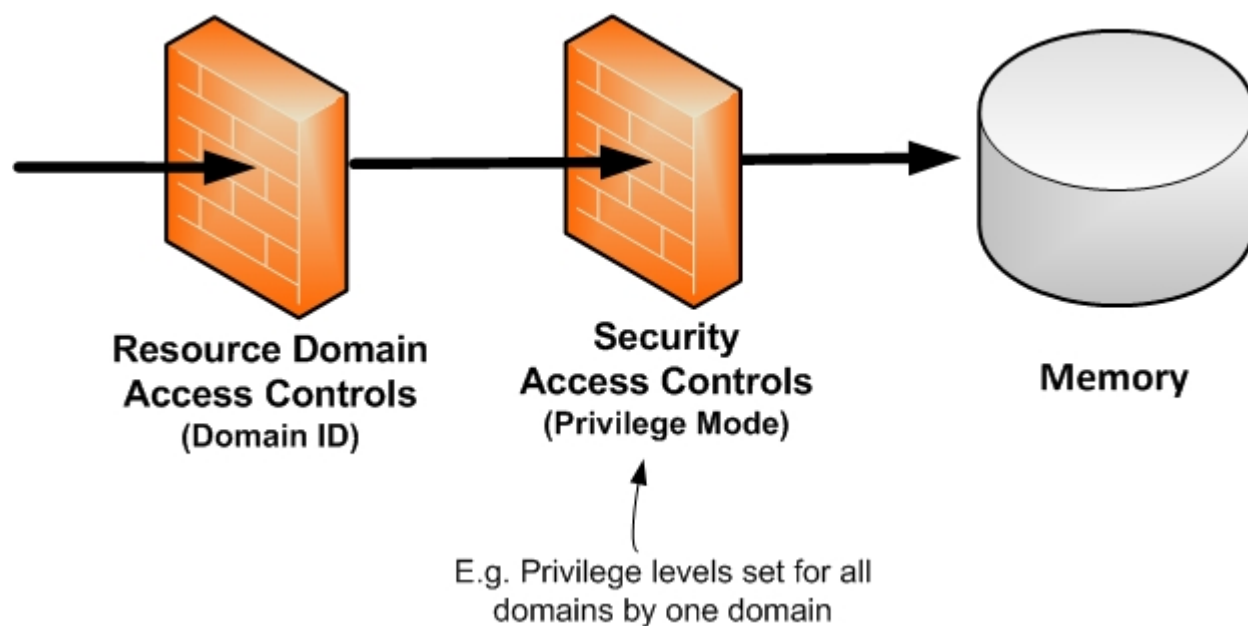


Figure 3-3. Access Control to Memory

Therefore, access to the security controls should be restricted to the most trustworthy operating mode of the core and privilege levels should be coordinated to ensure that shared peripherals and memory regions are accessible by both cores. For instance, if a memory region is designated for secure accesses then all domain masters that share that region must have secure privileges.

3.2.3 Modes of Operation

The RDC provides access controls to the resource domain subsystem. When the device is in a low power mode then some memory regions in the subsystem may be powered off. RDC responds to the impacted memory regions by automatically reconfiguring the memory regions once power returns and blocking access to those memory regions until the reconfiguration process is complete.

3.2.3.1 Low Power Modes

The RDC loads configuration information for memory regions (MRSA, MRSE, MRC) into access control mechanisms (gaskets) at the memory interface. The location of this configuration information may reside inside power domains that lose power during sleep modes for energy savings. To restore configuration information upon return from sleep mode, the RDC receives a global power control signal indicating power is restored. The RDC then automatically reconfigures the memory regions with the configuration information.

During reconfiguration, access is blocked to the previously powered down memories. When the RDC completes reconfiguration it issues an interrupt and allows access to the memory regions. Only the powered down regions are blocked during the reconfiguration. Memory regions in the "always-on" power domain (still powered during sleep mode) remain available according to the programmed access rights. If no memory regions were enabled then the powered down regions are available immediately when power is restored.

The figure below shows the Global Power Control signal which RDC uses to invalidate the configuration upon deassertion and to restore the configuration when re-asserted. The configuration is valid and bus transactions allowed once the memory regions have been restored.

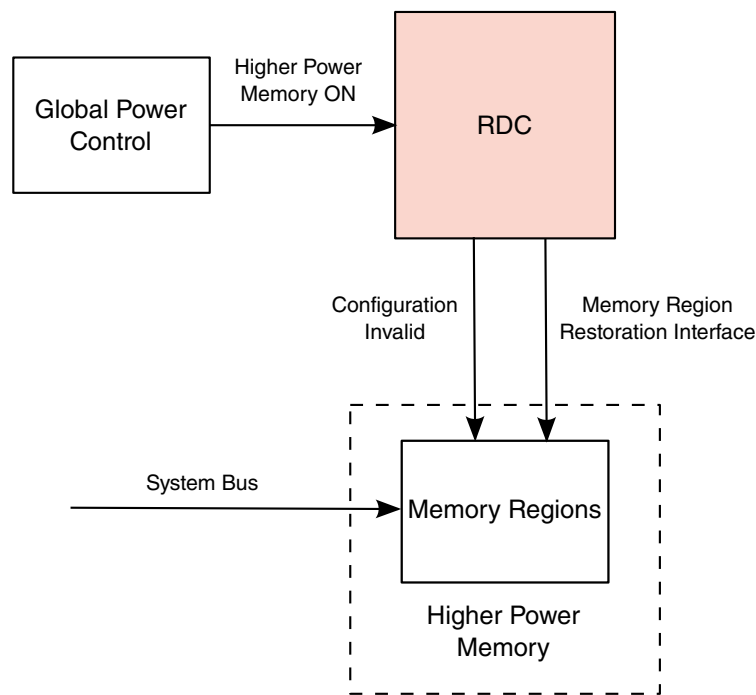


Figure 3-4. Memory Restoration Signaling

3.2.4 Programming Interface

This section provides product specific details describing the mapping of resources - peripherals, bus masters, and memory regions - to corresponding resource domain controls RDC registers.

The RDC and RDC_SEMA42 register maps are combined in this chapter. The base address for the one RDC map and two SEMA42 maps are each separated by 4KB. While there are two SEMA42 submodules and therefore two sets of SEMA42 registers, this chapter describes one. Please refer to the peripheral memory map for the base addresses of the RDC and SEMA42 modules.

3.2.4.1 Master Assignment Registers

Table 3-2. Master Assignment Mapping

Master	RDC MDA Register
Quad A53	RDC_MDA0
M4	RDC_MDA1

Table continues on the next page...

Table 3-2. Master Assignment Mapping (continued)

Master	RDC MDA Register
PCIE_CTRL1	RDC_MDA2
PCIE_CTRL2	RDC_MDA3
VPU Decoders	RDC_MDA4
LCDIF	RDC_MDA5
CSI-1	RDC_MDA6
CSI-2	RDC_MDA7
Coresight	RDC_MDA8
DAP	RDC_MDA9
CAAM	RDC_MDA10
SDMA (p)	RDC_MDA11
SDMA (b)	RDC_MDA12
APBHDMA	RDC_MDA13
RAWNAND	RDC_MDA14
uSDHC1	RDC_MDA15
uSDHC2	RDC_MDA16
Display Controller	RDC_MDA17
GPU	RDC_MDA18
USB1	RDC_MDA19
USB2	RDC_MDA20
TESTPORT	RDC_MDA21
ENET1_TX	RDC_MDA22
ENET1_RX	RDC_MDA23
SDMA2 (p)	RDC_MDA24
SDMA2 (b)	RDC_MDA24
SDMA2 to SPBA2	RDC_MDA24
Reserved	RDC_MDA25
SDMA1 to SPBA1	RDC_MDA26

3.2.4.2 Peripheral Mapping

Each peripheral has a corresponding resource domain assignment register in the RDC and semaphore lock register in the RDC_SEMA42 module. The following table shows allocation of the RDC PDAP and RDC_SEMA4 GATE registers for peripheral resource domain assignment.

NOTE

Access control of the RDC registers can be programmed using the respective PDAP register. The default setting of the PDAP

register for the RDC allows access from all domains. Use caution when restricting access of the RDC registers to avoid conditions where access to the RDC registers is needed but no master is assigned to a domain with access rights to the RDC.

Table 3-3. RDC Peripheral Mapping

Peripheral	RDC PDAP register	RDC_SEMA42 block/gate register
GPIO1	RDC_PDAP0	SEMA42 B1 / G0
GPIO2	RDC_PDAP1	SEMA42 B1 / G1
GPIO3	RDC_PDAP2	SEMA42 B1 / G2
GPIO4	RDC_PDAP3	SEMA42 B1 / G3
GPIO5	RDC_PDAP4	SEMA42 B1 / G4
Reserved	RDC_PDAP5	SEMA42 B1 / G5
ANA_TSENSOR	RDC_PDAP6	SEMA42 B1 / G6
ANA_OSC	RDC_PDAP7	SEMA42 B1 / G7
WDOG1	RDC_PDAP8	SEMA42 B1 / G8
WDOG2	RDC_PDAP9	SEMA42 B1 / G9
WDOG3	RDC_PDAP10	SEMA42 B1 / G10
Reserved	RDC_PDAP11	SEMA42 B1 / G11
SDMA2	RDC_PDAP12	SEMA42 B1 / G12
GPT1	RDC_PDAP13	SEMA42 B1 / G13
GPT2	RDC_PDAP14	SEMA42 B1 / G14
GPT3	RDC_PDAP15	SEMA42 B1 / G15
Reserved	RDC_PDAP16	SEMA42 B1 / G16
ROMCP	RDC_PDAP17	SEMA42 B1 / G17
LCDIF	RDC_PDAP18	SEMA42 B1 / G18
IOMUXC	RDC_PDAP19	SEMA42 B1 / G19
IOMUXC_GPR	RDC_PDAP20	SEMA42 B1 / G20
OCOTP_CTRL	RDC_PDAP21	SEMA42 B1 / G21
ANA_PLL	RDC_PDAP22	SEMA42 B1 / G22
SNVS_HP	RDC_PDAP23	SEMA42 B1 / G23
CCM	RDC_PDAP24	SEMA42 B1 / G24
SRC	RDC_PDAP25	SEMA42 B1 / G25
GPC	RDC_PDAP26	SEMA42 B1 / G26
SEMAPHORE1	RDC_PDAP27	SEMA42 B1 / G27
SEMAPHORE2	RDC_PDAP28	SEMA42 B1 / G28
RDC	RDC_PDAP29	SEMA42 B1 / G29
CSU	RDC_PDAP30	SEMA42 B1 / G30
Reserved	RDC_PDAP31	SEMA42 B1 / G31
DC_MST0	RDC_PDAP32	SEMA42 B1 / G32
DC_MST1	RDC_PDAP33	SEMA42 B1 / G33

Table continues on the next page...

Table 3-3. RDC Peripheral Mapping (continued)

Peripheral	RDC PDAP register	RDC_SEMA42 block/gate register
DC_MST2	RDC_PDAP34	SEMA42 B1 / G34
DC_MST3	RDC_PDAP35	SEMA42 B1 / G35
HDMI_SEC	RDC_PDAP36	SEMA42 B1 / G36
Reserved	RDC_PDAP37	SEMA42 B1 / G37
PWM1	RDC_PDAP38	SEMA42 B1 / G38
PWM2	RDC_PDAP39	SEMA42 B1 / G39
PWM3	RDC_PDAP40	SEMA42 B1 / G40
PWM4	RDC_PDAP41	SEMA42 B1 / G41
System_Counter_RD	RDC_PDAP42	SEMA42 B1 / G42
System_Counter_CMP	RDC_PDAP43	SEMA42 B1 / G43
System_Counter_CTRL	RDC_PDAP44	SEMA42 B1 / G44
HDMI_CTRL	RDC_PDAP45	SEMA42 B1 / G45
GPT6	RDC_PDAP46	SEMA42 B1 / G46
GPT5	RDC_PDAP47	SEMA42 B1 / G47
GPT4	RDC_PDAP48	SEMA42 B1 / G48
Reserved	RDC_PDAP49	SEMA42 B1 / G49
Reserved	RDC_PDAP50	SEMA42 B1 / G50
Reserved	RDC_PDAP51	SEMA42 B1 / G51
Reserved	RDC_PDAP52	SEMA42 B1 / G52
Reserved	RDC_PDAP53	SEMA42 B1 / G53
Reserved	RDC_PDAP54	SEMA42 B1 / G54
Reserved	RDC_PDAP55	SEMA42 B1 / G55
TZASC	RDC_PDAP56	SEMA42 B1 / G56
Reserved	RDC_PDAP57	SEMA42 B1 / G57
Reserved	RDC_PDAP58	SEMA42 B1 / G58
MTR	RDC_PDAP59	SEMA42 B1 / G59
PERFMON1	RDC_PDAP60	SEMA42 B1 / G60
PERFMON2	RDC_PDAP61	SEMA42 B1 / G61
PLATFORM_CTRL	RDC_PDAP62	SEMA42 B1 / G62
QoS	RDC_PDAP63	SEMA42 B1 / G63
MIPI_PHY	RDC_PDAP64	SEMA42 B2 / G0
MIPI_DSI	RDC_PDAP65	SEMA42 B2 / G1
I2C1	RDC_PDAP66	SEMA42 B2 / G2
I2C2	RDC_PDAP67	SEMA42 B2 / G3
I2C3	RDC_PDAP68	SEMA42 B2 / G4
I2C4	RDC_PDAP69	SEMA42 B2 / G5
UART4	RDC_PDAP70	SEMA42 B2 / G6
MIPI_CSI1	RDC_PDAP71	SEMA42 B2 / G7
MIPI_CSI_PHY1	RDC_PDAP72	SEMA42 B2 / G8

Table continues on the next page...

Table 3-3. RDC Peripheral Mapping (continued)

Peripheral	RDC PDAP register	RDC_SEMA42 block/gate register
CSI1	RDC_PDAP73	SEMA42 B2 / G9
MU_A	RDC_PDAP74	SEMA42 B2 / G10
MU_B	RDC_PDAP75	SEMA42 B2 / G11
SEMAPHORE_HS	RDC_PDAP76	SEMA42 B2 / G12
Reserved for SDMA2 internal memory	RDC_PDAP77	SEMA42 B2 / G13
SAI1	RDC_PDAP78	SEMA42 B2 / G14
Reserved	RDC_PDAP79	SEMA42 B2 / G15
SAI6	RDC_PDAP80	SEMA42 B2 / G16
SAI5	RDC_PDAP81	SEMA42 B2 / G17
SAI4	RDC_PDAP82	SEMA42 B2 / G18
Reserved for SDMA2 internal memory	RDC_PDAP83	SEMA42 B2 / G19
uSDHC1	RDC_PDAP84	SEMA42 B2 / G20
uSDHC2	RDC_PDAP85	SEMA42 B2 / G21
MIPI_CSI2	RDC_PDAP86	SEMA42 B2 / G22
MIPI_CSI_PHY2	RDC_PDAP87	SEMA42 B2 / G23
CSI2	RDC_PDAP88	SEMA42 B2 / G24
Reserved for SDMA2 internal memory	RDC_PDAP89	SEMA42 B2 / G25
SPBA2	RDC_PDAP90	SEMA42 B2 / G26
QSPI	RDC_PDAP91	SEMA42 B2 / G27
Reserved	RDC_PDAP92	SEMA42 B2 / G28
SDMA1	RDC_PDAP93	SEMA42 B2 / G29
ENET1	RDC_PDAP94	SEMA42 B2 / G30
Reserved	RDC_PDAP95	SEMA42 B2 / G31
Reserved for SDMA internal memory	RDC_PDAP96	SEMA42 B2 / G32
SPDIF1	RDC_PDAP97	SEMA42 B2 / G33
eCSPI1	RDC_PDAP98	SEMA42 B2 / G34
eCSPI2	RDC_PDAP99	SEMA42 B2 / G35
eCSPI3	RDC_PDAP100	SEMA42 B2 / G36
Reserved	RDC_PDAP101	SEMA42 B2 / G37
UART1	RDC_PDAP102	SEMA42 B2 / G38
Reserved for SDMA internal registers	RDC_PDAP103	SEMA42 B2 / G39
UART3	RDC_PDAP104	SEMA42 B2 / G40
UART2	RDC_PDAP105	SEMA42 B2 / G41
SPDIF2	RDC_PDAP106	SEMA42 B2 / G42
SAI2	RDC_PDAP107	SEMA42 B2 / G43
SAI3	RDC_PDAP108	SEMA42 B2 / G44
Reserved	RDC_PDAP109	SEMA42 B2 / G45
Reserved for SDMA internal registers	RDC_PDAP110	SEMA42 B2 / G46
SPBA1	RDC_PDAP111	SEMA42 B2 / G47

Table continues on the next page...

Table 3-3. RDC Peripheral Mapping (continued)

Peripheral	RDC PDAP register	RDC_SEMA42 block/gate register
module_en_glbl[0]	RDC_PDAP112	SEMA42 B2 / G48
module_en_glbl[0]	RDC_PDAP113	SEMA42 B2 / G49
CAAM	RDC_PDAP114	SEMA42 B2 / G50

3.2.4.3 Memory Region Map

The number of memories with domain isolation support varies per device. The number of memory regions for a particular memory and the size of those regions varies per memory gasket. Each region of memory has a set of registers to define the boundaries of the region based on start and end addresses, a control register to set the domain access permissions and enable the region, and a status register to determine if access was denied to a region.

For this device, refer to the table below to determine the memories with domain support, the number of regions for each memory, the region resolution, the identifying numbers for the sets of memory region registers, and the addresses of the RDC registers to access the sets of Memory Region registers.

Table 3-4. Memory Region Mapping

Memory/Port	Number of Regions	Region Resolution	Memory Region Register Set Number (e.g. MRSA, MREA, MRC, MRVS)	Register Address Range
DRAM	8	4 KB	0-7	0x800-0x87C
PCIe2	4	4 KB	8-11	0x880-0x8BC
QSPI1	8	4 KB	12-19	0x8C0-0x93C
PCIe1	4	4 KB	20-23	0x940-0x97C
OCRAM	5	128 B	24-28	0x980-0x9CC
OCRAM_S	5	128 B	29-33	0x9D0-0xA1C
TCM	5	128 B	34-38	0xA20-0xA6C
GIC	4	4 KB	39-42	0xA70-0xAAC
USBMIX	4	4 KB	43-46	0xAB0-0xAEC
GPU	4	4 KB	47-50	0xAF0-0xB2C
VPU (Decoder)	4	4 KB	51-54	0xB30-0xB6C
DEBUG (DAP)	4	4 KB	55-58	0xB70-0xBAC
DDRC (REG)	5	4 KB	59-63	0xBB0-0xBFC

3.2.4.4 Memory Region Access Control

The RDC also supports multiple regions with flexible access permission control for each memory space. The memory region access control is defined in the table below. The security and exclusive support for DRAM is done by the TZASC and the DRAM controller itself. Note that the RDC and TrustZone features are completely independent of each other but will be used together.

Table 3-5. Memory Region Access Control

Memory Space	Regions Supported	Granularity	Security Support	Exclusive Support
DRAM	8	4 KB	Disable	Disable
QSPI	8	4 KB	Enable	Enable
PCIe	8	4 KB	Enable	Enable
OCRAM	5	128 KB	Disable	Enable
OCRAM_S	5	128 KB	Disable	Enable
TCM	5	128 KB	Enable	Enable
GIC	4	4 KB	Enable	Disable
USBMIX	4	4 KB	Enable	Disable
GPU	4	4 KB	Enable	Disable
VPU (Decoder)	4	4 KB	Enable	Disable
VPU (Peripheral)	4	4 KB	Enable	Disable
DRAM Register	4	4 KB	Enable	Disable
DEBUG	4	4 KB	Enable	Disable

3.2.5 RDC Memory Map/Register Definition

RDC memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
303D_0000	Version Information (RDC_VIR)	32	R	0376_E204h	3.2.5.1/61
303D_0024	Status (RDC_STAT)	32	R/W	0000_0100h	3.2.5.2/62
303D_0028	Interrupt and Control (RDC_INTCTRL)	32	R/W	0000_0000h	3.2.5.3/63
303D_002C	Interrupt Status (RDC_INTSTAT)	32	R/W	See section	3.2.5.4/63
303D_0200	Master Domain Assignment (RDC_MDA0)	32	R/W	0000_0000h	3.2.5.5/64
303D_0204	Master Domain Assignment (RDC_MDA1)	32	R/W	0000_0000h	3.2.5.5/64
303D_0208	Master Domain Assignment (RDC_MDA2)	32	R/W	0000_0000h	3.2.5.5/64

Table continues on the next page...

RDC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303D_020C	Master Domain Assignment (RDC_MDA3)	32	R/W	0000_0000h	3.2.5.5/64
303D_0210	Master Domain Assignment (RDC_MDA4)	32	R/W	0000_0000h	3.2.5.5/64
303D_0214	Master Domain Assignment (RDC_MDA5)	32	R/W	0000_0000h	3.2.5.5/64
303D_0218	Master Domain Assignment (RDC_MDA6)	32	R/W	0000_0000h	3.2.5.5/64
303D_021C	Master Domain Assignment (RDC_MDA7)	32	R/W	0000_0000h	3.2.5.5/64
303D_0220	Master Domain Assignment (RDC_MDA8)	32	R/W	0000_0000h	3.2.5.5/64
303D_0224	Master Domain Assignment (RDC_MDA9)	32	R/W	0000_0000h	3.2.5.5/64
303D_0228	Master Domain Assignment (RDC_MDA10)	32	R/W	0000_0000h	3.2.5.5/64
303D_022C	Master Domain Assignment (RDC_MDA11)	32	R/W	0000_0000h	3.2.5.5/64
303D_0230	Master Domain Assignment (RDC_MDA12)	32	R/W	0000_0000h	3.2.5.5/64
303D_0234	Master Domain Assignment (RDC_MDA13)	32	R/W	0000_0000h	3.2.5.5/64
303D_0238	Master Domain Assignment (RDC_MDA14)	32	R/W	0000_0000h	3.2.5.5/64
303D_023C	Master Domain Assignment (RDC_MDA15)	32	R/W	0000_0000h	3.2.5.5/64
303D_0240	Master Domain Assignment (RDC_MDA16)	32	R/W	0000_0000h	3.2.5.5/64
303D_0244	Master Domain Assignment (RDC_MDA17)	32	R/W	0000_0000h	3.2.5.5/64
303D_0248	Master Domain Assignment (RDC_MDA18)	32	R/W	0000_0000h	3.2.5.5/64
303D_024C	Master Domain Assignment (RDC_MDA19)	32	R/W	0000_0000h	3.2.5.5/64
303D_0250	Master Domain Assignment (RDC_MDA20)	32	R/W	0000_0000h	3.2.5.5/64
303D_0254	Master Domain Assignment (RDC_MDA21)	32	R/W	0000_0000h	3.2.5.5/64
303D_0258	Master Domain Assignment (RDC_MDA22)	32	R/W	0000_0000h	3.2.5.5/64
303D_025C	Master Domain Assignment (RDC_MDA23)	32	R/W	0000_0000h	3.2.5.5/64
303D_0260	Master Domain Assignment (RDC_MDA24)	32	R/W	0000_0000h	3.2.5.5/64
303D_0264	Master Domain Assignment (RDC_MDA25)	32	R/W	0000_0000h	3.2.5.5/64
303D_0268	Master Domain Assignment (RDC_MDA26)	32	R/W	0000_0000h	3.2.5.5/64
303D_0400	Peripheral Domain Access Permissions (RDC_PDAP0)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0404	Peripheral Domain Access Permissions (RDC_PDAP1)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0408	Peripheral Domain Access Permissions (RDC_PDAP2)	32	R/W	0000_00FFh	3.2.5.6/65
303D_040C	Peripheral Domain Access Permissions (RDC_PDAP3)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0410	Peripheral Domain Access Permissions (RDC_PDAP4)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0414	Peripheral Domain Access Permissions (RDC_PDAP5)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0418	Peripheral Domain Access Permissions (RDC_PDAP6)	32	R/W	0000_00FFh	3.2.5.6/65
303D_041C	Peripheral Domain Access Permissions (RDC_PDAP7)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0420	Peripheral Domain Access Permissions (RDC_PDAP8)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0424	Peripheral Domain Access Permissions (RDC_PDAP9)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0428	Peripheral Domain Access Permissions (RDC_PDAP10)	32	R/W	0000_00FFh	3.2.5.6/65
303D_042C	Peripheral Domain Access Permissions (RDC_PDAP11)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0430	Peripheral Domain Access Permissions (RDC_PDAP12)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0434	Peripheral Domain Access Permissions (RDC_PDAP13)	32	R/W	0000_00FFh	3.2.5.6/65

Table continues on the next page...

RDC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303D_0438	Peripheral Domain Access Permissions (RDC_PDAP14)	32	R/W	0000_00FFh	3.2.5.6/65
303D_043C	Peripheral Domain Access Permissions (RDC_PDAP15)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0440	Peripheral Domain Access Permissions (RDC_PDAP16)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0444	Peripheral Domain Access Permissions (RDC_PDAP17)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0448	Peripheral Domain Access Permissions (RDC_PDAP18)	32	R/W	0000_00FFh	3.2.5.6/65
303D_044C	Peripheral Domain Access Permissions (RDC_PDAP19)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0450	Peripheral Domain Access Permissions (RDC_PDAP20)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0454	Peripheral Domain Access Permissions (RDC_PDAP21)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0458	Peripheral Domain Access Permissions (RDC_PDAP22)	32	R/W	0000_00FFh	3.2.5.6/65
303D_045C	Peripheral Domain Access Permissions (RDC_PDAP23)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0460	Peripheral Domain Access Permissions (RDC_PDAP24)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0464	Peripheral Domain Access Permissions (RDC_PDAP25)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0468	Peripheral Domain Access Permissions (RDC_PDAP26)	32	R/W	0000_00FFh	3.2.5.6/65
303D_046C	Peripheral Domain Access Permissions (RDC_PDAP27)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0470	Peripheral Domain Access Permissions (RDC_PDAP28)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0474	Peripheral Domain Access Permissions (RDC_PDAP29)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0478	Peripheral Domain Access Permissions (RDC_PDAP30)	32	R/W	0000_00FFh	3.2.5.6/65
303D_047C	Peripheral Domain Access Permissions (RDC_PDAP31)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0480	Peripheral Domain Access Permissions (RDC_PDAP32)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0484	Peripheral Domain Access Permissions (RDC_PDAP33)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0488	Peripheral Domain Access Permissions (RDC_PDAP34)	32	R/W	0000_00FFh	3.2.5.6/65
303D_048C	Peripheral Domain Access Permissions (RDC_PDAP35)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0490	Peripheral Domain Access Permissions (RDC_PDAP36)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0494	Peripheral Domain Access Permissions (RDC_PDAP37)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0498	Peripheral Domain Access Permissions (RDC_PDAP38)	32	R/W	0000_00FFh	3.2.5.6/65
303D_049C	Peripheral Domain Access Permissions (RDC_PDAP39)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04A0	Peripheral Domain Access Permissions (RDC_PDAP40)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04A4	Peripheral Domain Access Permissions (RDC_PDAP41)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04A8	Peripheral Domain Access Permissions (RDC_PDAP42)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04AC	Peripheral Domain Access Permissions (RDC_PDAP43)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04B0	Peripheral Domain Access Permissions (RDC_PDAP44)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04B4	Peripheral Domain Access Permissions (RDC_PDAP45)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04B8	Peripheral Domain Access Permissions (RDC_PDAP46)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04BC	Peripheral Domain Access Permissions (RDC_PDAP47)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04C0	Peripheral Domain Access Permissions (RDC_PDAP48)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04C4	Peripheral Domain Access Permissions (RDC_PDAP49)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04C8	Peripheral Domain Access Permissions (RDC_PDAP50)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04CC	Peripheral Domain Access Permissions (RDC_PDAP51)	32	R/W	0000_00FFh	3.2.5.6/65

Table continues on the next page...

RDC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303D_04D0	Peripheral Domain Access Permissions (RDC_PDAP52)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04D4	Peripheral Domain Access Permissions (RDC_PDAP53)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04D8	Peripheral Domain Access Permissions (RDC_PDAP54)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04DC	Peripheral Domain Access Permissions (RDC_PDAP55)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04E0	Peripheral Domain Access Permissions (RDC_PDAP56)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04E4	Peripheral Domain Access Permissions (RDC_PDAP57)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04E8	Peripheral Domain Access Permissions (RDC_PDAP58)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04EC	Peripheral Domain Access Permissions (RDC_PDAP59)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04F0	Peripheral Domain Access Permissions (RDC_PDAP60)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04F4	Peripheral Domain Access Permissions (RDC_PDAP61)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04F8	Peripheral Domain Access Permissions (RDC_PDAP62)	32	R/W	0000_00FFh	3.2.5.6/65
303D_04FC	Peripheral Domain Access Permissions (RDC_PDAP63)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0500	Peripheral Domain Access Permissions (RDC_PDAP64)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0504	Peripheral Domain Access Permissions (RDC_PDAP65)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0508	Peripheral Domain Access Permissions (RDC_PDAP66)	32	R/W	0000_00FFh	3.2.5.6/65
303D_050C	Peripheral Domain Access Permissions (RDC_PDAP67)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0510	Peripheral Domain Access Permissions (RDC_PDAP68)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0514	Peripheral Domain Access Permissions (RDC_PDAP69)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0518	Peripheral Domain Access Permissions (RDC_PDAP70)	32	R/W	0000_00FFh	3.2.5.6/65
303D_051C	Peripheral Domain Access Permissions (RDC_PDAP71)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0520	Peripheral Domain Access Permissions (RDC_PDAP72)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0524	Peripheral Domain Access Permissions (RDC_PDAP73)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0528	Peripheral Domain Access Permissions (RDC_PDAP74)	32	R/W	0000_00FFh	3.2.5.6/65
303D_052C	Peripheral Domain Access Permissions (RDC_PDAP75)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0530	Peripheral Domain Access Permissions (RDC_PDAP76)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0534	Peripheral Domain Access Permissions (RDC_PDAP77)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0538	Peripheral Domain Access Permissions (RDC_PDAP78)	32	R/W	0000_00FFh	3.2.5.6/65
303D_053C	Peripheral Domain Access Permissions (RDC_PDAP79)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0540	Peripheral Domain Access Permissions (RDC_PDAP80)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0544	Peripheral Domain Access Permissions (RDC_PDAP81)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0548	Peripheral Domain Access Permissions (RDC_PDAP82)	32	R/W	0000_00FFh	3.2.5.6/65
303D_054C	Peripheral Domain Access Permissions (RDC_PDAP83)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0550	Peripheral Domain Access Permissions (RDC_PDAP84)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0554	Peripheral Domain Access Permissions (RDC_PDAP85)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0558	Peripheral Domain Access Permissions (RDC_PDAP86)	32	R/W	0000_00FFh	3.2.5.6/65
303D_055C	Peripheral Domain Access Permissions (RDC_PDAP87)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0560	Peripheral Domain Access Permissions (RDC_PDAP88)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0564	Peripheral Domain Access Permissions (RDC_PDAP89)	32	R/W	0000_00FFh	3.2.5.6/65

Table continues on the next page...

RDC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
303D_0568	Peripheral Domain Access Permissions (RDC_PDAP90)	32	R/W	0000_00FFh	3.2.5.6/65
303D_056C	Peripheral Domain Access Permissions (RDC_PDAP91)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0570	Peripheral Domain Access Permissions (RDC_PDAP92)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0574	Peripheral Domain Access Permissions (RDC_PDAP93)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0578	Peripheral Domain Access Permissions (RDC_PDAP94)	32	R/W	0000_00FFh	3.2.5.6/65
303D_057C	Peripheral Domain Access Permissions (RDC_PDAP95)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0580	Peripheral Domain Access Permissions (RDC_PDAP96)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0584	Peripheral Domain Access Permissions (RDC_PDAP97)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0588	Peripheral Domain Access Permissions (RDC_PDAP98)	32	R/W	0000_00FFh	3.2.5.6/65
303D_058C	Peripheral Domain Access Permissions (RDC_PDAP99)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0590	Peripheral Domain Access Permissions (RDC_PDAP100)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0594	Peripheral Domain Access Permissions (RDC_PDAP101)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0598	Peripheral Domain Access Permissions (RDC_PDAP102)	32	R/W	0000_00FFh	3.2.5.6/65
303D_059C	Peripheral Domain Access Permissions (RDC_PDAP103)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05A0	Peripheral Domain Access Permissions (RDC_PDAP104)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05A4	Peripheral Domain Access Permissions (RDC_PDAP105)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05A8	Peripheral Domain Access Permissions (RDC_PDAP106)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05AC	Peripheral Domain Access Permissions (RDC_PDAP107)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05B0	Peripheral Domain Access Permissions (RDC_PDAP108)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05B4	Peripheral Domain Access Permissions (RDC_PDAP109)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05B8	Peripheral Domain Access Permissions (RDC_PDAP110)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05BC	Peripheral Domain Access Permissions (RDC_PDAP111)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05C0	Peripheral Domain Access Permissions (RDC_PDAP112)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05C4	Peripheral Domain Access Permissions (RDC_PDAP113)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05C8	Peripheral Domain Access Permissions (RDC_PDAP114)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05CC	Peripheral Domain Access Permissions (RDC_PDAP115)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05D0	Peripheral Domain Access Permissions (RDC_PDAP116)	32	R/W	0000_00FFh	3.2.5.6/65
303D_05D4	Peripheral Domain Access Permissions (RDC_PDAP117)	32	R/W	0000_00FFh	3.2.5.6/65
303D_0800	Memory Region Start Address (RDC_MRSA0)	32	R/W	Undefined	3.2.5.7/66
303D_0804	Memory Region End Address (RDC_MREA0)	32	R/W	Undefined	3.2.5.8/68
303D_0808	Memory Region Control (RDC_MRC0)	32	R/W	0000_00FFh	3.2.5.9/69
303D_080C	Memory Region Violation Status (RDC_MRVS0)	32	R/W	0000_0000h	3.2.5.10/70
303D_0810	Memory Region Start Address (RDC_MRSA1)	32	R/W	Undefined	3.2.5.7/66
303D_0814	Memory Region End Address (RDC_MREA1)	32	R/W	Undefined	3.2.5.8/68
303D_0818	Memory Region Control (RDC_MRC1)	32	R/W	0000_00FFh	3.2.5.9/69
303D_081C	Memory Region Violation Status (RDC_MRVS1)	32	R/W	0000_0000h	3.2.5.10/70

Table continues on the next page...

RDC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303D_0820	Memory Region Start Address (RDC_MRSA2)	32	R/W	Undefined	3.2.5.7/66
303D_0824	Memory Region End Address (RDC_MREA2)	32	R/W	Undefined	3.2.5.8/68
303D_0828	Memory Region Control (RDC_MRC2)	32	R/W	0000_00FFh	3.2.5.9/69
303D_082C	Memory Region Violation Status (RDC_MRVS2)	32	R/W	0000_0000h	3.2.5.10/70
303D_0830	Memory Region Start Address (RDC_MRSA3)	32	R/W	Undefined	3.2.5.7/66
303D_0834	Memory Region End Address (RDC_MREA3)	32	R/W	Undefined	3.2.5.8/68
303D_0838	Memory Region Control (RDC_MRC3)	32	R/W	0000_00FFh	3.2.5.9/69
303D_083C	Memory Region Violation Status (RDC_MRVS3)	32	R/W	0000_0000h	3.2.5.10/70
303D_0840	Memory Region Start Address (RDC_MRSA4)	32	R/W	Undefined	3.2.5.7/66
303D_0844	Memory Region End Address (RDC_MREA4)	32	R/W	Undefined	3.2.5.8/68
303D_0848	Memory Region Control (RDC_MRC4)	32	R/W	0000_00FFh	3.2.5.9/69
303D_084C	Memory Region Violation Status (RDC_MRVS4)	32	R/W	0000_0000h	3.2.5.10/70
303D_0850	Memory Region Start Address (RDC_MRSA5)	32	R/W	Undefined	3.2.5.7/66
303D_0854	Memory Region End Address (RDC_MREA5)	32	R/W	Undefined	3.2.5.8/68
303D_0858	Memory Region Control (RDC_MRC5)	32	R/W	0000_00FFh	3.2.5.9/69
303D_085C	Memory Region Violation Status (RDC_MRVS5)	32	R/W	0000_0000h	3.2.5.10/70
303D_0860	Memory Region Start Address (RDC_MRSA6)	32	R/W	Undefined	3.2.5.7/66
303D_0864	Memory Region End Address (RDC_MREA6)	32	R/W	Undefined	3.2.5.8/68
303D_0868	Memory Region Control (RDC_MRC6)	32	R/W	0000_00FFh	3.2.5.9/69
303D_086C	Memory Region Violation Status (RDC_MRVS6)	32	R/W	0000_0000h	3.2.5.10/70
303D_0870	Memory Region Start Address (RDC_MRSA7)	32	R/W	Undefined	3.2.5.7/66
303D_0874	Memory Region End Address (RDC_MREA7)	32	R/W	Undefined	3.2.5.8/68
303D_0878	Memory Region Control (RDC_MRC7)	32	R/W	0000_00FFh	3.2.5.9/69
303D_087C	Memory Region Violation Status (RDC_MRVS7)	32	R/W	0000_0000h	3.2.5.10/70
303D_0880	Memory Region Start Address (RDC_MRSA8)	32	R/W	Undefined	3.2.5.7/66
303D_0884	Memory Region End Address (RDC_MREA8)	32	R/W	Undefined	3.2.5.8/68
303D_0888	Memory Region Control (RDC_MRC8)	32	R/W	0000_00FFh	3.2.5.9/69
303D_088C	Memory Region Violation Status (RDC_MRVS8)	32	R/W	0000_0000h	3.2.5.10/70
303D_0890	Memory Region Start Address (RDC_MRSA9)	32	R/W	Undefined	3.2.5.7/66
303D_0894	Memory Region End Address (RDC_MREA9)	32	R/W	Undefined	3.2.5.8/68
303D_0898	Memory Region Control (RDC_MRC9)	32	R/W	0000_00FFh	3.2.5.9/69
303D_089C	Memory Region Violation Status (RDC_MRVS9)	32	R/W	0000_0000h	3.2.5.10/70

Table continues on the next page...

RDC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303D_08A0	Memory Region Start Address (RDC_MRSA10)	32	R/W	Undefined	3.2.5.7/66
303D_08A4	Memory Region End Address (RDC_MREA10)	32	R/W	Undefined	3.2.5.8/68
303D_08A8	Memory Region Control (RDC_MRC10)	32	R/W	0000_00FFh	3.2.5.9/69
303D_08AC	Memory Region Violation Status (RDC_MRVS10)	32	R/W	0000_0000h	3.2.5.10/70
303D_08B0	Memory Region Start Address (RDC_MRSA11)	32	R/W	Undefined	3.2.5.7/66
303D_08B4	Memory Region End Address (RDC_MREA11)	32	R/W	Undefined	3.2.5.8/68
303D_08B8	Memory Region Control (RDC_MRC11)	32	R/W	0000_00FFh	3.2.5.9/69
303D_08BC	Memory Region Violation Status (RDC_MRVS11)	32	R/W	0000_0000h	3.2.5.10/70
303D_08C0	Memory Region Start Address (RDC_MRSA12)	32	R/W	Undefined	3.2.5.7/66
303D_08C4	Memory Region End Address (RDC_MREA12)	32	R/W	Undefined	3.2.5.8/68
303D_08C8	Memory Region Control (RDC_MRC12)	32	R/W	0000_00FFh	3.2.5.9/69
303D_08CC	Memory Region Violation Status (RDC_MRVS12)	32	R/W	0000_0000h	3.2.5.10/70
303D_08D0	Memory Region Start Address (RDC_MRSA13)	32	R/W	Undefined	3.2.5.7/66
303D_08D4	Memory Region End Address (RDC_MREA13)	32	R/W	Undefined	3.2.5.8/68
303D_08D8	Memory Region Control (RDC_MRC13)	32	R/W	0000_00FFh	3.2.5.9/69
303D_08DC	Memory Region Violation Status (RDC_MRVS13)	32	R/W	0000_0000h	3.2.5.10/70
303D_08E0	Memory Region Start Address (RDC_MRSA14)	32	R/W	Undefined	3.2.5.7/66
303D_08E4	Memory Region End Address (RDC_MREA14)	32	R/W	Undefined	3.2.5.8/68
303D_08E8	Memory Region Control (RDC_MRC14)	32	R/W	0000_00FFh	3.2.5.9/69
303D_08EC	Memory Region Violation Status (RDC_MRVS14)	32	R/W	0000_0000h	3.2.5.10/70
303D_08F0	Memory Region Start Address (RDC_MRSA15)	32	R/W	Undefined	3.2.5.7/66
303D_08F4	Memory Region End Address (RDC_MREA15)	32	R/W	Undefined	3.2.5.8/68
303D_08F8	Memory Region Control (RDC_MRC15)	32	R/W	0000_00FFh	3.2.5.9/69
303D_08FC	Memory Region Violation Status (RDC_MRVS15)	32	R/W	0000_0000h	3.2.5.10/70
303D_0900	Memory Region Start Address (RDC_MRSA16)	32	R/W	Undefined	3.2.5.7/66
303D_0904	Memory Region End Address (RDC_MREA16)	32	R/W	Undefined	3.2.5.8/68
303D_0908	Memory Region Control (RDC_MRC16)	32	R/W	0000_00FFh	3.2.5.9/69
303D_090C	Memory Region Violation Status (RDC_MRVS16)	32	R/W	0000_0000h	3.2.5.10/70
303D_0910	Memory Region Start Address (RDC_MRSA17)	32	R/W	Undefined	3.2.5.7/66
303D_0914	Memory Region End Address (RDC_MREA17)	32	R/W	Undefined	3.2.5.8/68
303D_0918	Memory Region Control (RDC_MRC17)	32	R/W	0000_00FFh	3.2.5.9/69
303D_091C	Memory Region Violation Status (RDC_MRVS17)	32	R/W	0000_0000h	3.2.5.10/70

Table continues on the next page...

RDC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303D_0920	Memory Region Start Address (RDC_MRSA18)	32	R/W	Undefined	3.2.5.7/66
303D_0924	Memory Region End Address (RDC_MREA18)	32	R/W	Undefined	3.2.5.8/68
303D_0928	Memory Region Control (RDC_MRC18)	32	R/W	0000_00FFh	3.2.5.9/69
303D_092C	Memory Region Violation Status (RDC_MRVS18)	32	R/W	0000_0000h	3.2.5.10/70
303D_0930	Memory Region Start Address (RDC_MRSA19)	32	R/W	Undefined	3.2.5.7/66
303D_0934	Memory Region End Address (RDC_MREA19)	32	R/W	Undefined	3.2.5.8/68
303D_0938	Memory Region Control (RDC_MRC19)	32	R/W	0000_00FFh	3.2.5.9/69
303D_093C	Memory Region Violation Status (RDC_MRVS19)	32	R/W	0000_0000h	3.2.5.10/70
303D_0940	Memory Region Start Address (RDC_MRSA20)	32	R/W	Undefined	3.2.5.7/66
303D_0944	Memory Region End Address (RDC_MREA20)	32	R/W	Undefined	3.2.5.8/68
303D_0948	Memory Region Control (RDC_MRC20)	32	R/W	0000_00FFh	3.2.5.9/69
303D_094C	Memory Region Violation Status (RDC_MRVS20)	32	R/W	0000_0000h	3.2.5.10/70
303D_0950	Memory Region Start Address (RDC_MRSA21)	32	R/W	Undefined	3.2.5.7/66
303D_0954	Memory Region End Address (RDC_MREA21)	32	R/W	Undefined	3.2.5.8/68
303D_0958	Memory Region Control (RDC_MRC21)	32	R/W	0000_00FFh	3.2.5.9/69
303D_095C	Memory Region Violation Status (RDC_MRVS21)	32	R/W	0000_0000h	3.2.5.10/70
303D_0960	Memory Region Start Address (RDC_MRSA22)	32	R/W	Undefined	3.2.5.7/66
303D_0964	Memory Region End Address (RDC_MREA22)	32	R/W	Undefined	3.2.5.8/68
303D_0968	Memory Region Control (RDC_MRC22)	32	R/W	0000_00FFh	3.2.5.9/69
303D_096C	Memory Region Violation Status (RDC_MRVS22)	32	R/W	0000_0000h	3.2.5.10/70
303D_0970	Memory Region Start Address (RDC_MRSA23)	32	R/W	Undefined	3.2.5.7/66
303D_0974	Memory Region End Address (RDC_MREA23)	32	R/W	Undefined	3.2.5.8/68
303D_0978	Memory Region Control (RDC_MRC23)	32	R/W	0000_00FFh	3.2.5.9/69
303D_097C	Memory Region Violation Status (RDC_MRVS23)	32	R/W	0000_0000h	3.2.5.10/70
303D_0980	Memory Region Start Address (RDC_MRSA24)	32	R/W	Undefined	3.2.5.7/66
303D_0984	Memory Region End Address (RDC_MREA24)	32	R/W	Undefined	3.2.5.8/68
303D_0988	Memory Region Control (RDC_MRC24)	32	R/W	0000_00FFh	3.2.5.9/69
303D_098C	Memory Region Violation Status (RDC_MRVS24)	32	R/W	0000_0000h	3.2.5.10/70
303D_0990	Memory Region Start Address (RDC_MRSA25)	32	R/W	Undefined	3.2.5.7/66
303D_0994	Memory Region End Address (RDC_MREA25)	32	R/W	Undefined	3.2.5.8/68
303D_0998	Memory Region Control (RDC_MRC25)	32	R/W	0000_00FFh	3.2.5.9/69
303D_099C	Memory Region Violation Status (RDC_MRVS25)	32	R/W	0000_0000h	3.2.5.10/70

Table continues on the next page...

RDC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303D_09A0	Memory Region Start Address (RDC_MRSA26)	32	R/W	Undefined	3.2.5.7/66
303D_09A4	Memory Region End Address (RDC_MREA26)	32	R/W	Undefined	3.2.5.8/68
303D_09A8	Memory Region Control (RDC_MRC26)	32	R/W	0000_00FFh	3.2.5.9/69
303D_09AC	Memory Region Violation Status (RDC_MRVS26)	32	R/W	0000_0000h	3.2.5.10/70
303D_09B0	Memory Region Start Address (RDC_MRSA27)	32	R/W	Undefined	3.2.5.7/66
303D_09B4	Memory Region End Address (RDC_MREA27)	32	R/W	Undefined	3.2.5.8/68
303D_09B8	Memory Region Control (RDC_MRC27)	32	R/W	0000_00FFh	3.2.5.9/69
303D_09BC	Memory Region Violation Status (RDC_MRVS27)	32	R/W	0000_0000h	3.2.5.10/70
303D_09C0	Memory Region Start Address (RDC_MRSA28)	32	R/W	Undefined	3.2.5.7/66
303D_09C4	Memory Region End Address (RDC_MREA28)	32	R/W	Undefined	3.2.5.8/68
303D_09C8	Memory Region Control (RDC_MRC28)	32	R/W	0000_00FFh	3.2.5.9/69
303D_09CC	Memory Region Violation Status (RDC_MRVS28)	32	R/W	0000_0000h	3.2.5.10/70
303D_09D0	Memory Region Start Address (RDC_MRSA29)	32	R/W	Undefined	3.2.5.7/66
303D_09D4	Memory Region End Address (RDC_MREA29)	32	R/W	Undefined	3.2.5.8/68
303D_09D8	Memory Region Control (RDC_MRC29)	32	R/W	0000_00FFh	3.2.5.9/69
303D_09DC	Memory Region Violation Status (RDC_MRVS29)	32	R/W	0000_0000h	3.2.5.10/70
303D_09E0	Memory Region Start Address (RDC_MRSA30)	32	R/W	Undefined	3.2.5.7/66
303D_09E4	Memory Region End Address (RDC_MREA30)	32	R/W	Undefined	3.2.5.8/68
303D_09E8	Memory Region Control (RDC_MRC30)	32	R/W	0000_00FFh	3.2.5.9/69
303D_09EC	Memory Region Violation Status (RDC_MRVS30)	32	R/W	0000_0000h	3.2.5.10/70
303D_09F0	Memory Region Start Address (RDC_MRSA31)	32	R/W	Undefined	3.2.5.7/66
303D_09F4	Memory Region End Address (RDC_MREA31)	32	R/W	Undefined	3.2.5.8/68
303D_09F8	Memory Region Control (RDC_MRC31)	32	R/W	0000_00FFh	3.2.5.9/69
303D_09FC	Memory Region Violation Status (RDC_MRVS31)	32	R/W	0000_0000h	3.2.5.10/70
303D_0A00	Memory Region Start Address (RDC_MRSA32)	32	R/W	Undefined	3.2.5.7/66
303D_0A04	Memory Region End Address (RDC_MREA32)	32	R/W	Undefined	3.2.5.8/68
303D_0A08	Memory Region Control (RDC_MRC32)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0A0C	Memory Region Violation Status (RDC_MRVS32)	32	R/W	0000_0000h	3.2.5.10/70
303D_0A10	Memory Region Start Address (RDC_MRSA33)	32	R/W	Undefined	3.2.5.7/66
303D_0A14	Memory Region End Address (RDC_MREA33)	32	R/W	Undefined	3.2.5.8/68
303D_0A18	Memory Region Control (RDC_MRC33)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0A1C	Memory Region Violation Status (RDC_MRVS33)	32	R/W	0000_0000h	3.2.5.10/70

Table continues on the next page...

RDC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303D_0A20	Memory Region Start Address (RDC_MRSA34)	32	R/W	Undefined	3.2.5.7/66
303D_0A24	Memory Region End Address (RDC_MREA34)	32	R/W	Undefined	3.2.5.8/68
303D_0A28	Memory Region Control (RDC_MRC34)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0A2C	Memory Region Violation Status (RDC_MRVS34)	32	R/W	0000_0000h	3.2.5.10/70
303D_0A30	Memory Region Start Address (RDC_MRSA35)	32	R/W	Undefined	3.2.5.7/66
303D_0A34	Memory Region End Address (RDC_MREA35)	32	R/W	Undefined	3.2.5.8/68
303D_0A38	Memory Region Control (RDC_MRC35)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0A3C	Memory Region Violation Status (RDC_MRVS35)	32	R/W	0000_0000h	3.2.5.10/70
303D_0A40	Memory Region Start Address (RDC_MRSA36)	32	R/W	Undefined	3.2.5.7/66
303D_0A44	Memory Region End Address (RDC_MREA36)	32	R/W	Undefined	3.2.5.8/68
303D_0A48	Memory Region Control (RDC_MRC36)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0A4C	Memory Region Violation Status (RDC_MRVS36)	32	R/W	0000_0000h	3.2.5.10/70
303D_0A50	Memory Region Start Address (RDC_MRSA37)	32	R/W	Undefined	3.2.5.7/66
303D_0A54	Memory Region End Address (RDC_MREA37)	32	R/W	Undefined	3.2.5.8/68
303D_0A58	Memory Region Control (RDC_MRC37)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0A5C	Memory Region Violation Status (RDC_MRVS37)	32	R/W	0000_0000h	3.2.5.10/70
303D_0A60	Memory Region Start Address (RDC_MRSA38)	32	R/W	Undefined	3.2.5.7/66
303D_0A64	Memory Region End Address (RDC_MREA38)	32	R/W	Undefined	3.2.5.8/68
303D_0A68	Memory Region Control (RDC_MRC38)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0A6C	Memory Region Violation Status (RDC_MRVS38)	32	R/W	0000_0000h	3.2.5.10/70
303D_0A70	Memory Region Start Address (RDC_MRSA39)	32	R/W	Undefined	3.2.5.7/66
303D_0A74	Memory Region End Address (RDC_MREA39)	32	R/W	Undefined	3.2.5.8/68
303D_0A78	Memory Region Control (RDC_MRC39)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0A7C	Memory Region Violation Status (RDC_MRVS39)	32	R/W	0000_0000h	3.2.5.10/70
303D_0A80	Memory Region Start Address (RDC_MRSA40)	32	R/W	Undefined	3.2.5.7/66
303D_0A84	Memory Region End Address (RDC_MREA40)	32	R/W	Undefined	3.2.5.8/68
303D_0A88	Memory Region Control (RDC_MRC40)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0A8C	Memory Region Violation Status (RDC_MRVS40)	32	R/W	0000_0000h	3.2.5.10/70
303D_0A90	Memory Region Start Address (RDC_MRSA41)	32	R/W	Undefined	3.2.5.7/66
303D_0A94	Memory Region End Address (RDC_MREA41)	32	R/W	Undefined	3.2.5.8/68
303D_0A98	Memory Region Control (RDC_MRC41)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0A9C	Memory Region Violation Status (RDC_MRVS41)	32	R/W	0000_0000h	3.2.5.10/70

Table continues on the next page...

RDC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303D_0AA0	Memory Region Start Address (RDC_MRSA42)	32	R/W	Undefined	3.2.5.7/66
303D_0AA4	Memory Region End Address (RDC_MREA42)	32	R/W	Undefined	3.2.5.8/68
303D_0AA8	Memory Region Control (RDC_MRC42)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0AAC	Memory Region Violation Status (RDC_MRVS42)	32	R/W	0000_0000h	3.2.5.10/70
303D_0AB0	Memory Region Start Address (RDC_MRSA43)	32	R/W	Undefined	3.2.5.7/66
303D_0AB4	Memory Region End Address (RDC_MREA43)	32	R/W	Undefined	3.2.5.8/68
303D_0AB8	Memory Region Control (RDC_MRC43)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0ABC	Memory Region Violation Status (RDC_MRVS43)	32	R/W	0000_0000h	3.2.5.10/70
303D_0AC0	Memory Region Start Address (RDC_MRSA44)	32	R/W	Undefined	3.2.5.7/66
303D_0AC4	Memory Region End Address (RDC_MREA44)	32	R/W	Undefined	3.2.5.8/68
303D_0AC8	Memory Region Control (RDC_MRC44)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0ACC	Memory Region Violation Status (RDC_MRVS44)	32	R/W	0000_0000h	3.2.5.10/70
303D_0AD0	Memory Region Start Address (RDC_MRSA45)	32	R/W	Undefined	3.2.5.7/66
303D_0AD4	Memory Region End Address (RDC_MREA45)	32	R/W	Undefined	3.2.5.8/68
303D_0AD8	Memory Region Control (RDC_MRC45)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0ADC	Memory Region Violation Status (RDC_MRVS45)	32	R/W	0000_0000h	3.2.5.10/70
303D_0AE0	Memory Region Start Address (RDC_MRSA46)	32	R/W	Undefined	3.2.5.7/66
303D_0AE4	Memory Region End Address (RDC_MREA46)	32	R/W	Undefined	3.2.5.8/68
303D_0AE8	Memory Region Control (RDC_MRC46)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0AEC	Memory Region Violation Status (RDC_MRVS46)	32	R/W	0000_0000h	3.2.5.10/70
303D_0AF0	Memory Region Start Address (RDC_MRSA47)	32	R/W	Undefined	3.2.5.7/66
303D_0AF4	Memory Region End Address (RDC_MREA47)	32	R/W	Undefined	3.2.5.8/68
303D_0AF8	Memory Region Control (RDC_MRC47)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0AFC	Memory Region Violation Status (RDC_MRVS47)	32	R/W	0000_0000h	3.2.5.10/70
303D_0B00	Memory Region Start Address (RDC_MRSA48)	32	R/W	Undefined	3.2.5.7/66
303D_0B04	Memory Region End Address (RDC_MREA48)	32	R/W	Undefined	3.2.5.8/68
303D_0B08	Memory Region Control (RDC_MRC48)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0B0C	Memory Region Violation Status (RDC_MRVS48)	32	R/W	0000_0000h	3.2.5.10/70
303D_0B10	Memory Region Start Address (RDC_MRSA49)	32	R/W	Undefined	3.2.5.7/66
303D_0B14	Memory Region End Address (RDC_MREA49)	32	R/W	Undefined	3.2.5.8/68
303D_0B18	Memory Region Control (RDC_MRC49)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0B1C	Memory Region Violation Status (RDC_MRVS49)	32	R/W	0000_0000h	3.2.5.10/70

Table continues on the next page...

RDC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
303D_0B20	Memory Region Start Address (RDC_MRSA50)	32	R/W	Undefined	3.2.5.7/66
303D_0B24	Memory Region End Address (RDC_MREA50)	32	R/W	Undefined	3.2.5.8/68
303D_0B28	Memory Region Control (RDC_MRC50)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0B2C	Memory Region Violation Status (RDC_MRVS50)	32	R/W	0000_0000h	3.2.5.10/70
303D_0B30	Memory Region Start Address (RDC_MRSA51)	32	R/W	Undefined	3.2.5.7/66
303D_0B34	Memory Region End Address (RDC_MREA51)	32	R/W	Undefined	3.2.5.8/68
303D_0B38	Memory Region Control (RDC_MRC51)	32	R/W	0000_00FFh	3.2.5.9/69
303D_0B3C	Memory Region Violation Status (RDC_MRVS51)	32	R/W	0000_0000h	3.2.5.10/70

3.2.5.1 Version Information (RDC_VIR)

The VIR provides version information including the number of domains, number of master slots, number of peripheral slots, and number of memory regions.

Address: 303D_0000h base + 0h offset = 303D_0000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				NRGN								NPER								NMSTR								NDID			
W																																
Reset	0	0	0	0	0	0	1	1	0	1	1	1	0	1	1	0	1	1	1	0	0	0	1	0	0	0	0	0	0	1	0	0

RDC_VIR field descriptions

Field	Description
31–28 Reserved	This field is reserved.
27–20 NRGN	Number of Memory Regions Indicates the number of memory regions in this instance of the RDC.
19–12 NPER	Number of Peripherals Indicates the number of peripherals that can be isolated or safe-shared
11–4 NMSTR	Number of Masters Indicates the number of masters supported by this instance of RDC.
NDID	Number of Domains Indicates the number of domain ids supported by this instance of the RDC. Add one to the register value to get the actual number of domains.

3.2.5.2 Status (RDC_STAT)

Address: 303D_0000h base + 24h offset = 303D_0024h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								PDS	Reserved				DID		
W																
Reset	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0

RDC_STAT field descriptions

Field	Description
31–9 Reserved	This field is reserved.
8 PDS	Power Domain Status Indicates if the "Power Down" memory regions are powered and available. Power Down memory regions are only those memory regions susceptible to power outage for power savings are unavailable if this is zero. "Always-On" memory regions remain available. Always On memory regions are those regions that are not powered down unless the entire SoC is powered down. This signal remains low until all access controls have been restored to the domain. 0 Power Down Domain is OFF 1 Power Down Domain is ON
7–4 Reserved	This field is reserved.
DID	Domain ID The Domain ID of the core or bus master that is reading this. The value is different for requests from different domains.

3.2.5.3 Interrupt and Control (RDC_INTCTRL)

Address: 303D_0000h base + 28h offset = 303D_0028h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															RCI_EN
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

RDC_INTCTRL field descriptions

Field	Description
31–1 Reserved	This field is reserved.
0 RCI_EN	Restoration Complete Interrupt Interrupt generated when the RDC has completed restoring state to a recently re-powered memory regions. 0 Interrupt Disabled 1 Interrupt Enabled

3.2.5.4 Interrupt Status (RDC_INTSTAT)

Indication of Interrupt Pending for State Restoration

Address: 303D_0000h base + 2Ch offset = 303D_002Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															INT
W																w1c
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

RDC_INTSTAT field descriptions

Field	Description
31–1 Reserved	This field is reserved.
0 INT	Interrupt Status Indicates state of interrupt signal for state restoration. This is that status of the interrupt enabled in RDC_INTCTRL. Write one to interrupt status to clear it. 0 No Interrupt Pending 1 Interrupt Pending

3.2.5.5 Master Domain Assignment (RDC_MDAn)

Address: 303D_0000h base + 200h offset + (4d × i), where i=0d to 26d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	LCK	Reserved														
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DID
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

RDC_MDAn field descriptions

Field	Description
31 LCK	0 Not Locked 1 Locked
30–2 Reserved	This field is reserved.
DID	Domain ID Indicates the domain to which the Master is assigned 00 Master assigned to Processing Domain 0 01 Master assigned to Processing Domain 1 10 Master assigned to Processing Domain 2 11 Master assigned to Processing Domain 3

3.2.5.6 Peripheral Domain Access Permissions (RDC_PDAP_n)

Address: 303D_0000h base + 400h offset + (4d × i), where i=0d to 117d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W	LCK	SREQ	Reserved													
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W	Reserved								D3R	D3W	D2R	D2W	D1R	D1W	D0R	D0W
Reset	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

RDC_PDAP_n field descriptions

Field	Description
31 LCK	Peripheral Permissions Lock When set prevents further modification of the Peripheral Domain Access Permissions (sticky bit until reset) 0 Not Locked 1 Locked
30 SREQ	Semaphore Required When set the hardware semaphore state enforces the semaphore lock. If a domain has access permissions and a semaphore has locked a shared peripheral then only the domain holding the semaphore signal can access this peripheral. 0 Semaphores have no effect 1 Semaphores are enforced
29–8 Reserved	This field is reserved.
7 D3R	Domain 3 Read Access 0 No Read Access 1 Read Access Allowed
6 D3W	Domain 3 Write Access 0 No Write Access 1 Write Access Allowed
5 D2R	Domain 2 Read Access 0 No Read Access 1 Read Access Allowed
4 D2W	Domain 2 Write Access 0 No Write Access 1 Write Access Allowed
3 D1R	Domain 1 Read Access

Table continues on the next page...

RDC_PDAP_n field descriptions (continued)

Field	Description
	0 No Read Access 1 Read Access Allowed
2 D1W	Domain 1 Write Access 0 No Write Access 1 Write Access Allowed
1 D0R	Domain 0 Read Access 0 No Read Access 1 Read Access Allowed
0 D0W	Domain 0 Write Access 0 No Write Access 1 Write Access Allowed

3.2.5.7 Memory Region Start Address (RDC_MRSA_n)**NOTE**

The DDR space is 33-bit width. The RDC memory region registers are 32-bit width. The RDC configuration is the most significant bits in the DDR address space (32:1). To set the start address for this configuration, the MRSA value should be shifted 1-bit and added to the DDR base address. The example below illustrates how to calculate the proper start and end address value. Please refer to the Memory Map for the actual DDR base address.

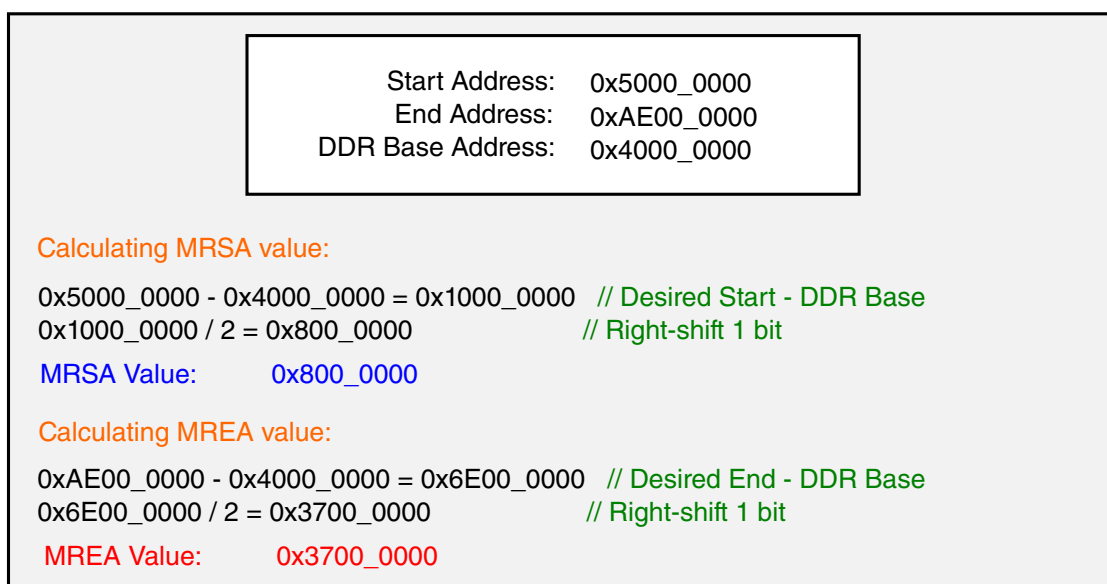


Figure 3-5. Calculating Address Value Example

Address: 303D_0000h base + 800h offset + (16d × i), where i=0d to 51d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	SADR																Reserved															
W																																
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	

* Notes:

- x = Undefined at reset.

RDC_MRSAn field descriptions

Field	Description
31–7 SADR	Start address for memory region Lower bound (inclusive) modulo the defined granularity byte size of a region. The region size (granularity) is defined for each Memory/Port in the Memory Region Map section. Region boundaries are aligned to the minimum possible region size for the Memory/Port.
Reserved	This field is reserved.

3.2.5.8 Memory Region End Address (RDC_MREAn)

NOTE

The DDR space is 33-bit width. The RDC memory region registers are 32-bit width. The RDC configuration is the most significant bits in the DDR address space (32:1). To set the start address for this configuration, the MRSA value should be shifted 1-bit and added to the DDR base address. The example below illustrates how to calculate the proper start and end address value. Please refer to the Memory Map for the actual DDR base address.

Start Address: 0x5000_0000
 End Address: 0xAE00_0000
 DDR Base Address: 0x4000_0000

Calculating MRSA value:

0x5000_0000 - 0x4000_0000 = 0x1000_0000 // Desired Start - DDR Base
 0x1000_0000 / 2 = 0x800_0000 // Right-shift 1 bit

MRSA Value: 0x800_0000

Calculating MREA value:

0xAE00_0000 - 0x4000_0000 = 0x6E00_0000 // Desired End - DDR Base
 0x6E00_0000 / 2 = 0x3700_0000 // Right-shift 1 bit

MREA Value: 0x3700_0000

Figure 3-6. Calculating Address Value Example

Address: 303D_0000h base + 804h offset + (16d × i), where i=0d to 51d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	EADR																Reserved															
W																																
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

* Notes:

- x = Undefined at reset.

RDC_MREAn field descriptions

Field	Description
31–7 EADR	Upper bound for memory region

Table continues on the next page...

RDC_MREAn field descriptions (continued)

Field	Description
	Upper bound (exclusive) modulo the defined granularity byte size of a region. The region size (granularity) is defined for each Memory/Port in the Memory Region Map section. Region boundaries are aligned to the minimum possible region size for the Memory/Port.
Reserved	This field is reserved.

3.2.5.9 Memory Region Control (RDC_MRCn)

Address: 303D_0000h base + 808h offset + (16d × i), where i=0d to 51d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W	LCK	ENA	Reserved													
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W	Reserved								D3R	D3W	D2R	D2W	D1R	D1W	D0R	D0W
Reset	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

RDC_MRCn field descriptions

Field	Description
31 LCK	Region Lock Locks all region fields from further modification except ENA, which can be set but not reset after LCK is set. LCK is a sticky bit. 0 No Lock. All fields in this register may be modified. 1 Locked. No fields in this register may be modified except ENA, which may be set but not cleared.
30 ENA	Region Enable Activates the memory region. If the region is not activated then the permissions and address boundaries have not affect and the region will be fully accessible. 0 Memory region is not defined or restricted. 1 Memory boundaries, domain permissions and controls are in effect.
29–8 Reserved	This field is reserved.
7 D3R	Domain 3 Read Access to Region 0 Processing Domain 3 does not have Read access to the memory region 1 Processing Domain 3 has Read access to the memory region
6 D3W	Domain 3 Write Access to Region 0 Processing Domain 3 does not have Write access to the memory region 1 Processing Domain 3 has Read access to the memory region
5 D2R	Domain 2 Read Access to Region

Table continues on the next page...

RDC_MRCn field descriptions (continued)

Field	Description
	0 Processing Domain 2 does not have Read access to the memory region 1 Processing Domain 2 has Read access to the memory region
4 D2W	Domain 2 Write Access to Region 0 Processing Domain 2 does not have Write access to the memory region 1 Processing Domain 2 has Write access to the memory region
3 D1R	Domain 1 Read Access to Region 0 Processing Domain 1 does not have Read access to the memory region 1 Processing Domain 1 has Read access to the memory region
2 D1W	Domain 1 Write Access to Region 0 Processing Domain 1 does not have Write access to the memory region 1 Processing Domain 1 has Write access to the memory region
1 D0R	Domain 0 Read Access to Region 0 Processing Domain 0 does not have Read access to the memory region 1 Processing Domain 0 has Read access to the memory region
0 D0W	Domain 0 Write Access to Region 0 Processing Domain 0 does not have Write access to the memory region 1 Processing Domain 0 has Write access to the memory region

3.2.5.10 Memory Region Violation Status (RDC_MRVS_n)

Address: 303D_0000h base + 80Ch offset + (16d × i), where i=0d to 51d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	VADR															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	VADR											AD	Reserved		VDID	
W												w1c				
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

RDC_MRVS_n field descriptions

Field	Description
31–5 VADR	Violating Address The address of the denied access. The first access violation is captured. Subsequent violations are ignored until the status register is cleared. Contents are cleared upon reading the register. Clearing of contents occurs only when the status is read by the memory region's associated domain ID (s).

Table continues on the next page...

RDC_MRVS_n field descriptions (continued)

Field	Description
4 AD	Access Denied Access to a memory region denied. This bit is cleared when this bit is written by one of the allowed domains.
3–2 Reserved	This field is reserved.
VDID	Violating Domain ID The domain ID of the denied access. The first access violation is captured. Subsequent violations are ignored until the status register is cleared. Contents are cleared upon reading the register. 00 Processing Domain 0 01 Processing Domain 1 10 Processing Domain 2 11 Processing Domain 3

3.2.6 RDC SEMA42 Memory Map/Register Definition

Only Supervisor Mode accesses are allowed on these registers. User accesses generate an error termination.

RDC_SEMAPHORE memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
303B_0000	Gate Register (RDC_SEMAPHORE1_GATE0)	8	R/W	00h	3.2.6.1/75
303B_0001	Gate Register (RDC_SEMAPHORE1_GATE1)	8	R/W	00h	3.2.6.1/75
303B_0002	Gate Register (RDC_SEMAPHORE1_GATE2)	8	R/W	00h	3.2.6.1/75
303B_0003	Gate Register (RDC_SEMAPHORE1_GATE3)	8	R/W	00h	3.2.6.1/75
303B_0004	Gate Register (RDC_SEMAPHORE1_GATE4)	8	R/W	00h	3.2.6.1/75
303B_0005	Gate Register (RDC_SEMAPHORE1_GATE5)	8	R/W	00h	3.2.6.1/75
303B_0006	Gate Register (RDC_SEMAPHORE1_GATE6)	8	R/W	00h	3.2.6.1/75
303B_0007	Gate Register (RDC_SEMAPHORE1_GATE7)	8	R/W	00h	3.2.6.1/75
303B_0008	Gate Register (RDC_SEMAPHORE1_GATE8)	8	R/W	00h	3.2.6.1/75
303B_0009	Gate Register (RDC_SEMAPHORE1_GATE9)	8	R/W	00h	3.2.6.1/75
303B_000A	Gate Register (RDC_SEMAPHORE1_GATE10)	8	R/W	00h	3.2.6.1/75
303B_000B	Gate Register (RDC_SEMAPHORE1_GATE11)	8	R/W	00h	3.2.6.1/75
303B_000C	Gate Register (RDC_SEMAPHORE1_GATE12)	8	R/W	00h	3.2.6.1/75
303B_000D	Gate Register (RDC_SEMAPHORE1_GATE13)	8	R/W	00h	3.2.6.1/75
303B_000E	Gate Register (RDC_SEMAPHORE1_GATE14)	8	R/W	00h	3.2.6.1/75

Table continues on the next page...

RDC_SEMAPHORE memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303B_000F	Gate Register (RDC_SEMAPHORE1_GATE15)	8	R/W	00h	3.2.6.1/75
303B_0010	Gate Register (RDC_SEMAPHORE1_GATE16)	8	R/W	00h	3.2.6.1/75
303B_0011	Gate Register (RDC_SEMAPHORE1_GATE17)	8	R/W	00h	3.2.6.1/75
303B_0012	Gate Register (RDC_SEMAPHORE1_GATE18)	8	R/W	00h	3.2.6.1/75
303B_0013	Gate Register (RDC_SEMAPHORE1_GATE19)	8	R/W	00h	3.2.6.1/75
303B_0014	Gate Register (RDC_SEMAPHORE1_GATE20)	8	R/W	00h	3.2.6.1/75
303B_0015	Gate Register (RDC_SEMAPHORE1_GATE21)	8	R/W	00h	3.2.6.1/75
303B_0016	Gate Register (RDC_SEMAPHORE1_GATE22)	8	R/W	00h	3.2.6.1/75
303B_0017	Gate Register (RDC_SEMAPHORE1_GATE23)	8	R/W	00h	3.2.6.1/75
303B_0018	Gate Register (RDC_SEMAPHORE1_GATE24)	8	R/W	00h	3.2.6.1/75
303B_0019	Gate Register (RDC_SEMAPHORE1_GATE25)	8	R/W	00h	3.2.6.1/75
303B_001A	Gate Register (RDC_SEMAPHORE1_GATE26)	8	R/W	00h	3.2.6.1/75
303B_001B	Gate Register (RDC_SEMAPHORE1_GATE27)	8	R/W	00h	3.2.6.1/75
303B_001C	Gate Register (RDC_SEMAPHORE1_GATE28)	8	R/W	00h	3.2.6.1/75
303B_001D	Gate Register (RDC_SEMAPHORE1_GATE29)	8	R/W	00h	3.2.6.1/75
303B_001E	Gate Register (RDC_SEMAPHORE1_GATE30)	8	R/W	00h	3.2.6.1/75
303B_001F	Gate Register (RDC_SEMAPHORE1_GATE31)	8	R/W	00h	3.2.6.1/75
303B_0020	Gate Register (RDC_SEMAPHORE1_GATE32)	8	R/W	00h	3.2.6.1/75
303B_0021	Gate Register (RDC_SEMAPHORE1_GATE33)	8	R/W	00h	3.2.6.1/75
303B_0022	Gate Register (RDC_SEMAPHORE1_GATE34)	8	R/W	00h	3.2.6.1/75
303B_0023	Gate Register (RDC_SEMAPHORE1_GATE35)	8	R/W	00h	3.2.6.1/75
303B_0024	Gate Register (RDC_SEMAPHORE1_GATE36)	8	R/W	00h	3.2.6.1/75
303B_0025	Gate Register (RDC_SEMAPHORE1_GATE37)	8	R/W	00h	3.2.6.1/75
303B_0026	Gate Register (RDC_SEMAPHORE1_GATE38)	8	R/W	00h	3.2.6.1/75
303B_0027	Gate Register (RDC_SEMAPHORE1_GATE39)	8	R/W	00h	3.2.6.1/75
303B_0028	Gate Register (RDC_SEMAPHORE1_GATE40)	8	R/W	00h	3.2.6.1/75
303B_0029	Gate Register (RDC_SEMAPHORE1_GATE41)	8	R/W	00h	3.2.6.1/75
303B_002A	Gate Register (RDC_SEMAPHORE1_GATE42)	8	R/W	00h	3.2.6.1/75
303B_002B	Gate Register (RDC_SEMAPHORE1_GATE43)	8	R/W	00h	3.2.6.1/75
303B_002C	Gate Register (RDC_SEMAPHORE1_GATE44)	8	R/W	00h	3.2.6.1/75
303B_002D	Gate Register (RDC_SEMAPHORE1_GATE45)	8	R/W	00h	3.2.6.1/75
303B_002E	Gate Register (RDC_SEMAPHORE1_GATE46)	8	R/W	00h	3.2.6.1/75
303B_002F	Gate Register (RDC_SEMAPHORE1_GATE47)	8	R/W	00h	3.2.6.1/75
303B_0030	Gate Register (RDC_SEMAPHORE1_GATE48)	8	R/W	00h	3.2.6.1/75
303B_0031	Gate Register (RDC_SEMAPHORE1_GATE49)	8	R/W	00h	3.2.6.1/75
303B_0032	Gate Register (RDC_SEMAPHORE1_GATE50)	8	R/W	00h	3.2.6.1/75
303B_0033	Gate Register (RDC_SEMAPHORE1_GATE51)	8	R/W	00h	3.2.6.1/75
303B_0034	Gate Register (RDC_SEMAPHORE1_GATE52)	8	R/W	00h	3.2.6.1/75

Table continues on the next page...

RDC_SEMAPHORE memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303B_0035	Gate Register (RDC_SEMAPHORE1_GATE53)	8	R/W	00h	3.2.6.1/75
303B_0036	Gate Register (RDC_SEMAPHORE1_GATE54)	8	R/W	00h	3.2.6.1/75
303B_0037	Gate Register (RDC_SEMAPHORE1_GATE55)	8	R/W	00h	3.2.6.1/75
303B_0038	Gate Register (RDC_SEMAPHORE1_GATE56)	8	R/W	00h	3.2.6.1/75
303B_0039	Gate Register (RDC_SEMAPHORE1_GATE57)	8	R/W	00h	3.2.6.1/75
303B_003A	Gate Register (RDC_SEMAPHORE1_GATE58)	8	R/W	00h	3.2.6.1/75
303B_003B	Gate Register (RDC_SEMAPHORE1_GATE59)	8	R/W	00h	3.2.6.1/75
303B_003C	Gate Register (RDC_SEMAPHORE1_GATE60)	8	R/W	00h	3.2.6.1/75
303B_003D	Gate Register (RDC_SEMAPHORE1_GATE61)	8	R/W	00h	3.2.6.1/75
303B_003E	Gate Register (RDC_SEMAPHORE1_GATE62)	8	R/W	00h	3.2.6.1/75
303B_003F	Gate Register (RDC_SEMAPHORE1_GATE63)	8	R/W	00h	3.2.6.1/75
303B_0040	Reset Gate Write (RDC_SEMAPHORE1_RSTGT_W)	16	R/W	0000h	3.2.6.2/76
303B_0040	Reset Gate Read (RDC_SEMAPHORE1_RSTGT_R)	16	R/W	0000h	3.2.6.3/77
303C_0000	Gate Register (RDC_SEMAPHORE2_GATE0)	8	R/W	00h	3.2.6.1/75
303C_0001	Gate Register (RDC_SEMAPHORE2_GATE1)	8	R/W	00h	3.2.6.1/75
303C_0002	Gate Register (RDC_SEMAPHORE2_GATE2)	8	R/W	00h	3.2.6.1/75
303C_0003	Gate Register (RDC_SEMAPHORE2_GATE3)	8	R/W	00h	3.2.6.1/75
303C_0004	Gate Register (RDC_SEMAPHORE2_GATE4)	8	R/W	00h	3.2.6.1/75
303C_0005	Gate Register (RDC_SEMAPHORE2_GATE5)	8	R/W	00h	3.2.6.1/75
303C_0006	Gate Register (RDC_SEMAPHORE2_GATE6)	8	R/W	00h	3.2.6.1/75
303C_0007	Gate Register (RDC_SEMAPHORE2_GATE7)	8	R/W	00h	3.2.6.1/75
303C_0008	Gate Register (RDC_SEMAPHORE2_GATE8)	8	R/W	00h	3.2.6.1/75
303C_0009	Gate Register (RDC_SEMAPHORE2_GATE9)	8	R/W	00h	3.2.6.1/75
303C_000A	Gate Register (RDC_SEMAPHORE2_GATE10)	8	R/W	00h	3.2.6.1/75
303C_000B	Gate Register (RDC_SEMAPHORE2_GATE11)	8	R/W	00h	3.2.6.1/75
303C_000C	Gate Register (RDC_SEMAPHORE2_GATE12)	8	R/W	00h	3.2.6.1/75
303C_000D	Gate Register (RDC_SEMAPHORE2_GATE13)	8	R/W	00h	3.2.6.1/75
303C_000E	Gate Register (RDC_SEMAPHORE2_GATE14)	8	R/W	00h	3.2.6.1/75
303C_000F	Gate Register (RDC_SEMAPHORE2_GATE15)	8	R/W	00h	3.2.6.1/75
303C_0010	Gate Register (RDC_SEMAPHORE2_GATE16)	8	R/W	00h	3.2.6.1/75
303C_0011	Gate Register (RDC_SEMAPHORE2_GATE17)	8	R/W	00h	3.2.6.1/75
303C_0012	Gate Register (RDC_SEMAPHORE2_GATE18)	8	R/W	00h	3.2.6.1/75
303C_0013	Gate Register (RDC_SEMAPHORE2_GATE19)	8	R/W	00h	3.2.6.1/75
303C_0014	Gate Register (RDC_SEMAPHORE2_GATE20)	8	R/W	00h	3.2.6.1/75
303C_0015	Gate Register (RDC_SEMAPHORE2_GATE21)	8	R/W	00h	3.2.6.1/75
303C_0016	Gate Register (RDC_SEMAPHORE2_GATE22)	8	R/W	00h	3.2.6.1/75
303C_0017	Gate Register (RDC_SEMAPHORE2_GATE23)	8	R/W	00h	3.2.6.1/75
303C_0018	Gate Register (RDC_SEMAPHORE2_GATE24)	8	R/W	00h	3.2.6.1/75

Table continues on the next page...

RDC_SEMAPHORE memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303C_0019	Gate Register (RDC_SEMAPHORE2_GATE25)	8	R/W	00h	3.2.6.1/75
303C_001A	Gate Register (RDC_SEMAPHORE2_GATE26)	8	R/W	00h	3.2.6.1/75
303C_001B	Gate Register (RDC_SEMAPHORE2_GATE27)	8	R/W	00h	3.2.6.1/75
303C_001C	Gate Register (RDC_SEMAPHORE2_GATE28)	8	R/W	00h	3.2.6.1/75
303C_001D	Gate Register (RDC_SEMAPHORE2_GATE29)	8	R/W	00h	3.2.6.1/75
303C_001E	Gate Register (RDC_SEMAPHORE2_GATE30)	8	R/W	00h	3.2.6.1/75
303C_001F	Gate Register (RDC_SEMAPHORE2_GATE31)	8	R/W	00h	3.2.6.1/75
303C_0020	Gate Register (RDC_SEMAPHORE2_GATE32)	8	R/W	00h	3.2.6.1/75
303C_0021	Gate Register (RDC_SEMAPHORE2_GATE33)	8	R/W	00h	3.2.6.1/75
303C_0022	Gate Register (RDC_SEMAPHORE2_GATE34)	8	R/W	00h	3.2.6.1/75
303C_0023	Gate Register (RDC_SEMAPHORE2_GATE35)	8	R/W	00h	3.2.6.1/75
303C_0024	Gate Register (RDC_SEMAPHORE2_GATE36)	8	R/W	00h	3.2.6.1/75
303C_0025	Gate Register (RDC_SEMAPHORE2_GATE37)	8	R/W	00h	3.2.6.1/75
303C_0026	Gate Register (RDC_SEMAPHORE2_GATE38)	8	R/W	00h	3.2.6.1/75
303C_0027	Gate Register (RDC_SEMAPHORE2_GATE39)	8	R/W	00h	3.2.6.1/75
303C_0028	Gate Register (RDC_SEMAPHORE2_GATE40)	8	R/W	00h	3.2.6.1/75
303C_0029	Gate Register (RDC_SEMAPHORE2_GATE41)	8	R/W	00h	3.2.6.1/75
303C_002A	Gate Register (RDC_SEMAPHORE2_GATE42)	8	R/W	00h	3.2.6.1/75
303C_002B	Gate Register (RDC_SEMAPHORE2_GATE43)	8	R/W	00h	3.2.6.1/75
303C_002C	Gate Register (RDC_SEMAPHORE2_GATE44)	8	R/W	00h	3.2.6.1/75
303C_002D	Gate Register (RDC_SEMAPHORE2_GATE45)	8	R/W	00h	3.2.6.1/75
303C_002E	Gate Register (RDC_SEMAPHORE2_GATE46)	8	R/W	00h	3.2.6.1/75
303C_002F	Gate Register (RDC_SEMAPHORE2_GATE47)	8	R/W	00h	3.2.6.1/75
303C_0030	Gate Register (RDC_SEMAPHORE2_GATE48)	8	R/W	00h	3.2.6.1/75
303C_0031	Gate Register (RDC_SEMAPHORE2_GATE49)	8	R/W	00h	3.2.6.1/75
303C_0032	Gate Register (RDC_SEMAPHORE2_GATE50)	8	R/W	00h	3.2.6.1/75
303C_0033	Gate Register (RDC_SEMAPHORE2_GATE51)	8	R/W	00h	3.2.6.1/75
303C_0034	Gate Register (RDC_SEMAPHORE2_GATE52)	8	R/W	00h	3.2.6.1/75
303C_0035	Gate Register (RDC_SEMAPHORE2_GATE53)	8	R/W	00h	3.2.6.1/75
303C_0036	Gate Register (RDC_SEMAPHORE2_GATE54)	8	R/W	00h	3.2.6.1/75
303C_0037	Gate Register (RDC_SEMAPHORE2_GATE55)	8	R/W	00h	3.2.6.1/75
303C_0038	Gate Register (RDC_SEMAPHORE2_GATE56)	8	R/W	00h	3.2.6.1/75
303C_0039	Gate Register (RDC_SEMAPHORE2_GATE57)	8	R/W	00h	3.2.6.1/75
303C_003A	Gate Register (RDC_SEMAPHORE2_GATE58)	8	R/W	00h	3.2.6.1/75
303C_003B	Gate Register (RDC_SEMAPHORE2_GATE59)	8	R/W	00h	3.2.6.1/75
303C_003C	Gate Register (RDC_SEMAPHORE2_GATE60)	8	R/W	00h	3.2.6.1/75
303C_003D	Gate Register (RDC_SEMAPHORE2_GATE61)	8	R/W	00h	3.2.6.1/75
303C_003E	Gate Register (RDC_SEMAPHORE2_GATE62)	8	R/W	00h	3.2.6.1/75

Table continues on the next page...

RDC_SEMAPHORE memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
303C_003F	Gate Register (RDC_SEMAPHORE2_GATE63)	8	R/W	00h	3.2.6.1/75
303C_0040	Reset Gate Write (RDC_SEMAPHORE2_RSTGT_W)	16	R/W	0000h	3.2.6.2/76
303C_0040	Reset Gate Read (RDC_SEMAPHORE2_RSTGT_R)	16	R/W	0000h	3.2.6.3/77

3.2.6.1 Gate Register (RDC_SEMAPHOREx_GATE_n)

Each semaphore gate is implemented in a 4-bit finite state machine, right-justified in a byte data structure. The hardware uses the logical bus master number (master_index) in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. Attempted writes with a data value that is neither the unlock value nor the appropriate lock value (master_index + 1) are simply treated as "no operation" and do not affect any gate state. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes. Processor dex values can be found in [AIPSTZ Memory Map/Register Definition](#).

Address: Base address + 0h offset + (1d × i), where i=0d to 63d

Bit	7	6	5	4	3	2	1	0
Read	0		LDOM		GTFSM			
Write								
Reset	0	0	0	0	0	0	0	0

RDC_SEMAPHOREx_GATE_n field descriptions

Field	Description
7–6 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
5–4 LDOM	Read-only bits. They indicate which domain had currently locked the gate. 00 The gate is locked by domain 0. (True if bits [3:0] do not equal 0000.) 01 The gate has been locked by domain 1. 10 The gate has been locked by domain 2. 11 The gate has been locked by domain 3.
GTFSM	Gate Finite State Machine. The state of the gate reflects the last processor that locked it, which can be useful during system debug.

Table continues on the next page...

RDC_SEMAPHOREx_GATE_n field descriptions (continued)

Field	Description
	The hardware gate is maintained in a 16-state implementation, defined as:
0000	The gate is unlocked (free).
0001	The gate has been locked by processor with master_index = 0.
0010	The gate has been locked by processor with master_index = 1.
0011	The gate has been locked by processor with master_index = 2.
0100	The gate has been locked by processor with master_index = 3.
0101	The gate has been locked by processor with master_index = 4.
0110	The gate has been locked by processor with master_index = 5.
0111	The gate has been locked by processor with master_index = 6.
1000	The gate has been locked by processor with master_index = 7.
1001	The gate has been locked by processor with master_index = 8.
1010	The gate has been locked by processor with master_index = 9.
1011	The gate has been locked by processor with master_index = 10.
1100	The gate has been locked by processor with master_index = 11.
1101	The gate has been locked by processor with master_index = 12.
1110	The gate has been locked by processor with master_index = 13.
1111	The gate has been locked by processor with master_index = 14.

3.2.6.2 Reset Gate Write (RDC_SEMAPHOREx_RSTGT_W)

Although the intent of the hardware gate implementation specifies a protocol where the locking processor must unlock the gate, it is recognized that system operation may require a reset function to re-initialize the state of any gate(s) without requiring a system-level reset.

To support this special gate reset requirement, the RDC Semaphores module implements a "secure" reset mechanism that allows a hardware gate (or all the gates) to be initialized by following a specific dual-write access pattern. Using a technique similar to that required for the servicing of a software watchdog timer, the secure gate reset requires two consecutive writes with predefined data patterns from the same processor to force the clearing of the specified gate(s). The required access pattern is:

1. A processor performs a 16-bit write to the RDC_SEMA42RSTGT memory location. The least significant byte (RDC_SEMA42RSTGT[RSTGDP]) must be 0xE2; the most significant byte is a "don't_care" for this reference.
2. The same processor then performs a second 16-bit write to the RDC_SEMA42RSTGT location. For this write, the lower byte (RDC_SEMA42RSTGT[RSTGDP]) is the logical complement of the first data pattern (0x1D) and the upper byte (RDC_SEMA42RSTGT[RSTGTN]) specifies the gate(s) to be reset. This gate field can specify a single gate be cleared, or else that all gates are to be cleared. If the same processor writes incorrect data on the second

access or another processor performs the second write access, the special gate reset sequence is aborted and no error signal will be asserted.

3. Reads of the RDC_SEMA42RSTGT location return information on the 2-bit state machine (RDC_SEMA42RSTGT[RSTGSM]) that implements this function, the bus master performing the reset (RDC_SEMA42RSTGT[RSTGMS]), and the gate number(s) last cleared (RDC_SEMA42RSTGT[RSTGTN]). Reads of the RDC_SEMA42RSTGT register do not affect the secure reset finite state machine in any manner.

Address: Base address + 40h offset

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read	RSTGTN								0							
Write									RSTGDP							
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

RDC_SEMAPHOREx_RSTGT_W field descriptions

Field	Description
15–8 RSTGTN	Reset Gate Number. This 8-bit field specifies the specific hardware gate to be reset. This field is updated by the second write. If RSTGTN < 64, then reset the single gate defined by RSTGTN, else reset all the gates.
RSTGDP	Reset Gate Data Pattern. This write-only field is accessed with the specified data patterns on the two consecutive writes to enable the gate reset mechanism. For the first write, RSTGDP = 0xE2 while the second write requires RSTGDP = 0x1D.

3.2.6.3 Reset Gate Read (RDC_SEMAPHOREx_RSTGT_R)

Although the intent of the hardware gate implementation specifies a protocol where the locking processor must unlock the gate, it is recognized that system operation may require a reset function to re-initialize the state of any gate(s) without requiring a system-level reset.

To support this special gate reset requirement, the RDC Semaphores module implements a "secure" reset mechanism that allows a hardware gate (or all the gates) to be initialized by following a specific dual-write access pattern. Using a technique similar to that required for the servicing of a software watchdog timer, the secure gate reset requires two consecutive writes with predefined data patterns from the same processor to force the clearing of the specified gate(s). The required access pattern is:

1. A processor performs a 16-bit write to the RDC_SEMA42RSTGT memory location. The least significant byte (RDC_SEMA42RSTGT[RSTGDP]) must be 0xE2; the most significant byte is a "don't_care" for this reference.
2. The same processor then performs a second 16-bit write to the RDC_SEMA42RSTGT location. For this write, the lower byte

(RDC_SEMA42RSTGT[RSTGDP]) is the logical complement of the first data pattern (0x1D) and the upper byte (RDC_SEMA42RSTGT[RSTGTN]) specifies the gate(s) to be reset. This gate field can specify a single gate be cleared, or else that all gates are to be cleared. If the same processor writes incorrect data on the second access or another processor performs the second write access, the special gate reset sequence is aborted and no error signal will be asserted.

- Reads of the RDC_SEMA42RSTGT location return information on the 2-bit state machine (RDC_SEMA42RSTGT[RSTGSM]) that implements this function, the bus master performing the reset (RDC_SEMA42RSTGT[RSTGMS]), and the gate number(s) last cleared (RDC_SEMA42RSTGT[RSTGTN]). Reads of the RDC_SEMA42RSTGT register do not affect the secure reset finite state machine in any manner.

Address: Base address + 40h offset

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read	RSTGTN								0		RSTGSM		RSTGMS			
Write																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

RDC_SEMAPHOREx_RSTGT_R field descriptions

Field	Description
15–8 RSTGTN	Reset Gate Number. This 8-bit field specifies the specific hardware gate to be reset. This field is updated by the second write. If RSTGTN < 64, then reset the single gate defined by RSTGTN, else reset all the gates.
7–6 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
5–4 RSTGSM	Reset Gate Finite State Machine. Reads of the RDC_SEMA42RSTGT register return the encoded state machine value. Note the RSTGSM = 10 state is valid for only a single machine cycle, so it is impossible for a read to return this value. The reset state machine is maintained in a 2-bit, 3-state implementation, defined as: 00 Idle, waiting for the first data pattern write. 01 Waiting for the second data pattern write. 10 The 2-write sequence has completed. Generate the specified gate reset(s). After the reset is performed, this machine returns to the idle (waiting for first data pattern write) state. The "01" state persists for only one clock cycle. Software will never be able to observe this state. 11 This state encoding is never used and therefore reserved.
RSTGMS	Reset Gate Bus Master. This 4-bit read-only field records the logical number of the bus master performing the gate reset function. The reset function requires that the two consecutive writes to this register must be initiated by the same bus master to succeed. This field is updated each time a write to this register occurs. The association between system bus master port numbers, the associated bus master device, and the logical processor number is SoC-specific. Consult the device reference manual for this information.

Chapter 4

ARM Platform and Debug

4.1 ARM Cortex A53 Platform (A53)

4.1.1 Overview

The Cortex-A53 cluster is a mid-range, low-power processor that implements the ARMv8-A architecture. The Cortex-A53 cluster has four cores, each with an L1 memory system and a single shared L2 cache that has a set of additional functions, which are included in a single APR region.

The core supports debug through real-time trace via ETM, and static debug via JTAG. The core platform supports static debug through the debug logic to SoC. This includes the capability of real-time trace via ARM's CoreSight ETM modules. The CTI and CTM modules allow cross-triggering of internal and external trigger sources.

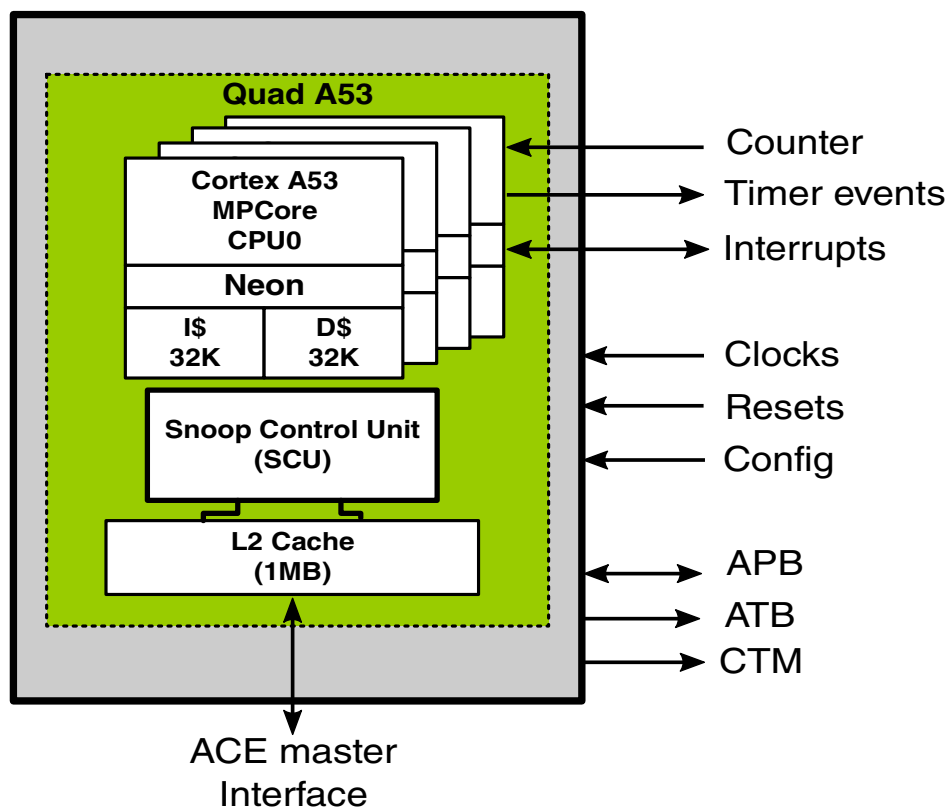


Figure 4-1. Cortex-A53 Block Diagram

4.1.1.1 Features

- 4x cores
- L1 instruction cache 32K with parity
- L1 data cache 32K with SECDED
- Advanced SIMD (NEON) per core
- Crypto extension per core
- CPU cache protection per core
- AMBA 4 ACE interface
- No ACP is included
- utilizes the ARMv8 debug map
- 1MB of L2 Cache
 - 1M with SECDED
 - Input latency 2 cycles
 - Output latency 2 cycles
 - SCU-L2 cache protection

4.1.2 Configuration

The configuration of the bus, cores and memory are detailed in the features and sections below. The core revision is r0p4-51rel0.

4.1.3 Performance

This section will discuss power and clocking performance of the Cortex-A53 Platform.

4.1.3.1 Power

There are several power states supported by the A53 Core Complex. Supported primary states are listed below:

- **Run** – At least one of 4 cores is running. The other cores may either be running, clock gated, or powered down.
- **L2 only coherent** – The L2 is powered up and servicing snoops. The cores are stopped (either powered down or clock gated). In this state the cache is retained coherent to the system.
- **L2 only non-coherent** – Both cores are stopped and powered down, the logic in the L2 controller are retaining the arrays only. In this state the L2 is not coherent, and as a result, the other AP cores must also be stopped.
- **Cluster Off** – The L2 has been flushed from the L2 only coherent state, using the HW flush mechanism (no core required). Then the cluster is fully powered off including the L2 arrays.

The power supply for the cluster is separated into two regions. The first is the control domain and second is for the remainder of the cluster (core and cluster domains). These regions are listed below:

- **Control domain** – The AON control domain contains the controls for powering up and down the rest of the core. The control domain is always powered on first and powered off last.
- **Core domains** – The core domains contains the whole core. The supply for the core domains is the same as the cluster domain, but require separate power down switches.
- **Cluster domain** – The cluster domain contains the rest of the cluster outside of the cores, which includes the shared logic of the L2 memory. The supply for the cluster power domain is the same as the core domains, but does not require power switches as it's shut down externally.

Isolation cells are required between each of these domains and is controlled by the signals on the control domain. These cells are necessary to force the output signal to be isolated to null values when the local power shuts down.

4.1.3.2 Clocking

The A53 platform is provided a main processor clock that supplies the component clocks to the cluster components, including CoreSight debug components. The maximum frequency targets are specified in the chip datasheet.

The cores are intended to support up to 1.5 GHz dependent on forward bias within the operating temperature range. Please see the datasheet for more information. The clocks are described in the table below:

Table 4-1. A53 Clocks

Clock Signal	Clock Name	Frequency	Description
Main Clock	CLKIN	Target	Main input clock.
APB Clock	PCLKENDBG	CLKIN/4	APB clock controls the timing on the debug port. Fixed frequency ratio to main frequency.
ACE Bus Clock	ACLKENM	CLKIN/2	ACE Interface bus clock. Controls the timing on the interface to CCI, but is not synchronous to CCI. Frequency is fixed ratio.
ATB Clock	ATCLKEN	CLKIN/4	ATB clock provides the clocks or outgoing trace. This clock needs to be reasonably high to enable sending samples out, but also needs to be the same as CNTCLKEN. Link to same signal as CNTCLKEN (1:4 core frequency fixed ratio).
Input Counter Clock	CNTCLKEN	CLKIN/4	Input counter clock. Timing is identical to debug port. The frequency is a fixed ratio (1:4) with the core clock. The same signal may be used for both inputs.

4.1.4 Platform sub-blocks

The sections below discuss the high-level overview of the ARM® Cortex®-A53 Platform components.

4.1.4.1 ARM Cortex-A53 MPCore Processor

The information presented in this section focuses on the design aspects of the ARM Cortex-A53 MPCore in the AP subsystem. The Cortex-A53 processor is a mid-range, low-power processor that implements the ARMv8-A architecture. The A53 complex has four cores, each with an L1 memory system and a single shared L2 cache.

The ARM Cortex-A53 MPCore processor consists of:

- Tightly-coupled L2 cache and an integrated Snoop Control Unit (SCU), connecting the four cores within the cluster providing cluster memory coherency, and a configurable coherent external interface supporting AMBA4 bus architecture.
- The Cortex-A53 implements the ARM Generic Interrupt Controller v3 (GICv3) architecture.

4.1.4.2 Advanced SIMD (Neon)

The Cortex-A53 processor supports the Advanced SIMD and Scalar Floating-point instructions in the A64 instruction set, and the Advanced SIMD and VFP instructions in the A32 and T32 instruction sets.

The ARMv8 architecture eliminates the concept of version numbers for Advanced SIMD and Floating-point in the AArch64 execution state.

4.1.4.3 Generic Interrupt Controller (GIC)

The Generic Interrupt Controller (GIC) supports and manages interrupts in the cluster. The GIC provides registers for managing:

- Interrupt sources
- Interrupt behavior
- Interrupt routing to one or more cores

The Cortex-A53 processor implements the GIC CPU interface as described in the Generic Interrupt Controller (GICv4) architecture. This interfaces with an external GICv3 or GICv4 interrupt distributor component within the system. The GICv4 architecture supports:

- Two security states
- Interrupt virtualization
- Software-generated Interrupts (SGIs)
- Message Based Interrupts
- System register access
- Memory-mapped register access
- Interrupt masking and prioritization

- Cluster environments
- Wake-up events in power management environments

The GIC includes interrupt grouping functionality that supports:

- Signaling interrupt groups to the target core using either the IRQ or the FIQ exception request, based on software configuration
- A unified scheme for handling the priority of Group 0 and Group 1 interrupts.

4.1.4.4 L1 Cache

The L1 instruction memory system has the following features:

- 32KB Instruction Cache
 - Instruction side cache line length of 64 bytes
 - 2-way set associative L1 Instruction cache
 - 128-bit read interface to the L2 memory system
- 32KB Data Cache
 - Data side cache line length of 64 bytes
 - 4-way set associative L1 Data cache
 - 256-bit write interface to the L2 memory system
 - 128-bit read interface to the L2 memory system
 - Read buffer that services the Data Cache Unit (DCU), the Instruction Fetch Unit (IFU) and the TLB
 - 64-bit read path from the data L1 memory system to the datapath
 - 128-bit write path from the datapath to the L1 memory system
 - Support for three outstanding data cache misses
 - Merging store buffer capability. This handles writes to:
 - Device memory
 - Normal Cacheable memory
 - Normal Non-cacheable memory
 - Data side prefetch engine

4.1.4.5 L2 Cache

The L2 cache consists of an integrated Snoop Control Unit (SCU), connecting four cores within the A53 cluster. The SCU also has duplicate copies of the L1 Data cache tags for coherency support. The L2 memory system interfaces to the external memory system through an AMBA 128-bit bus. The tightly-coupled L2 cache includes the following features:

- 1MB shared cache size
- AMBA 4 ACE Interface
- Fixed line length of 64 bytes

- Physically indexed and tagged cache
- 16-way set-associative cache structure
- No ACP support
- ECC/parity support

4.2 ARM Cortex M4 Platform (M4)

4.2.1 Overview

This block details the ARM Cortex-M4 core and platform. The Cortex-M4 implements the ARMv7- ME instruction set architecture (ISA) and adds significant capabilities with DSP and SIMD extensions. The ARM Cortex-M4 core provides additional general processing capability to the SoC with lower power and fast interrupt response time.

The Cortex-M4 also includes a single-precision floating-point unit (FPU) and two 32-bit system bus interfaces. The Cortex-M4 implementation includes two tightly coupled local memories, two cache memories connected to bus interfaces, 64-bit system bus interconnect, and supports a 32-byte cache line size.

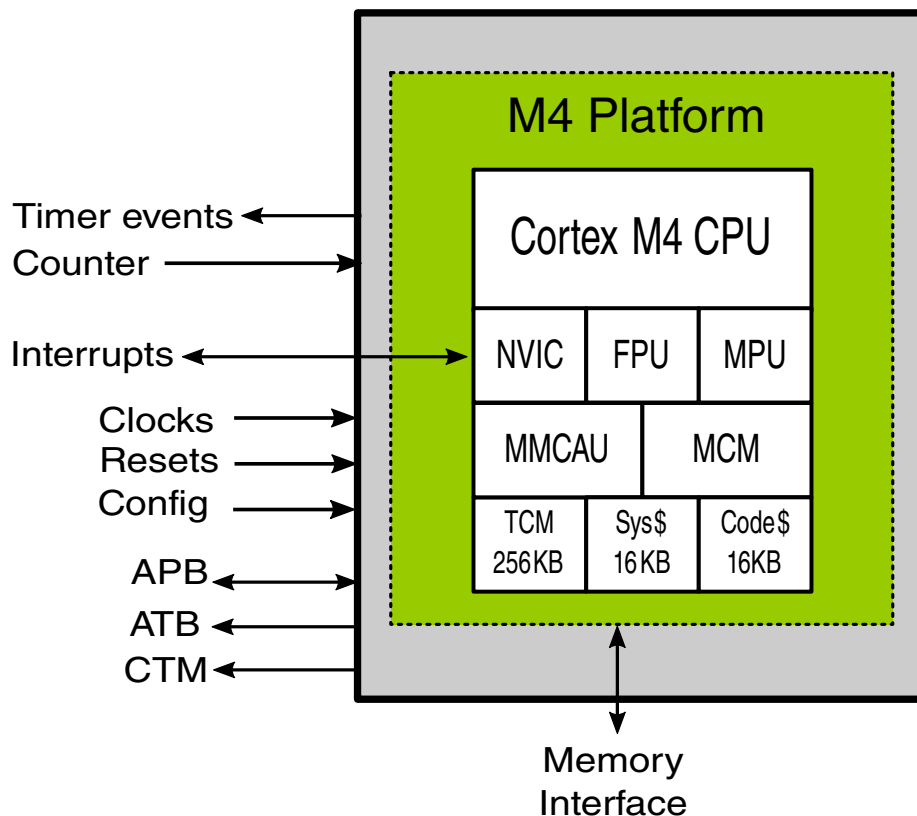


Figure 4-2. Cortex-M4 Block Diagram

4.2.1.1 Features

The features of the Cortex-M4 platform are detailed below:

- 1x Cortex-M4 processor
- AHB LMEM (Local Memory Controller) including controllers for TCM and cache memories
- 256 KB TCM (128 KB TCMU, 128 KB TCML)
- 16 KB Code Bus Cache
- 16 KB System Bus Cache
- ECC for TCM memories and parity for code and system caches
- Integrated Nested Vector Interrupt Controller (NVIC)
- Wakeup Interrupt Controller (WIC)
- FPU (Floating Point Unit)
- Core MPU (Memory Protection Unit)
- Support for exclusive access on the system bus
- MMCAU (Crypto Acceleration Unit)
- MCM (Miscellaneous Control Module)

4.2.2 Configuration

The configuration of the bus, core, and memory are detailed in the features and sections below.

Table 4-2. Cortex-M4 Configuration

Parameter	Description	Setting
NUM_IRQ	Number of Interrupts	128
LVL_WIDTH (translates to 2 ⁿ levels)	Number of interrupt priority bits	4
MPU_PRESENT	Exclude (0) or include (1) optional ARM MPU	1
DEBUG_LVL	Level of debug support	3
TRACE_LVL	Level of trace support 0 = No Trace 1 = ITM and DWT 2 = ITM, DWT, and ETM 3 = ITM, DWT, ETM, and HTM	2
RESET_ALL_REGS	Select whether all registers are reset (1)	1
JTAG_PRESENT	SWJ-DP (1) SW-DP (0)	0
CLKGATE_PRESENT	Disable (0) or enable (1) instantiation of architectural clock gates at all levels	1

Table continues on the next page...

Table 4-2. Cortex-M4 Configuration (continued)

WIC_PRESENT	Exclude (0) or include (1) optional ARM provided WIC	1
WIC_LINES	Select number of interrupts and/or events that WIC is sensitive to	NUM_IRQ+3
FPU_PRESENT	Exclude (0) or include (1) Floating Point Unit (FPU)	1
BB_PRESENT	Exclude (0) or include (1) Bit Banding logic	0
CONST_AHB_CTRL	Specifies whether the external AHB-Lite buses maintain control information during wait stated transfers	0

4.2.3 Performance

This section will discuss power and clocking performance of the Cortex-M4 Platform.

4.2.3.1 Power

There are several power states supported by the Cortex-M4. Supported primary core states are listed below:

- **Run** - Normal run state. CM4 Platform, TCM and cache memories on nominal voltage.
- **Sleep** – Wait power state. Depending on power state, CM4 Platform, TCM and cache memories can either be in nominal or low-voltage. State recovery through NVIC. NVIC clock is still running in this state.
- **Deep Sleep** – Stop power state. Depending on power state, CM4 Platform, TCM and cache memories can either be in nominal or low-voltage. State recovery through AWIC. Clocks can be completely stopped in this state.

4.2.3.2 Clocking

The M4 platform is provided a main processor clock that supplies the component clocks to the cluster components. The maximum frequency targets are specified in the chip datasheet. Please see the datasheet for more information. The clocks are described in the table below:

Table 4-3. CM4 Clocks

Clock Signal	Clock Name	Target Frequency	Description
Core gated clock	cpu_hclk	266 MHz	Cortex-M4 core clock.
Core Free-Running Clock	cpu_fclk	266 MHz	CM4 NVIC and timer clock

Table continues on the next page...

Table 4-3. CM4 Clocks (continued)

TCM Controller Clock	tcmc_hclk	266 MHz	CM4 platform TCM controller clock
Platform Clock	hclk	266 MHz	CM4 platform AXBS fabric and bus masters. Synchronous to the core clock.
Bus Clock	ipg_clk	133 MHz	Clock for bus slaves and peripherals. Synchronous to the system clock.

4.2.4 Platform sub-blocks

The sections below discuss the high-level overview of the ARM® Cortex®-M4 Platform components.

4.2.4.1 ARM Cortex-M4 Processor

The Cortex-M4 processor is a low-power processor that features low gate count, low interrupt latency, and low-cost debug. The Cortex-M4 includes floating point arithmetic functionality. The processor is intended for deeply embedded applications that require fast interrupt response features.

The ARM Cortex-M4 processor consists of:

- A processor core
- A Nested Vectored Interrupt Controller (NVIC) closely integrated with the processor core to achieve low-latency interrupt processing
- Multiple high-performance bus interfaces
- A low-cost debug solution with the optional ability to:
 - Implement breakpoints and code patches
 - Implement watchpoints, tracing, and system profiling
 - Support printf() style debugging
 - Bridge to a Trace Port Analyzer (TPA)
- Memory Protection Unit (MPU)
- Floating Point Unit (FPU)
- Miscellaneous Control Module (MCM)
- The Cortex-M4 implements the ARM the ARMv7-M architecture.

4.2.4.2 Nested Vectored Interrupt Controller (NVIC)

The M4 platform includes a Nested Vectored Interrupt Controller (NVIC) that is closely integrated with the processor core to achieve low-latency interrupt processing. The platform supports 16 priority levels for interrupts. The NVIC IPR registers will define 4 bits per IRQ.

4.2.4.3 Floating Point Unit (FPU)

The Cortex-M4 platform supports Floating Point Unit (FPU). The FPU implements the single precision variant of the ARMv7-M Floating-Point Extension (FPv4-SP), which supports single-precision add, subtract, multiply, divide, multiply and accumulate, and square root operations. FPU also provides conversions between fixed-point and floating-point data formats, and floating-point constant instructions.

The FPU provides floating-point computation functionality that is compliant with the ANSI/IEEE Std 754-2008, IEEE Standard for Binary Floating-Point Arithmetic, referred to as the IEEE 754 standard. The FPU contains 32 single-precision extension registers, which you can also access as 16 doubleword registers for load, store, and move operations.

4.2.4.4 Memory Protection Unit (MPU)

The Cortex-M4 platform supports Memory Protection Unit (MPU). The MPU enforces privilege rules, separates processes, and enforces access rules to memory, and supports the standard ARMv7 Protected Memory System Architecture model. The MPU provides full support for:

- Protection regions
- Overlapping protection regions, with ascending region priority:
 - 7 = highest priority
 - 0 = lowest priority
- Access permissions
- Exporting memory attributes to the system

MPU mismatches and permission violations invoke the programmable-priority MemManage fault handler. You can use the MPU to:

- Enforce privilege rules
- Separate processes
- Enforce access rules

4.2.4.5 Tightly-Coupled Memory (TCM)

The Cortex-M4 uses Tightly-Coupled Memory (TCM) to perform low-latency memory operations including speculative read accesses. The TCM is split between upper and lower regions with each TCM interface operating independently of each other.

The Cortex-M4 supports 32-bit ECC error detection and correction for the TCM memories. TCM ECC can indicate to the processor that an access must be retried to return the corrected data.

Because AHB-Lite does not support write data strobes when accessing AHB-Lite slaves from an AXI master, care must be taken not to generate transactions that have partial strobes. Make sure to not have unaligned accessing to TCM from an AXI master. For example, when writing data to TCM from A53, ensure every write strobe address is 64bit aligned. When the MMU is enabled, the TCM memory range must have the MT_DEVICE_NGSRNE type attribute set. This will avoid A53 sparse writes to the TCM memory region.

4.2.5 Local Memory Controller (LMEM)

The Local Memory Controller provides the Arm[®]Cortex-M4[™] processor with tightly-coupled processor-local memories and bus paths to all slave memory spaces.

4.2.5.1 LMEM Block Diagram

The Cortex-M4 processor has a modified 32-bit Harvard bus architecture. Using a 32-bit address space, low-order addresses (0x0000_0000 through 0x1FFF_FFFF) use the Processor Code (PC) bus, and high-order addresses (0x2000_0000 through 0xFFFF_FFFF) use the Processor System (PS) bus. As the bus names imply, normal operation has code accesses on the PC bus and data accesses on the PS bus.

This device has been augmented with tightly-coupled memories for the PC and PS buses. The memories include RAMs and caches. These local memories provide zero wait state access to RAM and cacheable address spaces.

The local memory controller includes four memory controllers and their attached memories:

- SRAM lower (SRAM_L) controller via the PC bus
- SRAM upper (SRAM_U) controller via the PS bus
- Cache memory controller via the PC bus
- Cache memory controller via the PS bus

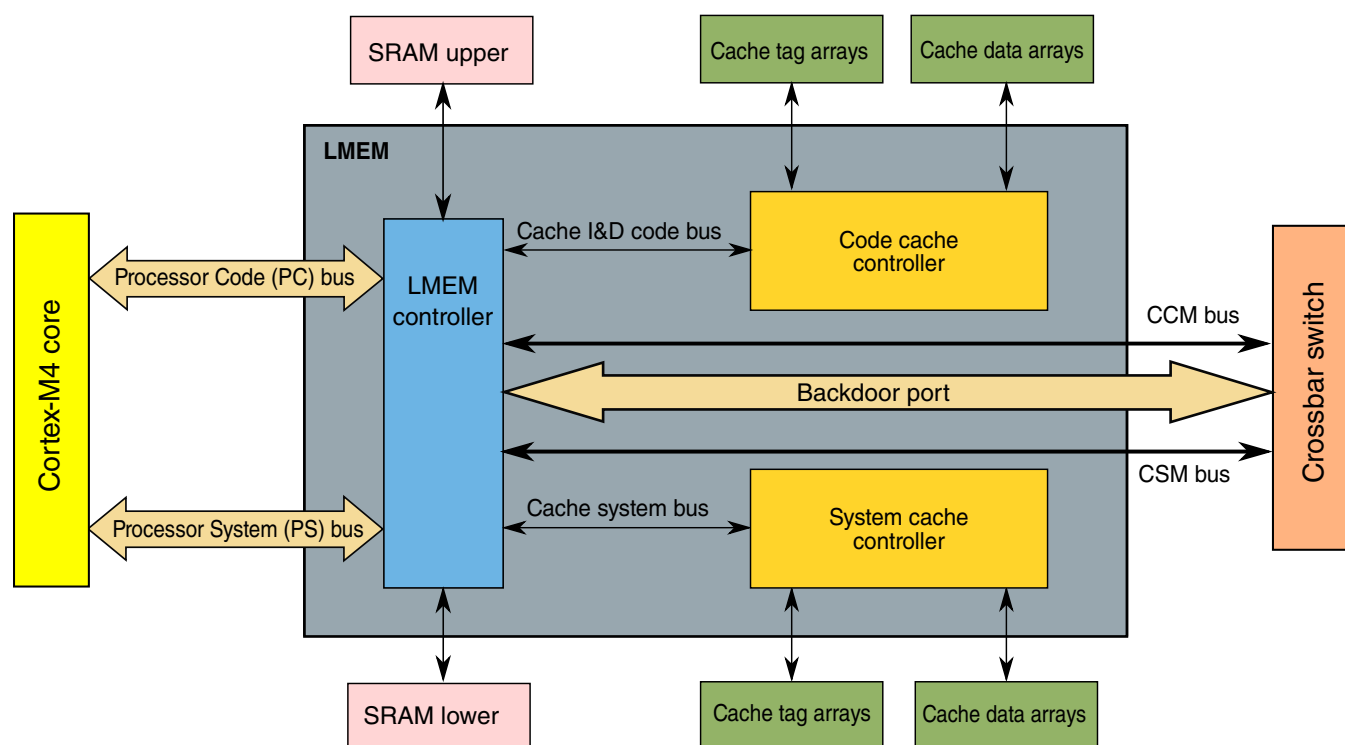


Figure 4-3. Local memory controller block diagram

NOTE

The SRAM and cache controllers reside within the LMEM, but the single-port synchronous RAM arrays used by these controllers are external.

The LMEM contains address decode logic for the PC and PS buses. This logic routes the core's accesses to the various system resources. The address spaces are device-specific and are specified in the device's Chip Configuration chapter.

4.2.5.2 Cache features

A cache is a block of high-speed memory locations containing address information (commonly known as a tag) and the associated data. The purpose is to decrease the average time of a memory access. Caches operate on two principles of locality:

- Spatial locality — An access to one location is likely to be followed by accesses from adjacent locations (for example, sequential instruction execution or usage of a data structure).
- Temporal locality — An access to an area of memory is likely to be repeated within a short time period (for example, execution of a code loop).

To minimize the quantity of control information stored, the spatial locality property is used to group several locations together under the same tag. This logical block is commonly known as a cache line.

When data is loaded into a cache, access times for subsequent loads and stores are reduced, resulting in overall performance benefits. An access to information already in a cache is known as a cache hit, and other accesses are called cache misses.

Normally, caches are self-managing, with the updates occurring automatically. Whenever the processor wants to access a cacheable location, the cache is checked. If the access is a cache hit, the access occurs immediately. Otherwise, a location is allocated and the cache line is loaded from memory. Different cache topologies and access policies are possible. However, they must comply with the memory coherency model of the underlying architecture.

Caches introduce a number of potential problems, mainly because of:

- memory accesses occurring at times other than when the programmer would normally expect them,
- the existence of multiple physical locations where a data item can be held.

The local memory controller supports three modes of operation:

1. Write-through — access to address spaces with this cache mode are cacheable.
 - A read miss on the input bus causes a line read on the output bus of a 32-byte-aligned memory address containing the desired address. This miss data is loaded into the cache and is marked as valid and not modified.
 - A write-through read hit to a valid cache location returns data from the cache with no output bus access.
 - A write-through write miss bypasses the cache and writes to the output bus (no allocate on write miss policy for write-through mode spaces).
 - A write-through write hit updates the cache hit data and writes to the output bus.
2. Write-back — access to address spaces with this cache mode are cacheable.
 - A write-back read miss on the input bus will cause a line read on the output bus of a 32-byte-aligned memory address containing the desired address. This miss data is loaded into the cache and marked as valid and not modified.
 - A write-back read hit to a valid cache location will return data from the cache with no output bus access.
 - A write-back write miss will do a "read-to-write" (allocate on write miss policy for write-back mode spaces). A line read on the output bus of a 16 byte aligned memory address containing the desired write address is performed. This miss data is loaded into the cache and marked as valid and modified; and the write data will then update the appropriate cache data locations.

3. Non-cacheable — access to address spaces with this cache mode are not cacheable. These accesses bypass the cache and access the output bus.

4.2.5.3 LMEM Function

The LMEM receives the following requests:

- Core master bus requests on the Processor Code (PC) bus,
- Core master bus requests on the Processor Space (PS) bus, and
- SRAM controller requests from all other bus masters on the backdoor port.

The LMEM address decode logic routes these accesses and also provides any crossbar switch slave target logic. Finally, the Local Memory controller provides the needed MPU connections for checking all SRAM controller and cacheable accesses.

The programming model for the Code and System Caches is accessed via the core's Private Peripheral Bus (PPB).

4.2.5.3.1 Processor Code accesses

Processor Code accesses are routed to the SRAM_L if they are mapped to that space. All other PC accesses are routed to the Code Cache Memory Controller. This controller then processes the cacheable accesses as needed, while bypassing the non-cacheable, cache write-through, cache miss, and cache maintenance accesses to the CCM bus and the crossbar switch using the Master0 port.

4.2.5.3.2 Processor Space accesses

Processor Space accesses are routed to the SRAM_U if they are mapped to that space. All other PS accesses are routed to the PS Cache Memory Controller. This controller then processes the cacheable accesses as needed, while bypassing the non-cacheable, cache write-through, cache miss, and cache maintenance accesses to the CCM bus and the crossbar switch using the Master1 port.

4.2.5.3.3 Backdoor port accesses

All LMEM backdoor port accesses are for the SRAM controller. These accesses go to the SRAM_L or the SRAM_U depending on their specific address.

4.2.5.3.4 SRAM Function

4.2.5.3.4.1 SRAM Configuration

The figure below shows how the SRAM controller is configured.

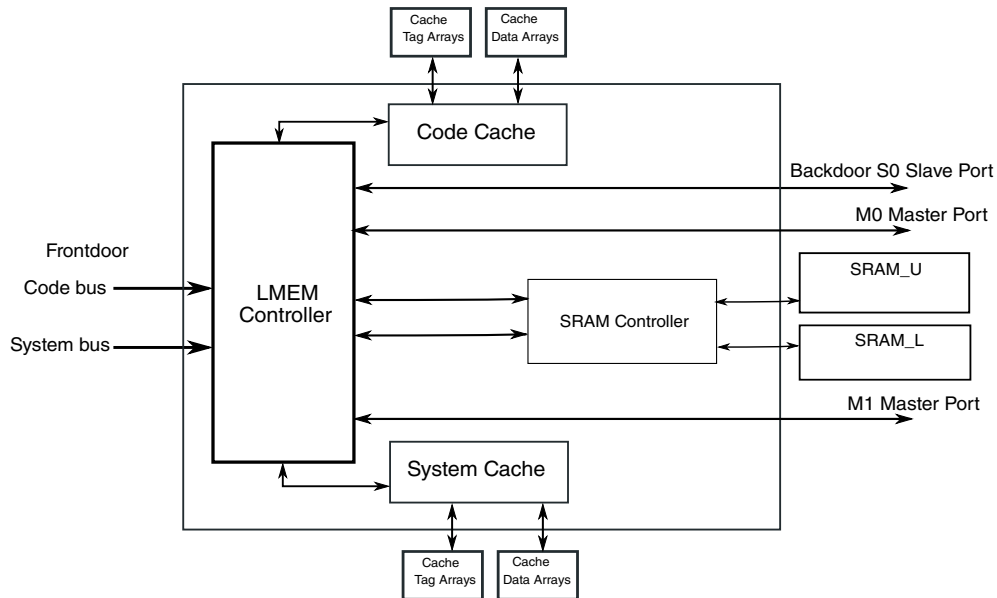


Figure 4-4. SRAM Configuration

4.2.5.3.4.2 SRAM Arrays

The on-chip SRAM is split into two logical arrays, SRAM_L and SRAM_U.

From equal-sized memories, valid address ranges for SRAM_L and SRAM_U are then defined as:

- $\text{SRAM_L} = 0x1FFE_0000 - (0x1FFE_0000 + \text{SRAM_size}/2)$
- $\text{SRAM_U} = 0x2000_0000 - (0x2000_0000 + \text{SRAM_size}/2)$

4.2.5.3.4.3 SRAM accesses

The SRAM is split into two logical arrays that are 64-bits wide:

- **SRAM_L** — Accessible by the code bus of the Cortex-M4 core and by the backdoor port.
- **SRAM_U** — Accessible by the system bus of the Cortex-M4 core and by the backdoor port.

The backdoor port makes the SRAM accessible to the non-core bus masters (such as DMA).

The figure below illustrates the SRAM accesses within the device.

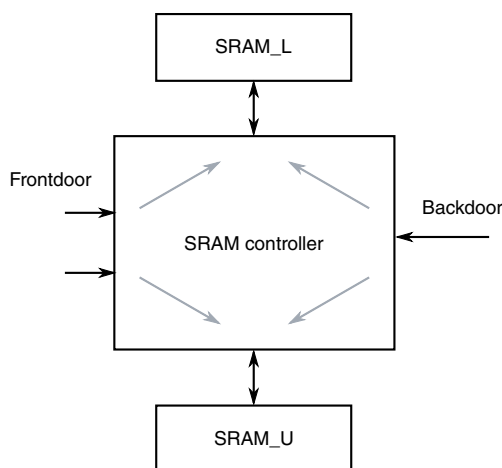


Figure 4-5. SRAM access diagram

The following simultaneous accesses can be made to different logical halves of the SRAM:

- Core code and core system
- Core code and non-core master
- Core system and non-core master

NOTE

Two non-core masters cannot access SRAM simultaneously. The required arbitration and serialization is provided by the crossbar switch. The SRAM_{L,U} arbitration is controlled by the SRAM controller based on the configuration bits in the MCM module.

4.2.5.3.5 Cache Function

The caches on this device are structured as follows. Both caches have a 2-way set-associative cache structure with a total size of 32 KBytes. The caches have a 32-bit address, 64-bit data paths and a 32-byte line size. The cache tags and data storage use single-port, synchronous RAMs.

For these 16-KByte caches, each cache TAG function uses two 256 x 22-bit RAM arrays and the cache DATA function uses two 1024 x 32-bit RAM arrays. The cache TAG entries store 20 bits of upper address as well as a modified and valid bit per cache line. The cache DATA entries store eight bytes of code or data.

All normal cache accesses use physical addresses. This leads to the following cache address use:

CACHE - 16 KByte size = (256 sets) x (32-byte lines) x (2-way set associative)

TAG:

- Only address[31:29] and [21:13] are implemented. All other bits are tied to zero. Only the memories below are supported:
 - address[31] - DDR space, first 2M (0x8000_0000 - 0x801F_FFFF)
 - address[30:29] - FlexSPI channel A, first 2M (0x6000_0000 - 0x601F_FFFF)
 - address[30:29]+[21] - FlexSPI channel A, second 2M (0x6020_0000 - 0x603F_FFFF)
 - address[29]+[21] - OCRAM (0x2020_0000 - 0x203F_FFFF)

NOTE

To use cache, user needs to configure MPU to set those memories as cacheable and all the other memories set as non-cacheable.

DATA

- address[31:13] not used
- address[12:5] used to select one of 256 sets
- address[4:2] used to select one of eight 32-bit words within a set
- address[1:0] used to select the byte within the 32-bit word

4.2.5.3.6 Cache Control

The Code and System Caches are disabled at reset. Cache tag and data arrays are not cleared at reset. Therefore, to enable the caches, cache commands must be done to clear and initialize the required tag array bits and to configure and enable the caches.

4.2.5.3.6.1 Cache set commands

The cache set commands may operate on:

- all of way 0,
- all of way 1, or
- all of both ways (complete cache).

Cache set commands are initiated using the upper bits in the CCR register. Cache set commands perform their operation on the cache independent of the cache enable bit, CCR[ENCACHE].

A cache set command is initiated by setting the CCR[GO] bit. This bit also acts as a busy bit for set commands. It stays set while the command is active and is cleared by the hardware when the set command completes.

Supported cache set commands are given in the table below. Set commands work as follows:

- Invalidate – Unconditionally clear valid and modify bits of a cache entry.
- Push – Push a cache entry if it is valid and modified, then clear the modify bit. If entry not valid or not modified, leave as is.
- Clear – Push a cache entry if it is valid and modified, then clear the valid and modify bits. If entry not valid or not modified, clear the valid bit.

Table 4-4. Cache Set Commands

CCR[27:24]				Command
PUSH W1	INVW1	PUSH W0	INVW0	
0	0	0	0	NOP
0	0	0	1	Invalidate all way 0
0	0	1	0	Push all way 0
0	0	1	1	Clear all way 0
0	1	0	0	Invalidate all way 1
0	1	0	1	Invalidate all way 1; invalidate all way 0 (invalidate cache)
0	1	1	0	Invalidate all way 1; push all way 0
0	1	1	1	Invalidate all way 1; clear all way 0
1	0	0	0	Push all way 1
1	0	0	1	Push all way 1; invalidate all way 0
1	0	1	0	Push all way 1; push all way 0 (push cache)
1	0	1	1	Push all way 1; clear all way 0
1	1	0	0	Clear all way 1
1	1	0	1	Clear all way 1; invalidate all way 0
1	1	1	0	Clear all way 1; push all way 0
1	1	1	1	Clear all way 1; clear all way 0 (clear cache)

After a reset, complete an invalidate cache command before using the cache. It is possible to combine the cache invalidate command with the cache enable. That is, setting CCR to 0x8500_0003 will invalidate the cache and enable the cache and write buffer.

4.2.5.3.6.2 Cache line commands

Cache line commands operate on a single line in the cache at a time. Cache line commands can be performed using a physical or cache address.

- A cache address consists of a set address and a way select. The line command acts on the specified cache line.
- Cache line commands with physical addresses first search both ways of the cache set specified by bits [11:4] of the physical address. If they hit, the commands perform their action on the hit way.

Cache line commands are specified using the upper bits in the CLCR register. Cache line commands perform their operation on the cache independent of the cache enable bit (CCR[ENCACHE]). Using a cache address, the command can be completely specified using the CLCR register. Using a physical address, the command must also use the CSAR register to specify the physical address.

A line cache command is initiated by setting the line command go bit (CLCR[LGO] or CSAR[LGO]). This bit also acts as a busy bit for line commands. It stays set while the command is active and is cleared by the hardware when the command completes.

The CLCR[27:24] bits select the line command as follows:

Table 4-5. Cache Line Commands

CLCR[27:24]			Command
LACC	LADSEL	LCMD	
0	0	00	Search by cache address and way
0	0	01	Invalidate by cache address and way
0	0	10	Push by cache address and way
0	0	11	Clear by cache address and way
0	1	00	Search by physical address
0	1	01	Invalidate by physical address
0	1	10	Push by physical address
0	1	11	Clear by physical address
1	0	00	Write by cache address and way
1	0	01	Reserved, NOP
1	0	10	Reserved, NOP
1	0	11	Reserved, NOP
1	1	xx	Reserved, NOP

4.2.5.3.6.2.1 Executing a series of line commands using cache addresses

A series of line commands with incremental cache addresses can be performed by just writing to the CLCR.

- Place the command in CLCR[27:24],
- Set the way (CLCR[WSEL]) and tag/data (CLCR[TDSEL]) controls as needed,

- Place the cache address in CLCR[CACHEADDR], and
- Set the line command go bit (CLCR[LGO]).

When one line command completes, initiate the next command by following these steps:

- Increment the cache address (at bit 2 to step through data or at bit 4 to step through lines), and
- Set the line command go bit (CLCR[LGO]).

4.2.5.3.6.2.2 Executing a series of line commands using physical addresses

Perform a series of line commands with incremental physical addresses using the following steps:

- Write to the CLCR.
 - Place the command in CLCR[27:24]
 - Set the tag/data (CLCR[TDSEL]) control
- Place the physical address in CSAR[PHYADDR] and set the line command go bit (CSAR[LGO]).

When one line command completes, initiate the next command by following these steps:

- Increment the physical address (at bit 2 to step through data or at bit 4 to step through lines), and
- Set the line command go bit (CSAR[LGO]).

The line command go bit is shared between the CLCR and CSAR registers, so that the above steps can be completed in a single write to the CSAR register.

4.2.5.3.6.2.3 Line command results

At completion of a line command, the CLCR register contains information on the initial state of the line targeted by the command. For line commands with cache addresses, this information is read before the line command action is performed from the targeted cache line. For line commands with physical addresses, this information is read on a hit before the line command action is performed from the hit cache line or has initial valid bit cleared if the command misses. In general, if the valid indicator (CLCR[LCIVB]) is cleared, the targeted line was invalid at the start of the line command and no line operation was performed.

Table 4-6. Line command results

CLCR[22:20]			For cache address commands	For physical address commands
LCWAY	LCIMB	LCIVB		
0	0	0	Way 0 line was invalid	No hit
0	0	1	Way 0 valid, not modified	Way 0 valid, not modified
0	1	0	Way 0 line was invalid	No hit
0	1	1	Way 0 valid and modified	Way 0 valid and modified
1	0	0	Way 1 line was invalid	No hit
1	0	1	Way 1 valid, not modified	Way 1 valid, not modified
1	1	0	Way 1 line was invalid	No hit
1	1	1	Way 1 valid and modified	Way 1 valid and modified

At completion of a line command other than a write, the CCVR (Cache R/W Value Register) contains information on the initial state of the line tag or data targeted by the command. For line commands, CLCR[TDSEL] selects between tag and data. If the line command used a physical address and missed, the data is don't care. For write commands, the CCVR holds the write data.

4.2.6 Miscellaneous Control Module (MCM)

The Miscellaneous Control Module (MCM) provides a myriad of miscellaneous control functions.

4.2.6.1 MCM features

The MCM includes the following features:

- Program-visible information on the platform configuration and revision

4.2.6.2 MCM Interrupts

The MCM generates the following interrupt requests:

- Normal interrupt

4.2.6.2.1 Normal interrupt

The MCM's normal interrupt is generated if any of the following is true:

- ISCR[ETBI] is set, when
 - The ETB counter is enabled, ETBCC[NTEN] = 1
 - The ETB count expires
 - The response to counter expiration is a normal interrupt, ETBCC[RSPT] = 01

4.2.7 LMEM Memory Map/Register Definition

LMEM memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
E008_2000	Cache control register (LMEM_PCCCR)	32	R/W	0000_0000h	4.2.7.1/101
E008_2004	Cache line control register (LMEM_PCCLCR)	32	R/W	0000_0000h	4.2.7.2/103
E008_2008	Cache search address register (LMEM_PCCSAR)	32	R/W	0000_0000h	4.2.7.3/105
E008_200C	Cache read/write value register (LMEM_PCCCVR)	32	R/W	0000_0000h	4.2.7.4/106
E008_2800	Cache control register (LMEM_PSCCR)	32	R/W	0000_0000h	4.2.7.5/107
E008_2804	Cache line control register (LMEM_PSCLCR)	32	R/W	0000_0000h	4.2.7.6/108
E008_2808	Cache search address register (LMEM_PSCSAR)	32	R/W	0000_0000h	4.2.7.7/111
E008_280C	Cache read/write value register (LMEM_PSCCVR)	32	R/W	0000_0000h	4.2.7.8/112

4.2.7.1 Cache control register (LMEM_PCCCR)

Address: E008_2000h base + 0h offset = E008_2000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

LMEM_PCCCR field descriptions

Field	Description
31 GO	Initiate Cache Command

Table continues on the next page...

LMEM_PCCCR field descriptions (continued)

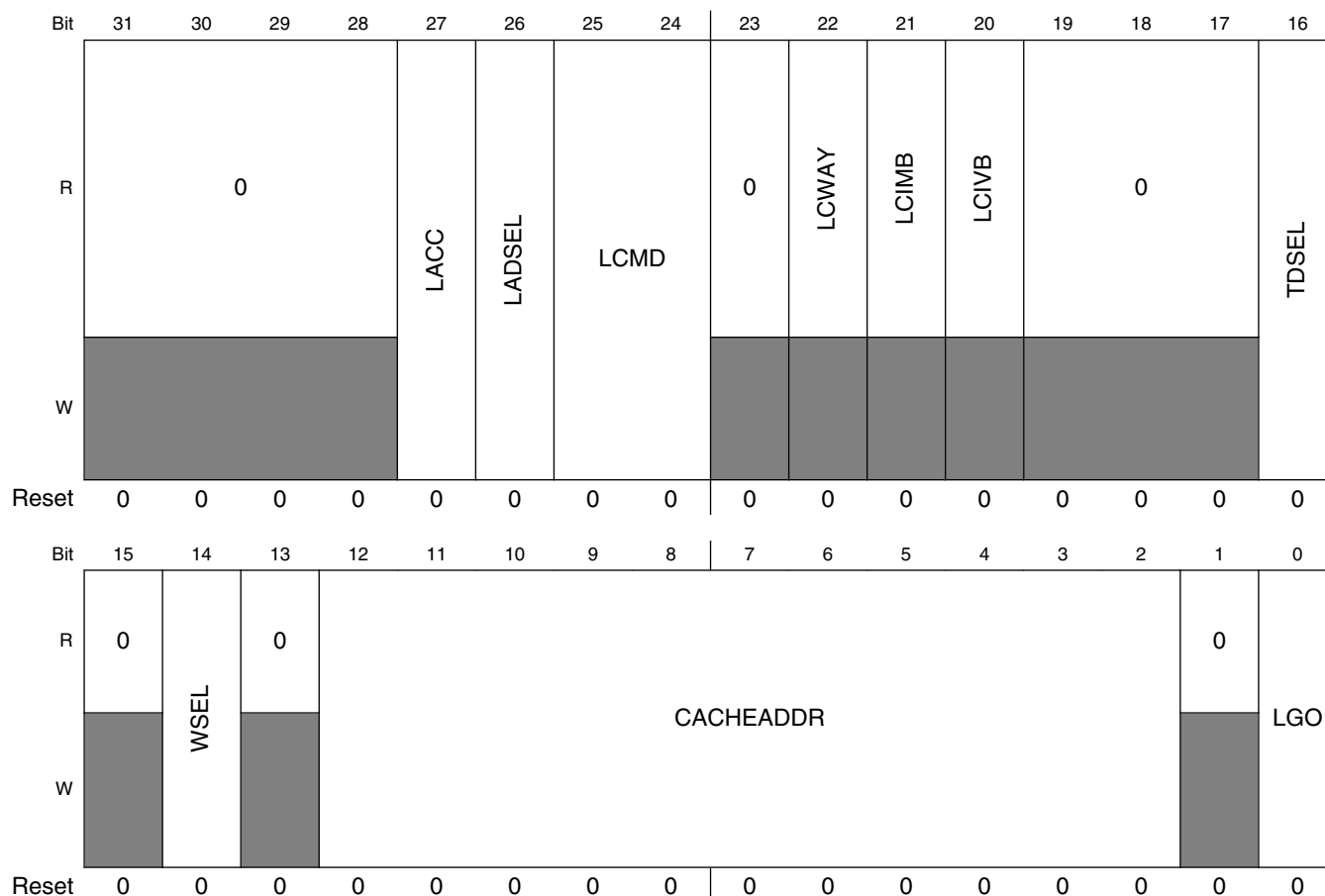
Field	Description
	Setting this bit initiates the cache command indicated by bits 27-24. Reading this bit indicates if a command is active NOTE: This bit stays set until the command completes. Writing zero has no effect. 0 Write: no effect. Read: no cache command active. 1 Write: initiate command indicated by bits 27-24. Read: cache command active.
30–28 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
27 PUSHW1	Push Way 1 0 No operation 1 When setting the GO bit, push all modified lines in way 1
26 INVW1	Invalidate Way 1 NOTE: If the PUSHW1 and INVW1 bits are set, then after setting the GO bit, push all modified lines in way 1 and invalidate all lines in way 1 (clear way 1). 0 No operation 1 When setting the GO bit, invalidate all lines in way 1
25 PUSHW0	Push Way 0 0 No operation 1 When setting the GO bit, push all modified lines in way 0
24 INVW0	Invalidate Way 0 NOTE: If the PUSHW0 and INVW0 bits are set, then after setting the GO bit, push all modified lines in way 0 and invalidate all lines in way 0 (clear way 0). 0 No operation 1 When setting the GO bit, invalidate all lines in way 0.
23–4 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
3 PCCR3	Forces no allocation on cache misses (must also have ACCR2 asserted)
2 PCCR2	Forces all cacheable spaces to write through
1 ENWRBUF	Enable Write Buffer 0 Write buffer disabled 1 Write buffer enabled
0 ENCACHE	Cache enable 0 Cache disabled 1 Cache enabled

4.2.7.2 Cache line control register (LMEM_PCCLCR)

This register defines specific line-sized cache operations to be performed using a specific cache line address or a physical address.

If a physical address is specified, both ways of the cache are searched, and the command is only performed on the way which hits.

Address: E008_2000h base + 4h offset = E008_2004h



LMEM_PCCLCR field descriptions

Field	Description
31–28 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
27 LACC	Line access type 0 Read 1 Write
26 LADSEL	Line Address Select When using the cache address, the way must also be specified in CLCR[WSEL].

Table continues on the next page...

LMEM_PCCLCR field descriptions (continued)

Field	Description
	When using the physical address, both ways are searched and the command is performed only if a hit. 0 Cache address 1 Physical address
25–24 LCMD	Line Command 00 Search and read or write 01 Invalidate 10 Push 11 Clear
23 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
22 LCWAY	Line Command Way Indicates the way used by the line command.
21 LCIMB	Line Command Initial Modified Bit If command used cache address and way, then this bit shows the initial state of the modified bit If command used physical address and a hit, then this bit shows the initial state of the modified bit. If a miss, this bit reads zero.
20 LCIVB	Line Command Initial Valid Bit If command used cache address and way, then this bit shows the initial state of the valid bit If command used physical address and a hit, then this bit shows the initial state of the valid bit. If a miss, this bit reads zero.
19–17 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
16 TDSEL	Tag/Data Select Selects tag or data for search and read or write commands. 0 Data 1 Tag
15 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
14 WSEL	Way select Selects the way for line commands. 0 Way 0 1 Way 1
13 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
12–2 CACHEADDR	Cache address CLCR[11:4] bits are used to access the tag arrays CLCR[11:2] bits are used to access the data arrays
1 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.

Table continues on the next page...

LMEM_PCCLCR field descriptions (continued)

Field	Description
0 LGO	Initiate Cache Line Command Setting this bit initiates the cache line command indicated by bits 27-24. Reading this bit indicates if a line command is active NOTE: This bit stays set until the command completes. Writing zero has no effect. NOTE: This bit is shared with CSAR[LGO] 0 Write: no effect. Read: no line command active. 1 Write: initiate line command indicated by bits 27-24. Read: line command active.

4.2.7.3 Cache search address register (LMEM_PCCSAR)

The CSAR register is used to define the explicit cache address or the physical address for line-sized commands specified in the CLCR[LADSEL] bit.

Address: E008_2000h base + 8h offset = E008_2008h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PHYADDR															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PHYADDR															LGO
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

LMEM_PCCSAR field descriptions

Field	Description
31–2 PHYADDR	Physical Address PHYADDR represents bits [31:2] of the system address. CSAR[31:12] bits are used for tag compare CSAR[11:4] bits are used to access the tag arrays CSAR[11:2] bits are used to access the data arrays
1 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
0 LGO	Initiate Cache Line Command Setting this bit initiates the cache line command indicated by bits 27-24. Reading this bit indicates if a line command is active NOTE: This bit stays set until the command completes. Writing zero has no effect. NOTE: This bit is shared with CLCR[LGO]

Table continues on the next page...

LMEM_PCCSAR field descriptions (continued)

Field	Description
0	Write: no effect. Read: no line command active.
1	Write: initiate line command indicated by bits CLCR[27:24]. Read: line command active.

4.2.7.4 Cache read/write value register (LMEM_PCCCVR)

The CCVR register is used to source write data or return read data for the commands specified in the CLCR register.

Address: E008_2000h base + Ch offset = E008_200Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
	DATA																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

LMEM_PCCCVR field descriptions

Field	Description
DATA	Cache read/write Data For tag search, read or write: • CCVR[31:12] bits are used for tag array R/W value • CCVR[11:4] bits are used for tag set address on reads; unused on writes • CCVR[3:2] bits are reserved For data search, read or write: • CCVR[31:0] bits are used for data array R/W value

4.2.7.5 Cache control register (LMEM_PSCCR)

Address: E008_2000h base + 800h offset = E008_2800h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R		0			PUSHW1	INVW1	PUSHW0	INVW0	0							
W	GO															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0														ENWRBUF	ENCACHE
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

LMEM_PSCCR field descriptions

Field	Description
31 GO	Initiate Cache Command Setting this bit initiates the cache command indicated by bits 27-24. Reading this bit indicates if a command is active NOTE: This bit stays set until the command completes. Writing zero has no effect. 0 Write: no effect. Read: no cache command active. 1 Write: initiate command indicated by bits 27-24. Read: cache command active.
30–28 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
27 PUSHW1	Push Way 1 0 No operation 1 When setting the GO bit, push all modified lines in way 1
26 INVW1	Invalidate Way 1 NOTE: If the PUSHW1 and INVW1 bits are set, then after setting the GO bit, push all modified lines in way 1 and invalidate all lines in way 1 (clear way 1). 0 No operation 1 When setting the GO bit, invalidate all lines in way 1
25 PUSHW0	Push Way 0 0 No operation 1 When setting the GO bit, push all modified lines in way 0
24 INVW0	Invalidate Way 0

Table continues on the next page...

LMEM_PSCCR field descriptions (continued)

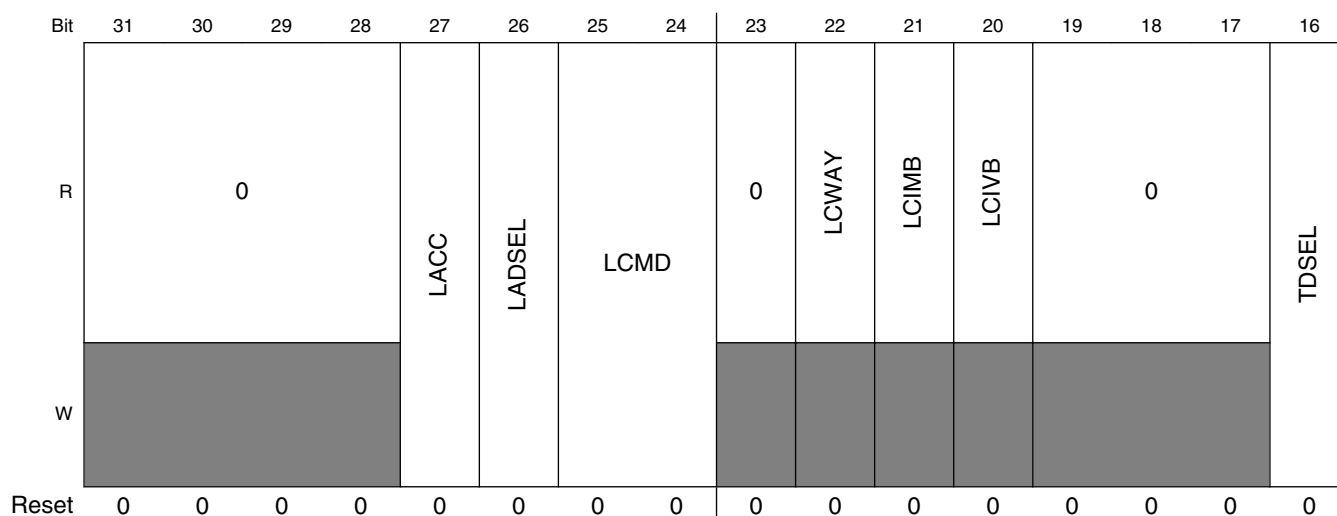
Field	Description
	NOTE: If the PUSHW0 and INVW0 bits are set, then after setting the GO bit, push all modified lines in way 0 and invalidate all lines in way 0 (clear way 0). 0 No operation 1 When setting the GO bit, invalidate all lines in way 0.
23–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
1 ENWRBUF	Enable Write Buffer 0 Write buffer disabled 1 Write buffer enabled
0 ENCACHE	Cache enable 0 Cache disabled 1 Cache enabled

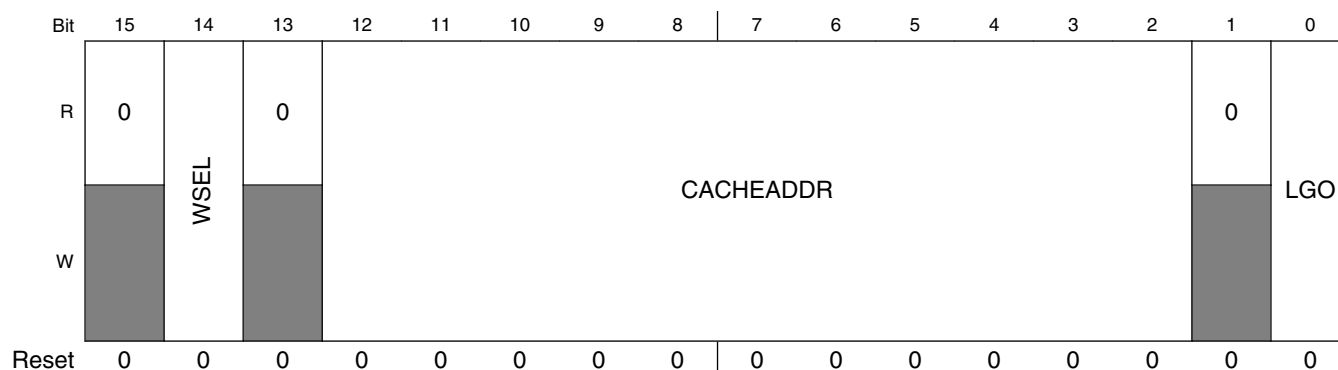
4.2.7.6 Cache line control register (LMEM_PSCLCR)

This register defines specific line-sized cache operations to be performed using a specific cache line address or a physical address.

If a physical address is specified, both ways of the cache are searched, and the command is only performed on the way which hits.

Address: E008_2000h base + 804h offset = E008_2804h





LMEM_PSCLCR field descriptions

Field	Description
31–28 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
27 LACC	Line access type 0 Read 1 Write
26 LADSEL	Line Address Select When using the cache address, the way must also be specified in CLCR[WSEL]. When using the physical address, both ways are searched and the command is performed only if a hit. 0 Cache address 1 Physical address
25–24 LCMD	Line Command 00 Search and read or write 01 Invalidate 10 Push 11 Clear
23 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
22 LCWAY	Line Command Way Indicates the way used by the line command.
21 LCIMB	Line Command Initial Modified Bit If command used cache address and way, then this bit shows the initial state of the modified bit If command used physical address and a hit, then this bit shows the initial state of the modified bit. If a miss, this bit reads zero.
20 LCIVB	Line Command Initial Valid Bit If command used cache address and way, then this bit shows the initial state of the valid bit If command used physical address and a hit, then this bit shows the initial state of the valid bit. If a miss, this bit reads zero.
19–17 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.

Table continues on the next page...

LMEM_PSCLCR field descriptions (continued)

Field	Description
16 TDSEL	Tag/Data Select Selects tag or data for search and read or write commands. 0 Data 1 Tag
15 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
14 WSEL	Way select Selects the way for line commands. 0 Way 0 1 Way 1
13 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
12–2 CACHEADDR	Cache address CLCR[11:4] bits are used to access the tag arrays CLCR[11:2] bits are used to access the data arrays
1 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
0 LGO	Initiate Cache Line Command Setting this bit initiates the cache line command indicated by bits 27-24. Reading this bit indicates if a line command is active NOTE: This bit stays set until the command completes. Writing zero has no effect. NOTE: This bit is shared with CSAR[LGO] 0 Write: no effect. Read: no line command active. 1 Write: initiate line command indicated by bits 27-24. Read: line command active.

4.2.7.7 Cache search address register (LMEM_PSCSAR)

The CSAR register is used to define the explicit cache address or the physical address for line-sized commands specified in the CLCR[LADSEL] bit.

Address: E008_2000h base + 808h offset = E008_2808h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PHYADDR															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PHYADDR															0
W																LGO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

LMEM_PSCSAR field descriptions

Field	Description
31–2 PHYADDR	Physical Address PHYADDR represents bits [31:2] of the system address. CSAR[31:12] bits are used for tag compare CSAR[11:4] bits are used to access the tag arrays CSAR[11:2] bits are used to access the data arrays
1 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
0 LGO	Initiate Cache Line Command Setting this bit initiates the cache line command indicated by bits 27-24. Reading this bit indicates if a line command is active NOTE: This bit stays set until the command completes. Writing zero has no effect. NOTE: This bit is shared with CLCR[LGO] 0 Write: no effect. Read: no line command active. 1 Write: initiate line command indicated by bits CLCR[27:24]. Read: line command active.

4.2.7.8 Cache read/write value register (LMEM_PSCCVR)

The CCVR register is used to source write data or return read data for the commands specified in the CLCR register.

Address: E008_2000h base + 80Ch offset = E008_280Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DATA																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

LMEM_PSCCVR field descriptions

Field	Description
DATA	Cache read/write Data For tag search, read or write: • CCVR[31:12] bits are used for tag array R/W value • CCVR[11:4] bits are used for tag set address on reads; unused on writes • CCVR[3:2] bits are reserved For data search, read or write: • CCVR[31:0] bits are used for data array R/W value

4.2.8 MCM Memory Map/Register Definition

MCM memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
E008_0008	Crossbar Switch (AXBS) Slave Configuration (MCM_PLASC)	16	R	0002h	4.2.8.1/113
E008_000A	Crossbar Switch (AXBS) Master Configuration (MCM_PLAMC)	16	R	0003h	4.2.8.2/113
E008_000C	Crossbar Switch (AXBS) Control Register (MCM_PLACR)	32	R/W	0000_0000h	4.2.8.3/114
E008_0020	Fault address register (MCM_FADR)	32	R	Undefined	4.2.8.4/114
E008_0024	Fault attributes register (MCM_FATR)	32	R	Undefined	4.2.8.5/115
E008_0028	Fault data register (MCM_FDR)	32	R	Undefined	4.2.8.6/117

4.2.8.1 Crossbar Switch (AXBS) Slave Configuration (MCM_PLASC)

PLASC is a 16-bit read-only register identifying the presence/absence of bus slave connections to the device's crossbar switch.

Address: E008_0000h base + 8h offset = E008_0008h

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read	0								ASC							
Write																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

MCM_PLASC field descriptions

Field	Description
15–8 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
ASC	Each bit in the ASC field indicates whether there is a corresponding connection to the crossbar switch's slave input port. 0 A bus slave connection to AXBS input port <i>n</i> is absent 1 A bus slave connection to AXBS input port <i>n</i> is present

4.2.8.2 Crossbar Switch (AXBS) Master Configuration (MCM_PLAMC)

PLAMC is a 16-bit read-only register identifying the presence/absence of bus master connections to the device's crossbar switch.

Address: E008_0000h base + Ah offset = E008_000Ah

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read	0								AMC							
Write																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1

MCM_PLAMC field descriptions

Field	Description
15–8 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
AMC	Each bit in the AMC field indicates whether there is a corresponding connection to the AXBS master input port.

Table continues on the next page...

MCM_PLAMC field descriptions (continued)

Field	Description
0	A bus master connection to AXBS input port <i>n</i> is absent
1	A bus master connection to AXBS input port <i>n</i> is present

4.2.8.3 Crossbar Switch (AXBS) Control Register (MCM_PLACR)

The PLACR register selects the arbitration policy for the crossbar masters.

Address: E008_0000h base + Ch offset = E008_000Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																Reserved															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

MCM_PLACR field descriptions

Field	Description
31–9 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
Reserved	This field is reserved.

4.2.8.4 Fault address register (MCM_FADR)

When a properly-enabled cache write buffer error interrupt event is detected, the faulting address is captured in the MCM_FADR register. The MCM logic supports capturing a single cache write buffer bus error event; if a subsequent error is detected before the captured error information has been read from the corresponding registers and the MCM_ISCR[CWBER] indicator cleared, the MCM_FATR[BEOVR] flag is set. However, no additional information is captured.

The bits in this register are set by hardware and signaled by the assertion of MCM_ISCR[CWBER]. Attempted writes to this location are terminated with an error.

Address: E008_0000h base + 20h offset = E008_0020h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ADDRESS																															
W																																
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

* Notes:

- x = Undefined at reset.

MCM_FADR field descriptions

Field	Description
ADDRESS	Fault address

4.2.8.5 Fault attributes register (MCM_FATR)

When a properly-enabled cache write buffer error interrupt event is detected, the faulting attributes are captured in the MCM_FATR register.

The bits in this register are set by hardware and signaled by the assertion of MCM_ISCR[CWBER]. Attempted writes to this location are terminated with an error.

Address: E008_0000h base + 24h offset = E008_0024h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	BEOVR	0														
W																
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0				BEMN				BEWT	0	BESZ		0		BEMD	BEDA
W																
Reset	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*	x*

* Notes:

- x = Undefined at reset.

MCM_FATR field descriptions

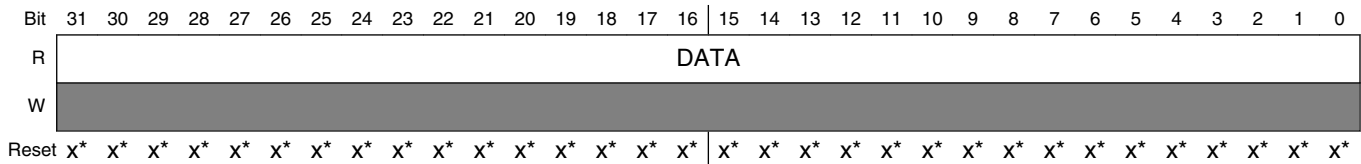
Field	Description
31 BEOVR	Bus error overrun Indicates if another cache write buffer bus error is detected before system software has retrieved all the error information from the original event, this overrun flag is set. The window of time is defined from the detection of the original cache write buffer error termination until the MCM_ISCR[CWBER] is written with a 1 to clear it and rearm the capture logic. This bit is set by the hardware and cleared whenever software writes a 1 to the CWBER bit. 0 No bus error overrun 1 Bus error overrun occurred. The FADR and FDR registers and the other FATR bits are not updated to reflect this new bus error.
30–12 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
11–8 BEMN	Bus error master number Crossbar switch bus master number of the captured cache write buffer bus error. For this device, this value is always 0x1.
7 BEWT	Bus error write Indicates the type of system bus access when the error was detected. Since this logic is monitoring data transfers from the cache write buffer, this bit is always a logical one, signaling a write operation. 0 Read access 1 Write access
6 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
5–4 BESZ	Bus error size Indicates the size of the cache write buffer access when the error was detected. 00 8-bit access 01 16-bit access 10 32-bit access 11 Reserved
3–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
1 BEMD	Bus error privilege level Indicates the privilege level of the cache write buffer access when the error was detected. 0 User mode 1 Supervisor/privileged mode
0 BEDA	Bus error access type Indicates the type of cache write buffer access when the error was detected. This attribute is always a logical one signaling a data reference. 0 Instruction 1 Data

4.2.8.6 Fault data register (MCM_FDR)

When a properly-enabled cache write buffer error interrupt event is detected, the faulting data is captured in the MCM_FDR register.

The bits in this register are set by hardware and signaled by the assertion of MCM_ISCR[CWBER]. For byte and halfword writes, only the accessed byte lanes contain valid data; the contents of the other bytes are undefined. Attempted writes to this location are terminated with an error.

Address: E008_0000h base + 28h offset = E008_0028h



* Notes:

- x = Undefined at reset.

MCM_FDR field descriptions

Field	Description
DATA	Fault data

4.3 Messaging Unit (MU)

4.3.1 Overview

The Messaging Unit module enables two processors within the SoC to communicate and coordinate by passing messages (e.g. data, status and control) through the MU interface. The MU also provides the ability for one processor to signal the other processor using interrupts.

Because the MU manages the messaging between processors, the MU uses different clocks (from each side of the different peripheral buses). Therefore, the MU must synchronize the accesses from one side to the other. The MU accomplishes synchronization using two sets of matching registers (Processor A-facing, Processor B-facing).

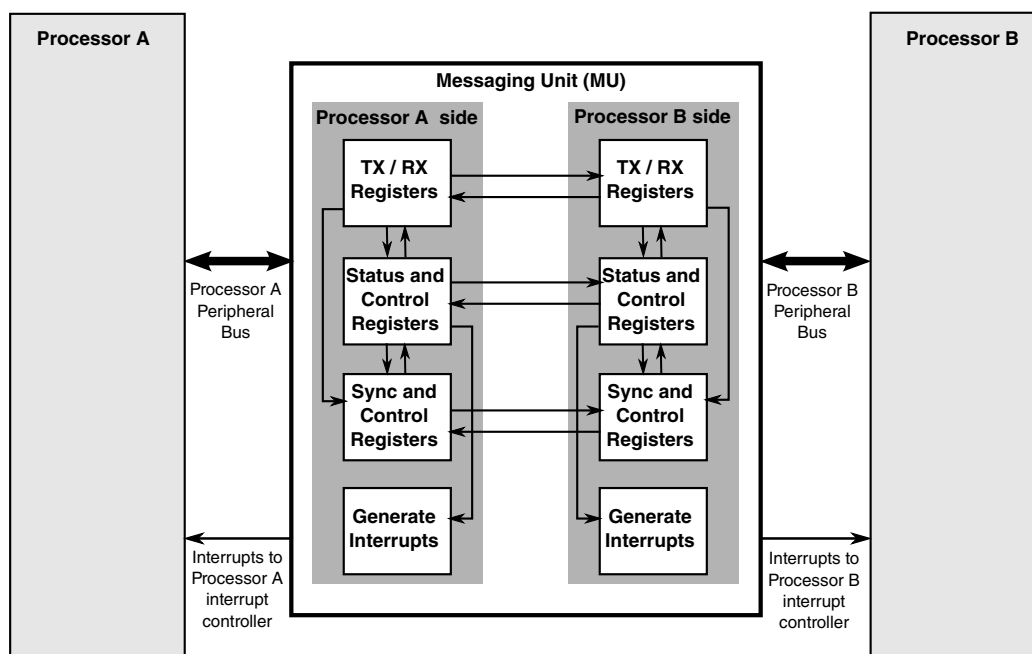


Figure 4-6. MU Block Diagram

4.3.1.1 Features

The MU includes the following features:

- Messaging control by interrupts or by polling
- The Processor B can take the Processor A out of low-power modes by asserting one of the above twelve interrupts to the Processor A and vice versa
- Symmetrical processor interfaces with each side supporting the following:
 - Four general-purpose interrupt requests reflected to the other side
 - Three general-purpose flags reflected to the other side
 - Four receive registers with maskable interrupt
 - Four transmit registers with maskable interrupt

4.3.1.2 Modes of Operation

The MU supports the modes described in the indicated sections:

- [Operating Modes](#)
- [Low Power Modes](#)

4.3.2 Functional Description

Table 4-7. Major Features of the MU

Major Feature	Description
Interprocessor Interrupts	The MU has 12 interrupt sources on each side (Processor A-side, Processor B-side) that are used for signaling the other processor. The interrupts can be used for notification of RX/TX events and general-purpose signaling between the processors.
MU Reset	- The Processor A can issue a reset to the entire MU, using a control bit (MUR) in the Processor A Control Register (ACR). - The MUR bit is a self-clearing bit.
Processor B Boot Configuration	- The Boot Source for Processor B can be configured with the BBOOT bits in the ACR register. - Boot Source Options are: - DMEM Base Address - IMEM Base Address - Address 0x00 - The value at reset is loaded from Flash IFR
Processor B Reset Hold	- Processor B can be held in reset following any reset event. This is done by setting the BRSTH bit in the ACR register. - Processor B will be released from reset when this bit is cleared. - The value at reset is loaded from Flash IFR.
Processor A/B Clock Enable	The Processor A/B platform clock can be enabled to continue running when Processor A/B enters Stop Mode, until Processor B/A also enters Stop Mode. This allows Processor B/A to continue accessing peripherals on Processor A/B's AIPS bus even when it has entered a Stop Mode.
Status and Control Communications between Cores	- The MU provides a way for the two cores to communicate using the status and control registers present on both the Processor B and Processor A sides of the MU. - The status register of one MU side reflects the status of the other MU side. - The control register is used for control operations, such as enabling an interrupt and sending an interrupt to the other processor.
Synchronized Message Transfers between Cores	- The transfer of data messages between cores uses transmit empty and receive full flags provided on both sides of the MU. - The update of these transmit and receive flags is accomplished using a synchronization mechanism. There is inherent latency between updating the flag on one side and reflecting its status on other side. For more about latency, see Event Update Timing
Accessing Shared Memory Directly and Avoiding Collisions	- For sending data or messages from one MU-side to the other MU-side, the MU provides 4 transmit registers and 4 receive registers on each side of the MU. - The Processor A or Processor B can access shared memory resources of the SoC directly. However, to avoid simultaneous access to shared memory by both cores, the MU provides a method (to prevent simultaneous access) using interrupts and transmit-receive registers for both processors.
Support for Different Clocks in the Two Cores	- The heart of the MU module is the event control mechanism, which synchronizes the access of one MU-side to the other MU-side, because these two MU-sides can operate using different clocks. - Formulated event update latency.
Memory-Mapped Registers	The MU is connected as a peripheral under the Peripheral bus on both sides—on the Processor A-side, the Processor A Peripheral Bus, and on the Processor B-side, the Processor B Peripheral Bus.

4.3.2.1 Processor A Side Memory-Mapping

The messaging, control, and status registers of the Processor A-side for the MU are mapped to the Processor A memory as a regular peripheral. The Peripheral bus data bus is 32 bits wide inside the MU module.

4.3.2.2 Processor B Side Memory-Mapping

The messaging, control, and status registers of the Processor B-side for the MU are mapped to the Processor B memory as a regular peripheral. The Peripheral bus data bus is 32 bits wide inside the MU module.

4.3.2.3 MU Messaging

The MU provides 32-bit status and control registers to the Processor B and Processor A sides for control operations (such as interrupts and reset), and for status checking of the other MU-side.

For messaging, the MU has four, 32-bit write-only transmit registers and four, 32-bit read-only receive registers on the Processor B and Processor A-sides. These registers are used for sending messages to each other. These messages can also be controlled using the 3 general purpose flags provided in the control and status registers of either MU-side.

4.3.2.3.1 Programmer Model

The messaging logic is used in conjunction with external memory. You have various messaging methods, which you can use to implement a messaging protocol. Some of these messages could mean “I have just written a message of N words, starting at offset X in the memory,” or “I have just finished reading the previous data block that was sent.” Having the messaging logic independent from the memory array does not restrict you to a predefined hardware protocol. On the other hand, the software needed to manage the messaging is short and straightforward.

Most of the messaging mechanisms are symmetric; they are duplicated and are available on both the Processor B-side and the Processor A-side. The messaging mechanisms are:

- Four, 32-bit write-only transmit registers, which are each reflected in four, read-only receive registers in the other processor’s side. You can use these registers to transfer 32-bit word messages or frame information of messages written to the shared memory (number of words, initial address, and message type code).

- A write to a transmit register on the transmitter side clears a “transmitter empty” bit in the Status Register on the transmitter side, and sets a “receiver full” bit in the Status Register on the receiver side. The setting of the bit at the receiver side can optionally trigger an interrupt at the receiver side (maskable receive interrupt).
- A read of one of the receive registers at the receiver side clears the “receiver full” bit in the Status Register at the receiver side, and sets the “transmitter empty” bit in the Status Register on the transmitter side. The setting of the “transmitter empty” bit can optionally trigger an interrupt at the transmitter side (maskable transmit interrupt).
- Four general purpose flags are reflected in the Status Register on the receiver side
- A read/write access to any reserved location and a write to a read-only register on the Processor A-side of the MU will generate a module transfer error acknowledge to the Processor A.
- A read/write access to any reserved location and write to a read-only register on the Processor B-side of the MU will generate a module transfer error acknowledge to the Processor B.

4.3.2.3.2 Messaging Examples

The following are messaging examples:

- **Passing short messages:** Transmit register(s) can be used to pass short messages from one to four words in length. For example, when a four-word message is desired, only one of the registers needs to have its corresponding interrupt enable bit set at the receiver side; the message’s first three words are written to the registers whose interrupt is masked, and the fourth word is written to the other register (which triggers an interrupt at the receiver side).
- **Passing frame information:** Transmit registers can be used to pass frame information for long messages written to the shared system (SDRAM and SyncFLASH). Such frame information normally includes a start address, number of words, and perhaps a message type code.
- **Passing event notices and requests:** Events and requests that do not include data words can be signaled from the Processor B to the Processor A using the general interrupts, such as acknowledging that a long message was read from the shared system memory.
- **Passing fixed length data:** Formatted data with a fixed length can be written in predetermined locations in the shared memory. A processor can use a general interrupt (Processor A or Processor B) to signal the other processor that the data is ready.
- **Passing announcements:** The three flags can be used by a processor to announce its current program state or other billboard messages to the other processor.

Figure 4-7 shows the MU registers schematic.

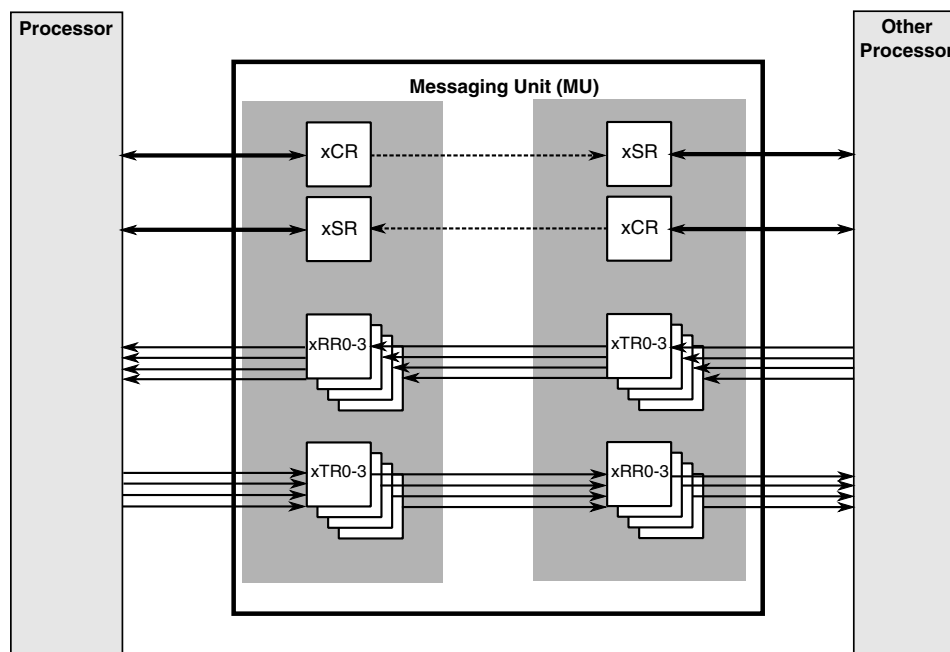


Figure 4-7. MU Registers

4.3.2.4 Operating Modes

This section describes all functional operation modes of the module.

4.3.2.5 Low Power Modes

This section describes the low power operating modes of the MU module.

4.3.2.5.1 Low Power Clocks and Synchronization

The Processor B and the Processor A clocks operate at different frequencies and from different sources. The MU design does not assume any frequency relationship between the Processor A and the Processor B clocks. Be aware, however, that the frequency relationship affects the MU's throughput performance.

- The data buffers and control logic of each MU-side operate with its corresponding clock.

4.3.2.5.2 Processor Low Power Modes

The Processors have four power modes:

- Run
- WAIT

- STOP
- DSM

The Processor can be awakened from a low-power mode by any enabled Processor side MU interrupt, as reflected in the xSR “status” register (RF0–3, TE0–3, GIP0–3 bits are set) and enabled in the xCR control register. Using these bits, the Processor can actively control when to wake the other Processor.

While the Processor is in STOP mode (such that the xSR register bits cannot be updated with events), special logic drives the enabled Processor interrupts directly from the other Processor-side (instead of from the xSR register).

While the Processor is in STOP mode, the asynchronous Processor interrupt will be asserted to wake the Processor:

- If any transmit data register of the other Processor-side is full, because of a write to it (transmit data register); that is, its “empty” bit in the xSR register is cleared while its corresponding receive interrupt is enabled on the Processor-side.
- If any receive data register of the other Processor-side is empty, because of a read on the other Processor -side; that is, its “full” bit in the xSR register is cleared while its corresponding transmit interrupt is enabled on the Processor-side.
- If any general purpose interrupt is set in the xCR register while the corresponding interrupt is enabled on the Processor-side.
- If the other Processor issues a non-maskable interrupt to the Processor.

The logic enables the other Processor to operate independently while the Processor is in any power mode (including STOP). However, the Processor power mode change protocol should be handled with care regarding:

- The interrupts that are enabled on the Processor-side
- The events that could be triggered by the other Processor-side
- The compatibility with the other Processor protocol of entering STOP mode

If the Processor is in STOP mode and an event on the other Processor is triggered, the EP bit (in the xSR register) will remain high until the Processor wakes up.

Before entering STOP mode, the Processor programmer should verify that the EP bit (in the xSR register) is cleared. This check is needed to ensure that all pending updates from the Processor, including the power mode change when STOP or WAIT is executed, will be updated in the xSR register.

- If the other Processor is in STOP mode or DSM mode, the EP bit (in the xSR register) may be stuck high; in this case, the Processor need not check the EP bit before entering STOP mode.

4.3.2.6 Event Update Timing

Each processor's MU messaging side (Processor B or Processor A) has a hardware mechanism to send "event update requests" to the other processor's side. An "event" is considered when any information change should be reflected at the Status Register of the receiving processor. The event update latency is the delay between the event being ready at one processor and the resulting update at the Status Register of the other processor.

- The minimum event latency is "1 clock of the sending side" + "2 1/2 clocks of the receiving side". The minimum case is if there is no event pending when the new event occurs.
- The maximum event latency is "6 clocks of the sending side" + "6 1/2 clocks of the receiving side." The maximum case is if the event occurred just after a previous event was sent to the other side. The event update latency will vary between the above-mentioned minimum and maximum latencies, depending on the time at which the subsequent event is triggered.

4.3.2.7 Interrupts

The MU controls the Processor B interrupt requests to the Processor A, and the Processor A interrupt requests to the Processor B. This section describes all the interrupts that the module generates.

4.3.2.7.1 Interrupts to the Processors

There are 12 interrupt sources from the MU to the Processors:

- Four receive interrupts (asserted when the Processors receive full bits are set and enabled in the xCR register) for each of the receive registers
- Four transmit interrupts (asserted when the Processor transmit empty bits are set and enabled in the xCR register) for each of the transmit registers
- Four general purpose interrupts (asserted when the GIP bits are set and enabled in the xCR register)

All the interrupts are maskable in the Processor Control Register (xCR). The MU does not assume any internal priority of these interrupts. Multiple interrupts (for example, Receive 0 and Receive 1 interrupts or any of the transmit and general purpose interrupts) can be asserted at one time. The priority of these interrupts should be resolved by the interrupt controller at the chip level.

The General Purpose Interrupt Pending bits (GIP0, GIP1, GIP2, and GIP3) should be cleared by the software (as part of the interrupt service routine) to de-assert the request to the interrupt controller.

4.3.2.7.2 General Purpose Interrupt Clearing Sequence

When a Processor writes to the general interrupt bit (GIR), the write event is synchronized to the other Processor clock to set the general interrupt request pending bit (GIP). When the GIP bit is set, and if the general purpose interrupt is enabled on the transmitting Processor side (GIE bit is set), then the receiving Processor general purpose interrupt is issued to the transmitting Processor. The transmitting Processor clears this interrupt by writing a “1” on the GIP bit. The interrupt is de-asserted as soon as the GIP bit is written. The write event of the GIP bit is synchronized to the other Processor clock. The synchronized signal clears the GIR bit. The software should not write the GIR bit again until the GIR bit is cleared.

4.3.2.8 Interrupt Messaging Protocols

4.3.2.8.1 Messaging Protocols using Interrupts

The example below describes a four-word messaging sequence sent by the Processor to the other Processor.

In this example, the first, second, and third receive interrupts are disabled, and the fourth receive interrupt is enabled. We write registers sequentially for $n = 0, 1, 2, 3$. For $n = 0, 1, 2$, the interrupts are disabled, therefore no interrupt will go to the other core (although interrupt conditions occur). For $n = 3$, the interrupt is enabled, and the last Receive Interrupt request is generated.

1. Write Sequence

- The Processor writes the message information sequentially to its Transmit Registers 0, 1, 2.
- When the write to the Transmit Register 3 occurs, the RF3 bit of the xSR is set after synchronization, and it immediately trigger the Receive 3 interrupt to the other Processor.

2. Read Sequence

- The other Processor receives the Receive 3 interrupt and starts reading the message transferred from the receive registers.
- After Receive Register 3 is read, the interrupt bit is cleared.

Figure 4-8 shows the programmer’s model of a messaging protocol using transmit and receive registers. Use Table 4-8 and Figure 4-8 to understand the generalized protocol sequence.

Table 4-8. Interrupt Messaging Protocol (Generalized)

Sequence	Action	Description
1	Processor A Data write	A data write to the ATRn register by Processor A is immediately reflected in the Processor B BRRn register.
2	Clear Tx Empty bit and Set Rx Full bit	The data write to the ATRn register - Clears the transmitter empty bit (TEn) in the Processor A Transmit Status Register - Sets the receiver full bit (RFn) in the Processor B Receive Status Register
3	Generate Receive Interrupt request	The setting of the receiver full bit (RFn) in the Receive Status Register generates a Receive Interrupt request to Processor B.
4	Processor B Data read	After receiving the Receive Interrupt request, Processor B performs a data read of the BRRn register.
5	Clear Rx Full bit and Set Tx Empty bit	Reading the data out of the BRRn register - Clears the receiver full bit (RFn) in the Processor B Receive Status Register - Sets the transmitter empty bit (TEn) in the Processor A Transmit Status Register
6	Generate Transmit Interrupt request	The setting of the transmitter empty bit (TEn) in the Transmit Status Register generates a Transmit Interrupt request to Processor A.

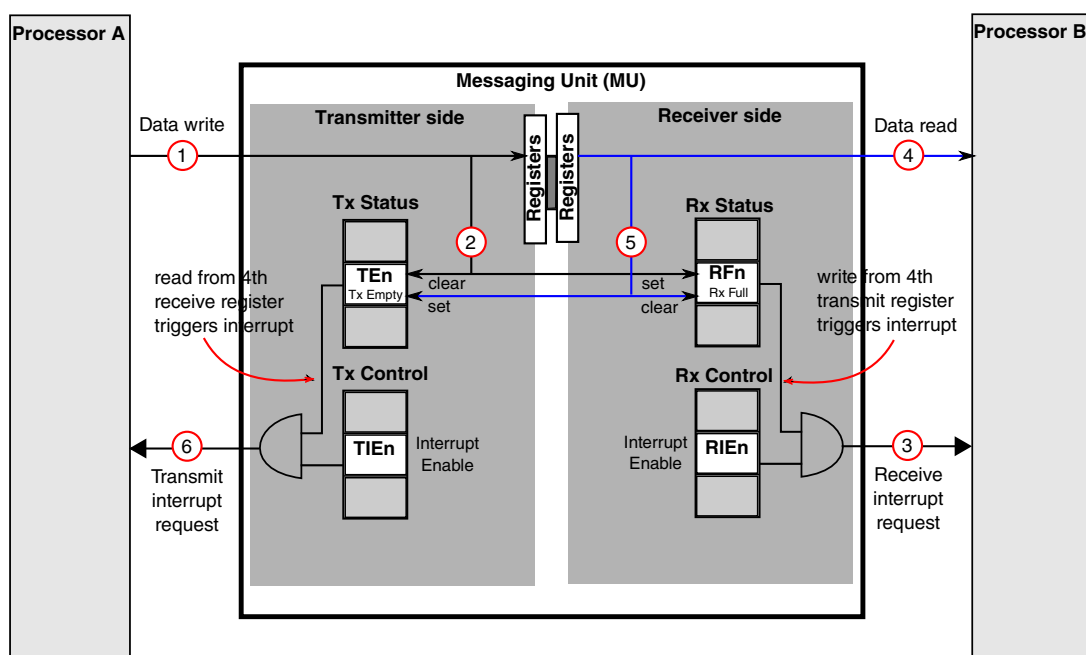


Figure 4-8. Messaging Model Using Transmit and Receive Registers

NOTE

The Transmit registers can be used to pass frame information on long messages written to the shared memory. Such frame

information would typically include an initial address, number of words, and perhaps a message type code.

The messaging hardware can be used by software to implement messaging protocols for a wide array of message types. Full support is given for both interrupt and polling management schemes.

4.3.2.8.2 Messaging Protocols using Event Interrupts

Events and requests that do not include data words can be signaled from the Processor B to the Processor A using the two general interrupts.

Formatted data with a fixed length can be written in predetermined locations in the shared memory. A processor can use a general purpose interrupt to signal the other processor that the data is ready.

The three flags can be used by a processor to announce to the other processor the program state it is currently in, or to announce similar messages.

[Table 4-9](#) and [Figure 4-9](#) describe the event sequence when the Processor triggers an interrupt.

Table 4-9. Interrupt Messaging Protocol (Generalized)

Sequence	Action	Description
1	Processor A sets General Interrupt request bit	Processor A sets its associated General Interrupt request bit (GIRn = 1) in the control register (ACR).
2	General Interrupt Request Pending status bit is set	The General Interrupt Request Pending status bit (GIPn) in the status register (BSR) is set to "1"
3	General Interrupt request to Processor B is generated	Setting the GIPn bit generates the General Interrupt request to Processor B (Interrupt Request Enable bit, GIEn, must be set for Processor B)
4	Processor B reads status register	The Processor B reads the GIPn bit in the BSR register.
5	Processor B services the interrupt	-
6	Processor B sets GIPn bit to clear interrupt	The Processor B writes "1" to the corresponding GIPn bit to clear the interrupt
7	GIRn bit is cleared	Setting the GIPn bit to "1" clears the General Interrupt request bit (GIRn) in the Processor A control register (ACR).

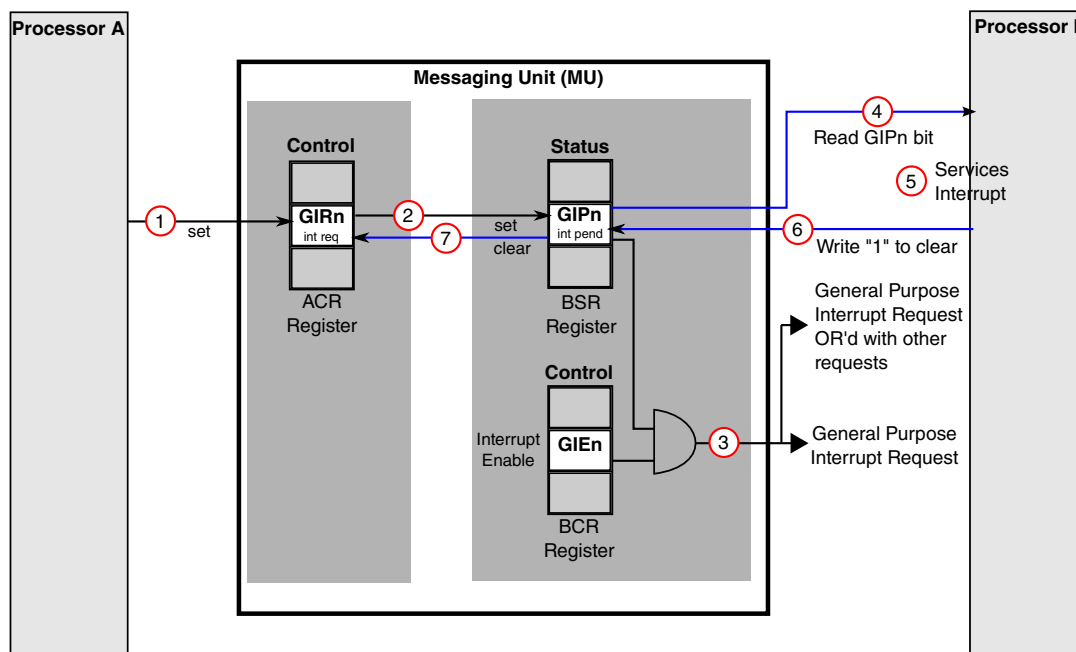


Figure 4-9. Messaging Model Using a General Purpose Interrupt

4.3.2.9 Exclusive Access to Shared Memory

You can use the MU to signal one processor about its current access to the shared memory, so that the data is not overwritten by the other processor during the exclusive memory access period.

The following tables describe the signaling protocol that the Processor A uses to inform the Processor B about its current access (write) to the shared memory, assuming that the set of bits and registers (GIR0 bit, BRR0 register, BTR0 register, GIR0 bit, ARR0 register, ATR0 register) are reserved to support exclusive access to the shared memory protocol.

Table 4-10. How the Processor A Performs an Exclusive Access to Shared Memory

Sequence	Action	Description
1	Processor A sends GIRn request to Processor B using Processor A control register	When the Processor A wants to perform an exclusive access to the shared memory, the Processor A sends an GIR0 request to the Processor B.
2	Processor A sends an exclusive-access request using a transmit data register (ATRn)	The Processor A will send an exclusive-access request (command, location, and length of target access) to Processor B using a selected transmit data register (ATR0).
3	Processor A waits for a dedicated interrupt from Processor B	The Processor A waits for a dedicated interrupt (as an acknowledgement) triggered by the Processor B before proceeding.

Table continues on the next page...

Table 4-10. How the Processor A Performs an Exclusive Access to Shared Memory (continued)

Sequence	Action	Description
4	Processor A accesses shared memory	After receiving a dedicated interrupt from the Processor B, Processor A proceeds.

Table 4-11. How the Processor B Scans for Transaction Information

Sequence	Action	Description
1	Processor B receives an interrupt from a receive data register (BRRn)	-
2	Processor B reads the receive data register (BRRn)	-
3	Processor B scans the receive data register contents	For transaction information (whether Processor A has requested an exclusive-access)

Table 4-12. How the Processor B Accepts Exclusive Access by Processor A

Sequence	Action	Description
1	Processor B triggers a dedicated interrupt	Processor B acknowledges the Processor A request by triggering a dedicated interrupt (ack) to the Processor A.
2	Processor B sends a code message to Processor A	Along with the acknowledge interrupt, the Processor B sends a code message to the Processor A through the selected transmit register (BTRn). The message informs the Processor A that it can exclusively access the shared memory.

Table 4-13. How the Processor B Rejects Exclusive Access by Processor A

Sequence	Action	Description
1	Processor B ignores Processor A request for exclusive access	If the Processor B does not want to give go-ahead permission to the Processor A, Processor B ignores the exclusive access request.

4.3.2.10 Packet Data Transfers

The following example describes the packet transfer sequence between the Processor B and Processor A subsystems:

Table 4-14. Packet Data Transfer Sequence

Action	Sequence	Description
Processor B requests DMA	1	The Processor B sends a DMA request to initiate the packet data transfer

Table continues on the next page...

Table 4-14. Packet Data Transfer Sequence (continued)

Action	Sequence	Description
DMA data transfer	2	DMA acknowledges.
	3	DMA starts transferring data from the specified Processor B location to the specified shared memory
	4	DMA interrupts the Processor B to signal that the packet transfer has finished.
Processor B informs Processor A that data is in shared memory	5	Using an MU Processor B-side transmit register, the Processor B sends a packet information message to the Processor A to inform the Processor A of the arrival of new packet data that is stored in shared memory . The message contains the command, location, and length of packet data information.
Processor A receives interrupt	6	The Processor A receives an interrupt (assuming its corresponding Processor A MU-side receive interrupt is enabled), and the pending processing task becomes active and processes packet data from memory.
Processor A reads data, writes data	7	The Processor A reads or processes packet data from shared memory.
	8	The Processor A writes the result from packet processing to a separate buffer.
Processor A informs Processor B that transfer is finished	9	After the processing of the packet data finishes, the Processor A informs the Processor B (using the MU Processor A-side transmit register, ATRn).
Processor A sends interrupt to Processor B (request for more data)	10	The Processor B receives the next interrupt from the Processor A, in which the Processor A requests more packet data.

4.3.2.11 MU Resets

The MU has two sources of reset, and each reset has a different function from the MU or system perspective.

- One asynchronous system that is connected to both sides of the MU interface.
- One programmable hardware reset (MUR bit) in the ACR register (on the Processor A-side).

Table 4-15. MU programmable resets

Reset	Description
Processor A MU reset	• Processor A MU Reset bit (MUR) of the ACR register • The MUR reset affects the messaging section on both the Processor A and the Processor B sides. The MUR reset causes all control and status registers to return to their default values and all internal states to be cleared. • It is up to the Processor A software to decide whether to use the MUR reset or not. • The instruction immediately following assertion of the MUR bit should not write to MU registers. Such a write may be overwritten by the reset sequence and the register will remain with the reset value. You should wait at least one instruction (after assertion of the MUR bit) before attempting a write to MU registers.

After issuing MUR bit reset events, the Processor A programmer can verify that the reset sequence on the Processor B-side has ended, by checking the RS bit in the ASR register.

NOTE

MUR bit assertion is a delicate operation because it affects the other side's registers asynchronously. MUR bit assertion may cause unpredictable behavior if, for example, the Processor B is concurrently testing an MU register bit (TE bit in Processor B SR register). Before asserting the MUR bit, you should verify that the Processor B is not presently engaged in an MU signalling activity.

4.3.3 Software Restrictions

This section describes certain software restrictions when accessing the MU.

4.3.3.1 General Restrictions

This section lists the restrictions that apply to both the sides (Processor A, Processor B) of the MU.

4.3.3.1.1 Write-After-Write to a Transmit Register

A write to a transmit register signals the receiver side that data is ready for retrieval.

- Writing to the transmit register again without verifying that the data was retrieved is prohibited, because the transmitter side has no way of knowing the exact time that the receiver will attempt to retrieve the data.
- Before attempting to write the transmit register again, the transmitter side should wait for a “Transmitter Empty” interrupt, or should poll the “Transmitter Empty” bit in the Status Register.
- Failure to follow this restriction may result in the wrong data being read on the receiver side of the MU.

4.3.3.1.2 Read-After-Read from a Receive Register

A read of a receive register signals the transmitter side that data can be written to that register. In the same way, the receiver processor should not read a receive register before receiving a “Receiver Full” interrupt or polling the “Receiver Full” bit in the Status Register.

- Reading the receive register again without verifying that the data was written is prohibited, because the receiver side has no way of knowing the exact time that the transmitter will attempt to write the data.
- Before attempting to read the receive register again, the receiver side should wait for a “Receiver Full” interrupt, or should poll the “Receiver Full” bit in the Status Register.
- Failure to follow this restriction may result in the wrong data being written on the transmitter side of the MU.

4.3.3.2 Processor Restrictions

This section lists the restrictions that apply each side of the processor in the MU.

4.3.3.2.1 Before Entering Low Power Mode

Before entering Low Power mode, the Processor should verify that the Processor Event Pending (EP) bit in the Status Register is cleared.

- If the Event Pending bit (EP) is still set to “1”, then the Processor should wait and poll the EP bit until it is cleared, before executing the LPM instruction.
- Note that if the other Processor is in Low Power mode (programmed for clock gating in CCM), the EP bit may be stuck high. In this case, the other Processor clock must be turned ON to get the EP bit cleared before the Processor can enter Low Power mode.
- To discover which power mode the other Processor is in, the Processor can check the PM bits in the xSR register.

4.3.3.2.2 Before Setting a General Interrupt Request Bit (GIR0–3)

Before setting a General Interrupt Request bit (GIR0–3), you must verify that the GIR_n bit is cleared, which means that a general interrupt is not pending. Generally, setting the GIR_n bit while the bit is set to “1” will be ignored, but in some cases it may issue a second interrupt. This restriction is meant to prevent this indeterministic behavior.

4.3.3.2.3 Reset Bit Restrictions

The reset bit (MUR, HR) restrictions are:

- Before asserting the MUR bit in the ACR register, verify that the Processor B-side is not engaged in some MU activity.
- Do not write to an MU register in the instruction immediately after the assertion of the MUR bit in the ACR register, because the written data can be overridden by the reset value.

4.3.4 MU Processor A-side Memory Map/Register Definition

This section contains the detailed register descriptions for the Processor A-side MU registers.

MUA memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
30AA_0000	Processor A Transmit Register 0 (MUA_ATR0)	32	R/W	0000_0000h	4.3.4.1/133
30AA_0004	Processor A Transmit Register 1 (MUA_ATR1)	32	R/W	0000_0000h	4.3.4.2/134
30AA_0008	Processor A Transmit Register 2 (MUA_ATR2)	32	R/W	0000_0000h	4.3.4.3/135
30AA_000C	Processor A Transmit Register 3 (MUA_ATR3)	32	R/W	0000_0000h	4.3.4.4/135
30AA_0010	Processor A Receive Register 0 (MUA_ARR0)	32	R	0000_0000h	4.3.4.5/136
30AA_0014	Processor A Receive Register 1 (MUA_ARR1)	32	R	0000_0000h	4.3.4.6/137
30AA_0018	Processor A Receive Register 2 (MUA_ARR2)	32	R	0000_0000h	4.3.4.7/137
30AA_001C	Processor A Receive Register 3 (MUA_ARR3)	32	R	0000_0000h	4.3.4.8/138
30AA_0020	Processor A Status Register (MUA_ASR)	32	R/W	00F0_0080h	4.3.4.9/139
30AA_0024	Processor A Control Register (MUA_ACR)	32	R/W	0000_0000h	4.3.4.10/142

4.3.4.1 Processor A Transmit Register 0 (MUA_ATR0)

Use Processor A Transmit Register 0 (ATR0, 32-bit, write-only) to transmit a message or data to the Processor B.

- You can only write to the ATR0 register when the TE0 bit in ASR register is set to “1”.
- Reading the ATR0 register returns all zeros.

Address: 30AA_0000h base + 0h offset = 30AA_0000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

MUA_ATR0 field descriptions

Field	Description
ATR0	Processor A Transmit Register 0. (Write-only)

MUA_ATR0 field descriptions (continued)

Field	Description
	- Data written to the ATR0 register is reflected on the Processor B-side in the Processor B Receive Register 0 (BRR0). The ATR0 and BRR0 registers are not double-buffered—a write to the ATR0 register overrides the data readable at the BRR0 register. - A write to the transmit register clears a “transmitter empty” bit (TE0) in the Processor A Status Register (ASR) on the transmitter side, and sets a “receiver full” bit (RF0) in the Processor B Status Register (BSR) on the receiver side (optionally triggering an interrupt 0 on the Processor B-side). - Any write to the ATR0 register will update all status information.

4.3.4.2 Processor A Transmit Register 1 (MUA_ATR1)

Use Processor A Transmit Register 1 (ATR1, 32-bit, write-only) to transmit a message or data to the Processor B.

- You can only write to the ATR1 register when the TE1 bit in ASR register is set to “1”.
- Reading the ATR1 register returns all zeros.

Address: 30AA_0000h base + 4h offset = 30AA_0004h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
	ATR1																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

MUA_ATR1 field descriptions

Field	Description
ATR1	Processor A Transmit Register 1. (Write-only) - Data written to the ATR1 register is reflected on the Processor B-side in the Processor B Receive Register 1 (BRR1). The ATR1 and BRR1 registers are not double-buffered—a write to the ATR1 register overrides the data readable at the BRR1 register. - A write to the transmit register clears a “transmitter empty” bit (TE1) in the Processor A Status Register (ASR) on the transmitter side, and sets a “receiver full” bit (RF1) in the Processor B Status Register (BSR) on the receiver side (optionally triggering an interrupt 1 on the Processor B-side). - Any write to the ATR1 register will update all status information.

4.3.4.3 Processor A Transmit Register 2 (MUA_ATR2)

Use Processor A Transmit Register 2 (ATR2, 32-bit, write-only) to transmit a message or data to the Processor B.

- You can only write to the ATR2 register when the TE2 bit in ASR register is set to “1”.
- Reading the ATR2 register returns all zeros.

Address: 30AA_0000h base + 8h offset = 30AA_0008h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
	ATR2																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

MUA_ATR2 field descriptions

Field	Description
ATR2	Processor A Transmit Register 2. (Write-only) • Data written to the ATR2 register is reflected on the Processor B-side in the Processor B Receive Register 2 (BRR2). The ATR2 and BRR2 registers are not double-buffered—a write to the ATR2 register overrides the data readable at the BRR2 register. • A write to the transmit register clears a “transmitter empty” bit (TE2) in the Processor A Status Register (ASR) on the transmitter side, and sets a “receiver full” bit (RF2) in the Processor B Status Register (BSR) on the receiver side (optionally triggering an interrupt 2 on the Processor B-side). • Any write to the ATR2 register will update all status information.

4.3.4.4 Processor A Transmit Register 3 (MUA_ATR3)

Use Processor A Transmit Register 3 (ATR3, 32-bit, write-only) to transmit a message or data to the Processor B.

- You can only write to the ATR3 register when the TE3 bit in ASR register is set to “1”.
- Reading the ATR3 register returns all zeros.

Address: 30AA_0000h base + Ch offset = 30AA_000Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
	ATR3																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

MUA_ATR3 field descriptions

Field	Description
ATR3	Processor A Transmit Register 3. (Write-only) - Data written to the ATR3 register is reflected on the Processor B-side in the Processor B Receive Register 3 (BRR3). The ATR3 and BRR3 registers are not double-buffered—a write to the ATR3 register overrides the data readable at the BRR3 register. - A write to the transmit register clears a “transmitter empty” bit (TE3) in the Processor A Status Register (ASR) on the transmitter side, and sets a “receiver full” bit (RF3) in the Processor B Status Register (BSR) on the receiver side (optionally triggering an interrupt 3 on the Processor B-side). - Any write to the ATR3 register will update all status information.

4.3.4.5 Processor A Receive Register 0 (MUA_ARR0)

Use Processor A Receive Register 0 (ARR0, 32-bit, read-only) to receive a message or data from the Processor B.

- Data written to the BTR0 register is immediately reflected in the ARR0 register.
- You can only read the ARR0 register when the RF0 bit in the ASR register is set to “1”.
- Writing to the ARR0 register generates an error response to the Processor A.

Address: 30AA_0000h base + 10h offset = 30AA_0010h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ARR0																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

MUA_ARR0 field descriptions

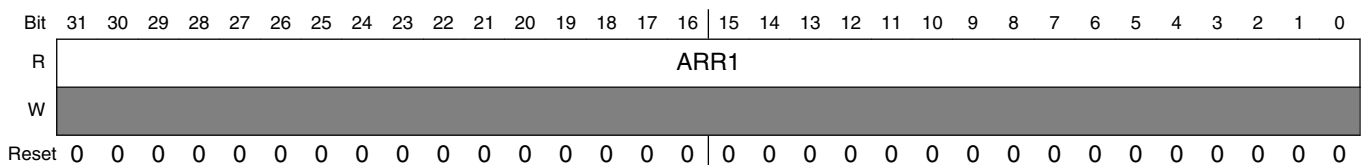
Field	Description
ARR0	Processor A Receive Register 0. (Read-only) - Reflects the data written to Processor B Transmit Register 0 (BTR0). - Reading the ARR0 register clears the “receiver full” bit (RF0) in the Processor A Status Register (ASR) on the receiver side, and sets the “transmitter empty” bit (TE0) in the Processor B Status Register on the transmitter side (optionally triggering a transmit interrupt 0 on the Processor B-side). - Any read of the ARR0 register will update all status information.

4.3.4.6 Processor A Receive Register 1 (MUA_ARR1)

Use Processor A Receive Register 1 (ARR1, 32-bit, read-only) to receive a message or data from the Processor B.

- Data written to the BTR1 register is immediately reflected in the ARR1 register.
- You can only read the ARR1 register when the RF1 bit in the ASR register is set to “1”.
- Writing to the ARR1 register generates an error response to the Processor A.

Address: 30AA_0000h base + 14h offset = 30AA_0014h



MUA_ARR1 field descriptions

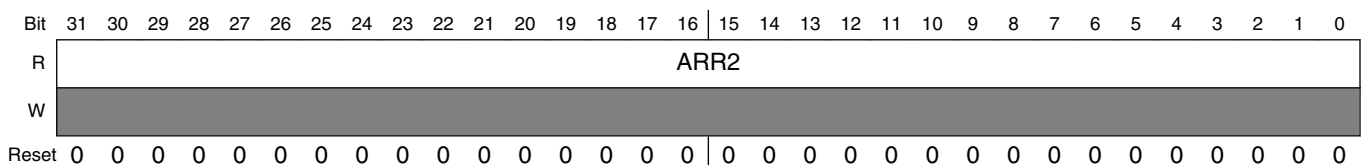
Field	Description
ARR1	Processor A Receive Register 1. (Read-only) • Reflects the data written to Processor B Transmit Register 1 (BTR1). • Reading the ARR1 register clears the “receiver full” bit (RF1) in the Processor A Status Register (ASR) on the receiver side, and sets the “transmitter empty” bit (TE1) in the Processor B Status Register on the transmitter side (optionally triggering a transmit interrupt 1 on the Processor B-side). • Any read of the ARR1 register will update all status information.

4.3.4.7 Processor A Receive Register 2 (MUA_ARR2)

Use Processor A Receive Register 2 (ARR2, 32-bit, read-only) to receive a message or data from the Processor B.

- Data written to the BTR2 register is immediately reflected in the ARR2 register.
- You can only read the ARR2 register when the RF2 bit in the ASR register is set to “1”.
- Writing to the ARR2 register generates an error response to the Processor A.

Address: 30AA_0000h base + 18h offset = 30AA_0018h



MUA_ARR2 field descriptions

Field	Description
ARR2	Processor A Receive Register 2. (Read-only) - Reflects the data written to Processor B Transmit Register 1 (BTR2). - Reading the ARR2 register clears the “receiver full” bit (RF2) in the Processor A Status Register (ASR) on the receiver side, and sets the “transmitter empty” bit (TE2) in the Processor B Status Register on the transmitter side (optionally triggering a transmit interrupt 2 on the Processor B-side). - Any read of the ARR2 register will update all status information.

4.3.4.8 Processor A Receive Register 3 (MUA_ARR3)

Use Processor A Receive Register 3 (ARR3, 32-bit, read-only) to receive a message or data from the Processor B.

- Data written to the BTR3 register is immediately reflected in the ARR3 register.
- You can only read the ARR3 register when the RF3 bit in the ASR register is set to “1”.
- Writing to the ARR3 register generates an error response to the Processor A.

Address: 30AA_0000h base + 1Ch offset = 30AA_001Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ARR3																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

MUA_ARR3 field descriptions

Field	Description
ARR3	Processor A Receive Register 3. (Read-only) - Reflects the data written to Processor B Transmit Register 3 (BTR3). - Reading the ARR3 register clears the “receiver full” bit (RF3) in the Processor A Status Register (ASR) on the receiver side, and sets the “transmitter empty” bit (TE3) in the Processor B Status Register on the transmitter side (optionally triggering a transmit interrupt 3 on the Processor B-side). - Any read of the ARR3 register will update all status information.

4.3.4.9 Processor A Status Register (MUA_ASR)

Use the Processor A Status Register (ASR, 32-bit, read-write) to show interrupt status from the Processor B, general purpose flags, and to set dual function control-status bits.

- Some dual-purpose bits are set by the MU logic, and cleared by the Processor A-side programmer
- Other dual-purpose bits are set by the Processor A-side programmer, and cleared by the MU logic.

Address: 30AA_0000h base + 20h offset = 30AA_0020h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	GIPn				RFn				TEn				Reserved			
W																
Reset	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								BRDIP	FUP	BRS	Reserved		EP	Reserved	Fn
W																
Reset	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0

MUA_ASR field descriptions

Field	Description
31–28 GIPn	For $n = \{0, 1, 2, 3\}$ Processor A General Interrupt Request n Pending. (Read-Write) • GIPn bit signals the Processor A that the GIPn bit in the BCR register on the Processor B-side was set from “0” to “1”. If the GIPn bit in the ACR register is set to “1”, a General Interrupt n request is issued. • The GIPn bit is cleared by writing it back as “1”. Writing “0”, or writing “1” when the GIPn bit is cleared is ignored. Use this feature in the interrupt routine, where the GIPn bit is cleared in order to de-assert the interrupt request source at the interrupt controller. The proper bit clearing sequence is: clear an Processor A register, set the desired bit in it (Processor A register), and write it to the ASR register, thus clearing the GIPn bit. • GIPn bit is cleared when the MU is reset.

Table continues on the next page...

MUA_ASR field descriptions (continued)

Field	Description
	0 Processor A general purpose interrupt n is not pending. (default) 1 Processor A general purpose interrupt n is pending.
27–24 RFn	For n = {0, 1, 2, 3} Processor A Receive Register n Full. (Read-only) - The RFn bit is set to “1” when the BTRn register is written on the Processor B-side. - After the RFn bit is set to “1”, the RFn bit signals the Processor A-side that new data is ready to be read by the Processor A in the ARRn register, and a Receive n interrupt is issued on the Processor A-side (if the RIEn bit in the ACR register has been set to “1”). - RFn bit is cleared when the ARRn register is read, and when the MU is reset. 0 ARRn register is not full (default). 1 ARRn register has received data from BTRn register and is ready to be read by the Processor A.
23–20 TEn	For n = {0, 1, 2, 3} Processor A Transmit Register n Empty. (Read-only) - The TEn bit is set to “1” after the BRRn register is read on the Processor B-side. - After the TEn bit is set to “1”, the TEn bit signals the Processor A-side that the ATRn register is ready to be written on the Processor A-side, and a Transmit n interrupt is issued on the Processor A-side (if the TEn bit in the ACR register is set to “1”). - TEn bit is cleared after the ATRn register is written on the Processor A-side. - TEn bit is set to “1” when the MU is reset. 0 ATRn register is not empty. 1 ATRn register is empty (default).
19–10 -	This field is reserved. Reserved.
9 BRDIP	Processor B Reset De-asserted Interrupt Pending. (Read-Write) - BRDIP bit signals the Processor A-side that the Processor B-side has come out of reset. - BRDIP bit is set to “1” after the MU Processor B-side comes out of reset, after synchronization. The interrupt generated by a Processor B-side reset de-assertion is ORed with the Processor A general purpose interrupt 3. The Processor A general purpose interrupt 3 is issued when the Processor B-side comes out of reset (if the interrupt is enabled). - To clear the BRDIP bit, write “1”, which also clears general purpose interrupt 3. - When Processor A-side of MU comes out of reset BRDIP bit has value “0”(default). Then Processor A sees the status of Processor B-side and if Processor B-side has come out of reset then BRDIP bit goes high. This takes 5-6 clock cycles. And if you read BRDIP bit now you will see it as high although its reset value was “0”. 0 The Processor A general purpose interrupt 3, because of a Processor B-side reset de-assertion, is cleared (default). 1 The Processor B-side is out of reset.
8 FUP	Processor A Flags Update Pending. (Read-only) - FUP bit is set to “1” when the Processor A-side sends a Flags Update request to the Processor B-side. - A Flags Update request is generated when the ABF[2:0] bits of the ACR register change. No flag update changes are allowed while the FUP bit is set to “1”. Any write to the ABF[2:0] bits, while the FUP bit is set to “1”, will not generate a Flags Update event, and the ABF[2:0] bits will stay unchanged. - FUP bit is cleared when this Flags Update request is internally acknowledged (that the flag is updated) from the MU Processor B-side, and during MU reset.

Table continues on the next page...

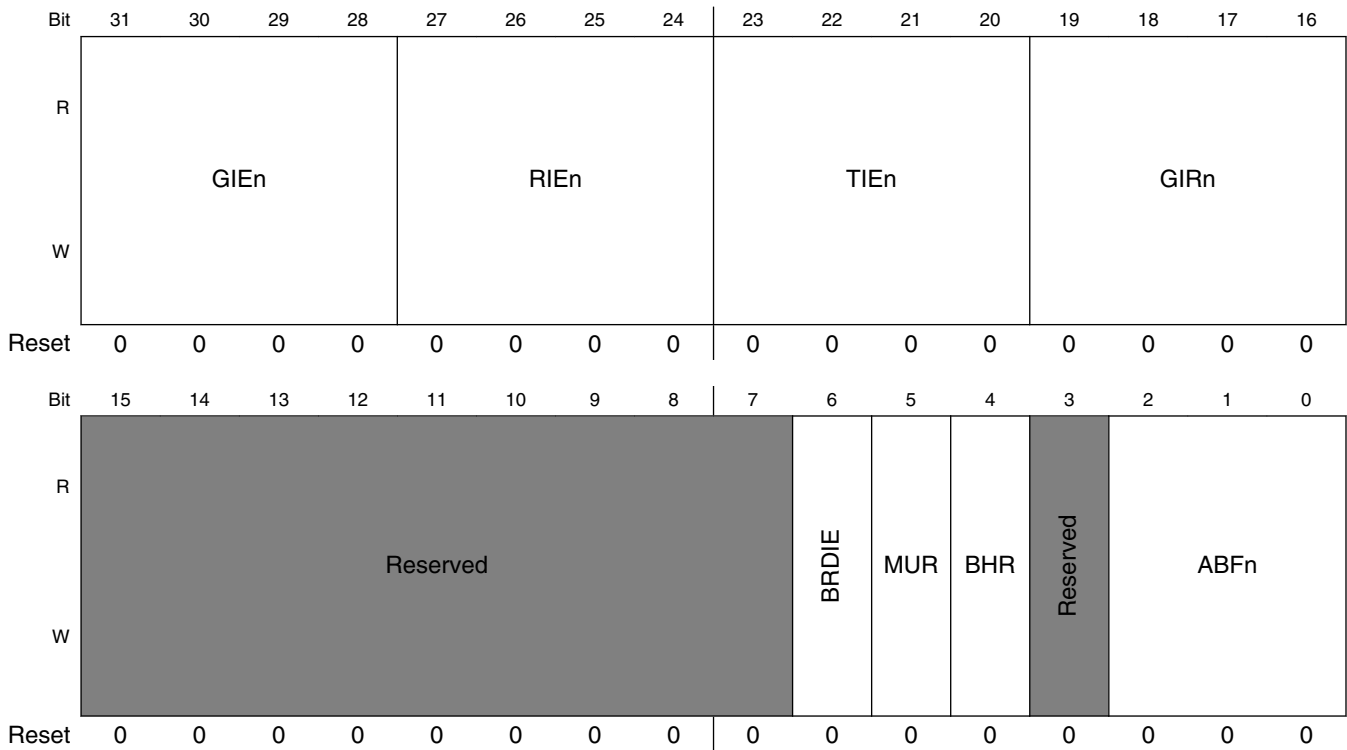
MUA_ASR field descriptions (continued)

Field	Description
	0 No flags updated, initiated by the Processor A, in progress (default) 1 Processor A initiated flags update, processing
7 BRS	Processor B-side Reset State. (Read-only) • BRS bit indicates if the Processor B-side of the MU is in a reset state or not. • If the BRS bit is set to “1”, then the Processor B-side of the MU is still in the reset state. • If the BRS bit is cleared, then the Processor B-side of the MU are out of reset. • The BRS bit is set to “1” during: a Processor B system reset, or an MU reset (caused by setting the MUR bit at the ACR register). • The BRS bit is cleared when the reset sequence on the Processor B-side of the MU ends. After issuing any of the reset events mentioned previously, you should verify that the BRS bit is cleared before starting any accesses. • When Processor A side of MU comes out of reset BRS bit has value “1”(default). Then Processor A sees the status of Processor B-side and if Processor B-side has come out of reset then BRS bit goes low. This takes 5-6 clock cycles. And if you read BRS bit now you will see it as low although its reset value was “1”. 0 The Processor B-side of the MU is not in reset. 1 The Processor B-side of the MU is in reset.
6–5 -	This field is reserved. Reserved
4 EP	Processor A-Side Event Pending. (Read-only) • EP bit is set to “1” when the Processor A-side mechanism sends an event update request to the Processor B-side. • EP bit is cleared when the event update acknowledge is received. An “event” is any hardware message that is reflected in the BSR register on the Processor B-side (for example, “transmit register 0 written”). During normal operations, you do not have to deal with the state of the EP bit because the event update mechanism works automatically. • To ensure events have been posted to Processor B before entering STOP mode, you should verify that the EP bit is cleared. If EP bit is set to “1”, you should wait and continue to poll it (EP bit) before entering STOP mode. • Reading the ASR register (to check the EP bit) should be the last access to the MU that should be performed before entering STOP or WAIT modes; otherwise, the EP bit may be set by subsequent additional actions. • The EP bit is cleared when the MU resets. 0 The Processor A-side event is not pending (default). 1 The Processor A-side event is pending.
3 -	This field is reserved. Reserved.
Fn	For $n = \{0, 1, 2\}$ Processor A-Side Flag n. (Read-only) • Fn bit is the Processor A-side flag that reflects the values written to the BAFn bit in the Processor B control register. • Every time that the BAFn bit is written, the BAFn bit write event updates the Fn bit after the event update latency, which is measured in terms of the number of clocks of the Processor B and the Processor A. 0 BAFn bit in BCR register is written 0 (default). 1 BAFn bit in BCR register is written 1.

4.3.4.10 Processor A Control Register (MUA_ACR)

Use the Processor A Control Register (ACR, 32-bit, read-write) to enable the MU interrupts on the Processor A-side, and trigger events and interrupts on the Processor B-side (general purpose interrupt, flag update).

Address: 30AA_0000h base + 24h offset = 30AA_0024h



MUA_ACR field descriptions

Field	Description
31–28 GIE _n	For n = {0, 1, 2, 3} Processor A General Purpose Interrupt Enable n. (Read-Write) When GIE_n=0 corresponds to the high order bit and GIE₃ corresponds to the low order bit. - GIE_n bit enables Processor A General Interrupt n. - If GIE_n bit is set to “1” (enabled), then a General Interrupt n request is issued when the GIP_n bit in the ASR register is set to “1”. - If GIE_n is cleared (disabled), then the value of the GIP_n bit is ignored and no General Interrupt n request will be issued. - GIE_n bit is cleared when the MU resets. 0 Disables Processor A General Interrupt n. (default) 1 Enables Processor A General Interrupt n.
27–24 RIE _n	For n = {0, 1, 2, 3} Processor A Receive Interrupt Enable n. (Read-Write)

Table continues on the next page...

MUA_ACR field descriptions (continued)

Field	Description
	- RIEn bit enables Processor A Receive Interrupt n. - If RIEn bit is set to “1” (enabled), then an Processor A Receive Interrupt n request is issued when the RFn bit in the ASR register is set to “1”. - If RIEn bit is cleared (disabled), then the value of the RFn bit is ignored and no Processor A Receive Interrupt n request will be issued. - RIEn bit is cleared when the MU resets. 0 Disables Processor A Receive Interrupt n. (default) 1 Enables Processor A Receive Interrupt n.
23–20 TIE _n	For n = {0, 1, 2, 3} Processor A Transmit Interrupt Enable n. (Read-Write) - TIE_n bit enables Processor A Transmit Interrupt n. - If TIE_n bit is set to “1” (enabled), then an Processor A Transmit Interrupt n request is issued when the TEn bit in the ASR register is set to “1”. - If TIE_n bit is cleared (disabled), then the value of the TEn bit is ignored and no Processor A Transmit Interrupt n request will be issued. - TIE_n bit is cleared when the MU resets. 0 Disables Processor A Transmit Interrupt n. (default) 1 Enables Processor A Transmit Interrupt n.
19–16 GIR _n	For n = {0, 1, 2, 3} Processor A General Purpose Interrupt Request n. (Read-Write) - Writing “1” to the GIR_n bit sets the GIP_n bit in the BSR register on the Processor B-side. If the GIE_n bit in the BCR register is set to “1” on the Processor B-side, a General Purpose Interrupt n request is triggered. - The GIR_n bit is cleared if the GIP_n bit (in the BSR register on the Processor B-side) is cleared by writing it (GIP_n bit) as “1”, thereby signalling the Processor A that the interrupt was accepted (cleared by the software). The GIP_n bit cannot be written as “0” on the Processor A-side. - To ensure proper operations, you must verify that the GIR_n bit is cleared (meaning that there is no pending interrupt) before setting it (GIR_n bit). - GIR_n bit is cleared when the MU resets. 0 Processor A General Interrupt n is not requested to the Processor B (default). 1 Processor A General Interrupt n is requested to the Processor B.
15–7 -	This field is reserved. Reserved.
6 BRDIE	Processor B Reset De-assertion Interrupt Enable. (Read-Write) - BRDIE bit enables Processor A General Interrupt 3. - If BRDIE bit is set to “1”, then General Interrupt 3 request is issued to the Processor A when the BRDIP bit in the ASR register is set to “1”. - If BRDIE is cleared, then the value of the BRDIP bit is ignored and no General Interrupt 3 request will be issued. - The BRDIE bit is cleared when the MU resets. 0 Disables the Processor A General Purpose Interrupt 3 request due to the Processor B reset de-assertion to the Processor A. Processor B reset deassertion causes Processor B and MU-Processor B side to come out of reset thus setting BRDIP bit to “1”. 1 Enables Processor A General Purpose Interrupt 3 request due to the Processor B reset de-assertion to the Processor A.
5 MUR	Processor A MU Reset.

Table continues on the next page...

MUA_ACR field descriptions (continued)

Field	Description
	- Setting MUR bit to “1” resets both the Processor B and the Processor A sides of the MU module, forcing all control and status registers to return to their default values (except the BHR bit in the ACR register and BHRM bit in BCR register), and all internal states to be cleared. - Before setting the MUR bit to “1”, it is advisable to interrupt the Processor B, because setting the MUR bit may affect the ongoing Processor B program. - After setting the MUR bit, you should monitor the value of the BRS bit in the ASR register to know when the reset sequence on the Processor B-side has ended. - MUR bit can only be written as “1”. - MUR bit is always read as “0”. - MUR bit is cleared during the MU reset sequence. 0 N/A. Self clearing bit (default). 1 Asserts the Processor A MU reset.
4 BHR	Processor B Hardware Reset. (Read-Write) - BHR bit asserts and de-asserts the hardware reset of the Processor B. - Set BHR bit to “1” to start a hardware reset of the Processor B. - Clear the BHR bit to de-assert the Processor B hardware reset input. - Assert the BHR bit for a minimum of 3 clock cycles of network clock (sampling clock in SRC) clock. The BRS bit in MU_ASR register (b[7]) indicates the state of the Processor B. As soon as the Processor B goes into Reset (BRS bit is set to “1”), the BHR bit can be de-asserted. - Strobe-setting the BHR bit will not cause an internal MU reset but will be routed outside MU to Processor B domain reset logic - After clearing the BHR bit, monitor the value of the BRS bit at the ASR to know when the Processor B reset sequence has ended. - The BHR reset issued by the Processor A to the Processor B is maskable by the Processor B (according to the settings of the BHRM bit in the BCR register). If the BHRM bit (in the BCR register) is set to “1”, then the BHR reset is masked; if the BHRM bit (in the BCR register) is cleared (default), then the BHR reset is enabled. - The BHR bit does not return to the reset value during the software (MUR) reset. 0 De-assert Hardware reset to the Processor B. (default) 1 Assert Hardware reset to the Processor B.
3 -	This field is reserved. Reserved
ABFn	For n = {0, 1, 2} Processor A to Processor B Flag n. (Read-Write) - ABFn bit is a read-write flag that is reflected in Fn bit in the BSR register on the Processor B-side. - ABFn bit is cleared when the MU resets. 0 N/A. Self clearing bit (default). 1 Asserts the Processor A MU reset.

4.3.5 MU Processor B-side Memory Map/Register Definition

This section contains the detailed register descriptions for the Processor B-side MU registers.

MUB memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
30AB_0000	Processor B Transmit Register 0 (MUB_BTR0)	32	R/W	0000_0000h	4.3.5.1/145
30AB_0004	Processor B Transmit Register 1 (MUB_BTR1)	32	R/W	0000_0000h	4.3.5.2/146
30AB_0008	Processor B Transmit Register 2 (MUB_BTR2)	32	R/W	0000_0000h	4.3.5.3/146
30AB_000C	Processor B Transmit Register 3 (MUB_BTR3)	32	R/W	0000_0000h	4.3.5.4/147
30AB_0010	Processor B Receive Register 0 (MUB_BRR0)	32	R	0000_0000h	4.3.5.5/148
30AB_0014	Processor B Receive Register 1 (MUB_BRR1)	32	R	0000_0000h	4.3.5.6/148
30AB_0018	Processor B Receive Register 2 (MUB_BRR2)	32	R	0000_0000h	4.3.5.7/149
30AB_001C	Processor B Receive Register 3 (MUB_BRR3)	32	R	0000_0000h	4.3.5.8/150
30AB_0020	Processor B Status Register (MUB_BSR)	32	R/W	00F0_0080h	4.3.5.9/151
30AB_0024	Processor B Control Register (MUB_BCR)	32	R/W	0000_0000h	4.3.5.10/154

4.3.5.1 Processor B Transmit Register 0 (MUB_BTR0)

Use Processor B Transmit Register 0 (BTR0, 32-bit, write-only) to transmit a message or data to the Processor A.

- You can only write to the BTR0 register when the TE0 bit in BSR register is set to “1”.
- Reading the BTR0 register returns all zeros.

Address: 30AB_0000h base + 0h offset = 30AB_0000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

MUB_BTR0 field descriptions

Field	Description
BTR0	Processor B Transmit Register 0. (Write-only) • Data written to the BTR0 register is reflected on the Processor A-side in the Processor A Receive Register 0 (ARR0). The BTR0 and ARR0 registers are not double-buffered—a write to the BTR0 register overrides the data readable at the ARR0 register. • A write to the transmit register clears a “transmitter empty” bit (TE0) in the Processor B Status Register (BSR) on the transmitter side, and sets a “receiver full” bit (RF0) in the Processor A Status Register (ASR) on the receiver side (optionally triggering an interrupt 0 on the Processor A-side). • Any write to the BTR0 register will update all status information.

4.3.5.2 Processor B Transmit Register 1 (MUB_BTR1)

Use Processor B Transmit Register 1 (BTR1, 32-bit, write-only) to transmit a message or data to the Processor A.

- You can only write to the BTR1 register when the TE1 bit in BSR register is set to “1”.
- Reading the BTR1 register returns all zeros.

Address: 30AB_0000h base + 4h offset = 30AB_0004h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

MUB_BTR1 field descriptions

Field	Description
BTR1	Processor B Transmit Register 1. (Write-only) • Data written to the BTR1 register is reflected on the Processor A-side in the Processor A Receive Register 1 (ARR1). The BTR1 and ARR1 registers are not double-buffered—a write to the BTR1 register overrides the data readable at the ARR1 register. • A write to the transmit register clears a “transmitter empty” bit (TE1) in the Processor B Status Register (BSR) on the transmitter side, and sets a “receiver full” bit (RF1) in the Processor A Status Register (ASR) on the receiver side (optionally triggering an interrupt 1 on the Processor A-side). • Any write to the BTR1 register will update all status information.

4.3.5.3 Processor B Transmit Register 2 (MUB_BTR2)

Use Processor B Transmit Register 2 (BTR2, 32-bit, write-only) to transmit a message or data to the Processor A.

- You can only write to the BTR2 register when the TE2 bit in BSR register is set to “1”.
- Reading the BTR2 register returns all zeros.

Address: 30AB_0000h base + 8h offset = 30AB_0008h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

MUB_BTR2 field descriptions

Field	Description
BTR2	Processor B Transmit Register 2. (Write-only) - Data written to the BTR2 register is reflected on the Processor A-side in the Processor A Receive Register 2 (ARR2). The BTR2 and ARR2 registers are not double-buffered—a write to the BTR2 register overrides the data readable at the ARR2 register. - A write to the transmit register clears a “transmitter empty” bit (TE2) in the Processor B Status Register (BSR) on the transmitter side, and sets a “receiver full” bit (RF2) in the Processor A Status Register (ASR) on the receiver side (optionally triggering an interrupt 2 on the Processor A-side). - Any write to the BTR2 register will update all status information.

4.3.5.4 Processor B Transmit Register 3 (MUB_BTR3)

Use Processor B Transmit Register 3 (BTR3, 32-bit, write-only) to transmit a message or data to the Processor A.

- You can only write to the BTR3 register when the TE3 bit in BSR register is set to “1”.
- Reading the BTR3 register returns all zeros.

Address: 30AB_0000h base + Ch offset = 30AB_000Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BTR3																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

MUB_BTR3 field descriptions

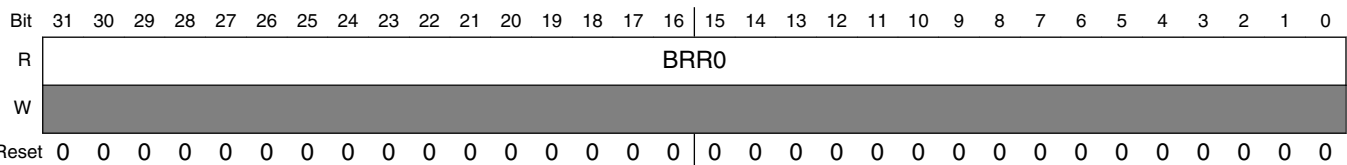
Field	Description
BTR3	Processor B Transmit Register 3. (Write-only) - Data written to the BTR3 register is reflected on the Processor A-side in the Processor A Receive Register 3 (ARR3). The BTR3 and ARR3 registers are not double-buffered—a write to the BTR3 register overrides the data readable at the ARR3 register. - A write to the transmit register clears a “transmitter empty” bit (TE3) in the Processor B Status Register (BSR) on the transmitter side, and sets a “receiver full” bit (RF3) in the Processor A Status Register (ASR) on the receiver side (optionally triggering an interrupt 3 on the Processor A-side). - Any write to the BTR3 register will update all status information.

4.3.5.5 Processor B Receive Register 0 (MUB_BRR0)

Use Processor B Receive Register 0 (BRR0, 32-bit, read-only) to receive a message or data from the Processor A.

- Data written to the ATR0 register is immediately reflected in the BRR0 register.
- You can only read the BRR0 register when the RF0 bit in the BSR register is set to “1”.
- Writing to the BRR0 register generates an error response to the Processor B.

Address: 30AB_0000h base + 10h offset = 30AB_0010h



MUB_BRR0 field descriptions

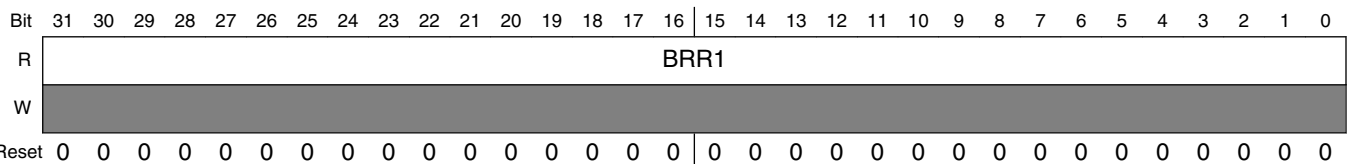
Field	Description
BRR0	Processor B Receive Register 0. (Read-only) • Reflects the data written to Processor A Transmit Register 0 (ATR0). • Reading the BRR0 register clears the “receiver full” bit (RF0) in the Processor B Status Register (BSR) on the receiver side, and sets the “transmitter empty” bit (TE0) in the Processor A Status Register on the transmitter side (optionally triggering a transmit interrupt 0 on the Processor A-side). • Any read of the BRR0 register will update all status information.

4.3.5.6 Processor B Receive Register 1 (MUB_BRR1)

Use Processor B Receive Register 1 (BRR1, 32-bit, read-only) to receive a message or data from the Processor A.

- Data written to the ATR1 register is immediately reflected in the BRR1 register.
- You can only read the BRR1 register when the RF1 bit in the BSR register is set to “1”.
- Writing to the BRR1 register generates an error response to the Processor B.

Address: 30AB_0000h base + 14h offset = 30AB_0014h



MUB_BRR1 field descriptions

Field	Description
BRR1	Processor B Receive Register 1. (Read-only) - Reflects the data written to Processor A Transmit Register 1 (ATR1). - Reading the BRR1 register clears the “receiver full” bit (RF1) in the Processor B Status Register (BSR) on the receiver side, and sets the “transmitter empty” bit (TE1) in the Processor A Status Register on the transmitter side (optionally triggering a transmit interrupt 1 on the Processor A-side). - Any read of the BRR1 register will update all status information.

4.3.5.7 Processor B Receive Register 2 (MUB_BRR2)

Use Processor B Receive Register 2 (BRR2, 32-bit, read-only) to receive a message or data from the Processor A.

- Data written to the ATR2 register is immediately reflected in the BRR2 register.
- You can only read the BRR2 register when the RF2 bit in the BSR register is set to “1”.
- Writing to the BRR2 register generates an error response to the Processor B.

Address: 30AB_0000h base + 18h offset = 30AB_0018h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BRR2																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

MUB_BRR2 field descriptions

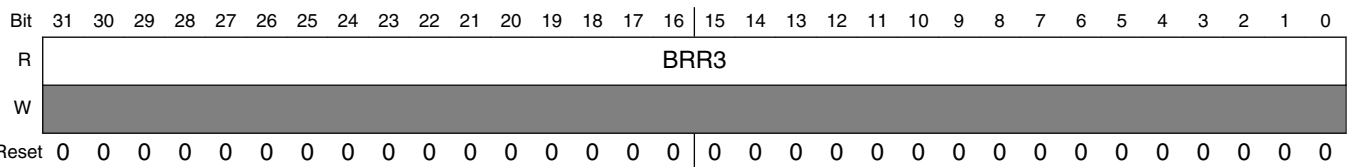
Field	Description
BRR2	Processor B Receive Register 2. (Read-only) - Reflects the data written to Processor A Transmit Register 1 (ATR2). - Reading the BRR2 register clears the “receiver full” bit (RF2) in the Processor B Status Register (BSR) on the receiver side, and sets the “transmitter empty” bit (TE2) in the Processor A Status Register on the transmitter side (optionally triggering a transmit interrupt 2 on the Processor A-side). - Any read of the BRR2 register will update all status information.

4.3.5.8 Processor B Receive Register 3 (MUB_BRR3)

Use Processor B Receive Register 3 (BRR3, 32-bit, read-only) to receive a message or data from the Processor A.

- Data written to the ATR3 register is immediately reflected in the BRR3 register.
- You can only read the BRR3 register when the RF3 bit in the BSR register is set to “1”.
- Writing to the BRR3 register generates an error response to the Processor B.

Address: 30AB_0000h base + 1Ch offset = 30AB_001Ch



MUB_BRR3 field descriptions

Field	Description
BRR3	Processor B Receive Register 3. (Read-only) • Reflects the data written to Processor A Transmit Register 3 (ATR3). • Reading the BRR3 register clears the “receiver full” bit (RF3) in the Processor B Status Register (BSR) on the receiver side, and sets the “transmitter empty” bit (TE3) in the Processor A Status Register on the transmitter side (optionally triggering a transmit interrupt 3 on the Processor A-side). • Any read of the BRR3 register will update all status information.

4.3.5.9 Processor B Status Register (MUB_BSR)

Use the Processor B Status Register (BSR, 32-bit, read-write) to show interrupt status from the Processor B, general purpose flags, the Processor A power mode, and to set dual function control-status bits.

- Dual-purpose bits are set by the Processor B-side programmer, and cleared by the MU logic.

Address: 30AB_0000h base + 20h offset = 30AB_0020h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	GIPn				RFn				TEn				Reserved			
W																
Reset	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								FUP	ARS	APM		Reserved	Fn		
W																
Reset	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0

MUB_BSR field descriptions

Field	Description
31–28 GIPn	For n = {0, 1, 2, 3} Processor B General Interrupt Request n Pending. (Read-Write) • GIPn bit signals the Processor B that the GIPn bit in the ACR register on the Processor A-side was set from “0” to “1”. If the GIPn bit in the BCR register is set to “1”, a General Interrupt n request is issued. • The GIPn bit is cleared by writing it back as “1”. Writing “0”, or writing “1” when the GIPn bit is cleared is ignored. Use this feature in the interrupt routine, where the GIPn bit is cleared in order to de-assert the interrupt request source at the interrupt controller. • GIPn bit is cleared when the MU is reset. 0 Processor B general purpose interrupt n is not pending. (default) 1 Processor B general purpose interrupt n is pending.

Table continues on the next page...

MUB_BSR field descriptions (continued)

Field	Description
27–24 RFn	For n = {0, 1, 2, 3} Processor B Receive Register n Full. (Read-only) - RFn bit signals to the Processor B-side that new data was written by the Processor A to the ATRn register, and is ready to be read by the Processor B in the BRRn register. - The RFn bit is set to “1” when the ATRn register is written on the Processor A-side. - After the RFn bit is set to “1”, the RFn bit signals the Processor B-side that new data is ready to be read by the Processor B in the BRRn register, and a Receive n interrupt is issued on the Processor A-side (if the RIEn bit in the BCR register has been set to “1”). - RFn bit is cleared when the BRRn register is read, and when the MU is reset. 0 BRRn register is not full (default). 1 BRRn register has received data from ATRn register and is ready to be read by the Processor B.
23–20 TEn	For n = {0, 1, 2, 3} Processor B Transmit Register n Empty. (Read-only) - When TEEn = “1”, it signals to the Processor B-side that the BTRn register is ready to be written on the Processor B-side. - The TEEn bit is set to “1” after the ARRn register is read on the Processor A-side. - Setting TEEn bit will issue a transmit n interrupt on the Processor B-side (if the TIEn bit in the BCR register is set to “1”). - TEEn bit is cleared after the BTRn register is written on the Processor B-side. - TEEn bit is set to “1” when the MU is reset. 0 BTRn register is not empty. 1 BTRn register is empty (default).
19–9 Reserved	This field is reserved.
8 FUP	Processor B Flags Update Pending. (Read-only) - FUP bit is set to “1” when the Processor B-side sends a Flags Update request to the Processor A-side. - A Flags Update request is generated when the BAF[2:0] bits of the BCR register change. No flag update changes are allowed while the FUP bit is set to “1”. Any write to the BAF[2:0] bits, while the FUP bit is set to “1”, will not generate a Flags Update event, and the BAF[2:0] bits will stay unchanged. - FUP bit is cleared when this Flags Update request is internally acknowledged (that the flag is updated) from the MU Processor A-side, and during MU reset. 0 No flags updated, initiated by the Processor B, in progress (default) 1 Processor B initiated flags update, processing
7 ARS	Processor A Reset State. (Read-only) - ARS bit indicates if the Processor A-side of the MU is in a reset state or not. - If the ARS bit is set to “1”, then the Processor A-side of the MU is still in the reset state. - If the ARS bit is cleared, then both the Processor A and the Processor A-side of the MU are out of reset. - The ARS bit is set to “1” during: a Processor A system reset, or an MU reset (caused by setting the MUR bit at the BCR register).

Table continues on the next page...

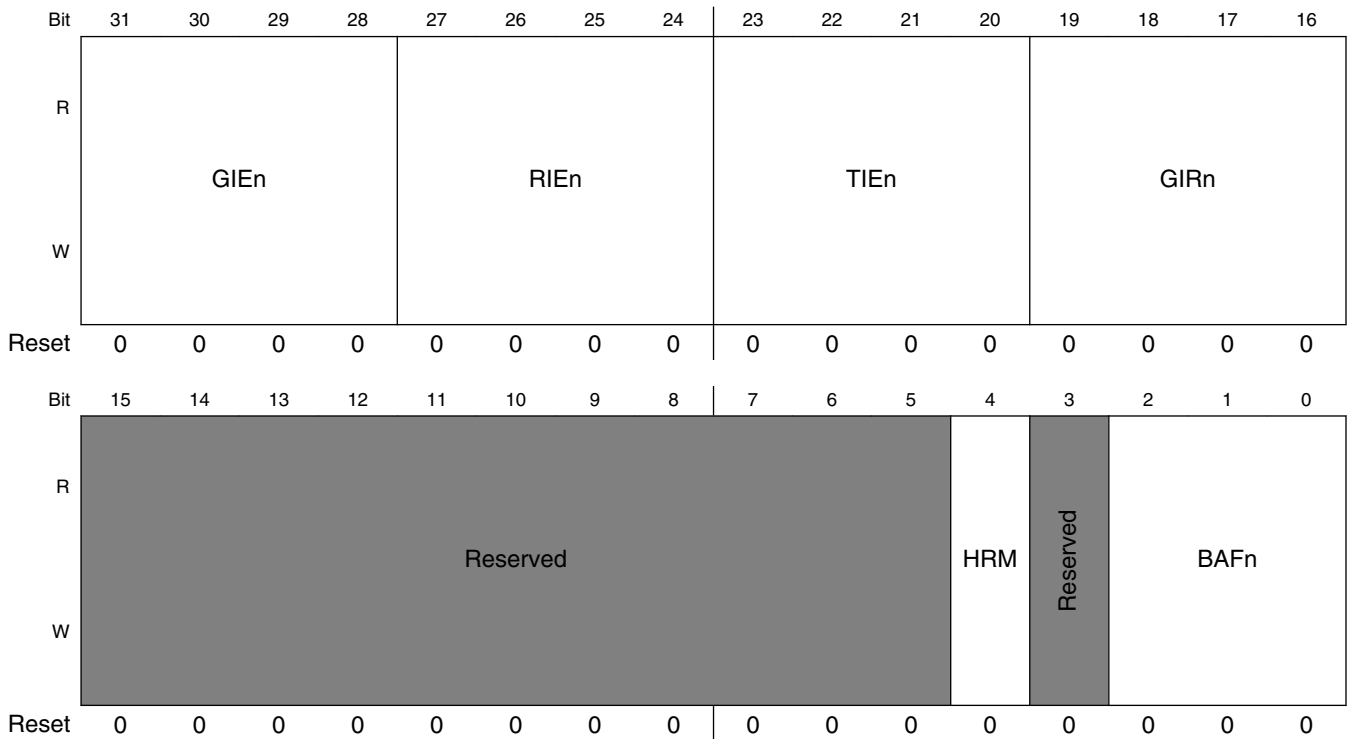
MUB_BSR field descriptions (continued)

Field	Description
	- The ARS bit is cleared when the reset sequence on the Processor A-side of the MU ends. After issuing any of the three reset events mentioned previously, you should verify that the ARS bit is cleared before starting any accesses. - When Processor B side of MU comes out of reset ARS bit has value "1"(default).Then Processor B sees the status of Processor A side and if Processor A has come out of reset then ARS bit goes low.This takes 5-6 clock cycles.And if you read ARS bit now you will see it as low although its reset value was "1" . 0 The Processor A or the Processor A-side of the MU is not in reset. 1 The Processor A or the Processor A-side of the MU is in reset.
6–5 APM	Processor A Power Mode. (Read-only) APM[1:0] bits indicate the Processor A power mode. 00 The System is in Run Mode. 01 The System is in WAIT Mode. 10 Reserved. 11 The System is in STOP Mode.
4 EP	Processor B-Side Event Pending. (Read-only) - EP bit is set to "1" when the Processor B-side mechanism sends an event update request to the Processor A-side. - EP bit is cleared when the event update acknowledge is received. An "event" is any hardware message that is reflected in the ASR register on the Processor A-side (for example, "transmit register 0 written"). During normal operations, you do not have to deal with the state of the EP bit because the event update mechanism works automatically. - To ensure events have been posted to Processor A before entering STOP mode, you should verify that the EP bit is cleared. If EP bit is set to "1", you should wait and continue to poll it (EP bit) before entering STOP mode. - Reading the BSR register (to check the EP bit) should be the last access to the MU that should be performed before entering STOP mode; otherwise, the EP bit may be set by subsequent additional actions. - Due to Processor B pipeline effects, three NOP operations (or their timing equivalent) should be given after an instruction that sets an event before the EP bit can reflect this event. - The EP bit is cleared when the MU resets. 0 The Processor B-side event is not pending (default). 1 The Processor B-side event is pending.
3 Reserved	This field is reserved.
Fn	For n = {0, 1, 2} Processor B-Side Flag n. (Read-only) - Fn bit is the Processor B-side flag that reflects the values written to the ABFn bit in the Processor A control register. - Every time that the ABFn bit is written, the ABFn bit write event updates the Fn bit after the event update latency, which is measured in terms of the number of clocks of the Processor A and the Processor B. 0 ABFn bit in ACR register is written 0 (default). 1 ABFn bit in ACR register is written 1.

4.3.5.10 Processor B Control Register (MUB_BCR)

Use the Processor B Control Register (BCR, 32-bit, read-write) to enable the MU interrupts on the Processor B-side, and trigger events and interrupts on the Processor A-side (wake from STOP, hardware reset, flag update).

Address: 30AB_0000h base + 24h offset = 30AB_0024h



MUB_BCR field descriptions

Field	Description
31–28 GIEn	For n = {0, 1, 2, 3} Processor B General Purpose Interrupt Enable n. (Read-Write) - GIEn bit enables Processor B General Interrupt n. - If GIEn bit is set to “1” (enabled), then a General Interrupt n request is issued when the GIPn bit in the BSR register is set to “1”. - If GIEn is cleared (disabled), then the value of the GIPn bit is ignored and no General Interrupt n request will be issued. - GIEn bit is cleared when the MU resets. 0 Disables Processor B General Interrupt n. (default) 1 Enables Processor B General Interrupt n.
27–24 RIEn	For n = {0, 1, 2, 3} Processor B Receive Interrupt Enable n. (Read-Write) RIEn bit enables Processor B Receive Interrupt n.

Table continues on the next page...

MUB_BCR field descriptions (continued)

Field	Description
	- If RIEn bit is set to “1” (enabled), then an Processor B Receive Interrupt n request is issued when the RFn bit in the BSR register is set to “1”. - If RIEn bit is cleared (disabled), then the value of the RFn bit is ignored and no Processor B Receive Interrupt n request will be issued. - RIEn bit is cleared when the MU resets. 0 Disables Processor B Receive Interrupt n. (default) 1 Enables Processor B Receive Interrupt n.
23–20 TIE _n	For n = {0, 1, 2, 3} Processor B Transmit Interrupt Enable n. (Read-Write) - TIE_n bit enables Processor B Transmit Interrupt n. - If TIE_n bit is set to “1” (enabled), then an Processor B Transmit Interrupt n request is issued when the TEn bit in the BSR register is set to “1”. - If TIE_n bit is cleared (disabled), then the value of the TEn bit is ignored and no Processor B Transmit Interrupt n request will be issued. - TIE_n bit is cleared when the MU resets. 0 Disables Processor B Transmit Interrupt n. (default) 1 Enables Processor B Transmit Interrupt n.
19–16 GIR _n	For n = {0, 1, 2, 3} Processor B General Purpose Interrupt Request n. (Read-Write) - Writing “1” to the GIR_n bit sets the GIP_n bit in the ASR register on the Processor A-side. If the GIEn bit in the ACR register is set to “1” on the Processor A-side, a General Purpose Interrupt n request is triggered. - The GIR_n bit is cleared if the GIP_n bit (in the ASR register on the Processor A-side) is cleared by writing it (GIP_n bit) as “1”, thereby signalling the Processor B that the interrupt was accepted (cleared by the software). The GIP_n bit cannot be written as “0” on the Processor B-side. - To ensure proper operations, you must verify that the GIR_n bit is cleared (meaning that there is no pending interrupt) before setting it (GIR_n bit). - GIR_n bit is cleared when the MU resets. 0 Processor B General Interrupt n is not requested to the Processor A (default). 1 Processor B General Interrupt n is requested to the Processor A.
15–5 Reserved	This field is reserved.
4 HRM	Processor B Hardware Reset Mask. (Read-Write) - The Processor A can give a hardware reset to the Processor B by setting the BHR bit in the ACR Register to “1”. - When the HRM bit is set to “1” by the Processor B, the BHR reset issued by the Processor A is masked (disabled by the Processor B). - When the HRM bit is cleared, the BHR reset issued by the Processor A to the Processor B is not masked (enabled by the Processor B). 0 BHR bit in ACR is not masked, enables the hardware reset to the Processor B (default after hardware reset). 1 BHR bit in ACR is masked, disables the hardware reset request to the Processor B.
3 Reserved	This field is reserved.
BAF _n	For n = {0, 1, 2} Processor B to Processor A Flag n. (Read-Write)

Table continues on the next page...

MUB_BCR field descriptions (continued)

Field	Description
	- BAFn bit is a read-write flag that is reflected in Fn bit in the ASR register on the Processor A-side. - BAFn bit is cleared when the MU resets.
0	Clears the Fn bit in the ASR register.
1	Sets the Fn bit in the ASR register.

4.4 Semaphore (SEMA4)

4.4.1 Overview

The IPS_Semaphores module provides a platform IPS slave device which implements 16 hardware-enforced gates with the following features:

- Module definition supports 16 hardware-enforced gates in a dual-processor configuration, where cp0 is core processor 0 and cp1 is core processor 1
 - Hardware gates appear as a 16-entry byte-size array with read and write accesses
 - Processors lock gates by writing "processor_number+1" to the appropriate gate and must read back the gate value to verify the lock operation was successful
 - Once locked, the gate is unlocked by a write of zeroes from the locking processor
 - Optional interrupt notification after a failed lock write provides a mechanism to indicate when the gate is unlocked
 - Secure reset mechanisms are supported to clear the contents of individual semaphore gates or notification logic, as well as a clear_all capability
 - Programming model allocates memory space to support up to 8 processors and up to 64 gates

A simplified block diagram of the Semaphores module is shown in [Figure 4-10](#). In the diagram, the register blocks named gate0, gate1,..., gate 15 include the finite state machines implementing the semaphore gates plus the interrupt notification logic.

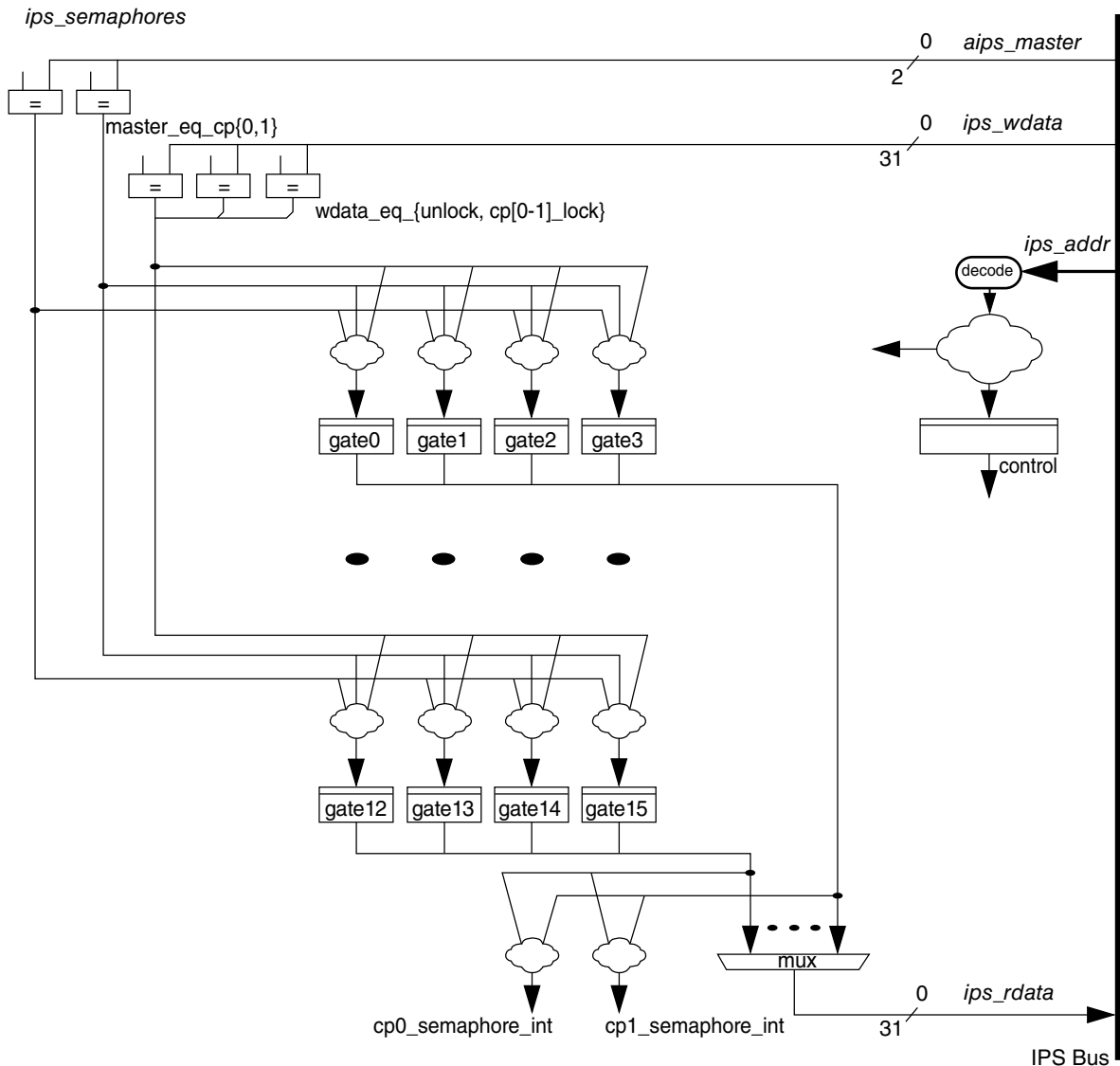


Figure 4-10. IPS_Semaphores Block Diagram

4.4.1.1 Features

The Semaphores module implements hardware-enforced semaphores as an IPS-mapped slave peripheral device. The feature set includes:

- Support for 16 hardware-enforced gates in a dual-processor configuration
 - Each hardware gate appears as a 3-state, 2-bit state machine, with all 16 gates mapped as a byte-size array
 - 3-state implementation

if gate = 0b00, then state = unlocked

if gate = 0b01, then state = locked by processor 0

if gate = 0b10, then state = locked by processor 1

- Uses the bus master number as a reference attribute plus the specified data patterns to validate all write operations
- Once locked, the gate can (and must) be unlocked by a write of zeroes from the locking processor
- Optional interrupt notification after a failed lock write provides a mechanism to indicate when the gate is unlocked
- Secure reset mechanisms are supported to clear the contents of individual gates or notification logic, as well as a clear_all capability
- Memory-mapped IPS slave peripheral platform module
 - Interface to the IPS bus for programming-model accesses
 - Two outputs (one per processor) for interrupt notification of failed lock writes

4.4.1.2 Modes of Operation

The Semaphores module does not support any special modes of operation. As a slave peripheral memory-mapped device located on the platform's IPS slave bus, it responds based strictly on the memory addresses of the connected bus. The IPS bus is used to access the Semaphores ' programming model.

4.4.2 External Signal Description

The Semaphores module does not include any external interfaces.

4.4.3 Functional Description

In this section, the functional operation of the Semaphores module, specifically the state machines of the SEMA4_GATE_n and SEMA4_CP_nNTF registers are detailed.

4.4.3.1 SEMA4_GATE_n Operation

Recall each of the SEMA4_GATE_n registers implements a 2-bit, 3-state machine. The state transitions for each gate are shown in the following figure.

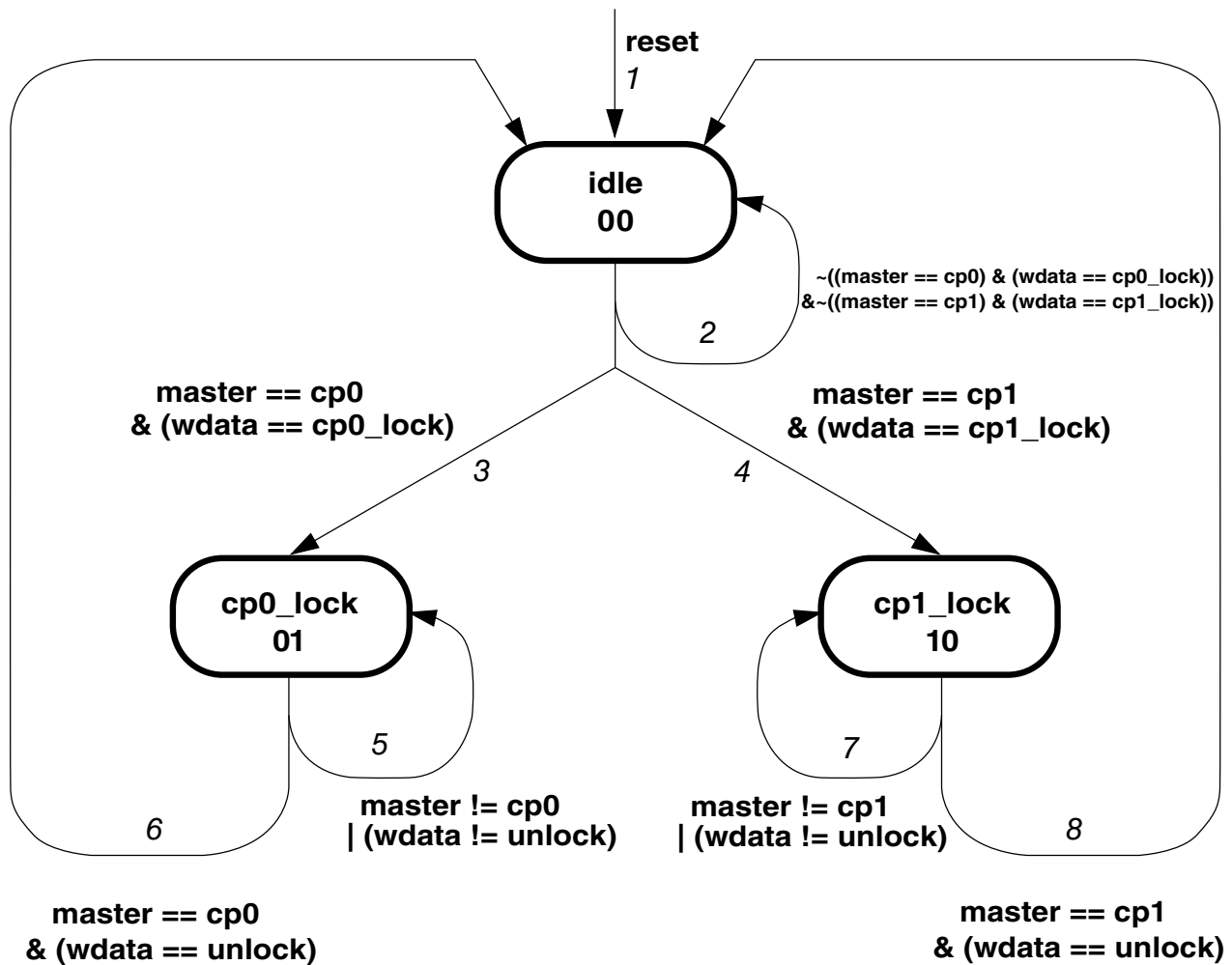


Figure 4-11. SEMA4_GATEn State Machine

The bus master number is used to identify core processor 0 (cp0) or core processor 1 (cp1). The Standard (or Reduced) Product Platform passes the AHB bus master number (hmaster[2:0]) through the AIPS (or AIPS-Lite) controller and drives an aips_master[2:0] output to the Semaphores module as an IPS sideband signal.

The state transitions for SEMA4_GATEn are defined in the following table.

Table 4-16. SEMA4_GATEn State Transitions

Current State	Next State	Transition	Description
—	idle	1	Any reset, whether a system reset or an individual gate reset, unconditionally forces the gate into the idle state.
idle	idle	2	Unless a write of the appropriate lock value from the corresponding processor occurs, the gate remains in the idle state.
idle	cp0_lock	3	When a write of the "cp0_lock" data value is initiated by processor 0, the gate transitions into the cp0_lock state.

Table continues on the next page...

**Table 4-16. SEMA4_GATEn State Transitions
(continued)**

Current State	Next State	Transition	Description
idle	cp1_lock	4	When a write of the "cp1_lock" value is initiated by processor 1, the gate transitions into the cp1_lock state.
cp0_lock	cp0_lock	5	Once in this state, the gate remains here if any attempted write is not from cp0 with the unlock data value.
cp0_lock	idle	6	The gate returns to the idle (unlocked) state once a write from cp0 with the unlock data value occurs.
cp1_lock	cp1_lock	7	Once in this state, the gate remains here if any attempted write is not from cp1 with the unlock data value.
cp1_lock	idle	8	The gate returns to the idle (unlocked) state once a write from cp1 with the unlock data value occurs.

4.4.3.2 SEMA4_CPnNTF Operation

The failed lock write notification interrupt request is implemented in a 3-bit, 5-state machine which records failed lock attempts and transitions based on gate locking and unlocking. Two specific states are encoded and program-visible as SEMA4_CP0NTF[GNn] and SEMA4_CP1NTF[GNn]. See the following figure.

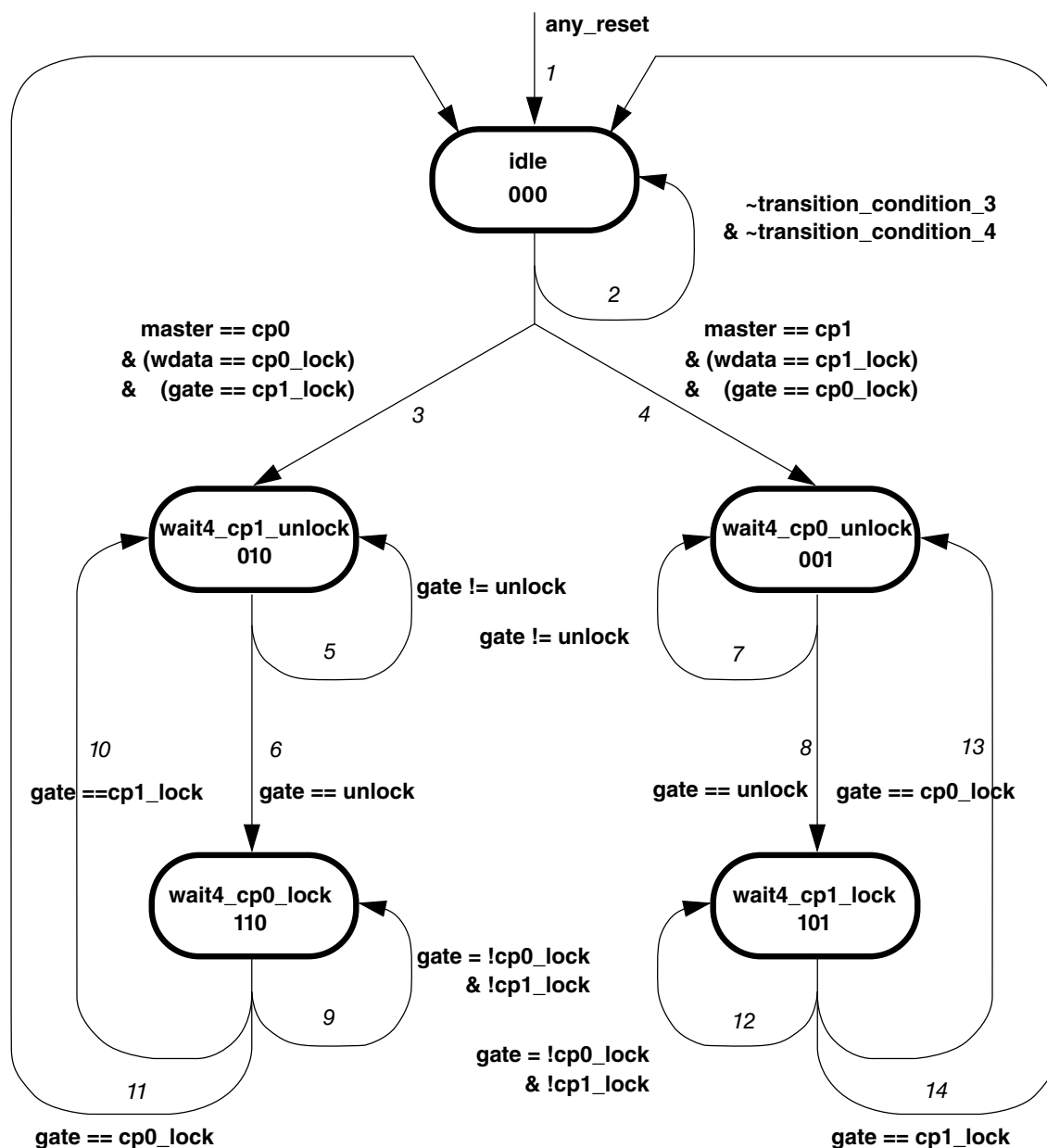


Figure 4-12. IRQ Notification State Machine

The state transitions of the IRQ notification function are defined in the following. Specific states of this machine are program-visible as the SEMA4_CPnNTF registers. In particular, two states are program-visible:

```

if state = wait4_cp0_lock (0b110) // generate cp0_semaphore_int if properly enabled
    then SEMA4_CP0NTF[GnN] = 1; else SEMA4_CP0NTF[GnN] = 0
if state = wait4_cp1_lock (0b101) // generate cp1_semaphore_int if properly enabled
    then SEMA4_CP1NTF[GnN] = 1; else SEMA4_CP1NTF[GnN] = 0
  
```

Table 4-17. IRQ Notification State Transitions

Current State	Next State	Transition	Description
—	idle	1	Any reset, including a system reset or an individual notification or secure gate reset, unconditionally forces the machine into the idle state.
idle	idle	2	Unless a write of the appropriate lock value from the corresponding processor to an already-locked gate occurs, the machine remains in the idle state.
idle	wait4_cp1_unlock	3	When a write of the "cp0_lock" data value is initiated by processor 0 but the gate is already locked by cp1, the machine transitions into this state, where it waits for cp1 to unlock the gate.
idle	wait4_cp0_unlock	4	When a write of the "cp1_lock" data value is initiated by processor 1 but the gate is already locked by cp0, the machine transitions into this state, where it waits for cp0 to unlock the gate.
wait4_cp1_unlock	wait4_cp1_unlock	5	Once in this state, the machine remains here until the gate is unlocked.
wait4_cp1_unlock	wait4_cp0_lock	6	From this state, the machine transitions into the next state, waiting for cp0 to lock the gate, once it has been unlocked.
wait4_cp0_unlock	wait4_cp0_unlock	7	Once in this state, the machine remains here until the gate is unlocked.
wait4_cp0_unlock	wait4_cp1_lock	8	From this state, the machine transitions into the next state, waiting for cp1 to lock the gate, once it has been unlocked.
wait4_cp0_lock	wait4_cp0_lock	9	In this state, the machine generates the notification interrupt (if properly-enabled) and remains here until the gate is locked by processor 0 or the gate is again locked by processor 1.
wait4_cp0_lock	wait4_cp1_unlock	10	In this state, the machine generates the notification interrupt (if properly-enabled) and transitions if the gate is again locked by processor 1. With this transition, the notification interrupt request is negated.
wait4_cp0_lock	idle	11	In this state, the machine generates the notification interrupt (if properly-enabled) and transitions if the gate is finally locked by processor 0. With this transition, the notification interrupt request is negated.
wait4_cp1_lock	wait4_cp1_lock	12	In this state, the machine generates the notification interrupt (if properly-enabled) and remains here until the gate is locked by processor 1 or the gate is again locked by processor 0.
wait4_cp1_lock	wait4_cp0_unlock	13	In this state, the machine generates the notification interrupt (if properly-enabled) and transitions if the gate is again locked by processor 0. With this transition, the notification interrupt request is negated.
wait4_cp1_lock	idle	14	In this state, the machine generates the notification interrupt (if properly-enabled) and transitions if the gate is finally locked by processor 1. With this transition, the notification interrupt request is negated.

The Semaphores module generates two interrupt request output signals, one per processor, combining the SEMA4_CPnINE and SEMA4_CPnNTF registers, where the boolean equations are:

```

cp0_semaphore_int
=   sema4_cp0ine[ine0]    &   sema4_cp0ntf[gn0]
  |   sema4_cp0ine[ine1]    &   sema4_cp0ntf[gn1]
  |   sema4_cp0ine[ine2]    &   sema4_cp0ntf[gn2]
  |   ...
  |   sema4_cp0ine[ine15]   &   sema4_cp0ntf[gn15]
cp1_semaphore_int
=   sema4_cp1ine[ine0]    &   sema4_cp1ntf[gn0]
  |   sema4_cp1ine[ine1]    &   sema4_cp1ntf[gn1]
  |   sema4_cp1ine[ine2]    &   sema4_cp1ntf[gn2]
  |   ...
  |   sema4_cp1ine[ine15]   &   sema4_cp1ntf[gn15]

```

4.4.4 Initialization Information

The reset state of the IPS_Semaphores module allows it to begin operation without the need for any further initialization. All the internal state machines are cleared by any reset event, allowing the module to immediately begin operation.

4.4.5 Application Information

In an operational multi-core system, most interactions involving the Semaphores module involves reads and writes to the SEMA4_GATE_n registers for implementation of the hardware-enforced software gate functions. Typical code segments for gate functions perform the following operations:

- To lock (close) a gate
 - The processor performs a byte write of "logical_processor_number + 1" to gate[i]
 - The processor reads back gate[i] and checks for a value of "logical_processor_number + 1"

If the compare indicates the expected value, then the gate is locked; proceed with the protected code segment. If the compare does not indicate the expected value, the lock operation failed; repeat the process beginning with byte write to gate[i] in spin-wait loop, or proceed with another execution path and wait for failed lock interrupt notification.

A simple C-language example of a `gateLock` function is shown in the following figure. This function follows the Hennessy/Patterson example described in [Multi-Core Programming 101: Software Gates](#).

Semaphore (SEMA4)

```
#define UNLOCK    0
#define CP0_LOCK  1
#define CP1_LOCK  2

void gateLock (n)
int  n;                /* gate number to lock */
{
    int i;
    int current_value;
    int locked_value;

    i = processor_number(); /* obtain logical CPU number */

    if (i == 0)
        locked_value = CP0_LOCK;
    else
        locked_value = CP1_LOCK;

    /* read the current value of the gate and wait until the state == UNLOCK */
    do {
        current_value = gate[n];
    } while (current_value != UNLOCK);

    /* the current value of the gate == UNLOCK. attempt to lock the gate for this
       processor. spin-wait in this loop until gate ownership is obtained */
    do {
        gate[n] = locked_value; /* write gate with processor_number + 1 */
        current_value = gate[n]; /* read gate to verify ownership was obtained */
    } while (current_value != locked_value);
}
```

Figure 4-13. Sample gateLock Function

- To unlock (open) a gate
 - After completing the protected code segment, the locking processor performs a byte write of zeroes to gate[i], opening (unlocking) the gate

A few comments on the logical CPU number are appropriate. In this example, a reference to `processor_number()` is used to retrieve this hardware configuration value. Typically, the logical processor numbers are defined by a hardwired input vector to the individual cores. The exact method for accessing the logical processor number varies by architecture. For PowerPC cores, there is a processor ID register (PIR) which is SPR 286 and contains this value. A single instruction can be used to move the contents of the PIR into a general-purpose register: `mf spr rx, 286` where `rx` is the destination GPRn. Other architectures may support a specific instruction to move the contents of the logical processor number into a general-purpose register, e.g., `rdcpn rx` for a "read CPU number" instruction.

If the optional failed lock IRQ notification mechanisms are used, then accesses to the related registers (SEMA4_CPnINE, SEMA4_CPnNTF) are required. Note that there is no required negation of the failed lock write notification interrupt as the request is automatically negated by the Semaphores module once the gate has been successfully locked by the "failing" processor.

Finally, in the event a system state requires a software-controlled reset of a gate or IRQ notification register(s), accesses to the secure reset control registers (SEMA4_RSTGT, SEMA4_RSTNTF) are required. For these situations, it is recommended that the appropriate IRQ notification enable(s) (SEMA4_CPnINE) bits be disabled *before* initiating the secure reset 2-write sequence to avoid any race conditions involving spurious notification interrupt requests.

4.4.6 Memory map and register definition

The Semaphores module provides an IPS programming model mapped to an SPP-standard on-platform 16 KB space. The description here specifies a dual-core configuration with 16 semaphore gates. All the register names are prefixed with "Sema4" as an abbreviation for the full module name.

The programming model is referenced using 8-, 16- and 32-bit accesses. Reads can use any reference size, while writes are generally restricted to the size of the register. Exceptions to the write size restrictions are detailed in the individual register descriptions. Attempted references using inappropriate access sizes, to undefined (reserved) addresses, or with a non-supported access type (for example, a write to a read-only register) generate an IPS error termination.

Finally, the programming model allocates space for a definition with up to 64 gates and up to 8 processor cores, even though this definition is considerably larger than any currently-planned module implementations. The number of gates and supported processor cores are independent; there is no relationship between these two system variables.

The 16 KB Semaphores programming model map is shown in the following table.

SEMA4 memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
30AC_0000	Semaphores Gate 0 Register (SEMA4_Gate00)	8	R/W	00h	4.4.6.1/166
30AC_0001	Semaphores Gate 1 Register (SEMA4_Gate01)	8	R/W	00h	4.4.6.2/167
30AC_0002	Semaphores Gate 2 Register (SEMA4_Gate02)	8	R/W	00h	4.4.6.3/168
30AC_0003	Semaphores Gate 3 Register (SEMA4_Gate03)	8	R/W	00h	4.4.6.4/169
30AC_0004	Semaphores Gate 4 Register (SEMA4_Gate04)	8	R/W	00h	4.4.6.5/170
30AC_0005	Semaphores Gate 5 Register (SEMA4_Gate05)	8	R/W	00h	4.4.6.6/171
30AC_0006	Semaphores Gate 6 Register (SEMA4_Gate06)	8	R/W	00h	4.4.6.7/172
30AC_0007	Semaphores Gate 7 Register (SEMA4_Gate07)	8	R/W	00h	4.4.6.8/173

Table continues on the next page...

SEMA4 memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
30AC_0008	Semaphores Gate 8 Register (SEMA4_Gate08)	8	R/W	00h	4.4.6.9/174
30AC_0009	Semaphores Gate 9 Register (SEMA4_Gate09)	8	R/W	00h	4.4.6.10/175
30AC_000A	Semaphores Gate 10 Register (SEMA4_Gate10)	8	R/W	00h	4.4.6.11/176
30AC_000B	Semaphores Gate 11 Register (SEMA4_Gate11)	8	R/W	00h	4.4.6.12/177
30AC_000C	Semaphores Gate 12 Register (SEMA4_Gate12)	8	R/W	00h	4.4.6.13/178
30AC_000D	Semaphores Gate 13 Register (SEMA4_Gate13)	8	R/W	00h	4.4.6.14/179
30AC_000E	Semaphores Gate 14 Register (SEMA4_Gate14)	8	R/W	00h	4.4.6.15/180
30AC_000F	Semaphores Gate 15 Register (SEMA4_Gate15)	8	R/W	00h	4.4.6.16/181
30AC_0040	Semaphores Processor n IRQ Notification Enable (SEMA4_CP0INE)	16	R/W	0000h	4.4.6.17/182
30AC_0048	Semaphores Processor n IRQ Notification Enable (SEMA4_CP1INE)	16	R/W	0000h	4.4.6.17/182
30AC_0080	Semaphores Processor n IRQ Notification (SEMA4_CP0NTF)	16	R	0000h	4.4.6.18/184
30AC_0088	Semaphores Processor n IRQ Notification (SEMA4_CP1NTF)	16	R	0000h	4.4.6.18/184
30AC_0100	Semaphores (Secure) Reset Gate n (SEMA4_RSTGT)	16	R/W	0000h	4.4.6.19/186
30AC_0104	Semaphores (Secure) Reset IRQ Notification (SEMA4_RSTNTF)	16	R/W	0000h	4.4.6.20/187

4.4.6.1 Semaphores Gate 0 Register (SEMA4_Gate00)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate

register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + 0h offset = 30AC_0000h

Bit	7	6	5	4	3	2	1	0
Read	0						GTFSM	
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_Gate00 field descriptions

Field	Description
7–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free). 01 The gate has been locked by processor 0. 10 The gate has been locked by processor 1. 11 This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.2 Semaphores Gate 1 Register (SEMA4_Gate01)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Memory map and register definition

Address: 30AC_0000h base + 1h offset = 30AC_0001h

Bit	7	6	5	4	3	2	1	0
Read	0						GTFSM	
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_Gate01 field descriptions

Field	Description
7–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free). 01 The gate has been locked by processor 0. 10 The gate has been locked by processor 1. 11 This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.3 Semaphores Gate 2 Register (SEMA4_Gate02)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + 2h offset = 30AC_0002h

Bit	7	6	5	4	3	2	1	0
Read	0						GTFSM	
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_Gate02 field descriptions

Field	Description
7–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free). 01 The gate has been locked by processor 0. 10 The gate has been locked by processor 1. 11 This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.4 Semaphores Gate 3 Register (SEMA4_Gate03)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + 3h offset = 30AC_0003h

Bit	7	6	5	4	3	2	1	0
Read	0						GTFSM	
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_Gate03 field descriptions

Field	Description
7–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine.

Table continues on the next page...

SEMA4_Gate03 field descriptions (continued)

Field	Description
	Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation .
	NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug.
00	The gate is unlocked (free).
01	The gate has been locked by processor 0.
10	The gate has been locked by processor 1.
11	This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.5 Semaphores Gate 4 Register (SEMA4_Gate04)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + 4h offset = 30AC_0004h

Bit	7	6	5	4	3	2	1	0
Read	0						GTFSM	
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_Gate04 field descriptions

Field	Description
7-2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation .

Table continues on the next page...

SEMA4_Gate04 field descriptions (continued)

Field	Description
	NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug.
00	The gate is unlocked (free).
01	The gate has been locked by processor 0.
10	The gate has been locked by processor 1.
11	This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.6 Semaphores Gate 5 Register (SEMA4_Gate05)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + 5h offset = 30AC_0005h

Bit	7	6	5	4	3	2	1	0
Read	0						GTFSM	
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_Gate05 field descriptions

Field	Description
7-2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free).

Table continues on the next page...

SEMA4_Gate05 field descriptions (continued)

Field	Description
01	The gate has been locked by processor 0.
10	The gate has been locked by processor 1.
11	This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.7 Semaphores Gate 6 Register (SEMA4_Gate06)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + 6h offset = 30AC_0006h

Bit	7	6	5	4	3	2	1	0
Read	0						GTFSM	
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_Gate06 field descriptions

Field	Description
7-2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free). 01 The gate has been locked by processor 0. 10 The gate has been locked by processor 1. 11 This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.8 Semaphores Gate 7 Register (SEMA4_Gate07)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + 7h offset = 30AC_0007h

Bit	7	6	5	4	3	2	1	0
Read	0						GTFSM	
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_Gate07 field descriptions

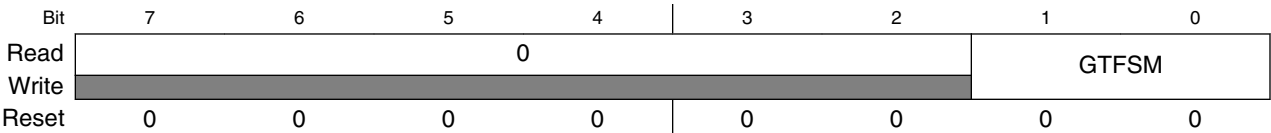
Field	Description
7–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free). 01 The gate has been locked by processor 0. 10 The gate has been locked by processor 1. 11 This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.9 Semaphores Gate 8 Register (SEMA4_Gate08)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + 8h offset = 30AC_0008h



SEMA4_Gate08 field descriptions

Field	Description
7–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free). 01 The gate has been locked by processor 0. 10 The gate has been locked by processor 1. 11 This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.10 Semaphores Gate 9 Register (SEMA4_Gate09)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + 9h offset = 30AC_0009h

Bit	7	6	5	4	3	2	1	0
Read	0						GTFSM	
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_Gate09 field descriptions

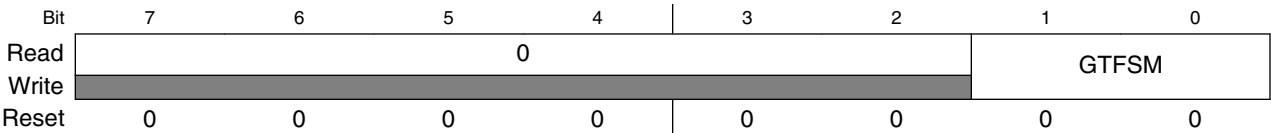
Field	Description
7–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free). 01 The gate has been locked by processor 0. 10 The gate has been locked by processor 1. 11 This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.11 Semaphores Gate 10 Register (SEMA4_Gate10)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + Ah offset = 30AC_000Ah



SEMA4_Gate10 field descriptions

Field	Description
7–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free). 01 The gate has been locked by processor 0. 10 The gate has been locked by processor 1. 11 This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.12 Semaphores Gate 11 Register (SEMA4_Gate11)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + Bh offset = 30AC_000Bh

Bit	7	6	5	4	3	2	1	0
Read	0						GTFSM	
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_Gate11 field descriptions

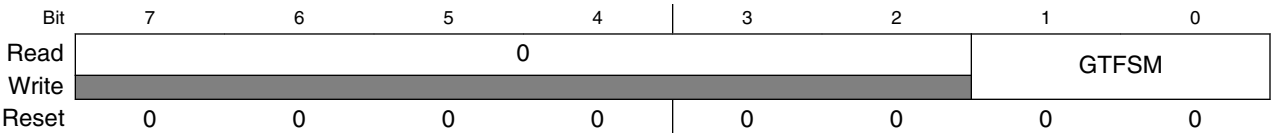
Field	Description
7–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free). 01 The gate has been locked by processor 0. 10 The gate has been locked by processor 1. 11 This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.13 Semaphores Gate 12 Register (SEMA4_Gate12)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + Ch offset = 30AC_000Ch



SEMA4_Gate12 field descriptions

Field	Description
7–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free). 01 The gate has been locked by processor 0. 10 The gate has been locked by processor 1. 11 This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.14 Semaphores Gate 13 Register (SEMA4_Gate13)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + Dh offset = 30AC_000Dh

Bit	7	6	5	4	3	2	1	0
Read	0						GTFSM	
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_Gate13 field descriptions

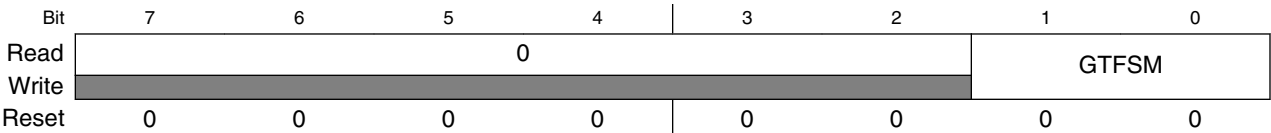
Field	Description
7–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free). 01 The gate has been locked by processor 0. 10 The gate has been locked by processor 1. 11 This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.15 Semaphores Gate 14 Register (SEMA4_Gate14)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + Eh offset = 30AC_000Eh



SEMA4_Gate14 field descriptions

Field	Description
7–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free). 01 The gate has been locked by processor 0. 10 The gate has been locked by processor 1. 11 This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.16 Semaphores Gate 15 Register (SEMA4_Gate15)

Each semaphore gate is implemented in a 2-bit finite state machine, right-justified in a byte data structure. The hardware uses the bus master number in conjunction with the data patterns to validate all attempted write operations. Only processor bus masters can modify the gate registers. Once locked, a gate can (and must) be opened (unlocked) by the locking processor core.

Multiple gate values can be read in a single access, but only a single gate can be updated via a write operation at a time. 16- and 32-bit writes to multiple gates are allowed, but the write data operand must only update the state of a single gate. A byte write data value of 0x03 is defined as "no operation" and does not affect the state of the corresponding gate register. Attempts to write multiple gates in a single aligned access with a size larger than an 8-bit (byte) reference generate an error termination and do not allow any gate state changes

Address: 30AC_0000h base + Fh offset = 30AC_000Fh

Bit	7	6	5	4	3	2	1	0
Read	0						GTFSM	
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_Gate15 field descriptions

Field	Description
7–2 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
GTFSM	Gate Finite State Machine. Gate Finite State Machine. The hardware gate is maintained in a 3-state implementation-unlocked, locked by processor 0 or locked by processor 1. For more details, see SEMA4_GATEn Operation . NOTE: The state of the gate reflects the last processor that locked it, which can be useful during system debug. 00 The gate is unlocked (free). 01 The gate has been locked by processor 0. 10 The gate has been locked by processor 1. 11 This state encoding is never used and therefore reserved. Attempted writes of 0x03 are treated as "no operation" and do not affect the gate state machine.

4.4.6.17 Semaphores Processor n IRQ Notification Enable (SEMA4_CPnINE)

The application of a hardware semaphore module provides an opportunity for implementation of helpful system-level features. An example is an optional mechanism to generate a processor interrupt after a failed lock attempt. Recall traditional software gate functions execute a spin-wait loop in an effort to obtain and lock the referenced gate. With this module, the processor that fails in the lock attempt could continue with other tasks and allow a properly-enabled notification interrupt to return its execution to the original lock function.

The optional notification interrupt function consists of two registers for each processor: an interrupt notification enable register (SEMA4_CPnINE) and the interrupt request register (SEMA4_CPnNTF). To support implementations with more than 16 gates, these registers can be referenced with aligned 16- or 32-bit accesses. For the SEMA4_CPnINE registers, unimplemented bits read as zeroes, and writes are ignored.

Address: 30AC_0000h base + 40h offset + (8d × i), where i=0d to 1d

Bit	15	14	13	12	11	10	9	8
Read	INE8	INE9	INE10	INE11	INE12	INE13	INE14	INE15
Write								
Reset	0	0	0	0	0	0	0	0

Bit	7	6	5	4	3	2	1	0
Read	INE0	INE1	INE2	INE3	INE4	INE5	INE6	INE7
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_CPnINE field descriptions

Field	Description
15 INE8	Interrupt Request Notification Enable 8. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 8. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
14 INE9	Interrupt Request Notification Enable 9. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 9. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
13 INE10	Interrupt Request Notification Enable 10. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 10. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
12 INE11	Interrupt Request Notification Enable 11. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 11.

Table continues on the next page...

SEMA4_CPnINE field descriptions (continued)

Field	Description
	0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
11 INE12	Interrupt Request Notification Enable 12. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 12. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
10 INE13	Interrupt Request Notification Enable 13. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 13. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
9 INE14	Interrupt Request Notification Enable 14. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 14. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
8 INE15	Interrupt Request Notification Enable 15. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 15. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
7 INE0	Interrupt Request Notification Enable 0. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 0. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
6 INE1	Interrupt Request Notification Enable 1. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 1. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
5 INE2	Interrupt Request Notification Enable 2. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 2. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
4 INE3	Interrupt Request Notification Enable 3. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 3. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
3 INE4	Interrupt Request Notification Enable 4. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 4. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
2 INE5	Interrupt Request Notification Enable 5. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 5.

Table continues on the next page...

SEMA4_CPnINE field descriptions (continued)

Field	Description
	0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
1 INE6	Interrupt Request Notification Enable 6. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 6. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.
0 INE7	Interrupt Request Notification Enable 7. This field is a bitmap to enable the generation of an interrupt notification from a failed attempt to lock gate 7. 0 The generation of the notification interrupt is disabled. 1 The generation of the notification interrupt is enabled.

4.4.6.18 Semaphores Processor n IRQ Notification (SEMA4_CPnNTF)

The Semaphores module optionally allows the processor that fails in the lock attempt to continue with other tasks and allow a properly-enabled notification interrupt to return its execution to the original lock function rather than simply execute in a spin-wait loop.

The optional notification interrupt mechanism consists of two registers for each processor: an interrupt notification enable register (SEMA4_CPnINE) and the read-only notification interrupt request register (SEMA4_CPnNTF). To support implementations with more than 16 gates, these registers can be referenced with aligned 16- or 32-bit accesses. For the SEMA4_CPnNTF registers, unimplemented bits read as zeroes.

The notification interrupt is generated via a unique finite state machine, one per hardware gate. This machine operates in the following manner:

1. When an attempted lock fails, the FSM enters a first state where it waits until the gate is unlocked.
2. Once unlocked, the FSM enters a second state where it generates an interrupt request to the “failed lock” processor.
3. When the “failed lock” processor succeeds in locking the gate, the IRQ is automatically negated and the FSM returns to the idle state. However, if the other processor again locks the gate, the FSM returns to the first state, negates the interrupt request, and then waits for the gate to be unlocked (again).

The notification interrupt request is implemented in a 3-bit, 5-state machine, where two specific states are encoded and program-visible as SEMA4_CP0NTF[GNn] and SEMA4_CP1NTF[GNn].

Address: 30AC_0000h base + 80h offset + (8d × i), where i=0d to 1d

Bit	15	14	13	12	11	10	9	8
Read	GN8	GN9	GN10	GN11	GN12	GN13	GN14	GN15
Write								
Reset	0	0	0	0	0	0	0	0

Bit	7	6	5	4	3	2	1	0
Read	GN0	GN1	GN2	GN3	GN4	GN5	GN6	GN7
Write								
Reset	0	0	0	0	0	0	0	0

SEMA4_CPnNTF field descriptions

Field	Description
15 GN8	Gate 8 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 8. For more details, see SEMA4_CPnNTF Operation .
14 GN9	Gate 9 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 9. For more details, see SEMA4_CPnNTF Operation .
13 GN10	Gate 10 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 10. For more details, see SEMA4_CPnNTF Operation .
12 GN11	Gate 11 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 11. For more details, see SEMA4_CPnNTF Operation .
11 GN12	Gate 12 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 12. For more details, see SEMA4_CPnNTF Operation .
10 GN13	Gate 13 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 13. For more details, see SEMA4_CPnNTF Operation .
9 GN14	Gate 14 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 14. For more details, see SEMA4_CPnNTF Operation .
8 GN15	Gate 15 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 15. For more details, see SEMA4_CPnNTF Operation .
7 GN0	Gate 0 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 0. For more details, see SEMA4_CPnNTF Operation .
6 GN1	Gate 1 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 1. For more details, see SEMA4_CPnNTF Operation .
5 GN2	Gate 2 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 2. For more details, see SEMA4_CPnNTF Operation .
4 GN3	Gate 3 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 3. For more details, see SEMA4_CPnNTF Operation .
3 GN4	Gate 4 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 4. For more details, see SEMA4_CPnNTF Operation .
2 GN5	Gate 5 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 5. For more details, see SEMA4_CPnNTF Operation .
1 GN6	Gate 6 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 6. For more details, see SEMA4_CPnNTF Operation .
0 GN7	Gate 7 Notification. This read-only field is a bitmap of the interrupt request notification from a failed attempt to lock gate 7. For more details, see SEMA4_CPnNTF Operation .

4.4.6.19 Semaphores (Secure) Reset Gate n (SEMA4_RSTGT)

Although the intent of the hardware gate implementation specifies a protocol where the locking processor must unlock the gate, it is recognized that system operation may require a reset function to re-initialize the state of any gate(s) without requiring a system-level reset.

To support this special gate reset requirement, the Semaphores module implements a "secure" reset mechanism which allows a hardware gate (or all the gates) to be initialized by following a specific dual-write access pattern. Using a technique similar to that required for the servicing of a software watchdog timer, the secure gate reset requires two consecutive writes with predefined data patterns from the same processor to force the clearing of the specified gate(s). The required access pattern is:

1. A processor performs a 16-bit write to the SEMA4_RSTGT memory location. The most significant byte (SEMA4_RSTGT[RSTGDP]) must be 0xe2; the least significant byte is a "don't_care" for this reference.
2. The same processor then performs a second 16-bit write to the SEMA4_RSTGT location. For this write, the upper byte (SEMA4_RSTGT[RSTGDP]) is the logical complement of the first data pattern (0x1d) and the lower byte (SEMA4_RSTGT[RSTGTN]) specifies the gate(s) to be reset. This gate field can specify a single gate be cleared, or that all gates are cleared.
3. Reads of the SEMA4_RSTGT location return information on the 2-bit state machine (SEMA4_RSTGT[RSTGSM]) which implements this function, the bus master performing the reset (SEMA4_RSTGT[RSTGMS]) and the gate number(s) last cleared (SEMA4_RSTGT[RSTGTN]). Reads of the SEMA4_RSTGT register do not affect the secure reset finite state machine in any manner.

Address: 30AC_0000h base + 100h offset = 30AC_0100h

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read	RSTGTN								RSTGSM_RSTGMS_RSTGDP							
Write																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SEMA4_RSTGT field descriptions

Field	Description
15–8 RSTGTN	Reset Gate Number. This 8-bit field specifies the specific hardware gate to be reset. This field is updated by the second write. If RSTGTN < 64, then reset the single gate defined by RSTGTN, else reset all the gates. The corresponding secure IRQ notification state machine(s) are also reset.
RSTGSM_ RSTGMS_ RSTGDP	NOTE: This field contains subfields that vary depending on whether it is being read or written. Sub-fields indicated as having read access are valid only for read operations. Sub-fields indicated as having

Table continues on the next page...

SEMA4_RSTGT field descriptions (continued)

Field	Description		
	write access are valid only for write operations. Bit numbering in the descriptions begins with the most significant bit numbered 0. See the following table for details.		
	Access	Sub-Field	Description
	Read-Only	7-6 Reserved	Reserved. Always reads 0.
		5-4 RSTGSM	Reset Gate Finite State Machine. Reads of the SEMA4_RSTGT register return the encoded state machine value. The reset state machine is maintained in a 2-bit, 3-state implementation, defined as: 00 Idle, waiting for the first data pattern write. 01 Waiting for the second data pattern write. 10 The 2-write sequence has completed. Generate the specified gate reset(s). After the reset is performed, this machine returns to the idle (waiting for first data pattern write) state. Note that the RSTGSM = 0b10 state is valid for only a single machine cycle, so it is impossible for a read to return this value 11 This state encoding is never used and therefore reserved.
		3 Reserved	Reserved. Always reads 0.
		2-0 RSTGMS	Reset Gate Bus Master. This 3-bit read-only field records the logical number of the bus master performing the gate reset function. The reset function requires that the two consecutive writes to this register be initiated by the same bus master to succeed. This field is updated each time a write to this register occurs. The association between system bus master port numbers, the associated bus master device and the logical processor number is SoC-specific. See the chip configuration chapter for this information.
	Write-Only	7-0 RSTGDP	Reset Gate Data Pattern. This write-only field is accessed with the specified data patterns on the two consecutive writes to enable the gate reset mechanism. For the first write, RSTGDP = 0xe2 while the second write requires RSTGDP = 0x1d.

4.4.6.20 Semaphores (Secure) Reset IRQ Notification (SEMA4_RSTNTF)

As with the case of the secure reset function and the hardware gates, it is recognized that system operation may require a reset function to re-initialize the state of the IRQ notification logic without requiring a system-level reset.

To support this special notification reset requirement, the Semaphores module implements a "secure" reset mechanism which allows an IRQ notification (or all the notifications) to be initialized by following a specific dual-write access pattern. When successful, the specified IRQ notification state machine(s) are reset. Using a technique

similar to that required for the servicing of a software watchdog timer, the secure reset mechanism requires two consecutive writes with predefined data patterns from the same processor to force the clearing of the IRQ notification(s). The required access pattern is:

1. A processor performs a 16-bit write to the SEMA4_RSTNTF memory location. The most significant byte (SEMA4_RSTNTF[RSTNDP]) must be 0x47; the least significant byte is a "don't_care" for this reference.
2. The same processor then performs a second 16-bit write to the SEMA4_RSTNTF location. For this write, the upper byte (SEMA4_RSTNTF[RSTNDP]) is the logical complement of the first data pattern (0xb8) and the lower byte (SEMA4_RSTNTF[RSTNTN]) specifies the notification(s) to be reset. This field can specify a single notification be cleared, or that all notifications are cleared.
3. Reads of the SEMA4_RSTNTF location return information on the 2-bit state machine (SEMA4_RSTNTF[RSTNSM]) which implements this function, the bus master performing the reset (SEMA4_RSTNTF[RSTNMS]) and the notification number(s) last cleared (SEMA4_RSTNTF[RSTNTN]). Reads of the SEMA4_RSTNTF register do not affect the secure reset finite state machine in any manner.

Address: 30AC_0000h base + 104h offset = 30AC_0104h

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read	RSTNTN								RSTNSM_RSTNMS_RSTNDP							
Write																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SEMA4_RSTNTF field descriptions

Field	Description								
15–8 RSTNTN	Reset Notification Number. This 8-bit field specifies the specific IRQ notification state machine to be reset. This field is updated by the second write. If RSTNTN < 64, then reset the single IRQ notification machine defined by RSTNTN, else reset all the notifications.								
RSTNSM_ RSTNMS_ RSTNDP	NOTE: This field contains subfields that vary depending on whether it is being read or written. Sub-fields indicated as having read access are valid only for read operations. Sub-fields indicated as having write access are valid only for write operations. Bit numbering in the descriptions begins with the most significant bit numbered 0. See the following table for details. <table><tr><th>Access</th><th>Sub-Field</th><th>Description</th></tr><tr><td rowspan="2">Read-Only</td><td>7-6 Reserved</td><td>Reserved. Always reads 0.</td></tr><tr><td>5-4 RSTNSM</td><td>Reset Notification Finite State Machine. Reads of the SEMA4_RSTNTF register return the encoded state machine value. The reset state machine is maintained in a 2-bit, 3-state implementation, defined as: 00 Idle, waiting for the first data pattern write. 01 Waiting for the second data pattern write.</td></tr></table>	Access	Sub-Field	Description	Read-Only	7-6 Reserved	Reserved. Always reads 0.	5-4 RSTNSM	Reset Notification Finite State Machine. Reads of the SEMA4_RSTNTF register return the encoded state machine value. The reset state machine is maintained in a 2-bit, 3-state implementation, defined as: 00 Idle, waiting for the first data pattern write. 01 Waiting for the second data pattern write.
Access	Sub-Field	Description							
Read-Only	7-6 Reserved	Reserved. Always reads 0.							
	5-4 RSTNSM	Reset Notification Finite State Machine. Reads of the SEMA4_RSTNTF register return the encoded state machine value. The reset state machine is maintained in a 2-bit, 3-state implementation, defined as: 00 Idle, waiting for the first data pattern write. 01 Waiting for the second data pattern write.							

Table continues on the next page...

SEMA4_RSTNTF field descriptions (continued)

Field	Description		
	Access	Sub-Field	Description
			10 The 2-write sequence has completed. Generate the specified notification reset(s). After the reset is performed, this machine returns to the idle (waiting for first data pattern write) state. Note the RSTNSM = 10 state is valid for only a single machine cycle, so it is impossible for a read to return this value.
			11 This state encoding is never used and therefore reserved..
		3 Reserved	Reserved. Always reads 0.
		2-0 RSTNMS	Reset Notification Bus Master. This 3-bit read-only field records the logical number of the bus master performing the notification reset function. The reset function requires that the two consecutive writes to this register be initiated by the same bus master to succeed. This field is updated each time a write to this register occurs. The association between system bus master port numbers, the associated bus master device and the logical processor number is SoC-specific. See the chip configuration chapter for this information.
	Write-Only	7-0 RSTNDP	Reset Notification Data Pattern. This write-only field is accessed with the specified data patterns on the two consecutive writes to enable the notification reset mechanism. For the first write, RSTNDP = 0x47 while the second write requires RSTNDP = 0xb8.

4.5 On-Chip RAM Memory Controller (OCRAM)

4.5.1 Overview

Various options are provided for adding a pipeline or wait-states in a read/write access, in order to ensure flexible timing control at both high and low frequencies.

The internal block diagram is shown in the figure below.

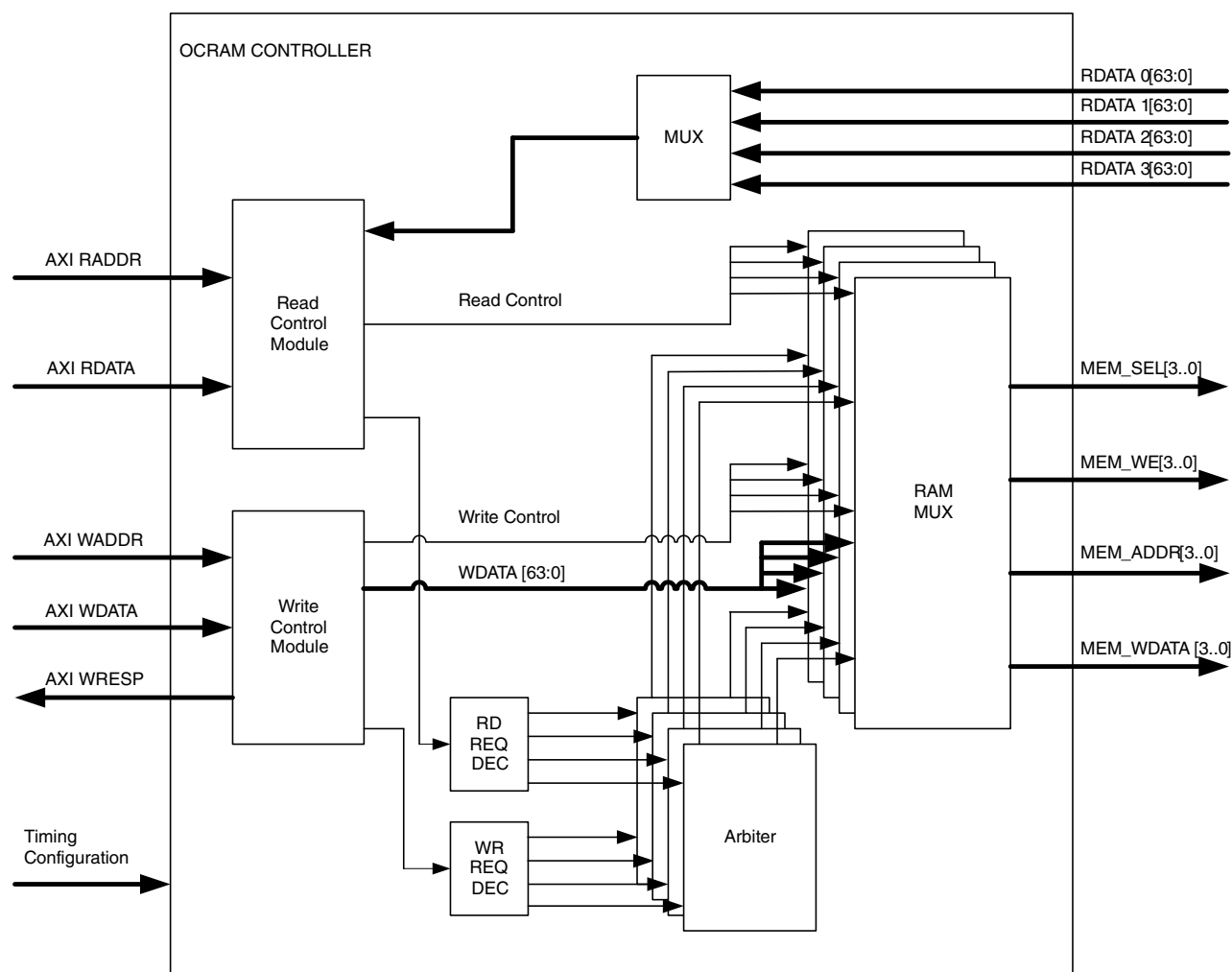


Figure 4-14. On-chip RAM Block Diagram

4.5.2 Basic Functions

4.5.2.1 Read/Write Arbitration

The detailed rules used in arbitration are as follows:

- If there is no granted read or write in the last cycle, and there is only a read request or a write request, the request will be granted.
- If there is no granted read or write in the last cycle, and there are both read or write requests coming in at the same time, the read request will be granted first.

- If a granted read/write transaction has just finished, the write/read request will have the higher priority in the next cycle.
- If the first read/write access request in a transaction is granted, all the data transfer in this burst will be finished before the next arbitration begins, that is, the round-robin arbitration mechanism is based on AXI transaction, not data access.

4.5.3 Advanced Features

This section describes some advanced features designed to avoid timing issues when the on-chip RAM is working at high frequency.

4.5.3.1 Read Data Wait State

When the wait state is enabled, it will take 2 cycles for each read access (each beat of a read burst).

This can avoid the potential timing problem caused by the longer memory access time at higher frequency.

When this feature is disabled, it only takes 1 clock cycle to finish a read transaction. That is, read data is available in the next cycle of read request becomes valid on the bus.

4.5.3.2 Read Address Pipeline

When this feature is enabled, the read address from the AXI master is delayed 1 cycle before it can be accepted by the on-chip RAM.

This can avoid setup time issues for the read access on the memory cell at high frequency. Enabling this feature can cost, at most, 1 more clock cycle for each AXI read transaction, that is, at most 1 more clock cycle for each read burst with multiple beats of data.

When this feature is disabled, the read address from the AXI master can be accepted by the on-chip RAM without delay, and data can become ready for master at next clock cycle (if no other access and no read data wait).

4.5.3.3 Write Data Pipeline

When this feature is enabled, the write data from the AXI master would be delayed 1 cycle before it can be accepted by the on-chip RAM.

This can avoid setup time issue for the write access on the memory cell at high frequency. Enabling this feature would cost at most 1 more clock cycle for each AXI write transaction, that is, at most 1 more clock cycle for each write burst with multiple beats of data.

When this feature is disabled, the write data from the AXI master can be accepted by the on-chip RAM without delay, and data can be written to memory at this cycle (if no other access and write address is also ready at this cycle).

4.5.3.4 Write Address Pipeline

When this feature is enabled, the write address from the AXI master would be delayed 1 cycle before it can be accepted by the on-chip RAM.

This can avoid setup time issue for the write access on the memory cell at high frequency. Enabling this feature would take at most 1 more clock cycle for each AXI write transaction, that is, at most 1 more clock cycle for each write burst with multiple beats of data.

When this feature is disabled, the write address from the AXI master can be accepted by the on-chip RAM without delay, and data can be written to memory at this cycle (if no other access and write data is also ready at this cycle).

4.5.4 Programmable Registers

There are no programmable registers in this block;

4.6 Network Interconnect Bus System (NOC)

4.6.1 Overview

This section provides an overview of the on chip Network Interconnection Bus System. The Bus System is composed of two kinds of IP

- NIC-301 (Network Inter-Connect) AXI arbiter IP

The NIC-301 (by Arm Ltd.) is a configurable AXI arbiter between several masters and slaves. The NIC-301 IP is designed so that many configuration options are

selected at the hardware design stage, determined by SoC characteristics and needs, while several other configuration options are software-controlled.

- NoC (Network on Chip) fabric IP

The NoC (by Arteris Ltd.) is a configurable high efficiency and performance fabric IP. Similar to the NIC-301 IP the majority design options are configured during hardware design stage. Meanwhile those application specific configurations are required to be configured by software.

NOTE

The NIC-301 and NoC default settings are configured by NXP's board support package (BSP), and in most cases should not be modified by the customer. The default settings have gone through exhaustive testing during the validation of the part, and have proven to work well for the part's intended target applications. Changes to the default settings may result in a degradation in system performance.

4.6.2 External Signals

There are no external I/O interfaces for NIC-301 and NoC.

4.7 AHB to IP Bridge (AIPSTZ)

4.7.1 Overview

This section provides an overview of the AHB to IP Bridge (AIPSTZ). This particular peripheral is designed as the bridge between AHB bus and peripherals with the lower bandwidth IP Slave (IPS) buses.

4.7.1.1 Features

The following list summarizes the key features of the bridge:

- The bridge supports the IPS slave bus signals.
- The bridge supports 8-, 16-, and 32-bit IPS peripherals. (Accesses larger than the size of a peripheral are not supported, except to 32-bit memory.)
- The bridge supports a pair of IPS accesses for 64-bit and certain misaligned AHB transfers to 32-bit memory in 64-bit platforms.

- The bridge directly supports up to 32 64-Kbyte external IPS peripherals, and 2 global external IPS peripheral spaces. The bridge occupies 1 MBytes of total address space.
- The bridge provides configurable per-block and per-master access protections. Access permissions are based on bus master (e.g. DMA or core) privilege levels and resource domain. More details on the protection features and configuration can be found in the Security Reference Manual
- Peripheral read transactions require a minimum of 2 hclk clocks, and unbuffered write transactions require a minimum of 3 hclk clocks.
- The bridge uses one single asynchronous reset and one global clock.

4.7.2 Clocks

The following table describes the clock sources for AIPSTZ. Please see [Clock Controller Module \(CCM\)](#) for clock setting, configuration and gating information.

Table 4-18. AIPSTZ Clocks

Clock name	Clock Root	Description
hclk	ahb_clk_root	Module clock

4.7.3 Functional Description

The AIPS bridge serves as a protocol translator between the AHB system bus and the IP bus.

Support is provided for generating a pair of 32-bit IP bus accesses when targeted by a 64-bit system bus access, or a misaligned access which crosses a 32-bit boundary. No other bus-sizing access support is provided.

The AHB to IP bridge is the interface between the AHB and on-chip IPS peripherals, which are sub-blocks containing readable/writable control and status registers.

The AHB master reads and writes these registers through the AIPSTZ. The bridge generates block enables, the block address, transfer attributes, byte enables and write data as inputs to the IPS peripherals. The bridge captures read data from the IPS interface and drives it on the AHB.

Each bridge that connects to the IPS (or peripherals) are referred as AIPS. The chip has several separate AIPS modules, and peripherals are grouped and assigned under each AIPS block. The list of peripherals are indicated as n-1, ... and n-x for AIPS-1, ... and AIPS-x respectively.

AIPS occupies a 1-Mbyte portion of the address space. The register maps of the IPS peripherals are located on 64-Kbyte boundaries. Each IPS peripheral is allocated one 64-Kbyte block of the memory map, and is activated by one of the block enables from the bridge. Up to thirty-two 64-Kbyte external IPS peripherals may be implemented, occupying contiguous blocks of 64-Kbytes. Two global external IPS block enables are available for the remaining address space to allow for customization and expansion of addressed peripheral devices. In addition, a single "non-global" block enable is also asserted whenever any of the thirty-two non-global block enables is asserted.

The bridge is responsible for indicating to IPS peripherals if an access is in supervisor or user mode. It may block user mode accesses to certain IPS peripherals or it may allow the individual IPS peripherals to determine if user mode accesses are allowed. In addition, peripherals may be designated as write-protected.

The bridge supports the notion of "trusted" masters for security purposes. Masters may be individually designated as trusted for reads, trusted for writes, or trusted for both reads and writes, as well as being forced to look as though all accesses from a master are in user-mode privilege level. Refer to [AIPSTZ Memory Map/Register Definition](#) for more information.

The AIPSTZ prevents access to a peripheral if the transaction originated from a source from a resource domain that has been explicitly omitted. Resource domains are assigned in the RDC submodule. Please refer to the RDC chapter for programming details.

All peripheral devices are expected to only require aligned accesses equal to or smaller in size than the peripheral size. An exception to this rule is supported for 32-bit peripherals to allow memory to be placed on the IPS.

4.7.4 Access Protections

The AIPSTZ bridge provides programmable access protections for both masters and peripherals. It allows the privilege level of a master to be overridden, forcing it to user-mode privilege, and allows masters to be designated as trusted or untrusted.

Peripherals may require supervisor privilege level for access, may restrict access to a trusted master only, and may be write-protected. IP bus peripherals are subject to access control policies set in both CSU registers and AIPSTZ registers. An access is blocked if it is denied by either policy.

Masters and peripherals are assigned to one or more resource domains in the RDC submodule (see the RDC chapter for details). Depending on RDC programming, masters transactions through the AIPSTZ may or may not be allowed access to peripherals in different resource domains.

4.7.5 Access Support

Aligned 64-bit accesses, aligned and misaligned word and half word accesses, as well as byte accesses are supported for 32-bit peripherals. Misaligned accesses are supported to allow memory to be placed on the IPS.

Peripheral registers must not be misaligned, although no explicit checking is performed by the AIPS bridge. The bridge will perform two IPS transfers for 64-bit accesses, word accesses with byte offsets of 1, 2, or 3, and for half word accesses with a byte offset of 3. All other accesses will be performed with a single IPS transfer.

Only aligned half word and byte accesses are supported for 16-bit peripherals. All other accesses types are unsupported, and results of such accesses are undefined. They are not terminated with an error response.

Only byte accesses are supported for 8-bit peripherals. All other accesses types are unsupported, and results of such accesses are undefined. They are not terminated with an error response.

4.7.6 Initialization Information

The AIPS bridge should be programmed before use.

The following registers should be initialized: The Master Privilege Registers (AIPSTZ_MPRs), the Peripheral Access Control registers (AIPSTZ_PACRs), and the Off-platform Peripheral Access Control registers (AIPSTZ_OPACRs) described in [AIPSTZ Memory Map/Register Definition](#).

4.7.6.1 Security Block

The AIPSTZ contains a security block that is connected to each off-platform peripheral. This block filters accesses based on write/read, non-secure, and supervisor signals.

Each peripheral can be individually configured to allow or deny each of the following transactions as described in the table below:

Table 4-19. Peripheral Access Configuration options

Config Bit	Write	Non-Secure	Supervisor	Meaning
0	0	0	0	Secure User Read
1	0	0	1	Secure Supervisor Read

Table continues on the next page...

Table 4-19. Peripheral Access Configuration options (continued)

Config Bit	Write	Non-Secure	Supervisor	Meaning
2	0	1	0	Non-Secure User Read
3	0	1	1	Non-Secure Supervisor Read
4	1	0	0	Secure User Write
5	1	0	1	Secure Supervisor Write
6	1	1	0	Non-Secure User Write
7	1	1	1	Non-Secure Supervisor Write

Each peripheral has a security configuration (sec_config_X) input for determining whether to allow or deny a given access type. These are 8-bit vectors, with each bit corresponding to one of the transactions above as listed in the Config Bit column of [Table 4-19](#). If the bit is asserted (1'b1), the transaction is allowed. If the bit is negated (1'b0), the transaction is not allowed.

For example, if peripheral 0 is configured as follows:

sec_config_0 [7:0] = 8'b0011_0011

This peripheral can only be accessed by secure transactions. Bits 0, 1, 4, and 5 are asserted and these bits refer to the four types of secure transactions. If an insecure transaction is attempted to this peripheral, it will result in an error.

Eight bits per peripheral across an entire system can result in a large number of configuration bits that must be assigned and controlled, most likely in a series of registers in another block. To reduce the number of register bits required predefined sets of security profiles can be defined and encapsulated in an external security translation block. The table below describes one set of security profiles that has been proposed for use with the AIPSTZ.

Table 4-20. Security Levels

CSU_SEC_LEVEL	Non-Secure User	Non-Secure Supervisor	Secure User	Secure Supervisor
0	RD+WR	RD+WR	RD+WR	RD+WR
1	NOT ALLOWED	RD+WR	RD+WR	RD+WR
2	Read Only	Read Only	RD+WR	RD+WR
3	NOT ALLOWED	Read Only	RD+WR	RD+WR
4	NOT ALLOWED	NOT ALLOWED	RD+WR	RD+WR
5	NOT ALLOWED	NOT ALLOWED	NOT ALLOWED	RD+WR
6	NOT ALLOWED	NOT ALLOWED	Read Only	Read Only
7	NOT ALLOWED	NOT ALLOWED	NOT ALLOWED	NOT ALLOWED

Information regarding CSU is provided in the Security Reference Manual. Contact your NXP representative for information about obtaining this document.

A 3-bit input, 8-bit output translation block can be used such that only three register bits are required to set the security profile and the translation block will drive the correct 8-bit configuration vector. Each peripheral connected to the AIPSTZ would require this translation block. The top level AIPSTZ has this three bit input line `csu_sec_level[2:0]' corresponding to each peripheral X.

4.7.7 AIPSTZ Memory Map/Register Definition

The memory map for the AIPS SW-visible registers is shown in the table below.

The MPROT and OPACR fields are 4 bits in width. Some bits may be reserved depending on device.

AIPSTZ memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3000_0000	Master Priviledge Registers (AIPSTZ_MPR)	32	R/W	7700_0000h	4.7.7.1/198
3000_0040	Off-Platform Peripheral Access Control Registers (AIPSTZ_OPACR)	32	R/W	4444_4444h	4.7.7.2/201
3000_0044	Off-Platform Peripheral Access Control Registers (AIPSTZ_OPACR1)	32	R/W	4444_4444h	4.7.7.3/204
3000_0048	Off-Platform Peripheral Access Control Registers (AIPSTZ_OPACR2)	32	R/W	4444_4444h	4.7.7.4/207
3000_004C	Off-Platform Peripheral Access Control Registers (AIPSTZ_OPACR3)	32	R/W	4444_4444h	4.7.7.5/210
3000_0050	Off-Platform Peripheral Access Control Registers (AIPSTZ_OPACR4)	32	R/W	4444_4444h	4.7.7.6/213

4.7.7.1 Master Priviledge Registers (AIPSTZ_MPR)

Each AIPSTZ_MPR specifies 16 4-bit fields defining the access privilege level associated with a bus master in the platform, as well as specifying whether write accesses from this master are bufferable shown in [Table 4-21](#)

The registers provide one field per bus master, where field 15 corresponds to master 15, field 14 to master 14,... field 0 to master 0 (typically the processor core). The master index allocation is shown in the table below.

Table 4-21. MPROT Field

Bit	Field	Description
3	MBW	Master Buffer Writes - This bit determines whether the AIPSTZ is enabled to buffer writes from this master.
2	MTR	Master Trusted for Reads - This bit determines whether the master is trusted for read accesses.
1	MTW	Master Trusted for Writes - This bit determines whether the master is trusted for write accesses.
0	MPL	Master Privilege Level - This bit determines how the privilege level of the master is determined.

NOTE

The reset value is set to 0000_0000_7700_0000, which makes master 0 and master 1 (Arm CORE) the trusted masters. Trusted software can change the settings after reset.

Table 4-22. Master Index Allocation

Master Index	Master Name	Comments
Master 0	All masters excluding Arm core	Share the same number allocation.
Master 1	Arm A53	
Master 4	SDMA	
Master 6	M4	

Address: 3000_0000h base + 0h offset = 3000_0000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W	MPROT0				MPROT1				MPROT2				MPROT3				Reserved				MPROT5				Reserved							
Reset	0	1	1	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

AIPSTZ_MPR field descriptions

Field	Description
31–28 MPROT0	Master 0 Privilege, Buffer, Read, Write Control xxx0 MPL0 — Accesses from this master are forced to user-mode (ips_supervisor_access is forced to zero) regardless of the hprot[1] access attribute. xxx1 MPL1 — Accesses from this master are not forced to user-mode. The hprot[1] access attribute is used directly to determine ips_supervisor_access. xx0x MTW0 — This master is not trusted for write accesses. xx1x MTW1 — This master is trusted for write accesses. x0xx MTR0 — This master is not trusted for read accesses. x1xx MTR1 — This master is trusted for read accesses.

Table continues on the next page...

AIPSTZ_MPR field descriptions (continued)

Field	Description
	0xxx MBW0 — Write accesses from this master are not bufferable 1xxx MBW1 — Write accesses from this master are allowed to be buffered
27–24 MPROT1	Master 1 Privilege, Buffer, Read, Write Control xxx0 MPL0 — Accesses from this master are forced to user-mode (ips_supervisor_access is forced to zero) regardless of the hprot[1] access attribute. xxx1 MPL1 — Accesses from this master are not forced to user-mode. The hprot[1] access attribute is used directly to determine ips_supervisor_access. xx0x MTW0 — This master is not trusted for write accesses. xx1x MTW1 — This master is trusted for write accesses. x0xx MTR0 — This master is not trusted for read accesses. x1xx MTR1 — This master is trusted for read accesses. 0xxx MBW0 — Write accesses from this master are not bufferable 1xxx MBW1 — Write accesses from this master are allowed to be buffered
23–20 MPROT2	Master 2 Privilege, Buffer, Read, Write Control xxx0 MPL0 — Accesses from this master are forced to user-mode (ips_supervisor_access is forced to zero) regardless of the hprot[1] access attribute. xxx1 MPL1 — Accesses from this master are not forced to user-mode. The hprot[1] access attribute is used directly to determine ips_supervisor_access. xx0x MTW0 — This master is not trusted for write accesses. xx1x MTW1 — This master is trusted for write accesses. x0xx MTR0 — This master is not trusted for read accesses. x1xx MTR1 — This master is trusted for read accesses. 0xxx MBW0 — Write accesses from this master are not bufferable 1xxx MBW1 — Write accesses from this master are allowed to be buffered
19–16 MPROT3	Master 3 Privilege, Buffer, Read, Write Control. xxx0 MPL0 — Accesses from this master are forced to user-mode (ips_supervisor_access is forced to zero) regardless of the hprot[1] access attribute. xxx1 MPL1 — Accesses from this master are not forced to user-mode. The hprot[1] access attribute is used directly to determine ips_supervisor_access. xx0x MTW0 — This master is not trusted for write accesses. xx1x MTW1 — This master is trusted for write accesses. x0xx MTR0 — This master is not trusted for read accesses. x1xx MTR1 — This master is trusted for read accesses. 0xxx MBW0 — Write accesses from this master are not bufferable 1xxx MBW1 — Write accesses from this master are allowed to be buffered
15–12 -	This field is reserved. Reserved
11–8 MPROT5	Master 5 Privilege, Buffer, Read, Write Control. xxx0 MPL0 — Accesses from this master are forced to user-mode (ips_supervisor_access is forced to zero) regardless of the hprot[1] access attribute. xxx1 MPL1 — Accesses from this master are not forced to user-mode. The hprot[1] access attribute is used directly to determine ips_supervisor_access. xx0x MTW0 — This master is not trusted for write accesses. xx1x MTW1 — This master is trusted for write accesses. x0xx MTR0 — This master is not trusted for read accesses.

Table continues on the next page...

AIPSTZ_MPR field descriptions (continued)

Field	Description
x1xx	MTR1 — This master is trusted for read accesses.
0xxx	MBW0 — Write accesses from this master are not bufferable
1xxx	MBW1 — Write accesses from this master are allowed to be buffered
-	This field is reserved. Reserved

4.7.7.2 Off-Platform Peripheral Access Control Registers (AIPSTZ_OPACR)

Each of the off-platform peripherals have an Off-platform Peripheral Access Control Register (AIPSTZ_OPACR) which defines the access levels supported by the given block.

Each AIPSTZ_OPACR has the following format shown in [Table 4-23](#)

Table 4-23. OPAC Field

Bit	Field	Description
3	BW	Buffer Writes - This bit determines whether write accesses to this peripheral are allowed to be buffered. ¹
2	SP	Supervisor Protect - This bit determines whether the peripheral requires supervisor privilege level for access.
1	WP	Write Protect - This bit determines whether the peripheral allows write accesses.
0	TP	Trusted Protect - This bit determines whether the peripheral allows accesses from an untrusted master.

1. Buffered writes are not available for AIPSTZ. This bit should be set to '0'.

Address: 3000_0000h base + 40h offset = 3000_0040h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																																
R																																																																
W	OPAC0								OPAC1								OPAC2								OPAC3								OPAC4								OPAC5								OPAC6								OPAC7							
Reset	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0																																

AIPSTZ_OPACR field descriptions

Field	Description
31–28 OPAC0	Off-platform Peripheral Access Control 0 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.

Table continues on the next page...

AIPSTZ_OPACR field descriptions (continued)

Field	Description
	x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
27–24 OPAC1	Off-platform Peripheral Access Control 1 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
23–20 OPAC2	Off-platform Peripheral Access Control 2 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
19–16 OPAC3	Off-platform Peripheral Access Control 3 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses.

Table continues on the next page...

AIPSTZ_OPACR field descriptions (continued)

Field	Description
	x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
15–12 OPAC4	Off-platform Peripheral Access Control 4 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
11–8 OPAC5	Off-platform Peripheral Access Control 5 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
7–4 OPAC6	Off-platform Peripheral Access Control 6 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the

Table continues on the next page...

AIPSTZ_OPACR field descriptions (continued)

Field	Description
	master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
0xxx	BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ.
1xxx	BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
OPAC7	Off-platform Peripheral Access Control 7
xxx0	TP0 — Accesses from an untrusted master are allowed.
xxx1	TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
xx0x	WP0 — This peripheral allows write accesses.
xx1x	WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
x0xx	SP0 — This peripheral does not require supervisor privilege level for accesses.
x1xx	SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
0xxx	BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ.
1xxx	BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.

4.7.7.3 Off-Platform Peripheral Access Control Registers (AIPSTZ_OPACR1)

Each of the off-platform peripherals have an Off-platform Peripheral Access Control Register (AIPSTZ_OPACR) which defines the access levels supported by the given block.

Each AIPSTZ_OPACR has the following format shown in [Table 4-23](#)

Address: 3000_0000h base + 44h offset = 3000_0044h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																																
R																																																																
W	OPAC8								OPAC9								OPAC10								OPAC11								OPAC12								OPAC13								OPAC14								OPAC15							
Reset	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0																																

AIPSTZ_OPACR1 field descriptions

Field	Description
31–28 OPAC8	Off-platform Peripheral Access Control 8
xxx0	TP0 — Accesses from an untrusted master are allowed.
xxx1	TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
xx0x	WP0 — This peripheral allows write accesses.
xx1x	WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.

Table continues on the next page...

AIPSTZ_OPACR1 field descriptions (continued)

Field	Description
	x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
27–24 OPAC9	Off-platform Peripheral Access Control 9 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
23–20 OPAC10	Off-platform Peripheral Access Control 10 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
19–16 OPAC11	Off-platform Peripheral Access Control 11 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses.

Table continues on the next page...

AIPSTZ_OPACR1 field descriptions (continued)

Field	Description
	x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
15–12 OPAC12	Off-platform Peripheral Access Control 12 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
11–8 OPAC13	Off-platform Peripheral Access Control 13 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
7–4 OPAC14	Off-platform Peripheral Access Control 14 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the

Table continues on the next page...

AIPSTZ_OPACR1 field descriptions (continued)

Field	Description
	master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
0xxx	BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ.
1xxx	BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
OPAC15	Off-platform Peripheral Access Control 15
xxx0	TP0 — Accesses from an untrusted master are allowed.
xxx1	TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
xx0x	WP0 — This peripheral allows write accesses.
xx1x	WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
x0xx	SP0 — This peripheral does not require supervisor privilege level for accesses.
x1xx	SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
0xxx	BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ.
1xxx	BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.

4.7.7.4 Off-Platform Peripheral Access Control Registers (AIPSTZ_OPACR2)

Each of the off-platform peripherals have an Off-platform Peripheral Access Control Register (AIPSTZ_OPACR) which defines the access levels supported by the given block.

Each AIPSTZ_OPACR has the following format shown in [Table 4-23](#)

Address: 3000_0000h base + 48h offset = 3000_0048h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																																
R																																																																
W	OPAC16								OPAC17								OPAC18								OPAC19								OPAC20								OPAC21								OPAC22								OPAC23							
Reset	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0																																

AIPSTZ_OPACR2 field descriptions

Field	Description
31–28 OPAC16	Off-platform Peripheral Access Control 16
xxx0	TP0 — Accesses from an untrusted master are allowed.
xxx1	TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
xx0x	WP0 — This peripheral allows write accesses.

Table continues on the next page...

AIPSTZ_OPACR2 field descriptions (continued)

Field	Description
	xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
27–24 OPAC17	Off-platform Peripheral Access Control 17 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
23–20 OPAC18	Off-platform Peripheral Access Control 18 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
19–16 OPAC19	Off-platform Peripheral Access Control 19 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.

Table continues on the next page...

AIPSTZ_OPACR2 field descriptions (continued)

Field	Description
	x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
15–12 OPAC20	Off-platform Peripheral Access Control 20 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
11–8 OPAC21	Off-platform Peripheral Access Control 21 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
7–4 OPAC22	Off-platform Peripheral Access Control 22 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses.

Table continues on the next page...

AIPSTZ_OPACR2 field descriptions (continued)

Field	Description
x1xx	SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
0xxx	BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ.
1xxx	BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
OPAC23	Off-platform Peripheral Access Control 23
xxx0	TP0 — Accesses from an untrusted master are allowed.
xxx1	TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
xx0x	WP0 — This peripheral allows write accesses.
xx1x	WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
x0xx	SP0 — This peripheral does not require supervisor privilege level for accesses.
x1xx	SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
0xxx	BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ.
1xxx	BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.

4.7.7.5 Off-Platform Peripheral Access Control Registers (AIPSTZ_OPACR3)

Each of the off-platform peripherals have an Off-platform Peripheral Access Control Register (AIPSTZ_OPACR) which defines the access levels supported by the given block.

Each AIPSTZ_OPACR has the following format shown in [Table 4-23](#)

Address: 3000_0000h base + 4Ch offset = 3000_004Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																																
R																																																																
W	OPAC24								OPAC25								OPAC26								OPAC27								OPAC28								OPAC29								OPAC30								OPAC31							
Reset	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0																																

AIPSTZ_OPACR3 field descriptions

Field	Description
31–28 OPAC24	Off-platform Peripheral Access Control 24
xxx0	TP0 — Accesses from an untrusted master are allowed.
xxx1	TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.

Table continues on the next page...

AIPSTZ_OPACR3 field descriptions (continued)

Field	Description
	xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
27–24 OPAC25	Off-platform Peripheral Access Control 25 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
23–20 OPAC26	Off-platform Peripheral Access Control 26 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
19–16 OPAC27	Off-platform Peripheral Access Control 27 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses.

Table continues on the next page...

AIPSTZ_OPACR3 field descriptions (continued)

Field	Description
	xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
15–12 OPAC28	Off-platform Peripheral Access Control 28 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
11–8 OPAC29	Off-platform Peripheral Access Control 29 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. x0xx SP0 — This peripheral does not require supervisor privilege level for accesses. x1xx SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. 0xxx BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ. 1xxx BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
7–4 OPAC30	Off-platform Peripheral Access Control 30 xxx0 TP0 — Accesses from an untrusted master are allowed. xxx1 TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus. xx0x WP0 — This peripheral allows write accesses. xx1x WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.

Table continues on the next page...

AIPSTZ_OPACR3 field descriptions (continued)

Field	Description
x0xx	SP0 — This peripheral does not require supervisor privilege level for accesses.
x1xx	SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
0xxx	BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ.
1xxx	BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
OPAC31	Off-platform Peripheral Access Control 31
xxx0	TP0 — Accesses from an untrusted master are allowed.
xxx1	TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
xx0x	WP0 — This peripheral allows write accesses.
xx1x	WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
x0xx	SP0 — This peripheral does not require supervisor privilege level for accesses.
x1xx	SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
0xxx	BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ.
1xxx	BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.

4.7.7.6 Off-Platform Peripheral Access Control Registers (AIPSTZ_OPACR4)

Each of the off-platform peripherals have an Off-platform Peripheral Access Control Register (AIPSTZ_OPACR) which defines the access levels supported by the given block.

Each AIPSTZ_OPACR has the following format shown in [Table 4-23](#)

Address: 3000_0000h base + 50h offset = 3000_0050h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0								
R	OPAC32								OPAC33								Reserved																							
W	OPAC32								OPAC33								Reserved																							
Reset	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0								

AIPSTZ_OPACR4 field descriptions

Field	Description
31–28 OPAC32	Off-platform Peripheral Access Control 32
xxx0	TP0 — Accesses from an untrusted master are allowed.

Table continues on the next page...

AIPSTZ_OPACR4 field descriptions (continued)

Field	Description
xxx1	TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
xx0x	WP0 — This peripheral allows write accesses.
xx1x	WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
x0xx	SP0 — This peripheral does not require supervisor privilege level for accesses.
x1xx	SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
0xxx	BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ.
1xxx	BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
27–24 OPAC33	Off-platform Peripheral Access Control 33
xxx0	TP0 — Accesses from an untrusted master are allowed.
xxx1	TP1 — Accesses from an untrusted master are not allowed. If an access is attempted by an untrusted master, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
xx0x	WP0 — This peripheral allows write accesses.
xx1x	WP1 — This peripheral is write protected. If a write access is attempted, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
x0xx	SP0 — This peripheral does not require supervisor privilege level for accesses.
x1xx	SP1 — This peripheral requires supervisor privilege level for accesses. The master privilege level must indicate supervisor via the hprot[1] access attribute, and the MPROTx[MPL] control bit for the master must be set. If not, the access is terminated with an error response and no peripheral access is initiated on the IPS bus.
0xxx	BW0 — Write accesses to this peripheral are not bufferable by the AIPSTZ.
1xxx	BW1 — Write accesses to this peripheral are allowed to be buffered by the AIPSTZ.
-	This field is reserved. Reserved

4.8 Shared Peripheral Bus Arbiter (SPBA)

4.8.1 Overview

The Shared Peripheral Bus Arbiter (SPBA) is a three-to-one IP Bus interface arbiter. Three masters arbitrate for shared peripheral access through the SPBA.

The SPBA has three primary functions:

- The IP Bus Line switches a master to one peripheral

- The Masters arbiter arbitrates between the three masters to solve concurrent access or restricted access to peripherals
- The Control Registers and Ownership Control includes a set of registers which are reachable through software and permit the access scheme to be defined for each peripheral (Resource Ownership and Access Control). It generates signals for the external steering logic of interrupts and DMA signals.

The figure below shows the SPBA block diagram

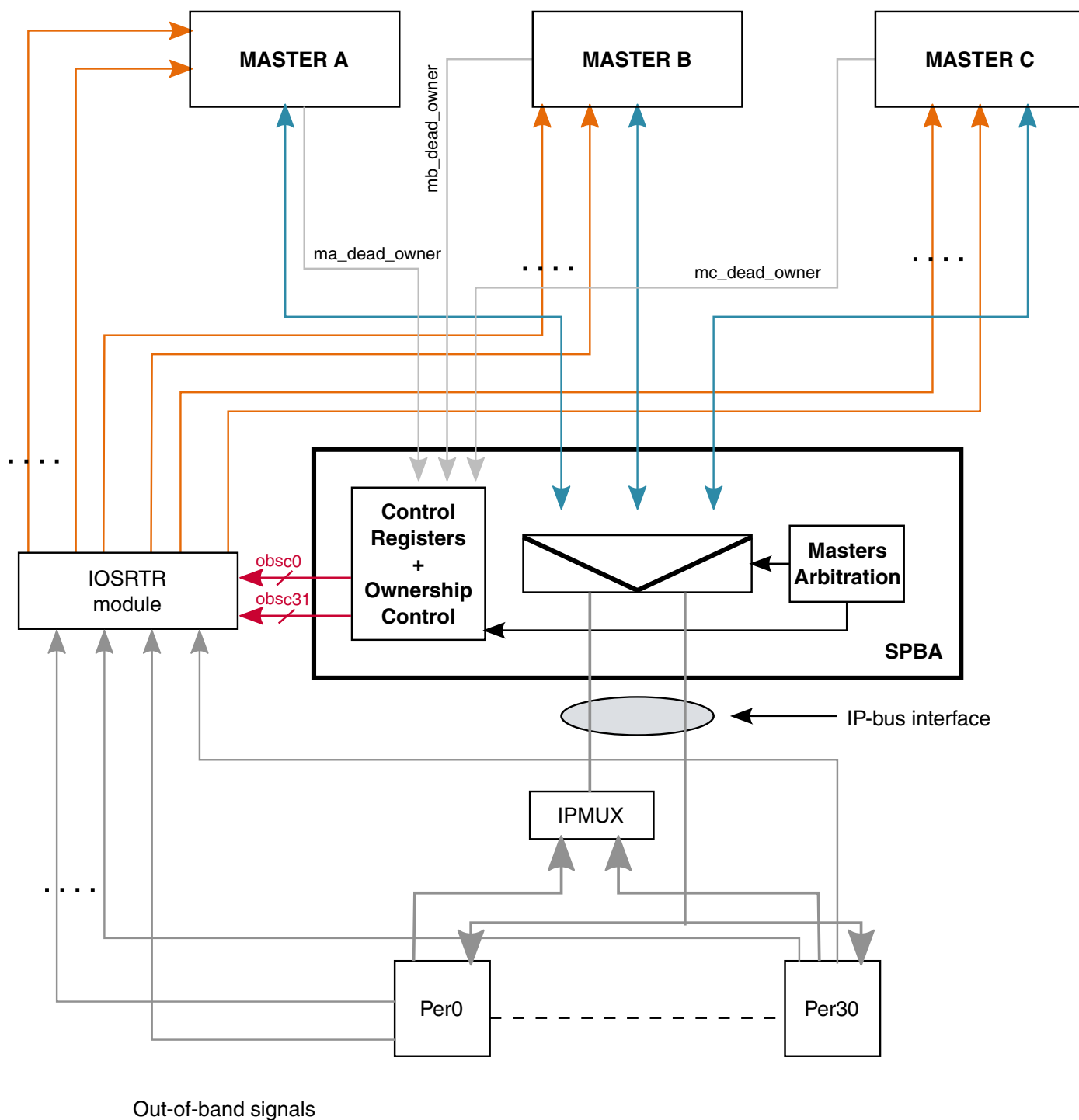


Figure 4-15. SPBA Block Diagram

4.8.1.1 Features

The SPBA includes the following features:

- Three IP Bus masters arbitration: Master A, B and C
- Support for DMA masters
- 32-bit data
- Supports up to 31 shared peripherals, each consuming 64 kilobytes of address space
- SPBA can be considered the 32nd peripheral, used for resource ownership and access control of the 31 peripherals
- Provides 31 sets of out of band steering control (OBSC) signals to the off-block steering logic
- Operating frequency up to 67 MHz
- Clocks: ipg_clk, ipg_clk_s

4.8.1.2 Modes of operation

SPBA behavior is transparent when accessing a peripheral, though it has these distinct modes of operation.

Reset/Abort

The SPBA has a hardware reset which initializes all registers, arbitration and peripherals rights registers (PRRs).

An abort signal input is provided allowing each master to abort its current access and release ownership (in case of master reset sequence).

Functional

Once a master request is granted, its IP Bus signals are steered to the requested peripheral.

Standby

No clock needed. The SPBA needs clocks only during access to the PRRs, arbitration, and abort phases. It generates two clock enable signals indicating when the clocks must be provided.

Configuration

During this phase, a master accesses the SPBA PRRs. The SPBA memory-mapped registers are seen as a shared peripheral.

4.8.2 Clocks

The table found here describes the clock sources for SPBA.

Please see [Clock Controller Module \(CCM\)](#) for clock setting, configuration and gating information.

Table 4-24. SPBA Clocks

Clock name	Clock Root	Description
ipg_clk	ipg_clk_root	Peripheral clock
ipg_clk_s	ipg_clk_root	Peripheral access clock

4.8.3 Functional description

4.8.3.1 Masters arbitration

The arbitration mechanism determines which port will control the master port, based on a simple round-robin arbitration scheme.

There are several use cases to consider.

- Only one master request per access. The master is switched to the shared peripheral bus, without arbitration. [Figure 4-16](#) shows the MB request on the global module enable signal, served without wait state.
- If two masters simultaneously access SPBA, the last granted master is held off using the <master>_ips_xfr_wait output signal (default value is high). When the master is granted sips_xfr_wait, shared IP Bus peripheral is connected to <master>_ips_xfr_wait outputs.
- If three masters simultaneously access SPBA, then the last two granted masters are held off using <master>_ips_xfr_wait. [Figure 4-17](#) shows a case in which the last two accesses granted are MA and MB. The requests are used even if they are in the same cycle.
- If after reset, at the first multiple access, no master has been granted, the priority is static: Master A (MA), Master B (MB) and last Master C (MC) port.
- No master request. No master switch to shared peripherals.

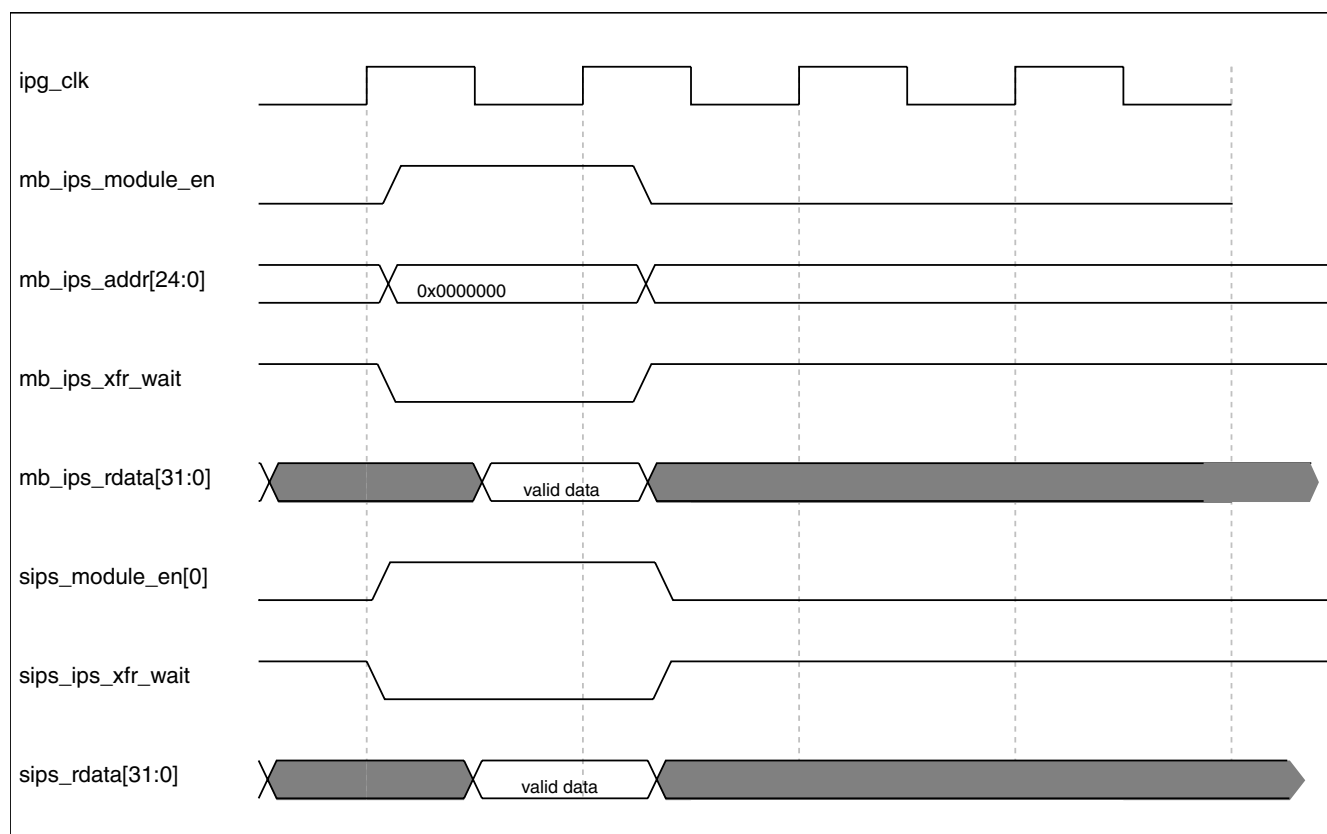


Figure 4-16. Example of one master request, no SPBA arbitration

The following figure assumes MA and MB have been the last two masters granted in the previous transfers (MA then MB).

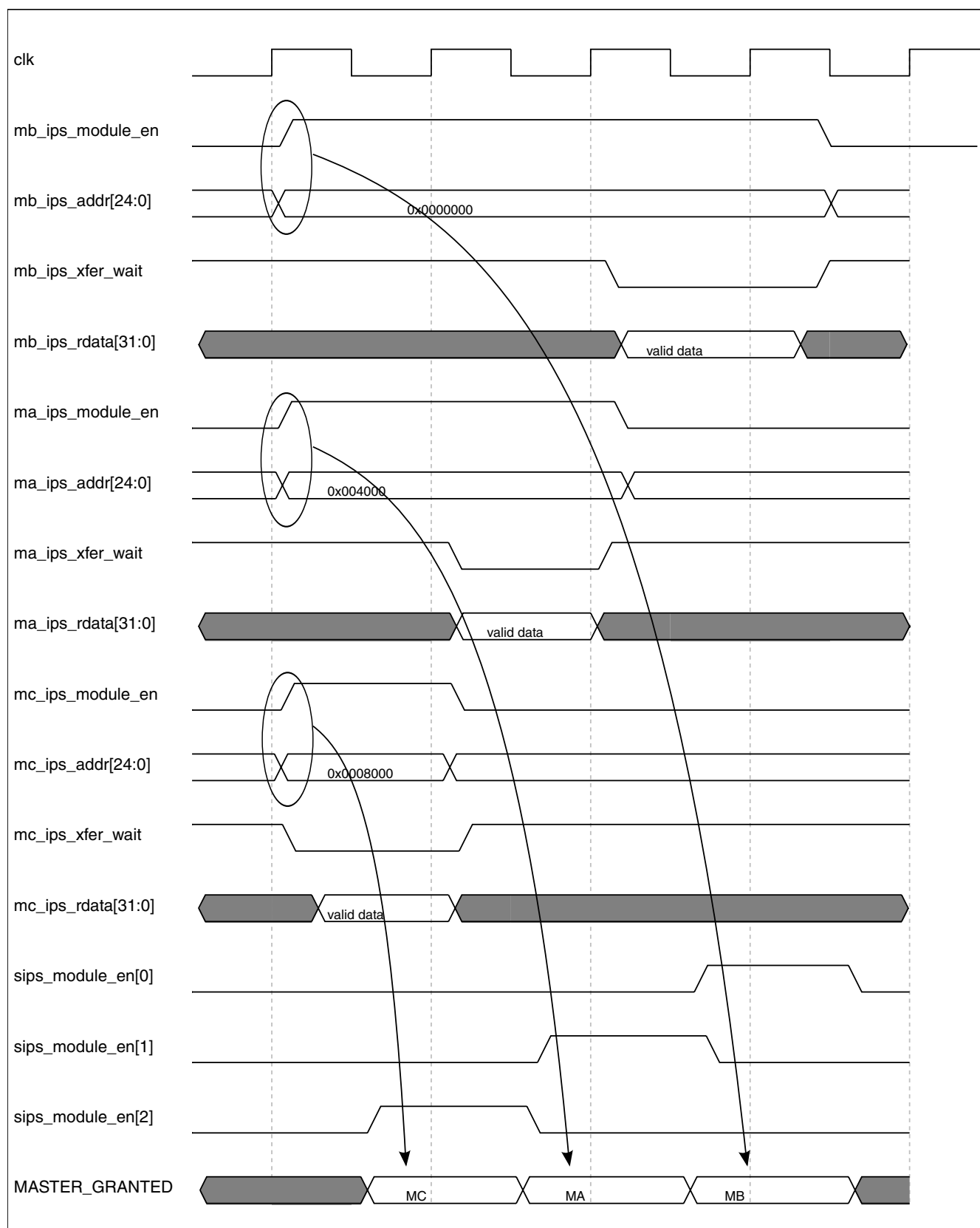


Figure 4-17. Example of three master requests: Masters already granted are "waited";

4.8.4 Resource ownership control

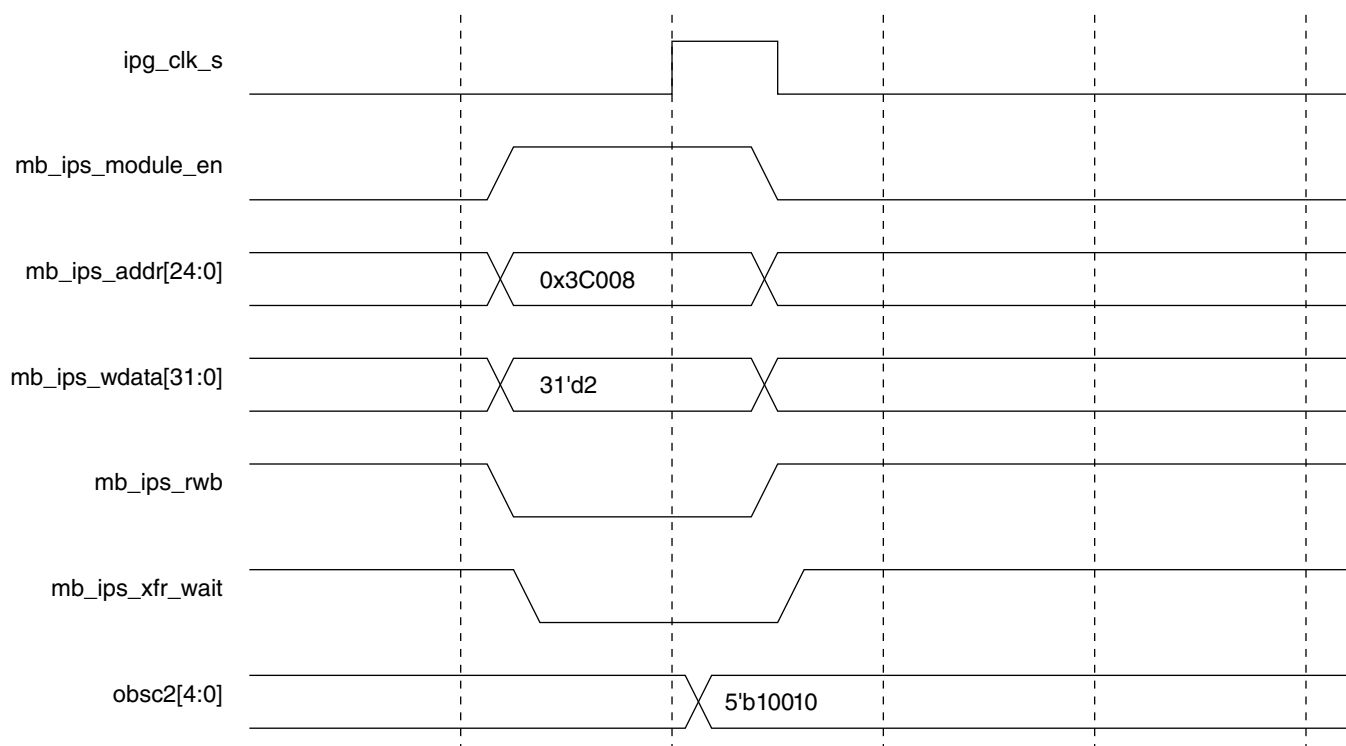
The resource ownership control regulates access to the shared peripherals and determines the steering of out-of-band signals.

4.8.4.1 Access control

4.8.4.1.1 Peripheral access

The peripheral access (resource access) of the requesting master is given by the corresponding RAR bit of the Peripheral Right Register. It determines if the master has access privilege to the resource.

Any attempt at access made by a requesting master whose access privilege bit is not set (in the PRR) is terminated with a bus error (<master>_ips_xfr_err is asserted by SPBA logic). The master that owns the resource can lock the peripheral for itself and/or grant other masters access to the peripheral by setting the appropriate bit(s) in the RAR field.



Master B is taking ownership of peripheral 2 by writing 3'b010 in the SPBA peripheral 2 right register (rarfield). This ownership can be checked on obsc2 output as roi2[1:0] = 2'b10 and rar2[2:0] = 3'b010 (obsc[4:0] = {roi2[1], roi2[0], rar2[2], rar2[1], rar2[0]}).

Figure 4-18. Example of one master B gaining ownership of peripheral 2

4.8.4.1.2 Peripheral Right Register access

The ROI bits of the Peripheral Right Register (PRR) determine which master is allowed to make write access to PRR. The identification of the requesting master is compared to the ROI bits of the PRR to determine if the master has ownership of the corresponding register.

Any attempted write access to a PRR already owned by another master will be ignored.

4.8.4.2 Owner election

When the peripheral is not owned by any master (ROI="00", after coming out of reset for instance), the first master to perform successfully a write to the RAR bits of the PRR is granted ownership of the peripheral and its associated PRR.

After writing to the PRR (RAR bit(s)), the master must read it back to make sure that it was granted ownership. If the RMO field is 2'b11, then the ownership claim is successful. If RMO is 2'b10, another master claimed ownership before this master was able to complete its write. This resolves the case in which two or more masters attempt to write the PRR at the same time; only the first master will be granted ownership. However all masters must read the PRR to determine if this case occurred, and if so, whether they were the first master which was granted ownership.

NOTE

A master that has been granted ownership of the PRR does not automatically have the right access to the peripheral; it must still set its own RAR bits in the PRR to access the peripheral.

4.8.4.3 Ending ownership

Ownership may be voluntarily ended by the owning master, or automatically upon assertion of a master-specific dead_owner signal.

The former is appropriate for software-controlled yielding of ownership. The latter is appropriate for automatic yielding of ownership when the owner has gone into reset.

When a master is reset, it clears the ROI bits of the PRRs owned by the corresponding master. When the owner is dead (in reset), all peripherals previously owned by that master must be changed to the un-owned state.

NOTE

It is the programmer's responsibility to make sure the peripherals are placed in an appropriate state before ending ownership.

4.8.4.3.1 Software Controlled Ownership Ending

The ROI bits will be automatically cleared when the master that owns the PRR access right clears (write) the RAR bits ([Table 2](#)).

It will then end the ownership of the PRR.

4.8.4.4 The Un-owned State

During the time when the peripheral is un-owned (i.e the ROI field contains all 0's), all masters have full access to it (RAR bits can then be modified by a master if ROI[1:0] = 2'b0).

In such cases it is necessary for software to ensure any necessary coherency in the resource, there is no hardware protection.

4.8.5 SPBA Memory Map/Register Definition

The SPBA control registers (Peripheral Right Registers) are mapped as a virtual shared peripheral.

SPBA can support up to 31 shared peripherals. Each of them has its own Peripheral Right Register (PRR) accessible within the SPBA memory-mapped registers, and consists of the Requesting Master Owner, the Resource Owner ID and the Resource Access Right fields.

SPBA memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
300F_0000	Peripheral Rights Register (SPBA2_PRR0)	32	R/W	0000_0007h	4.8.5.1/226
300F_0004	Peripheral Rights Register (SPBA2_PRR1)	32	R/W	0000_0007h	4.8.5.1/226
300F_0008	Peripheral Rights Register (SPBA2_PRR2)	32	R/W	0000_0007h	4.8.5.1/226
300F_000C	Peripheral Rights Register (SPBA2_PRR3)	32	R/W	0000_0007h	4.8.5.1/226
300F_0010	Peripheral Rights Register (SPBA2_PRR4)	32	R/W	0000_0007h	4.8.5.1/226
300F_0014	Peripheral Rights Register (SPBA2_PRR5)	32	R/W	0000_0007h	4.8.5.1/226
300F_0018	Peripheral Rights Register (SPBA2_PRR6)	32	R/W	0000_0007h	4.8.5.1/226
300F_001C	Peripheral Rights Register (SPBA2_PRR7)	32	R/W	0000_0007h	4.8.5.1/226
300F_0020	Peripheral Rights Register (SPBA2_PRR8)	32	R/W	0000_0007h	4.8.5.1/226
300F_0024	Peripheral Rights Register (SPBA2_PRR9)	32	R/W	0000_0007h	4.8.5.1/226
300F_0028	Peripheral Rights Register (SPBA2_PRR10)	32	R/W	0000_0007h	4.8.5.1/226
300F_002C	Peripheral Rights Register (SPBA2_PRR11)	32	R/W	0000_0007h	4.8.5.1/226
300F_0030	Peripheral Rights Register (SPBA2_PRR12)	32	R/W	0000_0007h	4.8.5.1/226
300F_0034	Peripheral Rights Register (SPBA2_PRR13)	32	R/W	0000_0007h	4.8.5.1/226
300F_0038	Peripheral Rights Register (SPBA2_PRR14)	32	R/W	0000_0007h	4.8.5.1/226
300F_003C	Peripheral Rights Register (SPBA2_PRR15)	32	R/W	0000_0007h	4.8.5.1/226

Table continues on the next page...

SPBA memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
300F_0040	Peripheral Rights Register (SPBA2_PRR16)	32	R/W	0000_0007h	4.8.5.1/226
300F_0044	Peripheral Rights Register (SPBA2_PRR17)	32	R/W	0000_0007h	4.8.5.1/226
300F_0048	Peripheral Rights Register (SPBA2_PRR18)	32	R/W	0000_0007h	4.8.5.1/226
300F_004C	Peripheral Rights Register (SPBA2_PRR19)	32	R/W	0000_0007h	4.8.5.1/226
300F_0050	Peripheral Rights Register (SPBA2_PRR20)	32	R/W	0000_0007h	4.8.5.1/226
300F_0054	Peripheral Rights Register (SPBA2_PRR21)	32	R/W	0000_0007h	4.8.5.1/226
300F_0058	Peripheral Rights Register (SPBA2_PRR22)	32	R/W	0000_0007h	4.8.5.1/226
300F_005C	Peripheral Rights Register (SPBA2_PRR23)	32	R/W	0000_0007h	4.8.5.1/226
300F_0060	Peripheral Rights Register (SPBA2_PRR24)	32	R/W	0000_0007h	4.8.5.1/226
300F_0064	Peripheral Rights Register (SPBA2_PRR25)	32	R/W	0000_0007h	4.8.5.1/226
300F_0068	Peripheral Rights Register (SPBA2_PRR26)	32	R/W	0000_0007h	4.8.5.1/226
300F_006C	Peripheral Rights Register (SPBA2_PRR27)	32	R/W	0000_0007h	4.8.5.1/226
300F_0070	Peripheral Rights Register (SPBA2_PRR28)	32	R/W	0000_0007h	4.8.5.1/226
300F_0074	Peripheral Rights Register (SPBA2_PRR29)	32	R/W	0000_0007h	4.8.5.1/226
300F_0078	Peripheral Rights Register (SPBA2_PRR30)	32	R/W	0000_0007h	4.8.5.1/226
300F_007C	Peripheral Rights Register (SPBA2_PRR31)	32	R/W	0000_0007h	4.8.5.1/226
308F_0000	Peripheral Rights Register (SPBA1_PRR0)	32	R/W	0000_0007h	4.8.5.1/226
308F_0004	Peripheral Rights Register (SPBA1_PRR1)	32	R/W	0000_0007h	4.8.5.1/226
308F_0008	Peripheral Rights Register (SPBA1_PRR2)	32	R/W	0000_0007h	4.8.5.1/226
308F_000C	Peripheral Rights Register (SPBA1_PRR3)	32	R/W	0000_0007h	4.8.5.1/226
308F_0010	Peripheral Rights Register (SPBA1_PRR4)	32	R/W	0000_0007h	4.8.5.1/226
308F_0014	Peripheral Rights Register (SPBA1_PRR5)	32	R/W	0000_0007h	4.8.5.1/226
308F_0018	Peripheral Rights Register (SPBA1_PRR6)	32	R/W	0000_0007h	4.8.5.1/226
308F_001C	Peripheral Rights Register (SPBA1_PRR7)	32	R/W	0000_0007h	4.8.5.1/226
308F_0020	Peripheral Rights Register (SPBA1_PRR8)	32	R/W	0000_0007h	4.8.5.1/226
308F_0024	Peripheral Rights Register (SPBA1_PRR9)	32	R/W	0000_0007h	4.8.5.1/226
308F_0028	Peripheral Rights Register (SPBA1_PRR10)	32	R/W	0000_0007h	4.8.5.1/226
308F_002C	Peripheral Rights Register (SPBA1_PRR11)	32	R/W	0000_0007h	4.8.5.1/226
308F_0030	Peripheral Rights Register (SPBA1_PRR12)	32	R/W	0000_0007h	4.8.5.1/226
308F_0034	Peripheral Rights Register (SPBA1_PRR13)	32	R/W	0000_0007h	4.8.5.1/226
308F_0038	Peripheral Rights Register (SPBA1_PRR14)	32	R/W	0000_0007h	4.8.5.1/226
308F_003C	Peripheral Rights Register (SPBA1_PRR15)	32	R/W	0000_0007h	4.8.5.1/226
308F_0040	Peripheral Rights Register (SPBA1_PRR16)	32	R/W	0000_0007h	4.8.5.1/226
308F_0044	Peripheral Rights Register (SPBA1_PRR17)	32	R/W	0000_0007h	4.8.5.1/226
308F_0048	Peripheral Rights Register (SPBA1_PRR18)	32	R/W	0000_0007h	4.8.5.1/226
308F_004C	Peripheral Rights Register (SPBA1_PRR19)	32	R/W	0000_0007h	4.8.5.1/226
308F_0050	Peripheral Rights Register (SPBA1_PRR20)	32	R/W	0000_0007h	4.8.5.1/226
308F_0054	Peripheral Rights Register (SPBA1_PRR21)	32	R/W	0000_0007h	4.8.5.1/226

Table continues on the next page...

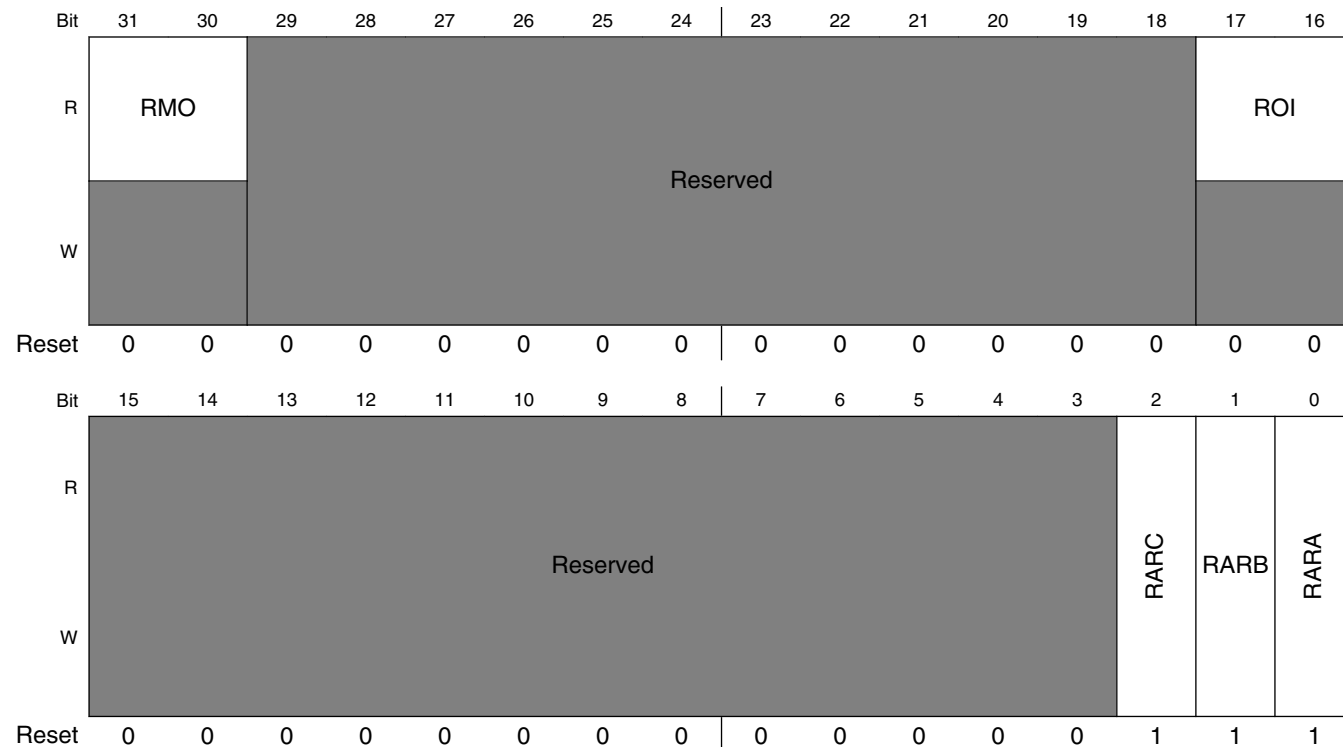
SPBA memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
308F_0058	Peripheral Rights Register (SPBA1_PRR22)	32	R/W	0000_0007h	4.8.5.1/226
308F_005C	Peripheral Rights Register (SPBA1_PRR23)	32	R/W	0000_0007h	4.8.5.1/226
308F_0060	Peripheral Rights Register (SPBA1_PRR24)	32	R/W	0000_0007h	4.8.5.1/226
308F_0064	Peripheral Rights Register (SPBA1_PRR25)	32	R/W	0000_0007h	4.8.5.1/226
308F_0068	Peripheral Rights Register (SPBA1_PRR26)	32	R/W	0000_0007h	4.8.5.1/226
308F_006C	Peripheral Rights Register (SPBA1_PRR27)	32	R/W	0000_0007h	4.8.5.1/226
308F_0070	Peripheral Rights Register (SPBA1_PRR28)	32	R/W	0000_0007h	4.8.5.1/226
308F_0074	Peripheral Rights Register (SPBA1_PRR29)	32	R/W	0000_0007h	4.8.5.1/226
308F_0078	Peripheral Rights Register (SPBA1_PRR30)	32	R/W	0000_0007h	4.8.5.1/226
308F_007C	Peripheral Rights Register (SPBA1_PRR31)	32	R/W	0000_0007h	4.8.5.1/226

4.8.5.1 Peripheral Rights Register (SPBAx_PRRn)

This register controls master ownership and access for a peripheral.

Address: Base address + 0h offset + (4d × i), where i=0d to 31d



SPBAX_PRRn field descriptions

Field	Description
31–30 RMO	Requesting Master Owner. This 2-bit register field indicates if the corresponding resource is owned by the requesting master or not. This register is reset to 2'b0 if ROI = 2'b0. 00 UNOWNED — The resource is unowned. 01 Reserved. 10 ANOTHER_MASTER — The resource is owned by another master. 11 REQUESTING_MASTER — The resource is owned by the requesting master.
29–18 -	This field is reserved. Reserved
17–16 ROI	Resource Owner ID. This field indicates which master (one at a time) can access to the PRR for rights modification. This is a read-only register. After reset, ROI bits are cleared ("00" -> un-owned resource). A master performing a write access to the an un-owned PRR will get its ID automatically written into ROI, while modifying RARx bits. It can then read back the RMO, RAR, ROI bits to make sure RMO returns the right value, ROI bits contain its ID and RARx bits are correctly asserted. Then no other master (whom ID is different from the one stored in ROI) will be able to modify RAR fields. Owner master of a peripheral can assert its dead_owner signal, or write 1'b0 in the RARx to release the ownership (ROI[1:0] reset to 2'b0). 00 UNOWNED — Unowned resource. 01 MASTER_A — The resource is owned by master A port. 10 MASTER_B — The resource is owned by master B port. 11 MASTER_C — The resource is owned by master C port.
15–3 -	This field is reserved. Reserved
2 RARC	Resource Access Right. Control and Status bit for master C. This field indicates whether master C can access the peripheral. From 0 up to 3 masters can have permission to access a resource (all the master can be granted on a peripheral, but only one access at a time will be granted by SPBA). 0 PROHIBITED — Access to peripheral is not allowed. 1 ALLOWED — Access to peripheral is granted.
1 RARB	Resource Access Right. Control and Status bit for master B. This field indicates whether master B can access the peripheral. From 0 up to 3 masters can have permission to access a resource (all the master can be granted on a peripheral, but only one access at a time will be granted by SPBA). 0 PROHIBITED — Access to peripheral is not allowed. 1 ALLOWED — Access to peripheral is granted.
0 RARA	Resource Access Right. Control and Status bit for master A. This field indicates whether master A can access the peripheral. From 0 up to 3 masters can have permission to access a resource (all the master can be granted on a peripheral, but only one access at a time will be granted by SPBA). 0 PROHIBITED — Access to peripheral is not allowed. 1 ALLOWED — Access to peripheral is granted.

4.9 ROM Controller with Patch (ROMCP)

4.9.1 Overview

The Read Only Memory Controller with ROM Patch (ROMC) acts as an interface between the Arm advanced high-performance bus (AHB - Lite) and the Read Only Memory. The ROMC consists of a ROM Controller and a ROM Patch. The ROM Patch is used to either patch code routines or fix data tables in the ROM area. There is an IP Bus interface to access the ROM Patch Registers. The following figure depicts the main functional sub-blocks of the ROMC.

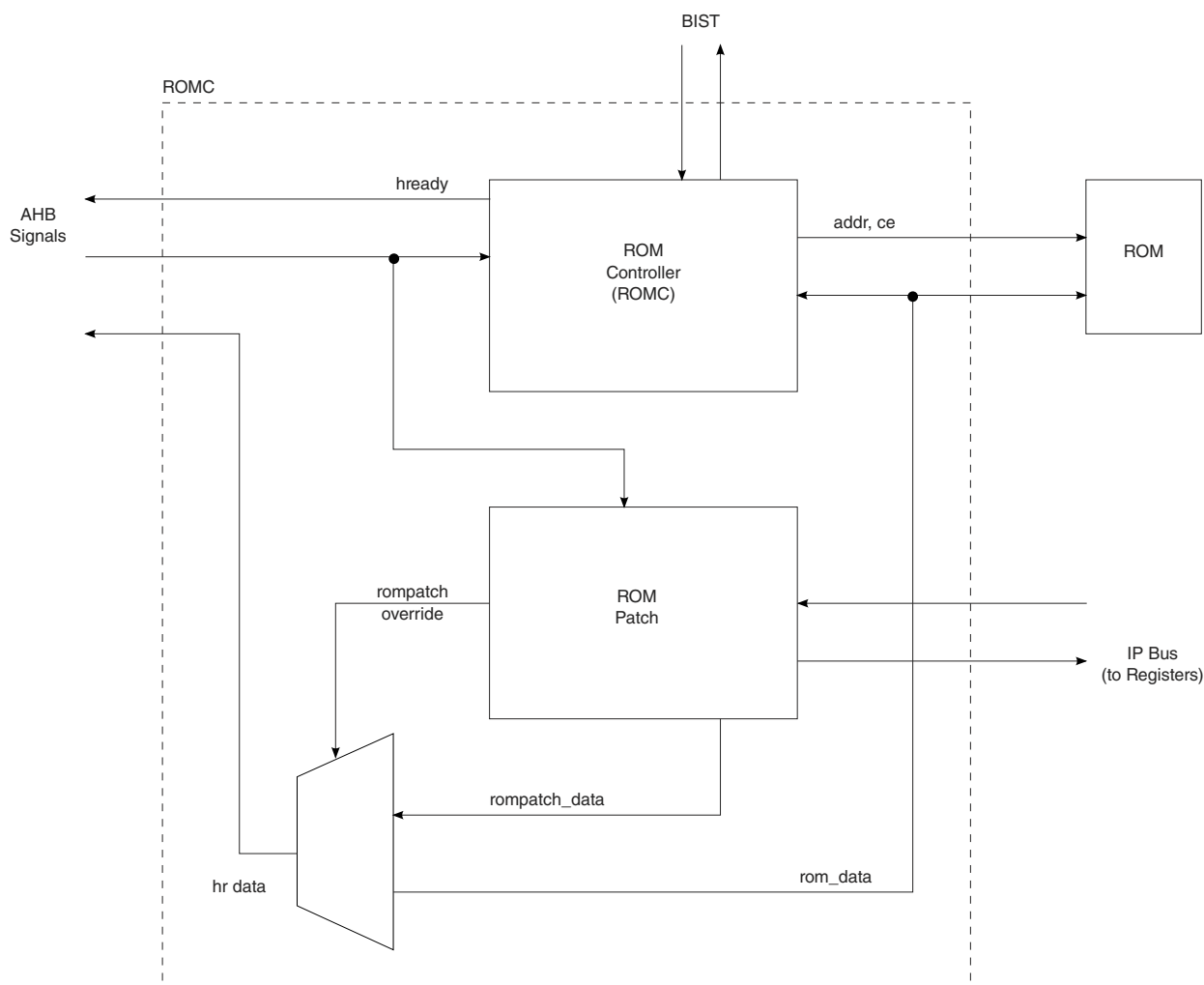


Figure 4-19. ROMC Block Diagram

4.9.1.1 Features

- Supports ROM size ranges from 16 Kbyte up to 4 Mbyte with increments of 1 Kbyte
- Supports opcode patching for a maximum of 16 different addresses in 4 Mbytes of ROM space
- Supports one-word data fixes for a max of 8 memory locations in 4 Mbytes of ROM space
- Supports patching of the Reset Vector (at 0x0000_0000) to allow external booting

4.9.1.2 Modes of Operation

There are two modes of operation: normal mode and BIST mode. In normal mode, the ROMC ensures correct reads from the ROM, assuming the memory complies with the characteristics and requirements for which the ROMC was designed.

4.9.1.2.1 Low Power Mode

There are two clock enables that are used to switch off parts of the ROMC logic when inactive. The first clock enable is used to disable the ROM Controller when the master connected to the AHB interface is not initiating a read to the ROM. The second clock enable is used to disable the registers used to program the ROM patch feature when the registers are not being accessed.

4.9.2 Clocks

The table found here describes the clock sources for ROMCP.

Please see [Clock Controller Module \(CCM\)](#) for clock setting, configuration and gating information.

Table 4-25. ROMCP Clocks

Clock name	Clock Root	Description
hclk	ahb_clk_root	System / bus clock
hclk_reg	ipg_clk_root	System access clock
96krom_CLK	ahb_clk_root	ROM clock

4.9.3 Memory Map

4.9.3.1 ROM Memory Map in detail

The ROMC supports ROM sizes with a range of 16 Kbyte to 4 Mbyte with an increment of 1 Kbyte. The 16 Kbyte lower limit was chosen because the minimum size of security code on an Arm platform is approximately 16 Kbyte of code, which is only accessible in supervisor mode. Note that it is the MMU that controls whether any region of memory is secure.

The exception vectors must be secured as well, and must be put in the same area as the security code. Since they must reside at address 0x0000_0000, the entire 16 Kbyte of ROM which can only be accessible in supervisor mode is located at the very beginning of the platform memory map.

If the user chooses not to use the security code, a memory size smaller than 16 Kbyte can be connected to the platform (minimum of 1 Kbyte). The MMU can be programmed to allow any kind of access into this memory. However, if the ROM size is less than 16 Kbyte, memory aliasing will occur for all invalid addresses greater than the memory size but within the 16 Kbyte of space.

For ROM sizes bigger than 16 Kbyte, the rest of its physical size resides at the address starting at 0x0040_4000 (4 M+16 Kbyte) going up to [0x0040_4000 + (mem. size - 16Kbyte)].

4.9.4 Functional Description

This section is divided up into the ROM Controller Functional Description and the ROMC functional description.

4.9.4.1 ROM Controller (ROMC) Functional Description

4.9.4.1.1 Functionality overview

The ROMC serves two main functions. First, as an interface between the AHB-Lite bus on an Arm platform and the ROM. Second, it drives and receives several signals for the BIST engine. In normal mode of operation, the ROMC monitors the AHB-Lite for memory access requests and performs the memory operation to the ROM.

The ROMC includes the option to wait state all accesses from either the Arm or non-Arm masters to ROM in the event that timing requirements will not allow single HCLK clock cycle reads. If a wait state is required, the static inputs `rom_wait_arm` or

rom_wait_alt_mstr can be set to 1 and accesses will take two HCLK clock cycles. If wait states are not required, rom_wait_arm or rom_wait_alt_mstr can be set to 0 and accesses will take one HCLK clock cycle to complete.

4.9.4.2 ROMC Functional Description

4.9.4.2.1 ROMC Disabling

All the bits in the ROMC_ROMPATCHENL register are cleared on Reset, disabling all the address comparators. Once the comparators have been enabled, the ROMC functions of data fixing and opcode patching can be quickly disabled by setting the DIS bit in the ROMC_ROMPATCHCNTL register. This bit is used to enable secure operations in which patching functions need to be disabled. This bit is cleared on Reset.

4.9.4.2.2 ROMC Event Priority

The ROMC has a total of 16 address comparators. The first 8 (0 through 7) comparators can be programmed for the data fixing function (through the 8 data fix enable bits in the ROMC_ROMPATCHCNTL register) while the rest are for opcode patching by default. This allows for potential multiple matching events involving both data fixing and opcode patch types. In these cases the ROMC assigns the highest priority to a data fixing event.

For example, if the ROMC is set up to data fix a certain address with comparator 4 and also opcode patch the same address with comparator 7, it will let comparator 4 have higher priority in indicating a match, and data from ROMC_ROMPATCHD4 will be put on the rompatch_romc_hrdata bus as the override value.

If multiple address matches of the same type level occur concurrently, then the ROMC will choose the source number based on the one with the highest source number. For example, the ROMC is setup to data fix the same location with address comparators 4 and 7, then address comparator 7 will have higher priority in indicating a match, and the value from ROMC_ROMPATCHD7 will be put on the rompatch_romc_hrdata bus as the override value. The same priority applies for an opcode patch event, except the override data is in the form of an SWI instruction with the comment field set to the source number with the highest priority.

4.9.4.2.3 Data Fixing

The data fixing feature allows ROM data to be updated by direct replacement when it is being read. This data usually originates from data tables, but can include Arm instructions. To enable data fixing on a certain address, this address value is written in to one of the first eight (0 through 7) of ROMC_ROMPATCHAxx registers and the same

numbered bit set in the ROMC_ROMPATCHENL and ROMC_ROMPATCHCNTL registers. The data to be used for replacement is placed in the corresponding ROMC_ROMPATCHDxx.

The ROMC looks for a read access to ROM (either code fetch or data load) by snooping the AHB interface for read transactions. The address is compared with the values stored in the ROMC_ROMPATCHAxx[22:2] registers. If a match occurs from one of the comparators, the ROMC places the value in the corresponding ROMC_ROMPATCHDxx register on the read data bus by overriding the read data coming from the actual ROM (see the mux in [Figure 4-19](#)). The value on the read data bus is maintained until hready is asserted to terminate the access. In data fixing, the entire word is replaced so if a byte or half-word access occurs on a "data fix" location, the entire data word is replaced. The word being replaced is word aligned. (The two LSBs of the matching ROMC_ROMPATCHAxx are ignored in the data fix operation.)

4.9.4.2.4 Opcode Patching

The opcode patch feature provides the Arm core a mechanism to fetch updated versions of code routines that were originally programmed in ROM. This patching mechanism makes use of the SWI (software interrupt instruction) and a table of function pointers residing in writable memory. The opcode being patched is replaced with a SWI instruction by the ROMC. Subsequent processing of the SWI reads from a function pointers table to obtain the address of the replacement code. Execution resumes with this code patch.

To enable opcode patching of a certain address, this address value is written into one of the ROMPATCHAxx registers and the corresponding bit set in the ROMPATCHENL to enable the associated comparator. The register's LSB (ROMC_ROMPATCHxx[0]) should be set if THUMB mode patching is in effect for this address. The ROMC identifies a ROM read access by snooping the AHB interface. The address is compared with the values stored in the ROMC_ROMPATCHAxx[22:2] registers. If a match occurs from one of the comparators, the ROMC generates the opcode of a software interrupt (SWI) instruction with the comment field containing the number of the matching address comparator. This opcode and comment is placed on the read data bus until hready is asserted by the ROM controller to terminate the read access.

The type of SWI generated, (that is, either Arm or THUMB), is determined by the LSB of the ROMC_ROMPATCHAxx register associated with the opcode patch. This bit is cleared for Arm mode (32 bits). The ROMC generates a 32-bit SWI (opcode field is 0xEF, occupying bits [31:24] of the word), with the least significant 5 bits of the 24-bit comment field (bits [23:0]) containing the number of the matching address comparator.

The rest of the comment field is filled with zeros. This means that the ROMC will use 16 of the 16777216 possible software interrupts. The ROMC overrides the read data from the ROM.

If the LSB of the matching ROMC_ROMPATCHAxx register is set, the opcode patch is in THUMB mode (16 bits or half word). The ROMC generates a 16-bit SWI instruction (opcode field is 0xDF, occupying bits [15:8] of the half word) with the least significant 5 bits of the 8-bit comment field containing with the source number of the address comparator. The rest of the comments field is filled with zeros. This means that the ROMC will use 16 of the 256 possible software interrupts. The ROMC puts this 16 bit SWI instruction value on the proper half of the rompatch_romc_hrdata bus. The other half is zeroed out. Which half of the bus contains the SWI opcode and comment depends on the mode (Big Endian or Little Endian) and the bit 1 of the matching ROMC_ROMPATCHAxx register. In Little Endian mode, the lower half is bits {15:0} and the upper half is bits {31:16}. The order is reversed in Big Endian mode.

In Little Endian mode (bigend signal negated), if bit 1 of the matching ROMC_ROMPATCHAxx is cleared (lower half word selected) then the SWI instruction is put on the lower 16 bits of the read data bus and the upper 16 bits are zeroed out. Only the lower 16 bits of the read data bus is overwritten by the ROMC data. If ROMC_ROMPATCHAxx[1] is set (upper half word selected), the SWI instruction is put on the upper 16 bits of the read data bus and the lower 16 bits are zeroed out. Only the upper 16 bits of the read data bus is overwritten.

In Big Endian mode (bigend asserted), if bit 1 of the matching ROMC_ROMPATCHAxx is cleared (lower half word selected) then the SWI instruction is put on the upper 16 bits of the read data bus while the lower 16 bits are zeroed out. Only the upper 16 bits of the read data bus is overwritten. If ROMC_ROMPATCHAxx[1] is set (upper word selected), the SWI instruction is put on the lower 16 bits and the upper 16 bits are zeroed out. Only the lower 16 bits of the read data bus is overwritten.

The eventual execution of the SWI causes the Arm to save the CPSR in SPSR_SVC, the address of the next instruction after the SWI in R14_SVC, enter Supervisor mode, and fetch the SWI vector at 0x8, which then takes it to a handler for further processing as described in the next section.

4.9.4.2.4.1 Typical Software Response to Opcode Patch

When the SWI handler executes it needs to determine whether the SWI was generated by the ROMC. This is done by loading the SWI instruction and extracting its comment field. The state of the Arm core (Arm or THUMB) when the SWI was executed dictates

whether to load the instruction word (Arm) or half word (THUMB). This state information can be determined by testing the T bit (bit 5) of the SPSR. If it's set, the execution was in THUMB mode.

By convention, if the comment field of the SWI is greater than 16, the software interrupt was initiated by software (i.e. an operating system call), and a branch is taken to the appropriate handler routine for further processing. If the comment field is less than 16, the SWI was generated by the ROMC performing a code patch operation. In this case, the software then reads from a table of function pointers, using the value in the SWI comment field as the index into the table. The value that is read is the address of the code patch. This value is loaded into the PC to begin the execution of the code patch. The following code segment illustrates a typical handling of the SWI.

```

stmfd      sp!, {r0-r1,lr}      @ push register onto SWI stack
mrs        r0, spsr             @ get saved status register
tst        r0, #0x20            @ check if call was in THUMB mode
ldrneh     r0, [lr,#-2]         @ yes: load opcode half-word and
bicne      r0, r0, #0xff00      @ yes: extract THUMB comment
ldreq      r0, [lr,#-4]         @ no: load opcode word and
biceq      r0, r0, #0xff000000  @ no: extract ARM comment
                                @ now r0 has comment field
cmp        r0, #16             @ compare to 16 (maximum for ROMC)
ldrlt      lr, =rompatch_tbl_ptr @ < 16: get top of current ROMC
                                @ table; global variable which is
                                @ changeable per context
ldrlt      r1, [lr, r0, lsl #2] @ < 16: read function pointer from
                                @ table assumed an array of pointers
                                @ patch functions
strlt      r1, [sp, #8]        @ < 16: store function pointer onto
                                @ stack in position of link register
ldmpltfd   sp!, {r0-r1,pc}^     @ < 16: "fake" return from SWI, will
                                @ vector core to appropriate patch
                                @ function and set core back to previous
                                @ mode of operating
ldr        r1, =swi_hdlr       @ >= 16: pointer to standard SWI
                                @ handler
mov        lr, pc              @ >= 16: set link register
bx         r1                  @ >= 16: jump to standard SWI
                                @ handler
ldmfd      sp!, {r0-r1,pc}^     @ >= 16: pop registers from stack

```

4.9.4.2.5 External Boot Feature

Following a Reset event, the Arm issues an instruction fetch of the Reset Vector from address 0x0. This instruction, normally residing in ROM is usually a branch to a Reset handler or boot code which also normally resides in ROM. The ROMC external boot feature allows the bypassing of this code, using a different boot code residing perhaps in external memory.

This feature uses the data fix mechanism and works as follows: if the boot_int signal is negated when a Reset event occurred, the ROMC will perform a data fix of the Reset Vector at 0x0 with the following instruction (opcode 0xE59FF00C):

```

ldr        pc, [pc, #12]        @ read 0x0000_0014 for reset_vector

```

The value of PC when this instruction is executed is 8 so that a PC relative offset of 12 makes the source address 20 or 0x14. When this instruction executes, the Arm core reads from address 0x0000_0014, triggering a ROMC data fix operation which places the value taken from the external boot address on the read data bus, with the two LSBs zeroed out. This value is returned to the Arm to be placed in the PC causing code fetch and execution to start from that address.

4.9.4.2.6 Alternate Masters and ROMC

The ROMC sits on the AHB bus of the internal ROM (ROMC). This means that the ROMC can modify values on the read data bus going to the master. Therefore, any master which reads an opcode patched or data patched location will read patched data.

4.9.5 ROMCP Memory Map/Register Definition

All registers are accessible through an IP Bus and can only be accessed in privileged mode. These registers can only be written with 32-bits stores and are clocked by hclk_reg.

ROMC memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3031_00D4	ROMC Data Registers (ROMC_ROMPATCH7D)	32	R/W	0000_0000h	4.9.5.1/236
3031_00D8	ROMC Data Registers (ROMC_ROMPATCH6D)	32	R/W	0000_0000h	4.9.5.1/236
3031_00DC	ROMC Data Registers (ROMC_ROMPATCH5D)	32	R/W	0000_0000h	4.9.5.1/236
3031_00E0	ROMC Data Registers (ROMC_ROMPATCH4D)	32	R/W	0000_0000h	4.9.5.1/236
3031_00E4	ROMC Data Registers (ROMC_ROMPATCH3D)	32	R/W	0000_0000h	4.9.5.1/236
3031_00E8	ROMC Data Registers (ROMC_ROMPATCH2D)	32	R/W	0000_0000h	4.9.5.1/236
3031_00EC	ROMC Data Registers (ROMC_ROMPATCH1D)	32	R/W	0000_0000h	4.9.5.1/236
3031_00F0	ROMC Data Registers (ROMC_ROMPATCH0D)	32	R/W	0000_0000h	4.9.5.1/236
3031_00F4	ROMC Control Register (ROMC_ROMPATCHCNTL)	32	R/W	0840_0000h	4.9.5.2/237
3031_00F8	ROMC Enable Register High (ROMC_ROMPATCHENH)	32	R	0000_0000h	4.9.5.3/238
3031_00FC	ROMC Enable Register Low (ROMC_ROMPATCHENL)	32	R/W	0000_0000h	4.9.5.4/238
3031_0100	ROMC Address Registers (ROMC_ROMPATCH0A)	32	R/W	0000_0000h	4.9.5.5/239
3031_0104	ROMC Address Registers (ROMC_ROMPATCH1A)	32	R/W	0000_0000h	4.9.5.5/239
3031_0108	ROMC Address Registers (ROMC_ROMPATCH2A)	32	R/W	0000_0000h	4.9.5.5/239
3031_010C	ROMC Address Registers (ROMC_ROMPATCH3A)	32	R/W	0000_0000h	4.9.5.5/239
3031_0110	ROMC Address Registers (ROMC_ROMPATCH4A)	32	R/W	0000_0000h	4.9.5.5/239

Table continues on the next page...

ROMC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3031_0114	ROMC Address Registers (ROMC_ROMPATCH5A)	32	R/W	0000_0000h	4.9.5.5/239
3031_0118	ROMC Address Registers (ROMC_ROMPATCH6A)	32	R/W	0000_0000h	4.9.5.5/239
3031_011C	ROMC Address Registers (ROMC_ROMPATCH7A)	32	R/W	0000_0000h	4.9.5.5/239
3031_0120	ROMC Address Registers (ROMC_ROMPATCH8A)	32	R/W	0000_0000h	4.9.5.5/239
3031_0124	ROMC Address Registers (ROMC_ROMPATCH9A)	32	R/W	0000_0000h	4.9.5.5/239
3031_0128	ROMC Address Registers (ROMC_ROMPATCH10A)	32	R/W	0000_0000h	4.9.5.5/239
3031_012C	ROMC Address Registers (ROMC_ROMPATCH11A)	32	R/W	0000_0000h	4.9.5.5/239
3031_0130	ROMC Address Registers (ROMC_ROMPATCH12A)	32	R/W	0000_0000h	4.9.5.5/239
3031_0134	ROMC Address Registers (ROMC_ROMPATCH13A)	32	R/W	0000_0000h	4.9.5.5/239
3031_0138	ROMC Address Registers (ROMC_ROMPATCH14A)	32	R/W	0000_0000h	4.9.5.5/239
3031_013C	ROMC Address Registers (ROMC_ROMPATCH15A)	32	R/W	0000_0000h	4.9.5.5/239
3031_0208	ROMC Status Register (ROMC_ROMPATCHSR)	32	w1c	0000_0000h	4.9.5.6/240

4.9.5.1 ROMC Data Registers (ROMC_ROMPATCHnD)

The ROMC data registers (ROMC_ROMPATCH 7D through ROMC_ROMPATCH 0D) store the data to use for the 8 1-word data fix events. Each register is associated with an address comparator (7 through 0). When a data fixing event occurs, the value in the data register corresponding to the comparator that has the address match is put on the romc_hrdata[31:0] bus until romc_hready is asserted by the ROM controller to terminate the access. A MUX external to the ROMC will select this data over that of romc_hrdata[31:0] in returning read data to the Arm core. The selection is done with the control bus rompatch_romc_hrdata_ovr[1:0] with both bits asserted by the ROMC.

If more than one address comparators match, the highest-numbered one takes precedence, and the value in corresponding data register is used for the patching event.

Address: 3031_0000h base + D4h offset + (4d × i), where i=0d to 7d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DATAx																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

ROMC_ROMPATCHnD field descriptions

Field	Description
DATAx	Data Fix Registers - Stores the data used for 1-word data fix operations. The values stored within these registers do not affect the writes to the memory system. They are selected over the read data from ROM when a data fix event occurs. If any part of the 1-word data fix is read, then the entire word is replaced. Therefore, a byte or half-word read will cause the ROMC to replace the entire word. The word is word address aligned.

4.9.5.2 ROMC Control Register (ROMC_ROMPATCHCNTL)

The ROMC control register (ROMC_ROMPATCHCNTL) contains the block disable bit and the data fix enable bits. The block disable bit provides a means to disable the ROMC data fix and opcode patching functions, even when the address comparators are enabled. The External Boot feature is not affected by this bit. The eight data fix enable bits (0 through 7), when set, assign the associated address comparators to data fix operations

NOTE

Bits 27 and 22 always read as 1s.

Address: 3031_0000h base + F4h offset = 3031_00F4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved		DIS	Reserved												
W																
Reset	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								DATAFIX							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

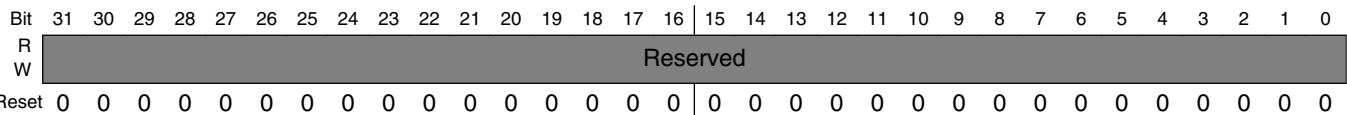
ROMC_ROMPATCHCNTL field descriptions

Field	Description
31–30 -	This field is reserved. Reserved
29 DIS	ROMC Disable -- This bit, when set, disables all ROMC operations. This bit is used to enable secure operations. 0 Does not affect any ROMC functions (default) 1 Disable all ROMC functions: data fixing, and opcode patching
28–8 -	This field is reserved. Reserved
DATAFIX	Data Fix Enable - Controls the use of the first 8 address comparators for 1-word data fix or for code patch routine. 0 Address comparator triggers a opcode patch 1 Address comparator triggers a data fix

4.9.5.3 ROMC Enable Register High (ROMC_ROMPATCHENH)

The ROMC enable register high (ROMC_ROMPATCHENH) and ROMC enable register low (ROMC_ROMPATCHENL) control whether or not the associated address comparator can trigger a opcode patch or data fix event. This implementation of the ROMC only has 16 comparators, therefore ROMC_ROMPATCHENH and the upper half of ROMC_ROMPATCHENL are read-only. ROMC_ROMPATCHENL[15:0] are associated with comparators 15 through 0. ROMC_ROMPATCHENLH[31:0] would have been associated with comparators 63 through 32.

Address: 3031_0000h base + F8h offset = 3031_00F8h



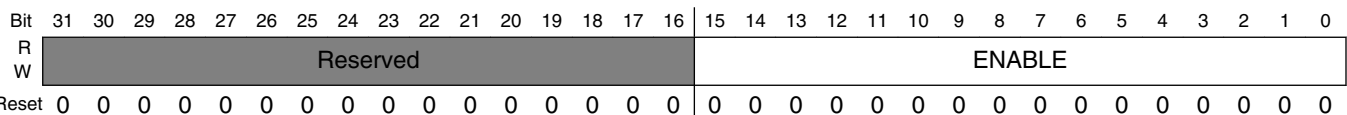
ROMC_ROMPATCHENH field descriptions

Field	Description
-	This field is reserved. Reserved

4.9.5.4 ROMC Enable Register Low (ROMC_ROMPATCHENL)

The ROMC enable register high (ROMC_ROMPATCHENH) and ROMC enable register low (ROMC_ROMPATCHENL) control whether or not the associated address comparator can trigger a opcode patch or data fix event. This implementation of the ROMC only has 16 comparators, therefore ROMC_ROMPATCHENH and the upper half of ROMC_ROMPATCHENL are read-only. ROMC_ROMPATCHENL[15:0] are associated with comparators 15 through 0. ROMC_ROMPATCHENLH[31:0] would have been associated with comparators 63 through 32.

Address: 3031_0000h base + FCh offset = 3031_00FCh



ROMC_ROMPATCHENL field descriptions

Field	Description
31–16 Reserved	This field is reserved.
ENABLE	Enable Address Comparator - This bit enables the corresponding address comparator to trigger an event. 0 Address comparator disabled 1 Address comparator enabled, ROMC will trigger a opcode patch or data fix event upon matching of the associated address

4.9.5.5 ROMC Address Registers (ROMC_ROMPATCHnA)

The ROMC address registers (ROMC_ROMPATCHA0 through ROMC_ROMPATCHA15) store the memory addresses where opcode patching begins and data fixing occurs. The address registers ROMC_ROMPATCHA0 through ROMC_ROMPATCHA15 are each 21 bits wide and dedicated to one 4 Mbyte memory space. Bits 21 through 2 are address bits, to be compared with romc_haddr[21:2] for a match; bit 1 is also an address bit used for half word selection. Bit 0 is the mode bit (set to 1 for THUMB mode). 1-word data fixing can only be used on the first 8 of the address comparators. ROMC_ROMPATCHA0 through ROMC_ROMPATCHA15 are associated each with address comparators 0 through 15.

Address: 3031_0000h base + 100h offset + (4d × i), where i=0d to 15d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved								ADDRX							
W	Reserved								ADDRX							
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ADDRX															THUMB
W	ADDRX															THUMB
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

ROMC_ROMPATCHnA field descriptions

Field	Description
31–23 -	This field is reserved. Reserved

Table continues on the next page...

ROMC_ROMPATCHnA field descriptions (continued)

Field	Description
22–1 ADDRX	Address Comparator Registers - Indicates the memory address to be watched. All 16 registers can be used for code patch address comparison. Only the first 8 registers can be used for a 1-word data fix address comparison. Bit 1 is ignored if data fix. Only used in code patch
0 THUMBX	THUMB Comparator Select - Indicates that this address will trigger a THUMB opcode patch or an Arm opcode patch. If this watchpoint is selected to be a data fix, then this bit is ignored as all data fixes are 1-word data fixes. 0 Arm patch 1 THUMB patch (ignore if data fix)

4.9.5.6 ROMC Status Register (ROMC_ROMPATCHSR)

The ROMC status register (ROMC_ROMPATCHSR) indicates the current state of the ROMC and the source number of the most recent address comparator event.

Address: 3031_0000h base + 208h offset = 3031_0208h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved														SW	Reserved
W															w1c	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved										SOURCE					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

ROMC_ROMPATCHSR field descriptions

Field	Description
31–18 -	This field is reserved. Reserved
17 SW	ROMC AHB Multiple Address Comparator matches Indicator - Indicates that multiple address comparator matches occurred. Writing a 1 to this bit will clear this it. 0 no event or comparator collisions 1 a collision has occurred
16–6 -	This field is reserved. Reserved
SOURCE	ROMC Source Number - Binary encoding of the number of the address comparator which has an address match in the most recent patch event on ROMC AHB. If multiple matches occurred, the highest priority source number is used.

Table continues on the next page...

ROMC_ROMPATCHSR field descriptions (continued)

Field	Description
0	Address Comparator 0 matched
1	Address Comparator 1 matched
15	Address Comparator 15 matched

4.10 Watchdog Timer (WDOG)

4.10.1 Overview

The Watchdog Timer (WDOG) protects against system failures by providing a method by which to escape from unexpected events or programming errors.

Once the WDOG is activated, it must be serviced by the software on a periodic basis. If servicing does not take place, the timer times out. Upon timeout, the WDOG asserts the internal system reset signal, WDOG_RESET_B_DEB to the System Reset Controller (SRC).

There is also a provision for WDOG signal assertion by timeout counter expiration. There is an option of programmable interrupt generation before the counter actually times out. The time at which the interrupt needs to be generated prior to counter timeout is programmable. There is a power down counter which is enabled out of any reset (POR, Warm/Cold). This counter has a fixed timeout period of 16 seconds, upon which it asserts the WDOG signal.

Flow diagrams for the timeout counter, power down counter and interrupt operations are shown in [Flow Diagrams](#).

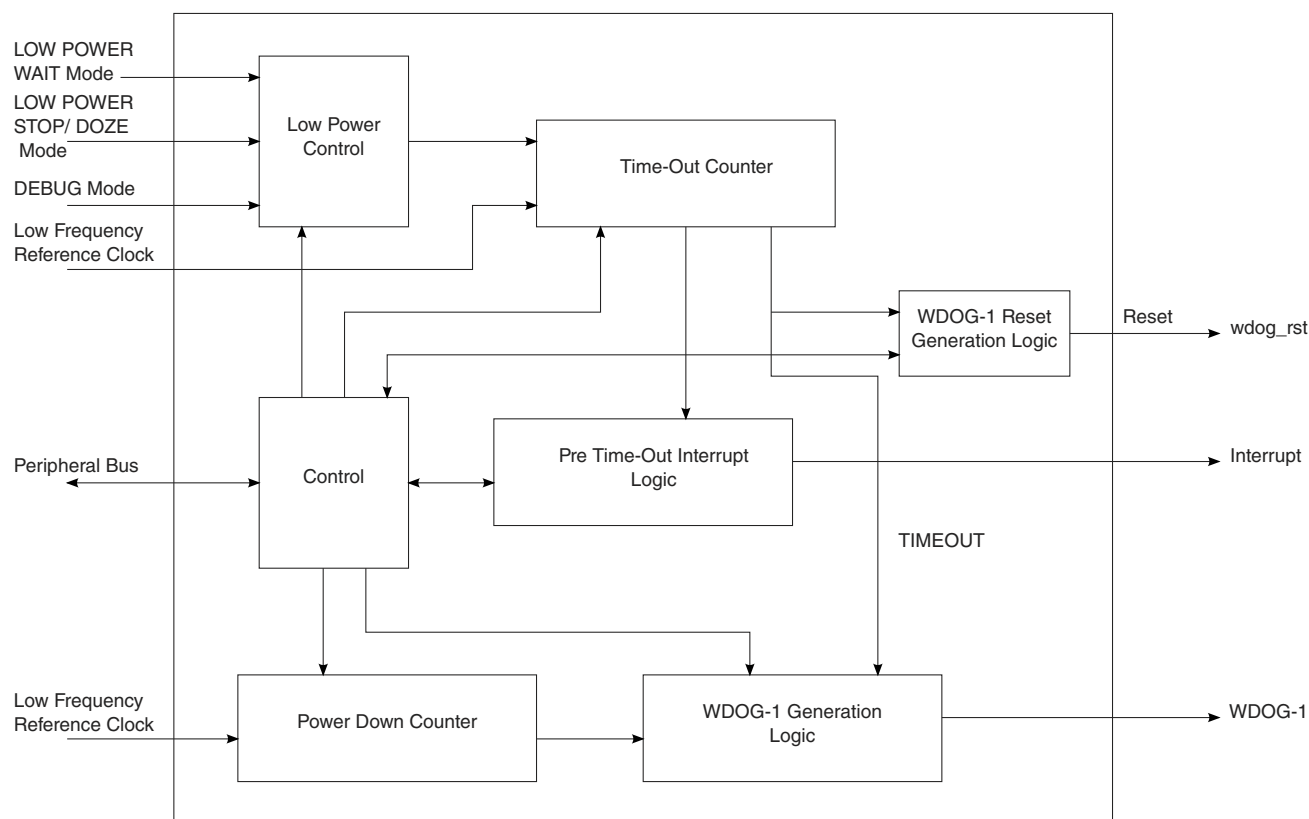


Figure 4-20. WDOG Diagram

4.10.1.1 Features

The WDOG features are listed below:

- Configurable timeout counter with timeout periods from 0.5 to 128 seconds which, after timeout expiration, result in the assertion of WDOG_RESET_B_DEB reset signal .
- Time resolution of 0.5 seconds
- Configurable timeout counter that can be programmed to run or stop during low-power modes
- Configurable timeout counter that can be programmed to run or stop during DEBUG mode
- Programmable interrupt generation prior to timeout
- The duration between interrupt and timeout events can be programmed from 0 to 127.5 seconds in steps of 0.5 seconds.
- Power down counter with fixed timeout period of 16 seconds, which if not disabled after reset will assert WDOG_B signal low

- Power down counter will be enabled out of any reset (POR, Warm / Cold reset) by default.

4.10.2 Clocks

This section describes clocks and special clocking requirements of the block.

The WDOG uses the low frequency reference clock for its counter and control operations. The peripheral bus clock is used for register read/write operations.

The following table describes the clock sources for WDOG. Please see [Introduction](#) for clock setting, configuration and gating information.

Table 4-26. WDOG Clocks

Clock name	Clock Root	Description
ipg_clk	ipg_clk_root	IP Global functional clock. All functionality inside the WDOG module is synchronized to this clock.
ipg_clk_s	ipg_clk_root	IP slave bus clock. This clock is synchronized to ipg_clk and is only used for register read/write operations.
ipg_clk_32k	ckil_sync_clk_root	Low frequency (32.768 kHz) clock that continues to run in low-power mode. It is assumed that the Clock Controller will provide this clock signal synchronized to ipg_clk in the normal mode, and switch to a non-synchronized signal in low-power mode when the ipg_clk is off.

4.10.3 Functional description

This section provides a complete functional description of the block.

4.10.3.1 Timeout event

The WDOG provides timeout periods from 0.5 to 128 seconds with a time resolution of 0.5 seconds.

The user can determine the timeout period by writing to the WDOG timeout field (WT[7:0]) in the [Watchdog Control Register \(WDOG_WCR\)](#). The WDOG must be enabled by setting the WDE bit of [Watchdog Control Register \(WDOG_WCR\)](#) for the timeout counter to start running. After the WDOG is enabled, the counter is activated, loads the timeout value and begins to count down from this programmed value. The timer will time out when the counter reaches zero and the WDOG outputs a system reset signal, WDOG_RESET_B_DEB and asserts WDOG_B (WDT bit should be set in [Watchdog Control Register \(WDOG_WCR\)](#)).

However, the timeout condition can be prevented by reloading the counter with the new timeout value (WT[7:0] of WDOG_WCR) if a service routine (see [Servicing WDOG to reload the counter](#)) is performed before the counter reaches zero. If any system errors occur which prevent the software from servicing the [Watchdog Service Register \(WDOG_WSR\)](#), the timeout condition occurs. By performing the service routine, the WDOG reloads its counter to the timeout value indicated by bits WT[7:0] of the [Watchdog Control Register \(WDOG_WCR\)](#) and it restarts the countdown.

A system reset will reset the counter and place it in the idle state at any time during the countdown. The counter flow diagram is shown in [Flow Diagrams](#).

NOTE

The timeout value is reloaded to the counter either at the time WDOG is enabled or after the service routine has been performed.

4.10.3.1.1 Servicing WDOG to reload the counter

To reload a timeout value to the counter the proper service sequence begins by writing 0x_5555 followed by 0x_AAAA to the [Watchdog Service Register \(WDOG_WSR\)](#). Any number of instructions can be executed between the two writes. If the WDOG_WSR is not loaded with 0x_5555 prior to writing 0x_AAAA to the WDOG_WSR, the counter is not reloaded. If any value other than 0x_AAAA is written to the WDOG_WSR after 0x_5555, the counter is not reloaded. This service sequence will reload the counter with the timeout value WT[7:0] of [Watchdog Control Register \(WDOG_WCR\)](#). The timeout value can be changed at any point; it is reloaded when WDOG is serviced by the core.

4.10.3.2 Interrupt event

Prior to timeout, the WDOG can generate an interrupt which can be considered a warning that timeout will occur shortly.

The duration between interrupt event and timeout event can be controlled by writing to the WICT field of [Watchdog Interrupt Control Register \(WDOG_WICR\)](#). It can vary between 0 and 127.5 seconds. If the WDOG is serviced ([Servicing WDOG to reload the counter](#)) before the interrupt generation, the counter will be reloaded with the timeout value WT[7:0] of [Watchdog Control Register \(WDOG_WCR\)](#) and the interrupt will not be triggered.

4.10.3.3 Power-down counter event

The power-down counter inside WDOG will be enabled out of reset. This counter has a fixed timeout value of 16 seconds, after which it will drive the WDOG_B signal low.

To prevent this, the software must disable this counter by clearing the PDE bit of [Watchdog Miscellaneous Control Register \(WDOG_WMCR\)](#) within 16 seconds of reset deassertion. Once disabled, this counter can't be enabled again until the next system reset occurs. This feature is intended to prevent the hanging up of cores after reset, as WDOG is not enabled out of reset.

4.10.3.4 Low power modes

4.10.3.4.1 STOP and DOZE mode

If the WDOG timer disable bit for low power STOP and DOZE mode (WDZST) bit in the [Watchdog Control Register \(WDOG_WCR\)](#), is cleared, the WDOG timer continues to operate using the low frequency reference clock. If the low power enable (WDZST) bit is set, the WDOG timer operation will be suspended in low power STOP or DOZE mode. Upon exiting low power STOP or DOZE mode, the WDOG operation returns to what it was prior to entering the STOP or DOZE mode.

4.10.3.4.2 WAIT mode

If the WDOG timer disable bit for low power WAIT mode (WDW) bit in the [Watchdog Control Register \(WDOG_WCR\)](#), is cleared, the WDOG timer continues to operate using the low frequency reference clock. If the low power WAIT enable (WDW) bit is set, the WDOG timer operation will be suspended. Upon exiting low power WAIT mode, the WDOG operation returns to what it was prior to entering the WAIT mode.

NOTE

The WDOG timer won't be able to detect events that happen for periods shorter than one low frequency reference clock cycle. For example, in repeated WAIT mode entry or exit, if the RUN mode time is less than one low frequency reference clock cycle and if the WDW bit is set, the WDOG timer may never time out, even though the system is in RUN mode for a finite duration; WDOG may not see a low frequency reference clock edge during its wake time.

4.10.3.5 Debug mode

The WDOG timer can be configured for continual operation, or for suspension during debug mode. If the WDOG debug enable (WDBG) bit is set in the [Watchdog Control Register \(WDOG_WCR\)](#), the WDOG timer operation is suspended in debug mode. If the WDBG bit is set and the debug mode is entered, WDOG timer operation is suspended after two low frequency reference clocks. Similarly, WDOG timer operation continues after two low frequency reference clocks of debug mode exit. Register read and write accesses in debug mode continue to function normally. Also, while in debug mode, the WDE bit of [Watchdog Control Register \(WDOG_WCR\)](#) can be enabled/disabled directly. If the WDOG debug enable (WDBG) bit is cleared then WDOG timer operation is not suspended. The power-down counter is not affected by debug mode entry/exit.

NOTE

If the WDE bit of [Watchdog Control Register \(WDOG_WCR\)](#) is set/cleared while in debug mode, it remains set/cleared even after exiting debug mode.

4.10.3.6 Operations

4.10.3.6.1 Watchdog reset generation

The WDOG generated reset signal WDOG_RESET_B_DEB is asserted by the following operations:

- A software write to the Software Reset Signal (SRS) bit of the [Watchdog Control Register \(WDOG_WCR\)](#).
- WDOG timeout. See [Timeout event](#).

The `wdog_rst` will be asserted for one clock cycle of low frequency reference clock for both a timeout condition and a software write occurrence. It remains asserted for 1 clock cycle of low frequency reference clock even if a system reset is asserted in between.

[Figure 4-22](#) shows the timing diagram of this signal due to a timeout condition.

4.10.3.6.2 WDOG_B generation

The WDOG asserts WDOG_B in the following scenarios:

- Software write to WDA bit of [Watchdog Control Register \(WDOG_WCR\)](#). WDOG_B signal remains asserted as long as the WDA bit is "0".
- WDOG timeout condition, WDT bit of [Watchdog Control Register \(WDOG_WCR\)](#) must be set for this scenario. A description of the timeout condition can be found in the [Timeout event](#). WDOG_B signal remains asserted until a power-on reset (POR)

occurs. It gets cleared after the POR occurs (not due to any other system reset).

Figure 4-23 shows the timing diagram of WDOG_B due to timeout condition.

- WDOG power-down counter timeout, PDE bit of [Watchdog Miscellaneous Control Register \(WDOG_WMCR\)](#) should not be cleared for this scenario. A description of this counter can be found in the [Power-down counter event](#). WDOG_B signal remains asserted for one clock cycle of low frequency reference clock.

Figure 4-21 shows the scenarios under which WDOG_B gets asserted.

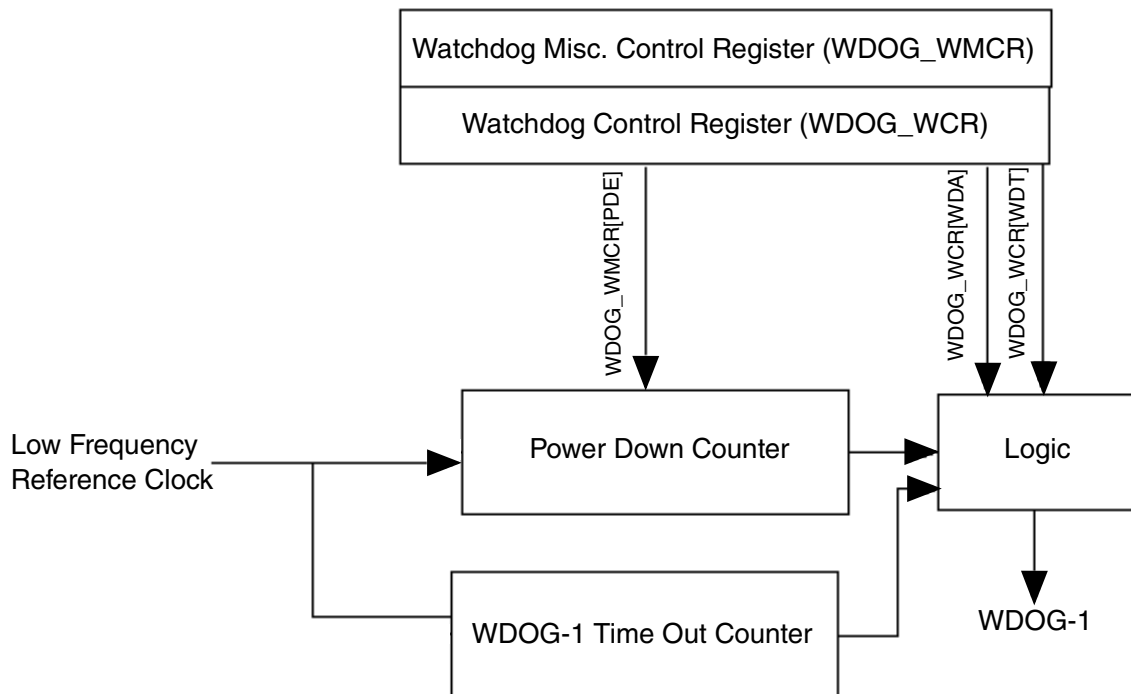


Figure 4-21. WDOG_B generation

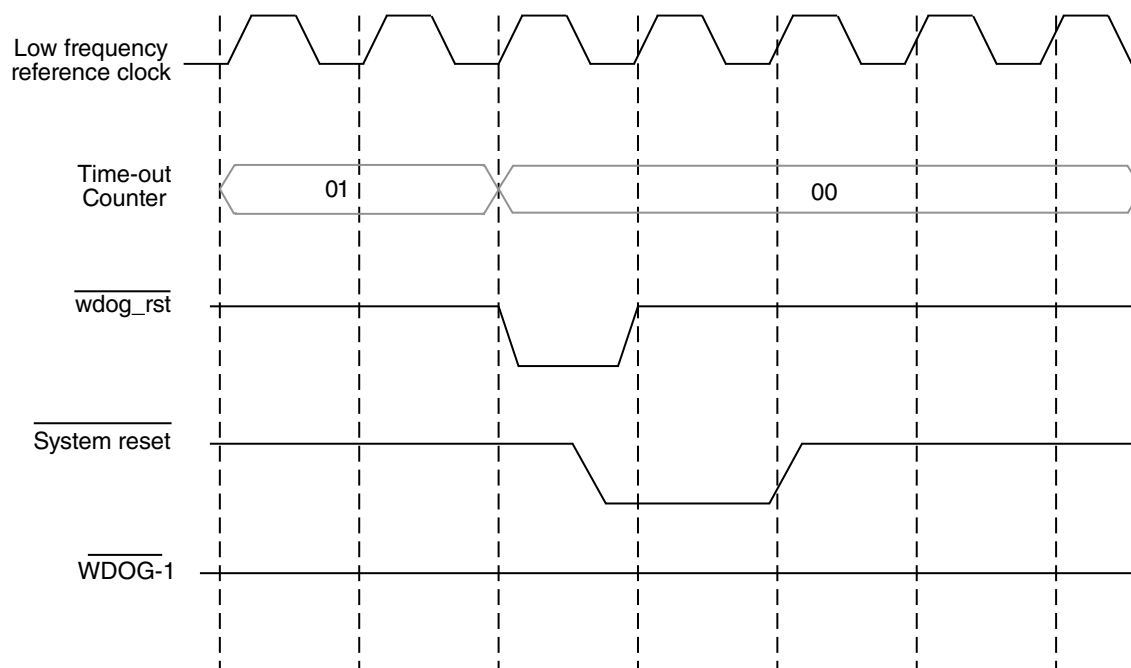


Figure 4-22. WDOG timeout condition/WDT bit is not set

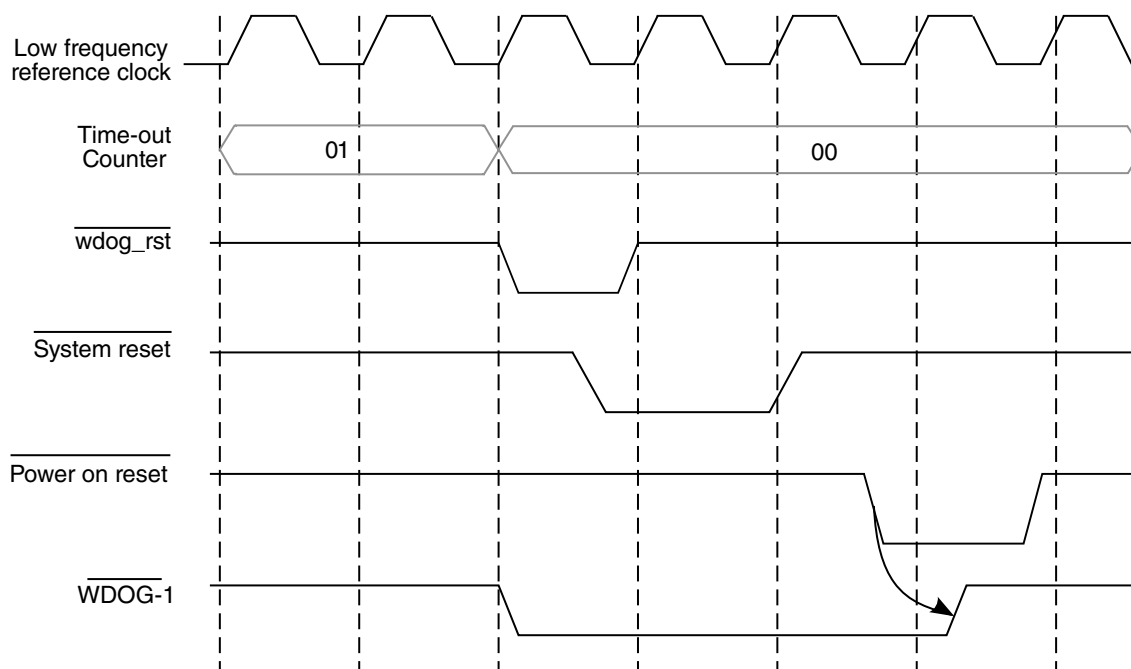


Figure 4-23. WDOG timeout condition/WDT bit is set

4.10.3.7 Reset

The block is reset by a system reset and the WDOG counter will be disabled. The power-down counter is enabled and starts counting.

4.10.3.8 Interrupt

The WDOG has the feature of Interrupt generation before timeout.

The interrupt will be generated only if the WIE bit in [Watchdog Interrupt Control Register \(WDOG_WICR\)](#) is set. The exact time at which the interrupt should occur (prior to timeout) depends on the value of WICT field of [Watchdog Interrupt Control Register \(WDOG_WICR\)](#). For example, if the WICT field has a value 0x04, then the interrupt will be generated two seconds prior to timeout. Once the interrupt is triggered the WTIS bit in [Watchdog Interrupt Control Register \(WDOG_WICR\)](#) will be set. The software needs to clear this bit to deassert the interrupt. If the WDOG is serviced before the interrupt generation then the counter will be reloaded with the timeout value WT[7:0] of [Watchdog Control Register \(WDOG_WCR\)](#) and interrupt would not be triggered.

4.10.3.9 Flow Diagrams

A flow diagram of WDOG operation is shown below.

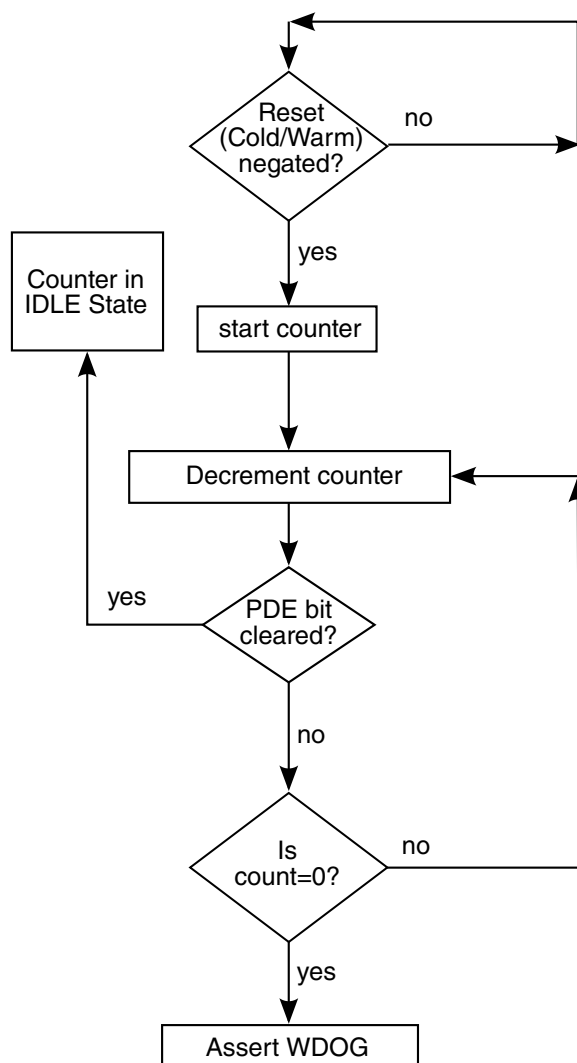


Figure 4-24. Power-Down Counter Flow Diagram

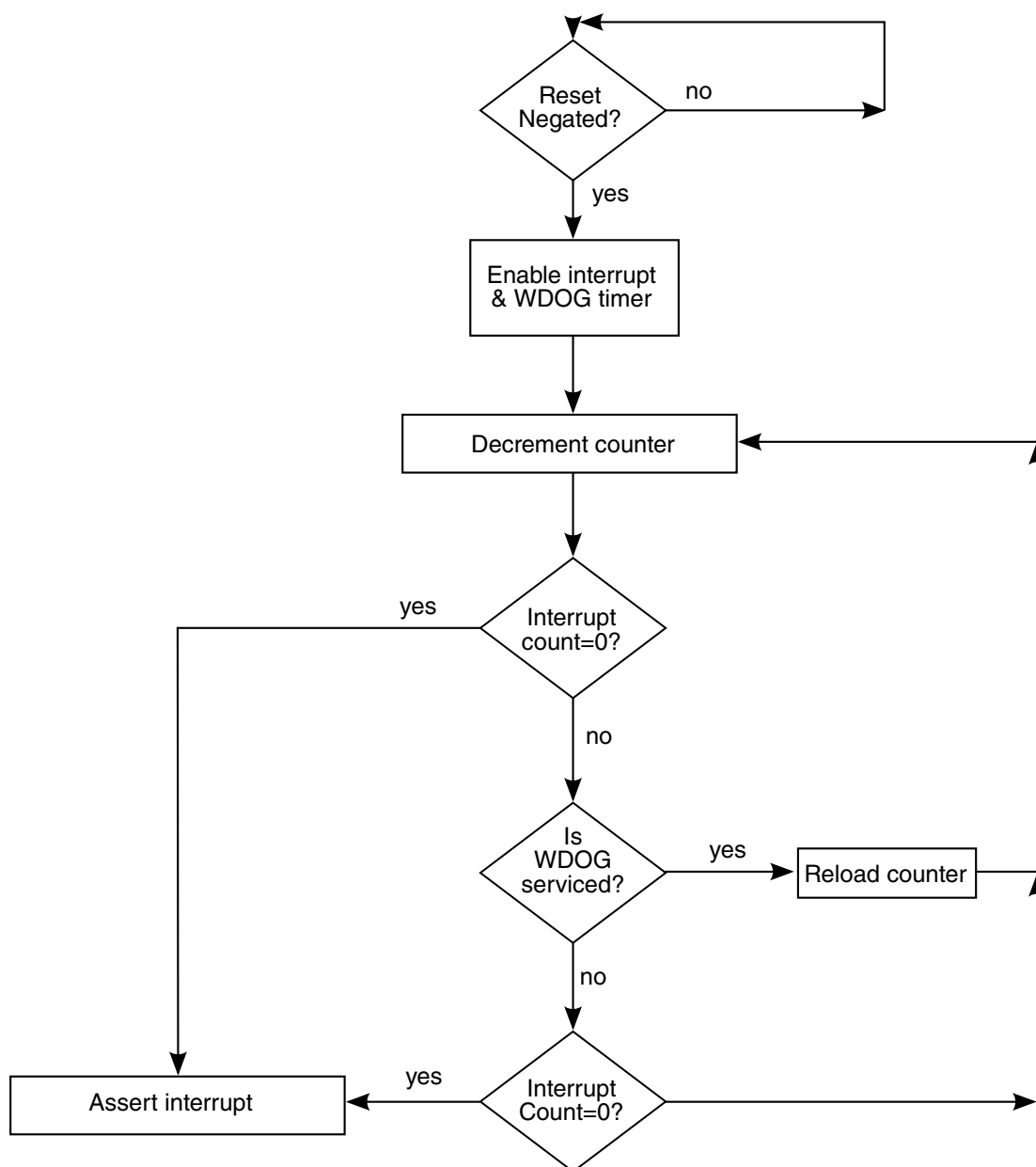


Figure 4-25. Interrupt Generation Flow Diagram

4.10.4 Initialization

The following sequence should be performed for WDOG initialization.

- PDE bit of [Watchdog Miscellaneous Control Register \(WDOG_WMCR\)](#) should be cleared to disable the power down counter.

Watchdog Timer (WDOG)

- WT field of [Watchdog Control Register \(WDOG_WCR\)](#) should be programmed for sufficient timeout value.
- WDOG should be enabled by setting WDE bit of [Watchdog Control Register \(WDOG_WCR\)](#) so that the timeout counter loads the WT field value of [Watchdog Control Register \(WDOG_WCR\)](#) and starts counting.

4.10.5 WDOG Memory Map/Register Definition

The WDOG Memory Map/Register Definition can be found here.

The WDOG has user-accessible, 16-bit registers used to configure, operate, and monitor the state of the Watchdog Timer. Byte operations can be performed on these registers. If a 32-bit access is performed, the WDOG will not generate a peripheral bus error but will behave normally, like a 16-Bit access, making read/write possible. A 32-Bit access should be avoided, as the system may go to an unknown state.

WDOG memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3028_0000	Watchdog Control Register (WDOG1_WCR)	16	R/W	0030h	4.10.5.1/ 253
3028_0002	Watchdog Service Register (WDOG1_WSR)	16	R/W	0000h	4.10.5.2/ 255
3028_0004	Watchdog Reset Status Register (WDOG1_WRSR)	16	R	0000h	4.10.5.3/ 255
3028_0006	Watchdog Interrupt Control Register (WDOG1_WICR)	16	R/W	0004h	4.10.5.4/ 256
3028_0008	Watchdog Miscellaneous Control Register (WDOG1_WMCR)	16	R/W	0001h	4.10.5.5/ 257
3029_0000	Watchdog Control Register (WDOG2_WCR)	16	R/W	0030h	4.10.5.1/ 253
3029_0002	Watchdog Service Register (WDOG2_WSR)	16	R/W	0000h	4.10.5.2/ 255
3029_0004	Watchdog Reset Status Register (WDOG2_WRSR)	16	R	0000h	4.10.5.3/ 255
3029_0006	Watchdog Interrupt Control Register (WDOG2_WICR)	16	R/W	0004h	4.10.5.4/ 256
3029_0008	Watchdog Miscellaneous Control Register (WDOG2_WMCR)	16	R/W	0001h	4.10.5.5/ 257
302A_0000	Watchdog Control Register (WDOG3_WCR)	16	R/W	0030h	4.10.5.1/ 253

Table continues on the next page...

WDOG memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
302A_0002	Watchdog Service Register (WDOG3_WSR)	16	R/W	0000h	4.10.5.2/ 255
302A_0004	Watchdog Reset Status Register (WDOG3_WRSR)	16	R	0000h	4.10.5.3/ 255
302A_0006	Watchdog Interrupt Control Register (WDOG3_WICR)	16	R/W	0004h	4.10.5.4/ 256
302A_0008	Watchdog Miscellaneous Control Register (WDOG3_WMCR)	16	R/W	0001h	4.10.5.5/ 257

4.10.5.1 Watchdog Control Register (WDOGx_WCR)

The Watchdog Control Register (WDOG_WCR) controls the WDOG operation.

- WDZST, WDBG and WDW are write-once only bits. Once the software does a write access to these bits, they will be locked and cannot be reprogrammed until the next system reset assertion.
- WDE is a write one once only bit. Once software performs a write "1" operation to this bit it cannot be reset/cleared until the next system reset.
- WDT is also a write one once only bit. Once software performs a write "1" operation to this bit it cannot be reset/cleared until the next POR. This bit does not get reset/cleared due to any system reset.

Address: Base address + 0h offset

Bit	15	14	13	12	11	10	9	8
Read	WT							
Write								
Reset	0	0	0	0	0	0	0	0
Bit	7	6	5	4	3	2	1	0
Read	WDW	SRE	WDA	SRS	WDT	WDE	WDBG	WDZST
Write								
Reset	0	0	1	1	0	0	0	0

WDOGx_WCR field descriptions

Field	Description
15–8 WT	Watchdog Time-out Field. This 8-bit field contains the time-out value that is loaded into the Watchdog counter after the service routine has been performed or after the Watchdog is enabled. After reset, WT[7:0] must have a value written to it before enabling the Watchdog otherwise count value of zero which is 0.5 seconds is loaded into the counter. NOTE: The time-out value can be written at any point of time but it is loaded to the counter at the time when WDOG is enabled or after the service routine has been performed. For more information see Timeout event.

Table continues on the next page...

WDOGx_WCR field descriptions (continued)

Field	Description
	0x00 - 0.5 Seconds (Default). 0x01 - 1.0 Seconds. 0x02 - 1.5 Seconds. 0x03 - 2.0 Seconds. 0xff - 128 Seconds.
7 WDW	Watchdog Disable for Wait. This bit determines the operation of WDOG during Low Power WAIT mode. This is a write once only bit. 0 Continue WDOG timer operation (Default). 1 Suspend WDOG timer operation.
6 SRE	Software Reset Extension. Required to be set to 1 when used in conjunction with the Software Reset Signal (SRS). 0 Reserved 1 This bit must be set to 1.
5 WDA	WDOG_B assertion. Controls the software assertion of the WDOG_B signal. 0 Assert WDOG_B output. 1 No effect on system (Default).
4 SRS	Software Reset Signal. Controls the software assertion of the WDOG-generated reset signal WDOG_RESET_B_DEB. This bit automatically resets to 1 after it has been asserted to 0. For proper operation of this function, the SRE bit in this register must be set to 1. NOTE: This bit does not generate the software reset to the block. 0 Assert system reset signal. 1 No effect on the system (Default).
3 WDT	WDOG_B Time-out assertion. Determines if the WDOG_B gets asserted upon a Watchdog Time-out Event. This is a write-one once only bit. NOTE: There is no effect on WDOG_RESET_B_DEB (WDOG Reset) upon writing on this bit. WDOG_B gets asserted along with WDOG_RESET_B_DEB if this bit is set. 0 No effect on WDOG_B (Default). 1 Assert WDOG_B upon a Watchdog Time-out event.
2 WDE	Watchdog Enable. Enables or disables the WDOG block. This is a write one once only bit. It is not possible to clear this bit by a software write, once the bit is set. NOTE: This bit can be set/reset in debug mode (exception). 0 Disable the Watchdog (Default). 1 Enable the Watchdog.
1 WDBG	Watchdog DEBUG Enable. Determines the operation of the WDOG during DEBUG mode. This bit is write once only. 0 Continue WDOG timer operation (Default). 1 Suspend the watchdog timer.
0 WDZST	Watchdog Low Power. Determines the operation of the WDOG during low-power modes. This bit is write once-only. NOTE: The WDOG can continue/suspend the timer operation in the low-power modes (STOP and DOZE mode).

Table continues on the next page...

WDOGx_WCR field descriptions (continued)

Field	Description
0	Continue timer operation (Default).
1	Suspend the watchdog timer.

4.10.5.2 Watchdog Service Register (WDOGx_WSR)

When enabled, the WDOG requires that a service sequence be written to the Watchdog Service Register (WSR) to prevent the timeout condition.

NOTE

Executing the service sequence will reload the WDOG timeout counter.

Address: Base address + 2h offset

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read	WSR															
Write																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

WDOGx_WSR field descriptions

Field	Description
WSR	Watchdog Service Register. This 16-bit field contains the Watchdog service sequence. Both writes must occur in the order listed prior to the time-out, but any number of instructions can be executed between the two writes. The service sequence must be performed as follows: 0x5555 Write to the Watchdog Service Register (WDOG_WSR). 0xAAAA Write to the Watchdog Service Register (WDOG_WSR).

4.10.5.3 Watchdog Reset Status Register (WDOGx_WRSR)

The WRSR is a read-only register that records the source of the output reset assertion. It is not cleared by a hard reset. Therefore, only one bit in the WRSR will always be asserted high. The register will always indicate the source of the last reset generated due to WDOG. Read access to this register is with one wait state. Any write performed on this register will generate a Peripheral Bus Error .

A reset can be generated by the following sources, as listed in priority from highest to lowest:

- Watchdog Time-out
- Software Reset

Watchdog Timer (WDOG)

Address: Base address + 4h offset

Bit	15	14	13	12	11	10	9	8
Read	0							
Write								
Reset	0	0	0	0	0	0	0	0
Bit	7	6	5	4	3	2	1	0
Read	0			POR	0		TOUT	SFTW
Write								
Reset	0	0	0	0	0	0	0	0

WDOGx_WRSR field descriptions

Field	Description
15–5 Reserved	This read-only field is reserved and always has the value 0.
4 POR	Power On Reset. Indicates whether the reset is the result of a power on reset. 0 Reset is not the result of a power on reset. 1 Reset is the result of a power on reset.
3–2 Reserved	This read-only field is reserved and always has the value 0.
1 TOUT	Timeout. Indicates whether the reset is the result of a WDOG timeout. 0 Reset is not the result of a WDOG timeout. 1 Reset is the result of a WDOG timeout.
0 SFTW	Software Reset. Indicates whether the reset is the result of a WDOG software reset by asserting SRS bit 0 Reset is not the result of a software reset. 1 Reset is the result of a software reset.

4.10.5.4 Watchdog Interrupt Control Register (WDOGx_WICR)

The WDOG_WICR controls the WDOG interrupt generation.

Address: Base address + 6h offset

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read	WIE	WTIS	0						WICT							
Write		w1c														
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0

WDOGx_WICR field descriptions

Field	Description
15 WIE	Watchdog Timer Interrupt enable bit. Reset value is 0.

Table continues on the next page...

WDOGx_WICR field descriptions (continued)

Field	Description
	NOTE: This bit is a write once only bit. Once the software does a write access to this bit, it will get locked and cannot be reprogrammed until the next system reset assertion 0 Disable Interrupt (Default). 1 Enable Interrupt.
14 WTIS	Watchdog Timer Interrupt Status bit will reflect the timer interrupt status, whether interrupt has occurred or not. Once the interrupt has been triggered software must clear this bit by writing 1 to it. 0 No interrupt has occurred (Default). 1 Interrupt has occurred
13–8 Reserved	This read-only field is reserved and always has the value 0.
WICT	Watchdog Interrupt Count Time-out (WICT) field determines, how long before the counter time-out must the interrupt occur. The reset value is 0x04 implies interrupt will occur 2 seconds before time-out. The maximum value that can be programmed to WICT field is 127.5 seconds with a resolution of 0.5 seconds. NOTE: This field is write once only. Once the software does a write access to this field, it will get locked and cannot be reprogrammed until the next system reset assertion. 0x00 WICT[7:0] = Time duration between interrupt and time-out is 0 seconds. 0x01 WICT[7:0] = Time duration between interrupt and time-out is 0.5 seconds. 0x04 WICT[7:0] = Time duration between interrupt and time-out is 2 seconds (Default). 0xff WICT[7:0] = Time duration between interrupt and time-out is 127.5 seconds.

4.10.5.5 Watchdog Miscellaneous Control Register (WDOGx_WMCR)

WDOG_WMCR Controls the Power Down counter operation.

Address: Base address + 8h offset

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read	0															PDE
Write																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

WDOGx_WMCR field descriptions

Field	Description
15–1 Reserved	This read-only field is reserved and always has the value 0.
0 PDE	Power Down Enable bit. Reset value of this bit is 1, which means the power down counter inside the WDOG is enabled after reset. The software must write 0 to this bit to disable the counter within 16 seconds of reset de-assertion. Once disabled this counter cannot be enabled again. See Power-down counter event for operation of this counter. NOTE: This bit is write-one once only bit. Once software sets this bit it cannot be reset until the next system reset.

Table continues on the next page...

WDOGx_WMCR field descriptions (continued)

Field	Description
0	Power Down Counter of WDOG is disabled.
1	Power Down Counter of WDOG is enabled (Default).

4.11 System Counter (SYS_CTR)

4.11.1 Overview

The System Counter (SYS_CTR) is a programmable system counter, which provides a shared time base to multiple processors. It is intended for applications where the counter is always powered on, and supports multiple unrelated clocks.

4.11.1.1 Block Diagram

The block diagram of the System Counter is shown below.

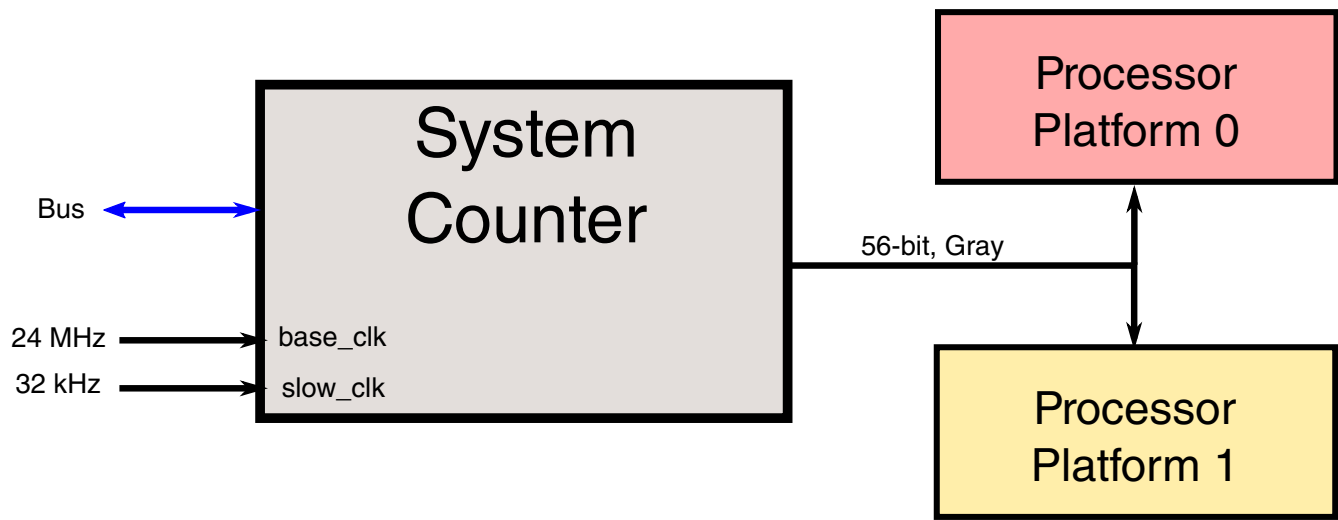


Figure 4-26. System Counter Block Diagram

4.11.1.2 Features

- Two counter clock frequencies
 - Base clock for normal operation
 - Alternate clock for low power operation
- 56-bit counter width

- Gray coded counter output for distribution to the processor timers
- 2 Compare Frames
 - Each compare frame contains a 64-bit compare value
 - Each compare frame contains programmable interrupt generation

4.11.2 Functional Description

The System Counter inputs two counter clock sources and outputs a gray coded counter value and interrupt signals (one per compare frame) to the platform's interrupt controller.

4.11.2.1 Operation

After reset, the System Counter is disabled with count value reset to zero and base frequency selected. Once the counter is enabled, it will increment the appropriate value on each rising edge of the selected clock. Because the System Counter is handling a 56-bit count value across multiple clock domains, synchronization is necessary. The System Counter provides synchronization mechanisms between the various clock domains.

When the system switches the counter's clock source, there is a short pause while the clock multiplexer is handling the clock transition. In order to maintain an accurate count value, the clock control logic employs two offset counters; one for the base-to-slow transition and one for the slow-to-base transition. These offset counters only operate during the clock transition time to compensate for the idled source clock. Both counters run off of the base clock. The transition offset values are added to the system count value at the appropriate time when the counter's clock is restored.

NOTE

Both base clock and alternate clock must be running when changing frequencies.

4.11.2.2 Clocks

The System Counter clocks are shown in the table below.

Table 4-27. Clocks

Clock	Description
ipg_clk	Peripheral Clock
ipg_clk_s	Gated peripheral clock for register transactions
base_clk	Base Clock. This clock is used during normal operation. It is internally divided by 3 before use.

Table continues on the next page...

Table 4-27. Clocks (continued)

Clock	Description
slow_clk	Slow Clock. This clock is used during low power mode. It is internally divided by 64 before use.
ctr_clk	Counter Clock. The ctr_clk is generated by selecting either the base_clk or slow_clk. Because the compare frame must generate an interrupt when either clock is selected, most of the compare frame logic resides in this clock domain.

4.11.3 Programming Model

The System Counter module contains three programmer's model sections. Each section has an independent module enable to allow individual secure access control.

Table 4-28. Programming Model

Programming Model	Description
CNTControlBase	This model controls the counter frequency, enable, debug, and count value. This model also contains the frequency modes table which supplies control and status for the frequency of the System Counter.
CNTReadBase	This model allows the user to read the count value.
CNTCompareBase	This model controls the compare value frequency, enable, interrupt mask, and interrupt status for each compare frame.

4.11.4 Memory Map and Register Description

SYS_CTR_CONTROL Hardware Register Format Summary

SYS_CTR_CONTROL memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
306C_0000	Counter Control Register (SYS_CTR_CONTROL_CNTCR)	32	R/W	0000_0000h	4.11.4.1/261
306C_0004	Counter Status Register (SYS_CTR_CONTROL_CNTSR)	32	R	0000_0100h	4.11.4.2/263
306C_0008	Counter Count Value Low Register (SYS_CTR_CONTROL_CNTCV0)	32	R/W	0000_0000h	4.11.4.3/264
306C_000C	Counter Count Value High Register (SYS_CTR_CONTROL_CNTCV1)	32	R/W	0000_0000h	4.11.4.4/265

Table continues on the next page...

SYS_CTR_CONTROL memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
306C_0020	Frequency Modes Table 0 Register (SYS_CTR_CONTROL_CNTFID0)	32	R	007A_1200h	4.11.4.5/265
306C_0024	Frequency Modes Table 1 Register (SYS_CTR_CONTROL_CNTFID1)	32	R	0000_0200h	4.11.4.6/266
306C_0028	Frequency Modes Table 2 Register (SYS_CTR_CONTROL_CNTFID2)	32	R	0000_0000h	4.11.4.7/266
306C_0FD0	Counter ID Register (SYS_CTR_CONTROL_CNTID0)	32	R	0000_0000h	4.11.4.8/267
306A_0000	Counter Count Value Low Register (SYS_CTR_READ_CNTRCV0)	32	R	0000_0000h	4.11.4.1/268
306A_0004	Counter Count Value High Register (SYS_CTR_READ_CNTRCV1)	32	R	0000_0000h	4.11.4.2/268
306A_0FD0	Counter ID Register (SYS_CTR_READ_CNTID0)	32	R	0000_0000h	4.11.4.3/269
306B_0020	Compare Count Value Low Register (SYS_CTR_COMPARE_CMPCV0)	32	R	0000_0000h	4.11.4.1/269
306B_0024	Compare Count Value High Register (SYS_CTR_COMPARE_CMPCV1)	32	R	0000_0000h	4.11.4.2/270
306B_002C	Compare Control Register (SYS_CTR_COMPARE_CMPCR)	32	R/W	0000_0000h	4.11.4.3/270
306B_0FD0	Counter ID Register (SYS_CTR_COMPARE_CNTID0)	32	R	0000_0000h	4.11.4.4/272

4.11.4.1 Counter Control Register (SYS_CTR_CONTROL_CNTCR)

The 32-bit Counter Control Register defines the basic operating configuration of the System Counter.

The System Counter operates using a fixed base frequency. However, the counter can increment at a lower, alternate frequency than the base frequency, using a correspondingly larger increment. For example, it can increment by 15625 and run at a frequency of 1/15625 of the base frequency. The two frequencies available are identified in the frequency modes table. These two frequencies are the base frequency (table entry 0) and the lower, alternate frequency (table entry 1). Setting the FCR1 bit selects the alternate frequency. Setting the FCR0 bit selects the base frequency. Setting or clearing both FCR0 and FCR1 will have no effect; the frequency will not change.

Memory Map and Register Description

Address: 306C_0000h base + 0h offset = 306C_0000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved							FCR1	FCR0	Reserved					HDBG	EN
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

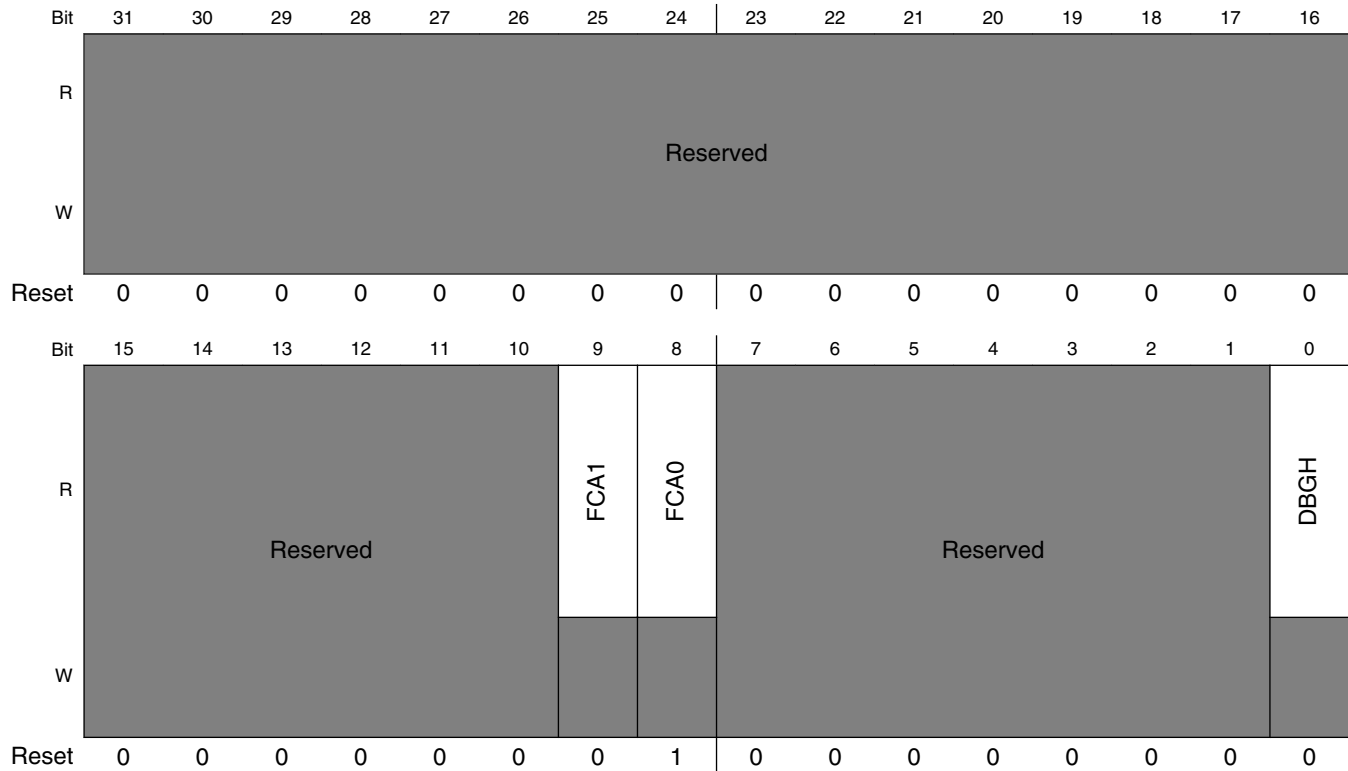
SYS_CTR_CONTROL_CNTCR field descriptions

Field	Description
31–10 -	This field is reserved. Reserved
9 FCR1	Frequency Change Request, ID 1 0 No change. 1 Select frequency modes table entry 1, the base frequency.
8 FCR0	Frequency Change Request, ID 0 0 No change. 1 Select frequency modes table entry 0, the base frequency.
7–2 -	This field is reserved. Reserved
1 HDBG	Enable Debug 0 The assertion of the debug input is ignored. 1 The assertion of the debug input causes the System Counter to halt.
0 EN	Enable Counting 0 Counter disabled 1 Counter enabled

4.11.4.2 Counter Status Register (SYS_CTR_CONTROL_CNTSR)

The system counter status register provides information concerning the clock frequency and debug state.

Address: 306C_0000h base + 4h offset = 306C_0004h



SYS_CTR_CONTROL_CNTSR field descriptions

Field	Description
31–10 -	This field is reserved. Reserved
9 FCA1	Frequency Change Acknowledge, ID 1 0 Base frequency is not selected. 1 Base frequency is selected.
8 FCA0	Frequency Change Acknowledge, ID 0 0 Base frequency is not selected. 1 Base frequency is selected.
7–1 -	This field is reserved. Reserved
0 DBGH	Debug Halt

Table continues on the next page...

SYS_CTR_CONTROL_CNTSR field descriptions (continued)

Field	Description
0	Counter is not halted by debug.
1	Counter is halted by debug.

4.11.4.3 Counter Count Value Low Register (SYS_CTR_CONTROL_CNTCV0)

The Counter Count Value Low register indicates the current count value bits 31-0.

NOTE

Writes to the CNTCV registers must be performed while operating on the base frequency only. Writes to these registers while running on the alternate frequency may have unpredictable results.

Address: 306C_0000h base + 8h offset = 306C_0008h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SYS_CTR_CONTROL_CNTCV0 field descriptions

Field	Description
CNTCV0	Counter Count Value bits [31:0]

4.11.4.4 Counter Count Value High Register (SYS_CTR_CONTROL_CNTCV1)

The Counter Count Value High register indicates the current count value bits 63-32.

NOTE

Writes to the CNTCV registers must be performed while operating on the base frequency only. Writes to these registers while running on the alternate frequency may have unpredictable results.

Address: 306C_0000h base + Ch offset = 306C_000Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								CNTCV1																							
W	Reserved								CNTCV1																							
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SYS_CTR_CONTROL_CNTCV1 field descriptions

Field	Description
31-25 -	This field is reserved. Reserved
CNTCV1	Counter Count Value bits [55:32]

4.11.4.5 Frequency Modes Table 0 Register (SYS_CTR_CONTROL_CNTFID0)

The Counter Frequency ID registers is the frequency modes table starting at offset 0x020.

Table entries are 32-bits, and each entry specifies a system counter update frequency, in Hz. The first entry in the table specifies the base frequency of the system counter. To ensure overall counter accuracy is maintained, any subsequent entries in the table are exact divisors of the base frequency.

When the system timer is operating at a lower frequency than the base frequency, the increment applied at each counter update is given by: $\text{increment} = (\text{base_frequency}) / (\text{selected_frequency})$. A 32-bit word of zero value marks the end of the table. That is, the word of memory immediately after the last entry in the table is zero.

Memory Map and Register Description

Address: 306C_0000h base + 20h offset = 306C_0020h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CNTFID0																															
W																																
Reset	0	0	0	0	0	0	0	0	0	1	1	1	1	0	1	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0

SYS_CTR_CONTROL_CNTFID0 field descriptions

Field	Description
CNTFID0	Base Frequency (24 MHz /3 = 8 MHz)

4.11.4.6 Frequency Modes Table 1 Register (SYS_CTR_CONTROL_CNTFID1)

The Counter Frequency ID registers is the frequency modes table starting at offset 0x020.

Table entries are 32-bits, and each entry specifies a system counter update frequency, in Hz. The first entry in the table specifies the base frequency of the system counter. To ensure overall counter accuracy is maintained, any subsequent entries in the table are exact divisors of the base frequency.

When the system timer is operating at a lower frequency than the base frequency, the increment applied at each counter update is given by: $\text{increment} = (\text{base_frequency}) / (\text{selected_frequency})$ A 32-bit word of zero value marks the end of the table. That is, the word of memory immediately after the last entry in the table is zero.

Address: 306C_0000h base + 24h offset = 306C_0024h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CNTFID1																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0

SYS_CTR_CONTROL_CNTFID1 field descriptions

Field	Description
CNTFID1	Alternate Frequency (32 kHz /64 = 512 Hz)

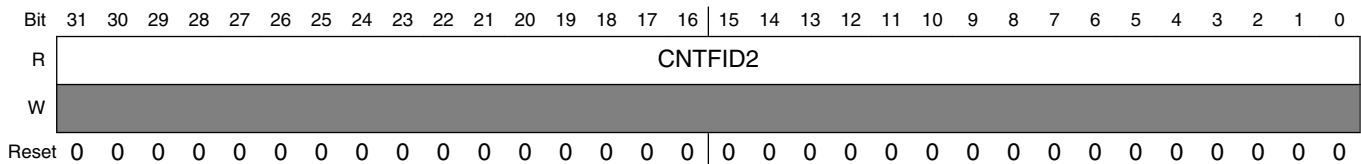
4.11.4.7 Frequency Modes Table 2 Register (SYS_CTR_CONTROL_CNTFID2)

The Counter Frequency ID registers is the frequency modes table starting at offset 0x020.

Table entries are 32-bits, and each entry specifies a system counter update frequency, in Hz. The first entry in the table specifies the base frequency of the system counter. To ensure overall counter accuracy is maintained, any subsequent entries in the table are exact divisors of the base frequency.

When the system timer is operating at a lower frequency than the base frequency, the increment applied at each counter update is given by: $\text{increment} = (\text{base_frequency}) / (\text{selected_frequency})$. A 32-bit word of zero value marks the end of the table. That is, the word of memory immediately after the last entry in the table is zero.

Address: 306C_0000h base + 28h offset = 306C_0028h



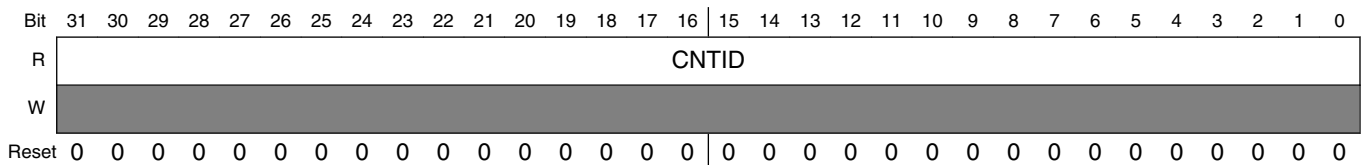
SYS_CTR_CONTROL_CNTFID2 field descriptions

Field	Description
CNTFID2	End Marker

4.11.4.8 Counter ID Register (SYS_CTR_CONTROL_CNTID0)

The Counter ID register indicates the architecture version 0.

Address: 306C_0000h base + FD0h offset = 306C_0FD0h



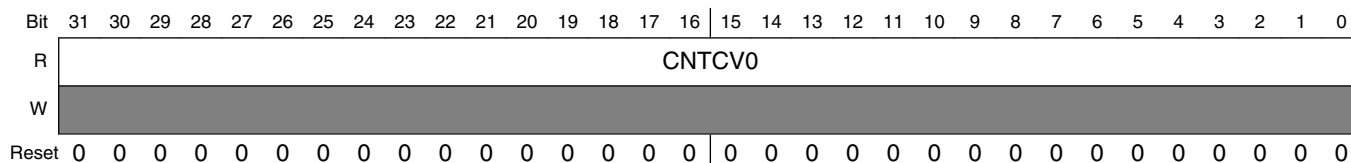
SYS_CTR_CONTROL_CNTID0 field descriptions

Field	Description
CNTID	Counter Identification. Counter ID 0.

4.11.4.1 Counter Count Value Low Register (SYS_CTR_READ_CNTCV0)

The Counter Count Value Low register indicates the current count value bits 31-0.

Address: 306A_0000h base + 0h offset = 306A_0000h



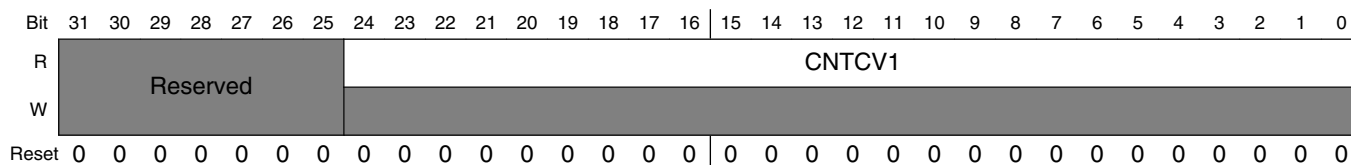
SYS_CTR_READ_CNTCV0 field descriptions

Field	Description
CNTCV0	Counter Count Value bits [31:0]

4.11.4.2 Counter Count Value High Register (SYS_CTR_READ_CNTCV1)

The Counter Count Value High register indicates the current count value bits 63-32.

Address: 306A_0000h base + 4h offset = 306A_0004h



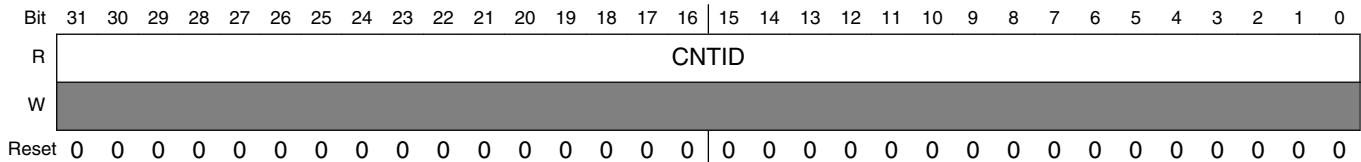
SYS_CTR_READ_CNTCV1 field descriptions

Field	Description
31–25 -	This field is reserved. Reserved
CNTCV1	Counter Count Value bits [55:32]. Bits[63:56] are always zero.

4.11.4.3 Counter ID Register (SYS_CTR_READ_CNTID0)

The Counter ID register indicates the architecture version 0.

Address: 306A_0000h base + FD0h offset = 306A_0FD0h



SYS_CTR_READ_CNTID0 field descriptions

Field	Description
CNTID	Counter Identification. Counter ID 0.

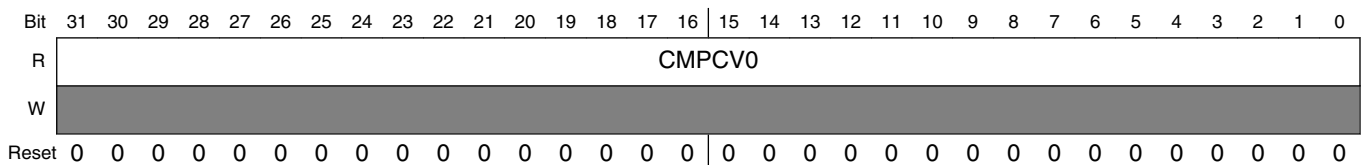
4.11.4.1 Compare Count Value Low Register (SYS_CTR_COMPARE_CMPCV0)

The Compare Count Value Low register indicates the current count value bits 31-0.

NOTE

Writes to the CMPCV registers must be performed while operating on the base frequency only. Writes to these registers while running on the alternate frequency may have unpredictable results.

Address: 306B_0000h base + 20h offset = 306B_0020h



SYS_CTR_COMPARE_CMPCV0 field descriptions

Field	Description
CMPCV0	Compare Count Value bits [31:0]

4.11.4.2 Compare Count Value High Register (SYS_CTR_COMPARE_CMPCV1)

The Compare Count Value High register indicates the current count value bits 63-32.

NOTE

Writes to the CMPCV registers must be performed while operating on the base frequency only. Writes to these registers while running on the alternate frequency may have unpredictable results.

Address: 306B_0000h base + 24h offset = 306B_0024h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved							CMPCV1																								
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SYS_CTR_COMPARE_CMPCV1 field descriptions

Field	Description
31-25 -	This field is reserved. Reserved
CMPCV1	Compare Count Value bits [55:32]. Bits[63:56] are always zero.

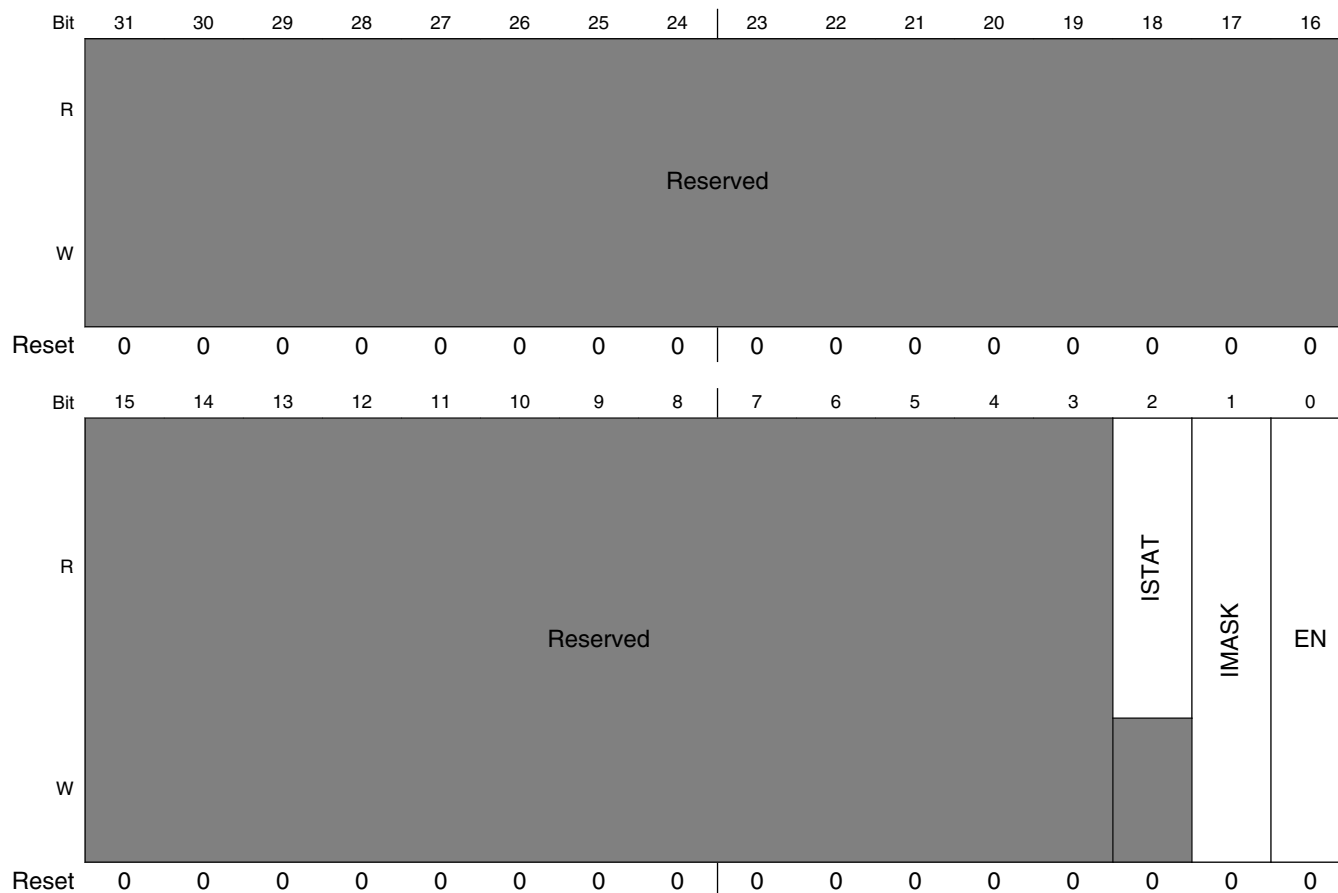
4.11.4.3 Compare Control Register (SYS_CTR_COMPARE_CMPCR)

The compare control register provides control and status of the compare function. When enabled, the ISTAT bit indicates whether the counter value is greater than or equal to the value in the compare value register (CMPCV). The ISTAT equation is:

$$\text{ISTAT} = (\text{CNTCV} \geq \text{CMPCV})$$

ISTAT takes no account of the value of the IMASK bit. If ISTAT is set to 1 and IMASK is 0, then the interrupt request is asserted. Clearing the enable bit (EN=0) will clear the status bit (ISTAT=0) and will negate the interrupt output signal.

Address: 306B_0000h base + 2Ch offset = 306B_002Ch

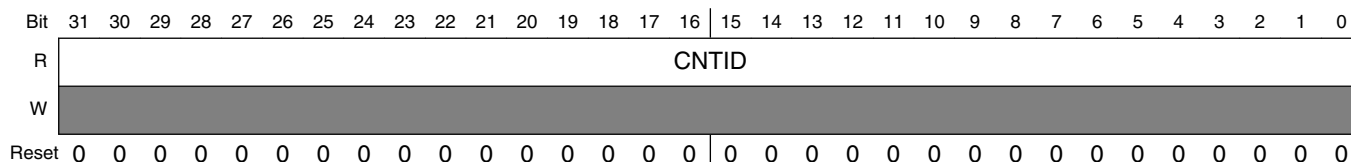
**SYS_CTR_COMPARE_CMPCR field descriptions**

Field	Description
31–3 -	This field is reserved. Reserved
2 ISTAT	Compare (interrupt) status 0 Counter value is less than the compare value or compare is disabled. 1 Counter value is greater than or equal to the compare value and compare is enabled.
1 IMASK	Interrupt request mask 0 Interrupt output signal is not masked. 1 Interrupt output signal is masked.
0 EN	Enable the compare function 0 Compare disabled 1 Compare enabled

4.11.4.4 Counter ID Register (SYS_CTR_COMPARE_CNTID0)

The Counter ID register indicates the architecture version 0.

Address: 306B_0000h base + FD0h offset = 306B_0FD0h



SYS_CTR_COMPARE_CNTID0 field descriptions

Field	Description
CNTID	Counter Identification. Counter ID 0.

The read frame registers are all read-only. These registers read the same values as the control frame registers for the count value and counter ID. They are processed via a separate mechanism from the control frame to allow nonsecure, user mode access.

SYS_CTR_READ memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
306C_0000	Counter Control Register (SYS_CTR_CONTROL_CNTCR)	32	R/W	0000_0000h	4.11.4.1/261
306C_0004	Counter Status Register (SYS_CTR_CONTROL_CNTSR)	32	R	0000_0100h	4.11.4.2/263
306C_0008	Counter Count Value Low Register (SYS_CTR_CONTROL_CNTCV0)	32	R/W	0000_0000h	4.11.4.3/264
306C_000C	Counter Count Value High Register (SYS_CTR_CONTROL_CNTCV1)	32	R/W	0000_0000h	4.11.4.4/265
306C_0020	Frequency Modes Table 0 Register (SYS_CTR_CONTROL_CNTFID0)	32	R	007A_1200h	4.11.4.5/265
306C_0024	Frequency Modes Table 1 Register (SYS_CTR_CONTROL_CNTFID1)	32	R	0000_0200h	4.11.4.6/266
306C_0028	Frequency Modes Table 2 Register (SYS_CTR_CONTROL_CNTFID2)	32	R	0000_0000h	4.11.4.7/266
306C_0FD0	Counter ID Register (SYS_CTR_CONTROL_CNTID0)	32	R	0000_0000h	4.11.4.8/267
306A_0000	Counter Count Value Low Register (SYS_CTR_READ_CNTCV0)	32	R	0000_0000h	4.11.4.1/268
306A_0004	Counter Count Value High Register (SYS_CTR_READ_CNTCV1)	32	R	0000_0000h	4.11.4.2/268
306A_0FD0	Counter ID Register (SYS_CTR_READ_CNTID0)	32	R	0000_0000h	4.11.4.3/269

SYS_CTR_READ memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
306B_0020	Compare Count Value Low Register (SYS_CTR_COMPARE_CMPCV0)	32	R	0000_0000h	4.11.4.1/269
306B_0024	Compare Count Value High Register (SYS_CTR_COMPARE_CMPCV1)	32	R	0000_0000h	4.11.4.2/270
306B_002C	Compare Control Register (SYS_CTR_COMPARE_CMPCR)	32	R/W	0000_0000h	4.11.4.3/270
306B_0FD0	Counter ID Register (SYS_CTR_COMPARE_CNTID0)	32	R	0000_0000h	4.11.4.4/272

4.11.4.1 Counter Control Register (SYS_CTR_CONTROL_CNTCR)

The 32-bit Counter Control Register defines the basic operating configuration of the System Counter.

The System Counter operates using a fixed base frequency. However, the counter can increment at a lower, alternate frequency than the base frequency, using a correspondingly larger increment. For example, it can increment by 15625 and run at a frequency of 1/15625 of the base frequency. The two frequencies available are identified in the frequency modes table. These two frequencies are the base frequency (table entry 0) and the lower, alternate frequency (table entry 1). Setting the FCR1 bit selects the alternate frequency. Setting the FCR0 bit selects the base frequency. Setting or clearing both FCR0 and FCR1 will have no effect; the frequency will not change.

Address: 306C_0000h base + 0h offset = 306C_0000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W	Reserved															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved							FCR1	FCR0	Reserved					HDBG	EN
W	Reserved							FCR1	FCR0	Reserved					HDBG	EN
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

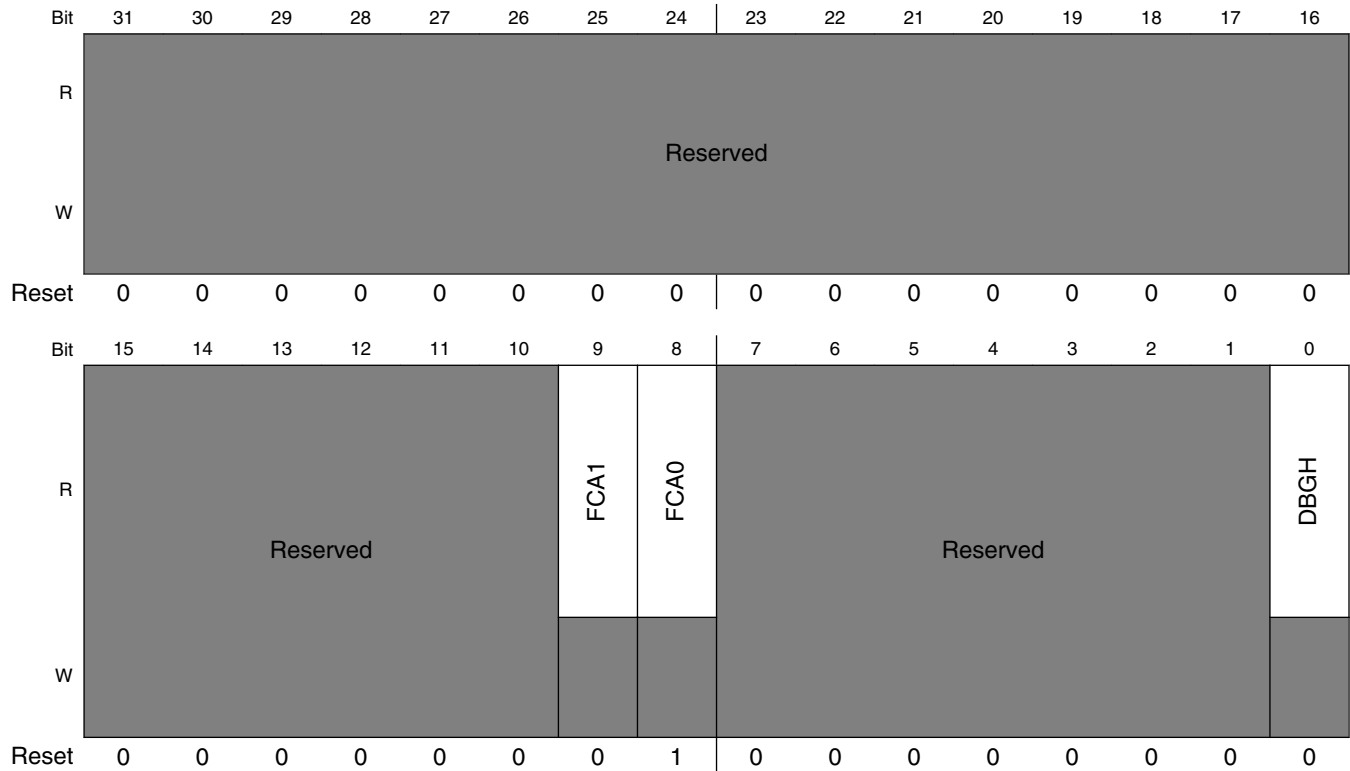
SYS_CTR_CONTROL_CNTCR field descriptions

Field	Description
31–10 -	This field is reserved. Reserved
9 FCR1	Frequency Change Request, ID 1 0 No change. 1 Select frequency modes table entry 1, the base frequency.
8 FCR0	Frequency Change Request, ID 0 0 No change. 1 Select frequency modes table entry 0, the base frequency.
7–2 -	This field is reserved. Reserved
1 HDBG	Enable Debug 0 The assertion of the debug input is ignored. 1 The assertion of the debug input causes the System Counter to halt.
0 EN	Enable Counting 0 Counter disabled 1 Counter enabled

4.11.4.2 Counter Status Register (SYS_CTR_CONTROL_CNTSR)

The system counter status register provides information concerning the clock frequency and debug state.

Address: 306C_0000h base + 4h offset = 306C_0004h



SYS_CTR_CONTROL_CNTSR field descriptions

Field	Description
31–10 -	This field is reserved. Reserved
9 FCA1	Frequency Change Acknowledge, ID 1 0 Base frequency is not selected. 1 Base frequency is selected.
8 FCA0	Frequency Change Acknowledge, ID 0 0 Base frequency is not selected. 1 Base frequency is selected.
7–1 -	This field is reserved. Reserved
0 DBGH	Debug Halt

Table continues on the next page...

SYS_CTR_CONTROL_CNTSR field descriptions (continued)

Field	Description
0	Counter is not halted by debug.
1	Counter is halted by debug.

4.11.4.3 Counter Count Value Low Register (SYS_CTR_CONTROL_CNTCV0)

The Counter Count Value Low register indicates the current count value bits 31-0.

NOTE

Writes to the CNTCV registers must be performed while operating on the base frequency only. Writes to these registers while running on the alternate frequency may have unpredictable results.

Address: 306C_0000h base + 8h offset = 306C_0008h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CNTCV0																															
W	CNTCV0																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SYS_CTR_CONTROL_CNTCV0 field descriptions

Field	Description
CNTCV0	Counter Count Value bits [31:0]

4.11.4.4 Counter Count Value High Register (SYS_CTR_CONTROL_CNTCV1)

The Counter Count Value High register indicates the current count value bits 63-32.

NOTE

Writes to the CNTCV registers must be performed while operating on the base frequency only. Writes to these registers while running on the alternate frequency may have unpredictable results.

Address: 306C_0000h base + Ch offset = 306C_000Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SYS_CTR_CONTROL_CNTCV1 field descriptions

Field	Description
31-25 -	This field is reserved. Reserved
CNTCV1	Counter Count Value bits [55:32]

4.11.4.5 Frequency Modes Table 0 Register (SYS_CTR_CONTROL_CNTFID0)

The Counter Frequency ID registers is the frequency modes table starting at offset 0x020.

Table entries are 32-bits, and each entry specifies a system counter update frequency, in Hz. The first entry in the table specifies the base frequency of the system counter. To ensure overall counter accuracy is maintained, any subsequent entries in the table are exact divisors of the base frequency.

When the system timer is operating at a lower frequency than the base frequency, the increment applied at each counter update is given by: $\text{increment} = (\text{base_frequency}) / (\text{selected_frequency})$. A 32-bit word of zero value marks the end of the table. That is, the word of memory immediately after the last entry in the table is zero.

Memory Map and Register Description

Address: 306C_0000h base + 20h offset = 306C_0020h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CNTFID0																															
W																																
Reset	0	0	0	0	0	0	0	0	0	1	1	1	1	0	1	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0

SYS_CTR_CONTROL_CNTFID0 field descriptions

Field	Description
CNTFID0	Base Frequency (24 MHz /3 = 8 MHz)

4.11.4.6 Frequency Modes Table 1 Register (SYS_CTR_CONTROL_CNTFID1)

The Counter Frequency ID registers is the frequency modes table starting at offset 0x020.

Table entries are 32-bits, and each entry specifies a system counter update frequency, in Hz. The first entry in the table specifies the base frequency of the system counter. To ensure overall counter accuracy is maintained, any subsequent entries in the table are exact divisors of the base frequency.

When the system timer is operating at a lower frequency than the base frequency, the increment applied at each counter update is given by: $\text{increment} = (\text{base_frequency}) / (\text{selected_frequency})$ A 32-bit word of zero value marks the end of the table. That is, the word of memory immediately after the last entry in the table is zero.

Address: 306C_0000h base + 24h offset = 306C_0024h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CNTFID1																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0

SYS_CTR_CONTROL_CNTFID1 field descriptions

Field	Description
CNTFID1	Alternate Frequency (32 kHz /64 = 512 Hz)

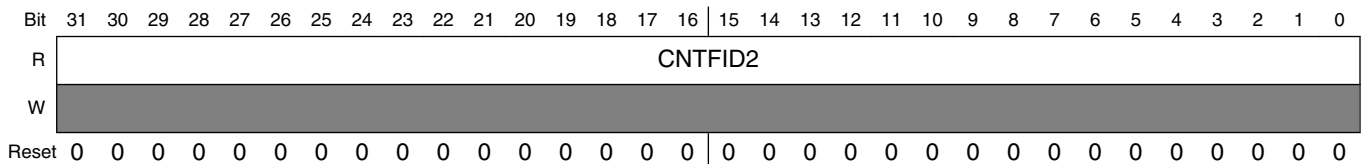
4.11.4.7 Frequency Modes Table 2 Register (SYS_CTR_CONTROL_CNTFID2)

The Counter Frequency ID registers is the frequency modes table starting at offset 0x020.

Table entries are 32-bits, and each entry specifies a system counter update frequency, in Hz. The first entry in the table specifies the base frequency of the system counter. To ensure overall counter accuracy is maintained, any subsequent entries in the table are exact divisors of the base frequency.

When the system timer is operating at a lower frequency than the base frequency, the increment applied at each counter update is given by: $\text{increment} = (\text{base_frequency}) / (\text{selected_frequency})$. A 32-bit word of zero value marks the end of the table. That is, the word of memory immediately after the last entry in the table is zero.

Address: 306C_0000h base + 28h offset = 306C_0028h



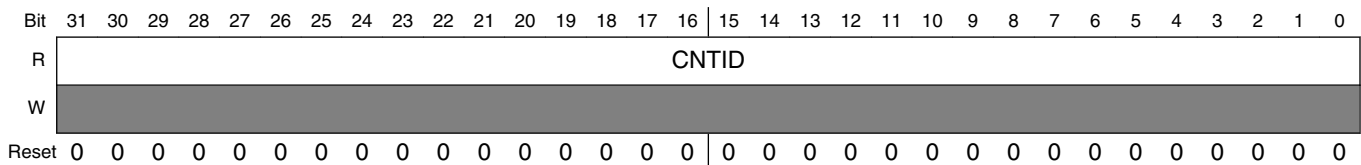
SYS_CTR_CONTROL_CNTFID2 field descriptions

Field	Description
CNTFID2	End Marker

4.11.4.8 Counter ID Register (SYS_CTR_CONTROL_CNTID0)

The Counter ID register indicates the architecture version 0.

Address: 306C_0000h base + FD0h offset = 306C_0FD0h



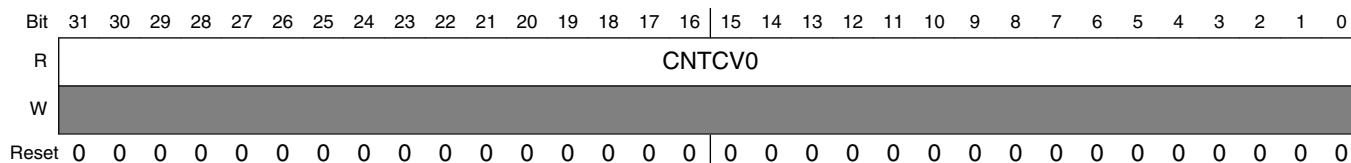
SYS_CTR_CONTROL_CNTID0 field descriptions

Field	Description
CNTID	Counter Identification. Counter ID 0.

4.11.4.1 Counter Count Value Low Register (SYS_CTR_READ_CNTCV0)

The Counter Count Value Low register indicates the current count value bits 31-0.

Address: 306A_0000h base + 0h offset = 306A_0000h



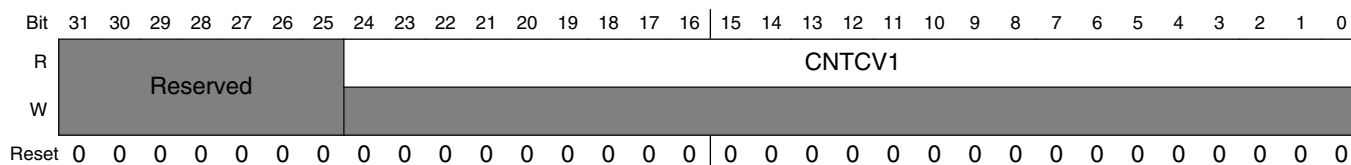
SYS_CTR_READ_CNTCV0 field descriptions

Field	Description
CNTCV0	Counter Count Value bits [31:0]

4.11.4.2 Counter Count Value High Register (SYS_CTR_READ_CNTCV1)

The Counter Count Value High register indicates the current count value bits 63-32.

Address: 306A_0000h base + 4h offset = 306A_0004h



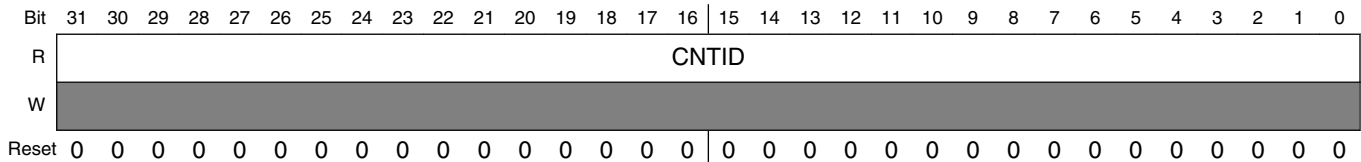
SYS_CTR_READ_CNTCV1 field descriptions

Field	Description
31–25 -	This field is reserved. Reserved
CNTCV1	Counter Count Value bits [55:32]. Bits[63:56] are always zero.

4.11.4.3 Counter ID Register (SYS_CTR_READ_CNTID0)

The Counter ID register indicates the architecture version 0.

Address: 306A_0000h base + FD0h offset = 306A_0FD0h



SYS_CTR_READ_CNTID0 field descriptions

Field	Description
CNTID	Counter Identification. Counter ID 0.

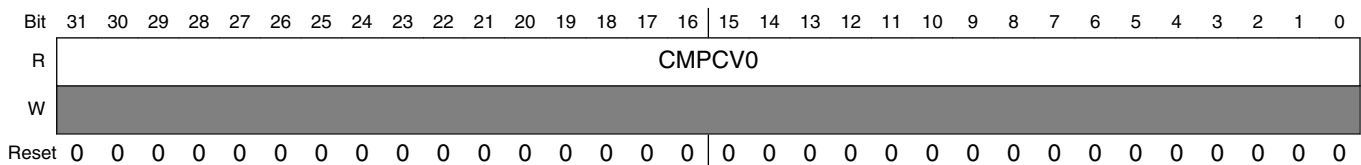
4.11.4.1 Compare Count Value Low Register (SYS_CTR_COMPARE_CMPCV0)

The Compare Count Value Low register indicates the current count value bits 31-0.

NOTE

Writes to the CMPCV registers must be performed while operating on the base frequency only. Writes to these registers while running on the alternate frequency may have unpredictable results.

Address: 306B_0000h base + 20h offset = 306B_0020h



SYS_CTR_COMPARE_CMPCV0 field descriptions

Field	Description
CMPCV0	Compare Count Value bits [31:0]

4.11.4.2 Compare Count Value High Register (SYS_CTR_COMPARE_CMPCV1)

The Compare Count Value High register indicates the current count value bits 63-32.

NOTE

Writes to the CMPCV registers must be performed while operating on the base frequency only. Writes to these registers while running on the alternate frequency may have unpredictable results.

Address: 306B_0000h base + 24h offset = 306B_0024h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved							CMPCV1																								
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SYS_CTR_COMPARE_CMPCV1 field descriptions

Field	Description
31-25 -	This field is reserved. Reserved
CMPCV1	Compare Count Value bits [55:32]. Bits[63:56] are always zero.

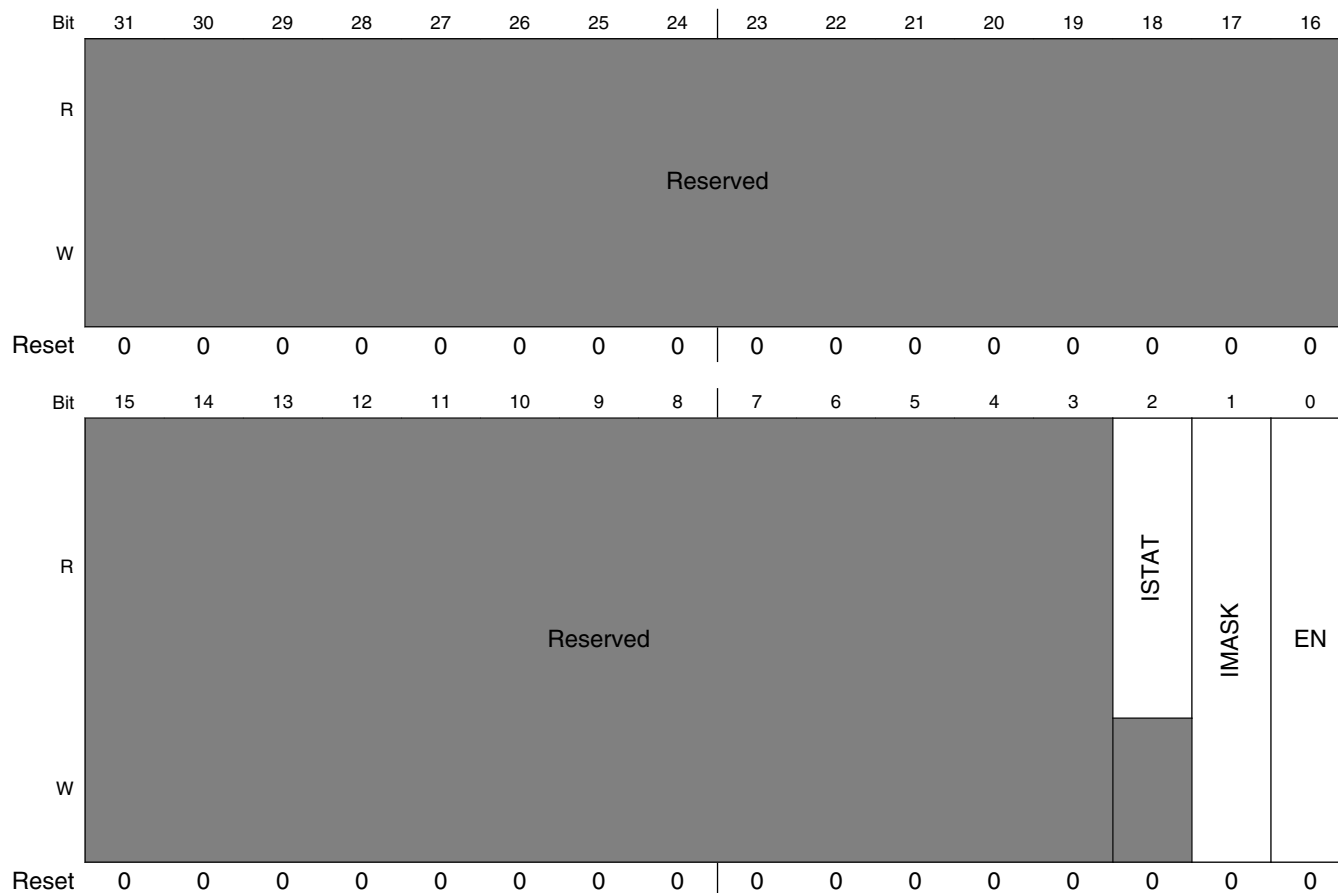
4.11.4.3 Compare Control Register (SYS_CTR_COMPARE_CMPCR)

The compare control register provides control and status of the compare function. When enabled, the ISTAT bit indicates whether the counter value is greater than or equal to the value in the compare value register (CMPCV). The ISTAT equation is:

$$\text{ISTAT} = (\text{CNTCV} \geq \text{CMPCV})$$

ISTAT takes no account of the value of the IMASK bit. If ISTAT is set to 1 and IMASK is 0, then the interrupt request is asserted. Clearing the enable bit (EN=0) will clear the status bit (ISTAT=0) and will negate the interrupt output signal.

Address: 306B_0000h base + 2Ch offset = 306B_002Ch

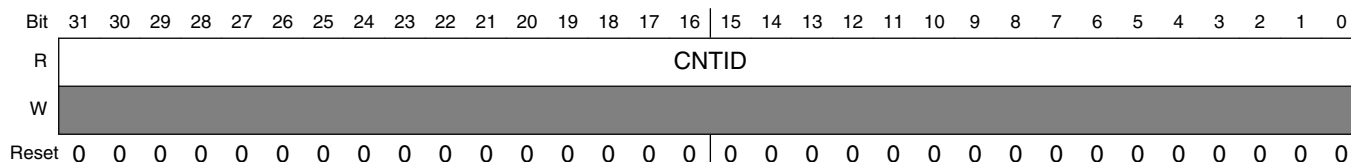
**SYS_CTR_COMPARE_CMPCR field descriptions**

Field	Description
31–3 -	This field is reserved. Reserved
2 ISTAT	Compare (interrupt) status 0 Counter value is less than the compare value or compare is disabled. 1 Counter value is greater than or equal to the compare value and compare is enabled.
1 IMASK	Interrupt request mask 0 Interrupt output signal is not masked. 1 Interrupt output signal is masked.
0 EN	Enable the compare function 0 Compare disabled 1 Compare enabled

4.11.4.4 Counter ID Register (SYS_CTR_COMPARE_CNTID0)

The Counter ID register indicates the architecture version 0.

Address: 306B_0000h base + FD0h offset = 306B_0FD0h



SYS_CTR_COMPARE_CNTID0 field descriptions

Field	Description
CNTID	Counter Identification. Counter ID 0.

Each compare frame consists of a 256 byte region. Each compare frame has its own compare value and control register. Each compare frame is capable of generating one maskable interrupt.

SYS_CTR_COMPARE memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
306C_0000	Counter Control Register (SYS_CTR_CONTROL_CNTCR)	32	R/W	0000_0000h	4.11.4.1/261
306C_0004	Counter Status Register (SYS_CTR_CONTROL_CNTSR)	32	R	0000_0100h	4.11.4.2/263
306C_0008	Counter Count Value Low Register (SYS_CTR_CONTROL_CNTCV0)	32	R/W	0000_0000h	4.11.4.3/264
306C_000C	Counter Count Value High Register (SYS_CTR_CONTROL_CNTCV1)	32	R/W	0000_0000h	4.11.4.4/265
306C_0020	Frequency Modes Table 0 Register (SYS_CTR_CONTROL_CNTFID0)	32	R	007A_1200h	4.11.4.5/265
306C_0024	Frequency Modes Table 1 Register (SYS_CTR_CONTROL_CNTFID1)	32	R	0000_0200h	4.11.4.6/266
306C_0028	Frequency Modes Table 2 Register (SYS_CTR_CONTROL_CNTFID2)	32	R	0000_0000h	4.11.4.7/266
306C_0FD0	Counter ID Register (SYS_CTR_CONTROL_CNTID0)	32	R	0000_0000h	4.11.4.8/267
306A_0000	Counter Count Value Low Register (SYS_CTR_READ_CNTCV0)	32	R	0000_0000h	4.11.4.1/268
306A_0004	Counter Count Value High Register (SYS_CTR_READ_CNTCV1)	32	R	0000_0000h	4.11.4.2/268
306A_0FD0	Counter ID Register (SYS_CTR_READ_CNTID0)	32	R	0000_0000h	4.11.4.3/269

SYS_CTR_COMPARE memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
306B_0020	Compare Count Value Low Register (SYS_CTR_COMPARE_CMPCV0)	32	R	0000_0000h	4.11.4.1/269
306B_0024	Compare Count Value High Register (SYS_CTR_COMPARE_CMPCV1)	32	R	0000_0000h	4.11.4.2/270
306B_002C	Compare Control Register (SYS_CTR_COMPARE_CMPCR)	32	R/W	0000_0000h	4.11.4.3/270
306B_0FD0	Counter ID Register (SYS_CTR_COMPARE_CNTID0)	32	R	0000_0000h	4.11.4.4/272

4.11.4.1 Counter Control Register (SYS_CTR_CONTROL_CNTCR)

The 32-bit Counter Control Register defines the basic operating configuration of the System Counter.

The System Counter operates using a fixed base frequency. However, the counter can increment at a lower, alternate frequency than the base frequency, using a correspondingly larger increment. For example, it can increment by 15625 and run at a frequency of 1/15625 of the base frequency. The two frequencies available are identified in the frequency modes table. These two frequencies are the base frequency (table entry 0) and the lower, alternate frequency (table entry 1). Setting the FCR1 bit selects the alternate frequency. Setting the FCR0 bit selects the base frequency. Setting or clearing both FCR0 and FCR1 will have no effect; the frequency will not change.

Address: 306C_0000h base + 0h offset = 306C_0000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W	Reserved															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved							FCR1	FCR0	Reserved					HDBG	EN
W	Reserved							FCR1	FCR0	Reserved					HDBG	EN
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

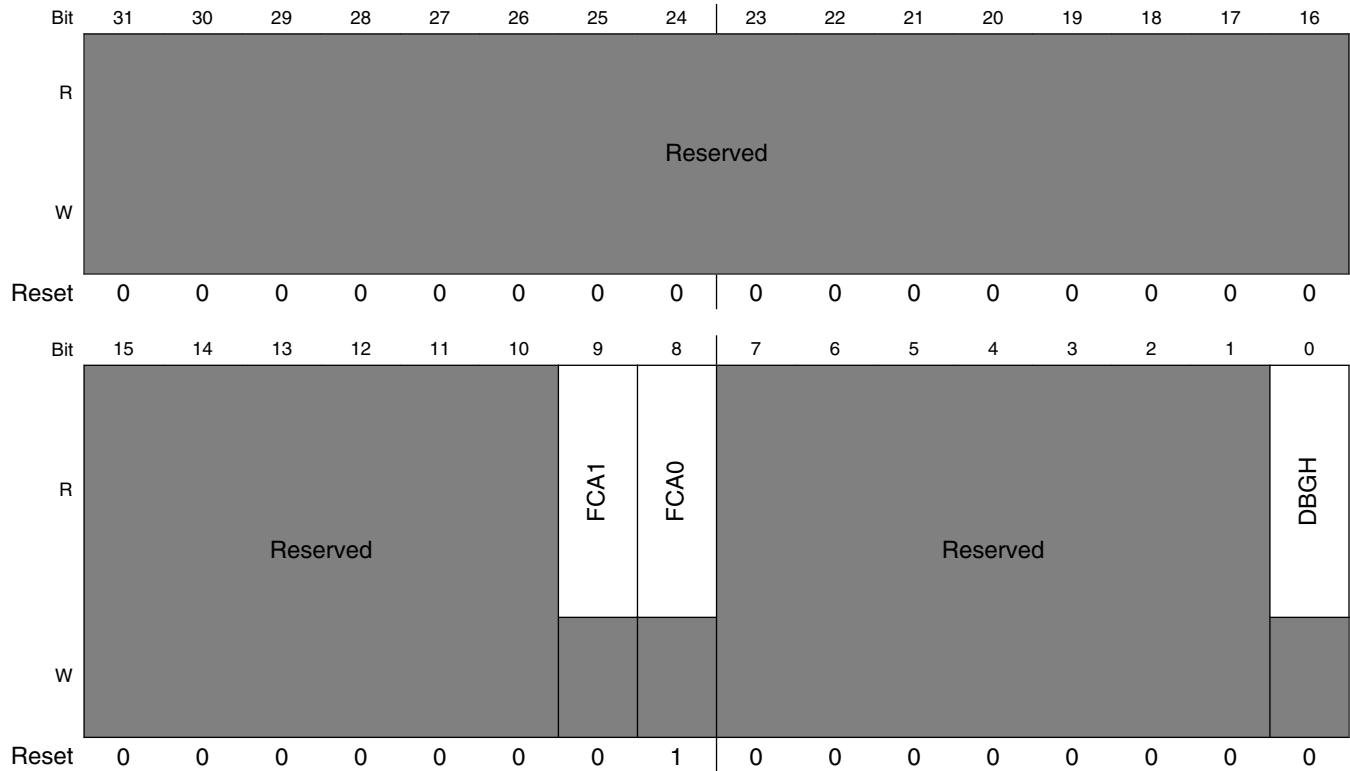
SYS_CTR_CONTROL_CNTCR field descriptions

Field	Description
31–10 -	This field is reserved. Reserved
9 FCR1	Frequency Change Request, ID 1 0 No change. 1 Select frequency modes table entry 1, the base frequency.
8 FCR0	Frequency Change Request, ID 0 0 No change. 1 Select frequency modes table entry 0, the base frequency.
7–2 -	This field is reserved. Reserved
1 HDBG	Enable Debug 0 The assertion of the debug input is ignored. 1 The assertion of the debug input causes the System Counter to halt.
0 EN	Enable Counting 0 Counter disabled 1 Counter enabled

4.11.4.2 Counter Status Register (SYS_CTR_CONTROL_CNTSR)

The system counter status register provides information concerning the clock frequency and debug state.

Address: 306C_0000h base + 4h offset = 306C_0004h



SYS_CTR_CONTROL_CNTSR field descriptions

Field	Description
31–10 -	This field is reserved. Reserved
9 FCA1	Frequency Change Acknowledge, ID 1 0 Base frequency is not selected. 1 Base frequency is selected.
8 FCA0	Frequency Change Acknowledge, ID 0 0 Base frequency is not selected. 1 Base frequency is selected.
7–1 -	This field is reserved. Reserved
0 DBGH	Debug Halt

Table continues on the next page...

SYS_CTR_CONTROL_CNTSR field descriptions (continued)

Field	Description
0	Counter is not halted by debug.
1	Counter is halted by debug.

4.11.4.3 Counter Count Value Low Register (SYS_CTR_CONTROL_CNTCV0)

The Counter Count Value Low register indicates the current count value bits 31-0.

NOTE

Writes to the CNTCV registers must be performed while operating on the base frequency only. Writes to these registers while running on the alternate frequency may have unpredictable results.

Address: 306C_0000h base + 8h offset = 306C_0008h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CNTCV0																															
W	CNTCV0																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SYS_CTR_CONTROL_CNTCV0 field descriptions

Field	Description
CNTCV0	Counter Count Value bits [31:0]

4.11.4.4 Counter Count Value High Register (SYS_CTR_CONTROL_CNTCV1)

The Counter Count Value High register indicates the current count value bits 63-32.

NOTE

Writes to the CNTCV registers must be performed while operating on the base frequency only. Writes to these registers while running on the alternate frequency may have unpredictable results.

Address: 306C_0000h base + Ch offset = 306C_000Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								CNTCV1																							
W	Reserved								CNTCV1																							
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SYS_CTR_CONTROL_CNTCV1 field descriptions

Field	Description
31–25 -	This field is reserved. Reserved
CNTCV1	Counter Count Value bits [55:32]

4.11.4.5 Frequency Modes Table 0 Register (SYS_CTR_CONTROL_CNTFID0)

The Counter Frequency ID registers is the frequency modes table starting at offset 0x020.

Table entries are 32-bits, and each entry specifies a system counter update frequency, in Hz. The first entry in the table specifies the base frequency of the system counter. To ensure overall counter accuracy is maintained, any subsequent entries in the table are exact divisors of the base frequency.

When the system timer is operating at a lower frequency than the base frequency, the increment applied at each counter update is given by: $\text{increment} = (\text{base_frequency}) / (\text{selected_frequency})$. A 32-bit word of zero value marks the end of the table. That is, the word of memory immediately after the last entry in the table is zero.

Memory Map and Register Description

Address: 306C_0000h base + 20h offset = 306C_0020h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CNTFID0																															
W																																
Reset	0	0	0	0	0	0	0	0	0	1	1	1	1	0	1	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0

SYS_CTR_CONTROL_CNTFID0 field descriptions

Field	Description
CNTFID0	Base Frequency (24 MHz /3 = 8 MHz)

4.11.4.6 Frequency Modes Table 1 Register (SYS_CTR_CONTROL_CNTFID1)

The Counter Frequency ID registers is the frequency modes table starting at offset 0x020.

Table entries are 32-bits, and each entry specifies a system counter update frequency, in Hz. The first entry in the table specifies the base frequency of the system counter. To ensure overall counter accuracy is maintained, any subsequent entries in the table are exact divisors of the base frequency.

When the system timer is operating at a lower frequency than the base frequency, the increment applied at each counter update is given by: $\text{increment} = (\text{base_frequency}) / (\text{selected_frequency})$ A 32-bit word of zero value marks the end of the table. That is, the word of memory immediately after the last entry in the table is zero.

Address: 306C_0000h base + 24h offset = 306C_0024h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CNTFID1																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0

SYS_CTR_CONTROL_CNTFID1 field descriptions

Field	Description
CNTFID1	Alternate Frequency (32 kHz /64 = 512 Hz)

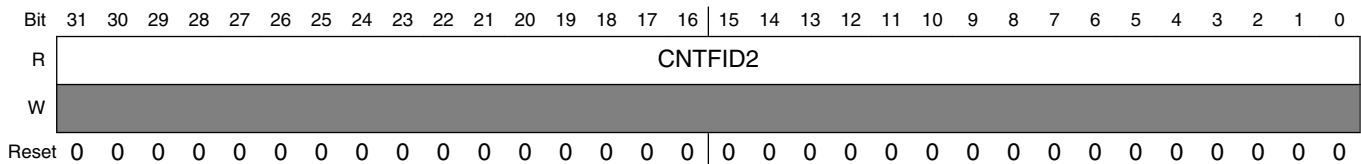
4.11.4.7 Frequency Modes Table 2 Register (SYS_CTR_CONTROL_CNTFID2)

The Counter Frequency ID registers is the frequency modes table starting at offset 0x020.

Table entries are 32-bits, and each entry specifies a system counter update frequency, in Hz. The first entry in the table specifies the base frequency of the system counter. To ensure overall counter accuracy is maintained, any subsequent entries in the table are exact divisors of the base frequency.

When the system timer is operating at a lower frequency than the base frequency, the increment applied at each counter update is given by: $\text{increment} = (\text{base_frequency}) / (\text{selected_frequency})$. A 32-bit word of zero value marks the end of the table. That is, the word of memory immediately after the last entry in the table is zero.

Address: 306C_0000h base + 28h offset = 306C_0028h



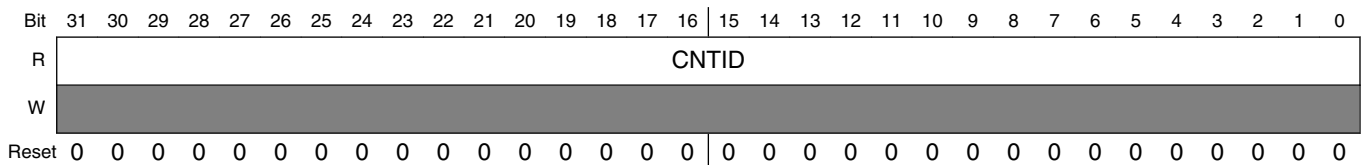
SYS_CTR_CONTROL_CNTFID2 field descriptions

Field	Description
CNTFID2	End Marker

4.11.4.8 Counter ID Register (SYS_CTR_CONTROL_CNTID0)

The Counter ID register indicates the architecture version 0.

Address: 306C_0000h base + FD0h offset = 306C_0FD0h



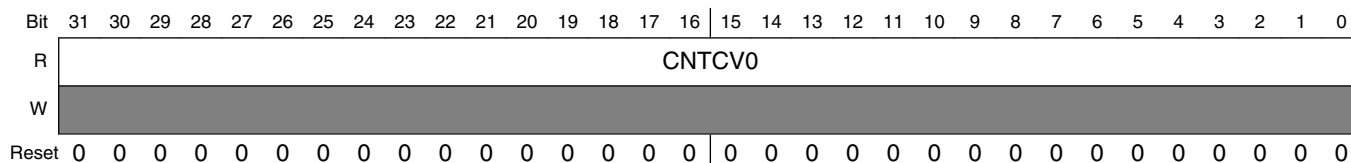
SYS_CTR_CONTROL_CNTID0 field descriptions

Field	Description
CNTID	Counter Identification. Counter ID 0.

4.11.4.1 Counter Count Value Low Register (SYS_CTR_READ_CNTCV0)

The Counter Count Value Low register indicates the current count value bits 31-0.

Address: 306A_0000h base + 0h offset = 306A_0000h



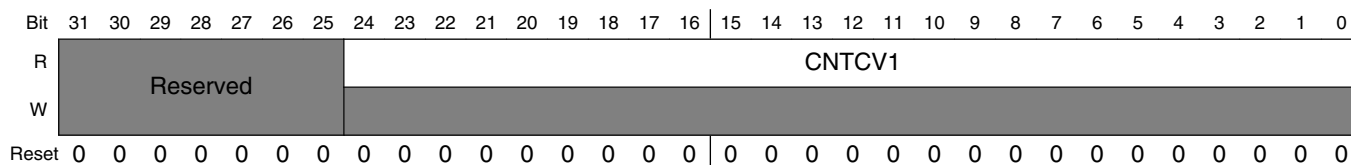
SYS_CTR_READ_CNTCV0 field descriptions

Field	Description
CNTCV0	Counter Count Value bits [31:0]

4.11.4.2 Counter Count Value High Register (SYS_CTR_READ_CNTCV1)

The Counter Count Value High register indicates the current count value bits 63-32.

Address: 306A_0000h base + 4h offset = 306A_0004h



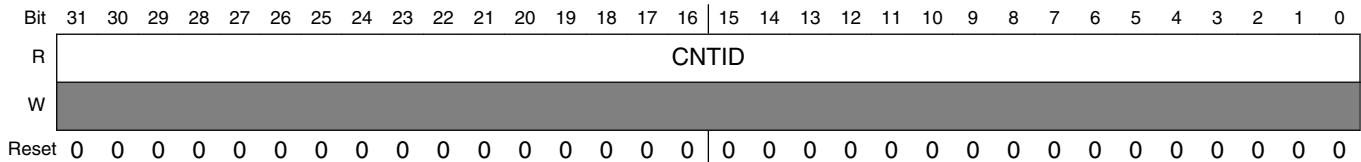
SYS_CTR_READ_CNTCV1 field descriptions

Field	Description
31–25 -	This field is reserved. Reserved
CNTCV1	Counter Count Value bits [55:32]. Bits[63:56] are always zero.

4.11.4.3 Counter ID Register (SYS_CTR_READ_CNTID0)

The Counter ID register indicates the architecture version 0.

Address: 306A_0000h base + FD0h offset = 306A_0FD0h



SYS_CTR_READ_CNTID0 field descriptions

Field	Description
CNTID	Counter Identification. Counter ID 0.

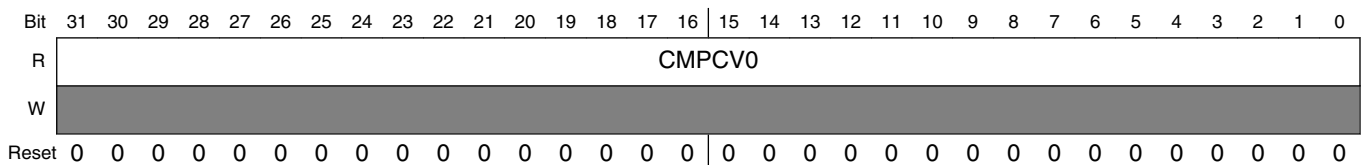
4.11.4.1 Compare Count Value Low Register (SYS_CTR_COMPARE_CMPCV0)

The Compare Count Value Low register indicates the current count value bits 31-0.

NOTE

Writes to the CMPCV registers must be performed while operating on the base frequency only. Writes to these registers while running on the alternate frequency may have unpredictable results.

Address: 306B_0000h base + 20h offset = 306B_0020h



SYS_CTR_COMPARE_CMPCV0 field descriptions

Field	Description
CMPCV0	Compare Count Value bits [31:0]

4.11.4.2 Compare Count Value High Register (SYS_CTR_COMPARE_CMPCV1)

The Compare Count Value High register indicates the current count value bits 63-32.

NOTE

Writes to the CMPCV registers must be performed while operating on the base frequency only. Writes to these registers while running on the alternate frequency may have unpredictable results.

Address: 306B_0000h base + 24h offset = 306B_0024h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved							CMPCV1																								
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SYS_CTR_COMPARE_CMPCV1 field descriptions

Field	Description
31-25 -	This field is reserved. Reserved
CMPCV1	Compare Count Value bits [55:32]. Bits[63:56] are always zero.

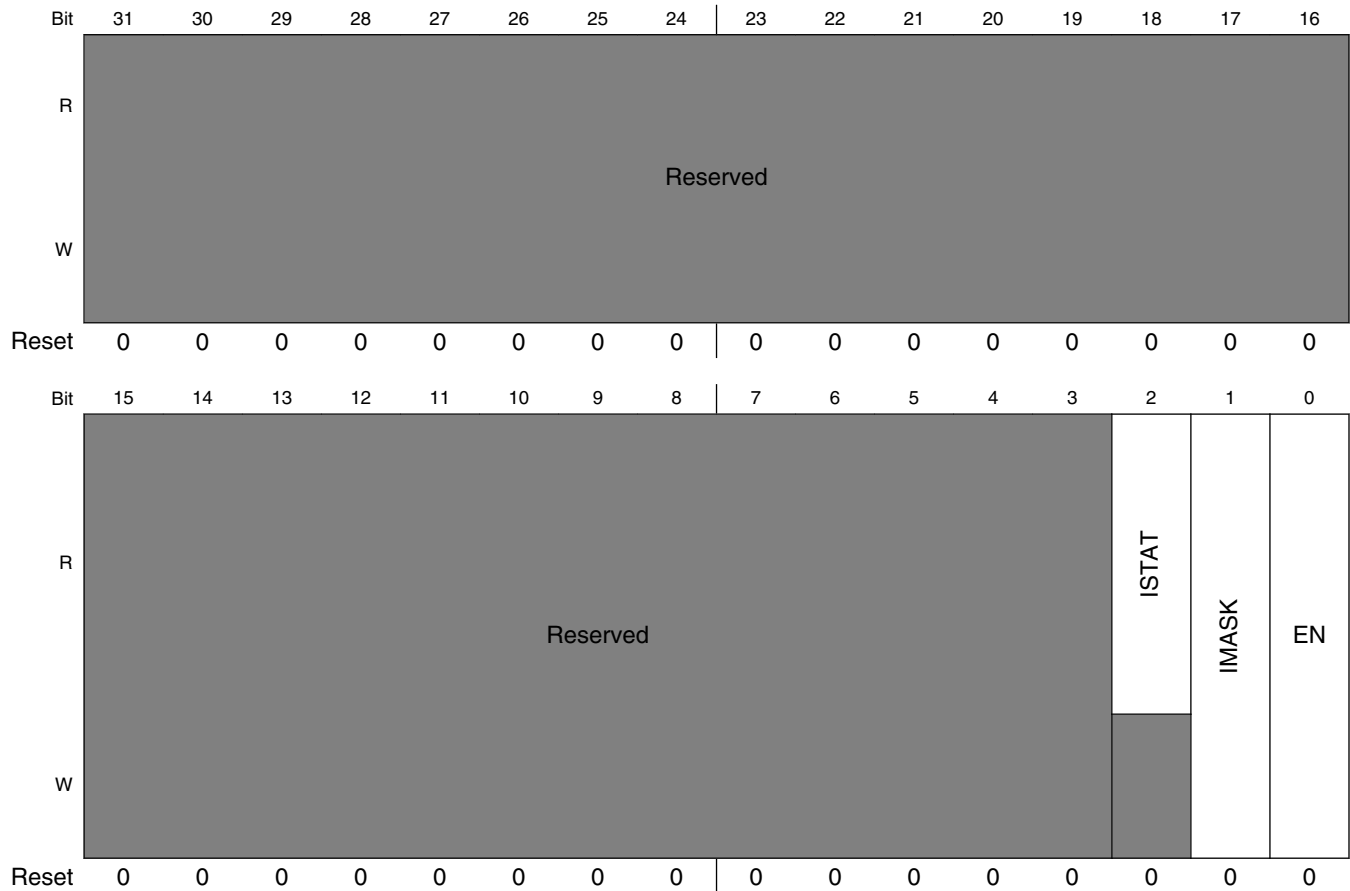
4.11.4.3 Compare Control Register (SYS_CTR_COMPARE_CMPCR)

The compare control register provides control and status of the compare function. When enabled, the ISTAT bit indicates whether the counter value is greater than or equal to the value in the compare value register (CMPCV). The ISTAT equation is:

$$\text{ISTAT} = (\text{CNTCV} \geq \text{CMPCV})$$

ISTAT takes no account of the value of the IMASK bit. If ISTAT is set to 1 and IMASK is 0, then the interrupt request is asserted. Clearing the enable bit (EN=0) will clear the status bit (ISTAT=0) and will negate the interrupt output signal.

Address: 306B_0000h base + 2Ch offset = 306B_002Ch

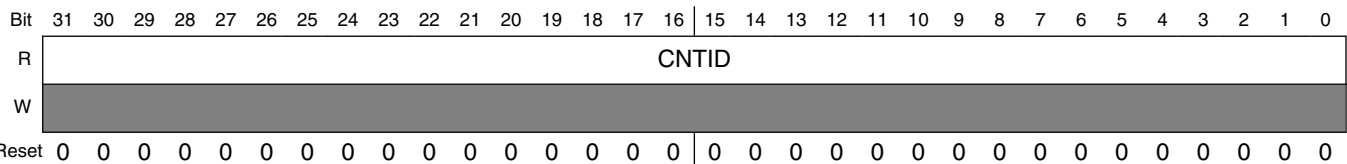
**SYS_CTR_COMPARE_CMPCR field descriptions**

Field	Description
31–3 -	This field is reserved. Reserved
2 ISTAT	Compare (interrupt) status 0 Counter value is less than the compare value or compare is disabled. 1 Counter value is greater than or equal to the compare value and compare is enabled.
1 IMASK	Interrupt request mask 0 Interrupt output signal is not masked. 1 Interrupt output signal is masked.
0 EN	Enable the compare function 0 Compare disabled 1 Compare enabled

4.11.4.4 Counter ID Register (SYS_CTR_COMPARE_CNTID0)

The Counter ID register indicates the architecture version 0.

Address: 306B_0000h base + FD0h offset = 306B_0FD0h



SYS_CTR_COMPARE_CNTID0 field descriptions

Field	Description
CNTID	Counter Identification. Counter ID 0.

4.12 TrustZone Address Space Controller (TZASC)

4.12.1 Overview

The TrustZone Address Space Controller (TZASC) protects security-sensitive SW and data in a trusted execution environment against potentially compromised SW running on the platform.

The TZASC block diagram is shown in figure below.

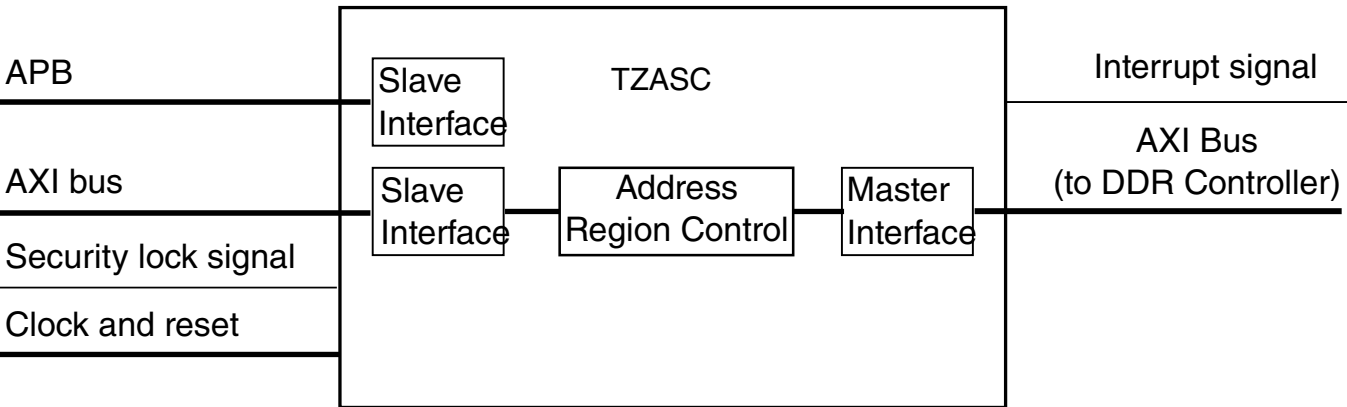


Figure 4-27. TZASC Block Diagram

The TZASC is an IP by Arm ("CoreLink™ TrustZone Address Space Controller TZC-380"), designed to provide configurable protection over program (SW) memory space.

The main features of TZASC are:

- Supports 16 independent address regions
- Access controls are independently programmable for each address region
- Sensitive registers may be locked
- Host interrupt may be programmed to signal attempted access control violations
- AXI master/slave interfaces for transactions
- APB slave interface for configuration and status reporting

NOTE

When illegal access is blocked by TZASC, the AXI ID read from the TZASC is not the AXI ID from the bus master. It cannot be used to identify the master which sends out the illegal access.

4.12.2 Clocks

The table found here describes the clock sources for TZASC.

See [Clock Controller Module \(CCM\)](#) for clock setting, configuration, and gating information.

Table 4-29. TZASC Clocks

Clock name	Clock Root	Description
aclk	ccm_clk_root	Module clock

4.12.3 Address Mapping in various memory mapping modes

The TZASC region base address starts at the beginning of DDR memory space (0x40000000) instead of the beginning of memory map (0x00000000). In this case the addresses configured in TZASC controller will be 1GB (0x40000000) offset and does not match the local addresses.

For example, setting region_setup_low_x=0xBE000000 maps DDR_ADDR=0xFE000000, the same behavior is observed with fail_address_x registers.

Memory "aliasing" implications on TZASC settings - in systems which does not utilize the maximal supported DDR space the controller is designed for, the whole DDR memory map becomes "aliased" (replicated) by the size of the physical memory used. In such cases, the TZASC must be configured to protect all aliased regions as well (i.e. effectively reducing the number of available TZASC regions, since all aliased regions must be handled, for each "real" space needing protection).

For complete details on TZASC functionality and the programming model, see the Arm document, "CoreLink™ TrustZone Address Space Controller TZC-380 Technical Reference Manual, (Rev r0p1 or newer)", available at <http://infocenter.arm.com>.

4.13 System Debug

4.13.1 Debug

4.13.1.1 Debug Architecture

The chapter describes the debug architecture of the chip.

4.13.1.2 Debug System Features

The chip debug is based on Arm's CoreSight platform, with support for Quad-core A53 platform and Cortex-M4 core. The key features of the debug system include:

- Support 5-pins(JTAG) interface.
- Support both non-intrusive and halt-mode trace/debug options.
- MDM-AP registers for debugger to control mutli-core halt/resume cores.
- Trace Memory Controller (TMC) is used to enable capturing trace.
- 4KB in SOC trace block.
- ETR is used to allow routing trace data to system memory.
- Support ARM real time trace interface (TPIU) 16-bit @133MHz.
- Support cross trigger between Quad Cortex-A53 and Cortex-M4.
- 4 JTAG security levels, via SJC security functions together with e-Fuse (challenge response, field return, intrusive detection)

4.13.1.3 System level debug architecture

The debug architecture is shown in the following figure:

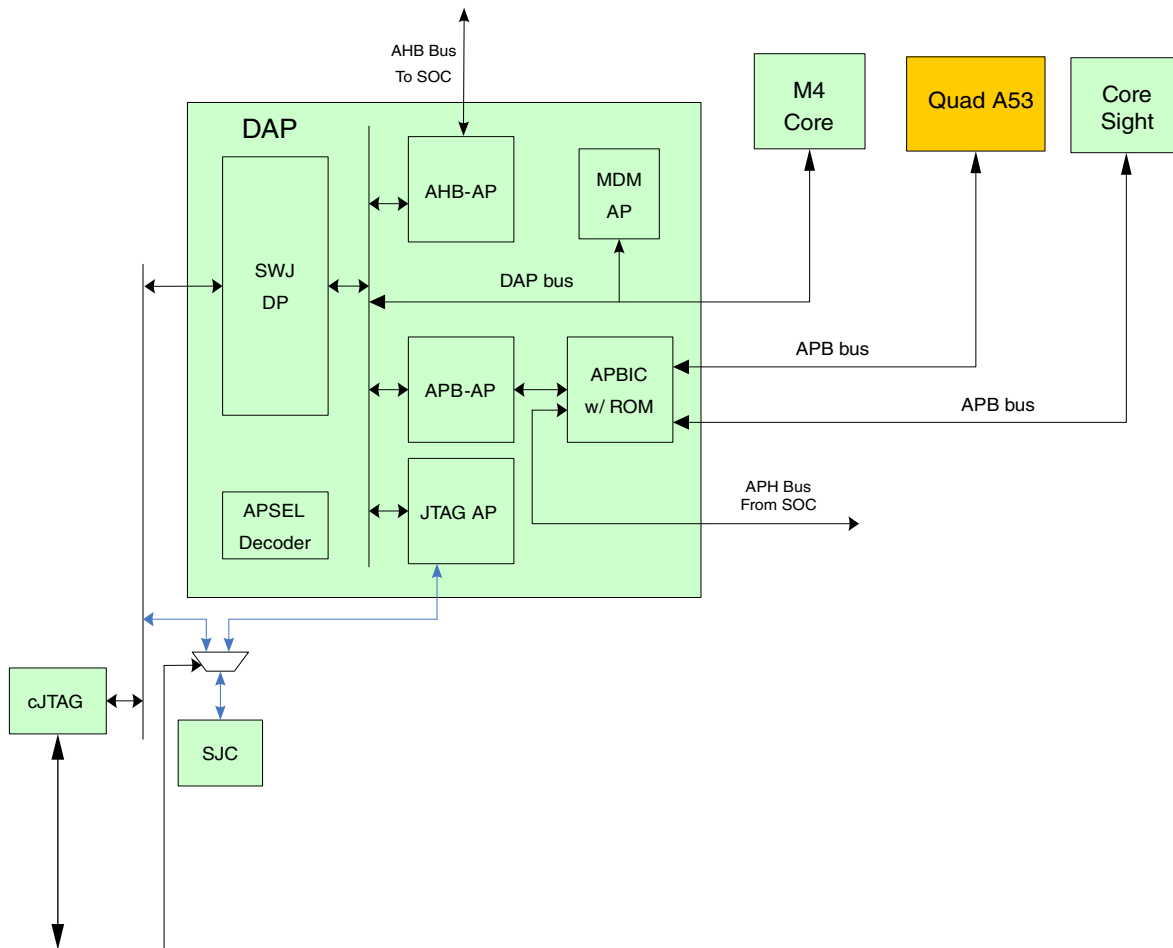


Figure 4-28. Debug Architecture Diagram

4.13.1.4 Functional description

This section gives a brief overview of the modules that are implemented within the Cortex-M/Cortex-A Core Platform. The debug blocks are part of the overall CoreSight platform debug system, which include the ETR, CTM, CTI, ATB replicator, APB address decode, TPIU and DAP. The CoreSight™ compatible Embedded Trace Macrocell (ETM) enables traces of program flow to be collected, compressed, and fed into the trace infrastructure. The Cross Trigger Interface (CTI) is included to provide a common programming model for use by the debug tools, control the trigger sources, and interface to the Cross Trigger Matrix (CTM). The debug is controlled via an ARM Debug Access Port (DAP).

4.13.1.4.1 Embedded Trace Router (ETR)

The Embedded Trace Router (ETR) is a CoreSight trace component which buffers trace data before dumping that data into DRAM through the AXI bus (via AXI master port). The output of the funnel is the ETR, and has the ability to buffer a certain amount before writing the data to DRAM.

4.13.1.4.2 Embedded Trace Macrocell (ETM)

The Embedded Trace Macrocell (ETM) is a CoreSight component. The ETM trace unit is a module that performs real-time instruction flow tracing based on the Embedded Trace Macrocell (ETM) architecture ETMv4.

4.13.1.4.3 Cross Trigger Matrix (CTM)

The CoreSight CTI channel signals from all the cores are combined using a Cross Trigger Matrix (CTM) block so that a single cross trigger channel interface is presented in the Cortex processor. This module can combine four internal channel interfaces corresponding to each core along with one external channel interface.

In the Cortex processor CTM, the external channel output is driven by the OR output of all internal channel outputs. Each internal channel input is driven by the OR output of internal channel outputs of all other CTIs in addition to the external channel input.

4.13.1.4.4 Cross Trigger Interface (CTI)

The Cortex processor has a single external cross trigger channel interface. This external interface is connected to the CoreSight Cross Trigger Interface (CTI) interface corresponding to each core through a Cross Trigger Matrix (CTM). A number of Embedded Cross Trigger (ECT) trigger inputs and trigger outputs are connected between debug components in the Cortex-A processor and CoreSight CTI blocks.

The CTI enables the debug logic, ETM trace unit, and performance monitoring, to interact with each other and with other CoreSight components. This is called cross triggering. For example, you configure the CTI to generate an interrupt when the ETM trace unit trigger event occurs.

4.13.1.4.5 AMBA Trace Bus Interface (ATB)

The AMBA Trace Bus (ATB) CoreSight debug components are part of the Trace stream block. The ATB components consist of the ATB Funnel and ATB Replicator. The ATB Funnel is used to merge several streams together to produce a single output to the ATB Replicator. The ATB Replicator enables two trace sinks to be wired together and receive ATB trace data from the same trace source.

4.13.1.4.6 Advanced Peripheral Bus Interface (APB)

The APB asynchronous interface / bridge connects several debug components. Each component has a single separate APB interface from the APB-IC (w/ ROM), which hooks up to a separate APB asynchronous component. The APB interface connects the core debug functions to the DAP.

4.13.1.4.7 Instrumentation Trace Macrocell (ITM)

The Instrumentation Trace Macrocell (ITM) is a application-driven trace source that supports printf style debugging to trace operating system and application events. The ITM generates diagnostic system information as packets. Multiple sources can generate packets. If multiple sources generate packets at the same time, the ITM arbitrates the order in which packets are output. These sources in decreasing order of priority are:

- **Software trace.** Software can write directly to ITM stimulus registers to generate packets.
- **Hardware trace.** The DWT generates these packets, and the ITM outputs them.
- **Time stamping.** Timestamps are generated relative to packets. The ITM contains a 21-bit counter to generate the timestamp. The Cortex-M processor clock or the bitclock rate of the Serial Wire Viewer (SWV) output clocks the counter.

4.13.1.4.8 Data Watchpoint and Trace (DWT)

The Data Watchpoint and Trace (DWT) debug unit provides watchpoints, data tracing, and system profiling for the processor.

4.13.1.4.9 Debug Access Port (DAP)

The Debug Access Port (DAP) is a physical port that connects to external debug tools. The DAP is part of the standardized ARM Debug Interface, and provides a bridge between a JTAG pin interface and on-chip memory mapped peripherals via the DAP bus. Transactions generated by the DAP are referred to as External Debugger Accesses.

4.13.1.5 JTAG topology

There is only one JTAG on the chip, and two JTAG modes are supported. Select via the JTAG_MOD pin.

- Debug mode: JTAG_MOD == 0, DAP is the only TAP controller in the daisy chain. SJC will be attached to JTAG-AP of DAP.
- Test mode: JTAG_MOD == 1, SJC is the only TAP controller in the daisy chain. 1149.1-compliant, and support 1149.6 AC coupled test.

When the JTAG interface is in Debug Mode, it can be operating in standard 5-pin JTAG interface. cJTAG/SWD interface is not supported by this chip.

4.14 System JTAG Controller (SJC)

4.14.1 Overview

The System JTAG Controller (SJC) provides debug and test control with the maximum security.

The test access port (TAP) is designed to support features compatible with the IEEE Standard 1149.1 v2001 (JTAG).

The figure below shows an overview of the JTAG architecture.

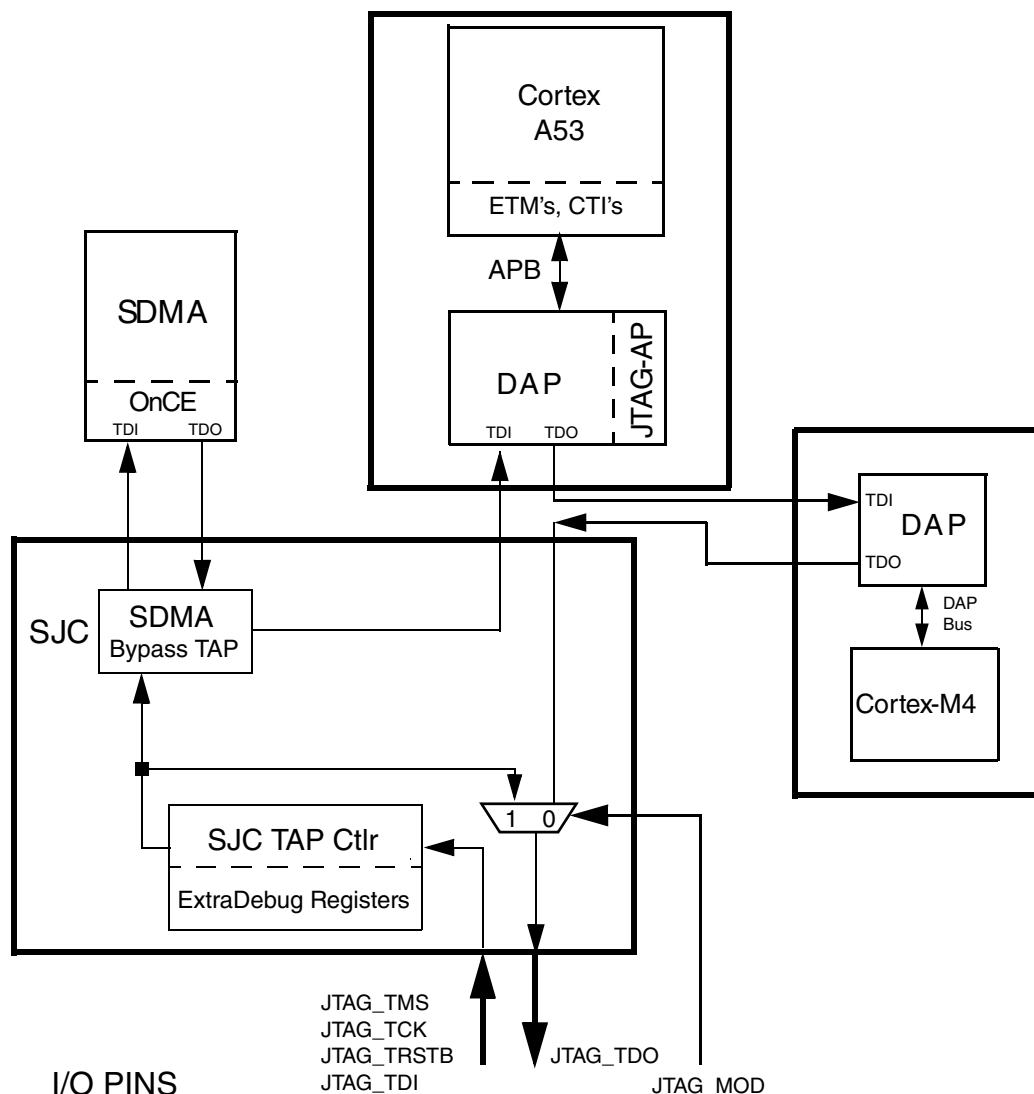


Figure 4-29. System JTAG Controller (SJC) Block Diagram

4.14.1.1 Features

The JTAG interface of chip, shared by SJC and DFT_TAP (for IEEE1149.1), provides the following capabilities:

- JTAG IEEE1149.1 mandatory instructions, see [EXTEST Instruction](#), [SAMPLE/PRELOAD Instruction](#), and [BYPASS Instruction](#).
- JTAG IEEE1149.1 optional instructions, see [ID_CODE Instruction \(SJC IDCODE / DFT_TAP IDCODE\)](#), [HIGHZ Instruction](#), and [CLAMP Instruction](#).

- JTAG IEEE P1149.1 (standard JTAG) interface to off-chip test and development equipment for true IEEE 1149.1 compliance, used primarily for board-level implementation of boundary scan.
- Debug-related control and status, such as putting selected cores into reset and/or debug mode and the ability to monitor individual core status signals via JTAG.
- Provides means for accessing each OnCE/ICE TAP controller independently to control a target system (see [Modes of Operation](#)).
- ExtraDebug logic (see [ENABLE_ExtraDebug Instruction](#)).
- The maximum clock speed of the SJC is one-eighth of the lowest frequency of the accessed OnCE/ICE. For example in normal operation (no core in low-power mode), this frequency is one-eighth of the SDMA frequency if this core is present in the TDI-TDO chain (serially connected with other cores or standalone). The user must also consider the 25 MHz frequency limitation on the CE bus.
- Core compliant modes to support standalone core debuggers (see [Modes of Operation](#)).
- Multi-cores daisy chained mode (default one) to support multi-core debuggers (see [Modes of Operation](#)).

Detailed information about the SJC is provided in the Security Reference Manual. Contact your NXP representative for information about obtaining this document.

4.14.1.2 Modes of Operation

The SJC modes are controlled through both the TAP select register (SJC_TSR) and the MOD input port.

The MOD port (typically connected to pad of the same name) selects between two possible topologies of TAP connections while using SJC functions, as seen at SoC level:

- Negating it (this should be the default state) selects all the TAPs (SJC, SDMA, DAP and Arm/ETM) to be connected in the TDI-TDO chain, which is referred to as "daisy chain" mode, throughout this chapter.
- Asserting it only selects the SJC TAP to be connected in the TDI-TDO chain.

SJC features are enabled by configuring the SJC input pin: MOD. Refer to the following table for MOD settings details:

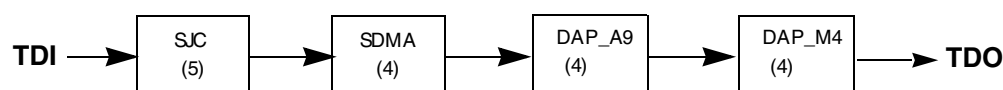
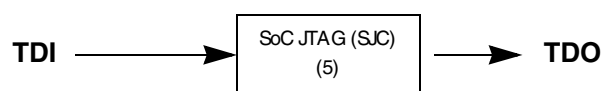
Table 4-30. SJC Modes

MOD	Name	Description
0	Daisy chain ALL	For common SW debug (High speed and production)
1	SJC only	SJC compliant mode

NOTE

IEEE1149.1 standard features are enabled by the following IO configurations: (BOOT_MODE0, BOOT_MODE1, GPIO1_IO00, GPIO1_IO01, GPIO1_IO02, GPIO1_IO03, TEST_MODE) = (0000011)

The following figure shows the SJC mode selection flow. The numbers shown in parenthesis below each block name indicates the TAP's IR length.

MOD = 0**MOD = 1**

(number in brackets lists IR length of given TAP)

Figure 4-30. SJC Mode Selection Using MOD Pin Sampling

The Connect SDMA bit inside TAP select register controls the SDMA TAP bypass.

- When negated (should be the default state), the SDMA TAP is bypassed with a single D-FF (Flip-flop) during Shift-Dr path
- When asserted SDMA TAP is connected inside the chain
- When taking the SDMA into bypass or out of bypass (by writing to tapsel reg), additional cycle with TMS '0' should be given

The TAP selection block (TSB) provides a simple method of integrating various pieces of IP that have embedded TAPs.

- Provides a way to connect up multiple TAPs within a single SoC
- Follow the state of SJC TAP, and when the Test-Logic-Reset (TLR) state is reached, reset all TAPs

The figure below shows the TAP Selection Block and SOC TAP Chain Scheme.

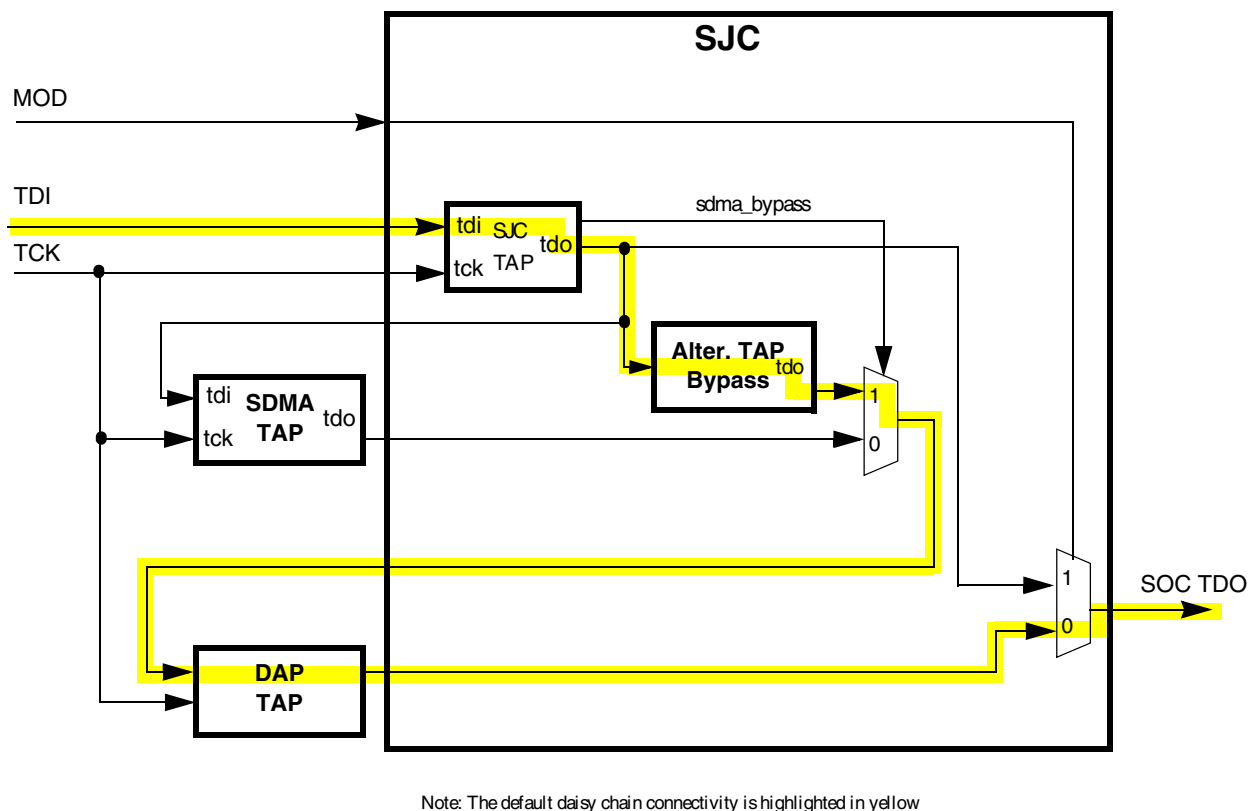


Figure 4-31. TAP Selection Block and SoC TAP Chain Scheme

NOTE

It is the responsibility of the user to ensure that in any configuration of the TAP controllers chosen, all of the TAPs in the chain comply with the demands of TCK clock frequency as well as the required ratio between TCK clock frequency and that of the core's to which the TAP refers.

4.14.2 TAP Selection Block (TSB)

As described in [Modes of Operation](#), the SJC can access cores in different modes selected through a TSB.

4.14.2.1 Select Mode Using Software

Conceptually, the SJC_TSR is a data register which is accessed through Access TSR IR instruction of SJC TAP.

The following figure shows the process of using reserved IR to access the SJC_TSR.

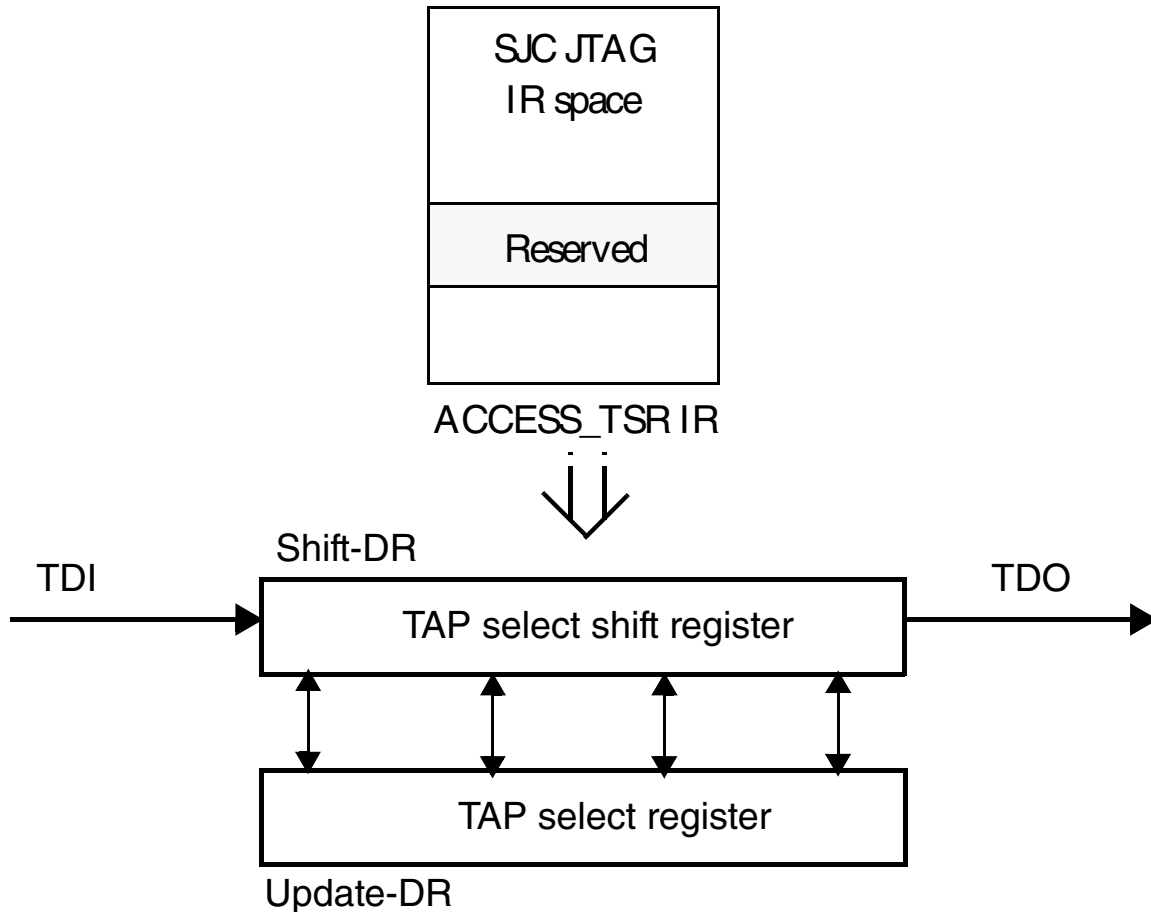


Figure 4-32. Using Reserved IR to Access the TAP Select Register (SJC_TSR)

The SJC_TSR can only be changed during the update-DR state of the TSB JTAG state machine. This is necessary to prevent a TAP that is being selected from losing synchronization with the TSB state machine when the TSB state machine returns to run-test-idle. Therefore, an associated shift register for the SJC_TSR is loaded into the SJC_TSR during the update-DR state (see the figure above). The shift register must also capture the state of the SJC_TSR when in the Capture-DR state for visibility of the contents of the SJC_TSR. See [TAP Select Instruction](#) , for more information.

4.14.3 Boundary Scan Register (BSR)

The Boundary Scan Register (BSR) in the JTAG implementation contains bits for all device signal and clock pins and associated control signals.

All SoC bidirectional pins have a single register bit in the boundary scan register for pin data, and are controlled by an associated control bit in the boundary scan register.

4.14.4 SJC Instruction Register (SJIR)

The SJC Instruction register is provided in the following table. The SJC Instruction register is 5 bits wide.

Table 4-31. SJC Instruction Register (SJIR)

Code					SJC IR
B4	B3	B2	B1	B0	
0	0	0	0	0	SJC IDCODE
0	0	0	0	1	Reserved
0	0	0	1	0	
0	0	0	1	1	Reserved
0	0	1	0	0	ENABLE_ExtraDebug
0	0	1	0	1	ENTER_DEBUG (secured)
0	0	1	1	0	Reserved
0	0	1	1	1	TAP select
0	1	0	0	0	
0	1	0	0	1	
0	1	0	1	0	Reserved
0	1	0	1	1	Reserved
0	1	1	0	0	Security Output challenge
0	1	1	0	1	Security Enter response
-	-	-	-	-	Reserved
1	1	1	1	1	Reserved

The instruction register is reset to 0b00000 in the test-logic-reset controller state which is equivalent to the SJC IDCODE instruction.

During the capture-IR controller state, the parallel inputs to the instruction register are loaded with the code 01 in the least significant bits as required by the standard; the most significant bits are loaded with the values 00, leading to a capture value of 0b000001.

4.14.4.1 DFT_TAP JTAG Instruction Register (DJIR)

The DFT_TAP JTAG Instruction register is provided in the following table. The DFT_TAP JTAG Instruction register is 4 bits wide.

Table 4-32. DFT_TAP JTAG Instruction Register (DJIR)

Code				DFT_TAP IR
B3	B2	B1	B0	
1	1	1	1	BYPASS
0	0	1	0	EXTEST
0	0	1	1	SAMPLE/PRELOAD
0	1	1	0	CLAMP
0	1	0	0	DFT_TAP IDCODE
0	1	1	1	HIGHZ

The DFT_TAP JTAG instruction register is reset to 0b0000, which is reserved.

4.14.4.2 ID_CODE Instruction (SJC IDCODE / DFT_TAP IDCODE)

Selects the ID register, and the system logic controls the I/O pins. This instruction is provided as a public instruction to allow the manufacturer, part number and version of a component to be determined through the TAP.

The table below shows the ID register configuration.

Table 4-33. ID Configuration Register (SJC IDCODE / DFT_TAP IDCODE)

IDCODE				ID Configuration Register													
	BIT 31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	BIT 16	
	Version Information[3:0]				Part Number (Bits 27-16)												
TYPE	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	
RESET	0	0	0	0	1	0	0	0	1	0	0	0	1	1	1	0	
Note:																	
	BIT 15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	BIT 0	
	Part Number (Bits 15-12)				Manufacturer Identity												1
TYPE	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	
RESET	0	0	1	0	0	0	0	0	0	0	0	1	1	1	0	1	
Note:																	

Table 4-34. SJC ID Configuration Register Description (SJC IDCODE)

Field	Description
31-28 Version Information	IC/SoC Version information number. Initial value: '0000' This number is subject to changes, for new IC/SoC (System On A Chip) revision releases.
27-12 Part Number	Customer Part Number The 16-bit Part Number value is unique for every NXP SoC / IC. See System Debug chapter for exact register value for a specific SoC.
11-1 Manufacturer Identity	Manufacturer Identity NXP Manufacturer Identity code. Bits [11:1] - 00000001110
0	Tied to logic 1.

Table 4-35. DFT_TAP ID Configuration Register Description (DFT_TAP IDCODE)

Field	Description
31-28 Version Information	IC/SoC Version information number. Initial value: '0001' This number is subject to changes, for new IC/SoC (System On A Chip) revision releases.
27-12 Part Number	Customer Part Number The 16-bit Part Number value is unique for every NXP SoC / IC. Value: 1100111110000000 See System Debug chapter for exact register value for a specific SoC.
11-1 Manufacturer Identity	Manufacturer Identity NXP Manufacturer Identity code. Value: 01010101001
0	Tied to logic 1.

One application of the ID register is to distinguish the manufacturer(s) of components on a board when multiple sourcing is used. As more components emerge which conform to the IEEE 1149.1 standard, it is desirable to allow for a system diagnostic controller unit to blindly interrogate a board design to determine the type of each component in each location. This information is also available for factory process monitoring and for failure mode analysis of assembled boards.

Once the IDCODE instruction is decoded, it selects the ID register which is a 32 Bit data register. Because the bypass register loads a logic 0 at the start of a scan cycle, whereas the ID register loads a logic 1 into its least significant bit, examination of the first bit of data shifted out of a component during a test data scan sequence immediate following exit from Test-Logic-Reset controller state shows whether such a register is included in

the design. When the IDCODE instruction is selected, the operation of the test logic has no effect on the operation of the on-chip system logic as required by the IEEE 1149.1 standard.

4.14.4.3 SAMPLE/PRELOAD Instruction

Selects the boundary scan register and the system logic controls the I/O pins.

The SAMPLE/PRELOAD instruction provides two separate functions:

- First, it provides a means to obtain a snapshot of system data and control signals. The snapshot occurs on the rising edge of TCK in the capture-DR controller state. The data can be observed by shifting it transparently through the boundary scan register.
- The second function of SAMPLE/PRELOAD is to initialize the boundary scan register output cells prior to selection of EXTEST. This initialization ensures that known data appears on the outputs when entering the EXTEST instruction.

NOTE

Because there is no internal synchronization between the JTAG clock (TCK) and the system clock (CLK), the user must provide some form of external synchronization to achieve meaningful results.

For more details on the function and use of SAMPLE/PRELOAD, refer to the appropriate IEEE 1149.1 document.

4.14.4.4 EXTEST Instruction

Selects the boundary scan register, and the 1149.1 test logic has control of the I/O pins.

By using the TAP controller, the register is capable of:

- Scanning user-defined values into the output buffers,
- Capturing values presented to input pins
- Controlling the direction of bidirectional pins,
- Controlling the output drive of tri-statable output pins.

For more details on the function and use of EXTEST, refer to the appropriate IEEE 1149.1 document.

The EXTEST instruction also asserts internal reset for the cores (through CCM, refer to [Figure 4-35](#)) to force a predictable internal state while performing external boundary scan operations.

4.14.4.5 HIGHZ Instruction

All output drivers, including the two-state drivers, are turned off (that is, high impedance). The instruction selects the bypass register.

In this mode, all internal pullup resistors on all the pins (except for the TMS, TDI, TCK, TRSTB pins) are disabled. This disabling functionality is not built into SJC, but should be implemented by some logic in the SOC/IO Pads.

For more details on the function and use of HIGHZ, refer to the IEEE 1149.1 document.

The HIGHZ instruction also asserts internal reset for the cores (through CCM, refer to [Figure 4-35](#)) to force a predictable internal state while performing external boundary scan operations.

4.14.4.6 BYPASS Instruction

Selects the single Bit bypass register and the system logic controls the I/O pins.

This creates a shift-register path from TDI to the bypass register and, finally, to TDO, circumventing the boundary scan register. This instruction is used to enhance test efficiency when a component other than the SoC Core based device becomes the device under test.

When the bypass register is selected by the current instruction, the shift-register stage is set to a logic zero on the rising edge of TCK in the capture-DR controller state. Therefore, the first bit to be shifted out after selecting the bypass register is always a logic zero.

For more details on the function and use of BYPASS, refer to the appropriate IEEE 1149.1 document.

4.14.4.7 CLAMP Instruction

The CLAMP instruction is established initially with the SAMPLE and PRELOAD instructions.

It drives preset values onto the outputs of devices and then, unlike the SAMPLE and PRELOAD instructions, it selects the bypass register between TDI and TDO. CLAMP can be used to set values on the outputs of certain devices to avoid bus contention problems.

4.14.4.8 ENABLE_ExtraDebug Instruction

The TDI and TDO pins are connected directly to the ExtraDebug registers, the SJC TAP controller remaining connected to TDI and TMS.

The ExtraDebug shift register consists of 38 bits (maximum) comprising a 32-bits data field (maximum length, see [Accessing ExtraDebug Registers](#)), a 5 bits address field and read/write bit. On a register read, the data field does not need to be filled in. The particular ExtraDebug register connected between TDI and TDO at a given time is selected by the ExtraDebug controller depending on the ExtraDebug Address being currently decoded. All communication with the ExtraDebug controller is done through the Select-DR-Scan path of the JTAG TAP Controller.

4.14.4.9 ENTER_DEBUG instruction

The ENTER_DEBUG instruction is used to generate a debug request event to SDMA and the Arm Core Platform simultaneously (practically, inherited minimal skew is expected, due to difference in event signal propagation in the different modules).

The TDI and TDO are connected to the Instruction Register (IR). After the acknowledgment of the Debug Mode is received (can be checked by reading the Core Status Register part of the ExtraDebug logic), the user can perform system debug functions on the cores.

NOTE

The ENTER_DEBUG event issue to the cores, can be masked, by bits in DCR register.

NOTE

It is user's responsibility to shift-in another IR value (like IDCODE) before trying to bring the cores out of debug mode, as the debug request signals to the cores remains asserted as long as ENTER_DEBUG IR is in place.

NOTE

The user need to check that cores are in debug mode (watching debug acknowledge signal) before leaving ENTER_DEBUG instruction, otherwise debug request might not take affect.

4.14.4.10 TAP Select Instruction

By means of TAP select instruction a user can access TAP select register and by controlling its only bit SDMA Bypass, control whether SDMA TAP is bypassed or not.

Table 4-36. TAP Select Register (TSR)

	TAP Select Register
	BIT 0
	Connect SDMA
TYPE	rw
RESET	0
Note:	

Table 4-37. TAP Select Register Description

Field	Description
0	Connect SDMA
SDMA Bypass	Control whether SDMA TAP is bypassed or not: • 0 - SDMA TAP is bypassed by the alternate TAP inside SJC (emulating 4-bit IR and 1-bit bypass path). • 1 - SDMA TAP is connected to the TDI-TDO chain. NOTE: Additional cycle with TMS '0' should be inserted, after writing to this register, to allow the SDMA tap be sync before SDMA get into / out of bypass.

4.14.5 Security

JTAG manipulation is one of the known hackers' ways of executing unauthorized program code, getting control over the OS and run code in privileged modes.

The SJC provides a debug access to several H/W blocks including the Arm processor and the system bus. This allows for program control and manipulation as well as visibility into system peripherals and memory. The ETM and NEXUS interfaces allow bus transactions to be traced. Together these tools provide the hacker all the access needed to completely comprise the system. Means must be provided to block any malicious JTAG access.

The SJC provides a way of regulating the JTAG access.

The following are the different JTAG security modes:

- **Mode #1: No Debug-Maximum Security.** All security sensitive JTAG features are permanently blocked.

- **Mode #2: Secure JTAG**-High security. JTAG use is regulated by secret key based authentication mechanism.
- **Mode #3: JTAG Enabled**-Low security. JTAG always enabled.

The JTAG security modes are configured using eFUSES which can be burned after packaging by applying electrical signals. The fuse burning is an irreversible process, once a fuse is burned (e-fuse or laser fuse) it is impossible to change the fuse back to the un-burned state.

4.14.5.1 JTAG Security Modes

JTAG can be in one of JTAG security modes which is selected by setting the SJC eFuse configuration. The physical location of the fuses is not in the SJC.

4.14.5.1.1 Mode 1: No Debug - Maximum Security

No Debug JTAG security mode provides the highest security level.

In this mode, all JTAG features are disabled except for:

- ScanBoundary Scan
- PLL bypass- Bypass Arm or/and USB PLL.
- Visibility of the following status bits: power mode - normal, standby, stop, shutdown, and so on

These features do not reduce the security level of the product, and they allow to perform important tests and board connectivity checks.

4.14.5.1.2 Mode 2: Secure JTAG - High Security

The Secure JTAG mode limits the JTAG access by using challenge/response based authentication mechanism. Any access to JTAG port is being checked. Only authorized debug devices (that is, devices having the right response) can access the JTAG, unauthorized JTAG access attempts are denied.

The intent of this mode is to allow return field testing. When a secured JTAG device is being returned for debugging, this mode allows authorized re-activation of the JTAG.

4.14.5.1.2.1 Challenge/Response Mechanism in System JTAG Mode

When SJC is in System JTAG mode the authentication process is as follows:

1. Shift Output Challenge instruction to IR.

2. Passing through Capture-DR state of the SJC and by performing Shift-DR operations Challenge code can be accessed from TDO.
3. Shift Enter Response instruction to IR. By performing Shift-DR, operations enter Response code value through TDI. As Update-DR state is entered, Response code is compared with the correct one.

In Fixed challenge-response pair mode, each part has its individual challenge - response pair which is determined at manufacturing time, and does not change later on. The SJC compares the user's response to the expected response.

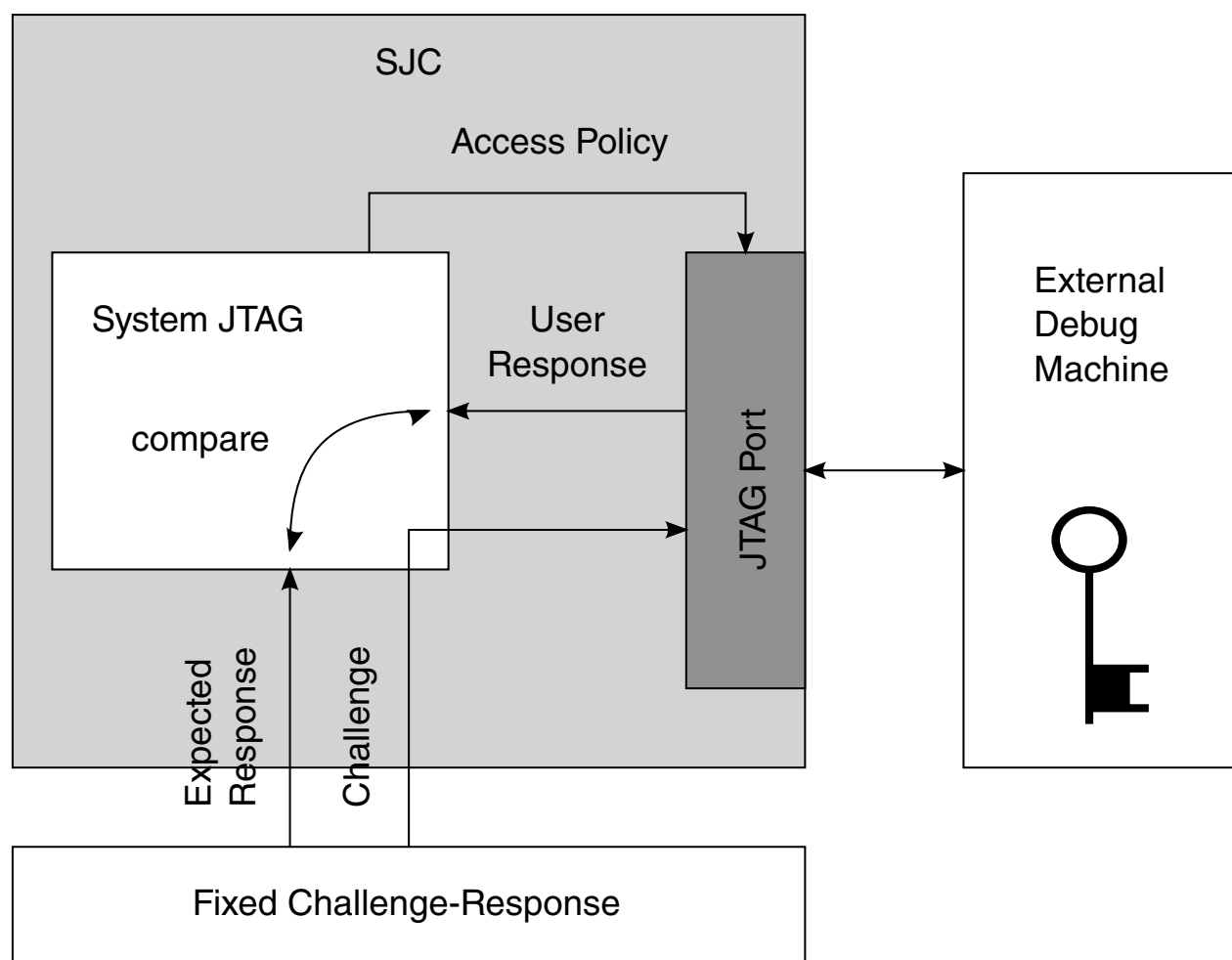


Figure 4-33. Mode #2 - Secure JTAG with Fixed Challenge-response Pair

4.14.5.1.3 Mode 3: JTAG Enabled - Low Security

In the JTAG Enabled JTAG security mode, all JTAG features are enabled.

4.14.5.2 Software Enabled JTAG

To increase the flexibility of the SJC, an option to enable the JTAG via software is added and is available only in Secure JTAG mode. By writing '1' to HAB_JDE (HAB JTAG DEBUG ENABLE) bit in the eFuse controller module, the JTAG is opened, regardless of its security mode. It is the responsibility of software to assert or negate this bit.

Additionally, a corresponding lock bit is available (in the eFuse control module) to ensure that only trusted software is able to set the JDE bit. When the LOCK bit is set, no future change of JDE is possible, until the next POR (power-on-reset) cycle.

The platform initialization software should set the LOCK bit for JDE bit before transferring control to the application code.

The S/W JTAG enable allows JTAG enabling without activating the challenge-Response mechanism (which requires JTAG access tool enhancement or special H/W). The JTAG S/W enable does not allow debug in case of boot or memory fault as it requires reset before entering debug.

This feature can be permanently blocked by burning the dedicated eFuse.

NOTE

The S/W enabled JTAG feature reduces the overall security level of the system as it relies on S/W protections. If this feature is not required, it is strongly recommended to burn the JTAG_HEO eFuse which disables this feature.

4.14.5.3 Kill Trace

The kill trace signal disables any output of the ETM block. The ETM can be accessed either via JTAG port and/or by direct software code. Blocking the JTAG port also yields assertion of the kill trace signal. This resulted in blocking of trace port. The intention of this action is to block any attempt to break into the system via software manipulation of the debug modules. The kill trace, when active, prevents trace output even in case where it can be activated via chip pin.

The kill trace feature needs to be activated by burning a dedicated eFuse. If the fuse is left intact, kill trace is never activated as seen in [Figure 4-34](#).

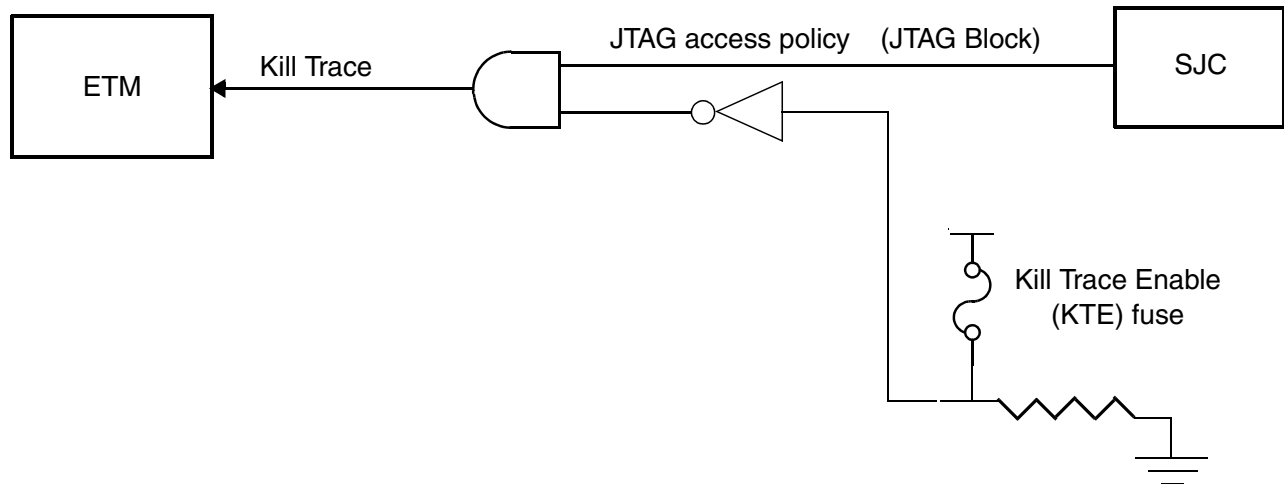


Figure 4-34. Kill Trace eFuse

The kill trace is asserted when "kill trace enable" fuse is burned and "ipt_secur_block" signal in SJC is asserted, which happens when at least one of the following is true:

- Mode #2 (Secure JTAG) and no code has been entered
- Mode #2 (Secure JTAG) with burned Bypass and Re-enable fuses
- Mode #2 (Secure JTAG) with incorrect response entered
- Mode #1 (No debug)
- TRST_B signal is active
- POR has not ever been asserted

4.14.5.4 SJC Disable Fuse

In addition to the different JTAG security modes that are implemented internally in the System JTAG Controller (SJC), there is an option to disable the SJC functionality by eFuse configuration. This creates additional JTAG mode that is, JTAG Disabled with highest level of JTAG protection. In this mode all JTAG features are disabled.

Specifically, the following debug features are disabled in addition to the features that were already disabled in No Debug JTAG mode:

- Boundary scan register (SJC_BSR)
- Non-Secure JTAG control registers (PLL configuration, Deterministic Reset, PLL bypass)
- Non-Secure JTAG status registers (Core status)
- Chip Identification Code (IDCODE)

4.14.6 Functional Description

This section provides a complete functional description of the block.

4.14.6.1 Static Core Debug

The SJC JTAG TAP controller is fully compatible with the IEEE 1149.1a-2001 Standard Test Access Port and Boundary Scan Architecture specifications.

The Arm platform has an integrated JTAG interface and a TAP controller to manage its own ICE. Also it can access an embedded trace ETM interface, see Arm core and ETM Technical reference guide for more information.

The SDMA has a TAP controller to manage its own OnCE, see SDMA OnCE specifications for more details.

The OnCE and ICE provide a mean of interacting with the cores and their peripherals non-intrusively so that a user may examine registers, memories to facilitate hardware and software development. Refer to [TAP Selection Block \(TSB\)](#), for more information.

4.14.6.2 Reset Mechanism

The following figure shows the SJC reset logic

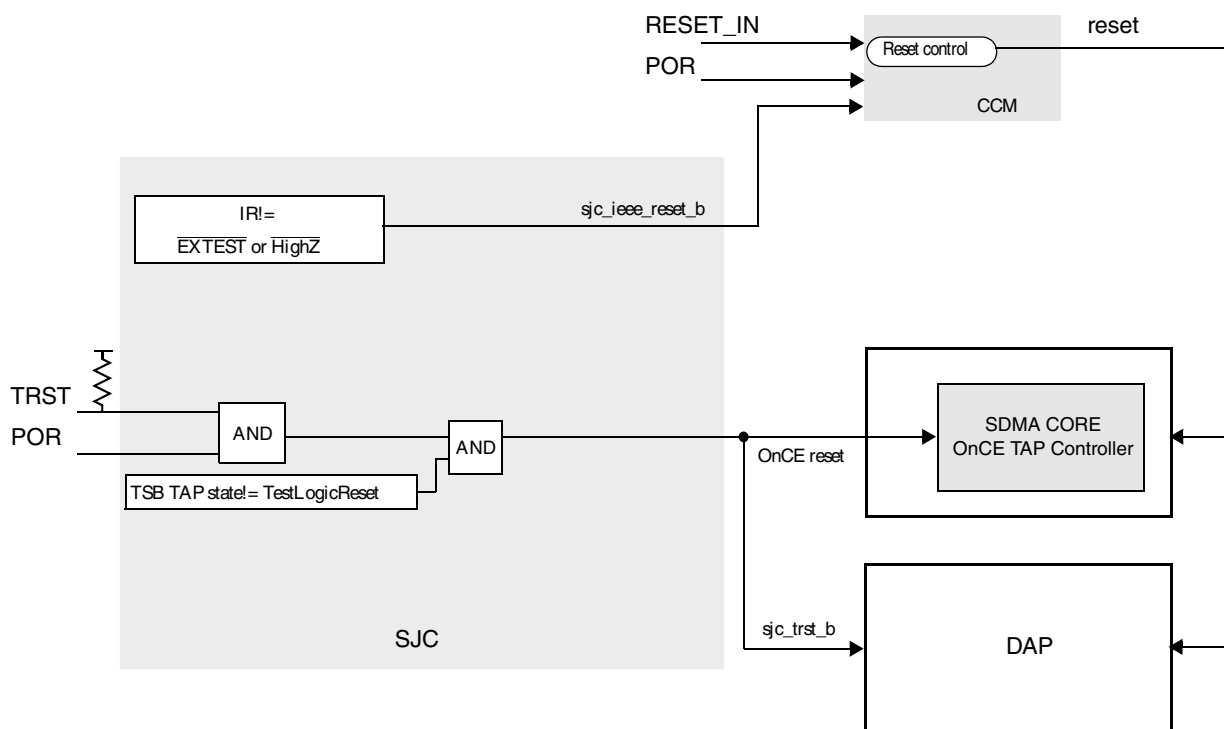


Figure 4-35. SJC Reset Logic

NOTE

- Asserting TRSTB in any scan mode resets the TCR losing the testmode configuration and selects default TAP.
- SJC generates an IEEE reset signal to the CCM when in one of the IEEE modes HIGHZ or EXTEST. This signal generates a system reset to the cores until exit from one of these modes.
- The TSB generates Once/ICE reset (either TRSTB if implemented or other) when its TAP state reaches Test-Logic-Reset (meaning that TAP accessed is also reaching Test-Logic-Reset).

4.14.7 Initialization/Application Information

The control afforded by the output enable signals using the boundary scan register and the EXTEST instruction requires a compatible circuit-board test environment to avoid device-destructive configurations. The user must avoid situations in which the SJC output drivers are enabled into actively driven networks.

There are two constraints related to the JTAG interface:

- Ensure that the JTAG test logic is kept transparent to the system logic by forcing TAP into the Test-Logic-Reset controller state. During power-up, SJC's internal TRSTB is asserted as IC's POR_B is asserted which forces the TAP controller into this state. After that, if TMS either remains unconnected or is connected to VCC, then the TAP controller cannot leave the Test-Logic-Reset state, regardless of the state of TCK.
- DE_B is an IO pin with pullup and care must be taken of the direction when driving this signal.

4.14.8 SJC Memory Map/Register Definition

In addition to the standard accessible JTAG registers (per IEEE1149.1 standard) listed in [SJC Instruction Register \(SJIR\)](#) , the chip contains the following registers accessed using the ExtraDebug mechanism, controlled via "ENABLE_ExtraDebug" IR instruction.

NOTE

SJC registers are only accessible by JTAG interface. They are not memory mapped to processor address space, so the absolute addresses provided by default in the SJC memory map are not valid.

This section assumes the JTAG controller is accessed in standalone mode or daisy chained (defined by TAP Selection Block) using the appropriate TSB configuration.

See "System Debug" chapter for more details about the general purpose register descriptions that are unique to this chip.

SJC memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
0	General Purpose Unsecured Status Register 1 (SJC_GPUSR1)	32	R	0000_0000h	4.14.8.1/323
1	General Purpose Unsecured Status Register 2 (SJC_GPUSR2)	32	R	0000_0000h	4.14.8.2/325
2	General Purpose Unsecured Status Register 3 (SJC_GPUSR3)	32	R	0000_0000h	4.14.8.3/325
3	General Purpose Secured Status Register (SJC_GPSSR)	32	R	0000_0000h	4.14.8.4/326
4	Debug Control Register (SJC_DCR)	32	R/W	0000_0000h	4.14.8.5/327

Table continues on the next page...

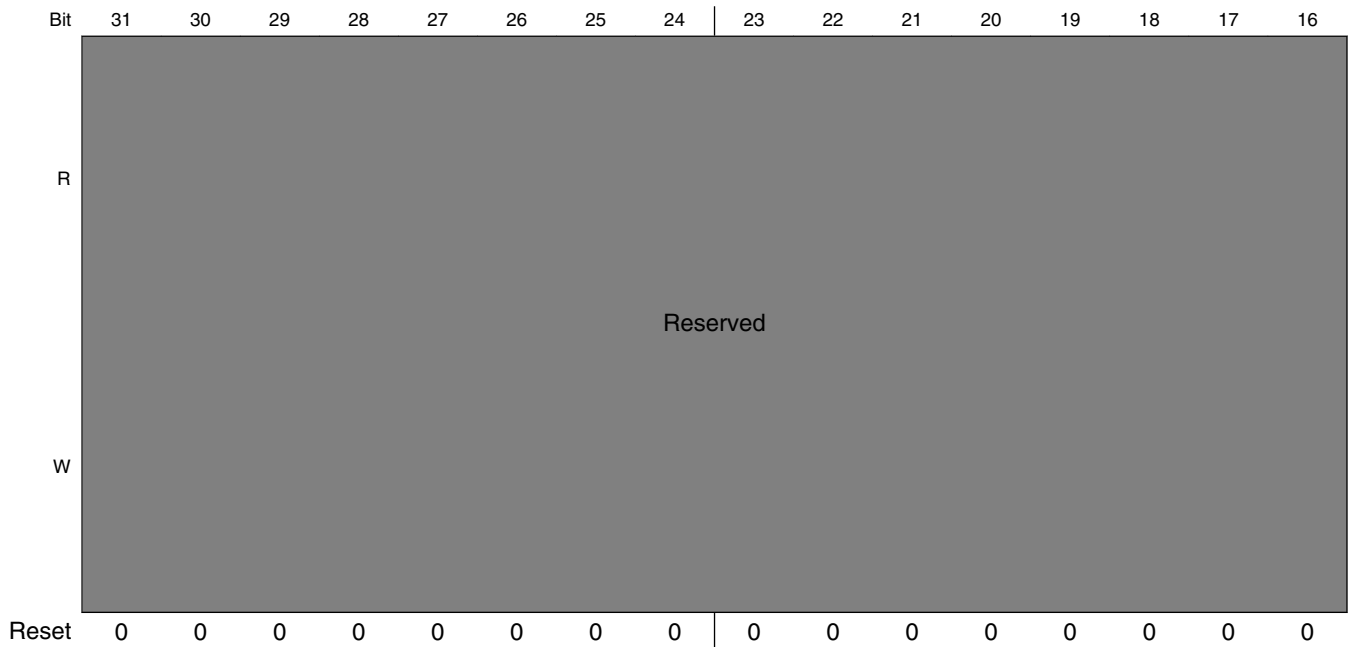
SJC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
5	Security Status Register (SJC_SSR)	32	R	See section	4.14.8.6/329
7	General Purpose Clocks Control Register (SJC_GPCCR)	32	R/W	0000_0000h	4.14.8.7/332

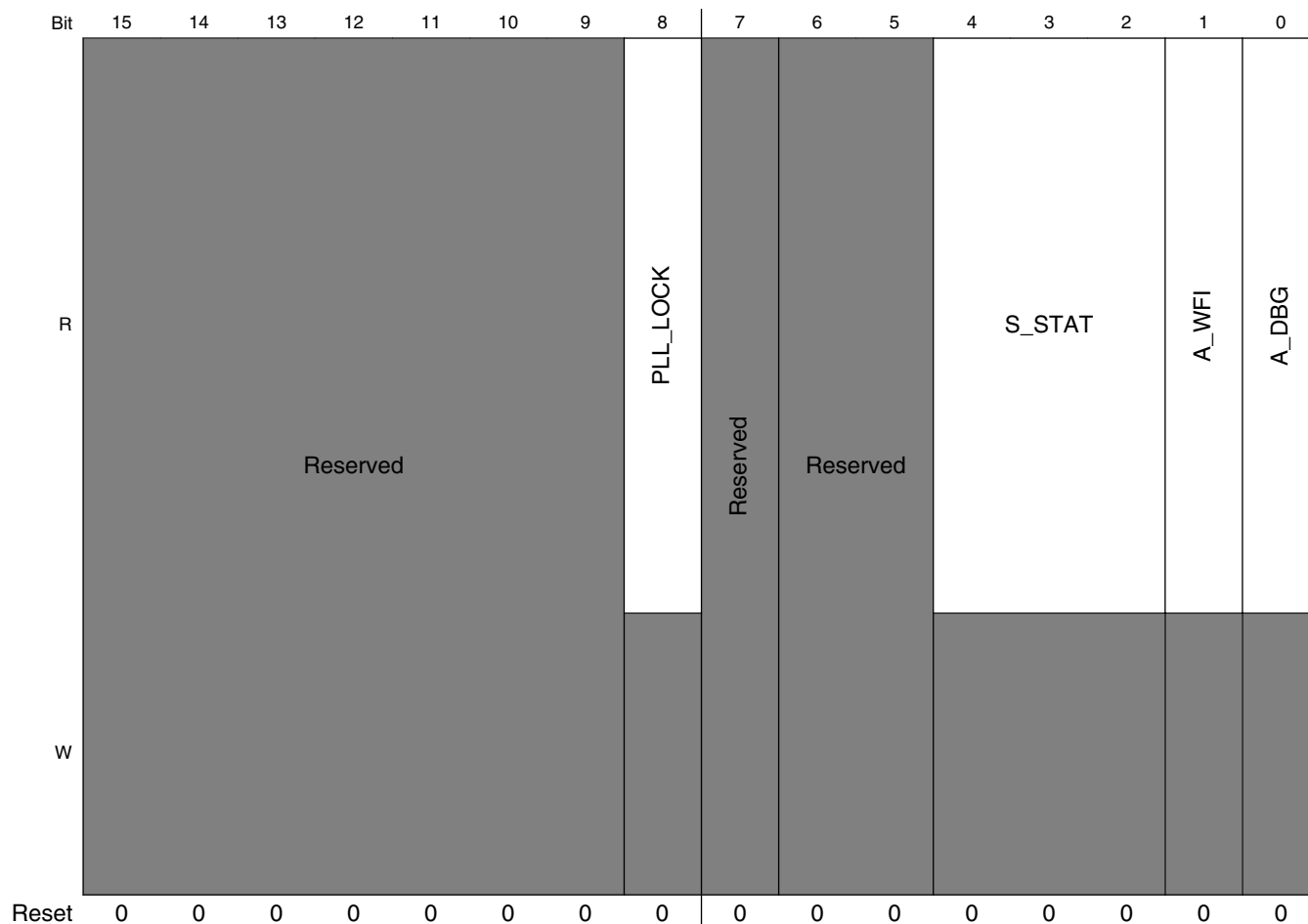
4.14.8.1 General Purpose Unsecured Status Register 1 (SJC_GPUSR1)

The General Purpose Unsecured Status Register 1 is a read only register used to check the status of the different Cores and of the PLL. The rest of its bits are for general purpose use.

Address: 0h base + 0h offset = 0h



System JTAG Controller (SJC)



SJC_GPUSR1 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved.
8 PLL_LOCK	PLL_LOCK A Combined PLL-Lock flag indicator, for all the PLL's.
7 -	This field is reserved. Reserved
6–5 -	This field is reserved. Reserved.
4–2 S_STAT	3 LSBits of SDMA core statusH.
1 A_WFI	Arm core wait-for interrupt bit Bit 1 is the Arm core standbywfi (stand by wait-for interrupt). When this bit is HIGH, Arm core is in wait for interrupt mode.
0 A_DBG	Arm core debug status bit Bit 0 is the Arm core DBGACK (debug acknowledge) DBGACK can be overwritten in the Arm core DCR to force a particular DBGACK value. Consequently interpretation of the DBGACK value is highly dependent on the debug sequence. When this bit is HIGH, Arm core is in debug.

4.14.8.2 General Purpose Unsecured Status Register 2 (SJC_GPUSR2)

Address: 0h base + 1h offset = 1h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved																STBYWFE				S_STAT				STBYWFI							
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SJC_GPUSR2 field descriptions

Field	Description
31–12 -	This field is reserved. Reserved
11–8 STBYWFE	STBYWFE[3:0] Reflecting the "Standby Wait For Event" signals of all cores.
7–4 S_STAT	S_STAT[3:0] SDMA debug status bits: debug_core_state[3:0]
STBYWFI	STBYWFI[3:0] These bits provide status of "Standby Wait-For-Interrupt" state of all Arm cores.

4.14.8.3 General Purpose Unsecured Status Register 3 (SJC_GPUSR3)

Address: 0h base + 2h offset = 2h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved													SYS_WAIT	IPG_STOP	IPG_WAIT
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SJC_GPUSR3 field descriptions

Field	Description
31–3 -	This field is reserved. Reserved

Table continues on the next page...

SJC_GPUSR3 field descriptions (continued)

Field	Description
2 SYS_WAIT	System In wait Indication on System in wait mode (from CCM).
1 IPG_STOP	IPG_STOP CCM's "ipg_stop" signal indication
0 IPG_WAIT	IPG_WAIT CCM's "ipg_wait" signal indication

4.14.8.4 General Purpose Secured Status Register (SJC_GPSSR)

The General Purpose Secured Status Register is a read-only register used to check the status of the different critical information in the SoC. This register cannot be accessed in secure modes.

Address: 0h base + 3h offset = 3h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	GPSSR																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SJC_GPSSR field descriptions

Field	Description
GPSSR	General Purpose Secured Status Register Register is used for testing and debug.

4.14.8.5 Debug Control Register (SJC_DCR)

This register is used to control propagation of debug request from DE_B pad to the cores and debug signals from internal logic to the DE_B pad.

Address: 0h base + 4h offset = 4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								DIRECT_ARM_REQ_EN	DIRECT_SDMA_REQ_EN	Reserved	DEBUG_OBS	Reserved	DE_TO_SDMA	DE_TO_ARM	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SJC_DCR field descriptions

Field	Description
31–7 -	This field is reserved. Reserved
6 DIRECT_ARM_REQ_EN	Pass Debug Enable event from DE_B pin to Arm platform debug request signal(s). This bit controls the propagation of debug request DE_B to the Arm platform. 0 Disable propagation of system debug to (DE_B pin) to Arm platform. 1 Enable propagation of system debug to (DE_B pin) to Arm platform.
5 DIRECT_SDMA_REQ_EN	Debug enable of the sdma debug request This bit controls the propagation of debug request DE_B to the sdma. 0 Disable propagation of system debug to (DE_B pin) to sdma. 1 Enable propagation of system debug to (DE_B pin) to sdma.
4 -	This field is reserved. Reserved

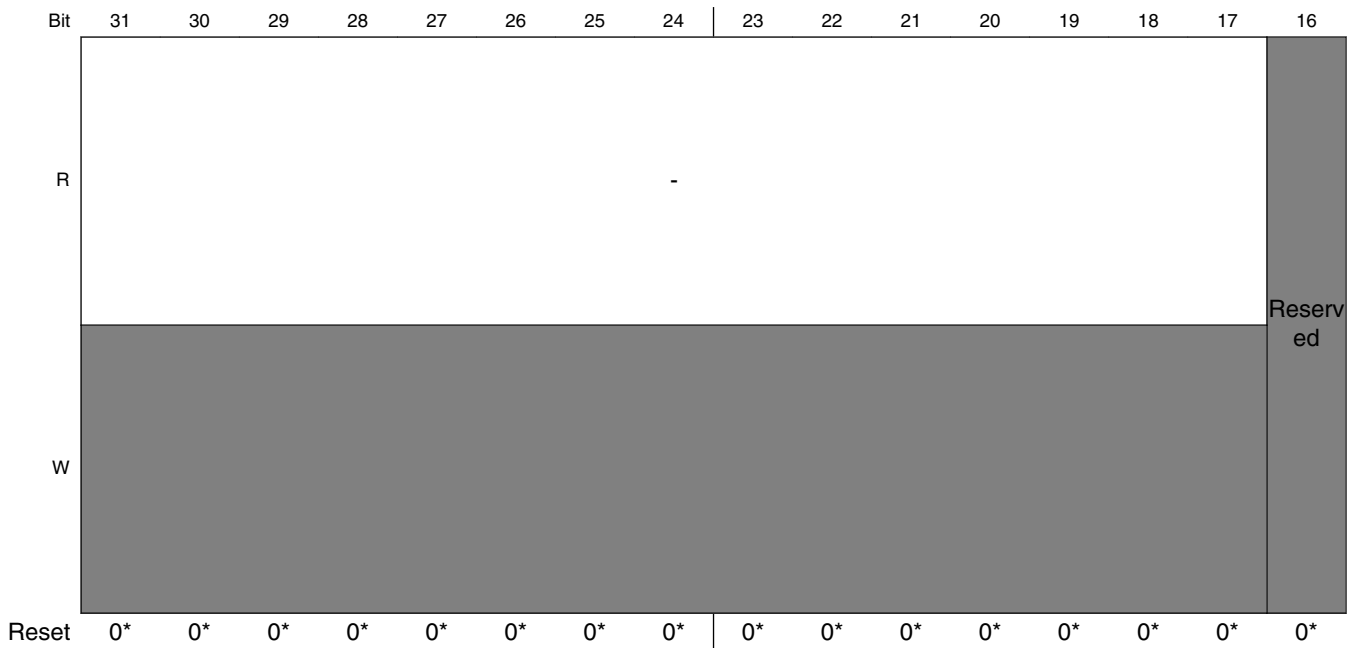
Table continues on the next page...

SJC_DCR field descriptions (continued)

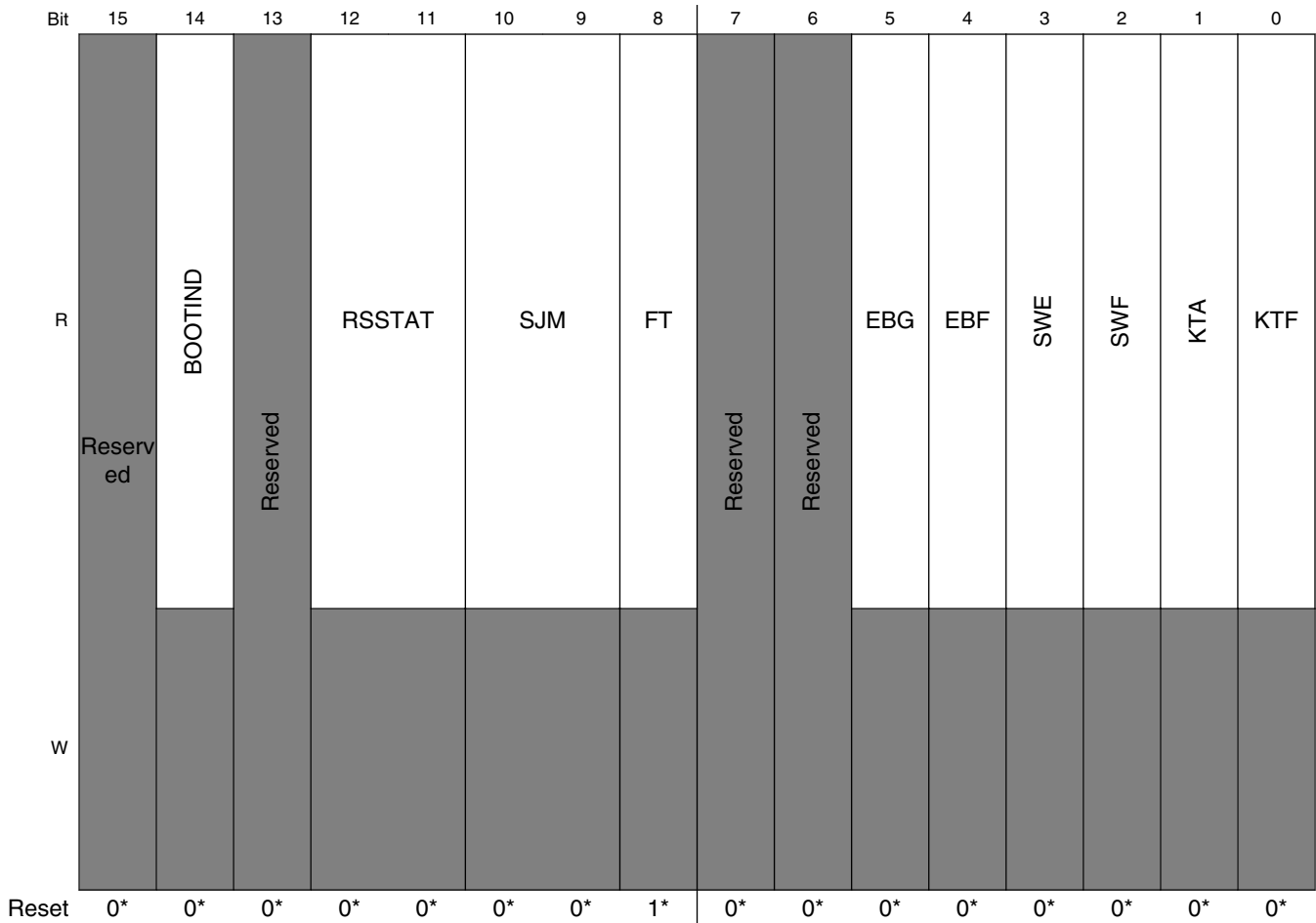
Field	Description
3 DEBUG_OBS	Debug observability This bit controls the propagation of the "system debug" input to SJC (driven by the ECT logic), to the DE_B pad. (This logic can be used to pass debug acknowledge event from ECT out to the PAD, for example). The SJC's "system_debug" input is tied to logic HIGH value, therefore, set of "debug_obs" bit, will result in unconditional assertion of DE_B pad. 0 Disable propagation of system debug to DE_B pin 1 Unconditional assertion of pad DE_B
2 -	This field is reserved. Reserved
1 DE_TO_SDMA	SDMA debug request input propagation This bit controls the propagation of debug request to SDMA, when the JTAG state machine is put in "ENTER_DEBUG" IR instruction.. 0 Disable propagation of debug request to SDMA 1 Enable propagation of debug request to SDMA
0 DE_TO_ARM	Arm platform debug request input propagation This bit controls the propagation of debug request to Arm platform ("dbgreq"), when the JTAG state machine is put in "ENTER_DEBUG" IR instruction. 0 Disable propagation of debug request to Arm platform 1 Enable propagation of debug request to Arm platform

4.14.8.6 Security Status Register (SJC_SSR)

Address: 0h base + 5h offset = 5h



System JTAG Controller (SJC)



- * Notes:
- The SJM reset value, reflects the JTAG security state, as defined by status of JTAG_SMODE[1:0] fuses. See the [SJM](#) bitfield description for details on valid values.

SJC_SSR field descriptions

Field	Description
31–17 -	Reserved.
16–15 -	This field is reserved. Reserved
14 BOOTIND	Boot Indication Inverted Internal Boot indication, i.e inverse of SRC: "src_int_boot" signal
13 -	This field is reserved. Reserved
12–11 RSSTAT	Response status Response status bits 00 Response wasn't entered 01 Response was entered but not verified

Table continues on the next page...

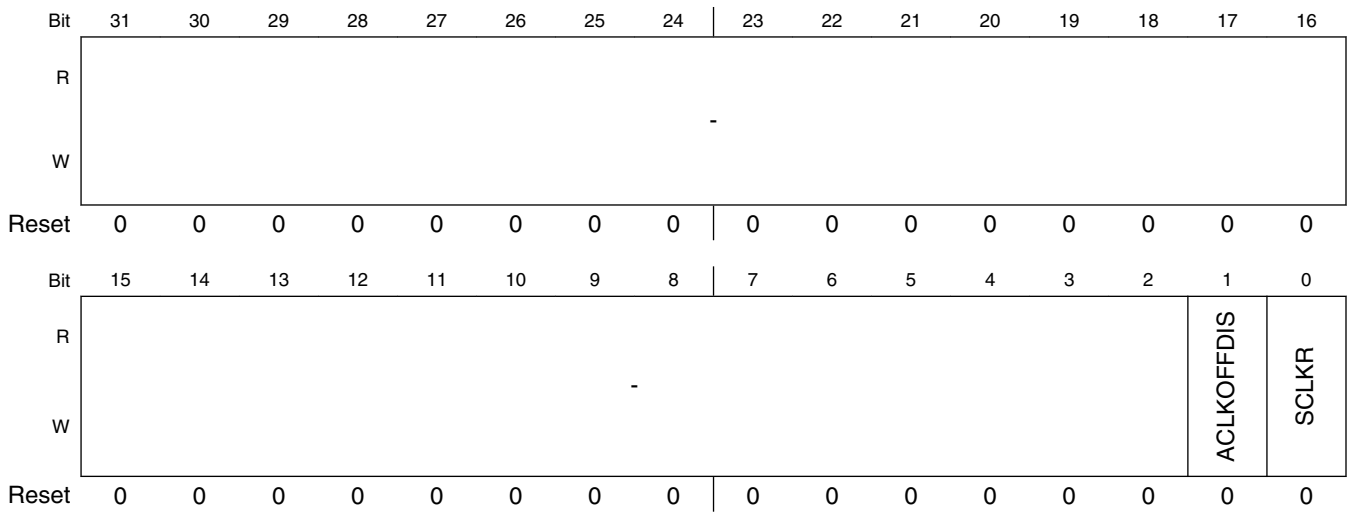
SJC_SSR field descriptions (continued)

Field	Description
	10 Response was entered and is incorrect 11 Response is correct
10–9 SJM	SJC Secure mode Secure JTAG mode, as set by external fuses. 00 No debug (#1) 01 Secure JTAG (#2) 10 Reserved 11 JTAG enabled (#3)
8 FT	Fuse type Fuse type bit - e-fuse or laser fuse 0 E-fuse technology 1 Laser fuse technology
7 -	This field is reserved. Reserved
6 -	This field is reserved. Reserved
5 EBG	External boot granted External boot enabled, requested and granted 1 granted 0 not granted
4 EBF	External Boot fuse Status of the external boot disable fuse 0 (intact) - external boot is allowed 1 (burned) - external boot is disabled
3 SWE	SW enable SW JTAG enable status 1 enabled 0 disabled
2 SWF	Software JTAG enable fuse Status of the no SW disable JTAG fuse 0 (intact) - SW enable possible 1 (intact) - no SW enable possible
1 KTA	Kill Trace is active 1 active 0 not active
0 KTF	Kill Trace Enable fuse value 0 (intact) - kill trace is never active 1 (burned) - kill trace functionality enabled

4.14.8.7 General Purpose Clocks Control Register (SJC_GPCCR)

This register is used to configure clock related modes in SOC, see System Configuration chapter for more information. Those bits are directly connected to JTAG outputs. Bit 0 of GPCCR controls SDMA clocks invocation. When out of reset, the SDMA is in sleep mode with no SDMA clock running. Unlike events, debug requests does not wake SDMA if it is in sleep mode. The debug request is recognized by the SDMA only when it exits sleep mode upon reception of an event. To be able to enter debug mode even if no event is triggered, the SDMA clock on bit needs to be set prior to sending the debug request (clear at reset).

Address: 0h base + 7h offset = 7h



SJC_GPCCR field descriptions

Field	Description
31–2 -	Reserved
1 ACLKOFFDIS	Disable/prevent Arm platform clock/power shutdown
0 SCLKR	SDMA Clock ON Register - This bit forces the clock on of the SDMA

Chapter 5

Clocks and Power Management

5.1 Clock Control Module (CCM)

5.1.1 Overview

Clock Control Module (CCM) manages the on-chip module clocks. CCM receives clocks from PLLs and oscillators and creates clocks for on-chip peripherals through a set of multiplexers, dividers and gates. When entering or exiting a low power mode, CCM automatically turns on and off PLLs and peripheral clocks.

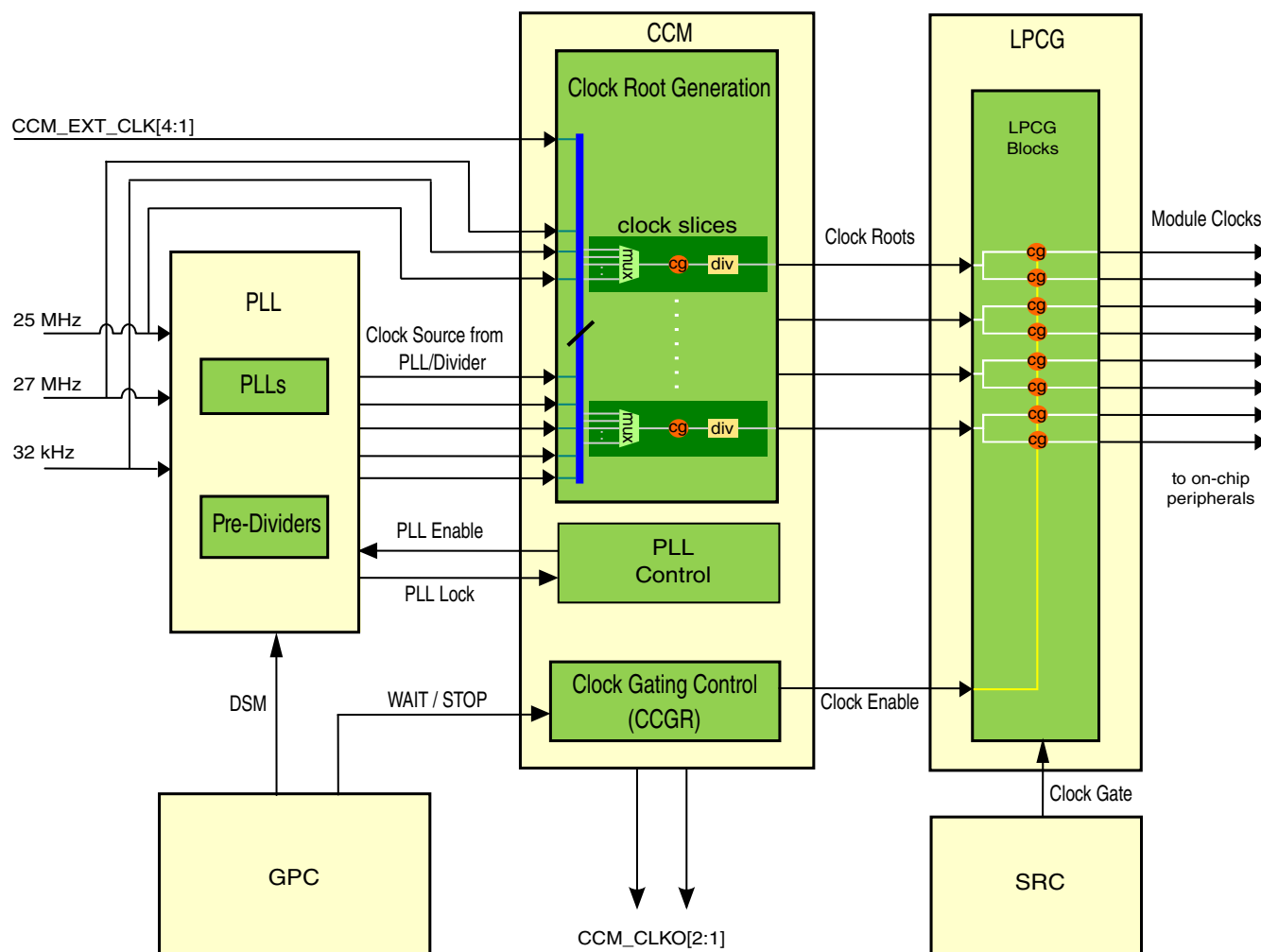


Figure 5-1. CCM Block Diagram

5.1.2 Clock Root Selects

The table below details the clock root slices and clock source inputs.

NOTE

The value of all clock root slice registers are zero after POR with the exception of DRAM clock. Please see System Boot for ROM reset values and default frequency settings for the clock root slices.

Table 5-1. Clock Root Table

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
0	ARM_A53_CLK_ROOT	1000	000 - 25M_REF_CLK 001 - ARM_PLL_CLK 010 - SYSTEM_PLL2_DIV2 011 - SYSTEM_PLL2_CLK 100 - SYSTEM_PLL1_CLK 101 - SYSTEM_PLL1_DIV2 110 - AUDIO_PLL1_CLK 111 - SYSTEM_PLL3_CLK
1	ARM_M4_CLK_ROOT	266	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV5 010 - SYSTEM_PLL2_DIV4 011 - SYSTEM_PLL1_DIV3 100 - SYSTEM_PLL1_CLK 101 - AUDIO_PLL1_CLK 110 - VIDEO_PLL1_CLK 111 - SYSTEM_PLL3_CLK
2	VPU_A53_CLK_ROOT	800	000 - 25M_REF_CLK 001 - ARM_PLL_CLK 010 - SYSTEM_PLL2_DIV2 011 - SYSTEM_PLL2_CLK 100 - SYSTEM_PLL1_CLK 101 - SYSTEM_PLL1_DIV2 110 - AUDIO_PLL1_CLK 111 - VPU_PLL_CLK
3	GPU_CORE_CLK_ROOT	800	000 - 25M_REF_CLK 001 - GPU_PLL_CLK 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL3_CLK 100 - SYSTEM_PLL2_CLK 101 - AUDIO_PLL1_CLK 110 - VIDEO_PLL1_CLK 111 - AUDIO_PLL2_CLK
4	GPU_SHADER_CLK_ROOT	1000	000 - 25M_REF_CLK 001 - GPU_PLL_CLK 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL3_CLK 100 - SYSTEM_PLL2_CLK

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			101 - AUDIO_PLL1_CLK 110 - VIDEO_PLL1_CLK 111 - AUDIO_PLL2_CLK
16	MAIN_AXI_CLK_ROOT	333	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV3 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL2_DIV4 100 - SYSTEM_PLL2_CLK 101 - AUDIO_PLL1_CLK 110 - VIDEO_PLL1_CLK 111 - SYSTEM_PLL1_DIV8
17	ENET_AXI_CLK_ROOT	266	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV3 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL2_DIV4 100 - SYSTEM_PLL2_DIV5 101 - AUDIO_PLL1_CLK 110 - VIDEO_PLL1_CLK 111 - SYSTEM_PLL3_CLK
18	NAND_USDHC_BUS_CLK_ROOT	266	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV3 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL2_DIV5 100 - SYSTEM_PLL1_DIV6 101 - SYSTEM_PLL3_CLK 110 - SYSTEM_PLL2_DIV4 111 - AUDIO_PLL1_CLK
19	VPU_BUS_CLK_ROOT	800	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_CLK 010 - VPU_PLL_CLK 011 - AUDIO_PLL2_CLK 100 - SYSTEM_PLL3_CLK 101 - SYSTEM_PLL2_CLK 110 - SYSTEM_PLL2_DIV5 111 - SYSTEM_PLL1_DIV8
20	DISPLAY_AXI_CLK_ROOT	800	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV8 010 - SYSTEM_PLL1_CLK

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			011 - SYSTEM_PLL3_CLK 100 - SYSTEM_PLL1_DIV20 101 - AUDIO_PLL2_CLK 110 - EXT_CLK_1 111 - EXT_CLK_4
21	DISPLAY_APB_CLK_ROOT	200	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV8 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL3_CLK 100 - SYSTEM_PLL1_DIV20 101 - AUDIO_PLL2_CLK 110 - EXT_CLK_1 111 - EXT_CLK_3
22	DISPLAY_RTRM_CLK_ROOT	400	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_CLK 010 - SYSTEM_PLL2_DIV5 011 - SYSTEM_PLL1_DIV2 100 - AUDIO_PLL1_CLK 101 - VIDEO_PLL1_CLK 110 - EXT_CLK_2 111 - EXT_CLK_3
23	USB_BUS_CLK_ROOT	400	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV2 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL2_DIV10 100 - SYSTEM_PLL2_DIV5 101 - EXT_CLK_2 110 - EXT_CLK_4 111 - AUDIO_PLL2_CLK
24	GPU_AXI_CLK_ROOT	800	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_CLK 010 - GPU_PLL_CLK 011 - SYSTEM_PLL3_CLK 100 - SYSTEM_PLL2_CLK 101 - AUDIO_PLL1_CLK 110 - VIDEO_PLL1_CLK 111 - AUDIO_PLL2_CLK
25	GPU_AHB_CLK_ROOT	400	000 - 25M_REF_CLK

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			001 - SYSTEM_PLL1_CLK 010 - GPU_PLL_CLK 011 - SYSTEM_PLL3_CLK 100 - SYSTEM_PLL2_CLK 101 - AUDIO_PLL1_CLK 110 - VIDEO_PLL1_CLK 111 - AUDIO_PLL2_CLK
26	NOC_CLK_ROOT	800	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_CLK 010 - SYSTEM_PLL3_CLK 011 - SYSTEM_PLL2_CLK 100 - SYSTEM_PLL2_DIV2 101 - AUDIO_PLL1_CLK 110 - VIDEO_PLL1_CLK 111 - AUDIO_PLL2_CLK
27	NOC_APB_CLK_ROOT	800	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV2 010 - SYSTEM_PLL3_CLK 011 - SYSTEM_PLL2_DIV3 100 - SYSTEM_PLL2_DIV5 101 - SYSTEM_PLL1_CLK 110 - AUDIO_PLL1_CLK 111 - VIDEO_PLL1_CLK
32	AHB_CLK_ROOT	133	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV6 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL1_DIV2 100 - SYSTEM_PLL2_DIV8 101 - SYSTEM_PLL3_CLK 110 - AUDIO_PLL1_CLK 111 - VIDEO_PLL1_CLK
34	AUDIO_AHB_CLK_ROOT	500	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV2 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL2_CLK 100 - SYSTEM_PLL2_DIV6 101 - SYSTEM_PLL3_CLK 110 - AUDIO_PLL1_CLK

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			111 - VIDEO_PLL1_CLK
36	MIPI_DSI_ESC_RX_CLK_ROOT	133	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV10 011 - SYSTEM_PLL1_CLK 100 - SYSTEM_PLL2_CLK 101 - SYSTEM_PLL3_CLK 110 - EXT_CLK_3 111 - AUDIO_PLL2_CLK
64	DRAM_ALT_CLK_ROOT	800	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_CLK 010 - SYSTEM_PLL1_DIV8 011 - SYSTEM_PLL2_DIV2 100 - SYSTEM_PLL2_DIV4 101 - SYSTEM_PLL1_DIV2 110 - AUDIO_PLL1_CLK 111 - SYSTEM_PLL1_DIV3
65	DRAM_APB_CLK_ROOT	200	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV5 010 - SYSTEM_PLL1_DIV20 011 - SYSTEM_PLL1_DIV5 100 - SYSTEM_PLL1_CLK 101 - SYSTEM_PLL3_CLK 110 - SYSTEM_PLL2_DIV4 111 - AUDIO_PLL2_CLK
66	VPU_G1_CLK_ROOT	600	000 - 25M_REF_CLK 001 - VPU_PLL_CLK 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL2_CLK 100 - SYSTEM_PLL1_DIV8 101 - SYSTEM_PLL2_DIV8 110 - SYSTEM_PLL3_CLK 111 - AUDIO_PLL1_CLK
67	VPU_G2_CLK_ROOT	300	000 - 25M_REF_CLK 001 - VPU_PLL_CLK 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL2_CLK 100 - SYSTEM_PLL1_DIV8

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			101 - SYSTEM_PLL2_DIV8 110 - SYSTEM_PLL3_CLK 111 - AUDIO_PLL1_CLK
68	DISPLAY_DTRC_CLK_ROOT	600	000 - 25M_REF_CLK 001 - VIDEO_PLL2_CLK 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL2_CLK 100 - SYSTEM_PLL1_DIV5 101 - VIDEO_PLL1_CLK 110 - SYSTEM_PLL3_CLK 111 - AUDIO_PLL2_CLK
69	DISPLAY_DC8000_CLK_ROOT	600	000 - 25M_REF_CLK 001 - VIDEO_PLL2_CLK 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL2_CLK 100 - SYSTEM_PLL1_DIV5 101 - VIDEO_PLL1_CLK 110 - SYSTEM_PLL3_CLK 111 - AUDIO_PLL2_CLK
70	PCIE1_CTRL_CLK_ROOT	250	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV4 010 - SYSTEM_PLL2_DIV5 011 - SYSTEM_PLL1_DIV3 100 - SYSTEM_PLL1_CLK 101 - SYSTEM_PLL2_DIV2 110 - SYSTEM_PLL2_DIV3 111 - SYSTEM_PLL3_CLK
71	PCIE1_PHY_CLK_ROOT	100	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL2_DIV2 011 - EXT_CLK_1 100 - EXT_CLK_2 101 - EXT_CLK_3 110 - EXT_CLK_4 111 - SYSTEM_PLL1_DIV2
72	PCIE1_AUX_CLK_ROOT	10	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV5 010 - SYSTEM_PLL2_DIV20

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			011 - SYSTEM_PLL3_CLK 100 - SYSTEM_PLL2_DIV10 101 - SYSTEM_PLL1_DIV10 110 - SYSTEM_PLL1_DIV5 111 - SYSTEM_PLL1_DIV4
73	DC_PIXEL_CLK_ROOT	594	000 - 25M_REF_CLK 001 - VIDEO_PLL1_CLK 010 - AUDIO_PLL2_CLK 011 - AUDIO_PLL1_CLK 100 - SYSTEM_PLL1_CLK 101 - SYSTEM_PLL2_CLK 110 - SYSTEM_PLL3_CLK 111 - EXT_CLK_4
74	LCDIF_PIXEL_CLK_ROOT	148.5	000 - 25M_REF_CLK 001 - VIDEO_PLL1_CLK 010 - AUDIO_PLL2_CLK 011 - AUDIO_PLL1_CLK 100 - SYSTEM_PLL1_CLK 101 - SYSTEM_PLL2_CLK 110 - SYSTEM_PLL3_CLK 111 - EXT_CLK_4
75	SAI1_CLK_ROOT	66	000 - 25M_REF_CLK 001 - AUDIO_PLL1_CLK 010 - AUDIO_PLL2_CLK 011 - VIDEO_PLL1_CLK 100 - SYSTEM_PLL1_DIV6 101 - 27M_REF_CLK 110 - EXT_CLK_1 111 - EXT_CLK_2
76	SAI2_CLK_ROOT	66	000 - 25M_REF_CLK 001 - AUDIO_PLL1_CLK 010 - AUDIO_PLL2_CLK 011 - VIDEO_PLL1_CLK 100 - SYSTEM_PLL1_DIV6 101 - 27M_REF_CLK 110 - EXT_CLK_2 111 - EXT_CLK_3
77	SAI3_CLK_ROOT	66	000 - 25M_REF_CLK

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			001 - AUDIO_PLL1_CLK 010 - AUDIO_PLL2_CLK 011 - VIDEO_PLL1_CLK 100 - SYSTEM_PLL1_DIV6 101 - 27M_REF_CLK 110 - EXT_CLK_3 111 - EXT_CLK_4
78	SAI4_CLK_ROOT	66	000 - 25M_REF_CLK 001 - AUDIO_PLL1_CLK 010 - AUDIO_PLL2_CLK 011 - VIDEO_PLL1_CLK 100 - SYSTEM_PLL1_DIV6 101 - 27M_REF_CLK 110 - EXT_CLK_1 111 - EXT_CLK_2
79	SAI5_CLK_ROOT	66	000 - 25M_REF_CLK 001 - AUDIO_PLL1_CLK 010 - AUDIO_PLL2_CLK 011 - VIDEO_PLL1_CLK 100 - SYSTEM_PLL1_DIV6 101 - 27M_REF_CLK 110 - EXT_CLK_2 111 - EXT_CLK_3
80	SAI6_CLK_ROOT	66	000 - 25M_REF_CLK 001 - AUDIO_PLL1_CLK 010 - AUDIO_PLL2_CLK 011 - VIDEO_PLL1_CLK 100 - SYSTEM_PLL1_DIV6 101 - 27M_REF_CLK 110 - EXT_CLK_3 111 - EXT_CLK_4
81	SPDIF1_CLK_ROOT	66	000 - 25M_REF_CLK 001 - AUDIO_PLL1_CLK 010 - AUDIO_PLL2_CLK 011 - VIDEO_PLL1_CLK 100 - SYSTEM_PLL1_DIV6 101 - 27M_REF_CLK 110 - EXT_CLK_2

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			111 - EXT_CLK_3
82	SPDIF2_CLK_ROOT	66	000 - 25M_REF_CLK 001 - AUDIO_PLL1_CLK 010 - AUDIO_PLL2_CLK 011 - VIDEO_PLL1_CLK 100 - SYSTEM_PLL1_DIV6 101 - 27M_REF_CLK 110 - EXT_CLK_3 111 - EXT_CLK_4
83	ENET_REF_CLK_ROOT	125	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV8 010 - SYSTEM_PLL2_DIV20 011 - SYSTEM_PLL2_DIV10 100 - SYSTEM_PLL1_DIV5 101 - AUDIO_PLL1_CLK 110 - VIDEO_PLL1_CLK 111 - EXT_CLK_4
84	ENET_TIMER_CLK_ROOT	125	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - AUDIO_PLL1_CLK 011 - EXT_CLK_1 100 - EXT_CLK_2 101 - EXT_CLK_3 110 - EXT_CLK_4 111 - VIDEO_PLL1_CLK
85	ENET_PHY_REF_CLK_ROOT	125	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV20 010 - SYSTEM_PLL2_DIV8 011 - SYSTEM_PLL2_DIV5 100 - SYSTEM_PLL2_DIV2 101 - AUDIO_PLL1_CLK 110 - VIDEO_PLL1_CLK 111 - AUDIO_PLL2_CLK
86	NAND_CLK_ROOT	500	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV2 010 - AUDIO_PLL1_CLK 011 - SYSTEM_PLL1_DIV2 100 - AUDIO_PLL2_CLK

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			101 - SYSTEM_PLL3_CLK 110 - SYSTEM_PLL2_DIV4 111 - VIDEO_PLL1_CLK
87	QSPI_CLK_ROOT	400	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV2 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL2_DIV2 100 - AUDIO_PLL2_CLK 101 - SYSTEM_PLL1_DIV3 110 - SYSTEM_PLL3_CLK 111 - SYSTEM_PLL1_DIV8
88	USDHC1_CLK_ROOT	400	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV2 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL2_DIV2 100 - SYSTEM_PLL3_CLK 101 - SYSTEM_PLL1_DIV3 110 - AUDIO_PLL2_CLK 111 - SYSTEM_PLL1_DIV8
89	USDHC2_CLK_ROOT	400	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV2 010 - SYSTEM_PLL1_CLK 011 - SYSTEM_PLL2_DIV2 100 - SYSTEM_PLL3_CLK 101 - SYSTEM_PLL1_DIV3 110 - AUDIO_PLL2_CLK 111 - SYSTEM_PLL1_DIV8
90	I2C1_CLK_ROOT	66	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV5 010 - SYSTEM_PLL2_DIV20 011 - SYSTEM_PLL3_CLK 100 - AUDIO_PLL1_CLK 101 - VIDEO_PLL1_CLK 110 - AUDIO_PLL2_CLK 111 - SYSTEM_PLL1_DIV6
91	I2C2_CLK_ROOT	66	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV5 010 - SYSTEM_PLL2_DIV20

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			011 - SYSTEM_PLL3_CLK 100 - AUDIO_PLL1_CLK 101 - VIDEO_PLL1_CLK 110 - AUDIO_PLL2_CLK 111 - SYSTEM_PLL1_DIV6
92	I2C3_CLK_ROOT	66	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV5 010 - SYSTEM_PLL2_DIV20 011 - SYSTEM_PLL3_CLK 100 - AUDIO_PLL1_CLK 101 - VIDEO_PLL1_CLK 110 - AUDIO_PLL2_CLK 111 - SYSTEM_PLL1_DIV6
93	I2C4_CLK_ROOT	66	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV5 010 - SYSTEM_PLL2_DIV20 011 - SYSTEM_PLL3_CLK 100 - AUDIO_PLL1_CLK 101 - VIDEO_PLL1_CLK 110 - AUDIO_PLL2_CLK 111 - SYSTEM_PLL1_DIV6
94	UART1_CLK_ROOT	80	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV10 010 - SYSTEM_PLL2_DIV5 011 - SYSTEM_PLL2_DIV10 100 - SYSTEM_PLL3_CLK 101 - EXT_CLK_2 110 - EXT_CLK_4 111 - AUDIO_PLL2_CLK
95	UART2_CLK_ROOT	80	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV10 010 - SYSTEM_PLL2_DIV5 011 - SYSTEM_PLL2_DIV10 100 - SYSTEM_PLL3_CLK 101 - EXT_CLK_2 110 - EXT_CLK_3 111 - AUDIO_PLL2_CLK
96	UART3_CLK_ROOT	80	000 - 25M_REF_CLK

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			001 - SYSTEM_PLL1_DIV10 010 - SYSTEM_PLL2_DIV5 011 - SYSTEM_PLL2_DIV10 100 - SYSTEM_PLL3_CLK 101 - EXT_CLK_2 110 - EXT_CLK_4 111 - AUDIO_PLL2_CLK
97	UART4_CLK_ROOT	80	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV10 010 - SYSTEM_PLL2_DIV5 011 - SYSTEM_PLL2_DIV10 100 - SYSTEM_PLL3_CLK 101 - EXT_CLK_2 110 - EXT_CLK_3 111 - AUDIO_PLL2_CLK
98	USB_CORE_REF_CLK_ROOT	125	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV8 010 - SYSTEM_PLL1_DIV20 011 - SYSTEM_PLL2_DIV10 100 - SYSTEM_PLL2_DIV5 101 - EXT_CLK_2 110 - EXT_CLK_3 111 - AUDIO_PLL2_CLK
99	USB_PHY_REF_CLK_ROOT	100	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV8 010 - SYSTEM_PLL1_DIV20 011 - SYSTEM_PLL2_DIV10 100 - SYSTEM_PLL2_DIV5 101 - EXT_CLK_2 110 - EXT_CLK_3 111 - AUDIO_PLL2_CLK
100	GIC_CLK_ROOT	400	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV5 010 - SYSTEM_PLL1_DIV20 011 - SYSTEM_PLL2_DIV10 100 - SYSTEM_PLL1_CLK 101 - EXT_CLK_2 110 - EXT_CLK_4

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			111 - AUDIO_PLL2_CLK
101	ECSPI1_CLK_ROOT	80	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV5 010 - SYSTEM_PLL1_DIV20 011 - SYSTEM_PLL1_DIV5 100 - SYSTEM_PLL1_CLK 101 - SYSTEM_PLL3_CLK 110 - SYSTEM_PLL2_DIV4 111 - AUDIO_PLL2_CLK
102	ECSPI2_CLK_ROOT	80	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV5 010 - SYSTEM_PLL1_DIV20 011 - SYSTEM_PLL1_DIV5 100 - SYSTEM_PLL1_CLK 101 - SYSTEM_PLL3_CLK 110 - SYSTEM_PLL2_DIV4 111 - AUDIO_PLL2_CLK
103	PWM1_CLK_ROOT	66	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV5 011 - SYSTEM_PLL1_DIV20 100 - SYSTEM_PLL3_CLK 101 - EXT_CLK_1 110 - SYSTEM_PLL1_DIV10 111 - VIDEO_PLL1_CLK
104	PWM2_CLK_ROOT	66	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV5 011 - SYSTEM_PLL1_DIV20 100 - SYSTEM_PLL3_CLK 101 - EXT_CLK_1 110 - SYSTEM_PLL1_DIV10 111 - VIDEO_PLL1_CLK
105	PWM3_CLK_ROOT	66	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV5 011 - SYSTEM_PLL1_DIV20 100 - SYSTEM_PLL3_CLK

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			101 - EXT_CLK_2 110 - SYSTEM_PLL1_DIV10 111 - VIDEO_PLL1_CLK
106	PWM4_CLK_ROOT	66	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV5 011 - SYSTEM_PLL1_DIV20 100 - SYSTEM_PLL3_CLK 101 - EXT_CLK_2 110 - SYSTEM_PLL1_DIV10 111 - VIDEO_PLL1_CLK
107	GPT1_CLK_ROOT	100	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV2 011 - SYSTEM_PLL1_DIV20 100 - VIDEO_PLL1_CLK 101 - SYSTEM_PLL1_DIV10 110 - AUDIO_PLL1_CLK 111 - EXT_CLK_1
108	GPT2_CLK_ROOT	100	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV2 011 - SYSTEM_PLL1_DIV20 100 - VIDEO_PLL1_CLK 101 - SYSTEM_PLL1_DIV10 110 - AUDIO_PLL1_CLK 111 - EXT_CLK_2
109	GPT3_CLK_ROOT	100	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV2 011 - SYSTEM_PLL1_DIV20 100 - VIDEO_PLL1_CLK 101 - SYSTEM_PLL1_DIV10 110 - AUDIO_PLL1_CLK 111 - EXT_CLK_3
110	GPT4_CLK_ROOT	100	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV2

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			011 - SYSTEM_PLL1_DIV20 100 - VIDEO_PLL1_CLK 101 - SYSTEM_PLL1_DIV10 110 - AUDIO_PLL1_CLK 111 - EXT_CLK_1
111	GPT5_CLK_ROOT	100	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV2 011 - SYSTEM_PLL1_DIV20 100 - VIDEO_PLL1_CLK 101 - SYSTEM_PLL1_DIV10 110 - AUDIO_PLL1_CLK 111 - EXT_CLK_2
112	GPT6_CLK_ROOT	100	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV2 011 - SYSTEM_PLL1_DIV20 100 - VIDEO_PLL1_CLK 101 - SYSTEM_PLL1_DIV10 110 - AUDIO_PLL1_CLK 111 - EXT_CLK_3
113	TRACE_CLK_ROOT	133	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV6 010 - SYSTEM_PLL1_DIV5 011 - VPU_PLL_CLK 100 - SYSTEM_PLL2_DIV8 101 - SYSTEM_PLL3_CLK 110 - EXT_CLK_1 111 - EXT_CLK_3
114	WDOG_CLK_ROOT	66	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV6 010 - SYSTEM_PLL1_DIV5 011 - VPU_PLL_CLK 100 - SYSTEM_PLL2_DIV8 101 - SYSTEM_PLL3_CLK 110 - SYSTEM_PLL1_DIV10 111 - SYSTEM_PLL2_DIV6
115	WRCLK_CLK_ROOT	40	000 - 25M_REF_CLK

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			001 - SYSTEM_PLL1_DIV20 010 - VPU_PLL_CLK 011 - SYSTEM_PLL3_CLK 100 - SYSTEM_PLL2_DIV5 101 - SYSTEM_PLL1_DIV3 110 - SYSTEM_PLL2_DIV2 111 - SYSTEM_PLL1_DIV8
116	IPP_DO_CLKO1	266	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_CLK 010 - 27M_REF_CLK 011 - SYSTEM_PLL1_DIV4 100 - AUDIO_PLL2_CLK 101 - SYSTEM_PLL2_DIV2 110 - VPU_PLL_CLK 111 - SYSTEM_PLL1_DIV10
117	IPP_DO_CLKO2	266	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV5 010 - SYSTEM_PLL1_DIV2 011 - SYSTEM_PLL2_DIV6 100 - SYSTEM_PLL3_CLK 101 - AUDIO_PLL1_CLK 110 - VIDEO_PLL1_CLK 111 - 32K_REF_CLK
118	MIPI_DSI_CORE_CLK_ROOT	266	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV3 010 - SYSTEM_PLL2_DIV4 011 - SYSTEM_PLL1_CLK 100 - SYSTEM_PLL2_CLK 101 - SYSTEM_PLL3_CLK 110 - AUDIO_PLL2_CLK 111 - VIDEO_PLL1_CLK
119	MIPI_DSI_PHY_REF_CLK_ROOT	125	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV8 010 - SYSTEM_PLL2_DIV10 011 - SYSTEM_PLL1_CLK 100 - SYSTEM_PLL2_CLK 101 - EXT_CLK_2 110 - AUDIO_PLL2_CLK

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			111 - VIDEO_PLL1_CLK
120	MIPI_DSI_DBI_CLK_ROOT	266	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV3 010 - SYSTEM_PLL2_DIV10 011 - SYSTEM_PLL1_CLK 100 - SYSTEM_PLL2_CLK 101 - SYSTEM_PLL3_CLK 110 - AUDIO_PLL2_CLK 111 - VIDEO_PLL1_CLK
121	OLD_MIPI_DSI_ESC_CLK_ROOT	133	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV10 011 - SYSTEM_PLL1_CLK 100 - SYSTEM_PLL2_CLK 101 - SYSTEM_PLL3_CLK 110 - EXT_CLK_3 111 - AUDIO_PLL2_CLK
122	MIPI_CSI1_CORE_CLK_ROOT	266	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV3 010 - SYSTEM_PLL2_DIV4 011 - SYSTEM_PLL1_CLK 100 - SYSTEM_PLL2_CLK 101 - SYSTEM_PLL3_CLK 110 - AUDIO_PLL2_CLK 111 - VIDEO_PLL1_CLK
123	MIPI_CSI1_PHY_REF_CLK_ROOT	333	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV3 010 - SYSTEM_PLL2_DIV10 011 - SYSTEM_PLL1_CLK 100 - SYSTEM_PLL2_CLK 101 - EXT_CLK_2 110 - AUDIO_PLL2_CLK 111 - VIDEO_PLL1_CLK
124	MIPI_CSI1_ESC_CLK_ROOT	133	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV10 011 - SYSTEM_PLL1_CLK 100 - SYSTEM_PLL2_CLK

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			101 - SYSTEM_PLL3_CLK 110 - EXT_CLK_3 111 - AUDIO_PLL2_CLK
125	MIPI_CSI2_CORE_CLK_ROOT	266	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV3 010 - SYSTEM_PLL2_DIV4 011 - SYSTEM_PLL1_CLK 100 - SYSTEM_PLL2_CLK 101 - SYSTEM_PLL3_CLK 110 - AUDIO_PLL2_CLK 111 - VIDEO_PLL1_CLK
126	MIPI_CSI2_PHY_REF_CLK_ROOT	333	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV3 010 - SYSTEM_PLL2_DIV10 011 - SYSTEM_PLL1_CLK 100 - SYSTEM_PLL2_CLK 101 - EXT_CLK_2 110 - AUDIO_PLL2_CLK 111 - VIDEO_PLL1_CLK
127	MIPI_CSI2_ESC_CLK_ROOT	133	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV10 011 - SYSTEM_PLL1_CLK 100 - SYSTEM_PLL2_CLK 101 - SYSTEM_PLL3_CLK 110 - EXT_CLK_3 111 - AUDIO_PLL2_CLK
128	PCIE2_CTRL_CLK_ROOT	250	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV4 010 - SYSTEM_PLL2_DIV5 011 - SYSTEM_PLL1_DIV3 100 - SYSTEM_PLL1_CLK 101 - SYSTEM_PLL2_DIV2 110 - SYSTEM_PLL2_DIV3 111 - SYSTEM_PLL3_CLK
129	PCIE2_PHY_CLK_ROOT	100	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL2_DIV2

Table continues on the next page...

Table 5-1. Clock Root Table (continued)

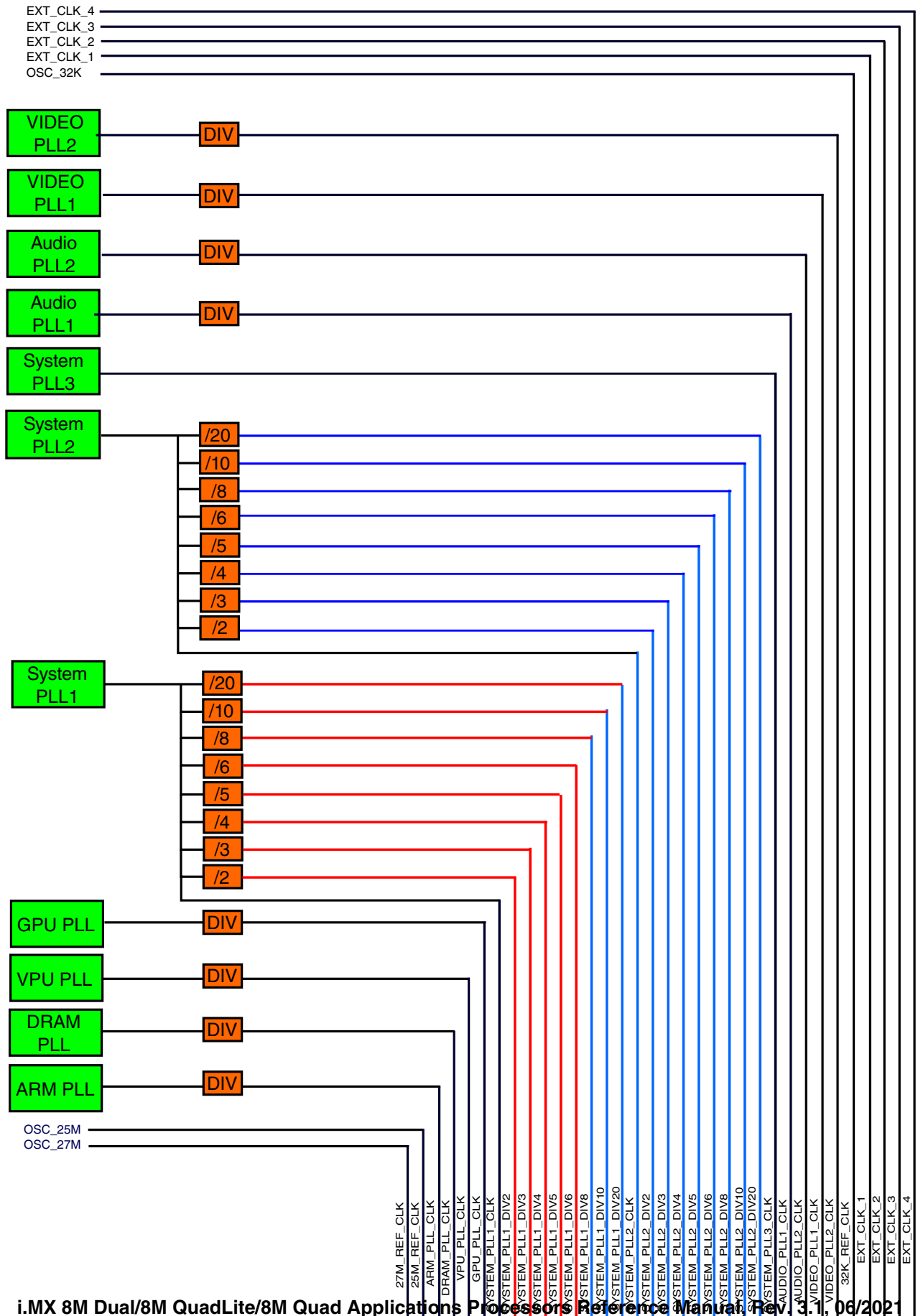
Clock Slice	Clock Root	Max Frequency (MHz)	Source Select (CCM_TARGET_ROOT[MUX])
			011 - EXT_CLK_1 100 - EXT_CLK_2 101 - EXT_CLK_3 110 - EXT_CLK_4 111 - SYSTEM_PLL1_DIV2
130	PCIE2_AUX_CLK_ROOT	10	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV5 010 - SYSTEM_PLL2_DIV20 011 - SYSTEM_PLL3_CLK 100 - SYSTEM_PLL2_DIV10 101 - SYSTEM_PLL1_DIV10 110 - SYSTEM_PLL1_DIV5 111 - SYSTEM_PLL1_DIV4
131	ECSPI3_CLK_ROOT	80	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV5 010 - SYSTEM_PLL1_DIV20 011 - SYSTEM_PLL1_DIV5 100 - SYSTEM_PLL1_CLK 101 - SYSTEM_PLL3_CLK 110 - SYSTEM_PLL2_DIV4 111 - AUDIO_PLL2_CLK
132	OLD_MIPI_DSI_ESC_RX_CLK_ROOT	60	000 - 25M_REF_CLK 001 - SYSTEM_PLL2_DIV10 010 - SYSTEM_PLL1_DIV10 011 - SYSTEM_PLL1_CLK 100 - SYSTEM_PLL2_CLK 101 - SYSTEM_PLL3_CLK 110 - EXT_CLK_3 111 - AUDIO_PLL2_CLK
133	DISPLAY_HDMI_CLK_ROOT	200	000 - 25M_REF_CLK 001 - SYSTEM_PLL1_DIV4 010 - SYSTEM_PLL2_DIV5 011 - VPU_PLL_CLK 100 - SYSTEM_PLL1_CLK 101 - SYSTEM_PLL2_CLK 110 - SYSTEM_PLL3_CLK 111 - EXT_CLK_4

5.1.3 Clock Tree

The figure below illustrates the clock sources from the PLLs.

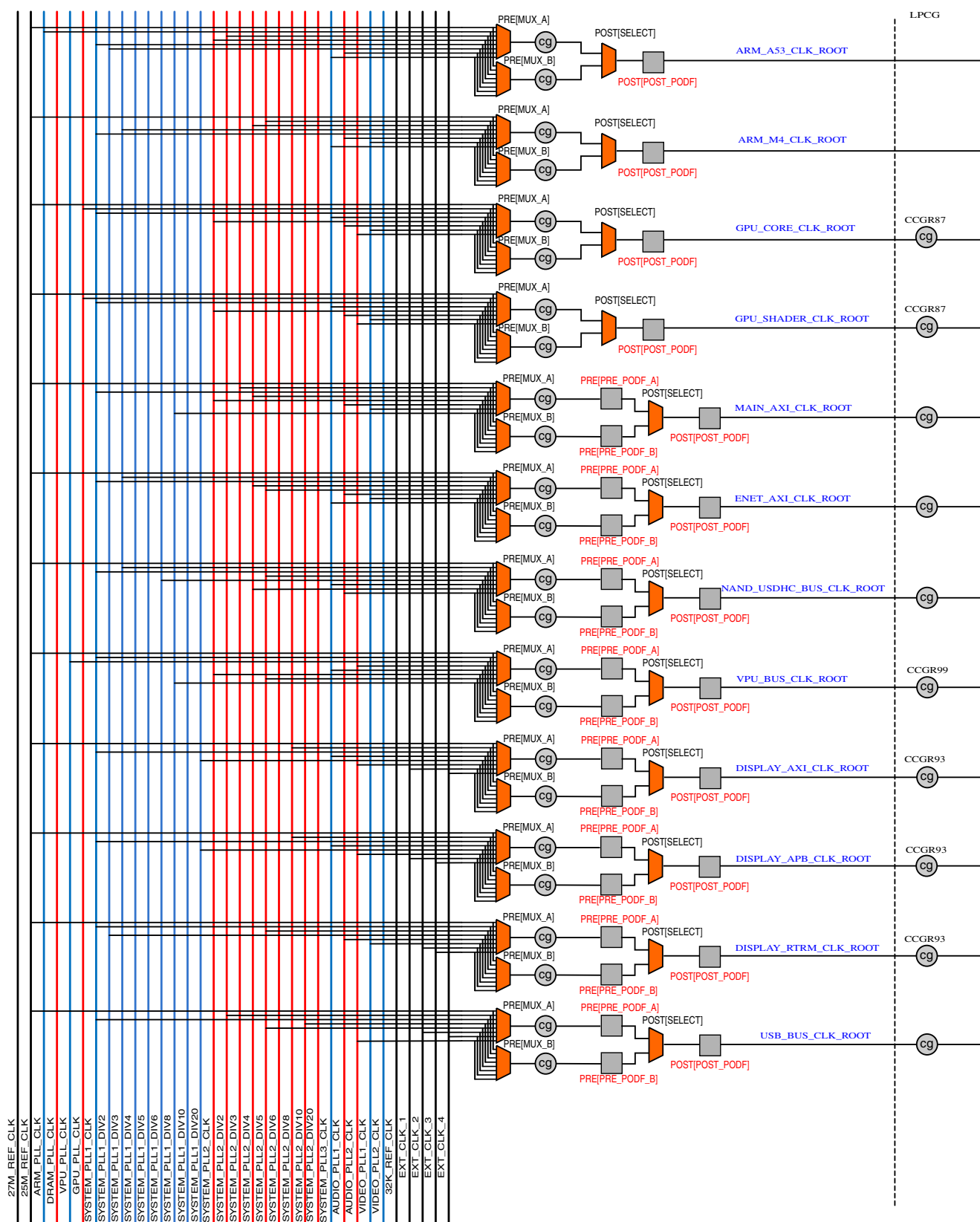
NOTE

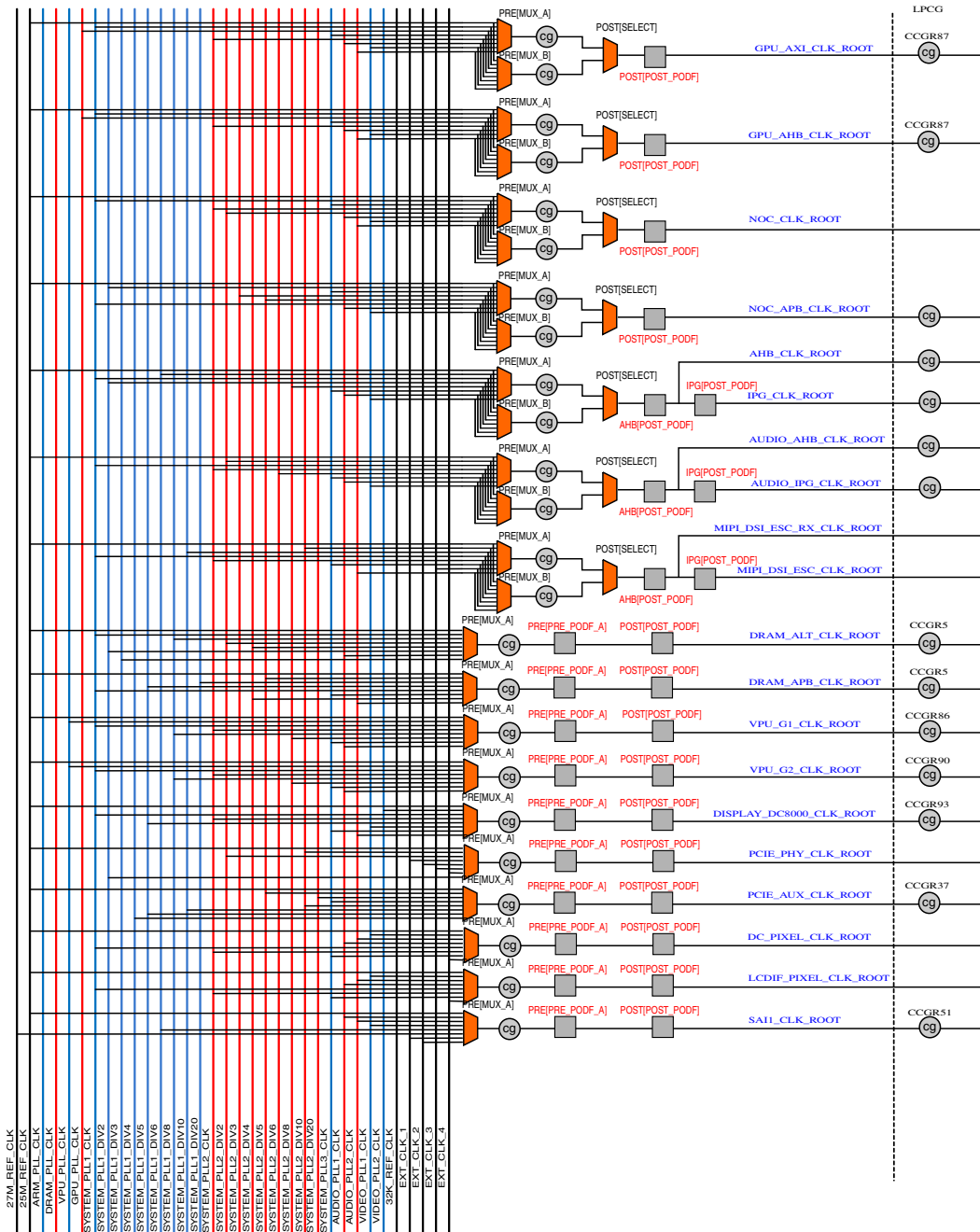
Some clock gates illustrated below are symbolic of distributed clock gates (multiple) and not a sole clock gate. These clock slices typically source multiple IP (e.g. bus clocks).

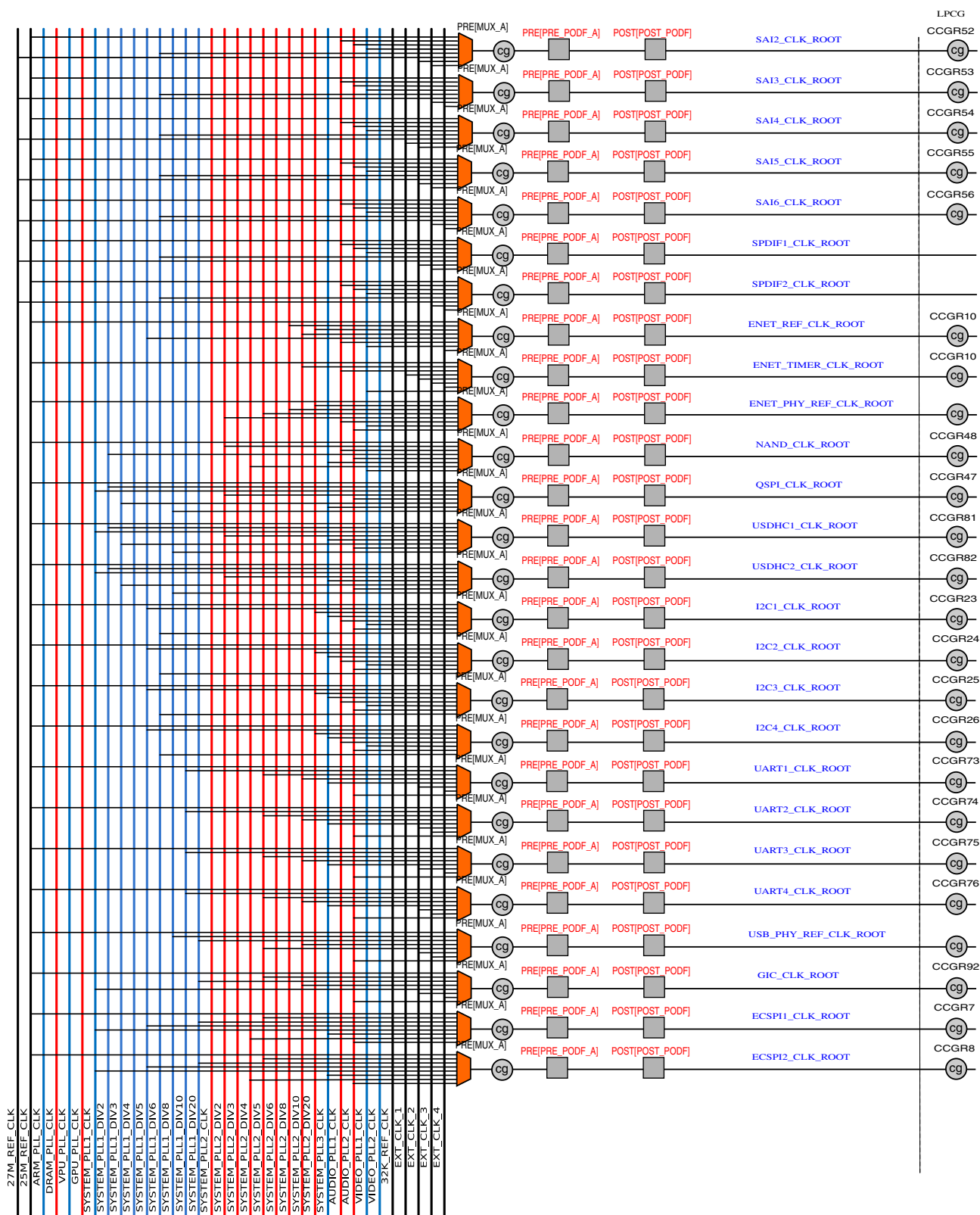


i.MX 8M Dual/8M QuadLite/8M Quad Applications Processor Reference Manual, Rev. 3.1, 06/2021

The figure below illustrates the clock slices of CCM and clock root generation.







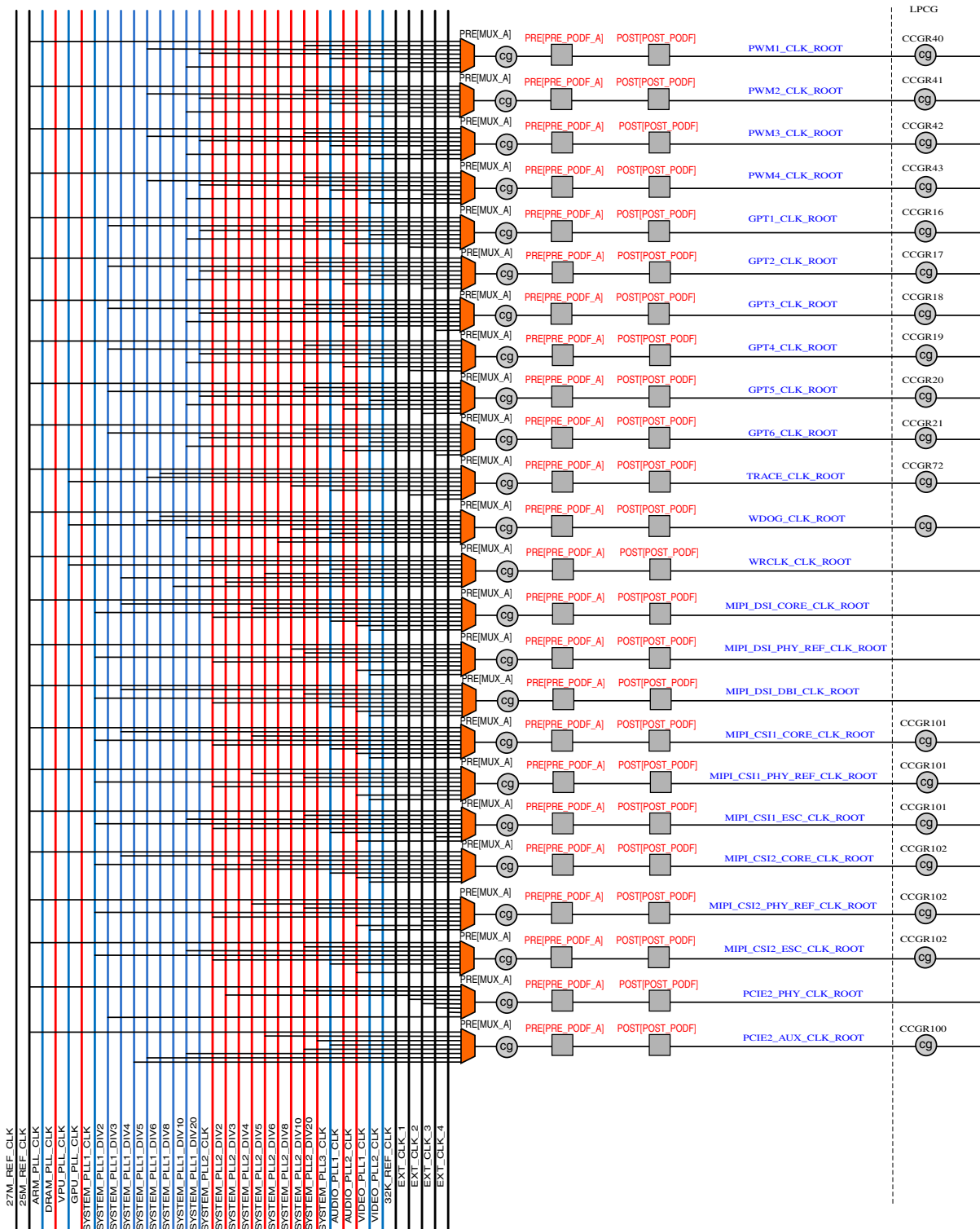


Figure 5-3. CCM Clock Tree Root Slices

5.1.4 System Clocks

The table below shows the CCM output module clock connectivity and gating.

Table 5-2. System Clocks and Gating

Module	Module Clock (instance.clock)	Clock Root	Module Clock Gating Enable (CCGR)
AIPS_TZ	aips_tz1.hclk	ccm_ahb_clk_root	clk_enable_ipmux1 (CCGR28)
	ipmux1.master_clk	ccm_ahb_clk_root	clk_enable_ipmux1 (CCGR28)
	ipmux1.slave_clk	ccm_ipg_clk_root	clk_enable_ipmux1 (CCGR28)
	aips_tz2.hclk	ccm_ahb_clk_root	clk_enable_ipmux2 (CCGR29)
	ipmux2.master_clk	ccm_ahb_clk_root	clk_enable_ipmux2 (CCGR29)
	ipmux2.slave_clk	ccm_ipg_clk_root	clk_enable_ipmux2 (CCGR29)
	aips_tz3.hclk	ccm_ahb_clk_root	clk_enable_ipmux3 (CCGR30)
	ipmux3.master_clk	ccm_ahb_clk_root	clk_enable_ipmux3 (CCGR30)
	ipmux3.slave_clk	ccm_ipg_clk_root	clk_enable_ipmux3 (CCGR30)
APBHDMA	apbhdma.hclk	ccm_nand_usdhc_bus_clk_root	clk_enable_rawnand (CCGR48)
	apbhdma_sec.mst_hclk	ccm_nand_usdhc_bus_clk_root	clk_enable_rawnand (CCGR48)
	rawnand.u_bch_input_apb_clk	ccm_nand_usdhc_bus_clk_root	clk_enable_rawnand (CCGR48)
	rawnand.u_gpmi_bch_input_bch_clk	ccm_nand_usdhc_bus_clk_root	clk_enable_rawnand (CCGR48)
	rawnand.u_gpmi_bch_input_gpmi_io_clk	ccm_nand_clk_root	clk_enable_rawnand (CCGR48)
	rawnand.U_gpmi_input_apb_clk	ccm_nand_usdhc_bus_clk_root	clk_enable_rawnand (CCGR48)
CAAM	caam.aclk	ccm_ahb_clk_root	1
	caam.ipg_clk	ccm_ipg_clk_root	1
	caam.ipg_clk_s	ccm_ipg_clk_root	1
	caam_exsc.aclk_exsc	ccm_ahb_clk_root	1
	caam_mem.clk	ccm_ahb_clk_root	1
CM4	cm4.cm4_cti_clk	ccm_arm_m4_clk_root	GPC controlled
	cm4.cm4_fclk	ccm_arm_m4_clk_root	GPC controlled

Table continues on the next page...

Table 5-2. System Clocks and Gating (continued)

Module	Module Clock (instance.clock)	Clock Root	Module Clock Gating Enable (CCGR)
	cm4.cm4_hclk	ccm_arm_m4_clk_root	GPC controlled
	cm4.dap_clk	ccm_ahb_clk_root	GPC controlled
	cm4.ipg_clk_nic	ccm_arm_m4_clk_root	GPC controlled
	cm4.tmc_hclk	ccm_arm_m4_clk_root	GPC controlled
	cm4_mem.tmc_hclk	ccm_arm_m4_clk_root	GPC controlled
	cm4_sec.ipg_clk	ccm_ipg_clk_root	GPC controlled
	cm4_sec.ipg_clk_s	ccm_ipg_clk_root	GPC controlled
	cm4_sec.mst_hclk	ccm_arm_m4_clk_root	GPC controlled
CSI	csi2_1.clk_vid	ccm_mipi_csi1_phy_ref_clk_root	1
	csi2_1.clk	ccm_mipi_csi1_core_clk_root	clk_enable_mipi_csi1 (CCGR101)
	csi2_1.clk_esc	ccm_mipi_csi1_esc_clk_root	clk_enable_mipi_csi1 (CCGR101)
	csi2_1.pclk	ccm_mipi_csi1_core_clk_root	1
	csi2_1.clk_ui	ccm_mipi_csi1_phy_ref_clk_root	clk_enable_mipi_csi1 (CCGR101)
	csi2_2.clk_vid	ccm_mipi_csi2_phy_ref_clk_root	1
	csi2_2.clk	ccm_mipi_csi2_core_clk_root	clk_enable_mipi_csi2 (CCGR102)
	csi2_2.clk_esc	ccm_mipi_csi2_esc_clk_root	clk_enable_mipi_csi2 (CCGR102)
	csi2_2.pclk	ccm_mipi_csi2_core_clk_root	1
	csi2_2.clk_ui	ccm_mipi_csi2_phy_ref_clk_root	clk_enable_mipi_csi2 (CCGR102)
CSU	csu.ipg_clk_s	ccm_ipg_clk_root	clk_enable_csu (CCGR3)
DAP	dap.dapclk_2_2	ccm_ahb_clk_root	clk_enable_debug (CCGR4)
ECSPI	ecspi1.ipg_clk	ccm_ipg_clk_root	clk_enable_ecspi1 (CCGR7)
	ecspi1.ipg_clk_per	ccm_ecspi1_clk_root	clk_enable_ecspi1 (CCGR7)
	ecspi1.ipg_clk_s	ccm_ipg_clk_root	clk_enable_ecspi1 (CCGR7)
	ecspi2.ipg_clk	ccm_ipg_clk_root	clk_enable_ecspi2 (CCGR8)
	ecspi2.ipg_clk_per	ccm_ecspi2_clk_root	clk_enable_ecspi2 (CCGR8)
	ecspi2.ipg_clk_s	ccm_ipg_clk_root	clk_enable_ecspi2 (CCGR8)
	ecspi3.ipg_clk	ccm_ipg_clk_root	clk_enable_ecspi3 (CCGR9)
	ecspi3.ipg_clk_per	ccm_ecspi3_clk_root	clk_enable_ecspi3 (CCGR9)
	ecspi3.ipg_clk_s	ccm_ipg_clk_root	clk_enable_ecspi3 (CCGR9)
ENET	enet1.ipp_ind_mac0_txclk	ccm_enet_ref_clk_root	clk_enable_enet1 (CCGR10)
	enet1.ipg_clk	ccm_enet_axi_clk_root	clk_enable_enet1 (CCGR10)
	enet1.ipg_clk_mac0	ccm_enet_axi_clk_root	clk_enable_enet1 (CCGR10)
	enet1.ipg_clk_mac0_s	ccm_enet_axi_clk_root	clk_enable_enet1 (CCGR10)

Table continues on the next page...

Table 5-2. System Clocks and Gating (continued)

Module	Module Clock (instance.clock)	Clock Root	Module Clock Gating Enable (CCGR)
	enet1.ipg_clk_s	ccm_enet_axi_clk_root	clk_enable_enet1 (CCGR10)
	enet1.ipg_clk_time	ccm_enet_timer_clk_root	clk_enable_enet1 (CCGR10)
	enet1_mem.mac0_rxmem_clk	ccm_enet_axi_clk_root	clk_enable_enet1 (CCGR10)
	enet1_mem.mac0_txmem_clk	ccm_enet_axi_clk_root	clk_enable_enet1 (CCGR10)
GPIO	gpio1.ipg_clk_s	ccm_ipg_clk_root	clk_enable_gpio1 (CCGR11)
	gpio2.ipg_clk_s	ccm_ipg_clk_root	clk_enable_gpio2 (CCGR12)
	gpio3.ipg_clk_s	ccm_ipg_clk_root	clk_enable_gpio3 (CCGR13)
	gpio4.ipg_clk_s	ccm_ipg_clk_root	clk_enable_gpio4 (CCGR14)
	gpio5.ipg_clk_s	ccm_ipg_clk_root	clk_enable_gpio5 (CCGR15)
GPT	gpt1.ipg_clk	ccm_gpt1_clk_root	clk_enable_gpt1 (CCGR16)
	gpt1.ipg_clk_24m	anamix_osc_25m_clk	clk_enable_gpt1 (CCGR16)
	gpt1.ipg_clk_highfreq	ccm_gpt1_clk_root	clk_enable_gpt1 (CCGR16)
	gpt1.ipg_clk_s	ccm_gpt1_clk_root	clk_enable_gpt1 (CCGR16)
	gpt2.ipg_clk	ccm_gpt2_clk_root	clk_enable_gpt2 (CCGR17)
	gpt2.ipg_clk_24m	anamix_osc_25m_clk	clk_enable_gpt2 (CCGR17)
	gpt2.ipg_clk_highfreq	ccm_gpt2_clk_root	clk_enable_gpt2 (CCGR17)
	gpt2.ipg_clk_s	ccm_gpt2_clk_root	clk_enable_gpt2 (CCGR17)
	gpt3.ipg_clk	ccm_gpt3_clk_root	clk_enable_gpt3 (CCGR18)
	gpt3.ipg_clk_24m	anamix_osc_25m_clk	clk_enable_gpt3 (CCGR18)
	gpt3.ipg_clk_highfreq	ccm_gpt3_clk_root	clk_enable_gpt3 (CCGR18)
	gpt3.ipg_clk_s	ccm_gpt3_clk_root	clk_enable_gpt3 (CCGR18)
	gpt4.ipg_clk	ccm_gpt4_clk_root	clk_enable_gpt4 (CCGR19)
	gpt4.ipg_clk_24m	anamix_osc_25m_clk	clk_enable_gpt4 (CCGR19)
	gpt4.ipg_clk_highfreq	ccm_gpt4_clk_root	clk_enable_gpt4 (CCGR19)
	gpt4.ipg_clk_s	ccm_gpt4_clk_root	clk_enable_gpt4 (CCGR19)
	gpt5.ipg_clk	ccm_gpt5_clk_root	clk_enable_gpt5 (CCGR20)
	gpt5.ipg_clk_24m	anamix_osc_25m_clk	clk_enable_gpt5 (CCGR20)
	gpt5.ipg_clk_highfreq	ccm_gpt5_clk_root	clk_enable_gpt5 (CCGR20)
	gpt5.ipg_clk_s	ccm_gpt5_clk_root	clk_enable_gpt5 (CCGR20)
	gpt6.ipg_clk	ccm_gpt6_clk_root	clk_enable_gpt6 (CCGR21)
	gpt6.ipg_clk_24m	anamix_osc_25m_clk	clk_enable_gpt6 (CCGR21)
	gpt6.ipg_clk_highfreq	ccm_gpt6_clk_root	clk_enable_gpt6 (CCGR21)
	gpt6.ipg_clk_s	ccm_gpt6_clk_root	clk_enable_gpt6 (CCGR21)
I2C	i2c1.ipg_clk_patref	ccm_i2c1_clk_root	clk_enable_i2c1 (CCGR23)
	i2c1.ipg_clk_s	ccm_i2c1_clk_root	clk_enable_i2c1 (CCGR23)
	i2c2.ipg_clk_patref	ccm_i2c2_clk_root	clk_enable_i2c2 (CCGR24)
	i2c2.ipg_clk_s	ccm_i2c2_clk_root	clk_enable_i2c2 (CCGR24)
	i2c3.ipg_clk_patref	ccm_i2c3_clk_root	clk_enable_i2c3 (CCGR25)
	i2c3.ipg_clk_s	ccm_i2c3_clk_root	clk_enable_i2c3 (CCGR25)

Table continues on the next page...

Table 5-2. System Clocks and Gating (continued)

Module	Module Clock (instance.clock)	Clock Root	Module Clock Gating Enable (CCGR)
IOMUXC	i2c4.ipg_clk_patref	ccm_i2c4_clk_root	clk_enable_i2c4 (CCGR26)
	i2c4.ipg_clk_s	ccm_i2c4_clk_root	clk_enable_i2c4 (CCGR26)
	iomuxc.ipg_clk_s	ccm_ipg_clk_root	clk_enable_iomux (CCGR27)
	iomuxc_gpr.ipg_clk_s	ccm_ipg_clk_root	clk_enable_iomux (CCGR27)
	iomux.ipt_clk_io	ccm_ipg_clk_root	clk_enable_iomux (CCGR27)
LCD	lcdif.pix_clk	ccm_lcdif_pixel_clk_root	1
	lcdif.apb_clk	ccm_main_axi_clk_root	1
DSI	mipi.CLKREF	ccm_mipi_dsi_phy_ref_clk_ro ot	1
	mipi.pclk	ccm_main_axi_clk_root	1
	mipi.RxClkEsc	ccm_mipi_dsi_esc_rx_clk_roo t	1
	mipi.TxClkEsc	ccm_mipi_dsi_esc_clk_root	1
MU	mu.ipg_clk_dsp	ccm_ipg_clk_root	clk_enable_mu (CCGR33)
	mu.ipg_clk_mcu	ccm_ipg_clk_root	clk_enable_mu (CCGR33)
	mu.ipg_clk_s_dsp	ccm_ipg_clk_root	clk_enable_mu (CCGR33)
	mu.ipg_clk_s_mcu	ccm_ipg_clk_root	clk_enable_mu (CCGR33)
OCOTP	ocotp.ipg_clk	ccm_ipg_clk_root	clk_enable_ocotp (CCGR34)
	ocotp.ipg_clk_s	ccm_ipg_clk_root	clk_enable_ocotp (CCGR34)
OCRAM	ocram_ctrl.clk	ccm_main_axi_clk_root	clk_enable_ocram (CCGR35)
	ocram_ctrl_s.clk	ccm_ahb_clk_root	clk_enable_ocram_s (CCGR36)
	ocram_exsc.aclk_exsc	ccm_main_axi_clk_root	clk_enable_ocram (CCGR35)
	ocram_exsc.ipg_clk	ccm_ipg_clk_root	clk_enable_ocram (CCGR35)
	ocram_mem.clk	ccm_main_axi_clk_root	clk_enable_ocram (CCGR35)
	ocram_s_exsc.aclk_exsc	ccm_ahb_clk_root	clk_enable_ocram_s (CCGR36)
	ocram_s_exsc.ipg_clk	ccm_ipg_clk_root	clk_enable_ocram_s (CCGR36)
	ocram_s_mem.clk	ccm_ahb_clk_root	clk_enable_ocram_s (CCGR36)
PCIE	pcie_clk_rst.auxclk	ccm_pcie_aux_clk_root	clk_enable_pcie (CCGR37)
	pcie_clk_rst.mstr_axi_clk	ccm_main_axi_clk_root	clk_enable_pcie (CCGR37)
	pcie_clk_rst.slv_axi_clk	ccm_main_axi_clk_root	clk_enable_pcie (CCGR37)
	pcie_ctrl.mstr_aclk	ccm_main_axi_clk_root	clk_enable_pcie (CCGR37)
	pcie_ctrl.slv_aclk	ccm_main_axi_clk_root	clk_enable_pcie (CCGR37)
	pcie_exsc.aclk_exsc	ccm_main_axi_clk_root	clk_enable_pcie (CCGR37)
	pcie_exsc.ipg_clk	ccm_ipg_clk_root	clk_enable_pcie (CCGR37)
	pcie_mem.mstr_axi_clk	ccm_main_axi_clk_root	clk_enable_pcie (CCGR37)
	pcie_mem.slv_axi_clk	ccm_main_axi_clk_root	clk_enable_pcie (CCGR37)

Table continues on the next page...

Table 5-2. System Clocks and Gating (continued)

Module	Module Clock (instance.clock)	Clock Root	Module Clock Gating Enable (CCGR)
	pcie2_clk_rst.auxclk	ccm_pcie_aux_clk_root	clk_enable_pcie2 (CCGR100)
	pcie2_clk_rst.mstr_axi_clk	ccm_main_axi_clk_root	clk_enable_pcie2 (CCGR100)
	pcie2_clk_rst.slv_axi_clk	ccm_main_axi_clk_root	clk_enable_pcie2 (CCGR100)
	pcie2_ctrl.mstr_aclk	ccm_main_axi_clk_root	clk_enable_pcie2 (CCGR100)
	pcie2_ctrl.slv_aclk	ccm_main_axi_clk_root	clk_enable_pcie2 (CCGR100)
	pcie2_exsc.aclk_exsc	ccm_main_axi_clk_root	clk_enable_pcie2 (CCGR100)
	pcie2_exsc.ipg_clk	ccm_ipg_clk_root	clk_enable_pcie2 (CCGR100)
	pcie2_mem.mstr_axi_clk	ccm_main_axi_clk_root	clk_enable_pcie2 (CCGR100)
	pcie2_mem.slv_axi_clk	ccm_main_axi_clk_root	clk_enable_pcie2 (CCGR100)
PCIE_PHY	pcie_phy.ref_alt_clk_p	ccm_pcie_phy_clk_root	1
	pcie2_phy.ref_alt_clk_p	ccm_pcie2_phy_clk_root	1
PERFMON	perfmon1.apb_clk	ccm_ipg_clk_root	clk_enable_perfmon1 (CCGR38)
	perfmon1.axi0_ACLK	ccm_main_axi_clk_root	clk_enable_perfmon1 (CCGR38)
	perfmon2.apb_clk	ccm_ipg_clk_root	clk_enable_perfmon2 (CCGR39)
	perfmon2.axi0_ACLK	ccm_main_axi_clk_root	clk_enable_perfmon2 (CCGR39)
PWM	pwm1.ipg_clk	ccm_pwm1_clk_root	clk_enable_pwm1 (CCGR40)
	pwm1.ipg_clk_high_freq	ccm_pwm1_clk_root	clk_enable_pwm1 (CCGR40)
	pwm1.ipg_clk_s	ccm_pwm1_clk_root	clk_enable_pwm1 (CCGR40)
	pwm2.ipg_clk	ccm_pwm2_clk_root	clk_enable_pwm2 (CCGR41)
	pwm2.ipg_clk_high_freq	ccm_pwm2_clk_root	clk_enable_pwm2 (CCGR41)
	pwm2.ipg_clk_s	ccm_pwm2_clk_root	clk_enable_pwm2 (CCGR41)
	pwm3.ipg_clk	ccm_pwm3_clk_root	clk_enable_pwm3 (CCGR42)
	pwm3.ipg_clk_high_freq	ccm_pwm3_clk_root	clk_enable_pwm3 (CCGR42)
	pwm3.ipg_clk_s	ccm_pwm3_clk_root	clk_enable_pwm3 (CCGR42)
	pwm4.ipg_clk	ccm_pwm4_clk_root	clk_enable_pwm4 (CCGR43)
	pwm4.ipg_clk_high_freq	ccm_pwm4_clk_root	clk_enable_pwm4 (CCGR43)
	pwm4.ipg_clk_s	ccm_pwm4_clk_root	clk_enable_pwm4 (CCGR43)
QSPI	qspi.ahb_clk	ccm_ahb_clk_root	clk_enable_qspi (CCGR47)
	qspi.ipg_clk	ccm_ipg_clk_root	clk_enable_qspi (CCGR47)
	qspi.ipg_clk_4xsif	ccm_qspi_clk_root	clk_enable_qspi (CCGR47)
	qspi.ipg_clk_s	ccm_ipg_clk_root	clk_enable_qspi (CCGR47)
	qspi_sec.ipg_clk	ccm_ipg_clk_root	clk_enable_qspi (CCGR47)
	qspi_sec.ipg_clk_s	ccm_ipg_clk_root	clk_enable_qspi (CCGR47)
	qspi_sec.mst_hclk	ccm_ahb_clk_root	clk_enable_qspi (CCGR47)
RDC	rdc.ipg_clk_s	ccm_ipg_clk_root	clk_enable_rdc (CCGR49)
	rdc.ipg_clk	ccm_ipg_clk_root	clk_enable_rdc (CCGR49)

Table continues on the next page...

Table 5-2. System Clocks and Gating (continued)

Module	Module Clock (instance.clock)	Clock Root	Module Clock Gating Enable (CCGR)
	rdc_mem.ipg_clk	ccm_ipg_clk_root	clk_enable_rdc (CCGR49)
ROMCP	romcp.hclk	ccm_ahb_clk_root	clk_enable_rom (CCGR50)
	romcp.hclk_reg	ccm_ipg_clk_root	clk_enable_rom (CCGR50)
	romcp_mem.rom_CLK	ccm_ahb_clk_root	clk_enable_rom (CCGR50)
	romcp_sec.mst_hclk	ccm_ahb_clk_root	clk_enable_rom (CCGR50)
SAI	sai1.ipg_clk	ccm_audio_ipg_clk_root	clk_enable_sai1 (CCGR51)
	sai1.ipg_clk_s	ccm_audio_ipg_clk_root	clk_enable_sai1 (CCGR51)
	sai1.ipg_clk_sai_mclk_1	ccm_sai1_clk_root	clk_enable_sai1 (CCGR51)
	sai1.ipt_clk_sai_bclk	ccm_sai1_clk_root	clk_enable_sai1 (CCGR51)
	sai1.ipt_clk_sai_bclk_b	ccm_sai1_clk_root	clk_enable_sai1 (CCGR51)
	sai2.ipg_clk	ccm_ipg_clk_root	clk_enable_sai2 (CCGR52)
	sai2.ipg_clk_s	ccm_ipg_clk_root	clk_enable_sai2 (CCGR52)
	sai2.ipg_clk_sai_mclk_1	ccm_sai2_clk_root	clk_enable_sai2 (CCGR52)
	sai2.ipt_clk_sai_bclk	ccm_sai2_clk_root	clk_enable_sai2 (CCGR52)
	sai2.ipt_clk_sai_bclk_b	ccm_sai2_clk_root	clk_enable_sai2 (CCGR52)
	sai3.ipg_clk	ccm_ipg_clk_root	clk_enable_sai3 (CCGR53)
	sai3.ipg_clk_s	ccm_ipg_clk_root	clk_enable_sai3 (CCGR53)
	sai3.ipg_clk_sai_mclk_1	ccm_sai3_clk_root	clk_enable_sai3 (CCGR53)
	sai3.ipt_clk_sai_bclk	ccm_sa3_clk_root	clk_enable_sai3 (CCGR53)
	sai3.ipt_clk_sai_bclk_b	ccm_sa3_clk_root	clk_enable_sai3 (CCGR53)
	sai4.ipg_clk	ccm_audio_ipg_clk_root	clk_enable_sai4 (CCGR54)
	sai4.ipg_clk_s	ccm_audio_ipg_clk_root	clk_enable_sai4 (CCGR54)
	sai4.ipg_clk_sai_mclk_1	ccm_sai4_clk_root	clk_enable_sai4 (CCGR54)
	sai4.ipt_clk_sai_bclk	ccm_sai4_clk_root	clk_enable_sai4 (CCGR54)
	sai4.ipt_clk_sai_bclk_b	ccm_sai4_clk_root	clk_enable_sai4 (CCGR54)
	sai5.ipg_clk	ccm_audio_ipg_clk_root	clk_enable_sai5 (CCGR55)
	sai5.ipg_clk_s	ccm_audio_ipg_clk_root	clk_enable_sai5 (CCGR55)
	sai5.ipg_clk_sai_mclk_1	ccm_sai5_clk_root	clk_enable_sai5 (CCGR55)
	sai5.ipt_clk_sai_bclk	ccm_sai5_clk_root	clk_enable_sai5 (CCGR55)
	sai5.ipt_clk_sai_bclk_b	ccm_sai5_clk_root	clk_enable_sai5 (CCGR55)
	sai6.ipg_clk	ccm_audio_ipg_clk_root	clk_enable_sai6 (CCGR56)
	sai6.ipg_clk_s	ccm_audio_ipg_clk_root	clk_enable_sai6 (CCGR56)
	sai6.ipg_clk_sai_mclk_1	ccm_sai6_clk_root	clk_enable_sai6 (CCGR56)
	sai6.ipt_clk_sai_bclk	ccm_sai6_clk_root	clk_enable_sai6 (CCGR56)
	sai6.ipt_clk_sai_bclk_b	ccm_sai6_clk_root	clk_enable_sai6 (CCGR56)
SCTR	sctr.ipg_clk	ccm_ipg_clk_root	clk_enable_sctr (CCGR57)
	sctr.ipg_clk_s	ccm_ipg_clk_root	clk_enable_sctr (CCGR57)
	sctr.scan_clk	anamix_osc_25m_clk	clk_enable_sctr (CCGR57)
	sctr.sys_ctr_base_clk	anamix_osc_25m_clk	clk_enable_sctr (CCGR57)

Table continues on the next page...

Table 5-2. System Clocks and Gating (continued)

Module	Module Clock (instance.clock)	Clock Root	Module Clock Gating Enable (CCGR)
SDMA	sdma1.ips_hostctrl_clk	ccm_ipg_clk_root	clk_enable_sdma1 (CCGR58)
	sdma1.sdma_ap_ahb_clk	ccm_ahb_clk_root	clk_enable_sdma1 (CCGR58)
	sdma1.sdma_core_clk	ccm_ipg_clk_root	clk_enable_sdma1 (CCGR58)
	sdma2.ips_hostctrl_clk	ccm_audio_ipg_clk_root	clk_enable_sdma2 (CCGR59)
	sdma2.sdma_ap_ahb_clk	ccm_audio_ahb_clk_root	clk_enable_sdma2 (CCGR59)
	sdma2.sdma_core_clk	ccm_audio_ipg_clk_root	clk_enable_sdma2 (CCGR59)
SEC	sec_wrapper.clk	ccm_ipg_clk_root	clk_enable_sec_debug (CCGR60)
SEMA42	sema1.clk	ccm_ipg_clk_root	clk_enable_sema1 (CCGR61)
	sema2.clk	ccm_ipg_clk_root	clk_enable_sema2 (CCGR62)
SIM	sim_display.cm4clk	ccm_arm_m4_clk_root	GPC controlled
	sim_display.mainclk	ccm_main_axi_clk_root	clk_enable_sim_display (CCGR63)
	sim_display.mainclk_r	ccm_main_axi_clk_root	clk_enable_sim_display (CCGR63)
	sim_enet.mainclk	ccm_enet_axi_clk_root	clk_enable_sim_enet (CCGR64)
	sim_enet.mainclk_r	ccm_enet_axi_clk_root	clk_enable_sim_enet (CCGR64)
	sim_m.mainclk	ccm_ahb_clk_root	clk_enable_sim_m (CCGR65)
	sim_m.mainclk_r	ccm_ahb_clk_root	clk_enable_sim_m (CCGR65)
	sim_m.usdhcclk	ccm_nand_usdhc_bus_clk_root	clk_enable_sim_m (CCGR65)
	sim_m.usdhcclk_r	ccm_nand_usdhc_bus_clk_root	clk_enable_sim_m (CCGR65)
	sim_main.cm4clk	ccm_arm_m4_clk_root	GPC controlled
	sim_main.enetclk	ccm_enet_axi_clk_root	clk_enable_sim_enet (CCGR64)
	sim_main.mainclk	ccm_main_axi_clk_root	clk_enable_sim_main (CCGR66)
	sim_main.mainclk_r	ccm_main_axi_clk_root	clk_enable_sim_main (CCGR66)
	sim_main.per_mclk	ccm_ahb_clk_root	clk_enable_sim_m (CCGR65)
	sim_main.per_sclk	ccm_ahb_clk_root	clk_enable_sim_s (CCGR67)
	sim_main.usdhcclk	ccm_nand_usdhc_bus_clk_root	clk_enable_sim_m (CCGR65)
	sim_main.wakeupclk	ccm_ahb_clk_root	clk_enable_sim_wakeup (CCGR68)
	sim_s.apbhdmaclk	ccm_nand_usdhc_bus_clk_root	clk_enable_rawnand (CCGR48)
	sim_s.gpv4clk	ccm_enet_axi_clk_root	clk_enable_sim_enet (CCGR64)
	sim_s.mainclk	ccm_ahb_clk_root	clk_enable_sim_s (CCGR67)

Table continues on the next page...

Table 5-2. System Clocks and Gating (continued)

Module	Module Clock (instance.clock)	Clock Root	Module Clock Gating Enable (CCGR)
	sim_s.mainclk_r	ccm_ahb_clk_root	clk_enable_sim_s (CCGR67)
	sim_s.weimclk	ccm_ahb_clk_root	1
	sim_wakeup.mainclk	ccm_ahb_clk_root	clk_enable_sim_wakeup (CCGR68)
	sim_wakeup.mainclk_r	ccm_ahb_clk_root	clk_enable_sim_wakeup (CCGR68)
	pl301_audio.displayclk	ccm_main_axi_clk_root	clk_enable_sim_display (CCGR63)
SNVS	snvs_hs_wrapper.ipg_clk	ccm_ipg_clk_root	clk_enable_snvs (CCGR71)
	snvs_hs_wrapper.ipg_clk_s	ccm_ipg_clk_root	clk_enable_snvs (CCGR71)
	snvsmix.ipg_clk_root	ccm_ipg_clk_root	1
SPBA	spba1.ipg_clk	ccm_ipg_clk_root	clk_enable_ipmux3 (CCGR30)
	spba1.ipg_clk_s	ccm_ipg_clk_root	clk_enable_ipmux3 (CCGR30)
	spba2.ipg_clk	ccm_audio_ipg_clk_root	1
	spba2.ipg_clk_s	ccm_audio_ipg_clk_root	1
SPDIF	spdif1.extal_clk	anamix_osc_25m_clk	1
	spdif1.gclkw_t0	ccm_spdif1_clk_root	1
	spdif1.ipg_clk_s	ccm_ipg_clk_root	1
	spdif1.tx_clk	ccm_spdif1_clk_root	1
	spdif1.tx_clk1	ccm_spdif1_clk_root	1
	spdif1.tx_clk2	spdif1_ext_clk_root	1
	spdif1.tx_clk3	ccm_spdif1_clk_root	1
	spdif1.tx_clk4	ccm_spdif1_clk_root	1
	spdif1.tx_clk5	ccm_spdif1_clk_root	1
	spdif2.extal_clk	anamix_osc_25m_clk	1
	spdif2.gclkw_t0	ccm_spdif2_clk_root	1
	spdif2.ipg_clk_s	ccm_ipg_clk_root	1
	spdif2.tx_clk	ccm_spdif2_clk_root	1
	spdif2.tx_clk1	ccm_spdif2_clk_root	1
	spdif2.tx_clk2	ccm_spdif2_clk_root	1
	spdif2.tx_clk3	ccm_spdif2_clk_root	1
	spdif2.tx_clk4	ccm_spdif2_clk_root	1
	spdif2.tx_clk5	ccm_spdif2_clk_root	1
TRACE	coresight.DBGCLK	main_axi_clk_root	clk_enable_trace (CCGR72)
	coresight.traceclk	ccm_trace_clk_root	clk_enable_trace (CCGR72)
	coresight_mem.cs_etf_clk	main_axi_clk_root	clk_enable_trace (CCGR72)
UART	uart1.ipg_clk	ccm_ipg_clk_root	clk_enable_uart1 (CCGR73)
	uart1.ipg_clk_s	ccm_ipg_clk_root	clk_enable_uart1 (CCGR73)

Table continues on the next page...

Table 5-2. System Clocks and Gating (continued)

Module	Module Clock (instance.clock)	Clock Root	Module Clock Gating Enable (CCGR)
	uart1.ipg_perclk	ccm_uart1_clk_root	clk_enable_uart1 (CCGR73)
	uart2.ipg_clk	ccm_ipg_clk_root	clk_enable_uart2 (CCGR74)
	uart2.ipg_clk_s	ccm_ipg_clk_root	clk_enable_uart2 (CCGR74)
	uart2.ipg_perclk	ccm_uart2_clk_root	clk_enable_uart2 (CCGR74)
	uart3.ipg_clk	ccm_ipg_clk_root	clk_enable_uart3 (CCGR75)
	uart3.ipg_clk_s	ccm_ipg_clk_root	clk_enable_uart3 (CCGR75)
	uart3.ipg_perclk	ccm_uart3_clk_root	clk_enable_uart3 (CCGR75)
	uart4.ipg_clk	ccm_ipg_clk_root	clk_enable_uart4 (CCGR76)
	uart4.ipg_clk_s	ccm_ipg_clk_root	clk_enable_uart4 (CCGR76)
	uart4.ipg_perclk	ccm_uart4_clk_root	clk_enable_uart4 (CCGR76)
USB	usb.clk	ccm_ipg_clk_root	clk_enable_usb (CCGR22)
USDHC	usdhc1.hclk	ccm_nand_usdhc_bus_clk_root	clk_enable_usdhc1 (CCGR81)
	usdhc1.ipg_clk	ccm_ipg_clk_root	clk_enable_usdhc1 (CCGR81)
	usdhc1.ipg_clk_perclk	ccm_usdhc1_clk_root	clk_enable_usdhc1 (CCGR81)
	usdhc1.ipg_clk_s	ccm_ipg_clk_root	clk_enable_usdhc1 (CCGR81)
	usdhc2.hclk	ccm_nand_usdhc_bus_clk_root	clk_enable_usdhc2 (CCGR82)
	usdhc2.ipg_clk	ccm_ipg_clk_root	clk_enable_usdhc2 (CCGR82)
	usdhc2.ipg_clk_perclk	ccm_usdhc2_clk_root	clk_enable_usdhc2 (CCGR82)
	usdhc2.ipg_clk_s	ccm_ipg_clk_root	clk_enable_usdhc2 (CCGR82)
WDOG	wdog1.ipg_clk	ccm_wdog_clk_root	clk_enable_wdog1 (CCGR83)
	wdog1.ipg_clk_s	ccm_wdog_clk_root	clk_enable_wdog1 (CCGR83)
	wdog2.ipg_clk	ccm_wdog_clk_root	clk_enable_wdog2 (CCGR84)
	wdog2.ipg_clk_s	ccm_wdog_clk_root	clk_enable_wdog2 (CCGR84)
	wdog3.ipg_clk	ccm_wdog_clk_root	clk_enable_wdog3 (CCGR85)
	wdog3.ipg_clk_s	ccm_wdog_clk_root	clk_enable_wdog3 (CCGR85)

5.1.5 Functional Description

The following sections describe the functional details of CCM.

5.1.5.1 Input Clocks

The table below describes the input clock sources that supply the muxes to the clock slices in the clock root generator. Please see [PLL Interface](#) for PLL programming information.

Control Register (CCM_PLL_CTRLn)	Input Clock	Frequency (MHz)	Description
12	ARM_PLL_CLK	1600	Arm PLL
13	GPU_PLL_CLK	1600	GPU PLL clock output
14	VPU_PLL_CLK	800	VPU PLL clock output
15	DRAM_PLL1_CLK	800	DDR PLL
16	SYSTEM_PLL1_CLK	800	System PLL1 output clock
17	SYSTEM_PLL1_DIV2	400	System PLL1 divided 2 clock output
18	SYSTEM_PLL1_DIV3	266	System PLL1 divided 3 clock output
19	SYSTEM_PLL1_DIV4	200	System PLL1 divided 4 clock output
20	SYSTEM_PLL1_DIV5	160	System PLL1 divided 5 clock output
21	SYSTEM_PLL1_DIV6	133	System PLL1 divided 6 clock output
22	SYSTEM_PLL1_DIV8	100	System PLL1 divided 8 clock output
23	SYSTEM_PLL1_DIV10	80	System PLL1 divided 10 clock output
24	SYSTEM_PLL1_DIV20	40	System PLL divided 20 clock output
25	SYSTEM_PLL2_CLK	1000	System PLL2 output clock
26	SYSTEM_PLL2_DIV2	500	System PLL2 divided 2 clock output
27	SYSTEM_PLL2_DIV3	333	System PLL2 divided 3 clock output
28	SYSTEM_PLL2_DIV4	250	System PLL2 divided 4 clock output
29	SYSTEM_PLL2_DIV5	200	System PLL2 divided 5 clock output
30	SYSTEM_PLL2_DIV6	166	System PLL2 divided 6 clock output
31	SYSTEM_PLL2_DIV8	125	System PLL2 divided 8 clock output
32	SYSTEM_PLL2_DIV10	100	System PLL2 divided 10 clock output
33	SYSTEM_PLL2_DIV20	50	System PLL2 divided 20 clock output
34	SYSTEM_PLL3_CLK	1000	System PLL3 output clock
35	AUDIO_PLL1_CLK	650	Audio PLL1 clock output
36	AUDIO_PLL2_CLK	650	Audio PLL2 clock output
37	VIDEO_PLL1_CLK	650	Video PLL1 clock output
38	VIDEO_PLL2_CLK	600	Video PLL2 clock output
external source	32K_REF_CLK	0.032	32K oscillator clock output
external source	25M_REF_CLK	25	25M oscillator clock output
external source	27M_REF_CLK	27	27M oscillator clock output
external source	EXT_CLK_1	133	Clock input from external IO
external source	EXT_CLK_2	133	Clock input from external IO
external source	EXT_CLK_3	133	Clock input from external IO
external source	EXT_CLK_4	133	Clock input from external IO

NOTE

CKIL_SYNC needs to be configured the same as the PLL for ipg_clk

5.1.5.2 CKIL Synchronizer

CCM provides a synchronized version of the 32K clock called CKIL Synchronizer (CKIL_SYNC). The CKIL_SYNC clock is generated and synchronized by the IPG_CLK_ROOT when IPG_CLK_ROOT is active.

When the system enters low-power mode that requires the shutdown of IPG_CLK_ROOT, the CKIL Synchronizer needs to be bypassed and fed directly from the XTALOSC 32K source.

The control for the CKIL_SYNC bypass comes from source control. The control for CKIL_SYNC bypass needs to be configured exactly the same as the controls for the PLL output that is used to generate IPG_CLK_ROOT.

In cases where a module ipg_clk is not tied to IPG_CLK_ROOT, the bypass signal to the ipg_clk is controlled by LPCG. To avoid issues on their 32K input, the LPCG must also be shutdown at the same time as the PLL clock output for that particular ipg_clk generation.

5.1.5.3 Clock Components

The details of the CCM clock components are detailed below.

5.1.5.3.1 Clock Divider

Synchronized clock dividers can be used to perform integer division on the source clock frequency. The divider guarantees a clean clock signal on its output during the change of the divide factor. Dividers perform a $1/(N+1)$ divide. A glitch can cause a sync-divider to go into an unrecoverable state, therefore a clean clock signal must be provided to a sync-divider.

When updating the divider value, a read enable signal (read_en) deasserts to stop the related control registers from fetching the divider value. To change a divider factor value, the bus enters a transition state and the divide logic latches the divider factor value. After this value is latched, the read_en is reasserted. The divider will begin to update its internal divide logic. After it finishes updating, the divider sends an acknowledge signal (sw_ack) to indicate a new value is in effect.

5.1.5.3.2 Clock Switching Multiplexer

Clock switching multiplexers can guarantee a clean clock signal when switching between 2 clock sources. Both clock inputs must be active when switching. Glitches in the selected source may be transferred to the output clock while the de-selected source is blocked. Glitches that occur during switching may cause an unpredictable state, and will recover in 2~3 cycles.

Clock switching multiplexers first shutdown the current active clock, then switch. After receiving an acknowledge that the current clock stopped. The multiplexer turns on the new selected clock source. As the acknowledge of turning on the new clock source is received, the source switch finishes.

5.1.5.3.3 Clock Gate

A clock gate cell is used to gate clocks. When gated, the clock will stop at 0. A clock gate cell can accept glitches on its clock inputs. If the gate cell is off, it will block or absorb glitches. If the gate cell is on, it may pass glitches to its output. A gate cell needs 2~3 clock cycles to change states. The state during this period is either on or off and will recover in 2~3 cycles, but cannot be predicted which state it is in during this period.

5.1.5.3.4 8 to 1 Multiplexer

The 8-to-1 multiplexer is a combinational multiplexer that can switch anytime. The multiplexer output does not guarantee a clean clock signal. During switching, the output must be gated or all 8 inputs will require the same value.

5.1.5.4 Clock Slices

There are several types of clock generation slices in CCM. The slices are categorized as Core, Bus, Peripheral (IP), and DRAM.

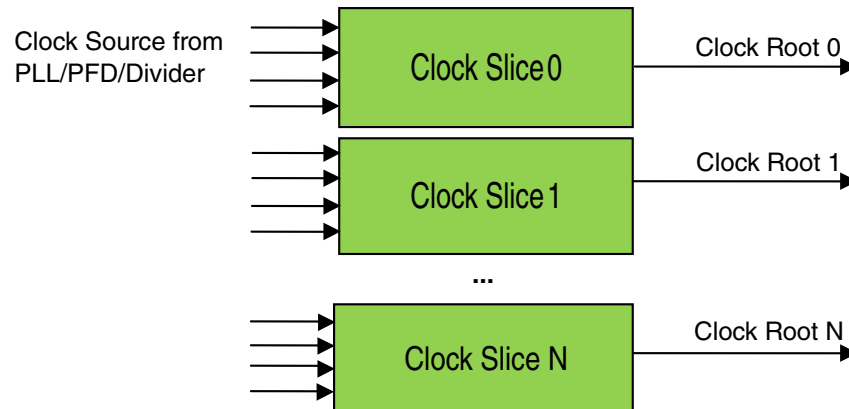


Figure 5-4. Clock Slices

5.1.5.4.1 Core clock slice

Core clock slices are designed for high speed, non-stop clock generation, typically for an Arm core. A core clock slice is comprised of a post divider and clock switching multiplexer. Each has a clock gate and a clock multiplexer inside. To run at high frequency, post divider is 3 bits, which is half the bit width of other slices. The two 8-to-1 multiplexers are not glitch-less, so switching them should only be done when their output is not routed to a clock output.

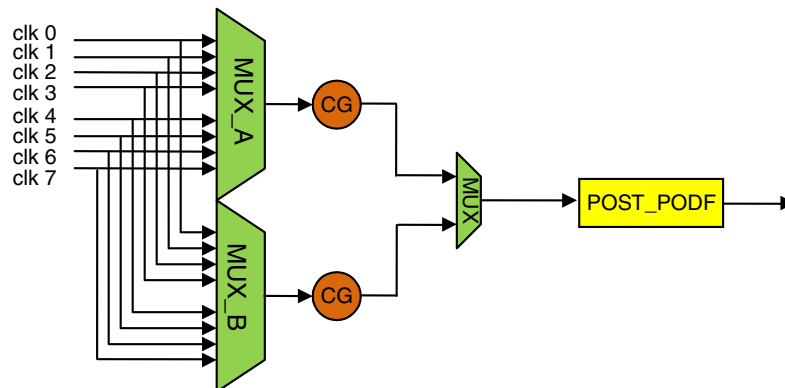


Figure 5-5. Core clock slice

5.1.5.4.2 Bus clock slice

Bus clock slices are comprised of a post divider and a clock switching multiplexer. Each has a pre-divider, clock gate and a clock multiplexer inside. The pre-divider is three bits and provides a maximum division factor of 8. The post-divider is six bits and can divide

down the clock input to 1MHz on clock output. The two 8-to-1 multiplexers are not glitch-less, so switching should only be done when their output is not routed to clock output.

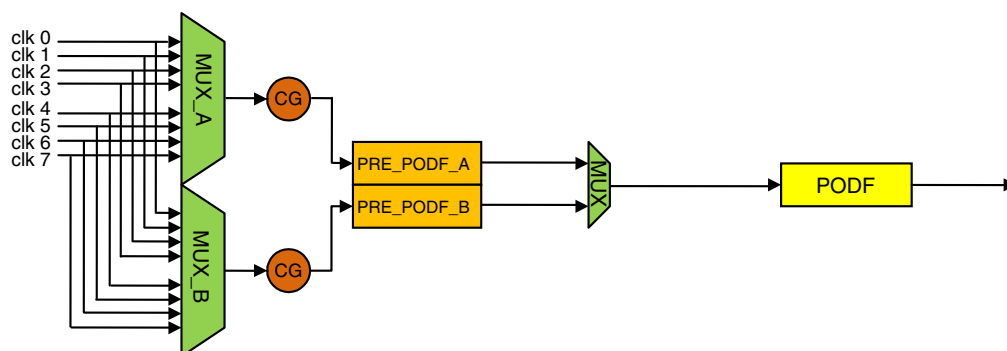


Figure 5-6. Bus clock slice

5.1.5.4.3 Peripheral clock slice

IP peripheral clock slices are comprised of a post-divider, pre-divider, clock gate and a clock multiplexer. The pre-divider is three bit and can divide down by a factor of 8. The post-divider is six bit and can divide down the input clock to 1MHz on clock output. The 8-to-1 multiplexer is not glitch-less, so switching should only be done when their output is not routed to clock output.

IP clock slices must be stopped to change the clock source.

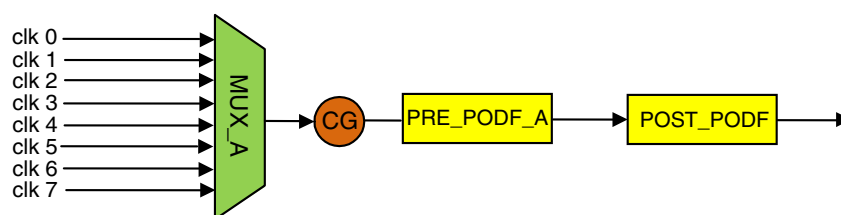


Figure 5-7. Peripheral clock slice

5.1.5.4.4 SSCG and Fractional PLLs

The SSCG and Fractional-N PLLs are detailed below. The SSCG block diagram is shown below. Please see CCM ANALOG for more information.

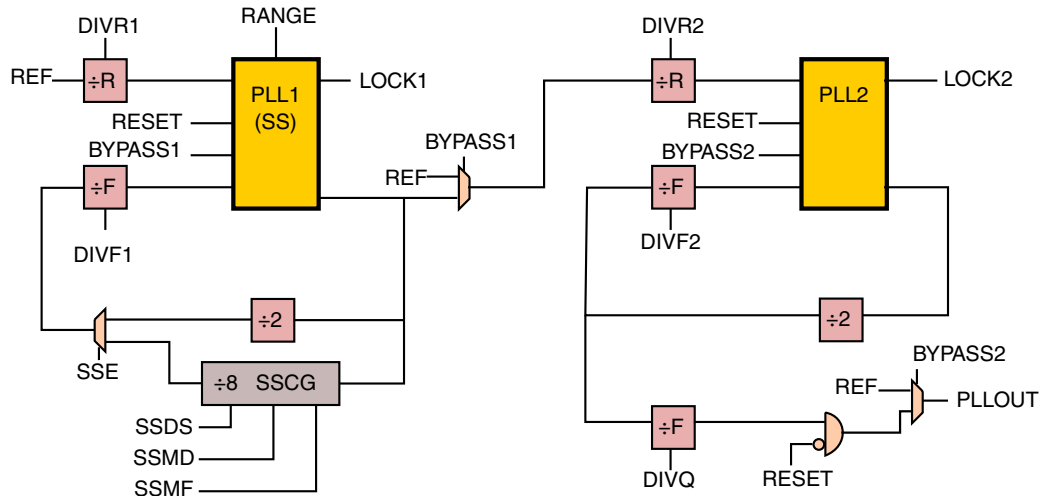


Figure 5-8. SSCG PLL Block Diagram

Formula for SSCG PLLOUT:

- SSE=0: PLLOUT = REF/DIVR1 * 2 * DIVF1/DIVR2 * DIVF2/DIVQ
- SSE=1: PLLOUT = REF/DIVR1 * 8 * DIVF1/DIVR2 * DIVF2/DIVQ

The ARM PLL, GPU PLL, VPU PLL, Audio PLL1/2, and Video PLL1 are fractional PLLs. The frequency on these can be tuned to be very accurate to meet audio and video interface requirements.

The figure below shows the Fractional PLL block diagram.

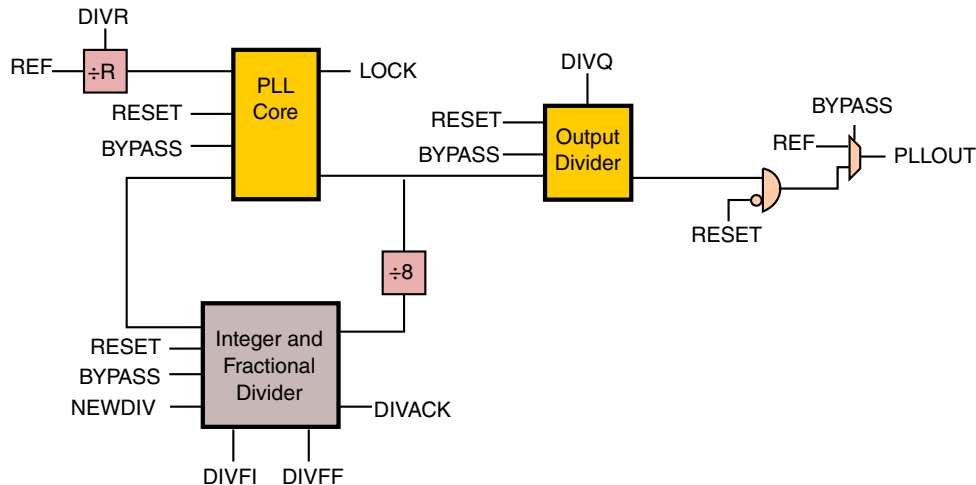


Figure 5-9. Fractional PLL Block Diagram

Formula for Fraction PLLOUT:

- $PLL_{OUT} = REF / DIVR_VAL * 8 * DIVF_VAL / DIVQ_VAL$
- Where, $DIVF_VAL = 1 + DIVFI + (DIVFF / 2^{24})$

5.1.5.5 Clock gate control

CCM can perform automatic clock shutdown of on-chip peripherals according to system low power mode. Each logic domain can respectively declare its dependency level on each clock. If a clock is detected that is not dependant on any domain, it will be shutdown to save power.

Clock generation inside the CCM creates a clock root for on-chip peripherals. Before the clock root goes into peripherals via low power clock gating cells. By controlling these LPCGs, CCM can manage on-chip peripheral clocks.

Clock gate controls use active clock gating, which means the low power clock gating (LPCG) requires an active clock root. The clock generation module only performs multiplexing, gating and dividing on clock sources. Therefore, the clock root from the generation module will stop when the corresponding clock source stops.

The ENABLE bit must be set for the clock root that LPCG is actively gating.

5.1.5.6 Clock source control

CCM can perform automatic PLL shutdown of CCM_ANALOG according to system low power mode. Each logic domain can respectively declare its dependency level on each clock. If a PLL is detected not depended by any domain, it will be shutdown to save power.

PLL in ANATOP can be set be controlled by CCM via setting some CCM override bits. Also there is control on 32K IPG_CLK_ROOT synchronized version inside clock source control module.

For PLLs, we have channel on every PLL, every PFD and every divider. PLL is the source of PFDs and dividers. For any clock, its source must be left on when it is kept on. Behavior is undefined if this rule is violated.

For a shutdown clock source, if it is declared depended by writing clock source control registers, the controlling logic will turn on the source immediately, while the setting goes into a shadow register. After clock source get ready, the setting will be accepted by source control logic, and copied from shadow register to setting register.

5.1.5.7 Access control

CCM can implement its own access control based on domain to provide more precise control on shared resources. Access controls are implemented on clock root generation, clock gate control, and clock source control. Access control logic does not impact read access, but blocks unauthentic write access.

Access controls on clock root generation are independent between every clock root. A sticky authentic fail flag is set when a domain writes to a register and authentication fails. The access control logic contains a whitelist and a semaphore. By default, each clock root's access control logic is disabled after power-on reset. Software can enable access control anytime after reset.

NOTE

Once access logic is enabled, it cannot be disabled until the next power-on reset.

Table 5-3. Whitelist

Enabled	Write access will be authenticated before being performed.
Disabled	every access on protected item will be performed.

NOTE

Only domains that are on the whitelist can perform write access to this clock root when access control is enabled.

Table 5-4. Semaphore

Enabled	A domain must obtain the semaphore's ownership before its write access can be authenticated. Only a domain on the whitelist can obtain the ownership and the ownership will last until the domain explicitly releases the ownership. Semaphore obtain will fail if it is already fetched by some other domain.
Disabled	Authentic check will check only on whitelist.

NOTE

Semaphore is intended to help software keep the clock root from unexpectedly changing.

Access control of clock gate and clock source control is performed in a simple operation. Every domain can only write on the bits for it's own setting. Any write to irrespective domain will be ignored.

5.1.5.8 System level considerations

Clock shutdown strategy

Any clock shutdown should first shutdown the LPCG, then the PLL. If the LPCG is configured not to shutdown, the clock root for the LPCG should not stop either. Violating this rule leads the system into an unpredictable state.

Core clock root frequency

If the core clock is set lower than one third of the IPG clock, SRC needs to generate a longer reset signal to match the requirement from the Arm core. This typically happens when the Arm core runs at some divided value of the XTAL 24M while IPG clock is supplied by the PLL.

DRAM clock

The DRAM PHYM clock needs a clock frequency faster than 400MHz.

USB OTG CLOCK

The USB clock may not be a reliable clock source in some applications. This clock may stop when USB cable is disconnected.

5.1.6 Programming Guide

5.1.6.1 Set, Clear, and Toggle register features

Every register of the CCM has set, clear, and toggle features.

The set feature for a given register is located at:

Base Address of the register + 0x04

The clear feature for a given register is located at:

Base Address of the register + 0x08

The toggle feature is located at:

Base Address of the register + 0x0c

Read from all 4 locations to get the current register value. Writing to the base register bits sets the register to the write value. Writing 1 to the set register bits sets them to 1, while writing 0 has no effect. Writing 1 to the clear register bits clears them to 0, while writing 0 has no effect. Writing 1 to the toggle register bits sets them to invert the value, while writing 0 has no effect.

5.1.6.2 PLL Interface

CCM can control PLLs inside CCM_ANALOG when entering or leaving low-power mode. Software must set the PLL override inside CCM_ANALOG before entering low-power mode after power-on reset (POR).

There are four levels of low-power modes in a logic domain:

- Not needed
- Needed in RUN
- Needed in RUN and WAIT
- Needed in RUN, WAIT, and STOP

CCM only takes action while domain status are switching between STOP (DEEP SLEEP mode is considered the same as STOP). There are 4 domains that can be assigned. Any CPU platform can be assigned to any domain by RDC. If a domain is empty, the domain is considered as STOP.

Each domain can declare its dependency to CCM. The use of any clock, without declaring it in its own domain, is not permitted. A domain declares its dependency on a clock by writing the dependency level. Settings against behavior in low-power mode are as follows:

Table 5-5. Domain Dependency

Domain Level	Run	Wait	Stop / Deep sleep
0			
1	Required		
2	Required	Required	
3	Required	Required	Required

Table 5-6. CCGR Program Interface

CC GR		Domain3				Domain2				Domain1				Domain0			
	31-16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				Level1	Level0			Level1	Level0			Level1	Level0			Level1	Level0

Each domain can change only bits assigned to control access. Any irrelevant write to it will be ignored. For example, Domain 0 can only write to bits [1:0]. Any bits written to, other than bits [1:0], will be ignored. Other domains can read all of the other domain settings. The default value for domain 0 is 2, and will enter STOP mode after shutting down. When the default value of the other domain setting is 0, it will not be required.

When setting clock source, the settings will not take effect immediately. The setting will enter the shadow register first. If a PLL shutdown or new setting enters the shadow register to declare dependency on the PLL, the PLL will turn on immediately. When the PLL is ready, the setting in shadow register will be updated to the new setting. During this period, the pending bit will be set and cleared. Then CCM will send the PLL control signal as a shadow register and inform GPC the PLL status according to the setting register. In other cases, the setting will be updated from the shadow register immediately. Clock sources have dependency on each other.

NOTE

Do not shutdown the parent clock when the required child clock is active. Attempting to do so will lead to unpredictable and unrecoverable behavior. It is recommended to shutdown the parent clock and child clock together.

5.1.6.3 CCGR Interface

Before a clock root goes to on-chip peripherals, the clock root is distributed through low power clock gates (LPCG). These LPCG are implemented to automatically perform clock shutdown when a domain enters and leaves a low-power state.

There are four levels of low-power modes in a logic domain:

- Not needed
- Needed in RUN
- Needed in RUN and WAIT
- Needed in RUN, WAIT, and STOP

CCM only takes action while domain status are switching between STOP (DEEP SLEEP mode is considered the same as STOP). There are 4 domains that can be assigned. Any CPU platform can be assigned to any domain by RDC. If a domain is empty, the domain is considered as STOP.

Each domain can declare its dependency to CCM. The use of any clock, without declaring it in its own domain, is not permitted. A domain declares its dependency on a clock by writing the dependency level. Settings against behavior in low-power mode are as follows:

Table 5-7. Domain Dependency

Domain Level	RUN	WAIT	STOP/ DEEP SLEEP
0			
1	Required		
2	Required	Required	

Table continues on the next page...

Table 5-7. Domain Dependency (continued)

3	Required	Required	Required
---	----------	----------	----------

Table 5-8. CCGR Program Interface

CC GR		Domain3				Domain2				Domain1				Domain0			
	31-16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				Level1	Level0			Level1	Level0			Level1	Level0			Level1	Level0

Each domain can change only bits assigned to control access. Any irrelevant write to it will be ignored. For example, Domain 0 can only write to bits [1:0]. Any bits written to, other than bits [1:0], will be ignored. Other domains can read all of the other domain settings. The default value for domain 0 is 2, and will enter STOP mode after shutting down. When the default value of the other domain setting is 0, it will not be required.

The table below lists the CCM Clock Gating Register (CCGR) and associated offset for each LPCG enable.

NOTE

Not all CCGRs are mapped.

Table 5-9. CCGR Mapping Table

Gating Register	LPCG Enable	Offset
CCM_CCGR0	DVFS	0x4000
CCM_CCGR1	Anamix	0x4010
CCM_CCGR2	CPU	0x4020
CCM_CCGR3	CSU	0x4030
CCM_CCGR4	Debug	0x4040
CCM_CCGR5	DDR1	0x4050
CCM_CCGR6	Reserved	0x4060
CCM_CCGR7	ECSPI1	0x4070
CCM_CCGR8	ECSPI2	0x4080
CCM_CCGR9	ECSPI3	0x4090
CCM_CCGR10	ENET1	0x40A0
CCM_CCGR11	GPIO1	0x40B0
CCM_CCGR12	GPIO2	0x40C0
CCM_CCGR13	GPIO3	0x40D0
CCM_CCGR14	GPIO4	0x40E0
CCM_CCGR15	GPIO5	0x40F0
CCM_CCGR16	GPT1	0x4100

Table continues on the next page...

Table 5-9. CCGR Mapping Table (continued)

Gating Register	LPCG Enable	Offset
CCM_CCGR17	GPT2	0x4110
CCM_CCGR18	GPT3	0x4120
CCM_CCGR19	GPT4	0x4130
CCM_CCGR20	GPT5	0x4140
CCM_CCGR21	GPT6	0x4150
CCM_CCGR22	HS	0x4160
CCM_CCGR23	I2C1	0x4170
CCM_CCGR24	I2C2	0x4180
CCM_CCGR25	I2C3	0x4190
CCM_CCGR26	I2C4	0x41A0
CCM_CCGR27	IOMUX	0x41B0
CCM_CCGR28	IOMUX1	0x41C0
CCM_CCGR29	IOMUX2	0x41D0
CCM_CCGR30	IOMUX3	0x41E0
CCM_CCGR31	IOMUX4	0x41F0
CCM_CCGR32	M4	0x4200
CCM_CCGR33	MU	0x4210
CCM_CCGR34	OCOTP	0x4220
CCM_CCGR35	OCRAM	0x4230
CCM_CCGR36	OCRAM_s	0x4240
CCM_CCGR37	PCIE	0x4250
CCM_CCGR38	PERFMON1	0x4260
CCM_CCGR39	PERFMON2	0x4270
CCM_CCGR40	PWM1	0x4280
CCM_CCGR41	PWM2	0x4290
CCM_CCGR42	PWM3	0x42A0
CCM_CCGR43	PWM4	0x42B0
CCM_CCGR44	QoS	0x42C0
CCM_CCGR45	Dismix	0x42D0
CCM_CCGR46	Megamix	0x42E0
CCM_CCGR47	QSPI	0x42F0
CCM_CCGR48	NAND (APBHDMA, GPPI, BCH)	0x4300
CCM_CCGR49	RDC	0x4310
CCM_CCGR50	ROM	0x4320
CCM_CCGR51	SAI1	0x4330
CCM_CCGR52	SAI2	0x4340
CCM_CCGR53	SAI3	0x4350
CCM_CCGR54	SAI4	0x4360
CCM_CCGR55	SAI5	0x4370

Table continues on the next page...

Table 5-9. CCGR Mapping Table (continued)

Gating Register	LPCG Enable	Offset
CCM_CCGR56	SAI6	0x4380
CCM_CCGR57	SCTR	0x4390
CCM_CCGR58	SDMA1	0x43A0
CCM_CCGR59	SDMA2	0x43B0
CCM_CCGR60	SEC_DEBUG	0x43C0
CCM_CCGR61	SEMA1	0x43D0
CCM_CCGR62	SEMA2	0x43E0
CCM_CCGR63	SIM_display	0x43F0
CCM_CCGR64	SIM_ENET	0x4400
CCM_CCGR65	SIM_m	0x4410
CCM_CCGR66	SIM_main	0x4420
CCM_CCGR67	SIM_s	0x4430
CCM_CCGR68	SIM_wakeup	0x4440
CCM_CCGR69	SIM_USB	0x4450
CCM_CCGR70	SIM_VPU	0x4460
CCM_CCGR71	SNVS	0x4470
CCM_CCGR72	Trace	0x4480
CCM_CCGR73	UART1	0x4490
CCM_CCGR74	UART2	0x44A0
CCM_CCGR75	UART3	0x44B0
CCM_CCGR76	UART4	0x44C0
CCM_CCGR77	USB_CTRL1	0x44D0
CCM_CCGR78	USB_CTRL2	0x44E0
CCM_CCGR79	USB_PHY1	0x44F0
CCM_CCGR80	USB_PHY2	0x4500
CCM_CCGR81	USDHC1	0x4510
CCM_CCGR82	USDHC2	0x4520
CCM_CCGR83	WDOG1	0x4530
CCM_CCGR84	WDOG2	0x4540
CCM_CCGR85	WDOG3	0x4550
CCM_CCGR86	VA53	0x4560
CCM_CCGR87	GPU	0x4570
CCM_CCGR88	HEVC	0x4580
CCM_CCGR89	AVC	0x4590
CCM_CCGR90	VP9	0x45A0
CCM_CCGR91	HEVC_inter	0x45B0
CCM_CCGR92	GIC	0x45C0
CCM_CCGR93	Display	0x45D0
CCM_CCGR94	HDMI	0x45E0

Table continues on the next page...

Table 5-9. CCGR Mapping Table (continued)

Gating Register	LPCG Enable	Offset
CCM_CCGR95	HDMI_phy	0x45F0
CCM_CCGR96	XTALOSC	0x4600
CCM_CCGR97	PLL	0x4610
CCM_CCGR98	TEMPSENSOR	0x4620
CCM_CCGR99	VPU_DEC	0x4630
CCM_CCGR100	PCIE2	0x4640
CCM_CCGR101	MIPI_CSI1	0x4650
CCM_CCGR102	MIPI_CSI2	0x4660

5.1.6.4 Target Interface

The Target Interface is optimized to simplify software operation. Using this interface, all clock roots are in the same program model with the same register bit field mapping. The software does not handle the details of the clock slice and clock slice types. Software writes the desired settings to the register, and the internal hardware logic generates a required sequence to achieve the desired settings.

The Target Interface requires the software to provide whether a clock is active, the clock source number to be selected, pre-divide value, and post-divide value. If a clock slice does not support a setting, that setting is simply ignored, and will not effect the supported fields.

$$\text{Freq} = (\text{clock source freq})/(\text{pre_div}+1)/(\text{post_div}+1)$$

The internal logic sequence of the Target Interface guarantees a clean clock on output without frequency overshoot. A requirement of the Target Interface's software is that the target clock source is active.

The Target Interface sequence begins by opening all clocks, applying highest divider value, switching to the new clock source, then decreasing divider value to the target frequency. If Shutdown is requested, it will be performed last.

The clock output is always active when using the Target Interface. For intermediate frequency requests, the Target Interface chooses the lowest frequency source to avoid frequency overshoot on the Peripheral clock slices. For Core and Bus clock slices, the clock switching multiplexer is used to guarantee smooth clock switching.

A write operation on a target interface completes once the output clock is running at the desired setting. Software polling is not necessary to determine clock stability.

Clock Control Module (CCM)

STEP	STATE	OPERATION
0	SMART_IDLE	Idle state, no write operation is pending
1	SMART_WAIT_READY	State occurs when a write access is received, wait for every field to be ready
2	SMART_APPLY_GATE1	Open all branches, all gates inside clock slices
3	SMART_WAIT_GATE1	Wait for gate applied
4	SMART_APPLY_PODF1	apply post divider and post divider for if new value generate slower clock
5	SMART_WAIT_PODF1	Wait for divider accept new value
6	SMART_APPLY_GATE2	shutdown spare branch if exist, else shutdown working branch
7	SMART_WAIT_GATE2	Wait for shutdown operation complete
8	SMART_APPLY_MUX	Change multiplexer to new source on spare branch if exist, else switch working one
9	SMART_APPLY_GATE3	open gates on all branches
10	SMART_WAIT_GATE3	Wait for gates opened
11	SMART_APPLY_SWITCH	Switch clock switching multiplexer if there is one
12	SMART_WAIT_SWITCH	Wait for clock switching multiplexer switch
13	SMART_APPLY_PODF2	apply post divider and post divider for if new value generate faster clock
14	SMART_WAIT_PODF2	Wait for divider accept new value
15	SMART_APPLY_GATE4	apply clock gate setting, shutdown spare one if there is
16	SMART_WAIT_GATE4	Wait for gate applied
17	SMART_APPLY_AUTO	apply auto and auto divider
18	SMART_DONE	Finish, wait for bus operation complete

5.1.6.5 Normal Interface

Normal interface provide more controllable thing that target interface. And also provide protections against dangerous operation.

Normal interface provides safe sequences to handle each clock component, divider, gate, multiplexer. But it is software that needs to care the order and relationship between updating components.

Writing to this interface will complete immediately, and internal logic will continue try to apply written values to clock components. A busy flag will be assert during applying.

Field access rule:

1. Only one field can be modified a time
2. No field can be modified when any field pending
3. Not violate change condition in following table

FIELD	CHANGE CONDITION	FINISH CONDITION
Auto		immediate
Auto-divider		immediate
Bypass	Gatea active and gateb active	Bypass switch complete
Post-divider		New divider value applied
Gate B	Bypass disable	New gate value active
Pre-divider B	Bypass disable and gateb not gated	New divider value applied
MUX B	Bypass disable and gateb gating	immediate
Gate A	Bypass	New gate value active
Pre-divider A	Bypass and gatea not gated	New divider value applied
MUX A	Bypass and gatea gating	immediate

- Error will be reported if access rules violated.
- Unsafe or ambiguous access will be ignored.

If a write access as blocked by normal interface, the write operation will be ignored. And a sticky bit “violate” will be set. The bit will last until software clears it explicitly. The violate bit is 4 bits inside CCM, each for a logic domain. Each domain can read and clear the bit for itself, the bits for other domain is neither visible nor clearable.

5.1.7 CCM Memory Map/Register Definition

The Memory Map below represents the full array for CCM.

NOTE

Not all mapped Clock Slices and CCGRs are tied to functional components.

Please see the following for the functional mapping tables and information:

- CCM_PLL_CTRL - [Input Clocks](#)
- CCM_TARGET_ROOT - [Clock Root Selects](#)
- CCM_CCGR - [CCGR Interface](#)

CCM memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_0000	General Purpose Register (CCM_GPR0)	32	R/W	0000_0000h	5.1.7.1/539
3038_0004	General Purpose Register (CCM_GPR0_SET)	32	R/W	0000_0000h	5.1.7.1/539
3038_0008	General Purpose Register (CCM_GPR0_CLR)	32	R/W	0000_0000h	5.1.7.1/539
3038_000C	General Purpose Register (CCM_GPR0_TOG)	32	R/W	0000_0000h	5.1.7.1/539

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_0800	CCM PLL Control Register (CCM_PLL_CTRL0)	32	R/W	0000_0002h	5.1.7.2/540
3038_0804	CCM PLL Control Register (CCM_PLL_CTRL0_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0808	CCM PLL Control Register (CCM_PLL_CTRL0_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_080C	CCM PLL Control Register (CCM_PLL_CTRL0_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0810	CCM PLL Control Register (CCM_PLL_CTRL1)	32	R/W	0000_0002h	5.1.7.2/540
3038_0814	CCM PLL Control Register (CCM_PLL_CTRL1_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0818	CCM PLL Control Register (CCM_PLL_CTRL1_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_081C	CCM PLL Control Register (CCM_PLL_CTRL1_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0820	CCM PLL Control Register (CCM_PLL_CTRL2)	32	R/W	0000_0002h	5.1.7.2/540
3038_0824	CCM PLL Control Register (CCM_PLL_CTRL2_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0828	CCM PLL Control Register (CCM_PLL_CTRL2_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_082C	CCM PLL Control Register (CCM_PLL_CTRL2_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0830	CCM PLL Control Register (CCM_PLL_CTRL3)	32	R/W	0000_0002h	5.1.7.2/540
3038_0834	CCM PLL Control Register (CCM_PLL_CTRL3_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0838	CCM PLL Control Register (CCM_PLL_CTRL3_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_083C	CCM PLL Control Register (CCM_PLL_CTRL3_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0840	CCM PLL Control Register (CCM_PLL_CTRL4)	32	R/W	0000_0002h	5.1.7.2/540
3038_0844	CCM PLL Control Register (CCM_PLL_CTRL4_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0848	CCM PLL Control Register (CCM_PLL_CTRL4_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_084C	CCM PLL Control Register (CCM_PLL_CTRL4_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0850	CCM PLL Control Register (CCM_PLL_CTRL5)	32	R/W	0000_0002h	5.1.7.2/540
3038_0854	CCM PLL Control Register (CCM_PLL_CTRL5_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0858	CCM PLL Control Register (CCM_PLL_CTRL5_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_085C	CCM PLL Control Register (CCM_PLL_CTRL5_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0860	CCM PLL Control Register (CCM_PLL_CTRL6)	32	R/W	0000_0002h	5.1.7.2/540
3038_0864	CCM PLL Control Register (CCM_PLL_CTRL6_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0868	CCM PLL Control Register (CCM_PLL_CTRL6_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_086C	CCM PLL Control Register (CCM_PLL_CTRL6_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0870	CCM PLL Control Register (CCM_PLL_CTRL7)	32	R/W	0000_0002h	5.1.7.2/540
3038_0874	CCM PLL Control Register (CCM_PLL_CTRL7_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0878	CCM PLL Control Register (CCM_PLL_CTRL7_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_087C	CCM PLL Control Register (CCM_PLL_CTRL7_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0880	CCM PLL Control Register (CCM_PLL_CTRL8)	32	R/W	0000_0002h	5.1.7.2/540
3038_0884	CCM PLL Control Register (CCM_PLL_CTRL8_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0888	CCM PLL Control Register (CCM_PLL_CTRL8_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_088C	CCM PLL Control Register (CCM_PLL_CTRL8_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0890	CCM PLL Control Register (CCM_PLL_CTRL9)	32	R/W	0000_0002h	5.1.7.2/540
3038_0894	CCM PLL Control Register (CCM_PLL_CTRL9_SET)	32	R/W	0000_0002h	5.1.7.3/542

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_0898	CCM PLL Control Register (CCM_PLL_CTRL9_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_089C	CCM PLL Control Register (CCM_PLL_CTRL9_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_08A0	CCM PLL Control Register (CCM_PLL_CTRL10)	32	R/W	0000_0002h	5.1.7.2/540
3038_08A4	CCM PLL Control Register (CCM_PLL_CTRL10_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_08A8	CCM PLL Control Register (CCM_PLL_CTRL10_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_08AC	CCM PLL Control Register (CCM_PLL_CTRL10_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_08B0	CCM PLL Control Register (CCM_PLL_CTRL11)	32	R/W	0000_0002h	5.1.7.2/540
3038_08B4	CCM PLL Control Register (CCM_PLL_CTRL11_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_08B8	CCM PLL Control Register (CCM_PLL_CTRL11_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_08BC	CCM PLL Control Register (CCM_PLL_CTRL11_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_08C0	CCM PLL Control Register (CCM_PLL_CTRL12)	32	R/W	0000_0002h	5.1.7.2/540
3038_08C4	CCM PLL Control Register (CCM_PLL_CTRL12_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_08C8	CCM PLL Control Register (CCM_PLL_CTRL12_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_08CC	CCM PLL Control Register (CCM_PLL_CTRL12_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_08D0	CCM PLL Control Register (CCM_PLL_CTRL13)	32	R/W	0000_0002h	5.1.7.2/540
3038_08D4	CCM PLL Control Register (CCM_PLL_CTRL13_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_08D8	CCM PLL Control Register (CCM_PLL_CTRL13_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_08DC	CCM PLL Control Register (CCM_PLL_CTRL13_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_08E0	CCM PLL Control Register (CCM_PLL_CTRL14)	32	R/W	0000_0002h	5.1.7.2/540
3038_08E4	CCM PLL Control Register (CCM_PLL_CTRL14_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_08E8	CCM PLL Control Register (CCM_PLL_CTRL14_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_08EC	CCM PLL Control Register (CCM_PLL_CTRL14_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_08F0	CCM PLL Control Register (CCM_PLL_CTRL15)	32	R/W	0000_0002h	5.1.7.2/540
3038_08F4	CCM PLL Control Register (CCM_PLL_CTRL15_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_08F8	CCM PLL Control Register (CCM_PLL_CTRL15_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_08FC	CCM PLL Control Register (CCM_PLL_CTRL15_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0900	CCM PLL Control Register (CCM_PLL_CTRL16)	32	R/W	0000_0002h	5.1.7.2/540
3038_0904	CCM PLL Control Register (CCM_PLL_CTRL16_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0908	CCM PLL Control Register (CCM_PLL_CTRL16_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_090C	CCM PLL Control Register (CCM_PLL_CTRL16_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0910	CCM PLL Control Register (CCM_PLL_CTRL17)	32	R/W	0000_0002h	5.1.7.2/540
3038_0914	CCM PLL Control Register (CCM_PLL_CTRL17_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0918	CCM PLL Control Register (CCM_PLL_CTRL17_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_091C	CCM PLL Control Register (CCM_PLL_CTRL17_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0920	CCM PLL Control Register (CCM_PLL_CTRL18)	32	R/W	0000_0002h	5.1.7.2/540
3038_0924	CCM PLL Control Register (CCM_PLL_CTRL18_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0928	CCM PLL Control Register (CCM_PLL_CTRL18_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_092C	CCM PLL Control Register (CCM_PLL_CTRL18_TOG)	32	R/W	0000_0002h	5.1.7.5/546

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_0930	CCM PLL Control Register (CCM_PLL_CTRL19)	32	R/W	0000_0002h	5.1.7.2/540
3038_0934	CCM PLL Control Register (CCM_PLL_CTRL19_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0938	CCM PLL Control Register (CCM_PLL_CTRL19_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_093C	CCM PLL Control Register (CCM_PLL_CTRL19_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0940	CCM PLL Control Register (CCM_PLL_CTRL20)	32	R/W	0000_0002h	5.1.7.2/540
3038_0944	CCM PLL Control Register (CCM_PLL_CTRL20_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0948	CCM PLL Control Register (CCM_PLL_CTRL20_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_094C	CCM PLL Control Register (CCM_PLL_CTRL20_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0950	CCM PLL Control Register (CCM_PLL_CTRL21)	32	R/W	0000_0002h	5.1.7.2/540
3038_0954	CCM PLL Control Register (CCM_PLL_CTRL21_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0958	CCM PLL Control Register (CCM_PLL_CTRL21_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_095C	CCM PLL Control Register (CCM_PLL_CTRL21_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0960	CCM PLL Control Register (CCM_PLL_CTRL22)	32	R/W	0000_0002h	5.1.7.2/540
3038_0964	CCM PLL Control Register (CCM_PLL_CTRL22_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0968	CCM PLL Control Register (CCM_PLL_CTRL22_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_096C	CCM PLL Control Register (CCM_PLL_CTRL22_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0970	CCM PLL Control Register (CCM_PLL_CTRL23)	32	R/W	0000_0002h	5.1.7.2/540
3038_0974	CCM PLL Control Register (CCM_PLL_CTRL23_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0978	CCM PLL Control Register (CCM_PLL_CTRL23_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_097C	CCM PLL Control Register (CCM_PLL_CTRL23_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0980	CCM PLL Control Register (CCM_PLL_CTRL24)	32	R/W	0000_0002h	5.1.7.2/540
3038_0984	CCM PLL Control Register (CCM_PLL_CTRL24_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0988	CCM PLL Control Register (CCM_PLL_CTRL24_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_098C	CCM PLL Control Register (CCM_PLL_CTRL24_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0990	CCM PLL Control Register (CCM_PLL_CTRL25)	32	R/W	0000_0002h	5.1.7.2/540
3038_0994	CCM PLL Control Register (CCM_PLL_CTRL25_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0998	CCM PLL Control Register (CCM_PLL_CTRL25_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_099C	CCM PLL Control Register (CCM_PLL_CTRL25_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_09A0	CCM PLL Control Register (CCM_PLL_CTRL26)	32	R/W	0000_0002h	5.1.7.2/540
3038_09A4	CCM PLL Control Register (CCM_PLL_CTRL26_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_09A8	CCM PLL Control Register (CCM_PLL_CTRL26_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_09AC	CCM PLL Control Register (CCM_PLL_CTRL26_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_09B0	CCM PLL Control Register (CCM_PLL_CTRL27)	32	R/W	0000_0002h	5.1.7.2/540
3038_09B4	CCM PLL Control Register (CCM_PLL_CTRL27_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_09B8	CCM PLL Control Register (CCM_PLL_CTRL27_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_09BC	CCM PLL Control Register (CCM_PLL_CTRL27_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_09C0	CCM PLL Control Register (CCM_PLL_CTRL28)	32	R/W	0000_0002h	5.1.7.2/540
3038_09C4	CCM PLL Control Register (CCM_PLL_CTRL28_SET)	32	R/W	0000_0002h	5.1.7.3/542

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_09C8	CCM PLL Control Register (CCM_PLL_CTRL28_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_09CC	CCM PLL Control Register (CCM_PLL_CTRL28_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_09D0	CCM PLL Control Register (CCM_PLL_CTRL29)	32	R/W	0000_0002h	5.1.7.2/540
3038_09D4	CCM PLL Control Register (CCM_PLL_CTRL29_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_09D8	CCM PLL Control Register (CCM_PLL_CTRL29_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_09DC	CCM PLL Control Register (CCM_PLL_CTRL29_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_09E0	CCM PLL Control Register (CCM_PLL_CTRL30)	32	R/W	0000_0002h	5.1.7.2/540
3038_09E4	CCM PLL Control Register (CCM_PLL_CTRL30_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_09E8	CCM PLL Control Register (CCM_PLL_CTRL30_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_09EC	CCM PLL Control Register (CCM_PLL_CTRL30_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_09F0	CCM PLL Control Register (CCM_PLL_CTRL31)	32	R/W	0000_0002h	5.1.7.2/540
3038_09F4	CCM PLL Control Register (CCM_PLL_CTRL31_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_09F8	CCM PLL Control Register (CCM_PLL_CTRL31_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_09FC	CCM PLL Control Register (CCM_PLL_CTRL31_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0A00	CCM PLL Control Register (CCM_PLL_CTRL32)	32	R/W	0000_0002h	5.1.7.2/540
3038_0A04	CCM PLL Control Register (CCM_PLL_CTRL32_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0A08	CCM PLL Control Register (CCM_PLL_CTRL32_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_0A0C	CCM PLL Control Register (CCM_PLL_CTRL32_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0A10	CCM PLL Control Register (CCM_PLL_CTRL33)	32	R/W	0000_0002h	5.1.7.2/540
3038_0A14	CCM PLL Control Register (CCM_PLL_CTRL33_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0A18	CCM PLL Control Register (CCM_PLL_CTRL33_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_0A1C	CCM PLL Control Register (CCM_PLL_CTRL33_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0A20	CCM PLL Control Register (CCM_PLL_CTRL34)	32	R/W	0000_0002h	5.1.7.2/540
3038_0A24	CCM PLL Control Register (CCM_PLL_CTRL34_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0A28	CCM PLL Control Register (CCM_PLL_CTRL34_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_0A2C	CCM PLL Control Register (CCM_PLL_CTRL34_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0A30	CCM PLL Control Register (CCM_PLL_CTRL35)	32	R/W	0000_0002h	5.1.7.2/540
3038_0A34	CCM PLL Control Register (CCM_PLL_CTRL35_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0A38	CCM PLL Control Register (CCM_PLL_CTRL35_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_0A3C	CCM PLL Control Register (CCM_PLL_CTRL35_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0A40	CCM PLL Control Register (CCM_PLL_CTRL36)	32	R/W	0000_0002h	5.1.7.2/540
3038_0A44	CCM PLL Control Register (CCM_PLL_CTRL36_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0A48	CCM PLL Control Register (CCM_PLL_CTRL36_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_0A4C	CCM PLL Control Register (CCM_PLL_CTRL36_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_0A50	CCM PLL Control Register (CCM_PLL_CTRL37)	32	R/W	0000_0002h	5.1.7.2/540
3038_0A54	CCM PLL Control Register (CCM_PLL_CTRL37_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0A58	CCM PLL Control Register (CCM_PLL_CTRL37_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_0A5C	CCM PLL Control Register (CCM_PLL_CTRL37_TOG)	32	R/W	0000_0002h	5.1.7.5/546

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_0A60	CCM PLL Control Register (CCM_PLL_CTRL38)	32	R/W	0000_0002h	5.1.7.2/540
3038_0A64	CCM PLL Control Register (CCM_PLL_CTRL38_SET)	32	R/W	0000_0002h	5.1.7.3/542
3038_0A68	CCM PLL Control Register (CCM_PLL_CTRL38_CLR)	32	R/W	0000_0002h	5.1.7.4/544
3038_0A6C	CCM PLL Control Register (CCM_PLL_CTRL38_TOG)	32	R/W	0000_0002h	5.1.7.5/546
3038_4000	CCM Clock Gating Register (CCM_CCGR0)	32	R/W	0000_0002h	5.1.7.6/548
3038_4004	CCM Clock Gating Register (CCM_CCGR0_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4008	CCM Clock Gating Register (CCM_CCGR0_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_400C	CCM Clock Gating Register (CCM_CCGR0_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4010	CCM Clock Gating Register (CCM_CCGR1)	32	R/W	0000_0002h	5.1.7.6/548
3038_4014	CCM Clock Gating Register (CCM_CCGR1_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4018	CCM Clock Gating Register (CCM_CCGR1_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_401C	CCM Clock Gating Register (CCM_CCGR1_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4020	CCM Clock Gating Register (CCM_CCGR2)	32	R/W	0000_0002h	5.1.7.6/548
3038_4024	CCM Clock Gating Register (CCM_CCGR2_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4028	CCM Clock Gating Register (CCM_CCGR2_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_402C	CCM Clock Gating Register (CCM_CCGR2_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4030	CCM Clock Gating Register (CCM_CCGR3)	32	R/W	0000_0002h	5.1.7.6/548
3038_4034	CCM Clock Gating Register (CCM_CCGR3_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4038	CCM Clock Gating Register (CCM_CCGR3_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_403C	CCM Clock Gating Register (CCM_CCGR3_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4040	CCM Clock Gating Register (CCM_CCGR4)	32	R/W	0000_0002h	5.1.7.6/548
3038_4044	CCM Clock Gating Register (CCM_CCGR4_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4048	CCM Clock Gating Register (CCM_CCGR4_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_404C	CCM Clock Gating Register (CCM_CCGR4_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4050	CCM Clock Gating Register (CCM_CCGR5)	32	R/W	0000_0002h	5.1.7.6/548
3038_4054	CCM Clock Gating Register (CCM_CCGR5_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4058	CCM Clock Gating Register (CCM_CCGR5_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_405C	CCM Clock Gating Register (CCM_CCGR5_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4060	CCM Clock Gating Register (CCM_CCGR6)	32	R/W	0000_0002h	5.1.7.6/548
3038_4064	CCM Clock Gating Register (CCM_CCGR6_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4068	CCM Clock Gating Register (CCM_CCGR6_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_406C	CCM Clock Gating Register (CCM_CCGR6_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4070	CCM Clock Gating Register (CCM_CCGR7)	32	R/W	0000_0002h	5.1.7.6/548
3038_4074	CCM Clock Gating Register (CCM_CCGR7_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4078	CCM Clock Gating Register (CCM_CCGR7_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_407C	CCM Clock Gating Register (CCM_CCGR7_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4080	CCM Clock Gating Register (CCM_CCGR8)	32	R/W	0000_0002h	5.1.7.6/548
3038_4084	CCM Clock Gating Register (CCM_CCGR8_SET)	32	R/W	0000_0002h	5.1.7.7/550

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4088	CCM Clock Gating Register (CCM_CCGR8_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_408C	CCM Clock Gating Register (CCM_CCGR8_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4090	CCM Clock Gating Register (CCM_CCGR9)	32	R/W	0000_0002h	5.1.7.6/548
3038_4094	CCM Clock Gating Register (CCM_CCGR9_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4098	CCM Clock Gating Register (CCM_CCGR9_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_409C	CCM Clock Gating Register (CCM_CCGR9_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_40A0	CCM Clock Gating Register (CCM_CCGR10)	32	R/W	0000_0002h	5.1.7.6/548
3038_40A4	CCM Clock Gating Register (CCM_CCGR10_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_40A8	CCM Clock Gating Register (CCM_CCGR10_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_40AC	CCM Clock Gating Register (CCM_CCGR10_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_40B0	CCM Clock Gating Register (CCM_CCGR11)	32	R/W	0000_0002h	5.1.7.6/548
3038_40B4	CCM Clock Gating Register (CCM_CCGR11_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_40B8	CCM Clock Gating Register (CCM_CCGR11_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_40BC	CCM Clock Gating Register (CCM_CCGR11_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_40C0	CCM Clock Gating Register (CCM_CCGR12)	32	R/W	0000_0002h	5.1.7.6/548
3038_40C4	CCM Clock Gating Register (CCM_CCGR12_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_40C8	CCM Clock Gating Register (CCM_CCGR12_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_40CC	CCM Clock Gating Register (CCM_CCGR12_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_40D0	CCM Clock Gating Register (CCM_CCGR13)	32	R/W	0000_0002h	5.1.7.6/548
3038_40D4	CCM Clock Gating Register (CCM_CCGR13_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_40D8	CCM Clock Gating Register (CCM_CCGR13_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_40DC	CCM Clock Gating Register (CCM_CCGR13_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_40E0	CCM Clock Gating Register (CCM_CCGR14)	32	R/W	0000_0002h	5.1.7.6/548
3038_40E4	CCM Clock Gating Register (CCM_CCGR14_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_40E8	CCM Clock Gating Register (CCM_CCGR14_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_40EC	CCM Clock Gating Register (CCM_CCGR14_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_40F0	CCM Clock Gating Register (CCM_CCGR15)	32	R/W	0000_0002h	5.1.7.6/548
3038_40F4	CCM Clock Gating Register (CCM_CCGR15_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_40F8	CCM Clock Gating Register (CCM_CCGR15_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_40FC	CCM Clock Gating Register (CCM_CCGR15_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4100	CCM Clock Gating Register (CCM_CCGR16)	32	R/W	0000_0002h	5.1.7.6/548
3038_4104	CCM Clock Gating Register (CCM_CCGR16_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4108	CCM Clock Gating Register (CCM_CCGR16_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_410C	CCM Clock Gating Register (CCM_CCGR16_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4110	CCM Clock Gating Register (CCM_CCGR17)	32	R/W	0000_0002h	5.1.7.6/548
3038_4114	CCM Clock Gating Register (CCM_CCGR17_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4118	CCM Clock Gating Register (CCM_CCGR17_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_411C	CCM Clock Gating Register (CCM_CCGR17_TOG)	32	R/W	0000_0002h	5.1.7.9/554

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4120	CCM Clock Gating Register (CCM_CCGR18)	32	R/W	0000_0002h	5.1.7.6/548
3038_4124	CCM Clock Gating Register (CCM_CCGR18_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4128	CCM Clock Gating Register (CCM_CCGR18_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_412C	CCM Clock Gating Register (CCM_CCGR18_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4130	CCM Clock Gating Register (CCM_CCGR19)	32	R/W	0000_0002h	5.1.7.6/548
3038_4134	CCM Clock Gating Register (CCM_CCGR19_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4138	CCM Clock Gating Register (CCM_CCGR19_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_413C	CCM Clock Gating Register (CCM_CCGR19_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4140	CCM Clock Gating Register (CCM_CCGR20)	32	R/W	0000_0002h	5.1.7.6/548
3038_4144	CCM Clock Gating Register (CCM_CCGR20_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4148	CCM Clock Gating Register (CCM_CCGR20_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_414C	CCM Clock Gating Register (CCM_CCGR20_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4150	CCM Clock Gating Register (CCM_CCGR21)	32	R/W	0000_0002h	5.1.7.6/548
3038_4154	CCM Clock Gating Register (CCM_CCGR21_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4158	CCM Clock Gating Register (CCM_CCGR21_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_415C	CCM Clock Gating Register (CCM_CCGR21_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4160	CCM Clock Gating Register (CCM_CCGR22)	32	R/W	0000_0002h	5.1.7.6/548
3038_4164	CCM Clock Gating Register (CCM_CCGR22_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4168	CCM Clock Gating Register (CCM_CCGR22_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_416C	CCM Clock Gating Register (CCM_CCGR22_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4170	CCM Clock Gating Register (CCM_CCGR23)	32	R/W	0000_0002h	5.1.7.6/548
3038_4174	CCM Clock Gating Register (CCM_CCGR23_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4178	CCM Clock Gating Register (CCM_CCGR23_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_417C	CCM Clock Gating Register (CCM_CCGR23_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4180	CCM Clock Gating Register (CCM_CCGR24)	32	R/W	0000_0002h	5.1.7.6/548
3038_4184	CCM Clock Gating Register (CCM_CCGR24_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4188	CCM Clock Gating Register (CCM_CCGR24_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_418C	CCM Clock Gating Register (CCM_CCGR24_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4190	CCM Clock Gating Register (CCM_CCGR25)	32	R/W	0000_0002h	5.1.7.6/548
3038_4194	CCM Clock Gating Register (CCM_CCGR25_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4198	CCM Clock Gating Register (CCM_CCGR25_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_419C	CCM Clock Gating Register (CCM_CCGR25_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_41A0	CCM Clock Gating Register (CCM_CCGR26)	32	R/W	0000_0002h	5.1.7.6/548
3038_41A4	CCM Clock Gating Register (CCM_CCGR26_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_41A8	CCM Clock Gating Register (CCM_CCGR26_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_41AC	CCM Clock Gating Register (CCM_CCGR26_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_41B0	CCM Clock Gating Register (CCM_CCGR27)	32	R/W	0000_0002h	5.1.7.6/548
3038_41B4	CCM Clock Gating Register (CCM_CCGR27_SET)	32	R/W	0000_0002h	5.1.7.7/550

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_41B8	CCM Clock Gating Register (CCM_CCGR27_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_41BC	CCM Clock Gating Register (CCM_CCGR27_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_41C0	CCM Clock Gating Register (CCM_CCGR28)	32	R/W	0000_0002h	5.1.7.6/548
3038_41C4	CCM Clock Gating Register (CCM_CCGR28_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_41C8	CCM Clock Gating Register (CCM_CCGR28_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_41CC	CCM Clock Gating Register (CCM_CCGR28_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_41D0	CCM Clock Gating Register (CCM_CCGR29)	32	R/W	0000_0002h	5.1.7.6/548
3038_41D4	CCM Clock Gating Register (CCM_CCGR29_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_41D8	CCM Clock Gating Register (CCM_CCGR29_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_41DC	CCM Clock Gating Register (CCM_CCGR29_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_41E0	CCM Clock Gating Register (CCM_CCGR30)	32	R/W	0000_0002h	5.1.7.6/548
3038_41E4	CCM Clock Gating Register (CCM_CCGR30_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_41E8	CCM Clock Gating Register (CCM_CCGR30_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_41EC	CCM Clock Gating Register (CCM_CCGR30_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_41F0	CCM Clock Gating Register (CCM_CCGR31)	32	R/W	0000_0002h	5.1.7.6/548
3038_41F4	CCM Clock Gating Register (CCM_CCGR31_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_41F8	CCM Clock Gating Register (CCM_CCGR31_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_41FC	CCM Clock Gating Register (CCM_CCGR31_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4200	CCM Clock Gating Register (CCM_CCGR32)	32	R/W	0000_0002h	5.1.7.6/548
3038_4204	CCM Clock Gating Register (CCM_CCGR32_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4208	CCM Clock Gating Register (CCM_CCGR32_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_420C	CCM Clock Gating Register (CCM_CCGR32_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4210	CCM Clock Gating Register (CCM_CCGR33)	32	R/W	0000_0002h	5.1.7.6/548
3038_4214	CCM Clock Gating Register (CCM_CCGR33_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4218	CCM Clock Gating Register (CCM_CCGR33_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_421C	CCM Clock Gating Register (CCM_CCGR33_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4220	CCM Clock Gating Register (CCM_CCGR34)	32	R/W	0000_0002h	5.1.7.6/548
3038_4224	CCM Clock Gating Register (CCM_CCGR34_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4228	CCM Clock Gating Register (CCM_CCGR34_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_422C	CCM Clock Gating Register (CCM_CCGR34_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4230	CCM Clock Gating Register (CCM_CCGR35)	32	R/W	0000_0002h	5.1.7.6/548
3038_4234	CCM Clock Gating Register (CCM_CCGR35_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4238	CCM Clock Gating Register (CCM_CCGR35_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_423C	CCM Clock Gating Register (CCM_CCGR35_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4240	CCM Clock Gating Register (CCM_CCGR36)	32	R/W	0000_0002h	5.1.7.6/548
3038_4244	CCM Clock Gating Register (CCM_CCGR36_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4248	CCM Clock Gating Register (CCM_CCGR36_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_424C	CCM Clock Gating Register (CCM_CCGR36_TOG)	32	R/W	0000_0002h	5.1.7.9/554

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4250	CCM Clock Gating Register (CCM_CCGR37)	32	R/W	0000_0002h	5.1.7.6/548
3038_4254	CCM Clock Gating Register (CCM_CCGR37_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4258	CCM Clock Gating Register (CCM_CCGR37_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_425C	CCM Clock Gating Register (CCM_CCGR37_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4260	CCM Clock Gating Register (CCM_CCGR38)	32	R/W	0000_0002h	5.1.7.6/548
3038_4264	CCM Clock Gating Register (CCM_CCGR38_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4268	CCM Clock Gating Register (CCM_CCGR38_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_426C	CCM Clock Gating Register (CCM_CCGR38_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4270	CCM Clock Gating Register (CCM_CCGR39)	32	R/W	0000_0002h	5.1.7.6/548
3038_4274	CCM Clock Gating Register (CCM_CCGR39_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4278	CCM Clock Gating Register (CCM_CCGR39_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_427C	CCM Clock Gating Register (CCM_CCGR39_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4280	CCM Clock Gating Register (CCM_CCGR40)	32	R/W	0000_0002h	5.1.7.6/548
3038_4284	CCM Clock Gating Register (CCM_CCGR40_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4288	CCM Clock Gating Register (CCM_CCGR40_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_428C	CCM Clock Gating Register (CCM_CCGR40_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4290	CCM Clock Gating Register (CCM_CCGR41)	32	R/W	0000_0002h	5.1.7.6/548
3038_4294	CCM Clock Gating Register (CCM_CCGR41_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4298	CCM Clock Gating Register (CCM_CCGR41_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_429C	CCM Clock Gating Register (CCM_CCGR41_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_42A0	CCM Clock Gating Register (CCM_CCGR42)	32	R/W	0000_0002h	5.1.7.6/548
3038_42A4	CCM Clock Gating Register (CCM_CCGR42_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_42A8	CCM Clock Gating Register (CCM_CCGR42_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_42AC	CCM Clock Gating Register (CCM_CCGR42_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_42B0	CCM Clock Gating Register (CCM_CCGR43)	32	R/W	0000_0002h	5.1.7.6/548
3038_42B4	CCM Clock Gating Register (CCM_CCGR43_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_42B8	CCM Clock Gating Register (CCM_CCGR43_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_42BC	CCM Clock Gating Register (CCM_CCGR43_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_42C0	CCM Clock Gating Register (CCM_CCGR44)	32	R/W	0000_0002h	5.1.7.6/548
3038_42C4	CCM Clock Gating Register (CCM_CCGR44_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_42C8	CCM Clock Gating Register (CCM_CCGR44_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_42CC	CCM Clock Gating Register (CCM_CCGR44_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_42D0	CCM Clock Gating Register (CCM_CCGR45)	32	R/W	0000_0002h	5.1.7.6/548
3038_42D4	CCM Clock Gating Register (CCM_CCGR45_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_42D8	CCM Clock Gating Register (CCM_CCGR45_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_42DC	CCM Clock Gating Register (CCM_CCGR45_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_42E0	CCM Clock Gating Register (CCM_CCGR46)	32	R/W	0000_0002h	5.1.7.6/548
3038_42E4	CCM Clock Gating Register (CCM_CCGR46_SET)	32	R/W	0000_0002h	5.1.7.7/550

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_42E8	CCM Clock Gating Register (CCM_CCGR46_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_42EC	CCM Clock Gating Register (CCM_CCGR46_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_42F0	CCM Clock Gating Register (CCM_CCGR47)	32	R/W	0000_0002h	5.1.7.6/548
3038_42F4	CCM Clock Gating Register (CCM_CCGR47_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_42F8	CCM Clock Gating Register (CCM_CCGR47_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_42FC	CCM Clock Gating Register (CCM_CCGR47_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4300	CCM Clock Gating Register (CCM_CCGR48)	32	R/W	0000_0002h	5.1.7.6/548
3038_4304	CCM Clock Gating Register (CCM_CCGR48_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4308	CCM Clock Gating Register (CCM_CCGR48_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_430C	CCM Clock Gating Register (CCM_CCGR48_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4310	CCM Clock Gating Register (CCM_CCGR49)	32	R/W	0000_0002h	5.1.7.6/548
3038_4314	CCM Clock Gating Register (CCM_CCGR49_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4318	CCM Clock Gating Register (CCM_CCGR49_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_431C	CCM Clock Gating Register (CCM_CCGR49_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4320	CCM Clock Gating Register (CCM_CCGR50)	32	R/W	0000_0002h	5.1.7.6/548
3038_4324	CCM Clock Gating Register (CCM_CCGR50_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4328	CCM Clock Gating Register (CCM_CCGR50_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_432C	CCM Clock Gating Register (CCM_CCGR50_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4330	CCM Clock Gating Register (CCM_CCGR51)	32	R/W	0000_0002h	5.1.7.6/548
3038_4334	CCM Clock Gating Register (CCM_CCGR51_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4338	CCM Clock Gating Register (CCM_CCGR51_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_433C	CCM Clock Gating Register (CCM_CCGR51_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4340	CCM Clock Gating Register (CCM_CCGR52)	32	R/W	0000_0002h	5.1.7.6/548
3038_4344	CCM Clock Gating Register (CCM_CCGR52_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4348	CCM Clock Gating Register (CCM_CCGR52_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_434C	CCM Clock Gating Register (CCM_CCGR52_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4350	CCM Clock Gating Register (CCM_CCGR53)	32	R/W	0000_0002h	5.1.7.6/548
3038_4354	CCM Clock Gating Register (CCM_CCGR53_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4358	CCM Clock Gating Register (CCM_CCGR53_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_435C	CCM Clock Gating Register (CCM_CCGR53_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4360	CCM Clock Gating Register (CCM_CCGR54)	32	R/W	0000_0002h	5.1.7.6/548
3038_4364	CCM Clock Gating Register (CCM_CCGR54_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4368	CCM Clock Gating Register (CCM_CCGR54_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_436C	CCM Clock Gating Register (CCM_CCGR54_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4370	CCM Clock Gating Register (CCM_CCGR55)	32	R/W	0000_0002h	5.1.7.6/548
3038_4374	CCM Clock Gating Register (CCM_CCGR55_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4378	CCM Clock Gating Register (CCM_CCGR55_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_437C	CCM Clock Gating Register (CCM_CCGR55_TOG)	32	R/W	0000_0002h	5.1.7.9/554

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4380	CCM Clock Gating Register (CCM_CCGR56)	32	R/W	0000_0002h	5.1.7.6/548
3038_4384	CCM Clock Gating Register (CCM_CCGR56_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4388	CCM Clock Gating Register (CCM_CCGR56_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_438C	CCM Clock Gating Register (CCM_CCGR56_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4390	CCM Clock Gating Register (CCM_CCGR57)	32	R/W	0000_0002h	5.1.7.6/548
3038_4394	CCM Clock Gating Register (CCM_CCGR57_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4398	CCM Clock Gating Register (CCM_CCGR57_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_439C	CCM Clock Gating Register (CCM_CCGR57_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_43A0	CCM Clock Gating Register (CCM_CCGR58)	32	R/W	0000_0002h	5.1.7.6/548
3038_43A4	CCM Clock Gating Register (CCM_CCGR58_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_43A8	CCM Clock Gating Register (CCM_CCGR58_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_43AC	CCM Clock Gating Register (CCM_CCGR58_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_43B0	CCM Clock Gating Register (CCM_CCGR59)	32	R/W	0000_0002h	5.1.7.6/548
3038_43B4	CCM Clock Gating Register (CCM_CCGR59_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_43B8	CCM Clock Gating Register (CCM_CCGR59_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_43BC	CCM Clock Gating Register (CCM_CCGR59_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_43C0	CCM Clock Gating Register (CCM_CCGR60)	32	R/W	0000_0002h	5.1.7.6/548
3038_43C4	CCM Clock Gating Register (CCM_CCGR60_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_43C8	CCM Clock Gating Register (CCM_CCGR60_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_43CC	CCM Clock Gating Register (CCM_CCGR60_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_43D0	CCM Clock Gating Register (CCM_CCGR61)	32	R/W	0000_0002h	5.1.7.6/548
3038_43D4	CCM Clock Gating Register (CCM_CCGR61_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_43D8	CCM Clock Gating Register (CCM_CCGR61_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_43DC	CCM Clock Gating Register (CCM_CCGR61_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_43E0	CCM Clock Gating Register (CCM_CCGR62)	32	R/W	0000_0002h	5.1.7.6/548
3038_43E4	CCM Clock Gating Register (CCM_CCGR62_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_43E8	CCM Clock Gating Register (CCM_CCGR62_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_43EC	CCM Clock Gating Register (CCM_CCGR62_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_43F0	CCM Clock Gating Register (CCM_CCGR63)	32	R/W	0000_0002h	5.1.7.6/548
3038_43F4	CCM Clock Gating Register (CCM_CCGR63_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_43F8	CCM Clock Gating Register (CCM_CCGR63_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_43FC	CCM Clock Gating Register (CCM_CCGR63_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4400	CCM Clock Gating Register (CCM_CCGR64)	32	R/W	0000_0002h	5.1.7.6/548
3038_4404	CCM Clock Gating Register (CCM_CCGR64_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4408	CCM Clock Gating Register (CCM_CCGR64_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_440C	CCM Clock Gating Register (CCM_CCGR64_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4410	CCM Clock Gating Register (CCM_CCGR65)	32	R/W	0000_0002h	5.1.7.6/548
3038_4414	CCM Clock Gating Register (CCM_CCGR65_SET)	32	R/W	0000_0002h	5.1.7.7/550

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4418	CCM Clock Gating Register (CCM_CCGR65_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_441C	CCM Clock Gating Register (CCM_CCGR65_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4420	CCM Clock Gating Register (CCM_CCGR66)	32	R/W	0000_0002h	5.1.7.6/548
3038_4424	CCM Clock Gating Register (CCM_CCGR66_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4428	CCM Clock Gating Register (CCM_CCGR66_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_442C	CCM Clock Gating Register (CCM_CCGR66_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4430	CCM Clock Gating Register (CCM_CCGR67)	32	R/W	0000_0002h	5.1.7.6/548
3038_4434	CCM Clock Gating Register (CCM_CCGR67_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4438	CCM Clock Gating Register (CCM_CCGR67_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_443C	CCM Clock Gating Register (CCM_CCGR67_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4440	CCM Clock Gating Register (CCM_CCGR68)	32	R/W	0000_0002h	5.1.7.6/548
3038_4444	CCM Clock Gating Register (CCM_CCGR68_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4448	CCM Clock Gating Register (CCM_CCGR68_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_444C	CCM Clock Gating Register (CCM_CCGR68_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4450	CCM Clock Gating Register (CCM_CCGR69)	32	R/W	0000_0002h	5.1.7.6/548
3038_4454	CCM Clock Gating Register (CCM_CCGR69_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4458	CCM Clock Gating Register (CCM_CCGR69_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_445C	CCM Clock Gating Register (CCM_CCGR69_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4460	CCM Clock Gating Register (CCM_CCGR70)	32	R/W	0000_0002h	5.1.7.6/548
3038_4464	CCM Clock Gating Register (CCM_CCGR70_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4468	CCM Clock Gating Register (CCM_CCGR70_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_446C	CCM Clock Gating Register (CCM_CCGR70_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4470	CCM Clock Gating Register (CCM_CCGR71)	32	R/W	0000_0002h	5.1.7.6/548
3038_4474	CCM Clock Gating Register (CCM_CCGR71_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4478	CCM Clock Gating Register (CCM_CCGR71_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_447C	CCM Clock Gating Register (CCM_CCGR71_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4480	CCM Clock Gating Register (CCM_CCGR72)	32	R/W	0000_0002h	5.1.7.6/548
3038_4484	CCM Clock Gating Register (CCM_CCGR72_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4488	CCM Clock Gating Register (CCM_CCGR72_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_448C	CCM Clock Gating Register (CCM_CCGR72_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4490	CCM Clock Gating Register (CCM_CCGR73)	32	R/W	0000_0002h	5.1.7.6/548
3038_4494	CCM Clock Gating Register (CCM_CCGR73_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4498	CCM Clock Gating Register (CCM_CCGR73_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_449C	CCM Clock Gating Register (CCM_CCGR73_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_44A0	CCM Clock Gating Register (CCM_CCGR74)	32	R/W	0000_0002h	5.1.7.6/548
3038_44A4	CCM Clock Gating Register (CCM_CCGR74_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_44A8	CCM Clock Gating Register (CCM_CCGR74_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_44AC	CCM Clock Gating Register (CCM_CCGR74_TOG)	32	R/W	0000_0002h	5.1.7.9/554

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_44B0	CCM Clock Gating Register (CCM_CCGR75)	32	R/W	0000_0002h	5.1.7.6/548
3038_44B4	CCM Clock Gating Register (CCM_CCGR75_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_44B8	CCM Clock Gating Register (CCM_CCGR75_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_44BC	CCM Clock Gating Register (CCM_CCGR75_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_44C0	CCM Clock Gating Register (CCM_CCGR76)	32	R/W	0000_0002h	5.1.7.6/548
3038_44C4	CCM Clock Gating Register (CCM_CCGR76_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_44C8	CCM Clock Gating Register (CCM_CCGR76_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_44CC	CCM Clock Gating Register (CCM_CCGR76_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_44D0	CCM Clock Gating Register (CCM_CCGR77)	32	R/W	0000_0002h	5.1.7.6/548
3038_44D4	CCM Clock Gating Register (CCM_CCGR77_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_44D8	CCM Clock Gating Register (CCM_CCGR77_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_44DC	CCM Clock Gating Register (CCM_CCGR77_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_44E0	CCM Clock Gating Register (CCM_CCGR78)	32	R/W	0000_0002h	5.1.7.6/548
3038_44E4	CCM Clock Gating Register (CCM_CCGR78_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_44E8	CCM Clock Gating Register (CCM_CCGR78_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_44EC	CCM Clock Gating Register (CCM_CCGR78_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_44F0	CCM Clock Gating Register (CCM_CCGR79)	32	R/W	0000_0002h	5.1.7.6/548
3038_44F4	CCM Clock Gating Register (CCM_CCGR79_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_44F8	CCM Clock Gating Register (CCM_CCGR79_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_44FC	CCM Clock Gating Register (CCM_CCGR79_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4500	CCM Clock Gating Register (CCM_CCGR80)	32	R/W	0000_0002h	5.1.7.6/548
3038_4504	CCM Clock Gating Register (CCM_CCGR80_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4508	CCM Clock Gating Register (CCM_CCGR80_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_450C	CCM Clock Gating Register (CCM_CCGR80_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4510	CCM Clock Gating Register (CCM_CCGR81)	32	R/W	0000_0002h	5.1.7.6/548
3038_4514	CCM Clock Gating Register (CCM_CCGR81_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4518	CCM Clock Gating Register (CCM_CCGR81_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_451C	CCM Clock Gating Register (CCM_CCGR81_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4520	CCM Clock Gating Register (CCM_CCGR82)	32	R/W	0000_0002h	5.1.7.6/548
3038_4524	CCM Clock Gating Register (CCM_CCGR82_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4528	CCM Clock Gating Register (CCM_CCGR82_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_452C	CCM Clock Gating Register (CCM_CCGR82_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4530	CCM Clock Gating Register (CCM_CCGR83)	32	R/W	0000_0002h	5.1.7.6/548
3038_4534	CCM Clock Gating Register (CCM_CCGR83_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4538	CCM Clock Gating Register (CCM_CCGR83_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_453C	CCM Clock Gating Register (CCM_CCGR83_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4540	CCM Clock Gating Register (CCM_CCGR84)	32	R/W	0000_0002h	5.1.7.6/548
3038_4544	CCM Clock Gating Register (CCM_CCGR84_SET)	32	R/W	0000_0002h	5.1.7.7/550

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4548	CCM Clock Gating Register (CCM_CCGR84_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_454C	CCM Clock Gating Register (CCM_CCGR84_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4550	CCM Clock Gating Register (CCM_CCGR85)	32	R/W	0000_0002h	5.1.7.6/548
3038_4554	CCM Clock Gating Register (CCM_CCGR85_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4558	CCM Clock Gating Register (CCM_CCGR85_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_455C	CCM Clock Gating Register (CCM_CCGR85_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4560	CCM Clock Gating Register (CCM_CCGR86)	32	R/W	0000_0002h	5.1.7.6/548
3038_4564	CCM Clock Gating Register (CCM_CCGR86_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4568	CCM Clock Gating Register (CCM_CCGR86_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_456C	CCM Clock Gating Register (CCM_CCGR86_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4570	CCM Clock Gating Register (CCM_CCGR87)	32	R/W	0000_0002h	5.1.7.6/548
3038_4574	CCM Clock Gating Register (CCM_CCGR87_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4578	CCM Clock Gating Register (CCM_CCGR87_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_457C	CCM Clock Gating Register (CCM_CCGR87_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4580	CCM Clock Gating Register (CCM_CCGR88)	32	R/W	0000_0002h	5.1.7.6/548
3038_4584	CCM Clock Gating Register (CCM_CCGR88_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4588	CCM Clock Gating Register (CCM_CCGR88_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_458C	CCM Clock Gating Register (CCM_CCGR88_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4590	CCM Clock Gating Register (CCM_CCGR89)	32	R/W	0000_0002h	5.1.7.6/548
3038_4594	CCM Clock Gating Register (CCM_CCGR89_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4598	CCM Clock Gating Register (CCM_CCGR89_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_459C	CCM Clock Gating Register (CCM_CCGR89_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_45A0	CCM Clock Gating Register (CCM_CCGR90)	32	R/W	0000_0002h	5.1.7.6/548
3038_45A4	CCM Clock Gating Register (CCM_CCGR90_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_45A8	CCM Clock Gating Register (CCM_CCGR90_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_45AC	CCM Clock Gating Register (CCM_CCGR90_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_45B0	CCM Clock Gating Register (CCM_CCGR91)	32	R/W	0000_0002h	5.1.7.6/548
3038_45B4	CCM Clock Gating Register (CCM_CCGR91_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_45B8	CCM Clock Gating Register (CCM_CCGR91_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_45BC	CCM Clock Gating Register (CCM_CCGR91_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_45C0	CCM Clock Gating Register (CCM_CCGR92)	32	R/W	0000_0002h	5.1.7.6/548
3038_45C4	CCM Clock Gating Register (CCM_CCGR92_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_45C8	CCM Clock Gating Register (CCM_CCGR92_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_45CC	CCM Clock Gating Register (CCM_CCGR92_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_45D0	CCM Clock Gating Register (CCM_CCGR93)	32	R/W	0000_0002h	5.1.7.6/548
3038_45D4	CCM Clock Gating Register (CCM_CCGR93_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_45D8	CCM Clock Gating Register (CCM_CCGR93_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_45DC	CCM Clock Gating Register (CCM_CCGR93_TOG)	32	R/W	0000_0002h	5.1.7.9/554

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_45E0	CCM Clock Gating Register (CCM_CCGR94)	32	R/W	0000_0002h	5.1.7.6/548
3038_45E4	CCM Clock Gating Register (CCM_CCGR94_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_45E8	CCM Clock Gating Register (CCM_CCGR94_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_45EC	CCM Clock Gating Register (CCM_CCGR94_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_45F0	CCM Clock Gating Register (CCM_CCGR95)	32	R/W	0000_0002h	5.1.7.6/548
3038_45F4	CCM Clock Gating Register (CCM_CCGR95_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_45F8	CCM Clock Gating Register (CCM_CCGR95_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_45FC	CCM Clock Gating Register (CCM_CCGR95_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4600	CCM Clock Gating Register (CCM_CCGR96)	32	R/W	0000_0002h	5.1.7.6/548
3038_4604	CCM Clock Gating Register (CCM_CCGR96_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4608	CCM Clock Gating Register (CCM_CCGR96_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_460C	CCM Clock Gating Register (CCM_CCGR96_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4610	CCM Clock Gating Register (CCM_CCGR97)	32	R/W	0000_0002h	5.1.7.6/548
3038_4614	CCM Clock Gating Register (CCM_CCGR97_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4618	CCM Clock Gating Register (CCM_CCGR97_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_461C	CCM Clock Gating Register (CCM_CCGR97_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4620	CCM Clock Gating Register (CCM_CCGR98)	32	R/W	0000_0002h	5.1.7.6/548
3038_4624	CCM Clock Gating Register (CCM_CCGR98_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4628	CCM Clock Gating Register (CCM_CCGR98_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_462C	CCM Clock Gating Register (CCM_CCGR98_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4630	CCM Clock Gating Register (CCM_CCGR99)	32	R/W	0000_0002h	5.1.7.6/548
3038_4634	CCM Clock Gating Register (CCM_CCGR99_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4638	CCM Clock Gating Register (CCM_CCGR99_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_463C	CCM Clock Gating Register (CCM_CCGR99_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4640	CCM Clock Gating Register (CCM_CCGR100)	32	R/W	0000_0002h	5.1.7.6/548
3038_4644	CCM Clock Gating Register (CCM_CCGR100_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4648	CCM Clock Gating Register (CCM_CCGR100_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_464C	CCM Clock Gating Register (CCM_CCGR100_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4650	CCM Clock Gating Register (CCM_CCGR101)	32	R/W	0000_0002h	5.1.7.6/548
3038_4654	CCM Clock Gating Register (CCM_CCGR101_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4658	CCM Clock Gating Register (CCM_CCGR101_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_465C	CCM Clock Gating Register (CCM_CCGR101_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4660	CCM Clock Gating Register (CCM_CCGR102)	32	R/W	0000_0002h	5.1.7.6/548
3038_4664	CCM Clock Gating Register (CCM_CCGR102_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4668	CCM Clock Gating Register (CCM_CCGR102_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_466C	CCM Clock Gating Register (CCM_CCGR102_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4670	CCM Clock Gating Register (CCM_CCGR103)	32	R/W	0000_0002h	5.1.7.6/548
3038_4674	CCM Clock Gating Register (CCM_CCGR103_SET)	32	R/W	0000_0002h	5.1.7.7/550

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4678	CCM Clock Gating Register (CCM_CCGR103_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_467C	CCM Clock Gating Register (CCM_CCGR103_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4680	CCM Clock Gating Register (CCM_CCGR104)	32	R/W	0000_0002h	5.1.7.6/548
3038_4684	CCM Clock Gating Register (CCM_CCGR104_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4688	CCM Clock Gating Register (CCM_CCGR104_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_468C	CCM Clock Gating Register (CCM_CCGR104_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4690	CCM Clock Gating Register (CCM_CCGR105)	32	R/W	0000_0002h	5.1.7.6/548
3038_4694	CCM Clock Gating Register (CCM_CCGR105_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4698	CCM Clock Gating Register (CCM_CCGR105_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_469C	CCM Clock Gating Register (CCM_CCGR105_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_46A0	CCM Clock Gating Register (CCM_CCGR106)	32	R/W	0000_0002h	5.1.7.6/548
3038_46A4	CCM Clock Gating Register (CCM_CCGR106_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_46A8	CCM Clock Gating Register (CCM_CCGR106_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_46AC	CCM Clock Gating Register (CCM_CCGR106_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_46B0	CCM Clock Gating Register (CCM_CCGR107)	32	R/W	0000_0002h	5.1.7.6/548
3038_46B4	CCM Clock Gating Register (CCM_CCGR107_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_46B8	CCM Clock Gating Register (CCM_CCGR107_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_46BC	CCM Clock Gating Register (CCM_CCGR107_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_46C0	CCM Clock Gating Register (CCM_CCGR108)	32	R/W	0000_0002h	5.1.7.6/548
3038_46C4	CCM Clock Gating Register (CCM_CCGR108_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_46C8	CCM Clock Gating Register (CCM_CCGR108_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_46CC	CCM Clock Gating Register (CCM_CCGR108_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_46D0	CCM Clock Gating Register (CCM_CCGR109)	32	R/W	0000_0002h	5.1.7.6/548
3038_46D4	CCM Clock Gating Register (CCM_CCGR109_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_46D8	CCM Clock Gating Register (CCM_CCGR109_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_46DC	CCM Clock Gating Register (CCM_CCGR109_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_46E0	CCM Clock Gating Register (CCM_CCGR110)	32	R/W	0000_0002h	5.1.7.6/548
3038_46E4	CCM Clock Gating Register (CCM_CCGR110_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_46E8	CCM Clock Gating Register (CCM_CCGR110_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_46EC	CCM Clock Gating Register (CCM_CCGR110_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_46F0	CCM Clock Gating Register (CCM_CCGR111)	32	R/W	0000_0002h	5.1.7.6/548
3038_46F4	CCM Clock Gating Register (CCM_CCGR111_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_46F8	CCM Clock Gating Register (CCM_CCGR111_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_46FC	CCM Clock Gating Register (CCM_CCGR111_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4700	CCM Clock Gating Register (CCM_CCGR112)	32	R/W	0000_0002h	5.1.7.6/548
3038_4704	CCM Clock Gating Register (CCM_CCGR112_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4708	CCM Clock Gating Register (CCM_CCGR112_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_470C	CCM Clock Gating Register (CCM_CCGR112_TOG)	32	R/W	0000_0002h	5.1.7.9/554

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4710	CCM Clock Gating Register (CCM_CCGR113)	32	R/W	0000_0002h	5.1.7.6/548
3038_4714	CCM Clock Gating Register (CCM_CCGR113_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4718	CCM Clock Gating Register (CCM_CCGR113_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_471C	CCM Clock Gating Register (CCM_CCGR113_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4720	CCM Clock Gating Register (CCM_CCGR114)	32	R/W	0000_0002h	5.1.7.6/548
3038_4724	CCM Clock Gating Register (CCM_CCGR114_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4728	CCM Clock Gating Register (CCM_CCGR114_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_472C	CCM Clock Gating Register (CCM_CCGR114_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4730	CCM Clock Gating Register (CCM_CCGR115)	32	R/W	0000_0002h	5.1.7.6/548
3038_4734	CCM Clock Gating Register (CCM_CCGR115_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4738	CCM Clock Gating Register (CCM_CCGR115_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_473C	CCM Clock Gating Register (CCM_CCGR115_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4740	CCM Clock Gating Register (CCM_CCGR116)	32	R/W	0000_0002h	5.1.7.6/548
3038_4744	CCM Clock Gating Register (CCM_CCGR116_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4748	CCM Clock Gating Register (CCM_CCGR116_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_474C	CCM Clock Gating Register (CCM_CCGR116_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4750	CCM Clock Gating Register (CCM_CCGR117)	32	R/W	0000_0002h	5.1.7.6/548
3038_4754	CCM Clock Gating Register (CCM_CCGR117_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4758	CCM Clock Gating Register (CCM_CCGR117_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_475C	CCM Clock Gating Register (CCM_CCGR117_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4760	CCM Clock Gating Register (CCM_CCGR118)	32	R/W	0000_0002h	5.1.7.6/548
3038_4764	CCM Clock Gating Register (CCM_CCGR118_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4768	CCM Clock Gating Register (CCM_CCGR118_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_476C	CCM Clock Gating Register (CCM_CCGR118_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4770	CCM Clock Gating Register (CCM_CCGR119)	32	R/W	0000_0002h	5.1.7.6/548
3038_4774	CCM Clock Gating Register (CCM_CCGR119_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4778	CCM Clock Gating Register (CCM_CCGR119_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_477C	CCM Clock Gating Register (CCM_CCGR119_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4780	CCM Clock Gating Register (CCM_CCGR120)	32	R/W	0000_0002h	5.1.7.6/548
3038_4784	CCM Clock Gating Register (CCM_CCGR120_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4788	CCM Clock Gating Register (CCM_CCGR120_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_478C	CCM Clock Gating Register (CCM_CCGR120_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4790	CCM Clock Gating Register (CCM_CCGR121)	32	R/W	0000_0002h	5.1.7.6/548
3038_4794	CCM Clock Gating Register (CCM_CCGR121_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4798	CCM Clock Gating Register (CCM_CCGR121_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_479C	CCM Clock Gating Register (CCM_CCGR121_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_47A0	CCM Clock Gating Register (CCM_CCGR122)	32	R/W	0000_0002h	5.1.7.6/548
3038_47A4	CCM Clock Gating Register (CCM_CCGR122_SET)	32	R/W	0000_0002h	5.1.7.7/550

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_47A8	CCM Clock Gating Register (CCM_CCGR122_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_47AC	CCM Clock Gating Register (CCM_CCGR122_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_47B0	CCM Clock Gating Register (CCM_CCGR123)	32	R/W	0000_0002h	5.1.7.6/548
3038_47B4	CCM Clock Gating Register (CCM_CCGR123_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_47B8	CCM Clock Gating Register (CCM_CCGR123_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_47BC	CCM Clock Gating Register (CCM_CCGR123_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_47C0	CCM Clock Gating Register (CCM_CCGR124)	32	R/W	0000_0002h	5.1.7.6/548
3038_47C4	CCM Clock Gating Register (CCM_CCGR124_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_47C8	CCM Clock Gating Register (CCM_CCGR124_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_47CC	CCM Clock Gating Register (CCM_CCGR124_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_47D0	CCM Clock Gating Register (CCM_CCGR125)	32	R/W	0000_0002h	5.1.7.6/548
3038_47D4	CCM Clock Gating Register (CCM_CCGR125_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_47D8	CCM Clock Gating Register (CCM_CCGR125_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_47DC	CCM Clock Gating Register (CCM_CCGR125_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_47E0	CCM Clock Gating Register (CCM_CCGR126)	32	R/W	0000_0002h	5.1.7.6/548
3038_47E4	CCM Clock Gating Register (CCM_CCGR126_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_47E8	CCM Clock Gating Register (CCM_CCGR126_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_47EC	CCM Clock Gating Register (CCM_CCGR126_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_47F0	CCM Clock Gating Register (CCM_CCGR127)	32	R/W	0000_0002h	5.1.7.6/548
3038_47F4	CCM Clock Gating Register (CCM_CCGR127_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_47F8	CCM Clock Gating Register (CCM_CCGR127_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_47FC	CCM Clock Gating Register (CCM_CCGR127_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4800	CCM Clock Gating Register (CCM_CCGR128)	32	R/W	0000_0002h	5.1.7.6/548
3038_4804	CCM Clock Gating Register (CCM_CCGR128_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4808	CCM Clock Gating Register (CCM_CCGR128_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_480C	CCM Clock Gating Register (CCM_CCGR128_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4810	CCM Clock Gating Register (CCM_CCGR129)	32	R/W	0000_0002h	5.1.7.6/548
3038_4814	CCM Clock Gating Register (CCM_CCGR129_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4818	CCM Clock Gating Register (CCM_CCGR129_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_481C	CCM Clock Gating Register (CCM_CCGR129_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4820	CCM Clock Gating Register (CCM_CCGR130)	32	R/W	0000_0002h	5.1.7.6/548
3038_4824	CCM Clock Gating Register (CCM_CCGR130_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4828	CCM Clock Gating Register (CCM_CCGR130_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_482C	CCM Clock Gating Register (CCM_CCGR130_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4830	CCM Clock Gating Register (CCM_CCGR131)	32	R/W	0000_0002h	5.1.7.6/548
3038_4834	CCM Clock Gating Register (CCM_CCGR131_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4838	CCM Clock Gating Register (CCM_CCGR131_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_483C	CCM Clock Gating Register (CCM_CCGR131_TOG)	32	R/W	0000_0002h	5.1.7.9/554

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4840	CCM Clock Gating Register (CCM_CCGR132)	32	R/W	0000_0002h	5.1.7.6/548
3038_4844	CCM Clock Gating Register (CCM_CCGR132_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4848	CCM Clock Gating Register (CCM_CCGR132_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_484C	CCM Clock Gating Register (CCM_CCGR132_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4850	CCM Clock Gating Register (CCM_CCGR133)	32	R/W	0000_0002h	5.1.7.6/548
3038_4854	CCM Clock Gating Register (CCM_CCGR133_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4858	CCM Clock Gating Register (CCM_CCGR133_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_485C	CCM Clock Gating Register (CCM_CCGR133_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4860	CCM Clock Gating Register (CCM_CCGR134)	32	R/W	0000_0002h	5.1.7.6/548
3038_4864	CCM Clock Gating Register (CCM_CCGR134_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4868	CCM Clock Gating Register (CCM_CCGR134_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_486C	CCM Clock Gating Register (CCM_CCGR134_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4870	CCM Clock Gating Register (CCM_CCGR135)	32	R/W	0000_0002h	5.1.7.6/548
3038_4874	CCM Clock Gating Register (CCM_CCGR135_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4878	CCM Clock Gating Register (CCM_CCGR135_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_487C	CCM Clock Gating Register (CCM_CCGR135_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4880	CCM Clock Gating Register (CCM_CCGR136)	32	R/W	0000_0002h	5.1.7.6/548
3038_4884	CCM Clock Gating Register (CCM_CCGR136_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4888	CCM Clock Gating Register (CCM_CCGR136_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_488C	CCM Clock Gating Register (CCM_CCGR136_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4890	CCM Clock Gating Register (CCM_CCGR137)	32	R/W	0000_0002h	5.1.7.6/548
3038_4894	CCM Clock Gating Register (CCM_CCGR137_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4898	CCM Clock Gating Register (CCM_CCGR137_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_489C	CCM Clock Gating Register (CCM_CCGR137_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_48A0	CCM Clock Gating Register (CCM_CCGR138)	32	R/W	0000_0002h	5.1.7.6/548
3038_48A4	CCM Clock Gating Register (CCM_CCGR138_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_48A8	CCM Clock Gating Register (CCM_CCGR138_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_48AC	CCM Clock Gating Register (CCM_CCGR138_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_48B0	CCM Clock Gating Register (CCM_CCGR139)	32	R/W	0000_0002h	5.1.7.6/548
3038_48B4	CCM Clock Gating Register (CCM_CCGR139_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_48B8	CCM Clock Gating Register (CCM_CCGR139_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_48BC	CCM Clock Gating Register (CCM_CCGR139_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_48C0	CCM Clock Gating Register (CCM_CCGR140)	32	R/W	0000_0002h	5.1.7.6/548
3038_48C4	CCM Clock Gating Register (CCM_CCGR140_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_48C8	CCM Clock Gating Register (CCM_CCGR140_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_48CC	CCM Clock Gating Register (CCM_CCGR140_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_48D0	CCM Clock Gating Register (CCM_CCGR141)	32	R/W	0000_0002h	5.1.7.6/548
3038_48D4	CCM Clock Gating Register (CCM_CCGR141_SET)	32	R/W	0000_0002h	5.1.7.7/550

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_48D8	CCM Clock Gating Register (CCM_CCGR141_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_48DC	CCM Clock Gating Register (CCM_CCGR141_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_48E0	CCM Clock Gating Register (CCM_CCGR142)	32	R/W	0000_0002h	5.1.7.6/548
3038_48E4	CCM Clock Gating Register (CCM_CCGR142_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_48E8	CCM Clock Gating Register (CCM_CCGR142_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_48EC	CCM Clock Gating Register (CCM_CCGR142_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_48F0	CCM Clock Gating Register (CCM_CCGR143)	32	R/W	0000_0002h	5.1.7.6/548
3038_48F4	CCM Clock Gating Register (CCM_CCGR143_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_48F8	CCM Clock Gating Register (CCM_CCGR143_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_48FC	CCM Clock Gating Register (CCM_CCGR143_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4900	CCM Clock Gating Register (CCM_CCGR144)	32	R/W	0000_0002h	5.1.7.6/548
3038_4904	CCM Clock Gating Register (CCM_CCGR144_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4908	CCM Clock Gating Register (CCM_CCGR144_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_490C	CCM Clock Gating Register (CCM_CCGR144_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4910	CCM Clock Gating Register (CCM_CCGR145)	32	R/W	0000_0002h	5.1.7.6/548
3038_4914	CCM Clock Gating Register (CCM_CCGR145_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4918	CCM Clock Gating Register (CCM_CCGR145_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_491C	CCM Clock Gating Register (CCM_CCGR145_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4920	CCM Clock Gating Register (CCM_CCGR146)	32	R/W	0000_0002h	5.1.7.6/548
3038_4924	CCM Clock Gating Register (CCM_CCGR146_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4928	CCM Clock Gating Register (CCM_CCGR146_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_492C	CCM Clock Gating Register (CCM_CCGR146_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4930	CCM Clock Gating Register (CCM_CCGR147)	32	R/W	0000_0002h	5.1.7.6/548
3038_4934	CCM Clock Gating Register (CCM_CCGR147_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4938	CCM Clock Gating Register (CCM_CCGR147_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_493C	CCM Clock Gating Register (CCM_CCGR147_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4940	CCM Clock Gating Register (CCM_CCGR148)	32	R/W	0000_0002h	5.1.7.6/548
3038_4944	CCM Clock Gating Register (CCM_CCGR148_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4948	CCM Clock Gating Register (CCM_CCGR148_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_494C	CCM Clock Gating Register (CCM_CCGR148_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4950	CCM Clock Gating Register (CCM_CCGR149)	32	R/W	0000_0002h	5.1.7.6/548
3038_4954	CCM Clock Gating Register (CCM_CCGR149_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4958	CCM Clock Gating Register (CCM_CCGR149_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_495C	CCM Clock Gating Register (CCM_CCGR149_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4960	CCM Clock Gating Register (CCM_CCGR150)	32	R/W	0000_0002h	5.1.7.6/548
3038_4964	CCM Clock Gating Register (CCM_CCGR150_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4968	CCM Clock Gating Register (CCM_CCGR150_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_496C	CCM Clock Gating Register (CCM_CCGR150_TOG)	32	R/W	0000_0002h	5.1.7.9/554

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4970	CCM Clock Gating Register (CCM_CCGR151)	32	R/W	0000_0002h	5.1.7.6/548
3038_4974	CCM Clock Gating Register (CCM_CCGR151_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4978	CCM Clock Gating Register (CCM_CCGR151_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_497C	CCM Clock Gating Register (CCM_CCGR151_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4980	CCM Clock Gating Register (CCM_CCGR152)	32	R/W	0000_0002h	5.1.7.6/548
3038_4984	CCM Clock Gating Register (CCM_CCGR152_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4988	CCM Clock Gating Register (CCM_CCGR152_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_498C	CCM Clock Gating Register (CCM_CCGR152_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4990	CCM Clock Gating Register (CCM_CCGR153)	32	R/W	0000_0002h	5.1.7.6/548
3038_4994	CCM Clock Gating Register (CCM_CCGR153_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4998	CCM Clock Gating Register (CCM_CCGR153_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_499C	CCM Clock Gating Register (CCM_CCGR153_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_49A0	CCM Clock Gating Register (CCM_CCGR154)	32	R/W	0000_0002h	5.1.7.6/548
3038_49A4	CCM Clock Gating Register (CCM_CCGR154_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_49A8	CCM Clock Gating Register (CCM_CCGR154_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_49AC	CCM Clock Gating Register (CCM_CCGR154_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_49B0	CCM Clock Gating Register (CCM_CCGR155)	32	R/W	0000_0002h	5.1.7.6/548
3038_49B4	CCM Clock Gating Register (CCM_CCGR155_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_49B8	CCM Clock Gating Register (CCM_CCGR155_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_49BC	CCM Clock Gating Register (CCM_CCGR155_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_49C0	CCM Clock Gating Register (CCM_CCGR156)	32	R/W	0000_0002h	5.1.7.6/548
3038_49C4	CCM Clock Gating Register (CCM_CCGR156_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_49C8	CCM Clock Gating Register (CCM_CCGR156_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_49CC	CCM Clock Gating Register (CCM_CCGR156_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_49D0	CCM Clock Gating Register (CCM_CCGR157)	32	R/W	0000_0002h	5.1.7.6/548
3038_49D4	CCM Clock Gating Register (CCM_CCGR157_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_49D8	CCM Clock Gating Register (CCM_CCGR157_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_49DC	CCM Clock Gating Register (CCM_CCGR157_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_49E0	CCM Clock Gating Register (CCM_CCGR158)	32	R/W	0000_0002h	5.1.7.6/548
3038_49E4	CCM Clock Gating Register (CCM_CCGR158_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_49E8	CCM Clock Gating Register (CCM_CCGR158_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_49EC	CCM Clock Gating Register (CCM_CCGR158_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_49F0	CCM Clock Gating Register (CCM_CCGR159)	32	R/W	0000_0002h	5.1.7.6/548
3038_49F4	CCM Clock Gating Register (CCM_CCGR159_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_49F8	CCM Clock Gating Register (CCM_CCGR159_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_49FC	CCM Clock Gating Register (CCM_CCGR159_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4A00	CCM Clock Gating Register (CCM_CCGR160)	32	R/W	0000_0002h	5.1.7.6/548
3038_4A04	CCM Clock Gating Register (CCM_CCGR160_SET)	32	R/W	0000_0002h	5.1.7.7/550

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4A08	CCM Clock Gating Register (CCM_CCGR160_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4A0C	CCM Clock Gating Register (CCM_CCGR160_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4A10	CCM Clock Gating Register (CCM_CCGR161)	32	R/W	0000_0002h	5.1.7.6/548
3038_4A14	CCM Clock Gating Register (CCM_CCGR161_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4A18	CCM Clock Gating Register (CCM_CCGR161_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4A1C	CCM Clock Gating Register (CCM_CCGR161_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4A20	CCM Clock Gating Register (CCM_CCGR162)	32	R/W	0000_0002h	5.1.7.6/548
3038_4A24	CCM Clock Gating Register (CCM_CCGR162_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4A28	CCM Clock Gating Register (CCM_CCGR162_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4A2C	CCM Clock Gating Register (CCM_CCGR162_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4A30	CCM Clock Gating Register (CCM_CCGR163)	32	R/W	0000_0002h	5.1.7.6/548
3038_4A34	CCM Clock Gating Register (CCM_CCGR163_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4A38	CCM Clock Gating Register (CCM_CCGR163_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4A3C	CCM Clock Gating Register (CCM_CCGR163_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4A40	CCM Clock Gating Register (CCM_CCGR164)	32	R/W	0000_0002h	5.1.7.6/548
3038_4A44	CCM Clock Gating Register (CCM_CCGR164_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4A48	CCM Clock Gating Register (CCM_CCGR164_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4A4C	CCM Clock Gating Register (CCM_CCGR164_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4A50	CCM Clock Gating Register (CCM_CCGR165)	32	R/W	0000_0002h	5.1.7.6/548
3038_4A54	CCM Clock Gating Register (CCM_CCGR165_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4A58	CCM Clock Gating Register (CCM_CCGR165_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4A5C	CCM Clock Gating Register (CCM_CCGR165_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4A60	CCM Clock Gating Register (CCM_CCGR166)	32	R/W	0000_0002h	5.1.7.6/548
3038_4A64	CCM Clock Gating Register (CCM_CCGR166_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4A68	CCM Clock Gating Register (CCM_CCGR166_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4A6C	CCM Clock Gating Register (CCM_CCGR166_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4A70	CCM Clock Gating Register (CCM_CCGR167)	32	R/W	0000_0002h	5.1.7.6/548
3038_4A74	CCM Clock Gating Register (CCM_CCGR167_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4A78	CCM Clock Gating Register (CCM_CCGR167_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4A7C	CCM Clock Gating Register (CCM_CCGR167_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4A80	CCM Clock Gating Register (CCM_CCGR168)	32	R/W	0000_0002h	5.1.7.6/548
3038_4A84	CCM Clock Gating Register (CCM_CCGR168_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4A88	CCM Clock Gating Register (CCM_CCGR168_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4A8C	CCM Clock Gating Register (CCM_CCGR168_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4A90	CCM Clock Gating Register (CCM_CCGR169)	32	R/W	0000_0002h	5.1.7.6/548
3038_4A94	CCM Clock Gating Register (CCM_CCGR169_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4A98	CCM Clock Gating Register (CCM_CCGR169_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4A9C	CCM Clock Gating Register (CCM_CCGR169_TOG)	32	R/W	0000_0002h	5.1.7.9/554

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4AA0	CCM Clock Gating Register (CCM_CCGR170)	32	R/W	0000_0002h	5.1.7.6/548
3038_4AA4	CCM Clock Gating Register (CCM_CCGR170_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4AA8	CCM Clock Gating Register (CCM_CCGR170_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4AAC	CCM Clock Gating Register (CCM_CCGR170_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4AB0	CCM Clock Gating Register (CCM_CCGR171)	32	R/W	0000_0002h	5.1.7.6/548
3038_4AB4	CCM Clock Gating Register (CCM_CCGR171_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4AB8	CCM Clock Gating Register (CCM_CCGR171_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4ABC	CCM Clock Gating Register (CCM_CCGR171_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4AC0	CCM Clock Gating Register (CCM_CCGR172)	32	R/W	0000_0002h	5.1.7.6/548
3038_4AC4	CCM Clock Gating Register (CCM_CCGR172_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4AC8	CCM Clock Gating Register (CCM_CCGR172_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4ACC	CCM Clock Gating Register (CCM_CCGR172_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4AD0	CCM Clock Gating Register (CCM_CCGR173)	32	R/W	0000_0002h	5.1.7.6/548
3038_4AD4	CCM Clock Gating Register (CCM_CCGR173_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4AD8	CCM Clock Gating Register (CCM_CCGR173_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4ADC	CCM Clock Gating Register (CCM_CCGR173_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4AE0	CCM Clock Gating Register (CCM_CCGR174)	32	R/W	0000_0002h	5.1.7.6/548
3038_4AE4	CCM Clock Gating Register (CCM_CCGR174_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4AE8	CCM Clock Gating Register (CCM_CCGR174_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4AEC	CCM Clock Gating Register (CCM_CCGR174_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4AF0	CCM Clock Gating Register (CCM_CCGR175)	32	R/W	0000_0002h	5.1.7.6/548
3038_4AF4	CCM Clock Gating Register (CCM_CCGR175_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4AF8	CCM Clock Gating Register (CCM_CCGR175_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4AFC	CCM Clock Gating Register (CCM_CCGR175_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4B00	CCM Clock Gating Register (CCM_CCGR176)	32	R/W	0000_0002h	5.1.7.6/548
3038_4B04	CCM Clock Gating Register (CCM_CCGR176_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4B08	CCM Clock Gating Register (CCM_CCGR176_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4B0C	CCM Clock Gating Register (CCM_CCGR176_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4B10	CCM Clock Gating Register (CCM_CCGR177)	32	R/W	0000_0002h	5.1.7.6/548
3038_4B14	CCM Clock Gating Register (CCM_CCGR177_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4B18	CCM Clock Gating Register (CCM_CCGR177_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4B1C	CCM Clock Gating Register (CCM_CCGR177_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4B20	CCM Clock Gating Register (CCM_CCGR178)	32	R/W	0000_0002h	5.1.7.6/548
3038_4B24	CCM Clock Gating Register (CCM_CCGR178_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4B28	CCM Clock Gating Register (CCM_CCGR178_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4B2C	CCM Clock Gating Register (CCM_CCGR178_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4B30	CCM Clock Gating Register (CCM_CCGR179)	32	R/W	0000_0002h	5.1.7.6/548
3038_4B34	CCM Clock Gating Register (CCM_CCGR179_SET)	32	R/W	0000_0002h	5.1.7.7/550

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4B38	CCM Clock Gating Register (CCM_CCGR179_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4B3C	CCM Clock Gating Register (CCM_CCGR179_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4B40	CCM Clock Gating Register (CCM_CCGR180)	32	R/W	0000_0002h	5.1.7.6/548
3038_4B44	CCM Clock Gating Register (CCM_CCGR180_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4B48	CCM Clock Gating Register (CCM_CCGR180_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4B4C	CCM Clock Gating Register (CCM_CCGR180_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4B50	CCM Clock Gating Register (CCM_CCGR181)	32	R/W	0000_0002h	5.1.7.6/548
3038_4B54	CCM Clock Gating Register (CCM_CCGR181_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4B58	CCM Clock Gating Register (CCM_CCGR181_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4B5C	CCM Clock Gating Register (CCM_CCGR181_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4B60	CCM Clock Gating Register (CCM_CCGR182)	32	R/W	0000_0002h	5.1.7.6/548
3038_4B64	CCM Clock Gating Register (CCM_CCGR182_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4B68	CCM Clock Gating Register (CCM_CCGR182_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4B6C	CCM Clock Gating Register (CCM_CCGR182_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4B70	CCM Clock Gating Register (CCM_CCGR183)	32	R/W	0000_0002h	5.1.7.6/548
3038_4B74	CCM Clock Gating Register (CCM_CCGR183_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4B78	CCM Clock Gating Register (CCM_CCGR183_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4B7C	CCM Clock Gating Register (CCM_CCGR183_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4B80	CCM Clock Gating Register (CCM_CCGR184)	32	R/W	0000_0002h	5.1.7.6/548
3038_4B84	CCM Clock Gating Register (CCM_CCGR184_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4B88	CCM Clock Gating Register (CCM_CCGR184_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4B8C	CCM Clock Gating Register (CCM_CCGR184_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4B90	CCM Clock Gating Register (CCM_CCGR185)	32	R/W	0000_0002h	5.1.7.6/548
3038_4B94	CCM Clock Gating Register (CCM_CCGR185_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4B98	CCM Clock Gating Register (CCM_CCGR185_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4B9C	CCM Clock Gating Register (CCM_CCGR185_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4BA0	CCM Clock Gating Register (CCM_CCGR186)	32	R/W	0000_0002h	5.1.7.6/548
3038_4BA4	CCM Clock Gating Register (CCM_CCGR186_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4BA8	CCM Clock Gating Register (CCM_CCGR186_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4BAC	CCM Clock Gating Register (CCM_CCGR186_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4BB0	CCM Clock Gating Register (CCM_CCGR187)	32	R/W	0000_0002h	5.1.7.6/548
3038_4BB4	CCM Clock Gating Register (CCM_CCGR187_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4BB8	CCM Clock Gating Register (CCM_CCGR187_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4BBC	CCM Clock Gating Register (CCM_CCGR187_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4BC0	CCM Clock Gating Register (CCM_CCGR188)	32	R/W	0000_0002h	5.1.7.6/548
3038_4BC4	CCM Clock Gating Register (CCM_CCGR188_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4BC8	CCM Clock Gating Register (CCM_CCGR188_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4BCC	CCM Clock Gating Register (CCM_CCGR188_TOG)	32	R/W	0000_0002h	5.1.7.9/554

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_4BD0	CCM Clock Gating Register (CCM_CCGR189)	32	R/W	0000_0002h	5.1.7.6/548
3038_4BD4	CCM Clock Gating Register (CCM_CCGR189_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4BD8	CCM Clock Gating Register (CCM_CCGR189_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4BDC	CCM Clock Gating Register (CCM_CCGR189_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4BE0	CCM Clock Gating Register (CCM_CCGR190)	32	R/W	0000_0002h	5.1.7.6/548
3038_4BE4	CCM Clock Gating Register (CCM_CCGR190_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4BE8	CCM Clock Gating Register (CCM_CCGR190_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4BEC	CCM Clock Gating Register (CCM_CCGR190_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_4BF0	CCM Clock Gating Register (CCM_CCGR191)	32	R/W	0000_0002h	5.1.7.6/548
3038_4BF4	CCM Clock Gating Register (CCM_CCGR191_SET)	32	R/W	0000_0002h	5.1.7.7/550
3038_4BF8	CCM Clock Gating Register (CCM_CCGR191_CLR)	32	R/W	0000_0002h	5.1.7.8/552
3038_4BFC	CCM Clock Gating Register (CCM_CCGR191_TOG)	32	R/W	0000_0002h	5.1.7.9/554
3038_8000	Target Register (CCM_TARGET_ROOT0)	32	R/W	1000_0000h	5.1.7.10/556
3038_8004	Target Register (CCM_TARGET_ROOT0_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8008	Target Register (CCM_TARGET_ROOT0_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_800C	Target Register (CCM_TARGET_ROOT0_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8010	Miscellaneous Register (CCM_MISC0)	32	R/W	0000_0000h	5.1.7.14/564
3038_8014	Miscellaneous Register (CCM_MISC_ROOT0_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8018	Miscellaneous Register (CCM_MISC_ROOT0_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_801C	Miscellaneous Register (CCM_MISC_ROOT0_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8020	Post Divider Register (CCM_POST0)	32	R/W	0000_0000h	5.1.7.18/568
3038_8024	Post Divider Register (CCM_POST_ROOT0_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8028	Post Divider Register (CCM_POST_ROOT0_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_802C	Post Divider Register (CCM_POST_ROOT0_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8030	Pre Divider Register (CCM_PRE0)	32	R/W	1000_0000h	5.1.7.22/580
3038_8034	Pre Divider Register (CCM_PRE_ROOT0_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8038	Pre Divider Register (CCM_PRE_ROOT0_CLR)	32	R/W	0000_0000h	5.1.7.24/586

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_803C	Pre Divider Register (CCM_PRE_ROOT0_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8070	Access Control Register (CCM_ACCESS_CTRL0)	32	R/W	0000_0000h	5.1.7.26/592
3038_8074	Access Control Register (CCM_ACCESS_CTRL_ROOT0_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8078	Access Control Register (CCM_ACCESS_CTRL_ROOT0_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_807C	Access Control Register (CCM_ACCESS_CTRL_ROOT0_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8080	Target Register (CCM_TARGET_ROOT1)	32	R/W	1000_0000h	5.1.7.10/556
3038_8084	Target Register (CCM_TARGET_ROOT1_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8088	Target Register (CCM_TARGET_ROOT1_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_808C	Target Register (CCM_TARGET_ROOT1_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8090	Miscellaneous Register (CCM_MISC1)	32	R/W	0000_0000h	5.1.7.14/564
3038_8094	Miscellaneous Register (CCM_MISC_ROOT1_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8098	Miscellaneous Register (CCM_MISC_ROOT1_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_809C	Miscellaneous Register (CCM_MISC_ROOT1_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_80A0	Post Divider Register (CCM_POST1)	32	R/W	0000_0000h	5.1.7.18/568
3038_80A4	Post Divider Register (CCM_POST_ROOT1_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_80A8	Post Divider Register (CCM_POST_ROOT1_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_80AC	Post Divider Register (CCM_POST_ROOT1_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_80B0	Pre Divider Register (CCM_PRE1)	32	R/W	1000_0000h	5.1.7.22/580
3038_80B4	Pre Divider Register (CCM_PRE_ROOT1_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_80B8	Pre Divider Register (CCM_PRE_ROOT1_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_80BC	Pre Divider Register (CCM_PRE_ROOT1_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_80F0	Access Control Register (CCM_ACCESS_CTRL1)	32	R/W	0000_0000h	5.1.7.26/592

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_80F4	Access Control Register (CCM_ACCESS_CTRL_ROOT1_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_80F8	Access Control Register (CCM_ACCESS_CTRL_ROOT1_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_80FC	Access Control Register (CCM_ACCESS_CTRL_ROOT1_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8100	Target Register (CCM_TARGET_ROOT2)	32	R/W	1000_0000h	5.1.7.10/556
3038_8104	Target Register (CCM_TARGET_ROOT2_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8108	Target Register (CCM_TARGET_ROOT2_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_810C	Target Register (CCM_TARGET_ROOT2_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8110	Miscellaneous Register (CCM_MISC2)	32	R/W	0000_0000h	5.1.7.14/564
3038_8114	Miscellaneous Register (CCM_MISC_ROOT2_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8118	Miscellaneous Register (CCM_MISC_ROOT2_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_811C	Miscellaneous Register (CCM_MISC_ROOT2_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8120	Post Divider Register (CCM_POST2)	32	R/W	0000_0000h	5.1.7.18/568
3038_8124	Post Divider Register (CCM_POST_ROOT2_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8128	Post Divider Register (CCM_POST_ROOT2_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_812C	Post Divider Register (CCM_POST_ROOT2_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8130	Pre Divider Register (CCM_PRE2)	32	R/W	1000_0000h	5.1.7.22/580
3038_8134	Pre Divider Register (CCM_PRE_ROOT2_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8138	Pre Divider Register (CCM_PRE_ROOT2_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_813C	Pre Divider Register (CCM_PRE_ROOT2_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8170	Access Control Register (CCM_ACCESS_CTRL2)	32	R/W	0000_0000h	5.1.7.26/592
3038_8174	Access Control Register (CCM_ACCESS_CTRL_ROOT2_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8178	Access Control Register (CCM_ACCESS_CTRL_ROOT2_CLR)	32	R/W	0000_0000h	5.1.7.28/597

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_817C	Access Control Register (CCM_ACCESS_CTRL_ROOT2_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8180	Target Register (CCM_TARGET_ROOT3)	32	R/W	1000_0000h	5.1.7.10/556
3038_8184	Target Register (CCM_TARGET_ROOT3_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8188	Target Register (CCM_TARGET_ROOT3_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_818C	Target Register (CCM_TARGET_ROOT3_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8190	Miscellaneous Register (CCM_MISC3)	32	R/W	0000_0000h	5.1.7.14/564
3038_8194	Miscellaneous Register (CCM_MISC_ROOT3_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8198	Miscellaneous Register (CCM_MISC_ROOT3_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_819C	Miscellaneous Register (CCM_MISC_ROOT3_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_81A0	Post Divider Register (CCM_POST3)	32	R/W	0000_0000h	5.1.7.18/568
3038_81A4	Post Divider Register (CCM_POST_ROOT3_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_81A8	Post Divider Register (CCM_POST_ROOT3_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_81AC	Post Divider Register (CCM_POST_ROOT3_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_81B0	Pre Divider Register (CCM_PRE3)	32	R/W	1000_0000h	5.1.7.22/580
3038_81B4	Pre Divider Register (CCM_PRE_ROOT3_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_81B8	Pre Divider Register (CCM_PRE_ROOT3_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_81BC	Pre Divider Register (CCM_PRE_ROOT3_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_81F0	Access Control Register (CCM_ACCESS_CTRL3)	32	R/W	0000_0000h	5.1.7.26/592
3038_81F4	Access Control Register (CCM_ACCESS_CTRL_ROOT3_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_81F8	Access Control Register (CCM_ACCESS_CTRL_ROOT3_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_81FC	Access Control Register (CCM_ACCESS_CTRL_ROOT3_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8200	Target Register (CCM_TARGET_ROOT4)	32	R/W	1000_0000h	5.1.7.10/556

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8204	Target Register (CCM_TARGET_ROOT4_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8208	Target Register (CCM_TARGET_ROOT4_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_820C	Target Register (CCM_TARGET_ROOT4_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8210	Miscellaneous Register (CCM_MISC4)	32	R/W	0000_0000h	5.1.7.14/564
3038_8214	Miscellaneous Register (CCM_MISC_ROOT4_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8218	Miscellaneous Register (CCM_MISC_ROOT4_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_821C	Miscellaneous Register (CCM_MISC_ROOT4_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8220	Post Divider Register (CCM_POST4)	32	R/W	0000_0000h	5.1.7.18/568
3038_8224	Post Divider Register (CCM_POST_ROOT4_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8228	Post Divider Register (CCM_POST_ROOT4_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_822C	Post Divider Register (CCM_POST_ROOT4_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8230	Pre Divider Register (CCM_PRE4)	32	R/W	1000_0000h	5.1.7.22/580
3038_8234	Pre Divider Register (CCM_PRE_ROOT4_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8238	Pre Divider Register (CCM_PRE_ROOT4_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_823C	Pre Divider Register (CCM_PRE_ROOT4_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8270	Access Control Register (CCM_ACCESS_CTRL4)	32	R/W	0000_0000h	5.1.7.26/592
3038_8274	Access Control Register (CCM_ACCESS_CTRL_ROOT4_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8278	Access Control Register (CCM_ACCESS_CTRL_ROOT4_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_827C	Access Control Register (CCM_ACCESS_CTRL_ROOT4_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8280	Target Register (CCM_TARGET_ROOT5)	32	R/W	1000_0000h	5.1.7.10/556
3038_8284	Target Register (CCM_TARGET_ROOT5_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8288	Target Register (CCM_TARGET_ROOT5_CLR)	32	R/W	0000_0000h	5.1.7.12/560

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_828C	Target Register (CCM_TARGET_ROOT5_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8290	Miscellaneous Register (CCM_MISC5)	32	R/W	0000_0000h	5.1.7.14/564
3038_8294	Miscellaneous Register (CCM_MISC_ROOT5_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8298	Miscellaneous Register (CCM_MISC_ROOT5_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_829C	Miscellaneous Register (CCM_MISC_ROOT5_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_82A0	Post Divider Register (CCM_POST5)	32	R/W	0000_0000h	5.1.7.18/568
3038_82A4	Post Divider Register (CCM_POST_ROOT5_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_82A8	Post Divider Register (CCM_POST_ROOT5_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_82AC	Post Divider Register (CCM_POST_ROOT5_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_82B0	Pre Divider Register (CCM_PRE5)	32	R/W	1000_0000h	5.1.7.22/580
3038_82B4	Pre Divider Register (CCM_PRE_ROOT5_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_82B8	Pre Divider Register (CCM_PRE_ROOT5_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_82BC	Pre Divider Register (CCM_PRE_ROOT5_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_82F0	Access Control Register (CCM_ACCESS_CTRL5)	32	R/W	0000_0000h	5.1.7.26/592
3038_82F4	Access Control Register (CCM_ACCESS_CTRL_ROOT5_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_82F8	Access Control Register (CCM_ACCESS_CTRL_ROOT5_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_82FC	Access Control Register (CCM_ACCESS_CTRL_ROOT5_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8300	Target Register (CCM_TARGET_ROOT6)	32	R/W	1000_0000h	5.1.7.10/556
3038_8304	Target Register (CCM_TARGET_ROOT6_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8308	Target Register (CCM_TARGET_ROOT6_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_830C	Target Register (CCM_TARGET_ROOT6_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8310	Miscellaneous Register (CCM_MISC6)	32	R/W	0000_0000h	5.1.7.14/564

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8314	Miscellaneous Register (CCM_MISC_ROOT6_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8318	Miscellaneous Register (CCM_MISC_ROOT6_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_831C	Miscellaneous Register (CCM_MISC_ROOT6_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8320	Post Divider Register (CCM_POST6)	32	R/W	0000_0000h	5.1.7.18/568
3038_8324	Post Divider Register (CCM_POST_ROOT6_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8328	Post Divider Register (CCM_POST_ROOT6_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_832C	Post Divider Register (CCM_POST_ROOT6_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8330	Pre Divider Register (CCM_PRE6)	32	R/W	1000_0000h	5.1.7.22/580
3038_8334	Pre Divider Register (CCM_PRE_ROOT6_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8338	Pre Divider Register (CCM_PRE_ROOT6_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_833C	Pre Divider Register (CCM_PRE_ROOT6_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8370	Access Control Register (CCM_ACCESS_CTRL6)	32	R/W	0000_0000h	5.1.7.26/592
3038_8374	Access Control Register (CCM_ACCESS_CTRL_ROOT6_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8378	Access Control Register (CCM_ACCESS_CTRL_ROOT6_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_837C	Access Control Register (CCM_ACCESS_CTRL_ROOT6_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8380	Target Register (CCM_TARGET_ROOT7)	32	R/W	1000_0000h	5.1.7.10/556
3038_8384	Target Register (CCM_TARGET_ROOT7_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8388	Target Register (CCM_TARGET_ROOT7_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_838C	Target Register (CCM_TARGET_ROOT7_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8390	Miscellaneous Register (CCM_MISC7)	32	R/W	0000_0000h	5.1.7.14/564
3038_8394	Miscellaneous Register (CCM_MISC_ROOT7_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8398	Miscellaneous Register (CCM_MISC_ROOT7_CLR)	32	R/W	0000_0000h	5.1.7.16/566

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_839C	Miscellaneous Register (CCM_MISC_ROOT7_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_83A0	Post Divider Register (CCM_POST7)	32	R/W	0000_0000h	5.1.7.18/568
3038_83A4	Post Divider Register (CCM_POST_ROOT7_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_83A8	Post Divider Register (CCM_POST_ROOT7_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_83AC	Post Divider Register (CCM_POST_ROOT7_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_83B0	Pre Divider Register (CCM_PRE7)	32	R/W	1000_0000h	5.1.7.22/580
3038_83B4	Pre Divider Register (CCM_PRE_ROOT7_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_83B8	Pre Divider Register (CCM_PRE_ROOT7_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_83BC	Pre Divider Register (CCM_PRE_ROOT7_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_83F0	Access Control Register (CCM_ACCESS_CTRL7)	32	R/W	0000_0000h	5.1.7.26/592
3038_83F4	Access Control Register (CCM_ACCESS_CTRL_ROOT7_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_83F8	Access Control Register (CCM_ACCESS_CTRL_ROOT7_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_83FC	Access Control Register (CCM_ACCESS_CTRL_ROOT7_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8400	Target Register (CCM_TARGET_ROOT8)	32	R/W	1000_0000h	5.1.7.10/556
3038_8404	Target Register (CCM_TARGET_ROOT8_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8408	Target Register (CCM_TARGET_ROOT8_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_840C	Target Register (CCM_TARGET_ROOT8_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8410	Miscellaneous Register (CCM_MISC8)	32	R/W	0000_0000h	5.1.7.14/564
3038_8414	Miscellaneous Register (CCM_MISC_ROOT8_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8418	Miscellaneous Register (CCM_MISC_ROOT8_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_841C	Miscellaneous Register (CCM_MISC_ROOT8_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8420	Post Divider Register (CCM_POST8)	32	R/W	0000_0000h	5.1.7.18/568

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8424	Post Divider Register (CCM_POST_ROOT8_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8428	Post Divider Register (CCM_POST_ROOT8_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_842C	Post Divider Register (CCM_POST_ROOT8_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8430	Pre Divider Register (CCM_PRE8)	32	R/W	1000_0000h	5.1.7.22/580
3038_8434	Pre Divider Register (CCM_PRE_ROOT8_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8438	Pre Divider Register (CCM_PRE_ROOT8_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_843C	Pre Divider Register (CCM_PRE_ROOT8_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8470	Access Control Register (CCM_ACCESS_CTRL8)	32	R/W	0000_0000h	5.1.7.26/592
3038_8474	Access Control Register (CCM_ACCESS_CTRL_ROOT8_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8478	Access Control Register (CCM_ACCESS_CTRL_ROOT8_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_847C	Access Control Register (CCM_ACCESS_CTRL_ROOT8_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8480	Target Register (CCM_TARGET_ROOT9)	32	R/W	1000_0000h	5.1.7.10/556
3038_8484	Target Register (CCM_TARGET_ROOT9_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8488	Target Register (CCM_TARGET_ROOT9_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_848C	Target Register (CCM_TARGET_ROOT9_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8490	Miscellaneous Register (CCM_MISC9)	32	R/W	0000_0000h	5.1.7.14/564
3038_8494	Miscellaneous Register (CCM_MISC_ROOT9_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8498	Miscellaneous Register (CCM_MISC_ROOT9_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_849C	Miscellaneous Register (CCM_MISC_ROOT9_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_84A0	Post Divider Register (CCM_POST9)	32	R/W	0000_0000h	5.1.7.18/568
3038_84A4	Post Divider Register (CCM_POST_ROOT9_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_84A8	Post Divider Register (CCM_POST_ROOT9_CLR)	32	R/W	0000_0000h	5.1.7.20/574

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_84AC	Post Divider Register (CCM_POST_ROOT9_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_84B0	Pre Divider Register (CCM_PRE9)	32	R/W	1000_0000h	5.1.7.22/580
3038_84B4	Pre Divider Register (CCM_PRE_ROOT9_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_84B8	Pre Divider Register (CCM_PRE_ROOT9_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_84BC	Pre Divider Register (CCM_PRE_ROOT9_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_84F0	Access Control Register (CCM_ACCESS_CTRL9)	32	R/W	0000_0000h	5.1.7.26/592
3038_84F4	Access Control Register (CCM_ACCESS_CTRL_ROOT9_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_84F8	Access Control Register (CCM_ACCESS_CTRL_ROOT9_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_84FC	Access Control Register (CCM_ACCESS_CTRL_ROOT9_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8500	Target Register (CCM_TARGET_ROOT10)	32	R/W	1000_0000h	5.1.7.10/556
3038_8504	Target Register (CCM_TARGET_ROOT10_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8508	Target Register (CCM_TARGET_ROOT10_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_850C	Target Register (CCM_TARGET_ROOT10_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8510	Miscellaneous Register (CCM_MISC10)	32	R/W	0000_0000h	5.1.7.14/564
3038_8514	Miscellaneous Register (CCM_MISC_ROOT10_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8518	Miscellaneous Register (CCM_MISC_ROOT10_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_851C	Miscellaneous Register (CCM_MISC_ROOT10_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8520	Post Divider Register (CCM_POST10)	32	R/W	0000_0000h	5.1.7.18/568
3038_8524	Post Divider Register (CCM_POST_ROOT10_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8528	Post Divider Register (CCM_POST_ROOT10_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_852C	Post Divider Register (CCM_POST_ROOT10_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8530	Pre Divider Register (CCM_PRE10)	32	R/W	1000_0000h	5.1.7.22/580

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8534	Pre Divider Register (CCM_PRE_ROOT10_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8538	Pre Divider Register (CCM_PRE_ROOT10_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_853C	Pre Divider Register (CCM_PRE_ROOT10_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8570	Access Control Register (CCM_ACCESS_CTRL10)	32	R/W	0000_0000h	5.1.7.26/592
3038_8574	Access Control Register (CCM_ACCESS_CTRL_ROOT10_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8578	Access Control Register (CCM_ACCESS_CTRL_ROOT10_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_857C	Access Control Register (CCM_ACCESS_CTRL_ROOT10_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8580	Target Register (CCM_TARGET_ROOT11)	32	R/W	1000_0000h	5.1.7.10/556
3038_8584	Target Register (CCM_TARGET_ROOT11_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8588	Target Register (CCM_TARGET_ROOT11_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_858C	Target Register (CCM_TARGET_ROOT11_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8590	Miscellaneous Register (CCM_MISC11)	32	R/W	0000_0000h	5.1.7.14/564
3038_8594	Miscellaneous Register (CCM_MISC_ROOT11_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8598	Miscellaneous Register (CCM_MISC_ROOT11_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_859C	Miscellaneous Register (CCM_MISC_ROOT11_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_85A0	Post Divider Register (CCM_POST11)	32	R/W	0000_0000h	5.1.7.18/568
3038_85A4	Post Divider Register (CCM_POST_ROOT11_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_85A8	Post Divider Register (CCM_POST_ROOT11_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_85AC	Post Divider Register (CCM_POST_ROOT11_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_85B0	Pre Divider Register (CCM_PRE11)	32	R/W	1000_0000h	5.1.7.22/580
3038_85B4	Pre Divider Register (CCM_PRE_ROOT11_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_85B8	Pre Divider Register (CCM_PRE_ROOT11_CLR)	32	R/W	0000_0000h	5.1.7.24/586

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_85BC	Pre Divider Register (CCM_PRE_ROOT11_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_85F0	Access Control Register (CCM_ACCESS_CTRL11)	32	R/W	0000_0000h	5.1.7.26/592
3038_85F4	Access Control Register (CCM_ACCESS_CTRL_ROOT11_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_85F8	Access Control Register (CCM_ACCESS_CTRL_ROOT11_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_85FC	Access Control Register (CCM_ACCESS_CTRL_ROOT11_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8600	Target Register (CCM_TARGET_ROOT12)	32	R/W	1000_0000h	5.1.7.10/556
3038_8604	Target Register (CCM_TARGET_ROOT12_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8608	Target Register (CCM_TARGET_ROOT12_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_860C	Target Register (CCM_TARGET_ROOT12_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8610	Miscellaneous Register (CCM_MISC12)	32	R/W	0000_0000h	5.1.7.14/564
3038_8614	Miscellaneous Register (CCM_MISC_ROOT12_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8618	Miscellaneous Register (CCM_MISC_ROOT12_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_861C	Miscellaneous Register (CCM_MISC_ROOT12_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8620	Post Divider Register (CCM_POST12)	32	R/W	0000_0000h	5.1.7.18/568
3038_8624	Post Divider Register (CCM_POST_ROOT12_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8628	Post Divider Register (CCM_POST_ROOT12_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_862C	Post Divider Register (CCM_POST_ROOT12_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8630	Pre Divider Register (CCM_PRE12)	32	R/W	1000_0000h	5.1.7.22/580
3038_8634	Pre Divider Register (CCM_PRE_ROOT12_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8638	Pre Divider Register (CCM_PRE_ROOT12_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_863C	Pre Divider Register (CCM_PRE_ROOT12_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8670	Access Control Register (CCM_ACCESS_CTRL12)	32	R/W	0000_0000h	5.1.7.26/592

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8674	Access Control Register (CCM_ACCESS_CTRL_ROOT12_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8678	Access Control Register (CCM_ACCESS_CTRL_ROOT12_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_867C	Access Control Register (CCM_ACCESS_CTRL_ROOT12_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8680	Target Register (CCM_TARGET_ROOT13)	32	R/W	1000_0000h	5.1.7.10/556
3038_8684	Target Register (CCM_TARGET_ROOT13_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8688	Target Register (CCM_TARGET_ROOT13_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_868C	Target Register (CCM_TARGET_ROOT13_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8690	Miscellaneous Register (CCM_MISC13)	32	R/W	0000_0000h	5.1.7.14/564
3038_8694	Miscellaneous Register (CCM_MISC_ROOT13_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8698	Miscellaneous Register (CCM_MISC_ROOT13_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_869C	Miscellaneous Register (CCM_MISC_ROOT13_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_86A0	Post Divider Register (CCM_POST13)	32	R/W	0000_0000h	5.1.7.18/568
3038_86A4	Post Divider Register (CCM_POST_ROOT13_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_86A8	Post Divider Register (CCM_POST_ROOT13_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_86AC	Post Divider Register (CCM_POST_ROOT13_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_86B0	Pre Divider Register (CCM_PRE13)	32	R/W	1000_0000h	5.1.7.22/580
3038_86B4	Pre Divider Register (CCM_PRE_ROOT13_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_86B8	Pre Divider Register (CCM_PRE_ROOT13_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_86BC	Pre Divider Register (CCM_PRE_ROOT13_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_86F0	Access Control Register (CCM_ACCESS_CTRL13)	32	R/W	0000_0000h	5.1.7.26/592
3038_86F4	Access Control Register (CCM_ACCESS_CTRL_ROOT13_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_86F8	Access Control Register (CCM_ACCESS_CTRL_ROOT13_CLR)	32	R/W	0000_0000h	5.1.7.28/597

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_86FC	Access Control Register (CCM_ACCESS_CTRL_ROOT13_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8700	Target Register (CCM_TARGET_ROOT14)	32	R/W	1000_0000h	5.1.7.10/556
3038_8704	Target Register (CCM_TARGET_ROOT14_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8708	Target Register (CCM_TARGET_ROOT14_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_870C	Target Register (CCM_TARGET_ROOT14_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8710	Miscellaneous Register (CCM_MISC14)	32	R/W	0000_0000h	5.1.7.14/564
3038_8714	Miscellaneous Register (CCM_MISC_ROOT14_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8718	Miscellaneous Register (CCM_MISC_ROOT14_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_871C	Miscellaneous Register (CCM_MISC_ROOT14_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8720	Post Divider Register (CCM_POST14)	32	R/W	0000_0000h	5.1.7.18/568
3038_8724	Post Divider Register (CCM_POST_ROOT14_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8728	Post Divider Register (CCM_POST_ROOT14_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_872C	Post Divider Register (CCM_POST_ROOT14_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8730	Pre Divider Register (CCM_PRE14)	32	R/W	1000_0000h	5.1.7.22/580
3038_8734	Pre Divider Register (CCM_PRE_ROOT14_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8738	Pre Divider Register (CCM_PRE_ROOT14_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_873C	Pre Divider Register (CCM_PRE_ROOT14_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8770	Access Control Register (CCM_ACCESS_CTRL14)	32	R/W	0000_0000h	5.1.7.26/592
3038_8774	Access Control Register (CCM_ACCESS_CTRL_ROOT14_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8778	Access Control Register (CCM_ACCESS_CTRL_ROOT14_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_877C	Access Control Register (CCM_ACCESS_CTRL_ROOT14_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8780	Target Register (CCM_TARGET_ROOT15)	32	R/W	1000_0000h	5.1.7.10/556

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8784	Target Register (CCM_TARGET_ROOT15_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8788	Target Register (CCM_TARGET_ROOT15_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_878C	Target Register (CCM_TARGET_ROOT15_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8790	Miscellaneous Register (CCM_MISC15)	32	R/W	0000_0000h	5.1.7.14/564
3038_8794	Miscellaneous Register (CCM_MISC_ROOT15_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8798	Miscellaneous Register (CCM_MISC_ROOT15_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_879C	Miscellaneous Register (CCM_MISC_ROOT15_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_87A0	Post Divider Register (CCM_POST15)	32	R/W	0000_0000h	5.1.7.18/568
3038_87A4	Post Divider Register (CCM_POST_ROOT15_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_87A8	Post Divider Register (CCM_POST_ROOT15_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_87AC	Post Divider Register (CCM_POST_ROOT15_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_87B0	Pre Divider Register (CCM_PRE15)	32	R/W	1000_0000h	5.1.7.22/580
3038_87B4	Pre Divider Register (CCM_PRE_ROOT15_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_87B8	Pre Divider Register (CCM_PRE_ROOT15_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_87BC	Pre Divider Register (CCM_PRE_ROOT15_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_87F0	Access Control Register (CCM_ACCESS_CTRL15)	32	R/W	0000_0000h	5.1.7.26/592
3038_87F4	Access Control Register (CCM_ACCESS_CTRL_ROOT15_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_87F8	Access Control Register (CCM_ACCESS_CTRL_ROOT15_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_87FC	Access Control Register (CCM_ACCESS_CTRL_ROOT15_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8800	Target Register (CCM_TARGET_ROOT16)	32	R/W	1000_0000h	5.1.7.10/556
3038_8804	Target Register (CCM_TARGET_ROOT16_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8808	Target Register (CCM_TARGET_ROOT16_CLR)	32	R/W	0000_0000h	5.1.7.12/560

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_880C	Target Register (CCM_TARGET_ROOT16_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8810	Miscellaneous Register (CCM_MISC16)	32	R/W	0000_0000h	5.1.7.14/564
3038_8814	Miscellaneous Register (CCM_MISC_ROOT16_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8818	Miscellaneous Register (CCM_MISC_ROOT16_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_881C	Miscellaneous Register (CCM_MISC_ROOT16_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8820	Post Divider Register (CCM_POST16)	32	R/W	0000_0000h	5.1.7.18/568
3038_8824	Post Divider Register (CCM_POST_ROOT16_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8828	Post Divider Register (CCM_POST_ROOT16_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_882C	Post Divider Register (CCM_POST_ROOT16_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8830	Pre Divider Register (CCM_PRE16)	32	R/W	1000_0000h	5.1.7.22/580
3038_8834	Pre Divider Register (CCM_PRE_ROOT16_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8838	Pre Divider Register (CCM_PRE_ROOT16_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_883C	Pre Divider Register (CCM_PRE_ROOT16_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8870	Access Control Register (CCM_ACCESS_CTRL16)	32	R/W	0000_0000h	5.1.7.26/592
3038_8874	Access Control Register (CCM_ACCESS_CTRL_ROOT16_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8878	Access Control Register (CCM_ACCESS_CTRL_ROOT16_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_887C	Access Control Register (CCM_ACCESS_CTRL_ROOT16_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8880	Target Register (CCM_TARGET_ROOT17)	32	R/W	1000_0000h	5.1.7.10/556
3038_8884	Target Register (CCM_TARGET_ROOT17_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8888	Target Register (CCM_TARGET_ROOT17_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_888C	Target Register (CCM_TARGET_ROOT17_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8890	Miscellaneous Register (CCM_MISC17)	32	R/W	0000_0000h	5.1.7.14/564

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8894	Miscellaneous Register (CCM_MISC_ROOT17_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8898	Miscellaneous Register (CCM_MISC_ROOT17_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_889C	Miscellaneous Register (CCM_MISC_ROOT17_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_88A0	Post Divider Register (CCM_POST17)	32	R/W	0000_0000h	5.1.7.18/568
3038_88A4	Post Divider Register (CCM_POST_ROOT17_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_88A8	Post Divider Register (CCM_POST_ROOT17_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_88AC	Post Divider Register (CCM_POST_ROOT17_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_88B0	Pre Divider Register (CCM_PRE17)	32	R/W	1000_0000h	5.1.7.22/580
3038_88B4	Pre Divider Register (CCM_PRE_ROOT17_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_88B8	Pre Divider Register (CCM_PRE_ROOT17_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_88BC	Pre Divider Register (CCM_PRE_ROOT17_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_88F0	Access Control Register (CCM_ACCESS_CTRL17)	32	R/W	0000_0000h	5.1.7.26/592
3038_88F4	Access Control Register (CCM_ACCESS_CTRL_ROOT17_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_88F8	Access Control Register (CCM_ACCESS_CTRL_ROOT17_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_88FC	Access Control Register (CCM_ACCESS_CTRL_ROOT17_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8900	Target Register (CCM_TARGET_ROOT18)	32	R/W	1000_0000h	5.1.7.10/556
3038_8904	Target Register (CCM_TARGET_ROOT18_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8908	Target Register (CCM_TARGET_ROOT18_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_890C	Target Register (CCM_TARGET_ROOT18_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8910	Miscellaneous Register (CCM_MISC18)	32	R/W	0000_0000h	5.1.7.14/564
3038_8914	Miscellaneous Register (CCM_MISC_ROOT18_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8918	Miscellaneous Register (CCM_MISC_ROOT18_CLR)	32	R/W	0000_0000h	5.1.7.16/566

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_891C	Miscellaneous Register (CCM_MISC_ROOT18_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8920	Post Divider Register (CCM_POST18)	32	R/W	0000_0000h	5.1.7.18/568
3038_8924	Post Divider Register (CCM_POST_ROOT18_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8928	Post Divider Register (CCM_POST_ROOT18_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_892C	Post Divider Register (CCM_POST_ROOT18_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8930	Pre Divider Register (CCM_PRE18)	32	R/W	1000_0000h	5.1.7.22/580
3038_8934	Pre Divider Register (CCM_PRE_ROOT18_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8938	Pre Divider Register (CCM_PRE_ROOT18_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_893C	Pre Divider Register (CCM_PRE_ROOT18_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8970	Access Control Register (CCM_ACCESS_CTRL18)	32	R/W	0000_0000h	5.1.7.26/592
3038_8974	Access Control Register (CCM_ACCESS_CTRL_ROOT18_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8978	Access Control Register (CCM_ACCESS_CTRL_ROOT18_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_897C	Access Control Register (CCM_ACCESS_CTRL_ROOT18_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8980	Target Register (CCM_TARGET_ROOT19)	32	R/W	1000_0000h	5.1.7.10/556
3038_8984	Target Register (CCM_TARGET_ROOT19_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8988	Target Register (CCM_TARGET_ROOT19_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_898C	Target Register (CCM_TARGET_ROOT19_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8990	Miscellaneous Register (CCM_MISC19)	32	R/W	0000_0000h	5.1.7.14/564
3038_8994	Miscellaneous Register (CCM_MISC_ROOT19_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8998	Miscellaneous Register (CCM_MISC_ROOT19_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_899C	Miscellaneous Register (CCM_MISC_ROOT19_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_89A0	Post Divider Register (CCM_POST19)	32	R/W	0000_0000h	5.1.7.18/568

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_89A4	Post Divider Register (CCM_POST_ROOT19_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_89A8	Post Divider Register (CCM_POST_ROOT19_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_89AC	Post Divider Register (CCM_POST_ROOT19_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_89B0	Pre Divider Register (CCM_PRE19)	32	R/W	1000_0000h	5.1.7.22/580
3038_89B4	Pre Divider Register (CCM_PRE_ROOT19_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_89B8	Pre Divider Register (CCM_PRE_ROOT19_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_89BC	Pre Divider Register (CCM_PRE_ROOT19_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_89F0	Access Control Register (CCM_ACCESS_CTRL19)	32	R/W	0000_0000h	5.1.7.26/592
3038_89F4	Access Control Register (CCM_ACCESS_CTRL_ROOT19_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_89F8	Access Control Register (CCM_ACCESS_CTRL_ROOT19_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_89FC	Access Control Register (CCM_ACCESS_CTRL_ROOT19_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8A00	Target Register (CCM_TARGET_ROOT20)	32	R/W	1000_0000h	5.1.7.10/556
3038_8A04	Target Register (CCM_TARGET_ROOT20_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8A08	Target Register (CCM_TARGET_ROOT20_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_8A0C	Target Register (CCM_TARGET_ROOT20_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8A10	Miscellaneous Register (CCM_MISC20)	32	R/W	0000_0000h	5.1.7.14/564
3038_8A14	Miscellaneous Register (CCM_MISC_ROOT20_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8A18	Miscellaneous Register (CCM_MISC_ROOT20_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_8A1C	Miscellaneous Register (CCM_MISC_ROOT20_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8A20	Post Divider Register (CCM_POST20)	32	R/W	0000_0000h	5.1.7.18/568
3038_8A24	Post Divider Register (CCM_POST_ROOT20_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8A28	Post Divider Register (CCM_POST_ROOT20_CLR)	32	R/W	0000_0000h	5.1.7.20/574

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8A2C	Post Divider Register (CCM_POST_ROOT20_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8A30	Pre Divider Register (CCM_PRE20)	32	R/W	1000_0000h	5.1.7.22/580
3038_8A34	Pre Divider Register (CCM_PRE_ROOT20_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8A38	Pre Divider Register (CCM_PRE_ROOT20_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_8A3C	Pre Divider Register (CCM_PRE_ROOT20_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8A70	Access Control Register (CCM_ACCESS_CTRL20)	32	R/W	0000_0000h	5.1.7.26/592
3038_8A74	Access Control Register (CCM_ACCESS_CTRL_ROOT20_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8A78	Access Control Register (CCM_ACCESS_CTRL_ROOT20_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_8A7C	Access Control Register (CCM_ACCESS_CTRL_ROOT20_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8A80	Target Register (CCM_TARGET_ROOT21)	32	R/W	1000_0000h	5.1.7.10/556
3038_8A84	Target Register (CCM_TARGET_ROOT21_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8A88	Target Register (CCM_TARGET_ROOT21_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_8A8C	Target Register (CCM_TARGET_ROOT21_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8A90	Miscellaneous Register (CCM_MISC21)	32	R/W	0000_0000h	5.1.7.14/564
3038_8A94	Miscellaneous Register (CCM_MISC_ROOT21_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8A98	Miscellaneous Register (CCM_MISC_ROOT21_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_8A9C	Miscellaneous Register (CCM_MISC_ROOT21_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8AA0	Post Divider Register (CCM_POST21)	32	R/W	0000_0000h	5.1.7.18/568
3038_8AA4	Post Divider Register (CCM_POST_ROOT21_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8AA8	Post Divider Register (CCM_POST_ROOT21_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_8AAC	Post Divider Register (CCM_POST_ROOT21_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8AB0	Pre Divider Register (CCM_PRE21)	32	R/W	1000_0000h	5.1.7.22/580

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_8AB4	Pre Divider Register (CCM_PRE_ROOT21_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8AB8	Pre Divider Register (CCM_PRE_ROOT21_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_8ABC	Pre Divider Register (CCM_PRE_ROOT21_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8AF0	Access Control Register (CCM_ACCESS_CTRL21)	32	R/W	0000_0000h	5.1.7.26/592
3038_8AF4	Access Control Register (CCM_ACCESS_CTRL_ROOT21_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8AF8	Access Control Register (CCM_ACCESS_CTRL_ROOT21_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_8AFC	Access Control Register (CCM_ACCESS_CTRL_ROOT21_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8B00	Target Register (CCM_TARGET_ROOT22)	32	R/W	1000_0000h	5.1.7.10/556
3038_8B04	Target Register (CCM_TARGET_ROOT22_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8B08	Target Register (CCM_TARGET_ROOT22_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_8B0C	Target Register (CCM_TARGET_ROOT22_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8B10	Miscellaneous Register (CCM_MISC22)	32	R/W	0000_0000h	5.1.7.14/564
3038_8B14	Miscellaneous Register (CCM_MISC_ROOT22_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8B18	Miscellaneous Register (CCM_MISC_ROOT22_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_8B1C	Miscellaneous Register (CCM_MISC_ROOT22_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8B20	Post Divider Register (CCM_POST22)	32	R/W	0000_0000h	5.1.7.18/568
3038_8B24	Post Divider Register (CCM_POST_ROOT22_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8B28	Post Divider Register (CCM_POST_ROOT22_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_8B2C	Post Divider Register (CCM_POST_ROOT22_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8B30	Pre Divider Register (CCM_PRE22)	32	R/W	1000_0000h	5.1.7.22/580
3038_8B34	Pre Divider Register (CCM_PRE_ROOT22_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8B38	Pre Divider Register (CCM_PRE_ROOT22_CLR)	32	R/W	0000_0000h	5.1.7.24/586

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8B3C	Pre Divider Register (CCM_PRE_ROOT22_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8B70	Access Control Register (CCM_ACCESS_CTRL22)	32	R/W	0000_0000h	5.1.7.26/592
3038_8B74	Access Control Register (CCM_ACCESS_CTRL_ROOT22_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8B78	Access Control Register (CCM_ACCESS_CTRL_ROOT22_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_8B7C	Access Control Register (CCM_ACCESS_CTRL_ROOT22_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8B80	Target Register (CCM_TARGET_ROOT23)	32	R/W	1000_0000h	5.1.7.10/556
3038_8B84	Target Register (CCM_TARGET_ROOT23_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8B88	Target Register (CCM_TARGET_ROOT23_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_8B8C	Target Register (CCM_TARGET_ROOT23_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8B90	Miscellaneous Register (CCM_MISC23)	32	R/W	0000_0000h	5.1.7.14/564
3038_8B94	Miscellaneous Register (CCM_MISC_ROOT23_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8B98	Miscellaneous Register (CCM_MISC_ROOT23_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_8B9C	Miscellaneous Register (CCM_MISC_ROOT23_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8BA0	Post Divider Register (CCM_POST23)	32	R/W	0000_0000h	5.1.7.18/568
3038_8BA4	Post Divider Register (CCM_POST_ROOT23_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8BA8	Post Divider Register (CCM_POST_ROOT23_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_8BAC	Post Divider Register (CCM_POST_ROOT23_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8BB0	Pre Divider Register (CCM_PRE23)	32	R/W	1000_0000h	5.1.7.22/580
3038_8BB4	Pre Divider Register (CCM_PRE_ROOT23_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8BB8	Pre Divider Register (CCM_PRE_ROOT23_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_8BBC	Pre Divider Register (CCM_PRE_ROOT23_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8BF0	Access Control Register (CCM_ACCESS_CTRL23)	32	R/W	0000_0000h	5.1.7.26/592

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8BF4	Access Control Register (CCM_ACCESS_CTRL_ROOT23_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8BF8	Access Control Register (CCM_ACCESS_CTRL_ROOT23_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_8BFC	Access Control Register (CCM_ACCESS_CTRL_ROOT23_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8C00	Target Register (CCM_TARGET_ROOT24)	32	R/W	1000_0000h	5.1.7.10/556
3038_8C04	Target Register (CCM_TARGET_ROOT24_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8C08	Target Register (CCM_TARGET_ROOT24_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_8C0C	Target Register (CCM_TARGET_ROOT24_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8C10	Miscellaneous Register (CCM_MISC24)	32	R/W	0000_0000h	5.1.7.14/564
3038_8C14	Miscellaneous Register (CCM_MISC_ROOT24_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8C18	Miscellaneous Register (CCM_MISC_ROOT24_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_8C1C	Miscellaneous Register (CCM_MISC_ROOT24_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8C20	Post Divider Register (CCM_POST24)	32	R/W	0000_0000h	5.1.7.18/568
3038_8C24	Post Divider Register (CCM_POST_ROOT24_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8C28	Post Divider Register (CCM_POST_ROOT24_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_8C2C	Post Divider Register (CCM_POST_ROOT24_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8C30	Pre Divider Register (CCM_PRE24)	32	R/W	1000_0000h	5.1.7.22/580
3038_8C34	Pre Divider Register (CCM_PRE_ROOT24_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8C38	Pre Divider Register (CCM_PRE_ROOT24_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_8C3C	Pre Divider Register (CCM_PRE_ROOT24_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8C70	Access Control Register (CCM_ACCESS_CTRL24)	32	R/W	0000_0000h	5.1.7.26/592
3038_8C74	Access Control Register (CCM_ACCESS_CTRL_ROOT24_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8C78	Access Control Register (CCM_ACCESS_CTRL_ROOT24_CLR)	32	R/W	0000_0000h	5.1.7.28/597

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8C7C	Access Control Register (CCM_ACCESS_CTRL_ROOT24_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8C80	Target Register (CCM_TARGET_ROOT25)	32	R/W	1000_0000h	5.1.7.10/556
3038_8C84	Target Register (CCM_TARGET_ROOT25_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8C88	Target Register (CCM_TARGET_ROOT25_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_8C8C	Target Register (CCM_TARGET_ROOT25_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8C90	Miscellaneous Register (CCM_MISC25)	32	R/W	0000_0000h	5.1.7.14/564
3038_8C94	Miscellaneous Register (CCM_MISC_ROOT25_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8C98	Miscellaneous Register (CCM_MISC_ROOT25_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_8C9C	Miscellaneous Register (CCM_MISC_ROOT25_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8CA0	Post Divider Register (CCM_POST25)	32	R/W	0000_0000h	5.1.7.18/568
3038_8CA4	Post Divider Register (CCM_POST_ROOT25_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8CA8	Post Divider Register (CCM_POST_ROOT25_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_8CAC	Post Divider Register (CCM_POST_ROOT25_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8CB0	Pre Divider Register (CCM_PRE25)	32	R/W	1000_0000h	5.1.7.22/580
3038_8CB4	Pre Divider Register (CCM_PRE_ROOT25_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8CB8	Pre Divider Register (CCM_PRE_ROOT25_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_8CBC	Pre Divider Register (CCM_PRE_ROOT25_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8CF0	Access Control Register (CCM_ACCESS_CTRL25)	32	R/W	0000_0000h	5.1.7.26/592
3038_8CF4	Access Control Register (CCM_ACCESS_CTRL_ROOT25_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8CF8	Access Control Register (CCM_ACCESS_CTRL_ROOT25_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_8CFC	Access Control Register (CCM_ACCESS_CTRL_ROOT25_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8D00	Target Register (CCM_TARGET_ROOT26)	32	R/W	1000_0000h	5.1.7.10/556

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8D04	Target Register (CCM_TARGET_ROOT26_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8D08	Target Register (CCM_TARGET_ROOT26_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_8D0C	Target Register (CCM_TARGET_ROOT26_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8D10	Miscellaneous Register (CCM_MISC26)	32	R/W	0000_0000h	5.1.7.14/564
3038_8D14	Miscellaneous Register (CCM_MISC_ROOT26_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8D18	Miscellaneous Register (CCM_MISC_ROOT26_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_8D1C	Miscellaneous Register (CCM_MISC_ROOT26_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8D20	Post Divider Register (CCM_POST26)	32	R/W	0000_0000h	5.1.7.18/568
3038_8D24	Post Divider Register (CCM_POST_ROOT26_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8D28	Post Divider Register (CCM_POST_ROOT26_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_8D2C	Post Divider Register (CCM_POST_ROOT26_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8D30	Pre Divider Register (CCM_PRE26)	32	R/W	1000_0000h	5.1.7.22/580
3038_8D34	Pre Divider Register (CCM_PRE_ROOT26_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8D38	Pre Divider Register (CCM_PRE_ROOT26_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_8D3C	Pre Divider Register (CCM_PRE_ROOT26_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8D70	Access Control Register (CCM_ACCESS_CTRL26)	32	R/W	0000_0000h	5.1.7.26/592
3038_8D74	Access Control Register (CCM_ACCESS_CTRL_ROOT26_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8D78	Access Control Register (CCM_ACCESS_CTRL_ROOT26_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_8D7C	Access Control Register (CCM_ACCESS_CTRL_ROOT26_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8D80	Target Register (CCM_TARGET_ROOT27)	32	R/W	1000_0000h	5.1.7.10/556
3038_8D84	Target Register (CCM_TARGET_ROOT27_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8D88	Target Register (CCM_TARGET_ROOT27_CLR)	32	R/W	0000_0000h	5.1.7.12/560

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8D8C	Target Register (CCM_TARGET_ROOT27_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8D90	Miscellaneous Register (CCM_MISC27)	32	R/W	0000_0000h	5.1.7.14/564
3038_8D94	Miscellaneous Register (CCM_MISC_ROOT27_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8D98	Miscellaneous Register (CCM_MISC_ROOT27_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_8D9C	Miscellaneous Register (CCM_MISC_ROOT27_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8DA0	Post Divider Register (CCM_POST27)	32	R/W	0000_0000h	5.1.7.18/568
3038_8DA4	Post Divider Register (CCM_POST_ROOT27_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8DA8	Post Divider Register (CCM_POST_ROOT27_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_8DAC	Post Divider Register (CCM_POST_ROOT27_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8DB0	Pre Divider Register (CCM_PRE27)	32	R/W	1000_0000h	5.1.7.22/580
3038_8DB4	Pre Divider Register (CCM_PRE_ROOT27_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8DB8	Pre Divider Register (CCM_PRE_ROOT27_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_8DBC	Pre Divider Register (CCM_PRE_ROOT27_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8DF0	Access Control Register (CCM_ACCESS_CTRL27)	32	R/W	0000_0000h	5.1.7.26/592
3038_8DF4	Access Control Register (CCM_ACCESS_CTRL_ROOT27_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8DF8	Access Control Register (CCM_ACCESS_CTRL_ROOT27_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_8DFC	Access Control Register (CCM_ACCESS_CTRL_ROOT27_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8E00	Target Register (CCM_TARGET_ROOT28)	32	R/W	1000_0000h	5.1.7.10/556
3038_8E04	Target Register (CCM_TARGET_ROOT28_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8E08	Target Register (CCM_TARGET_ROOT28_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_8E0C	Target Register (CCM_TARGET_ROOT28_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8E10	Miscellaneous Register (CCM_MISC28)	32	R/W	0000_0000h	5.1.7.14/564

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8E14	Miscellaneous Register (CCM_MISC_ROOT28_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8E18	Miscellaneous Register (CCM_MISC_ROOT28_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_8E1C	Miscellaneous Register (CCM_MISC_ROOT28_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8E20	Post Divider Register (CCM_POST28)	32	R/W	0000_0000h	5.1.7.18/568
3038_8E24	Post Divider Register (CCM_POST_ROOT28_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8E28	Post Divider Register (CCM_POST_ROOT28_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_8E2C	Post Divider Register (CCM_POST_ROOT28_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8E30	Pre Divider Register (CCM_PRE28)	32	R/W	1000_0000h	5.1.7.22/580
3038_8E34	Pre Divider Register (CCM_PRE_ROOT28_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8E38	Pre Divider Register (CCM_PRE_ROOT28_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_8E3C	Pre Divider Register (CCM_PRE_ROOT28_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8E70	Access Control Register (CCM_ACCESS_CTRL28)	32	R/W	0000_0000h	5.1.7.26/592
3038_8E74	Access Control Register (CCM_ACCESS_CTRL_ROOT28_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8E78	Access Control Register (CCM_ACCESS_CTRL_ROOT28_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_8E7C	Access Control Register (CCM_ACCESS_CTRL_ROOT28_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8E80	Target Register (CCM_TARGET_ROOT29)	32	R/W	1000_0000h	5.1.7.10/556
3038_8E84	Target Register (CCM_TARGET_ROOT29_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8E88	Target Register (CCM_TARGET_ROOT29_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_8E8C	Target Register (CCM_TARGET_ROOT29_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8E90	Miscellaneous Register (CCM_MISC29)	32	R/W	0000_0000h	5.1.7.14/564
3038_8E94	Miscellaneous Register (CCM_MISC_ROOT29_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8E98	Miscellaneous Register (CCM_MISC_ROOT29_CLR)	32	R/W	0000_0000h	5.1.7.16/566

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8E9C	Miscellaneous Register (CCM_MISC_ROOT29_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8EA0	Post Divider Register (CCM_POST29)	32	R/W	0000_0000h	5.1.7.18/568
3038_8EA4	Post Divider Register (CCM_POST_ROOT29_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8EA8	Post Divider Register (CCM_POST_ROOT29_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_8EAC	Post Divider Register (CCM_POST_ROOT29_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8EB0	Pre Divider Register (CCM_PRE29)	32	R/W	1000_0000h	5.1.7.22/580
3038_8EB4	Pre Divider Register (CCM_PRE_ROOT29_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8EB8	Pre Divider Register (CCM_PRE_ROOT29_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_8EBC	Pre Divider Register (CCM_PRE_ROOT29_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8EF0	Access Control Register (CCM_ACCESS_CTRL29)	32	R/W	0000_0000h	5.1.7.26/592
3038_8EF4	Access Control Register (CCM_ACCESS_CTRL_ROOT29_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8EF8	Access Control Register (CCM_ACCESS_CTRL_ROOT29_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_8EFC	Access Control Register (CCM_ACCESS_CTRL_ROOT29_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8F00	Target Register (CCM_TARGET_ROOT30)	32	R/W	1000_0000h	5.1.7.10/556
3038_8F04	Target Register (CCM_TARGET_ROOT30_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8F08	Target Register (CCM_TARGET_ROOT30_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_8F0C	Target Register (CCM_TARGET_ROOT30_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8F10	Miscellaneous Register (CCM_MISC30)	32	R/W	0000_0000h	5.1.7.14/564
3038_8F14	Miscellaneous Register (CCM_MISC_ROOT30_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8F18	Miscellaneous Register (CCM_MISC_ROOT30_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_8F1C	Miscellaneous Register (CCM_MISC_ROOT30_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8F20	Post Divider Register (CCM_POST30)	32	R/W	0000_0000h	5.1.7.18/568

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8F24	Post Divider Register (CCM_POST_ROOT30_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8F28	Post Divider Register (CCM_POST_ROOT30_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_8F2C	Post Divider Register (CCM_POST_ROOT30_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8F30	Pre Divider Register (CCM_PRE30)	32	R/W	1000_0000h	5.1.7.22/580
3038_8F34	Pre Divider Register (CCM_PRE_ROOT30_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8F38	Pre Divider Register (CCM_PRE_ROOT30_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_8F3C	Pre Divider Register (CCM_PRE_ROOT30_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8F70	Access Control Register (CCM_ACCESS_CTRL30)	32	R/W	0000_0000h	5.1.7.26/592
3038_8F74	Access Control Register (CCM_ACCESS_CTRL_ROOT30_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8F78	Access Control Register (CCM_ACCESS_CTRL_ROOT30_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_8F7C	Access Control Register (CCM_ACCESS_CTRL_ROOT30_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_8F80	Target Register (CCM_TARGET_ROOT31)	32	R/W	1000_0000h	5.1.7.10/556
3038_8F84	Target Register (CCM_TARGET_ROOT31_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_8F88	Target Register (CCM_TARGET_ROOT31_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_8F8C	Target Register (CCM_TARGET_ROOT31_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_8F90	Miscellaneous Register (CCM_MISC31)	32	R/W	0000_0000h	5.1.7.14/564
3038_8F94	Miscellaneous Register (CCM_MISC_ROOT31_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_8F98	Miscellaneous Register (CCM_MISC_ROOT31_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_8F9C	Miscellaneous Register (CCM_MISC_ROOT31_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_8FA0	Post Divider Register (CCM_POST31)	32	R/W	0000_0000h	5.1.7.18/568
3038_8FA4	Post Divider Register (CCM_POST_ROOT31_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_8FA8	Post Divider Register (CCM_POST_ROOT31_CLR)	32	R/W	0000_0000h	5.1.7.20/574

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_8FAC	Post Divider Register (CCM_POST_ROOT31_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_8FB0	Pre Divider Register (CCM_PRE31)	32	R/W	1000_0000h	5.1.7.22/580
3038_8FB4	Pre Divider Register (CCM_PRE_ROOT31_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_8FB8	Pre Divider Register (CCM_PRE_ROOT31_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_8FBC	Pre Divider Register (CCM_PRE_ROOT31_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_8FF0	Access Control Register (CCM_ACCESS_CTRL31)	32	R/W	0000_0000h	5.1.7.26/592
3038_8FF4	Access Control Register (CCM_ACCESS_CTRL_ROOT31_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_8FF8	Access Control Register (CCM_ACCESS_CTRL_ROOT31_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_8FFC	Access Control Register (CCM_ACCESS_CTRL_ROOT31_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9000	Target Register (CCM_TARGET_ROOT32)	32	R/W	1000_0000h	5.1.7.10/556
3038_9004	Target Register (CCM_TARGET_ROOT32_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9008	Target Register (CCM_TARGET_ROOT32_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_900C	Target Register (CCM_TARGET_ROOT32_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9010	Miscellaneous Register (CCM_MISC32)	32	R/W	0000_0000h	5.1.7.14/564
3038_9014	Miscellaneous Register (CCM_MISC_ROOT32_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9018	Miscellaneous Register (CCM_MISC_ROOT32_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_901C	Miscellaneous Register (CCM_MISC_ROOT32_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9020	Post Divider Register (CCM_POST32)	32	R/W	0000_0000h	5.1.7.18/568
3038_9024	Post Divider Register (CCM_POST_ROOT32_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9028	Post Divider Register (CCM_POST_ROOT32_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_902C	Post Divider Register (CCM_POST_ROOT32_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9030	Pre Divider Register (CCM_PRE32)	32	R/W	1000_0000h	5.1.7.22/580

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9034	Pre Divider Register (CCM_PRE_ROOT32_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9038	Pre Divider Register (CCM_PRE_ROOT32_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_903C	Pre Divider Register (CCM_PRE_ROOT32_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9070	Access Control Register (CCM_ACCESS_CTRL32)	32	R/W	0000_0000h	5.1.7.26/592
3038_9074	Access Control Register (CCM_ACCESS_CTRL_ROOT32_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9078	Access Control Register (CCM_ACCESS_CTRL_ROOT32_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_907C	Access Control Register (CCM_ACCESS_CTRL_ROOT32_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9080	Target Register (CCM_TARGET_ROOT33)	32	R/W	1000_0000h	5.1.7.10/556
3038_9084	Target Register (CCM_TARGET_ROOT33_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9088	Target Register (CCM_TARGET_ROOT33_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_908C	Target Register (CCM_TARGET_ROOT33_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9090	Miscellaneous Register (CCM_MISC33)	32	R/W	0000_0000h	5.1.7.14/564
3038_9094	Miscellaneous Register (CCM_MISC_ROOT33_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9098	Miscellaneous Register (CCM_MISC_ROOT33_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_909C	Miscellaneous Register (CCM_MISC_ROOT33_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_90A0	Post Divider Register (CCM_POST33)	32	R/W	0000_0000h	5.1.7.18/568
3038_90A4	Post Divider Register (CCM_POST_ROOT33_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_90A8	Post Divider Register (CCM_POST_ROOT33_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_90AC	Post Divider Register (CCM_POST_ROOT33_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_90B0	Pre Divider Register (CCM_PRE33)	32	R/W	1000_0000h	5.1.7.22/580
3038_90B4	Pre Divider Register (CCM_PRE_ROOT33_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_90B8	Pre Divider Register (CCM_PRE_ROOT33_CLR)	32	R/W	0000_0000h	5.1.7.24/586

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_90BC	Pre Divider Register (CCM_PRE_ROOT33_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_90F0	Access Control Register (CCM_ACCESS_CTRL33)	32	R/W	0000_0000h	5.1.7.26/592
3038_90F4	Access Control Register (CCM_ACCESS_CTRL_ROOT33_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_90F8	Access Control Register (CCM_ACCESS_CTRL_ROOT33_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_90FC	Access Control Register (CCM_ACCESS_CTRL_ROOT33_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9100	Target Register (CCM_TARGET_ROOT34)	32	R/W	1000_0000h	5.1.7.10/556
3038_9104	Target Register (CCM_TARGET_ROOT34_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9108	Target Register (CCM_TARGET_ROOT34_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_910C	Target Register (CCM_TARGET_ROOT34_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9110	Miscellaneous Register (CCM_MISC34)	32	R/W	0000_0000h	5.1.7.14/564
3038_9114	Miscellaneous Register (CCM_MISC_ROOT34_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9118	Miscellaneous Register (CCM_MISC_ROOT34_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_911C	Miscellaneous Register (CCM_MISC_ROOT34_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9120	Post Divider Register (CCM_POST34)	32	R/W	0000_0000h	5.1.7.18/568
3038_9124	Post Divider Register (CCM_POST_ROOT34_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9128	Post Divider Register (CCM_POST_ROOT34_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_912C	Post Divider Register (CCM_POST_ROOT34_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9130	Pre Divider Register (CCM_PRE34)	32	R/W	1000_0000h	5.1.7.22/580
3038_9134	Pre Divider Register (CCM_PRE_ROOT34_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9138	Pre Divider Register (CCM_PRE_ROOT34_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_913C	Pre Divider Register (CCM_PRE_ROOT34_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9170	Access Control Register (CCM_ACCESS_CTRL34)	32	R/W	0000_0000h	5.1.7.26/592

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9174	Access Control Register (CCM_ACCESS_CTRL_ROOT34_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9178	Access Control Register (CCM_ACCESS_CTRL_ROOT34_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_917C	Access Control Register (CCM_ACCESS_CTRL_ROOT34_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9180	Target Register (CCM_TARGET_ROOT35)	32	R/W	1000_0000h	5.1.7.10/556
3038_9184	Target Register (CCM_TARGET_ROOT35_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9188	Target Register (CCM_TARGET_ROOT35_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_918C	Target Register (CCM_TARGET_ROOT35_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9190	Miscellaneous Register (CCM_MISC35)	32	R/W	0000_0000h	5.1.7.14/564
3038_9194	Miscellaneous Register (CCM_MISC_ROOT35_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9198	Miscellaneous Register (CCM_MISC_ROOT35_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_919C	Miscellaneous Register (CCM_MISC_ROOT35_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_91A0	Post Divider Register (CCM_POST35)	32	R/W	0000_0000h	5.1.7.18/568
3038_91A4	Post Divider Register (CCM_POST_ROOT35_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_91A8	Post Divider Register (CCM_POST_ROOT35_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_91AC	Post Divider Register (CCM_POST_ROOT35_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_91B0	Pre Divider Register (CCM_PRE35)	32	R/W	1000_0000h	5.1.7.22/580
3038_91B4	Pre Divider Register (CCM_PRE_ROOT35_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_91B8	Pre Divider Register (CCM_PRE_ROOT35_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_91BC	Pre Divider Register (CCM_PRE_ROOT35_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_91F0	Access Control Register (CCM_ACCESS_CTRL35)	32	R/W	0000_0000h	5.1.7.26/592
3038_91F4	Access Control Register (CCM_ACCESS_CTRL_ROOT35_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_91F8	Access Control Register (CCM_ACCESS_CTRL_ROOT35_CLR)	32	R/W	0000_0000h	5.1.7.28/597

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_91FC	Access Control Register (CCM_ACCESS_CTRL_ROOT35_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9200	Target Register (CCM_TARGET_ROOT36)	32	R/W	1000_0000h	5.1.7.10/556
3038_9204	Target Register (CCM_TARGET_ROOT36_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9208	Target Register (CCM_TARGET_ROOT36_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_920C	Target Register (CCM_TARGET_ROOT36_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9210	Miscellaneous Register (CCM_MISC36)	32	R/W	0000_0000h	5.1.7.14/564
3038_9214	Miscellaneous Register (CCM_MISC_ROOT36_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9218	Miscellaneous Register (CCM_MISC_ROOT36_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_921C	Miscellaneous Register (CCM_MISC_ROOT36_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9220	Post Divider Register (CCM_POST36)	32	R/W	0000_0000h	5.1.7.18/568
3038_9224	Post Divider Register (CCM_POST_ROOT36_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9228	Post Divider Register (CCM_POST_ROOT36_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_922C	Post Divider Register (CCM_POST_ROOT36_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9230	Pre Divider Register (CCM_PRE36)	32	R/W	1000_0000h	5.1.7.22/580
3038_9234	Pre Divider Register (CCM_PRE_ROOT36_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9238	Pre Divider Register (CCM_PRE_ROOT36_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_923C	Pre Divider Register (CCM_PRE_ROOT36_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9270	Access Control Register (CCM_ACCESS_CTRL36)	32	R/W	0000_0000h	5.1.7.26/592
3038_9274	Access Control Register (CCM_ACCESS_CTRL_ROOT36_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9278	Access Control Register (CCM_ACCESS_CTRL_ROOT36_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_927C	Access Control Register (CCM_ACCESS_CTRL_ROOT36_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9280	Target Register (CCM_TARGET_ROOT37)	32	R/W	1000_0000h	5.1.7.10/556

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9284	Target Register (CCM_TARGET_ROOT37_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9288	Target Register (CCM_TARGET_ROOT37_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_928C	Target Register (CCM_TARGET_ROOT37_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9290	Miscellaneous Register (CCM_MISC37)	32	R/W	0000_0000h	5.1.7.14/564
3038_9294	Miscellaneous Register (CCM_MISC_ROOT37_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9298	Miscellaneous Register (CCM_MISC_ROOT37_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_929C	Miscellaneous Register (CCM_MISC_ROOT37_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_92A0	Post Divider Register (CCM_POST37)	32	R/W	0000_0000h	5.1.7.18/568
3038_92A4	Post Divider Register (CCM_POST_ROOT37_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_92A8	Post Divider Register (CCM_POST_ROOT37_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_92AC	Post Divider Register (CCM_POST_ROOT37_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_92B0	Pre Divider Register (CCM_PRE37)	32	R/W	1000_0000h	5.1.7.22/580
3038_92B4	Pre Divider Register (CCM_PRE_ROOT37_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_92B8	Pre Divider Register (CCM_PRE_ROOT37_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_92BC	Pre Divider Register (CCM_PRE_ROOT37_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_92F0	Access Control Register (CCM_ACCESS_CTRL37)	32	R/W	0000_0000h	5.1.7.26/592
3038_92F4	Access Control Register (CCM_ACCESS_CTRL_ROOT37_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_92F8	Access Control Register (CCM_ACCESS_CTRL_ROOT37_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_92FC	Access Control Register (CCM_ACCESS_CTRL_ROOT37_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9300	Target Register (CCM_TARGET_ROOT38)	32	R/W	1000_0000h	5.1.7.10/556
3038_9304	Target Register (CCM_TARGET_ROOT38_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9308	Target Register (CCM_TARGET_ROOT38_CLR)	32	R/W	0000_0000h	5.1.7.12/560

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_930C	Target Register (CCM_TARGET_ROOT38_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9310	Miscellaneous Register (CCM_MISC38)	32	R/W	0000_0000h	5.1.7.14/564
3038_9314	Miscellaneous Register (CCM_MISC_ROOT38_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9318	Miscellaneous Register (CCM_MISC_ROOT38_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_931C	Miscellaneous Register (CCM_MISC_ROOT38_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9320	Post Divider Register (CCM_POST38)	32	R/W	0000_0000h	5.1.7.18/568
3038_9324	Post Divider Register (CCM_POST_ROOT38_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9328	Post Divider Register (CCM_POST_ROOT38_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_932C	Post Divider Register (CCM_POST_ROOT38_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9330	Pre Divider Register (CCM_PRE38)	32	R/W	1000_0000h	5.1.7.22/580
3038_9334	Pre Divider Register (CCM_PRE_ROOT38_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9338	Pre Divider Register (CCM_PRE_ROOT38_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_933C	Pre Divider Register (CCM_PRE_ROOT38_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9370	Access Control Register (CCM_ACCESS_CTRL38)	32	R/W	0000_0000h	5.1.7.26/592
3038_9374	Access Control Register (CCM_ACCESS_CTRL_ROOT38_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9378	Access Control Register (CCM_ACCESS_CTRL_ROOT38_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_937C	Access Control Register (CCM_ACCESS_CTRL_ROOT38_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9380	Target Register (CCM_TARGET_ROOT39)	32	R/W	1000_0000h	5.1.7.10/556
3038_9384	Target Register (CCM_TARGET_ROOT39_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9388	Target Register (CCM_TARGET_ROOT39_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_938C	Target Register (CCM_TARGET_ROOT39_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9390	Miscellaneous Register (CCM_MISC39)	32	R/W	0000_0000h	5.1.7.14/564

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9394	Miscellaneous Register (CCM_MISC_ROOT39_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9398	Miscellaneous Register (CCM_MISC_ROOT39_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_939C	Miscellaneous Register (CCM_MISC_ROOT39_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_93A0	Post Divider Register (CCM_POST39)	32	R/W	0000_0000h	5.1.7.18/568
3038_93A4	Post Divider Register (CCM_POST_ROOT39_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_93A8	Post Divider Register (CCM_POST_ROOT39_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_93AC	Post Divider Register (CCM_POST_ROOT39_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_93B0	Pre Divider Register (CCM_PRE39)	32	R/W	1000_0000h	5.1.7.22/580
3038_93B4	Pre Divider Register (CCM_PRE_ROOT39_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_93B8	Pre Divider Register (CCM_PRE_ROOT39_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_93BC	Pre Divider Register (CCM_PRE_ROOT39_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_93F0	Access Control Register (CCM_ACCESS_CTRL39)	32	R/W	0000_0000h	5.1.7.26/592
3038_93F4	Access Control Register (CCM_ACCESS_CTRL_ROOT39_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_93F8	Access Control Register (CCM_ACCESS_CTRL_ROOT39_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_93FC	Access Control Register (CCM_ACCESS_CTRL_ROOT39_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9400	Target Register (CCM_TARGET_ROOT40)	32	R/W	1000_0000h	5.1.7.10/556
3038_9404	Target Register (CCM_TARGET_ROOT40_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9408	Target Register (CCM_TARGET_ROOT40_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_940C	Target Register (CCM_TARGET_ROOT40_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9410	Miscellaneous Register (CCM_MISC40)	32	R/W	0000_0000h	5.1.7.14/564
3038_9414	Miscellaneous Register (CCM_MISC_ROOT40_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9418	Miscellaneous Register (CCM_MISC_ROOT40_CLR)	32	R/W	0000_0000h	5.1.7.16/566

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_941C	Miscellaneous Register (CCM_MISC_ROOT40_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9420	Post Divider Register (CCM_POST40)	32	R/W	0000_0000h	5.1.7.18/568
3038_9424	Post Divider Register (CCM_POST_ROOT40_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9428	Post Divider Register (CCM_POST_ROOT40_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_942C	Post Divider Register (CCM_POST_ROOT40_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9430	Pre Divider Register (CCM_PRE40)	32	R/W	1000_0000h	5.1.7.22/580
3038_9434	Pre Divider Register (CCM_PRE_ROOT40_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9438	Pre Divider Register (CCM_PRE_ROOT40_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_943C	Pre Divider Register (CCM_PRE_ROOT40_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9470	Access Control Register (CCM_ACCESS_CTRL40)	32	R/W	0000_0000h	5.1.7.26/592
3038_9474	Access Control Register (CCM_ACCESS_CTRL_ROOT40_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9478	Access Control Register (CCM_ACCESS_CTRL_ROOT40_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_947C	Access Control Register (CCM_ACCESS_CTRL_ROOT40_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9480	Target Register (CCM_TARGET_ROOT41)	32	R/W	1000_0000h	5.1.7.10/556
3038_9484	Target Register (CCM_TARGET_ROOT41_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9488	Target Register (CCM_TARGET_ROOT41_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_948C	Target Register (CCM_TARGET_ROOT41_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9490	Miscellaneous Register (CCM_MISC41)	32	R/W	0000_0000h	5.1.7.14/564
3038_9494	Miscellaneous Register (CCM_MISC_ROOT41_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9498	Miscellaneous Register (CCM_MISC_ROOT41_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_949C	Miscellaneous Register (CCM_MISC_ROOT41_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_94A0	Post Divider Register (CCM_POST41)	32	R/W	0000_0000h	5.1.7.18/568

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_94A4	Post Divider Register (CCM_POST_ROOT41_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_94A8	Post Divider Register (CCM_POST_ROOT41_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_94AC	Post Divider Register (CCM_POST_ROOT41_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_94B0	Pre Divider Register (CCM_PRE41)	32	R/W	1000_0000h	5.1.7.22/580
3038_94B4	Pre Divider Register (CCM_PRE_ROOT41_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_94B8	Pre Divider Register (CCM_PRE_ROOT41_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_94BC	Pre Divider Register (CCM_PRE_ROOT41_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_94F0	Access Control Register (CCM_ACCESS_CTRL41)	32	R/W	0000_0000h	5.1.7.26/592
3038_94F4	Access Control Register (CCM_ACCESS_CTRL_ROOT41_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_94F8	Access Control Register (CCM_ACCESS_CTRL_ROOT41_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_94FC	Access Control Register (CCM_ACCESS_CTRL_ROOT41_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9500	Target Register (CCM_TARGET_ROOT42)	32	R/W	1000_0000h	5.1.7.10/556
3038_9504	Target Register (CCM_TARGET_ROOT42_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9508	Target Register (CCM_TARGET_ROOT42_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_950C	Target Register (CCM_TARGET_ROOT42_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9510	Miscellaneous Register (CCM_MISC42)	32	R/W	0000_0000h	5.1.7.14/564
3038_9514	Miscellaneous Register (CCM_MISC_ROOT42_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9518	Miscellaneous Register (CCM_MISC_ROOT42_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_951C	Miscellaneous Register (CCM_MISC_ROOT42_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9520	Post Divider Register (CCM_POST42)	32	R/W	0000_0000h	5.1.7.18/568
3038_9524	Post Divider Register (CCM_POST_ROOT42_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9528	Post Divider Register (CCM_POST_ROOT42_CLR)	32	R/W	0000_0000h	5.1.7.20/574

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_952C	Post Divider Register (CCM_POST_ROOT42_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9530	Pre Divider Register (CCM_PRE42)	32	R/W	1000_0000h	5.1.7.22/580
3038_9534	Pre Divider Register (CCM_PRE_ROOT42_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9538	Pre Divider Register (CCM_PRE_ROOT42_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_953C	Pre Divider Register (CCM_PRE_ROOT42_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9570	Access Control Register (CCM_ACCESS_CTRL42)	32	R/W	0000_0000h	5.1.7.26/592
3038_9574	Access Control Register (CCM_ACCESS_CTRL_ROOT42_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9578	Access Control Register (CCM_ACCESS_CTRL_ROOT42_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_957C	Access Control Register (CCM_ACCESS_CTRL_ROOT42_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9580	Target Register (CCM_TARGET_ROOT43)	32	R/W	1000_0000h	5.1.7.10/556
3038_9584	Target Register (CCM_TARGET_ROOT43_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9588	Target Register (CCM_TARGET_ROOT43_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_958C	Target Register (CCM_TARGET_ROOT43_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9590	Miscellaneous Register (CCM_MISC43)	32	R/W	0000_0000h	5.1.7.14/564
3038_9594	Miscellaneous Register (CCM_MISC_ROOT43_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9598	Miscellaneous Register (CCM_MISC_ROOT43_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_959C	Miscellaneous Register (CCM_MISC_ROOT43_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_95A0	Post Divider Register (CCM_POST43)	32	R/W	0000_0000h	5.1.7.18/568
3038_95A4	Post Divider Register (CCM_POST_ROOT43_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_95A8	Post Divider Register (CCM_POST_ROOT43_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_95AC	Post Divider Register (CCM_POST_ROOT43_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_95B0	Pre Divider Register (CCM_PRE43)	32	R/W	1000_0000h	5.1.7.22/580

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_95B4	Pre Divider Register (CCM_PRE_ROOT43_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_95B8	Pre Divider Register (CCM_PRE_ROOT43_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_95BC	Pre Divider Register (CCM_PRE_ROOT43_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_95F0	Access Control Register (CCM_ACCESS_CTRL43)	32	R/W	0000_0000h	5.1.7.26/592
3038_95F4	Access Control Register (CCM_ACCESS_CTRL_ROOT43_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_95F8	Access Control Register (CCM_ACCESS_CTRL_ROOT43_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_95FC	Access Control Register (CCM_ACCESS_CTRL_ROOT43_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9600	Target Register (CCM_TARGET_ROOT44)	32	R/W	1000_0000h	5.1.7.10/556
3038_9604	Target Register (CCM_TARGET_ROOT44_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9608	Target Register (CCM_TARGET_ROOT44_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_960C	Target Register (CCM_TARGET_ROOT44_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9610	Miscellaneous Register (CCM_MISC44)	32	R/W	0000_0000h	5.1.7.14/564
3038_9614	Miscellaneous Register (CCM_MISC_ROOT44_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9618	Miscellaneous Register (CCM_MISC_ROOT44_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_961C	Miscellaneous Register (CCM_MISC_ROOT44_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9620	Post Divider Register (CCM_POST44)	32	R/W	0000_0000h	5.1.7.18/568
3038_9624	Post Divider Register (CCM_POST_ROOT44_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9628	Post Divider Register (CCM_POST_ROOT44_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_962C	Post Divider Register (CCM_POST_ROOT44_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9630	Pre Divider Register (CCM_PRE44)	32	R/W	1000_0000h	5.1.7.22/580
3038_9634	Pre Divider Register (CCM_PRE_ROOT44_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9638	Pre Divider Register (CCM_PRE_ROOT44_CLR)	32	R/W	0000_0000h	5.1.7.24/586

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_963C	Pre Divider Register (CCM_PRE_ROOT44_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9670	Access Control Register (CCM_ACCESS_CTRL44)	32	R/W	0000_0000h	5.1.7.26/592
3038_9674	Access Control Register (CCM_ACCESS_CTRL_ROOT44_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9678	Access Control Register (CCM_ACCESS_CTRL_ROOT44_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_967C	Access Control Register (CCM_ACCESS_CTRL_ROOT44_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9680	Target Register (CCM_TARGET_ROOT45)	32	R/W	1000_0000h	5.1.7.10/556
3038_9684	Target Register (CCM_TARGET_ROOT45_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9688	Target Register (CCM_TARGET_ROOT45_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_968C	Target Register (CCM_TARGET_ROOT45_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9690	Miscellaneous Register (CCM_MISC45)	32	R/W	0000_0000h	5.1.7.14/564
3038_9694	Miscellaneous Register (CCM_MISC_ROOT45_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9698	Miscellaneous Register (CCM_MISC_ROOT45_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_969C	Miscellaneous Register (CCM_MISC_ROOT45_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_96A0	Post Divider Register (CCM_POST45)	32	R/W	0000_0000h	5.1.7.18/568
3038_96A4	Post Divider Register (CCM_POST_ROOT45_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_96A8	Post Divider Register (CCM_POST_ROOT45_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_96AC	Post Divider Register (CCM_POST_ROOT45_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_96B0	Pre Divider Register (CCM_PRE45)	32	R/W	1000_0000h	5.1.7.22/580
3038_96B4	Pre Divider Register (CCM_PRE_ROOT45_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_96B8	Pre Divider Register (CCM_PRE_ROOT45_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_96BC	Pre Divider Register (CCM_PRE_ROOT45_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_96F0	Access Control Register (CCM_ACCESS_CTRL45)	32	R/W	0000_0000h	5.1.7.26/592

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_96F4	Access Control Register (CCM_ACCESS_CTRL_ROOT45_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_96F8	Access Control Register (CCM_ACCESS_CTRL_ROOT45_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_96FC	Access Control Register (CCM_ACCESS_CTRL_ROOT45_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9700	Target Register (CCM_TARGET_ROOT46)	32	R/W	1000_0000h	5.1.7.10/556
3038_9704	Target Register (CCM_TARGET_ROOT46_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9708	Target Register (CCM_TARGET_ROOT46_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_970C	Target Register (CCM_TARGET_ROOT46_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9710	Miscellaneous Register (CCM_MISC46)	32	R/W	0000_0000h	5.1.7.14/564
3038_9714	Miscellaneous Register (CCM_MISC_ROOT46_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9718	Miscellaneous Register (CCM_MISC_ROOT46_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_971C	Miscellaneous Register (CCM_MISC_ROOT46_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9720	Post Divider Register (CCM_POST46)	32	R/W	0000_0000h	5.1.7.18/568
3038_9724	Post Divider Register (CCM_POST_ROOT46_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9728	Post Divider Register (CCM_POST_ROOT46_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_972C	Post Divider Register (CCM_POST_ROOT46_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9730	Pre Divider Register (CCM_PRE46)	32	R/W	1000_0000h	5.1.7.22/580
3038_9734	Pre Divider Register (CCM_PRE_ROOT46_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9738	Pre Divider Register (CCM_PRE_ROOT46_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_973C	Pre Divider Register (CCM_PRE_ROOT46_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9770	Access Control Register (CCM_ACCESS_CTRL46)	32	R/W	0000_0000h	5.1.7.26/592
3038_9774	Access Control Register (CCM_ACCESS_CTRL_ROOT46_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9778	Access Control Register (CCM_ACCESS_CTRL_ROOT46_CLR)	32	R/W	0000_0000h	5.1.7.28/597

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_977C	Access Control Register (CCM_ACCESS_CTRL_ROOT46_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9780	Target Register (CCM_TARGET_ROOT47)	32	R/W	1000_0000h	5.1.7.10/556
3038_9784	Target Register (CCM_TARGET_ROOT47_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9788	Target Register (CCM_TARGET_ROOT47_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_978C	Target Register (CCM_TARGET_ROOT47_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9790	Miscellaneous Register (CCM_MISC47)	32	R/W	0000_0000h	5.1.7.14/564
3038_9794	Miscellaneous Register (CCM_MISC_ROOT47_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9798	Miscellaneous Register (CCM_MISC_ROOT47_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_979C	Miscellaneous Register (CCM_MISC_ROOT47_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_97A0	Post Divider Register (CCM_POST47)	32	R/W	0000_0000h	5.1.7.18/568
3038_97A4	Post Divider Register (CCM_POST_ROOT47_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_97A8	Post Divider Register (CCM_POST_ROOT47_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_97AC	Post Divider Register (CCM_POST_ROOT47_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_97B0	Pre Divider Register (CCM_PRE47)	32	R/W	1000_0000h	5.1.7.22/580
3038_97B4	Pre Divider Register (CCM_PRE_ROOT47_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_97B8	Pre Divider Register (CCM_PRE_ROOT47_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_97BC	Pre Divider Register (CCM_PRE_ROOT47_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_97F0	Access Control Register (CCM_ACCESS_CTRL47)	32	R/W	0000_0000h	5.1.7.26/592
3038_97F4	Access Control Register (CCM_ACCESS_CTRL_ROOT47_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_97F8	Access Control Register (CCM_ACCESS_CTRL_ROOT47_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_97FC	Access Control Register (CCM_ACCESS_CTRL_ROOT47_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9800	Target Register (CCM_TARGET_ROOT48)	32	R/W	1000_0000h	5.1.7.10/556

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9804	Target Register (CCM_TARGET_ROOT48_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9808	Target Register (CCM_TARGET_ROOT48_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_980C	Target Register (CCM_TARGET_ROOT48_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9810	Miscellaneous Register (CCM_MISC48)	32	R/W	0000_0000h	5.1.7.14/564
3038_9814	Miscellaneous Register (CCM_MISC_ROOT48_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9818	Miscellaneous Register (CCM_MISC_ROOT48_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_981C	Miscellaneous Register (CCM_MISC_ROOT48_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9820	Post Divider Register (CCM_POST48)	32	R/W	0000_0000h	5.1.7.18/568
3038_9824	Post Divider Register (CCM_POST_ROOT48_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9828	Post Divider Register (CCM_POST_ROOT48_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_982C	Post Divider Register (CCM_POST_ROOT48_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9830	Pre Divider Register (CCM_PRE48)	32	R/W	1000_0000h	5.1.7.22/580
3038_9834	Pre Divider Register (CCM_PRE_ROOT48_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9838	Pre Divider Register (CCM_PRE_ROOT48_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_983C	Pre Divider Register (CCM_PRE_ROOT48_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9870	Access Control Register (CCM_ACCESS_CTRL48)	32	R/W	0000_0000h	5.1.7.26/592
3038_9874	Access Control Register (CCM_ACCESS_CTRL_ROOT48_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9878	Access Control Register (CCM_ACCESS_CTRL_ROOT48_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_987C	Access Control Register (CCM_ACCESS_CTRL_ROOT48_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9880	Target Register (CCM_TARGET_ROOT49)	32	R/W	1000_0000h	5.1.7.10/556
3038_9884	Target Register (CCM_TARGET_ROOT49_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9888	Target Register (CCM_TARGET_ROOT49_CLR)	32	R/W	0000_0000h	5.1.7.12/560

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_988C	Target Register (CCM_TARGET_ROOT49_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9890	Miscellaneous Register (CCM_MISC49)	32	R/W	0000_0000h	5.1.7.14/564
3038_9894	Miscellaneous Register (CCM_MISC_ROOT49_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9898	Miscellaneous Register (CCM_MISC_ROOT49_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_989C	Miscellaneous Register (CCM_MISC_ROOT49_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_98A0	Post Divider Register (CCM_POST49)	32	R/W	0000_0000h	5.1.7.18/568
3038_98A4	Post Divider Register (CCM_POST_ROOT49_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_98A8	Post Divider Register (CCM_POST_ROOT49_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_98AC	Post Divider Register (CCM_POST_ROOT49_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_98B0	Pre Divider Register (CCM_PRE49)	32	R/W	1000_0000h	5.1.7.22/580
3038_98B4	Pre Divider Register (CCM_PRE_ROOT49_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_98B8	Pre Divider Register (CCM_PRE_ROOT49_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_98BC	Pre Divider Register (CCM_PRE_ROOT49_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_98F0	Access Control Register (CCM_ACCESS_CTRL49)	32	R/W	0000_0000h	5.1.7.26/592
3038_98F4	Access Control Register (CCM_ACCESS_CTRL_ROOT49_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_98F8	Access Control Register (CCM_ACCESS_CTRL_ROOT49_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_98FC	Access Control Register (CCM_ACCESS_CTRL_ROOT49_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9900	Target Register (CCM_TARGET_ROOT50)	32	R/W	1000_0000h	5.1.7.10/556
3038_9904	Target Register (CCM_TARGET_ROOT50_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9908	Target Register (CCM_TARGET_ROOT50_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_990C	Target Register (CCM_TARGET_ROOT50_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9910	Miscellaneous Register (CCM_MISC50)	32	R/W	0000_0000h	5.1.7.14/564

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9914	Miscellaneous Register (CCM_MISC_ROOT50_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9918	Miscellaneous Register (CCM_MISC_ROOT50_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_991C	Miscellaneous Register (CCM_MISC_ROOT50_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9920	Post Divider Register (CCM_POST50)	32	R/W	0000_0000h	5.1.7.18/568
3038_9924	Post Divider Register (CCM_POST_ROOT50_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9928	Post Divider Register (CCM_POST_ROOT50_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_992C	Post Divider Register (CCM_POST_ROOT50_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9930	Pre Divider Register (CCM_PRE50)	32	R/W	1000_0000h	5.1.7.22/580
3038_9934	Pre Divider Register (CCM_PRE_ROOT50_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9938	Pre Divider Register (CCM_PRE_ROOT50_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_993C	Pre Divider Register (CCM_PRE_ROOT50_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9970	Access Control Register (CCM_ACCESS_CTRL50)	32	R/W	0000_0000h	5.1.7.26/592
3038_9974	Access Control Register (CCM_ACCESS_CTRL_ROOT50_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9978	Access Control Register (CCM_ACCESS_CTRL_ROOT50_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_997C	Access Control Register (CCM_ACCESS_CTRL_ROOT50_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9980	Target Register (CCM_TARGET_ROOT51)	32	R/W	1000_0000h	5.1.7.10/556
3038_9984	Target Register (CCM_TARGET_ROOT51_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9988	Target Register (CCM_TARGET_ROOT51_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_998C	Target Register (CCM_TARGET_ROOT51_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9990	Miscellaneous Register (CCM_MISC51)	32	R/W	0000_0000h	5.1.7.14/564
3038_9994	Miscellaneous Register (CCM_MISC_ROOT51_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9998	Miscellaneous Register (CCM_MISC_ROOT51_CLR)	32	R/W	0000_0000h	5.1.7.16/566

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_999C	Miscellaneous Register (CCM_MISC_ROOT51_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_99A0	Post Divider Register (CCM_POST51)	32	R/W	0000_0000h	5.1.7.18/568
3038_99A4	Post Divider Register (CCM_POST_ROOT51_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_99A8	Post Divider Register (CCM_POST_ROOT51_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_99AC	Post Divider Register (CCM_POST_ROOT51_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_99B0	Pre Divider Register (CCM_PRE51)	32	R/W	1000_0000h	5.1.7.22/580
3038_99B4	Pre Divider Register (CCM_PRE_ROOT51_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_99B8	Pre Divider Register (CCM_PRE_ROOT51_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_99BC	Pre Divider Register (CCM_PRE_ROOT51_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_99F0	Access Control Register (CCM_ACCESS_CTRL51)	32	R/W	0000_0000h	5.1.7.26/592
3038_99F4	Access Control Register (CCM_ACCESS_CTRL_ROOT51_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_99F8	Access Control Register (CCM_ACCESS_CTRL_ROOT51_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_99FC	Access Control Register (CCM_ACCESS_CTRL_ROOT51_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9A00	Target Register (CCM_TARGET_ROOT52)	32	R/W	1000_0000h	5.1.7.10/556
3038_9A04	Target Register (CCM_TARGET_ROOT52_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9A08	Target Register (CCM_TARGET_ROOT52_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_9A0C	Target Register (CCM_TARGET_ROOT52_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9A10	Miscellaneous Register (CCM_MISC52)	32	R/W	0000_0000h	5.1.7.14/564
3038_9A14	Miscellaneous Register (CCM_MISC_ROOT52_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9A18	Miscellaneous Register (CCM_MISC_ROOT52_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_9A1C	Miscellaneous Register (CCM_MISC_ROOT52_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9A20	Post Divider Register (CCM_POST52)	32	R/W	0000_0000h	5.1.7.18/568

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9A24	Post Divider Register (CCM_POST_ROOT52_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9A28	Post Divider Register (CCM_POST_ROOT52_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_9A2C	Post Divider Register (CCM_POST_ROOT52_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9A30	Pre Divider Register (CCM_PRE52)	32	R/W	1000_0000h	5.1.7.22/580
3038_9A34	Pre Divider Register (CCM_PRE_ROOT52_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9A38	Pre Divider Register (CCM_PRE_ROOT52_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_9A3C	Pre Divider Register (CCM_PRE_ROOT52_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9A70	Access Control Register (CCM_ACCESS_CTRL52)	32	R/W	0000_0000h	5.1.7.26/592
3038_9A74	Access Control Register (CCM_ACCESS_CTRL_ROOT52_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9A78	Access Control Register (CCM_ACCESS_CTRL_ROOT52_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_9A7C	Access Control Register (CCM_ACCESS_CTRL_ROOT52_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9A80	Target Register (CCM_TARGET_ROOT53)	32	R/W	1000_0000h	5.1.7.10/556
3038_9A84	Target Register (CCM_TARGET_ROOT53_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9A88	Target Register (CCM_TARGET_ROOT53_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_9A8C	Target Register (CCM_TARGET_ROOT53_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9A90	Miscellaneous Register (CCM_MISC53)	32	R/W	0000_0000h	5.1.7.14/564
3038_9A94	Miscellaneous Register (CCM_MISC_ROOT53_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9A98	Miscellaneous Register (CCM_MISC_ROOT53_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_9A9C	Miscellaneous Register (CCM_MISC_ROOT53_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9AA0	Post Divider Register (CCM_POST53)	32	R/W	0000_0000h	5.1.7.18/568
3038_9AA4	Post Divider Register (CCM_POST_ROOT53_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9AA8	Post Divider Register (CCM_POST_ROOT53_CLR)	32	R/W	0000_0000h	5.1.7.20/574

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9AAC	Post Divider Register (CCM_POST_ROOT53_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9AB0	Pre Divider Register (CCM_PRE53)	32	R/W	1000_0000h	5.1.7.22/580
3038_9AB4	Pre Divider Register (CCM_PRE_ROOT53_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9AB8	Pre Divider Register (CCM_PRE_ROOT53_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_9ABC	Pre Divider Register (CCM_PRE_ROOT53_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9AF0	Access Control Register (CCM_ACCESS_CTRL53)	32	R/W	0000_0000h	5.1.7.26/592
3038_9AF4	Access Control Register (CCM_ACCESS_CTRL_ROOT53_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9AF8	Access Control Register (CCM_ACCESS_CTRL_ROOT53_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_9AFC	Access Control Register (CCM_ACCESS_CTRL_ROOT53_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9B00	Target Register (CCM_TARGET_ROOT54)	32	R/W	1000_0000h	5.1.7.10/556
3038_9B04	Target Register (CCM_TARGET_ROOT54_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9B08	Target Register (CCM_TARGET_ROOT54_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_9B0C	Target Register (CCM_TARGET_ROOT54_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9B10	Miscellaneous Register (CCM_MISC54)	32	R/W	0000_0000h	5.1.7.14/564
3038_9B14	Miscellaneous Register (CCM_MISC_ROOT54_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9B18	Miscellaneous Register (CCM_MISC_ROOT54_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_9B1C	Miscellaneous Register (CCM_MISC_ROOT54_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9B20	Post Divider Register (CCM_POST54)	32	R/W	0000_0000h	5.1.7.18/568
3038_9B24	Post Divider Register (CCM_POST_ROOT54_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9B28	Post Divider Register (CCM_POST_ROOT54_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_9B2C	Post Divider Register (CCM_POST_ROOT54_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9B30	Pre Divider Register (CCM_PRE54)	32	R/W	1000_0000h	5.1.7.22/580

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9B34	Pre Divider Register (CCM_PRE_ROOT54_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9B38	Pre Divider Register (CCM_PRE_ROOT54_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_9B3C	Pre Divider Register (CCM_PRE_ROOT54_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9B70	Access Control Register (CCM_ACCESS_CTRL54)	32	R/W	0000_0000h	5.1.7.26/592
3038_9B74	Access Control Register (CCM_ACCESS_CTRL_ROOT54_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9B78	Access Control Register (CCM_ACCESS_CTRL_ROOT54_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_9B7C	Access Control Register (CCM_ACCESS_CTRL_ROOT54_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9B80	Target Register (CCM_TARGET_ROOT55)	32	R/W	1000_0000h	5.1.7.10/556
3038_9B84	Target Register (CCM_TARGET_ROOT55_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9B88	Target Register (CCM_TARGET_ROOT55_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_9B8C	Target Register (CCM_TARGET_ROOT55_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9B90	Miscellaneous Register (CCM_MISC55)	32	R/W	0000_0000h	5.1.7.14/564
3038_9B94	Miscellaneous Register (CCM_MISC_ROOT55_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9B98	Miscellaneous Register (CCM_MISC_ROOT55_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_9B9C	Miscellaneous Register (CCM_MISC_ROOT55_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9BA0	Post Divider Register (CCM_POST55)	32	R/W	0000_0000h	5.1.7.18/568
3038_9BA4	Post Divider Register (CCM_POST_ROOT55_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9BA8	Post Divider Register (CCM_POST_ROOT55_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_9BAC	Post Divider Register (CCM_POST_ROOT55_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9BB0	Pre Divider Register (CCM_PRE55)	32	R/W	1000_0000h	5.1.7.22/580
3038_9BB4	Pre Divider Register (CCM_PRE_ROOT55_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9BB8	Pre Divider Register (CCM_PRE_ROOT55_CLR)	32	R/W	0000_0000h	5.1.7.24/586

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9BBC	Pre Divider Register (CCM_PRE_ROOT55_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9BF0	Access Control Register (CCM_ACCESS_CTRL55)	32	R/W	0000_0000h	5.1.7.26/592
3038_9BF4	Access Control Register (CCM_ACCESS_CTRL_ROOT55_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9BF8	Access Control Register (CCM_ACCESS_CTRL_ROOT55_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_9BFC	Access Control Register (CCM_ACCESS_CTRL_ROOT55_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9C00	Target Register (CCM_TARGET_ROOT56)	32	R/W	1000_0000h	5.1.7.10/556
3038_9C04	Target Register (CCM_TARGET_ROOT56_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9C08	Target Register (CCM_TARGET_ROOT56_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_9C0C	Target Register (CCM_TARGET_ROOT56_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9C10	Miscellaneous Register (CCM_MISC56)	32	R/W	0000_0000h	5.1.7.14/564
3038_9C14	Miscellaneous Register (CCM_MISC_ROOT56_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9C18	Miscellaneous Register (CCM_MISC_ROOT56_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_9C1C	Miscellaneous Register (CCM_MISC_ROOT56_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9C20	Post Divider Register (CCM_POST56)	32	R/W	0000_0000h	5.1.7.18/568
3038_9C24	Post Divider Register (CCM_POST_ROOT56_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9C28	Post Divider Register (CCM_POST_ROOT56_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_9C2C	Post Divider Register (CCM_POST_ROOT56_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9C30	Pre Divider Register (CCM_PRE56)	32	R/W	1000_0000h	5.1.7.22/580
3038_9C34	Pre Divider Register (CCM_PRE_ROOT56_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9C38	Pre Divider Register (CCM_PRE_ROOT56_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_9C3C	Pre Divider Register (CCM_PRE_ROOT56_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9C70	Access Control Register (CCM_ACCESS_CTRL56)	32	R/W	0000_0000h	5.1.7.26/592

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9C74	Access Control Register (CCM_ACCESS_CTRL_ROOT56_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9C78	Access Control Register (CCM_ACCESS_CTRL_ROOT56_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_9C7C	Access Control Register (CCM_ACCESS_CTRL_ROOT56_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9C80	Target Register (CCM_TARGET_ROOT57)	32	R/W	1000_0000h	5.1.7.10/556
3038_9C84	Target Register (CCM_TARGET_ROOT57_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9C88	Target Register (CCM_TARGET_ROOT57_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_9C8C	Target Register (CCM_TARGET_ROOT57_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9C90	Miscellaneous Register (CCM_MISC57)	32	R/W	0000_0000h	5.1.7.14/564
3038_9C94	Miscellaneous Register (CCM_MISC_ROOT57_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9C98	Miscellaneous Register (CCM_MISC_ROOT57_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_9C9C	Miscellaneous Register (CCM_MISC_ROOT57_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9CA0	Post Divider Register (CCM_POST57)	32	R/W	0000_0000h	5.1.7.18/568
3038_9CA4	Post Divider Register (CCM_POST_ROOT57_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9CA8	Post Divider Register (CCM_POST_ROOT57_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_9CAC	Post Divider Register (CCM_POST_ROOT57_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9CB0	Pre Divider Register (CCM_PRE57)	32	R/W	1000_0000h	5.1.7.22/580
3038_9CB4	Pre Divider Register (CCM_PRE_ROOT57_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9CB8	Pre Divider Register (CCM_PRE_ROOT57_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_9CBC	Pre Divider Register (CCM_PRE_ROOT57_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9CF0	Access Control Register (CCM_ACCESS_CTRL57)	32	R/W	0000_0000h	5.1.7.26/592
3038_9CF4	Access Control Register (CCM_ACCESS_CTRL_ROOT57_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9CF8	Access Control Register (CCM_ACCESS_CTRL_ROOT57_CLR)	32	R/W	0000_0000h	5.1.7.28/597

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9CFC	Access Control Register (CCM_ACCESS_CTRL_ROOT57_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9D00	Target Register (CCM_TARGET_ROOT58)	32	R/W	1000_0000h	5.1.7.10/556
3038_9D04	Target Register (CCM_TARGET_ROOT58_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9D08	Target Register (CCM_TARGET_ROOT58_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_9D0C	Target Register (CCM_TARGET_ROOT58_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9D10	Miscellaneous Register (CCM_MISC58)	32	R/W	0000_0000h	5.1.7.14/564
3038_9D14	Miscellaneous Register (CCM_MISC_ROOT58_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9D18	Miscellaneous Register (CCM_MISC_ROOT58_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_9D1C	Miscellaneous Register (CCM_MISC_ROOT58_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9D20	Post Divider Register (CCM_POST58)	32	R/W	0000_0000h	5.1.7.18/568
3038_9D24	Post Divider Register (CCM_POST_ROOT58_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9D28	Post Divider Register (CCM_POST_ROOT58_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_9D2C	Post Divider Register (CCM_POST_ROOT58_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9D30	Pre Divider Register (CCM_PRE58)	32	R/W	1000_0000h	5.1.7.22/580
3038_9D34	Pre Divider Register (CCM_PRE_ROOT58_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9D38	Pre Divider Register (CCM_PRE_ROOT58_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_9D3C	Pre Divider Register (CCM_PRE_ROOT58_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9D70	Access Control Register (CCM_ACCESS_CTRL58)	32	R/W	0000_0000h	5.1.7.26/592
3038_9D74	Access Control Register (CCM_ACCESS_CTRL_ROOT58_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9D78	Access Control Register (CCM_ACCESS_CTRL_ROOT58_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_9D7C	Access Control Register (CCM_ACCESS_CTRL_ROOT58_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9D80	Target Register (CCM_TARGET_ROOT59)	32	R/W	1000_0000h	5.1.7.10/556

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9D84	Target Register (CCM_TARGET_ROOT59_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9D88	Target Register (CCM_TARGET_ROOT59_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_9D8C	Target Register (CCM_TARGET_ROOT59_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9D90	Miscellaneous Register (CCM_MISC59)	32	R/W	0000_0000h	5.1.7.14/564
3038_9D94	Miscellaneous Register (CCM_MISC_ROOT59_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9D98	Miscellaneous Register (CCM_MISC_ROOT59_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_9D9C	Miscellaneous Register (CCM_MISC_ROOT59_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9DA0	Post Divider Register (CCM_POST59)	32	R/W	0000_0000h	5.1.7.18/568
3038_9DA4	Post Divider Register (CCM_POST_ROOT59_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9DA8	Post Divider Register (CCM_POST_ROOT59_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_9DAC	Post Divider Register (CCM_POST_ROOT59_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9DB0	Pre Divider Register (CCM_PRE59)	32	R/W	1000_0000h	5.1.7.22/580
3038_9DB4	Pre Divider Register (CCM_PRE_ROOT59_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9DB8	Pre Divider Register (CCM_PRE_ROOT59_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_9DBC	Pre Divider Register (CCM_PRE_ROOT59_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9DF0	Access Control Register (CCM_ACCESS_CTRL59)	32	R/W	0000_0000h	5.1.7.26/592
3038_9DF4	Access Control Register (CCM_ACCESS_CTRL_ROOT59_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9DF8	Access Control Register (CCM_ACCESS_CTRL_ROOT59_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_9DFC	Access Control Register (CCM_ACCESS_CTRL_ROOT59_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9E00	Target Register (CCM_TARGET_ROOT60)	32	R/W	1000_0000h	5.1.7.10/556
3038_9E04	Target Register (CCM_TARGET_ROOT60_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9E08	Target Register (CCM_TARGET_ROOT60_CLR)	32	R/W	0000_0000h	5.1.7.12/560

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9E0C	Target Register (CCM_TARGET_ROOT60_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9E10	Miscellaneous Register (CCM_MISC60)	32	R/W	0000_0000h	5.1.7.14/564
3038_9E14	Miscellaneous Register (CCM_MISC_ROOT60_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9E18	Miscellaneous Register (CCM_MISC_ROOT60_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_9E1C	Miscellaneous Register (CCM_MISC_ROOT60_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9E20	Post Divider Register (CCM_POST60)	32	R/W	0000_0000h	5.1.7.18/568
3038_9E24	Post Divider Register (CCM_POST_ROOT60_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9E28	Post Divider Register (CCM_POST_ROOT60_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_9E2C	Post Divider Register (CCM_POST_ROOT60_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9E30	Pre Divider Register (CCM_PRE60)	32	R/W	1000_0000h	5.1.7.22/580
3038_9E34	Pre Divider Register (CCM_PRE_ROOT60_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9E38	Pre Divider Register (CCM_PRE_ROOT60_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_9E3C	Pre Divider Register (CCM_PRE_ROOT60_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9E70	Access Control Register (CCM_ACCESS_CTRL60)	32	R/W	0000_0000h	5.1.7.26/592
3038_9E74	Access Control Register (CCM_ACCESS_CTRL_ROOT60_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9E78	Access Control Register (CCM_ACCESS_CTRL_ROOT60_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_9E7C	Access Control Register (CCM_ACCESS_CTRL_ROOT60_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9E80	Target Register (CCM_TARGET_ROOT61)	32	R/W	1000_0000h	5.1.7.10/556
3038_9E84	Target Register (CCM_TARGET_ROOT61_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9E88	Target Register (CCM_TARGET_ROOT61_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_9E8C	Target Register (CCM_TARGET_ROOT61_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9E90	Miscellaneous Register (CCM_MISC61)	32	R/W	0000_0000h	5.1.7.14/564

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9E94	Miscellaneous Register (CCM_MISC_ROOT61_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9E98	Miscellaneous Register (CCM_MISC_ROOT61_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_9E9C	Miscellaneous Register (CCM_MISC_ROOT61_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9EA0	Post Divider Register (CCM_POST61)	32	R/W	0000_0000h	5.1.7.18/568
3038_9EA4	Post Divider Register (CCM_POST_ROOT61_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9EA8	Post Divider Register (CCM_POST_ROOT61_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_9EAC	Post Divider Register (CCM_POST_ROOT61_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9EB0	Pre Divider Register (CCM_PRE61)	32	R/W	1000_0000h	5.1.7.22/580
3038_9EB4	Pre Divider Register (CCM_PRE_ROOT61_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9EB8	Pre Divider Register (CCM_PRE_ROOT61_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_9EBC	Pre Divider Register (CCM_PRE_ROOT61_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9EF0	Access Control Register (CCM_ACCESS_CTRL61)	32	R/W	0000_0000h	5.1.7.26/592
3038_9EF4	Access Control Register (CCM_ACCESS_CTRL_ROOT61_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9EF8	Access Control Register (CCM_ACCESS_CTRL_ROOT61_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_9EFC	Access Control Register (CCM_ACCESS_CTRL_ROOT61_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9F00	Target Register (CCM_TARGET_ROOT62)	32	R/W	1000_0000h	5.1.7.10/556
3038_9F04	Target Register (CCM_TARGET_ROOT62_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9F08	Target Register (CCM_TARGET_ROOT62_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_9F0C	Target Register (CCM_TARGET_ROOT62_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9F10	Miscellaneous Register (CCM_MISC62)	32	R/W	0000_0000h	5.1.7.14/564
3038_9F14	Miscellaneous Register (CCM_MISC_ROOT62_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9F18	Miscellaneous Register (CCM_MISC_ROOT62_CLR)	32	R/W	0000_0000h	5.1.7.16/566

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_9F1C	Miscellaneous Register (CCM_MISC_ROOT62_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9F20	Post Divider Register (CCM_POST62)	32	R/W	0000_0000h	5.1.7.18/568
3038_9F24	Post Divider Register (CCM_POST_ROOT62_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_9F28	Post Divider Register (CCM_POST_ROOT62_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_9F2C	Post Divider Register (CCM_POST_ROOT62_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_9F30	Pre Divider Register (CCM_PRE62)	32	R/W	1000_0000h	5.1.7.22/580
3038_9F34	Pre Divider Register (CCM_PRE_ROOT62_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_9F38	Pre Divider Register (CCM_PRE_ROOT62_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_9F3C	Pre Divider Register (CCM_PRE_ROOT62_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_9F70	Access Control Register (CCM_ACCESS_CTRL62)	32	R/W	0000_0000h	5.1.7.26/592
3038_9F74	Access Control Register (CCM_ACCESS_CTRL_ROOT62_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_9F78	Access Control Register (CCM_ACCESS_CTRL_ROOT62_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_9F7C	Access Control Register (CCM_ACCESS_CTRL_ROOT62_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_9F80	Target Register (CCM_TARGET_ROOT63)	32	R/W	1000_0000h	5.1.7.10/556
3038_9F84	Target Register (CCM_TARGET_ROOT63_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_9F88	Target Register (CCM_TARGET_ROOT63_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_9F8C	Target Register (CCM_TARGET_ROOT63_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_9F90	Miscellaneous Register (CCM_MISC63)	32	R/W	0000_0000h	5.1.7.14/564
3038_9F94	Miscellaneous Register (CCM_MISC_ROOT63_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_9F98	Miscellaneous Register (CCM_MISC_ROOT63_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_9F9C	Miscellaneous Register (CCM_MISC_ROOT63_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_9FA0	Post Divider Register (CCM_POST63)	32	R/W	0000_0000h	5.1.7.18/568

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3038_9FA4	Post Divider Register (CCM_POST_ROOT63_SET)	32	R/W	0000_0000h	5.1.7.19/ 571
3038_9FA8	Post Divider Register (CCM_POST_ROOT63_CLR)	32	R/W	0000_0000h	5.1.7.20/ 574
3038_9FAC	Post Divider Register (CCM_POST_ROOT63_TOG)	32	R/W	0000_0000h	5.1.7.21/ 577
3038_9FB0	Pre Divider Register (CCM_PRE63)	32	R/W	1000_0000h	5.1.7.22/ 580
3038_9FB4	Pre Divider Register (CCM_PRE_ROOT63_SET)	32	R/W	0000_0000h	5.1.7.23/ 583
3038_9FB8	Pre Divider Register (CCM_PRE_ROOT63_CLR)	32	R/W	0000_0000h	5.1.7.24/ 586
3038_9FBC	Pre Divider Register (CCM_PRE_ROOT63_TOG)	32	R/W	0000_0000h	5.1.7.25/ 589
3038_9FF0	Access Control Register (CCM_ACCESS_CTRL63)	32	R/W	0000_0000h	5.1.7.26/ 592
3038_9FF4	Access Control Register (CCM_ACCESS_CTRL_ROOT63_SET)	32	R/W	0000_0000h	5.1.7.27/ 594
3038_9FF8	Access Control Register (CCM_ACCESS_CTRL_ROOT63_CLR)	32	R/W	0000_0000h	5.1.7.28/ 597
3038_9FFC	Access Control Register (CCM_ACCESS_CTRL_ROOT63_TOG)	32	R/W	0000_0000h	5.1.7.29/ 599
3038_A000	Target Register (CCM_TARGET_ROOT64)	32	R/W	1000_0000h	5.1.7.10/ 556
3038_A004	Target Register (CCM_TARGET_ROOT64_SET)	32	R/W	0000_0000h	5.1.7.11/ 558
3038_A008	Target Register (CCM_TARGET_ROOT64_CLR)	32	R/W	0000_0000h	5.1.7.12/ 560
3038_A00C	Target Register (CCM_TARGET_ROOT64_TOG)	32	R/W	0000_0000h	5.1.7.13/ 562
3038_A010	Miscellaneous Register (CCM_MISC64)	32	R/W	0000_0000h	5.1.7.14/ 564
3038_A014	Miscellaneous Register (CCM_MISC_ROOT64_SET)	32	R/W	0000_0000h	5.1.7.15/ 565
3038_A018	Miscellaneous Register (CCM_MISC_ROOT64_CLR)	32	R/W	0000_0000h	5.1.7.16/ 566
3038_A01C	Miscellaneous Register (CCM_MISC_ROOT64_TOG)	32	R/W	0000_0000h	5.1.7.17/ 567
3038_A020	Post Divider Register (CCM_POST64)	32	R/W	0000_0000h	5.1.7.18/ 568
3038_A024	Post Divider Register (CCM_POST_ROOT64_SET)	32	R/W	0000_0000h	5.1.7.19/ 571
3038_A028	Post Divider Register (CCM_POST_ROOT64_CLR)	32	R/W	0000_0000h	5.1.7.20/ 574

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A02C	Post Divider Register (CCM_POST_ROOT64_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A030	Pre Divider Register (CCM_PRE64)	32	R/W	1000_0000h	5.1.7.22/580
3038_A034	Pre Divider Register (CCM_PRE_ROOT64_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A038	Pre Divider Register (CCM_PRE_ROOT64_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A03C	Pre Divider Register (CCM_PRE_ROOT64_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A070	Access Control Register (CCM_ACCESS_CTRL64)	32	R/W	0000_0000h	5.1.7.26/592
3038_A074	Access Control Register (CCM_ACCESS_CTRL_ROOT64_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A078	Access Control Register (CCM_ACCESS_CTRL_ROOT64_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A07C	Access Control Register (CCM_ACCESS_CTRL_ROOT64_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A080	Target Register (CCM_TARGET_ROOT65)	32	R/W	1000_0000h	5.1.7.10/556
3038_A084	Target Register (CCM_TARGET_ROOT65_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A088	Target Register (CCM_TARGET_ROOT65_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A08C	Target Register (CCM_TARGET_ROOT65_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A090	Miscellaneous Register (CCM_MISC65)	32	R/W	0000_0000h	5.1.7.14/564
3038_A094	Miscellaneous Register (CCM_MISC_ROOT65_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A098	Miscellaneous Register (CCM_MISC_ROOT65_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A09C	Miscellaneous Register (CCM_MISC_ROOT65_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A0A0	Post Divider Register (CCM_POST65)	32	R/W	0000_0000h	5.1.7.18/568
3038_A0A4	Post Divider Register (CCM_POST_ROOT65_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A0A8	Post Divider Register (CCM_POST_ROOT65_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A0AC	Post Divider Register (CCM_POST_ROOT65_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A0B0	Pre Divider Register (CCM_PRE65)	32	R/W	1000_0000h	5.1.7.22/580

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A0B4	Pre Divider Register (CCM_PRE_ROOT65_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A0B8	Pre Divider Register (CCM_PRE_ROOT65_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A0BC	Pre Divider Register (CCM_PRE_ROOT65_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A0F0	Access Control Register (CCM_ACCESS_CTRL65)	32	R/W	0000_0000h	5.1.7.26/592
3038_A0F4	Access Control Register (CCM_ACCESS_CTRL_ROOT65_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A0F8	Access Control Register (CCM_ACCESS_CTRL_ROOT65_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A0FC	Access Control Register (CCM_ACCESS_CTRL_ROOT65_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A100	Target Register (CCM_TARGET_ROOT66)	32	R/W	1000_0000h	5.1.7.10/556
3038_A104	Target Register (CCM_TARGET_ROOT66_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A108	Target Register (CCM_TARGET_ROOT66_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A10C	Target Register (CCM_TARGET_ROOT66_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A110	Miscellaneous Register (CCM_MISC66)	32	R/W	0000_0000h	5.1.7.14/564
3038_A114	Miscellaneous Register (CCM_MISC_ROOT66_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A118	Miscellaneous Register (CCM_MISC_ROOT66_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A11C	Miscellaneous Register (CCM_MISC_ROOT66_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A120	Post Divider Register (CCM_POST66)	32	R/W	0000_0000h	5.1.7.18/568
3038_A124	Post Divider Register (CCM_POST_ROOT66_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A128	Post Divider Register (CCM_POST_ROOT66_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A12C	Post Divider Register (CCM_POST_ROOT66_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A130	Pre Divider Register (CCM_PRE66)	32	R/W	1000_0000h	5.1.7.22/580
3038_A134	Pre Divider Register (CCM_PRE_ROOT66_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A138	Pre Divider Register (CCM_PRE_ROOT66_CLR)	32	R/W	0000_0000h	5.1.7.24/586

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A13C	Pre Divider Register (CCM_PRE_ROOT66_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A170	Access Control Register (CCM_ACCESS_CTRL66)	32	R/W	0000_0000h	5.1.7.26/592
3038_A174	Access Control Register (CCM_ACCESS_CTRL_ROOT66_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A178	Access Control Register (CCM_ACCESS_CTRL_ROOT66_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A17C	Access Control Register (CCM_ACCESS_CTRL_ROOT66_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A180	Target Register (CCM_TARGET_ROOT67)	32	R/W	1000_0000h	5.1.7.10/556
3038_A184	Target Register (CCM_TARGET_ROOT67_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A188	Target Register (CCM_TARGET_ROOT67_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A18C	Target Register (CCM_TARGET_ROOT67_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A190	Miscellaneous Register (CCM_MISC67)	32	R/W	0000_0000h	5.1.7.14/564
3038_A194	Miscellaneous Register (CCM_MISC_ROOT67_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A198	Miscellaneous Register (CCM_MISC_ROOT67_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A19C	Miscellaneous Register (CCM_MISC_ROOT67_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A1A0	Post Divider Register (CCM_POST67)	32	R/W	0000_0000h	5.1.7.18/568
3038_A1A4	Post Divider Register (CCM_POST_ROOT67_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A1A8	Post Divider Register (CCM_POST_ROOT67_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A1AC	Post Divider Register (CCM_POST_ROOT67_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A1B0	Pre Divider Register (CCM_PRE67)	32	R/W	1000_0000h	5.1.7.22/580
3038_A1B4	Pre Divider Register (CCM_PRE_ROOT67_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A1B8	Pre Divider Register (CCM_PRE_ROOT67_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A1BC	Pre Divider Register (CCM_PRE_ROOT67_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A1F0	Access Control Register (CCM_ACCESS_CTRL67)	32	R/W	0000_0000h	5.1.7.26/592

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A1F4	Access Control Register (CCM_ACCESS_CTRL_ROOT67_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A1F8	Access Control Register (CCM_ACCESS_CTRL_ROOT67_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A1FC	Access Control Register (CCM_ACCESS_CTRL_ROOT67_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A200	Target Register (CCM_TARGET_ROOT68)	32	R/W	1000_0000h	5.1.7.10/556
3038_A204	Target Register (CCM_TARGET_ROOT68_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A208	Target Register (CCM_TARGET_ROOT68_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A20C	Target Register (CCM_TARGET_ROOT68_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A210	Miscellaneous Register (CCM_MISC68)	32	R/W	0000_0000h	5.1.7.14/564
3038_A214	Miscellaneous Register (CCM_MISC_ROOT68_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A218	Miscellaneous Register (CCM_MISC_ROOT68_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A21C	Miscellaneous Register (CCM_MISC_ROOT68_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A220	Post Divider Register (CCM_POST68)	32	R/W	0000_0000h	5.1.7.18/568
3038_A224	Post Divider Register (CCM_POST_ROOT68_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A228	Post Divider Register (CCM_POST_ROOT68_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A22C	Post Divider Register (CCM_POST_ROOT68_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A230	Pre Divider Register (CCM_PRE68)	32	R/W	1000_0000h	5.1.7.22/580
3038_A234	Pre Divider Register (CCM_PRE_ROOT68_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A238	Pre Divider Register (CCM_PRE_ROOT68_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A23C	Pre Divider Register (CCM_PRE_ROOT68_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A270	Access Control Register (CCM_ACCESS_CTRL68)	32	R/W	0000_0000h	5.1.7.26/592
3038_A274	Access Control Register (CCM_ACCESS_CTRL_ROOT68_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A278	Access Control Register (CCM_ACCESS_CTRL_ROOT68_CLR)	32	R/W	0000_0000h	5.1.7.28/597

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A27C	Access Control Register (CCM_ACCESS_CTRL_ROOT68_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A280	Target Register (CCM_TARGET_ROOT69)	32	R/W	1000_0000h	5.1.7.10/556
3038_A284	Target Register (CCM_TARGET_ROOT69_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A288	Target Register (CCM_TARGET_ROOT69_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A28C	Target Register (CCM_TARGET_ROOT69_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A290	Miscellaneous Register (CCM_MISC69)	32	R/W	0000_0000h	5.1.7.14/564
3038_A294	Miscellaneous Register (CCM_MISC_ROOT69_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A298	Miscellaneous Register (CCM_MISC_ROOT69_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A29C	Miscellaneous Register (CCM_MISC_ROOT69_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A2A0	Post Divider Register (CCM_POST69)	32	R/W	0000_0000h	5.1.7.18/568
3038_A2A4	Post Divider Register (CCM_POST_ROOT69_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A2A8	Post Divider Register (CCM_POST_ROOT69_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A2AC	Post Divider Register (CCM_POST_ROOT69_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A2B0	Pre Divider Register (CCM_PRE69)	32	R/W	1000_0000h	5.1.7.22/580
3038_A2B4	Pre Divider Register (CCM_PRE_ROOT69_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A2B8	Pre Divider Register (CCM_PRE_ROOT69_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A2BC	Pre Divider Register (CCM_PRE_ROOT69_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A2F0	Access Control Register (CCM_ACCESS_CTRL69)	32	R/W	0000_0000h	5.1.7.26/592
3038_A2F4	Access Control Register (CCM_ACCESS_CTRL_ROOT69_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A2F8	Access Control Register (CCM_ACCESS_CTRL_ROOT69_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A2FC	Access Control Register (CCM_ACCESS_CTRL_ROOT69_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A300	Target Register (CCM_TARGET_ROOT70)	32	R/W	1000_0000h	5.1.7.10/556

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A304	Target Register (CCM_TARGET_ROOT70_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A308	Target Register (CCM_TARGET_ROOT70_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A30C	Target Register (CCM_TARGET_ROOT70_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A310	Miscellaneous Register (CCM_MISC70)	32	R/W	0000_0000h	5.1.7.14/564
3038_A314	Miscellaneous Register (CCM_MISC_ROOT70_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A318	Miscellaneous Register (CCM_MISC_ROOT70_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A31C	Miscellaneous Register (CCM_MISC_ROOT70_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A320	Post Divider Register (CCM_POST70)	32	R/W	0000_0000h	5.1.7.18/568
3038_A324	Post Divider Register (CCM_POST_ROOT70_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A328	Post Divider Register (CCM_POST_ROOT70_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A32C	Post Divider Register (CCM_POST_ROOT70_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A330	Pre Divider Register (CCM_PRE70)	32	R/W	1000_0000h	5.1.7.22/580
3038_A334	Pre Divider Register (CCM_PRE_ROOT70_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A338	Pre Divider Register (CCM_PRE_ROOT70_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A33C	Pre Divider Register (CCM_PRE_ROOT70_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A370	Access Control Register (CCM_ACCESS_CTRL70)	32	R/W	0000_0000h	5.1.7.26/592
3038_A374	Access Control Register (CCM_ACCESS_CTRL_ROOT70_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A378	Access Control Register (CCM_ACCESS_CTRL_ROOT70_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A37C	Access Control Register (CCM_ACCESS_CTRL_ROOT70_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A380	Target Register (CCM_TARGET_ROOT71)	32	R/W	1000_0000h	5.1.7.10/556
3038_A384	Target Register (CCM_TARGET_ROOT71_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A388	Target Register (CCM_TARGET_ROOT71_CLR)	32	R/W	0000_0000h	5.1.7.12/560

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A38C	Target Register (CCM_TARGET_ROOT71_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A390	Miscellaneous Register (CCM_MISC71)	32	R/W	0000_0000h	5.1.7.14/564
3038_A394	Miscellaneous Register (CCM_MISC_ROOT71_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A398	Miscellaneous Register (CCM_MISC_ROOT71_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A39C	Miscellaneous Register (CCM_MISC_ROOT71_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A3A0	Post Divider Register (CCM_POST71)	32	R/W	0000_0000h	5.1.7.18/568
3038_A3A4	Post Divider Register (CCM_POST_ROOT71_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A3A8	Post Divider Register (CCM_POST_ROOT71_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A3AC	Post Divider Register (CCM_POST_ROOT71_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A3B0	Pre Divider Register (CCM_PRE71)	32	R/W	1000_0000h	5.1.7.22/580
3038_A3B4	Pre Divider Register (CCM_PRE_ROOT71_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A3B8	Pre Divider Register (CCM_PRE_ROOT71_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A3BC	Pre Divider Register (CCM_PRE_ROOT71_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A3F0	Access Control Register (CCM_ACCESS_CTRL71)	32	R/W	0000_0000h	5.1.7.26/592
3038_A3F4	Access Control Register (CCM_ACCESS_CTRL_ROOT71_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A3F8	Access Control Register (CCM_ACCESS_CTRL_ROOT71_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A3FC	Access Control Register (CCM_ACCESS_CTRL_ROOT71_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A400	Target Register (CCM_TARGET_ROOT72)	32	R/W	1000_0000h	5.1.7.10/556
3038_A404	Target Register (CCM_TARGET_ROOT72_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A408	Target Register (CCM_TARGET_ROOT72_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A40C	Target Register (CCM_TARGET_ROOT72_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A410	Miscellaneous Register (CCM_MISC72)	32	R/W	0000_0000h	5.1.7.14/564

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A414	Miscellaneous Register (CCM_MISC_ROOT72_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A418	Miscellaneous Register (CCM_MISC_ROOT72_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A41C	Miscellaneous Register (CCM_MISC_ROOT72_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A420	Post Divider Register (CCM_POST72)	32	R/W	0000_0000h	5.1.7.18/568
3038_A424	Post Divider Register (CCM_POST_ROOT72_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A428	Post Divider Register (CCM_POST_ROOT72_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A42C	Post Divider Register (CCM_POST_ROOT72_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A430	Pre Divider Register (CCM_PRE72)	32	R/W	1000_0000h	5.1.7.22/580
3038_A434	Pre Divider Register (CCM_PRE_ROOT72_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A438	Pre Divider Register (CCM_PRE_ROOT72_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A43C	Pre Divider Register (CCM_PRE_ROOT72_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A470	Access Control Register (CCM_ACCESS_CTRL72)	32	R/W	0000_0000h	5.1.7.26/592
3038_A474	Access Control Register (CCM_ACCESS_CTRL_ROOT72_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A478	Access Control Register (CCM_ACCESS_CTRL_ROOT72_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A47C	Access Control Register (CCM_ACCESS_CTRL_ROOT72_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A480	Target Register (CCM_TARGET_ROOT73)	32	R/W	1000_0000h	5.1.7.10/556
3038_A484	Target Register (CCM_TARGET_ROOT73_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A488	Target Register (CCM_TARGET_ROOT73_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A48C	Target Register (CCM_TARGET_ROOT73_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A490	Miscellaneous Register (CCM_MISC73)	32	R/W	0000_0000h	5.1.7.14/564
3038_A494	Miscellaneous Register (CCM_MISC_ROOT73_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A498	Miscellaneous Register (CCM_MISC_ROOT73_CLR)	32	R/W	0000_0000h	5.1.7.16/566

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A49C	Miscellaneous Register (CCM_MISC_ROOT73_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A4A0	Post Divider Register (CCM_POST73)	32	R/W	0000_0000h	5.1.7.18/568
3038_A4A4	Post Divider Register (CCM_POST_ROOT73_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A4A8	Post Divider Register (CCM_POST_ROOT73_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A4AC	Post Divider Register (CCM_POST_ROOT73_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A4B0	Pre Divider Register (CCM_PRE73)	32	R/W	1000_0000h	5.1.7.22/580
3038_A4B4	Pre Divider Register (CCM_PRE_ROOT73_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A4B8	Pre Divider Register (CCM_PRE_ROOT73_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A4BC	Pre Divider Register (CCM_PRE_ROOT73_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A4F0	Access Control Register (CCM_ACCESS_CTRL73)	32	R/W	0000_0000h	5.1.7.26/592
3038_A4F4	Access Control Register (CCM_ACCESS_CTRL_ROOT73_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A4F8	Access Control Register (CCM_ACCESS_CTRL_ROOT73_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A4FC	Access Control Register (CCM_ACCESS_CTRL_ROOT73_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A500	Target Register (CCM_TARGET_ROOT74)	32	R/W	1000_0000h	5.1.7.10/556
3038_A504	Target Register (CCM_TARGET_ROOT74_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A508	Target Register (CCM_TARGET_ROOT74_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A50C	Target Register (CCM_TARGET_ROOT74_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A510	Miscellaneous Register (CCM_MISC74)	32	R/W	0000_0000h	5.1.7.14/564
3038_A514	Miscellaneous Register (CCM_MISC_ROOT74_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A518	Miscellaneous Register (CCM_MISC_ROOT74_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A51C	Miscellaneous Register (CCM_MISC_ROOT74_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A520	Post Divider Register (CCM_POST74)	32	R/W	0000_0000h	5.1.7.18/568

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A524	Post Divider Register (CCM_POST_ROOT74_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A528	Post Divider Register (CCM_POST_ROOT74_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A52C	Post Divider Register (CCM_POST_ROOT74_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A530	Pre Divider Register (CCM_PRE74)	32	R/W	1000_0000h	5.1.7.22/580
3038_A534	Pre Divider Register (CCM_PRE_ROOT74_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A538	Pre Divider Register (CCM_PRE_ROOT74_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A53C	Pre Divider Register (CCM_PRE_ROOT74_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A570	Access Control Register (CCM_ACCESS_CTRL74)	32	R/W	0000_0000h	5.1.7.26/592
3038_A574	Access Control Register (CCM_ACCESS_CTRL_ROOT74_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A578	Access Control Register (CCM_ACCESS_CTRL_ROOT74_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A57C	Access Control Register (CCM_ACCESS_CTRL_ROOT74_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A580	Target Register (CCM_TARGET_ROOT75)	32	R/W	1000_0000h	5.1.7.10/556
3038_A584	Target Register (CCM_TARGET_ROOT75_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A588	Target Register (CCM_TARGET_ROOT75_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A58C	Target Register (CCM_TARGET_ROOT75_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A590	Miscellaneous Register (CCM_MISC75)	32	R/W	0000_0000h	5.1.7.14/564
3038_A594	Miscellaneous Register (CCM_MISC_ROOT75_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A598	Miscellaneous Register (CCM_MISC_ROOT75_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A59C	Miscellaneous Register (CCM_MISC_ROOT75_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A5A0	Post Divider Register (CCM_POST75)	32	R/W	0000_0000h	5.1.7.18/568
3038_A5A4	Post Divider Register (CCM_POST_ROOT75_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A5A8	Post Divider Register (CCM_POST_ROOT75_CLR)	32	R/W	0000_0000h	5.1.7.20/574

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A5AC	Post Divider Register (CCM_POST_ROOT75_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A5B0	Pre Divider Register (CCM_PRE75)	32	R/W	1000_0000h	5.1.7.22/580
3038_A5B4	Pre Divider Register (CCM_PRE_ROOT75_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A5B8	Pre Divider Register (CCM_PRE_ROOT75_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A5BC	Pre Divider Register (CCM_PRE_ROOT75_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A5F0	Access Control Register (CCM_ACCESS_CTRL75)	32	R/W	0000_0000h	5.1.7.26/592
3038_A5F4	Access Control Register (CCM_ACCESS_CTRL_ROOT75_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A5F8	Access Control Register (CCM_ACCESS_CTRL_ROOT75_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A5FC	Access Control Register (CCM_ACCESS_CTRL_ROOT75_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A600	Target Register (CCM_TARGET_ROOT76)	32	R/W	1000_0000h	5.1.7.10/556
3038_A604	Target Register (CCM_TARGET_ROOT76_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A608	Target Register (CCM_TARGET_ROOT76_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A60C	Target Register (CCM_TARGET_ROOT76_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A610	Miscellaneous Register (CCM_MISC76)	32	R/W	0000_0000h	5.1.7.14/564
3038_A614	Miscellaneous Register (CCM_MISC_ROOT76_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A618	Miscellaneous Register (CCM_MISC_ROOT76_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A61C	Miscellaneous Register (CCM_MISC_ROOT76_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A620	Post Divider Register (CCM_POST76)	32	R/W	0000_0000h	5.1.7.18/568
3038_A624	Post Divider Register (CCM_POST_ROOT76_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A628	Post Divider Register (CCM_POST_ROOT76_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A62C	Post Divider Register (CCM_POST_ROOT76_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A630	Pre Divider Register (CCM_PRE76)	32	R/W	1000_0000h	5.1.7.22/580

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A634	Pre Divider Register (CCM_PRE_ROOT76_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A638	Pre Divider Register (CCM_PRE_ROOT76_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A63C	Pre Divider Register (CCM_PRE_ROOT76_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A670	Access Control Register (CCM_ACCESS_CTRL76)	32	R/W	0000_0000h	5.1.7.26/592
3038_A674	Access Control Register (CCM_ACCESS_CTRL_ROOT76_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A678	Access Control Register (CCM_ACCESS_CTRL_ROOT76_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A67C	Access Control Register (CCM_ACCESS_CTRL_ROOT76_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A680	Target Register (CCM_TARGET_ROOT77)	32	R/W	1000_0000h	5.1.7.10/556
3038_A684	Target Register (CCM_TARGET_ROOT77_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A688	Target Register (CCM_TARGET_ROOT77_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A68C	Target Register (CCM_TARGET_ROOT77_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A690	Miscellaneous Register (CCM_MISC77)	32	R/W	0000_0000h	5.1.7.14/564
3038_A694	Miscellaneous Register (CCM_MISC_ROOT77_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A698	Miscellaneous Register (CCM_MISC_ROOT77_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A69C	Miscellaneous Register (CCM_MISC_ROOT77_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A6A0	Post Divider Register (CCM_POST77)	32	R/W	0000_0000h	5.1.7.18/568
3038_A6A4	Post Divider Register (CCM_POST_ROOT77_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A6A8	Post Divider Register (CCM_POST_ROOT77_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A6AC	Post Divider Register (CCM_POST_ROOT77_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A6B0	Pre Divider Register (CCM_PRE77)	32	R/W	1000_0000h	5.1.7.22/580
3038_A6B4	Pre Divider Register (CCM_PRE_ROOT77_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A6B8	Pre Divider Register (CCM_PRE_ROOT77_CLR)	32	R/W	0000_0000h	5.1.7.24/586

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A6BC	Pre Divider Register (CCM_PRE_ROOT77_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A6F0	Access Control Register (CCM_ACCESS_CTRL77)	32	R/W	0000_0000h	5.1.7.26/592
3038_A6F4	Access Control Register (CCM_ACCESS_CTRL_ROOT77_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A6F8	Access Control Register (CCM_ACCESS_CTRL_ROOT77_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A6FC	Access Control Register (CCM_ACCESS_CTRL_ROOT77_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A700	Target Register (CCM_TARGET_ROOT78)	32	R/W	1000_0000h	5.1.7.10/556
3038_A704	Target Register (CCM_TARGET_ROOT78_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A708	Target Register (CCM_TARGET_ROOT78_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A70C	Target Register (CCM_TARGET_ROOT78_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A710	Miscellaneous Register (CCM_MISC78)	32	R/W	0000_0000h	5.1.7.14/564
3038_A714	Miscellaneous Register (CCM_MISC_ROOT78_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A718	Miscellaneous Register (CCM_MISC_ROOT78_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A71C	Miscellaneous Register (CCM_MISC_ROOT78_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A720	Post Divider Register (CCM_POST78)	32	R/W	0000_0000h	5.1.7.18/568
3038_A724	Post Divider Register (CCM_POST_ROOT78_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A728	Post Divider Register (CCM_POST_ROOT78_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A72C	Post Divider Register (CCM_POST_ROOT78_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A730	Pre Divider Register (CCM_PRE78)	32	R/W	1000_0000h	5.1.7.22/580
3038_A734	Pre Divider Register (CCM_PRE_ROOT78_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A738	Pre Divider Register (CCM_PRE_ROOT78_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A73C	Pre Divider Register (CCM_PRE_ROOT78_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A770	Access Control Register (CCM_ACCESS_CTRL78)	32	R/W	0000_0000h	5.1.7.26/592

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A774	Access Control Register (CCM_ACCESS_CTRL_ROOT78_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A778	Access Control Register (CCM_ACCESS_CTRL_ROOT78_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A77C	Access Control Register (CCM_ACCESS_CTRL_ROOT78_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A780	Target Register (CCM_TARGET_ROOT79)	32	R/W	1000_0000h	5.1.7.10/556
3038_A784	Target Register (CCM_TARGET_ROOT79_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A788	Target Register (CCM_TARGET_ROOT79_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A78C	Target Register (CCM_TARGET_ROOT79_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A790	Miscellaneous Register (CCM_MISC79)	32	R/W	0000_0000h	5.1.7.14/564
3038_A794	Miscellaneous Register (CCM_MISC_ROOT79_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A798	Miscellaneous Register (CCM_MISC_ROOT79_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A79C	Miscellaneous Register (CCM_MISC_ROOT79_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A7A0	Post Divider Register (CCM_POST79)	32	R/W	0000_0000h	5.1.7.18/568
3038_A7A4	Post Divider Register (CCM_POST_ROOT79_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A7A8	Post Divider Register (CCM_POST_ROOT79_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A7AC	Post Divider Register (CCM_POST_ROOT79_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A7B0	Pre Divider Register (CCM_PRE79)	32	R/W	1000_0000h	5.1.7.22/580
3038_A7B4	Pre Divider Register (CCM_PRE_ROOT79_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A7B8	Pre Divider Register (CCM_PRE_ROOT79_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A7BC	Pre Divider Register (CCM_PRE_ROOT79_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A7F0	Access Control Register (CCM_ACCESS_CTRL79)	32	R/W	0000_0000h	5.1.7.26/592
3038_A7F4	Access Control Register (CCM_ACCESS_CTRL_ROOT79_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A7F8	Access Control Register (CCM_ACCESS_CTRL_ROOT79_CLR)	32	R/W	0000_0000h	5.1.7.28/597

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A7FC	Access Control Register (CCM_ACCESS_CTRL_ROOT79_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A800	Target Register (CCM_TARGET_ROOT80)	32	R/W	1000_0000h	5.1.7.10/556
3038_A804	Target Register (CCM_TARGET_ROOT80_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A808	Target Register (CCM_TARGET_ROOT80_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A80C	Target Register (CCM_TARGET_ROOT80_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A810	Miscellaneous Register (CCM_MISC80)	32	R/W	0000_0000h	5.1.7.14/564
3038_A814	Miscellaneous Register (CCM_MISC_ROOT80_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A818	Miscellaneous Register (CCM_MISC_ROOT80_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A81C	Miscellaneous Register (CCM_MISC_ROOT80_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A820	Post Divider Register (CCM_POST80)	32	R/W	0000_0000h	5.1.7.18/568
3038_A824	Post Divider Register (CCM_POST_ROOT80_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A828	Post Divider Register (CCM_POST_ROOT80_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A82C	Post Divider Register (CCM_POST_ROOT80_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A830	Pre Divider Register (CCM_PRE80)	32	R/W	1000_0000h	5.1.7.22/580
3038_A834	Pre Divider Register (CCM_PRE_ROOT80_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A838	Pre Divider Register (CCM_PRE_ROOT80_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A83C	Pre Divider Register (CCM_PRE_ROOT80_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A870	Access Control Register (CCM_ACCESS_CTRL80)	32	R/W	0000_0000h	5.1.7.26/592
3038_A874	Access Control Register (CCM_ACCESS_CTRL_ROOT80_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A878	Access Control Register (CCM_ACCESS_CTRL_ROOT80_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A87C	Access Control Register (CCM_ACCESS_CTRL_ROOT80_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A880	Target Register (CCM_TARGET_ROOT81)	32	R/W	1000_0000h	5.1.7.10/556

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A884	Target Register (CCM_TARGET_ROOT81_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A888	Target Register (CCM_TARGET_ROOT81_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A88C	Target Register (CCM_TARGET_ROOT81_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A890	Miscellaneous Register (CCM_MISC81)	32	R/W	0000_0000h	5.1.7.14/564
3038_A894	Miscellaneous Register (CCM_MISC_ROOT81_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A898	Miscellaneous Register (CCM_MISC_ROOT81_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A89C	Miscellaneous Register (CCM_MISC_ROOT81_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A8A0	Post Divider Register (CCM_POST81)	32	R/W	0000_0000h	5.1.7.18/568
3038_A8A4	Post Divider Register (CCM_POST_ROOT81_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A8A8	Post Divider Register (CCM_POST_ROOT81_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A8AC	Post Divider Register (CCM_POST_ROOT81_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A8B0	Pre Divider Register (CCM_PRE81)	32	R/W	1000_0000h	5.1.7.22/580
3038_A8B4	Pre Divider Register (CCM_PRE_ROOT81_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A8B8	Pre Divider Register (CCM_PRE_ROOT81_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A8BC	Pre Divider Register (CCM_PRE_ROOT81_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A8F0	Access Control Register (CCM_ACCESS_CTRL81)	32	R/W	0000_0000h	5.1.7.26/592
3038_A8F4	Access Control Register (CCM_ACCESS_CTRL_ROOT81_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A8F8	Access Control Register (CCM_ACCESS_CTRL_ROOT81_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A8FC	Access Control Register (CCM_ACCESS_CTRL_ROOT81_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A900	Target Register (CCM_TARGET_ROOT82)	32	R/W	1000_0000h	5.1.7.10/556
3038_A904	Target Register (CCM_TARGET_ROOT82_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A908	Target Register (CCM_TARGET_ROOT82_CLR)	32	R/W	0000_0000h	5.1.7.12/560

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A90C	Target Register (CCM_TARGET_ROOT82_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A910	Miscellaneous Register (CCM_MISC82)	32	R/W	0000_0000h	5.1.7.14/564
3038_A914	Miscellaneous Register (CCM_MISC_ROOT82_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A918	Miscellaneous Register (CCM_MISC_ROOT82_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A91C	Miscellaneous Register (CCM_MISC_ROOT82_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A920	Post Divider Register (CCM_POST82)	32	R/W	0000_0000h	5.1.7.18/568
3038_A924	Post Divider Register (CCM_POST_ROOT82_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A928	Post Divider Register (CCM_POST_ROOT82_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A92C	Post Divider Register (CCM_POST_ROOT82_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A930	Pre Divider Register (CCM_PRE82)	32	R/W	1000_0000h	5.1.7.22/580
3038_A934	Pre Divider Register (CCM_PRE_ROOT82_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A938	Pre Divider Register (CCM_PRE_ROOT82_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A93C	Pre Divider Register (CCM_PRE_ROOT82_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A970	Access Control Register (CCM_ACCESS_CTRL82)	32	R/W	0000_0000h	5.1.7.26/592
3038_A974	Access Control Register (CCM_ACCESS_CTRL_ROOT82_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A978	Access Control Register (CCM_ACCESS_CTRL_ROOT82_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A97C	Access Control Register (CCM_ACCESS_CTRL_ROOT82_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_A980	Target Register (CCM_TARGET_ROOT83)	32	R/W	1000_0000h	5.1.7.10/556
3038_A984	Target Register (CCM_TARGET_ROOT83_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_A988	Target Register (CCM_TARGET_ROOT83_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_A98C	Target Register (CCM_TARGET_ROOT83_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_A990	Miscellaneous Register (CCM_MISC83)	32	R/W	0000_0000h	5.1.7.14/564

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_A994	Miscellaneous Register (CCM_MISC_ROOT83_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_A998	Miscellaneous Register (CCM_MISC_ROOT83_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_A99C	Miscellaneous Register (CCM_MISC_ROOT83_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_A9A0	Post Divider Register (CCM_POST83)	32	R/W	0000_0000h	5.1.7.18/568
3038_A9A4	Post Divider Register (CCM_POST_ROOT83_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_A9A8	Post Divider Register (CCM_POST_ROOT83_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_A9AC	Post Divider Register (CCM_POST_ROOT83_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_A9B0	Pre Divider Register (CCM_PRE83)	32	R/W	1000_0000h	5.1.7.22/580
3038_A9B4	Pre Divider Register (CCM_PRE_ROOT83_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_A9B8	Pre Divider Register (CCM_PRE_ROOT83_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_A9BC	Pre Divider Register (CCM_PRE_ROOT83_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_A9F0	Access Control Register (CCM_ACCESS_CTRL83)	32	R/W	0000_0000h	5.1.7.26/592
3038_A9F4	Access Control Register (CCM_ACCESS_CTRL_ROOT83_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_A9F8	Access Control Register (CCM_ACCESS_CTRL_ROOT83_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_A9FC	Access Control Register (CCM_ACCESS_CTRL_ROOT83_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_AA00	Target Register (CCM_TARGET_ROOT84)	32	R/W	1000_0000h	5.1.7.10/556
3038_AA04	Target Register (CCM_TARGET_ROOT84_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_AA08	Target Register (CCM_TARGET_ROOT84_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_AA0C	Target Register (CCM_TARGET_ROOT84_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_AA10	Miscellaneous Register (CCM_MISC84)	32	R/W	0000_0000h	5.1.7.14/564
3038_AA14	Miscellaneous Register (CCM_MISC_ROOT84_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_AA18	Miscellaneous Register (CCM_MISC_ROOT84_CLR)	32	R/W	0000_0000h	5.1.7.16/566

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_AA1C	Miscellaneous Register (CCM_MISC_ROOT84_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_AA20	Post Divider Register (CCM_POST84)	32	R/W	0000_0000h	5.1.7.18/568
3038_AA24	Post Divider Register (CCM_POST_ROOT84_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_AA28	Post Divider Register (CCM_POST_ROOT84_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_AA2C	Post Divider Register (CCM_POST_ROOT84_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_AA30	Pre Divider Register (CCM_PRE84)	32	R/W	1000_0000h	5.1.7.22/580
3038_AA34	Pre Divider Register (CCM_PRE_ROOT84_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_AA38	Pre Divider Register (CCM_PRE_ROOT84_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_AA3C	Pre Divider Register (CCM_PRE_ROOT84_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_AA70	Access Control Register (CCM_ACCESS_CTRL84)	32	R/W	0000_0000h	5.1.7.26/592
3038_AA74	Access Control Register (CCM_ACCESS_CTRL_ROOT84_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_AA78	Access Control Register (CCM_ACCESS_CTRL_ROOT84_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_AA7C	Access Control Register (CCM_ACCESS_CTRL_ROOT84_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_AA80	Target Register (CCM_TARGET_ROOT85)	32	R/W	1000_0000h	5.1.7.10/556
3038_AA84	Target Register (CCM_TARGET_ROOT85_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_AA88	Target Register (CCM_TARGET_ROOT85_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_AA8C	Target Register (CCM_TARGET_ROOT85_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_AA90	Miscellaneous Register (CCM_MISC85)	32	R/W	0000_0000h	5.1.7.14/564
3038_AA94	Miscellaneous Register (CCM_MISC_ROOT85_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_AA98	Miscellaneous Register (CCM_MISC_ROOT85_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_AA9C	Miscellaneous Register (CCM_MISC_ROOT85_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_AAA0	Post Divider Register (CCM_POST85)	32	R/W	0000_0000h	5.1.7.18/568

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_AAA4	Post Divider Register (CCM_POST_ROOT85_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_AAA8	Post Divider Register (CCM_POST_ROOT85_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_AAAC	Post Divider Register (CCM_POST_ROOT85_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_AAB0	Pre Divider Register (CCM_PRE85)	32	R/W	1000_0000h	5.1.7.22/580
3038_AAB4	Pre Divider Register (CCM_PRE_ROOT85_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_AAB8	Pre Divider Register (CCM_PRE_ROOT85_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_AABC	Pre Divider Register (CCM_PRE_ROOT85_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_AAF0	Access Control Register (CCM_ACCESS_CTRL85)	32	R/W	0000_0000h	5.1.7.26/592
3038_AAF4	Access Control Register (CCM_ACCESS_CTRL_ROOT85_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_AAF8	Access Control Register (CCM_ACCESS_CTRL_ROOT85_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_AAFc	Access Control Register (CCM_ACCESS_CTRL_ROOT85_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_AB00	Target Register (CCM_TARGET_ROOT86)	32	R/W	1000_0000h	5.1.7.10/556
3038_AB04	Target Register (CCM_TARGET_ROOT86_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_AB08	Target Register (CCM_TARGET_ROOT86_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_AB0C	Target Register (CCM_TARGET_ROOT86_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_AB10	Miscellaneous Register (CCM_MISC86)	32	R/W	0000_0000h	5.1.7.14/564
3038_AB14	Miscellaneous Register (CCM_MISC_ROOT86_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_AB18	Miscellaneous Register (CCM_MISC_ROOT86_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_AB1C	Miscellaneous Register (CCM_MISC_ROOT86_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_AB20	Post Divider Register (CCM_POST86)	32	R/W	0000_0000h	5.1.7.18/568
3038_AB24	Post Divider Register (CCM_POST_ROOT86_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_AB28	Post Divider Register (CCM_POST_ROOT86_CLR)	32	R/W	0000_0000h	5.1.7.20/574

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_AB2C	Post Divider Register (CCM_POST_ROOT86_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_AB30	Pre Divider Register (CCM_PRE86)	32	R/W	1000_0000h	5.1.7.22/580
3038_AB34	Pre Divider Register (CCM_PRE_ROOT86_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_AB38	Pre Divider Register (CCM_PRE_ROOT86_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_AB3C	Pre Divider Register (CCM_PRE_ROOT86_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_AB70	Access Control Register (CCM_ACCESS_CTRL86)	32	R/W	0000_0000h	5.1.7.26/592
3038_AB74	Access Control Register (CCM_ACCESS_CTRL_ROOT86_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_AB78	Access Control Register (CCM_ACCESS_CTRL_ROOT86_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_AB7C	Access Control Register (CCM_ACCESS_CTRL_ROOT86_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_AB80	Target Register (CCM_TARGET_ROOT87)	32	R/W	1000_0000h	5.1.7.10/556
3038_AB84	Target Register (CCM_TARGET_ROOT87_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_AB88	Target Register (CCM_TARGET_ROOT87_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_AB8C	Target Register (CCM_TARGET_ROOT87_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_AB90	Miscellaneous Register (CCM_MISC87)	32	R/W	0000_0000h	5.1.7.14/564
3038_AB94	Miscellaneous Register (CCM_MISC_ROOT87_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_AB98	Miscellaneous Register (CCM_MISC_ROOT87_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_AB9C	Miscellaneous Register (CCM_MISC_ROOT87_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_ABA0	Post Divider Register (CCM_POST87)	32	R/W	0000_0000h	5.1.7.18/568
3038_ABA4	Post Divider Register (CCM_POST_ROOT87_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_ABA8	Post Divider Register (CCM_POST_ROOT87_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_ABAC	Post Divider Register (CCM_POST_ROOT87_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_ABB0	Pre Divider Register (CCM_PRE87)	32	R/W	1000_0000h	5.1.7.22/580

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_ABB4	Pre Divider Register (CCM_PRE_ROOT87_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_ABB8	Pre Divider Register (CCM_PRE_ROOT87_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_ABBC	Pre Divider Register (CCM_PRE_ROOT87_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_ABF0	Access Control Register (CCM_ACCESS_CTRL87)	32	R/W	0000_0000h	5.1.7.26/592
3038_ABF4	Access Control Register (CCM_ACCESS_CTRL_ROOT87_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_ABF8	Access Control Register (CCM_ACCESS_CTRL_ROOT87_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_ABFC	Access Control Register (CCM_ACCESS_CTRL_ROOT87_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_AC00	Target Register (CCM_TARGET_ROOT88)	32	R/W	1000_0000h	5.1.7.10/556
3038_AC04	Target Register (CCM_TARGET_ROOT88_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_AC08	Target Register (CCM_TARGET_ROOT88_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_AC0C	Target Register (CCM_TARGET_ROOT88_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_AC10	Miscellaneous Register (CCM_MISC88)	32	R/W	0000_0000h	5.1.7.14/564
3038_AC14	Miscellaneous Register (CCM_MISC_ROOT88_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_AC18	Miscellaneous Register (CCM_MISC_ROOT88_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_AC1C	Miscellaneous Register (CCM_MISC_ROOT88_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_AC20	Post Divider Register (CCM_POST88)	32	R/W	0000_0000h	5.1.7.18/568
3038_AC24	Post Divider Register (CCM_POST_ROOT88_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_AC28	Post Divider Register (CCM_POST_ROOT88_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_AC2C	Post Divider Register (CCM_POST_ROOT88_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_AC30	Pre Divider Register (CCM_PRE88)	32	R/W	1000_0000h	5.1.7.22/580
3038_AC34	Pre Divider Register (CCM_PRE_ROOT88_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_AC38	Pre Divider Register (CCM_PRE_ROOT88_CLR)	32	R/W	0000_0000h	5.1.7.24/586

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_AC3C	Pre Divider Register (CCM_PRE_ROOT88_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_AC70	Access Control Register (CCM_ACCESS_CTRL88)	32	R/W	0000_0000h	5.1.7.26/592
3038_AC74	Access Control Register (CCM_ACCESS_CTRL_ROOT88_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_AC78	Access Control Register (CCM_ACCESS_CTRL_ROOT88_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_AC7C	Access Control Register (CCM_ACCESS_CTRL_ROOT88_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_AC80	Target Register (CCM_TARGET_ROOT89)	32	R/W	1000_0000h	5.1.7.10/556
3038_AC84	Target Register (CCM_TARGET_ROOT89_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_AC88	Target Register (CCM_TARGET_ROOT89_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_AC8C	Target Register (CCM_TARGET_ROOT89_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_AC90	Miscellaneous Register (CCM_MISC89)	32	R/W	0000_0000h	5.1.7.14/564
3038_AC94	Miscellaneous Register (CCM_MISC_ROOT89_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_AC98	Miscellaneous Register (CCM_MISC_ROOT89_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_AC9C	Miscellaneous Register (CCM_MISC_ROOT89_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_ACA0	Post Divider Register (CCM_POST89)	32	R/W	0000_0000h	5.1.7.18/568
3038_ACA4	Post Divider Register (CCM_POST_ROOT89_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_ACA8	Post Divider Register (CCM_POST_ROOT89_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_ACAC	Post Divider Register (CCM_POST_ROOT89_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_ACB0	Pre Divider Register (CCM_PRE89)	32	R/W	1000_0000h	5.1.7.22/580
3038_ACB4	Pre Divider Register (CCM_PRE_ROOT89_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_ACB8	Pre Divider Register (CCM_PRE_ROOT89_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_ACBC	Pre Divider Register (CCM_PRE_ROOT89_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_ACF0	Access Control Register (CCM_ACCESS_CTRL89)	32	R/W	0000_0000h	5.1.7.26/592

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_ACF4	Access Control Register (CCM_ACCESS_CTRL_ROOT89_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_ACF8	Access Control Register (CCM_ACCESS_CTRL_ROOT89_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_ACFC	Access Control Register (CCM_ACCESS_CTRL_ROOT89_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_AD00	Target Register (CCM_TARGET_ROOT90)	32	R/W	1000_0000h	5.1.7.10/556
3038_AD04	Target Register (CCM_TARGET_ROOT90_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_AD08	Target Register (CCM_TARGET_ROOT90_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_AD0C	Target Register (CCM_TARGET_ROOT90_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_AD10	Miscellaneous Register (CCM_MISC90)	32	R/W	0000_0000h	5.1.7.14/564
3038_AD14	Miscellaneous Register (CCM_MISC_ROOT90_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_AD18	Miscellaneous Register (CCM_MISC_ROOT90_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_AD1C	Miscellaneous Register (CCM_MISC_ROOT90_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_AD20	Post Divider Register (CCM_POST90)	32	R/W	0000_0000h	5.1.7.18/568
3038_AD24	Post Divider Register (CCM_POST_ROOT90_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_AD28	Post Divider Register (CCM_POST_ROOT90_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_AD2C	Post Divider Register (CCM_POST_ROOT90_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_AD30	Pre Divider Register (CCM_PRE90)	32	R/W	1000_0000h	5.1.7.22/580
3038_AD34	Pre Divider Register (CCM_PRE_ROOT90_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_AD38	Pre Divider Register (CCM_PRE_ROOT90_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_AD3C	Pre Divider Register (CCM_PRE_ROOT90_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_AD70	Access Control Register (CCM_ACCESS_CTRL90)	32	R/W	0000_0000h	5.1.7.26/592
3038_AD74	Access Control Register (CCM_ACCESS_CTRL_ROOT90_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_AD78	Access Control Register (CCM_ACCESS_CTRL_ROOT90_CLR)	32	R/W	0000_0000h	5.1.7.28/597

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_AD7C	Access Control Register (CCM_ACCESS_CTRL_ROOT90_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_AD80	Target Register (CCM_TARGET_ROOT91)	32	R/W	1000_0000h	5.1.7.10/556
3038_AD84	Target Register (CCM_TARGET_ROOT91_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_AD88	Target Register (CCM_TARGET_ROOT91_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_AD8C	Target Register (CCM_TARGET_ROOT91_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_AD90	Miscellaneous Register (CCM_MISC91)	32	R/W	0000_0000h	5.1.7.14/564
3038_AD94	Miscellaneous Register (CCM_MISC_ROOT91_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_AD98	Miscellaneous Register (CCM_MISC_ROOT91_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_AD9C	Miscellaneous Register (CCM_MISC_ROOT91_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_ADA0	Post Divider Register (CCM_POST91)	32	R/W	0000_0000h	5.1.7.18/568
3038_ADA4	Post Divider Register (CCM_POST_ROOT91_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_ADA8	Post Divider Register (CCM_POST_ROOT91_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_ADAC	Post Divider Register (CCM_POST_ROOT91_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_ADB0	Pre Divider Register (CCM_PRE91)	32	R/W	1000_0000h	5.1.7.22/580
3038_ADB4	Pre Divider Register (CCM_PRE_ROOT91_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_ADB8	Pre Divider Register (CCM_PRE_ROOT91_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_ADBC	Pre Divider Register (CCM_PRE_ROOT91_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_ADF0	Access Control Register (CCM_ACCESS_CTRL91)	32	R/W	0000_0000h	5.1.7.26/592
3038_ADF4	Access Control Register (CCM_ACCESS_CTRL_ROOT91_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_ADF8	Access Control Register (CCM_ACCESS_CTRL_ROOT91_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_ADFC	Access Control Register (CCM_ACCESS_CTRL_ROOT91_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_AE00	Target Register (CCM_TARGET_ROOT92)	32	R/W	1000_0000h	5.1.7.10/556

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_AE04	Target Register (CCM_TARGET_ROOT92_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_AE08	Target Register (CCM_TARGET_ROOT92_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_AE0C	Target Register (CCM_TARGET_ROOT92_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_AE10	Miscellaneous Register (CCM_MISC92)	32	R/W	0000_0000h	5.1.7.14/564
3038_AE14	Miscellaneous Register (CCM_MISC_ROOT92_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_AE18	Miscellaneous Register (CCM_MISC_ROOT92_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_AE1C	Miscellaneous Register (CCM_MISC_ROOT92_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_AE20	Post Divider Register (CCM_POST92)	32	R/W	0000_0000h	5.1.7.18/568
3038_AE24	Post Divider Register (CCM_POST_ROOT92_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_AE28	Post Divider Register (CCM_POST_ROOT92_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_AE2C	Post Divider Register (CCM_POST_ROOT92_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_AE30	Pre Divider Register (CCM_PRE92)	32	R/W	1000_0000h	5.1.7.22/580
3038_AE34	Pre Divider Register (CCM_PRE_ROOT92_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_AE38	Pre Divider Register (CCM_PRE_ROOT92_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_AE3C	Pre Divider Register (CCM_PRE_ROOT92_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_AE70	Access Control Register (CCM_ACCESS_CTRL92)	32	R/W	0000_0000h	5.1.7.26/592
3038_AE74	Access Control Register (CCM_ACCESS_CTRL_ROOT92_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_AE78	Access Control Register (CCM_ACCESS_CTRL_ROOT92_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_AE7C	Access Control Register (CCM_ACCESS_CTRL_ROOT92_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_AE80	Target Register (CCM_TARGET_ROOT93)	32	R/W	1000_0000h	5.1.7.10/556
3038_AE84	Target Register (CCM_TARGET_ROOT93_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_AE88	Target Register (CCM_TARGET_ROOT93_CLR)	32	R/W	0000_0000h	5.1.7.12/560

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_AE8C	Target Register (CCM_TARGET_ROOT93_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_AE90	Miscellaneous Register (CCM_MISC93)	32	R/W	0000_0000h	5.1.7.14/564
3038_AE94	Miscellaneous Register (CCM_MISC_ROOT93_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_AE98	Miscellaneous Register (CCM_MISC_ROOT93_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_AE9C	Miscellaneous Register (CCM_MISC_ROOT93_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_AEA0	Post Divider Register (CCM_POST93)	32	R/W	0000_0000h	5.1.7.18/568
3038_AEA4	Post Divider Register (CCM_POST_ROOT93_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_AEA8	Post Divider Register (CCM_POST_ROOT93_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_AEAC	Post Divider Register (CCM_POST_ROOT93_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_AEB0	Pre Divider Register (CCM_PRE93)	32	R/W	1000_0000h	5.1.7.22/580
3038_AEB4	Pre Divider Register (CCM_PRE_ROOT93_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_AEB8	Pre Divider Register (CCM_PRE_ROOT93_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_AEBC	Pre Divider Register (CCM_PRE_ROOT93_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_AEF0	Access Control Register (CCM_ACCESS_CTRL93)	32	R/W	0000_0000h	5.1.7.26/592
3038_AEF4	Access Control Register (CCM_ACCESS_CTRL_ROOT93_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_AEF8	Access Control Register (CCM_ACCESS_CTRL_ROOT93_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_AEFC	Access Control Register (CCM_ACCESS_CTRL_ROOT93_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_AF00	Target Register (CCM_TARGET_ROOT94)	32	R/W	1000_0000h	5.1.7.10/556
3038_AF04	Target Register (CCM_TARGET_ROOT94_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_AF08	Target Register (CCM_TARGET_ROOT94_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_AF0C	Target Register (CCM_TARGET_ROOT94_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_AF10	Miscellaneous Register (CCM_MISC94)	32	R/W	0000_0000h	5.1.7.14/564

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_AF14	Miscellaneous Register (CCM_MISC_ROOT94_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_AF18	Miscellaneous Register (CCM_MISC_ROOT94_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_AF1C	Miscellaneous Register (CCM_MISC_ROOT94_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_AF20	Post Divider Register (CCM_POST94)	32	R/W	0000_0000h	5.1.7.18/568
3038_AF24	Post Divider Register (CCM_POST_ROOT94_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_AF28	Post Divider Register (CCM_POST_ROOT94_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_AF2C	Post Divider Register (CCM_POST_ROOT94_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_AF30	Pre Divider Register (CCM_PRE94)	32	R/W	1000_0000h	5.1.7.22/580
3038_AF34	Pre Divider Register (CCM_PRE_ROOT94_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_AF38	Pre Divider Register (CCM_PRE_ROOT94_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_AF3C	Pre Divider Register (CCM_PRE_ROOT94_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_AF70	Access Control Register (CCM_ACCESS_CTRL94)	32	R/W	0000_0000h	5.1.7.26/592
3038_AF74	Access Control Register (CCM_ACCESS_CTRL_ROOT94_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_AF78	Access Control Register (CCM_ACCESS_CTRL_ROOT94_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_AF7C	Access Control Register (CCM_ACCESS_CTRL_ROOT94_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_AF80	Target Register (CCM_TARGET_ROOT95)	32	R/W	1000_0000h	5.1.7.10/556
3038_AF84	Target Register (CCM_TARGET_ROOT95_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_AF88	Target Register (CCM_TARGET_ROOT95_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_AF8C	Target Register (CCM_TARGET_ROOT95_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_AF90	Miscellaneous Register (CCM_MISC95)	32	R/W	0000_0000h	5.1.7.14/564
3038_AF94	Miscellaneous Register (CCM_MISC_ROOT95_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_AF98	Miscellaneous Register (CCM_MISC_ROOT95_CLR)	32	R/W	0000_0000h	5.1.7.16/566

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_AF9C	Miscellaneous Register (CCM_MISC_ROOT95_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_AFA0	Post Divider Register (CCM_POST95)	32	R/W	0000_0000h	5.1.7.18/568
3038_AFA4	Post Divider Register (CCM_POST_ROOT95_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_AFA8	Post Divider Register (CCM_POST_ROOT95_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_AFAC	Post Divider Register (CCM_POST_ROOT95_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_AFB0	Pre Divider Register (CCM_PRE95)	32	R/W	1000_0000h	5.1.7.22/580
3038_AFB4	Pre Divider Register (CCM_PRE_ROOT95_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_AFB8	Pre Divider Register (CCM_PRE_ROOT95_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_AFBC	Pre Divider Register (CCM_PRE_ROOT95_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_AFF0	Access Control Register (CCM_ACCESS_CTRL95)	32	R/W	0000_0000h	5.1.7.26/592
3038_AFF4	Access Control Register (CCM_ACCESS_CTRL_ROOT95_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_AFF8	Access Control Register (CCM_ACCESS_CTRL_ROOT95_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_AFFC	Access Control Register (CCM_ACCESS_CTRL_ROOT95_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B000	Target Register (CCM_TARGET_ROOT96)	32	R/W	1000_0000h	5.1.7.10/556
3038_B004	Target Register (CCM_TARGET_ROOT96_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B008	Target Register (CCM_TARGET_ROOT96_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B00C	Target Register (CCM_TARGET_ROOT96_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B010	Miscellaneous Register (CCM_MISC96)	32	R/W	0000_0000h	5.1.7.14/564
3038_B014	Miscellaneous Register (CCM_MISC_ROOT96_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B018	Miscellaneous Register (CCM_MISC_ROOT96_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B01C	Miscellaneous Register (CCM_MISC_ROOT96_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B020	Post Divider Register (CCM_POST96)	32	R/W	0000_0000h	5.1.7.18/568

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B024	Post Divider Register (CCM_POST_ROOT96_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B028	Post Divider Register (CCM_POST_ROOT96_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B02C	Post Divider Register (CCM_POST_ROOT96_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B030	Pre Divider Register (CCM_PRE96)	32	R/W	1000_0000h	5.1.7.22/580
3038_B034	Pre Divider Register (CCM_PRE_ROOT96_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B038	Pre Divider Register (CCM_PRE_ROOT96_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B03C	Pre Divider Register (CCM_PRE_ROOT96_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B070	Access Control Register (CCM_ACCESS_CTRL96)	32	R/W	0000_0000h	5.1.7.26/592
3038_B074	Access Control Register (CCM_ACCESS_CTRL_ROOT96_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B078	Access Control Register (CCM_ACCESS_CTRL_ROOT96_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B07C	Access Control Register (CCM_ACCESS_CTRL_ROOT96_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B080	Target Register (CCM_TARGET_ROOT97)	32	R/W	1000_0000h	5.1.7.10/556
3038_B084	Target Register (CCM_TARGET_ROOT97_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B088	Target Register (CCM_TARGET_ROOT97_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B08C	Target Register (CCM_TARGET_ROOT97_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B090	Miscellaneous Register (CCM_MISC97)	32	R/W	0000_0000h	5.1.7.14/564
3038_B094	Miscellaneous Register (CCM_MISC_ROOT97_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B098	Miscellaneous Register (CCM_MISC_ROOT97_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B09C	Miscellaneous Register (CCM_MISC_ROOT97_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B0A0	Post Divider Register (CCM_POST97)	32	R/W	0000_0000h	5.1.7.18/568
3038_B0A4	Post Divider Register (CCM_POST_ROOT97_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B0A8	Post Divider Register (CCM_POST_ROOT97_CLR)	32	R/W	0000_0000h	5.1.7.20/574

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B0AC	Post Divider Register (CCM_POST_ROOT97_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B0B0	Pre Divider Register (CCM_PRE97)	32	R/W	1000_0000h	5.1.7.22/580
3038_B0B4	Pre Divider Register (CCM_PRE_ROOT97_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B0B8	Pre Divider Register (CCM_PRE_ROOT97_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B0BC	Pre Divider Register (CCM_PRE_ROOT97_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B0F0	Access Control Register (CCM_ACCESS_CTRL97)	32	R/W	0000_0000h	5.1.7.26/592
3038_B0F4	Access Control Register (CCM_ACCESS_CTRL_ROOT97_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B0F8	Access Control Register (CCM_ACCESS_CTRL_ROOT97_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B0FC	Access Control Register (CCM_ACCESS_CTRL_ROOT97_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B100	Target Register (CCM_TARGET_ROOT98)	32	R/W	1000_0000h	5.1.7.10/556
3038_B104	Target Register (CCM_TARGET_ROOT98_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B108	Target Register (CCM_TARGET_ROOT98_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B10C	Target Register (CCM_TARGET_ROOT98_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B110	Miscellaneous Register (CCM_MISC98)	32	R/W	0000_0000h	5.1.7.14/564
3038_B114	Miscellaneous Register (CCM_MISC_ROOT98_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B118	Miscellaneous Register (CCM_MISC_ROOT98_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B11C	Miscellaneous Register (CCM_MISC_ROOT98_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B120	Post Divider Register (CCM_POST98)	32	R/W	0000_0000h	5.1.7.18/568
3038_B124	Post Divider Register (CCM_POST_ROOT98_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B128	Post Divider Register (CCM_POST_ROOT98_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B12C	Post Divider Register (CCM_POST_ROOT98_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B130	Pre Divider Register (CCM_PRE98)	32	R/W	1000_0000h	5.1.7.22/580

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B134	Pre Divider Register (CCM_PRE_ROOT98_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B138	Pre Divider Register (CCM_PRE_ROOT98_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B13C	Pre Divider Register (CCM_PRE_ROOT98_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B170	Access Control Register (CCM_ACCESS_CTRL98)	32	R/W	0000_0000h	5.1.7.26/592
3038_B174	Access Control Register (CCM_ACCESS_CTRL_ROOT98_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B178	Access Control Register (CCM_ACCESS_CTRL_ROOT98_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B17C	Access Control Register (CCM_ACCESS_CTRL_ROOT98_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B180	Target Register (CCM_TARGET_ROOT99)	32	R/W	1000_0000h	5.1.7.10/556
3038_B184	Target Register (CCM_TARGET_ROOT99_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B188	Target Register (CCM_TARGET_ROOT99_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B18C	Target Register (CCM_TARGET_ROOT99_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B190	Miscellaneous Register (CCM_MISC99)	32	R/W	0000_0000h	5.1.7.14/564
3038_B194	Miscellaneous Register (CCM_MISC_ROOT99_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B198	Miscellaneous Register (CCM_MISC_ROOT99_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B19C	Miscellaneous Register (CCM_MISC_ROOT99_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B1A0	Post Divider Register (CCM_POST99)	32	R/W	0000_0000h	5.1.7.18/568
3038_B1A4	Post Divider Register (CCM_POST_ROOT99_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B1A8	Post Divider Register (CCM_POST_ROOT99_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B1AC	Post Divider Register (CCM_POST_ROOT99_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B1B0	Pre Divider Register (CCM_PRE99)	32	R/W	1000_0000h	5.1.7.22/580
3038_B1B4	Pre Divider Register (CCM_PRE_ROOT99_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B1B8	Pre Divider Register (CCM_PRE_ROOT99_CLR)	32	R/W	0000_0000h	5.1.7.24/586

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B1BC	Pre Divider Register (CCM_PRE_ROOT99_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B1F0	Access Control Register (CCM_ACCESS_CTRL99)	32	R/W	0000_0000h	5.1.7.26/592
3038_B1F4	Access Control Register (CCM_ACCESS_CTRL_ROOT99_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B1F8	Access Control Register (CCM_ACCESS_CTRL_ROOT99_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B1FC	Access Control Register (CCM_ACCESS_CTRL_ROOT99_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B200	Target Register (CCM_TARGET_ROOT100)	32	R/W	1000_0000h	5.1.7.10/556
3038_B204	Target Register (CCM_TARGET_ROOT100_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B208	Target Register (CCM_TARGET_ROOT100_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B20C	Target Register (CCM_TARGET_ROOT100_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B210	Miscellaneous Register (CCM_MISC100)	32	R/W	0000_0000h	5.1.7.14/564
3038_B214	Miscellaneous Register (CCM_MISC_ROOT100_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B218	Miscellaneous Register (CCM_MISC_ROOT100_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B21C	Miscellaneous Register (CCM_MISC_ROOT100_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B220	Post Divider Register (CCM_POST100)	32	R/W	0000_0000h	5.1.7.18/568
3038_B224	Post Divider Register (CCM_POST_ROOT100_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B228	Post Divider Register (CCM_POST_ROOT100_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B22C	Post Divider Register (CCM_POST_ROOT100_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B230	Pre Divider Register (CCM_PRE100)	32	R/W	1000_0000h	5.1.7.22/580
3038_B234	Pre Divider Register (CCM_PRE_ROOT100_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B238	Pre Divider Register (CCM_PRE_ROOT100_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B23C	Pre Divider Register (CCM_PRE_ROOT100_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B270	Access Control Register (CCM_ACCESS_CTRL100)	32	R/W	0000_0000h	5.1.7.26/592

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B274	Access Control Register (CCM_ACCESS_CTRL_ROOT100_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B278	Access Control Register (CCM_ACCESS_CTRL_ROOT100_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B27C	Access Control Register (CCM_ACCESS_CTRL_ROOT100_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B280	Target Register (CCM_TARGET_ROOT101)	32	R/W	1000_0000h	5.1.7.10/556
3038_B284	Target Register (CCM_TARGET_ROOT101_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B288	Target Register (CCM_TARGET_ROOT101_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B28C	Target Register (CCM_TARGET_ROOT101_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B290	Miscellaneous Register (CCM_MISC101)	32	R/W	0000_0000h	5.1.7.14/564
3038_B294	Miscellaneous Register (CCM_MISC_ROOT101_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B298	Miscellaneous Register (CCM_MISC_ROOT101_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B29C	Miscellaneous Register (CCM_MISC_ROOT101_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B2A0	Post Divider Register (CCM_POST101)	32	R/W	0000_0000h	5.1.7.18/568
3038_B2A4	Post Divider Register (CCM_POST_ROOT101_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B2A8	Post Divider Register (CCM_POST_ROOT101_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B2AC	Post Divider Register (CCM_POST_ROOT101_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B2B0	Pre Divider Register (CCM_PRE101)	32	R/W	1000_0000h	5.1.7.22/580
3038_B2B4	Pre Divider Register (CCM_PRE_ROOT101_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B2B8	Pre Divider Register (CCM_PRE_ROOT101_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B2BC	Pre Divider Register (CCM_PRE_ROOT101_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B2F0	Access Control Register (CCM_ACCESS_CTRL101)	32	R/W	0000_0000h	5.1.7.26/592
3038_B2F4	Access Control Register (CCM_ACCESS_CTRL_ROOT101_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B2F8	Access Control Register (CCM_ACCESS_CTRL_ROOT101_CLR)	32	R/W	0000_0000h	5.1.7.28/597

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B2FC	Access Control Register (CCM_ACCESS_CTRL_ROOT101_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B300	Target Register (CCM_TARGET_ROOT102)	32	R/W	1000_0000h	5.1.7.10/556
3038_B304	Target Register (CCM_TARGET_ROOT102_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B308	Target Register (CCM_TARGET_ROOT102_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B30C	Target Register (CCM_TARGET_ROOT102_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B310	Miscellaneous Register (CCM_MISC102)	32	R/W	0000_0000h	5.1.7.14/564
3038_B314	Miscellaneous Register (CCM_MISC_ROOT102_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B318	Miscellaneous Register (CCM_MISC_ROOT102_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B31C	Miscellaneous Register (CCM_MISC_ROOT102_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B320	Post Divider Register (CCM_POST102)	32	R/W	0000_0000h	5.1.7.18/568
3038_B324	Post Divider Register (CCM_POST_ROOT102_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B328	Post Divider Register (CCM_POST_ROOT102_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B32C	Post Divider Register (CCM_POST_ROOT102_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B330	Pre Divider Register (CCM_PRE102)	32	R/W	1000_0000h	5.1.7.22/580
3038_B334	Pre Divider Register (CCM_PRE_ROOT102_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B338	Pre Divider Register (CCM_PRE_ROOT102_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B33C	Pre Divider Register (CCM_PRE_ROOT102_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B370	Access Control Register (CCM_ACCESS_CTRL102)	32	R/W	0000_0000h	5.1.7.26/592
3038_B374	Access Control Register (CCM_ACCESS_CTRL_ROOT102_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B378	Access Control Register (CCM_ACCESS_CTRL_ROOT102_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B37C	Access Control Register (CCM_ACCESS_CTRL_ROOT102_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B380	Target Register (CCM_TARGET_ROOT103)	32	R/W	1000_0000h	5.1.7.10/556

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B384	Target Register (CCM_TARGET_ROOT103_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B388	Target Register (CCM_TARGET_ROOT103_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B38C	Target Register (CCM_TARGET_ROOT103_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B390	Miscellaneous Register (CCM_MISC103)	32	R/W	0000_0000h	5.1.7.14/564
3038_B394	Miscellaneous Register (CCM_MISC_ROOT103_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B398	Miscellaneous Register (CCM_MISC_ROOT103_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B39C	Miscellaneous Register (CCM_MISC_ROOT103_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B3A0	Post Divider Register (CCM_POST103)	32	R/W	0000_0000h	5.1.7.18/568
3038_B3A4	Post Divider Register (CCM_POST_ROOT103_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B3A8	Post Divider Register (CCM_POST_ROOT103_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B3AC	Post Divider Register (CCM_POST_ROOT103_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B3B0	Pre Divider Register (CCM_PRE103)	32	R/W	1000_0000h	5.1.7.22/580
3038_B3B4	Pre Divider Register (CCM_PRE_ROOT103_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B3B8	Pre Divider Register (CCM_PRE_ROOT103_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B3BC	Pre Divider Register (CCM_PRE_ROOT103_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B3F0	Access Control Register (CCM_ACCESS_CTRL103)	32	R/W	0000_0000h	5.1.7.26/592
3038_B3F4	Access Control Register (CCM_ACCESS_CTRL_ROOT103_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B3F8	Access Control Register (CCM_ACCESS_CTRL_ROOT103_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B3FC	Access Control Register (CCM_ACCESS_CTRL_ROOT103_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B400	Target Register (CCM_TARGET_ROOT104)	32	R/W	1000_0000h	5.1.7.10/556
3038_B404	Target Register (CCM_TARGET_ROOT104_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B408	Target Register (CCM_TARGET_ROOT104_CLR)	32	R/W	0000_0000h	5.1.7.12/560

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B40C	Target Register (CCM_TARGET_ROOT104_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B410	Miscellaneous Register (CCM_MISC104)	32	R/W	0000_0000h	5.1.7.14/564
3038_B414	Miscellaneous Register (CCM_MISC_ROOT104_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B418	Miscellaneous Register (CCM_MISC_ROOT104_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B41C	Miscellaneous Register (CCM_MISC_ROOT104_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B420	Post Divider Register (CCM_POST104)	32	R/W	0000_0000h	5.1.7.18/568
3038_B424	Post Divider Register (CCM_POST_ROOT104_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B428	Post Divider Register (CCM_POST_ROOT104_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B42C	Post Divider Register (CCM_POST_ROOT104_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B430	Pre Divider Register (CCM_PRE104)	32	R/W	1000_0000h	5.1.7.22/580
3038_B434	Pre Divider Register (CCM_PRE_ROOT104_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B438	Pre Divider Register (CCM_PRE_ROOT104_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B43C	Pre Divider Register (CCM_PRE_ROOT104_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B470	Access Control Register (CCM_ACCESS_CTRL104)	32	R/W	0000_0000h	5.1.7.26/592
3038_B474	Access Control Register (CCM_ACCESS_CTRL_ROOT104_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B478	Access Control Register (CCM_ACCESS_CTRL_ROOT104_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B47C	Access Control Register (CCM_ACCESS_CTRL_ROOT104_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B480	Target Register (CCM_TARGET_ROOT105)	32	R/W	1000_0000h	5.1.7.10/556
3038_B484	Target Register (CCM_TARGET_ROOT105_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B488	Target Register (CCM_TARGET_ROOT105_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B48C	Target Register (CCM_TARGET_ROOT105_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B490	Miscellaneous Register (CCM_MISC105)	32	R/W	0000_0000h	5.1.7.14/564

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B494	Miscellaneous Register (CCM_MISC_ROOT105_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B498	Miscellaneous Register (CCM_MISC_ROOT105_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B49C	Miscellaneous Register (CCM_MISC_ROOT105_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B4A0	Post Divider Register (CCM_POST105)	32	R/W	0000_0000h	5.1.7.18/568
3038_B4A4	Post Divider Register (CCM_POST_ROOT105_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B4A8	Post Divider Register (CCM_POST_ROOT105_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B4AC	Post Divider Register (CCM_POST_ROOT105_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B4B0	Pre Divider Register (CCM_PRE105)	32	R/W	1000_0000h	5.1.7.22/580
3038_B4B4	Pre Divider Register (CCM_PRE_ROOT105_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B4B8	Pre Divider Register (CCM_PRE_ROOT105_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B4BC	Pre Divider Register (CCM_PRE_ROOT105_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B4F0	Access Control Register (CCM_ACCESS_CTRL105)	32	R/W	0000_0000h	5.1.7.26/592
3038_B4F4	Access Control Register (CCM_ACCESS_CTRL_ROOT105_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B4F8	Access Control Register (CCM_ACCESS_CTRL_ROOT105_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B4FC	Access Control Register (CCM_ACCESS_CTRL_ROOT105_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B500	Target Register (CCM_TARGET_ROOT106)	32	R/W	1000_0000h	5.1.7.10/556
3038_B504	Target Register (CCM_TARGET_ROOT106_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B508	Target Register (CCM_TARGET_ROOT106_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B50C	Target Register (CCM_TARGET_ROOT106_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B510	Miscellaneous Register (CCM_MISC106)	32	R/W	0000_0000h	5.1.7.14/564
3038_B514	Miscellaneous Register (CCM_MISC_ROOT106_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B518	Miscellaneous Register (CCM_MISC_ROOT106_CLR)	32	R/W	0000_0000h	5.1.7.16/566

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B51C	Miscellaneous Register (CCM_MISC_ROOT106_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B520	Post Divider Register (CCM_POST106)	32	R/W	0000_0000h	5.1.7.18/568
3038_B524	Post Divider Register (CCM_POST_ROOT106_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B528	Post Divider Register (CCM_POST_ROOT106_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B52C	Post Divider Register (CCM_POST_ROOT106_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B530	Pre Divider Register (CCM_PRE106)	32	R/W	1000_0000h	5.1.7.22/580
3038_B534	Pre Divider Register (CCM_PRE_ROOT106_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B538	Pre Divider Register (CCM_PRE_ROOT106_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B53C	Pre Divider Register (CCM_PRE_ROOT106_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B570	Access Control Register (CCM_ACCESS_CTRL106)	32	R/W	0000_0000h	5.1.7.26/592
3038_B574	Access Control Register (CCM_ACCESS_CTRL_ROOT106_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B578	Access Control Register (CCM_ACCESS_CTRL_ROOT106_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B57C	Access Control Register (CCM_ACCESS_CTRL_ROOT106_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B580	Target Register (CCM_TARGET_ROOT107)	32	R/W	1000_0000h	5.1.7.10/556
3038_B584	Target Register (CCM_TARGET_ROOT107_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B588	Target Register (CCM_TARGET_ROOT107_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B58C	Target Register (CCM_TARGET_ROOT107_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B590	Miscellaneous Register (CCM_MISC107)	32	R/W	0000_0000h	5.1.7.14/564
3038_B594	Miscellaneous Register (CCM_MISC_ROOT107_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B598	Miscellaneous Register (CCM_MISC_ROOT107_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B59C	Miscellaneous Register (CCM_MISC_ROOT107_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B5A0	Post Divider Register (CCM_POST107)	32	R/W	0000_0000h	5.1.7.18/568

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B5A4	Post Divider Register (CCM_POST_ROOT107_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B5A8	Post Divider Register (CCM_POST_ROOT107_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B5AC	Post Divider Register (CCM_POST_ROOT107_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B5B0	Pre Divider Register (CCM_PRE107)	32	R/W	1000_0000h	5.1.7.22/580
3038_B5B4	Pre Divider Register (CCM_PRE_ROOT107_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B5B8	Pre Divider Register (CCM_PRE_ROOT107_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B5BC	Pre Divider Register (CCM_PRE_ROOT107_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B5F0	Access Control Register (CCM_ACCESS_CTRL107)	32	R/W	0000_0000h	5.1.7.26/592
3038_B5F4	Access Control Register (CCM_ACCESS_CTRL_ROOT107_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B5F8	Access Control Register (CCM_ACCESS_CTRL_ROOT107_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B5FC	Access Control Register (CCM_ACCESS_CTRL_ROOT107_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B600	Target Register (CCM_TARGET_ROOT108)	32	R/W	1000_0000h	5.1.7.10/556
3038_B604	Target Register (CCM_TARGET_ROOT108_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B608	Target Register (CCM_TARGET_ROOT108_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B60C	Target Register (CCM_TARGET_ROOT108_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B610	Miscellaneous Register (CCM_MISC108)	32	R/W	0000_0000h	5.1.7.14/564
3038_B614	Miscellaneous Register (CCM_MISC_ROOT108_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B618	Miscellaneous Register (CCM_MISC_ROOT108_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B61C	Miscellaneous Register (CCM_MISC_ROOT108_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B620	Post Divider Register (CCM_POST108)	32	R/W	0000_0000h	5.1.7.18/568
3038_B624	Post Divider Register (CCM_POST_ROOT108_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B628	Post Divider Register (CCM_POST_ROOT108_CLR)	32	R/W	0000_0000h	5.1.7.20/574

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B62C	Post Divider Register (CCM_POST_ROOT108_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B630	Pre Divider Register (CCM_PRE108)	32	R/W	1000_0000h	5.1.7.22/580
3038_B634	Pre Divider Register (CCM_PRE_ROOT108_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B638	Pre Divider Register (CCM_PRE_ROOT108_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B63C	Pre Divider Register (CCM_PRE_ROOT108_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B670	Access Control Register (CCM_ACCESS_CTRL108)	32	R/W	0000_0000h	5.1.7.26/592
3038_B674	Access Control Register (CCM_ACCESS_CTRL_ROOT108_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B678	Access Control Register (CCM_ACCESS_CTRL_ROOT108_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B67C	Access Control Register (CCM_ACCESS_CTRL_ROOT108_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B680	Target Register (CCM_TARGET_ROOT109)	32	R/W	1000_0000h	5.1.7.10/556
3038_B684	Target Register (CCM_TARGET_ROOT109_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B688	Target Register (CCM_TARGET_ROOT109_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B68C	Target Register (CCM_TARGET_ROOT109_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B690	Miscellaneous Register (CCM_MISC109)	32	R/W	0000_0000h	5.1.7.14/564
3038_B694	Miscellaneous Register (CCM_MISC_ROOT109_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B698	Miscellaneous Register (CCM_MISC_ROOT109_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B69C	Miscellaneous Register (CCM_MISC_ROOT109_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B6A0	Post Divider Register (CCM_POST109)	32	R/W	0000_0000h	5.1.7.18/568
3038_B6A4	Post Divider Register (CCM_POST_ROOT109_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B6A8	Post Divider Register (CCM_POST_ROOT109_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B6AC	Post Divider Register (CCM_POST_ROOT109_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B6B0	Pre Divider Register (CCM_PRE109)	32	R/W	1000_0000h	5.1.7.22/580

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B6B4	Pre Divider Register (CCM_PRE_ROOT109_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B6B8	Pre Divider Register (CCM_PRE_ROOT109_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B6BC	Pre Divider Register (CCM_PRE_ROOT109_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B6F0	Access Control Register (CCM_ACCESS_CTRL109)	32	R/W	0000_0000h	5.1.7.26/592
3038_B6F4	Access Control Register (CCM_ACCESS_CTRL_ROOT109_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B6F8	Access Control Register (CCM_ACCESS_CTRL_ROOT109_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B6FC	Access Control Register (CCM_ACCESS_CTRL_ROOT109_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B700	Target Register (CCM_TARGET_ROOT110)	32	R/W	1000_0000h	5.1.7.10/556
3038_B704	Target Register (CCM_TARGET_ROOT110_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B708	Target Register (CCM_TARGET_ROOT110_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B70C	Target Register (CCM_TARGET_ROOT110_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B710	Miscellaneous Register (CCM_MISC110)	32	R/W	0000_0000h	5.1.7.14/564
3038_B714	Miscellaneous Register (CCM_MISC_ROOT110_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B718	Miscellaneous Register (CCM_MISC_ROOT110_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B71C	Miscellaneous Register (CCM_MISC_ROOT110_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B720	Post Divider Register (CCM_POST110)	32	R/W	0000_0000h	5.1.7.18/568
3038_B724	Post Divider Register (CCM_POST_ROOT110_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B728	Post Divider Register (CCM_POST_ROOT110_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B72C	Post Divider Register (CCM_POST_ROOT110_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B730	Pre Divider Register (CCM_PRE110)	32	R/W	1000_0000h	5.1.7.22/580
3038_B734	Pre Divider Register (CCM_PRE_ROOT110_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B738	Pre Divider Register (CCM_PRE_ROOT110_CLR)	32	R/W	0000_0000h	5.1.7.24/586

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B73C	Pre Divider Register (CCM_PRE_ROOT110_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B770	Access Control Register (CCM_ACCESS_CTRL110)	32	R/W	0000_0000h	5.1.7.26/592
3038_B774	Access Control Register (CCM_ACCESS_CTRL_ROOT110_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B778	Access Control Register (CCM_ACCESS_CTRL_ROOT110_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B77C	Access Control Register (CCM_ACCESS_CTRL_ROOT110_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B780	Target Register (CCM_TARGET_ROOT111)	32	R/W	1000_0000h	5.1.7.10/556
3038_B784	Target Register (CCM_TARGET_ROOT111_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B788	Target Register (CCM_TARGET_ROOT111_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B78C	Target Register (CCM_TARGET_ROOT111_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B790	Miscellaneous Register (CCM_MISC111)	32	R/W	0000_0000h	5.1.7.14/564
3038_B794	Miscellaneous Register (CCM_MISC_ROOT111_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B798	Miscellaneous Register (CCM_MISC_ROOT111_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B79C	Miscellaneous Register (CCM_MISC_ROOT111_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B7A0	Post Divider Register (CCM_POST111)	32	R/W	0000_0000h	5.1.7.18/568
3038_B7A4	Post Divider Register (CCM_POST_ROOT111_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B7A8	Post Divider Register (CCM_POST_ROOT111_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B7AC	Post Divider Register (CCM_POST_ROOT111_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B7B0	Pre Divider Register (CCM_PRE111)	32	R/W	1000_0000h	5.1.7.22/580
3038_B7B4	Pre Divider Register (CCM_PRE_ROOT111_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B7B8	Pre Divider Register (CCM_PRE_ROOT111_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B7BC	Pre Divider Register (CCM_PRE_ROOT111_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B7F0	Access Control Register (CCM_ACCESS_CTRL111)	32	R/W	0000_0000h	5.1.7.26/592

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B7F4	Access Control Register (CCM_ACCESS_CTRL_ROOT111_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B7F8	Access Control Register (CCM_ACCESS_CTRL_ROOT111_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B7FC	Access Control Register (CCM_ACCESS_CTRL_ROOT111_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B800	Target Register (CCM_TARGET_ROOT112)	32	R/W	1000_0000h	5.1.7.10/556
3038_B804	Target Register (CCM_TARGET_ROOT112_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B808	Target Register (CCM_TARGET_ROOT112_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B80C	Target Register (CCM_TARGET_ROOT112_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B810	Miscellaneous Register (CCM_MISC112)	32	R/W	0000_0000h	5.1.7.14/564
3038_B814	Miscellaneous Register (CCM_MISC_ROOT112_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B818	Miscellaneous Register (CCM_MISC_ROOT112_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B81C	Miscellaneous Register (CCM_MISC_ROOT112_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B820	Post Divider Register (CCM_POST112)	32	R/W	0000_0000h	5.1.7.18/568
3038_B824	Post Divider Register (CCM_POST_ROOT112_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B828	Post Divider Register (CCM_POST_ROOT112_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B82C	Post Divider Register (CCM_POST_ROOT112_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B830	Pre Divider Register (CCM_PRE112)	32	R/W	1000_0000h	5.1.7.22/580
3038_B834	Pre Divider Register (CCM_PRE_ROOT112_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B838	Pre Divider Register (CCM_PRE_ROOT112_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B83C	Pre Divider Register (CCM_PRE_ROOT112_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B870	Access Control Register (CCM_ACCESS_CTRL112)	32	R/W	0000_0000h	5.1.7.26/592
3038_B874	Access Control Register (CCM_ACCESS_CTRL_ROOT112_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B878	Access Control Register (CCM_ACCESS_CTRL_ROOT112_CLR)	32	R/W	0000_0000h	5.1.7.28/597

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B87C	Access Control Register (CCM_ACCESS_CTRL_ROOT112_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B880	Target Register (CCM_TARGET_ROOT113)	32	R/W	1000_0000h	5.1.7.10/556
3038_B884	Target Register (CCM_TARGET_ROOT113_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B888	Target Register (CCM_TARGET_ROOT113_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B88C	Target Register (CCM_TARGET_ROOT113_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B890	Miscellaneous Register (CCM_MISC113)	32	R/W	0000_0000h	5.1.7.14/564
3038_B894	Miscellaneous Register (CCM_MISC_ROOT113_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B898	Miscellaneous Register (CCM_MISC_ROOT113_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B89C	Miscellaneous Register (CCM_MISC_ROOT113_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B8A0	Post Divider Register (CCM_POST113)	32	R/W	0000_0000h	5.1.7.18/568
3038_B8A4	Post Divider Register (CCM_POST_ROOT113_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B8A8	Post Divider Register (CCM_POST_ROOT113_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B8AC	Post Divider Register (CCM_POST_ROOT113_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B8B0	Pre Divider Register (CCM_PRE113)	32	R/W	1000_0000h	5.1.7.22/580
3038_B8B4	Pre Divider Register (CCM_PRE_ROOT113_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B8B8	Pre Divider Register (CCM_PRE_ROOT113_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B8BC	Pre Divider Register (CCM_PRE_ROOT113_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B8F0	Access Control Register (CCM_ACCESS_CTRL113)	32	R/W	0000_0000h	5.1.7.26/592
3038_B8F4	Access Control Register (CCM_ACCESS_CTRL_ROOT113_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B8F8	Access Control Register (CCM_ACCESS_CTRL_ROOT113_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B8FC	Access Control Register (CCM_ACCESS_CTRL_ROOT113_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B900	Target Register (CCM_TARGET_ROOT114)	32	R/W	1000_0000h	5.1.7.10/556

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B904	Target Register (CCM_TARGET_ROOT114_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B908	Target Register (CCM_TARGET_ROOT114_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_B90C	Target Register (CCM_TARGET_ROOT114_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B910	Miscellaneous Register (CCM_MISC114)	32	R/W	0000_0000h	5.1.7.14/564
3038_B914	Miscellaneous Register (CCM_MISC_ROOT114_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B918	Miscellaneous Register (CCM_MISC_ROOT114_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B91C	Miscellaneous Register (CCM_MISC_ROOT114_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B920	Post Divider Register (CCM_POST114)	32	R/W	0000_0000h	5.1.7.18/568
3038_B924	Post Divider Register (CCM_POST_ROOT114_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B928	Post Divider Register (CCM_POST_ROOT114_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B92C	Post Divider Register (CCM_POST_ROOT114_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B930	Pre Divider Register (CCM_PRE114)	32	R/W	1000_0000h	5.1.7.22/580
3038_B934	Pre Divider Register (CCM_PRE_ROOT114_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B938	Pre Divider Register (CCM_PRE_ROOT114_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B93C	Pre Divider Register (CCM_PRE_ROOT114_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B970	Access Control Register (CCM_ACCESS_CTRL114)	32	R/W	0000_0000h	5.1.7.26/592
3038_B974	Access Control Register (CCM_ACCESS_CTRL_ROOT114_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B978	Access Control Register (CCM_ACCESS_CTRL_ROOT114_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B97C	Access Control Register (CCM_ACCESS_CTRL_ROOT114_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_B980	Target Register (CCM_TARGET_ROOT115)	32	R/W	1000_0000h	5.1.7.10/556
3038_B984	Target Register (CCM_TARGET_ROOT115_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_B988	Target Register (CCM_TARGET_ROOT115_CLR)	32	R/W	0000_0000h	5.1.7.12/560

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_B98C	Target Register (CCM_TARGET_ROOT115_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_B990	Miscellaneous Register (CCM_MISC115)	32	R/W	0000_0000h	5.1.7.14/564
3038_B994	Miscellaneous Register (CCM_MISC_ROOT115_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_B998	Miscellaneous Register (CCM_MISC_ROOT115_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_B99C	Miscellaneous Register (CCM_MISC_ROOT115_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_B9A0	Post Divider Register (CCM_POST115)	32	R/W	0000_0000h	5.1.7.18/568
3038_B9A4	Post Divider Register (CCM_POST_ROOT115_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_B9A8	Post Divider Register (CCM_POST_ROOT115_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_B9AC	Post Divider Register (CCM_POST_ROOT115_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_B9B0	Pre Divider Register (CCM_PRE115)	32	R/W	1000_0000h	5.1.7.22/580
3038_B9B4	Pre Divider Register (CCM_PRE_ROOT115_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_B9B8	Pre Divider Register (CCM_PRE_ROOT115_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_B9BC	Pre Divider Register (CCM_PRE_ROOT115_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_B9F0	Access Control Register (CCM_ACCESS_CTRL115)	32	R/W	0000_0000h	5.1.7.26/592
3038_B9F4	Access Control Register (CCM_ACCESS_CTRL_ROOT115_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_B9F8	Access Control Register (CCM_ACCESS_CTRL_ROOT115_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_B9FC	Access Control Register (CCM_ACCESS_CTRL_ROOT115_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_BA00	Target Register (CCM_TARGET_ROOT116)	32	R/W	1000_0000h	5.1.7.10/556
3038_BA04	Target Register (CCM_TARGET_ROOT116_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_BA08	Target Register (CCM_TARGET_ROOT116_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_BA0C	Target Register (CCM_TARGET_ROOT116_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_BA10	Miscellaneous Register (CCM_MISC116)	32	R/W	0000_0000h	5.1.7.14/564

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_BA14	Miscellaneous Register (CCM_MISC_ROOT116_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_BA18	Miscellaneous Register (CCM_MISC_ROOT116_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_BA1C	Miscellaneous Register (CCM_MISC_ROOT116_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_BA20	Post Divider Register (CCM_POST116)	32	R/W	0000_0000h	5.1.7.18/568
3038_BA24	Post Divider Register (CCM_POST_ROOT116_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_BA28	Post Divider Register (CCM_POST_ROOT116_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_BA2C	Post Divider Register (CCM_POST_ROOT116_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_BA30	Pre Divider Register (CCM_PRE116)	32	R/W	1000_0000h	5.1.7.22/580
3038_BA34	Pre Divider Register (CCM_PRE_ROOT116_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_BA38	Pre Divider Register (CCM_PRE_ROOT116_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_BA3C	Pre Divider Register (CCM_PRE_ROOT116_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_BA70	Access Control Register (CCM_ACCESS_CTRL116)	32	R/W	0000_0000h	5.1.7.26/592
3038_BA74	Access Control Register (CCM_ACCESS_CTRL_ROOT116_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_BA78	Access Control Register (CCM_ACCESS_CTRL_ROOT116_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_BA7C	Access Control Register (CCM_ACCESS_CTRL_ROOT116_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_BA80	Target Register (CCM_TARGET_ROOT117)	32	R/W	1000_0000h	5.1.7.10/556
3038_BA84	Target Register (CCM_TARGET_ROOT117_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_BA88	Target Register (CCM_TARGET_ROOT117_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_BA8C	Target Register (CCM_TARGET_ROOT117_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_BA90	Miscellaneous Register (CCM_MISC117)	32	R/W	0000_0000h	5.1.7.14/564
3038_BA94	Miscellaneous Register (CCM_MISC_ROOT117_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_BA98	Miscellaneous Register (CCM_MISC_ROOT117_CLR)	32	R/W	0000_0000h	5.1.7.16/566

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_BA9C	Miscellaneous Register (CCM_MISC_ROOT117_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_BAA0	Post Divider Register (CCM_POST117)	32	R/W	0000_0000h	5.1.7.18/568
3038_BAA4	Post Divider Register (CCM_POST_ROOT117_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_BAA8	Post Divider Register (CCM_POST_ROOT117_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_BAAC	Post Divider Register (CCM_POST_ROOT117_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_BAB0	Pre Divider Register (CCM_PRE117)	32	R/W	1000_0000h	5.1.7.22/580
3038_BAB4	Pre Divider Register (CCM_PRE_ROOT117_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_BAB8	Pre Divider Register (CCM_PRE_ROOT117_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_BABC	Pre Divider Register (CCM_PRE_ROOT117_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_BAF0	Access Control Register (CCM_ACCESS_CTRL117)	32	R/W	0000_0000h	5.1.7.26/592
3038_BAF4	Access Control Register (CCM_ACCESS_CTRL_ROOT117_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_BAF8	Access Control Register (CCM_ACCESS_CTRL_ROOT117_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_BAFC	Access Control Register (CCM_ACCESS_CTRL_ROOT117_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_BB00	Target Register (CCM_TARGET_ROOT118)	32	R/W	1000_0000h	5.1.7.10/556
3038_BB04	Target Register (CCM_TARGET_ROOT118_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_BB08	Target Register (CCM_TARGET_ROOT118_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_BB0C	Target Register (CCM_TARGET_ROOT118_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_BB10	Miscellaneous Register (CCM_MISC118)	32	R/W	0000_0000h	5.1.7.14/564
3038_BB14	Miscellaneous Register (CCM_MISC_ROOT118_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_BB18	Miscellaneous Register (CCM_MISC_ROOT118_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_BB1C	Miscellaneous Register (CCM_MISC_ROOT118_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_BB20	Post Divider Register (CCM_POST118)	32	R/W	0000_0000h	5.1.7.18/568

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_BB24	Post Divider Register (CCM_POST_ROOT118_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_BB28	Post Divider Register (CCM_POST_ROOT118_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_BB2C	Post Divider Register (CCM_POST_ROOT118_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_BB30	Pre Divider Register (CCM_PRE118)	32	R/W	1000_0000h	5.1.7.22/580
3038_BB34	Pre Divider Register (CCM_PRE_ROOT118_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_BB38	Pre Divider Register (CCM_PRE_ROOT118_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_BB3C	Pre Divider Register (CCM_PRE_ROOT118_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_BB70	Access Control Register (CCM_ACCESS_CTRL118)	32	R/W	0000_0000h	5.1.7.26/592
3038_BB74	Access Control Register (CCM_ACCESS_CTRL_ROOT118_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_BB78	Access Control Register (CCM_ACCESS_CTRL_ROOT118_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_BB7C	Access Control Register (CCM_ACCESS_CTRL_ROOT118_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_BB80	Target Register (CCM_TARGET_ROOT119)	32	R/W	1000_0000h	5.1.7.10/556
3038_BB84	Target Register (CCM_TARGET_ROOT119_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_BB88	Target Register (CCM_TARGET_ROOT119_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_BB8C	Target Register (CCM_TARGET_ROOT119_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_BB90	Miscellaneous Register (CCM_MISC119)	32	R/W	0000_0000h	5.1.7.14/564
3038_BB94	Miscellaneous Register (CCM_MISC_ROOT119_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_BB98	Miscellaneous Register (CCM_MISC_ROOT119_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_BB9C	Miscellaneous Register (CCM_MISC_ROOT119_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_BBA0	Post Divider Register (CCM_POST119)	32	R/W	0000_0000h	5.1.7.18/568
3038_BBA4	Post Divider Register (CCM_POST_ROOT119_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_BBA8	Post Divider Register (CCM_POST_ROOT119_CLR)	32	R/W	0000_0000h	5.1.7.20/574

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_BBAC	Post Divider Register (CCM_POST_ROOT119_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_BBB0	Pre Divider Register (CCM_PRE119)	32	R/W	1000_0000h	5.1.7.22/580
3038_BBB4	Pre Divider Register (CCM_PRE_ROOT119_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_BBB8	Pre Divider Register (CCM_PRE_ROOT119_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_BBBC	Pre Divider Register (CCM_PRE_ROOT119_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_BBF0	Access Control Register (CCM_ACCESS_CTRL119)	32	R/W	0000_0000h	5.1.7.26/592
3038_BBF4	Access Control Register (CCM_ACCESS_CTRL_ROOT119_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_BBF8	Access Control Register (CCM_ACCESS_CTRL_ROOT119_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_BBFC	Access Control Register (CCM_ACCESS_CTRL_ROOT119_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_BC00	Target Register (CCM_TARGET_ROOT120)	32	R/W	1000_0000h	5.1.7.10/556
3038_BC04	Target Register (CCM_TARGET_ROOT120_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_BC08	Target Register (CCM_TARGET_ROOT120_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_BC0C	Target Register (CCM_TARGET_ROOT120_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_BC10	Miscellaneous Register (CCM_MISC120)	32	R/W	0000_0000h	5.1.7.14/564
3038_BC14	Miscellaneous Register (CCM_MISC_ROOT120_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_BC18	Miscellaneous Register (CCM_MISC_ROOT120_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_BC1C	Miscellaneous Register (CCM_MISC_ROOT120_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_BC20	Post Divider Register (CCM_POST120)	32	R/W	0000_0000h	5.1.7.18/568
3038_BC24	Post Divider Register (CCM_POST_ROOT120_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_BC28	Post Divider Register (CCM_POST_ROOT120_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_BC2C	Post Divider Register (CCM_POST_ROOT120_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_BC30	Pre Divider Register (CCM_PRE120)	32	R/W	1000_0000h	5.1.7.22/580

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_BC34	Pre Divider Register (CCM_PRE_ROOT120_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_BC38	Pre Divider Register (CCM_PRE_ROOT120_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_BC3C	Pre Divider Register (CCM_PRE_ROOT120_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_BC70	Access Control Register (CCM_ACCESS_CTRL120)	32	R/W	0000_0000h	5.1.7.26/592
3038_BC74	Access Control Register (CCM_ACCESS_CTRL_ROOT120_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_BC78	Access Control Register (CCM_ACCESS_CTRL_ROOT120_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_BC7C	Access Control Register (CCM_ACCESS_CTRL_ROOT120_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_BC80	Target Register (CCM_TARGET_ROOT121)	32	R/W	1000_0000h	5.1.7.10/556
3038_BC84	Target Register (CCM_TARGET_ROOT121_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_BC88	Target Register (CCM_TARGET_ROOT121_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_BC8C	Target Register (CCM_TARGET_ROOT121_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_BC90	Miscellaneous Register (CCM_MISC121)	32	R/W	0000_0000h	5.1.7.14/564
3038_BC94	Miscellaneous Register (CCM_MISC_ROOT121_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_BC98	Miscellaneous Register (CCM_MISC_ROOT121_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_BC9C	Miscellaneous Register (CCM_MISC_ROOT121_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_BCA0	Post Divider Register (CCM_POST121)	32	R/W	0000_0000h	5.1.7.18/568
3038_BCA4	Post Divider Register (CCM_POST_ROOT121_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_BCA8	Post Divider Register (CCM_POST_ROOT121_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_BCAC	Post Divider Register (CCM_POST_ROOT121_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_BCB0	Pre Divider Register (CCM_PRE121)	32	R/W	1000_0000h	5.1.7.22/580
3038_BCB4	Pre Divider Register (CCM_PRE_ROOT121_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_BCB8	Pre Divider Register (CCM_PRE_ROOT121_CLR)	32	R/W	0000_0000h	5.1.7.24/586

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_BCBC	Pre Divider Register (CCM_PRE_ROOT121_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_BCF0	Access Control Register (CCM_ACCESS_CTRL121)	32	R/W	0000_0000h	5.1.7.26/592
3038_BCF4	Access Control Register (CCM_ACCESS_CTRL_ROOT121_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_BCF8	Access Control Register (CCM_ACCESS_CTRL_ROOT121_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_BCFC	Access Control Register (CCM_ACCESS_CTRL_ROOT121_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_BD00	Target Register (CCM_TARGET_ROOT122)	32	R/W	1000_0000h	5.1.7.10/556
3038_BD04	Target Register (CCM_TARGET_ROOT122_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_BD08	Target Register (CCM_TARGET_ROOT122_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_BD0C	Target Register (CCM_TARGET_ROOT122_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_BD10	Miscellaneous Register (CCM_MISC122)	32	R/W	0000_0000h	5.1.7.14/564
3038_BD14	Miscellaneous Register (CCM_MISC_ROOT122_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_BD18	Miscellaneous Register (CCM_MISC_ROOT122_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_BD1C	Miscellaneous Register (CCM_MISC_ROOT122_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_BD20	Post Divider Register (CCM_POST122)	32	R/W	0000_0000h	5.1.7.18/568
3038_BD24	Post Divider Register (CCM_POST_ROOT122_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_BD28	Post Divider Register (CCM_POST_ROOT122_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_BD2C	Post Divider Register (CCM_POST_ROOT122_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_BD30	Pre Divider Register (CCM_PRE122)	32	R/W	1000_0000h	5.1.7.22/580
3038_BD34	Pre Divider Register (CCM_PRE_ROOT122_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_BD38	Pre Divider Register (CCM_PRE_ROOT122_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_BD3C	Pre Divider Register (CCM_PRE_ROOT122_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_BD70	Access Control Register (CCM_ACCESS_CTRL122)	32	R/W	0000_0000h	5.1.7.26/592

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_BD74	Access Control Register (CCM_ACCESS_CTRL_ROOT122_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_BD78	Access Control Register (CCM_ACCESS_CTRL_ROOT122_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_BD7C	Access Control Register (CCM_ACCESS_CTRL_ROOT122_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_BD80	Target Register (CCM_TARGET_ROOT123)	32	R/W	1000_0000h	5.1.7.10/556
3038_BD84	Target Register (CCM_TARGET_ROOT123_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_BD88	Target Register (CCM_TARGET_ROOT123_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_BD8C	Target Register (CCM_TARGET_ROOT123_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_BD90	Miscellaneous Register (CCM_MISC123)	32	R/W	0000_0000h	5.1.7.14/564
3038_BD94	Miscellaneous Register (CCM_MISC_ROOT123_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_BD98	Miscellaneous Register (CCM_MISC_ROOT123_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_BD9C	Miscellaneous Register (CCM_MISC_ROOT123_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_BDA0	Post Divider Register (CCM_POST123)	32	R/W	0000_0000h	5.1.7.18/568
3038_BDA4	Post Divider Register (CCM_POST_ROOT123_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_BDA8	Post Divider Register (CCM_POST_ROOT123_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_BDAC	Post Divider Register (CCM_POST_ROOT123_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_BDB0	Pre Divider Register (CCM_PRE123)	32	R/W	1000_0000h	5.1.7.22/580
3038_BDB4	Pre Divider Register (CCM_PRE_ROOT123_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_BDB8	Pre Divider Register (CCM_PRE_ROOT123_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_BDBC	Pre Divider Register (CCM_PRE_ROOT123_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_BDF0	Access Control Register (CCM_ACCESS_CTRL123)	32	R/W	0000_0000h	5.1.7.26/592
3038_BDF4	Access Control Register (CCM_ACCESS_CTRL_ROOT123_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_BDF8	Access Control Register (CCM_ACCESS_CTRL_ROOT123_CLR)	32	R/W	0000_0000h	5.1.7.28/597

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_BDFC	Access Control Register (CCM_ACCESS_CTRL_ROOT123_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_BE00	Target Register (CCM_TARGET_ROOT124)	32	R/W	1000_0000h	5.1.7.10/556
3038_BE04	Target Register (CCM_TARGET_ROOT124_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_BE08	Target Register (CCM_TARGET_ROOT124_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_BE0C	Target Register (CCM_TARGET_ROOT124_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_BE10	Miscellaneous Register (CCM_MISC124)	32	R/W	0000_0000h	5.1.7.14/564
3038_BE14	Miscellaneous Register (CCM_MISC_ROOT124_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_BE18	Miscellaneous Register (CCM_MISC_ROOT124_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_BE1C	Miscellaneous Register (CCM_MISC_ROOT124_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_BE20	Post Divider Register (CCM_POST124)	32	R/W	0000_0000h	5.1.7.18/568
3038_BE24	Post Divider Register (CCM_POST_ROOT124_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_BE28	Post Divider Register (CCM_POST_ROOT124_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_BE2C	Post Divider Register (CCM_POST_ROOT124_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_BE30	Pre Divider Register (CCM_PRE124)	32	R/W	1000_0000h	5.1.7.22/580
3038_BE34	Pre Divider Register (CCM_PRE_ROOT124_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_BE38	Pre Divider Register (CCM_PRE_ROOT124_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_BE3C	Pre Divider Register (CCM_PRE_ROOT124_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_BE70	Access Control Register (CCM_ACCESS_CTRL124)	32	R/W	0000_0000h	5.1.7.26/592
3038_BE74	Access Control Register (CCM_ACCESS_CTRL_ROOT124_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_BE78	Access Control Register (CCM_ACCESS_CTRL_ROOT124_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_BE7C	Access Control Register (CCM_ACCESS_CTRL_ROOT124_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_BE80	Target Register (CCM_TARGET_ROOT125)	32	R/W	1000_0000h	5.1.7.10/556

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_BE84	Target Register (CCM_TARGET_ROOT125_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_BE88	Target Register (CCM_TARGET_ROOT125_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_BE8C	Target Register (CCM_TARGET_ROOT125_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_BE90	Miscellaneous Register (CCM_MISC125)	32	R/W	0000_0000h	5.1.7.14/564
3038_BE94	Miscellaneous Register (CCM_MISC_ROOT125_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_BE98	Miscellaneous Register (CCM_MISC_ROOT125_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_BE9C	Miscellaneous Register (CCM_MISC_ROOT125_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_BEA0	Post Divider Register (CCM_POST125)	32	R/W	0000_0000h	5.1.7.18/568
3038_BEA4	Post Divider Register (CCM_POST_ROOT125_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_BEA8	Post Divider Register (CCM_POST_ROOT125_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_BEAC	Post Divider Register (CCM_POST_ROOT125_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_BEB0	Pre Divider Register (CCM_PRE125)	32	R/W	1000_0000h	5.1.7.22/580
3038_BEB4	Pre Divider Register (CCM_PRE_ROOT125_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_BEB8	Pre Divider Register (CCM_PRE_ROOT125_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_BEBC	Pre Divider Register (CCM_PRE_ROOT125_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_BEFC	Access Control Register (CCM_ACCESS_CTRL125)	32	R/W	0000_0000h	5.1.7.26/592
3038_BEFC	Access Control Register (CCM_ACCESS_CTRL_ROOT125_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_BEFC	Access Control Register (CCM_ACCESS_CTRL_ROOT125_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_BEFC	Access Control Register (CCM_ACCESS_CTRL_ROOT125_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_BF00	Target Register (CCM_TARGET_ROOT126)	32	R/W	1000_0000h	5.1.7.10/556
3038_BF04	Target Register (CCM_TARGET_ROOT126_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_BF08	Target Register (CCM_TARGET_ROOT126_CLR)	32	R/W	0000_0000h	5.1.7.12/560

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_BF0C	Target Register (CCM_TARGET_ROOT126_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_BF10	Miscellaneous Register (CCM_MISC126)	32	R/W	0000_0000h	5.1.7.14/564
3038_BF14	Miscellaneous Register (CCM_MISC_ROOT126_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_BF18	Miscellaneous Register (CCM_MISC_ROOT126_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_BF1C	Miscellaneous Register (CCM_MISC_ROOT126_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_BF20	Post Divider Register (CCM_POST126)	32	R/W	0000_0000h	5.1.7.18/568
3038_BF24	Post Divider Register (CCM_POST_ROOT126_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_BF28	Post Divider Register (CCM_POST_ROOT126_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_BF2C	Post Divider Register (CCM_POST_ROOT126_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_BF30	Pre Divider Register (CCM_PRE126)	32	R/W	1000_0000h	5.1.7.22/580
3038_BF34	Pre Divider Register (CCM_PRE_ROOT126_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_BF38	Pre Divider Register (CCM_PRE_ROOT126_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_BF3C	Pre Divider Register (CCM_PRE_ROOT126_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_BF70	Access Control Register (CCM_ACCESS_CTRL126)	32	R/W	0000_0000h	5.1.7.26/592
3038_BF74	Access Control Register (CCM_ACCESS_CTRL_ROOT126_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_BF78	Access Control Register (CCM_ACCESS_CTRL_ROOT126_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_BF7C	Access Control Register (CCM_ACCESS_CTRL_ROOT126_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_BF80	Target Register (CCM_TARGET_ROOT127)	32	R/W	1000_0000h	5.1.7.10/556
3038_BF84	Target Register (CCM_TARGET_ROOT127_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_BF88	Target Register (CCM_TARGET_ROOT127_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_BF8C	Target Register (CCM_TARGET_ROOT127_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_BF90	Miscellaneous Register (CCM_MISC127)	32	R/W	0000_0000h	5.1.7.14/564

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_BF94	Miscellaneous Register (CCM_MISC_ROOT127_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_BF98	Miscellaneous Register (CCM_MISC_ROOT127_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_BF9C	Miscellaneous Register (CCM_MISC_ROOT127_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_BFA0	Post Divider Register (CCM_POST127)	32	R/W	0000_0000h	5.1.7.18/568
3038_BFA4	Post Divider Register (CCM_POST_ROOT127_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_BFA8	Post Divider Register (CCM_POST_ROOT127_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_BFAC	Post Divider Register (CCM_POST_ROOT127_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_BFB0	Pre Divider Register (CCM_PRE127)	32	R/W	1000_0000h	5.1.7.22/580
3038_BFB4	Pre Divider Register (CCM_PRE_ROOT127_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_BFB8	Pre Divider Register (CCM_PRE_ROOT127_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_BFBC	Pre Divider Register (CCM_PRE_ROOT127_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_BFF0	Access Control Register (CCM_ACCESS_CTRL127)	32	R/W	0000_0000h	5.1.7.26/592
3038_BFF4	Access Control Register (CCM_ACCESS_CTRL_ROOT127_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_BFF8	Access Control Register (CCM_ACCESS_CTRL_ROOT127_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_BFFC	Access Control Register (CCM_ACCESS_CTRL_ROOT127_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C000	Target Register (CCM_TARGET_ROOT128)	32	R/W	1000_0000h	5.1.7.10/556
3038_C004	Target Register (CCM_TARGET_ROOT128_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C008	Target Register (CCM_TARGET_ROOT128_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_C00C	Target Register (CCM_TARGET_ROOT128_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C010	Miscellaneous Register (CCM_MISC128)	32	R/W	0000_0000h	5.1.7.14/564
3038_C014	Miscellaneous Register (CCM_MISC_ROOT128_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C018	Miscellaneous Register (CCM_MISC_ROOT128_CLR)	32	R/W	0000_0000h	5.1.7.16/566

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_C01C	Miscellaneous Register (CCM_MISC_ROOT128_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C020	Post Divider Register (CCM_POST128)	32	R/W	0000_0000h	5.1.7.18/568
3038_C024	Post Divider Register (CCM_POST_ROOT128_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C028	Post Divider Register (CCM_POST_ROOT128_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_C02C	Post Divider Register (CCM_POST_ROOT128_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C030	Pre Divider Register (CCM_PRE128)	32	R/W	1000_0000h	5.1.7.22/580
3038_C034	Pre Divider Register (CCM_PRE_ROOT128_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C038	Pre Divider Register (CCM_PRE_ROOT128_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_C03C	Pre Divider Register (CCM_PRE_ROOT128_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C070	Access Control Register (CCM_ACCESS_CTRL128)	32	R/W	0000_0000h	5.1.7.26/592
3038_C074	Access Control Register (CCM_ACCESS_CTRL_ROOT128_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C078	Access Control Register (CCM_ACCESS_CTRL_ROOT128_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_C07C	Access Control Register (CCM_ACCESS_CTRL_ROOT128_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C080	Target Register (CCM_TARGET_ROOT129)	32	R/W	1000_0000h	5.1.7.10/556
3038_C084	Target Register (CCM_TARGET_ROOT129_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C088	Target Register (CCM_TARGET_ROOT129_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_C08C	Target Register (CCM_TARGET_ROOT129_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C090	Miscellaneous Register (CCM_MISC129)	32	R/W	0000_0000h	5.1.7.14/564
3038_C094	Miscellaneous Register (CCM_MISC_ROOT129_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C098	Miscellaneous Register (CCM_MISC_ROOT129_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_C09C	Miscellaneous Register (CCM_MISC_ROOT129_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C0A0	Post Divider Register (CCM_POST129)	32	R/W	0000_0000h	5.1.7.18/568

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_C0A4	Post Divider Register (CCM_POST_ROOT129_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C0A8	Post Divider Register (CCM_POST_ROOT129_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_C0AC	Post Divider Register (CCM_POST_ROOT129_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C0B0	Pre Divider Register (CCM_PRE129)	32	R/W	1000_0000h	5.1.7.22/580
3038_C0B4	Pre Divider Register (CCM_PRE_ROOT129_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C0B8	Pre Divider Register (CCM_PRE_ROOT129_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_C0BC	Pre Divider Register (CCM_PRE_ROOT129_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C0F0	Access Control Register (CCM_ACCESS_CTRL129)	32	R/W	0000_0000h	5.1.7.26/592
3038_C0F4	Access Control Register (CCM_ACCESS_CTRL_ROOT129_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C0F8	Access Control Register (CCM_ACCESS_CTRL_ROOT129_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_C0FC	Access Control Register (CCM_ACCESS_CTRL_ROOT129_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C100	Target Register (CCM_TARGET_ROOT130)	32	R/W	1000_0000h	5.1.7.10/556
3038_C104	Target Register (CCM_TARGET_ROOT130_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C108	Target Register (CCM_TARGET_ROOT130_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_C10C	Target Register (CCM_TARGET_ROOT130_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C110	Miscellaneous Register (CCM_MISC130)	32	R/W	0000_0000h	5.1.7.14/564
3038_C114	Miscellaneous Register (CCM_MISC_ROOT130_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C118	Miscellaneous Register (CCM_MISC_ROOT130_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_C11C	Miscellaneous Register (CCM_MISC_ROOT130_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C120	Post Divider Register (CCM_POST130)	32	R/W	0000_0000h	5.1.7.18/568
3038_C124	Post Divider Register (CCM_POST_ROOT130_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C128	Post Divider Register (CCM_POST_ROOT130_CLR)	32	R/W	0000_0000h	5.1.7.20/574

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_C12C	Post Divider Register (CCM_POST_ROOT130_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C130	Pre Divider Register (CCM_PRE130)	32	R/W	1000_0000h	5.1.7.22/580
3038_C134	Pre Divider Register (CCM_PRE_ROOT130_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C138	Pre Divider Register (CCM_PRE_ROOT130_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_C13C	Pre Divider Register (CCM_PRE_ROOT130_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C170	Access Control Register (CCM_ACCESS_CTRL130)	32	R/W	0000_0000h	5.1.7.26/592
3038_C174	Access Control Register (CCM_ACCESS_CTRL_ROOT130_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C178	Access Control Register (CCM_ACCESS_CTRL_ROOT130_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_C17C	Access Control Register (CCM_ACCESS_CTRL_ROOT130_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C180	Target Register (CCM_TARGET_ROOT131)	32	R/W	1000_0000h	5.1.7.10/556
3038_C184	Target Register (CCM_TARGET_ROOT131_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C188	Target Register (CCM_TARGET_ROOT131_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_C18C	Target Register (CCM_TARGET_ROOT131_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C190	Miscellaneous Register (CCM_MISC131)	32	R/W	0000_0000h	5.1.7.14/564
3038_C194	Miscellaneous Register (CCM_MISC_ROOT131_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C198	Miscellaneous Register (CCM_MISC_ROOT131_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_C19C	Miscellaneous Register (CCM_MISC_ROOT131_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C1A0	Post Divider Register (CCM_POST131)	32	R/W	0000_0000h	5.1.7.18/568
3038_C1A4	Post Divider Register (CCM_POST_ROOT131_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C1A8	Post Divider Register (CCM_POST_ROOT131_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_C1AC	Post Divider Register (CCM_POST_ROOT131_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C1B0	Pre Divider Register (CCM_PRE131)	32	R/W	1000_0000h	5.1.7.22/580

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_C1B4	Pre Divider Register (CCM_PRE_ROOT131_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C1B8	Pre Divider Register (CCM_PRE_ROOT131_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_C1BC	Pre Divider Register (CCM_PRE_ROOT131_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C1F0	Access Control Register (CCM_ACCESS_CTRL131)	32	R/W	0000_0000h	5.1.7.26/592
3038_C1F4	Access Control Register (CCM_ACCESS_CTRL_ROOT131_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C1F8	Access Control Register (CCM_ACCESS_CTRL_ROOT131_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_C1FC	Access Control Register (CCM_ACCESS_CTRL_ROOT131_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C200	Target Register (CCM_TARGET_ROOT132)	32	R/W	1000_0000h	5.1.7.10/556
3038_C204	Target Register (CCM_TARGET_ROOT132_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C208	Target Register (CCM_TARGET_ROOT132_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_C20C	Target Register (CCM_TARGET_ROOT132_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C210	Miscellaneous Register (CCM_MISC132)	32	R/W	0000_0000h	5.1.7.14/564
3038_C214	Miscellaneous Register (CCM_MISC_ROOT132_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C218	Miscellaneous Register (CCM_MISC_ROOT132_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_C21C	Miscellaneous Register (CCM_MISC_ROOT132_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C220	Post Divider Register (CCM_POST132)	32	R/W	0000_0000h	5.1.7.18/568
3038_C224	Post Divider Register (CCM_POST_ROOT132_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C228	Post Divider Register (CCM_POST_ROOT132_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_C22C	Post Divider Register (CCM_POST_ROOT132_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C230	Pre Divider Register (CCM_PRE132)	32	R/W	1000_0000h	5.1.7.22/580
3038_C234	Pre Divider Register (CCM_PRE_ROOT132_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C238	Pre Divider Register (CCM_PRE_ROOT132_CLR)	32	R/W	0000_0000h	5.1.7.24/586

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_C23C	Pre Divider Register (CCM_PRE_ROOT132_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C270	Access Control Register (CCM_ACCESS_CTRL132)	32	R/W	0000_0000h	5.1.7.26/592
3038_C274	Access Control Register (CCM_ACCESS_CTRL_ROOT132_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C278	Access Control Register (CCM_ACCESS_CTRL_ROOT132_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_C27C	Access Control Register (CCM_ACCESS_CTRL_ROOT132_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C280	Target Register (CCM_TARGET_ROOT133)	32	R/W	1000_0000h	5.1.7.10/556
3038_C284	Target Register (CCM_TARGET_ROOT133_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C288	Target Register (CCM_TARGET_ROOT133_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_C28C	Target Register (CCM_TARGET_ROOT133_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C290	Miscellaneous Register (CCM_MISC133)	32	R/W	0000_0000h	5.1.7.14/564
3038_C294	Miscellaneous Register (CCM_MISC_ROOT133_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C298	Miscellaneous Register (CCM_MISC_ROOT133_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_C29C	Miscellaneous Register (CCM_MISC_ROOT133_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C2A0	Post Divider Register (CCM_POST133)	32	R/W	0000_0000h	5.1.7.18/568
3038_C2A4	Post Divider Register (CCM_POST_ROOT133_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C2A8	Post Divider Register (CCM_POST_ROOT133_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_C2AC	Post Divider Register (CCM_POST_ROOT133_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C2B0	Pre Divider Register (CCM_PRE133)	32	R/W	1000_0000h	5.1.7.22/580
3038_C2B4	Pre Divider Register (CCM_PRE_ROOT133_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C2B8	Pre Divider Register (CCM_PRE_ROOT133_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_C2BC	Pre Divider Register (CCM_PRE_ROOT133_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C2F0	Access Control Register (CCM_ACCESS_CTRL133)	32	R/W	0000_0000h	5.1.7.26/592

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_C2F4	Access Control Register (CCM_ACCESS_CTRL_ROOT133_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C2F8	Access Control Register (CCM_ACCESS_CTRL_ROOT133_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_C2FC	Access Control Register (CCM_ACCESS_CTRL_ROOT133_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C300	Target Register (CCM_TARGET_ROOT134)	32	R/W	1000_0000h	5.1.7.10/556
3038_C304	Target Register (CCM_TARGET_ROOT134_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C308	Target Register (CCM_TARGET_ROOT134_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_C30C	Target Register (CCM_TARGET_ROOT134_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C310	Miscellaneous Register (CCM_MISC134)	32	R/W	0000_0000h	5.1.7.14/564
3038_C314	Miscellaneous Register (CCM_MISC_ROOT134_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C318	Miscellaneous Register (CCM_MISC_ROOT134_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_C31C	Miscellaneous Register (CCM_MISC_ROOT134_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C320	Post Divider Register (CCM_POST134)	32	R/W	0000_0000h	5.1.7.18/568
3038_C324	Post Divider Register (CCM_POST_ROOT134_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C328	Post Divider Register (CCM_POST_ROOT134_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_C32C	Post Divider Register (CCM_POST_ROOT134_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C330	Pre Divider Register (CCM_PRE134)	32	R/W	1000_0000h	5.1.7.22/580
3038_C334	Pre Divider Register (CCM_PRE_ROOT134_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C338	Pre Divider Register (CCM_PRE_ROOT134_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_C33C	Pre Divider Register (CCM_PRE_ROOT134_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C370	Access Control Register (CCM_ACCESS_CTRL134)	32	R/W	0000_0000h	5.1.7.26/592
3038_C374	Access Control Register (CCM_ACCESS_CTRL_ROOT134_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C378	Access Control Register (CCM_ACCESS_CTRL_ROOT134_CLR)	32	R/W	0000_0000h	5.1.7.28/597

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_C37C	Access Control Register (CCM_ACCESS_CTRL_ROOT134_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C380	Target Register (CCM_TARGET_ROOT135)	32	R/W	1000_0000h	5.1.7.10/556
3038_C384	Target Register (CCM_TARGET_ROOT135_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C388	Target Register (CCM_TARGET_ROOT135_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_C38C	Target Register (CCM_TARGET_ROOT135_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C390	Miscellaneous Register (CCM_MISC135)	32	R/W	0000_0000h	5.1.7.14/564
3038_C394	Miscellaneous Register (CCM_MISC_ROOT135_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C398	Miscellaneous Register (CCM_MISC_ROOT135_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_C39C	Miscellaneous Register (CCM_MISC_ROOT135_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C3A0	Post Divider Register (CCM_POST135)	32	R/W	0000_0000h	5.1.7.18/568
3038_C3A4	Post Divider Register (CCM_POST_ROOT135_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C3A8	Post Divider Register (CCM_POST_ROOT135_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_C3AC	Post Divider Register (CCM_POST_ROOT135_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C3B0	Pre Divider Register (CCM_PRE135)	32	R/W	1000_0000h	5.1.7.22/580
3038_C3B4	Pre Divider Register (CCM_PRE_ROOT135_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C3B8	Pre Divider Register (CCM_PRE_ROOT135_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_C3BC	Pre Divider Register (CCM_PRE_ROOT135_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C3F0	Access Control Register (CCM_ACCESS_CTRL135)	32	R/W	0000_0000h	5.1.7.26/592
3038_C3F4	Access Control Register (CCM_ACCESS_CTRL_ROOT135_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C3F8	Access Control Register (CCM_ACCESS_CTRL_ROOT135_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_C3FC	Access Control Register (CCM_ACCESS_CTRL_ROOT135_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C400	Target Register (CCM_TARGET_ROOT136)	32	R/W	1000_0000h	5.1.7.10/556

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_C404	Target Register (CCM_TARGET_ROOT136_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C408	Target Register (CCM_TARGET_ROOT136_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_C40C	Target Register (CCM_TARGET_ROOT136_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C410	Miscellaneous Register (CCM_MISC136)	32	R/W	0000_0000h	5.1.7.14/564
3038_C414	Miscellaneous Register (CCM_MISC_ROOT136_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C418	Miscellaneous Register (CCM_MISC_ROOT136_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_C41C	Miscellaneous Register (CCM_MISC_ROOT136_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C420	Post Divider Register (CCM_POST136)	32	R/W	0000_0000h	5.1.7.18/568
3038_C424	Post Divider Register (CCM_POST_ROOT136_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C428	Post Divider Register (CCM_POST_ROOT136_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_C42C	Post Divider Register (CCM_POST_ROOT136_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C430	Pre Divider Register (CCM_PRE136)	32	R/W	1000_0000h	5.1.7.22/580
3038_C434	Pre Divider Register (CCM_PRE_ROOT136_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C438	Pre Divider Register (CCM_PRE_ROOT136_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_C43C	Pre Divider Register (CCM_PRE_ROOT136_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C470	Access Control Register (CCM_ACCESS_CTRL136)	32	R/W	0000_0000h	5.1.7.26/592
3038_C474	Access Control Register (CCM_ACCESS_CTRL_ROOT136_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C478	Access Control Register (CCM_ACCESS_CTRL_ROOT136_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_C47C	Access Control Register (CCM_ACCESS_CTRL_ROOT136_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C480	Target Register (CCM_TARGET_ROOT137)	32	R/W	1000_0000h	5.1.7.10/556
3038_C484	Target Register (CCM_TARGET_ROOT137_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C488	Target Register (CCM_TARGET_ROOT137_CLR)	32	R/W	0000_0000h	5.1.7.12/560

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_C48C	Target Register (CCM_TARGET_ROOT137_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C490	Miscellaneous Register (CCM_MISC137)	32	R/W	0000_0000h	5.1.7.14/564
3038_C494	Miscellaneous Register (CCM_MISC_ROOT137_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C498	Miscellaneous Register (CCM_MISC_ROOT137_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_C49C	Miscellaneous Register (CCM_MISC_ROOT137_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C4A0	Post Divider Register (CCM_POST137)	32	R/W	0000_0000h	5.1.7.18/568
3038_C4A4	Post Divider Register (CCM_POST_ROOT137_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C4A8	Post Divider Register (CCM_POST_ROOT137_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_C4AC	Post Divider Register (CCM_POST_ROOT137_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C4B0	Pre Divider Register (CCM_PRE137)	32	R/W	1000_0000h	5.1.7.22/580
3038_C4B4	Pre Divider Register (CCM_PRE_ROOT137_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C4B8	Pre Divider Register (CCM_PRE_ROOT137_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_C4BC	Pre Divider Register (CCM_PRE_ROOT137_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C4F0	Access Control Register (CCM_ACCESS_CTRL137)	32	R/W	0000_0000h	5.1.7.26/592
3038_C4F4	Access Control Register (CCM_ACCESS_CTRL_ROOT137_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C4F8	Access Control Register (CCM_ACCESS_CTRL_ROOT137_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_C4FC	Access Control Register (CCM_ACCESS_CTRL_ROOT137_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C500	Target Register (CCM_TARGET_ROOT138)	32	R/W	1000_0000h	5.1.7.10/556
3038_C504	Target Register (CCM_TARGET_ROOT138_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C508	Target Register (CCM_TARGET_ROOT138_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_C50C	Target Register (CCM_TARGET_ROOT138_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C510	Miscellaneous Register (CCM_MISC138)	32	R/W	0000_0000h	5.1.7.14/564

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_C514	Miscellaneous Register (CCM_MISC_ROOT138_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C518	Miscellaneous Register (CCM_MISC_ROOT138_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_C51C	Miscellaneous Register (CCM_MISC_ROOT138_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C520	Post Divider Register (CCM_POST138)	32	R/W	0000_0000h	5.1.7.18/568
3038_C524	Post Divider Register (CCM_POST_ROOT138_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C528	Post Divider Register (CCM_POST_ROOT138_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_C52C	Post Divider Register (CCM_POST_ROOT138_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C530	Pre Divider Register (CCM_PRE138)	32	R/W	1000_0000h	5.1.7.22/580
3038_C534	Pre Divider Register (CCM_PRE_ROOT138_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C538	Pre Divider Register (CCM_PRE_ROOT138_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_C53C	Pre Divider Register (CCM_PRE_ROOT138_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C570	Access Control Register (CCM_ACCESS_CTRL138)	32	R/W	0000_0000h	5.1.7.26/592
3038_C574	Access Control Register (CCM_ACCESS_CTRL_ROOT138_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C578	Access Control Register (CCM_ACCESS_CTRL_ROOT138_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_C57C	Access Control Register (CCM_ACCESS_CTRL_ROOT138_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C580	Target Register (CCM_TARGET_ROOT139)	32	R/W	1000_0000h	5.1.7.10/556
3038_C584	Target Register (CCM_TARGET_ROOT139_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C588	Target Register (CCM_TARGET_ROOT139_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_C58C	Target Register (CCM_TARGET_ROOT139_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C590	Miscellaneous Register (CCM_MISC139)	32	R/W	0000_0000h	5.1.7.14/564
3038_C594	Miscellaneous Register (CCM_MISC_ROOT139_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C598	Miscellaneous Register (CCM_MISC_ROOT139_CLR)	32	R/W	0000_0000h	5.1.7.16/566

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_C59C	Miscellaneous Register (CCM_MISC_ROOT139_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C5A0	Post Divider Register (CCM_POST139)	32	R/W	0000_0000h	5.1.7.18/568
3038_C5A4	Post Divider Register (CCM_POST_ROOT139_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C5A8	Post Divider Register (CCM_POST_ROOT139_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_C5AC	Post Divider Register (CCM_POST_ROOT139_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C5B0	Pre Divider Register (CCM_PRE139)	32	R/W	1000_0000h	5.1.7.22/580
3038_C5B4	Pre Divider Register (CCM_PRE_ROOT139_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C5B8	Pre Divider Register (CCM_PRE_ROOT139_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_C5BC	Pre Divider Register (CCM_PRE_ROOT139_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C5F0	Access Control Register (CCM_ACCESS_CTRL139)	32	R/W	0000_0000h	5.1.7.26/592
3038_C5F4	Access Control Register (CCM_ACCESS_CTRL_ROOT139_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C5F8	Access Control Register (CCM_ACCESS_CTRL_ROOT139_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_C5FC	Access Control Register (CCM_ACCESS_CTRL_ROOT139_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C600	Target Register (CCM_TARGET_ROOT140)	32	R/W	1000_0000h	5.1.7.10/556
3038_C604	Target Register (CCM_TARGET_ROOT140_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C608	Target Register (CCM_TARGET_ROOT140_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_C60C	Target Register (CCM_TARGET_ROOT140_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C610	Miscellaneous Register (CCM_MISC140)	32	R/W	0000_0000h	5.1.7.14/564
3038_C614	Miscellaneous Register (CCM_MISC_ROOT140_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C618	Miscellaneous Register (CCM_MISC_ROOT140_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_C61C	Miscellaneous Register (CCM_MISC_ROOT140_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C620	Post Divider Register (CCM_POST140)	32	R/W	0000_0000h	5.1.7.18/568

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_C624	Post Divider Register (CCM_POST_ROOT140_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C628	Post Divider Register (CCM_POST_ROOT140_CLR)	32	R/W	0000_0000h	5.1.7.20/574
3038_C62C	Post Divider Register (CCM_POST_ROOT140_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C630	Pre Divider Register (CCM_PRE140)	32	R/W	1000_0000h	5.1.7.22/580
3038_C634	Pre Divider Register (CCM_PRE_ROOT140_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C638	Pre Divider Register (CCM_PRE_ROOT140_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_C63C	Pre Divider Register (CCM_PRE_ROOT140_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C670	Access Control Register (CCM_ACCESS_CTRL140)	32	R/W	0000_0000h	5.1.7.26/592
3038_C674	Access Control Register (CCM_ACCESS_CTRL_ROOT140_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C678	Access Control Register (CCM_ACCESS_CTRL_ROOT140_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_C67C	Access Control Register (CCM_ACCESS_CTRL_ROOT140_TOG)	32	R/W	0000_0000h	5.1.7.29/599
3038_C680	Target Register (CCM_TARGET_ROOT141)	32	R/W	1000_0000h	5.1.7.10/556
3038_C684	Target Register (CCM_TARGET_ROOT141_SET)	32	R/W	0000_0000h	5.1.7.11/558
3038_C688	Target Register (CCM_TARGET_ROOT141_CLR)	32	R/W	0000_0000h	5.1.7.12/560
3038_C68C	Target Register (CCM_TARGET_ROOT141_TOG)	32	R/W	0000_0000h	5.1.7.13/562
3038_C690	Miscellaneous Register (CCM_MISC141)	32	R/W	0000_0000h	5.1.7.14/564
3038_C694	Miscellaneous Register (CCM_MISC_ROOT141_SET)	32	R/W	0000_0000h	5.1.7.15/565
3038_C698	Miscellaneous Register (CCM_MISC_ROOT141_CLR)	32	R/W	0000_0000h	5.1.7.16/566
3038_C69C	Miscellaneous Register (CCM_MISC_ROOT141_TOG)	32	R/W	0000_0000h	5.1.7.17/567
3038_C6A0	Post Divider Register (CCM_POST141)	32	R/W	0000_0000h	5.1.7.18/568
3038_C6A4	Post Divider Register (CCM_POST_ROOT141_SET)	32	R/W	0000_0000h	5.1.7.19/571
3038_C6A8	Post Divider Register (CCM_POST_ROOT141_CLR)	32	R/W	0000_0000h	5.1.7.20/574

Table continues on the next page...

CCM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3038_C6AC	Post Divider Register (CCM_POST_ROOT141_TOG)	32	R/W	0000_0000h	5.1.7.21/577
3038_C6B0	Pre Divider Register (CCM_PRE141)	32	R/W	1000_0000h	5.1.7.22/580
3038_C6B4	Pre Divider Register (CCM_PRE_ROOT141_SET)	32	R/W	0000_0000h	5.1.7.23/583
3038_C6B8	Pre Divider Register (CCM_PRE_ROOT141_CLR)	32	R/W	0000_0000h	5.1.7.24/586
3038_C6BC	Pre Divider Register (CCM_PRE_ROOT141_TOG)	32	R/W	0000_0000h	5.1.7.25/589
3038_C6F0	Access Control Register (CCM_ACCESS_CTRL141)	32	R/W	0000_0000h	5.1.7.26/592
3038_C6F4	Access Control Register (CCM_ACCESS_CTRL_ROOT141_SET)	32	R/W	0000_0000h	5.1.7.27/594
3038_C6F8	Access Control Register (CCM_ACCESS_CTRL_ROOT141_CLR)	32	R/W	0000_0000h	5.1.7.28/597
3038_C6FC	Access Control Register (CCM_ACCESS_CTRL_ROOT141_TOG)	32	R/W	0000_0000h	5.1.7.29/599

5.1.7.1 General Purpose Register (CCM_GPR0n)

GPR0

Address: 3038_0000h base + 0h offset + (4d × i), where i=0d to 3d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	GP0																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

CCM_GPR0n field descriptions

Field	Description
GP0	Timeout cycle count of ipg_clk, when perform read and write.

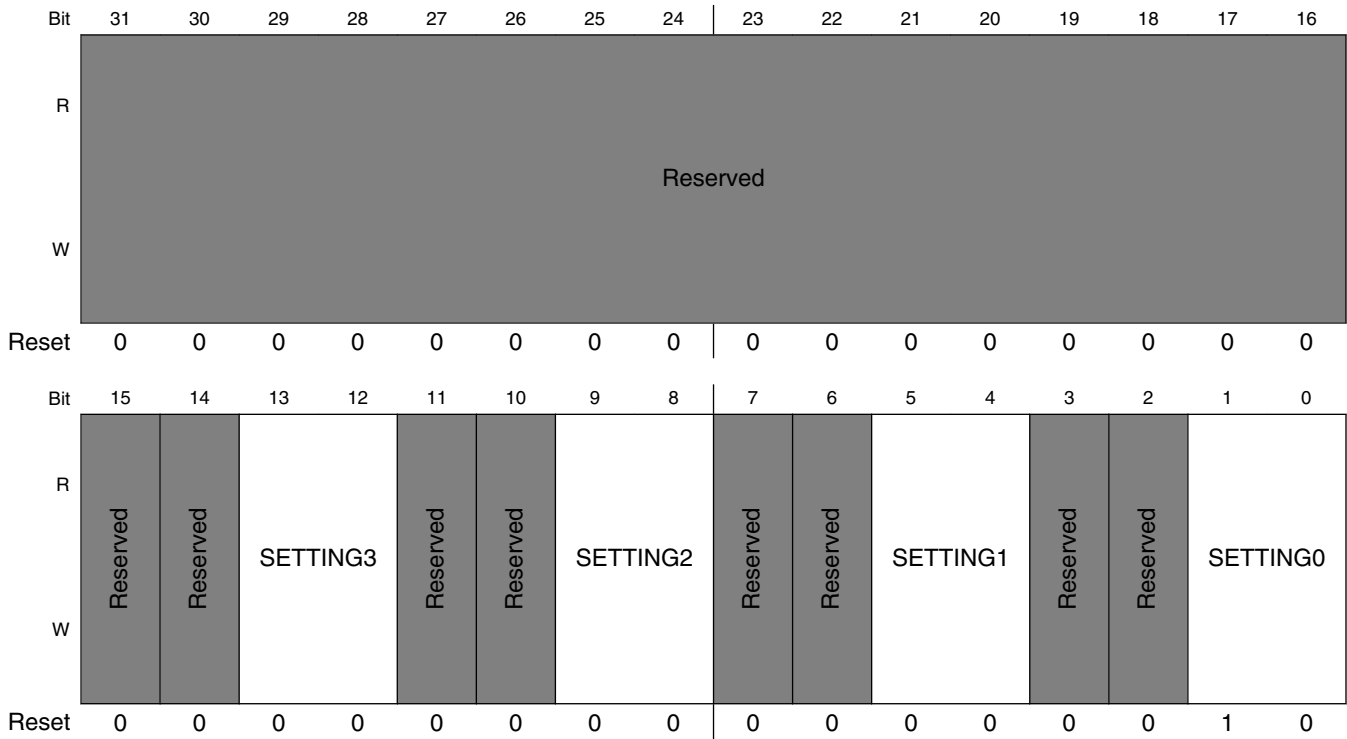
5.1.7.2 CCM PLL Control Register (CCM_PLL_CTRLn)

See [Input Clocks](#) for PLL control mapping.

NOTE

For the SoC to correctly power up after entering DSM, CCM_PLL_CTRLx must not be set to 0x0 or 0x3 for any domain in use.

Address: 3038_0000h base + 800h offset + (16d × i), where i=0d to 38d



CCM_PLL_CTRLn field descriptions

Field	Description
31–16 -	This field is reserved. Reserved
15 -	This field is reserved. Reserved
14 -	This field is reserved. Reserved
13–12 SETTING3	Clock gate control setting for domain 3. This field can only be written by domain 3 00 Domain clocks not needed

Table continues on the next page...

CCM_PLL_CTRL_n field descriptions (continued)

Field	Description
	01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
11 -	This field is reserved. Reserved
10 -	This field is reserved. Reserved
9–8 SETTING2	Clock gate control setting for domain 2. This field can only be written by domain 2 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
7 -	This field is reserved. Reserved
6 -	This field is reserved. Reserved
5–4 SETTING1	Clock gate control setting for domain 1. This field can only be written by domain 1. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
3 -	This field is reserved. Reserved
2 -	This field is reserved. Reserved
SETTING0	Clock gate control setting for domain 0. This field can only be written by domain 0. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time

5.1.7.3 CCM PLL Control Register (CCM_PLL_CTRLn_SET)

See [Input Clocks](#) for PLL control mapping.

NOTE

For the SoC to correctly power up after entering DSM, CCM_PLL_CTRLx must not be set to 0x0 or 0x3 for any domain in use.

Address: 3038_0000h base + 804h offset + (16d × i), where i=0d to 38d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved	Reserved	SETTING3		Reserved	Reserved	SETTING2		Reserved	Reserved	SETTING1		Reserved	Reserved	SETTING0	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

CCM_PLL_CTRLn_SET field descriptions

Field	Description
31–16 -	This field is reserved. Reserved
15 -	This field is reserved. Reserved
14 -	This field is reserved. Reserved
13–12 SETTING3	Clock gate control setting for domain 3. This field can only be written by domain 3 00 Domain clocks not needed

Table continues on the next page...

CCM_PLL_CTRLn_SET field descriptions (continued)

Field	Description
	01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
11 -	This field is reserved. Reserved
10 -	This field is reserved. Reserved
9–8 SETTING2	Clock gate control setting for domain 2. This field can only be written by domain 2 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
7 -	This field is reserved. Reserved
6 -	This field is reserved. Reserved
5–4 SETTING1	Clock gate control setting for domain 1. This field can only be written by domain 1. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
3 -	This field is reserved. Reserved
2 -	This field is reserved. Reserved
SETTING0	Clock gate control setting for domain 0. This field can only be written by domain 0. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time

5.1.7.4 CCM PLL Control Register (CCM_PLL_CTRLn_CLR)

See [Input Clocks](#) for PLL control mapping.

NOTE

For the SoC to correctly power up after entering DSM, CCM_PLL_CTRLx must not be set to 0x0 or 0x3 for any domain in use.

Address: 3038_0000h base + 808h offset + (16d × i), where i=0d to 38d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved	Reserved	SETTING3		Reserved	Reserved	SETTING2		Reserved	Reserved	SETTING1		Reserved	Reserved	SETTING0	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

CCM_PLL_CTRLn_CLR field descriptions

Field	Description
31–16 -	This field is reserved. Reserved
15 -	This field is reserved. Reserved
14 -	This field is reserved. Reserved
13–12 SETTING3	Clock gate control setting for domain 3. This field can only be written by domain 3 00 Domain clocks not needed

Table continues on the next page...

CCM_PLL_CTRLn_CLR field descriptions (continued)

Field	Description
	01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
11 -	This field is reserved. Reserved
10 -	This field is reserved. Reserved
9–8 SETTING2	Clock gate control setting for domain 2. This field can only be written by domain 2 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
7 -	This field is reserved. Reserved
6 -	This field is reserved. Reserved
5–4 SETTING1	Clock gate control setting for domain 1. This field can only be written by domain 1. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
3 -	This field is reserved. Reserved
2 -	This field is reserved. Reserved
SETTING0	Clock gate control setting for domain 0. This field can only be written by domain 0. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time

5.1.7.5 CCM PLL Control Register (CCM_PLL_CTRLn_TOG)

See [Input Clocks](#) for PLL control mapping.

NOTE

For the SoC to correctly power up after entering DSM, CCM_PLL_CTRLx must not be set to 0x0 or 0x3 for any domain in use.

Address: 3038_0000h base + 80Ch offset + (16d × i), where i=0d to 38d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved	Reserved	SETTING3		Reserved	Reserved	SETTING2		Reserved	Reserved	SETTING1		Reserved	Reserved	SETTING0	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

CCM_PLL_CTRLn_TOG field descriptions

Field	Description
31–16 -	This field is reserved. Reserved
15 -	This field is reserved. Reserved
14 -	This field is reserved. Reserved
13–12 SETTING3	Clock gate control setting for domain 3. This field can only be written by domain 3 00 Domain clocks not needed

Table continues on the next page...

CCM_PLL_CTRLn_TOG field descriptions (continued)

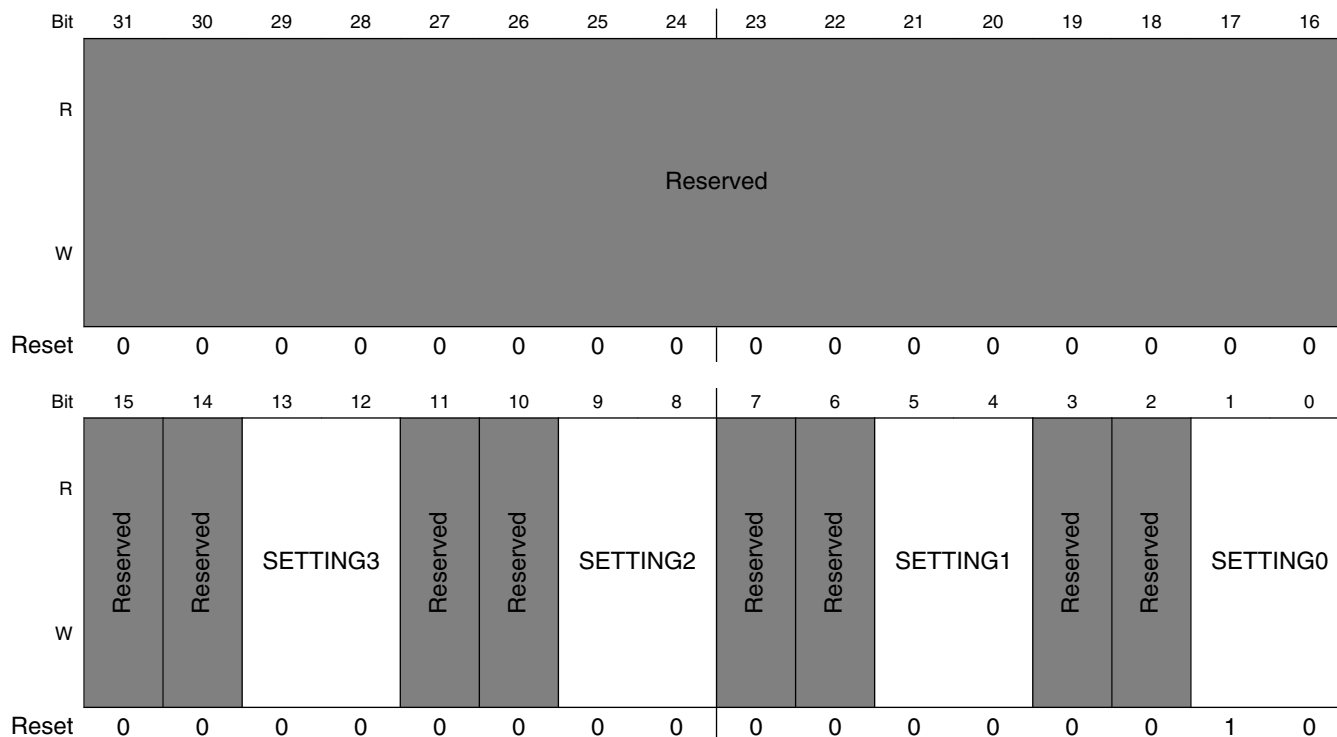
Field	Description
	01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
11 -	This field is reserved. Reserved
10 -	This field is reserved. Reserved
9–8 SETTING2	Clock gate control setting for domain 2. This field can only be written by domain 2 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
7 -	This field is reserved. Reserved
6 -	This field is reserved. Reserved
5–4 SETTING1	Clock gate control setting for domain 1. This field can only be written by domain 1. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
3 -	This field is reserved. Reserved
2 -	This field is reserved. Reserved
SETTING0	Clock gate control setting for domain 0. This field can only be written by domain 0. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time

5.1.7.6 CCM Clock Gating Register (CCM_CCGRn)

NOTE

Not all CCGRs are mapped. See [CCGR Interface](#) for CCGR mapping and clock gating information.

Address: 3038_0000h base + 4000h offset + (16d × i), where i=0d to 191d



CCM_CCGRn field descriptions

Field	Description
31–16 -	This field is reserved. Reserved
15 -	This field is reserved. Reserved
14 -	This field is reserved. Reserved
13–12 SETTING3	Clock gate control setting for domain 3. This field can only be written by domain 3 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time

Table continues on the next page...

CCM_CCGRn field descriptions (continued)

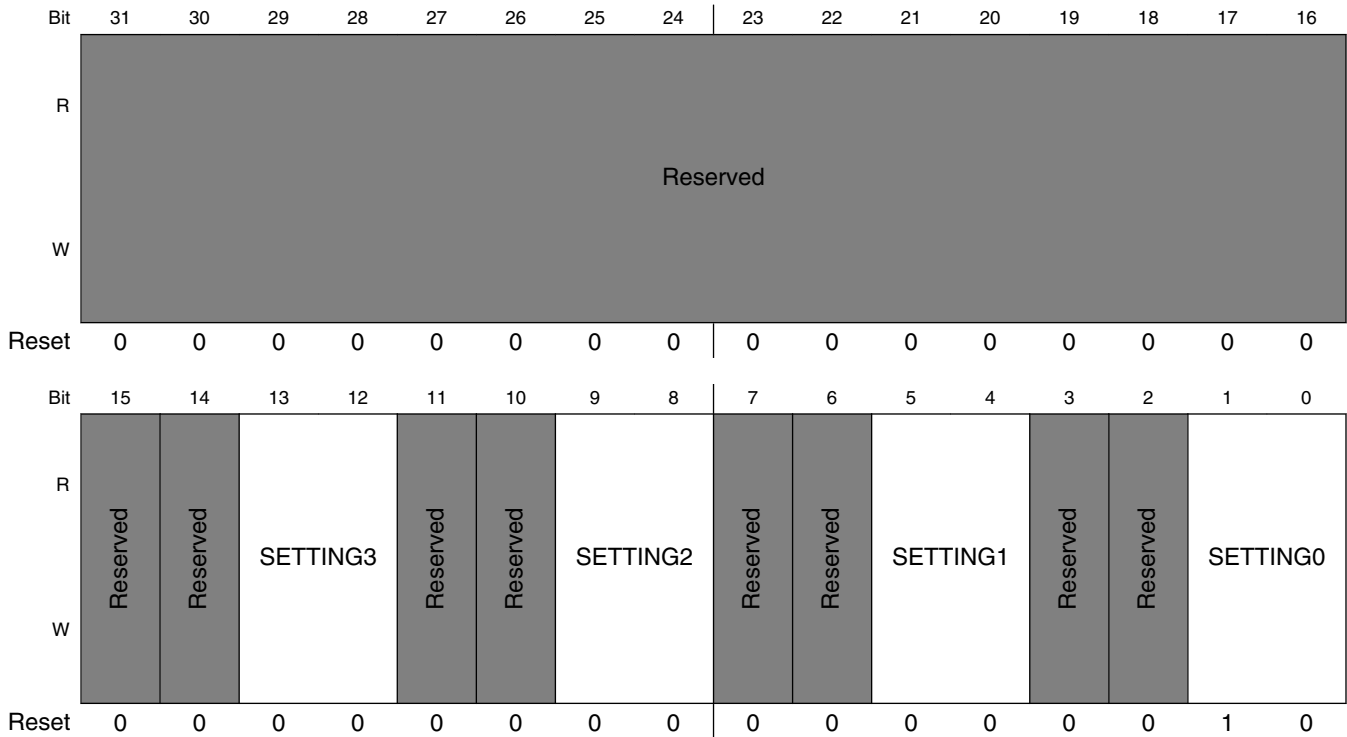
Field	Description
11 -	This field is reserved. Reserved
10 -	This field is reserved. Reserved
9–8 SETTING2	Clock gate control setting for domain 2. This field can only be written by domain 2 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
7 -	This field is reserved. Reserved
6 -	This field is reserved. Reserved
5–4 SETTING1	Clock gate control setting for domain 1. This field can only be written by domain 1. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
3 -	This field is reserved. Reserved
2 -	This field is reserved. Reserved
SETTING0	Clock gate control setting for domain 0. This field can only be written by domain 0. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time

5.1.7.7 CCM Clock Gating Register (CCM_CCGRn_SET)

NOTE

Not all CCGRs are mapped. See [CCGR Interface](#) for CCGR mapping and clock gating information.

Address: 3038_0000h base + 4004h offset + (16d × i), where i=0d to 191d



CCM_CCGRn_SET field descriptions

Field	Description
31–16 -	This field is reserved. Reserved
15 -	This field is reserved. Reserved
14 -	This field is reserved. Reserved
13–12 SETTING3	Clock gate control setting for domain 3. This field can only be written by domain 3 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time

Table continues on the next page...

CCM_CCGRn_SET field descriptions (continued)

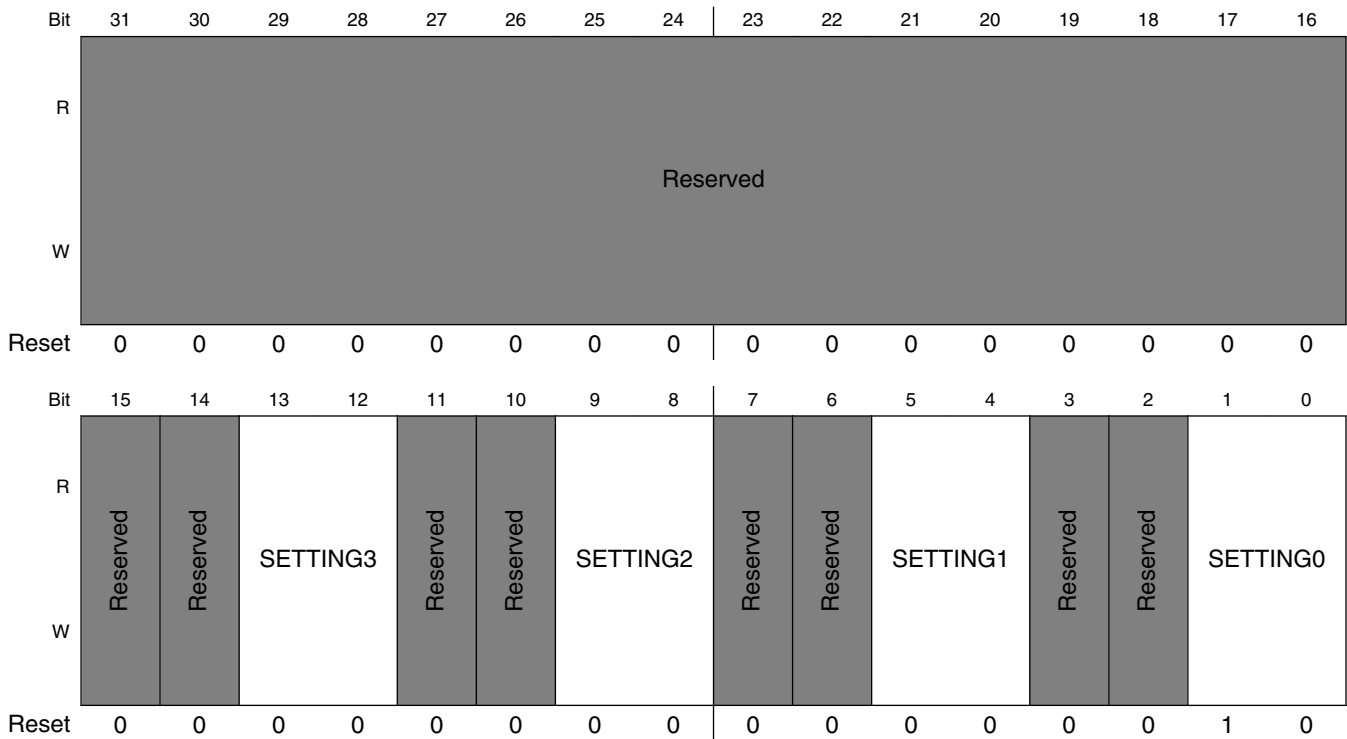
Field	Description
11 -	This field is reserved. Reserved
10 -	This field is reserved. Reserved
9–8 SETTING2	Clock gate control setting for domain 2. This field can only be written by domain 2 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
7 -	This field is reserved. Reserved
6 -	This field is reserved. Reserved
5–4 SETTING1	Clock gate control setting for domain 1. This field can only be written by domain 1. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
3 -	This field is reserved. Reserved
2 -	This field is reserved. Reserved
SETTING0	Clock gate control setting for domain 0. This field can only be written by domain 0. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time

5.1.7.8 CCM Clock Gating Register (CCM_CCGRn_CLR)

NOTE

Not all CCGRs are mapped. See [CCGR Interface](#) for CCGR mapping and clock gating information.

Address: 3038_0000h base + 4008h offset + (16d × i), where i=0d to 191d



CCM_CCGRn_CLR field descriptions

Field	Description
31–16 -	This field is reserved. Reserved
15 -	This field is reserved. Reserved
14 -	This field is reserved. Reserved
13–12 SETTING3	Clock gate control setting for domain 3. This field can only be written by domain 3 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time

Table continues on the next page...

CCM_CCGRn_CLR field descriptions (continued)

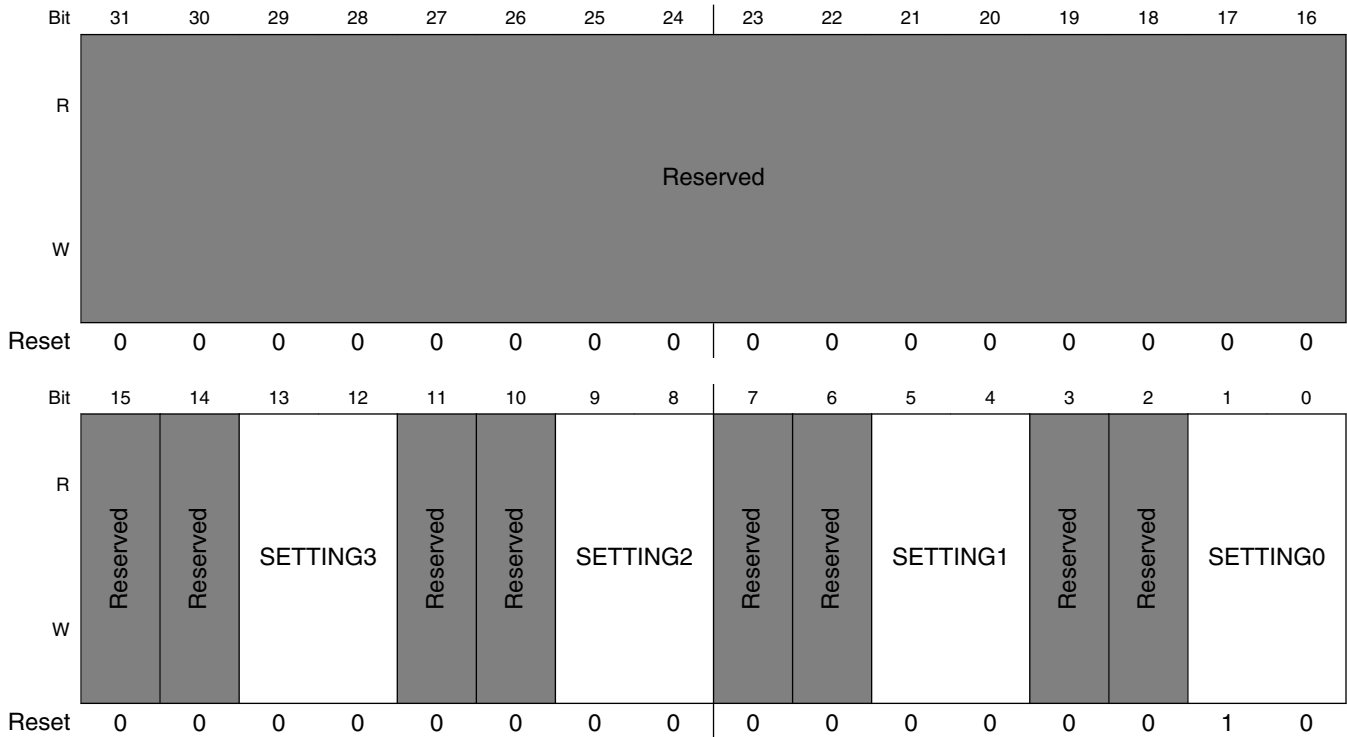
Field	Description
11 -	This field is reserved. Reserved
10 -	This field is reserved. Reserved
9–8 SETTING2	Clock gate control setting for domain 2. This field can only be written by domain 2 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
7 -	This field is reserved. Reserved
6 -	This field is reserved. Reserved
5–4 SETTING1	Clock gate control setting for domain 1. This field can only be written by domain 1. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
3 -	This field is reserved. Reserved
2 -	This field is reserved. Reserved
SETTING0	Clock gate control setting for domain 0. This field can only be written by domain 0. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time

5.1.7.9 CCM Clock Gating Register (CCM_CCGRn_TOG)

NOTE

Not all CCGRs are mapped. See [CCGR Interface](#) for CCGR mapping and clock gating information.

Address: 3038_0000h base + 400Ch offset + (16d × i), where i=0d to 191d



CCM_CCGRn_TOG field descriptions

Field	Description
31–16 -	This field is reserved. Reserved
15 -	This field is reserved. Reserved
14 -	This field is reserved. Reserved
13–12 SETTING3	Clock gate control setting for domain 3. This field can only be written by domain 3 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time

Table continues on the next page...

CCM_CCGRn_TOG field descriptions (continued)

Field	Description
11 -	This field is reserved. Reserved
10 -	This field is reserved. Reserved
9–8 SETTING2	Clock gate control setting for domain 2. This field can only be written by domain 2 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
7 -	This field is reserved. Reserved
6 -	This field is reserved. Reserved
5–4 SETTING1	Clock gate control setting for domain 1. This field can only be written by domain 1. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time
3 -	This field is reserved. Reserved
2 -	This field is reserved. Reserved
SETTING0	Clock gate control setting for domain 0. This field can only be written by domain 0. 00 Domain clocks not needed 01 Domain clocks needed when in RUN 10 Domain clocks needed when in RUN and WAIT 11 Domain clocks needed all the time

5.1.7.10 Target Register (CCM_TARGET_ROOT n)

See [Target Interface](#) for more information.

NOTE

See [Clock Root Selects](#) for clock root offsets and muxing information.

Address: 3038_0000h base + 8000h offset + (128d × i), where i=0d to 141d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved			ENABLE	Reserved	MUX			Reserved					PRE_PODF		
W																
Reset	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved										POST_PODF					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

CCM_TARGET_ROOT n field descriptions

Field	Description
31–29 -	This field is reserved. Reserved
28 ENABLE	Enable this clock 0 clock root is OFF 1 clock root is ON
27 -	This field is reserved. Reserved
26–24 MUX	Selection of clock sources This field is 1 bit long for DRAM and CORE

Table continues on the next page...

CCM_TARGET_ROOT n field descriptions (continued)

Field	Description
23–19 -	This field is reserved. Reserved
18–16 PRE_PODF	Pre divider divide the number Divider value is $n+1$ This field does not apply for CORE, DRAM, DRAM_PHYM 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
15–6 -	This field is reserved. Reserved
POST_PODF	Post divider divide number Divider value is $n + 1$. For CORE, this field is 3 bit long. For IPG, this field is 1 bit long. This field does not apply to DRAM_PHYM 000000 Divide by 1 000001 Divide by 2 000010 Divide by 3 000011 Divide by 4 000100 Divide by 5 000101 Divide by 6 : 111111 Divide by 64

5.1.7.11 Target Register (CCM_TARGET_ROOT n _SET)

See [Target Interface](#) for more information.

NOTE

See [Clock Root Selects](#) for clock root offsets and muxing information.

Address: 3038_0000h base + 8004h offset + (128d × i), where i=0d to 141d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved			ENABLE	Reserved	MUX			Reserved					PRE_PODF		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved										POST_PODF					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

CCM_TARGET_ROOT n _SET field descriptions

Field	Description
31–29 -	This field is reserved. Reserved
28 ENABLE	Enable this clock 0 clock root is OFF 1 clock root is ON
27 -	This field is reserved. Reserved
26–24 MUX	Selection of clock sources This field is 1 bit long for DRAM and CORE

Table continues on the next page...

CCM_TARGET_ROOT n _SET field descriptions (continued)

Field	Description
23–19 -	This field is reserved. Reserved
18–16 PRE_PODF	Pre divider divide the number Divider value is $n+1$ This field does not apply for CORE, DRAM, DRAM_PHYM 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
15–6 -	This field is reserved. Reserved
POST_PODF	Post divider divide the number Divider value is $n + 1$. For CORE, this field is 3 bit long. For IPG, this field is 1 bit long. This field does not apply to DRAM_PHYM 000000 Divide by 1 000001 Divide by 2 000010 Divide by 3 000011 Divide by 4 000100 Divide by 5 000101 Divide by 6 : 111111 Divide by 64

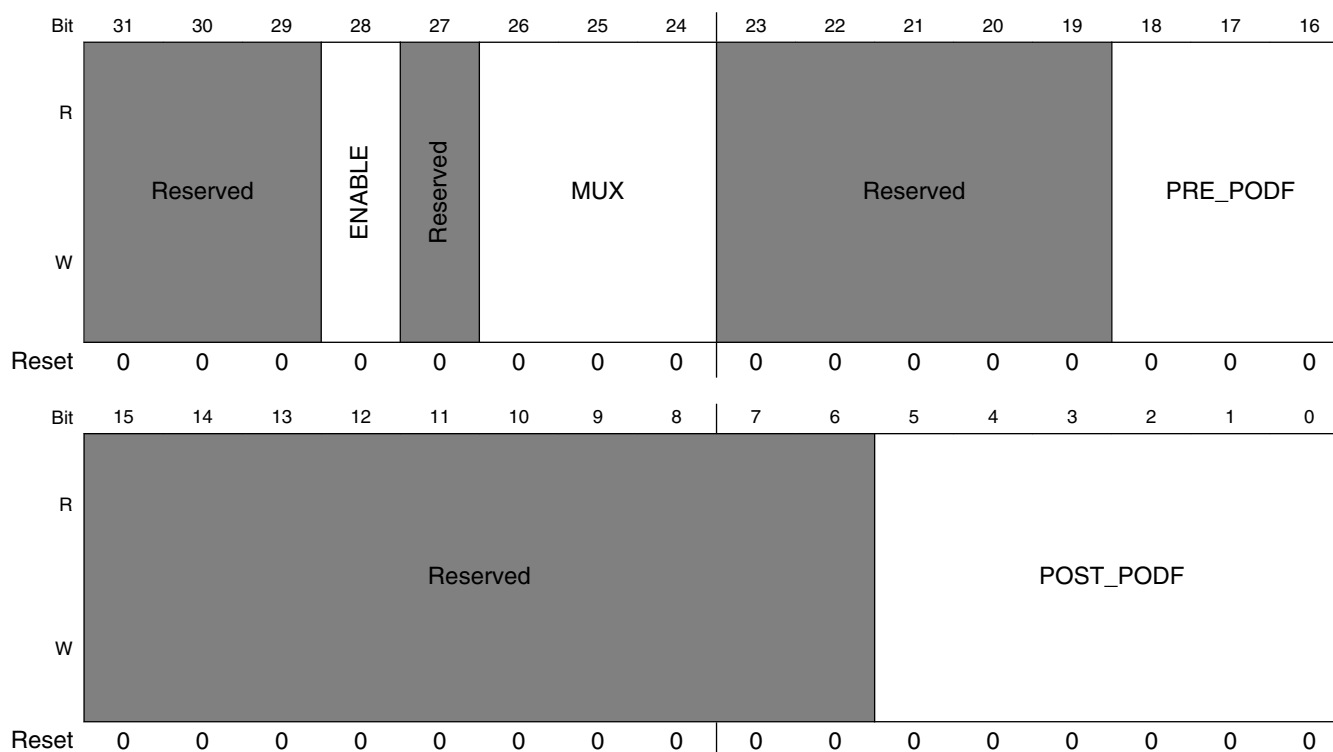
5.1.7.12 Target Register (CCM_TARGET_ROOT n _CLR)

See [Target Interface](#) for more information.

NOTE

See [Clock Root Selects](#) for clock root offsets and muxing information.

Address: 3038_0000h base + 8008h offset + (128d × i), where i=0d to 141d



CCM_TARGET_ROOT n _CLR field descriptions

Field	Description
31–29 -	This field is reserved. Reserved
28 ENABLE	Enable this clock 0 clock root is OFF 1 clock root is ON
27 -	This field is reserved. Reserved
26–24 MUX	Selection of clock sources This field is 1 bit long for DRAM and CORE

Table continues on the next page...

CCM_TARGET_ROOT n _CLR field descriptions (continued)

Field	Description
23–19 -	This field is reserved. Reserved
18–16 PRE_PODF	Pre divider divide the number Divider value is $n+1$ This field does not apply for CORE, DRAM, DRAM_PHYM 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
15–6 -	This field is reserved. Reserved
POST_PODF	Post divider divide the number Divider value is $n + 1$. For CORE, this field is 3 bit long. For IPG, this field is 1 bit long. This field does not apply to DRAM_PHYM 000000 Divide by 1 000001 Divide by 2 000010 Divide by 3 000011 Divide by 4 000100 Divide by 5 000101 Divide by 6 : 111111 Divide by 64

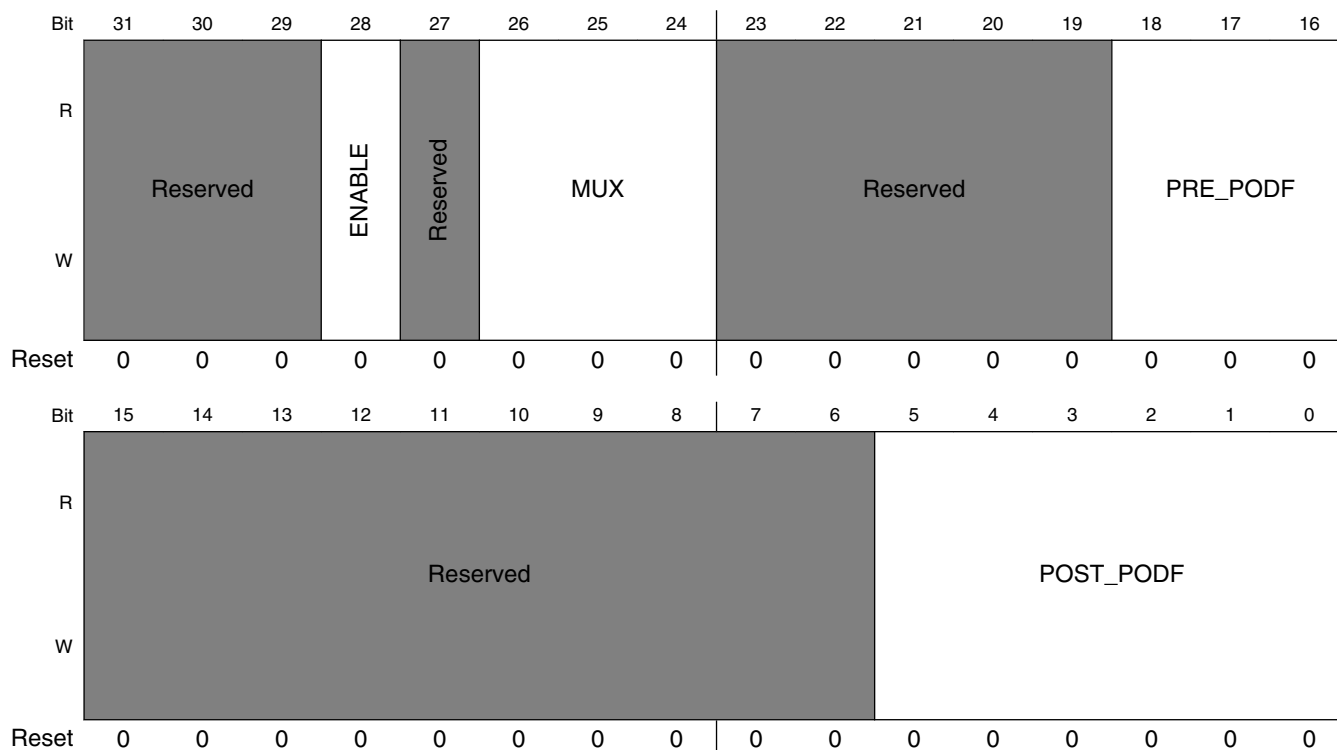
5.1.7.13 Target Register (CCM_TARGET_ROOT n _TOG)

See [Target Interface](#) for more information.

NOTE

See [Clock Root Selects](#) for clock root offsets and muxing information.

Address: 3038_0000h base + 800Ch offset + (128d × i), where i=0d to 141d



CCM_TARGET_ROOT n _TOG field descriptions

Field	Description
31–29 -	This field is reserved. Reserved
28 ENABLE	Enable this clock 0 clock root is OFF 1 clock root is ON
27 -	This field is reserved. Reserved
26–24 MUX	Selection of clock sources This field is 1 bit long for DRAM and CORE

Table continues on the next page...

CCM_TARGET_ROOT n _TOG field descriptions (continued)

Field	Description
23–19 -	This field is reserved. Reserved
18–16 PRE_PODF	Pre divide divide number Divider value is $n+1$ This field does not apply for CORE, DRAM, DRAM_PHYM 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
15–6 -	This field is reserved. Reserved
POST_PODF	Post divider divide number Divider value is $n + 1$. For CORE, this field is 3 bit long. For IPG, this field is 1 bit long. This field does not apply to DRAM_PHYM 000000 Divide by 1 000001 Divide by 2 000010 Divide by 3 000011 Divide by 4 000100 Divide by 5 000101 Divide by 6 : 111111 Divide by 64

5.1.7.14 Miscellaneous Register (CCM_MISCN)

MISC

Address: 3038_0000h base + 8010h offset + (128d × i), where i=0d to 141d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved																
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved								VIOLATE	Reserved			TIMEOUT	Reserved			AUTHEN_FAIL
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

CCM_MISCN field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 VIOLATE	This sticky bit reflects access violation in normal interface of this clock. This bit has internal 4 bits, one for each domain. Violation from other domain is not visible or clearable. This file is cleared to 0 while write 1.
7–5 -	This field is reserved. Reserved
4 TIMEOUT	This sticky bit reflects time out happened during accessing this clock. This bit has internal 4 bits, one for each domain. Timeout from other domain is not visible or clearable. This file is cleared to 0 while write 1.
3–1 -	This field is reserved. Reserved
0 AUTHEN_FAIL	This sticky bit reflects access restricted by access control of this clock. This bit has internal 4 bits, one for each domain. Authentic fail from other domain is not visible or clearable. This file is cleared to 0 while write 1

5.1.7.15 Miscellaneous Register (CCM_MISC_ROOTn_SET)

Misc

Address: 3038_0000h base + 8014h offset + (128d × i), where i=0d to 141d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								VIOLATE				TIMEOUT			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

CCM_MISC_ROOTn_SET field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 VIOLATE	This sticky bit reflects access violation in normal interface of this clock. This bit has internal 4 bits, one for each domain. Violation from other domain is not visible or clearable. This file is cleared to 0 while write 1.
7–5 -	This field is reserved. Reserved
4 TIMEOUT	This sticky bit reflects time out happened during accessing this clock. This bit has internal 4 bits, one for each domain. Timeout from other domain is not visible or clearable. This file is cleared to 0 while write 1.
3–1 -	This field is reserved. Reserved
0 AUTHEN_FAIL	This sticky bit reflects access restricted by access control of this clock. This bit has internal 4 bits, one for each domain. Authentic fail from other domain is not visible or clearable. This file is cleared to 0 while write 1

5.1.7.16 Miscellaneous Register (CCM_MISC_ROOTn_CLR)

MISC

Address: 3038_0000h base + 8018h offset + (128d × i), where i=0d to 141d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								VIOLATE				TIMEOUT			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

CCM_MISC_ROOTn_CLR field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 VIOLATE	This sticky bit reflects access violation in normal interface of this clock. This bit has internal 4 bits, one for each domain. Violation from other domain is not visible or clearable. This file is cleared to 0 while write 1.
7–5 -	This field is reserved. Reserved
4 TIMEOUT	This sticky bit reflects time out happened during accessing this clock. This bit has internal 4 bits, one for each domain. Timeout from other domain is not visible or clearable. This file is cleared to 0 while write 1.
3–1 -	This field is reserved. Reserved
0 AUTHEN_FAIL	This sticky bit reflects access restricted by access control of this clock. This bit has internal 4 bits, one for each domain. Authentic fail from other domain is not visible or clearable. This file is cleared to 0 while write 1

5.1.7.17 Miscellaneous Register (CCM_MISC_ROOTn_TOG)

MISC

Address: 3038_0000h base + 801Ch offset + (128d × i), where i=0d to 141d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved																
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved								VIOLATE	Reserved			TIMEOUT	Reserved			AUTHEN_FAIL
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

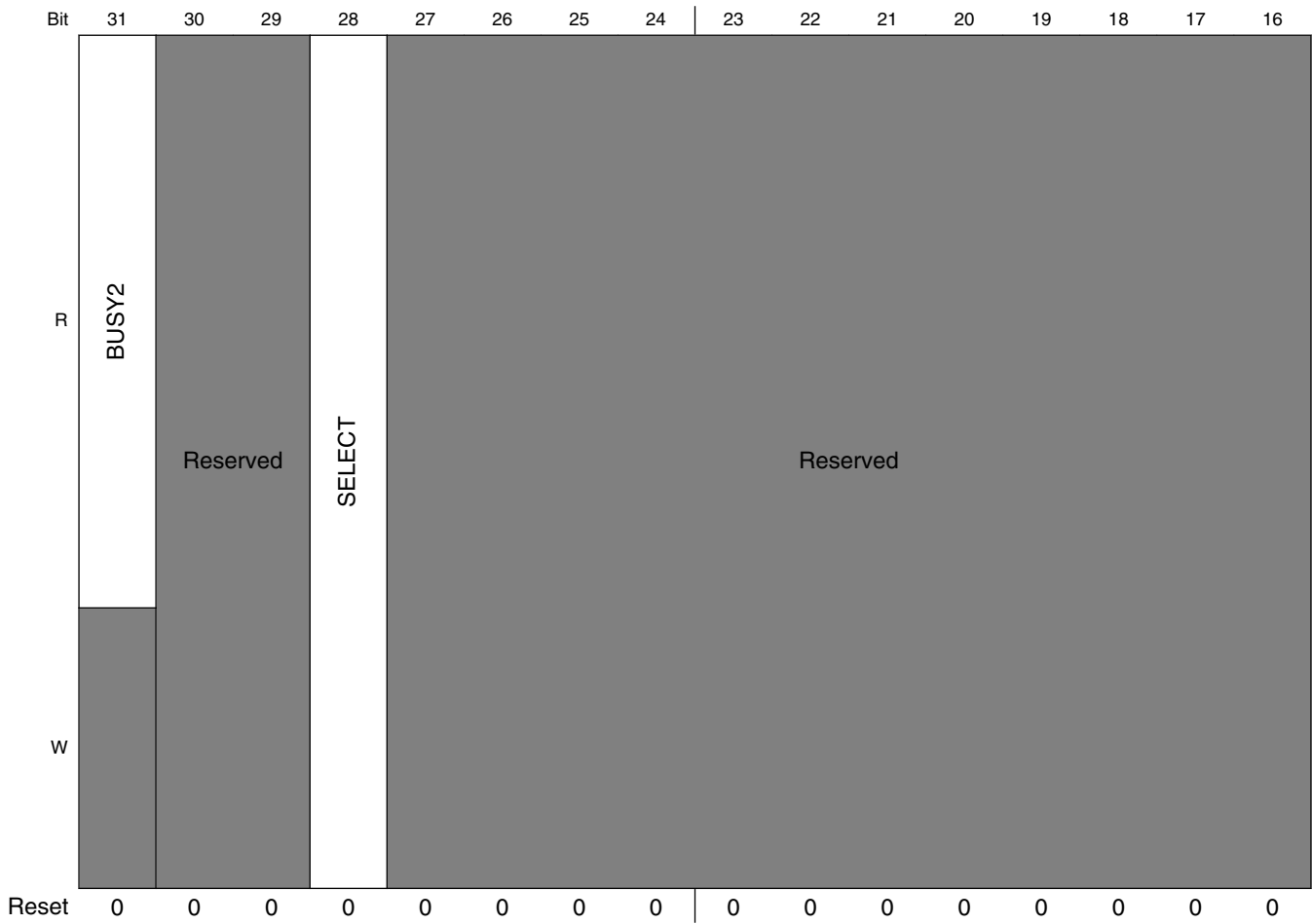
CCM_MISC_ROOTn_TOG field descriptions

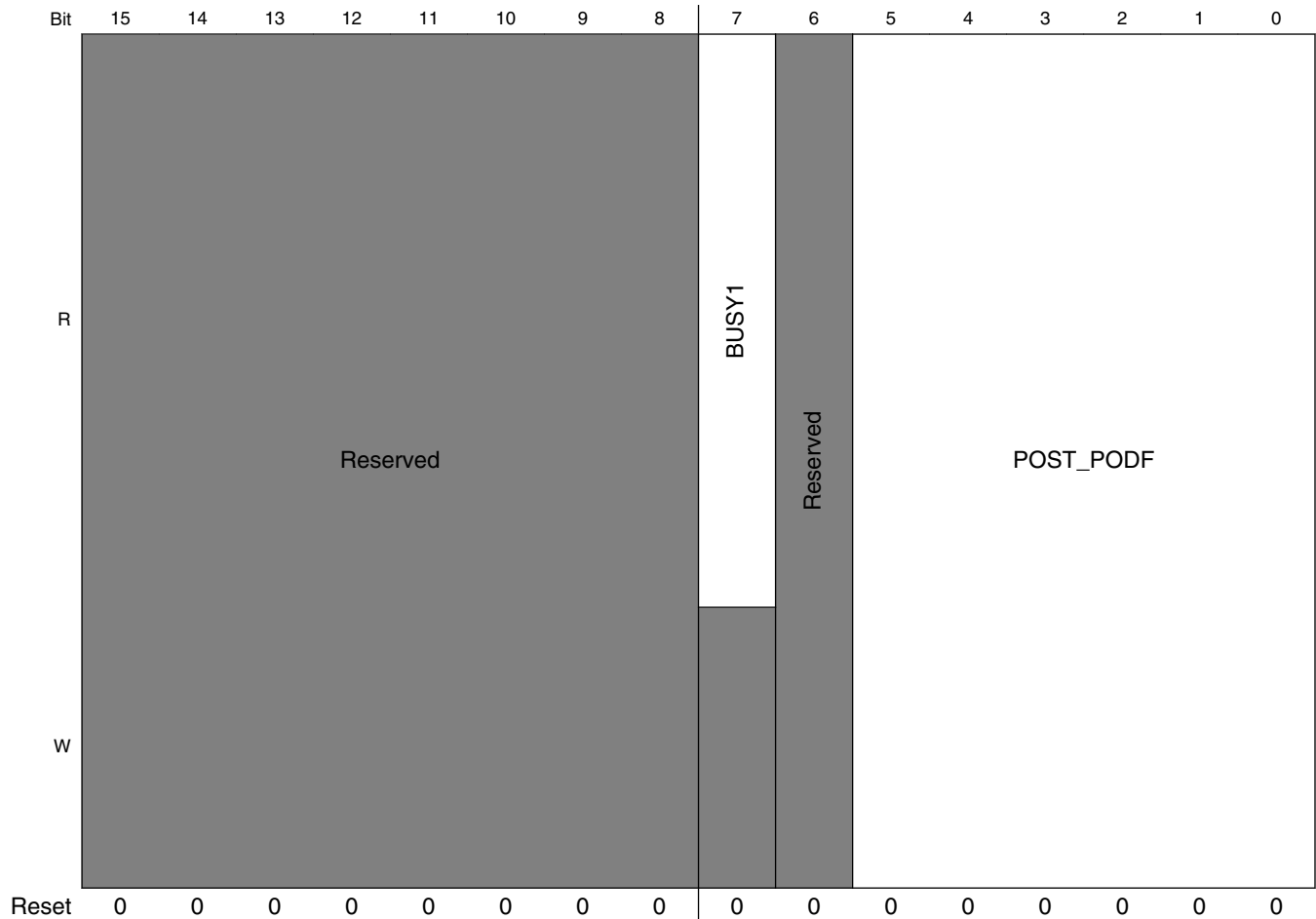
Field	Description
31–9 -	This field is reserved. Reserved
8 VIOLATE	This sticky bit reflects access violation in normal interface of this clock. This bit has internal 4 bits, one for each domain. Violation from other domain is not visible or clearable. This file is cleared to 0 while write 1.
7–5 -	This field is reserved. Reserved
4 TIMEOUT	This sticky bit reflects time out happened during accessing this clock. This bit has internal 4 bits, one for each domain. Timeout from other domain is not visible or clearable. This file is cleared to 0 while write 1.
3–1 -	This field is reserved. Reserved
0 AUTHEN_FAIL	This sticky bit reflects access restricted by access control of this clock. This bit has internal 4 bits, one for each domain. Authentic fail from other domain is not visible or clearable. This file is cleared to 0 while write 1

5.1.7.18 Post Divider Register (CCM_POST*n*)

Post Register

Address: 3038_0000h base + 8020h offset + (128d × i), where i=0d to 141d



CCM_POST n field descriptions

Field	Description
31 BUSY2	Clock switching multiplexer is applying new setting
30–29 -	This field is reserved. Reserved
28 SELECT	Selection of pre clock branches This field is not applicable to IP(Peripheral) Clock Slice, see Peripheral clock slice Peripheral clock slice 0 select branch A 1 select branch B
27–8 -	This field is reserved. Reserved
7 BUSY1	Post divider is applying new set value
6 -	This field is reserved. Reserved
POST_PODF	Post divider divide the number Divider value is $n + 1$

Table continues on the next page...

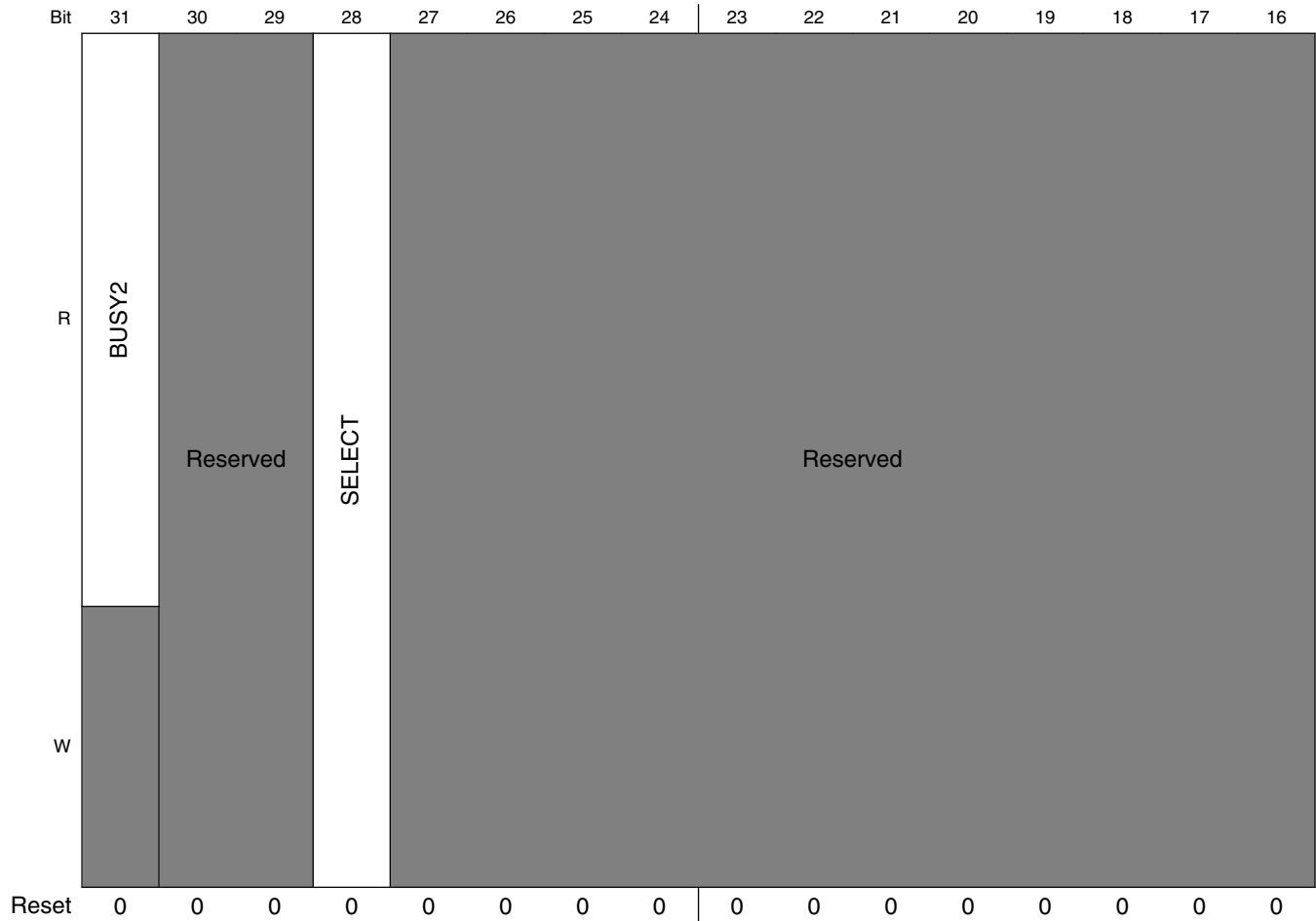
CCM_POST n field descriptions (continued)

Field	Description
	For CORE, this field is 3 bit long. For IPG, this field is 2 bit long. This field does not apply to DRAM_PHYM
	000000 Divide by 1
	000001 Divide by 2
	000010 Divide by 3
	000011 Divide by 4
	000100 Divide by 5
	000101 Divide by 6
	:
	111111 Divide by 64

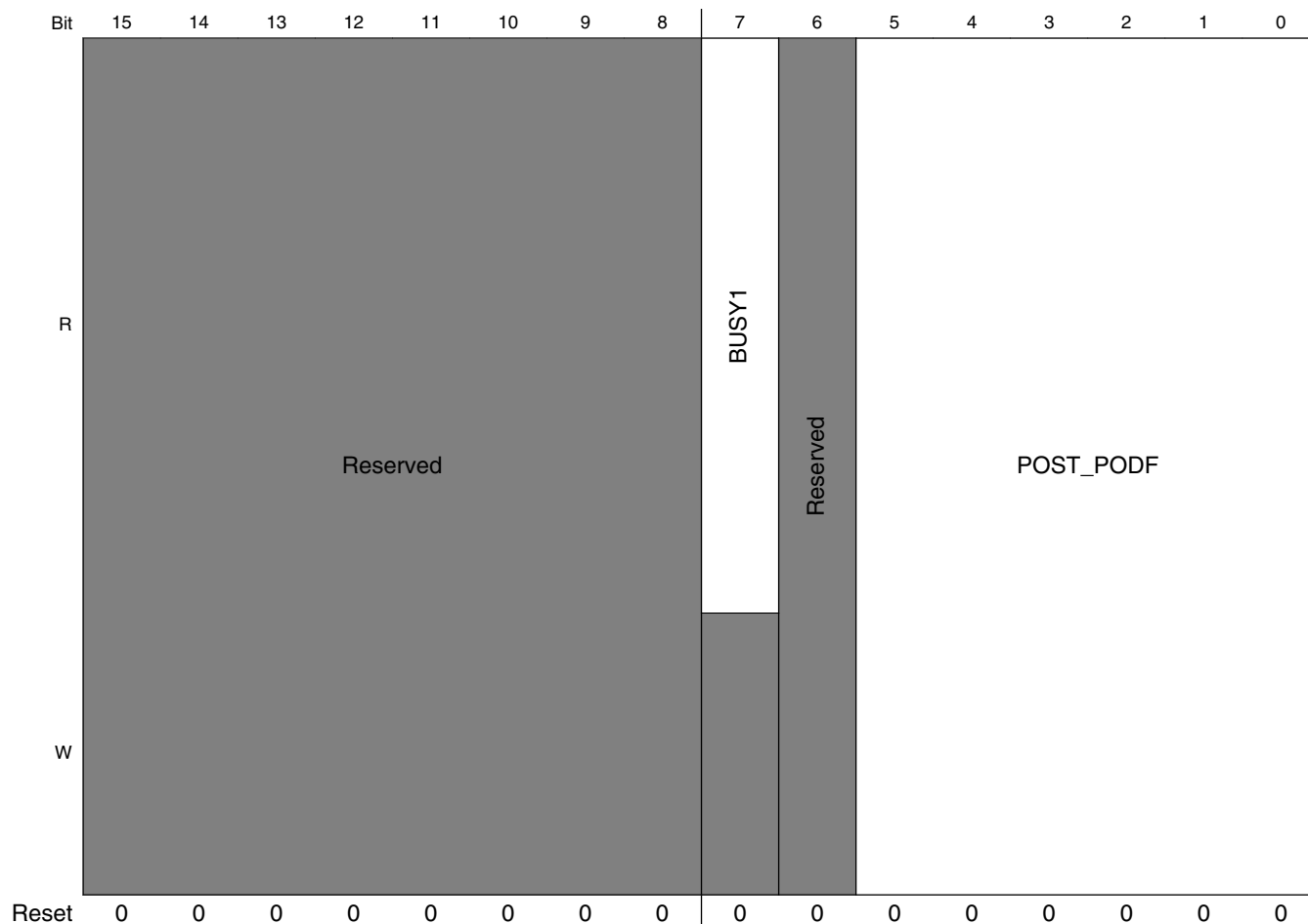
5.1.7.19 Post Divider Register (CCM_POST_ROOTn_SET)

Post Divider Register

Address: 3038_0000h base + 8024h offset + (128d × i), where i=0d to 141d



Clock Control Module (CCM)



CCM_POST_ROOTn_SET field descriptions

Field	Description
31 BUSY2	Clock switching multiplexer is applying new setting
30–29 -	This field is reserved. Reserved
28 SELECT	Selection of pre clock branches This field is not applicable to IP(Peripheral) Clock Slice, see Peripheral clock slice Peripheral clock slice 0 select branch A 1 select branch B
27–8 -	This field is reserved. Reserved
7 BUSY1	Post divider is applying new set value
6 -	This field is reserved. Reserved
POST_PODF	Post divider divide number Divider value is n + 1

Table continues on the next page...

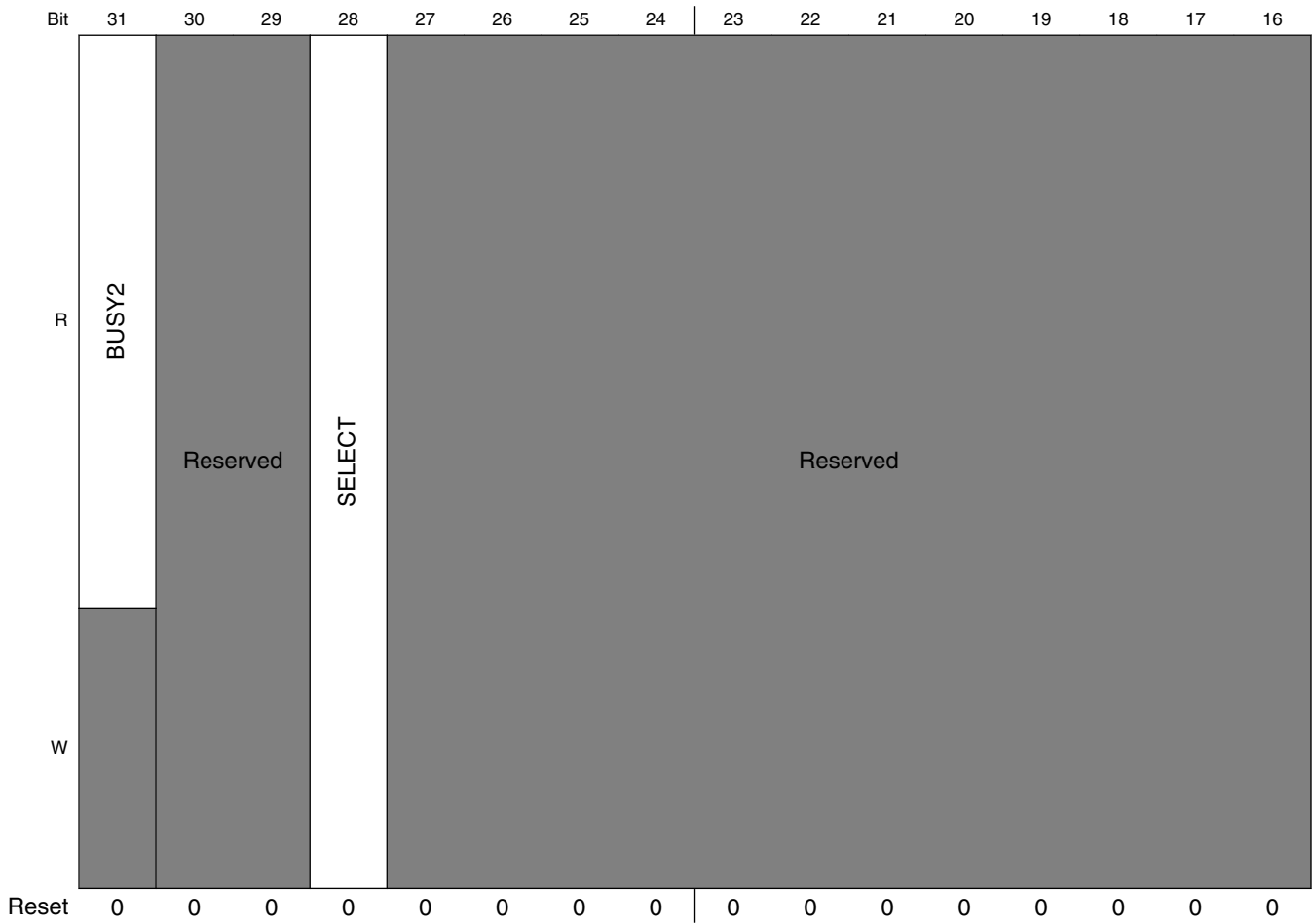
CCM_POST_ROOT n _SET field descriptions (continued)

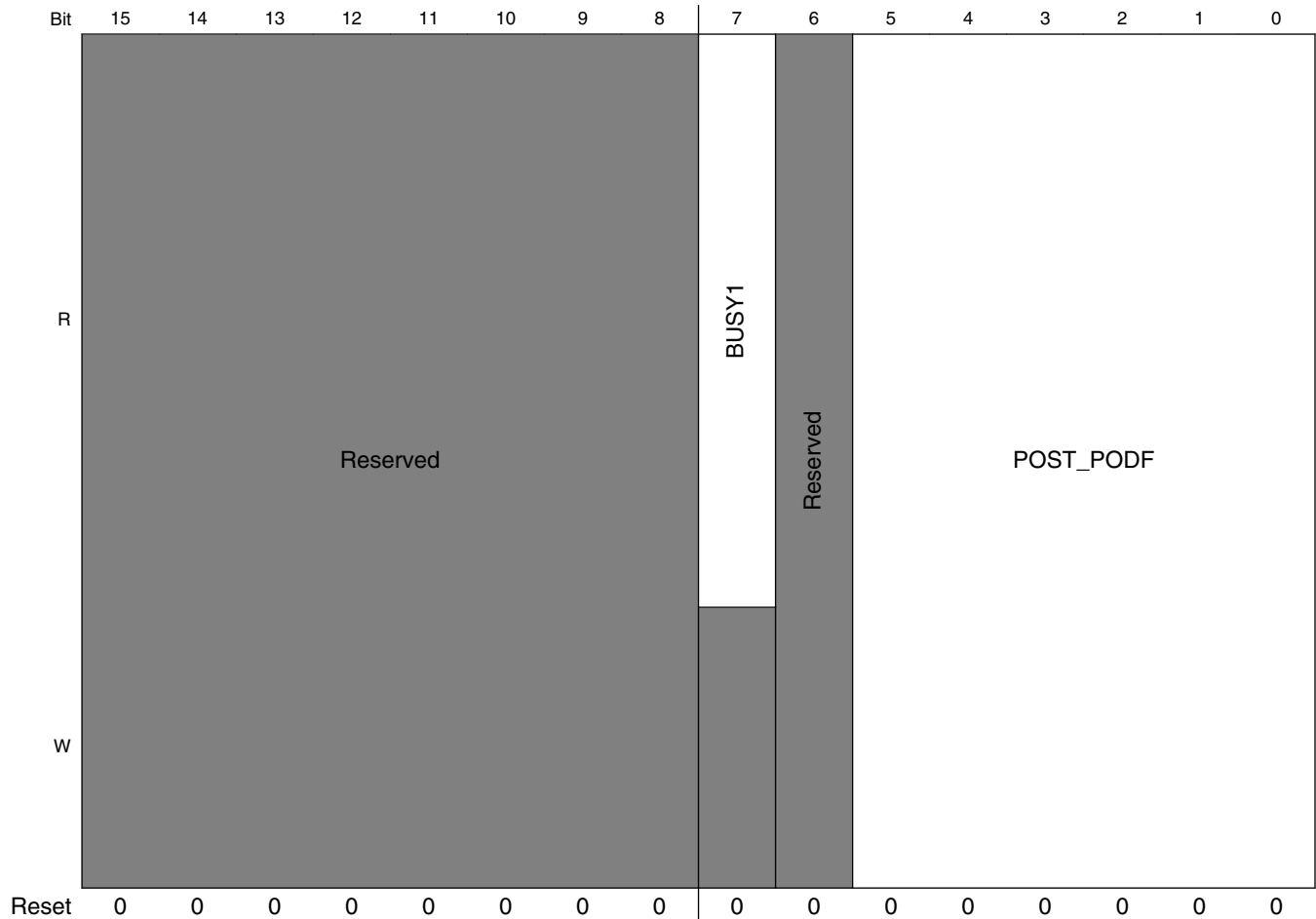
Field	Description
	For CORE, this field is 3 bit long.
	For IPG, this field is 2 bit long.
	This field does not apply to DRAM_PHYM
	000000 Divide by 1
	000001 Divide by 2
	000010 Divide by 3
	000011 Divide by 4
	000100 Divide by 5
	000101 Divide by 6
	:
	111111 Divide by 64

5.1.7.20 Post Divider Register (CCM_POST_ROOTn_CLR)

Post Root Register

Address: 3038_0000h base + 8028h offset + (128d × i), where i=0d to 141d





CCM_POST_ROOTn_CLR field descriptions

Field	Description
31 BUSY2	Clock switching multiplexer is applying new setting
30–29 -	This field is reserved. Reserved
28 SELECT	Selection of pre clock branches This field is not applicable to IP(Peripheral) Clock Slice, see Peripheral clock slice Peripheral clock slice 0 select branch A 1 select branch B
27–8 -	This field is reserved. Reserved
7 BUSY1	Post divider is applying new set value
6 -	This field is reserved. Reserved
POST_PODF	Post divider divide the number Divider value is $n + 1$

Table continues on the next page...

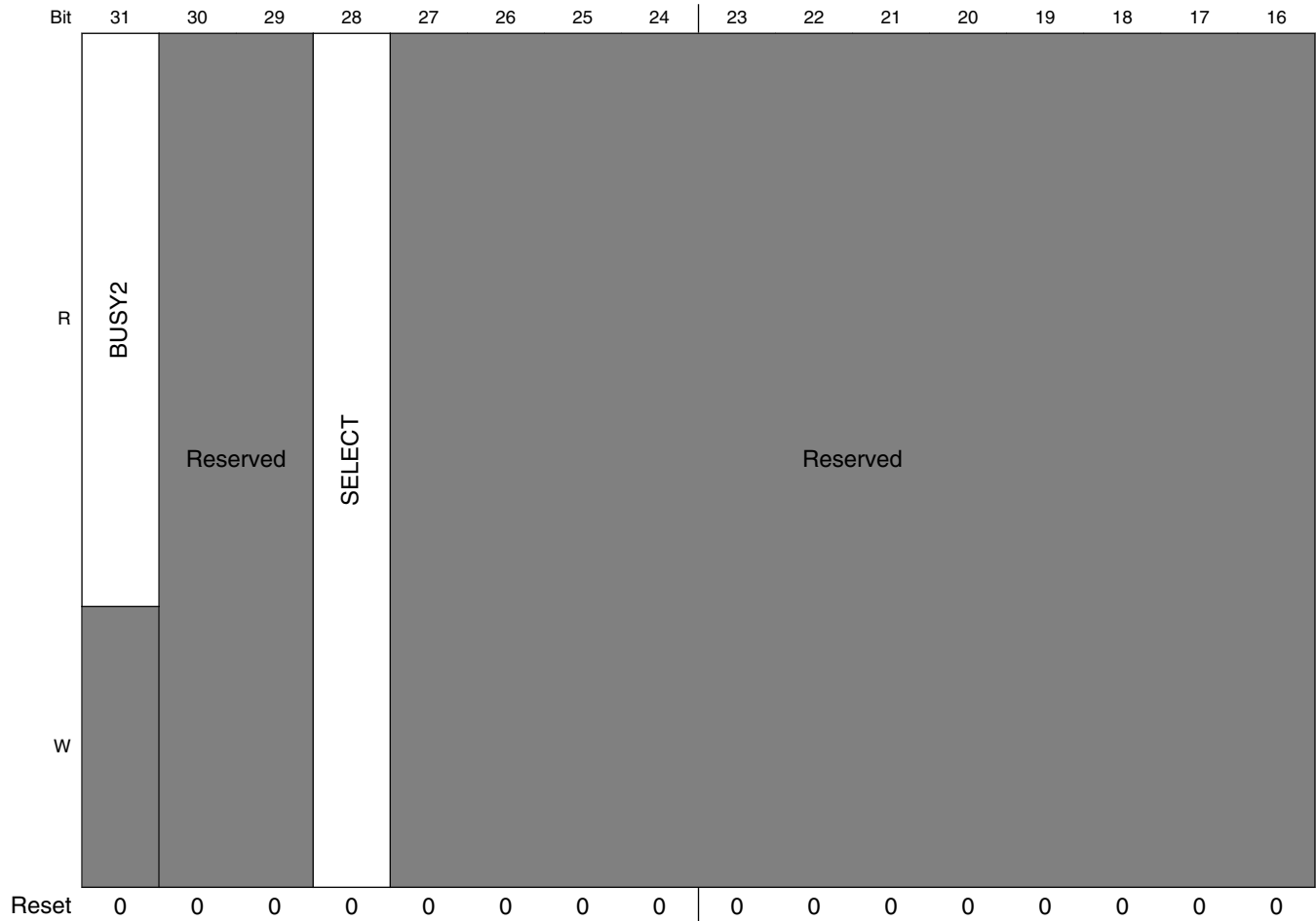
CCM_POST_ROOT n _CLR field descriptions (continued)

Field	Description
	For CORE, this field is 3 bit long. For IPG, this field is 2 bit long. This field does not apply to DRAM_PHYM
	000000 Divide by 1
	000001 Divide by 2
	000010 Divide by 3
	000011 Divide by 4
	000100 Divide by 5
	000101 Divide by 6
	:
	111111 Divide by 64

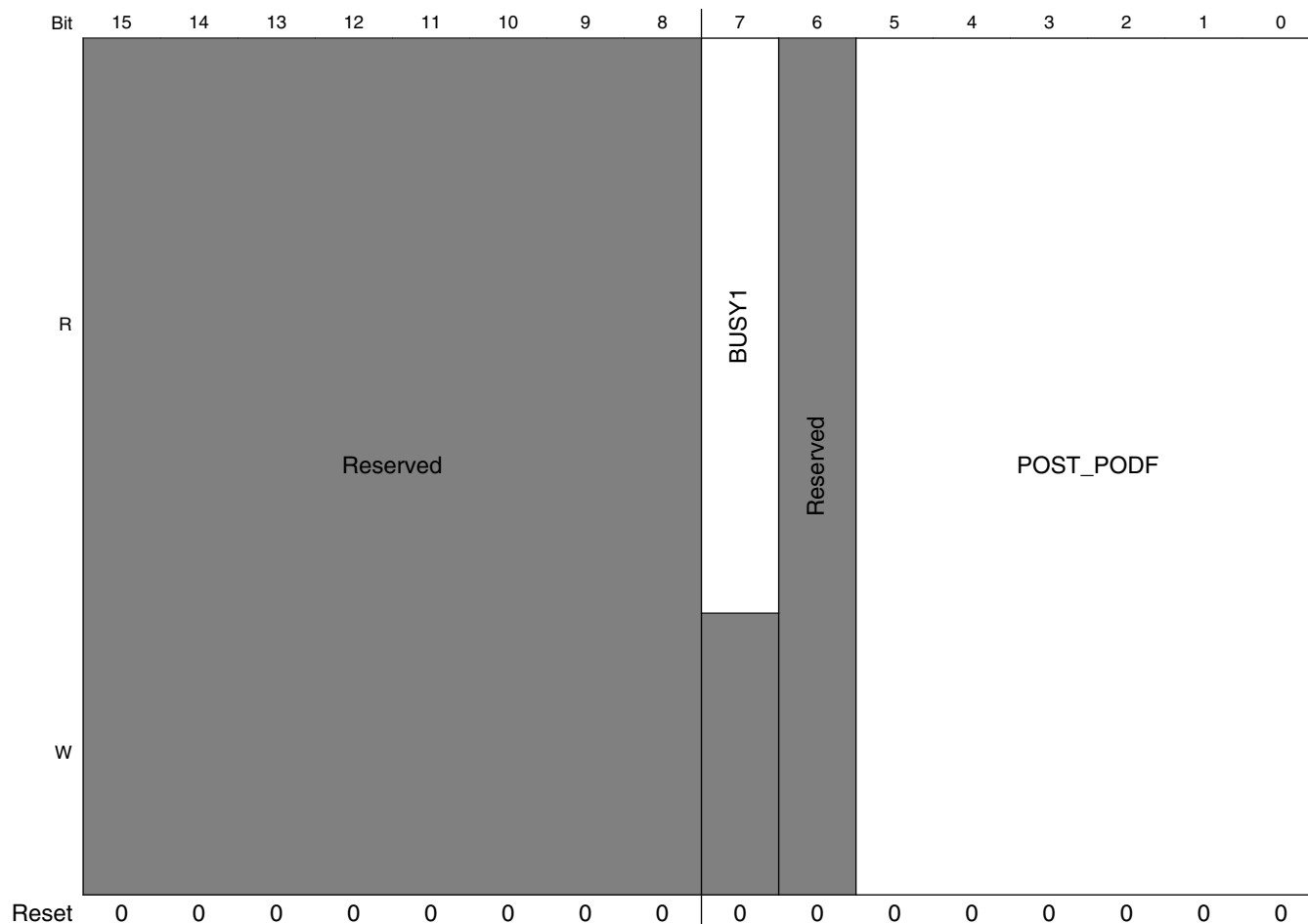
5.1.7.21 Post Divider Register (CCM_POST_ROOTn_TOG)

Post Root Register

Address: 3038_0000h base + 802Ch offset + (128d × i), where i=0d to 141d



Clock Control Module (CCM)



CCM_POST_ROOTn_TOG field descriptions

Field	Description
31 BUSY2	Clock switching multiplexer is applying new setting
30–29 -	This field is reserved. Reserved
28 SELECT	Selection of pre clock branches This field is not applicable to IP(Peripheral) Clock Slice, see Peripheral clock slice Peripheral clock slice 0 select branch A 1 select branch B
27–8 -	This field is reserved. Reserved
7 BUSY1	Post divider is applying new set value
6 -	This field is reserved. Reserved
POST_PODF	Post divider divide number Divider value is $n + 1$

Table continues on the next page...

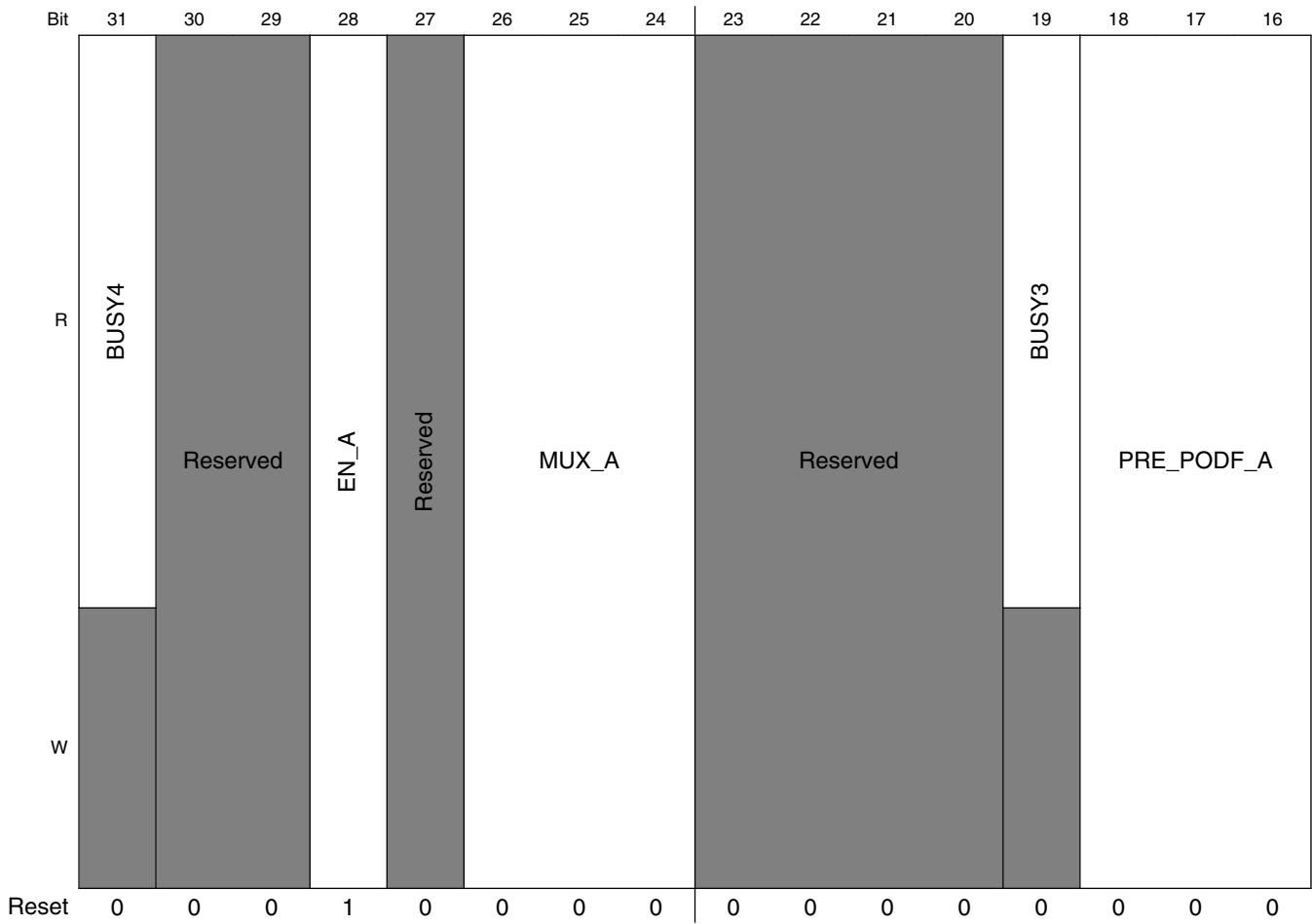
CCM_POST_ROOT n _TOG field descriptions (continued)

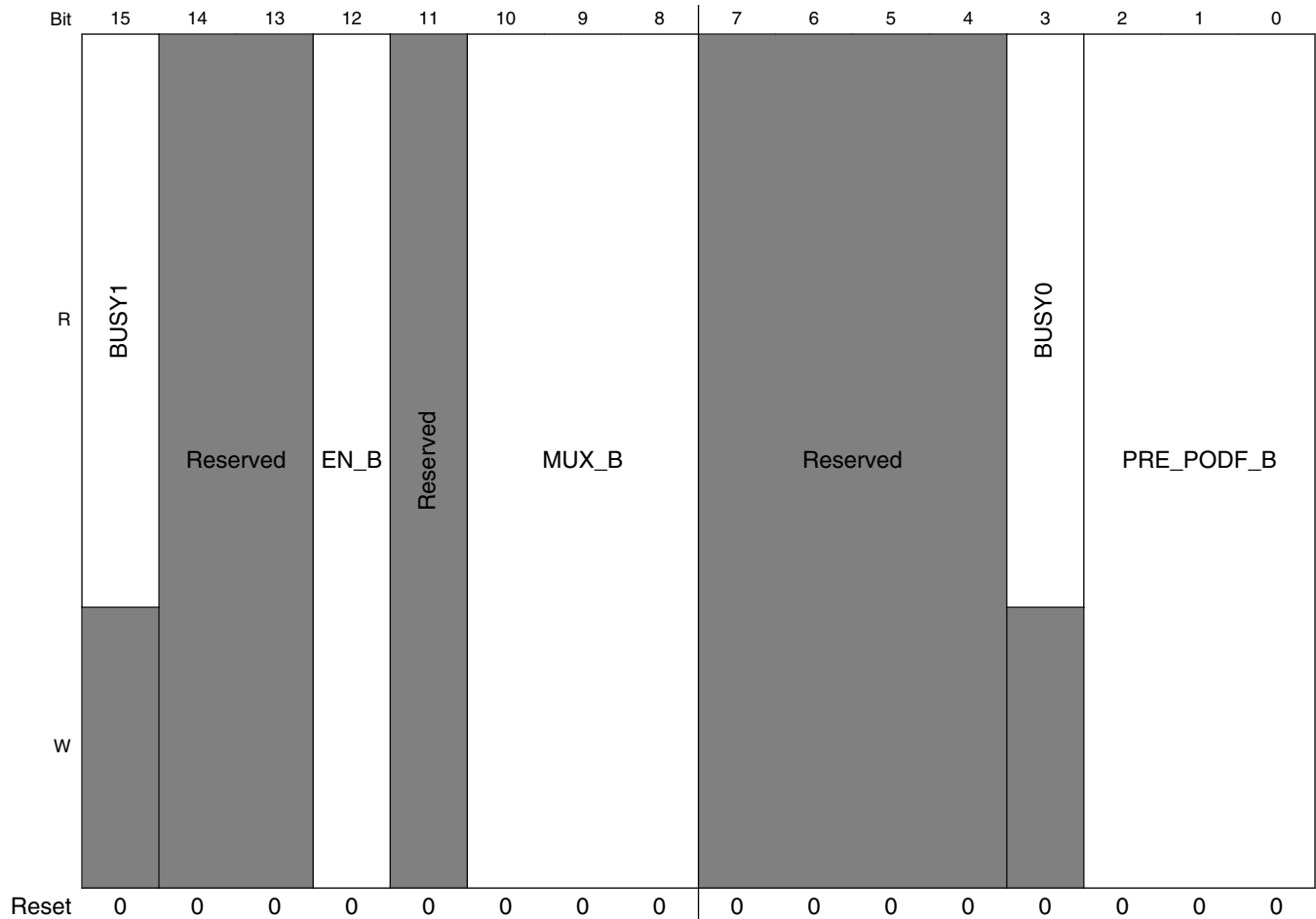
Field	Description
	For CORE, this field is 3 bit long.
	For IPG, this field is 2 bit long.
	This field does not apply to DRAM_PHYM
	000000 Divide by 1
	000001 Divide by 2
	000010 Divide by 3
	000011 Divide by 4
	000100 Divide by 5
	000101 Divide by 6
	:
	111111 Divide by 64

5.1.7.22 Pre Divider Register (CCM_PREn)

Pre Register

Address: 3038_0000h base + 8030h offset + (128d × i), where i=0d to 141d





CCM_PREN field descriptions

Field	Description
31 BUSY4	EN_A field is applied to field This field applies to DRAM and DRAM_PHYM
30–29 -	This field is reserved. Reserved
28 EN_A	Branch A clock gate control This field applies to DRAM and DRAM_PHYM 0 Clock shutdown 1 clock ON
27 -	This field is reserved. Reserved
26–24 MUX_A	Selection control of multiplexer of branch A This field applies to DRAM and DRAM_PHYM
23–20 -	This field is reserved. Reserved
19 BUSY3	Pre divider value for branch A is applied This field applies to DRAM and DRAM_PHYM

Table continues on the next page...

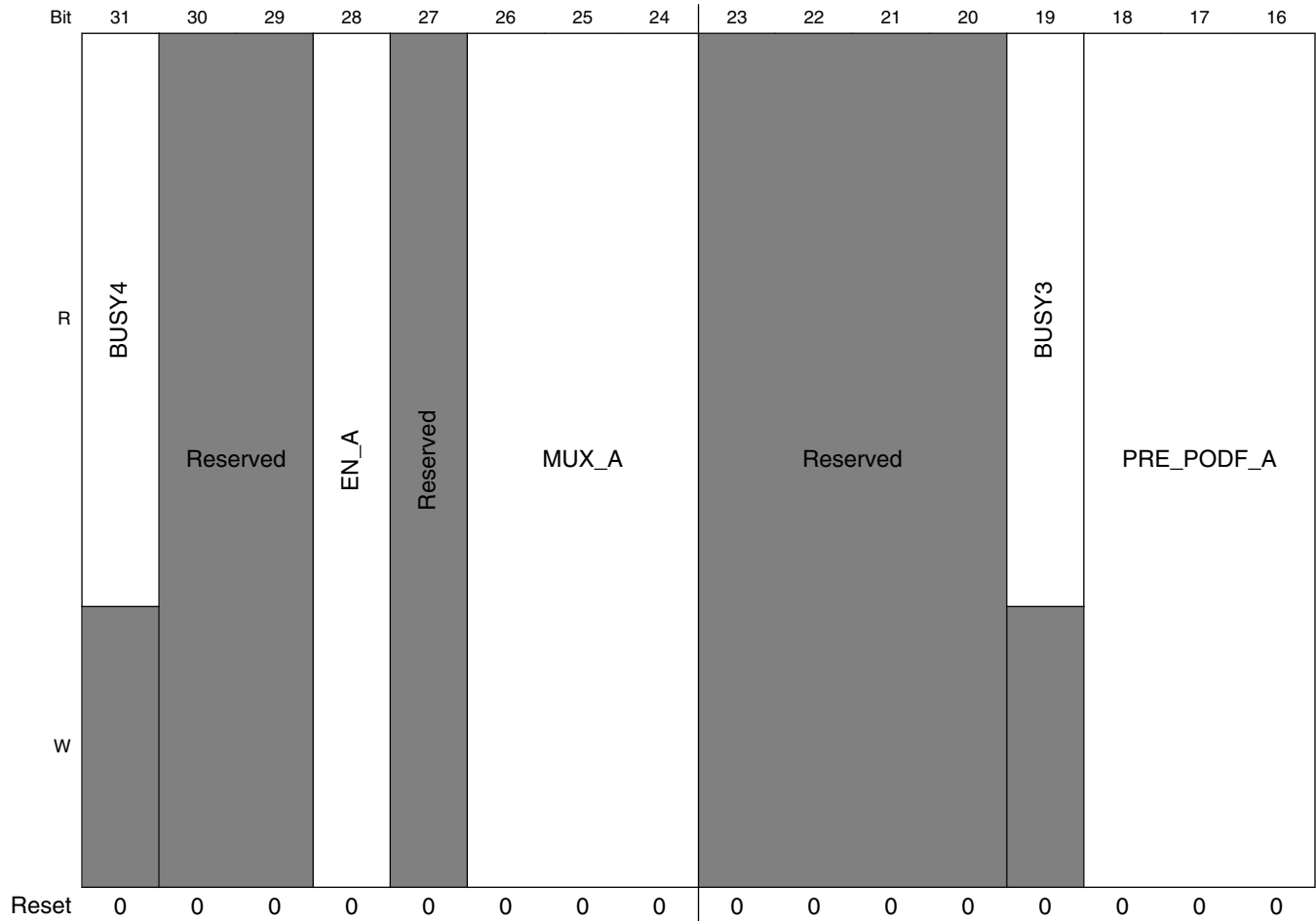
CCM_PREN field descriptions (continued)

Field	Description
18–16 PRE_PODF_A	Pre divider divide number for branch A Divider value is $n + 1$. This field does not apply for CORE, DRAM, DRAM_PHYM 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
15 BUSY1	EN_B is applied to field This field does not apply for CORE, IP, DRAM, DRAM_PHYM
14–13 -	This field is reserved. Reserved
12 EN_B	Branch B clock gate control This field does not apply for CORE, IP, DRAM, DRAM_PHYM 0 Clock shutdown 1 Clock ON
11 -	This field is reserved. Reserved
10–8 MUX_B	Selection control of multiplexer of branch B This field does not apply for CORE, IP, DRAM, DRAM_PHYM
7–4 -	This field is reserved. Reserved
3 BUSY0	Pre divider value for branch a is applying field does not apply for CORE, IP, DRAM, DRAM_PHYM
PRE_PODF_B	Pre divider divide number for branch B Divider value is $n + 1$. This field does not apply for CORE, IP, DRAM, DRAM_PHYM 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8

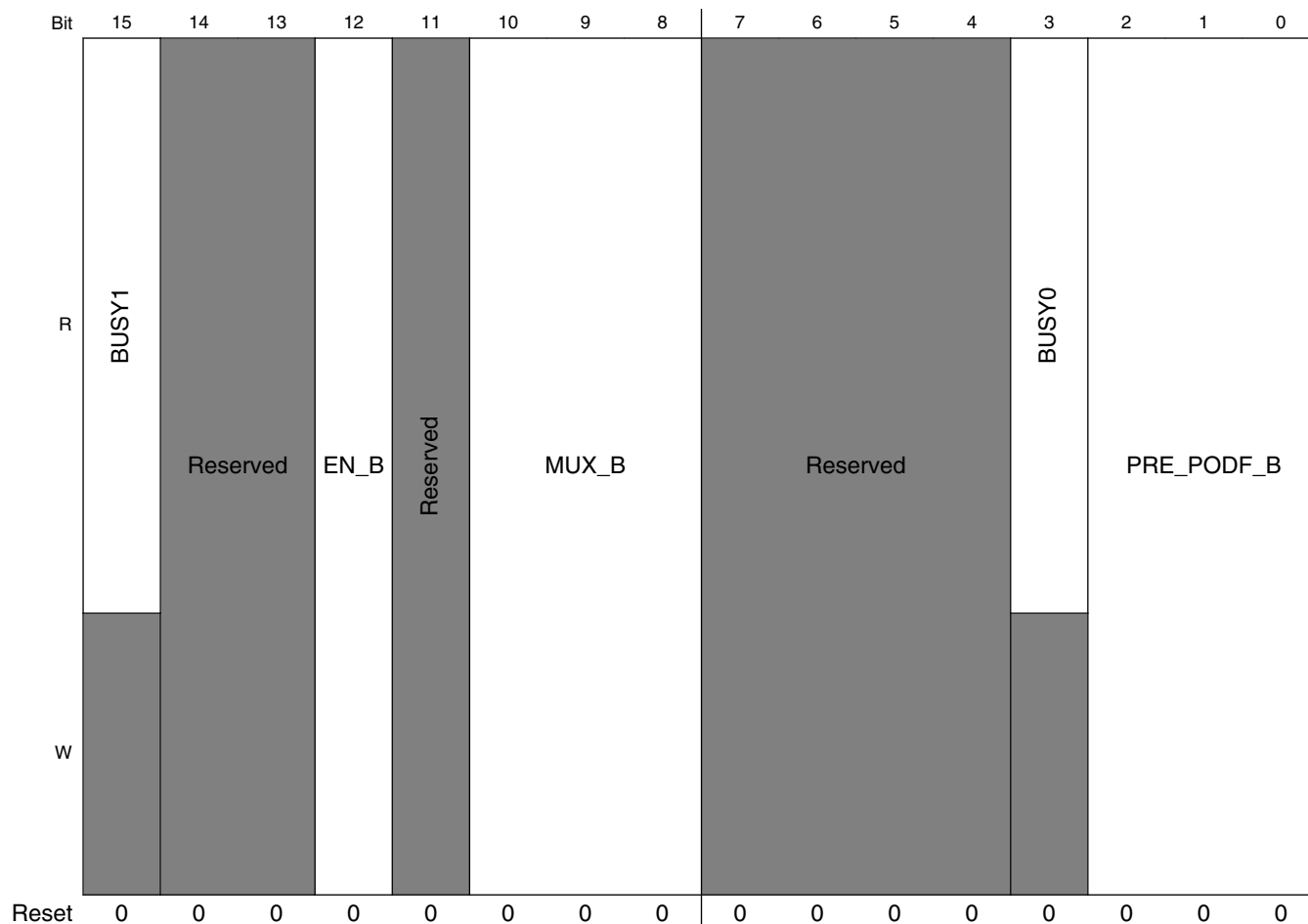
5.1.7.23 Pre Divider Register (CCM_PRE_ROOT n _SET)

Pre Divider Register

Address: 3038_0000h base + 8034h offset + (128d × i), where i=0d to 141d



Clock Control Module (CCM)



CCM_PRE_ROOTn_SET field descriptions

Field	Description
31 BUSY4	EN_A field is applied to field This field applies to DRAM and DRAM_PHYM
30–29 -	This field is reserved. Reserved
28 EN_A	Branch A clock gate control This field applies to DRAM and DRAM_PHYM 0 Clock shutdown 1 clock ON
27 -	This field is reserved. Reserved
26–24 MUX_A	Selection control of multiplexer of branch A This field applies to DRAM and DRAM_PHYM
23–20 -	This field is reserved. Reserved
19 BUSY3	Pre divider value for branch A is applied This field applies to DRAM and DRAM_PHYM

Table continues on the next page...

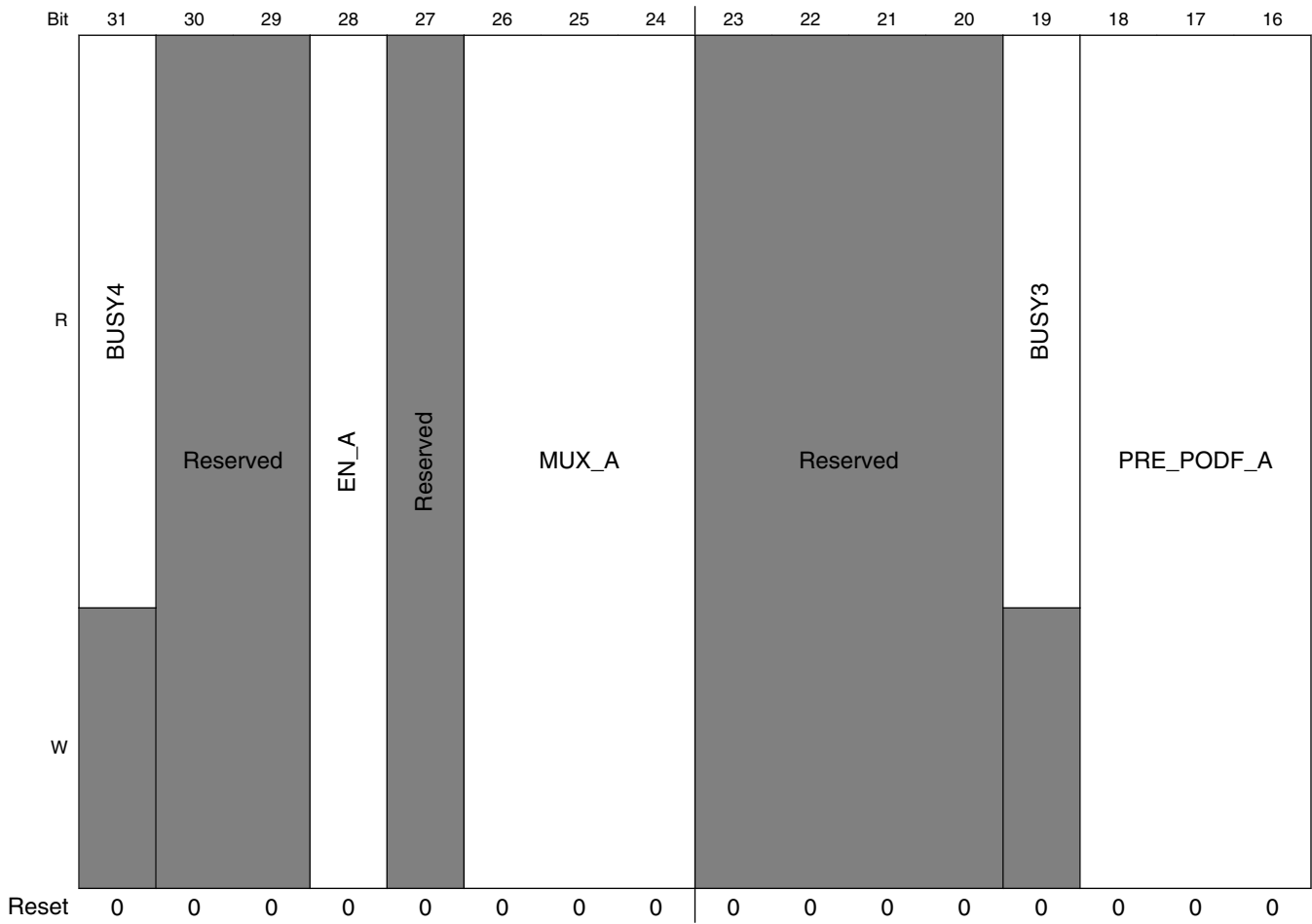
CCM_PRE_ROOTn_SET field descriptions (continued)

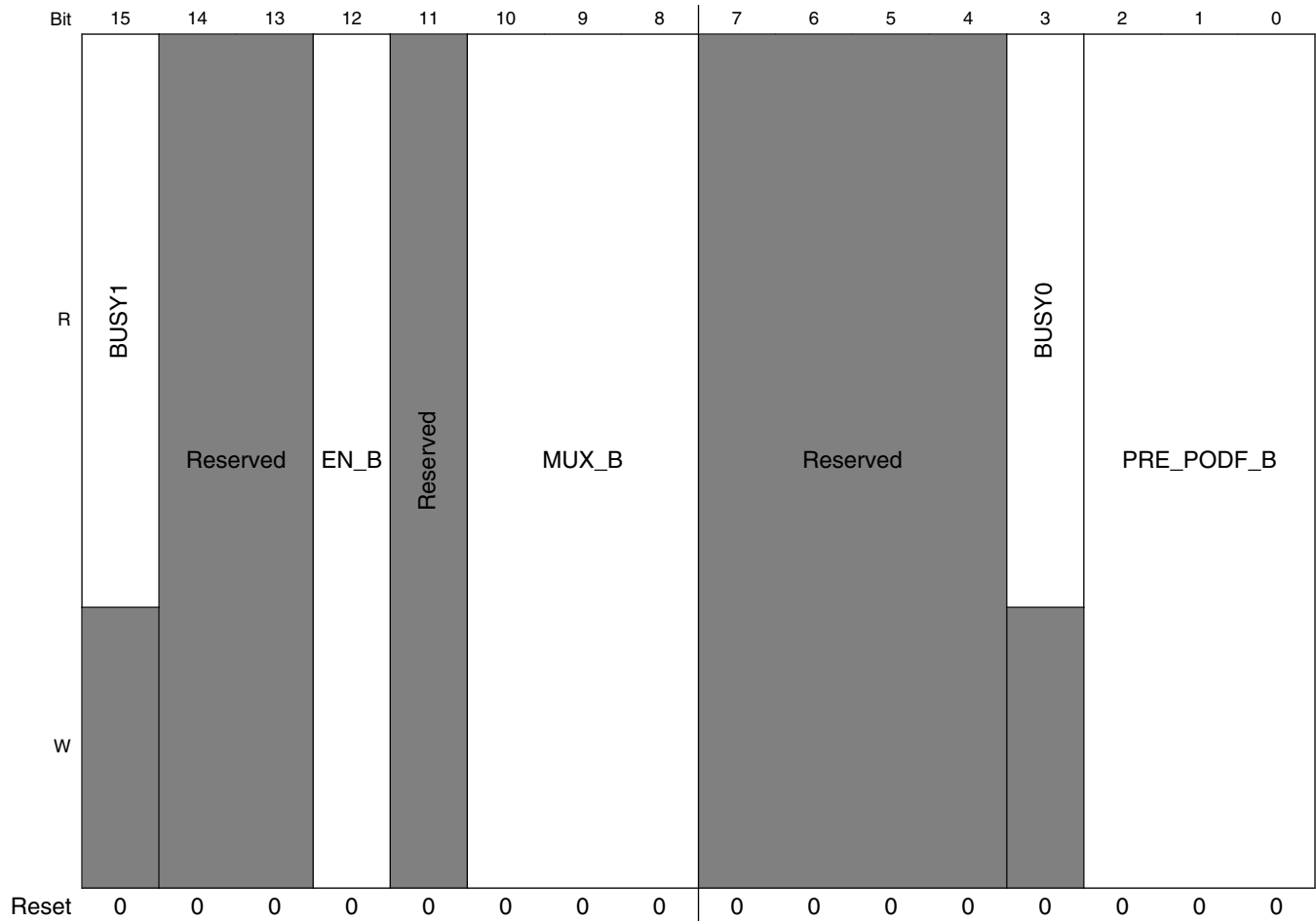
Field	Description
18–16 PRE_PODF_A	Pre divider divide number for branch A Divider value is $n + 1$. This field does not apply for CORE, DRAM, DRAM_PHYM 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
15 BUSY1	EN_B is applied to field This field does not apply for CORE, IP, DRAM, DRAM_PHYM
14–13 -	This field is reserved. Reserved
12 EN_B	Branch B clock gate control This field does not apply for CORE, IP, DRAM, DRAM_PHYM 0 Clock shutdown 1 Clock ON
11 -	This field is reserved. Reserved
10–8 MUX_B	Selection control of multiplexer of branch B This field does not apply for CORE, IP, DRAM, DRAM_PHYM
7–4 -	This field is reserved. Reserved
3 BUSY0	Pre divider value for branch A is applying field does not apply for CORE, IP, DRAM, DRAM_PHYM
PRE_PODF_B	Pre divider divide number for branch B Divider value is $n + 1$. This field does not apply for CORE, IP, DRAM, DRAM_PHYM 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8

5.1.7.24 Pre Divider Register (CCM_PRE_ROOTn_CLR)

Pre Root Register

Address: 3038_0000h base + 8038h offset + (128d × i), where i=0d to 141d





CCM_PRE_ROOTn_CLR field descriptions

Field	Description
31 BUSY4	EN_A field is applied to field This field applies to DRAM and DRAM_PHYM
30–29 -	This field is reserved. Reserved
28 EN_A	Branch A clock gate control This field applies to DRAM and DRAM_PHYM 0 Clock shutdown 1 clock ON
27 -	This field is reserved. Reserved
26–24 MUX_A	Selection control of multiplexer of branch A This field applies to DRAM and DRAM_PHYM
23–20 -	This field is reserved. Reserved
19 BUSY3	Pre divider value for branch A is applied This field applies to DRAM and DRAM_PHYM

Table continues on the next page...

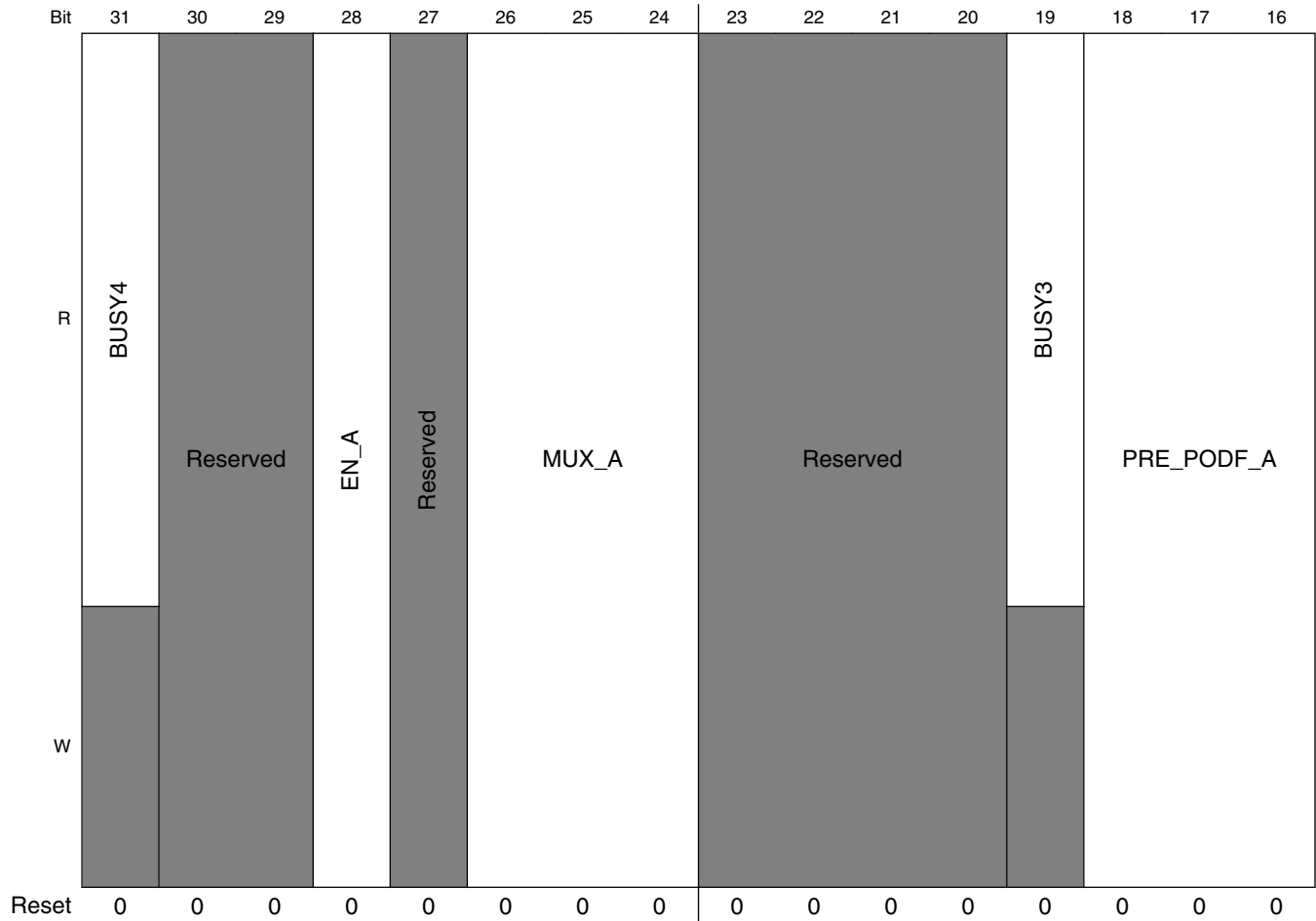
CCM_PRE_ROOT n _CLR field descriptions (continued)

Field	Description
18–16 PRE_PODF_A	Pre divider divide number for branch A Divider value is $n + 1$. This field does not apply for CORE, DRAM, DRAM_PHYM 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
15 BUSY1	EN_B is applied to field This field does not apply for CORE, IP, DRAM, DRAM_PHYM
14–13 -	This field is reserved. Reserved
12 EN_B	Branch B clock gate control This field does not apply for CORE, IP, DRAM, DRAM_PHYM 0 Clock shutdown 1 Clock ON
11 -	This field is reserved. Reserved
10–8 MUX_B	Selection control of multiplexer of branch B This field does not apply for CORE, IP, DRAM, DRAM_PHYM
7–4 -	This field is reserved. Reserved
3 BUSY0	Pre divider value for branch A is applied This field does not apply for CORE, IP, DRAM, DRAM_PHYM
PRE_PODF_B	Pre divider divide number for branch B Divider value is $n + 1$. This field does not apply for CORE, IP, DRAM, DRAM_PHYM 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8

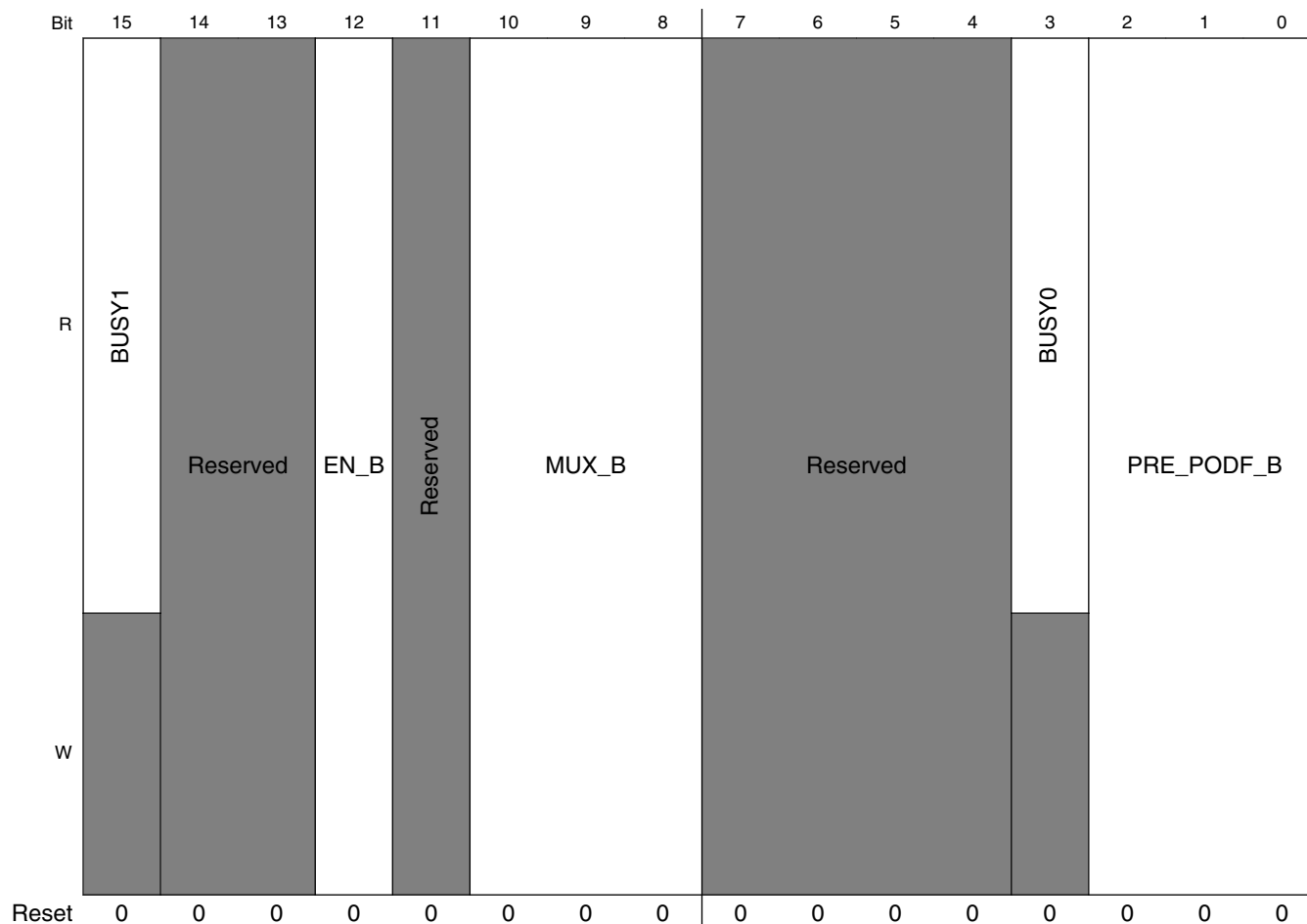
5.1.7.25 Pre Divider Register (CCM_PRE_ROOTn_TOG)

Pre Root Register

Address: 3038_0000h base + 803Ch offset + (128d × i), where i=0d to 141d



Clock Control Module (CCM)



CCM_PRE_ROOT_n_TOG field descriptions

Field	Description
31 BUSY4	EN_A field is applied to field This field applies to DRAM and DRAM_PHYM
30–29 -	This field is reserved. Reserved
28 EN_A	Branch A clock gate control This field applies to DRAM and DRAM_PHYM 0 Clock shutdown 1 clock ON
27 -	This field is reserved. Reserved
26–24 MUX_A	Selection control of multiplexer of branch A This field applies to DRAM and DRAM_PHYM
23–20 -	This field is reserved. Reserved
19 BUSY3	Pre divider value for branch A is applied This field applies to DRAM and DRAM_PHYM

Table continues on the next page...

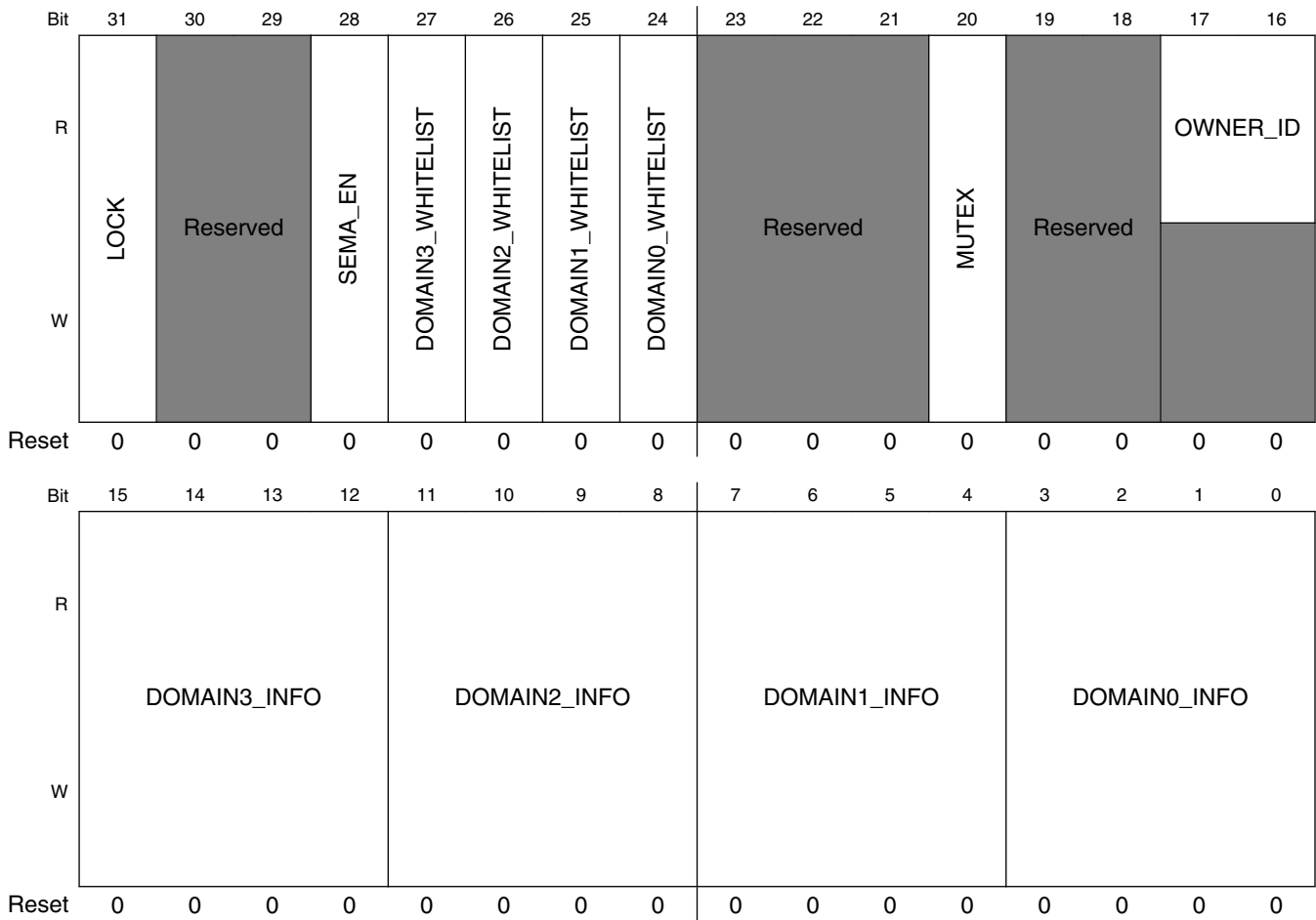
CCM_PRE_ROOT n _TOG field descriptions (continued)

Field	Description
18–16 PRE_PODF_A	Pre divider divide number for branch A Divider value is $n + 1$. This field does not apply for CORE, DRAM, DRAM_PHYM 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
15 BUSY1	EN_B is applied to field This field does not apply for CORE, IP, DRAM, DRAM_PHYM
14–13 -	This field is reserved. Reserved
12 EN_B	Branch B clock gate control This field does not apply for CORE, IP, DRAM, DRAM_PHYM 0 Clock shutdown 1 Clock ON
11 -	This field is reserved. Reserved
10–8 MUX_B	Selection control of multiplexer of branch B This field does not apply for CORE, IP, DRAM, DRAM_PHYM
7–4 -	This field is reserved. Reserved
3 BUSY0	Pre divider value for branch a is applied field does not apply for CORE, IP, DRAM, DRAM_PHYM
PRE_PODF_B	Pre divider divide number for branch B Divider value is $n + 1$. This field does not apply for CORE, IP, DRAM, DRAM_PHYM 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8

5.1.7.26 Access Control Register (CCM_ACCESS_CTRLn)

Access Control Register

Address: 3038_0000h base + 8070h offset + (128d × i), where i=0d to 141d



CCM_ACCESS_CTRLn field descriptions

Field	Description
31 LOCK	Lock this clock root to use access control This bit can be set to 1 by software, and can be cleared only by system reset. 0 Access control inactive 1 Access control active
30–29 -	This field is reserved. Reserved
28 SEMA_EN	Enable internal semaphore This field cannot be changed when lock bit is 1

Table continues on the next page...

CCM_ACCESS_CTRLn field descriptions (continued)

Field	Description
	0 Disable 1 Enable
27 DOMAIN3_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
26 DOMAIN2_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
25 DOMAIN1_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
24 DOMAIN0_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
23–21 -	This field is reserved. Reserved
20 MUTEX	Semaphore to control access 0 Semaphore is free to take 1 Semaphore is taken Write 0 Release semaphore Write 1 Acquire semaphore
19–18 -	This field is reserved. Reserved
17–16 OWNER_ID	Current domain that owns semaphore This field is meaningless when MUTEX is 0 0 domain0 1 domain1 2 domain2 3 domain3
15–12 DOMAIN3_INFO	Information from domain 3 to pass to others This field can only be changed by domain 3
11–8 DOMAIN2_INFO	Information from domain 2 to pass to others This field can only be changed by domain 2
7–4 DOMAIN1_INFO	Information from domain 1 to pass to others This field can only be changed by domain 1

Table continues on the next page...

CCM_ACCESS_CTRL n field descriptions (continued)

Field	Description
DOMAIN0_INFO	Information from domain 0 to pass to others This field can only be changed by domain 0

5.1.7.27 Access Control Register (CCM_ACCESS_CTRL_ROOT n _SET)

Access Control Register

Address: 3038_0000h base + 8074h offset + (128d × i), where i=0d to 141d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	LOCK	Reserved			SEMA_EN	DOMAIN3_WHITELIST	DOMAIN2_WHITELIST	DOMAIN1_WHITELIST	DOMAIN0_WHITELIST	Reserved			MUTEX	Reserved		OWNER_ID
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DOMAIN3_INFO				DOMAIN2_INFO				DOMAIN1_INFO				DOMAIN0_INFO			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

CCM_ACCESS_CTRL_ROOT n _SET field descriptions

Field	Description
31 LOCK	Lock this clock root to use access control This bit can be set to 1 by software, and can be cleared only by system reset.

Table continues on the next page...

CCM_ACCESS_CTRL_ROOT_n_SET field descriptions (continued)

Field	Description
	0 Access control inactive 1 Access control active
30–29 -	This field is reserved. Reserved
28 SEMA_EN	Enable internal semaphore This field cannot be changed when lock bit is 1 0 Disable 1 Enable
27 DOMAIN3_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
26 DOMAIN2_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
25 DOMAIN1_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
24 DOMAIN0_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
23–21 -	This field is reserved. Reserved
20 MUTEX	Semaphore to control access 0 Semaphore is free to take 1 Semaphore is taken Write 0 Release semaphore Write 1 Acquire semaphore
19–18 -	This field is reserved. Reserved
17–16 OWNER_ID	Current domain that owns semaphore This field is meaningless when MUTEX is 0 0 domain0 1 domain1 2 domain2 3 domain3

Table continues on the next page...

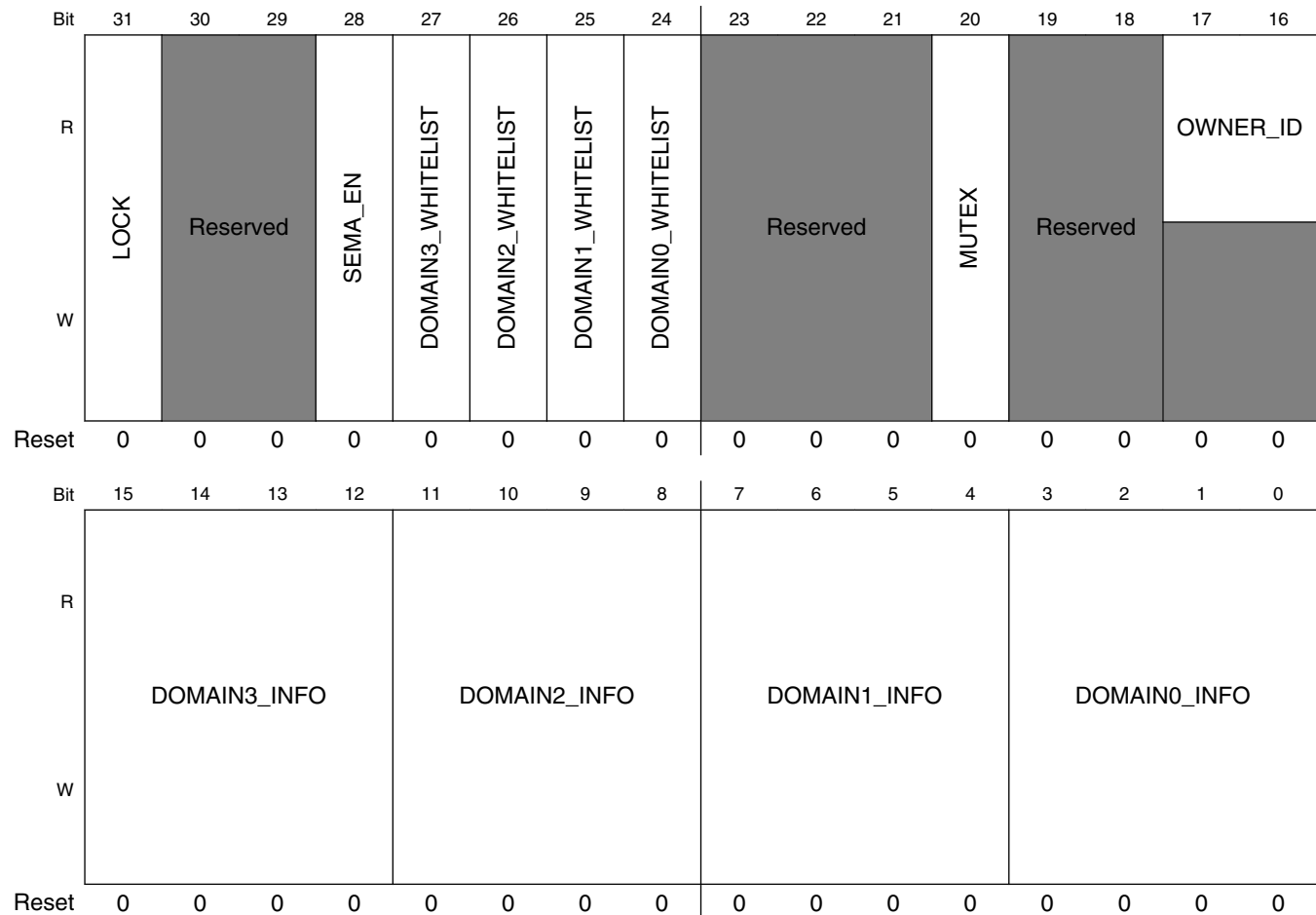
CCM_ACCESS_CTRL_ROOT n _SET field descriptions (continued)

Field	Description
15–12 DOMAIN3_INFO	Information from domain 3 to pass to others This field can only be changed by domain 3
11–8 DOMAIN2_INFO	Information from domain 2 to pass to others This field can only be changed by domain 2
7–4 DOMAIN1_INFO	Information from domain 1 to pass to others This field can only be changed by domain 1
DOMAIN0_INFO	Information from domain 0 to pass to others This field can only be changed by domain 0

5.1.7.28 Access Control Register (CCM_ACCESS_CTRL_ROOTn_CLR)

Access Control Register

Address: 3038_0000h base + 8078h offset + (128d × i), where i=0d to 141d



CCM_ACCESS_CTRL_ROOTn_CLR field descriptions

Field	Description
31 LOCK	Lock this clock root to use access control This bit can be set to 1 by software, and can be cleared only by system reset. 0 Access control inactive 1 Access control active
30–29 -	This field is reserved. Reserved

Table continues on the next page...

CCM_ACCESS_CTRL_ROOT n _CLR field descriptions (continued)

Field	Description
28 SEMA_EN	Enable internal semaphore This field cannot be changed when lock bit is 1 0 Disable 1 Enable
27 DOMAIN3_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
26 DOMAIN2_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
25 DOMAIN1_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
24 DOMAIN0_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
23–21 -	This field is reserved. Reserved
20 MUTEX	Semaphore to control access 0 Semaphore is free to take 1 Semaphore is taken Write 0 Release semaphore Write 1 Acquire semaphore
19–18 -	This field is reserved. Reserved
17–16 OWNER_ID	Current domain that owns semaphore This field is meaningless when MUTEX is 0 0 domain0 1 domain1 2 domain2 3 domain3
15–12 DOMAIN3_INFO	Information from domain 3 to pass to others This field can only be changed by domain 3
11–8 DOMAIN2_INFO	Information from domain 2 to pass to others This field can only be changed by domain 2

Table continues on the next page...

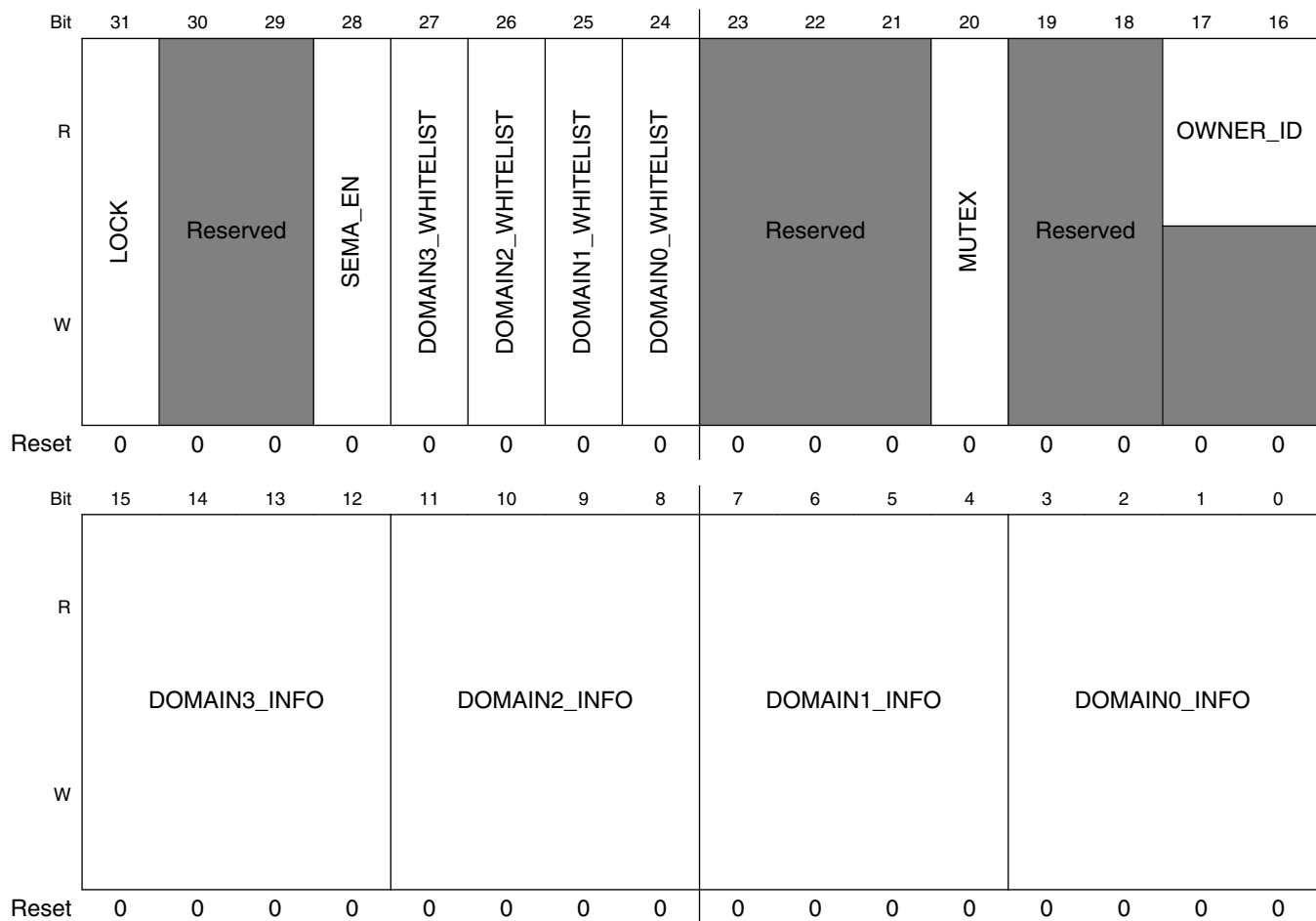
CCM_ACCESS_CTRL_ROOT_n_CLR field descriptions (continued)

Field	Description
7–4 DOMAIN1_INFO	Information from domain 1 to pass to others This field can only be changed by domain 1
DOMAIN0_INFO	Information from domain 0 to pass to others This field can only be changed by domain 0

5.1.7.29 Access Control Register (CCM_ACCESS_CTRL_ROOT_n_TOG)

Access Control Register

Address: 3038_0000h base + 807Ch offset + (128d × i), where i=0d to 141d



CCM_ACCESS_CTRL_ROOT_n_TOG field descriptions

Field	Description
31 LOCK	Lock this clock root to use access control This bit can be set to 1 by software, and can be cleared only by system reset. 0 Access control inactive 1 Access control active
30–29 -	This field is reserved. Reserved
28 SEMA_EN	Enable internal semaphore This field cannot be changed when lock bit is 1 0 Disable 1 Enable
27 DOMAIN3_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
26 DOMAIN2_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
25 DOMAIN1_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
24 DOMAIN0_ WHITELIST	White list of domains that can change setting of this clock root. Each domain has a corresponding bit. 0 Domain cannot change the setting 1 Domain can change the setting
23–21 -	This field is reserved. Reserved
20 MUTEX	Semaphore to control access 0 Semaphore is free to take 1 Semaphore is taken Write 0 Release semaphore Write 1 Acquire semaphore
19–18 -	This field is reserved. Reserved
17–16 OWNER_ID	Current domain that owns semaphore This field is meaningless when MUTEX is 0 0 domain0 1 domain1

Table continues on the next page...

CCM_ACCESS_CTRL_ROOTn_TOG field descriptions (continued)

Field	Description
	2 domain2 3 domain3
15–12 DOMAIN3_INFO	Information from domain 3 to pass to others This field can only be changed by domain 3
11–8 DOMAIN2_INFO	Information from domain 2 to pass to others This field can only be changed by domain 2
7–4 DOMAIN1_INFO	Information from domain 1 to pass to others This field can only be changed by domain 1
DOMAIN0_INFO	Information from domain 0 to pass to others This field can only be changed by domain 0

5.1.8 CCM Analog Memory Map/Register Definition**CCM_ANALOG memory map**

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3036_0000	AUDIO PLL1 Configuration 0 Register (CCM_ANALOG_AUDIO_PLL1_CFG0)	32	R/W	0020_C081h	5.1.8.1/603
3036_0004	AUDIO PLL1 Configuration 1 Register (CCM_ANALOG_AUDIO_PLL1_CFG1)	32	R/W	0000_0040h	5.1.8.2/606
3036_0008	AUDIO PLL2 Configuration 0 Register (CCM_ANALOG_AUDIO_PLL2_CFG0)	32	R/W	0020_C081h	5.1.8.3/607
3036_000C	AUDIO PLL2 Configuration 1 Register (CCM_ANALOG_AUDIO_PLL2_CFG1)	32	R/W	0000_0040h	5.1.8.4/610
3036_0010	VIDEO PLL Configuration 0 Register (CCM_ANALOG_VIDEO_PLL1_CFG0)	32	R/W	0020_C081h	5.1.8.5/611
3036_0014	VIDEO PLL Configuration 1 Register (CCM_ANALOG_VIDEO_PLL1_CFG1)	32	R/W	0000_0040h	5.1.8.6/614
3036_0018	GPU PLL Configuration 0 Register (CCM_ANALOG_GPU_PLL_CFG0)	32	R/W	0020_C081h	5.1.8.7/615
3036_001C	GPU PLL Configuration 1 Register (CCM_ANALOG_GPU_PLL_CFG1)	32	R/W	0000_004Fh	5.1.8.8/618
3036_0020	VPU PLL Configuration 0 Register (CCM_ANALOG_VPU_PLL_CFG0)	32	R/W	0020_C081h	5.1.8.9/619
3036_0024	VPU PLL Configuration 1 Register (CCM_ANALOG_VPU_PLL_CFG1)	32	R/W	0000_003Bh	5.1.8.10/622
3036_0028	ARM PLL Configuration 0 Register (CCM_ANALOG_ARM_PLL_CFG0)	32	R/W	0020_C080h	5.1.8.11/623
3036_002C	ARM PLL Configuration 1 Register (CCM_ANALOG_ARM_PLL_CFG1)	32	R/W	0000_004Fh	5.1.8.12/626

Table continues on the next page...

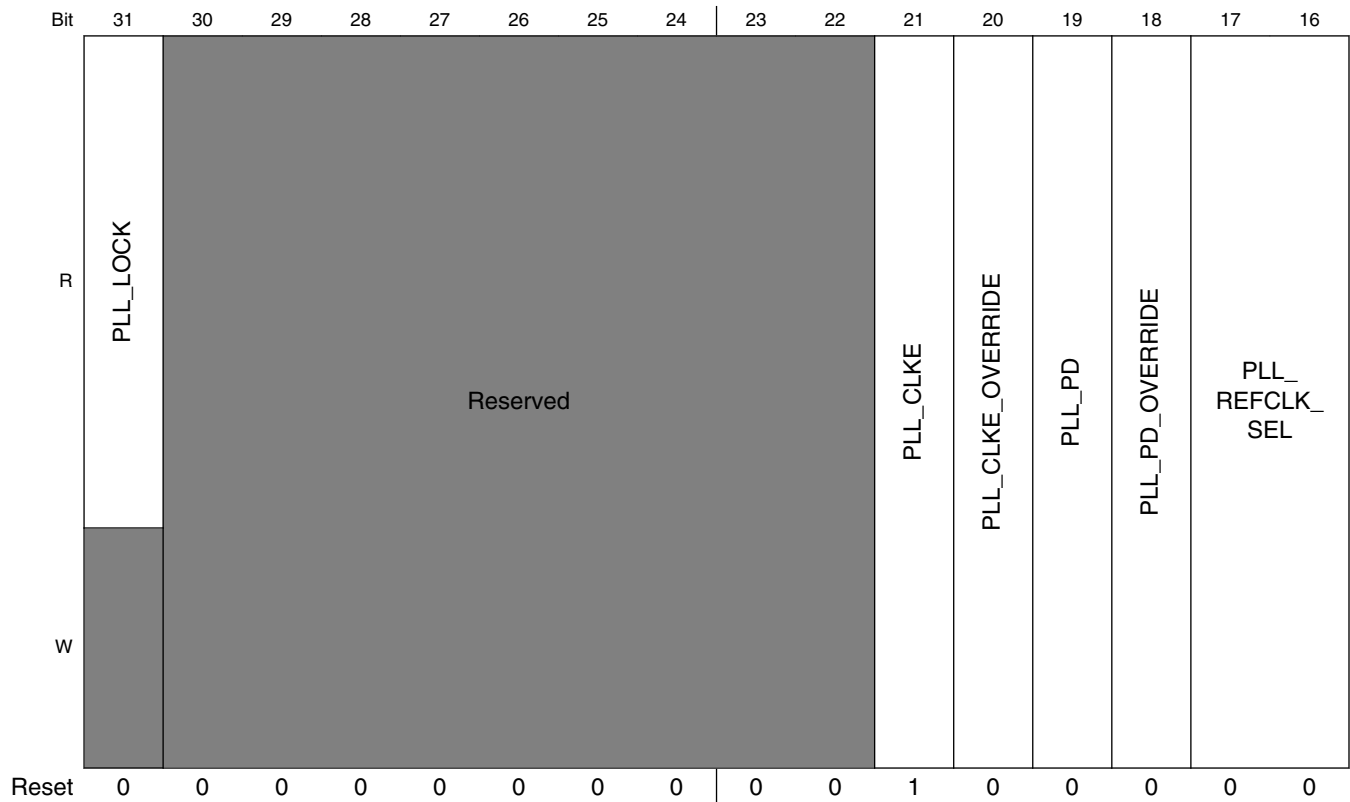
CCM_ANALOG memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3036_0030	System PLL Configuration 0 Register (CCM_ANALOG_SYS_PLL1_CFG0)	32	R/W	02AA_AA38h	5.1.8.13/627
3036_0034	System_PLL Configuration 1 Register (CCM_ANALOG_SYS_PLL1_CFG1)	32	R/W	0000_0000h	5.1.8.14/630
3036_0038	System_PLL Configuration 2 Register (CCM_ANALOG_SYS_PLL1_CFG2)	32	R/W	00BB_E580h	5.1.8.15/631
3036_003C	System PLL Configuration 0 Register (CCM_ANALOG_SYS_PLL2_CFG0)	32	R/W	02AA_AA38h	5.1.8.16/632
3036_0040	System_PLL Configuration 1 Register (CCM_ANALOG_SYS_PLL2_CFG1)	32	R/W	0000_0000h	5.1.8.17/635
3036_0044	System_PLL Configuration 2 Register (CCM_ANALOG_SYS_PLL2_CFG2)	32	R/W	00BB_E700h	5.1.8.18/636
3036_0048	System PLL Configuration 0 Register (CCM_ANALOG_SYS_PLL3_CFG0)	32	R/W	0000_0238h	5.1.8.19/637
3036_004C	System_PLL Configuration 1 Register (CCM_ANALOG_SYS_PLL3_CFG1)	32	R/W	0000_0000h	5.1.8.20/639
3036_0050	System_PLL Configuration 2 Register (CCM_ANALOG_SYS_PLL3_CFG2)	32	R/W	00BB_E700h	5.1.8.21/640
3036_0054	VIDEO PLL2 Configuration 0 Register (CCM_ANALOG_VIDEO_PLL2_CFG0)	32	R/W	0000_0238h	5.1.8.22/641
3036_0058	VIDEO PLL2 Configuration 1 Register (CCM_ANALOG_VIDEO_PLL2_CFG1)	32	R/W	0000_0000h	5.1.8.23/643
3036_005C	VIDEO PLL2 Configuration 2 Register (CCM_ANALOG_VIDEO_PLL2_CFG2)	32	R/W	00EC_6480h	5.1.8.24/644
3036_0060	DRAM PLL Configuration 0 Register (CCM_ANALOG_DRAM_PLL_CFG0)	32	R/W	0000_0238h	5.1.8.25/645
3036_0064	DRAM PLL Configuration 1 Register (CCM_ANALOG_DRAM_PLL_CFG1)	32	R/W	0000_0000h	5.1.8.26/647
3036_0068	DRAM PLL Configuration 2 Register (CCM_ANALOG_DRAM_PLL_CFG2)	32	R/W	00BB_E580h	5.1.8.27/648
3036_006C	DIGPROG Register (CCM_ANALOG_DIGPROG)	32	R	0082_4010h	5.1.8.28/649
3036_0070	Osc Misc Configuration Register (CCM_ANALOG_OSC_MISC_CFG)	32	R/W	0000_0014h	5.1.8.29/650
3036_0074	PLLOUT Monitor Configuration Register (CCM_ANALOG_PLLOUT_MONITOR_CFG)	32	R/W	0000_0000h	5.1.8.30/651
3036_0078	Fractional PLLOUT Divider Configuration Register (CCM_ANALOG_FRAC_PLLOUT_DIV_CFG)	32	R/W	0000_0000h	5.1.8.31/652
3036_007C	SCCG PLLOUT Divider Configuration Register (CCM_ANALOG_SCCG_PLLOUT_DIV_CFG)	32	R/W	0000_0000h	5.1.8.32/655

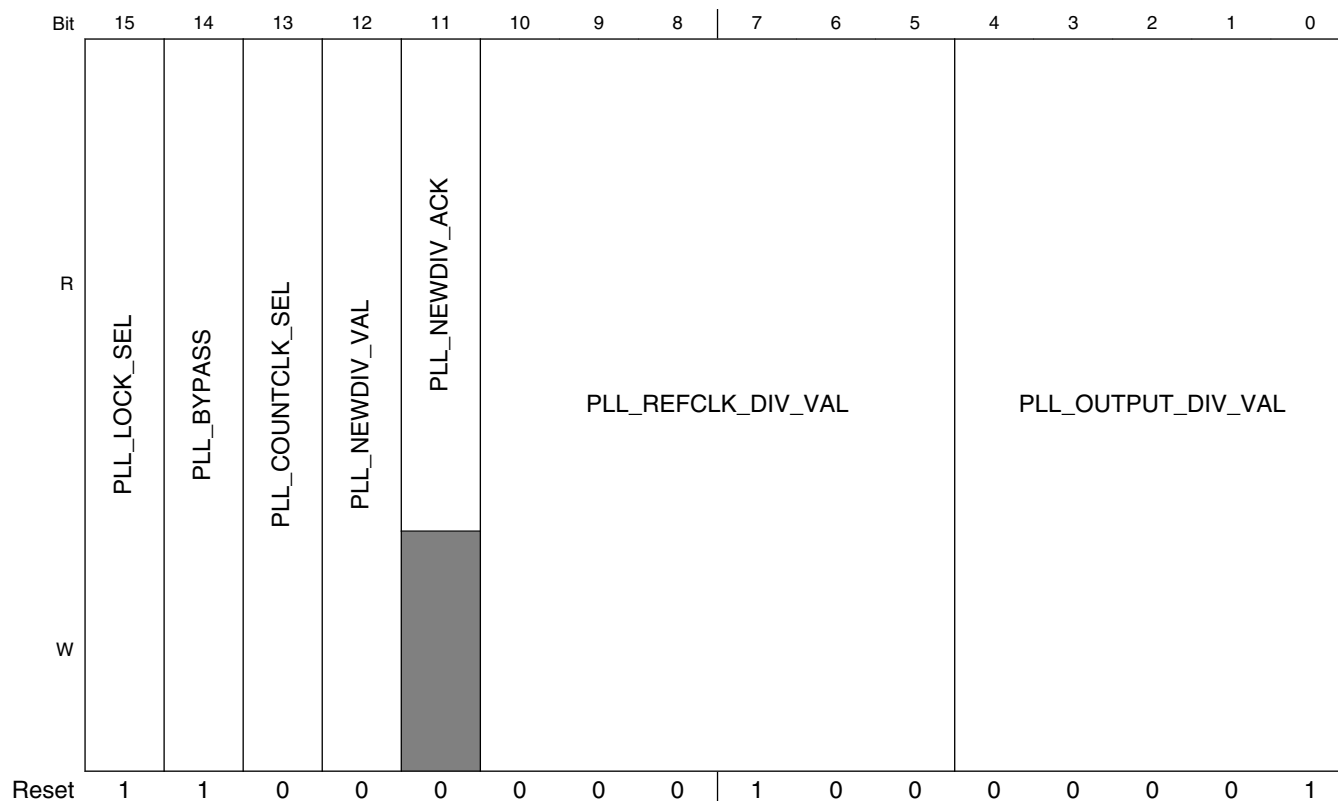
5.1.8.1 AUDIO PLL1 Configuration 0 Register (CCM_ANALOG_AUDIO_PLL1_CFG0)

AUDIO PLL1 Configuration 0 Register

Address: 3036_0000h base + 0h offset = 3036_0000h



Clock Control Module (CCM)



CCM_ANALOG_AUDIO_PLL1_CFG0 field descriptions

Field	Description
31 PLL_LOCK	PLL lock status active high
30–22 -	This field is reserved. Reserved
21 PLL_CLKE	PLL output clock clock gating enable active high
20 PLL_CLKE_OVERRIDE	Override the PLL_CLKE, clock gating enable signal from CCM block active high
19 PLL_PD	PLL output clock clock gating enable active high
18 PLL_PD_OVERRIDE	Override the PLL_PD, clock gating enable signal from CCM block active high
17–16 PLL_REFCLK_SEL	PLL reference clocks select 00 25M_REF_CLK 01 27M_REF_CLK 10 HDMI_PHY_27M_CLK 11 CLK_P_N
15 PLL_LOCK_SEL	PLL Lock signal select

Table continues on the next page...

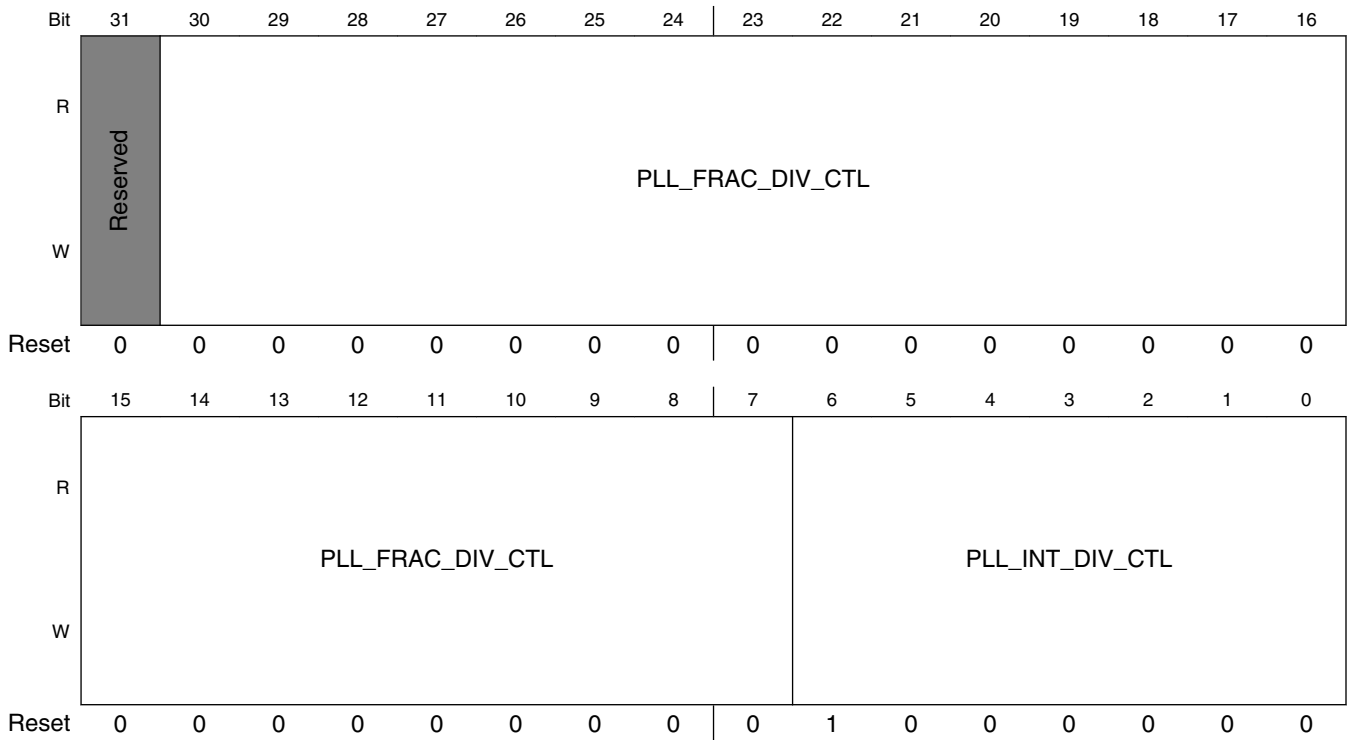
CCM_ANALOG_AUDIO_PLL1_CFG0 field descriptions (continued)

Field	Description
	00 Select PLL lock output 01 Select maximum lock time counter output
14 PLL_BYPASS	PLL bypass control active high
13 PLL_COUNTCLK_SEL	PLL maximum lock time counter clock select 0 25M_REF_CLK 1 27M_REF_CLK
12 PLL_NEWDIV_VAL	PLL new fraction divide input control active high
11 PLL_NEWDIV_ACK	PLL new fraction divide handshake signal
10–5 PLL_REFCLK_DIV_VAL	PLL reference clock divide value. Both REF and post_divide REF must be within the range 10MHz to 300MHz. Valid divider values are 1 to 64
PLL_OUTPUT_DIV_VAL	PLL output clock divide value Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz Valid divider values are even divides from 2 to 64 (binary value + 1) * 2 00000 - 2 00001 - 4 00010 - 6 . . . 11111 - 64

5.1.8.2 AUDIO PLL1 Configuration 1 Register
(CCM_ANALOG_AUDIO_PLL1_CFG1)

AUDIO PLL1 Configuration 1 Register

Address: 3036_0000h base + 4h offset = 3036_0004h



CCM_ANALOG_AUDIO_PLL1_CFG1 field descriptions

Field	Description
31 -	This field is reserved. Reserved
30–7 PLL_FRAC_DIV_CTL	PLL fraction divide control Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz Valid divider values are 1 to 2^24
PLL_INT_DIV_CTL	PLL Integer divide control Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz. Valid divider values are 1 to 32

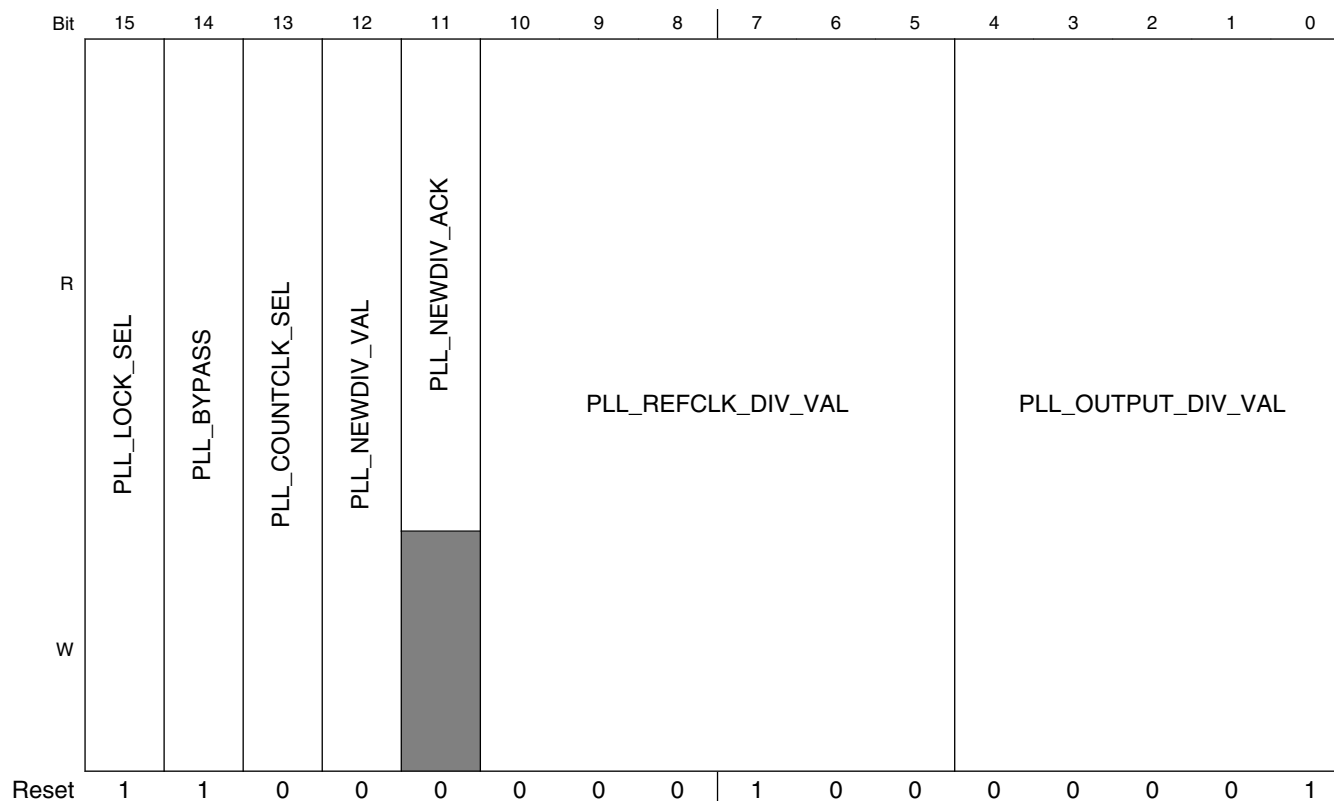
5.1.8.3 AUDIO PLL2 Configuration 0 Register (CCM_ANALOG_AUDIO_PLL2_CFG0)

AUDIO PLL2 Configuration 0 Register

Address: 3036_0000h base + 8h offset = 3036_0008h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PLL_LOCK	Reserved														
W											PLL_CLKE	PLL_CLKE_OVERRIDE	PLL_PD	PLL_PD_OVERRIDE	PLL_REFCLK_SEL	
Reset	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0

Clock Control Module (CCM)



CCM_ANALOG_AUDIO_PLL2_CFG0 field descriptions

Field	Description
31 PLL_LOCK	PLL lock status active high
30–22 -	This field is reserved. Reserved
21 PLL_CLKE	PLL output clock clock gating enable active high
20 PLL_CLKE_OVERRIDE	Override the PLL_CLKE, clock gating enable signal from CCM block active high
19 PLL_PD	PLL output clock clock gating enable active high
18 PLL_PD_OVERRIDE	Override the PLL_PD, clock gating enable signal from CCM block active high
17–16 PLL_REFCLK_SEL	PLL reference clocks select 00 25M_REF_CLK 01 27M_REF_CLK 10 HDMI_PHY_27M_CLK 11 CLK_P_N
15 PLL_LOCK_SEL	PLL Lock signal select

Table continues on the next page...

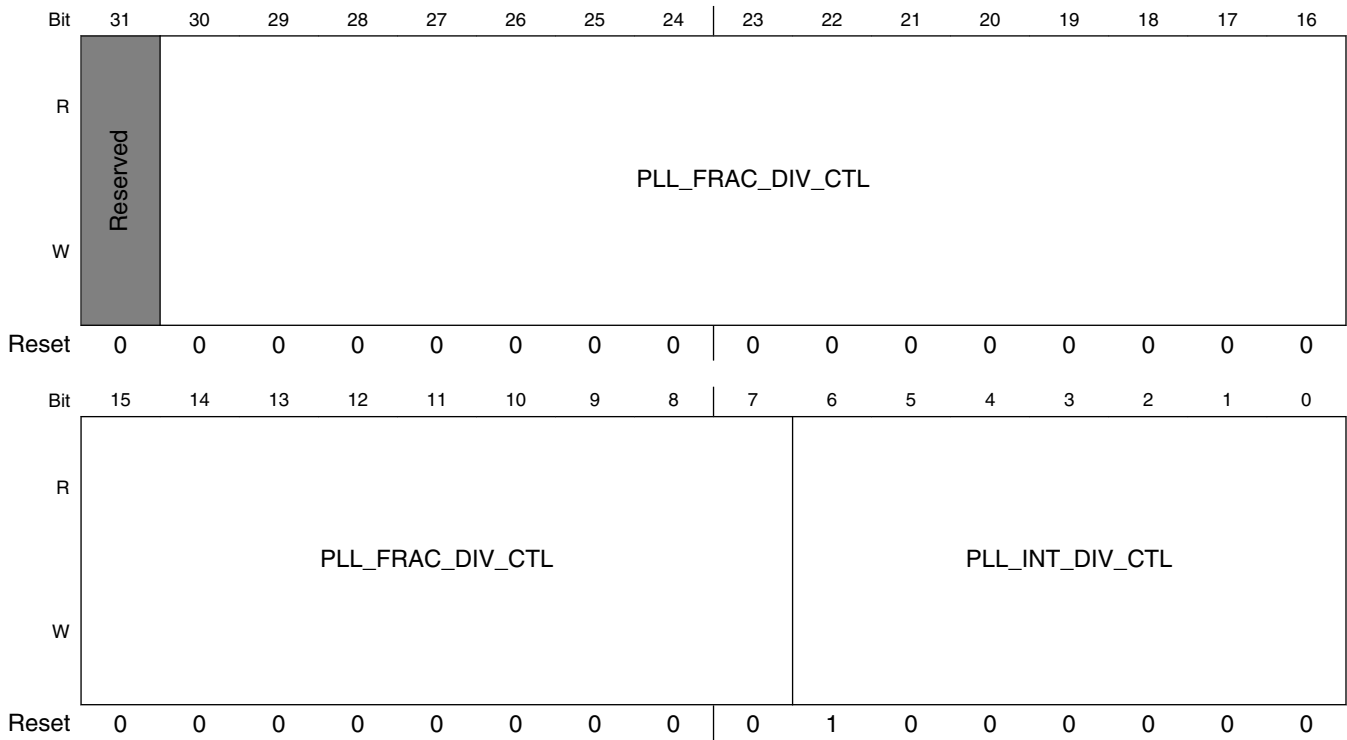
CCM_ANALOG_AUDIO_PLL2_CFG0 field descriptions (continued)

Field	Description
	00 Select PLL lock output 01 Select maximum lock time counter output
14 PLL_BYPASS	PLL bypass control active high
13 PLL_COUNTCLK_SEL	PLL maximum lock time counter clock select 0 25M_REF_CLK 1 27M_REF_CLK
12 PLL_NEWDIV_VAL	PLL new fraction divide input control active high
11 PLL_NEWDIV_ACK	PLL new fraction divide handshake signal
10–5 PLL_REFCLK_DIV_VAL	PLL reference clock divide value. Both REF and post_divide REF must be within the range 10MHz to 300MHz. Valid divider values are 1 to 64
PLL_OUTPUT_DIV_VAL	PLL output clock divide value Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz Valid divider values are even divides from 2 to 64 (binary value + 1) * 2 00000 - 2 00001 - 4 00010 - 6 . . . 11111 - 64

5.1.8.4 AUDIO PLL2 Configuration 1 Register (CCM_ANALOG_AUDIO_PLL2_CFG1)

AUDIO PLL2 Configuration 1 Register

Address: 3036_0000h base + Ch offset = 3036_000Ch



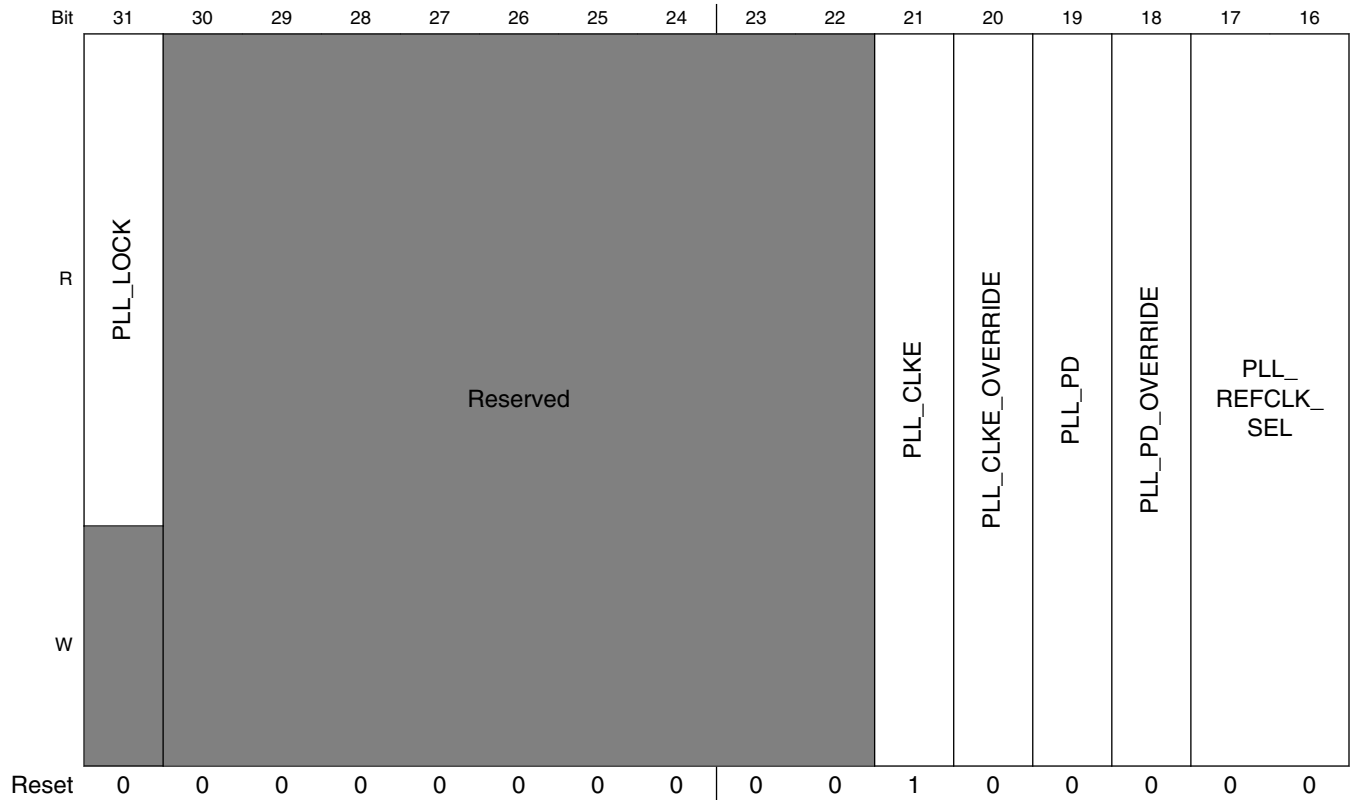
CCM_ANALOG_AUDIO_PLL2_CFG1 field descriptions

Field	Description
31 -	This field is reserved. Reserved
30-7 PLL_FRAC_DIV_CTL	PLL fraction divide control Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz Valid divider values are 1 to 2^24
PLL_INT_DIV_CTL	PLL Integer divide control Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz. Valid divider values are 1 to 32

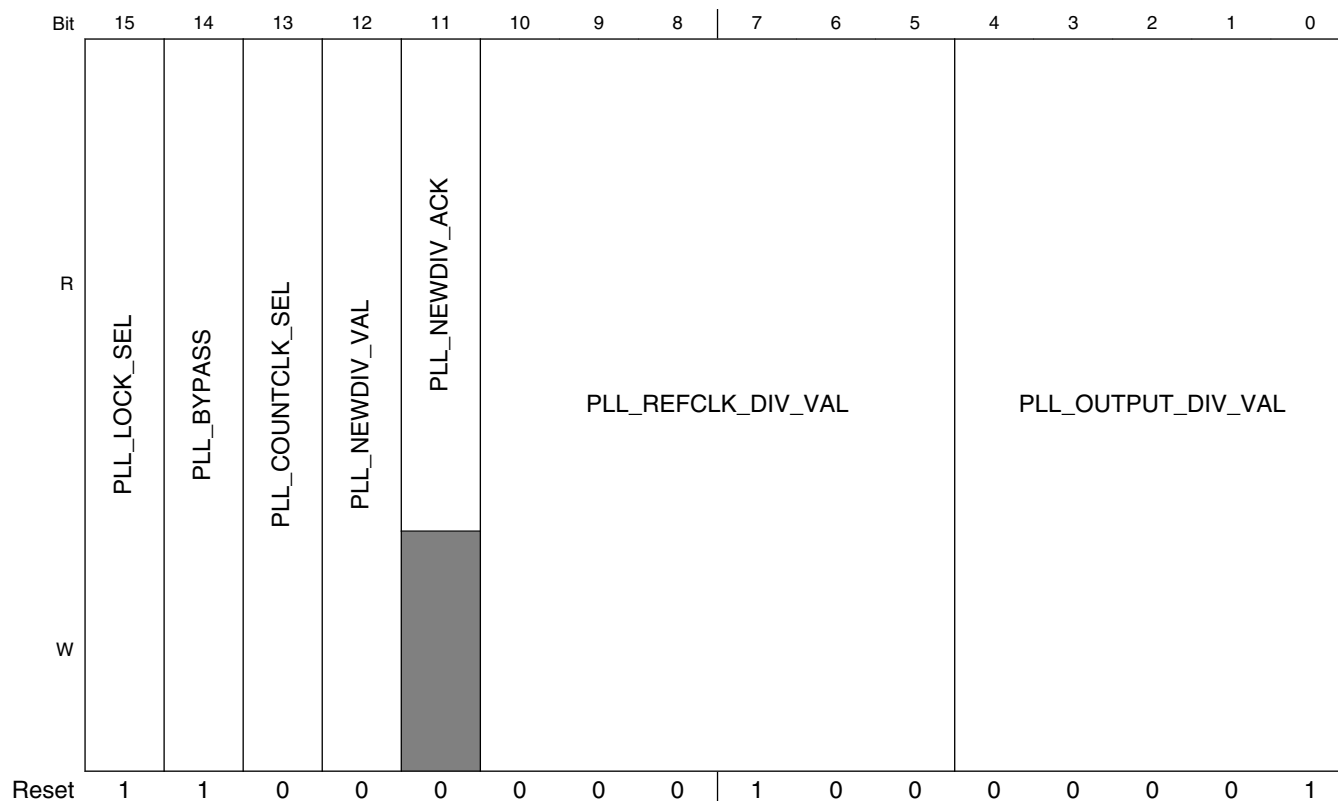
5.1.8.5 VIDEO PLL Configuration 0 Register (CCM_ANALOG_VIDEO_PLL1_CFG0)

VIDEO PLL Configuration 0 Register

Address: 3036_0000h base + 10h offset = 3036_0010h



Clock Control Module (CCM)



CCM_ANALOG_VIDEO_PLL1_CFG0 field descriptions

Field	Description
31 PLL_LOCK	PLL lock status active high
30–22 -	This field is reserved. Reserved
21 PLL_CLKE	PLL output clock clock gating enable active high
20 PLL_CLKE_OVERRIDE	Override the PLL_CLKE, clock gating enable signal from CCM block active high
19 PLL_PD	PLL output clock clock gating enable active high
18 PLL_PD_OVERRIDE	Override the PLL_PD, clock gating enable signal from CCM block active high
17–16 PLL_REFCLK_SEL	PLL reference clocks select 00 25M_REF_CLK 01 27M_REF_CLK 10 HDMI_PHY_27M_CLK 11 CLK_P_N
15 PLL_LOCK_SEL	PLL Lock signal select

Table continues on the next page...

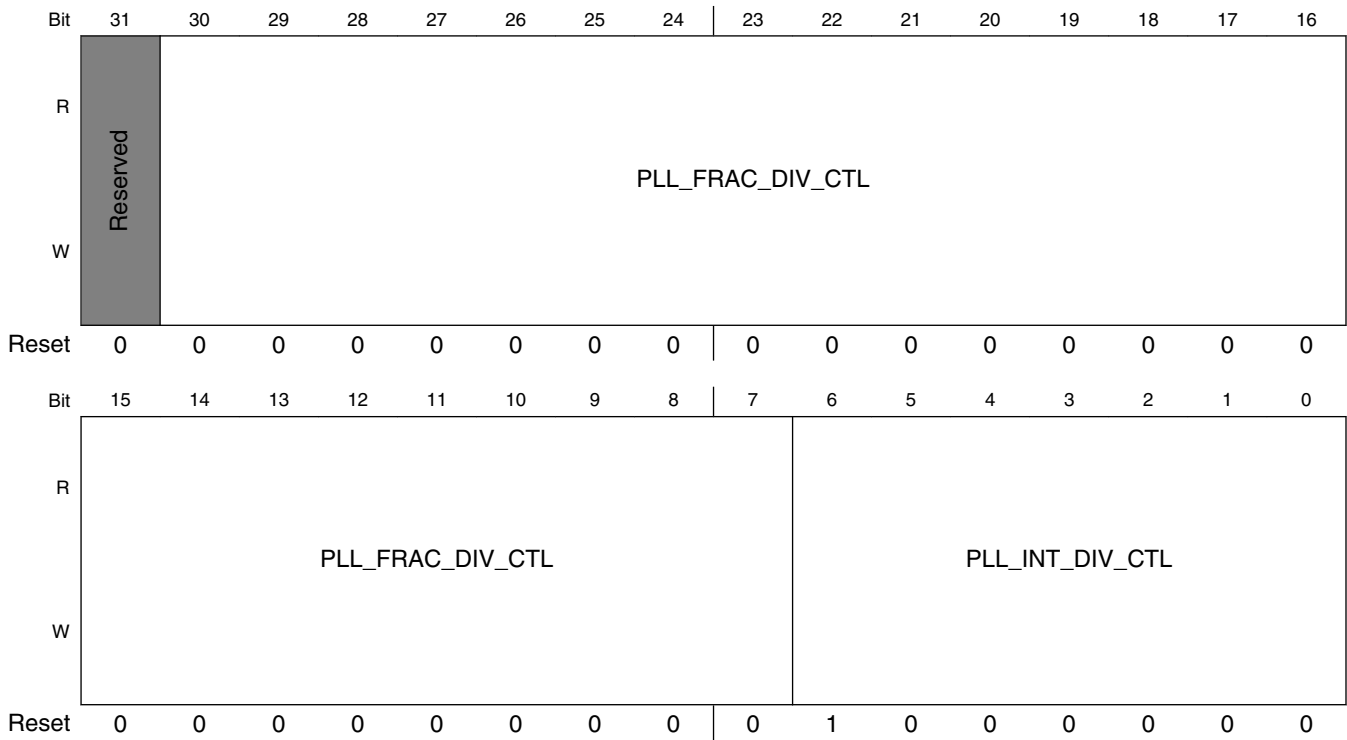
CCM_ANALOG_VIDEO_PLL1_CFG0 field descriptions (continued)

Field	Description
	00 Select PLL lock output 01 Select maximum lock time counter output
14 PLL_BYPASS	PLL bypass control active high
13 PLL_COUNTCLK_SEL	PLL maximum lock time counter clock select 0 25M_REF_CLK 1 27M_REF_CLK
12 PLL_NEWDIV_VAL	PLL new fraction divide input control active high
11 PLL_NEWDIV_ACK	PLL new fraction divide handshake signal
10–5 PLL_REFCLK_DIV_VAL	PLL reference clock divide value. Both REF and post_divide REF must be within the range 10MHz to 300MHz. Valid divider values are 1 to 64
PLL_OUTPUT_DIV_VAL	PLL output clock divide value Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz Valid divider values are even divides from 2 to 64 (binary value + 1) * 2 00000 - 2 00001 - 4 00010 - 6 . . . 11111 - 64

5.1.8.6 VIDEO PLL Configuration 1 Register
(CCM_ANALOG_VIDEO_PLL1_CFG1)

VIDEO PLL Configuration 1 Register

Address: 3036_0000h base + 14h offset = 3036_0014h



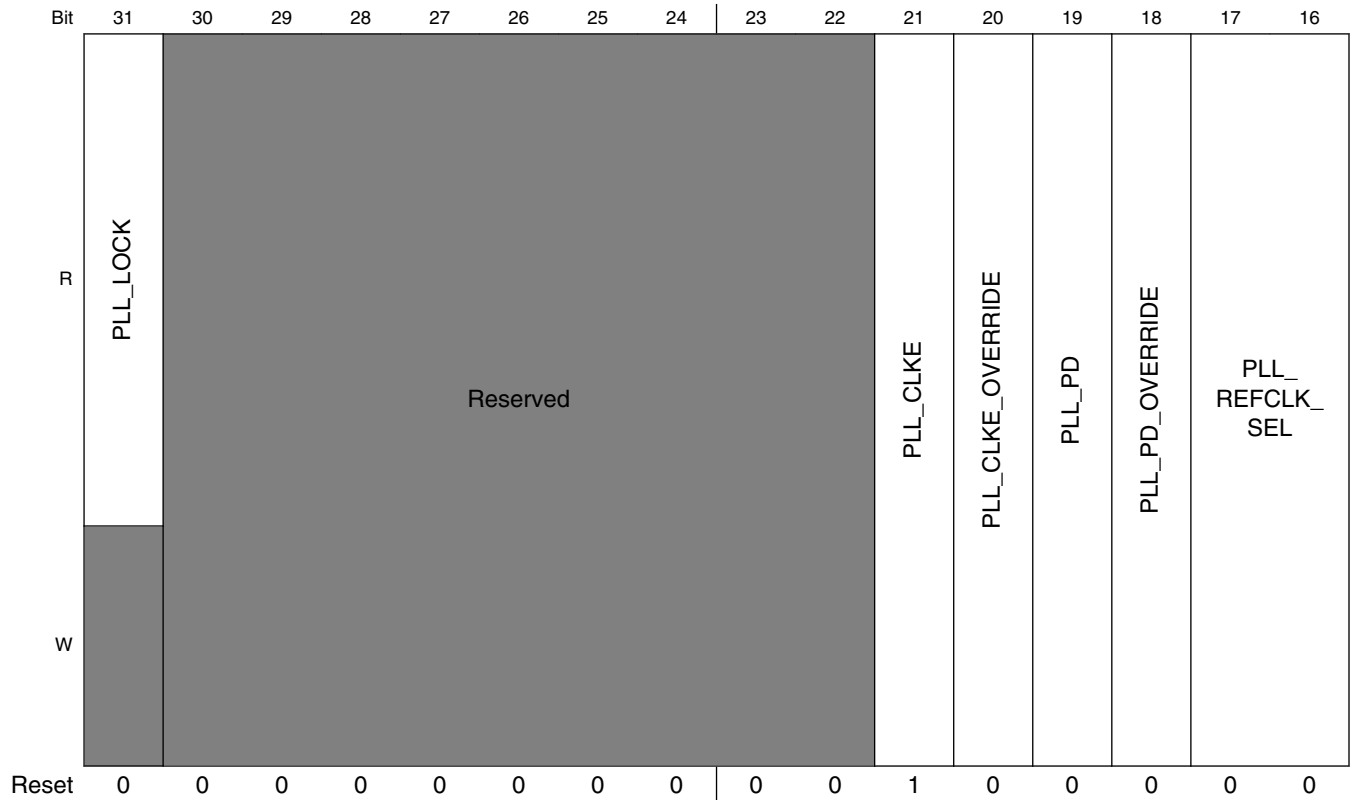
CCM_ANALOG_VIDEO_PLL1_CFG1 field descriptions

Field	Description
31 -	This field is reserved. Reserved
30–7 PLL_FRAC_DIV_CTL	PLL fraction divide control Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz Valid divider values are 1 to 2^24
PLL_INT_DIV_CTL	PLL Integer divide control Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz. Valid divider values are 1 to 32

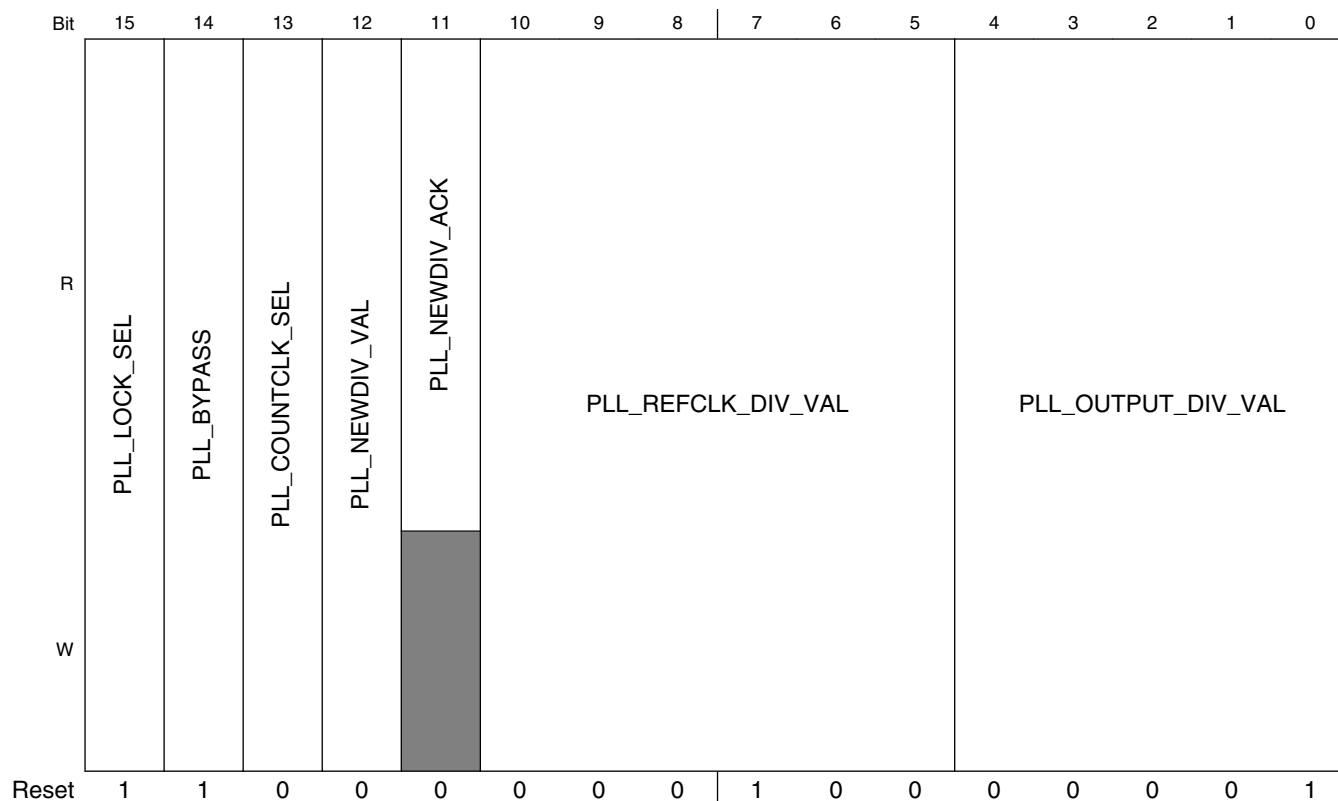
5.1.8.7 GPU PLL Configuration 0 Register (CCM_ANALOG_GPU_PLL_CFG0)

GPU PLL Configuration 0 Register

Address: 3036_0000h base + 18h offset = 3036_0018h



Clock Control Module (CCM)



CCM_ANALOG_GPU_PLL_CFG0 field descriptions

Field	Description
31 PLL_LOCK	PLL lock status active high
30–22 -	This field is reserved. Reserved
21 PLL_CLKE	PLL output clock clock gating enable active high
20 PLL_CLKE_OVERRIDE	Override the PLL_CLKE, clock gating enable signal from CCM block active high
19 PLL_PD	PLL output clock clock gating enable active high
18 PLL_PD_OVERRIDE	Override the PLL_PD, clock gating enable signal from CCM block active high
17–16 PLL_REFCLK_SEL	PLL reference clocks select 00 25M_REF_CLK 01 27M_REF_CLK 10 HDMI_PHY_27M_CLK 11 CLK_P_N
15 PLL_LOCK_SEL	PLL Lock signal select

Table continues on the next page...

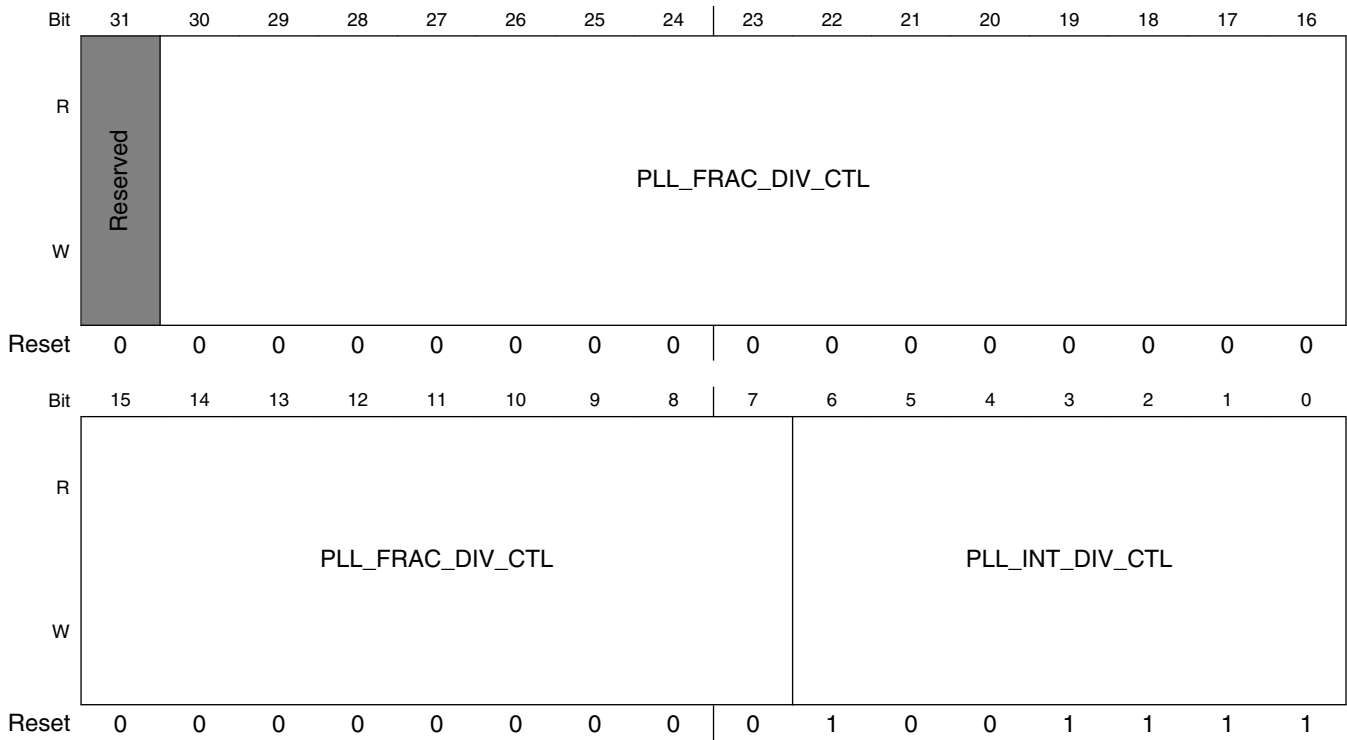
CCM_ANALOG_GPU_PLL_CFG0 field descriptions (continued)

Field	Description
	00 Select PLL lock output 01 Select maximum lock time counter output
14 PLL_BYPASS	PLL bypass control active high
13 PLL_COUNTCLK_SEL	PLL maximum lock time counter clock select 0 25M_REF_CLK 1 27M_REF_CLK
12 PLL_NEWDIV_VAL	PLL new fraction divide input control active high
11 PLL_NEWDIV_ACK	PLL new fraction divide handshake signal
10–5 PLL_REFCLK_DIV_VAL	PLL reference clock divide value. Both REF and post_divide REF must be within the range 10MHz to 300MHz. Valid divider values are 1 to 64
PLL_OUTPUT_DIV_VAL	PLL output clock divide value Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz Valid divider values are even divides from 2 to 64 (binary value + 1) * 2 00000 - 2 00001 - 4 00010 - 6 . . . 11111 - 64

5.1.8.8 GPU PLL Configuration 1 Register (CCM_ANALOG_GPU_PLL_CFG1)

GPU PLL Configuration 1 Register

Address: 3036_0000h base + 1Ch offset = 3036_001Ch



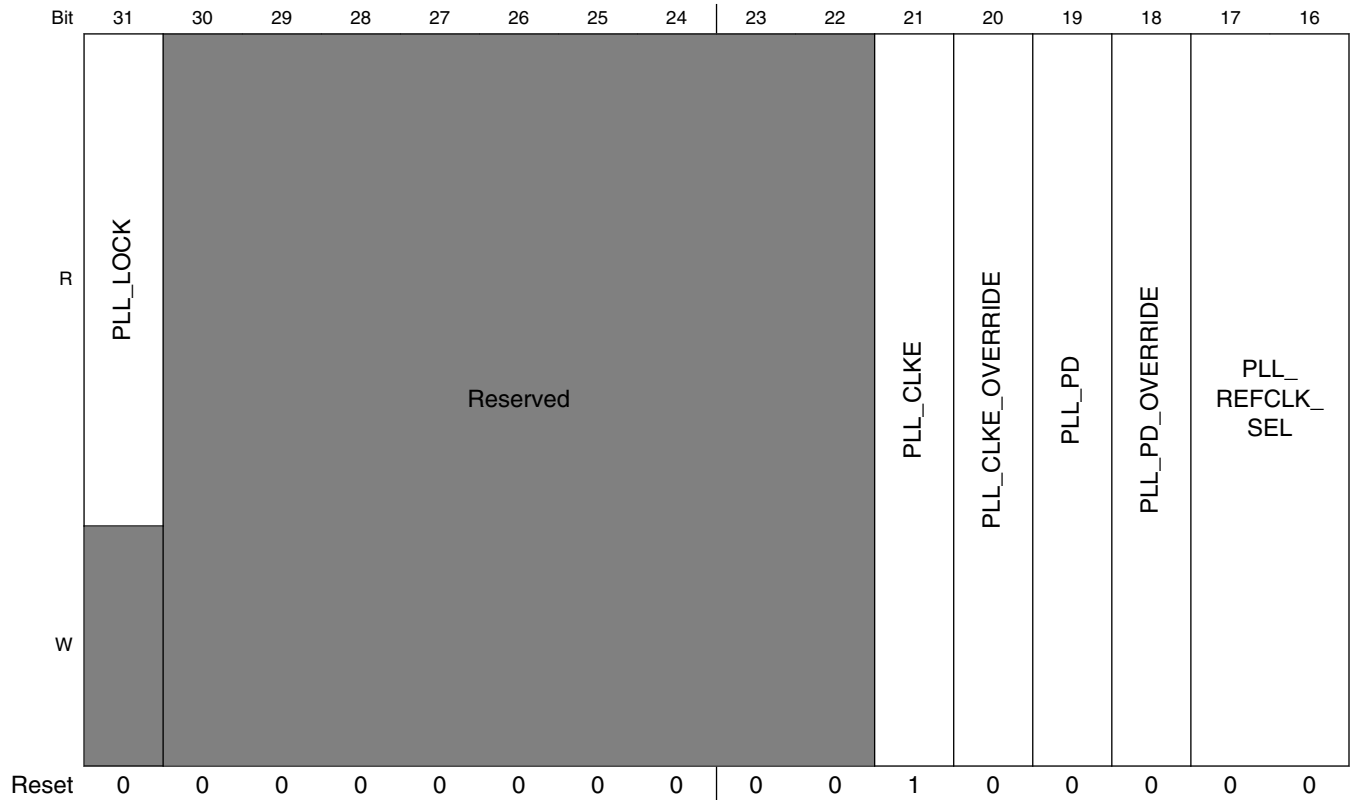
CCM_ANALOG_GPU_PLL_CFG1 field descriptions

Field	Description
31 -	This field is reserved. Reserved
30–7 PLL_FRAC_DIV_CTL	PLL fraction divide control Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz Valid divider values are 1 to 2^24
PLL_INT_DIV_CTL	PLL Integer divide control Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz. Valid divider values are 1 to 32

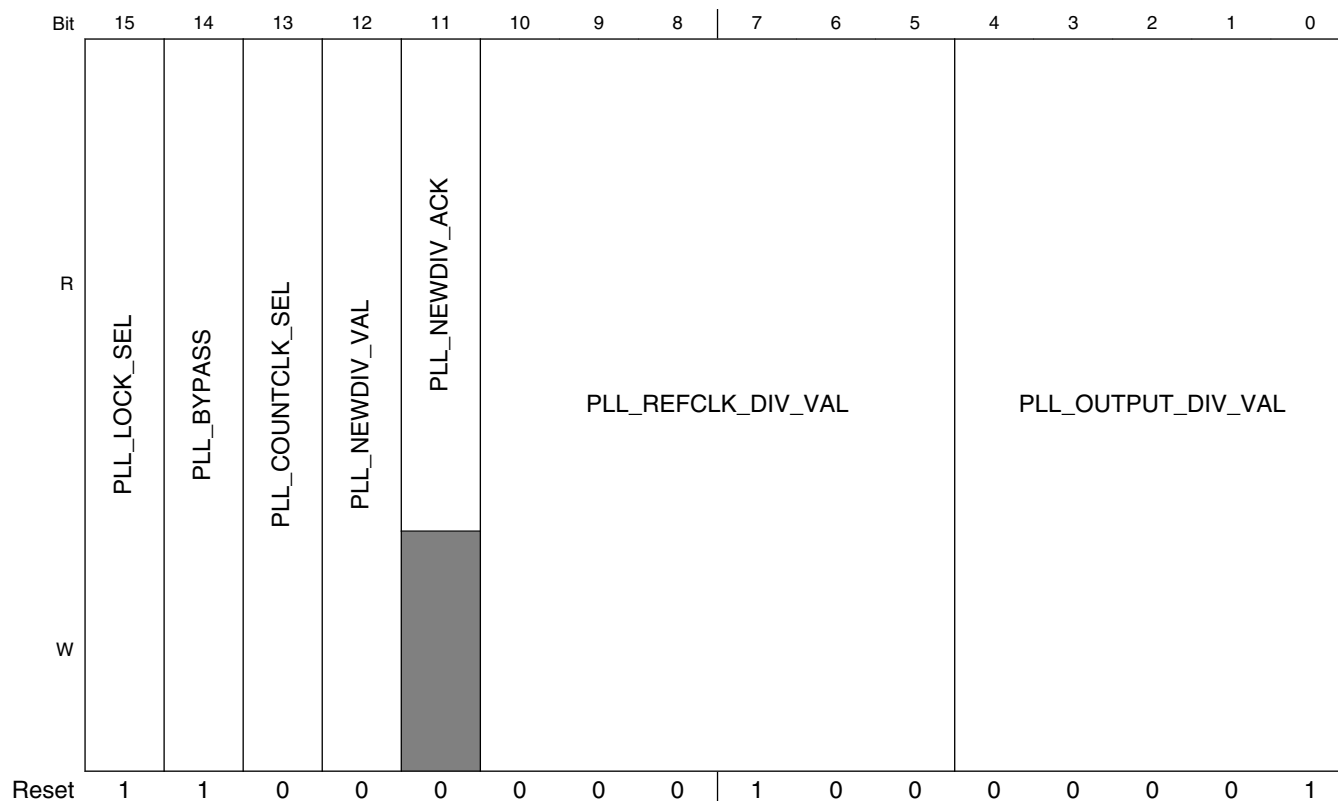
5.1.8.9 VPU PLL Configuration 0 Register (CCM_ANALOG_VPU_PLL_CFG0)

VPU PLL Configuration 0 Register

Address: 3036_0000h base + 20h offset = 3036_0020h



Clock Control Module (CCM)



CCM_ANALOG_VPU_PLL_CFG0 field descriptions

Field	Description
31 PLL_LOCK	PLL lock status active high
30–22 -	This field is reserved. Reserved
21 PLL_CLKE	PLL output clock clock gating enable active high
20 PLL_CLKE_OVERRIDE	Override the PLL_CLKE, clock gating enable signal from CCM block active high
19 PLL_PD	PLL output clock clock gating enable active high
18 PLL_PD_OVERRIDE	Override the PLL_PD, clock gating enable signal from CCM block active high
17–16 PLL_REFCLK_SEL	PLL reference clocks select 00 25M_REF_CLK 01 27M_REF_CLK 10 HDMI_PHY_27M_CLK 11 CLK_P_N
15 PLL_LOCK_SEL	PLL Lock signal select

Table continues on the next page...

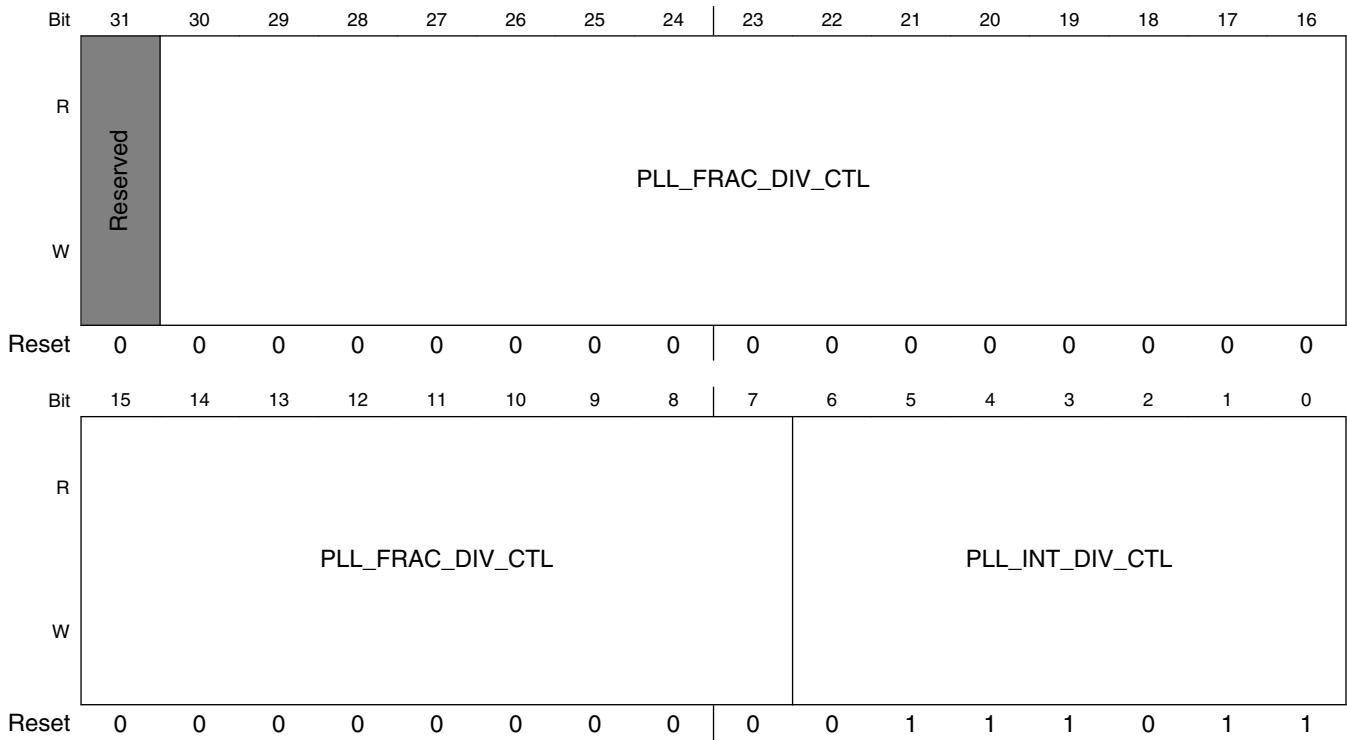
CCM_ANALOG_VPU_PLL_CFG0 field descriptions (continued)

Field	Description
	00 Select PLL lock output 01 Select maximum lock time counter output
14 PLL_BYPASS	PLL bypass control active high
13 PLL_COUNTCLK_SEL	PLL maximum lock time counter clock select 0 25M_REF_CLK 1 27M_REF_CLK
12 PLL_NEWDIV_VAL	PLL new fraction divide input control active high
11 PLL_NEWDIV_ACK	PLL new fraction divide handshake signal
10–5 PLL_REFCLK_DIV_VAL	PLL reference clock divide value. Both REF and post_divide REF must be within the range 10MHz to 300MHz. Valid divider values are 1 to 64
PLL_OUTPUT_DIV_VAL	PLL output clock divide value Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz Valid divider values are even divides from 2 to 64 (binary value + 1) * 2 00000 - 2 00001 - 4 00010 - 6 . . . 11111 - 64

5.1.8.10 VPU PLL Configuration 1 Register (CCM_ANALOG_VPU_PLL_CFG1)

VPU PLL Configuration 1 Register

Address: 3036_0000h base + 24h offset = 3036_0024h



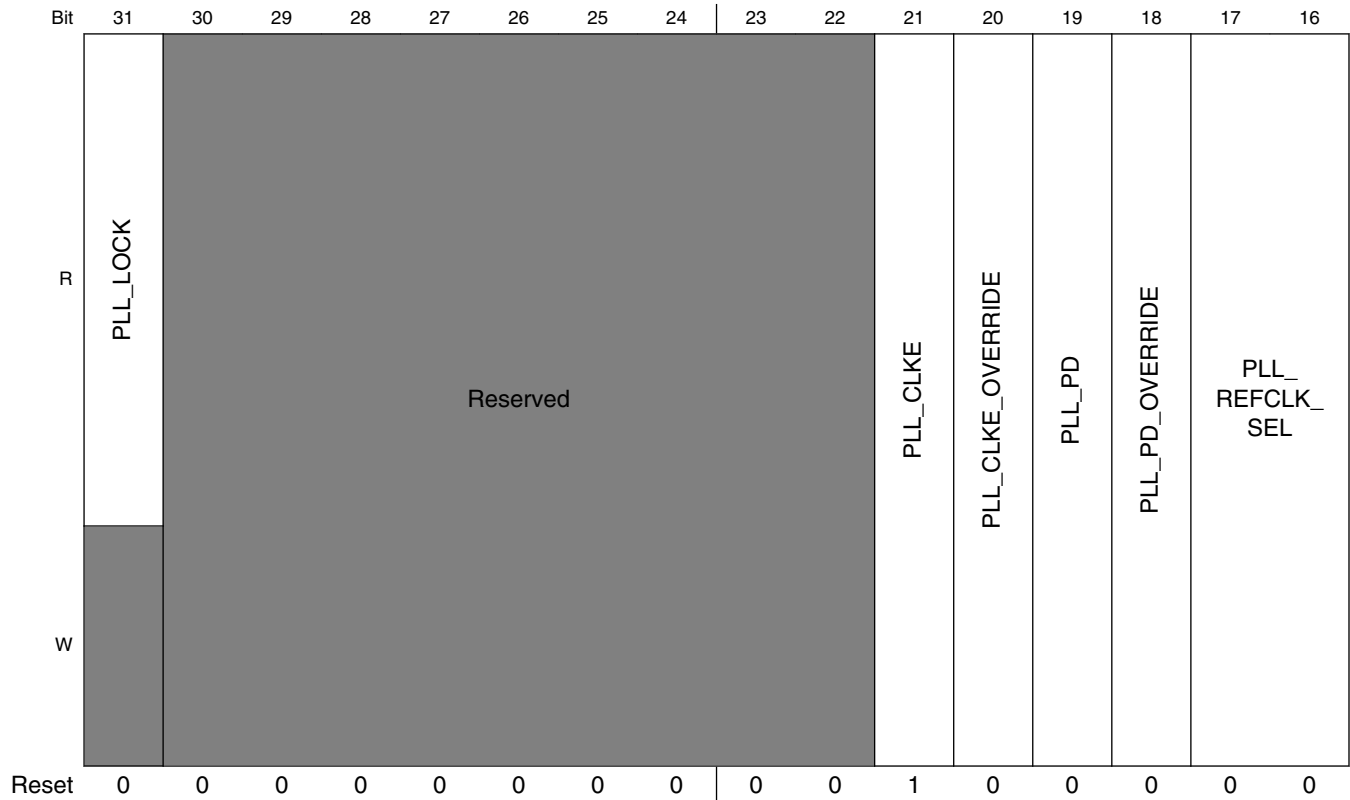
CCM_ANALOG_VPU_PLL_CFG1 field descriptions

Field	Description
31 -	This field is reserved. Reserved
30–7 PLL_FRAC_DIV_CTL	PLL fraction divide control Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz Valid divider values are 1 to 2^24
PLL_INT_DIV_CTL	PLL Integer divide control Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz. Valid divider values are 1 to 32

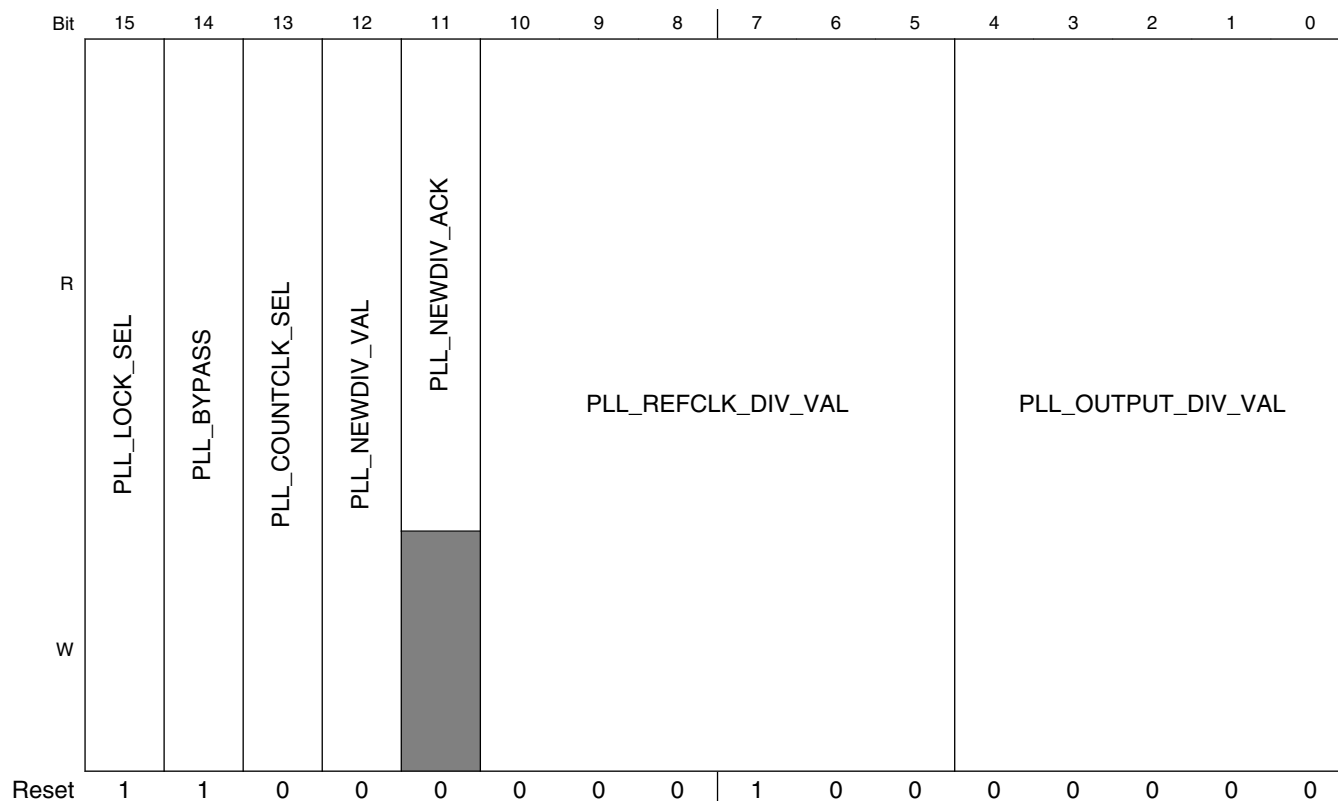
5.1.8.11 ARM PLL Configuration 0 Register (CCM_ANALOG_ARM_PLL_CFG0)

ARM PLL Configuration 0 Register

Address: 3036_0000h base + 28h offset = 3036_0028h



Clock Control Module (CCM)



CCM_ANALOG_ARM_PLL_CFG0 field descriptions

Field	Description
31 PLL_LOCK	PLL lock status active high
30–22 -	This field is reserved. Reserved
21 PLL_CLKE	PLL output clock clock gating enable active high
20 PLL_CLKE_OVERRIDE	Override the PLL_CLKE, clock gating enable signal from CCM block active high
19 PLL_PD	PLL output clock clock gating enable active high
18 PLL_PD_OVERRIDE	Override the PLL_PD, clock gating enable signal from CCM block active high
17–16 PLL_REFCLK_SEL	PLL reference clocks select 00 25M_REF_CLK 01 27M_REF_CLK 10 HDMI_PHY_27M_CLK 11 CLK_P_N
15 PLL_LOCK_SEL	PLL Lock signal select

Table continues on the next page...

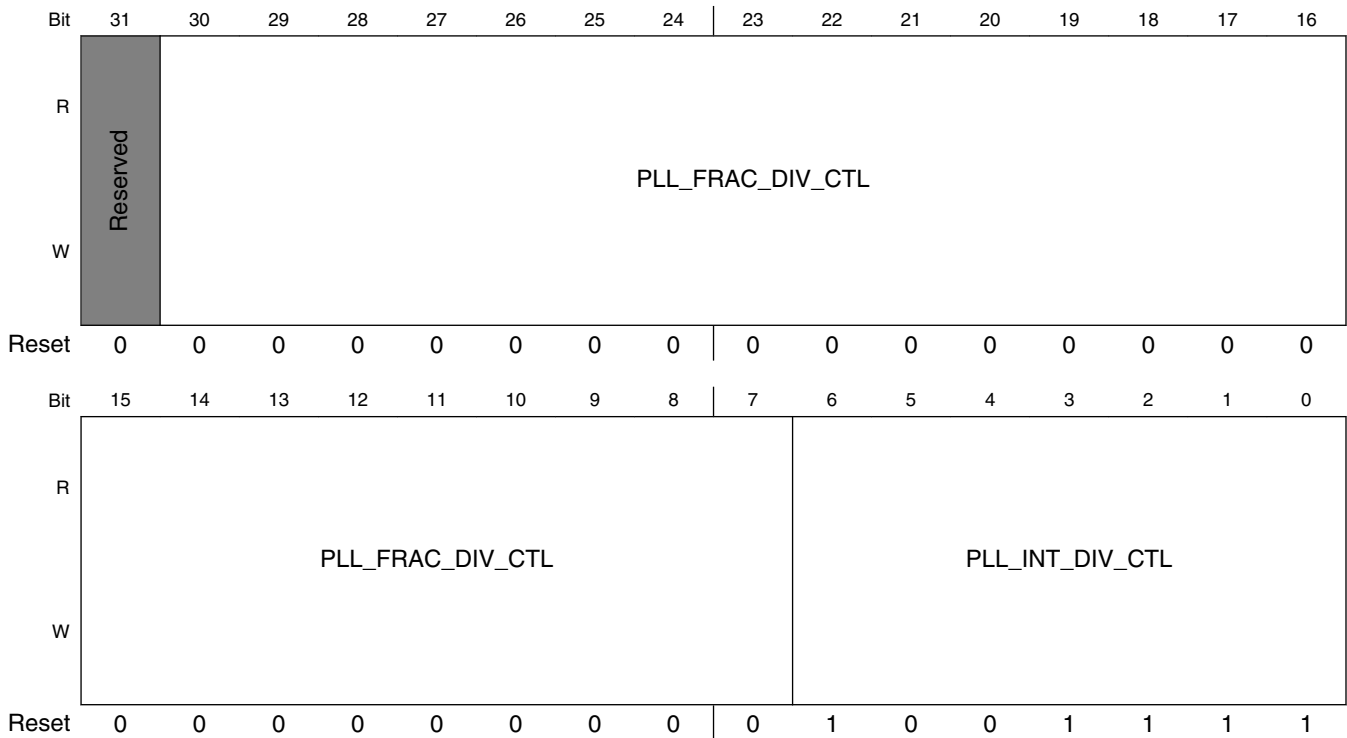
CCM_ANALOG_ARM_PLL_CFG0 field descriptions (continued)

Field	Description
	00 Select PLL lock output 01 Select maximum lock time counter output
14 PLL_BYPASS	PLL bypass control active high
13 PLL_COUNTCLK_SEL	PLL maximum lock time counter clock select 0 25M_REF_CLK 1 27M_REF_CLK
12 PLL_NEWDIV_VAL	PLL new fraction divide input control active high
11 PLL_NEWDIV_ACK	PLL new fraction divide handshake signal
10–5 PLL_REFCLK_DIV_VAL	PLL reference clock divide value. Both REF and post_divide REF must be within the range 10MHz to 300MHz. Valid divider values are 1 to 64
PLL_OUTPUT_DIV_VAL	PLL output clock divide value Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz Valid divider values are even divides from 2 to 64 (binary value + 1) * 2 00000 - 2 00001 - 4 00010 - 6 . . . 11111 - 64

5.1.8.12 ARM PLL Configuration 1 Register (CCM_ANALOG_ARM_PLL_CFG1)

ARM PLL Configuration 1 Register

Address: 3036_0000h base + 2Ch offset = 3036_002Ch



CCM_ANALOG_ARM_PLL_CFG1 field descriptions

Field	Description
31 -	This field is reserved. Reserved
30–7 PLL_FRAC_DIV_CTL	PLL fraction divide control Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz Valid divider values are 1 to 2^24
PLL_INT_DIV_CTL	PLL Integer divide control Settings must maintain the PLL operational range. Valid output clock range 30MHz to 2000MHz. Valid divider values are 1 to 32

5.1.8.13 System PLL Configuration 0 Register (CCM_ANALOG_SYS_PLL1_CFG0)

System PLL Configuration 0 Register

Address: 3036_0000h base + 30h offset = 3036_0030h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16																
R	PLL_LOCK		Reserved										PLL_CLKE		PLL_CLKE_OVERRIDE		PLL_DIV2_CLKE		PLL_DIV2_OVERRIDE		PLL_DIV3_CLKE		PLL_DIV3_OVERRIDE		PLL_DIV4_CLKE		PLL_DIV4_OVERRIDE		PLL_DIV5_CLKE		PLL_DIV5_OVERRIDE	
W																																
Reset	0	0	0	0	0	0	1	0	1	0	1	0	1	0	1	0																

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PLL_DIV6_CLKE		PLL_DIV6_OVERRIDE		PLL_DIV8_CLKE		PLL_DIV8_OVERRIDE		PLL_DIV10_CLKE		PLL_DIV10_OVERRIDE		PLL_DIV20_CLKE		PLL_DIV20_OVERRIDE	
W																
Reset	1	0	1	0	1	0	1	0	0	0	1	1	1	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PLL_PD		PLL_PD_OVERRIDE		PLL_BYPASS1		PLL_BYPASS2		PLL_LOCK_SEL		PLL_COUNTCLK_SEL		PLL_REFCLK_SEL			
W																
Reset	0	0	1	1	1	0	0	0	0	0	1	1	1	0	0	0

CCM_ANALOG_SYS_PLL1_CFG0 field descriptions

Field	Description
31 PLL_LOCK	PLL lock status active high
30–26 -	This field is reserved. Reserved
25 PLL_CLKE	PLL output clock clock gating enable active high
24 PLL_CLKE_OVERRIDE	Override the PLL_CLKE, clock gating enable signal from CCM block active high
23 PLL_DIV2_CLKE	PLL output clock divide by 2 clock gating enable active high
22 PLL_DIV2_OVERRIDE	Override the PLL_DIV2_CLKE, clock gating enable signal from CCM block active high
21 PLL_DIV3_CLKE	PLL output clock divide by 3 clock gating enable active high
20 PLL_DIV3_OVERRIDE	Override the PLL_DIV3_CLKE, clock gating enable signal from CCM block active high
19 PLL_DIV4_CLKE	PLL output clock divide by 4 clock gating enable active high
18 PLL_DIV4_OVERRIDE	Override the PLL_DIV4_CLKE, clock gating enable signal from CCM block active high
17 PLL_DIV5_CLKE	PLL output clock divide by 5 clock gating enable active high
16 PLL_DIV5_OVERRIDE	Override the PLL_DIV5_CLKE, clock gating enable signal from CCM block active high
15 PLL_DIV6_CLKE	PLL output clock divide by 6 clock gating enable active high
14 PLL_DIV6_OVERRIDE	Override the PLL_DIV6_CLKE, clock gating enable signal from CCM block active high
13 PLL_DIV8_CLKE	PLL output clock divide by 8 clock gating enable active high
12 PLL_DIV8_OVERRIDE	Override the PLL_DIV8_CLKE, clock gating enable signal from CCM block active high
11 PLL_DIV10_CLKE	PLL output clock divide by 10 clock gating enable active high
10 PLL_DIV10_OVERRIDE	Override the PLL_DIV10_CLKE, clock gating enable signal from CCM block active high
9 PLL_DIV20_CLKE	PLL output clock divide by 20 clock gating enable active high

Table continues on the next page...

CCM_ANALOG_SYS_PLL1_CFG0 field descriptions (continued)

Field	Description
8 PLL_DIV20_OVERRIDE	Override the PLL_DIV20_CLKE, clock gating enable signal from CCM block active high
7 PLL_PD	PLL output clock clock gating enable active high
6 PLL_PD_OVERRIDE	Override the PLL_PD, clock gating enable signal from CCM block active high
5 PLL_BYPASS1	Internal PLL1 bypass control active high
4 PLL_BYPASS2	Internal PLL2 bypass control active high
3 PLL_LOCK_SEL	PLL Lock signal select 00 Select PLL lock output 01 Select maximum lock time counter output
2 PLL_COUNTCLK_SEL	PLL maximum lock time counter clock select 0 25M_REF_CLK 1 27M_REF_CLK
PLL_REFCLK_SEL	PLL reference clocks select 00 25M_REF_CLK 01 27M_REF_CLK 10 HDMI_PHY_27M_CLK 11 CLK_P_N

5.1.8.14 System_PLL Configuration 1 Register (CCM_ANALOG_SYS_PLL1_CFG1)

System PLL Configuration 1 Register

Address: 3036_0000h base + 34h offset = 3036_0034h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								PLL_SS SDS	PLL_SSMD			PLL_SSMF			PLL_SSE
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

CCM_ANALOG_SYS_PLL1_CFG1 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 PLL_SS SDS	Selects between Spread Spectrum Center Spread and Down Spread Modes. 0 Center Spread 1 Down Spread
7–5 PLL_SS MDS	Controls Spread Spectrum modulation depth 000 0.25 001 0.5 010 0.75 011 1.0 100 1.5 101 2.0 110 3.0 111 4.0
4–1 PLL_SS MDF	Controls Spread Spectrum Modulation Frequency $F_{mod} = SSMF[3:0] \cdot 2 \cdot F_{vco1} / 251658$
0 PLL_SSE	Enables Spread Spectrum Mode active high

5.1.8.15 System_PLL Configuration 2 Register (CCM_ANALOG_SYS_PLL1_CFG2)

System PLL Configuration 2 Register

Address: 3036_0000h base + 38h offset = 3036_0038h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved				PLL_REF_DIVR1				PLL_REF_DIVR2					PLL_FEEDBACK_DIVF1		
W																
Reset	0	0	0	0	0	0	0	0	1	0	1	1	1	0	1	1
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PLL_FEEDBACK_DIVF1			PLL_FEEDBACK_DIVF2					PLL_OUTPUT_DIV_VAL						PLL_FILTER_RANGE	
W																
Reset	1	1	1	0	0	1	0	1	1	0	0	0	0	0	0	0

CCM_ANALOG_SYS_PLL1_CFG2 field descriptions

Field	Description
31–28 -	This field is reserved. Reserved
27–25 PLL_REF_DIVR1	Internal PLL1 reference clock divider value. REF must be within the range 25MHz to 235MHz. post_divide REF must be within the range 25MHz to 54MHz. Valid divider values are 1 to 8
24–19 PLL_REF_DIVR2	Internal PLL2 reference clock divider value. REF must be within the range 54MHz to 75MHz. post_divide REF must be within the range 25MHz to 54MHz. Valid divider values are 1 to 64
18–13 PLL_FEEDBACK_DIVF1	Internal PLL1 reference clock divider value. Fvco1 must be within the range 1600MHz to 2400MHz. Valid divider values are 1 to 64
12–7 PLL_FEEDBACK_DIVF2	Internal PLL2 reference clock divider value. REF must be within the range 20MHz to 1200MHz. Valid divider values are 1 to 64
6–1 PLL_OUTPUT_DIV_VAL	Internal PLL2 output clock divider value Fout must be within the range 20MHz ~ 1200MHz. Valid divider values are 1 ~ 64
0 PLL_FILTER_RANGE	This sets the internal PLL1 loop filter to work with the post-reference divider frequency. 0 25 to 35 MHz 1 35 to 54 MHz

5.1.8.16 System PLL Configuration 0 Register (CCM_ANALOG_SYS_PLL2_CFG0)

System PLL Configuration 0 Register

Address: 3036_0000h base + 3Ch offset = 3036_003Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PLL_LOCK	Reserved						PLL_CLKE_OVERRIDE	PLL_DIV2_CLKE	PLL_DIV2_OVERRIDE	PLL_DIV3_CLKE	PLL_DIV3_OVERRIDE	PLL_DIV4_CLKE	PLL_DIV4_OVERRIDE	PLL_DIV5_CLKE	PLL_DIV5_OVERRIDE
W																
Reset	0	0	0	0	0	0	1	0	1	0	1	0	1	0	1	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PLL_DIV6_CLKE	PLL_DIV6_OVERRIDE	PLL_DIV8_CLKE	PLL_DIV8_OVERRIDE	PLL_DIV10_CLKE	PLL_DIV10_OVERRIDE	PLL_DIV20_CLKE	PLL_DIV20_OVERRIDE	PLL_PD	PLL_PD_OVERRIDE	PLL_BYPASS1	PLL_BYPASS2	PLL_LOCK_SEL	PLL_COUNTCLK_SEL	PLL_REFCLK_SEL	
W																
Reset	1	0	1	0	1	0	1	0	0	0	1	1	1	0	0	0

CCM_ANALOG_SYS_PLL2_CFG0 field descriptions

Field	Description
31 PLL_LOCK	PLL lock status active high
30–26 -	This field is reserved. Reserved
25 PLL_CLKE	PLL output clock clock gating enable active high
24 PLL_CLKE_OVERRIDE	Override the PLL_CLKE, clock gating enable signal from CCM block active high
23 PLL_DIV2_CLKE	PLL output clock divide by 2 clock gating enable active high
22 PLL_DIV2_OVERRIDE	Override the PLL_DIV2_CLKE, clock gating enable signal from CCM block active high
21 PLL_DIV3_CLKE	PLL output clock divide by 3 clock gating enable active high
20 PLL_DIV3_OVERRIDE	Override the PLL_DIV3_CLKE, clock gating enable signal from CCM block active high
19 PLL_DIV4_CLKE	PLL output clock divide by 4 clock gating enable active high
18 PLL_DIV4_OVERRIDE	Override the PLL_DIV4_CLKE, clock gating enable signal from CCM block active high
17 PLL_DIV5_CLKE	PLL output clock divide by 5 clock gating enable active high
16 PLL_DIV5_OVERRIDE	Override the PLL_DIV5_CLKE, clock gating enable signal from CCM block active high
15 PLL_DIV6_CLKE	PLL output clock divide by 6 clock gating enable active high
14 PLL_DIV6_OVERRIDE	Override the PLL_DIV6_CLKE, clock gating enable signal from CCM block active high
13 PLL_DIV8_CLKE	PLL output clock divide by 8 clock gating enable active high
12 PLL_DIV8_OVERRIDE	Override the PLL_DIV8_CLKE, clock gating enable signal from CCM block active high
11 PLL_DIV10_CLKE	PLL output clock divide by 10 clock gating enable active high
10 PLL_DIV10_OVERRIDE	Override the PLL_DIV10_CLKE, clock gating enable signal from CCM block active high
9 PLL_DIV20_CLKE	PLL output clock divide by 20 clock gating enable active high

Table continues on the next page...

CCM_ANALOG_SYS_PLL2_CFG0 field descriptions (continued)

Field	Description
8 PLL_DIV20_OVERRIDE	Override the PLL_DIV20_CLKE, clock gating enable signal from CCM block active high
7 PLL_PD	PLL output clock clock gating enable active high
6 PLL_PD_OVERRIDE	Override the PLL_PD, clock gating enable signal from CCM block active high
5 PLL_BYPASS1	Internal PLL1 bypass control active high
4 PLL_BYPASS2	Internal PLL2 bypass control active high
3 PLL_LOCK_SEL	PLL Lock signal select 00 Select PLL lock output 01 Select maximum lock time counter output
2 PLL_COUNTCLK_SEL	PLL maximum lock time counter clock select 0 25M_REF_CLK 1 27M_REF_CLK
PLL_REFCLK_SEL	PLL reference clocks select 00 25M_REF_CLK 01 27M_REF_CLK 10 HDMI_PHY_27M_CLK 11 CLK_P_N

5.1.8.17 System_PLL Configuration 1 Register (CCM_ANALOG_SYS_PLL2_CFG1)

System PLL Configuration 1 Register

Address: 3036_0000h base + 40h offset = 3036_0040h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								PLL_SS SDS	PLL_SSMD			PLL_SSMF			PLL_SSE
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

CCM_ANALOG_SYS_PLL2_CFG1 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 PLL_SS SDS	Selects between Spread Spectrum Center Spread and Down Spread Modes. 0 Center Spread 1 Down Spread
7–5 PLL_S SM D	Controls Spread Spectrum modulation depth 000 0.25 001 0.5 010 0.75 011 1.0 100 1.5 101 2.0 110 3.0 111 4.0
4–1 PLL_S SM F	Controls Spread Spectrum Modulation Frequency $F_{mod} = SSMF[3:0] \cdot 2 \cdot F_{vco1} / 251658$
0 PLL_S S E	Enables Spread Spectrum Mode active high

5.1.8.18 System_PLL Configuration 2 Register (CCM_ANALOG_SYS_PLL2_CFG2)

System PLL Configuration 2 Register

Address: 3036_0000h base + 44h offset = 3036_0044h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved				PLL_REF_DIVR1				PLL_REF_DIVR2					PLL_FEEDBACK_DIVF1		
W																
Reset	0	0	0	0	0	0	0	0	1	0	1	1	1	0	1	1
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PLL_FEEDBACK_DIVF1			PLL_FEEDBACK_DIVF2					PLL_OUTPUT_DIV_VAL						PLL_FILTER_RANGE	
W																
Reset	1	1	1	0	0	1	1	1	0	0	0	0	0	0	0	0

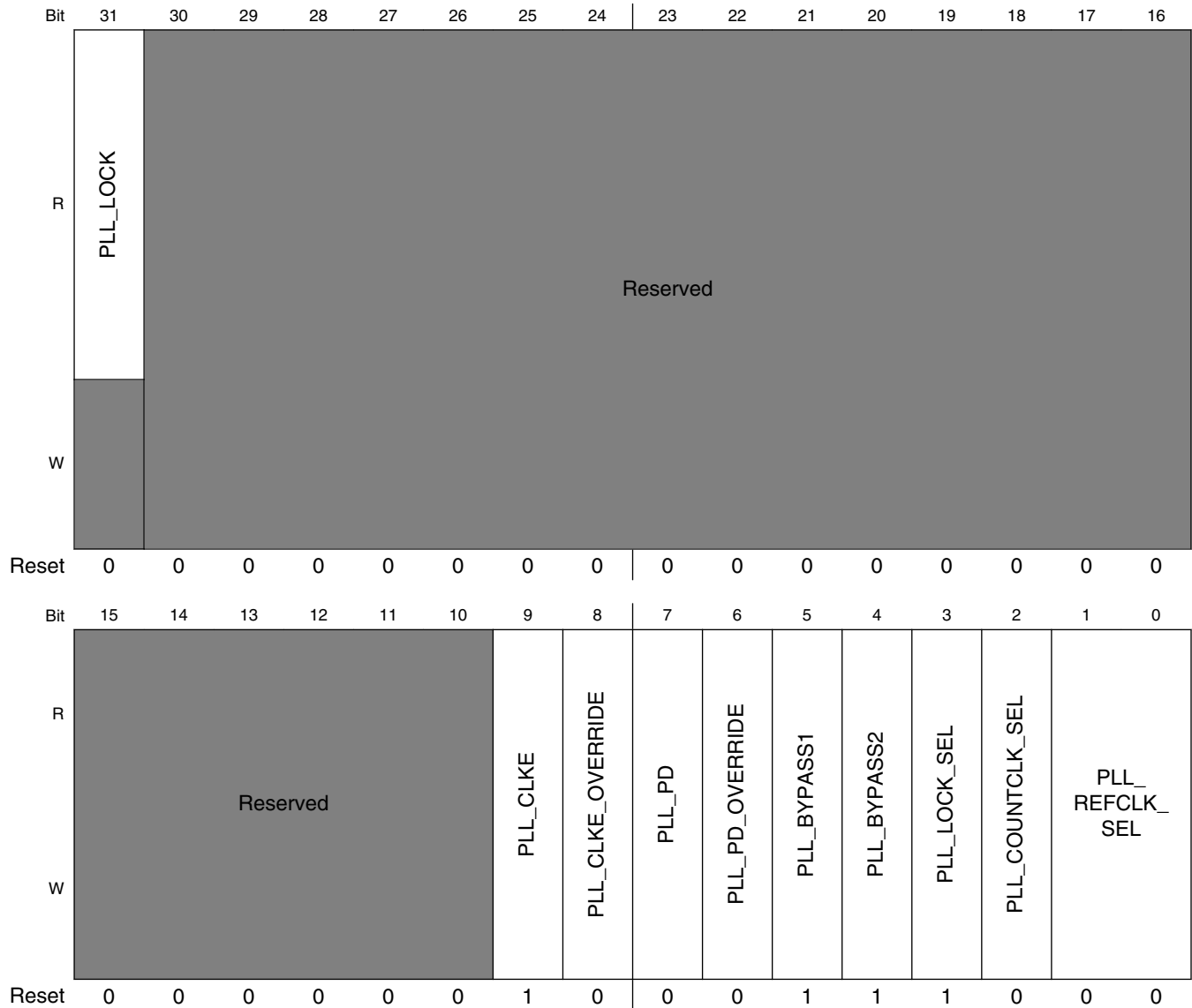
CCM_ANALOG_SYS_PLL2_CFG2 field descriptions

Field	Description
31–28 -	This field is reserved. Reserved
27–25 PLL_REF_DIVR1	Internal PLL1 reference clock divider value. REF must be within the range 25MHz to 235MHz. post_divide REF must be within the range 25MHz to 54MHz. Valid divider values are 1 to 8
24–19 PLL_REF_DIVR2	Internal PLL2 reference clock divider value. REF must be within the range 54MHz to 75MHz. post_divide REF must be within the range 25MHz to 54MHz. Valid divider values are 1 to 64
18–13 PLL_FEEDBACK_DIVF1	Internal PLL1 reference clock divider value. Fvco1 must be within the range 1600MHz to 2400MHz. Valid divider values are 1 to 64
12–7 PLL_FEEDBACK_DIVF2	Internal PLL2 reference clock divider value. REF must be within the range 20MHz to 1200MHz. Valid divider values are 1 to 64
6–1 PLL_OUTPUT_DIV_VAL	Internal PLL2 output clock divider value Fout must be within the range 20MHz ~ 1200MHz. Valid divider values are 1 ~ 64
0 PLL_FILTER_RANGE	This sets the internal PLL1 loop filter to work with the post-reference divider frequency. 0 25 to 35 MHz 1 35 to 54 MHz

5.1.8.19 System PLL Configuration 0 Register (CCM_ANALOG_SYS_PLL3_CFG0)

System PLL Configuration 0 Register

Address: 3036_0000h base + 48h offset = 3036_0048h



CCM_ANALOG_SYS_PLL3_CFG0 field descriptions

Field	Description
31 PLL_LOCK	PLL lock status active high

Table continues on the next page...

CCM_ANALOG_SYS_PLL3_CFG0 field descriptions (continued)

Field	Description
30–10 -	This field is reserved. Reserved
9 PLL_CLKE	PLL output clock divide by 20 clock gating enable active high
8 PLL_CLKE_OVERRIDE	Override the PLL_DIV20_CLKE, clock gating enable signal from CCM block active high
7 PLL_PD	PLL output clock clock gating enable active high
6 PLL_PD_OVERRIDE	Override the PLL_PD, clock gating enable signal from CCM block active high
5 PLL_BYPASS1	Internal PLL1 bypass control active high
4 PLL_BYPASS2	Internal PLL2 bypass control active high
3 PLL_LOCK_SEL	PLL Lock signal select 00 Select PLL lock output 01 Select maximum lock time counter output
2 PLL_COUNTCLK_SEL	PLL maximum lock time counter clock select 0 25M_REF_CLK 1 27M_REF_CLK
PLL_REFCLK_SEL	PLL reference clocks select 00 25M_REF_CLK 01 27M_REF_CLK 10 HDMI_PHY_27M_CLK 11 CLK_P_N

5.1.8.20 System_PLL Configuration 1 Register (CCM_ANALOG_SYS_PLL3_CFG1)

System PLL Configuration 1 Register

Address: 3036_0000h base + 4Ch offset = 3036_004Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								PLL_SS SDS	PLL_SSMD			PLL_SSMF			PLL_SSE
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

CCM_ANALOG_SYS_PLL3_CFG1 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 PLL_SS SDS	Selects between Spread Spectrum Center Spread and Down Spread Modes. 0 Center Spread 1 Down Spread
7–5 PLL_S SMD	Controls Spread Spectrum modulation depth 000 0.25 001 0.5 010 0.75 011 1.0 100 1.5 101 2.0 110 3.0 111 4.0
4–1 PLL_S SMF	Controls Spread Spectrum Modulation Frequency $F_{mod} = SSMF[3:0] \cdot 2 \cdot F_{vco1} / 251658$
0 PLL_S SSE	Enables Spread Spectrum Mode active high

5.1.8.21 System_PLL Configuration 2 Register (CCM_ANALOG_SYS_PLL3_CFG2)

System PLL Configuration 2 Register

Address: 3036_0000h base + 50h offset = 3036_0050h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved				PLL_REF_DIVR1				PLL_REF_DIVR2					PLL_FEEDBACK_DIVF1		
W																
Reset	0	0	0	0	0	0	0	0	1	0	1	1	1	0	1	1
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PLL_FEEDBACK_DIVF1			PLL_FEEDBACK_DIVF2					PLL_OUTPUT_DIV_VAL						PLL_FILTER_RANGE	
W																
Reset	1	1	1	0	0	1	1	1	0	0	0	0	0	0	0	0

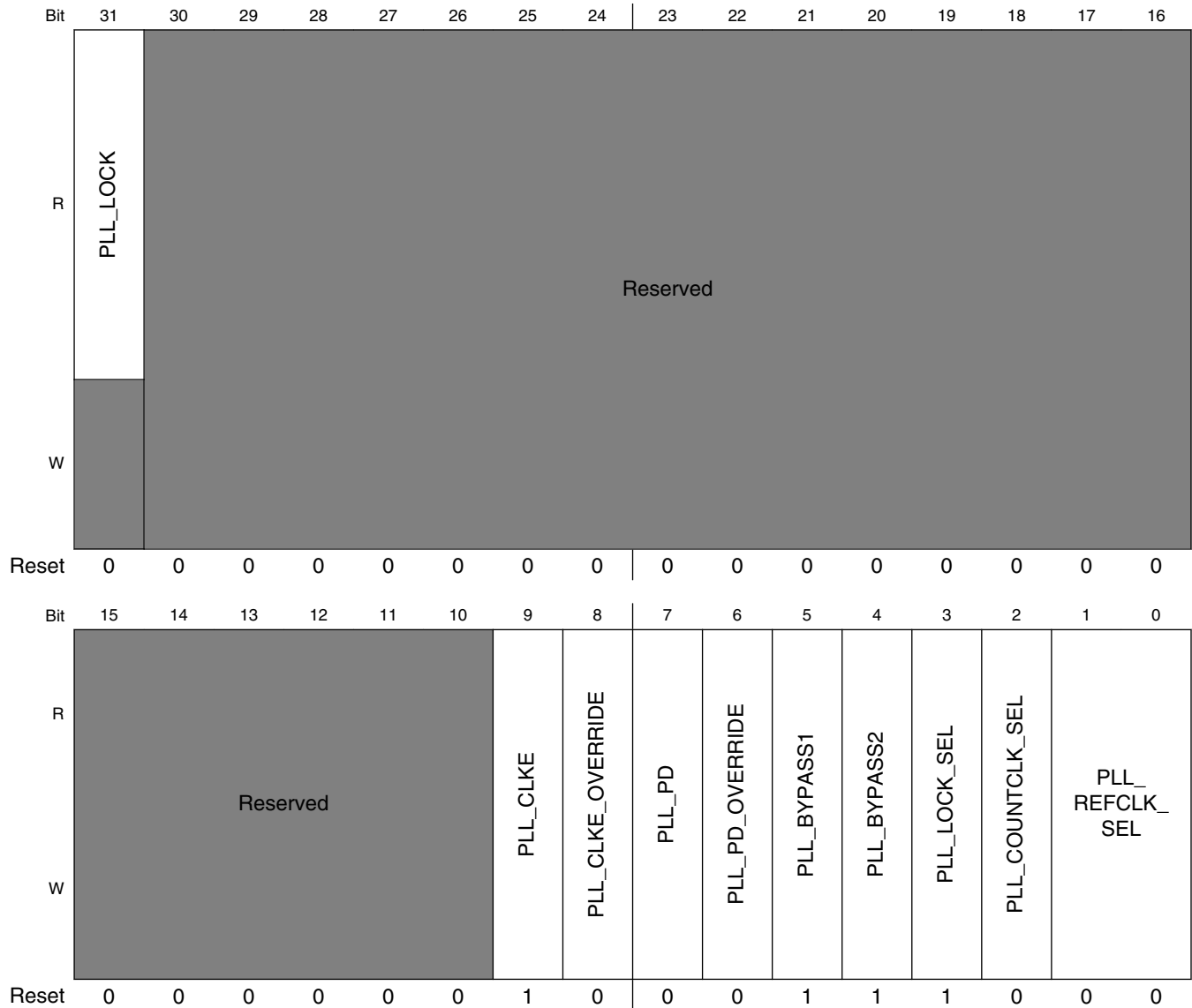
CCM_ANALOG_SYS_PLL3_CFG2 field descriptions

Field	Description
31–28 -	This field is reserved. Reserved
27–25 PLL_REF_DIVR1	Internal PLL1 reference clock divider value. REF must be within the range 25MHz to 235MHz. post_divide REF must be within the range 25MHz to 54MHz. Valid divider values are 1 to 8
24–19 PLL_REF_DIVR2	Internal PLL2 reference clock divider value. REF must be within the range 54MHz to 75MHz. post_divide REF must be within the range 25MHz to 54MHz. Valid divider values are 1 to 64
18–13 PLL_FEEDBACK_DIVF1	Internal PLL1 reference clock divider value. Fvco1 must be within the range 1600MHz to 2400MHz. Valid divider values are 1 to 64
12–7 PLL_FEEDBACK_DIVF2	Internal PLL2 reference clock divider value. REF must be within the range 20MHz to 1200MHz. Valid divider values are 1 to 64
6–1 PLL_OUTPUT_DIV_VAL	Internal PLL2 output clock divider value Fout must be within the range 20MHz ~ 1200MHz. Valid divider values are 1 ~ 64
0 PLL_FILTER_RANGE	This sets the internal PLL1 loop filter to work with the post-reference divider frequency. 0 25 to 35 MHz 1 35 to 54 MHz

5.1.8.22 VIDEO PLL2 Configuration 0 Register (CCM_ANALOG_VIDEO_PLL2_CFG0)

VIDEO PLL2 Configuration 0 Register

Address: 3036_0000h base + 54h offset = 3036_0054h



CCM_ANALOG_VIDEO_PLL2_CFG0 field descriptions

Field	Description
31 PLL_LOCK	PLL lock status active high

Table continues on the next page...

CCM_ANALOG_VIDEO_PLL2_CFG0 field descriptions (continued)

Field	Description
30–10 -	This field is reserved. Reserved
9 PLL_CLKE	PLL output clock clock gating enable active high
8 PLL_CLKE_OVERRIDE	Override the PLL_CLKE, clock gating enable signal from CCM block active high
7 PLL_PD	PLL output clock clock gating enable active high
6 PLL_PD_OVERRIDE	Override the PLL_PD, clock gating enable signal from CCM block active high
5 PLL_BYPASS1	Internal PLL1 bypass control active high
4 PLL_BYPASS2	Internal PLL2 bypass control active high
3 PLL_LOCK_SEL	PLL Lock signal select 00 Select PLL lock output 01 Select maximum lock time counter output
2 PLL_COUNTCLK_SEL	PLL maximum lock time counter clock select 0 25M_REF_CLK 1 27M_REF_CLK
PLL_REFCLK_SEL	PLL reference clocks select 00 25M_REF_CLK 01 27M_REF_CLK 10 HDMI_PHY_27M_CLK 11 CLK_P_N

5.1.8.23 VIDEO PLL2 Configuration 1 Register (CCM_ANALOG_VIDEO_PLL2_CFG1)

VIDEO PLL2 Configuration 1 Register

Address: 3036_0000h base + 58h offset = 3036_0058h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								PLL_SS SSDS			PLL_SSMF				PLL_SSE
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

CCM_ANALOG_VIDEO_PLL2_CFG1 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 PLL_SS SDS	Selects between Spread Spectrum Center Spread and Down Spread Modes. 0 Center Spread 1 Down Spread
7–5 PLL_SS MDS	Controls Spread Spectrum modulation depth 000 0.25 001 0.5 010 0.75 011 1.0 100 1.5 101 2.0 110 3.0 111 4.0
4–1 PLL_SS MDF	Controls Spread Spectrum Modulation Frequency $F_{mod} = SSMF[3:0] \cdot 2 \cdot F_{vco1} / 251658$
0 PLL_SS SE	Enables Spread Spectrum Mode active high

5.1.8.24 VIDEO PLL2 Configuration 2 Register (CCM_ANALOG_VIDEO_PLL2_CFG2)

VIDEO PLL2 Configuration 2 Register

Address: 3036_0000h base + 5Ch offset = 3036_005Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved				PLL_REF_DIVR1				PLL_REF_DIVR2					PLL_FEEDBACK_DIVF1		
W																
Reset	0	0	0	0	0	0	0	0	1	1	1	0	1	1	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PLL_FEEDBACK_DIVF1			PLL_FEEDBACK_DIVF2					PLL_OUTPUT_DIV_VAL						PLL_FILTER_RANGE	
W																
Reset	0	1	1	0	0	1	0	0	1	0	0	0	0	0	0	0

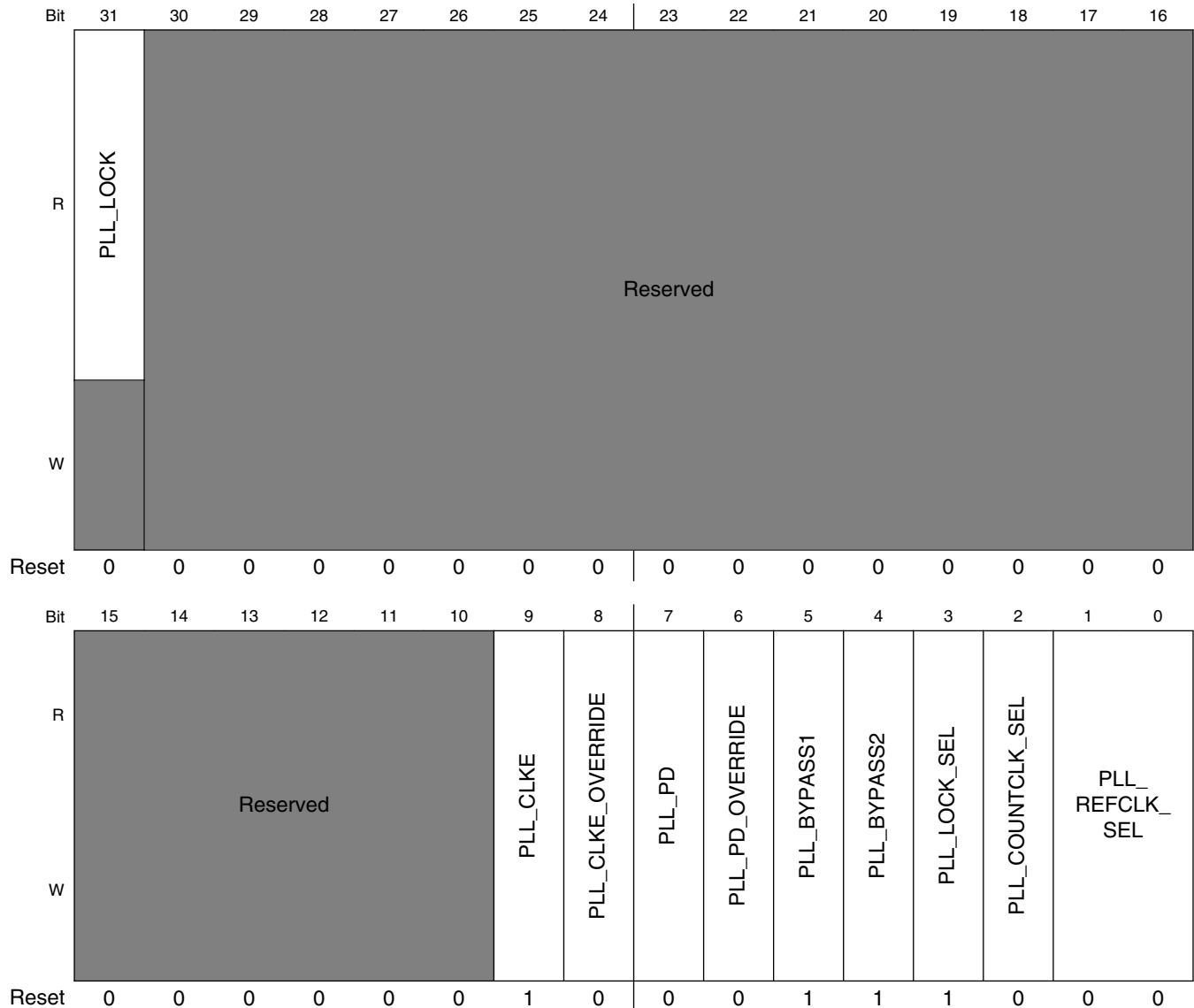
CCM_ANALOG_VIDEO_PLL2_CFG2 field descriptions

Field	Description
31–28 -	This field is reserved. Reserved
27–25 PLL_REF_DIVR1	Internal PLL1 reference clock divider value. REF must be within the range 25MHz to 235MHz. post_divide REF must be within the range 25MHz to 54MHz. Valid divider values are 1 to 8
24–19 PLL_REF_DIVR2	Internal PLL2 reference clock divider value. REF must be within the range 54MHz to 75MHz. post_divide REF must be within the range 25MHz to 54MHz. Valid divider values are 1 to 64
18–13 PLL_FEEDBACK_DIVF1	Internal PLL1 reference clock divider value. Fvco1 must be within the range 1600MHz to 2400MHz. Valid divider values are 1 to 64
12–7 PLL_FEEDBACK_DIVF2	Internal PLL2 reference clock divider value. REF must be within the range 20MHz to 1200MHz. Valid divider values are 1 to 64
6–1 PLL_OUTPUT_DIV_VAL	Internal PLL2 output clock divider value Fout must be within the range 20MHz ~ 1200MHz. Valid divider values are 1 ~ 64
0 PLL_FILTER_RANGE	This sets the internal PLL1 loop filter to work with the post-reference divider frequency. 0 25 to 35 MHz 1 35 to 54 MHz

5.1.8.25 DRAM PLL Configuration 0 Register (CCM_ANALOG_DRAM_PLL_CFG0)

DRAM PLL Configuration 0 Register

Address: 3036_0000h base + 60h offset = 3036_0060h



CCM_ANALOG_DRAM_PLL_CFG0 field descriptions

Field	Description
31 PLL_LOCK	PLL lock status active high

Table continues on the next page...

CCM_ANALOG_DRAM_PLL_CFG0 field descriptions (continued)

Field	Description
30–10 -	This field is reserved. Reserved
9 PLL_CLKE	PLL output clock clock gating enable active high
8 PLL_CLKE_OVERRIDE	Override the PLL_CLKE, clock gating enable signal from CCM block active high
7 PLL_PD	PLL output clock clock gating enable active high
6 PLL_PD_OVERRIDE	Override the PLL_PD, clock gating enable signal from CCM block active high
5 PLL_BYPASS1	Internal PLL1 bypass control active high
4 PLL_BYPASS2	Internal PLL2 bypass control active high
3 PLL_LOCK_SEL	PLL Lock signal select 00 Select PLL lock output 01 Select maximum lock time counter output
2 PLL_COUNTCLK_SEL	PLL maximum lock time counter clock select 0 25M_REF_CLK 1 27M_REF_CLK
PLL_REFCLK_SEL	PLL reference clocks select 00 25M_REF_CLK 01 27M_REF_CLK 10 HDMI_PHY_27M_CLK 11 CLK_P_N

5.1.8.26 DRAM PLL Configuration 1 Register (CCM_ANALOG_DRAM_PLL_CFG1)

DRAM PLL Configuration 1 Register

Address: 3036_0000h base + 64h offset = 3036_0064h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								PLL_SS SSDS		PLL_SSM D			PLL_SSM F		PLL_S SSE
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

CCM_ANALOG_DRAM_PLL_CFG1 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 PLL_SS SDS	Selects between Spread Spectrum Center Spread and Down Spread Modes. 0 Center Spread 1 Down Spread
7–5 PLL_SSM D	Controls Spread Spectrum modulation depth 000 0.25 001 0.5 010 0.75 011 1.0 100 1.5 101 2.0 110 3.0 111 4.0
4–1 PLL_SSM F	Controls Spread Spectrum Modulation Frequency $F_{mod} = SSMF[3:0] \cdot 2 \cdot F_{vco1} / 251658$
0 PLL_S SSE	Enables Spread Spectrum Mode active high

5.1.8.27 DRAM PLL Configuration 2 Register (CCM_ANALOG_DRAM_PLL_CFG2)

DRAM PLL Configuration 2 Register

Address: 3036_0000h base + 68h offset = 3036_0068h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved				PLL_REF_DIVR1				PLL_REF_DIVR2					PLL_FEEDBACK_DIVF1		
W																
Reset	0	0	0	0	0	0	0	0	1	0	1	1	1	0	1	1
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PLL_FEEDBACK_DIVF1			PLL_FEEDBACK_DIVF2					PLL_OUTPUT_DIV_VAL						PLL_FILTER_RANGE	
W																
Reset	1	1	1	0	0	1	0	1	1	0	0	0	0	0	0	0

CCM_ANALOG_DRAM_PLL_CFG2 field descriptions

Field	Description
31–28 -	This field is reserved. Reserved
27–25 PLL_REF_DIVR1	Internal PLL1 reference clock divider value. REF must be within the range 25MHz to 235MHz. post_divide REF must be within the range 25MHz to 54MHz. Valid divider values are 1 to 8
24–19 PLL_REF_DIVR2	Internal PLL2 reference clock divider value. REF must be within the range 54MHz to 75MHz. post_divide REF must be within the range 25MHz to 54MHz. Valid divider values are 1 to 64
18–13 PLL_FEEDBACK_DIVF1	Internal PLL1 reference clock divider value. Fvco1 must be within the range 1600MHz to 2400MHz. Valid divider values are 1 to 64
12–7 PLL_FEEDBACK_DIVF2	Internal PLL2 reference clock divider value. REF must be within the range 20MHz to 1200MHz. Valid divider values are 1 to 64
6–1 PLL_OUTPUT_DIV_VAL	Internal PLL2 output clock divider value Fout must be within the range 20MHz ~ 1200MHz. Valid divider values are 1 ~ 64
0 PLL_FILTER_RANGE	This sets the internal PLL1 loop filter to work with the post-reference divider frequency. 0 25 to 35 MHz 1 35 to 54 MHz

5.1.8.28 DIGPROG Register (CCM_ANALOG_DIGPROG)

DIGPROG Register

Address: 3036_0000h base + 6Ch offset = 3036_006Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								DIGPROG_MAJOR_UPPER								DIGPROG_MAJOR_LOWER								DIGPROG_MINOR							
W																																
Reset	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0

CCM_ANALOG_DIGPROG field descriptions

Field	Description
31–24 -	This field is reserved. Reserved
23–16 DIGPROG_ MAJOR_UPPER	Bit[7:4] is 0x8, stands for “i.MX8” Bit[3:0] is 0x2, stands for “M”
15–8 DIGPROG_ MAJOR_LOWER	Bit[7:4] is 0x4, stands for “Quad” Bit[3:0] is 0x0, stands for “Lite”
DIGPROG_ MINOR	Bit[7:4] is the base layer revision, Bit[3:0] is the metal layer revision 0x10 stands for Tapeout 1.0

5.1.8.29 Osc Misc Configuration Register (CCM_ANALOG_OSC_MISC_CFG)

Osc Misc Register

Address: 3036_0000h base + 70h offset = 3036_0070h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												OSC_27M_CLKE	OSC_27M_CLKE_OVERRIDE	OSC_25M_CLKE	OSC_25M_CLKE_OVERRIDE
W													OSC_27M_CLKE	OSC_27M_CLKE_OVERRIDE	OSC_25M_CLKE	OSC_25M_CLKE_OVERRIDE
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0

CCM_ANALOG_OSC_MISC_CFG field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 OSC_27M_CLKE	27MHz OSC output clock gating enable active high
3 OSC_27M_CLKE_OVERRIDE	Override the OSC_27M_CLKE, clock gating enable signal from CCM block active high
2 OSC_25M_CLKE	25MHz OSC output clock gating enable active high
1 OSC_25M_CLKE_OVERRIDE	Override the OSC_27M_CLKE, clock gating enable signal from CCM block active high
0 OSC_32K_SEL	32KHz OSC input select 0 25M_REF_CLK_DIV800 1 RTC

5.1.8.30 PLLOUT Monitor Configuration Register (CCM_ANALOG_PLLOUT_MONITOR_CFG)

PLLOUT Monitor Configuration Register contains bits to control the clock that will be generated on the CCM clock mapped to CLK2_P/N.

Address: 3036_0000h base + 74h offset = 3036_0074h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved											PLLOUT_ MONITOR_CKE	PLLOUT_MONITOR_CLK_ SEL			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

CCM_ANALOG_PLLOUT_MONITOR_CFG field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 PLLOUT_ MONITOR_CKE	Clock monitor output clock gating enable active high
PLLOUT_ MONITOR_CLK_ SEL	Clock monitor output clock select: 00 25M_REF_CLK 01 27M_REF_CLK 10 HDMI_PHY_27M_CLK 11 CLK1_P_N 100 OSC_32K_CLK 101 AUDIO_PLL1_CLK 110 AUDIO_PLL2_CLK 111 GPU_PLL_CLK 1000 VPU_PLL_CLK 1001 VIDEO_PLL1_CLK

Table continues on the next page...

CCM_ANALOG_PLLOUT_MONITOR_CFG field descriptions (continued)

Field	Description
1010	ARM_PLL_CLK
1011	SYSTEM_PLL1_CLK
1100	SYSTEM_PLL2_CLK
1101	SYSTEM_PLL3_CLK
1110	VIDEO_PLL2_CLK
1111	DRAM_PLL_CLK

5.1.8.31 Fractional PLLOUT Divider Configuration Register (CCM_ANALOG_FRAC_PLLOUT_DIV_CFG)

Fractional PLLOUT Divider Configuration Register

Address: 3036_0000h base + 78h offset = 3036_0078h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved									ARM_PLL_DIV_VAL			Reserved	VPU_PLL_DIV_VAL		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved	GPU_PLL_DIV_VAL				Reserved	VIDEO_PLL1_DIV_VAL			Reserved	AUDIO_PLL2_DIV_VAL			Reserved	AUDIO_PLL1_DIV_VAL	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

CCM_ANALOG_FRAC_PLLOUT_DIV_CFG field descriptions

Field	Description
31–23 -	This field is reserved. Reserved

Table continues on the next page...

CCM_ANALOG_FRAC_PLLOUT_DIV_CFG field descriptions (continued)

Field	Description
22–20 ARM_PLL_DIV_ VAL	ARM PLL clock divider value, for test purpose. 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
19 -	This field is reserved. Reserved
18–16 VPU_PLL_DIV_ VAL	VPU PLL clock divider value, for test purpose. 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
15 -	This field is reserved. Reserved
14–12 GPU_PLL_DIV_ VAL	GPU PLL clock divider value, for test purpose. 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
11 -	This field is reserved. Reserved
10–8 VIDEO_PLL1_ DIV_VAL	VIDEO PLL1 clock divider value, for test purpose. 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
7 -	This field is reserved. Reserved

Table continues on the next page...

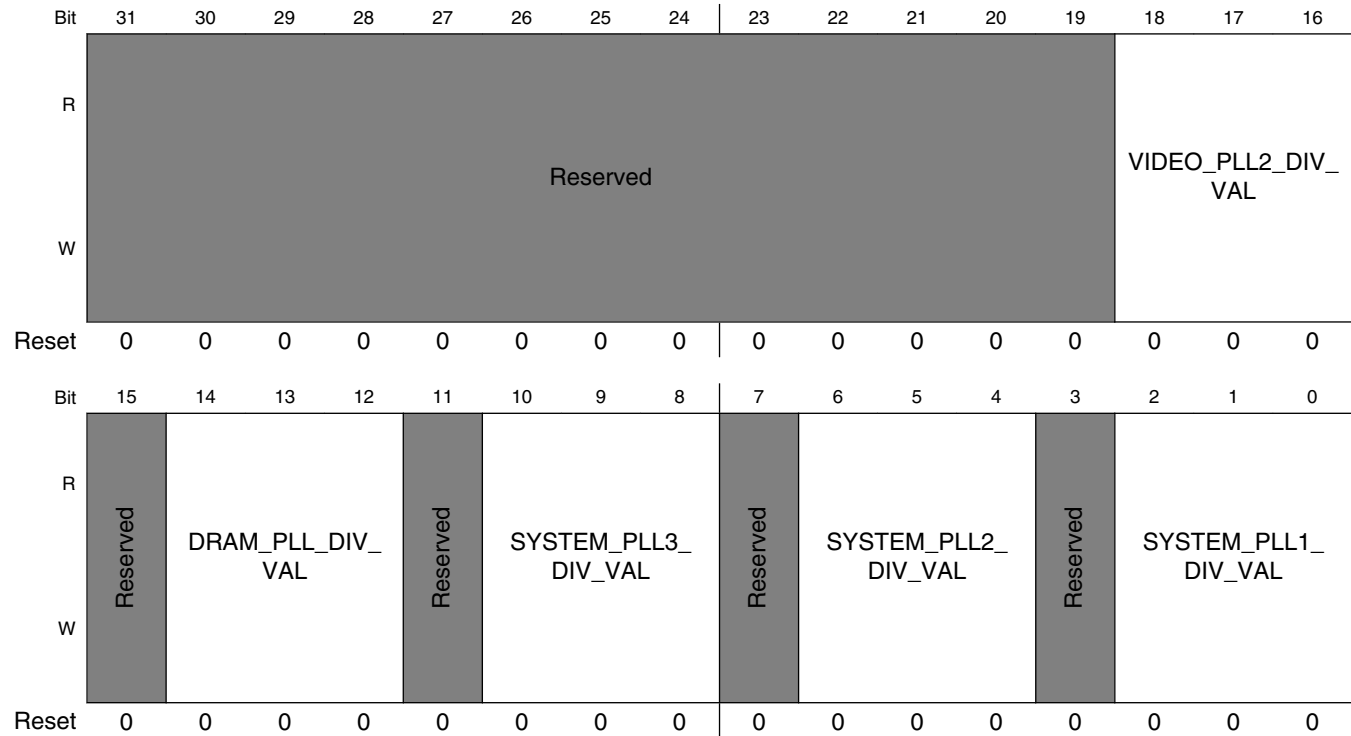
CCM_ANALOG_FRAC_PLLOUT_DIV_CFG field descriptions (continued)

Field	Description
6-4 AUDIO_PLL2_ DIV_VAL	AUDIO PLL2 clock divider value, for test purpose. 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
3 -	This field is reserved. Reserved
AUDIO_PLL1_ DIV_VAL	AUDIO PLL1 clock divider value, for test purpose. 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8

5.1.8.32 SCCG PLLOUT Divider Configuration Register (CCM_ANALOG_SCCG_PLLOUT_DIV_CFG)

SCCG PLLOUT Divider Configuration Register

Address: 3036_0000h base + 7Ch offset = 3036_007Ch



CCM_ANALOG_SCCG_PLLOUT_DIV_CFG field descriptions

Field	Description
31–19 -	This field is reserved. Reserved
18–16 VIDEO_PLL2_DIV_VAL	VIDEO PLL2 clock divider value, for test purpose. 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
15 -	This field is reserved. Reserved

Table continues on the next page...

CCM_ANALOG_SCCG_PLLOUT_DIV_CFG field descriptions (continued)

Field	Description
14–12 DRAM_PLL_ DIV_VAL	DRAM PLL clock divider value, for test purpose. 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
11 -	This field is reserved. Reserved
10–8 SYSTEM_PLL3_ DIV_VAL	System PLL3 clock divider value, for test purpose. 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
7 -	This field is reserved. Reserved
6–4 SYSTEM_PLL2_ DIV_VAL	System PLL2 clock divider value, for test purpose. 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8
3 -	This field is reserved. Reserved
SYSTEM_PLL1_ DIV_VAL	System PLL1 clock divider value, for test purpose. 000 Divide by 1 001 Divide by 2 010 Divide by 3 011 Divide by 4 100 Divide by 5 101 Divide by 6 110 Divide by 7 111 Divide by 8

5.2 General Power Controller (GPC)

5.2.1 Overview

The General Power Controller (GPC) module controls the following functions:

- Provide low power mode control for A53 and M4 platform
- Provide Power domain management all Arm and SOC power domain
- Provide domain control mechanism based on A53 and M4 CPU domain
- Provide handshake with CCM for clock management in low power mode
- Provide handshake with SRC for power down and power up sequence
- Provide handshake with Analog for Deep Sleep Mode control

5.2.2 Features

The General Power Controller (GPC) module controls the following functions:

- Support programmable feature for WAIT/STOP/DSM low power mode
- Support time slot based power domain control
- Support flexible sleep and wakeup condition
- Support domain control for multi CPU platforms system
- All register accessed by IP bus
- Interface for the following IPs:
 - CCM – clock controller module
 - SRC – system reset controller
 - ANALOG – miscellaneous analog control

5.2.3 Block Diagram

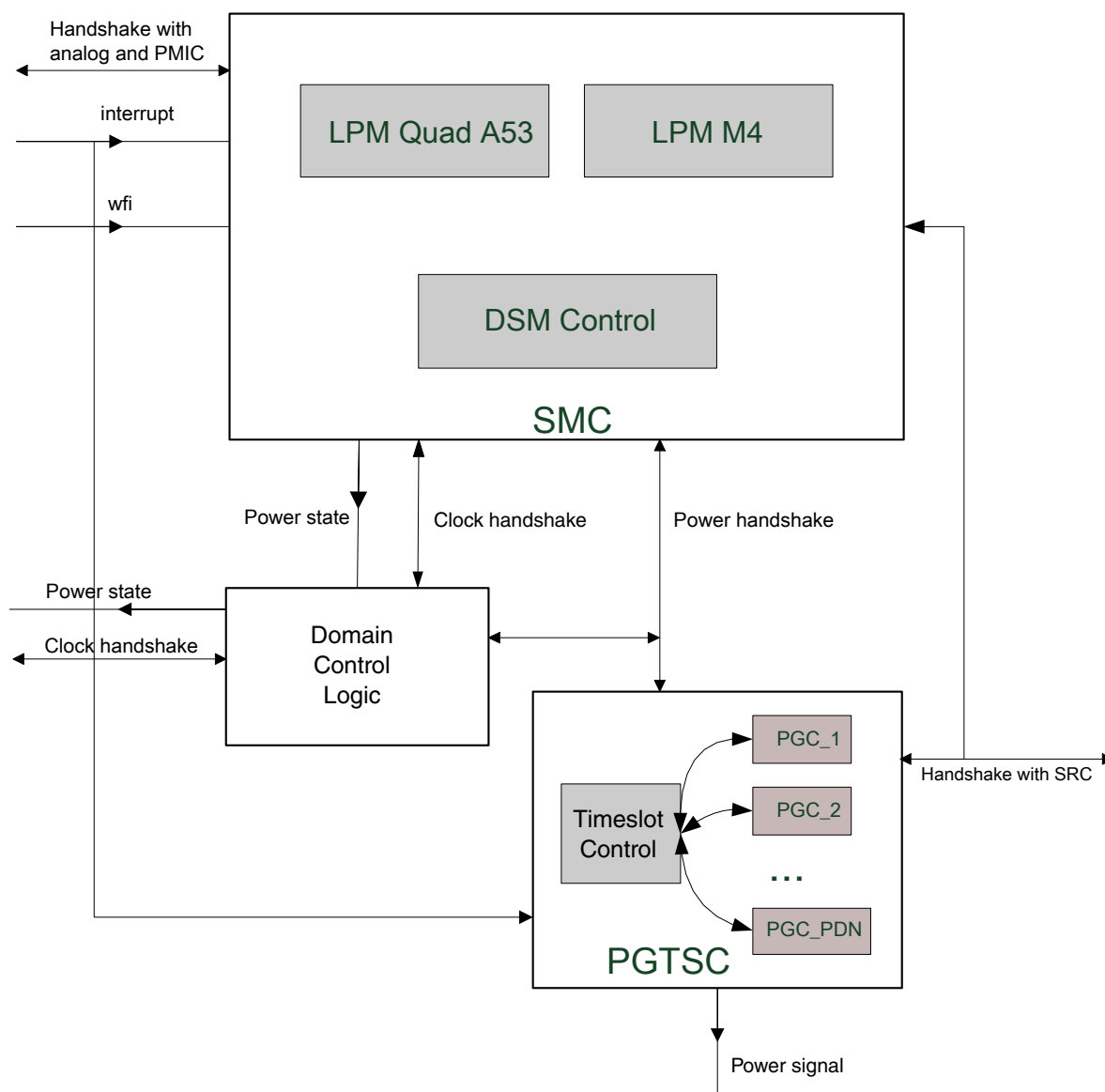


Figure 5-10. GPC Block Diagram

The GPC module contains two sub-modules: System Mode Controller (SMC) and Power Gating Time Slot Control (PGTSC):

- GPC Top: the top level GPC. It also includes the top memory map and registers, domain control information, and memory low power control.
- System Mode Controller (SMC):
 - The SMC supports two low power modes (LPM), WAIT and STOP. Each LPM corresponds to one mode for A53 platform and one mode for M4 platform.

- SMC controls the power sequence in Deep Sleep Mode (DSM)
- SMC support the power up and power down of A53 core0/core1/core2/core3 by IRQ/WFI signals without LPM triggered
- SMC can translate the LPM request for A53 and M4 platform to power up and power down request to PGTSC.
- Power Gating Time Slot Control (PGTSC):
 - The Power Gating Controller (PGC) is a power management component that controls the power-down and power-up sequencing of individual subsystems. For subsystems to be completely powered down in low power modes, a specific sequence of power control signals must be followed. The sequence timing is programmable using the PGC control registers.
 - There are 20 PGCs in the chip, all of them can be power down/up with a software trigger and all of them can be mapped to 20 timing slots and power-up and power down by request from SMC.

5.2.4 Functional Description

5.2.4.1 RUN mode

This is the normal/functional operating mode. In this mode, the CPU runs in its normal operational mode.

5.2.4.2 Low power mode

There are two CPU platforms (each of them represents a CPU domain): Quad core Cortex A53 platform and Cortex M4 platform. Each platform supports two low power modes: WAIT mode and STOP mode.

5.2.4.2.1 WAIT mode

In this low power mode:

- LPCG can be defined to be shut off or not in wait mode for each CPU domain
- PLL can be defined to be shut off or not in wait mode for each CPU domain

NOTE

The PLLs will only been closed in non-fast wake-up mode, relevant bit are

GPC_SLPCR[EN_A53_FASTWUP_WAIT_MODE] and
GPC_SLPCR[EN_M4_FASTWUP_WAIT_MODE]

- CPU clock can be defined been shut off or not in wait for each CPU platform.
(GPC_LPCR_A53_BSC[CPU_CLK_ON_LPM] and
GPC_LPCR_M4[CPU_CLK_ON_LPM])
- Power of different power domain can be defined be shut off or not in wait mode for each platform domain
- Some peripherals may go to wait mode along with A53 or M4 platform.

5.2.4.2.2 STOP mode

In this low power mode:

- LPCG can be defined been shut off or not in stop mode for each CPU domain
- PLL can be defined been shut off or not in stop mode for each CPU domain

NOTE

The PLLs will only been closed in non-fast wake-up mode,
relevant bit are

GPC_SLPCR[EN_A53_FASTWUP_STOP_MODE] and
GPC_SLPCR[EN_M4_FASTWUP_STOP_MODE]

- CPU clock can be defined been shut off or not in stop for each CPU
(GPC_LPCR_A53_BSC[CPU_CLK_ON_LPM] and
GPC_LPCR_M4[CPU_CLK_ON_LPM])
- Power of different power domain can be defined be shut off or not in stop mode for each platform domain
- Some peripherals may go to stop mode along with A53 or M4 platform.

5.2.4.3 Deep Sleep Mode

The Deep Sleep Mode (DSM) is a system low power mode.

In this mode:

- On-chip OSC can be defined to be shut off or not in DSM (GPC_SLPCR[SBYOS])
- PMIC can be defined to be stand-by mode or not in DSM (GPC_SLPCR[VSTBY])
- Regulator can be defined to be BYPASS mode or not in DSM
(GPC_SLPCR[RBC_EN])
- Memory can be defined to go to retention mode or not in
DSM(GPC_MLPCR[MEMLP_CTL_DIS])
- A53 platform power (VDD_ARM) can be defined to be shut off or not in DSM.

NOTE

CCM configuration must make sure close all PLLs before system goes to DSM

NOTE

For the SoC to correctly power up after entering DSM, CCM_PLL_CTRLx must not be set to 0x0 or 0x3 for any domain in use.

5.2.4.4 LPM Sleep Process

CPU platform will go to WAIT/STOP under the following conditions:

- LPM registers (GPC_LPCR_A53_BSC[LPM0], GPC_LPCR_A53_BSC[LPM1], GPC_LPCR_A53_BSC2[LPM2], GPC_LPCR_A53_BSC2[LPM3], GPC_LPCR_M4[LPM]) are set to WAIT or STOP.

NOTE

Since A53 platform has four cores, the so each core has its own LPM register: GPC_LPCR_A53_BSC[LPM0] and GPC_LPCR_A53_BSC[LPM1], GPC_LPCR_A53_BSC2[LPM2] and GPC_LPCR_A53_BSC2[LPM3]. The unified LPM of A53 will be generated with the lower LPM of the cores.

- Asserting the WFI signal will trigger CPU sleep process. There are five WFIs that come from A53 platform: WFI_core0, WFI_core1, WFI_core2, WFI_core3, and WFI_scu. The A53 platform will go to LPM when all WFIs are asserted. If the GPC_LPCR_A53_AD[EN_C0_WFI_PDN] or GPC_LPCR_A53_AD[EN_C1_WFI_PDN] or GPC_LPCR_A53_AD[EN_C2_WFI_PDN] or GPC_LPCR_A53_AD[EN_C3_WFI_PDN] bit is set, the LPM trigger condition will be a little different. See [Power control for A53 Platform](#) for more information. Only one WFI comes from M4 platform. The M4 platform will go to LPM when WFI_M4 asserted.

NOTE

WFI condition can be masked by register bits GPC_LPCR_A53_BSC[MASK_n_WFI] and GPC_LPCR_M4[MASK_M4_WFI]

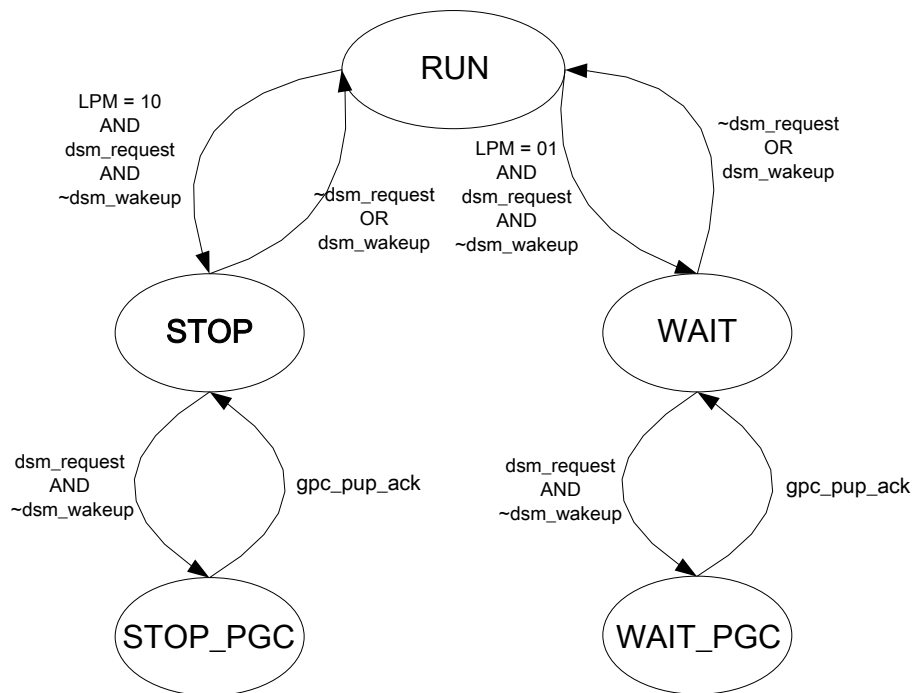


Figure 5-11. LPM transition inside CPU platform

System will go to DSM under the following conditions:

- Both A53 and M4 are STOP mode.
- Both GPC_SLPCR[EN_A53_FASTWUP_STOP_MODE] and GPC_SLPCR[EN_M4_FASTWUP_STOP_MODE] are not set.
- GPC_SLPCR[EN_DSM] is set.

NOTE

If GPC_LPCR_M4[MASK_DSM_TRIGGER] is set, the system will go to DSM when A53 goes STOP mode and GPC_SLPCR[EN_A53_FASTWUP_STOP_MODE] is not set. If GPC_LPCR_A53_BSC[MASK_DSM_TRIGGER] is set, the system will go to DSM when M4 goes to STOP mode and GPC_SLPCR[EN_M4_FASTWUP_STOP_MODE] not set. GPC_LPCR_M4[MASK_DSM_TRIGGER] and GPC_LPCR_A53_BSC[MASK_DSM_TRIGGER] cannot be set at the same time.

5.2.4.5 LPM Wake Up Process

DSM, STOP, WAIT mode will be woken up by interrupts:

- The CPU platforms share the same IRQ sources in this chip. Software can use GPC_IMRn_CORE0_A53, GPC_IMRn_CORE1_A53, GPC_IMRn_CORE2_A53, GPC_IMRn_CORE3_A53, and GPC_IMRn_M4 to separate the 128 bits IRQ sources to A53 core0, core1, core2, and core3, and M4 platform.
- The A53 core0, core1, core2, and core3 IRQ can also be from GIC source (defined by GPC_LPCR_A53_BSC[IRQ_SRC_C3], GPC_LPCR_A53_BSC[IRQ_SRC_C2], GPC_LPCR_A53_BSC[IRQ_SRC_C1], and GPC_LPCR_A53_BSC[IRQ_SRC_C0]) and if it is chosen from GIC source the GPC_IMRn_x_A53 will lose its function. See [Power control for A53 Platform](#) for more information.
- Interrupts for both A53 and M4 will cause the system wake up from DSM, A53 interrupt will wake up A53 from LPM, M4 interrupt will wake up M4 from LPM.

5.2.5 Power Gating Controller (PGC) Overview

The Power Gating Controller (PGC) is a power management component that controls the power-down and power-up sequencing of individual subsystems. For subsystems to be completely powered down in low power modes, a specific sequence of power control signals must be followed. The sequence timing is programmable using the PGC control registers.

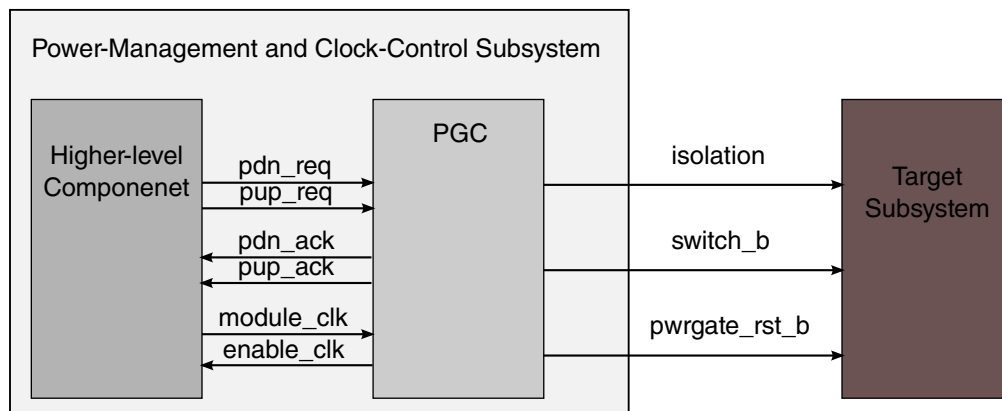


Figure 5-12. Power Gating Controller (PGC)

5.2.5.1 PGC power domains

The following table lists the PGCs in the chip and the corresponding power domain. There are three types of PGC - CPU, MIX, and PU. The power for the PUs (Power Units) can be controlled by the GPC.

PGC type	Type	Power domain
PGC_C0	CPU	Core0 of A53 platform
PGC_C1	CPU	Core1 of A53 platform
PGC_C2	CPU	Core2 of A53 platform
PGC_C3	CPU	Core3 of A53 platform
PGC_SCU	CPU	SCU/L2 cache RAM of A53 platform
PGC_MF	MIX	Fastmix and Megamix
PGC_OTG1	PU	USB OTG1 PHY
PGC_OTG2	PU	USB OTG2 PHY
PGC_PCIE	PU	PCIE1 PHY
PGC_MIPI	PU	MIPI DSI PHY
PGC_DDR1	PU	DDR1
PGC_DDR2	PU	DDR2
PGC_VPU	PU	VPU
PGC_GPU	PU	GPU
PGC_HDMI	PU	HDMI
PGC_DISP	PU	DISP
PGC_MIPI_CSI1	PU	MIPI CSI1 PHY
PGC_MIPI_CSI2	PU	MIPI CSI2 PHY
PGC_PCIE2	PU	PCIE2 PHY

5.2.5.2 Trigger to PGC: Hardware and Software Requests

All PGCs (except fastmix/megamix PGC) can be power up/down by hardware or software request. Fastmix/megamix PGC can only be power up/down by hardware request.

The LPM controller for each platform can generate hardware power down or power up request. All hardware requests will be mapped to “timeslot controller” before they goes to relevant PGC (see “Time slot control for PGCs” for more information).

The CPU can also generate software power up or power down request to relevant PGCs. The software trigger will not be mapped to timeslot control. If there are PGCs in software PDN/PUP sequence the request from LPM will be masked. The software trigger will also be failed if the timeslot control is in “busy” state.

All power up/down request to PGCs will be mapped to domain control module (see “Domain control for PGCs ”).

PGC_C0, PGC_C1, PGC_C2, and PGC_C3 can be triggered by its own “WFI/IRQ” without LPM trigger and time slot. See [Power control for A53 Platform](#) for more information.

5.2.5.3 Time slot control mechanism for PGCs

GPC uses a time slot controller to control the PGC sub-systems, such as the PGC in CPU0, CPU1, CPU2, CPU3, MIX, PCIE PHY, etc. We use it for below reasons when system wakes from low power mode.

1. Support flexible power down/up sequence for different sub-system
2. Sub-system can be power up in different slot to avoid large ramping up current in case they start ramping up at the same time

There are a total of 21 time slots used in the chip, one or more PGCs are used for power up or power down in each of these slots. The time slot controller will sample power up/down requests at slot0. If there are power up/down requests from SMC, it will scan from slot0 to slot20. When the scan process comes to one slot which has a defined power up or power down for one or more PGCs (defined by “SLTn_CFG”), the power up/down sequence of relevant PGCs will happen in that slot. Otherwise, the relevant slot will be skipped.

The next slot will not begin until the all PGCs finish their power up or power down process in current time slot. When all the 20 slots are finished, slot controller will jump to IDLE state and monitor new request from SMC.

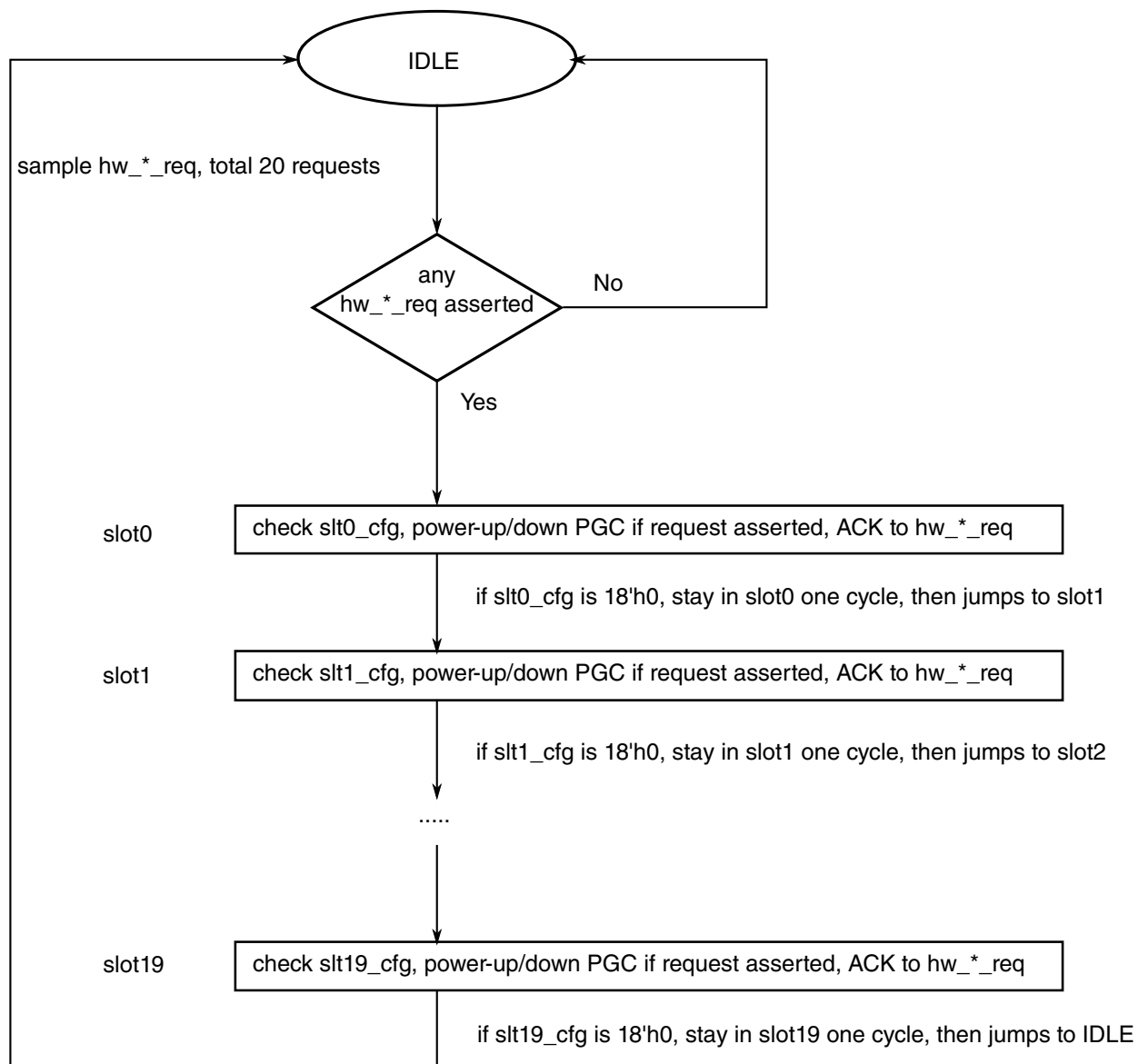


Figure 5-13. Slot controller processing flow

NOTE

PGC_SCU should be “always-on” to PGC_C0 PGC_C1, PGC_C2, and PGC_C3. This means PGC_SCU should be power up earlier than PGC_C0/PGC_C1/PGC_C2/PGC_C3 and should be power down later than PGC_C0/PGC_C1/PGC_C2/PGC_C3 (see example code 1 and 2). If we arrange A53 Cx/A53 SCU power down/up in same slot, special setting is required (see example code 2).

NOTE

When the system enters/exists ALL_OFF or L2_RETENTION mode, PGC_MF should be power up earlier than PGC_C0/PGC_C1/PGC_C2/PGC_C3/PGC_SCU. We can arrange MIX PGC power up in earlier slot than A53 Cx/SCU power up slot (See example code 1 and 2).

5.2.5.4 Handshake between LPM controller and time slot controller

The figure below shows an example of A53 “into” LPM sequence. We want to power up fastmix/megamix, A53 SCU and A53 core0 in time slot0 /slot1/slot2 respectively. Request from low power mode controller will be mapped to relevant PGCs according to the rules listed above. We choose acknowledge from PGC_core0 as the acknowledge for A53 LPM power down request (relevant register bits are defined in “PGC_ACK_SEL_A53”). The A53 LPM will regard the three PGCs as a virtual big PGC and it will cancel all power up request when it receive the acknowledge signals from PGC_core0.

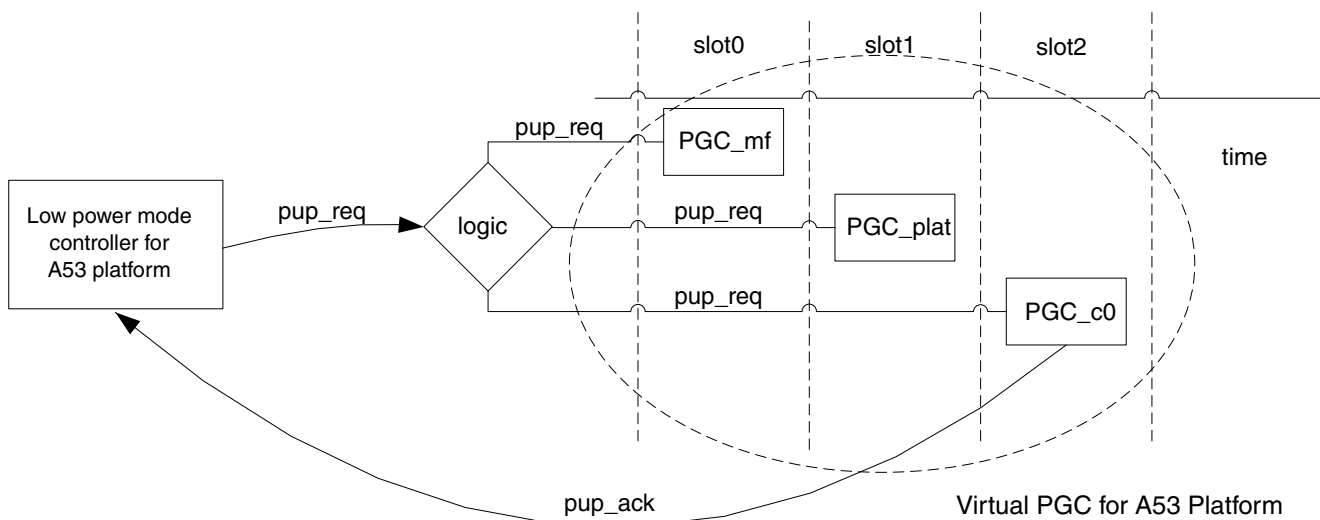


Figure 5-14. A53 into LPM sequence

NOTE

If a PGC is mapped to two CPU domain (refer to “Domain control for PGCs ”for more information), it cannot be selected as the power down acknowledge for both of the CPU platform. “PGC_ACK_SEL_A53”, “PGC_ACK_SEL_A53_PU”, and “PGC_ACK_SEL_M4” are should be chosen for the last PGC in power up or power down sequence in the time slot. If there is no PGC be power up/power down with LPM sequence, the

“dummy” acknowledge should be selected. Only one PGC should be selected for power down or power up acknowledge for one CPU platform.

5.2.6 Power control for A53 Platform

5.2.6.1 A53 Platform power domains and power modes

There are six power domains inside SEC dual core Cortex A53 platform: Core0, Core1, Core2, Core3, SCU, and L2 RAM. There are six power states in A53 platform:

Power State	PDCPU0	PDCPU1	PDCPU2	PDCPU3	PDPLAT	PDL2	VDD_ARM
ALL_ON	ON	ON	ON	ON	ON	ON	ON
THREE_CPU_ON	3 CPUs are ON, 1 CPU is OFF				ON	ON	ON
TWO_CPU_ON	2 CPUs are ON, 2 CPUs are OFF				ON	ON	ON
ONE_CPU_ON	1 CPU is ON, 3 CPUs are OFF				ON	ON	ON
ALL_CPU_OFF	OFF	OFF	OFF	OFF	ON	ON	ON
L2_RETENTION	OFF	OFF	OFF	OFF	OFF	RET	ON
ALL_OFF	OFF	OFF	OFF	OFF	OFF	OFF	ON
POWER_DOWN	OFF	OFF	OFF	OFF	OFF	OFF	OFF

NOTE

In all six power states, “ALL_ON”, “THREE_CPU_ON”, “TWO_CPU_ON”, and “ONE_CPU_ON” can exist in all RUN, WAIT or STOP mode of A53 platform. “L2_RETENTION” and “ALL_OFF” can only exist in WAIT or STOP mode of A53 platform.

5.2.6.2 Power down process for the A53 Platform

5.2.6.2.1 Power down of Core0, Core1, Core2, and Core3 in the A53 Platform

The power of core0, core1, core2, and core3 can be shut off along with the LPM process, as show in the following figure:

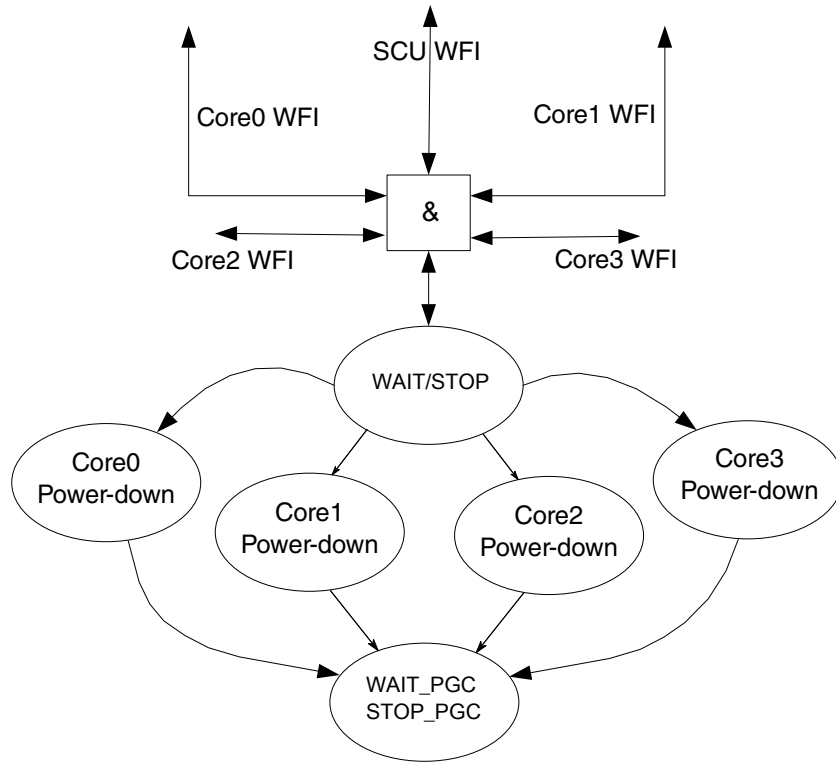


Figure 5-15. Power down of Core0, Core1, Core2, and Core3

WFIs from A53 platform will trigger the A53 platform LPM and the power of core0, core1, core2, or core3 will be shut off when “lpcr_a53_ad.en_c0_pdn”, “lpcr_a53_ad.en_c1_pdn”, “lpcr_a53_ad.en_c2_pdn”, or “lpcr_a53_ad.en_c3_pdn” enabled in this process. This mode should be used when core0 is used as the leading core of A53 platform.

The power of core0, core1, core2, and core3 can also be shut off in RUN mode: in this mode “LPCR_A53_AD.en_c0_wfi_pdn”, “LPCR_A53_AD.en_c1_wfi_pdn”, “LPCR_A53_AD.en_c2_wfi_pdn”, and “LPCR_A53_AD.en_c3_wfi_pdn” should be set and the condition to trigger A53 LPM will be some different:

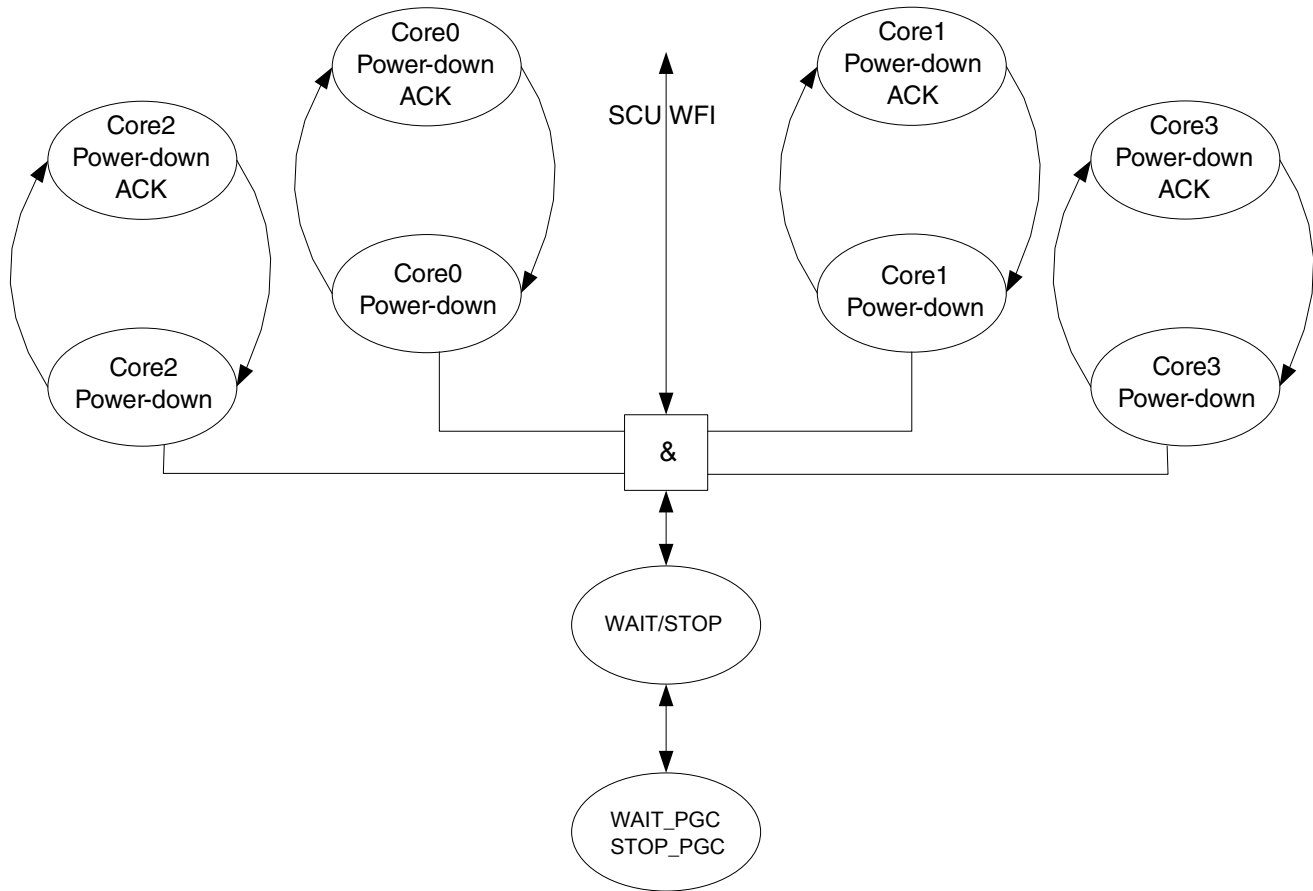


Figure 5-16. Power down of Core0, Core1, Core2, and Core3 in RUN mode

In this mode, core0/core1/core2/core3 power down process should not be disturbed by IRQs.

5.2.6.2.2 Power down of SCU and L2 Cache RAM

Power domain SCU and L2 cache RAM is controlled by one PGC – “PGC_PLAT” and they can only be power down in LPM process (when “LPCR_A53_AD.en_plat_pdn” is set). There is another bit “LPCR_A53_AD.l2pge” which will decide if L2 cache RAM need to be in retention mode when SCU domain is power down.

5.2.6.3 Power up process for the A53 Platform

- The core0, core1, core2, and core3 can be powered up with the exit of A53 LPM. The relevant bits are “LPCR_A53_AD.en_c0_pup”, “LPCR_A53_AD.en_c1_pup”, “LPCR_A53_AD.en_c2_pup”, and “LPCR_A53_AD.en_c3_pup”.
- The core0, core1, core2, and core3 can also be powered up by interrupt signal in RUN mode.

- The relevant bit are “LPCR_A53_AD.en_c0_irq_pup”, “LPCR_A53_AD.en_c1_irq_pup”, “LPCR_A53_AD.en_c2_irq_pup”, and “LPCR_A53_AD.en_c3_irq_pup”.
- The interrupt signal can be chosen from GIC or directly from IRQ with mask in GPCv2.
- The relevant select bits are “LPCR_A53_BSC.irq_src_c0”, “LPCR_A53_BSC.irq_src_c1”, “LPCR_A53_BSC.irq_src_c2”, “LPCR_A53_BSC.irq_src_c3”, and “LPCR_A53_BSC.irq_src_a53_wup”. (LPCR_A53_BSC[30],LPCR_A53_BSC[23:22],LPCR_A53_BSC.)

{LPCR_A53_BSC[30],LPCR_A53_BSC[23:22],LPCR_A53_BSC[29:28]}	Usage	Restriction
5'b00000	Use IRQ trigger A53 LPM and use IRQ to power up core0 to core3	None
5'b01111	Use GIC trigger A53 LPM and GIC to power up core0 to core3	SCU cannot power down in LPM, CPU clock cannot stop in LPM
5'b11111	Use IRQ trigger A53 LPM and GIC to power up core0 to core3	SCU cannot power down in LPM

As show in the table above, core0/core1/core2/core3 can only be power up by its own interrupt in RUN mode of A53 platform.

There are three combination of

{LPCR_A53_BSC[30],LPCR_A53_BSC[23:22],LPCR_A53_BSC[29:28]}:

1. In the first case, “IMRn_CORE0_A53, IMRn_CORE1_A53, IMRn_CORE2_A53, IMRn_CORE3_A53” are used to separate the 128 bits interrupts for core0, core1, core2, and core3 of A53 platform and also used as the interrupt mask for A53 LPM.
2. In the second case, “IMRn_CORE0_A53, IMRn_CORE1_A53, IMRn_CORE2_A53, IMRn_CORE3_A53” are not used, GIC setting are used to separate interrupts for core0, core1, core2, and core3 of A53 platform and also used as the interrupt mask for A53 LPM.
3. In the third case, “IMRn_CORE0_A53, IMRn_CORE1_A53, IMRn_CORE2_A53, IMRn_CORE3_A53” is used as the mask for interrupt for A53 LPM, GIC setting are used to separate interrupts for core0, core1, core2, and core3.

5.2.7 Power control for the M4 Platform

M4 LPM is same as A53 LPM. M4 platform doesn't have its own power domain and exist as a part of Megamix. M4 LPM power down request cannot power down the fastmix/megamix directly. There is a virtual PGC reserved for M4, the virtual PGC will

do nothing else except generating an acknowledge signal and this signal can be chosen as the acknowledge signal for M4 platform. Make sure virtual PGC power up/down slot same with fastmix/megamix PGC power up/down slot (See example code 3).

5.2.8 Domain control for PGCs

The following rules are used for PGC power up/down with domain mapping control:

1. For PGCs inside CPU platform (since M4 doesn't have its own power domain, only PGCs in A53 platform are referred): for hardware trigger (including both power up and power down) only the LPM request from its own platform will take effect; for software trigger (including both power up and power down) (the relevant register bit are "CPU_PGC_SW_PUP_REQ" and "CPU_PGC_SW_PDN_REQ"), only the software running in its own platform will take effect.
2. For PGCs outside CPU platform(MIX and PU PGCs), register bits "PGC_CPU_MAPPING" will map MIX and PU PGCs to A53 or M4 CPU domain:
 - One PGC can be mapped to one or both of A53 and M4 domain;
 - For hardware power up request, if a PGC is mapped to any CPU domain, the PGC will be powered up when the corresponding CPU platform sends out its power up request.
 - For software power up, if a PGC is mapped any CPU platform, the PGC can be powered up by software running in corresponding CPU.(the relevant register bits are "MIX_PGC_SW_PUP_REQ" and "PU_PGC_SW_PUP_REQ")
 - For hardware power down, if a power domain is mapped to only one CPU domain, the relevant PGC can be powered down when the corresponding CPU platform sends out its power down request; If a power domain is mapped to both two CPU domain, when one CPU platform sends out its hardware power down request, the relevant PGC will power down with any of the following condition satisfied: the other CPU already in LPM; the other CPU want to power down relevant PGC (the relevant register are "A53_MIX_PDN_FLG, M4_MIX_PDN_FLG, A53_PU_PDN_FLG, M4_PU_PDN_FLG")
 - For software power down, if a power domain is mapped to only one CPU domain, the PGC can be powered down by software running in corresponding CPU. (the relevant register bits are "MIX_PGC_SW_PUP_REQ" and "PU_PGC_SW_PUP_REQ"). If a power domain is mapped to both two CPU domain, when one CPU platform sends out its software power down request, the relevant PGC will power down with any of the following condition satisfied: the other CPU already in LPM; the other CPU want to power down relevant PGC (the relevant register are "A53_MIX_PDN_FLG, M4_MIX_PDN_FLG, A53_PU_PDN_FLG, M4_PU_PDN_FLG")
 - The access of "PGC_CPU_MAPPING" is controlled by domain information from RDC

5.2.9 Example Code

Below are code examples for entering specific power scenarios

5.2.9.1 Example Code 1

```
//ARM enters into ALL_OFF(STOP) mode and enable DSM :
//after "wfi", MIX/C0/C1/C2/C3 power down in SLOT0, SCU power down in SLOT1 when
//after "GPT1_INT" arrived, MIX power up in SLOT2, SCU power up in SLOT3, C0 power
up in SLOT4
//IMRx_CORE0_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x30, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x34, 0xFF7FFFFF); //[23] : GPT1 used as wakeup
source
reg32_write(GPC_IPS_BASE_ADDR + 0x38, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x3C, 0xFFFFFFFF);
//IMRx_CORE1_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x40, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x44, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x48, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x4C, 0xFFFFFFFF);
//IMRx_CORE2_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x1C0, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1C4, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1C8, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1CC, 0xFFFFFFFF);
//IMRx_CORE3_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x1D0, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1D4, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1D8, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1DC, 0xFFFFFFFF);
//LPCR_A53_BSC
reg32_write(GPC_IPS_BASE_ADDR, 0x0000000A) ;
//[30],[23:22],[29:28] : A53_C0/A53_C1/A53_C2/A53_C3/LPM wakeup from external INT
//[14] : CLOCK OFF during STOP mode
//[3:0] : STOP mode
//LPCR_A53_BSC2
reg32_write(GPC_IPS_BASE_ADDR + 0x108, 0x0000000A) ;
//[3:0] : STOP mode
//LPCR_A53_AD
reg32_write(GPC_IPS_BASE_ADDR + 0x4, 0x0A0B0A1A) ;
//[16] : 1(ALL_OFF mode); 0(L2 retention mode)
//[27]/[25]/[11]/[9]: A53_C0/A53_C1/A53_C2/A53_C3 power up with A53 LPM PUP REQ
//[4] : A53_SCU power down with A53 LPM PDN REQ
//[19]/[17]/[3]/[1] : A53_C0/A53_C1/A53_C2/A53_C3 powr down with A53 LPM PDN REQ
//LPCR_M4
reg32_write(GPC_IPS_BASE_ADDR + 0x8, 0x80000000) ;
//[31] : DSM ignore to check M4 low power state
//[1:0] : M4 LPM run mode
//SLPCR
reg32_write(GPC_IPS_BASE_ADDR + 0x14, 0xe000ffa7) ;
//[31] : enable DSM
//[30] : enable regulator bypass
//[5:3] : wait 64 ckil clock cycles
//[2] : enable PMIC standby
//[1] : enable OSC power down
//[0] : bypass PMIC ready handshake
//PGC_ACK_SEL_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x24, 0x00010004) ;
//[2] : A53_SCU PGC as LPM power down ack
//[16] : A53_C0 PGC as LPM power up ack
```

General Power Controller (GPC)

```
//SLT_CFG0
reg32_write(GPC_IPS_BASE_ADDR + 0xB0, 0x00000055) ;
//[6]/[4]/[2]/[0] : A53_C0/A53_C1/ A53_C2/A53_C3 power down in SLOT0
//SLT_CFG0_PU
reg32_write(GPC_IPS_BASE_ADDR + 0x200, 0x00000001) ;
//[0] : fastmix/megamix power down in SLOT0
//SLT_CFG1
reg32_write(GPC_IPS_BASE_ADDR + 0xB4, 0x00000100) ;
//[8] : A53_SCU power down in SLOT1
//SLT_CFG2_PU
reg32_write(GPC_IPS_BASE_ADDR + 0x208, 0x00000002) ;
//[1] : fastmix/megamix power up in SLOT2
//SLT_CFG3
reg32_write(GPC_IPS_BASE_ADDR + 0xBC, 0x00000200) ;
//[9] : A53_SCU power up in SLOT3
//SLT_CFG4
reg32_write(GPC_IPS_BASE_ADDR + 0xC0, 0x00000002) ;
//[1] : A53_C0 power up in SLOT4, A53_C1/ A53_C2/A53_C3 not power up
//PGC_CPU_MAPPING
reg32_write(GPC_IPS_BASE_ADDR + 0xEC, 0x00000001) ;
//[0] : MIX PGC mapping to A53 LPM
//A53 PGC
reg32_write(GPC_IPS_BASE_ADDR + 0x800, reg32_read(GPC_IPS_BASE_ADDR + 0x800) |
0x00000001) ;
// enable A53_C0 PGC power down
reg32_write(GPC_IPS_BASE_ADDR + 0x840, reg32_read(GPC_IPS_BASE_ADDR + 0x840) |
0x00000001) ;
// enable A53_C1 PGC power down
reg32_write(GPC_IPS_BASE_ADDR + 0x880, reg32_read(GPC_IPS_BASE_ADDR + 0x880) |
0x00000001) ;
// enable A53_C2 PGC power down
reg32_write(GPC_IPS_BASE_ADDR + 0x8C0, reg32_read(GPC_IPS_BASE_ADDR + 0x8C0) |
0x00000001) ;
// enable A53_C3 PGC power down
reg32_write(GPC_IPS_BASE_ADDR + 0x900, reg32_read(GPC_IPS_BASE_ADDR + 0x900) |
0x00000001) ;
// enable A53_SCU PGC power down
reg32_write(GPC_IPS_BASE_ADDR + 0x910, (0x59 << 10) | 0x5B | (0x51 << 20) ) ;
// change nL2retn/mempwr/dftrm to meet SCU power up timing
//fastmix/megamix PGC
reg32_write(GPC_IPS_BASE_ADDR + 0xA00, reg32_read(GPC_IPS_BASE_ADDR + 0xA00) |
0x00000001) ;
// enable MIX PGC power down
```

5.2.9.2 Example Code 2

```
//ARM enters into L2_RETENTION(STOP) mode and enable DSM :
//after "wfi", MIX/C0/C1/C2/C3/SCU power down in SLOT0
//after "GPT1_INT" arrived, MIX power up in SLOT1, SCU/C0 power up in SLOT2
//IMRx_CORE0_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x30, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x34, 0xFF7FFFFFFF);
//[23] : GPT1 used as wakeup source
reg32_write(GPC_IPS_BASE_ADDR + 0x38, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x3C, 0xFFFFFFFF);
//IMRx_CORE1_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x40, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x44, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x48, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x4C, 0xFFFFFFFF);
//IMRx_CORE2_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x1C0, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1C4, 0xFFFFFFFF);
```

```

reg32_write(GPC_IPS_BASE_ADDR + 0x1C8, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1CC, 0xFFFFFFFF);
//IMRx_CORE3_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x1D0, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1D4, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1D8, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1DC, 0xFFFFFFFF);
//LPCR_A53_BSC
reg32_write(GPC_IPS_BASE_ADDR, 0x0000000A) ;
//[30],[23:22],[29:28] : A53_C0/A53_C1/A53_C2/A53_C3/LPM wakeup from external INT
//[14] : CLOCK OFF during STOP mode
//[3:0] : STOP mode
//LPCR_A53_BSC2
reg32_write(GPC_IPS_BASE_ADDR + 0x108, 0x0000000A) ;
//[3:0] : STOP mode
//LPCR_A53_AD
reg32_write(GPC_IPS_BASE_ADDR + 0x4, 0x0A0A0A1A) ;
//[16] : 1(ALL_OFF mode); 0(L2 retention mode)
//[27]/[25]/[11]/[9] : A53_C0/A53_C1/A53_C2/A53_C3 power up with A53 LPM PUP REQ
//[4] : A53_SCU power down with A53 LPM PDN REQ
//[19]/[17]/[3]/[1] : A53_C0/A53_C1/A53_C2/A53_C3 powr down with A53 LPM PDN REQ
//LPCR_M4
reg32_write(GPC_IPS_BASE_ADDR + 0x8, 0x80000000) ;
//[31] : DSM ignore to check M4 low power state
//[1:0] : M4 LPM run mode
//SLPCR
reg32_write(GPC_IPS_BASE_ADDR + 0x14, 0xe000ffa7) ;
//[31] : enable DSM
//[30] : enable regulator bypass
//[5:3] : wait 64 ckil clock cycles
//[2] : enable PMIC standby
//[1] : enable OSC power down
//[0] : bypass PMIC ready handshake
//PGC_ACK_SEL_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x24, 0x00010004) ;
//[2] : A53_SCU PGC as LPM power down ack
//[16] : A53_C0 PGC as LPM power up ack
//SLT_CFG0
reg32_write(GPC_IPS_BASE_ADDR + 0xB0, 0x00000155) ;
//[6]/[4]/[2]/[0] : A53_C0/A53_C1/ A53_C2/A53_C3 power down in SLOT0
//[8] : A53_SCU power down in SLOT0
//SLT_CFG0_PU
reg32_write(GPC_IPS_BASE_ADDR + 0x200, 0x00000001) ;
//[0] : fastmix/megamix power down in SLOT0
//A53_Cx/SCU are power down in same slot. Special setting is required( see below
PGC setting #A )
//SLT_CFG1_PU
reg32_write(GPC_IPS_BASE_ADDR + 0x204, 0x00000002) ;
//[1] : fastmix/megamix power up in SLOT1
//SLT_CFG2
reg32_write(GPC_IPS_BASE_ADDR + 0xB8, 0x00000202) ;
//[9] : A53_SCU power up in SLOT1
//[1] : A53_C0 power up in SLOT1, A53_C1/ A53_C2/ A53_C3 not power up
//A53_Cx/SCU are power up in same slot. Special setting is required( see below
PGC setting #B )
//PGC_CPU_MAPPING
reg32_write(GPC_IPS_BASE_ADDR + 0xEC, 0x00000001) ;
//[0] : MIX PGC mapping to A53 LPM
//Special PGC setting #A for C0/C1/C2/C3/SCU power down in same slot(SCU should
be always ON comparing to C0/C1/C2/C3)
reg32_write(GPC_IPS_BASE_ADDR + 0x808, (reg32_read(GPC_IPS_BASE_ADDR + 0x808) &
0xFFFFFC00) | 0x0801) ;
// set C0.ISO2SW = 8 ; C0.ISO = 1;
reg32_write(GPC_IPS_BASE_ADDR + 0x848, (reg32_read(GPC_IPS_BASE_ADDR + 0x848) &
0xFFFFFC00) | 0x0801) ;
// set C1.ISO2SW = 8 ; C1.ISO = 1;
reg32_write(GPC_IPS_BASE_ADDR + 0x888, (reg32_read(GPC_IPS_BASE_ADDR + 0x888) &
0xFFFFFC00) | 0x0801) ;
// set C2.ISO2SW = 8 ; C2.ISO = 1;
reg32_write(GPC_IPS_BASE_ADDR + 0x8C8, (reg32_read(GPC_IPS_BASE_ADDR + 0x8C8) &

```

General Power Controller (GPC)

```
0xFFFFC0C0) | 0x0801 ) ;
// set C3.ISO2SW = 8 ; C3.ISO = 1;
reg32_write(GPC_IPS_BASE_ADDR +0x908, (reg32_read(GPC_IPS_BASE_ADDR +0x908) &
0xFFFFC0C0) | 0x1001 ) ;
// set SCU.ISO2SW = 16 ; SCU.ISO = 1;
//Special PGC setting #B for A53_Cx/SCU power up in same slot(SCU should be
always ON comparing to C0/C1/C2/C3)
reg32_write(GPC_IPS_BASE_ADDR +0x804, (reg32_read(GPC_IPS_BASE_ADDR +0x804) &
0xFF800040) | 0x11 | (0x20 << 7) ) ;
// set C0.SW = 0x11 ; C0.SW2ISO = 0x20 ;
reg32_write(GPC_IPS_BASE_ADDR +0x844, (reg32_read(GPC_IPS_BASE_ADDR +0x844) &
0xFF800040) | 0x11 | (0x20 << 7) ) ;
// set C1.SW = 0x11 ; C1.SW2ISO = 0x20 ;
reg32_write(GPC_IPS_BASE_ADDR +0x884, (reg32_read(GPC_IPS_BASE_ADDR +0x884) &
0xFF800040) | 0x11 | (0x20 << 7) ) ;
// set C2.SW = 0x11 ; C2.SW2ISO = 0x20 ;
reg32_write(GPC_IPS_BASE_ADDR +0x8C4, (reg32_read(GPC_IPS_BASE_ADDR +0x8C4) &
0xFF800040) | 0x11 | (0x20 << 7) ) ;
// set C3.SW = 0x11 ; C3.SW2ISO = 0x20 ;
reg32_write(GPC_IPS_BASE_ADDR +0x904, (reg32_read(GPC_IPS_BASE_ADDR +0x904) &
0xFF800040) | 0x1 | (0x0f << 7) ) ;
// set SCU.SW = 0x1 ; SCU.SW2ISO = 0x0f ;
//A53 PGC
reg32_write(GPC_IPS_BASE_ADDR +0x800, reg32_read(GPC_IPS_BASE_ADDR +0x800) |
0x00000001) ;
// enable A53_C0 PGC power down
reg32_write(GPC_IPS_BASE_ADDR +0x840, reg32_read(GPC_IPS_BASE_ADDR +0x840) |
0x00000001) ;
// enable A53_C1 PGC power down
reg32_write(GPC_IPS_BASE_ADDR +0x880, reg32_read(GPC_IPS_BASE_ADDR +0x880) |
0x00000001) ;
// enable A53_C2 PGC power down
reg32_write(GPC_IPS_BASE_ADDR +0x8C0, reg32_read(GPC_IPS_BASE_ADDR +0x8C0) |
0x00000001) ;
// enable A53_C3 PGC power down
reg32_write(GPC_IPS_BASE_ADDR +0x900, reg32_read(GPC_IPS_BASE_ADDR +0x900) |
0x00000001) ;
// enable A53_SCU PGC power down
reg32_write(GPC_IPS_BASE_ADDR +0x910, (0x59 << 10) | 0x5B | (0x51 << 20) ) ;
// change nl2retn/mempwr/dftrm to meet SCU power up timing
//fastmix/megamix PGC
reg32_write(GPC_IPS_BASE_ADDR +0xA00, reg32_read(GPC_IPS_BASE_ADDR +0xA00) |
0x00000001) ;
// enable MIX PGC power down
```

5.2.9.3 Example Code 3

```
//A53/M4 both enters into low power mode. A53/M4 are in different master domain.
//MIX are mapping to both A53 and M4
//after A53/M4 enters into low power mode, MIX will be also power down.A53/M4
enters into low power mode any time
//either A53 or M4 exists from low power mode, MIX will be also power up
//IMRx_CORE0_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x30, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x34, 0xFF7FFFFF);
//[23] : GPT1 used as ARM wakeup source
reg32_write(GPC_IPS_BASE_ADDR + 0x38, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x3C, 0xFFFFFFFF);
//IMRx_CORE1_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x40, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x44, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x48, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x4C, 0xFFFFFFFF);
```



```

//IMRx_CORE2_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x1C0, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1C4, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1C8, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1CC, 0xFFFFFFFF);
//IMRx_CORE3_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x1D0, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1D4, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1D8, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x1DC, 0xFFFFFFFF);
//IMRx_M4
reg32_write(GPC_IPS_BASE_ADDR + 0x50, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x54, 0xFFBFFFFF);
//[22] : GPT2 used as M4 wakeup source
reg32_write(GPC_IPS_BASE_ADDR + 0x58, 0xFFFFFFFF);
reg32_write(GPC_IPS_BASE_ADDR + 0x5C, 0xFFFFFFFF);
//LPCR_A53_BSC
reg32_write(GPC_IPS_BASE_ADDR, 0x0000000A) ;
//[30],[23:22],[29:28] : A53_C0/A53_C1/A53_C2/A53_C3/LPM wakeup from external INT
//[14] : CLOCK OFF during STOP mode
//[3:0] : STOP mode
//LPCR_A53_BSC2
reg32_write(GPC_IPS_BASE_ADDR + 0x108, 0x0000000A) ;
//[3:0] : STOP mode
//LPCR_A53_AD
reg32_write(GPC_IPS_BASE_ADDR + 0x4, 0x0A0B0A1A) ;
//[16] : 1(ALL_OFF mode); 0(L2 retention mode)
//[27]/[25]/[11]/[9] : A53_C0/A53_C1/A53_C2/A53_C3 power up with A53 LPM PUP REQ
//[4] : A53_SCU power down with A53 LPM PDN REQ
//[19]/[17]/[3]/[1] : A53_C0/A53_C1/A53_C2/A53_C3 powr down with A53 LPM PDN REQ
//LPCR_M4
reg32_write(GPC_IPS_BASE_ADDR + 0x8, 0x00003FFE) ;
//[31] : 0(check M4 low power state to enter into DSM)
//[14] : 0(M4 clock OFF during low power mode)
//[3:2] : enable M4 virtual PGC power up/down with M4 LPM
//[1:0] : M4 LPM STOP mode
//SLPCR
reg32_write(GPC_IPS_BASE_ADDR + 0x14, 0xe00ffa7) ;
//[31] : enable DSM
//[30] : enable regulator bypass
//[5:3] : wait 64 ckil clock cycles
//[2] : enable PMIC standby
//[1] : enable OSC power down
//[0] : bypass PMIC ready handshake
//PGC_ACK_SEL_A53
reg32_write(GPC_IPS_BASE_ADDR + 0x24, 0x00010004) ;
//[2] : A53_SCU PGC as LPM power down ack
//[16] : A53_C0 PGC as LPM power up ack
//PGC_ACK_SEL_M4
reg32_write(GPC_IPS_BASE_ADDR + 0x28, 0x00010001) ;
//[0] : M4 virtual PGC as M4 LPM power down ack
//[16] : M4 virtual PGC as M4 LPM power up ack
//GPC_MISC
reg32_write(GPC_IPS_BASE_ADDR + 0x2C, reg32_read(GPC_IPS_BASE_ADDR + 0x2C) |
0x100) ;
//[8] : not mask M4 power down request to M4 virtual PGC
//SLT_CFG0
reg32_write(GPC_IPS_BASE_ADDR + 0xB0, 0x00000155) ;
//[6]/[4]/[2]/[0] : A53_C0/A53_C1/ A53_C2/A53_C3 power down in SLOT0
//[8] : A53_SCU power down in SLOT0
//SLT_CFG0_PU
reg32_write(GPC_IPS_BASE_ADDR + 0x200, 0x00010001) ;
//[0] : fastmix/megamix power down in SLOT0
//[12] : M4 virtual PGC power down in SLOT0, same with fastmix/megamix PGC power
down slot
//A53_Cx/SCU are power down in same slot. Special setting is required( see below
PGC setting #A )
//SLT_CFG1_PU
reg32_write(GPC_IPS_BASE_ADDR + 0x204, 0x00002002) ;
//[2] : fastmix/megamix power up in SLOT1

```

General Power Controller (GPC)

```
up slot      //[13] : M4 virtual PGC power up in SLOT1, same with fastmix/megamix PGC power
            //SLT_CFG2
            reg32_write(GPC_IPS_BASE_ADDR + 0xB8, 0x00000202) ;
            //[9] : A53_SCU power up in SLOT1
            //[1] : A53_C0 power up in SLOT1, A53_C1/ A53_C2/ A53_C3 not power up
            //A53_Cx/SCU are power up in same slot. Special setting is required( see below
PGC setting #B )
            //PGC_CPU_MAPPING
            reg32_write(GPC_IPS_BASE_ADDR + 0xEC, 0x00010001) ;
            //[0] : MIX PGC mapping to A53 LPM
            //[16] : MIX PGC mapping to M4 LPM
            //Special PGC setting #A for C0/C1/SCU power down in same slot(SCU should be
always ON comparing to C0/C1/C2/C3)
            reg32_write(GPC_IPS_BASE_ADDR + 0x808, (reg32_read(GPC_IPS_BASE_ADDR + 0x808) &
0xFFFFFC00) | 0x0801) ;
            // set C0.ISO2SW = 8 ; C0.ISO = 1;
            reg32_write(GPC_IPS_BASE_ADDR + 0x848, (reg32_read(GPC_IPS_BASE_ADDR + 0x848) &
0xFFFFFC00) | 0x0801) ;
            // set C1.ISO2SW = 8 ; C1.ISO = 1;
            reg32_write(GPC_IPS_BASE_ADDR + 0x848, (reg32_read(GPC_IPS_BASE_ADDR + 0x848) &
0xFFFFFC00) | 0x0801) ;
            // set C2.ISO2SW = 8 ; C2.ISO = 1;
            reg32_write(GPC_IPS_BASE_ADDR + 0x888, (reg32_read(GPC_IPS_BASE_ADDR + 0x888) &
0xFFFFFC00) | 0x0801) ;
            // set C3.ISO2SW = 8 ; C3.ISO = 1;
            reg32_write(GPC_IPS_BASE_ADDR + 0x908, (reg32_read(GPC_IPS_BASE_ADDR + 0x908) &
0xFFFFFC00) | 0x1001) ;
            // set SCU.ISO2SW = 16 ; SCU.ISO = 1;
            //Special PGC setting #B for A53_Cx/SCU power up in same slot(SCU should be
always ON comparing to C0/C1/C2/C3)
            reg32_write(GPC_IPS_BASE_ADDR + 0x804, (reg32_read(GPC_IPS_BASE_ADDR + 0x804) &
0xFF800040) | 0x10 | (0x1f << 7) ) ;
            // set C0.SW = 0x10 ; C0.SW2ISO = 0x1f ;
            reg32_write(GPC_IPS_BASE_ADDR + 0x844, (reg32_read(GPC_IPS_BASE_ADDR + 0x844) &
0xFF800040) | 0x10 | (0x1f << 7) ) ;
            // set C1.SW = 0x10 ; C1.SW2ISO = 0x1f ;
            reg32_write(GPC_IPS_BASE_ADDR + 0x844, (reg32_read(GPC_IPS_BASE_ADDR + 0x844) &
0xFF800040) | 0x10 | (0x1f << 7) ) ;
            // set C2.SW = 0x10 ; C2.SW2ISO = 0x1f ;
            reg32_write(GPC_IPS_BASE_ADDR + 0x884, (reg32_read(GPC_IPS_BASE_ADDR + 0x884) &
0xFF800040) | 0x10 | (0x1f << 7) ) ;
            // set C3.SW = 0x10 ; C3.SW2ISO = 0x1f ;
            reg32_write(GPC_IPS_BASE_ADDR + 0x904, (reg32_read(GPC_IPS_BASE_ADDR + 0x904) &
0xFF800040) | 0x1 | (0x0f << 7) ) ;
            // set SCU.SW = 0x1 ; SCU.SW2ISO = 0x0f ;
            //A53 PGC
            reg32_write(GPC_IPS_BASE_ADDR + 0x800, reg32_read(GPC_IPS_BASE_ADDR + 0x800) |
0x00000001) ;
            // enable A53_C0 PGC power down
            reg32_write(GPC_IPS_BASE_ADDR + 0x840, reg32_read(GPC_IPS_BASE_ADDR + 0x840) |
0x00000001) ;
            // enable A53_C1 PGC power down
            reg32_write(GPC_IPS_BASE_ADDR + 0x840, reg32_read(GPC_IPS_BASE_ADDR + 0x840) |
0x00000001) ;
            // enable A53_C2 PGC power down
            reg32_write(GPC_IPS_BASE_ADDR + 0x880, reg32_read(GPC_IPS_BASE_ADDR + 0x880) |
0x00000001) ;
            // enable A53_C3 PGC power down
            reg32_write(GPC_IPS_BASE_ADDR + 0x900, reg32_read(GPC_IPS_BASE_ADDR + 0x900) |
0x00000001) ;
            // enable A53_SCU PGC power down
            reg32_write(GPC_IPS_BASE_ADDR + 0x910, (0x59 << 10) | 0x5B | (0x51 << 20) ) ;
            // change nl2retn/mempwr/dftrm to meet SCU power up timing
            //fastmix/megamix PGC
            reg32_write(GPC_IPS_BASE_ADDR + 0xA00, reg32_read(GPC_IPS_BASE_ADDR + 0xA00) |
0x00000001) ;
            // enable MIX PGC power down
```

5.2.9.4 Example Code 4

```

// software power up/down PCIE PHY
//power up PCIE PHY
reg32_write ( GPC_IPS_BASE_ADDR + 0xEC, reg32_read(GPC_IPS_BASE_ADDR + 0xEC) |
0x8 );
//map PCIE PGC to A53
reg32_write ( GPC_IPS_BASE_ADDR + 0xF8 , reg32_read(GPC_IPS_BASE_ADDR + 0xF8) |
0x2 );
//trigger sw Power up Request
while( read(GPC_IPS_BASE_ADDR + 0xF8) & 0x2 );
//wait software power up request self clear
//power down PCIE PHY
reg32_write ( GPC_IPS_BASE_ADDR + 0xEC, reg32_read(GPC_IPS_BASE_ADDR + 0xEC) |
0x8 );
//map PCIE PGC to A53
reg32_write ( GPC_IPS_BASE_ADDR + 0xC40, reg32_read(GPC_IPS_BASE_ADDR + 0xC40) |
0x1 );
// enable PCIE PGC power down
reg32_write ( GPC_IPS_BASE_ADDR + 0x104 , reg32_read(GPC_IPS_BASE_ADDR + 0x104)
| 0x2 );
//trigger sw Power Down Request
while( read(GPC_IPS_BASE_ADDR + 0x104) & 0x2 );
//wait software power down request self clear

```

5.2.10 GPC Memory Map/Register Definition

Detailed descriptions of each register can be found below.

The total GPC memory map is 4KB

Table 5-10. Memory Regions

Address Range(offset)	Region
0x000 - 0x3FF	GPC configuration register
0x400 - 0x7FF	Reserved
0x800 - 0x9FF	CPU and SCU type PGC register base address
0xA00 - 0xBFF	MIX type PGC register base address
0xC00 - 0xFFF	PU type PGC register base address

Each PGC (CPU type, MIX type, PU type) will occupy 64 Bytes address space, the specific base address of each PGC are listed as below.

- 0x800 ~ 0x83F : PGC for A53 core0
- 0x840 ~ 0x87F: PGC for A53 core1
- 0x880 ~ 0x8BF: PGC for A53 core2

General Power Controller (GPC)

- 0x8C0 ~ 0x8FF: PGC for A53 core3
- 0x900 ~ 0x93F: PGC for A53 SCU
- 0xA00 ~ 0xA3F: PGC for fastmix/megamix
- 0xC00 ~ 0xC3F: PGC for MIPI DSI PHY
- 0xC40 ~ 0xC7F: PGC for PCIE1 PHY
- 0xC80 ~ 0xCBF: USB_OTG1
- 0xCC0 ~ 0xCFF: USB_OTG2
- 0xD00 ~ 0xD3F: Reserved
- 0xD40 ~ 0xD7F: DDR1
- 0xD80 ~ 0xDBF: Reserved
- 0xDC0 ~ 0xDFF: GPUMIX
- 0xE00 ~ 0xE3F: VPUMIX
- 0xE40 ~ 0xE7F: HDMI
- 0xE80 ~ 0xEBF: DISPMIX
- 0xEC0 ~ 0xEFF: MIPI CSI1 PHY
- 0xF00 ~ 0xF3F: MIPI CSI2 PHY
- 0xF40 ~ 0xF7F: PCIE2 PHY

For more specific information about PGC register definition, please see the register definition for each PGC.

GPC memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303A_0000	Basic Low power control register of A53 platform (GPC_LPCR_A53_BSC)	32	R/W	0000_3FF0h	5.2.10.1/ 686
303A_0004	Advanced Low power control register of A53 platform (GPC_LPCR_A53_AD)	32	R/W	0000_0020h	5.2.10.2/ 689
303A_0008	Low power control register of CPU1 (GPC_LPCR_M4)	32	R/W	0000_3FF0h	5.2.10.3/ 691
303A_0014	System low power control register (GPC_SLPCR)	32	R/W	E000_FF82h	5.2.10.4/ 693
303A_0018	MASTER LPM Handshake (GPC_MST_CPU_MAPPING)	32	R/W	0000_00FFh	5.2.10.5/ 695
303A_0020	Memory low power control register (GPC_MLPCR)	32	R/W	0101_0100h	5.2.10.6/ 696
303A_0024	PGC acknowledge signal selection of A53 platform (GPC_PGC_ACK_SEL_A53)	32	R/W	8000_8000h	5.2.10.7/ 697
303A_0028	PGC acknowledge signal selection of M4 platform (GPC_PGC_ACK_SEL_M4)	32	R/W	8000_8000h	5.2.10.8/ 699
303A_002C	GPC Miscellaneous register (GPC_MISC)	32	R/W	0000_0021h	5.2.10.9/ 700
303A_0030	IRQ masking register 1 of A53 core0 (GPC_IMR1_CORE0_A53)	32	R/W	0000_0000h	5.2.10.10/ 701

Table continues on the next page...

GPC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
303A_0034	IRQ masking register 2 of A53 core0 (GPC_IMR2_CORE0_A53)	32	R/W	0000_0000h	5.2.10.11/702
303A_0038	IRQ masking register 3 of A53 core0 (GPC_IMR3_CORE0_A53)	32	R/W	0000_0000h	5.2.10.12/702
303A_003C	IRQ masking register 4 of A53 core0 (GPC_IMR4_CORE0_A53)	32	R/W	0000_0000h	5.2.10.13/703
303A_0040	IRQ masking register 1 of A53 core1 (GPC_IMR1_CORE1_A53)	32	R/W	0000_0000h	5.2.10.14/703
303A_0044	IRQ masking register 2 of A53 core1 (GPC_IMR2_CORE1_A53)	32	R/W	0000_0000h	5.2.10.15/703
303A_0048	IRQ masking register 3 of A53 core1 (GPC_IMR3_CORE1_A53)	32	R/W	0000_0000h	5.2.10.16/704
303A_004C	IRQ masking register 4 of A53 core1 (GPC_IMR4_CORE1_A53)	32	R/W	0000_0000h	5.2.10.17/704
303A_0050	IRQ masking register 1 of M4 (GPC_IMR1_M4)	32	R/W	0000_0000h	5.2.10.18/705
303A_0054	IRQ masking register 2 of M4 (GPC_IMR2_M4)	32	R/W	0000_0000h	5.2.10.19/705
303A_0058	IRQ masking register 3 of M4 (GPC_IMR3_M4)	32	R/W	0000_0000h	5.2.10.20/705
303A_005C	IRQ masking register 4 of M4 (GPC_IMR4_M4)	32	R/W	0000_0000h	5.2.10.21/706
303A_0070	IRQ status register 1 of A53 (GPC_ISR1_A53)	32	R	0000_0000h	5.2.10.22/706
303A_0074	IRQ status register 2 of A53 (GPC_ISR2_A53)	32	R	0000_0000h	5.2.10.23/707
303A_0078	IRQ status register 3 of A53 (GPC_ISR3_A53)	32	R	0000_0000h	5.2.10.24/707
303A_007C	IRQ status register 4 of A53 (GPC_ISR4_A53)	32	R	0000_0000h	5.2.10.25/707
303A_0080	IRQ status register 1 of M4 (GPC_ISR1_M4)	32	R	0000_0000h	5.2.10.26/708
303A_0084	IRQ status register 2 of M4 (GPC_ISR2_M4)	32	R	0000_0000h	5.2.10.27/708
303A_0088	IRQ status register 3 of M4 (GPC_ISR3_M4)	32	R	0000_0000h	5.2.10.28/709
303A_008C	IRQ status register 4 of M4 (GPC_ISR4_M4)	32	R	0000_0000h	5.2.10.29/709
303A_00B0	Slot configure register for CPUs (GPC_SLT0_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00B4	Slot configure register for CPUs (GPC_SLT1_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00B8	Slot configure register for CPUs (GPC_SLT2_CFG)	32	R/W	0000_0000h	5.2.10.30/709

Table continues on the next page...

GPC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303A_00BC	Slot configure register for CPUs (GPC_SLT3_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00C0	Slot configure register for CPUs (GPC_SLT4_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00C4	Slot configure register for CPUs (GPC_SLT5_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00C8	Slot configure register for CPUs (GPC_SLT6_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00CC	Slot configure register for CPUs (GPC_SLT7_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00D0	Slot configure register for CPUs (GPC_SLT8_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00D4	Slot configure register for CPUs (GPC_SLT9_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00D8	Slot configure register for CPUs (GPC_SLT10_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00DC	Slot configure register for CPUs (GPC_SLT11_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00E0	Slot configure register for CPUs (GPC_SLT12_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00E4	Slot configure register for CPUs (GPC_SLT13_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00E8	Slot configure register for CPUs (GPC_SLT14_CFG)	32	R/W	0000_0000h	5.2.10.30/709
303A_00EC	PGC CPU mapping (GPC_PGC_CPU_0_1_MAPPING)	32	R/W	0000_0000h	5.2.10.31/712
303A_00F0	CPU PGC software power up trigger (GPC_CPU_PGC_SW_PUP_REQ)	32	R/W	0000_0000h	5.2.10.32/714
303A_00F4	MIX PGC software power up trigger (GPC_MIX_PGC_SW_PUP_REQ)	32	R/W	0000_0000h	5.2.10.33/715
303A_00F8	PU PGC software up trigger (GPC_PU_PGC_SW_PUP_REQ)	32	R/W	0000_0000h	5.2.10.34/716
303A_00FC	CPU PGC software down trigger (GPC_CPU_PGC_SW_PDN_REQ)	32	R/W	0000_0000h	5.2.10.35/717
303A_0100	MIX PGC software power down trigger (GPC_MIX_PGC_SW_PDN_REQ)	32	R/W	0000_0000h	5.2.10.36/718
303A_0104	PU PGC software down trigger (GPC_PU_PGC_SW_PDN_REQ)	32	R/W	0000_0000h	5.2.10.37/719
303A_0108	Basic Low power control register of A53 platform (GPC_LPCR_A53_BSC2)	32	R/W	0000_0000h	5.2.10.38/721
303A_0130	CPU PGC software up trigger status1 (GPC_CPU_PGC_PUP_STATUS1)	32	R	0000_0000h	5.2.10.39/722
303A_0134	A53 MIX software up trigger status register (GPC_A53_MIX_PGC_PUP_STATUS0)	32	R	0000_0000h	5.2.10.40/724

Table continues on the next page...

GPC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303A_0138	A53 MIX software up trigger status register (GPC_A53_MIX_PGC_PUP_STATUS1)	32	R	0000_0000h	5.2.10.40/724
303A_013C	A53 MIX software up trigger status register (GPC_A53_MIX_PGC_PUP_STATUS2)	32	R	0000_0000h	5.2.10.40/724
303A_0140	M4 MIX PGC software up trigger status register (GPC_M4_MIX_PGC_PUP_STATUS0)	32	R	0000_0000h	5.2.10.41/725
303A_0144	M4 MIX PGC software up trigger status register (GPC_M4_MIX_PGC_PUP_STATUS1)	32	R	0000_0000h	5.2.10.41/725
303A_0148	M4 MIX PGC software up trigger status register (GPC_M4_MIX_PGC_PUP_STATUS2)	32	R	0000_0000h	5.2.10.41/725
303A_014C	A53 PU software up trigger status register (GPC_A53_PU_PGC_PUP_STATUS0)	32	R	0000_0000h	5.2.10.42/727
303A_0150	A53 PU software up trigger status register (GPC_A53_PU_PGC_PUP_STATUS1)	32	R	0000_0000h	5.2.10.42/727
303A_0154	A53 PU software up trigger status register (GPC_A53_PU_PGC_PUP_STATUS2)	32	R	0000_0000h	5.2.10.42/727
303A_0158	M4 PU PGC software up trigger status register (GPC_M4_PU_PGC_PUP_STATUS0)	32	R	0000_0000h	5.2.10.43/730
303A_015C	M4 PU PGC software up trigger status register (GPC_M4_PU_PGC_PUP_STATUS1)	32	R	0000_0000h	5.2.10.43/730
303A_0160	M4 PU PGC software up trigger status register (GPC_M4_PU_PGC_PUP_STATUS2)	32	R	0000_0000h	5.2.10.43/730
303A_0170	CPU PGC software dn trigger status1 (GPC_CPU_PGC_PDN_STATUS1)	32	R	0000_0000h	5.2.10.44/733
303A_0174	A53 MIX software down trigger status register (GPC_A53_MIX_PGC_PDN_STATUS0)	32	R	0000_0000h	5.2.10.45/735
303A_0178	A53 MIX software down trigger status register (GPC_A53_MIX_PGC_PDN_STATUS1)	32	R	0000_0000h	5.2.10.45/735
303A_017C	A53 MIX software down trigger status register (GPC_A53_MIX_PGC_PDN_STATUS2)	32	R	0000_0000h	5.2.10.45/735
303A_0180	M4 MIX PGC software power down trigger status register (GPC_M4_MIX_PGC_PDN_STATUS0)	32	R	0000_0000h	5.2.10.46/737
303A_0184	M4 MIX PGC software power down trigger status register (GPC_M4_MIX_PGC_PDN_STATUS1)	32	R	0000_0000h	5.2.10.46/737
303A_0188	M4 MIX PGC software power down trigger status register (GPC_M4_MIX_PGC_PDN_STATUS2)	32	R	0000_0000h	5.2.10.46/737
303A_018C	A53 PU PGC software down trigger status (GPC_A53_PU_PGC_PDN_STATUS0)	32	R	0000_0000h	5.2.10.47/738
303A_0190	A53 PU PGC software down trigger status (GPC_A53_PU_PGC_PDN_STATUS1)	32	R	0000_0000h	5.2.10.47/738
303A_0194	A53 PU PGC software down trigger status (GPC_A53_PU_PGC_PDN_STATUS2)	32	R	0000_0000h	5.2.10.47/738
303A_0198	M4 PU PGC software down trigger status (GPC_M4_PU_PGC_PDN_STATUS0)	32	R	0000_0000h	5.2.10.48/741

Table continues on the next page...

GPC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
303A_019C	M4 PU PGC software down trigger status (GPC_M4_PU_PGC_PDN_STATUS1)	32	R	0000_0000h	5.2.10.48/741
303A_01A0	M4 PU PGC software down trigger status (GPC_M4_PU_PGC_PDN_STATUS2)	32	R	0000_0000h	5.2.10.48/741
303A_01B0	A53 MIX PDN FLG (GPC_A53_MIX_PDN_FLG)	32	R/W	0000_0000h	5.2.10.49/745
303A_01B4	A53 PU PDN FLG (GPC_A53_PU_PDN_FLG)	32	R/W	0000_0000h	5.2.10.50/746
303A_01B8	M4 MIX PDN FLG (GPC_M4_MIX_PDN_FLG)	32	R/W	0000_0000h	5.2.10.51/747
303A_01BC	M4 PU PDN FLG (GPC_M4_PU_PDN_FLG)	32	R/W	0000_0000h	5.2.10.52/748
303A_01C0	IRQ masking register 1 of A53 core2 (GPC_IMR1_CORE2_A53)	32	R/W	0000_0000h	5.2.10.53/748
303A_01C4	IRQ masking register 2 of A53 core2 (GPC_IMR2_CORE2_A53)	32	R/W	0000_0000h	5.2.10.54/749
303A_01C8	IRQ masking register 3 of A53 core2 (GPC_IMR3_CORE2_A53)	32	R/W	0000_0000h	5.2.10.55/749
303A_01CC	IRQ masking register 4 of A53 core2 (GPC_IMR4_CORE2_A53)	32	R/W	0000_0000h	5.2.10.56/750
303A_01D0	IRQ masking register 1 of A53 core3 (GPC_IMR1_CORE3_A53)	32	R/W	0000_0000h	5.2.10.57/750
303A_01D4	IRQ masking register 2 of A53 core3 (GPC_IMR2_CORE3_A53)	32	R/W	0000_0000h	5.2.10.58/751
303A_01D8	IRQ masking register 3 of A53 core3 (GPC_IMR3_CORE3_A53)	32	R/W	0000_0000h	5.2.10.59/751
303A_01DC	IRQ masking register 4 of A53 core3 (GPC_IMR4_CORE3_A53)	32	R/W	0000_0000h	5.2.10.60/752
303A_01E0	PGC acknowledge signal selection of A53 platform for PUs (GPC_ACK_SEL_A53_PU)	32	R/W	0000_0000h	5.2.10.61/752
303A_01E4	PGC acknowledge signal selection of M4 platform for PUs (GPC_ACK_SEL_M4_PU)	32	R/W	0000_0000h	5.2.10.62/755
303A_01E8	Slot configure register for PGC CPUs (GPC_SLT15_CFG)	32	R/W	0000_0000h	5.2.10.63/757
303A_01EC	Slot configure register for PGC CPUs (GPC_SLT16_CFG)	32	R/W	0000_0000h	5.2.10.63/757
303A_01F0	Slot configure register for PGC CPUs (GPC_SLT17_CFG)	32	R/W	0000_0000h	5.2.10.63/757
303A_01F4	Slot configure register for PGC CPUs (GPC_SLT18_CFG)	32	R/W	0000_0000h	5.2.10.63/757
303A_01F8	Slot configure register for PGC CPUs (GPC_SLT19_CFG)	32	R/W	0000_0000h	5.2.10.63/757
303A_01FC	Power handshake register (GPC_PU_PWRHSK)	32	R/W	0000_FFFFh	5.2.10.64/760

Table continues on the next page...

GPC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303A_0200	Slot configure register for PGC PUs (GPC_SLT0_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0208	Slot configure register for PGC PUs (GPC_SLT1_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0210	Slot configure register for PGC PUs (GPC_SLT2_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0218	Slot configure register for PGC PUs (GPC_SLT3_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0220	Slot configure register for PGC PUs (GPC_SLT4_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0228	Slot configure register for PGC PUs (GPC_SLT5_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0230	Slot configure register for PGC PUs (GPC_SLT6_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0238	Slot configure register for PGC PUs (GPC_SLT7_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0240	Slot configure register for PGC PUs (GPC_SLT8_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0248	Slot configure register for PGC PUs (GPC_SLT9_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0250	Slot configure register for PGC PUs (GPC_SLT10_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0258	Slot configure register for PGC PUs (GPC_SLT11_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0260	Slot configure register for PGC PUs (GPC_SLT12_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0268	Slot configure register for PGC PUs (GPC_SLT13_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0270	Slot configure register for PGC PUs (GPC_SLT14_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0278	Slot configure register for PGC PUs (GPC_SLT15_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0280	Slot configure register for PGC PUs (GPC_SLT16_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0288	Slot configure register for PGC PUs (GPC_SLT17_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0290	Slot configure register for PGC PUs (GPC_SLT18_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762
303A_0298	Slot configure register for PGC PUs (GPC_SLT19_CFG_PU)	32	R/W	0000_0000h	5.2.10.65/762

5.2.10.1 Basic Low power control register of A53 platform (GPC_LPCR_A53_BSC)

NOTE

LPCR_A53_BSC[CPU_CLK_ON_LPM] should be set 1'b1 when using A53 low power debug feature

NOTE

Always set LPM1/LPM0 with same value

Address: 303A_0000h base + 0h offset = 303A_0000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	MASK_DSM_TRIGGER	IRQ_SRC_A53_WUP	IRQ_SRC_C1	IRQ_SRC_C0	Reserved	MASK_L2CC_WFI	Reserved	MASK_SCU_WFI	IRQ_SRC_C3	IRQ_SRC_C2	Reserved	Reserved	MASK_CORE3_WFI	MASK_CORE2_WFI	MASK_CORE1_WFI	MASK_CORE0_WFI
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved	CPU_CLK_ON_LPM	Reserved						MST0_LPM_HSK_MASK	Reserved	Reserved	LPM1	LPM0			
W																
Reset	0	0	1	1	1	1	1	1	1	1	1	1	0	0	0	0

GPC_LPCR_A53_BSC field descriptions

Field	Description
31 MASK_DSM_TRIGGER	DSM Trigger Mask 0 DSM trigger of A53 platform will not be masked 1 DSM trigger of A53 platform will be masked
30 IRQ_SRC_A53_WUP	LPCR_A53_BSC[IRQ_SRC_C0], LPCR_A53_BSC[IRQ_SRC_C1], LPCR_A53_BSC[IRQ_SRC_C2], LPCR_A53_BSC[IRQ_SRC_C3], and LPCR_A53_BSC[IRQ_SRC_A53_WUP] work together to decide the wake up source for A53 LPM and core0/core1/core2/core3 power. See “Power up process for A53 platform” for more specific information.

Table continues on the next page...

GPC_LPCR_A53_BSC field descriptions (continued)

Field	Description
	0 LPM wakeup source be “OR” result of LPCR_A53_BSC[IRQ_SRC_C0]/LPCR_A53_BSC[IRQ_SRC_C1]/LPCR_A53_BSC[IRQ_SRC_C2]/LPCR_A53_BSC[IRQ_SRC_C3] setting 1 LPM wakeup source from external INT[127:0], masked by IMR0
29 IRQ_SRC_C1	LPCR_A53_BSC[IRQ_SRC_C0], LPCR_A53_BSC[IRQ_SRC_C1], LPCR_A53_BSC[IRQ_SRC_C2], LPCR_A53_BSC[IRQ_SRC_C3], and LPCR_A53_BSC[IRQ_SRC_A53_WUP] work together to decide the wake up source for A53 LPM and core0/core1/core2/core3 power. See “Power up process for A53 platform” for more specific information. 0 core1 wakeup source from external INT[127:0], masked by IMR1 refer to “Power up process for A53 platform” for more specific information 1 core1 wakeup source from GIC(nFIQ[1]/nIRQ[1]), SCU should not be power down during low power mode when this bit is set to 1'b1
28 IRQ_SRC_C0	LPCR_A53_BSC[IRQ_SRC_C0], LPCR_A53_BSC[IRQ_SRC_C1], LPCR_A53_BSC[IRQ_SRC_C2], LPCR_A53_BSC[IRQ_SRC_C3], and LPCR_A53_BSC[IRQ_SRC_A53_WUP] work together to decide the wake up source for A53 LPM and core0/core1/core2/core3 power. See “Power up process for A53 platform” for more specific information. 0 core0 wakeup source from external INT[127:0], masked by IMR0 refer to “Power up process for A53 platform” for more specific information 1 core0 wakeup source from GIC(nFIQ[0]/nIRQ[0]), SCU should not be power down during low power mode when this bit is set to 1'b1
27 -	This field is reserved. Reserved
26 MASK_L2CC_WFI	L2 cache controller Wait For Interrupt Mask Register 0 WFI for L2 cache controller is not masked 1 WFI for L2 cache controller is masked
25 -	This field is reserved. Reserved
24 MASK_SCU_WFI	SCU Wait For Interrupt Mask Register 0 WFI for SCU is not masked 1 WFI for SCU is masked
23 IRQ_SRC_C3	LPCR_A53_BSC[IRQ_SRC_C0], LPCR_A53_BSC[IRQ_SRC_C1], LPCR_A53_BSC[IRQ_SRC_C2], LPCR_A53_BSC[IRQ_SRC_C3], and LPCR_A53_BSC[IRQ_SRC_A53_WUP] work together to decide the wake up source for A53 LPM and core0/core1/core2/core3 power. 0 core3 wakeup source from external INT[127:0], masked by IMR1. See Power Up Process for A53 Platform for more specific information. 1 core3 wakeup source from external GIC(nFIQ[1]/nIRQ[1]), SCU should not be powered down during low power mode when this bit is set to 1'b1.
22 IRQ_SRC_C2	LPCR_A53_BSC[IRQ_SRC_C0], LPCR_A53_BSC[IRQ_SRC_C1], LPCR_A53_BSC[IRQ_SRC_C2], LPCR_A53_BSC[IRQ_SRC_C3], and LPCR_A53_BSC[IRQ_SRC_A53_WUP] work together to decide the wake up source for A53 LPM and core0/core1/core2/core3 power. 0 core2 wakeup source from external INT[127:0], masked by IMR1. See Power Up Process for A53 Platform for more specific information. 1 core2 wakeup source from external GIC(nFIQ[1]/nIRQ[1]), SCU should not be powered down during low power mode when this bit is set to 1'b1.

Table continues on the next page...

GPC_LPCR_A53_BSC field descriptions (continued)

Field	Description
21–20 -	This field is reserved.
19 MASK_CORE3_WFI	CORE3 Wait For Interrupt Mask 0 WFI for CORE3 is not masked 1 WFI for CORE3 is masked
18 MASK_CORE2_WFI	CORE2 Wait For Interrupt Mask 0 WFI for CORE2 is not masked 1 WFI for CORE2 is masked
17 MASK_CORE1_WFI	CORE1 Wait For Interrupt Mask 0 WFI for CORE1 is not masked 1 WFI for CORE1 is masked
16 MASK_CORE0_WFI	CORE0 Wait For Interrupt Mask 0 WFI for CORE0 is not masked 1 WFI for CORE0 is masked
15 -	This field is reserved. Reserved
14 CPU_CLK_ON_LPM	Define if A53 clocks will be disabled on wait/stop mode. 0 A53 clock disabled on wait/stop mode 1 A53 clock enabled on wait/stop mode
13–7 -	This field is reserved. Reserved
6 MST0_LPM_HSK_MASK	MASTER0 LPM handshake mask MASTER0(SCU) will handshake with GPC in LPM, follow this when you want this master power off. This bit should be used together with MST_CPU_MAPPING[0]. If you want power of SCU, use this setting: LPCR_A53_BSC[6]=0; MST_CPU_MAPPING[0]=1 Otherwise use: LPCR_A53_BSC[6]=1; MST_CPU_MAPPING[0]=0 0 enable MASTER0 LPM handshake, wait ACK from MASTER0 1 disable MASTER0 LPM handshake, mask ACK from MASTER0
5–4 -	This field is reserved. Reserved
3–2 LPM1	CORE1 Setting the low power mode that system will enter on next assertion of dsm_request signal. 00 Remain in RUN mode 01 Transfer to WAIT mode 10 Transfer to STOP mode 11 Reserved
LPM0	CORE0 Setting the low power mode that system will enter on next assertion of dsm_request signal. 00 Remain in RUN mode 01 Transfer to WAIT mode

Table continues on the next page...

GPC_LPCR_A53_BSC field descriptions (continued)

Field	Description
10	Transfer to STOP mode
11	Reserved

5.2.10.2 Advanced Low power control register of A53 platform (GPC_LPCR_A53_AD)

Address: 303A_0000h base + 4h offset = 303A_0004h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	L2PGE	Reserved				EN_C3_PUP	EN_C3_IRQ_PUP	EN_C2_PUP	EN_C2_IRQ_PUP	EN_C3_WFI_PDN_DIS	EN_C2_WFI_PDN_DIS	EN_C1_WFI_PDN_DIS	EN_C0_WFI_PDN_DIS	EN_C3_PDN	EN_C3_WFI_PDN	EN_C2_PDN	EN_C2_WFI_PDN
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				EN_C1_PUP	EN_C1_IRQ_PUP	EN_C0_PUP	EN_C0_IRQ_PUP	Reserved		EN_L2_WFI_PDN	EN_PLAT_PDN	EN_C1_PDN	EN_C1_WFI_PDN	EN_C0_PDN	EN_C0_WFI_PDN
W																
Reset	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0

GPC_LPCR_A53_AD field descriptions

Field	Description
31 L2PGE	0 L2 cache RAM will power down with SCU power domain in A53 platform (used for ALL_OFF mode) 1 L2 cache RAM will not power down with SCU power domain in A53 platform (used for ALL_OFF mode)
30–28 -	This field is reserved. Reserved
27 EN_C3_PUP	0 CORE3 will not power up with lower power mode request 1 CORE3 will power up with low power mode request (only used wake up from CPU_OFF)
26 EN_C3_IRQ_PUP	0 CORE3 will not power up with IRQ request 1 CORE3 will power up with IRQ request
25 EN_C2_PUP	0 CORE2 will not power up with lower power mode request 1 CORE2 will power up with low power mode request (only used wake up from CPU_OFF)
24 EN_C2_IRQ_PUP	0 CORE2 will not power up with IRQ request 1 CORE2 will power up with IRQ request

Table continues on the next page...

GPC_LPCR_A53_AD field descriptions (continued)

Field	Description
23 EN_C3_WFI_PDN_DIS	0 Disable WFI power down core3 1 Enable WFI power down core3
22 EN_C2_WFI_PDN_DIS	0 Disable WFI power down core2 1 Enable WFI power down core2
21 EN_C1_WFI_PDN_DIS	0 Disable WFI power down core1 1 Enable WFI power down core1
20 EN_C0_WFI_PDN_DIS	0 Disable WFI power down core0 1 Enable WFI power down core0
19 EN_C3_PDN	0 CORE3 will not be power down with low power mode request 1 CORE3 will be power down with low power mode request
18 EN_C3_WFI_PDN	0 CORE3 will not be power down with WFI request 1 CORE3 will be power down with WFI request
17 EN_C2_PDN	0 CORE2 will not be power down with low power mode request 1 CORE2 will be power down with low power mode request
16 EN_C2_WFI_PDN	0 CORE2 will not be power down with WFI request 1 CORE2 will be power down with WFI request
15–12 -	This field is reserved. Reserved
11 EN_C1_PUP	0 CORE1 will not power up with low power mode request (only used wake up from CPU01_OFF mode) 1 CORE1 will power up with low power mode request
10 EN_C1_IRQ_PUP	0 CORE1 will not power up with IRQ request 1 CORE1 will power up with IRQ request
9 EN_C0_PUP	(only used wake up from CPU01_OFF mode) 0 CORE0 will not power up with low power mode request 1 CORE0 will power up with low power mode request
8 EN_C0_IRQ_PUP	0 CORE0 will not power up with IRQ request 1 CORE0 will power up with IRQ request
7–6 -	This field is reserved. Reserved
5 EN_L2_WFI_PDN	NOTE: Before reset, L2 WFI is 1 and make GPC generate an error DSM request. This bit is used to mask the L2 WFI before reset. After reset, L2 WFI change to 0, and functions are OK, SW must clear this bit at the beginning of code. 0 SCU and L2 will not be power down with WFI request 1 SCU and L2 will be power down with WFI request (default)
4 EN_PLAT_PDN	0 SCU and L2 cache RAM will not be power down with low power mode request 1 SCU and L2 cache RAM will be power down with low power mode request

Table continues on the next page...

GPC_LPCR_A53_AD field descriptions (continued)

Field	Description
3 EN_C1_PDN	0 CORE1 will not be power down with low power mode request 1 CORE1 will be power down with low power mode request
2 EN_C1_WFI_PDN	0 CORE1 will not be power down with WFI request 1 CORE1 will be power down with WFI request
1 EN_C0_PDN	0 CORE0 will not be power down with low power mode request 1 CORE0 will be power down with low power mode request
0 EN_C0_WFI_PDN	0 CORE0 will not be power down with WFI request 1 CORE0 will be power down with WFI request

5.2.10.3 Low power control register of CPU1 (GPC_LPCR_M4)**NOTE**

LPCR_M4[CPU_CLK_ON_LPM] should be set 1'b0 if M4 goes to LPM without trigger power down of related domains

Address: 303A_0000h base + 8h offset = 303A_0008h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	MASK_DSM_TRIGGER	Reserved														MASK_M4_WFI
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved	CPU_CLK_ON_LPM	Reserved										EN_M4_PUP	EN_M4_PDN	LPM0	
W																
Reset	0	0	1	1	1	1	1	1	1	1	1	1	0	0	0	0

GPC_LPCR_M4 field descriptions

Field	Description
31 MASK_DSM_TRIGGER	M4 WFI Mask 0 DSM trigger of M4 platform will not be masked 1 DSM trigger of M4 platform will be masked
30–17 -	This field is reserved. Reserved
16 MASK_M4_WFI	M4 WFI Mask 0 WFI for M4 is not masked 1 WFI for M4 is masked
15 -	This field is reserved. Reserved
14 CPU_CLK_ON_LPM	Define if M4 clocks will be disabled on wait/stop mode. 0 M4 clock disabled on wait/stop mode. 1 M4 clock enabled on wait/stop mode.
13–4 -	This field is reserved. Reserved
3 EN_M4_PUP	Enable m4 virtual PGC power up with LPM enter
2 EN_M4_PDN	Enable m4 virtual PGC power down with LPM enter
LPM0	Setting the low power mode that system will enter on next assertion of dsm_request signal. 00 Remain in RUN mode 01 Transfer to WAIT mode 10 Transfer to STOP mode 11 Reserved

5.2.10.4 System low power control register (GPC_SLPCR)

NOTE

SLPCR[VSTBY] must be set to 1'b1 if SLPCCR[RBC_EN] is set to 1'b1; SLPCCR[SBYOS] must be set to 1'b1 if SLPCCR[VSTBY] is set to 1'b1.

Address: 303A_0000h base + 14h offset = 303A_0014h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16		
R	EN_DSM	RBC_EN	REG_BYPASS_COUNT							DISABLE_A53_IS_DSM	Reserved				EN_M4_FASTWUP_STOP_MODE	EN_M4_FASTWUP_WAIT_MODE	EN_A53_FASTWUP_STOP_MODE	EN_A53_FASTWUP_WAIT_MODE
W																		
Reset	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
R	OSCCNT								COSC_EN	COSC_PWRDOWN	STBY_COUNT			VSTBY	SBYOS	BYPASS_PMIC_READY		
W																		
Reset	1	1	1	1	1	1	1	1	1	0	0	0	0	0	1	0		

GPC_SLPCR field descriptions

Field	Description
31 EN_DSM	DSM enable 0 DSM disabled 1 DSM enabled
30 RBC_EN	Enable for REG_BYPASS_COUNTER. If enabled, REG_BYPASS signal will be asserted after REG_BYPASS_COUNT clocks of CKIL, after standby voltage is requested. If standby voltage is not requested REG_BYPASS won't be asserted, even if counter is enabled. 0 REG_BYPASS_COUNTER disabled 1 REG_BYPASS_COUNTER enabled
29–24 REG_BYPASS_COUNT	Counter for REG_BYPASS signal assertion after standby voltage request by PMIC_STBY_REQ. 000000 no delay 000001 1 CKIL clock period delay 111111 63 CKIL clock period delay

Table continues on the next page...

GPC_SLPCR field descriptions (continued)

Field	Description
23 DISABLE_A53_I S_DSM	0 Enable A53 isolation signal in DSM 1 Disable A53 isolation signal in DSM
22–20 -	This field is reserved. Reserved
19 EN_M4_FASTW UP_STOP_MOD E	Enable M4 fast wake up stop mode, relevant PLLs will not be closed in this mode.
18 EN_M4_FASTW UP_WAIT_MOD E	Enable M4 fast wake up wait mode, relevant PLLs will not be closed in this mode.
17 EN_A53_FASTW UP_STOP_MOD E	Enable A53 fast wake up stop mode, relevant PLLs will not be closed in this mode.
16 EN_A53_FASTW UP_WAIT_MOD E	Enable A53 fast wake up wait mode, relevant PLLs will not be closed in this mode.
15–8 OSCCNT	Oscillator ready counter value. These bits define value of 32KHz counter, that serve as counter for oscillator lock time. This is used for oscillator lock time. Current estimation is ~5ms. This counter will be used in sequence out of DSM and if sbyos bit was defined. GPC will wait the “OSCCNT” number of cycles before it notify CCM to open the relevant PLLs. 00000000 count 1 ckil 11111111 count 256 ckils
7 COSC_EN	On-chip oscillator enable bit - this bit value is reflected on the output cosc_en. The system will start with on-chip oscillator enabled to supply source for the PLLs. Software can change this bit if a transition to the bypass PLL clocks was performed for all the PLLs. In cases that this bit is changed from '0' to '1' then GPC will enable the on-chip oscillator and after counting oscnt ckil clock cycles before it notify CCM to open the relevant PLLs. The cosc_en bit should be changed only when on-chip oscillator is not chosen as the clock source. 0 Disable on-chip oscillator 1 Enable on-chip oscillator
6 COSC_ PWRDOWN	In run mode, software can manually control powering down of on chip oscillator, i.e. generating '1' on cosc_pwrdown signal. If software manually powered down the on chip oscillator, then sbyos functionality for on-chip oscillator will be bypassed. The manual closing of on-chip oscillator should be performed only in case the reference oscillator is not the source of all the clocks generation. 0 On-chip oscillator will not be powered down, i.e. cosc_pwrdown = 0 1 On-chip oscillator will be powered down, i.e. cosc_pwrdown = 1
5–3 STBY_COUNT	Standby counter definition. These two bits define, in the case of stop exit (if VSTBY bit was set), the amount of time GPC will wait between PMIC_STBY_REQ negation and the check of assertion of PMIC_READY. 000 GPC will wait 4 ckil clock cycles 001 GPC will wait 8 ckil clock cycles

Table continues on the next page...

GPC_SLPCR field descriptions (continued)

Field	Description
	010 GPC will wait 16 ckil clock cycles 011 GPC will wait 32 ckil clock cycles 100 GPC will wait 64 ckil clock cycles 101 GPC will wait 128 ckil clock cycles 110 GPC will wait 256 ckil clock cycles 111 GPC will wait 512 ckil clock cycles
2 VSTBY	Voltage standby request bit. This bit defines if PMIC_STBY_REQ pin, which notifies external power management IC to move from functional voltage to standby voltage, will be asserted in stop mode. 0 Voltage will not be changed to standby voltage after next entrance to stop mode. (PMIC_STBY_REQ will remain negated - '0') 1 Voltage will be changed to standby voltage after next entrance to stop mode.
1 SBYOS	Standby clock oscillator bit. This bit defines if cosc_pwrdown, which power down the on chip oscillator, will be asserted in DSM. 0 On chip oscillator will not be powered down, after next entrance to DSM. 1 On chip oscillator will be powered down, after next entrance to DSM. When returning from DSM, external oscillator will be enabled again, on chip oscillator will return to oscillator mode , and after oscnt count GPC will continue with the exit from DSM process.
0 BYPASS_PMIC_READY	By asserting this bit GPC will bypass waiting for PMIC_READY signal when coming out of DSM. This should be used for PMIC's that don't support the PMIC_READY signal. 0 Don't bypass the PMIC_READY signal - GPC will wait for its assertion during exit of low power mode if standby voltage was enabled 1 Bypass the PMIC_READY signal - GPC will not wait for its assertion during exit of low power mode if standby voltage was enabled

5.2.10.5 MASTER LPM Handshake (GPC_MST_CPU_MAPPING)

Address: 303A_0000h base + 18h offset = 303A_0018h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	MEMLP_RET_PGGEN															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	MEMLP_RET_PGGEN															MST0_CPU_MAPPING
W																
Reset	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

GPC_MST_CPU_MAPPING field descriptions

Field	Description
31–1 MEMLP_RET_PGEN	Delay counter for "retnx" and "pgen"
0 MST0_CPU_MAPPING	MASTER0 CPU Mapping SCU LPM handshake mask. This bit should be used together with LPCR_A7_BSC[6]. If you want power of SCU, use this setting: LPCR_A7_BSC[6]=0; MST_CPU_MAPPING[0]=1 Otherwise, use this: LPCR_A7_BSC[6]=1; MST_CPU_MAPPING[0]=0 0 GPC will not send out power off requirement 1 GPC will send out power off requirement

5.2.10.6 Memory low power control register (GPC_MLPCR)

Address: 303A_0000h base + 20h offset = 303A_0020h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	MEMLP_RET_PGEN								MEM_EXT_CNT							
W																
Reset	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	MEMLP_ENT_CNT								Reserved					ROMLP_PDN_DIS	MEMLP_RET_SEL	MEMLP_CTL_DIS
W																
Reset	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0

GPC_MLPCR field descriptions

Field	Description
31–24 MEMLP_RET_PGEN	Delay counter for "retnx" and "pgen"
23–16 MEM_EXT_CNT	Delay counter to start existing from memory low power
15–8 MEMLP_ENT_CNT	Delay counter to make sure all clock off after pll_dis_req is issued by smc

Table continues on the next page...

GPC_MLPCR field descriptions (continued)

Field	Description
7-3 -	This field is reserved. Reserved
2 ROMLP_PDN_ DIS	ROM shut down control 0 Enable ROM shut down control(should also enable RAM low power control); 1 Disable ROM shut down control
1 MEMLP_RET_ SEL	Retention select 0 retention mode 2 1 retention mode 1
0 MEMLP_CTL_ DIS	RAM low-power control 0 Enable RAM low power control 1 Disable RAM low power control

5.2.10.7 PGC acknowledge signal selection of A53 platform (GPC_PGC_ACK_SEL_A53)

The register can only be accessed by A53 platform

Address: 303A_0000h base + 24h offset = 303A_0024h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	A53_PGC_PUP_ ACK	A53_C3_PGC_ PUP_ACK	A53_C2_PGC_ PUP_ACK	Reserved										A53_PLAT_PGC_ PUP_ACK	A53_C1_PGC_ PUP_ACK	A53_C0_PGC_ PUP_ACK
W																
Reset	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	A53_PGC_PDN_ ACK	A53_C3_PGC_ PDN_ACK	A53_C2_PGC_ PDN_ACK	Reserved										A53_PLAT_PGC_ PDN_ACK	A53_C1_PGC_ PDN_ACK	A53_C0_PGC_ PDN_ACK
W																
Reset	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_PGC_ACK_SEL_A53 field descriptions

Field	Description
31 A53_PGC_PUP_ACK	Select power up acknowledge signal of A53 (dummy) PGC as the power up acknowledge for A53 LPM.
30 A53_C3_PGC_PUP_ACK	Select power up acknowledge signal of A53 CORE3 PGC as the power up acknowledge for A53 LPM.
29 A53_C2_PGC_PUP_ACK	Select power up acknowledge signal of A53 CORE2 PGC as the power up acknowledge for A53 LPM.
28–19 -	This field is reserved. Reserved
18 A53_PLAT_PGC_PUP_ACK	Select power up acknowledge signal of A53 PLATFORM PGC as the power up acknowledge for A53 LPM.
17 A53_C1_PGC_PUP_ACK	Select power up acknowledge signal of A53 CORE1 PGC as the power up acknowledge for A53 LPM.
16 A53_C0_PGC_PUP_ACK	Select power up acknowledge signal of A53 CORE0 PGC as the power up acknowledge for A53 LPM.
15 A53_PGC_PDN_ACK	Select power down acknowledge signal of A53 (dummy) PGC as the power down acknowledge for A53 LPM.
14 A53_C3_PGC_PDN_ACK	Select power down acknowledge signal of A53 CORE3 PGC as the power down acknowledge for A53 LPM.
13 A53_C2_PGC_PDN_ACK	Select power down acknowledge signal of A53 CORE2 PGC as the power down acknowledge for A53 LPM.
12–3 -	This field is reserved. Reserved
2 A53_PLAT_PGC_PDN_ACK	Select power down acknowledge signal of A53 PLATFORM PGC as the power down acknowledge for A53 LPM.
1 A53_C1_PGC_PDN_ACK	Select power down acknowledge signal of A53 CORE1 PGC as the power down acknowledge for A53 LPM.
0 A53_C0_PGC_PDN_ACK	Select power down acknowledge signal of A53 CORE0 PGC as the power down acknowledge for A53 LPM.

5.2.10.8 PGC acknowledge signal selection of M4 platform (GPC_PGC_ACK_SEL_M4)

This register can only be accessed by the M4 platform.

NOTE

“dummy” PGC cannot be mapped to time slot control. “virtual” PGC can be mapped to time slot control. When virtual PGC is used, below setting is required -
GPC_MISC[M4_PDN_REQ_MASK] should be set to 1'b1 and arrange virtual GPC in same slot with MIX. power/up slot (See example code 3). MIX PGC may possibly power down later than A53 platform power down when virtual PGC is used.

Address: 303A_0000h base + 28h offset = 303A_0028h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	M4_DUMMY_PGC_PUP_ACK	Reserved														M4_VIRTUAL_PGC_PUP_ACK
W																
Reset	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	M4_DUMMY_PGC_PDN_ACK	Reserved														M4_VIRTUAL_PGC_PDN_ACK
W																
Reset	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_PGC_ACK_SEL_M4 field descriptions

Field	Description
31 M4_DUMMY_PGC_PUP_ACK	Select power up acknowledge signal of M4 (dummy) PGC as the power up acknowledge for M4 LPM.
30–17 -	This field is reserved. Reserved

Table continues on the next page...

GPC_PGC_ACK_SEL_M4 field descriptions (continued)

Field	Description
16 M4_VIRTUAL_PGC_PUP_ACK	Select power up acknowledge signal of M4 virtual PGC as the power up acknowledge for M4 LPM. M4 virtual PGC only acknowledge power up request in the end of current slot time
15 M4_DUMMY_PGC_PDN_ACK	Select power down acknowledge signal of M4 (dummy) PGC as the power down acknowledge for M4 LPM.
14–1 -	This field is reserved. Reserved
0 M4_VIRTUAL_PGC_PDN_ACK	Select power down acknowledge signal of M4 virtual PGC as the power down acknowledge for M4 LPM. M4 virtual PGC only acknowledge power down request in the end of current slot time

5.2.10.9 GPC Miscellaneous register (GPC_MISC)

Address: 303A_0000h base + 2Ch offset = 303A_002Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved							M4_BYPASS_PUP_MASK	A53_BYPASS_PUP_MASK	Reserved						
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved							M4_PDN_REQ_MASK	Reserved		GPC_IRQ_MASK	Reserved			A53_SLEEP_HOLD_REQ_B	M4_SLEEP_HOLD_REQ_B
W																
Reset	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1

GPC_MISC field descriptions

Field	Description
31–26 -	This field is reserved. Reserved
25 M4_BYPASS_PUP_MASK	

Table continues on the next page...

GPC_MISC field descriptions (continued)

Field	Description
24 A53_BYPASS_PUP_MASK	
23–9 -	This field is reserved. Reserved
8 M4_PDN_REQ_MASK	M4 power-down mask 0 M4 power down request to virtual M4 PGC will be masked. 1 M4 power down request to virtual M4 PGC will not be masked. Set this bit to 1'b1 when M4 virtual PGC is used.
7–6 -	This field is reserved. Reserved
5 GPC_IRQ_MASK	GPC interrupt/event masking 0 Not masked 1 Interrupt / event is masked
4–2 -	This field is reserved. Reserved
1 A53_SLEEP_HOLD_REQ_B	A53 sleep hold 0 Hold A53 platform in sleep mode. This bit is a software control bit to A53 platform. 1 Don't hold A53 platform in sleep mode.
0 M4_SLEEP_HOLD_REQ_B	M4 sleep hold 0 Hold M4 platform in sleep mode. This bit is a software control bit to M4 platform. 1 Don't hold M4 platform in sleep mode.

5.2.10.10 IRQ masking register 1 of A53 core0 (GPC_IMR1_CORE0_A53)

The four IMRn_CORE0_A53 (n = 1,2,3,4) registers are used as interrupt mask for A53 core0.

Address: 303A_0000h base + 30h offset = 303A_0030h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR1_CORE0_A53																															
W	IMR1_CORE0_A53																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR1_CORE0_A53 field descriptions

Field	Description
IMR1_CORE0_A53	A53 core0 IRQ[31:0] masking bits:

GPC_IMR1_CORE0_A53 field descriptions (continued)

Field	Description
0	IRQ not masked
1	IRQ masked

5.2.10.11 IRQ masking register 2 of A53 core0 (GPC_IMR2_CORE0_A53)

Address: 303A_0000h base + 34h offset = 303A_0034h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR2_CORE0_A53																															
W	IMR2_CORE0_A53																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR2_CORE0_A53 field descriptions

Field	Description
IMR2_CORE0_A53	A53 core0 IRQ[63:32] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.12 IRQ masking register 3 of A53 core0 (GPC_IMR3_CORE0_A53)

Address: 303A_0000h base + 38h offset = 303A_0038h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR3_CORE0_A53																															
W	IMR3_CORE0_A53																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR3_CORE0_A53 field descriptions

Field	Description
IMR3_CORE0_A53	A53 core0 IRQ[95:64] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.13 IRQ masking register 4 of A53 core0 (GPC_IMR4_CORE0_A53)

Address: 303A_0000h base + 3Ch offset = 303A_003Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR4_CORE0_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR4_CORE0_A53 field descriptions

Field	Description
IMR4_CORE0_A53	A53 core0 IRQ[127:96] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.14 IRQ masking register 1 of A53 core1 (GPC_IMR1_CORE1_A53)

The four IMRn_CORE1_A53 (n = 1,2,3,4) registers are used as interrupt mask for A53 core1.

Address: 303A_0000h base + 40h offset = 303A_0040h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR1_CORE1_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR1_CORE1_A53 field descriptions

Field	Description
IMR1_CORE1_A53	A53 core1 IRQ[31:0] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.15 IRQ masking register 2 of A53 core1 (GPC_IMR2_CORE1_A53)

General Power Controller (GPC)

Address: 303A_0000h base + 44h offset = 303A_0044h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR2_CORE1_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPC_IMR2_CORE1_A53 field descriptions

Field	Description
IMR2_CORE1_A53	A53 core1 IRQ[63:32] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.16 IRQ masking register 3 of A53 core1 (GPC_IMR3_CORE1_A53)

Address: 303A_0000h base + 48h offset = 303A_0048h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR3_CORE1_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPC_IMR3_CORE1_A53 field descriptions

Field	Description
IMR3_CORE1_A53	A53 core1 IRQ[95:64] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.17 IRQ masking register 4 of A53 core1 (GPC_IMR4_CORE1_A53)

Address: 303A_0000h base + 4Ch offset = 303A_004Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR4_CORE1_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPC_IMR4_CORE1_A53 field descriptions

Field	Description
IMR4_CORE1_A53	A53 core1 IRQ[127:96] masking bits:

GPC_IMR4_CORE1_A53 field descriptions (continued)

Field	Description
0	IRQ not masked
1	IRQ masked

5.2.10.18 IRQ masking register 1 of M4 (GPC_IMR1_M4)

The four IMR_n_M4 (n = 1,2,3,4) registers are used as interrupt mask for M4.

Address: 303A_0000h base + 50h offset = 303A_0050h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR1_M4																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPC_IMR1_M4 field descriptions

Field	Description
IMR1_M4	M4 IRQ[31:0] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.19 IRQ masking register 2 of M4 (GPC_IMR2_M4)

Address: 303A_0000h base + 54h offset = 303A_0054h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR2_M4																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPC_IMR2_M4 field descriptions

Field	Description
IMR2_M4	M4 IRQ[63:32] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.20 IRQ masking register 3 of M4 (GPC_IMR3_M4)

General Power Controller (GPC)

Address: 303A_0000h base + 58h offset = 303A_0058h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR3_M4																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR3_M4 field descriptions

Field	Description
IMR3_M4	M4 IRQ[95:64] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.21 IRQ masking register 4 of M4 (GPC_IMR4_M4)

Address: 303A_0000h base + 5Ch offset = 303A_005Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR4_M4																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR4_M4 field descriptions

Field	Description
IMR4_M4	M4 IRQ[127:96] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.22 IRQ status register 1 of A53 (GPC_ISR1_A53)

The four ISR_n_A53 (n = 1,2,3,4) registers, all of them are read only register

Address: 303A_0000h base + 70h offset = 303A_0070h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ISR1_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_ISR1_A53 field descriptions

Field	Description
ISR1_A53	A53 IRQ[31:0] status

5.2.10.23 IRQ status register 2 of A53 (GPC_ISR2_A53)

Address: 303A_0000h base + 74h offset = 303A_0074h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ISR2_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_ISR2_A53 field descriptions

Field	Description
ISR2_A53	A53 IRQ[63:32] status

5.2.10.24 IRQ status register 3 of A53 (GPC_ISR3_A53)

Address: 303A_0000h base + 78h offset = 303A_0078h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ISR3_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_ISR3_A53 field descriptions

Field	Description
ISR3_A53	A53 IRQ[95:64] status

5.2.10.25 IRQ status register 4 of A53 (GPC_ISR4_A53)

General Power Controller (GPC)

Address: 303A_0000h base + 7Ch offset = 303A_007Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ISR4_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_ISR4_A53 field descriptions

Field	Description
ISR4_A53	A53 IRQ[127:96] status

5.2.10.26 IRQ status register 1 of M4 (GPC_ISR1_M4)

The four ISR_n_M4 (n = 1,2,3,4) registers, all of them are read only register

Address: 303A_0000h base + 80h offset = 303A_0080h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ISR1_M4																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_ISR1_M4 field descriptions

Field	Description
ISR1_M4	M4 IRQ[31:0] status

5.2.10.27 IRQ status register 2 of M4 (GPC_ISR2_M4)

Address: 303A_0000h base + 84h offset = 303A_0084h

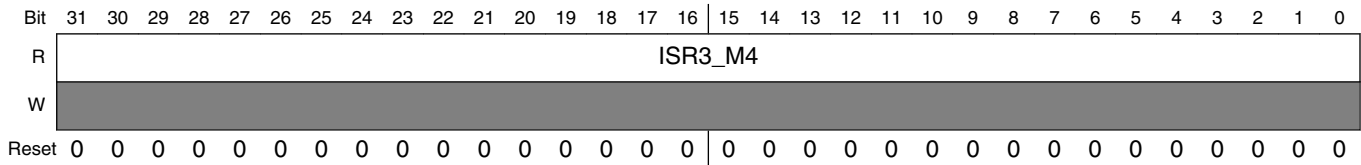
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ISR2_M4																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_ISR2_M4 field descriptions

Field	Description
ISR2_M4	M4 IRQ[63:32] status

5.2.10.28 IRQ status register 3 of M4 (GPC_ISR3_M4)

Address: 303A_0000h base + 88h offset = 303A_0088h

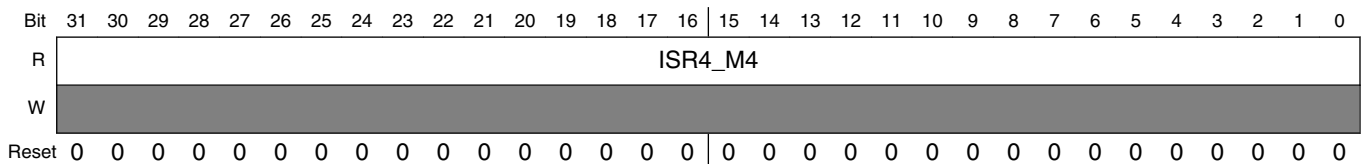


GPC_ISR3_M4 field descriptions

Field	Description
ISR3_M4	M4 IRQ[95:64] status

5.2.10.29 IRQ status register 4 of M4 (GPC_ISR4_M4)

Address: 303A_0000h base + 8Ch offset = 303A_008Ch



GPC_ISR4_M4 field descriptions

Field	Description
ISR4_M4	M4 IRQ[127:96] status

5.2.10.30 Slot configure register for CPUs (GPC_SLTn_CFG)

There are 20 slots in each SLTn_CFG(n = 0~19) that define the power up or power down behavior of one or more A53 core, NOC, or SCU PGC in each slot. This array contains slots 0 to 14, see Memory Map for slots 15 to 19.

In each “SLTn_cfg”, 2 bits (slt_cfg[1:0])are reserved for each PGC:

- 2'b01 (slot controller will power down relevant PGC in corresponding slot if hardware power down request asserted)
- 2'b10 (slot controller will power up relevant PGC in corresponding slot if hardware power up request asserted)
- 2'b00 or 2'b11 (not power down or power up behavior in relevant slot)

General Power Controller (GPC)

The specific bits assignment for each PGC is shown in the table below.

	PGCx	PGCx-1	..	PGC2	PGC1	PGC0
SLT0_CFG	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]
SLT1_CFG	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]
:	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]
SLTn_CFG	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]

Address: 303A_0000h base + B0h offset + (4d × i), where i=0d to 14d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved																
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved							SCU_PUP_SLOT_CONTROL	SCU_PDN_SLOT_CONTROL	CORE3_A53_PUP_SLOT_CONTROL	CORE3_A53_PDN_SLOT_CONTROL	CORE2_A53_PUP_SLOT_CONTROL	CORE2_A53_PDN_SLOT_CONTROL	CORE1_A53_PUP_SLOT_CONTROL	CORE1_A53_PDN_SLOT_CONTROL	CORE0_A53_PUP_SLOT_CONTROL	CORE0_A53_PDN_SLOT_CONTROL
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPC_SLTn_CFG field descriptions

Field	Description
31–10 -	This field is reserved.
9 SCU_PUP_SLOT_CONTROL	SCU Power-up slot control
8 SCU_PDN_SLOT_CONTROL	SCU Power-down slot control
7 CORE3_A53_PUP_SLOT_CONTROL	CORE3 A53 Power-up slot control

Table continues on the next page...

GPC_SLTn_CFG field descriptions (continued)

Field	Description
6 CORE3_A53_ PDN_SLOT_ CONTROL	CORE3 A53 Power-down slot control
5 CORE2_A53_ PUP_SLOT_ CONTROL	CORE2 A53 Power-up slot control
4 CORE2_A53_ PDN_SLOT_ CONTROL	CORE2 A53 Power-down slot control
3 CORE1_A53_ PUP_SLOT_ CONTROL	CORE1 A53 Power-up slot control
2 CORE1_A53_ PDN_SLOT_ CONTROL	CORE1 A53 Power-down slot control
1 CORE0_A53_ PUP_SLOT_ CONTROL	CORE0 A53 Power-up slot control
0 CORE0_A53_ PDN_SLOT_ CONTROL	CORE0 A53 Power-down slot control

5.2.10.31 PGC CPU mapping (GPC_PGC_CPU_0_1_MAPPING)

Address: 303A_0000h base + ECh offset = 303A_00ECh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PCIE2_M4_DOMAIN	MIPI_CSI2_M4_DOMAIN	MIPI_CSI1_M4_DOMAIN	DISP_M4_DOMAIN	HDMI_M4_DOMAIN	VPU_M4_DOMAIN	GPU_M4_DOMAIN	DDR2_M4_DOMAIN	DDR1_M4_DOMAIN	Reserved	OTG2_M4_DOMAIN	OTG1_M4_DOMAIN	PCIE_M4_DOMAIN	MIPI_M4_DOMAIN	Reserved	MF_M4_DOMAIN
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PCIE2_A53_DOMAIN	MIPI_CSI2_A53_DOMAIN	MIPI_CSI1_A53_DOMAIN	DISP_A53_DOMAIN	HDMI_A53_DOMAIN	VPU_A53_DOMAIN	GPU_A53_DOMAIN	DDR2_A53_DOMAIN	DDR1_A53_DOMAIN	Reserved	OTG2_A53_DOMAIN	OTG1_A53_DOMAIN	PCIE_A53_DOMAIN	MIPI_A53_DOMAIN	Reserved	MF_A53_DOMAIN
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_PGC_CPU_0_1_MAPPING field descriptions

Field	Description
31 PCIE2_M4_DOMAIN	PCIE2_M4_DOMAIN
30 MIPI_CSI2_M4_DOMAIN	MIPI_CSI2_M4_DOMAIN
29 MIPI_CSI1_M4_DOMAIN	MIPI_CSI1_M4_DOMAIN
28 DISP_M4_DOMAIN	DISP_M4_DOMAIN
27 HDMI_M4_DOMAIN	HDMI_M4_DOMAIN

Table continues on the next page...

GPC_PGC_CPU_0_1_MAPPING field descriptions (continued)

Field	Description
26 VPU_M4_ DOMAIN	VPU_M4_DOMAIN
25 GPU_M4_DOMA IN	GPU_M4_DOMAIN
24 DDR2_M4_DOM AIN	DDR2_M4_DOMAIN
23 DDR1_M4_DOM AIN	DDR1_M4_DOMAIN
22 -	This field is reserved.
21 OTG2_M4_ DOMAIN	OTG2_M4_DOMAIN
20 OTG1_M4_DOM AIN	OTG1_M4_DOMAIN
19 PCIE_M4_ DOMAIN	PCIE_M4_DOMAIN
18 MIPI_M4_DOMAI N	MIPI_M4_DOMAIN
17 -	This field is reserved. Reserved
16 MF_M4_ DOMAIN	MF_M4_DOMAIN
15 PCIE2_A53_DO MAIN	PCIE2_A53_DOMAIN
14 MIPI_CSI2_A53_ DOMAIN	MIPI_CSI2_A53_DOMAIN
13 MIPI_CSI1_A53_ DOMAIN	MIPI_CSI1_A53_DOMAIN
12 DISP_A53_ DOMAIN	DISP_A53_DOMAIN
11 HDMI_A53_DOM AIN	HDMI_A53_DOMAIN
10 VPU_A53_ DOMAIN	VPU_A53_DOMAIN

Table continues on the next page...

GPC_PGC_CPU_0_1_MAPPING field descriptions (continued)

Field	Description
9 GPU_A53_DOMAIN	GPU_A53_DOMAIN
8 DDR2_A53_DOMAIN	DDR2_A53_DOMAIN
7 DDR1_A53_DOMAIN	DDR1_A53_DOMAIN
6 -	This field is reserved.
5 OTG2_A53_DOMAIN	OTG2_A53_DOMAIN
4 OTG1_A53_DOMAIN	OTG1_A53_DOMAIN
3 PCIE_A53_DOMAIN	PCIE_A53_DOMAIN
2 MIPI_A53_DOMAIN	MIPI A53 DOMAIN
1 -	This field is reserved. Reserved
0 MF_A53_DOMAIN	MF_A53_DOMAIN

5.2.10.32 CPU PGC software power up trigger (GPC_CPU_PGC_SW_PUP_REQ)

Address: 303A_0000h base + F0h offset = 303A_00F0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

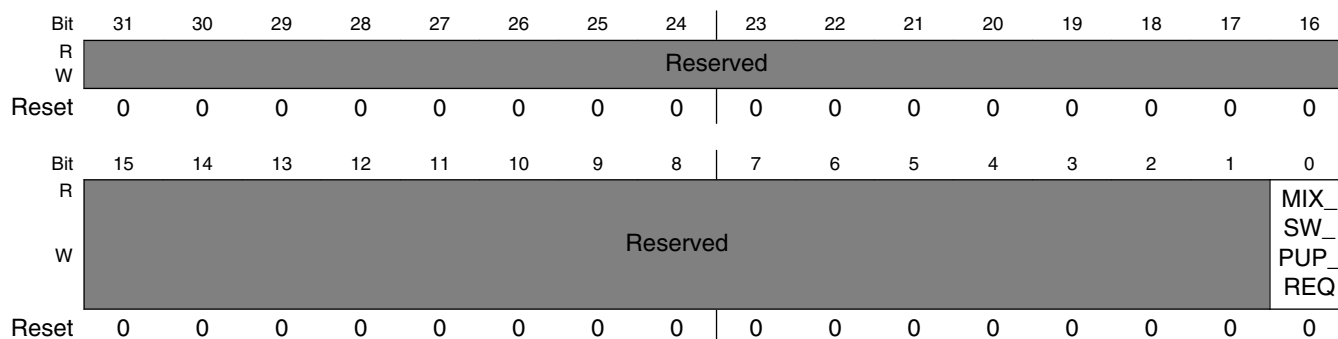


GPC_CPU_PGC_SW_PUP_REQ field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 CORE3_A53_SW_PUP_REQ	Software power up trigger for Core3 A53 PGC
3 CORE2_A53_SW_PUP_REQ	Software power up trigger for Core2 A53 PGC
2 SCU_A53_SW_PUP_REQ	Software power up trigger for SCU A53 PGC
1 CORE1_A53_SW_PUP_REQ	Software power up trigger for Core1 A53 PGC
0 CORE0_A53_SW_PUP_REQ	Software power up trigger for Core0 A53 PGC

5.2.10.33 MIX PGC software power up trigger (GPC_MIX_PGC_SW_PUP_REQ)

Address: 303A_0000h base + F4h offset = 303A_00F4h



GPC_MIX_PGC_SW_PUP_REQ field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 MIX_SW_PUP_REQ	Software power up trigger for MIX PGC

5.2.10.34 PU PGC software up trigger (GPC_PU_PGC_SW_PUP_REQ)

Address: 303A_0000h base + F8h offset = 303A_00F8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		PCIE2_SW_PUP_REQ	MIPI_CSI2_SW_PUP_REQ Q	MIPI_CSI1_SW_PUP_REQ Q	DISP_SW_PUP_REQ	HDMI_SW_PUP_REQ	VPU_SW_PUP_REQ	GPU_SW_PUP_REQ	DDR2_SW_PUP_REQ	DDR1_SW_PUP_REQ	Reserved	USB_OTG2_SW_PUP_R EQ	USB_OTG1_SW_PUP_R EQ	PCIE_SW_PUP_REQ	MIPI_SW_PUP_REQ
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_PU_PGC_SW_PUP_REQ field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13 PCIE2_SW_PUP_REQ	Software power up trigger for PCIE2
12 MIPI_CSI2_SW_PUP_REQ	Software power up trigger for MIPI_CSI2

Table continues on the next page...

GPC_PU_PGC_SW_PUP_REQ field descriptions (continued)

Field	Description
11 MIPI_CSI1_SW_PUP_REQ	Software power up trigger for MIPI_CSI1
10 DISP_SW_PUP_REQ	Software power up trigger for DISP
9 HDMI_SW_PUP_REQ	Software power up trigger for HDMI
8 VPU_SW_PUP_REQ	Software power up trigger for VPU
7 GPU_SW_PUP_REQ	Software power up trigger for GPU
6 DDR2_SW_PUP_REQ	Software power up trigger for DDR2
5 DDR1_SW_PUP_REQ	Software power up trigger for DDR1
4 -	This field is reserved.
3 USB_OTG2_SW_PUP_REQ	Software power up trigger for USB_OTG2
2 USB_OTG1_SW_PUP_REQ	Software power up trigger for USB_OTG1
1 PCIE_SW_PUP_REQ	Software power up trigger for PCIE
0 MIPI_SW_PUP_REQ	Software power up trigger for MIPI

5.2.10.35 CPU PGC software down trigger (GPC_CPU_PGC_SW_PDN_REQ)

General Power Controller (GPC)

Address: 303A_0000h base + FCh offset = 303A_00FCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												CORE3_A53_SW_PUP_REQ	CORE2_A53_SW_PUP_REQ	SCU_A53_SW_PD_N_REQ	CORE1_A53_SW_PDN_REQ
W													CORE0_A53_SW_PDN_REQ			
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_CPU_PGC_SW_PDN_REQ field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 CORE3_A53_SW_PUP_REQ	Software power up trigger for Core3 A53 PGC
3 CORE2_A53_SW_PUP_REQ	Software power up trigger for Core2 A53 PGC
2 SCU_A53_SW_PDN_REQ	Software power down trigger for SCU A53 PGC
1 CORE1_A53_SW_PDN_REQ	Software power down trigger for Core1 A53 PGC
0 CORE0_A53_SW_PDN_REQ	Software power down trigger for Core0 A53 PGC

5.2.10.36 MIX PGC software power down trigger (GPC_MIX_PGC_SW_PDN_REQ)

Address: 303A_0000h base + 100h offset = 303A_0100h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															MIX_SW_PDN_REQ
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_MIX_PGC_SW_PDN_REQ field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 MIX_SW_PDN_REQ	Software power down trigger for MIX PGC

5.2.10.37 PU PGC software down trigger (GPC_PU_PGC_SW_PDN_REQ)

Address: 303A_0000h base + 104h offset = 303A_0104h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

General Power Controller (GPC)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		PCIE2_SW_PDN_REQ	MIPI_CSI2_SW_PDN_REQ	MIPI_CSI1_SW_PDN_REQ	DISP_SW_PDN_REQ	HDMI_SW_PDN_REQ	VPU_SW_PDN_REQ	GPU_SW_PDN_REQ	DDR2_SW_PDN_REQ	DDR1_SW_PDN_REQ	Reserved	USB_OTG2_SW_PDN_REQ	USB_OTG1_SW_PDN_REQ	PCIE_SW_PDN_REQ	MIPI_SW_PDN_REQ
W				Q	Q								EQ	EQ		
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_PU_PGC_SW_PDN_REQ field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13 PCIE2_SW_PDN_REQ	Software power down trigger for PCIE2
12 MIPI_CSI2_SW_PDN_REQ	Software power down trigger for MIPI_CSI2
11 MIPI_CSI1_SW_PDN_REQ	Software power down trigger for MIPI_CSI1
10 DISP_SW_PDN_REQ	Software power down trigger for DISP
9 HDMI_SW_PDN_REQ	Software power down trigger for HDMI
8 VPU_SW_PDN_REQ	Software power down trigger for VPU
7 GPU_SW_PDN_REQ	Software power down trigger for GPU
6 DDR2_SW_PDN_REQ	Software power down trigger for DDR2
5 DDR1_SW_PDN_REQ	Software power down trigger for DDR1
4 -	This field is reserved.
3 USB_OTG2_SW_PDN_REQ	Software power down trigger for USB_OTG2

Table continues on the next page...

GPC_PU_PGC_SW_PDN_REQ field descriptions (continued)

Field	Description
2 USB_OTG1_SW_PDN_REQ	Software power down trigger for USB_OTG1
1 PCIE_SW_PDN_REQ	Software power down trigger for PCIE
0 MIPI_SW_PDN_REQ	Software power down trigger for MIPI

5.2.10.38 Basic Low power control register of A53 platform (GPC_LPCR_A53_BSC2)

Address: 303A_0000h base + 108h offset = 303A_0108h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_LPCR_A53_BSC2 field descriptions

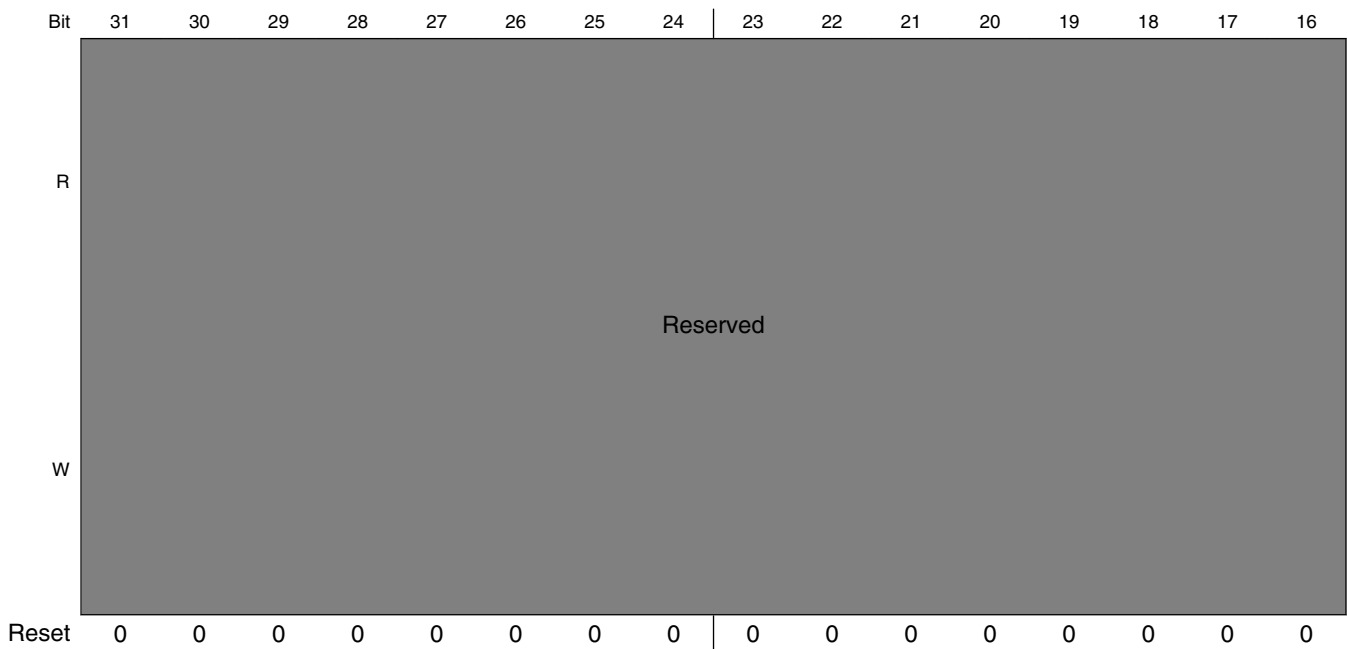
Field	Description
31–4 -	This field is reserved. Reserved
3–2 LPM3	CORE3 Setting the low power mode that system will enter on next assertion of dsm_request signal. 00 Remain in RUN mode 01 Transfer to WAIT mode 10 Transfer to STOP mode 11 Reserved
LPM2	CORE2 Setting the low power mode that system will enter on next assertion of dsm_request signal. 00 Remain in RUN mode 01 Transfer to WAIT mode 10 Transfer to STOP mode 11 Reserved

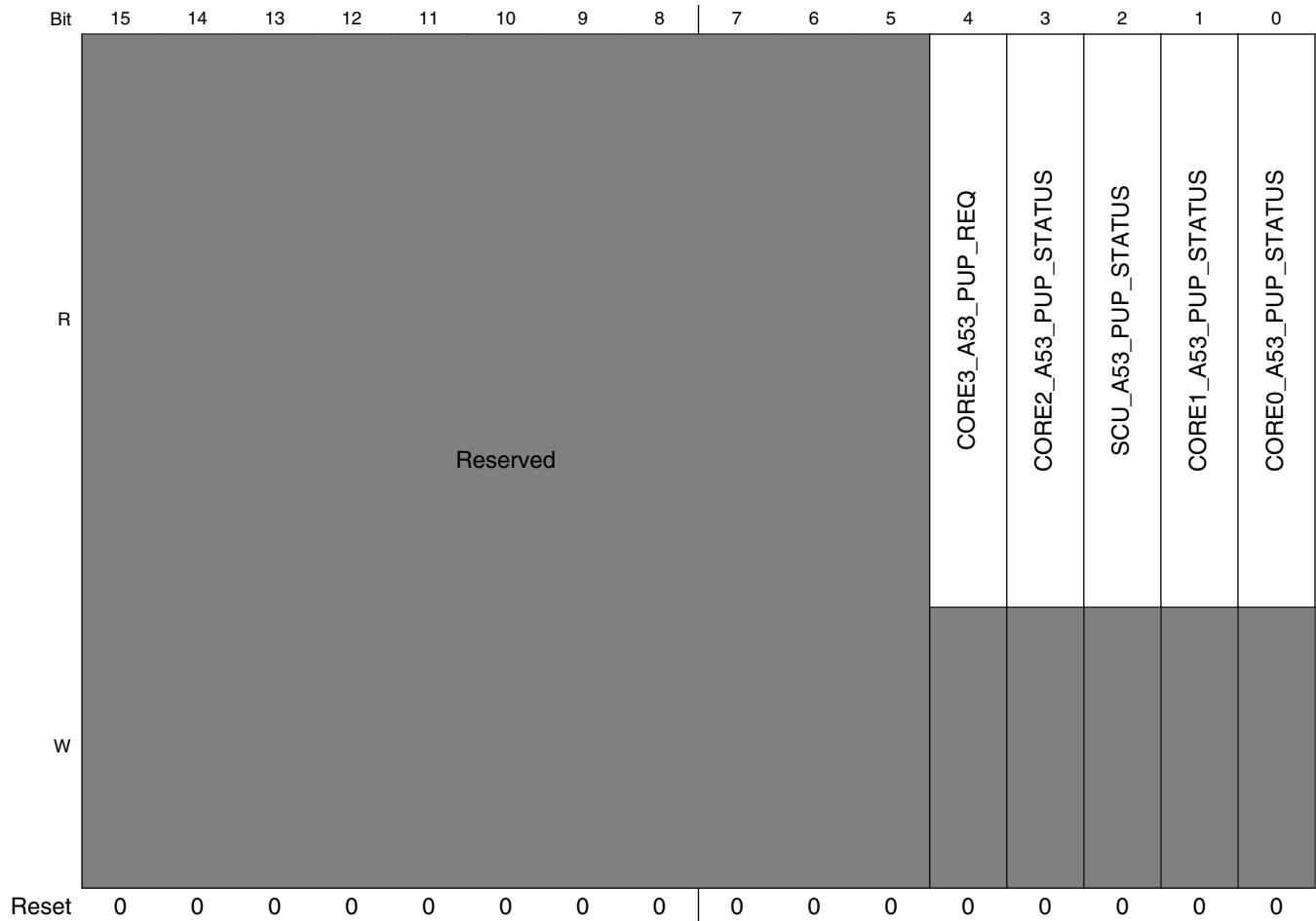
5.2.10.39 CPU PGC software up trigger status1 (GPC_CPU_PGC_PUP_STATUS1)

CPU_PGC_PUP_STATUS1 is a read only register, represents the results for power up software trigger for CPU type PGCs.

The field description is show in table below, the value of “1'b1” represent the software power up trigger failed because the relevant PGC is in a power down process. The relevant bit will be cleared after a success operation of power up software trigger for CPU type PGCs.

Address: 303A_0000h base + 130h offset = 303A_0130h





GPC_CPU_PGC_PUP_STATUS1 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 CORE3_A53_PUP_REQ	
3 CORE2_A53_PUP_STATUS	
2 SCU_A53_PUP_STATUS	
1 CORE1_A53_PUP_STATUS	
0 CORE0_A53_PUP_STATUS	

5.2.10.40 A53 MIX software up trigger status register (GPC_A53_MIX_PGC_PUP_STATUS_n)

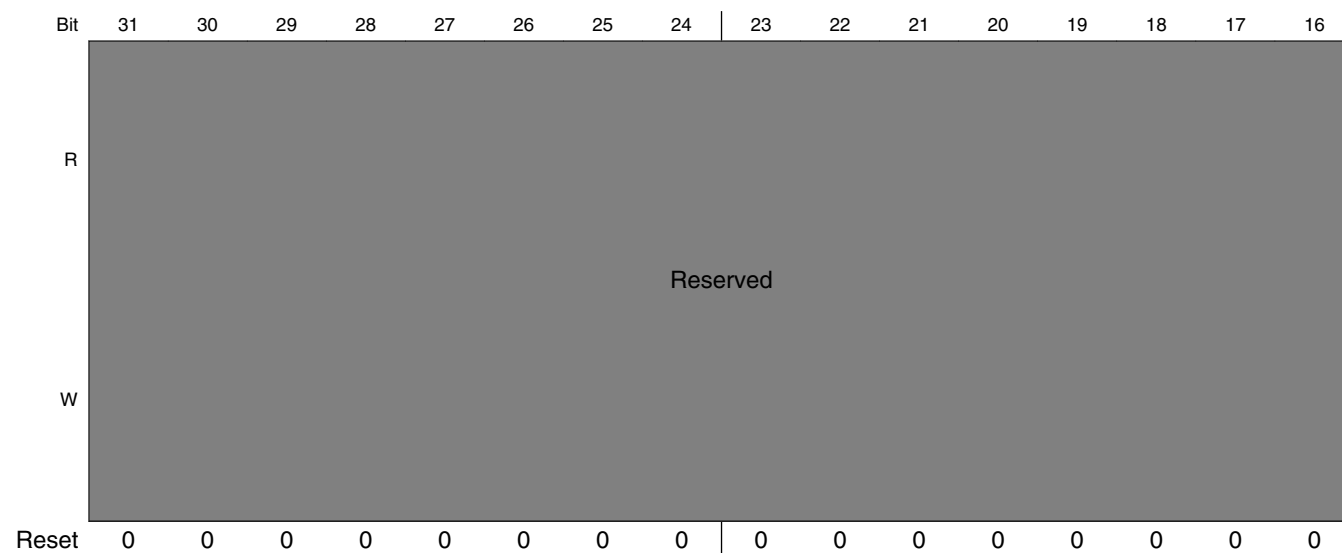
A53_MIX_PGC_PUP_STATUS_n (n = 0,1,2) are a read only register, represents the results for power up software trigger from A53 platform to MIX type PGCs.

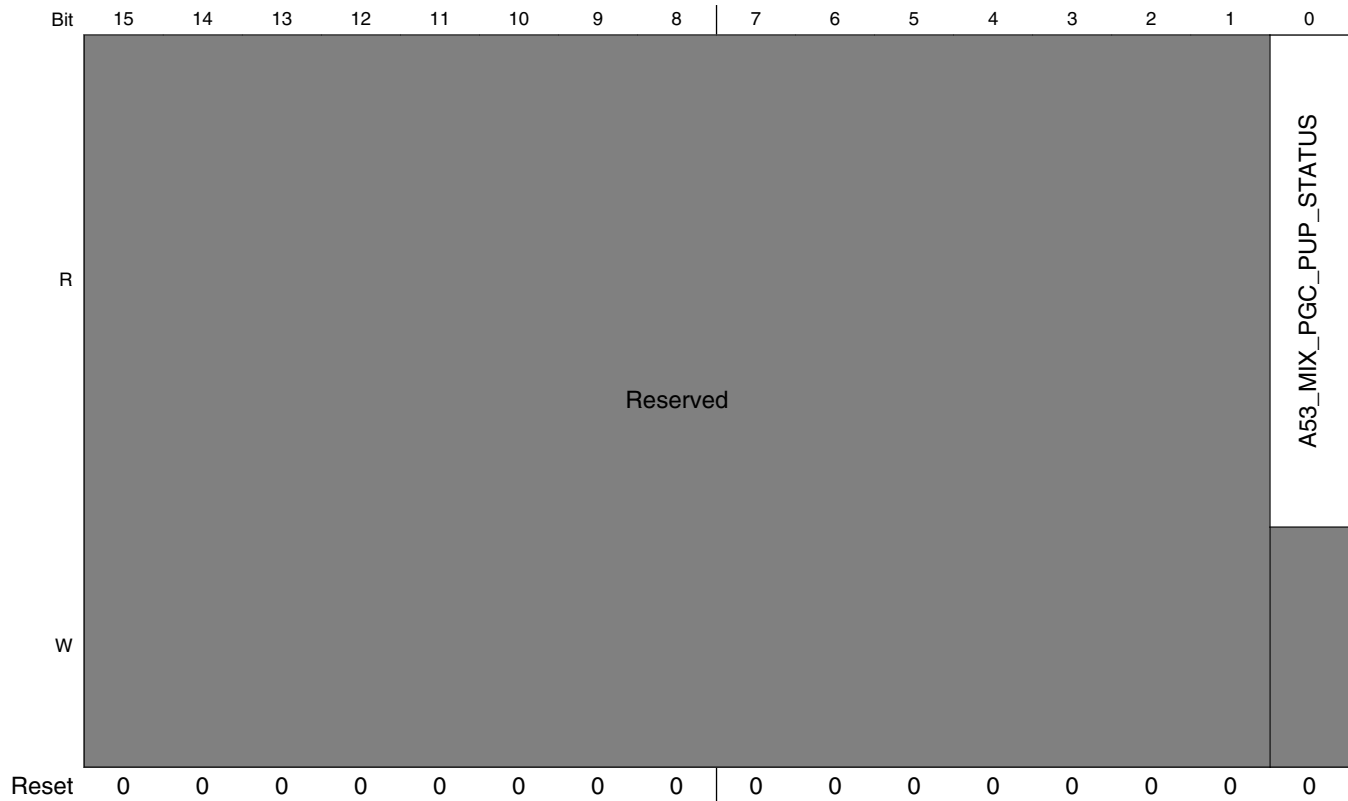
A53_MIX_PGC_PUP_STATUS0: value of “1'b1” represent the software power up trigger failed because domain control condition. The relevant bit will be cleared after a success operation of power up software trigger for MIX type PGCs.

A53_MIX_PGC_PUP_STATUS1: value of “1'b1” represent the software power up trigger failed because the relevant PGC is in a power up process. The relevant bit will be cleared after a success operation of power up software trigger for MIX type PGCs.

A53_MIX_PGC_PUP_STATUS2: value of “1'b1” represent the software power up trigger failed because time slot control is busy. The relevant bit will be cleared after a success operation of power up software trigger for MIX type PGCs.

Address: 303A_0000h base + 134h offset + (4d × i), where i=0d to 2d



**GPC_A53_MIX_PGC_PUP_STATUS_n field descriptions**

Field	Description
31–1 -	This field is reserved. Reserved
0 A53_MIX_PGC_PUP_STATUS	

5.2.10.41 M4 MIX PGC software up trigger status register (GPC_M4_MIX_PGC_PUP_STATUS_n)

M4_MIX_PGC_PUP_STATUS_n (n = 0,1,2) are a read only register, represents the results for power up software trigger from M4 platform to MIX type PGCs.

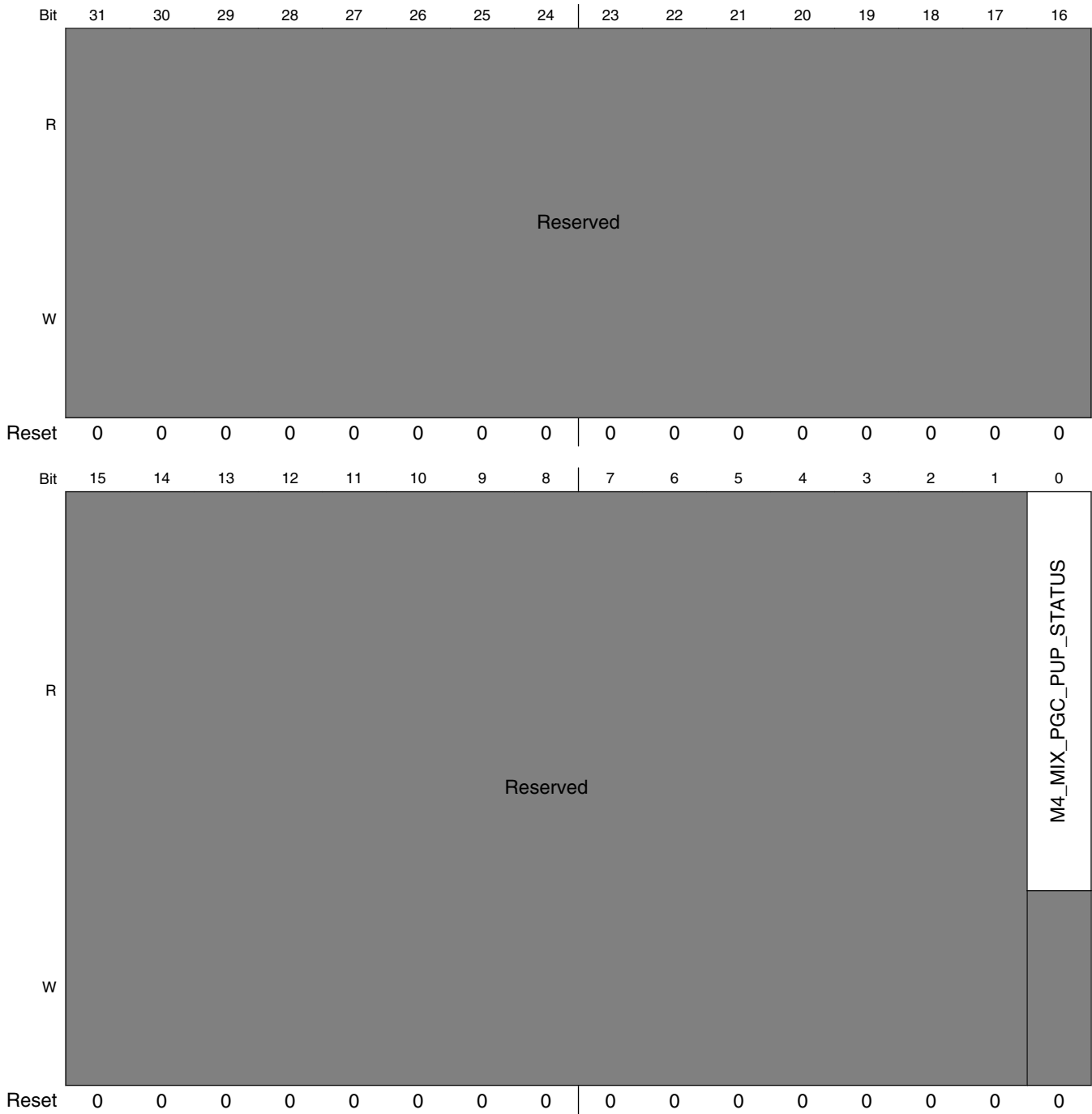
M4_MIX_PGC_PUP_STATUS0: value of “1'b1” represent the software power up trigger failed because domain control condition. The relevant bit will be cleared after a success operation of power up software trigger for MIX type PGCs.

M4_MIX_PGC_PUP_STATUS1: value of “1'b1” represent the software power up trigger failed because the relevant PGC is in a power up process. The relevant bit will be cleared after a success operation of power up software trigger for MIX type PGCs.

General Power Controller (GPC)

M4_MIX_PGC_PUP_STATUS2: value of “1'b1” represent the software power up trigger failed because time slot control is busy. The relevant bit will be cleared after a success operation of power up software trigger for MIX type PGCs.

Address: 303A_0000h base + 140h offset + (4d × i), where i=0d to 2d



GPC_M4_MIX_PGC_PUP_STATUS_n field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 M4_MIX_PGC_PUP_STATUS	

5.2.10.42 A53 PU software up trigger status register (GPC_A53_PU_PGC_PUP_STATUS_n)

A53_PU_PGC_PUP_STATUS_n (n = 0,1,2) are a read only register, represents the results for power up software trigger from A53 platform to PU type PGCs.

A53_PU_PGC_PUP_STATUS0: value of “1'b1” represent the software power up trigger failed because domain control condition. The relevant bit will be cleared after a success operation of power up software trigger for PU type PGCs.

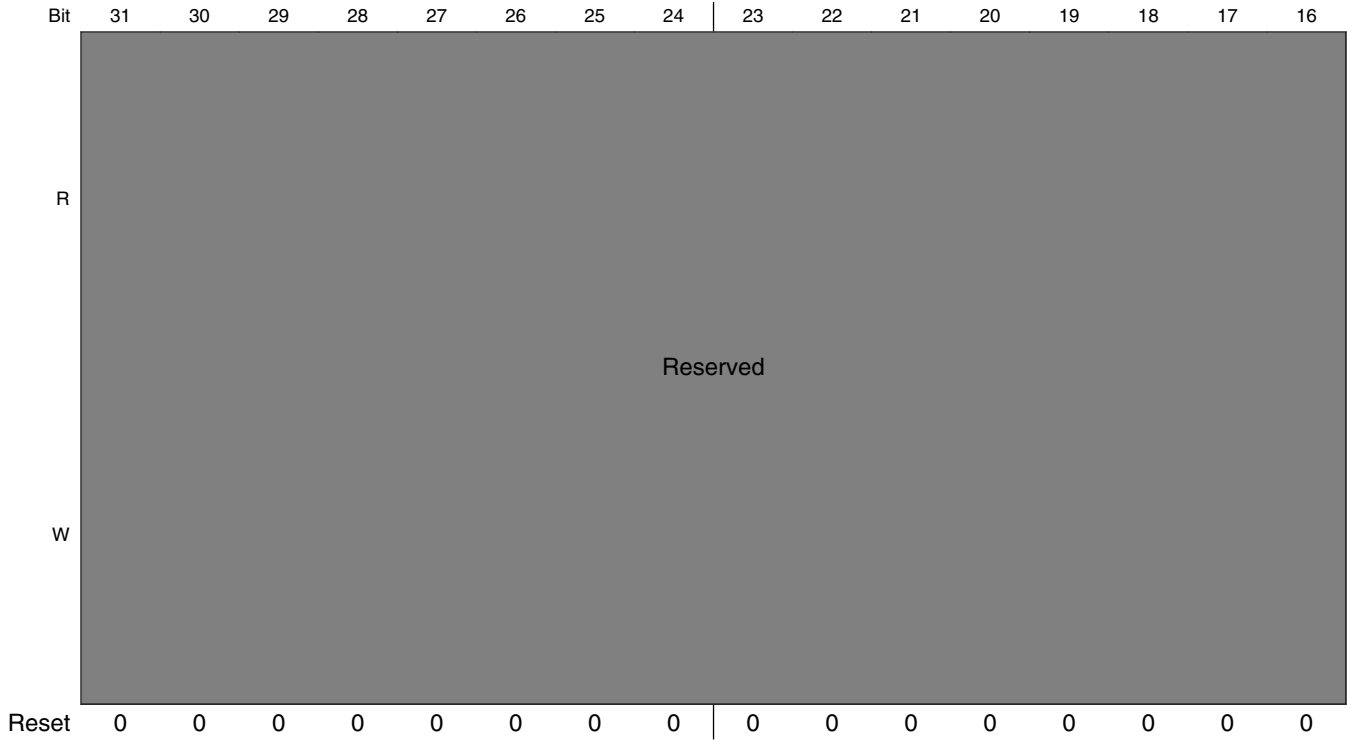
A53_PU_PGC_PUP_STATUS1: value of “1'b1” represent the software power up trigger failed because the relevant PGC is in a power up process. The relevant bit will be cleared after a success operation of power up software trigger for PU type PGCs.

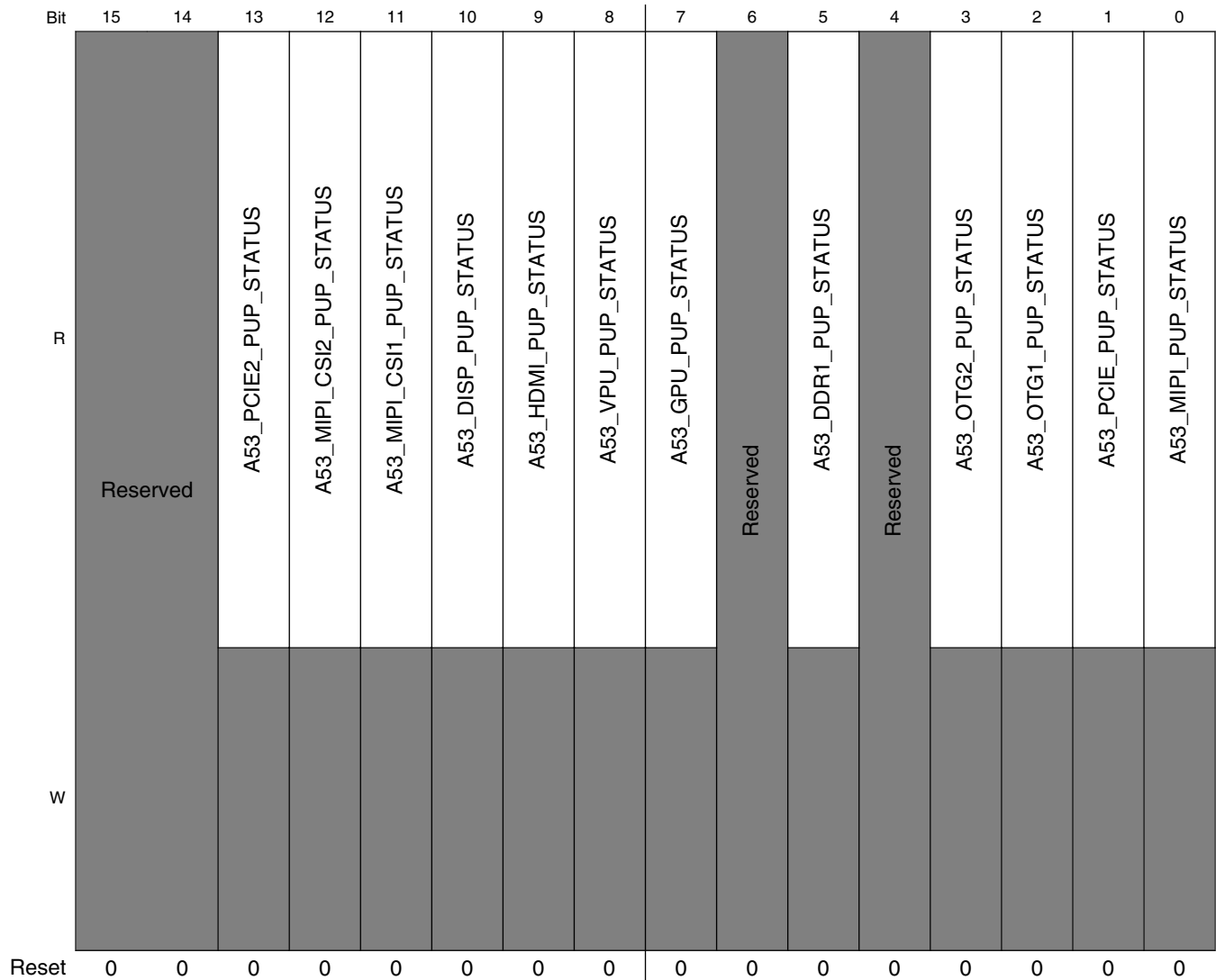
A53_PU_PGC_PUP_STATUS2: value of “1'b1” represent the software power up trigger failed because time slot control is busy. The relevant bit will be cleared after a success operation of power up software trigger for PU type PGCs.



General Power Controller (GPC)

Address: 303A_0000h base + 14Ch offset + (4d × i), where i=0d to 2d



GPC_A53_PU_PGC_PUP_STATUS n field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13 A53_PCIE2_PUP_STATUS	
12 A53_MIPI_CSI2_PUP_STATUS	
11 A53_MIPI_CSI1_PUP_STATUS	
10 A53_DISP_PUP_STATUS	

Table continues on the next page...

GPC_A53_PU_PGC_PUP_STATUS_n field descriptions (continued)

Field	Description
9 A53_HDMI_PUP_ _STATUS	
8 A53_VPU_PUP_ _STATUS	
7 A53_GPU_PUP_ _STATUS	
6 -	This field is reserved.
5 A53_DDR1_PUP_ _STATUS	
4 -	This field is reserved.
3 A53_OTG2_PUP_ _STATUS	
2 A53_OTG1_PUP_ _STATUS	
1 A53_PCIE_PUP_ _STATUS	
0 A53_MIPI_PUP_ _STATUS	

5.2.10.43 M4 PU PGC software up trigger status register (GPC_M4_PU_PGC_PUP_STATUS_n)

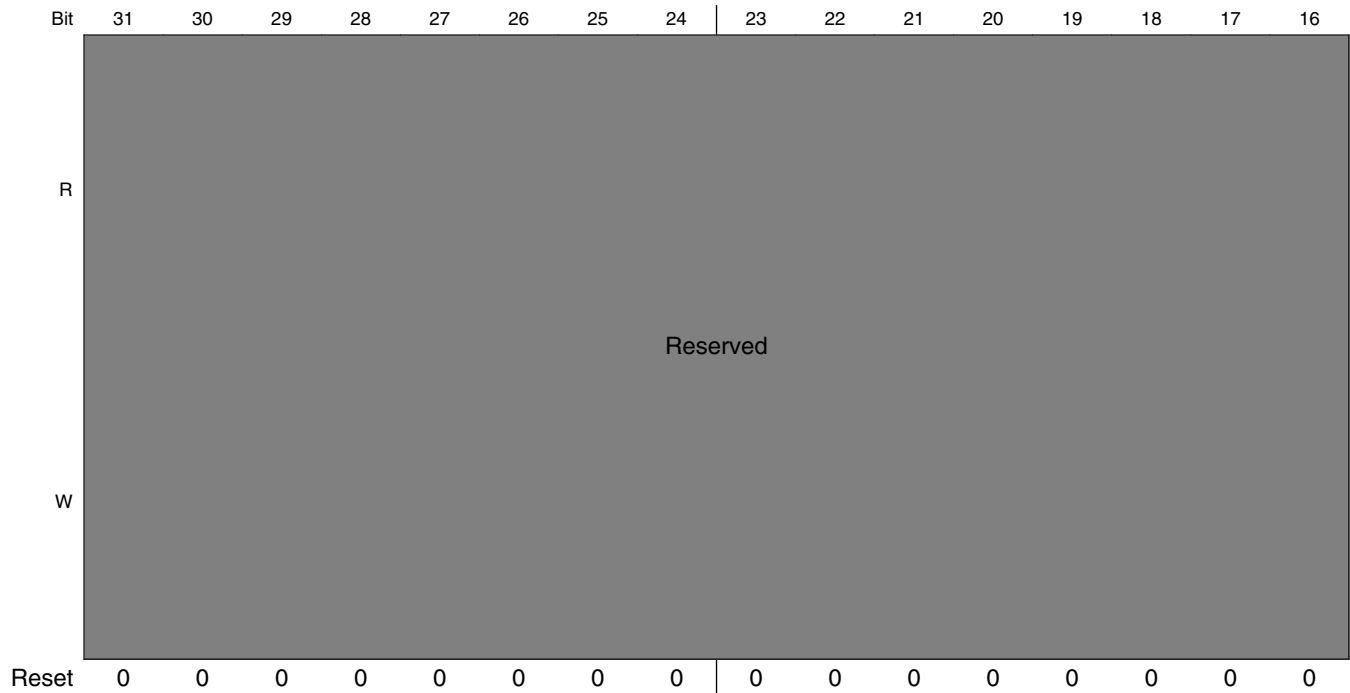
M4_PU_PGC_PUP_STATUS_n (n = 0,1,2) are a read only register, represents the results for power up software trigger from M4 platform to PU type PGCs.

M4_PU_PGC_PUP_STATUS0: value of “1'b1” represent the software power up trigger failed because domain control condition. The relevant bit will be cleared after a success operation of power up software trigger for PU type PGCs.

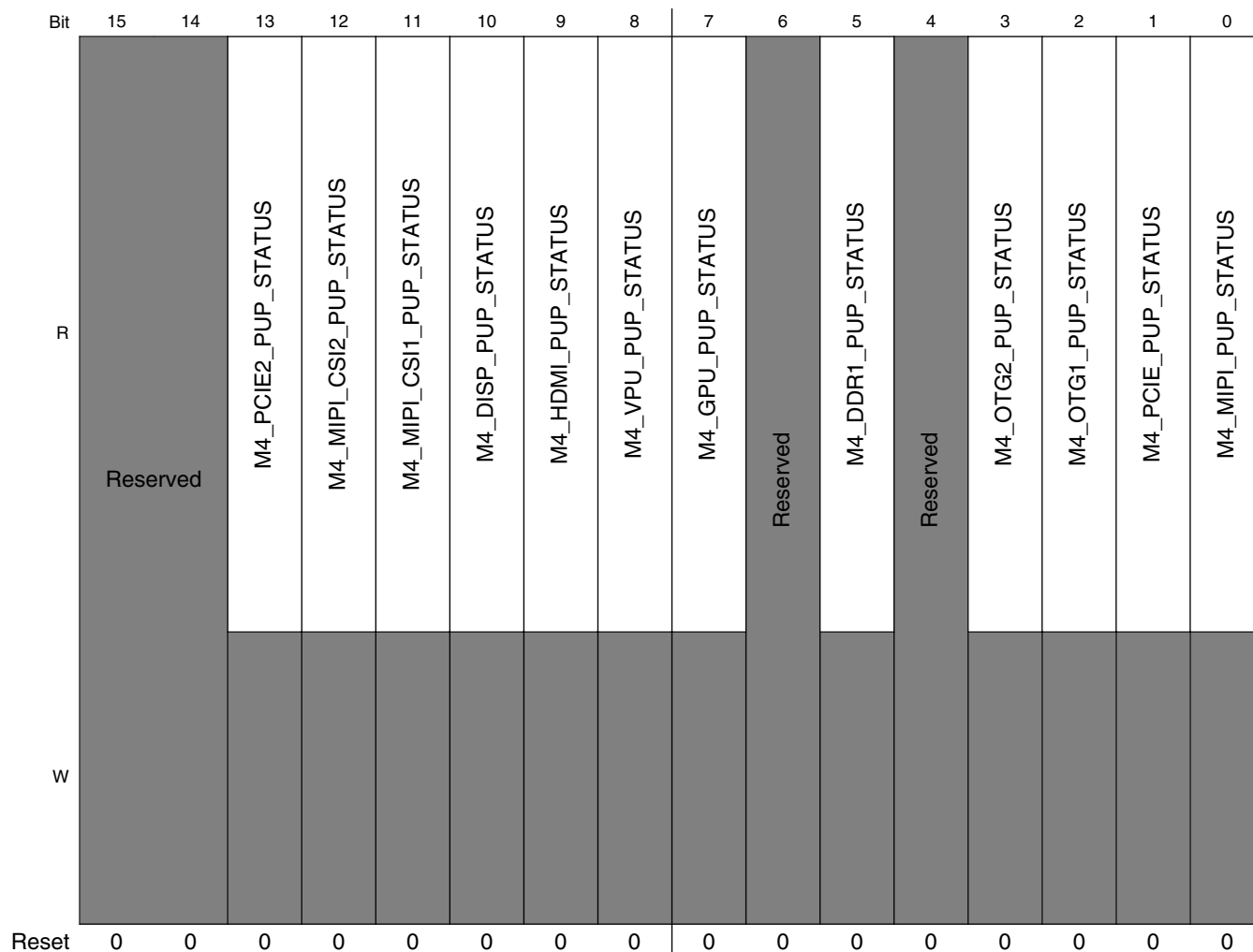
M4_PU_PGC_PUP_STATUS1: value of “1'b1” represent the software power up trigger failed because the relevant PGC is in a power up process. The relevant bit will be cleared after a success operation of power up software trigger for PU type PGCs.

M4_PU_PGC_PUP_STATUS2: value of “1'b1” represent the software power up trigger failed because time slot control is busy. The relevant bit will be cleared after a success operation of power up software trigger for PU type PGCs.

Address: 303A_0000h base + 158h offset + (4d × i), where i=0d to 2d



General Power Controller (GPC)



GPC_M4_PU_PGC_PUP_STATUSn field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13 M4_PCIE2_PUP_STATUS	
12 M4_MIPI_CSI2_PUP_STATUS	
11 M4_MIPI_CSI1_PUP_STATUS	
10 M4_DISP_PUP_STATUS	
9 M4_HDMI_PUP_STATUS	

Table continues on the next page...

GPC_M4_PU_PGC_PUP_STATUS_n field descriptions (continued)

Field	Description
8 M4_VPU_PUP_ STATUS	
7 M4_GPU_PUP_ STATUS	
6 -	This field is reserved.
5 M4_DDR1_PUP_ STATUS	
4 -	This field is reserved.
3 M4_OTG2_PUP_ STATUS	
2 M4_OTG1_PUP_ STATUS	
1 M4_PCIE_PUP_ STATUS	
0 M4_MIPI_PUP_ STATUS	

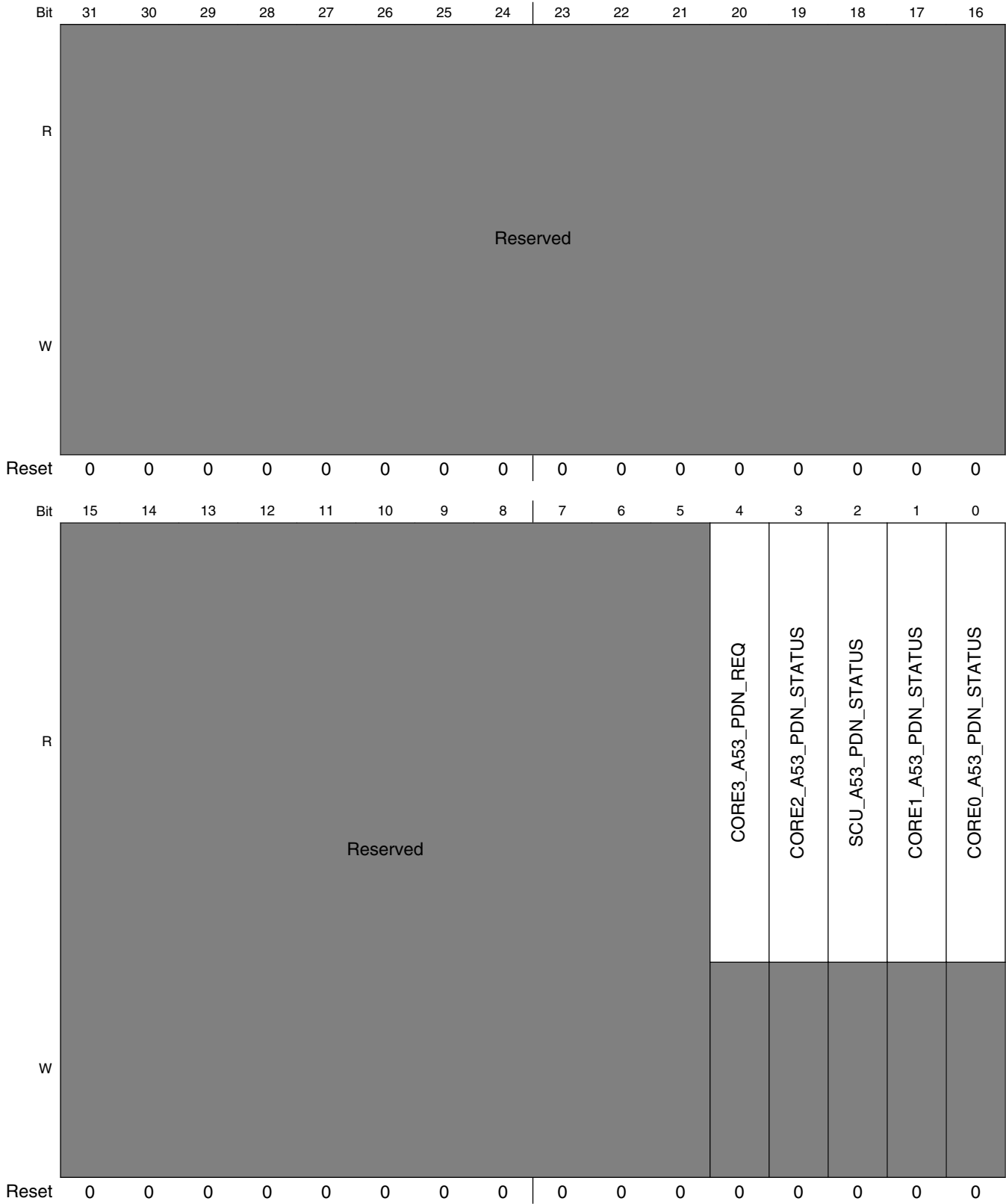
**5.2.10.44 CPU PGC software dn trigger status1
(GPC_CPU_PGC_PDN_STATUS1)**

CPU_PGC_PDN_STATUS1 is a read only register, represents the results for power DN software trigger for CPU type PGCs.

The field description is show in table below, the value of “1'b1” represent the software power DN trigger failed because the relevant PGC is in a power down process. The relevant bit will be cleared after a success operation of power DN software trigger for CPU type PGCs.

General Power Controller (GPC)

Address: 303A_0000h base + 170h offset = 303A_0170h



GPC_CPU_PGC_PDN_STATUS1 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 CORE3_A53_PDN_REQ	
3 CORE2_A53_PDN_STATUS	
2 SCU_A53_PDN_STATUS	
1 CORE1_A53_PDN_STATUS	
0 CORE0_A53_PDN_STATUS	

5.2.10.45 A53 MIX software down trigger status register (GPC_A53_MIX_PGC_PDN_STATUSn)

A53_MIX_PGC_PDN_STATUSn (n = 0,1,2) are a read only register, represents the results for power down software trigger from A53 platform to MIX type PGCs.

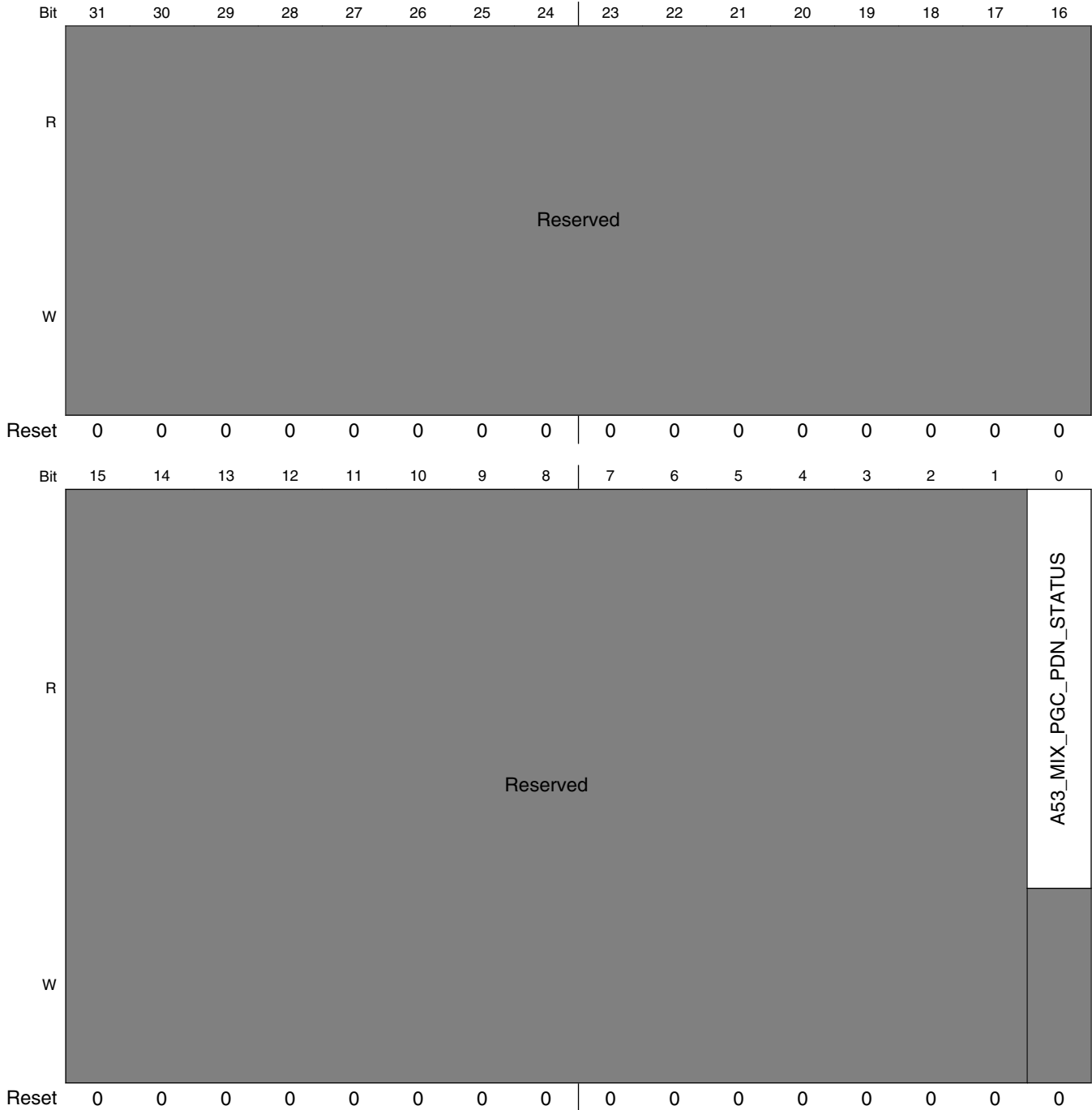
A53_MIX_PGC_PDN_STATUS0: value of “1'b1” represent the software power up trigger failed because domain control condition. The relevant bit will be cleared after a success operation of power down software trigger for MIX type PGCs.

A53_MIX_PGC_PDN_STATUS1: value of “1'b1” represent the software power up trigger failed because the relevant PGC is in a power up process. The relevant bit will be cleared after a success operation of power down software trigger for MIX type PGCs.

A53_MIX_PGC_PDN_STATUS2: value of “1'b1” represent the software power up trigger failed because time slot control is busy. The relevant bit will be cleared after a success operation of power down software trigger for MIX type PGCs.

General Power Controller (GPC)

Address: 303A_0000h base + 174h offset + (4d × i), where i=0d to 2d



GPC_A53_MIX_PGC_PDN_STATUSn field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 A53_MIX_PGC_PDN_STATUS	

5.2.10.46 M4 MIX PGC software power down trigger status register (GPC_M4_MIX_PGC_PDN_STATUS_n)

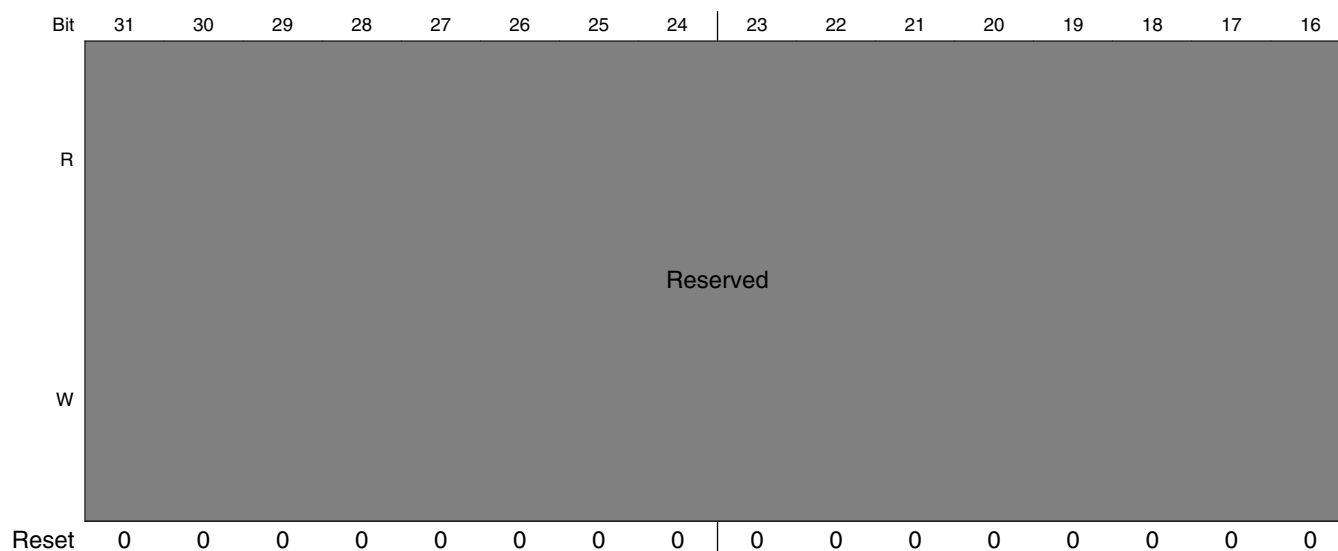
M4_MIX_PGC_PDN_STATUS_n (n = 0,1,2) are a read only register, represents the results for power down software trigger from M4 platform to MIX type PGCs.

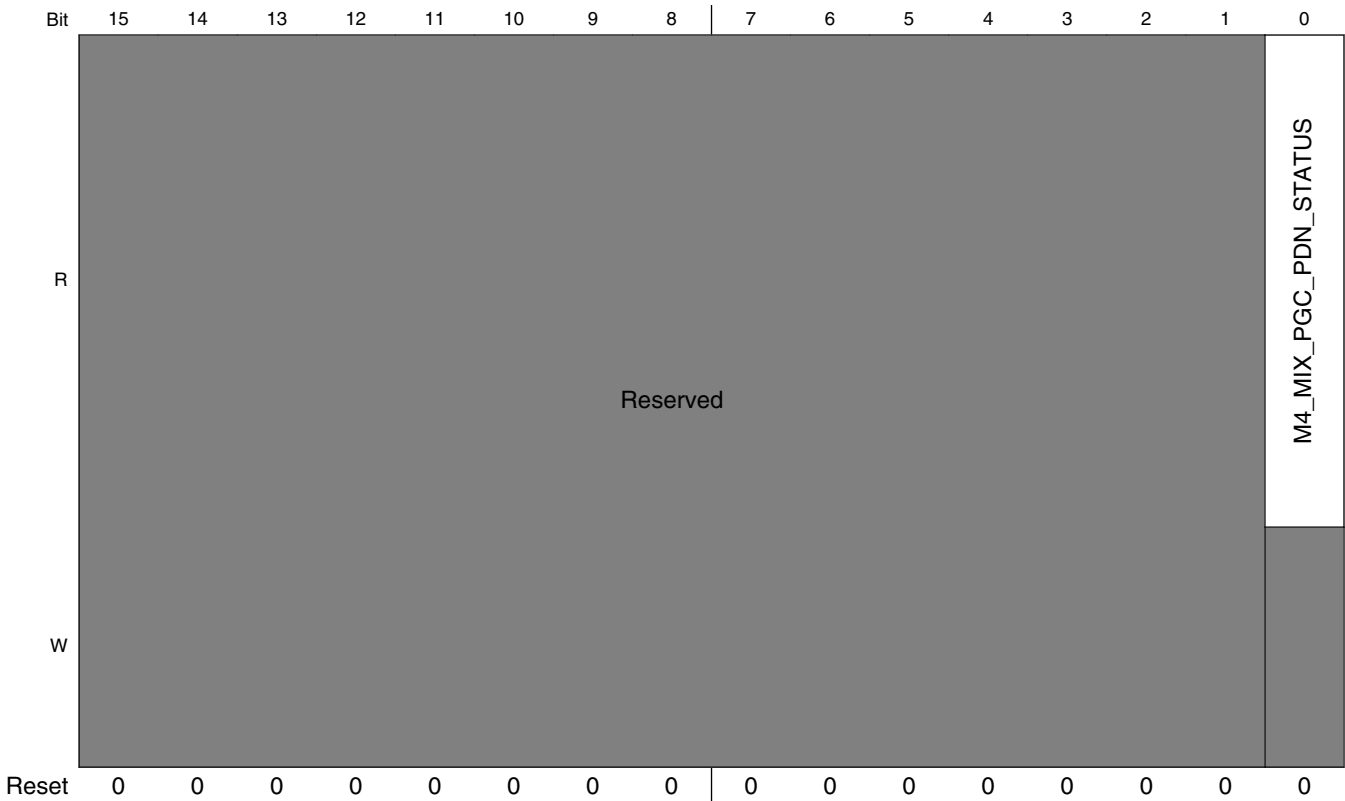
M4_MIX_PGC_PDN_STATUS0: value of “1'b1” represent the software power up trigger failed because domain control condition. The relevant bit will be cleared after a success operation of power down software trigger for MIX type PGCs.

M4_MIX_PGC_PDN_STATUS1: value of “1'b1” represent the software power up trigger failed because the relevant PGC is in a power up process. The relevant bit will be cleared after a success operation of power down software trigger for MIX type PGCs.

M4_MIX_PGC_PDN_STATUS2: value of “1'b1” represent the software power up trigger failed because time slot control is busy. The relevant bit will be cleared after a success operation of power down software trigger for MIX type PGCs.

Address: 303A_0000h base + 180h offset + (4d × i), where i=0d to 2d





GPC_M4_MIX_PGC_PDN_STATUSn field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 M4_MIX_PGC_PDN_STATUS	

5.2.10.47 A53 PU PGC software down trigger status (GPC_A53_PU_PGC_PDN_STATUSn)

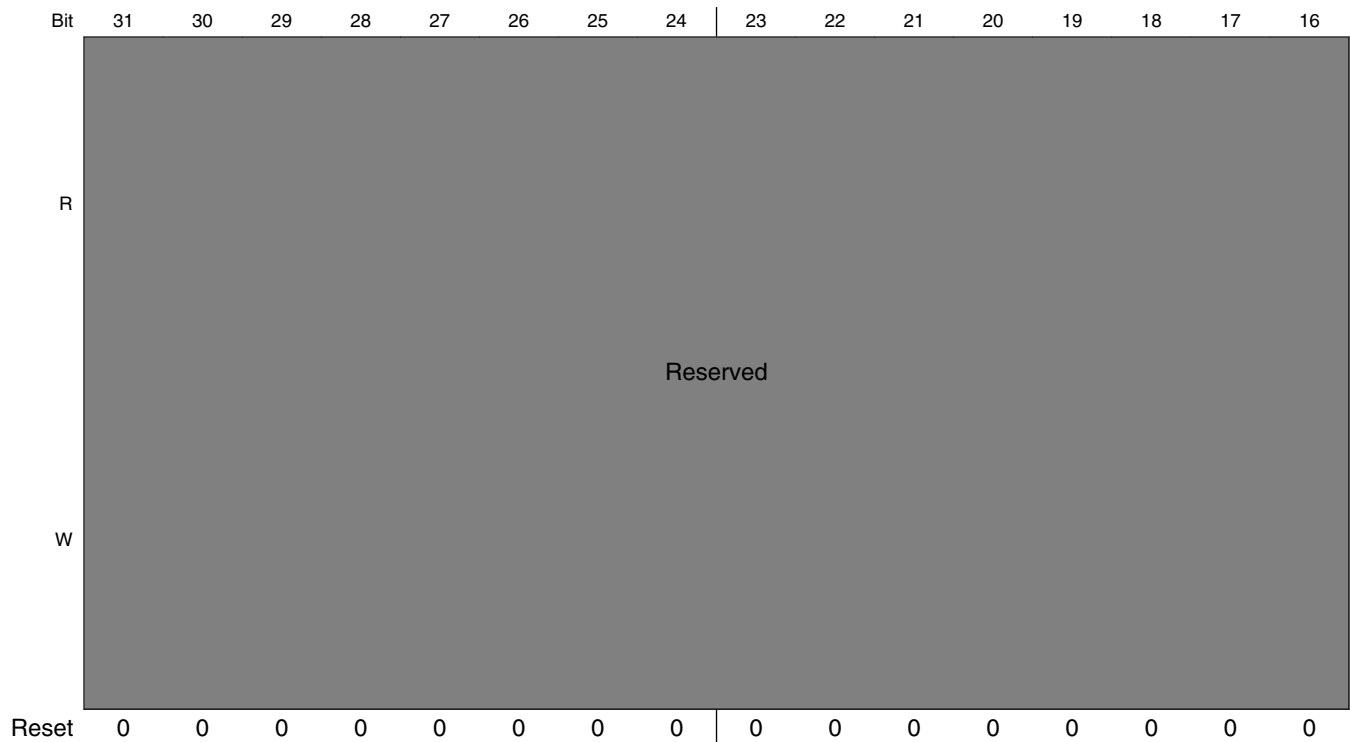
A53_PU_PGC_PDN_STATUSn (n = 0,1,2) are a read only register, represents the results for power DN software trigger from A53 platform to PU type PGCs.

A53_PU_PGC_PDN_STATUS0: value of “1'b1” represent the software power DN trigger failed because domain control condition. The relevant bit will be cleared after a success operation of power DN software trigger for PU type PGCs.

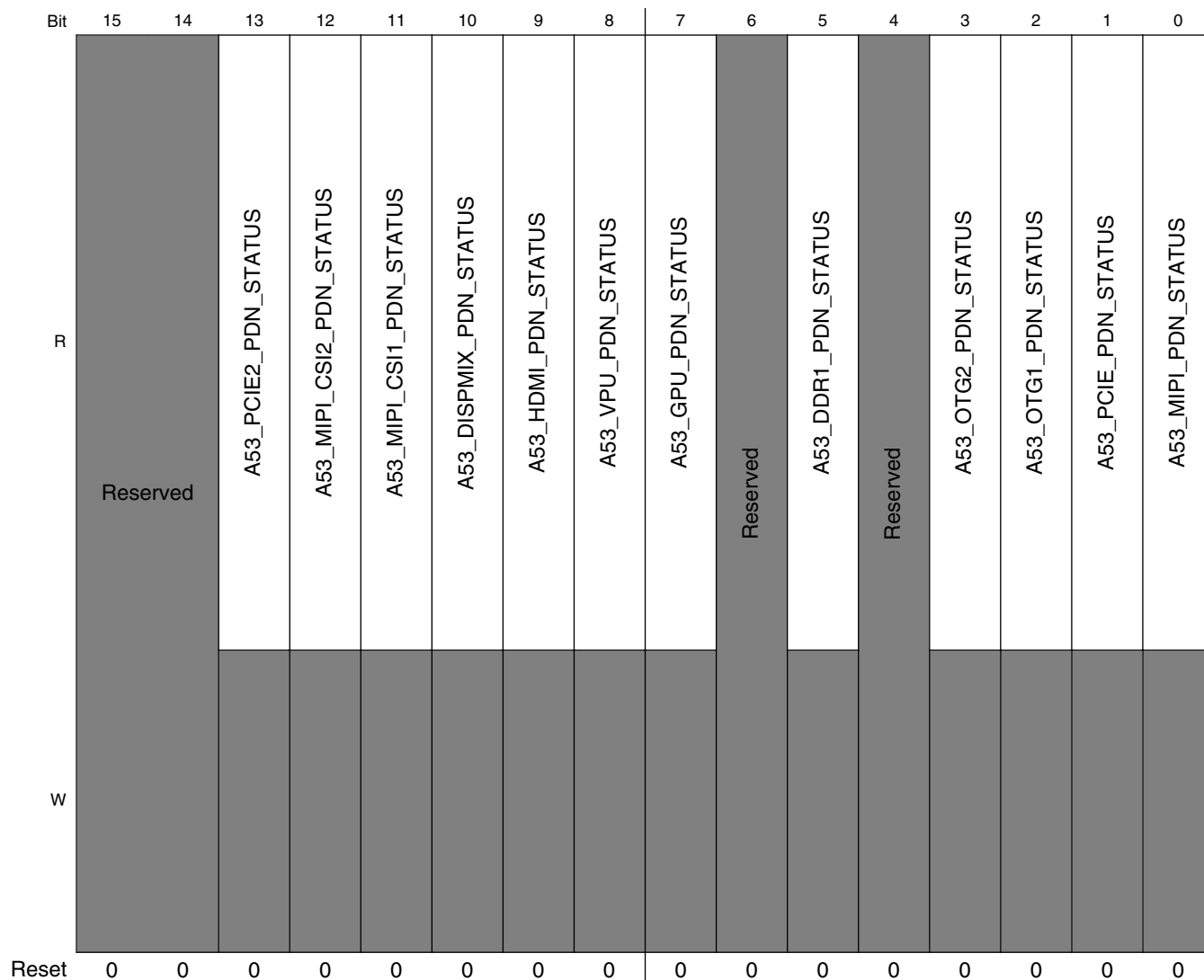
A53_PU_PGC_PDN_STATUS1: value of “1'b1” represent the software power DN trigger failed because the relevant PGC is in a power DN process. The relevant bit will be cleared after a success operation of power DN software trigger for PU type PGCs.

A53_PU_PGC_PDN_STATUS2: value of “1'b1” represent the software power DN trigger failed because time slot control is busy. The relevant bit will be cleared after a success operation of power DN software trigger for PU type PGCs.

Address: 303A_0000h base + 18Ch offset + (4d × i), where i=0d to 2d



General Power Controller (GPC)



GPC_A53_PU_PGC_PDN_STATUSn field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13 A53_PCIE2_PDN_STATUS	
12 A53_MIPI_CSI2_PDN_STATUS	
11 A53_MIPI_CSI1_PDN_STATUS	
10 A53_DISPMIX_PDN_STATUS	

Table continues on the next page...

GPC_A53_PU_PGC_PDN_STATUS_n field descriptions (continued)

Field	Description
9 A53_HDMI_PDN_STATUS	
8 A53_VPU_PDN_STATUS	
7 A53_GPU_PDN_STATUS	
6 -	This field is reserved.
5 A53_DDR1_PDN_STATUS	
4 -	This field is reserved.
3 A53_OTG2_PDN_STATUS	
2 A53_OTG1_PDN_STATUS	
1 A53_PCIE_PDN_STATUS	
0 A53_MIPI_PDN_STATUS	

5.2.10.48 M4 PU PGC software down trigger status (GPC_M4_PU_PGC_PDN_STATUS_n)

M4_PU_PGC_PDN_STATUS_n (n = 0,1,2) are a read only register, represents the results for power DN software trigger from M4 platform to PU type PGCs.

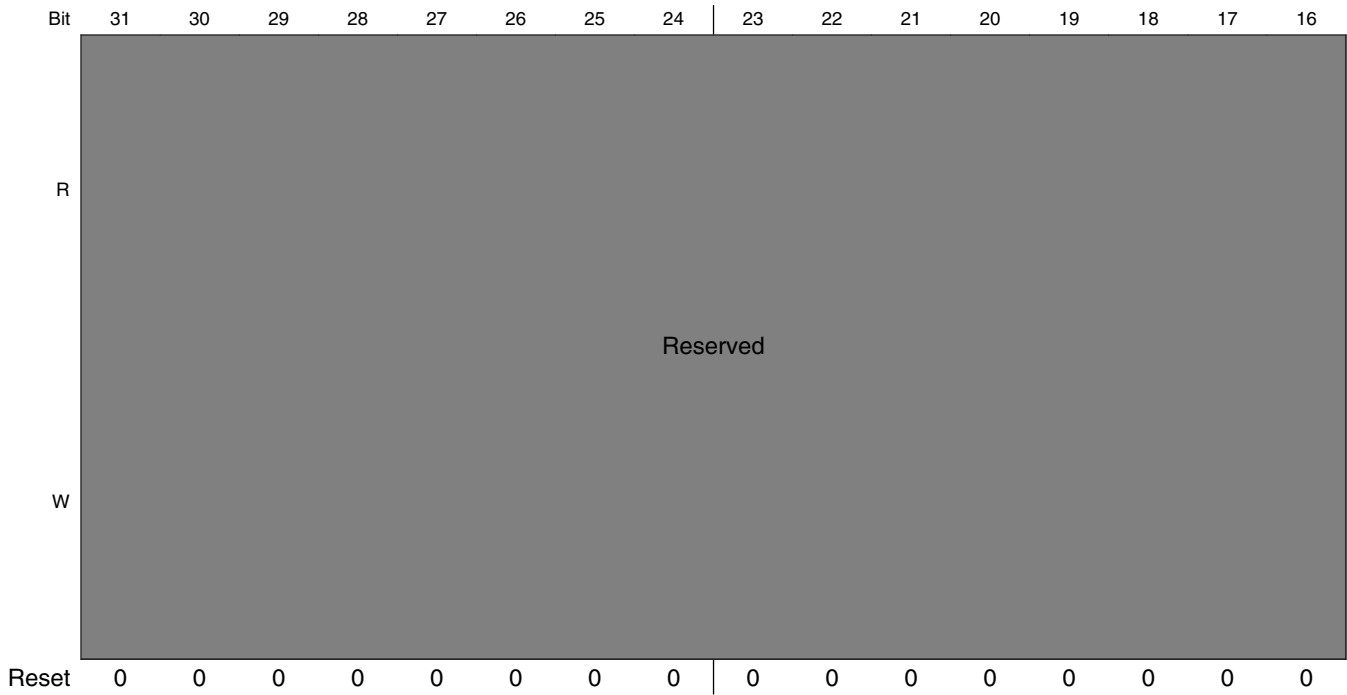
M4_PU_PGC_PDN_STATUS0: value of “1'b1” represent the software power DN trigger failed because domain control condition. The relevant bit will be cleared after a success operation of power DN software trigger for PU type PGCs.

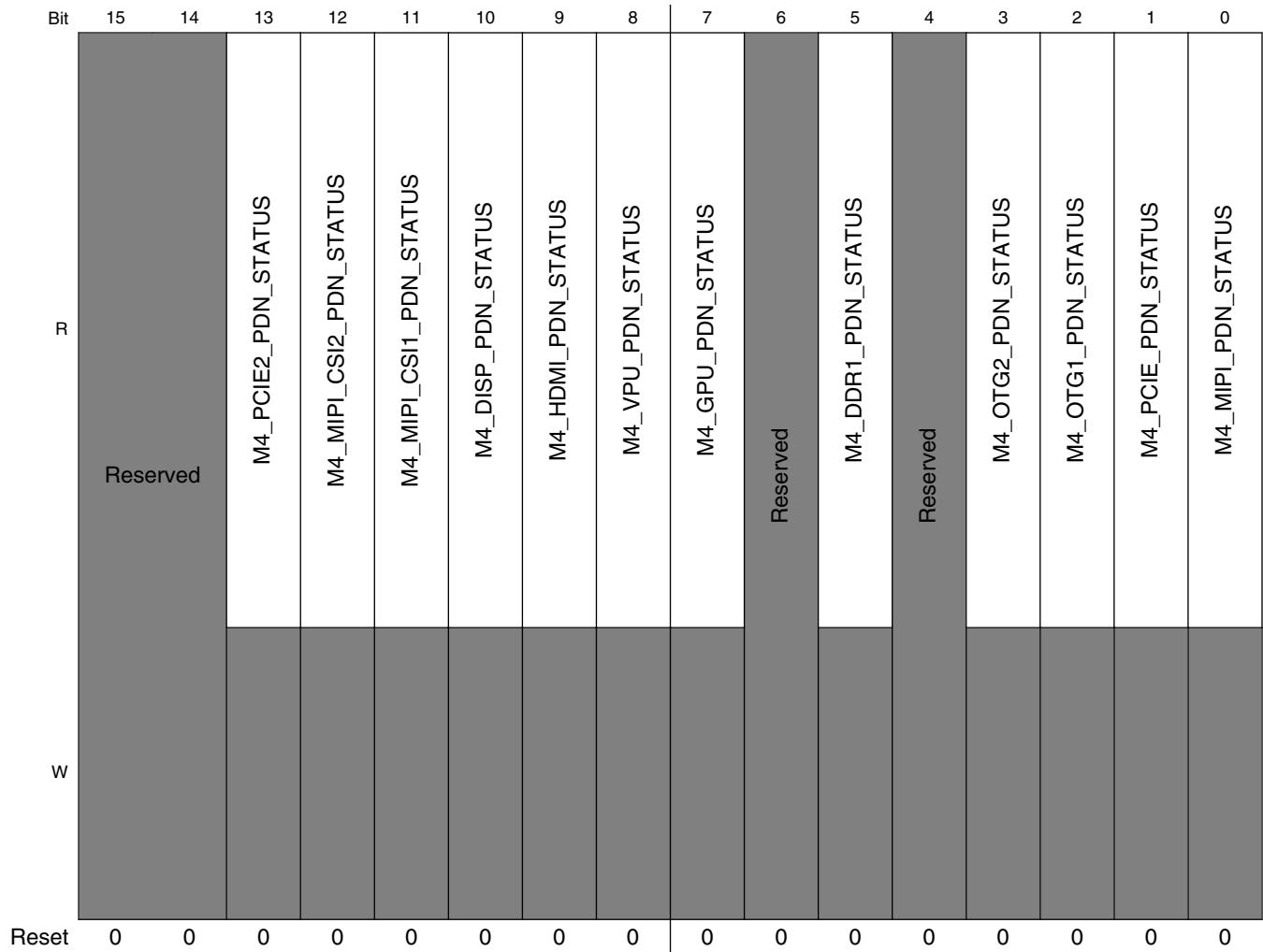
M4_PU_PGC_PDN_STATUS1: value of “1'b1” represent the software power DN trigger failed because the relevant PGC is in a power DN process. The relevant bit will be cleared after a success operation of power DN software trigger for PU type PGCs.

General Power Controller (GPC)

M4_PU_PGC_PDN_STATUS2: value of “1'b1” represent the software power DN trigger failed because time slot control is busy. The relevant bit will be cleared after a success operation of power DN software trigger for PU type PGCs.

Address: 303A_0000h base + 198h offset + (4d × i), where i=0d to 2d





GPC_M4_PU_PGC_PDN_STATUSn field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13 M4_PCIE2_PDN_STATUS	
12 M4_MIPI_CS12_PDN_STATUS	
11 M4_MIPI_CS11_PDN_STATUS	
10 M4_DISP_PDN_STATUS	
9 M4_HDMI_PDN_STATUS	

Table continues on the next page...

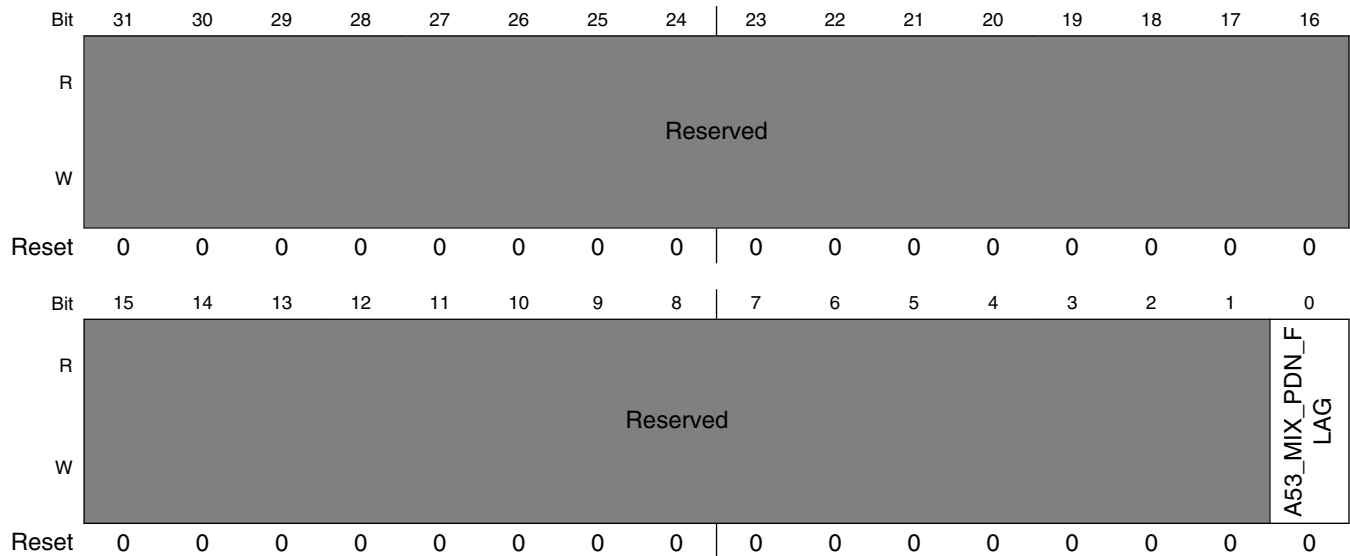
GPC_M4_PU_PGC_PDN_STATUS_n field descriptions (continued)

Field	Description
8 M4_VPU_PDN_ STATUS	
7 M4_GPU_PDN_ STATUS	
6 -	This field is reserved.
5 M4_DDR1_PDN_ STATUS	
4 -	This field is reserved.
3 M4_OTG2_PDN_ STATUS	
2 M4_OTG1_PDN_ STATUS	
1 M4_PCIE_PDN_ STATUS	
0 M4_MIPI_PDN_ STATUS	

5.2.10.49 A53 MIX PDN FLG (GPC_A53_MIX_PDN_FLG)

This is flag bit relevant domain control, represents A53 CPU platform wants to power down MIX PGC. The register can only be accessed by A53 platform.

Address: 303A_0000h base + 1B0h offset = 303A_01B0h



GPC_A53_MIX_PDN_FLG field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 A53_MIX_PDN_ FLAG	A53 MIX power-down flag

5.2.10.50 A53 PU PDN FLG (GPC_A53_PU_PDN_FLG)

The register field is show in the table below. The 1'b1 represents A53 CPU platform wants to power down certain PU PGC. The register is a read only register. The register bits will be set when corresponding A53 software power down trigger happens and will be clear when corresponding A53 software power up trigger happens.

Address: 303A_0000h base + 1B4h offset = 303A_01B4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved																A53_PU_PDN_FLG															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPC_A53_PU_PDN_FLG field descriptions

Field	Description
31-14 -	This field is reserved. Reserved
A53_PU_PDN_FLG	A53 PGC power-down flag

5.2.10.51 M4 MIX PDN FLG (GPC_M4_MIX_PDN_FLG)

This is flag bit relevant domain control, represents M4 CPU platform wants to power down MIX PGC. The register can only be accessed by M4 platform.

Address: 303A_0000h base + 1B8h offset = 303A_01B8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															M4_MIX_PDN_FLAG
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_M4_MIX_PDN_FLG field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 M4_MIX_PDN_FLAG	M4_MIX power-down flag

5.2.10.52 M4 PU PDN FLG (GPC_M4_PU_PDN_FLG)

The register field is show in the table below. The 1'b1 represents M4 CPU platform wants to power down certain PU PGC. The register is a read only register. The register bits will be set when corresponding M4 software power down trigger happens and will be clear when corresponding M4 software power up trigger happens.

Address: 303A_0000h base + 1BCh offset = 303A_01BCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W	Reserved																M4_PU_PDN_FLG															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPC_M4_PU_PDN_FLG field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
M4_PU_PDN_FLG	M4 power-down flag

5.2.10.53 IRQ masking register 1 of A53 core2 (GPC_IMR1_CORE2_A53)

The four IMRn_CORE2_A53 (n = 1,2,3,4) registers are used as interrupt mask for A53 core2.

Address: 303A_0000h base + 1C0h offset = 303A_01C0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR1_CORE2_A53																															
W	IMR1_CORE2_A53																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPC_IMR1_CORE2_A53 field descriptions

Field	Description
IMR1_CORE2_A53	A53 core2 IRQ[31:0] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.54 IRQ masking register 2 of A53 core2 (GPC_IMR2_CORE2_A53)

Address: 303A_0000h base + 1C4h offset = 303A_01C4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR2_CORE2_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR2_CORE2_A53 field descriptions

Field	Description
IMR2_CORE2_A53	A53 core2 IRQ[63:32] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.55 IRQ masking register 3 of A53 core2 (GPC_IMR3_CORE2_A53)

Address: 303A_0000h base + 1C8h offset = 303A_01C8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR3_CORE2_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR3_CORE2_A53 field descriptions

Field	Description
IMR3_CORE2_A53	A53 core2 IRQ[95:64] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.56 IRQ masking register 4 of A53 core2 (GPC_IMR4_CORE2_A53)

Address: 303A_0000h base + 1CCh offset = 303A_01CCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR4_CORE2_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR4_CORE2_A53 field descriptions

Field	Description
IMR4_CORE2_A53	A53 core2 IRQ[127:96] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.57 IRQ masking register 1 of A53 core3 (GPC_IMR1_CORE3_A53)

The four IMRn_CORE2_A53 (n = 1,2,3,4) registers are used as interrupt mask for A53 core3.

Address: 303A_0000h base + 1D0h offset = 303A_01D0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR1_CORE3_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR1_CORE3_A53 field descriptions

Field	Description
IMR1_CORE3_A53	A53 core3 IRQ[31:0] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.58 IRQ masking register 2 of A53 core3 (GPC_IMR2_CORE3_A53)

Address: 303A_0000h base + 1D4h offset = 303A_01D4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR2_CORE3_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR2_CORE3_A53 field descriptions

Field	Description
IMR2_CORE3_A53	A53 core3 IRQ[63:32] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.59 IRQ masking register 3 of A53 core3 (GPC_IMR3_CORE3_A53)

Address: 303A_0000h base + 1D8h offset = 303A_01D8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR3_CORE3_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR3_CORE3_A53 field descriptions

Field	Description
IMR3_CORE3_A53	A53 core3 IRQ[95:64] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.60 IRQ masking register 4 of A53 core3 (GPC_IMR4_CORE3_A53)

Address: 303A_0000h base + 1DCh offset = 303A_01DCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	IMR4_CORE3_A53																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_IMR4_CORE3_A53 field descriptions

Field	Description
IMR4_CORE3_A53	A53 core3 IRQ[127:96] masking bits: 0 IRQ not masked 1 IRQ masked

5.2.10.61 PGC acknowledge signal selection of A53 platform for PUs (GPC_ACK_SEL_A53_PU)

Address: 303A_0000h base + 1E0h offset = 303A_01E0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PCIE2_PGC_PUP_ACK	MIPI_CSI2_PGC_PUP_A CK	MIPI_CSI1_PGC_PUP_A CK	DISP_PGC_PUP_ACK	HDMI_PGC_PUP_ACK	VPU_PGC_PUP_ACK	GPU_PGC_PUP_ACK	DDR2_PGC_PUP_ACK	DDR1_PGC_PUP_ACK	Reserved	USB_OTG2_PGC_PUP_ ACK	USB_OTG1_PGC_PUP_ ACK	PCIE_PGC_PUP_ACK	MIPI_PGC_PUP_ACK	Reserved	MF_PGC_PUP_ACK
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	PCIE2_PGC_PDN_ACK	MIPI_CSI2_PGC_PDN_A CK	MIPI_CSI1_PGC_PDN_A CK	DISP_PGC_PDN_ACK	HDMI_PGC_PDN_ACK	VPU_PGC_PDN_ACK	GPU_PGC_PDN_ACK	DDR2_PGC_PDN_ACK	DDR1_PGC_PDN_ACK	Reserved	USB_OTG2_PGC_PDN_ ACK	USB_OTG1_PGC_PDN_ ACK	PCIE_PGC_PDN_ACK	MIPI_PGC_PDN_ACK	Reserved	MF_PGC_PDN_ACK

GPC_ACK_SEL_A53_PU field descriptions

Field	Description
31 PCIE2_PGC_PU P_ACK	Select power down acknowledge signal of PCIE2 PGC as the power up acknowledge for A53 LPM.
30 MIPI_CSI2_PGC _PUP_ACK	Select power down acknowledge signal of MIPI_CSI2 PGC as the power up acknowledge for A53 LPM.
29 MIPI_CSI1_PGC _PUP_ACK	Select power down acknowledge signal of MIPI_CSI1 PGC as the power up acknowledge for A53 LPM.
28 DISP_PGC_ PUP_ACK	Select power down acknowledge signal of DISP PGC as the power up acknowledge for A53 LPM.
27 HDMI_PGC_PUP _ACK	Select power down acknowledge signal of HDMI PGC as the power up acknowledge for A53 LPM.
26 VPU_PGC_PUP_ ACK	Select power down acknowledge signal of VPU PGC as the power up acknowledge for A53 LPM.
25 GPU_PGC_ PUP_ACK	Select power down acknowledge signal of GPU PGC as the power up acknowledge for A53 LPM.
24 DDR2_PGC_PU P_ACK	Select power down acknowledge signal of DDR2 PGC as the power up acknowledge for A53 LPM.
23 DDR1_PGC_ PUP_ACK	Select power down acknowledge signal of DDR1 PGC as the power up acknowledge for A53 LPM.
22 -	This field is reserved.
21 USB_OTG2_ PGC_PUP_ACK	Select power down acknowledge signal of USB_OTG2 PGC as the power up acknowledge for A53 LPM.
20 USB_OTG1_ PGC_PUP_ACK	Select power down acknowledge signal of USB_OTG1 PGC as the power up acknowledge for A53 LPM.

Table continues on the next page...

GPC_ACK_SEL_A53_PU field descriptions (continued)

Field	Description
19 PCIE_PGC_PUP_ACK	Select power down acknowledge signal of PCIE PGC as the power up acknowledge for A53 LPM.
18 MIPI_PGC_PUP_ACK	Select power down acknowledge signal of MIPI PGC as the power up acknowledge for A53 LPM.
17 -	This field is reserved.
16 MF_PGC_PUP_ACK	Select power down acknowledge signal of MIX PGC as the power up acknowledge for A53 LPM.
15 PCIE2_PGC_PD_N_ACK	Select power down acknowledge signal of PCIE2 PGC as the power down acknowledge for A53 LPM.
14 MIPI_CSI2_PGC_PDN_ACK	Select power down acknowledge signal of MIPI_CSI2 PGC as the power down acknowledge for A53 LPM.
13 MIPI_CSI1_PGC_PDN_ACK	Select power down acknowledge signal of MIPI_CSI1 PGC as the power down acknowledge for A53 LPM.
12 DISP_PGC_PDN_ACK	Select power down acknowledge signal of DISP PGC as the power down acknowledge for A53 LPM.
11 HDMI_PGC_PDN_ACK	Select power down acknowledge signal of HDMI PGC as the power down acknowledge for A53 LPM.
10 VPU_PGC_PDN_ACK	Select power down acknowledge signal of VPU PGC as the power down acknowledge for A53 LPM.
9 GPU_PGC_PDN_ACK	Select power down acknowledge signal of GPU PGC as the power down acknowledge for A53 LPM.
8 DDR2_PGC_PD_N_ACK	Select power down acknowledge signal of DDR2 PGC as the power down acknowledge for A53 LPM.
7 DDR1_PGC_PDN_ACK	Select power down acknowledge signal of DDR1 PGC as the power down acknowledge for A53 LPM.
6 -	This field is reserved. Reserved
5 USB_OTG2_PGC_PDN_ACK	Select power down acknowledge signal of USB_OTG2 PGC as the power down acknowledge for A53 LPM.
4 USB_OTG1_PGC_PDN_ACK	Select power down acknowledge signal of USB_OTG1 PGC as the power down acknowledge for A53 LPM.
3 PCIE_PGC_PDN_ACK	Select power down acknowledge signal of PCIE PGC as the power down acknowledge for A53 LPM.

Table continues on the next page...

GPC_ACK_SEL_A53_PU field descriptions (continued)

Field	Description
2 MIPI_PGC_PDN_ACK	Select power down acknowledge signal of MIPI PGC as the power down acknowledge for A53 LPM.
1 -	This field is reserved. Reserved
0 MF_PGC_PDN_ACK	Select power down acknowledge signal of MIX PGC as the power down acknowledge for A53 LPM.

5.2.10.62 PGC acknowledge signal selection of M4 platform for PUs (GPC_ACK_SEL_M4_PU)

Address: 303A_0000h base + 1E4h offset = 303A_01E4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PCIE2_PGC_PUP_ACK	MIPI_CSI2_PGC_PUP_A CK	MIPI_CSI1_PGC_PUP_A CK	DISP_PGC_PUP_ACK	HDMI_PGC_PUP_ACK	VPU_PGC_PUP_ACK	GPU_PGC_PUP_ACK	DDR2_PGC_PUP_ACK	DDR1_PGC_PUP_ACK	Reserved	USB_OTG2_PGC_PUP_ ACK	USB_OTG1_PGC_PUP_ ACK	PCIE_PGC_PUP_ACK	MIPI_PGC_PUP_ACK	Reserved	MF_PGC_PUP_ACK
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PCIE2_PGC_PDN_ACK	MIPI_CSI2_PGC_PDN_A CK	MIPI_CSI1_PGC_PDN_A CK	DISP_PGC_PDN_ACK	HDMI_PGC_PDN_ACK	VPU_PGC_PDN_ACK	GPU_PGC_PDN_ACK	DDR2_PGC_PDN_ACK	DDR1_PGC_PDN_ACK	Reserved	USB_OTG2_PGC_PDN_ ACK	USB_OTG1_PGC_PDN_ ACK	PCIE_PGC_PDN_ACK	MIPI_PGC_PDN_ACK	Reserved	MF_PGC_PDN_ACK
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_ACK_SEL_M4_PU field descriptions

Field	Description
31 PCIE2_PGC_PU_P_ACK	Select power down acknowledge signal of PCIE2 PGC as the power up acknowledge for M4 LPM.

Table continues on the next page...

GPC_ACK_SEL_M4_PU field descriptions (continued)

Field	Description
30 MIPI_CSI2_PGC_PUP_ACK	Select power down acknowledge signal of MIPI_CSI2 PGC as the power up acknowledge for M4 LPM.
29 MIPI_CSI1_PGC_PUP_ACK	Select power down acknowledge signal of MIPI_CSI1 PGC as the power up acknowledge for M4 LPM.
28 DISP_PGC_PUP_ACK	Select power down acknowledge signal of DISP PGC as the power up acknowledge for M4 LPM.
27 HDMI_PGC_PUP_ACK	Select power down acknowledge signal of HDMI PGC as the power up acknowledge for M4 LPM.
26 VPU_PGC_PUP_ACK	Select power down acknowledge signal of VPU PGC as the power up acknowledge for M4 LPM.
25 GPU_PGC_PUP_ACK	Select power down acknowledge signal of GPU PGC as the power up acknowledge for M4 LPM.
24 DDR2_PGC_PUP_ACK	Select power down acknowledge signal of DDR2 PGC as the power up acknowledge for M4 LPM.
23 DDR1_PGC_PUP_ACK	Select power down acknowledge signal of DDR1 PGC as the power up acknowledge for M4 LPM.
22 -	This field is reserved.
21 USB_OTG2_PGC_PUP_ACK	Select power down acknowledge signal of USB_OTG2 PGC as the power up acknowledge for M4 LPM.
20 USB_OTG1_PGC_PUP_ACK	Select power down acknowledge signal of USB_OTG1 PGC as the power up acknowledge for M4 LPM.
19 PCIE_PGC_PUP_ACK	Select power down acknowledge signal of PCIE PGC as the power up acknowledge for M4 LPM.
18 MIPI_PGC_PUP_ACK	Select power down acknowledge signal of MIPI PGC as the power up acknowledge for M4 LPM.
17 -	This field is reserved.
16 MF_PGC_PUP_ACK	Select power down acknowledge signal of MIX PGC as the power up acknowledge for M4 LPM.
15 PCIE2_PGC_PDN_ACK	Select power down acknowledge signal of PCIE2 PGC as the power down acknowledge for M4 LPM.
14 MIPI_CSI2_PGC_PDN_ACK	Select power down acknowledge signal of MIPI_CSI2 PGC as the power down acknowledge for M4 LPM.

Table continues on the next page...

GPC_ACK_SEL_M4_PU field descriptions (continued)

Field	Description
13 MIPI_CSI1_PGC_PDN_ACK	Select power down acknowledge signal of MIPI_CSI1 PGC as the power down acknowledge for M4 LPM.
12 DISP_PGC_PDN_ACK	Select power down acknowledge signal of DISP PGC as the power down acknowledge for M4 LPM.
11 HDMI_PGC_PDN_ACK	Select power down acknowledge signal of HDMI PGC as the power down acknowledge for M4 LPM.
10 VPU_PGC_PDN_ACK	Select power down acknowledge signal of VPU PGC as the power down acknowledge for M4 LPM.
9 GPU_PGC_PDN_ACK	Select power down acknowledge signal of GPU PGC as the power down acknowledge for M4 LPM.
8 DDR2_PGC_PDN_ACK	Select power down acknowledge signal of DDR2 PGC as the power down acknowledge for M4 LPM.
7 DDR1_PGC_PDN_ACK	Select power down acknowledge signal of DDR1 PGC as the power down acknowledge for M4 LPM.
6 -	This field is reserved.
5 USB_OTG2_PGC_PDN_ACK	Select power down acknowledge signal of USB_OTG2 PGC as the power down acknowledge for M4 LPM.
4 USB_OTG1_PGC_PDN_ACK	Select power down acknowledge signal of USB_OTG1 PGC as the power down acknowledge for M4 LPM.
3 PCIE_PGC_PDN_ACK	Select power down acknowledge signal of PCIE PGC as the power down acknowledge for M4 LPM.
2 MIPI_PGC_PDN_ACK	Select power down acknowledge signal of MIPI PGC as the power down acknowledge for M4 LPM.
1 -	This field is reserved.
0 MF_PGC_PDN_ACK	Select power down acknowledge signal of MIX PGC as the power down acknowledge for M4 LPM.

5.2.10.63 Slot configure register for PGC CPUs (GPC_SLTn_CFG)

There are 20 slots in each SLTn_CFG(n = 0~19) that define the power up or power down behavior of one or more A53 cores, NOC, or SCU PGC in each slot. This array contains slots 15 to 19, see Memory Map for slots 0 to 14.

General Power Controller (GPC)

In each “SLTn_cfg”, 2 bits (slt_cfg[1:0])are reserved for each PGC:

- 2'b01 (slot controller will power down relevant PGC in corresponding slot if hardware power down request asserted)
- 2'b10 (slot controller will power up relevant PGC in corresponding slot if hardware power up request asserted)
- 2'b00 or 2'b11 (not power down or power up behavior in relevant slot)

The specific bits assignment for each PGC is shown in the table below.

	PGCx	PGCx-1	..	PGC2	PGC1	PGC0
SLT0_CFG	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]
SLT1_CFG	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]
:	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]
SLTn_CFG	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]	slt_cfg[1:0]

Address: 303A_0000h base + 1E8h offset + (4d × i), where i=0d to 4d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved																
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved							SCU_PUP_SLOT_CONTROL	SCU_PDN_SLOT_CONTROL	CORE3_A53_PUP_SLOT_CONTROL	CORE3_A53_PDN_SLOT_CONTROL	CORE2_A53_PUP_SLOT_CONTROL	CORE2_A53_PDN_SLOT_CONTROL	CORE1_A53_PUP_SLOT_CONTROL	CORE1_A53_PDN_SLOT_CONTROL	CORE0_A53_PUP_SLOT_CONTROL	CORE0_A53_PDN_SLOT_CONTROL
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPC_SLTn_CFG field descriptions

Field	Description
31–10 -	This field is reserved.
9 SCU_PUP_SLOT_CONTROL	SCU Power-up slot control

Table continues on the next page...

GPC_SLTn_CFG field descriptions (continued)

Field	Description
8 SCU_PDN_ SLOT_ CONTROL	SCU Power-down slot control
7 CORE3_A53_ PUP_SLOT_ CONTROL	CORE3 A53 Power-up slot control
6 CORE3_A53_ PDN_SLOT_ CONTROL	CORE3 A53 Power-down slot control
5 CORE2_A53_ PUP_SLOT_ CONTROL	CORE2 A53 Power-up slot control
4 CORE2_A53_ PDN_SLOT_ CONTROL	CORE2 A53 Power-down slot control
3 CORE1_A53_ PUP_SLOT_ CONTROL	CORE1 A53 Power-up slot control
2 CORE1_A53_ PDN_SLOT_ CONTROL	CORE1 A53 Power-down slot control
1 CORE0_A53_ PUP_SLOT_ CONTROL	CORE0 A53 Power-up slot control
0 CORE0_A53_ PDN_SLOT_ CONTROL	CORE0 A53 Power-down slot control

5.2.10.64 Power handshake register (GPC_PU_PWRHSK)

Address: 303A_0000h base + 1FCh offset = 303A_01FCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved					GPC_GPUMIX_ PWRDNACKN	GPC_VPUMIX_ PWRDNACKN	GPC_DISP_ PWRDNACKN	GPC_DDR2_AXI_ CACTIVE	GPC_DDR2_AXI_ CSYSACK	GPC_DDR2_CORE_ CACTIVE	GPC_DDR2_CORE_ CSYSACK	GPC_DDR1_AXI_ CACTIVE	GPC_DDR1_AXI_ CSYSACK	GPC_DDR1_CORE_ CACTIVE	GPC_DDR1_CORE_ CSYSACK
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved									GPC_GPUMIX_ PWRDNREQN	GPC_VPUMIX_ PWRDNREQN	GPC_DISPMIX_ PWRDNREQN	GPC_DDR2_AXI_ CSYSREQ	GPC_DDR2_CORE_ CSYSREQ	GPC_DDR1_AXI_ CSYSREQ	GPC_DDR1_CORE_ CSYSREQ
W																
Reset	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

GPC_PU_PWRHSK field descriptions

Field	Description
31–27 -	This field is reserved.
26 GPC_GPUMIX_ PWRDNACKN	GPU ADB400 power down ack. Active 0
25 GPC_VPUMIX_ PWRDNACKN	VPU ADB400 power down ack. Active 0
24 GPC_DISP_ PWRDNACKN	DISP ADB400 power down ack. Active 0
23 GPC_DDR2_ AXI_CACTIVE	DDR2 AXI Clock Active
22 GPC_DDR2_ AXI_CSYSACK	DDR2 AXI Low-Power Request ack

Table continues on the next page...

GPC_PU_PWRHSK field descriptions (continued)

Field	Description
21 GPC_DDR2_ CORE_CAACTIVE	DDR2 controller Hardware Low-Power Clock active
20 GPC_DDR2_ CORE_ CSYSACK	DDR2 controller Hardware Low_Power ack
19 GPC_DDR1_ AXI_CAACTIVE	DDR1 AXI Clock Active
18 GPC_DDR1_ AXI_CSYSACK	DDR1 AXI Low-Power Request ack
17 GPC_DDR1_ CORE_CAACTIVE	DDR1 controller Hardware Low-Power Clock active
16 GPC_DDR1_ CORE_ CSYSACK	DDR1 controller Hardware Low_Power ack
15–7 -	This field is reserved.
6 GPC_GPUMIX_ PWRDNREQN	GPU ADB400 power down request. Active 0
5 GPC_VPUMIX_ PWRDNREQN	VPU ADB400 power down request. Active 0
4 GPC_DISPMIX_ PWRDNREQN	DISPMIX ADB400 power down request. Active 0
3 GPC_DDR2_ AXI_CSYSREQ	DDR2 AXI Low-Power Request
2 GPC_DDR2_ CORE_ CSYSREQ	DDR2 controller Hardware Low-Power Request
1 GPC_DDR1_ AXI_CSYSREQ	DDR1 AXI Low-Power Request
0 GPC_DDR1_ CORE_ CSYSREQ	DDR1 controller Hardware Low-Power Request

5.2.10.65 Slot configure register for PGC PUs (GPC_SLTn_CFG_PU)

There are 20 slots in each SLTn_CFG_PU (n = 0~19) that define the power up or power down behavior of PU PGC in each slot. See PGC power domains section for list of PGC PUs.

Address: 303A_0000h base + 200h offset + (8d × i), where i=0d to 19d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W	PCIE2_PUP_SLOT_CO NTROL	PCIE2_PDN_SLOT_CO NTROL	MIPI_CSI2_PUP_SLOT _CONTROL	MIPI_CSI2_PDN_SLOT _CONTROL	MIPI_CSI1_PUP_SLOT _CONTROL	MIPI_CSI1_PDN_SLOT _CONTROL	DISP_PUP_SLOT_ CONTROL	DISP_PDN_SLOT_ CONTROL	HDMI_PUP_SLOT_CO NTROL	HDMI_PDN_SLOT_CO NTROL	VPU_PUP_SLOT_ CONTROL	VPU_PDN_SLOT_ CONTROL	GPU_PUP_SLOT_ CONTROL	GPU_PDN_SLOT_ CONTROL	DDR2_PUP_SLOT_CO NTROL	DDR2_PDN_SLOT_CO NTROL
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W	DDR1_PUP_SLOT_ CONTROL	DDR1_PDN_SLOT_ CONTROL	M4_PUP_SLOT_CONT ROL	M4_PDN_SLOT_CONT ROL	Reserved		OTG2_PUP_SLOT_ CONTROL	OTG2_PDN_SLOT_ CONTROL	OTG1_PUP_SLOT_ CONTROL	OTG1_PDN_SLOT_ CONTROL	PCIE_PUP_SLOT_ CONTROL	PCIE_PDN_SLOT_ CONTROL	MIPI_PUP_SLOT_ CONTROL	MIPI_PDN_SLOT_ CONTROL	MF_PUP_SLOT_ CONTROL	MF_PDN_SLOT_ CONTROL
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPC_SLTn_CFG_PU field descriptions

Field	Description
31 PCIE2_PUP_SLOT_CONTROL	PCIE2 Power-up slot control
30 PCIE2_PDN_SLOT_CONTROL	PCIE2 Power-down slot control
29 MIPI_CSI2_PUP_SLOT_CONTROL	MIPI_CSI2 Power-up slot control
28 MIPI_CSI2_PDN_SLOT_CONTROL	MIPI_CSI2 Power-down slot control

Table continues on the next page...

GPC_SLTn_CFG_PU field descriptions (continued)

Field	Description
27 MIPI_CSI1_PUP_ _SLOT_CONTR OL	MIPI_CSI1 Power-up slot control
26 MIPI_CSI1_PDN_ _SLOT_CONTR OL	MIPI_CSI1 Power-down slot control
25 DISP_PUP_ SLOT_ CONTROL	DISP Power-up slot control
24 DISP_PDN_ SLOT_ CONTROL	DISP Power-down slot control
23 HDMI_PUP_SLO T_CONTROL	HDMI Power-up slot control
22 HDMI_PDN_SLO T_CONTROL	HDMI Power-down slot control
21 VPU_PUP_ SLOT_ CONTROL	VPU Power-up slot control
20 VPU_PDN_ SLOT_ CONTROL	VPU Power-down slot control
19 GPU_PUP_ SLOT_ CONTROL	GPU Power-up slot control
18 GPU_PDN_ SLOT_ CONTROL	GPU Power-down slot control
17 DDR2_PUP_SLO T_CONTROL	DDR2 Power-up slot control
16 DDR2_PDN_SLO T_CONTROL	DDR2 Power-down slot control
15 DDR1_PUP_ SLOT_ CONTROL	DDR1 Power-up slot control

Table continues on the next page...

GPC_SLT_n_CFG_PU field descriptions (continued)

Field	Description
14 DDR1_PDN_ SLOT_ CONTROL	DDR1 Power-down slot control
13 M4_PUP_SLOT_ CONTROL	M4 Power-up slot control
12 M4_PDN_SLOT_ CONTROL	M4 Power-down slot control
11–10 -	This field is reserved.
9 OTG2_PUP_ SLOT_ CONTROL	OTG2 Power-up slot control
8 OTG2_PDN_ SLOT_ CONTROL	OTG2 Power-down slot control
7 OTG1_PUP_ SLOT_ CONTROL	OTG1 Power-up slot control
6 OTG1_PDN_ SLOT_ CONTROL	OTG1 Power-down slot control
5 PCIE_PUP_ SLOT_ CONTROL	PCIE Power-up slot control
4 PCIE_PDN_ SLOT_ CONTROL	SCU Power-down slot control
3 MIPI_PUP_ SLOT_ CONTROL	MIPI Power-up slot control
2 MIPI_PDN_ SLOT_ CONTROL	MIPI Power-down slot control
1 MF_PUP_SLOT_ CONTROL	MF Power-up slot control
0 MF_PDN_SLOT_ CONTROL	MF Power-down slot control

5.2.11 GPC PGC Memory Map/Register Definition

There are numerous PGC inside GPCv2, with 4 different types: CPU/SCU/MIX/PU. Each PGC type has 4 different control words PGC_CTRL, PGC_PUPSCR, PGC_PDNSCR, and PGC_SR. Different PGC types may have different field definition in these four registers. There is another extra control word PGC_AUXSW which has different field definition for PCIE/MIPI PGC and SCU type PGC.

The total GPC memory map is 4KB

Table 5-11. Memory Regions

Address Range(offset)	Region
0x000 - 0x3FF	GPC configuration register
0x400 - 0x7FF	Reserved
0x800 - 0x9FF	CPU and SCU type PGC register base address
0xA00 - 0xBFF	MIX type PGC register base address
0xC00 - 0xFFFF	PU type PGC register base address

Each PGC (CPU type, MIX type, PU type) will occupy 64 Bytes address space, the specific base address of each PGC are listed as below.

- 0x800 ~ 0x83F: PGC for A53 core0
- 0x840 ~ 0x87F: PGC for A53 core1
- 0x880 ~ 0x8BF: PGC for A53 core2
- 0x8C0 ~ 0x8FF: PGC for A53 core3
- 0x900 ~ 0x93F: PGC for A53 SCU
- 0xC00 ~ 0xC3F: PGC for MIPI DSI PHY (PU0)
- 0xC40 ~ 0xC7F: PGC for PCIE1 PHY (PU1)
- 0xC80 ~ 0xCBF: USB_OTG1 (PU2)
- 0xCC0 ~ 0xCFF: USB_OTG2 (PU3)
- 0xD00 ~ 0xD3F: Reserved (PU4)
- 0xD40 ~ 0xD7F: DDR1 (PU5)
- 0xD80 ~ 0xDBF: Reserved (PU6)
- 0xDC0 ~ 0xDFF: GPUMIX (PU7)
- 0xE00 ~ 0xE3F: VPUMIX (PU8)
- 0xE40 ~ 0xE7F: HDMI (PU9)
- 0xE80 ~ 0xEBF: DISPMIX (PU10)
- 0xEC0 ~ 0xEFF: MIPI CSI1 PHY (PU11)

General Power Controller (GPC)

- 0xF00 ~ 0xF3F: MIPI CSI2 PHY (PU12)
- 0xF40 ~ 0xF7F: PCIE2 PHY (PU13)

GPC_PGC memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303A_0800	GPC PGC Control Register for PGC CPUs (GPC_PGC_A53CORE0_CTRL)	32	R/W	0604_0202h	5.2.11.1/ 771
303A_0804	GPC PGC Up Sequence Control Register (GPC_PGC_A53CORE0_PUPSCR)	32	R/W	0009_97C1h	5.2.11.2/ 772
303A_0808	GPC PGC Down Sequence Control Register (GPC_PGC_A53CORE0_PDNSCR)	32	R/W	2100_0801h	5.2.11.3/ 773
303A_080C	GPC PGC Status Register (GPC_PGC_A53CORE0_SR)	32	R/W	0000_1000h	5.2.11.4/ 775
303A_0810	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_A53CORE0_AUXSW)	32	R/W	0000_0131h	5.2.11.5/ 778
303A_0840	GPC PGC Control Register for PGC CPUs (GPC_PGC_A53CORE1_CTRL)	32	R/W	0604_0202h	5.2.11.1/ 771
303A_0844	GPC PGC Up Sequence Control Register (GPC_PGC_A53CORE1_PUPSCR)	32	R/W	0009_97C1h	5.2.11.2/ 772
303A_0848	GPC PGC Down Sequence Control Register (GPC_PGC_A53CORE1_PDNSCR)	32	R/W	2100_0801h	5.2.11.3/ 773
303A_084C	GPC PGC Status Register (GPC_PGC_A53CORE1_SR)	32	R/W	0000_1000h	5.2.11.4/ 775
303A_0850	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_A53CORE1_AUXSW)	32	R/W	0000_0131h	5.2.11.5/ 778
303A_0880	GPC PGC Control Register for PGC CPUs (GPC_PGC_A53CORE2_CTRL)	32	R/W	0604_0202h	5.2.11.1/ 771
303A_0884	GPC PGC Up Sequence Control Register (GPC_PGC_A53CORE2_PUPSCR)	32	R/W	0009_97C1h	5.2.11.2/ 772
303A_0888	GPC PGC Down Sequence Control Register (GPC_PGC_A53CORE2_PDNSCR)	32	R/W	2100_0801h	5.2.11.3/ 773
303A_088C	GPC PGC Status Register (GPC_PGC_A53CORE2_SR)	32	R/W	0000_1000h	5.2.11.4/ 775
303A_0890	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_A53CORE2_AUXSW)	32	R/W	0000_0131h	5.2.11.5/ 778
303A_08C0	GPC PGC Control Register for PGC CPUs (GPC_PGC_A53CORE3_CTRL)	32	R/W	0604_0202h	5.2.11.1/ 771
303A_08C4	GPC PGC Up Sequence Control Register (GPC_PGC_A53CORE3_PUPSCR)	32	R/W	0009_97C1h	5.2.11.2/ 772
303A_08C8	GPC PGC Down Sequence Control Register (GPC_PGC_A53CORE3_PDNSCR)	32	R/W	2100_0801h	5.2.11.3/ 773
303A_08CC	GPC PGC Status Register (GPC_PGC_A53CORE3_SR)	32	R/W	0000_1000h	5.2.11.4/ 775
303A_08D0	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_A53CORE3_AUXSW)	32	R/W	0000_0131h	5.2.11.5/ 778

Table continues on the next page...

GPC_PGC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
303A_0900	GPC PGC Control Register for PGC CPUs (GPC_PGC_A53SCU_CTRL)	32	R/W	0604_0202h	5.2.11.1/771
303A_0904	GPC PGC Up Sequence Control Register (GPC_PGC_A53SCU_PUPSCR)	32	R/W	0009_97C1h	5.2.11.2/772
303A_0908	GPC PGC Down Sequence Control Register (GPC_PGC_A53SCU_PDNSCR)	32	R/W	2100_0801h	5.2.11.3/773
303A_090C	GPC PGC Status Register (GPC_PGC_A53SCU_SR)	32	R/W	0000_1000h	5.2.11.4/775
303A_0910	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_A53SCU_AUXSW)	32	R/W	0000_0131h	5.2.11.5/778
303A_0A10	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_MIX_AUXSW)	32	R/W	0000_0131h	5.2.11.6/779
303A_0A40	GPC PGC Control Register for PGC MIX. (GPC_PGC_NOC_MIX_CTRL)	32	R/W	0604_0202h	5.2.11.7/781
303A_0A44	GPC PGC Up Sequence Control Register (GPC_PGC_NOC_MIX_PUPSCR)	32	R/W	0009_97C1h	5.2.11.8/782
303A_0A48	GPC PGC Down Sequence Control Register (GPC_PGC_NOC_MIX_PDNSCR)	32	R/W	2100_0801h	5.2.11.9/783
303A_0A4C	GPC PGC Status Register (GPC_PGC_NOC_MIX_SR)	32	R/W	0000_1000h	5.2.11.10/785
303A_0C00	GPC PGC Control Register for PGC PUs (GPC_PGC_PU0_CTRL)	32	R/W	0604_0202h	5.2.11.11/788
303A_0C04	GPC PGC Up Sequence Control Register (GPC_PGC_PU0_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789
303A_0C08	GPC PGC Down Sequence Control Register (GPC_PGC_PU0_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0C0C	GPC PGC Status Register (GPC_PGC_PU0_SR)	32	R/W	0000_1000h	5.2.11.14/792
303A_0C10	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU0_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795
303A_0C40	GPC PGC Control Register for PGC PUs (GPC_PGC_PU1_CTRL)	32	R/W	0604_0202h	5.2.11.11/788
303A_0C44	GPC PGC Up Sequence Control Register (GPC_PGC_PU1_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789
303A_0C48	GPC PGC Down Sequence Control Register (GPC_PGC_PU1_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0C4C	GPC PGC Status Register (GPC_PGC_PU1_SR)	32	R/W	0000_1000h	5.2.11.14/792
303A_0C50	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU1_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795
303A_0C80	GPC PGC Control Register for PGC PUs (GPC_PGC_PU2_CTRL)	32	R/W	0604_0202h	5.2.11.11/788
303A_0C84	GPC PGC Up Sequence Control Register (GPC_PGC_PU2_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789

Table continues on the next page...

GPC_PGC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303A_0C88	GPC PGC Down Sequence Control Register (GPC_PGC_PU2_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0C8C	GPC PGC Status Register (GPC_PGC_PU2_SR)	32	R/W	0000_1000h	5.2.11.14/792
303A_0C90	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU2_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795
303A_0CC0	GPC PGC Control Register for PGC PUs (GPC_PGC_PU3_CTRL)	32	R/W	0604_0202h	5.2.11.11/788
303A_0CC4	GPC PGC Up Sequence Control Register (GPC_PGC_PU3_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789
303A_0CC8	GPC PGC Down Sequence Control Register (GPC_PGC_PU3_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0CCC	GPC PGC Status Register (GPC_PGC_PU3_SR)	32	R/W	0000_1000h	5.2.11.14/792
303A_0CD0	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU3_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795
303A_0D00	GPC PGC Control Register for PGC PUs (GPC_PGC_PU4_CTRL)	32	R/W	0604_0202h	5.2.11.11/788
303A_0D04	GPC PGC Up Sequence Control Register (GPC_PGC_PU4_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789
303A_0D08	GPC PGC Down Sequence Control Register (GPC_PGC_PU4_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0D0C	GPC PGC Status Register (GPC_PGC_PU4_SR)	32	R/W	0000_1000h	5.2.11.14/792
303A_0D10	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU4_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795
303A_0D40	GPC PGC Control Register for PGC PUs (GPC_PGC_PU5_CTRL)	32	R/W	0604_0202h	5.2.11.11/788
303A_0D44	GPC PGC Up Sequence Control Register (GPC_PGC_PU5_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789
303A_0D48	GPC PGC Down Sequence Control Register (GPC_PGC_PU5_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0D4C	GPC PGC Status Register (GPC_PGC_PU5_SR)	32	R/W	0000_1000h	5.2.11.14/792
303A_0D50	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU5_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795
303A_0D80	GPC PGC Control Register for PGC PUs (GPC_PGC_PU6_CTRL)	32	R/W	0604_0202h	5.2.11.11/788
303A_0D84	GPC PGC Up Sequence Control Register (GPC_PGC_PU6_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789
303A_0D88	GPC PGC Down Sequence Control Register (GPC_PGC_PU6_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0D8C	GPC PGC Status Register (GPC_PGC_PU6_SR)	32	R/W	0000_1000h	5.2.11.14/792

Table continues on the next page...

GPC_PGC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
303A_0D90	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU6_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795
303A_0DC0	GPC PGC Control Register for PGC PUs (GPC_PGC_PU7_CTRL)	32	R/W	0604_0202h	5.2.11.11/788
303A_0DC4	GPC PGC Up Sequence Control Register (GPC_PGC_PU7_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789
303A_0DC8	GPC PGC Down Sequence Control Register (GPC_PGC_PU7_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0DCC	GPC PGC Status Register (GPC_PGC_PU7_SR)	32	R/W	0000_1000h	5.2.11.14/792
303A_0DD0	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU7_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795
303A_0E00	GPC PGC Control Register for PGC PUs (GPC_PGC_PU8_CTRL)	32	R/W	0604_0202h	5.2.11.11/788
303A_0E04	GPC PGC Up Sequence Control Register (GPC_PGC_PU8_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789
303A_0E08	GPC PGC Down Sequence Control Register (GPC_PGC_PU8_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0E0C	GPC PGC Status Register (GPC_PGC_PU8_SR)	32	R/W	0000_1000h	5.2.11.14/792
303A_0E10	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU8_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795
303A_0E40	GPC PGC Control Register for PGC PUs (GPC_PGC_PU9_CTRL)	32	R/W	0604_0202h	5.2.11.11/788
303A_0E44	GPC PGC Up Sequence Control Register (GPC_PGC_PU9_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789
303A_0E48	GPC PGC Down Sequence Control Register (GPC_PGC_PU9_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0E4C	GPC PGC Status Register (GPC_PGC_PU9_SR)	32	R/W	0000_1000h	5.2.11.14/792
303A_0E50	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU9_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795
303A_0E80	GPC PGC Control Register for PGC PUs (GPC_PGC_PU10_CTRL)	32	R/W	0604_0202h	5.2.11.11/788
303A_0E84	GPC PGC Up Sequence Control Register (GPC_PGC_PU10_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789
303A_0E88	GPC PGC Down Sequence Control Register (GPC_PGC_PU10_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0E8C	GPC PGC Status Register (GPC_PGC_PU10_SR)	32	R/W	0000_1000h	5.2.11.14/792
303A_0E90	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU10_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795
303A_0EC0	GPC PGC Control Register for PGC PUs (GPC_PGC_PU11_CTRL)	32	R/W	0604_0202h	5.2.11.11/788

Table continues on the next page...

GPC_PGC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
303A_0EC4	GPC PGC Up Sequence Control Register (GPC_PGC_PU11_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789
303A_0EC8	GPC PGC Down Sequence Control Register (GPC_PGC_PU11_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0ECC	GPC PGC Status Register (GPC_PGC_PU11_SR)	32	R/W	0000_1000h	5.2.11.14/792
303A_0ED0	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU11_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795
303A_0F00	GPC PGC Control Register for PGC PUs (GPC_PGC_PU12_CTRL)	32	R/W	0604_0202h	5.2.11.11/788
303A_0F04	GPC PGC Up Sequence Control Register (GPC_PGC_PU12_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789
303A_0F08	GPC PGC Down Sequence Control Register (GPC_PGC_PU12_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0F0C	GPC PGC Status Register (GPC_PGC_PU12_SR)	32	R/W	0000_1000h	5.2.11.14/792
303A_0F10	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU12_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795
303A_0F40	GPC PGC Control Register for PGC PUs (GPC_PGC_PU13_CTRL)	32	R/W	0604_0202h	5.2.11.11/788
303A_0F44	GPC PGC Up Sequence Control Register (GPC_PGC_PU13_PUPSCR)	32	R/W	0009_97C1h	5.2.11.12/789
303A_0F48	GPC PGC Down Sequence Control Register (GPC_PGC_PU13_PDNSCR)	32	R/W	2100_0801h	5.2.11.13/790
303A_0F4C	GPC PGC Status Register (GPC_PGC_PU13_SR)	32	R/W	0000_1000h	5.2.11.14/792
303A_0F50	GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_PU13_AUXSW)	32	R/W	0000_0131h	5.2.11.15/795

5.2.11.1 GPC PGC Control Register for PGC CPUs (GPC_PGC_nCTRL)

GPC PGC Control Register for the PGC CPUs. See the PGC Memory Map for the assignments.

Address: 303A_0000h base + 800h offset + (64d × i), where i=0d to 4d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16								
R	Reserved				MEMPWR_TCD1_TDR_TRM								Reserved				L2RETN_TCD1_TDR							
W																								
Reset	0	0	0	0	0	1	1	0	0	0	0	0	0	1	0	0								

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0										
R	Reserved				DFTRAM_TCD1								Reserved				L2RSTDIS								PCR	
W																										
Reset	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0										

GPC_PGC_nCTRL field descriptions

Field	Description
31–30 -	This field is reserved. Reserved
29–24 MEMPWR_ TCD1_TDR_ TRM	After scu pdn_req, count this value to assert A53 mempwr to 1'b1 NOTE: Can't be programmed to zero (This register control only for SCU Type PGC)
23–22 -	This field is reserved. Reserved
21–16 L2RETN_TCD1_ TDR	After scu pdn_req, count this value to assert A53 l2retn to 1'b0 NOTE: Can't be programmed to zero (This register control only for SCU Type PGC)
15–14 -	This field is reserved. Reserved

Table continues on the next page...

GPC_PGC_nCTRL field descriptions (continued)

Field	Description
13–8 DFTRAM_TCD1	After scu pdn_req, count this value to assert A53 dftram to 1'b1 NOTE: Can't be programmed to zero (This register control only for SCU Type PGC)
7 -	This field is reserved. Reserved
6–1 L2RSTDIS	After scu pdn_req, count this value to assert A53 l2rstdis to 1'b1, it will be clear automatically once any of A53 core0/core1/core2/core3 is wakeup NOTE: Can't be programmed to zero (This register control only for SCU Type PGC)
0 PCR	Power Control NOTE: PCR must not change from power-down request (pdn_req) assertion until the target subsystem is completely powered up. 0 Do not switch off power even if pdn_req is asserted. 1 Switch off power when pdn_req is asserted.

5.2.11.2 GPC PGC Up Sequence Control Register (GPC_PGC_nPUPSCR)

GPC PGC Up Sequence Control Register

Address: 303A_0000h base + 804h offset + (64d × i), where i=0d to 4d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PUP_SCALL_SCALLOUT_CNT								SW2ISO							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	SW2ISO								PUP_WAIT_SCALLOUT		SW					
W																
Reset	1	0	0	1	0	1	1	1	1	1	0	0	0	0	0	1

GPC_PGC_nPUPSCR field descriptions

Field	Description
31–23 PUP_SCALL_ SCALLOUT_CNT	After SCALL asserting to 1'b0, count this value to complete switch power up NOTE: Only valid when pup_wait_scall_out is set to 1'b0. Can't be programmed to zero (This register control only for MIX Type PGC)
22–7 SW2ISO	After asserting switch_b, the PGC waits a number of clocks equal to the value of SW2ISO before negating isolation.
6 PUP_WAIT_ SCALL_OUT	After SCALL asserting to 1'b0, wait handshake signal SCALL_OUT to return to 1'b0 (This register control only for MIX Type PGC)
SW	After a power-up request (pup_req assertion), the PGC waits a number of clocks equal to the value of SW before asserting switch_b NOTE: SW must not be programmed to zero.

5.2.11.3 GPC PGC Down Sequence Control Register (GPC_PGC_nPDNSCR)

GPC PGC Down Sequence Control Register

Address: 303A_0000h base + 808h offset + (64d × i), where i=0d to 4d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PUP_SCPRE_SCALL_CNT								PDN_SCALL_SCALLOUT_CNT							
W																
Reset	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				ISO2SW				PDN_WAIT_SCALL_OUT	Reserved		ISO				
W																
Reset	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1

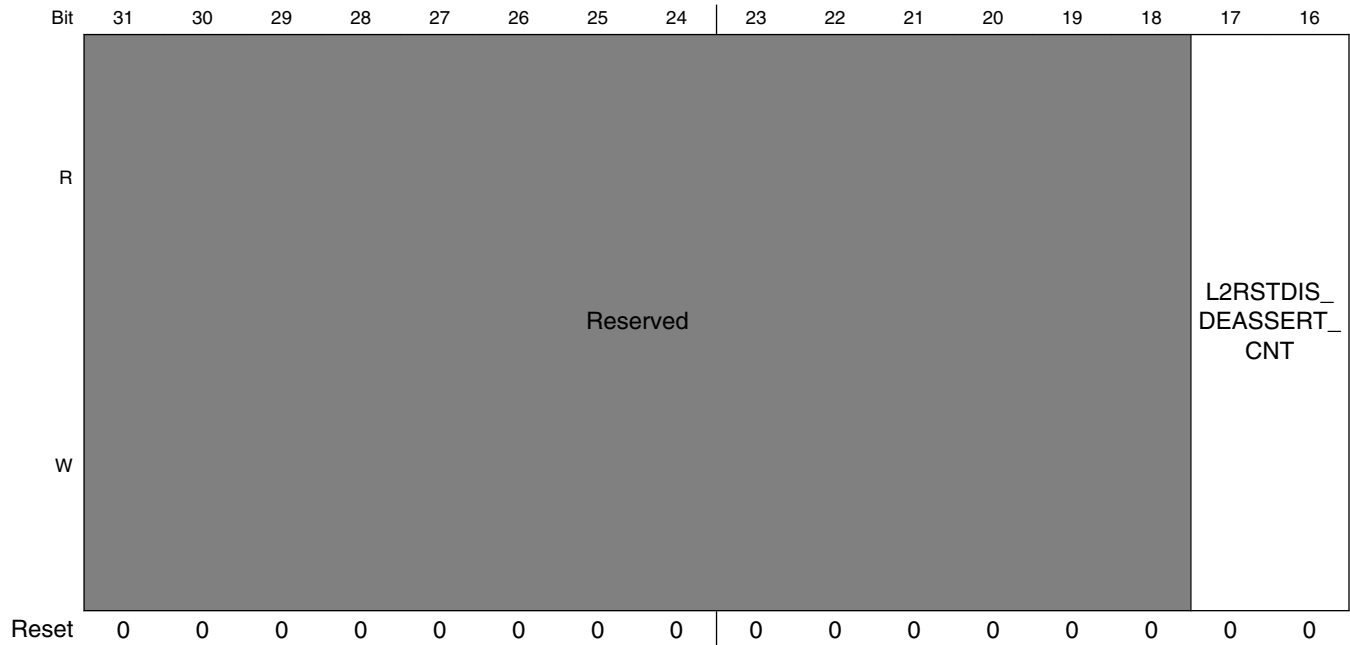
GPC_PGC_nPDNSCR field descriptions

Field	Description
31–24 PUP_SCPRE_ SCALL_CNT	After SCPRE asserting to 1'b0, count this value to assert SCALL to 1'b0 (This register control only for MIX Type PGC)
23–16 PDN_SCALL_ SCALLOUT_CNT	Default 8'h0. After SCALL asserting to 1'b1, count this value to complete switch power down. NOTE: This control is only valid when pdn_wait_scall_out is set to 1'b0. It can be programmed to zero. (This register control only for MIX Type PGC)
15–14 -	This field is reserved. Reserved
13–8 ISO2SW	After asserting isolation(by pdn_req assertion), the PGC waits a number of clocks equal to the value of ISO2SW before negating switch_b NOTE: ISO2SW must not be programmed to zero.
7 PDN_WAIT_ SCALL_OUT	Default 1'b0 If set to 1'b1, after SCALL asserting to 1'b1, wait handshake signal SCALL_OUT to change to 1'b1. NOTE: This register control is only for MIX Type PGC.
6 -	This field is reserved. Reserved
ISO	After a power-down request (pdn_req assertion), the PGC waits a number of clocks equal to the value of ISO before asserting isolation NOTE: ISO must not be programmed to zero.

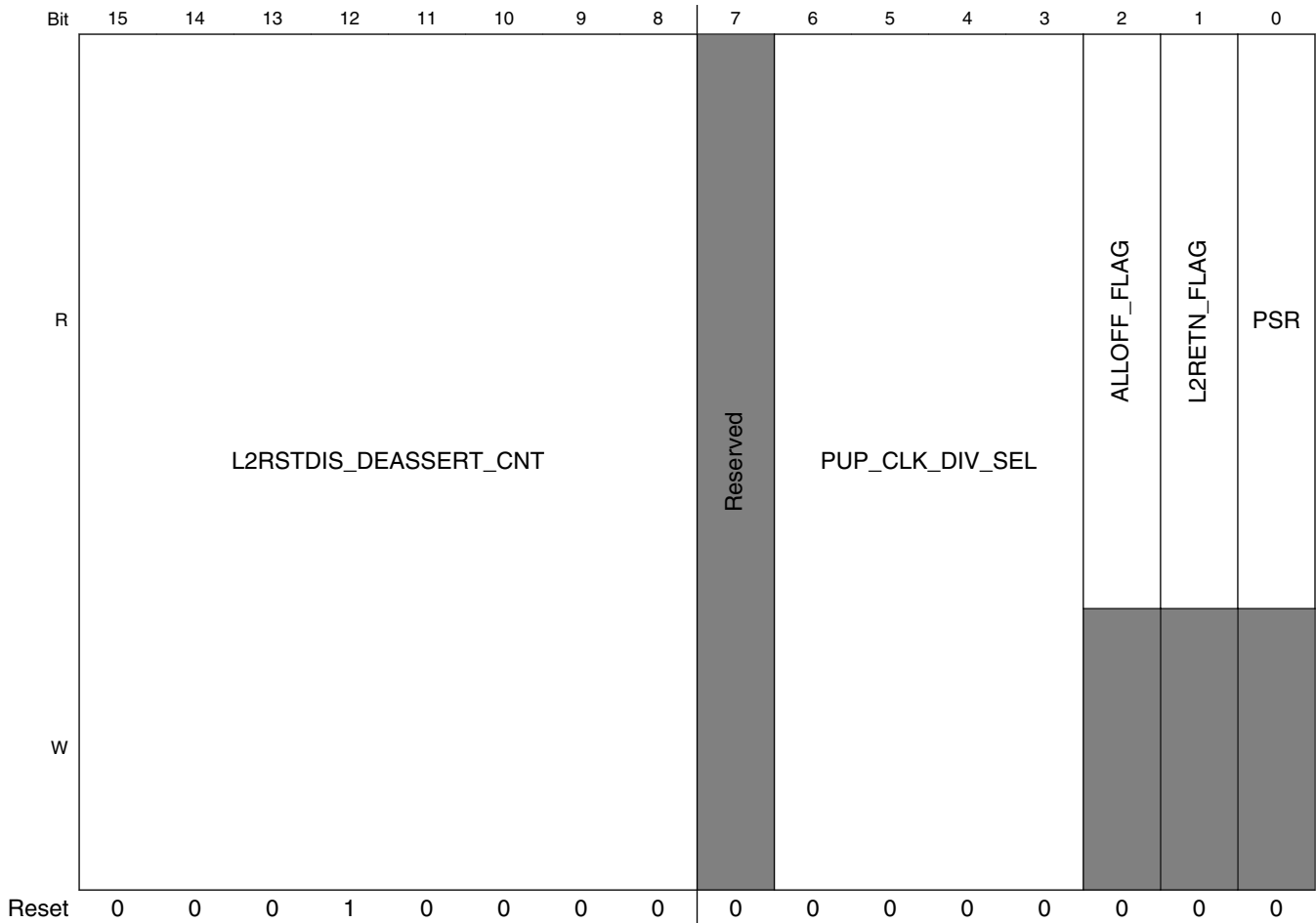
5.2.11.4 GPC PGC Status Register (GPC_PGC_nSR)

GPC PGC Status Register

Address: 303A_0000h base + 80Ch offset + (64d × i), where i=0d to 4d



General Power Controller (GPC)



GPC_PGC_nSR field descriptions

Field	Description
31-18 -	This field is reserved. Reserved
17-8 L2RSTDIS_ DEASSERT_ CNT	Count this value to de-assert L2RSTDISABLE to LOW after CPU0 or CPU1 power up NOTE: This value can't be programmed to zero (This register control only for SCU Type PGC)
7 -	This field is reserved. Reserved
6-3 PUP_CLK_DIV_ SEL	Clock divider select for the clock of power up counter(count_clk is 32KHz for CPU/SCU type PGC, ipg_clk(66MHz) for MIX/PU Type PGC) 0000 1 0001 1/2 count_clk 0010 1/4 count_clk 0011 1/8 count_clk 0100 1/16 count_clk 0101 1/32 count_clk 0110 1/64 count_clk 0111 1/128 count_clk

Table continues on the next page...

GPC_PGC_nSR field descriptions (continued)

Field	Description
	1000 1/256 count_clk 1001 1/512 count_clk 1010 1/1024 count_clk 1011 1/2056 count_clk 1100 1/4096 count_clk 1101 1/8192 count_clk 1110 1/16384 count_clk 1111 1/32768 count_clk
² ALLOFF_FLAG	All-off flag. NOTE: Software should write “1” to clear this flag after A53 is wakeup from ALL_OFF mode, otherwise, it will always keep to 1 (This register control only for SCU Type PGC) 0 A53 is not wakeup from ALL_OFF mode. 1 A53 is wakeup from ALL_OFF mode.
¹ L2RETN_FLAG	L2 Retention Flag NOTE: Software should write “1” to clear this flag after A53 is wakeup from L2 retention mode, otherwise it will always keep to 1 (This register control only for SCU Type PGC) 0 A53 is not wakeup from L2 retention mode. 1 A53 is wakeup from L2 retention mode.
⁰ PSR	Power status. When in functional (or software-controlled debug) mode, PGC hardware sets PSR as soon as any of the power control output changes its state to one. Write one to clear this bit. Software should clear this bit after power up; otherwise, PSR continues to reflect the power status of the initial power down. 0 The target subsystem was not powered down for the previous power-down request. 1 The target subsystem was powered down for the previous power-down request.

5.2.11.5 GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_nAUXSW)

GPC PGC Auxiliary Power Switch Control Register. The register has different field definition for PCIE/MIPI PGC and SCU Type PGC.

Address: 303A_0000h base + 810h offset + (64d × i), where i=0d to 4d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved												PDN_CLK_DIV_SEL			
W	Reserved												PDN_CLK_DIV_SEL			
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		ISO2SW2						Reserved		SW2					
W	Reserved		ISO2SW2						Reserved		SW2					
Reset	0	0	0	0	0	0	0	1	0	0	1	1	0	0	0	1

GPC_PGC_nAUXSW field descriptions

Field	Description
31–20 -	This field is reserved. Reserved
19–16 PDN_CLK_DIV_SEL	Clock divider select for the clock of power down counter 0000 1 0001 1/2 count_clk 0010 1/4 count_clk 0011 1/8 count_clk 0100 1/16 count_clk 0101 1/32 count_clk 0110 1/64 count_clk 0111 1/128 count_clk 1000 1/256 count_clk 1001 1/512 count_clk 1010 1/1024 count_clk 1011 1/2056 count_clk 1100 1/4096 count_clk 1101 1/8192 count_clk 1110 1/16384 count_clk 1111 1/32768 count_clk
15–14 -	This field is reserved. Reserved
13–8 ISO2SW2	after asserting isolation by power-down request(pdn_req assertion), the PGC waits a number of clocks equal to the value of ISO2SW2 before negating switch2_b(1P8 Power)

Table continues on the next page...

GPC_PGC_nAUXSW field descriptions (continued)

Field	Description
	NOTE: ISO2SW2 must not be programmed to zero. 0 A53 is not wakeup from ALL_OFF mode. 1 A53 is wakeup from ALL_OFF mode.
7–6 -	This field is reserved. Reserved
SW2	After a power-up request (pup_req assertion), the PGC waits a number of clocks equal to the value of SW2 before asserting switch2_b(1P8 Power) NOTE: SW2 must not be programmed to zero.

5.2.11.6 GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_MIX_AUXSW)

GPC PGC Auxiliary Power Switch Control Register. The register has different field definition for PCIE/MIPI PGC and SCU Type PGC.

Address: 303A_0000h base + A10h offset = 303A_0A10h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved												PDN_CLK_DIV_SEL			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		ISO2SW2						Reserved		SW2					
W																
Reset	0	0	0	0	0	0	0	1	0	0	1	1	0	0	0	1

GPC_PGC_MIX_AUXSW field descriptions

Field	Description
31–20 -	This field is reserved. Reserved
19–16 PDN_CLK_DIV_SEL	Clock divider select for the clock of power down counter 0000 1 0001 1/2 count_clk 0010 1/4 count_clk 0011 1/8 count_clk 0100 1/16 count_clk 0101 1/32 count_clk

Table continues on the next page...

GPC_PGC_MIX_AUXSW field descriptions (continued)

Field	Description
	0110 1/64 count_clk 0111 1/128 count_clk 1000 1/256 count_clk 1001 1/512 count_clk 1010 1/1024 count_clk 1011 1/2056 count_clk 1100 1/4096 count_clk 1101 1/8192 count_clk 1110 1/16384 count_clk 1111 1/32768 count_clk
15–14 -	This field is reserved. Reserved
13–8 ISO2SW2	after asserting isolation by power-down request(pdn_req assertion), the PGC waits a number of clocks equal to the value of ISO2SW2 before negating switch2_b(1P8 Power) NOTE: ISO2SW2 must not be programmed to zero. 0 A53 is not wakeup from ALL_OFF mode. 1 A53 is wakeup from ALL_OFF mode.
7–6 -	This field is reserved. Reserved
SW2	After a power-up request (pup_req assertion), the PGC waits a number of clocks equal to the value of SW2 before asserting switch2_b(1P8 Power) NOTE: SW2 must not be programmed to zero.

5.2.11.7 GPC PGC Control Register for PGC MIX. (GPC_PGC_NOC_MIX_CTRL)

GPC PGC Control Register.

Address: 303A_0000h base + A40h offset = 303A_0A40h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved		MEMPWR_TCD1_TDR_TRM						Reserved		L2RETN_TCD1_TDR					
W																
Reset	0	0	0	0	0	1	1	0	0	0	0	0	0	1	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		DFTRAM_TCD1						Reserved	L2RSTDIS						MIX_PCR
W																
Reset	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0

GPC_PGC_NOC_MIX_CTRL field descriptions

Field	Description
31–30 -	This field is reserved. Reserved
29–24 MEMPWR_TCD1_TDR_TRM	After scu pdn_req, count this value to assert A53 mempwr to 1'b1 NOTE: Can't be programmed to zero (This register control only for SCU Type PGC)
23–22 -	This field is reserved. Reserved
21–16 L2RETN_TCD1_TDR	After scu pdn_req, count this value to assert A53 l2retn to 1'b0 NOTE: Can't be programmed to zero (This register control only for SCU Type PGC)
15–14 -	This field is reserved. Reserved

Table continues on the next page...

GPC_PGC_NOC_MIX_CTRL field descriptions (continued)

Field	Description
13–8 DFTRAM_TCD1	After scu pdn_req, count this value to assert A53 dftram to 1'b1 NOTE: Can't be programmed to zero (This register control only for SCU Type PGC)
7 -	This field is reserved. Reserved
6–1 L2RSTDIS	After scu pdn_req, count this value to assert A53 l2rstdis to 1'b1, it will be clear automatically once any of A53 core0/core1/core2/core3 is wakeup NOTE: Can't be programmed to zero (This register control only for SCU Type PGC)
0 MIX_PCR	Power Control NOTE: PCR must not change from power-down request (pdn_req) assertion until the target subsystem is completely powered up. 0 Do not switch off power even if pdn_req is asserted. 1 Switch off power when pdn_req is asserted.

5.2.11.8 GPC PGC Up Sequence Control Register (GPC_PGC_NOC_MIX_PUPSCR)

GPC PGC Up Sequence Control Register

Address: 303A_0000h base + A44h offset = 303A_0A44h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PUP_SCALL_SCALLOUT_CNT								SW2ISO							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	SW2ISO								PUP_WAIT_SCALL_OUT		SW					
W																
Reset	1	0	0	1	0	1	1	1	1	1	0	0	0	0	0	1

GPC_PGC_NOC_MIX_PUPSCR field descriptions

Field	Description
31–23 PUP_SCALL_ SCALLOUT_CNT	After SCALL asserting to 1'b0, count this value to complete switch power up NOTE: Only valid when pup_wait_scall_out is set to 1'b0. Can't be programmed to zero (This register control only for MIX Type PGC)
22–7 SW2ISO	After asserting switch_b, the PGC waits a number of clocks equal to the value of SW2ISO before negating isolation.
6 PUP_WAIT_ SCALL_OUT	After SCALL asserting to 1'b0, wait handshake signal SCALL_OUT to return to 1'b0 (This register control only for MIX Type PGC)
SW	After a power-up request (pup_req assertion), the PGC waits a number of clocks equal to the value of SW before asserting switch_b NOTE: SW must not be programmed to zero.

5.2.11.9 GPC PGC Down Sequence Control Register (GPC_PGC_NOC_MIX_PDNSCR)

GPC PGC Down Sequence Control Register

Address: 303A_0000h base + A48h offset = 303A_0A48h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PUP_SCPRE_SCALL_CNT								PDN_SCALL_SCALLOUT_CNT							
W	PUP_SCPRE_SCALL_CNT								PDN_SCALL_SCALLOUT_CNT							
Reset	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		ISO2SW						PDN_WAIT_SCALL_OUT	Reserved	ISO					
W	Reserved		ISO2SW						PDN_WAIT_SCALL_OUT	Reserved	ISO					
Reset	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1

GPC_PGC_NOC_MIX_PDNSCR field descriptions

Field	Description
31–24 PUP_SCPRE_ SCALL_CNT	After SCPRE asserting to 1'b0, count this value to assert SCALL to 1'b0 (This register control only for MIX Type PGC)
23–16 PDN_SCALL_ SCALLOUT_CNT	Default 8'h0. After SCALL asserting to 1'b1, count this value to complete switch power down. NOTE: This control is only valid when pdn_wait_scall_out is set to 1'b0. It can be programmed to zero. (This register control only for MIX Type PGC)
15–14 -	This field is reserved. Reserved
13–8 ISO2SW	After asserting isolation(by pdn_req assertion), the PGC waits a number of clocks equal to the value of ISO2SW before negating switch_b NOTE: ISO2SW must not be programmed to zero.
7 PDN_WAIT_ SCALL_OUT	Default 1'b0 If set to 1'b1, after SCALL asserting to 1'b1, wait handshake signal SCALL_OUT to change to 1'b1. NOTE: This register control is only for MIX Type PGC.
6 -	This field is reserved. Reserved
ISO	After a power-down request (pdn_req assertion), the PGC waits a number of clocks equal to the value of ISO before asserting isolation NOTE: ISO must not be programmed to zero.

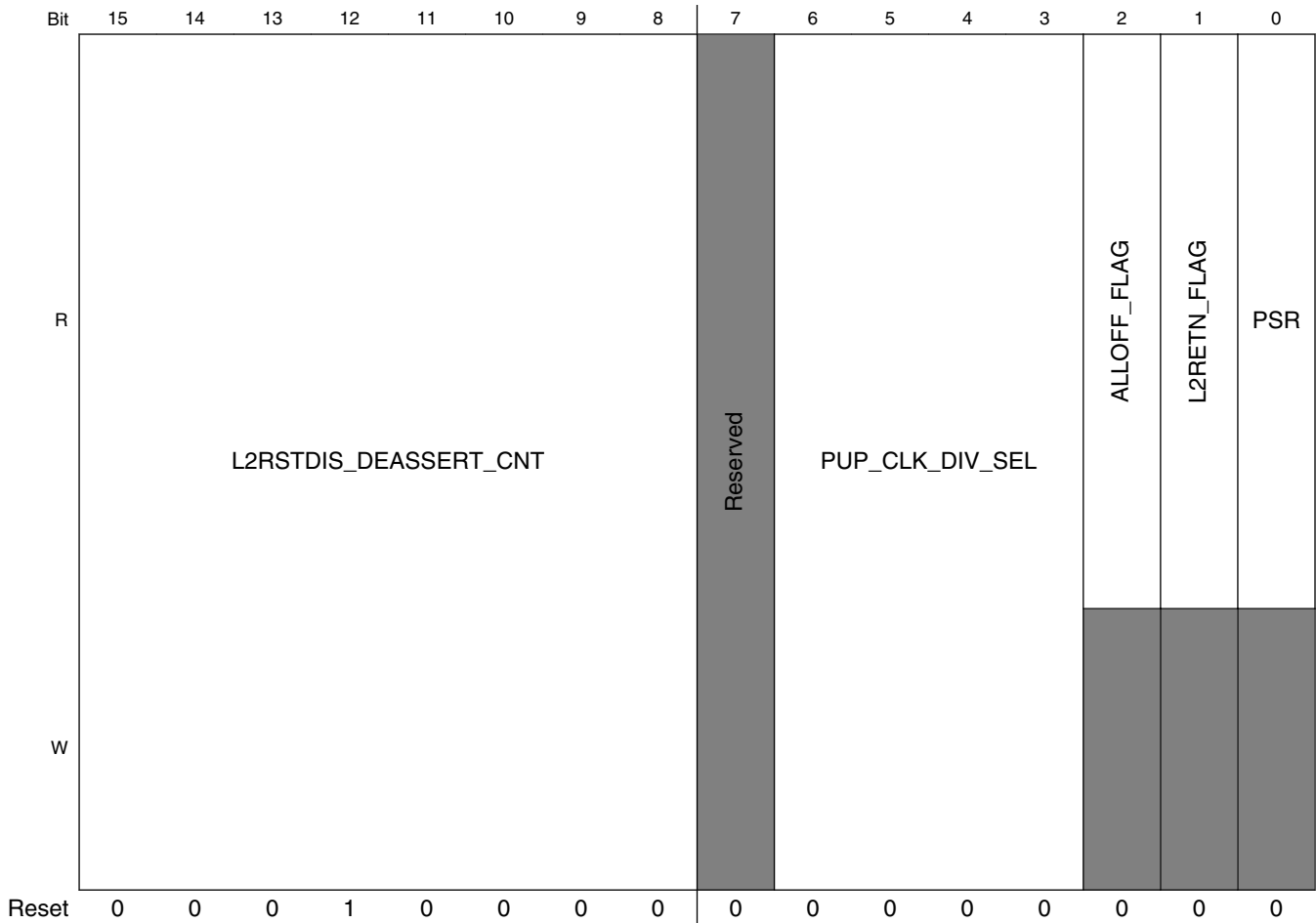
5.2.11.10 GPC PGC Status Register (GPC_PGC_NOC_MIX_SR)

GPC PGC Status Register

Address: 303A_0000h base + A4Ch offset = 303A_0A4Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved														L2RSTDIS_ DEASSERT_ CNT	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

General Power Controller (GPC)



GPC_PGC_NOC_MIX_SR field descriptions

Field	Description
31–18 -	This field is reserved. Reserved
17–8 L2RSTDIS_ DEASSERT_ CNT	Count this value to de-assert L2RSTDISABLE to LOW after CPU0 or CPU1 power up NOTE: This value can't be programmed to zero (This register control only for SCU Type PGC)
7 -	This field is reserved. Reserved
6–3 PUP_CLK_DIV_ SEL	Clock divider select for the clock of power up counter(count_clk is 32KHz for CPU/SCU type PGC, ipg_clk(66MHz) for MIX/PU Type PGC) 0000 1 0001 1/2 count_clk 0010 1/4 count_clk 0011 1/8 count_clk 0100 1/16 count_clk 0101 1/32 count_clk 0110 1/64 count_clk 0111 1/128 count_clk

Table continues on the next page...

GPC_PGC_NOC_MIX_SR field descriptions (continued)

Field	Description
	1000 1/256 count_clk 1001 1/512 count_clk 1010 1/1024 count_clk 1011 1/2056 count_clk 1100 1/4096 count_clk 1101 1/8192 count_clk 1110 1/16384 count_clk 1111 1/32768 count_clk
² ALLOFF_FLAG	All-off flag. NOTE: Software should write “1” to clear this flag after A53 is wakeup from ALL_OFF mode, otherwise, it will always keep to 1 (This register control only for SCU Type PGC) 0 A53 is not wakeup from ALL_OFF mode. 1 A53 is wakeup from ALL_OFF mode.
¹ L2RETN_FLAG	L2 Retention Flag NOTE: Software should write “1” to clear this flag after A53 is wakeup from L2 retention mode, otherwise it will always keep to 1 (This register control only for SCU Type PGC) 0 A53 is not wakeup from L2 retention mode. 1 A53 is wakeup from L2 retention mode.
⁰ PSR	Power status. When in functional (or software-controlled debug) mode, PGC hardware sets PSR as soon as any of the power control output changes its state to one. Write one to clear this bit. Software should clear this bit after power up; otherwise, PSR continues to reflect the power status of the initial power down. 0 The target subsystem was not powered down for the previous power-down request. 1 The target subsystem was powered down for the previous power-down request.

5.2.11.11 GPC PGC Control Register for PGC PUs (GPC_PGC_nCTRL)

GPC PGC Control Register for the PUs. See the PGC Memory Map for the assignments.

Address: 303A_0000h base + C00h offset + (64d × i), where i=0d to 13d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16		
R	Reserved		MEMPWR_TCD1_TDR_TRM							Reserved		L2RETN_TCD1_TDR						
W																		
Reset	0	0	0	0	0	1	1	0	0	0	0	0	0	1	0	0		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
R	Reserved		DFTRAM_TCD1							Reserved	L2RSTDIS							PCR
W																		
Reset	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0		

GPC_PGC_nCTRL field descriptions

Field	Description
31–30 -	This field is reserved. Reserved
29–24 MEMPWR_TCD1_TDR_TRM	After scu pdn_req, count this value to assert A53 mempwr to 1'b1 NOTE: Can't be programmed to zero (This register control only for SCU Type PGC)
23–22 -	This field is reserved. Reserved
21–16 L2RETN_TCD1_TDR	After scu pdn_req, count this value to assert A53 l2retn to 1'b0 NOTE: Can't be programmed to zero (This register control only for SCU Type PGC)
15–14 -	This field is reserved. Reserved

Table continues on the next page...

GPC_PGC_nCTRL field descriptions (continued)

Field	Description
13–8 DFTRAM_TCD1	After scu pdn_req, count this value to assert A53 dftram to 1'b1 NOTE: Can't be programmed to zero (This register control only for SCU Type PGC)
7 -	This field is reserved. Reserved
6–1 L2RSTDIS	After scu pdn_req, count this value to assert A53 l2rstdis to 1'b1, it will be clear automatically once any of A53 core0/core1/core2/core3 is wakeup NOTE: Can't be programmed to zero (This register control only for SCU Type PGC)
0 PCR	Power Control NOTE: PCR must not change from power-down request (pdn_req) assertion until the target subsystem is completely powered up. 0 Do not switch off power even if pdn_req is asserted. 1 Switch off power when pdn_req is asserted.

5.2.11.12 GPC PGC Up Sequence Control Register (GPC_PGC_nPUPSCR)

GPC PGC Up Sequence Control Register

Address: 303A_0000h base + C04h offset + (64d × i), where i=0d to 13d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PUP_SCALL_SCALLOUT_CNT								SW2ISO							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	SW2ISO								PUP_WAIT_SCALL_OUT		SW					
W																
Reset	1	0	0	1	0	1	1	1	1	1	0	0	0	0	0	1

GPC_PGC_nPUPSCR field descriptions

Field	Description
31–23 PUP_SCALL_ SCALLOUT_CNT	After SCALL asserting to 1'b0, count this value to complete switch power up NOTE: Only valid when pup_wait_scall_out is set to 1'b0. Can't be programmed to zero (This register control only for MIX Type PGC)
22–7 SW2ISO	After asserting switch_b, the PGC waits a number of clocks equal to the value of SW2ISO before negating isolation.
6 PUP_WAIT_ SCALL_OUT	After SCALL asserting to 1'b0, wait handshake signal SCALL_OUT to return to 1'b0 (This register control only for MIX Type PGC)
SW	After a power-up request (pup_req assertion), the PGC waits a number of clocks equal to the value of SW before asserting switch_b NOTE: SW must not be programmed to zero.

5.2.11.13 GPC PGC Down Sequence Control Register (GPC_PGC_nPDNSCR)

GPC PGC Down Sequence Control Register

Address: 303A_0000h base + C08h offset + (64d × i), where i=0d to 13d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PUP_SCPRE_SCALL_CNT								PDN_SCALL_SCALLOUT_CNT							
W																
Reset	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				ISO2SW				PDN_WAIT_SCALL_OUT	Reserved	ISO					
W																
Reset	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1

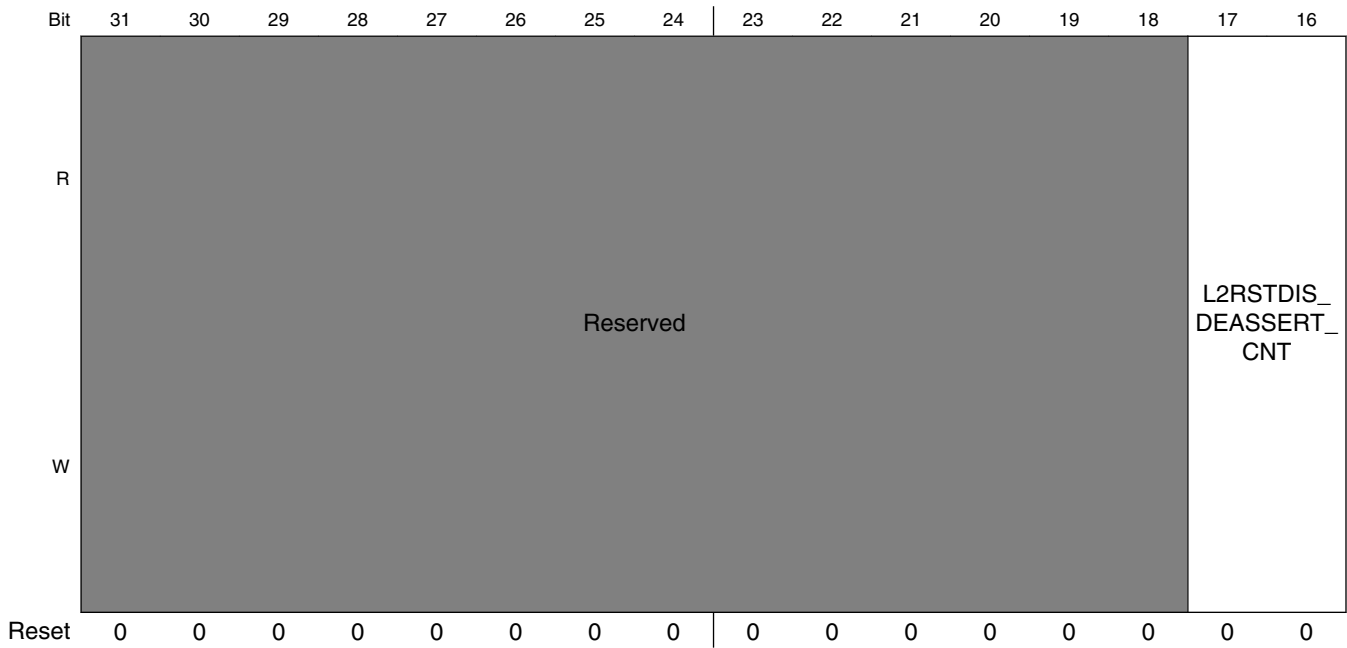
GPC_PGC_nPDNSCR field descriptions

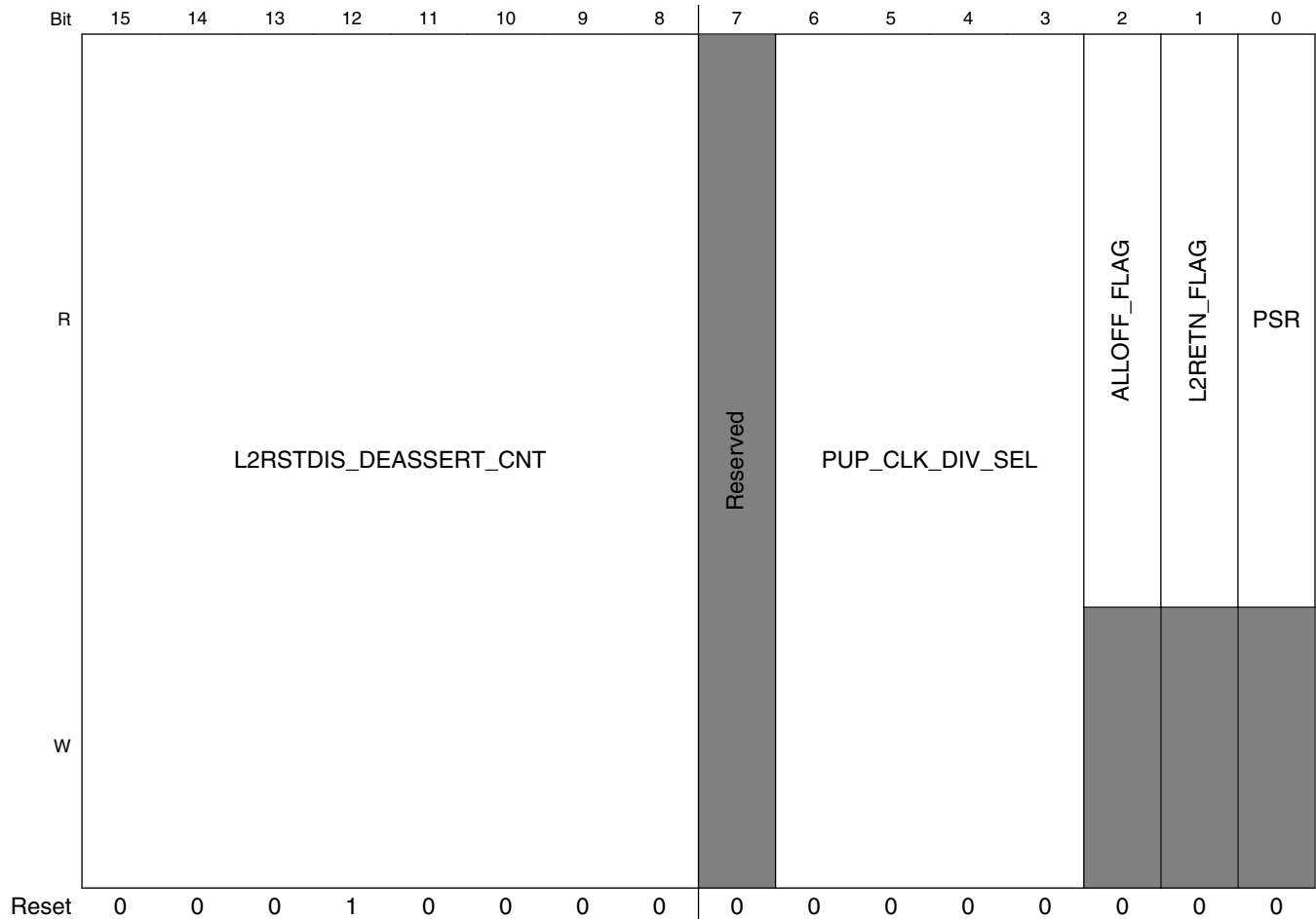
Field	Description
31–24 PUP_SCPRE_ SCALL_CNT	After SCPRE asserting to 1'b0, count this value to assert SCALL to 1'b0 (This register control only for MIX Type PGC)
23–16 PDN_SCALL_ SCALLOUT_CNT	Default 8'h0. After SCALL asserting to 1'b1, count this value to complete switch power down. NOTE: This control is only valid when pdn_wait_scall_out is set to 1'b0. It can be programmed to zero. (This register control only for MIX Type PGC)
15–14 -	This field is reserved. Reserved
13–8 ISO2SW	After asserting isolation(by pdn_req assertion), the PGC waits a number of clocks equal to the value of ISO2SW before negating switch_b NOTE: ISO2SW must not be programmed to zero.
7 PDN_WAIT_ SCALL_OUT	Default 1'b0 If set to 1'b1, after SCALL asserting to 1'b1, wait handshake signal SCALL_OUT to change to 1'b1. NOTE: This register control is only for MIX Type PGC.
6 -	This field is reserved. Reserved
ISO	After a power-down request (pdn_req assertion), the PGC waits a number of clocks equal to the value of ISO before asserting isolation NOTE: ISO must not be programmed to zero.

5.2.11.14 GPC PGC Status Register (GPC_PGC_nSR)

GPC PGC Status Register

Address: 303A_0000h base + C0Ch offset + (64d × i), where i=0d to 13d





GPC_PGC_nSR field descriptions

Field	Description
31-18 -	This field is reserved. Reserved
17-8 L2RSTDIS_ DEASSERT_ CNT	Count this value to de-assert L2RSTDISABLE to LOW after CPU0 or CPU1 power up NOTE: This value can't be programmed to zero (This register control only for SCU Type PGC)
7 -	This field is reserved. Reserved
6-3 PUP_CLK_DIV_ SEL	Clock divider select for the clock of power up counter(count_clk is 32KHz for CPU/SCU type PGC, ipg_clk(66MHz) for MIX/PU Type PGC) 0000 1 0001 1/2 count_clk 0010 1/4 count_clk 0011 1/8 count_clk 0100 1/16 count_clk 0101 1/32 count_clk 0110 1/64 count_clk 0111 1/128 count_clk

Table continues on the next page...

GPC_PGC_nSR field descriptions (continued)

Field	Description
	1000 1/256 count_clk 1001 1/512 count_clk 1010 1/1024 count_clk 1011 1/2056 count_clk 1100 1/4096 count_clk 1101 1/8192 count_clk 1110 1/16384 count_clk 1111 1/32768 count_clk
2 ALLOFF_FLAG	All-off flag. NOTE: Software should write “1” to clear this flag after A53 is wakeup from ALL_OFF mode, otherwise, it will always keep to 1 (This register control only for SCU Type PGC) 0 A53 is not wakeup from ALL_OFF mode. 1 A53 is wakeup from ALL_OFF mode.
1 L2RETN_FLAG	L2 Retention Flag NOTE: Software should write “1” to clear this flag after A53 is wakeup from L2 retention mode, otherwise it will always keep to 1 (This register control only for SCU Type PGC) 0 A53 is not wakeup from L2 retention mode. 1 A53 is wakeup from L2 retention mode.
0 PSR	Power status. When in functional (or software-controlled debug) mode, PGC hardware sets PSR as soon as any of the power control output changes its state to one. Write one to clear this bit. Software should clear this bit after power up; otherwise, PSR continues to reflect the power status of the initial power down. 0 The target subsystem was not powered down for the previous power-down request. 1 The target subsystem was powered down for the previous power-down request.

5.2.11.15 GPC PGC Auxiliary Power Switch Control Register (GPC_PGC_nAUXSW)

GPC PGC Auxiliary Power Switch Control Register. The register has different field definition for PCIE/MIPI PGC and SCU Type PGC.

Address: 303A_0000h base + C10h offset + (64d × i), where i=0d to 13d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved												PDN_CLK_DIV_SEL			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		ISO2SW2						Reserved		SW2					
W																
Reset	0	0	0	0	0	0	0	1	0	0	1	1	0	0	0	1

GPC_PGC_nAUXSW field descriptions

Field	Description
31–20 -	This field is reserved. Reserved
19–16 PDN_CLK_DIV_SEL	Clock divider select for the clock of power down counter 0000 1 0001 1/2 count_clk 0010 1/4 count_clk 0011 1/8 count_clk 0100 1/16 count_clk 0101 1/32 count_clk 0110 1/64 count_clk 0111 1/128 count_clk 1000 1/256 count_clk 1001 1/512 count_clk 1010 1/1024 count_clk 1011 1/2056 count_clk 1100 1/4096 count_clk 1101 1/8192 count_clk 1110 1/16384 count_clk 1111 1/32768 count_clk
15–14 -	This field is reserved. Reserved
13–8 ISO2SW2	

Table continues on the next page...

GPC_PGC_nAUXSW field descriptions (continued)

Field	Description
	after asserting isolation by power-down request(pdn_req assertion), the PGC waits a number of clocks equal to the value of ISO2SW2 before negating switch2_b(1P8 Power) NOTE: ISO2SW2 must not be programmed to zero. 0 A53 is not wakeup from ALL_OFF mode. 1 A53 is wakeup from ALL_OFF mode.
7–6 -	This field is reserved. Reserved
SW2	After a power-up request (pup_req assertion), the PGC waits a number of clocks equal to the value of SW2 before asserting switch2_b(1P8 Power) NOTE: SW2 must not be programmed to zero.

5.3 Crystal Oscillator (XTALOSC)

5.3.1 Overview

The chip has three XTAL modules, 25MHz XTAL module, 27MHz XTAL module and 32KHz XTAL Module. The 25MHz and 27MHz XTAL modules are instantiated from the same XTAL IP, which includes:

- 25/27MHz crystal oscillator to generate reference clock
- Digital control logics for the XTAL

The 32KHz XTAL module uses a different IP and it is used as the clock source for the RTC, located in the SNVS.

The 25MHz oscillator will be used as the primary clock source for the PLLs to generate the clock for CPU, BUS, and high-speed interfaces. For fractional PLLs, the 25MHz clock from the oscillator can be used as the PLL reference clock directly.

The 27MHz oscillator will be used as the reference clock for HDMI PHY. It can also be used as the alternative source for the PLLs.

Each XTAL module supports the following modes through register configuration:

- Normal oscillator mode - In normal mode, the XTAL IP generates stable square wave based on the crystal oscillator input.
- Bypass mode - In bypass mode, an external clock can be input through the XTAL pad. The IP supports up to 50MHz external clock in bypass mode.

5.3.2 XTALOSC Memory Map/Register Definition

XTALOSC memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3027_0000	25M Oscillator Control Configuration Register (XTALOSC_OSC25M_CTL_CFG)	32	R/W	0180_2070h	5.3.2.1/797
3027_8000	27M Oscillator Control Configuration Register (XTALOSC_OSC27M_CTL_CFG)	32	R/W	0180_2070h	5.3.2.2/799

5.3.2.1 25M Oscillator Control Configuration Register (XTALOSC_OSC25M_CTL_CFG)

25M Oscillator Control Configuration Register

Address: 3027_0000h base + 0h offset = 3027_0000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	OSC_BYPSS	OSC_GM_TST_SEL	Reserved							OSC_EOCV						
W																
Reset	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	OSC_INT_MASK	Reserved	OSC_OK_BYPASS	OSC_DIV					OSC_INT_STU	OSC_GM_SEL			OSC_HYST_CTL	OSC_ALC_CTL	Reserved	
W																
Reset	0	0	1	0	0	0	0	0	0	1	1	1	0	0	0	0

XTALOSC_OSC25M_CTL_CFG field descriptions

Field	Description
31 OSC_BYPSS	Crystal Oscillator bypass This bit specifies whether the oscillator should be bypassed or not. Software can only set this bit. System reset is needed to reset this bit. 0 Oscillator output is used as root clock. 1 EXTAL is used as root clock
30 OSC_GM_TST_SEL	Test mode GM measurement 0 Normal run mode 1 Enable test mode measurement of GM
29–24 -	This field is reserved. Reserved
23–16 OSC_EOCV	End of Count Value These bits specify the end of count value to be used for comparison by the oscillator stabilization counter OSCCNT after reset or whenever it is switched on from the off state. This counting period ensures that external oscillator clock signal is stable before it can be selected by the system. When oscillator counter reaches the value EOCV 512, oscillator available interrupt request is generated. The OSCCNT counter is kept under reset if oscillator bypass mode is selected
15 OSC_INT_MASK	Crystal oscillator clock interrupt mask This bit masks the I_OSC interrupt bit. 0 Crystal oscillator clock interrupt is masked 1 Crystal oscillator clock interrupt is enabled
14 -	This field is reserved. Reserved
13 OSC_OK_BYPASS	OSC ok output bypass
12–8 OSC_DIV	Crystal oscillator clock division factor These bits specify the crystal oscillator output clock division factor. The output clock is divided by the factor OSCDIV+1
7 OSC_INT_STU	Crystal oscillator clock interrupt This bit is set by hardware when OSCCNT counter reaches the count value EOCV x 512. It is cleared by software by writing 1. 0 No oscillator clock interrupt occurred 1 Oscillator clock interrupt pending
6–4 OSC_GM_SEL	Crystal overdrive protection
3 OSC_HYST_CTL	Hysteresis Control 0 Enable hysteresis control 1 Disable hysteresis control
2 OSC_ALC_CTL	Automatic Level Controller Enable 0 Enable automatic level controller 1 Disable automatic level controller
-	This field is reserved. Reserved

5.3.2.2 27M Oscillator Control Configuration Register (XTALOSC_OSC27M_CTL_CFG)

27M Oscillator Control Configuration Register

Address: 3027_0000h base + 8000h offset = 3027_8000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	OSC_BYPSS	OSC_GM_TST_SEL	Reserved							OSC_EOCV						
W																
Reset	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	OSC_INT_MASK	Reserved	OSC_OK_BYPASS	OSC_DIV					OSC_INT_STU	OSC_GM_SEL			OSC_HYST_CTL	OSC_ALC_CTL	Reserved	
W																
Reset	0	0	1	0	0	0	0	0	0	1	1	1	0	0	0	0

XTALOSC_OSC27M_CTL_CFG field descriptions

Field	Description
31 OSC_BYPSS	Crystal Oscillator bypass This bit specifies whether the oscillator should be bypassed or not. Software can only set this bit. System reset is needed to reset this bit. 0 Oscillator output is used as root clock. 1 EXTAL is used as root clock
30 OSC_GM_TST_SEL	Test mode GM measurement 0 Normal run mode 1 Enable test mode measurement of GM
29–24 -	This field is reserved. Reserved
23–16 OSC_EOCV	End of Count Value These bits specify the end of count value to be used for comparison by the oscillator stabilization counter OSCCNT after reset or whenever it is switched on from the off state. This counting period ensures that external oscillator clock signal is stable before it can be selected by the system. When

Table continues on the next page...

XTALOSC_OSC27M_CTL_CFG field descriptions (continued)

Field	Description
	oscillator counter reaches the value EOCV 512, oscillator available interrupt request is generated. The OSCCNT counter is kept under reset if oscillator bypass mode is selected
15 OSC_INT_MASK	Crystal oscillator clock interrupt mask This bit masks the I_OSC interrupt bit. 0 Crystal oscillator clock interrupt is masked 1 Crystal oscillator clock interrupt is enabled
14 -	This field is reserved. Reserved
13 OSC_OK_BYPASS	OSC ok output bypass
12–8 OSC_DIV	Crystal oscillator clock division factor These bits specify the crystal oscillator output clock division factor. The output clock is divided by the factor OSCDIV+1
7 OSC_INT_STU	Crystal oscillator clock interrupt This bit is set by hardware when OSCCNT counter reaches the count value EOCV x 512. It is cleared by software by writing 1. 0 No oscillator clock interrupt occurred 1 Oscillator clock interrupt pending
6–4 OSC_GM_SEL	Crystal overdrive protection
3 OSC_HYST_CTL	Hysteresis Control 0 Enable hysteresis control 1 Disable hysteresis control
2 OSC_ALC_CTL	Automatic Level Controller Enable 0 Enable automatic level controller 1 Disable automatic level controller
-	This field is reserved. Reserved

5.4 Thermal Management Unit (TMU)

5.4.1 TMU as implemented on the chip

The local sensors for each channel are put at different locations on the chip to have more accurate temperature monitoring for certain IP. Since the die size is not big, only 3 local sensors are used on this chip. The sensors are put next to high-power modules such as CPU, GPU. The table below shows the local sensor placement.

Table 5-12. Local Sensor Placement

Temperature	Sensor
0	ARM
1	GPU
2	VPU
3-15	Reserved

5.4.2 Thermal Monitoring Unit Introduction

The Thermal Monitoring Unit (TMU) monitors and reports the temperature from one or more remote temperature measurement sites located on chip.

5.4.2.1 TMU Overview

The TMU has access to multiple temperature measurement sites strategically located on the chip. It monitors these sites and can signal an alarm if a programmed threshold is ever exceeded. The upper and lower temperature range is continuously captured. A set of reporting registers allow for reading the current temperature at monitored sites.

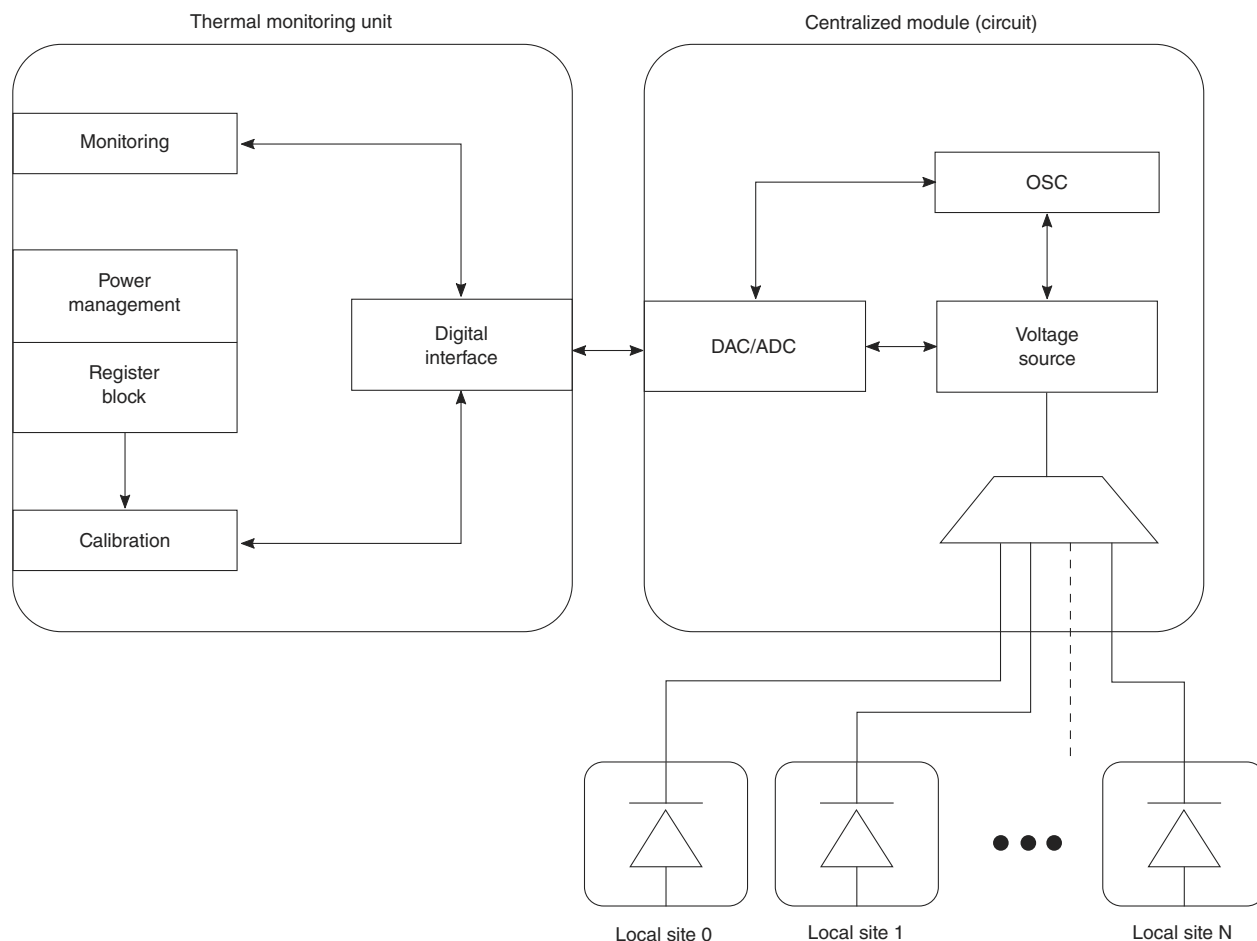


Figure 5-17. Thermal Monitoring Unit Block Diagram

5.4.2.2 Features

The temperature management unit features:

- Temperature measurement range 0-125°C.
- Calibration
 - Calibration table loaded from boot code ROM using pre-boot loader *or* initialization of calibration table through software
- Monitoring
 - Single-, or multi-site monitoring
 - Programmable monitoring interval
 - Out-of-range indication
 - High/low temperature range monitoring
 - Immediate and average temperature monitoring

- Average temperature monitoring programmable low-pass filtering
- Programmable monitoring thresholds for normal and critical alarm
- Reporting
 - Immediate and average temperature reporting for all monitor sites

5.4.2.3 Modes of Operation

The TMU has one mode of operation:

- Monitoring

The mode register monitoring enable bit, TMR[ME], determines if the unit is in active monitoring mode or in power saving mode.

The table below describes bit settings required for each TMU mode of operation.

Table 5-13. TMU Mode Bit Setting

Modes with Features	TMR[ME]
Monitoring mode disabled	0
Monitoring mode enabled	1

5.4.3 TMU register descriptions

The table shows the memory map for management of the TMU resources.

5.4.3.1 TMU Memory map

TMU base address: 3026_0000h

Offset	Register	Width (In bits)	Access	Reset value
0h	TMU mode register (TMR)	32	RW	0000_0000h
4h	TMU status register (TSR)	32	RO	0000_0000h
8h	TMU monitor temperature measurement interval register (TMTMIR)	32	RW	0000_0000h
20h	TMU interrupt enable register (TIER)	32	RW	0000_0000h
24h	TMU interrupt detect register (TIDR)	32	W1C	0000_0000h
28h	TMU interrupt site capture register (TISCR)	32	RW	0000_0000h

Table continues on the next page...

Thermal Management Unit (TMU)

Offset	Register	Width (In bits)	Access	Reset value
2Ch	TMU interrupt critical site capture register (TICSCR)	32	RW	0000_0000h
40h	TMU monitor high temperature capture register (TMHTCR)	32	RO	0000_0000h
44h	TMU monitor low temperature capture register (TMLTCR)	32	RO	0000_0000h
50h	TMU monitor high temperature immediate threshold register (TMHT ITR)	32	RW	0000_0000h
54h	TMU monitor high temperature average threshold register (TMHT ATR)	32	RW	0000_0000h
58h	TMU monitor high temperature average critical threshold register (TMHTACTR)	32	RW	0000_0000h
80h	TMU temperature configuration register (TTCFGR)	32	RW	0000_0000h
84h	TMU sensor configuration register (TSCFGR)	32	RW	0000_0000h
100h	TMU report immediate temperature site register 0 (TRITSR0)	32	RO	0000_0000h
104h	TMU report average temperature site register 0 (TRATSR0)	32	RO	0000_0000h
110h	TMU report immediate temperature site register 1 (TRITSR1)	32	RO	0000_0000h
114h	TMU report average temperature site register 1 (TRATSR1)	32	RO	0000_0000h
120h	TMU report immediate temperature site register 2 (TRITSR2)	32	RO	0000_0000h
124h	TMU report average temperature site register 2 (TRATSR2)	32	RO	0000_0000h
F10h	TMU temperature range 0 control register (TTR0CR)	32	RW	000B_0000h
F14h	TMU temperature range 1 control register (TTR1CR)	32	RW	000A_0026h
F18h	TMU temperature range 2 control register (TTR2CR)	32	RW	0008_0048h
F1Ch	TMU temperature range 3 control register (TTR3CR)	32	RW	0007_0061h

5.4.3.2 TMU mode register (TMR)

5.4.3.2.1 Offset

Register	Offset
TMR	0h

5.4.3.2.2 Function

The TMU mode register allows software to control the operation of the thermal monitoring.

5.4.3.2.3 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W	ME	Reserved			ALPF	Reserved										
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W	MSITE			Reserved												
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

5.4.3.2.4 Fields

Field	Function
31 ME	Monitoring mode enable. 0 No monitoring. 1 Monitoring of sites as defined by field MSITE. Before enabling the TMU for monitoring, the TMU must be configured, see section Initialization Information. Failure to properly initialize the configuration table may result in boundedly undefined behavior.
30-28 —	Reserved
27-26 ALPF	Average low pass filter setting. 00 1.0 01 0.5 10 0.25 11 0.125 The average temperature is calculated as: $ALPF \times Current_Temp + (1 - ALPF) \times Average_Temp$. If no previous (average) temperature is valid, current temperature is used. For proper operation, this field should only change when monitoring is disabled.
25-16 —	Reserved
15-13 MSITE	Monitoring site select 0 - 2. By setting the select bit for a temperature sensor site, it is enabled and included in all monitoring functions. For proper operation, this field should only change when monitoring is disabled. If no site is selected, site 0 is monitored by default.
12-0 —	Reserved

5.4.3.3 TMU status register (TSR)

5.4.3.3.1 Offset

Register	Offset
TSR	4h

5.4.3.3.2 Function

The TMU status register reports the monitoring and calibration status during operation.

5.4.3.3.3 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved	MIE	ORL	ORH	Reserved											
W	Reserved				Reserved											
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

5.4.3.3.4 Fields

Field	Function
31 —	Reserved
30 MIE	Monitoring interval exceeded. 0 Monitoring interval not exceeded. 1 Monitoring interval exceeded. The time required to perform measurement of all monitored sites has exceeded the monitoring interval as defined by TMTMIR. This bit will clear automatically when TMU monitoring is (re-)enabled or the monitoring interval register, TMTMIR, is written.
29 ORL	Out-of-range low temperature measurement detected. A temperature sensor detected a temperature reading below the lowest measurable temperature of 0 degrees Celsius. This bit will clear automatically when TMU monitoring is (re-)enabled.

Table continues on the next page...

Field	Function
28 ORH	Out-of-range high temperature measurement detected. A temperature sensor detected a temperature reading above the highest measurable temperature of 125 °C. This bit will clear automatically when TMU monitoring is (re-)enabled.
27-0 —	Reserved

5.4.3.4 TMU monitor temperature measurement interval register (TMTMIR)

5.4.3.4.1 Offset

Register	Offset
TMTMIR	8h

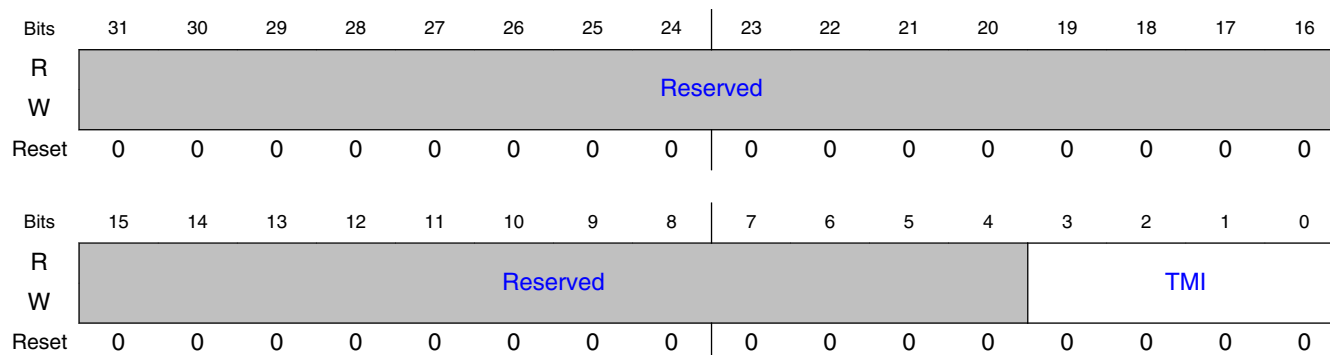
5.4.3.4.2 Function

The TMU monitor temperature measurement interval register determines at what frequency temperature sensors are read. All enabled monitored sites are read once in the duration of the time interval. The status bit TSR[MIE] will be set if the temperature measurement takes longer than the set interval. Software should consider increasing the interval or reducing the number of active sites if the interval is exceeded. Disabling the interval allows for continuous monitoring.

NOTE

The time it takes for one temperature measurement is dependent on the temperatures measured and mode settings, thus there is no fixed time advertised for a single temperature measurements.

5.4.3.4.3 Diagram



5.4.3.4.4 Fields

Field	Function																																																																																										
31-4 —	Reserved																																																																																										
3-0 TMI	Temperature monitoring interval in seconds. For proper operation, this field should only change when monitoring is disabled, TMR[ME]=0. For lower platform speeds, the time increases proportionally, while the opposite is true for higher platform speeds. <table><tr><th>Setting</th><th colspan="4">Platform Clock Frequency</th></tr><tr><td></td><td>333 MHz</td><td>400 MHz</td><td>667 MHz</td><td>800 MHz</td></tr><tr><td>0000</td><td>0.03 s</td><td>0.02 s</td><td>0.015 s</td><td>0.01 s</td></tr><tr><td>0001</td><td>0.06 s</td><td>0.04 s</td><td>0.03 s</td><td>0.02 s</td></tr><tr><td>0010</td><td>0.10 s</td><td>0.08 s</td><td>0.05 s</td><td>0.04 s</td></tr><tr><td>0011</td><td>0.20 s</td><td>0.17 s</td><td>0.10 s</td><td>0.08 s</td></tr><tr><td>0100</td><td>0.40 s</td><td>0.34 s</td><td>0.20 s</td><td>0.17 s</td></tr><tr><td>0101</td><td>0.80 s</td><td>0.67 s</td><td>0.40 s</td><td>0.34 s</td></tr><tr><td>0110</td><td>1.60 s</td><td>1.34 s</td><td>0.80 s</td><td>0.67 s</td></tr><tr><td>0111</td><td>3.2 s</td><td>2.7 s</td><td>1.6 s</td><td>1.34 s</td></tr><tr><td>1000</td><td>6.4 s</td><td>5.4 s</td><td>3.2 s</td><td>2.7 s</td></tr><tr><td>1001</td><td>12.8 s</td><td>10.7 s</td><td>6.4 s</td><td>5.4 s</td></tr><tr><td>1010</td><td>25.8 s</td><td>21.5 s</td><td>12.9 s</td><td>10.7 s</td></tr><tr><td>1011</td><td>51.6 s</td><td>42.9 s</td><td>25.8 s</td><td>21.5 s</td></tr><tr><td>1100</td><td>103 s</td><td>85.9 s</td><td>51.5 s</td><td>42.9 s</td></tr><tr><td>1101</td><td>206.0 s</td><td>171.8 s</td><td>103.0 s</td><td>85.9 s</td></tr><tr><td>1110</td><td>412.2 s</td><td>343.6 s</td><td>206.1 s</td><td>171.8 s</td></tr><tr><td>1111</td><td colspan="4">Disabled</td></tr></table>	Setting	Platform Clock Frequency					333 MHz	400 MHz	667 MHz	800 MHz	0000	0.03 s	0.02 s	0.015 s	0.01 s	0001	0.06 s	0.04 s	0.03 s	0.02 s	0010	0.10 s	0.08 s	0.05 s	0.04 s	0011	0.20 s	0.17 s	0.10 s	0.08 s	0100	0.40 s	0.34 s	0.20 s	0.17 s	0101	0.80 s	0.67 s	0.40 s	0.34 s	0110	1.60 s	1.34 s	0.80 s	0.67 s	0111	3.2 s	2.7 s	1.6 s	1.34 s	1000	6.4 s	5.4 s	3.2 s	2.7 s	1001	12.8 s	10.7 s	6.4 s	5.4 s	1010	25.8 s	21.5 s	12.9 s	10.7 s	1011	51.6 s	42.9 s	25.8 s	21.5 s	1100	103 s	85.9 s	51.5 s	42.9 s	1101	206.0 s	171.8 s	103.0 s	85.9 s	1110	412.2 s	343.6 s	206.1 s	171.8 s	1111	Disabled			
Setting	Platform Clock Frequency																																																																																										
	333 MHz	400 MHz	667 MHz	800 MHz																																																																																							
0000	0.03 s	0.02 s	0.015 s	0.01 s																																																																																							
0001	0.06 s	0.04 s	0.03 s	0.02 s																																																																																							
0010	0.10 s	0.08 s	0.05 s	0.04 s																																																																																							
0011	0.20 s	0.17 s	0.10 s	0.08 s																																																																																							
0100	0.40 s	0.34 s	0.20 s	0.17 s																																																																																							
0101	0.80 s	0.67 s	0.40 s	0.34 s																																																																																							
0110	1.60 s	1.34 s	0.80 s	0.67 s																																																																																							
0111	3.2 s	2.7 s	1.6 s	1.34 s																																																																																							
1000	6.4 s	5.4 s	3.2 s	2.7 s																																																																																							
1001	12.8 s	10.7 s	6.4 s	5.4 s																																																																																							
1010	25.8 s	21.5 s	12.9 s	10.7 s																																																																																							
1011	51.6 s	42.9 s	25.8 s	21.5 s																																																																																							
1100	103 s	85.9 s	51.5 s	42.9 s																																																																																							
1101	206.0 s	171.8 s	103.0 s	85.9 s																																																																																							
1110	412.2 s	343.6 s	206.1 s	171.8 s																																																																																							
1111	Disabled																																																																																										

5.4.3.5 TMU interrupt enable register (TIER)

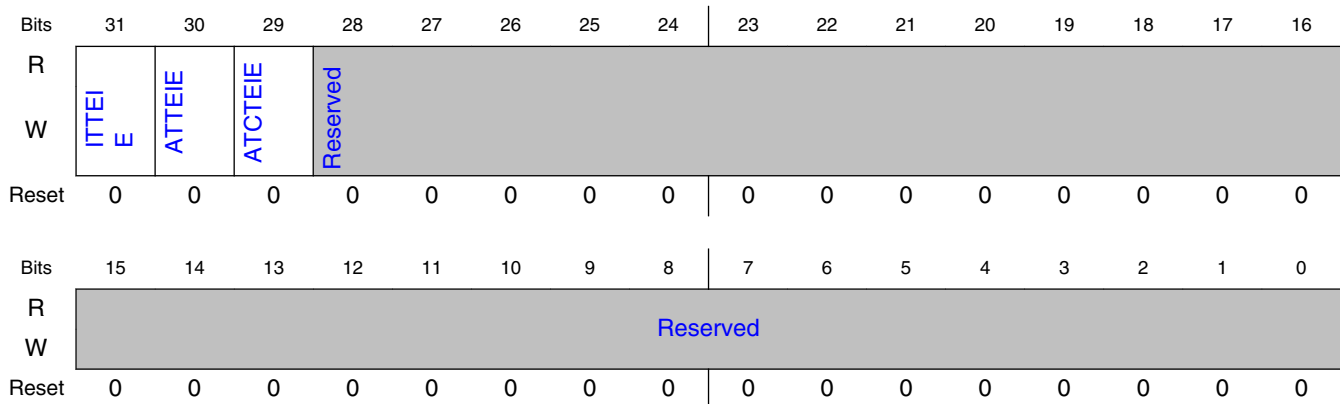
5.4.3.5.1 Offset

Register	Offset
TIER	20h

5.4.3.5.2 Function

The TMU interrupt enable register determines if a detected status condition should cause a system interrupt. A system interrupt occurs if a bit in this register is set and the corresponding bit in the interrupt detect register is also set. To clear the interrupt, write a 1 to the interrupt detect register.

5.4.3.5.3 Diagram



5.4.3.5.4 Fields

Field	Function
31 ITTEIE	Immediate temperature threshold exceeded interrupt enable. 0 Disabled. 1 Interrupt enabled. Generate an interrupt if TIDR[ITTE] is set.
30 ATTEIE	Average temperature threshold exceeded interrupt enable. 0 Disabled. 1 Interrupt enabled. Generate an interrupt if TIDR[ATTE] is set.

Table continues on the next page...

Thermal Management Unit (TMU)

Field	Function
29 ATCTEIE	Average temperature critical threshold exceeded interrupt enable. 0 Disabled. 1 Interrupt enabled. Generate an interrupt if TIDR[ATCTE] is set.
28-0 —	Reserved

5.4.3.6 TMU interrupt detect register (TIDR)

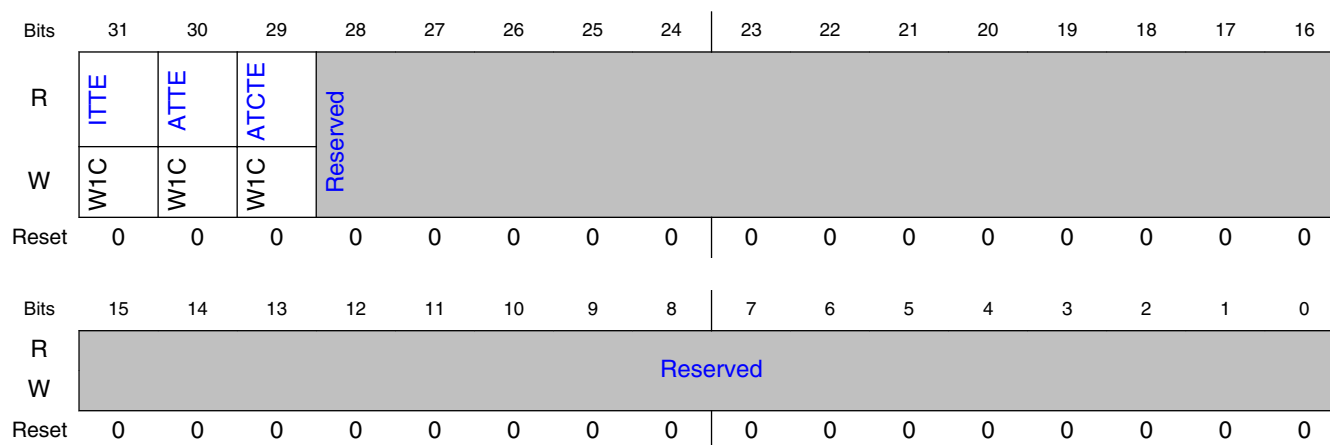
5.4.3.6.1 Offset

Register	Offset
TIDR	24h

5.4.3.6.2 Function

The TMU interrupt detect register indicates if an status condition was detected that could generate an interrupt. Write 1 to clear the detected condition and the interrupt, if enabled.

5.4.3.6.3 Diagram



5.4.3.6.4 Fields

Field	Function
31	Immediate temperature threshold exceeded. Write 1 to clear.

Table continues on the next page...

Field	Function
ITTE	0 No threshold exceeded. 1 Immediate temperature threshold, as defined by TMHTITR, has been exceeded by one or more monitored sites. This includes an out-of-range measured temperature above 85 °C. The sites which has exceeded the threshold are captured in TISCR[ISITE].
30 ATTE	Average temperature threshold exceeded. Write 1 to clear. 0 No threshold exceeded. 1 Average temperature threshold, as defined by TMHTATR, has been exceeded by one or more monitored sites. The sites which has exceeded the threshold are captured in TISCR[ASITE].
29 ATCTE	Average temperature critical threshold exceeded. Write 1 to clear. 0 No threshold exceeded. 1 Average temperature critical threshold, as defined by TMHTACTR, has been exceeded by one or more monitored sites. The sites which has exceeded the threshold are captured in TICSCR[CASITE].
28-0 —	Reserved

5.4.3.7 TMU interrupt site capture register (TISCR)

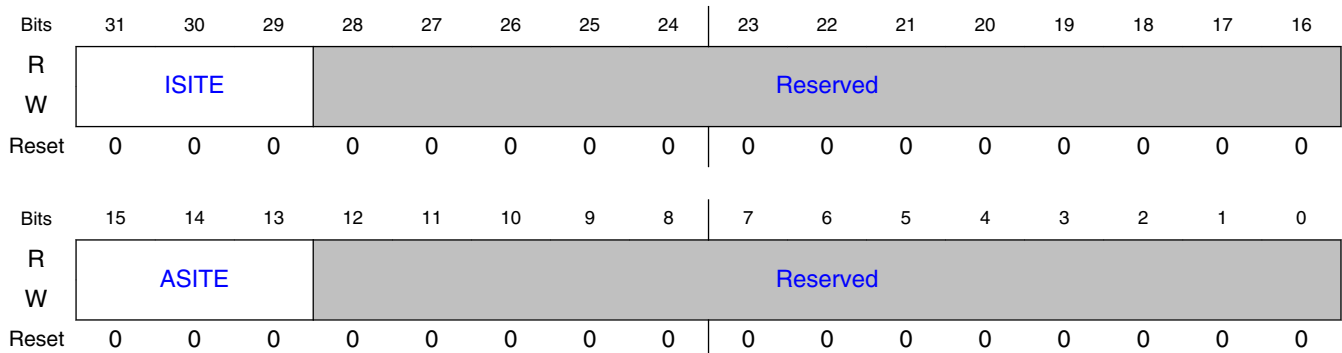
5.4.3.7.1 Offset

Register	Offset
TISCR	28h

5.4.3.7.2 Function

The TMU interrupt site capture register holds information about the temperature sensor site associated with a detected interrupt event.

5.4.3.7.3 Diagram



5.4.3.7.4 Fields

Field	Function
31-29 ISITE	Temperature sensor site associated with the setting of TIDR[ITTE]. This field has the same bit representation as TMR[MSITE]. Software should clear this field after handling the detected interrupt events ITTE.
28-16 —	Reserved
15-13 ASITE	Temperature sensor site associated with the setting of TIDR[ATTE] . This field has the same bit representation as TMR[MSITE]. Software should clear this field after handling the detected interrupt event ATTE.
12-0 —	Reserved

5.4.3.8 TMU interrupt critical site capture register (TICSCR)

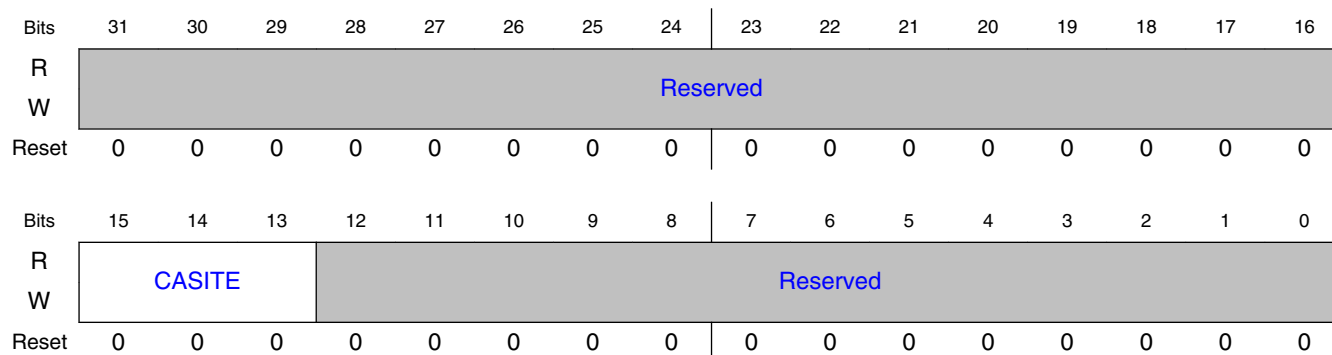
5.4.3.8.1 Offset

Register	Offset
TICSCR	2Ch

5.4.3.8.2 Function

The TMU interrupt critical site capture register holds information about the temperature sensor site associated with a detected critical interrupt event.

5.4.3.8.3 Diagram



5.4.3.8.4 Fields

Field	Function
31-16 —	Reserved
15-13 CASITE	Temperature sensor site associated with the setting of TIDR[ATCTE] . This field has the same bit representation as TMR[MSITE]. Software should clear this field after handling the detected critical interrupt event ATCTE.
12-0 —	Reserved

5.4.3.9 TMU monitor high temperature capture register (TMHTCR)

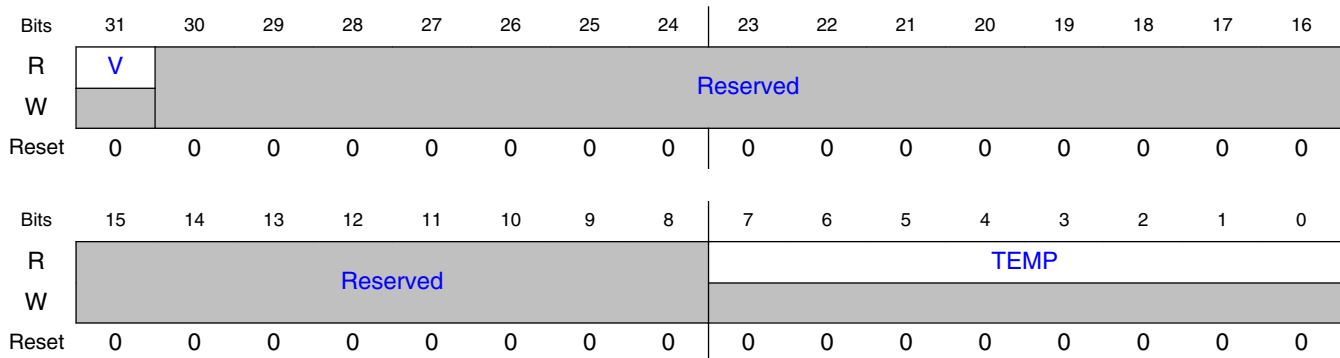
5.4.3.9.1 Offset

Register	Offset
TMHTCR	40h

5.4.3.9.2 Function

This TMU monitor register captures and record the highest temperature reached for any one enabled monitored site within temperature sensor range.

5.4.3.9.3 Diagram



5.4.3.9.4 Fields

Field	Function
31 V	Valid reading. 0 Temperature reading is not valid due to no measured temperature within the sensor range of 0-125 °C for an enabled monitored site. 1 Temperature reading is valid. (Re-)enabling the TMU will automatically clear this bit and start a new search.
30-8 —	Reserved
7-0 TEMP	Highest temperature recorded in degrees Celcius by any enabled monitored site. Valid when V=1. 0-85 °C Sensor range 86-255 °C Reserved

5.4.3.10 TMU monitor low temperature capture register (TMLTCCR)

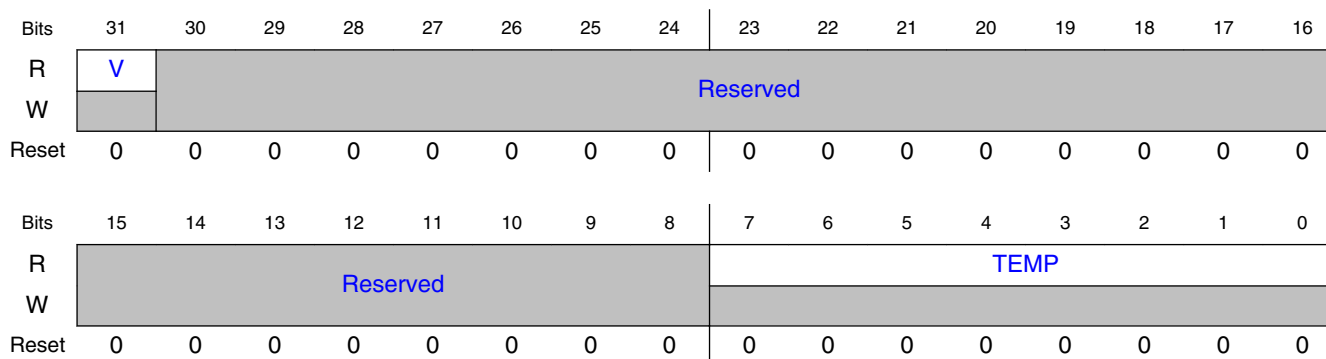
5.4.3.10.1 Offset

Register	Offset
TMLTCCR	44h

5.4.3.10.2 Function

This TMU monitor register captures and record the lowest temperature reached for any one enabled monitored site within temperature sensor range.

5.4.3.10.3 Diagram



5.4.3.10.4 Fields

Field	Function
31 V	Valid reading. 0 Temperature reading is not valid due to no measured temperature within the sensor range of 0-85 °C for an enabled monitored site. 1 Temperature reading is valid. (Re-)enabling the TMU will automatically clear this bit and start a new search.
30-8 —	Reserved
7-0 TEMP	Lowest temperature recorded in degrees Celcius by any enabled monitored site. Valid when V=1. 0-85 °C Sensor range 86-255 °C Reserved

5.4.3.11 TMU monitor high temperature immediate threshold register (TMHTITR)

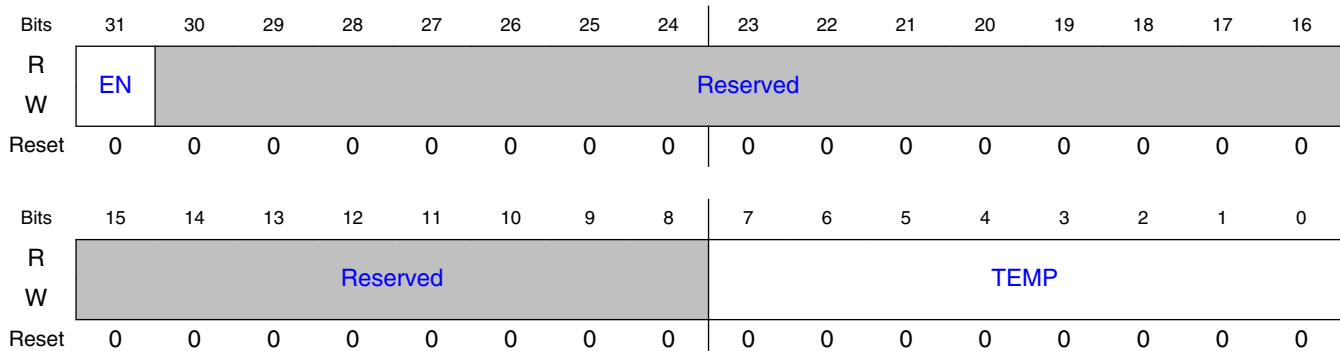
5.4.3.11.1 Offset

Register	Offset
TMHTITR	50h

5.4.3.11.2 Function

This TMU monitor register determines the high current temperature threshold for generating the TIDR[ITTE] event.

5.4.3.11.3 Diagram



5.4.3.11.4 Fields

Field	Function
31 EN	Enable threshold. 0 Disabled. 1 Threshold enabled.
30-8 —	Reserved
7-0 TEMP	High temperature immediate threshold value. Determines the current upper temperature threshold, for any enabled monitored site, that if exceeded will cause TIDR[ITTE] to be set when EN=1. 0-85 °C Sensor range 86-255 °C Reserved

5.4.3.12 TMU monitor high temperature average threshold register (TMHTATR)

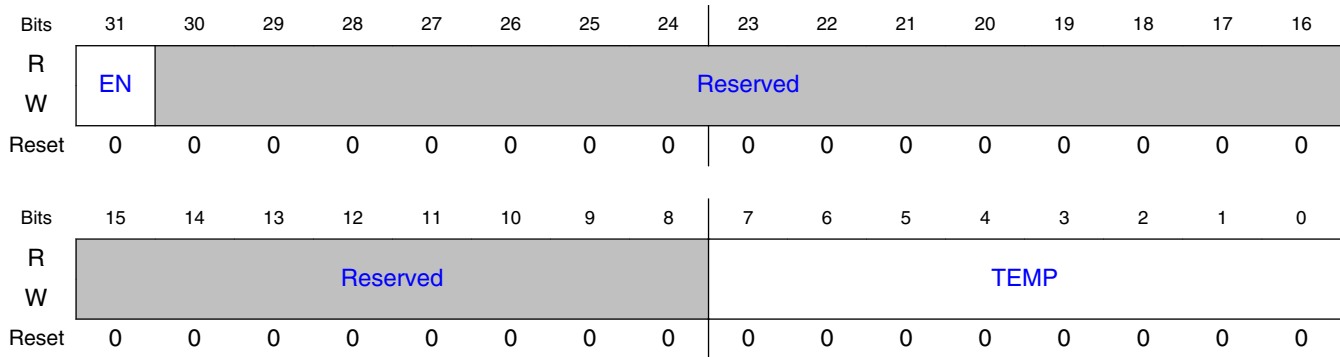
5.4.3.12.1 Offset

Register	Offset
TMHTATR	54h

5.4.3.12.2 Function

This TMU monitor register determines the high average temperature threshold for generating the TIDR[ATTE] event. The low-pass filter setting, TMR[ALPF], determines the function for calculating average temperature.

5.4.3.12.3 Diagram



5.4.3.12.4 Fields

Field	Function
31 EN	Enable threshold. 0 Disabled. 1 Threshold enabled.
30-8 —	Reserved
7-0 TEMP	High temperature average threshold value. Determines the average upper temperature threshold, for any enabled monitor site, that if exceeded will cause TIDR[ATTE] to be set when EN=1. 0-85 °C Sensor range 86-255 °C Reserved

5.4.3.13 TMU monitor high temperature average critical threshold register (TMHTACTR)

5.4.3.13.1 Offset

Register	Offset
TMHTACTR	58h

5.4.3.13.2 Function

This TMU monitor register determines the high average critical temperature threshold for generating the TIDR[ATCTE] event. The low-pass filter setting, TMR[ALPF], determines the function for calculating average temperature.

5.4.3.13.3 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	EN Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								TEMP							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

5.4.3.13.4 Fields

Field	Function
31 EN	Enable threshold. 0 Disabled. 1 Threshold enabled.
30-8 —	Reserved
7-0 TEMP	High temperature average critical threshold value. Determines the average upper critical temperature threshold, for any enabled monitored site, that if exceeded will cause TIDR[ATCTE] to be set when EN=1. 0-85 °C Sensor range 86-255 °C Reserved

5.4.3.14 TMU temperature configuration register (TTCFGR)

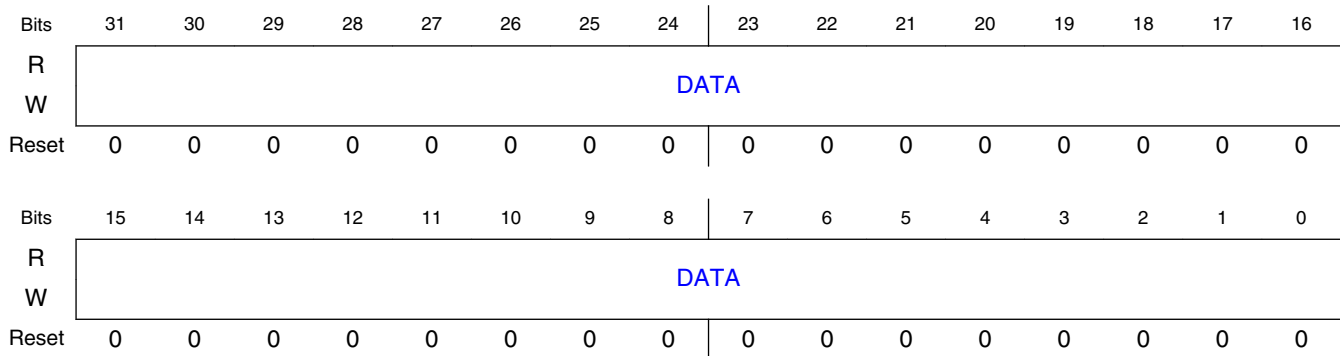
5.4.3.14.1 Offset

Register	Offset
TTCFGR	80h

5.4.3.14.2 Function

The TMU temperature configuration register, in conjunction with the sensor configuration register, is used to initialize the internal sensor translation table used during monitoring. This register pair defines indirect access to the table. See the section Initialization Information.

5.4.3.14.3 Diagram



5.4.3.14.4 Fields

Field	Function
31-0 DATA	Sensor data.

5.4.3.15 TMU sensor configuration register (TSCFGR)

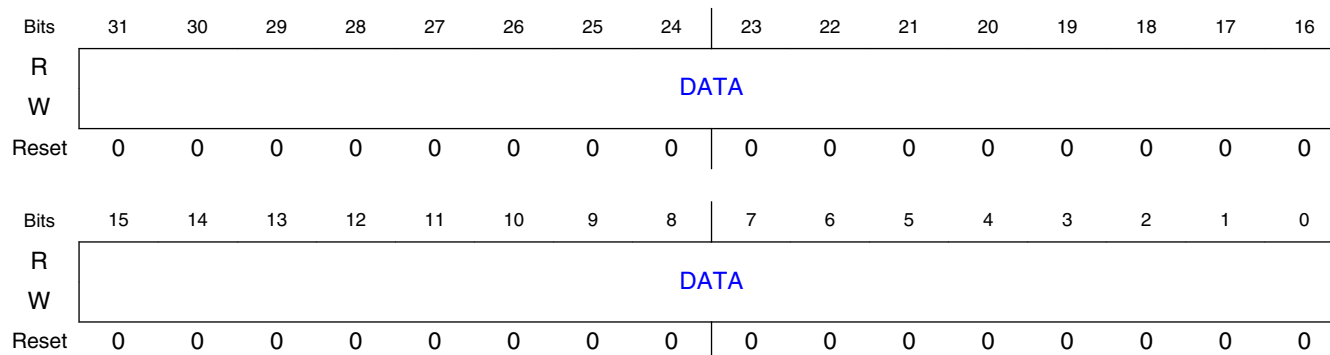
5.4.3.15.1 Offset

Register	Offset
TSCFGR	84h

5.4.3.15.2 Function

The TMU sensor configuration register, in conjunction with the temperature configuration register, is used to initialize the internal sensor translation table used during monitoring. This register pair defines indirect access to the table. Reading this register will return the data from the translation table as defined by TTCFGR. See the section Initialization Information.

5.4.3.15.3 Diagram



5.4.3.15.4 Fields

Field	Function
31-0 DATA	Sensor data.

5.4.3.16 TMU report immediate temperature site register a (TRITSR0 - TRITSR2)

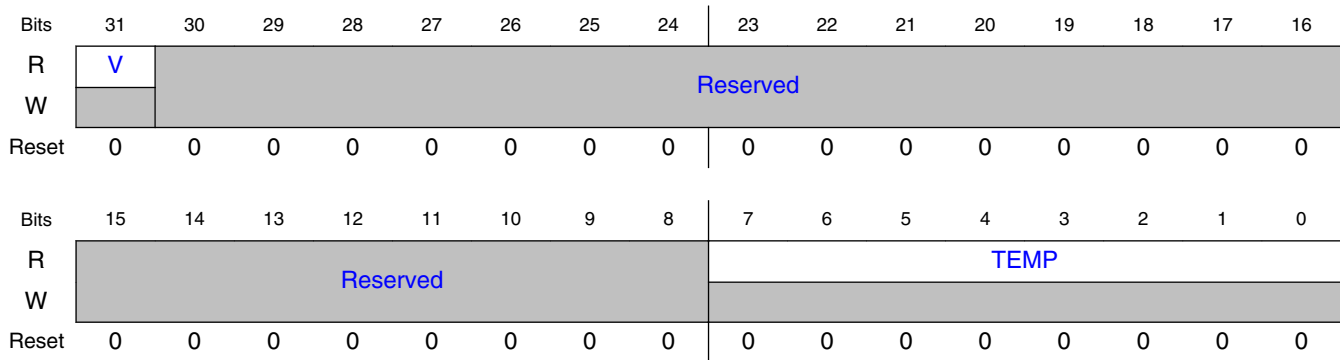
5.4.3.16.1 Offset

Register	Offset
TRITSR0	100h
TRITSR1	110h
TRITSR2	120h

5.4.3.16.2 Function

This TMU report register returns the last measured temperature at site. The site must be part of the list of enabled monitored sites as defined by TMR[MSITE].

5.4.3.16.3 Diagram



5.4.3.16.4 Fields

Field	Function
31 V	Valid measured temperature. 0 Not valid. Temperature out of sensor range or first measurement still pending. 1 Valid.
30-8 —	Reserved
7-0 TEMP	Last temperature reading at site when V=1.

5.4.3.17 TMU report average temperature site register a (TRATSR0 - TRATSR2)

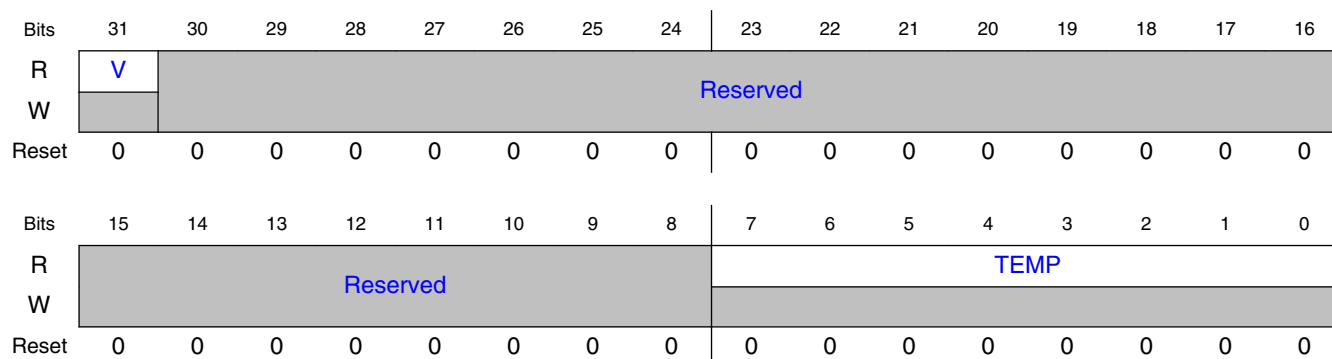
5.4.3.17.1 Offset

Register	Offset
TRATSR0	104h
TRATSR1	114h
TRATSR2	124h

5.4.3.17.2 Function

This TMU report register returns the average measured temperature at site. The site must be part of the list of enabled monitored sites as defined by TMR[MSITE].

5.4.3.17.3 Diagram



5.4.3.17.4 Fields

Field	Function
31 V	Valid measured temperature. 0 Not valid. Temperature out of sensor range or first measurement still pending. 1 Valid.
30-8 —	Reserved
7-0 TEMP	Average temperature reading at site when V=1.

5.4.3.18 TMU temperature range 0 control register (TTR0CR)

5.4.3.18.1 Offset

Register	Offset
TTR0CR	F10h

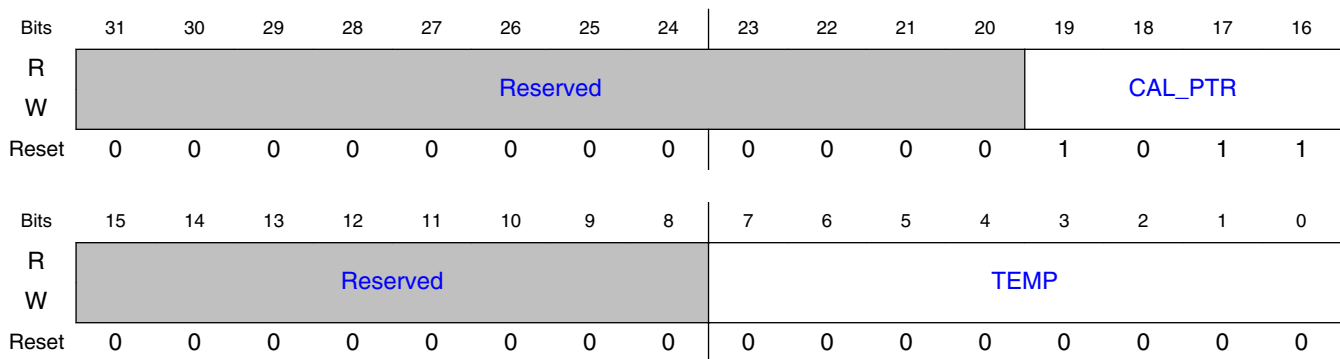
5.4.3.18.2 Function

The TMU temperature range control registers allow for defining the four temperature ranges that each sensor covers. A starting temperature range and number of calibration points, up to 16, is given. The temperature ranges may overlap to remove possible sample error, when searching for an accurate sensor reading. The default temperature calibration points are shown below.

Table 5-14. Default Temperature Configuration Points

Configuration Range	Starting Temperature (Celsius)	Temperature Configuration Points (Celsius)	Number of Points
0	0	0, 4, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44	12
1	38	38, 42, 46, 50, 54, 58, 62, 66, 70, 74, 78	11
2	72	72, 76, 80, 84, 88, 92, 96, 100, 104	9
3	97	97, 101, 105, 109, 113, 117, 121, 125	8

5.4.3.18.3 Diagram



5.4.3.18.4 Fields

Field	Function
31-20 —	Reserved
19-16 CAL_PTR	Temperature configuration points. 0000 Reserved 0001 2 points 0010 3 points ... 1111 16 points
15-8	Reserved

Table continues on the next page...

Field	Function
—	
7-0 TEMP	Starting temperature in Celsius for range.

5.4.3.19 TMU temperature range 1 control register (TTR1CR)

5.4.3.19.1 Offset

Register	Offset
TTR1CR	F14h

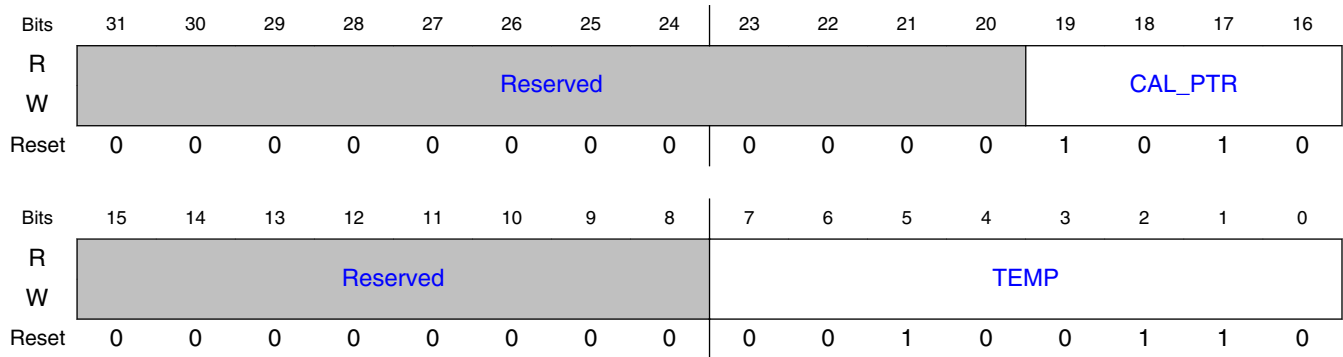
5.4.3.19.2 Function

The TMU temperature range control registers allow for defining the four temperature ranges that each sensor covers. A starting temperature range and number of calibration points, up to 16, is given. The temperature ranges may overlap to remove possible sample error, when searching for an accurate sensor reading. The default temperature calibration points are shown below.

Table 5-15. Default Temperature Configuration Points

Configuration Range	Starting Temperature (Celsius)	Temperature Configuration Points (Celsius)	Number of Points
0	0	0, 4, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44	12
1	38	38, 42, 46, 50, 54, 58, 62, 66, 70, 74, 78	11
2	72	72, 76, 80, 84, 88, 92, 96, 100, 104	9
3	97	97, 101, 105, 109, 113, 117, 121, 125	8

5.4.3.19.3 Diagram



5.4.3.19.4 Fields

Field	Function
31-20 —	Reserved
19-16 CAL_PTR	Temperature configuration points. 0000 Reserved 0001 2 points 0010 3 points ... 1111 16 points
15-8 —	Reserved
7-0 TEMP	Starting temperature in Celsius for range.

5.4.3.20 TMU temperature range 2 control register (TTR2CR)

5.4.3.20.1 Offset

Register	Offset
TTR2CR	F18h

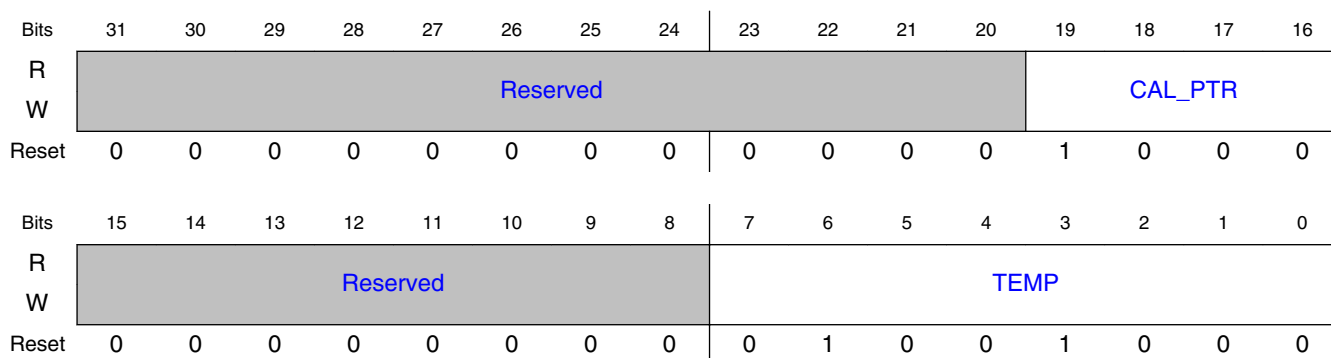
5.4.3.20.2 Function

The TMU temperature range control registers allow for defining the four temperature ranges that each sensor covers. A starting temperature range and number of calibration points, up to 16, is given. The temperature ranges may overlap to remove possible sample error, when searching for an accurate sensor reading. The default temperature calibration points are shown below.

Table 5-16. Default Temperature Configuration Points

Configuration Range	Starting Temperature (Celsius)	Temperature Configuration Points (Celsius)	Number of Points
0	0	0, 4, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44	12
1	38	38, 42, 46, 50, 54, 58, 62, 66, 70, 74, 78	11
2	72	72, 76, 80, 84, 88, 92, 96, 100, 104	9
3	97	97, 101, 105, 109, 113, 117, 121, 125	8

5.4.3.20.3 Diagram



5.4.3.20.4 Fields

Field	Function
31-20 —	Reserved
19-16 CAL_PTR	Temperature configuration points. 0000 Reserved 0001 2 points 0010 3 points ... 1111 16 points
15-8	Reserved

Table continues on the next page...

Field	Function
—	
7-0 TEMP	Starting temperature in Celsius for range.

5.4.3.21 TMU temperature range 3 control register (TTR3CR)

5.4.3.21.1 Offset

Register	Offset
TTR3CR	F1Ch

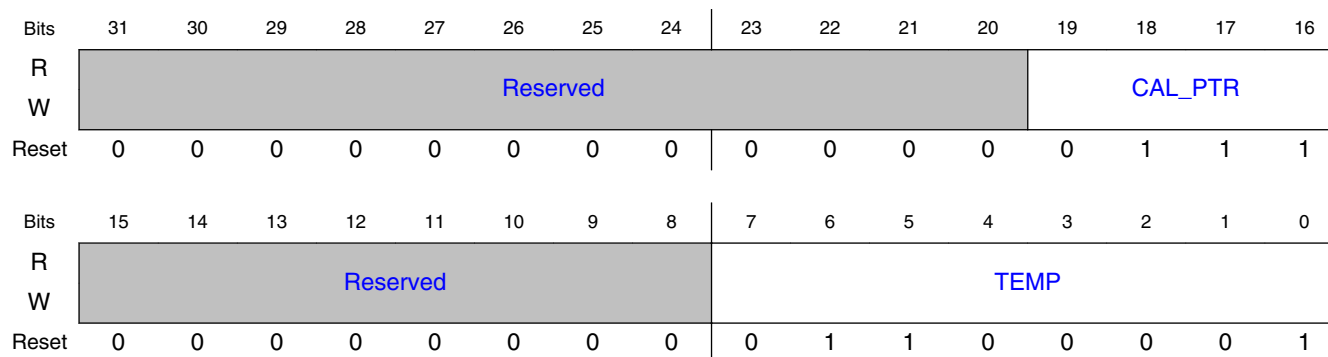
5.4.3.21.2 Function

The TMU temperature range control registers allow for defining the four temperature ranges that each sensor covers. A starting temperature range and number of calibration points, up to 16, is given. The temperature ranges may overlap to remove possible sample error, when searching for an accurate sensor reading. The default temperature calibration points are shown below.

Table 5-17. Default Temperature Configuration Points

Configuration Range	Starting Temperature (Celsius)	Temperature Configuration Points (Celsius)	Number of Points
0	0	0, 4, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44	12
1	38	38, 42, 46, 50, 54, 58, 62, 66, 70, 74, 78	11
2	72	72, 76, 80, 84, 88, 92, 96, 100, 104	9
3	97	97, 101, 105, 109, 113, 117, 121, 125	8

5.4.3.21.3 Diagram



5.4.3.21.4 Fields

Field	Function
31-20 —	Reserved
19-16 CAL_PTR	Temperature configuration points. 0000 Reserved 0001 2 points 0010 3 points ... 1111 16 points
15-8 —	Reserved
7-0 TEMP	Starting temperature in Celsius for range.

5.4.4 Functional Description

The following sections describe the functionality of the TMU in details.

5.4.4.1 Monitoring

Monitoring is the process of reading enabled temperature sensor sites on-chip at regular intervals and taking appropriate actions, such as alarming the user when the temperature exceeds a programmed temperature threshold.

During monitoring, all enabled temperature sensor sites are periodically measured for temperature starting with site 0 and ending with site 2. The interval at which the sensors are read is set in TMTMIR and should reflect the maximum interval time required to accurately capture temperature changes. If the measurement interval has not expired after the last active site has been read, the sensor logic enters low-power mode. If the interval has expired when the last active site is read, the measurement interval exceeded bit, TSR[MIE], is set and the next active site is read immediately. If the interval has been exceeded, user may opt to reduce number of sites monitored or increase the interval, if possible.

For each site the current and average temperature is logged. The average temperature is calculated based on the low-pass filter function in the mode register. If any of the set temperature threshold registers are exceeded, the corresponding interrupt detect bit is set in TIDR. Interrupts are enabled through the interrupt enable register, TIER.

Process for enabling monitoring mode:

1. Clear the interrupt detect register, TIDR.
2. Clear interrupt site capture register (TISCR) and interrupt critical site capture register (TICSCR).
3. Enable interrupt handling by setting the appropriate bits in TIER.
4. Set the temperature threshold registers TMHTITR, TMHTATR and TMHTACTR.
5. Set the monitoring interval register, TMTMIR.
6. Enable monitor mode by setting TMR[ME]=1. Sites to monitor are controlled by setting TMR[MSITE]. There should be at least one active site enabled. Set other mode control bits as needed.
7. If the monitoring interval is too short as indicated by TSR[MIE], the interval may need to be increased or number of sites reduced.

5.4.4.2 Reporting

The TMU can directly report the current and average temperature for a particular temperature sensor site during monitoring mode by reading one of the report registers per site. The report uses the last measurement done by the monitoring process and requires a site to be actively monitored for accurate reading. Reading a site which has last measured an invalid temperature outside the sensor range of 0-85 degrees Celsius, will have the valid bit cleared.

If monitoring is disabled, the last temperature measurement remains for the site(s) previously monitored and can still be read using the report registers knowing that the temperature reported is no longer accurate. This method can be used to capture the temperature at multiple sites in time, but does not allow for continuous monitoring.

Chapter 6

SNVS, Reset, Fuse, and Boot

6.1 System Boot

6.1.1 Overview

The boot process begins at the Power-On Reset (POR) where the hardware reset logic forces the Arm core to begin the execution starting from the on-chip boot ROM.

The boot ROM code uses the state of the internal register `BOOT_MODE[1:0]` as well as the state of various eFUSEs and/or GPIO settings to determine the boot flow behavior of the device.

The main features of the ROM include:

- Support for booting from various boot devices
- Serial downloader support (USB OTG)
- Device Configuration Data (DCD) and plugin
- Wake-up from the low-power modes

The boot ROM supports these boot devices:

- NAND flash
- SD/MMC

The boot ROM uses the state of the `BOOT_MODE` and eFUSEs to determine the boot device. For development purposes, the eFUSEs used to determine the boot device may be overridden using the GPIO pin inputs.

The boot ROM code also allows to download the programs to be run on the device. The example is a provisioning program that can make further use of the serial connection to provide a boot device with a new image. Typically, the provisioning program is downloaded to the internal RAM and allows to program the boot devices, such as the SD/MMC flash. The ROM serial downloader uses a high-speed USB in a non-stream mode connection.

The Device Configuration Data (DCD) feature allows the boot ROM code to obtain the SOC configuration data from an external program image residing on the boot device. As an example, the DCD can be used to program the DDR controller for optimal settings, improving the boot performance. The DCD is restricted to the memory areas and peripheral addresses that are considered essential for the boot purposes (see [Write data command](#)).

A key feature of the boot ROM is the ability to perform a secure boot, also known as a High-Assurance Boot (HAB). This is supported by the HAB security library which is a subcomponent of the ROM code. The HAB uses a combination of hardware and software together with the Public Key Infrastructure (PKI) protocol to protect the system from executing unauthorized programs. Before the HAB allows the user image to execute, the image must be signed. The signing process is done during the image build process by the private key holder and the signatures are then included as a part of the final program image. If configured to do so, the ROM verifies the signatures using the public keys included in the program image. A secure boot with HAB can be performed on all boot devices supported on the chip in addition to the serial downloader. The HAB library in the boot ROM also provides the API functions, allowing the additional boot chain components (bootloaders) to extend the secure boot chain. The out-of-fab setting for the SEC_CONFIG is the open configuration, in which the ROM/HAB performs the image authentication, but all authentication errors are ignored and the image is still allowed to execute.

6.1.2 Boot modes

During boot, the core's behavior is defined by the boot mode pin settings, as described in [Boot mode pin settings](#).

6.1.2.1 Boot mode pin settings

The device has four boot modes (one is reserved for NXP use). The boot mode is selected based on the binary value stored in the internal BOOT_MODE register.

The BOOT_MODE is initialized by sampling the BOOT_MODE0 and BOOT_MODE1 inputs on the rising edge of the POR_B. After these inputs are sampled, their subsequent state does not affect the contents of the BOOT_MODE internal register. The state of the

internal BOOT_MODE register may be read from the BMOD[1:0] field of the SRC Boot Mode Register (SRC_SBMR2). The available boot modes are: Boot From Fuses, serial boot via USB, and Internal Boot. See this table for settings:

Table 6-1. Boot MODE pin settings

BOOT_MODE[1:0]	Boot Type
00	Boot From Fuses
01	Serial Downloader
10	Internal Boot
11	Reserved

6.1.2.2 High-level boot sequence

The figure found here show the high-level boot ROM code flow.

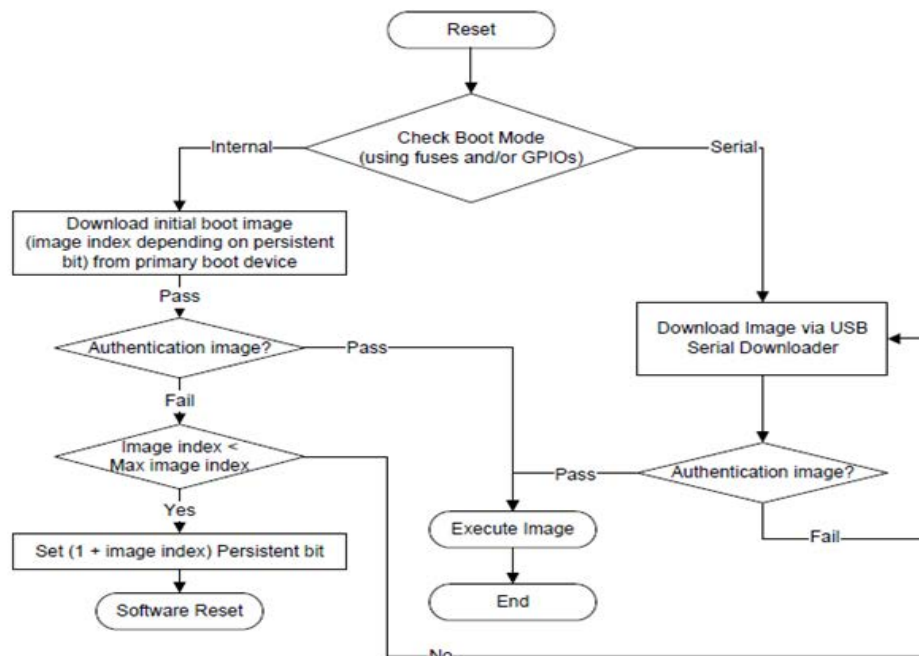


Figure 6-1. Boot flow

6.1.2.3 Boot From Fuses mode (BOOT_MODE[1:0] = 00b)

A value of 00b in the BOOT_MODE[1:0] register selects the Boot From Fuses mode.

This mode is similar to the Internal Boot mode described in [Internal Boot mode](#) (`BOOT_MODE[1:0] = 0b10`) with one difference. In this mode, the GPIO boot override pins are ignored. The boot ROM code uses the boot eFUSE settings only. This mode also supports a secure boot using HAB.

If set to Boot From Fuses, the boot flow is controlled by the `BT_FUSE_SEL` eFUSE value. If `BT_FUSE_SEL = 0`, indicating that the boot device (for example, flash, SD/MMC) was not programmed yet, the boot flow jumps directly to the Serial Downloader. If `BT_FUSE_SEL = 1`, the normal boot flow is followed, where the ROM attempts to boot from the selected boot device.

The first time a board is used, the default eFUSES may be configured incorrectly for the hardware on the platform. In such case, the Boot ROM code may try to boot from a device that does not exist. This may cause an electrical/logic violation on some pads. Using the Boot From Fuses mode addresses this problem.

Setting the `BT_FUSE_SEL=0` forces the ROM code to jump directly to the Serial Downloader. This allows a bootloader to be downloaded which can then provision the boot device with a program image and blow the `BT_FUSE_SEL` and the other boot configuration eFUSES. After the reset, the boot ROM code determines that the `BT_FUSE_SEL` is blown (`BT_FUSE_SEL = 1`) and the ROM code performs an internal boot according to the new eFUSE settings. This allows the user to set `BOOT_MODE[1:0]=00b` on a production device and burn the fuses on the same device (by forcing the entry to the Serial Downloader), without changing the value of the `BOOT_MODE[1:0]` or the pullups/pulldowns on the `BOOT_MODE` pins.

6.1.2.4 Serial Downloader

The Serial Downloader provides a means to download a program image to the chip over the USB serial connection.

In this mode, the ROM programs the WDOG1 for a time-out specified by the fuse WDOG Time-out Select (See fusemap for details) if the `WDOG_ENABLE` eFuse is 1 and continuously polls for the USB connection. If no activity is found on the USB OTG1 and the watchdog timer expires, the Arm core is reset.

NOTE

After the downloaded image is loaded, it is responsible for managing the watchdog resets properly.

This figure shows the USB boot flow:

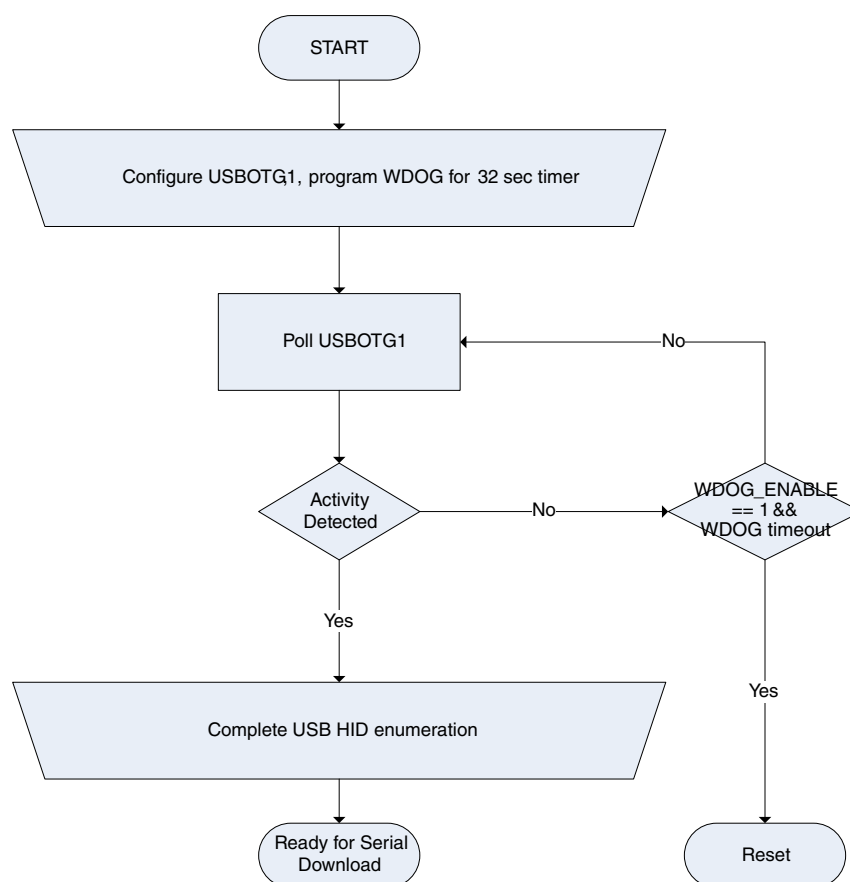


Figure 6-2. Serial Downloader boot flow

NOTE

Before going into USB serial mode, Boot ROM detect SD/MMC card on USDHC2 port. If a card is inserted, ROM will try to boot from it. This is the so-called Manufacture SD/MMC boot. This feature can be disabled by blowing fuse “Disable SD/MMC Manufacture Mode”. See [SD/MMC manufacture mode](#) for details.

6.1.2.5 Internal Boot mode (BOOT_MODE[1:0] = 0b10)

A value of 0b10 in the BOOT_MODE[1:0] register selects the Internal Boot mode. In this mode, the processor continues to execute the boot code from the internal boot ROM.

The boot code performs the hardware initialization, loads the program image from the chosen boot device, performs the image validation using the HAB library (see [Boot security settings](#)), and then jumps to an address derived from the program image. If an error occurs during the internal boot, the boot code jumps to the Serial Downloader (see [Serial Downloader](#)). A secure boot using the HAB is possible in all the three boot modes.

When set to the Internal Boot, the boot flow may be controlled by a combination of eFUSE settings with an option of overriding the fuse settings using the General Purpose I/O (GPIO) pins. The GPIO Boot Select FUSE (BT_FUSE_SEL) determines whether the ROM uses the GPIO pins for a selected number of configuration parameters or eFUSES in this mode.

- If BT_FUSE_SEL = 1, all boot options are controlled by the eFUSES described in [Table 6-2](#).
- If BT_FUSE_SEL = 0, the specific boot configuration parameters may be set using the GPIO pins rather than eFUSES. The fuses that can be overridden when in this mode are indicated in the GPIO column of [Table 6-2](#). [Table 6-3](#) provides the details of the GPIO pins.

The use of the GPIO overrides is intended for development since these pads are used for other purposes in the deployed products. NXP recommends controlling the boot configuration by the eFUSES in the deployed products and reserving the use of the GPIO mode for the development and testing purposes only.

6.1.2.6 Boot security settings

The internal boot modes use one of three security configurations.

- Closed: This level is intended for use with shipping-secure products. All HAB functions are executed and the security hardware is initialized (the Security Controller or SNVS enters the Secure state), the DCD is processed if present, and the program image is authenticated by the HAB before its execution. All detected errors are logged, and the boot flow is aborted with the control being passed to the serial downloader. At this level, the execution does not leave the internal ROM unless the target executable image is authenticated.
- Open: This level is intended for use in non-secure products or during the development phases of a secure product. All HAB functions are executed as for a closed device. The security hardware is initialized (except for the SNVS which is left in the Non-Secure state), the DCD is processed if present, and the program image is authenticated by the HAB before its execution. All detected errors are logged, but have no influence on the boot flow which continues as if the errors did not occur. This configuration is useful for a secure product development because the program

image runs even if the authentication data is missing or incorrect, and the error log can be examined to determine the cause of the authentication failure.

- Field Return: This level is intended for the parts returned from the shipped products.

NOTE

The DIR_BT_DIS eFuse must be programmed prior to shipping a device in a security enabled configuration. If the this eFuse is not blown, the system is not secure.

6.1.3 Device configuration

This section describes the external inputs that control the behavior of the Boot ROM code.

This includes the boot device selection (SD, MMC, and so on), boot device configuration (SD bus width, speed, and so on), and other. In general, the source for this configuration comes from the eFUSES embedded inside the chip. However, certain configuration parameters can be sourced from the GPIO pins, allowing further flexibility during the development process.

6.1.3.1 Boot eFUSE descriptions

This table is a comprehensive list of the configuration parameters that the ROM uses.

Table 6-2. Boot eFUSE descriptions

Fuse Address	Configuration	Definition	GPIO ¹	Shipped value	Settings ²
BT_FUSE_SEL 0x470[28]	OEM	In the Internal Boot mode BOOT_MODE[1:0] = 10, the BT_FUSE_SEL fuse determines whether the boot settings indicated by a Yes in the GPIO column are controlled by the GPIO pins or the eFUSE settings in the On-Chip OTP Controller (OCOTP). In the Boot From Fuse mode BOOT_MODE[1:0]	NA	0	If BOOT_MODE[1:0] = 0b10: 0—The bits of the SBMR are overridden by the GPIO pins. 1—The specific bits of the SBMR are controlled by the eFUSE settings. If BOOT_MODE[1:0] = 0b00 0—The BOOT configuration eFuses are not programmed yet.

Table continues on the next page...

**Table 6-2. Boot eFUSE descriptions
(continued)**

		= 00, the BT_FUSE_SEL fuse indicates whether the bit configuration eFuses are programmed.			The boot flow jumps to the serial downloader. 1—The BOOT configuration eFuses are programmed. The regular boot flow is performed.
FIELD_RETURN	OEM	Enables the NXP reserved modes.	NA	0	0—The NXP reserved modes are enabled/disabled based on the DIR_BT_DIS value. 1—The NXP reserved modes are enabled.
BT_DCACHE_DISABLE	OEM	The D Cache disable bit used by the boot ROM for fast HAB processing.	No	0	0—D Cache is enabled by the ROM during the boot. 1—D Cache is disabled by the ROM during the boot.
L1 I-Cache DISABLE 0x480[12]	OEM	L1 I Cache disable bit used by the boot during the entire execution.	No	0	0—L1 I Cache is enabled by the ROM during the boot. 1—L1 I Cache is disabled by the ROM during the boot.
BT_FREQ 0x480[9]	OEM	Boot frequency selection	No	0	0—Arm—792 MHz, DDR—396 MHz, AXI— 396 MHz 1—Arm—396 MHz, DDR—307 MHz, AXI— 307 MHz. The Arm/DDR/AXI frequency may be lower if the LP_BOOT is blown.
LPB_BOOT 0x480[15:14]	OEM	USB Low-Power Boot	No	00	0x—LPB Disable 10—Divide by 2 11—Divide by 4

Table continues on the next page...

**Table 6-2. Boot eFUSE descriptions
(continued)**

BT_LPB_POLARITY 0x480[13]	OEM	USB Low-Power Boot GPIO polarity	No	0	0—Low on the GPIO pad indicates the lowpower condition. 1—High on the GPIO pad indicates the low-power condition.
WDOG_ENABLE 0x480[10]	OEM	Watchdog reset counter enable	No	0	0—The watchdog reset counter is disabled during the serial downloader. 1—The watchdog reset counter is enabled during the serial downloader.
MMC_DLL_DLY[6: 0] 0x490[14:8]	OEM	uSDHC Delay Line settings	No	0000000	uSDHC Delay Line settings
SRK_REVOKE[2:0] 0x670[2:0]	OEM	SRK revocation mask	No	000	SRK revocation mask
DISABLE_SDMMC _MFG 0x490[23]	OEM	Disable the SDMMC manufacture mode	Yes	0	0—enable the SD/MMC MFG mode 1—disable the SD/MMC MFG mode
USDHC_PAD_SET TINGS 0x490[31:24] NAND_PAD_SETT INGS 0x4A0[31:24]	OEM	Override values for the SD/MMC and NAND boot modes	No	00000000	Override the following IO PAD settings: [1:0] Driver Strength [2] Slew Rate [3] Hysteresis [4] Pull/Keeper select [6:5] Pull up/down config [7] Reserved.
USDHC_OVERRID E_PAD_SETTING S 0x490[17]	OEM	Overrides the HYS bit for the SD pads	No	0	Override the IO PAD setting HYS to 1 for the SD pads.
NAND_OVERRID E_PAD_SETTINGS 0x4B0[7]	OEM	Overrides the HYS bit for the SD pads	No	0	Override the IO PAD setting HYS to 1 for the SD pads.

Table continues on the next page...

**Table 6-2. Boot eFUSE descriptions
(continued)**

eMMC_4.4_RESET _TO_PRE- IDLE_STATE 0x490[21]	OEM	ROM resets the boot device in the pre-idle state using the eMMC 4.4 feature, CMD0 with the argument value 0xf0f0f0f0.	No	0	Applicable for booting from the eMMC 4.4 spec or greater version devices. The fuse must not be blown for the eMMC 4.3 or lesser spec version devices.
---	-----	---	----	---	---

1. This setting can be overridden by the GPIO settings when the BT_FUSE_SEL fuse is intact. See [GPIO Boot Overrides](#) for the corresponding GPIO pin.
2. 0 = intact fuse and 1= blown fuse

6.1.3.2 GPIO boot overrides

This table provides a list of the GPIO boot overrides:

Table 6-3. GPIO override contact assignments

Package pin	Direction on reset	eFuse
BOOT_MODE1	Input	Boot mode selection
BOOT_MODE0	Input	
SAI1_RXD0	Input	BOOT_CFG[0]
SAI1_RXD1	Input	BOOT_CFG[1]
SAI1_RXD2	Input	BOOT_CFG[2]
SAI1_RXD3	Input	BOOT_CFG[3]
SAI1_RXD4	Input	BOOT_CFG[4]
SAI1_RXD5	Input	BOOT_CFG[5]
SAI1_RXD6	Input	BOOT_CFG[6]
SAI1_RXD7	Input	BOOT_CFG[7]
SAI1_TXD0	Input	BOOT_CFG[8]
SAI1_TXD1	Input	BOOT_CFG[9]
SAI1_TXD2	Input	BOOT_CFG[10]
SAI1_TXD3	Input	BOOT_CFG[11]
SAI1_TXD4	Input	BOOT_CFG[12]
SAI1_TXD5	Input	BOOT_CFG[13]
SAI1_TXD6	Input	BOOT_CFG[14]
SAI1_TXD7	Input	BOOT_CFG[15]

The input pins provided are sampled at boot, and can be used to override the corresponding eFUSE values, depending on the setting of the BT_FUSE_SEL fuse.

6.1.3.3 Device Configuration Data (DCD)

The DCD is the configuration information contained in the program image (external to the ROM) that the ROM interprets to configure various on-chip peripherals. See [Device Configuration Data \(DCD\)](#) for more details on DCD.

6.1.4 Device initialization

This section describes the details of the ROM and provides the initialization details.

This includes details on:

- The ROM memory map
- The RAM memory map
- On-chip blocks that the ROM must use or change the POR register default values
- Clock initialization
- Enabling the MMU/L2 cache
- Exception handling and interrupt handling

6.1.4.1 Internal ROM/RAM memory map

These figures show the iROM memory map:

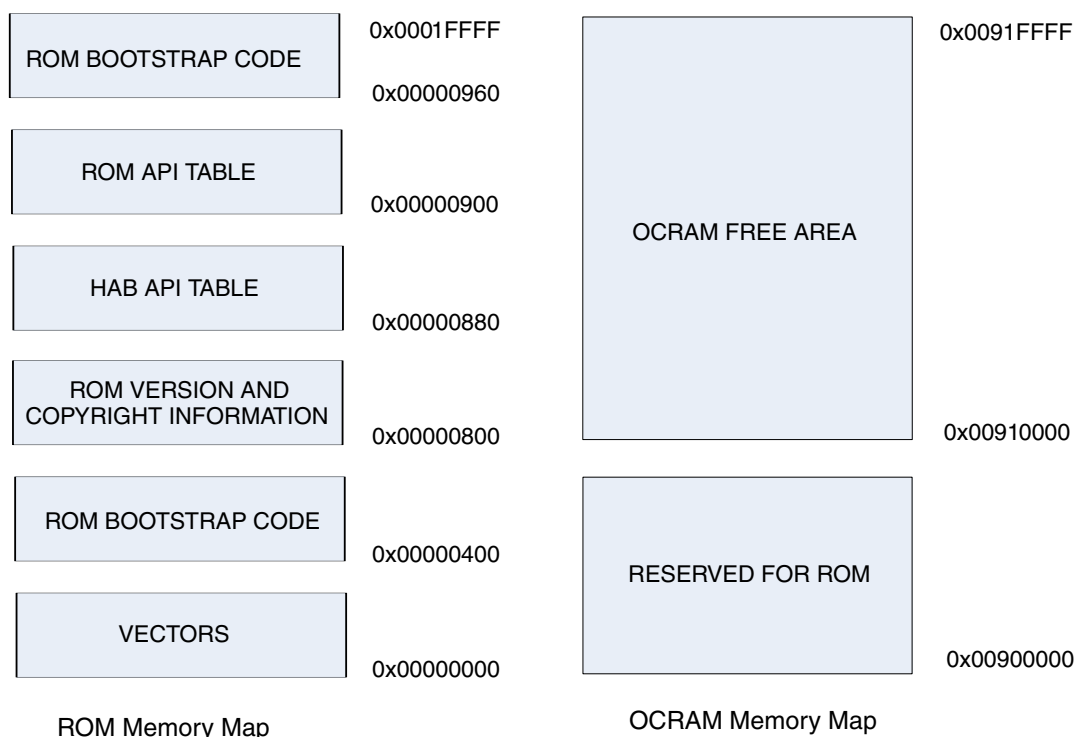


Figure 6-3. Internal ROM and RAM memory map

NOTE

If no ROM/HAB APIs are being used, the entire OCRAM region can be used freely after the boot.

6.1.4.2 Boot block activation

The boot ROM affects a number of different hardware blocks which are activated and play a vital role in the boot flow.

The ROM configures and uses the following blocks (listed in an alphabetical order) during the boot process. Note that the blocks actually used depend on the boot mode and the boot device selection:

- APBH—the DMA engine to drive the GPMI module
- BCH—62-bit error correction hardware engine with the AXI bus master and a private connection to the GPMI
- CCM—Clock Control Module
- GPMI—NAND controller pin interface
- OCOTP_CTRL—On-Chip OTP Controller; the OCOTP contains the eFUSES
- IOMUXC—I/O Multiplexer Control which allows the GPIO use to override the eFUSE boot settings;

- IOMUXC GPR—I/O Multiplexer Control General-Purpose Registers
- CAAM—Cryptographic Acceleration and Assurance Module
- SNVS—Secure Non-Volatile Storage
- SRC—System Reset Controller
- USB—used for the serial download of a boot device provisioning program
- USDHC—Ultra-Secure Digital Host Controller
- WDOG-1—Watchdog timer

6.1.4.3 Clocks at boot time

The table below show the various clocks and their sources used by the ROM.

After the reset, each Arm core has access to all peripherals. The ROM code disables the clocks listed in the following table, except for the boot devices listed in the second column.

Table 6-4. PLL setting by ROM

PLL name	Frequency	Comment
ARM_PLL	1000 MHz	
SYS_PLL1	800 MHz	
SYS_PLL2	1000 MHz	
SYS_PLL3	1000 MHz	

NOTE

All other PLLs are in the default status.

Table 6-5. Clock root setting by ROM

Clock Name	Frequency (MHz)	Source	Enable
ARM_A53_ROOT	1000	arm_pll_clk	Yes
ARM_M4_CLK_ROOT	200	system_pll2_200m_clk	
AHB_CLK_ROOT	133	system_pll1_133m_clk	Yes
MAIN_AXI_CLK_ROOT	333	system_pll2_333m_clk	Yes
VPU_A53_CLK_ROOT	800	arm_pll_clk	No
DRAM_ALT_CLK_ROOT	800	system_pll1_800m_clk	yes
NAND_CLK_ROOT	500	system_pll2_500m_clk	Enabled by driver
NAND_USDHC_BUS_CLK_ROOT	266	system_pll1_266m_clk	Enabled by driver
USB_BUS_CLK_ROOT		system_pll2_500m_clk	Enabled by driver
NOC_CLK_ROOT	400	system_pll1_800m_clk	Yes
USDHC1_CLK_ROOT	200	system_pll1_400m_clk	Enabled by driver
USDHC2_CLK_ROOT	200	system_pll1_400m_clk	Enabled by driver

Table continues on the next page...

Table 6-5. Clock root setting by ROM (continued)

Clock Name	Frequency (MHz)	Source	Enable
USB_PHY_REF_CLK_ROOT		system_pll1_100m_clk	Enabled by driver
ECSPi1_CLK_ROOT	200	system_pll2_200m_clk	No
ECSPi2_CLK_ROOT	200	system_pll2_200m_clk	No
ECSPi3_CLK_ROOT	200	system_pll2_200m_clk	No
WRCLK_CLK_ROOT		system_pll1_40m_clk	No

NOTE

All other clock roots are in the default status.

Table 6-6. NAND_CLK_ROOT setting

NAND data rate	NAND_CLK_ROOT source	Frequency
Async/Legacy NAND	system_pll1_400m_clk	25 MHz
Sync 40M	system_pll1_400m_clk	40 MHz
Toggle/Sync 66M	system_pll1_400m_clk	66 MHz
Toggle 80M	system_pll1_400m_clk	80 MHz
Sync 100M	system_pll1_400m_clk	100 MHz
Toggle/Sync 133M	system_pll1_400m_clk	133 MHz
Sync 160M	system_pll1_400m_clk	133 MHz
Toggle/Sync 200M	system_pll1_400m_clk	200 MHz

NOTE

The NAND_CLK_ROOT source depends on the NAND data rate.

The ROM code disables the clocks listed in the following table, except for the boot devices listed in the "Enabled for boot device" column below.

Table 6-7. CCGR setting by ROM

CCGR Register	LPCG Enable	Enabled for boot device
CCM_CCGR0	Dvfs	
CCM_CCGR1	Anamix	
CCM_CCGR2	Cpu	
CCM_CCGR3	Csu	Security related
CCM_CCGR4	debug	
CCM_CCGR5	Dram1	
CCM_CCGR6	reserved	
CCM_CCGR7	Ecspi1	
CCM_CCGR8	Ecspi2	

Table continues on the next page...

Table 6-7. CCGR setting by ROM (continued)

CCGR Register	LPCG Enable	Enabled for boot device
CCM_CCGR9	Ecspi3	
CCM_CCGR10	Enet1	
CCM_CCGR11	Gpio1	Never gate GPIO clock. uSDHC and test mode use GPIO.
CCM_CCGR12	Gpio2	
CCM_CCGR13	Gpio3	
CCM_CCGR14	Gpio4	
CCM_CCGR15	Gpio5	
CCM_CCGR16	Gpt1	Used by ROM as tick. Keep no changed so it is 25MHz.
CCM_CCGR17	Gpt2	Can be used in DCD. Keep no changed so it is 25MHz.
CCM_CCGR18	Gpt3	
CCM_CCGR19	Gpt4	
CCM_CCGR20	Gpt5	
CCM_CCGR21	Gpt6	
CCM_CCGR22	Hs	
CCM_CCGR23	I2c1	No I2C to be enabled
CCM_CCGR24	I2c2	
CCM_CCGR25	I2c3	
CCM_CCGR26	I2c4	
CCM_CCGR27	Iomux	
CCM_CCGR28	Iomux1	
CCM_CCGR29	Iomux2	
CCM_CCGR30	Iomux3	
CCM_CCGR31	Iomux4	
CCM_CCGR32	M4	
CCM_CCGR33	Mu	
CCM_CCGR34	Ocotp	
CCM_CCGR35	Ocram	
CCM_CCGR36	Ocram_s	
CCM_CCGR37	Pcie	
CCM_CCGR38	Perfmon1	
CCM_CCGR39	Perfmon2	
CCM_CCGR40	Pwm1	
CCM_CCGR41	Pwm2	
CCM_CCGR42	Pwm3	
CCM_CCGR43	Pwm4	
CCM_CCGR44	Qos	
CCM_CCGR45	Dismix	

Table continues on the next page...

Table 6-7. CCGR setting by ROM (continued)

CCGR Register	LPCG Enable	Enabled for boot device
CCM_CCGR46	Megamix	
CCM_CCGR47		
CCM_CCGR48	Rawnand	Will be gated on if boot from NAND is issued.
CCM_CCGR49	Rdc	Never used by ROM. Gate it off.
CCM_CCGR50	Rom	
CCM_CCGR51	Sai1	
CCM_CCGR52	Sai2	
CCM_CCGR53	Sai3	
CCM_CCGR54	Sai4	
CCM_CCGR55	Sai5	
CCM_CCGR56	Sai6	
CCM_CCGR57	Sctr	System counter. Do not gate off.
CCM_CCGR58	Sdma1	Not used by ROM. Gate it off.
CCM_CCGR59	Sdma2	
CCM_CCGR60	Sec_debug	
CCM_CCGR61	Sema1	Not used by ROM. Gate it off.
CCM_CCGR62	Sema2	
CCM_CCGR63	Sim_display	Bus clock. Do not gate off.
CCM_CCGR64	Sim_enet	Bus clock. Do not gate off.
CCM_CCGR65	Sim_m	Bus clock. Do not gate off.
CCM_CCGR66	Sim_main	Bus clock. Do not gate off.
CCM_CCGR67	Sim_s	Bus clock. Do not gate off.
CCM_CCGR68	Sim_wakeup	Bus clock. Do not gate off.
CCM_CCGR69	Sim_usb	Bus clock. Do not gate off.
CCM_CCGR70	Sim_vpu	Bus clock. Do not gate off.
CCM_CCGR71	Snvs	Secure Non-Volatile Storage
CCM_CCGR72	Trace	
CCM_CCGR73	Uart1	
CCM_CCGR74	Uart2	
CCM_CCGR75	Uart3	
CCM_CCGR76	Uart4	
CCM_CCGR77	Usb_ctrl1	Used by ROM USB driver
CCM_CCGR78	Usb_ctrl2	Not used by ROM. Gate it off.
CCM_CCGR79	Usb_phy1	Used by ROM USB driver. Keep it gated on so the PHY can work earlier.
CCM_CCGR80	Usb_phy2	Not used by ROM. Keep it on, otherwise the PHY PLL's reference clock (24MHz) will be gated off when PLL is still enabled, and the PHY may not work normally.

Table continues on the next page...

Table 6-7. CCGR setting by ROM (continued)

CCGR Register	LPCG Enable	Enabled for boot device
CCM_CCGR81	Usdhc1	Used by USDHC driver
CCM_CCGR82	Usdhc2	Used by USDHC driver
CCM_CCGR83	Wdog1	WDOG1 used by ROM.
CCM_CCGR84	Wdog2	
CCM_CCGR85	Wdog3	
CCM_CCGR86	Va53	
CCM_CCGR87	Gpu	
CCM_CCGR88	Hevc	
CCM_CCGR89	Avc	
CCM_CCGR90	Vp9	
CCM_CCGR91	Hevc_inter	
CCM_CCGR92	Gic	Leave on for Software.
CCM_CCGR93	Display	
CCM_CCGR94	Hdmi	
CCM_CCGR95	Hdmi_phy	
CCM_CCGR96	Xtal	
CCM_CCGR97	Pll	
CCM_CCGR98	Tsensor	
CCM_CCGR99	Vpu_dec	
CCM_CCGR100	Pcie2	
CCM_CCGR101	Mipi_csi1	
CCM_CCGR102	Mipi_csi2	

6.1.4.4 Exception handling

The exception vectors located at the start of the ROM are used to map all the Arm exceptions (except the reset exception) to a duplicate exception vector table in the internal RAM.

During the boot phase of CPU0, the RAM vectors point to the serial downloader in the ROM.

After the boot, the program image can overwrite the vectors as required. The code shown below is used to map the ROM exception vector table to the duplicate exception vector table in the RAM.

Mapping ROM Exception Vector Table

6.1.4.5 Interrupt handling during boot

No special interrupt-handling routines are required during the boot process. The interrupts are disabled during the boot ROM execution and may be enabled in a later boot stage.

6.1.4.6 Persistent bits

Some modes of the boot ROM require the registers that keep their values after a warm reset. The SRC General-Purpose registers are used for this purpose.

See this table for persistent bits list and description:

Table 6-8. Persistent bits

Bit name	Bit location	Description
PERSIST_BLOCK_REWRITE	SRC_GPR10[29]	This bit is used as a warning. It identifies that there are errors in the NAND blocks that hold the application image. See NAND flash for more details.
PERSISTENT_ENTRY1[31:0]	SRC_GPR3[31:0]	Holds the entry function for the CPU1 to wake up from the low-power mode.
PERSISTENT_ARG1[31:0]	SRC_GPR4[31:0]	Holds the argument of the entry function for the CPU1 to wake up from the low-power mode.

6.1.5 Boot devices (internal boot)

The chip supports these boot flash devices:

- Raw NAND (MLC and SLC), and Toggle-mode NAND flash through GPMI-2 interface, located at CS0. Page sizes of 2 KB, 4 KB, and 8 KB. The bus widths of 8-bit with 2 through 62-bit BCH hardware ECC (Error Correction) are supported.
- SD/MMC/eSD/SDXC/eMMC4.4 via USDHC interface, supporting high capacity cards.

The selection of the external boot device type is controlled by the BOOT_CFG eFUSEs. See this table for more details:

Table 6-9. Boot device selection

BOOT_CFG[14:12]	Boot device
001	SD/eSD
010	MMC/eMMC
011	NAND

6.1.5.1 NAND flash

The boot ROM supports a number of MLC/SLC NAND flash devices from different vendors and LBA NAND flash devices. The Error Correction and Control (ECC) subblock (BCH) is used to detect the errors.

6.1.5.1.1 NAND eFUSE configuration

The boot ROM determines the configuration of the external NAND flash by parameters, either provided by the eFUSE, or sampled on the GPIO pins during boot. See [Table 6-10](#) for parameters details.

NOTE

BOOT_CFGx sampled on the GPIO pins depends on the BT_FUSE_SEL setting. See [Boot Fusemap](#) for details.

6.1.5.1.2 NAND flash boot flow and Boot Control Blocks (BCB)

There are two BCB data structures:

- FCB
- DBBT

As a part of the NAND media initialization, the ROM driver uses safe NAND timings to search for the Firmware Configuration Block (FCB) that contains the optimum NAND timings, the page address of the Discovered Bad Block Table (DBBT) Search Area, and the start page address of the primary and secondary firmware.

The hardware ECC level to use is embedded inside the FCB block. The FCB data structure is also protected using the ECC. The driver reads raw 2112 bytes of the first sector and runs through the software ECC engine that determines whether the FCB data is valid or not.

If the FCB is found, the optimum NAND timings are loaded for further reads. If the ECC fails, or the fingerprints do not match, the Block Search state machine increments the page number to the Search Stride number of pages to read for the next BCB until the SearchCount pages have been read.

If the search fails to find a valid FCB, the NAND driver responds with an error and the boot ROM enters the serial download mode.

The FCB contains the page address of the DBBT Search Area, and the page address for primary and secondary boot images. The DBBT is searched in the DBBT Search Area, just like the FCB is searched. After the FCB is read, the DBBT is loaded, and the primary or secondary boot image is loaded using the starting page address from the FCB.

This figure shows the state diagram of the FCB search:

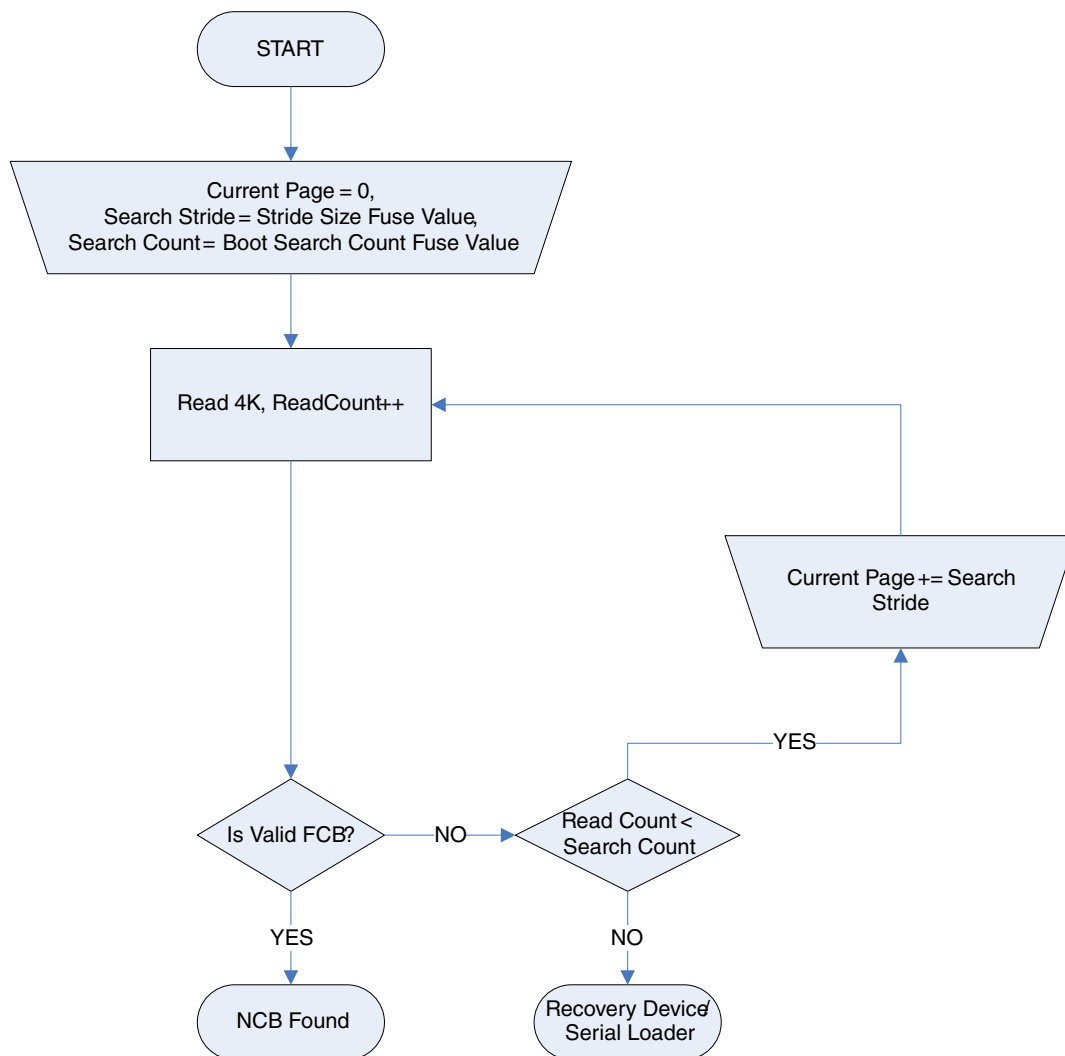


Figure 6-4. FCB search flow

When the FCB is found, the boot ROM searches for the Discovered Bad Blocks Table (DBBT). If the DBBT Search Area is 0 in the FCB, the ROM assumes that there are no bad blocks on the NAND device boot area. See this figure for the DBBT search flow:

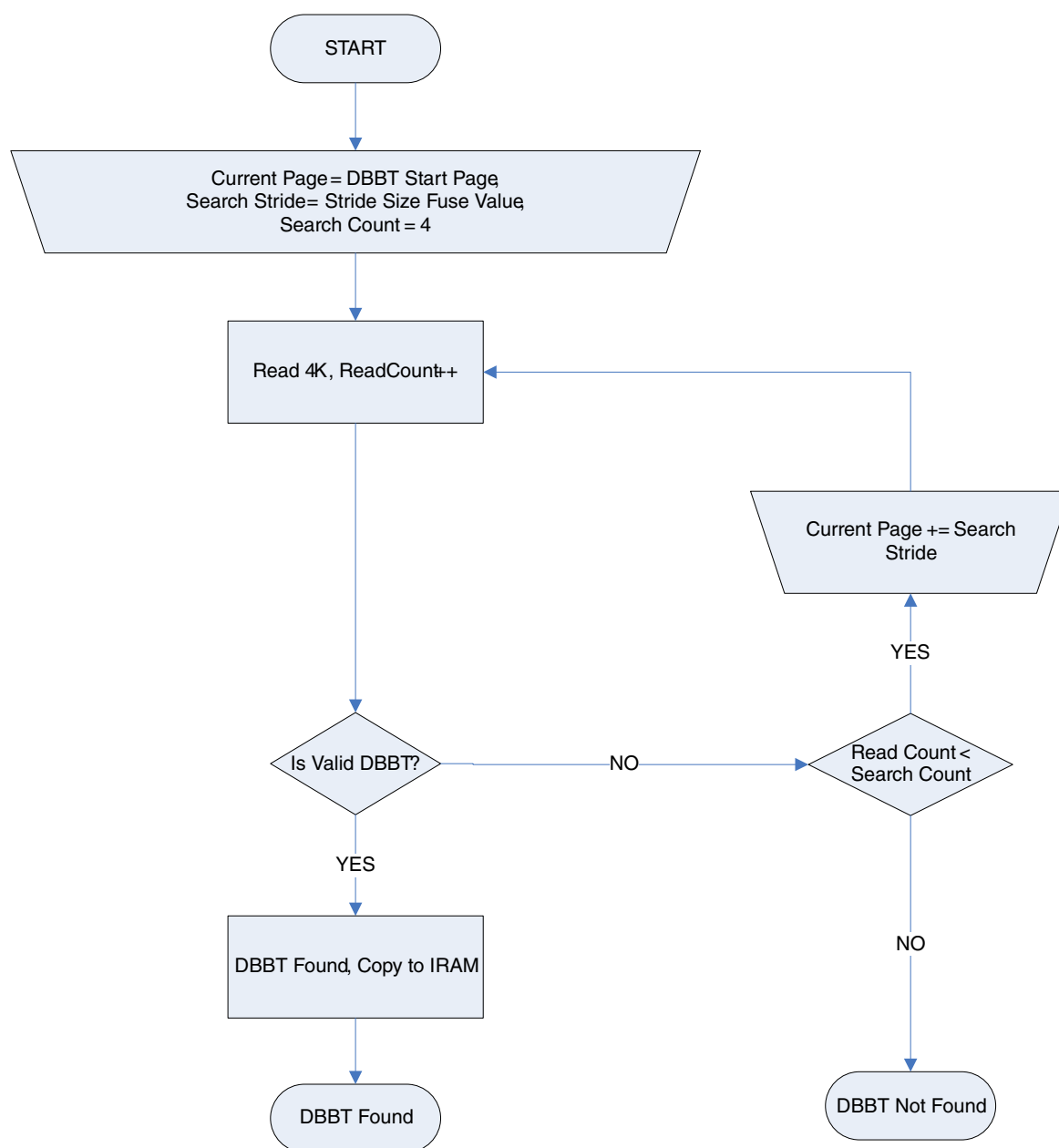


Figure 6-5. DBBT search flow

The BCB search and load function also monitors the ECC correction threshold and sets the `PERSIST_BLOCK_REWRITE` persistent bit if the threshold exceeds the maximum ECC correction ability.

If there is a page with a number of errors higher than ECC can correct during the primary image read, the boot ROM turns on the `PERSIST_SECONDARY_BOOT` bit and performs the software reset (After the software reset, the secondary image is used).

If there is a page with number of errors higher than ECC can correct during secondary image read, the boot ROM goes to the serial loader.

6.1.5.1.3 Firmware configuration block

The FCB is the first sector in the first good block. The FCB must be present at each search stride of the search area.

The search area contains copies of the FCB at each stride distance, so, in case the first NAND block becomes corrupted, the ROM finds its copy in the next NAND block. The search area must span over at least two NAND blocks. The location information for the DBBT search area, FW1, and FW2 are all specified in the FCB. This table shows the flash control block structure:

Table 6-10. Flash control block structure

Name	Start byte	Size in bytes	Description
Reserved	0	4	Reserved for Fingerprint #1(Checksum)
FingerPrint	4	4	32-bit word with a value of 0x20424346, in ascii "FCB"
Version	8	4	32-bit version number; this version of FCB is 0x00000001
m_NANDTiming	12	8	8 B of data for eight NAND timing parameters from the NAND datasheet. The eight parameters are: m_NandTiming[0]=data_setup, m_NandTiming[1]=data_hold, m_NandTiming[2]=address_setup, m_NandTiming[3]=dsample_time, m_NandTiming[4]=nand_timing_state, m_NandTiming[5]=REA, m_NandTiming[6]=RLOH, m_NandTiming[7]=RHOH. The ROM only uses the first four parameters, but the FCB provides space for other four parameters to be used by the bootloader or other applications.
PageDataSize	20	4	The number of bytes of data in a page. Typically, this is 2048 bytes for 2112 bytes page size or 4096 bytes for 4314/4224 bytes page size or 8192 for 8568 bytes page size.
TotalPageSize	24	4	The total number of bytes in a page. Typically, 2112 for 2-KB page or 4224 or 4314 for 4-KB page or 8568 for 8-KB page.
SectorsPerBlock	28	4	The number of pages per block. Typically 64 or 128 or depending on the NAND device type.

Table continues on the next page...

Table 6-10. Flash control block structure (continued)

Name	Start byte	Size in bytes	Description
NumberOfNANDs	32	4	Not used by ROM
TotalInternalDie	36	4	Not used by ROM
CellType	40	4	Not used by ROM
EccBlockNEccType	44	4	Value from 0 to is used to set the BCH Error Correction level 0, 2, 4, .. or 62 for Block BN of ECC page, used in configuring the BCH62 page layout registers.
EccBlock0Size	48	4	Size of block B0 used in configuring the BCH62 page-layout registers.
EccBlockNSize	52	4	Size of block BN used in configuring the BCH62 page-layout registers.
EccBlock0EccType	56	4	Value from 0 to used to set the BCH Error Correction level 0, 2, 4, .. or 62 for Block BN of ECC page, used in configuring the BCH62 page layout registers.
MetadataBytes	60	4	Size of metadata bytes used in configuring the BCH62 page-layout registers.
NumEccBlocksPerPage	64	4	Number of the ECC blocks BN not including B0. This value is used in configuring the BCH62 page-layout registers.
EccBlockNEccLevelSDK	68	4	Not used by ROM
EccBlock0SizeSDK	72	4	Not used by ROM
EccBlockNSizeSDK	76	4	Not used by ROM
EccBlock0EccLevelSDK	80	4	Not used by ROM
NumEccBlocksPerPageSDK	84	4	Not used by ROM
MetadataBytesSDK	88	4	Not used by ROM
EraseThreshold	92	4	Not used by ROM
Firmware1_startingPage	104	4	Page number address where the first copy of bootable firmware is located.
Firmware2_startingPage	108	4	Page number address where the second copy of bootable firmware is located.
PagesInFirmware1	112	4	Size of the first copy of firmware in pages.
PagesInFirmware2	116	4	Size of the second copy of firmware in pages.
DBBTSearchAreaStartAddress	120	4	Page address for the bad block table search area.
BadBlockMarkerByte	124	4	This is an input offset in the BCH page for the ROM to swap with the first byte of metadata after reading a page using the BCH62. The ROM supports the restoration of manufacturer-marked bad block markers in the page and this offset is the bad block marker offset location.
BadBlockMarkerStartBit	128	4	This is an input bit offset in the BadBlockMarkerByte for the ROM to use when swapping eight bits with the first byte of metadata.

Table continues on the next page...

Table 6-10. Flash control block structure (continued)

Name	Start byte	Size in bytes	Description
BBMarkerPhysicalOffset	132	4	This is the offset where the manufacturer leaves the bad block marker on a page.
BCHType	136	4	0 for BCH20 and 1 for BCH62. The chip is backwards compatible to BCH20 and this field tells the ROM to use the BCH20 or BCH62 block.
TMTiming2_ReadLatency	140	4	Toggle mode NAND timing parameter read latency, the ROM uses this value to configure the timing2 register of the GPML.
TMTiming2_PreambleDelay	144	4	Toggle mode NAND timing parameter Preamble Delay. The ROM uses this value to configure the timing2 register of the GPML.
TMTiming2_CEDelay	148	4	Toggle mode NAND timing parameter CE Delay. The ROM uses this value to configure the timing2 register of the GPML.
TMTiming2_PostambleDelay	152	4	Toggle mode NAND timing parameter Postamble Delay. The ROM uses this value to configure the timing2 register of the GPML.
TMTiming2_CmdAddPause	156	4	Toggle mode NAND timing parameter Cmd Add Pause. The ROM uses this value to configure the timing2 register of the GPML.
TMTiming2_DataPause	160	4	Toggle mode NAND timing parameter Data Pause. The ROM uses this value to configure the timing2 register of the GPML.
TMSpeed	164	4	This is the toggle mode speed for the ROM to configure the gpml clock. 0 for 33 MHz, 1 for 40 MHz, and 2 for 66 MHz.
TMTiming1_BusyTimeout	168	4	Toggle mode NAND timing parameter Busy Timeout. The ROM uses this value to configure the timing1 register of the GPML.
DISBBM	172	4	If 0, the ROM swaps the BadBlockMarkerByte with metadata[0] after reading a page using the BCH62. If the value is 1, the ROM does not swap.
BBMark_spare_offset	176	4	The offset in the metadata place which stores the data in the bad block marker place.
Onfi_sync_enable	180	4	Enable the Onfi nand sync mode support.
Onfi_sync_speed	184	4	Speed for the Onfi nand sync mode: 0 - 24 MHz, 1 - 33 MHz, 2 - 40 MHz, 3 - 50 MHz, 4 - 66 MHz, 5 - 80 MHz, 6 - 100 MHz, 7 - 133 MHz, 8 - 160 MHz, 9 - 200 MHz
Onfi_syncNANDData	188	28	The parameters for the Onfi nand sync mode timing. They are read_latency, ce_delay, preamble_delay, postamble_delay, cmdadd_pause, data_pause, and busy_timeout.
DISBB_Search	216	4	Disable the bad block search function when reading the firmware, only using DBBT.

The FCB data structure is protected using a 62-bit ECC. The layout of the FCB page is illustrated in this figure:



Figure 6-6. Layout of the FCB page

The detailed parameters of the FCB pages are listed in this table:

Table 6-11. Parameters setting for FCB page

Parameter	Value
TotalPageSize	2048+64=2112
MetadataBytes	32
EccBlock0Size	128
EccBlock0EccType	31
BCHType	0
EccBlockNSize	128
EccBlockNEccType	31
NumEccBlocksPerPage	7

To reduce the disturbances caused by a neighboring cell in the FCB page in the NAND chip, a randomizer is enabled when reading the FCB page. BCH ECC has a Randomizer module that is interfaced through the GPMI APBHDMA chain. The Randomizer can generate random data based on BCH ECC encoded/decoded data. It can be employed to reduce the disturbances caused by a neighboring cell in the NAND chip, thus reducing bit errors. The randomizer is used to reduce the bit errors in the FCB. Ensure that the randomizer is enabled when burning the FCB pages in the NAND flash. To control the randomizer for the pages (except for FCB), a new field called Randomizer_Enable is added into the FCB structure. If the Randomizer_Enable field is set to 0, the randomizer is disabled. Reading the pages (except for FCB) being set to a non-zero value enables the randomizer. For detailed randomizer information, see [Randomizer](#).

6.1.5.1.4 Discovered Bad Block Table (DBBT)

See this table for the DBBT format:

Table 6-12. DBBT structure

Name	Start byte	Size in bytes	Description
reserved	0	4	-
FingerPrint	4	4	32-bit word with a value of 0x44424254, in ascii "DBBT"
Version	8	4	32-bit version number; this version of DBBT is 0x00000001
reserved	12	4	-
DBBT_NUM_OF_PAGES	16	4	Size of the DBBT in pages
reserved	20	4*PageSize-20	-
reserved	4*PageSize	4	-
Number of Entries	4*PageSize + 4	4	Number of bad blocks
Bad Block Number	4*PageSize + 8	4	First bad block number
Bad Block Number	4*PageSize + 12	4	Second bad block number
-	-	-	Next bad block number
-	-	-	-
Last bad block number	-	-	Last bad block number

6.1.5.1.5 Bad block handling in ROM

During the firmware boot, at the block boundary, the Bad Block table is searched for a match to the next block.

If no match is found, the next block can be loaded. If a match is found, the block must be skipped and the next block checked.

If the Bad Block table start page is null, check the manufactory made Bad Block marker. The location of the Bad Block maker is at the first three or last three pages in every block of the NAND flash. The NAND manufacturers normally use one byte in the spare area of certain pages within a block to mark that a block is bad or not. A value of 0xFF means good block, non-FF means bad block.

To preserve the BI (bad block information), the flash updater or gang programmer applications must swap the Bad Block Information (BI) data to byte 0 of the metadata area for every page before programming the NAND flash. When the ROM loads the firmware, it copies back the value at metadata[0] to the BI offset in the page data. This figure shows how the factory bad block marker is preserved:

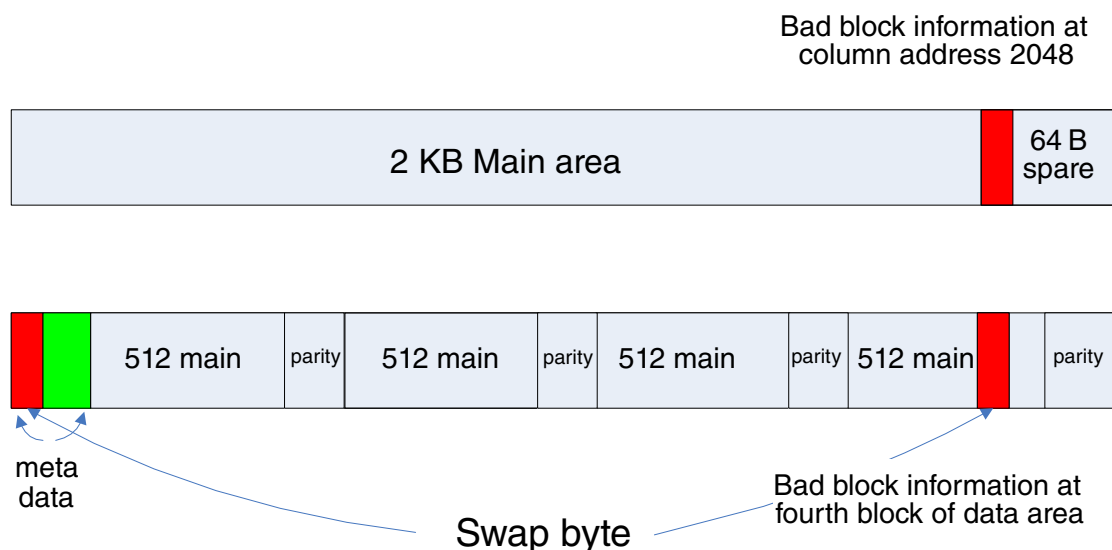


Figure 6-7. Factory bad block marker preservation

In the FCB structure, there are two elements (`m_u32BadBlockMarkerByte` and `m_u32BadBlockMarkerStartBit`) to indicate the byte and bit place in the page data that the manufacturer marked the bad block marker.

6.1.5.1.6 Toggle mode DDR NAND boot

If the `BT_TOGGLEMODE` efuse is blown, the ROM does the following to boot from the Samsung's toggle mode DDR NAND.

6.1.5.1.6.1 GPMI and BCH clocks configuration

The ROM sets the clock source and the dividers in the CCM registers.

If the `BOOT_CFG` is set (toggle mode), the GPMI/BCH CLK source is PLL2PFD4, and running at 66 MHz, otherwise the GPMI/ BCH CLK souce is PLL3, running at 24 MHz. The ROM sets the default values to `timing0`, `timing1`, and `timing2` gpmi registers for 24 MHz clock speed. It uses the `BOOT_CFG` fuse to configure the GPMI `timing2` register parameters preamble delay and read latency. The default value for these parameters is 2 when the fuses are not blown.

The default timing parameter values used by the ROM for the toggle-mode device are:

- `Timing0.ADDRESS_SETUP = 5`
- `Timing0.DATA_SETUP = 10`
- `Timing0.DATA_HOLD = 10`
- `Timing1.DEVICE_BUSY_TIMEOUT = 0 x 500`

- Timing2.READ_LATENCY = BOOT_CFG if blown, otherwise 2
- Timing2.CE_DELAY = 2
- Timing2.PREAMBLE_DELAY = BOOT_CFG if blown, otherwise 2
- Timing2.POSTAMBLE_DELAY = 3
- Timing2.CMDADD_PAUSE = 4
- Timing2.DATA_PAUSE = 6

The default timing parameters can be overridden by the TMTiming2_ReadLatency, TMTiming2_PreambleDelay, TMTiming2_CEDelay, TMTiming2_PostambleDelay, TMTiming2_CmdAddPause, and TMTiming2_DataPause parameters of the FCB.

6.1.5.1.6.2 Setup DMA for DDR transfers

In the DMA descriptors, the GPMI is configured to read the page data at a double data rate, the word length is set to 16, and the transfer count to a half of the page size.

6.1.5.1.6.3 Reconfigure timing and speed using values in FCB

After reading the FCB page with the GPMI set to default timings and a speed of 33 MHz, the ROM reconfigures the CCM dividers to run the gpmi/bch clks to a desired speed specified in the FCB for the rest of the boot process. The GPMI timing registers are also reconfigured to the values specified in the FCB.

The GPMI speed can be configured using the FCB parameter TMSpeed:

- 0—25 MHz
- 1—33 MHz
- 2—40 MHz
- 3—50 MHz
- 4—66 MHz
- 5—80 MHz
- 6—100 MHz
- 7—133 MHz
- 8—133 MHz
- 9—200 MHz

The GPMI timing0 register fields data_setup, data_hold, and address_setup are set to the values specified for the data_setup and data_hold and address_setup in the FCB member m_NANDTiming.

The GPMI timing1.DEVICE_BUSY_TIMEOUT is set to the value specified in the FCB member TMTiming1_BusyTimeout.

The GPMI timing2 register values are set using the FCB members TMTiming2.READ_LATENCY, CE_DELAY, PREAMBLE_DELAY, POSTAMBLE_DELAY, CMDADD_PAUSE, and DATA_PAUSE.

6.1.5.1.7 Typical NAND page organization

6.1.5.1.7.1 BCH ECC page organization

The first data block is called block 0 and the rest of the blocks are called block N. A separate ECC level scan is used for block 0 and block N.

The metadata bytes must be located at the beginning of a page, starting at byte 0, followed by the data block 0, the ECC bytes for data block 0, the block 1 and its ECC bytes, and so on, up until the N data blocks. The ECC level for the block 0 can be different from the ECC level for the rest of the blocks.

For the NAND boot with page-size restrictions and the data block size restricted to 512 B, only few combinations of the ECC for block 0 and block N are possible.

This figure shows the valid layout for 2112-byte sized page.

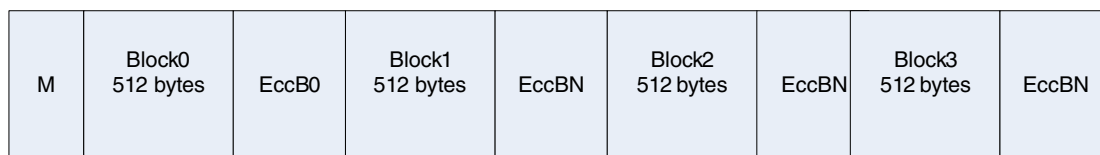


Figure 6-8. Valid layout for 2112-byte sized page

The example below is for 13 bits of parity (GF13). The number of ECC bits required for a data block is calculated using the (ECC_Correction_Level * 13) bits.

In the above layout, the ECC size for EccB0 and EccBN must be selected to not exceed a total page size of 2112 bytes. The EccB0 and EccBN can be one of the 2, 4, 6, 8, 10, 12, 14, 16, 18, and 20 bits on the ECC correction level. The total bytes are:

$$[M + (\text{data_block_size} \times 4) + ([\text{EccB0} + (\text{EccBN} \times 3)] \times 13) / 8] \leq 2112;$$

M = metadata bytes and data_block_size is 512.

There are four data blocks of 512 bytes each in a page of 2-KB page sized NAND. The values of EccB0 and EccBN must be such that the above calculation does not result in a value greater than 2112 bytes.

M	Block0 512 bytes	EccB0	Block1 512 bytes	EccBN	Block2 512 bytes	EccBN	Block3 512 bytes	EccBN
	Block4 512 bytes	EccBN	Block5 512 bytes	EccBN	Block6 512 bytes	EccBN	Block7 512 bytes	EccBN

Figure 6-9. Valid layout for 4-KB sized page

Different NAND manufacturers have different sizes for a 4-KB page; 4314 bytes is typical.

$$[M + (\text{data_block_size} \times 8) + ([\text{EccB0} + (\text{EccBN} \times 7)] \times 13) / 8] \leq 4314;$$

M= metadata bytes and data_block_size is 512.

There are eight data blocks of 512 bytes each in a page of a 4-KB page sized NAND. The values of the EccB0 and EccBN must be such that the above calculation does not result in a value greater than the size of a page in a 4-KB page NAND.

6.1.5.1.7.2 Metadata

The number of bytes used for the metadata is specified in the FCB. The metadata for the BCH encoded pages is placed at the beginning of a page. The ROM only cares about the first byte of metadata to swap it with a bad block marker byte in the page data after each page read; it is important to have at least one byte for the metadata bytes field in the FCB data structure.

6.1.5.1.8 IOMUX configuration for NAND

The following table shows the RawNAND IOMUX pin configuration.

Table 6-13. NAND IOMUX pin configuration

Signal	Pad name
NAND_CLE	SD3_CLK.alt1
NAND_ALE	SD3_CMD.alt1
NAND_WP_B	SAI1_MCLK.alt1
NAND_RE_B	SD3_STROBE.alt1
NAND_WE_B	SD3_RESET_B.alt1
NAND_READY_B	SAI1_TXD.alt1
NAND_DQS	SAI1_TXFS.alt1

Table continues on the next page...

Table 6-13. NAND IOMUX pin configuration (continued)

NAND_CE0_B	SAI1_TXC.alt1
NAND_DATA00	SD3_DATA0.alt1
NAND_DATA01	SD3_DATA1.alt1
NAND_DATA02	SD3_DATA2.alt2
NAND_DATA03	SD3_DATA3.alt3
NAND_DATA04	SD3_DATA4.alt4
NAND_DATA05	SD3_DATA5.alt5
NAND_DATA06	SD3_DATA6.alt6
NAND_DATA07	SD3_DATA7.alt7

6.1.5.2 Expansion device

The ROM supports booting from the MMC/eMMC and SD/eSD compliant devices.

6.1.5.2.1 Expansion device eFUSE configuration

The SD/MMC/eSD/eMMC/SDXC boot can be performed using either the USDHC ports, based on the setting of the BOOT_CFG[11:10] fuse or it is associated to the GPIO input value at the boot.

All USDHC ports support the fast boot. See this table for details:

Table 6-14. USDHC boot eFUSE descriptions

Fuse	Config	Definition	GPIO ¹	Shipped value	Settings
BOOT_CFG[7]	OEM	Fast boot support	Yes	000	0 - Normal boot 1 - Fast boot
BOOT_CFG[6:4]	OEM	Bus width	Yes	000	0 - SD/eSD/SDXC 1 - MMC/eMMC
BOOT_CFG[3:1]	OEM	SD speed mode	Yes	000	SD speed selection speed 000 - Normal/SDR12 001 - High/SDR25 010 - SDR50 011 - SDR104 else - Reserved
		MMC Speed Mode (CFG[3:2])	Yes	00	MMC speed selection 00 - Normal 01 - High

Table continues on the next page...

**Table 6-14. USDHC boot eFUSE descriptions
(continued)**

Fuse	Config	Definition	GPIO ¹	Shipped value	Settings
					else - Reserved
		USDHC1 IO Voltage Selection (CFG[1])	Yes	0	USDHC1 IO VOLTAGE SELECTION (only for MMC/eMMC boot) 0 - 3.3 V 1 - 1.8 V
BOOT_CFG[0]	OEM	USDHC2 IO VOLTAGE	Yes	0	USDHC2 IO VOLTAGE SELECTION (only for MMC/eMMC boot) 0 - 3.3 V 1 - 1.8 V
BOOT_CFG[15:12]	OEM	Boot device selection	Yes	0000	0001 - Boot from SD/eSD 0010 - Boot from MMC/eMMC
BOOT_CFG[11:10]	OEM	USDHC port selection	Yes	00	00 - USDHC-1 01 - USDHC-2 10 - USDHC-3 else - reserved
BOOT_CFG[9]	OEM	SD power cycle enable/ eMMC reset enable	Yes	0	SD power cycle/eMMC reset 0 - Disabled 1 - Enabled
BOOT_CFG[8]	OEM	USDHC loopback clock selection	Yes	0	USDHC loopback clock source selection 0 - Through SD pad 1 - Direct
0x490[14:8]	OEM	SD/MMC DLL DLY config	No	0	Delay target for USDHC DLL, it is applied to the slave mode target delay or overrides the mode target delay, depending on the DLL override fuse bit value.
0x490[15]	OEM	USDHC DLL override enabled	No	0	0 - No override 1 - Override
0x490[16]	OEM	USDHC DLL enabled	No	0	0 - Disable the DLL for SD/eMMC 1 - Enable the DLL for SD/eMMC
0x490[17]	OEM	USDHC override pad settings selection			0 - Use default pad settings 1 - Override the USDHC pad settings by using the PAD_SETTINGS value
0x490[18]	OEM	USDHC_IOMUX_SION_BIT_ENABLE	No	0	0 - Disable 1 - Enable

Table continues on the next page...

**Table 6-14. USDHC boot eFUSE descriptions
(continued)**

Fuse	Config	Definition	GPIO ¹	Shipped value	Settings
0x490[19]	OEM	ENABLE_EMMC_5K_PULLUP	No	0	0 - 47 K pullup 1 - 5 K pullup
0x490[20]	OEM	USDHC_PAD_PULL_DOWN	No	0	0 - No action 1 - Pull down
0x490[21]	OEM	Issue pre-idle command enabled (for eMMC4.4)	No	0	0 - Enable 1 - Disable
0x490[23]	OEM	Disable SDMMC manufacture mode	No	0	0 - Enable 1 - Disable
0x490[31:24]	OEM	USDHC pad setting override	No	0	Override pad settings default if 0x490[17] is set
0x4A0[0]	OEM	Fast boot acknowledge enable	No	0	0 - Boot Ack disabled 1 - Boot Ack enabled
0x4A0[1]	OEM	USDHC3 IO voltage selection	No	0	0 - 3.3 V 1 - 1.8 V
0x4A0[2]	OEM	uSDHC power-off polarity selection	No	0	0 - Low 1 - High
0x4A0[3]	OEM	uSDHC power cycle delay selection	No	0	0 - 5 ms 1 - 2.5 ms
0x4A0[5:4]	OEM	uSDHC power cycle interval	No	0	00 - 20 ms 01 - 10 ms 10 - 5 ms 11 - 2.5 ms

1. The setting can be overridden by the GPIO settings when the BT_FUSE_SEL fuse is intact. See [GPIO boot overrides](#) for the corresponding GPIO pin.

The boot code supports these standards:

- MMCv4.4 or less
- eMMCv4.4 or less
- SDv2.0 or less
- eSDv2.10 rev-0.9, with or without FAST_BOOT
- SDXCv3.0

The MMC/SD/eSD/SDXC/eMMC can be connected to any of the USDHC blocks and can be booted by copying 4 KB of data from the MMC/SD/eSD/eMMC device to the internal RAM. After checking the Image Vector Table header value (0xD1) from program image, the ROM code performs a DCD check. After a successful DCD

extraction, the ROM code extracts from the Boot Data Structure the destination pointer and length of image to be copied to the RAM device from where the code execution occurs.

The maximum image size to load into the SD/MMC boot is 32 MB. This is due to a limited number of uSDHC ADMA Buffer Descriptors allocated by the ROM.

NOTE

The initial 4 KB of the program image must contain the IVT, DCD, and the Boot Data structures.

Table 6-15. SD/MMC frequencies

	SD	MMC	MMC (DDR mode)
Identification (KHz)	347.22		
Normal-speed mode (MHz)	25	20	25
High-speed mode (MHz)	50	40	50
UHSI SDR50 (MHz)	100		
UHSI SDR104 (MHz)	200		

NOTE

The boot ROM code reads the application image length and the application destination pointer from the image.

6.1.5.2.2 MMC and eMMC boot

This table provides the MMC and eMMC boot details.

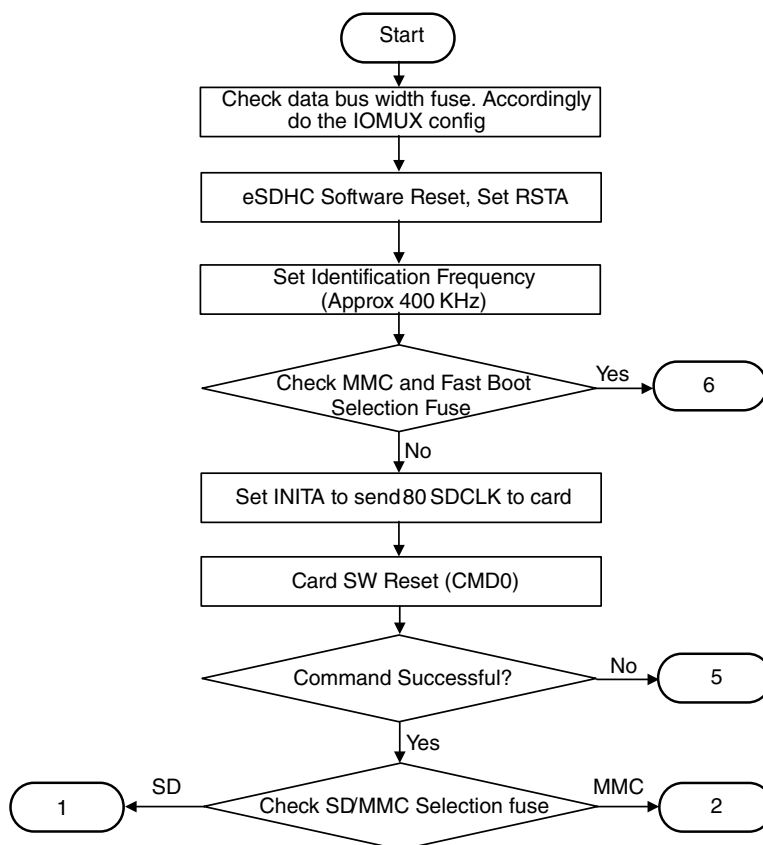
Table 6-16. MMC and eMMC boot details

Normal boot mode	During the initialization (normal boot mode), the MMC frequency is set to 347.22 KHz. When the MMC card enters the identification portion of the initialization, the voltage validation is performed, and the ROM boot code checks the high-voltage settings and the card capacity. The ROM boot code supports both the high-capacity and low-capacity MMC/eMMC cards. After the initialization phase is complete, the ROM boot code switches to a higher frequency (20 MHz in the normal boot mode or 40 MHz in the high-speed mode). The eMMC is also interfaced via the USDHC and follows the same flow as the MMC. The boot partition can be selected for an MMC4.x card after the card initialization is complete. The ROM code reads the BOOT_PARTITION_ENABLE field in the Ext_CSD[179] to get the boot partition to be set. If there is no boot partition mentioned in the BOOT_PARTITION_ENABLE field or the user partition was mentioned, the ROM boots from the user partition.
------------------	--

Table continues on the next page...

Table 6-16. MMC and eMMC boot details (continued)

eMMC4.3 or eMMC4.4 device supporting special boot mode	If using an eMMC4.3 or eMMC4.4 device that supports the special boot mode, it can be initiated by pulling the CMD line low. If the BOOT ACK is enabled, the eMMC4.3/eMMC4.4 device sends the BOOT ACK via the DATA lines and the ROM can read the BOOT ACK [S010E] to identify the eMMC4.3/eMMC4.4 device. If the BOOT ACK is enabled, the ROM waits 50 ms to get the BOOT ACK. If BOOT ACK is disabled ROM waits 1 second for data. If the BOOT ACK or data was received, the eMMC4.3/eMMC4.4 is booted in the "boot mode", otherwise the eMMC4.3/eMMC4.4 boots as a normal MMC card from the selected boot partition. This boot mode can be selected by the BOOT_CFG (fast boot) fuse. The BOOT ACK is selected by the fuse.
eMMC4.4 device	If using the eMMC4.4 device, the Double Data Rate (DDR) mode can be used. This mode can be selected by the BOOT_CFG2[7:5] (bus width) fuse.

**Figure 6-10. Expansion device boot flow (1 of 6)**

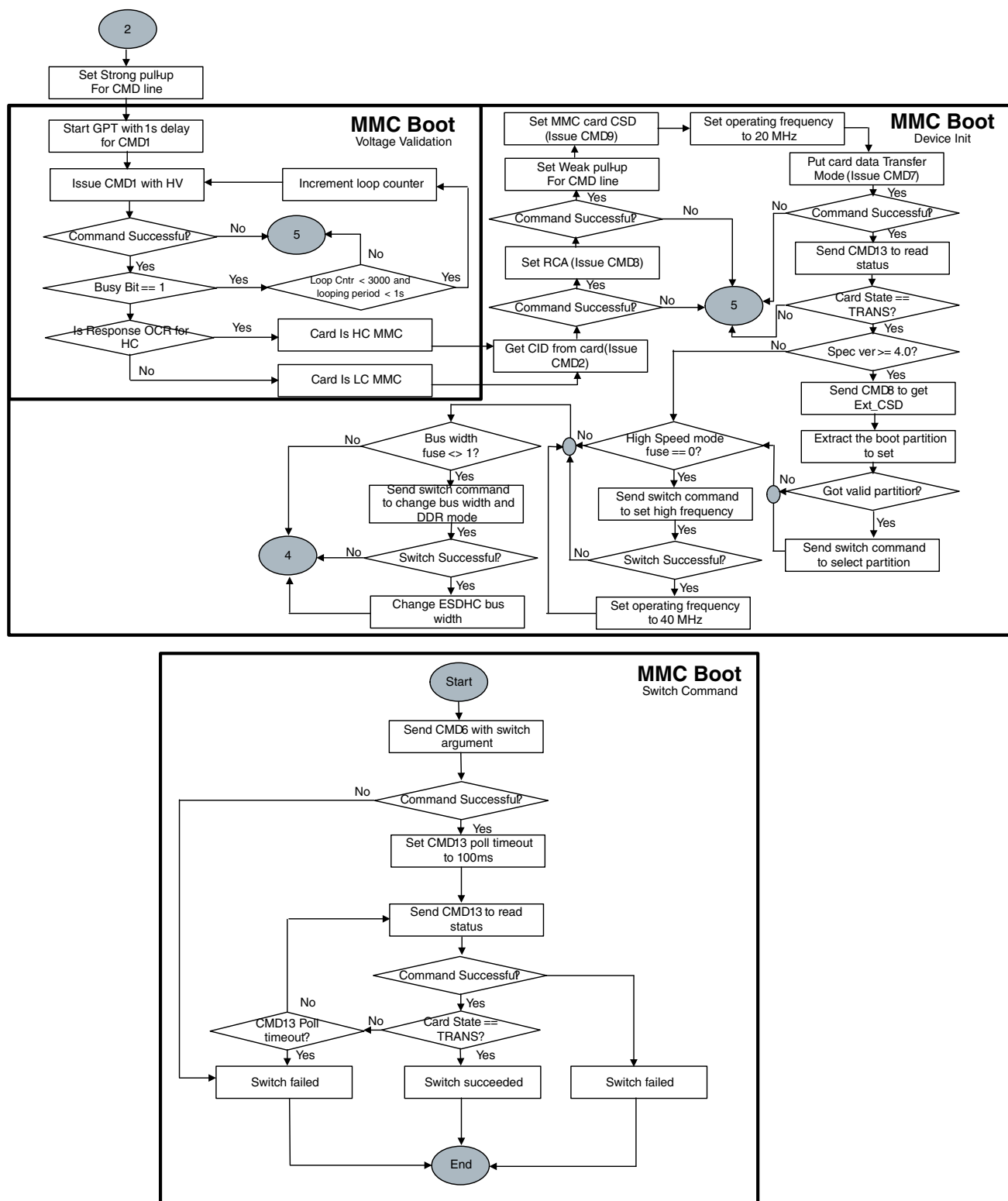


Figure 6-11. Expansion device (MMC) boot flow (2 of 6)

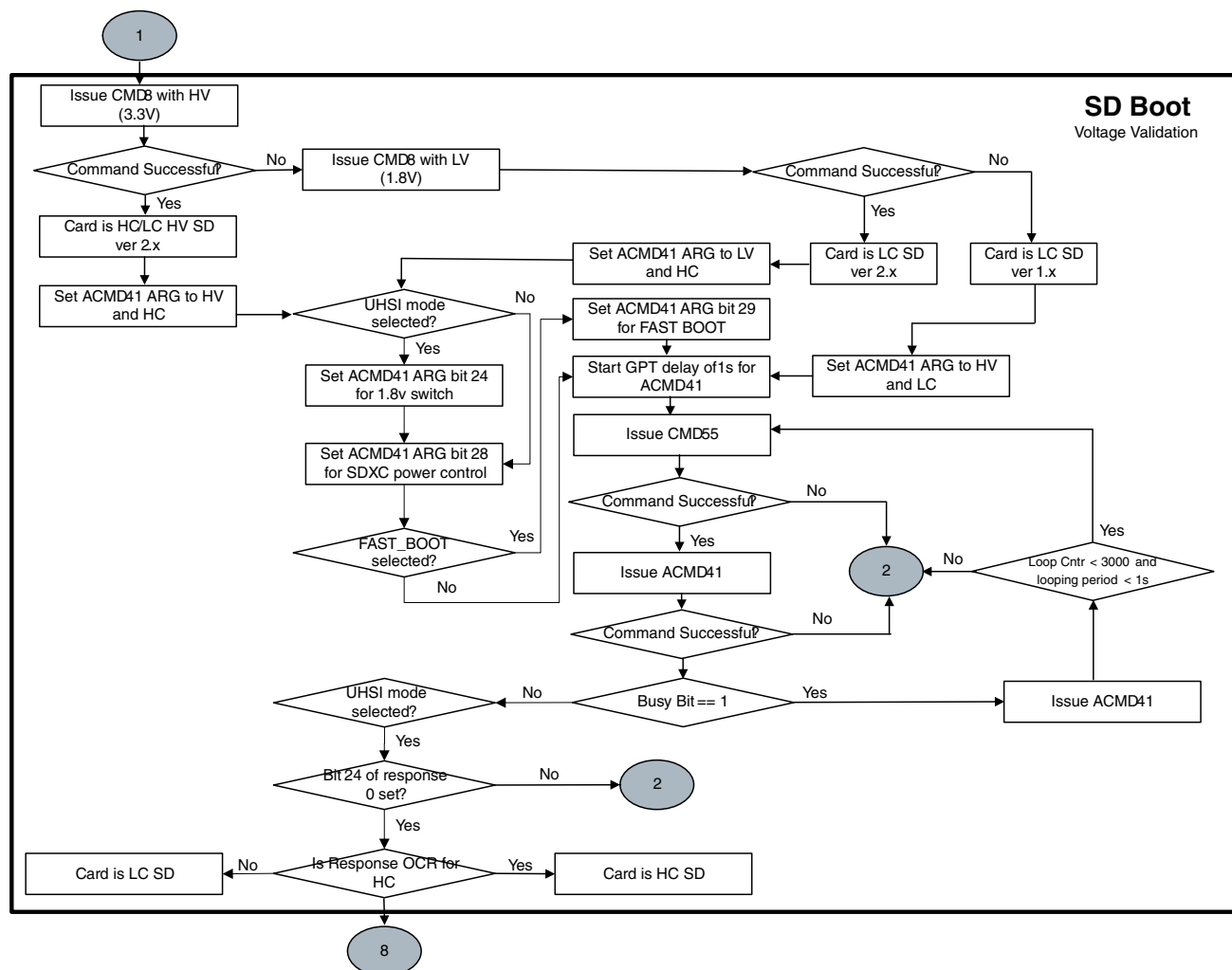


Figure 6-12. Expansion device (SD/eSD/SDXC) boot flow (3 of 6) part 1

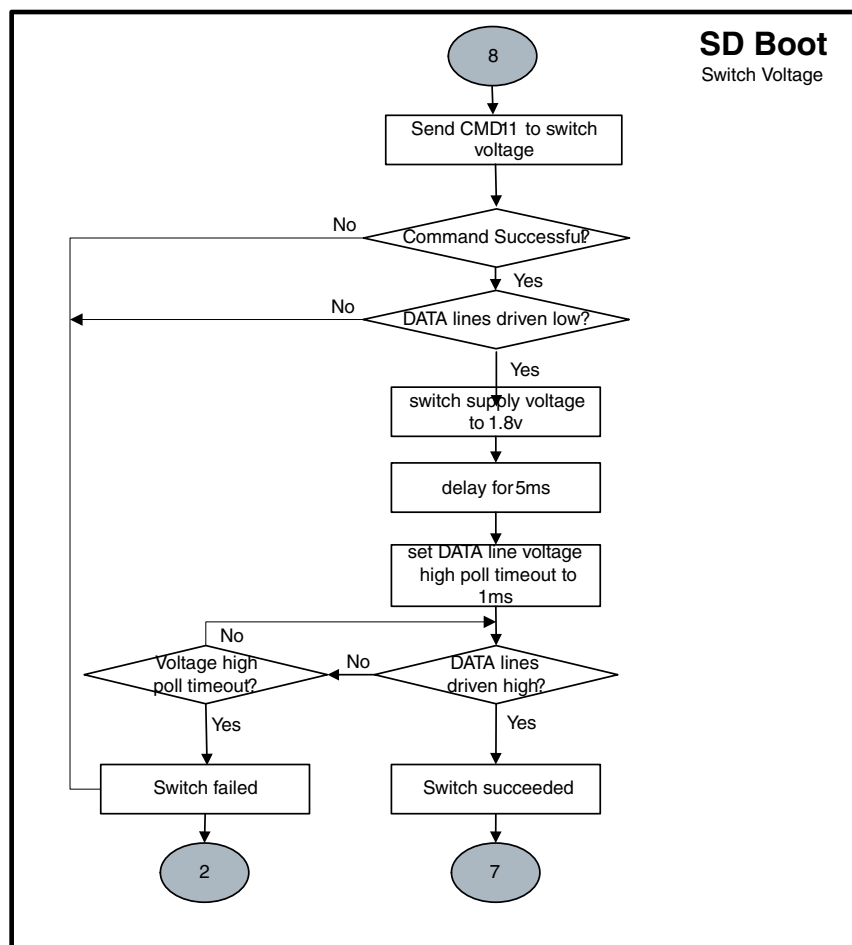


Figure 6-13. Expansion device (SD/eSD/SDXC) boot flow (3 of 6) part 2

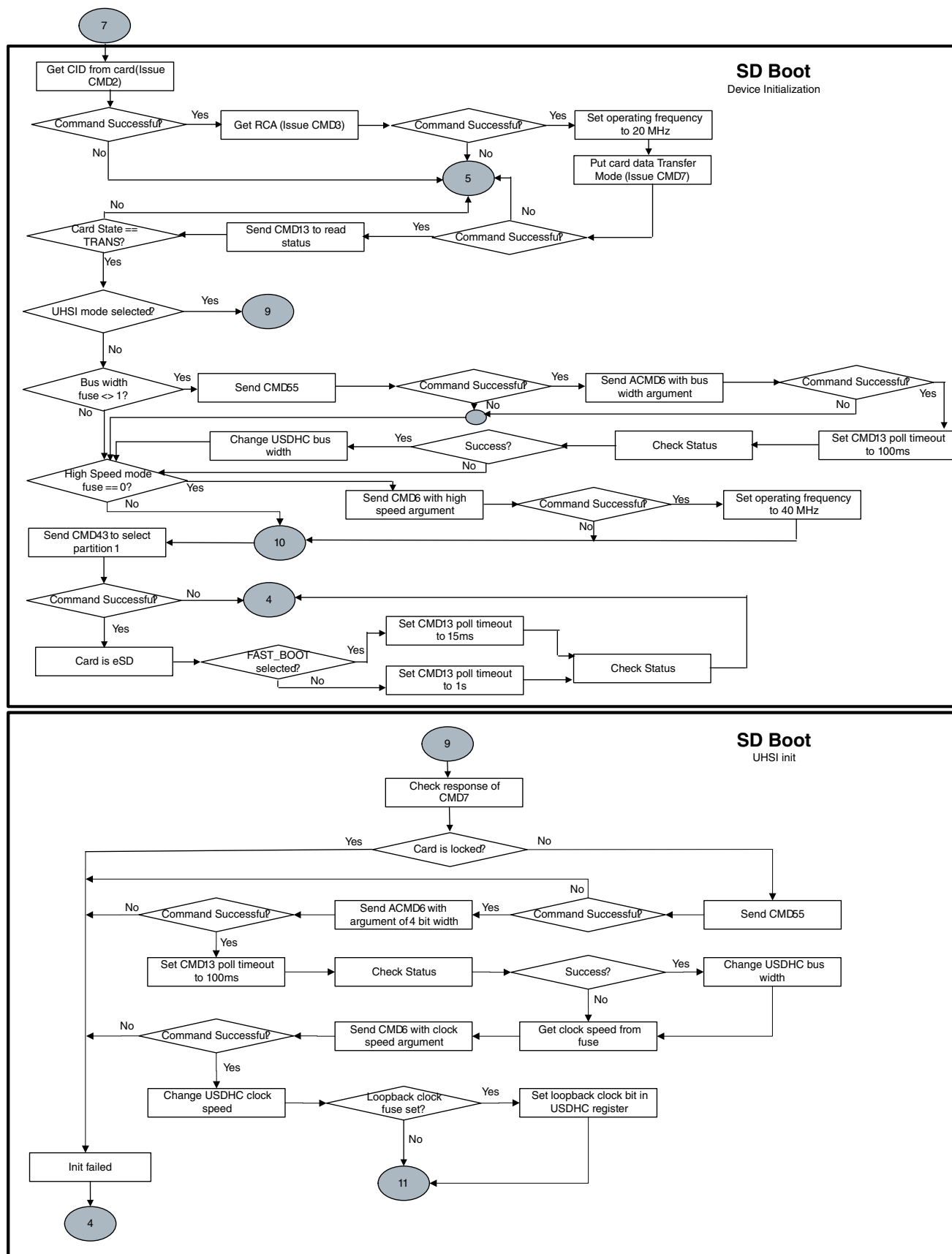


Figure 6-14. Expansion device (MMCSD/eSD/SDXC) boot flow (4 of 6)
i.MX 8M Dual/8M QuadLite/8M Quad Applications Processors Reference Manual, Rev. 3.1, 06/2021

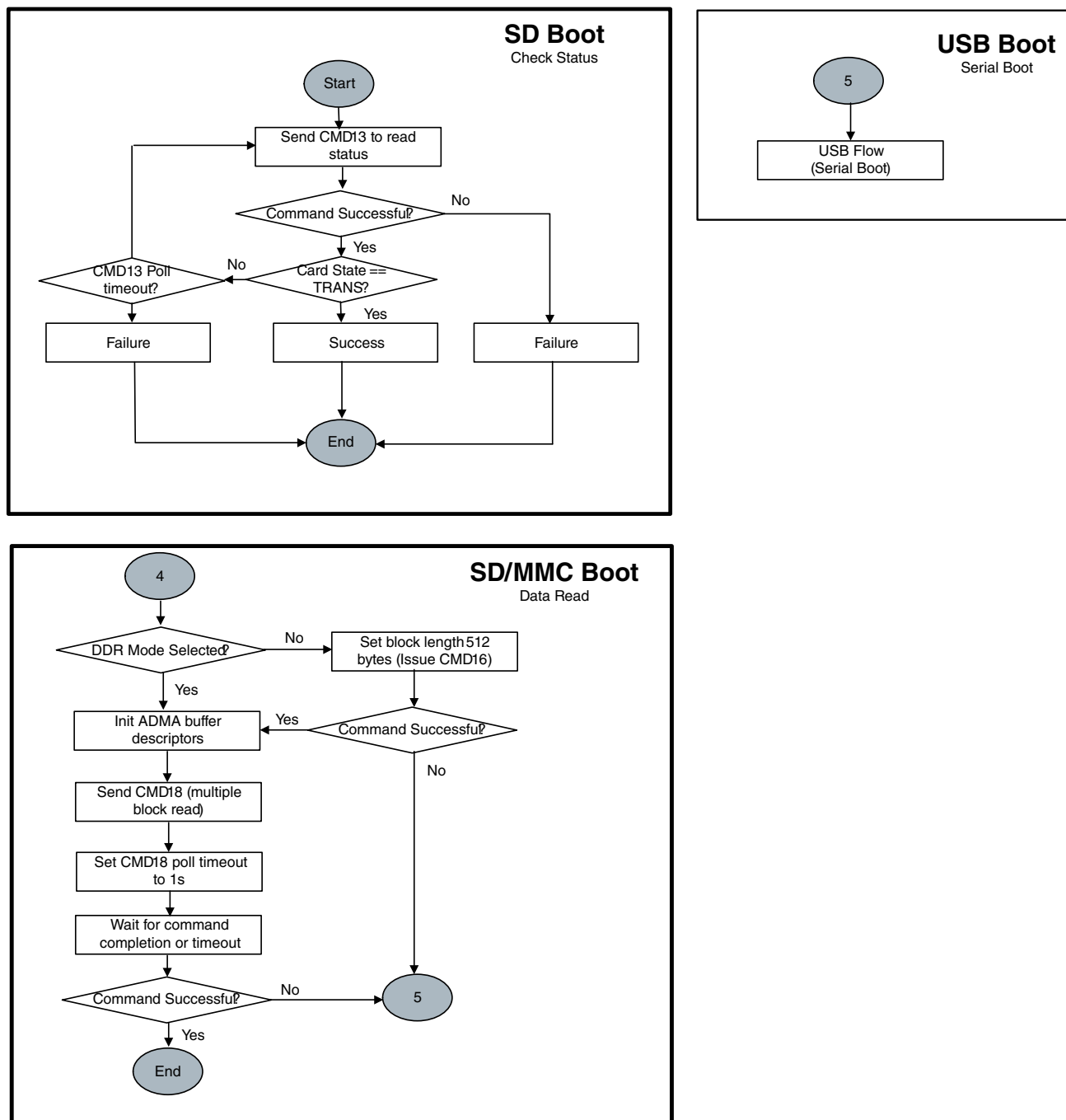


Figure 6-15. Expansion device (SD/eSD) boot flow (5 of 6)

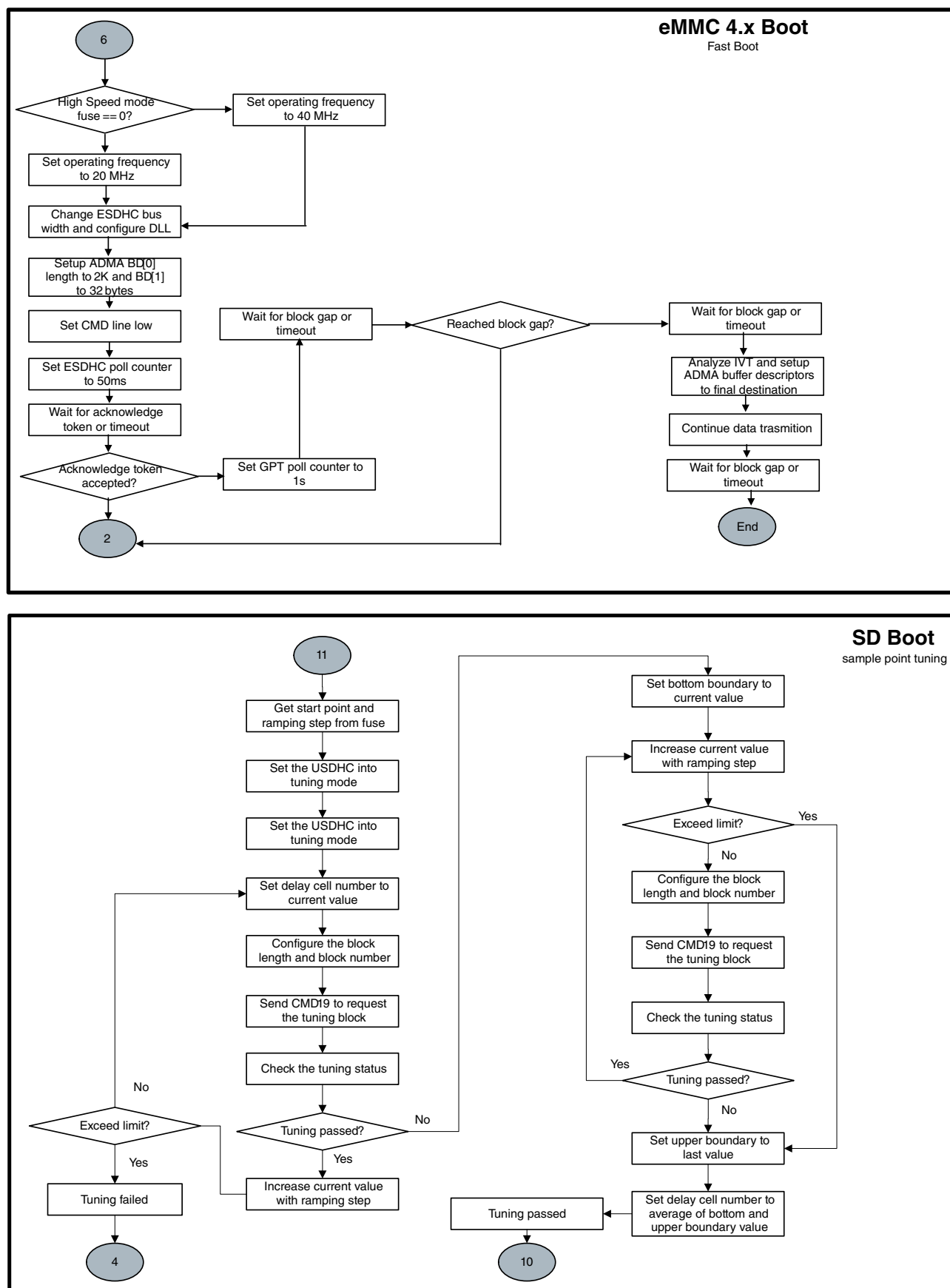


Figure 6-16. Expansion device boot flow (6 of 6)

i.MX 8M Dual/8M QuadLite/8M Quad Applications Processors Reference Manual, Rev. 3.1, 06/2021

6.1.5.2.3 SD, eSD, and SDXC

After the normal boot mode initialization begins, the SD/eSD/SDXC frequency is set to 347.22 kHz. During the identification phase, the SD/eSD/SDXC card voltage validation is performed. During the voltage validation, the boot code first checks with the high-voltage settings; if that fails, it checks with the low-voltage settings.

The capacity of the card is also checked. The boot code supports the high-capacity and low-capacity SD/eSD/SDXC cards after the voltage validation card initialization is done.

During the card initialization, the ROM boot code attempts to set the boot partition for all SD, eSD, and SDXC devices. If this fails, the boot code assumes that the card is a normal SD or SDXC card. If it does not fail, the boot code assumes it is an eSD card. After the initialization phase is over, the boot code switches to a higher frequency (25 MHz in the normal-speed mode or 50 MHz in the high-speed mode). The ROM also supports the FAST_BOOT mode booting from the eSD card. This mode can be selected by the BOOT_CFG[4] (Fast Boot) fuse described in .

For the UHSI cards, the clock speed fuses can be set to SDR50 or SDR104 on ports. This enables the voltage switch process to set the signaling voltage to 1.8 V during the voltage validation. The bus width is fixed at a 4-bit width and a sampling point tuning process is needed to calibrate the number of the delay cells. If the SD Loopback Clock eFuse is set, the feedback clock comes directly from the loopback SD clock, instead of the card clock (by default). The SD clock speed can be selected by the BOOT_CFG[3:2], and the SD Loopback Clock is selected by the BOOT_CFG[0].

The UHSI calibration start value (MMC_DLL_DLY[6:0]) and the step value can be set to optimize the sample point tuning process.

If the SD Power Cycle Enable eFuse is 1, the ROM sets the SD_RST pad low, waits for 5 ms, and then sets the SD_RST pad high. If the SD_RST pad is connected to the SD power supply enable logic on board, it enables the power cycle of the SD card. This may be crucial in case the SD logic is in the 1.8 V states and must be reset to the 3.3 V states.

6.1.5.2.4 IOMUX configuration for SD/MMC

Table 6-17. SD/MMC IOMUX pin configuration

Signal	USDHC1	USDHC2
CLK	SD1_CLK.alt0	SD2_CLK.alt0
CMD	SD1_CMD.alt0	SD2_CMD.alt0
DATA0	SD1_DATA0.alt0	SD2_DATA0.alt0
DATA1	SD1_DATA1.alt0	SD2_DATA1.alt0
DATA2	SD1_DATA2.alt0	SD2_DATA2.alt0

Table continues on the next page...

Table 6-17. SD/MMC IOMUX pin configuration (continued)

Signal	USDHC1	USDHC2
DATA3	SD1_DATA3.alt0	SD2_DATA3.alt0
DATA4	SD1_DATA4.alt0	-
DATA5	SD1_DATA5.alt0	-
DATA6	SD1_DATA6.alt0	-
DATA7	SD1_DATA7.alt0	-
VSELECT	-	GPIO1_IO04.alt1
RESET_B	SD1_RESET_B.alt0	SD2_RESET_B.alt0
CD_B	-	SD2_CD_B.alt0
WP	-	SD2_WP.alt0

6.1.6 Program image

This section describes the data structures that are required to be included in the user's program image. The program image consists of:

- Image vector table—a list of pointers located at a fixed address that the ROM examines to determine where the other components of the program image are located.
- Boot data—a table that indicates the program image location, program image size in bytes, and the plugin flag.
- Device configuration data—IC configuration data.
- User code and data.

6.1.6.1 Image Vector Table and Boot Data

The Image Vector Table (IVT) is the data structure that the ROM reads from the boot device supplying the program image containing the required data components to perform a successful boot.

The IVT includes the program image entry point, a pointer to Device Configuration Data (DCD) and other pointers used by the ROM during the boot process. The ROM locates the IVT at a fixed address that is determined by the boot device connected to the Chip. The IVT offset from the base address and initial load region size for each boot device

type is defined in the table below. The location of the IVT is the only fixed requirement by the ROM. The remainder of the image memory map is flexible and is determined by the contents of the IVT.

Table 6-18. Image Vector Table Offset and Initial Load Region Size

Boot Device Type	Image Vector Table Offset	Initial Load Region Size
NAND	1 Kbyte = 0x4000 bytes	8 Kbyte
SD/eSD/MMC/eMMC normal boot	33 Kbyte = 0x8400 bytes	8 Kbyte
eMMC Fast boot	1 Kbyte = 0x400 bytes	8 Kbyte

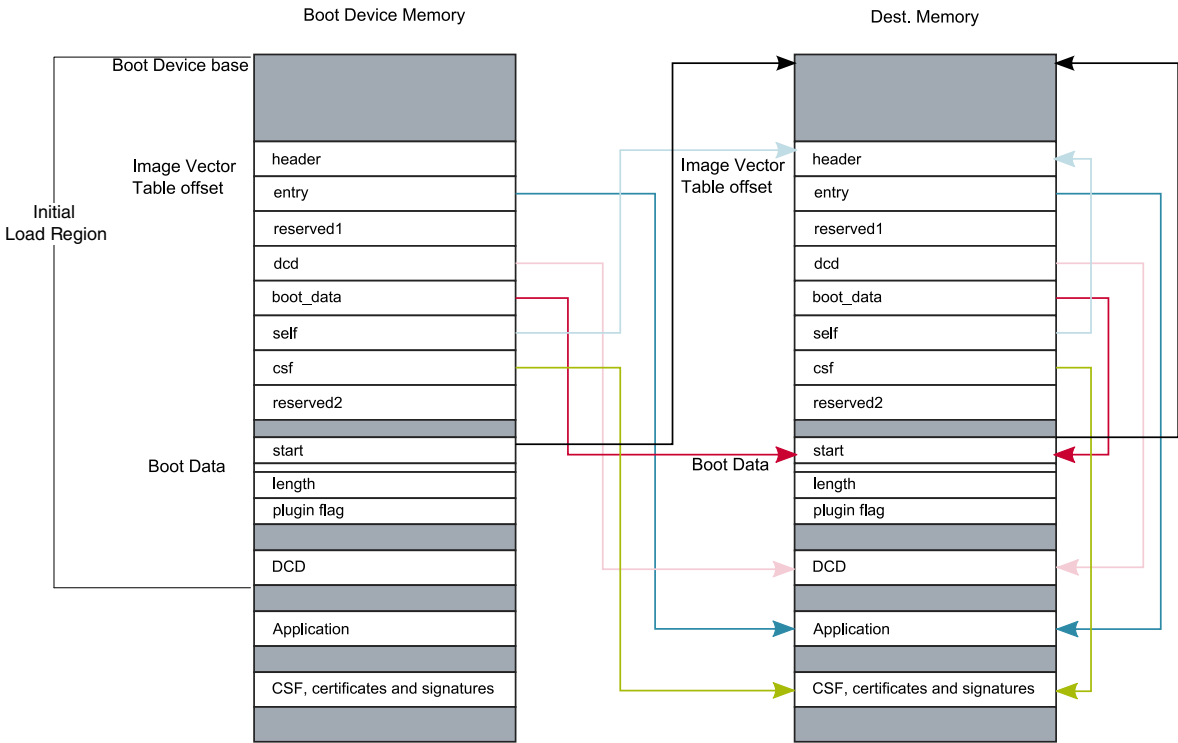


Figure 6-17. Image Vector Table

6.1.6.1.1 Image vector table structure

The IVT has the following format where each entry is a 32-bit word:

Table 6-19. IVT format

header
entry: Absolute address of the first instruction to execute from the image

Table continues on the next page...

Table 6-19. IVT format (continued)

reserved1: Reserved and should be zero
dcd: Absolute address of the image DCD. The DCD is optional so this field may be set to NULL if no DCD is required. See Device Configuration Data (DCD) for further details on the DCD.
boot data: Absolute address of the boot data
self: Absolute address of the IVT. Used internally by the ROM.
csf: Absolute address of the Command Sequence File (CSF) used by the HAB library. See High-Assurance Boot (HAB) for details on the secure boot using HAB. This field must be set to NULL when not performing a secure boot
reserved2: Reserved and should be zero

Figure 6-18 shows the IVT header format:

Tag	Length	Version
-----	--------	---------

Figure 6-18. IVT header format

where:

Tag: A single byte field set to 0xD1

Length: a two byte field in big endian format containing the overall length of the IVT, in bytes, including the header. (the length is fixed and must have a value of 32 bytes)

Version: A single byte field set to 0x40 or 0x41

6.1.6.1.2 Boot data structure

The boot data must follow the format defined in the table found here, each entry is a 32-bit word.

Table 6-20. Boot data format

start	Absolute address of the image
length	Size of the program image
plugin	Plugin flag (see Plugin image)

6.1.6.2 HDMI image boot up

The chip also supports HDMI image boot up for HDMI firmware. In addition to the standard image, there is a special image flagged as HDMI image (see [Boot data structure](#)). And normally it will come before the standard image.

ROM will check the HDMI image viability first. If the HDMI image is present and the HDMI device is not disabled by the eFuse, it will load the image into HDMI RAM and do the authentication. The image will include both the HDMI firmware and authentication information in it.

If the authentication is succeeded, then the firmware will be started and verified. ROM will also load the HDMI/ HDCP related keys and then lock it.

If the authentication is failed, then the HDMI memory will be cleared and locked. If the HDMI is disabled, ROM will release the JTAG access at early boot up stage otherwise it will be released after HDMI image processing.

After processing HDMI image, the ROM process will move to standard image boot up sequence.

The HDMI image is generated by NXP CDT tool and firmware will be provided by customer. For more details about generating HDMI image, please refer to NXP CDT user guide.

The following figure shows the HDMI image boot flow. See [High-level boot sequence](#) for standard image boot flow.

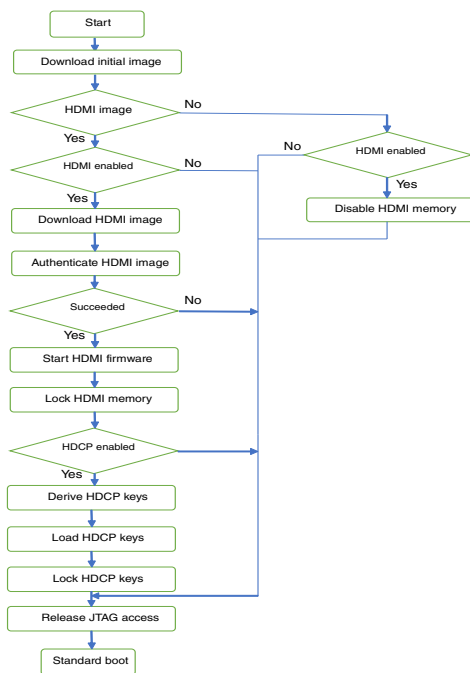


Figure 6-19. HDMI image boot flow

6.1.6.2.1 HDMI and A53 boot image location on boot device

The new requirement for i.MX 8M Dual/8M QuadLite/8M Quad ROM is to locate HDMI image on the boot device, load it into the HDMI block's i-ram and d-ram memories, authenticate it, and then finally locate, load, and authenticate the A53 boot image.

The chip can boot from the following boot media:

1. eMMC/SD
2. NAND

On the boot media, the HDMI image will be placed first followed by the A53 image. The distance between the two images is different for different types of boot media.

For eMMC/SD, the A53 image should start on the next 512-byte sector where HDMI image ends. Typically, here is the layout for eMMC/SD device for Normal and fast boot methods:

Table 6-21. eMMC/SD BOOT layout

eMMC/SD BOOT			
NORMAL BOOT	SECTOR/LBA #	BYTE OFFSET	FAST BOOT
32K SPACE FOR GUID PARTITION TABLE	0	0	START OF HDMI IMAGE
	1	512	
	2	1024	HDMI IMAGE IVT AND BOOT DATA
	...		
	...		...
	16	8192	96K HDMI IMAGE
	.	.	64K HDMI IMEM
	...		28K HDMI DMEM FOLLOWED BY COPY OF IVT AND BOOT DATA CSF DATA
START OF HDMI IMAGE	64	32768	
SECONDARY IMAGE TABLE	65		
HDMI IMAGE IVT AND BOOT DATA	66		
	...		
...	...		
92K HDMI IMAGE DATA	80	40960	
64K HDMI IMEM	...	...	
28K HDMI DMEM	208	106496	START OF A53 IMAGE
	210	107520	A53 IMAGE IVT AND BOOT DATA
	...	...	DCD
	...	...	...
	...	...	CSF DATA
	...	...	...

Table continues on the next page...

Table 6-21. eMMC/SD BOOT layout (continued)

eMMC/SD BOOT			
NORMAL BOOT	SECTOR/LBA #	BYTE OFFSET	FAST BOOT
	...	...	
	...	...	A53 IMAGE DATA
COPY OF IVT AND BOOT DATA	264		
CSF DATA	264 Sector, Byte 64		
...	...		
START OF A53 IMAGE	272	139264	
SECONDARY IMAGE TABLE	273		
A53 IMAGE IVT AND BOOT DATA	274	140288	
DCD			
...			
CSF DATA			
...	.		
	.		
A53 IMAGE DATA			
...			
SECOND COPY OF HDMI IMAGE			
SECOND COPY OF BOOT IMAGE			

For NAND device, the A53 image should start on the next new block where HDMI image ends. Typically, here is the layout for NAND device:

Table 6-22. NAND layout

NAND BOOT	
BLOCK #	NAND BOOT
0	FCB SEARCH AREA
...	...
4	DBBT SEARCH AREA
...	...
8	START OF HDMI IMAGE
	SECONDARY IMAGE TABLE
	HDMI IMAGE IVT AND BOOT DATA
	CSF DATA
	...

Table continues on the next page...

Table 6-22. NAND layout (continued)

NAND BOOT	
BLOCK #	NAND BOOT
9	96K HDMI IMAGE 64K HDMI IMEM 28K HDMI DMEM FOLLOWED BY COPY OF IVT AND BOOT DATA CSF DATA
	START OF A53 IMAGE
	SECONDARY IMAGE TABLE
	A53 IMAGE IVT AND BOOT DATA
	DCD
	...
	CSF DATA
	...
	.
	A53 IMAGE DATA
	.
	.
	.
	.
10	START OF HDMI IMAGE COPY 2
	.
	HDMI IMAGE IVT AND BOOT DATA
	CSF DATA
	...
	96K HDMI IMAGE 64K HDMI IMEM 28K HDMI DMEM FOLLOWED BY COPY OF IVT AND BOOT DATA CSF DATA
	.
	.
	.
	.
	.
	.
11	START OF A53 IMAGE COPY 2
	A53 IMAGE IVT AND BOOT DATA
	DCD
	...
	CSF DATA
	...
	...
	A53 IMAGE DATA
	.

Table continues on the next page...

Table 6-22. NAND layout (continued)

NAND BOOT	
BLOCK #	NAND BOOT
	.
	.
	.

6.1.6.3 Device Configuration Data (DCD)

Upon reset, the chip uses the default register values for all peripherals in the system. However, these settings typically are not ideal for achieving the optimal system performance and there are even some peripherals that must be configured before they can be used.

The DCD is a configuration information contained in the program image (external to the ROM) that the ROM interprets to configure various peripherals on the chip.

For example, the EIM default settings allow the core to interface to a NOR flash device immediately after the reset. This allows the chip to interface with any NOR flash device, but has the disadvantage of slow performance. Additionally, some components (such as DDR) require some sequence of register programming as a part of the configuration before it is ready to be used. The DCD feature can be used to program the EIM registers and the DDR Controller registers to the optimal settings.

The ROM determines the location of the DCD table based on the information located in the Image Vector Table (IVT). See [Image Vector Table and Boot Data](#) for more details. The DCD table shown below is a big-endian byte array of the allowable DCD commands. The maximum size of the DCD is limited to 1768 B.

Header
[CMD]
[CMD]
...

Figure 6-20. DCD data format

The DCD header is 4 B with the following format:

Tag	Length	Version
-----	--------	---------

Figure 6-21. DCD header format

where:

Tag: A single-byte field set to 0xD2

Length: a two-byte field in the big-endian format containing the overall length of the DCD (in bytes) including the header

Version: A single-byte field set to 0x41

6.1.6.3.1 Write data command

The write data command is used to write a list of given 1-, 2- or 4-byte values (or bitmasks) to a corresponding list of target addresses.

The format of the write data command (in a big-endian byte array) is shown in this table:

Table 6-23. Write data command format

Tag	Length	Parameter
	Address	
	Value/Mask	
	[Address]	
	[Value/Mask]	
	...	
	[Address]	
	[Value/Mask]	

where:

Tag: a single-byte field set to 0xCC

Length: a two-byte field in a big-endian format, containing the length of the Write Data Command (in bytes) including the header

Address: the target address to which the data must be written

Value/Mask: the data value (or bitmask) to be written to the preceding address

The parameter field is a single byte divided into the bitfields, as follows:

Table 6-24. Write data command parameter field

7	6	5	4	3	2	1	0
flags					bytes		

where

System Boot

bytes: the width of the target locations in bytes (either 1, 2, or 4)

flags: control flags for the command behavior

Data Mask = bit 3: if set, only specific bits may be overwritten at the target address (otherwise all bits may be overwritten)

Data Set = bit 4: if set, the bits at the target address are overwritten with this flag (otherwise it is ignored)

One or more target address and value/bitmask pairs can be specified. The same bytes' and flags' parameters apply to all locations in the command.

When successful, this command writes to each target address in accordance with the flags as follows:

Table 6-25. Interpretation of write data command flags

"Mask"	"Set"	Action	Interpretation
0	0	*address = val_msk	Write value
0	1	*address = val_msk	Write value
1	0	*address &= ~val_msk	Clear bitmask
1	1	*address = val_msk	Set bitmask

NOTE

If any of the target addresses does not have the same alignment as the data width indicated in the parameter field, none of the values are written.

If any of the values are larger or any of the bitmasks are wider than permitted by the data width indicated in the parameter field, none of the values are written.

If any of the target addresses do not lie within the allowed region, none of the values are written. The list of allowable blocks and target addresses for the chip are provided below.

6.1.6.3.2 Check data command

The check data command is used to test for a given 1-, 2-, or 4-byte bitmasks from a source address.

The check data command is a big-endian byte array with the format shown in this table:

Table 6-26. Check data command format

Tag	Length	Parameter
	Address	
	Mask	
	[Count]	

where:

Tag: a single-byte field set to 0xCF

Length: a two-byte field in the big-endian format containing the length of the check data command (in bytes) including the header

Address: the source address to test

Mask: the bit mask to test

Count: an optional poll count; If the count is not specified, this command polls indefinitely

until the exit condition is met. If count = 0, this command behaves as for the NOP.

The parameter field is a single byte divided into bitfields, as follows:

Table 6-27. Check data command parameter field

7	6	5	4	3	2	1	0
flags					bytes		

where

bytes: the width of target locations in bytes (either 1, 2, or 4)

flags: control flags for the command behavior

Data Mask = bit 3: if set, only the specific bits may be overwritten at a target address (otherwise all bits may be overwritten)

Data Set = bit 4: if set, the bits at the target address are overwritten with this flag (otherwise it is ignored)

This command polls the source address until either the exit condition is satisfied, or the poll count is reached. The exit condition is determined by the flags as follows:

Table 6-28. Interpretation of check data command flags

"Mask"	"Set"	Action	Interpretation
0	0	(*address & mask) == 0	All bits clear
0	1	(*address & mask) == mask	All bits set
1	0	(*address & mask) != mask	Any bit clear
1	1	(*address & mask) != 0	Any bit set

NOTE

If the source address does not have the same alignment as the data width indicated in the parameter field, the value is not read.

If the bitmask is wider than permitted by the data width indicated in the parameter field, the value is not read.

6.1.6.3.3 NOP command

This command has no effect.

The format of the NOP command is a big-endian four-byte array, as shown in this table:

Table 6-29. NOP command format

Tag	Length	Undefined
-----	--------	-----------

where:

Tag: a single-byte field set to 0xC0
Length: a two-byte field in big endian containing the length of the NOP command in bytes (fixed to a value of 4)
Undefined: this byte is ignored and can be set to any value.

6.1.6.3.4 Unlock command

The unlock command is used to prevent specific engine features from being locked when exiting the ROM.

The format of the unlock command (in a big-endian byte array) is shown in this table:

Table 6-30. Unlock command format

Tag	Length	Eng
	Value	
	Value	
	...	
	Value	

where:

NOTE

This command may not be used in the DCD structure if the SEC_CONFIG is configured as closed.

6.1.7 Plugin image

The ROM supports a limited number of boot devices. When using other devices as a boot source (for example, Ethernet, CDRom, or USB), the supported boot device must be used (typically serial ROM) as a firmware to provide the missing boot drivers. Additionally, the plugin can be customized to support boot drivers, which is more flexible when performing the device initialization, such as condition judging, delay assertion, or to apply custom settings to the boot device and memory system.

In addition to the standard images, the chip also supports plugin images. The plugin images return the execution to the ROM whereas the standard image does not.

The boot ROM detects the image type using the plugin flag of the boot data structure (see [Boot data structure](#)). If the plugin flag is 1, then the ROM uses the image as a plugin function. The function must initialize the boot device and copy the program image to the final location. At the end, the plugin function must return with the program image parameters. (See [High-level boot sequence](#) for details about the boot flow).

The boot ROM authenticates the plugin image before running the plugin function and then authenticates the program image.

The plugin function must follow the API described below:

```
typedef BOOLEAN (*plugin_download_f)(void **start, size_t *bytes, UINT32
*ivt_offset);
```

ARGUMENTS PASSED:

- start - the image load address on exit.
- bytes - the image size on exit.
- ivt_offset - the offset (in bytes) of the IVT from the image start address on exit.

RETURN VALUE:

- 1 - success
- 0 - failure

6.1.8 Serial Downloader

The Serial Downloader provides a means to download a program image to the chip over the USB serial connection.

In this mode, the ROM programs the WDOG1 for a time-out specified by the fuse WDOG Time-out Select (See fusemap for details) if the WDOG_ENABLE eFuse is 1 and continuously polls for the USB connection. If no activity is found on the USB OTG1 and the watchdog timer expires, the Arm core is reset.

NOTE

After the downloaded image is loaded, it is responsible for managing the watchdog resets properly.

This figure shows the USB boot flow:

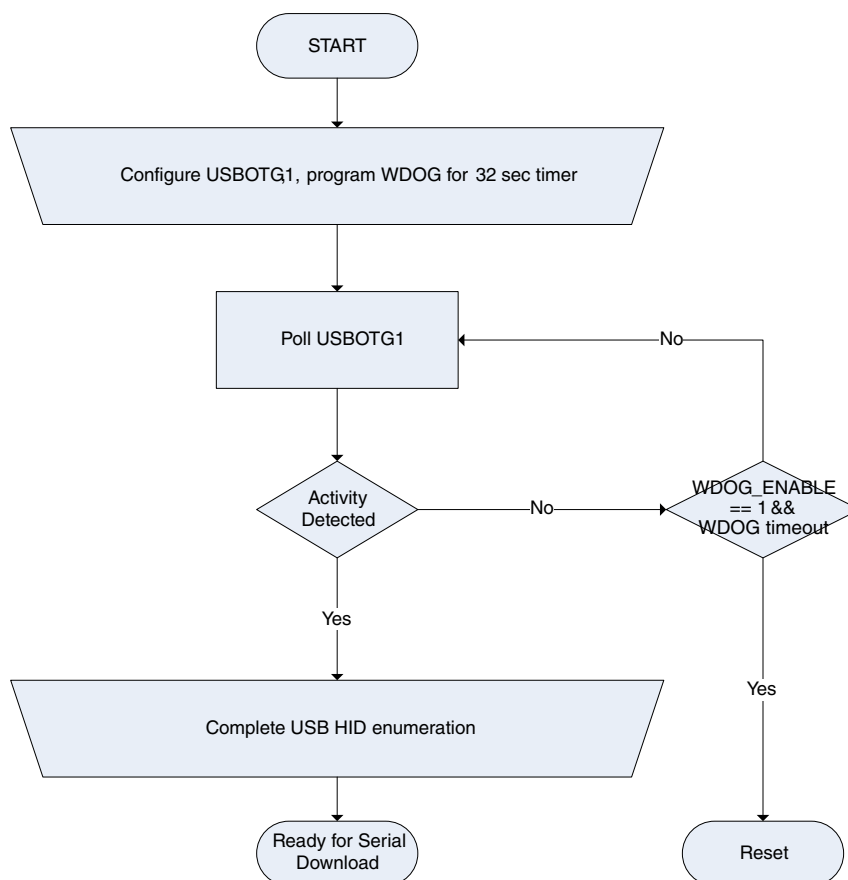


Figure 6-22. Serial Downloader boot flow

NOTE

Before going into USB serial mode, Boot ROM detect SD/MMC card on USDHC2 port. If a card is inserted, ROM will try to boot from it. This is the so-called Manufacture SD/MMC boot. This feature can be disabled by blowing fuse “Disable SD/MMC Manufacture Mode”. See [SD/MMC manufacture mode](#) for details.

6.1.8.1 USB

The USB support is composed of the USB (core controller, compliant with the USB 3.0 specification) and the USBPHY (HS USB transceiver).

The ROM supports the USB OTG port for boot purposes. The other USB ports on the chip are not supported for boot purposes.

The USB Driver is implemented as a USB HID class. A collection of four HID reports are used to implement the SDP protocol for data transfers, as described in [Table 6-31](#).

Table 6-31. USB HID reports

Report ID (first byte)	Transfer endpoint	Direction	Length	Description
1	control OUT	Host to device	17 B	SDP command from the host to the device.
2	control OUT	Host to device	Up to 1025 B	Data associated with the report 1 SDP command.
3	interrupt	Device to host	5 B	HAB security configuration. The device sends 0x12343412 in the closed mode and 0x56787856 in the open mode.
4	interrupt	Device to host	Up to 65 B	Data in response to the SDP command in report 1.

6.1.8.1.1 USB configuration details

The USB OTG function device driver supports a high speed (HS for UTMI) non-stream mode with a maximal packet size of 512 B and a low-level USB OTG function.

The VID/PID and strings for the are listed in the following table.

Table 6-32. VID/PID and strings for USB device driver

Descriptor	Value
VID (NXP vendor ID)	0x1FC9
PID ¹	0x012B
String Descriptor1 (manufacturer)	NXP Semiconductors
String Descriptor2 (product)	SE Blank M850 SP Blank M850 NS Blank M850 FR Blank M850
String Descriptor4	NXP Flash
String Descriptor5	NXP Flash

1. Allocation based on the BPN (Before Part Number)

6.1.8.1.2 IOMUX configuration for USB

The interface signals of the UTMI PHY are not configured in the IOMUX,, except for the USB_OTGn_ID pins. The USB ID pin function is configured using the USBNC_n_CTRL2[ID_DIG_SEL] and the IOMUXC_USB_OTGn_ID_SELECT_INPUT register. The remaining pins of the UTMI PHY interface use the dedicated contacts on the IC. See the chip data sheet for details.

6.1.8.2 Serial Download Protocol (SDP)

The 16-byte SDP command from the host to device is sent using the HID report 1.

This table describes the 16-byte SDP command data structure:

Table 6-33. 16-byte SDP command data structure

BYTE offset	Size	Name	Description
0	2	COMMAND TYPE	These commands are supported for the ROM: • 0x0101 READ_REGISTER • 0x0202 WRITE_REGISTER • 0x0404 WRITE_FILE • 0x0505 ERROR_STATUS • 0x0A0A DCD_WRITE • 0x0B0B JUMP_ADDRESS
2	4	ADDRESS	Only relevant for these commands: READ_REGISTER, WRITE_REGISTER, WRITE_FILE, DCD_WRITE, and JUMP_ADDRESS. For the READ_REGISTER and WRITE_REGISTER commands, this field is the address to a register. For the WRITE_FILE and JUMP_ADDRESS commands, this field is an address to the internal or external memory address.
6	1	FORMAT	Format of access, 0x8 for an 8-bit access, 0x10 for a 16-bit access, and 0x20 for a 32-bit access. Only relevant for the READ_REGISTER and WRITE_REGISTER commands.
7	4	DATA COUNT	Size of the data to read or write. Only relevant for the WRITE_FILE, READ_REGISTER, WRITE_REGISTER, and DCD_WRITE commands. For the WRITE_FILE and DCD_WRITE commands, the DATA COUNT is in the byte units.
11	4	DATA	The value to write. Only relevant for the WRITE_REGISTER command.
15	1	RESERVED	Reserved

6.1.8.2.1 SDP commands

The SDP commands are described in the following sections.

6.1.8.2.1.1 READ_REGISTER

The transaction for the READ_REGISTER command consists of these reports: Report1 for the command, Report3 for the security configuration, and Report4 for the response or the register value.

The register to read is specified in the ADDRESS field of the SDP command. The first device sends Report3 with the security configuration followed by the Report4 with the bytes read at a given address. If the count is greater than 64, multiple reports with the report id 4 are sent until the entire data requested by the host is sent. The STATUS is either 0x12343412 for the closed parts and 0x56787856 for the open or field return parts.

Report1, Command, Host to Device:

1	Valid values for the READ_REGISTER COMMAND, ADDRESS, FORMAT, DATA_COUNT
---	---

ID 16-byte SDP command

Report3, Response, Device to Host:

3	Four bytes indicating the security configuration
---	--

ID 4 bytes status

Report4, Response, Device to Host: first response report

4	Register value
---	----------------

ID 4 bytes of data containing the register value. If the number of bytes requested is less than 4, the remaining bytes must be ignored by the host.

Multiple reports of the report id 4 are sent until the entire requested data is sent.

Report4, Response, Device to Host: last response report

4	Register value
---	----------------

ID 64 bytes of data containing the register value. If the number of bytes requested is less than 64, the remaining bytes must be ignored by the host.

6.1.8.2.1.2 WRITE_REGISTER

The transaction for the WRITE_REGISTER command consists of these reports: Report1 for the command, Report3 for the security configuration and Report4 for the write status.

The host sends Report1 with the WRITE_REGISTER command. The register to write is specified in the ADDRESS field of the SDP command of Report1, with the FORMAT field set to the data type (number of bits to write, either 8, 16, or 32) and the value to write in the DATA field of the SDP command. The device writes the DATA to the register address and returns the WRITE_COMPLETE code using Report4 and the security configuration using Report3 to complete the transaction.

Report1, Command, Host to Device:

1	Valid values for WRITE_REGISTER COMMAND, ADDRESS, FORMAT, DATA_COUNT and DATA
---	---

ID 16-byte SDP command

Report3, Response, Device to Host:

3	4 bytes indicating the security configuration
---	---

ID 4 bytes status

Report4, Response, Device to Host:

4	WRITE_COMPLETE (0x128A8A12) status
---	------------------------------------

ID 64 bytes data with the first 4 bytes to indicate that the write is completed with code 0x128A8A12. On failure, the device reports the HAB error status.

6.1.8.2.1.3 WRITE_FILE

The transaction for the WRITE_FILE command consists of these reports: Report1 for the command phase, Report2 for the data phase, Report3 for the HAB mode, and Report4 to indicate that the data are received in full.

The size of each Report2 is limited to 1024 bytes (limitation of the USB HID protocol). Hence, multiple Report2 packets are sent by the host in the data phase until the entire data is transferred to the device. When the entire data (DATA_COUNT bytes) is received, the device sends Report3 with the HAB mode and Report4 with 0x88888888, indicating that the file download completed.

Report1, Host to Device:

1	Valid values for WRITE_FILE COMMAND, ADDRESS, DATA_COUNT
---	--

ID 16-byte SDP command

=====Optional Begin=====

Host sends the ERROR_STATUS command to query if the HAB rejected the address

===== Optional End=====

Report2, Host to Device:

2	File data
---	-----------

ID Max 1024 bytes data per report

Report2, Host to Device:

2	File data
---	-----------

ID Max 1024 bytes data per report

Report3, Device to Host:

3	4 bytes indicating security configuration
---	---

ID 4 bytes status

Report4, Response, Device to Host:

4	COMPLETE (0x88888888) status
---	------------------------------

ID 64 bytes data with the first four bytes to indicate that the file download completed with code 0x88888888. On failure, the device reports the HAB error status.

6.1.8.2.1.4 ERROR_STATUS

The transaction for the SDP command ERROR_STATUS consists of three reports.

Report1 is used by the host to send the command; the device sends global error status in four bytes of Report4 after returning the security configuration in Report3. When the device receives the ERROR_STATUS command, it returns the global error status that is updated for each command. This command is useful to find out whether the last command resulted in a device error or succeeded.

Report1, Command, Host to Device:

1	ERROR_STATUS COMMAND
---	----------------------

ID 16-byte SDP Command

Report3, Response, Device to Host:

3	Four bytes indicating the security configuration
---	--

ID 4 bytes status

Report4, Response, Device to Host:

4	Four bytes Error status
---	-------------------------

ID first 4 bytes status in 64 bytes Report4

6.1.8.2.1.5 DCD_WRITE

The SDP command DCD_WRITE is used by the host to send multiple register writes in one shot. This command is provided to speed up the process of programming the register writes (such as to configure an external RAM device).

The command goes with Report1 from the host with COMMAND TYPE set to DCD_WRITE, ADDRESS which is used as a temporary location of the DCD data, and DATA_COUNT to the number of bytes sent in the data out phase. In the data phase, the host sends the data for a number of registers using Report2. The device completes the transaction with Report3 indicating the security configuration and Report4 with the WRITE_COMPLETE code 0x12828212.

Report1, Command, Host to Device:

1	DCD_WRITE COMMAND, ADDRESS, DATA_COUNT
---	--

ID 16-byte SDP Command

Report2, Data, Host to Device:

2	DCD binary data
---	-----------------

ID Max 1024 bytes per report

Report3, Response, Device to Host:

3	Four bytes indicating the security configuration
---	--

ID 4 bytes status

Report4, Response, Device to Host:

4	WRITE_COMPLETE (0x128A8A12) status
---	------------------------------------

ID 64 bytes report with the first four bytes to indicate that the write completed with the code 0x128A8A12. On failure, the device reports the HAB error status.

See [Device Configuration Data \(DCD\)](#) for the DCD format description.

6.1.8.2.1.6 SKIP_DCD_HEADER

The SDP command SKIP_DCD_HEADER is used by the host to inform the device to skip the DCD configuration within the download image.

If the download image must be run on the DDR, the DCD configuration data must be built into the image. In case the host issued DCD_WRITE to push the DCD configuration data to the device for the DDR initialization, the image with the DCD information causes the ROM to initialize the DDR twice, and may cause the initialization processing to hang.

The SKIP_DCD_HEADER command informs the device to skip the DCD configuration within the download image and avoid this issue.

This command is typically sent after JUMP_ADDRESS. This command is sent by the host in the command-phase of the transaction using Report1, there is no data phase for this command. The device completes the transaction with Report3 indicating the security configuration and Report4 with the OK_ACK code 0x900DD009.

Report1, Command, Host to Device:

1	SKIP_DCD_HEADER
---	-----------------

ID 16-byte SDP Command

Report3, Response, Device to Host:

3	Four bytes indicating the security configuration
---	--

ID 4 bytes status

Report4, Response, Device to Host:

4	OK_ACK (0x900DD009)
---	---------------------

6.1.8.2.1.7 JUMP_ADDRESS

The SDP command JUMP_ADDRESS is the last command that the host can send to the device. After this command, the device jumps to the address specified in the ADDRESS field of the SDP command and starts to execute.

This command usually follows after the WRITE_FILE command. The command is sent by the host in the command-phase of the transaction using Report1. There is no data phase for this command, but the device sends the status Report3 to complete the transaction. If the authentication fails, it also sends Report4 with the HAB error status.

Report1, Command, Host to Device:

1	JUMP_ADDRESS COMMAND, ADDRESS
---	-------------------------------

ID 16-byte SDP Command

Report3, Response, Device to Host:

3	Four bytes indicating the security configuration
---	--

ID 4 bytes status

This report is sent by the device only in case of an error jumping to the given address, or if the device reports error in Report4, Response, Device to Host:

4	Four bytes HAB error status
---	-----------------------------

ID 4 bytes status, 64 bytes report length

6.1.9 Low-power boot

The ROM supports the low-power boot. If the LPB_BOOT fuses are blown, the chip checks if there is a low-power condition via the pad. If there is a low-power boot condition, the ROM applies division factors on the ARM, DDR, AXI, and AHB root clocks based on the LPB_BOOT fuses value (see the table below). The polarity of the low-power boot condition on the pad is set by the BT_LPB_POLARITY fuse (see the following figure).

Table 6-34. Low-power boot frequencies

LPB_BOOT	Boot Frequencies=0	Boot Frequencies=1
00	ARM_A53_CLK_ROOT= 1000 MHz AHB_CLK_ROOT= 133 MHz MAIN_AXI_CLK_ROOT= 333 MHz	ARM_A53_CLK_ROOT= 500 MHz AHB_CLK_ROOT= 133 MHz MAIN_AXI_CLK_ROOT= 166 MHz
01	ARM_A53_CLK_ROOT= 1000 MHz AHB_CLK_ROOT= 133 MHz MAIN_AXI_CLK_ROOT= 333 MHz	ARM_A53_CLK_ROOT= 500 MHz AHB_CLK_ROOT= 133 MHz MAIN_AXI_CLK_ROOT= 166 MHz
10	ARM_A53_CLK_ROOT= 500 MHz AHB_CLK_ROOT= 133 MHz MAIN_AXI_CLK_ROOT= 166 MHz	ARM_A53_CLK_ROOT= 250 MHz AHB_CLK_ROOT= 133 MHz MAIN_AXI_CLK_ROOT= 83.3 MHz
11	ARM_A53_CLK_ROOT= 250 MHz AHB_CLK_ROOT= 133 MHz MAIN_AXI_CLK_ROOT= 83.3 MHz	ARM_A53_CLK_ROOT= 250 MHz AHB_CLK_ROOT= 133 MHz MAIN_AXI_CLK_ROOT= 41.67 MHz

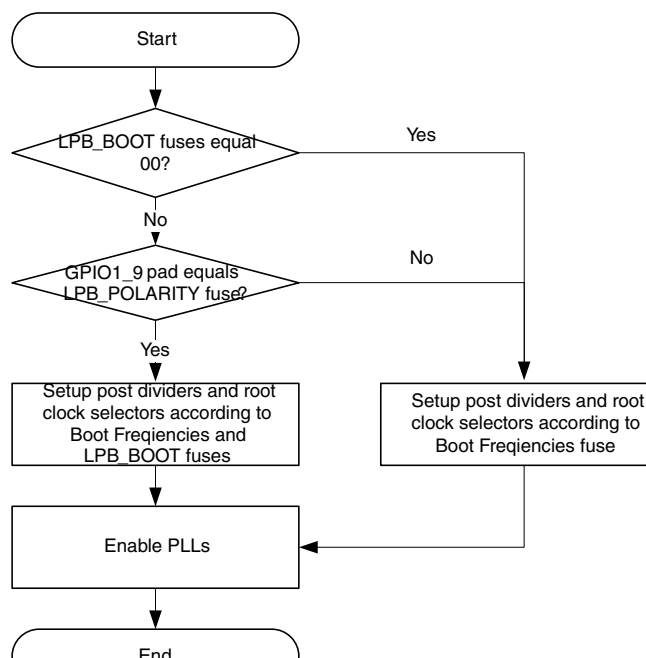


Figure 6-23. Low-power boot flow

6.1.10 SD/MMC manufacture mode

When the internal boot and recover boot (if enabled) failed, the boot goes to the SD/MMC manufacture mode before the serial download mode. In the manufacture mode, one bit bus width is used despite of the fuse setting.

By default, the SD/MMC manufacture mode is enabled. Blow the fuse of the `DISABLE_SDMMC_MFG` to disable it.

NOTE

A secondary boot is not supported in the SD/MMC manufacture mode.

NOTE

ROM does not support loading of HDMI image in SD/MMC manufacturing mode.

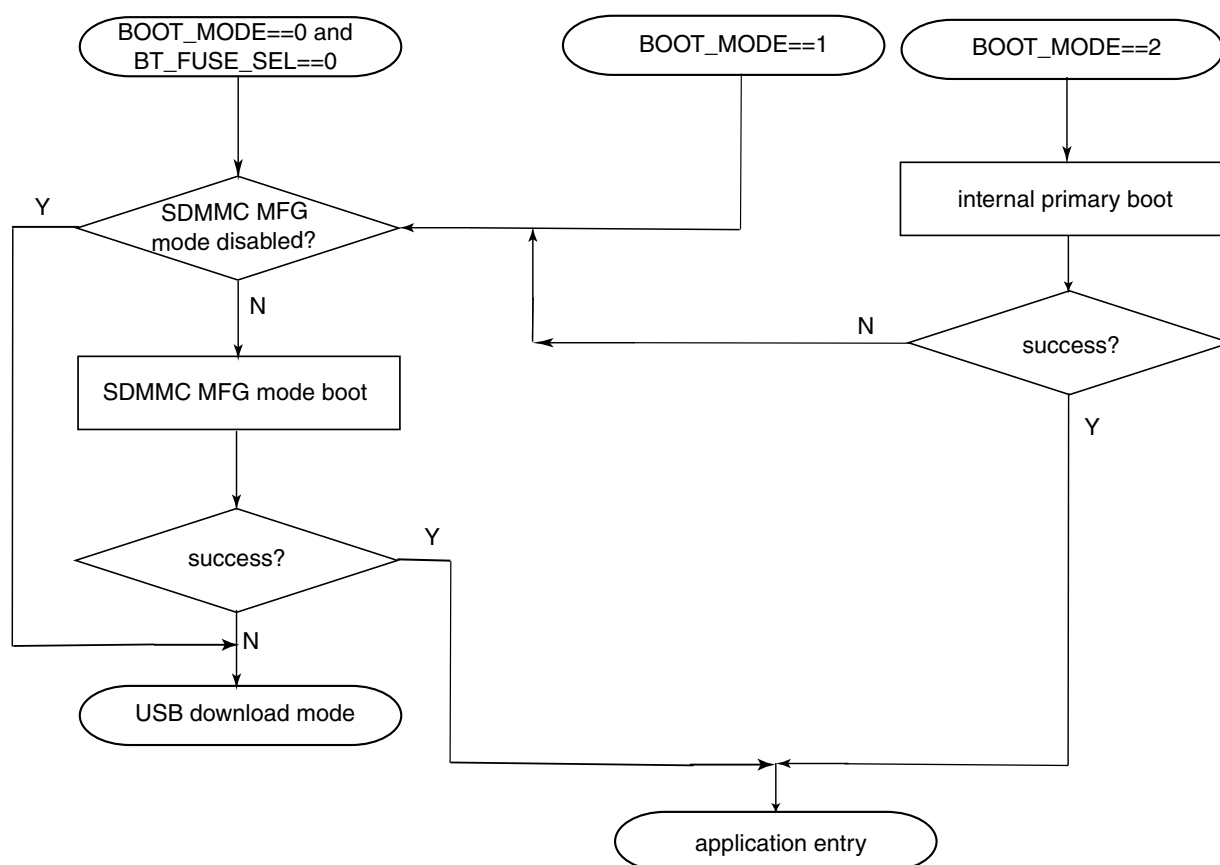


Figure 6-24. SD/MMC manufacture boot flow

NOTE

The EEPROM recovery mode is not supported on this chip.

6.1.10.1 Using manufacture mode / serial download mode with eMMC

Manufacture mode is intended to allow a system to boot from a SD/MMC card on a board with unprogrammed boot media or to upgrade the image on a boot device. For manufacture mode, the boot ROM assumes if there is a SD/MMC card present indicated by the uSDHC card detect (CD) signal pulled low, then there is a valid image on the card.

If an unprogrammed eMMC device is connected to the uSDHC port(s) on which manufacture mode is supported and the CD signal is low, the ROM will attempt manufacturing mode with the eMMC device, which does not contain a valid image. The ROM loads the invalid image, and this will cause the ROM code to fail to enter serial download mode, resulting in a reset of the system by the ROM. To enter serial download mode in this case, the CD pin should be pulled up so the ROM does not detect a eMMC device present and will bypass the manufacture mode to enter serial download mode.

6.1.11 High-Assurance Boot (HAB)

The High Assurance Boot (HAB) component of the ROM protects against the potential threat of attackers modifying the areas of code or data in the programmable memory to make it behave in an incorrect manner. The HAB also prevents the attempts to gain access to features which must not be available.

The integration of the HAB feature with the ROM code ensures that the chip does not enter an operational state if the existing hardware security blocks detected a condition that may be a security threat or if the areas of memory deemed to be important were modified. The HAB uses the RSA digital signatures to enforce these policies.

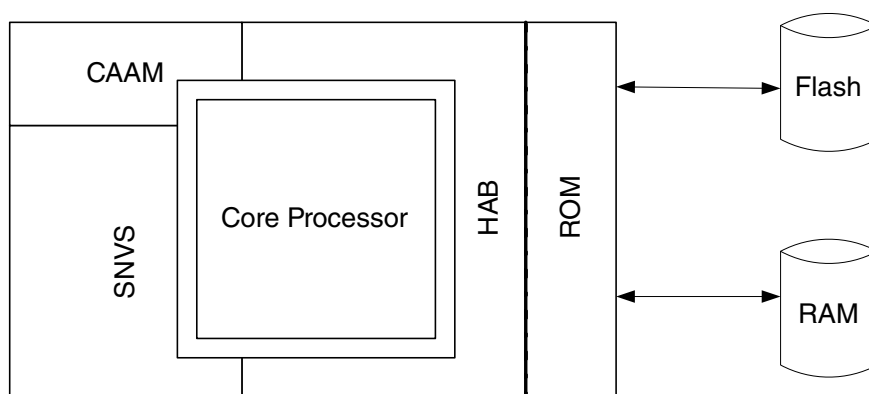


Figure 6-25. Secure boot components

The figure above illustrates the components used during a secure boot using HAB. The HAB interfaces with the SNVS to make sure that the system security state is as expected. The HAB also uses the CAAM hardware block to accelerate the SHA-256 message digest operations performed during the signature verifications and AES-128 operations for the encrypted boot operations. The HAB also includes a software implementation of SHA-256 for cases where a hardware accelerator can't be used. The RSA key sizes supported are 1024, 2048, 3072, and 4096 bits. The RSA signature verification operations are performed by a software implementation contained in the HAB library. The main features supported by the HAB are:

- X.509 public key certificate support

- CMS signature format support
- Proprietary encrypted boot support. Note that the encrypted boot depends on the CAAM hardware module. When the CAAM is disabled (when the EXPORT_CONTROL fuse is blown), the encrypted boot is not available.

NOTE

NXP provides the reference Code Signing Tool (CST) for key generation, certificate generation, and code signing for use with the HAB library. The CST can be found by searching for "IMX_CST_TOOL" at <http://www.nxp.com>.

NOTE

For further details on using the secure boot feature using HAB, refer to *Secure Boot on i.MX 50, i.MX 53, i.MX 6 and i.MX 7 Series using HABv4 (AN4581)*.

6.1.11.1 HAB API vector table addresses

For devices that perform a secure boot, the HAB library may be called by the boot stages that execute after the ROM code.

NOTE

For additional information on the secure boot including the HAB API, refer to *HABv4 RVT Guidelines and Recommendations (AN12263)*.

6.1.12 Boot information for software

To address the requirement that the boot image may need to get the basic boot information when getting out of the boot process, the boot software information (Boot_SW_Info) is provisioned by the ROM.

The base address of the Boot_SW_Info (the SW information pointer is recorded at 0x9e8 for ROM A0 and 0x968 for ROM B0). The software must read 0x9e8 for ROM A0 or 0x968 for ROM B0 to get the base address of the Boot_SW_Info, and parse the Boot_SW_Info content to get the boot information.

Table 6-35. Boot_SW_Info structure

Offset	Byte3	Byte2	Byte1	Byte0
0x0	Reserved	Boot Device Type	Boot Device Instance	Reserved
0x4	Arm core frequency (in Hz)			

Table continues on the next page...

**Table 6-35. Boot_SW_Info structure
(continued)**

Offset	Byte3	Byte2	Byte1	Byte0
0x8	AXI bus frequency (in Hz)			
0xC	DDR frequency (in Hz)			
0x10	GPT1 input clock frequency (in Hz)			
0x14	Reserved			
0x18				
0x1C				

NOTE

The boot ROM sets the GPT1 in a free-running mode with a 32-kHz input clock.

Boot device type mapping:

- 0x1 - SD card or eSD chip
- 0x2 - MMC card or eMMC chip
- 0x3 - NAND chip

Boot device instance: The instance index of the boot device, starting from 0.

6.2 Fusemap

6.2.1 Boot Fusemap

The following section details the various modes and selection of the required boot devices. A separate map is given for each and every boot device. The device select is specified by BOOT_CFG[14:12] fuses listed below.

Table 6-36. Boot Device Select

Boot Device	BOOT_CFG[14]	BOOT_CFG[13]	BOOT_CFG[12]
SD/eSD	0	0	1
MMC/eMMC	0	1	0
NAND	0	1	1

NOTE

Fuses marked as “Reserved” are reserved for NXP internal (and future) use only. Customers should not attempt to burn these, as

the IC behavior may be unpredictable. The reserved fuses can be read as either 0 or 1.

Table 6-37. SD/eSD Boot Fusemap

Addr	7	6	5	4	3	2	1	0
0x470[15:8]	Reserved	Boot Device Select			Port Select: 00 - SD1 01 - SD2		Power Cycle Enable 0 - Disable 1 - Enable	SD Loopback Clock Source SEL (SDR50 and SDR104 Only) 0 - Disable 1 - Enable
0x470[7:0]	Fast Boot: 0 - Regular 1 - Fast Boot	Reserved		Bus Width: 0 - 1-bit 1 - 4-bit	Speed 000 - Normal/SDR12 001 - High/SDR25 010 - SDR50 011 - SDR104 101 - Reserved for DDR50 Others - Reserved			Reserved

Table 6-38. MMC/eMMC Boot Fusemap

Addr	7	6	5	4	3	2	1	0
0x470[15:8]	Reserved	Boot Device Select			Port Select: 00 - SD1 01 - SD2		Power Cycle Enable 0 - Disable 1 - Enable	SD Loopback Clock Source SEL (SDR50 and SDR104 Only) 0 - Disable 1 - Enable
0x470[7:0]	Fast Boot: 0 - Regular 1 - Fast Boot	Bus Width: 000 - 1-bit 001 - 4-bit 010 - 8-bit 101 - 4-bit DDR (MMC 4.4) 110 - 8-bit DDR (MMC 4.4) Else - reserved			Speed 00 - Normal 01 - High 10 - Reserved for HS200 11 - Reserved		USDHC1 IO VOLTAGE SELECTION 0 - 3.3V 1 - 1.8V	USDHC2 IO VOLTAGE SELECTION 0 - 3.3V 1 - 1.8V

Table 6-39. NAND Boot Fusemap

Addr	7	6	5	4	3	2	1	0
0x470[15:8]	Reserved	Boot Device Select			Pages in Block: 00 - 128 01 - 64 10 - 32 11 - 256		NAND Row Address Bytes: 00 - 3 01 - 2 10 - 4 11 - 5	
0x470[7:0]	BT_TOGGL EMODE 0 - Raw NAND 1 - Toggle mode	BOOT_SEARCH_COUN T 00 - 2 01 - 2 10 - 4 11 - 8		Toggle Mode 33MHz Preamble Delay, Read Latency 000 - 16 GPMICLK cycles 001 - 1 GPMICLK cycles 010 - 2 GPMICLK cycles 011 - 3 GPMICLK cycles 100 - 4 GPMICLK cycles 101 - 5 GPMICLK cycles 110 - 6 GPMICLK cycles 111 - 7 GPMICLK cycles 1111- 15 GPMICLK cycles				Reserved

Table 6-40. Boot Fusemap

Addr	7	6	5	4	3	2	1	0
0x480[7:0]	Reserved							
0x480[15:8]	LPB_BOOT (Core/DDR/ Bus) 00/01 - LPB Disable 10 - Div by2 11 - Div by 4	BT_LPB_P OLARITY (GPIO polarity)	L1 I-Cache DISABLE	TZASC_EN ABLE	WDOG_EN ABLE 0 - Disabled 1 - Enabled	Boot Frequencies (ARM/DDR) 0 - 800 / 800 MHz 1 - 400 / 400 MHz	Reserved	
0x480[23:16]	NOC_ID_R EMAP_BYP ASS	SDP_READ _DISABLE	SDP_DISA BLE	FORCE_IN TERNAL_B OOT	Reserved	WDOG Timeout Select 000 - 64s 001 - 32s 010 - 16s 011 - 8s 100 - 4s Others - Reserved		
0x480[31:24]	Reserved						UART Serial Download Disable 0 - Enabled	

Table continues on the next page...

Table 6-40. Boot Fusemap (continued)

Addr	7	6	5	4	3	2	1	0
								1 - Disabled
0x490[7:0]	Reserved for uSDHC future use							
0x490[15:8]	USDHC DLL Select 0 - DLL Slave Mode for 1 - DLL Override Mode	MMC_DLL_DLY[6:0] Delay target for USDHC DLL, it is applied to slave mode target delay or override mode target delay depends on DLL Override fuse bit value.						
0x490[23:16]	Disable SDMMC Manufacture mode 0 - Enable 1 - Disable	USDHC_CMD_OE_PRE_EN (SD/MMC debug)	eMMC 4.4 - RESET TO PRE-IDLE STATE 0 - enable 1 - disable	USDHC_PAD_PULL_DOWN 0 - no action 1 - pull down	ENABLE_EMMC_22K_PULLUP 0 - 47K pullup 1 - 22K pullup	USDHC_IO_MUX_SION_BIT_ENABLE 0 - Disable 1 - Enable	USDHC Override Pad Settings (using PAD_SETTINGS value)	USDHC_DLL_ENABLE 0 - Disable DLL for SD/eMMC 1 - Enable DLL for SD/eMMC
0x490[31:24]	USDHCPAD_SETTINGS[7:0]							
0x4A0[7:0]	SD Calibration Step 00 - 1		uSDHC Power Cycle Interval 00 - 20ms 01 - 10ms 10 - 5ms 11 - 2.5		uSDHC Power Cycle Delay 0 - 5ms 1 - 2.5ms	uSDHC Power On Polarity 0 - Low 1 - High	DCD Size 0 - Default Setting 1 - Force to 100K	Fast Boot Acknowledge Disable 0 - Boot Ack Disabled 1 - Boot Ack Enabled
0x4A0[15:8]	NAND_READ_CMD_CODE1[7:0]							
0x4A0[23:16]	NAND_READ_CMD_CODE2[7:0]							
0x4A0[31:24]	NAND_PAD_SETTINGS[7:0]							
0x4B0[7:0]	Override NAND Pad Settings (using PAD_SETTINGS value)	GPMI Read DDR DLL Target Value 0000 - 7 0001 - 1 0111 - 0 1111 - 15				0x4B0[1:0] NAND Number of Devices: 00 - 1 01 - 2 10 - 4 11 - Reserved		
0x4B0[15:8]	READ_RETRY_SEQ_ID[3:0] 0000 - do not use read retry (RR) sequence embedded in ROM 0001 - Micron 20nm RR sequence 0010 - Toshiba A19nm RR sequence 0011 - Toshiba 19nm RR sequence 0100 - SanDisk 19nm RR sequence				Reserved			

Table continues on the next page...

Table 6-40. Boot Fusemap (continued)

Addr	7	6	5	4	3	2	1	0
	0101 - SanDisk 1ynmRR sequence 0110 - Hynix 20nm A Die RR sequence 0111 - Hynix 26nm RR sequence 1000 - Hynix 20nm B Die RR sequence 1001 - Hynix 20nm C Die RR sequence Others - Reserved							
0x4B0[23:16]	Reserved							
0x4B0[31:24]	RNG_TRIM[7:0]							

6.2.2 Lock Fusemap

Table 6-41 describes the functions of various lock fuses.

Table 6-41. Lock Fuses

Addr	7	6	5	4	3	2	1	0
0x400[7:0]	Reserved		Reserved		BOOT_CFG_LOCK 1x - OP x1 - WP		TESTER_LOCK 1x - OP x1 - WP	
0x400[15:8]	MAC_ADDR_LOCK 1x - OP x1 - WP		USB_ID_LOCK 1x - OP x1 - WP		Reserved	SJC_RESP_LOCK WRP,OP,R DP	SRK_LOCK	Reserved
0x400[23:16]	GP2_LOCK 1x - OP x1 - WP		GP1_LOCK 1x - OP x1 - WP		Reserved		Reserved	Reserved
0x400[31:24]	Reserved							

NOTE

TESTER_LOCK programmed by NXP / set at factory

6.2.3 Fusemap Descriptions Table

NOTE

Definitions for fuse settings are as follows:

- Unlock - The controlled field can be read, sensed, burned, or overridden in the corresponding OCOTP shadow register.

- **Override Project (OP)** - The controlled field can be read, sensed, or burned in the corresponding OCOTP shadow register.
- **Write Project (WP)** - The controlled field can be read, sensed, or overridden in the corresponding OCOTP shadow register.
- **OP + WP** - The controlled field can only be read or sensed in the corresponding OCOTP shadow register. It cannot be burned or overridden.
- **Lock** - The controlled field cannot be read, burned, or overridden.

Table 6-42. Fusemap Descriptions

Fuse Address	Fuses Name	Number of Fuses	Fuses Function	Setting	Used by
0x400[1:0]	TESTER_LOCK	2	Lock for tester related fuses at 0x400-0x460.	00 - Unlock (The controlled field can be read, sensed, burned or overridden in the corresponded OCOTP shadow register) 10 - OP (Override Protect, the controlled field can be read, sensed or burned, but can't be overridden in the corresponded OCOTP shadow register) 01 - WP (Write Protect, the controlled field can be read, sensed or overridden in the corresponded OCOTP shadow register, but can't be burned) 11 - OP + WP (The controlled field can be read or sensed only, but can't be burned or overridden in the corresponded OCOTP shadow register)	OCOTP
0x400[3:2]	BOOT_CFG_LOCK	2	Lock for BOOT related fuses at 0x470-4B0.	00 - Unlock (The controlled field can be read, sensed, burned or overridden in the corresponded OCOTP shadow register) 10 - OP (Override Protect, the controlled field can be read, sensed or burned, but can't be overridden in the corresponded OCOTP shadow register)	OCOTP

Table continues on the next page...

Table 6-42. Fusemap Descriptions (continued)

Fuse Address	Fuses Name	Number of Fuses	Fuses Function	Setting	Used by
				01 - WP (Write Protect, the controlled field can be read, sensed or overridden in the corresponded OCOTP shadow register, but can't be burned) 11 - OP + WP (The controlled field can be read or sensed only, but can't be burned or overridden in the corresponded OCOTP shadow register)	
0x400[8:4]	Reserved	5	Reserved	Reserved	Reserved
0x400[9]	SRK_LOCK	1	Lock for SRK_HASH[255:0] fuses.	0 - Unlock (The controlled field can be read, sensed, burned or overridden in the corresponded OCOTP shadow register) 1 - OP + WP (The controlled field can be read or sensed only, but can't be burned or overridden in the corresponded OCOTP shadow register)	OCOTP
0x400[10]	SJC_RESP_LOCK	1	Lock for SJC_RESP[55:0] fuses.	0 - Unlock (The controlled field can be read, sensed, burned or overridden in the corresponded OCOTP shadow register) 1 - Lock (The controlled field can't be read, overridden nor burned)	OCOTP
0x400[11]	Reserved	1	Reserved	Reserved	Reserved
0x400[13:12]	USB_ID_LOCK	2	Lock for USB_PID and USB_VID fuses.	00 - Unlock (The controlled field can be read, sensed, burned or overridden in the corresponded OCOTP shadow register) 10 - OP (Override Protect, the controlled field can be read, sensed or burned, but can't be overridden in the corresponded OCOTP shadow register) 01 - WP (Write Protect, the controlled field can be read, sensed or overridden in the	OCOTP

Table continues on the next page...

Table 6-42. Fusemap Descriptions (continued)

Fuse Address	Fuses Name	Number of Fuses	Fuses Function	Setting	Used by
				corresponded OCOTP shadow register, but can't be burned) 11 - OP + WP (The controlled field can be read or sensed only, but can't be burned or overridden in the corresponded OCOTP shadow register)	
0x400[15:14]	MAC_ADDR_LOCK	2	Lock for MAC_ADDR fuses.	00 - Unlock (The controlled field can be read, sensed, burned or overridden in the corresponded OCOTP shadow register) 10 - OP (Override Protect, the controlled field can be read, sensed or burned, but can't be overridden in the corresponded OCOTP shadow register) 01 - WP (Write Protect, the controlled field can be read, sensed or overridden in the corresponded OCOTP shadow register, but can't be burned) 11 - OP + WP (The controlled field can be read or sensed only, but can't be burned or overridden in the corresponded OCOTP shadow register)	OCOTP
0x400[19:16]	Reserved	4	Reserved	Reserved	Reserved
0x400[21:20]	GP1_LOCK	2	Lock for GP1[63:0] fuses.	00 - Unlock (The controlled field can be read, sensed, burned or overridden in the corresponded OCOTP shadow register) 10 - OP (Override Protect, the controlled field can be read, sensed or burned, but can't be overridden in the corresponded OCOTP shadow register) 01 - WP (Write Protect, the controlled field can be read, sensed or overridden in the	OCOTP

Table continues on the next page...

Table 6-42. Fusemap Descriptions (continued)

Fuse Address	Fuses Name	Number of Fuses	Fuses Function	Setting	Used by															
				corresponded OCOTP shadow register, but can't be burned) 11 - OP + WP (The controlled field can be read or sensed only, but can't be burned or overridden in the corresponded OCOTP shadow register)																
0x400[23:22]	GP2_LOCK	2	Lock for GP2[63:0] fuses.	00 - Unlock (The controlled field can be read, sensed, burned or overridden in the corresponded OCOTP shadow register) 10 - OP (Override Protect, the controlled field can be read, sensed or burned, but can't be overridden in the corresponded OCOTP shadow register) 01 - WP (Write Protect, the controlled field can be read, sensed or overridden in the corresponded OCOTP shadow register, but can't be burned) 11 - OP + WP (The controlled field can be read or sensed only, but can't be burned or overridden in the corresponded OCOTP shadow register)	OCOTP															
0x400[31:24]	Reserved	8	Reserved	Reserved	Reserved															
0x410-0x420	SJC_CHALL[63:0] / UNIQUE_ID[63:0]	32	The SJC CHALLENGE / Unique ID	-	SJC, SW															
0x430	Reserved	32	Reserved	Reserved	Reserved															
0x440[7:0]	Reserved	8	Reserved	Reserved	Reserved															
0x440[13:8]	SPEED_GRADING	2	Burned by tester program, for indicating IC core speed. (Hot burn may not be used).	<table><tr><th>FC A[5:4]</th><th>FH A[3:2]</th><th>FR AL[1:0]</th><th>MH z</th><th>P/N</th></tr><tr><td>xx</td><td>xx</td><td>00</td><td>800</td><td>08</td></tr><tr><td>xx</td><td>xx</td><td>01</td><td>1000</td><td>10</td></tr></table> Table continues on the next page...	FC A[5:4]	FH A[3:2]	FR AL[1:0]	MH z	P/N	xx	xx	00	800	08	xx	xx	01	1000	10	PROD / SW
FC A[5:4]	FH A[3:2]	FR AL[1:0]	MH z	P/N																
xx	xx	00	800	08																
xx	xx	01	1000	10																

Table continues on the next page...

Table 6-42. Fusemap Descriptions (continued)

Fuse Address	Fuses Name	Number of Fuses	Fuses Function	Setting					Used by
				FC A[5:4]	FH A[3:2]	FR AL[1:0]	MH z	P/N	
				xx	xx	10	1300	13	
				xx	xx	11	1500	15	
0x440[31:10]	Reserved	22	Reserved	Reserved					Reserved
0x450[1:0]	NUM_A53_CORES	2	Number of A53 CPU cores available.	00 - 4 cores 01 - 1 cores 10 - 2 cores 11 - Reserved					SRC, SJC, SW
0x450[7:2]	Reserved	6	Reserved	Reserved					Reserved
0x450[8]	M4_DISABLE	1	Disable M4 Core.	0 - enabled 1 - disabled					M4
0x450[9]	M4_MPU_DISABLE	1	Disable M4 MPU IP.	0 - enabled 1 - disabled					M4
0x450[10]	M4_FPU_DISABLE	1	Disable M4 FPU IP.	0 - enabled 1 - disabled					M4
0x450[11]	USB_OTG1_DISABLE	1	Disable USB OTG1 IP.	0 - enabled 1 - disabled					USB OTG1
0x450[12]	USB_OTG2_DISABLE	1	Disable USB OTG2 IP.	0 - enabled 1 - disabled					USB OTG2
0x450[13]	DOLBY_DISABLE	1	Disable Dolby HDR IP.	0 - enabled 1 - disabled					Display
0x450[17:14]	Reserved	4	Reserved	Reserved					Reserved
0x450[18]	VPU_VP9_DISABLE	1	Disable VP9 Decoder in VPU	0 - enabled 1 - disabled					VPU
0x450[19]	VPU_HEVC_DISABLE	1	Disable HEVC Decoder in VPU	0 - enabled 1 - disabled					VPU
0x450[20]	VPU_AVC_DISABLE	1	Disable AVC Decoder in VPU	0 - enabled 1 - disabled					VPU
0x450[21]	VPU_DISABLE	1	Disable VPU IP	0 - enabled 1 - disabled					VPU
0x450[22]	PCIE1_DISABLE	1	Disable PCIe-1 IP.	0 - enabled 1 - disabled					PCIE

Table continues on the next page...

Table 6-42. Fusemap Descriptions (continued)

Fuse Address	Fuses Name	Number of Fuses	Fuses Function	Setting	Used by
0x450[23]	PCIE2_DISABLE	1	Disable PCIe-2 IP.	0 - enabled 1 - disabled	PCIE
0x450[24]	GPU_DISABLE	1	Disable GPU IP.	0 - enabled 1 - disabled	GPU
0x450[25]	HDMI_DISABLE	1	Disable HDMI IP.	0 - enabled 1 - disabled	HDMI
0x450[26]	DC_DISABLE	1	Disable Display Controller IP.	0 - enabled 1 - disabled	Display
0x450[27]	HDCP_DISABLE	1	Disable HDCP Function	0 - enabled 1 - disabled	HDMI
0x450[28]	MIPI_DSI_DISABLE	1	Disable MIPI DSI IP.	0 - enabled 1 - disabled	MIPI DSI
0x450[29]	ENET_DISABLE	1	Disable ENET IP.	0 - enabled 1 - disabled	ENET
0x450[30]	MIPI_CSI1_DISABLE	1	Disable MIPI CSI1 IP.	0 - enabled 1 - disabled	MIPI CSI
0x450[31]	MIPI_CSI2_DISABLE	1	Disable MIPI CSI2 IP.	0 - enabled 1 - disabled	MIPI CSI
0x460	Reserved	32	Reserved	Reserved	Reserved
0x470[15:0]	BOOT_CFG	16	BOOT configuration register, Usage varies, depending on selected boot device.	See boot fusemap for details.	SRC SW(ROM)
0x470[26:16]	Reserved	10	Reserved	Reserved	Reserved
0x470[27]	Reserved	2	Reserved	Reserved	Reserved
0x470[28]	BT_FUSE_SEL	1	Determines, whether using fuses for boot configuration, or GPIO /Serial loader.	If boot_mode="00" (Development) 0=Boot mode configuration is taken from GPIOs. 1=Boot mode configuration is taken from fuses. If boot_mode="10" (Production) 0 - Boot using Serial Loader (USB) 1- Boot mode configuration is taken from fuses.	SRC SW(ROM)
0x470[29]	FORCE_COLD_BOOT(SBMR)	1	Force cold boot when A53 core come out of reset. Reflected in SBMR reg of SRC	Fuse Function: 0 – Default behavior allowing a fast recovery from low power modes. That is, the	SRC SW(ROM)

Table continues on the next page...

Table 6-42. Fusemap Descriptions (continued)

Fuse Address	Fuses Name	Number of Fuses	Fuses Function	Setting	Used by
				ROM is allowed to jump to the address previously programmed in the SRC persistent register. 1 – Fast recovery path in the ROM is not allowed and a cold boot is always performed. Customers wanting a higher level of security should burn this fuse.	
0x470[31:30]	Reserved	2	Reserved	Reserved	Reserved
0x480[31:0]	BOOT_CFG_PARAMETER	32	BOOT configuration parameters, Usage varies, depending on selected boot device.	See boot fusemap for details.	SW (ROM)
0x490[31:0]	BOOT_CFG_PARAMETER	32	BOOT configuration parameters, Usage varies, depending on selected boot device.	See boot fusemap for details.	SW (ROM)
0x4A0[31:0]	BOOT_CFG_PARAMETER	32	BOOT configuration parameters, Usage varies, depending on selected boot device.	See boot fusemap for details.	SW (ROM)
0x4B0[31:0]	BOOT_CFG_PARAMETER	32	BOOT configuration parameters, Usage varies, depending on selected boot device.	See boot fusemap for details.	SW (ROM)
0x4C0-0x500	Reserved	384	Reserved	Reserved	Reserved
0x580[31:0]	Reserved	256	Reserved	Reserved	Reserved
0x600[23:0]	SJC_RESP[55:0]	56	Response reference value for the secure JTAG controller	-	SJC
0x610[31:24]	Reserved	8	Reserved	Reserved	Reserved
0x620[15:0]	USB_VID[31:0]	16	USB VID	-	SW
0x620[31:16]	USB_PID[31:0]	16	USB PID	-	SW
0x630[31:0]	Reserved	32	Reserved	Reserved	Reserved
0x640[15:0]	MAC_ADDR[47:0]	48	Reserved for customers/SW	-	SW
0x650[31:16]	Reserved	48	Reserved	Reserved	Reserved
0x670-0x700	Reserved	544	Reserved	Reserved	Reserved
0x780[31:0]	GP1[63:0]	64	General Purpose fuse register #1	-	SW
0x7A0[31:0]	GP2[63:0]	64	General Purpose fuse register #2	-	SW
0x7C0-0x7E0	Reserved	96	Reserved	Reserved	Reserved

Table continues on the next page...

Table 6-42. Fusemap Descriptions (continued)

Fuse Address	Fuses Name	Number of Fuses	Fuses Function	Setting	Used by
0x7F0[23:0]	Reserved	24	Reserved	Reserved	Reserved
0x800-0x13F0	Reserved	6144	Reserved	Reserved	Reserved

6.3 On-Chip OTP Controller (OCOTP_CTRL)

6.3.1 Overview

This section contains information describing the requirements for the on-chip eFuse OTP controller along with details about the block functionality and implementation.

In this document, the words "eFuse" and "OTP" are interchangeable. OCOTP refers to the hardware block itself.

6.3.1.1 Features

The OCOTP provides the following features:

- Loading and housing of fuse content into shadow registers.
- Generation of HWV_FUSE (hardware visible fuse bus) and the HWV_REG bus which is made up of volatile PIO register based "fuses". The HWV_REG bits come from the SCS (Software Controllable Signals) register.
- Generation of STICKY_REG which is consist of sticky register bits.
- Provide program-protect and read-protect eFuse.
- Provide override and read protection of shadow register.

6.3.2 Clocks

The table found here describes the clock sources for OCOTP.

Please see [Clock Controller Module \(CCM\)](#) for clock setting, configuration and gating information.

Table 6-43. OCOTP Clocks

Clock name	Clock Root	Description
ipg_clk	ipg_clk_root	Peripheral clock

6.3.3 Top-Level Symbol and Functional Overview

The figure found here shows the OCOTP system level diagram.

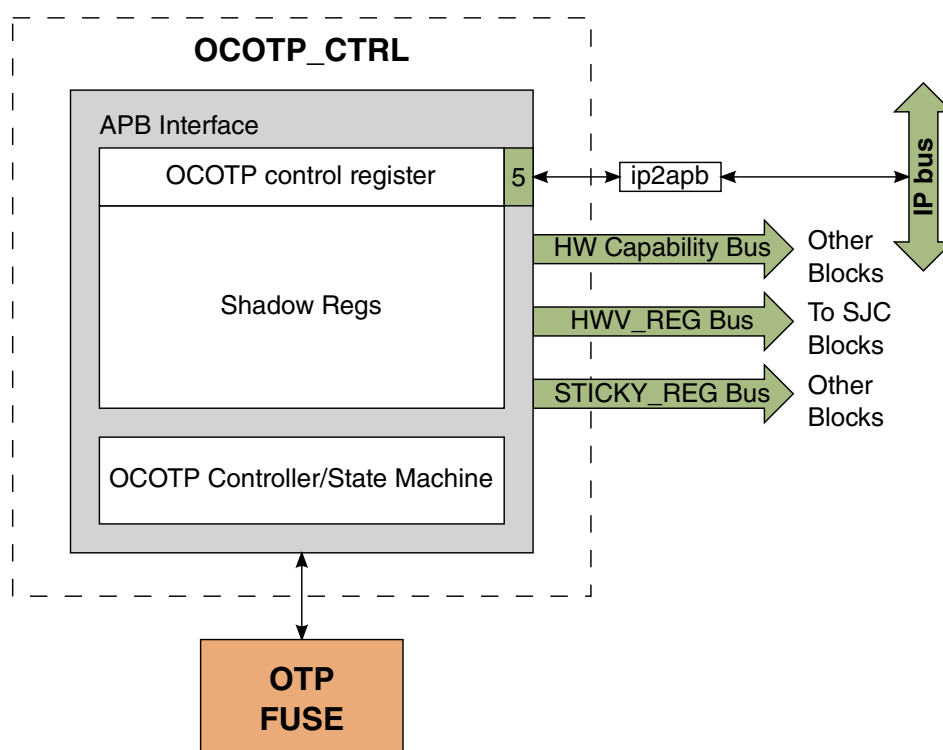


Figure 6-26. OCOTP System Level Diagram

6.3.3.1 Operation

The IP bus interface of the OCOTP provides two functions.

- Configure control registers for programming and reading fuse .
- Override and read shadow registers.

6.3.3.1.1 Shadow Register Reload

All fuse words in are shadowed. Therefore, fuse information is available through memory mapped shadow registers. If fuses are subsequently programmed, the shadow registers should be reloaded to keep them coherent with the fuse bank arrays.

The "reload shadows" feature allows the user to force a reload of the shadow registers (including HW_OCOTP_LOCK) without having to reset the device. To force a reload, complete the following steps:

1. Check that HW_OCOTP_CTRL[BUSY] and HW_OCOTP_CTRL[ERROR] are clear. Overlapped accesses are not supported by the controller. Any pending write , read or reload must be completed before a new access can be requested.
2. Set the HW_OCOTP_CTRL[RELOAD_SHADOWS] bit. OCOTP will read all the fuse one by one and put it into corresponding shadow register.
3. Wait for HW_OCOTP_CTRL[BUSY] and HW_OCOTP_CTRL[RELOAD_SHADOWS] to be cleared by the controller.

The controller will automatically clear the HW_OCOTP_CTRL[RELOAD_SHADOWS] bit after the successful completion of the operation.

6.3.3.1.2 Fuse and Shadow Register Read

All shadow registers are always readable through the APB bus except some secret keys regions. When their corresponding fuse lock bits are set, the shadow registers also become read locked. After read locking, reading from these registers will return 0xBADABADA.

In addition HW_OCOTP_CTRL[ERROR] will be set. It must be cleared by software before any new write , read or reload access can be issued. Subsequent reads to unlocked shadow locations will still work successfully however.

To read fuse words directly from correctly complete the following steps:

1. Check that HW_OCOTP_CTRL[BUSY] and HW_OCOTP_CTRL[ERROR] are clear. Overlapped accesses are not supported by the controller. Any pending write, read or reload must be completed before a read access can be requested.
2. Write the requested to HW_OCOTP_CTRL[ADDR].

6.3.3.1.3 Fuse and Shadow Register Writes

Shadow register bits can be overridden by software until the corresponding fuse lock bit for the region is set. When the lock shadow bit is set, the shadow registers for that lock region become write locked. The LOCK shadow register also has no shadow or fuse lock bits but it is always read only.

In order to avoid "rogue" code performing erroneous writes to OTP, a special unlocking sequence is required for writes to the fuse banks. To program fuse bank correctly complete the following steps:

1. Program HW_OCOTP_TIMING[STROBE_PROG] and fields with timing values to match the current frequency of the ipg_clk. OTP writes will work at maximum bus frequencies as long as the parameters are set correctly.
2. Check that HW_OCOTP_CTRL[BUSY] and HW_OCOTP_CTRL[ERROR] are clear. Overlapped accesses are not supported by the controller. Any pending write or reload must be completed before a write access can be requested.
3. Write the requested to HW_OCOTP_CTRL[ADDR] and program the unlock code into HW_OCOTP_CTRL[WR_UNLOCK]. This must be programmed for each write access. The lock code is documented in the register description. Both the unlock code and address can be written in the same operation.

It should be noted that write latencies to OTP are numbers of 10 micro-seconds. Write latencies is based on amount of bit filed which is 1. For example : program half fuse bits in one word need 10 us x 16.

For further details of OTP read/write operations see [eFUSE].

HW_OCOTP_CTRL[ERROR] will be set under the following conditions:

- A write is performed to a shadow register during a shadow reload (essentially, while HW_OCOTP_CTRL[RELOAD_SHADOWS] is set. In addition, the contents of the shadow register shall not be updated.
- A write is performed to a shadow register which has been locked.
- A read is performed to from a shadow register which has been read locked.
- A program is performed to a fuse which has been .
- A read is performed to from a fuse which has been read locked.

6.3.3.1.4 Write Postamble

Due to internal electrical characteristics of the OTP during writes, all OTP operations following a write must be separated by 2 us after the clearing of HW_OCOTP_CTRL_BUSY following the write. This guarantees programming voltages on-chip to reach a steady state when exiting a write sequence. This includes reads, shadow reloads, or other writes.

A recommended software sequence to meet the postamble requirements is as follows:

- Issue the write and poll for BUSY (as per [Fuse Shadow Memory Footprint](#)).
- After BUSY is clear, wait an additional 2 us.
- Perform the next OTP operation.

6.3.3.2 Fuse Shadow Memory Footprint

The OTP memory footprint shows in the following figure. The registers are grouped by lock region. Their names correspond to the PIO register and fusemap names.

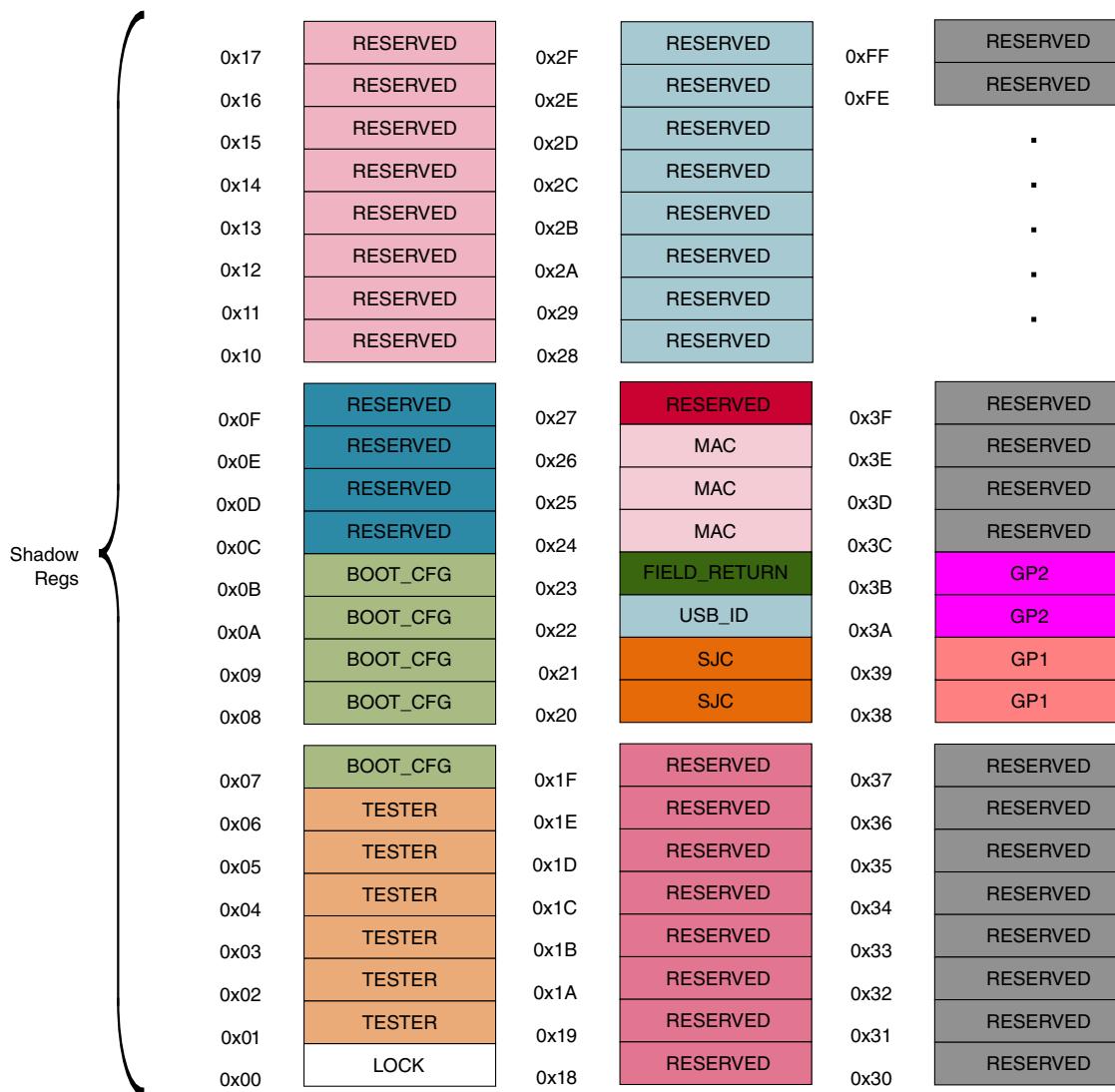


Figure 6-27. OTP Memory Footprint

6.3.3.3 OTP Read/Write Timing Parameters

There are timing fields contained in the HW_OCOTP_TIMING register that specify counter limit values, which are used to specify the signal timing.

Both two timing parameters are specified in ipg_clk cycles. Since the ipg_clk frequency can be set to a range of values, these parameters must be adjusted with the clock to yield the appropriate delay.

6.3.3.4 Hardware Visible Fuses

The hwv_fuse bus emanates from the OCOTP block and goes to various other blocks inside the chip. This bus is made up of the shadow register bits for .

Only a subset of these fuse bits are currently used by the hardware. The fuse bits are initially copied from the banks after reset is deasserted. When all fuse bits are loaded into their shadow registers, the OCOTP asserts the fuse_latched output signal.

The hwv_reg bus also comes from the OCOTP. Its source is the HW_OCOTP_SCS register. This register has 1 defined bit, the HAB_JDE bit, that is connected to the SJC block. The SCS bits are intended to be used as volatile fuse bits under software control. Additional bits will be defined as needed in future implementations.

The system-wide reset sequence must be coordinated by the system reset controller, so that the hwv_fuse and hwv_reg buses are stable and reflect the values of the fuses before they are used by the rest of the system.

6.3.3.5 Behavior During Reset

The OCOTP is always active. The shadow registers automatically load the appropriate OTP contents after reset is deasserted. During this load-time HW_OCOTP_CTRL_BUSY is set. The load time is similar to that of a "reload shadow" operation.

6.3.4 Fuse Map

See the Fusemap chapter of this reference manual for more information.

6.3.5 OCOTP Memory Map/Register Definition

NOTE

When write/read unimplemented register address in ocotp_ctrl, ocotp_ctrl will not send error and read data will be 0.

OCOTP Hardware Register Format Summary

OCOTP memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3035_0000	OTP Controller Control Register (OCOTP_HW_OCOTP_CTRL)	32	R/W	0000_0000h	6.3.5.1/920
3035_0004	OTP Controller Control Register (OCOTP_HW_OCOTP_CTRL_SET)	32	R/W	0000_0000h	6.3.5.1/920
3035_0008	OTP Controller Control Register (OCOTP_HW_OCOTP_CTRL_CLR)	32	R/W	0000_0000h	6.3.5.1/920
3035_000C	OTP Controller Control Register (OCOTP_HW_OCOTP_CTRL_TOG)	32	R/W	0000_0000h	6.3.5.1/920
3035_0010	OTP Controller Timing Register (OCOTP_HW_OCOTP_TIMING)	32	R/W	0400_0000h	6.3.5.2/921
3035_0020	OTP Controller Write Data Register (OCOTP_HW_OCOTP_DATA)	32	R/W	0000_0000h	6.3.5.3/922
3035_0030	OTP Controller Write Data Register (OCOTP_HW_OCOTP_READ_CTRL)	32	R/W	0000_0000h	6.3.5.4/923
3035_0040	OTP Controller Read Data Register (OCOTP_HW_OCOTP_READ_FUSE_DATA)	32	R/W	0000_0000h	6.3.5.5/924
3035_0050	Sticky bit Register (OCOTP_HW_OCOTP_SW_STICKY)	32	R/W	0000_0000h	6.3.5.6/925
3035_0060	Software Controllable Signals Register (OCOTP_HW_OCOTP_SCS)	32	R/W	0000_0000h	6.3.5.7/926
3035_0064	Software Controllable Signals Register (OCOTP_HW_OCOTP_SCS_SET)	32	R/W	0000_0000h	6.3.5.7/926
3035_0068	Software Controllable Signals Register (OCOTP_HW_OCOTP_SCS_CLR)	32	R/W	0000_0000h	6.3.5.7/926
3035_006C	Software Controllable Signals Register (OCOTP_HW_OCOTP_SCS_TOG)	32	R/W	0000_0000h	6.3.5.7/926
3035_0090	OTP Controller Version Register (OCOTP_HW_OCOTP_VERSION)	32	R/W	0148_1299h	6.3.5.8/927
3035_0400	Value of OTP Bank0 Word0 (Lock controls) (OCOTP_HW_OCOTP_LOCK)	32	R/W	0000_0000h	6.3.5.9/928
3035_0410	Value of OTP Bank0 Word1 (Tester Info.) (OCOTP_HW_OCOTP_TESTER0)	32	R/W	0000_0000h	6.3.5.10/930

Table continues on the next page...

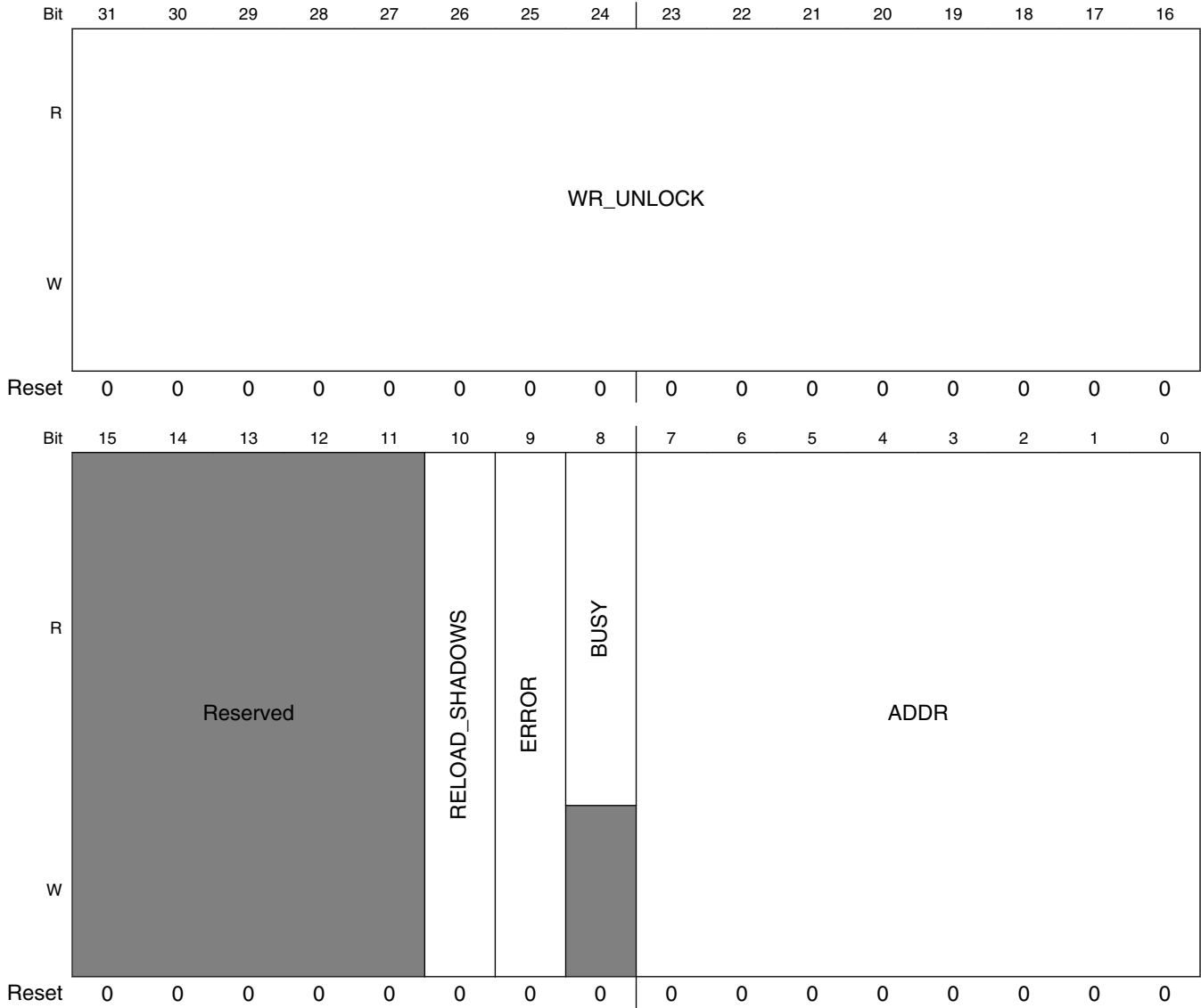
OCOTP memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3035_0420	Value of OTP Bank0 Word2 (tester Info.) (OCOTP_HW_OCOTP_TESTER1)	32	R/W	0000_0000h	6.3.5.11/930
3035_0430	Value of OTP Bank0 Word3 (Tester Info.) (OCOTP_HW_OCOTP_TESTER2)	32	R/W	0000_0000h	6.3.5.12/931
3035_0440	Value of OTP Bank1 Word0 (Tester Info.) (OCOTP_HW_OCOTP_TESTER3)	32	R/W	0000_0000h	6.3.5.13/931
3035_0450	Value of OTP Bank1 Word1 (Tester Info.) (OCOTP_HW_OCOTP_TESTER4)	32	R/W	0000_0000h	6.3.5.14/932
3035_0460	Value of OTP Bank1 Word2 (Tester Info.) (OCOTP_HW_OCOTP_TESTER5)	32	R/W	0000_0000h	6.3.5.15/932
3035_0470	Value of OTP Bank1 Word3 (Boot Configuration Info.) (OCOTP_HW_OCOTP_BOOT_CFG0)	32	R/W	0000_0000h	6.3.5.16/933
3035_0480	Value of OTP Bank2 Word0 (Boot Configuration Info.) (OCOTP_HW_OCOTP_BOOT_CFG1)	32	R/W	0000_0000h	6.3.5.17/933
3035_0490	Value of OTP Bank2 Word1 (Boot Configuration Info.) (OCOTP_HW_OCOTP_BOOT_CFG2)	32	R/W	0000_0000h	6.3.5.18/934
3035_04A0	Value of OTP Bank2 Word2 (Boot Configuration Info.) (OCOTP_HW_OCOTP_BOOT_CFG3)	32	R/W	0000_0000h	6.3.5.19/934
3035_04B0	Value of OTP Bank2 Word3 (BOOT Configuration Info.) (OCOTP_HW_OCOTP_BOOT_CFG4)	32	R/W	0000_0000h	6.3.5.20/935
3035_0600	Value of OTP Bank8 Word0 (Secure JTAG Response Field) (OCOTP_HW_OCOTP_SJC_RESP0)	32	R/W	0000_0000h	6.3.5.21/935
3035_0610	Value of OTP Bank8 Word1 (Secure JTAG Response Field) (OCOTP_HW_OCOTP_SJC_RESP1)	32	R/W	0000_0000h	6.3.5.22/936
3035_0620	Value of OTP Bank8 Word2 (USB ID info) (OCOTP_HW_OCOTP_USB_ID)	32	R/W	0000_0000h	6.3.5.23/936
3035_0640	Value of OTP Bank9 Word0 (MAC Address) (OCOTP_HW_OCOTP_MAC_ADDR0)	32	R/W	0000_0000h	6.3.5.24/937
3035_0650	Value of OTP Bank9 Word1 (MAC Address) (OCOTP_HW_OCOTP_MAC_ADDR1)	32	R/W	0000_0000h	6.3.5.25/937
3035_0660	Value of OTP Bank9 Word2 (MAC Address) (OCOTP_HW_OCOTP_MAC_ADDR2)	32	R/W	0000_0000h	6.3.5.26/938
3035_0780	Value of OTP Bank14 Word0 () (OCOTP_HW_OCOTP_GP10)	32	R/W	0000_0000h	6.3.5.27/938
3035_0790	Value of OTP Bank14 Word1 () (OCOTP_HW_OCOTP_GP11)	32	R/W	0000_0000h	6.3.5.28/939
3035_07A0	Value of OTP Bank14 Word2 () (OCOTP_HW_OCOTP_GP20)	32	R/W	0000_0000h	6.3.5.29/939
3035_07B0	Value of OTP Bank14 Word3 () (OCOTP_HW_OCOTP_GP21)	32	R/W	0000_0000h	6.3.5.30/939

6.3.5.1 OTP Controller Control Register (OCOTP_HW_OCOTP_CTRLn)

The OCOTP Control and Status Register provides the necessary software interface for performing read and write operations to the On-Chip OTP (One-Time Programmable ROM). The control fields such as WR_UNLOCK, ADDR and BUSY/ERROR may be used in conjunction with the HW_OCOTP_DATA register to perform write operations. Read operations to the On-Chip OTP are involving ADDR, BUSY/ERROR bit field and HW_OCOTP_READ_CTRL register. Read value is saved in HW_OCOTP_READ_FUSE_DATA register.

Address: 3035_0000h base + 0h offset + (4d × i), where i=0d to 3d



OCOTP_HW_OCOTP_CTRLn field descriptions

Field	Description
31–16 WR_UNLOCK	Write 0x3E77 to enable OTP write accesses. NOTE: This register must be unlocked on a write-by-write basis (a write is initiated when HW_OCOTP_DATA is written), so the UNLOCK bitfield must contain the correct key value during all writes to HW_OCOTP_DATA, otherwise a write shall not be initiated. This field is automatically cleared after a successful write completion (clearing of BUSY).
15–11 -	This field is reserved. Reserved
10 RELOAD_SHADOWS	Set to force re-loading the shadow registers (HW/SW capability and LOCK). This operation will automatically set BUSY. Once the shadow registers have been re-loaded, BUSY and RELOAD_SHADOWS are automatically cleared by the controller.
9 ERROR	Set by the controller when an access to a locked region(OTP or shadow register) is requested. Must be cleared before any further access can be performed. This bit can only be set by the controller. This bit is also set if the Pin interface is active and software requests an access to the OTP. In this instance, the ERROR bit cannot be cleared until the Pin interface access has completed. Reset this bit by writing a one to the SCT clear address space and not by a general write.
8 BUSY	OTP controller status bit. When active, no new write access or read access to OTP(including RELOAD_SHADOWS) can be performed. Cleared by controller when access complete. After reset (or after setting RELOAD_SHADOWS), this bit is set by the controller until the HW/SW and LOCK registers are successfully copied, after which time it is automatically cleared by the controller.
ADDR	OTP write and read access address register. Specifies one of 128 word address locations (0x00 - 0x7f). If a valid access is accepted by the controller, the controller makes an internal copy of this value. This internal copy will not update until the access is complete.

6.3.5.2 OTP Controller Timing Register (OCOTP_HW_OCOTP_TIMING)

This register specifies timing parameters for programming and reading the OCOTP fuse array.

Address: 3035_0000h base + 10h offset = 3035_0010h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	RSRVD0				WAIT								STROBE_READ				RELAX				STROBE_PROG											
W																																
Reset	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_TIMING field descriptions

Field	Description
31–28 RSRVD0	These bits always read back zero.
27–22 WAIT	This count value specifies time interval between auto read and write access in one time program. It is given in number of ipg_clk periods.

Table continues on the next page...

OCOTP_HW_OCOTP_TIMING field descriptions (continued)

Field	Description
21–16 STROBE_READ	This count value specifies the strobe period in one time read OTP. $Trd = ((STROBE_READ+1) - 2*(RELAX+1)) / ipg_clk_freq$. It is given in number of ipg_clk periods.
15–12 RELAX	This count value specifies the time to add to all default timing parameters other than the Tpgm and Trd. It is given in number of ipg_clk periods.
STROBE_PROG	This count value specifies the strobe period in one time write OTP. $Tpgm = ((STROBE_PROG+1) - 2*(RELAX+1)) / ipg_clk_freq$. It is given in number of ipg_clk periods.

6.3.5.3 OTP Controller Write Data Register (OCOTP_HW_OCOTP_DATA)

This register is used in conjunction with HW_OCOTP_CTRL to perform one-time writes to the OTP.

Address: 3035_0000h base + 20h offset = 3035_0020h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
	DATA																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

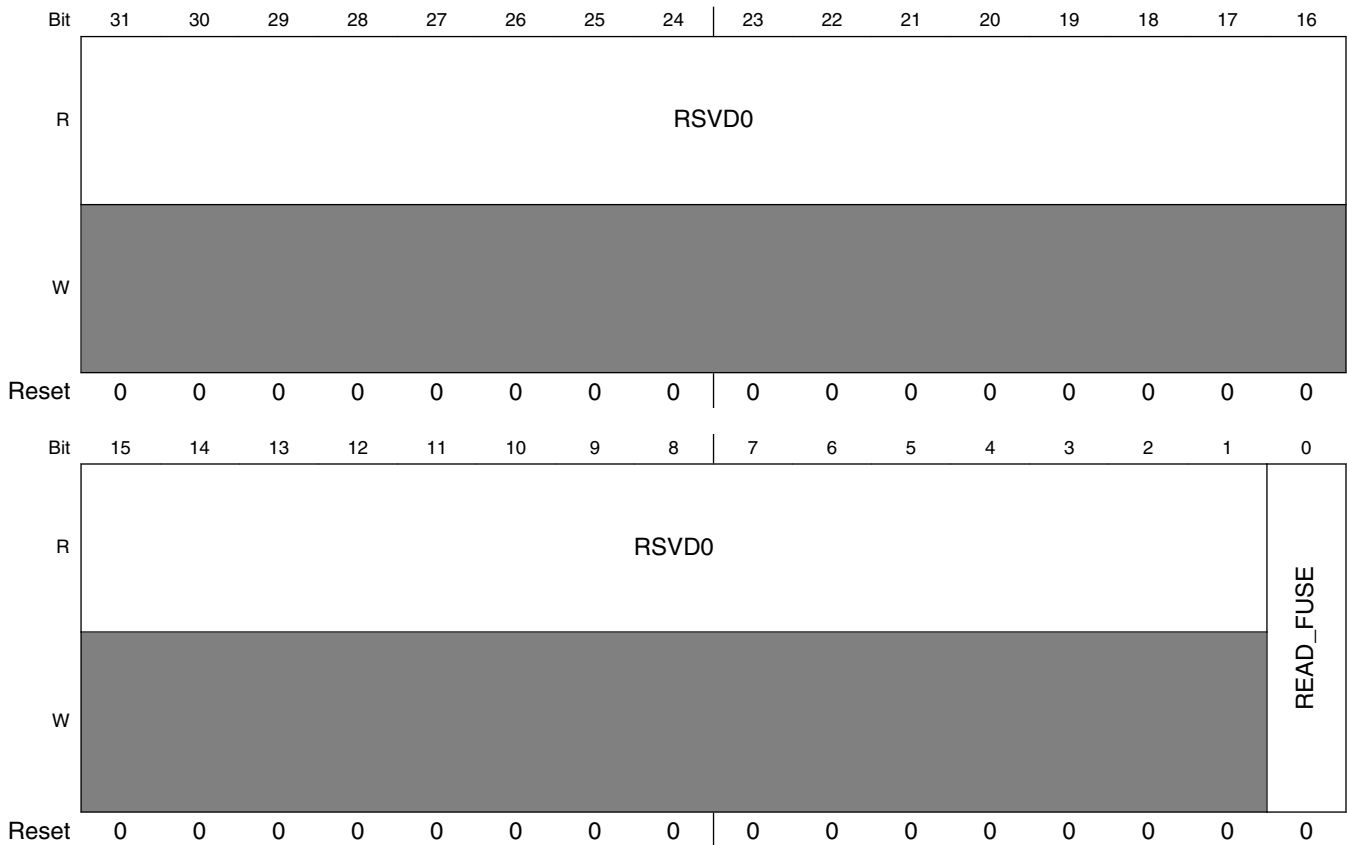
OCOTP_HW_OCOTP_DATA field descriptions

Field	Description
DATA	Used to initiate a write to OTP.

6.3.5.4 OTP Controller Write Data Register (OCOTP_HW_OCOTP_READ_CTRL)

This register is used in conjunction with HW_OCOTP_CTRL to perform one time read to the OTP.

Address: 3035_0000h base + 30h offset = 3035_0030h



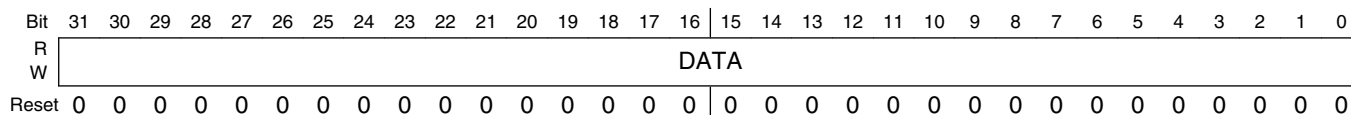
OCOTP_HW_OCOTP_READ_CTRL field descriptions

Field	Description
31–1 RSVD0	Reserved
0 READ_FUSE	Used to initiate a read to OTP.

6.3.5.5 OTP Controller Read Data Register (OCOTP_HW_OCOTP_READ_FUSE_DATA)

The data read from OTP

Address: 3035 0000h base + 40h offset = 3035 0040h



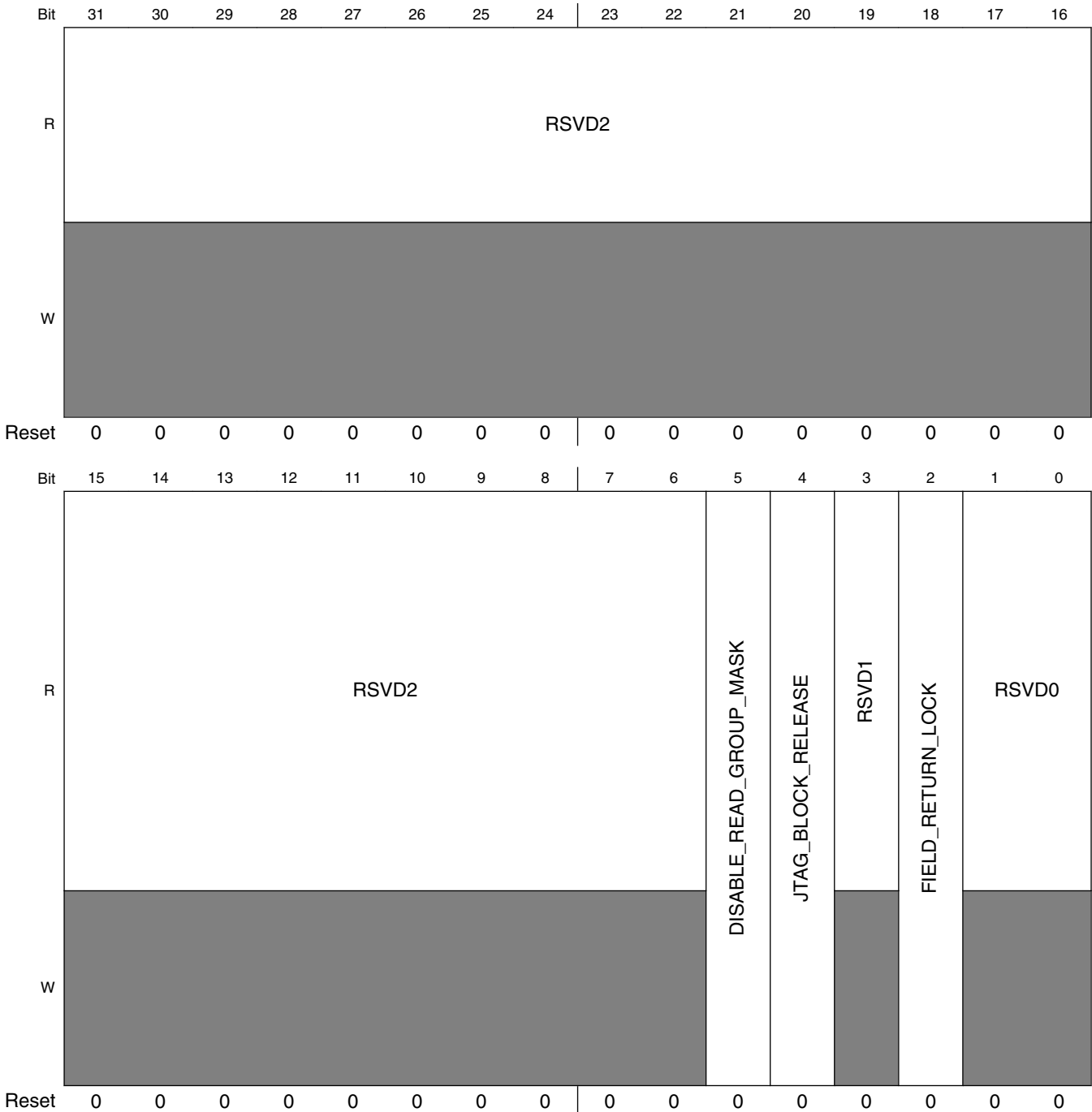
OCOTP_HW_OCOTP_READ_FUSE_DATA field descriptions

Field	Description
DATA	The data read from OTP

6.3.5.6 Sticky bit Register (OCOTP_HW_OCOTP_SW_STICKY)

Some sticky bits are used by SW to lock some fuse area , shadow registers and other features.

Address: 3035_0000h base + 50h offset = 3035_0050h



OCOTP_HW_OCOTP_SW_STICKY field descriptions

Field	Description
31–6 RSVD2	Reserved
5 DISABLE_READ_GROUP_MASK	Shadow register write and OTP write lock for GROUP_MASK region. When set, the writing of this region's shadow register and OTP fuse word are blocked. Once this bit is set, it is always high unless a POR is issued.
4 JTAG_BLOCK_RELEASE	Set by Arm during Boot after DTCP is initialized and before test mode entry. * 0 (Default) - JTAG is blocked (subject to other conditions). * 1 - JTAG block is released (subject to other controls). Once this bit is set, it is always high unless a POR is issued.
3 RSVD1	Reserved
2 FIELD_RETURN_LOCK	Shadow register write and OTP write lock for FIELD_RETURN region. When set, the writing of this region's shadow register and OTP fuse word are blocked. Once this bit is set, it is always high unless a POR is issued.
RSVD0	Reserved

6.3.5.7 Software Controllable Signals Register (OCOTP_HW_OCOTP_SCSn)

This register holds volatile configuration values that can be set and locked by trusted software. All values are returned to their default values after POR.

Address: 3035_0000h base + 60h offset + (4d × i), where i=0d to 3d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	LOCK	SPARE														
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	SPARE															HAB_JDE
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_SCSn field descriptions

Field	Description
31 LOCK	When set, all of the bits in this register are locked and can not be changed through SW programming. This bit is only reset after a POR is issued.

Table continues on the next page...

OCOTP_HW_OCOTP_SCS_n field descriptions (continued)

Field	Description
30–1 SPARE	Unallocated read/write bits for implementation specific software use.
0 HAB_JDE	HAB JTAG Debug Enable. This bit is used by the HAB to enable JTAG debugging, assuming that a properly signed command to do so is found and validated by the HAB. The HAB must lock the register before passing control to the OS whether or not JTAG debugging has been enabled. Once JTAG is enabled by this bit, it can not be disabled unless the system is reset by POR. 0: JTAG debugging is not enabled by the HAB (it may still be enabled by other mechanisms). 1: JTAG debugging is enabled by the HAB (though this signal may be gated off).

6.3.5.8 OTP Controller Version Register (OCOTP_HW_OCOTP_VERSION)

This register indicates the RTL version in use.

Address: 3035_0000h base + 90h offset = 3035_0090h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	MAJOR								MINOR								STEP															
W																																
Reset	0	0	0	0	0	0	0	1	0	1	0	0	1	0	0	0	0	0	0	1	0	0	1	0	1	0	0	1	1	0	0	1

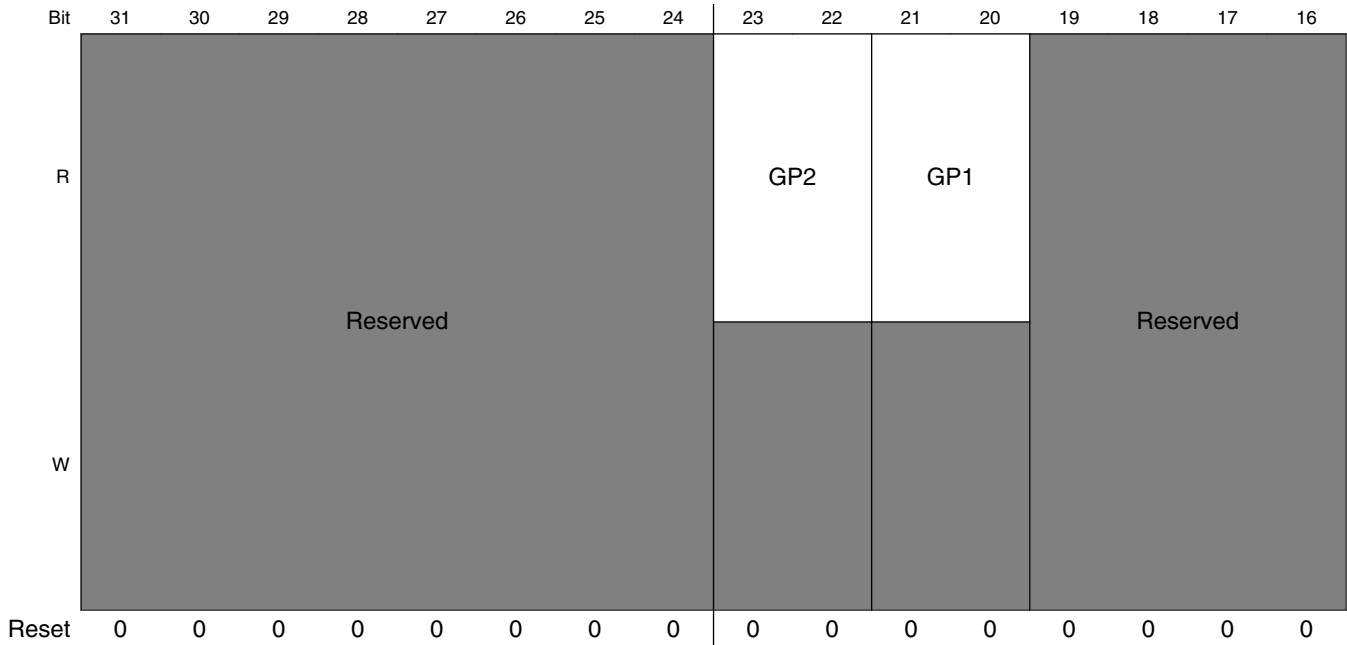
OCOTP_HW_OCOTP_VERSION field descriptions

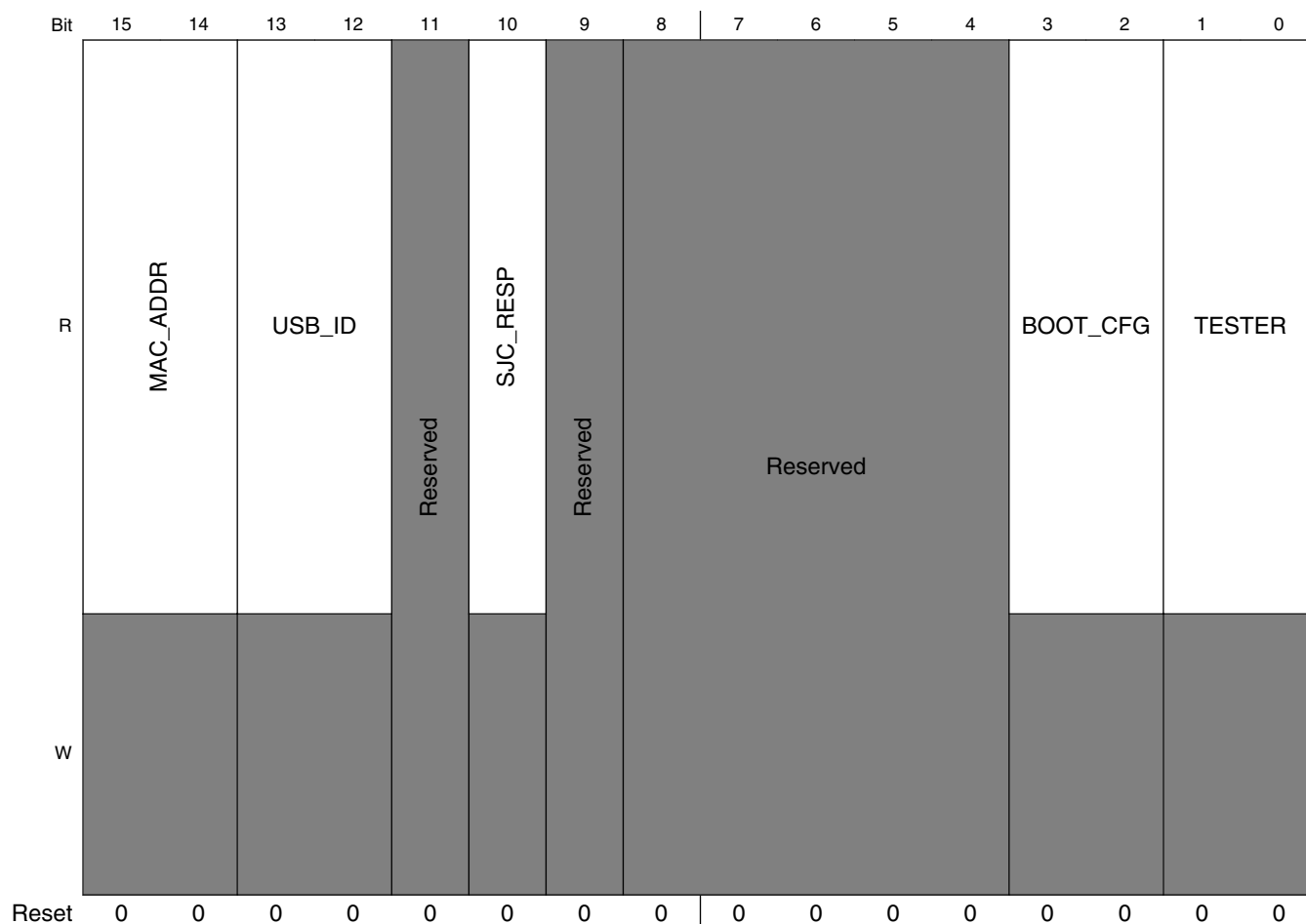
Field	Description
31–24 MAJOR	Fixed read-only value reflecting the MAJOR field of the RTL version.
23–16 MINOR	Fixed read-only value reflecting the MINOR field of the RTL version.
STEP	Fixed read-only value reflecting the stepping of the RTL version.

6.3.5.9 Value of OTP Bank0 Word0 (Lock controls) (OCOTP_HW_OCOTP_LOCK)

Shadowed memory mapped access to OTP Bank 0, word 0.

Address: 3035_0000h base + 400h offset = 3035_0400h





OCOTP_HW_OCOTP_LOCK field descriptions

Field	Description
31–24 -	This field is reserved. Reserved
23–22 GP2	Status of shadow register and OTP write lock for gp2 region. When bit 1 is set, the writing of this region's shadow register is blocked. When bit 0 is set, the writing of this region's OTP fuse word is blocked.
21–20 GP1	Status of shadow register and OTP write lock for gp1 region. When bit 1 is set, the writing of this region's shadow register is blocked. When bit 0 is set, the writing of this region's OTP fuse word is blocked.
19–16 -	This field is reserved. Reserved
15–14 MAC_ADDR	Status of shadow register and OTP write lock for mac_addr region. When bit 1 is set, the writing of this region's shadow register is blocked. When bit 0 is set, the writing of this region's OTP fuse word is blocked.
13–12 USB_ID	Status of shadow register and OTP write lock for usb_id region. When bit 1 is set, the writing of this region's shadow register is blocked. When bit 0 is set, the writing of this region's OTP fuse word is blocked.
11 -	This field is reserved. Reserved

Table continues on the next page...

OCOTP_HW_OCOTP_LOCK field descriptions (continued)

Field	Description
10 SJC_RESP	Status of shadow register read and write, OTP read and write lock for sjc_resp region. When set, the writing of this region's shadow register and OTP fuse word are blocked. The read of this region's shadow register and OTP fuse word are also blocked.
9 -	This field is reserved. Reserved
8–4 -	This field is reserved. Reserved
3–2 BOOT_CFG	Status of shadow register and OTP write lock for boot_cfg region. When bit 1 is set, the writing of this region's shadow register is blocked. When bit 0 is set, the writing of this region's OTP fuse word is blocked.
TESTER	Status of shadow register and OTP write lock for tester region. When bit 1 is set, the writing of this region's shadow register is blocked. When bit 0 is set, the writing of this region's OTP fuse word is blocked.

**6.3.5.10 Value of OTP Bank0 Word1 (Tester Info.)
(OCOTP_HW_OCOTP_TESTER0)**

Shadowed memory mapped access to OTP Bank 0, word 1.

Address: 3035_0000h base + 410h offset = 3035_0410h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
	BITS																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_TESTER0 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 0, word 1. These bits become read-only after the HW_OCOTP_LOCK_TESTER[1] bit is set.

**6.3.5.11 Value of OTP Bank0 Word2 (tester Info.)
(OCOTP_HW_OCOTP_TESTER1)**

shadowed memory mapped access to OTP Bank 0, word 2.

Address: 3035_0000h base + 420h offset = 3035_0420h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
	BITS																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

OCOTP_HW_OCOTP_TESTER1 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 0, word 2. These bits become read-only after the HW_OCOTP_LOCK_TESTER[1] bit is set.

6.3.5.12 Value of OTP Bank0 Word3 (Tester Info.) (OCOTP_HW_OCOTP_TESTER2)

Shadowed memory mapped access to OTP Bank 0, word 3.

Address: 3035_0000h base + 430h offset = 3035_0430h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
	BITS																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

OCOTP_HW_OCOTP_TESTER2 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 0, word 3. These bits become read-only after the HW_OCOTP_LOCK_TESTER[1] bit is set.

6.3.5.13 Value of OTP Bank1 Word0 (Tester Info.) (OCOTP_HW_OCOTP_TESTER3)

Non-shadowed memory mapped access to OTP Bank 1, word 0.

Address: 3035_0000h base + 440h offset = 3035_0440h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
	BITS																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

OCOTP_HW_OCOTP_TESTER3 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 1, word 0. These bits become read-only after the HW_OCOTP_LOCK_TESTER[1] bit is set.

6.3.5.14 Value of OTP Bank1 Word1 (Tester Info.) (OCOTP_HW_OCOTP_TESTER4)

Shadowed memory mapped access to OTP Bank 1, word 1.

Address: 3035_0000h base + 450h offset = 3035_0450h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_TESTER4 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 1, word 1. These bits become read-only after the HW_OCOTP_LOCK_TESTER[1] bit is set.

6.3.5.15 Value of OTP Bank1 Word2 (Tester Info.) (OCOTP_HW_OCOTP_TESTER5)

Shadowed memory mapped access to OTP Bank 1, word 2.

Address: 3035_0000h base + 460h offset = 3035_0460h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_TESTER5 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 1, word 2. These bits become read-only after the HW_OCOTP_LOCK_TESTER[1] bit is set.

6.3.5.16 Value of OTP Bank1 Word3 (Boot Configuration Info.) (OCOTP_HW_OCOTP_BOOT_CFG0)

Shadowed memory mapped access to OTP Bank 1, word 3.

Address: 3035_0000h base + 470h offset = 3035_0470h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_BOOT_CFG0 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 1, word 3. These bits become read-only after the HW_OCOTP_LOCK_BOOT_CFG[1] bit is set.

6.3.5.17 Value of OTP Bank2 Word0 (Boot Configuration Info.) (OCOTP_HW_OCOTP_BOOT_CFG1)

Shadowed memory mapped access to OTP bank 2, word 0.

Address: 3035_0000h base + 480h offset = 3035_0480h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_BOOT_CFG1 field descriptions

Field	Description
BITS	Reflects value of OTP bank 2, word 0. These bits become read-only after the HW_OCOTP_LOCK_BOOT_CFG[1] bit is set.

6.3.5.18 Value of OTP Bank2 Word1 (Boot Configuration Info.) (OCOTP_HW_OCOTP_BOOT_CFG2)

Shadowed memory mapped access to OTP bank 2, word 1.

Address: 3035_0000h base + 490h offset = 3035_0490h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_BOOT_CFG2 field descriptions

Field	Description
BITS	Reflects value of OTP bank 2, word 1. These bits become read-only after the HW_OCOTP_LOCK_BOOT_CFG[1] bit is set.

6.3.5.19 Value of OTP Bank2 Word2 (Boot Configuration Info.) (OCOTP_HW_OCOTP_BOOT_CFG3)

Shadowed memory mapped access to OTP bank 2, word 2.

Address: 3035_0000h base + 4A0h offset = 3035_04A0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_BOOT_CFG3 field descriptions

Field	Description
BITS	Reflects value of OTP bank 2, word 2. These bits become read-only after the HW_OCOTP_LOCK_BOOT_CFG[1] bit is set.

6.3.5.20 Value of OTP Bank2 Word3 (BOOT Configuration Info.) (OCOTP_HW_OCOTP_BOOT_CFG4)

Shadowed memory mapped access to OTP bank 2, word 3.

Address: 3035_0000h base + 4B0h offset = 3035_04B0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_BOOT_CFG4 field descriptions

Field	Description
BITS	Reflects value of OTP bank 2, word 3. These bits become read-only after the HW_OCOTP_LOCK_BOOT_CFG[1] bit is set.

6.3.5.21 Value of OTP Bank8 Word0 (Secure JTAG Response Field) (OCOTP_HW_OCOTP_SJC_RESP0)

Shadowed memory mapped access to OTP Bank 8, word 0.

Address: 3035_0000h base + 600h offset = 3035_0600h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_SJC_RESP0 field descriptions

Field	Description
BITS	Shadow register for the SJC_RESP Key word0 (Copy of OTP Bank 8, word 0). These bits can be not read and written after the HW_OCOTP_LOCK_SJC_RESP bit is set. If read, returns 0xBADA_BADA and sets HW_OCOTP_CTRL[ERROR].

6.3.5.22 Value of OTP Bank8 Word1 (Secure JTAG Response Field) (OCOTP_HW_OCOTP_SJC_RESP1)

Shadowed memory mapped access to OTP Bank 8, word 1.

Address: 3035_0000h base + 610h offset = 3035_0610h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_SJC_RESP1 field descriptions

Field	Description
BITS	Shadow register for the SJC_RESP Key word1 (Copy of OTP Bank 8, word 1). These bits can be not read and wrotten after the HW_OCOTP_LOCK_SJC_RESP bit is set. If read, returns 0xBADA_BADA and sets HW_OCOTP_CTRL[ERROR].

6.3.5.23 Value of OTP Bank8 Word2 (USB ID info) (OCOTP_HW_OCOTP_USB_ID)

Shadowed memory mapped access to OTP Bank 8, word 2.

Address: 3035_0000h base + 620h offset = 3035_0620h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_USB_ID field descriptions

Field	Description
BITS	Reflects value of OTP Bank 8, word 2.

6.3.5.24 Value of OTP Bank9 Word0 (MAC Address) (OCOTP_HW_OCOTP_MAC_ADDR0)

Shadowed memory mapped access to OTP Bank 9, word 0.

Address: 3035_0000h base + 640h offset = 3035_0640h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_MAC_ADDR0 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 9, word 0.

6.3.5.25 Value of OTP Bank9 Word1 (MAC Address) (OCOTP_HW_OCOTP_MAC_ADDR1)

Shadowed memory mapped access to OTP Bank 9, word 1.

Address: 3035_0000h base + 650h offset = 3035_0650h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_MAC_ADDR1 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 9, word 1.

6.3.5.26 Value of OTP Bank9 Word2 (MAC Address) (OCOTP_HW_OCOTP_MAC_ADDR2)

Shadowed memory mapped access to OTP Bank 9, word 2.

Address: 3035_0000h base + 660h offset = 3035_0660h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_MAC_ADDR2 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 9, word 2.

6.3.5.27 Value of OTP Bank14 Word0 () (OCOTP_HW_OCOTP_GP10)

Shadowed memory mapped access to OTP Bank 14, word 0.

Address: 3035_0000h base + 780h offset = 3035_0780h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OCOTP_HW_OCOTP_GP10 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 14, word 0.

6.3.5.28 Value of OTP Bank14 Word1 () (OCOTP_HW_OCOTP_GP11)

Shadowed memory mapped access to OTP Bank 14, word 1.

Address: 3035_0000h base + 790h offset = 3035_0790h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

OCOTP_HW_OCOTP_GP11 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 14, word 1.

6.3.5.29 Value of OTP Bank14 Word2 () (OCOTP_HW_OCOTP_GP20)

Shadowed memory mapped access to OTP Bank 14, word 2.

Address: 3035_0000h base + 7A0h offset = 3035_07A0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

OCOTP_HW_OCOTP_GP20 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 14, word 2.

6.3.5.30 Value of OTP Bank14 Word3 () (OCOTP_HW_OCOTP_GP21)

Shadowed memory mapped access to OTP Bank 14, word 3.

Address: 3035_0000h base + 7B0h offset = 3035_07B0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BITS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

OCOTP_HW_OCOTP_GP21 field descriptions

Field	Description
BITS	Reflects value of OTP Bank 14, word 3.

6.4 Secure Non-Volatile Storage (SNVS)

6.4.1 SNVS introduction

SNVS is a companion module to the CAAM module.

SNVS incorporates both security and non-security functionality. The SNVS non-security functionality is described in this document, but the SNVS security functionality is described only in the Security Reference Manual.

SNVS non-security functions:

- Realtime Counter (RTC) - a software accessible realtime counter
 - RTC can be set to the value in the SRTC
- General Purpose Register - a set of registers used to hold 128 bits of data specified by software
 - If the SNVS_LP power input is connected to an uninterrupted power supply, the GPR value is maintained when main SoC is powered off
- Chip power-on/power-off - If the SNVS_LP power input is connected to an uninterrupted power supply and the Power On button input signal is connected to a power button external to the chip, logic within SNVS_LP can be used to wake the chip from a power down.

6.4.1.1 SNVS feature list

The following table summarizes the features of SNVS:

Table 6-44. SNVS feature list

Feature	Description	Links for Further Information
Real time counter (RTC)	• The RTC is driven by a dedicated clock, which is off when the system power is down. • Programmable time alarm interrupt	SNVS_HP Real Time Counter
General-purpose register	• The general-purpose register is available to software to store 128 bits of data.	Using the General-Purpose Register

Table continues on the next page...

Table 6-44. SNVS feature list (continued)

Feature	Description	Links for Further Information
	- The general-purpose register is zeroized when a security violation is detected. - If the SNVS_LP power input is connected to an uninterrupted power supply (see SNVS power domains), the general-purpose register value is retained even if the main chip is powered down.	
Register access restrictions	Some registers/values can be written only once per boot cycle.	privileged and non-privileged registers
Wakeup from power off	- Input signal from off chip requests SNVS_LP to power on the main SoC (Assuming that the SNVS_LP power input is connected to an uninterrupted power supply (see SNVS power domains). - Hardware debounces the input signal using software-specified signal bounce characteristics	LP Wake-Up Interrupt Enable

6.4.1.2 SNVS functional description

SNVS implements several non-security features that involve software interaction:

- reading or writing the Realtime Counter (RTC) (This is a non-privileged operation.) - software can also instruct SNVS to load the current SRTC value into the RTC
- reading or writing the General Purpose Register (GPR) (Note that there may be a significant delay when reading or writing registers in the LP section if the LP clock is different from the HP clock.)

The following sections describe in more detail the operation of SNVS.

6.4.2 SNVS Structure

SNVS is organized as two major sub-modules:

- Low-Power Section of SNVS (SNVS_LP)

The SNVS_LP section provides hardware that enables secure storage and protection of sensitive data. The SNVS module is designed to safely hold security-related data such as cryptographic key, time counter, monotonic counter, and general purpose security information.

The SNVS_LP block implements the following functional units:

- Control and Status Registers
- General Purpose Registers

When the LP section is connected to an uninterrupted power supply the state of these registers is maintained even when the main chip power is off. (see [SNVS power domains](#))

- High-Power Section of SNVS (SNVS_HP)

The SNVS_HP section contains all SNVS status and configuration registers. It implements all features that enable system communication and provisioning of the SNVS_LP section.

The SNVS_HP provides an interface between SNVS_LP and the rest of the system.

The SNVS_HP block implements the following functional units:

- IP Bus Interface
- SNVS_LP Interface
- Zeroizable Master Key Programming Mechanism
- Real Time Counter with Alarm Control and Status Registers
- Control and status registers

SNVS_HP is in the chip's power supply domain and thus receives power along with the rest of the chip.

The following figure illustrates the structure of SNVS.

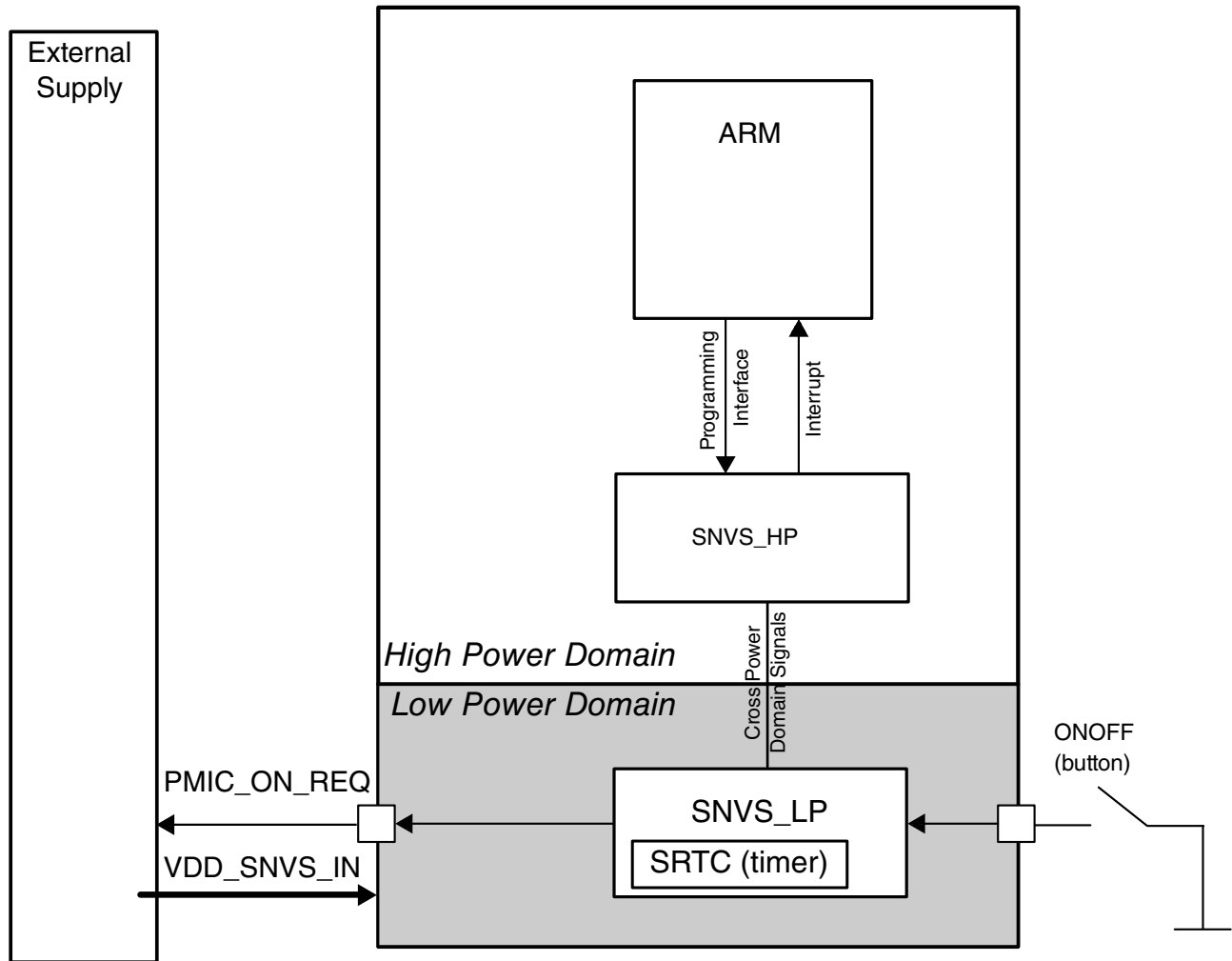


Figure 6-28. SNVS Block Diagram

6.4.2.1 SNVS power domains

In some versions of SNVS (including this version), the LP (Low Power) section is implemented in an independent power domain from the HP (High Power) section, and most other logic on the chip. Throughout the SNVS documentation whenever mention is made of "always-on" logic, this assumes a version of SNVS that implements an independent power domain for the LP section, and that the power for this section is supplied by an uninterrupted power supply. The purpose for the independent power domain is so that data can be retained and certain logic can remain functional even when the main chip logic is powered down. But this is possible only if the LP domain remains powered via an uninterrupted power supply when the main chip power domain is powered off. Usually this uninterrupted power supply would be a battery, with possibly some power management logic to power the LP section from main power (and perhaps

recharge the battery) when main power is on, and switch to battery power when the main power is off. In versions of SNVS with an independent LP power domain the LP section can be electrically isolated from the rest of the chip logic to ensure that its logic does not get corrupted when the main chip is powered down. If the battery runs down or is removed, an LP POR will occur when the LP section next powers up. Note that some OEMs may choose to connect LP power to HP/main chip power and dispense with a battery. In that case the SNVS will operate the same as an SNVS without an independent LP power domain. No state will be retained in the LP section when the chip is powered down, and an LP POR will occur whenever there is an HP POR.

6.4.2.2 Power glitch detector (PGD)

SNVS_LP incorporates a mechanism to detect glitches on the SNVS_LP power supply that might cause the LP control, status, and secure counter values to change. The mechanism works as follows:

1. The PGD register (LPPGDR) is loaded with the known specific value 4173_6166h as part of the SNVS initialization process.
2. Subsequently, this register's value is continuously compared to the hardwired value 4173_6166h.
3. If the comparison indicates that any bit has changed, a power glitch violation is asserted.

Power glitch detection is always enabled and cannot be disabled. At LP POR this register is reset to all 0's, so the hardwired comparison fails and a power glitch violation is reported. Therefore, before programming any feature in the SNVS the power glitch violation should be cleared. The initialization software should write the proper value (4173_6166h) into LPPGDR (see [SNVS_LP Power Glitch Detector Register \(LPPGDR\)](#)) and should then clear the power glitch record in the LP status register (see [SNVS_LP Status Register \(LPSR\)](#)).

The following figure shows the PGD mechanism.

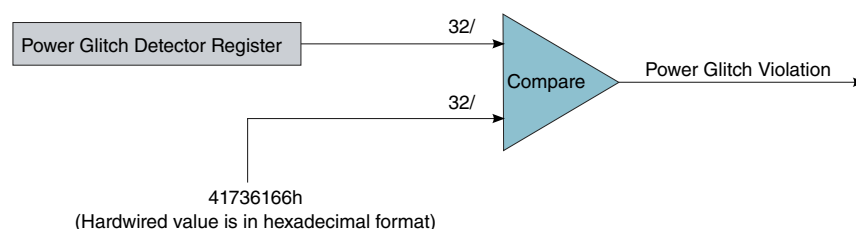


Figure 6-29. Power glitch detector

6.4.2.3 SNVS clock sources

The SNVS has the following clock sources:

- System peripheral clock input. This clock is used by the SNVS's internal logic, for example, the Security State Machine. This clock can be gated outside of the module when the SNVS indicates that it is not in use.
- HP RTC clock. This clock is used by SNVS_HP real-time counter. This clock does not need to be synchronous with other clocks.

6.4.3 Runtime Procedures

SNVS implements a number of features that are intended to be accessed by software at runtime (as opposed to accessed at boot time). These features include:

- Real Time Clock (see [SNVS_HP Real Time Counter](#))
- General Purpose Register (see [Using the General-Purpose Register](#))

Procedures for using these features are described in the following sections.

6.4.3.1 Using SNVS Timer Facilities

SNVS incorporates timer facilities that can optionally generate an interrupt at a specified time. As described in the following sections, SNVS_HP incorporates a Real Time Counter that is available for general use, and SNVS_LP incorporates a Secure Real Time Counter intended for security applications.

6.4.3.1.1 SNVS_HP Real Time Counter

SNVS_HP implements a real time counter that can be read or written by any application; it has no privileged software access restrictions. When the chip is powered down the RTC is not active and it is reset at chip POR. The RTC can be used to generate a functional interrupt request either at a specific time, or at a specific frequency, or both. To generate an interrupt request at a specific time HPTA_EN is set to 0, the desired time is written to HPTA_MS and HPTA_LS and then HPTA_EN is set to 1. HPTA_EN, HPTA_MS and HPTA_LS can be written by any software that has access to SNVS registers; there are no privileged access restrictions. The counter can be synchronized to the SNVS_LP SRTC by writing to the HP_TS bit of SNVS_HP Control Register. This is particularly useful if the SNVS_LP is powered from an uninterrupted power source because the RTC can then be set from a chip-internal time source.

6.4.3.1.2 RTC/SRTC control bits setting

All SNVS registers are programmed from the register bus, consequently any software-initiated changes are synchronized with the IP clock. Several registers can also change synchronously with the RTC/SRTC clock after they are programmed. To avoid IP clock and RTC/SRTC clock synchronization issues, the following values can be changed only when the corresponding function is disabled.

Table 6-45. RTC/SRTC synchronized values list

Function	Value/register	Control bit setting
HP section		
HP Real Time Counter	HPRTCMR and HPRTCLR Registers	RTC_EN = 0 : HPRTCMR/HPRTCLR can be programmed RTC_EN = 1 : HPRTCMR/HPRTCLR cannot be programmed
HP Time Alarm	HPTAMR and HPTALR Registers	HPTA_EN = 0 : HPTAMR/HPTALR can be programmed HPTA_EN = 1 : HPTAMR/HPTALR cannot be programmed
LP section		
LP Secure Real Time Counter	LPRTCMR and LPRTCLR Registers	SRTC_ENV = 0 : LPRTCMR/LPRTCLR can be programmed SRTC_ENV = 1 : LPRTCMR/LPRTCLR cannot be programmed
LP Time Alarm	LPTAR Register	LPTA_EN = 0 : LPTAR can be programmed LPTA_EN = 1 : LPTAR cannot be programmed

Use the following steps to program synchronized values:

1. Check the enable bit value. If set, clear it.
2. Verify that the enable bit is cleared. There are two reasons to verify the enable bit's setting:
 - Enable bit clearing does not happen immediately; it takes three IP clock cycles and two RTC/SRTC clock cycles to change the enable bit's value.
 - If the enable bit is locked for programming, it cannot be cleared.
3. Program the desired value.
4. Set the enable bit; it takes three IP clock cycles and two RTC/SRTC clock cycles for the bit to set.

NOTE

Incrementing the value programmed into RTC/SRTC registers by two compensates for the two RTC/SRTC clock cycle delay that is required to enable the counter.

6.4.3.1.3 Reading RTC and SRTC values

Software should follow the following procedure to ensure that it has read correct data from the RTC (HPRTCMR and HPRTCLR) and SRTC (LPSRTCMR and LPSRTCLR) registers:

- Read the most-significant half and the least-significant half of the RTC/SRTC and then read both halves again. If the values read are the same both times, the value is correct.
- If the two consecutive pairs of reads yield different results, perform two more reads.

The worst case scenario may require three sessions of two consecutive pairs of reads. There are several reasons that the values may be incorrectly read initially:

- Synchronization issues between the RTC/SRTC clock and the system clock
- Since the counter continues to increment, there may be a carry from the least-significant 32-bits to the most-significant bits in between reading the two halves of the counter

6.4.3.2 Using Other SNVS Registers

The section below describes how to use the General Purpose Register.

6.4.3.2.1 Using the General-Purpose Register

SNVS implements a 128-bit general-purpose register allows software to store a small amount of data. To maintain backward compatibility with versions of SNVS that implement only a 32-bit general purpose register, the most-significant word of the general purpose register is aliased to the original legacy address, and to maintain backward compatibility with versions of snvs_module_name that implement a 128-bit general purpose register, the most-significant half of the general purpose register is aliased to the previous legacy address address. The data in the GPR will be retained during system power-down mode as long as the SNVS_LP remains powered by an uninterrupted power source.

6.4.4 Reset and Initialization of SNVS

SNVS is implemented in two sections (HP and LP) that both must be initialized by software. If the SNVS_LP is powered by an uninterrupted power source that is separate from main SoC power, then SNVS can operate in either of two modes, depending upon whether the main SoC power is on or off. During main SoC power-down SNVS_HP is powered-down, but SNVS_LP is powered from the backup power supply and is electrically isolated from the rest of the chip. In this mode SNVS_LP keeps its registers' values but the LP registers cannot be read or written. During main SoC power-up the isolation of SNVS_LP is disabled and both SNVS_HP and SNVS_LP are powered from the main SoC power. Both LP and HP registers can be read and written (locks and privilege modes permitting). Signals between the SNVS_HP and SNVS_LP sections are enabled and all SNVS functions are operational.

Since the HP and LP sections reside in different power domains, the POR for the two sections can occur at different times. If the SNVS_LP section remains powered by an uninterrupted power source when the main SoC power is off, SNVS_LP is initialized rarely, typically once when the device is first powered on and again whenever the battery is replaced. During main SoC power-up the isolation of SNVS_LP is disabled and both SNVS_HP and SNVS_LP are powered from the main SoC power. Signals between the SNVS_HP and SNVS_LP sections are enabled and all SNVS functions are operational. The SNVS_HP section is powered from the main SoC power, so it must be initialized after the device is powered on.

6.4.5 SNVS register descriptions

This section contains detailed register descriptions for the SNVS registers. Each description includes a standard register diagram and register table. The register table provides detailed descriptions of the register bit and field functions, in bit order.

SNVS registers consist of two types:

- Privileged read/write accessible
- Non-privileged read/write accessible

Privileged read/write accessible registers can only be accessed for read/write by privileged software. Unauthorized write accesses are ignored, and unauthorized read accesses return zero. Non-privileged software can access privileged access registers when the non-privileged software access enable bit is set in the SNVS_HP Command Register.

- Non-Secure
- Trusted
- Secure

Non-privileged read/write accessible registers are read/write accessible by any software.

The LP register values are set only on LP POR and are unaffected by System (HP) POR. The HP registers are set only on System POR and are unaffected by LP POR.

The following table shows the SNVS main memory map.

NOTE

For more information on security-related bitfields, see the Security Reference Manual.

6.4.5.1 SNVS Memory map

SNVS base address: 3037_0000h

Offset	Register	Width (In bits)	Access	Reset value
4h	SNVS_HP Command Register (HPCOMR)	32	RW	0000_0000h
8h	SNVS_HP Control Register (HPCR)	32	RW	0000_0000h
14h	SNVS_HP Status Register (HPSR)	32	RW	8000_0000h
24h	SNVS_HP Real Time Counter MSB Register (HPRTCMR)	32	RW	0000_0000h
28h	SNVS_HP Real Time Counter LSB Register (HPRTCLR)	32	RW	0000_0000h
2Ch	SNVS_HP Time Alarm MSB Register (HPTAMR)	32	RW	0000_0000h
30h	SNVS_HP Time Alarm LSB Register (HPTALR)	32	RW	0000_0000h
34h	SNVS_LP Lock Register (LPLR)	32	RW	0000_0000h
38h	SNVS_LP Control Register (LPCR)	32	RW	0000_0020h
4Ch	SNVS_LP Status Register (LPSR)	32	RW	0000_0008h
5Ch	SNVS_LP Secure Monotonic Counter MSB Register (LPSMCMR)	32	RW	0000_0000h
60h	SNVS_LP Secure Monotonic Counter LSB Register (LPSMCLR)	32	RW	0000_0000h
64h	SNVS_LP Power Glitch Detector Register (LPPGDR)	32	RW	0000_0000h
68h	SNVS_LP General Purpose Register 0 (legacy alias) (LPGPR0_legacy_alias)	32	RW	0000_0000h
90h - 9Ch	SNVS_LP General Purpose Registers 0 .. 3 (LPGPR0_alias - LPGPR3_alias)	32	RW	0000_0000h
100h - 10Ch	SNVS_LP General Purpose Registers 0 .. 3 (LPGPR0 - LPGPR3)	32	RW	0000_0000h
BF8h	SNVS_HP Version ID Register 1 (HPVIDR1)	32	RO	003E_0103h
BFCh	SNVS_HP Version ID Register 2 (HPVIDR2)	32	RO	0600_0300h

6.4.5.2 SNVS_HP Command Register (HPCOMR)

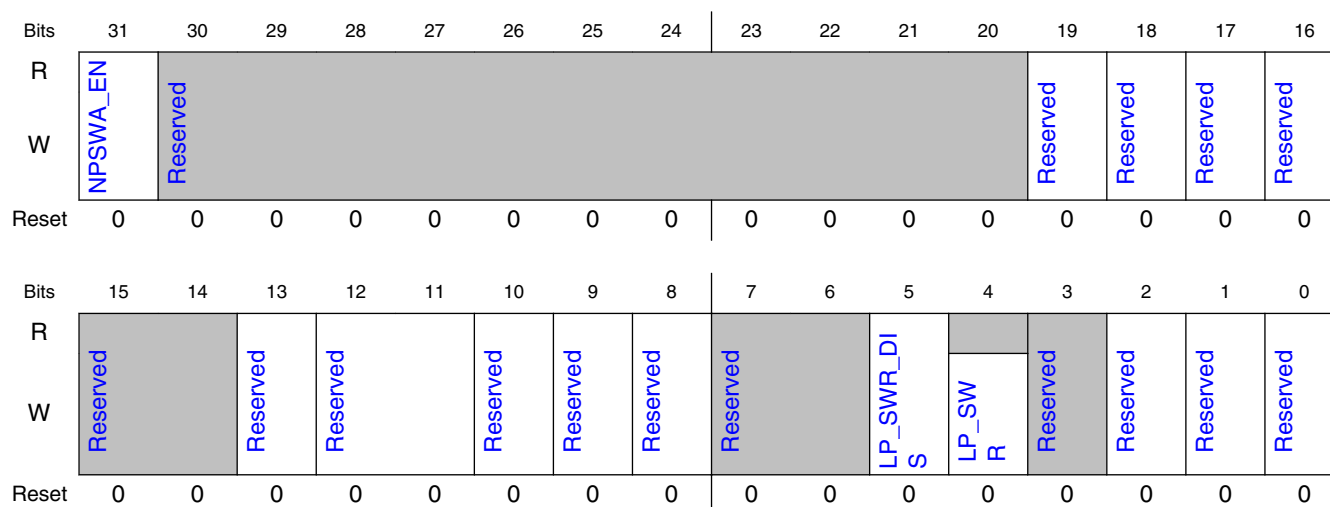
6.4.5.2.1 Offset

Register	Offset
HPCOMR	4h

6.4.5.2.2 Function

The SNVS_HP Command Register contains the command, configuration, and control bits for the SNVS block. This is a privileged write register.

6.4.5.2.3 Diagram



6.4.5.2.4 Fields

Field	Function
31 NPSWA_EN	Non-Privileged Software Access Enable When set, allows non-privileged software to access all SNVS registers, including those that are privileged software read/write access only. 0 Only privileged software can access privileged registers 1 Any software can access privileged registers
30-20	Reserved

Table continues on the next page...

Field	Function
—	
19 —	Reserved
18 —	Reserved
17 —	Reserved
16 —	Reserved
15-14 —	Reserved
13 —	Reserved
12-11 —	Reserved
10 —	Reserved
9 —	Reserved
8 —	Reserved
7-6 —	Reserved
5 LP_SWR_DIS	LP Software Reset Disable When set, disables the LP software reset. Once set, this bit can only be reset by the system reset. 0b - LP software reset is enabled 1b - LP software reset is disabled
4 LP_SWR	LP Software Reset When set to 1, the registers in the SNVS_LP section are reset. 0b - No Action 1b - Reset LP section
3 —	Reserved
2 —	Reserved
1 —	Reserved
0 —	Reserved

6.4.5.3 SNVS_HP Control Register (HPCR)

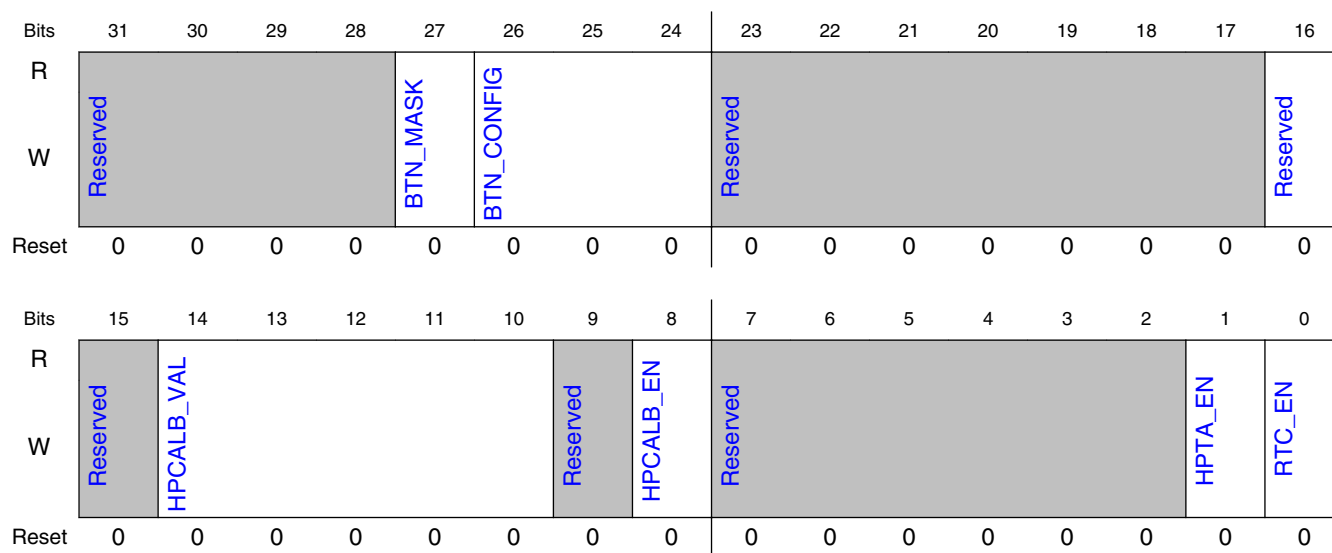
6.4.5.3.1 Offset

Register	Offset
HPCR	8h

6.4.5.3.2 Function

The SNVS_HP Control Register contains various control bits of the HP section of SNVS. This is *not* a privileged write register.

6.4.5.3.3 Diagram



6.4.5.3.4 Fields

Field	Function
31-28 —	Reserved
27 BTN_MASK	Button interrupt mask. This bit is used to mask the ipi_snvs_btn_int_b (button) interrupt request. 0: Interrupt disabled

Table continues on the next page...

Field	Function
	1: Interrupt enabled
26-24 BTN_CONFIG	Button Configuration. This field is used to configure which feature of the button (BTN) input signal constitutes "active". 000: Button signal is active high 001: Button signal is active low 010: Button signal is active on the falling edge 011: Button signal is active on the rising edge 100: Button signal is active on any edge All other patterns are Reserved
23-17 —	Reserved
16 —	Reserved
15 —	Reserved
14-10 HPCALB_VAL	HP Calibration Value Defines signed calibration value for the HP Real Time Counter. This field can be programmed only when RTC Calibration is disabled (HPCALB_EN is not set). This is a 5-bit 2's complement value, hence the allowable calibration values are in the range from -16 to +15 counts per 32768 ticks of the counter. 00000b - +0 counts per each 32768 ticks of the counter 00001b - +1 counts per each 32768 ticks of the counter 00010b - +2 counts per each 32768 ticks of the counter 01111b - +15 counts per each 32768 ticks of the counter 10000b - -16 counts per each 32768 ticks of the counter 10001b - -15 counts per each 32768 ticks of the counter 11110b - -2 counts per each 32768 ticks of the counter 11111b - -1 counts per each 32768 ticks of the counter
9 —	Reserved
8 HPCALB_EN	HP Real Time Counter Calibration Enabled Indicates that the time calibration mechanism is enabled. 0b - HP Timer calibration disabled 1b - HP Timer calibration enabled
7-2 —	Reserved
1 HPTA_EN	HP Time Alarm Enable When set, the time alarm interrupt is generated if the value in the HP Time Alarm Registers is equal to the value of the HP Real Time Counter. 0b - HP Time Alarm Interrupt is disabled 1b - HP Time Alarm Interrupt is enabled
0 RTC_EN	HP Real Time Counter Enable. This bit syncs with the 32KHz clock. It won't update with the bus clock. 0b - RTC is disabled 1b - RTC is enabled

6.4.5.4 SNVS_HP Status Register (HPSR)

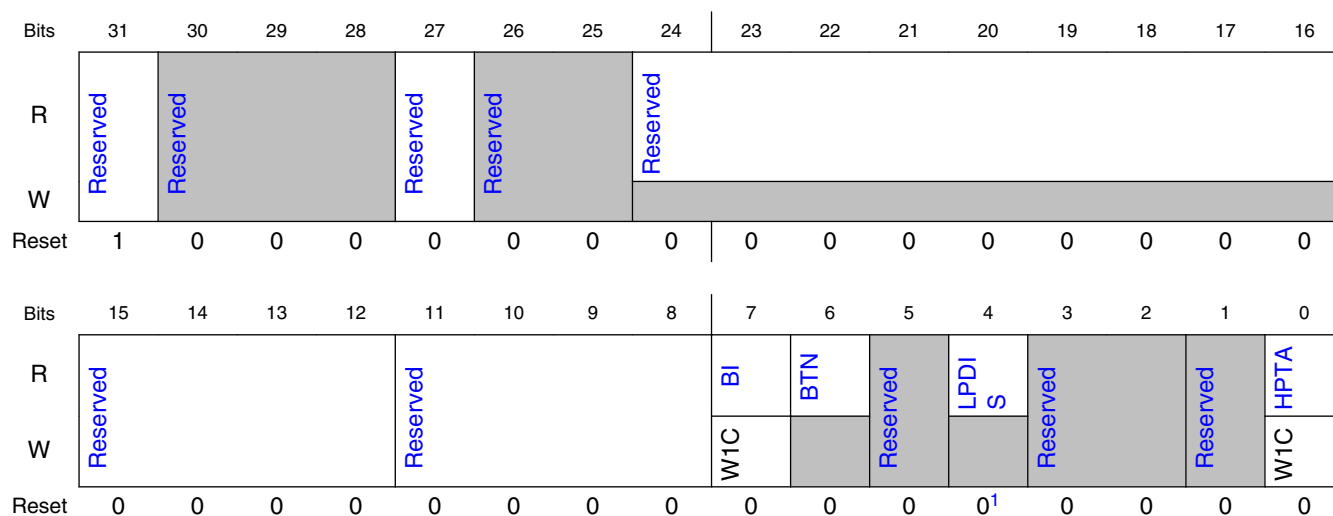
6.4.5.4.1 Offset

Register	Offset
HPSR	14h

6.4.5.4.2 Function

The HP Status Register reflects the internal state of the SNVS. This is *not* a privileged write register.

6.4.5.4.3 Diagram



1. The value of Low Power Disable is determined by the *no_battery* input signal to SNVS.

6.4.5.4.4 Fields

Field	Function
31	Reserved
—	
30-28	Reserved
—	
27	Reserved

Table continues on the next page...

Field	Function
—	
26-25 —	Reserved
24-16 —	Reserved
15-12 —	Reserved
11-8 —	Reserved
7 BI	Button Interrupt Signal ipi_snvs_btn_int_b was asserted.
6 BTN	Button Value of the BTN input. This is the external button used for PMIC control. 0: BTN not pressed 1: BTN pressed
5 —	Reserved
4 LPDIS	Low Power Disable If 1, the low power section has been disabled by means of an input signal to SNVS.
3-2 —	Reserved
1 —	Reserved
0 HPTA	HP Time Alarm Indicates that the HP Time Alarm has occurred since this bit was last cleared. 0b - No time alarm interrupt occurred. 1b - A time alarm interrupt occurred.

6.4.5.5 SNVS_HP Real Time Counter MSB Register (HPRTC MR)

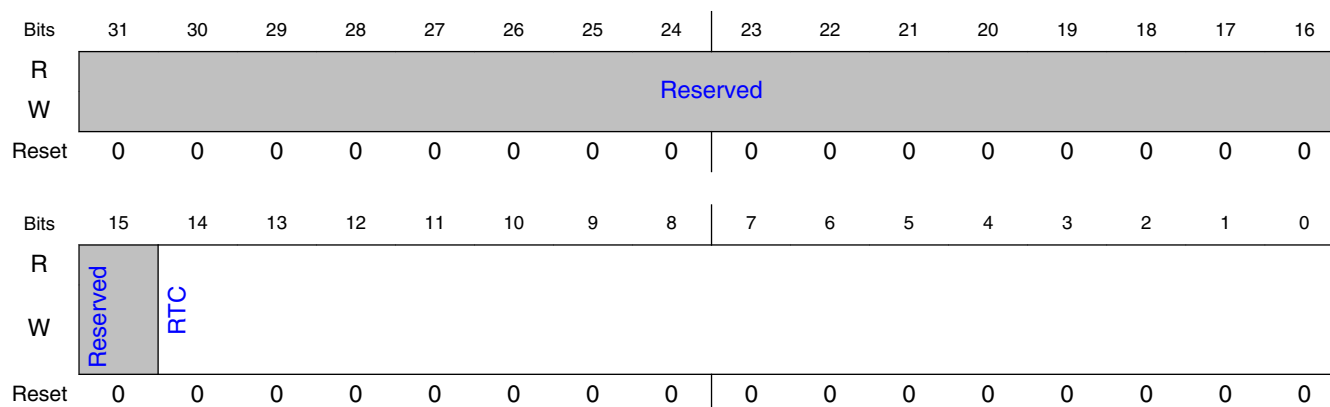
6.4.5.5.1 Offset

Register	Offset
HPRTC MR	24h

6.4.5.5.2 Function

The SNVS_HP Real Time Counter MSB register contains the 15 most-significant bits of the HP Real Time Counter. This is *not* a privileged write register.

6.4.5.5.3 Diagram



6.4.5.5.4 Fields

Field	Function
31-15 —	Reserved
14-0 RTC	HP Real Time Counter The most-significant 15 bits of the RTC. This register can be programmed only when RTC is not active (RTC_EN bit is not set).

6.4.5.6 SNVS_HP Real Time Counter LSB Register (HPRTCLR)

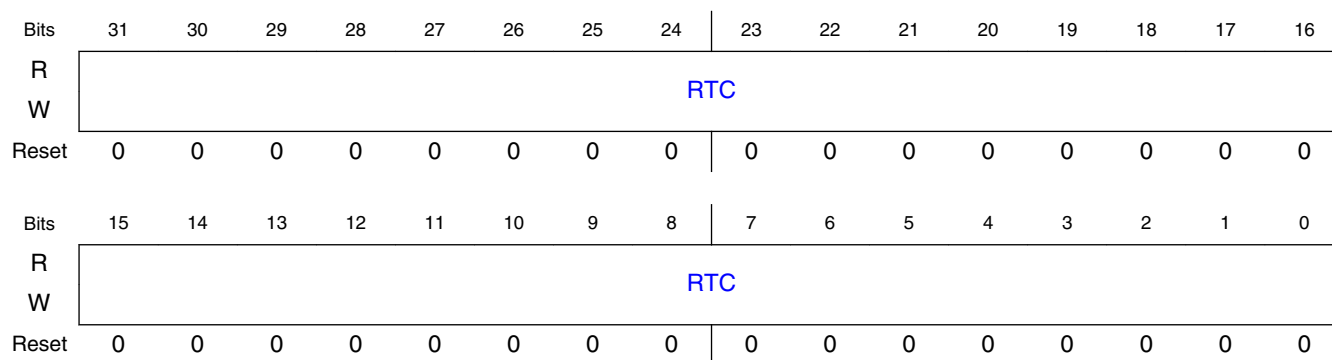
6.4.5.6.1 Offset

Register	Offset
HPRTCLR	28h

6.4.5.6.2 Function

The SNVS_HP Real Time Counter LSB register contains the 32 least-significant bits of the HP real time counter. This is *not* a privileged write register.

6.4.5.6.3 Diagram



6.4.5.6.4 Fields

Field	Function
31-0	HP Real Time Counter
RTC	least-significant 32 bits. This register can be programmed only when RTC is not active (RTC_EN bit is not set).

6.4.5.7 SNVS_HP Time Alarm MSB Register (HPTAMR)

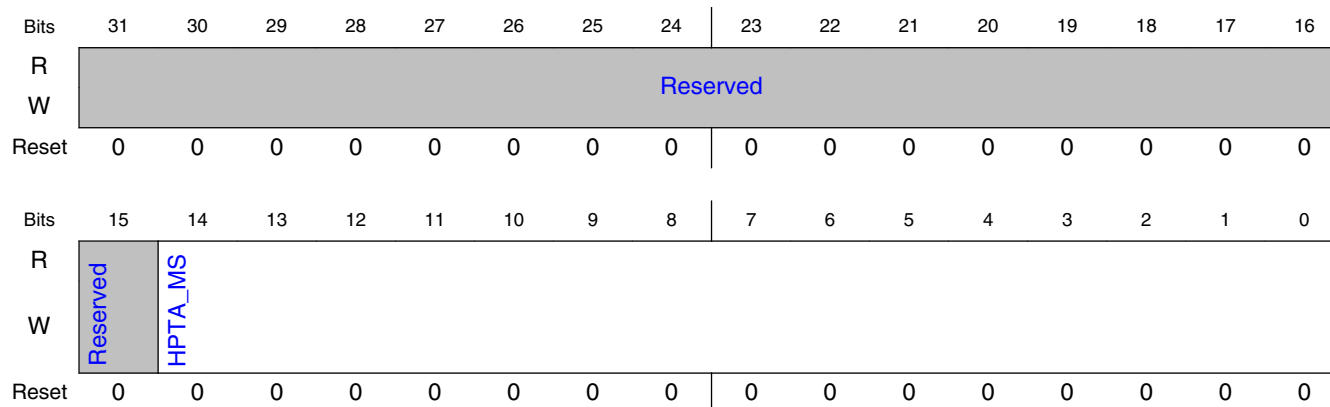
6.4.5.7.1 Offset

Register	Offset
HPTAMR	2Ch

6.4.5.7.2 Function

The SNVS_HP Time Alarm MSB register contains the most-significant bits of the SNVS_HP Time Alarm value. This is *not* a privileged write register.

6.4.5.7.3 Diagram



6.4.5.7.4 Fields

Field	Function
31-15	Reserved
—	
14-0	HP Time Alarm, most-significant 15 bits.
HPTA_MS	This register can be programmed only when HP time alarm is disabled (HPTA_EN bit is not set).

6.4.5.8 SNVS_HP Time Alarm LSB Register (HPTALR)

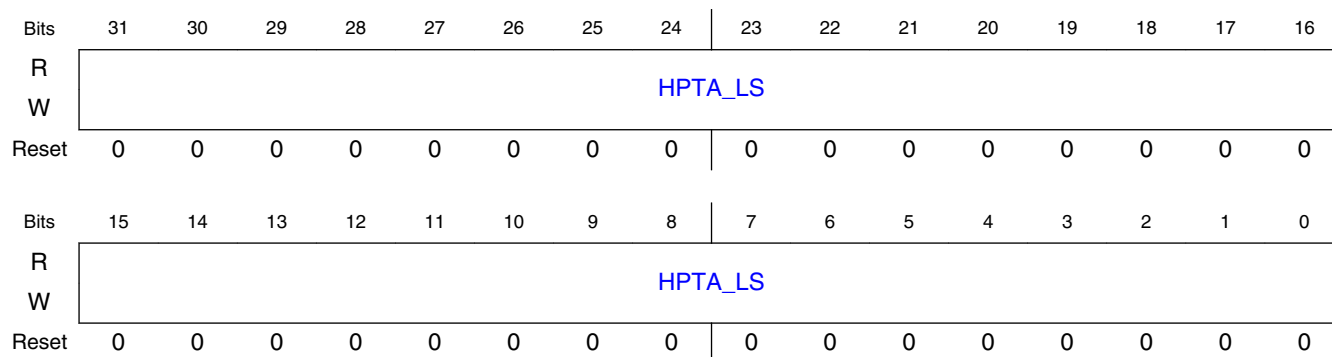
6.4.5.8.1 Offset

Register	Offset
HPTALR	30h

6.4.5.8.2 Function

The SNVS_HP Time Alarm LSB register contains the 32 least-significant bits of the SNVS_HP Time Alarm value. This is *not* a privileged write register.

6.4.5.8.3 Diagram



6.4.5.8.4 Fields

Field	Function
31-0	HP Time Alarm, 32 least-significant bits.
HPTA_LS	This register can be programmed only when HP time alarm is disabled (HPTA_EN bit is not set).

6.4.5.9 SNVS_LP Lock Register (LPLR)

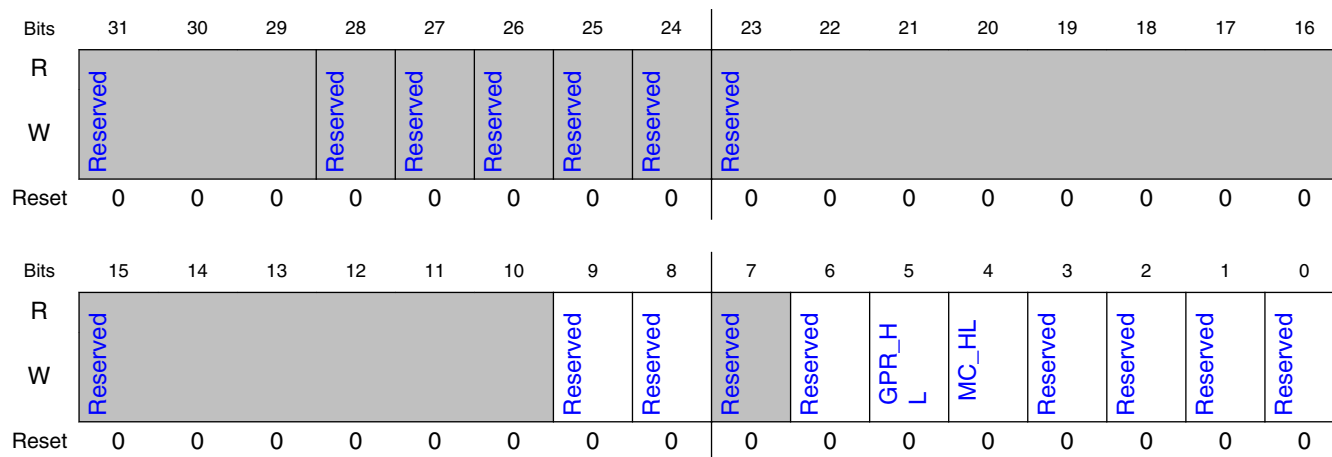
6.4.5.9.1 Offset

Register	Offset
LPLR	34h

6.4.5.9.2 Function

The SNVS_LP Lock Register contains lock bits for the SNVS_LP registers. This is a privileged write register.

6.4.5.9.3 Diagram



6.4.5.9.4 Fields

Field	Function
31-29 —	Reserved
28 —	Reserved
27 —	Reserved
26 —	Reserved
25 —	Reserved
24 —	Reserved
23-10 —	Reserved
9 —	Reserved
8 —	Reserved
7 —	Reserved
6 —	Reserved
5	General Purpose Register Hard Lock

Table continues on the next page...

Field	Function
GPR_HL	When set, prevents any writes to the GPR. Once set, this bit can only be reset by the LP POR. 0b - Write access is allowed. 1b - Write access is not allowed.
4 MC_HL	Monotonic Counter Hard Lock When set, prevents any writes (increments) to the MC Registers and MC_ENV bit. Once set, this bit can only be reset by the LP POR. 0b - Write access (increment) is allowed. 1b - Write access (increment) is not allowed.
3 —	Reserved
2 —	Reserved
1 —	Reserved
0 —	Reserved

6.4.5.10 SNVS_LP Control Register (LPCR)

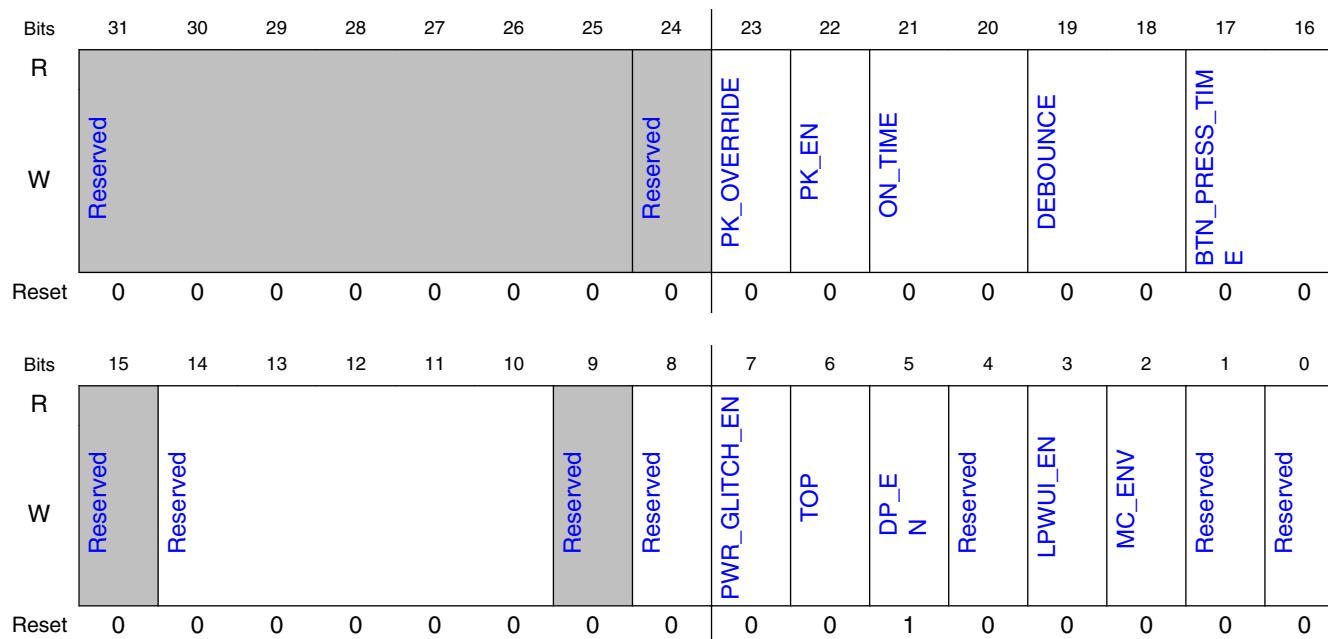
6.4.5.10.1 Offset

Register	Offset
LPCR	38h

6.4.5.10.2 Function

The SNVS_LP Control Register contains various control bits of the LP section of SNVS. This is a privileged write register.

6.4.5.10.3 Diagram



6.4.5.10.4 Fields

Field	Function
31-25 —	Reserved
24 —	Reserved
23 PK_OVERRIDE	PMIC On Request Override The value written to PK_OVERRIDE will be asserted on output signal snvs_lp_pk_override. That signal is used to override the IOMUX control for the PMIC I/O pad.
22 PK_EN	PMIC On Request Enable The value written to PK_EN will be asserted on output signal snvs_lp_pk_en. That signal is used to turn off the pullup/pulldown circuitry in the PMIC I/O pad.
21-20 ON_TIME	The ON_TIME field is used to configure the period of time after BTN is asserted before pmic_en_b is asserted to turn on the SoC power. 00: 500msec off->on transition time 01: 50msec off->on transition time 10: 100msec off->on transition time 11: 0msec off->on transition time
19-18 DEBOUNCE	This field configures the amount of debounce time for the BTN input signal. 00: 50msec debounce 01: 100msec debounce

Table continues on the next page...

Field	Function
	10: 500msec debounce 11: 0msec debounce
17-16 BTN_PRESS_TIME	This field configures the button press time out values for the PMIC Logic. 00 : 5 secs 01 : 10 secs 10 : 15 secs 11 : long press disabled (pmic_en_b will not be asserted regardless of how long BTN is asserted)
15 —	Reserved
14-10 —	Reserved
9 —	Reserved
8 —	Reserved
7 PWR_GLITCH_EN	Power Glitch Enable By default the detection of a power glitch does not cause the pmic_en_b signal to be asserted. Setting the Power Glitch Enable bit to 1 enables the power glitch event for the PMIC. 0 - disabled 1 - enabled
6 TOP	Turn off System Power Asserting this bit causes a signal to be sent to the Power Management IC to turn off the system power. This bit will clear once power is off. This bit is only valid when the Dumb PMIC is enabled. 0b - Leave system power on. 1b - Turn off system power.
5 DP_EN	Dumb PMIC Enabled When set, software can control the system power. When cleared, the system requires a Smart PMIC to automatically turn power off. 0b - Smart PMIC enabled. 1b - Dumb PMIC enabled.
4 —	Reserved
3 LPWUI_EN	LP Wake-Up Interrupt Enable This interrupt line should be connected to the external pin and is intended to inform the external chip about an SNVS_LP event (MC rollover, SRTC rollover, or time alarm). This wake-up signal can be asserted only when the chip (HP section) is powered down, and the LP section is isolated. 0 LP wake-up interrupt is disabled. 1 LP wake-up interrupt is enabled.
2 MC_ENV	Monotonic Counter Enabled and Valid When set, the MC can be incremented (by write transaction to the LPSMCMR or LPSMCLR). Once MC_SL or MC_HL bit is set this bit can be changed only by LP software reset or LP POR. 0b - MC is disabled or invalid.

Table continues on the next page...

Secure Non-Volatile Storage (SNVS)

Field	Function
	1b - MC is enabled and valid.
1 —	Reserved
0 —	Reserved

6.4.5.11 SNVS_LP Status Register (LPSR)

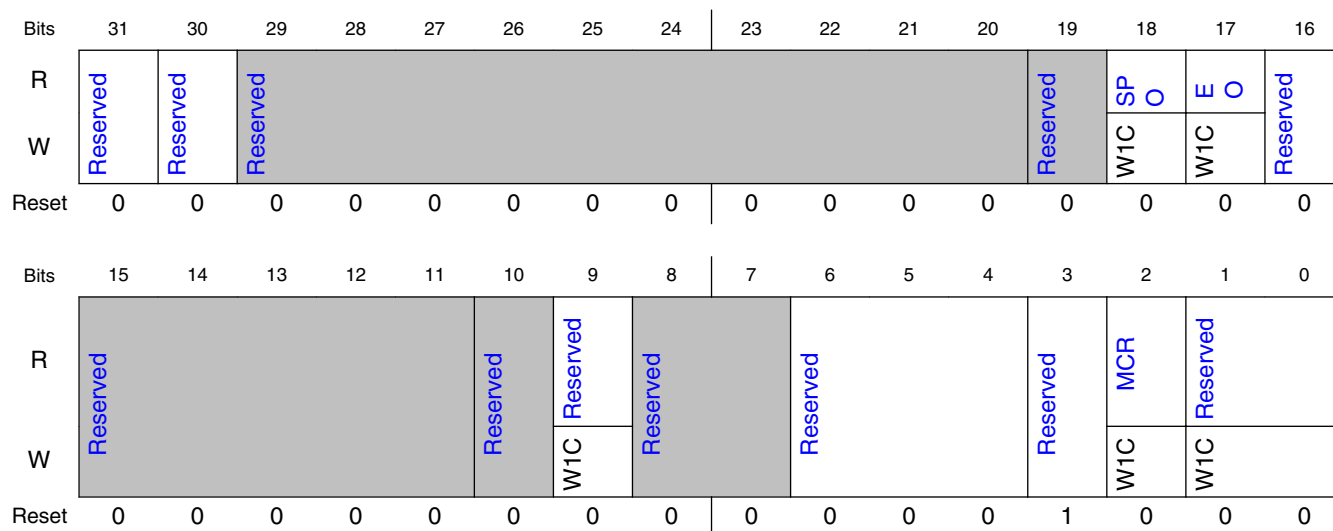
6.4.5.11.1 Offset

Register	Offset
LPSR	4Ch

6.4.5.11.2 Function

The SNVS_LP Status Register reflects the internal state and behavior of the SNVS_LP. This is a privileged write register.

6.4.5.11.3 Diagram



6.4.5.11.4 Fields

Field	Function
31 —	Reserved
30 —	Reserved
29-20 —	Reserved
19 —	Reserved
18 SPO	Set Power Off The SPO bit is set when the power button is pressed longer than the configured debounce time. Writing to the SPO bit will clear the set_pwr_off_irq interrupt. 0b - Set Power Off was not detected. 1b - Set Power Off was detected.
17 EO	Emergency Off This bit is set when a power off is requested. 0b - Emergency off was not detected. 1b - Emergency off was detected.
16 —	Reserved
15-11 —	Reserved
10 —	Reserved
9 —	Reserved
8-7 —	Reserved
6-4 —	Reserved
3 —	Reserved
2 MCR	Monotonic Counter Rollover 0b - MC has not reached its maximum value. 1b - MC has reached its maximum value.
1-0 —	Reserved

6.4.5.12 SNVS_LP Secure Monotonic Counter MSB Register (LPSMCMR)

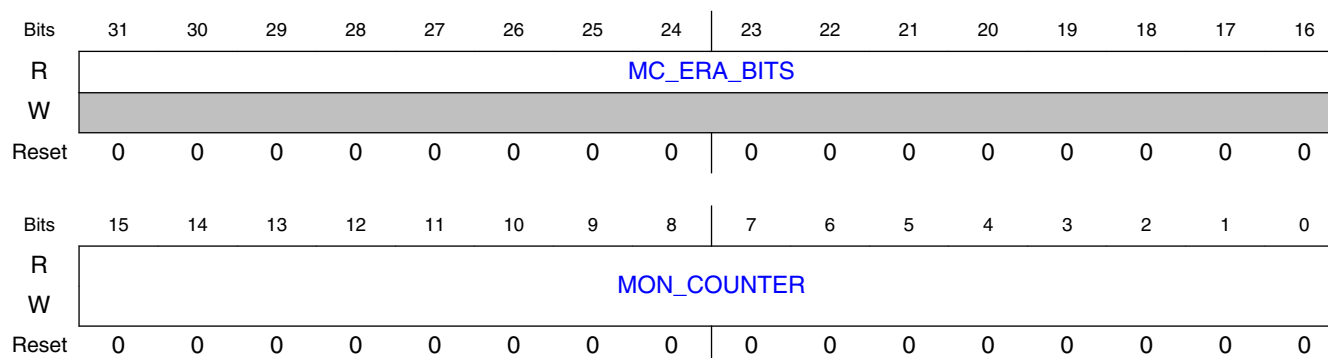
6.4.5.12.1 Offset

Register	Offset
LPSMCMR	5Ch

6.4.5.12.2 Function

The SNVS_LP Secure Monotonic Counter MSB Register contains the monotonic counter era bits and the most-significant 16 bits of the monotonic counter. The monotonic counter is incremented by one if there is a write command to the LPSMCMR or LPSMCLR register. This is a non-privileged read-only register.

6.4.5.12.3 Diagram



6.4.5.12.4 Fields

Field	Function
31-16 MC_ERA_BITS	Monotonic Counter Era Bits These bits are inputs to the module and typically connect to fuses. When the Monotonic Counter is in use (i.e. enabled and valid and powered by an uninterrupted power source), and the boot software detects that the Monotonic Counter most-significant 16 Bits and Monotonic Counter LSB Register have been reset (MC_ENV=0), the boot software can take action to ensure that the value in the monotonic counter remains monotonic (i.e. never decreasing). The action is to blow an additional MC_ERA_BITS fuse. Since the MC_ERA_BITS field forms the most-significant field of the monotonic counter, blowing an additional fuse guarantees that the new monotonic counter value is higher than any previous value. Since the Monotonic Counter is reset on an LP Software Reset, an excessive number of MC_ERA_BITS fuses may be consumed if LP Software Reset is used repeatedly.
15-0	Monotonic Counter most-significant 16 Bits

Field	Function
MON_COUNTER	Note that writing to this register does <i>not</i> change the value of this field to the value that was written. The 48-bit monotonic counter value (consisting of LPSMCMR[MON_COUNTER] prepended to LPSMCLR[MON_COUNTER]) is incremented by one when: • A write transaction to the LPSMCMR or LPSMCLR register is detected. • The MC_ENV bit is set. • MC_SL and MC_HL bits are not set. This value can be reset only by LP software reset or LP POR.

6.4.5.13 SNVS_LP Secure Monotonic Counter LSB Register (LPSMCLR)

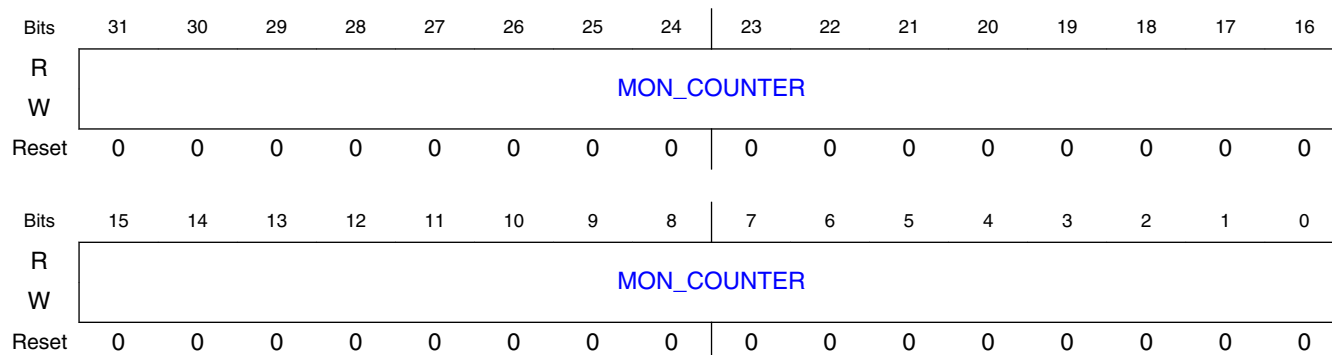
6.4.5.13.1 Offset

Register	Offset
LPSMCLR	60h

6.4.5.13.2 Function

The SNVS_LP Secure Monotonic Counter LSB Register contains the 32 least-significant bits of the monotonic counter. The MC is incremented by one if there is a write command to the LPSMCMR or LPSMCLR register. This is a non-privileged read-only register.

6.4.5.13.3 Diagram



6.4.5.13.4 Fields

Field	Function
31-0 MON_COUNTER	Monotonic Counter bits Note that writing to this register does <i>not</i> change the value of this field to the value that was written. The 48-bit monotonic counter value (consisting of LPSMCMR[MON_COUNTER] prepended to LPSMCLR[MON_COUNTER]) is incremented by one when: • A write transaction to the LPSMCMR or LPSMCLR register is detected. • The MC_ENV bit is set. • MC_SL and MC_HL bits are not set. This value can be reset only by LP software reset or LP POR.

6.4.5.14 SNVS_LP Power Glitch Detector Register (LPPGDR)

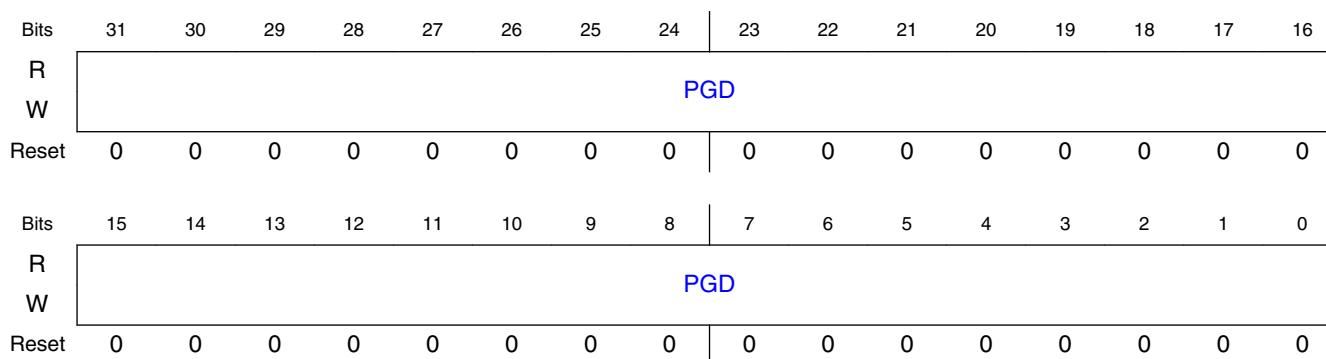
6.4.5.14.1 Offset

Register	Offset
LPPGDR	64h

6.4.5.14.2 Function

The SNVS_LP Power Glitch Detector Register is a 32-bit read/write register that is used for storing the power glitch detector value, as described in [Power glitch detector \(PGD\)](#). This is a privileged write register.

6.4.5.14.3 Diagram



6.4.5.14.4 Fields

Field	Function
31-0 PGD	Power Glitch Detector Value

6.4.5.15 SNVS_LP General Purpose Register 0 (legacy alias) (LPGPR0_legacy_alias)

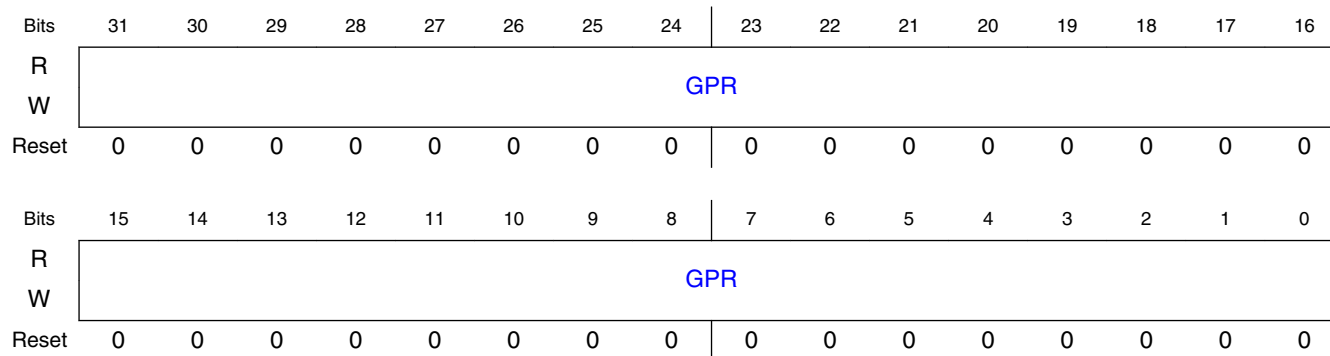
6.4.5.15.1 Offset

Register	Offset
LPGPR0_legacy_alias	68h

6.4.5.15.2 Function

See register [SNVS_LP General Purpose Registers 0 .. 3 \(LPGPR0 - LPGPR3\)](#).

6.4.5.15.3 Diagram



6.4.5.15.4 Fields

Field	Function
31-0	General Purpose Register
GPR	When GPR_SL or GPR_HL bit is set, the register cannot be programmed.

6.4.5.16 SNVS_LP General Purpose Registers 0 .. 3 (LPGPR0_alias - LPGPR3_alias)

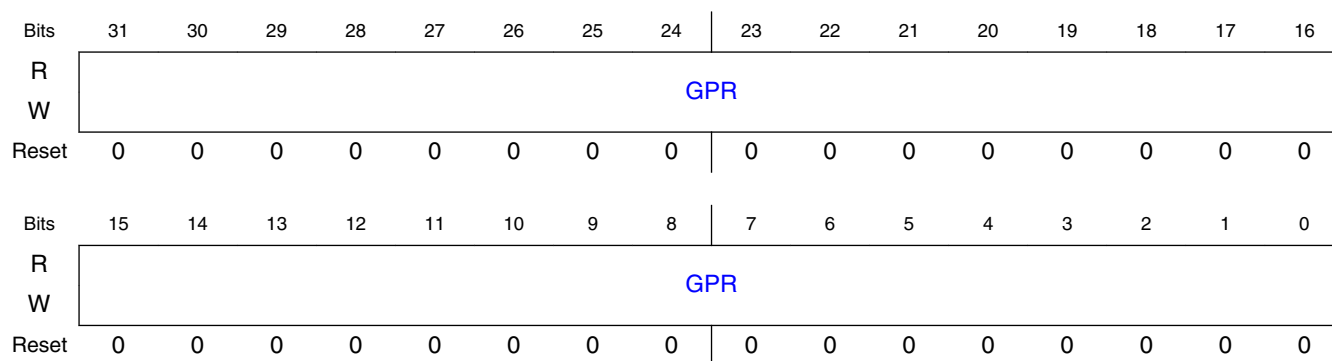
6.4.5.16.1 Offset

Register	Offset
LPGPR0_alias	90h
LPGPR1_alias	94h
LPGPR2_alias	98h
LPGPR3_alias	9Ch

6.4.5.16.2 Function

See register [SNVS_LP General Purpose Registers 0 .. 3 \(LPGPR0 - LPGPR3\)](#).

6.4.5.16.3 Diagram



6.4.5.16.4 Fields

Field	Function
31-0	General Purpose Register
GPR	When GPR_SL or GPR_HL bit is set, the register cannot be programmed.

6.4.5.17 SNVS_LP General Purpose Registers 0 .. 3 (LPGPR0 - LPGPR3)

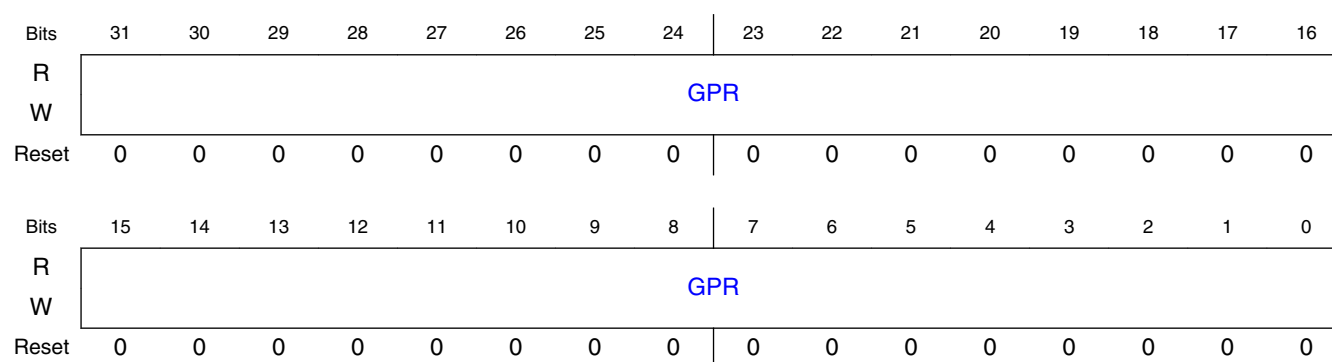
6.4.5.17.1 Offset

Register	Offset
LPGPR0	100h
LPGPR1	104h
LPGPR2	108h
LPGPR3	10Ch

6.4.5.17.2 Function

The SNVS_LP General Purpose Register is a 128-bit read/write register located in SNVS_LP, which can be used by any application for retaining data during an SoC power-down mode. This is a privileged read/write register. The full GPR register is accessed as 4 32-bit registers located in successive word addresses starting at offset 100h. For backward compatibility with earlier versions of SNVS, LPGPR0..LPGPR3 are aliased at the earlier offset of 90h and LPGPR0 is also aliased at its original offset of 68h. New software should access the GPR register at the preferred offset of 100h. The GPR will be automatically zeroized when an enabled security event occurs, unless GPR zeroization is disabled via the GPR_Z_DIS bit in the LP Control Register.

6.4.5.17.3 Diagram



6.4.5.17.4 Fields

Field	Function
31-0	General Purpose Register

Field	Function
GPR	When GPR_SL or GPR_HL bit is set, the register cannot be programmed.

6.4.5.18 SNVS_HP Version ID Register 1 (HPVIDR1)

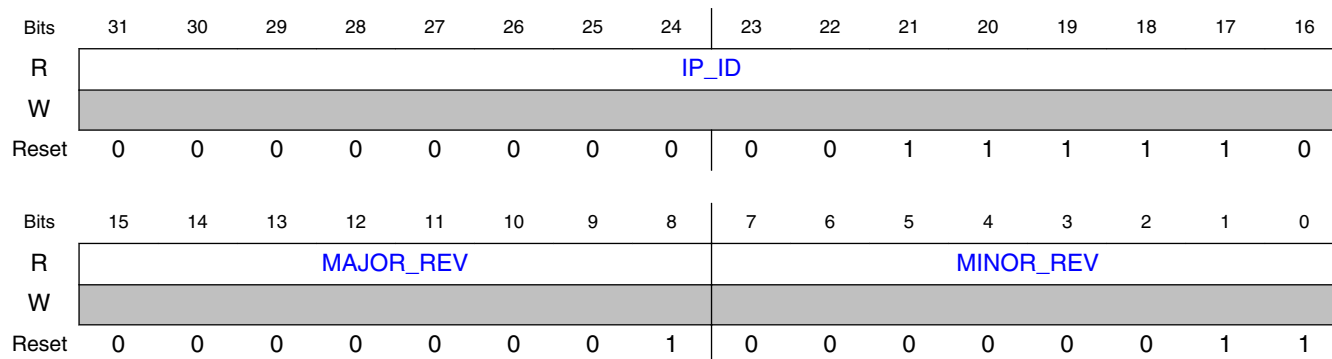
6.4.5.18.1 Offset

Register	Offset
HPVIDR1	BF8h

6.4.5.18.2 Function

The SNVS_HP Version ID Register 1 is a non-privileged read-only register that contains the current version of the SNVS. The version consists of a module ID, a major version number, and a minor version number.

6.4.5.18.3 Diagram



6.4.5.18.4 Fields

Field	Function
31-16 IP_ID	SNVS block ID
15-8 MAJOR_REV	SNVS block major version number
7-0 MINOR_REV	SNVS block minor version number

6.4.5.19 SNVS_HP Version ID Register 2 (HPVIDR2)

6.4.5.19.1 Offset

Register	Offset
HPVIDR2	BFCh

6.4.5.19.2 Function

The SNVS_HP Version ID Register 2 is a non-privileged read-only register that indicates the current version of the SNVS. Version ID register 2 consists of the following fields: integration options, ECO revision, and configuration options.

6.4.5.19.3 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	IP_ERA								INTG_OPT							
W																
Reset	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ECO_REV								CONFIG_OPT							
W																
Reset	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0

6.4.5.19.4 Fields

Field	Function
31-24 IP_ERA	IP Era 00h - Era 1 or 2 03h - Era 3 04h - Era 4 05h - Era 5
23-16 INTG_OPT	SNVS Integration Options
15-8	SNVS ECO Revision

Table continues on the next page...

Field	Function
ECO_REV	
7-0 CONFIG_OPT	SNVS Configuration Options

6.5 System Reset Controller (SRC)

6.5.1 SRC Overview

The System Reset Controller (SRC) controls the reset and boot operation of the SoC.

It is responsible for the generation of all reset signals and boot decoding.

The reset controller determines the source and the type of reset, such as POR, COLD, and performs the necessary reset qualification and stretching sequences. Based on the type of reset, the reset logic generates the reset sequence for the entire IC. Whenever the chip is powered on, the reset is issued through SRC_ONOFF signal and the entire chip is reset.

6.5.1.1 Features

The SRC includes the following features.

- Receives and handles the resets from all the reset sources
- Resets the appropriate domains based upon the resets sources and the nature of the reset
- Latches the SRC_BOOT_MODE pins and common configuration signals from the internal fuse

6.5.2 Clocks

The table found here describes the clock sources for SRC.

Please see [Clock Controller Module \(CCM\)](#) for clock setting, configuration and gating information.

Table 6-46. SRC Clocks

Clock name	Clock Root	Description
ipg_clk	ipg_clk_root	Peripheral clock
ipg_clk_s	ipg_clk_root	Peripheral access clock

6.5.3 Top-level resets, power-up sequence and external supply integration

Information found here defines chip resets, power-up sequence, and external supply integration.

6.5.3.1 Reset and Power-up Flow

The chip presumes the following reset and power-up flow:

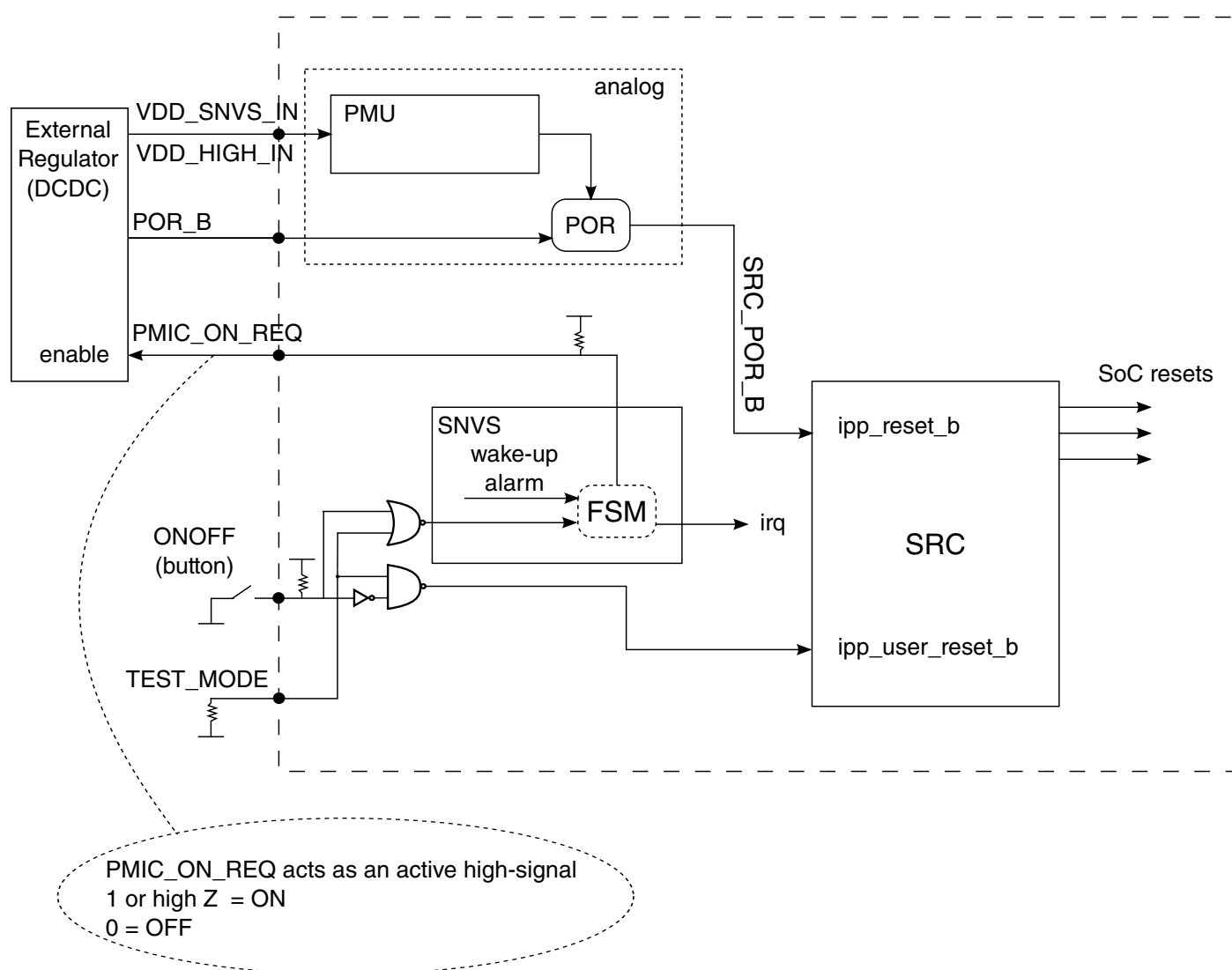


Figure 6-30. Chip reset scheme under PMU control

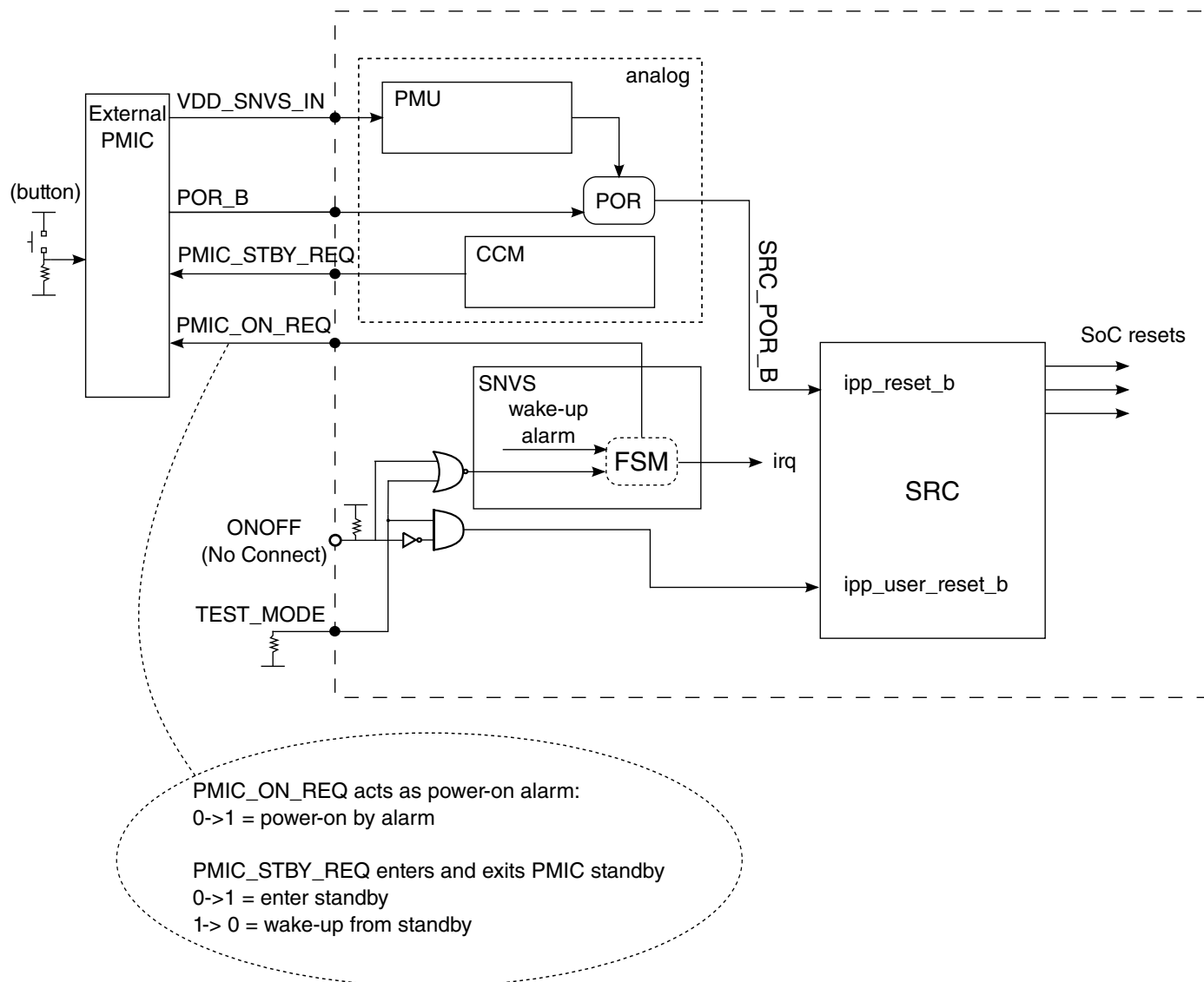


Figure 6-31. Chip reset scheme under external PMIC control

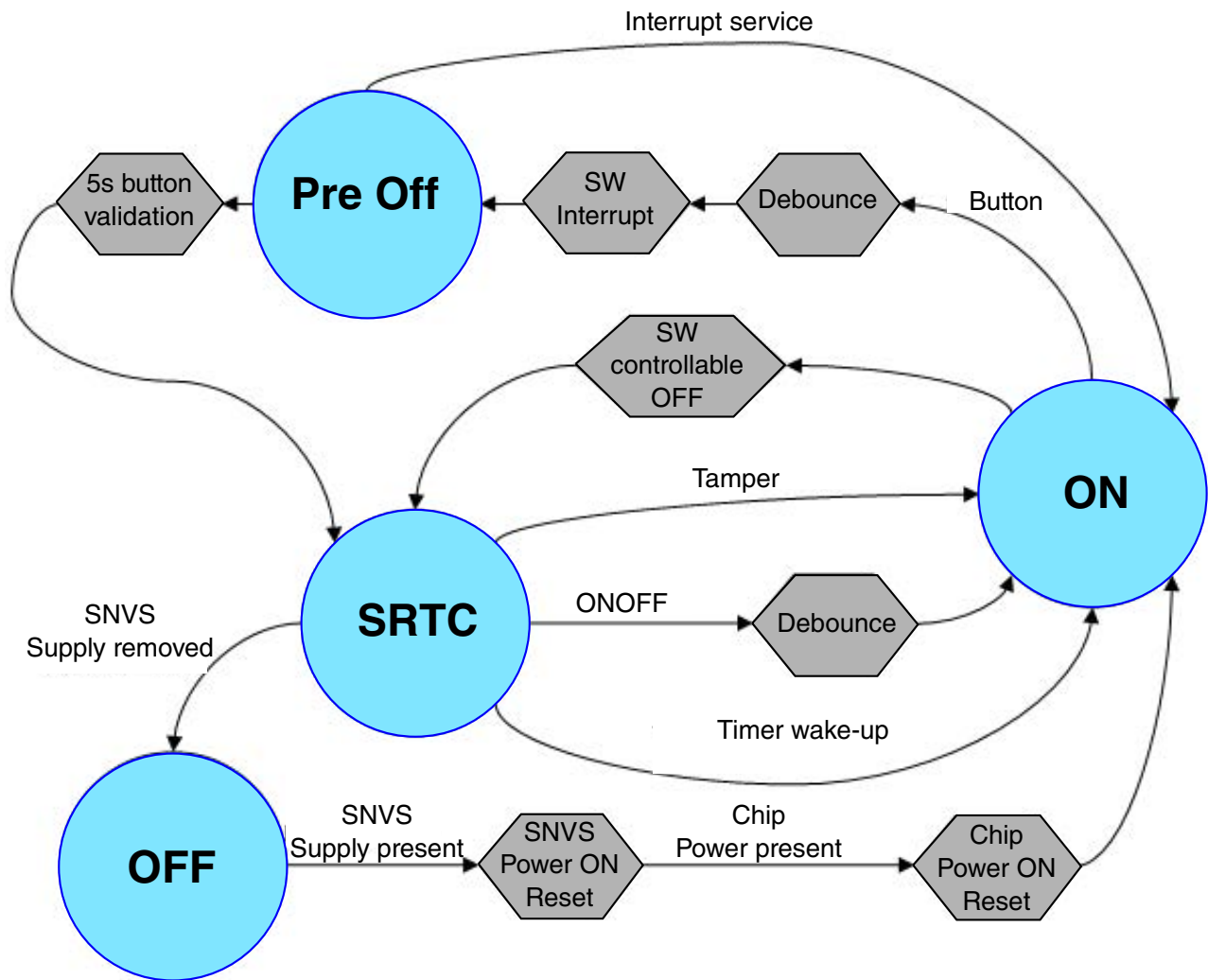


Figure 6-32. Chip on/off state flow diagram

6.5.3.2 Finite-State Machine (FSM)

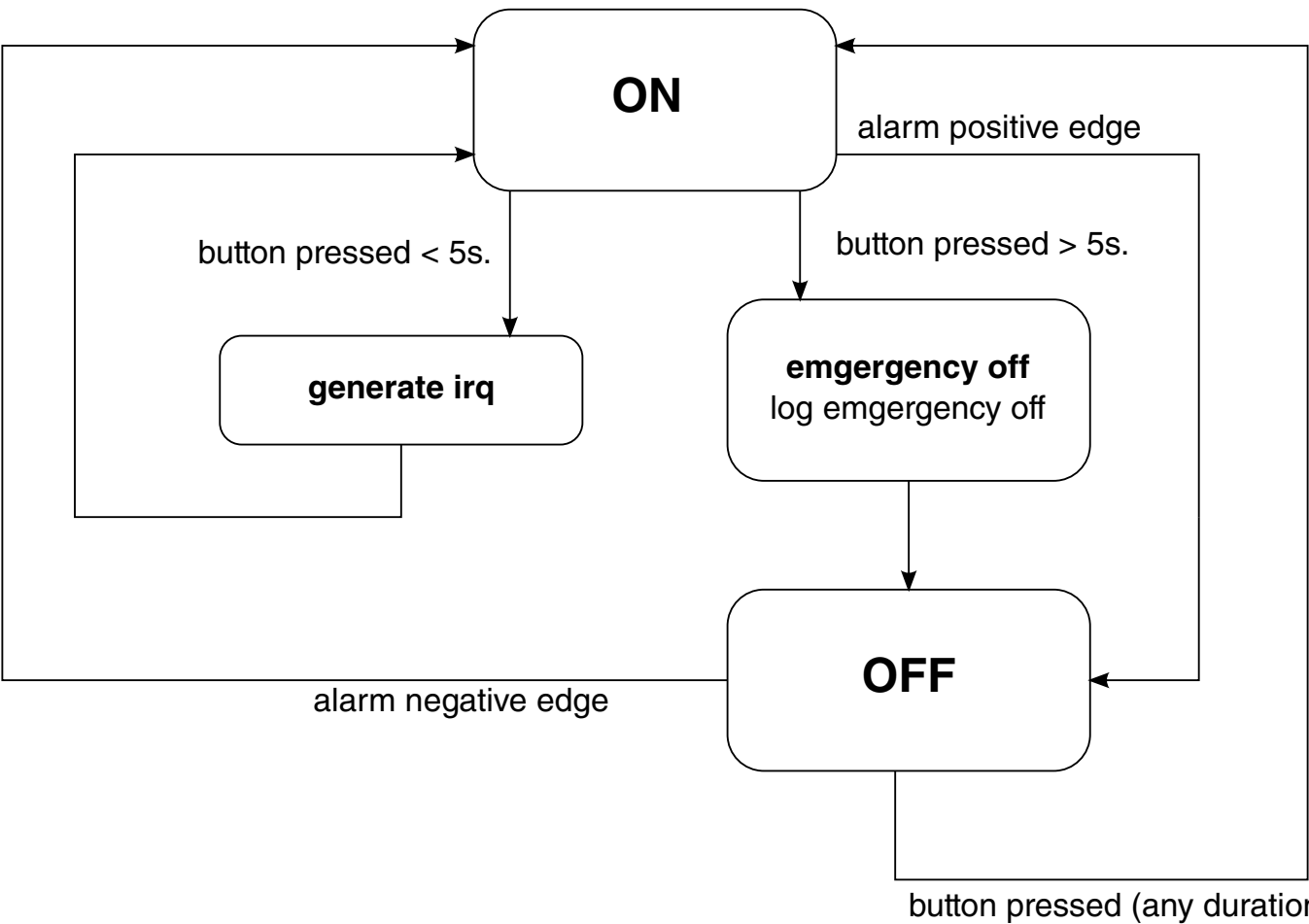


Figure 6-33. FSM

6.5.3.3 Power mode transitions

Table 6-47. Power mode transitions

Power mode	Configuration with external PMIC	Configuration with internal PMIC
ON, first time	1. SoC power supply is connected to SNVS. 2. When button is pressed, PMIC powers on.	1. SoC power supply is connected to SNVS. 2. When button is pressed, 'state' goes ON, PMIC_ON_REQ goes '1'. 3. External regulator is enabled.
Normal ON to OFF, by button	1. Button is pressed for a short duration on the external PMIC. 2. Interrupt request (irq) is sent to SoC from external PMIC. 3. SoC is programming PMIC for power off when standby is asserted.	1. SoC button is pressed for a short duration. 2. Interrupt request (irq) is sent to SoC from FSM. 3. Alarm timer is set up by software routine and started. 4. Upon alarm_in assertion to '1', PMIC_ON_REQ goes '0'. 5. External regulator goes OFF.

Table continues on the next page...

Table 6-47. Power mode transitions (continued)

Power mode	Configuration with external PMIC	Configuration with internal PMIC
	4. In CCM STOP mode, Standby is asserted, PMIC gates SoC supplies.	
Emergency ON to OFF, by button	1. Button is pressed for an extended time on the external PMIC. 2. PMIC is powering off.	1. Button is pressed for longer than 5 seconds on the SoC. 2. FSM validates button pressed for 5 seconds. 3. Emergency power off is logged, PMIC_ON_REQ goes '0', alarm_mask goes '1'. 4. External regulator goes OFF.
OFF to ON, by button	1. Button is pressed on the external PMIC. 2. PMIC powers ON.	1. Button is pressed on the SoC. 2. PMIC_ON_REQ goes '1', alarm_mask goes '0'. 3. External regulator powers ON.
OFF to ON, by timer alarm	1. Timer alarm in SNVS is programmed by software before SoC goes OFF. 2. SoC enters OFF mode. 3. Upon timer limit, wake up alarm goes '0'. PMIC_ON_REQ goes '1'. 4. PMIC receives assertion of PMIC_ON_REQ and wakes up.	1. Timer alarm in SNVS is programmed by software before SoC goes OFF. 2. SoC enters OFF mode. 3. Upon timer limit, wake up alarm goes '0'. PMIC_ON_REQ goes '1'. 4. External regulator is enabled by PMIC_ON_REQ = 1.

6.5.4 Power-On Reset and power sequencing

This module generates an internal POR_B signal that is logically AND'ed with any externally applied SRC_POR_B signal. The internal POR_B signal will be held low until all of the following conditions are met:

- 4ms after the external power supply VDDHIGH_IN is valid
- 1ms after the VDD_SOC_CAP supply is valid

The 4ms and 1ms delays are derived from counting the 32 kHz RTC clock cycles; the accuracy depends on the accuracy of the RTC. When the RTC crystal is either absent or in the process of powering up, an internal ring oscillator will be the source of RTC, which is not as accurate as the crystal.

6.5.4.1 External POR using SRC_POR_B

If the external SRC_POR_B signal is used to control the processor POR, SRC_POR_B must remain low (asserted) until the VDD_ARM_CAP and VDD_SOC_CAP supplies are stable.

6.5.4.2 Internal POR

If the external SRC_POR_B signal is not used (always held high or left unconnected), the processor defaults to the internal POR function (PMU controls generation of the POR based on the power supplies).

If the internal POR function is used, the following power supply requirements must be met:

- VDD_ARM_IN and VDD_SOC_IN may be supplied from the same source, or
- VDD_SOC_IN can be supplied before VDD_ARM_IN with a maximum delay of 1 ms.

6.5.5 Functional Description

6.5.5.1 Reset Control

This section details the reset control of this device.

6.5.5.1.1 Reset inputs and outputs

The reset control logic receives reset requests from all potential reset sources. All the immediate sources of reset are directly passed to the reset stretching block, whereas the resets requiring qualification are passed on to the reset qualification logic before they are sent to the reset stretching block.

All reset inputs and outputs are described in the following figure:

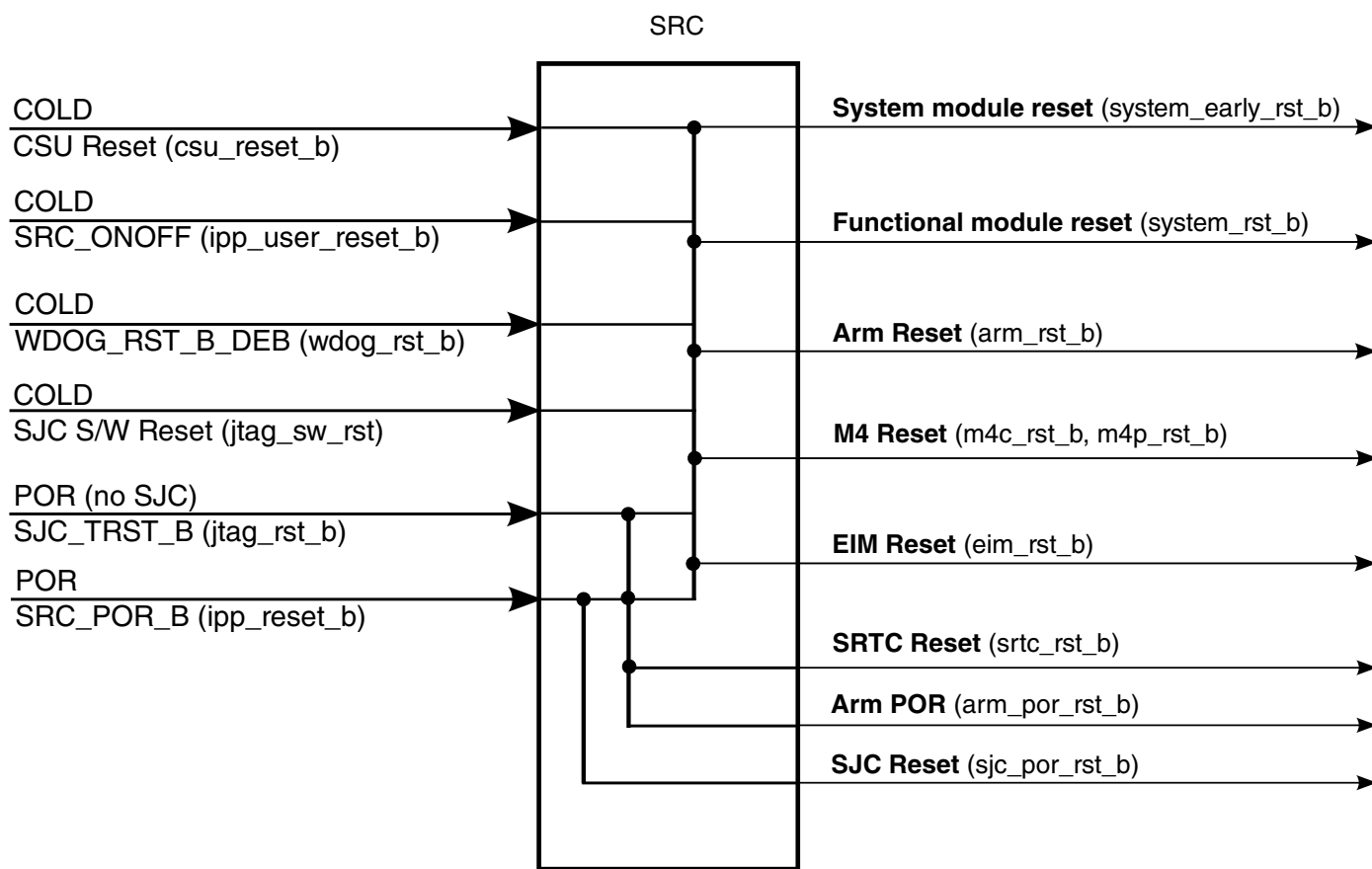


Figure 6-34. SRC inputs and outputs

The reset types and modules they affect are shown in [Table 6-48](#). As there is no chip POR, the POR_B is used to reset the entire chip including test logic and JTAG modules.

NOTE

All resets are expected to be active low except `jtag_sw_rst`.

Table 6-48. SRC reset functionality

SoC Modules	POR	COLD
System modules (PLLs, fuses, etc)	yes	yes
Functional modules	yes	yes
Arm	yes	yes
Arm SoC	yes	yes
M4 Core	yes	yes
M4 Platform	yes	yes
Arm POR	yes	no
Arm debug	yes	no
SJC	yes	no
SRTC	yes	no

The reset priorities are POR (strongest) and COLD (weakest). If a stronger reset is asserted during the sequence of a weaker reset, then the weaker sequence will be overridden, and the stronger reset sequence will commence. There is no priority within a reset type (POR, etc). If a reset is asserted during the reset sequence of the same type, the reset sequence will be interrupted and restarted.

The following lists the functionality of each of these reset outputs:

- `system_early_rst_b` - Resets the system modules that need to start first as CCM, OCOTP_CTRL, FUSEBOX, etc.
- `system_rst_b` - Resets functional modules
- `arm_rst_b` - Resets Arm module (on regular system reset)
- `arm_por_rst_b` - Resets Arm POR input
- `arm_soc_rst_b` - Reset for Arm SOC
- `m4c_rst_b` - Reset for M4 core
- `m4p_rst_b` - Reset for M4 platform
- `arm_dbg_rst_b` - Reset debug logic of Arm
- `test_logic_rst_b` - Reset test logic (IOMUXC, DAP)
- `sjc_por_rst_b` - Reset to SJC
- `srtc_rst_b` - Resets SRTC

NOTE

It is assumed that each reset source will deassert after its assertion, either due to reset generated to the system from SRC, or by negation of the reset source (if it came from an external source to the chip). In the latter case, the reset source is assumed to be held for at least 2 XTALI clocks so it can be sampled by SRC.

6.5.5.1.2 Reset Handling

6.5.5.1.2.1 POR (SRC_POR_B)

`SRC_POR_B` is an external reset signal. When the chip is powered up, the reset signal is passed through the `POR_B` pin indicating power-up sequence. The SRC resets the entire chip including the JTAG (SJC) module. All SRC registers will be reset during the POR sequence.

As soon as `SRC_POR_B` occurs, all resets are asserted and the entire chip is reset by SRC. The `SRC_POR_B` is stretched for 2 XTALI cycles and the stretching sequence takes place after 2 XTALI clocks of `POR_B` pin deassertion.

The `srtc_rst_b` signal is deasserted together with `SRC_POR_B` signal. The output are also deasserted after the stretching of `SRC_POR_B` has deasserted.

The `sjc_por_rst_b` signal is deasserted together with `SRC_POR_B` signal. The output is also deasserted after the stretching of `SRC_POR_B` has deasserted.

After the above resets deassert, `system_early_rst_b` reset is deasserted after 2 XTALI clocks. The `system_early_rst_b` is used for the CCM and PLL-IPs to start generating PLL clock outputs and the system root clocks.

When the system root clocks are ready, the CCM will assert `system_clk_ready` signal. This signal is generated during the start sequence in the CCM and it involves the preparation of the PLLs to generate clock roots for functional operation.

SRC then enables `OCOTP_CTRL` and fusebox clocks, so that fuses can be loaded to `OCOTP_CTRL`.

- SRC will prepare the boot information
- After 8 ipg cycles, resets to all modules will be de-asserted
- After 8 ipg cycles, system clocks will be enabled (`en_system_clk`).

6.5.5.1.2.2 COLD RESET

The sequence is similar to `SRC_POR_B` except the memory repair operation is not performed.

After the reset source deasserts, `system_early_rst_b` reset is deasserted after at least 2 XTALI clocks. The `system_early_rst_b` is used for the CCM and PLL-IPs to start generating PLL clock outputs and the system root clocks.

After the system root clocks are ready, the CCM will assert `system_clk_ready` signal. This signal is generated during the start sequence in the CCM and it involves the preparation of the PLLs to generate clock roots for functional operation. See CCM for more information.

After `system_clk_ready` arrives at the SRC, it will enable `OCOTP_CTRL` and fusebox clocks, so that fuses can be loaded to `OCOTP_CTRL`. `OCOTP_CTRL` will notify with `iim_ready_flag` when the fusebox loading finishes.

- SRC will prepare the boot information
- After 8 ipg cycles resets to all modules will be deasserted
- After 8 ipg cycles, system clocks will be enabled (`en_system_clk`).

6.5.5.2 Parallel Reset Requests

SRC will follow the following rules in the case of parallel reset requests:

1. The order of strength of resets is POR - strongest, COLD - weakest
2. If a stronger reset is asserted during weaker reset sequence, then the stronger reset will take over and the stronger reset process will commence. The following cases fall into this category:
 - POR reset request in the middle of cold reset process - the cold will be stopped and the POR sequence will start.
3. If a weaker reset is asserted during stronger reset sequence, then the stronger reset sequence will continue without interference. If at the end of the stronger reset process the weaker request is still asserted then the weaker sequence will commence. The following cases fall into this category:
 - COLD reset requests in the middle of POR reset process - the POR process will continue without interference.
4. If a similar reset request is asserted during the process of reset handling, then the process of reset handling will start over (with the same process). The following cases fall into this category:
 - POR reset request in the middle of POR reset process - the POR process will start over.
 - COLD reset request in the middle of COLD reset process - the COLD process will start over.

6.5.5.3 Boot Mode Control

6.5.5.3.1 BOOT_MODE Pin Latching

The exact boot sequence is controlled by the values of the BOOT_MODE pins on this device.

The value of the BOOT_MODE pins will be latched after the OCOTP_CTRL asserts the fuse read completion flag. After latching, the values of the BOOT_MODE pins are used to determine the booting options of the core as described in the SRC_SBMRx registers.

The boot mode general purpose bits can be provided to the SRC from either e-fuses or GPIO signals. The gpio_bt_sel e-fuse defines the source to be used to derive the boot information. When gpio_bt_sel is set, e-fuses are used. When cleared, GPIO signals are used.

The boot information is provided in SRC_SBMR1 register. The figure below shows the selection of boot mode information.

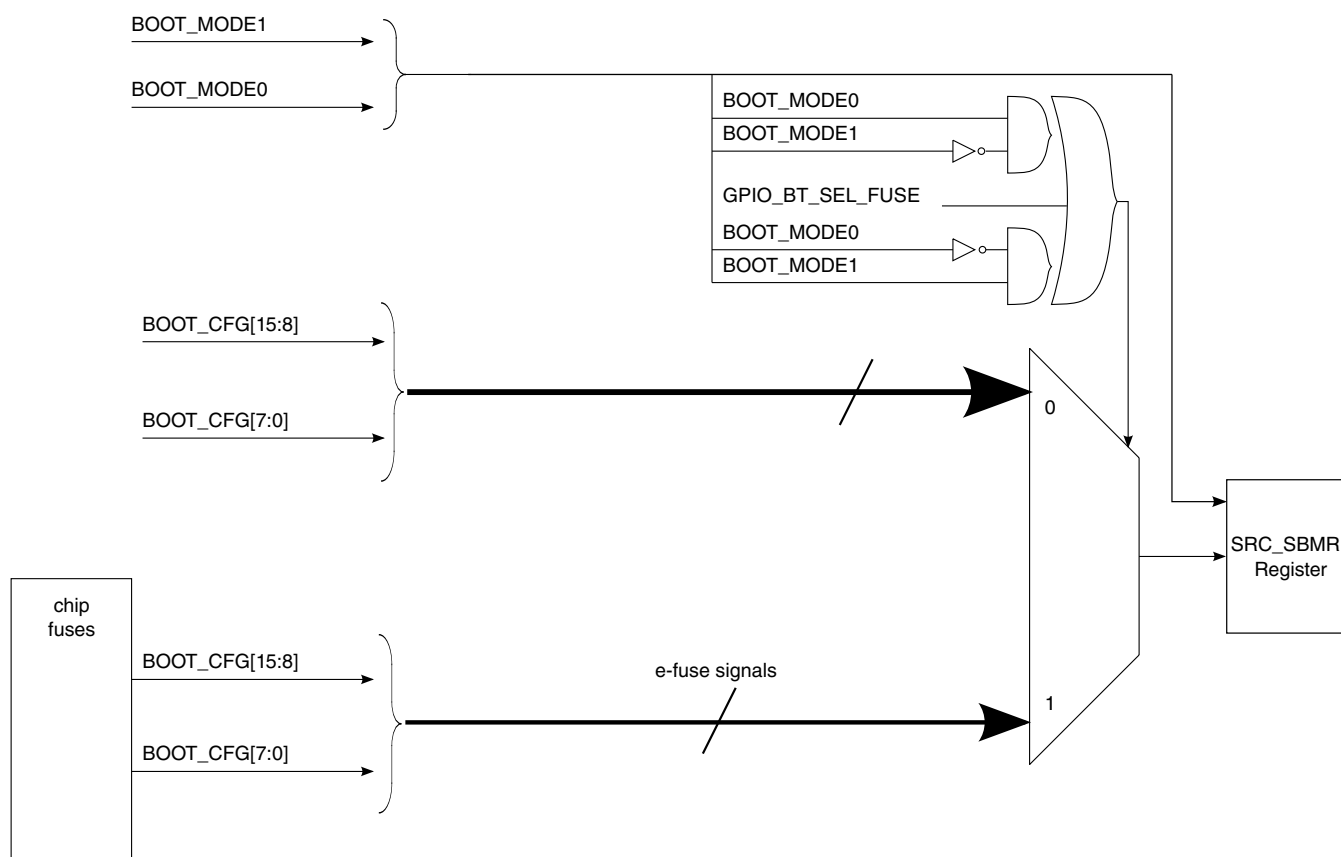


Figure 6-35. Boot mode information

6.5.6 SRC Memory Map/Register Definition

SRC memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3039_0000	SRC Reset Control Register (SRC_SCR)	32	R/W	0000_00A0h	6.5.6.1/987
3039_0004	A53 Reset Control Register (SRC_A53RCR0)	32	R/W	000A_0000h	6.5.6.2/989
3039_0008	A53 Reset Control Register (SRC_A53RCR1)	32	R/W	0000_0001h	6.5.6.3/994
3039_000C	M4 Reset Control Register (SRC_M4RCR)	32	R/W	0000_00A8h	6.5.6.4/996
3039_0020	USB OTG PHY1 Reset Control Register (SRC_USBOPHY1_RCR)	32	R/W	0000_0001h	6.5.6.5/999
3039_0024	USB OTG PHY2 Reset Control Register (SRC_USBOPHY2_RCR)	32	R/W	0000_0001h	6.5.6.6/1000

Table continues on the next page...

SRC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3039_0028	MIPI PHY Reset Control Register (SRC_MIPIPHY_RCR)	32	R/W	0000_0000h	6.5.6.7/1002
3039_002C	PCIE PHY Reset Control Register (SRC_PCIEPHY_RCR)	32	R/W	0000_000Ah	6.5.6.8/1004
3039_0030	HDMI Reset Control Register (SRC_HDMI_RCR)	32	R/W	0000_0000h	6.5.6.9/1007
3039_0034	DISPLAY Reset Control Register (SRC_DISP_RCR)	32	R/W	0000_0000h	6.5.6.10/1008
3039_0040	GPU Reset Control Register (SRC_GPU_RCR)	32	R/W	0000_0000h	6.5.6.11/1010
3039_0044	VPU Reset Control Register (SRC_VPU_RCR)	32	R/W	0000_0000h	6.5.6.12/1011
3039_0048	PCIE2 Reset Control Register (SRC_PCIE2_RCR)	32	R/W	0000_000Ah	6.5.6.13/1013
3039_004C	MIPI CSI1 PHY Reset Control Register (SRC_MIPIPHY1_RCR)	32	R/W	0000_0000h	6.5.6.14/1016
3039_0050	MIPI CSI2 PHY Reset Control Register (SRC_MIPIPHY2_RCR)	32	R/W	0000_0000h	6.5.6.15/1018
3039_0058	SRC Boot Mode Register 1 (SRC_SBMR1)	32	R	0000_0000h	6.5.6.16/1019
3039_005C	SRC Reset Status Register (SRC_SRSR)	32	R/W	0000_0001h	6.5.6.17/1020
3039_0068	SRC Interrupt Status Register (SRC_SISR)	32	R/W	0000_0000h	6.5.6.18/1023
3039_006C	SRC Interrupt Mask Register (SRC_SIMR)	32	R/W	0000_03FFh	6.5.6.19/1026
3039_0070	SRC Boot Mode Register 2 (SRC_SBMR2)	32	R	0000_0000h	6.5.6.20/1030
3039_0074	SRC General Purpose Register 1 (SRC_GPR1)	32	R/W	0000_0000h	6.5.6.21/1032
3039_0078	SRC General Purpose Register 2 (SRC_GPR2)	32	R/W	0000_0000h	6.5.6.22/1032
3039_007C	SRC General Purpose Register 3 (SRC_GPR3)	32	R/W	0000_0000h	6.5.6.23/1032
3039_0080	SRC General Purpose Register 4 (SRC_GPR4)	32	R/W	0000_0000h	6.5.6.24/1033
3039_0084	SRC General Purpose Register 5 (SRC_GPR5)	32	R/W	0000_0000h	6.5.6.25/1033
3039_0088	SRC General Purpose Register 6 (SRC_GPR6)	32	R/W	0000_0000h	6.5.6.26/1033
3039_008C	SRC General Purpose Register 7 (SRC_GPR7)	32	R/W	0000_0000h	6.5.6.27/1034
3039_0090	SRC General Purpose Register 8 (SRC_GPR8)	32	R/W	0000_0000h	6.5.6.28/1034

Table continues on the next page...

SRC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3039_0094	SRC General Purpose Register 9 (SRC_GPR9)	32	R/W	0000_0000h	6.5.6.29/1034
3039_0098	SRC General Purpose Register 10 (SRC_GPR10)	32	R/W	0000_0000h	6.5.6.30/1035
3039_1000	SRC DDR Controller Reset Control Register (SRC_DDRC_RCR)	32	R/W	0000_000Fh	6.5.6.31/1036
3039_1004	SRC DDRC2 Controller Reset Control Register (SRC_DDRC2_RCR)	32	R/W	0000_000Fh	6.5.6.32/1038

6.5.6.1 SRC Reset Control Register (SRC_SCR)

The reset control register (SCR), contains bits that control operation of the reset controller.

Address: 3039_0000h base + 0h offset = 3039_0000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DOM_EN	LOCK	Reserved				DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved					
W	DOM_EN	LOCK	Reserved				DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved					
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								MASK_TEMPSENSE_RESET				Reserved			
W	Reserved								MASK_TEMPSENSE_RESET				Reserved			
Reset	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0

SRC_SCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock

Table continues on the next page...

SRC_SCR field descriptions (continued)

Field	Description
	NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–8 -	This field is reserved. Reserved
7–4 MASK_ TEMPSENSE_ RESET	Mask tempsense_reset source. If these 4 bits are coded from A to 5 then, the tempsense_reset input to SRC will be masked and the tempsense_reset will not create a reset to the chip. 0101 tempsense_reset is masked 1010 tempsense_reset is not masked
-	This field is reserved. Reserved

6.5.6.2 A53 Reset Control Register (SRC_A53RCR0)

The A53 Reset Control Register (A53RCR), contains bits that control the A53 reset generation.

Address: 3039_0000h base + 4h offset = 3039_0004h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DOM_EN	LOCK	Reserved				DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved		A53_L2RESET	A53_SOC_DBG_RESET	MASK_WDOG1_RST	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	A53_ETM_RESET3	A53_ETM_RESET2	A53_ETM_RESET1	A53_ETM_RESET0	A53_DBG_RESET3	A53_DBG_RESET2	A53_DBG_RESET1	A53_DBG_RESET0	A53_CORE_RESET3	A53_CORE_RESET2	A53_CORE_RESET1	A53_CORE_RESET0	A53_CORE_POR_RESET3	A53_CORE_POR_RESET2	A53_CORE_POR_RESET1	A53_CORE_POR_RESET0
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_A53RCR0 field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1.

Table continues on the next page...

SRC_A53RCR0 field descriptions (continued)

Field	Description
	0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–22 -	This field is reserved. Reserved
21 A53_L2RESET	Software reset for A53 Snoop Control Unit (SCU). NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert SCU reset 1 assert SCU reset
20 A53_SOC_DBG_RESET	Software reset for system level debug reset. It initializes the shared Debug APB, the CTI, and the CTM. It also causes: • A53_dbgreset[3:0] and A53_etmreset[3:0] to be asserted • debug logic in the processor power domain and in the debug power domain to be reset NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert system level debug reset 1 assert system level debug reset
19–16 MASK_WDOG1_RST	Mask wdog1_rst_b source. If these 4 bits are coded from A to 5 then, the wdog1_rst_b input to SRC will be masked and the wdog1_rst_b will not create a reset to the chip. NOTE: During the time the WDOG event is masked using SRC logic, it is likely that the WDOG Reset Status Register (WRSR) bit 1 (which indicates a WDOG timeout event) will get asserted. software / OS developer must prepare for this case. Re-enabling the WDOG is possible, by unmasking it in SRC, though it must be preceded by servicing the WDOG. However, for the case that the event has been asserted, the status bit (WRSR bit-1) will remain asserted, regardless of servicing the WDOG module. (Hardware reset is the only way to cause the de-assertion of that bit). Any other code will be coded to 1010 i.e. wdog1_rst_b is not masked 0101 wdog1_rst_b is masked 1010 wdog1_rst_b is not masked

Table continues on the next page...

SRC_A53RCR0 field descriptions (continued)

Field	Description
15 A53_ETM_ RESET3	Software reset for core3 ETM only. NOTE: This is a self clearing bit. Once it is set to 1, the rest process will begin, and once it finished, this bit will be self-cleared. 0 do not assert core3 ETM reset 1 assert core3 ETM reset
14 A53_ETM_ RESET2	Software reset for core2 ETM only. NOTE: This is a self clearing bit. Once it is set to 1, the rest process will begin, and once it finished, this bit will be self-cleared. 0 do not assert core2 ETM reset 1 assert core2 ETM reset
13 A53_ETM_ RESET1	Software reset for core1 ETM only. NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core1 ETM reset 1 assert core1 ETM reset
12 A53_ETM_ RESET0	Software reset for core0 ETM only. NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core0 ETM reset 1 assert core0 ETM reset
11 A53_DBG_ RESET3	Software reset for core3 debug only. It initialize the debug, and breakpoint and watchpoint logic in the core3 processor power domain. It also reset the debug logic for core1 processor, which is in the debug power domain. NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core3 debug reset 1 assert core3 debug reset
10 A53_DBG_ RESET2	Software reset for core2 debug only. It initialize the debug, and breakpoint and watchpoint logic in the core2 processor power domain. It also reset the debug logic for core1 processor, which is in the debug power domain. NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core2 debug reset 1 assert core2 debug reset
9 A53_DBG_ RESET1	Software reset for core1 debug only. It initialize the debug, and breakpoint and watchpoint logic in the core1 processor power domain. It also reset the debug logic for core1 processor, which is in the debug power domain.

Table continues on the next page...

SRC_A53RCR0 field descriptions (continued)

Field	Description
	NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core1 debug reset 1 assert core1 debug reset
8 A53_DBG_RESET0	Software reset for core0 debug only. It initialize the debug, and breakpoint and watchpoint logic in the core1 processor power domain. It also reset the debug logic for core1 processor, which is in the debug power domain. NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core0 debug reset 1 assert core0 debug reset
7 A53_CORE_RESET3	Software reset for core3 only. It initializes the processor logic in the core1 processor power domains, not including the debug, breakpoint and watchpoint logic. NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core3 reset 1 assert core3 reset
6 A53_CORE_RESET2	Software reset for core2 only. It initializes the processor logic in the core1 processor power domains, not including the debug, breakpoint and watchpoint logic. NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core2 reset 1 assert core2 reset
5 A53_CORE_RESET1	Software reset for core1 only. It initializes the processor logic in the core1 processor power domains, not including the debug, breakpoint and watchpoint logic. NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core1 reset 1 assert core1 reset
4 A53_CORE_RESET0	Software reset for core0 only. It initializes the processor logic in the core0 processor power domains, not including the debug, breakpoint and watchpoint logic. NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core0 reset 1 assert core0 reset
3 A53_CORE_POR_RESET3	POR reset for A53 core3 only. It initializes all the core1 processor logic, including CPU Debug, and breakpoint and watchpoint logic in the core3 processor power domains

Table continues on the next page...

SRC_A53RCR0 field descriptions (continued)

Field	Description
	NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core3 reset 1 assert core3 reset
2 A53_CORE_ POR_RESET2	POR reset for A53 core2 only. It initializes all the core1 processor logic, including CPU Debug, and breakpoint and watchpoint logic in the core2 processor power domains NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core2 reset 1 assert core2 reset
1 A53_CORE_ POR_RESET1	POR reset for A53 core1 only. It initializes all the core1 processor logic, including CPU Debug, and breakpoint and watchpoint logic in the core1 processor power domains NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core1 reset 1 assert core1 reset
0 A53_CORE_ POR_RESET0	POR reset for A53 core0 only. It initializes all the core0 processor logic, including CPU Debug, and breakpoint and watchpoint logic in the core0 processor power domains NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. 0 do not assert core0 reset 1 assert core0 reset

6.5.6.3 A53 Reset Control Register (SRC_A53RCR1)

The A53 Reset Control Register (A53RCR), contains bits that control the A53 reset generation.

Address: 3039_0000h base + 8h offset = 3039_0008h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R			Reserved		DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								A53_RST_SLOW				A53_CORE3_ENABLE	A53_CORE2_ENABLE	A53_CORE1_ENABLE	A53_CORE0_ENABLE
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

SRC_A53RCR1 field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–7 -	This field is reserved. Reserved
6–4 A53_RST_SLOW	A53_RST_SLOW
3 A53_CORE3_ENABLE	core 3 enable 0 core3 is disabled 1 core3 is enabled
2 A53_CORE2_ENABLE	core 2 enable 0 core2 is disabled 1 core2 is enabled
1 A53_CORE1_ENABLE	core 1 enable

Table continues on the next page...

SRC_A53RCR1 field descriptions (continued)

Field	Description
	0 core1 is disabled 1 core1 is enabled
0 A53_CORE0_ENABLE	Always 1, can't be changed.

6.5.6.4 M4 Reset Control Register (SRC_M4RCR)

The M4 Reset Control Register (M4RCR), contains bits that control the M4 reset generation.

Address: 3039_0000h base + Ch offset = 3039_000Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DOM_EN	LOCK	Reserved			DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved						
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved							WDOG3_RST_OPTION	WDOG3_RST_OPTION_M4	MASK_WDOG3_RST				ENABLE_M4	SW_M4P_RST	SW_M4C_RST
W																
Reset	0	0	0	0	0	0	0	0	1	0	1	0	1	0	0	0

SRC_M4RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock

Table continues on the next page...

SRC_M4RCR field descriptions (continued)

Field	Description
	NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–10 -	This field is reserved. Reserved
9 WDOG3_RST_OPTION	Wdog3_rst_b option 0 Wdog3_rst_b asserts M4 reset 1 Wdog3_rst_b asserts global reset
8 WDOG3_RST_OPTION_M4	Wdog3_rst_b option for M4. This bit is only effective when wdog3_rst_option is set to 1. 0 wdog3_rst_b Reset M4 core only 1 Reset both M4 core and platform
7–4 MASK_WDOG3_RST	Mask wdog3_rst_b source. If these 4 bits are coded from A to 5 then, the wdog3_rst_b input to SRC will be masked and the wdog3_rst_b will not create a reset to the chip. NOTE: During the time the WDOG3 event is masked using SRC logic, it is likely that the WDOG3 Reset Status Register (WRSR) bit 1 (which indicates a WDOG3 timeout event) will get asserted. Software / OS developer must prepare for this case. Re-enabling the WDOG3 is possible, by unmasking it in SRC, though it must be preceded by servicing the WDOG3. However, for the case that the event has been asserted, the status bit (WRSR bit-1) will remain asserted, regardless of servicing the WDOG3 module. (Hardware reset is the only way to cause the de-assertion of that bit). Any other code will be coded to 1010 i.e. wdog3_rst_b is not masked 0101 wdog3_rst_b is masked 1010 wdog3_rst_b is not masked
3 ENABLE_M4	Enable M4 0 M4 is disabled 1 M4 is enabled

Table continues on the next page...

SRC_M4RCR field descriptions (continued)

Field	Description
2 SW_M4P_RST	Self-clearing SW reset for M4 platform NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. Software can determine that the reset has finished once this bit is cleared. Software can also configure SRC to generate interrupt once the software has finished. Please refer to SRC_SISR register for details. 0 do not assert M4 platform reset 1 assert M4 platform reset
1 SW_M4C_RST	Self-clearing SW reset for M4 core NOTE: This is a self clearing bit. Once it is set to 1, the reset process will begin, and once it finishes, this bit will be self cleared. Software can determine that the reset has finished once this bit is cleared. Software can also configure SRC to generate interrupt once the software has finished. Please refer to SRC_SISR register for details. 0 do not assert M4 core reset 1 assert M4 core reset
0 SW_M4C_NON_SCLR_RST	Non-self-clearing SW reset for M4 core 0 do not assert M4 core reset 1 assert M4 core reset

6.5.6.5 USB OTG PHY1 Reset Control Register (SRC_USBOPHY1_RCR)

The USB OTG PHY1 Reset Control Register (SRC_IP_RCR2), contains bits that control the USB OTG PHY1 reset generation.

Address: 3039_0000h base + 20h offset = 3039_0020h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DOM_EN	LOCK	Reserved				DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															OTG1_PHY_RESET
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

SRC_USBOPHY1_RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1.

Table continues on the next page...

SRC_USBOPHY1_RCR field descriptions (continued)

Field	Description
	0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–1 -	This field is reserved. Reserved
0 OTG1_PHY_ RESET	0 Don't reset USB OTG1 PHY 1 Reset USB OTG1 PHY

6.5.6.6 USB OTG PHY2 Reset Control Register (SRC_USBOPHY2_RCR)

The USB OTG PHY2 Reset Control Register (SRC_IP_RCR2), contains bits that control the USB OTG PHY2 reset generation.

Address: 3039_0000h base + 24h offset = 3039_0024h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

SRC_USBOPHY2_RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–1 -	This field is reserved. Reserved
0 OTG2_PHY_RESET	0 Don't reset USB OTG2 PHY 1 Reset USB OTG2 PHY

6.5.6.7 MIPI PHY Reset Control Register (SRC_MIIPHY_RCR)

The MIPI PHY Reset Control Register (SRC_MIIPHY_RCR), contains bits that control the MIPI PHYreset generation.

Address: 3039_0000h base + 28h offset = 3039_0028h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DOM_EN	LOCK	Reserved			DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved						
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved										MIPI_DSI_PCLK_RESET_N	MIPI_DSI_ESC_RESET_N	MIPI_DSI_DPI_RESET_N	MIPI_DSI_RESET_N	MIPI_DSI_RESET_BYTE_N	Reserved
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_MIIPHY_RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified

Table continues on the next page...

SRC_MIPIPHY_RCR field descriptions (continued)

Field	Description
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–6 -	This field is reserved. Reserved
5 MIPI_DSI_ PCLK_RESET_N	0 Reset 1 Don't reset
4 MIPI_DSI_ESC_ RESET_N	0 Reset 1 Don't reset
3 MIPI_DSI_DPI_ RESET_N	0 Reset 1 Don't reset
2 MIPI_DSI_ RESET_N	0 Reset 1 Don't reset
1 MIPI_DSI_ RESET_BYTE_N	0 Reset 1 Don't reset
0 -	This field is reserved. Reserved

6.5.6.8 PCIE PHY Reset Control Register (SRC_PCIEPHY_RCR)

The PCIE PHY Control Register (SRC_PCIEPHY_RCR), contains bits that control the PCIE PHY reset generation.

Address: 3039_0000h base + 2Ch offset = 3039_002Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DOM_EN	LOCK	Reserved				DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved					PCIE_CTRL_APP_XFER_PENDING
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PCIE_CTRL_APP_UNLOCK_MSG	PCIE_CTRL_SYS_INT	Reserved	PCIE_CTRL_CFG_L1_AUX	PCIE_CTRL_APPS_TURNOFF	PCIE_CTRL_APPS_PME	PCIE_CTRL_APPS_EXIT	PCIE_CTRL_APPS_ENTER	PCIE_CTRL_APPS_READY	PCIE_CTRL_APPS_EN	PCIE_CTRL_APPS_RST	PCIE_CTRL_APPS_CLK_REQ	PCIEPHY_PERST	PCIEPHY_BTN	PCIEPHY_G_RST	PCIE_PHY_POWER_ON_RESET_N
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0

SRC_PCIEPHY_RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified

Table continues on the next page...

SRC_PCIEPHY_RCR field descriptions (continued)

Field	Description
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–17 -	This field is reserved. Reserved
16 PCIE_CTRL_APP_XFER_PENDING	PCIE_CTRL_APP_XFER_PENDING
15 PCIE_CTRL_APP_UNLOCK_MSG	PCIE_CTRL_APP_UNLOCK_MSG
14 PCIE_CTRL_SYS_INT	PCIE_CTRL_SYS_INT
13 -	This field is reserved.
12 PCIE_CTRL_CFG_L1_AUX	Pcie_ctrl_cfg_l1_aux_clk_switch_core_clk_gate_en
11 PCIE_CTRL_APPS_TURNOFF	Pcie_ctrl_apps_pm_xmt_turnoff
10 PCIE_CTRL_APPS_PME	Pcie_ctrl_apps_pm_xmt_pme
9 PCIE_CTRL_APPS_EXIT	Pcie_ctrl_app_req_exit_l1
8 PCIE_CTRL_APPS_ENTER	Pcie_ctrl_app_req_entr_l1

Table continues on the next page...

SRC_PCIEPHY_RCR field descriptions (continued)

Field	Description
7 PCIE_CTRL_APPS_READY	Pcie_ctrl_app_ready_entr_l23
6 PCIE_CTRL_APPS_EN	Pcie_ctrl_app_ltssm_enable
5 PCIE_CTRL_APPS_RST	Pcie_ctrl_app_init_rst
4 PCIE_CTRL_APPS_CLK_REQ	Pcie_ctrl_app_clk_req_n
3 PCIEPHY_PERST	Pciephy_perst
2 PCIEPHY_BTN	PCIE PHY button
1 PCIEPHY_G_RST	PCIE PHY Global Reset
0 PCIE_PHY_POWER_ON_RESET_N	PCIE_PHY_POWER_ON_RESET_N

6.5.6.9 HDMI Reset Control Register (SRC_HDMI_RCR)

The HDMI Control Register contains bits that control the HDMI reset generation.

Address: 3039_0000h base + 30h offset = 3039_0030h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DOM_EN	LOCK	Reserved				DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															HDMI_PHY_APB_RESET
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_HDMI_RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register

Table continues on the next page...

SRC_HDMI_RCR field descriptions (continued)

Field	Description
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–1 -	This field is reserved. Reserved
0 HDMI_PHY_ APB_RESET	Active 1

6.5.6.10 DISPLAY Reset Control Register (SRC_DISP_RCR)

The DISPLAY Control Register contains bits that control the DISPLAY reset generation.

Address: 3039_0000h base + 34h offset = 3039_0034h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DOM_EN	LOCK	Reserved				DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DISP_RESET
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_DISP_RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set.

Table continues on the next page...

SRC_DISP_RCR field descriptions (continued)

Field	Description
	0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–1 -	This field is reserved. Reserved
0 DISP_RESET	0 Don't reset dispmix 1 Reset dispmix

6.5.6.11 GPU Reset Control Register (SRC_GPU_RCR)

The GPU Control Register contains bits that control the GPU reset generation.

Address: 3039_0000h base + 40h offset = 3039_0040h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DOM_EN	LOCK	Reserved		DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															GPU_RESET
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_GPU_RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1.

Table continues on the next page...

SRC_GPU_RCR field descriptions (continued)

Field	Description
	0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–1 -	This field is reserved. Reserved
0 GPU_RESET	GPU_RESET

6.5.6.12 VPU Reset Control Register (SRC_VPU_RCR)

The VPU Control Register contains bits that control the VPU reset generation.

Address: 3039_0000h base + 44h offset = 3039_0044h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DOM_EN	LOCK	Reserved			DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved						
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															VPU_RESET
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_VPU_RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set.

Table continues on the next page...

SRC_VPU_RCR field descriptions (continued)

Field	Description
	0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–1 -	This field is reserved. Reserved
0 VPU_RESET	VPU_RESET

6.5.6.13 PCIE2 Reset Control Register (SRC_PCIE2_RCR)

The PCIE2 Control Register contains bits that control the PCIE2 reset generation.

Address: 3039_0000h base + 48h offset = 3039_0048h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DOM_EN	LOCK	Reserved				DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved					PCIE_CTRL_APP_XFER_PENDING
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PCIE_CTRL_APP_UNLOCK_MSG	PCIE_CTRL_SYS_INT	Reserved	PCIE_CTRL_CFG_L1_AUX	PCIE_CTRL_APPS_TURNOFF	PCIE_CTRL_APPS_PME	PCIE_CTRL_APPS_EXIT	PCIE_CTRL_APPS_ENTER	PCIE_CTRL_APPS_READY	PCIE_CTRL_APPS_EN	PCIE_CTRL_APPS_RST	PCIE_CTRL_APPS_CLK_REQ	PCIE_PERST	PCIE_BTN	PCIE_G_RST	PCIE_PHY_POWER_ON_RESET_N
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0

SRC_PCIE2_RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved

Table continues on the next page...

SRC_PCIE2_RCR field descriptions (continued)

Field	Description
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–17 -	This field is reserved. Reserved
16 PCIE_CTRL_APP_XFER_PENDING	PCIE_CTRL_APP_XFER_PENDING
15 PCIE_CTRL_APP_UNLOCK_MSG	PCIE_CTRL_APP_UNLOCK_MSG
14 PCIE_CTRL_SYS_INT	PCIE_CTRL_SYS_INT
13 -	This field is reserved.
12 PCIE_CTRL_CFG_L1_AUX	Pcie_ctrl_cfg_l1_aux_clk_switch_core_clk_gate_en
11 PCIE_CTRL_APPS_TURNOFF	Pcie_ctrl_apps_pm_xmt_turnoff
10 PCIE_CTRL_APPS_PME	Pcie_ctrl_apps_pm_xmt_pme
9 PCIE_CTRL_APPS_EXIT	Pcie_ctrl_app_req_exit_l1
8 PCIE_CTRL_APPS_ENTER	Pcie_ctrl_app_req_entr_l1

Table continues on the next page...

SRC_PCIE2_RCR field descriptions (continued)

Field	Description
7 PCIE_CTRL_APPS_READY	Pcie_ctrl_app_ready_entr_l23
6 PCIE_CTRL_APPS_EN	Pcie_ctrl_app_ltssm_enable
5 PCIE_CTRL_APPS_RST	Pcie_ctrl_app_init_rst
4 PCIE_CTRL_APPS_CLK_REQ	Pcie_ctrl_app_clk_req_n
3 PCIE_PERST	Pcie_perst
2 PCIE_BTN	PCIE2 button
1 PCIE_G_RST	PCIE Global Reset
0 PCIE_PHY_POWER_ON_RESET_N	PCIE_PHY_POWER_ON_RESET_N

6.5.6.14 MIPI CSI1 PHY Reset Control Register (SRC_MIPIPHY1_RCR)

The MIPI CSI1 PHY Control Register contains bits that control the MIPI CSI1 PHY reset generation.

Address: 3039_0000h base + 4Ch offset = 3039_004Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W	DOM_EN	LOCK	Reserved		DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved							
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															
W	Reserved												MIPI_CSI1_ESC_RESET		MIPI_CSI1_PHY_REF_RESET	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_MIPIPHY1_RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved

Table continues on the next page...

SRC_MIPIPHY1_RCR field descriptions (continued)

Field	Description
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–3 -	This field is reserved. Reserved
2 MIPI_CSI1_ ESC_RESET	MIPI_CSI1_ESC_RESET
1 MIPI_CSI1_ PHY_REF_ RESET	MIPI_CSI1_PHY_REF_RESET
0 MIPI_CSI1_ CORE_RESET	MIPI_CSI1_CORE_RESET

6.5.6.15 MIPI CSI2 PHY Reset Control Register (SRC_MIPIPHY2_RCR)

The MIPI CSI2 PHY Control Register contains bits that control the MIPI CSI2 PHY reset generation.

Address: 3039_0000h base + 50h offset = 3039_0050h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W	DOM_EN	LOCK	Reserved		DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved							
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															
W	Reserved												MIPI_CSI2_ESC_RESET		MIPI_CSI2_PHY_REF_RESET	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_MIPIPHY2_RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved

Table continues on the next page...

SRC_MIIPHY2_RCR field descriptions (continued)

Field	Description
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–3 -	This field is reserved. Reserved
2 MIPI_CSI2_ ESC_RESET	MIPI_CSI2_ESC_RESET
1 MIPI_CSI2_ PHY_REF_ RESET	MIPI_CSI2_PHY_REF_RESET
0 MIPI_CSI2_ CORE_RESET	MIPI_CSI2_CORE_RESET

6.5.6.16 SRC Boot Mode Register 1 (SRC_SBMR1)

The Boot Mode register (SBMR) contains bits that reflect the status of Boot Mode Pins of the chip. The reset value is configuration dependent (depending on boot/fuses/IO pads).

Address: 3039_0000h base + 58h offset = 3039_0058h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	BOOT_CFG																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_SBMR1 field descriptions

Field	Description
BOOT_CFG	Refer to fusemap.

6.5.6.17 SRC Reset Status Register (SRC_SRSR)

The SRSR is a write to one clear register which records the source of the reset events for the chip. The SRC reset status register will capture all the reset sources that have occurred. This register is reset on ipp_reset_b. This is a read-write register.

For bit[9-0] - writing zero does not have any effect. Writing one will clear the corresponding bit. The individual bits can be cleared by writing one to that bit. When the system comes out of reset, this register will have bits set corresponding to all the reset sources that occurred during system reset. Software has to take care to clear this register by writing one after every reset that occurs so that the register will contain the information of recently occurred reset.

Address: 3039_0000h base + 5Ch offset = 3039_005Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R									0							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0						tempense_rst_b	wdog4_rst_b	wdog3_rst_b	jtag_sw_rst	jtag_rst_b	wdog1_rst_b	ipp_user_reset_b	csu_reset_b	0	ipp_reset_b
W								w1c	w1c	w1c	w1c	w1c	w1c	w1c		w1c
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

SRC_SRSR field descriptions

Field	Description
31–10 Reserved	This read-only field is reserved and always has the value 0.
9 tempense_rst_b	Temper Sensor software reset. Indicates whether the reset was the result of software reset from on-chip Temperature Sensor. 0 Reset is not a result of software reset from Temperature Sensor. 1 Reset is a result of software reset from Temperature Sensor.
8 wdog4_rst_b	IC Watchdog4 Time-out reset. Indicates whether the reset was the result of the watchdog4 time-out event. 0 Reset is not a result of the watchdog4 time-out event. 1 Reset is a result of the watchdog4 time-out event.
7 wdog3_rst_b	IC Watchdog3 Time-out reset. Indicates whether the reset was the result of the watchdog3 time-out event. 0 Reset is not a result of the watchdog3 time-out event. 1 Reset is a result of the watchdog3 time-out event.
6 jtag_sw_rst	JTAG software reset. Indicates whether the reset was the result of software reset from JTAG. 0 Reset is not a result of software reset from JTAG. 1 Reset is a result of software reset from JTAG.
5 jtag_rst_b	HIGH - Z JTAG reset. Indicates whether the reset was the result of HIGH-Z reset from JTAG. 0 Reset is not a result of HIGH-Z reset from JTAG. 1 Reset is a result of HIGH-Z reset from JTAG.
4 wdog1_rst_b	IC Watchdog1 Time-out reset. Indicates whether the reset was the result of the watchdog time-out event. 0 Reset is not a result of the watchdog1 time-out event. 1 Reset is a result of the watchdog1 time-out event.
3 ipp_user_reset_b	Indicates whether the reset was the result of the ipp_user_reset_b qualified reset. 0 Reset is not a result of the ipp_user_reset_b qualified as COLD reset event. 1 Reset is a result of the ipp_user_reset_b qualified as COLD reset event.

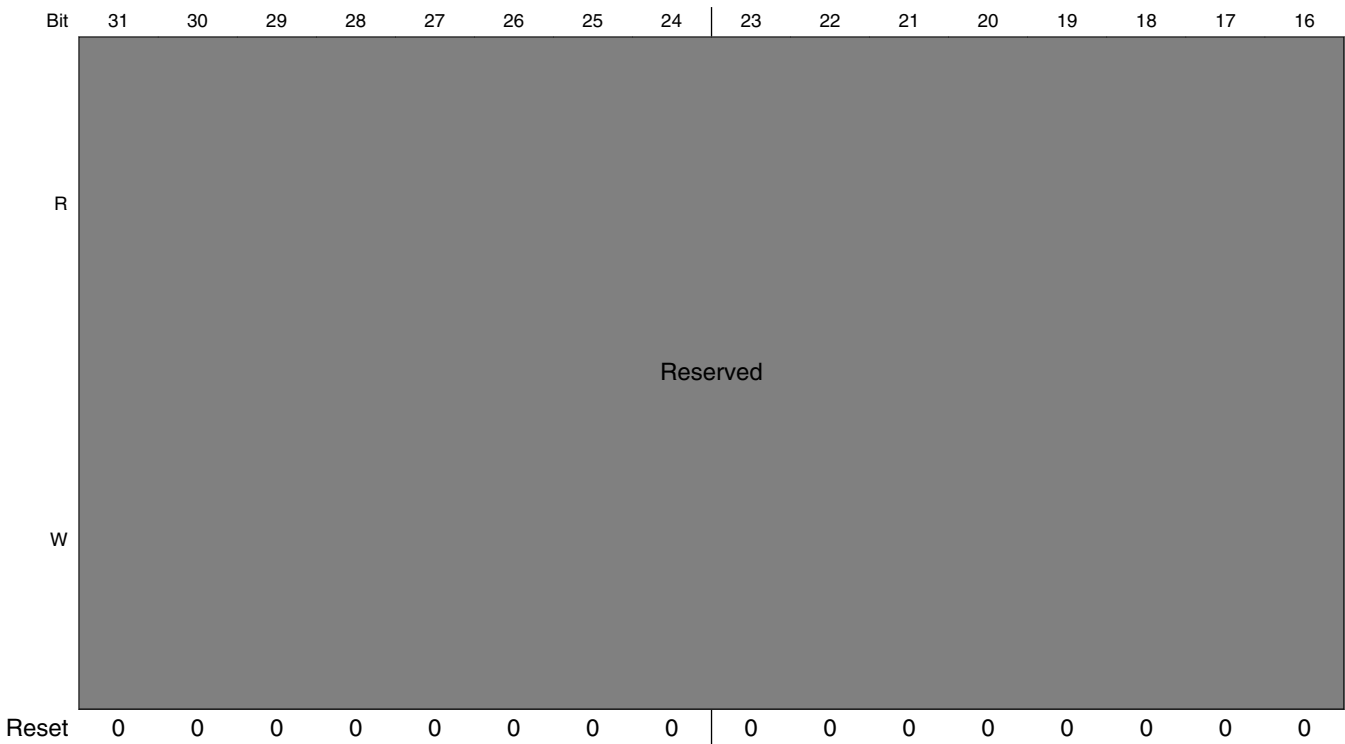
Table continues on the next page...

SRC_SRSR field descriptions (continued)

Field	Description
2 csu_reset_b	Indicates whether the reset was the result of the csu_reset_b input. NOTE: If case the csu_reset_b occurred during a WARM reset process, during the phase that ipg_clk is not available yet, then the occurrence of CSU reset will not be reflected in this bit. 0 Reset is not a result of the csu_reset_b event. 1 Reset is a result of the csu_reset_b event.
1 Reserved	This read-only field is reserved and always has the value 0.
0 ipp_reset_b	Indicates whether reset was the result of ipp_reset_b pin (Power-up sequence) 0 Reset is not a result of ipp_reset_b pin. 1 Reset is a result of ipp_reset_b pin.

6.5.6.18 SRC Interrupt Status Register (SRC_SISR)

Address: 3039_0000h base + 68h offset = 3039_0068h



System Reset Controller (SRC)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Reserved															Reserved
		MIPI_CSI2_PHY_PASSED_RESET	MIPI_CSI1_PHY_PASSED_RESET	PCIE2_PHY_PASSED_RESET	VPU_PASSED_RESET	GPU_PASSED_RESET	M4P_PASSED_RESET	M4C_PASSED_RESET	DISPLAY_PASSED_RESET	HDMI_PASSED_RESET	PCIE1_PHY_PASSED_RESET	MIPIPHY_PASSED_RESET	OTGPHY2_PASSED_RESET	OTGPHY1_PASSED_RESET	HSICPHY_PASSED_RESET	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_SISR field descriptions

Field	Description
31–15 -	This field is reserved. Reserved
14 MIPI_CSI2_ PHY_PASSED_ RESET	Interrupt generated to indicate that MIPI CSI2 PHY passed software reset and is ready to be used 0 interrupt generated not due to MIPI CSI2 PHY reset 1 interrupt generated due to MIPI CSI2 PHY reset
13 MIPI_CSI1_ PHY_PASSED_ RESET	Interrupt generated to indicate that MIPI CSI1 PHY passed software reset and is ready to be used 0 interrupt generated not due to MIPI CSI1 PHY reset 1 interrupt generated due to MIPI CSI1 PHY reset
12 PCIE2_PHY_ PASSED_ RESET	Interrupt generated to indicate that PCIE2 PHY passed software reset and is ready to be used 0 interrupt generated not due to PCIE2 PHY reset 1 interrupt generated due to PCIE2 PHY reset

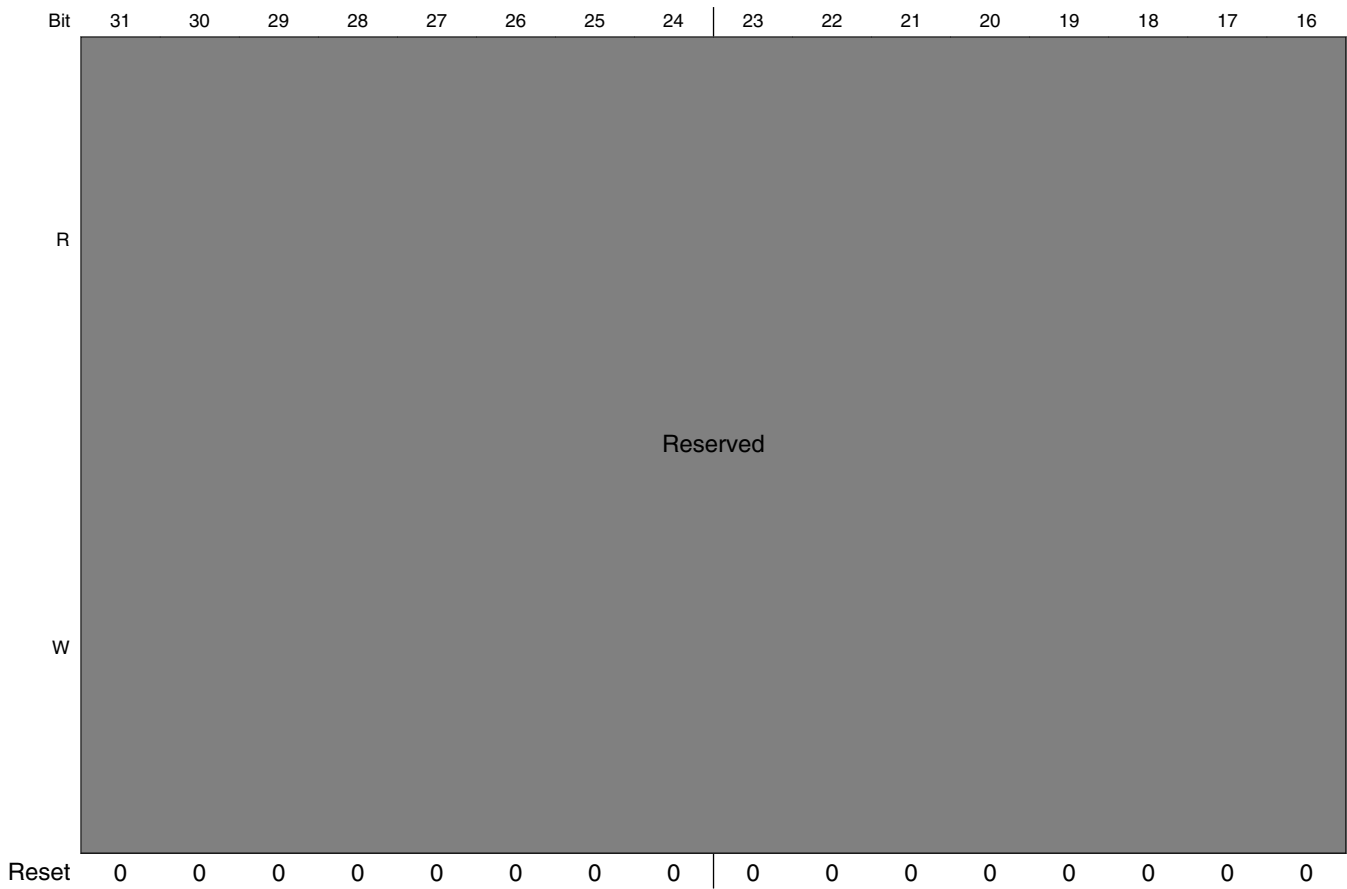
Table continues on the next page...

SRC_SISR field descriptions (continued)

Field	Description
11 VPU_PASSED_ RESET	Interrupt generated to indicate that VPU passed software reset and is ready to be used 0 interrupt generated not due to VPU reset 1 interrupt generated due to VPU reset
10 GPU_PASSED_ RESET	Interrupt generated to indicate that GPU passed software reset and is ready to be used 0 interrupt generated not due to GPU reset 1 interrupt generated due to GPU reset
9 M4P_PASSED_ RESET	Interrupt generated to indicate that m4 platform passed software reset and is ready to be used 0 interrupt generated not due to m4 platform reset 1 interrupt generated due to m4 platform reset
8 M4C_PASSED_ RESET	Interrupt generated to indicate that m4 core passed software reset and is ready to be used 0 interrupt generated not due to m4 core reset 1 interrupt generated due to m4 core reset
7 DISPLAY_ PASSED_ RESET	Interrupt generated to indicate that DISPLAY passed software reset and is ready to be used 0 Interrupt generated not due to DISPLAY passed reset 1 Interrupt generated due to DISPLAY passed reset
6 HDMI_PASSED_ RESET	Interrupt generated to indicate that HDMI passed software reset and is ready to be used 0 Interrupt generated not due to HDMI passed reset 1 Interrupt generated due to HDMI passed reset
5 PCIE1_PHY_ PASSED_ RESET	Interrupt generated to indicate that PCIE1 PHY passed software reset and is ready to be used 0 Interrupt generated not due to PCIE1 PHY passed reset 1 Interrupt generated due to PCIE1 PHY passed reset
4 MIPIPHY_ PASSED_ RESET	Interrupt generated to indicate that MIPI PHY passed software reset and is ready to be used 0 Interrupt generated not due to MIPI PHY passed reset 1 Interrupt generated due to MIPI PHY passed reset
3 OTGPHY2_ PASSED_ RESET	Interrupt generated to indicate that OTG PHY2 passed software reset and is ready to be used 0 Interrupt generated not due to OTG PHY2 passed reset 1 Interrupt generated due to OTG PHY2 passed reset
2 OTGPHY1_ PASSED_ RESET	Interrupt generated to indicate that OTG PHY1 passed software reset and is ready to be used 0 Interrupt generated not due to OTG PHY1 passed reset 1 Interrupt generated due to OTG PHY1 passed reset
1 HSICPHY_ PASSED_ RESET	Interrupt generated to indicate that HSIC PHY passed software reset and is ready to be used 0 Interrupt generated not due to HSIC PHY passed reset 1 Interrupt generated due to HSIC PHY passed reset
0 -	This field is reserved. Reserved

6.5.6.19 SRC Interrupt Mask Register (SRC_SIMR)

Address: 3039_0000h base + 6Ch offset = 3039_006Ch



Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved	MASK_MIPI_CSI2_PHY_PASSED_RESET	MASK_MIPI_CSI1_PHY_PASSED_RESET	MASK_PCIE2_PHY_PASSED_RESET	MASK_VPU_PASSED_RESET	MASK_GPU_PASSED_RESET	MASK_M4P_PASSED_RESET	MASK_M4C_PASSED_RESET	MASK_DISPLAY_PASSED_RESET	MASK_HDMI_PASSED_RESET	MASK_PCIE1_PHY_PASSED_RESET	MASK_MIPIPHY_PASSED_RESET	MASK_OTGPHY2_PASSED_RESET	MASK_OTGPHY1_PASSED_RESET	MASK_HSICPHY_PASSED_RESET	Reserved
W																
Reset	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1

SRC_SIMR field descriptions

Field	Description
31–15 -	This field is reserved. Reserved
14 MASK_MIPI_CSI2_PHY_PASSED_RESET	Mask interrupt generation due to MIPI CSI2 PHY passed reset 0 do not mask interrupt due to MIPI CSI2 PHY passed reset - interrupt will be created 1 mask interrupt due to MIPI CSI2 PHY passed reset

Table continues on the next page...

SRC_SIMR field descriptions (continued)

Field	Description
13 MASK_MIPI_ CSI1_PHY_ PASSED_ RESET	Mask interrupt generation due to MIPI CSI1 PHY passed reset 0 do not mask interrupt due to MIPI CSI1 PHY passed reset - interrupt will be created 1 mask interrupt due to MIPI CSI1 PHY passed reset
12 MASK_PCIE2_ PHY_PASSED_ RESET	Mask interrupt generation due to PCIE2 PHY passed reset 0 do not mask interrupt due to PCIE2 PHY passed reset - interrupt will be created 1 mask interrupt due to PCIE2 PHY passed reset
11 MASK_VPU_ PASSED_ RESET	Mask interrupt generation due to VPU passed reset 0 do not mask interrupt due to VPU passed reset - interrupt will be created 1 mask interrupt due to VPU passed reset
10 MASK_GPU_ PASSED_ RESET	Mask interrupt generation due to GPU passed reset 0 do not mask interrupt due to GPU passed reset - interrupt will be created 1 mask interrupt due to GPU passed reset
9 MASK_M4P_ PASSED_ RESET	mask interrupt generation due to m4 platform passed reset 0 do not mask interrupt due to m4 platform passed reset - interrupt will be created 1 mask interrupt due to m4platform passed reset
8 MASK_M4C_ PASSED_ RESET	mask interrupt generation due to m4 core passed reset 0 do not mask interrupt due to m4 core passed reset - interrupt will be created 1 mask interrupt due to m4 core passed reset
7 MASK_ DISPLAY_ PASSED_ RESET	Mask interrupt generation due to display passed reset 0 do not mask interrupt due to display passed reset - interrupt will be created 1 mask interrupt due to display passed reset
6 MASK_HDMI_ PASSED_ RESET	Mask interrupt generation due to HDMI passed reset 0 do not mask interrupt due to HDMI passed reset - interrupt will be created 1 mask interrupt due to HDMI passed reset
5 MASK_PCIE1_ PHY_PASSED_ RESET	Mask interrupt generation due to PCIE1 PHY passed reset 0 do not mask interrupt due to PCIE1 PHY passed reset - interrupt will be created 1 mask interrupt due to PCIE1 PHY passed reset
4 MASK_ MIPIPHY_ PASSED_ RESET	mask interrupt generation due to MIPI PHY passed reset 0 do not mask interrupt due to MIPI PHY passed reset - interrupt will be created 1 mask interrupt due to MIPI PHY passed reset
3 MASK_ OTGPHY2_ PASSED_ RESET	mask interrupt generation due to OTG PHY2 passed reset 0 do not mask interrupt due to OTG PHY2 passed reset - interrupt will be created 1 mask interrupt due to OTG PHY2 passed reset

Table continues on the next page...

SRC_SIMR field descriptions (continued)

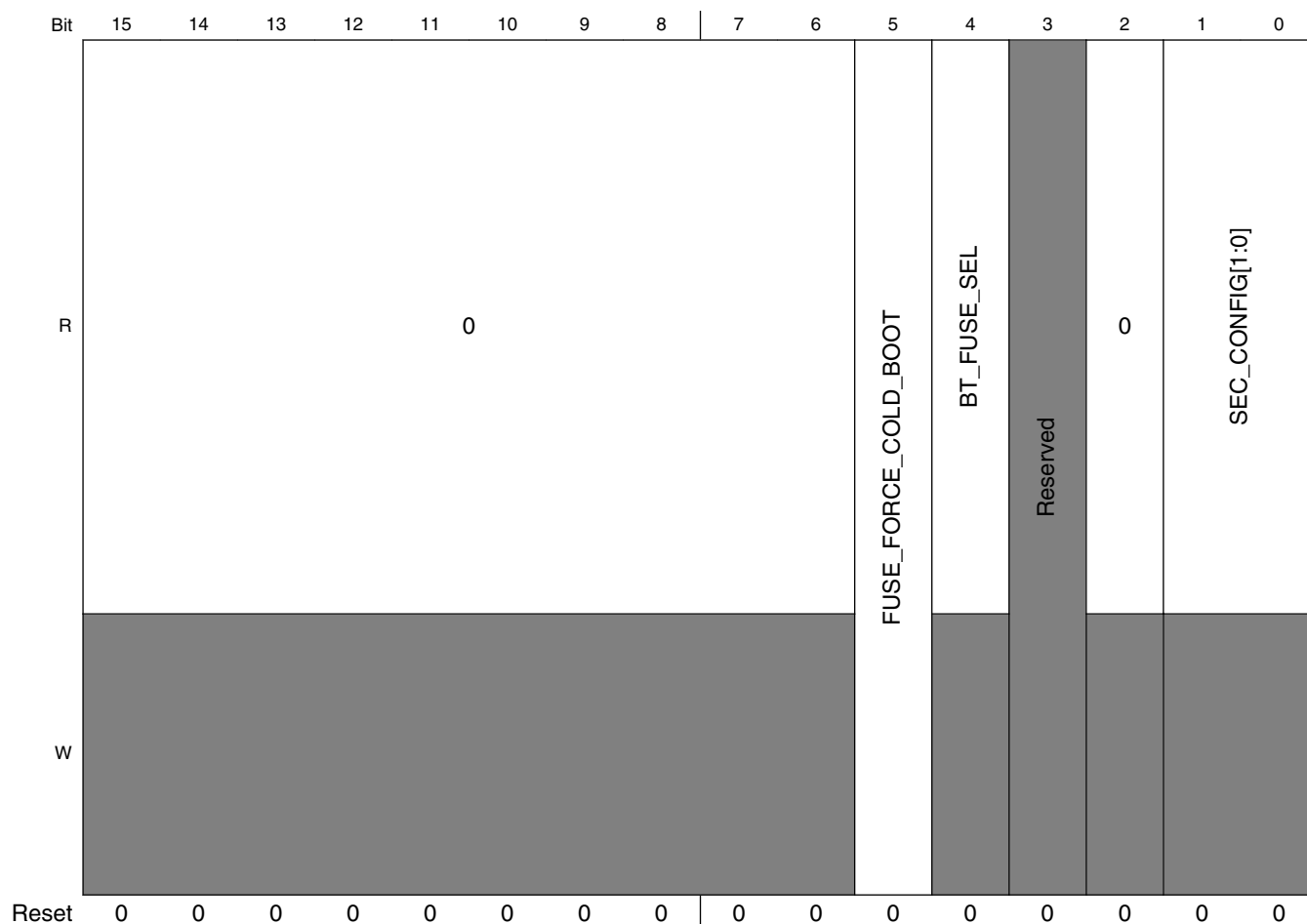
Field	Description
2 MASK_ OTGPHY1_ PASSED_ RESET	mask interrupt generation due to OTG PHY1 passed reset 0 do not mask interrupt due to OTG PHY1 passed reset - interrupt will be created 1 mask interrupt due to OTG PHY1 passed reset
1 MASK_ HSICPHY_ PASSED_ RESET	mask interrupt generation due to HSIC PHY passed reset 0 do not mask interrupt due to HSIC PHY passed reset - interrupt will be created 1 mask interrupt due to HSIC PHY passed reset
0 -	This field is reserved. Reserved

6.5.6.20 SRC Boot Mode Register 2 (SRC_SBMR2)

The Boot Mode register (SBMR), contains bits that reflect the status of Boot Mode Pins of the chip. The default values for those bits depends on the values of pins/fuses during reset sequence.

Address: 3039_0000h base + 70h offset = 3039_0070h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0						BMOD		0							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0



SRC_SBMR2 field descriptions

Field	Description
31–26 Reserved	This read-only field is reserved and always has the value 0.
25–24 BMOD	BMOD shows the latched state of the BOOT_MODE1 and BOOT_MODE0 signals on the rising edge of POR_B. See the Boot mode pin settings section of System Boot.
23–6 Reserved	This read-only field is reserved and always has the value 0.
5 FUSE_FORCE_COLD_BOOT	See Fusemap for additional information.
4 BT_FUSE_SEL	BT_FUSE_SEL (connected to gpio bt_fuse_sel) shows the state of the BT_FUSE_SEL fuse. See Fusemap for additional information on this fuse.
3 -	This field is reserved. Reserved
2 Reserved	This read-only field is reserved and always has the value 0.
SEC_CONFIG[1:0]	SEC_CONFIG[1] shows the state of the SEC_CONFIG[1] fuse. See Fusemap for additional information on this fuse. SEC_CONFIG[0] shows the state of the SEC_CONFIG[0] fuse. This fuse is shown as reserved in Fusemap (address 0x440[1]) because it does not have a user-relevant function.

6.5.6.21 SRC General Purpose Register 1 (SRC_GPR1)

Address: 3039_0000h base + 74h offset = 3039_0074h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PERSISTENT_ENTRY0																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_GPR1 field descriptions

Field	Description
PERSISTENT_ENTRY0	Holds entry function for core0 for waking-up from low power mode. The SRC ensures that the register value will persist across system resets.

6.5.6.22 SRC General Purpose Register 2 (SRC_GPR2)

Address: 3039_0000h base + 78h offset = 3039_0078h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PERSISTENT_ARG0																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_GPR2 field descriptions

Field	Description
PERSISTENT_ARG0	Holds argument of entry function for core0 for waking-up from low power mode. The SRC ensures that the register value will persist across system resets.

6.5.6.23 SRC General Purpose Register 3 (SRC_GPR3)

Address: 3039_0000h base + 7Ch offset = 3039_007Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PERSISTENT_ENTRY1																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_GPR3 field descriptions

Field	Description
PERSISTENT_ENTRY1	Holds entry function for core1. The SRC ensures that the register value will persist across system resets.

6.5.6.24 SRC General Purpose Register 4 (SRC_GPR4)

Address: 3039_0000h base + 80h offset = 3039_0080h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PERSISTENT_ARG1																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_GPR4 field descriptions

Field	Description
PERSISTENT_ARG1	Holds argument of entry function for core1. The SRC ensures that the register value will persist across system resets.

6.5.6.25 SRC General Purpose Register 5 (SRC_GPR5)

Address: 3039_0000h base + 84h offset = 3039_0084h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PERSISTENT_ENTRY2																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_GPR5 field descriptions

Field	Description
PERSISTENT_ENTRY2	Holds entry function for core2. The SRC ensures that the register value will persist across system resets.

6.5.6.26 SRC General Purpose Register 6 (SRC_GPR6)

Address: 3039_0000h base + 88h offset = 3039_0088h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PERSISTENT_ARG2																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_GPR6 field descriptions

Field	Description
PERSISTENT_ARG2	Holds argument of entry function for core2. The SRC ensures that the register value will persist across system resets.

6.5.6.27 SRC General Purpose Register 7 (SRC_GPR7)

Address: 3039_0000h base + 8Ch offset = 3039_008Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PERSISTENT_ENTRY3																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_GPR7 field descriptions

Field	Description
PERSISTENT_ENTRY3	Holds entry function for core3. The SRC ensures that the register value will persist across system resets.

6.5.6.28 SRC General Purpose Register 8 (SRC_GPR8)

Address: 3039_0000h base + 90h offset = 3039_0090h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PERSISTENT_ARG3																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_GPR8 field descriptions

Field	Description
PERSISTENT_ARG3	Holds argument of entry function for core3. The SRC ensures that the register value will persist across system resets.

6.5.6.29 SRC General Purpose Register 9 (SRC_GPR9)

Reserved for Internal Use.

Address: 3039_0000h base + 94h offset = 3039_0094h

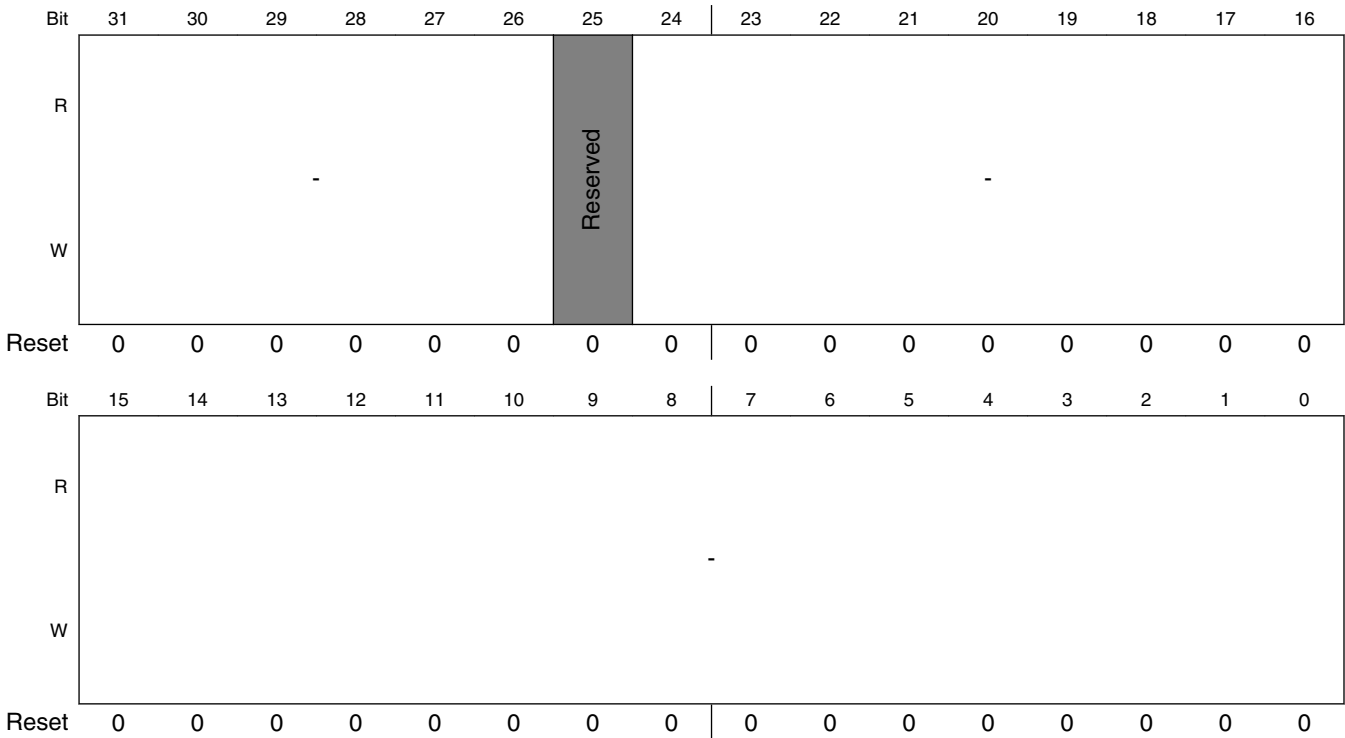
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SRC_GPR9 field descriptions

Field	Description
-	This field is reserved. Reserved.

6.5.6.30 SRC General Purpose Register 10 (SRC_GPR10)

Address: 3039_0000h base + 98h offset = 3039_0098h



SRC_GPR10 field descriptions

Field	Description
31–26 -	Read/write bits, for general purpose NOTE: Reset only by POR
25 -	This field is reserved. Reserved.
-	Read/write bits, for general purpose NOTE: Reset only by POR

6.5.6.31 SRC DDR Controller Reset Control Register (SRC_DDRC_RCR)

DDR Controller Reset Control Register

Address: 3039_0000h base + 1000h offset = 3039_1000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DOM_EN	LOCK	Reserved				DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												DDRC1_PHY_PWROKIN	DDRC1_PHY_RESET	DDRC1_CORE_RST	DDRC1_PRST
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1

SRC_DDRC_RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1.

Table continues on the next page...

SRC_DDRC_RCR field descriptions (continued)

Field	Description
	0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain0 cannot write to this register. 1 This register is assigned to domain0. The master from domain0 can write to this register
23–4 -	This field is reserved. Reserved
3 DDRC1_PHY_ PWROKIN	0 De-assert DDR controller 1 Assert DDR Controller
2 DDRC1_PHY_ RESET	0 De-assert DDR controller 1 Assert DDR Controller
1 DDRC1_CORE_ RST	DDR Controller core_ddrc_rstn and aresetn. This bit is set to 1 automatically when fastmix is powered off. 0 De-assert DDR controller aresetn and core_ddrc_rstn 1 Assert DDR Controller preset and DDR PHY reset
0 DDRC1_PRST	DDR Controller preset and DDR PHY reset. This bit is set to 1 automatically when fastmix is powered off. NOTE: This reset can only be released when DDR Controller clock inputs are active and stable for 30 cycles 0 De-assert DDR Controller preset and DDR PHY reset 1 Assert DDR Controller preset and DDR PHY reset

6.5.6.32 SRC DDRC2 Controller Reset Control Register (SRC_DDRC2_RCR)

DDRC2 Controller Reset Control Register

Address: 3039_0000h base + 1004h offset = 3039_1004h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DOM_EN	LOCK	Reserved				DOMAIN3	DOMAIN2	DOMAIN1	DOMAIN0	Reserved					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												DDRC1_PHY_PWROKIN	DDRC1_PHY_RESET	DDRC2_CORE_RST	DDRC2_PRST
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1

SRC_DDRC2_RCR field descriptions

Field	Description
31 DOM_EN	Domain Control enable for this register. NOTE: [31:30] and [27:24] areas are not controlled by this bit. [31:30] and [27:24] area can be modified by any masters from any domains when Lock bit is not set. 0 Disables domain control. All of this register's bits except [31:30] and [27:24] can be modified by any masters 1 Enables domain control. All of this register's bits except [31:30] and [27:24] can only be modified by the masters from the domains specified in [27:24] area.
30 LOCK	Domain control bits lock NOTE: Lock bit is a write-once register, once it is set to 1, it can't be write to 0 0 [31] and [27:24] bits can be modified 1 [31] and [27:24] bits cannot be modified
29–28 -	This field is reserved. Reserved
27 DOMAIN3	Domain3 assignment control. Effective when dom_en is set to 1.

Table continues on the next page...

SRC_DDRC2_RCR field descriptions (continued)

Field	Description
	0 This register is not assigned to domain3. The master from domain3 cannot write to this register. 1 This register is assigned to domain3. The master from domain3 can write to this register
26 DOMAIN2	Domain2 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain2. The master from domain2 cannot write to this register. 1 This register is assigned to domain2. The master from domain2 can write to this register
25 DOMAIN1	Domain1 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain1. The master from domain1 cannot write to this register. 1 This register is assigned to domain1. The master from domain1 can write to this register
24 DOMAIN0	Domain0 assignment control. Effective when dom_en is set to 1. 0 This register is not assigned to domain0. The master from domain3 cannot write to this register. 1 This register is assigned to domain0. The master from domain3 can write to this register
23–4 -	This field is reserved. Reserved
3 DDRC1_PHY_ PWROKIN	0 De-assert DDR controller 1 Assert DDR Controller
2 DDRC1_PHY_ RESET	0 De-assert DDR controller 1 Assert DDR Controller
1 DDRC2_CORE_ RST	DDRC2 Controller core_ddrc_rstn and aresetn. This bit is set to 1 automatically when fastmix is powered off. 0 De-assert DDR controller aresetn and core_ddrc_rstn 1 Assert DDR Controller preset and DDR PHY reset
0 DDRC2_PRST	DDRC2 Controller preset and DDR PHY reset. This bit is set to 1 automatically when fastmix is powered off. This bit is used as src_system_rst_b of DDR1. NOTE: This reset can only be released when DDR Controller clock inputs are active and stable for 30 cycles 0 De-assert DDRC2 Controller preset and DDR PHY reset 1 Assert DDRC2 Controller preset and DDR PHY reset

Chapter 7

Interrupts and DMA

7.1 Interrupts and DMA Events

7.1.1 Overview

This chapter provides the interrupt assignments of the ARM domain in [A53 Interrupts](#), [CM4 Interrupts](#), and the DMA events in [SDMA event mapping](#)

7.1.2 A53 Interrupts

The Global Interrupt Controller (GIC) collects up to 128 interrupt requests from all the chip sources and provides an interface to the Cortex A53 CPU.

Each interrupt can be configured as a normal or a secure interrupt. Software force registers and software priority masking are also supported. The following table describes the A53 interrupt sources.

Table 7-1. ARM Domain Interrupt Summary

IRQ	Module	Logic	Interrupt Description
IRQ	MODULE	LOGIC	Description
0	boot	-	Used to notify cores on exception condition while boot
1	dap	-	DAP Interrupt
2	sdma1	-	AND of all 48 SDMA interrupts (events) from all the channels
3	gpu	-	GPU Interrupt
4	snvs_lp_wrapper	OR	ON-OFF button press shorter than 5 secs (pulse event)
4	snvs_hp_wrapper	OR	ON-OFF button press shorter than 5 secs (pulse event)
5	lcdif	-	LCDIF Interrupt

Table continues on the next page...

Table 7-1. ARM Domain Interrupt Summary (continued)

IRQ	Module	Logic	Interrupt Description
6	spdif1	OR	SPDIF Rx interrupt
6	spdif1	OR	SPDIF Tx interrupt
7	h264dec	-	h264 Decoder Interrupt
8	vpudma	-	VPU DMA Interrupt
9	qos	-	QOS Interrupt
10	wdog3	-	Watchdog Timer reset
11	hs	-	HS Interrupt Request
12	apbhdma	OR	GPMI operation channel 0 description complete interrupt
12	apbhdma	OR	GPMI operation channel 1 description complete interrupt
12	apbhdma	OR	GPMI operation channel 2 description complete interrupt
12	apbhdma	OR	GPMI operation channel 3 description complete interrupt
13	spdif2	OR	SPDIF Rx interrupt
13	spdif2	OR	SPDIF Tx interrupt
14	rawnand	-	BCH operation complete interrupt
15	rawnand	-	GPMI operation TIMEOUT ERROR interrupt
16	hdmi_ips	-	HDMI IRQ
17	hdmi_ips	-	HDMI IRQ
18	hdmi_ips	-	HDMI IRQ
19	snvs_hp_wrapper	-	SRTC Consolidated Interrupt. Non TZ.
20	snvs_hp_wrapper	-	SRTC Security Interrupt. TZ.
21	csu	-	CSU Interrupt Request. Indicates to the processor that one or more alarm inputs were asserted
22	usdhc1	-	uSDHC1 Enhanced SDHC Interrupt Request
23	usdhc2	-	uSDHC2 Enhanced SDHC Interrupt Request
24	dc8000_control	-	DC8000 Display Controller IRQ
25	DTRC_wrapper	-	DTRC IRQ
26	uart1	-	UART-1 ORed interrupt
27	uart2	-	UART-2 ORed interrupt
28	uart3	-	UART-3 ORed interrupt
29	uart4	-	UART-4 ORed interrupt
30	vp9dec	-	VP9 Decoder Interrupt
31	ecspi1	-	eCSPI1 interrupt request line to the core.
32	ecspi2	-	eCSPI2 interrupt request line to the core.
33	ecspi3	-	eCSPI3 interrupt request line to the core.
34	mipi_dsi	-	MIPI-DSI interrupt
35	i2c1	-	I2C-1 Interrupt

Table continues on the next page...

Table 7-1. ARM Domain Interrupt Summary (continued)

IRQ	Module	Logic	Interrupt Description
36	i2c2	-	I2C-2 Interrupt
37	i2c3	-	I2C-3 Interrupt
38	i2c4	-	I2C-4 Interrupt
39	rdc	-	RDC interrupt
40	usb1	-	USB-1 Interrupt
41	usb2	-	USB-2 Interrupt
42	csi1	-	CSI-1 Interrupt
43	csi2	-	CSI-2 Interrupt
44	mipi_csi1	-	MIPI-CSI-1 Interrupt
45	mipi_csi2	-	MIPI-CSI-2 Interrupt
46	gpt6	-	OR of GPT Rollover interrupt line, Input Capture 1 & 2 lines, Output Compare 1,2 &3 Interrupt lines
47	sctr	-	System Counter Interrupt [0]
48	sctr	-	System Counter Interrupt [1]
49	anamix	OR	TempSensor (Temperature alarm).
49	anamix	OR	TempSensor (Temperature critical alarm).
49	Reserved	-	Reserved
50	sai3	OR	SAI3 Receive Interrupt
50	sai3	OR	SAI3 Receive Async Interrupt
50	sai3	OR	SAI3 Transmit Interrupt
50	sai3	OR	SAI3 Transmit Async Interrupt
51	gpt5	-	OR of GPT Rollover interrupt line, Input Capture 1 & 2 lines, Output Compare 1,2 &3 Interrupt lines
52	gpt4	-	OR of GPT Rollover interrupt line, Input Capture 1 & 2 lines, Output Compare 1,2 &3 Interrupt lines
53	gpt3	-	OR of GPT Rollover interrupt line, Input Capture 1 & 2 lines, Output Compare 1,2 &3 Interrupt lines
54	gpt2	-	OR of GPT Rollover interrupt line, Input Capture 1 & 2 lines, Output Compare 1,2 &3 Interrupt lines
55	gpt1	-	OR of GPT Rollover interrupt line, Input Capture 1 & 2 lines, Output Compare 1,2 &3 Interrupt lines
56	gpio1	-	Active HIGH Interrupt from INT7 from GPIO
57	gpio1	-	Active HIGH Interrupt from INT6 from GPIO
58	gpio1	-	Active HIGH Interrupt from INT5 from GPIO
59	gpio1	-	Active HIGH Interrupt from INT4 from GPIO
60	gpio1	-	Active HIGH Interrupt from INT3 from GPIO
61	gpio1	-	Active HIGH Interrupt from INT2 from GPIO

Table continues on the next page...

Table 7-1. ARM Domain Interrupt Summary (continued)

IRQ	Module	Logic	Interrupt Description
62	gpio1	-	Active HIGH Interrupt from INT1 from GPIO
63	gpio1	-	Active HIGH Interrupt from INT0 from GPIO
64	gpio1	-	Combined interrupt indication for GPIO1 signal 0 throughout 15
65	gpio1	-	Combined interrupt indication for GPIO1 signal 16 throughout 31
66	gpio2	-	Combined interrupt indication for GPIO2 signal 0 throughout 15
67	gpio2	-	Combined interrupt indication for GPIO2 signal 16 throughout 31
68	gpio3	-	Combined interrupt indication for GPIO3 signal 0 throughout 15
69	gpio3	-	Combined interrupt indication for GPIO3 signal 16 throughout 31
70	gpio4	-	Combined interrupt indication for GPIO4 signal 0 throughout 15
71	gpio4	-	Combined interrupt indication for GPIO4 signal 16 throughout 31
72	gpio5	-	Combined interrupt indication for GPIO5 signal 0 throughout 15
73	gpio5	-	Combined interrupt indication for GPIO5 signal 16 throughout 31
74	pcie_ctrl2	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
75	pcie_ctrl2	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
76	pcie_ctrl2	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
77	pcie_ctrl2	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
78	wdog1	-	Watchdog 1 Timer reset
79	wdog2	-	Watchdog 2 Timer reset
80	pcie_ctrl2	-	Channels [63:32] interrupts requests
81	pwm1	-	Cumulative interrupt line. OR of Rollover Interrupt line, Compare Interrupt line and FIFO Waterlevel crossing interrupt line
82	pwm2	-	Cumulative interrupt line. OR of Rollover Interrupt line, Compare Interrupt line and FIFO Waterlevel crossing interrupt line
83	pwm3	-	Cumulative interrupt line. OR of Rollover Interrupt line, Compare Interrupt line and FIFO Waterlevel crossing interrupt line
84	pwm4	-	Cumulative interrupt line. OR of Rollover Interrupt line, Compare Interrupt line and FIFO Waterlevel crossing interrupt line
85	ccmsrcgpcmix	-	CCM, Interrupt Request 1

Table continues on the next page...

Table 7-1. ARM Domain Interrupt Summary (continued)

IRQ	Module	Logic	Interrupt Description
86	ccmsrcgpcmix	-	CCM, Interrupt Request 2
87	ccmsrcgpcmix	-	GPC, Interrupt Request 1
88	mu	-	Interrupt to A53
89	ccmsrcgpcmix	-	SRC interrupt request
90	sai5	OR	SAI5 Receive Interrupt
90	sai5	OR	SAI5 Receive Async Interrupt
90	sai5	OR	SAI5 Transmit Interrupt
90	sai5	OR	SAI5 Transmit Async Interrupt
90	sai6	OR	SAI6 Receive Interrupt
90	sai6	OR	SAI6 Receive Async Interrupt
90	sai6	OR	SAI6 Transmit Interrupt
90	sai6	OR	SAI6 Transmit Async Interrupt
91	rtic	-	RTIC Interrupt
92	cpu	OR	Performance Unit Interrupts from Quad-A53 platform (internally: PMUIRQ[0])
92	cpu	OR	Performance Unit Interrupts from Quad-A53 platform (internally: PMUIRQ[1])
92	cpu	OR	Performance Unit Interrupts from Quad-A53 platform (internally: PMUIRQ[2])
92	cpu	OR	Performance Unit Interrupts from Quad-A53 platform (internally: PMUIRQ[3])
93	cpu	OR	CTI trigger outputs from Quad-A53 platform (internal: nCTIIRQ[0])
93	cpu	OR	CTI trigger outputs from Quad-A53 platform (internal: nCTIIRQ[1])
93	cpu	OR	CTI trigger outputs from Quad-A53 platform (internal: nCTIIRQ[2])
93	cpu	OR	CTI trigger outputs from Quad-A53 platform (internal: nCTIIRQ[3])
94	ccmsrcgpcmix	-	Combined CPU wdog interrupts (4x) out of SRC.
95	sai1	OR	SAI1 Receive Interrupt
95	sai1	OR	SAI1 Receive Async Interrupt
95	sai1	OR	SAI1 Transmit Interrupt
95	sai1	OR	SAI1 Transmit Async Interrupt
96	sai2	OR	SAI2 Receive Interrupt
96	sai2	OR	SAI2 Receive Async Interrupt
96	sai2	OR	SAI2 Transmit Interrupt
96	sai2	OR	SAI2 Transmit Async Interrupt
97	mu	-	Interrupt to M4
98	ddr	-	Interrupt for performance monitor in DRAM controller
99	ddr	-	-

Table continues on the next page...

Table 7-1. ARM Domain Interrupt Summary (continued)

IRQ	Module	Logic	Interrupt Description
100	sai4	OR	SAI4 Receive Interrupt
100	sai4	OR	SAI4 Receive Async Interrupt
100	sai4	OR	SAI4 Transmit Interrupt
100	sai4	OR	SAI4 Transmit Async Interrupt
101	cpu	-	Error indicator for AXI transaction with a write response error condition
102	cpu	-	Error indicator for L2 RAM double-bit ECC error
103	sdma2	-	AND of all 48 SDMA interrupts (events) from all the channels
104	Reserved	-	Reserved
105	caam_wrapper	-	CAAM interrupt queue for JQ
106	caam_wrapper	-	CAAM interrupt queue for JQ
107	qspi	-	QSPI Interrupt
108	tzasc	-	TZASC (PL380) interrupt
109	Reserved	-	Reserved
110	Reserved	-	Reserved
111	Reserved	-	Reserved
112	perfmon1	-	General interrupt
113	perfmon2	-	General interrupt
114	caam_wrapper	-	CAAM interrupt queue for JQ
115	caam_wrapper	-	Recoverable error interrupt
116	hs	-	HS Interrupt Request
117	hevcdec	-	HEVC Decoder Interrupt
118	enet1	OR	MAC 0 Receive Buffer Done
118	enet1	OR	MAC 0 Receive Frame Done
118	enet1	OR	MAC 0 Transmit Buffer Done
118	enet1	OR	MAC 0 Transmit Frame Done
119	enet1	OR	MAC 0 Receive Buffer Done
119	enet1	OR	MAC 0 Receive Frame Done
119	enet1	OR	MAC 0 Transmit Buffer Done
119	enet1	OR	MAC 0 Transmit Frame Done
120	enet1	OR	MAC 0 Periodic Timer Overflow
120	enet1	OR	MAC 0 Time Stamp Available
120	enet1	OR	MAC 0 Payload Receive Error
120	enet1	OR	MAC 0 Transmit FIFO Underrun
120	enet1	OR	MAC 0 Collision Retry Limit
120	enet1	OR	MAC 0 Late Collision
120	enet1	OR	MAC 0 Ethernet Bus Error
120	enet1	OR	MAC 0 MII Data Transfer Done
120	enet1	OR	MAC 0 Receive Buffer Done

Table continues on the next page...

Table 7-1. ARM Domain Interrupt Summary (continued)

IRQ	Module	Logic	Interrupt Description
120	enet1	OR	MAC 0 Receive Frame Done
120	enet1	OR	MAC 0 Transmit Buffer Done
120	enet1	OR	MAC 0 Transmit Frame Done
120	enet1	OR	MAC 0 Graceful Stop
120	enet1	OR	MAC 0 Babbling Transmit Error
120	enet1	OR	MAC 0 Babbling Receive Error
120	enet1	OR	MAC 0 Receive Flush Frame0
120	enet1	OR	MAC 0 Receive Flush Frame1
120	enet1	OR	MAC 0 Receive Flush Frame2
120	enet1	OR	MAC 0 Wakeup Request (sync)
120	enet1	OR	MAC 0 Babbling Receive Error
120	enet1	OR	MAC 0 Wakeup Request (sync)
121	enet1	-	MAC 0 1588 Timer Interrupt – synchronous
122	pcie_ctrl1	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
123	pcie_ctrl1	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
124	pcie_ctrl1	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
125	pcie_ctrl1	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
126	Reserved	-	Reserved
127	pcie_ctrl1	-	Channels [63:32] interrupts requests

7.1.3 CM4 Interrupts

The Nested Vectored Interrupt Controller (NVIC) collects up to 128 interrupt requests from all chip sources and provides an interface to the Cortex M4 Core.

The following table describes the M4 interrupt sources.

Table 7-2. CM4 Interrupt Summary

IRQ	Module	Logic	Interrupt Description
IRQ	MODULE	LOGIC	Description
0	boot	-	Used to notify cores on exception condition while boot
1	dap	-	DAP Interrupt
2	sdma1	-	AND of all 48 SDMA interrupts (events) from all the channels

Table continues on the next page...

Table 7-2. CM4 Interrupt Summary (continued)

IRQ	Module	Logic	Interrupt Description
3	gpu	-	GPU Interrupt
4	snvs_lp_wrapper	OR	ON-OFF button press shorter than 5 secs (pulse event)
4	snvs_hp_wrapper	OR	ON-OFF button press shorter than 5 secs (pulse event)
5	lcdif	-	LCDIF Interrupt
6	spdif1	OR	SPDIF Rx interrupt
6	spdif1	OR	SPDIF Tx interrupt
7	h264dec	-	h264 Decoder Interrupt
8	vpudma	-	VPU DMA Interrupt
9	qos	-	QOS Interrupt
10	wdog3	-	Watchdog Timer reset
11	hs	-	HS Interrupt Request
12	apbhdma	OR	GPMI operation channel 0 description complete interrupt
12	apbhdma	OR	GPMI operation channel 1 description complete interrupt
12	apbhdma	OR	GPMI operation channel 2 description complete interrupt
12	apbhdma	OR	GPMI operation channel 3 description complete interrupt
13	spdif2	OR	SPDIF Rx interrupt
13	spdif2	OR	SPDIF Tx interrupt
14	rawnand	-	BCH operation complete interrupt
15	rawnand	-	GPMI operation TIMEOUT ERROR interrupt
16	hdmi_ips	-	HDMI IRQ
17	hdmi_ips	-	HDMI IRQ
18	hdmi_ips	-	HDMI IRQ
19	snvs_hp_wrapper	-	SRTC Consolidated Interrupt. Non TZ.
20	snvs_hp_wrapper	-	SRTC Security Interrupt. TZ.
21	csu	-	CSU Interrupt Request. Indicates to the processor that one or more alarm inputs were asserted
22	usdhc1	-	uSDHC1 Enhanced SDHC Interrupt Request
23	usdhc2	-	uSDHC2 Enhanced SDHC Interrupt Request
24	dc8000_control	-	DC8000 Display Controller IRQ
25	DTRC_wrapper	-	DTRC IRQ
26	uart1	-	UART-1 ORed interrupt
27	uart2	-	UART-2 ORed interrupt
28	uart3	-	UART-3 ORed interrupt
29	uart4	-	UART-4 ORed interrupt
30	vp9dec	-	VP9 Decoder Interrupt

Table continues on the next page...

Table 7-2. CM4 Interrupt Summary (continued)

IRQ	Module	Logic	Interrupt Description
31	ecspi1	-	eCSPI1 interrupt request line to the core.
32	ecspi2	-	eCSPI2 interrupt request line to the core.
33	ecspi3	-	eCSPI3 interrupt request line to the core.
34	mipi_dsi	-	MIPI-DSI interrupt
35	i2c1	-	I2C-1 Interrupt
36	i2c2	-	I2C-2 Interrupt
37	i2c3	-	I2C-3 Interrupt
38	i2c4	-	I2C-4 Interrupt
39	rdc	-	RDC interrupt
40	usb1	-	USB-1 Interrupt
41	usb2	-	USB-2 Interrupt
42	csi1	-	CSI-1 Interrupt
43	csi2	-	CSI-2 Interrupt
44	mipi_csi1	-	MIPI-CSI-1 Interrupt
45	mipi_csi2	-	MIPI-CSI-2 Interrupt
46	gpt6	-	OR of GPT Rollover interrupt line, Input Capture 1 & 2 lines, Output Compare 1,2 &3 Interrupt lines
47	sctr	-	System Counter Interrupt [0]
48	sctr	-	System Counter Interrupt [1]
49	anamax	OR	TempSensor (Temperature alarm).
49	anamax	OR	TempSensor (Temperature critical alarm).
49	Reserved	-	Reserved
50	sai3	OR	SAI3 Receive Interrupt
50	sai3	OR	SAI3 Receive Async Interrupt
50	sai3	OR	SAI3 Transmit Interrupt
50	sai3	OR	SAI3 Transmit Async Interrupt
51	gpt5	-	OR of GPT Rollover interrupt line, Input Capture 1 & 2 lines, Output Compare 1,2 &3 Interrupt lines
52	gpt4	-	OR of GPT Rollover interrupt line, Input Capture 1 & 2 lines, Output Compare 1,2 &3 Interrupt lines
53	gpt3	-	OR of GPT Rollover interrupt line, Input Capture 1 & 2 lines, Output Compare 1,2 &3 Interrupt lines
54	gpt2	-	OR of GPT Rollover interrupt line, Input Capture 1 & 2 lines, Output Compare 1,2 &3 Interrupt lines
55	gpt1	-	OR of GPT Rollover interrupt line, Input Capture 1 & 2 lines, Output Compare 1,2 &3 Interrupt lines
56	gpio1	-	Active HIGH Interrupt from INT7 from GPIO

Table continues on the next page...

Table 7-2. CM4 Interrupt Summary (continued)

IRQ	Module	Logic	Interrupt Description
57	gpio1	-	Active HIGH Interrupt from INT6 from GPIO
58	gpio1	-	Active HIGH Interrupt from INT5 from GPIO
59	gpio1	-	Active HIGH Interrupt from INT4 from GPIO
60	gpio1	-	Active HIGH Interrupt from INT3 from GPIO
61	gpio1	-	Active HIGH Interrupt from INT2 from GPIO
62	gpio1	-	Active HIGH Interrupt from INT1 from GPIO
63	gpio1	-	Active HIGH Interrupt from INT0 from GPIO
64	gpio1	-	Combined interrupt indication for GPIO1 signal 0 throughout 15
65	gpio1	-	Combined interrupt indication for GPIO1 signal 16 throughout 31
66	gpio2	-	Combined interrupt indication for GPIO2 signal 0 throughout 15
67	gpio2	-	Combined interrupt indication for GPIO2 signal 16 throughout 31
68	gpio3	-	Combined interrupt indication for GPIO3 signal 0 throughout 15
69	gpio3	-	Combined interrupt indication for GPIO3 signal 16 throughout 31
70	gpio4	-	Combined interrupt indication for GPIO4 signal 0 throughout 15
71	gpio4	-	Combined interrupt indication for GPIO4 signal 16 throughout 31
72	gpio5	-	Combined interrupt indication for GPIO5 signal 0 throughout 15
73	gpio5	-	Combined interrupt indication for GPIO5 signal 16 throughout 31
74	pcie_ctrl2	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
75	pcie_ctrl2	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
76	pcie_ctrl2	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
77	pcie_ctrl2	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
78	wdog1	-	Watchdog 1 Timer reset
79	wdog2	-	Watchdog 2 Timer reset
80	pcie_ctrl2	-	Channels [63:32] interrupts requests
81	pwm1	-	Cumulative interrupt line. OR of Rollover Interrupt line, Compare Interrupt line and FIFO Waterlevel crossing interrupt line
82	pwm2	-	Cumulative interrupt line. OR of Rollover Interrupt line, Compare Interrupt line and FIFO Waterlevel crossing interrupt line

Table continues on the next page...

Table 7-2. CM4 Interrupt Summary (continued)

IRQ	Module	Logic	Interrupt Description
83	pwm3	-	Cumulative interrupt line. OR of Rollover Interrupt line, Compare Interrupt line and FIFO Waterlevel crossing interrupt line
84	pwm4	-	Cumulative interrupt line. OR of Rollover Interrupt line, Compare Interrupt line and FIFO Waterlevel crossing interrupt line
85	ccmsrcgpcmix	-	CCM, Interrupt Request 1
86	ccmsrcgpcmix	-	CCM, Interrupt Request 2
87	ccmsrcgpcmix	-	GPC, Interrupt Request 1
88	mu	-	Interrupt to A53
89	ccmsrcgpcmix	-	SRC interrupt request
90	sai5	OR	SAI5 Receive Interrupt
90	sai5	OR	SAI5 Receive Async Interrupt
90	sai5	OR	SAI5 Transmit Interrupt
90	sai5	OR	SAI5 Transmit Async Interrupt
90	sai6	OR	SAI6 Receive Interrupt
90	sai6	OR	SAI6 Receive Async Interrupt
90	sai6	OR	SAI6 Transmit Interrupt
90	sai6	OR	SAI6 Transmit Async Interrupt
91	rtic	-	RTIC Interrupt
92	cpu	OR	Performance Unit Interrupts from Quad-A53 platform (internally: PMUIRQ[0])
92	cpu	OR	Performance Unit Interrupts from Quad-A53 platform (internally: PMUIRQ[1])
92	cpu	OR	Performance Unit Interrupts from Quad-A53 platform (internally: PMUIRQ[2])
92	cpu	OR	Performance Unit Interrupts from Quad-A53 platform (internally: PMUIRQ[3])
93	cpu	OR	CTI trigger outputs from Quad-A53 platform (internal: nCTIIRQ[0])
93	cpu	OR	CTI trigger outputs from Quad-A53 platform (internal: nCTIIRQ[1])
93	cpu	OR	CTI trigger outputs from Quad-A53 platform (internal: nCTIIRQ[2])
93	cpu	OR	CTI trigger outputs from Quad-A53 platform (internal: nCTIIRQ[3])
94	ccmsrcgpcmix	-	Combined CPU wdog interrupts (4x) out of SRC.
95	sai1	OR	SAI1 Receive Interrupt
95	sai1	OR	SAI1 Receive Async Interrupt
95	sai1	OR	SAI1 Transmit Interrupt
95	sai1	OR	SAI1 Transmit Async Interrupt
96	sai2	OR	SAI2 Receive Interrupt
96	sai2	OR	SAI2 Receive Async Interrupt

Table continues on the next page...

Table 7-2. CM4 Interrupt Summary (continued)

IRQ	Module	Logic	Interrupt Description
96	sai2	OR	SAI2 Transmit Interrupt
96	sai2	OR	SAI2 Transmit Async Interrupt
97	mu	-	Interrupt to M4
98	ddr	-	Interrupt for performance monitor in DRAM controller
99	ddr	-	-
100	sai4	OR	SAI4 Receive Interrupt
100	sai4	OR	SAI4 Receive Async Interrupt
100	sai4	OR	SAI4 Transmit Interrupt
100	sai4	OR	SAI4 Transmit Async Interrupt
101	cpu	-	Error indicator for AXI transaction with a write response error condition
102	cpu	-	Error indicator for L2 RAM double-bit ECC error
103	sdma2	-	AND of all 48 SDMA interrupts (events) from all the channels
104	Reserved	-	Reserved
105	caam_wrapper	-	CAAM interrupt queue for JQ
106	caam_wrapper	-	CAAM interrupt queue for JQ
107	qspi	-	QSPI Interrupt
108	tzasc	-	TZASC (PL380) interrupt
109	Reserved	-	Reserved
110	Reserved	-	Reserved
111	Reserved	-	Reserved
112	perfmon1	-	General interrupt
113	perfmon2	-	General interrupt
114	caam_wrapper	-	CAAM interrupt queue for JQ
115	caam_wrapper	-	Recoverable error interrupt
116	hs	-	HS Interrupt Request
117	hevcdec	-	HEVC Decoder Interrupt
118	enet1	OR	MAC 0 Receive Buffer Done
118	enet1	OR	MAC 0 Receive Frame Done
118	enet1	OR	MAC 0 Transmit Buffer Done
118	enet1	OR	MAC 0 Transmit Frame Done
119	enet1	OR	MAC 0 Receive Buffer Done
119	enet1	OR	MAC 0 Receive Frame Done
119	enet1	OR	MAC 0 Transmit Buffer Done
119	enet1	OR	MAC 0 Transmit Frame Done
120	enet1	OR	MAC 0 Periodic Timer Overflow
120	enet1	OR	MAC 0 Time Stamp Available
120	enet1	OR	MAC 0 Payload Receive Error

Table continues on the next page...

Table 7-2. CM4 Interrupt Summary (continued)

IRQ	Module	Logic	Interrupt Description
120	enet1	OR	MAC 0 Transmit FIFO Underrun
120	enet1	OR	MAC 0 Collision Retry Limit
120	enet1	OR	MAC 0 Late Collision
120	enet1	OR	MAC 0 Ethernet Bus Error
120	enet1	OR	MAC 0 MII Data Transfer Done
120	enet1	OR	MAC 0 Receive Buffer Done
120	enet1	OR	MAC 0 Receive Frame Done
120	enet1	OR	MAC 0 Transmit Buffer Done
120	enet1	OR	MAC 0 Transmit Frame Done
120	enet1	OR	MAC 0 Graceful Stop
120	enet1	OR	MAC 0 Babbling Transmit Error
120	enet1	OR	MAC 0 Babbling Receive Error
120	enet1	OR	MAC 0 Receive Flush Frame0
120	enet1	OR	MAC 0 Receive Flush Frame1
120	enet1	OR	MAC 0 Receive Flush Frame2
120	enet1	OR	MAC 0 Wakeup Request (sync)
120	enet1	OR	MAC 0 Babbling Receive Error
120	enet1	OR	MAC 0 Wakeup Request (sync)
121	enet1	-	MAC 0 1588 Timer Interrupt – synchronous
122	pcie_ctrl1	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
123	pcie_ctrl1	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
124	pcie_ctrl1	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
125	pcie_ctrl1	-	Coming from GLUE logic, of set/reset FF, driven by PCIE signals.
126	Reserved	-	Reserved
127	pcie_ctrl1	-	Channels [63:32] interrupts requests

7.1.4 SDMA event mapping

The following table shows the DMA request signals for peripherals in the chip.

Table 7-3. SDMA1 event mapping

SDMA	Module	Description
0	ecspi1	eCSPI1 Rx request
1	ecspi1	eCSPI1 Tx request

Table continues on the next page...

Table 7-3. SDMA1 event mapping (continued)

SDMA	Module	Description
2	ecspi2	eCSPI2 Rx request
3	ecspi2	eCSPI2 Tx request
4	-	Reserved
5	-	Reserved
6	-	Reserved
7	-	Reserved
8	spdif1	SPDIF1 Rx DMA request
9	spdif1	SPDIF1 Tx DMA request
10	sai2	SAI-2 receive DMA request
11	sai2	SAI-2 transmit DMA request
12	sai3	SAI-3 receive DMA request
13	sai3	SAI-3 transmit DMA request
14	iomux	external DMA from pad through IOMUX #1
15	iomux	external DMA from pad through IOMUX #2
16	spdif2	SPDIF2 Rx DMA request
17	spdif2	SPDIF2 Tx DMA request
18	i2c1	I2C1 DMA event
19	i2c2	I2C2 DMA event
20	i2c3	I2C3 DMA event
21	i2c4	I2C4 DMA event
22	uart1	Rx FIFO
23	uart1	Tx FIFO
24	uart2	Rx FIFO
25	uart2	Tx FIFO
26	uart3	Rx FIFO
27	uart3	Tx FIFO
28	uart4	Rx FIFO
29	uart4	Tx FIFO
30	-	Reserved
31	-	Reserved
32	-	Reserved
33	-	Reserved
34	-	Reserved
35	-	Reserved
36	qspi1	QSPI DMA TX request
37	qspi1	QSPI DMA RX request
38	gpt1	GPT1 counter event
39	gpt2	GPT2 counter event
40	gpt3	GPT3 counter event

Table continues on the next page...

Table 7-3. SDMA1 event mapping (continued)

SDMA	Module	Description
41	-	Reserved
42	-	Reserved
43	-	Reserved
44	enet1	ENET1 1588 Event2 out
45	enet1	ENET1 1588 Event0 out
46	enet1	ENET1 1588 Event3 out
47	enet1	ENET1 1588 Event1 out

Table 7-4. SDMA2 event mapping

SDMA	Module	Description
SDMA	MODULE	Description
0	sai4	SAI-4 receive DMA request
1	sai4	SAI-4 transmit DMA request
2	sai5	SAI-5 receive DMA request
3	sai5	SAI-5 transmit DMA request
4	sai6	SAI-6 receive DMA request
5	sai6	SAI-6 transmit DMA request
6	-	Reserved
7	-	Reserved
8	sai1	SAI-1 receive DMA request
9	sai1	SAI-1 transmit DMA request
10	-	Reserved
11	-	Reserved
12	-	Reserved
13	-	Reserved
14	iomux	external DMA from pad through IOMUX #1
15	iomux	external DMA from pad through IOMUX #2
16	-	Reserved
17	-	Reserved
18	-	Reserved
19	-	Reserved
20	-	Reserved
21	-	Reserved
22	-	Reserved
23	-	Reserved
24	-	Reserved
25	-	Reserved
26	-	Reserved

Table continues on the next page...

Table 7-4. SDMA2 event mapping (continued)

SDMA	Module	Description
27	-	Reserved
28	-	Reserved
29	-	Reserved
30	-	Reserved
31	-	Reserved
32	-	Reserved
33	-	Reserved
34	-	Reserved
35	-	Reserved
36	-	Reserved
37	-	Reserved
38	gpt4	GPT4 counter event
39	gpt5	GPT5 counter event
40	gpt6	GPT6 counter event
41	-	Reserved
42	-	Reserved
43	-	Reserved
44	-	Reserved
45	-	Reserved
46	-	Reserved
47	-	Reserved

7.2 Smart Direct Memory Access Controller (SDMA)

7.2.1 Overview

The Smart Direct Memory Access (SDMA) controller offers highly-competitive DMA features combined with software-based virtual-DMA flexibility. It enables data transfers between peripheral I/O devices and internal/external memories.

The SDMA controller helps maximize system performance by off-loading the Arm core in dynamic data routing.

7.2.1.1 Block Diagram

The figure below shows a block diagram of the SDMA controller. It includes the custom RISC core along with its RAM, ROM, DMA units, and the scheduler.

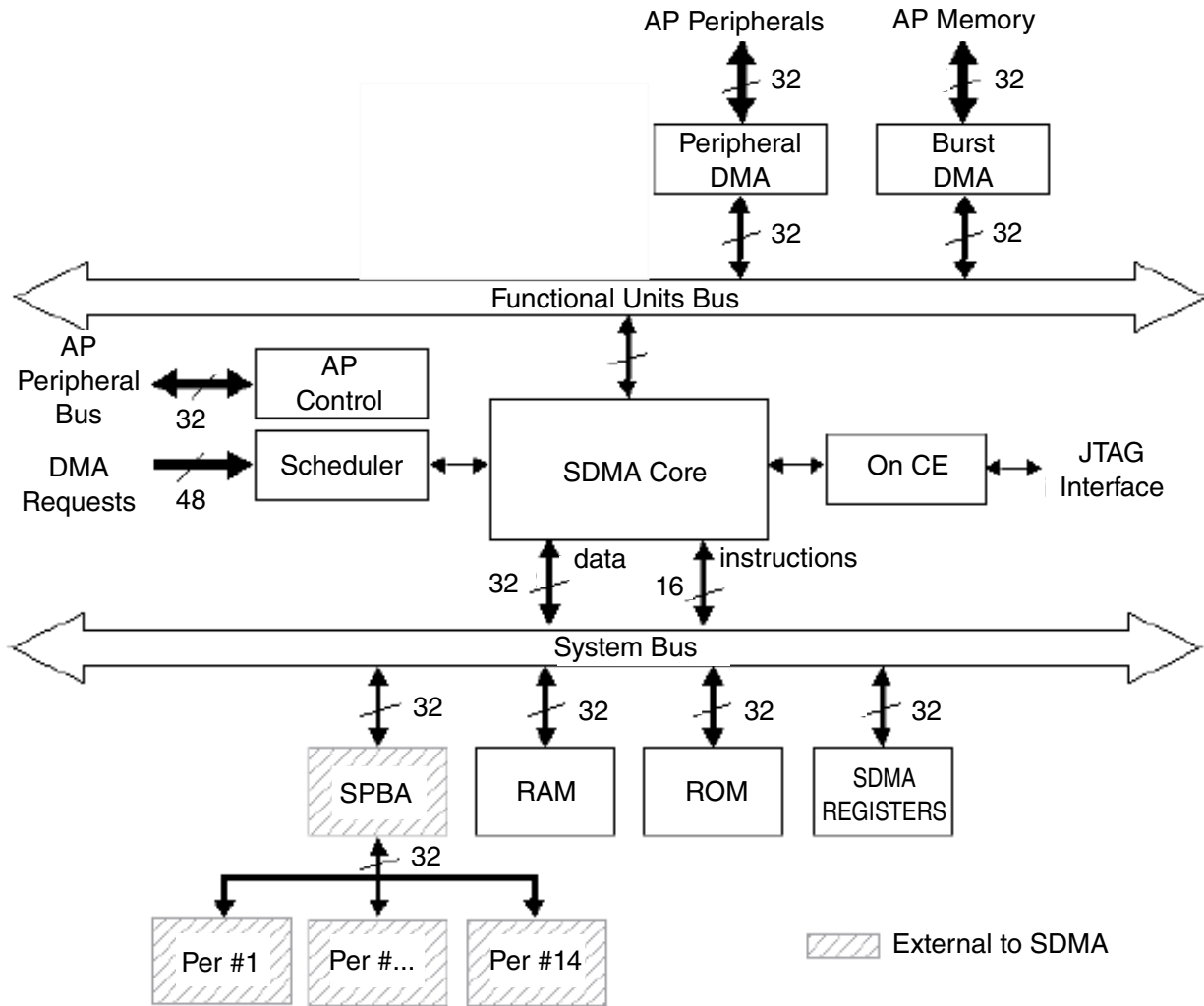


Figure 7-1. SDMA Block Diagram

The SDMA core executes short routines that perform DMA transfers; these routines are called *scripts*. The SDMA core interfaces to its own memory via the SDMA system bus. The SDMA system bus supports a 32-bit data path and a 16-bit address bus. The system bus datapath is used for both 16-bit instruction (program) memory access and 32-bit data access. DMA units interface to the core via the Functional Unit Bus and use dedicated registers to perform DMA transfers.

The SDMA memory contains a ROM and a RAM. The ROM contains startup scripts (for example, boot code) and other common utilities, which are referenced by the scripts that reside in the RAM. The internal RAM is divided into a context area and a script area (more details about this mapping are available in [Instruction Memory Map](#) and [Data Memory Map](#)).

Every transfer channel requires one context area to keep the contents of all the core and unit registers while inactive. Channel scripts are downloaded into the internal RAM by the SDMA using a dedicated channel that is started during the boot sequence. Downloads are invoked using commands and pointers provided by the Arm platform. Every channel contains a corresponding channel script located in RAM and/or ROM that can be reconfigured independently as-needed. Channel scripts can be stored in an external memory and downloaded when needed. The SDMA can be configured with any mixture of scripts to enable an endless combination of supported services.

The scheduler monitors and detects DMA requests, mapping them to channels, and mapping individual channels to a pre-configured priority. At any given point, the scheduler presents the highest priority channel that requires service to the SDMA core. A special SDMA core instruction is used to "conditionally yield" the current channel being executed to an eligible channel that requires service. If (and only if) there is an eligible channel pending, will the current channel execution be preempted.

There are two yield instructions that differently determine the eligible channels: In the first version, eligible channels are pending channels with a strictly higher priority than the current channel priority. In the second version (yieldge), eligible channels are pending channels with a priority that is greater or equal to the current channel priority. The scheduler detects devices that need service through its 48 DMA request inputs. After a request is detected, the scheduler determines the channel(s) that is (are) triggered by this request and marks it (them) as pending in the "Channel Pending (EP)" register. The priorities of all the pending channels are continuously evaluated in order to update the highest pending priority. The channel pending flag is cleared by the channel script when the transfer has completed.

The Arm platform control block contains the control registers used to configure the 32 individual channels. There are 48 Channel Enable registers, and every register maps one DMA request to any desired combination of channels. The 32 Priority registers are used to assign a programmable 1-of-7 level priority to every possible channel. This block also contains all other control registers that the Arm platform can access.

The 48 DMA requests that are connected to the scheduler come from a variety of sources. The "receive register full" and "transmit register empty" signals found in the UART and USB ports are typical examples of DMA requests that can be connected to the SDMA. These requests can be used to trigger a specific SDMA channel, or several channels.

There is an OnCE compatible debug port for product development. The OnCE includes support for setting breakpoints, single-step and trace, and register dump capability. In addition, all memory locations are accessible from the debug port.

7.2.1.2 Features

The following are the SDMA features:

- Multi-channel DMA supporting up to 32 time-division multiplexed DMA channels
- Hardware or software driven triggers for each channel
- 48 hardware driven triggers that can be mapped to any channel.
- Memory accesses including linear addressing, FIFO addressing and 2D addressing
- Fast context-switching with two-level, priority-based preemptive multi-tasking
- 16-bit instruction-set micro-RISC engine (the SDMA core)
- Two DMA units with some or all the following features:
 - Auto-flush and prefetch capability
 - Flexible address management (increment, decrement, and no address changes on source and destination address)
 - Misaligned data-transfer support
 - Uni-directional and bi-directional flows (copy mode)
 - Up to eight-word buffers for configurable burst transfers
- Support of byte-swapping
- An available API and library of scripts
- Little-Endian and Big-Endian modes
- Hardware handshakes for low-power entry sequence
- Security support to lock contents of the SDMA script RAM.
- 4-Kbyte ROM containing startup scripts (for example, boot code) and other common utilities that can be referenced by RAM-located scripts
- 8-Kbyte RAM area is divided into a processor context area and a code space area used to store channel scripts that are downloaded from the system memory
- Debug support, including a OnCE port, real-time monitors, and embedded cross-trigger events
- Supported clock frequencies in process:
 - Configurable clock options for the SDMA core and the Arm platform DMA units
 - 1:2 ratio with maximum of SDMA core running at Arm platform Peripheral Bus speed and DMA running at max DMA frequency.
 - 1:1 ratio when both SDMA core and Arm platform DMA clocks are set to the Arm platform Peripheral Bus speed.
- Peripheral bus interface for configuration register programming by the Arm platform

- The SDMA RISC engine (arithmetic and logic operations), which is referred to as the "SDMA core."
- An internal peripheral bus connected to the Shared Peripherals Bus Interface (SPBA) that enables access to up to 14 shared peripherals. SDMA supports 32-bit accesses to word peripherals and 16-bit accesses to half-word peripherals.
- The peripheral DMA unit that is hooked-up to the Arm platform Crossbar Switch to service Arm peripherals
- The burst DMA unit is able to perform burst accesses to the external memory
- All the DMA units are 32-bit AHB masters. They are connected to different buses, thus allowing concurrent accesses.

7.2.2 Clocks

The following table describes the clock sources for SDMA. Please see [Clock Controller Module \(CCM\)](#) for clock setting, configuration and gating information. For functional information regarding module clocks, see [SDMA Clocks and Low Power Modes](#).

Table 7-5. SDMA Clocks

Clock name	Clock Root	Description
events_sync_clk (clk)	ahb_clk_root	Arm peripheral / events clock
ips_hostctrl_clk	ipg_clk_root	Host control clock
ap_ahb_clk	ahb_clk_root	Arm platform bus clock
core_clk	ipg_clk_root	Module / Core clock
tck	-	JTAG access clock

7.2.3 Functional Description

The figure below shows the SDMA topology, and is composed of the following components:

- SDMA Core ([SDMA Core](#))
- SDMA Scheduler ([Scheduler](#))
- Functional Units:
 - Burst DMA ([Burst DMA Unit](#))
 - Peripheral DMA ([Peripheral DMA Unit](#))
- Arm platform Control for Arm control register access.
- Internal RAM and ROM Memory ([SDMA Programming Model](#))
- OnCE debug Port ([The OnCE Controller](#))

The functional unit bus provides access by the SDMA core to the DMA units. The system bus provides access to SDMA internal memory and also supports up to 14 peripherals.

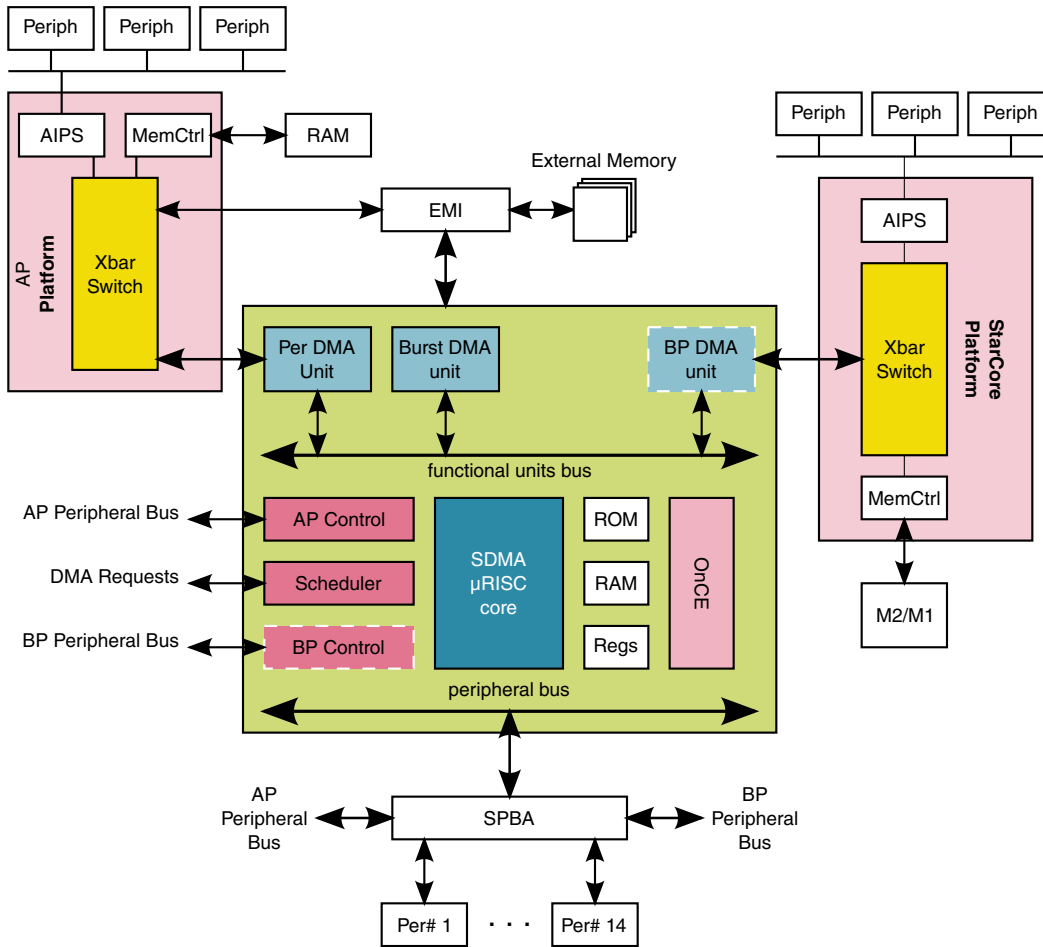


Figure 7-2. SDMA Connections

7.2.3.1 SDMA Core

The SDMA core is a customized RISC-like processor that is specifically developed to control DMA units and perform L1 tasks like byte-stuffing or framing.

The SDMA core incorporates on-chip debug capability using the OnCE.

The SDMA core is based on a 32-bit register architecture with 16-bit instructions. There are eight general purpose 32-bit registers, four flags (T, LM, SF, and DF), and four PCU registers (PC, RPC, SPC, and EPC) that can address 16,384 16-bit instructions.

7.2.3.1.1 SDMA Core Structure

The figure found here shows the structure of the SDMA core. It also shows the different registers, calculation resources, and possible data movements.

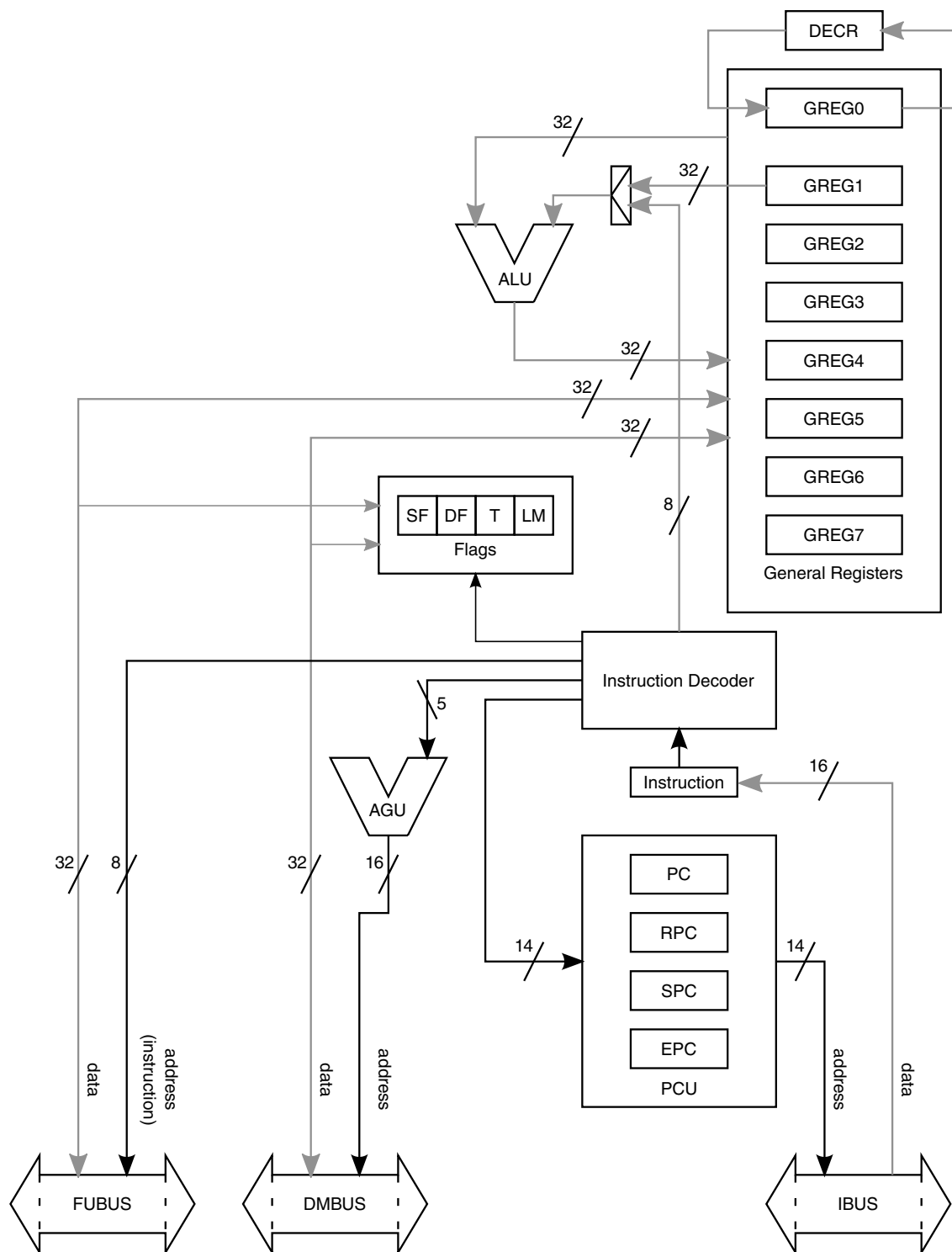


Figure 7-3. SDMA Core

- The Program Control Unit (PCU) is described in [Program Control Unit \(PCU\)](#). It handles the state of the core and generates the instruction fetch addresses. Instructions are retrieved from the Instruction Bus (IBUS) and stored in the SDMA core instruction register prior to their decoding. The PCU contains the following registers:
 - The Program Counter (PC) contains the address of the current instruction.
 - The Return Program Counter (RPC) contains the address of the instruction that follows a jump to the subroutine.
 - The Start Program Counter (SPC) contains the address of the first instruction of the current hardware loop.
 - End Program Counter (EPC) contains the address of the last instruction of the current hardware loop.
- The other core registers are the general purpose registers (GREGn) and the flags.
 - The general purpose registers can be used to hold data and addresses. They can be loaded with immediate values (for example, 8-bit data that are encoded in the instruction), results of calculations that were performed with the ALU, 32-bit data that comes from the memory or peripherals via the Data Memory Bus (DMBUS), 32-bit data that comes from the DMAs via the Functional Units Bus (FUBUS) or another general purpose register. Their content can be the operands of the ALU, the data to send on either bus (DMBUS or FUBUS), or a pointer to memory (DMBUS address).
 - The general register 0 (GREG0) is also the hardware loop counter. In hardware loops, it cannot be used for any other purpose. This register uses a dedicated decrement unit (DECR) shown in [Figure 7-3](#).
 - The flags reflect the status of operations:
 - SF and DF are set when the last load or store on either bus (FUBUS or DMBUS) received an error response.
 - LM is set when the core is executing instructions inside a hardware loop.
 - T is set when the ALU operation result was 0 or the loop counter reaches 0 (the latter is preponderant when an ALU operation is the last instruction of a hardware loop).
- The ALU has two operands: any general register and either a second general register or an immediate value. The result is always stored into the first general register. A NOP function can be utilized by moving a register's contents into itself (For example, the instruction: mov R0,R0).
- The 16-bit instructions are fetched via the instruction bus (IBUS) whose address is driven by the PC. The SDMA RAM and ROM are visible to the core as 16-bit devices through this interface.
- The memory (RAM and ROM), memory mapped registers, and external peripherals are accessed via the DMBUS. The address is always taken from a general register whose content is added to a 5-bit immediate value. This is the only available

addressing mode. The DMBUS is a 32-bit data bus. Except for the peripherals that are external to the SDMA, the address accuracy is the 32-bit word (for example, adding 1 to an address points to the next word, not the next byte).

- The functional units are accessed via the FUBUS connection. The data is exchanged with any general register, but the address (which in fact is the instruction and the selector of the functional unit) comes from an 8-bit field of the corresponding load or store.

7.2.3.1.2 Program Control Unit (PCU)

This part of the SDMA core is dedicated to the control of the RISC engine, as implied by the instructions that are executed. Its behavior is determined by the instruction type and the inputs of the SDMA.

It contains the PC, RPC, SPC, and EPC registers that are described in [SDMA Core Structure](#).

7.2.3.1.2.1 Instruction Types

The state sequence and the delay of execution vary according to the type of the instruction. There are six possible categories of instructions, as follows:

1. Standard: Most of the instructions belong to this category, and always last 1 cycle.
2. ldf/stf: These are respectively the load and store instructions that access the functional units. They last 1+n cycles where n is the number of wait-states of the targeted functional unit.
3. ld/st: These are the load and store instructions that access the memory and peripherals. They last 1+n cycles where n is the number of wait-states of the targeted device (1 for the ROM, RAM, and memory mapped registers, 1 + the external peripheral wait-states). These instructions always last at least two cycles, but the core is able to handle them in one cycle. The first wait-state is inserted outside the core.
4. Branch: These are all the instructions that cause the Program Counter to point to another instruction other than the following one (for example, one that breaks the sequential flow). There are the absolute jumps, the conditional branches, the jump to the sub-routines, and the return from the sub-routine.
5. Loop, Modified Load or Store: The hardware loop instruction modifies the potential behavior of any load or store inside the loop (for example, when the LM flag is set). A jump may be implied after any such load or store if it received an error. The error causes an early exit of the loop, which means a jump to the instruction that follows the one that is pointed to by EPC. An additional cycle is required by the PCU to perform the jump (+1 to the ld/st/ldf/stf original execution delay). Although there is

usually an implicit jump after the last instruction of the loop when the PC goes back to SPC, this is performed at no cycle cost.

6. Done: The done, yield, or yieldge instructions are used to control channel switching. When no channel switching is performed, these instructions last a single cycle. When there is a change of channel or context switch, the delay is variable and depends on many factors (as detailed in [Context Switching](#)).

7.2.3.1.2.2 PCU States

The PCU state is visible through outputs of the SDMA (see [Real-Time Debug Outputs](#)) or the OnCE status register(see [OnCE Status Register \(OSTAT\)](#)).

The PCU state is a four-bit field that can take the values shown in the following table. [Figure 7-4](#) shows the possible state transitions and the corresponding conditions.

Table 7-6. PCU States

Value	State	Description
0	Program	This is the usual instruction cycle.
1	Data	This state is inserted when there are wait-states during a load or a store on the data bus (ld/st type).
2	Change of Flow	This is the second cycle of any instruction that breaks the sequence of instructions (branch and done types). This state lasts only a single cycle; it is always followed by the Program state.
3	Error in Loop	This state is used when an error causes a hardware loop exit (loop-modified load or store type). This state only lasts a single cycle; it is always followed by the Program state.
4	Debug	The SDMA is stopped in debug mode.
5	Functional Unit	This state is inserted when there are wait-states during a load or a store on the functional units bus (ldf/stf type).
6	Sleep	No script is running: The core is idle after saving the last channel context.
7	Save	The context switch FSM is saving the current channel.
8	Program in Sleep	Same as Program except there is no associated channel, this state is used when instructions are executed after entering debug mode, whereas the core was in either Sleep mode.
9	Data in Sleep	This is the same as Data except there is no associated channel.
10	Change of Flow in Sleep	This is the same as Change of Flow except there is no associated channel. This state only lasts a single cycle, and is always followed by the Program in Sleep state.
11	Error in Loop in Sleep	This is the same as Error in Loop except there is no associated. channel. This state only lasts a single cycle, and is always followed by the Program in Sleep state.
12	Debug in Sleep	This is the same as Debug except the core was put in debug mode when no channel was active.
13	Functional Unit in Sleep	This is the same as Functional Unit except there is no associated channel.

Table continues on the next page...

Table 7-6. PCU States (continued)

Value	State	Description
14	Sleep after Reset	This shows that no script is running, and the core is idle after a reset. When a channel becomes active, no context is restored but the core starts its boot program located at address 0 (or the address available in register in Channel 0 Boot Address (SDMAARM_CHN0ADDR)).
15	Restore	The context switch FSM is restoring the next channel context.

7.2.3.1.3 SDMA Core Memory

The SDMA has two memory spaces: one for the instructions and one for the data. As both spaces share the same resources (ROM and RAM devices), the system bus manages possible conflicts when the core accesses the same resource for both an instruction read and a data read or write.

Program and data memory is further described in [Address Space](#).

Instructions of 16-bit width are stored in 32-bit wide devices and can be accessed as data. The mapping is Big Endian: an even instruction address (terminated by 0) accesses the most significant part of the 32-bit data (bits [31:16]), and an odd instruction address (terminated by 1) accesses the least significant part of the 32-bit data (bits [15:0]). Instructions can be fetched out of internal ROM or RAM.

Data can be read from ROM, RAM, memory mapped registers, and external peripherals, and written to the same devices (except the ROM).

The ROM contains bootload scripts, channel scripts, and common subroutines which may be referenced by channel scripts elsewhere in the ROM or RAM.

The RAM is divided into a context area and a code space area which may be used to store channel scripts. The RAM contains undefined values after a hardware reset. Channel scripts and initial context values are downloaded into RAM using channel 0 which is reserved for bootload functions.

7.2.3.2 Scheduler

All channel scheduling hardware is included in the Scheduler.

7.2.3.2.1 Primary Functions

The scheduler is a hardware-based design used to coordinate the timely execution of 32 virtual DMA channels by the SDMA core on the basis of channel status and priority.

The scheduler performs the following functions:

- Monitors, detects, and registers the occurrence of any one of the 48 DMA requests
- Links a specific request to a channel or group of channels (channel mapping)
- Ignores requests that are not mapped to a previously configured channel
- Maintains a list of all the channels that are requesting service
- Assigns a pre-programmed priority level (1 of 7) to every channel requesting service
- Detects and flags overrun/underrun conditions

7.2.3.2.2 Channels and DMA Requests

7.2.3.2.2.1 Channels

A Virtual Channel (hereafter simply called a channel) manages a flow of data through the SDMA. Flows are typically unidirectional.

The SDMA can have up to 32 simultaneously operating channels, numbered from 0 to 31. Channel 0 is usually dedicated to control the SDMA script downloading. All the channels can be assigned by the Arm platform software.

7.2.3.2.2.2 DMA Requests

A DMA request is caused by externally (for example, external to the SDMA) controlled conditions (for example, UART receive FIFO reaches a threshold). The SDMA currently supports up to 48 DMA requests.

7.2.3.2.2.3 Mapping from DMA Requests to Channels and Priorities

A channel can stall waiting on a single DMA request. A single DMA request can awake more than one channel (in fact, any request can awake any combination of channels).

The mapping between DMA requests and channels is program-controlled. There is a storage element assigned for each of the 48 requests that contains a bitmap table of the channels that are awakened by the event.

Every channel also has a three-bit register that indicates its priority.

7.2.3.2.3 Scheduler Functional Description

[Scheduler Overview](#) describes the behavior of the SDMA scheduler—from the channel enabling conditions to the highest priority pending channel selection.

7.2.3.2.3.1 Scheduler Overview

The scheduler algorithm is built in hardware. It is provided with possibilities for the Arm platform to control its behavior.

The scheduler processes incoming DMA requests, maps detected requests to 0, one, or several channels, maintains a list of channels that are requesting service (pending channels), identifies the top priority and its associated channel, and selects the next active channel when the current channel yields.

The following figure shows a functional overview.

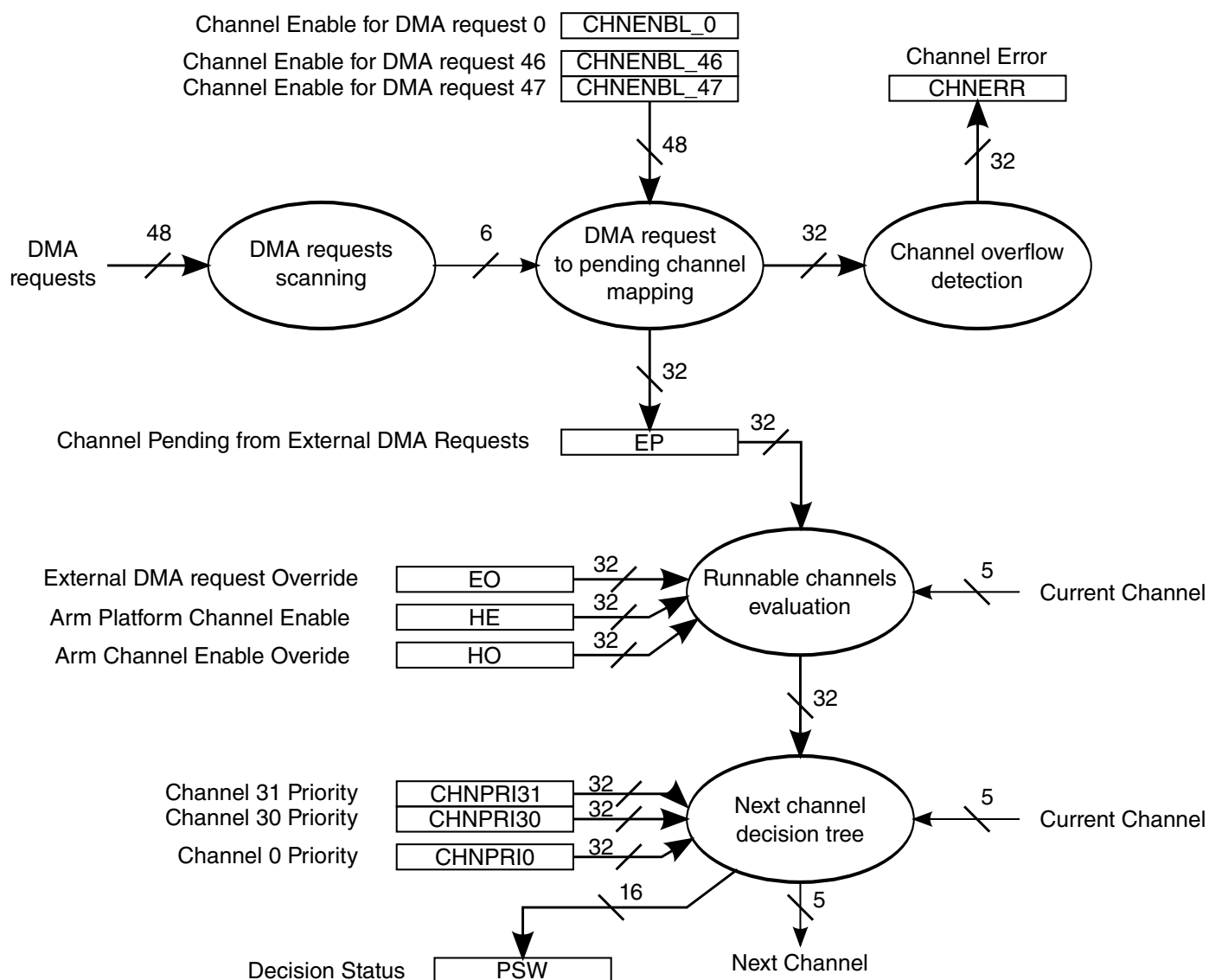


Figure 7-5. SDMA Hardware Scheduler

7.2.3.2.3.2 DMA Requests Scanning

The scheduler contains a 48-bit edge detection device that detects the rising edge of every DMA request and transmits the request number to the next stage.

The DMA requests are assumed to be generated on the same reference clock as the SDMA core clock; they are detected as soon as the signal goes from a 1-to-n-cycles low state to a 1-to-m-cycles high state.

This system is able to detect single-cycle pulses as well as level-based DMA requests such as a FIFO threshold crossing. In this case, the SDMA provides a memory mapped register that can be used by the channel script to monitor the DMA requests lines, and thus determines whether the data transfer is done or not done, and then continues with the transfer or closes the channel.

When several DMA requests are detected at the same time, they are forwarded to the next scheduler stage at the rate of one request per cycle. No request is lost.

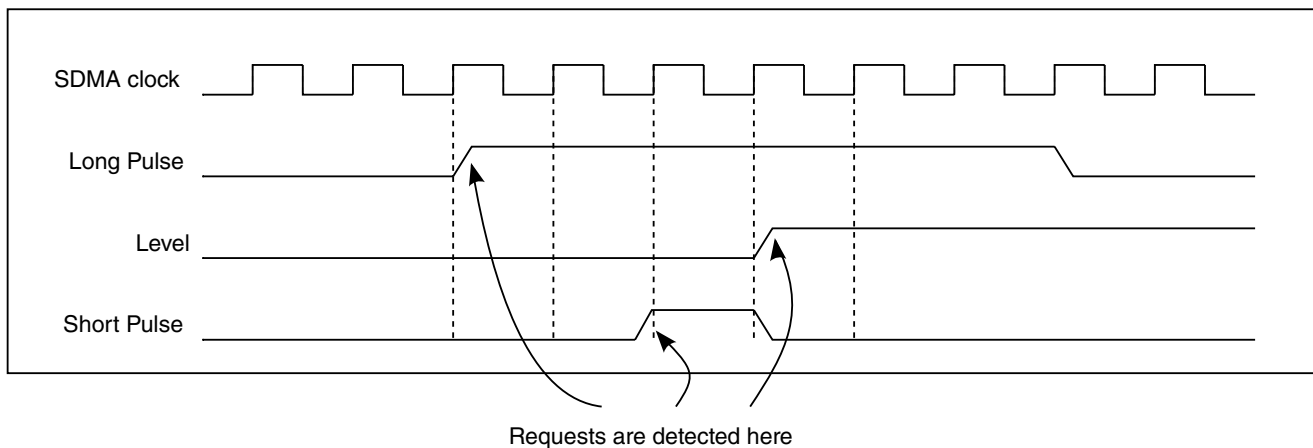


Figure 7-6. Examples of Valid DMA Requests

The DMA request inputs are connected to various sources that depend on the SoC. The exact list of DMA request inputs and their associated number is available in each respective project-specific chapter.

7.2.3.2.3.3 Mapping DMA Requests to Pending Channels

Whenever a DMA request is detected by the first stage, its number is used in the second stage to determine the channels that have to be activated.

This is performed with an array of 48 registers that are 32 bits wide: There are 48 Channel Enable Registers (CHNENBLn), one register per DMA request. The DMA request number selects the Channel Enable Registers, and every bit of this 32-bit register indicates that the corresponding channel must be activated when it is a 1.

This information is passed on the EP register. For every bit of the Channel Enable Register that is set, the corresponding bit of the EP register is also set, and the remaining bits of EP are left unchanged. The transformation of EP is summarized by the following equation:

$$EP = EP \text{ or } CHNENBLn$$

The EP register is used to know which channels require service because they received a DMA request.

Typical contents of the CHNENBLn registers are all 0s, except for a single bit set. For example, a DMA request triggers one channel, but all 0s or several 1s are possible. One DMA request could activate several channels, and the channel execution sequence can be controlled by the channel priorities and numbers, as explained in the next sections. The following table illustrates an example configuration.

NOTE

From the table, the DMA request 0 is programmed to simultaneously trigger channels 0, 1, and 31. Also, DMA requests 30-47 are not used in this example. The remaining channels 2 to 30, are configured to be triggered by DMA requests 29 to 1, respectively.

Table 7-7. Channel Enable RAM Programming Example

DMA Request Number	Channel																															
	31																															0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table continues on the next page...

Table 7-7. Channel Enable RAM Programming Example (continued)

DMA Request Number	Channel																															
	3																															0
7	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
13	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
14	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
17	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
18	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
19	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
20	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
21	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
22	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
23	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
24	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
25	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
26	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
27	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
28	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
29	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
30	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
31	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
32	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
33	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
34	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
35	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
36	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
37	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
38	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
39	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table continues on the next page...

Table 7-7. Channel Enable RAM Programming Example (continued)

DMA Request Number	Channel																															
	31																															0
40	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
41	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
42	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
43	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
44	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
45	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
46	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
47	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

7.2.3.2.3.4 Channel Overflow

A channel overflow occurs when a DMA request requires service from channel n by setting bit n of the register EP, but this bit is already set, meaning channel n is already pending. This can come from an overrun/underrun condition.

This detection is possible only when the DMA requests are pulses, because a level-based DMA request stays high until it is serviced, even though an underrun or overrun condition occurs, thus preventing another edge detection of the DMA request.

The channel overflow information is saved in the 32-bit CHNERR register (1 bit per channel). You can configure the SDMA to trigger an interrupt to the Arm platform when there are 1s in CHNERR. Every bit of CHNERR is masked with the corresponding bit of INTRMASK and if it gives a 1, the corresponding bit of INTR is set, triggering the interrupt.

7.2.3.2.3.5 Runnable Channels Evaluation

The EP register is used in conjunction with several other 32-bit registers to determine the channels that are runnable.

Registers EO, DO, HO and HE, are controlled by the Arm platform. EP is controlled by the DMA requests and their mapping to channels.

Several channels may be runnable at any given time. The i^{th} channel is runnable if (and only if) the condition below is true:

$(\text{HE}[i] \text{ or } \text{HO}[i]) \text{ and } (\text{DO}[i]) \text{ and } (\text{EP}[i] \text{ or } \text{EO}[i])$

After reset, the $\text{HE}[i]$, $\text{HO}[i]$, $\text{EP}[i]$, and $\text{EO}[i]$ bits are all cleared whereas the $\text{DO}[i]$ bits are all set. The functions associated with DO are not available for this device. When $\text{DO}[i]$ is set, the scheduler condition becomes:

$(\text{HE}[i] \text{ or } \text{HO}[i]) \text{ and } (\text{EP}[i] \text{ or } \text{EO}[i])$

The registers in these equations are controlled as follows:

- Arm platform (host) channel enable flag $\text{HE}[i]$ may be set or cleared by the Arm platform with the HSTART and STOP_STAT registers. It can also be cleared by the i^{th} channel script.

Typical usage is for the Arm platform to set this flag to activate the channel. The flag is cleared by the SDMA core when the transfer is done.

- Externally triggered channel pending flag $\text{EP}[i]$ is set by the scheduler when the channel was activated by a DMA request. It can be cleared by the i^{th} channel script.
- The Arm platform channel override flag $\text{HO}[i]$ may be set or cleared by the Arm platform. When set, it enables the i^{th} channel to run without the involvement of the Arm platform.

Typical usage is for the Arm platform to set this flag for channels that do not need Arm platform supervision such as channels that are controlled by DMA request events (EP).

- DO should always be set to 1 so that the runnable channel evaluation considers only HO , HE , EP , and EO .
- Externally triggered channel override flag $\text{EO}[i]$ may be set or cleared by the Arm platform. When set, it prevents the i^{th} channel from stopping and stalling on incoming peripheral DMA requests. This is the case when the channel is not handling data transfers with peripherals (for example, a memory to memory transfer).

The SDMA can clear the $\text{HE}[i]$, and $\text{EP}[i]$ bits by means of a done or notify instruction. The done instruction causes a reschedule; thus, enabling another channel to preempt the current one, while the notify instruction does not. The done and notify instructions can clear either $\text{HE}[i]$ or $\text{EP}[i]$ (never more than one at a time).

Table 7-8. Runnable Channel Selection Control

Register	Set by	Cleared By
HO	Write to HOSTOVR register	Write to HOSTOVR register

Table continues on the next page...

Table 7-8. Runnable Channel Selection Control (continued)

Register	Set by	Cleared By
HE	Write to HSTART register	Write to STOP_STAT register or by the channel script with the done or notify instructions.
DO	Write to DSPOVR register	Write to DSPOVR register
EO	Write to EVTOVER register	Write to EVTOVER register
EP	Set by external DMA request event input.	By the channel script with the done or notify instructions

7.2.3.2.3.6 Next Channel Decision Tree

The next channel number is computed from the runnable channels list, the current channel number, and their respective priorities.

It is re-evaluated every cycle, but is only used when the current channel yields or terminates by executing a yield, yieldge, or done instruction.

The decision tree is based on the selection of the runnable channel that has the highest priority.

The highest priority channel is selected according to the following rules:

- Runnable channels are sorted by priority.
- If one of the channels with the highest priority had been preempted by a channel with a higher priority, but did not want to yield to a channel of the same priority (for example, it executed a yield, not a yieldge), it is elected as the next channel.
- The channels that belong to the highest priority group are sorted by their number and the channel that has the highest number in this group becomes the next channel. For example, if priorities are the same, channel 31 will be selected before channel 30.

When the current channel requires a reschedule with a yield(ge) or a done instruction, the context switch decision is based on the instruction parameter, the current channel number and priority, and the next channel number and priority. The possible cases are all listed in the following table. The grayed cells correspond to unusual cases that should not occur with a typical usage of the SDMA.

Table 7-9. Channel Switching Decision with a yield, yield(ge), or done

Instruction	Current Channel	Next Channel	Priorities Comparison	New Running Channel/Comments
yield (done 0)	Runnable	Not runnable	none	Current
	Runnable	Runnable	Current > Next	Current
			Current = Next	Current
			Current < Next	Next ¹

Table continues on the next page...

Table 7-9. Channel Switching Decision with a yield, yield(ge), or done (continued)

Instruction	Current Channel	Next Channel	Priorities Comparison	New Running Channel/Comments
	Not runnable	Not runnable	none	none ² (occurs when the channel was disabled by the Arm platform)
	Not runnable	Runnable	none	Next ¹ (occurs when the channel was disabled by the Arm platform)
yieldge (done 1)	Runnable	Not runnable	none	Current
	Runnable	Runnable	Current > Next	Current
			Current = Next	Next ¹
			Current < Next	Next ¹
	Not runnable	Not runnable	none	none ² (occurs when the channel was disabled by the Arm platform)
	Not runnable	Runnable	none	Next ¹ (occurs when the channel was disabled by the Arm platform)
done (done>1)	Not runnable	Not runnable	none	none ²
	Runnable	Not runnable	none	Current ³ (occurs when the done instruction does not disable the channel runnable condition)
	Not runnable	Runnable	none	Next ¹
	Runnable	Runnable	none	Current ³ (occurs when the done instruction does not disable the channel runnable condition)

1. Current channel script execution is stopped, its context is saved; the next channel context is restored and its script execution resumes
2. Current channel context is saved and SDMA enters IDLE mode
3. Current channel context is saved, then restored, and the current channel script resumes execution

Finally, when the SDMA is in IDLE mode and a runnable channel is elected as the next channel, its context is immediately restored and the script execution resumes.

The *combinatorial-decision* tree supports dynamic modifications of the EP, EO, HE, HO, and DO flags as well as dynamic modifications of the channel priorities. The propagation times are detailed in [Scheduler Pipeline Timing Diagram](#).

The decision tree status is available in the PSW register, which is continuously updated. It contains the next channel priority, the next channel number, the current channel priority, and the current channel number. When a priority is read as 0, it means the channel is not runnable.

A few examples of decisions are presented below:

- Channel 31 is running with priority 5, channels 13 and 24 are pending with the same priority 5; channel 24 is eligible as the next channel since $24 > 13$.
- Channel 31 is running with priority 7, channels 13 and 24 are pending with priority 5; channel 31 is the next channel because its priority is greater than the other pending channels.
- Channels 7, 23, and 29 are pending with the same priority. Channel 7 is active and runs a yieldge; it is preempted by channel 29. After a period of time, channel 29 runs a yieldge, it is then preempted by channel 23 that is the selected channel since channel 29 is the current channel. Later, channel 23 runs a yieldge and is preempted by channel 29. Channels 23 and 29 will go on switching after every yieldge until one of them terminates. It is only at that point that channel 7 becomes eligible again.
- Channel 11 is running with priority 3, and channel 15 is pending with priority 4. When the channel 31 script executes a yield instruction, it gets preempted by channel 15; then channels 6 and 18 with priority 3 become pending. Because channel 11 was preempted after executing a yield and there is no pending channel with a strictly greater priority, it is eligible as the next channel (although its number $11 < 18$).

7.2.3.2.3.7 Scheduler State Diagram

The [Figure 7-7](#) summarizes the behavior of the SDMA scheduler with details about the exact mechanism of the priority decision tree. It is important to understand the scheduler is a hardwired pipeline, which means all the stages are performed simultaneously every cycle, but a change on any given stage is reflected on the next stage after the delays presented in [Scheduler Pipeline Timing Diagram](#).

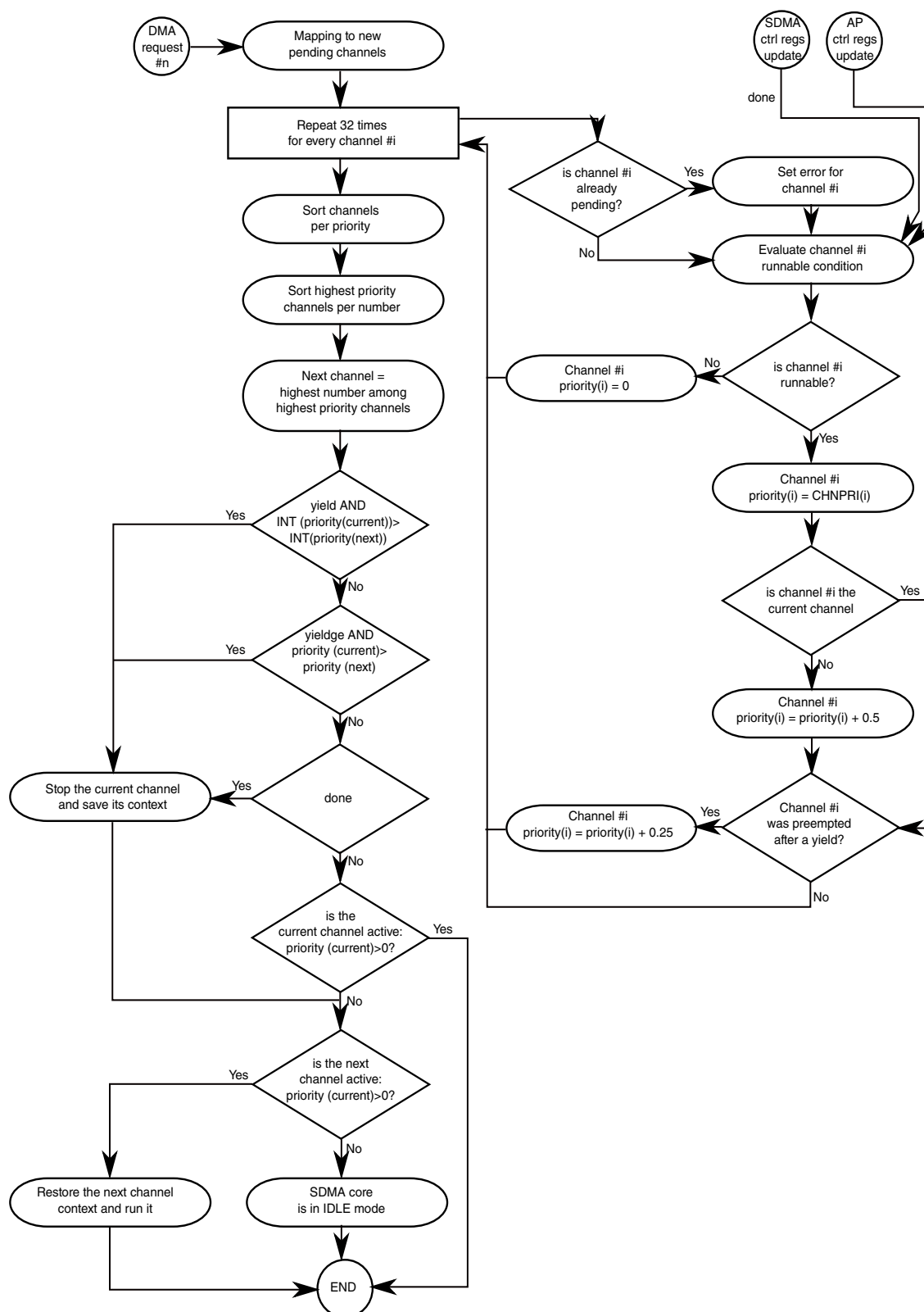


Figure 7-7. Scheduler State Diagram

7.2.3.2.3.8 Scheduler Pipeline Timing Diagram

The SDMA scheduler process of DMA-request and control-register modifications is not immediate.

The figure below shows the exact delays of all the tasks. The reference clock is the SDMA core clock.

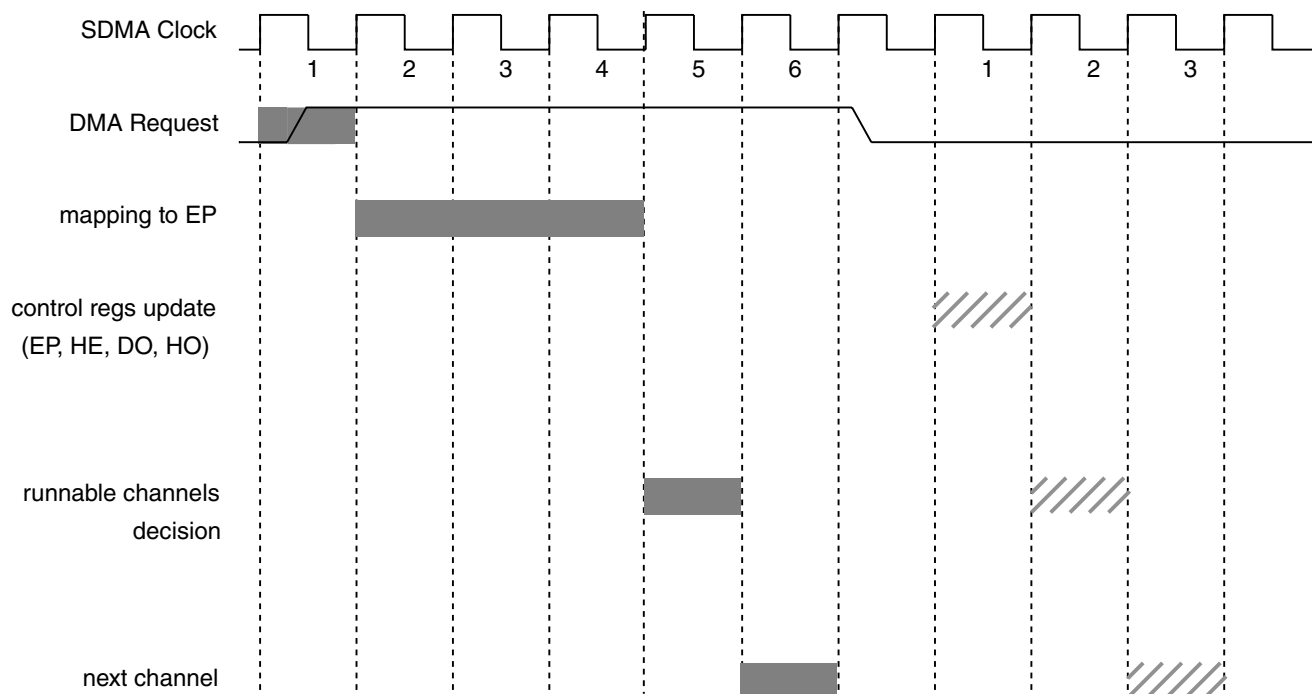


Figure 7-8. Scheduler Timing Diagram

Two numbers can be inferred from this timing diagram. First, it takes six SDMA core clock cycles to update the next channel from a DMA request. Second, it takes three SDMA core clock cycles to update the next channel from a direct modification of the condition registers (EP, DO, HE, or HO) by any processor. The processors that can modify these bits include SDMA with a done instruction or the Arm platform with a write access through the corresponding control port on their respective peripheral bus).

7.2.3.2.3.9 Channel-DMA Request Mapping

The 48 DMA request inputs to the SDMA scheduler are listed in project-specific chapters. Refer to the respective chapters for this information.

7.2.3.2.3.10 Examples: How to Start a Channel

A channel can be started when the following equation is true for channel i :

$(HE[i] \text{ or } HO[i]) \text{ and } (DO[i]) \text{ and } (EP[i] \text{ or } EO[i])$

Once this equation is true, the scheduler can start this channel according to the priority of all pending channels. Several examples of configuration are listed below:

1. To start a channel triggered by Arm platform software:
 - Initially, configure $HO[i]=0$, $DO[i]=1$, and $EO[i]=1$ using registers indicated in [Table 7-8](#).
 - Arm platform software triggers the channel by writing to the HSTART register to set $HE[i]=1$, thereby setting the above equation true.
2. To start a channel triggered by DMA request event.
 - Initially, configure $HO[i]=1$, $DO[i]=1$, and $EO[i]=0$ using registers indicated in [Table 7-8](#).
 - The DMA request is asserted to trigger the channel by setting $EP[i]=1$, which makes the above equation true.

7.2.3.2.4 Context Switching

On execution of a done or yield(*ge*) instruction, the current channel may be changed either because it has finished (which necessarily happens when the done instruction is executed), or it was preempted by a higher priority channel (which is possible but not systematic when the yield(*ge*) is executed).

Upon a channel change the SDMA goes through a context switch procedure.

When the current channel yields or ends, the context for that channel is saved into the context RAM locations for that channel. When the next channel starts running, its context is first restored from RAM.

Since context RAM is not yet initialized by reset, there will be no context restore at the beginning of the first channel (bootload channel) run after reset. It is expected that the bootload channel will be used to initialize the context for all other channels. When the bootload channel finishes running or yields, SDMA will enter its SAVE state and save that channel's context into RAM. Then, if the bootload channel is called again later, the context will be restored from RAM when the channel starts again.

The context structure for each channel is defined in [Context Switching-Programming](#) and [Table 7-14](#). There will be one context area reserved for each channel. When a channel ends or yields, the SDMA core registers are automatically saved into the context RAM and later restored from the context RAM when the channel is next run. The total RAM

space reserved for 32-channel contexts is either 3K or 4K depending on whether the SMSZ bit is set in the CHN0ADDR register, which enables an additional 8 words of scratch RAM for each context.

7.2.3.2.4.1 Context Switch Modes

The exact procedure to save the context of the old channel, and to restore the context of the new channel depends on the context switch mode selected by the Arm platform in the CONFIG control register.

The following are the context switch modes:

- By default, the "dynamic" context switch is set. This mode provides the most efficient context switch for an average of eight cycles to stop the current channel, save its context, restore the next channel context, and resume its execution. It consists of saving modified registers of the current channel in the background (for example, during the channel execution)-which leaves very few registers to save when the switch is decided-resuming execution of the next channel as soon as possible (for example, when the minimal set of registers is restored), and continuing the restore phase during this execution.
- In "dynamic with no loop" mode, the same principle is followed except the modified registers are only saved in the background when the loop flag is not set. This mode offers almost the same effectiveness as the previous one, but it prevents the system from accessing the RAM during loops to save power. This is the recommended mode for an efficient context-switch when the loop bodies are short.
- In "dynamic power" mode, no background saving is performed, which reduces power consumption to the minimum. The modified registers are only saved when the context switch starts. The restore phase is the same as before. This is the mode that achieves the optimal power consumption at the cost of a slower context-switch.
- In a "static" context switch, all the registers are saved when a context switch is decided, and all the registers are restored before starting the execution of the new channel. This mode enables a predictable behavior of the context switch since all the registers are restored prior to the channel start and all registers are saved after the channel termination.

NOTE

Static context mode should be used for the first channel called after reset to ensure that the all context RAM for that channel is initialized during the context SAVE phase when the channel is done or yields. Subsequent calls to the same channel or different channels may use any of the dynamic context modes. This will ensure that all context locations for the bootloader

channel are initialized, and prevent undefined values in context RAM from being loaded during the context restore if the channel is re-started later.

7.2.3.2.4.2 Context Switch Procedure

The Program Control Unit goes into the *save* state, the current context is spilled into memory, and the next channel context is restored according to the context-switch mode that was selected by the Arm platform.

The context switch procedure is as follows:

1. Load the current context's spill base address.
2. Spill the modified registers of the current channel to memory according to the selected context switch mode while the channel is running.

On a done or yield(ge) that causes the channel preemption, the PCU goes into the *save* state. In *static* mode, all the registers are saved; whereas, in either *dynamic* mode, the registers that were modified but not yet saved are then saved, and the PCU registers and flags are finally saved.

3. Put the SDMA core into *sleep* and wait for new channels to be serviced. This step is skipped if there are pending channels when the current channel is saved.

As soon as there is at least one pending channel, the PCU goes into its *restore* state to restore the context of the channel that was elected by the scheduler.

Once a channel is elected, it remains the current channel until its script requests a rescheduling operation with a done or yield(ge) instruction. That means the current channel cannot be modified by the Arm platform, even if it is no more runnable or if its priority is modified.

The Arm platform can however force a reschedule by writing the corresponding bit in the CONFIG register, which has the same effect as if the script had executed a done instruction. That feature should only be used to stop the SDMA in emergency cases.

4. Load the context base-address of the new channel.

In "static" mode, all the registers are restored. In either "dynamic" modes, only the PCU registers are restored.

The new channel is running. In "static" mode, no more activity regarding context restoring or saving is performed. In either "dynamic" modes, the registers are restored in the background every time an access to the context RAM is possible, and

priority is given to restoring the registers that are required by the next instruction to be executed. When a register has not been restored and the next instruction needs it, this instruction gets stalled until the register was restored.

In "dynamic" and "dynamic with no loop" modes, background saving of dirty registers is performed every time an access to the context RAM is possible and allowed by the context switch mode.

NOTE

The contents of a channel context space in the context RAM depends on the selected context switch mode. In "dynamic" and "dynamic with no loop" modes, the contents of the context RAM tend to match the contents of the SDMA registers (except for the PCU registers and flags that are never saved in the background). In "dynamic power" and "static" modes, the contents of the context RAM remain unchanged until the channel terminates with a done or gets preempted.

7.2.3.2.4.3 Context Map in Memory

Refer to [Context Switching-Programming](#).

7.2.3.3 Functional Units

The functional units are small systems that are used by the SDMA core to handle data transfers between the core and a bus domain external to the SDMA.

The SDMA core is able to control and exchange data with these systems by sending instructions and reading or writing data from/to the functional units' registers via the FUBUS. This is done with the ldf and stf instructions.

The following sections provide introductions to the available functional units. [Functional Units Programming Model](#) provides descriptions the functional units' behaviors.

7.2.3.3.1 Burst DMA Unit

The burst DMA unit enables the SDMA core to perform data transfers to and from the Arm platform memory.

It is optimized for accessing SDRAM-like devices. It does not provide control to assign a privilege level to the DMA access. The burst DMA unit provides the SDMA with means to do the following:

- Perform up to 8-beat read and write bursts to the Arm platform memory, which optimizes throughput when accessing SDRAM-type devices because of an internal, 36-byte FIFO
- Access the Arm platform memory at once or twice the SDMA core frequency
- Copy data from one Arm platform memory location to another Arm platform memory location at the Arm platform bus speed, which provides a very high throughput
- Control the method for addressing the Arm platform memory (automatic increment of addresses or frozen addresses-the former aimed at accessing RAM-like memory and the latter aimed at accessing single-address FIFOs)
- Enable or disable automatic prefetch when reading data from the Arm platform memory. When the prefetch mode is selected, the burst DMA automatically triggers external bursts to fill its FIFO without waiting for the SDMA core to request the corresponding data, greatly improving throughput.
- Rely on the DMA to automatically flush its FIFO content when there is enough data to generate an 8-beat burst to the Arm platform memory. Or, it forces a flush when a data transfer must terminate.
- In the former case, the SDMA core may only be stalled when it tries writing data and there is not enough room left in the FIFO. In the latter case, the core is stalled until the data is effectively written to the Arm platform memory.

In automatic flush mode, the core receives an acknowledge that does not reflect the actual error status when the data is effectively written into the Arm platform memory. This error status is retrieved by a later access to the burst DMA.

Terminating a write data transfer with a forced flush command guarantees that any bus error to the Arm platform memory is caught.

- Handle address alignment issues between the Arm platform memory map and the SDMA core data. This enables the core to read or write 32-bit data from the burst DMA, whereas the corresponding Arm platform address is not 32-bit aligned. This drastically improves the SDMA scripts' efficiency since the same loop that transfers 32 bits at a time can be used regardless of the start and end addresses in the Arm platform memory space.

This unit structure and registers are described in [Burst DMA Structure](#) and [Burst DMA Registers](#).

7.2.3.3.1.1 Burst DMA Structure

The burst DMA is essentially made up of a 36-byte FIFO, address registers, and a controlling state-machine. The 36-byte FIFO enables eight-word buffering with address alignment, and the state-machine manages clock adaptation when required.

The burst DMA is depicted in the figure below.

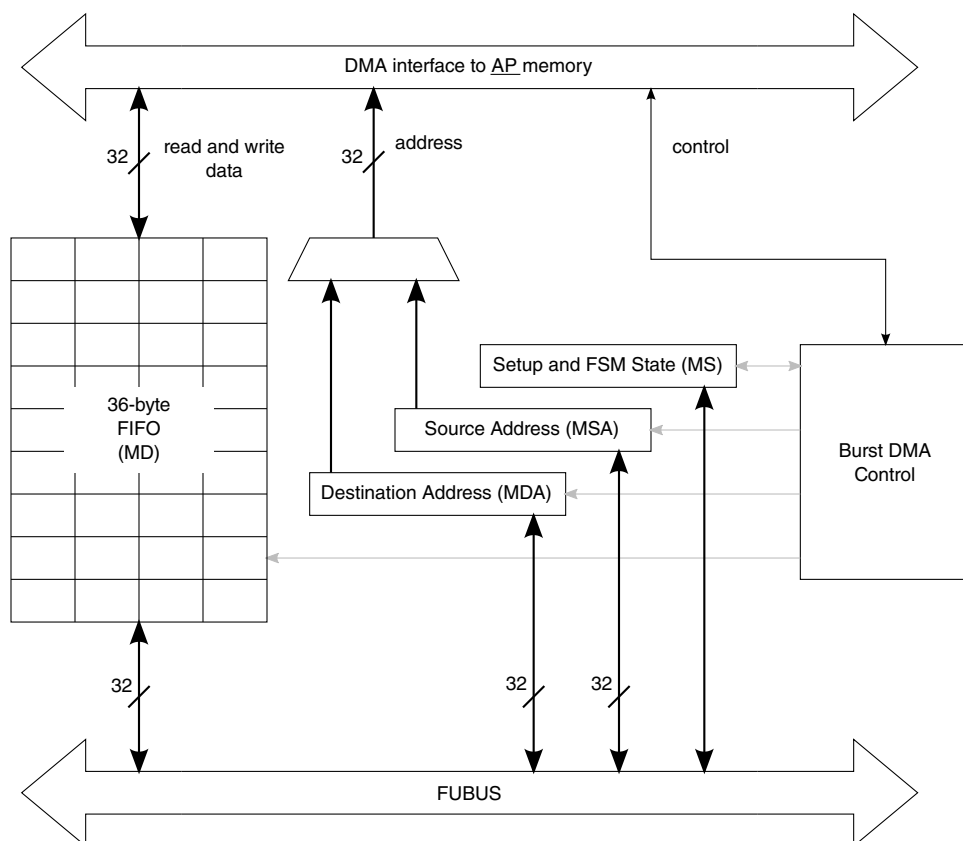


Figure 7-9. Burst DMA Structure

7.2.3.3.1.2 Burst DMA Registers

There are four registers, as follows, that may be accessed from the SDMA core:

- **MSA (Memory Source Address)** - Holds the source byte address in the Arm platform memory map for reading data from this location. This register is automatically modified every time the core reads new data from the FIFO.
- **MDA (Memory Destination Address)** - Holds the destination byte address in the Arm platform memory map for writing data to this location. This register is automatically modified every time the core writes new data into the FIFO.
- **MD (Memory Data)** - Labels the 36-byte FIFO access point: Reading a byte, halfword, or word from MD respectively retrieves the first 1, 2, or 4 bytes of the

FIFO (for example, the bytes that were stored first by the DMA state-machine when transferring data from the Arm platform memory).

- When the FIFO does not hold as many bytes as required by the SDMA core, the core is stalled until the missing bytes are read from the Arm platform memory. In the case of prefetch mode, the DMA controller decides when it should start a burst to Arm platform memory in order to reduce the risk to not have the required data for the future accesses of the core. When there is no prefetching, a burst is triggered when the required data is not available in the FIFO.

Writing a byte, halfword, or word to MD stores 1, 2, or 4 bytes, respectively, at the end of the FIFO (for example, these bytes are transmitted to the Arm platform memory after all the other bytes that were previously stored in the FIFO). When the FIFO does not have enough room left to hold the written data, the SDMA core is stalled until a sufficient amount of FIFO contents are flushed out to the Arm platform memory. Flushing is decided by the DMA controller when there are enough bytes in the FIFO to perform the largest allowed burst to Arm platform memory (the exact size depends on the burst start address and the AHB 1 Kbyte boundary rule).

However, the SDMA core has the ability to force the flushing operation at any time, for example, when at the end of the data transfer, prior to channel closure.

- MS (Memory Setup) - Contains the state of the burst DMA control, the two flags that define whether each address register is incremented after every access to the external memory, and another flag that is set when a bus error occurred.

7.2.3.3.1.3 Burst DMA Data Transfers

Three typical usages have been identified that involve the burst DMA: the data transfer startpoint, the endpoint, or both.

Every case requires a different procedure, as listed in the following sections:

7.2.3.3.1.3.1 Data Retrieval from the Arm platform Memory

The following steps retrieve data from Arm platform memory using the burst DMA unit:

- Set up the MS flags to reflect the mode for the source address (incremented or frozen according to the type of accessed device: memory or peripheral FIFO), then initialize the source address register itself (MSA).
- Read data from the FIFO using the *ldf MD* instruction as many times as needed. If an error occurred during the fetch from Arm platform memory, the DMA control tags the error status on the data and the SDMA core SF flag is set when reading this data from the FIFO.

7.2.3.3.1.3.2 Storing Data Into the Arm platform Memory

The following steps store data from Arm platform memory using the burst DMA unit:

- Set up the MS flags to reflect the mode for the destination address (incremented or frozen according to the type of accessed device: memory or peripheral FIFO), then initialize the destination address register itself (MDA).
- Store data into the FIFO using the *stf MD* instruction as many times as needed.
- When the transfer is finished and if the DMA worked in automatic flush mode, force the flush of the FIFO. This instruction is stalled until all the FIFO data is effectively sent to the Arm platform memory and the error status of the transfer is available in the DF flag.

7.2.3.3.1.3.3 Transferring Data Between Two Arm platform Memory Locations-Burst DMA Unit

The following steps copy data between two Arm platform memory locations using the burst DMA unit:

- Set up the MS flags to reflect the modes for the source and destination addresses (all the combinations are possible), then initialize the source address register (MSA) and the destination address register (MDA). Both addresses must be word-aligned.
- Use as many *stf MD* instructions with the *COPY* flag as needed. Every instruction triggers a burst read of a given number of words from the source address (this number is provided to the burst DMA via the SDMA core general purpose register, which is referenced in the *stf* instruction). Once all the data is loaded into the FIFO, the DMA empties it with a write burst of the same count to the destination address. The DMA acknowledges prior to instruction completion, which frees the SDMA core for other tasks at no delay cost.
- Once the transfer is done, there should be a final access to the burst DMA to check the error status.

7.2.3.3.2 Peripheral DMA Unit

The peripheral DMA unit is the second functional unit that connects the SDMA to the Arm platform memory.

Unlike the burst DMA, it does not support burst transfers and is optimized for accessing peripherals. It does not provide control to assign a privilege level to the DMA access. Its feature list comprises the following:

- Access to the Arm platform peripherals or memory at once or twice the SDMA core frequency

- Data copy from one Arm platform memory location to another Arm platform memory location at memory bus speed, improving throughput
- Control of the method for addressing the Arm platform memory (automatic increment or decrement of addresses or frozen addresses, the first ones aimed at accessing RAM-like memory and the last one aimed at accessing single-address FIFOs)
- Selectable automatic prefetch when reading data from the Arm platform memory. In prefetch mode, the peripheral DMA automatically fetches another data-without waiting for the SDMA core to request it-when its data register is empty, which improves the throughput
- Selectable automatic flush. In this mode, the SDMA core may only be stalled when it tries writing data and the previous write operation is not finished yet; whereas, in forced flush mode, the core is stalled until the data is effectively written to the Arm platform memory.
- In automatic flush mode, the core receives an acknowledge that does not reflect the actual error status when the data is effectively written into the Arm platform memory or the peripheral. This error status is retrieved by a later access to the peripheral DMA. Terminating a write data transfer with a forced flush command guarantees that any bus error to the Arm platform memory has been caught.

This unit structure and registers are described in [Peripheral DMA Structure](#) and [Peripheral DMA Registers](#).

7.2.3.3.2.1 Peripheral DMA Structure

The peripheral DMA is made up of a 32-bit data register, two address registers, and a controlling state-machine. The state-machine manages clock adaptation, when required.

It is shown in the following figure.

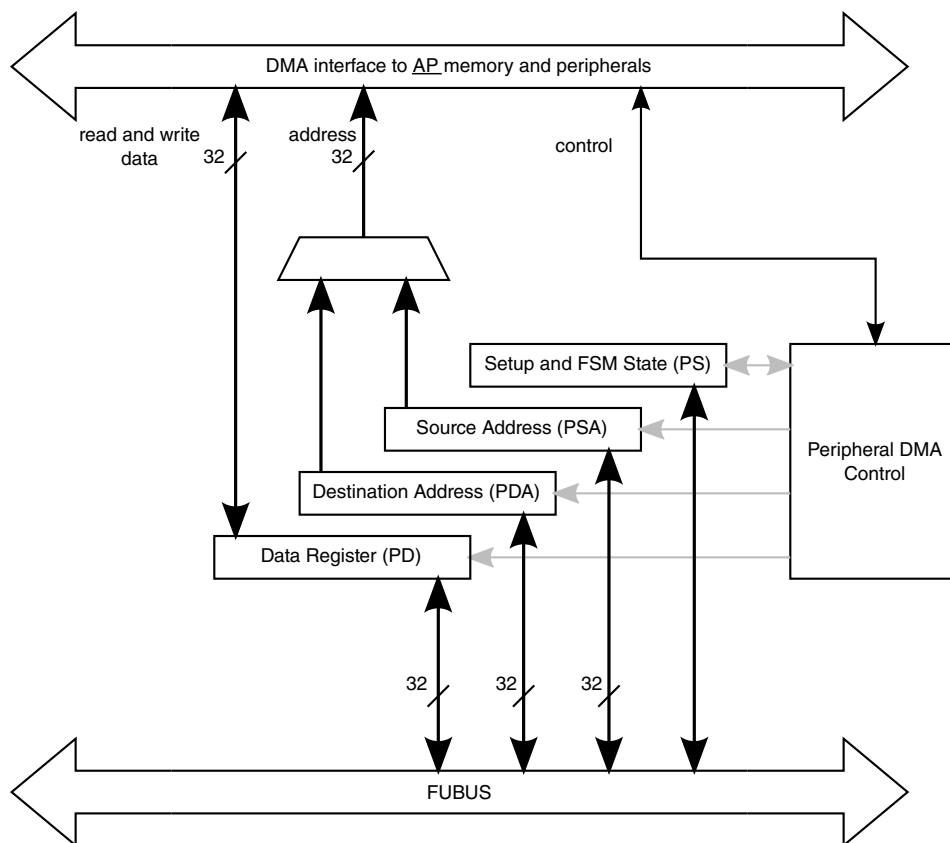


Figure 7-10. Peripheral DMA structure

7.2.3.3.2.2 Peripheral DMA Registers

According to [Figure 7-10](#), the peripheral DMA has four registers that may be read or written by the SDMA core:

- *PD (Peripheral Data)* is the DMA 32-bit data register.
- *PSA (Peripheral Source Address)* holds the source byte address in the Arm platform memory map for reading data from this location. This register is automatically modified every time the core reads a new data from PD.

- *PDA (Peripheral Destination Address)* holds the destination byte address in the Arm platform memory map for writing data to this location. This register is automatically modified every time the core writes a new data into PD.
- *PS (Peripheral Setup)* contains the state of the peripheral DMA control, two configuration fields that define the way address registers are modified after every data access, two additional configuration fields that define the data size to access the source and destination devices, and another field that contains the latest transfer error status.

7.2.3.3.2.3 Peripheral DMA Data Transfers

There are three typical usages that involve the peripheral DMA, whether it is the data transfer start-point, endpoint, or both.

Every case requires a different procedure, as described in [Data Retrieval from the Arm platform Memory or Peripheral](#), [Storing Data into the Arm platform Memory or Peripheral](#), and [Transferring Data Between Two Arm platform Memory Locations-Peripheral DMA Unit](#).

7.2.3.3.2.3.1 Data Retrieval from the Arm platform Memory or Peripheral

The following steps retrieve data from Arm platform memory using the peripheral DMA unit:

- Set up the PS fields to reflect the mode and data size for the source (incremented, decremented, or frozen address register; 8-bit, 16-bit, or 32-bit data transfers), then initialize the source address register itself (PSA) with an address that is aligned to the programmed data size.
- Read data from PD using the ldf PD instruction as many times as needed. If an error occurs during the fetch from the Arm platform memory or peripheral, the DMA control tags the error status on the data and the SDMA core SF flag is set when reading this data from PD.

7.2.3.3.2.3.2 Storing Data into the Arm platform Memory or Peripheral

The following steps store data to Arm platform memory using the peripheral DMA unit:

- Set up the PS fields to reflect the mode and data size for the destination (incremented, decremented, or frozen address register; 8-bit, 16-bit, or 32-bit data transfers), then initialize the destination address register itself (PDA) with an address that is aligned to the programmed data size.

- Store data into PD using the *stf PD* instruction as many times as needed.
- When the transfer is finished and if the peripheral DMA worked in automatic flush mode, force the flush of PD. This instruction is stalled until PD contents are effectively sent to the Arm platform memory or peripheral, and the error status of the transfer is available in the DF flag.

7.2.3.3.2.3.3 *Transferring Data Between Two Arm platform Memory Locations-Peripheral DMA Unit*

The following steps copy data between two Arm platform memory locations using the peripheral DMA unit:

- Set up the PS fields to reflect the modes and data size for the source and destination addresses (all the combinations of addressing modes are possible, but both data sizes must be identical), then initialize the source address register (PSA) and the destination address register (PDA). Both addresses must be aligned with the programmed data size.
- Use as many *stf PD* instructions with the *COPY* flag as needed. Every instruction triggers a single read from the source address; a single write of the received data immediately follows. The DMA acknowledges prior to instruction completion, which frees the SDMA core for other tasks at no delay cost.
- Once the transfer is done, there should be a final access to the peripheral DMA to check the error status.

7.2.3.4 SDMA Security Support

The SDMA provides support to SDMA software to block unauthorized updates to the scripts in RAM.

SDMA supports the following Security modes:

- Open Mode: has full control to load scripts and context into SDMA RAM. This is the default mode.
- Locked Mode: The Arm platform loads scripts and channel contexts at startup when it is still executing known safe software. When finished, it locks the SDMA to prevent further updates to RAM and selected registers. More details described in [Locked Mode](#).

7.2.3.4.1 Locked Mode

The LOCK bit in the SDMA_LOCK register provides support for SDMA scripts to freeze RAM contents after the initial bootload routine to prevent future unauthorized updates to SDMA RAM.

After initial RAM contents are uploaded, Arm platform software can set the LOCK bit to secure the RAM contents to prevent future updates by an unauthorized. After the LOCK bit is written with a '1', the SDMA is "locked" until reset.

The LOCK bit can be read in the SDMA's internal memory map in the LOCK register (see Section [SDMA LOCK \(SDMAARM_SDMA_LOCK\)](#)). SDMA scripts which load information into RAM can check the value of the LOCK bit to determine if an upload to RAM is allowed. If not allowed, the script can refuse to allow the request to copy data into the RAM to continue. The exact use of the LOCK bit in SDMA scripts for security control will be described in SDMA software documentation (see [SDMA Scripts](#)).

While SDMA is locked, attempts to write to the SDMA_LOCK, CHN0ADR, ILLINSTADDR, and ONCE_ENB registers will be ignored. All registers remain readable. Writes to other registers are still allowed.

Once the SDMA is locked, the LOCK bit can only be cleared by a reset. A hardware reset will always clear the LOCK bit. A software reset initiated by writing to the RESET register will only clear the LOCK bit if the SRESET_LOCK_CLR bit in the SDMA_LOCK register is set. Since SDMA_LOCK register cannot be updated if SDMA is locked, the SRESET_LOCK_CLR bit must be configured before setting the LOCK bit. The SRESET_LOCK_CLR bit will also be cleared by resets that clear the LOCK bit.

The SDMA RISC core uses the ILLINST and CHN0ADDR registers as pointers to determine where to jump to after an illegal instruction or upon boot after a reset. The LOCK bit prevents updates to these registers to protect against unauthorized changes to these pointers.

While SDMA is locked, the ONCE_ENB register cannot be written to prevent the OnCE under Arm platform control from being used to gain access to SDMA internal memory. If Arm platform control of the OnCE is enabled before setting the LOCK bit, the Arm platform can use the ONCE for debug purpose after LOCK is set.

7.2.3.5 OnCE and PCU Debug States

The SDMA has two different debug modes in which the OnCE performs debug instructions.

Refer to [Figure 7-4](#) for an example of the PCU states in debug. The following are the two debug states:

- When a channel is running (that is, when CCR and CCPRI are different from 0, which can be read in the PSW register), SDMA can execute a SoftBkpt instruction from the channel script or receive a debug request. When either happens, the SDMA enters its "Classical" *Debug* state, which is described in [OnCE and Real-Time Debug](#).
- When a channel is not running, the SDMA can be in *Sleep* state or in *Sleep after Reset* state. If a debug request is sent to the core, it enters its *Debug in Sleep* state. This debug mode works similarly to the "Classical" *Debug* state, except it returns to the original state (*Sleep* or *Sleep after Reset*) when the debug mode is left via the exec_core instruction of the OnCE. From this *Debug in Sleep* state, the SDMA can execute a program whereas no channel is running. If a new debug request is sent to the core or if a SoftBkpt is executed, it comes back to this *Debug in Sleep* state.

The OnCE is provided with several instructions that can be executed when the core is in either debug state. The following table summarizes the behavior of these OnCE debug instructions. There exists other secondary OnCE instructions that are described in [OnCE and Real-Time Debug](#).

Table 7-10. SDMA in Debug Mode

Instruction	Debug	Debug in Sleep
exec_once	exec_once <instruction> SDMA executes the <instruction> and returns to the <i>Debug</i> state. The Program Counter (PC) is not incremented. This command must not be used with an instruction that modifies the PC value.	exec_once <instruction> SDMA executes the <instruction> and returns to the <i>Debug in Sleep</i> state. The Program Counter (PC) is not incremented. This command must not be used with an instruction that modifies the PC value.
run_core	run_core <instruction> SDMA executes the <instruction>, leaves the <i>Debug</i> state and continues executing the channel script from the position where it stopped. This command must not be used with an instruction that modifies the PC value.	run_core <instruction> SDMA executes the <instruction> and returns to its <i>Sleep</i> or <i>Sleep after Reset</i> initial state. This command must not be used with an instruction that modifies the PC value.
exec_core	exec_core <instruction> It is similar to run_core except it requires an instruction that changes the PC value (jump, branch...): the SDMA jumps to the new PC value, leaves the <i>Debug</i> state and starts executing instructions from this new PC value.	exec_core <instruction> If the previous state was <i>Sleep after Reset</i> , the SDMA returns to this state, and Chn0Addr value overrides the PC value. Otherwise, the SDMA jumps to the new PC value and starts executing instructions from this new PC.

NOTE

The feature exec_core in *Debug in Sleep* after *Sleep after Reset* was added for the Channel boot (channel 0) to allow the debugger to return to *Sleep after Reset* state with a new PC

value. The SDMA will be ready to boot at the Chn0Addr address.

7.2.3.6 SDMA Clocks and Low Power Modes

The SDMA receives several root clocks from the SoC clock controller block and performs adaptive clock gating to optimize its power consumption. From a user standpoint, clock gating and power mode selection are fully automatized inside the SDMA.

Root clock control is available from the SoC clock controller block.

There are numerous clock sources that are used in the SDMA. They belong to one of two possible clock domains listed in the following table, and have frequency constraints within each domain. Clocks are considered asynchronous between domains.

Within the Arm platform/SDMA clock domain, all clocks must come from the same DPLL. The Arm platform DMA interfaces (peripheral DMA and burst DMA) receive their clock from the Arm platform DMA clock source whose frequency can be once or twice the frequency of the SDMA core clock. The DMA interfaces are designed to work at the Arm platform DMA frequency, but the SDMA core is physically limited to a maximum 104 MHz frequency. Since this is lower than the maximum Arm platform DMA frequency, the SDMA core clock is tied to the Arm platform peripheral clock frequency.

The Arm platform Peripheral Bus Clock source must be an exact sub-frequency of the SDMA Core clock source (any integer value greater or equal to 1).

Table 7-11. Clocking Scheme

Clock Domain	Source Clock	Comments
Arm platform	SDMA core (SDMA main core)	Source clock for the core and all its operations; this clock is thus used by most of the SDMA sub-blocks.
	Arm platform DMA	DMA interface for the peripheral DMA and the burst DMA. It is balanced with the main clock source, and its frequency is either once or twice the main clock frequency.
	Arm platform peripheral	Connection to the Arm platform peripheral bus. It is a sub-frequency of the main clock frequency.
JTAG	TCK	Clock for JTAG access, limited to maximum of 1/8 of the SDMA core clock frequency.

The JTAG clock is sampled by the SDMA main clock to determine its rising edge. This simplifies design and clock management, but it also adds a ratio constraint between those two clocks. It is guaranteed the JTAG interface works properly when the frequency of TCK is lower than 1/8th of the frequency of the SDMA main clock (which is about 8 MHz when the SDMA core clock frequency is 66 MHz).

7.2.3.6.1 Clock Gating and Low Power Modes

The SDMA automatically performs power saving without requiring user involvement. It implements two levels of automatic clock gating.

7.2.3.6.1.1 Coarse Clock Gating

Every sub-block clock comes from one of the five available sources, and is gated with the sub-block specific enabling condition.

The following table displays the sub-block clocks and their source. It also indicates the relationships that may exist between different sub-blocks clock enables.

Table 7-12. Sub-blocks Clocks

Sub-block	Source Clocks	Enabling Condition and Comments	Related Enabling Conditions
Core	SDMA Main Core	The core sub-block clock is running when the core is not in one of its sleep states (Sleep or Sleep after Reset) or there is a pending channel. Typically, the core sub-block clock is stopped once all the channels are processed and the core enters its sleep state. A new pending channel awakes the core sub-block clock.	None
Memories	SDMA Main Core	The clock activation only occurs during a core access.	Disabled when Core sub-block clock is disabled or no memory access in progress
Scheduler	SDMA Main Core	Its clock only runs when scheduling is needed: for example, when there are pending channels, upon reception of a DMA request, and anytime the Arm platform modifies the channel running conditions.	None
Arm platform Control	SDMA Main Core & Arm platform peripheral	The Arm platform peripheral clock is solely used to determine the frequency ratio with the SDMA main clock. The control registers' clock is based on <i>SDMA main clock</i> ; it is active when the Arm platform or the SDMA modifies the contents of one of these registers.	None
Burst DMA	SDMA Main Core & Arm platform DMA	The burst DMA has two clocks: The first clock is derived from the SDMA main core clock and drives registers that are connected to the FUBUS. The second clock is derived from the Arm platform DMA clock and drives registers that are connected to the Arm platform DMA bus outside the SDMA. Both clocks are enabled	Disabled when Core sub-block clock is disabled

Table continues on the next page...

Table 7-12. Sub-blocks Clocks (continued)

Sub-block	Source Clocks	Enabling Condition and Comments	Related Enabling Conditions
		during active phases of data transfers (for example, these clocks are turned off when the burst DMA is not used by the running channel script).	
Peripheral DMA	SDMA Main Core & Arm platform DMA	The peripheral DMA has two clocks: The first clock is derived from SDMA main clock and drives registers that are connected to the FUBUS. The second clock is derived from the Arm platform DMA clock and drives registers that are connected to the Arm platform DMA bus outside the SDMA. Both clocks are enabled during active phases of data transfers (for example, these clocks are turned off when the peripheral DMA is not used by the running channel script).	Disabled when Core sub-block clock is disabled
OnCE	SDMA Main Core	The OnCE clock is derived from main source clock. It is disabled by default. In order to use the OnCE, its clock must be explicitly turned on, either by enabling the OnCE access from the Arm platform peripheral bus (register ONCE_ENB), or by driving the clk_gating_off input pin high. This is a SDMA input whose driver depends on the SoC implementation (typically a JTAG controller). The OnCE also receives the TCK input, which is the JTAG clock. It does not use it as a functional clock; the TCK input is sampled instead. Refer to Synchronization Implementation .	When enabled, all other clocks are systematically on (clock gating is off)

7.2.3.6.1.2 Refined Clock Gating

The SDMA implements a second level of clock gating on a register-per-register basis.

Unlike the first level that covers all the SDMA flip-flops, except the synchronizers (only five flip-flops are always running), the second level is only available for eligible registers, which amounts to about 90% of the SDMA flip-flops.

These gated registers are only clocked when the hardware logic detects a new data loading. This additional gating further reduces dynamic power consumption.

7.2.3.6.1.3 Low Power Modes and User Control

Power savings are automatically managed by the SDMA hardware without any user involvement; however, one can distinguish three different power modes: SLEEP, RUN, and DEBUG.

The following table describes these modes, and shows how to switch from one mode to another.

Table 7-13. Power Modes

Power Mode	Sub-blocks							Comments
	Core	Mem ories	Sche duler	Arm platf orm Control	Burs t DMA	Perip heral DMA	OnC E	
SLEEP	off ¹	off	wait ²	wait	off	off	off	Set when the PCU state is either <i>Sleep</i> or <i>Sleep after Reset</i> and the SDMA is not in DEBUG mode. This is the default mode after reset.
RUN	on ³	wait	wait	wait	wait	wait	off	Set for the other PCU states that are reachable out of debug: <i>Program</i> , <i>Data</i> , <i>Change of Flow</i> , <i>Error in Loop</i> , <i>Debug</i> , <i>Functional Unit</i> , <i>Save</i> , or <i>Restore</i> .
DEBUG	on	on	on	on	on	on	on	Set regardless of the PCU state when clock gating is turned off to use the OnCE features (either <i>clk_gating_off</i> pin high or ONCE_ENB[0] set).

1. *off*: no clock

2. *wait*: only clocked when accessed or stimulated

3. *on*: clock is always running

It is possible to control the SDMA power mode. The procedures to force the SDMA into either mode are described in [SLEEP Mode](#).

7.2.3.6.1.3.1 SLEEP Mode

This is the default mode after reset; therefore, resetting the SDMA forces this mode.

However, the common procedure is as follows:

- Ensure the *clk_gating_off* pin is low and ONCE_ENB[0] is cleared.
- Disable all channels (via the STOP_STAT control register, and the HO, DO, EO if necessary).
- Wait for the active channels to complete or force a reschedule via the reschedule bit in the RESET register.
- The SDMA is in SLEEP mode making it possible to completely shut off its clock from the chip level clock controller using the procedure described in [Stop Mode Response](#).

7.2.3.6.1.3.2 RUN Mode

This is the default mode when a channel is running:

- Ensure the *clk_gating_off* pin is low and ONCE_ENB[0] is cleared.
- Activate at least one channel (via the HSTART control registers, a DMA request, and/or the HO, DO, EO register bits).

7.2.3.6.1.3.3 *DEBUG Mode*

The DEBUG mode must be set when one needs to use the debugging facilities of the SDMA.

- Ensure the SDMA clocks are running from the CCM.
- Set the *clk_gating_off* pin high or use the SDMA to set ONCE_ENB[0].

7.2.3.6.1.4 **Stop Mode Response**

The SDMA receives a stop request from the chip level clock controller. This request may be asserted when the chip enters the stop low power mode.

If the SDMA is running when the request is received, then the SDMA will complete all pending channels before returning to the SLEEP state. The SDMA sends an acknowledgement to the clock controller when the SLEEP state is entered indicating that the SDMA's clocks can be turned off.

7.2.3.6.2 **Reset**

After reset (either received from the reset block or a software reset required by the Arm platform), the SDMA is in IDLE mode. It will start its boot code located at address 0 once a channel is activated.

Activating a channel can be done by the Arm platform after programming a positive priority and setting the channel bit in the EVTPEND register.

There will not be a context RESTORE for the first channel (bootload channel) called after a reset because the context data in RAM has not been initialized. Static context mode should be used for the first channel called after reset to ensure that the all context RAM for that channel is initialized. Subsequent calls to the same channel or different channels may use any of the dynamic context modes

7.2.3.7 **Software Interface**

Appendix A fully describes the SDMA Application Programming Interface (API).

7.2.3.8 Initialization Information

This section discusses the following:

- [Hardware Reset](#)
- [Channel Script Execution](#)
- [Initialization and Script Execution Setup Sequence](#)

7.2.3.8.1 Hardware Reset

After reset, the program RAM, context RAM, data RAM, and RAM containing the channel enable registers (CHNENBLn) have unpredictable contents.

The active register set is assigned to channel 0 and the PC is initialized to all zeros. However, since the channel enable register is all zeros, there are no active channels and the SDMA is halted waiting for the boot channel to start.

The Arm platform will have to setup the SDMA in order to boot it. The CONFIG register must be initialized to determine the DMA/core clock ratio (1 or 2). Channel Enable Registers must also be initialized.

To start up the SDMA, the Arm platform first creates some channel control blocks (CCB) and buffer descriptors (BD) in Arm platform memory for the boot channel (channel 0) and then initializes the channel 0 pointer register (SDMA_MC0PTR) to the address of the first control block. [Data Structures for Boot Code and Channel Scripts](#) provides an overview of the data structure for the CCB and BD's. The SDMA_HSTART, SDMA_HOSTOVR and SDMA_EVT OVR registers are then configured according to [Runnable Channels Evaluation](#) to allow channel 0 to run.

Upon being enabled, the SDMA begins executing the script located at the address indicated by the Channel 0 Boot Address register (SDMA_CHN0ADDR) in the program memory. The reset value of SDMA_CHN0ADDR points to the default bootload script in ROM. This ROM script will read the channel 0 pointer register (SDMA_MC0PTR) to determine the location of the Channel Control Block (SDMA_CCB) in Arm platform memory. The script will then begin fetching by DMA the first channel control block which contains a pointer to the location channel 0 Buffer Descriptor chain which is also fetched via DMA. If the buffer descriptor contains a valid command, the script interprets the command in each buffer descriptor and proceeds to implement the command and move on to the next buffer descriptor control block. The buffer descriptor commands for channel zero are typically set up to load SDMA's program RAM, Data RAM, and initial values for the channel contexts. Some channel scripts expect particular parameters to be passed

There are two ways to make the SDMA boot on a user-defined script. The OnCE (either via its JTAG interface or its Arm platform Control interface) can be used to download any code in the SDMA RAM and force the SDMA to boot on that code. Also, the SDMA_CHN0ADDR register in the Arm platform programming model can be modified to point to user code in RAM which would need to either have been loaded via the ONCE or default bootload routine (ex before a S/W reset).

7.2.3.8.2 Channel Script Execution

The execution of an SDMA script depends on both the instructions that make up the script, the data context upon which it operates, and commands or parameters allowed to the buffer. All these items must be initialized before the script is allowed to execute.

Each of the 32 channels has a separate context, but may share scripts and locations in data RAM.

Channels are initialized by the Arm platform by using channel 0 to download any required scripts and data values and the channels initial context. The context contains all the initial values of the SDMA core registers. This includes the Program Counter (PC) which is set to the start of the desired script in SDMA program memory.

The Arm platform selects which trigger conditions that must occur for the channel to start by configuring the SDMA_CHNENBL, SDMA_HOSTOVR and SDMA_EVT OVR registers. The trigger events include Arm platform setting HE (SDMA_HSTART) or a hardware DMA request asserts an event input to SDMA. The channel can become active according to its priority compared with other runnable channels when the selected trigger(s) cause the condition described in [Runnable Channels Evaluation](#) to evaluate as true.

The specific parameters to be passed to each script in the buffer descriptor or context are documented in the software documentation for each script. Please refer to [SDMA Scripts](#) for complete script documentation. [Buffer Descriptor Format](#) provides an overview of the buffer descriptor format.

7.2.3.8.3 Initialization and Script Execution Setup Sequence

To summarize, the following steps are minimally required to setup SDMA and run channel scripts.

- Perform Hardware Reset. The program RAM, context RAM, data RAM and SDMA_CHNENBLn registers have unpredictable contents after this reset.
- Initialize SDMA_CHNENBLn registers to map DMA request events to desired channels.

- Configure SDMA_CHNPRIn registers to select priority for runnable channels. A non-zero priority is required for the channel to run.
- Configure the SDMA_CONFIG register to select DMA to SDMA core clock ratio .
- Set up channel control blocks and buffer descriptors in Arm platform to specify the loading of SDMA program RAM and channel contexts for each SDMA channel to be used. Reference [Data Structures for Boot Code and Channel Scripts](#).
- Configure SDMA_MC0PTR register with base address of Arm platform Channel Control Block base address.
- Initialize SDMA_CHNENBLn registers to map DMA request events to associated channel. Reference [Mapping DMA Requests to Pending Channels](#).
- Configure SDMA_CHNPRIn registers to set priority for each channel to be run.
- For each channel to be run, configure SDMA_HOSTOVR (HO) and SDMA_EVT0VR (EO) registers to select which events (hardware and/or software trigger events) must occur for the channel to be runnable. Reference [Runnable Channels Evaluation](#).
- Set bit 0 of the SDMA_HSTART register to set HE[0] and allow Channel 0 to run (assumes EO[0] and DO[0] were both set in previous step). This will cause SDMA to load the program RAM and channel contexts configured previously.
- Wait for Channel 0 to finish running. This is indicated by HI[0]=1 in the SDMA_SDMA_INTR register, or by optional interrupt to the Arm platform.
- Set the LOCK bit in the SDMA_SDMA_LOCK register to prevent un-authorized uploads of data to SDMA RAM.
- Additional channel scripts can now be run by enabling the selected software or hardware trigger event according to [Runnable Channels Evaluation](#).

7.2.3.9 SDMA Programming Model

This section describes the programming model for the SDMA RISC engine, including its processor, memory, and internal control registers.

All addresses are related to the internal SDMA memory map, which is completely different from the Arm platform memory maps. The Arm platform processor has no access to any hardware resource described, except when those resources are described in Arm Platform Memory Map and Control Register Summary. .

7.2.3.9.1 State and Registers Per Channel

The SDMA can be seen as a set of 32 identical devices that are able to perform one data transfer channel each. Only one channel can work at a time, but every channel state is available at any time.

This chapter lists the components of every channel state.

7.2.3.9.2 General Purpose Registers

Each channel has eight general purpose registers of 32 bits for use by scripts. General register 0 has a dedicated function for the loop instruction, but otherwise can be used for any purpose.

7.2.3.9.3 Functional Unit State

Each channel context has some state that is part of the functional units.

The specific allocation of this state is part of the functional unit definition that is described in [Burst DMA Unit Programming](#), [Peripheral DMA Unit Programming](#).

This state must be saved/restored on context switches.

7.2.3.9.3.1 Program Counter Register (PC)

The PC is 14 bits. Since instructions are 16 bits in width and all memory in the SDMA is 32 bits in width, the low order bit of the PC selects which half of the 32-bit word contains the current instruction.

A low order bit of zero selects the most significant half of the word.¹

7.2.3.9.3.2 Flags

Each channel has the following four flags:

- The T bit reflects the status of some arithmetic and test instructions. It is set when the result of an addition or a subtraction is zero and cleared otherwise. It is also the copy of the tested bits. Finally, it can also be set when the loop counter (GReg0) reaches zero. When the last instruction of the hardware loop is an operation that can modify the T flag, its effect on T is discarded and replaced by the GReg0 status.
- Two additional bits, SF and DF, are used to indicate error conditions resulting from loading data sources and storing to destinations, respectively. Access errors set these bits, and successful transactions clear them. They can also be cleared by specific instructions (CLRF and loop). The source fault (SF) is updated by the loads LD and LDF; the destination fault (DF) is updated by the stores ST and STF.

1. For example, big-Endian.

- Access errors are caused by several conditions including writing to the ROM, writing to a read-only memory mapped register, accessing an unmapped address, or any transfer error received by a peripheral when it is accessed.

The SF and DF flags have a major impact on the behavior of the hardware loop: If SF or DF is set when starting a hardware loop and it is not masked by the loop instruction, the loop body will not be executed. Inside the loop body, if a load or store sets the corresponding SF or DF flag, the loop exits immediately. Testing the status of the T flag at the end of the loop (as well as testing both SF and DF) tells if the loop exited abnormally as any anticipated exit prevents GReg0 from reaching the zero value and thus setting the T flag. This is also valid if the fault occurs at the last instruction of the last loop.

- The last flag is the loop mode flag, LM, which is composed of two bits. The most significant bit indicates when the processor is currently operating in loop mode. It is set by the loop instruction and is cleared after execution of the last instruction of the last loop. The least significant bit is set when the program counter points to the last instruction of a loop on the last path. It is used for a channel that is restored with this configuration to know that the next program counter is EPC. As with the dynamic context switch GReg0, which indicates when the program must get out of the loop, it can be restored only on the last instruction of the loop. This, however, is too late to fetch the next instruction after the loop.

7.2.3.9.3.3 Return Program Counter (RPC)

The RPC is 14 bits. It is set by the jump to the subroutine instructions and used by the return from the subroutine instructions.

Instructions are available to transfer its contents to and from a general register.

7.2.3.9.3.4 Loop Mode Start Program Counter (SPC)

The SPC is 14 bits. It is set by the loop instruction to the location immediately following it.

7.2.3.9.3.5 Loop Mode End Program Counter (EPC)

The EPC is 14 bits. It is set by the loop instruction to the location of the next instruction after the loop.

7.2.3.9.4 Context Switching-Programming

Each channel has a separate context consisting of the eight general purpose registers and additional registers representing the state of the functional units.

The active registers and functional units contain the context of the active channel. The context of inactive channels is stored in SDMA RAM, which is part of the SDMA address space.

In a function of the selected context switching mode ([Context Switching](#)), modified registers by the program can be saved in the channel RAM space while the program is going on. In every cycle, a write access to the RAM is possible.

On a done or yield(ge) instruction, SDMA goes into "real" context switching. In one of the dynamic modes, modified registers not previously saved, as well as the PC-Loop registers, are stored into the context area of the channel that will be closed. The new PC-Loop registers are loaded from the context area of the new channel. All other registers are restored while the program is executed, giving priority to registers used by the decoded instruction. Therefore, in the best case, only the PC and Loop registers should be saved and restored during this context-switching phase, which only requires five SDMA cycles.

In static mode, the context switch stores all registers in the old channel RAM space, and restores all registers from the new channel RAM space. It requires 26 SDMA cycles.

The address of the context memory for channel i is $CONTEXT_BASE + 24*i$ or $CONTEXT_BASE + 32*i$ where $CONTEXT_BASE$ equals 0x0800. The table below presents the layout of a channel context in memory:

Table 7-14. Layout of a Channel Context in Memory for SDMA

OFFSET	31	30	29-16	15	14	13-0
0	SF	-	RPC	T	-	PC
1	LM		EPC	DF	-	SPC
2	GR0					
3	GR1					
4	GR2					
5	GR3					
6	GR4					
7	GR5					
8	GR6					
9	GR7					
10	MDA (burst DMA)					
11	MSA (burst DMA)					
12	MS (burst DMA)					
13	MD (burst DMA)					

Table continues on the next page...

Table 7-14. Layout of a Channel Context in Memory for SDMA (continued)

14	PDA (peripheral DMA)
15	PSA (peripheral DMA)
16	PS (peripheral DMA)
17	PD (peripheral DMA)
18	
19	
20	Reserved ¹
21	Reserved ¹
22	Reserved ¹
23	Reserved ¹
24	Scratch RAM (optional)
25	Scratch RAM (optional)
26	Scratch RAM (optional)
27	Scratch RAM (optional)
28	Scratch RAM (optional)
29	Scratch RAM (optional)
30	Scratch RAM (optional)
31	Scratch RAM (optional)

7.2.3.9.5 Address Space

The SDMA has four internal buses which are listed here.

- The Instruction bus reads instructions from the memory. Its address map is described in [Instruction Memory Map](#).
- The Data bus (DMBUS) accesses the same memories as those visible on the Instruction bus, some memory-mapped registers (scheduler status and OnCE registers), and up to 14 peripherals. Its address map is described in [Data Memory Map](#).
- The Functional Units bus (FUBUS) accesses the , Burst DMA, Peripheral DMA . The addressing mechanism is further detailed in [Functional Units Programming Model](#).
- The Context Switch bus reads/writes registers into context-switch RAM space. It is a 64-bit bus dedicated for accessing this RAM space for updating the context of the running channel. While the program is going on, this bus has the lowest priority compared to the Instruction and Data buses, except for restoring a register needed for the decoded instruction to be executed. On the save part of a context switch (when the PCU is in its slave state), this is the only one used. On the restore part, the Instruction bus has the priority to read the next instruction at the restored PC and

otherwise the Context Switch bus is used. It is not possible to control the actual data transfers that occur on this bus.

7.2.3.9.5.1 Instruction Memory Map

The instruction memory map is based on a 14-bit address bus and a 16-bit data (instruction) bus.

Instructions are fetched from either program ROM or program RAM. An SDMA script is able to change the contents of the program RAM, which is also visible from the data bus.

The first two instruction locations (at 0 and 1) are special. Location 0 is where the PC is set on reset. Location 1 is where the PC is set upon the execution of an illegal instruction. It is expected that both of these locations will contain a jmp to handle routines.

Table 7-15. SDMA Instruction Memory Space

Device	SDMA Address (Hex)	Base Address Label	Block Name	WS	Description
ROM	0x0000 ↓ 0x07FF	SDMA_IBUS_ROM_ADDR	-	0	4 Kbyte internal ROM with boot code and standard routines.
RAM	0x1000 ↓ 0x1FFF	SDMA_IBUS_RAM_ADDR	-	0	8 Kbyte internal RAM with channels context and user data/routines.

7.2.3.9.5.2 Data Memory Map

All of the data accessible to SDMA scripts make up the data memory space of the SDMA.

This address space has several components:

- ROM (also visible on the Instruction bus)
- RAM (also visible on the Instruction bus)
- Shared Peripherals Registers
- SDMA Internal Registers (scheduler, OnCE, and registers that are also accessible by the Arm platform)

SDMA scripts can read and write to the context RAM, data RAM, shared peripheral registers, and internal registers.

The address range is 16 bits and the data width is 32 bits. When accessing peripheral registers (USB and so on), the data width may be different. The exact address map for the peripherals depends on the project (as presented in each respective chapter).

Data access is performed with *ld* and *st* instructions that take the address from a general purpose register in the core (GRegn). The mapping between the general purpose register contents and the address bus is given in the following table:

Table 7-16. GRegn to DMBUS Address Mapping

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sz	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
address															

Grayed bits are simply discarded but they must be cleared to ensure forward-script compatibility.

- sz (bit 31) indicates the peripheral data width: 0 is used for a 32-bit peripheral and 1 is used for a 16-bit peripheral.
- address (bits 15 down to 0) is the address of the accessed resource (internal memory, internal register, or shared peripheral).

Table 7-17. SDMA Data Memory Space

Device	SDMA Address (Hex)	Size	Description
ROM	0x0000 → 0x03FF	4 Kbyte	4 Kbyte internal ROM with boot code and standard routines
Reserved	0x0400 → 0x07FF	4 Kbyte	4 Kbyte Reserved
RAM	0x0800 → 0x0FFF	8 Kbyte	8 Kbyte internal RAM with channels contexts and user data/routines
per1	0x1000 → 0x1FFF	16 Kbyte	<i>peripheral 1</i> memory space (4 Kbyte peripheral's address space)
per2	0x2000 → 0x2FFF	16 Kbyte	<i>peripheral 2</i> memory space (4 Kbyte peripheral's address space)
per3	0x3000 → 0x3FFF	16 Kbyte	<i>peripheral 3</i> memory space (4 Kbyte peripheral's address space)
per4	0x4000 → 0x4FFF	16 Kbyte	<i>peripheral 4</i> memory space (4 Kbyte peripheral's address space)
per5	0x5000 → 0x5FFF	16 Kbyte	<i>peripheral 5</i> memory space (4 Kbyte peripheral's address space)
per6	0x6000 → 0x6FFF	16 Kbyte	<i>peripheral 6</i> memory space (4 Kbyte peripheral's address space)
Registers	0x7000 → 0x7FFF	16 Kbyte	Memory mapped registers
per7	0x8000 → 0x8FFF	16 Kbyte	<i>peripheral 7</i> memory space (4 Kbyte peripheral's address space)
per8	0x9000 → 0x9FFF	16 Kbyte	<i>peripheral 8</i> memory space (4 Kbyte peripheral's address space)
per9	0xA000 → 0xAFFF	16 Kbyte	<i>peripheral 9</i> memory space (4 Kbyte peripheral's address space)
per10	0xB000 → 0xBFFF	16 Kbyte	<i>peripheral 10</i> memory space (4 Kbyte peripheral's address space)
per11	0xC000 → 0xCFFF	16 Kbyte	<i>peripheral 11</i> memory space (4 Kbyte peripheral's address space)
per12	0xD000 → 0xDFFF	16 Kbyte	<i>peripheral 12</i> memory space (4 Kbyte peripheral's address space)
per13	0xE000 → 0xEFFF	16 Kbyte	<i>peripheral 13</i> memory space (4 Kbyte peripheral's address space)
per14	0xF000 → 0xFFFF	16 Kbyte	<i>peripheral 14</i> memory space (4 Kbyte peripheral's address space)

7.2.3.10 SDMA Initialization

Appendix A describes the setup of the SDMA . This section provides a quick description of several initialization procedures.

NOTE

There may be differences with the actual implementation in the API.

7.2.3.10.1 Hardware Reset-SDMA

After reset, the RAM that holds contexts, data, scripts, and the DMA request-channels matrix has unpredictable content.

The core registers are all reset to 0, including the PC; the PCU state is *Sleep after Reset*. No channel can be activated because all of the priorities are also reset to 0.

7.2.3.10.2 Standard Boot Sequence

The following is the standard boot sequence:

1. Initialize the CONFIG register-detailed in [Configuration Register \(SDMAARM_CONFIG\)](#)-to determine the Arm platform DMA/core clock ratio (1 or 2)
2. Initialize the DMA request-channels matrix (see [Channel Enable RAM \(SDMAARM_CHNENBL_n\)](#)).
3. Program the channel control registers-[Channel Event Override \(SDMAARM_EVTOVR\)](#), [Channel BP Override \(SDMAARM_DSPOVR\)](#), [Channel BP Override \(SDMA_HOSTOVR\)](#), and [Channel Event Pending \(SDMAARM_EVTPEND\)](#)-according to the channel allocation.
4. Perform any necessary setup as required by the standard boot script in ROM (this is described in Appendix A).
5. Trigger channel 0 with the [Channel Start \(SDMAARM_HSTART\)](#) register, which starts the execution of the ROM script starting at address 0. This boot downloads channel scripts and contexts in RAM.

7.2.3.10.3 User-Defined Boot Sequence

The following is a user-defined boot sequence:

1. Initialize the [Configuration Register \(SDMAARM_CONFIG\)](#)[Channel Enable RAM \(SDMAARM_CHNENBL_n\)](#), [Channel Event Override \(SDMAARM_EVTOVR\)](#), [Channel BP Override \(SDMAARM_DSPOVR\)](#), [Channel Arm platform Override \(SDMAARM_HOSTOVR\)](#), and [Channel Event Pending \(SDMAARM_EVTPEND\)](#).

2. Use the OnCE (either via its JTAG interface or its Arm platform control registers) to download any code in the SDMA RAM. [Accessing the Memory](#) describes how to write data to the RAM via the OnCE.
3. Use the OnCE instructions to make the PC default value point to the new boot script start address, or rely on the ROM startup script, which first jumps to the address in [Channel 0 Boot Address \(SDMAARM_CHN0ADDR\)](#). (This register default address points to the standard boot script.)

7.2.3.10.4 Script Loading and Context Initialization

The execution of an SDMA script depends on both the instructions that make up the script and the data context upon which it operates. Both must be initialized before the script is allowed to execute.

Each of the 32 channels has a separate data context, but may share scripts and locations in the data RAM.

The Arm platform manages the space in program RAM and data RAM. It also manages the assignment of SDMA channels to the device drivers that need them. Channels are initialized by the Arm platform via the channel 0 boot script. The boot channel downloads any required scripts with their data and the channels' initial contexts. Every context contains all the initial values of the registers, including the PC. Then the Arm platform can enable any channel that becomes active and begins fetching and executing instructions from its script.

7.2.3.11 Instruction Description

The following sections introduce the instruction of the SDMA.

Instruction set details are available in [Instruction Set](#).

7.2.3.11.1 Scheduling Instructions

The following are scheduling instructions:

- done-The instruction causes certain scheduling or interrupt bits to be set or cleared, which may cause a change in the schedule-ability of the running channel. Then the instruction causes the SDMA to evaluate the current scheduling priorities and to choose the highest priority ready channel. If this channel is not the current channel, a context switch will take place. If there are no runnable channels, the SDMA will enter the stopped mode. The done 5 has a special usage reserved for debug, as explained in [Debug Instructions](#).

- **yield**-These instructions are special cases of the done instruction. They do not modify the scheduling bits, but allow the highest pending channel (if it exists) to preempt the current channel if the pending channel priority is strictly greater than the current channel priority.
- **yieldge**-These instructions are special cases of the done instruction. They do not modify the scheduling bits, but allow the highest pending channel (if it exists) to preempt the current channel if the pending channel priority is strictly greater or equal to the current channel priority.
- **notify**-The notify instruction affects the scheduling bits, but does not cause rescheduling.

7.2.3.11.2 Conditional Branch Instructions

The conditional branch instructions of an 8-bit displacement, which is sign-extended and added to the current PC (which points to the next instruction) if the condition is satisfied.

Otherwise, control passes to the next sequential instruction.

- **BF**-Branch if False. The branch is taken if the T bit in the processor status is zero (false).
- **BT**-Branch if True. The branch is taken if the T bit in the processor status is one (true).
- **BSF**-Branch if Source Fault. The branch is taken if the SF bit in the processor status is one.
- **BDF**-Branch if Destination Fault. The branch is taken if the DF bit in the processor status is one.

7.2.3.11.3 Unconditional Jump Instructions

There are two varieties of unconditional control transfers: an absolute transfer and a through-register transfer.

Absolute transfers have a 14-bit address field that replaces the current PC.

- **JMP**-Jump. Causes the processor to jump to an absolute address encoded in the instruction itself.
- **JSR**-Jump to Subroutine. Causes the processor to jump to a subroutine, the address of which is encoded in the instruction itself.

- **JMPR**-Jump through Register. Causes the processor to jump to an absolute address contained in a General register. This instruction is meant to be used when more than one level of subroutines are required.
- **JSRR**-Jump to Subroutine through Register. Causes the processor to jump to a subroutine, the address of which is contained in a General register. This instruction is meant to be used when more than one level of subroutines are required.

7.2.3.11.4 Subroutine Return Instructions

The following are subroutine return instructions:

- **RET**-Return from Subroutine. The RET restores the contents of RPC to PC.
- **LDRPC**-Load from RPC to Register. The LDRPC instruction is meant to be used when more than one level of subroutines are required. It stores the contents of RPC in any General register.

7.2.3.11.5 Loop Instruction

The following is a loop instruction:

LOOP-Enters Loop Mode. Before entering loop mode, the loop instruction can optionally clear the fault flags (SF and/or DF) based on a 2-bit field in the instruction. This feature is linked to the fact that setting SF or DF in loop mode will cause an immediate exit of the loop.

7.2.3.11.6 Miscellaneous Instructions

The following are miscellaneous instructions:

- **CLRF**-Clear Fault Flags. This instruction clears any combination of SF and DF.
- **MOV r,s**-This moves data from GReg[s] to GReg[r].
- **LDI r,immediate**-This loads GReg[r] with a zero-extended immediate value.

7.2.3.11.7 Logic Instructions

The following are logic instructions:

- **XORr,s**-This performs an exclusive or between GReg[r] and GReg[s], and stores the result in GReg[r].
- **XORIr,immediate**-This performs an exclusive or between GReg[r] and a zero-extended immediate value, and stores the result in GReg[r].
- **ORr,s**-This performs an or between GReg[r] and GReg[s], and stores the result in GReg[r].

- **ORIr,immediate**-This performs an or between GReg[r] and a zero-extended immediate value and, stores the result in GReg[r].
- **ANDNr,s**-This performs an and between GReg[r] and the negated GReg[s], and stores the result in GReg[r].
- **ANDNIr,immediate**-This performs an and between GReg[r] and the negated zero-extended immediate value, and stores the result in GReg[r].
- **ANDr,s**-This performs an and between GReg[r] and GReg[s], and stores the result in GReg[r].
- **ANDIr,immediate**-This performs an and between GReg[r] and a zero-extended immediate value, and stores the result in GReg[r].

7.2.3.11.8 Arithmetic Instructions

Arithmetic instructions modify the T bit in the processor status according to the result of the operation. The T bit is set if the result is zero, otherwise it is cleared.

- **ADD r,s**-This performs the addition of GReg[r] and GReg[s], and stores the result in GReg[r].
- **ADDI r,immediate**-This performs the addition of GReg[r] and a zero-extended immediate value, and stores the result in GReg[r].
- **SUB r,s**-This performs the subtraction of GReg[s] from GReg[r], and stores the result in GReg[r].
- **SUBIr,immediate**-This performs the subtraction of a zero-extended immediate value from GReg[r], and stores the result in GReg[r].

7.2.3.11.9 Compare Instructions

Compare instructions modify the T bit in the processor status according to the result of the operation. The T bit is set if the comparison is true, otherwise it is cleared.

NOTE

Only one version of the immediate form is implemented. Non-equality comparisons to immediate values will require two instructions.

- **CMPEQ r,s**-This sets T when registers GReg[r] and GReg[s] are equal.
- **CMPEQIr,immediate**-This sets T when register GReg[r] and the zero-extended immediate value are equal.
- **CMPLTr,s**-This sets T when register GReg[r] is less than and not equal to GReg[s]. The comparison is signed.
- **CMPHS r,s**-This sets T when register GReg[r] is greater than or equal to GReg[s]. The comparison is signed.

7.2.3.11.10 Test Instructions

Test instructions modify the T bit in the processor status according to the result of the operation. The T bit is set if any bit in the result is one, otherwise it is cleared.

- TSTr,s-This performs an and between GReg[r] and GReg[s], and sets T if the result is not zero.
- TSTIr,immediate-This performs an and between GReg[r] and a zero-extended immediate value, and sets T if the result is not zero.

7.2.3.11.11 Byte Permutation Instructions

These instructions shuffle the bytes in a register. For the purpose of describing these instructions, have the bytes in a register be numbered from the most significant as b_3 , b_2 , b_1 , b_0 .

- RORBr-The rotate right byte. The result is b_0 , b_3 , b_2 , b_1 .
- REVBr-The reverse bytes in word. The result is b_0 , b_1 , b_2 , b_3 .
- REVBLOr-The reverse, two low-order bytes. The result is b_3 , b_2 , b_0 , b_1 .

7.2.3.11.12 Bit Shift Instructions

The following are bit shift instructions:

- ROR1r-The rotate right 1 bit. This instruction does a circular right shift of 1 bit.
- LSR1r-The logical shift right 1 bit. This instruction shifts all bits to the right by 1. The high order bit is replaced by a 0.
- ASR1r-The arithmetic shift right 1 bit. This instruction shifts all bits to the right by 1. The high order bit is replaced by itself.
- LSL1r-The logical shift left 1 bit. This instruction shifts all bits to the left by 1. The low order bit is replaced by zero.

7.2.3.11.13 Bit Manipulation Instructions

- BCLR1r,n-The bit clear is immediate; clears bit number i in register r .
- BSET1r,n-The bit set is immediate; sets bit number i in register r .
- BTST1r,n-The bit test is immediate; tests bit number i in register r (T becomes equal to the selected register bit).

7.2.3.11.14 SDMA Memory Access Instructions

All memory accesses are 32 bits.

Any memory location that is implemented with less than 32 bits (for example, peripheral registers) causes unimplemented bits to be read as 0s.

All memory accesses will cause either the SF or DF flags in the processor status to be set if they cause a fault.

What constitutes a fault, especially when accessing peripheral registers, is a property of the memory location.

- **LDr,(b,d)**-The load instruction creates an address by adding the displacement field (d) to the contents of the base register (b). The SDMA location at the resulting address is read and placed in the destination register (r).
- **STr,(b,d)**-The store instruction creates an address in the same manner as the load instruction. The register (r) is stored in the SDMA location at the resulting address.

7.2.3.11.15 Functional Unit Instructions

The functional unit instructions have an 8-bit field that is placed on the functional unit bus.

Some of these bits are used to select which functional unit should be involved in the transfer. The remaining bits are decoded by the selected functional unit so their specific use depends on the functional unit. See [Functional Units Programming Model](#).

There are two functional unit instructions, as follows:

- **LDFr,fub**-The 8-bit field is placed on the functional unit bus and a read is issued to the selected functional unit. As a result of this instruction, the SF may be set in the processor status.
- **STFr,fub**-The 8-bit field is placed on the functional unit bus and a write is issued to the selected functional unit. As a result of this instruction, the DF may be set in the processor status.

7.2.3.11.16 Illegal Instructions

All instruction encodings that are illegal cause the following actions:

- The current PC (which points to one beyond the offending instruction) is put in the EPC register.
- The loop mode bit is cleared.
- The PC is set to the value stored in the [Illegal Instruction Trap Address \(SDMAARM_ILLINSTADDR\)](#) register (the default value is 0x0001).

ILLEGAL-Although any instruction other than those indicated in the SDMA specification will trigger the illegal instruction mechanism, the **ILLEGAL** instruction code is preferred as it will always be kept as *illegal* in the possible future versions of the SDMA core.

7.2.3.11.17 Debug Instructions

The following are debug instructions:

- **SOFTBKPT**-The software breakpoint instruction causes the core to stop and enter debug mode. The core can then be accessed and started by the OnCE debug block only.
- **done 5**-This instruction is used for debugging, as it copies the contents of the PCU registers and flags to the context memory. Information on this instruction is described in [Saving the Context](#).
- **CpShReg**-This instruction copies the context memory into the PCU registers and flags. Modifying the corresponding memory location before executing this instruction enables you to have the channel continue from a new instruction address. This instruction is described in [Restoring the Context](#).

7.2.3.12 Functional Units Programming Model

The functional unit instructions cause an 8-bit code, found in the low eight bits of the instruction, to be asserted on the functional unit control bus.

Some of these bits are used to select one of several functional units. Functional units which can be selected include SDMA registers such as MSA and MSD which are not mapped in the SDMA memory map, and are accessible only through the functional unit bus. These Functional Unit Registers are listed in the following table. In order to establish a programming convention, assume the selection bits are some number of the most significant bits of the 8-bit code. Furthermore, some number of the least significant bits is decoded by a given functional unit to establish the type of operation to perform.

Table 7-18. Functional Unit Registers

Functional Unit	Register	Register Name	Section/Page
Burst DMA Unit Programming	SDMSA	Memory Source Address Register	Memory Source Address Register (MSA)
	MDA	Memory Destination Address Register	Memory Destination Address Register (MDA)
	MD	Memory Data Buffer Register	Memory Data Buffer Register (MD)

Table continues on the next page...

Table 7-18. Functional Unit Registers (continued)

Functional Unit	Register	Register Name	Section/Page
Peripheral DMA Unit Programming			(Write) Burst DMA Write (stf) (Read) Burst DMA Read (ldf)
	MS	Memory State Register	State Register (MS)
	PSA	Peripheral Source Address Register	Peripheral Source Address Register (PSA)
	PDA	Peripheral Destination Address Register	Peripheral Destination Address Register (PDA)
	PD	Peripheral Data Buffer Register	Peripheral Data Register (PD) (Write) Peripheral DMA Write (stf)-Write Mode (Read) Peripheral DMA Read (ldf)-Read Mode
	PS	Peripheral State Register	Peripheral State Register (PS)

More information regarding the functional units can be found in [Peripheral DMA Unit](#), and [Burst DMA Unit](#).

7.2.3.12.1 Burst DMA Unit Programming

The DMA instructions control the DMA state machine and may cause a DMA cycle on the associated memory bus.

There are four registers associated with the burst DMA unit: a Memory Source Address register (MSA), a Memory Destination Address register (MDA), a Memory Data buffer (MD), and a state register (MS). The burst DMA has two different uses:

- A data transfer between External Memory Interface and SDMA general register
- A data transfer in copy mode where blocks of data are transferred from the source address to the destination address

7.2.3.12.1.1 Memory Source Address Register (MSA)

The source address register contains the pointer into EXTMC memory associated with the next read data transfer. It has byte granularity.

Reading the register with the ldf instruction has no side effects, and gives the address value in the EXTMC memory of the next data that is read by the SDMA during an ldf MD instruction.

Writing the source address register has two side effects: If the prefetch bit is set, a DMA read cycle (8-word read access) is issued with the new address. Any data still located in the buffer is lost. If there is valid write data in the buffer, it is necessary to force the DMA to completely flush it out before modifying MSA to guarantee all the data is effectively written to memory.

The MSA register has two modes of programming:

- Frozen-In frozen mode, the MSA register is not modified after DMA accesses.
- Incremented (default mode)-In incremental mode, MSA is incremented by the number of bytes transferred during read cycles.

7.2.3.12.1.2 Memory Destination Address Register (MDA)

The destination address register contains the pointer into EXTMC memory associated with the next write data transfer. It has byte granularity.

Reading the MDA register with the ldf instruction has no side effects. It gives the address value in the EXTMC memory where the next SDMA data (stf r,MD instruction) is stored when MD FIFO is flushed.

Writing the destination address register has one side effect. Any data still located in the buffer is lost. If there is valid write data in the buffer, it is necessary to force the DMA to completely flush it out before modifying MDA to guarantee all the data is effectively written to memory.

The MDA register has two modes of programming:

- Frozen-In frozen mode, the MDA register is not modified after DMA accesses.
- Incremented (default mode)-The MDA register is incremented by the number of bytes transferred during write cycles.

7.2.3.12.1.3 Memory Data Buffer Register (MD)

The data buffer register consists of a bank of 36 bytes that behave like FIFO.

This FIFO stores the eight words received when a read burst is triggered by the DMA (DMA is in read mode).

The MD register is in write mode after a writing in MDA or after an stf MD instruction.

In that case, a burst write access is automatically triggered when there are more than eight words in MD. For bandwidth optimization, any transfers between DMA and the EXTMC controller are based on burst accesses.

An `ldf r,MD|SIZE` instruction that reads the data buffer may cause a DMA cycle, as follows:

- If there are less bytes in the FIFO than the size parameter of the instruction. For instance, if only two bytes are available in MD and a 4-byte read is requested, a burst read access is executed to complete the two bytes.
- If the prefetch bit is set, and after reading there is enough space in the FIFO to store a full burst, a burst read access is triggered.

An `stf r,MD|SIZE` instruction that writes to the data buffer may cause a DMA cycle if the number of written bytes in MD is higher than 32 (eight words) or if the flush bit is set.

When DMA is used for data transfer between SDMA and EXTMC (reading or writing), no immediate error is possible because the block manages a data misalignment issue; therefore, it is allowed to read/write a word to/from a half-word address. However, the addresses (source or destination) must belong to the EXTMC memory mapping. The only potential error, in this mode, would be the error sent back by the EXTMC controller when an access to a super-user page is detected. The whole transfer on the DMA associated bus will be considered successful when there are no errors seen on the bus during the transfer. In copy mode, an immediate error could be returned to SDMA as described in [Burst DMA Unit Error Management](#).

7.2.3.12.1.4 State Register (MS)

The state register contains the DMA state-machine value. It can be accessed in case of an error received during a transfer. MS is also accessed to set-up the conditional yielding feature.

The initialization value of this register is 0 and it consists of the following:

Table 7-19. SDMA_MS Structure

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	0	0	spriv	stype	0	0	dpriv	dtype
W																
R	0	0	0	0	y	d	e		0	0	n					
W																

Table 7-20. SDMA_MS Field Descriptions

Field	Description
31-22	Reserved

Table continues on the next page...

**Table 7-20. SDMA_MS Field Descriptions
(continued)**

Field	Description
21 spriv	The spriv value is ignored for this device. 0 = valid value 1 = Reserved
20 stype	Source Mode. Indicates if MSA has to be incremented (or not) during accesses. 0 Frozen-MSA is not modified. 1 Incremented-MSA is incremented by the number of transferred bytes during read access.
19-18	Reserved
17 dpriv	The dpriv value is ignored for this device. 0 = valid value 1 = Reserved
16 dtype	Destination Mode. Indicates if MDA has to be incremented (or not) during accesses. 0 Frozen-MDA is not modified. 1 Incremented-MDA is incremented by the number of transferred bytes during write access.
15-12	Reserved
11 y	Conditional Yielding selector. When selected, theyield/yieldge instructions will not switch channels if the Burst DMA is in Write Mode, and it has less than four bytes in its FIFO. This is aimed at reducing the number of inefficient FIFO flushes due to context switches. 0 Always yields 1 Yields conditionally (when there are less than four bytes in the FIFO in write mode)
10 d	Access Direction or DMA Mode. DMA is in write mode when data was written into MD by stf MD instructions, or if a previous DMA cycle on the external bus was a write access. Writing MDA or MSA changes the DMA mode to the respective value. DMA is in read mode when a previous DMA cycle was a read access, and DMA stays in read mode when data is read by SDMA with an ldf MD instruction. Reading MDA or MSA does not change the DMA mode. 0 Read Mode 1 Write Mode
9-8 e	Error. Indicates if the previous access was acknowledged with a bus error. 00 No error was received. 01 <i>reserved</i> 10 Error mode 11 error read burst
7-6	Reserved
5-0 n	Number of bytes in the MD FIFO.

7.2.3.12.1.5 Burst DMA Write (stf)

When received from a stf instruction, the function code bits are interpreted as follows, depending on the addressed register:

Table 7-21. STF Code Bits

Register	7	6	5	4	3	2	1	0
MSA	s		p	freeze	r			spriv
MDA								dpriv
MD			f	cpy			sz	
MS								

Table 7-22. STF Code Bit Field Descriptions

Field	Description
7-6 s	Functional Unit selector 00 for Burst DMA
5 p (MSA)	Prefetch Flag 0 No prefetch 1 Prefetch required from new MSA
5 f (MD)	Forced Flush Flag 0 Automatic flush 1 FIFO contents are flushed (including the new written data).
4 freeze (MSA/MDA)	Address Freeze Mode 0 Address is normally incremented. 1 Address is frozen.
4 cpy (MD)	Copy Mode selection 0 Write Mode 1 Copy Mode
3-2 r	Register selection 00 MSA 01 MDA 10 MD 11 MS
1-0 sz (MD/MS)	Transfer Size 00 size 0 (no data stored in the FIFO) 01 byte (8 bits) 10 half-word (16 bits) 11 word (32 bits)
0 spriv (MSA)	The spriv value is ignored for this device. 0 = valid value

Table continues on the next page...

Table 7-22. STF Code Bit Field Descriptions (continued)

Field	Description
	1 = Reserved
0 dpriv (MDA)	The dpriv value is ignored for this device. 0 = valid value 1 = Reserved

The possible write instructions are listed in the table below (unused bits should always be cleared).

Table 7-23. Burst DMA STF Instruction List

Binary	Assembly	Comments
00_0_0_00_00	stf r,MSA	Writes content of the SDMA general register (r) to the source address register. MSA is in incremented mode.
00_0_1_00_00	stf r,MSAIFR	Writes content of the SDMA general register (r) to the source address register. MSA is in frozen mode.
00_1_0_00_00	stf r,MSAIPF	Writes content of the SDMA general register (r) to the source address register, and starts a read burst access. MSA is in incremented mode.
00_1_1_00_00	stf r,MSAIPFIFR	Writes content of the SDMA general register (r) to the source address register, and starts a read burst access.
00_0_0_01_00	stf r,MDA	Writes content of the SDMA general register (r) to the destination address register. MDA is in incremented mode.
00_0_1_01_00	stf r,MDAIFR	Writes content of the SDMA general register (r) to the destination address register. MDA is in frozen mode.
00_1_0_10_00	stf r,MDISZ0IFL	No data transfers between the SDMA and MD, but all valid written data of the MD is flushed to the memory. An acknowledge or error is sent back to the SDMA core on transfer completion.
00_0_0_10_01	stf r,MDISZ8	8-bit (byte) transfer to write buffer MD
00_1_0_10_01	stf r,MDISZ8IFL	8-bit (byte) transfer to write buffer MD and flush after transfer. All valid written data of the MD is flushed to memory.
00_0_0_10_10	stf r,MDISZ16	16-bit (half-word) transfer to write buffer MD
00_1_0_10_10	stf r,MDISZ16IFL	16-bit (half-word) transfer to write buffer MD and flush after transfer. All valid written data of the MD is flushed to memory.
00_0_0_10_11	stf r,MDISZ32	32-bit (word) transfer to write buffer MD
00_1_0_10_11	stf r,MDISZ32IFL	32-bit (word) transfer to write buffer MD and flush after transfer. All valid written data of MD is flushed to memory.
00_0_1_10_00	stf r,MDICPY	No data transfer between SDMA and MD but starts a copy transfer whose length is given by the 4 LSB of r register. (Maximum burst length is eight words.)
00_0_0_11_11	stf r,MS	32-bit (word) transfer to status register MS
00_0_0_11_00	stf r,MSISZ0	Clears the error flag (if set). Other MS bits are unchanged; this instruction is also known as clref MS.

NOTE

When a flush bit is set, the SDMA flushes the FIFO including the newly written data. An acknowledge is sent to the core before the flush completes (except if size 0 is used). The goal of this flush bit is to force a flush, but it is recommended to use it only when needed (for example, when finishing a row of pixels during 2D data transfers). Indeed, if this bit is omitted and if there are more than 32 bytes in the FIFO, a burst write access is automatically triggered.

Since all the stf r,MD instructions (including the copy mode) acknowledge the SDMA core before the store is effective (except if size 0 is used), it is recommended to perform an ldf from MS before terminating a channel in order to check the final error status. (The ldf from MS will stall the core until all the data was flushed out and the transfer status is known.)

After every stf MD instruction, the MDA is incremented by the number of bytes that are written in MD, except when it is programmed in frozen mode.

7.2.3.12.1.6 Burst DMA Read (ldf)

When received from an ldf instruction, the function code bits are interpreted as follows, depending on the addressed register:

Table 7-24. LDF Code Bits

Register	7	6	5	4	3	2	1	0
MSA	s				r			
MDA								
MD			p				sz	
MS								

Table 7-25. LDF Code Bit Field Descriptions

Field	Description
7-6	Functional Unit selector
s	00 for Burst DMA
5	Prefetch Flag
p (MD)	0 no prefetch 1 automatic prefetch

Table continues on the next page...

Table 7-25. LDF Code Bit Field Descriptions (continued)

Field	Description
3-2 r	Register selection 00 MSA 01 MDA 10 MD 11 MS
1-0 sz (MD)	Transfer Size 00 reserved 01 byte (8 bits) 10 half-word (16 bits) 11 word (32 bits)

The table below lists the possible write instructions (unused bits should always be cleared).

Table 7-26. Burst DMA LDF Instruction List

Binary	Assembly	Comments
00_0_0_00_00	ldf r,MSA	Copies the source address register value into an SDMA general register. It gives the memory address of the next data that will be read with an ldf MD instruction.
00_0_0_01_00	ldf r,MDA	Copies the destination address register value into an SDMA general register. It gives the memory address where the next incoming data will be flushed.
00_0_0_10_01	ldf r,MDISZ8	8-bit (byte) read
00_1_0_10_01	ldf r,MDISZ8IPF	8-bit (byte) read. If after this reading and the MD FIFO is empty, a burst read access at the MSA address is triggered.
00_0_0_10_10	ldf r,MDISZ16	16-bit (half-word) read
00_1_0_10_10	ldf r,MDISZ16IPF	16-bit (half-word) read. If after this reading, and the MD FIFO is empty, a burst read access at the MSA address is triggered.
00_0_0_10_11	ldf r,MDISZ32	32-bit (word) read
00_1_0_10_11	ldf r,MDISZ32IPF	32-bit (word) read. If after this reading and the MD FIFO is empty, a burst read access at the MSA address is triggered.
00_0_0_11_00	ldf r,MS	Copy the status register value into an SDMA general register.

NOTE

Read data is 0-extended before writing in the SDMA general registers. When reading the MD register, the DMA takes data from the FIFO if it is available. If part or whole data is not in the FIFO, an external burst read access is performed to provide the missing data. The SDMA is stalled as long as the required read data is not complete.

After every reading, MSA is incremented by the number of read bytes from MD FIFO, except when MSA is programmed in frozen mode.

7.2.3.12.1.7 Prefetch/Flush and Auto-Flush Management-Burst DMA Unit

The prefetch and auto-flush management enables the SDMA RISC machine to go on while a DMA access is performed.

When the RISC core requires a prefetch ($p = 1$) to the Burst DMA, it will receive an immediate transfer acknowledge before the DMA has finished the external access. This enables the RISC core to do other things like accessing another DMA machine.

The basic principle in prefetch mode is for the DMA to anticipate data reads from the SDMA RISC engine by fetching external bursts of data as soon as there is enough space in the DMA FIFO to store it. If ever the RISC engine required data that is not available in the FIFO, the read acknowledge is delayed until the data is available, but it does not have to wait until the burst completes.

The auto-flush basic principle is similar: An automatic flush is triggered every time there are eight words to be written in the FIFO. If the FIFO is full and the RISC engine requires another write, it is stalled until the burst has started and enough space was freed in the FIFO to store that new data. This means the SDMA RISC engine does not have to wait for the completion of a burst to receive its acknowledge and continue its processing.

In particular, an auto-flush is executed when DMA is in write mode and if the following is true:

- If the FIFO is empty and the first write is to a word-aligned address of any size (ex: the 2 LSB of $MDA[1:0] = 0x0$), the auto-flush is triggered immediately after the write of the 32'nd byte.
- If the FIFO is empty, and if MDA is an odd byte address (1, 3, 5, 7,...) and an stf MDISZ8 is executed, the byte is flushed to memory. Once MDA increments to a word aligned address, the auto-flush will be triggered every 32 bytes.
- If the FIFO is empty, and if MDA is a half-word address (2, 6, 0xA,...) and an stf MDISZ16 is executed, the two bytes of the incoming data are flushed to memory. Once MDA increments to a word aligned address, the auto-flush will be triggered every 32 bytes.
- If the FIFO is empty, and if MDA is not a word-aligned address (ex 1, 2, 3, 5, 6, 7, 9,...), and an stf MDISZ32 is executed, the first 1 to 3 bytes will be flushed up to the next word aligned address. Afterwards, an auto-flush will be triggered each time the FIFO receives 32-bytes.

- Therefore, if an stf MD|SZ32 is executed with MDA equal to 0x1 and with an empty MD FIFO, the bytes located at addresses 1, 2, and 3 are flushed, and the byte located at address 4 remains in MD FIFO. This solves the misalignment issue. Additionally, the next write instructions (stf) complete the FIFO until it contains eight words; then a burst write is executed by the DMA to empty the FIFO. Protocol on the external bus does not support bursts of different data types (byte, half-word, or word).

For example, consider the case where data is written using a byte access, stf MD|SZ8. The value of MDA during the very first byte write determines when the auto-flush will occur as follows:

- If MDA=0x0, the flush occurs following the write of byte 32
- If MDA=0x1, the flush occurs following the write of byte 1, byte 3 and byte 35.
- If MDA=0x2, the flush occurs following the write of byte 2 and byte 34.
- If MDA=0x3, the flush occurs following the write of byte 1 and byte 33.
- If MDA=0x4, the flush occurs following the write of byte 32

The flush command forces the DMA to flush all MD valid bytes to the EXTMC controller. An acknowledge is sent immediately to the SDMA, and any potential error is reported on a future access. It is thus essential to conclude a transfer with a last read from MS, which will stall the core until all data was flushed out and returned to the transfer status (acknowledge or error).

NOTE

During this kind of auto-flush (which occurs only at the beginning of a misaligned write transfer) no acknowledge is sent back to the SDMA, which is stalled until a flush is completed.

7.2.3.12.1.8 Data Alignment and Endianness-Burst DMA Unit

7.2.3.12.1.8.1 Burst DMA in Read Mode

For every read access to MD, the data returned to the SDMA core and the new FIFO state depends on the MSA status and the access size.

The FIFO is considered as a stack of 36 bytes: Data is fetched externally on a 32-bit bus, but the valid bytes only are stored in the FIFO and left-aligned (for a transfer of consecutive words, it is only the first word that may be truncated). The following table shows the FIFO byte alignment strategy and the corresponding MSA, the returned data, and the new FIFO state for any access size of an internal read from MD.

Table 7-27. FIFO Read Configuration

Before read		Internal read access size	Read data	After read	
MSA[1:0]	FIFO state			MSA[1:0]	FIFO state
00	x0 x1 x2 x3 y0 y1 y2 y3 z0 z1 z2 z3 and so on...	sz8	00 00 00 x0	01	x1 x2 x3 y0 y1 y2 y3 z0
		sz16	00 00 x0 x1	10	x2 x3 y0 y1 y2 y3 z0 z1
		sz32	x0 x1 x2 x3	00	y0 y1 y2 y3 z0 z1 z2 z3
01	x1 x2 x3 y0 y1 y2 y3 z0 z1 z2 z3 t0 and so on...	sz8	00 00 00 x1	10	x2 x3 y0 y1 y2 y3 z0 z1
		sz16	00 00 x1 x2	11	x3 y0 y1 y2 y3 z0 z1 z2
		sz32	x1 x2 x3 y0	01	y1 y2 y3 z0 z1 z2 z3 t0
10	x2 x3 y0 y1 y2 y3 z0 z1 z2 z3 t0 t1 and so on...	sz8	00 00 00 x2	11	x3 y0 y1 y2 y3 z0 z1 z2
		sz16	00 00 x2 x3	00	y0 y1 y2 y3 z0 z1 z2 z3
		sz32	x2 x3 y0 y1	10	y2 y3 z0 z1 z2 z3 t0 t1
11	x3 y0 y1 y2 y3 z0 z1 z2 z3 t0 t1 t2 and so on...	sz8	00 00 00 x3	00	y0 y1 y2 y3 z0 z1 z2 z3
		sz16	00 00 x3 y0	01	y1 y2 y3 z0 z1 z2 z3 t0
		sz32	x3 y0 y1 y2	11	y3 z0 z1 z2 z3 t0 t1 t2

7.2.3.12.1.8.2 Burst DMA in Write Mode

For every write access to the MD, the new FIFO state depends on the MDA status and the access size.

The FIFO is considered as a stack of 36 bytes: Data is stored in the FIFO according to the internal access size and the former MDA value. The following table shows the FIFO byte alignment strategy corresponding to MDA, as well as the new FIFO state for any access size of an internal write to MD.

Table 7-28. FIFO Write Configuration

Before write		Internal write access size	Written data	After write	
MDA[1:0]	FIFO state			MDA[1:0]	FIFO state
00	tt uu vv ww ?? ?? ?? ?? ?? ?? ?? ?? and so on...	sz8	?? ?? ?? x0	01	tt uu vv ww x0 ?? ?? ?? ?? ?? ?? ??
		sz16	?? ?? x0 x1	10	tt uu vv ww x0 x1 ?? ?? ?? ?? ?? ??
		sz32	x0 x1 x2 x3	00	tt uu vv ww x0 x1 x2 x3 ?? ?? ?? ??
01	tt uu vv ww xx ?? ?? ?? ?? ?? ?? ?? and so on...	sz8	?? ?? ?? x0	10	tt uu vv ww xx x0 ?? ?? ?? ?? ?? ??
		sz16	?? ?? x0 x1	11	tt uu vv ww xx x0 x1 ?? ?? ?? ?? ??
		sz32	x0 x1 x2 x3	01	tt uu vv ww xx x0 x1 x2 x3 ?? ?? ??
10	tt uu vv ww xx yy ?? ?? ?? ?? ?? ?? and so on...	sz8	?? ?? ?? x0	11	tt uu vv ww xx yy x0 ?? ?? ?? ?? ??
		sz16	?? ?? x0 x1	00	tt uu vv ww xx yy x0 x1 ?? ?? ?? ??
		sz32	x0 x1 x2 x3	10	tt uu vv ww xx yy x0 x1 x2 x3 ?? ??
11	tt uu vv ww xx yy zz ?? ?? ?? ?? ?? and so on...	sz8	?? ?? ?? x0	00	tt uu vv ww xx yy zz x0 ?? ?? ?? ??
		sz16	?? ?? x0 x1	01	tt uu vv ww xx yy zz x0

Table continues on the next page...

Table 7-28. FIFO Write Configuration (continued)

Before write		Internal write access size	Written data	After write	
MDA[1:0]	FIFO state			MDA[1:0]	FIFO state
					x1 ?? ?? ??
		sz32	x0 x1 x2 x3	11	tt uu vv ww xx yy zz x0 x1 x2 x3 ??

NOTE

If the FIFO mode changes from a write to a read mode, all remaining written bytes in MD are lost but no error is returned. Typically, this happens if an ldf MD is executed after stf MD instructions. Before a mode change, it is recommended to force the flush of a potential remaining byte by a stfMD|SZ0|FL instruction. In the same way, if a FIFO mode changes from a read to a write mode, all prefetched data present in the FIFO is lost and no error is returned.

7.2.3.12.1.8.3 Endianness-Burst DMA Unit

Big and Little Endian are supported by the Burst DMA, but data is always stored in MD in Big Endian.

Byte manipulation is performed when data is exchanged with an Burst controller (for example, during read or write burst accesses).

7.2.3.12.1.9 Burst DMA Unit Copy Mode

A mechanism is available to perform fast Arm-to-Arm transfers.

Data does not flow through the SDMA core: It is kept in the DMA FIFO. This mechanism is selected when writing MD with a special option in the instruction code (copy flag).

It is possible to transfer up to eight words in one SDMA instruction (this does not mean in one cycle). In this mode, every time an stf MD|CPY is executed, a read burst is executed and directly followed by a write burst transfer. Burst transfers are limited to eight words. The size of the transfer (in words)-given by the SDMA general register (4 LSB)-is also limited to eight. The following SDMA code shows how 100 bytes could be copied from the MSA address to the MDA address. This is sample code only.

Burst DMA copy mode example

```

        ldi r0,@src
        stf r0,MSA                      // Source address setup
        ldi r1,@dst
        stf r1,MSA                      // Destination address setup
        ldi r0,0x64                     // data transfer counter
        ldi r1,0x8

MAIN_XFER:
        cmpbs r0,r1                    // Is r0 >= 0x8
        bf LAST_XFER                  // If not, jump to last transfer label
        stf r1,MD|CPY                  // Copy 8 words from MSA to MDA address.
        subi r0,0x8                    // Decrement counter
        jmp MAIN_XFER                  // return to main transfer loop

LAST_XFER:
        stf r0,MD|CPY

```

The main transfer loop is executed 12 times; then r0 equals 4 and the last transfer loop is run.

In this mode, an acknowledge is transmitted to the core as soon as the read burst can start; thus, a first copy instruction returns an immediate acknowledge and subsequent copy instructions will be acknowledged as soon as the previous copy has finished.

7.2.3.12.1.10 Burst DMA Unit Error Management

Another point to consider is the management of errors.

Because the DMA immediately sends an acknowledge to the RISC core (except for the stf MS|SZ0|FLS instruction), it assumes no error will occur. If an error occurs, it is flagged (transfer error acknowledge) for the following DMA access.

This should not be a problem if the DMA is used properly. The MD accesses are meant to stall the SDMA as little as possible to optimize throughput and hide calculation time. Therefore, final access to MS should be performed before closing a channel. This access waits until any pending operation is finished in the burst DMA and gather any remaining error.

In copy mode, an error could be immediately returned to the SDMA on execution of the ldf copy or stf copy instruction. It happens when MSA or MDA are not word addresses (for example, 0[4]). This is because copy mode must only be used for transferring a large packet of aligned data.

When an error is received during a *read* transfer to the external bus, which may occur during the burst accesses, the MD FIFO contains the valid beats of the burst, and the error flag of MS is set to 2'b11 (error read burst). It is possible to read MS ("n" field) to know how much valid data remains in MD and when MD is empty (after ldf instructions). The next read MD instruction sets the MS error flag to 2'b10 (error mode), and an error is sent back to the SDMA core. In error mode, it is possible to read MSA, which gives the address of the error data. Any attempt to read or write MD, or to modify MDA or MSA in error mode, gives rise to an error; therefore, an error flag must be reset by clearing MS at the end of the SDMA code section responsible for error management.

In "error read burst" mode, writing MDA, MSA, or MD, or starting a copy transfer by a stf MDICOPY instruction will cancel the error mode. The following table shows when an immediate error is sent back according to the executed instruction.

Table 7-29. Possibilities in ERROR READ BURST Mode

DMA Instruction	Immediate Error	Comments
stf rn, MD stf rn, MSA (IU IPF) stf rn, MDA stf rn, MDICOPY	NO	Error mode is reset. MSA, MDA, or MD are updated and a DMA cycle may start. For the stf MDICOPY, a copy loop is executed.
stf rn, MS	NO	MS is updated.
ldf rn, MS ldf rn, MSA ldf rn, MDA	NO	MS, MSA, and MDA could be read in ERROR READ mode without any side effects (for example, no DMA cycle is triggered).
ldf rn, MD	YES/NO	Immediate error if there is no more data available for read in the FIFO.

When an error is received during a *write* transfer, the error is reported to the next DMA access. In this case, an error is sent to the SDMA core and the DMA goes to its error mode. Reading MS gives the number of bytes that remain in MD; reading MDA gives the address of the error data. Any attempt to read or write MD, or to modify MDA or MSA in error mode, give rise to an error; therefore, an error flag must be reset by clearing MS at the end of the SDMA code section responsible for error management.

Table 7-30. Possibilities in ERROR Mode

DMA Instruction	Immediate Error	Comments
stf rn, MD stf rn, MSA stf rn, MDA	Yes	Any attempt to modify MD, MSA, MDA will raise an immediate error and burst DMA remains in error mode. When address registers are write-accessed, an error is returned.
stf rn, MS	No	This is the only way to exit error mode. MS[9:8] must be reset by an stf MSISZ0 instruction.
ldf rn, MS ldf rn, MSA ldf rn, MDA	No	MS, MSA, and MDA could be read in error mode without any side effects (for example, no DMA cycle is triggered).
ldf rn, MD	Yes	Whatever the DMA direction (read or write), an ldf rn triggers an immediate error.

7.2.3.12.1.11 Conditional Yielding-Burst DMA Unit

The standard SDMA transfer is based upon a hardware loop that has the following structure:

Hardware Loop

```

loop
load Rn,source          // can be ldf or ld
<computation>           // can be done through functional units
store Rn,dest           // can be st or stf
done 0                  // yield

```

This structure needs to be kept independent of the functional units' particularities regarding the context switch. However, there can be variations in the context switch's efficiency, which can depend on the number of data received up to that point, and on the data itself.

The DMA, with its 8-word burst capability, has a preferable context switch period when its address register is 8-word aligned: It is the only moment that occurs once every eight loops when the succession of bursts is not broken by the context switch. When this is not the case, a context switch requires the storing (or loading) of less than eight words, which requires separate accesses and is far less efficient. The rest of the 8-word packet is stored (or loaded) after the context restore, and this is done as separate accesses.

The proposed solution is a conditional yielding, which occurs only when the DMA is in an optimum state. It does not require any modification to the scripts. The condition is decided at the DMA level.

The DMA can be programmed in two modes-conditional or always-true-for every channel, which provides complete flexibility. By default, the DMA is not in conditional mode.

The DMA condition is computed from the FIFO fill level and the various modes, as follows:

- When copy mode is selected, regardless of the transfer direction ('read' or 'write'), the condition is always true.
- In read mode, the condition is always true.
- In write mode, the condition is true when there are four bytes or less in the FIFO; it is false when there are more than four bytes. The 4-byte limit comes from the possibility of saving those bytes as MD with absolutely no impact on the bus accesses.

The aim at conditional yielding is to avoid splitting bus accesses (especially bursts).

7.2.3.12.2 Peripheral DMA Unit Programming

The peripheral DMA unit is connected to the Multi-Layer DMA Crossbar Switch of the Arm platform.

Its goal is to perform data transfers between any blocks connected to the DMA bus of this platform. These blocks are either peripherals or memories. The peripheral DMA could be seen as the Arm platform DMA controller.

The DMA performs data transfers in three modes:

- Read mode, where data is read from peripherals or from memory connected to the Arm platform and copied in a SDMA general register.
- Write mode, where data of a general register has to be written in a peripheral or a memory.
- Copy mode, where data is read from a peripheral (or memory) at a source address (PSA) and automatically written to a peripheral (or memory) at a destination address (PDA).

In copy mode, no SDMA general register is involved as transferred data only goes through the data register of the DMA.

The peripheral DMA has three addressing modes: frozen, incremented, and decremented, as follows:

- Frozen mode-When source or destination addresses are frozen, their value is not modified after a transfer. This mode is typically used for addressing peripheral FIFOs located at a fixed address.
- Incremented mode-When source or destination addresses are in incremented mode, after every transfer they are incremented by the number of bytes transferred.
- Decrement mode-In decremented mode, addresses are decremented by the number of bytes transferred.

The peripheral DMA registers are as follows:

- Two, 32-bit address registers (PSA and PDA) that respectively contain the source address for a read access and the destination address for a write access
- A 32-bit status register (PS) that contains information on the peripheral DMA configuration, such as the number of valid bytes in the data register, the error flag, the source and destination address mode, and so on.
- A 32-bit data register (PD) that stores data involved in a data transfer

7.2.3.12.2.1 Peripheral Source Address Register (PSA)

The source address register contains a pointer to a source peripheral or a memory associated with the next read data transfer. It has byte granularity.

It is based on the following:

- A 32-bit register (PSA) to store the address value
- A 2-bit register (stype) to store the source address mode (frozen, incremented, or decremented)
- A 2-bit register (ssize) to store the source target data path size (byte, half-word, or word)

Reading the register with the ldf instruction has no side effects and gives the address value of the next data that will be read by the SDMA during an ldf MD instruction. Writing the source address register may have side effects. If there is valid write data in the data register and the source address is changed, the write data is discarded. If the prefetch bit is set, a DMA read cycle is issued with the new address.

When PSA is to be written, you must specify the source target address mode, providing its size (byte, half-word, or word). This enables omission of the size field in all ldf MD instructions. When DMA performs a read cycle, its size is given by the value of the PSA source size register (ssize). If source is a memory in incremented mode, first programmed in word mode (stf PSA|SZ32|I), and if an SDMA script needs to read bytes from this memory, the size of the source target must be updated before executing new accesses. The source address mode and its size are given by labels added to the stf PSA instruction as described in the write section. The ssize and stype registers are part of the DMA status register (PS).

Writing to PSA may issue an immediate error if the source size is not compatible with the value to be written into the PSA register. For instance, writing a 2 in PSA and specifying that it is memory-accessed in word mode creates an immediate error.

7.2.3.12.2.2 Peripheral Destination Address Register (PDA)

The destination address register contains a pointer to a source peripheral or a memory associated with the next write data transfer. It has byte granularity.

It is based on the following:

- A 32-bit register (PDA) to store the address value
- A 2-bit register (dtype) to store the destination address mode (frozen, incremented, or decremented)
- A 2-bit register (dsize) to store the destination target data path size (byte, half-word, or word)

Reading the register with the ldf instruction has no side effects, and gives the address value of the next data that will be written by SDMA during an stfMD instruction. Writing the destination register has no side effect. Similar to the PSA register, the destination address mode and source are specified in the stf PDA instruction and may also generate an error in case of incorrect programming.

7.2.3.12.2.3 Peripheral Data Register (PD)

The data register of the peripheral DMA is a 32-bit register. When the destination address is correctly set up, any writing to PD will automatically flush the new input data.

The number of SDMA bytes that will be transferred is given by the PDA size register. Unlike other SDMA DMAs, PD is not a FIFO: It is not used to accumulate bytes that from the SDMA and must be packed before being sent to external memories. In read mode, and if the source address is correctly set up, an ldf instruction will empty PD. If a prefetch is required along with the instruction, the DMA will initiate a new read transfer.

Reading PD in prefetch mode only stalls the SDMA when the prefetched data is not yet available. Writing PD only stalls the SDMA if the previous write operation was not completed. As soon as the previous operation is over, the acknowledge is sent back to the SDMA RISC engine.

An error flag-part of PS-is set when an external access fails. The error is thus reported to the next SDMA instruction that involves the peripheral DMA.

7.2.3.12.2.4 Peripheral State Register (PS)

The state register contains the DMA state-machine value. It can be accessed in case of an error received during a transfer.

Although all PS fields can be written by an stf instruction, it is recommended to access only the error bit (to reset it). Modifying other PS fields will provide an un-guaranteed DMA behavior.

The initialization value of PS is 0, and it consists of the following structure:

Table 7-31. PS Structure

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	ssize		stype		dsize		dtype	
W																
R	0	0	0	0	0	d	e		0	0	0	0	0	n		
W																

Table 7-32. PS Field Descriptions

Field	Description
31-24	Reserved
23-22 ssize	Source Target Size. Determines the size of the read transfers on the external bus. It should match the accessed device characteristics. 00 <i>reserved</i> 01 Byte (8 bits) 10 half-word (16 bits) 11 word (32 bits)
21-20 stype	Source address Mode. Determines whether PSA is incremented, decremented, or kept unmodified after every read from the external bus. 00 Frozen Mode 01 Incremented Mode 10 Decrement Mode 11 <i>reserved</i>
19-18 dsize	Destination Target Size. Determines the size of the write transfers on the external bus. It should match the accessed device characteristics. 00 <i>reserved</i> 01 Byte (8 bits) 10 half-word (16 bits) 11 word (32 bits)
17-16 dtype	Destination address Mode. Determines whether PDA is incremented, decremented, or kept unmodified after every write on the external bus. 00 Frozen Mode 01 Incremented Mode 10 Decrement Mode 11 <i>reserved</i>
15-11	Reserved
10 d	Direction Flag or DMA Mode. DMA is in write mode when data was written into PD by stf PD instructions, or if a previous DMA cycle on the external bus was a write access. Writing PDA or PSA does not change the DMA mode. DMA is in read mode when a previous DMA cycle was a read access, and DMA stays in read mode when data is read by the SDMA with an ldf PD instruction. Reading PDA or PSA does not change the DMA mode. 0 Read Mode 1 Write Mode
9-8 e	Error. Indicates if the previous access was acknowledged with a bus error. 00 No error was received. 01 <i>reserved</i> 10 Error mode 11 Error read
7-3	Reserved

Table continues on the next page...

Table 7-32. PS Field Descriptions (continued)

Field	Description
2-0 n	number of bytes in PD

NOTE

dtype, dsize, stype, and ssize are updated when PSA and PDA are written.

7.2.3.12.2.5 Peripheral DMA Write (stf)-Write Mode

When written by an stf instruction, the function code bits are interpreted as follows:

Table 7-33. STF Code Bits

Register	7	6	5	4	3	2	1	0
PSA	s		p	ar	am		sz	
PDA								
PD			pdsel					
PS			pssel					

Table 7-34. STF Code Bits Field Descriptions

Field	Description
7-6 s	Functional Unit selector 11 for Peripheral DMA
5 p (PSA)	Prefetch Flag 0 no prefetch 1 automatic prefetch
4 ar (PSA/PDA)	Address Register Selector 0 PSA 1 PDA
3-2 am (PSA/PDA)	Address Mode. Determines how PSA or PDA is modified after every read or write access to the PD. 00 Frozen-Address registers are not modified after the transfer. 01 Incremented-Address registers are incremented by the number of transferred bytes. 10 Decrement-Address registers are decremented by the number of transferred bytes. 11 Updated-PSA and PDA are not modified. Either address mode is not modified, but the width of the data path is updated by the sz field.
1-0 sz	Transfer Size 00 reserved

Table continues on the next page...

Table 7-34. STF Code Bits Field Descriptions (continued)

Field	Description
	01 byte (8 bits) 10 half-word (16 bits) 11 word (32 bits)
5-0 pdsel	PD access selector 001000 is the only valid option
5-0 pssel	PS access selector 111111 writes to PS 001100 only clears the error flag in PS

Due to the large number of possible stf instructions, the following table provides only a short list of all the possible write instructions:

Table 7-35. Peripheral DMA STF Instruction List

Binary	Assembly	Comments
11_00_00_01 11_00_00_10 11_00_00_11	stf Rn, PSAISZ8 IF stf Rn, PSAISZ16IF stf Rn, PSAISZ32IF	- Source is a byte, half-word, or word target at the Rn address. Any further PD read instructions will trigger a byte, half-word, or word access to the source. - Source address is frozen.
11_10_00_01 11_10_00_10 11_10_00_11	stf Rn, PSAISZ8 IFIPF stf Rn, PSAISZ16IFIPF stf Rn, PSAISZ32IFIPF	- Source is a byte, half-word, or word target at the Rn address. Any further PD read instructions will trigger a byte, half-word, or word access to the source. 1, 2, or 4 bytes are <i>fetched</i> from the peripheral source. - Source address is frozen.
11_00_01_01 11_00_01_10 11_00_01_11	stf Rn, PSAISZ8 II stf Rn, PSAISZ16II stf Rn, PSAISZ32II	- Source is a byte, half-word, or word target at the Rn address. Any further PD read instructions will trigger a byte, half-word, or word access to the source. - Source address is in incremented mode: PSA = PSA + 1, 2 or 4 after read PD.
11_10_01_01 11_10_01_10 11_10_01_11	stf Rn, PSAISZ8 IIIPF stf Rn, PSAISZ16IIIPF stf Rn, PSAISZ32IIIPF	- Source is a byte, half-word, or word target at the Rn address. Any further PD read instructions will trigger a byte, half-word, or word access to the source. - Source address is in incremented mode: PSA = PSA + 1, 2, or 4 after read PD. 1, 2, or 4 bytes are <i>fetched</i> from the peripheral source.
11_00_10_01 11_00_10_10 11_00_10_11	stf Rn, PSAISZ8 ID stf Rn, PSAISZ16ID stf Rn, PSAISZ32ID	- Source is a byte, half-word, or word target at the Rn address. Any further PD read instructions will trigger a byte, half-word, or word access to the source. - Source address is in incremented mode: PSA = PSA-1, 2, or 4 after read PD.
11_10_10_01 11_10_10_10 11_10_10_11	stf Rn, PSAISZ8 IDIPF stf Rn, PSAISZ16IDIPF stf Rn, PSAISZ32IDIPF	- Source is a byte, half-word, or word target at the Rn address. Any further PD read instructions will trigger a byte, half-word, or word access to the source. - Source address is in incremented mode: PSA = PSA-1, 2, or 4 after read PD. 1, 2, or 4 bytes are <i>fetched</i> from the peripheral source.

Table continues on the next page...

Table 7-35. Peripheral DMA STF Instruction List (continued)

Binary	Assembly	Comments
11_00_11_01 11_00_11_10 11_00_11_11	stf Rn, PSAISZ8 IU stf Rn, PSAISZ16 IU stf Rn, PSAI SZ32 IU	• <i>Update</i> source pointer to memory, which becomes a pointer to a memory accessed in byte, half-word, or word. • PSA value is not modified by Rn. • Bytes present in PD are lost.
11_10_11_01 11_10_11_10 11_10_11_11	stf Rn, PSAISZ8 IPFIU stf Rn, PSAISZ16 IPFIU stf Rn, PSAISZ32 IPFIU	• <i>Update</i> source pointer, which becomes a pointer to a target accessed in byte, half-word, or word. • PSA value is not modified by Rn. • Bytes present in PD are lost. • 1, 2, or 4 bytes are <i>fetched</i> from the memory source.
11_01_00_01 11_01_00_10 11_01_00_11	stf Rn, PDAISZ8 IF stf Rn, PDAISZ16IF stf Rn, PDAISZ32IF	• Destination is a byte, half-word, or word target at the Rn address, and any further PD write instructions will trigger byte, half-word, or word access to the destination. • Destination address is frozen.
11_01_01_01 11_01_01_10 11_01_01_11	stf Rn, PDAISZ8 II stf Rn, PDAISZ16II stf Rn, PDAI SZ32II	• Destination is a byte, half-word, or word target at the Rn address, and any further PD write instructions will trigger byte, half-word, or word access to the destination. • Destination address is in incremented mode: $PDA = PDA + 1, 2, \text{ or } 4$ after write PD.
11_01_10_01 11_01_10_10 11_01_10_11	stf Rn, PDAISZ8 ID stf Rn, PDAISZ16ID stf Rn, PDAISZ32ID	• Destination is a byte, half-word, or word target at the Rn address, and any further PD write instructions will trigger byte, half-word, or word access to the destination. • Destination address is in incremented mode: $PDA = PDA - 1, 2, \text{ or } 4$ after write PD.
11_01_11_01 11_01_11_10 11_01_11_11	stf Rn, PDAISZ8 IU stf Rn, PDAISZ16 IU stf Rn, PDAI SZ32 IU	• <i>Update</i> destination pointer to memory, which becomes a pointer to a memory accessed in byte, half-word, or word. • PDA value is not modified by Rn • bytes present in PD are lost
11_00_10_00	stf Rn, PD	• Write "dsize" bytes of Rn in PD and automatically flush to destination target
11_11_11_11	stf Rn, PS	• Write status register
11_00_11_00	stf Rn, clrefPS	• Clear error flag if set

NOTE

When writing PD, size information is not important: It is embedded in the dsize field of PDA register. If dsize is 1, 2, or 4, then one, two, or four bytes from Rn is written to the PD register, and automatically flushed out to the destination target.

7.2.3.12.2.6 Peripheral DMA Read (ldf)-Read Mode

When received from an ldf instruction, the function code bits are interpreted as follows.

Table 7-36. LDF Code Bits

Register	7	6	5	4	3	2	1	0
PSA	s			ar	a			
PDA								
PD			p	cpy				
PS			pssel					

Table 7-37. LDF Code Bits Descriptions

Field	Description
7-6 s	Functional Unit selector 11 for Peripheral DMA
5 p (PD)	Prefetch Flag 0 no prefetch 1 automatic prefetch
4 ar (PSA/PDA)	Address Register Selector 0 PSA 1 PDA
4 cpy (PD)	Copy Mode 0 standard access 1 copy mode access
3 a	Register Set selection 0 PSA or PDA 1 PD or PS
5-0 pssel	PS access selector 111111 is the only valid option to read PS

Table 7-38. Peripheral DMA LDF Instruction List

Binary	Assembly	Comments
11_0_0_0_000	ldf Rn, PSA	Reads 32-bit of PSA value
11_0_1_0_000	ldf Rn, PDA	Reads 32-bit of PDA value
11_0_0_1_000	ldf Rn, PD	Reads programmed source size bytes of PD (0-extended)
11_1_0_1_000	ldf Rn, PDIPF	Reads programmed source size bytes of PD (0-extended), and starts a prefetch at PSA address.
11_0_1_1_000	ldf Rn, PDICOPY	Starts a copy transfer from the source target at the PSA address to the destination target at the PDA address. No data transmits through Rn, but Rn contents are lost (Rn is loaded with PD temporary contents that are <i>not</i> the copied data).
11_111111	ldf Rn, PS	Reads 32-bit of PS value

NOTE

When reading PD, size information is not important: It is embedded in the ssize field of the PSA register. If ssize is 1, 2, or 4, the one, two, or four bytes is transferred from PD to Rn. Read data is 0-extended.

7.2.3.12.2.7 Peripheral DMA Unit Copy Mode

Like burst DMA, the peripheral DMA unit has a copy mode that is used when data transfers do not involve SDMA general registers.

Data is read from the source target at a PSA address, stored in PD, and then automatically flushed to the destination target at the PDA address. Copy mode is only available for transfers that involve two targets of the same data path width.

Since copy mode is invoked with an ldf instruction, the *loaded* general purpose register loses its previous contents. (However, the new contents are unpredictable as they depend on temporary values that are seen on the external DMA bus.)

7.2.3.12.2.8 Error Management

Peripheral DMA generates two kinds of errors: the immediate error that sanctioned incorrect register programming; and the error triggered by the previous access and stored in the error flag of PS until a DMA instruction is executed.

7.2.3.12.2.8.1 Immediate Errors

The following table lists all incorrect DMA register setups.

Table 7-39. Immediate Errors with Peripheral DMA

Rn[1:0] values	DMA instruction	Comments
0x01 0x11	stf Rn, PSAISZ16IF stf Rn, PSAISZ16II stf Rn, PDAISZ16IF stf Rn, PDAISZ16II	If PSA points to a half-word peripheral or to a half-word address in memory, its value must be 0 modulo 2.
0x01 0x10 0x11	stf Rn, PSAISZ32IF stf Rn, PSAISZ32II stf Rn, PDAISZ32IF stf Rn, PDAISZ32II	If PSA points to a word peripheral or to a word address in memory, its value must be 0 modulo 4.
PSA[1:0]-PDA[1:0]	DMA instruction	Comments

Table continues on the next page...

Table 7-39. Immediate Errors with Peripheral DMA (continued)

Rn[1:0] values	DMA instruction	Comments
0x01 0x10 0x11	stf Rn, PSAISZ32IU stf Rn, PDAISZ32IU	When PDA or PSA is updated and becomes a pointer to a word address in memory, its content must be 0 modulo 4.
0x01 0x11	stf Rn, PSAISZ16IU stf Rn, PDAISZ16IU	When PDA or PSA is updated and becomes a pointer to a half-word address in memory, its content must be 0 modulo 2.
Read/Write PD instruction	Comments	
stf Rn,PD ldf Rn,PD	If PDA size (dsize) has never been set up before an stf PD instruction (dsize=0) If PSA size (ssize) has never been set up before an ldf PD instruction (ssize=0)	
ldf Rn,PDICPY	Copy mode is possible only between two targets whose data path width is identical. It is P8↔P8, P16↔P16, or P32↔P32 regardless of the way the address registers are incremented.	

7.2.3.12.2.8.2 Data Transfer Errors

When PSA and PDA are correctly set up, the only error that may arise for an ldf PD or stf PD instruction would be the error of the previous DMA cycle.

Error handling is driven by a single consideration: When an error occurred during a data read on the DMA interface, this error should appear as a transfer error to the core when the core attempts to retrieve the data that was not successfully read from the accessed device (memory or peripheral).

When an error occurred during a write access to the DMA interface, the data is still available in PD and should not be destroyed by subsequent core accesses: The core must be warned about the error issue.

There are three error handling mechanisms for each case: [Read Error \(First Phase\)](#), [Write Error and Read Error \(Second Phase\)](#), and [Copy Mode Errors](#) handling.

7.2.3.12.2.8.3 Read Error (First Phase)

If an error occurred during a prefetch command, the peripheral DMA enters its ERROR READ mode (PS[9:8]=11). In this mode, the error is reported on the next ldf PD instruction and writing PSA, PDA, or PD will cancel the error flag.

The block returns no error mode and instructions are normally executed (a DMA cycle may be triggered). Similarly, initiating a copy transfer will reset the error flag and start a copy transfer. The following table details which instructions can be executed in this mode.

Table 7-40. Possibilities in ERROR READ Mode

DMA Instruction	Immediate Error	Comments
stf rn, PD stf rn, PSA (IU IPF) stf rn, PDA ldf rn, PDICOPY	NO	Error mode is reset, PSA or PDA are updated, or a write cycle is started. For the ldf PDICOPY, a copy loop is executed.
stf rn, PS	NO	PS is updated.
ldf rn, PS ldf rn, PSA ldf rn, PDA	NO	PS, PSA, and PDA could be read in ERROR READ mode without any side effects (for example, no DMA cycle is triggered).
ldf rn, PD	YES	Error of the previous read access is reported here and the peripheral DMA enters its ERROR mode.

7.2.3.12.2.8.4 Write Error and Read Error (Second Phase)

The peripheral DMA enters its ERROR mode (PS[9:8]=10) when the previous DMA write cycle failed, or, as explained in [Read Error \(First Phase\)](#), when an ldf PD is executed while the block is in ERROR READ mode. When a DMA cycle failed, address registers (PSA, PDA) are not modified and continue to point to the problematic address. In ERROR mode, stf instructions may raise an immediate error, and ldf instructions will not (as detailed in the table below).

Table 7-41. Possibilities in ERROR Mode

DMA Instruction	Immediate Error	Comments
stf rn, PD stf rn, PSA stf rn, PDA	YES	Any attempt to modify PD, PSA, or PDA will raise an immediate error, and the peripheral DMA stays in ERROR mode. When address registers are write accessed, an error is returned.
stf rn, PS	NO	This is the only way to exit the ERROR mode. PS[3] must be reset by an stf PS instruction.
ldf rn, PS ldf rn, PSA ldf rn, PDA	NO	PS, PSA, and PDA could be read in ERROR mode without any side effects (for example, no DMA cycle is triggered).
ldf rn, PD	YES	Whatever the DMA direction (read or write), an ldf rn, PD instruction will show an immediate error.

7.2.3.12.2.8.5 Copy Mode Errors

Because copy mode is a write access that follows a read access, there are two possible cases of bus error.

When the read access incurs a bus error, the peripheral DMA behaves exactly as described in [Read Error \(First Phase\)](#) and [Write Error and Read Error \(Second Phase\)](#) : It enters its ERROR READ mode, and so on.

When the error occurred during the write access of the copy transfer, the DMA enables the core to retrieve the data that was read because it is assumed the read from the peripheral removed the data from its source device. Therefore, the data to be flushed is still in PD. Any subsequent access to PD triggers an error to the core, which should execute its error handling procedure.

Once the ERROR mode is left (after writing to PS), it is possible for the core to retrieve the data in PD with an ldf instruction or try to flush PD contents once again (for example, when the error was due to a full FIFO and the script waited for the FIFO to be emptied) with another ldf instruction in copy mode. This latter instruction detects that there is valid data in PD, tries to flush it, and thus skips the read phase of the copy instruction. This is a different behavior from the usual stf PD instruction that overwrites PD with the selected General Purpose register contents. The same mechanism can be used any time PD holds data that is not written because of a bus error on the DMA interface; when the data was written via a copy instruction, or via the usual stf PD instruction.

7.2.3.12.2.8.6 Error Check Example

The following code illustrates an example checking for both immediate and data transfer errors on a store to the PD register. The first bdf instruction checks for an immediate error, but if a data transfer error occurred it is reported until the next instruction to access the Peripheral DMA. A second check of the error flags is done after the ldf PS instruction. The value of PS here can be ignored. The act of reading any register in Peripheral DMA while it is in an error mode that returns the error to the core to set either the SF or DF flag. Any error returned on an ldf command sets the SF flag and any error returned on an stf instruction sets the DF flag. This can create a situation as shown in the example where a bus error during a DMA write which would normally be considered as a destination fault is reported as a source fault because the error was reported to the SDMA core during an ldf instruction.

Table 7-42. Peripheral DMA Error Check

Function	Instruction	Comment
clrf	0	Clear SF and DF flags
stf	R4, PD	Write data to memory
bdf	error_routine	Check for immediate error from write to PD.
ldf	r3, PS	Read PS (PS value in R3 can be ignored)

Table continues on the next page...

Table 7-42. Peripheral DMA Error Check (continued)

Function	Instruction	Comment
bsf	error_routine	Check for bus error from "stf R4,PD". SF is set because it is a ldf instruction, even though the original error was a destination fault.

7.2.3.12.2.9 Peripheral DMA Unit Prefetch/Flush Management

There is no flush bit because every time data is stored in PD by a stf PD instruction—assuming PDA is correctly programmed—it is automatically flushed to the destination.

An acknowledge is returned in the cycle of the DMA instruction, and the SDMA is only stalled by an instruction that addresses the peripheral DMA when the previous DMA access is not over.

7.2.3.12.3 OnCE and Real-Time Debug

The On-Chip Emulation block (OnCE) is the debug interface to the SDMA.

It supports the access to all core internal devices (registers, memory, and so on), and provides a set of mechanisms that control the core. The OnCE is accessed by JTAG ports at the chip's board level, or by the host via its peripheral bus.

To reduce the size of the hardware material involved, all tasks supported by the OnCE are performed on the SDMA core. The architecture of the SDMA OnCE is relatively simple and very flexible.

The commands supported by the SDMA OnCE are listed in the following sections.

7.2.3.12.3.1 Memory and Register Access

A set of mechanisms is provided to access SDMA memory and register locations. Both reading and writing are allowed. The access is supported if the processor is in debug mode.

Those registers can also be accessed through the Arm platform Control interface when the OnCE is controlled by the Arm platform, as described in the "Using BP" section.

7.2.3.12.3.2 Hardware Breakpoints

An event detection unit is implemented to support memory breakpoints. The unit watches the data exchanged between the SDMA memory bus and the core.

A debug request is sent to the core when matching conditions occur. The unit supports mixed conditions based on address range, access type, and data value. Event detection unit configuration registers are memory mapped in the SDMA space (see [Arm platform Channel 0 Pointer \(SDMAARM_MC0PTR\)](#)): You can modify them through a regular memory access or the Arm platform control interface.

7.2.3.12.3.3 Watchpoints

One output pin is provided to monitor matching trigger conditions that are defined in the event detection unit.

7.2.3.12.3.4 Software Breakpoints

The SDMA instruction set contains a software breakpoint. Upon executing a software breakpoint instruction, the core suspends normal execution and enters debug mode.

No hardware step execution mode is implemented in the OnCE, but this feature may be implemented at the software level with this instruction.

7.2.3.12.3.5 Core Control

Commands are provided to monitor and control processor activity. You can halt the core, rerun the core from another address location, and get processor status.

Any hardware breakpoint on the instruction bus is not supported, but this feature may be implemented by inserting a software breakpoints program.

7.2.3.13 The OnCE Controller

The OnCE controller receives commands from the Arm platform or from the JTAG controller. Each command is interpreted before being sent to the core.

7.2.3.13.1 OnCE Commands

A small set of commands supports the communication between the OnCE and the external world.

This command set enables you to perform any of the following tasks: control processor activity, save core context, and execute an SDMA instruction from the OnCE. Combined together, these tasks perform more complex commands.

A full OnCE command contains a 4-bit instruction (the OnCE command opcode) and a variable length data field (the OnCE data). During command execution, the OnCE data is transferred in a OnCE internal register before being exchanged with the SDMA. Some data values are also exported. This mechanism creates a link between the processor and the external world. Nine commands are defined: The following table presents their formats.

Table 7-43. OnCE Command Opcode Values

Instruction Opcode	Name	Action	Register	Data Field Size	Mode
0000	rstatus	Reads the OnCE status register	STATUS	16-bit	normal/debug
0001	dmov	Updates general register GReg1	GREG1	32-bit	debug
0010	exec_once	Runs the instruction from the SDMA instruction register	INSTRUCTION	16-bit	debug
0011	run_core	Returns to normal execution	BYPASS	1-bit	debug
0100	exec_core	Returns to normal execution via a jump instruction that specifies the new address	INSTRUCTION	16-bit	debug
0101	debug_rqst	Stops the core after execution of current instruction	BYPASS	1-bit	normal
0110	rbuffer	Reads the real time buffer	RTB	32-bit	normal/debug
0111-1110	reserved	Reserved	BYPASS	1-bit	normal/debug
1111	bypass	Bypasses TARM platform controller	BYPASS	1-bit	normal/debug

Each instruction corresponds to a specific action performed on the OnCE. The nature of the associated data field is clearly identified. The dmov command is followed by a 32-bit data value (which is a data value for the SDMA); the exec_once and the exec_core commands are followed by a 16-bit data value (which is an instruction for the SDMA); the rstatus command is followed by a 16-bit control value (which is the content of the OnCE status register); the rbuffer command is followed by a 32-bit data value. The debug_rqst and the run_core commands are followed by a single bit data field (this is a bypass value). Finally, the bypass instruction enables the SDMA JTAG TAP controller to be daisy-chained with another JTAG TAP controller. This is a JTAG-only feature. The set of commands is simple, but enables you to perform any possible task on the SDMA during a debug process.

7.2.3.13.2 Sending Commands to the OnCE Controller

The JTAG access is the standard access to the OnCE, but sometimes the JTAG is not available to fix some bugs (if the chip is in production for instance), an additional access is then required. Therefore, one Arm platform access to the OnCE is provided.

7.2.3.13.2.1 Using the JTAG Interface

A serial access is performed through the five JTAG pins TCK, TRST, TMS, TDI, and TDO. A Test Access Port controller is provided to decode the TMS control signal.

It produces shift-enable signals (shift_ir and shift_dr), and updates enable signals (update_ir and update_dr). It is fully compliant with the IEEE 1149.1 testability (JTAG) standard.

During the shift_ir state, the command opcode is shifted into the OnCE controller (for example, the signal from the TDI pin is shifted into the command register and the TDO pin receives the signal shifted out). After transferring the four bits of the command, an update_ir signal is asserted and the command is decoded. The target data register is now clearly identified and the corresponding control signal is produced, as follows: bypass enable signal (bp_en), instruction enable signal (inst_en), data enable (data_en), and status enable signal (stat_en).

During the shift_dr state, the TDI signal is shifted into one of the following target registers: bypass register (1 bit), SDMA instruction register (16 bits), SDMA data register (32 bits), or OnCE status register (16 bits). The TDO pin is connected to the output of the selected register to receive the signals shifted out.

The JTAG access is disabled when the Arm platform access is enabled.

7.2.3.13.2.2 Using the Arm platform

The Arm platform access to the OnCE is not the standard access, but it is required if the JTAG is not available.

For example, if the SDMA ROM is out of use on a chip in production, and the Arm platform needs to download new code and restart the SDMA, the OnCE can easily perform this operation. This type of debug operation justifies the use of an Arm platform access to the OnCE.

To drive the OnCE, the Arm platform uses some registers contained in the Arm platform Control block of the SDMA. These registers are accessed through the Arm platform peripheral bus. Most of these registers are connected to another register in the OnCE controller. Thus, accessing one of these registers is equivalent to accessing the associated register in the OnCE controller.

The set of registers in the Arm platform Control block is listed below:

- **ONCE_ENB** register (1 bit, read/write)-This 1-bit register enables the Arm platform access to the OnCE. When this bit is set, the signals from the JTAG are ignored. When it is cleared, all writing operations to the following registers through the Host Control interface are ignored. This register is reset on a JTAG reset.
- **ONCE_CMD** register (4 bits, read/write)-This 4-bit register receives the command opcode. It is connected to the command register in the controller. A write access to this register causes the associated command to be executed on the OnCE. For example, after writing "0001" in this register, a `dmov` command is executed.

NOTE

On the Arm platform side, the `rstatus` and `bypass` commands are not supported. This register is reset on a JTAG reset.

- **ONCE_DATA** register (32 bits, read/write)-This 32-bit register is connected to the SDMA data register. This register is used when executing a `dmov` or `rbuffer` command.

NOTE

Before requesting a `dmov` command, the 32-bit data to transfer must be written in the **ONCE_DATA** register. At the end of the execution, the register is updated with `GReg1` former value. This register is reset on a JTAG reset.

- **ONCE_INSTR** register (16 bits, read/write)-This 16-bit register is connected to the SDMA instruction register. This register is used when executing an `exec_core` or an `exec_once` command.

NOTE

Before requesting an `exec_core` or an `exec_once` command, the appropriate instruction must be written in the **ONCE_INSTR** register. This register is reset on a JTAG reset.

- ONCE_STAT register (16 bits, read only)-A read access to the ONCE_STAT register returns the content of the OnCE status register (OSTAT). This register is read only.
- The bypass register is not useful when the Arm platform controls the OnCE, therefore no register is defined in the Arm platform Control block to access the bypass register.

7.2.3.13.2.3 Conflicts Between the JTAG and the Arm platform Accesses

When Arm platform access to the SDMA OnCE is enabled (that is, when the bit in the ONCE_ENB register is set), the JTAG access is disabled. This guarantees that the block is not accessed at the same time on both sides.

It is possible to check whether the JTAG access to the SDMA OnCE is enabled from the JTAG port. When the JTAG access is disabled, the SDMA TDO always returns 1. The check requires the following steps:

- Execute a dmov command from debug mode (with neither 0xffffffff nor 0x0 as dmov value: 0x5a5a5a5a is good).
- Execute another dmov command (the value here is not important).
- The returned value from the latter dmov command should be the original one if the JTAG access is enabled; if it is 0xffffffff instead of the original input value, this means the JTAG access is disabled.

7.2.3.13.3 Executing a Command from the OnCE

All the commands defined in [OnCE Commands](#) can be accessed through the JTAG. The Arm platform can access all these commands except the rstatus command.

On the Arm platform side, the OnCE status is directly accessed by reading the ONCE_STAT register.

7.2.3.13.3.1 Nature of the Commands

Two types of commands may be distinguished. First, there are two commands that do not interact with the core: rstatus and rbuffer. Those commands may be requested at any time: They do not depend on the core status.

NOTE

Each of these commands exports a data value or a status value from the SDMA.

There are also commands that interact with the core: `dmov`, `run_core`, `exec_core`, `exec_once`, and `debug_rqst`. These commands are core status dependent, as follows:

- During user mode only the `debug_rqst` is taken into account.
- During debug mode, all these commands are taken into account except the `debug_rqst`. For example, an `exec_once` command requested while not in debug mode has no effect.

7.2.3.13.2 Execution Request

The SDMA starts executing a task in debug mode when requested by the OnCE controller. The execution starting time depends on the type of access used to communicate with the OnCE.

If the JTAG is used, the request is sent after decoding the `update_dr` state in the TAP controller. Therefore, always cross this state when sending a command through the JTAG. If the OnCE is driven from the Arm platform side, the request is sent after detecting a write access to the `ONCE_CMD` register. All the registers involved in this operation must be loaded first.

The following is an example of an `exec_core` command execution from the Arm platform side: After writing '010' in the `ONCE_CMD` register, the OnCE controller asks the SDMA to execute the instruction contained in the `ONCE_INSTR` register. The instruction involved should be available in the `ONCE_INSTR` register before the beginning of the execution.

7.2.3.13.3 Command Execution

The following list shows the commands and details how each command is executed:

- `rstatus` command execution-The `rstatus` command exports the content of the OnCE status register (OSR). If the JTAG is used, the status information is captured in the OnCE status register during the `capture_dr` state, and shifted out after 16 TCK clock cycles in the `shift_dr` state. The `rstatus` command is not supported on the Arm platform side, but a status register is provided instead. The `rstatus` may be performed in both debug and user modes.
- `dmov` command execution-The `dmov` command accesses SDMA internal registers. Executing a `dmov` instruction exchanges the 32-bit data values between the SDMA data register and the general register `GReg[1]`.
- If the JTAG is used, the content of `GReg1` is captured in the SDMA data register during the `capture_dr` state, then it is shifted out after 32 TCK clock cycles in the `shift_dr` state. During the `update_dr` state, `GReg1` is updated with the new, shifted-in

32-bit data value. If the OnCE is driven from the Arm platform side, the data values contained in GReg1 and the SDMA data register are exchanged after detecting a write access to the ONCE_CMD register. The ONCE_DATA register must therefore be loaded first.

- **exec_once command execution**-The **exec_once** command executes the instruction loaded in the SDMA instruction register. The command may only be requested from debug mode. The SDMA returns to debug mode at the end of the execution.
- **Change of flow instructions as well as instructions that may cause a context switch are not supported:** The comprehensive list comprises **done/yield/yiedge** (except **done 5**), **BF, BT, BSF, BDF, JMP, JSR, JMPR, JSRR, RET, and LOOP**, as well as all the illegal instructions.

No other command should be requested before the SDMA returns to debug mode. The SDMA status (for example, whether it is in debug mode or not) can be detected by polling with the **rstatus OnCE** command, monitoring the **debug_mode** pin, or checking the [OnCE Status Register \(SDMAARM_ONCE_STAT\)](#) register via the Arm platform control interface.

NOTE

Most of the instructions are single-cycle, which omits the step of polling the status. Loads and stores to DMA units are typical instructions that might require this polling.

If the JTAG is used, the 16-bit instruction is shifted in the SDMA instruction register after 16 TCK clock cycles in the **shift_dr** state. A request is sent to the core when the **update_dr** state is decoded in the TAP controller. If the OnCE is driven from the Arm platform side, the request is sent to the SDMA when detecting a write access to the ONCE_CMD register. The ONCE_INSTR register must be therefore be loaded first.

- **run_core command execution**-The **run_core** command leaves debug mode and resume normal program execution. The next instruction executed is the last instruction decoded before entering debug mode. Be sure to restore core context before re-running the core. This procedure is detailed in [Restoring the Context](#).
- If the JTAG is used, a 1-bit bypass value is shifted in the bypass register in the **shift_dr** state. The SDMA is rerun when the **update_dr** state is decoded in the TAP controller. If the OnCE is driven from the Arm platform side, the core is rerun when detecting a write access to the ONCE_CMD register.
- **exec_core command execution**-The **exec_core** command resumes program execution from any address. The 16-bit instruction provided with the **exec_core** overwrites the last instruction decoded before entering debug mode. This command is designed to support change of flow instructions, so that a program execution can be restarted

from any address. After executing an `exec_core` command, the SDMA leaves debug mode. The `exec_core` command is usually used with a `jmp` instruction.

- If the JTAG is used, the 16-bit branch instruction is shifted in the SDMA instruction register after 16 TCK clock cycles in the `shift_dr` state. The SDMA is rerun when the `update_dr` state is decoded in the TAP controller. If the OnCE is driven from the Arm platform side, the SDMA reruns when detecting a write access to the `ONCE_CMD` register. The `ONCE_INSTR` register must therefore be loaded first. For example, to restart the SDMA from the program address `0x100`, the instruction loaded should be a jump to address `0x100` instruction.
- `debug_rqst` command execution-The `debug_rqst` command puts the SDMA in debug mode. If the JTAG is used, a 1-bit bypass value is shifted in the bypass register during the `shift_dr` state. A debug request is sent to the SDMA when the `update_dr` state is decoded in the TAP controller. If the OnCE is driven from the Arm platform side, the debug request is sent when detecting a write access to the `ONCE_CMD` register. When the SDMA is already in debug mode, this command is simply ignored.
- `rbuffer` command execution-The `rbuffer` command exports the content of the real time buffer (RTB). If the JTAG is used, the content of the real time buffer (RTB) is captured in the SDMA data register during the `capture_dr` state. The register is completely shifted out after maintaining the `shift_dr` state during 32 TCK clock cycles. If the OnCE is driven from the Arm platform side, the content of the RTB is captured in the `ONCE_DATA` register after detecting a write access to the `ONCE_CMD` register.
- `bypass` command execution-This command is only available from the JTAG interface. It enables daisy-chaining of the SDMA JTAG TAP controller with other JTAG TAP controllers. This command does not change the SDMA state and can be executed in any mode (run, debug, or sleep). It selects the bypass register of the TAP controller.

7.2.3.13.4 Registers Descriptions

See [SDMACORE](#), and [SDMAARM](#), for detailed information on each register.

7.2.3.13.4.1 Event Cell Counter Register (ECOUNT)

The event cell counter register is a 16-bit register that contains the number of times minus one that an event detection occurs before generating a debug request.

This register should be written before attempting to use the event detection counter during an event detection process. The event cell counter register is cleared on a JTAG reset.

7.2.3.13.4.2 Event Cell Address Registers (EAA or EAB)

The event cell contains two address registers-the event cell address register (a), called EAA, and the event cell address register (b), called EAB. Every address register is a 16-bit register that stores a user-defined address value. This value computes one of the following address conditions: `addra_cond` or `addrb_cond`. Every address register is cleared on a JTAG reset.

7.2.3.13.4.3 Event Cell Address Mask Register (EAM)

The event cell address mask register is a 16-bit register that contains a user-defined address mask value. This mask is applied to the address value latched from the memory address bus before comparing addresses.

NOTE

There is a common address mask value for the two address comparators. If bit *i* of this register is set, then bit *i* of the address value latched from the memory bus does not influence the result of the address comparison. The event cell address mask register is cleared on a JTAG reset.

7.2.3.13.4.4 Event Cell Data Register (ED)

The event cell data register is a 32-bit register that contains a user-defined data value. This data value is an input for the data comparator, which generates the `data_cond` condition.

The event cell data register is cleared on a JTAG reset.

7.2.3.13.4.5 Event Cell Data Mask Register (EDM)

The event cell data mask register is a 32-bit register that contains a user-defined data mask value. This mask is applied to the data value latched from the memory bus before comparing data.

Setting bit *i* of the event cell data mask register means that bit *i* of the data value latched from the address bus does not influence the result of the data comparison. The event cell data mask register is cleared on a JTAG reset.

7.2.3.13.4.6 Real Time Buffer Register (RTB)

The real Time Buffer register is a 32-bit register that stores and retrieves run-time information without putting the SDMA in debug mode.

Refer to [Real Time Buffer](#) for more details.

7.2.3.13.4.7 Event Control Register (ECTL)

The event cell control register is a 16-bit register that defines cell event occurrence conditions.

The event cell control register is cleared on a JTAG reset. See also [OnCE Event Detection Unit](#) for more details.

7.2.3.13.4.8 Trace Buffer (TB)

The Trace Buffer register retrieves the information in the Trace Buffer.

See [Trace Buffer](#) for more details.

7.2.3.13.4.9 OnCE Status Register (OSTAT)

The OnCE status register is a 16-bit register that contains processor and event detection unit status. The OSTAT is a read-only register.

Refer to [OnCE Status Register \(SDMAARM_ONCE_STAT\)](#) for detailed description of the individual fields in the OSTAT register.

The following figure shows the OSTAT structure.

Table 7-44. OnCE Status Register (OnCE)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PST[3:0]				RCV	EDR	ODR	SWB	MST					ECCR[2:0]		

Where PST[3:0] is the SDMA core state, RCV is set when the real-time buffer (RTB) is modified. EDR, ODR, and SWB are set, respectively, when the SDMA has entered debug mode because of an external debug request, a OnCE debug_rqst command, or a software breakpoint. MST is set when the OnCE is controlled from the Arm platform control interface, and when ECCR is a three-flag set that shows the event cell condition(s) that put the core in debug mode. The OSTAT never provides more than one reason for entering debug mode.

There are two ways of accessing OSTAT content, as follows:

1. Send an rstatus command to the OnCE controller through the JTAG, or read the ONCE_STAT register through the Arm platform access. Executing the rstatus command through the JTAG can be performed in both user and debug modes.
2. Perform an SDMA read access to the location in the SDMA core memory map (OSTAT register) debug mode using the exec_once command. With this method of access, the SDMA state reflected by the PST (processor status bit) is always DATA.

The register may also be accessed by a running application.

7.2.3.13.5 JTAG Interface Requirements

Because the signals received from the JTAG (running on TCK) are transferred to the OnCE controller (running on the SDMA clock), a synchronization mechanism is required.

7.2.3.13.5.1 TCK Speed Limitation

In the JTAG top-level layer, the TDO signal is always captured on a TCK falling edge. To guarantee a stable TDO signal from the SDMA during this operation, a falling edge detection is performed on TCK.

Before being latched in the *I* flip-flop (see [Figure 7-11](#)) on TCK falling edge, the TDO signal must be stable at the input of the flip-flop. This condition is verified if the TCK period is superior to the following delay:

worst-case edge detection delay + negative-edge signal propagation delay + JTAG top-level logic propagation delay

The frequency relationship, $TCK < CLK/8$, limitation guarantees that all operations are performed as expected.

7.2.3.13.5.2 Synchronization Implementation

The figure found here shows the synchronization mechanism.

Flip-flops tck0, tck1, and tck2 perform falling- and rising-edge detections on TCK. They generate the posedge_detected and negedge_detected nets that are used to sample the TDI and TMS inputs into the respective tdi and tms flip-flops, and update the tdo flip-flop to yield the TDO output. In the design, the only signal that might go metastable is the output of the tck0 flip-flop. This signal is captured in the tck1 flip-flop and no logical operation is performed on it to minimize a metastability propagation risk.

The TDI and TMS flip-flops also cannot go metastable: The propagation time of the rising-edge detection signal through tck0, tck1, and tck2 guarantees that the TDI and TMS inputs are stable when captured in the TDI and TMS flip-flops.

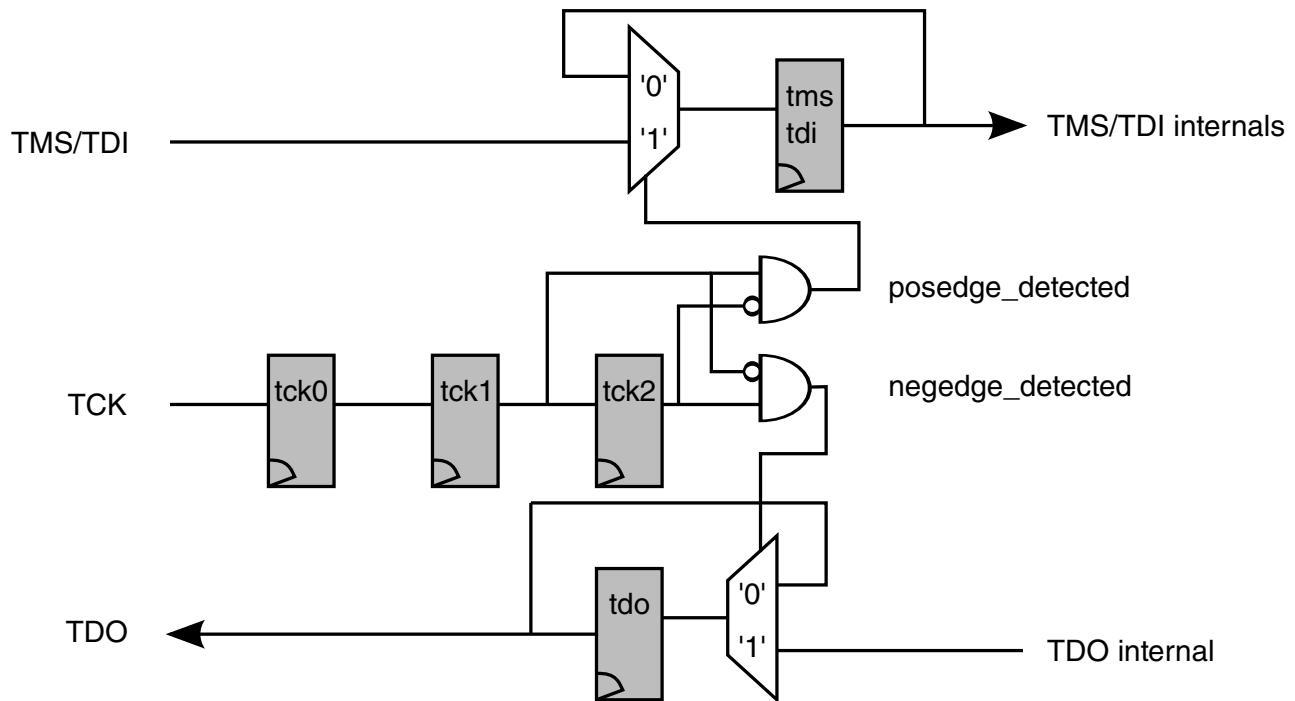


Figure 7-11. OnCE Synchronization Layer

The following figure shows synchronization timings. It takes three CLK clock cycles to synchronize TDI on the SDMA clock.

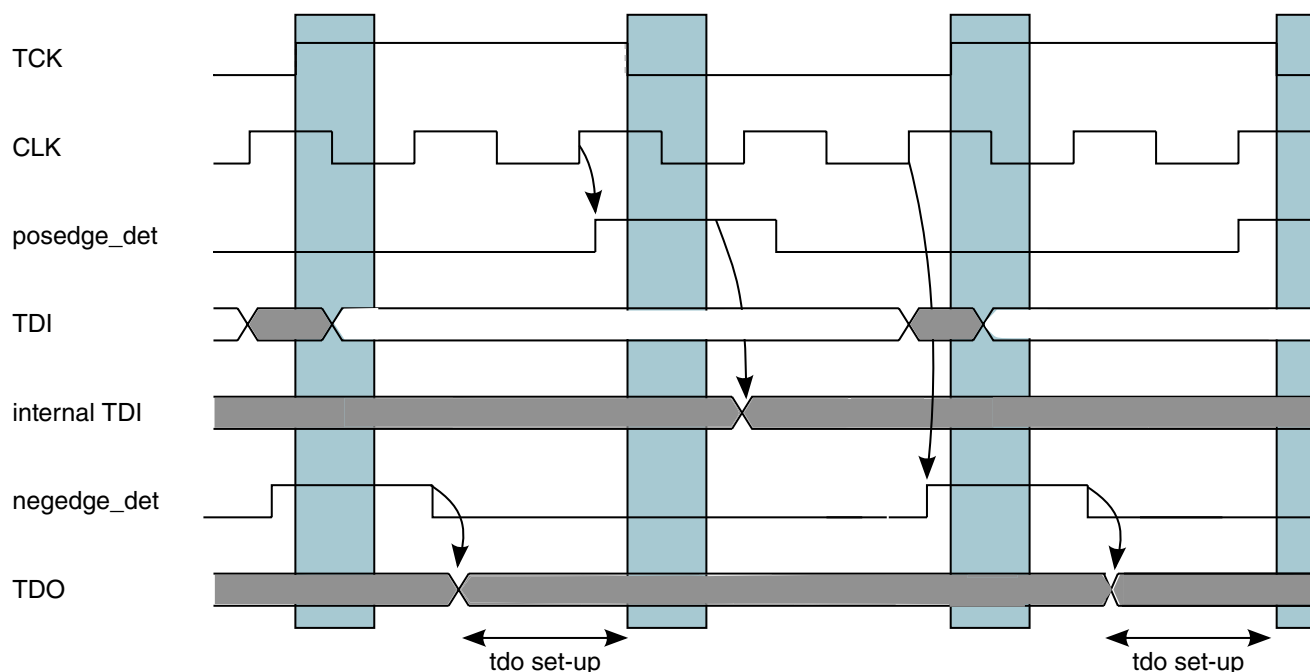


Figure 7-12. Synchronization Timings

7.2.3.13.5.3 JTAG Controller Start-Up Recommended Procedure

To ensure correct TAP controller initialization, it is recommended to use the following procedure:

1. Assert JTAG reset TRSTB (for example, set low).
2. Set TMS low.
3. Wait for 1 TCK clock.
4. Release JTAG reset TRSTB (for example, set high).
5. Wait for a minimum of five TCK cycles.

7.2.3.14 Using the OnCE

This section provides the elements necessary to run the OnCE during a debug process.

In addition to the basic set of commands described in [OnCE Commands](#), more complex commands can be built to meet users' requirements.

7.2.3.14.1 Activating Clocks in Debug Mode

For power consumption issues, some clocks in the SDMA are disabled when not needed.

This is the case for instances when the SDMA is in sleep mode. Clock gating management depends on the interface used to control the OnCE.

- For the JTAG access, the SDMA clock gating must be turned off via the `clk_gating_off` input.
- For the Arm platform access, the SDMA clock gating is automatically turned off when the Arm platform access is enabled (see [OnCE Enable \(SDMAARM_ONCE_ENB\)](#)).

7.2.3.14.2 Getting the Current Status

Most of the commands the OnCE supports have an impact on the status of the SDMA.

It is not permissible to request the execution of an instruction on the SDMA from the OnCE while the SDMA is not in debug mode. Such a violation may cause unpredictable behavior, and it might be necessary to reset the SDMA.

Therefore, the value of the PST bits provided in the OnCE status register should always be checked before sending any request to the SDMA.

7.2.3.14.3 Methods of Entering Debug Mode

A debug request may be asserted at any time, but it is not always taken into account immediately. Debug mode cannot be entered in the middle of an instruction, or during the save or restore states of a context switch.

The request is ignored when the core is already in debug mode. Refer to [Figure 7-4](#), which shows all possible transitions to the debug state, as there are several ways to enter debug mode.

7.2.3.14.3.1 External Debug Request During Reset

To enter debug mode after exiting reset, the external debug line has to be maintained high. This line is handled by the JTAG top-level block.

NOTE

The SDMA detects the debug requests only if the SDMA clock is running (see [Activating Clocks in Debug Mode](#)). The debug request line should not be maintained high when the SDMA is in debug mode.

NOTE

The `debug_rqst` command (from the OnCE command set) is not supported during system reset.

7.2.3.14.3.2 Debug Request During Normal Activity

During normal activity, the SDMA enters debug mode when the following is true:

1. If the debug request line from the JTAG top-level is asserted, or
2. If the OnCE controller receives a debug_rqst command.

The debug_rqst command can be sent by the JTAG access or by an access on the Arm platform side (if the Arm platform access is enabled).

7.2.3.14.3.3 Software Breakpoint Instruction

The SDMA enters debug mode at the end of the execution of a software breakpoint instruction. This instruction must be inserted in program flow executed by the core.

7.2.3.14.3.4 Event Detection Unit Matching Condition

If the event detection is enabled, a debug request is sent to the core after detecting a matching condition on the SDMA memory bus.

See [OnCE Event Detection Unit](#) for more details.

7.2.3.14.4 Executing Instructions in Debug Mode

The OnCE supports a mechanism to execute instructions in debug mode. If the SDMA is in debug mode, then the exec_once command can be used to execute an SDMA instruction from the OnCE controller. The SDMA returns to debug mode at the end of each execution.

Some instructions are not supported by the exec_once command: done/yield/yiedge (except done 5), BF, BT, BSF, BDF, JMP, JSR, JMPR, JSRR, RET, and LOOP, as well as all the illegal instructions are not supported.

NOTE

While instructions are executed in debug mode from the OnCE, the program counter of the SDMA is not incremented.

7.2.3.14.5 Command Sequences Examples

This section provides examples of command sequences that run the SDMA in debug mode. These sequences are available for both the Arm platform and JTAG accesses.

The following presents the syntax used in this section. The data field provided with each command is put in parenthesis with the command name. A '-' is used if the data field provided is a *don't care* value.

```
my_command(data_field);           // executing my_command with a data field
my_command(-);                   // executing my_command with a don't care data field
```

The value returned by the command (if there is one) is referred by an assignment. In case the value returned by the command is not used, the assignment is omitted. For an Arm platform access, the value returned (it is always a data value) is obtained by reading back into the SDMA data register.

```
data_out = my_command(data_in); // returning a data value
```

To clarify the syntax, the instructions' opcodes are referred to by their names. In practice, use the corresponding 16-bit encoding.

7.2.3.14.5.1 Getting the SDMA Status

NOTE

Before executing any command that affects the SDMA (like `dmov` or `exec_once`), check that the SDMA is in debug mode.

Use the following snippet:

```
rstatus();           // read SDMA status until the SDMA is in debug mode
...
rstatus();
```

If the SDMA is not in debug mode, then a debug request must be generated. In this case, the SDMA enters debug mode at the end of the execution of the current instruction. Use this snippet:

```
debug_rqst(-); // debug request
```

In the following sections, it is assumed that the SDMA was successfully put into debug mode.

7.2.3.14.5.2 Saving the Context

The first debug task is to save the SDMA context, which is the content of the eight general-purpose registers, the loop and PC-related registers, and the flags.

Use the general register `GReg[1]` as an intermediate register to export the entire context of the SDMA.

The following example shows how to save GReg[0], GReg[1], GReg[2] and GReg[3]. The sequence of commands used to export additional general registers is very similar to this.

Save GReg[0], GReg[1], GReg[2], and GReg[3]

```
GReg1_data = dmov(-);           // the value exported is the content of
GReg[1]                                           // puts the content of GReg[0] into
exec_once("mov GReg1, GReg0");
GReg[1]                                           // the value exported is the content of
GReg0_data = dmov(-);           // puts the content of GReg[2] into
GReg[0]                                           // the value exported is the content of
exec_once("mov GReg1, GReg2");
GReg[1]                                           // puts the content of GReg[3] into
GReg2_data = dmov(-);           // the value exported is the content of
GReg[2]                                           // puts the content of GReg[3] into
exec_once("mov GReg1, GReg3");
GReg[1]                                           // the value exported is the content of
GReg3_data = dmov(-);           // the value exported is the content of
GReg[3]
```

Get the value of the internal flags (SF, DF, T, and LM), of the loop related registers (EPC and SPC), and of the PC-related registers (PC and RPC). Use a done 5, which is the formatting instruction dedicated to the debug. This instruction formats the flags and the values contained in the registers. It also writes the resulting values into the channel context memory. It should not be used when entering debug from the IDLE state (for example, with no active channel script running on the SDMA), because it will update a channel context that may belong to any channel.

```
exec_once("done 5");           // formatting the value of flags and registers
```

At this point, the channel context should be up-to-date in memory, and debug operations should now be possible. However, the context can be exported with the following instructions:

Exporting the Context

```
dmov(ctx_base_addr);           // loading GReg[1] with the channel
context base address
exec_once("ld GReg0, (GReg1,0)"); // get RPC-PC into GReg0
exec_once("ld GReg1, (GReg1,1)"); // get SPC-EPC into GReg1
Loop_data = dmov(-);           // read back the value of Loop registers
exec_once("mov GReg1, GReg0");   // puts the PC info into GReg1
PC_data = dmov(-);             // reads back the content of the PC registers
```

After this sequence of operations, the entire SDMA context is exported via the OnCE.

7.2.3.14.5.3 Restoring the Context

At this point in the operation, restore the context of the SDMA. It can be different from the original context located in memory, and the content previously saved into the debugging application via the OnCE.

The example found hereshows how it is possible to modify the current channel context.

Modifying the Current Channel Context

```

dmov(Loop_data);                // put Loop former value into GReg[1]
exec_once("mov GReg0, GReg1");   // copy to GReg[0]
dmov(PC_data);                  // put PC former value into GReg[1]
exec_once("mov GReg2, GReg1");   // copy to GReg[2]
dmov(ctx_base_addr);             // put channel context base address into
GReg[1]
exec_once("st GReg0, (GReg1,1)"); // restore Loop context
exec_once("st GReg2, (GReg1,0)"); // restore PC context

```

Once the context in memory is the desired context (with or without applying the previous instruction sequence), it can be restored to the *real* PC and loop registers in the SDMA core:

```

exec_once("cpShReg");            // restore flags and PC & loop related registers

```

After this command, the SDMA core PC, RPC, SPC, EPC registers, as well as the flags contain the same data as what is stored in the context RAM for the current channel.

The following example shows how to restore the context of general registers GReg[0], GReg[1], GReg[2] and GReg[3].

Restoring the General Register Context

```

dmov(GReg3_data);                // put GReg[3] restore value in GReg[1]
exec_once("mov GReg3, GReg1");    // restore GReg[3]
dmov(GReg2_data);                // put GReg[2] restore value in GReg[1]
exec_once("mov GReg2, GReg1");    // restore GReg[2]
dmov(GReg0_data);                // put GReg[0] restore value in GReg[1]
exec_once("mov GReg0, GReg1");    // restore GReg[0]
dmov(GReg1_data);                // restore GReg[1]

```

At this point, it is possible to restart the normal program execution.

NOTE

Every SDMA core general register value can be modified by a mov instruction, which makes modification of these registers easy during debug. Unfortunately, there is no such instruction as a mov to directly modify the contents of either PCU register or flag (PC, RPC, SPC, EPC, T, LM, SF, or DF). The cpShReg instruction is meant to provide a means for changing these register contents via the context memory.

7.2.3.14.5.4 Accessing the Memory

In the example shown here, it is assumed that the SDMA context is entirely saved. If true, it is permissible to modify the general purpose registers during debugging activity.

To perform a memory read access, the target address is stored via the OnCE in GReg[1], then the load instruction is executed on the SDMA (the data loaded from the memory overwrites the address contained in GReg[1]), and then the result value is read back via the OnCE.

```
macro READ:                dmov(target_addr);                // put the target
address in GReg[1]         exec_once("ld GReg1, (GReg1, 0)"); // execute the
load instruction           res_data = dmov(-);               // exports the result
data value
```

For a memory write access, the target address is written in GReg[0], and the value to store is written in GReg[1]. Then the store instruction is executed on the SDMA.

```
macro WRITE:                dmov(target_addr);                // puts the
target address in GReg[1]   exec_once("mov GReg0, GReg1");     // puts the target
address in GReg[0]         dmov(target_data);                 // puts the target
data in GReg[1]            exec_once("st GReg1, (GReg0, 0)");  // performs the
store operation
```

This sequence is shown as an example; however, many other sequences are possible.

NOTE

This sequence of commands can also be applied to memory-mapped registers.

7.2.3.14.5.5 Resuming Program Execution

Before resuming program execution, it is assumed that the SDMA context is properly restored. There are two ways to restart the SDMA.

Start by executing the last instruction fetched before entering debug mode, as follows.

```
run_core(-);                // resume execution from where we stopped before
```

If necessary, restart the execution from a different address. In this case, use the `exec_core` command. The data field provided with this command must be the encoding of a jump instruction.

```
exec_core("jmp start_addr"); // rerun the SDMA from another address
```

In these two examples, the SDMA exits debug mode and keeps executing the code fetched from the memory.

7.2.3.14.5.6 Single Stepping in RAM

To execute a program step-by-step from the RAM, insert software breakpoints in the program flow at appropriate places so that the SDMA only executes one instruction before returning to debug mode.

First, read the next instruction to execute in the RAM. Then, depending on the value of this instruction, compute the address where a software breakpoint instruction should be inserted. The instruction at the corresponding address must be saved, and, the software breakpoint instruction is inserted. After restarting the SDMA, there is only one instruction executed before meeting the software breakpoint.

The following example shows the macro functions READ and WRITE, which correspond to the sequence of commands (described above) used to access the memory.

NOTE

The data read from the memory are 32-bit values, while the instructions are 16-bit values only. This is why it is best to only use addresses divided by two when accessing the memory.

READ and WRITE Macro Functions

```
next_instr = READ(run_addr/2);           // read the next instruction to execute
// the tool now has to compute the address where the breakpoint
// instruction should be inserted, this address is the "bkpt_addr"
instr_save = READ(bkpt_addr/2);         // save the instruction before
overwriting                               // store the bkpt instruction
STORE("bkpt instruction",bkpt_addr/2);
in memory
exec_core("jmp run_addr");               // rerun the SDMA
rstatus(-);                             // wait for the SDMA to enter debug mode
...
rstatus(-);
STORE(instr_save,bpkt_addr/2);           // restore the instruction
overwritten
```

In case of branched conditional instructions, a breakpoint instruction should be written at the two possible target addresses.

7.2.3.14.5.7 Single Stepping in ROM

No single-step mechanism is supported in ROM. The program code can be loaded in the RAM, where the single-step mechanism can be executed.

7.2.3.14.6 OnCE Event Detection Unit

The event detection unit watches signals from the data memory bus (DMBUS), which the SDMA core uses to access its RAM, ROM, and memory mapped registers.

A debug request is sent to the OnCE controller when user-defined conditions on address and/or data values are true.

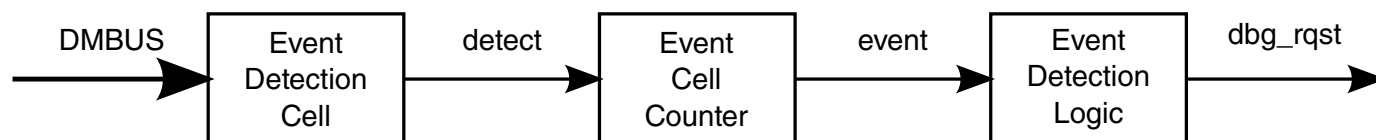


Figure 7-13. Event Detection Unit

A counter, provided with the detection cell, is decreased after an event detection. A debug request is sent to the core only when the counter reaches the value of 0. It is possible to disable the use of the counter if a debug request has to be generated after each event detection.

The event cell is the basic block that supports hardware breakpoints on an address value and/or data values coming from the SDMA memory bus. The trigger condition that generates the debug request is a mixed condition based on those values.

The following figure shows the event cell architecture. The event cell contains the address (stored in the memory address register) and the data (stored in the memory data register) used during the last memory access. There are some user-defined reference values located in memory mapped registers—the event cell addresses, the event cell address mask, the event cell data, and the event cell data mask. These registers are accessed by standard load/store instructions just like regular memory locations.

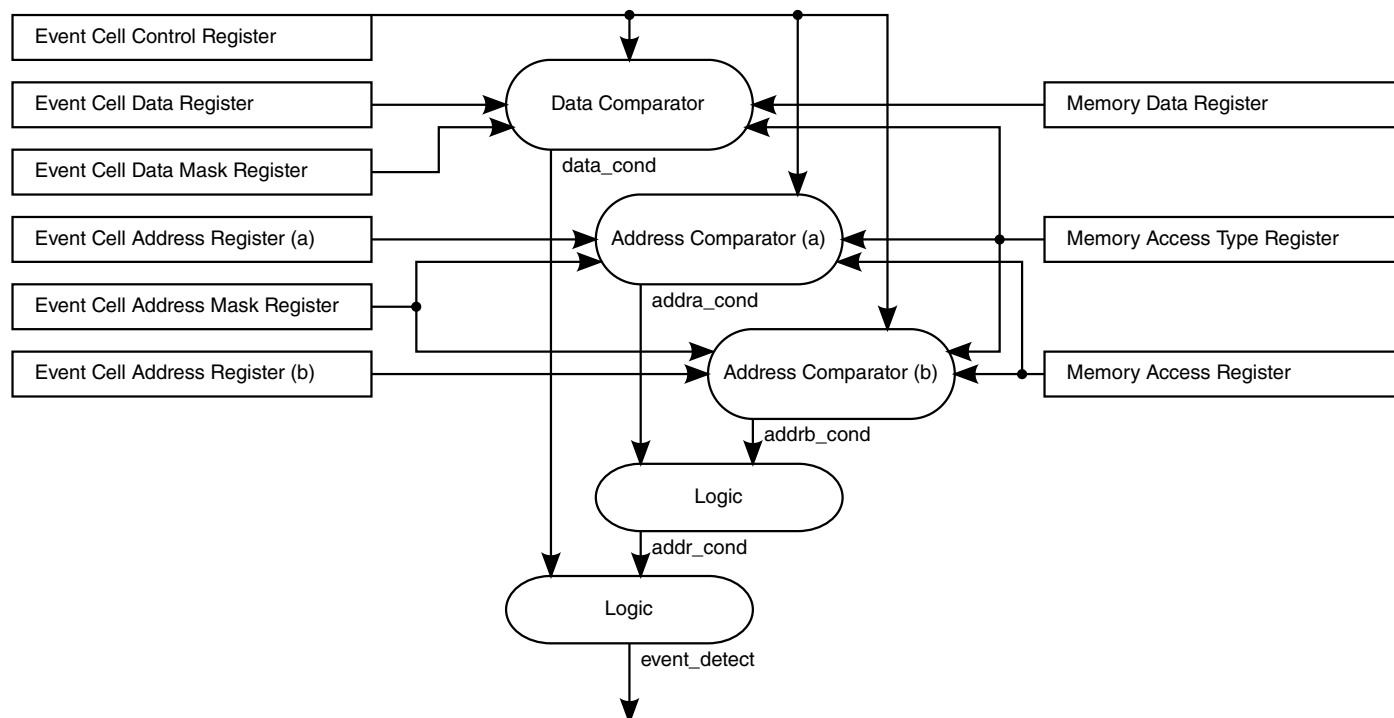


Figure 7-14. Event Cell Architecture

To define a memory breakpoint, three conditions are taken into account: The first two conditions are comparisons of the current memory address with user-defined reference addresses (these conditions are called addressA and addressB). The third condition consists of a comparison between the data received on the DMBUS and a user-defined reference data (this condition is called data). An intermediate address condition is set to express a dependency between addressA and addressB conditions.

7.2.3.14.7 Clock Gating and Reset

This section details how to use the clocks and handle the reset signals.

7.2.3.14.7.1 Clocks

Because the SDMA uses clock gating to save power, it is necessary to disable the clock gating and force the clocks to be enabled when using the OnCE.

When the OnCE is accessed through its JTAG interface, clock gating must be disabled outside the SDMA via a dedicated SDMA input port `clk_gating_off`. The reason why detection is not performed automatically by the SDMA internal hardware is that it would cost power to monitor activity on the JTAG interface.

When the OnCE is accessed through the Arm platform Control interface, clock gating is automatically turned off. This is done when bit 0 of the ONCE_ENB register (see [OnCE Enable \(SDMAARM_ONCE_ENB\)](#)) is set. A write access to this register is possible even when the OnCE clock is not running. If the Arm platform access is used, the bit in the ONCE_ENB register must be set before any attempt to access any other OnCE register.

7.2.3.14.7.2 Resets

The OnCE reset is different from the SDMA main reset.

Normally, activating the SDMA reset while keeping the OnCE reset inactive (when possible) enables you to reset the core without having to reprogram the OnCE.

7.2.3.14.8 Real Time Features

To rebuild the skeleton of a program execution, it is necessary to store the addresses of the program instructions where jumps are taken: A trace buffer is therefore provided. A real time buffer has also been added to receive data values written during a program execution.

The content of this register may be exported through JTAG ports without stopping the core.

7.2.3.14.8.1 Trace Buffer

The Trace Buffer is a 32-stage buffer that contains appropriate information to identify the 32 last changes of flow detected during a program execution.

The following figure shows an overview of the Trace Buffer.

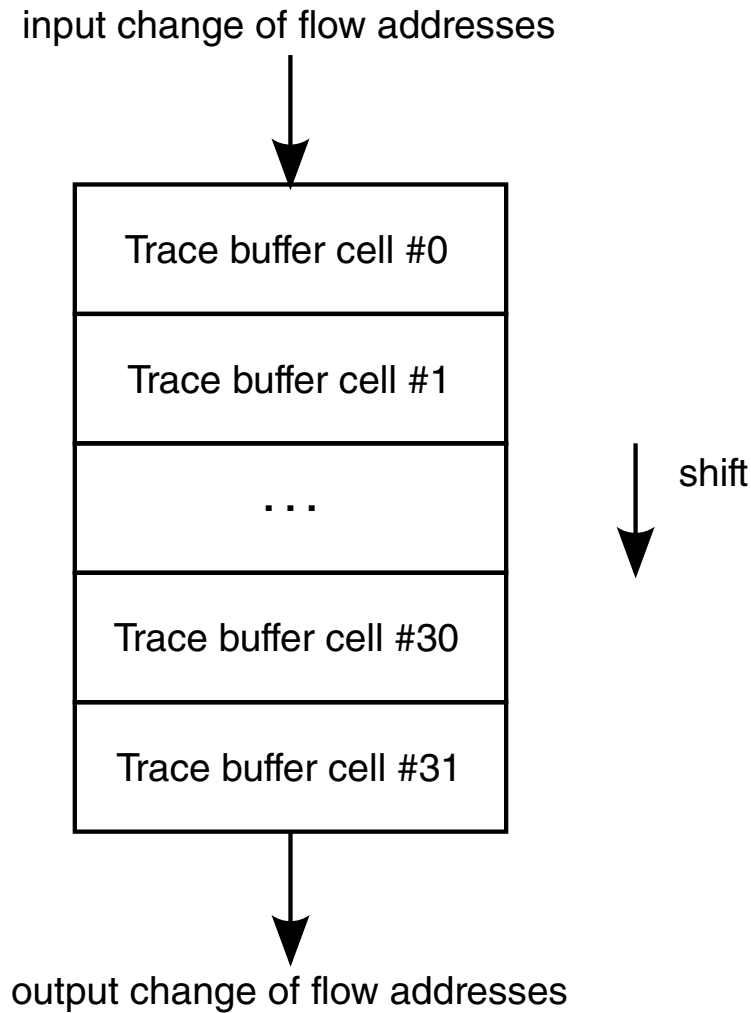


Figure 7-15. Trace Buffer

Each cell of the trace buffer contains two reference addresses and a flag. The flag is set when the addresses stored in the cell correspond to a valid change of flow; otherwise, the flag is cleared. The three most significant bits are unused.

After every change of flow detection, the address of current instruction and the address of the target instruction are stored at the top of the Trace Buffer (cell #0). The flag in the cell is set to indicate that a valid change of flow was detected. Former cell values are shifted one level down. The Trace Buffer contains the 32 last changes of flow. All the flags are reset on a software or a hardware reset, and after each transition from debug mode to user mode.

A memory mapped register of SDMA core, the Trace Buffer register (TB), is provided to read the content of the Trace Buffer. This operation should be done in debug mode. Performing a read access to the Trace Buffer register returns the content of the bottom of the Trace Buffer (cell #31). After every read access, the trace buffer is shifted one level down, and the flag at the top of the trace buffer is cleared.

A typical OnCE command sequence that retrieves the oldest change-of-flow information is as follows:

```
exec_once("mov r1, TB");           // stores the oldest change-of-flow in
GReg1                             // retrieves GReg1 contents
dmov(-);
```

This sequence requires the SDMA to be put in debug mode.

7.2.3.14.8.2 Real Time Buffer

The Real Time Buffer register (RTB) is a memory mapped register that can be accessed as a regular memory location by the SDMA core during program execution. This register is located in the OnCE.

Executing an `rbuffer` command (see [The OnCE Controller](#) for further details) exports the content of this register through JTAG ports.

When a write access is performed at the memory location corresponding to the RTB, the receive flag (for example, the RCV bit) is set in the OnCE Status Register (OSR). This flag is cleared at the end of the execution of a `rbuffer` command.

NOTE

Every write access to the RTB memory location updates the RTB register even if the RCV flag is set. The RTB is cleared on a JTAG reset.

7.2.3.14.8.3 Emulation Pin

The `debug_matched_event` emulation pin reflects the matching condition status detected by the Event Detection Unit.

Since it can be necessary to detect conditions without triggering debug requests, it is possible to disable the generation of debug requests by the Event Detection Unit and still have the matching condition available on the emulation pin. This can be done by clearing the EN flag in the ECTL register.

7.2.3.14.8.4 Real-Time Debug Outputs

The table found here shows the debug signals that are available at the SDMA boundaries. Their availability at chip boundaries depends on the project.

Table 7-45. Real-Time Debug Output Pins

Pin	Description
debug_core_state[3:0]	The core_state bits reflect the state of the SDMA core. • The "Program" state is the usual instruction execution cycle. • The "Data" state is inserted when there are wait-states during a load or a store on the data bus (ld or st). • The "Change of Flow" state is the second cycle of any instruction that breaks the sequence of instructions (jumps and channel switching instructions). • The "Change of Flow in Loop" state is used when an error causes a hardware loop exit. • The "Debug" state means the SDMA is in debug mode. • The "Functional Unit" state is inserted when there are wait-states during a load or a store on the functional units bus (ldf or stf). • In "Sleep" modes, no script is running (this is the core idle state); the "after Reset" is slightly different because no context restoring phase will happen when a channel is triggered: The script located at address 0 is executed (boot operation). • The "in Sleep" states are the same as above except they do not have any corresponding channel: they are used when entering debug mode after reset; the reason is that it is necessary to return to the "Sleep after Reset" state when leaving debug mode. 0 Program 1 Data 2 Change of Flow 3 Change of Flow in Loop 4 Debug 5 Functional Unit 6 Sleep 7 Context Switch Saving Channel 8 Program in Sleep 9 Data in Sleep 10 Change of Flow in Sleep 11 Change of Flow in Loop in Sleep 12 Debug in Sleep 13 Functional Unit in Sleep 14 Sleep after Reset 15 Context Switch Restoring Channel
debug_yield	Pulse that is active when a yield (done 0) or a yieldge (done 1) instruction is executed. 0 - 1 yield/yieldge executed
debug_core_run	Active when the SDMA core is executing instructions. 0 Debug or sleep mode

Table continues on the next page...

Table 7-45. Real-Time Debug Output Pins (continued)

Pin	Description
	1 Run mode
debug_event_channel_sel	Indicates if debug_event_channel displays current channel or last received event 0- debug_event_channel[5:0] gives the number of the current channel 1- debug_event_channel[5:0] gives the number of the last received event
debug_event_channel[5:0]	Gives the number of any DMA request as soon as it is received or the number of the current channel. The value of debug_event_channel_sel indicates if debug_event_channel displays the current channel or last received event. The signal debug_event_channel_sel must be observed to determine what information is provided on debug_event_chanel at any given time.
debug_pc[13:0]	Program Counter value; it has a meaning when the core is in run mode.
debug_mode	Set when the core is in debug. 0 - 1 Core is in debug
debug_bus_error	Set when an error was received during a load or a store (ld, st, ldf, or stf instruction) and registered in SF or DF flag. 0 No error during last load/store 1 Error during last load/store
debug_bus_device[4:0]	Indicates the device or functional unit that is accessed by the current instruction. The debug_bus_device output is always valid when in sleep mode, debug mode, or executing any instruction that does not access the functional units or the memory mapped devices, "no access" is output. 0 No access 1 MSA 2 MDA 3 MD 4 MS 5 PSA 6 PDA 7 PD 8 PS 9 RESERVED 10 RESERVED 11 RESERVED 12 RESERVED 13 CA 14 CS 15 Reserved 16 Memory (RAM or ROM) 17 Memory mapped register

Table continues on the next page...

Table 7-45. Real-Time Debug Output Pins (continued)

Pin	Description
	18 Peripheral #1 19 Peripheral #2 20 Peripheral #3 21 Peripheral #4 22 Peripheral #5 23 Peripheral #6 24 Peripheral #7 25 Peripheral #8 26 Peripheral #9 27 Peripheral #10 28 Peripheral #11 29 Peripheral #12 30 Peripheral #13 31 Peripheral #14
debug_bus_rwb	Indicates the direction of the access given by debug_bus_device 0 Write access (st or stf) 1 Read access (ld or ldf)
debug_matched_dmbus	Pulse indicating the OnCE event detection unit has detected a match on the data bus during an access to memory (RAM or ROM), a memory mapped register or a peripheral that is hooked to the SDMA. 0 - 1 data bus match detected
debug_rtbuffer_write	Pulse indicating when the real-time buffer is written by the core. 0 - 1 RTB was modified
debug_evt_chn_lines[7:0]	Eight lines that generate short pulses when DMA requests are received or channels are (re)started. Every line is controlled through two parameters defined in registers Cross-Trigger Events Configuration Register 1 (SDMAARM_XTRIG_CONF1) (as described in SDMAARM). The following two parameters are available for every line: • CNF-Indicates what is monitored on the line: 0 for a channel start, 1 for a DMA request reception • NUM[5:0]-Gives the number of the DMA request or channel to monitor

The matched_event emulation pin reflects the matching condition status detected by the Event Detection Unit. Because it can be necessary to detect conditions without triggering debug requests, it is possible to disable the generation of debug requests by the Event Detection Unit and still have the matching condition available on the emulation pin. This can be done by clearing the EN flag in the ECTL register.

All real-time debug outputs are disabled by default (for example, they are stuck to 0) to avoid power consumption when they are not used. They are enabled when bit 11 (RTDOBS) of the [Configuration Register \(SDMAARM_CONFIG\)](#) is set. Signals provided to the system JTAG controller for SDMA debug mode status will also be enabled when the *clk_gating_off* input is asserted.

7.2.4 Instruction Set

7.2.4.1 Instruction Encoding

This section presents a short summary of the instruction codes. All context switch instructions are listed for information only; they cannot function properly out of the context switch routine.

```

x...x - don't care
rrr - destination/source general register
sss - additional source general register
bbb - general register used as address base register
dddd - address displacement
nnnnn - bit number
uuuuuuuu - function unit command bits
pppppppp - branch displacement (signed)
iiiiiii - 8-bit immediate
jjj - control bit to clear
ff - flag to clear
00000jjj000000000 - done (done,yield,wait)
00000jjj000000001 - notify
00000xxx000000010 - reserved
00000xxx000000011 - reserved
00000xxx000000100 - reserved
00000000000000101 - softBkpt
00000000100000101 - reserved
00000010000000101 - reserved
00000011000000101 - reserved
00000100000000101 - reserved
00000101000000101 - reserved
00000110000000101 - reserved
00000111000000101 - reserved
00000000000000110 - ret
00000000100000110 - reserved
00000010000000110 - reserved
00000011000000110 - reserved
00000100000000110 - reserved
00000101000000110 - reserved
00000110000000110 - reserved
00000111000000110 - reserved

```

000000ff00000111	- clrf ff
0000010000000111	- reserved
0000010100000111	- reserved
0000011000000111	- reserved
0000011100000111	- illegal
00000rrr00001000	- jmp r
00000rrr00001001	- jsrr
00000rrr00001010	- ldrpc r
00000rrr00001011	- reserved
00000rrr000011xx	- reserved
00000rrr00010000	- revb
00000rrr00010001	- revblo
00000rrr00010010	- rorb
00000rrr00010011	- reserved
00000rrr00010100	- ror1
00000rrr00010101	- lsr1
00000rrr00010110	- asr1
00000rrr00010111	- lsl1
00000rrr001nnnnn	- bclri r,n
00000rrr010nnnnn	- bseti r,n
00000rrr011nnnnn	- btsti r,n
00000xxx10000xxx	- reserved
00000rrr10001sss	- mov
00000rrr10010sss	- xor
00000rrr10011sss	- add
00000rrr10100sss	- sub
00000rrr10101sss	- or
00000rrr10110sss	- andn
00000rrr10111sss	- and
00000rrr11000sss	- tst
00000rrr11001sss	- cmpeq
00000rrr11010sss	- cmplt
00000rrr11011sss	- cmphs
0000011011100000	- reserved
0000011011100001	- reserved
0000011011100010	- cpShReg
0000011011100011	- reserved
0000011011100100	- reserved
0000011011100101	- reserved
0000011011100110	- reserved
0000011011100111	- reserved
00000xxx11101xxx	- reserved
00000xxx11110xxx	- reserved
00000xxx11111xxx	- reserved
00001rrriiiiiiii	- ldi r,i
00010rrriiiiiiii	- xori r,i
00011rrriiiiiiii	- addi r,i
00100rrriiiiiiii	- subi r,i
00101rrriiiiiiii	- ori r,i
00110rrriiiiiiii	- andni r,i
00111rrriiiiiiii	- andi r,i
01000rrriiiiiiii	- tsti r,i
01001rrriiiiiiii	- cmpeqi r,i
01010rrrddddd bbb	- ld r,(d,b)
01011rrrddddd bbb	- st r,u
01100rrruuuuuuuu	- ldf r,u
01101rrruuuuuuuu	- stf r,u
011100xxxxxxxxxx	- reserved
011101xxxxxxxxxx	- reserved
011110ffnnnnnnnn	- Loop ff flags are reset
0111110pppppppp	- bf pc=pc+signed(pppppppp)+1
01111101pppppppp	- bt pc=pc+signed(pppppppp)+1
01111110pppppppp	- bsf pc=pc+signed(pppppppp)+1
01111111pppppppp	- bdf pc=pc+signed(pppppppp)+1
10aaaaaaaaaaaaaa	- jmp absolute
11aaaaaaaaaaaaaa	- jsr absolute

7.2.4.2 SDMA Instruction Set

This section describes all the useful instructions from the SDMA set.

Table 7-46. SDMA Instruction List

Instruction	Description	Page
ADD	Addition	ADD (Addition)
ADDI	Add with Immediate Value	ADDI (Add with Immediate Value)
AND	Logical AND	AND (Logical AND)
ANDI	Logical AND with Immediate Value	ANDI (Logical AND with Immediate Value)
ANDN	Logical AND NOT	ANDN (Logical AND NOT)
ANDNI	Logical AND with Negated Immediate Value	ANDNI (Logical AND with Negated Immediate Value)
ASR1	Arithmetic Shift Right by 1 Bit	ASR1 (Arithmetic Shift Right by 1 Bit)
BCLRI	Bit Clear Immediate	BCLRI1 (Bit Clear Immediate)
BDF	Conditional Branch if Destination Fault	BDF (Conditional Branch if Destination Fault)
BF	Conditional Branch if False	Functional Units Programming Model
BSETI	Bit Set Immediate	BSETI (Bit Set Immediate)
BSF	Conditional Branch if Source Fault	BSF (Conditional Branch if Source Fault)
BT	Conditional Branch if True	BT (Conditional Branch if True)
BTSTI	Bit Test immediate	BTSTI (Bit Test immediate)
CLRF	Clear Arm platform flags	CLRF (Clear Arm platform flags)
CMPEQ	Compare for Equal	CMPEQ (Compare for Equal)
CMPEQI	Compare with Immediate for Equal	CMPEQI (Compare with Immediate for Equal)
CMPHS	Compare for Higher or Same	CMPHS (Compare for Higher or Same)
CMPLT	Compare for Less Than	CMPLT (Compare for Less Than)
cpShReg	Update Context of PCU Registers and Flags	cpShReg (Update Context of PCU Registers and Flag)
DONE	DONE, Yield	DONE (DONE, Yield)
ILLEGAL	ILLEGAL Instruction	ILLEGAL (ILLEGAL Instruction)
JMP	Unconditional Jump Immediate	JMP (Unconditional Jump Immediate)
JMPR	Unconditional Jump	JMPR (Unconditional Jump)
JSR	Unconditional Jump to Subroutine Immediate	JSR (Unconditional Jump to Subroutine Immediate)
JSRR	Unconditional Jump to Subroutine	JSRR (Unconditional Jump to Subroutine)
LD	Load Register	LD (Load Register)
LDF	Load Register from Functional Unit	LDF (Load Register from Functional Unit)
LDI	Load Register with Immediate Value	LDI (Load Register with Immediate Value)
LDRPC	Load from RPC to Register	LDRPC (Load from RPC to Register)

Table continues on the next page...

**Table 7-46. SDMA Instruction List
(continued)**

Instruction	Description	Page
LOOP	Hardware Loop	LOOP (Hardware Loop)
LSL1	Logical Shift Left by 1 Bit	LSL1 (Logical Shift Left by 1 Bit)
LSR1	Logical Shift Right by 1 Bit	LSR1 (Logical Shift Right by 1 Bit)
MOV	Logical Move	MOV (Logical Move)
NOTIFY	Notify to Arm platform	NOTIFY (Notify to Arm platform)
OR	Logical OR	OR (Logical OR)
ORI	Logical OR with Immediate Value	ORI (Logical OR with Immediate Value)
RET	Return from Subroutine	RET (Return from Subroutine)
REVB	Reverse Byte Order	REVB (Reverse Byte Order)
REVBLO	Reverse Low Order Bytes	Reverse Low Order Bytes(REVBLO)
ROR1	Rotate Right by 1 Bit	ROR1 (Rotate Right by 1 Bit)
RORB	Rotate Right by 1 Byte	RORB (Rotate Right by 1 Byte)
SOFTBKPT	Software Breakpoint	SOFTBKPT (Software Breakpoint)
ST	Store Register	ST (Store Register)
STF	Store Register in Functional Unit	STF (Store Register in Functional Unit)
SUB	Subtract	SUB (Subtract)
SUBI	Subtract with Immediate	SUBI (Subtract with Immediate)
TST	Test with Zero	TST (Test with Zero)
TSTI	Test Immediate	TSTI (Test Immediate)
XOR	Logical Exclusive OR	XOR (Logical Exclusive OR)
XORI	Exclusive OR with Immediate	XORI (Exclusive OR with Immediate)

7.2.4.2.1 ADD (Addition)

Operation:

$GReg[r] \leftarrow GReg[s] + GReg[r]$

$T \leftarrow (GReg[r] == 0)$

Assembler:

Syntax: `add r,s`

Example: `add 0,3`

ADD GReg[3] and GReg[0] and store the result in GReg[0]

CPU Flags: T

Cycles: 1

Description: Performs the ADDition of the source general register s and the destination general register r , and stores the result in the destination general register r . The T flag is set if the result of the operation is 0. It is cleared if the result is not 0.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	1	0	0	1	1	s	s	s

Instruction Fields:

rrr / sss - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.2 ADDI (Add with Immediate Value)

Operation:

$\text{GReg}[r] \leftarrow \text{GReg}[r] + \text{immediate}$

$T \leftarrow (\text{GReg}[r] == 0)$

Assembler:

Syntax: `addi r,immediate`

Example: `add 6,112`

ADD GReg[6] and decimal value 112 and store the result in GReg[6]

CPU Flags: T

Cycles: 1

Description: Adds a 0-extended immediate value to a general register; stores the result in the general register. The flag T is set when the result of the operation is 0; otherwise, it is cleared. The immediate value is the low-order byte of the instruction and has a maximum value of 255 (0xFF).

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	r	r	r	i	i	i	i	i	i	i	i

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

iiiiiii - immediate value:

00000000 - 0

00000001 - 1

...

11111110 - 254

11111111 - 255

7.2.4.2.3 AND (Logical AND)**Operation:** $\text{GReg}[r] \leftarrow \text{GReg}[s] \& \text{GReg}[r]$ **Assembler:**

Syntax: and r,s

Example: and 1,2

AND GReg[1] and GReg[2] and store the result in GReg[1]

CPU Flags: Unaffected

Cycles: 1

Description: Performs the AND of the source general register s and the destination general register r , and stores the result in the destination general register r .

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	1	0	1	1	1	s	s	s

Instruction Fields:

rrr / sss - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.4 ANDI (Logical AND with Immediate Value)

Operation:

$\text{GReg}[r] \leftarrow \text{GReg}[r] \& \text{immediate}$

Assembler:

Syntax: `andi r,immediate`

Example: `andi 7,45`

AND GReg[7] and decimal value 45 and store the result in GReg[7]

CPU Flags: unaffected

Cycles: 1

Description: Performs an AND between a 0-extended immediate value and a general register; stores the result in the general register. The immediate value is the low-order byte of the instruction and has a maximum value of 255 (0xFF).

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	1	1	r	r	r	i	i	i	i	i	i	i	i

Instruction Fields:**rrr - register field:**

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

iiiiiii - immediate value:

00000000 - 0

00000001 - 1

...

11111110 - 254

11111111 - 255

7.2.4.2.5 ANDN (Logical AND NOT)**Operation:** $\text{GReg}[r] \leftarrow \sim\text{GReg}[s] \& \text{GReg}[r]$ **Assembler:**Syntax: `andn r,s`Example: `andn 3,4`

AND GReg[3] and NOT GReg[4] (bit inverted) and store the result in GReg[3]

CPU Flags: Unaffected

Cycles: 1

Description: Performs the AND of the negation of the source general register *s* and the destination general register *r*, and stores the result in the destination general register *r*.

Instruction Format:

Table 7-47. Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	1	0	1	1	0	s	s	s

Instruction Fields:

rrr /sss - destination register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.6 ANDNI (Logical AND with Negated Immediate Value)

Operation:

$\text{GReg}[r] \leftarrow \text{GReg}[r] \& \sim\text{immediate}$

Assembler:

Syntax: `andni r,immediate`

Example: `andni 0,2`

AND GReg[0] and decimal value -3 (inverted 32-bit value 2) and store the result in GReg[0]

CPU Flags: unaffected

Cycles: 1

Description: Performs an AND between the negation of a 0-extended 8-bit immediate value and a general register; stores the result in the general register. The immediate value is the low-order byte of the instruction and has a maximum value of 255 (0xFF).

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	1	0	r	r	r	i	i	i	i	i	i	i	i

Instruction Fields:**rrr - register field:**

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

iiiiiii - immediate value:

00000000 - 0

00000001 - 1

...

11111110 - 254

11111111 - 255

7.2.4.2.7 ASR1 (Arithmetic Shift Right by 1 Bit)**Operation:**

$$\text{GReg}[r]:\{b_{31}, b_{30}, \dots, b_1, b_0\} \leftarrow \text{GReg}[r]:\{b_{31}, b_{31}, b_{30}, \dots, b_1\}$$
Assembler:

Syntax: asr1 r

Example: asr1 3

divide by 2 the signed value of GReg[3] and store the result in GReg[3]

CPU Flags: Unaffected

Cycles: 1

Description: Shift the bits of any general register to the right and keep the same sign: The left bit (bit 31) is kept untouched.

Instruction Format:

Table 7-48. Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	0	0	0	1	0	1	1	0

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.8 BCLRI1 (Bit Clear Immediate)

Operation:

$\text{GReg}[r] : \{b_{31}, \dots, b(i+1), 0, b(i-1), \dots, b_0\} \leftarrow \text{GReg}[r] : \{b_{31}, \dots, b(i+1), b(i), b(i-1), \dots, b_0\}$

Assembler:

Syntax: `bclri r,i`

Example: `bclri 1,12`

clear bit 12 in GReg[1]

CPU Flags: Unaffected

Cycles: 1

Description: Clear the bit of register r specified by the 5-bit immediate field

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	0	0	1	i	i	i	i	i

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

iiii - immediate value:

00000 - 0

00001 - 1

...

11110 - 30

11111 - 31

7.2.4.2.9 BDF (Conditional Branch if Destination Fault)

Operation:

if (DF == 1) PC $\leftarrow$ PC + 1 + displacement else PC $\leftarrow$ PC + 1

Assembler:

Syntax: bdf label

Example: bdf LLL

Jump to LLL if DF is set, or go to the next instruction if DF is cleared; the displacement value is calculated by the assembler.

CPU Flags: Unaffected

Cycles: 2 when the branch is done, 1 otherwise

Description: If flag DF is set, jump to the new address that is calculated by adding the sign-extended 8-bit displacement to the next PC address. If flag DF is cleared, no jump is performed: The next instruction is located at the next PC address.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	1	1	1	1	1	p	p	p	p	p	p	p	p

Instruction Fields:

pppppppp - signed displacement field:

```
00000000 - 0
00000001 - 1
...
01111110 - 126
01111111 - 127
10000000 - (-128)
10000001 - (-127)
...
11111110 - (-2)
11111111 - (-1)
```

7.2.4.2.10 BF (Conditional Branch if False)

Operation:

```
if (T == 0)
    PC ← PC + 1 + displacement
else
    PC ← PC + 1
```

Assembler:

Syntax: bf label

Example: bf LLL

Jump to LLL if T is cleared, or go to the next instruction if T is set. The displacement value is calculated by the assembler.

CPU Flags: Unaffected

Cycles: 2 when the branch is done, 1 otherwise

Description: Conditional branch: If flag T is cleared, jump to the new address that is calculated by adding the sign-extended 8-bit displacement to the next PC address. If flag T is set, no jump is performed: The next instruction is located at the next PC address.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	1	1	1	0	0	p	p	p	p	p	p	p	p

Instruction Fields:

pppppppp - signed displacement field:

00000000 - 0
 00000001 - 1
 ...
 01111110 - 126
 01111111 - 127
 10000000 - (-128)
 10000001 - (-127)
 ...
 11111110 - (-2)
 11111111 - (-1)

7.2.4.2.11 BSETI (Bit Set Immediate)**Operation:**

$$\text{GReg}[r]:\{b_{31}, \dots, b(i+1), 1, b(i-1), \dots, b_0\} \leftarrow \text{GReg}[r]:\{b_{31}, \dots, b(i+1), b(i), b(i-1), \dots, b_0\}$$
Assembler:

Syntax: bseti r,i

Example: bseti 6,5

Set bit 5 in GReg[6]

CPU Flags: Unaffected

Cycles: 1

Description: Sets bit number i in the selected General Register.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	0	1	0	i	i	i	i	i

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

iiii - bit number field:

00000 - 0

00001 - 1

...

11110 - 30

11111 - 31

7.2.4.2.12 BSF (Conditional Branch if Source Fault)

Operation:

if (SF == 1) PC ← PC + 1 + displacement else PC ← PC + 1

Assembler:

Syntax: bsf label

Example: bsf LLL

Jump to LLL if SF is set, or go to the next instruction if SF is cleared. The displacement value is calculated by the assembler.

CPU Flags: Unaffected

Cycles: 2 when the branch is done, 1 otherwise

Description: Conditional branch: If flag SF is set, jump to the new address that is calculated by adding the sign-extended 8-bit displacement to the next PC address. If flag SF is cleared, no jump is performed: The next instruction is located at the next PC address.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	1	1	1	1	0	p	p	p	p	p	p	p	p

Instruction Fields:

pppppppp - signed displacement field:

```
00000000 - 0
00000001 - 1
...
01111110 - 126
01111111 - 127
10000000 - (-128)
10000001 - (-127)
...
11111110 - (-2)
11111111 - (-1)
```

7.2.4.2.13 BT (Conditional Branch if True)

Operation

```
if (T == 1)
    PC ← PC + 1 + displacement
else
    PC ← PC + 1
```

Assembler

```
Syntax: bt label
bt LLL
```

Jump to LLL if T is set, or go to the next instruction if T is cleared. The displacement value is calculated by the assembler.

CPU Flags: Unaffected

Cycles: 2 when the branch is done, 1 otherwise

Description: Conditional branch: If flag T is set, jump to the new address that is calculated by adding the sign-extended 8-bit displacement to the next PC address. If flag T is cleared, no jump is performed: The next instruction is located at the next PC address.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	1	1	1	0	1	p	p	p	p	p	p	p	p

pppppppp - signed displacement field:

00000000 - 0

00000001 - 1

...

01111110 - 126

01111111 - 127

10000000 - (-128)

10000001 - (-127)

...

11111110 - (-2)

11111111 - (-1)

7.2.4.2.14 BTSTI (Bit Test immediate)

Operation:

$T \leftarrow \text{GReg}[r]:b(i)$

Assembler:

Syntax: `btsti r,i`

Example: `btsti 2,29`

Test bit 29 in GReg[2] and copy its value in flag T

CPU flags: T

Cycles: 1

Description: T is loaded with the value of bit number i from the selected general register.

Instruction Format:

Table 7-49. Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	0	1	1	i	i	i	i	i

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

iiii - bit number field:

0000 - 0

0001 - 1

...

11110 - 30

11111 - 31

7.2.4.2.15 CLRF (Clear Arm platform flags)

Operation:

```
if (ff%2 == 0)
```

```
SF ← 0
```

```
if (ff/2 == 0)
```

```
DF ← 0
```

Assembler:

Syntax: clrf ff

Example: clrf 2

Clear flag SF and keep flag DF unchanged

CPU Flags: SF, DF

Cycles: 1

Description: Clears a selection of the Arm platform fault flags: SF, DF, both SF and DF or none can be cleared.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	f	f	0	0	0	0	0	1	1	1

Instruction Fields:

ff - flags field:

00 - clear SF and clear DF

01 - clear DF

10 - clear

SF 11 - no clear

7.2.4.2.16 CMPEQ (Compare for Equal)

Operation:

$T \leftarrow (GReg[s] == GReg[r])$

Assembler:

Syntax: cmpeq *r*, *s*

Example: cmpeq 7,5

Compare GReg[7] and GReg[5] and set flag T if they are equal

CPU flags: T

Cycles: 1

Description: Subtracts the destination general register *r* from the source general register *s*, and sets T if the result is 0, clears T if the result is not 0.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	1	1	0	0	1	s	s	s

Instruction Fields:

rrr / sss - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.17 CMPEQI (Compare with Immediate for Equal)

Operation:

$T \leftarrow (\text{GReg}[r] == \text{immediate})$

Assembler:

Syntax: `cmpeqi r,immediate`

Example: `cmpeqi 2,13`

Compare GReg[2] and decimal value 13 and set flag T if they are equal

CPU Flags: T

Cycles: 1

Description: Subtracts the 0-extended 8-bit immediate value from the general register, and sets T if the result is 0, clears T if the result is not 0. The immediate value is the low-order byte of the instruction.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	1	r	r	r	i	i	i	i	i	i	i	i

Instruction Fields:

rrr - destination register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

iiiiiii - immediate value:

00000000 - 0

00000001 - 1

...

11111110 - 254

7.2.4.2.18 CMPHS (Compare for Higher or Same)

Operation:

$$T \leftarrow (\text{GReg}[r] \geq \text{GReg}[s])$$

Assembler:

Syntax: `cmphs r,s`

Example: `cmphs 0,1`

Compare GReg[0] and GReg[1] and set flag T if GReg[0] is higher than or equal to GReg[1]

CPU Flags: T

Cycles: 1

Description: Compares the destination general register *r* and the source general register *s*, and sets T if the destination general register *r* is higher than or equal to the source general register *s*, clears T otherwise. The comparison is unsigned.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	1	1	0	1	1	s	s	s

Instruction Fields:

rrr / sss - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.19 CMPLT (Compare for Less Than)

Operation:

$$T \leftarrow (\text{GReg}[r] < \text{GReg}[s])$$
Assembler:Syntax: `cmplt r,s`Example: `cmplt 7,4`

Compare GReg[7] and GReg[4] and set flag T if GReg[7] is lower than GReg[4]

CPU Flags: T

Cycles: 1

Description: Compares the destination general register *r* and the source general register *s*, and sets T if the destination general register *r* is lower than the source general register *s*, clears T otherwise. The comparison is signed.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	1	1	0	1	0	s	s	s

rrr / sss - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.20 cpShReg (Update Context of PCU Registers and Flag)**Assembler:**Syntax: `cpShReg`

CPU Flags: none

Cycles: 1

Description: SF, RPC, T, PC, LM, EPC, DF, and SPC registers are updated according to the value of their corresponding bits in the context memory. This instruction must only be used in debug mode via the OnCE. It reverses the done 5 operation.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	1	0	1	1	1	0	0	0	1	0

7.2.4.2.21 DONE (DONE, Yield)

Operation:

```
if (jjj&6 == 2) HE[CCR] ← 0
```

```
if (jjj == 3) HI[CCR] ← 1
```

```
if (jjj == 4) EP[CCR] ← 0
```

```
if ((jjj == 0) && (NCP > CCP)) CCR ← NCR
```

```
else if ((jjj == 1) && (NCP >= CCP))
```

```
CCR ← NCR
```

```
else
```

```
CCR ← NCR
```

(CCR stands for Current Channel Register; NCR stands for Next Channel Register)

Assembler:

Syntax: done jjj

Example: done 3

Clear HE bit for the current channel, send an interrupt to the Arm platform for the current channel and reschedule.

CPU Flags: Unaffected

Cycles: Variable if a context switch is done, 1 otherwise

Description: Clears one of the channel enabling bits (HE or EP for the corresponding channel number) if required. Sends an interrupt to the corresponding Arm platform by setting the appropriate flag, if required (HI for the corresponding channel number). Reschedules according to the mode and the NCP (Next Channel Priority) and CCP (Current Channel Priority) values. According to the scheduling decision, the NCR (Next Channel Register) is copied to the CCR (Current Channel Register) and channel contexts are switched. If several channels with the same highest priority are pending, they are ordered by their number from 31 down to 0. The higher number is selected (for example, channel 26 is selected if channels 3, 12, 14, and 26 with the same highest priority are pending). If no flag is modified, the reschedule can allow the replacement of the current

channel by another channel with a priority strictly greater than the current channel priority (yield). Or, it can allow the replacement of the current channel by another channel with a priority greater than or equal to the current channel priority (yieldge). In the latter case, the selected channel will always be the first one with the same priority, starting from channel number 31 down to channel 0 (the current channel does not belong to the set of selectable channels).

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	j	j	j	0	0	0	0	0	0	0	0

jjj - Channel Flags field:

000 - No channel flags affected: Reschedule only if the next channel priority is greater than current channel priority (yield)

001 - No channel flags affected: Reschedule only if the next channel priority is greater than or equal to the current channel priority (yieldge)

010 - Clear HE for the current channel and reschedule 011 - Clear HE, set HI for the current channel and reschedule

100 - Clear EP for the current channel and reschedule

101 - Reserved for debug to copy relevant registers into context memory

110 - RESERVED

111 - RESERVED

For the scheduling rules, refer to [Scheduler Functional Description](#). Every possible done instruction is further described as follows:

- done 0/yield is executed by a channel script when it accepts preemption by a higher priority channel;
- done 1/yieldge is executed by a channel script when it accepts preemption by a higher priority channel and it also accepts a roll-up with other channels that have the same priority;
- done 2 is executed by a channel script that was triggered by a Arm platform start via the [Channel Start \(SDMAARM_HSTART\)](#) register, when its task is completed and it requires termination;
- done 3 is executed by a channel script that was triggered by a Arm platform start via the [Channel Start \(SDMAARM_HSTART\)](#) register, when its task is completed, it requires termination and it needs to trigger an interrupt to the Arm platform upon closure;

- done 4 is executed by a channel script that was triggered by a DMA request, when its task is completed and it requires termination;
- done 5 is used in debug mode only; it copies the PCU registers and flags to the context memory of the current channel;

7.2.4.2.22 ILLEGAL (ILLEGAL Instruction)

Operation:

PC ← 0001

Assembler:

Syntax: illegal

CPU Flags: Unaffected

Cycles: 2

Description: Jumps to the Illegal instruction routine located at address 0001. All unauthorized instructions result in an Illegal instruction behavior; however, the ILLEGAL instruction must be used to guarantee software compatibility with future versions of the SDMA.

Instruction Format

Table 7-50. Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	1	1	0	0	0	0	0	1	1	1

7.2.4.2.23 JMP (Unconditional Jump Immediate)

Operation:

PC ← absolute_address

Assembler:

Syntax: jmp label

Example: jmp LLL

The assembler translates the label to the exact address

CPU Flags: Unaffected

Cycles: 2

Description: Jumps to the absolute address contained the lower 14 bits of the instruction (the PC is a 14-bit register).

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	a	a	a	a	a	a	a	a	a	a	a	a	a	a

aaaaaaaaaaaaaaaa - address field:

0000000000000000 - 0

0000000000000001 - 1

...

111111111111110 - 16382

111111111111111 - 16383

7.2.4.2.24 JMPR (Unconditional Jump)

Operation:

$PC \leftarrow GReg[r]$

Assembler:

Syntax: `jmp r`

Example: `jmp 0`

Jump to address stored in GReg[0]

CPU Flags: Unaffected

Cycles: 2

Description: Jumps to the absolute address contained in a General Register.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	0	0	0	0	1	0	0	0

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.25 JSR (Unconditional Jump to Subroutine Immediate)

Operation:

RPC ← PC + 1

PC ← absolute_address

Assembler:

Syntax: jsr r

Example: jsr LLL

Jumps to subroutine starting at LLL; the assembler translates the label to exact address

CPU Flags: Unaffected

Cycles: 2

Description: Jumps to the subroutine located at the absolute address contained the lower 14 bits of the instruction (the PC is a 14-bit register).

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	a	a	a	a	a	a	a	a	a	a	a	a	a	a

aaaaaaaaaaaaaa - address field:

0000000000000000 - 0

0000000000000001 - 1

...

111111111111110 - 16382

111111111111111 - 16383

7.2.4.2.26 JSRR (Unconditional Jump to Subroutine)

Operation:

$$RPC \leftarrow PC + 1$$

$$PC \leftarrow GReg[r]$$
Assembler:

Syntax: jsrr r

Example: jsrr 5

Jumps to subroutine located at address stored in GReg[5]

CPU Flags: Unaffected

Cycles: 2

Description: Jumps to the subroutine at address contained in a General Register

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	0	0	0	0	1	0	0	1

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.27 LD (Load Register)**Operation:**

$$GReg[r] \leftarrow [GReg[b] + displacement]$$

if (transfer_error)

SF $\leftarrow$ 1

else

SF $\leftarrow$ 0

Assembler:

Syntax: `ld r, (b, displacement)`

Example: `ld 1, (2, 23)`

Loads data into GReg[1]; the data is located at address obtained by adding decimal value 23 to GReg[2]

CPU Flags: SF

Cycles: 2+n where n is 0 for ROM, RAM or memory mapped registers, and n is the number of wait-states of the peripheral for a peripheral access

Description: Adds a 5-bit 0-extended displacement to a base address in General Register b; the result is the address of the data to fetch on the DM bus. The data received from the bus is stored in the destination General Register r. If an error occurs during the transfer, the flag SF is set, else it is cleared.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	r	r	r	d	d	d	d	d	b	b	b

rrr / bbb - register field:

000 - GReg[0]

001 - GReg[1]

...

111 - GReg[7]

dddd - displacement value:

00000 - 0

00001 - 1

...

11111 - 31

7.2.4.2.28 LDF (Load Register from Functional Unit)**Operation:**

$\text{GReg}[r] \leftarrow [\text{fu_address}]$

if (transfer_error)

SF $\leftarrow$ 1

else

SF ← 0

fu_address is an 8-bit field and depends on addressed functional unit

Assembler:

Syntax: `ldf r, fu_address`

Example: `ldf 0, 13`

Loads data coming from the Burst DMA register MD into GReg[0]; it is a 32-bit access with no prefetch

CPU Flags: SF

Cycles: 1+n where n is the number of wait-states that may be inserted by the functional unit

Description: Sends an 8-bit address on the Functional Unit Bus (FU bus) and stores the data received from the bus in the destination General Register r. If an error occurs during the transfer, the flag SF is set, else it is cleared.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	r	r	r	f	f	f	f	f	f	f	f

See the following sections for more details of the LDF instruction usage with each functional unit:

- [Burst DMA Read \(ldf\)](#) for Burst DMA
- [Peripheral DMA Read \(ldf\)-Read Mode](#) for Peripheral DMA

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

fffffff - functional unit source register and action (unspecified values are reserved):

00000000 - MSA

00000100 - MDA

00001001 - MD byte

00001010 - MD halfword

00001011 - MD word

00001100 - MS

00101001 - MD byte - prefetch

00101010 - MD halfword - prefetch

00101011 - MD word - prefetch

01000000 - DSA

11000000 - PSA

11001000 - PD

11010000 - PDA

11011000 - PD in copy mode (rrr contents are lost)

11101000 - PD - prefetch next data

11111111 - PS

7.2.4.2.29 LDI (Load Register with Immediate Value)

Operation:

$\text{GReg}[r] \leftarrow \text{immediate}$

Assembler:

Syntax: `ldi r,immediate`

Example: `ldi 6,1`

loads decimal value 1 into GReg[6]

CPU Flags: Unaffected

Cycles: 1

Description: Stores a 0-extended immediate value in a General Register. The immediate value is the low-order byte of the instruction and has a maximum value of 255 (0xFF).

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	r	r	r	i	i	i	i	i	i	i	i

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

iiiiiii - immediate value:

00000000 - 0

00000001 - 1

...

11111110 - 254

11111111 - 255

7.2.4.2.30 LDRPC (Load from RPC to Register)

Operation:

GReg[r] ← RPC

Assembler:

Syntax: `ldrpc r`

Example: `ldrpc 3`

copies RPC to GReg[3]

CPU Flags: Unaffected

Cycles: 1

Description: Stores the contents of the RPC in a General Register. That instruction may be used to have more than one level of subroutines.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	0	0	0	0	1	0	1	0

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.31 LOOP (Hardware Loop)

Operation:

```

if (ff%2 == 0)
    SF ← 0
if (ff/2 == 0)
    DF ← 0
if ((GReg[0] == 0) || (SF == 1) || (DF == 1))
    PC ← PC + loop_size + 1
else
{
    SPC ← PC + 1
    EPC ← PC + loop_size + 1
    LM ← 1
    PC ← PC + 1
}

```

during every instruction execution in the loop:

```

if ((SF == 1) || (DF == 1))
{

```

```

    LM ← 0
    PC ← EPC
}
else if ((PC + 1) == EPC)
{
    GReg[0] ← GReg[0] - 1
    if (GReg[0] == 0)
    {
        LM ← 0
        PC ← EPC
    }
    else
        PC ← SPC
}
else
    PC ← nextPC(instruction)

```

after the execution of the last instruction of the loop body:

```

if (GReg[0] == 0)
    T ← 1
else
    T ← 0

```

Assembler:

Syntax: `loop n{,ff}`

Example: `loop 3,1`

Executes GReg[0] times the instructions comprised between PC+1 and PC+3 (included); ff=1 clears the DF flag before starting the loop. When omitted, the ff field is set to 0 (clearing both SF and DF).

CPU Flags: LM[1:0], T

Cycles: 2 when the loop count (GReg[0]) is 0 or SF or DF is set at loop start, 1+1 when the loop starts but exits abnormally (SF or DF set inside the loop which adds 1 cycle to the offending load or store to jump to EPC), 1 when the loop is executed normally

Description: The loop instruction executes a sequence of instructions several times. The number of times is given by the contents of GReg[0], the loop counter. SDMA will jump to the first instruction after the end of the loop if the value in GReg[0] is 0. Otherwise the SDMA enters loop mode. It sets the most significant bit of the LM flag that will only be reset once the last instruction of the last loop is executed. The instructions in the loop are executed GReg[0] times. The management of fault flags (SF and DF) is as follows. When entering the hardware loop, SF and DF can be cleared according to the ff field of the instruction. After that operation, if any flag is still set the loop will not be executed. The SDMA will jump to the first instruction after the end of the loop without entering loop mode. During the execution of the loop, if any fault flag is set by a LD, LDF, ST, or STF instruction, the SDMA will immediately exit loop mode and jump to the first instruction after the end of the loop. In that case, GReg0 is not decremented for that last piece of the loop body execution (even if the SF or DF flag is set at the last instruction of the loop body). The T flag reflects the state of GReg[0] after the end of the loop, which is an indicator of the complete execution of the loop. If the loop exited because of an error (SF or DF set), GReg[0] will not be 0 at the end of the loop, hence T will be cleared. If the loop executes without fault, GReg[0] will be 0 at the end of the loop, hence T will be set. The boundary case when a source or destination fault occurs at the last instruction of the last loop is considered as an anticipated exit of the loop, which causes the T flag to be cleared. If the last instruction executed before leaving the hardware loop also tries to modify the T flag, the flag is updated according to the value of GReg[0], NOT according to the result of the last executed instruction.

Limitations:

1. 1. Jump instructions (JMP, JMPR, JSR, JSRR, BF, BT, BSF, BDF) are not allowed inside the hardware loop.
2. 2. GReg[0] cannot be written to inside the hardware loop (it can be read).
3. 3. The empty loop (0 instruction in the body) is forbidden.
4. 4. If GReg[0] == 0 at the start of the loop, which causes a jump to EPC, the T flag is not updated.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	1	1	0	f	f	n	n	n	n	n	n	n	n

Instruction Fields:

ff - flags field:

00 - clear SF and clear DF

01 - clear DF

10 - clear SF

11 - no clear

nnnnnnnn - loop size

00000000 - empty loop: forbidden value

00000001 - 1 instruction in the loop

00000010 - 2 instructions in the loop

...

11111111 - 255 instructions in the loop

7.2.4.2.32 LSL1 (Logical Shift Left by 1 Bit)

Operation:

$\text{GReg}[r]:\{b30, \dots, b1, b0, 0\} \leftarrow \text{GReg}[r]:\{b31, b30, \dots, b1, b0\}$

Assembler:

Syntax: `lsl1 r`

Example: `lsl1 2`

multiplies by 2 the value in `GReg[2]`

CPU Flags: Unaffected

Cycles: 1

Description: Shift the bits of any General Register to the left. The right bit (bit 0) is set to 0. No overflow is detected by the hardware.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	0	0	0	1	0	1	1	1

Instruction Fields:

rrr - register field:

000 - `GReg[0]`

001 - `GReg[1]`

010 - `GReg[2]`

011 - `GReg[3]`

100 - `GReg[4]`

101 - `GReg[5]`

110 - `GReg[6]`

111 - GReg[7]

7.2.4.2.33 LSR1 (Logical Shift Right by 1 Bit)

Operation:

$$\text{GReg}[r] : \{0, b_{31}, b_{30}, \dots, b_1\} \leftarrow \text{GReg}[r] : \{b_{31}, b_{30}, \dots, b_1, b_0\}$$

Assembler:

Syntax: lsr1 r

Example: lsr1 4

divides by 2 the unsigned value contained in GReg[4]

CPU Flags: Unaffected

Cycles: 1

Description: Shift the bits of any General Register to the right. The left bit (bit 31) is set to 0.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	0	0	0	1	0	1	0	1

Instruction Fields:

rrr - destination register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.34 MOV (Logical Move)

Operation:

$$\text{GReg}[r] \leftarrow \text{GReg}[s]$$

Assembler:

Syntax: `mov r,s`

Example: `mov 4,0`

copies GReg[0] to GReg[4]

CPU Flags: Unaffected

Cycles: 1

Description: Move the contents of the source General Register *s* to the destination General Register *r*.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	1	0	0	0	1	s	s	s

Instruction Fields:

rrr / sss - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.35 NOTIFY (Notify to Arm platform)

Operation:

```

if (jjj & 4 == 0)
{
    if (jjj&2 == 2)
        HE[CCR] ← 0
    if (jjj&1== 1)
        HI[CCR] ← 1
}
else if (jjj == 4)

```

$EP[CCR] \leftarrow 0$

else

(CCR stands for Current Channel Register)

Assembler:

Syntax: notify jjj

Example: notify 3

clears the HE bit for the current channel and sends an interrupt to the Host for the current channel

CPU Flags: Unaffected

Cycles: 1

Description: Clears one of the channel enabling bits (HE or EP for the corresponding channel number) if required, sends an interrupt to the corresponding Arm platform by setting the appropriate flag if required (HI for the corresponding channel number).

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	j	j	j	0	0	0	0	0	0	0	1

jjj - Channel Flags field:

000 - unused

001 - set HI for the current channel

010 - clear HE for the current channel

011 - clear HE, set HI for the current channel

100 - clear EP for the current channel

101 - RESERVED

110 - RESERVED

111 - RESERVED

7.2.4.2.36 OR (Logical OR)

Operation:

$GReg[r] \leftarrow GReg[s] \mid GReg[r]$

Assembler:

Syntax: or r,s

Example: or 3,6

ORs GReg[3] and GReg[6] and stores the result in GReg[3]

CPU Flags: Unaffected

Cycles: 1

Description: Performs the OR of the source General Register s and the destination General Register r, and stores the result in the destination General Register r.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	1	0	1	0	1	s	s	s

Instruction Fields:

rrr / sss - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.37 ORI (Logical OR with Immediate Value)

Operation:

$\text{GReg}[r] \leftarrow \text{GReg}[r] \mid \text{immediate}$

Assembler:

Syntax: ori r,immediate

Example: ori 1,56

ORs GReg[1] and the decimal value 56 and stores the result in GReg[1]

CPU Flags: unaffected

Cycles: 1

Description: Performs an OR between a 0-extended 8-bit immediate value and a General Register; stores the result in the General Register. The immediate value is the low-order byte of the instruction and has a maximum value of 255 (0xFF).

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	1	r	r	r	i	i	i	i	i	i	i	i

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

iiiiiii - immediate value:

00000000 - 0

00000001 - 1

...

11111110 - 254

11111111 - 255

7.2.4.2.38 RET (Return from Subroutine)

Operation:

PC ← RPC

Assembler:

Syntax: ret

CPU Flags: Unaffected

Cycles: 2

Description: Return from subroutine.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0

7.2.4.2.39 REVB (Reverse Byte Order)**Operation:**

$\text{GReg}[r] : \{B3, B2, B1, B0\} \leftarrow \text{GReg}[r] : \{B0, B1, B2, B3\}$

Assembler:

Syntax: `revb r`

Example: `revb 5`

reverses bytes order in `GReg[5]`

CPU Flags: Unaffected

Cycles: 1

Description: Reverse the byte order of any General Register.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	0	0	0	1	0	0	0	0

Instruction Fields:

rrr - register field:

000 - `GReg[0]`

001 - `GReg[1]`

010 - `GReg[2]`

011 - `GReg[3]`

100 - `GReg[4]`

101 - `GReg[5]`

110 - `GReg[6]`

111 - `GReg[7]`

7.2.4.2.40 Reverse Low Order Bytes(REVBLO)

Operation:

$$\text{GReg}[r] : \{B3, B2, B0, B1\} \leftarrow \text{GReg}[r] : \{B3, B2, B1, B0\}$$

Assembler:

Syntax: revblo r

Example: revblo 0

reverses low order bytes in GReg[0]

CPU Flags: Unaffected

Cycles: 1

Description: Reverse both low order bytes of any General Register.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	0	0	0	1	0	0	0	1

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.41 ROR1 (Rotate Right by 1 Bit)

Operation:

$$\text{GReg}[r] : \{b0, b31, b30, \dots, b1\} \leftarrow \text{GReg}[r] : \{b31, b30, \dots, b1, b0\}$$

Assembler:

Syntax: ror1 r

Example: ror1 3

rotates bits to the right in GReg[3]

CPU Flags: Unaffected

Cycles: 1

Description: Rotate the bits of any General Register to the right.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	0	0	0	1	0	1	0	0

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.42 RORB (Rotate Right by 1 Byte)

Operation:

$\text{GReg}[r] : \{B0, B3, B2, B1\} \leftarrow \text{GReg}[r] : \{B3, B2, B1, B0\}$

Assembler:

Syntax: rorb r

Example: rorb 2

rotates bytes to the right in GReg[2]

CPU Flags: Unaffected

Cycles: 1

Description: Rotate the bytes of any General Register to the right.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	0	0	0	1	0	0	1	0

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.43 SOFTBKPT (Software Breakpoint)

Operation:

Stops the current script and enters debug mode

Assembler:

softbkpt

CPU Flags: Unaffected

Description: When the core executes this instruction, it has the same effect as receiving a debug request from the OnCE or via the external debug request input: the script execution halts, the PCU enters its debug state and waits for the OnCE commands that are described in [OnCE and Real-Time Debug](#).

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

7.2.4.2.44 ST (Store Register)

Operation:

[GReg[b] + displacement] ← GReg[r]

```

if (transfer_error)
    DF ← 1
else
    DF ← 0

```

Assembler:

Syntax: `st r,(b,displacement)`

Example: `st 7,(0,9)`

stores the value from GReg[7] into memory at address obtained by adding decimal value 9 to GReg[0]

CPU Flags: DF

Cycles: 2+n where n is 0 for ROM, RAM or memory mapped registers, and n is the number of wait-states of the peripheral for a peripheral access

Description: Adds a 5-bit 0-extended displacement to a base address in General Register b; the result is the address of the data to store on the DM bus. The data sent on the bus comes from the source General Register r. If an error occurs during the transfer, the flag DF is set, else it is cleared.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	1	r	r	r	d	d	d	d	d	b	b	b

Instruction Fields:

rrr / bbb - register field:

```

000 - GReg[0]
001 - GReg[1]
010 - GReg[2]
011 - GReg[3]
100 - GReg[4]
101 - GReg[5]
110 - GReg[6]
111 - GReg[7]

```

dddd - displacement value:

```

00000 - 0

```

00001 - 1
...
11111 - 31

7.2.4.2.45 STF (Store Register in Functional Unit)

Operation:

```
[fu_address] ← GReg[r] 0
if (transfer_error) 0
DF ← 1 0
else 0
DF ← 0
```

fu_address is an 8-bit field

Assembler:

Syntax: stf r, fu_address

Example: stf 3, 0x2B

stores the 32-bit contents of GReg[3] to the Burst DMA register MD; waits until the flush to external memory is completed

CPU Flags: DF

Cycles: 1+n where n is the number of wait-states that may be inserted by the functional unit

Description: Sends an 8-bit address on the Functional Unit Bus (FU bus) and sends the contents of the source General Register r on the bus. If an error occurs during the transfer, the flag DF is set, else it is cleared.

Table 7-51. Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	1	r	r	r	f	f	f	f	f	f	f	f

See the following sections for more details of the STF instruction usage with each functional unit:

- [Burst DMA Write \(stf\)](#) for Burst DMA
- [Peripheral DMA Write \(stf\)-Write Mode](#) for Peripheral DMA

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

fffffff - functional unit destination register and action (unspecified values are reserved):

00000000 - MSA in incremented mode

00000100 - MDA in incremented mode

00001001 - MD byte

00001010 - MD halfword

00001011 - MD word

00001100 - clear MS error flag

00001111 - MS

00010000 - MSA in frozen mode

00010100 - MDA in frozen mode

00011000 - MD in copy mode - number of words in rrr

00100000 - MSA in incremented mode - start prefetch

00101000 - MD no data - flush

00101001 - MD byte - flush

00101010 - MD halfword - flush

00101011 - MD word - flush

00110000 - MSA in frozen mode - start prefetch

11000001 - PSA in frozen mode - 8-bit data width
11000010 - PSA in frozen mode - 16-bit data width
11000011 - PSA in frozen mode - 32-bit data width
11000101 - PSA in incremented mode - 8-bit data width
11000110 - PSA in incremented mode - 16-bit data width
11000111 - PSA in incremented mode - 32-bit data width
11001000 - PD
11001001 - PSA in decremented mode - 8-bit data width
11001010 - PSA in decremented mode - 16-bit data width
11001011 - PSA in decremented mode - 32-bit data width
11001100 - clear PS error flag
11001101 - PSA data width becomes 8-bit
11001110 - PSA data width becomes 16-bit
11001111 - PSA data width becomes 32-bit
11010001 - PDA in frozen mode - 8-bit data width
11010010 - PDA in frozen mode - 16-bit data width
11010011 - PDA in frozen mode - 32-bit data width
11010101 - PDA in incremented mode - 8-bit data width
11010110 - PDA in incremented mode - 16-bit data width
11010111 - PDA in incremented mode - 32-bit data width
11011001 - PDA in decremented mode - 8-bit data width
11011010 - PDA in decremented mode - 16-bit data width
11011011 - PDA in decremented mode - 32-bit data width
11011101 - PDA data width becomes 8-bit
11011110 - PDA data width becomes 16-bit
11011111 - PDA data width becomes 32-bit

11100001 - PSA in frozen mode - 8-bit data width - prefetch data
 11100010 - PSA in frozen mode - 16-bit data width - prefetch data
 11100011 - PSA in frozen mode - 32-bit data width - prefetch data
 11100101 - PSA in incremented mode - 8-bit data width - prefetch data
 11100110 - PSA in incremented mode - 16-bit data width - prefetch data
 11100111 - PSA in incremented mode - 32-bit data width - prefetch data
 11101001 - PSA in decremented mode - 8-bit data width - prefetch data
 11101010 - PSA in decremented mode - 16-bit data width - prefetch data
 11101011 - PSA in decremented mode - 32-bit data width - prefetch data
 11101101 - PSA data width becomes 8-bit - prefetch data
 11101110 - PSA data width becomes 16-bit - prefetch data
 11101111 - PSA data width becomes 32-bit - prefetch data
 11111111- PS

7.2.4.2.46 SUB (Subtract)

Operation:

$$\text{GReg}[r] \leftarrow \text{GReg}[r] - \text{GReg}[s]$$

$$T \leftarrow (\text{GReg}[r] == 0)$$

Assembler:

Syntax: `sub r,s`

Example: `sub 4,7`

SUBtracts GReg[7] from GReg[4] and stores the result in GReg[4]

CPU Flags: T

Cycles: 1

Description: Subtracts the source General Register *s* from the destination General Register *r*, and stores the result in the destination General Register *r*. The T flag is set if the result of the operation is 0; it is cleared if the result is not 0.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	1	0	1	0	0	s	s	s

Instruction Fields:

rrr / sss - register fields:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.47 SUBI (Subtract with Immediate)

Operation:

$\text{GReg}[r] \leftarrow \text{GReg}[r] - \text{immediate}$

$T \leftarrow (\text{GReg}[r] == 0)$

Assembler:

Syntax: `sub r,immediate`

Example: `sub 1,255`

SUBtracts decimal value 255 from GReg[1] and stores the result in GReg[1]

CPU Flags: T

Cycles: 1

Description: Subtracts a 0-extended 8-bit immediate value from a General Register; stores the result in the General Register. The flag T is set when the result of the operation is 0; otherwise, it is cleared. The immediate value is the low-order byte of the instruction and has a maximum value of 255 (0xFF).

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	r	r	r	i	i	i	i	i	i	i	i

Instruction Fields:**rrr - register field:**

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

iiiiiii - immediate value:

00000000 - 0

00000001 - 1

...

11111110 - 254

11111111 - 255

7.2.4.2.48 TST (Test with Zero)**Operation:** $T \leftarrow ((\text{GReg}[s] \ \& \ \text{GReg}[r]) \neq 0)$ **Assembler:**Syntax: `tst r,s`Example: `tst 2,3`

ANDs GReg[2] and GReg[3] and sets T if the result is non-null

CPU Flags: T

Cycles: 1

Description: Performs the AND of the source General Register s and the destination General Register r, and sets T if the result is not 0, clears T if the result is 0.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	1	1	0	0	0	s	s	s

Instruction Fields:

rrr / sss - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.49 TSTI (Test Immediate)**Operation:** $T \leftarrow ((\text{GReg}[r] \& \text{immediate}) \neq 0)$ **Assembler:**Syntax: `tsti r,immediate`Example: `tsti 5,13`

ANDs GReg[5] and decimal value 13 and sets T if the result is non-null

CPU Flags: T

Cycles: 1

Description: Performs the AND of a 0-extended 8-bit immediate value and the destination General Register r, and sets T if the result is not 0, clears T if the result is 0. The immediate value is the low-order byte of the instruction and has a maximum value of 255 (0xFF).

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	r	r	r	i	i	i	i	i	i	i	i

Instruction Fields:

rrr - destination register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

iiiiiii - immediate value:

00000000 - 0

00000001 - 1

...

11111110 - 254

11111111 - 255

7.2.4.2.50 XOR (Logical Exclusive OR)

Operation:

$\text{GReg}[r] \leftarrow \text{GReg}[s] \wedge \text{GReg}[r]$

Assembler:

Syntax: `xor r,s`

Example: `xor 0,3`

XORs GReg[0] and GReg[3] and stores the result in GReg[0]

CPU Flags: Unaffected

Cycles: 1

Description: Performs the eXclusive OR of the source General Register s and the destination General Register r, and stores the result in the destination General Register r.

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	r	r	r	1	0	0	1	0	s	s	s

Instruction Fields:

rrr / sss - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

7.2.4.2.51 XORI (Exclusive OR with Immediate)

Operation:

$GReg[r] \leftarrow GReg[r] \wedge \text{immediate}$

Assembler:

Syntax: `xori r,immediate`

Example: `xor 7,5`

XORs GReg[5] and decimal value 5 and stores the result in GReg[7]

CPU Flags: Unaffected

Cycles: 1

Description: Performs an eXclusive OR between a 0-extended 8-bit immediate value and a General Register; stores the result in the General Register. The immediate value is the low-order byte of the instruction and has a maximum value of 255 (0xFF).

Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	0	r	r	r	i	i	i	i	i	i	i	i

Instruction Fields:

rrr - register field:

000 - GReg[0]

001 - GReg[1]

010 - GReg[2]

011 - GReg[3]

100 - GReg[4]

101 - GReg[5]

110 - GReg[6]

111 - GReg[7]

iiiiiii - immediate value:

00000000 - 0

00000001 - 1

...

11111110 - 254

11111111 - 255

7.2.4.2.52 YIELD, YIELDGE (DONE, Yield)

By default, unsupported assembler syntax. Can be aliased to the corresponding done instructions (yield = done 0; yieldge = done 1). Refer to the done instruction description [DONE \(DONE, Yield\)](#).

7.2.5 Software Restrictions

7.2.5.1 Unsupported Burst DMA Access Sequence

The SDMA does not support triggering a pre-fetch followed by a flush of the Burst DMA without reading or writing any data. If the flush occurs while the background pre-fetch DMA operation is still in progress, it could result in un-defined behavior.

An example of the sequence which could result in undefined results is shown in the following example:

Instruction sequence not supported

```

stf r1, MSA|PF      ; Update source address, triggers data pre-fetch in the
                        ; background
mov R0,R0            ; Execute multiple assembly instructions, none of which
                        ; read
mov R0,R0            ; or write data to/from MD
stf MD|SZ0|FL        ; Flush FIFO without writing data. If the pre-fetch is still
                        ; in progress when this instruction is executed, there
                        ; could be undefined operation

```

A work-around to avoid any undesirable results is to first read MD to ensure the pre-fetch is complete before the flush is attempted.

Work-Around to previous example

```

stf r1, MSA|PF      ; Update source address, triggers data pre-fetch.

```

```
mov R0,R0          ; Execute multiple assembly instructions, none of which
                   ; read
mov R0,R0          ; or write data to/from MD
ldf r2, MD         ; dummy read of MD to ensure pre-fetch is complete
                   ; before the next instruction
stf MD|SZ0|FL      ; Flush FIFO without writing data
```

7.2.6 Application Notes

7.2.6.1 Data Structures for Boot Code and Channel Scripts

SDMA boot code downloads the different channel contexts and the scripts that will be executed on SDMA channels during the application.

The boot code is run after reset when channel 0 is started by the Arm platform. The boot code is also known as channel 0 script.

The boot code is based on the Channel Control Block (CCB) and Buffer Descriptor (BD) mechanisms that are data structures located into the Arm platform memory space. With these data structures, it is possible to instruct SDMA to download scripts and contexts but also to dump a context or a script to a destination data buffer. Channel scripts also use the CCB and BD data structures to pass instructions and/or pointers to data to be copied.

The format, processing, and field definition of the CCB and BD are defined and performed entirely by the software script rather than the SDMA hardware. An overview of the format and structure is provided here, but for complete details refer to the SDMA software documentation (see [SDMA Scripts](#)).

The CCB and BD data structures are accessed by SDMA using DMA and processed by the SDMA scripts. The ROM contains common sub-routines for processing these data structures which may be called by the bootload and channel scripts.

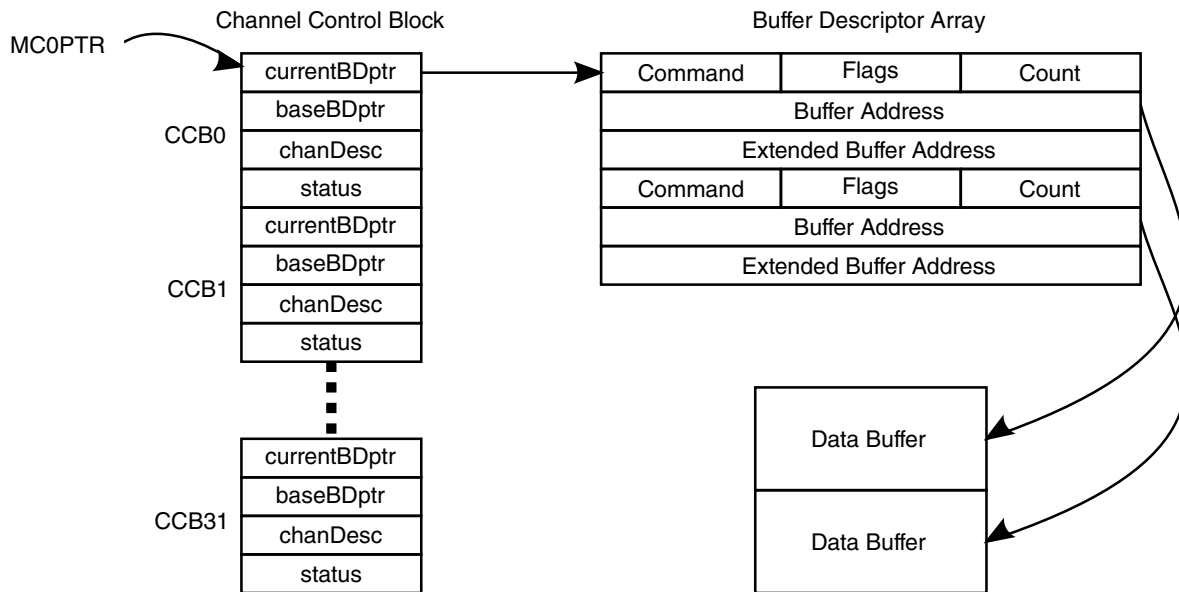


Figure 7-16. Data Structures Layout

The previous figure shows an example how these data structures are linked to pass command and pointers to data buffers. The SDMA's MC0PTR register holds the base address of the Channel 0 Control Block (CCB0). The Channel 0 control block holds a pointer to the array of buffer descriptors. The buffer descriptors are used to tell the channel 0 (boot channel) what to do as described [Buffer Descriptor Format](#).

7.2.6.1.1 Buffer Descriptor Format

Buffer descriptors are three longs (32-bit words) in size as, shown in the figure found [here](#).

A buffer descriptor describes the properties of the data buffer it points to. The buffer descriptors can be used for linear or circular data buffers in the Arm platform processor memory. The CCB contains a pointer to the base BD as well as the current BD.

Table 7-52. Buffer Descriptor

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Command								-	-	L	R	I	C	W	D	Count															
Buffer Address																															
Extended Buffer Address																															

Table 7-53. Buffer Descriptor Field Descriptions

Field	Description
31-24 Command	Command. The command field is used to differentiate operations performed within a script when the script accesses this particular buffer descriptor. The use of this field can be defined by the script. The command values defined for the bootstrap script are defined in Buffer Descriptor Commands for Bootstrap scripts . Refer to the individual script definition in script library documents in SDMA Scripts for command field definitions for other scripts.
23	Reserved
22	Reserved
21 L	Last Buffer Descriptor: This bit is set in SDMA IPC scripts to indicate to the receiving Core that the transfer has ended. Whenever the source finishes transferring the count it wanted to transfer, it sets LAST_BIT in the destination BD, to let the destination know that transfer is over. This bit also tells the destination software that when it processes the destination BDs, they need not process any BD after the BD with the LAST_BIT set. For example, when the DSP prepares a single buffer descriptor with count equals to 25 and Arm platform prepares a single buffer descriptor with count equals 100. When 25 bytes have been transferred from DSP to Arm platform, the DSP buffer descriptor is normally closed while the Arm platform buffer descriptor will have the L bit set and the byte count updated to 25.
20 R	erroR. Indicates an error occurred on the channel's buffer descriptor requested command. Some scripts may overwrite the command field with an error code indicating the source of the error. 0 No Error 1 Error
19 I	Interrupt. When SDMA has finished to process data transfer attached to this buffer descriptor, send an interrupt to the Arm platform. 0 No Interrupt 1 Interrupt the processor when BD is complete
18 C	Continuous. This buffer is allowed to receive multiple transmit buffers or is allowed to transmit to multiple receive buffers. The Continuous bit is decoded at the end of the processing of a BD to determine if the SDMA script must open a new BD to potentially continue the data transfer. 0 No further buffer descriptors 1 SDMA should move to the next Buffer descriptor after this one
17 W	Wrap. Indicates if this buffer descriptor is the last one for the channel control block. When encountering this bit set, the SDMA scripts updates the CurrentBD pointer to point to the first Buffer Descriptor of the array. This bit is set if the Arm platform wants to organize the array of BD in a circular way (like a ring). When all BD have been processed and if Wrap bit and Continuous bit are set in the last BD, the SDMA script will wrap around and it will try to re-open the first BD. 0 No Error 1 Wrap to first buffer descriptor after this one is processed.
16 D	D - "Done": bit 16: indicates the "ownership" of the buffer descriptor. When D=0 the host owns the buffer descriptor; when D=1 SDMA owns the buffer descriptor. In the case of the channel 0, D=1 indicates the SDMA has not yet processed this buffer, D=0 indicates the SDMA has processed this buffer. 0 Arm platform owns the buffer. 1 SDMA owns the buffer
15-0 Count	Count. the count field (bit 15-0) indicates the size of the data to be transmitted, the size of the data buffer pointed to by the buffer descriptor. The SDMA memory structure is different for program memory (16-bits shorts/half-words) and data memory (32-bits long). For channel 0 buffer descriptors, Count is expressed in 16-bit half-words when PM is addressed and in 32-bit words when DM is addressed. Count is typically expressed in bytes for other channel scripts, but the unit is dependant on the script.
31-0	Buffer address. Address pointer to the data buffer.
31-0	Extended buffer address. Additional pointer or other information required by some scripts.

The buffer descriptors form an array of programmable size. If the last buffer descriptor is marked by the Wrap flag-bit $W=1$, the array of buffer descriptor is treated as a ring with some logically continuous portion owned by the Arm platform with $D=0$, and the remainder owned by the SDMA with $D=1$. The count field of the buffer descriptor indicates how much data has been transmitted.

If Arm platform has prepared 3 buffers to be filled by the SDMA script, it has also prepared 3 BD, one for each buffer. The *Cont* and *Wrap* bits are used to organize the buffers in a circular way. For example, *CONTInous* bit is set to 1 in the 2 first BDs and *Wrap* is set in the 3rd BD. The SDMA script opens and processes BD#1. Since *CONTInous* bit is set for this BD, the SDMA will open the second BD and it will process it. Each time a BD is processed, its *Done* bit is reset by the SDMA. After the 3rd BD, if *CONTInous* is not set but if *Wrap* is set, the SDMA script stops here and the next time the channel will be triggered, the script will open the BD pointed by the currentBDptr pointer of the CCB and it will correspond to the first buffer descriptor.

If the *CONTInous* bit and *Wrap* bits are both set in the 3rd BD, the script will close it and it will try to open the first BD. An error may occur at this point if the BD#1 has already been processed and its *Done* bit is 0. The SDMA script cannot process a BD with a *Done* bit to 0. It means the BD is not ready to be processed. To avoid this situation, the *CONTInous* bit should not be set for the last BD if *Wrap* is set, and the Interrupt flag must set for the last BD. It will warn the owner of the BD that all the BDs have been processed and it has to re-set to 1 the *Done* bit of all the BD's if it desires the SDMA to fill them again. Basically, if the Arm platform expects the SDMA to fill up the buffers in a circular fashion, then it's the responsibility of the Arm platform to set the *Done* bit of a buffer descriptor at an appropriate time.

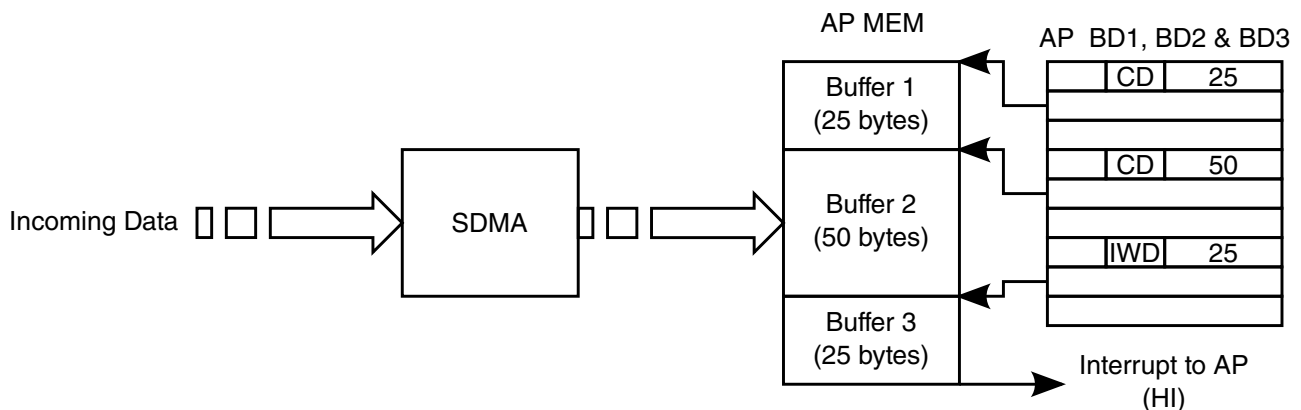


Figure 7-17. Buffer Descriptor Flow

The previous figure shows an example buffer descriptor flow. When the incoming data is stored and fills the first buffer of 25 bytes, the SDMA script opens the second BD because the CONTinuous bit was set. Then next incoming data is put in the second buffer. After receiving 50 bytes, the second buffer descriptor is also closed. The Done bit is reset and the third BD is opened. After receiving another 25 bytes, the third buffer is full and an interrupt is sent to the Arm platform because the Interrupt flag is set in the 3rd BD. The CONTinuous flag is not present the transfer is over. The next time the script will be triggered, the BD to be opened will be the first buffer descriptor since the Wrap flag was set in the 3rd BD. It is the Arm platform responsibility to set the Done bit of all the BD if it wants to use the same buffers.

7.2.6.1.2 Buffer Descriptor Commands for Bootload scripts

The command field of the buffer descriptor is defined separately for each script.

The following table lists the buffer descriptor commands defined for the channel 0 bootload script.

Table 7-54. Channel Zero Buffer Descriptor Commands

Command Field (binary)	Command	Description	Buffer Address	Extended Buffer Address
0000_0001 (0x01)	C0_SET_DM	Load SDMA data memory (RAM) from Arm platform memory buffer	Arm platform memory source address	SDMA memory destination address
0000_0010 (0x02)	C0_GET_DM	Copy SDMA data memory (RAM) to Arm platform memory buffer	Arm platform memory destination address	SDMA memory source address
0000_0100 (0x04)	C0_SET_PM	Load SDMA program memory (RAM) from Arm platform memory buffer	Arm platform memory source address	SDMA memory destination address
0000_0110 (0x06)	C0_GET_PM	Copy SDMA program memory (RAM) to Arm platform memory buffer	Arm platform memory destination address	SDMA memory source address
cccc_c111 (0x07 CHN)	C0_SETCTX	Load Context for channel cccc into SDMA RAM from Arm platform memory buffer	Arm Platform memory source address	-
cccc_c011 (0x03 CHN)	C0_GETCTXT	Copy Context for channel ccccc from SDMA RAM to Arm platform memory buffer	Arm platform memory destination address	-

The Channel 0 bootload commands are summarized as follows:

- **C0_SET_[PM-DM]**: load the buffer descriptor data in the SDMA local memory at the address pointed to by the "extended buffer address" field. The SDMA RAM can be seen as a Program Memory (PM, 16-bit address) or Data Memory (DM 32-bit address). When C0_SET_PM is used, the count field is expressed in "shorts" (16-bit half words), this command can be used to download scripts. When C0_SET_DM is

used, the count field is expressed in "long" (32-bit words), this command can be used to download channel contexts to the context channel area in RAM.

- **C0_GET_[PM-DM]**: write to the buffer descriptor's data buffer the content of the SDMA local memory from the address pointed to by the "extended buffer address" field for the length defined by the count in the buffer descriptor. **C0_GET_PM** is used to dump some part of the Program Memory (may be used to dump context of a channel), therefore count is expressed in "shorts"; while **C0_GET_DM** is used to dump to the buffer descriptor's data buffer, so the count field is in "longs."
- **C0_SETCTX**: load a context into the SDMA context page area. The handling script decodes the channel number from the 5 MSB of the command field of the buffer descriptor. Using the channel number the script computes the offset of the context data pointer for the channel relative to the context page base to use as the destination address in SDMA memory. Then the **C0_SET_DM** command explained above is invoked to load SDMA RAM from memory. The counter indicates the size in words of the context structure.
- **Command value**: (in binary) `cccc c111`, where `cccc` is the channel number (5 bits). For instance, `0x0F` means set context for channel 1, `0xFF` means set context for channel 31.
- **C0_GETCTX**: write to the buffer descriptor's data buffer the content of the SDMA context page area. The handling script decodes the channel number from the 5 MSB of the command field of the buffer descriptor. Using this channel number, the script computes the offset of the context data pointer for the channel relative to the context page base to use as the source address for the copy. Then the **C0_GET_DM** command explained above is invoked to copy the context to memory. The counter indicates the size in words of the context structure.
- **Command value**: (in binary): `cccc c011`, where `cccc` is the channel number (5 bits). For instance, `0x03` means get context of channel 1, `0xFB` means get context of channel 31.

NOTE

To download channel context, **C0_SETDM** and **C0_SETCTXT** command can be used but the second one is easier because the channel number is embedded into the command field, whereas with the **C0_SETDM**, the pointer to the channel context area must be written into the extended buffer address field of the buffer descriptor.

7.2.6.1.3 Example of Buffer Descriptors for Channel 0.

Figure 7-19 illustrates the buffer descriptors that must be set in Arm platform memory space, before execution of boot code, to download contexts and scripts of channels 1, 4, and 10. After boot code execution, SDMA memory will be populated with the different contexts and scripts as presented in the following figure.

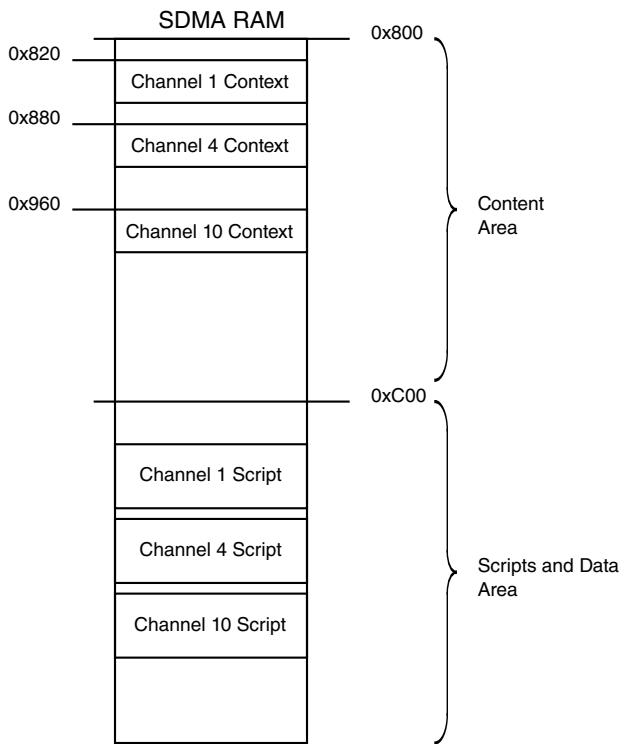


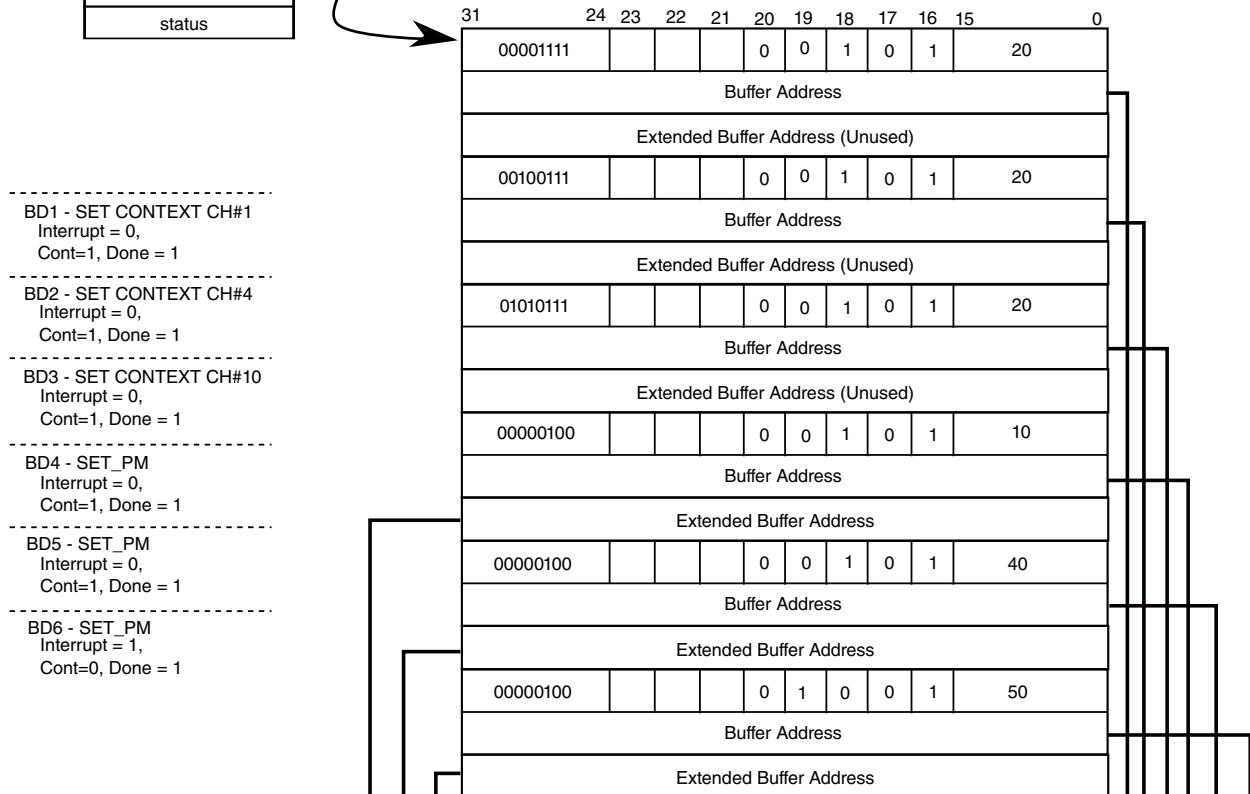
Figure 7-18. Example of SDMA RAM After Boot Session

SDMA Register

MCOPTR

Channel Control Block

CurrentBDptr
BaseBDptr
chanDesc
status

Channel 0 Buffer Descriptor Array

BD1 - SET CONTEXT CH#1
 Interrupt = 0,
 Cont=1, Done = 1

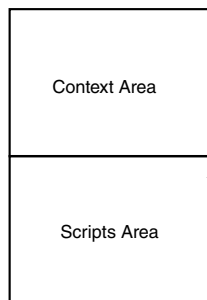
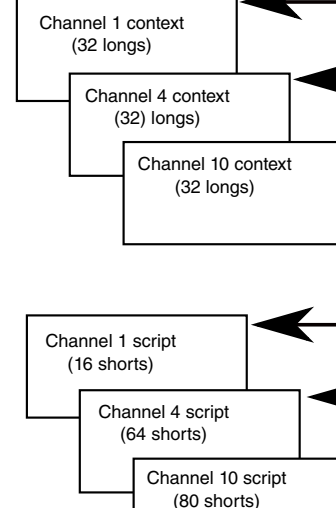
BD2 - SET CONTEXT CH#4
 Interrupt = 0,
 Cont=1, Done = 1

BD3 - SET CONTEXT CH#10
 Interrupt = 0,
 Cont=1, Done = 1

BD4 - SET_PM
 Interrupt = 0,
 Cont=1, Done = 1

BD5 - SET_PM
 Interrupt = 0,
 Cont=1, Done = 1

BD6 - SET_PM
 Interrupt = 1,
 Cont=0, Done = 1

SDMA RAM**AP Memory Space**

7.2.6.1.4 Channel Context

There are 32 channel context memory structures pointed to by the local save area pointer. These channel context memory structures are fixed.

The script in the SDMA computes the memory offset for a given channel based on the structure length and channel number. Figure below shows the structure of the channel context as it is saved in the SDMA local memory (RAM).

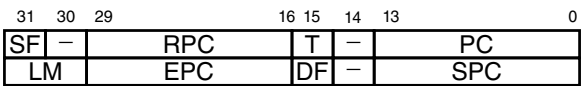
A channel context consists in 24 words, one per register. A total of 32 words are reserved for every channel. The additional 8 words are called scratch ram and they are dedicated to each channel. This memory area is commonly used for stack management.

The structure is divided in 4 areas:

- Channel status registers
- General purpose registers
- Functional units state registers reflecting the state of the Arm platform DMAs (Burst and Peripheral DMA).
- Scratch RAM

The details of the channel context status registers are described in the following figure.

The PC field of the first long register must point to the SDMA RAM address where the script that will be executed on the channel is located and this value equals the one stored in the extended buffer address of the buffer descriptor with C0_SETPM command.



SF: Source fault while loading data
RPC: Return program counter
T: Test bit: status of arithmetic and test instructions
PC: Program counter
LM: Loop mode
EPC: Loop end program counter
DF: Destination fault while storing data
SPC: Loop Start program counter

Figure 7-20. SDMA State Registers (ShPC, ShLoop)

7.2.6.2 Typical Data Transfer Supported by SDMA DMA Units

This section presents a library of SDMA scripts that perform data transfers through the peripheral DMA and the burst DMA units.

The Arm platform memory and peripherals are devices that either the peripheral DMA or the burst DMA can access. The scripts are given for a peripheral DMA whose address registers are programmed in incremented mode when internal memory is involved. See the following table for the summary.

Table 7-55. Typical Data Transfers Summary

Data Transfer	Peripheral DMA	Burst DMA	Comments
Arm platform External Memory ↔ Arm platform External Memory		3	Copy mode Script example, see Burst DMA Unit Copy Mode and External Memory to External Memory .
Arm platform Peripheral ↔ Arm platform Peripheral	3		Copy mode if same data path width Script example, see Peripheral to Peripheral Transfer .
Arm platform External Memory ↔ Arm platform Peripheral	3	3	Data transit through SDMA Script example, see Transfer Between Peripheral and External Memory .
Arm platform External Memory ↔ Arm platform Internal Memory		3	Copy mode Script example, see Transfer Between External Memory and Internal Memory .
Arm platform Internal Memory ↔ Arm platform Internal Memory		3	Copy mode Script example, see Internal Memory to Internal Memory .
Arm platform Internal memory ↔ Arm platform Peripheral	3		Data transit through SDMA Script example, see Transfer Between Peripheral and Internal Memory .

NOTE

These scripts are provided as examples of how to use DMA blocks to perform required data transfers: They are not "official" programs.

7.2.6.2.1 External Memory to External Memory

This section describes the SDMA script that performs data moves in external memory.

For this particular data transfer, only the burst DMA is used. It is programmed in copy mode, so no data transmits through an SDMA general register.

The SDMA core only monitors data transfer status. It is assumed source and destination address values are already present in two SDMA general registers (r1 and r2). For this example, it is also assumed that a 32-bit word-to-move for source-to-destination address is present in r0 and equals 64.

Data Moves in External Memory

```

1      stf r1,MSA                      // Source address setup
2      stf r2,MDA                      // Destination address setup
3      ldi r0,0x64                    // 64 words must be transferred from MSA to
MDA
4      ldi r1,0x8
MAIN_XFER:
5      cmphs r0,r1                    // Is r0 >= 0x8
6      bf LAST_XFER                  // If not, jump to last transfer label
7      stf r1,MD|CPY                  // Copy 8 words from MSA to MDA address.
8      subi r0,0x8                    // Decrement counter
9      jmp MAIN_XFER                  // return to main transfer loop
LAST_XFER:
10     stf r0,MD|CPY                  // perform last transfer

```

All instructions are performed in one cycle (jumps excepted). Instruction 7 triggers a copy transfer: A read burst access of 8-word starts, data is staged in MD and then a write burst of 8 words is executed. Instruction 8, 9, 5, and 6 are executed while the burst access is in progress. If this access is not complete when instruction 7 is executed a second time, SDMA stalls on this instruction as long as the previous copy transfer is not over. In this case, the instruction is no longer a one-cycle instruction.

During the main loop (MAIN_XFER), r1 always equals 8, so burst lengths are 8 words. On the last ldf |CPY instruction (10), r1 equals the remainder of r0 divided by 8; therefore, the length of bursts triggered in copy mode equal r1 value, which is between 1 and 7.

7.2.6.2.2 Peripheral to Peripheral Transfer

For this data transfer, only the peripheral DMA is used.

It is programmed in copy mode, so no data will transmit through the SDMA general register used in the ldf instruction, but the contents of the general register are lost. The SDMA core only monitors the transfer.

7.2.6.2.2.1 Source and Destination Target Have the Same Data Path Width

When the source and destination target have the same data path width, the following is true:

- Source target is a *half-word* (16-bit) peripheral located at address 0x1002.
- Destination is a *half-word* (16-bit) peripheral located at address 0x2006.

It is assumed the address values are already present in two SDMA general registers (r1, r2). The script for a transfer of 10 half-word is as follows:

Same Data Path Width for Source and Destination

```
//SETUP SECTION
1          stf r1, PSA|SZ16|F          //r1=0x1002 Source address register setup
2          stf r2, PDA|SZ16|F          //r2=0x2006 Destination address register
setup
3          bdf ERROR_ADDR_SETUP
4          ldi r0,0xa                    //loop counter is 10
//MAIN LOOP TRANSFER
copy_loop:

5          loop 2,0
6          ldf r7,PD|CPY                //Reads 1 half-word from src and writes to
dest.
7          yield
8          bdf ERROR_DURING_XFER
ERROR_ADDR_SETUP:                //correction of PSA/PDA setup and jumps to main loop transfer
ERROR_DURING_XFER:
//flag error is set,
//PS can be read to know if error occurs during read or write access.
```

If a data transfer must occur between two word peripherals, only the setup section should be updated. The transfer itself is always performed by the hardware loop instruction.

All instructions are executed in one cycle (change of flow excepted). On instruction 6, a single read access is triggered, read data is staged in PD, and a write-to-destination is executed. When the transfers are in progress, the SDMA can execute the next instructions in parallel. If instruction 6, which performs the copy transfer, is executed while the previous access is not over, SDMA is stalled and instruction ldf is a multi-cycle instruction.

7.2.6.2.2.2 Source and Destination Target Have a Different Data Path Width

When the source and destination target have a different data path width, copy mode cannot be used, and any attempt to initiate a copy transfer immediately raises an error, which is stored in the SF flag.

The following example shows the SDMA code that could transfer 10 words from a *word* (32-bit) peripheral to a *half-word* peripheral whose addresses are preliminary and stored in r1 and r2.

Different Data Path Width for Source and Destination

```

//SETUP SECTION
1      stf r1, PSA|SZ32|F|PF          //r1=0x1000 and prefetch data
2      stf r2, PDA|SZ16|F            //r2=0x2006
3      bdf ERROR_ADDR_SETUP
4      ldi r0,0xa                     //loop counter is 10
//MAIN LOOP TRANSFER
main_loop_xfer_16_16:
5      loop 6,0
6      ldf r7,PD                      //copy 32-bit of PD in r7
7      stf r7,PD                      //store 16 LSB of r7 in PD and a flush is
executed
8      rorb r7
9      rorb r7                        //16 MSB --> 16 LSB
10     stf r7,PD                      //store 16 LSB of r6 in PD and a flush.
11     yield

```

On instruction 1, when the source address register is programmed and a data prefetch is required, a read access is executed. In parallel, the SDMA executes instructions 2 to 5. On instruction 6, the SDMA tries to read data that was fetched by instruction 1. If data is ready, the ldf will be a one cycle instruction; otherwise, the SDMA is stalled as long as the read access is not finished. Then, the 16 LSB of the read data is stored in PD and automatically flushed to the destination peripheral. In parallel, the SDMA executes the rotation instructions (8, 9), and stores the 16 MSB of the read data into PD. If a previous write access is finished, instruction 10 will be a one-cycle instruction.

The main loop transfer may appear inefficient, but due to wait states imposed to the peripheral DMA each time an external access is performed, a software pipeline is in place. During the time needed to flush PD, the SDMA executes the move and rotation operations. SDMA executes instructions in parallel with DMA accesses.

7.2.6.2.3 Transfer Between Peripheral and External Memory

7.2.6.2.3.1 Peripheral to External Memory Transfer

A transfer from a peripheral to the external memory controller involves the peripheral DMA and the burst DMA.

The code for transferring 100 word from word peripheral to the external memory would be as follows:

Peripheral to External Memory Transfer

```

//SETUP SECTION source and destination addresses are already in r1 and r2
1      stf r1, PSA|SZ16|F|PF          //r1=0x1000 and prefetch 32-bit data
2      stf r2, MDA                    //r2=0x2000, setup burst DMA destination
address
3      bdf ERROR_ADDR_SETUP
4      ldi r0,0x64                     //loop counter is 100
5
//MAIN LOOP TRANSFER
6      loop 3,0
7      ldf r1,PD|PF                    // read 32 bits of PD and initiate a new read

```

```

access.
8          stf r1,MD|32                // store 32 bits of r1 in the MD fifo.
9          yield
10         ldf r1,PD                    // last word data is read
11         stf r1,MD|32|FL              // to flush all remaining bytes of MD

```

On instruction 1, the source address register of the peripheral DMA is programmed and data is fetched. This data is stored in PD and the SDMA reads PD during instruction 7, which is a one-cycle instruction that is read-access finished. On the same instruction (7), a data prefetch is required and a read access to the source peripheral is executed. In parallel, the SDMA stored the previous read data into the data register of MD. When MD (which is an eight-word FIFO) is full, a burst write access is executed to empty the FIFO. As long as the next SDMA instructions do not access the burst DMA, they will be one-cycle instructions. The following figures show how the peripheral DMA and burst DMA work in parallel.

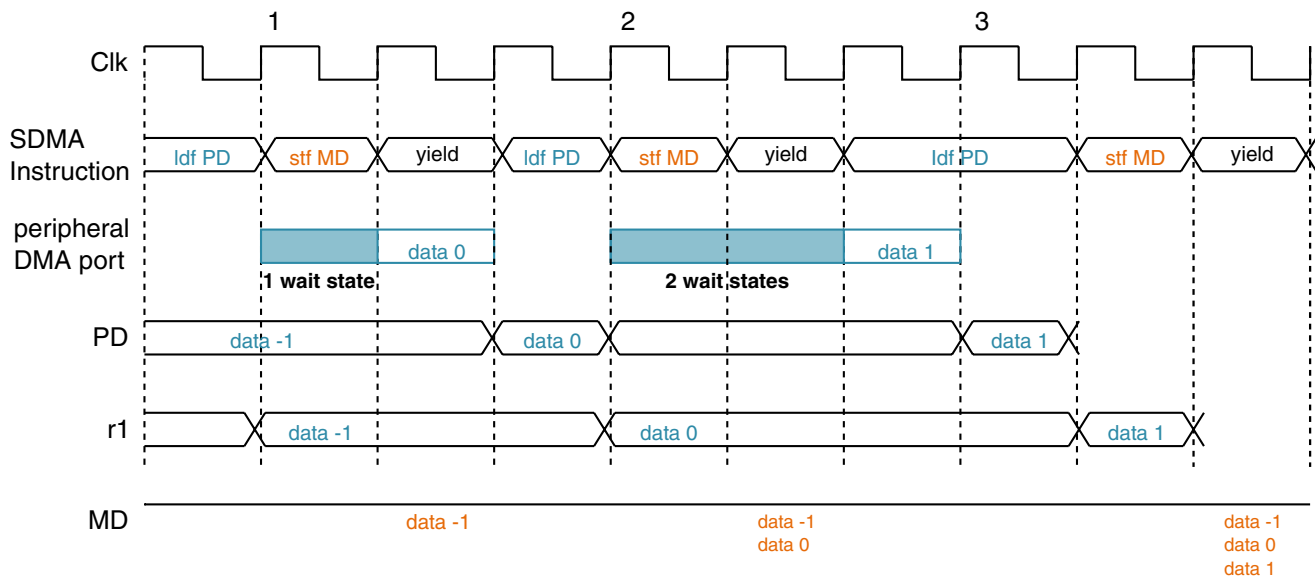


Figure 7-21. Peripheral to External Memory Example (1)

As seen in the figure above, the read access triggered by the ldf PD instruction is symbolized by the blue bar when in progress. After wait states, the read data (data 0, data 1) is stored in PD on the clk rising edge. On edge 2, data 0 is available in PD so it can be transferred to the SDMA general register r1, and then stored in MD FIFO. On edge 3, data 1 is not in PD; therefore, SDMA is stalled on the ldf instruction, which lasts two cycles. The figure below shows an example of when MD FIFO is full with data.

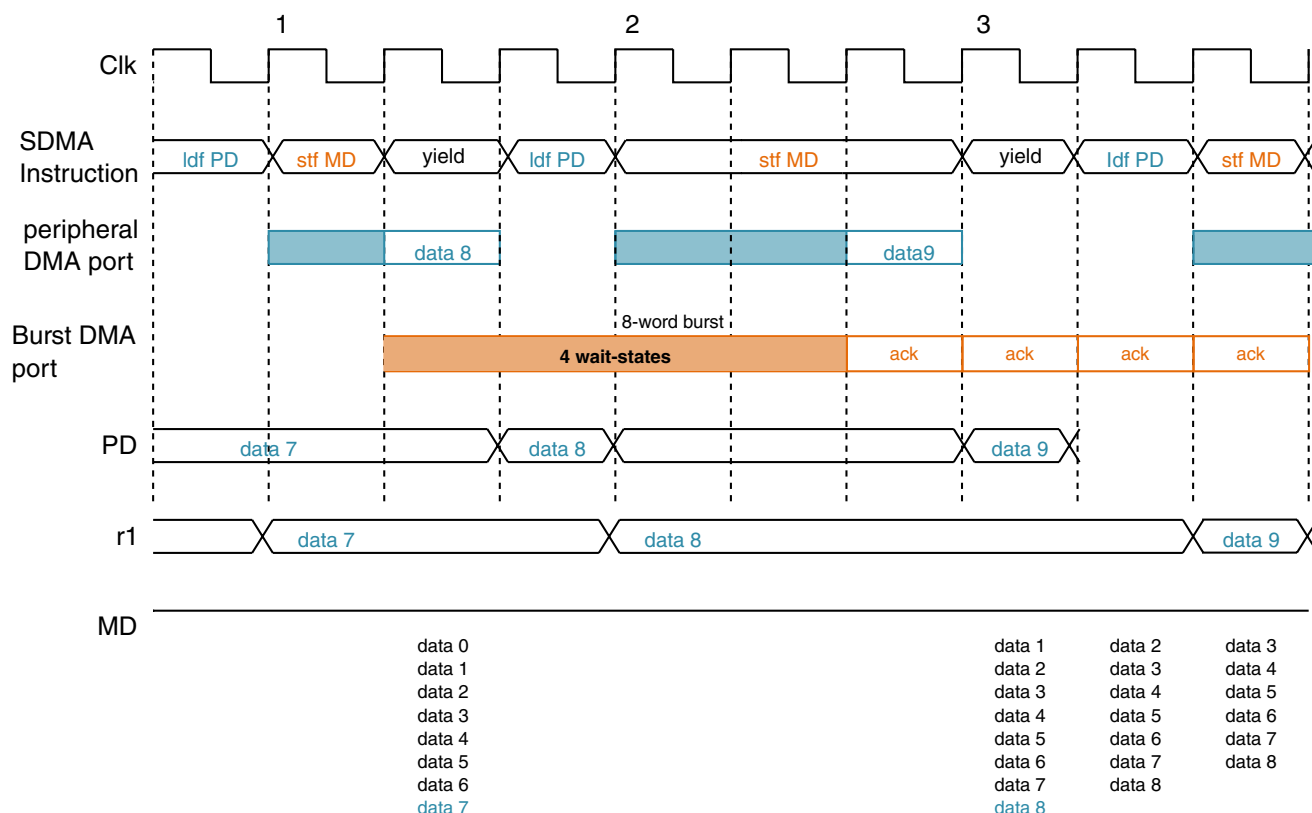


Figure 7-22. Peripheral to External Memory Example (2)

In the previous figure, the write bar means the burst DMA is performing a write burst access. The latency to have the first write acknowledge is four cycles. SDMA is stalled on instruction `stf` because no acknowledge was received, MD FIFO is full, and there is no empty slot to store data 9. When an acknowledge is sampled by the burst DMA, FIFO is shifted and data 8 is written. As long as there is at least one empty slot in MD FIFO, the `stf MD` instruction lasts one cycle.

7.2.6.2.3.2 External Memory to Peripheral Transfer

A transfer from the external memory to a peripheral involves the peripheral DMA and the burst DMA.

The code for transferring 100 word from external memory to a word peripheral would be as follows:

External Memory to Peripheral Transfer

```
//SETUP SECTION source and destination addresses are already in r1 and r2
1      stf r1, MSA|PF           //r1=0x1000 and starts a 8-word read burst
2      stf r2, PDA|SZ32|P       //r2=0x2010, setup peripheral DMA destination address
3      bdf ERROR_ADDR_SETUP
4      ldi r0,0x64              //loop counter is 100
//MAIN LOOP TRANSFER
```



```

6      loop 3,0
7      ldf r1,MD|32|PF          // read 32 bits of MD and initiate a new read access
                                   // if MD is empty after this reading.
8      stf r1,PD                // store 32 bits of r1 in the PD.
9      yield
10     ldf r1,MD|32              // last word data is read
11     stf r1,PD                // last write access

```

On instruction 1, a read burst of 8 words begins. Read data is staged into MD. On instruction 7 (and if data is available in MD), 32 bits are copied into r1. Then instruction 8 writes them into PD and an automatic flush is executed. The SDMA core, peripheral DMA, and burst DMA can work in parallel as long as no SDMA instruction tries to start a new write access on the peripheral DMA while the previous access is still in progress, or as long as there is data in MD when the SDMA tries to read it.

7.2.6.2.4 Transfer Between External Memory and Internal Memory

Since the internal memory (Arm platform RAM) is accessed via the peripheral DMA and the external memory is accessed via the burst DMA, the SDMA scripts that are described in [Transfer Between Peripheral and External Memory](#) can be reused. The exception is that the peripheral DMA address registers (PSA or PDA, depending on the script) should be programmed in incremented mode rather than frozen mode.

7.2.6.2.4.1 Internal Memory to Internal Memory

The internal memory can only be accessed via the peripheral DMA, so the script described in [Peripheral to Peripheral Transfer](#) can be reused with a different programming of the peripheral DMA address registers.

7.2.6.2.4.2 Transfer Between Peripheral and Internal Memory

For this transfer, the peripheral DMA is also used in copy mode.

The SDMA script is very similar to the one described in [Peripheral to Peripheral Transfer](#), except for the peripheral DMA address registers programming.

7.2.7 Arm Platform Memory Map and Control Register Definitions

The Arm platform controls the SDMA by means of several interface registers. Those registers are described in the current section.

All registers are clocked with the SDMA clock (which means the Arm platform must ensure that the SDMA clock is running when it wants to access any register).

SDMAARM memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
302C_0000	Arm platform Channel 0 Pointer (SDMAARM2_MC0PTR)	32	R/W	0000_0000h	7.2.7.1/1256
302C_0004	Channel Interrupts (SDMAARM2_INTR)	32	w1c	0000_0000h	7.2.7.2/1256
302C_0008	Channel Stop/Channel Status (SDMAARM2_STOP_STAT)	32	w1c	0000_0000h	7.2.7.3/1257
302C_000C	Channel Start (SDMAARM2_HSTART)	32	R/W	0000_0000h	7.2.7.4/1257
302C_0010	Channel Event Override (SDMAARM2_EVTOVR)	32	R/W	0000_0000h	7.2.7.5/1258
302C_0014	Channel BP Override (SDMAARM2_DSPOVR)	32	R/W	FFFF_FFFFh	7.2.7.6/1258
302C_0018	Channel Arm platform Override (SDMAARM2_HOSTOVR)	32	R/W	0000_0000h	7.2.7.7/1258
302C_001C	Channel Event Pending (SDMAARM2_EVTPEND)	32	w1c	0000_0000h	7.2.7.8/1259
302C_0024	Reset Register (SDMAARM2_RESET)	32	R	0000_0000h	7.2.7.9/1260
302C_0028	DMA Request Error Register (SDMAARM2_EVTERR)	32	R	0000_0000h	7.2.7.10/1261
302C_002C	Channel Arm platform Interrupt Mask (SDMAARM2_INTRMASK)	32	R/W	0000_0000h	7.2.7.11/1261
302C_0030	Schedule Status (SDMAARM2_PSW)	32	R	0000_0000h	7.2.7.12/1262
302C_0034	DMA Request Error Register (SDMAARM2_EVTERRDBG)	32	R	0000_0000h	7.2.7.13/1262
302C_0038	Configuration Register (SDMAARM2_CONFIG)	32	R/W	0000_0003h	7.2.7.14/1263
302C_003C	SDMA LOCK (SDMAARM2_SDMA_LOCK)	32	R/W	0000_0000h	7.2.7.15/1264
302C_0040	OnCE Enable (SDMAARM2_ONCE_ENB)	32	R/W	0000_0000h	7.2.7.16/1265

Table continues on the next page...

SDMAARM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
302C_0044	OnCE Data Register (SDMAARM2_ONCE_DATA)	32	R/W	0000_0000h	7.2.7.17/1265
302C_0048	OnCE Instruction Register (SDMAARM2_ONCE_INSTR)	32	R/W	0000_0000h	7.2.7.18/1266
302C_004C	OnCE Status Register (SDMAARM2_ONCE_STAT)	32	R	0000_E000h	7.2.7.19/1266
302C_0050	OnCE Command Register (SDMAARM2_ONCE_CMD)	32	R/W	0000_0000h	7.2.7.20/1268
302C_0058	Illegal Instruction Trap Address (SDMAARM2_ILLINSTADDR)	32	R/W	0000_0001h	7.2.7.21/1268
302C_005C	Channel 0 Boot Address (SDMAARM2_CHN0ADDR)	32	R/W	0000_0050h	7.2.7.22/1269
302C_0060	DMA Requests (SDMAARM2_EVT_MIRROR)	32	R	0000_0000h	7.2.7.23/1270
302C_0064	DMA Requests 2 (SDMAARM2_EVT_MIRROR2)	32	R	0000_0000h	7.2.7.24/1270
302C_0070	Cross-Trigger Events Configuration Register 1 (SDMAARM2_XTRIG_CONF1)	32	R/W	0000_0000h	7.2.7.25/1271
302C_0074	Cross-Trigger Events Configuration Register 2 (SDMAARM2_XTRIG_CONF2)	32	R/W	0000_0000h	7.2.7.26/1272
302C_0100	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI0)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0104	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI1)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0108	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI2)	32	R/W	0000_0000h	7.2.7.27/1273
302C_010C	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI3)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0110	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI4)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0114	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI5)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0118	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI6)	32	R/W	0000_0000h	7.2.7.27/1273
302C_011C	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI7)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0120	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI8)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0124	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI9)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0128	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI10)	32	R/W	0000_0000h	7.2.7.27/1273
302C_012C	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI11)	32	R/W	0000_0000h	7.2.7.27/1273

Table continues on the next page...

SDMAARM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
302C_0130	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI12)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0134	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI13)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0138	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI14)	32	R/W	0000_0000h	7.2.7.27/1273
302C_013C	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI15)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0140	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI16)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0144	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI17)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0148	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI18)	32	R/W	0000_0000h	7.2.7.27/1273
302C_014C	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI19)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0150	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI20)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0154	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI21)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0158	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI22)	32	R/W	0000_0000h	7.2.7.27/1273
302C_015C	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI23)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0160	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI24)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0164	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI25)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0168	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI26)	32	R/W	0000_0000h	7.2.7.27/1273
302C_016C	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI27)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0170	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI28)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0174	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI29)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0178	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI30)	32	R/W	0000_0000h	7.2.7.27/1273
302C_017C	Channel Priority Registers (SDMAARM2_SDMA_CHNPRI31)	32	R/W	0000_0000h	7.2.7.27/1273
302C_0200	Channel Enable RAM (SDMAARM2_CHNENBL0)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0204	Channel Enable RAM (SDMAARM2_CHNENBL1)	32	R/W	0000_0000h	7.2.7.28/1274

Table continues on the next page...

SDMAARM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
302C_0208	Channel Enable RAM (SDMAARM2_CHNENBL2)	32	R/W	0000_0000h	7.2.7.28/1274
302C_020C	Channel Enable RAM (SDMAARM2_CHNENBL3)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0210	Channel Enable RAM (SDMAARM2_CHNENBL4)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0214	Channel Enable RAM (SDMAARM2_CHNENBL5)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0218	Channel Enable RAM (SDMAARM2_CHNENBL6)	32	R/W	0000_0000h	7.2.7.28/1274
302C_021C	Channel Enable RAM (SDMAARM2_CHNENBL7)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0220	Channel Enable RAM (SDMAARM2_CHNENBL8)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0224	Channel Enable RAM (SDMAARM2_CHNENBL9)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0228	Channel Enable RAM (SDMAARM2_CHNENBL10)	32	R/W	0000_0000h	7.2.7.28/1274
302C_022C	Channel Enable RAM (SDMAARM2_CHNENBL11)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0230	Channel Enable RAM (SDMAARM2_CHNENBL12)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0234	Channel Enable RAM (SDMAARM2_CHNENBL13)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0238	Channel Enable RAM (SDMAARM2_CHNENBL14)	32	R/W	0000_0000h	7.2.7.28/1274
302C_023C	Channel Enable RAM (SDMAARM2_CHNENBL15)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0240	Channel Enable RAM (SDMAARM2_CHNENBL16)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0244	Channel Enable RAM (SDMAARM2_CHNENBL17)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0248	Channel Enable RAM (SDMAARM2_CHNENBL18)	32	R/W	0000_0000h	7.2.7.28/1274
302C_024C	Channel Enable RAM (SDMAARM2_CHNENBL19)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0250	Channel Enable RAM (SDMAARM2_CHNENBL20)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0254	Channel Enable RAM (SDMAARM2_CHNENBL21)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0258	Channel Enable RAM (SDMAARM2_CHNENBL22)	32	R/W	0000_0000h	7.2.7.28/1274
302C_025C	Channel Enable RAM (SDMAARM2_CHNENBL23)	32	R/W	0000_0000h	7.2.7.28/1274

Table continues on the next page...

SDMAARM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
302C_0260	Channel Enable RAM (SDMAARM2_CHNENBL24)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0264	Channel Enable RAM (SDMAARM2_CHNENBL25)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0268	Channel Enable RAM (SDMAARM2_CHNENBL26)	32	R/W	0000_0000h	7.2.7.28/1274
302C_026C	Channel Enable RAM (SDMAARM2_CHNENBL27)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0270	Channel Enable RAM (SDMAARM2_CHNENBL28)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0274	Channel Enable RAM (SDMAARM2_CHNENBL29)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0278	Channel Enable RAM (SDMAARM2_CHNENBL30)	32	R/W	0000_0000h	7.2.7.28/1274
302C_027C	Channel Enable RAM (SDMAARM2_CHNENBL31)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0280	Channel Enable RAM (SDMAARM2_CHNENBL32)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0284	Channel Enable RAM (SDMAARM2_CHNENBL33)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0288	Channel Enable RAM (SDMAARM2_CHNENBL34)	32	R/W	0000_0000h	7.2.7.28/1274
302C_028C	Channel Enable RAM (SDMAARM2_CHNENBL35)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0290	Channel Enable RAM (SDMAARM2_CHNENBL36)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0294	Channel Enable RAM (SDMAARM2_CHNENBL37)	32	R/W	0000_0000h	7.2.7.28/1274
302C_0298	Channel Enable RAM (SDMAARM2_CHNENBL38)	32	R/W	0000_0000h	7.2.7.28/1274
302C_029C	Channel Enable RAM (SDMAARM2_CHNENBL39)	32	R/W	0000_0000h	7.2.7.28/1274
302C_02A0	Channel Enable RAM (SDMAARM2_CHNENBL40)	32	R/W	0000_0000h	7.2.7.28/1274
302C_02A4	Channel Enable RAM (SDMAARM2_CHNENBL41)	32	R/W	0000_0000h	7.2.7.28/1274
302C_02A8	Channel Enable RAM (SDMAARM2_CHNENBL42)	32	R/W	0000_0000h	7.2.7.28/1274
302C_02AC	Channel Enable RAM (SDMAARM2_CHNENBL43)	32	R/W	0000_0000h	7.2.7.28/1274
302C_02B0	Channel Enable RAM (SDMAARM2_CHNENBL44)	32	R/W	0000_0000h	7.2.7.28/1274
302C_02B4	Channel Enable RAM (SDMAARM2_CHNENBL45)	32	R/W	0000_0000h	7.2.7.28/1274

Table continues on the next page...

SDMAARM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
302C_02B8	Channel Enable RAM (SDMAARM2_CHNENBL46)	32	R/W	0000_0000h	7.2.7.28/1274
302C_02BC	Channel Enable RAM (SDMAARM2_CHNENBL47)	32	R/W	0000_0000h	7.2.7.28/1274
302C_1000	SDMA DONE0 Configuration (SDMAARM2_DONE0_CONFIG)	32	R/W	1F1F_1F1Fh	7.2.7.29/1274
302C_1004	SDMA DONE1 Configuration (SDMAARM2_DONE1_CONFIG)	32	R/W	1F1F_1F1Fh	7.2.7.30/1276
30BD_0000	Arm platform Channel 0 Pointer (SDMAARM1_MC0PTR)	32	R/W	0000_0000h	7.2.7.1/1256
30BD_0004	Channel Interrupts (SDMAARM1_INTR)	32	w1c	0000_0000h	7.2.7.2/1256
30BD_0008	Channel Stop/Channel Status (SDMAARM1_STOP_STAT)	32	w1c	0000_0000h	7.2.7.3/1257
30BD_000C	Channel Start (SDMAARM1_HSTART)	32	R/W	0000_0000h	7.2.7.4/1257
30BD_0010	Channel Event Override (SDMAARM1_EVTOVR)	32	R/W	0000_0000h	7.2.7.5/1258
30BD_0014	Channel BP Override (SDMAARM1_DSPOVR)	32	R/W	FFFF_FFFFh	7.2.7.6/1258
30BD_0018	Channel Arm platform Override (SDMAARM1_HOSTOVR)	32	R/W	0000_0000h	7.2.7.7/1258
30BD_001C	Channel Event Pending (SDMAARM1_EVTPEND)	32	w1c	0000_0000h	7.2.7.8/1259
30BD_0024	Reset Register (SDMAARM1_RESET)	32	R	0000_0000h	7.2.7.9/1260
30BD_0028	DMA Request Error Register (SDMAARM1_EVTERR)	32	R	0000_0000h	7.2.7.10/1261
30BD_002C	Channel Arm platform Interrupt Mask (SDMAARM1_INTRMASK)	32	R/W	0000_0000h	7.2.7.11/1261
30BD_0030	Schedule Status (SDMAARM1_PSW)	32	R	0000_0000h	7.2.7.12/1262
30BD_0034	DMA Request Error Register (SDMAARM1_EVTERRDBG)	32	R	0000_0000h	7.2.7.13/1262
30BD_0038	Configuration Register (SDMAARM1_CONFIG)	32	R/W	0000_0003h	7.2.7.14/1263
30BD_003C	SDMA LOCK (SDMAARM1_SDMA_LOCK)	32	R/W	0000_0000h	7.2.7.15/1264
30BD_0040	OnCE Enable (SDMAARM1_ONCE_ENB)	32	R/W	0000_0000h	7.2.7.16/1265
30BD_0044	OnCE Data Register (SDMAARM1_ONCE_DATA)	32	R/W	0000_0000h	7.2.7.17/1265
30BD_0048	OnCE Instruction Register (SDMAARM1_ONCE_INSTR)	32	R/W	0000_0000h	7.2.7.18/1266

Table continues on the next page...

SDMAARM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
30BD_004C	OnCE Status Register (SDMAARM1_ONCE_STAT)	32	R	0000_E000h	7.2.7.19/1266
30BD_0050	OnCE Command Register (SDMAARM1_ONCE_CMD)	32	R/W	0000_0000h	7.2.7.20/1268
30BD_0058	Illegal Instruction Trap Address (SDMAARM1_ILLINSTADDR)	32	R/W	0000_0001h	7.2.7.21/1268
30BD_005C	Channel 0 Boot Address (SDMAARM1_CHN0ADDR)	32	R/W	0000_0050h	7.2.7.22/1269
30BD_0060	DMA Requests (SDMAARM1_EVT_MIRROR)	32	R	0000_0000h	7.2.7.23/1270
30BD_0064	DMA Requests 2 (SDMAARM1_EVT_MIRROR2)	32	R	0000_0000h	7.2.7.24/1270
30BD_0070	Cross-Trigger Events Configuration Register 1 (SDMAARM1_XTRIG_CONF1)	32	R/W	0000_0000h	7.2.7.25/1271
30BD_0074	Cross-Trigger Events Configuration Register 2 (SDMAARM1_XTRIG_CONF2)	32	R/W	0000_0000h	7.2.7.26/1272
30BD_0100	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI0)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0104	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI1)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0108	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI2)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_010C	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI3)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0110	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI4)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0114	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI5)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0118	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI6)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_011C	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI7)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0120	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI8)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0124	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI9)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0128	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI10)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_012C	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI11)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0130	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI12)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0134	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI13)	32	R/W	0000_0000h	7.2.7.27/1273

Table continues on the next page...

SDMAARM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
30BD_0138	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI14)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_013C	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI15)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0140	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI16)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0144	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI17)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0148	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI18)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_014C	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI19)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0150	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI20)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0154	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI21)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0158	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI22)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_015C	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI23)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0160	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI24)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0164	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI25)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0168	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI26)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_016C	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI27)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0170	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI28)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0174	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI29)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0178	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI30)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_017C	Channel Priority Registers (SDMAARM1_SDMA_CHNPRI31)	32	R/W	0000_0000h	7.2.7.27/1273
30BD_0200	Channel Enable RAM (SDMAARM1_CHNENBL0)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0204	Channel Enable RAM (SDMAARM1_CHNENBL1)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0208	Channel Enable RAM (SDMAARM1_CHNENBL2)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_020C	Channel Enable RAM (SDMAARM1_CHNENBL3)	32	R/W	0000_0000h	7.2.7.28/1274

Table continues on the next page...

SDMAARM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
30BD_0210	Channel Enable RAM (SDMAARM1_CHNENBL4)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0214	Channel Enable RAM (SDMAARM1_CHNENBL5)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0218	Channel Enable RAM (SDMAARM1_CHNENBL6)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_021C	Channel Enable RAM (SDMAARM1_CHNENBL7)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0220	Channel Enable RAM (SDMAARM1_CHNENBL8)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0224	Channel Enable RAM (SDMAARM1_CHNENBL9)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0228	Channel Enable RAM (SDMAARM1_CHNENBL10)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_022C	Channel Enable RAM (SDMAARM1_CHNENBL11)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0230	Channel Enable RAM (SDMAARM1_CHNENBL12)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0234	Channel Enable RAM (SDMAARM1_CHNENBL13)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0238	Channel Enable RAM (SDMAARM1_CHNENBL14)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_023C	Channel Enable RAM (SDMAARM1_CHNENBL15)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0240	Channel Enable RAM (SDMAARM1_CHNENBL16)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0244	Channel Enable RAM (SDMAARM1_CHNENBL17)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0248	Channel Enable RAM (SDMAARM1_CHNENBL18)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_024C	Channel Enable RAM (SDMAARM1_CHNENBL19)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0250	Channel Enable RAM (SDMAARM1_CHNENBL20)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0254	Channel Enable RAM (SDMAARM1_CHNENBL21)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0258	Channel Enable RAM (SDMAARM1_CHNENBL22)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_025C	Channel Enable RAM (SDMAARM1_CHNENBL23)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0260	Channel Enable RAM (SDMAARM1_CHNENBL24)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0264	Channel Enable RAM (SDMAARM1_CHNENBL25)	32	R/W	0000_0000h	7.2.7.28/1274

Table continues on the next page...

SDMAARM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
30BD_0268	Channel Enable RAM (SDMAARM1_CHNENBL26)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_026C	Channel Enable RAM (SDMAARM1_CHNENBL27)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0270	Channel Enable RAM (SDMAARM1_CHNENBL28)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0274	Channel Enable RAM (SDMAARM1_CHNENBL29)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0278	Channel Enable RAM (SDMAARM1_CHNENBL30)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_027C	Channel Enable RAM (SDMAARM1_CHNENBL31)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0280	Channel Enable RAM (SDMAARM1_CHNENBL32)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0284	Channel Enable RAM (SDMAARM1_CHNENBL33)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0288	Channel Enable RAM (SDMAARM1_CHNENBL34)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_028C	Channel Enable RAM (SDMAARM1_CHNENBL35)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0290	Channel Enable RAM (SDMAARM1_CHNENBL36)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0294	Channel Enable RAM (SDMAARM1_CHNENBL37)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_0298	Channel Enable RAM (SDMAARM1_CHNENBL38)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_029C	Channel Enable RAM (SDMAARM1_CHNENBL39)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_02A0	Channel Enable RAM (SDMAARM1_CHNENBL40)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_02A4	Channel Enable RAM (SDMAARM1_CHNENBL41)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_02A8	Channel Enable RAM (SDMAARM1_CHNENBL42)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_02AC	Channel Enable RAM (SDMAARM1_CHNENBL43)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_02B0	Channel Enable RAM (SDMAARM1_CHNENBL44)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_02B4	Channel Enable RAM (SDMAARM1_CHNENBL45)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_02B8	Channel Enable RAM (SDMAARM1_CHNENBL46)	32	R/W	0000_0000h	7.2.7.28/1274
30BD_02BC	Channel Enable RAM (SDMAARM1_CHNENBL47)	32	R/W	0000_0000h	7.2.7.28/1274

Table continues on the next page...

SDMAARM memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
30BD_1000	SDMA DONE0 Configuration (SDMAARM1_DONE0_CONFIG)	32	R/W	1F1F_1F1Fh	7.2.7.29/1274
30BD_1004	SDMA DONE1 Configuration (SDMAARM1_DONE1_CONFIG)	32	R/W	1F1F_1F1Fh	7.2.7.30/1276

7.2.7.1 Arm platform Channel 0 Pointer (SDMAARMx_MC0PTR)

Address: Base address + 0h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_MC0PTR field descriptions

Field	Description
MC0PTR	Channel 0 Pointer contains the 32-bit address, in Arm platform memory, of channel 0 control block (the boot channel). Appendix A fully describes the SDMA Application Programming Interface (API). The Arm platform has a read/write access and the SDMA has a read-only access.

7.2.7.2 Channel Interrupts (SDMAARMx_INTR)

Address: Base address + 4h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_INTR field descriptions

Field	Description
HI[31:0]	The Arm platform Interrupts register contains the 32 HI[i] bits. If any bit is set, it will cause an interrupt to the Arm platform. This register is a "write-ones" register to the Arm platform. When the Arm platform sets a bit in this register the corresponding HI[i] bit is cleared. The interrupt service routine should clear individual channel bits when their interrupts are serviced, failure to do so will cause continuous interrupts. The SDMA is responsible for setting the HI[i] bit corresponding to the current channel when the corresponding done instruction is executed.

7.2.7.3 Channel Stop/Channel Status (SDMAARMx_STOP_STAT)

Address: Base address + 8h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	HE																															
W	w1c																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_STOP_STAT field descriptions

Field	Description
HE	This 32-bit register gives access to the Arm platform Enable bits. There is one bit for every channel. This register is a "write-ones" register to the Arm platform. When the Arm platform writes 1 in bit <i>i</i> of this register, it clears the HE[<i>i</i>] and HSTART[<i>i</i>] bits. Reading this register yields the current state of the HE[<i>i</i>] bits.

7.2.7.4 Channel Start (SDMAARMx_HSTART)

Address: Base address + Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	HSTART_HE																															
W	w1c																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_HSTART field descriptions

Field	Description
HSTART_HE	The HSTART_HE registers are 32 bits wide with one bit for every channel. When a bit is written to 1, it enables the corresponding channel. Two physical registers are accessed with that address (HSTART and HE), which enables the Arm platform to trigger a channel a second time before the first trigger is processed. - This register is a "write-ones" register to the Arm platform. Neither HSTART[<i>i</i>] bit can be set while the corresponding HE[<i>i</i>] bit is cleared. - When the Arm platform tries to set the HSTART[<i>i</i>] bit by writing a one (if the corresponding HE[<i>i</i>] bit is clear), the bit in the HSTART[<i>i</i>] register will remain cleared and the HE[<i>i</i>] bit will be set. - If the corresponding HE[<i>i</i>] bit was already set, the HSTART[<i>i</i>] bit will be set. The next time the SDMA channel <i>i</i> attempts to clear the HE[<i>i</i>] bit by means of a done instruction, the bit in the HSTART[<i>i</i>] register will be cleared and the HE[<i>i</i>] bit will take the old value of the HSTART[<i>i</i>] bit. - Reading this register yields the current state of the HSTART[<i>i</i>] bits. This mechanism enables the Arm platform to pipeline two HSTART commands per channel.

7.2.7.5 Channel Event Override (SDMAARMx_EVTOVR)

Address: Base address + 10h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	EO																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_EVTOVR field descriptions

Field	Description
EO	The Channel Event Override register contains the 32 EO[i] bits. A bit set in this register causes the SDMA to ignore DMA requests when scheduling the corresponding channel.

7.2.7.6 Channel BP Override (SDMAARMx_DSPOVR)

Address: Base address + 14h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DO																															
W																																
Reset	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

SDMAARMx_DSPOVR field descriptions

Field	Description
DO	This register is reserved. All DO bits should be set to the reset value of 1. A setting of 0 will prevent SDMA channels from starting according to the condition described in Runnable Channels Evaluation. 0 - Reserved 1 - Reset value.

7.2.7.7 Channel Arm platform Override (SDMAARMx_HOSTOVR)

Address: Base address + 18h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	HO																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_HOSTOVR field descriptions

Field	Description
HO	The Channel Arm platform Override register contains the 32 HO[i] bits. A bit set in this register causes the SDMA to ignore the Arm platform enable bit (HE) when scheduling the corresponding channel.

7.2.7.8 Channel Event Pending (SDMAARMx_EVTPEND)

Address: Base address + 1Ch offset

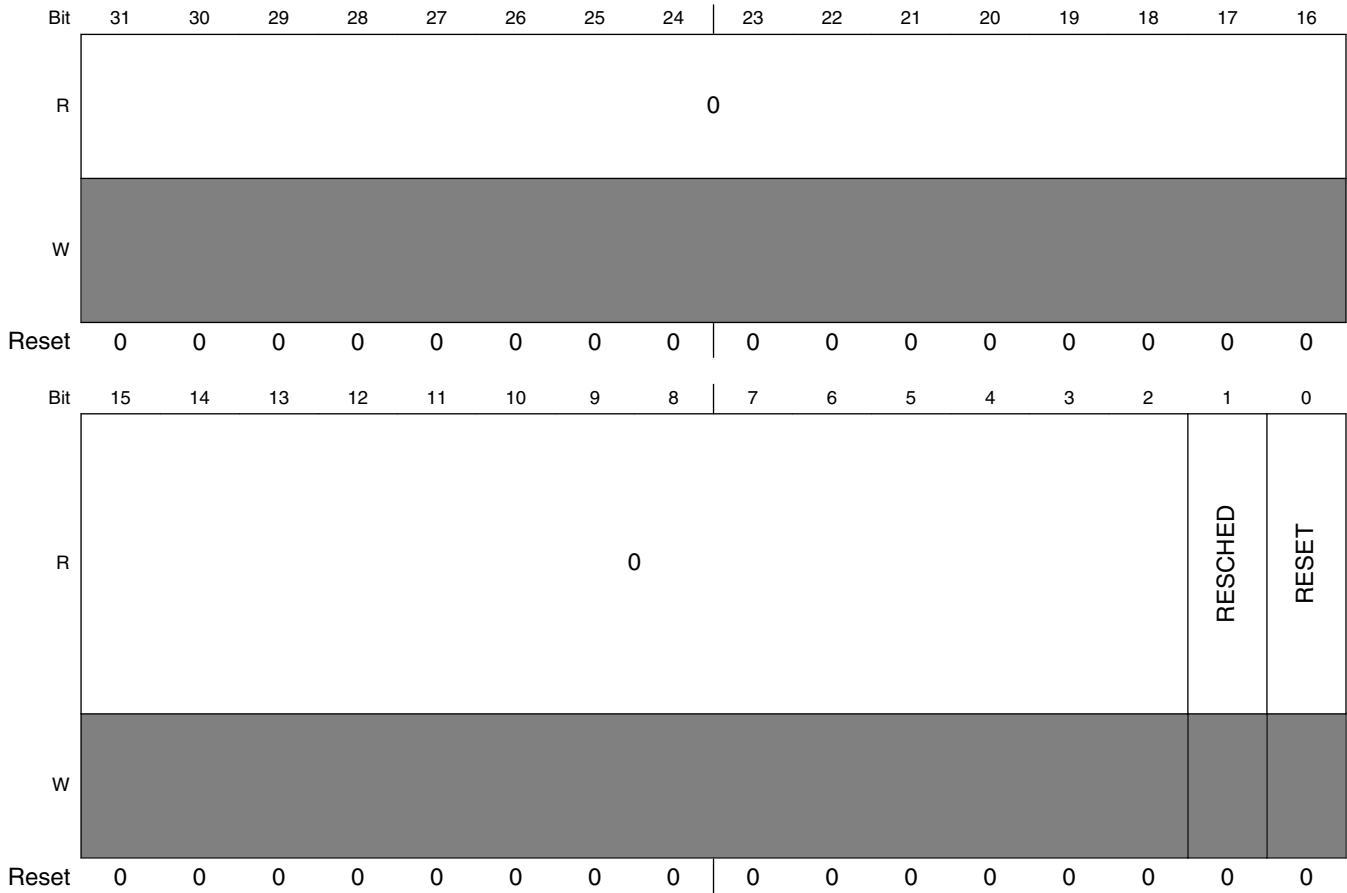
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	EP																															
W	w1c																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SDMAARMx_EVTPEND field descriptions

Field	Description
EP	The Channel Event Pending register contains the 32 EP[i] bits. Reading this register enables the Arm platform to determine what channels are pending after the reception of a DMA request. - Setting a bit in this register causes the SDMA to reevaluate scheduling as if a DMA request mapped on this channel had occurred. This is useful for starting up channels, so that initialization is done before awaiting the first request. The scheduler can also set bits in the EVTPEND register according to the received DMA requests. - The EP[i] bit may be cleared by the done instruction when running the channel <i>i</i> script. This is a "write-ones" mechanism: Writing a '0' does not clear the corresponding bit.

7.2.7.9 Reset Register (SDMAARMx_RESET)

Address: Base address + 24h offset



SDMAARMx_RESET field descriptions

Field	Description
31–2 Reserved	This read-only field is reserved and always has the value 0.
1 RESCHED	When set, this bit forces the SDMA to reschedule as if a script had executed a done instruction. This enables the Arm platform to recover from a runaway script on a channel by clearing its HE[i] bit via the STOP register, and then forcing a reschedule via the RESCHED bit. The RESCHED bit is cleared when the context switch starts.
0 RESET	When set, this bit causes the SDMA to be held in a software reset. The internal reset signal is held low 16 cycles; the RESET bit is automatically cleared when the internal reset signal rises.

7.2.7.10 DMA Request Error Register (SDMAARMx_EVTERR)

Address: Base address + 28h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CHNERR																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_EVTERR field descriptions

Field	Description
CHNERR	This register is used by the SDMA to warn the Arm platform when an incoming DMA request was detected and it triggers a channel that is already pending or being serviced. This probably means there is an overflow of data for that channel. - An interrupt is sent to the Arm platform if the corresponding channel bit is set in the INTRMASK register. - This is a "write-ones" register for the scheduler. It is only able to set the flags. The flags are cleared when the register is read by the Arm platform or during SDMA reset. - The CHNERR[i] bit is set when a DMA request that triggers channel <i>i</i> is received through the corresponding input pins and the EP[i] bit is already set; the EVTERR[i] bit is unaffected if the Arm platform tries to set the EP[i] bit, whereas, that EP[i] bit is already set.

7.2.7.11 Channel Arm platform Interrupt Mask (SDMAARMx_INTRMASK)

Address: Base address + 2Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_INTRMASK field descriptions

Field	Description
HIMASK	The Interrupt Mask Register contains 32 interrupt generation mask bits. If bit HIMASK[i] is set, the HI[i] bit is set and an interrupt is sent to the Arm platform when a DMA request error is detected on channel <i>i</i> (for example, EVTERR[i] is set).

7.2.7.12 Schedule Status (SDMAARMx_PSW)

Address: Base address + 30h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																NCP[2:0]			NCR[4:0]				CCP[2:0]			CCR[4:0]					
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SDMAARMx_PSW field descriptions

Field	Description
31–16 Reserved	This read-only field is reserved and always has the value 0.
15–13 NCP[2:0]	The Next Channel Priority gives the next pending channel priority. When the priority is 0, it means there is no pending channel and the NCR value has no meaning. 0 No running channel 1 Active channel priority
12–8 NCR[4:0]	The Next Channel Register indicates the number of the next scheduled pending channel with the highest priority.
7–4 CCP[2:0]	The Current Channel Priority indicates the priority of the current active channel. When the priority is 0, no channel is running: The SDMA is idle and the CCR value has no meaning. In the case that the SDMA has finished running the channel and has entered sleep state, CCP will indicate the priority of previous running channel. 0 No running channel 1 Active channel priority
CCR[4:0]	The Current Channel Register indicates the number of the channel that is being executed by the SDMA. SDMA. In the case that the SDMA has finished running the channel and has entered sleep state, CCR will indicate the previous running channel.

7.2.7.13 DMA Request Error Register (SDMAARMx_EVERRDBG)

Address: Base address + 34h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CHNERR																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_EVERRDBG field descriptions

Field	Description
CHNERR	This register is the same as EVERR, except reading it does not clear its contents. This address is meant to be used in debug mode. The Arm platform OnCE may check this register value without modifying it.

7.2.7.14 Configuration Register (SDMAARMx_CONFIG)

Address: Base address + 38h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0			DSPDMA	RTDOBS	0					ACR	0			CSM	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1

SDMAARMx_CONFIG field descriptions

Field	Description
31–13 Reserved	This read-only field is reserved and always has the value 0.
12 DSPDMA	This bit's function is reserved and should be configured as zero. 0 - Reset Value 1 - Reserved
11 RTDOBS	Indicates if Real-Time Debug pins are used: They do not toggle by default in order to reduce power consumption. 0 RTD pins disabled 1 RTD pins enabled
10–5 Reserved	This read-only field is reserved and always has the value 0.
4 ACR	Arm platform DMA / SDMA Core Clock Ratio. Selects the clock ratio between Arm platform DMA interfaces (burst DMA and peripheral DMA) and the internal SDMA core clock. The frequency selection is determined separately by the chip clock controller. This bit has to match the configuration of the chip clock controller that generates the clocks used in the SDMA. 0 Arm platform DMA interface frequency equals twice core frequency 1 Arm platform DMA interface frequency equals core frequency
3–2 Reserved	This read-only field is reserved and always has the value 0.
CSM	Selects the Context Switch Mode. The Arm platform has a read/write access. The SDMA cannot modify that register. The value at reset is 3, which selects the dynamic context switch by default. That register can be modified at anytime but the new context switch configuration will only be taken into account at the start of the next restore phase. NOTE: The first call to SDMA's channel 0 Bootload script after reset should use static context switch mode to ensure the context RAM for channel 0 is initialized in the channel SAVE Phase. After Channel 0 is run once, then any of the dynamic context modes can be used.

Table continues on the next page...

SDMAARMx_CONFIG field descriptions (continued)

Field	Description
0	static
1	dynamic low power
2	dynamic with no loop
3	dynamic

7.2.7.15 SDMA LOCK (SDMAARMx_SDMA_LOCK)

Address: Base address + 3Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_SDMA_LOCK field descriptions

Field	Description
31–2 Reserved	This read-only field is reserved and always has the value 0.
1 SRESET_LOCK_CLR	The SRESET_LOCK_CLR bit determine if the LOCK bit is cleared on a software reset triggered by writing to the RESET register. This bit cannot be changed if LOCK=1. SRESET_LOCK_CLR is cleared by conditions that clear the LOCK bit. 0 Software Reset does not clear the LOCK bit. 1 Software Reset clears the LOCK bit.
0 LOCK	The LOCK bit is used to restrict access to update SDMA script memory through ROM channel zero scripts and through the OnCE interface under Arm platform control. The LOCK bit is set: - The SDMA_LOCK, ONCE_ENB, CH0ADDR, and ILLINSTADDR registers cannot be written. These registers can be read, but writes are ignored. - SDMA software executing out of ROM or RAM may check the LOCK bit in the LOCK register Lock Status Register (SDMACORE_SDMA_LOCK) to determine if certain operations are allowed, such as up-loading new scripts.

Table continues on the next page...

SDMAARMx_SDMA_LOCK field descriptions (continued)

Field	Description
	Once the LOCK bit is set to 1, only a reset can clear it. The LOCK bit is cleared by a hardware reset. LOCK is cleared by a software reset only if SRESET_LOCK_CLR is set.
0	LOCK disengaged.
1	LOCK enabled.

7.2.7.16 OnCE Enable (SDMAARMx_ONCE_ENB)

Address: Base address + 40h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0															ENB
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_ONCE_ENB field descriptions

Field	Description
31–1 Reserved	This read-only field is reserved and always has the value 0.
0 ENB	The OnCE Enable register selects the OnCE control source: When cleared (0), the OnCE registers are accessed through the JTAG interface; when set (1), the OnCE registers may be accessed by the Arm platform through the addresses described, as follows. - After reset, the OnCE registers are accessed through the JTAG interface. - Writing a 1 to ENB enables the Arm platform to access the ONCE_* as any other SDMA control register. - When cleared (0), all the ONCE_xxx registers cannot be written. The value of ENB cannot be changed if the LOCK bit in the SDMA_LOCK register is set.

7.2.7.17 OnCE Data Register (SDMAARMx_ONCE_DATA)

Address: Base address + 44h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W	DATA																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_ONCE_DATA field descriptions

Field	Description
DATA	Data register of the OnCE JTAG controller. Refer to OnCE and Real-Time Debug for information on this register.

7.2.7.18 OnCE Instruction Register (SDMAARMx_ONCE_INSTR)

Address: Base address + 48h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																INSTR															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_ONCE_INSTR field descriptions

Field	Description
31–16 Reserved	This read-only field is reserved and always has the value 0.
INSTR	Instruction register of the OnCE JTAG controller. Refer to OnCE and Real-Time Debug for information on this register.

7.2.7.19 OnCE Status Register (SDMAARMx_ONCE_STAT)

Address: Base address + 4Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PST[3:0]				RCV	EDR	ODR	SWB	MST	0				ECDR		
W																
Reset	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_ONCE_STAT field descriptions

Field	Description
31–16 Reserved	This read-only field is reserved and always has the value 0.
15–12 PST[3:0]	The Processor Status bits reflect the state of the SDMA RISC engine. Its states are as follows: - The "Program" state is the usual instruction execution cycle. - The "Data" state is inserted when there are wait-states during a load or a store on the data bus (ld or st).

Table continues on the next page...

SDMAARMx_ONCE_STAT field descriptions (continued)

Field	Description
	- The "Change of Flow" state is the second cycle of any instruction that breaks the sequence of instructions (jumps and channel switching instructions). - The "Change of Flow in Loop" state is used when an error causes a hardware loop exit. - The "Debug" state means the SDMA is in debug mode. - The "Functional Unit" state is inserted when there are wait-states during a load or a store on the functional units bus (ldf or stf). - In "Sleep" modes, no script is running (this is the RISC engine idle state). The "after Reset" is slightly different because no context restoring phase will happen when a channel is triggered: The script located at address 0 will be executed (boot operation). - The "in Sleep" states are the same as above except they do not have any corresponding channel: They are used when entering debug mode after reset. The reason is that it is necessary to return to the "Sleep after Reset" state when leaving debug mode.
	0 Program 1 Data 2 Change of Flow 3 Change of Flow in Loop 4 Debug 5 Functional Unit 6 Sleep 7 Save 8 Program in Sleep 9 Data in Sleep 10 Change of Flow in Sleep 11 Change Flow in Loop in Sleep 12 Debug in Sleep 13 Functional Unit in Sleep 14 Sleep after Reset 15 Restore
11 RCV	After each write access to the real time buffer (RTB), the RCV bit is set. This bit is cleared after execution of an rbuffer command and on a JTAG reset.
10 EDR	This flag is raised when the SDMA has entered debug mode after an external debug request.
9 ODR	This flag is raised when the SDMA has entered debug mode after a OnCE debug request.
8 SWB	This flag is raised when the SDMA has entered debug mode after a software breakpoint.
7 MST	This flag is raised when the OnCE is controlled from the Arm platform peripheral interface. 0 The JTAG interface controls the OnCE. 1 The Arm platform peripheral interface controls the OnCE.
6–3 Reserved	This read-only field is reserved and always has the value 0.
ECDR	Event Cell Debug Request. If the debug request comes from the event cell, the reason for entering debug mode is given by the EDR bits. If all three bits of the EDR are reset, then it did not generate any debug request. If the cell did generate a debug request, then at least one of the EDR bits is set (the meaning of the encoding is given below). The encoding of the EDR bits is useful to find out more precisely why the debug request was generated. A debug request from an event cell is generated for a specific combination of the addra_cond, addrb_cond, and data_cond conditions. The value of those fields is given by the EDR bits.

Table continues on the next page...

SDMAARMx_ONCE_STAT field descriptions (continued)

Field	Description
0	1 matched addra_cond
1	1 matched addrb_cond
2	1 matched data_cond

7.2.7.20 OnCE Command Register (SDMAARMx_ONCE_CMD)

Address: Base address + 50h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																CMD															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_ONCE_CMD field descriptions

Field	Description
31–4 Reserved	This read-only field is reserved and always has the value 0.
CMD	Writing to this register will cause the OnCE to execute the command that is written. When needed, the ONCE_DATA and ONCE_INSTR registers should be loaded with the correct value before writing the command to that register. For a list of the OnCE commands and their usage, see OnCE and Real-Time Debug. NOTE: 7-15 reserved 0 rstatus 1 dmov 2 exec_once 3 run_core 4 exec_core 5 debug_rqst 6 rbuffer

7.2.7.21 Illegal Instruction Trap Address (SDMAARMx_ILLINSTADDR)

Address: Base address + 58h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																ILLINSTADDR															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

SDMAARMx_ILLINSTADDR field descriptions

Field	Description
31–14 Reserved	This read-only field is reserved and always has the value 0.
ILLINSTADDR	The Illegal Instruction Trap Address is the address where the SDMA jumps when an illegal instruction is executed. It is 0x0001 after reset. The value of ILLINSTADDR cannot be changed if the LOCK bit in the SDMA_LOCK register is set.

7.2.7.22 Channel 0 Boot Address (SDMAARMx_CHN0ADDR)

Address: Base address + 5Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	SMSZ	CHN0ADDR													
W																
Reset	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0

SDMAARMx_CHN0ADDR field descriptions

Field	Description
31–15 Reserved	This read-only field is reserved and always has the value 0.
14 SMSZ	The bit 14 (Scratch Memory Size) determines if scratch memory must be available after every channel context. After reset, it is equal to 0, which defines a RAM space of 24 words for each channel. All of this area stores the channel context. By setting this bit, 32 words are reserved for every channel context, which gives eight additional words that can be used by the channel script to store any type of data. Those words are never erased by the context switching mechanism. The value of SMSZ cannot be changed if the LOCK bit in the SDMA_LOCK register is set. 0 24 words per context 1 32 words per context
CHN0ADDR	This 14-bit register is used by the boot code of the SDMA. After reset, it points to the standard boot routine in ROM (channel 0 routine). By changing this address, you can perform a boot sequence with your own routine. The very first instructions of the boot code fetch the contents of this register (it is also mapped in the SDMA memory space) and jump to the given address. The reset value is 0x0050 (decimal 80). The value of CHN0ADDR cannot be changed if the LOCK bit in the SDMA_LOCK register is set.

7.2.7.23 DMA Requests (SDMAARMx_EVT_MIRROR)

Address: Base address + 60h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	EVENTS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_EVT_MIRROR field descriptions

Field	Description
EVENTS	This register reflects the DMA requests received by the SDMA for events 31-0. The Arm platform and the SDMA have a read-only access. There is one bit associated with each of 32 DMA request events. This information may be useful during debug of the blocks that generate the DMA requests. The EVT_MIRROR register is cleared following read access. 0 DMA request event not pending 1 DMA request event pending

7.2.7.24 DMA Requests 2 (SDMAARMx_EVT_MIRROR2)

Address: Base address + 64h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																EVENTS[47:32]															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_EVT_MIRROR2 field descriptions

Field	Description
31–16 Reserved	This read-only field is reserved and always has the value 0.
EVENTS[47:32]	This register reflects the DMA requests received by the SDMA for events 47-32. The Arm platform and the SDMA have a read-only access. There is one bit associated with each of DMA request events. This information may be useful during debug of the blocks that generate the DMA requests. The EVT_MIRROR2 register is cleared following read access. 0 - DMA request event not pending 1- DMA request event pending

7.2.7.25 Cross-Trigger Events Configuration Register 1 (SDMAARMx_XTRIG_CONF1)

Address: Base address + 70h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0	CNF3	NUM3[5:0]							0	CNF2	NUM2[5:0]				
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	CNF1	NUM1[5:0]							0	CNF0	NUM0[5:0]				
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_XTRIG_CONF1 field descriptions

Field	Description
31 Reserved	This read-only field is reserved and always has the value 0.
30 CNF3	Configuration of the SDMA event line number i that is connected to the cross-trigger. It determines whether the event line pulse is generated by the reception of a DMA request or by the starting of a channel script execution. 0 channel 1 DMA request
29–24 NUM3[5:0]	Contains the number of the DMA request or channel that triggers the pulse on the cross-trigger event line number i .
23 Reserved	This read-only field is reserved and always has the value 0.
22 CNF2	Configuration of the SDMA event line number i that is connected to the cross-trigger. It determines whether the event line pulse is generated by receiving a DMA request or by starting a channel script execution. 0 channel 1 DMA request
21–16 NUM2[5:0]	Contains the number of the DMA request or channel that triggers the pulse on the cross-trigger event line number i .
15 Reserved	This read-only field is reserved and always has the value 0.
14 CNF1	Configuration of the SDMA event line number i that is connected to the cross-trigger. It determines whether the event line pulse is generated by receiving a DMA request or by starting a channel script execution.

Table continues on the next page...

SDMAARMx_XTRIG_CONF1 field descriptions (continued)

Field	Description
	0 channel 1 DMA request
13–8 NUM1[5:0]	Contains the number of the DMA request or channel that triggers the pulse on the cross-trigger event line number <i>i</i> .
7 Reserved	This read-only field is reserved and always has the value 0.
6 CNF0	Configuration of the SDMA event line number <i>i</i> that is connected to the cross-trigger. It determines whether the event line pulse is generated by receiving a DMA request or by starting a channel script execution. 0 channel 1 DMA request
NUM0[5:0]	Contains the number of the DMA request or channel that triggers the pulse on the cross-trigger event line number <i>i</i> .

7.2.7.26 Cross-Trigger Events Configuration Register 2 (SDMAARMx_XTRIG_CONF2)

Address: Base address + 74h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0								0							
W		CNF7								CNF6						
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0								0							
W		CNF5								CNF4						
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_XTRIG_CONF2 field descriptions

Field	Description
31 Reserved	This read-only field is reserved and always has the value 0.
30 CNF7	Configuration of the SDMA event line number <i>i</i> that is connected to the cross-trigger. It determines whether the event line pulse is generated by receiving a DMA request or by starting a channel script execution. 0 channel 1 DMA request

Table continues on the next page...

SDMAARMx_XTRIG_CONF2 field descriptions (continued)

Field	Description
29–24 NUM7[5:0]	Contains the number of the DMA request or channel that triggers the pulse on the cross-trigger event line number <i>i</i> .
23 Reserved	This read-only field is reserved and always has the value 0.
22 CNF6	Configuration of the SDMA event line number <i>i</i> that is connected to the cross-trigger. It determines whether the event line pulse is generated by receiving a DMA request or by starting a channel script execution. 0 channel 1 DMA request
21–16 NUM6[5:0]	Contains the number of the DMA request or channel that triggers the pulse on the cross-trigger event line number <i>i</i> .
15 Reserved	This read-only field is reserved and always has the value 0.
14 CNF5	Configuration of the SDMA event line number <i>i</i> that is connected to the cross-trigger. It determines whether the event line pulse is generated by receiving a DMA request or by starting a channel script execution. 0 channel 1 DMA request
13–8 NUM5[5:0]	Contains the number of the DMA request or channel that triggers the pulse on the cross-trigger event line number <i>i</i> .
7 Reserved	This read-only field is reserved and always has the value 0.
6 CNF4	Configuration of the SDMA event line number <i>i</i> that is connected to the cross-trigger. It determines whether the event line pulse is generated by receiving a DMA request or by starting a channel script execution. 0 channel 1 DMA request
NUM4[5:0]	Contains the number of the DMA request or channel that triggers the pulse on the cross-trigger event line number <i>i</i> .

7.2.7.27 Channel Priority Registers (SDMAARMx_SDMA_CHNPRI_n)

Address: Base address + 100h offset + (4d × i), where i=0d to 31d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																CHNPRI _n															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_SDMA_CHNPRIn field descriptions

Field	Description
31–3 Reserved	This read-only field is reserved and always has the value 0.
CHNPRIn	This contains the priority of channel number <i>n</i> . Useful values are between 1 and 7; 0 is reserved by the SDMA hardware to determine when there is no pending channel. Reset value is 0, which prevents the channels from starting.

7.2.7.28 Channel Enable RAM (SDMAARMx_CHNENBLn)

Address: Base address + 200h offset + (4d × i), where i=0d to 47d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMAARMx_CHNENBLn field descriptions

Field	Description
ENBLn	This 32-bit value selects the channels that are triggered by the DMA request number <i>n</i> . If ENBLn[i] is set to 1, bit EP[i] will be set when the DMA request <i>n</i> is received. These 48 32-bit registers are physically located in a RAM, with no known reset value. It is thus essential for the Arm platform to program them before any DMA request is triggered to the SDMA, otherwise an unpredictable combination of channels may be started.

7.2.7.29 SDMA DONE0 Configuration (SDMAARMx_DONE0_CONFIG)

Address: Base address + 1000h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R			0								0					
W																
Reset	0	0	0	1	1	1	1	1	0	0	0	1	1	1	1	1

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R			0								0					
W																
Reset	0	0	0	1	1	1	1	1	0	0	0	1	1	1	1	1

SDMAARMx_DONE0_CONFIG field descriptions

Field	Description
31 DONE_SEL3	Select Done from SW or HW for channel 3 0 HW 1 SW
30 SW_DONE_DIS3	Disable SW Done for channel 3 0 Enable 1 Disable
29 Reserved	This read-only field is reserved and always has the value 0.
28–24 CH_SEL3	Select event for channel 3 when Done is selected from HW. HW Done will be asserted when the negative edge of selected event's Event Pending (EP) is detected. 00000 - Event 0 00001 - Event 1 ... 11111 - Event 31
23 DONE_SEL2	Select Done from SW or HW for channel 2 0 HW 1 SW
22 SW_DONE_DIS2	Disable SW Done for channel 2 0 Enable 1 Disable
21 Reserved	This read-only field is reserved and always has the value 0.
20–16 CH_SEL2	Select event for channel 2 when Done is selected from HW. HW Done will be asserted when the negative edge of selected event's Event Pending (EP) is detected. 00000 - Event 0 00001 - Event 1 ... 11111 - Event 31
15 DONE_SEL1	Select Done from SW or HW for channel 1 0 HW 1 SW
14 SW_DONE_DIS1	Disable SW Done for channel 1 0 Enable 1 Disable
13 Reserved	This read-only field is reserved and always has the value 0.
12–8 CH_SEL1	Select event for channel 1 when Done is selected from HW. HW Done will be asserted when the negative edge of selected event's Event Pending (EP) is detected.

Table continues on the next page...

SDMAARMx_DONE0_CONFIG field descriptions (continued)

Field	Description
	00000 - Event 0 00001 - Event 1 ... 11111 - Event 31
7 DONE_SEL0	Select Done from SW or HW for channel 0 0 HW 1 SW
6 SW_DONE_DIS0	Disable SW Done for channel 0 0 Enable 1 Disable
5 Reserved	This read-only field is reserved and always has the value 0.
CH_SEL0	Select event for channel 0 when Done is selected from HW. HW Done will be asserted when the negative edge of selected event's Event Pending (EP) is detected. 00000 - Event 0 00001 - Event 1 ... 11111 - Event 31

7.2.7.30 SDMA DONE1 Configuration (SDMAARMx_DONE1_CONFIG)

Address: Base address + 1004h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DONE_SEL7	SW_DONE_DIS7	0	CH_SEL7					DONE_SEL6	SW_DONE_DIS6	0	CH_SEL6				
W																
Reset	0	0	0	1	1	1	1	1	0	0	0	1	1	1	1	1
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DONE_SEL5	SW_DONE_DIS5	0	CH_SEL5					DONE_SEL4	SW_DONE_DIS4	0	CH_SEL4				
W																
Reset	0	0	0	1	1	1	1	1	0	0	0	1	1	1	1	1

SDMAARMx_DONE1_CONFIG field descriptions

Field	Description
31 DONE_SEL7	Select Done from SW or HW for channel 7 0 HW 1 SW
30 SW_DONE_DIS7	Disable SW Done for channel 7 0 Enable 1 Disable
29 Reserved	This read-only field is reserved and always has the value 0.
28–24 CH_SEL7	Select event for channel 7 when Done is selected from HW. HW Done will be asserted when the negative edge of selected event's Event Pending (EP) is detected. 00000 - Event 0 00001 - Event 1 ... 11111 - Event 31
23 DONE_SEL6	Select Done from SW or HW for channel 6 0 HW 1 SW
22 SW_DONE_DIS6	Disable SW Done for channel 6 0 Enable 1 Disable
21 Reserved	This read-only field is reserved and always has the value 0.
20–16 CH_SEL6	Select event for channel 6 when Done is selected from HW. HW Done will be asserted when the negative edge of selected event's Event Pending (EP) is detected. 00000 - Event 0 00001 - Event 1 ... 11111 - Event 31
15 DONE_SEL5	Select Done from SW or HW for channel 5 0 HW 1 SW
14 SW_DONE_DIS5	Disable SW Done for channel 5 0 Enable 1 Disable
13 Reserved	This read-only field is reserved and always has the value 0.
12–8 CH_SEL5	Select event for channel 5 when Done is selected from HW. HW Done will be asserted when the negative edge of selected event's Event Pending (EP) is detected.

Table continues on the next page...

SDMAARMx_DONE1_CONFIG field descriptions (continued)

Field	Description
	00000 - Event 0 00001 - Event 1 ... 11111 - Event 31
7 DONE_SEL4	Select Done from SW or HW for channel 4 0 HW 1 SW
6 SW_DONE_DIS4	Disable SW Done for channel 4 0 Enable 1 Disable
5 Reserved	This read-only field is reserved and always has the value 0.
CH_SEL4	Select event for channel 4 when Done is selected from HW. HW Done will be asserted when the negative edge of selected event's Event Pending (EP) is detected. 00000 - Event 0 00001 - Event 1 ... 11111 - Event 31

7.2.8 BP Memory Map and Control Register Definitions

The following section describes SDMA control registers available to the BP.

NOTE

These registers are physically implemented in all platforms, but are not accessible when the SDMA BP control port is not connected. Reset values are calculated to allow the system to work when those registers cannot be accessed.

All registers are clocked with the SDMA clock (which means the SDMA clock must be running when the BP wants to access any register).

SDMABP memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
302C_0000	Channel 0 Pointer (SDMABP2_DC0PTR)	32	R/W	0000_0000h	7.2.8.1/1279
302C_0004	Channel Interrupts (SDMABP2_INTR)	32	w1c	0000_0000h	7.2.8.2/1280
302C_0008	Channel Stop/Channel Status (SDMABP2_STOP_STAT)	32	R/W	0000_0000h	7.2.8.3/1280
302C_000C	Channel Start (SDMABP2_DSTART)	32	R	0000_0000h	7.2.8.4/1280
302C_0028	DMA Request Error Register (SDMABP2_EVTERR)	32	R	0000_0000h	7.2.8.5/1281
302C_002C	Channel DSP Interrupt Mask (SDMABP2_INTRMASK)	32	R/W	0000_0000h	7.2.8.6/1281
302C_0034	DMA Request Error Register (SDMABP2_EVTERRDBG)	32	R	0000_0000h	7.2.8.7/1282
30BD_0000	Channel 0 Pointer (SDMABP1_DC0PTR)	32	R/W	0000_0000h	7.2.8.1/1279
30BD_0004	Channel Interrupts (SDMABP1_INTR)	32	w1c	0000_0000h	7.2.8.2/1280
30BD_0008	Channel Stop/Channel Status (SDMABP1_STOP_STAT)	32	R/W	0000_0000h	7.2.8.3/1280
30BD_000C	Channel Start (SDMABP1_DSTART)	32	R	0000_0000h	7.2.8.4/1280
30BD_0028	DMA Request Error Register (SDMABP1_EVTERR)	32	R	0000_0000h	7.2.8.5/1281
30BD_002C	Channel DSP Interrupt Mask (SDMABP1_INTRMASK)	32	R/W	0000_0000h	7.2.8.6/1281
30BD_0034	DMA Request Error Register (SDMABP1_EVTERRDBG)	32	R	0000_0000h	7.2.8.7/1282

7.2.8.1 Channel 0 Pointer (SDMABPx_DC0PTR)

Address: Base address + 0h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMABPx_DC0PTR field descriptions

Field	Description
DC0PTR	Channel 0 Pointer contains the 32-bit address, in BP memory, of the array of channel control blocks starting with the one for channel 0 (the control channel). This register should be initialized by the BP before it enables a channel (for example, channel 0). See the API document SDMA Scripts User Manual for the use of this register. The BP has a read/write access and the SDMA has a read-only access.

7.2.8.2 Channel Interrupts (SDMABPx_INTR)

Address: Base address + 4h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DI																															
W	w1c																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMABPx_INTR field descriptions

Field	Description
DI	The BP Interrupts register contains the 32 DI[i] bits. If any bit is set, it will cause an interrupt to the BP. - This register is a "write-ones" register to the BP. When the BP sets a bit in this register, the corresponding DI[i] bit is cleared. - The interrupt service routine should clear individual channel bits when their interrupts are serviced; failure to do so will cause continuous interrupts. - The SDMA is responsible for setting the DI[i] bit corresponding to the current channel when the corresponding done instruction is executed.

7.2.8.3 Channel Stop/Channel Status (SDMABPx_STOP_STAT)

Address: Base address + 8h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DE																															
W	w1c																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMABPx_STOP_STAT field descriptions

Field	Description
DE	This 32-bit register gives access to the BP (DSP) Enable bits, DE. There is one bit for every channel. - This register is a "write-ones" register to the BP. - When the BP writes 1 in bit <i>i</i> of this register, it clears the DE[i] and DSTART[i] bits. - Reading this register yields the current state of the DE[i] bits.

7.2.8.4 Channel Start (SDMABPx_DSTART)

Address: Base address + Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DSTART_DE																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMABPx_DSTART field descriptions

Field	Description
DSTART_DE	The DSTART_DE registers are 32 bits wide with one bit for every channel. - When a bit is written to 1, it enables the corresponding channel. - Two physical registers are accessed with that address (DSTART and DE), which enables the BP to trigger a channel a second time before the first trigger was processed. - This register is a "write-ones" register to the BP. Neither DSTART[i] bit can be set while the corresponding DE[i] bit is cleared. - When the BP tries to set the DSTART[i] bit by writing a one (if the corresponding DE[i] bit is clear), the bit in the DSTART[i] register will remain cleared and the DE[i] bit will be set. If the corresponding DE[i] bit was already set, the DSTART[i] bit will be set. - The next time the SDMA channel <i>i</i> attempts to clear the DE[i] bit by means of a done instruction, the bit in the DSTART[i] register will be cleared and the DE[i] bit will take the old value of the DSTART[i] bit. - Reading this register yields the current state of the DSTART[i] bits. This mechanism enables the BP to pipeline two DSTART commands per channel.

7.2.8.5 DMA Request Error Register (SDMABPx_EVTERR)

Address: Base address + 28h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CHNERR																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMABPx_EVTERR field descriptions

Field	Description
CHNERR	This register is used by the SDMA to warn the BP when an incoming DMA request was detected; it then triggers a channel that is already pending or being serviced, which may mean there is an overflow of data for that channel. An interrupt is sent to the BP if the corresponding channel bit is set in the INTRMASK register. - This is a "write-ones" register for the scheduler. It is only able to set the flags. The flags are cleared when the register is read by the BP or during an SDMA reset. - The CHNERR[i] bit is set when a DMA request that triggers channel <i>i</i> is received through the corresponding input pins and the EP[i] bit is already set. The EVTERR[i] bit is unaffected if the BP tries to set the EP[i] bit when that EP[i] bit is already set.

7.2.8.6 Channel DSP Interrupt Mask (SDMABPx_INTRMASK)

Address: Base address + 2Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DIMASK																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMABPx_INTRMASK field descriptions

Field	Description
DIMASK	The Interrupt Mask Register contains 32 interrupt generation mask bits. If bit DIMASK[i] is set, the DI[i] bit is set and an interrupt is sent to the BP when a DMA request error is detected on channel <i>i</i> (for example, EVTERR[i] is set).

7.2.8.7 DMA Request Error Register (SDMABPx_EVTERRDBG)

Address: Base address + 34h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CHNERR																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMABPx_EVTERRDBG field descriptions

Field	Description
CHNERR	This register is the same as EVTERR except reading it does not clear its contents. This address is meant to be used in debug mode. The BP OnCE may check this register value without modifying it.

7.2.9 SDMA Internal (Core) Memory Map and Internal Register Definitions

The actual SDMA memory mapped registers are summarized in the following sections; for peripherals' memory maps, refer to the respective chapters.

The following definitions serve as a key for the SDMA internal register summary.

SDMACORE memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
302C_0000	Arm platform Channel 0 Pointer (SDMACORE2_MC0PTR)	32	R	0000_0000h	7.2.9.1/1284
302C_0008	Current Channel Pointer (SDMACORE2_CCPtr)	32	R	0000_0000h	7.2.9.2/1285
302C_000C	Current Channel Register (SDMACORE2_CCR)	32	R	0000_0000h	7.2.9.3/1285
302C_0010	Highest Pending Channel Register (SDMACORE2_NCR)	32	R	0000_0000h	7.2.9.4/1286

Table continues on the next page...

SDMACORE memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
302C_0014	External DMA Requests Mirror (SDMACORE2_EVENTS)	32	R	0000_0000h	7.2.9.5/1287
302C_0018	Current Channel Priority (SDMACORE2_CCPRI)	32	R	0000_0000h	7.2.9.6/1288
302C_001C	Next Channel Priority (SDMACORE2_NCPRI)	32	R	0000_0000h	7.2.9.7/1288
302C_0020	OnCE Event Cell Counter (SDMACORE2_ECOUNTER)	32	R/W	0000_0000h	7.2.9.8/1289
302C_0024	OnCE Event Cell Control Register (SDMACORE2_ECTL)	32	R/W	0000_0000h	7.2.9.9/1289
302C_0028	OnCE Event Address Register A (SDMACORE2_EAA)	32	R/W	0000_0000h	7.2.9.10/1291
302C_002C	OnCE Event Cell Address Register B (SDMACORE2_EAB)	32	R/W	0000_0000h	7.2.9.11/1291
302C_0030	OnCE Event Cell Address Mask (SDMACORE2_EAM)	32	R/W	0000_0000h	7.2.9.12/1291
302C_0034	OnCE Event Cell Data Register (SDMACORE2_ED)	32	R/W	0000_0000h	7.2.9.13/1292
302C_0038	OnCE Event Cell Data Mask (SDMACORE2_EDM)	32	R/W	0000_0000h	7.2.9.14/1292
302C_003C	OnCE Real-Time Buffer (SDMACORE2_RTBB)	32	R/W	0000_0000h	7.2.9.15/1293
302C_0040	OnCE Trace Buffer (SDMACORE2_TB)	32	R	0000_0000h	7.2.9.16/1293
302C_0044	OnCE Status (SDMACORE2_OSTAT)	32	R	0000_0000h	7.2.9.17/1294
302C_0048	Channel 0 Boot Address (SDMACORE2_MCHN0ADDR)	32	R	0000_0000h	7.2.9.18/1296
302C_004C	ENDIAN Status Register (SDMACORE2_ENDIANNES)	32	R	0000_0001h	7.2.9.19/1297
302C_0054	Lock Status Register (SDMACORE2_SDMA_LOCK)	32	R	0000_0000h	7.2.9.20/1298
302C_0058	External DMA Requests Mirror #2 (SDMACORE2_EVENTS2)	32	R	0000_0000h	7.2.9.21/1298
30BD_0000	Arm platform Channel 0 Pointer (SDMACORE1_MC0PTR)	32	R	0000_0000h	7.2.9.1/1284
30BD_0008	Current Channel Pointer (SDMACORE1_CCPTR)	32	R	0000_0000h	7.2.9.2/1285
30BD_000C	Current Channel Register (SDMACORE1_CCR)	32	R	0000_0000h	7.2.9.3/1285
30BD_0010	Highest Pending Channel Register (SDMACORE1_NCR)	32	R	0000_0000h	7.2.9.4/1286
30BD_0014	External DMA Requests Mirror (SDMACORE1_EVENTS)	32	R	0000_0000h	7.2.9.5/1287

Table continues on the next page...

SDMACORE memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
30BD_0018	Current Channel Priority (SDMACORE1_CCPRI)	32	R	0000_0000h	7.2.9.6/1288
30BD_001C	Next Channel Priority (SDMACORE1_NCPRI)	32	R	0000_0000h	7.2.9.7/1288
30BD_0020	OnCE Event Cell Counter (SDMACORE1_ECOUNT)	32	R/W	0000_0000h	7.2.9.8/1289
30BD_0024	OnCE Event Cell Control Register (SDMACORE1_ECTL)	32	R/W	0000_0000h	7.2.9.9/1289
30BD_0028	OnCE Event Address Register A (SDMACORE1_EAA)	32	R/W	0000_0000h	7.2.9.10/1291
30BD_002C	OnCE Event Cell Address Register B (SDMACORE1_EAB)	32	R/W	0000_0000h	7.2.9.11/1291
30BD_0030	OnCE Event Cell Address Mask (SDMACORE1_EAM)	32	R/W	0000_0000h	7.2.9.12/1291
30BD_0034	OnCE Event Cell Data Register (SDMACORE1_ED)	32	R/W	0000_0000h	7.2.9.13/1292
30BD_0038	OnCE Event Cell Data Mask (SDMACORE1_EDM)	32	R/W	0000_0000h	7.2.9.14/1292
30BD_003C	OnCE Real-Time Buffer (SDMACORE1_RTB)	32	R/W	0000_0000h	7.2.9.15/1293
30BD_0040	OnCE Trace Buffer (SDMACORE1_TB)	32	R	0000_0000h	7.2.9.16/1293
30BD_0044	OnCE Status (SDMACORE1_OSTAT)	32	R	0000_0000h	7.2.9.17/1294
30BD_0048	Channel 0 Boot Address (SDMACORE1_MCHN0ADDR)	32	R	0000_0000h	7.2.9.18/1296
30BD_004C	ENDIAN Status Register (SDMACORE1_ENDIANNES)	32	R	0000_0001h	7.2.9.19/1297
30BD_0054	Lock Status Register (SDMACORE1_SDMA_LOCK)	32	R	0000_0000h	7.2.9.20/1298
30BD_0058	External DMA Requests Mirror #2 (SDMACORE1_EVENTS2)	32	R	0000_0000h	7.2.9.21/1298

7.2.9.1 Arm platform Channel 0 Pointer (SDMACOREx_MC0PTR)

Address: Base address + 0h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	MC0PTR																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_MC0PTR field descriptions

Field	Description
MC0PTR	Contains the address-in the Arm platform memory space-of the initial SDMA context and scripts that are loaded by the SDMA boot script running on channel 0.

7.2.9.2 Current Channel Pointer (SDMACOREx_CCPtr)

Address: Base address + 8h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																CCPTR															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_CCPtr field descriptions

Field	Description
31–16 Reserved	This read-only field is reserved and always has the value 0.
CCPTR	Contains the start address of the context data for the current channel: Its value is <i>CONTEXT_BASE</i> + 24* <i>CCR</i> or <i>CONTEXT_BASE</i> + 32* <i>CCR</i> where <i>CONTEXT_BASE</i> = 0x0800. The value 24 or 32 is selected according to the programmed channel scratch RAM size in the register shown in Channel 0 Boot Address (SDMAARM_CHN0ADDR) .

7.2.9.3 Current Channel Register (SDMACOREx_CCR)

Address: Base address + Ch offset

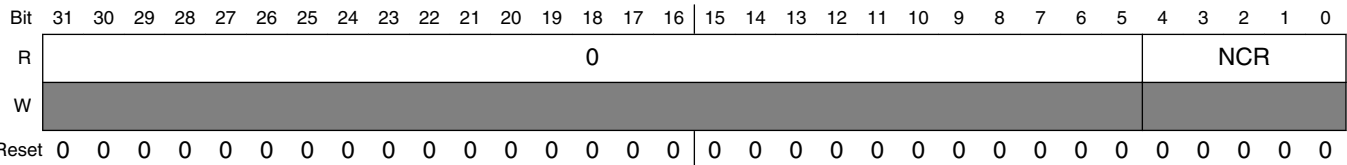
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																CCR															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_CCR field descriptions

Field	Description
31–5 Reserved	This read-only field is reserved and always has the value 0.
CCR	Contains the number of the current running channel whose context is installed. In the case that the SDMA has finished running the channel and has entered sleep state, CCR will indicate the previous running channel. The PST bits in the OSTAT register indicate when the SDMA is in sleep state.

7.2.9.4 Highest Pending Channel Register (SDMACOREx_NCR)

Address: Base address + 10h offset



SDMACOREx_NCR field descriptions

Field	Description
31–5 Reserved	This read-only field is reserved and always has the value 0.
NCR	Contains the number of the pending channel that the scheduler has selected to run next.

7.2.9.5 External DMA Requests Mirror (SDMACOREx_EVENTS)

NOTE

This register is very useful in the case of DMA requests that are active when a peripheral FIFO level is above the programmed watermark. The activation of the DMA request (rising edge) is detected by the SDMA logic and it can enable one or several channels. One of the channels accesses the peripheral and reads or writes a number of data that matches the watermark level (for example, if the watermark is four words, the channel reads or writes four words).

If the channel is effectively executed long after the DMA request was received, reading or writing the watermark number of data may not be sufficient to reset the DMA request (for example, if the FIFO watermark is four and at the channel execution it already contains nine pieces of data). This means no new rising edge may be detected by the SDMA, although there still remains transfers to perform. Therefore, if the channel were terminated at that time, it would not be restarted, causing potential overrun or underrun of the peripheral.

The proposed mechanism is for the channel to check this register after it has performed the "watermark" number of accesses to the peripheral. If the bit for the DMA request that triggers this channel is set, it means there is still another watermark number of data to transfer. This goes on until the bit is cleared. The same script can be used for multiple channels that require this behavior. The script can determine its channel number from the CCR register and infer the corresponding DMA request bit to check. It needs a reference table that is coherent with the request-channel matrix that the Arm platform programmed.

Address: Base address + 14h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	EVENTS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_EVENTS field descriptions

Field	Description
EVENTS	Reflects the status of the SDMA's external DMA requests. It is meant to allow any channel to monitor the states of these SDMA inputs. This register displays EVENTS 0-31. The EVENTS2 register displays events 32-47.

7.2.9.6 Current Channel Priority (SDMACOREx_CCPRI)

Address: Base address + 18h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																CCPRI															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SDMACOREx_CCPRI field descriptions

Field	Description
31–3 Reserved	This read-only field is reserved and always has the value 0.
CCPRI	Contains the 3-bit priority of the channel whose context is installed. It is 0 when no channel is running. NOTE: 1-7 current channel priority 0 no running channel

7.2.9.7 Next Channel Priority (SDMACOREx_NCPRI)

Address: Base address + 1Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																NCPRI															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SDMACOREx_NCPRI field descriptions

Field	Description
31–3 Reserved	This read-only field is reserved and always has the value 0.
NCPRI	Contains the 3-bit priority of the channel the scheduler has selected to run next. It is 0 when no other channel is pending.

7.2.9.8 OnCE Event Cell Counter (SDMACOREx_ECOUNTER)

Address: Base address + 20h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																ECOUNT															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_ECOUNTER field descriptions

Field	Description
31–16 Reserved	This read-only field is reserved and always has the value 0.
ECOUNT	The event cell counter contains the number of times minus one that an event detection must occur before generating a debug request. - This register should be written before any attempt to use the event detection counter during an event detection process. - The counter is cleared on a JTAG reset.

7.2.9.9 OnCE Event Cell Control Register (SDMACOREx_ECTL)

Address: Base address + 24h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0		EN	CNT	ECTC[1:0]	DTC[1:0]	ATC[1:0]	ABTC[1:0]	AATC[1:0]	ATS[1:0]						
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_ECTL field descriptions

Field	Description
31–14 Reserved	This read-only field is reserved and always has the value 0.
13 EN	Event Cell Enable. If the EN bit is set, the event cell is allowed to generate debug requests (the cell is awakened). If it is cleared, the event detection unit is disabled and no hardware breakpoint is generated, but matching conditions are still reflected on the emulation pin. 0 Cell is disabled. 1 Cell is enabled.
12 CNT	Event Counter Enable. The event counter enable bit determines if the cell counter is used during the event detection. In order to use the event counter during an event detection process, the event cell counter register should be loaded with a value equal to the number of times minus one that an event occurs before a debug request is sent. After every event detection, the counter is decreased. When the counter reaches

Table continues on the next page...

SDMACOREx_ECTL field descriptions (continued)

Field	Description
	the value 0, the event detection cell sends a debug request to the core. The event counter register should be written and the EN bit should be set before each new event detection process uses the event counter. 0 Counter is disabled. 1 Counter is enabled.
11–10 ECTC[1:0]	The event cell trigger condition bits select the combination of address and data matching conditions that generate the final address/data condition. During program execution, if this event cell trigger condition goes to 1, a debug request is sent to the SDMA. The EN bit must be set to enable the debug request generation. 00 address ONLY 01 data ONLY 10 address AND data 11 address OR data
9–8 DTC[1:0]	The data trigger condition bits define when data is considered matching after comparison with the data register of the event detection unit. The operations are performed on unsigned values. 00 equal 01 not equal 10 greater than 11 less than
7–6 ATC[1:0]	The address trigger condition bits select how the two address conditions (addressA and addressB) are combined to define the global address matching condition. The supported combinations are described, as follows. 00 addressA ONLY 01 addrA AND addrB 10 addrA OR addrB 11 reserved
5–4 ABTC[1:0]	The Address B Trigger Condition (ABTC) controls the operations performed by address comparator B. All operations are performed on unsigned values. This comparator B outputs the addressB condition. 00 equal 01 not equal 10 greater than 11 less than
3–2 AATC[1:0]	The Address A Trigger Condition (AATC) controls the operations performed by address comparator A. All operations are performed on unsigned values. This comparator A outputs the addressA condition. 00 equal 01 not equal 10 greater than 11 less than
ATS[1:0]	The access type select bits define the memory access type required on the SDMA memory bus. 00 read ONLY 01 write ONLY 10 read or write 11 -

7.2.9.10 OnCE Event Address Register A (SDMACOREx_EAA)

Address: Base address + 28h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																EAA															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_EAA field descriptions

Field	Description
31–16 Reserved	This read-only field is reserved and always has the value 0.
EAA	Event Cell Address Register A computes an address A condition. It is cleared on a JTAG reset.

7.2.9.11 OnCE Event Cell Address Register B (SDMACOREx_EAB)

Address: Base address + 2Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																EAB															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_EAB field descriptions

Field	Description
31–16 Reserved	This read-only field is reserved and always has the value 0.
EAB	Event Cell Address Register B computes an address B condition. It is cleared on a JTAG reset.

7.2.9.12 OnCE Event Cell Address Mask (SDMACOREx_EAM)

Address: Base address + 30h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																EAM															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_EAM field descriptions

Field	Description
31–16 Reserved	This read-only field is reserved and always has the value 0.

Table continues on the next page...

SDMACOREx_EAM field descriptions (continued)

Field	Description
EAM	The Event Cell Address Mask contains a user-defined address mask value. This mask is applied to the address value latched from the memory address bus before performing the address comparison. NOTE: There is a common address mask value for both address comparators. If bit <i>i</i> of this register is set, then bit <i>i</i> of the address value latched from the memory bus does not influence the result of the address comparison. The register is cleared on a JTAG reset.

7.2.9.13 OnCE Event Cell Data Register (SDMACOREx_ED)

Address: Base address + 34h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SDMACOREx_ED field descriptions

Field	Description
ED	The event cell data register contains a user defined data value. This data value is an input for the data comparator which generates the data condition. It is cleared on a JTAG reset.

7.2.9.14 OnCE Event Cell Data Mask (SDMACOREx_EDM)

Address: Base address + 38h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	EDM																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

SDMACOREx_EDM field descriptions

Field	Description
EDM	The event cell data mask register contains the user-defined data mask value. - This mask is applied to the data value latched from the memory bus before performing the data comparison. - Setting bit <i>i</i> of the event cell data mask register means that bit <i>i</i> of the data value latched from the address bus does not influence the result of the data comparison. - The data mask is cleared on a JTAG reset.

7.2.9.15 OnCE Real-Time Buffer (SDMACOREx_RTb)

Address: Base address + 3Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	RTB																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_RTb field descriptions

Field	Description
RTB	The Real Time Buffer register stores and retrieves run time information without putting the SDMA in debug mode. Writing to that register triggers a pulse on a specific real-time debug pin whose connection depends on the chip implementation. The RTB value can be accessed by the OnCE under Arm platform or JTAG control using the rbuffer command.

7.2.9.16 OnCE Trace Buffer (SDMACOREx_TB)

Address: Base address + 40h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0			TBF	TADDR											
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	TADDR		CHFADDR													
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_TB field descriptions

Field	Description
31–29 Reserved	This read-only field is reserved and always has the value 0.
28 TBF	The Trace Buffer Flag is set when the buffer contains the addresses of a valid change of flow. The contents of the buffer should be ignored otherwise. 0 Invalid information 1 Valid information
27–14 TADDR	The target address is the address taken after the execution of the change of flow instruction.
CHFADDR	The change of flow address is the address where the change of flow is taken when executing a change of flow instruction.

7.2.9.17 OnCE Status (SDMACOREx_OSTAT)

Address: Base address + 44h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PST[3:0]				RCV	EDR	ODR	SWB	MST	0				ECDR[2:0]		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_OSTAT field descriptions

Field	Description																												
31–16 Reserved	This read-only field is reserved and always has the value 0.																												
15–12 PST[3:0]	The Processor Status bits reflect the state of the SDMA RISC engine. - The "Program" state is the usual instruction execution cycle. - The "Data" state is inserted when there are wait-states during a load or a store on the data bus (ld or st). - The "Change of Flow" state is the second cycle of any instruction that breaks the sequence of instructions (jumps and channel-switching instructions). - The "Change of Flow in Loop" state is used when an error causes a hardware loop exit. - The "Debug" state means the SDMA is in debug mode. - The "Functional Unit" state is inserted when there are wait-states during a load or a store on the functional units bus (ldf or stf). - In "Sleep" modes, no script is running (this is the RISC engine idle state). The "after Reset" is slightly different because no context restoring phase will happen when a channel is triggered: The script located at address 0 will be executed (boot operation). - The "in Sleep" states are the same as above except they do not have any corresponding channel. They are used when entering debug mode after reset; the reason is that it is necessary to return to the "Sleep after Reset" state when leaving debug mode. <table> <tr><td>0</td><td>Program</td></tr> <tr><td>1</td><td>Data</td></tr> <tr><td>2</td><td>Change of Flow</td></tr> <tr><td>3</td><td>Change of Flow in Loop</td></tr> <tr><td>4</td><td>Debug</td></tr> <tr><td>5</td><td>Functional Unit</td></tr> <tr><td>6</td><td>Sleep</td></tr> <tr><td>7</td><td>Save</td></tr> <tr><td>8</td><td>Program in Sleep</td></tr> <tr><td>9</td><td>Data in Sleep</td></tr> <tr><td>10</td><td>Change of Flow in Sleep</td></tr> <tr><td>11</td><td>Change Flow Loop Sleep</td></tr> <tr><td>12</td><td>Debug in Sleep</td></tr> <tr><td>13</td><td>Functional Unit in Sleep</td></tr> </table>	0	Program	1	Data	2	Change of Flow	3	Change of Flow in Loop	4	Debug	5	Functional Unit	6	Sleep	7	Save	8	Program in Sleep	9	Data in Sleep	10	Change of Flow in Sleep	11	Change Flow Loop Sleep	12	Debug in Sleep	13	Functional Unit in Sleep
0	Program																												
1	Data																												
2	Change of Flow																												
3	Change of Flow in Loop																												
4	Debug																												
5	Functional Unit																												
6	Sleep																												
7	Save																												
8	Program in Sleep																												
9	Data in Sleep																												
10	Change of Flow in Sleep																												
11	Change Flow Loop Sleep																												
12	Debug in Sleep																												
13	Functional Unit in Sleep																												

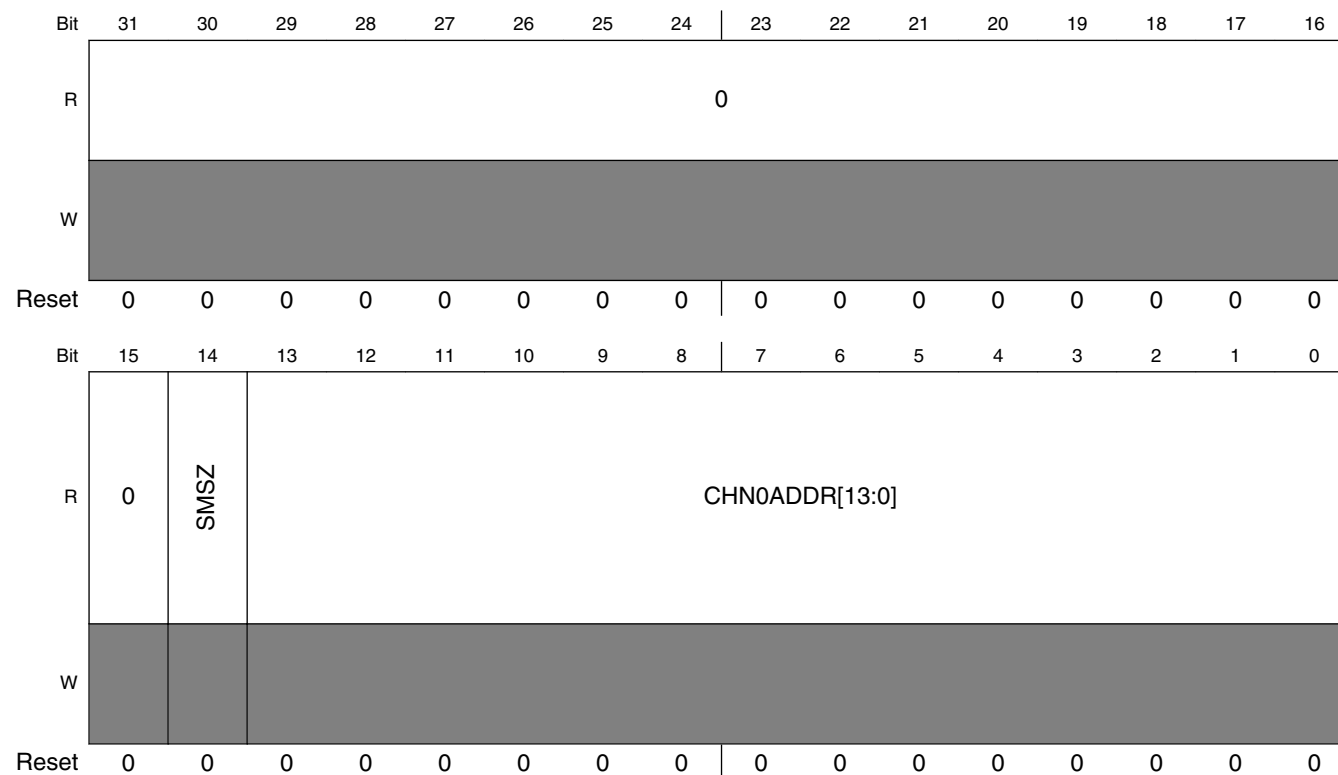
Table continues on the next page...

SDMACOREx_OSTAT field descriptions (continued)

Field	Description
	14 Sleep after Reset 15 Restore
11 RCV	After each write access to the real time buffer (RTB), the RCV bit is set. This bit is cleared after execution of an <code>rbuffer</code> command and on a JTAG reset.
10 EDR	This flag is raised when the SDMA has entered debug mode after an external debug request.
9 ODR	This flag is raised when the SDMA has entered debug mode after a OnCE debug request.
8 SWB	This flag is raised when the SDMA has entered debug mode after a software breakpoint.
7 MST	This flag is raised when the OnCE is controlled from the Arm platform peripheral interface. 0 JTAG interface controls the OnCE. 1 Arm platform peripheral interface controls the OnCE.
6–3 Reserved	This read-only field is reserved and always has the value 0.
ECDR[2:0]	Event Cell Debug Request. If the debug request comes from the event cell, the reason for entering debug mode is given by the EDR bits. The encoding of the EDR bits is useful to find out more precisely why the debug request was generated. A debug request from an event cell is generated for a specific combination of the addressA, addressB, and data conditions; the value of those fields is given by the EDR bits. If all three bits of the EDR are reset, then it did not generate any debug request. If the cell did generate a debug request, then at least one EDR bit is set; the meaning of the encoding is as follows: 0 1 matched addressA condition 1 1 matched addressB condition 2 1 matched data condition

7.2.9.18 Channel 0 Boot Address (SDMACOREx_MCHN0ADDR)

Address: Base address + 48h offset



SDMACOREx_MCHN0ADDR field descriptions

Field	Description
31–15 Reserved	This read-only field is reserved and always has the value 0.
14 SMSZ	The bit 14 (Scratch Memory Size) determines if scratch memory must be available after every channel context. After reset, it is equal to 0, which defines a RAM space of 24 words for each channel. All of this area stores the channel context. By setting this bit, 32 words are reserved for every channel context, which gives eight additional words that can be used by the channel script to store any type of data. Those words are never erased by the context switching mechanism. 0 24 words per context 1 32 words per context
CHN0ADDR[13:0]	Contains the address of the channel 0 routine programmed by the Arm platform; it is loaded into a general register at the very start of the boot and the SDMA jumps to the address it contains. By default, it points to the standard boot routine in ROM.

7.2.9.19 ENDIAN Status Register (SDMACOREx_ENDIANNES)

Address: Base address + 4Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0												0		APEND	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

SDMACOREx_ENDIANNES field descriptions

Field	Description
31–3 Reserved	This read-only field is reserved and always has the value 0.
2–1 Reserved	This read-only field is reserved and always has the value 0.
0 APEND	APEND indicates the endian mode of the Peripheral and Burst DMA interfaces. This bit is tied to logic '1' indicating little-endian mode. 0 - Arm platform is in big-endian mode 1 - Arm platform is in little-endian mode

7.2.9.20 Lock Status Register (SDMACOREx_SDMA_LOCK)

Address: Base address + 54h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0															LOCK
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_SDMA_LOCK field descriptions

Field	Description
31–1 Reserved	This read-only field is reserved and always has the value 0.
0 LOCK	The LOCK bit reports the value of the LOCK bit in the SDMA_LOCK status register. SDMA software may use this value to determine if certain operations such as loading of new scripts is allowed. 0 - LOCK bit clear 1 - LOCK bit set

7.2.9.21 External DMA Requests Mirror #2 (SDMACOREx_EVENTS2)

Address: Base address + 58h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0																EVENTS															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SDMACOREx_EVENTS2 field descriptions

Field	Description
31–16 Reserved	This read-only field is reserved and always has the value 0.
EVENTS	Reflects the status of the SDMA's external DMA requests. It is meant to allow any channel to monitor the states of these SDMA inputs. This register displays EVENTS 32-47. The separate EVENTS register displays events 0-31.

7.2.10 SDMA Peripheral Registers

Refer to the respective peripherals' chapters for more information.

Chapter 8

Chip IO and Pinmux

8.1 External Signals and Pin Multiplexing

8.1.1 Overview

The chip contains a limited number of pins, most of which have multiple signal options. These signal-to-pin and pin-to-signal options are selected by the input-output multiplexer called IOMUX. The IOMUX is also used to configure other pin characteristics, such as voltage level, drive strength, and hysteresis.

The muxing options table lists the external signals grouped by the module instance, the muxing options for each signal, and the registers used to route the signal to the chosen pad.

8.1.1.1 Muxing Options

Instance	Port	Pad	Mode
ARM_PLATFORM	ARM_PLATFORM_EVENTI	SAI1_TXC	ALT4
	ARM_PLATFORM_EVENTO	SAI1_TXFS	ALT4
	ARM_PLATFORM_TRACE00	SAI1_RXD0	ALT4
	ARM_PLATFORM_TRACE01	SAI1_RXD1	ALT4
	ARM_PLATFORM_TRACE10	SAI1_TXD2	ALT4
	ARM_PLATFORM_TRACE11	SAI1_TXD3	ALT4
	ARM_PLATFORM_TRACE12	SAI1_TXD4	ALT4
	ARM_PLATFORM_TRACE13	SAI1_TXD5	ALT4
	ARM_PLATFORM_TRACE14	SAI1_TXD6	ALT4
	ARM_PLATFORM_TRACE15	SAI1_TXD7	ALT4
	ARM_PLATFORM_TRACE02	SAI1_RXD2	ALT4
	ARM_PLATFORM_TRACE03	SAI1_RXD3	ALT4

Table continues on the next page...

External Signals and Pin Multiplexing

Instance	Port	Pad	Mode
	ARM_PLATFORM_TRACE04	SAI1_RXD4	ALT4
	ARM_PLATFORM_TRACE05	SAI1_RXD5	ALT4
	ARM_PLATFORM_TRACE06	SAI1_RXD6	ALT4
	ARM_PLATFORM_TRACE07	SAI1_RXD7	ALT4
	ARM_PLATFORM_TRACE08	SAI1_TXD0	ALT4
	ARM_PLATFORM_TRACE09	SAI1_TXD1	ALT4
	ARM_PLATFORM_TRACE_CLK	SAI1_RXFS	ALT4
	ARM_PLATFORM_TRACE_CTL	SAI1_RXC	ALT4
ANAMIX	REF_CLK_25M	GPIO1_IO01	ALT5
	REF_CLK_32K	GPIO1_IO00	ALT5
CCM	CCM_CLKO1	GPIO1_IO14	ALT6
	CCM_CLKO2	GPIO1_IO15	ALT6
	CCM_ENET_PHY_REF_CLK_ROOT	GPIO1_IO00	ALT1
	CCM_EXT_CLK1	GPIO1_IO00	ALT6
	CCM_EXT_CLK2	GPIO1_IO01	ALT6
	CCM_EXT_CLK3	GPIO1_IO06	ALT6
	CCM_EXT_CLK4	GPIO1_IO07	ALT6
	CCM_PMIC_READY	GPIO1_IO05	ALT5
	CCM_PMIC_READY	GPIO1_IO11	ALT5
	CCM_PMIC_STBY_REQ	PMIC_STBY_REQ	ALT0
	CCM_ENET_REF_CLK_ROOT (output)	ENET_TD2	ALT1
	CLK1_N	CLK1_N	No Muxing
	CLK1_P	CLK1_P	No Muxing
	CLK2_N	CLK2_N	No Muxing
	CLK2_P	CLK2_P	No Muxing
CJTAG	CJTAG_WRAPPER_MODE	JTAG_MOD	ALT0
	CJTAG_WRAPPER_TCK	JTAG_TCK	ALT0
	CJTAG_WRAPPER_TDI	JTAG_TDI	ALT0
	CJTAG_WRAPPER_TDO	JTAG_TDO	ALT0
	CJTAG_WRAPPER_TMS	JTAG_TMS	ALT0
	CJTAG_WRAPPER_TRST_B	JTAG_TRST_B	ALT0
DDR	DRAM_AC00	DRAM_AC00	No Muxing
	DRAM_AC01	DRAM_AC01	No Muxing
	DRAM_AC02	DRAM_AC02	No Muxing
	DRAM_AC03	DRAM_AC03	No Muxing
	DRAM_AC04	DRAM_AC04	No Muxing
	DRAM_AC05	DRAM_AC05	No Muxing
	DRAM_AC06	DRAM_AC06	No Muxing

Table continues on the next page...

Instance	Port	Pad	Mode
	DRAM_AC07p	DRAM_AC07	No Muxing
	DRAM_AC08	DRAM_AC08	No Muxing
	DRAM_AC09	DRAM_AC09	No Muxing
	DRAM_AC10	DRAM_AC10	No Muxing
	DRAM_AC11	DRAM_AC11	No Muxing
	DRAM_AC12	DRAM_AC12	No Muxing
	DRAM_AC13	DRAM_AC13	No Muxing
	DRAM_AC14	DRAM_AC14	No Muxing
	DRAM_AC15	DRAM_AC15	No Muxing
	DRAM_AC16	DRAM_AC16	No Muxing
	DRAM_AC17	DRAM_AC17	No Muxing
	DRAM_AC19	DRAM_AC19	No Muxing
	DRAM_AC20	DRAM_AC20	No Muxing
	DRAM_AC21	DRAM_AC21	No Muxing
	DRAM_AC22	DRAM_AC22	No Muxing
	DRAM_AC23	DRAM_AC23	No Muxing
	DRAM_AC24	DRAM_AC24	No Muxing
	DRAM_AC25	DRAM_AC25	No Muxing
	DRAM_AC26	DRAM_AC26	No Muxing
	DRAM_AC27	DRAM_AC27	No Muxing
	DRAM_AC28	DRAM_AC28	No Muxing
	DRAM_AC29	DRAM_AC29	No Muxing
	DRAM_AC30	DRAM_AC30	No Muxing
	DRAM_AC31	DRAM_AC31	No Muxing
	DRAM_AC32	DRAM_AC32	No Muxing
	DRAM_AC33	DRAM_AC33	No Muxing
	DRAM_AC34	DRAM_AC34	No Muxing
	DRAM_AC35	DRAM_AC35	No Muxing
	DRAM_AC36	DRAM_AC36	No Muxing
	DRAM_AC37	DRAM_AC37	No Muxing
	DRAM_AC38	DRAM_AC38	No Muxing
	DRAM_ALERT_N	DRAM_ALERT_N	No Muxing
	DRAM_DM0	DRAM_DM0	No Muxing
	DRAM_DM1	DRAM_DM1	No Muxing
	DRAM_DM2	DRAM_DM2	No Muxing
	DRAM_DM3	DRAM_DM3	No Muxing
	DRAM_DQ00	DRAM_DQ00	No Muxing
	DRAM_DQ01	DRAM_DQ01	No Muxing
	DRAM_DQ02	DRAM_DQ02	No Muxing
	DRAM_DQ03	DRAM_DQ03	No Muxing
	DRAM_DQ04	DRAM_DQ04	No Muxing

Table continues on the next page...

External Signals and Pin Multiplexing

Instance	Port	Pad	Mode
	DRAM_DQ05	DRAM_DQ05	No Muxing
	DRAM_DQ06	DRAM_DQ06	No Muxing
	DRAM_DQ07	DRAM_DQ07	No Muxing
	DRAM_DQ08	DRAM_DQ08	No Muxing
	DRAM_DQ09	DRAM_DQ09	No Muxing
	DRAM_DQ10	DRAM_DQ10	No Muxing
	DRAM_DQ11	DRAM_DQ11	No Muxing
	DRAM_DQ12	DRAM_DQ12	No Muxing
	DRAM_DQ13	DRAM_DQ13	No Muxing
	DRAM_DQ14	DRAM_DQ14	No Muxing
	DRAM_DQ15	DRAM_DQ15	No Muxing
	DRAM_DQ16	DRAM_DQ16	No Muxing
	DRAM_DQ17	DRAM_DQ17	No Muxing
	DRAM_DQ18	DRAM_DQ18	No Muxing
	DRAM_DQ19	DRAM_DQ19	No Muxing
	DRAM_DQ20	DRAM_DQ20	No Muxing
	DRAM_DQ21	DRAM_DQ21	No Muxing
	DRAM_DQ22	DRAM_DQ22	No Muxing
	DRAM_DQ23	DRAM_DQ23	No Muxing
	DRAM_DQ24	DRAM_DQ24	No Muxing
	DRAM_DQ25	DRAM_DQ25	No Muxing
	DRAM_DQ26	DRAM_DQ26	No Muxing
	DRAM_DQ27	DRAM_DQ27	No Muxing
	DRAM_DQ28	DRAM_DQ28	No Muxing
	DRAM_DQ29	DRAM_DQ29	No Muxing
	DRAM_DQ30	DRAM_DQ30	No Muxing
	DRAM_DQ31	DRAM_DQ31	No Muxing
	DRAM_DQS0_N	DRAM_DQS0_N	No Muxing
	DRAM_DQS0_P	DRAM_DQS0_P	No Muxing
	DRAM_DQS1_N	DRAM_DQS1_N	No Muxing
	DRAM_DQS1_P	DRAM_DQS1_P	No Muxing
	DRAM_DQS2_N	DRAM_DQS2_N	No Muxing
	DRAM_DQS2_P	DRAM_DQS2_P	No Muxing
	DRAM_DQS3_N	DRAM_DQS3_N	No Muxing
	DRAM_DQS3_P	DRAM_DQS3_P	No Muxing
	DRAM_RESET_N	DRAM_RESET_N	No Muxing
	DRAM_VREF	DRAM_VREF	No Muxing
	DRAM_ZN	DRAM_ZN	No Muxing
ECSPI1	ECSPI1_MISO	ECSPI1_MISO	ALT0
	ECSPI1_MOSI	ECSPI1_MOSI	ALT0
	ECSPI1_SCLK	ECSPI1_SCLK	ALT0

Table continues on the next page...

Instance	Port	Pad	Mode
ECSPI2	ECSPI1_SS0	ECSPI1_SS0	ALT0
	ECSPI2_MISO	ECSPI2_MISO	ALT0
	ECSPI2_MOSI	ECSPI2_MOSI	ALT0
	ECSPI2_SCLK	ECSPI2_SCLK	ALT0
ECSPI3	ECSPI2_SS0	ECSPI2_SS0	ALT0
	ECSPI3_MISO	UART2_RXD	ALT1
	ECSPI3_MOSI	UART1_TXD	ALT1
	ECSPI3_SCLK	UART1_RXD	ALT1
ENET1	ECSPI3_SS0	UART2_TXD	ALT1
	ENET1_1588_EVENT0_IN	GPIO1_IO08	ALT1
	ENET1_1588_EVENT0_OUT	GPIO1_IO09	ALT1
	ENET1_1588_EVENT1_IN	I2C2_SCL	ALT1
	ENET1_1588_EVENT1_OUT	I2C2_SDA	ALT1
	ENET1_MDC	ENET_MDC	ALT0
	ENET1_MDC	GPIO1_IO06	ALT1
	ENET1_MDC	I2C1_SCL	ALT1
	ENET1_MDIO	ENET_MDIO	ALT0
	ENET1_MDIO	GPIO1_IO07	ALT1
	ENET1_MDIO	I2C1_SDA	ALT1
	ENET1_RGMII_RD0	ENET_RD0	ALT0
	ENET1_RGMII_RD1	ENET_RD1	ALT0
	ENET1_RGMII_RD2	ENET_RD2	ALT0
	ENET1_RGMII_RD3	ENET_RD3	ALT0
	ENET1_RGMII_RX_CTL	ENET_RX_CTL	ALT0
	ENET1_RGMII_RXC	ENET_RXC	ALT0
	ENET1_RGMII_TD0	ENET_TD0	ALT0
	ENET1_RGMII_TD1	ENET_TD1	ALT0
	ENET1_RGMII_TD2	ENET_TD2	ALT0
	ENET1_RGMII_TD3	ENET_TD3	ALT0
	ENET1_RGMII_TX_CTL	ENET_TX_CTL	ALT0
	ENET1_RGMII_TXC	ENET_TXC	ALT0
	ENET1_RX_ER	ENET_RXC	ALT1
	ENET1_TX_ER	ENET_TXC	ALT1
	ENET1_TX_CLK (input)	ENET_TD2	ALT1
GPIO1	GPIO1_IO00	GPIO1_IO00	ALT0
	GPIO1_IO01	GPIO1_IO01	ALT0
	GPIO1_IO10	GPIO1_IO10	ALT0
	GPIO1_IO11	GPIO1_IO11	ALT0
	GPIO1_IO12	GPIO1_IO12	ALT0
	GPIO1_IO13	GPIO1_IO13	ALT0
	GPIO1_IO14	GPIO1_IO14	ALT0

Table continues on the next page...

External Signals and Pin Multiplexing

Instance	Port	Pad	Mode
	GPIO1_IO15	GPIO1_IO15	ALT0
	GPIO1_IO16	ENET_MDC	ALT5
	GPIO1_IO17	ENET_MDIO	ALT5
	GPIO1_IO18	ENET_TD3	ALT5
	GPIO1_IO19	ENET_TD2	ALT5
	GPIO1_IO02	GPIO1_IO02	ALT0
	GPIO1_IO20	ENET_TD1	ALT5
	GPIO1_IO21	ENET_TD0	ALT5
	GPIO1_IO22	ENET_TX_CTL	ALT5
	GPIO1_IO23	ENET_TXC	ALT5
	GPIO1_IO24	ENET_RX_CTL	ALT5
	GPIO1_IO25	ENET_RXC	ALT5
	GPIO1_IO26	ENET_RD0	ALT5
	GPIO1_IO27	ENET_RD1	ALT5
	GPIO1_IO28	ENET_RD2	ALT5
	GPIO1_IO29	ENET_RD3	ALT5
	GPIO1_IO03	GPIO1_IO03	ALT0
	GPIO1_IO04	GPIO1_IO04	ALT0
	GPIO1_IO05	GPIO1_IO05	ALT0
	GPIO1_IO06	GPIO1_IO06	ALT0
	GPIO1_IO07	GPIO1_IO07	ALT0
	GPIO1_IO08	GPIO1_IO08	ALT0
	GPIO1_IO09	GPIO1_IO09	ALT0
GPIO2	GPIO2_IO00	SD1_CLK	ALT5
	GPIO2_IO01	SD1_CMD	ALT5
	GPIO2_IO10	SD1_RESET_B	ALT5
	GPIO2_IO11	SD1_STROBE	ALT5
	GPIO2_IO12	SD2_CD_B	ALT5
	GPIO2_IO13	SD2_CLK	ALT5
	GPIO2_IO14	SD2_CMD	ALT5
	GPIO2_IO15	SD2_DATA0	ALT5
	GPIO2_IO16	SD2_DATA1	ALT5
	GPIO2_IO17	SD2_DATA2	ALT5
	GPIO2_IO18	SD2_DATA3	ALT5
	GPIO2_IO19	SD2_RESET_B	ALT5
	GPIO2_IO02	SD1_DATA0	ALT5
	GPIO2_IO20	SD2_WP	ALT5
	GPIO2_IO03	SD1_DATA1	ALT5
	GPIO2_IO04	SD1_DATA2	ALT5
	GPIO2_IO05	SD1_DATA3	ALT5
	GPIO2_IO06	SD1_DATA4	ALT5

Table continues on the next page...

Instance	Port	Pad	Mode
	GPIO2_IO07	SD1_DATA5	ALT5
	GPIO2_IO08	SD1_DATA6	ALT5
	GPIO2_IO09	SD1_DATA7	ALT5
GPIO3	GPIO3_IO00	NAND_ALE	ALT5
	GPIO3_IO01	NAND_CE0_B	ALT5
	GPIO3_IO10	NAND_DATA04	ALT5
	GPIO3_IO11	NAND_DATA05	ALT5
	GPIO3_IO12	NAND_DATA06	ALT5
	GPIO3_IO13	NAND_DATA07	ALT5
	GPIO3_IO14	NAND_DQS	ALT5
	GPIO3_IO15	NAND_RE_B	ALT5
	GPIO3_IO16	NAND_READY_B	ALT5
	GPIO3_IO17	NAND_WE_B	ALT5
	GPIO3_IO18	NAND_WP_B	ALT5
	GPIO3_IO19	SAI5_RXFS	ALT5
	GPIO3_IO02	NAND_CE1_B	ALT5
	GPIO3_IO20	SAI5_RXC	ALT5
	GPIO3_IO21	SAI5_RXD0	ALT5
	GPIO3_IO22	SAI5_RXD1	ALT5
	GPIO3_IO23	SAI5_RXD2	ALT5
	GPIO3_IO24	SAI5_RXD3	ALT5
	GPIO3_IO25	SAI5_MCLK	ALT5
	GPIO3_IO03	NAND_CE2_B	ALT5
	GPIO3_IO04	NAND_CE3_B	ALT5
	GPIO3_IO05	NAND_CLE	ALT5
	GPIO3_IO06	NAND_DATA00	ALT5
	GPIO3_IO07	NAND_DATA01	ALT5
	GPIO3_IO08	NAND_DATA02	ALT5
	GPIO3_IO09	NAND_DATA03	ALT5
GPIO4	GPIO4_IO00	SAI1_RXFS	ALT5
	GPIO4_IO01	SAI1_RXC	ALT5
	GPIO4_IO10	SAI1_TXFS	ALT5
	GPIO4_IO11	SAI1_TXC	ALT5
	GPIO4_IO12	SAI1_TXD0	ALT5
	GPIO4_IO13	SAI1_TXD1	ALT5
	GPIO4_IO14	SAI1_TXD2	ALT5
	GPIO4_IO15	SAI1_TXD3	ALT5
	GPIO4_IO16	SAI1_TXD4	ALT5
	GPIO4_IO17	SAI1_TXD5	ALT5
	GPIO4_IO18	SAI1_TXD6	ALT5
	GPIO4_IO19	SAI1_TXD7	ALT5

Table continues on the next page...

External Signals and Pin Multiplexing

Instance	Port	Pad	Mode
	GPIO4_IO02	SAI1_RXD0	ALT5
	GPIO4_IO20	SAI1_MCLK	ALT5
	GPIO4_IO21	SAI2_RXFS	ALT5
	GPIO4_IO22	SAI2_RXC	ALT5
	GPIO4_IO23	SAI2_RXD0	ALT5
	GPIO4_IO24	SAI2_TXFS	ALT5
	GPIO4_IO25	SAI2_TXC	ALT5
	GPIO4_IO26	SAI2_TXD0	ALT5
	GPIO4_IO27	SAI2_MCLK	ALT5
	GPIO4_IO28	SAI3_RXFS	ALT5
	GPIO4_IO29	SAI3_RXC	ALT5
	GPIO4_IO03	SAI1_RXD1	ALT5
	GPIO4_IO30	SAI3_RXD	ALT5
	GPIO4_IO31	SAI3_TXFS	ALT5
	GPIO4_IO04	SAI1_RXD2	ALT5
	GPIO4_IO05	SAI1_RXD3	ALT5
	GPIO4_IO06	SAI1_RXD4	ALT5
	GPIO4_IO07	SAI1_RXD5	ALT5
	GPIO4_IO08	SAI1_RXD6	ALT5
	GPIO4_IO09	SAI1_RXD7	ALT5
GPIO5	GPIO5_IO00	SAI3_TXC	ALT5
	GPIO5_IO01	SAI3_TXD	ALT5
	GPIO5_IO10	ECSPi2_SCLK	ALT5
	GPIO5_IO11	ECSPi2_MOSI	ALT5
	GPIO5_IO12	ECSPi2_MISO	ALT5
	GPIO5_IO13	ECSPi2_SS0	ALT5
	GPIO5_IO14	I2C1_SCL	ALT5
	GPIO5_IO15	I2C1_SDA	ALT5
	GPIO5_IO16	I2C2_SCL	ALT5
	GPIO5_IO17	I2C2_SDA	ALT5
	GPIO5_IO18	I2C3_SCL	ALT5
	GPIO5_IO19	I2C3_SDA	ALT5
	GPIO5_IO02	SAI3_MCLK	ALT5
	GPIO5_IO20	I2C4_SCL	ALT5
	GPIO5_IO21	I2C4_SDA	ALT5
	GPIO5_IO22	UART1_RXD	ALT5
	GPIO5_IO23	UART1_TXD	ALT5
	GPIO5_IO24	UART2_RXD	ALT5
	GPIO5_IO25	UART2_TXD	ALT5
	GPIO5_IO26	UART3_RXD	ALT5
	GPIO5_IO27	UART3_TXD	ALT5

Table continues on the next page...

Instance	Port	Pad	Mode
	GPIO5_IO28	UART4_RXD	ALT5
	GPIO5_IO29	UART4_TXD	ALT5
	GPIO5_IO03	SPDIF_TX	ALT5
	GPIO5_IO04	SPDIF_RX	ALT5
	GPIO5_IO05	SPDIF_EXT_CLK	ALT5
	GPIO5_IO06	ECSPI1_SCLK	ALT5
	GPIO5_IO07	ECSPI1_MOSI	ALT5
	GPIO5_IO08	ECSPI1_MISO	ALT5
	GPIO5_IO09	ECSPI1_SS0	ALT5
GPT1	GPT1_CAPTURE1	SAI3_RXFS	ALT1
	GPT1_CAPTURE2	SAI3_RXC	ALT1
	GPT1_CLK	SAI3_TXFS	ALT1
	GPT1_COMPARE1	SAI3_RXD	ALT1
	GPT1_COMPARE2	SAI3_TXC	ALT1
	GPT1_COMPARE3	SAI3_TXD	ALT1
GPT2	GPT2_CLK	I2C3_SCL	ALT2
GPT3	GPT3_CLK	I2C3_SDA	ALT2
HDMI	HDMI_AUX_N	HDMI_AUX_N	No Muxing
	HDMI_AUX_P	HDMI_AUX_P	No Muxing
	HDMI_CEC	HDMI_CEC	No Muxing
	HDMI_DDC_SCL	HDMI_DDC_SCL	No Muxing
	HDMI_DDC_SDA	HDMI_DDC_SDA	No Muxing
	HDMI_HPD	HDMI_HPD	No Muxing
	HDMI_REFCLK_N	HDMI_REFCLK_N	No Muxing
	HDMI_REFCLK_P	HDMI_REFCLK_P	No Muxing
	HDMI_REXT	HDMI_REXT	No Muxing
	HDMI_TX_N_LN_0	HDMI_TX_N_LN_0	No Muxing
	HDMI_TX_N_LN_1	HDMI_TX_N_LN_1	No Muxing
	HDMI_TX_N_LN_2	HDMI_TX_N_LN_2	No Muxing
	HDMI_TX_N_LN_3	HDMI_TX_N_LN_3	No Muxing
	HDMI_TX_P_LN_0	HDMI_TX_P_LN_0	No Muxing
	HDMI_TX_P_LN_1	HDMI_TX_P_LN_1	No Muxing
	HDMI_TX_P_LN_2	HDMI_TX_P_LN_2	No Muxing
	HDMI_TX_P_LN_3	HDMI_TX_P_LN_3	No Muxing
I2C1	I2C1_SDA	I2C1_SDA	ALT0
	I2C1_SCL	I2C1_SCL	ALT0
I2C2	I2C2_SDA	I2C2_SDA	ALT0
	I2C2_SCL	I2C2_SCL	ALT0
I2C3	I2C3_SDA	I2C3_SDA	ALT0
	I2C3_SCL	I2C3_SCL	ALT0
I2C4	I2C4_SDA	I2C4_SDA	ALT0

Table continues on the next page...

External Signals and Pin Multiplexing

Instance	Port	Pad	Mode
	I2C4_SCL	I2C4_SCL	ALT0
M4	M4_NMI	GPIO1_IO05	ALT1
MIPI_CSI1	MIPI_CSI1_CLK_N	MIPI_CSI1_CLK_N	No Muxing
	MIPI_CSI1_CLK_P	MIPI_CSI1_CLK_P	No Muxing
	MIPI_CSI1_DATA0_N	MIPI_CSI1_D0_N	No Muxing
	MIPI_CSI1_DATA0_P	MIPI_CSI1_D0_P	No Muxing
	MIPI_CSI1_DATA1_N	MIPI_CSI1_D1_N	No Muxing
	MIPI_CSI1_DATA1_P	MIPI_CSI1_D1_P	No Muxing
	MIPI_CSI1_DATA2_N	MIPI_CSI1_D2_N	No Muxing
	MIPI_CSI1_DATA2_P	MIPI_CSI1_D2_P	No Muxing
	MIPI_CSI1_DATA3_N	MIPI_CSI1_D3_N	No Muxing
	MIPI_CSI1_DATA3_P	MIPI_CSI1_D3_P	No Muxing
MIPI_CSI2	MIPI_CSI2_CLK_N	MIPI_CSI2_CLK_N	No Muxing
	MIPI_CSI2_CLK_P	MIPI_CSI2_CLK_P	No Muxing
	MIPI_CSI2_DATA0_N	MIPI_CSI2_D0_N	No Muxing
	MIPI_CSI2_DATA0_P	MIPI_CSI2_D0_P	No Muxing
	MIPI_CSI2_DATA1_N	MIPI_CSI2_D1_N	No Muxing
	MIPI_CSI2_DATA1_P	MIPI_CSI2_D1_P	No Muxing
	MIPI_CSI2_DATA2_N	MIPI_CSI2_D2_N	No Muxing
	MIPI_CSI2_DATA2_P	MIPI_CSI2_D2_P	No Muxing
	MIPI_CSI2_DATA3_N	MIPI_CSI2_D3_N	No Muxing
	MIPI_CSI2_DATA3_P	MIPI_CSI2_D3_P	No Muxing
MIPI_DSI	MIPI_DSI_CLK_N	MIPI_DSI_CLK_N	No Muxing
	MIPI_DSI_CLK_P	MIPI_DSI_CLK_P	No Muxing
	MIPI_DSI_DATA0_N	MIPI_DSI_D0_N	No Muxing
	MIPI_DSI_DATA0_P	MIPI_DSI_D0_P	No Muxing
	MIPI_DSI_DATA1_N	MIPI_DSI_D1_N	No Muxing
	MIPI_DSI_DATA1_P	MIPI_DSI_D1_P	No Muxing
	MIPI_DSI_DATA2_N	MIPI_DSI_D2_N	No Muxing
	MIPI_DSI_DATA2_P	MIPI_DSI_D2_P	No Muxing
	MIPI_DSI_DATA3_N	MIPI_DSI_D3_N	No Muxing
	MIPI_DSI_DATA3_P	MIPI_DSI_D3_P	No Muxing
	MIPI_DSI_REXT	MIPI_DSI_REXT	No Muxing
NAND	NAND_ALE	NAND_ALE	ALT0
	NAND_CE0_B	NAND_CE0_B	ALT0
	NAND_CE1_B	NAND_CE1_B	ALT0
	NAND_CE2_B	NAND_CE2_B	ALT0
	NAND_CE3_B	NAND_CE3_B	ALT0
	NAND_CLE	NAND_CLE	ALT0
	NAND_DATA00	NAND_DATA00	ALT0
	NAND_DATA01	NAND_DATA01	ALT0

Table continues on the next page...

Instance	Port	Pad	Mode
	NAND_DATA02	NAND_DATA02	ALT0
	NAND_DATA03	NAND_DATA03	ALT0
	NAND_DATA04	NAND_DATA04	ALT0
	NAND_DATA05	NAND_DATA05	ALT0
	NAND_DATA06	NAND_DATA06	ALT0
	NAND_DATA07	NAND_DATA07	ALT0
	NAND_DQS	NAND_DQS	ALT0
	NAND_RE_B	NAND_RE_B	ALT0
	NAND_READY_B	NAND_READY_B	ALT0
	NAND_WE_B	NAND_WE_B	ALT0
	NAND_WP_B	NAND_WP_B	ALT0
PCIE1	PCIE1_CLKREQ_B	I2C4_SCL	ALT2
	PCIE1_CLKREQ_B	UART4_RXD	ALT2
	PCIE1_REF_PAD_CLK_N	PCIE1_REF_PAD_CLK_N	No Muxing
	PCIE1_REF_PAD_CLK_P	PCIE1_REF_PAD_CLK_P	No Muxing
	PCIE1_RESREF	PCIE1_RESREF	No Muxing
	PCIE1_RXN_N	PCIE1_RXN_N	No Muxing
	PCIE1_RXN_P	PCIE1_RXN_P	No Muxing
	PCIE1_TXN_N	PCIE1_TXN_N	No Muxing
	PCIE1_TXN_P	PCIE1_TXN_P	No Muxing
PCIE2	PCIE2_CLKREQ_B	I2C4_SDA	ALT2
	PCIE2_CLKREQ_B	UART4_TXD	ALT2
	PCIE2_REF_PAD_CLK_N	PCIE2_REF_PAD_CLK_N	No Muxing
	PCIE2_REF_PAD_CLK_P	PCIE2_REF_PAD_CLK_P	No Muxing
	PCIE2_RESREF	PCIE2_RESREF	No Muxing
	PCIE2_RXN_N	PCIE2_RXN_N	No Muxing
	PCIE2_RXN_P	PCIE2_RXN_P	No Muxing
	PCIE2_TXN_N	PCIE2_TXN_N	No Muxing
	PCIE2_TXN_P	PCIE2_TXN_P	No Muxing
PWM1	PWM1_OUT	GPIO1_IO01	ALT1
	PWM1_OUT	I2C4_SDA	ALT1
	PWM1_OUT	SPDIF_EXT_CLK	ALT1
PWM2	PWM2_OUT	I2C4_SCL	ALT1
	PWM2_OUT	SPDIF_RX	ALT1
	PWM2_OUT	GPIO1_IO13	ALT5
PWM3	PWM3_OUT	I2C3_SDA	ALT1
	PWM3_OUT	SPDIF_TX	ALT1
	PWM3_OUT	GPIO1_IO14	ALT5
PWM4	PWM4_OUT	I2C3_SCL	ALT1
	PWM4_OUT	SAI3_MCLK	ALT1
	PWM4_OUT	GPIO1_IO15	ALT5

Table continues on the next page...

External Signals and Pin Multiplexing

Instance	Port	Pad	Mode
QSPI	QSPI_A_DATA00	NAND_DATA00	ALT1
	QSPI_A_DATA01	NAND_DATA01	ALT1
	QSPI_A_DATA02	NAND_DATA02	ALT1
	QSPI_A_DATA03	NAND_DATA03	ALT1
	QSPI_A_DQS	NAND_DQS	ALT1
	QSPI_A_SCLK	NAND_ALE	ALT1
	QSPI_A_SS0_B	NAND_CE0_B	ALT1
	QSPI_A_SS1_B	NAND_CE1_B	ALT1
	QSPI_B_DATA00	NAND_DATA04	ALT1
	QSPI_B_DATA01	NAND_DATA05	ALT1
	QSPI_B_DATA02	NAND_DATA06	ALT1
	QSPI_B_DATA03	NAND_DATA07	ALT1
	QSPI_B_DQS	NAND_RE_B	ALT1
	QSPI_B_SCLK	NAND_CLE	ALT1
	QSPI_B_SS0_B	NAND_CE2_B	ALT1
	QSPI_B_SS1_B	NAND_CE3_B	ALT1
SAI1	SAI1_MCLK	SAI1_MCLK	ALT0
	SAI1_RX_BCLK	SAI1_RXC	ALT0
	SAI1_RX_DATA00	SAI1_RXD0	ALT0
	SAI1_RX_DATA01	SAI1_RXD1	ALT0
	SAI1_RX_DATA02	SAI1_RXD2	ALT0
	SAI1_RX_DATA03	SAI1_RXD3	ALT0
	SAI1_RX_DATA04	SAI1_RXD4	ALT0
	SAI1_RX_DATA05	SAI1_RXD5	ALT0
	SAI1_RX_DATA06	SAI1_RXD6	ALT0
	SAI1_RX_DATA07	SAI1_RXD7	ALT0
	SAI1_RX_SYNC	SAI1_RXFS	ALT0
	SAI1_RX_SYNC	SAI1_RXD5	ALT3
	SAI1_TX_BCLK	SAI1_TXC	ALT0
	SAI1_TX_BCLK	SAI5_MCLK	ALT1
	SAI1_TX_BCLK	SAI1_MCLK	ALT2
	SAI1_TX_DATA00	SAI1_TXD0	ALT0
	SAI1_TX_DATA00	SAI5_RXFS	ALT1
	SAI1_TX_DATA01	SAI1_TXD1	ALT0
	SAI1_TX_DATA01	SAI5_RXC	ALT1
	SAI1_TX_DATA02	SAI1_TXD2	ALT0
	SAI1_TX_DATA02	SAI5_RXD0	ALT1
	SAI1_TX_DATA03	SAI1_TXD3	ALT0
	SAI1_TX_DATA03	SAI5_RXD1	ALT1
	SAI1_TX_DATA04	SAI1_TXD4	ALT0
	SAI1_TX_DATA04	SAI5_RXD2	ALT1

Table continues on the next page...

Instance	Port	Pad	Mode
	SAI1_TX_DATA04	SAI1_RXD7	ALT3
	SAI1_TX_DATA05	SAI1_TXD5	ALT0
	SAI1_TX_DATA05	SAI5_RXD3	ALT1
	SAI1_TX_DATA06	SAI1_TXD6	ALT0
	SAI1_TX_DATA07	SAI1_TXD7	ALT0
	SAI1_TX_SYNC	SAI1_TXFS	ALT0
	SAI1_TX_SYNC	SAI5_RXD1	ALT2
	SAI1_TX_SYNC	SAI5_RXD2	ALT2
	SAI1_TX_SYNC	SAI5_RXD3	ALT2
	SAI1_TX_SYNC	SAI1_RXD7	ALT2
SAI2	SAI2_MCLK	SAI2_MCLK	ALT0
	SAI2_RX_BCLK	SAI2_RXC	ALT0
	SAI2_RX_DATA00	SAI2_RXD0	ALT0
	SAI2_RX_SYNC	SAI2_RXFS	ALT0
	SAI2_TX_BCLK	SAI2_TXC	ALT0
	SAI2_TX_DATA00	SAI2_TXD0	ALT0
	SAI2_TX_SYNC	SAI2_TXFS	ALT0
SAI3	SAI3_MCLK	SAI3_MCLK	ALT0
	SAI3_RX_BCLK	SAI3_RXC	ALT0
	SAI3_RX_DATA00	SAI3_RXD	ALT0
	SAI3_RX_SYNC	SAI3_RXFS	ALT0
	SAI3_TX_BCLK	SAI3_TXC	ALT0
	SAI3_TX_DATA00	SAI3_TXD	ALT0
	SAI3_TX_SYNC	SAI3_TXFS	ALT0
SAI4	SAI4_MCLK	SAI5_MCLK	ALT2
SAI5	SAI5_MCLK	SAI1_MCLK	ALT1
	SAI5_MCLK	SAI2_MCLK	ALT1
	SAI5_MCLK	SAI5_MCLK	ALT0
	SAI5_MCLK	SAI3_MCLK	ALT2
	SAI5_RX_BCLK	SAI1_RXC	ALT1
	SAI5_RX_BCLK	SAI5_RXC	ALT0
	SAI5_RX_BCLK	SAI3_RXC	ALT2
	SAI5_RX_DATA00	SAI1_RXD0	ALT1
	SAI5_RX_DATA00	SAI5_RXD0	ALT0
	SAI5_RX_DATA00	SAI3_RXD	ALT2
	SAI5_RX_DATA01	SAI1_RXD1	ALT1
	SAI5_RX_DATA01	SAI5_RXD1	ALT0
	SAI5_RX_DATA01	SAI3_TXFS	ALT2
	SAI5_RX_DATA02	SAI1_RXD2	ALT1
	SAI5_RX_DATA02	SAI5_RXD2	ALT0
	SAI5_RX_DATA02	SAI3_TXC	ALT2

Table continues on the next page...

External Signals and Pin Multiplexing

Instance	Port	Pad	Mode
	SAI5_RX_DATA03	SAI1_RXD3	ALT1
	SAI5_RX_DATA03	SAI5_RXD3	ALT0
	SAI5_RX_DATA03	SAI3_TXD	ALT2
	SAI5_RX_SYNC	SAI1_RXFS	ALT1
	SAI5_RX_SYNC	SAI5_RXFS	ALT0
	SAI5_RX_SYNC	SAI3_RXFS	ALT2
	SAI5_TX_BCLK	SAI1_TXC	ALT1
	SAI5_TX_BCLK	SAI2_RXC	ALT1
	SAI5_TX_BCLK	SAI5_RXD2	ALT3
	SAI5_TX_DATA00	SAI1_TXD0	ALT1
	SAI5_TX_DATA00	SAI2_RXD0	ALT1
	SAI5_TX_DATA00	SAI5_RXD3	ALT3
	SAI5_TX_DATA01	SAI1_TXD1	ALT1
	SAI5_TX_DATA01	SAI2_TXFS	ALT1
	SAI5_TX_DATA02	SAI1_TXD2	ALT1
	SAI5_TX_DATA02	SAI2_TXC	ALT1
	SAI5_TX_DATA03	SAI1_TXD3	ALT1
	SAI5_TX_DATA03	SAI2_TXD0	ALT1
	SAI5_TX_SYNC	SAI1_TXFS	ALT1
	SAI5_TX_SYNC	SAI2_RXFS	ALT1
	SAI5_TX_SYNC	SAI5_RXD1	ALT3
SAI6	SAI6_MCLK	SAI1_RXD7	ALT1
	SAI6_MCLK	SAI1_TXD7	ALT1
	SAI6_RX_BCLK	SAI1_TXD4	ALT1
	SAI6_RX_BCLK	SAI1_RXD4	ALT2
	SAI6_RX_DATA00	SAI1_TXD5	ALT1
	SAI6_RX_DATA00	SAI1_RXD5	ALT2
	SAI6_RX_SYNC	SAI1_TXD6	ALT1
	SAI6_RX_SYNC	SAI1_RXD6	ALT2
	SAI6_TX_BCLK	SAI1_RXD4	ALT1
	SAI6_TX_BCLK	SAI1_TXD4	ALT2
	SAI6_TX_DATA00	SAI1_RXD5	ALT1
	SAI6_TX_DATA00	SAI1_TXD5	ALT2
	SAI6_TX_SYNC	SAI1_RXD6	ALT1
	SAI6_TX_SYNC	SAI1_TXD6	ALT2
SDMA1	SDMA1_EXT_EVENT00	GPIO1_IO03	ALT5
	SDMA1_EXT_EVENT01	GPIO1_IO04	ALT5
SJC	SJC_DE_B	GPIO1_IO02	ALT7
SDMA2	SDMA2_EXT_EVENT00	GPIO1_IO09	ALT5
	SDMA2_EXT_EVENT01	GPIO1_IO12	ALT5
SNVS	SNVS_ONOFF	ONOFF	ALT0

Table continues on the next page...

Instance	Port	Pad	Mode
	SNVS_PMIC_ON_REQ	PMIC_ON_REQ	ALT0
	SNVS_POR_B	POR_B	ALT0
	SNVS_RTC	RTC	ALT0
	SNVS_RTC_RESET_B	RTC_RESET_B	ALT0
SPDIF1	SPDIF1_EXT_CLK	SPDIF_EXT_CLK	ALT0
	SPDIF1_IN	SPDIF_RX	ALT0
	SPDIF1_OUT	SPDIF_TX	ALT0
SRC	SRC_BOOT_CFG00	SAI1_RXD0	ALT6
	SRC_BOOT_CFG01	SAI1_RXD1	ALT6
	SRC_BOOT_CFG10	SAI1_TXD2	ALT6
	SRC_BOOT_CFG11	SAI1_TXD3	ALT6
	SRC_BOOT_CFG12	SAI1_TXD4	ALT6
	SRC_BOOT_CFG13	SAI1_TXD5	ALT6
	SRC_BOOT_CFG14	SAI1_TXD6	ALT6
	SRC_BOOT_CFG15	SAI1_TXD7	ALT6
	SRC_BOOT_CFG02	SAI1_RXD2	ALT6
	SRC_BOOT_CFG03	SAI1_RXD3	ALT6
	SRC_BOOT_CFG04	SAI1_RXD4	ALT6
	SRC_BOOT_CFG05	SAI1_RXD5	ALT6
	SRC_BOOT_CFG06	SAI1_RXD6	ALT6
	SRC_BOOT_CFG07	SAI1_RXD7	ALT6
	SRC_BOOT_CFG08	SAI1_TXD0	ALT6
	SRC_BOOT_CFG09	SAI1_TXD1	ALT6
	SRC_BOOT_MODE0	BOOT_MODE0	ALT0
	SRC_BOOT_MODE1	BOOT_MODE1	ALT0
UART1	UART1_CTS_B	UART3_RXD	ALT1
	UART1_RTS_B	UART3_TXD	ALT1
	UART1_RX	UART1_RXD	ALT0
	UART1_TX	UART1_TXD	ALT0
UART2	UART2_CTS_B	UART4_RXD	ALT1
	UART2_RTS_B	UART4_TXD	ALT1
	UART2_RX	UART2_RXD	ALT0
	UART2_TX	UART2_TXD	ALT0
UART3	UART3_CTS_B	ECSPI1_MISO	ALT1
	UART3_RTS_B	ECSPI1_SS0	ALT1
	UART3_RX	ECSPI1_SCLK	ALT1
	UART3_RX	UART3_RXD	ALT0
	UART3_TX	ECSPI1_MOSI	ALT1
	UART3_TX	UART3_TXD	ALT0
UART4	UART4_CTS_B	ECSPI2_MISO	ALT1
	UART4_RTS_B	ECSPI2_SS0	ALT1

Table continues on the next page...

External Signals and Pin Multiplexing

Instance	Port	Pad	Mode
	UART4_RX	ECSPi2_SCLK	ALT1
	UART4_RX	UART4_RXD	ALT0
	UART4_TX	ECSPi2_MOSI	ALT1
	UART4_TX	UART4_TXD	ALT0
USB1	USB1_DN	USB1_DN	No Muxing
	USB1_DP	USB1_DP	No Muxing
	USB1_VBUS	USB1_VBUS	No Muxing
	USB1_ID	USB1_ID	No Muxing
	USB1_OTG_ID	GPIO1_IO10	ALT1
	USB1_OTG_OC	GPIO1_IO13	ALT1
	USB1_OTG_PWR	GPIO1_IO12	ALT1
	USB1_RESREF	USB1_RESREF	No Muxing
	USB1_RX_N	USB1_RX_N	No Muxing
	USB1_RX_P	USB1_RX_P	No Muxing
	USB1_TX_N	USB1_TX_N	No Muxing
	USB1_TX_P	USB1_TX_P	No Muxing
USB2	USB2_DN	USB2_DN	No Muxing
	USB2_DP	USB2_DP	No Muxing
	USB2_ID	USB2_ID	No Muxing
	USB2_OTG_ID	GPIO1_IO11	ALT1
	USB2_OTG_OC	GPIO1_IO15	ALT1
	USB2_OTG_PWR	GPIO1_IO14	ALT1
	USB2_RESREF	USB2_RESREF	No Muxing
	USB2_RX_N	USB2_RX_N	No Muxing
	USB2_RX_P	USB2_RX_P	No Muxing
	USB2_TX_N	USB2_TX_N	No Muxing
	USB2_TX_P	USB2_TX_P	No Muxing
	USB2_VBUS	USB2_VBUS	No Muxing
USDHC1	USDHC1_CD_B	GPIO1_IO06	ALT5
	USDHC1_CLK	SD1_CLK	ALT0
	USDHC1_CMD	SD1_CMD	ALT0
	USDHC1_DATA0	SD1_DATA0	ALT0
	USDHC1_DATA1	SD1_DATA1	ALT0
	USDHC1_DATA2	SD1_DATA2	ALT0
	USDHC1_DATA3	SD1_DATA3	ALT0
	USDHC1_DATA4	SD1_DATA4	ALT0
	USDHC1_DATA5	SD1_DATA5	ALT0
	USDHC1_DATA6	SD1_DATA6	ALT0
	USDHC1_DATA7	SD1_DATA7	ALT0
	USDHC1_RESET_B	SD1_RESET_B	ALT0
	USDHC1_STROBE	SD1_STROBE	ALT0

Table continues on the next page...

Instance	Port	Pad	Mode
	USDHC1_VSELECT	GPIO1_IO03	ALT1
	USDHC1_WP	GPIO1_IO07	ALT5
USDHC2	USDHC2_CD_B	SD2_CD_B	ALT0
	USDHC2_CLK	SD2_CLK	ALT0
	USDHC2_CMD	SD2_CMD	ALT0
	USDHC2_DATA0	SD2_DATA0	ALT0
	USDHC2_DATA1	SD2_DATA1	ALT0
	USDHC2_DATA2	SD2_DATA2	ALT0
	USDHC2_DATA3	SD2_DATA3	ALT0
	USDHC2_RESET_B	SD2_RESET_B	ALT0
	USDHC2_RESET_B	GPIO1_IO08	ALT5
	USDHC2_VSELECT	GPIO1_IO04	ALT1
	USDHC2_WP	SD2_WP	ALT0
WDOG1	WDOG1_WDOG_ANY	GPIO1_IO02	ALT5
	WDOG1_WDOG_B	GPIO1_IO02	ALT1
XTALOSC	XTALI_25M	XTALI_25M	No Muxing
	XTALI_27M	XTALI_27M	No Muxing
	XTALO_25M	XTALO_25M	No Muxing
	XTALO_27M	XTALO_27M	No Muxing

8.1.1.2 Muxing Options

NOTE

System STOP/WAIT status can be observed on pin. See [CCM_WAIT](#) and [CCM_STOP](#) in the table below.

Table 8-1. Muxing Options

Instance	Port	Pad	Mode
ADC1	ADC1_IN0	GPIO_AD_B1_11	-
	ADC1_IN1	GPIO_AD_B0_12	-
	ADC1_IN2	GPIO_AD_B0_13	-
	ADC1_IN3	GPIO_AD_B0_14	-
	ADC1_IN4	GPIO_AD_B0_15	-
	ADC1_IN5	GPIO_AD_B1_00	-
	ADC1_IN6	GPIO_AD_B1_01	-
	ADC1_IN7	GPIO_AD_B1_02	-
	ADC1_IN8	GPIO_AD_B1_03	-
	ADC1_IN9	GPIO_AD_B1_04	-
	ADC1_IN10	GPIO_AD_B1_05	-

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	ADC1_IN11	GPIO_AD_B1_06	-
	ADC1_IN12	GPIO_AD_B1_07	-
	ADC1_IN13	GPIO_AD_B1_08	-
	ADC1_IN14	GPIO_AD_B1_09	-
	ADC1_IN15	GPIO_AD_B1_10	-
ADC2	ADC2_IN0	GPIO_AD_B1_11	-
	ADC2_IN1	GPIO_AD_B1_12	-
	ADC2_IN2	GPIO_AD_B1_13	-
	ADC2_IN3	GPIO_AD_B1_14	-
	ADC2_IN4	GPIO_AD_B1_15	-
	ADC2_IN5	GPIO_AD_B1_00	-
	ADC2_IN6	GPIO_AD_B1_01	-
	ADC2_IN7	GPIO_AD_B1_02	-
	ADC2_IN8	GPIO_AD_B1_03	-
	ADC2_IN9	GPIO_AD_B1_04	-
	ADC2_IN10	GPIO_AD_B1_05	-
	ADC2_IN11	GPIO_AD_B1_06	-
	ADC2_IN12	GPIO_AD_B1_07	-
	ADC2_IN13	GPIO_AD_B1_08	-
	ADC2_IN14	GPIO_AD_B1_09	-
	ADC2_IN15	GPIO_AD_B1_10	-
ACMP1	ACMP1_IN0	GPIO_AD_B1_01	-
	ACMP1_IN1	GPIO_AD_B1_06	-
	ACMP1_IN2	GPIO_AD_B0_13	-
	ACMP1_IN3	GPIO_AD_B1_02	-
	ACMP1_IN4	GPIO_AD_B0_00	-
	ACMP1_IN5	GPIO_AD_B1_07	-
	ACMP1_IN6	GPIO_AD_B1_11	-
	ACMP1_OUT	GPIO_AD_B1_12	-
ACMP2	ACMP2_IN0	GPIO_AD_B1_01	-
	ACMP2_IN1	GPIO_AD_B1_06	-
	ACMP2_IN2	GPIO_AD_B0_14	-
	ACMP2_IN3	GPIO_AD_B1_03	-
	ACMP2_IN4	GPIO_AD_B0_01	-
	ACMP2_IN5	GPIO_AD_B1_08	-
	ACMP2_IN6	GPIO_AD_B1_12	-
	ACMP2_OUT	GPIO_AD_B1_13	-
ACMP3	ACMP3_IN0	GPIO_AD_B1_01	-
	ACMP3_IN1	GPIO_AD_B1_06	-
	ACMP3_IN2	GPIO_AD_B0_15	-

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	ACMP3_IN3	GPIO_AD_B1_04	-
	ACMP3_IN4	GPIO_AD_B0_02	-
	ACMP3_IN5	GPIO_AD_B1_09	-
	ACMP3_IN6	GPIO_AD_B1_13	-
	ACMP3_OUT	GPIO_AD_B1_14	-
ACMP4	ACMP4_IN0	GPIO_AD_B1_01	-
	ACMP4_IN1	GPIO_AD_B1_06	-
	ACMP4_IN2	GPIO_AD_B1_00	-
	ACMP4_IN3	GPIO_AD_B1_05	-
	ACMP4_IN4	GPIO_AD_B0_03	-
	ACMP4_IN5	GPIO_AD_B1_10	-
	ACMP4_IN6	GPIO_AD_B1_14	-
	ACMP4_OUT	GPIO_AD_B1_15	-
ARM PLATFORM	ARM_EVENT0	GPIO_B0_14	ALT2
	ARM_EVENT1	GPIO_B0_15	ALT2
CCM	CCM_PMIC_STBY_REQ	PMIC_STBY_REQ	ALT0
	CCM_PMIC_READY	GPIO_AD_B0_12	ALT1
		GPIO_EMC_32	ALT3
		GPIO_AD_B1_08	ALT3
		GPIO_AD_B1_01	ALT4
		GPIO_SD_B1_03	ALT6
	CCM_CLKO1	GPIO_SD_B0_04	ALT6
	CCM_CLKO2	GPIO_SD_B0_05	ALT6
	CCM_WAIT	GPIO_SD_B1_02	ALT6
	CCM_STOP	GPIO_SD_B1_04	ALT6
	CCM_CLK1_P	CCM_CLK1_P	No Muxing
	CCM_CLK1_N	CCM_CLK1_N	No Muxing
CSI	CSI_DATA00	GPIO_B1_10	ALT2
	CSI_DATA01	GPIO_B1_11	ALT2
	CSI_DATA02	GPIO_AD_B1_15	ALT4
		GPIO_AD_B0_11	ALT4
	CSI_DATA03	GPIO_AD_B1_14	ALT4
		GPIO_AD_B0_10	ALT4
	CSI_DATA04	GPIO_AD_B1_13	ALT4
		GPIO_AD_B0_09	ALT4
	CSI_DATA05	GPIO_AD_B1_12	ALT4
		GPIO_AD_B0_08	ALT4
	CSI_DATA06	GPIO_AD_B1_11	ALT4
		GPIO_AD_B0_07	ALT4
	CSI_DATA07	GPIO_AD_B1_10	ALT4

Table continues on the next page...

Table 8-1. Muxing Options (continued)

		GPIO_AD_B0_06	ALT4
	CSI_DATA08	GPIO_AD_B1_09	ALT4
		GPIO_AD_B0_05	ALT4
	CSI_DATA09	GPIO_AD_B1_08	ALT4
		GPIO_AD_B0_04	ALT4
	CSI_DATA10	GPIO_B1_09	ALT2
	CSI_DATA11	GPIO_B1_08	ALT2
	CSI_DATA12	GPIO_B1_07	ALT2
	CSI_DATA13	GPIO_B1_06	ALT2
	CSI_DATA14	GPIO_B1_05	ALT2
	CSI_DATA15	GPIO_B1_04	ALT2
	CSI_DATA16	GPIO_EMC_37	ALT4
	CSI_DATA17	GPIO_EMC_36	ALT4
	CSI_DATA18	GPIO_EMC_35	ALT4
	CSI_DATA19	GPIO_EMC_34	ALT4
	CSI_DATA20	GPIO_EMC_33	ALT4
	CSI_DATA21	GPIO_EMC_32	ALT4
	CSI_DATA22	GPIO_EMC_31	ALT4
	CSI_DATA23	GPIO_EMC_30	ALT4
	CSI_PIXCLK	GPIO_B1_12	ALT2
		GPIO_AD_B1_04	ALT4
	CSI_VSYNC	GPIO_B1_13	ALT2
		GPIO_AD_B1_06	ALT4
		GPIO_AD_B0_14	ALT4
	CSI_HSYNC	GPIO_B1_14	ALT2
		GPIO_AD_B1_07	ALT4
		GPIO_AD_B0_15	ALT4
	CSI_MCLK	GPIO_B1_15	ALT2
		GPIO_AD_B1_05	ALT4
	CSI_FIELD	GPIO_EMC_38	ALT4
ENET	ENET_MDC	GPIO_B1_14	ALT0
		GPIO_AD_B1_04	ALT1
		GPIO_EMC_40	ALT4
	ENET_MDIO	GPIO_B1_15	ALT0
		GPIO_AD_B1_05	ALT1
		GPIO_EMC_41	ALT4
	ENET_TX_DATA0	GPIO_EMC_22	ALT3
		GPIO_B1_07	ALT3
	ENET_TX_DATA1	GPIO_EMC_21	ALT3
		GPIO_B1_08	ALT3

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	ENET_TX_DATA2	GPIO_AD_B0_05	ALT2
	ENET_TX_DATA3	GPIO_AD_B0_04	ALT2
	ENET_TX_ER	GPIO_AD_B0_07	ALT2
	ENET_TX_EN	GPIO_EMC_24	ALT3
		GPIO_B1_09	ALT3
	ENET_TX_CLK	GPIO_B1_10	ALT3
		GPIO_EMC_25	ALT3
	ENET_RX_DATA0	GPIO_B1_04	ALT3
		GPIO_EMC_20	ALT3
	ENET_RX_DATA1	GPIO_B1_05	ALT3
		GPIO_EMC_19	ALT3
	ENET_RX_DATA2	GPIO_AD_B0_09	ALT2
	ENET_RX_DATA3	GPIO_AD_B0_08	ALT2
	ENET_RX_ER	GPIO_B1_11	ALT3
		GPIO_EMC_26	ALT3
	ENET_RX_EN	GPIO_B1_06	ALT3
		GPIO_EMC_23	ALT3
	ENET_RX_CLK	GPIO_AD_B0_06	ALT2
	ENET_CRS	GPIO_AD_B0_10	ALT2
	ENET_COL	GPIO_AD_B0_11	ALT2
	ENET_REF_CLK	GPIO_EMC_25	ALT4
		GPIO_B1_10	ALT6
	ENET_1588_EVENT0_OUT	GPIO_AD_B0_14	ALT3
		GPIO_AD_B0_10	ALT7
		GPIO_B1_13	ALT3
	ENET_1588_EVENT0_IN	GPIO_AD_B0_15	ALT3
		GPIO_AD_B0_11	ALT7
		GPIO_B1_12	ALT3
	ENET_1588_EVENT1_OUT	GPIO_AD_B0_12	ALT6
	ENET_1588_EVENT1_IN	GPIO_AD_B0_13	ALT6
	ENET_1588_EVENT2_OUT	GPIO_AD_B1_02	ALT4
	ENET_1588_EVENT2_IN	GPIO_AD_B1_03	ALT4
	ENET_1588_EVENT3_OUT	GPIO_AD_B0_07	ALT7
	ENET_1588_EVENT3_IN	GPIO_AD_B0_08	ALT7
ENET2	ENET2_MDC	GPIO_EMC_38	ALT8
		GPIO_B0_00	ALT8
	ENET2_MDIO	GPIO_EMC_39	ALT8
		GPIO_B0_01	ALT8
	ENET2_TDATA0	GPIO_EMC_30	ALT8
		GPIO_B0_12	ALT8

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	GPIO_B1_14	ALT8
ENET2_TDATA1	GPIO_EMC_31	ALT8
	GPIO_B0_13	ALT8
	GPIO_B1_15	ALT8
ENET2_TDATA2	GPIO_B0_05	ALT8
ENET2_TDATA3	GPIO_B0_04	ALT8
ENET2_TX_CLK	GPIO_EMC_33	ALT8
	GPIO_B0_15	ALT8
	GPIO_SD_B0_01	ALT8
ENET2_TX_EN	GPIO_EMC_32	ALT8
	GPIO_B0_14	ALT8
	GPIO_SD_B0_00	ALT8
ENET2_TX_ER	GPIO_B0_07	ALT8
ENET2_RDATA0	GPIO_EMC_35	ALT8
	GPIO_B1_01	ALT8
	GPIO_SD_B0_03	ALT8
ENET2_RDATA1	GPIO_EMC_36	ALT8
	GPIO_B1_02	ALT8
	GPIO_SD_B0_04	ALT8
ENET2_RDATA2	GPIO_B0_09	ALT8
ENET2_RDATA3	GPIO_B0_08	ALT8
ENET2_RX_CLK	GPIO_B0_06	ALT8
ENET2_RX_EN	GPIO_EMC_37	ALT8
	GPIO_B1_03	ALT8
	GPIO_SD_B0_05	ALT8
ENET2_RX_ER	GPIO_EMC_34	ALT8
	GPIO_B1_00	ALT8
	GPIO_SD_B0_02	ALT8
ENET2_REF_CLK2	GPIO_EMC_33	ALT9
	GPIO_B0_15	ALT9
	GPIO_SD_B0_01	ALT9
ENET2_1588_EVENT0_IN	GPIO_AD_B1_01	ALT8
	GPIO_B0_03	ALT8
ENET2_COL	GPIO_B0_11	ALT8
ENET2_CRS	GPIO_B0_10	ALT8
ENET2_1588_EVENT1_IN	GPIO_AD_B1_11	ALT8
ENET2_1588_EVENT2_IN	GPIO_AD_B1_13	ALT8
ENET2_1588_EVENT3_IN	GPIO_AD_B1_15	ALT8
ENET2_1588_EVENT0_OUT	GPIO_AD_B1_00	ALT8
	GPIO_B0_02	ALT8

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	ENET2_1588_EVENT1_OUT	GPIO_AD_B1_10	ALT8
	ENET2_1588_EVENT2_OUT	GPIO_AD_B1_12	ALT8
	ENET2_1588_EVENT3_OUT	GPIO_AD_B1_14	ALT8
EWM	EWM_OUT_B	GPIO_AD_B0_01	ALT6
		GPIO_AD_B0_13	ALT3
		GPIO_AD_B1_11	ALT1
FLEXIO1	FLEXIO1_D00	GPIO_EMC_00	ALT4
	FLEXIO1_D01	GPIO_EMC_01	ALT4
	FLEXIO1_D02	GPIO_EMC_02	ALT4
	FLEXIO1_D03	GPIO_EMC_03	ALT4
	FLEXIO1_D04	GPIO_EMC_04	ALT4
	FLEXIO1_D05	GPIO_EMC_05	ALT4
	FLEXIO1_D06	GPIO_EMC_06	ALT4
	FLEXIO1_D07	GPIO_EMC_07	ALT4
	FLEXIO1_D08	GPIO_EMC_08	ALT4
	FLEXIO1_D09	GPIO_EMC_09	ALT4
	FLEXIO1_D10	GPIO_EMC_10	ALT4
	FLEXIO1_D11	GPIO_EMC_11	ALT4
	FLEXIO1_D12	GPIO_EMC_26	ALT4
	FLEXIO1_D13	GPIO_EMC_27	ALT4
	FLEXIO1_D14	GPIO_EMC_28	ALT4
	FLEXIO1_D15	GPIO_EMC_29	ALT4
FLEXIO2	FLEXIO2_D00	GPIO_B0_00	ALT4
	FLEXIO2_D01	GPIO_B0_01	ALT4
	FLEXIO2_D02	GPIO_B0_02	ALT4
	FLEXIO2_D03	GPIO_B0_03	ALT4
	FLEXIO2_D04	GPIO_B0_04	ALT4
	FLEXIO2_D05	GPIO_B0_05	ALT4
	FLEXIO2_D06	GPIO_B0_06	ALT4
	FLEXIO2_D07	GPIO_B0_07	ALT4
	FLEXIO2_D08	GPIO_B0_08	ALT4
	FLEXIO2_D09	GPIO_B0_09	ALT4
	FLEXIO2_D10	GPIO_B0_10	ALT4
	FLEXIO2_D11	GPIO_B0_11	ALT4
	FLEXIO2_D12	GPIO_B0_12	ALT4
	FLEXIO2_D13	GPIO_B0_13	ALT4
	FLEXIO2_D14	GPIO_B0_14	ALT4
	FLEXIO2_D15	GPIO_B0_15	ALT4
	FLEXIO2_D16	GPIO_B1_00	ALT4
	FLEXIO2_D17	GPIO_B1_01	ALT4

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	FLEXIO2_D18	GPIO_B1_02	ALT4
	FLEXIO2_D19	GPIO_B1_03	ALT4
	FLEXIO2_D20	GPIO_B1_04	ALT4
	FLEXIO2_D21	GPIO_B1_05	ALT4
	FLEXIO2_D22	GPIO_B1_06	ALT4
	FLEXIO2_D23	GPIO_B1_07	ALT4
	FLEXIO2_D24	GPIO_B1_08	ALT4
	FLEXIO2_D25	GPIO_B1_09	ALT4
	FLEXIO2_D26	GPIO_B1_10	ALT4
	FLEXIO2_D27	GPIO_B1_11	ALT4
	FLEXIO2_D28	GPIO_B1_12	ALT4
	FLEXIO2_D29	GPIO_B1_13	ALT4
	FLEXIO2_D30	GPIO_B1_14	ALT4
	FLEXIO2_D31	GPIO_B1_15	ALT4
FLEXIO3	FLEXIO3_D00	GPIO_AD_B1_00	ALT9
	FLEXIO3_D01	GPIO_AD_B1_01	ALT9
	FLEXIO3_D02	GPIO_AD_B1_02	ALT9
	FLEXIO3_D03	GPIO_AD_B1_03	ALT9
	FLEXIO3_D04	GPIO_AD_B1_04	ALT9
	FLEXIO3_D05	GPIO_AD_B1_05	ALT9
	FLEXIO3_D06	GPIO_AD_B1_06	ALT9
	FLEXIO3_D07	GPIO_AD_B1_07	ALT9
	FLEXIO3_D08	GPIO_AD_B1_08	ALT9
	FLEXIO3_D09	GPIO_AD_B1_09	ALT9
	FLEXIO3_D10	GPIO_AD_B1_10	ALT9
	FLEXIO3_D11	GPIO_AD_B1_11	ALT9
	FLEXIO3_D12	GPIO_AD_B1_12	ALT9
	FLEXIO3_D13	GPIO_AD_B1_13	ALT9
	FLEXIO3_D14	GPIO_AD_B1_14	ALT9
	FLEXIO3_D15	GPIO_AD_B1_15	ALT9
	FLEXIO3_D16	GPIO_B1_00	ALT9
	FLEXIO3_D17	GPIO_B1_01	ALT9
	FLEXIO3_D18	GPIO_B1_02	ALT9
	FLEXIO3_D19	GPIO_B1_03	ALT9
	FLEXIO3_D20	GPIO_B1_04	ALT9
	FLEXIO3_D21	GPIO_B1_05	ALT9
	FLEXIO3_D22	GPIO_B1_06	ALT9
	FLEXIO3_D23	GPIO_B1_07	ALT9
	FLEXIO3_D24	GPIO_B1_08	ALT9
	FLEXIO3_D25	GPIO_B1_09	ALT9

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	FLEXIO3_D26	GPIO_B1_10	ALT9
	FLEXIO3_D27	GPIO_B1_11	ALT9
	FLEXIO3_D28	GPIO_B1_12	ALT9
	FLEXIO3_D29	GPIO_B1_13	ALT9
	FLEXIO3_D30	GPIO_B1_14	ALT9
	FLEXIO3_D31	GPIO_B1_15	ALT9
FLEXPWM1	FLEXPWM1_PWM0_A	GPIO_EMC_23	ALT1
		GPIO_SD_B0_00	ALT1
	FLEXPWM1_PWM0_B	GPIO_SD_B0_01	ALT1
		GPIO_EMC_24	ALT1
	FLEXPWM1_PWM1_A	GPIO_EMC_25	ALT1
		GPIO_SD_B0_02	ALT1
	FLEXPWM1_PWM1_B	GPIO_EMC_26	ALT1
		GPIO_SD_B0_03	ALT1
	FLEXPWM1_PWM2_A	GPIO_SD_B0_04	ALT1
		GPIO_EMC_27	ALT1
	FLEXPWM1_PWM2_B	GPIO_SD_B0_05	ALT1
		GPIO_EMC_28	ALT1
	FLEXPWM1_PWM3_A	GPIO_EMC_38	ALT1
		GPIO_SD_B1_00	ALT2
		GPIO_AD_B0_10	ALT1
		GPIO_EMC_12	ALT4
		GPIO_B1_00	ALT6
	FLEXPWM1_PWM3_B	GPIO_EMC_39	ALT1
		GPIO_B1_01	ALT6
		GPIO_AD_B0_11	ALT1
		GPIO_SD_B1_01	ALT2
		GPIO_EMC_13	ALT4
	FLEXPWM1_PWM0_X	GPIO_AD_B0_02	ALT4
	FLEXPWM1_PWM1_X	GPIO_AD_B0_03	ALT4
	FLEXPWM1_PWM2_X	GPIO_AD_B0_12	ALT4
	FLEXPWM1_PWM3_X	GPIO_AD_B0_13	ALT4
FLEXPWM2	FLEXPWM2_PWM0_A	GPIO_B0_06	ALT2
		GPIO_EMC_06	ALT1
	FLEXPWM2_PWM0_B	GPIO_B0_07	ALT2
		GPIO_EMC_07	ALT1
	FLEXPWM2_PWM1_A	GPIO_EMC_08	ALT1
		GPIO_B0_08	ALT2
	FLEXPWM2_PWM1_B	GPIO_EMC_09	ALT1
		GPIO_B0_09	ALT2

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	FLEXPWM2_PWM2_A	GPIO_EMC_10	ALT1
		GPIO_B0_10	ALT2
	FLEXPWM2_PWM2_B	GPIO_EMC_11	ALT1
		GPIO_B0_11	ALT2
	FLEXPWM2_PWM3_A	GPIO_AD_B0_09	ALT1
		GPIO_SD_B1_02	ALT2
		GPIO_EMC_19	ALT1
		GPIO_B1_02	ALT6
		GPIO_AD_B0_00	ALT0
	FLEXPWM2_PWM3_B	GPIO_AD_B0_01	ALT0
		GPIO_SD_B1_03	ALT2
		GPIO_B1_03	ALT6
		GPIO_EMC_20	ALT1
FLEXPWM3	FLEXPWM3_PWM0_A	GPIO_EMC_29	ALT1
	FLEXPWM3_PWM0_B	GPIO_EMC_30	ALT1
	FLEXPWM3_PWM1_A	GPIO_EMC_31	ALT1
	FLEXPWM3_PWM1_B	GPIO_EMC_32	ALT1
	FLEXPWM3_PWM2_A	GPIO_EMC_33	ALT1
	FLEXPWM3_PWM2_B	GPIO_EMC_34	ALT1
	FLEXPWM3_PWM3_A	GPIO_EMC_21	ALT1
	FLEXPWM3_PWM3_B	GPIO_EMC_22	ALT1
FLEXPWM4	FLEXPWM4_PWM0_A	GPIO_AD_B1_08	ALT1
		GPIO_EMC_00	ALT1
	FLEXPWM4_PWM0_B	GPIO_EMC_01	ALT1
	FLEXPWM4_PWM1_A	GPIO_AD_B1_09	ALT1
		GPIO_EMC_02	ALT1
	FLEXPWM4_PWM1_B	GPIO_EMC_03	ALT1
	FLEXPWM4_PWM2_A	GPIO_B1_14	ALT1
		GPIO_EMC_04	ALT1
	FLEXPWM4_PWM2_B	GPIO_EMC_05	ALT1
	FLEXPWM4_PWM3_A	GPIO_B1_15	ALT1
		GPIO_EMC_17	ALT1
	FLEXPWM4_PWM3_B	GPIO_EMC_18	ALT1
FLEXCAN1	FLEXCAN1_TX	GPIO_AD_B1_08	ALT2
		GPIO_B0_02	ALT2
		GPIO_EMC_17	ALT3
		GPIO_SD_B1_02	ALT4
	FLEXCAN1_RX	GPIO_AD_B1_09	ALT2
		GPIO_B0_03	ALT2
		GPIO_EMC_18	ALT3

Table continues on the next page...

Table 8-1. Muxing Options (continued)

FLEXCAN2	FLEXCAN2_TX	GPIO_SD_B1_03	ALT4
		GPIO_AD_B0_02	ALT0
		GPIO_EMC_09	ALT3
		GPIO_B1_08	ALT6
		GPIO_AD_B0_14	ALT6
	FLEXCAN2_RX	GPIO_AD_B0_03	ALT0
		GPIO_EMC_10	ALT3
		GPIO_AD_B0_15	ALT6
		GPIO_B1_09	ALT6
FLEXCAN3	FLEXCAN3_RX	GPIO_AD_B0_11	ALT8
		GPIO_AD_B0_15	ALT8
		GPIO_EMC_37	ALT9
	FLEXCAN3_TX	GPIO_AD_B0_10	ALT8
		GPIO_AD_B0_14	ALT8
		GPIO_EMC_36	ALT9
FLEXSPI A	FLEXSPI_A_DATA0	GPIO_AD_B1_13	ALT0
		GPIO_SD_B1_08	ALT1
	FLEXSPI_A_DATA1	GPIO_AD_B1_12	ALT0
		GPIO_SD_B1_09	ALT1
	FLEXSPI_A_DATA2	GPIO_AD_B1_11	ALT0
		GPIO_SD_B1_10	ALT1
	FLEXSPI_A_DATA3	GPIO_AD_B1_10	ALT0
		GPIO_SD_B1_11	ALT1
	FLEXSPI_A_DQS	GPIO_AD_B1_09	ALT0
		GPIO_SD_B1_05	ALT1
	FLEXSPI_A_SCLK	GPIO_AD_B1_14	ALT0
		GPIO_SD_B1_07	ALT1
	FLEXSPI_A_SS0_B	GPIO_AD_B1_15	ALT0
		GPIO_SD_B1_06	ALT1
	FLEXSPI_A_SS1_B	GPIO_AD_B1_08	ALT0
		GPIO_SD_B0_00	ALT6
		GPIO_SD_B1_04	ALT4
FLEXSPI B	FLEXSPI_B_DATA0	GPIO_AD_B1_07	ALT0
		GPIO_SD_B1_03	ALT1
	FLEXSPI_B_DATA1	GPIO_AD_B1_06	ALT0
		GPIO_SD_B1_02	ALT1
	FLEXSPI_B_DATA2	GPIO_AD_B1_05	ALT0
		GPIO_SD_B1_01	ALT1
	FLEXSPI_B_DATA3	GPIO_AD_B1_04	ALT0
		GPIO_SD_B1_00	ALT1

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	FLEXSPI_B_DQS	GPIO_SD_B0_05	ALT4
	FLEXSPI_B_SCLK	GPIO_SD_B1_04	ALT1
	FLEXSPI_B_SS0_B	GPIO_SD_B0_04	ALT4
		GPIO_SD_B1_05	ALT4
	FLEXSPI_B_SS1_B	GPIO_SD_B0_01	ALT6
FLEXSPI2_A_SS1_B	GPIO_EMC_22	ALT8	
FLEXSPI2_B_DQS	GPIO_EMC_11	ALT8	
FLEXSPI2_B_SS0_B	GPIO_EMC_10	ALT8	
FLEXSPI2_B_SS1_B	GPIO_EMC_09	ALT8	
GPIO1	GPIO1_IO00	GPIO_AD_B0_00	ALT5
	GPIO1_IO01	GPIO_AD_B0_01	ALT5
	GPIO1_IO02	GPIO_AD_B0_02	ALT5
	GPIO1_IO03	GPIO_AD_B0_03	ALT5
	GPIO1_IO04	GPIO_AD_B0_04	ALT5
	GPIO1_IO05	GPIO_AD_B0_05	ALT5
	GPIO1_IO06	GPIO_AD_B0_06	ALT5
	GPIO1_IO07	GPIO_AD_B0_07	ALT5
	GPIO1_IO08	GPIO_AD_B0_08	ALT5
	GPIO1_IO09	GPIO_AD_B0_09	ALT5
	GPIO1_IO10	GPIO_AD_B0_10	ALT5
	GPIO1_IO11	GPIO_AD_B0_11	ALT5
	GPIO1_IO12	GPIO_AD_B0_12	ALT5
	GPIO1_IO13	GPIO_AD_B0_13	ALT5
	GPIO1_IO14	GPIO_AD_B0_14	ALT5
	GPIO1_IO15	GPIO_AD_B0_15	ALT5
	GPIO1_IO16	GPIO_AD_B1_00	ALT5
	GPIO1_IO17	GPIO_AD_B1_01	ALT5
	GPIO1_IO18	GPIO_AD_B1_02	ALT5
	GPIO1_IO19	GPIO_AD_B1_03	ALT5
	GPIO1_IO20	GPIO_AD_B1_04	ALT5
	GPIO1_IO21	GPIO_AD_B1_05	ALT5
	GPIO1_IO22	GPIO_AD_B1_06	ALT5
	GPIO1_IO23	GPIO_AD_B1_07	ALT5
	GPIO1_IO24	GPIO_AD_B1_08	ALT5
	GPIO1_IO25	GPIO_AD_B1_09	ALT5
	GPIO1_IO26	GPIO_AD_B1_10	ALT5
	GPIO1_IO27	GPIO_AD_B1_11	ALT5
	GPIO1_IO28	GPIO_AD_B1_12	ALT5
	GPIO1_IO29	GPIO_AD_B1_13	ALT5
	GPIO1_IO30	GPIO_AD_B1_14	ALT5

Table continues on the next page...

Table 8-1. Muxing Options (continued)

GPIO2	GPIO1_IO31	GPIO_AD_B1_15	ALT5
	GPIO2_IO00	GPIO_B0_00	ALT5
	GPIO2_IO01	GPIO_B0_01	ALT5
	GPIO2_IO02	GPIO_B0_02	ALT5
	GPIO2_IO03	GPIO_B0_03	ALT5
	GPIO2_IO04	GPIO_B0_04	ALT5
	GPIO2_IO05	GPIO_B0_05	ALT5
	GPIO2_IO06	GPIO_B0_06	ALT5
	GPIO2_IO07	GPIO_B0_07	ALT5
	GPIO2_IO08	GPIO_B0_08	ALT5
	GPIO2_IO09	GPIO_B0_09	ALT5
	GPIO2_IO10	GPIO_B0_10	ALT5
	GPIO2_IO11	GPIO_B0_11	ALT5
	GPIO2_IO12	GPIO_B0_12	ALT5
	GPIO2_IO13	GPIO_B0_13	ALT5
	GPIO2_IO14	GPIO_B0_14	ALT5
	GPIO2_IO15	GPIO_B0_15	ALT5
	GPIO2_IO16	GPIO_B1_00	ALT5
	GPIO2_IO17	GPIO_B1_01	ALT5
	GPIO2_IO18	GPIO_B1_02	ALT5
	GPIO2_IO19	GPIO_B1_03	ALT5
	GPIO2_IO20	GPIO_B1_04	ALT5
	GPIO2_IO21	GPIO_B1_05	ALT5
	GPIO2_IO22	GPIO_B1_06	ALT5
	GPIO2_IO23	GPIO_B1_07	ALT5
	GPIO2_IO24	GPIO_B1_08	ALT5
	GPIO2_IO25	GPIO_B1_09	ALT5
	GPIO2_IO26	GPIO_B1_10	ALT5
	GPIO2_IO27	GPIO_B1_11	ALT5
	GPIO2_IO28	GPIO_B1_12	ALT5
	GPIO2_IO29	GPIO_B1_13	ALT5
	GPIO2_IO30	GPIO_B1_14	ALT5
	GPIO2_IO31	GPIO_B1_15	ALT5
GPIO3	GPIO3_IO00	GPIO_SD_B1_00	ALT5
	GPIO3_IO01	GPIO_SD_B1_01	ALT5
	GPIO3_IO02	GPIO_SD_B1_02	ALT5
	GPIO3_IO03	GPIO_SD_B1_03	ALT5
	GPIO3_IO04	GPIO_SD_B1_04	ALT5
	GPIO3_IO05	GPIO_SD_B1_05	ALT5
	GPIO3_IO06	GPIO_SD_B1_06	ALT5

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	GPIO3_IO07	GPIO_SD_B1_07	ALT5
	GPIO3_IO08	GPIO_SD_B1_08	ALT5
	GPIO3_IO09	GPIO_SD_B1_09	ALT5
	GPIO3_IO10	GPIO_SD_B1_10	ALT5
	GPIO3_IO11	GPIO_SD_B1_11	ALT5
	GPIO3_IO12	GPIO_SD_B0_00	ALT5
	GPIO3_IO13	GPIO_SD_B0_01	ALT5
	GPIO3_IO14	GPIO_SD_B0_02	ALT5
	GPIO3_IO15	GPIO_SD_B0_03	ALT5
	GPIO3_IO16	GPIO_SD_B0_04	ALT5
	GPIO3_IO17	GPIO_SD_B0_05	ALT5
	GPIO3_IO18	GPIO_EMC_32	ALT5
	GPIO3_IO19	GPIO_EMC_33	ALT5
	GPIO3_IO20	GPIO_EMC_34	ALT5
	GPIO3_IO21	GPIO_EMC_35	ALT5
	GPIO3_IO22	GPIO_EMC_36	ALT5
	GPIO3_IO23	GPIO_EMC_37	ALT5
	GPIO3_IO24	GPIO_EMC_38	ALT5
	GPIO3_IO25	GPIO_EMC_39	ALT5
	GPIO3_IO26	GPIO_EMC_40	ALT5
	GPIO3_IO27	GPIO_EMC_41	ALT5
GPIO4	GPIO4_IO0	GPIO_EMC_00	ALT5
	GPIO4_IO1	GPIO_EMC_01	ALT5
	GPIO4_IO2	GPIO_EMC_02	ALT5
	GPIO4_IO3	GPIO_EMC_03	ALT5
	GPIO4_IO4	GPIO_EMC_04	ALT5
	GPIO4_IO5	GPIO_EMC_05	ALT5
	GPIO4_IO6	GPIO_EMC_06	ALT5
	GPIO4_IO7	GPIO_EMC_07	ALT5
	GPIO4_IO8	GPIO_EMC_08	ALT5
	GPIO4_IO9	GPIO_EMC_09	ALT5
	GPIO4_IO10	GPIO_EMC_10	ALT5
	GPIO4_IO11	GPIO_EMC_11	ALT5
	GPIO4_IO12	GPIO_EMC_12	ALT5
	GPIO4_IO13	GPIO_EMC_13	ALT5
	GPIO4_IO14	GPIO_EMC_14	ALT5
	GPIO4_IO15	GPIO_EMC_15	ALT5
	GPIO4_IO16	GPIO_EMC_16	ALT5
	GPIO4_IO17	GPIO_EMC_17	ALT5
	GPIO4_IO18	GPIO_EMC_18	ALT5

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	GPIO4_IO19	GPIO_EMC_19	ALT5
	GPIO4_IO20	GPIO_EMC_20	ALT5
	GPIO4_IO21	GPIO_EMC_21	ALT5
	GPIO4_IO22	GPIO_EMC_22	ALT5
	GPIO4_IO23	GPIO_EMC_23	ALT5
	GPIO4_IO24	GPIO_EMC_24	ALT5
	GPIO4_IO25	GPIO_EMC_25	ALT5
	GPIO4_IO26	GPIO_EMC_26	ALT5
	GPIO4_IO27	GPIO_EMC_27	ALT5
	GPIO4_IO28	GPIO_EMC_28	ALT5
	GPIO4_IO29	GPIO_EMC_29	ALT5
	GPIO4_IO30	GPIO_EMC_30	ALT5
	GPIO4_IO31	GPIO_EMC_31	ALT5
GPIO5	GPIO5_IO00	WAKEUP	ALT5
	GPIO5_IO01	PMIC_ON_REQ	ALT5
	GPIO5_IO02	PMIC_STBY_REQ	ALT5
GPT1	GPT1_CLK	GPIO_AD_B0_13	ALT1
		GPIO_B1_04	ALT8
	GPT1_CAPTURE1	GPIO_EMC_24	ALT4
		GPIO_B1_05	ALT8
	GPT1_CAPTURE2	GPIO_EMC_23	ALT4
		GPIO_B1_06	ALT8
	GPT1_COMPARE1	GPIO_EMC_35	ALT2
		GPIO_B1_07	ALT8
	GPT1_COMPARE2	GPIO_EMC_36	ALT2
		GPIO_B1_08	ALT8
GPT2	GPT2_CLK	GPIO_AD_B0_09	ALT7
		GPIO_AD_B1_02	ALT8
	GPT2_CAPTURE1	GPIO_EMC_41	ALT1
		GPIO_AD_B1_03	ALT8
	GPT2_CAPTURE2	GPIO_EMC_40	ALT1
		GPIO_AD_B1_04	ALT8
	GPT2_COMPARE1	GPIO_AD_B0_06	ALT1
		GPIO_AD_B1_05	ALT8
	GPT2_COMPARE2	GPIO_AD_B0_07	ALT1
		GPIO_AD_B1_06	ALT8
	GPT2_COMPARE3	GPIO_AD_B0_08	ALT1
		GPIO_AD_B1_07	ALT8

Table continues on the next page...

Table 8-1. Muxing Options (continued)

JTAG	JTAG_TMS	GPIO_AD_B0_06	ALT0
	JTAG_TCK	GPIO_AD_B0_07	ALT0
	JTAG_MOD	GPIO_AD_B0_08	ALT0
	JTAG_TDI	GPIO_AD_B0_09	ALT0
	JTAG_TDO	GPIO_AD_B0_10	ALT0
	JTAG_TRSTB	GPIO_AD_B0_11	ALT0
	JTAG_DE_B	GPIO_EMC_01	ALT7
SWD	SWD_DIO	GPIO_AD_B0_06	ALT0
	SWD_CLK	GPIO_AD_B0_07	ALT0
TPIU	ARM_TRACE0	GPIO_B0_04	ALT3
	ARM_TRACE1	GPIO_B0_05	ALT3
	ARM_TRACE2	GPIO_B0_06	ALT3
	ARM_TRACE3	GPIO_B0_07	ALT3
	ARM_TRACE_CLK	GPIO_B0_12	ALT2
	ARM_TRACE_SWO	GPIO_B0_13	ALT2
KPP	KPP_ROW0	GPIO_AD_B1_14	ALT7
	KPP_COL0	GPIO_AD_B1_15	ALT7
	KPP_ROW1	GPIO_AD_B1_12	ALT7
	KPP_COL1	GPIO_AD_B1_13	ALT7
	KPP_ROW2	GPIO_AD_B1_10	ALT7
	KPP_COL2	GPIO_AD_B1_11	ALT7
	KPP_ROW3	GPIO_AD_B1_08	ALT7
	KPP_COL3	GPIO_AD_B1_09	ALT7
	KPP_ROW4	GPIO_AD_B1_06	ALT7
	KPP_COL4	GPIO_AD_B1_07	ALT7
	KPP_ROW5	GPIO_AD_B1_04	ALT7
	KPP_COL5	GPIO_AD_B1_05	ALT7
	KPP_ROW6	GPIO_AD_B1_02	ALT7
	KPP_COL6	GPIO_AD_B1_03	ALT7
	KPP_ROW7	GPIO_AD_B1_00	ALT7
	KPP_COL7	GPIO_AD_B1_01	ALT7
LCD	LCD_CLK	GPIO_B0_00	ALT0
	LCD_ENABLE	GPIO_B0_01	ALT0
	LCD_HSYNC	GPIO_B0_02	ALT0
	LCD_VSYNC	GPIO_B0_03	ALT0
	LCD_DATA00	GPIO_B0_04	ALT0
	LCD_DATA01	GPIO_B0_05	ALT0
	LCD_DATA02	GPIO_B0_06	ALT0
	LCD_DATA03	GPIO_B0_07	ALT0
	LCD_DATA04	GPIO_B0_08	ALT0

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	LCD_DATA05	GPIO_B0_09	ALT0
	LCD_DATA06	GPIO_B0_10	ALT0
	LCD_DATA07	GPIO_B0_11	ALT0
	LCD_DATA08	GPIO_B0_12	ALT0
	LCD_DATA09	GPIO_B0_13	ALT0
	LCD_DATA10	GPIO_B0_14	ALT0
	LCD_DATA11	GPIO_B0_15	ALT0
	LCD_DATA12	GPIO_B1_00	ALT0
	LCD_DATA13	GPIO_B1_01	ALT0
	LCD_DATA14	GPIO_B1_02	ALT0
	LCD_DATA15	GPIO_B1_03	ALT0
	LCD_DATA16	GPIO_B1_04	ALT0
	LCD_DATA17	GPIO_B1_05	ALT0
	LCD_DATA18	GPIO_B1_06	ALT0
	LCD_DATA19	GPIO_B1_07	ALT0
	LCD_DATA20	GPIO_B1_08	ALT0
	LCD_DATA21	GPIO_B1_09	ALT0
	LCD_DATA22	GPIO_B1_10	ALT0
	LCD_DATA23	GPIO_B1_11	ALT0
LPI2C1	LPI2C1_SCL	GPIO_AD_B1_00	ALT3
		GPIO_SD_B1_04	ALT2
	LPI2C1_SDA	GPIO_SD_B1_05	ALT2
		GPIO_AD_B1_01	ALT3
	LPI2C1_SCLS	GPIO_AD_B0_00	ALT4
	LPI2C1_SDAS	GPIO_AD_B0_01	ALT4
LPI2C2	LPI2C2_SCL	GPIO_AD_B0_02	ALT6
	LPI2C2_SDA	GPIO_B0_04	ALT2
		GPIO_SD_B1_11	ALT3
		GPIO_B0_05	ALT2
		GPIO_SD_B1_10	ALT3
LPI2C3	LPI2C3_SCL	GPIO_AD_B1_07	ALT1
		GPIO_EM_C22	ALT2
		GPIO_SD_B0_00	ALT2
	LPI2C3_SDA	GPIO_AD_B1_06	ALT1
		GPIO_EM_C21	ALT2
		GPIO_SD_B0_01	ALT2
LPI2C4	LPI2C4_SCL	GPIO_AD_B0_12	ALT0
		GPIO_EM_C12	ALT2
	LPI2C4_SDA	GPIO_EM_C11	ALT2
		GPIO_AD_B0_13	ALT0

Table continues on the next page...

Table 8-1. Muxing Options (continued)

LPSPI1	LPSPI1_PCS0	GPIO_EMC_30	ALT3
		GPIO_SD_B0_01	ALT4
	LPSPI1_PCS1	GPIO_EMC_31	ALT3
	LPSPI1_PCS2	GPIO_EMC_40	ALT2
	LPSPI1_PCS3	GPIO_EMC_41	ALT2
	LPSPI1_SCK	GPIO_SD_B0_00	ALT4
		GPIO_EMC_27	ALT3
	LPSPI1_SOUT	GPIO_SD_B0_02	ALT4
		GPIO_EMC_28	ALT3
	LPSPI1_SIN	GPIO_EMC_29	ALT3
		GPIO_SD_B0_03	ALT4
LPSPI2	LPSPI2_PCS0	GPIO_SD_B1_06	ALT4
		GPIO_EMC_01	ALT2
	LPSPI2_PCS1	GPIO_EMC_14	ALT4
	LPSPI2_PCS2	GPIO_SD_B1_10	ALT4
	LPSPI2_PCS3	GPIO_SD_B1_11	ALT4
	LPSPI2_SCK	GPIO_SD_B1_07	ALT4
		GPIO_EMC_00	ALT2
	LPSPI2_SOUT	GPIO_EMC_02	ALT2
		GPIO_SD_B1_08	ALT4
	LPSPI2_SIN	GPIO_SD_B1_09	ALT4
		GPIO_EMC_03	ALT2
LPSPI3	LPSPI3_PCS0	GPIO_AD_B1_12	ALT2
		GPIO_AD_B0_03	ALT7
	LPSPI3_PCS1	GPIO_AD_B0_04	ALT7
	LPSPI3_PCS2	GPIO_AD_B0_05	ALT7
	LPSPI3_PCS3	GPIO_AD_B0_06	ALT7
	LPSPI3_SCK	GPIO_AD_B1_15	ALT2
		GPIO_AD_B0_00	ALT7
	LPSPI3_SOUT	GPIO_AD_B0_01	ALT7
		GPIO_AD_B1_14	ALT2
	LPSPI3_SIN	GPIO_AD_B1_13	ALT2
		GPIO_AD_B0_02	ALT7
LPSPI4	LPSPI4_PCS0	GPIO_B1_04	ALT1
		GPIO_B0_00	ALT3
	LPSPI4_PCS1	GPIO_B1_03	ALT2
	LPSPI4_PCS2	GPIO_B1_02	ALT2
	LPSPI4_PCS3	GPIO_B1_11	ALT6
	LPSPI4_SCK	GPIO_B1_07	ALT1
		GPIO_B0_03	ALT3

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	LPSPI4_SOUT	GPIO_B1_06	ALT1
		GPIO_B0_02	ALT3
	LPSPI4_SIN	GPIO_B1_05	ALT1
		GPIO_B0_01	ALT3
LPUART1	LPUART1_TXD	GPIO_AD_B0_12	ALT2
	LPUART1_RXD	GPIO_AD_B0_13	ALT2
	LPUART1_CTS_B	GPIO_AD_B0_14	ALT2
	LPUART1_RTS_B	GPIO_AD_B0_15	ALT2
LPUART2	LPUART2_TXD	GPIO_SD_B1_11	ALT2
		GPIO_AD_B1_02	ALT2
	LPUART2_RXD	GPIO_AD_B1_03	ALT2
		GPIO_SD_B1_10	ALT2
	LPUART2_CTS_B	GPIO_AD_B1_00	ALT2
	LPUART2_RTS_B	GPIO_AD_B1_01	ALT2
LPUART3	LPUART3_TXD	GPIO_AD_B1_06	ALT2
		GPIO_B0_08	ALT3
		GPIO_EMC_13	ALT2
	LPUART3_RXD	GPIO_EMC_14	ALT2
		GPIO_B0_09	ALT3
		GPIO_AD_B1_07	ALT2
	LPUART3_CTS_B	GPIO_EMC_15	ALT2
		GPIO_AD_B1_04	ALT2
	LPUART3_RTS_B	GPIO_AD_B1_05	ALT2
		GPIO_EMC_16	ALT2
LPUART4	LPUART4_TXD	GPIO_SD_B1_00	ALT4
		GPIO_B1_00	ALT2
		GPIO_EMC_19	ALT2
	LPUART4_RXD	GPIO_SD_B1_01	ALT4
		GPIO_EMC_20	ALT2
		GPIO_B1_01	ALT2
	LPUART4_CTS_B	GPIO_EMC_17	ALT2
	LPUART4_RTS_B	GPIO_EMC_18	ALT2
LPUART5	LPUART5_TXD	GPIO_EMC_23	ALT2
		GPIO_B1_12	ALT1
	LPUART5_RXD	GPIO_B1_13	ALT1
		GPIO_EMC_24	ALT2
	LPUART5_CTS_B	GPIO_EMC_28	ALT2
	LPUART5_RTS_B	GPIO_EMC_27	ALT2
LPUART6	LPUART6_TXD	GPIO_EMC_25	ALT2
		GPIO_AD_B0_02	ALT2

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	LPUART6_RXD	GPIO_AD_B0_03	ALT2
		GPIO_EMC_26	ALT2
	LPUART6_CTS_B	GPIO_EMC_30	ALT2
	LPUART6_RTS_B	GPIO_EMC_29	ALT2
LPUART7	LPUART7_TXD	GPIO_SD_B1_08	ALT2
		GPIO_EMC_31	ALT2
	LPUART7_RXD	GPIO_EMC_32	ALT2
		GPIO_SD_B1_09	ALT2
	LPUART7_CTS_B	GPIO_SD_B1_06	ALT2
	LPUART7_RTS_B	GPIO_SD_B1_07	ALT2
LPUART8	LPUART8_TXD	GPIO_EMC_38	ALT2
		GPIO_AD_B1_10	ALT2
		GPIO_SD_B0_04	ALT2
	LPUART8_RXD	GPIO_SD_B0_05	ALT2
		GPIO_AD_B1_11	ALT2
		GPIO_EMC_39	ALT2
	LPUART8_CTS_B	GPIO_SD_B0_02	ALT2
	LPUART8_RTS_B	GPIO_SD_B0_03	ALT2
MQS	MQS_RIGHT	GPIO_AD_B0_04	ALT1
		GPIO_B0_00	ALT2
		GPIO_EMC_13	ALT3
	MQS_LEFT	GPIO_EMC_14	ALT3
		GPIO_B0_01	ALT2
		GPIO_AD_B0_05	ALT1
PIT	PIT_TRIGGER0	GPIO_AD_B0_04	ALT6
TMR1 (QUAD TIMER)	TMR1_TIMER0	GPIO_B0_00	ALT1
	TMR1_TIMER1	GPIO_B0_01	ALT1
	TMR1_TIMER2	GPIO_B0_02	ALT1
	TMR1_TIMER3	GPIO_B1_08	ALT1
TMR2 (QUAD TIMER)	TMR2_TIMER0	GPIO_EMC_19	ALT4
		GPIO_B0_03	ALT1
	TMR2_TIMER1	GPIO_EMC_20	ALT4
		GPIO_B0_04	ALT1
	TMR2_TIMER2	GPIO_EMC_21	ALT4
		GPIO_B0_05	ALT1
	TMR2_TIMER3	GPIO_EMC_22	ALT4
		GPIO_B1_09	ALT1
TMR3 (QUAD TIMER)	TMR3_TIMER0	GPIO_AD_B1_00	ALT1
		GPIO_EMC_15	ALT4
		GPIO_B0_06	ALT1

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	TMR3_TIMER1	GPIO_AD_B1_01	ALT1
		GPIO_EMC_16	ALT4
		GPIO_B0_07	ALT1
	TMR3_TIMER2	GPIO_EMC_17	ALT4
		GPIO_AD_B1_02	ALT1
		GPIO_B0_08	ALT1
	TMR3_TIMER3	GPIO_B1_10	ALT1
		GPIO_AD_B1_03	ALT1
		GPIO_EMC_18	ALT4
TMR4 (QUAD TIMER)	TMR4_TIMER0	GPIO_B0_09	ALT1
	TMR4_TIMER1	GPIO_B0_10	ALT1
	TMR4_TIMER2	GPIO_B0_11	ALT1
	TMR4_TIMER3	GPIO_B1_11	ALT1
SAI1	SAI1_TX_DATA0	GPIO_AD_B1_13	ALT3
		GPIO_B1_01	ALT3
		GPIO_SD_B1_07	ALT3
	SAI1_TX_DATA1 and SAI1_RX_DATA3	GPIO_B0_12	ALT3
		GPIO_SD_B1_02	ALT3
	SAI1_TX_DATA2 and SAI1_RX_DATA2	GPIO_B0_11	ALT3
		GPIO_SD_B1_01	ALT3
	SAI1_TX_DATA3 and SAI1_RX_DATA1	GPIO_B0_10	ALT3
		GPIO_SD_B1_00	ALT3
	SAI1_TX_BCLK	GPIO_SD_B1_08	ALT3
		GPIO_B1_02	ALT3
		GPIO_AD_B1_14	ALT3
	SAI1_TX_SYNC	GPIO_AD_B1_15	ALT3
		GPIO_B1_03	ALT3
		GPIO_SD_B1_09	ALT3
	SAI1_RX_DATA0	GPIO_B1_00	ALT3
		GPIO_AD_B1_12	ALT3
		GPIO_SD_B1_06	ALT3
	SAI1_RX_BCLK	GPIO_AD_B1_11	ALT3
		GPIO_B0_15	ALT3
		GPIO_SD_B1_05	ALT3
	SAI1_RX_SYNC	GPIO_AD_B1_10	ALT3
		GPIO_SD_B1_04	ALT3
		GPIO_B0_14	ALT3
	SAI1_MCLK	GPIO_B0_13	ALT3
		GPIO_SD_B1_03	ALT3
		GPIO_AD_B1_09	ALT3

Table continues on the next page...

Table 8-1. Muxing Options (continued)

SAI2	SAI2_TX_DATA	GPIO_EMC_04	ALT2
		GPIO_AD_B0_09	ALT3
	SAI2_TX_BCLK	GPIO_AD_B0_05	ALT3
		GPIO_EMC_06	ALT2
	SAI2_TX_SYNC	GPIO_AD_B0_04	ALT3
		GPIO_EMC_05	ALT2
	SAI2_RX_DATA	GPIO_AD_B0_08	ALT3
		GPIO_EMC_08	ALT2
	SAI2_RX_BCLK	GPIO_EMC_10	ALT2
		GPIO_AD_B0_06	ALT3
	SAI2_RX_SYNC	GPIO_EMC_09	ALT2
		GPIO_AD_B0_07	ALT3
SAI2_MCLK	GPIO_EMC_07	ALT2	
	GPIO_AD_B0_10	ALT3	
SAI3	SAI3_TX_DATA	GPIO_EMC_36	ALT3
		GPIO_SD_B1_01	ALT8
	SAI3_TX_BCLK	GPIO_EMC_38	ALT3
		GPIO_SD_B1_03	ALT8
	SAI3_TX_SYNC	GPIO_EMC_39	ALT3
		GPIO_SD_B1_02	ALT8
	SAI3_RX_DATA	GPIO_EMC_33	ALT3
		GPIO_SD_B1_00	ALT8
	SAI3_RX_BCLK	GPIO_EMC_35	ALT3
		GPIO_SD_B1_06	ALT8
	SAI3_RX_SYNC	GPIO_EMC_34	ALT3
		GPIO_SD_B1_05	ALT8
SAI3_MCLK	GPIO_EMC_37	ALT3	
	GPIO_SD_B1_04	ALT8	
SEMC	SEMC_DATA00	GPIO_EMC_00	ALT0
	SEMC_DATA01	GPIO_EMC_01	ALT0
	SEMC_DATA02	GPIO_EMC_02	ALT0
	SEMC_DATA03	GPIO_EMC_03	ALT0
	SEMC_DATA04	GPIO_EMC_04	ALT0
	SEMC_DATA05	GPIO_EMC_05	ALT0
	SEMC_DATA06	GPIO_EMC_06	ALT0
	SEMC_DATA07	GPIO_EMC_07	ALT0
	SEMC_DATA08	GPIO_EMC_30	ALT0
	SEMC_DATA09	GPIO_EMC_31	ALT0
	SEMC_DATA10	GPIO_EMC_32	ALT0
	SEMC_DATA11	GPIO_EMC_33	ALT0

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	SEMC_DATA12	GPIO_EMC_34	ALT0
	SEMC_DATA13	GPIO_EMC_35	ALT0
	SEMC_DATA14	GPIO_EMC_36	ALT0
	SEMC_DATA15	GPIO_EMC_37	ALT0
	SEMC_ADDR00	GPIO_EMC_09	ALT0
	SEMC_ADDR01	GPIO_EMC_10	ALT0
	SEMC_ADDR02	GPIO_EMC_11	ALT0
	SEMC_ADDR03	GPIO_EMC_12	ALT0
	SEMC_ADDR04	GPIO_EMC_13	ALT0
	SEMC_ADDR05	GPIO_EMC_14	ALT0
	SEMC_ADDR06	GPIO_EMC_15	ALT0
	SEMC_ADDR07	GPIO_EMC_16	ALT0
	SEMC_ADDR08	GPIO_EMC_17	ALT0
	SEMC_ADDR09	GPIO_EMC_18	ALT0
	SEMC_ADDR10	GPIO_EMC_23	ALT0
	SEMC_ADDR11	GPIO_EMC_19	ALT0
	SEMC_ADDR12	GPIO_EMC_20	ALT0
	SEMC_DM0	GPIO_EMC_08	ALT0
	SEMC_DM1	GPIO_EMC_38	ALT0
	SEMC_BA0	GPIO_EMC_21	ALT0
	SEMC_BA1	GPIO_EMC_22	ALT0
	SEMC_CAS	GPIO_EMC_24	ALT0
	SEMC_RAS	GPIO_EMC_25	ALT0
	SEMC_CLK	GPIO_EMC_26	ALT0
	SEMC_CKE	GPIO_EMC_27	ALT0
	SEMC_WE	GPIO_EMC_28	ALT0
	SEMC_CS0	GPIO_EMC_29	ALT0
	SEMC_DQS	GPIO_EMC_39	ALT0
	SEMC_RDY	GPIO_EMC_40	ALT0
	SEMC_CSX0	GPIO_EMC_41	ALT0
	SEMC_CSX1	GPIO_SD_B1_07	ALT0
	SEMC_CSX1	GPIO_B0_00	ALT6
	SEMC_CSX2	GPIO_SD_B1_08	ALT6
	SEMC_CSX2	GPIO_B0_01	ALT6
	SEMC_CSX3	GPIO_B0_02	ALT6
SNVS	SNVS_PMIC_ON_REQ	PMIC_ON_REQ	ALT0
	SNVS_VIO_5_CTL	GPIO_EMC_18	ALT6
	SNVS_VIO_5_B	GPIO_EMC_19	ALT6
SPDIF	SPDIF_OUT	GPIO_EMC_15	ALT3
		GPIO_AD_B1_05	ALT3

Table continues on the next page...

Table 8-1. Muxing Options (continued)

		GPIO_AD_B1_02	ALT3
	SPDIF_IN	GPIO_EMC_16	ALT3
		GPIO_AD_B1_03	ALT3
	SPDIF_SR_CLK	GPIO_AD_B1_04	ALT3
	SPDIF_LOCK	GPIO_AD_B1_06	ALT3
	SPDIF_EXT_CLK	GPIO_AD_B1_07	ALT3
SRC	SRC_POR_B	POR_B	ALT0 (No Muxing)
	SRC_RESET_B	ONOFF	ALT0 (No Muxing)
	SRC_BOOT_MODE0	GPIO_AD_B0_04	ALT0
	SRC_BOOT_MODE1	GPIO_AD_B0_05	ALT0
	SRC_BT_CFG00	GPIO_B0_04	ALT6
	SRC_BT_CFG01	GPIO_B0_05	ALT6
	SRC_BT_CFG02	GPIO_B0_06	ALT6
	SRC_BT_CFG03	GPIO_B0_07	ALT6
	SRC_BT_CFG04	GPIO_B0_08	ALT6
	SRC_BT_CFG05	GPIO_B0_09	ALT6
	SRC_BT_CFG06	GPIO_B0_10	ALT6
	SRC_BT_CFG07	GPIO_B0_11	ALT6
	SRC_BT_CFG08	GPIO_B0_12	ALT6
	SRC_BT_CFG09	GPIO_B0_13	ALT6
	SRC_BT_CFG10	GPIO_B0_14	ALT6
	SRC_BT_CFG11	GPIO_B0_15	ALT6
USB	USB_OTG1_PWR	GPIO_AD_B0_02	ALT3
		GPIO_AD_B1_01	ALT0
	USB_OTG1_OC	GPIO_AD_B0_03	ALT3
		GPIO_AD_B1_03	ALT0
	USB_OTG1_ID	GPIO_AD_B0_01	ALT3
		GPIO_AD_B1_02	ALT0
	USB_OTG2_PWR	GPIO_EMC_41	ALT3
		GPIO_AD_B0_15	ALT0
	USB_OTG2_OC	GPIO_EMC_40	ALT3
		GPIO_AD_B0_14	ALT0
USB PHY 1	USB_PHY1_TSTI_TX_LS_M ODE	GPIO_AD_B0_00	ALT3
		GPIO_AD_B1_00	ALT0
	USB_PHY1_TSTI_TX_HS_M ODE	GPIO_EMC_01	ALT6
	USB_PHY1_TSTI_TX_DN	GPIO_EMC_02	ALT6
	USB_PHY1_TSTO_RX_SQU ELCH	GPIO_EMC_03	ALT6

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	USB_PHY1_TSTO_RX_DISC ON_DET	GPIO_EMC_04	ALT6
	USB_PHY1_TSTO_RX_HS_ RXD	GPIO_EMC_05	ALT6
	USB_PHY1_TSTO_RX_FS_ RXD	GPIO_EMC_07	ALT6
	USB_PHY1_TSTI_TX_DP	GPIO_EMC_08	ALT6
	USB_PHY1_TSTI_TX_EN	GPIO_EMC_09	ALT6
	USB_PHY1_TSTI_TX_HIZ	GPIO_EMC_10	ALT6
	USB_PHY1_TSTO_PLL_CLK 20DIV	GPIO_EMC_12	ALT6
USB PHY 2	USB_PHY2_TSTO_RX_FS_ RXD	GPIO_EMC_06	ALT6
	USB_PHY2_TSTO_RX_HS_ RXD	GPIO_EMC_11	ALT6
	USB_PHY2_TSTO_PLL_CLK 20DIV	GPIO_EMC_13	ALT6
	USB_PHY2_TSTO_RX_SQU ELCH	GPIO_EMC_14	ALT6
	USB_PHY2_TSTO_RX_DISC ON_DET	GPIO_EMC_15	ALT6
USDHC1	USDHC1_CMD	GPIO_SD_B0_00	ALT0
	USDHC1_CLK	GPIO_SD_B0_01	ALT0
	USDHC1_DATA0	GPIO_SD_B0_02	ALT0
	USDHC1_DATA1	GPIO_SD_B0_03	ALT0
	USDHC1_DATA2	GPIO_SD_B0_04	ALT0
	USDHC1_DATA3	GPIO_SD_B0_05	ALT0
	USDHC1_RESET_B	GPIO_AD_B0_00	ALT6
		GPIO_B1_15	ALT6
		GPIO_EMC_33	ALT2
	USDHC1_VSELECT	GPIO_EMC_41	ALT6
		GPIO_AD_B1_01	ALT6
		GPIO_B1_14	ALT6
		GPIO_EMC_34	ALT2
	USDHC1_WP	GPIO_EMC_36	ALT6
		GPIO_AD_B1_00	ALT6
		GPIO_B1_13	ALT6
		GPIO_EMC_12	ALT3
	USDHC1_CD_B	GPIO_AD_B1_02	ALT6
		GPIO_EMC_35	ALT6
		GPIO_B1_12	ALT6
USDHC2	USDHC2_CMD	GPIO_AD_B1_08	ALT6
		GPIO_SD_B1_05	ALT0

Table continues on the next page...

Table 8-1. Muxing Options (continued)

	USDHC2_CLK	GPIO_AD_B1_09	ALT6
		GPIO_SD_B1_04	ALT0
	USDHC2_DATA0	GPIO_AD_B1_04	ALT6
		GPIO_SD_B1_03	ALT0
	USDHC2_DATA1	GPIO_AD_B1_05	ALT6
		GPIO_SD_B1_02	ALT0
	USDHC2_DATA2	GPIO_AD_B1_06	ALT6
		GPIO_SD_B1_01	ALT0
	USDHC2_DATA3	GPIO_AD_B1_07	ALT6
		GPIO_SD_B1_00	ALT0
	USDHC2_DATA4	GPIO_AD_B1_12	ALT6
		GPIO_SD_B1_08	ALT0
	USDHC2_DATA5	GPIO_AD_B1_13	ALT6
		GPIO_SD_B1_09	ALT0
	USDHC2_DATA6	GPIO_AD_B1_14	ALT6
		GPIO_SD_B1_10	ALT0
	USDHC2_DATA7	GPIO_AD_B1_15	ALT6
		GPIO_SD_B1_11	ALT0
	USDHC2_RESET_B	GPIO_AD_B1_11	ALT6
		GPIO_EMC_40	ALT6
		GPIO_EMC_11	ALT3
		GPIO_SD_B1_06	ALT0
	USDHC2_WP	GPIO_AD_B1_10	ALT6
		GPIO_EMC_37	ALT6
	USDHC2_VSELECT	GPIO_EMC_38	ALT6
	USDHC2_CD_B	GPIO_AD_B1_03	ALT6
		GPIO_EMC_39	ALT6
WDOG1	WDOG1_B	GPIO_B1_13	ALT0
		GPIO_AD_B1_10	ALT1
		GPIO_EMC_39	ALT4
		GPIO_AD_B0_11	ALT3
		GPIO_AD_B1_00	ALT4
	WDOG1_RESET_B_DEB	GPIO_AD_B0_15	ALT7
	WDOG1_ANY	GPIO_AD_B0_14	ALT7
WDOG2	WDOG2_B	GPIO_AD_B0_12	ALT3
	WDOG2_RESET_B_DEB	GPIO_B0_03	ALT6
XBAR1	XBAR_INOUT02	GPIO_EMC_00	ALT3
		GPIO_B1_14	ALT3
	XBAR_INOUT03	GPIO_EMC_01	ALT3
		GPIO_B1_15	ALT3

Table continues on the next page...

Table 8-1. Muxing Options (continued)

XBAR_INOUT04	GPIO_SD_B0_00	ALT3
	GPIO_EMC_02	ALT3
XBAR_INOUT05	GPIO_EMC_03	ALT3
	GPIO_SD_B0_01	ALT3
XBAR_INOUT06	GPIO_SD_B0_02	ALT3
	GPIO_EMC_04	ALT3
XBAR_INOUT07	GPIO_EMC_05	ALT3
	GPIO_SD_B0_03	ALT3
XBAR_INOUT08	GPIO_SD_B0_04	ALT3
	GPIO_EMC_06	ALT3
XBAR_INOUT09	GPIO_EMC_07	ALT3
	GPIO_SD_B0_05	ALT3
XBAR_INOUT10	GPIO_B0_12	ALT1
XBAR_INOUT11	GPIO_B0_13	ALT1
XBAR_INOUT12	GPIO_B0_14	ALT1
XBAR_INOUT13	GPIO_B0_15	ALT1
XBAR_INOUT14	GPIO_B1_00	ALT1
	GPIO_AD_B0_00	ALT1
XBAR_INOUT15	GPIO_AD_B0_01	ALT1
	GPIO_B1_01	ALT1
XBAR_INOUT16	GPIO_B1_02	ALT1
	GPIO_AD_B0_02	ALT1
XBAR_INOUT17	GPIO_AD_B0_03	ALT1
	GPIO_EMC_08	ALT3
	GPIO_AD_B0_05	ALT6
	GPIO_B1_03	ALT1
XBAR_INOUT18	GPIO_EMC_35	ALT1
	GPIO_AD_B0_06	ALT6
XBAR_INOUT19	GPIO_EMC_14	ALT1
	GPIO_AD_B0_07	ALT6
XBAR_INOUT20	GPIO_AD_B0_08	ALT6
	GPIO_EMC_15	ALT1
XBAR_INOUT21	GPIO_EMC_16	ALT1
	GPIO_AD_B0_09	ALT6
XBAR_INOUT22	GPIO_AD_B0_10	ALT6
	GPIO_EMC_36	ALT1
XBAR_INOUT23	GPIO_EMC_37	ALT1
	GPIO_AD_B0_11	ALT6
XBAR_INOUT24	GPIO_AD_B0_14	ALT1
	GPIO_EMC_12	ALT1

Table continues on the next page...

Table 8-1. Muxing Options (continued)

XTALOSC	XBAR_INOUT25	GPIO_AD_B0_15	ALT1
		GPIO_EMC_13	ALT1
	REF_CLK_32K	GPIO_AD_B0_00	ALT2
	REF_CLK_24M	GPIO_AD_B0_01	ALT2
		GPIO_AD_B0_03	ALT6
		GPIO_AD_B0_13	ALT7
	XTALI	XTALI	No Muxing
	XTALO	XTALO	No Muxing

8.2 IOMUX Controller (IOMUXC)

8.2.1 Overview

The IOMUX Controller (IOMUXC), together with the IOMUX, enables the IC to share one pad to several functional blocks. This sharing is done by multiplexing the pad's input and output signals.

Every module requires a specific pad setting (such as pull up or keeper), and for each pad, there are up to 8 muxing options (called ALT modes). The pad settings parameters are controlled by the IOMUXC.

The IOMUX consists only of combinatorial logic combined from several basic IOMUX cells. Each basic IOMUX cell handles only one pad signal's muxing.

[Figure 8-1](#) illustrates the IOMUX/IOMUXC connectivity in the system.

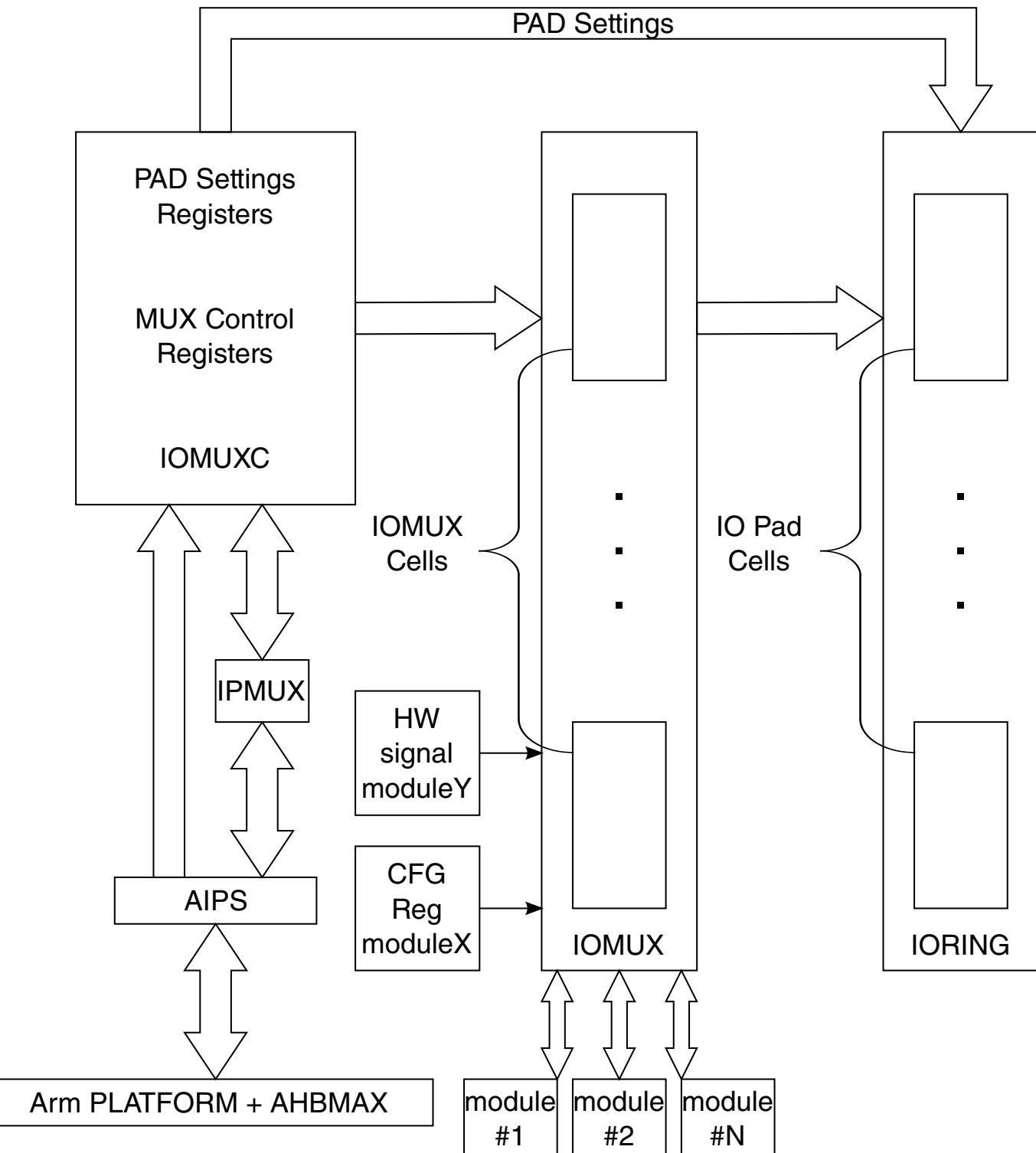


Figure 8-1. IOMUX SoC Level Block Diagram

8.2.1.1 Features

The IOMUXC features are:

- 32-bit software mux control registers (IOMUXC_SW_MUX_CTL_PAD_<PAD NAME> or IOMUXC_SW_MUX_CTL_GRP_<GROUP NAME>) to configure 1 of 8 alternate (ALT) MUX_MODE fields of each pad or a predefined group of pads and to enable the forcing of an input path of the pad(s) (SION bit).
- 32-bit software pad control registers (IOMUXC_SW_PAD_CTL_PAD_<PAD NAME> or IOMUXC_SW_PAD_CTL_GRP_<GROUP NAME>) to configure specific pad settings of each pad, or a predefined group of pads.
- 32-bit general purpose registers - several (GPR0 to GPR_n) 32-bit registers according to SoC requirements for any usage.
- 32-bit input select control registers to control the input path to a module when more than one pad drives this module input.

Each SW MUX/PAD CTL IOMUXC register handles only one pad or one pad's group.

Only the minimum number of registers required by software are implemented by hardware. For example, if only ALT0 and ALT1 modes are used on Pad x then only one bit register will be generated as the MUX_MODE control field in the software mux control register of Pad x.

The software mux control registers may allow the forcing of pads to become input (input path enabled) regardless of the functional direction driven. This may be useful for loopback and GPIO data capture.

8.2.2 Clocks

The table found here describes the clock sources for IOMUXC.

Please see [Clock Controller Module \(CCM\)](#) for clock setting, configuration and gating information.

Table 8-2. IOMUXC Clocks

Clock name	Clock Root	Description
ipt_clk_io		IO clock
ipg_clk_s	ipg_clk_root	Peripheral access clock

8.2.3 Functional description

This section provides a complete functional description of the block.

The IOMUXC consists of two sub-blocks:

- IOMUXC_REGISTERS includes all of the IOMUXC registers (see [Features](#)).
- IOMUXC_LOGIC includes all of the IOMUXC combinatorial logic (IP interface controls, address decoder, observability muxes).

The IOMUX consists of a number (about the number of pads in the SoC) of basic iomux_cell units. If only one functional mode is required for a specific pad, there is no need for IOMUX and the signals can be connected directly from the module to the I/O. The IOMUX cell is required whenever two or more functional modes are required for a specific pad or when one functional mode and the one test mode are required.

The basic iomux_cell design, which allows two levels of HW signal control (in ALT6 and ALT7 modes - ALT7 gets highest priority) is shown in [Figure 8-2](#).

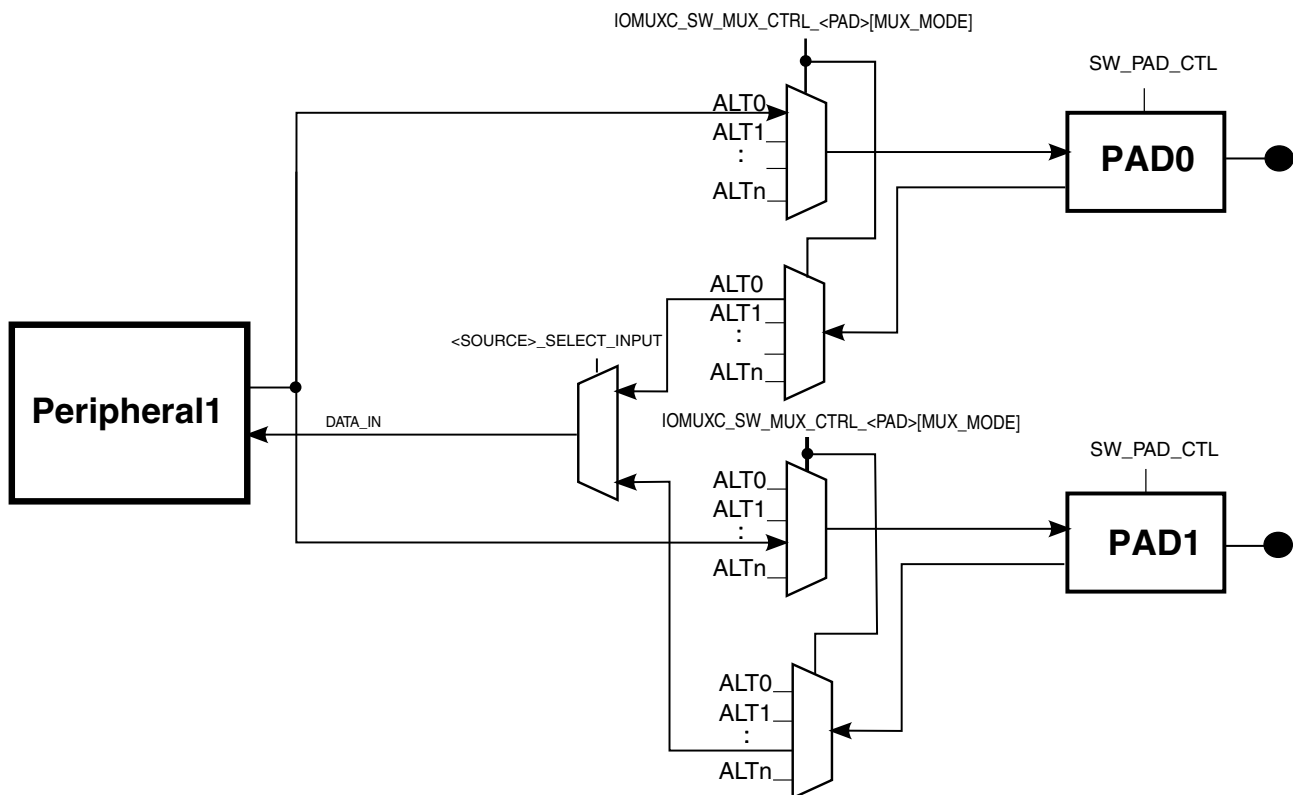


Figure 8-2. IOMUX Cell Block Diagram

8.2.3.1 ALT6 and ALT7 extended muxing modes

The ALT7 and ALT6 extended muxing modes allow any signal in the system (such as fuse, pad input, JTAG, or software register) to override any software configuration and to force the ALT6/ALT7 muxing mode.

It also allows an IOMUX software register to control a group of pads.

8.2.3.2 SW Loopback through SION bit

A limited option exists to override the default pad functionality and force the input path to be active (`ipp_ibe==1'b1`) regardless of the value driven by the corresponding module. This can be done by setting the SION (Software Input On) bit in the IOMUXC_SW_MUX_CTL register (when available) to "1".

Uses include:

- LoopBack - Module x drives the pad and also receives pad value as an input.
- GPIO Capture - Module x drives the pad and the value is captured by GPIO.

8.2.3.3 Daisy chain - multi pads driving same module input pin

In some cases, more than one pad may drive a single module input pin. Such cases require the addition of one more level of IOMUXing; all of these input signals are muxed, and a dedicated software controlled register controls the mux in order to select the required input path.

A module port involved in "daisy chain" requires two software configuration commands, one for selecting the mode for this pad (programmable via the IOMUXC_SW_MUX_CTL_<PAD> registers) and one for defining it as the input path (via the daisy chain registers).

This means that a module port involved in "daisy chain" requires two software configuration commands, one for selecting the mode for this pad (programmable via the IOMUXC_SW_MUX_CTL_<PAD> registers) and one for defining it as the input path (via the daisy chain registers). The daisy chain is illustrated in the figure below.

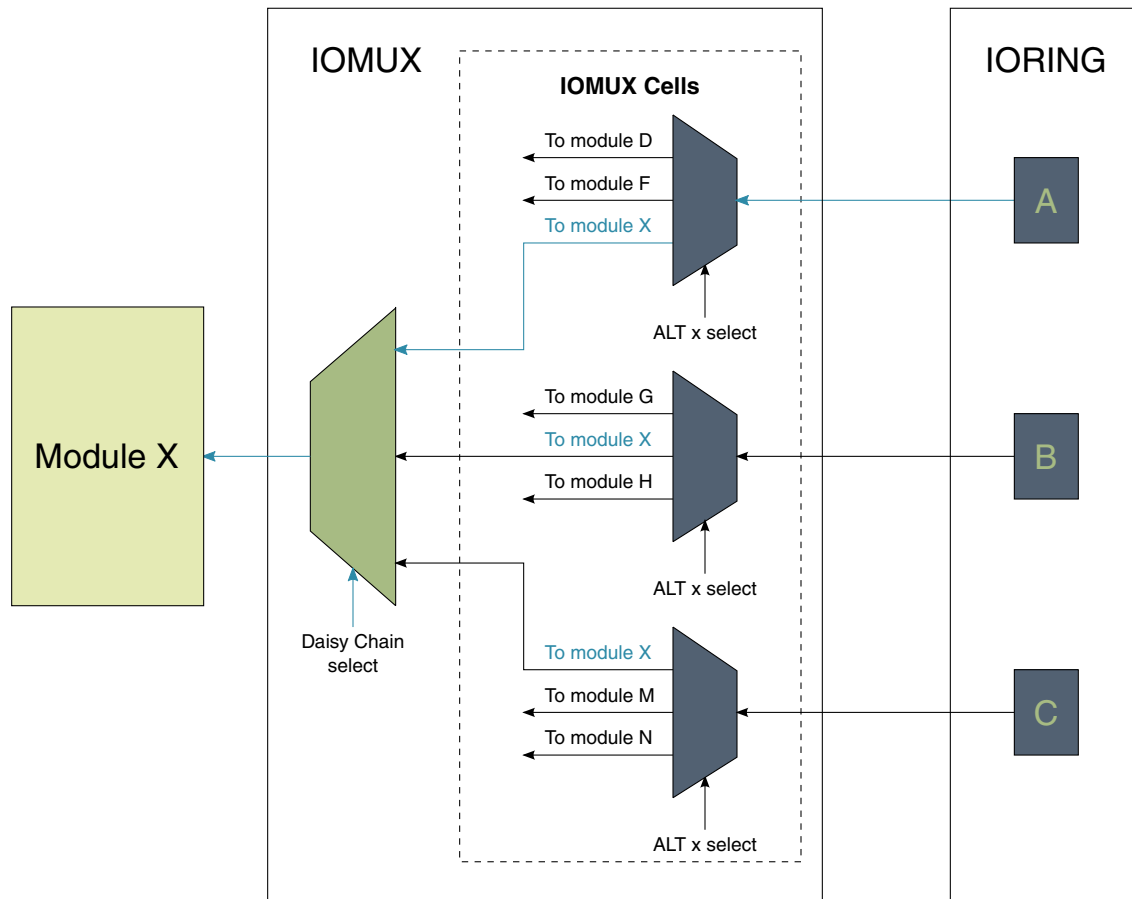


Figure 8-3. Daisy chain illustration

8.2.4 IOMUXC GPR Memory Map/Register Definition

IOMUXC_GPR memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3034_0000	GPR0 General Purpose Register (IOMUXC_GPR_GPR0)	32	R/W	0000_0000h	8.2.4.1/1352
3034_0004	GPR1 General Purpose Register (IOMUXC_GPR_GPR1)	32	R/W	0000_0000h	8.2.4.2/1353
3034_0008	GPR2 General Purpose Register (IOMUXC_GPR_GPR2)	32	R/W	0000_0000h	8.2.4.3/1354
3034_000C	GPR3 General Purpose Register (IOMUXC_GPR_GPR3)	32	R/W	0000_00FFh	8.2.4.4/1356
3034_0010	GPR4 General Purpose Register (IOMUXC_GPR_GPR4)	32	R/W	0000_0000h	8.2.4.5/1360
3034_0014	GPR5 General Purpose Register (IOMUXC_GPR_GPR5)	32	R/W	0000_0000h	8.2.4.6/1363
3034_0018	GPR6 General Purpose Register (IOMUXC_GPR_GPR6)	32	R/W	0000_0000h	8.2.4.7/1364
3034_001C	GPR7 General Purpose Register (IOMUXC_GPR_GPR7)	32	R/W	0000_0000h	8.2.4.8/1364
3034_0020	GPR8 General Purpose Register (IOMUXC_GPR_GPR8)	32	R/W	0000_0000h	8.2.4.9/1365
3034_0024	GPR9 General Purpose Register (IOMUXC_GPR_GPR9)	32	R/W	0000_0000h	8.2.4.10/1365
3034_0028	GPR10 General Purpose Register (IOMUXC_GPR_GPR10)	32	R/W	0000_0008h	8.2.4.11/1366
3034_002C	GPR11 General Purpose Register (IOMUXC_GPR_GPR11)	32	R/W	0000_0000h	8.2.4.12/1367
3034_0030	GPR12 General Purpose Register (IOMUXC_GPR_GPR12)	32	R/W	0000_4400h	8.2.4.13/1369
3034_0034	GPR13 General Purpose Register (IOMUXC_GPR_GPR13)	32	R/W	0000_0000h	8.2.4.14/1371
3034_0038	GPR14 General Purpose Register (IOMUXC_GPR_GPR14)	32	R/W	4940_9100h	8.2.4.15/1373
3034_003C	GPR15 General Purpose Register (IOMUXC_GPR_GPR15)	32	R/W	6188_FFFFh	8.2.4.16/1374
3034_0040	GPR16 General Purpose Register (IOMUXC_GPR_GPR16)	32	R/W	4940_9100h	8.2.4.17/1375
3034_0044	GPR17 General Purpose Register (IOMUXC_GPR_GPR17)	32	R/W	6188_FFFFh	8.2.4.18/1377
3034_0048	GPR18 General Purpose Register (IOMUXC_GPR_GPR18)	32	R/W	0000_0000h	8.2.4.19/1377
3034_004C	GPR19 General Purpose Register (IOMUXC_GPR_GPR19)	32	R	0000_0000h	8.2.4.20/1378

Table continues on the next page...

IOMUXC_GPR memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3034_0050	GPR20 General Purpose Register (IOMUXC_GPR_GPR20)	32	R/W	0000_0000h	8.2.4.21/ 1378
3034_0054	GPR21 General Purpose Register (IOMUXC_GPR_GPR21)	32	R/W	0000_0000h	8.2.4.22/ 1379
3034_0058	GPR22 General Purpose Register (IOMUXC_GPR_GPR22)	32	R	0000_0000h	8.2.4.23/ 1379
3034_005C	GPR23 General Purpose Register (IOMUXC_GPR_GPR23)	32	R	0000_0000h	8.2.4.24/ 1380
3034_0060	GPR24 General Purpose Register (IOMUXC_GPR_GPR24)	32	R	0000_0000h	8.2.4.25/ 1382
3034_0064	GPR25 General Purpose Register (IOMUXC_GPR_GPR25)	32	R	0000_0000h	8.2.4.26/ 1385
3034_0068	GPR26 General Purpose Register (IOMUXC_GPR_GPR26)	32	R	0000_0000h	8.2.4.27/ 1385
3034_006C	GPR27 General Purpose Register (IOMUXC_GPR_GPR27)	32	R	0000_0000h	8.2.4.28/ 1386
3034_0070	GPR28 General Purpose Register (IOMUXC_GPR_GPR28)	32	R	0000_0000h	8.2.4.29/ 1386
3034_0074	GPR29 General Purpose Register (IOMUXC_GPR_GPR29)	32	R/W	0000_0000h	8.2.4.30/ 1387
3034_0078	GPR30 General Purpose Register (IOMUXC_GPR_GPR30)	32	R	0000_0000h	8.2.4.31/ 1388
3034_007C	GPR31 General Purpose Register (IOMUXC_GPR_GPR31)	32	R	0000_0000h	8.2.4.32/ 1389
3034_0080	GPR32 General Purpose Register (IOMUXC_GPR_GPR32)	32	R	0000_0000h	8.2.4.33/ 1391
3034_0084	GPR33 General Purpose Register (IOMUXC_GPR_GPR33)	32	R	0000_0000h	8.2.4.34/ 1392
3034_0088	GPR34 General Purpose Register (IOMUXC_GPR_GPR34)	32	R/W	0000_0800h	8.2.4.35/ 1393
3034_008C	GPR35 General Purpose Register (IOMUXC_GPR_GPR35)	32	R	0000_0000h	8.2.4.36/ 1395
3034_0090	GPR36 General Purpose Register (IOMUXC_GPR_GPR36)	32	R/W	0000_0000h	8.2.4.37/ 1398
3034_0094	GPR37 General Purpose Register (IOMUXC_GPR_GPR37)	32	R	0000_0000h	8.2.4.38/ 1399
3034_0098	GPR38 General Purpose Register (IOMUXC_GPR_GPR38)	32	R	0000_0000h	8.2.4.39/ 1401
3034_009C	GPR39 General Purpose Register (IOMUXC_GPR_GPR39)	32	R	0000_0000h	8.2.4.40/ 1403
3034_00A0	GPR40 General Purpose Register (IOMUXC_GPR_GPR40)	32	R	0000_0000h	8.2.4.41/ 1405
3034_00A4	GPR41 General Purpose Register (IOMUXC_GPR_GPR41)	32	R/W	0000_0800h	8.2.4.42/ 1407

Table continues on the next page...

IOMUXC_GPR memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3034_00A8	GPR42 General Purpose Register (IOMUXC_GPR_GPR42)	32	R	0000_0000h	8.2.4.43/1409
3034_00AC	GPR43 General Purpose Register (IOMUXC_GPR_GPR43)	32	R/W	0000_0000h	8.2.4.44/1412
3034_00B0	GPR44 General Purpose Register (IOMUXC_GPR_GPR44)	32	R	0000_0000h	8.2.4.45/1413
3034_00B4	GPR45 General Purpose Register (IOMUXC_GPR_GPR45)	32	R	0000_0000h	8.2.4.46/1415
3034_00B8	GPR46 General Purpose Register (IOMUXC_GPR_GPR46)	32	R	0000_0000h	8.2.4.47/1417
3034_00BC	GPR47 General Purpose Register (IOMUXC_GPR_GPR47)	32	R	0000_0000h	8.2.4.48/1419

8.2.4.1 GPR0 General Purpose Register (IOMUXC_GPR_GPR0)

GPR Register

Address: 3034_0000h base + 0h offset = 3034_0000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

IOMUXC_GPR_GPR0 field descriptions

Field	Description
-	This field is reserved. Reserved

8.2.4.2 GPR1 General Purpose Register (IOMUXC_GPR_GPR1)

GPR Register

Address: 3034_0000h base + 4h offset = 3034_0004h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DBG_ACK				Reserved				TZASC1_SECURE_ BOOT_LOCK	Reserved				Reserved		Reserved
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	GPR_ANAMIX_IPT_ MODE	Reserved	ENET1_TX_CLK_SEL	IRQ	Reserved											
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR1 field descriptions

Field	Description
31–28 DBG_ACK	Debug Acknowledge
27–24 -	This field is reserved. Reserved
23 TZASC1_ SECURE_ BOOT_LOCK	TZASC-1 Secure Boot Lock 0 Secure boot lock is disabled 1 Secure boot lock is enabled
22–18 -	This field is reserved. Reserved
17 -	This field is reserved.
16 -	This field is reserved. Reserved

Table continues on the next page...

IOMUXC_GPR_GPR1 field descriptions (continued)

Field	Description
15 GPR_ANAMIX_IPT_MODE	
14 -	This field is reserved. Reserved
13 ENET1_TX_CLK_SEL	ENET1 reference clock mode select 0 Gets ENET1 TX reference clk. This clock is also output to pins via the IOMUX. ENET_REF_CLK1 function. 1 Gets ENET1 TX reference clk from the ENET1_TX_CLK pin. In this use case, an external OSC provides the clock for both the external PHY and the internal controller
12 IRQ	Interrupt signal which is connected to CPU IRQS[0]. Used to notify cores on exception condition while boot.
-	This field is reserved. Reserved

8.2.4.3 GPR2 General Purpose Register (IOMUXC_GPR_GPR2)**GPR Register**

Address: 3034_0000h base + 8h offset = 3034_0008h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved										GPR_SAI6_EXT_MCLK_EN GPR_SAI5_EXT_MCLK_EN GPR_SAI4_EXT_MCLK_EN GPR_SAI3_EXT_MCLK_EN GPR_SAI2_EXT_MCLK_EN GPR_SAI1_EXT_MCLK_EN					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

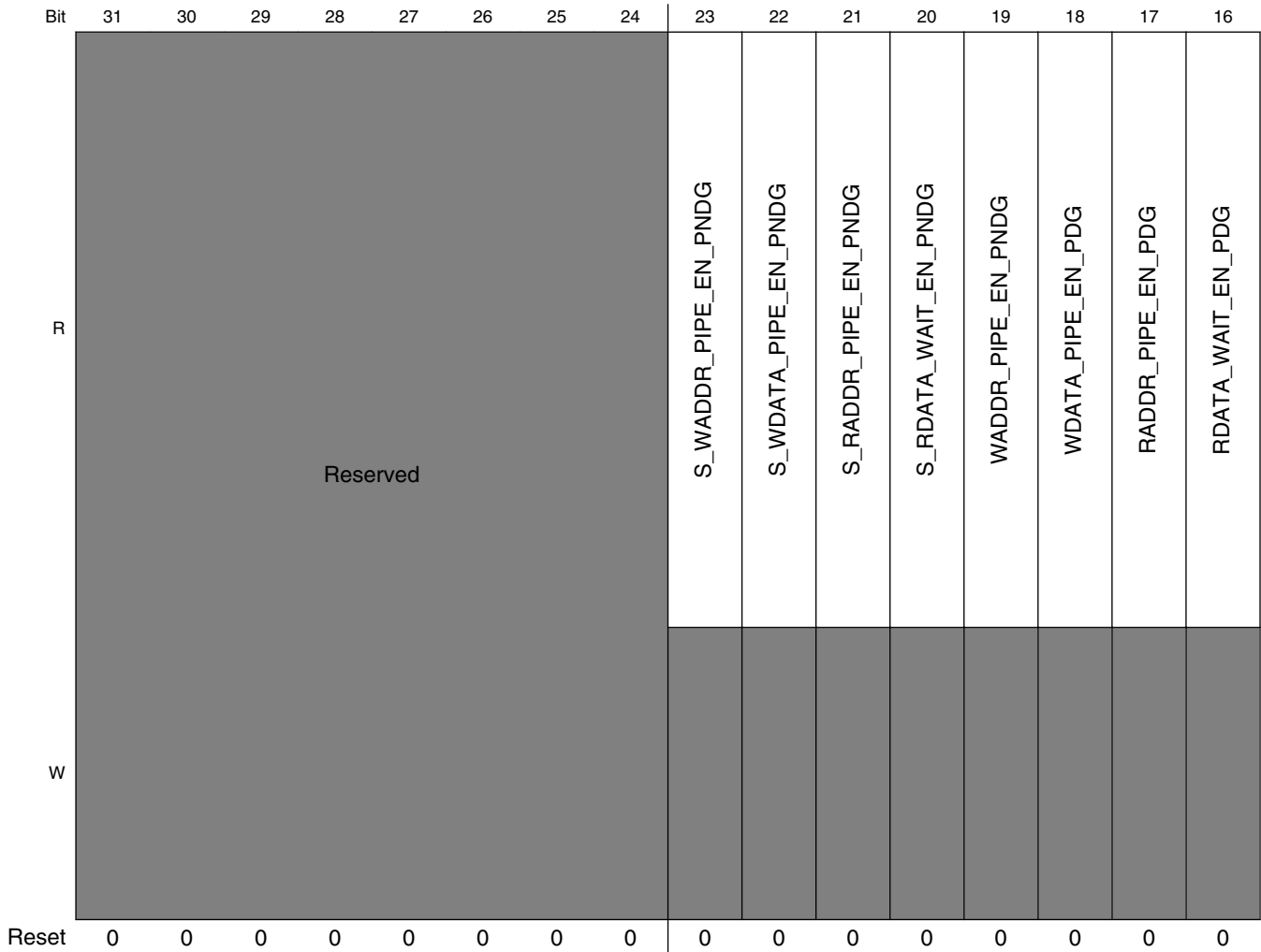
IOMUXC_GPR_GPR2 field descriptions

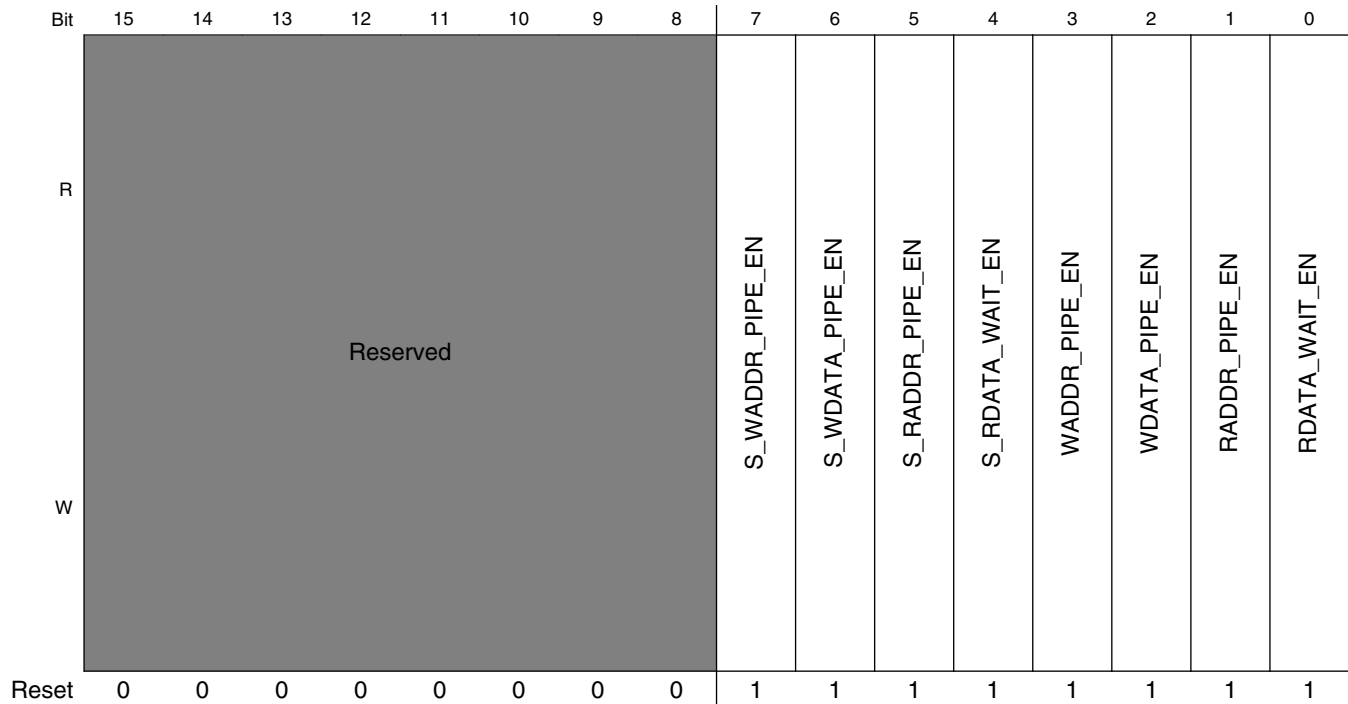
Field	Description
31–6 -	This field is reserved. Reserved
5 GPR_SAI6_ EXT_MCLK_EN	SAI6 External MCLK Enable
4 GPR_SAI5_ EXT_MCLK_EN	SAI5 External MCLK Enable
3 GPR_SAI4_ EXT_MCLK_EN	SAI4 External MCLK Enable
2 GPR_SAI3_ EXT_MCLK_EN	SAI3 External MCLK Enable
1 GPR_SAI2_ EXT_MCLK_EN	SAI2 External MCLK Enable
0 GPR_SAI1_ EXT_MCLK_EN	SAI1 External MCLK Enable

8.2.4.4 GPR3 General Purpose Register (IOMUXC_GPR_GPR3)

GPR Register

Address: 3034_0000h base + Ch offset = 3034_000Ch





IOMUXC_GPR_GPR3 field descriptions

Field	Description
31–24 -	This field is reserved. Reserved
23 S_WADDR_PIPE_EN_PNDG	State Retention On-chip RAM write address pipeline enable update is pending 0 write address pipeline enable configuration valid 1 write address pipeline enable bit changed
22 S_WDATA_PIPE_EN_PNDG	State Retention On-chip RAM write data pipeline enable update is pending 0 write data pipeline enable configuration valid 1 write data pipeline enable bit changed
21 S_RADDR_PIPE_EN_PNDG	State Retention On-chip RAM read address pipeline enable update is pending 0 read address pipeline enable configuration valid 1 read address pipeline enable bit changed
20 S_RDATA_WAIT_EN_PNDG	State Retention On-chip RAM read data wait state control update is pending 0 read data wait state control configuration valid 1 read data wait state control bit changed
19 WADDR_PIPE_EN_PNDG	On-chip RAM write address pipeline enable update is pending 0 write address pipeline enable configuration valid 1 write address pipeline enable bit changed
18 WDATA_PIPE_EN_PDG	On-chip RAM write data pipeline enable update is pending 0 write data pipeline enable configuration valid 1 write data pipeline enable bit changed

Table continues on the next page...

IOMUXC_GPR_GPR3 field descriptions (continued)

Field	Description
17 RADDR_PIPE_EN_PDG	On-chip RAM read address pipeline enable update is pending 0 read address pipeline enable configuration valid 1 read address pipeline enable bit changed
16 RDATA_WAIT_EN_PDG	On-chip RAM read data wait state control update is pending 0 read data wait state control configuration valid 1 read data wait state control bit changed
15–8 -	This field is reserved. Reserved
7 S_WADDR_PIPE_EN	State Retention On-chip RAM write address pipeline enable For description, please refer to WADDR_PIPE_EN bit
6 S_WDATA_PIPE_EN	State Retention On-chip RAM write data pipeline enable For description, please refer to WDATA_PIPE_EN bit
5 S_RADDR_PIPE_EN	State Retention On-chip RAM read address pipeline enable For description, please refer to RADDR_PIPE_EN bit
4 S_RDATA_WAIT_EN	State Retention On-chip RAM read data wait state control For description, please refer to RDATA_WAIT_EN bit
3 WADDR_PIPE_EN	On-chip RAM write address pipeline enable When this feature is enabled, the write address from the AXI master would be delayed 1 cycle before it can be accepted by the on-chip RAM. This can avoid setup time issue for the write access on the memory cell at high frequency. Enable this feature would cost at most 1 more clock cycle for each AXI write transaction, i.e., at most 1 more clock cycle for each write burst with multiple beats of data. When this feature is disabled, the write address from the AXI master can be accepted by the on-chip RAM without delay, and data can be written to memory at this cycle (if no other access and write data is also ready at this cycle). 0 write address pipeline is disabled 0 write address pipeline is disabled 1 write address pipeline is enabled
2 WDATA_PIPE_EN	On-chip RAM write data pipeline enable When this feature is enabled, the write data from the AXI master would be delayed 1 cycle before it can be accepted by the on-chip RAM. This can avoid setup time issue for the write access on the memory cell at high frequency. Enable this feature would cost at most 1 more clock cycle for each AXI write transaction, i.e., at most 1 more clock cycle for each write burst with multiple beats of data. When this feature is disabled, the write data from the AXI master can be accepted by the on-chip RAM without delay, and data can be written to memory at this cycle (if no other access and write address is also ready at this cycle). 0 write data pipeline is disabled 1 write data pipeline is enabled
1 RADDR_PIPE_EN	On-chip RAM read address pipeline enable When this feature is enabled, the read address from the AXI master would be delayed 1 cycle before it can be accepted by the on-chip RAM. This can avoid setup time issue for the read access on the memory cell at high frequency. Enable this feature would cost at most 1 more clock cycle for each AXI read

Table continues on the next page...

IOMUXC_GPR_GPR3 field descriptions (continued)

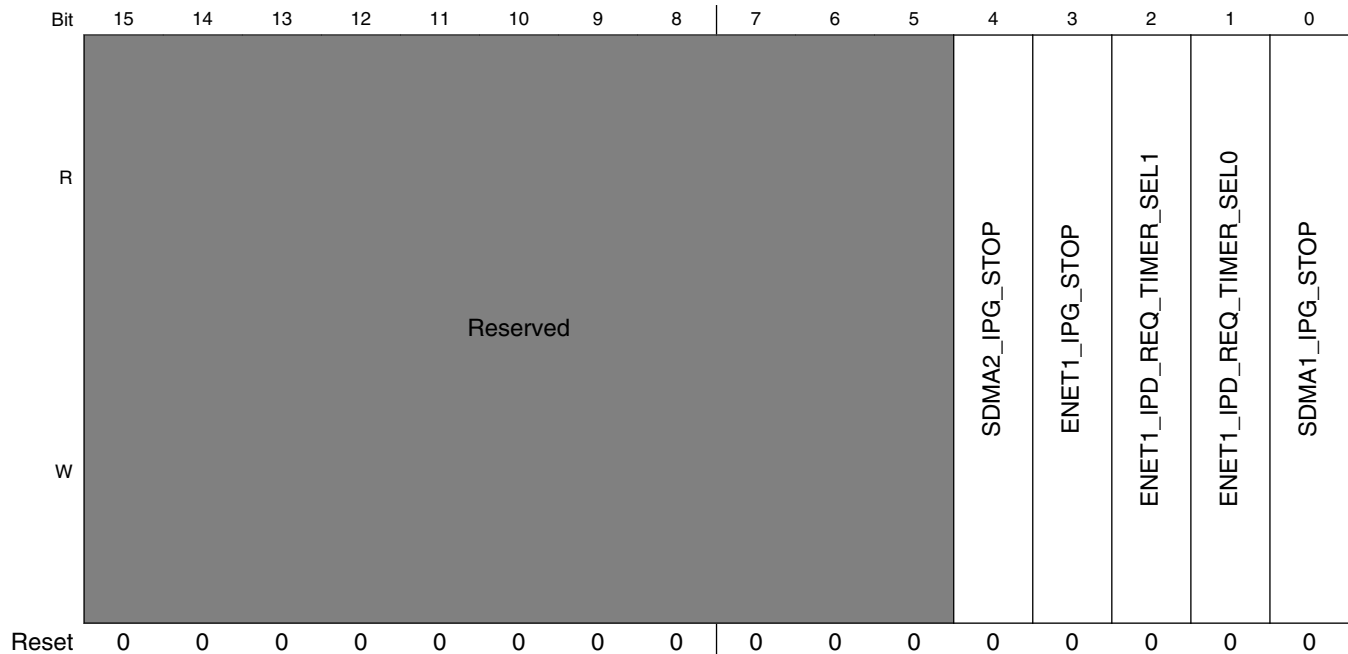
Field	Description
	transaction, i.e., at most 1 more clock cycle for each read burst with multiple beats of data. When this feature is disabled, the read address from the AXI master can be accepted by the on-chip RAM without delay, and data can become ready for master at next clock cycle (if no other access and no read data wait). 0 read address pipeline is disabled 1 read address pipeline is enabled
0 RDATA_WAIT_ EN	On-chip RAM read data wait state control When the read data wait state is enabled, it will cost 2 cycles for each read access, (each beat of a read burst). This can avoid the potential timing problem caused by the relatively longer memory access time at higher frequency. When this feature is disabled, it only costs 1 clock cycle to finish a read transaction, i.e., get read data back in the next cycle of read request becomes valid on the bus. 0 read data wait state disabled 1 read data wait state enabled

8.2.4.5 GPR4 General Purpose Register (IOMUXC_GPR_GPR4)

GPR Register

Address: 3034_0000h base + 10h offset = 3034_0010h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0					SAI6_IPG_STOP_ACK	SAI5_IPG_STOP_ACK	SAI4_IPG_STOP_ACK	SAI3_IPG_STOP_ACK	SAI2_IPG_STOP_ACK	SAI1_IPG_STOP_ACK	SDMA2_IPG_STOP_ACK	ENET1_IPG_STOP_ACK	Reserved		SDMA1_IPG_STOP_ACK
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

**IOMUXC_GPR_GPR4 field descriptions**

Field	Description
31–27 Reserved	This read-only field is reserved and always has the value 0.
26 SAI6_IPG_STOP_ACK	SAI6 stop acknowledge 0 stop acknowledge is not asserted 1 stop acknowledge is asserted, peripheral is in STOP mode
25 SAI5_IPG_STOP_ACK	SAI5 stop acknowledge 0 stop acknowledge is not asserted 1 stop acknowledge is asserted, peripheral is in STOP mode
24 SAI4_IPG_STOP_ACK	SAI4 stop acknowledge 0 stop acknowledge is not asserted 1 stop acknowledge is asserted, peripheral is in STOP mode
23 SAI3_IPG_STOP_ACK	SAI3 stop acknowledge 0 stop acknowledge is not asserted 1 stop acknowledge is asserted, peripheral is in STOP mode
22 SAI2_IPG_STOP_ACK	SAI2 stop acknowledge 0 stop acknowledge is not asserted 1 stop acknowledge is asserted, peripheral is in STOP mode
21 SAI1_IPG_STOP_ACK	SAI1 stop acknowledge 0 stop acknowledge is not asserted 1 stop acknowledge is asserted, peripheral is in STOP mode

Table continues on the next page...

IOMUXC_GPR_GPR4 field descriptions (continued)

Field	Description
20 SDMA2_IPG_STOP_ACK	SDMA2 stop acknowledge 0 stop acknowledge is not asserted 1 stop acknowledge is asserted, peripheral is in STOP mode
19 ENET1_IPG_STOP_ACK	ENET1 stop acknowledge 0 stop acknowledge is not asserted 1 stop acknowledge is asserted, peripheral is in STOP mode
18–17 -	This field is reserved. Reserved
16 SDMA1_IPG_STOP_ACK	SDMA1 stop acknowledge 0 stop acknowledge is not asserted 1 stop acknowledge is asserted, peripheral is in STOP mode
15–5 -	This field is reserved. Reserved
4 SDMA2_IPG_STOP	SDMA2 stop request 0 stop request off 1 stop request on
3 ENET1_IPG_STOP	ENET1 stop request 0 stop request off 1 stop request on
2 ENET1_IPD_REQ_TIMER_SEL1	ENET1 IPD_REQ Timer Select 1 0 Select ipd_req_mac0_timer3 to SDMA IRQ 47 1 Select ipd_req_mac0_timer1 to SDMA IRQ 47
1 ENET1_IPD_REQ_TIMER_SEL0	ENET1 IPD_REQ Timer Select 0 0 Select ipd_req_mac0_timer2 to SDMA IRQ 45 1 Select ipd_req_mac0_timer0 to SDMA IRQ 45
0 SDMA1_IPG_STOP	SDMA1 stop request 0 stop request off 1 stop request on

8.2.4.6 GPR5 General Purpose Register (IOMUXC_GPR_GPR5)

GPR Register

Address: 3034_0000h base + 14h offset = 3034_0014h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved												WDOG3_MASK	Reserved		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								WDOG2_MASK	WDOG1_MASK	Reserved		HDMI_CEC_PD	HDMI_DDC_SCL_PD	HDMI_DDC_SDA_PD	HDMI_HPD_PD
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR5 field descriptions

Field	Description
31–21 -	This field is reserved. Reserved
20 WDOG3_MASK	WDOG3 Timeout Mask 0 WDOG3 Timeout behaves normally 1 WDOG3 Timeout is masked
19–8 -	This field is reserved. Reserved
7 WDOG2_MASK	WDOG2 Timeout Mask 0 WDOG2 Timeout behaves normally 1 WDOG2 Timeout is masked
6 WDOG1_MASK	WDOG1 Timeout Mask 0 WDOG1 Timeout behaves normally 1 WDOG1 Timeout is masked
5–4 -	This field is reserved. Reserved
3 HDMI_CEC_PD	Connect to hdmi_cec_pd_pad

Table continues on the next page...

IOMUXC_GPR_GPR5 field descriptions (continued)

Field	Description
2 HDMI_DDC_ SCL_PD	Connect to hdmi_ddc_scl_pd_pad
1 HDMI_DDC_ SDA_PD	Connect to hdmi_ddc_sda_pd_pad
0 HDMI_HPD_PD	Connect to hdmi_hpd_pd_pad

8.2.4.7 GPR6 General Purpose Register (IOMUXC_GPR_GPR6)

GPR Register

Address: 3034_0000h base + 18h offset = 3034_0018h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

IOMUXC_GPR_GPR6 field descriptions

Field	Description
-	This field is reserved. Reserved

8.2.4.8 GPR7 General Purpose Register (IOMUXC_GPR_GPR7)

GPR Register

Address: 3034_0000h base + 1Ch offset = 3034_001Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved																															
W	Reserved																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

IOMUXC_GPR_GPR7 field descriptions

Field	Description
-	This field is reserved. Reserved

8.2.4.9 GPR8 General Purpose Register (IOMUXC_GPR_GPR8)

GPR Register

Address: 3034_0000h base + 20h offset = 3034_0020h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR8 field descriptions

Field	Description
-	This field is reserved. Reserved

8.2.4.10 GPR9 General Purpose Register (IOMUXC_GPR_GPR9)

GPR Register

Address: 3034_0000h base + 24h offset = 3034_0024h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

IOMUXC_GPR_GPR9 field descriptions

Field	Description
-	This field is reserved. Reserved

8.2.4.11 GPR10 General Purpose Register (IOMUXC_GPR_GPR10)

GPR Register

Address: 3034_0000h base + 28h offset = 3034_0028h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved												EXC_ERR_RESP_EN_LOCK	SEC_ERR_RESP_EN_LOCK	TZASC_ID_SWAP_BYPASS_LOCK	TZASC_EN_LOCK
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												EXC_ERR_RESP_EN	SEC_ERR_RESP_EN	TZASC_ID_SWAP_BYPASS	TZASC_EN
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

IOMUXC_GPR_GPR10 field descriptions

Field	Description
31–20 -	This field is reserved. Reserved
19 EXC_ERR_RESP_EN_LOCK	Lock bit for EXC_ERR_RESP_EN
18 SEC_ERR_RESP_EN_LOCK	Lock bit for SEC_ERR_RESP_EN
17 TZASC_ID_SWAP_BYPASS_LOCK	Lock bit for TZASC_ID_SWAP_BYPASS
16 TZASC_EN_LOCK	Lock bit for TZASC_EN

Table continues on the next page...

IOMUXC_GPR_GPR10 field descriptions (continued)

Field	Description
15–4 -	This field is reserved. Reserved
3 EXC_ERR_RESP_EN	Security exclusive access error response enable for all security gaskets (on both AHB and AXI busses). Enables an ERR response on the AXI vs an OK response for an exclusive access error. 0 OK response on the AXI for an exclusive access error 1 ERR response on the AXI for an exclusive access error
2 SEC_ERR_RESP_EN	Security error response enable for all security gaskets (on both AHB and AXI busses) This is a "lock" type bit 0 OKAY response 1 SLVERR response
1 TZASC_ID_SWAP_BYPASS	Connect to id_swap_bypass input on tzasc_id_wrap
0 TZASC_EN	Connect to tzasc_en input on tzasc_id_wrap

8.2.4.12 GPR11 General Purpose Register (IOMUXC_GPR_GPR11)**GPR Register**

Address: 3034_0000h base + 2Ch offset = 3034_002Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved		OCRAM_S_TZ_ADDR_LOCK			OCRAM_S_TZ_EN_LOCK	Reserved					OCRAM_TZ_ADDR_LOCK					OCRAM_TZ_EN_LOCK
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved		OCRAM_S_TZ_ADDR			OCRAM_S_TZ_EN	Reserved					OCRAM_TZ_ADDR					OCRAM_TZ_EN
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

IOMUXC_GPR_GPR11 field descriptions

Field	Description
31–30 -	This field is reserved. Reserved
29–27 OCRAM_S_TZ_ADDR_LOCK	Lock Bits
26 OCRAM_S_TZ_EN_LOCK	Lock Bit
25–22 -	This field is reserved. Reserved
21–17 OCRAM_TZ_ADDR_LOCK	Lock Bits
16 OCRAM_TZ_EN_LOCK	Lock Bit
15–14 -	This field is reserved. Reserved
13–11 OCRAM_S_TZ_ADDR	State Retention OCRM TrustZone (TZ) start address. This is the start address of the secure memory region within the State Retention OCRM memory space is 4KB granularity. The start address affects the State Retention OCRM transactions only if OCRM_S_TZ_EN bit is set. The State Retention OCRM TZ ENDADDR is not configurable and is set to the end of State Retention OCRM memory space. These are "lock" type bits
10 OCRAM_S_TZ_EN	State Retention OCRM TrustZone (TZ) enable This is a "lock" type bit 0 The TrustZone feature is disabled. Entire State Retention OCRM space is available for all access types (secure/non-secure/user/supervisor). 1 The TrustZone feature is enabled. Access to address in the range specified by [ENDADDR:STARTADDR] follows the execution mode access policy described in CSU chapter.
9–6 -	This field is reserved. Reserved
5–1 OCRAM_TZ_ADDR	OCRAM TrustZone (TZ) start address. This is the start address of the secure memory region within the OCRM memory space is 4KB granularity. The start address affects the OCRM transactions only if OCRM_TZ_EN bit is set. The OCRM TZ ENDADDR is not configurable and is set to the end of OCRM memory space. These are "lock" type bits
0 OCRAM_TZ_EN	OCRAM TrustZone (TZ) enable This is a "lock" type bit 0 The TrustZone feature is disabled. Entire OCRM space is available for all access types (secure/non-secure/user/supervisor). 1 The TrustZone feature is enabled. Access to address in the range specified by [ENDADDR:STARTADDR] follows the execution mode access policy described in CSU chapter.

8.2.4.13 GPR12 General Purpose Register (IOMUXC_GPR_GPR12)

GPR Register

Address: 3034_0000h base + 30h offset = 3034_0030h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16		
R	PCIE_DIAG_BUS_SEL			PCIE2_CTRL_DIAG_CTRL_BUS			PCIE2_CTRL_DIAG_STATUS_BUS_SELECT			Reserved		PCIE1_CTRL_DIAG_CTRL_BUS			PCIE1_CTRL_DIAG_STATUS_BUS_SELECT			Reserved
W																		
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PCIE1_CTRL_DEVICE_TYPE						PCIE2_CTRL_DEVICE_TYPE				Reserved					
W																
Reset	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR12 field descriptions

Field	Description
31 PCIE_DIAG_BUS_SEL	Control the source of the PCIE DIAG STATUS bus, PCIe1 or PCIe2.
30–29 PCIE2_CTRL_DIAG_CTRL_BUS	PCI Express Diagnostic Control Bus
28–25 PCIE2_CTRL_DIAG_STATUS_BUS_SELECT	PCI Express Diagnostic Status Bus Select
24–23 -	This field is reserved. Reserved

Table continues on the next page...

IOMUXC_GPR_GPR12 field descriptions (continued)

Field	Description
22–21 PCIE1_CTRL_ DIAG_CTRL_ BUS	PCI Express Diagnostic Control Bus
20–17 PCIE1_CTRL_ DIAG_STATUS_ BUS_SELECT	PCI Express Diagnostic Status Bus Select
16 -	This field is reserved. Reserved
15–12 PCIE1_CTRL_ DEVICE_TYPE	PCI Express device/port type 0000 PCI Express endpoint 0001 Legacy PCI Express endpoint 0100 Root port of PCI Express root complex
11–8 PCIE2_CTRL_ DEVICE_TYPE	PCI Express device/port type 0000 PCI Express endpoint 0001 Legacy PCI Express endpoint 0100 Root port of PCI Express root complex
-	This field is reserved. Reserved

8.2.4.14 GPR13 General Purpose Register (IOMUXC_GPR_GPR13)

GPR Register

Address: 3034_0000h base + 34h offset = 3034_0034h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved	AWCACHE_PCIE2	ARCACHE_PCIE2	ARCACHE_LCDIF_EN	AWCACHE_PCIE1_EN	ARCACHE_PCIE1_EN	AWCACHE_PCIE2_EN	ARCACHE_PCIE2_EN	Reserved	ARCACHE_LCDIF	AWCACHE_PCIE1	ARCACHE_PCIE1	MIPI_MUX_INV	MIPI_MUX_SEL	AWCACHE_USDHC	ARCACHE_USDHC
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR13 field descriptions

Field	Description
31–15 -	This field is reserved. Reserved
14 AWCACHE_PCIE2	PCIe AXI Master Port AWCACHE Override Value Note: this bit only takes effect when AWCACHE_PCIE_EN = 1 0 Drive PCIe AXI Master Port AWCACHE[1] to 0 1 Drive PCIe AXI Master Port AWCACHE[1] to 1
13 ARCACHE_PCIE2	PCIe AXI Master Port ARCACHE Override Value Note: this bit only takes effect when ARCACHE_PCIE_EN = 1 0 Drive PCIe AXI Master Port ARCACHE[1] to 0 1 Drive PCIe AXI Master Port ARCACHE[1] to 1
12 ARCACHE_LCDIF_EN	LCDIF AXI Master Port ARCACHE Override Enable

Table continues on the next page...

IOMUXC_GPR_GPR13 field descriptions (continued)

Field	Description
	0 LCDIF AXI Master Port ARCACHE[1] driven by LCDIF 1 LCDIF AXI Master Port ARCACHE[1] driven to constant value specified by the ARCACHE_LCDIF bit
11 AWCACHE_ PCIE1_EN	PCle AXI Master Port AWCACHE Override Enable 0 PCle AXI Master Port AWCACHE[1] driven by PCle 1 PCle AXI Master Port AWCACHE[1] driven to constant value specified by the AWCACHE_PCIE1 bit
10 ARCACHE_ PCIE1_EN	PCle AXI Master Port ARCACHE Override Enable 0 PCle AXI Master Port ARCACHE[1] driven by PCle 1 PCle AXI Master Port ARCACHE[1] driven to constant value specified by the ARCACHE_PCIE1 bit
9 AWCACHE_ PCIE2_EN	PCle AXI Master Port AWCACHE Override Enable 0 PCIE Primary AXI Master Port AWCACHE[1] driven by PCIE 1 PCIE Primary AXI Master Port AWCACHE[1] driven to constant value specified by the AWCACHE_PCIE2 bit
8 ARCACHE_ PCIE2_EN	PCle AXI Master Port ARCACHE Override Enable 0 PCIE Primary AXI Master Port ARCACHE[1] driven by PCIE 1 PCIE Primary AXI Master Port ARCACHE[1] driven to constant value specified by the ARCACHE_PCIE2 bit
7 -	This field is reserved. Reserved
6 ARCACHE_ LCDIF	LCDIF AXI Master Port ARCACHE Override Value Note: this bit only takes effect when ARCACHE_LCDIF_EN = 1 0 Drive LCDIF AXI Master Port ARCACHE[1] to 0 1 Drive LCDIF AXI Master Port ARCACHE[1] to 1
5 AWCACHE_ PCIE1	PCle AXI Master Port AWCACHE Override Value Note: this bit only takes effect when AWCACHE_PCIE_EN = 1 0 Drive PCle AXI Master Port AWCACHE[1] to 0 1 Drive PCle AXI Master Port AWCACHE[1] to 1
4 ARCACHE_ PCIE1	PCle AXI Master Port ARCACHE Override Value Note: this bit only takes effect when ARCACHE_PCIE_EN = 1 0 Drive PCle AXI Master Port ARCACHE[1] to 0 1 Drive PCle AXI Master Port ARCACHE[1] to 1
3 MIPI_MUX_INV	MIPI MUX INV
2 MIPI_MUX_SEL	MIPI MUX SEL
1 AWCACHE_ USDHC	USDHC 1-3 AXI Master AWCACHE Override Value Note: this bit always overrides 0 Drive USDHC AXI Master AWCACHE[1] to 0 1 Drive USDHC AXI Master AWCACHE[1] to 1

Table continues on the next page...

IOMUXC_GPR_GPR13 field descriptions (continued)

Field	Description
0 ARCCACHE_ USDHC	USDHC 1-3 AXI Master ARCCACHE Override Value Note: this bit always overrides 0 Drive USDHC AXI Master ARCCACHE[1] to 0 1 Drive USDHC AXI Master ARCCACHE[1] to 1

8.2.4.15 GPR14 General Purpose Register (IOMUXC_GPR_GPR14)**GPR Register**

Address: 3034_0000h base + 38h offset = 3034_0038h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PCIE1_PHY_LOS_BIAS			PCIE1_PHY_LOS_LEVEL					PCIE1_PHY_RX0_EQ			PCIE1_PHY_TX0_TERM_OFFSET				
W																
Reset	0	1	0	0	1	0	0	1	0	1	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PCIE1_PHY_TX_VBOOST_LVL			PCIE1_VREG_BYPASS	PCIE1_CLKREQ_B_OVERRIDE	PCIE1_CLKREQ_B_OVERRIDE_EN	PCIE1_REF_USE_PAD	PCIE1_APP_CLK_PM_EN	Reserved							
W																
Reset	1	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR14 field descriptions

Field	Description
31–29 PCIE1_PHY_LOS_BIAS	To PCIe PHY
28–24 PCIE1_PHY_LOS_LEVEL	To PCIe PHY

Table continues on the next page...

IOMUXC_GPR_GPR14 field descriptions (continued)

Field	Description
23–21 PCIE1_PHY_ RX0_EQ	To PCIe PHY
20–16 PCIE1_PHY_ TX0_TERM_ OFFSET	To PCIe PHY
15–13 PCIE1_PHY_TX_ VBOOST_LVL	To PCIe PHY
12 PCIE1_VREG_ BYPASS	To PCIe PHY If the PCIe externally-provided PCIE_VPH is at 3.3V, then it is necessary to use the internal voltage regulator (1.8V). To use this internal voltage regulator, this bitfield should be set to 0. 0 Use regulator 1 Bypass regulator
11 PCIE1_ CLKREQ_B_ OVERRIDE	Control the PCIE_CLKREQ_B to the pad together with CLKREQ_B from controller
10 PCIE1_ CLKREQ_B_ OVERRIDE_EN	Control the PCIE_CLKREQ_B to the pad together with CLKREQ_B from controller
9 PCIE1_REF_ USE_PAD	To PCIe PHY
8 PCIE1_APP_ CLK_PM_EN	To PCIe PHY
-	This field is reserved. Reserved

8.2.4.16 GPR15 General Purpose Register (IOMUXC_GPR_GPR15)

GPR Register

Address: 3034_0000h base + 3Ch offset = 3034_003Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W	PCIE1_PCS_TX_ DEEMPH_GEN1						PCIE1_PCS_TX_ DEEMPH_GEN2_ 3P5DB						PCIE1_PCS_TX_ DEEMPH_GEN2_ 6DB						PCIE1_PCS_TX_ SWING_FULL						PCIE1_PCS_TX_ SWING_LOW							
Reset	0	1	1	0	0	0	0	1	1	0	0	0	1	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

IOMUXC_GPR_GPR15 field descriptions

Field	Description
31–26 PCIE1_PCS_TX_ DEEMPH_GEN1	To PCIe PHY
25–20 PCIE1_PCS_TX_ DEEMPH_ GEN2_3P5DB	To PCIe PHY
19–14 PCIE1_PCS_TX_ DEEMPH_ GEN2_6DB	To PCIe PHY
13–7 PCIE1_PCS_TX_ SWING_FULL	To PCIe PHY
PCIE1_PCS_TX_ SWING_LOW	To PCIe PHY

8.2.4.17 GPR16 General Purpose Register (IOMUXC_GPR_GPR16)**GPR Register**

Address: 3034_0000h base + 40h offset = 3034_0040h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PCIE2_PHY_LOS_ BIAS			PCIE2_PHY_LOS_LEVEL					PCIE2_PHY_RX0_ EQ			PCIE2_PHY_TX0_TERM_OFFSET				
W																
Reset	0	1	0	0	1	0	0	1	0	1	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PCIE2_PHY_TX_ VBOOST_LVL			PCIE2_VREG_BYPASS	PCIE2_CLKREQ_B_ OVERRIDE	PCIE2_CLKREQ_B_ OVERRIDE_EN	PCIE2_REF_USE_PAD	PCIE2_APP_CLK_PM_ EN	Reserved							
W																
Reset	1	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR16 field descriptions

Field	Description
31–29 PCIE2_PHY_ LOS_BIAS	To PCIe PHY
28–24 PCIE2_PHY_ LOS_LEVEL	To PCIe PHY
23–21 PCIE2_PHY_ RX0_EQ	To PCIe PHY
20–16 PCIE2_PHY_ TX0_TERM_ OFFSET	To PCIe PHY
15–13 PCIE2_PHY_TX_ VBOOST_LVL	To PCIe PHY
12 PCIE2_VREG_ BYPASS	To PCIe PHY If the PCIe externally-provided PCIE_VPH is at 3.3V, then it is necessary to use the internal voltage regulator (1.8V). To use this internal voltage regulator, this bitfield should be set to 0. 0 Use regulator 1 Bypass regulator
11 PCIE2_ CLKREQ_B_ OVERRIDE	Control the PCIE_CLKREQ_B to the pad together with CLKREQ_B from controller
10 PCIE2_ CLKREQ_B_ OVERRIDE_EN	Control the PCIE_CLKREQ_B to the pad together with CLKREQ_B from controller
9 PCIE2_REF_ USE_PAD	To PCIe PHY
8 PCIE2_APP_ CLK_PM_EN	To PCIe PHY
-	This field is reserved. Reserved

8.2.4.18 GPR17 General Purpose Register (IOMUXC_GPR_GPR17)

GPR Register

Address: 3034_0000h base + 44h offset = 3034_0044h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W	PCIE2_PCS_TX_DEEMPH_GEN1						PCIE2_PCS_TX_DEEMPH_GEN2_3P5DB						PCIE2_PCS_TX_DEEMPH_GEN2_6DB						PCIE2_PCS_TX_SWING_FULL						PCIE2_PCS_TX_SWING_LOW							
Reset	0	1	1	0	0	0	0	1	1	0	0	0	1	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

IOMUXC_GPR_GPR17 field descriptions

Field	Description
31–26 PCIE2_PCS_TX_ DEEMPH_GEN1	To PCIe PHY
25–20 PCIE2_PCS_TX_ DEEMPH_ GEN2_3P5DB	To PCIe PHY
19–14 PCIE2_PCS_TX_ DEEMPH_ GEN2_6DB	To PCIe PHY
13–7 PCIE2_PCS_TX_ SWING_FULL	To PCIe PHY
PCIE2_PCS_TX_ SWING_LOW	To PCIe PHY

8.2.4.19 GPR18 General Purpose Register (IOMUXC_GPR_GPR18)

GPR Register

Address: 3034_0000h base + 48h offset = 3034_0048h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved																															
W	Reserved																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

IOMUXC_GPR_GPR18 field descriptions

Field	Description
-	This field is reserved. Reserved

8.2.4.20 GPR19 General Purpose Register (IOMUXC_GPR_GPR19)

GPR Register

Address: 3034_0000h base + 4Ch offset = 3034_004Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PCIE_DIAG_STATUS																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR19 field descriptions

Field	Description
PCIE_DIAG_STATUS	PCIe DIAG Status Bus

8.2.4.21 GPR20 General Purpose Register (IOMUXC_GPR_GPR20)

GPR Register

Address: 3034_0000h base + 50h offset = 3034_0050h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR20 field descriptions

Field	Description
-	This field is reserved. Reserved

8.2.4.22 GPR21 General Purpose Register (IOMUXC_GPR_GPR21)

GPR Register

Address: 3034_0000h base + 54h offset = 3034_0054h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR21 field descriptions

Field	Description
-	This field is reserved. Reserved

8.2.4.23 GPR22 General Purpose Register (IOMUXC_GPR_GPR22)

GPR Register

Address: 3034_0000h base + 58h offset = 3034_0058h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								CPU_				CPU_				Reserved															
W									STANDBYWF				STANDBYWF																			
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR22 field descriptions

Field	Description
31–24 -	This field is reserved. Reserved
23–20 CPU_	Status of CPU STANDBYWFE low power states
STANDBYWFE	MSB: status of core 3 STANDBYWFE low power state LSB: status of core 0 STANDBYWFE low power state
19–16 CPU_	Status of CPU STANDBYWF I low power states
STANDBYWF I	MSB: status of core 3 STANDBYWF I low power state LSB: status of core 0 STANDBYWF I low power state
-	This field is reserved. Reserved

8.2.4.24 GPR23 General Purpose Register (IOMUXC_GPR_GPR23)

GPR Register

Address: 3034_0000h base + 5Ch offset = 3034_005Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DDSI_DPHY_ TURNAROUND	DSI_TRIGGER_ REQ	DSI_ TRIGGER_ SEND		DSI_TX_ULPS_ENABLE				DSI_ HSEL		DSI_NOCAL	DSI_RCALT		DSI_RTERM_SEL	DSI_RX_ RCAL	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR23 field descriptions

Field	Description
31–16 -	This field is reserved. Reserved
15 DDSI_DPHY_ TURNAROUND	For DSI Control
14 DSI_TRIGGER_ REQ	For DSI Control
13–12 DSI_TRIGGER_ SEND	For DSI Control
11–7 DSI_TX_ULPS_ ENABLE	For DSI Control
6 DSI_HSEL	For DSI Control
5 DSI_NOCAL	For DSI Control
4–3 DSI_RCALT	For DSI Control

Table continues on the next page...

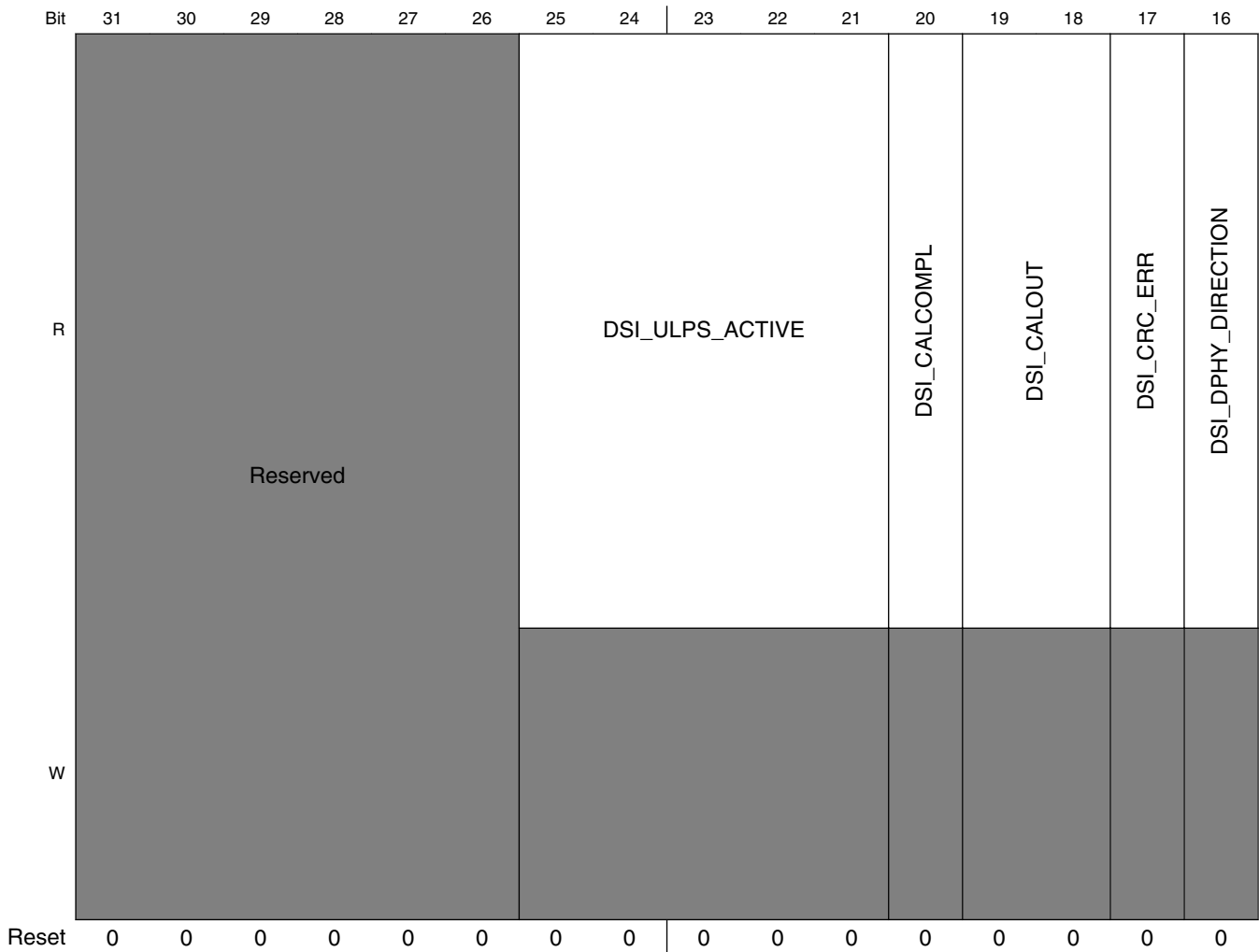
IOMUXC_GPR_GPR23 field descriptions (continued)

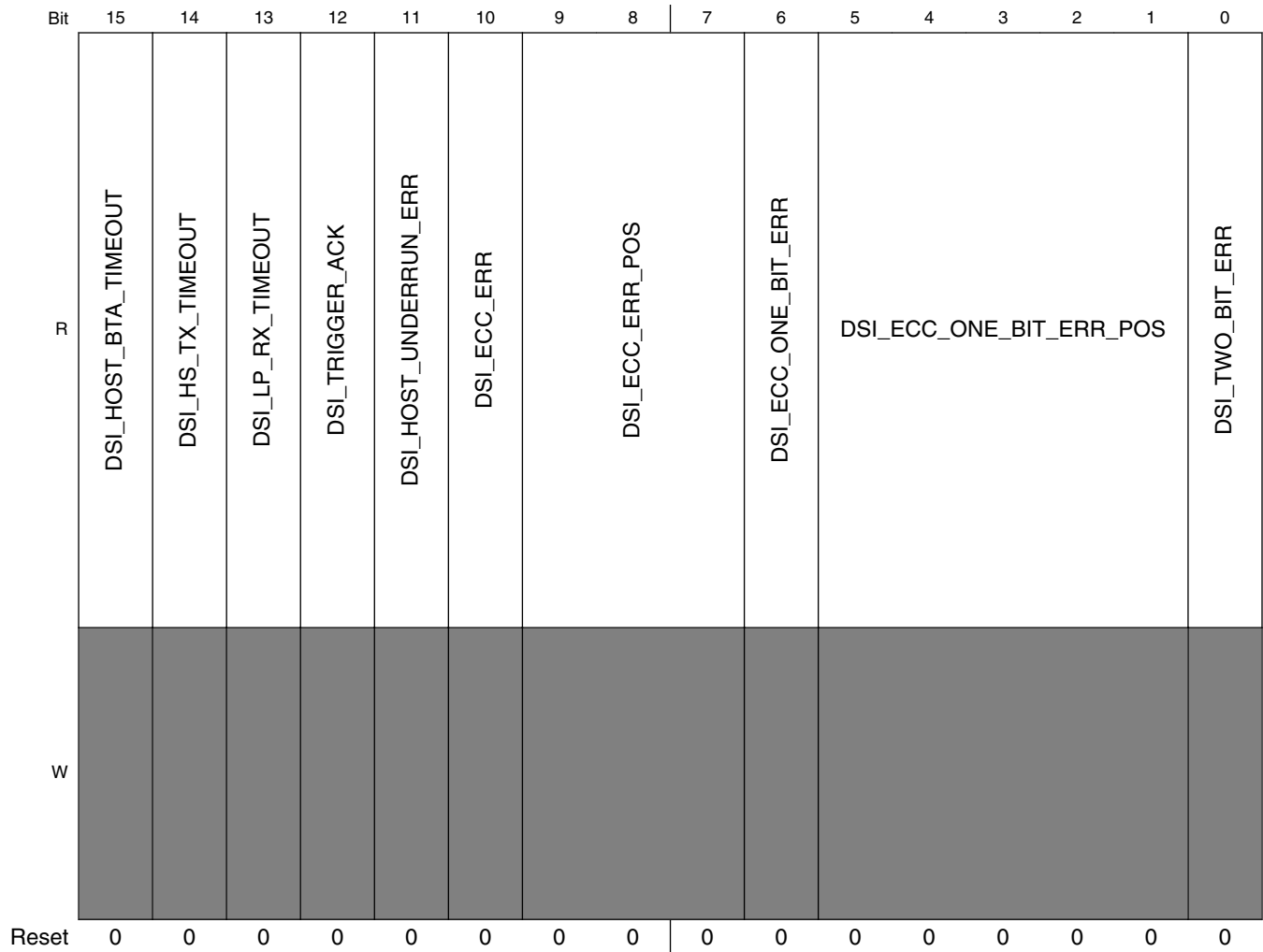
Field	Description
2 DSI_RTERM_ SEL	For DSI Control
DSI_RX_RCAL	For DSI Control

8.2.4.25 GPR24 General Purpose Register (IOMUXC_GPR_GPR24)

GPR Register

Address: 3034_0000h base + 60h offset = 3034_0060h



**IOMUXC_GPR_GPR24 field descriptions**

Field	Description
31–26 -	This field is reserved. Reserved
25–21 DSI_ULPS_ ACTIVE	for DSI status
20 DSI_CALCOMPL	for DSI status
19–18 DSI_CALOUT	for DSI status
17 DSI_CRC_ERR	for DSI status
16 DSI_DPHY_ DIRECTION	for DSI status

Table continues on the next page...

IOMUXC_GPR_GPR24 field descriptions (continued)

Field	Description
15 DSI_HOST_ BTA_TIMEOUT	for DSI status
14 DSI_HS_TX_ TIMEOUT	for DSI status
13 DSI_LP_RX_ TIMEOUT	for DSI status
12 DSI_TRIGGER_ ACK	for DSI status
11 DSI_HOST_ UNDERRUN_ ERR	for DSI status
10 DSI_ECC_ERR	for DSI status
9–7 DSI_ECC_ERR_ POS	for DSI status
6 DSI_ECC_ONE_ BIT_ERR	for DSI status
5–1 DSI_ECC_ONE_ BIT_ERR_POS	for DSI status
0 DSI_TWO_BIT_ ERR	for DSI status

8.2.4.26 GPR25 General Purpose Register (IOMUXC_GPR_GPR25)

GPR Register

Address: 3034_0000h base + 64h offset = 3034_0064h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved		DSI_UI_STATUS3_RO													
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DSI_UI_STATUS3_RO															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR25 field descriptions

Field	Description
31–30 -	This field is reserved. Reserved
DSI_UI_STATUS3_RO	for DSI status: host_ui_status[125:96]

8.2.4.27 GPR26 General Purpose Register (IOMUXC_GPR_GPR26)

GPR Register

Address: 3034_0000h base + 68h offset = 3034_0068h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DSI_UI_STATUS2_RO																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR26 field descriptions

Field	Description
DSI_UI_STATUS2_RO	for DSI status: host_ui_status[95:64]

8.2.4.28 GPR27 General Purpose Register (IOMUXC_GPR_GPR27)

GPR Register

Address: 3034_0000h base + 6Ch offset = 3034_006Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DSI_UI_STATUS1_RO																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR27 field descriptions

Field	Description
DSI_UI_STATUS1_RO	for DSI status: host_ui_status[63:32]

8.2.4.29 GPR28 General Purpose Register (IOMUXC_GPR_GPR28)

GPR Register

Address: 3034_0000h base + 70h offset = 3034_0070h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DSI_UI_STATUS0_RO																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

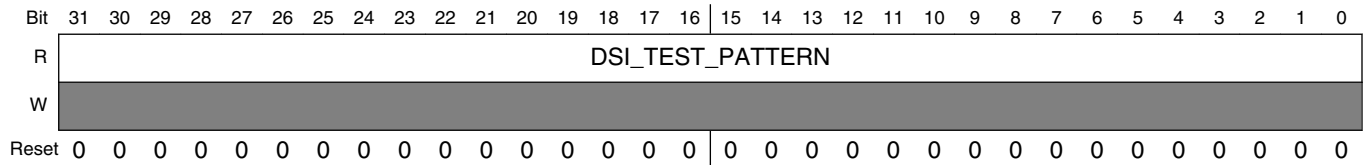
IOMUXC_GPR_GPR28 field descriptions

Field	Description
DSI_UI_STATUS0_RO	for DSI status: host_ui_status[31:0]

8.2.4.30 GPR29 General Purpose Register (IOMUXC_GPR_GPR29)

GPR Register

Address: 3034_0000h base + 74h offset = 3034_0074h



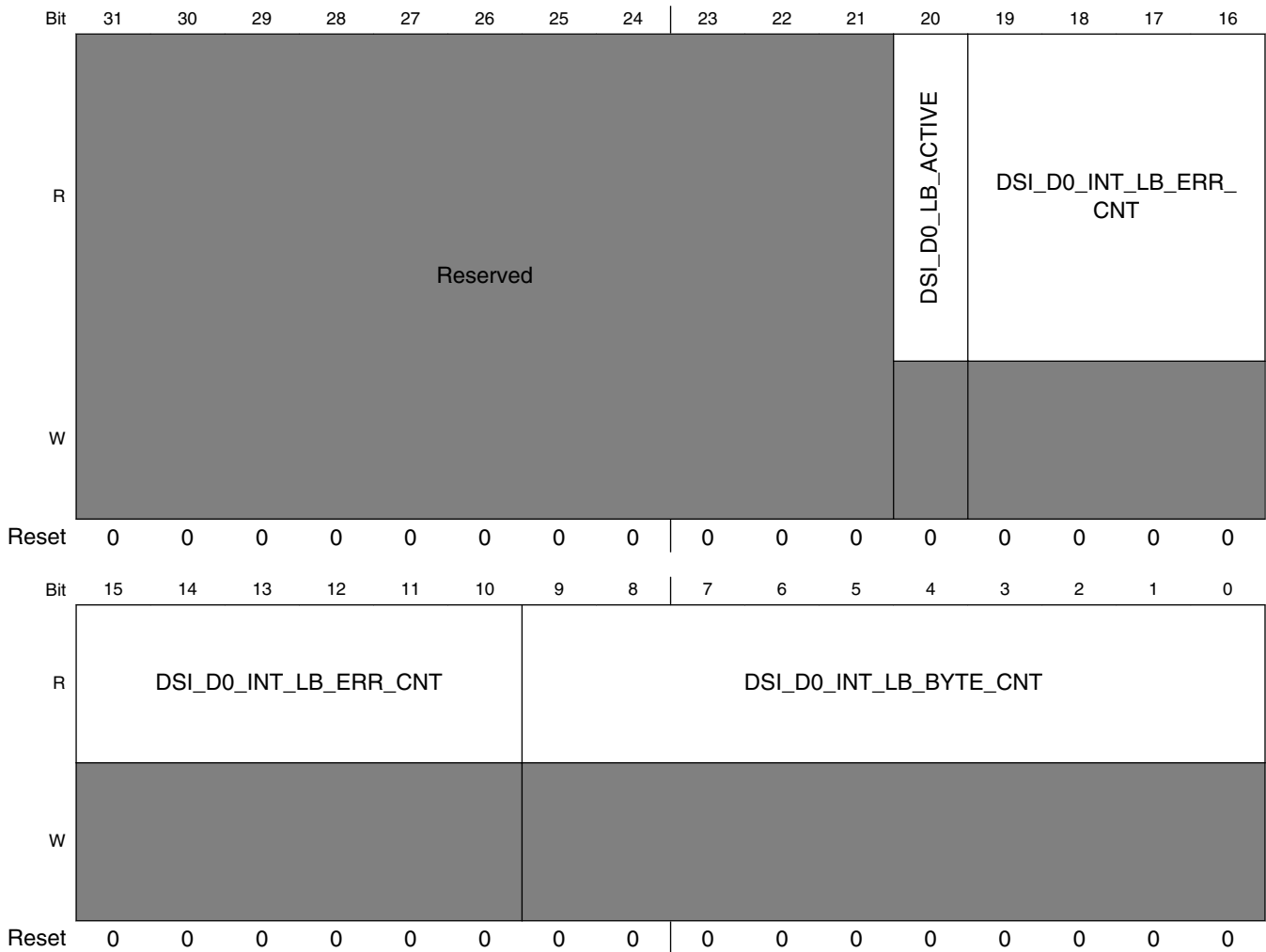
IOMUXC_GPR_GPR29 field descriptions

Field	Description
DSI_TEST_PATTERN	for DSI test control

8.2.4.31 GPR30 General Purpose Register (IOMUXC_GPR_GPR30)

GPR Register

Address: 3034_0000h base + 78h offset = 3034_0078h



IOMUXC_GPR_GPR30 field descriptions

Field	Description
31–21 -	This field is reserved. Reserved
20 DSI_D0_LB_ACTIVE	for DSI test status
19–10 DSI_D0_INT_LB_ERR_CNT	for DSI test status

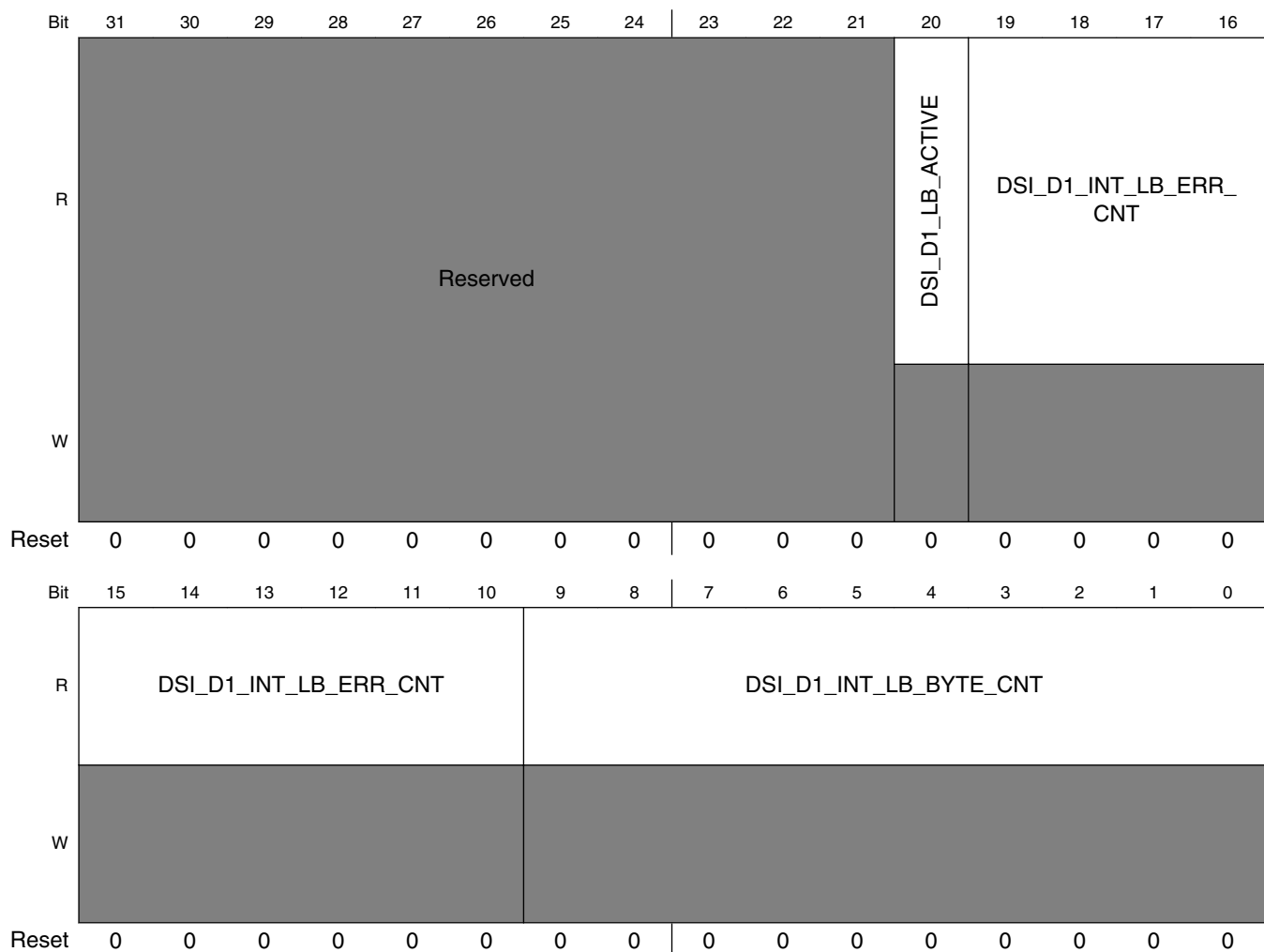
Table continues on the next page...

IOMUXC_GPR_GPR30 field descriptions (continued)

Field	Description
DSI_D0_INT_LB_BYTE_CNT	for DSI test status

8.2.4.32 GPR31 General Purpose Register (IOMUXC_GPR_GPR31)**GPR Register**

Address: 3034_0000h base + 7Ch offset = 3034_007Ch



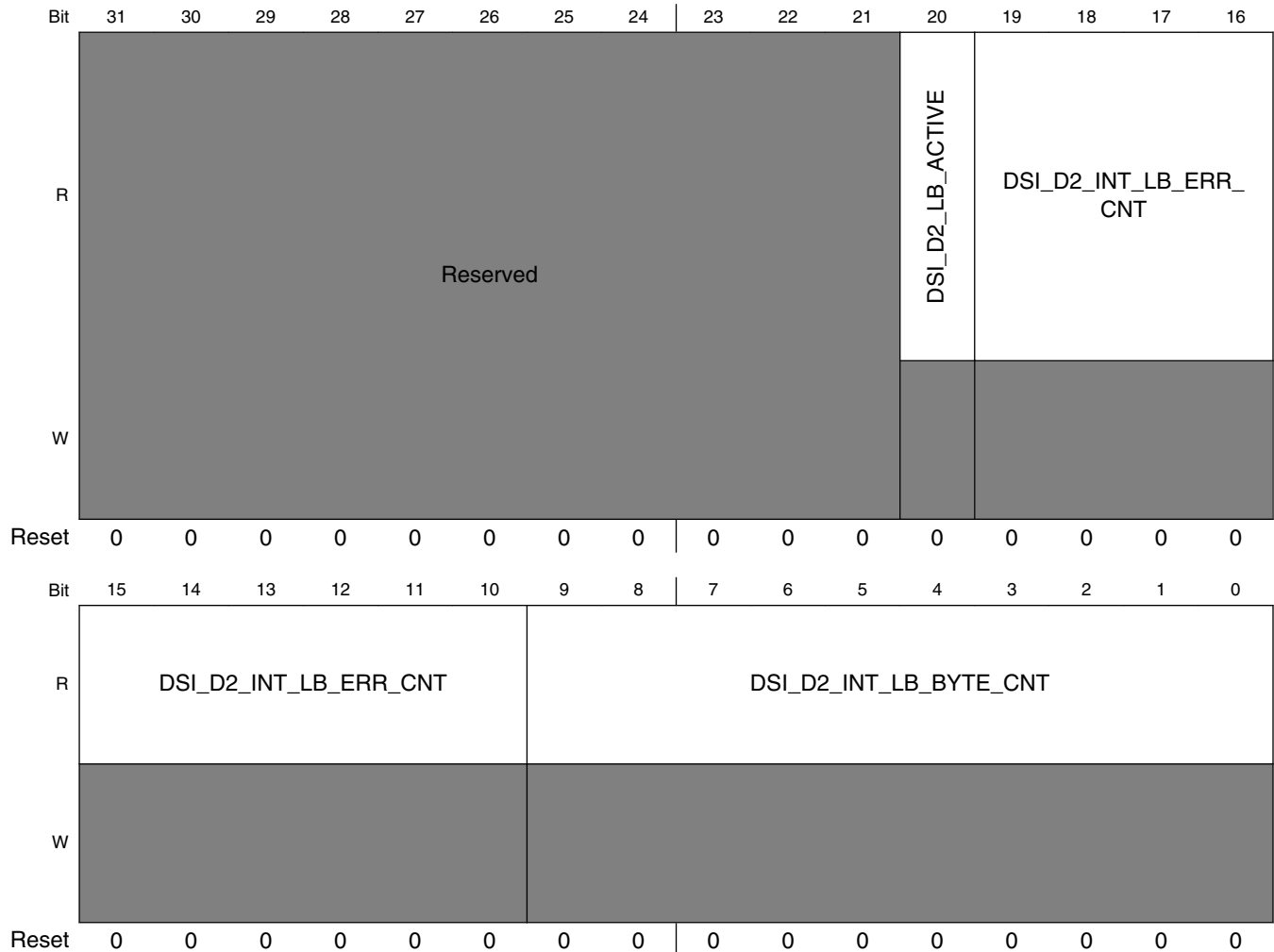
IOMUXC_GPR_GPR31 field descriptions

Field	Description
31–21 -	This field is reserved. Reserved
20 DSI_D1_LB_ ACTIVE	for DSI test status
19–10 DSI_D1_INT_ LB_ERR_CNT	for DSI test status
DSI_D1_INT_ LB_BYTE_CNT	for DSI test status

8.2.4.33 GPR32 General Purpose Register (IOMUXC_GPR_GPR32)

GPR Register

Address: 3034_0000h base + 80h offset = 3034_0080h



IOMUXC_GPR_GPR32 field descriptions

Field	Description
31–21 -	This field is reserved. Reserved
20 DSI_D2_LB_ACTIVE	for DSI test status
19–10 DSI_D2_INT_LB_ERR_CNT	for DSI test status

Table continues on the next page...

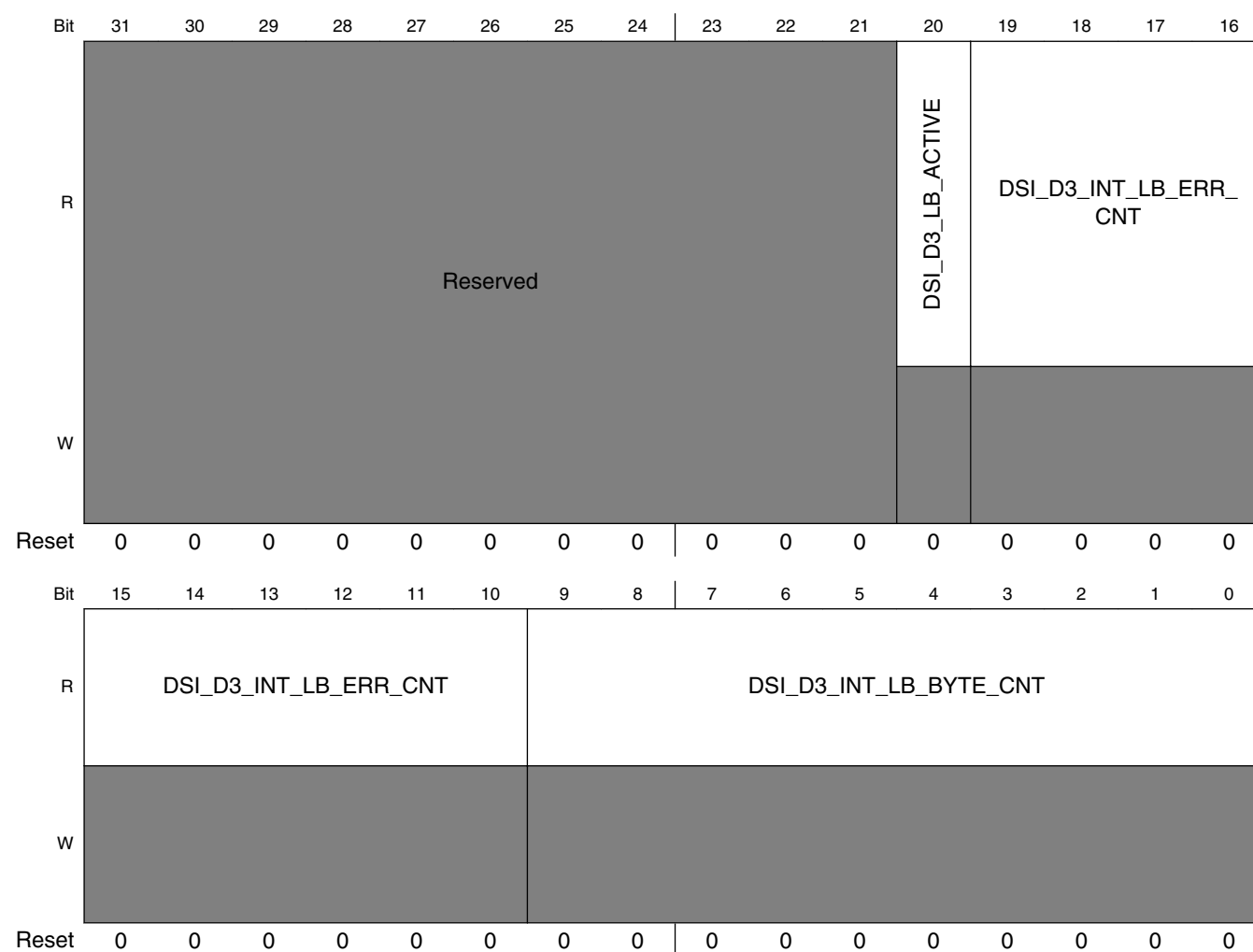
IOMUXC_GPR_GPR32 field descriptions (continued)

Field	Description
DSI_D2_INT_LB_BYTE_CNT	for DSI test status

8.2.4.34 GPR33 General Purpose Register (IOMUXC_GPR_GPR33)

GPR Register

Address: 3034_0000h base + 84h offset = 3034_0084h



IOMUXC_GPR_GPR33 field descriptions

Field	Description
31–21 -	This field is reserved. Reserved
20 DSI_D3_LB_ ACTIVE	for DSI test status
19–10 DSI_D3_INT_ LB_ERR_CNT	for DSI test status
DSI_D3_INT_ LB_BYTE_CNT	for DSI test status

8.2.4.35 GPR34 General Purpose Register (IOMUXC_GPR_GPR34)**GPR Register**

Address: 3034_0000h base + 88h offset = 3034_0088h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		CSI2_1_RX_ENABLE	CSI2_1_VID_INTFC_ENB	CSI2_1_PD_RX	CSI2_1_HSEL	CSI2_1_AUTO_PD_EN	CSI2_1_CONT_CLK_MODE	CSI2_1_S_PRG_RXHS_SETTLE						CSI2_1_RX_RCAL	
W																
Reset	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR34 field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13 CSI2_1_RX_ ENABLE	MIPI CSI2_1 Controller Receive Enable, active high. When deasserted, the RX controller will pause data reception at the next packet boundary. Since the CSI-2 protocol does not allow the Receiver to pause data, when rx_enable is deasserted the RX Controller still receives data from the RX DPHY but it does not forward the receive packets to the user interface but instead discards the received data. When rx_enable

Table continues on the next page...

IOMUXC_GPR_GPR34 field descriptions (continued)

Field	Description
	is asserted, the RX controller will wait for the start of the next packet before allowing data to be sent out over the user interface.
12 CSI2_1_VID_ INTFC_ENB	MIPI CSI2_1 controller video interface enable. 0 Disable 1 Enable
11 CSI2_1_PD_RX	MIPI CSI2_1 D-PHY Power Down input. When high, all blocks are powered down.
10 CSI2_1_HSEL	MIPI CSI2_1 D-PHY High Speed Select 0 1.0 Gbps operation 1 1.2 Gbps operation
9 CSI2_1_AUTO_ PD_EN	MIPI CSI2_1 D-PHY Power down inactive lanes reported by CFG_NUM_LANES input bus. 0 Inactive lanes are powered up and driving LP11. 1 Inactive lanes are powered down.
8 CSI2_1_CONT_ CLK_MODE	MIPI CSI2_1 D-PHY slave clock lane feature enable to maintain HS reception state during continuous clock mode operation. 0 Feature disable 1 Feature enabled
7–2 CSI2_1_S_PRG_ RXHS_SETTLE	MIPI CSI2_1 D-PHY program T_HS_SETTLE bits. HS-RX waits for Time-out T_HS_SETTLE in order to neglect transition effects.
CSI2_1_RX_ RCAL	MIPI CSI2_1 D-PHY On-chip termination control bits for manual calibration of HS-RX. 00 25% higher than mid-range. Highest impedance setting. 01 15% higher than mid-range. 10 Mid-range impedance setting. 11 25% lower than mid-range. Lowest impedance setting.

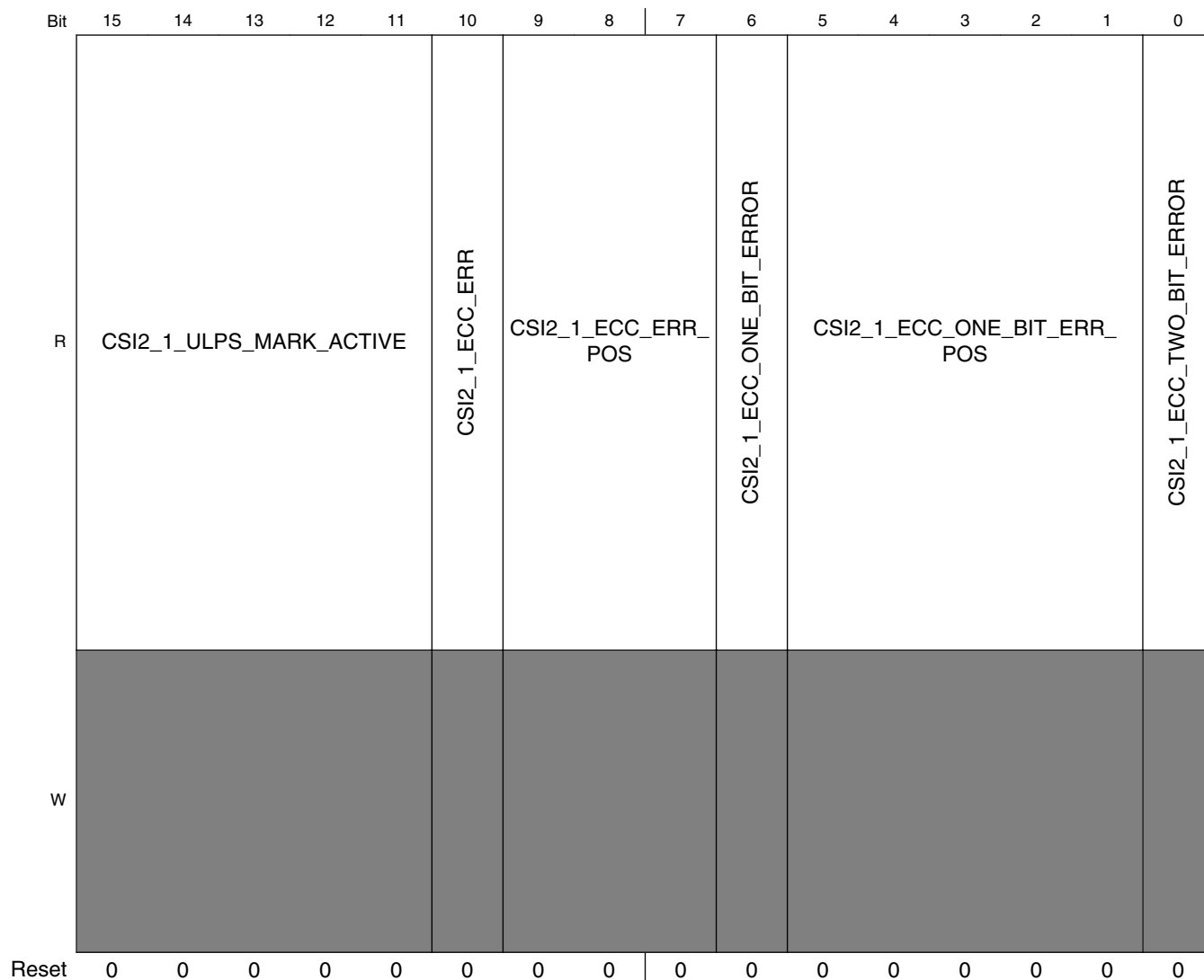
8.2.4.36 GPR35 General Purpose Register (IOMUXC_GPR_GPR35)

GPR Register

Address: 3034_0000h base + 8Ch offset = 3034_008Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved							CSI2_1_ERR_SEND_LEVEL	CSI2_1_ERR_FIFO_WR_OVFL	CSI2_1_ULPS_ACTIVE				CSI2_1_CRC_ERR		CSI2_1_RX_DPHY_RDY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUX Controller (IOMUXC)



IOMUXC_GPR_GPR35 field descriptions

Field	Description
31–25 -	This field is reserved. Reserved
24 CSI2_1_ERR_SEND_LEVEL	For MIPI CSI2_1 Controller, this flag indicates the video interface pixel fifo did not accumulate enough sample to trigger its send level setting before the end of an image data packet was detected. This flag would generally mean that the <code>cfg_vid_p_fifo_send_level</code> is set to high.
23 CSI2_1_ERR_FIFO_WR_OVFL	For MIPI CSI2_1 Controller, this flag indicates the video interface pixel fifo overflowed on a pixel write. This means the output video port is not removing pixels quickly enough to keep up with the input data rate. This flag would generally mean the <code>clk_vid</code> rate is too low.
22–18 CSI2_1_ULPS_ACTIVE	MIPI CSI2_1 Controller Receive Ultra Low Power State active. Bits assert high when corresponding clock or data lane is in ULPS mode. Bit [0] – Clock Lane Bit [1] – Data Lane 0 Bit [2] – Data Lane 1

Table continues on the next page...

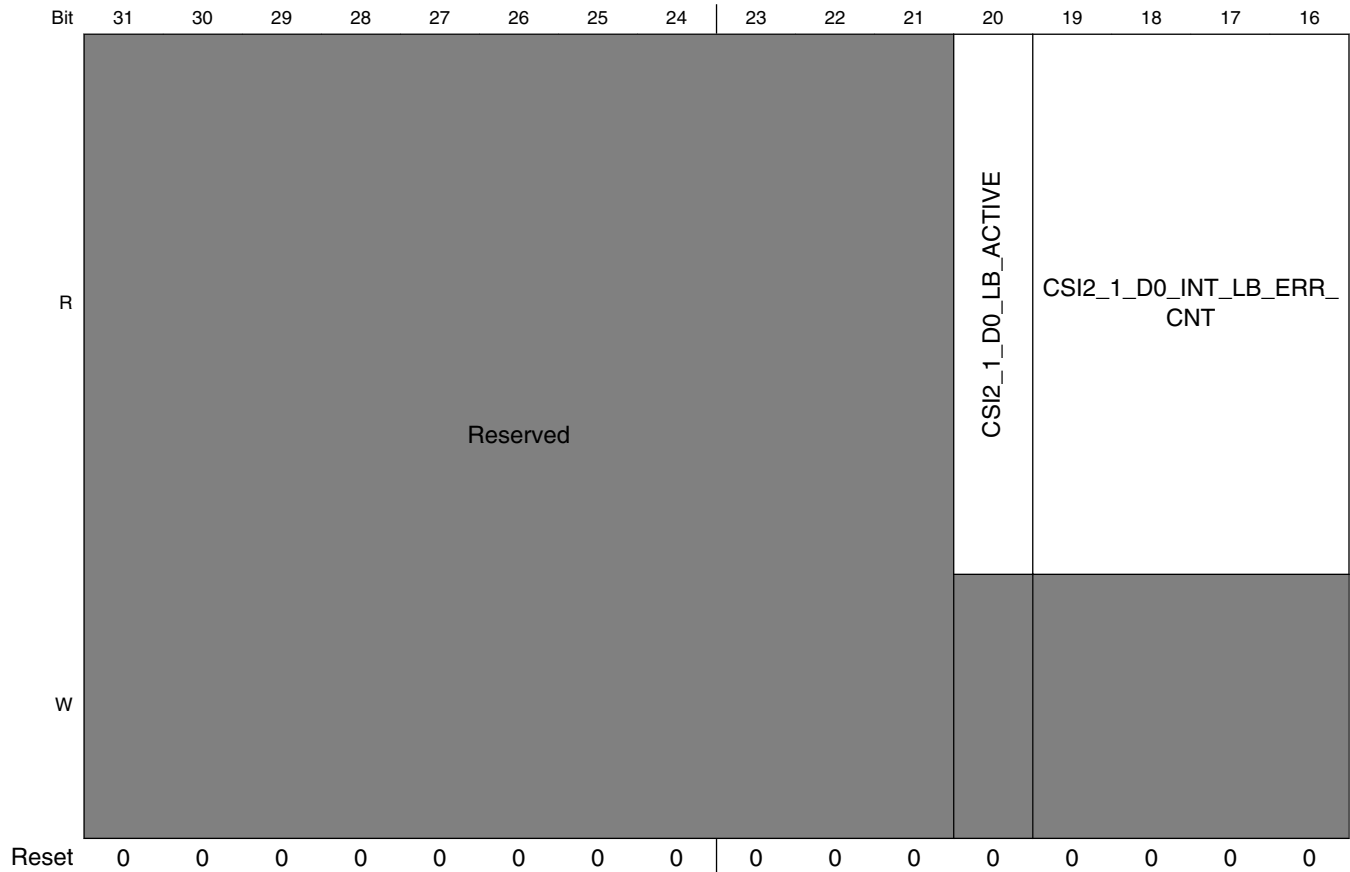
IOMUXC_GPR_GPR35 field descriptions (continued)

Field	Description
	Bit [3] – Data Lane 2 Bit [4] – Data Lane 3
17 CSI2_1_CRC_ERR	MIPI CSI2_1 Controller CRC status. Asserts high when the CRC calculated on the received data does not match the CRC the transmitter sent at the end of the packet. Valid when the eop_out output asserts.
16 CSI2_1_RX_DPHY_RDY	MIPI CSI2_1 D-PHY ready status 0 Not ready 1 Ready
15–11 CSI2_1_ULPS_MARK_ACTIVE	MIPI CSI2_1 Controller Receive Ultra Low Power State Mark status. Bits assert high when corresponding clock or data lane has exited ULPS and entered Mark-1 state. Lanes will remain in Mark-1 state for a minimum of 1ms as per the MIPI DPHY specification. While in Mark-1 state, no other active, like High Speed data transmission, is allowed. Bit [0] – Clock Lane Bit [1] – Data Lane 0 Bit [2] – Data Lane 1 Bit [3] – Data Lane 2 Bit [4] – Data Lane 3
10 CSI2_1_ECC_ERR	MIPI CSI2_1: Error detected in the ECC bits. This signal is no longer supported.
9–7 CSI2_1_ECC_ERR_POS	MIPI CSI2_1: Position of the corrected single bit error in the packet header. Valid when pkt_hdr_valid is asserted high.
6 CSI2_1_ECC_ONE_BIT_ERROR	MIPI CSI2_1: Two packet header bit errors were detected and not corrected, active high and is valid when pkt_hdr_is high.
5–1 CSI2_1_ECC_ONE_BIT_ERR_POS	MIPI CSI2_1: Position of the corrected single bit error in the packet header. Valid when pkt_hdr_valid is asserted high.
0 CSI2_1_ECC_TWO_BIT_ERROR	MIPI CSI2_1: Two packet header bit errors were detected and not corrected, active high and is valid when pkt_hdr_is high.

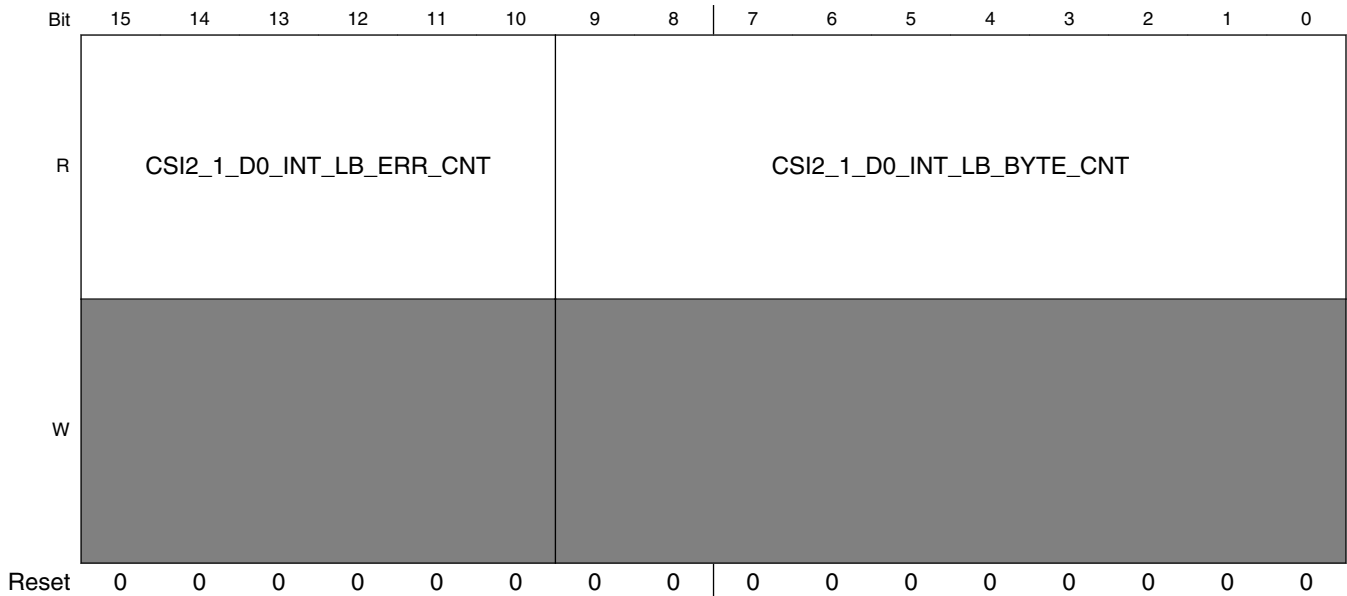
8.2.4.38 GPR37 General Purpose Register (IOMUXC_GPR_GPR37)

GPR Register

Address: 3034_0000h base + 94h offset = 3034_0094h



IOMUX Controller (IOMUXC)



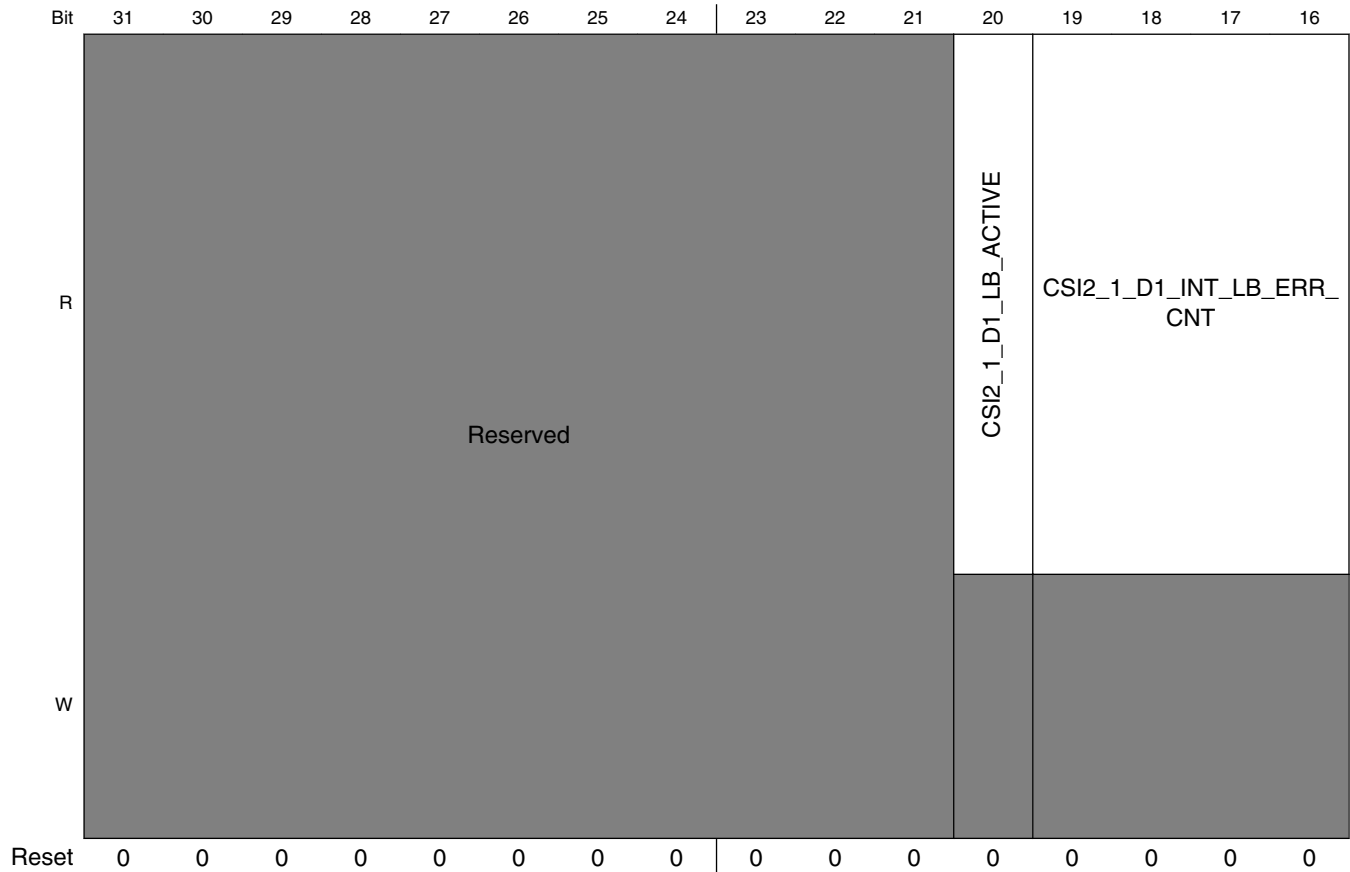
IOMUXC_GPR_GPR37 field descriptions

Field	Description
31–21 -	This field is reserved. Reserved
20 CSI2_1_D0_LB_ACTIVE	for DSI test status
19–10 CSI2_1_D0_INT_LB_ERR_CNT	for DSI test status
CSI2_1_D0_INT_LB_BYTE_CNT	for DSI test status

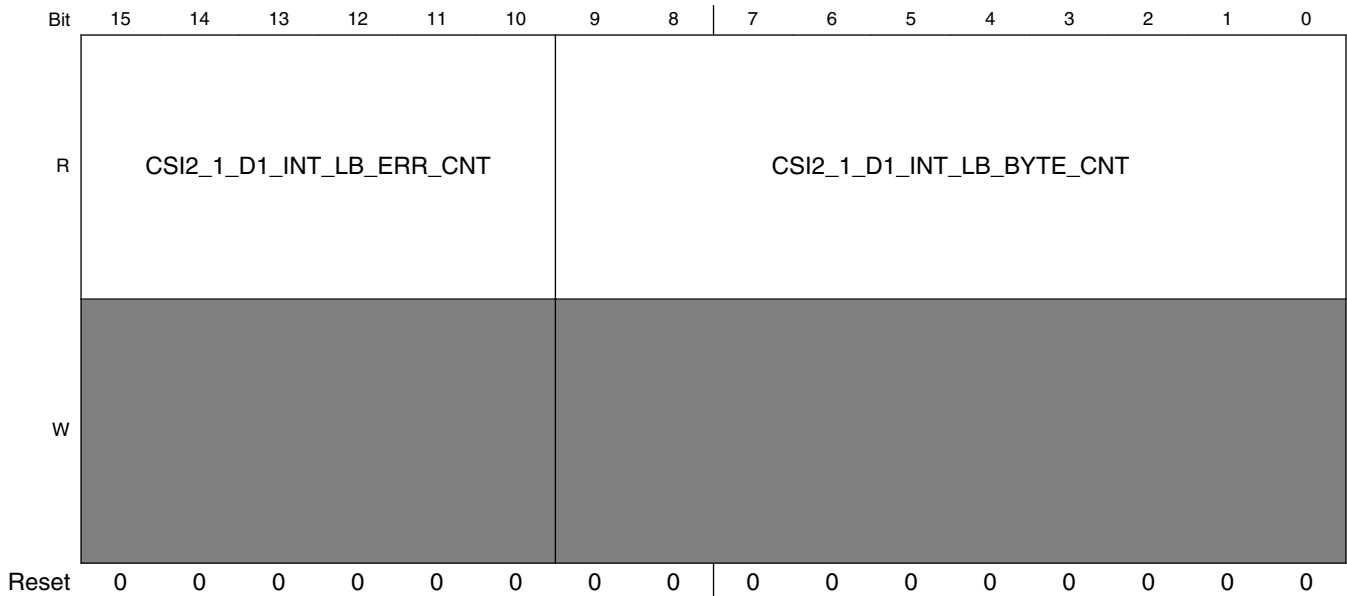
8.2.4.39 GPR38 General Purpose Register (IOMUXC_GPR_GPR38)

GPR Register

Address: 3034_0000h base + 98h offset = 3034_0098h



IOMUX Controller (IOMUXC)



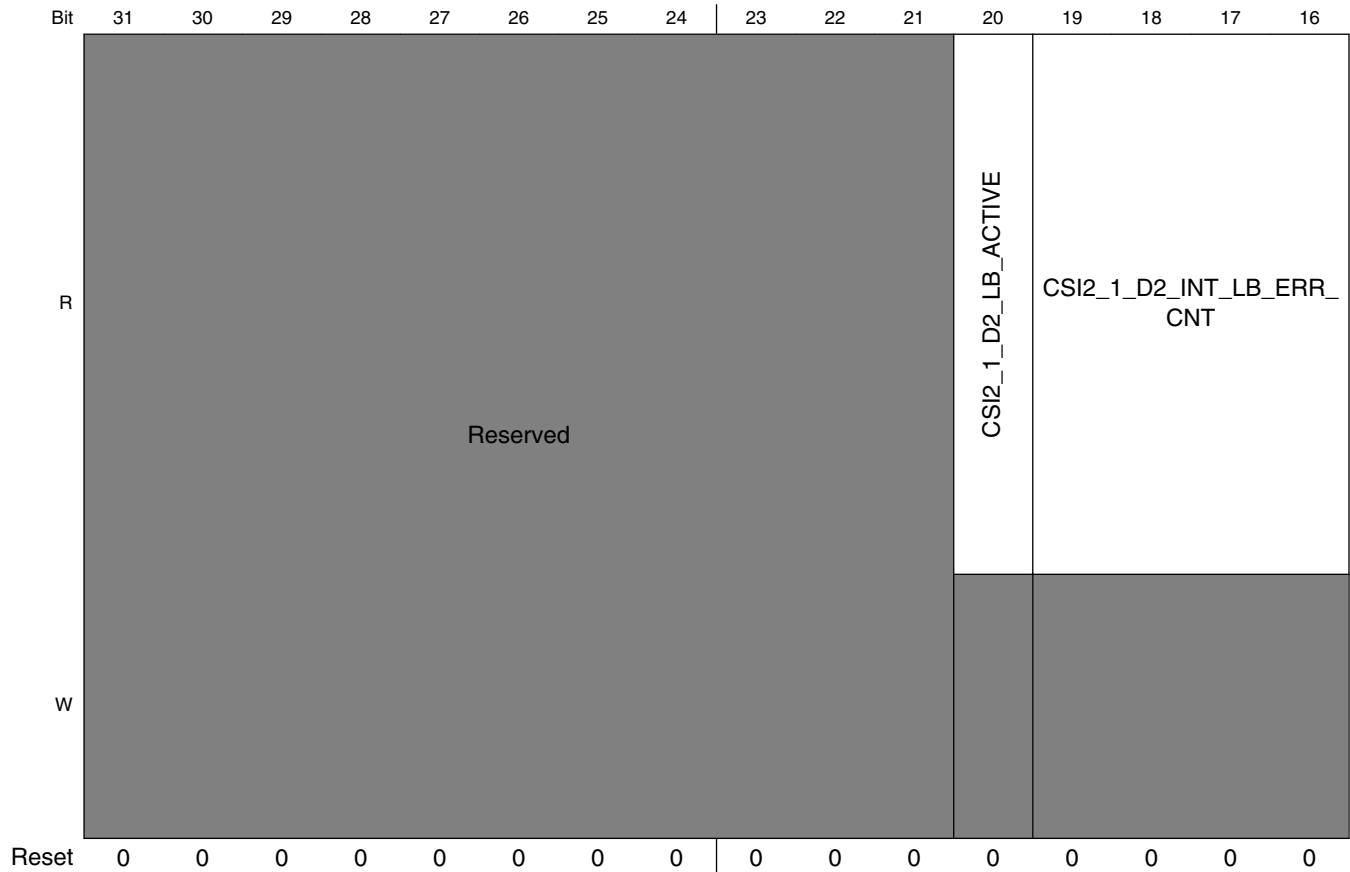
IOMUXC_GPR_GPR38 field descriptions

Field	Description
31–21 -	This field is reserved. Reserved
20 CSI2_1_D1_LB_ACTIVE	for DSI test status
19–10 CSI2_1_D1_INT_LB_ERR_CNT	for DSI test status
CSI2_1_D1_INT_LB_BYTE_CNT	for DSI test status

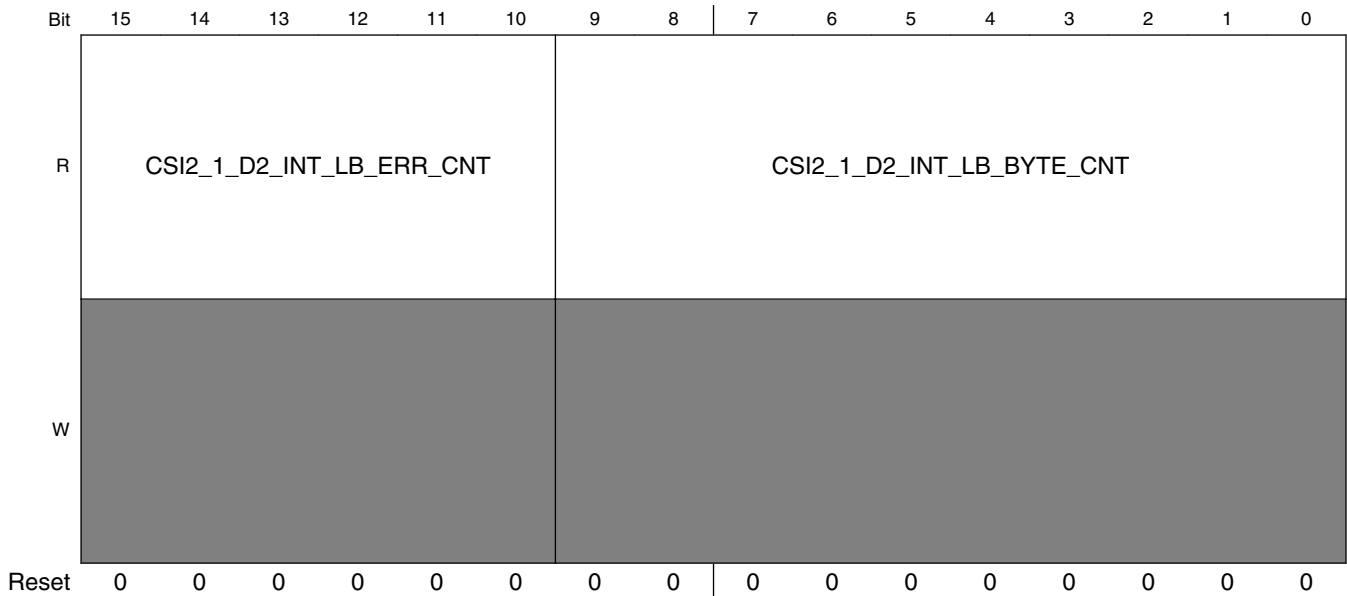
8.2.4.40 GPR39 General Purpose Register (IOMUXC_GPR_GPR39)

GPR Register

Address: 3034_0000h base + 9Ch offset = 3034_009Ch



IOMUX Controller (IOMUXC)



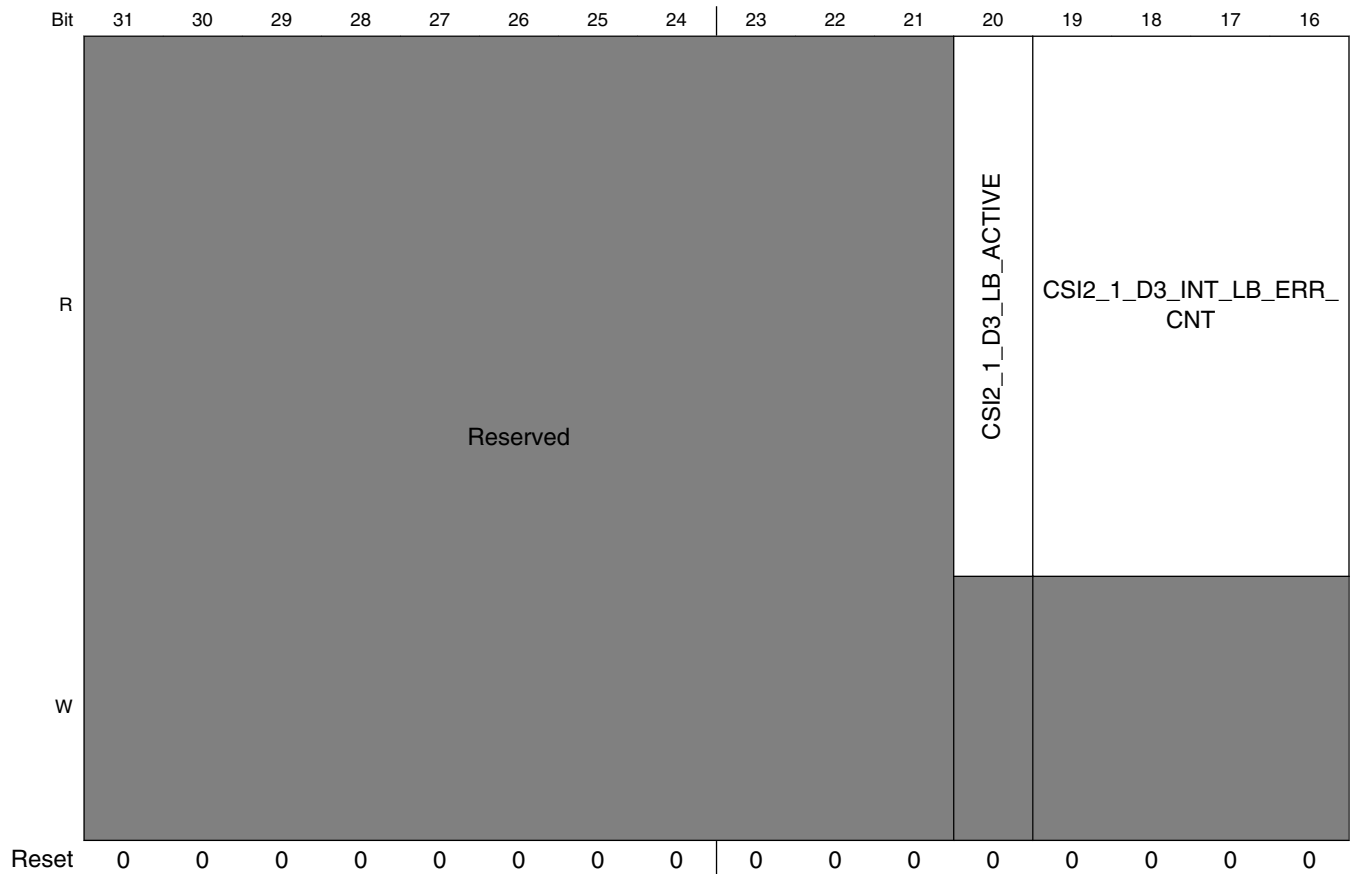
IOMUXC_GPR_GPR39 field descriptions

Field	Description
31–21 -	This field is reserved. Reserved
20 CSI2_1_D2_LB_ACTIVE	for DSI test status
19–10 CSI2_1_D2_INT_LB_ERR_CNT	for DSI test status
CSI2_1_D2_INT_LB_BYTE_CNT	for DSI test status

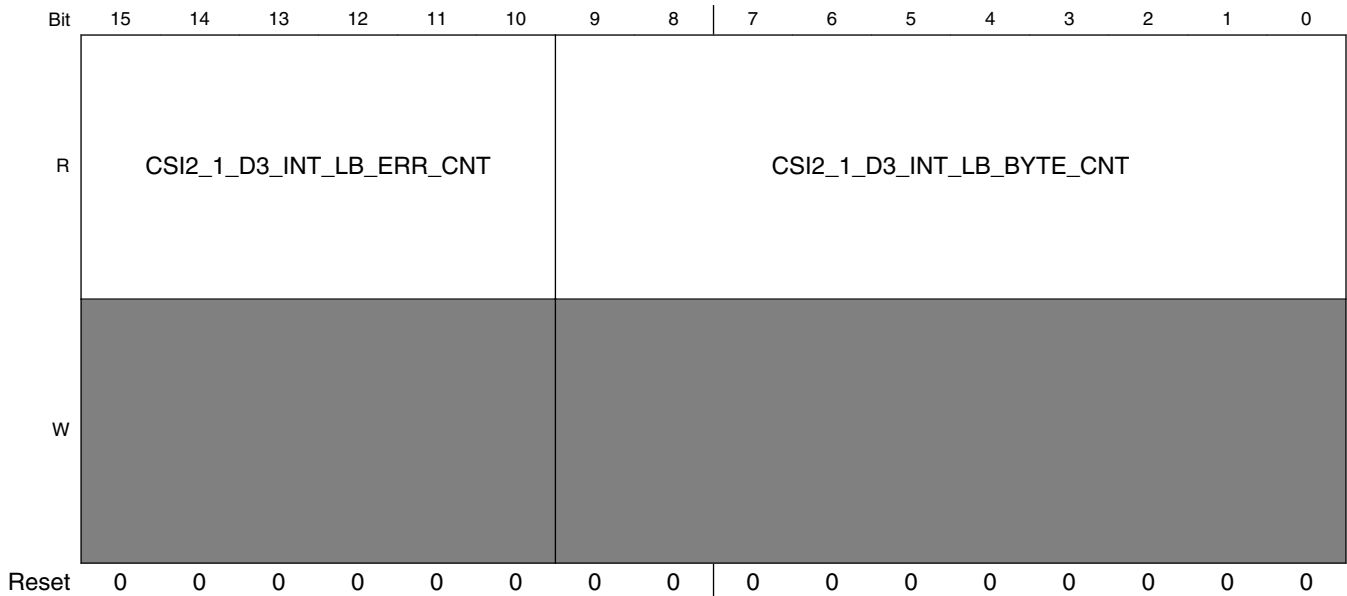
8.2.4.41 GPR40 General Purpose Register (IOMUXC_GPR_GPR40)

GPR Register

Address: 3034_0000h base + A0h offset = 3034_00A0h



IOMUX Controller (IOMUXC)



IOMUXC_GPR_GPR40 field descriptions

Field	Description
31–21 -	This field is reserved. Reserved
20 CSI2_1_D3_LB_ACTIVE	for DSI test status
19–10 CSI2_1_D3_INT_LB_ERR_CNT	for DSI test status
CSI2_1_D3_INT_LB_BYTE_CNT	for DSI test status

8.2.4.42 GPR41 General Purpose Register (IOMUXC_GPR_GPR41)

GPR Register

Address: 3034_0000h base + A4h offset = 3034_00A4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		CSI2_2_RX_ENABLE	CSI2_2_VID_INTFC_ENB	CSI2_2_PD_RX	CSI2_2_HSEL	CSI2_2_AUTO_PD_EN	CSI2_2_CONT_CLK_MODE	CSI2_2_S_PRG_RXHS_SETTLE						CSI2_2_RX_RCAL	
W																
Reset	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_GPR_GPR41 field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13 CSI2_2_RX_ENABLE	MIPI CSI2_2 Controller Receive Enable, active high. When deasserted, the RX controller will pause data reception at the next packet boundary. Since the CSI-2 protocol does not allow the Receiver to pause data, when rx_enable is deasserted the RX Controller still receives data from the RX DPHY but it does not forward the receive packets to the user interface but instead discards the received data. When rx_enable is asserted, the RX controller will wait for the start of the next packet before allowing data to be sent out over the user interface.
12 CSI2_2_VID_INTFC_ENB	MIPI CSI2_2 controller video interface enable. 0 Disable 1 Enable
11 CSI2_2_PD_RX	MIPI CSI2_2 D-PHY Power Down input. When high, all blocks are powered down.
10 CSI2_2_HSEL	MIPI CSI2_2 D-PHY High Speed Select 0 1.0 Gbps operation 1 1.2 Gbps operation

Table continues on the next page...

IOMUXC_GPR_GPR41 field descriptions (continued)

Field	Description
9 CSI2_2_AUTO_ PD_EN	MIPI CSI2_2 D-PHY Power down inactive lanes reported by CFG_NUM_LANES input bus. 0 Inactive lanes are powered up and driving LP11. 1 Inactive lanes are powered down.
8 CSI2_2_CONT_ CLK_MODE	MIPI CSI2_2 D-PHY slave clock lane feature enable to maintain HS reception state during continuous clock mode operation. 0 Feature disable 1 Feature enable
7-2 CSI2_2_S_PRG_ RXHS_SETTLE	MIPI CSI2_2 D-PHY program T_HS_SETTLE bits. HS-RX waits for Time-out T_HS_SETTLE in order to neglect transition effects.
CSI2_2_RX_ RCAL	MIPI CSI2_2 D-PHY On-chip termination control bits for manual calibration of HS-RX. 00 25% higher than mid-range. Highest impedance setting. 01 15% higher than mid-range. 10 Mid-range impedance setting. 11 25% lower than mid-range. Lowest impedance setting.

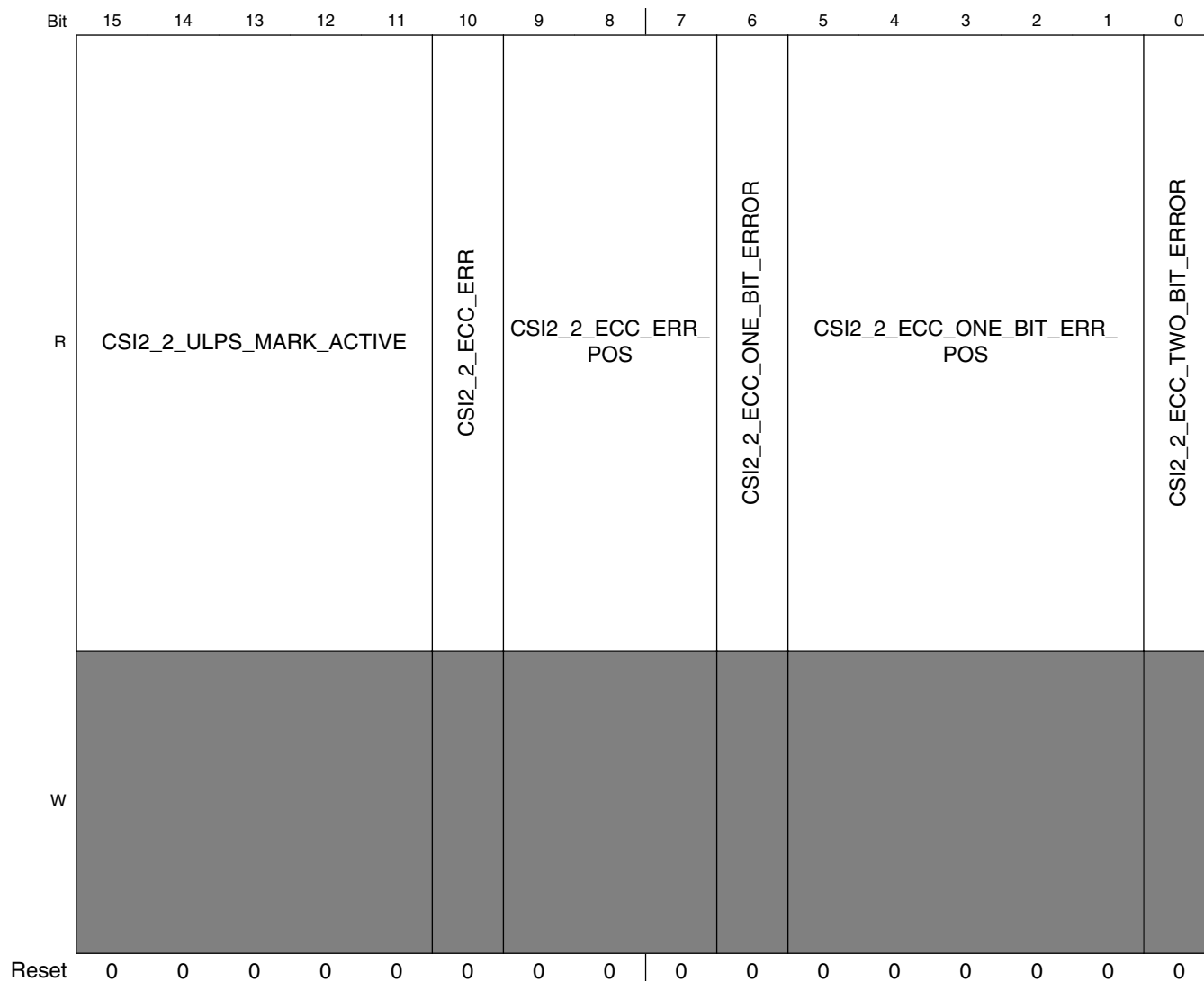
8.2.4.43 GPR42 General Purpose Register (IOMUXC_GPR_GPR42)

GPR Register

Address: 3034_0000h base + A8h offset = 3034_00A8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved							CSI2_2_ERR_SEND_LEVEL	CSI2_2_ERR_FIFO_WR_OVFL	CSI2_2_ULPS_ACTIVE				CSI2_2_CRC_ERR		CSI2_2_RX_DPHY_RDY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUX Controller (IOMUXC)



IOMUXC_GPR_GPR42 field descriptions

Field	Description
31–25 -	This field is reserved. Reserved
24 CSI2_2_ERR_SEND_LEVEL	For MIPI CSI2_2 Controller, this flag indicates the video interface pixel fifo did not accumulate enough sample to trigger its send level setting before the end of an image data packet was detected. This flag would generally mean that the <code>cfg_vid_p_fifo_send_level</code> is set to high.
23 CSI2_2_ERR_FIFO_WR_OVFL	For MIPI CSI2_2 Controller, this flag indicates the video interface pixel fifo overflowed on a pixel write. This means the output video port is not removing pixels quickly enough to keep up with the input data rate. This flag would generally mean the <code>clk_vid</code> rate is too low.
22–18 CSI2_2_ULPS_ACTIVE	MIPI CSI2_2 Controller Receive Ultra Low Power State active. Bits assert high when corresponding clock or data lane is in ULPS mode. Bit [0] – Clock Lane Bit [1] – Data Lane 0 Bit [2] – Data Lane 1

Table continues on the next page...

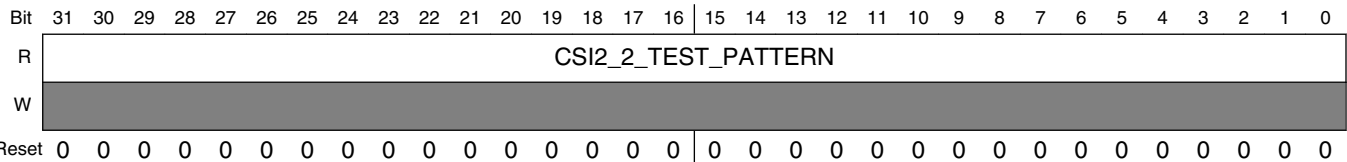
IOMUXC_GPR_GPR42 field descriptions (continued)

Field	Description
	Bit [3] – Data Lane 2 Bit [4] – Data Lane 3
17 CSI2_2_CRC_ERR	MIPI CSI2_2 Controller CRC status. Asserts high when the CRC calculated on the received data does not match the CRC the transmitter sent at the end of the packet. Valid when the eop_out output asserts.
16 CSI2_2_RX_DPHY_RDY	MIPI CSI2_2 D-PHY ready status 0 Not ready 1 Ready
15–11 CSI2_2_ULPS_MARK_ACTIVE	MIPI CSI2_2 Controller Receive Ultra Low Power State Mark status. Bits assert high when corresponding clock or data lane has exited ULPS and entered Mark-1 state. Lanes will remain in Mark-1 state for a minimum of 1ms as per the MIPI DPHY specification. While in Mark-1 state, no other active, like High Speed data transmission, is allowed. Bit [0] – Clock Lane Bit [1] – Data Lane 0 Bit [2] – Data Lane 1 Bit [3] – Data Lane 2 Bit [4] – Data Lane 3
10 CSI2_2_ECC_ERR	MIPI CSI2_2: Error detected in the ECC bits. This signal is no longer supported.
9–7 CSI2_2_ECC_ERR_POS	MIPI CSI2_2: Position of the corrected single bit error in the packet header. Valid when pkt_hdr_valid is asserted high.
6 CSI2_2_ECC_ONE_BIT_ERROR	MIPI CSI2_2: Two packet header bit errors were detected and not corrected, active high and is valid when pkt_hdr_is high.
5–1 CSI2_2_ECC_ONE_BIT_ERR_POS	MIPI CSI2_2: Position of the corrected single bit error in the packet header. Valid when pkt_hdr_valid is asserted high.
0 CSI2_2_ECC_TWO_BIT_ERROR	MIPI CSI2_2: Two packet header bit errors were detected and not corrected, active high and is valid when pkt_hdr_is high.

8.2.4.44 GPR43 General Purpose Register (IOMUXC_GPR_GPR43)

GPR Register

Address: 3034_0000h base + ACh offset = 3034_00ACh



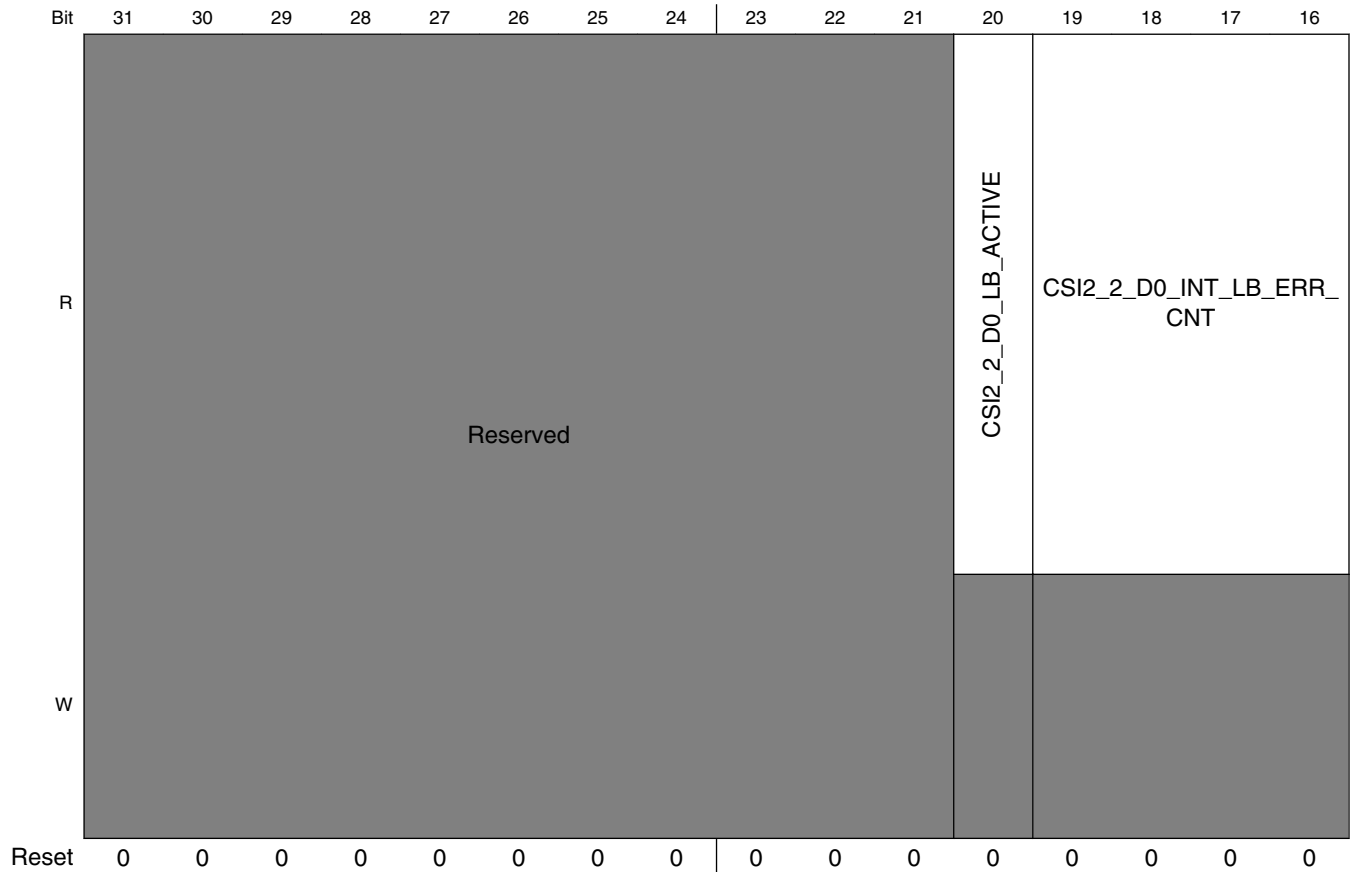
IOMUXC_GPR_GPR43 field descriptions

Field	Description
CSI2_2_TEST_PATTERN	for CSI2_2 test control

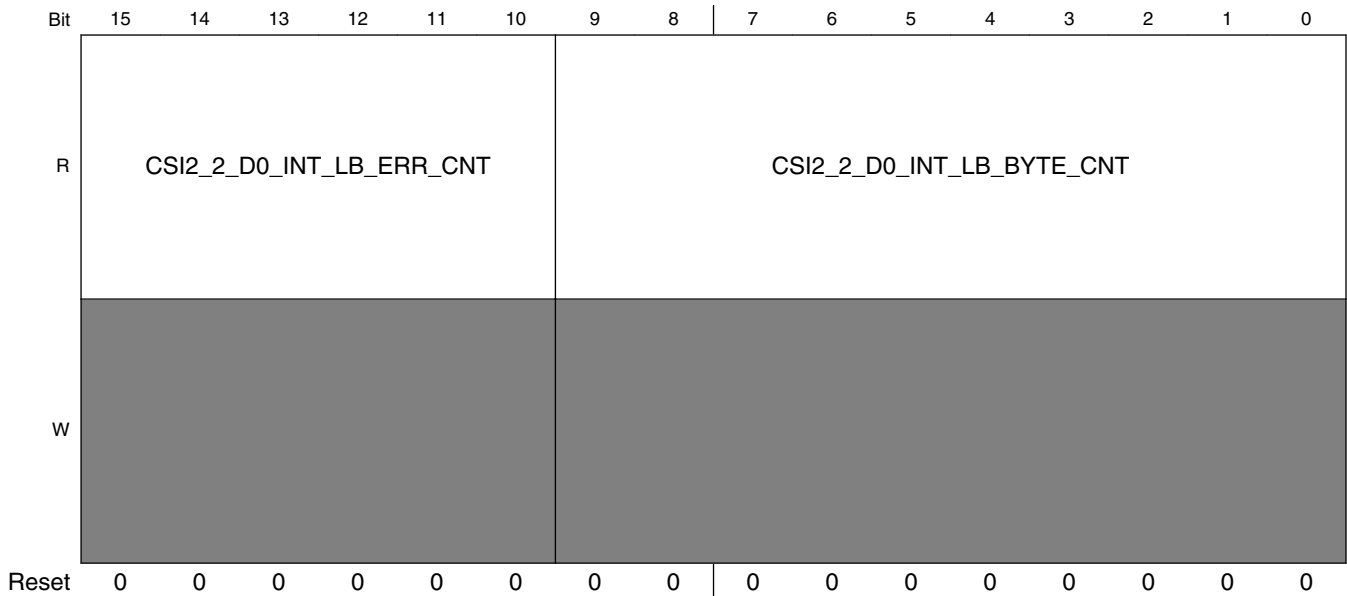
8.2.4.45 GPR44 General Purpose Register (IOMUXC_GPR_GPR44)

GPR Register

Address: 3034_0000h base + B0h offset = 3034_00B0h



IOMUX Controller (IOMUXC)



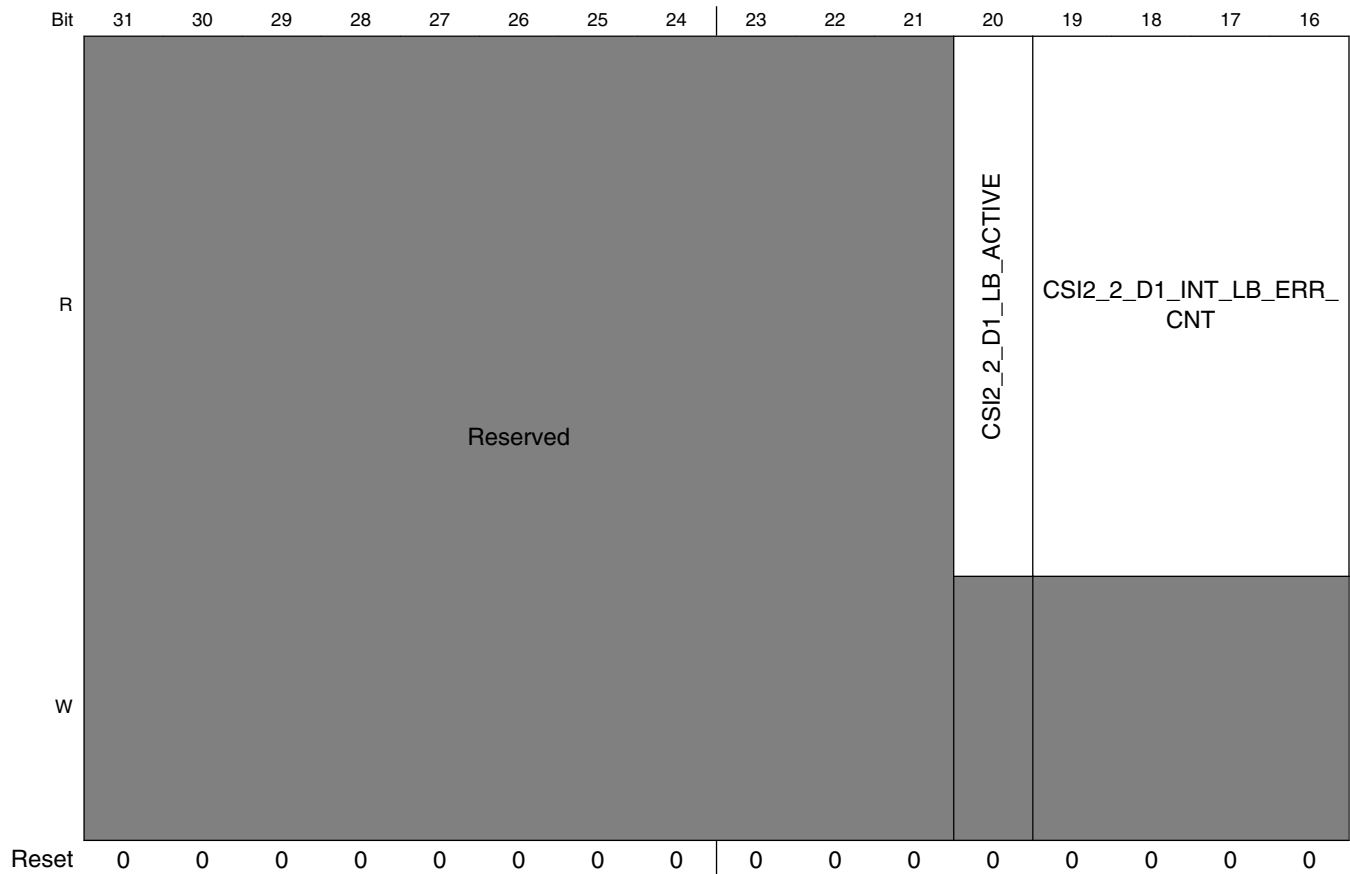
IOMUXC_GPR_GPR44 field descriptions

Field	Description
31–21 -	This field is reserved. Reserved
20 CSI2_2_D0_LB_ACTIVE	for DSI test status
19–10 CSI2_2_D0_INT_LB_ERR_CNT	for DSI test status
CSI2_2_D0_INT_LB_BYTE_CNT	for DSI test status

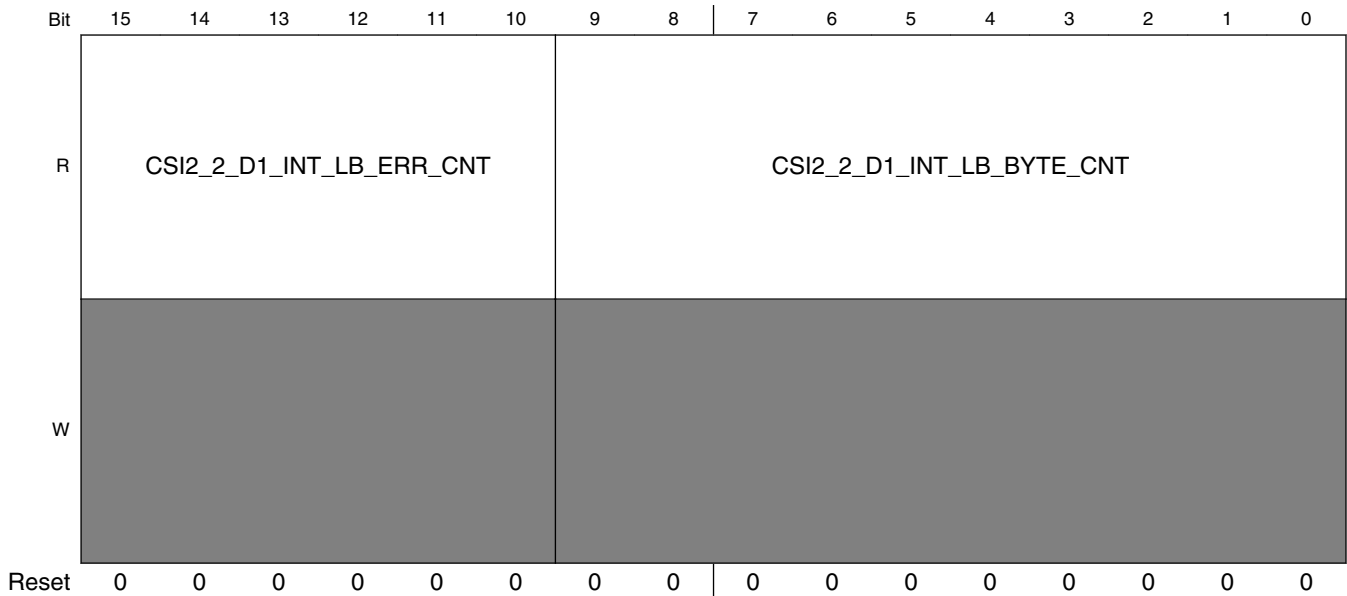
8.2.4.46 GPR45 General Purpose Register (IOMUXC_GPR_GPR45)

GPR Register

Address: 3034_0000h base + B4h offset = 3034_00B4h



IOMUX Controller (IOMUXC)



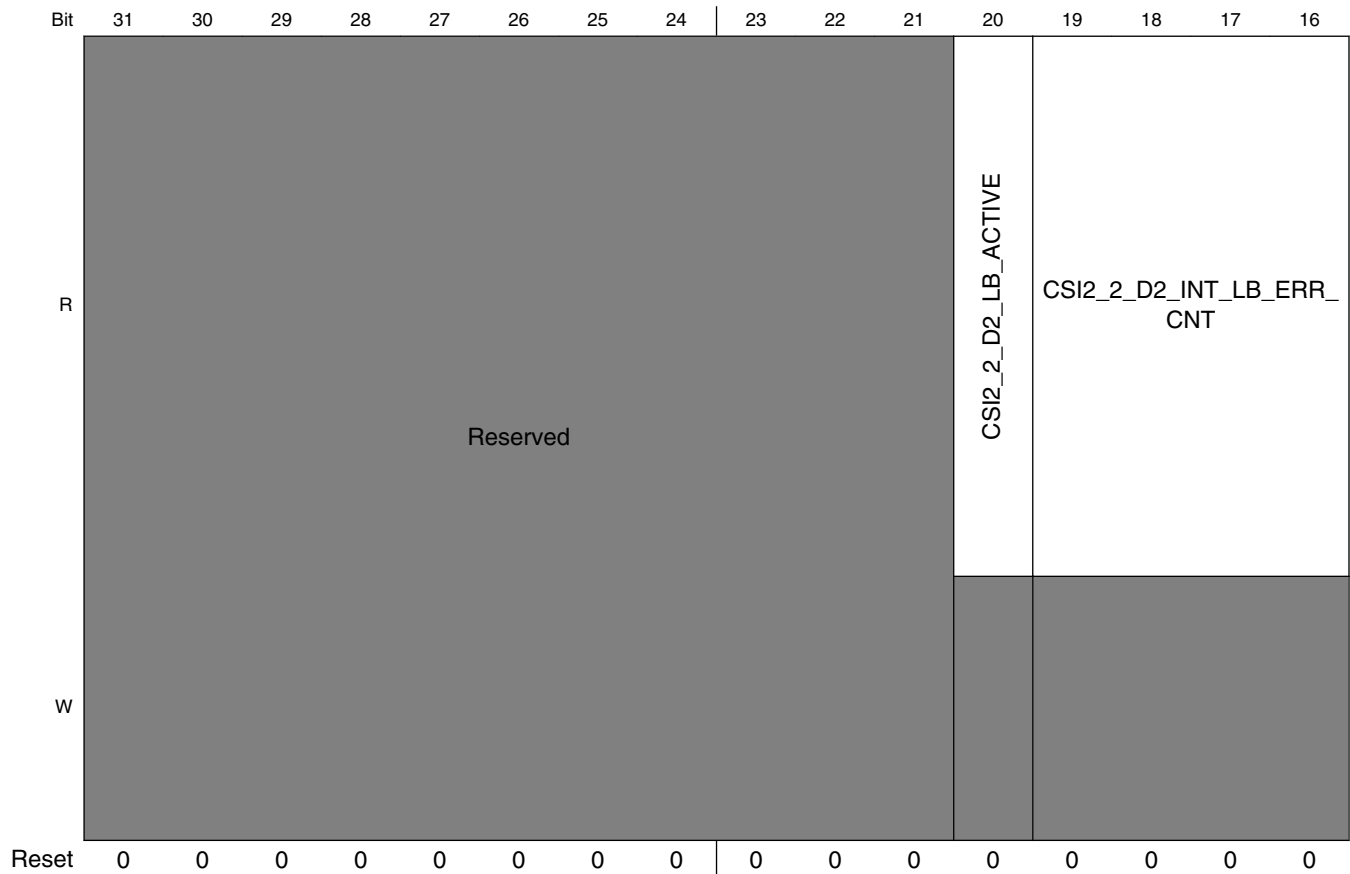
IOMUXC_GPR_GPR45 field descriptions

Field	Description
31–21 -	This field is reserved. Reserved
20 CSI2_2_D1_LB_ACTIVE	for DSI test status
19–10 CSI2_2_D1_INT_LB_ERR_CNT	for DSI test status
CSI2_2_D1_INT_LB_BYTE_CNT	for DSI test status

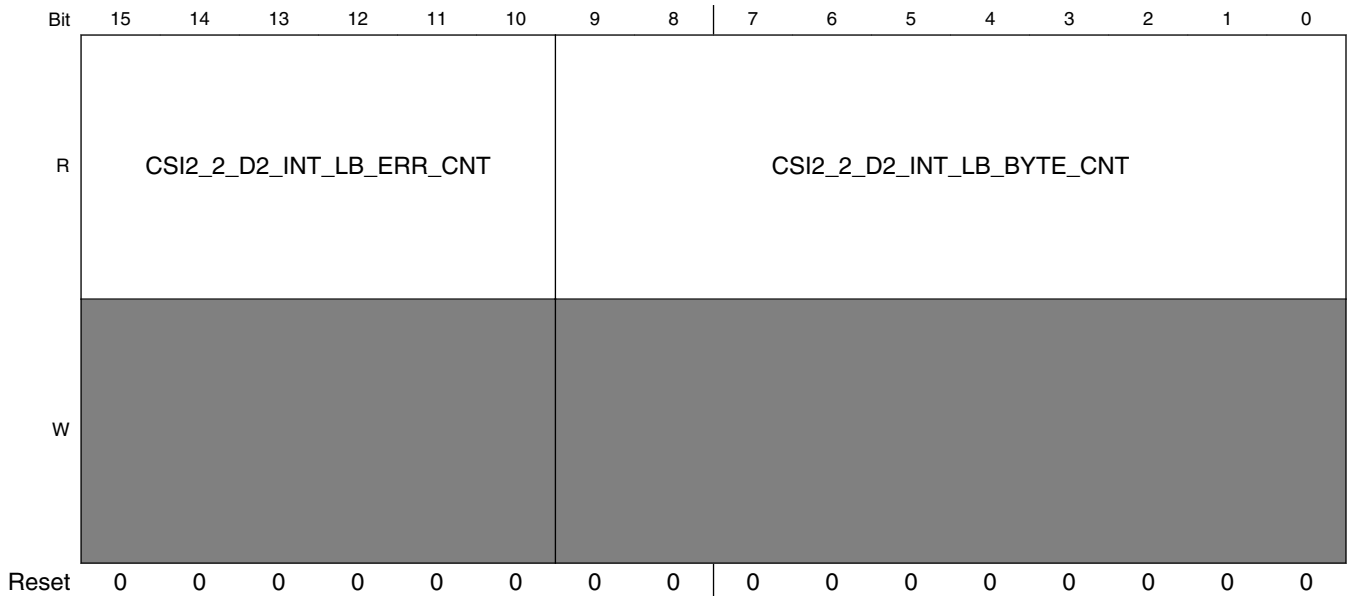
8.2.4.47 GPR46 General Purpose Register (IOMUXC_GPR_GPR46)

GPR Register

Address: 3034_0000h base + B8h offset = 3034_00B8h



IOMUX Controller (IOMUXC)



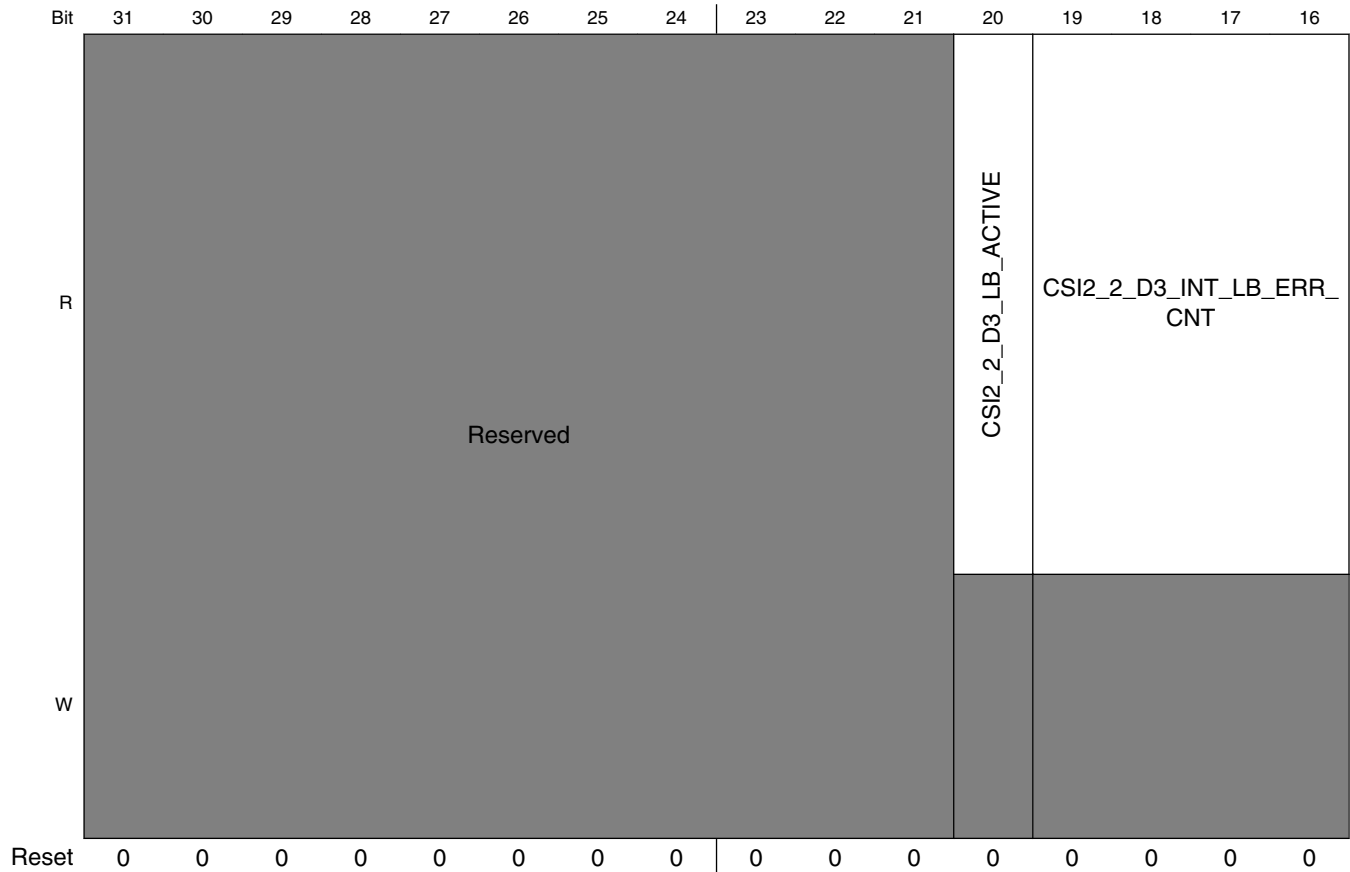
IOMUXC_GPR_GPR46 field descriptions

Field	Description
31–21 -	This field is reserved. Reserved
20 CSI2_2_D2_LB_ACTIVE	for DSI test status
19–10 CSI2_2_D2_INT_LB_ERR_CNT	for DSI test status
CSI2_2_D2_INT_LB_BYTE_CNT	for DSI test status

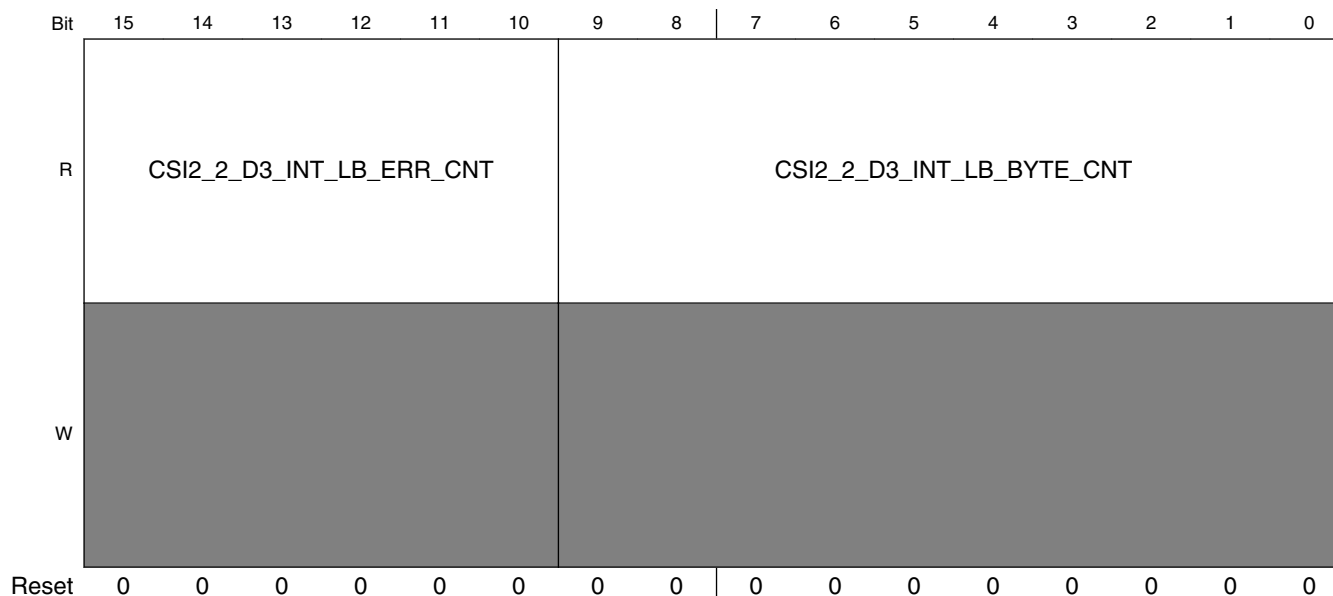
8.2.4.48 GPR47 General Purpose Register (IOMUXC_GPR_GPR47)

GPR Register

Address: 3034_0000h base + BCh offset = 3034_00BCh



IOMUX Controller (IOMUXC)



IOMUXC_GPR_GPR47 field descriptions

Field	Description
31–21 -	This field is reserved. Reserved
20 CSI2_2_D3_LB_ACTIVE	for DSI test status
19–10 CSI2_2_D3_INT_LB_ERR_CNT	for DSI test status
CSI2_2_D3_INT_LB_BYTE_CNT	for DSI test status

8.2.5 IOMUXC Memory Map/Register Definition

IOMUXC memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3033_0014	SW_MUX_CTL_PAD_PMIC_STBY_REQ SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_PMIC_STBY_REQ)	32	R/W	0000_0000h	8.2.5.1/1436
3033_0018	SW_MUX_CTL_PAD_PMIC_ON_REQ SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_PMIC_ON_REQ)	32	R/W	0000_0000h	8.2.5.2/1436
3033_001C	SW_MUX_CTL_PAD_ONOFF SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ONOFF)	32	R/W	0000_0000h	8.2.5.3/1437

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3033_0020	SW_MUX_CTL_PAD_POR_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_POR_B)	32	R/W	0000_0000h	8.2.5.4/1438
3033_0024	SW_MUX_CTL_PAD_RTC_RESET_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_RTC_RESET_B)	32	R/W	0000_0000h	8.2.5.5/1438
3033_0028	SW_MUX_CTL_PAD_GPIO1_IO00 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO00)	32	R/W	0000_0000h	8.2.5.6/1439
3033_002C	SW_MUX_CTL_PAD_GPIO1_IO01 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO01)	32	R/W	0000_0000h	8.2.5.7/1440
3033_0030	SW_MUX_CTL_PAD_GPIO1_IO02 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO02)	32	R/W	0000_0000h	8.2.5.8/1442
3033_0034	SW_MUX_CTL_PAD_GPIO1_IO03 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO03)	32	R/W	0000_0000h	8.2.5.9/1443
3033_0038	SW_MUX_CTL_PAD_GPIO1_IO04 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO04)	32	R/W	0000_0000h	8.2.5.10/1444
3033_003C	SW_MUX_CTL_PAD_GPIO1_IO05 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO05)	32	R/W	0000_0000h	8.2.5.11/1445
3033_0040	SW_MUX_CTL_PAD_GPIO1_IO06 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO06)	32	R/W	0000_0000h	8.2.5.12/1446
3033_0044	SW_MUX_CTL_PAD_GPIO1_IO07 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO07)	32	R/W	0000_0000h	8.2.5.13/1447
3033_0048	SW_MUX_CTL_PAD_GPIO1_IO08 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO08)	32	R/W	0000_0000h	8.2.5.14/1448
3033_004C	SW_MUX_CTL_PAD_GPIO1_IO09 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO09)	32	R/W	0000_0000h	8.2.5.15/1449
3033_0050	SW_MUX_CTL_PAD_GPIO1_IO10 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO10)	32	R/W	0000_0000h	8.2.5.16/1451
3033_0054	SW_MUX_CTL_PAD_GPIO1_IO11 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO11)	32	R/W	0000_0000h	8.2.5.17/1452
3033_0058	SW_MUX_CTL_PAD_GPIO1_IO12 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO12)	32	R/W	0000_0000h	8.2.5.18/1453
3033_005C	SW_MUX_CTL_PAD_GPIO1_IO13 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO13)	32	R/W	0000_0000h	8.2.5.19/1454
3033_0060	SW_MUX_CTL_PAD_GPIO1_IO14 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO14)	32	R/W	0000_0000h	8.2.5.20/1455
3033_0064	SW_MUX_CTL_PAD_GPIO1_IO15 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO15)	32	R/W	0000_0000h	8.2.5.21/1456
3033_0068	SW_MUX_CTL_PAD_ENET_MDC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_MDC)	32	R/W	0000_0005h	8.2.5.22/1457
3033_006C	SW_MUX_CTL_PAD_ENET_MDIO SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_MDIO)	32	R/W	0000_0005h	8.2.5.23/1458
3033_0070	SW_MUX_CTL_PAD_ENET_TD3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_TD3)	32	R/W	0000_0005h	8.2.5.24/1459
3033_0074	SW_MUX_CTL_PAD_ENET_TD2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_TD2)	32	R/W	0000_0005h	8.2.5.25/1460

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3033_0078	SW_MUX_CTL_PAD_ENET_TD1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_TD1)	32	R/W	0000_0005h	8.2.5.26/1461
3033_007C	SW_MUX_CTL_PAD_ENET_TD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_TD0)	32	R/W	0000_0005h	8.2.5.27/1462
3033_0080	SW_MUX_CTL_PAD_ENET_TX_CTL SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_TX_CTL)	32	R/W	0000_0005h	8.2.5.28/1463
3033_0084	SW_MUX_CTL_PAD_ENET_TXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_TXC)	32	R/W	0000_0005h	8.2.5.29/1464
3033_0088	SW_MUX_CTL_PAD_ENET_RX_CTL SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_RX_CTL)	32	R/W	0000_0005h	8.2.5.30/1465
3033_008C	SW_MUX_CTL_PAD_ENET_RXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_RXC)	32	R/W	0000_0005h	8.2.5.31/1466
3033_0090	SW_MUX_CTL_PAD_ENET_RD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_RD0)	32	R/W	0000_0005h	8.2.5.32/1467
3033_0094	SW_MUX_CTL_PAD_ENET_RD1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_RD1)	32	R/W	0000_0005h	8.2.5.33/1468
3033_0098	SW_MUX_CTL_PAD_ENET_RD2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_RD2)	32	R/W	0000_0005h	8.2.5.34/1469
3033_009C	SW_MUX_CTL_PAD_ENET_RD3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_RD3)	32	R/W	0000_0005h	8.2.5.35/1470
3033_00A0	SW_MUX_CTL_PAD_SD1_CLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_CLK)	32	R/W	0000_0005h	8.2.5.36/1471
3033_00A4	SW_MUX_CTL_PAD_SD1_CMD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_CMD)	32	R/W	0000_0005h	8.2.5.37/1472
3033_00A8	SW_MUX_CTL_PAD_SD1_DATA0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA0)	32	R/W	0000_0005h	8.2.5.38/1473
3033_00AC	SW_MUX_CTL_PAD_SD1_DATA1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA1)	32	R/W	0000_0005h	8.2.5.39/1474
3033_00B0	SW_MUX_CTL_PAD_SD1_DATA2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA2)	32	R/W	0000_0005h	8.2.5.40/1475
3033_00B4	SW_MUX_CTL_PAD_SD1_DATA3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA3)	32	R/W	0000_0005h	8.2.5.41/1476
3033_00B8	SW_MUX_CTL_PAD_SD1_DATA4 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA4)	32	R/W	0000_0005h	8.2.5.42/1477
3033_00BC	SW_MUX_CTL_PAD_SD1_DATA5 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA5)	32	R/W	0000_0005h	8.2.5.43/1478
3033_00C0	SW_MUX_CTL_PAD_SD1_DATA6 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA6)	32	R/W	0000_0005h	8.2.5.44/1479
3033_00C4	SW_MUX_CTL_PAD_SD1_DATA7 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA7)	32	R/W	0000_0005h	8.2.5.45/1480
3033_00C8	SW_MUX_CTL_PAD_SD1_RESET_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_RESET_B)	32	R/W	0000_0005h	8.2.5.46/1481
3033_00CC	SW_MUX_CTL_PAD_SD1_STROBE SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_STROBE)	32	R/W	0000_0005h	8.2.5.47/1482

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3033_00D0	SW_MUX_CTL_PAD_SD2_CD_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_CD_B)	32	R/W	0000_0005h	8.2.5.48/1483
3033_00D4	SW_MUX_CTL_PAD_SD2_CLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_CLK)	32	R/W	0000_0005h	8.2.5.49/1484
3033_00D8	SW_MUX_CTL_PAD_SD2_CMD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_CMD)	32	R/W	0000_0005h	8.2.5.50/1485
3033_00DC	SW_MUX_CTL_PAD_SD2_DATA0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_DATA0)	32	R/W	0000_0005h	8.2.5.51/1486
3033_00E0	SW_MUX_CTL_PAD_SD2_DATA1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_DATA1)	32	R/W	0000_0005h	8.2.5.52/1487
3033_00E4	SW_MUX_CTL_PAD_SD2_DATA2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_DATA2)	32	R/W	0000_0005h	8.2.5.53/1488
3033_00E8	SW_MUX_CTL_PAD_SD2_DATA3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_DATA3)	32	R/W	0000_0005h	8.2.5.54/1489
3033_00EC	SW_MUX_CTL_PAD_SD2_RESET_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_RESET_B)	32	R/W	0000_0005h	8.2.5.55/1490
3033_00F0	SW_MUX_CTL_PAD_SD2_WP SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_WP)	32	R/W	0000_0005h	8.2.5.56/1491
3033_00F4	SW_MUX_CTL_PAD_NAND_ALE SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_ALE)	32	R/W	0000_0005h	8.2.5.57/1492
3033_00F8	SW_MUX_CTL_PAD_NAND_CE0_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_CE0_B)	32	R/W	0000_0005h	8.2.5.58/1493
3033_00FC	SW_MUX_CTL_PAD_NAND_CE1_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_CE1_B)	32	R/W	0000_0005h	8.2.5.59/1494
3033_0100	SW_MUX_CTL_PAD_NAND_CE2_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_CE2_B)	32	R/W	0000_0005h	8.2.5.60/1495
3033_0104	SW_MUX_CTL_PAD_NAND_CE3_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_CE3_B)	32	R/W	0000_0005h	8.2.5.61/1496
3033_0108	SW_MUX_CTL_PAD_NAND_CLE SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_CLE)	32	R/W	0000_0005h	8.2.5.62/1497
3033_010C	SW_MUX_CTL_PAD_NAND_DATA00 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA00)	32	R/W	0000_0005h	8.2.5.63/1498
3033_0110	SW_MUX_CTL_PAD_NAND_DATA01 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA01)	32	R/W	0000_0005h	8.2.5.64/1499
3033_0114	SW_MUX_CTL_PAD_NAND_DATA02 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA02)	32	R/W	0000_0005h	8.2.5.65/1500
3033_0118	SW_MUX_CTL_PAD_NAND_DATA03 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA03)	32	R/W	0000_0005h	8.2.5.66/1501
3033_011C	SW_MUX_CTL_PAD_NAND_DATA04 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA04)	32	R/W	0000_0005h	8.2.5.67/1502
3033_0120	SW_MUX_CTL_PAD_NAND_DATA05 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA05)	32	R/W	0000_0005h	8.2.5.68/1503
3033_0124	SW_MUX_CTL_PAD_NAND_DATA06 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA06)	32	R/W	0000_0005h	8.2.5.69/1504

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3033_0128	SW_MUX_CTL_PAD_NAND_DATA07 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA07)	32	R/W	0000_0005h	8.2.5.70/1505
3033_012C	SW_MUX_CTL_PAD_NAND_DQS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DQS)	32	R/W	0000_0005h	8.2.5.71/1506
3033_0130	SW_MUX_CTL_PAD_NAND_RE_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_RE_B)	32	R/W	0000_0005h	8.2.5.72/1507
3033_0134	SW_MUX_CTL_PAD_NAND_READY_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_READY_B)	32	R/W	0000_0005h	8.2.5.73/1508
3033_0138	SW_MUX_CTL_PAD_NAND_WE_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_WE_B)	32	R/W	0000_0005h	8.2.5.74/1509
3033_013C	SW_MUX_CTL_PAD_NAND_WP_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_WP_B)	32	R/W	0000_0005h	8.2.5.75/1510
3033_0140	SW_MUX_CTL_PAD_SAI5_RXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_RXFS)	32	R/W	0000_0005h	8.2.5.76/1511
3033_0144	SW_MUX_CTL_PAD_SAI5_RXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_RXC)	32	R/W	0000_0005h	8.2.5.77/1512
3033_0148	SW_MUX_CTL_PAD_SAI5_RXD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD0)	32	R/W	0000_0005h	8.2.5.78/1513
3033_014C	SW_MUX_CTL_PAD_SAI5_RXD1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD1)	32	R/W	0000_0005h	8.2.5.79/1514
3033_0150	SW_MUX_CTL_PAD_SAI5_RXD2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD2)	32	R/W	0000_0005h	8.2.5.80/1515
3033_0154	SW_MUX_CTL_PAD_SAI5_RXD3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD3)	32	R/W	0000_0005h	8.2.5.81/1516
3033_0158	SW_MUX_CTL_PAD_SAI5_MCLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_MCLK)	32	R/W	0000_0005h	8.2.5.82/1517
3033_015C	SW_MUX_CTL_PAD_SAI1_RXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXFS)	32	R/W	0000_0005h	8.2.5.83/1518
3033_0160	SW_MUX_CTL_PAD_SAI1_RXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXC)	32	R/W	0000_0005h	8.2.5.84/1520
3033_0164	SW_MUX_CTL_PAD_SAI1_RXD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD0)	32	R/W	0000_0005h	8.2.5.85/1521
3033_0168	SW_MUX_CTL_PAD_SAI1_RXD1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD1)	32	R/W	0000_0005h	8.2.5.86/1522
3033_016C	SW_MUX_CTL_PAD_SAI1_RXD2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD2)	32	R/W	0000_0005h	8.2.5.87/1524
3033_0170	SW_MUX_CTL_PAD_SAI1_RXD3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD3)	32	R/W	0000_0005h	8.2.5.88/1525
3033_0174	SW_MUX_CTL_PAD_SAI1_RXD4 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD4)	32	R/W	0000_0005h	8.2.5.89/1526
3033_0178	SW_MUX_CTL_PAD_SAI1_RXD5 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD5)	32	R/W	0000_0005h	8.2.5.90/1528
3033_017C	SW_MUX_CTL_PAD_SAI1_RXD6 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD6)	32	R/W	0000_0005h	8.2.5.91/1529

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3033_0180	SW_MUX_CTL_PAD_SAI1_RXD7 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD7)	32	R/W	0000_0005h	8.2.5.92/1530
3033_0184	SW_MUX_CTL_PAD_SAI1_TXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXFS)	32	R/W	0000_0005h	8.2.5.93/1532
3033_0188	SW_MUX_CTL_PAD_SAI1_TXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXC)	32	R/W	0000_0005h	8.2.5.94/1533
3033_018C	SW_MUX_CTL_PAD_SAI1_TXD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD0)	32	R/W	0000_0005h	8.2.5.95/1534
3033_0190	SW_MUX_CTL_PAD_SAI1_TXD1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD1)	32	R/W	0000_0005h	8.2.5.96/1535
3033_0194	SW_MUX_CTL_PAD_SAI1_TXD2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD2)	32	R/W	0000_0005h	8.2.5.97/1537
3033_0198	SW_MUX_CTL_PAD_SAI1_TXD3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD3)	32	R/W	0000_0005h	8.2.5.98/1538
3033_019C	SW_MUX_CTL_PAD_SAI1_TXD4 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD4)	32	R/W	0000_0005h	8.2.5.99/1539
3033_01A0	SW_MUX_CTL_PAD_SAI1_TXD5 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD5)	32	R/W	0000_0005h	8.2.5.100/1541
3033_01A4	SW_MUX_CTL_PAD_SAI1_TXD6 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD6)	32	R/W	0000_0005h	8.2.5.101/1542
3033_01A8	SW_MUX_CTL_PAD_SAI1_TXD7 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD7)	32	R/W	0000_0005h	8.2.5.102/1543
3033_01AC	SW_MUX_CTL_PAD_SAI1_MCLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_MCLK)	32	R/W	0000_0005h	8.2.5.103/1545
3033_01B0	SW_MUX_CTL_PAD_SAI2_RXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_RXFS)	32	R/W	0000_0005h	8.2.5.104/1546
3033_01B4	SW_MUX_CTL_PAD_SAI2_RXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_RXC)	32	R/W	0000_0005h	8.2.5.105/1547
3033_01B8	SW_MUX_CTL_PAD_SAI2_RXD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_RXD0)	32	R/W	0000_0005h	8.2.5.106/1548
3033_01BC	SW_MUX_CTL_PAD_SAI2_TXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_TXFS)	32	R/W	0000_0005h	8.2.5.107/1549
3033_01C0	SW_MUX_CTL_PAD_SAI2_TXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_TXC)	32	R/W	0000_0005h	8.2.5.108/1550
3033_01C4	SW_MUX_CTL_PAD_SAI2_TXD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_TXD0)	32	R/W	0000_0005h	8.2.5.109/1551
3033_01C8	SW_MUX_CTL_PAD_SAI2_MCLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_MCLK)	32	R/W	0000_0005h	8.2.5.110/1552
3033_01CC	SW_MUX_CTL_PAD_SAI3_RXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_RXFS)	32	R/W	0000_0005h	8.2.5.111/1553
3033_01D0	SW_MUX_CTL_PAD_SAI3_RXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_RXC)	32	R/W	0000_0005h	8.2.5.112/1554
3033_01D4	SW_MUX_CTL_PAD_SAI3_RXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_RXD)	32	R/W	0000_0005h	8.2.5.113/1555

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3033_01D8	SW_MUX_CTL_PAD_SAI3_TXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_TXFS)	32	R/W	0000_0005h	8.2.5.114/1556
3033_01DC	SW_MUX_CTL_PAD_SAI3_TXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_TXC)	32	R/W	0000_0005h	8.2.5.115/1557
3033_01E0	SW_MUX_CTL_PAD_SAI3_TXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_TXD)	32	R/W	0000_0005h	8.2.5.116/1558
3033_01E4	SW_MUX_CTL_PAD_SAI3_MCLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_MCLK)	32	R/W	0000_0005h	8.2.5.117/1559
3033_01E8	SW_MUX_CTL_PAD_SPDIF_TX SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SPDIF_TX)	32	R/W	0000_0005h	8.2.5.118/1560
3033_01EC	SW_MUX_CTL_PAD_SPDIF_RX SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SPDIF_RX)	32	R/W	0000_0005h	8.2.5.119/1561
3033_01F0	SW_MUX_CTL_PAD_SPDIF_EXT_CLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SPDIF_EXT_CLK)	32	R/W	0000_0005h	8.2.5.120/1562
3033_01F4	SW_MUX_CTL_PAD_ECSP11_SCLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSP11_SCLK)	32	R/W	0000_0005h	8.2.5.121/1563
3033_01F8	SW_MUX_CTL_PAD_ECSP11_MOSI SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSP11_MOSI)	32	R/W	0000_0005h	8.2.5.122/1564
3033_01FC	SW_MUX_CTL_PAD_ECSP11_MISO SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSP11_MISO)	32	R/W	0000_0005h	8.2.5.123/1565
3033_0200	SW_MUX_CTL_PAD_ECSP11_SS0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSP11_SS0)	32	R/W	0000_0005h	8.2.5.124/1566
3033_0204	SW_MUX_CTL_PAD_ECSP12_SCLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSP12_SCLK)	32	R/W	0000_0005h	8.2.5.125/1567
3033_0208	SW_MUX_CTL_PAD_ECSP12_MOSI SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSP12_MOSI)	32	R/W	0000_0005h	8.2.5.126/1568
3033_020C	SW_MUX_CTL_PAD_ECSP12_MISO SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSP12_MISO)	32	R/W	0000_0005h	8.2.5.127/1569
3033_0210	SW_MUX_CTL_PAD_ECSP12_SS0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSP12_SS0)	32	R/W	0000_0005h	8.2.5.128/1570
3033_0214	SW_MUX_CTL_PAD_I2C1_SCL SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C1_SCL)	32	R/W	0000_0005h	8.2.5.129/1571
3033_0218	SW_MUX_CTL_PAD_I2C1_SDA SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C1_SDA)	32	R/W	0000_0005h	8.2.5.130/1572
3033_021C	SW_MUX_CTL_PAD_I2C2_SCL SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C2_SCL)	32	R/W	0000_0005h	8.2.5.131/1573
3033_0220	SW_MUX_CTL_PAD_I2C2_SDA SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C2_SDA)	32	R/W	0000_0005h	8.2.5.132/1574
3033_0224	SW_MUX_CTL_PAD_I2C3_SCL SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C3_SCL)	32	R/W	0000_0005h	8.2.5.133/1575
3033_0228	SW_MUX_CTL_PAD_I2C3_SDA SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C3_SDA)	32	R/W	0000_0005h	8.2.5.134/1576
3033_022C	SW_MUX_CTL_PAD_I2C4_SCL SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C4_SCL)	32	R/W	0000_0005h	8.2.5.135/1577

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3033_0230	SW_MUX_CTL_PAD_I2C4_SDA SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C4_SDA)	32	R/W	0000_0005h	8.2.5.136/1578
3033_0234	SW_MUX_CTL_PAD_UART1_RXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART1_RXD)	32	R/W	0000_0005h	8.2.5.137/1579
3033_0238	SW_MUX_CTL_PAD_UART1_TXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART1_TXD)	32	R/W	0000_0005h	8.2.5.138/1580
3033_023C	SW_MUX_CTL_PAD_UART2_RXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART2_RXD)	32	R/W	0000_0005h	8.2.5.139/1581
3033_0240	SW_MUX_CTL_PAD_UART2_TXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART2_TXD)	32	R/W	0000_0005h	8.2.5.140/1582
3033_0244	SW_MUX_CTL_PAD_UART3_RXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART3_RXD)	32	R/W	0000_0005h	8.2.5.141/1583
3033_0248	SW_MUX_CTL_PAD_UART3_TXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART3_TXD)	32	R/W	0000_0005h	8.2.5.142/1584
3033_024C	SW_MUX_CTL_PAD_UART4_RXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART4_RXD)	32	R/W	0000_0005h	8.2.5.143/1585
3033_0250	SW_MUX_CTL_PAD_UART4_TXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART4_TXD)	32	R/W	0000_0005h	8.2.5.144/1586
3033_0254	SW_PAD_CTL_PAD_TEST_MODE SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_TEST_MODE)	32	R/W	0000_0001h	8.2.5.145/1587
3033_0258	SW_PAD_CTL_PAD_BOOT_MODE0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_BOOT_MODE0)	32	R/W	0000_0081h	8.2.5.146/1588
3033_025C	SW_PAD_CTL_PAD_BOOT_MODE1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_BOOT_MODE1)	32	R/W	0000_0081h	8.2.5.147/1590
3033_0260	SW_PAD_CTL_PAD_JTAG_MOD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_JTAG_MOD)	32	R/W	0000_1801h	8.2.5.148/1591
3033_0264	SW_PAD_CTL_PAD_JTAG_TRST_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_JTAG_TRST_B)	32	R/W	0000_0041h	8.2.5.149/1593
3033_0268	SW_PAD_CTL_PAD_JTAG_TDI SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_JTAG_TDI)	32	R/W	0000_0041h	8.2.5.150/1594
3033_026C	SW_PAD_CTL_PAD_JTAG_TMS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_JTAG_TMS)	32	R/W	0000_0041h	8.2.5.151/1595
3033_0270	SW_PAD_CTL_PAD_JTAG_TCK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_JTAG_TCK)	32	R/W	0000_0041h	8.2.5.152/1596
3033_0274	SW_PAD_CTL_PAD_JTAG_TDO SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_JTAG_TDO)	32	R/W	0000_0041h	8.2.5.153/1597
3033_0278	SW_PAD_CTL_PAD_RTC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_RTC)	32	R/W	0000_0001h	8.2.5.154/1599
3033_027C	SW_PAD_CTL_PAD_PMIC_STBY_REQ SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_PMIC_STBY_REQ)	32	R/W	0000_000Ch	8.2.5.155/1600
3033_0280	SW_PAD_CTL_PAD_PMIC_ON_REQ SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_PMIC_ON_REQ)	32	R/W	0000_186Ch	8.2.5.156/1602
3033_0284	SW_PAD_CTL_PAD_ONOFF SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ONOFF)	32	R/W	0000_00CCh	8.2.5.157/1604

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3033_0288	SW_PAD_CTL_PAD_POR_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_POR_B)	32	R/W	0000_00CCh	8.2.5.158/1605
3033_028C	SW_PAD_CTL_PAD_RTC_RESET_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_RTC_RESET_B)	32	R/W	0000_00CCh	8.2.5.159/1607
3033_0290	SW_PAD_CTL_PAD_GPIO1_IO00 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO00)	32	R/W	0000_0014h	8.2.5.160/1608
3033_0294	SW_PAD_CTL_PAD_GPIO1_IO01 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO01)	32	R/W	0000_0016h	8.2.5.161/1610
3033_0298	SW_PAD_CTL_PAD_GPIO1_IO02 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO02)	32	R/W	0000_0056h	8.2.5.162/1611
3033_029C	SW_PAD_CTL_PAD_GPIO1_IO03 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO03)	32	R/W	0000_0016h	8.2.5.163/1613
3033_02A0	SW_PAD_CTL_PAD_GPIO1_IO04 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO04)	32	R/W	0000_0016h	8.2.5.164/1614
3033_02A4	SW_PAD_CTL_PAD_GPIO1_IO05 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO05)	32	R/W	0000_0056h	8.2.5.165/1616
3033_02A8	SW_PAD_CTL_PAD_GPIO1_IO06 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO06)	32	R/W	0000_0016h	8.2.5.166/1617
3033_02AC	SW_PAD_CTL_PAD_GPIO1_IO07 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO07)	32	R/W	0000_1816h	8.2.5.167/1619
3033_02B0	SW_PAD_CTL_PAD_GPIO1_IO08 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO08)	32	R/W	0000_0016h	8.2.5.168/1621
3033_02B4	SW_PAD_CTL_PAD_GPIO1_IO09 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO09)	32	R/W	0000_0016h	8.2.5.169/1622
3033_02B8	SW_PAD_CTL_PAD_GPIO1_IO10 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO10)	32	R/W	0000_0016h	8.2.5.170/1624
3033_02BC	SW_PAD_CTL_PAD_GPIO1_IO11 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO11)	32	R/W	0000_0016h	8.2.5.171/1625
3033_02C0	SW_PAD_CTL_PAD_GPIO1_IO12 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO12)	32	R/W	0000_0016h	8.2.5.172/1627
3033_02C4	SW_PAD_CTL_PAD_GPIO1_IO13 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO13)	32	R/W	0000_0016h	8.2.5.173/1628
3033_02C8	SW_PAD_CTL_PAD_GPIO1_IO14 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO14)	32	R/W	0000_0016h	8.2.5.174/1630
3033_02CC	SW_PAD_CTL_PAD_GPIO1_IO15 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO15)	32	R/W	0000_0016h	8.2.5.175/1631
3033_02D0	SW_PAD_CTL_PAD_ENET_MDC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_MDC)	32	R/W	0000_0016h	8.2.5.176/1633
3033_02D4	SW_PAD_CTL_PAD_ENET_MDIO SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_MDIO)	32	R/W	0000_0016h	8.2.5.177/1634
3033_02D8	SW_PAD_CTL_PAD_ENET_TD3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_TD3)	32	R/W	0000_0016h	8.2.5.178/1636
3033_02DC	SW_PAD_CTL_PAD_ENET_TD2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_TD2)	32	R/W	0000_0016h	8.2.5.179/1637

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3033_02E0	SW_PAD_CTL_PAD_ENET_TD1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_TD1)	32	R/W	0000_0016h	8.2.5.180/1639
3033_02E4	SW_PAD_CTL_PAD_ENET_TD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_TD0)	32	R/W	0000_0016h	8.2.5.181/1640
3033_02E8	SW_PAD_CTL_PAD_ENET_TX_CTL SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_TX_CTL)	32	R/W	0000_1816h	8.2.5.182/1642
3033_02EC	SW_PAD_CTL_PAD_ENET_TXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_TXC)	32	R/W	0000_0016h	8.2.5.183/1644
3033_02F0	SW_PAD_CTL_PAD_ENET_RX_CTL SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_RX_CTL)	32	R/W	0000_0016h	8.2.5.184/1645
3033_02F4	SW_PAD_CTL_PAD_ENET_RXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_RXC)	32	R/W	0000_0016h	8.2.5.185/1647
3033_02F8	SW_PAD_CTL_PAD_ENET_RD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_RD0)	32	R/W	0000_0016h	8.2.5.186/1648
3033_02FC	SW_PAD_CTL_PAD_ENET_RD1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_RD1)	32	R/W	0000_0016h	8.2.5.187/1650
3033_0300	SW_PAD_CTL_PAD_ENET_RD2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_RD2)	32	R/W	0000_0016h	8.2.5.188/1651
3033_0304	SW_PAD_CTL_PAD_ENET_RD3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_RD3)	32	R/W	0000_0016h	8.2.5.189/1653
3033_0308	SW_PAD_CTL_PAD_SD1_CLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_CLK)	32	R/W	0000_0016h	8.2.5.190/1654
3033_030C	SW_PAD_CTL_PAD_SD1_CMD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_CMD)	32	R/W	0000_0016h	8.2.5.191/1656
3033_0310	SW_PAD_CTL_PAD_SD1_DATA0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA0)	32	R/W	0000_0016h	8.2.5.192/1657
3033_0314	SW_PAD_CTL_PAD_SD1_DATA1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA1)	32	R/W	0000_0016h	8.2.5.193/1659
3033_0318	SW_PAD_CTL_PAD_SD1_DATA2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA2)	32	R/W	0000_0016h	8.2.5.194/1660
3033_031C	SW_PAD_CTL_PAD_SD1_DATA3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA3)	32	R/W	0000_1816h	8.2.5.195/1662
3033_0320	SW_PAD_CTL_PAD_SD1_DATA4 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA4)	32	R/W	0000_0016h	8.2.5.196/1664
3033_0324	SW_PAD_CTL_PAD_SD1_DATA5 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA5)	32	R/W	0000_0016h	8.2.5.197/1665
3033_0328	SW_PAD_CTL_PAD_SD1_DATA6 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA6)	32	R/W	0000_0016h	8.2.5.198/1667
3033_032C	SW_PAD_CTL_PAD_SD1_DATA7 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA7)	32	R/W	0000_0016h	8.2.5.199/1668
3033_0330	SW_PAD_CTL_PAD_SD1_RESET_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_RESET_B)	32	R/W	0000_0016h	8.2.5.200/1670
3033_0334	SW_PAD_CTL_PAD_SD1_STROBE SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_STROBE)	32	R/W	0000_0016h	8.2.5.201/1671

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3033_0338	SW_PAD_CTL_PAD_SD2_CD_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_CD_B)	32	R/W	0000_0016h	8.2.5.202/1673
3033_033C	SW_PAD_CTL_PAD_SD2_CLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_CLK)	32	R/W	0000_0016h	8.2.5.203/1674
3033_0340	SW_PAD_CTL_PAD_SD2_CMD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_CMD)	32	R/W	0000_0016h	8.2.5.204/1676
3033_0344	SW_PAD_CTL_PAD_SD2_DATA0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_DATA0)	32	R/W	0000_1816h	8.2.5.205/1677
3033_0348	SW_PAD_CTL_PAD_SD2_DATA1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_DATA1)	32	R/W	0000_0016h	8.2.5.206/1679
3033_034C	SW_PAD_CTL_PAD_SD2_DATA2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_DATA2)	32	R/W	0000_0016h	8.2.5.207/1681
3033_0350	SW_PAD_CTL_PAD_SD2_DATA3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_DATA3)	32	R/W	0000_0016h	8.2.5.208/1682
3033_0354	SW_PAD_CTL_PAD_SD2_RESET_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_RESET_B)	32	R/W	0000_0016h	8.2.5.209/1684
3033_0358	SW_PAD_CTL_PAD_SD2_WP SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_WP)	32	R/W	0000_0016h	8.2.5.210/1685
3033_035C	SW_PAD_CTL_PAD_NAND_ALE SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_ALE)	32	R/W	0000_0016h	8.2.5.211/1687
3033_0360	SW_PAD_CTL_PAD_NAND_CE0_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_CE0_B)	32	R/W	0000_0016h	8.2.5.212/1688
3033_0364	SW_PAD_CTL_PAD_NAND_CE1_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_CE1_B)	32	R/W	0000_0016h	8.2.5.213/1690
3033_0368	SW_PAD_CTL_PAD_NAND_CE2_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_CE2_B)	32	R/W	0000_0016h	8.2.5.214/1691
3033_036C	SW_PAD_CTL_PAD_NAND_CE3_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_CE3_B)	32	R/W	0000_0016h	8.2.5.215/1693
3033_0370	SW_PAD_CTL_PAD_NAND_CLE SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_CLE)	32	R/W	0000_0016h	8.2.5.216/1694
3033_0374	SW_PAD_CTL_PAD_NAND_DATA00 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA00)	32	R/W	0000_0016h	8.2.5.217/1696
3033_0378	SW_PAD_CTL_PAD_NAND_DATA01 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA01)	32	R/W	0000_0016h	8.2.5.218/1697
3033_037C	SW_PAD_CTL_PAD_NAND_DATA02 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA02)	32	R/W	0000_0016h	8.2.5.219/1699
3033_0380	SW_PAD_CTL_PAD_NAND_DATA03 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA03)	32	R/W	0000_1816h	8.2.5.220/1700
3033_0384	SW_PAD_CTL_PAD_NAND_DATA04 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA04)	32	R/W	0000_0016h	8.2.5.221/1702
3033_0388	SW_PAD_CTL_PAD_NAND_DATA05 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA05)	32	R/W	0000_0016h	8.2.5.222/1704
3033_038C	SW_PAD_CTL_PAD_NAND_DATA06 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA06)	32	R/W	0000_0016h	8.2.5.223/1705

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3033_0390	SW_PAD_CTL_PAD_NAND_DATA07 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA07)	32	R/W	0000_0016h	8.2.5.224/1707
3033_0394	SW_PAD_CTL_PAD_NAND_DQS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DQS)	32	R/W	0000_0016h	8.2.5.225/1708
3033_0398	SW_PAD_CTL_PAD_NAND_RE_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_RE_B)	32	R/W	0000_0016h	8.2.5.226/1710
3033_039C	SW_PAD_CTL_PAD_NAND_READY_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_READY_B)	32	R/W	0000_0016h	8.2.5.227/1711
3033_03A0	SW_PAD_CTL_PAD_NAND_WE_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_WE_B)	32	R/W	0000_0016h	8.2.5.228/1713
3033_03A4	SW_PAD_CTL_PAD_NAND_WP_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_WP_B)	32	R/W	0000_0016h	8.2.5.229/1714
3033_03A8	SW_PAD_CTL_PAD_SAI5_RXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_RXFS)	32	R/W	0000_0016h	8.2.5.230/1716
3033_03AC	SW_PAD_CTL_PAD_SAI5_RXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_RXC)	32	R/W	0000_0016h	8.2.5.231/1717
3033_03B0	SW_PAD_CTL_PAD_SAI5_RXD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD0)	32	R/W	0000_0016h	8.2.5.232/1719
3033_03B4	SW_PAD_CTL_PAD_SAI5_RXD1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD1)	32	R/W	0000_1816h	8.2.5.233/1720
3033_03B8	SW_PAD_CTL_PAD_SAI5_RXD2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD2)	32	R/W	0000_0016h	8.2.5.234/1722
3033_03BC	SW_PAD_CTL_PAD_SAI5_RXD3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD3)	32	R/W	0000_0016h	8.2.5.235/1724
3033_03C0	SW_PAD_CTL_PAD_SAI5_MCLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_MCLK)	32	R/W	0000_0016h	8.2.5.236/1725
3033_03C4	SW_PAD_CTL_PAD_SAI1_RXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXFS)	32	R/W	0000_0016h	8.2.5.237/1727
3033_03C8	SW_PAD_CTL_PAD_SAI1_RXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXC)	32	R/W	0000_0016h	8.2.5.238/1728
3033_03CC	SW_PAD_CTL_PAD_SAI1_RXD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD0)	32	R/W	0000_0016h	8.2.5.239/1730
3033_03D0	SW_PAD_CTL_PAD_SAI1_RXD1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD1)	32	R/W	0000_0016h	8.2.5.240/1731
3033_03D4	SW_PAD_CTL_PAD_SAI1_RXD2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD2)	32	R/W	0000_0016h	8.2.5.241/1733
3033_03D8	SW_PAD_CTL_PAD_SAI1_RXD3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD3)	32	R/W	0000_0016h	8.2.5.242/1734
3033_03DC	SW_PAD_CTL_PAD_SAI1_RXD4 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD4)	32	R/W	0000_0016h	8.2.5.243/1736
3033_03E0	SW_PAD_CTL_PAD_SAI1_RXD5 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD5)	32	R/W	0000_0016h	8.2.5.244/1737
3033_03E4	SW_PAD_CTL_PAD_SAI1_RXD6 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD6)	32	R/W	0000_0016h	8.2.5.245/1739

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3033_03E8	SW_PAD_CTL_PAD_SAI1_RXD7 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD7)	32	R/W	0000_0016h	8.2.5.246/1740
3033_03EC	SW_PAD_CTL_PAD_SAI1_TXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXFS)	32	R/W	0000_1816h	8.2.5.247/1742
3033_03F0	SW_PAD_CTL_PAD_SAI1_TXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXC)	32	R/W	0000_0016h	8.2.5.248/1744
3033_03F4	SW_PAD_CTL_PAD_SAI1_TXD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD0)	32	R/W	0000_0016h	8.2.5.249/1745
3033_03F8	SW_PAD_CTL_PAD_SAI1_TXD1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD1)	32	R/W	0000_0016h	8.2.5.250/1747
3033_03FC	SW_PAD_CTL_PAD_SAI1_TXD2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD2)	32	R/W	0000_0016h	8.2.5.251/1748
3033_0400	SW_PAD_CTL_PAD_SAI1_TXD3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD3)	32	R/W	0000_0016h	8.2.5.252/1750
3033_0404	SW_PAD_CTL_PAD_SAI1_TXD4 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD4)	32	R/W	0000_0016h	8.2.5.253/1751
3033_0408	SW_PAD_CTL_PAD_SAI1_TXD5 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD5)	32	R/W	0000_0016h	8.2.5.254/1753
3033_040C	SW_PAD_CTL_PAD_SAI1_TXD6 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD6)	32	R/W	0000_0016h	8.2.5.255/1754
3033_0410	SW_PAD_CTL_PAD_SAI1_TXD7 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD7)	32	R/W	0000_0016h	8.2.5.256/1756
3033_0414	SW_PAD_CTL_PAD_SAI1_MCLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_MCLK)	32	R/W	0000_0016h	8.2.5.257/1757
3033_0418	SW_PAD_CTL_PAD_SAI2_RXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_RXFS)	32	R/W	0000_0016h	8.2.5.258/1759
3033_041C	SW_PAD_CTL_PAD_SAI2_RXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_RXC)	32	R/W	0000_0016h	8.2.5.259/1760
3033_0420	SW_PAD_CTL_PAD_SAI2_RXD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_RXD0)	32	R/W	0000_0016h	8.2.5.260/1762
3033_0424	SW_PAD_CTL_PAD_SAI2_TXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_TXFS)	32	R/W	0000_1816h	8.2.5.261/1763
3033_0428	SW_PAD_CTL_PAD_SAI2_TXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_TXC)	32	R/W	0000_0016h	8.2.5.262/1765
3033_042C	SW_PAD_CTL_PAD_SAI2_TXD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_TXD0)	32	R/W	0000_0016h	8.2.5.263/1767
3033_0430	SW_PAD_CTL_PAD_SAI2_MCLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_MCLK)	32	R/W	0000_0016h	8.2.5.264/1768
3033_0434	SW_PAD_CTL_PAD_SAI3_RXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_RXFS)	32	R/W	0000_0016h	8.2.5.265/1770
3033_0438	SW_PAD_CTL_PAD_SAI3_RXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_RXC)	32	R/W	0000_0016h	8.2.5.266/1771
3033_043C	SW_PAD_CTL_PAD_SAI3_RXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_RXD)	32	R/W	0000_0016h	8.2.5.267/1773

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3033_0440	SW_PAD_CTL_PAD_SAI3_TXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_TXFS)	32	R/W	0000_0016h	8.2.5.268/1774
3033_0444	SW_PAD_CTL_PAD_SAI3_TXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_TXC)	32	R/W	0000_1816h	8.2.5.269/1776
3033_0448	SW_PAD_CTL_PAD_SAI3_TXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_TXD)	32	R/W	0000_0016h	8.2.5.270/1778
3033_044C	SW_PAD_CTL_PAD_SAI3_MCLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_MCLK)	32	R/W	0000_0016h	8.2.5.271/1779
3033_0450	SW_PAD_CTL_PAD_SPDIF_TX SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SPDIF_TX)	32	R/W	0000_0016h	8.2.5.272/1781
3033_0454	SW_PAD_CTL_PAD_SPDIF_RX SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SPDIF_RX)	32	R/W	0000_0016h	8.2.5.273/1782
3033_0458	SW_PAD_CTL_PAD_SPDIF_EXT_CLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SPDIF_EXT_CLK)	32	R/W	0000_0016h	8.2.5.274/1784
3033_045C	SW_PAD_CTL_PAD_ECSP11_SCLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSP11_SCLK)	32	R/W	0000_0016h	8.2.5.275/1785
3033_0460	SW_PAD_CTL_PAD_ECSP11_MOSI SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSP11_MOSI)	32	R/W	0000_0016h	8.2.5.276/1787
3033_0464	SW_PAD_CTL_PAD_ECSP11_MISO SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSP11_MISO)	32	R/W	0000_0016h	8.2.5.277/1788
3033_0468	SW_PAD_CTL_PAD_ECSP11_SS0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSP11_SS0)	32	R/W	0000_0016h	8.2.5.278/1790
3033_046C	SW_PAD_CTL_PAD_ECSP12_SCLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSP12_SCLK)	32	R/W	0000_1816h	8.2.5.279/1791
3033_0470	SW_PAD_CTL_PAD_ECSP12_MOSI SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSP12_MOSI)	32	R/W	0000_0016h	8.2.5.280/1793
3033_0474	SW_PAD_CTL_PAD_ECSP12_MISO SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSP12_MISO)	32	R/W	0000_0016h	8.2.5.281/1795
3033_0478	SW_PAD_CTL_PAD_ECSP12_SS0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSP12_SS0)	32	R/W	0000_0016h	8.2.5.282/1796
3033_047C	SW_PAD_CTL_PAD_I2C1_SCL SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C1_SCL)	32	R/W	0000_0016h	8.2.5.283/1798
3033_0480	SW_PAD_CTL_PAD_I2C1_SDA SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C1_SDA)	32	R/W	0000_0016h	8.2.5.284/1799
3033_0484	SW_PAD_CTL_PAD_I2C2_SCL SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C2_SCL)	32	R/W	0000_0016h	8.2.5.285/1801
3033_0488	SW_PAD_CTL_PAD_I2C2_SDA SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C2_SDA)	32	R/W	0000_0016h	8.2.5.286/1802
3033_048C	SW_PAD_CTL_PAD_I2C3_SCL SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C3_SCL)	32	R/W	0000_1816h	8.2.5.287/1804
3033_0490	SW_PAD_CTL_PAD_I2C3_SDA SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C3_SDA)	32	R/W	0000_0016h	8.2.5.288/1806
3033_0494	SW_PAD_CTL_PAD_I2C4_SCL SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C4_SCL)	32	R/W	0000_0016h	8.2.5.289/1807

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3033_0498	SW_PAD_CTL_PAD_I2C4_SDA SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C4_SDA)	32	R/W	0000_0016h	8.2.5.290/1809
3033_049C	SW_PAD_CTL_PAD_UART1_RXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART1_RXD)	32	R/W	0000_0016h	8.2.5.291/1810
3033_04A0	SW_PAD_CTL_PAD_UART1_TXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART1_TXD)	32	R/W	0000_0016h	8.2.5.292/1812
3033_04A4	SW_PAD_CTL_PAD_UART2_RXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART2_RXD)	32	R/W	0000_0016h	8.2.5.293/1813
3033_04A8	SW_PAD_CTL_PAD_UART2_TXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART2_TXD)	32	R/W	0000_0016h	8.2.5.294/1815
3033_04AC	SW_PAD_CTL_PAD_UART3_RXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART3_RXD)	32	R/W	0000_1816h	8.2.5.295/1816
3033_04B0	SW_PAD_CTL_PAD_UART3_TXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART3_TXD)	32	R/W	0000_0016h	8.2.5.296/1818
3033_04B4	SW_PAD_CTL_PAD_UART4_RXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART4_RXD)	32	R/W	0000_0016h	8.2.5.297/1820
3033_04B8	SW_PAD_CTL_PAD_UART4_TXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART4_TXD)	32	R/W	0000_0016h	8.2.5.298/1821
3033_04BC	CCM_PMIC_READY_SELECT_INPUT DAISY Register (IOMUXC_CCM_PMIC_READY_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.299/1823
3033_04C0	ENET1_MDIO_SELECT_INPUT DAISY Register (IOMUXC_ENET1_MDIO_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.300/1823
3033_04C4	SAI1_RX_SYNC_SELECT_INPUT DAISY Register (IOMUXC_SAI1_RX_SYNC_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.301/1824
3033_04C8	SAI1_TX_BCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI1_TX_BCLK_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.302/1825
3033_04CC	SAI1_TX_SYNC_SELECT_INPUT DAISY Register (IOMUXC_SAI1_TX_SYNC_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.303/1825
3033_04D0	SAI5_RX_BCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI5_RX_BCLK_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.304/1826
3033_04D4	SAI5_RXD0_SELECT_INPUT DAISY Register (IOMUXC_SAI5_RXD0_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.305/1827
3033_04D8	SAI5_RXD1_SELECT_INPUT DAISY Register (IOMUXC_SAI5_RXD1_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.306/1827
3033_04DC	SAI5_RXD2_SELECT_INPUT DAISY Register (IOMUXC_SAI5_RXD2_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.307/1828
3033_04E0	SAI5_RXD3_SELECT_INPUT DAISY Register (IOMUXC_SAI5_RXD3_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.308/1829
3033_04E4	SAI5_RX_SYNC_SELECT_INPUT DAISY Register (IOMUXC_SAI5_RX_SYNC_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.309/1829
3033_04E8	SAI5_TX_BCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI5_TX_BCLK_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.310/1830
3033_04EC	SAI5_TX_SYNC_SELECT_INPUT DAISY Register (IOMUXC_SAI5_TX_SYNC_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.311/1831

Table continues on the next page...

IOMUXC memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3033_04F0	UART1_RTS_B_SELECT_INPUT DAISY Register (IOMUXC_UART1_RTS_B_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.312/1832
3033_04F4	UART1_RXD_SELECT_INPUT DAISY Register (IOMUXC_UART1_RXD_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.313/1833
3033_04F8	UART2_RTS_B_SELECT_INPUT DAISY Register (IOMUXC_UART2_RTS_B_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.314/1834
3033_04FC	UART2_RXD_SELECT_INPUT DAISY Register (IOMUXC_UART2_RXD_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.315/1835
3033_0500	UART3_RTS_B_SELECT_INPUT DAISY Register (IOMUXC_UART3_RTS_B_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.316/1836
3033_0504	UART3_RXD_SELECT_INPUT DAISY Register (IOMUXC_UART3_RXD_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.317/1836
3033_0508	UART4_RTS_B_SELECT_INPUT DAISY Register (IOMUXC_UART4_RTS_B_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.318/1837
3033_050C	UART4_RXD_SELECT_INPUT DAISY Register (IOMUXC_UART4_RXD_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.319/1838
3033_0510	SAI6_RX_BCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI6_RX_BCLK_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.320/1839
3033_0514	SAI6_RXD0_SELECT_INPUT DAISY Register (IOMUXC_SAI6_RXD0_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.321/1840
3033_0518	SAI6_RX_SYNC_SELECT_INPUT DAISY Register (IOMUXC_SAI6_RX_SYNC_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.322/1841
3033_051C	SAI6_TX_BCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI6_TX_BCLK_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.323/1842
3033_0520	SAI6_TX_SYNC_SELECT_INPUT DAISY Register (IOMUXC_SAI6_TX_SYNC_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.324/1843
3033_0524	PCIE1_CLKREQ_B_SELECT_INPUT DAISY Register (IOMUXC_PCIE1_CLKREQ_B_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.325/1844
3033_0528	PCIE2_CLKREQ_B_SELECT_INPUT DAISY Register (IOMUXC_PCIE2_CLKREQ_B_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.326/1845
3033_052C	SAI5_MCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI5_MCLK_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.327/1845
3033_0530	SAI6_MCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI6_MCLK_SELECT_INPUT)	32	R/W	0000_0000h	8.2.5.328/1846

8.2.5.1 SW_MUX_CTL_PAD_PMIC_STBY_REQ SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_PMIC_STBY_REQ)

SW_MUX_CTL Register

Address: 3033_0000h base + 14h offset = 3033_0014h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SW_MUX_CTL_PAD_PMIC_STBY_REQ field descriptions

Field	Description
31–7 -	This field is reserved. Reserved
6 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad PMIC_STBY_REQ is determined by functionality 1 SION_ENABLED — Force Input Path of pad PMIC_STBY_REQ
-	This field is reserved. Reserved

8.2.5.2 SW_MUX_CTL_PAD_PMIC_ON_REQ SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_PMIC_ON_REQ)

SW_MUX_CTL Register

Address: 3033_0000h base + 18h offset = 3033_0018h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SW_MUX_CTL_PAD_PMIC_ON_REQ field descriptions

Field	Description
31–7 -	This field is reserved. Reserved
6 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad PMIC_ON_REQ is determined by functionality 1 SION_ENABLED — Force Input Path of pad PMIC_ON_REQ
-	This field is reserved. Reserved

8.2.5.3 SW_MUX_CTL_PAD_ONOFF SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ONOFF)**SW_MUX_CTL Register**

Address: 3033_0000h base + 1Ch offset = 3033_001Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SW_MUX_CTL_PAD_ONOFF field descriptions

Field	Description
31–7 -	This field is reserved. Reserved
6 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ONOFF is determined by functionality 1 SION_ENABLED — Force Input Path of pad ONOFF
-	This field is reserved. Reserved

8.2.5.4 SW_MUX_CTL_PAD_POR_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_POR_B)

SW_MUX_CTL Register

Address: 3033_0000h base + 20h offset = 3033_0020h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved									SION	Reserved					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SW_MUX_CTL_PAD_POR_B field descriptions

Field	Description
31–7 -	This field is reserved. Reserved
6 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad POR_B is determined by functionality 1 SION_ENABLED — Force Input Path of pad POR_B
-	This field is reserved. Reserved

8.2.5.5 SW_MUX_CTL_PAD_RTC_RESET_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_RTC_RESET_B)

SW_MUX_CTL Register

Address: 3033_0000h base + 24h offset = 3033_0024h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved									SION	Reserved					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SW_MUX_CTL_PAD_RTC_RESET_B field descriptions

Field	Description
31–7 -	This field is reserved. Reserved
6 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad RTC_RESET_B is determined by functionality 1 SION_ENABLED — Force Input Path of pad RTC_RESET_B
-	This field is reserved. Reserved

8.2.5.6 SW_MUX_CTL_PAD_GPIO1_IO00 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO00)**SW_MUX_CTL Register**

Address: 3033_0000h base + 28h offset = 3033_0028h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved										SION	Reserved	MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO00 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO00 field descriptions (continued)

Field	Description
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO00 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO00
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: GPIO1_IO00 000 ALT0_GPIO1_IO00 — Select mux mode: ALT0 mux port: IO00 of instance: GPIO1 001 ALT1_CCM_ENET_PHY_REF_CLK_ROOT — Select mux mode: ALT1 mux port: ENET_PHY_REF_CLK_ROOT of instance: CCM 101 ALT5_ANAMIX_REF_CLK_32K — Select mux mode: ALT5 mux port: REF_CLK_32K of instance: ANAMIX 110 ALT6_CCM_EXT_CLK1 — Select mux mode: ALT6 mux port: EXT_CLK1 of instance: CCM

8.2.5.7 SW_MUX_CTL_PAD_GPIO1_IO01 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO01)**SW_MUX_CTL Register**

Address: 3033_0000h base + 2Ch offset = 3033_002Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												SION	Reserved	MUX_MODE	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

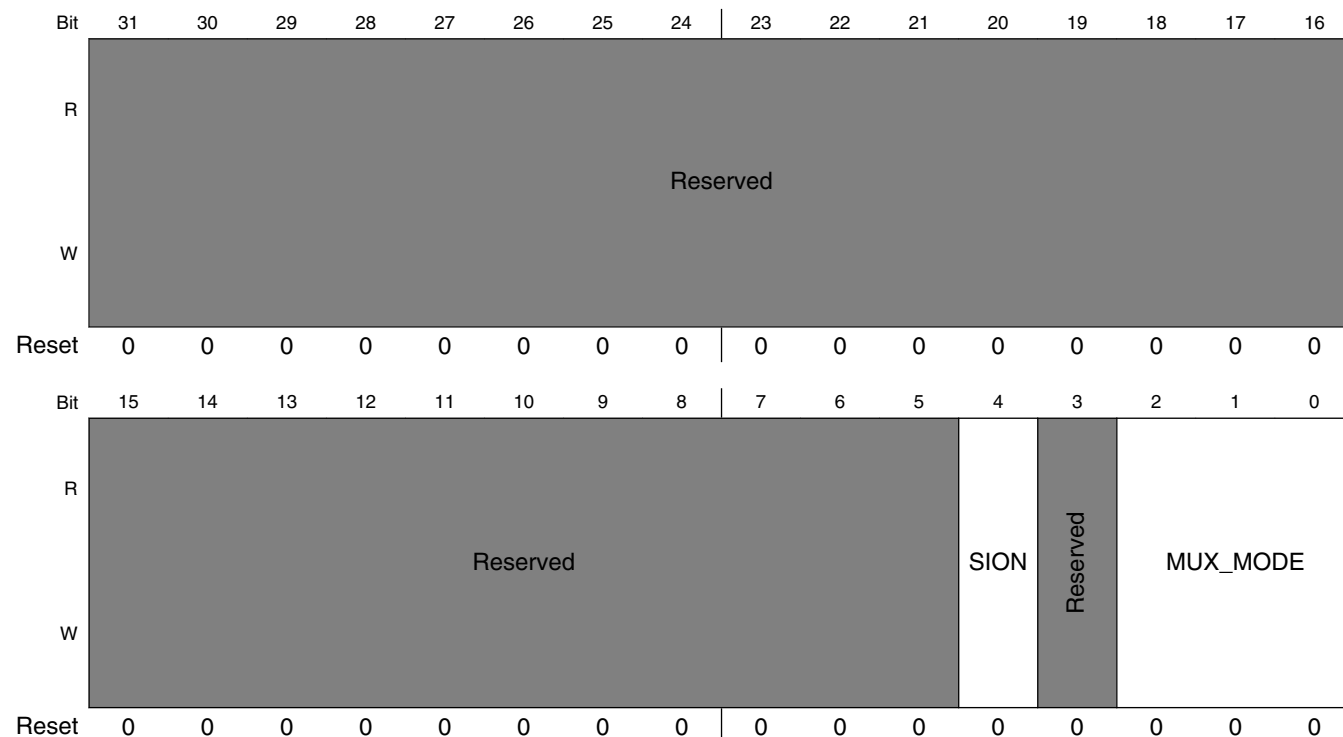
IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO01 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO01 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO01
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: GPIO1_IO01 000 ALT0_GPIO1_IO01 — Select mux mode: ALT0 mux port: IO01 of instance: GPIO1 001 ALT1_PWM1_OUT — Select mux mode: ALT1 mux port: OUT of instance: PWM1 101 ALT5_ANAMIX_REF_CLK_25M — Select mux mode: ALT5 mux port: REF_CLK_25M of instance: ANAMIX 110 ALT6_CCM_EXT_CLK2 — Select mux mode: ALT6 mux port: EXT_CLK2 of instance: CCM

8.2.5.8 SW_MUX_CTL_PAD_GPIO1_IO02 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO02)

SW_MUX_CTL Register

Address: 3033_0000h base + 30h offset = 3033_0030h



IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO02 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO02 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO02
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: GPIO1_IO02 000 ALT0_GPIO1_IO02 — Select mux mode: ALT0 mux port: IO02 of instance: GPIO1 001 ALT1_WDOG1_WDOG_B — Select mux mode: ALT1 mux port: WDOG_B of instance: WDOG1

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO02 field descriptions (continued)

Field	Description
101	ALT5_WDOG1_WDOG_ANY — Select mux mode: ALT5 mux port: WDOG_ANY of instance: WDOG1
111	ALT7_SJC_DE_B — Select mux mode: ALT7 mux port: DE_B of instance: SJC

8.2.5.9 SW_MUX_CTL_PAD_GPIO1_IO03 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO03)**SW_MUX_CTL Register**

Address: 3033_0000h base + 34h offset = 3033_0034h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO03 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO03 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO03

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO03 field descriptions (continued)

Field	Description
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: GPIO1_IO03 000 ALT0_GPIO1_IO03 — Select mux mode: ALT0 mux port: IO03 of instance: GPIO1 001 ALT1_USDHC1_VSELECT — Select mux mode: ALT1 mux port: VSELECT of instance: USDHC1 101 ALT5_SDMA1_EXT_EVENT0 — Select mux mode: ALT5 mux port: EXT_EVENT0 of instance: SDMA1

8.2.5.10 SW_MUX_CTL_PAD_GPIO1_IO04 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO04)**SW_MUX_CTL Register**

Address: 3033_0000h base + 38h offset = 3033_0038h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved											SION	Reserved	MUX_MODE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO04 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO04 field descriptions (continued)

Field	Description
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO04 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO04
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: GPIO1_IO04 000 ALT0_GPIO1_IO04 — Select mux mode: ALT0 mux port: IO04 of instance: GPIO1 001 ALT1_USDHC2_VSELECT — Select mux mode: ALT1 mux port: VSELECT of instance: USDHC2 101 ALT5_SDMA1_EXT_EVENT1 — Select mux mode: ALT5 mux port: EXT_EVENT1 of instance: SDMA1

8.2.5.11 SW_MUX_CTL_PAD_GPIO1_IO05 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO05)**SW_MUX_CTL Register**

Address: 3033_0000h base + 3Ch offset = 3033_003Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved											SION	Reserved	MUX_MODE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO05 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO05 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO05
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: GPIO1_IO05 000 ALT0_GPIO1_IO05 — Select mux mode: ALT0 mux port: IO05 of instance: GPIO1 001 ALT1_M4_NMI — Select mux mode: ALT1 mux port: NMI of instance: M4 101 ALT5_CCM_PMIC_READY — Select mux mode: ALT5 mux port: PMIC_READY of instance: CCM

8.2.5.12 SW_MUX_CTL_PAD_GPIO1_IO06 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO06)

SW_MUX_CTL Register

Address: 3033_0000h base + 40h offset = 3033_0040h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved										SION	Reserved	MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO06 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO06 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO06
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: GPIO1_IO06 000 ALT0_GPIO1_IO06 — Select mux mode: ALT0 mux port: IO06 of instance: GPIO1 001 ALT1_ENET1_MDC — Select mux mode: ALT1 mux port: MDC of instance: ENET1 101 ALT5_USDHC1_CD_B — Select mux mode: ALT5 mux port: CD_B of instance: USDHC1 110 ALT6_CCM_EXT_CLK3 — Select mux mode: ALT6 mux port: EXT_CLK3 of instance: CCM

8.2.5.13 SW_MUX_CTL_PAD_GPIO1_IO07 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO07)**SW_MUX_CTL Register**

Address: 3033_0000h base + 44h offset = 3033_0044h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												SION	Reserved	MUX_MODE	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO07 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO07 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO07
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: GPIO1_IO07 000 ALT0_GPIO1_IO07 — Select mux mode: ALT0 mux port: IO07 of instance: GPIO1 001 ALT1_ENET1_MDIO — Select mux mode: ALT1 mux port: MDIO of instance: ENET1 101 ALT5_USDHC1_WP — Select mux mode: ALT5 mux port: WP of instance: USDHC1 110 ALT6_CCM_EXT_CLK4 — Select mux mode: ALT6 mux port: EXT_CLK4 of instance: CCM

8.2.5.14 SW_MUX_CTL_PAD_GPIO1_IO08 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO08)**SW_MUX_CTL Register**

Address: 3033_0000h base + 48h offset = 3033_0048h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												SION	Reserved	MUX_MODE	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO08 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO08 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO08
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: GPIO1_IO08 000 ALT0_GPIO1_IO08 — Select mux mode: ALT0 mux port: IO08 of instance: GPIO1 001 ALT1_ENET1_1588_EVENT0_IN — Select mux mode: ALT1 mux port: 1588_EVENT0_IN of instance: ENET1 101 ALT5_USDHC2_RESET_B — Select mux mode: ALT5 mux port: RESET_B of instance: USDHC2

8.2.5.15 SW_MUX_CTL_PAD_GPIO1_IO09 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO09)**SW_MUX_CTL Register**

Address: 3033_0000h base + 4Ch offset = 3033_004Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved										SION	Reserved	MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

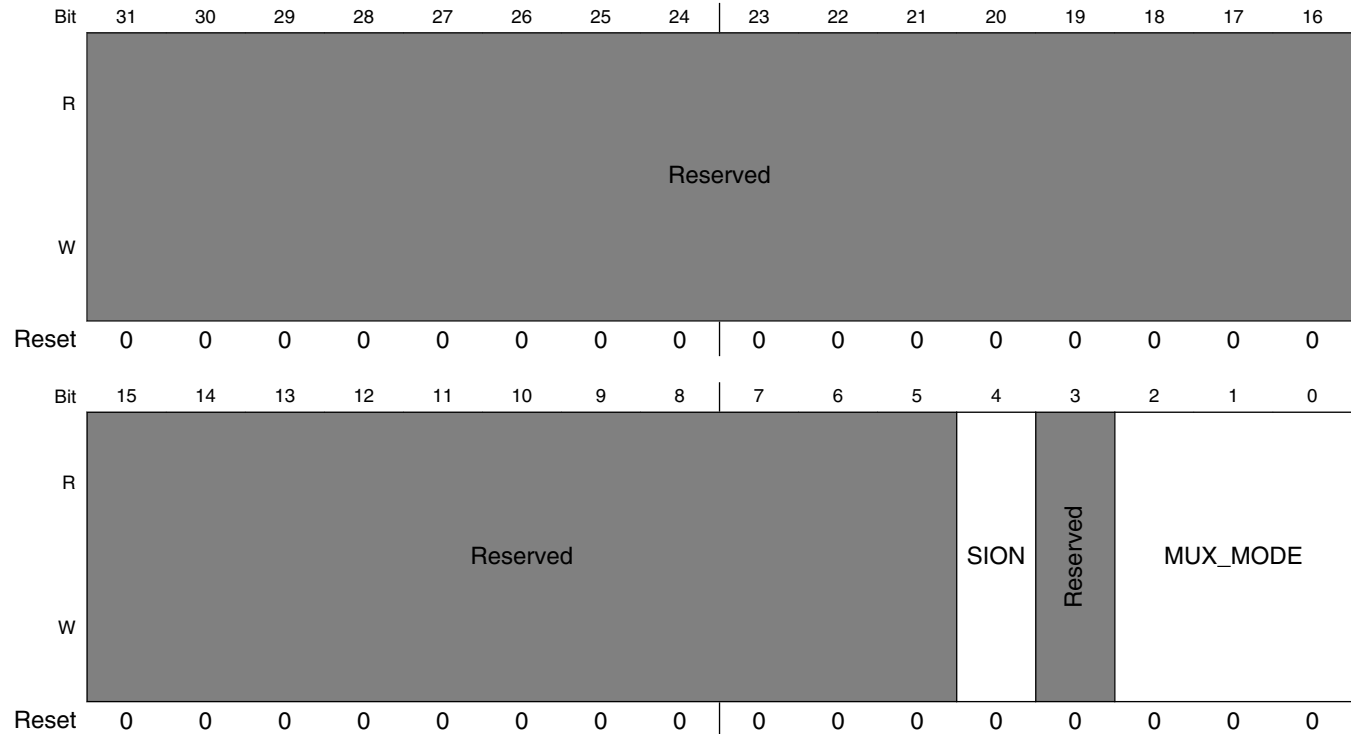
IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO09 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO09 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO09
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: GPIO1_IO09 000 ALT0_GPIO1_IO09 — Select mux mode: ALT0 mux port: IO09 of instance: GPIO1 001 ALT1_ENET1_1588_EVENT0_OUT — Select mux mode: ALT1 mux port: 1588_EVENT0_OUT of instance: ENET1 101 ALT5_SDMA2_EXT_EVENT0 — Select mux mode: ALT5 mux port: EXT_EVENT0 of instance: SDMA2

8.2.5.16 SW_MUX_CTL_PAD_GPIO1_IO10 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO10)

SW_MUX_CTL Register

Address: 3033_0000h base + 50h offset = 3033_0050h



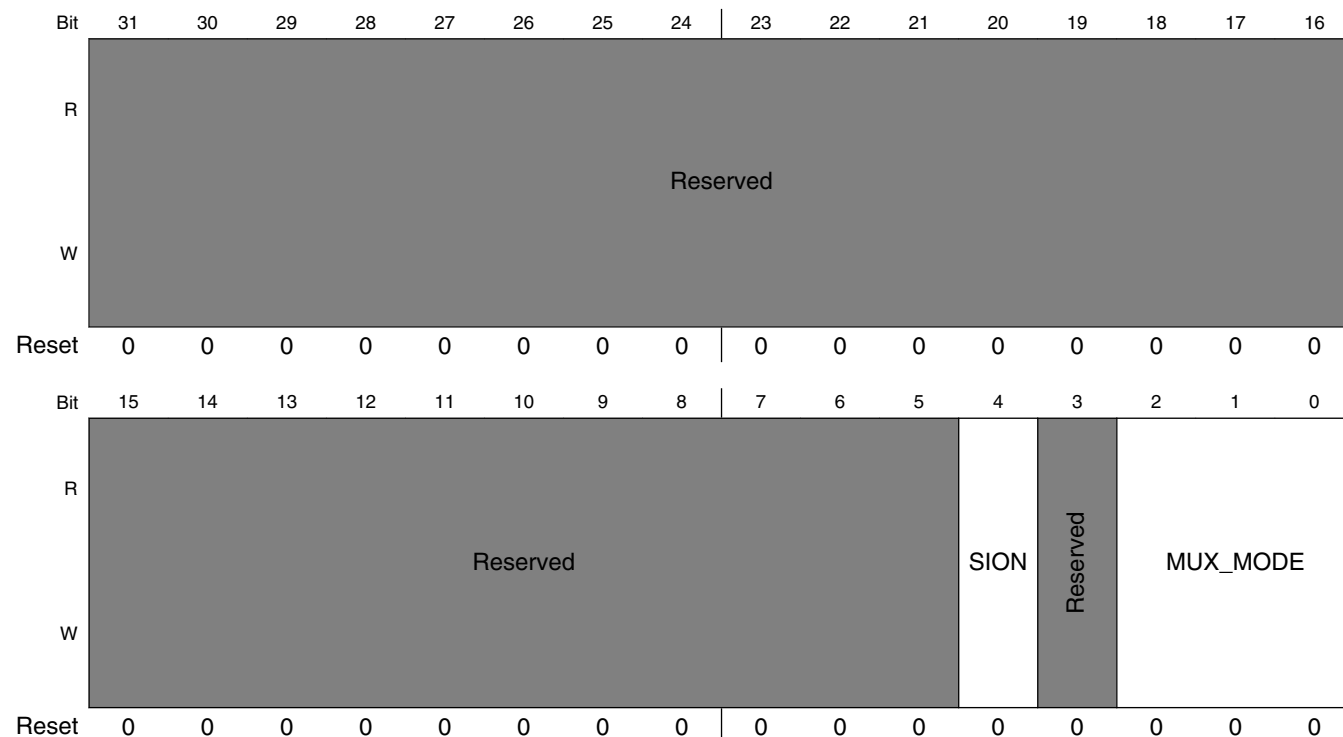
IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO10 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO10 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO10
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: GPIO1_IO10 000 ALT0_GPIO1_IO10 — Select mux mode: ALT0 mux port: IO10 of instance: GPIO1 001 ALT1_USB1_OTG_ID — Select mux mode: ALT1 mux port: OTG_ID of instance: USB1

8.2.5.17 SW_MUX_CTL_PAD_GPIO1_IO11 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO11)

SW_MUX_CTL Register

Address: 3033_0000h base + 54h offset = 3033_0054h



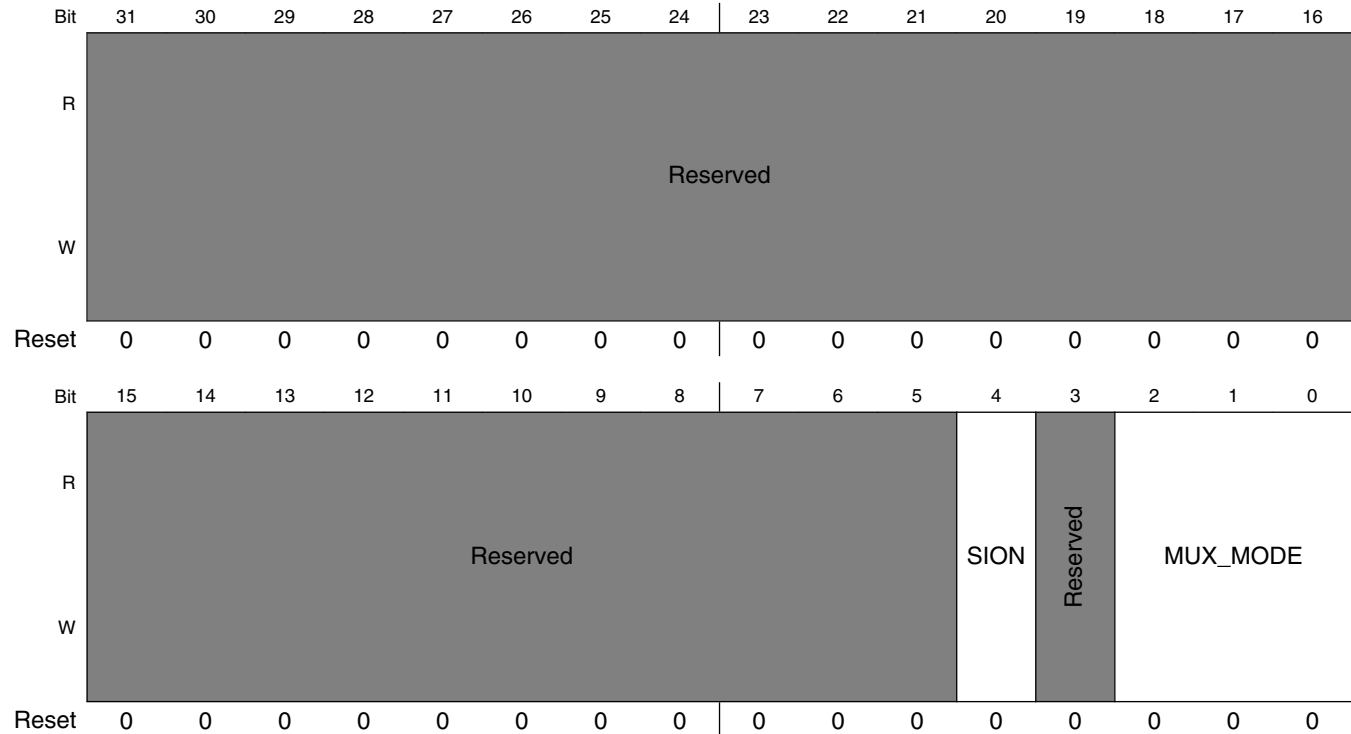
IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO11 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO11 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO11
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: GPIO1_IO11 000 ALT0_GPIO1_IO11 — Select mux mode: ALT0 mux port: IO11 of instance: GPIO1 001 ALT1_USB2_OTG_ID — Select mux mode: ALT1 mux port: OTG_ID of instance: USB2 101 ALT5_CCM_PMIC_READY — Select mux mode: ALT5 mux port: PMIC_READY of instance: CCM

8.2.5.18 SW_MUX_CTL_PAD_GPIO1_IO12 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO12)

SW_MUX_CTL Register

Address: 3033_0000h base + 58h offset = 3033_0058h



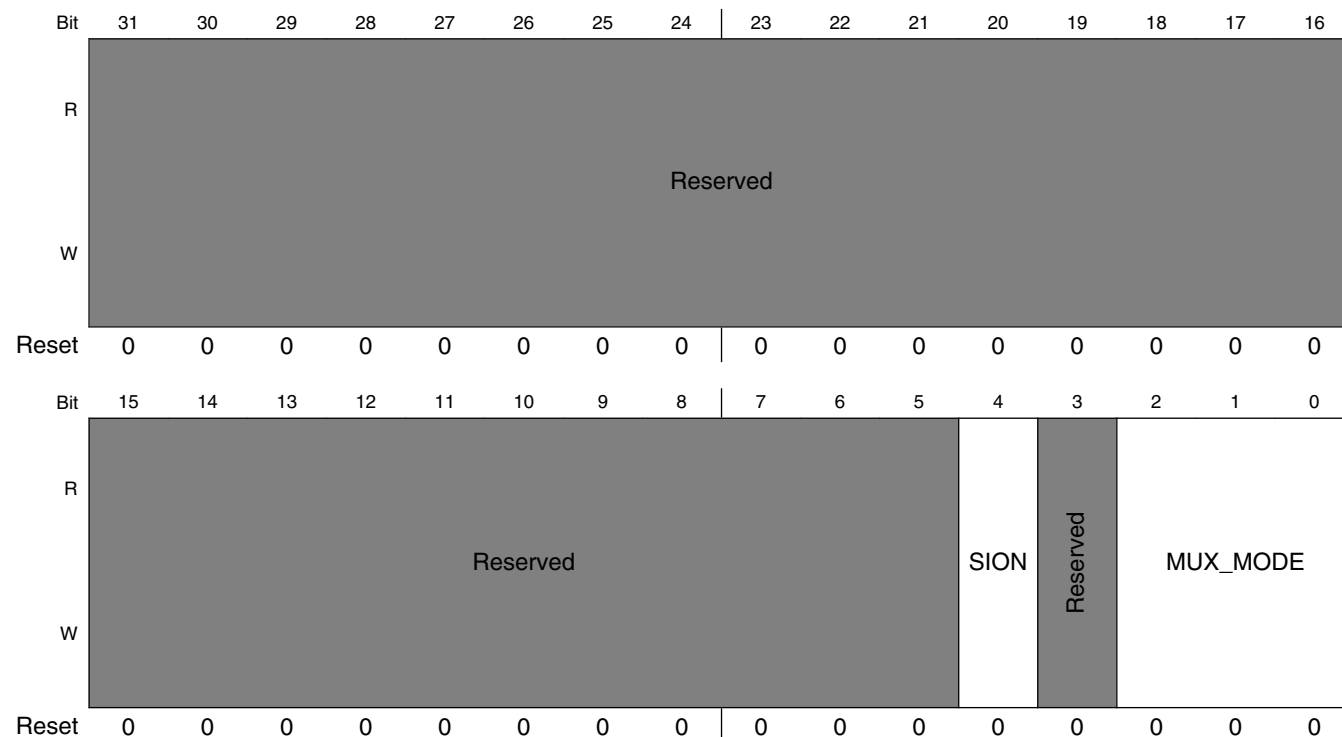
IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO12 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO12 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO12
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: GPIO1_IO12 000 ALT0_GPIO1_IO12 — Select mux mode: ALT0 mux port: IO12 of instance: GPIO1 001 ALT1_USB1_OTG_PWR — Select mux mode: ALT1 mux port: OTG_PWR of instance: USB1 101 ALT5_SDMA2_EXT_EVENT1 — Select mux mode: ALT5 mux port: EXT_EVENT1 of instance: SDMA2

8.2.5.19 SW_MUX_CTL_PAD_GPIO1_IO13 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO13)

SW_MUX_CTL Register

Address: 3033_0000h base + 5Ch offset = 3033_005Ch



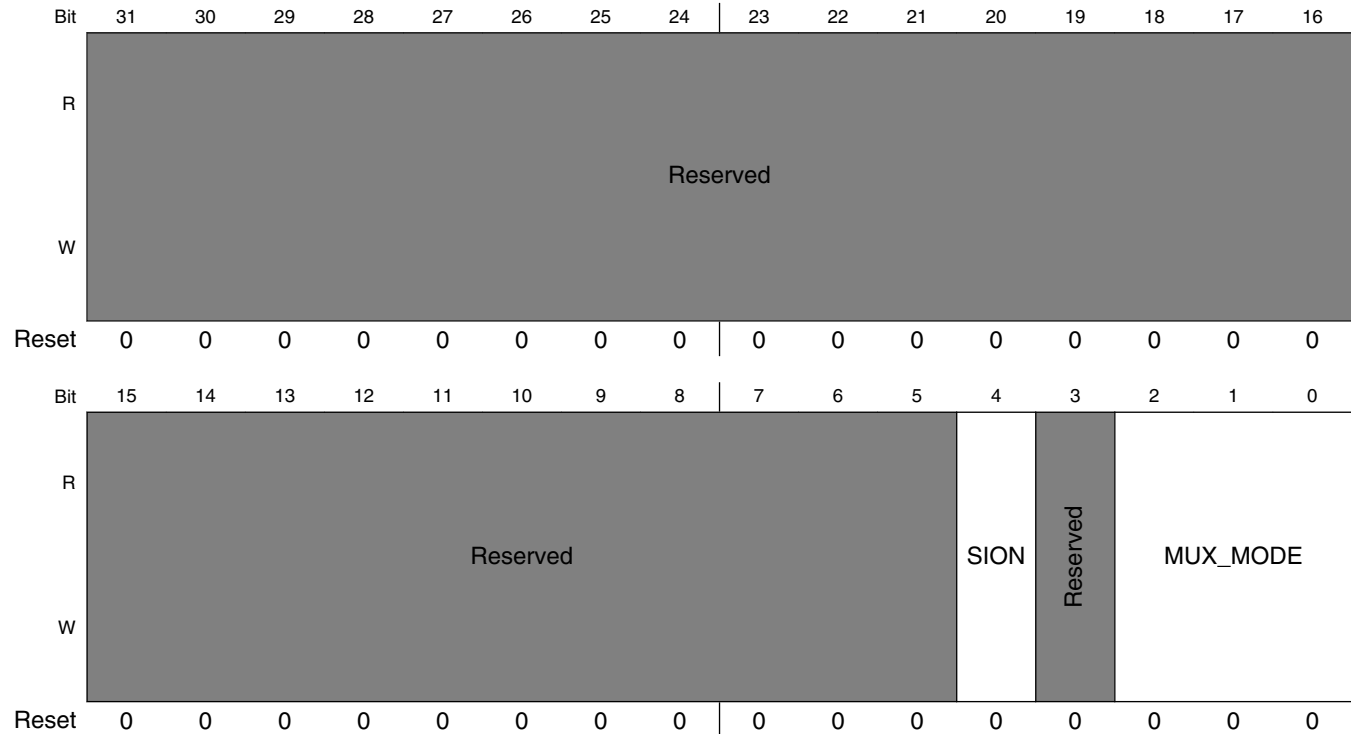
IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO13 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO13 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO13
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: GPIO1_IO13 000 ALT0_GPIO1_IO13 — Select mux mode: ALT0 mux port: IO13 of instance: GPIO1 001 ALT1_USB1_OTG_OC — Select mux mode: ALT1 mux port: OTG_OC of instance: USB1 101 ALT5_PWM2_OUT — Select mux mode: ALT5 mux port: OUT of instance: PWM2

8.2.5.20 SW_MUX_CTL_PAD_GPIO1_IO14 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO14)

SW_MUX_CTL Register

Address: 3033_0000h base + 60h offset = 3033_0060h



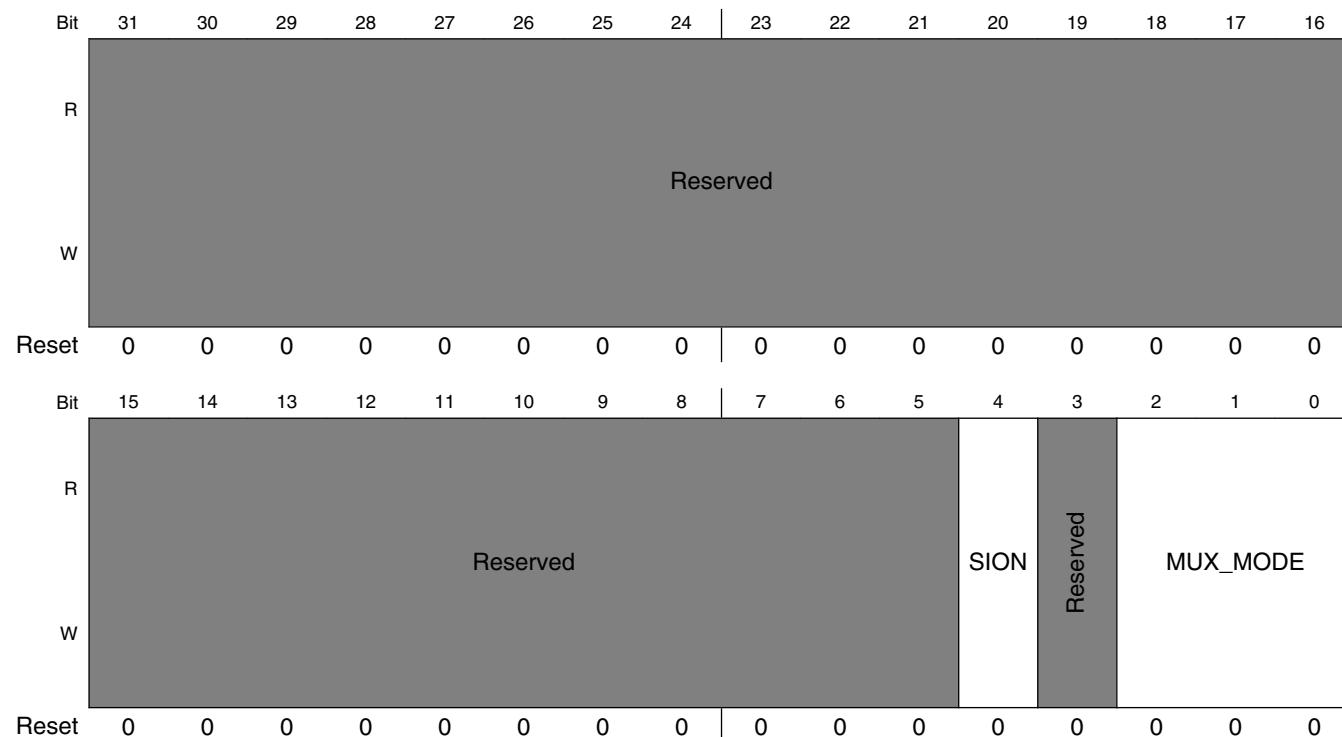
IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO14 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO14 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO14
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: GPIO1_IO14 000 ALT0_GPIO1_IO14 — Select mux mode: ALT0 mux port: IO14 of instance: GPIO1 001 ALT1_USB2_OTG_PWR — Select mux mode: ALT1 mux port: OTG_PWR of instance: USB2 101 ALT5_PWM3_OUT — Select mux mode: ALT5 mux port: OUT of instance: PWM3 110 ALT6_CCM_CLKO1 — Select mux mode: ALT6 mux port: CLKO1 of instance: CCM

8.2.5.21 SW_MUX_CTL_PAD_GPIO1_IO15 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO15)

SW_MUX_CTL Register

Address: 3033_0000h base + 64h offset = 3033_0064h



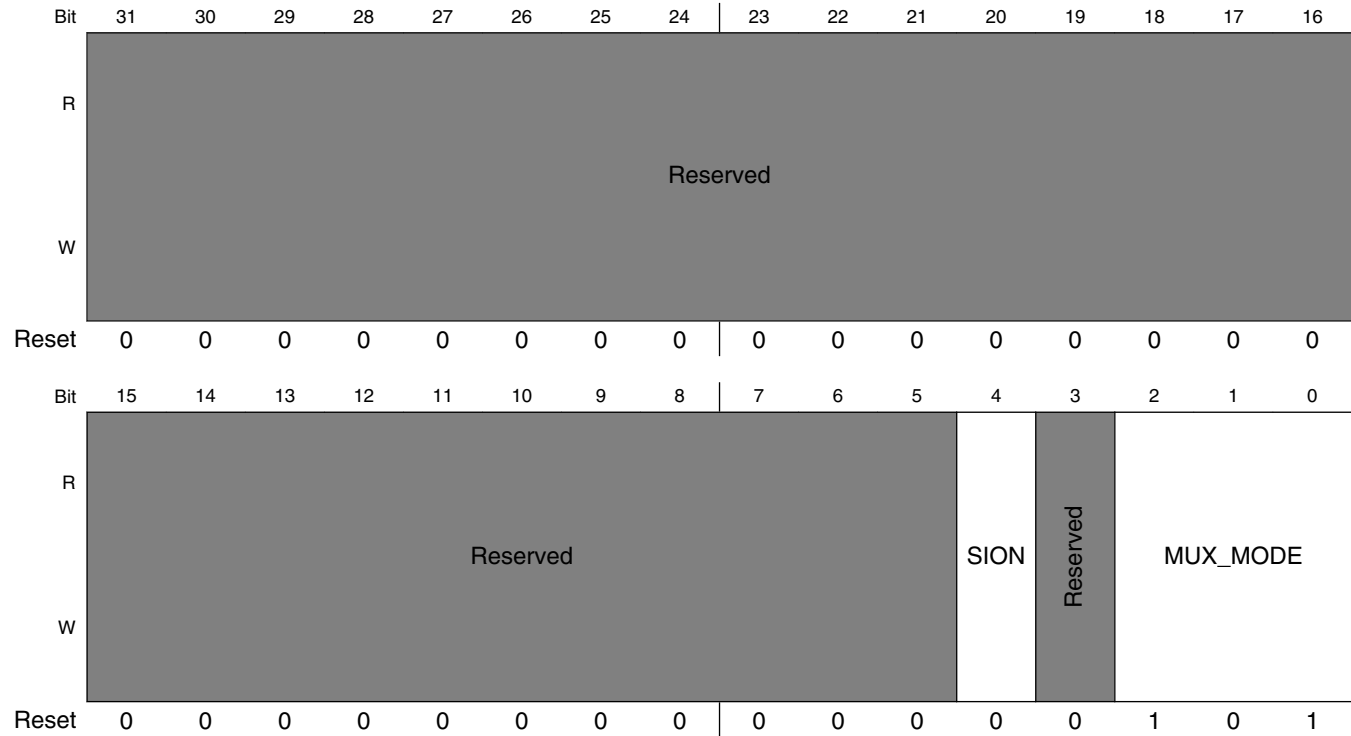
IOMUXC_SW_MUX_CTL_PAD_GPIO1_IO15 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad GPIO1_IO15 is determined by functionality 1 SION_ENABLED — Force Input Path of pad GPIO1_IO15
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: GPIO1_IO15 000 ALT0_GPIO1_IO15 — Select mux mode: ALT0 mux port: IO15 of instance: GPIO1 001 ALT1_USB2_OTG_OC — Select mux mode: ALT1 mux port: OTG_OC of instance: USB2 101 ALT5_PWM4_OUT — Select mux mode: ALT5 mux port: OUT of instance: PWM4 110 ALT6_CCM_CLKO2 — Select mux mode: ALT6 mux port: CLKO2 of instance: CCM

8.2.5.22 SW_MUX_CTL_PAD_ENET_MDC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_MDC)

SW_MUX_CTL Register

Address: 3033_0000h base + 68h offset = 3033_0068h



IOMUXC_SW_MUX_CTL_PAD_ENET_MDC field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_MDC is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_MDC
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: ENET_MDC 000 ALT0_ENET1_MDC — Select mux mode: ALT0 mux port: MDC of instance: ENET1 101 ALT5_GPIO1_IO16 — Select mux mode: ALT5 mux port: IO16 of instance: GPIO1

8.2.5.23 SW_MUX_CTL_PAD_ENET_MDIO SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_MDIO)

SW_MUX_CTL Register

Address: 3033_0000h base + 6Ch offset = 3033_006Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

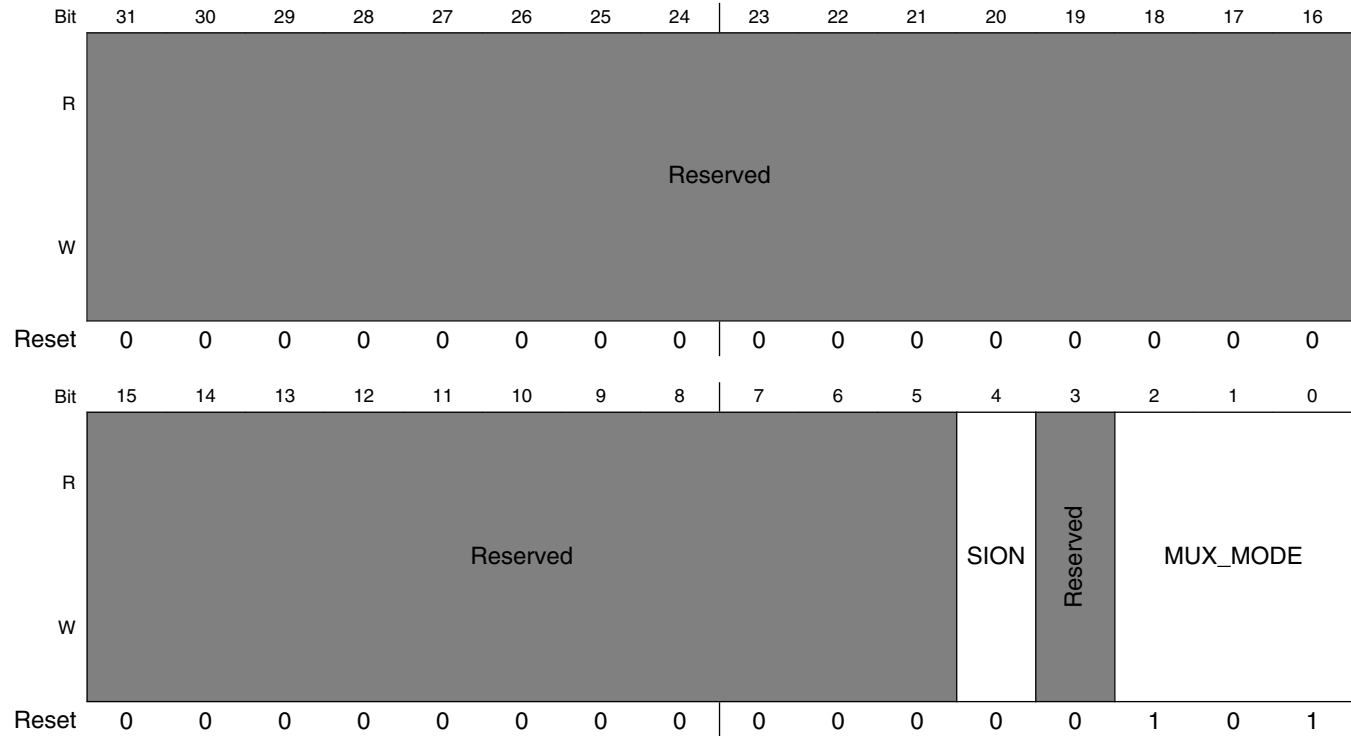
IOMUXC_SW_MUX_CTL_PAD_ENET_MDIO field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_MDIO is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_MDIO
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: ENET_MDIO 000 ALT0_ENET1_MDIO — Select mux mode: ALT0 mux port: MDIO of instance: ENET1 101 ALT5_GPIO1_IO17 — Select mux mode: ALT5 mux port: IO17 of instance: GPIO1

8.2.5.24 SW_MUX_CTL_PAD_ENET_TD3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_TD3)

SW_MUX_CTL Register

Address: 3033_0000h base + 70h offset = 3033_0070h



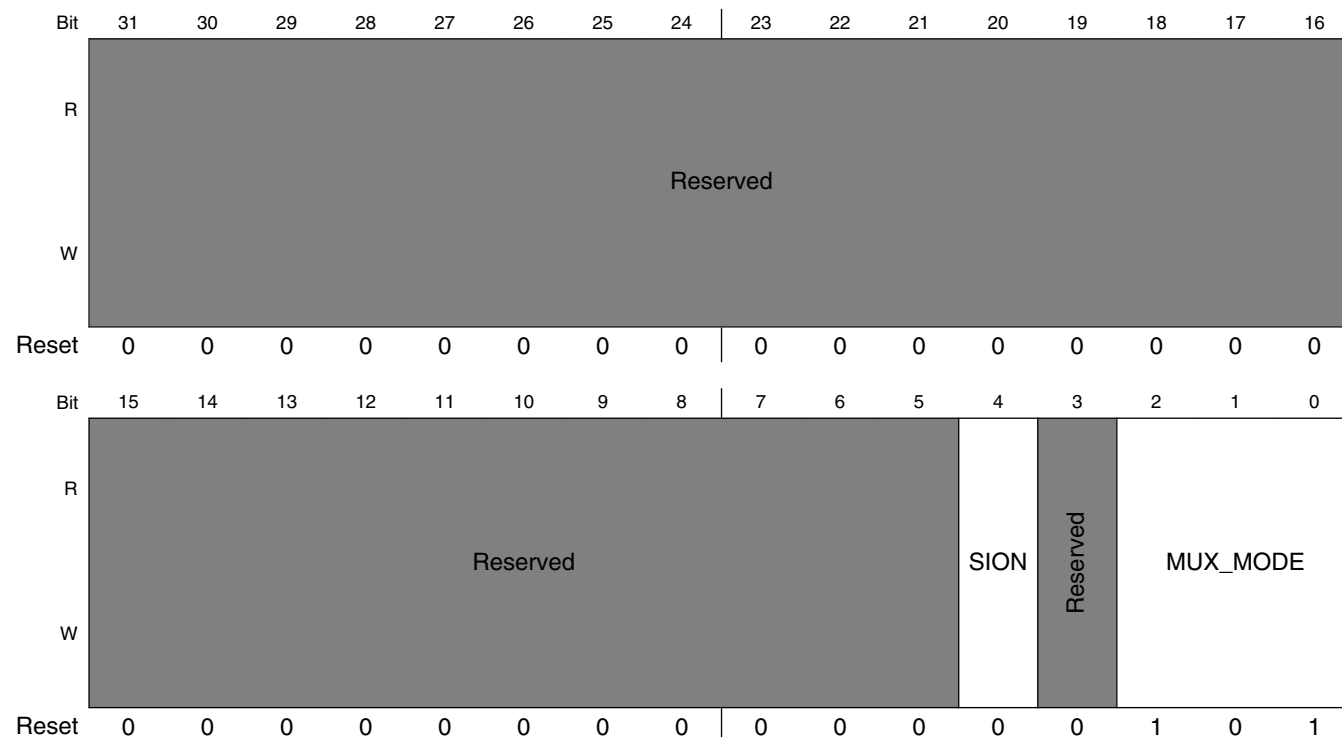
IOMUXC_SW_MUX_CTL_PAD_ENET_TD3 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_TD3 is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_TD3
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: ENET_TD3 000 ALT0_ENET1_RGMII_TD3 — Select mux mode: ALT0 mux port: RGMII_TD3 of instance: ENET1 101 ALT5_GPIO1_IO18 — Select mux mode: ALT5 mux port: IO18 of instance: GPIO1

8.2.5.25 SW_MUX_CTL_PAD_ENET_TD2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_TD2)

SW_MUX_CTL Register

Address: 3033_0000h base + 74h offset = 3033_0074h



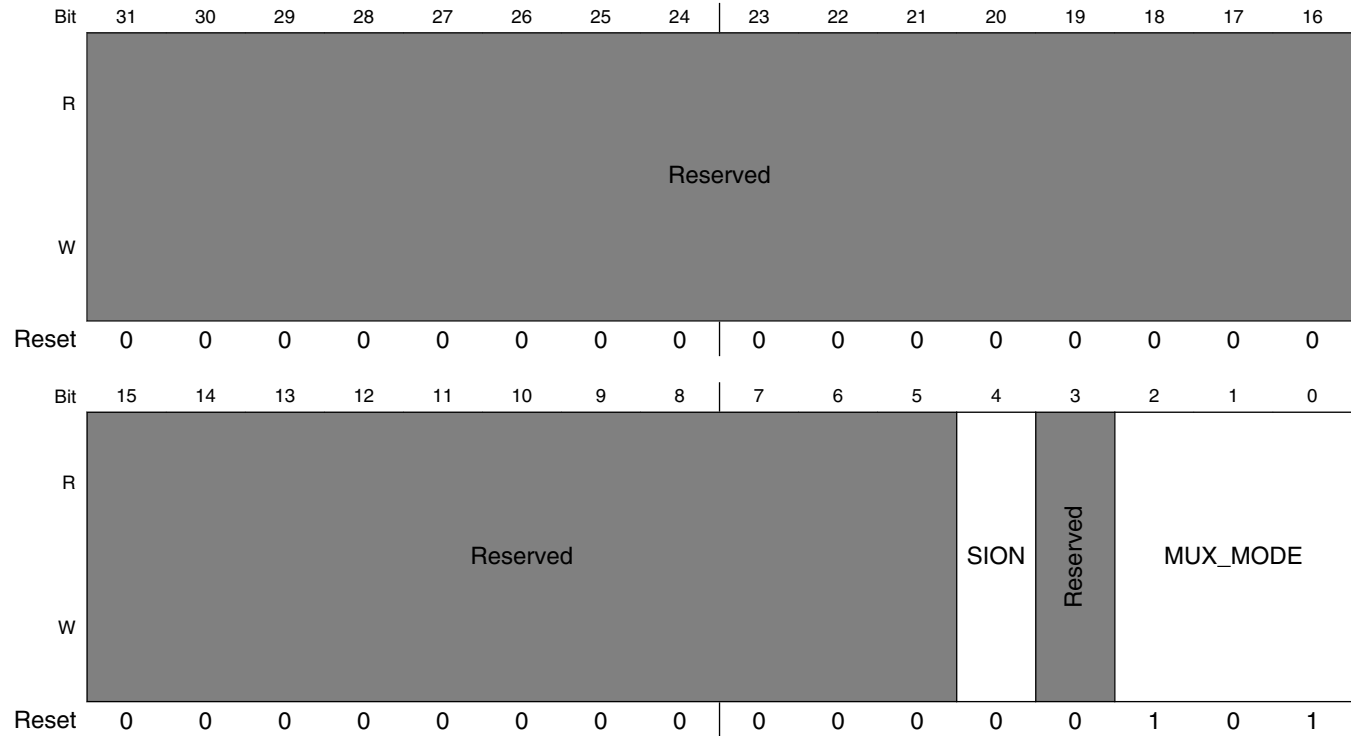
IOMUXC_SW_MUX_CTL_PAD_ENET_TD2 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_TD2 is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_TD2
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: ENET_TD2 000 ALT0_ENET1_RGMII_TD2 — Select mux mode: ALT0 mux port: RGMII_TD2 of instance: ENET1 001 ALT1_ENET1_TX_CLK — Select mux mode: ALT1 mux port: TX_CLK of instance: ENET1 (input), mux port: ENET_REF_CLK_ROOT of CCM (output) 101 ALT5_GPIO1_IO19 — Select mux mode: ALT5 mux port: IO19 of instance: GPIO1

8.2.5.26 SW_MUX_CTL_PAD_ENET_TD1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_TD1)

SW_MUX_CTL Register

Address: 3033_0000h base + 78h offset = 3033_0078h



IOMUXC_SW_MUX_CTL_PAD_ENET_TD1 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_TD1 is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_TD1
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: ENET_TD1 000 ALT0_ENET1_RGMII_TD1 — Select mux mode: ALT0 mux port: RGMII_TD1 of instance: ENET1 101 ALT5_GPIO1_IO20 — Select mux mode: ALT5 mux port: IO20 of instance: GPIO1

8.2.5.27 SW_MUX_CTL_PAD_ENET_TD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_TD0)

SW_MUX_CTL Register

Address: 3033_0000h base + 7Ch offset = 3033_007Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

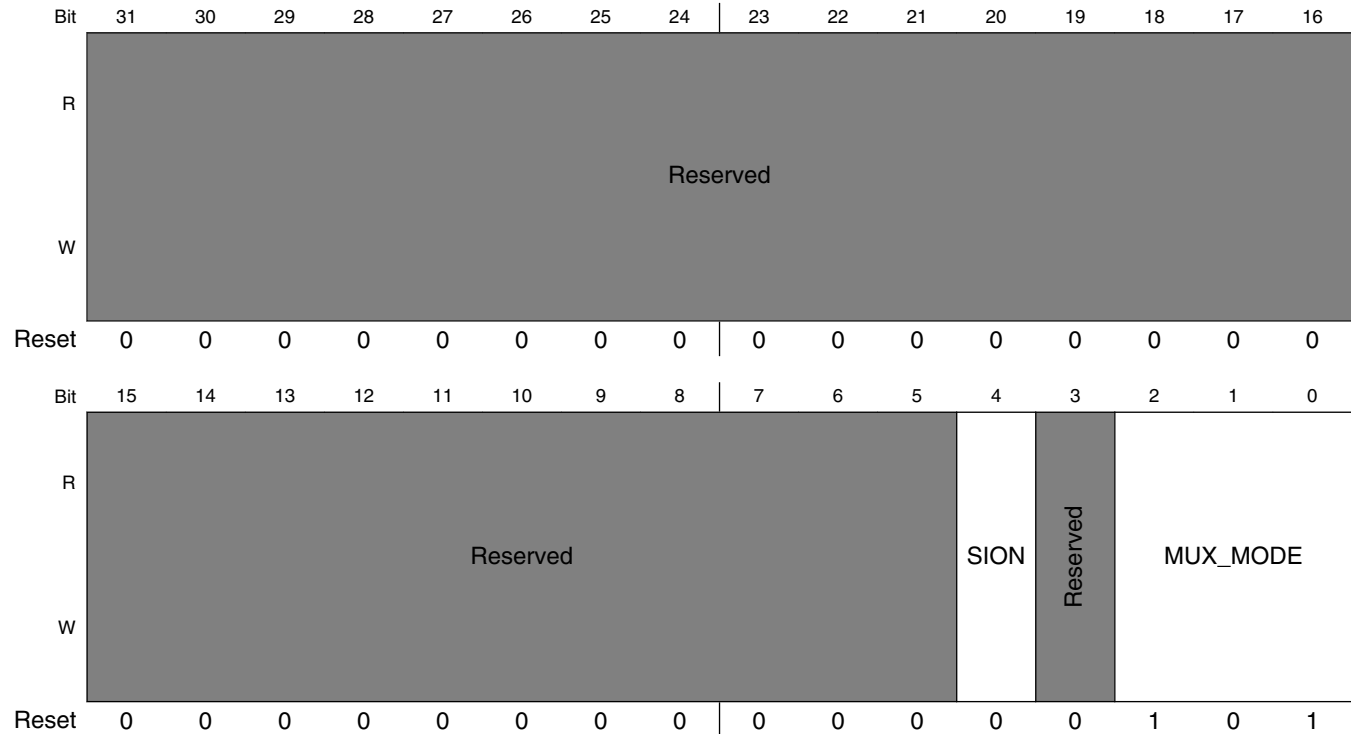
IOMUXC_SW_MUX_CTL_PAD_ENET_TD0 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_TD0 is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_TD0
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: ENET_TD0 000 ALT0_ENET1_RGMII_TD0 — Select mux mode: ALT0 mux port: RGMII_TD0 of instance: ENET1 101 ALT5_GPIO1_IO21 — Select mux mode: ALT5 mux port: IO21 of instance: GPIO1

8.2.5.28 SW_MUX_CTL_PAD_ENET_TX_CTL SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_TX_CTL)

SW_MUX_CTL Register

Address: 3033_0000h base + 80h offset = 3033_0080h



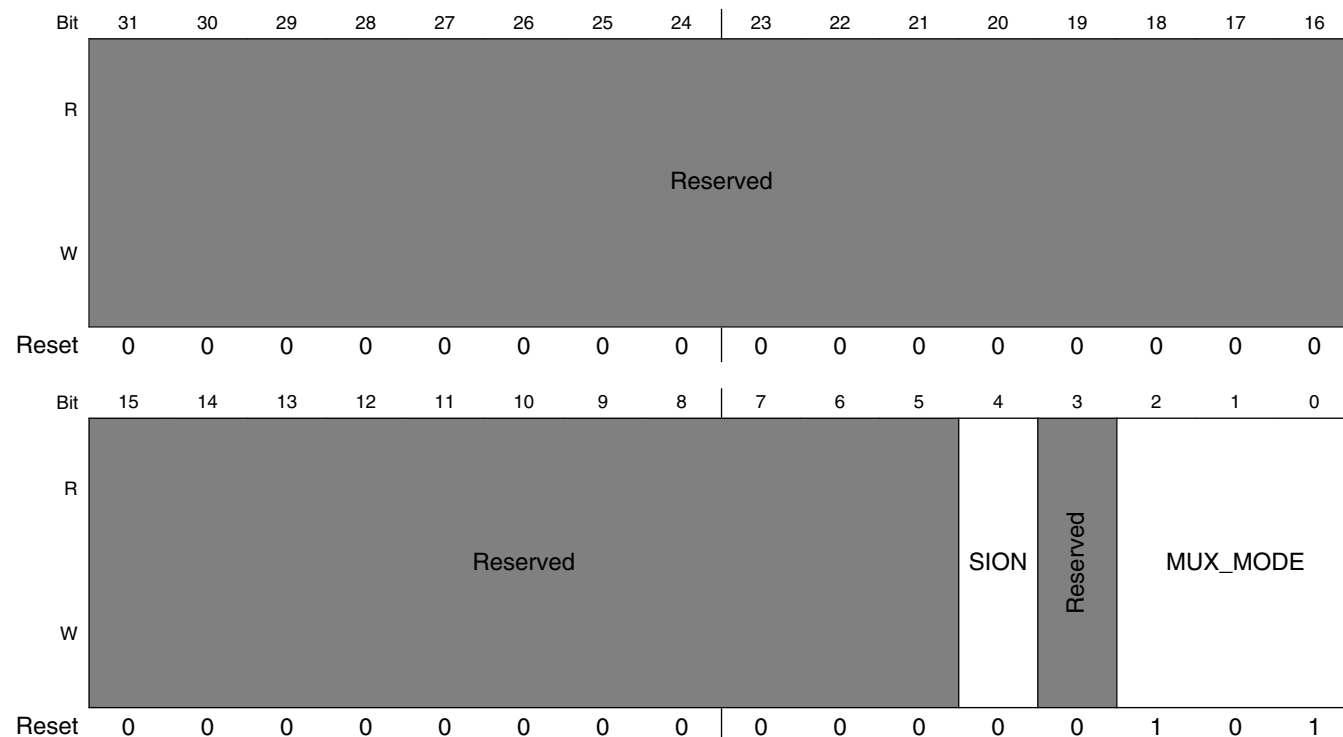
IOMUXC_SW_MUX_CTL_PAD_ENET_TX_CTL field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_TX_CTL is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_TX_CTL
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: ENET_TX_CTL 000 ALT0_ENET1_RGMII_TX_CTL — Select mux mode: ALT0 mux port: RGMII_TX_CTL of instance: ENET1 101 ALT5_GPIO1_IO22 — Select mux mode: ALT5 mux port: IO22 of instance: GPIO1

8.2.5.29 SW_MUX_CTL_PAD_ENET_TXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_TXC)

SW_MUX_CTL Register

Address: 3033_0000h base + 84h offset = 3033_0084h



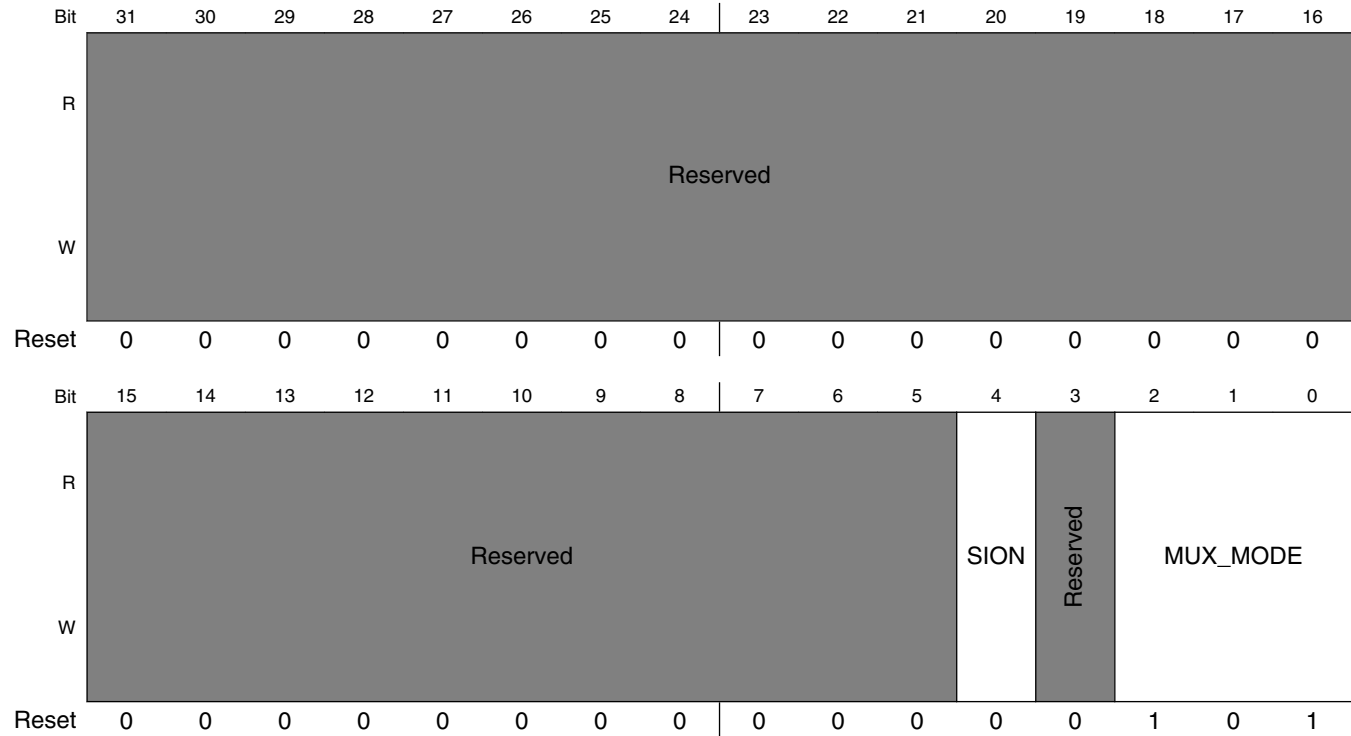
IOMUXC_SW_MUX_CTL_PAD_ENET_TXC field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_TXC is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_TXC
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: ENET_TXC 000 ALT0_ENET1_RGMII_TXC — Select mux mode: ALT0 mux port: RGMII_TXC of instance: ENET1 001 ALT1_ENET1_TX_ER — Select mux mode: ALT1 mux port: TX_ER of instance: ENET1 101 ALT5_GPIO1_IO23 — Select mux mode: ALT5 mux port: IO23 of instance: GPIO1

8.2.5.30 SW_MUX_CTL_PAD_ENET_RX_CTL SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_RX_CTL)

SW_MUX_CTL Register

Address: 3033_0000h base + 88h offset = 3033_0088h



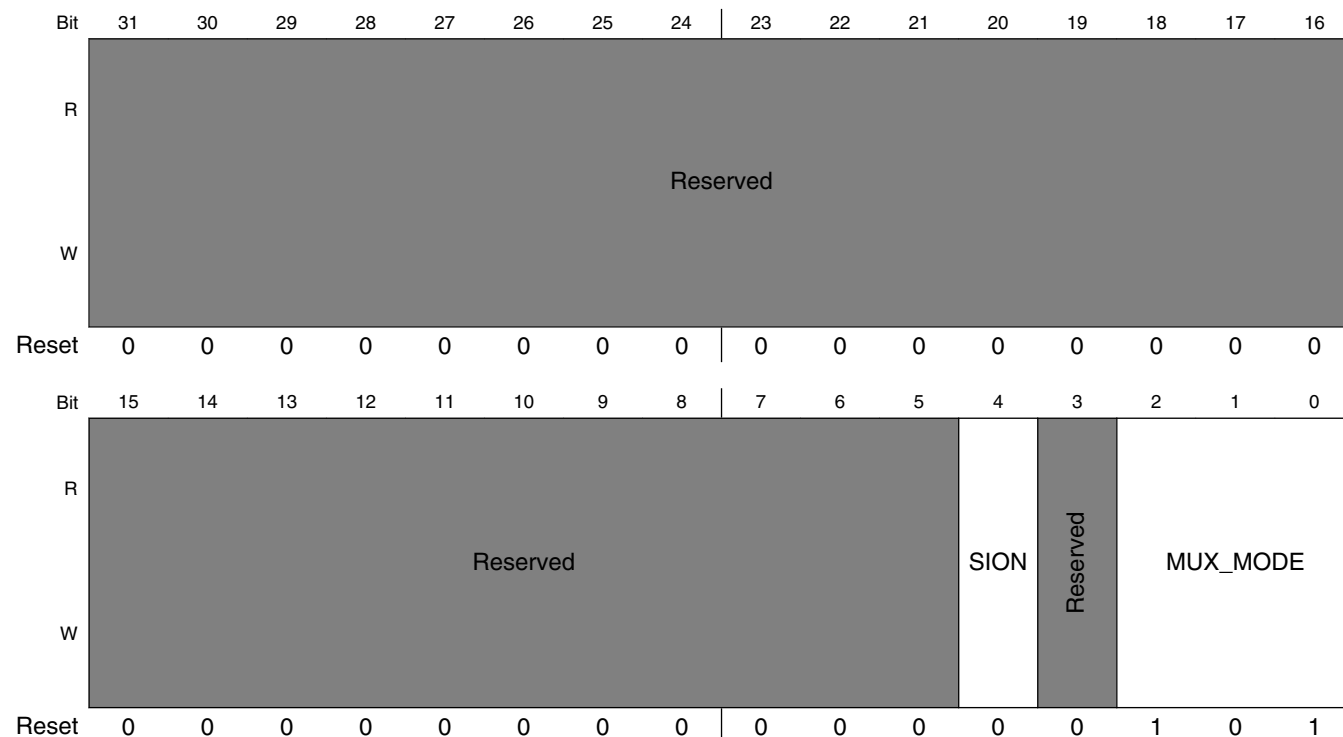
IOMUXC_SW_MUX_CTL_PAD_ENET_RX_CTL field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_RX_CTL is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_RX_CTL
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: ENET_RX_CTL 000 ALT0_ENET1_RGMII_RX_CTL — Select mux mode: ALT0 mux port: RGMII_RX_CTL of instance: ENET1 101 ALT5_GPIO1_IO24 — Select mux mode: ALT5 mux port: IO24 of instance: GPIO1

8.2.5.31 SW_MUX_CTL_PAD_ENET_RXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_RXC)

SW_MUX_CTL Register

Address: 3033_0000h base + 8Ch offset = 3033_008Ch



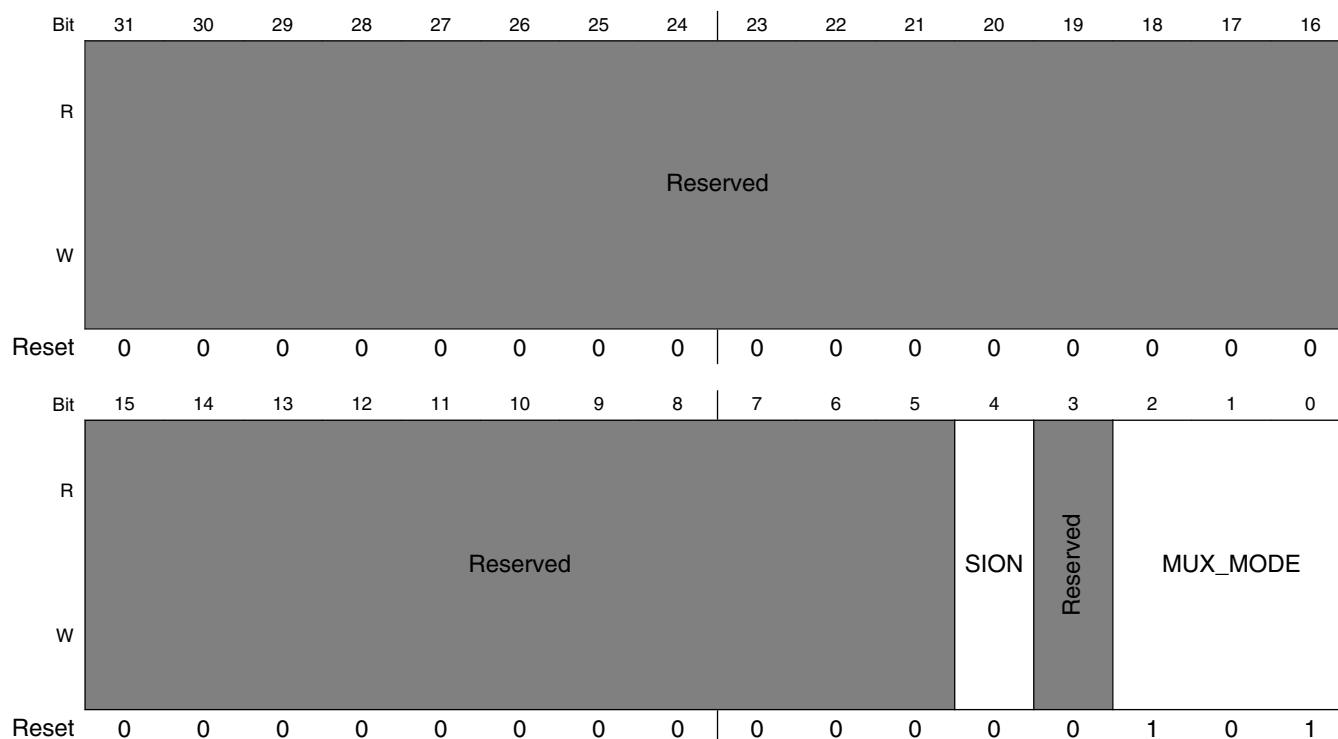
IOMUXC_SW_MUX_CTL_PAD_ENET_RXC field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_RXC is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_RXC
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: ENET_RXC 000 ALT0_ENET1_RGMII_RXC — Select mux mode: ALT0 mux port: RGMII_RXC of instance: ENET1 001 ALT1_ENET1_RX_ER — Select mux mode: ALT1 mux port: RX_ER of instance: ENET1 101 ALT5_GPIO1_IO25 — Select mux mode: ALT5 mux port: IO25 of instance: GPIO1

8.2.5.32 SW_MUX_CTL_PAD_ENET_RD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_RD0)

SW_MUX_CTL Register

Address: 3033_0000h base + 90h offset = 3033_0090h



IOMUXC_SW_MUX_CTL_PAD_ENET_RD0 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_RD0 is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_RD0
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: ENET_RD0 000 ALT0_ENET1_RGMII_RD0 — Select mux mode: ALT0 mux port: RGMII_RD0 of instance: ENET1 101 ALT5_GPIO1_IO26 — Select mux mode: ALT5 mux port: IO26 of instance: GPIO1

8.2.5.33 SW_MUX_CTL_PAD_ENET_RD1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_RD1)

SW_MUX_CTL Register

Address: 3033_0000h base + 94h offset = 3033_0094h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

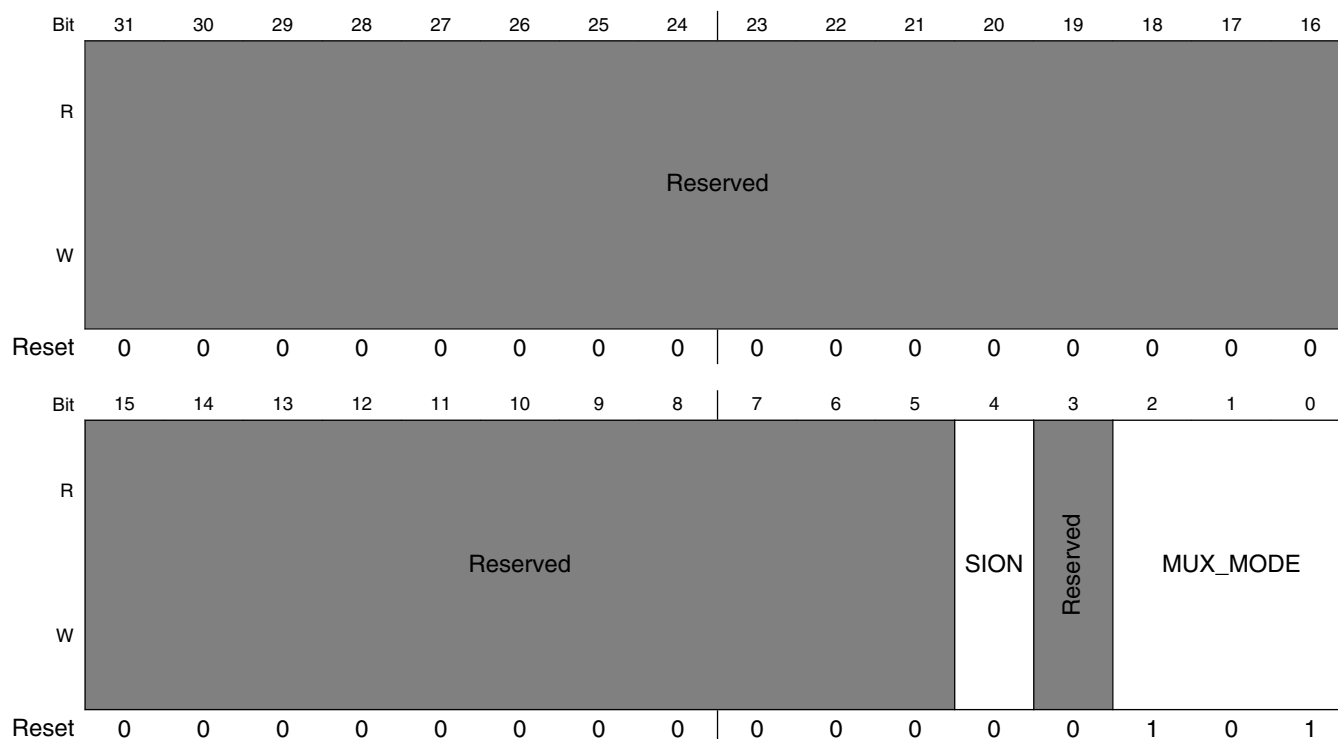
IOMUXC_SW_MUX_CTL_PAD_ENET_RD1 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_RD1 is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_RD1
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: ENET_RD1 000 ALT0_ENET1_RGMII_RD1 — Select mux mode: ALT0 mux port: RGMII_RD1 of instance: ENET1 101 ALT5_GPIO1_IO27 — Select mux mode: ALT5 mux port: IO27 of instance: GPIO1

8.2.5.34 SW_MUX_CTL_PAD_ENET_RD2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_RD2)

SW_MUX_CTL Register

Address: 3033_0000h base + 98h offset = 3033_0098h



IOMUXC_SW_MUX_CTL_PAD_ENET_RD2 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_RD2 is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_RD2
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: ENET_RD2 000 ALT0_ENET1_RGMII_RD2 — Select mux mode: ALT0 mux port: RGMII_RD2 of instance: ENET1 101 ALT5_GPIO1_IO28 — Select mux mode: ALT5 mux port: IO28 of instance: GPIO1

8.2.5.35 SW_MUX_CTL_PAD_ENET_RD3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ENET_RD3)

SW_MUX_CTL Register

Address: 3033_0000h base + 9Ch offset = 3033_009Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

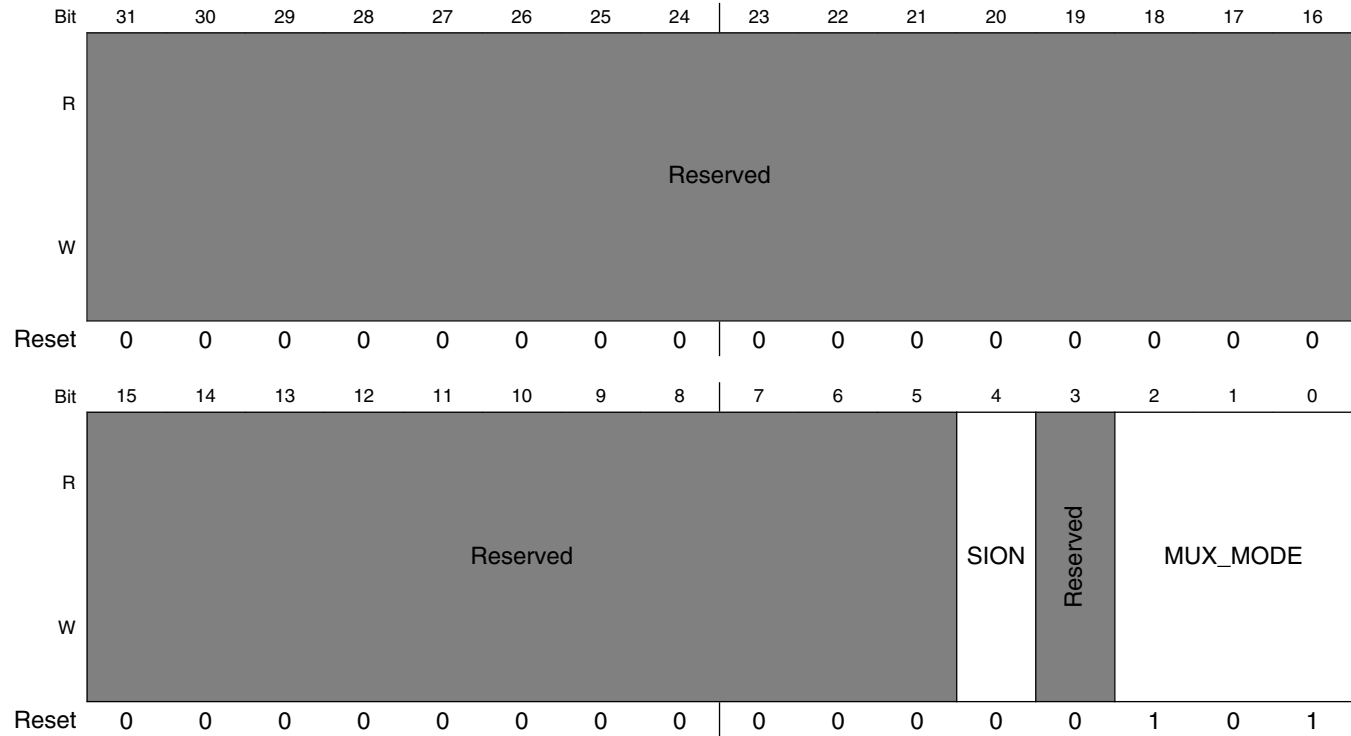
IOMUXC_SW_MUX_CTL_PAD_ENET_RD3 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ENET_RD3 is determined by functionality 1 SION_ENABLED — Force Input Path of pad ENET_RD3
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: ENET_RD3 000 ALT0_ENET1_RGMII_RD3 — Select mux mode: ALT0 mux port: RGMII_RD3 of instance: ENET1 101 ALT5_GPIO1_IO29 — Select mux mode: ALT5 mux port: IO29 of instance: GPIO1

8.2.5.36 SW_MUX_CTL_PAD_SD1_CLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_CLK)

SW_MUX_CTL Register

Address: 3033_0000h base + A0h offset = 3033_00A0h



IOMUXC_SW_MUX_CTL_PAD_SD1_CLK field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD1_CLK is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD1_CLK
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD1_CLK 000 ALT0_USDHC1_CLK — Select mux mode: ALT0 mux port: CLK of instance: USDHC1 101 ALT5_GPIO2_IO00 — Select mux mode: ALT5 mux port: IO00 of instance: GPIO2

8.2.5.37 SW_MUX_CTL_PAD_SD1_CMD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_CMD)

SW_MUX_CTL Register

Address: 3033_0000h base + A4h offset = 3033_00A4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

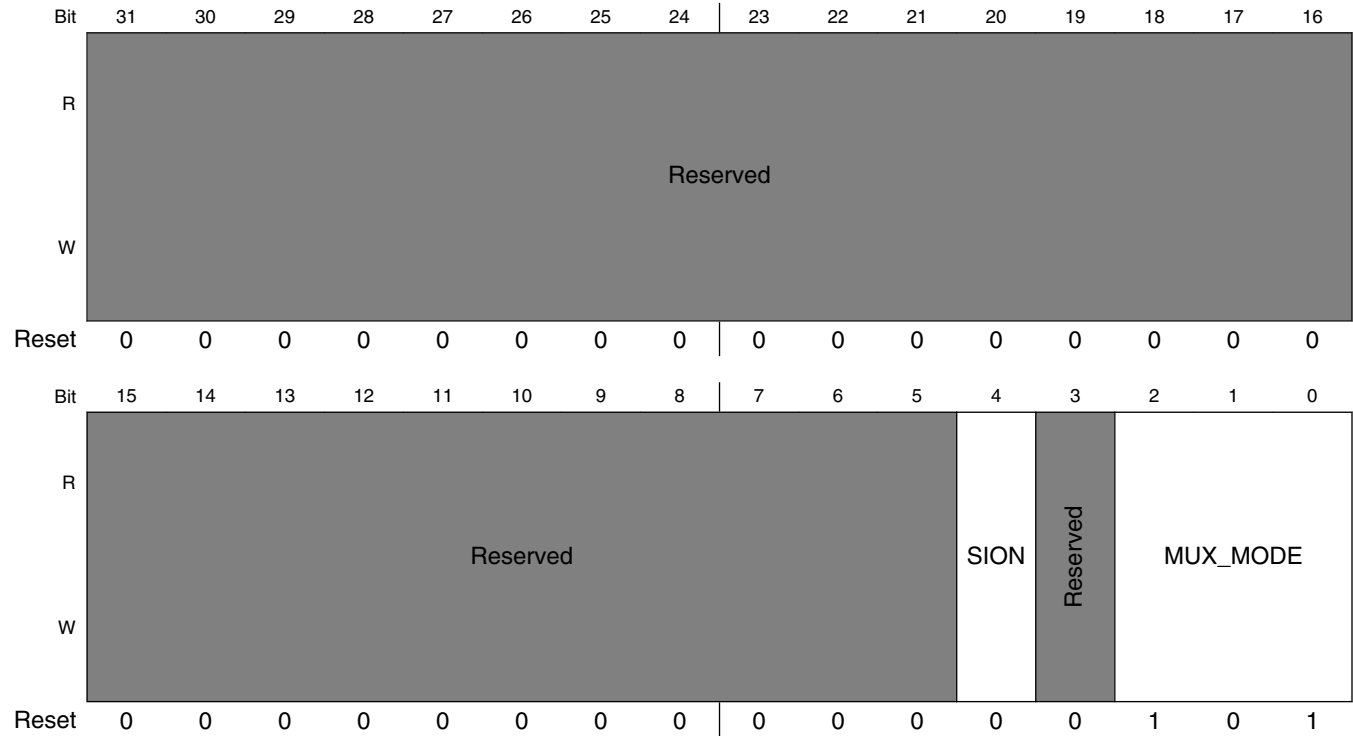
IOMUXC_SW_MUX_CTL_PAD_SD1_CMD field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD1_CMD is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD1_CMD
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD1_CMD 000 ALT0_USDHC1_CMD — Select mux mode: ALT0 mux port: CMD of instance: USDHC1 101 ALT5_GPIO2_IO01 — Select mux mode: ALT5 mux port: IO01 of instance: GPIO2

8.2.5.38 SW_MUX_CTL_PAD_SD1_DATA0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA0)

SW_MUX_CTL Register

Address: 3033_0000h base + A8h offset = 3033_00A8h



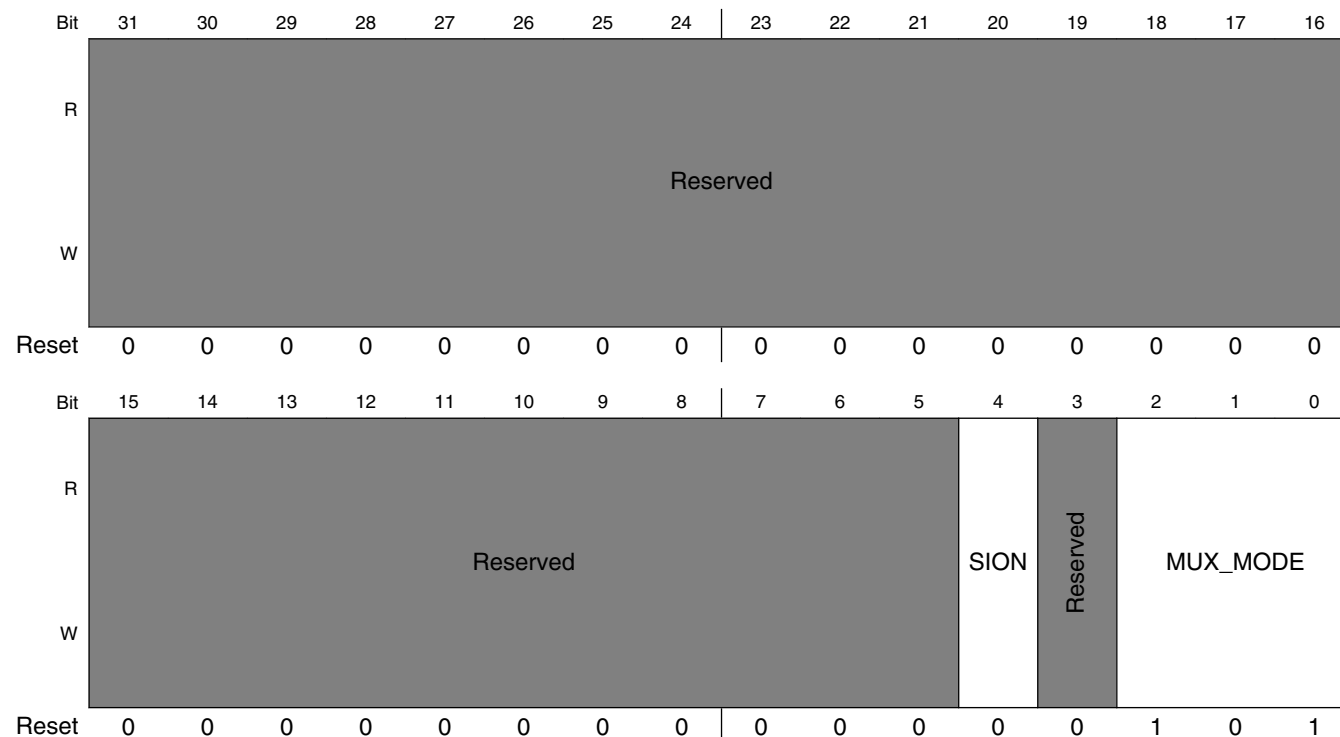
IOMUXC_SW_MUX_CTL_PAD_SD1_DATA0 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD1_DATA0 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD1_DATA0
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD1_DATA0 000 ALT0_USDHC1_DATA0 — Select mux mode: ALT0 mux port: DATA0 of instance: USDHC1 101 ALT5_GPIO2_IO02 — Select mux mode: ALT5 mux port: IO02 of instance: GPIO2

8.2.5.39 SW_MUX_CTL_PAD_SD1_DATA1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA1)

SW_MUX_CTL Register

Address: 3033_0000h base + ACh offset = 3033_00ACh



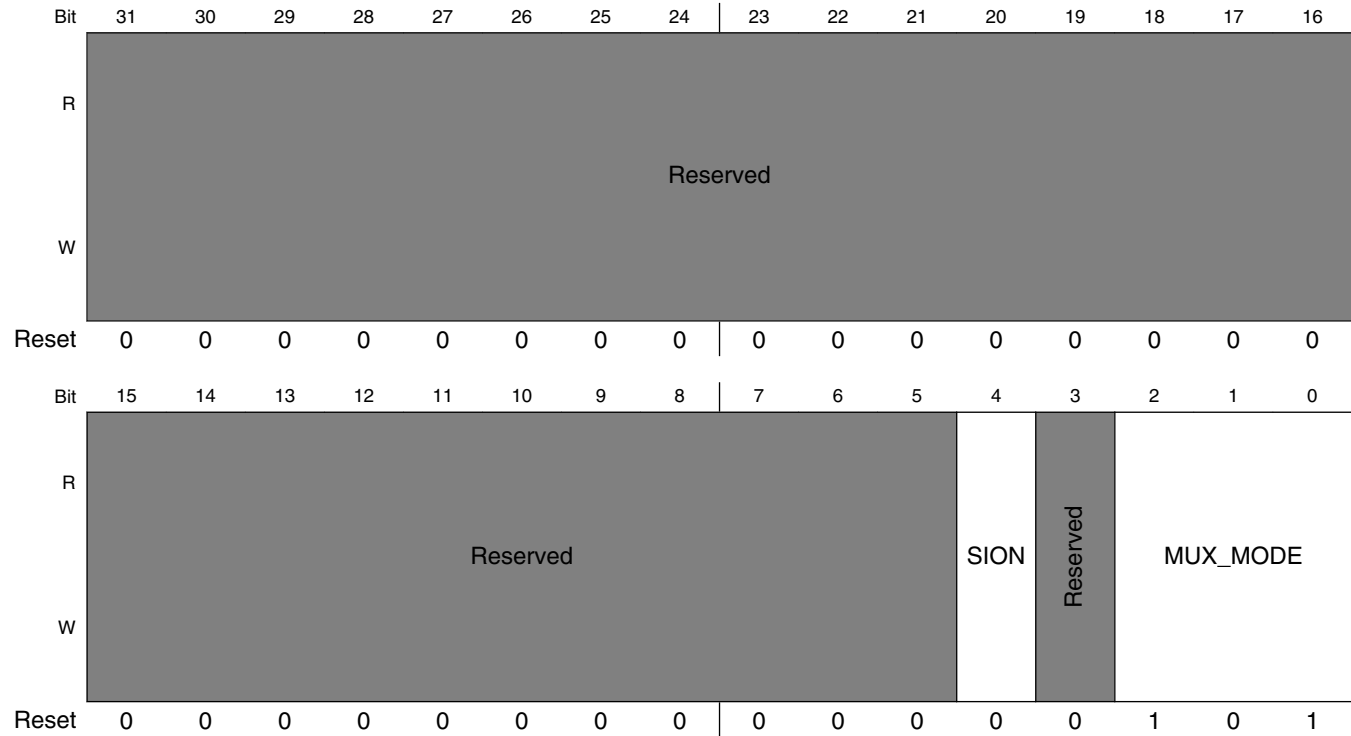
IOMUXC_SW_MUX_CTL_PAD_SD1_DATA1 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD1_DATA1 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD1_DATA1
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD1_DATA1 000 ALT0_USDHC1_DATA1 — Select mux mode: ALT0 mux port: DATA1 of instance: USDHC1 101 ALT5_GPIO2_IO03 — Select mux mode: ALT5 mux port: IO03 of instance: GPIO2

8.2.5.40 SW_MUX_CTL_PAD_SD1_DATA2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA2)

SW_MUX_CTL Register

Address: 3033_0000h base + B0h offset = 3033_00B0h



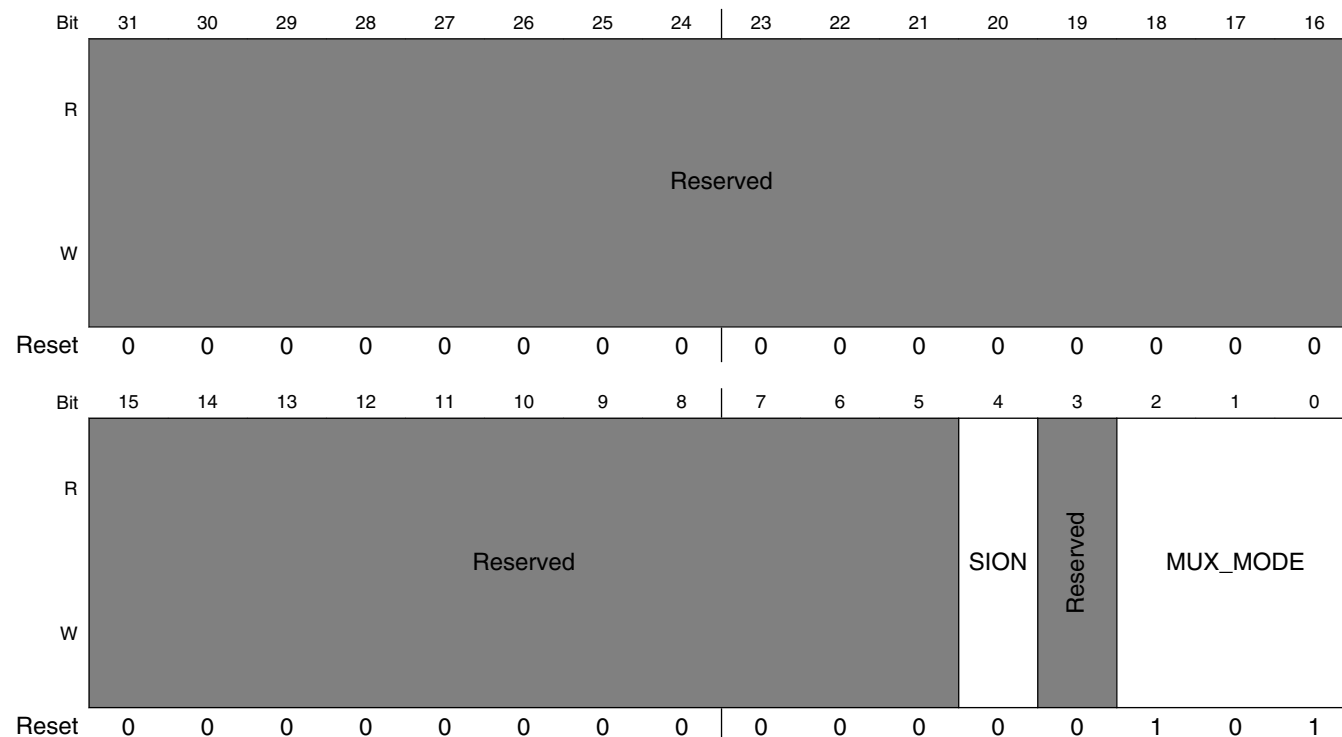
IOMUXC_SW_MUX_CTL_PAD_SD1_DATA2 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD1_DATA2 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD1_DATA2
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD1_DATA2 000 ALT0_USDHC1_DATA2 — Select mux mode: ALT0 mux port: DATA2 of instance: USDHC1 101 ALT5_GPIO2_IO04 — Select mux mode: ALT5 mux port: IO04 of instance: GPIO2

8.2.5.41 SW_MUX_CTL_PAD_SD1_DATA3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA3)

SW_MUX_CTL Register

Address: 3033_0000h base + B4h offset = 3033_00B4h



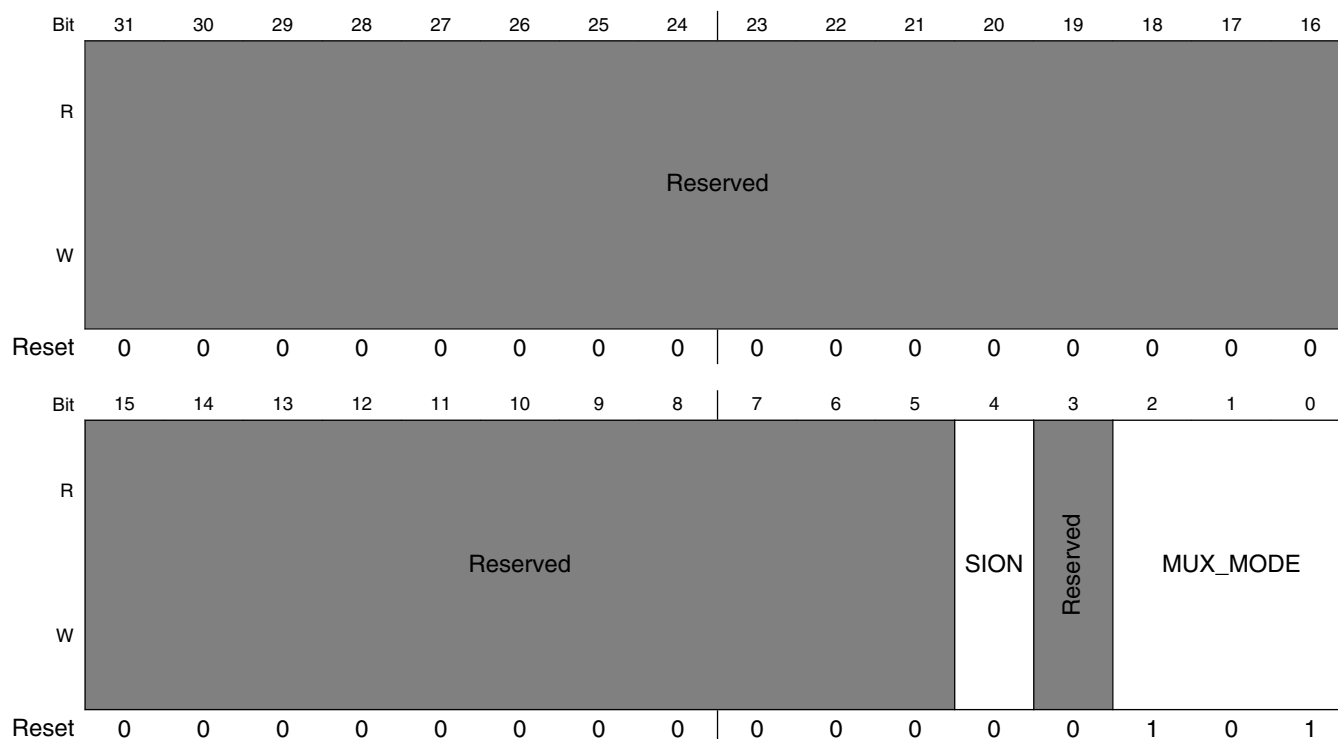
IOMUXC_SW_MUX_CTL_PAD_SD1_DATA3 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD1_DATA3 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD1_DATA3
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD1_DATA3 000 ALT0_USDHC1_DATA3 — Select mux mode: ALT0 mux port: DATA3 of instance: USDHC1 101 ALT5_GPIO2_IO05 — Select mux mode: ALT5 mux port: IO05 of instance: GPIO2

8.2.5.42 SW_MUX_CTL_PAD_SD1_DATA4 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA4)

SW_MUX_CTL Register

Address: 3033_0000h base + B8h offset = 3033_00B8h



IOMUXC_SW_MUX_CTL_PAD_SD1_DATA4 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD1_DATA4 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD1_DATA4
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD1_DATA4 000 ALT0_USDHC1_DATA4 — Select mux mode: ALT0 mux port: DATA4 of instance: USDHC1 101 ALT5_GPIO2_IO06 — Select mux mode: ALT5 mux port: IO06 of instance: GPIO2

8.2.5.43 SW_MUX_CTL_PAD_SD1_DATA5 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA5)

SW_MUX_CTL Register

Address: 3033_0000h base + BCh offset = 3033_00BCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION				Reserved		MUX_MODE	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

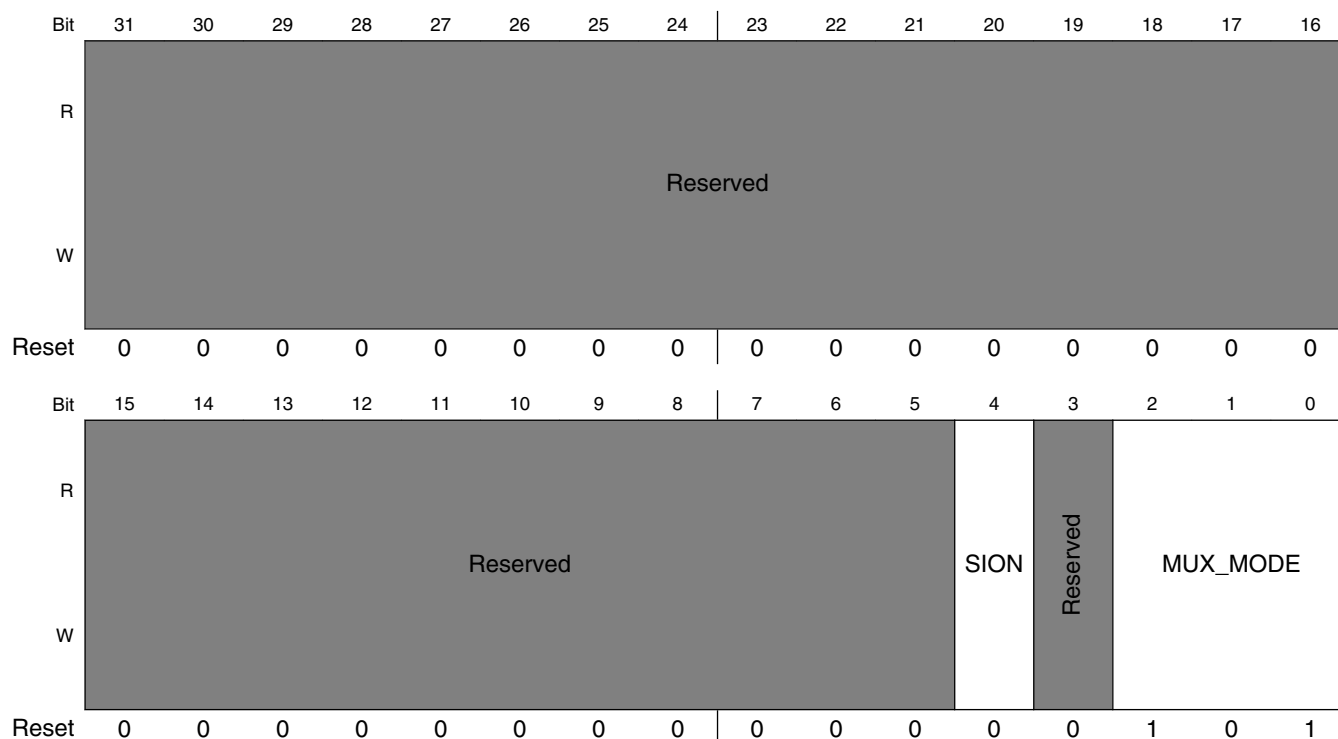
IOMUXC_SW_MUX_CTL_PAD_SD1_DATA5 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD1_DATA5 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD1_DATA5
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD1_DATA5 000 ALT0_USDHC1_DATA5 — Select mux mode: ALT0 mux port: DATA5 of instance: USDHC1 101 ALT5_GPIO2_IO07 — Select mux mode: ALT5 mux port: IO07 of instance: GPIO2

8.2.5.44 SW_MUX_CTL_PAD_SD1_DATA6 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA6)

SW_MUX_CTL Register

Address: 3033_0000h base + C0h offset = 3033_00C0h



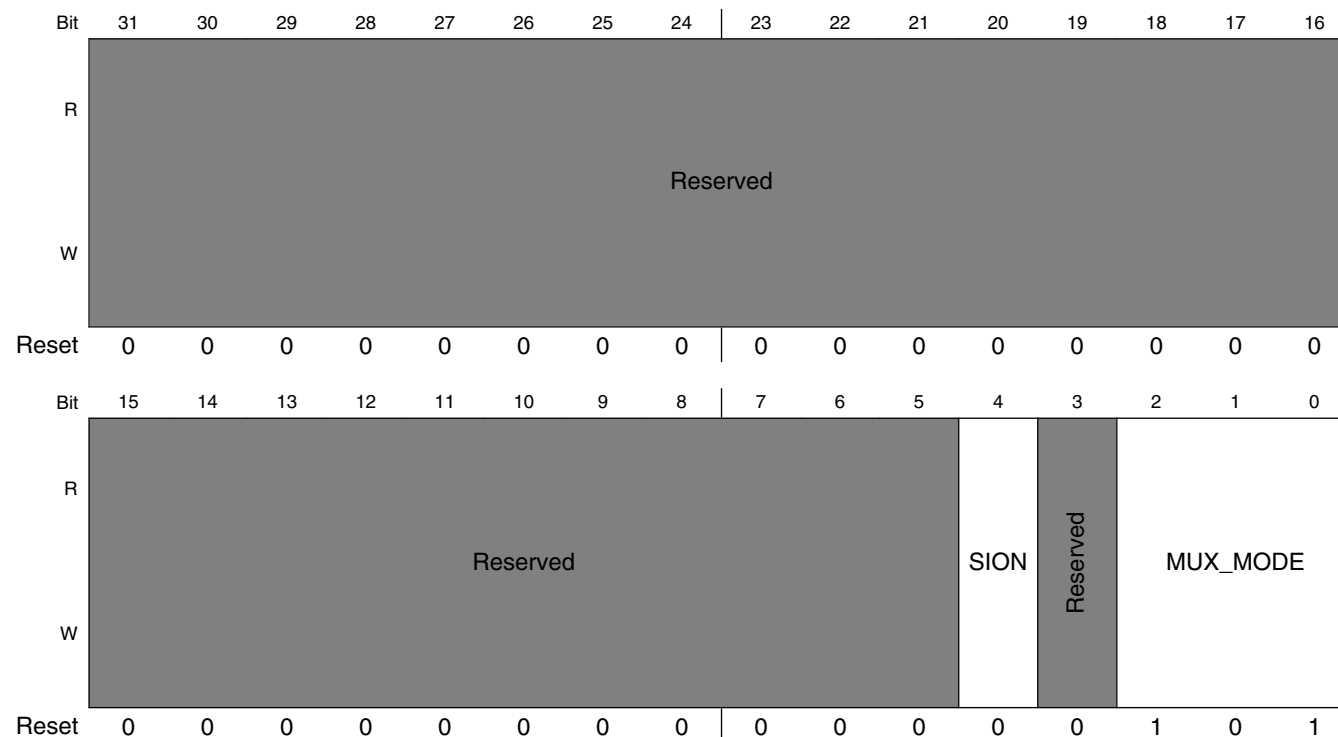
IOMUXC_SW_MUX_CTL_PAD_SD1_DATA6 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD1_DATA6 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD1_DATA6
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD1_DATA6 000 ALT0_USDHC1_DATA6 — Select mux mode: ALT0 mux port: DATA6 of instance: USDHC1 101 ALT5_GPIO2_IO08 — Select mux mode: ALT5 mux port: IO08 of instance: GPIO2

8.2.5.45 SW_MUX_CTL_PAD_SD1_DATA7 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_DATA7)

SW_MUX_CTL Register

Address: 3033_0000h base + C4h offset = 3033_00C4h



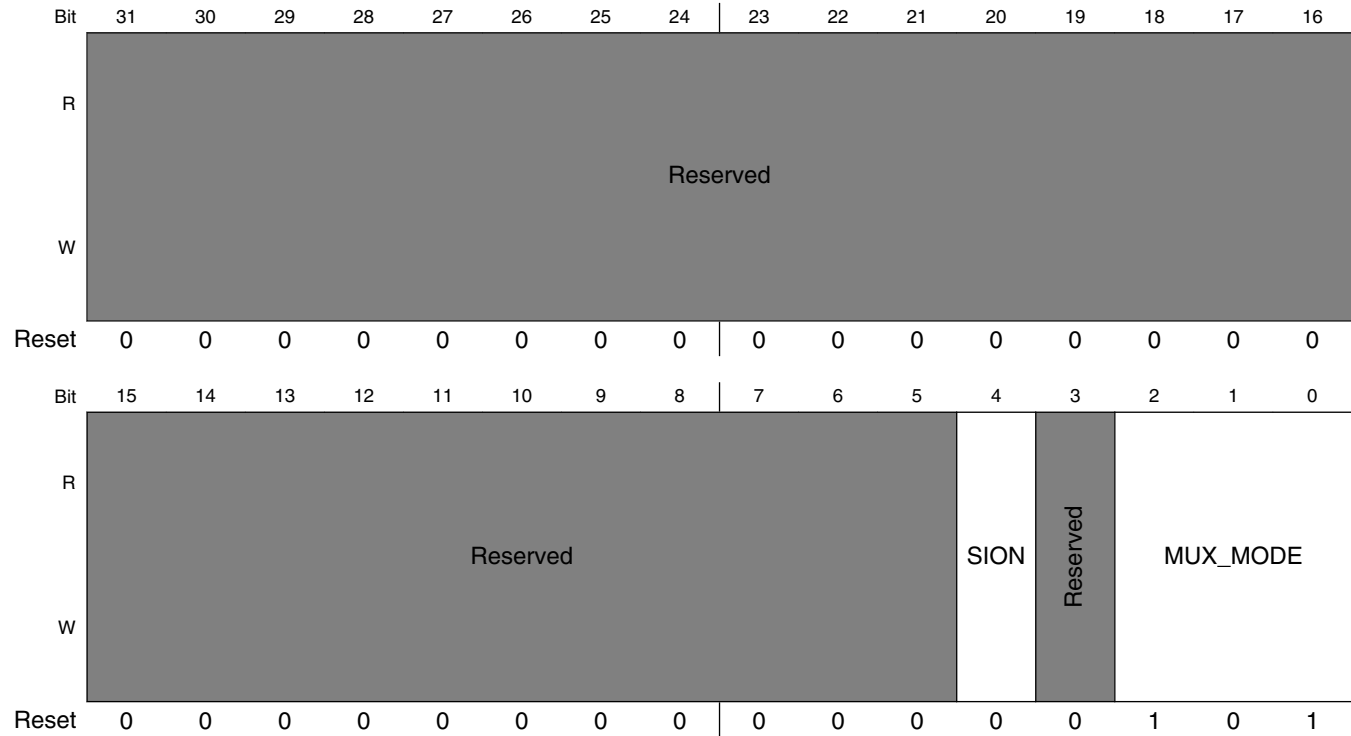
IOMUXC_SW_MUX_CTL_PAD_SD1_DATA7 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD1_DATA7 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD1_DATA7
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD1_DATA7 000 ALT0_USDHC1_DATA7 — Select mux mode: ALT0 mux port: DATA7 of instance: USDHC1 101 ALT5_GPIO2_IO09 — Select mux mode: ALT5 mux port: IO09 of instance: GPIO2

8.2.5.46 SW_MUX_CTL_PAD_SD1_RESET_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_RESET_B)

SW_MUX_CTL Register

Address: 3033_0000h base + C8h offset = 3033_00C8h



IOMUXC_SW_MUX_CTL_PAD_SD1_RESET_B field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD1_RESET_B is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD1_RESET_B
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD1_RESET_B 000 ALT0_USDHC1_RESET_B — Select mux mode: ALT0 mux port: RESET_B of instance: USDHC1 101 ALT5_GPIO2_IO10 — Select mux mode: ALT5 mux port: IO10 of instance: GPIO2

8.2.5.47 SW_MUX_CTL_PAD_SD1_STROBE SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD1_STROBE)

SW_MUX_CTL Register

Address: 3033_0000h base + CCh offset = 3033_00CCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

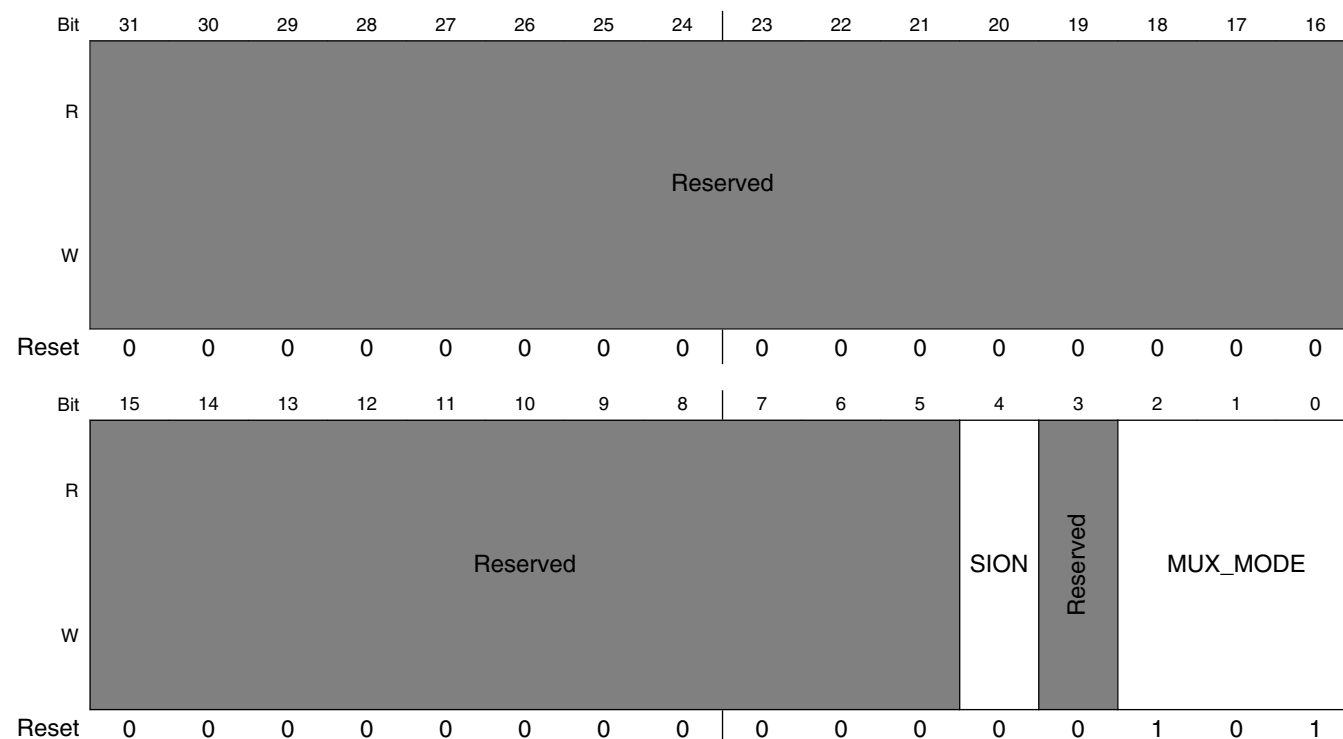
IOMUXC_SW_MUX_CTL_PAD_SD1_STROBE field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD1_STROBE is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD1_STROBE
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD1_STROBE 000 ALT0_USDHC1_STROBE — Select mux mode: ALT0 mux port: STROBE of instance: USDHC1 101 ALT5_GPIO2_IO11 — Select mux mode: ALT5 mux port: IO11 of instance: GPIO2

8.2.5.48 SW_MUX_CTL_PAD_SD2_CD_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_CD_B)

SW_MUX_CTL Register

Address: 3033_0000h base + D0h offset = 3033_00D0h



IOMUXC_SW_MUX_CTL_PAD_SD2_CD_B field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD2_CD_B is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD2_CD_B
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD2_CD_B 000 ALT0_USDHC2_CD_B — Select mux mode: ALT0 mux port: CD_B of instance: USDHC2 101 ALT5_GPIO2_IO12 — Select mux mode: ALT5 mux port: IO12 of instance: GPIO2

8.2.5.49 SW_MUX_CTL_PAD_SD2_CLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_CLK)

SW_MUX_CTL Register

Address: 3033_0000h base + D4h offset = 3033_00D4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION				Reserved	MUX_MODE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

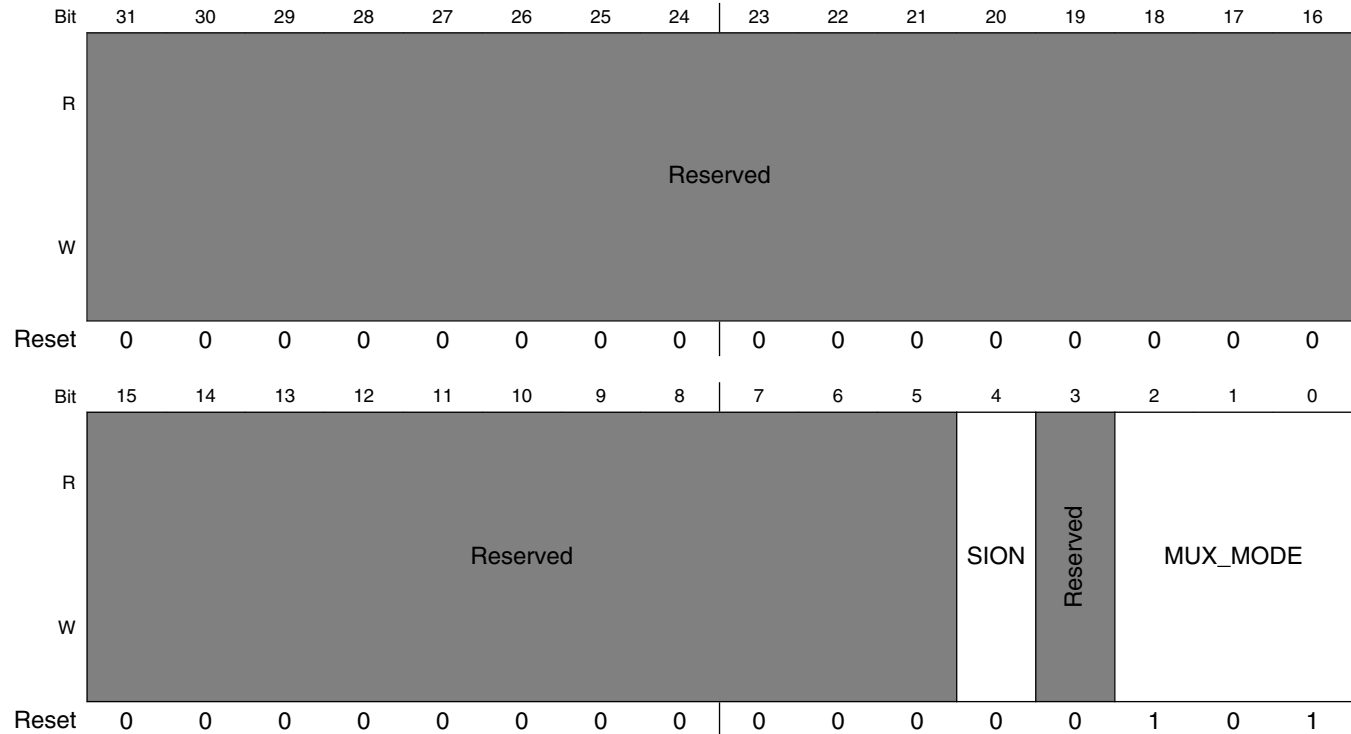
IOMUXC_SW_MUX_CTL_PAD_SD2_CLK field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD2_CLK is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD2_CLK
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD2_CLK 000 ALT0_USDHC2_CLK — Select mux mode: ALT0 mux port: CLK of instance: USDHC2 101 ALT5_GPIO2_IO13 — Select mux mode: ALT5 mux port: IO13 of instance: GPIO2

8.2.5.50 SW_MUX_CTL_PAD_SD2_CMD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_CMD)

SW_MUX_CTL Register

Address: 3033_0000h base + D8h offset = 3033_00D8h



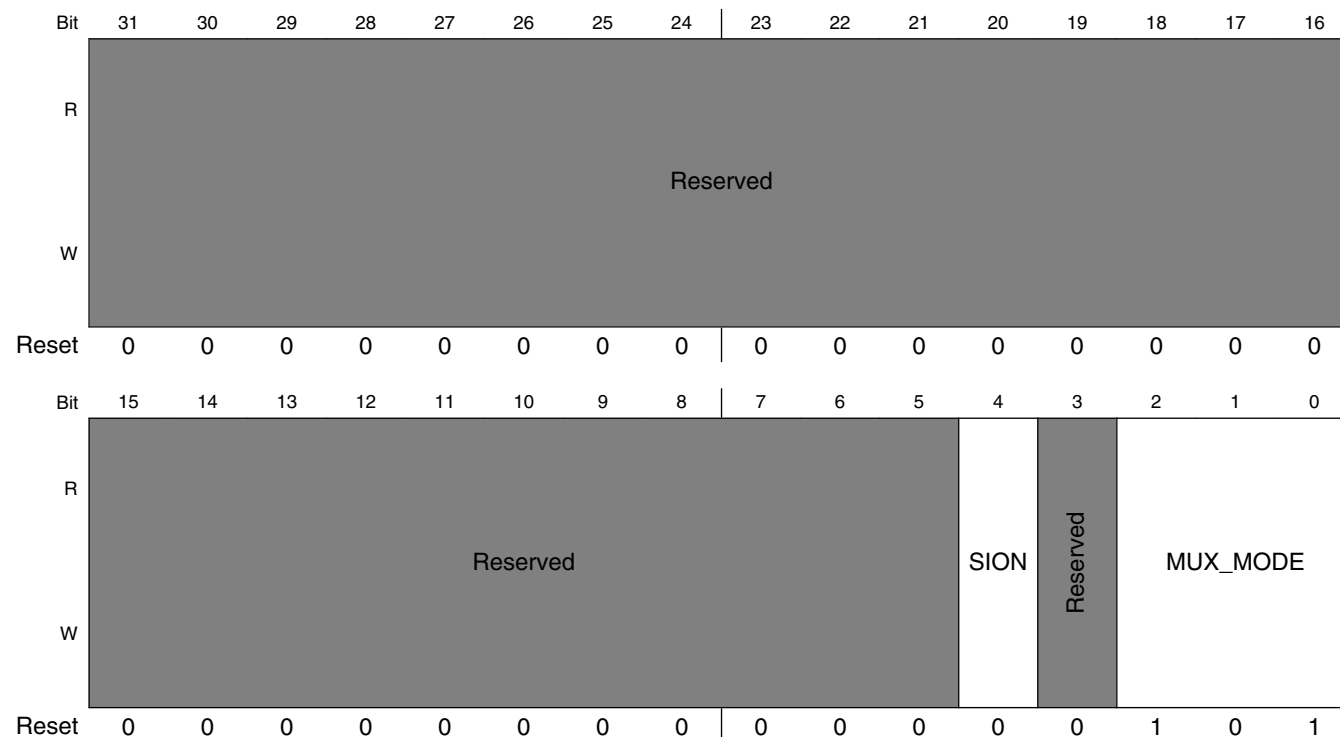
IOMUXC_SW_MUX_CTL_PAD_SD2_CMD field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD2_CMD is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD2_CMD
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD2_CMD 000 ALT0_USDHC2_CMD — Select mux mode: ALT0 mux port: CMD of instance: USDHC2 101 ALT5_GPIO2_IO14 — Select mux mode: ALT5 mux port: IO14 of instance: GPIO2

8.2.5.51 SW_MUX_CTL_PAD_SD2_DATA0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_DATA0)

SW_MUX_CTL Register

Address: 3033_0000h base + DCh offset = 3033_00DCh



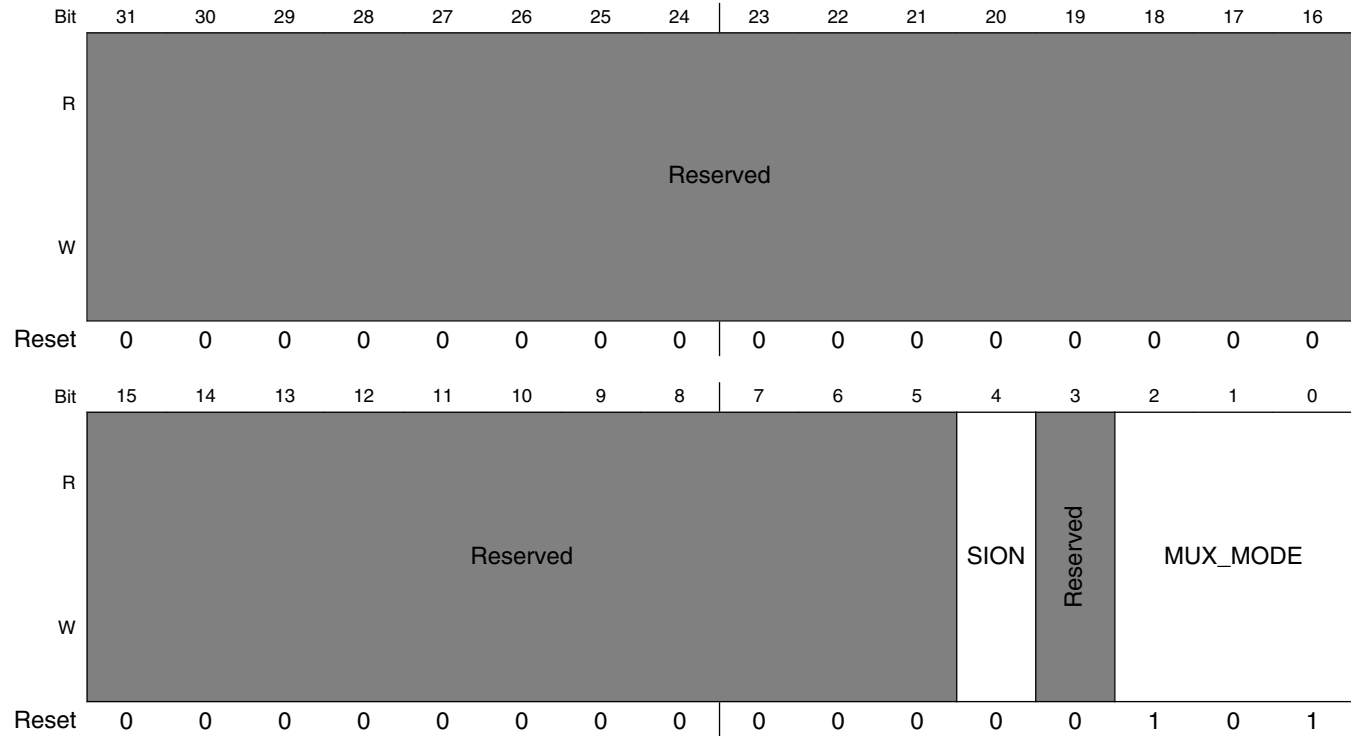
IOMUXC_SW_MUX_CTL_PAD_SD2_DATA0 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD2_DATA0 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD2_DATA0
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD2_DATA0 000 ALT0_USDHC2_DATA0 — Select mux mode: ALT0 mux port: DATA0 of instance: USDHC2 101 ALT5_GPIO2_IO15 — Select mux mode: ALT5 mux port: IO15 of instance: GPIO2

8.2.5.52 SW_MUX_CTL_PAD_SD2_DATA1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_DATA1)

SW_MUX_CTL Register

Address: 3033_0000h base + E0h offset = 3033_00E0h



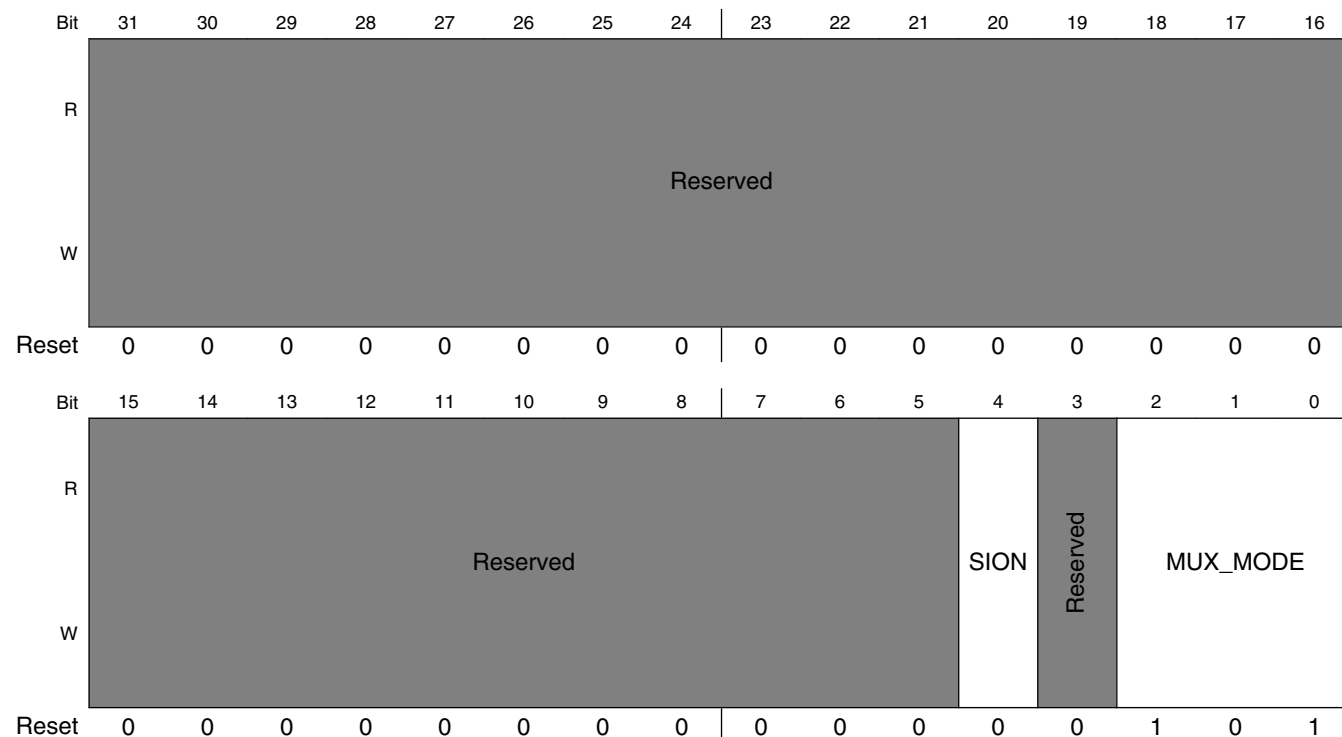
IOMUXC_SW_MUX_CTL_PAD_SD2_DATA1 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD2_DATA1 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD2_DATA1
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD2_DATA1 000 ALT0_USDHC2_DATA1 — Select mux mode: ALT0 mux port: DATA1 of instance: USDHC2 101 ALT5_GPIO2_IO16 — Select mux mode: ALT5 mux port: IO16 of instance: GPIO2

8.2.5.53 SW_MUX_CTL_PAD_SD2_DATA2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_DATA2)

SW_MUX_CTL Register

Address: 3033_0000h base + E4h offset = 3033_00E4h



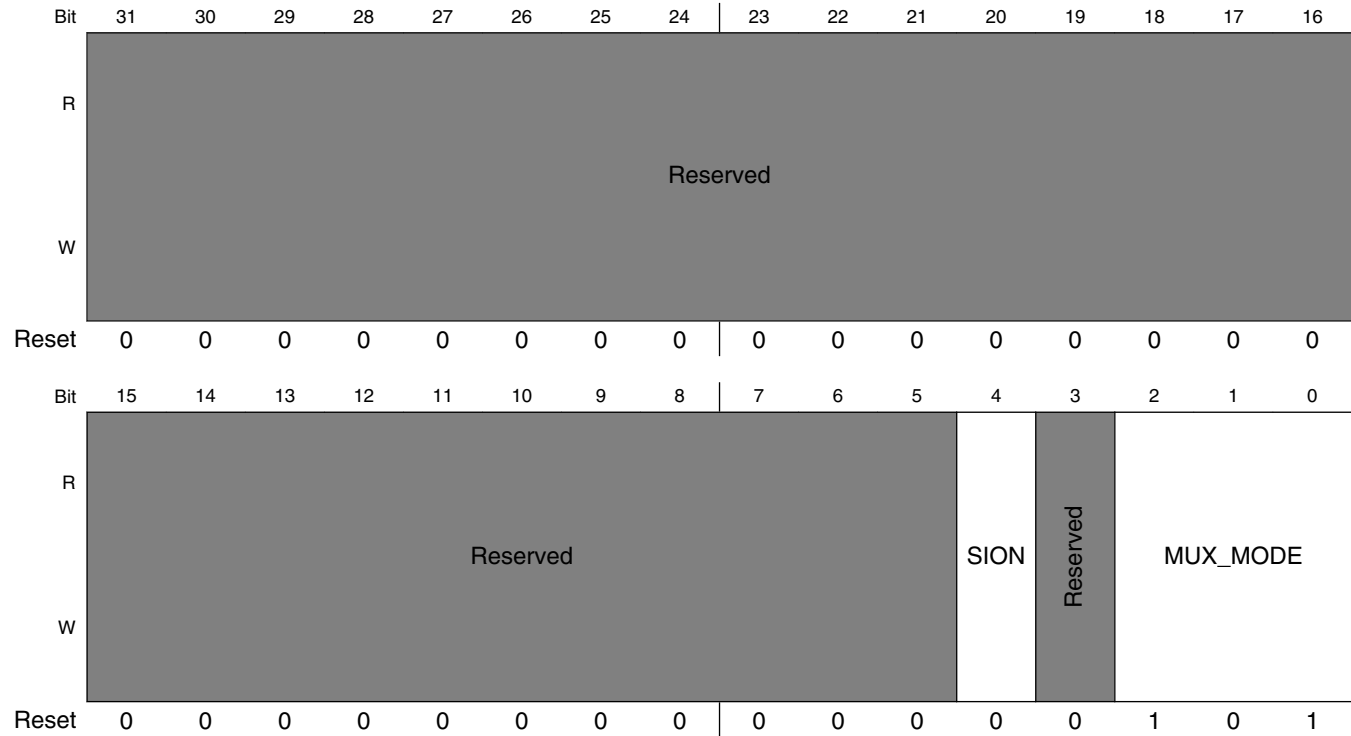
IOMUXC_SW_MUX_CTL_PAD_SD2_DATA2 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD2_DATA2 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD2_DATA2
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD2_DATA2 000 ALT0_USDHC2_DATA2 — Select mux mode: ALT0 mux port: DATA2 of instance: USDHC2 101 ALT5_GPIO2_IO17 — Select mux mode: ALT5 mux port: IO17 of instance: GPIO2

8.2.5.54 SW_MUX_CTL_PAD_SD2_DATA3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_DATA3)

SW_MUX_CTL Register

Address: 3033_0000h base + E8h offset = 3033_00E8h



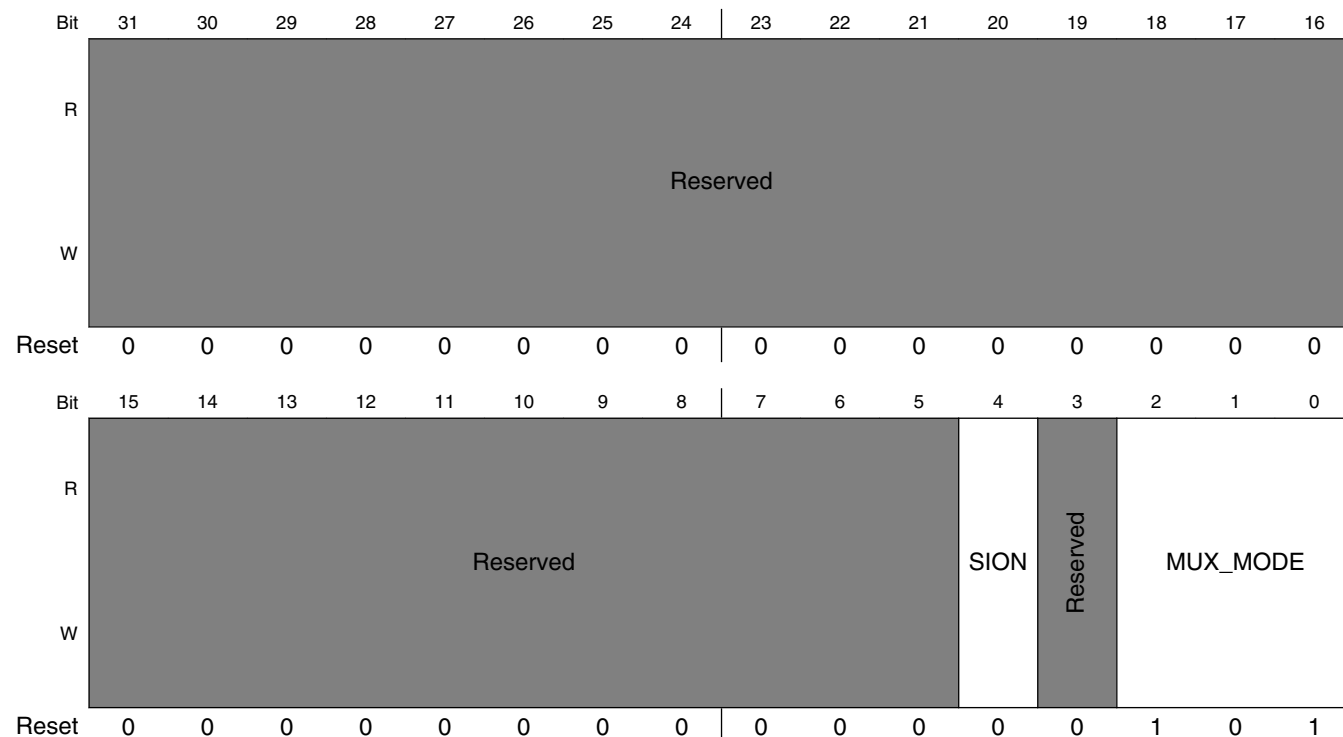
IOMUXC_SW_MUX_CTL_PAD_SD2_DATA3 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD2_DATA3 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD2_DATA3
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD2_DATA3 000 ALT0_USDHC2_DATA3 — Select mux mode: ALT0 mux port: DATA3 of instance: USDHC2 101 ALT5_GPIO2_IO18 — Select mux mode: ALT5 mux port: IO18 of instance: GPIO2

8.2.5.55 SW_MUX_CTL_PAD_SD2_RESET_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_RESET_B)

SW_MUX_CTL Register

Address: 3033_0000h base + ECh offset = 3033_00ECh



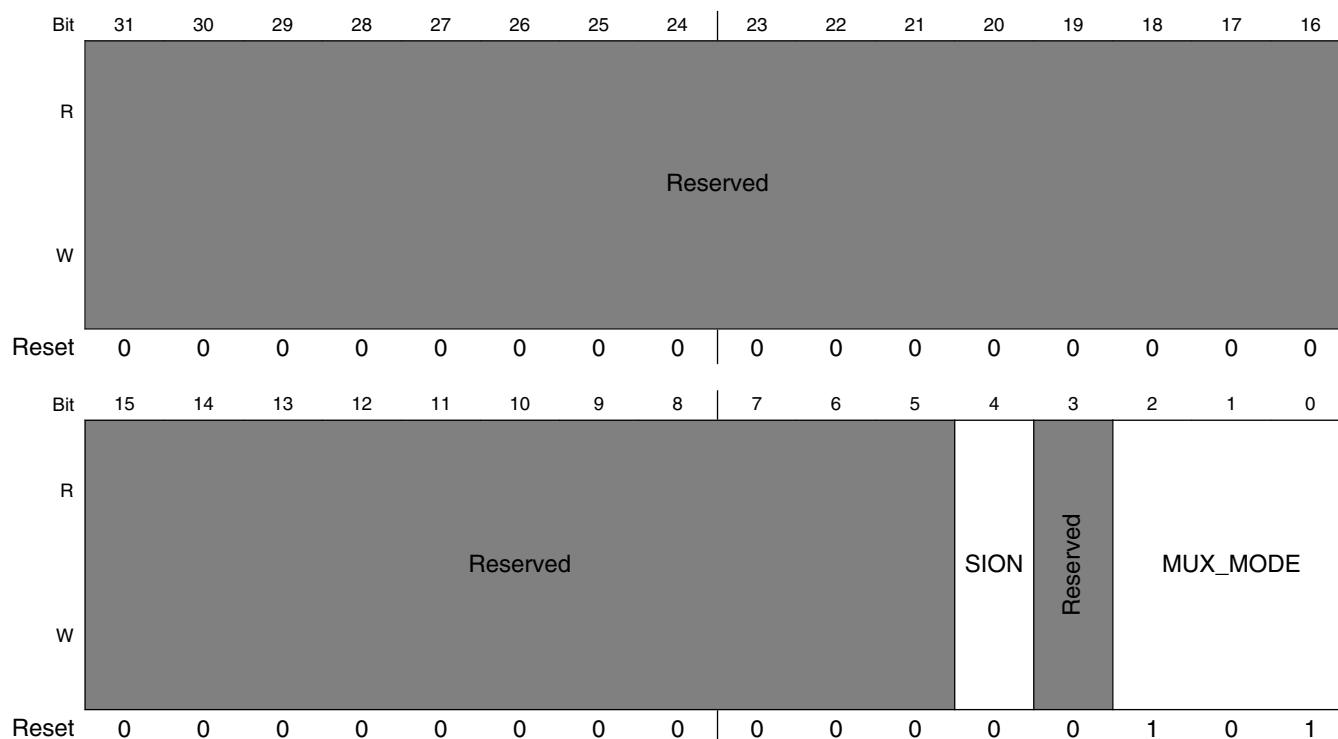
IOMUXC_SW_MUX_CTL_PAD_SD2_RESET_B field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD2_RESET_B is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD2_RESET_B
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD2_RESET_B 000 ALT0_USDHC2_RESET_B — Select mux mode: ALT0 mux port: RESET_B of instance: USDHC2 101 ALT5_GPIO2_IO19 — Select mux mode: ALT5 mux port: IO19 of instance: GPIO2

8.2.5.56 SW_MUX_CTL_PAD_SD2_WP SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SD2_WP)

SW_MUX_CTL Register

Address: 3033_0000h base + F0h offset = 3033_00F0h



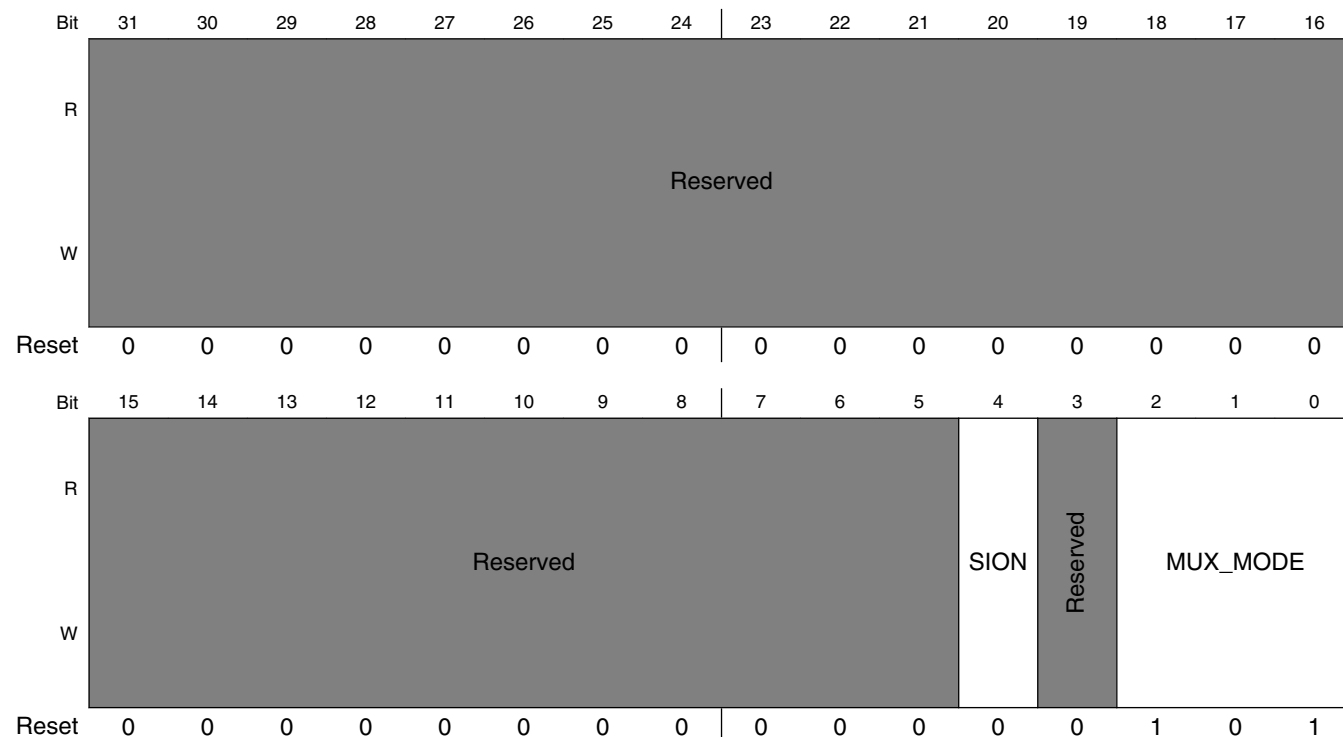
IOMUXC_SW_MUX_CTL_PAD_SD2_WP field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SD2_WP is determined by functionality 1 SION_ENABLED — Force Input Path of pad SD2_WP
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: SD2_WP 000 ALT0_USDHC2_WP — Select mux mode: ALT0 mux port: WP of instance: USDHC2 101 ALT5_GPIO2_IO20 — Select mux mode: ALT5 mux port: IO20 of instance: GPIO2

8.2.5.57 SW_MUX_CTL_PAD_NAND_ALE SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_ALE)

SW_MUX_CTL Register

Address: 3033_0000h base + F4h offset = 3033_00F4h



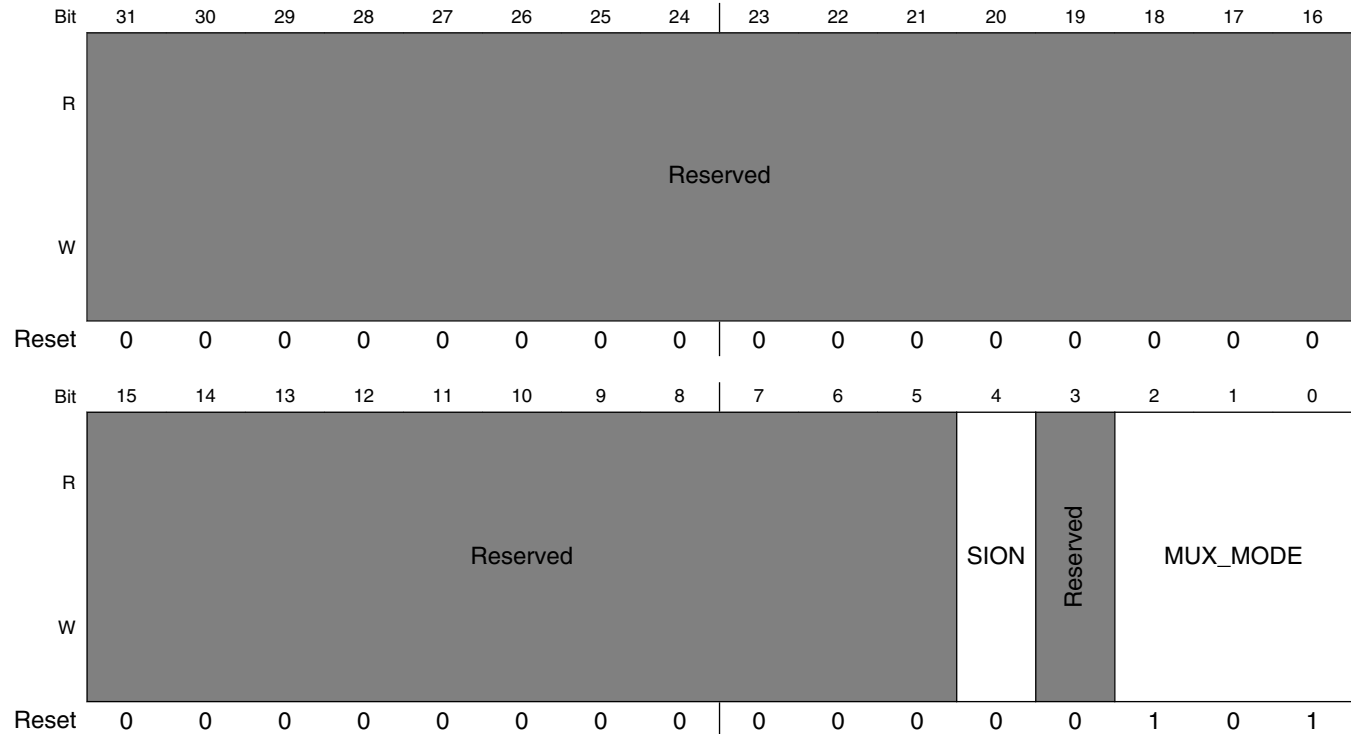
IOMUXC_SW_MUX_CTL_PAD_NAND_ALE field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_ALE is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_ALE
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_ALE 000 ALT0_RAWNAND_ALE — Select mux mode: ALT0 mux port: ALE of instance: RAWNAND 001 ALT1_QSPI_A_SCLK — Select mux mode: ALT1 mux port: A_SCLK of instance: QSPI 101 ALT5_GPIO3_IO00 — Select mux mode: ALT5 mux port: IO00 of instance: GPIO3

8.2.5.58 SW_MUX_CTL_PAD_NAND_CE0_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_CE0_B)

SW_MUX_CTL Register

Address: 3033_0000h base + F8h offset = 3033_00F8h



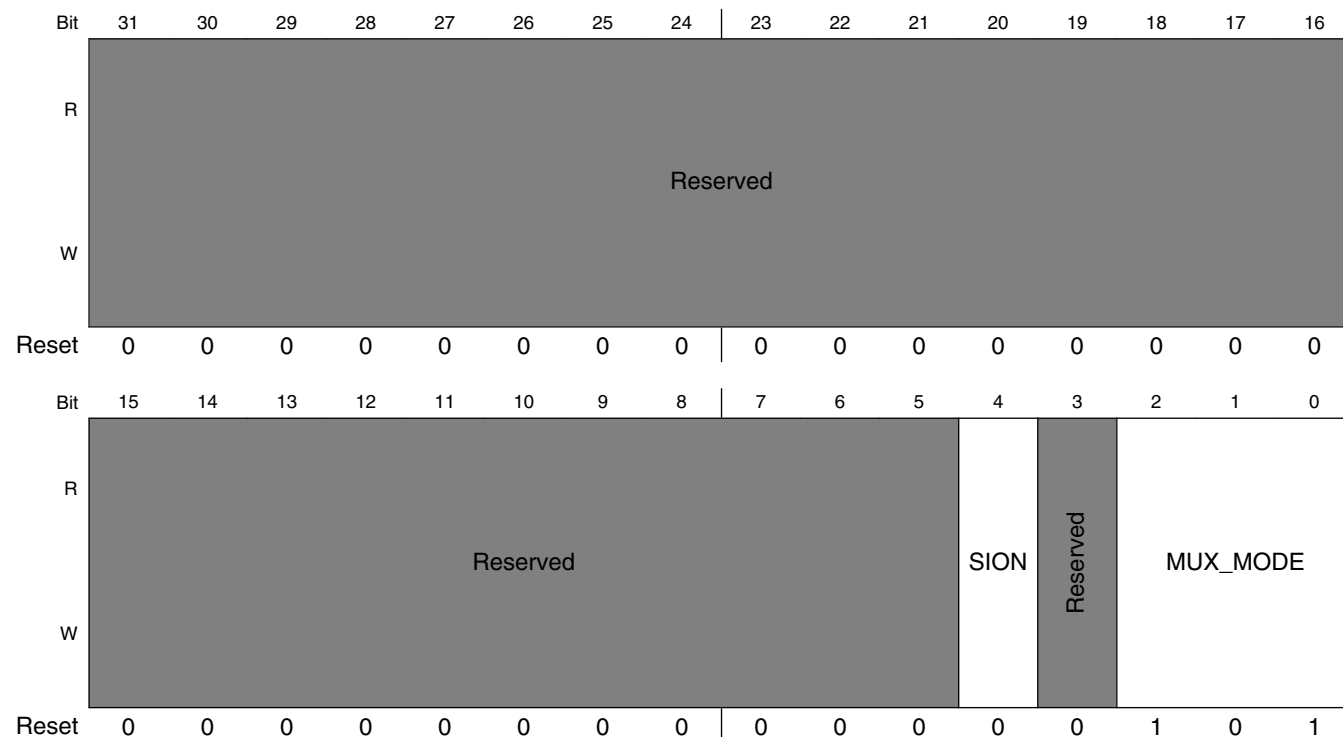
IOMUXC_SW_MUX_CTL_PAD_NAND_CE0_B field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_CE0_B is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_CE0_B
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_CE0_B 000 ALT0_RAWNAND_CE0_B — Select mux mode: ALT0 mux port: CE0_B of instance: RAWNAND 001 ALT1_QSPI_A_SS0_B — Select mux mode: ALT1 mux port: A_SS0_B of instance: QSPI 101 ALT5_GPIO3_IO01 — Select mux mode: ALT5 mux port: IO01 of instance: GPIO3

8.2.5.59 SW_MUX_CTL_PAD_NAND_CE1_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_CE1_B)

SW_MUX_CTL Register

Address: 3033_0000h base + FCh offset = 3033_00FCh



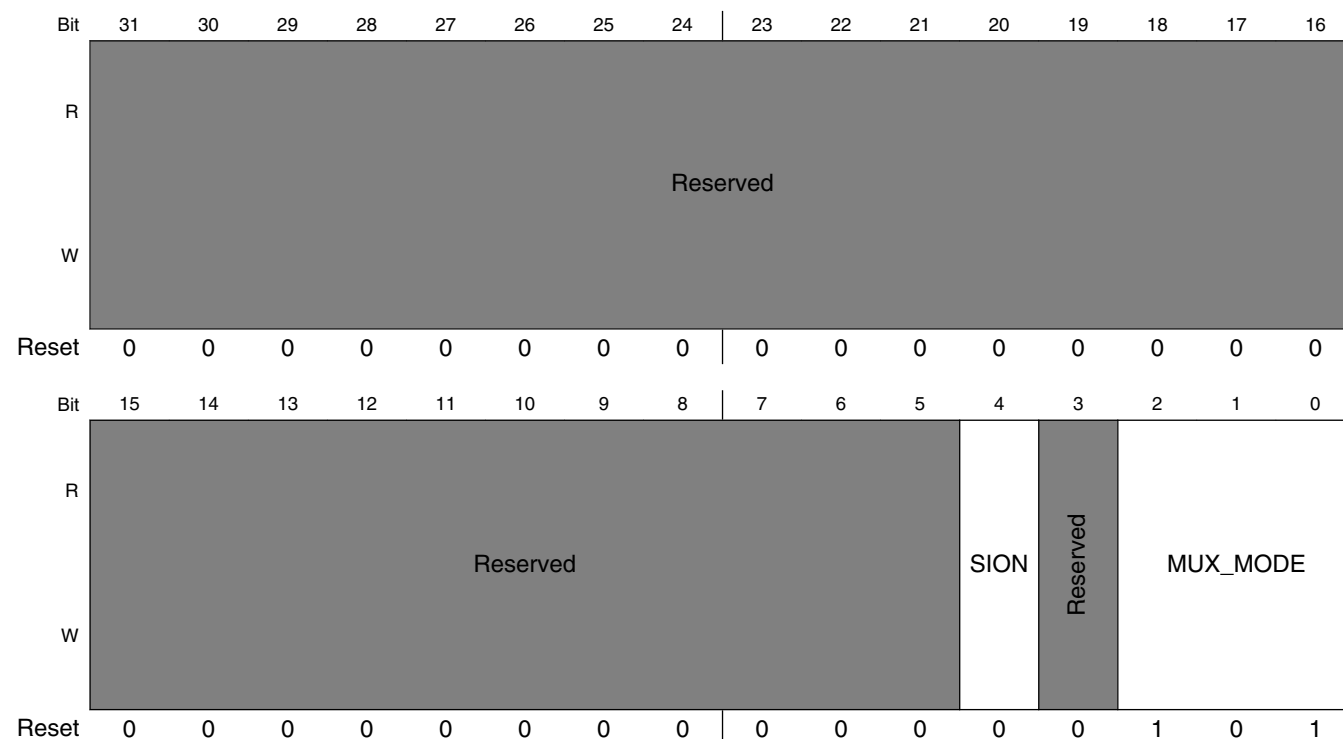
IOMUXC_SW_MUX_CTL_PAD_NAND_CE1_B field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_CE1_B is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_CE1_B
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_CE1_B 000 ALT0_RAWNAND_CE1_B — Select mux mode: ALT0 mux port: CE1_B of instance: RAWNAND 001 ALT1_QSPI_A_SS1_B — Select mux mode: ALT1 mux port: A_SS1_B of instance: QSPI 101 ALT5_GPIO3_IO02 — Select mux mode: ALT5 mux port: IO02 of instance: GPIO3

8.2.5.60 SW_MUX_CTL_PAD_NAND_CE2_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_CE2_B)

SW_MUX_CTL Register

Address: 3033_0000h base + 100h offset = 3033_0100h



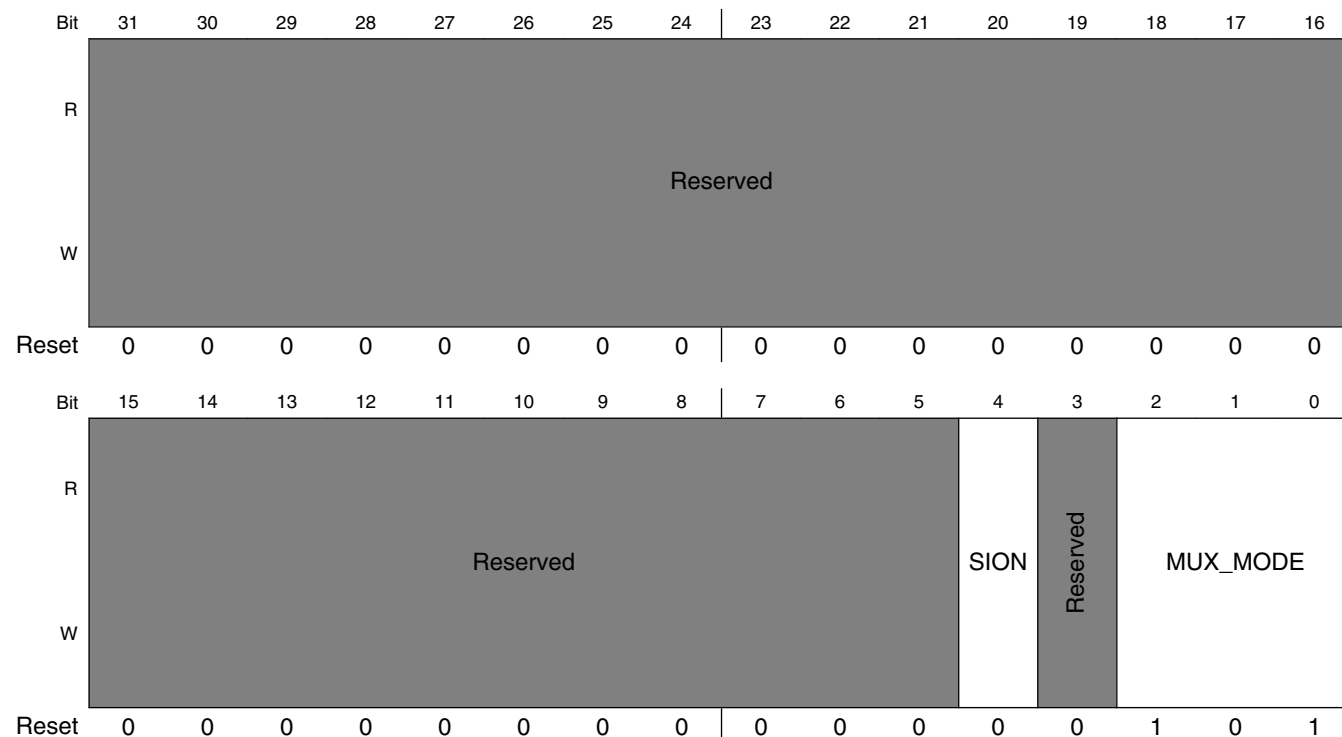
IOMUXC_SW_MUX_CTL_PAD_NAND_CE2_B field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_CE2_B is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_CE2_B
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_CE2_B 000 ALT0_RAWNAND_CE2_B — Select mux mode: ALT0 mux port: CE2_B of instance: RAWNAND 001 ALT1_QSPI_B_SS0_B — Select mux mode: ALT1 mux port: B_SS0_B of instance: QSPI 101 ALT5_GPIO3_IO03 — Select mux mode: ALT5 mux port: IO03 of instance: GPIO3

8.2.5.61 SW_MUX_CTL_PAD_NAND_CE3_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_CE3_B)

SW_MUX_CTL Register

Address: 3033_0000h base + 104h offset = 3033_0104h



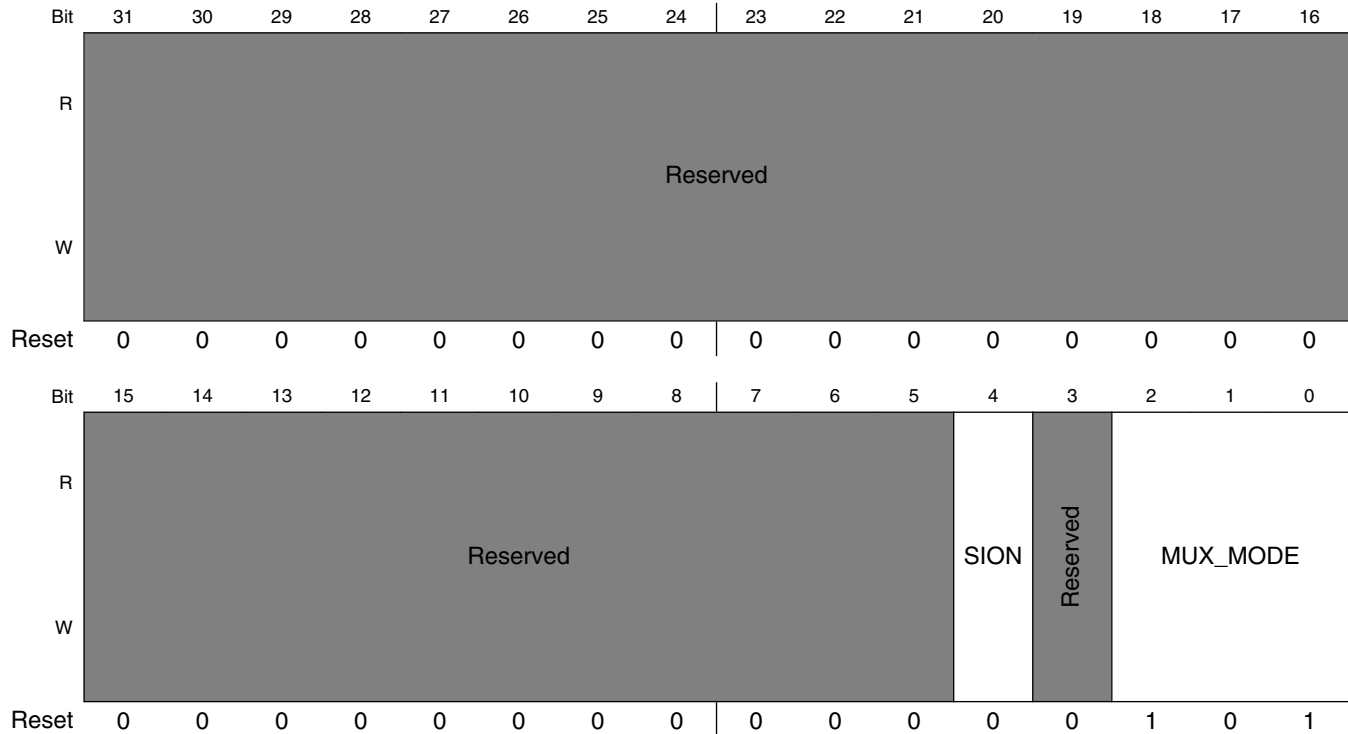
IOMUXC_SW_MUX_CTL_PAD_NAND_CE3_B field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_CE3_B is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_CE3_B
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_CE3_B 000 ALT0_RAWNAND_CE3_B — Select mux mode: ALT0 mux port: CE3_B of instance: RAWNAND 001 ALT1_QSPI_B_SS1_B — Select mux mode: ALT1 mux port: B_SS1_B of instance: QSPI 101 ALT5_GPIO3_IO04 — Select mux mode: ALT5 mux port: IO04 of instance: GPIO3

8.2.5.62 SW_MUX_CTL_PAD_NAND_CLE SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_CLE)

SW_MUX_CTL Register

Address: 3033_0000h base + 108h offset = 3033_0108h



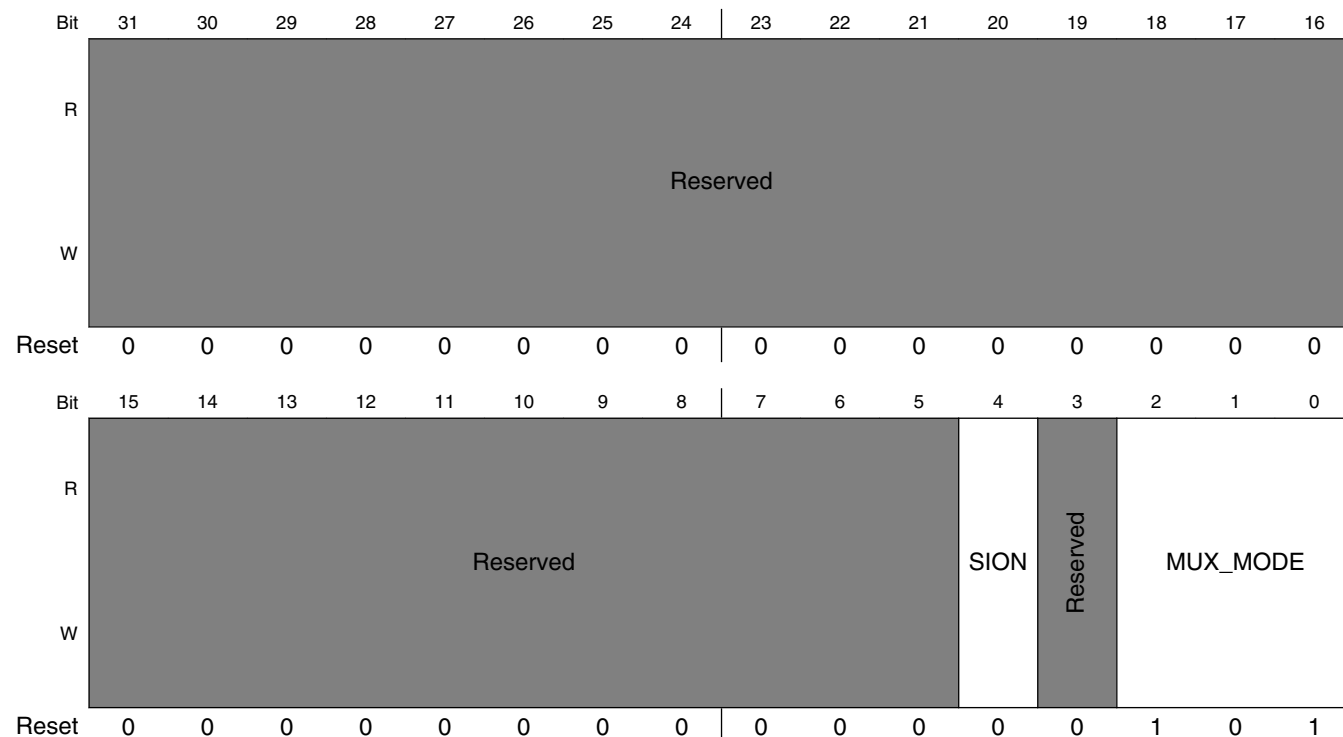
IOMUXC_SW_MUX_CTL_PAD_NAND_CLE field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_CLE is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_CLE
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_CLE 000 ALT0_RAWNAND_CLE — Select mux mode: ALT0 mux port: CLE of instance: RAWNAND 001 ALT1_QSPI_B_SCLK — Select mux mode: ALT1 mux port: B_SCLK of instance: QSPI 101 ALT5_GPIO3_IO05 — Select mux mode: ALT5 mux port: IO05 of instance: GPIO3

8.2.5.63 SW_MUX_CTL_PAD_NAND_DATA00 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA00)

SW_MUX_CTL Register

Address: 3033_0000h base + 10Ch offset = 3033_010Ch



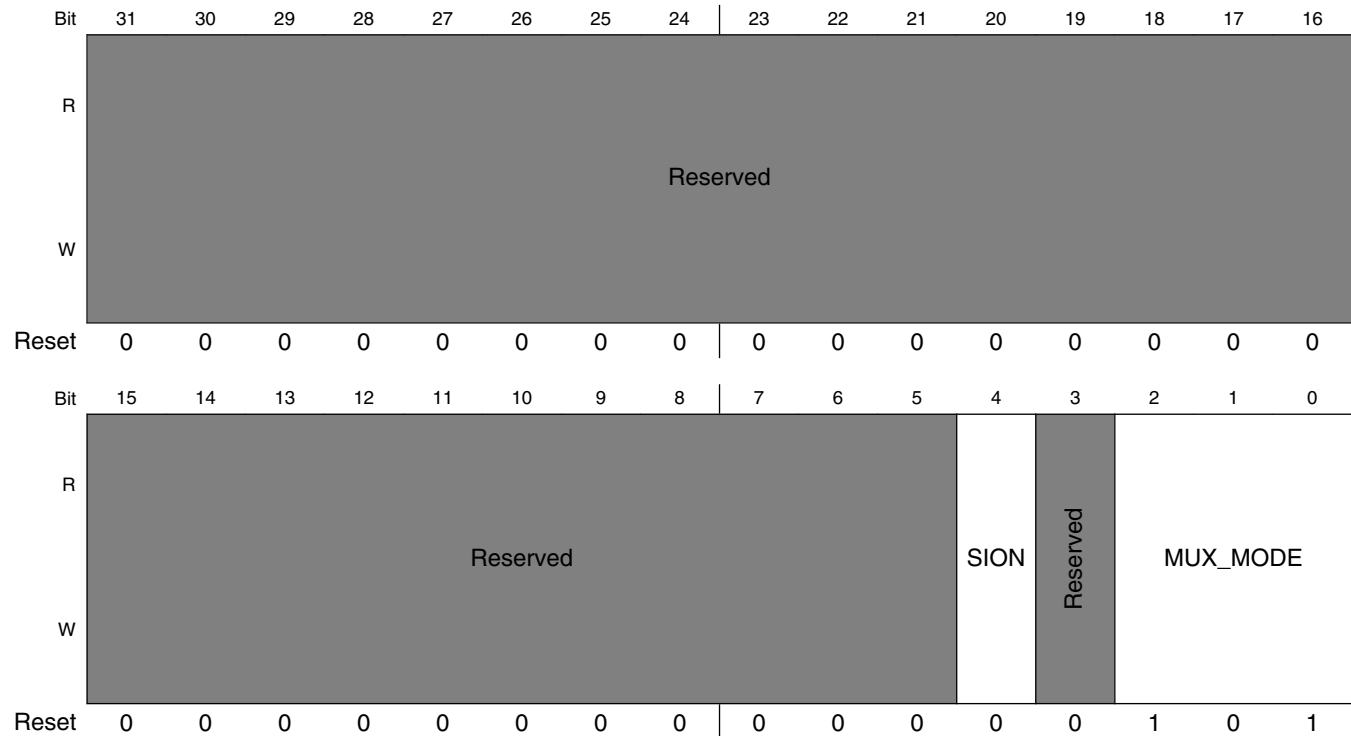
IOMUXC_SW_MUX_CTL_PAD_NAND_DATA00 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_DATA00 is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_DATA00
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_DATA00 000 ALT0_RAWNAND_DATA00 — Select mux mode: ALT0 mux port: DATA00 of instance: RAWNAND 001 ALT1_QSPI_A_DATA0 — Select mux mode: ALT1 mux port: A_DATA0 of instance: QSPI 101 ALT5_GPIO3_IO06 — Select mux mode: ALT5 mux port: IO06 of instance: GPIO3

8.2.5.64 SW_MUX_CTL_PAD_NAND_DATA01 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA01)

SW_MUX_CTL Register

Address: 3033_0000h base + 110h offset = 3033_0110h



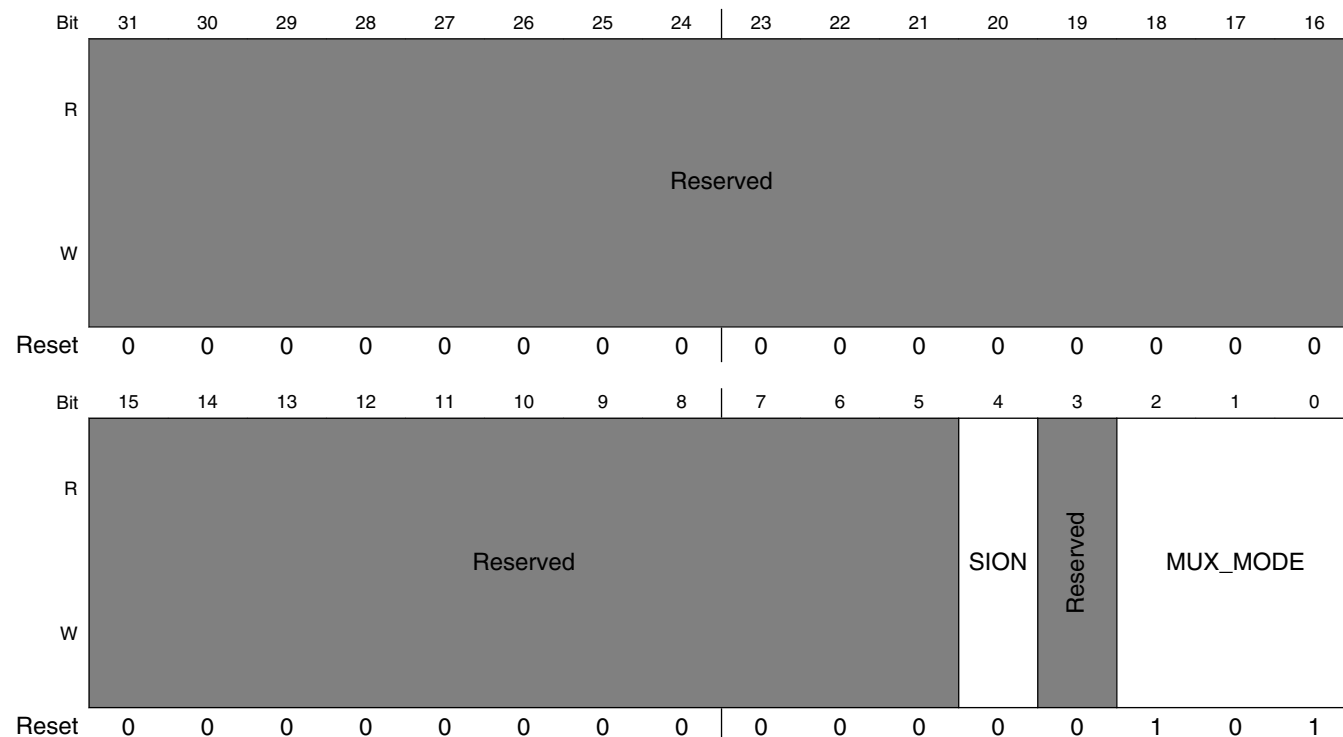
IOMUXC_SW_MUX_CTL_PAD_NAND_DATA01 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_DATA01 is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_DATA01
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_DATA01 000 ALT0_RAWNAND_DATA01 — Select mux mode: ALT0 mux port: DATA01 of instance: RAWNAND 001 ALT1_QSPI_A_DATA1 — Select mux mode: ALT1 mux port: A_DATA1 of instance: QSPI 101 ALT5_GPIO3_IO07 — Select mux mode: ALT5 mux port: IO07 of instance: GPIO3

8.2.5.65 SW_MUX_CTL_PAD_NAND_DATA02 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA02)

SW_MUX_CTL Register

Address: 3033_0000h base + 114h offset = 3033_0114h



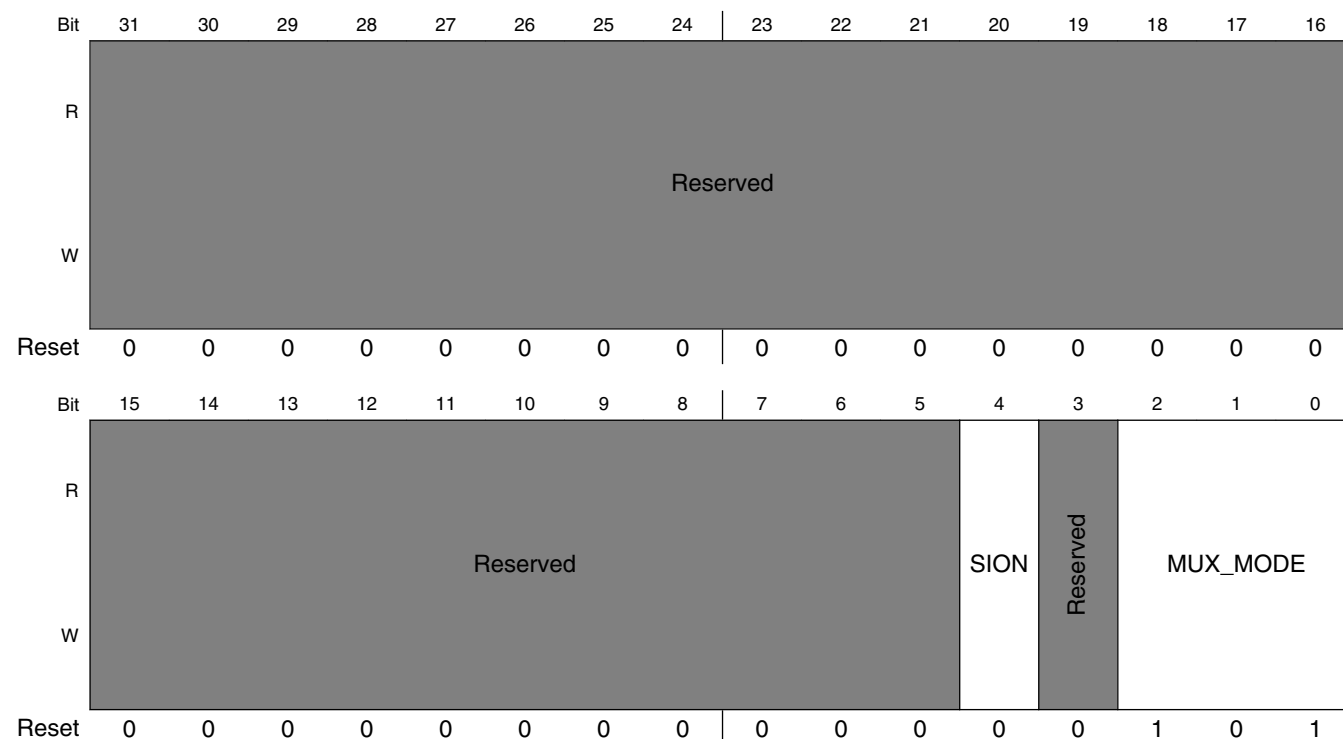
IOMUXC_SW_MUX_CTL_PAD_NAND_DATA02 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_DATA02 is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_DATA02
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_DATA02 000 ALT0_RAWNAND_DATA02 — Select mux mode: ALT0 mux port: DATA02 of instance: RAWNAND 001 ALT1_QSPI_A_DATA2 — Select mux mode: ALT1 mux port: A_DATA2 of instance: QSPI 101 ALT5_GPIO3_IO08 — Select mux mode: ALT5 mux port: IO08 of instance: GPIO3

8.2.5.66 SW_MUX_CTL_PAD_NAND_DATA03 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA03)

SW_MUX_CTL Register

Address: 3033_0000h base + 118h offset = 3033_0118h



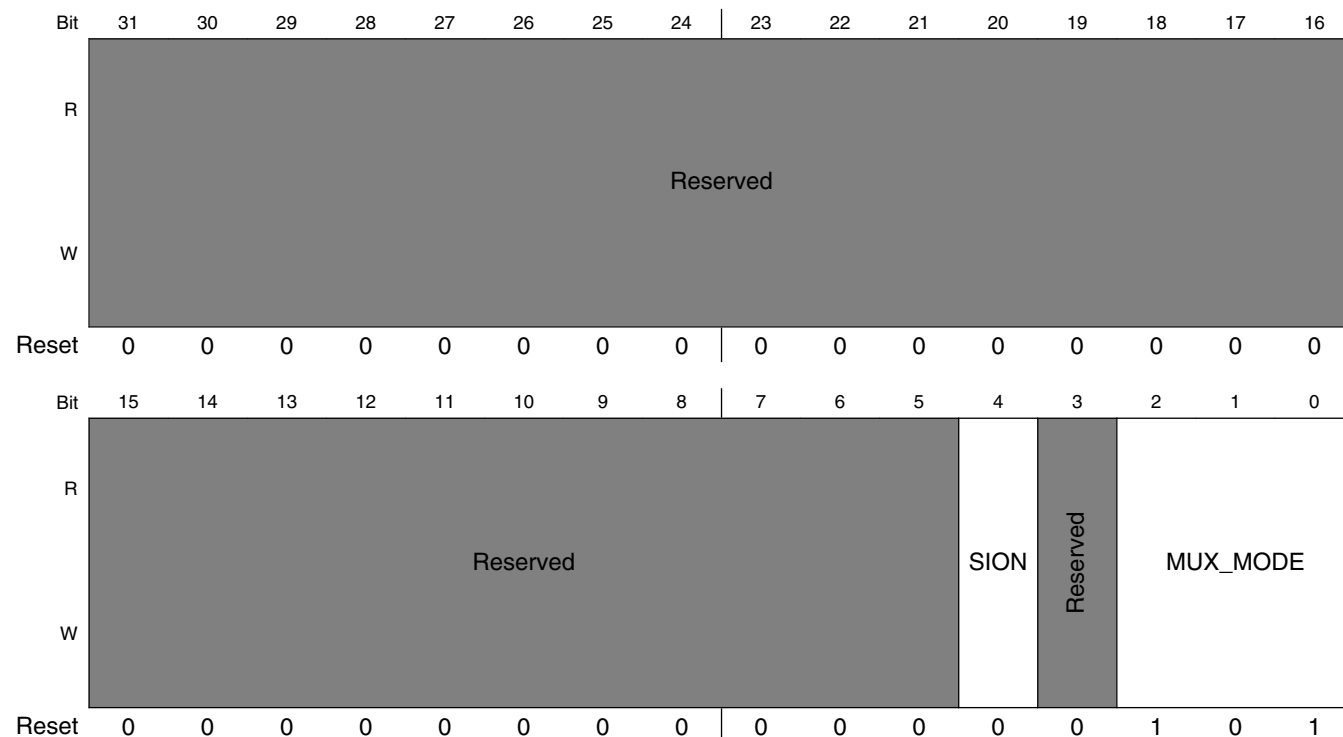
IOMUXC_SW_MUX_CTL_PAD_NAND_DATA03 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_DATA03 is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_DATA03
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_DATA03 000 ALT0_RAWNAND_DATA03 — Select mux mode: ALT0 mux port: DATA03 of instance: RAWNAND 001 ALT1_QSPI_A_DATA3 — Select mux mode: ALT1 mux port: A_DATA3 of instance: QSPI 101 ALT5_GPIO3_IO09 — Select mux mode: ALT5 mux port: IO09 of instance: GPIO3

8.2.5.67 SW_MUX_CTL_PAD_NAND_DATA04 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA04)

SW_MUX_CTL Register

Address: 3033_0000h base + 11Ch offset = 3033_011Ch



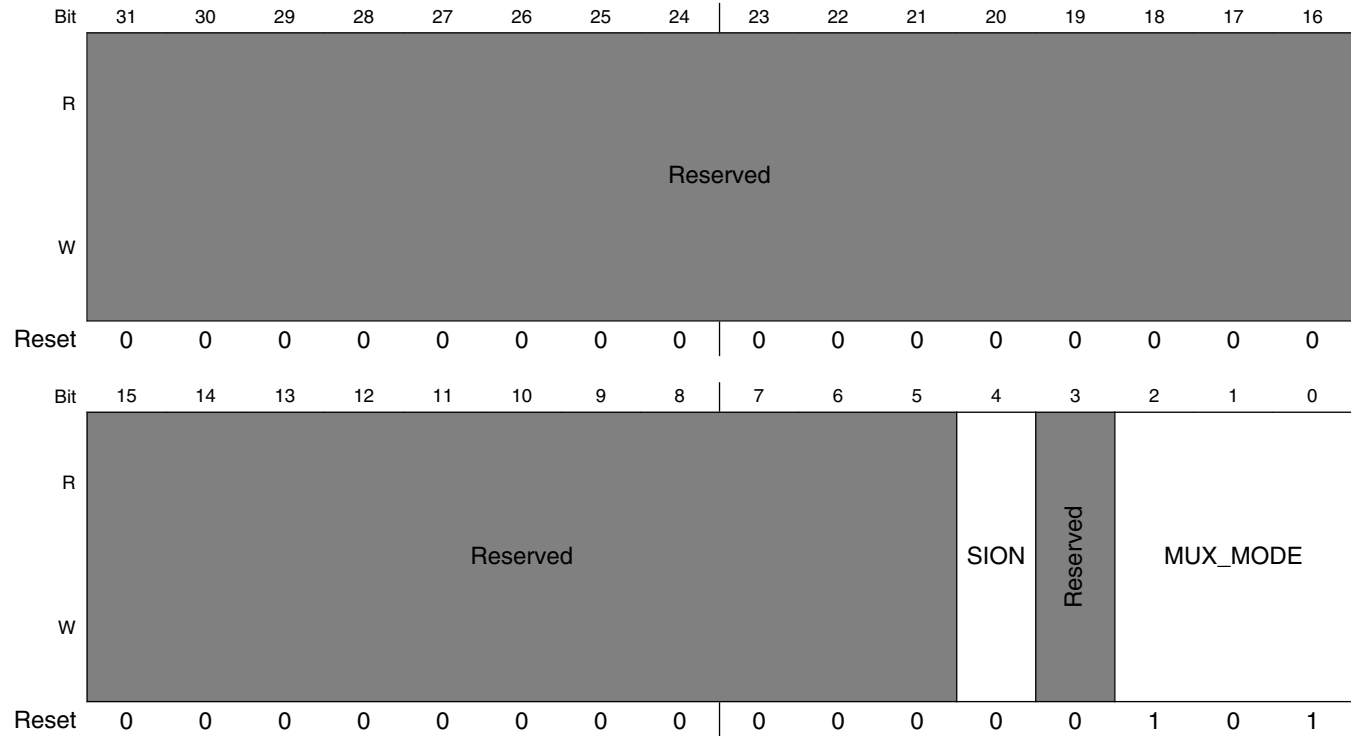
IOMUXC_SW_MUX_CTL_PAD_NAND_DATA04 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_DATA04 is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_DATA04
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_DATA04 000 ALT0_RAWNAND_DATA04 — Select mux mode: ALT0 mux port: DATA04 of instance: RAWNAND 001 ALT1_QSPI_B_DATA0 — Select mux mode: ALT1 mux port: B_DATA0 of instance: QSPI 101 ALT5_GPIO3_IO10 — Select mux mode: ALT5 mux port: IO10 of instance: GPIO3

8.2.5.68 SW_MUX_CTL_PAD_NAND_DATA05 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA05)

SW_MUX_CTL Register

Address: 3033_0000h base + 120h offset = 3033_0120h



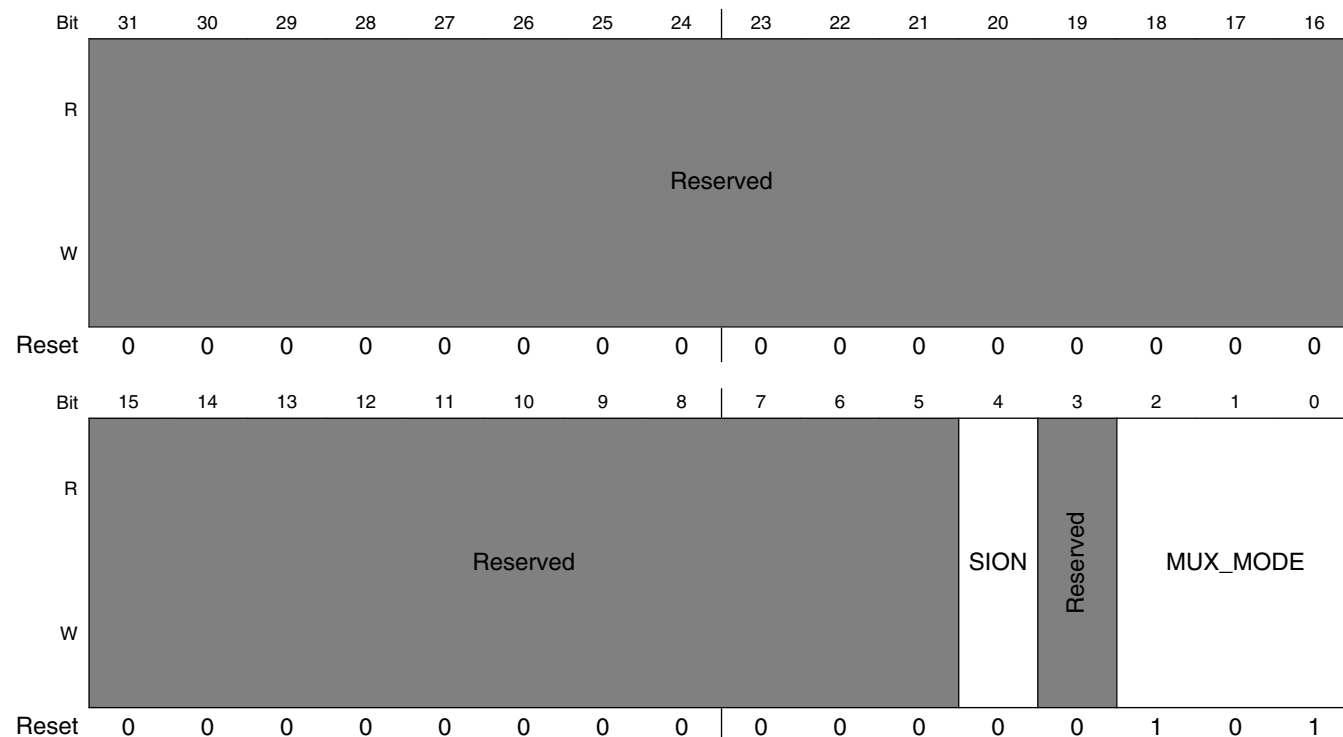
IOMUXC_SW_MUX_CTL_PAD_NAND_DATA05 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_DATA05 is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_DATA05
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_DATA05 000 ALT0_RAWNAND_DATA05 — Select mux mode: ALT0 mux port: DATA05 of instance: RAWNAND 001 ALT1_QSPI_B_DATA1 — Select mux mode: ALT1 mux port: B_DATA1 of instance: QSPI 101 ALT5_GPIO3_IO11 — Select mux mode: ALT5 mux port: IO11 of instance: GPIO3

8.2.5.69 SW_MUX_CTL_PAD_NAND_DATA06 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA06)

SW_MUX_CTL Register

Address: 3033_0000h base + 124h offset = 3033_0124h



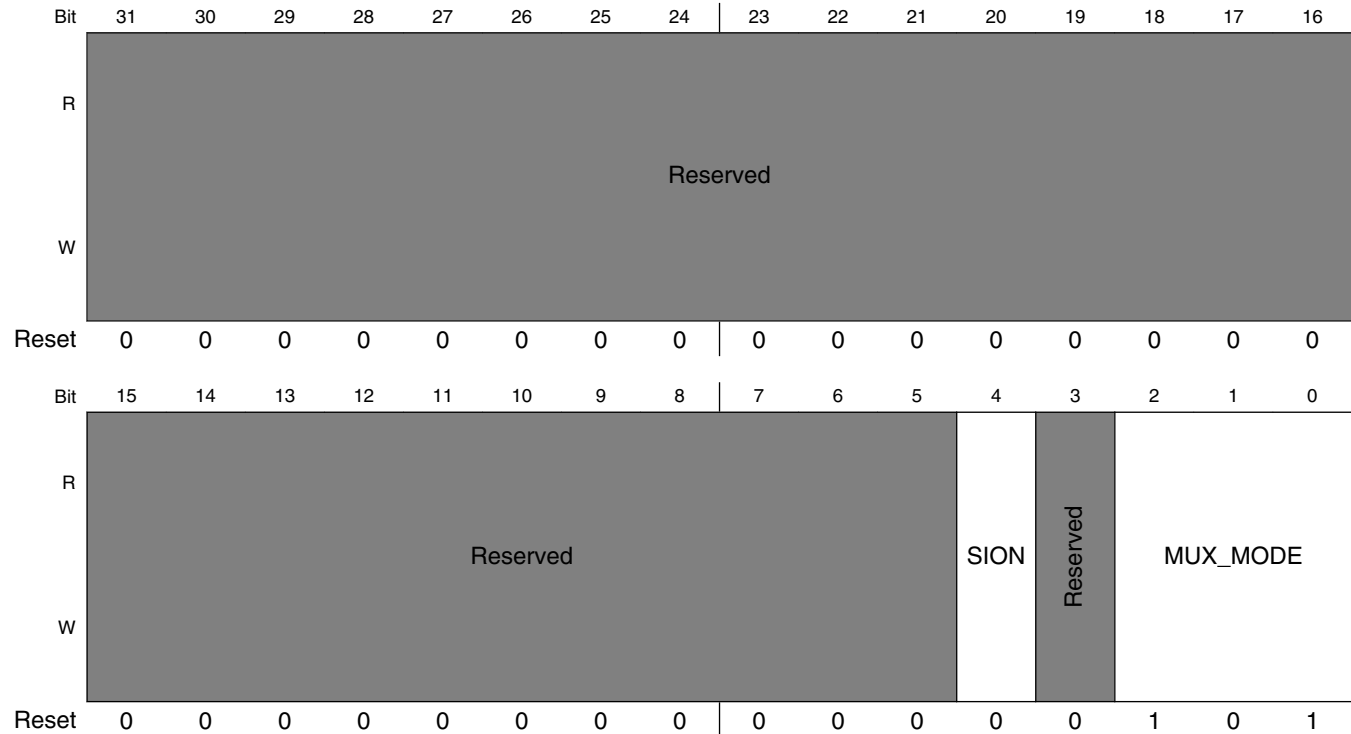
IOMUXC_SW_MUX_CTL_PAD_NAND_DATA06 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_DATA06 is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_DATA06
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_DATA06 000 ALT0_RAWNAND_DATA06 — Select mux mode: ALT0 mux port: DATA06 of instance: RAWNAND 001 ALT1_QSPI_B_DATA2 — Select mux mode: ALT1 mux port: B_DATA2 of instance: QSPI 101 ALT5_GPIO3_IO12 — Select mux mode: ALT5 mux port: IO12 of instance: GPIO3

8.2.5.70 SW_MUX_CTL_PAD_NAND_DATA07 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DATA07)

SW_MUX_CTL Register

Address: 3033_0000h base + 128h offset = 3033_0128h



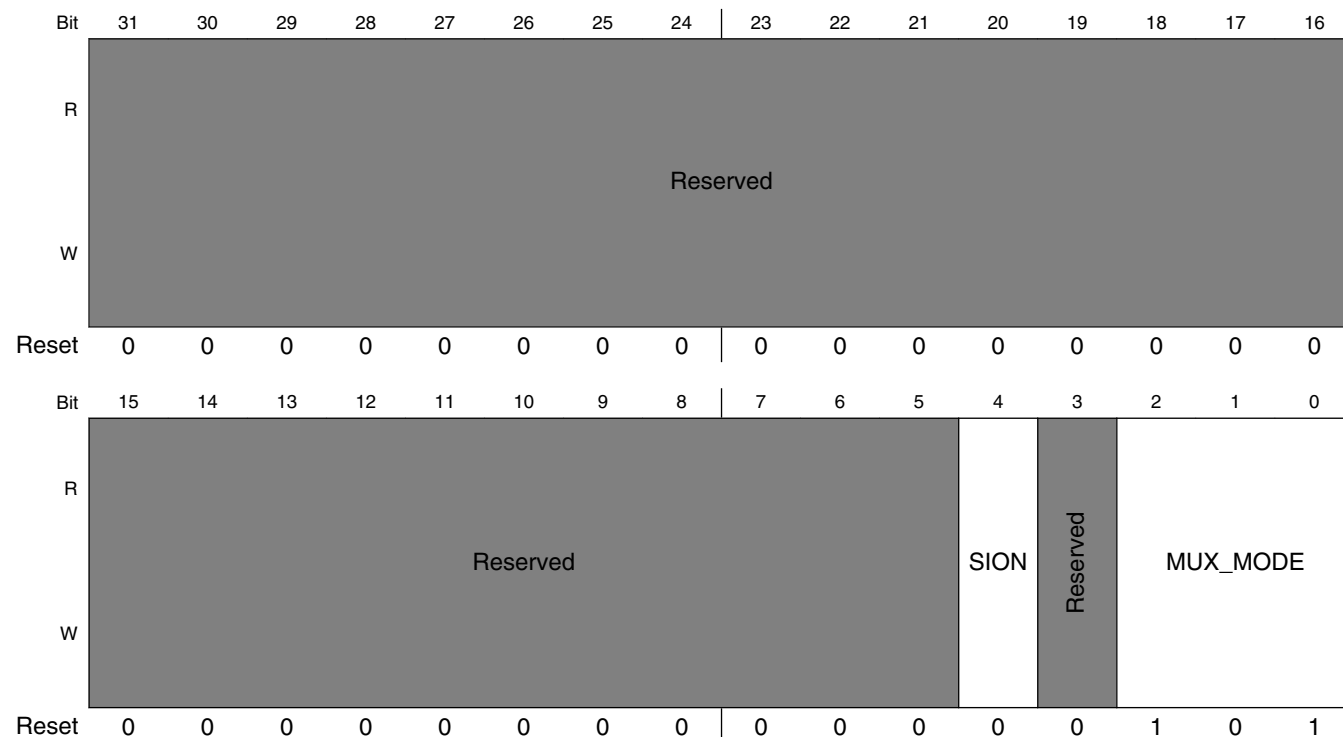
IOMUXC_SW_MUX_CTL_PAD_NAND_DATA07 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_DATA07 is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_DATA07
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_DATA07 000 ALT0_RAWNAND_DATA07 — Select mux mode: ALT0 mux port: DATA07 of instance: RAWNAND 001 ALT1_QSPI_B_DATA3 — Select mux mode: ALT1 mux port: B_DATA3 of instance: QSPI 101 ALT5_GPIO3_IO13 — Select mux mode: ALT5 mux port: IO13 of instance: GPIO3

8.2.5.71 SW_MUX_CTL_PAD_NAND_DQS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_DQS)

SW_MUX_CTL Register

Address: 3033_0000h base + 12Ch offset = 3033_012Ch



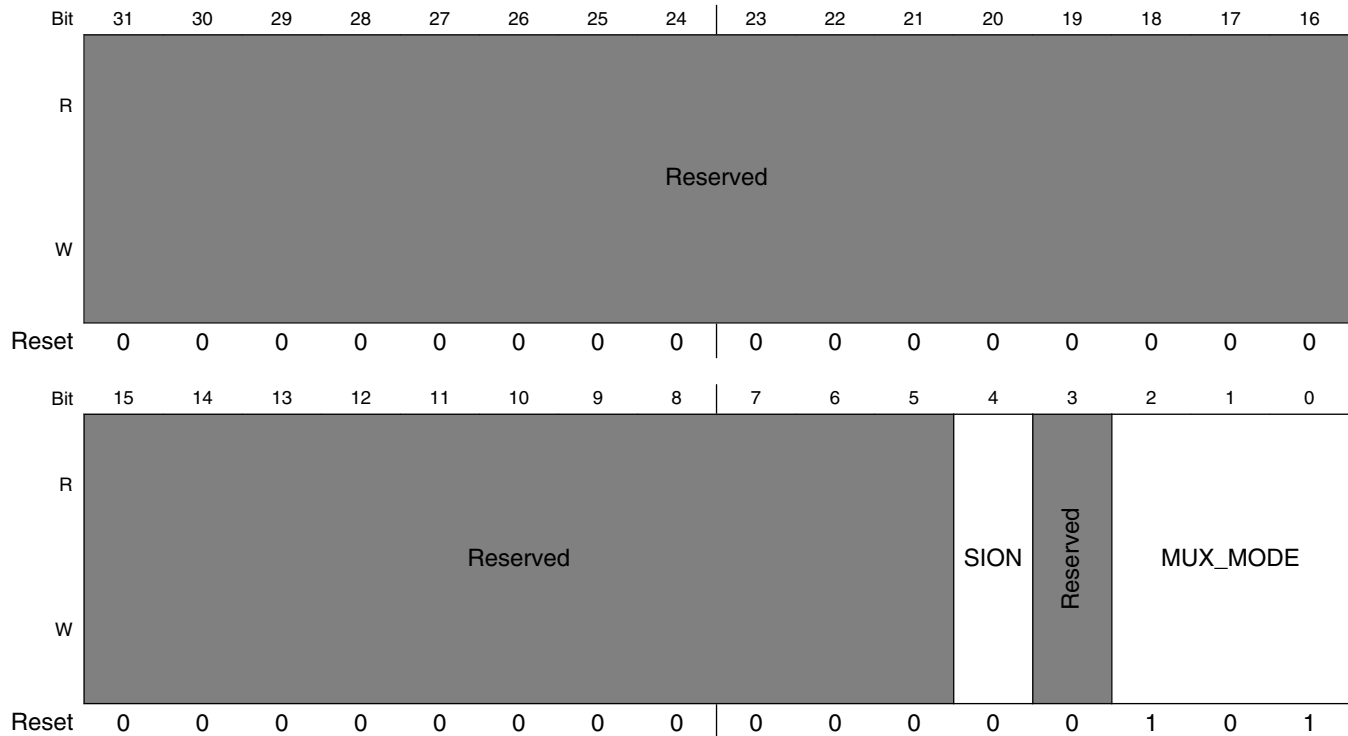
IOMUXC_SW_MUX_CTL_PAD_NAND_DQS field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_DQS is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_DQS
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_DQS 000 ALT0_RAWNAND_DQS — Select mux mode: ALT0 mux port: DQS of instance: RAWNAND 001 ALT1_QSPI_A_DQS — Select mux mode: ALT1 mux port: A_DQS of instance: QSPI 101 ALT5_GPIO3_IO14 — Select mux mode: ALT5 mux port: IO14 of instance: GPIO3

8.2.5.72 SW_MUX_CTL_PAD_NAND_RE_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_RE_B)

SW_MUX_CTL Register

Address: 3033_0000h base + 130h offset = 3033_0130h



IOMUXC_SW_MUX_CTL_PAD_NAND_RE_B field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_RE_B is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_RE_B
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: NAND_RE_B 000 ALT0_RAWNAND_RE_B — Select mux mode: ALT0 mux port: RE_B of instance: RAWNAND 001 ALT1_QSPI_B_DQS — Select mux mode: ALT1 mux port: B_DQS of instance: QSPI 101 ALT5_GPIO3_IO15 — Select mux mode: ALT5 mux port: IO15 of instance: GPIO3

8.2.5.73 SW_MUX_CTL_PAD_NAND_READY_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_READY_B)

SW_MUX_CTL Register

Address: 3033_0000h base + 134h offset = 3033_0134h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

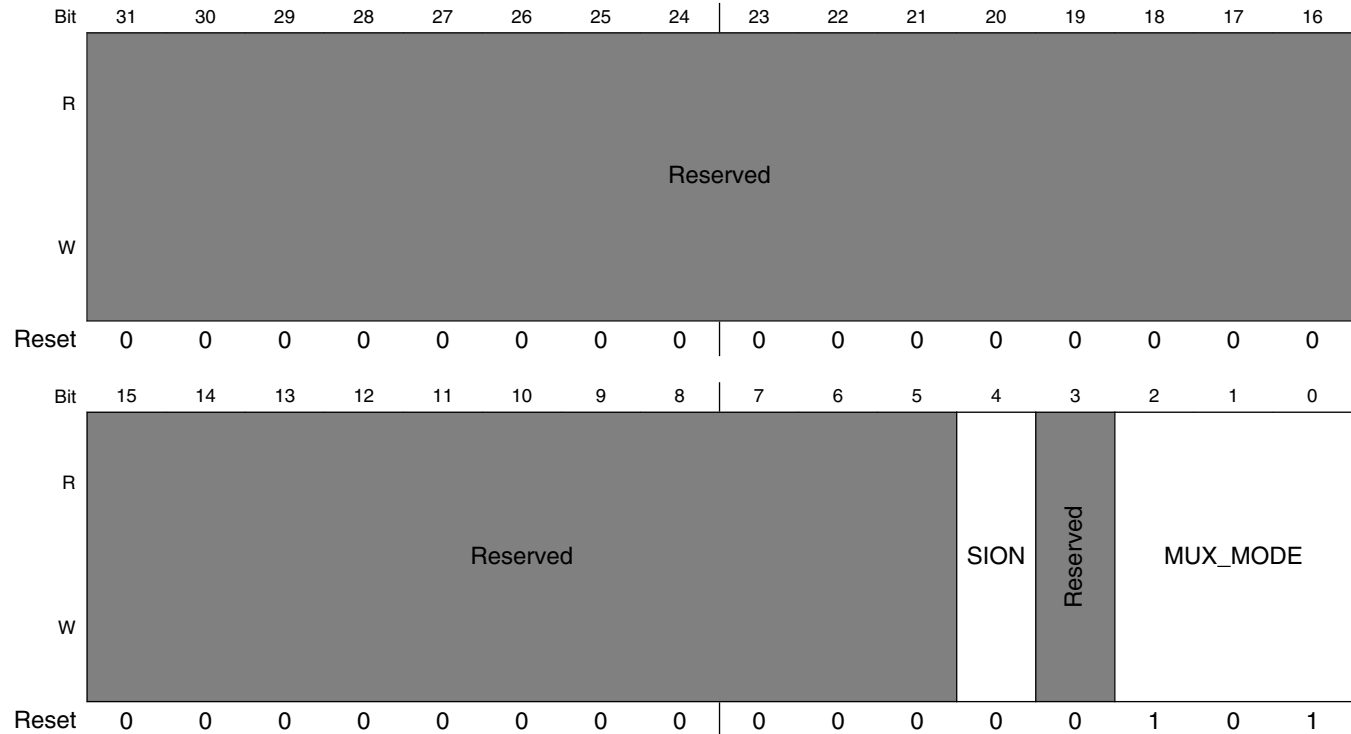
IOMUXC_SW_MUX_CTL_PAD_NAND_READY_B field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_READY_B is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_READY_B
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: NAND_READY_B 000 ALT0_RAWNAND_READY_B — Select mux mode: ALT0 mux port: READY_B of instance: RAWNAND 101 ALT5_GPIO3_IO16 — Select mux mode: ALT5 mux port: IO16 of instance: GPIO3

8.2.5.74 SW_MUX_CTL_PAD_NAND_WE_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_WE_B)

SW_MUX_CTL Register

Address: 3033_0000h base + 138h offset = 3033_0138h



IOMUXC_SW_MUX_CTL_PAD_NAND_WE_B field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_WE_B is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_WE_B
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: NAND_WE_B 000 ALT0_RAWNAND_WE_B — Select mux mode: ALT0 mux port: WE_B of instance: RAWNAND 101 ALT5_GPIO3_IO17 — Select mux mode: ALT5 mux port: IO17 of instance: GPIO3

8.2.5.75 SW_MUX_CTL_PAD_NAND_WP_B SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_NAND_WP_B)

SW_MUX_CTL Register

Address: 3033_0000h base + 13Ch offset = 3033_013Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION				Reserved	MUX_MODE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

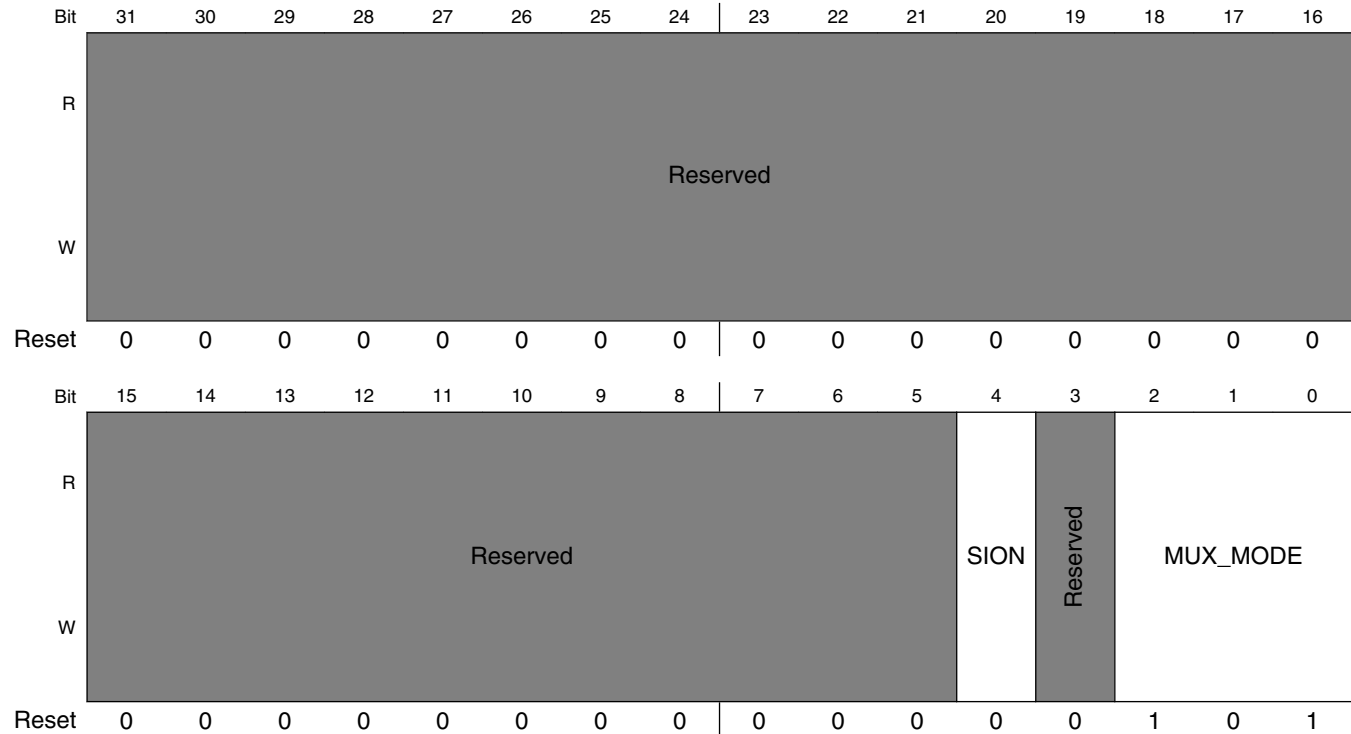
IOMUXC_SW_MUX_CTL_PAD_NAND_WP_B field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad NAND_WP_B is determined by functionality 1 SION_ENABLED — Force Input Path of pad NAND_WP_B
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 2 iomux modes to be used for pad: NAND_WP_B 000 ALT0_RAWNAND_WP_B — Select mux mode: ALT0 mux port: WP_B of instance: RAWNAND 101 ALT5_GPIO3_IO18 — Select mux mode: ALT5 mux port: IO18 of instance: GPIO3

8.2.5.76 SW_MUX_CTL_PAD_SAI5_RXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_RXFS)

SW_MUX_CTL Register

Address: 3033_0000h base + 140h offset = 3033_0140h



IOMUXC_SW_MUX_CTL_PAD_SAI5_RXFS field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI5_RXFS is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI5_RXFS
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: SAI5_RXFS 000 ALT0_SAI5_RX_SYNC — Select mux mode: ALT0 mux port: RX_SYNC of instance: SAI5 001 ALT1_SAI1_TX_DATA0 — Select mux mode: ALT1 mux port: TX_DATA0 of instance: SAI1 101 ALT5_GPIO3_IO19 — Select mux mode: ALT5 mux port: IO19 of instance: GPIO3

8.2.5.77 SW_MUX_CTL_PAD_SAI5_RXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_RXC)

SW_MUX_CTL Register

Address: 3033_0000h base + 144h offset = 3033_0144h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved											SION	Reserved	MUX_MODE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI5_RXC field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI5_RXC is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI5_RXC
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: SAI5_RXC 000 ALT0_SAI5_RX_BCLK — Select mux mode: ALT0 mux port: RX_BCLK of instance: SAI5 001 ALT1_SAI1_TX_DATA1 — Select mux mode: ALT1 mux port: TX_DATA1 of instance: SAI1 101 ALT5_GPIO3_IO20 — Select mux mode: ALT5 mux port: IO20 of instance: GPIO3

8.2.5.78 SW_MUX_CTL_PAD_SAI5_RXD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD0)

SW_MUX_CTL Register

Address: 3033_0000h base + 148h offset = 3033_0148h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD0 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI5_RXD0 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI5_RXD0
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: SAI5_RXD0 000 ALT0_SAI5_RX_DATA0 — Select mux mode: ALT0 mux port: RX_DATA0 of instance: SAI5 001 ALT1_SAI1_TX_DATA2 — Select mux mode: ALT1 mux port: TX_DATA2 of instance: SAI1 101 ALT5_GPIO3_IO21 — Select mux mode: ALT5 mux port: IO21 of instance: GPIO3

8.2.5.79 SW_MUX_CTL_PAD_SAI5_RXD1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD1)

SW_MUX_CTL Register

Address: 3033_0000h base + 14Ch offset = 3033_014Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD1 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI5_RXD1 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI5_RXD1
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 5 iomux modes to be used for pad: SAI5_RXD1 000 ALT0_SAI5_RX_DATA1 — Select mux mode: ALT0 mux port: RX_DATA1 of instance: SAI5 001 ALT1_SAI1_TX_DATA3 — Select mux mode: ALT1 mux port: TX_DATA3 of instance: SAI1 010 ALT2_SAI1_TX_SYNC — Select mux mode: ALT2 mux port: TX_SYNC of instance: SAI1

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD1 field descriptions (continued)

Field	Description
011 ALT3_SAI5_TX_SYNC	Select mux mode: ALT3 mux port: TX_SYNC of instance: SAI5
101 ALT5_GPIO3_IO22	Select mux mode: ALT5 mux port: IO22 of instance: GPIO3

8.2.5.80 SW_MUX_CTL_PAD_SAI5_RXD2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD2)**SW_MUX_CTL Register**

Address: 3033_0000h base + 150h offset = 3033_0150h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved										SION	Reserved	MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD2 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI5_RXD2 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI5_RXD2
3 -	This field is reserved. Reserved

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD2 field descriptions (continued)

Field	Description
MUX_MODE	MUX Mode Select Field Select 1 of 5 iomux modes to be used for pad: SAI5_RXD2 000 ALT0_SAI5_RX_DATA2 — Select mux mode: ALT0 mux port: RX_DATA2 of instance: SAI5 001 ALT1_SAI1_TX_DATA4 — Select mux mode: ALT1 mux port: TX_DATA4 of instance: SAI1 010 ALT2_SAI1_TX_SYNC — Select mux mode: ALT2 mux port: TX_SYNC of instance: SAI1 011 ALT3_SAI5_TX_BCLK — Select mux mode: ALT3 mux port: TX_BCLK of instance: SAI5 101 ALT5_GPIO3_IO23 — Select mux mode: ALT5 mux port: IO23 of instance: GPIO3

8.2.5.81 SW_MUX_CTL_PAD_SAI5_RXD3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD3)**SW_MUX_CTL Register**

Address: 3033_0000h base + 154h offset = 3033_0154h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved											SION	Reserved	MUX_MODE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD3 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI5_RXD3 field descriptions (continued)

Field	Description
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI5_RXD3 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI5_RXD3
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 5 iomux modes to be used for pad: SAI5_RXD3 000 ALT0_SAI5_RX_DATA3 — Select mux mode: ALT0 mux port: RX_DATA3 of instance: SAI5 001 ALT1_SAI1_TX_DATA5 — Select mux mode: ALT1 mux port: TX_DATA5 of instance: SAI1 010 ALT2_SAI1_TX_SYNC — Select mux mode: ALT2 mux port: TX_SYNC of instance: SAI1 011 ALT3_SAI5_TX_DATA0 — Select mux mode: ALT3 mux port: TX_DATA0 of instance: SAI5 101 ALT5_GPIO3_IO24 — Select mux mode: ALT5 mux port: IO24 of instance: GPIO3

8.2.5.82 SW_MUX_CTL_PAD_SAI5_MCLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI5_MCLK)**SW_MUX_CTL Register**

Address: 3033_0000h base + 158h offset = 3033_0158h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved											SION	Reserved	MUX_MODE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI5_MCLK field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI5_MCLK is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI5_MCLK
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: SAI5_MCLK 000 ALT0_SAI5_MCLK — Select mux mode: ALT0 mux port: MCLK of instance: SAI5 001 ALT1_SAI1_TX_BCLK — Select mux mode: ALT1 mux port: TX_BCLK of instance: SAI1 010 ALT2_SAI4_MCLK — Select mux mode: ALT2 mux port: MCLK of instance: SAI4 101 ALT5_GPIO3_IO25 — Select mux mode: ALT5 mux port: IO25 of instance: GPIO3

8.2.5.83 SW_MUX_CTL_PAD_SAI1_RXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXFS)**SW_MUX_CTL Register**

Address: 3033_0000h base + 15Ch offset = 3033_015Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												SION	Reserved	MUX_MODE	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

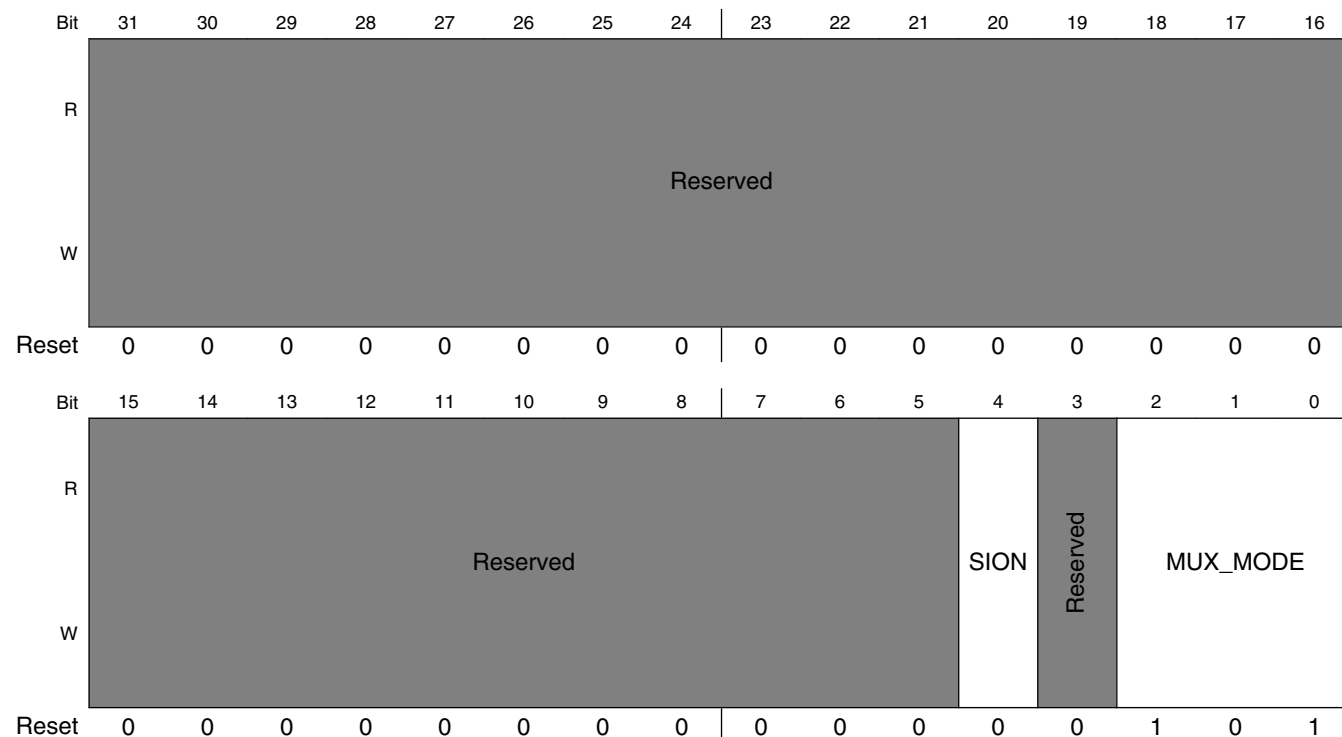
IOMUXC_SW_MUX_CTL_PAD_SAI1_RXFS field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_RXFS is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_RXFS
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: SAI1_RXFS 000 ALT0_SAI1_RX_SYNC — Select mux mode: ALT0 mux port: RX_SYNC of instance: SAI1 001 ALT1_SAI5_RX_SYNC — Select mux mode: ALT1 mux port: RX_SYNC of instance: SAI5 100 ALT4_CORESIGHT_TRACE_CLK — Select mux mode: ALT4 mux port: TRACE_CLK of instance: CORESIGHT 101 ALT5_GPIO4_IO00 — Select mux mode: ALT5 mux port: IO00 of instance: GPIO4

8.2.5.84 SW_MUX_CTL_PAD_SAI1_RXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXC)

SW_MUX_CTL Register

Address: 3033_0000h base + 160h offset = 3033_0160h



IOMUXC_SW_MUX_CTL_PAD_SAI1_RXC field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_RXC is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_RXC
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: SAI1_RXC 000 ALT0_SAI1_RX_BCLK — Select mux mode: ALT0 mux port: RX_BCLK of instance: SAI1 001 ALT1_SAI5_RX_BCLK — Select mux mode: ALT1 mux port: RX_BCLK of instance: SAI5

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXC field descriptions (continued)

Field	Description
100	ALT4_CORESIGHT_TRACE_CTL — Select mux mode: ALT4 mux port: TRACE_CTL of instance: CORESIGHT
101	ALT5_GPIO4_IO01 — Select mux mode: ALT5 mux port: IO01 of instance: GPIO4

8.2.5.85 SW_MUX_CTL_PAD_SAI1_RXD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD0)**SW_MUX_CTL Register**

Address: 3033_0000h base + 164h offset = 3033_0164h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved																
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved											SION		Reserved	MUX_MODE		
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD0 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_RXD0 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_RXD0

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD0 field descriptions (continued)

Field	Description
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 5 iomux modes to be used for pad: SAI1_RXD0 000 ALT0_SAI1_RX_DATA0 — Select mux mode: ALT0 mux port: RX_DATA0 of instance: SAI1 001 ALT1_SAI5_RX_DATA0 — Select mux mode: ALT1 mux port: RX_DATA0 of instance: SAI5 100 ALT4_CORESIGHT_TRACE0 — Select mux mode: ALT4 mux port: TRACE0 of instance: CORESIGHT 101 ALT5_GPIO4_IO02 — Select mux mode: ALT5 mux port: IO02 of instance: GPIO4 110 ALT6_SRC_BOOT_CFG0 — Select mux mode: ALT6 mux port: BOOT_CFG0 of instance: SRC

8.2.5.86 SW_MUX_CTL_PAD_SAI1_RXD1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD1)

SW_MUX_CTL Register

Address: 3033_0000h base + 168h offset = 3033_0168h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												SION	Reserved	MUX_MODE	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD1 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_RXD1 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_RXD1
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 5 iomux modes to be used for pad: SAI1_RXD1 000 ALT0_SAI1_RX_DATA1 — Select mux mode: ALT0 mux port: RX_DATA1 of instance: SAI1 001 ALT1_SAI5_RX_DATA1 — Select mux mode: ALT1 mux port: RX_DATA1 of instance: SAI5 100 ALT4_CORESIGHT_TRACE1 — Select mux mode: ALT4 mux port: TRACE1 of instance: CORESIGHT 101 ALT5_GPIO4_IO03 — Select mux mode: ALT5 mux port: IO03 of instance: GPIO4 110 ALT6_SRC_BOOT_CFG1 — Select mux mode: ALT6 mux port: BOOT_CFG1 of instance: SRC

8.2.5.87 SW_MUX_CTL_PAD_SAI1_RXD2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD2)

SW_MUX_CTL Register

Address: 3033_0000h base + 16Ch offset = 3033_016Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved											SION	Reserved	MUX_MODE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD2 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_RXD2 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_RXD2
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 5 iomux modes to be used for pad: SAI1_RXD2 000 ALT0_SAI1_RX_DATA2 — Select mux mode: ALT0 mux port: RX_DATA2 of instance: SAI1 001 ALT1_SAI5_RX_DATA2 — Select mux mode: ALT1 mux port: RX_DATA2 of instance: SAI5

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD2 field descriptions (continued)

Field	Description
100	ALT4_CORESIGHT_TRACE2 — Select mux mode: ALT4 mux port: TRACE2 of instance: CORESIGHT
101	ALT5_GPIO4_IO04 — Select mux mode: ALT5 mux port: IO04 of instance: GPIO4
110	ALT6_SRC_BOOT_CFG2 — Select mux mode: ALT6 mux port: BOOT_CFG2 of instance: SRC

8.2.5.88 SW_MUX_CTL_PAD_SAI1_RXD3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD3)**SW_MUX_CTL Register**

Address: 3033_0000h base + 170h offset = 3033_0170h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD3 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_RXD3 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_RXD3

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD3 field descriptions (continued)

Field	Description
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 5 iomux modes to be used for pad: SAI1_RXD3 000 ALT0_SAI1_RX_DATA3 — Select mux mode: ALT0 mux port: RX_DATA3 of instance: SAI1 001 ALT1_SAI5_RX_DATA3 — Select mux mode: ALT1 mux port: RX_DATA3 of instance: SAI5 100 ALT4_CORESIGHT_TRACE3 — Select mux mode: ALT4 mux port: TRACE3 of instance: CORESIGHT 101 ALT5_GPIO4_IO05 — Select mux mode: ALT5 mux port: IO05 of instance: GPIO4 110 ALT6_SRC_BOOT_CFG3 — Select mux mode: ALT6 mux port: BOOT_CFG3 of instance: SRC

8.2.5.89 SW_MUX_CTL_PAD_SAI1_RXD4 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD4)

SW_MUX_CTL Register

Address: 3033_0000h base + 174h offset = 3033_0174h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION				Reserved	MUX_MODE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD4 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_RXD4 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_RXD4
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 6 iomux modes to be used for pad: SAI1_RXD4 000 ALT0_SAI1_RX_DATA4 — Select mux mode: ALT0 mux port: RX_DATA4 of instance: SAI1 001 ALT1_SAI6_TX_BCLK — Select mux mode: ALT1 mux port: TX_BCLK of instance: SAI6 010 ALT2_SAI6_RX_BCLK — Select mux mode: ALT2 mux port: RX_BCLK of instance: SAI6 100 ALT4_CORESIGHT_TRACE4 — Select mux mode: ALT4 mux port: TRACE4 of instance: CORESIGHT 101 ALT5_GPIO4_IO06 — Select mux mode: ALT5 mux port: IO06 of instance: GPIO4 110 ALT6_SRC_BOOT_CFG4 — Select mux mode: ALT6 mux port: BOOT_CFG4 of instance: SRC

8.2.5.90 SW_MUX_CTL_PAD_SAI1_RXD5 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD5)

SW_MUX_CTL Register

Address: 3033_0000h base + 178h offset = 3033_0178h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD5 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_RXD5 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_RXD5
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 7 iomux modes to be used for pad: SAI1_RXD5 000 ALT0_SAI1_RX_DATA5 — Select mux mode: ALT0 mux port: RX_DATA5 of instance: SAI1 001 ALT1_SAI6_TX_DATA0 — Select mux mode: ALT1 mux port: TX_DATA0 of instance: SAI6 010 ALT2_SAI6_RX_DATA0 — Select mux mode: ALT2 mux port: RX_DATA0 of instance: SAI6

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD5 field descriptions (continued)

Field	Description
011	ALT3_SAI1_RX_SYNC — Select mux mode: ALT3 mux port: RX_SYNC of instance: SAI1
100	ALT4_CORESIGHT_TRACE5 — Select mux mode: ALT4 mux port: TRACE5 of instance: CORESIGHT
101	ALT5_GPIO4_IO07 — Select mux mode: ALT5 mux port: IO07 of instance: GPIO4
110	ALT6_SRC_BOOT_CFG5 — Select mux mode: ALT6 mux port: BOOT_CFG5 of instance: SRC

8.2.5.91 SW_MUX_CTL_PAD_SAI1_RXD6 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD6)**SW_MUX_CTL Register**

Address: 3033_0000h base + 17Ch offset = 3033_017Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION				Reserved	MUX_MODE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD6 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD6 field descriptions (continued)

Field	Description
0	SION_DISABLED — Input Path of pad SAI1_RXD6 is determined by functionality
1	SION_ENABLED — Force Input Path of pad SAI1_RXD6
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 6 iomux modes to be used for pad: SAI1_RXD6 000 ALT0_SAI1_RX_DATA6 — Select mux mode: ALT0 mux port: RX_DATA6 of instance: SAI1 001 ALT1_SAI6_TX_SYNC — Select mux mode: ALT1 mux port: TX_SYNC of instance: SAI6 010 ALT2_SAI6_RX_SYNC — Select mux mode: ALT2 mux port: RX_SYNC of instance: SAI6 100 ALT4_CORESIGHT_TRACE6 — Select mux mode: ALT4 mux port: TRACE6 of instance: CORESIGHT 101 ALT5_GPIO4_IO08 — Select mux mode: ALT5 mux port: IO08 of instance: GPIO4 110 ALT6_SRC_BOOT_CFG6 — Select mux mode: ALT6 mux port: BOOT_CFG6 of instance: SRC

8.2.5.92 SW_MUX_CTL_PAD_SAI1_RXD7 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD7)

SW_MUX_CTL Register

Address: 3033_0000h base + 180h offset = 3033_0180h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												SION	Reserved	MUX_MODE	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI1_RXD7 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_RXD7 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_RXD7
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 7 iomux modes to be used for pad: SAI1_RXD7 000 ALT0_SAI1_RX_DATA7 — Select mux mode: ALT0 mux port: RX_DATA7 of instance: SAI1 001 ALT1_SAI6_MCLK — Select mux mode: ALT1 mux port: MCLK of instance: SAI6 010 ALT2_SAI1_TX_SYNC — Select mux mode: ALT2 mux port: TX_SYNC of instance: SAI1 011 ALT3_SAI1_TX_DATA4 — Select mux mode: ALT3 mux port: TX_DATA4 of instance: SAI1 100 ALT4_CORESIGHT_TRACE7 — Select mux mode: ALT4 mux port: TRACE7 of instance: CORESIGHT 101 ALT5_GPIO4_IO09 — Select mux mode: ALT5 mux port: IO09 of instance: GPIO4 110 ALT6_SRC_BOOT_CFG7 — Select mux mode: ALT6 mux port: BOOT_CFG7 of instance: SRC

8.2.5.93 SW_MUX_CTL_PAD_SAI1_TXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXFS)

SW_MUX_CTL Register

Address: 3033_0000h base + 184h offset = 3033_0184h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI1_TXFS field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_TXFS is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_TXFS
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: SAI1_TXFS 000 ALT0_SAI1_TX_SYNC — Select mux mode: ALT0 mux port: TX_SYNC of instance: SAI1 001 ALT1_SAI5_TX_SYNC — Select mux mode: ALT1 mux port: TX_SYNC of instance: SAI5

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI1_TXFS field descriptions (continued)

Field	Description
100	ALT4_CORESIGHT_EVENTO — Select mux mode: ALT4 mux port: EVENTO of instance: CORESIGHT
101	ALT5_GPIO4_IO10 — Select mux mode: ALT5 mux port: IO10 of instance: GPIO4

8.2.5.94 SW_MUX_CTL_PAD_SAI1_TXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXC)**SW_MUX_CTL Register**

Address: 3033_0000h base + 188h offset = 3033_0188h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved																
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved											SION		Reserved	MUX_MODE		
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	

IOMUXC_SW_MUX_CTL_PAD_SAI1_TXC field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_TXC is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_TXC

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI1_TXC field descriptions (continued)

Field	Description
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: SAI1_TXC 000 ALT0_SAI1_TX_BCLK — Select mux mode: ALT0 mux port: TX_BCLK of instance: SAI1 001 ALT1_SAI5_TX_BCLK — Select mux mode: ALT1 mux port: TX_BCLK of instance: SAI5 100 ALT4_CORESIGHT_EVENTI — Select mux mode: ALT4 mux port: EVENTI of instance: CORESIGHT 101 ALT5_GPIO4_IO11 — Select mux mode: ALT5 mux port: IO11 of instance: GPIO4

8.2.5.95 SW_MUX_CTL_PAD_SAI1_TXD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD0)

SW_MUX_CTL Register

Address: 3033_0000h base + 18Ch offset = 3033_018Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												SION	Reserved	MUX_MODE	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD0 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD0 field descriptions (continued)

Field	Description
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_TXD0 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_TXD0
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 5 iomux modes to be used for pad: SAI1_TXD0 000 ALT0_SAI1_TX_DATA0 — Select mux mode: ALT0 mux port: TX_DATA0 of instance: SAI1 001 ALT1_SAI5_TX_DATA0 — Select mux mode: ALT1 mux port: TX_DATA0 of instance: SAI5 100 ALT4_CORESIGHT_TRACE8 — Select mux mode: ALT4 mux port: TRACE8 of instance: CORESIGHT 101 ALT5_GPIO4_IO12 — Select mux mode: ALT5 mux port: IO12 of instance: GPIO4 110 ALT6_SRC_BOOT_CFG8 — Select mux mode: ALT6 mux port: BOOT_CFG8 of instance: SRC

8.2.5.96 SW_MUX_CTL_PAD_SAI1_TXD1 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD1)**SW_MUX_CTL Register**

Address: 3033_0000h base + 190h offset = 3033_0190h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												SION	Reserved	MUX_MODE	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

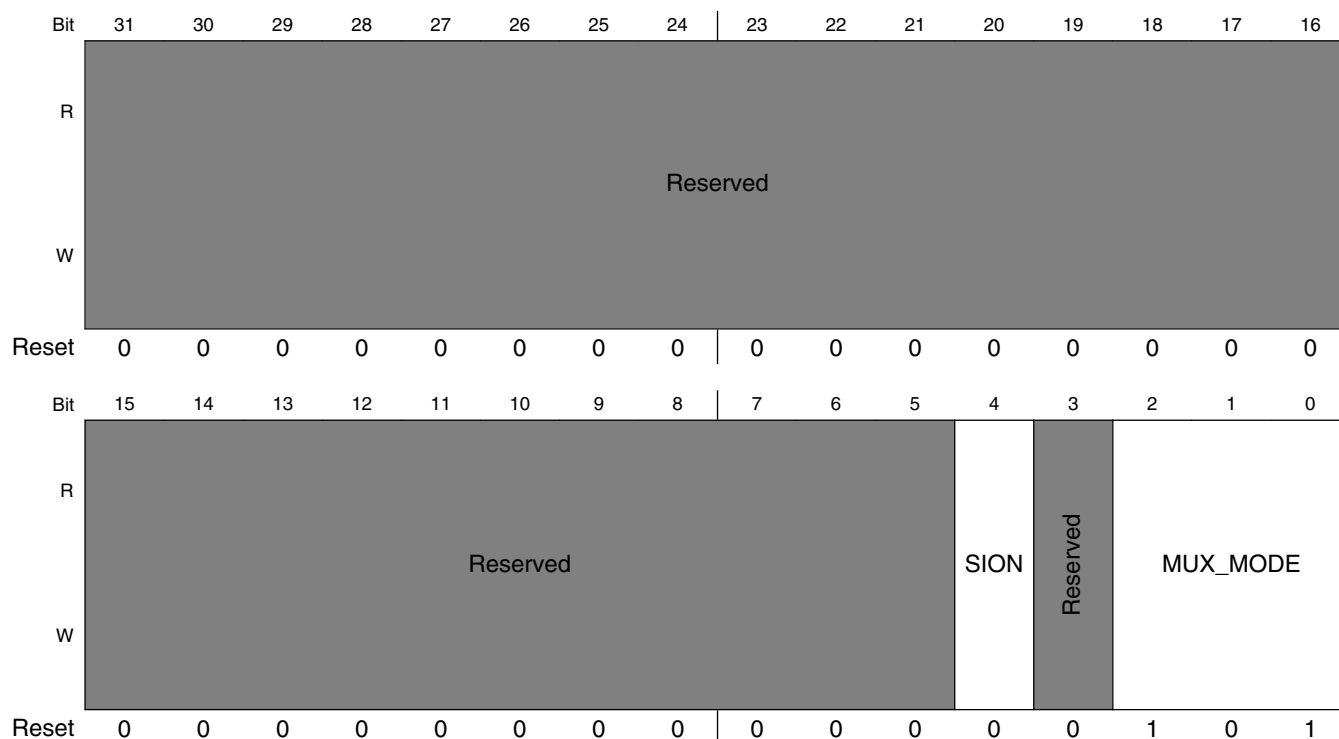
IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD1 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_TXD1 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_TXD1
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 5 iomux modes to be used for pad: SAI1_TXD1 000 ALT0_SAI1_TX_DATA1 — Select mux mode: ALT0 mux port: TX_DATA1 of instance: SAI1 001 ALT1_SAI5_TX_DATA1 — Select mux mode: ALT1 mux port: TX_DATA1 of instance: SAI5 100 ALT4_CORESIGHT_TRACE9 — Select mux mode: ALT4 mux port: TRACE9 of instance: CORESIGHT 101 ALT5_GPIO4_IO13 — Select mux mode: ALT5 mux port: IO13 of instance: GPIO4 110 ALT6_SRC_BOOT_CFG9 — Select mux mode: ALT6 mux port: BOOT_CFG9 of instance: SRC

8.2.5.97 SW_MUX_CTL_PAD_SAI1_TXD2 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD2)

SW_MUX_CTL Register

Address: 3033_0000h base + 194h offset = 3033_0194h



IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD2 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_TXD2 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_TXD2
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 5 iomux modes to be used for pad: SAI1_TXD2 000 ALT0_SAI1_TX_DATA2 — Select mux mode: ALT0 mux port: TX_DATA2 of instance: SAI1 001 ALT1_SAI5_TX_DATA2 — Select mux mode: ALT1 mux port: TX_DATA2 of instance: SAI5

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD2 field descriptions (continued)

Field	Description
100	ALT4_CORESIGHT_TRACE10 — Select mux mode: ALT4 mux port: TRACE10 of instance: CORESIGHT
101	ALT5_GPIO4_IO14 — Select mux mode: ALT5 mux port: IO14 of instance: GPIO4
110	ALT6_SRC_BOOT_CFG10 — Select mux mode: ALT6 mux port: BOOT_CFG10 of instance: SRC

8.2.5.98 SW_MUX_CTL_PAD_SAI1_TXD3 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD3)

SW_MUX_CTL Register

Address: 3033_0000h base + 198h offset = 3033_0198h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD3 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_TXD3 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_TXD3

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD3 field descriptions (continued)

Field	Description
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 5 iomux modes to be used for pad: SAI1_TXD3 000 ALT0_SAI1_TX_DATA3 — Select mux mode: ALT0 mux port: TX_DATA3 of instance: SAI1 001 ALT1_SAI5_TX_DATA3 — Select mux mode: ALT1 mux port: TX_DATA3 of instance: SAI5 100 ALT4_CORESIGHT_TRACE11 — Select mux mode: ALT4 mux port: TRACE11 of instance: CORESIGHT 101 ALT5_GPIO4_IO15 — Select mux mode: ALT5 mux port: IO15 of instance: GPIO4 110 ALT6_SRC_BOOT_CFG11 — Select mux mode: ALT6 mux port: BOOT_CFG11 of instance: SRC

8.2.5.99 SW_MUX_CTL_PAD_SAI1_TXD4 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD4)**SW_MUX_CTL Register**

Address: 3033_0000h base + 19Ch offset = 3033_019Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved										SION	Reserved	MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

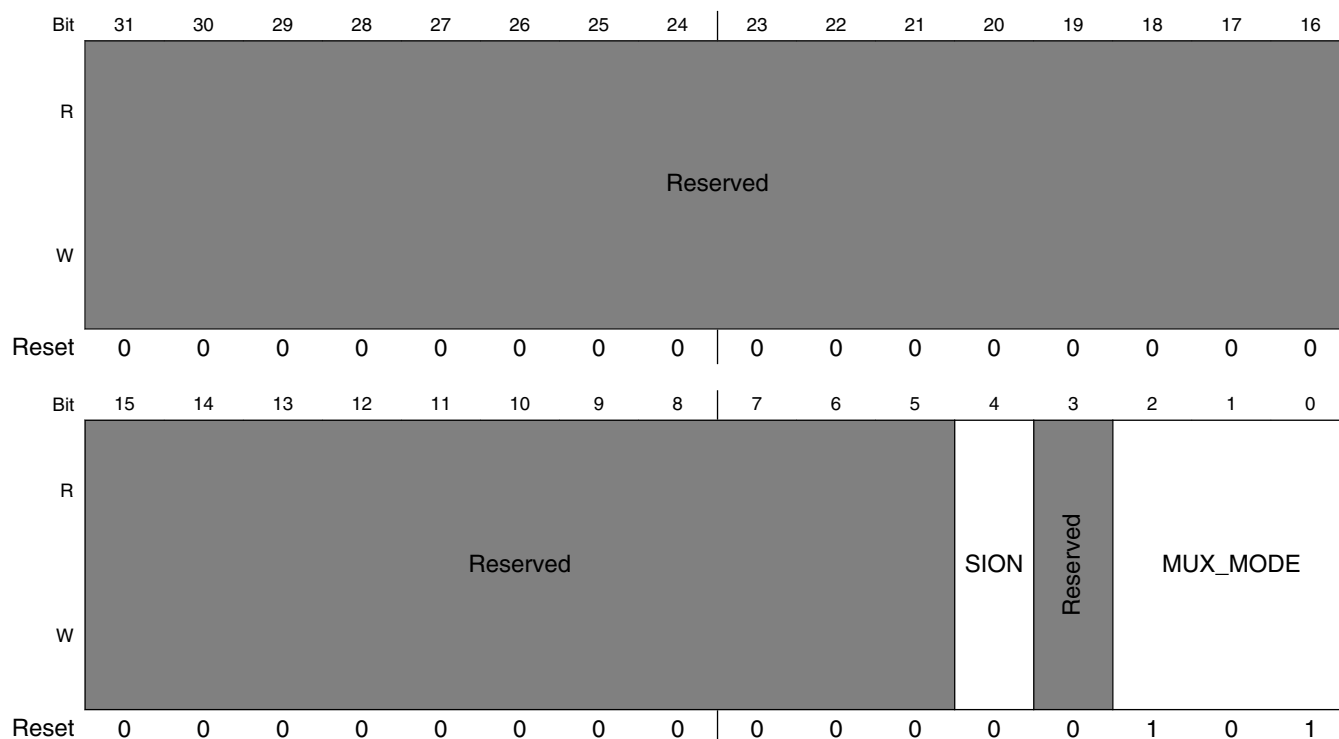
IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD4 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_TXD4 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_TXD4
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 6 iomux modes to be used for pad: SAI1_TXD4 000 ALT0_SAI1_TX_DATA4 — Select mux mode: ALT0 mux port: TX_DATA4 of instance: SAI1 001 ALT1_SAI6_RX_BCLK — Select mux mode: ALT1 mux port: RX_BCLK of instance: SAI6 010 ALT2_SAI6_TX_BCLK — Select mux mode: ALT2 mux port: TX_BCLK of instance: SAI6 100 ALT4_CORESIGHT_TRACE12 — Select mux mode: ALT4 mux port: TRACE12 of instance: CORESIGHT 101 ALT5_GPIO4_IO16 — Select mux mode: ALT5 mux port: IO16 of instance: GPIO4 110 ALT6_SRC_BOOT_CFG12 — Select mux mode: ALT6 mux port: BOOT_CFG12 of instance: SRC

8.2.5.100 SW_MUX_CTL_PAD_SAI1_TXD5 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD5)

SW_MUX_CTL Register

Address: 3033_0000h base + 1A0h offset = 3033_01A0h



IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD5 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_TXD5 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_TXD5
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 6 iomux modes to be used for pad: SAI1_TXD5 000 ALT0_SAI1_TX_DATA5 — Select mux mode: ALT0 mux port: TX_DATA5 of instance: SAI1 001 ALT1_SAI6_RX_DATA0 — Select mux mode: ALT1 mux port: RX_DATA0 of instance: SAI6 010 ALT2_SAI6_TX_DATA0 — Select mux mode: ALT2 mux port: TX_DATA0 of instance: SAI6

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD5 field descriptions (continued)

Field	Description
100	ALT4_CORESIGHT_TRACE13 — Select mux mode: ALT4 mux port: TRACE13 of instance: CORESIGHT
101	ALT5_GPIO4_IO17 — Select mux mode: ALT5 mux port: IO17 of instance: GPIO4
110	ALT6_SRC_BOOT_CFG13 — Select mux mode: ALT6 mux port: BOOT_CFG13 of instance: SRC

8.2.5.101 SW_MUX_CTL_PAD_SAI1_TXD6 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD6)

SW_MUX_CTL Register

Address: 3033_0000h base + 1A4h offset = 3033_01A4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD6 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_TXD6 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_TXD6

Table continues on the next page...

IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD6 field descriptions (continued)

Field	Description
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 6 iomux modes to be used for pad: SAI1_TXD6 000 ALT0_SAI1_TX_DATA6 — Select mux mode: ALT0 mux port: TX_DATA6 of instance: SAI1 001 ALT1_SAI6_RX_SYNC — Select mux mode: ALT1 mux port: RX_SYNC of instance: SAI6 010 ALT2_SAI6_TX_SYNC — Select mux mode: ALT2 mux port: TX_SYNC of instance: SAI6 100 ALT4_CORESIGHT_TRACE14 — Select mux mode: ALT4 mux port: TRACE14 of instance: CORESIGHT 101 ALT5_GPIO4_IO18 — Select mux mode: ALT5 mux port: IO18 of instance: GPIO4 110 ALT6_SRC_BOOT_CFG14 — Select mux mode: ALT6 mux port: BOOT_CFG14 of instance: SRC

8.2.5.102 SW_MUX_CTL_PAD_SAI1_TXD7 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD7)**SW_MUX_CTL Register**

Address: 3033_0000h base + 1A8h offset = 3033_01A8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												SION	Reserved	MUX_MODE	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

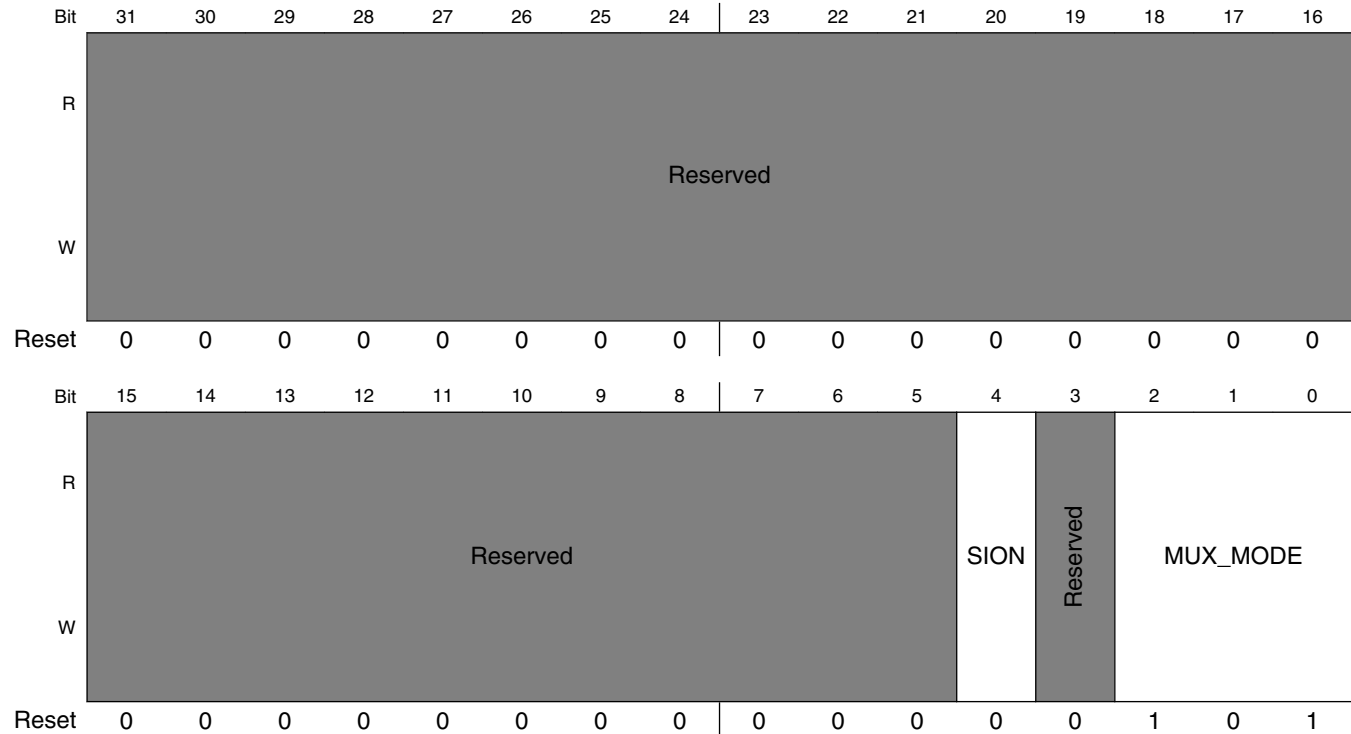
IOMUXC_SW_MUX_CTL_PAD_SAI1_TXD7 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_TXD7 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_TXD7
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 5 iomux modes to be used for pad: SAI1_TXD7 000 ALT0_SAI1_TX_DATA7 — Select mux mode: ALT0 mux port: TX_DATA7 of instance: SAI1 001 ALT1_SAI6_MCLK — Select mux mode: ALT1 mux port: MCLK of instance: SAI6 100 ALT4_CORESIGHT_TRACE15 — Select mux mode: ALT4 mux port: TRACE15 of instance: CORESIGHT 101 ALT5_GPIO4_IO19 — Select mux mode: ALT5 mux port: IO19 of instance: GPIO4 110 ALT6_SRC_BOOT_CFG15 — Select mux mode: ALT6 mux port: BOOT_CFG15 of instance: SRC

8.2.5.103 SW_MUX_CTL_PAD_SAI1_MCLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI1_MCLK)

SW_MUX_CTL Register

Address: 3033_0000h base + 1ACh offset = 3033_01ACh



IOMUXC_SW_MUX_CTL_PAD_SAI1_MCLK field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI1_MCLK is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI1_MCLK
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: SAI1_MCLK 000 ALT0_SAI1_MCLK — Select mux mode: ALT0 mux port: MCLK of instance: SAI1 001 ALT1_SAI5_MCLK — Select mux mode: ALT1 mux port: MCLK of instance: SAI5 010 ALT2_SAI1_TX_BCLK — Select mux mode: ALT2 mux port: TX_BCLK of instance: SAI1 101 ALT5_GPIO4_IO20 — Select mux mode: ALT5 mux port: IO20 of instance: GPIO4

8.2.5.104 SW_MUX_CTL_PAD_SAI2_RXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_RXFS)

SW_MUX_CTL Register

Address: 3033_0000h base + 1B0h offset = 3033_01B0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

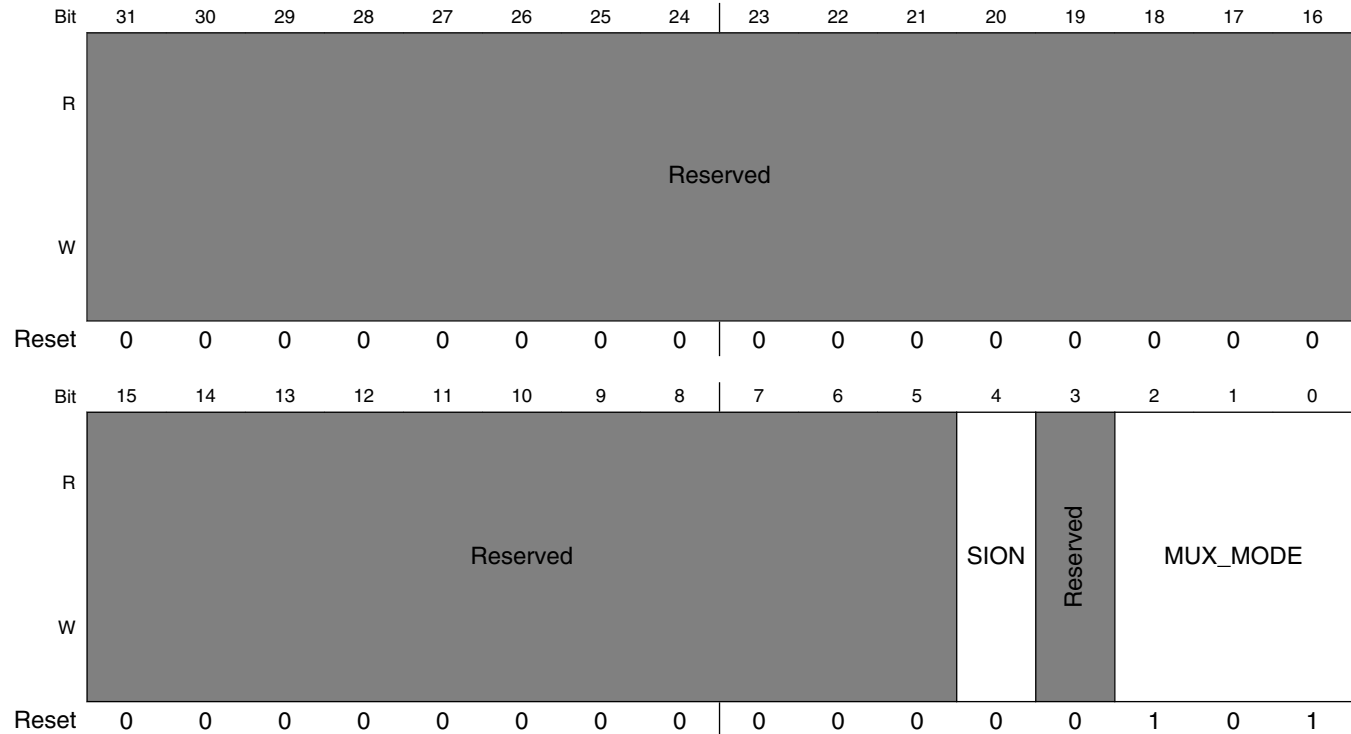
IOMUXC_SW_MUX_CTL_PAD_SAI2_RXFS field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI2_RXFS is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI2_RXFS
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: SAI2_RXFS 000 ALT0_SAI2_RX_SYNC — Select mux mode: ALT0 mux port: RX_SYNC of instance: SAI2 001 ALT1_SAI5_TX_SYNC — Select mux mode: ALT1 mux port: TX_SYNC of instance: SAI5 101 ALT5_GPIO4_IO21 — Select mux mode: ALT5 mux port: IO21 of instance: GPIO4

8.2.5.105 SW_MUX_CTL_PAD_SAI2_RXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_RXC)

SW_MUX_CTL Register

Address: 3033_0000h base + 1B4h offset = 3033_01B4h



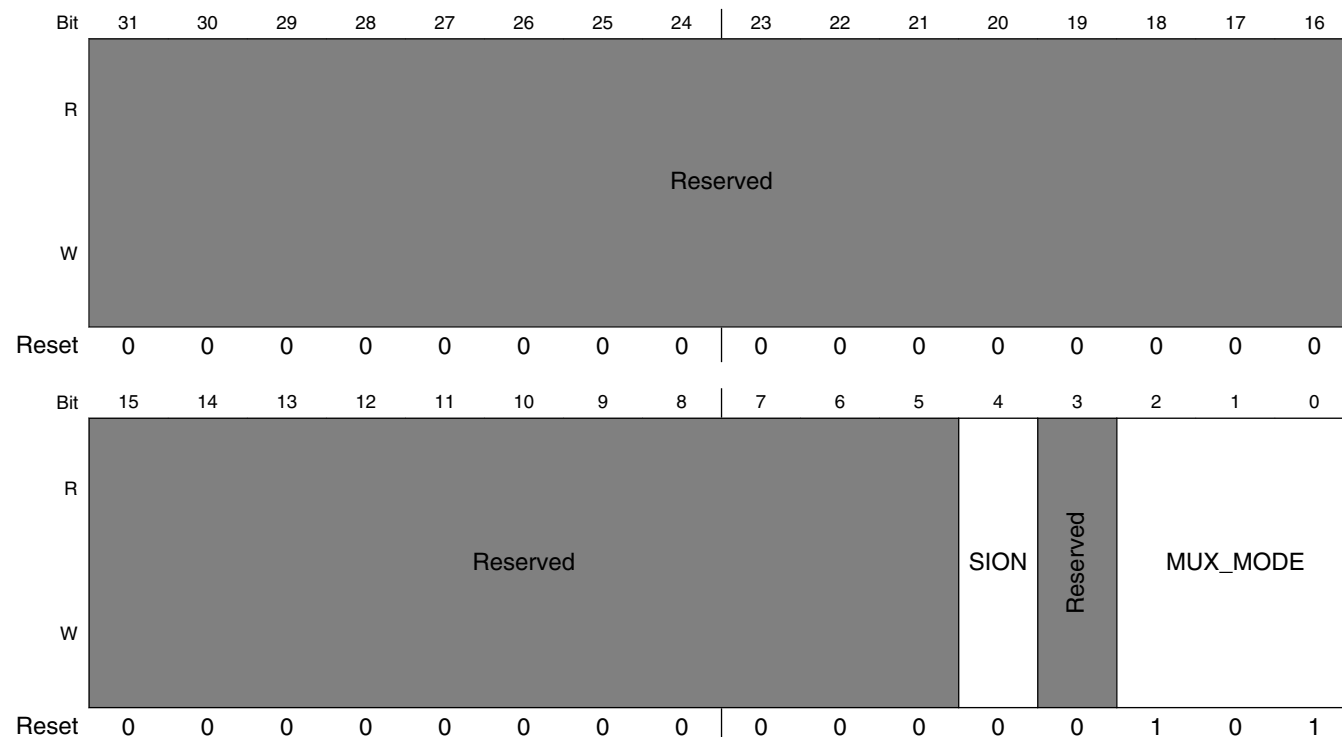
IOMUXC_SW_MUX_CTL_PAD_SAI2_RXC field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI2_RXC is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI2_RXC
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: SAI2_RXC 000 ALT0_SAI2_RX_BCLK — Select mux mode: ALT0 mux port: RX_BCLK of instance: SAI2 001 ALT1_SAI5_TX_BCLK — Select mux mode: ALT1 mux port: TX_BCLK of instance: SAI5 101 ALT5_GPIO4_IO22 — Select mux mode: ALT5 mux port: IO22 of instance: GPIO4

8.2.5.106 SW_MUX_CTL_PAD_SAI2_RXD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_RXD0)

SW_MUX_CTL Register

Address: 3033_0000h base + 1B8h offset = 3033_01B8h



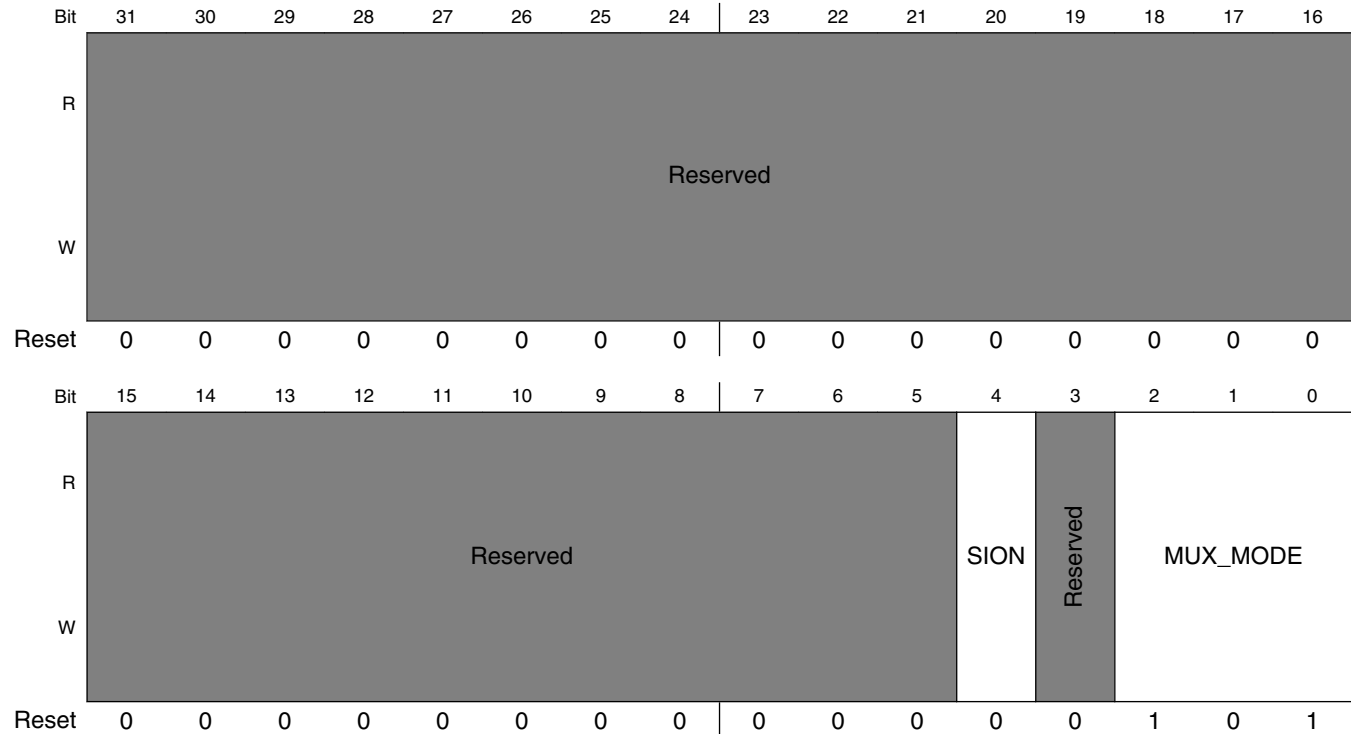
IOMUXC_SW_MUX_CTL_PAD_SAI2_RXD0 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI2_RXD0 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI2_RXD0
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: SAI2_RXD0 000 ALT0_SAI2_RX_DATA0 — Select mux mode: ALT0 mux port: RX_DATA0 of instance: SAI2 001 ALT1_SAI5_TX_DATA0 — Select mux mode: ALT1 mux port: TX_DATA0 of instance: SAI5 101 ALT5_GPIO4_IO23 — Select mux mode: ALT5 mux port: IO23 of instance: GPIO4

8.2.5.107 SW_MUX_CTL_PAD_SAI2_TXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_TXFS)

SW_MUX_CTL Register

Address: 3033_0000h base + 1BCh offset = 3033_01BCh



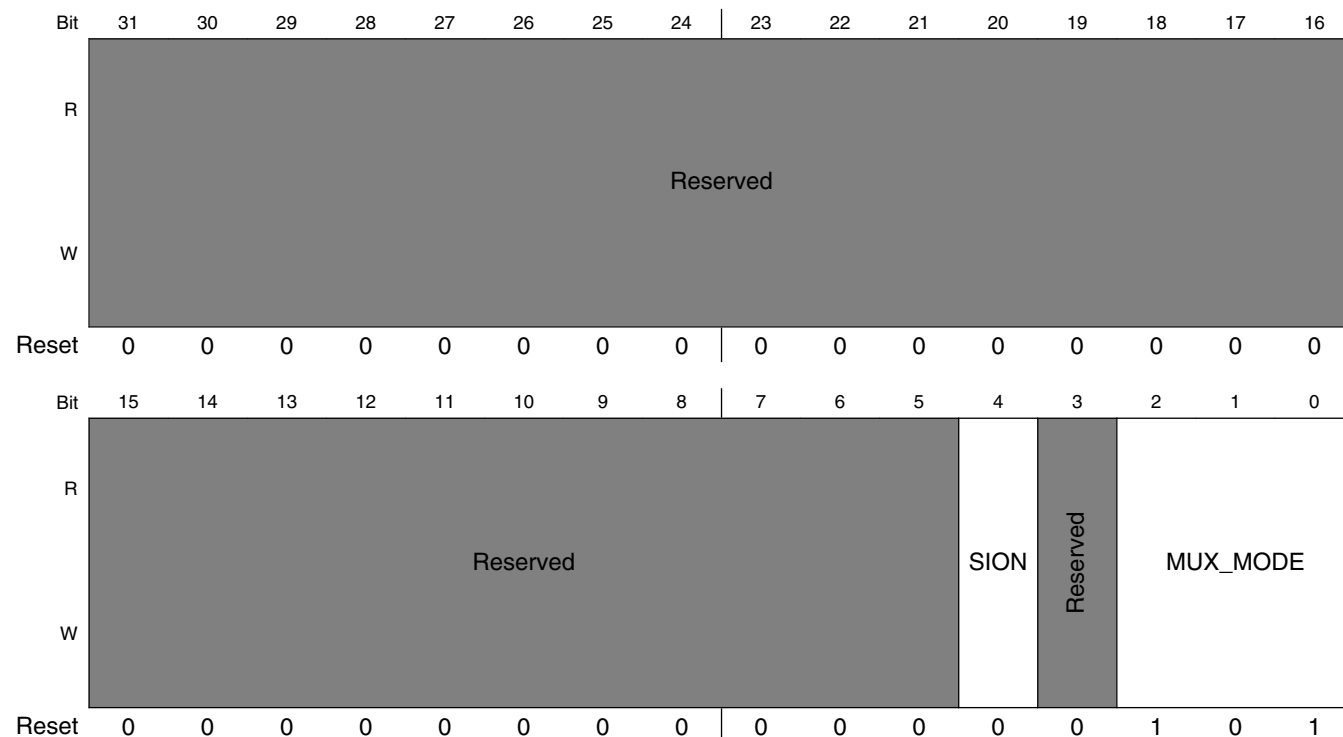
IOMUXC_SW_MUX_CTL_PAD_SAI2_TXFS field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI2_TXFS is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI2_TXFS
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: SAI2_TXFS 000 ALT0_SAI2_TX_SYNC — Select mux mode: ALT0 mux port: TX_SYNC of instance: SAI2 001 ALT1_SAI5_TX_DATA1 — Select mux mode: ALT1 mux port: TX_DATA1 of instance: SAI5 101 ALT5_GPIO4_IO24 — Select mux mode: ALT5 mux port: IO24 of instance: GPIO4

8.2.5.108 SW_MUX_CTL_PAD_SAI2_TXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_TXC)

SW_MUX_CTL Register

Address: 3033_0000h base + 1C0h offset = 3033_01C0h



IOMUXC_SW_MUX_CTL_PAD_SAI2_TXC field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI2_TXC is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI2_TXC
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: SAI2_TXC 000 ALT0_SAI2_TX_BCLK — Select mux mode: ALT0 mux port: TX_BCLK of instance: SAI2 001 ALT1_SAI5_TX_DATA2 — Select mux mode: ALT1 mux port: TX_DATA2 of instance: SAI5 101 ALT5_GPIO4_IO25 — Select mux mode: ALT5 mux port: IO25 of instance: GPIO4

8.2.5.109 SW_MUX_CTL_PAD_SAI2_TXD0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_TXD0)

SW_MUX_CTL Register

Address: 3033_0000h base + 1C4h offset = 3033_01C4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16		
R	Reserved																	
W																		
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
R	Reserved												SION		Reserved		MUX_MODE	
W																		
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1		

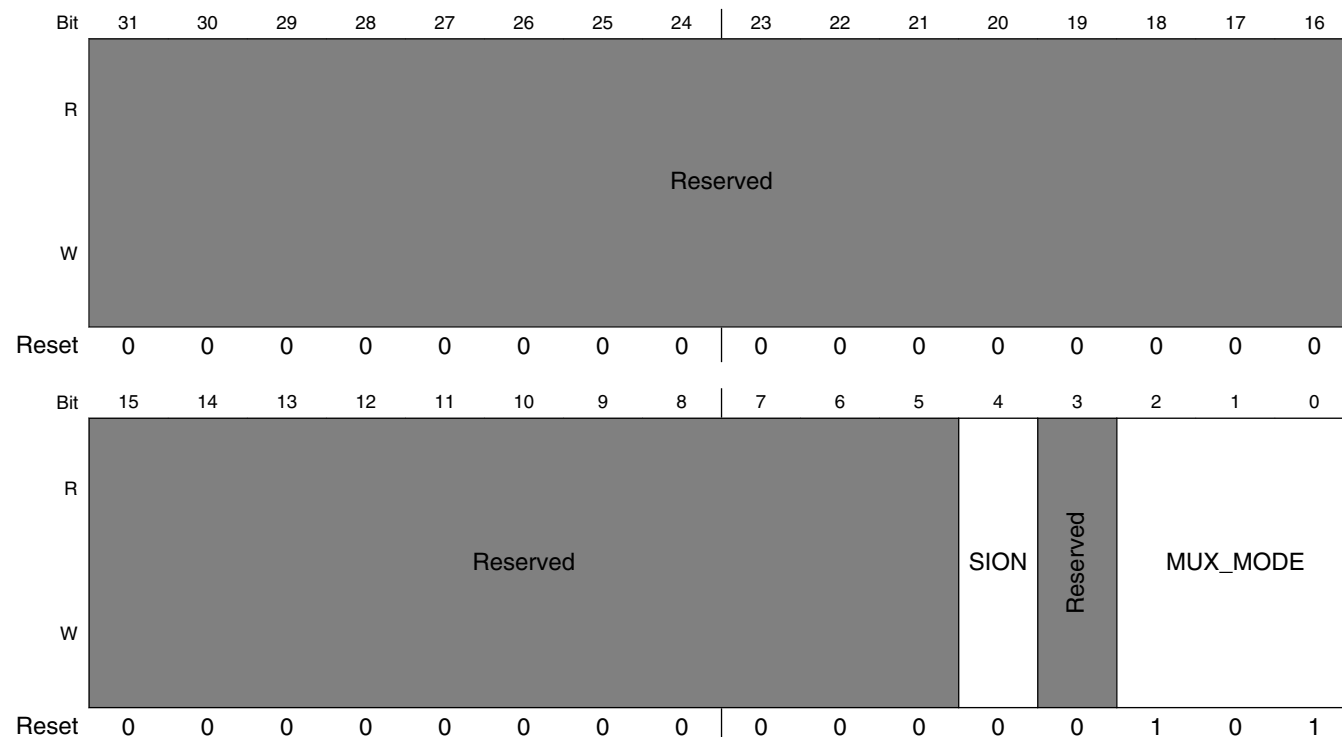
IOMUXC_SW_MUX_CTL_PAD_SAI2_TXD0 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI2_TXD0 is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI2_TXD0
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: SAI2_TXD0 000 ALT0_SAI2_TX_DATA0 — Select mux mode: ALT0 mux port: TX_DATA0 of instance: SAI2 001 ALT1_SAI5_TX_DATA3 — Select mux mode: ALT1 mux port: TX_DATA3 of instance: SAI5 101 ALT5_GPIO4_IO26 — Select mux mode: ALT5 mux port: IO26 of instance: GPIO4

8.2.5.110 SW_MUX_CTL_PAD_SAI2_MCLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI2_MCLK)

SW_MUX_CTL Register

Address: 3033_0000h base + 1C8h offset = 3033_01C8h



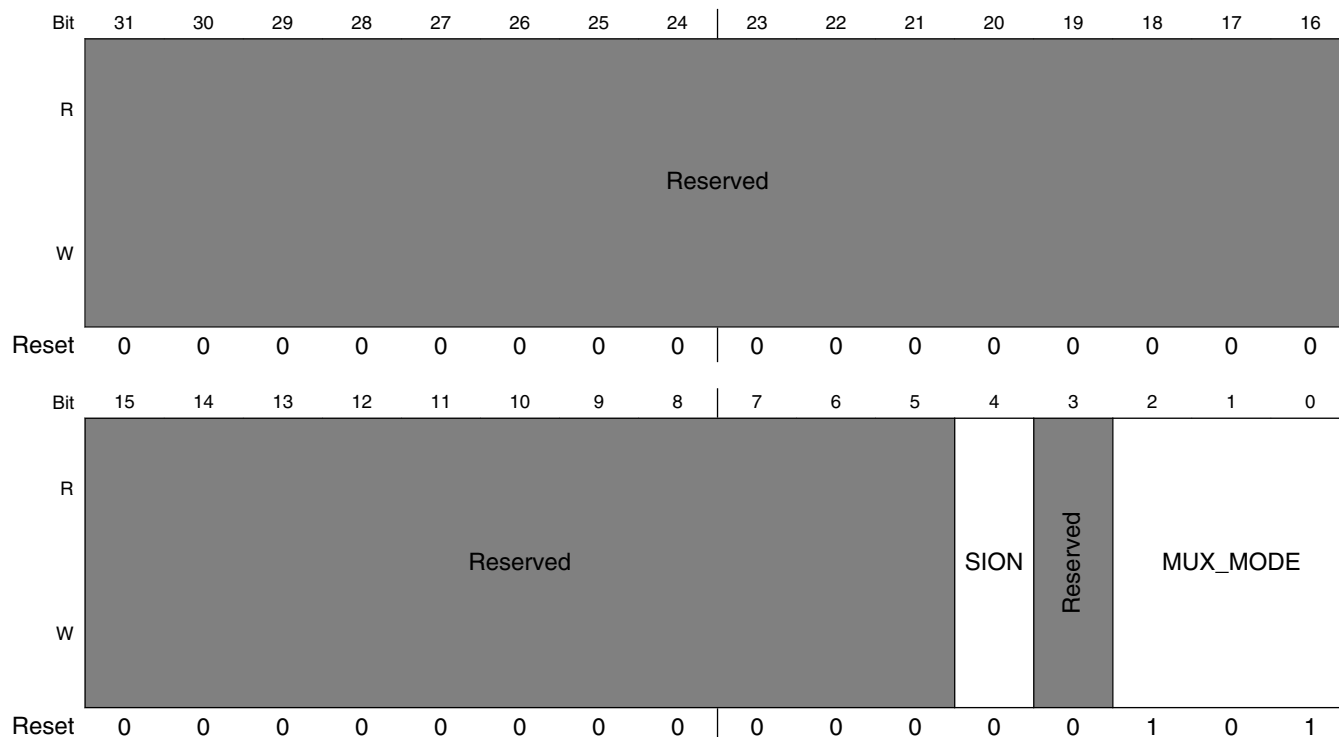
IOMUXC_SW_MUX_CTL_PAD_SAI2_MCLK field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI2_MCLK is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI2_MCLK
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: SAI2_MCLK 000 ALT0_SAI2_MCLK — Select mux mode: ALT0 mux port: MCLK of instance: SAI2 001 ALT1_SAI5_MCLK — Select mux mode: ALT1 mux port: MCLK of instance: SAI5 101 ALT5_GPIO4_IO27 — Select mux mode: ALT5 mux port: IO27 of instance: GPIO4

8.2.5.111 SW_MUX_CTL_PAD_SAI3_RXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_RXFS)

SW_MUX_CTL Register

Address: 3033_0000h base + 1CCh offset = 3033_01CCh



IOMUXC_SW_MUX_CTL_PAD_SAI3_RXFS field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI3_RXFS is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI3_RXFS
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: SAI3_RXFS 000 ALT0_SAI3_RX_SYNC — Select mux mode: ALT0 mux port: RX_SYNC of instance: SAI3 001 ALT1_GPT1_CAPTURE1 — Select mux mode: ALT1 mux port: CAPTURE1 of instance: GPT1 010 ALT2_SAI5_RX_SYNC — Select mux mode: ALT2 mux port: RX_SYNC of instance: SAI5 101 ALT5_GPIO4_IO28 — Select mux mode: ALT5 mux port: IO28 of instance: GPIO4

8.2.5.112 SW_MUX_CTL_PAD_SAI3_RXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_RXC)

SW_MUX_CTL Register

Address: 3033_0000h base + 1D0h offset = 3033_01D0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

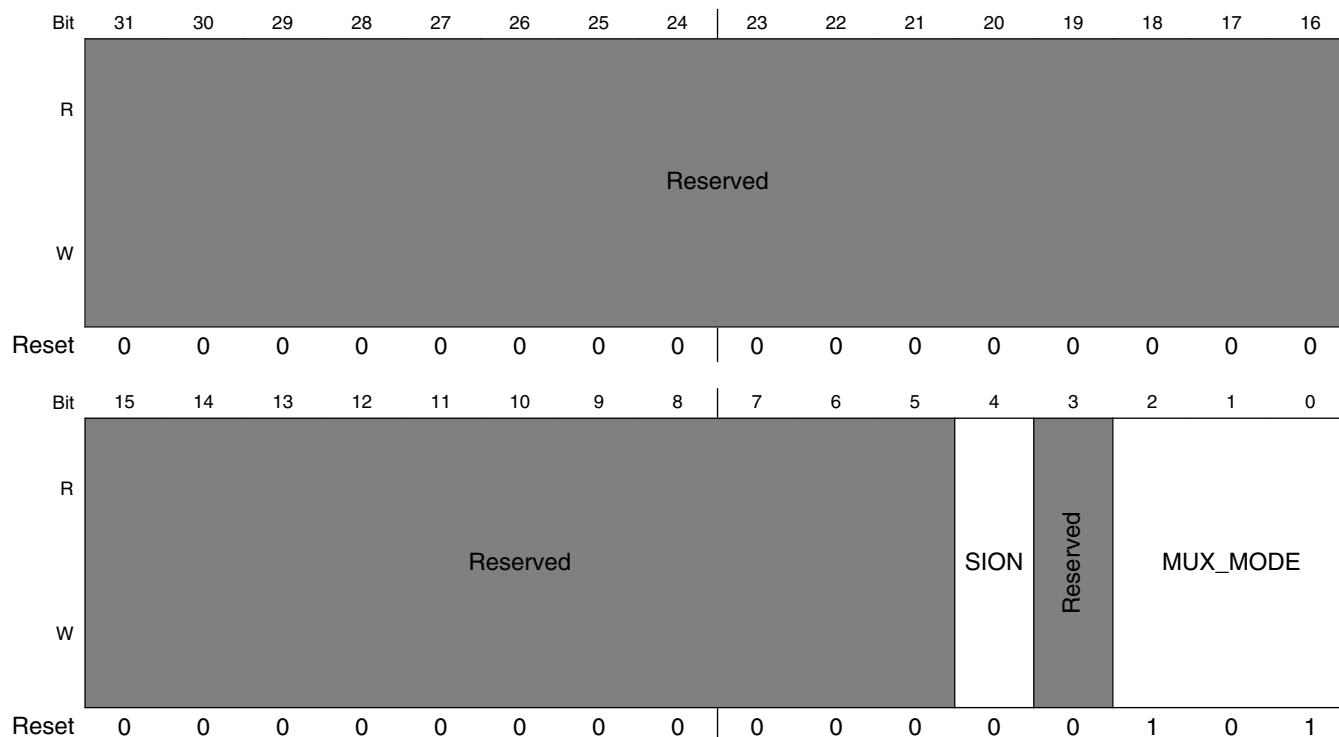
IOMUXC_SW_MUX_CTL_PAD_SAI3_RXC field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI3_RXC is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI3_RXC
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: SAI3_RXC 000 ALT0_SAI3_RX_BCLK — Select mux mode: ALT0 mux port: RX_BCLK of instance: SAI3 001 ALT1_GPT1_CAPTURE2 — Select mux mode: ALT1 mux port: CAPTURE2 of instance: GPT1 010 ALT2_SAI5_RX_BCLK — Select mux mode: ALT2 mux port: RX_BCLK of instance: SAI5 101 ALT5_GPIO4_IO29 — Select mux mode: ALT5 mux port: IO29 of instance: GPIO4

8.2.5.113 SW_MUX_CTL_PAD_SAI3_RXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_RXD)

SW_MUX_CTL Register

Address: 3033_0000h base + 1D4h offset = 3033_01D4h



IOMUXC_SW_MUX_CTL_PAD_SAI3_RXD field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI3_RXD is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI3_RXD
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: SAI3_RXD 000 ALT0_SAI3_RX_DATA0 — Select mux mode: ALT0 mux port: RX_DATA0 of instance: SAI3 001 ALT1_GPT1_COMPARE1 — Select mux mode: ALT1 mux port: COMPARE1 of instance: GPT1 010 ALT2_SAI5_RX_DATA0 — Select mux mode: ALT2 mux port: RX_DATA0 of instance: SAI5 101 ALT5_GPIO4_IO30 — Select mux mode: ALT5 mux port: IO30 of instance: GPIO4

8.2.5.114 SW_MUX_CTL_PAD_SAI3_TXFS SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_TXFS)

SW_MUX_CTL Register

Address: 3033_0000h base + 1D8h offset = 3033_01D8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

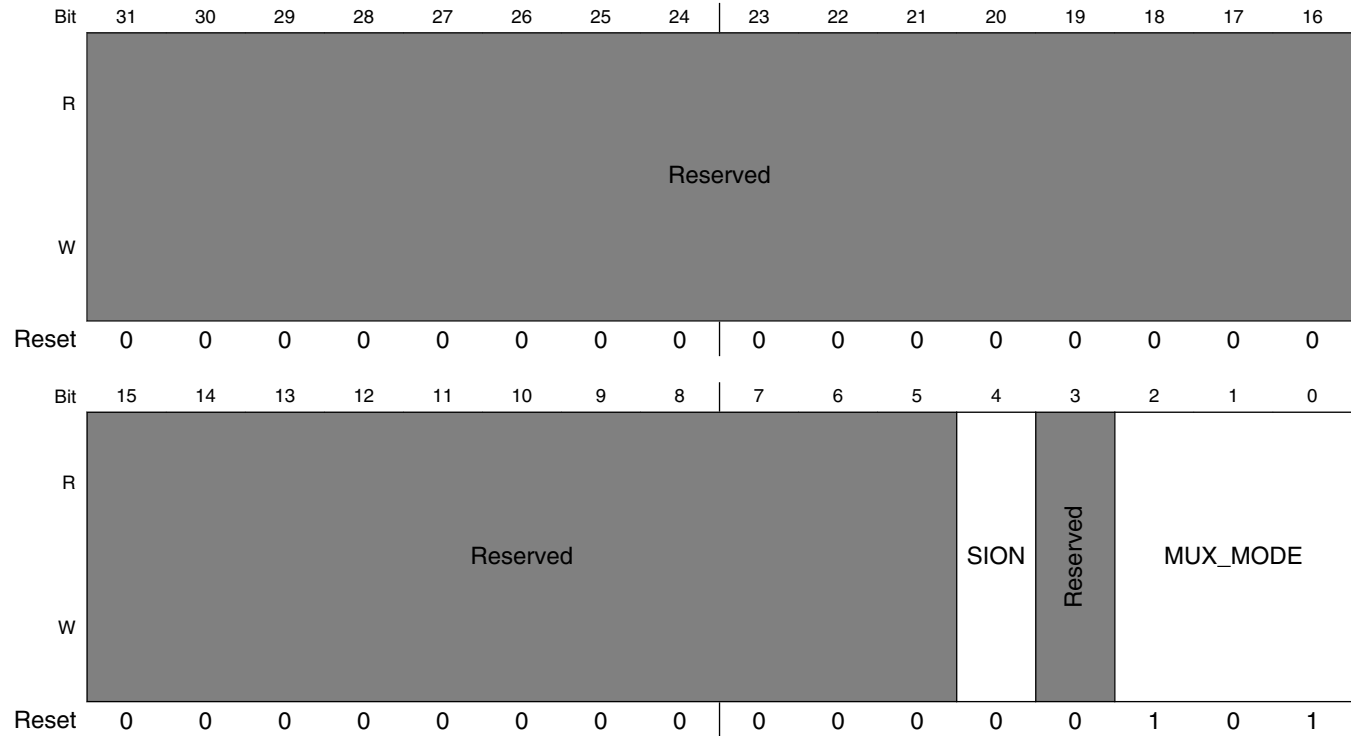
IOMUXC_SW_MUX_CTL_PAD_SAI3_TXFS field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI3_TXFS is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI3_TXFS
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: SAI3_TXFS 000 ALT0_SAI3_TX_SYNC — Select mux mode: ALT0 mux port: TX_SYNC of instance: SAI3 001 ALT1_GPT1_CLK — Select mux mode: ALT1 mux port: CLK of instance: GPT1 010 ALT2_SAI5_RX_DATA1 — Select mux mode: ALT2 mux port: RX_DATA1 of instance: SAI5 101 ALT5_GPIO4_IO31 — Select mux mode: ALT5 mux port: IO31 of instance: GPIO4

8.2.5.115 SW_MUX_CTL_PAD_SAI3_TXC SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_TXC)

SW_MUX_CTL Register

Address: 3033_0000h base + 1DCh offset = 3033_01DCh



IOMUXC_SW_MUX_CTL_PAD_SAI3_TXC field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI3_TXC is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI3_TXC
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: SAI3_TXC 000 ALT0_SAI3_TX_BCLK — Select mux mode: ALT0 mux port: TX_BCLK of instance: SAI3 001 ALT1_GPT1_COMPARE2 — Select mux mode: ALT1 mux port: COMPARE2 of instance: GPT1 010 ALT2_SAI5_RX_DATA2 — Select mux mode: ALT2 mux port: RX_DATA2 of instance: SAI5 101 ALT5_GPIO5_IO00 — Select mux mode: ALT5 mux port: IO00 of instance: GPIO5

8.2.5.116 SW_MUX_CTL_PAD_SAI3_TXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_TXD)

SW_MUX_CTL Register

Address: 3033_0000h base + 1E0h offset = 3033_01E0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION				Reserved	MUX_MODE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI3_TXD field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI3_TXD is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI3_TXD
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: SAI3_TXD 000 ALT0_SAI3_TX_DATA0 — Select mux mode: ALT0 mux port: TX_DATA0 of instance: SAI3 001 ALT1_GPT1_COMPARE3 — Select mux mode: ALT1 mux port: COMPARE3 of instance: GPT1 010 ALT2_SAI5_RX_DATA3 — Select mux mode: ALT2 mux port: RX_DATA3 of instance: SAI5 101 ALT5_GPIO5_IO01 — Select mux mode: ALT5 mux port: IO01 of instance: GPIO5

8.2.5.117 SW_MUX_CTL_PAD_SAI3_MCLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SAI3_MCLK)

SW_MUX_CTL Register

Address: 3033_0000h base + 1E4h offset = 3033_01E4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

IOMUXC_SW_MUX_CTL_PAD_SAI3_MCLK field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SAI3_MCLK is determined by functionality 1 SION_ENABLED — Force Input Path of pad SAI3_MCLK
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: SAI3_MCLK 000 ALT0_SAI3_MCLK — Select mux mode: ALT0 mux port: MCLK of instance: SAI3 001 ALT1_PWM4_OUT — Select mux mode: ALT1 mux port: OUT of instance: PWM4 010 ALT2_SAI5_MCLK — Select mux mode: ALT2 mux port: MCLK of instance: SAI5 101 ALT5_GPIO5_IO02 — Select mux mode: ALT5 mux port: IO02 of instance: GPIO5

8.2.5.118 SW_MUX_CTL_PAD_SPDIF_TX SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SPDIF_TX)

SW_MUX_CTL Register

Address: 3033_0000h base + 1E8h offset = 3033_01E8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												SION	Reserved	MUX_MODE	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

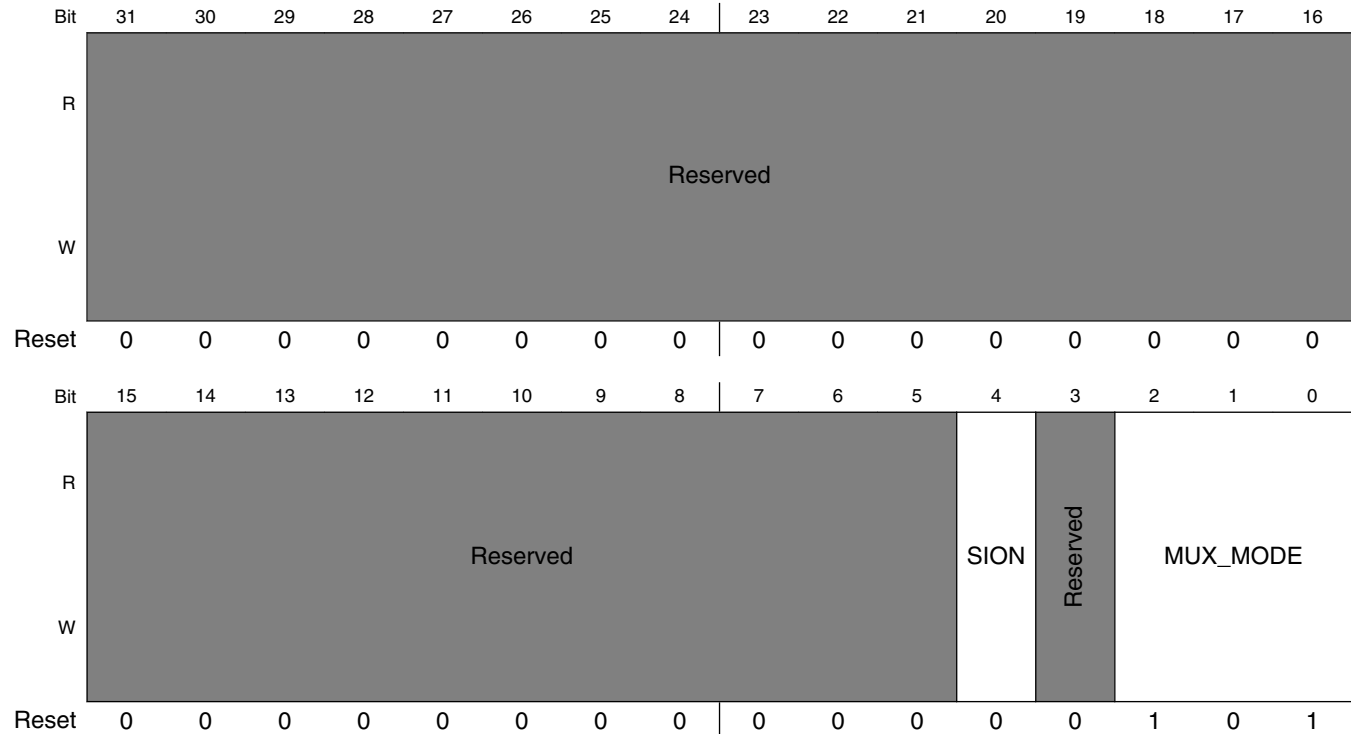
IOMUXC_SW_MUX_CTL_PAD_SPDIF_TX field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SPDIF_TX is determined by functionality 1 SION_ENABLED — Force Input Path of pad SPDIF_TX
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: SPDIF_TX 000 ALT0_SPDIF1_OUT — Select mux mode: ALT0 mux port: OUT of instance: SPDIF1 001 ALT1_PWM3_OUT — Select mux mode: ALT1 mux port: OUT of instance: PWM3 101 ALT5_GPIO5_IO03 — Select mux mode: ALT5 mux port: IO03 of instance: GPIO5

8.2.5.119 SW_MUX_CTL_PAD_SPDIF_RX SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SPDIF_RX)

SW_MUX_CTL Register

Address: 3033_0000h base + 1ECh offset = 3033_01ECh



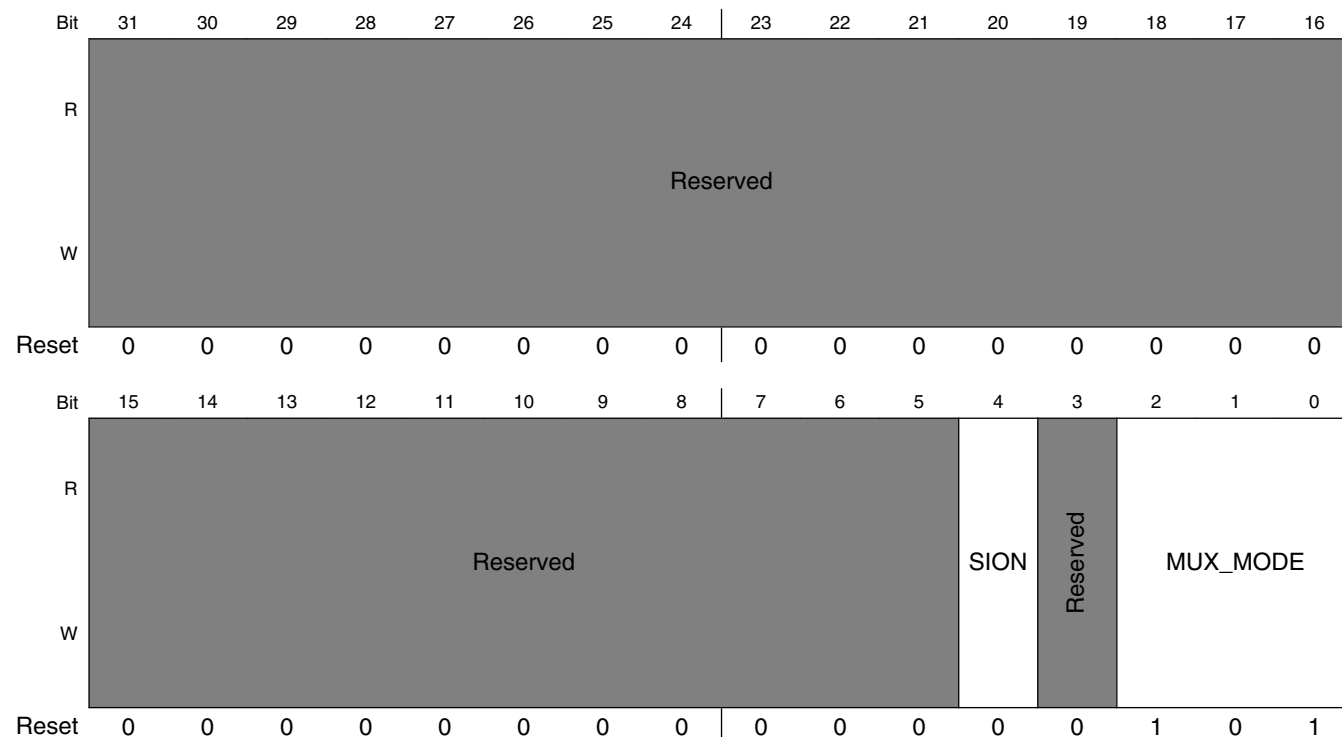
IOMUXC_SW_MUX_CTL_PAD_SPDIF_RX field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SPDIF_RX is determined by functionality 1 SION_ENABLED — Force Input Path of pad SPDIF_RX
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: SPDIF_RX 000 ALT0_SPDIF1_IN — Select mux mode: ALT0 mux port: IN of instance: SPDIF1 001 ALT1_PWM2_OUT — Select mux mode: ALT1 mux port: OUT of instance: PWM2 101 ALT5_GPIO5_IO04 — Select mux mode: ALT5 mux port: IO04 of instance: GPIO5

8.2.5.120 SW_MUX_CTL_PAD_SPDIF_EXT_CLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_SPDIF_EXT_CLK)

SW_MUX_CTL Register

Address: 3033_0000h base + 1F0h offset = 3033_01F0h



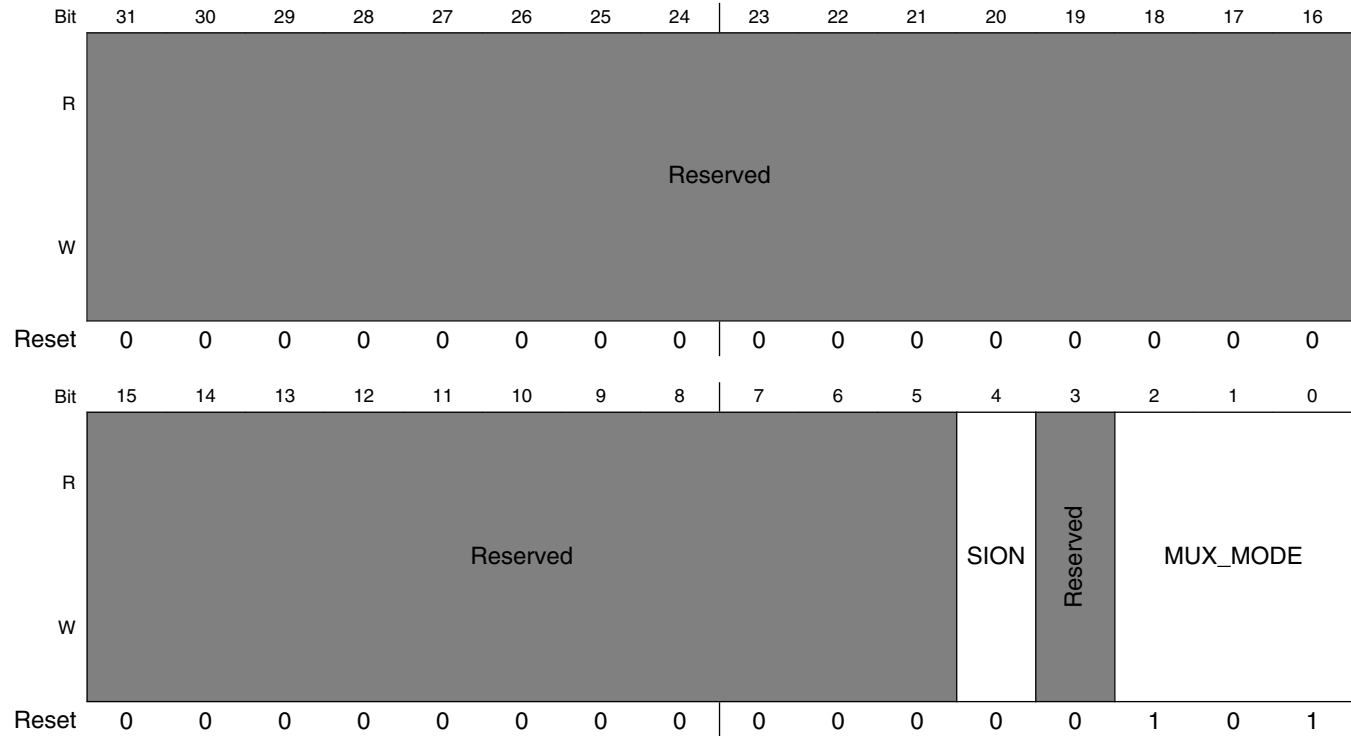
IOMUXC_SW_MUX_CTL_PAD_SPDIF_EXT_CLK field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad SPDIF_EXT_CLK is determined by functionality 1 SION_ENABLED — Force Input Path of pad SPDIF_EXT_CLK
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: SPDIF_EXT_CLK 000 ALT0_SPDIF1_EXT_CLK — Select mux mode: ALT0 mux port: EXT_CLK of instance: SPDIF1 001 ALT1_PWM1_OUT — Select mux mode: ALT1 mux port: OUT of instance: PWM1 101 ALT5_GPIO5_IO05 — Select mux mode: ALT5 mux port: IO05 of instance: GPIO5

8.2.5.121 SW_MUX_CTL_PAD_ECSP11_SCLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSP11_SCLK)

SW_MUX_CTL Register

Address: 3033_0000h base + 1F4h offset = 3033_01F4h



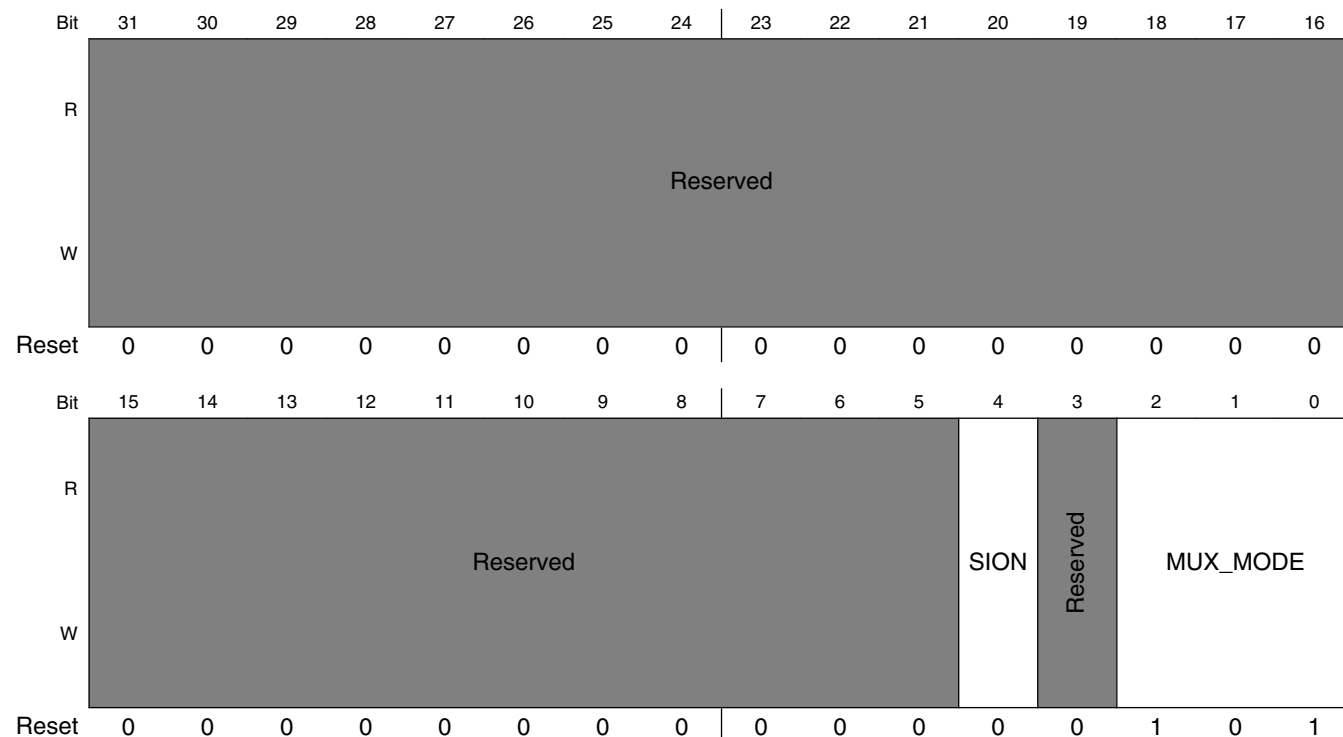
IOMUXC_SW_MUX_CTL_PAD_ECSP11_SCLK field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ECSP11_SCLK is determined by functionality 1 SION_ENABLED — Force Input Path of pad ECSP11_SCLK
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: ECSP11_SCLK 000 ALT0_ECSP11_SCLK — Select mux mode: ALT0 mux port: SCLK of instance: ECSP11 001 ALT1_UART3_RX — Select mux mode: ALT1 mux port: RX of instance: UART3 101 ALT5_GPIO5_IO06 — Select mux mode: ALT5 mux port: IO06 of instance: GPIO5

8.2.5.122 SW_MUX_CTL_PAD_ECSP11_MOSI SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSP11_MOSI)

SW_MUX_CTL Register

Address: 3033_0000h base + 1F8h offset = 3033_01F8h



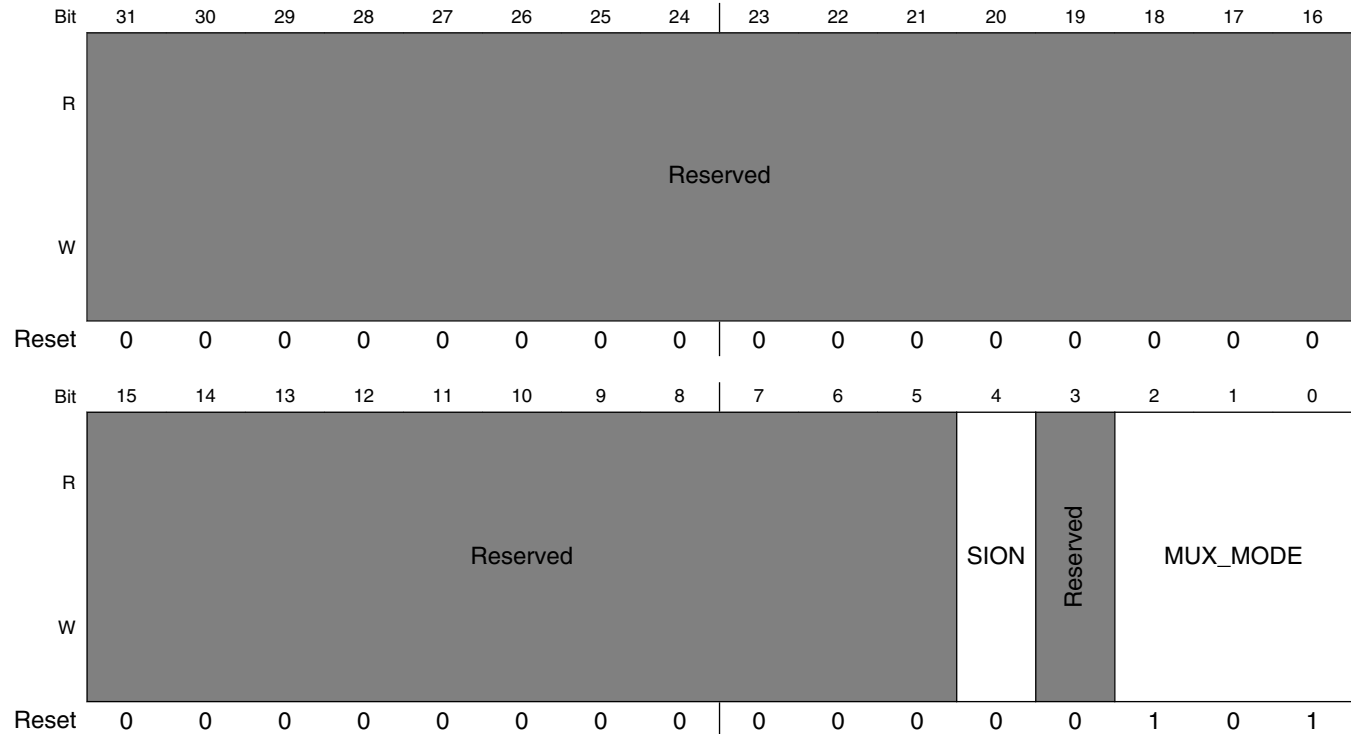
IOMUXC_SW_MUX_CTL_PAD_ECSP11_MOSI field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ECSP11_MOSI is determined by functionality 1 SION_ENABLED — Force Input Path of pad ECSP11_MOSI
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: ECSP11_MOSI 000 ALT0_ECSP11_MOSI — Select mux mode: ALT0 mux port: MOSI of instance: ECSP11 001 ALT1_UART3_TX — Select mux mode: ALT1 mux port: TX of instance: UART3 101 ALT5_GPIO5_IO07 — Select mux mode: ALT5 mux port: IO07 of instance: GPIO5

8.2.5.123 SW_MUX_CTL_PAD_ECSP11_MISO SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSP11_MISO)

SW_MUX_CTL Register

Address: 3033_0000h base + 1FCh offset = 3033_01FCh



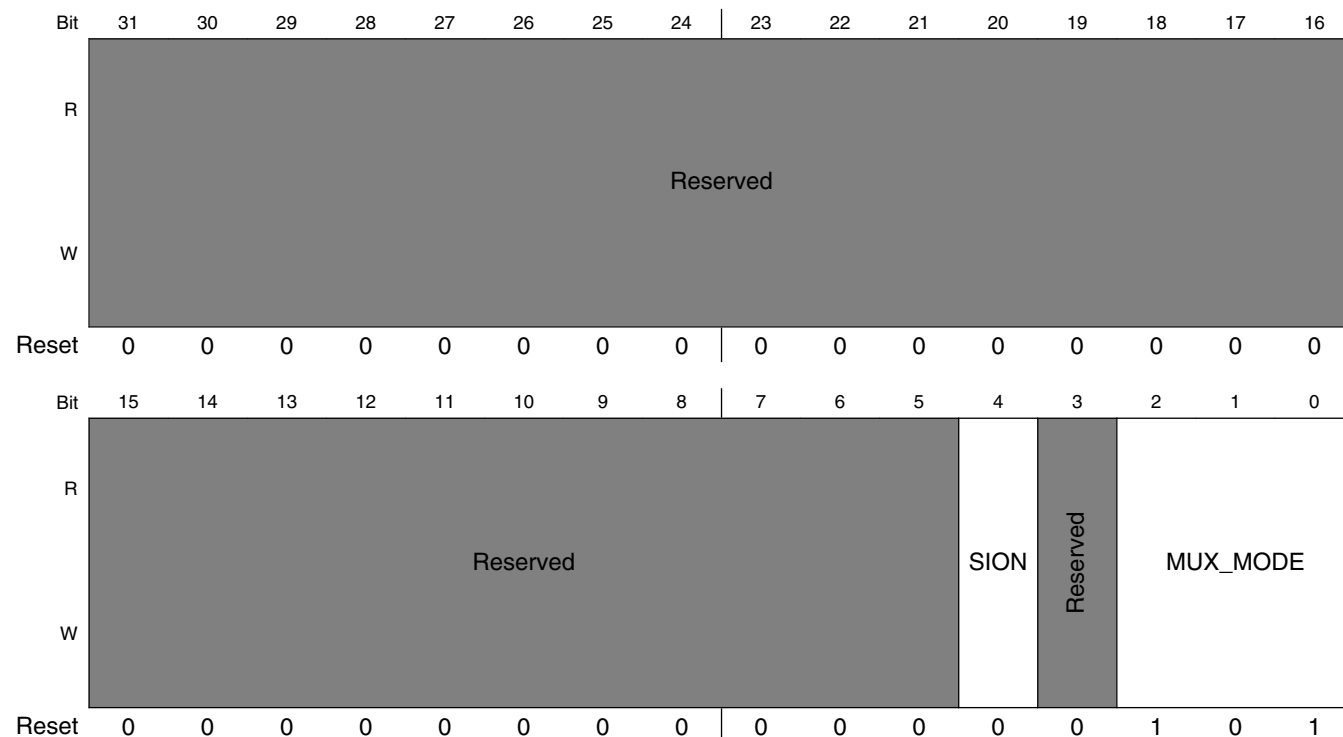
IOMUXC_SW_MUX_CTL_PAD_ECSP11_MISO field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ECSP11_MISO is determined by functionality 1 SION_ENABLED — Force Input Path of pad ECSP11_MISO
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: ECSP11_MISO 000 ALT0_ECSP11_MISO — Select mux mode: ALT0 mux port: MISO of instance: ECSP11 001 ALT1_UART3_CTS_B — Select mux mode: ALT1 mux port: CTS_B of instance: UART3 101 ALT5_GPIO5_IO08 — Select mux mode: ALT5 mux port: IO08 of instance: GPIO5

8.2.5.124 SW_MUX_CTL_PAD_ECSP1_SS0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSP1_SS0)

SW_MUX_CTL Register

Address: 3033_0000h base + 200h offset = 3033_0200h



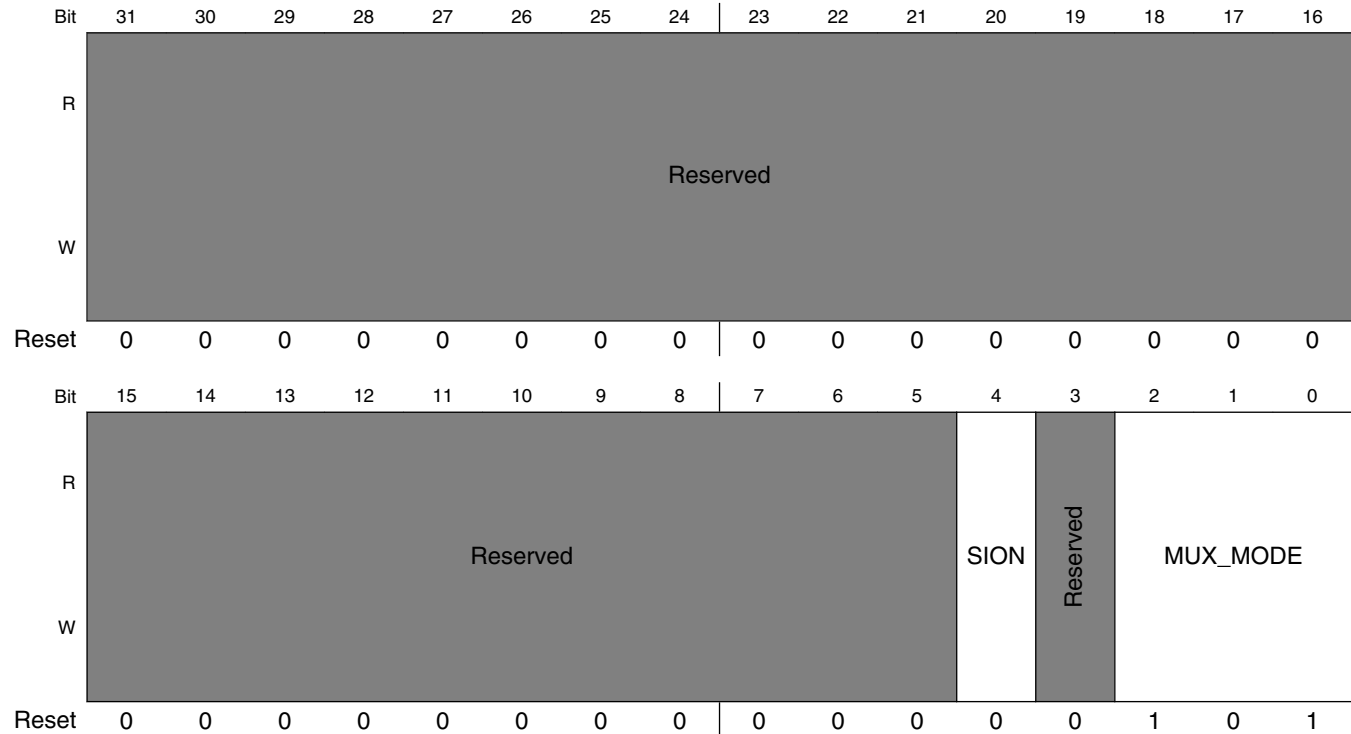
IOMUXC_SW_MUX_CTL_PAD_ECSP1_SS0 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ECSP1_SS0 is determined by functionality 1 SION_ENABLED — Force Input Path of pad ECSP1_SS0
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: ECSP1_SS0 000 ALT0_ECSP1_SS0 — Select mux mode: ALT0 mux port: SS0 of instance: ECSP1 001 ALT1_UART3_RTS_B — Select mux mode: ALT1 mux port: RTS_B of instance: UART3 101 ALT5_GPIO5_IO09 — Select mux mode: ALT5 mux port: IO09 of instance: GPIO5

8.2.5.125 SW_MUX_CTL_PAD_ECSPi2_SCLK SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSPi2_SCLK)

SW_MUX_CTL Register

Address: 3033_0000h base + 204h offset = 3033_0204h



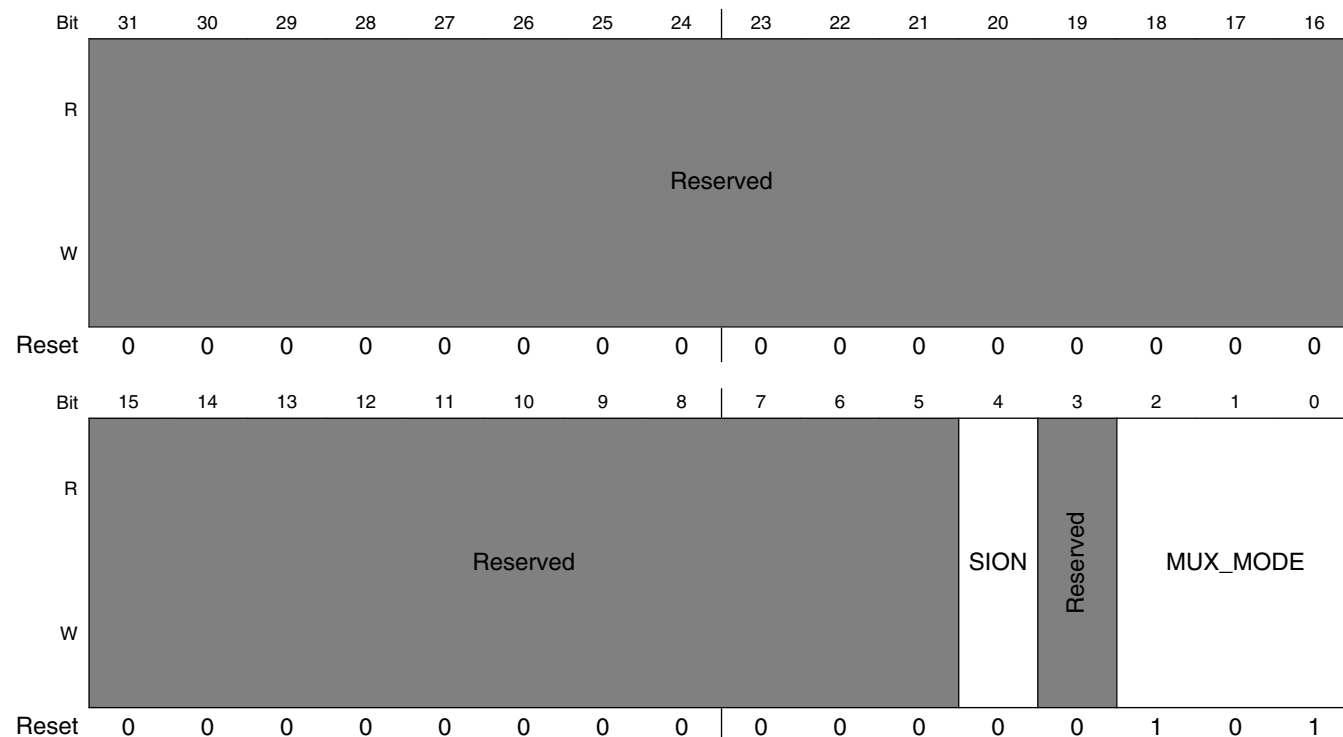
IOMUXC_SW_MUX_CTL_PAD_ECSPi2_SCLK field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ECSPi2_SCLK is determined by functionality 1 SION_ENABLED — Force Input Path of pad ECSPi2_SCLK
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: ECSPi2_SCLK 000 ALT0_ECSPi2_SCLK — Select mux mode: ALT0 mux port: SCLK of instance: ECSPi2 001 ALT1_UART4_RX — Select mux mode: ALT1 mux port: RX of instance: UART4 101 ALT5_GPIO5_IO10 — Select mux mode: ALT5 mux port: IO10 of instance: GPIO5

8.2.5.126 SW_MUX_CTL_PAD_ECSPi2_MOSI SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSPi2_MOSI)

SW_MUX_CTL Register

Address: 3033_0000h base + 208h offset = 3033_0208h



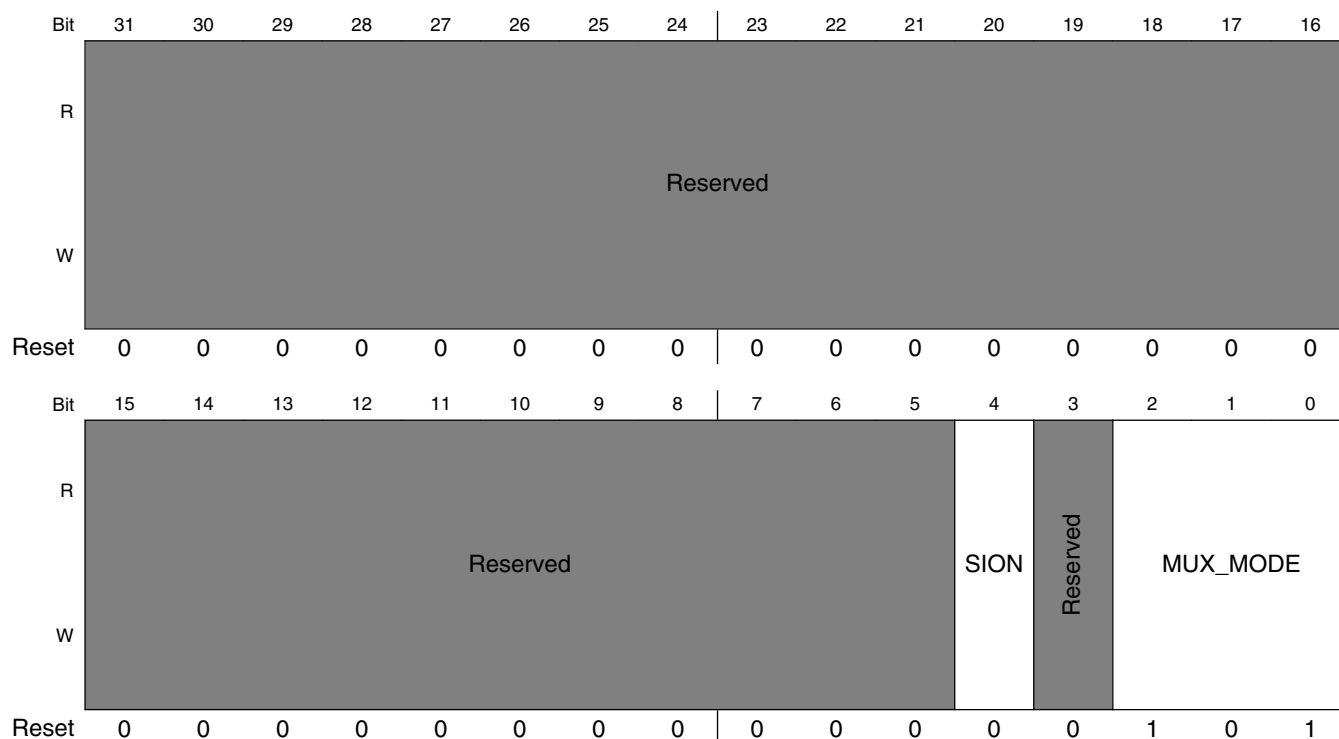
IOMUXC_SW_MUX_CTL_PAD_ECSPi2_MOSI field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ECSPi2_MOSI is determined by functionality 1 SION_ENABLED — Force Input Path of pad ECSPi2_MOSI
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: ECSPi2_MOSI 000 ALT0_ECSPi2_MOSI — Select mux mode: ALT0 mux port: MOSI of instance: ECSPi2 001 ALT1_UART4_TX — Select mux mode: ALT1 mux port: TX of instance: UART4 101 ALT5_GPIO5_IO11 — Select mux mode: ALT5 mux port: IO11 of instance: GPIO5

8.2.5.127 SW_MUX_CTL_PAD_ECSPi2_MISO SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSPi2_MISO)

SW_MUX_CTL Register

Address: 3033_0000h base + 20Ch offset = 3033_020Ch



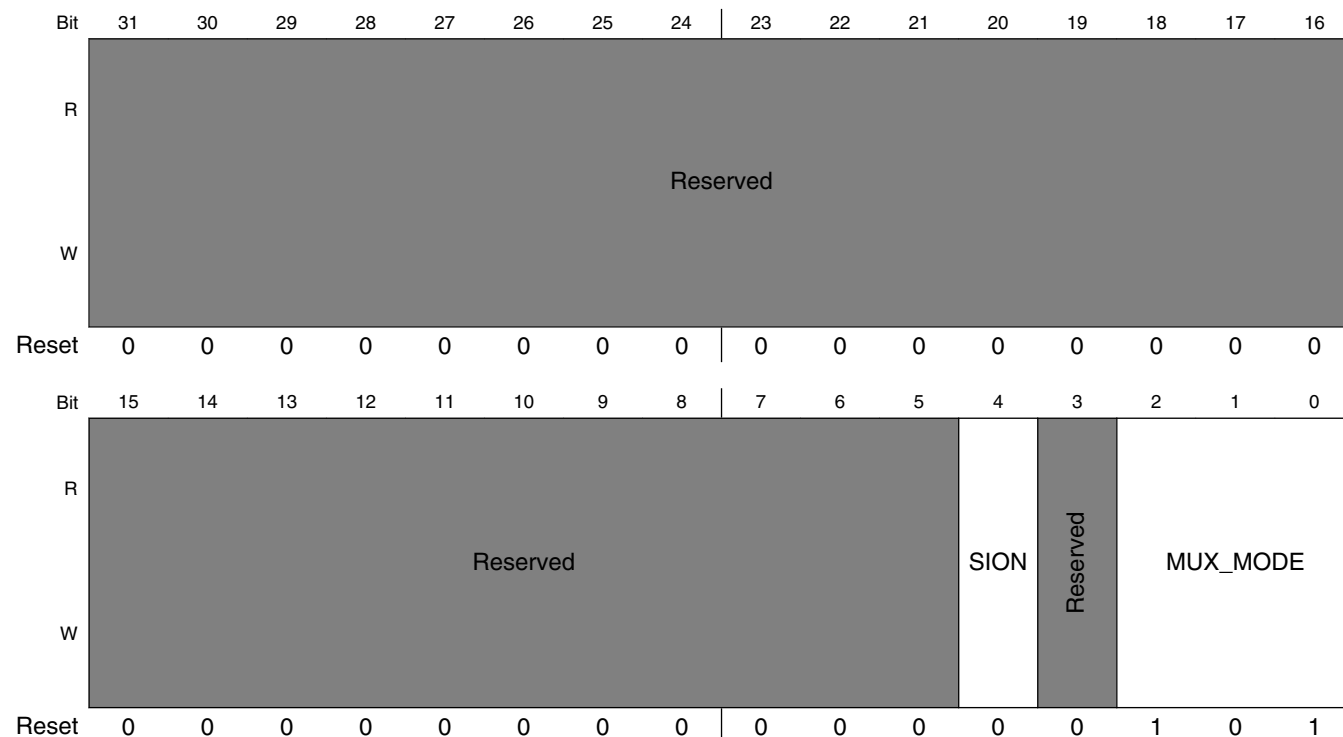
IOMUXC_SW_MUX_CTL_PAD_ECSPi2_MISO field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ECSPi2_MISO is determined by functionality 1 SION_ENABLED — Force Input Path of pad ECSPi2_MISO
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: ECSPi2_MISO 000 ALT0_ECSPi2_MISO — Select mux mode: ALT0 mux port: MISO of instance: ECSPi2 001 ALT1_UART4_CTS_B — Select mux mode: ALT1 mux port: CTS_B of instance: UART4 101 ALT5_GPIO5_IO12 — Select mux mode: ALT5 mux port: IO12 of instance: GPIO5

8.2.5.128 SW_MUX_CTL_PAD_ECSPi2_SS0 SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_ECSPi2_SS0)

SW_MUX_CTL Register

Address: 3033_0000h base + 210h offset = 3033_0210h



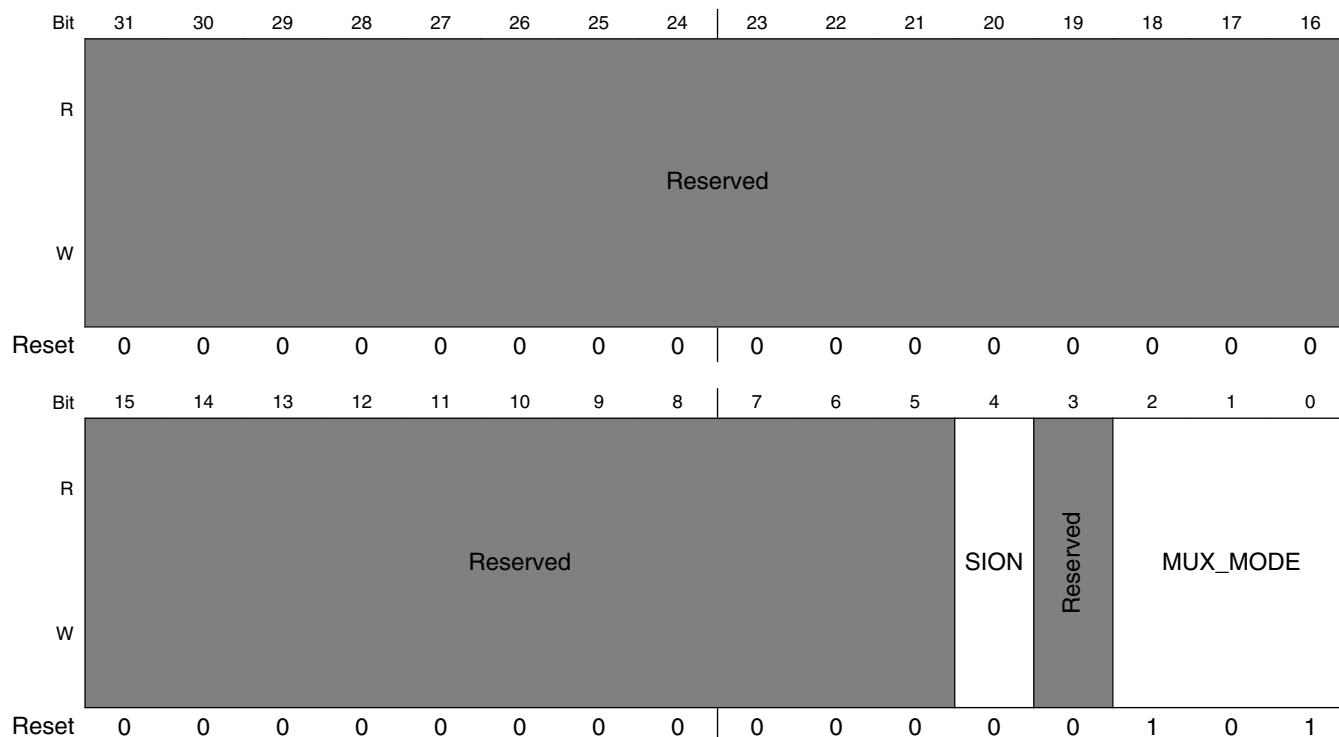
IOMUXC_SW_MUX_CTL_PAD_ECSPi2_SS0 field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad ECSPi2_SS0 is determined by functionality 1 SION_ENABLED — Force Input Path of pad ECSPi2_SS0
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: ECSPi2_SS0 000 ALT0_ECSPi2_SS0 — Select mux mode: ALT0 mux port: SS0 of instance: ECSPi2 001 ALT1_UART4_RTS_B — Select mux mode: ALT1 mux port: RTS_B of instance: UART4 101 ALT5_GPIO5_IO13 — Select mux mode: ALT5 mux port: IO13 of instance: GPIO5

8.2.5.129 SW_MUX_CTL_PAD_I2C1_SCL SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C1_SCL)

SW_MUX_CTL Register

Address: 3033_0000h base + 214h offset = 3033_0214h



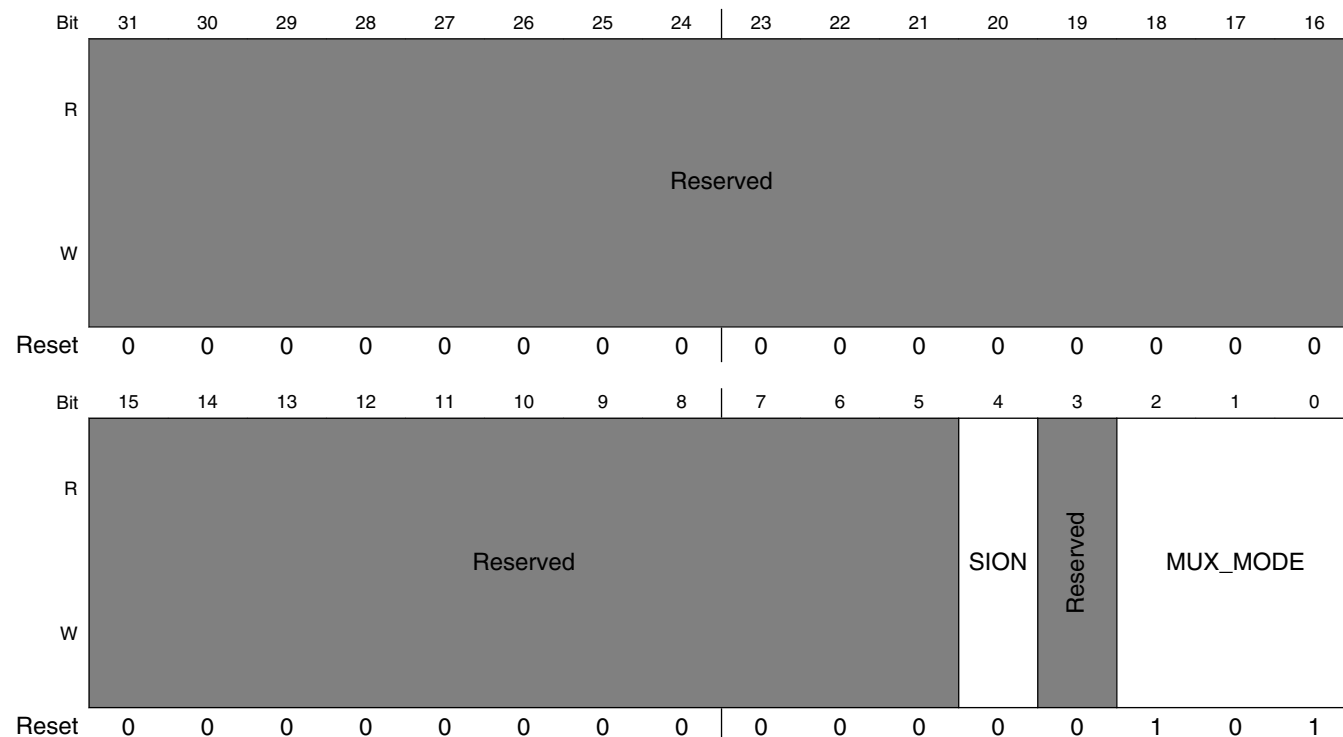
IOMUXC_SW_MUX_CTL_PAD_I2C1_SCL field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad I2C1_SCL is determined by functionality 1 SION_ENABLED — Force Input Path of pad I2C1_SCL
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: I2C1_SCL 000 ALT0_I2C1_SCL — Select mux mode: ALT0 mux port: SCL of instance: I2C1 001 ALT1_ENET1_MDC — Select mux mode: ALT1 mux port: MDC of instance: ENET1 101 ALT5_GPIO5_IO14 — Select mux mode: ALT5 mux port: IO14 of instance: GPIO5

8.2.5.130 SW_MUX_CTL_PAD_I2C1_SDA SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C1_SDA)

SW_MUX_CTL Register

Address: 3033_0000h base + 218h offset = 3033_0218h



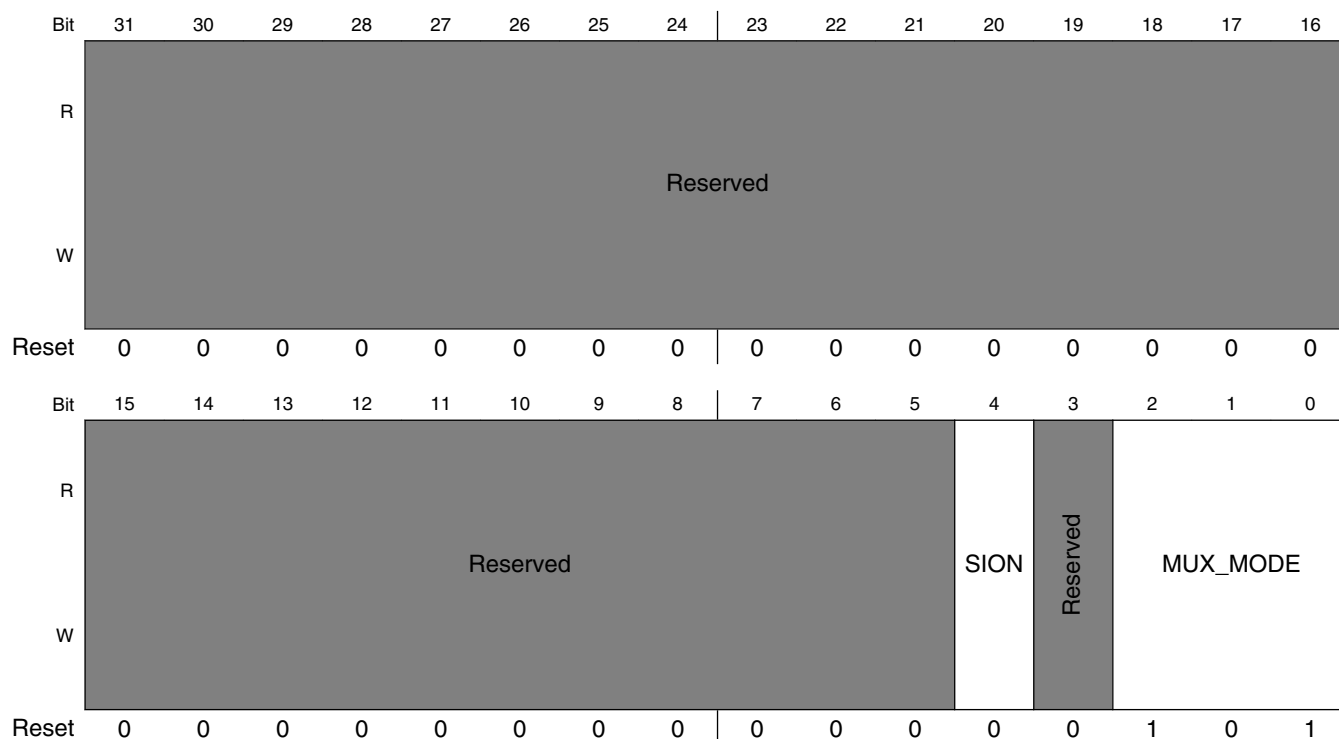
IOMUXC_SW_MUX_CTL_PAD_I2C1_SDA field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad I2C1_SDA is determined by functionality 1 SION_ENABLED — Force Input Path of pad I2C1_SDA
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: I2C1_SDA 000 ALT0_I2C1_SDA — Select mux mode: ALT0 mux port: SDA of instance: I2C1 001 ALT1_ENET1_MDIO — Select mux mode: ALT1 mux port: MDIO of instance: ENET1 101 ALT5_GPIO5_IO15 — Select mux mode: ALT5 mux port: IO15 of instance: GPIO5

8.2.5.131 SW_MUX_CTL_PAD_I2C2_SCL SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C2_SCL)

SW_MUX_CTL Register

Address: 3033_0000h base + 21Ch offset = 3033_021Ch



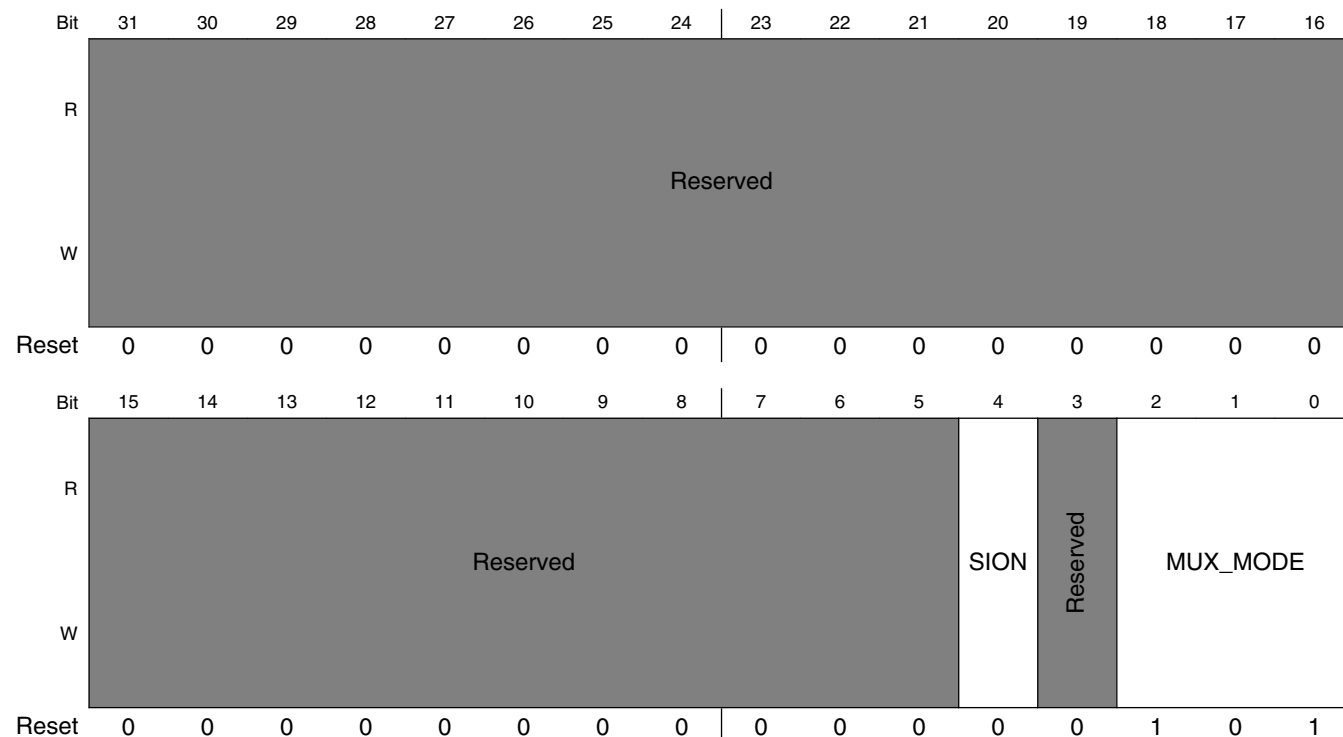
IOMUXC_SW_MUX_CTL_PAD_I2C2_SCL field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad I2C2_SCL is determined by functionality 1 SION_ENABLED — Force Input Path of pad I2C2_SCL
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: I2C2_SCL 000 ALT0_I2C2_SCL — Select mux mode: ALT0 mux port: SCL of instance: I2C2 001 ALT1_ENET1_1588_EVENT1_IN — Select mux mode: ALT1 mux port: 1588_EVENT1_IN of instance: ENET1 101 ALT5_GPIO5_IO16 — Select mux mode: ALT5 mux port: IO16 of instance: GPIO5

8.2.5.132 SW_MUX_CTL_PAD_I2C2_SDA SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C2_SDA)

SW_MUX_CTL Register

Address: 3033_0000h base + 220h offset = 3033_0220h



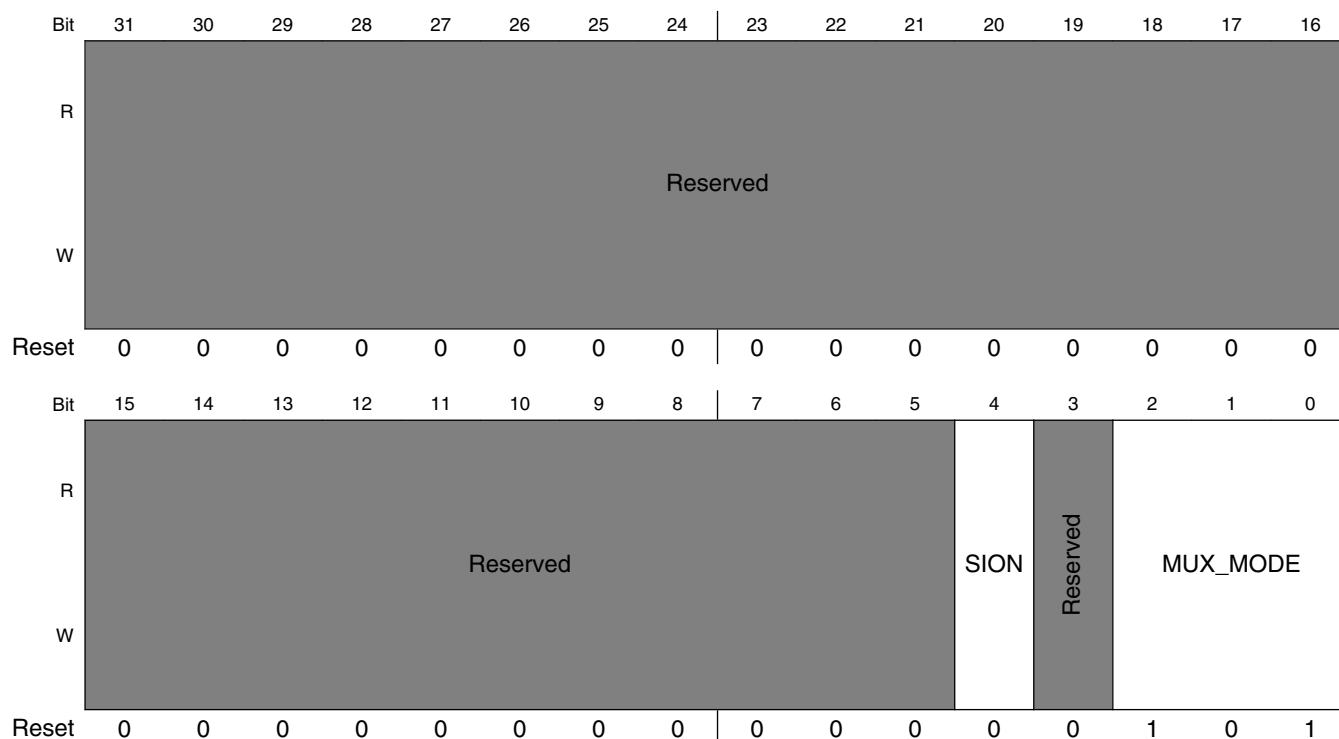
IOMUXC_SW_MUX_CTL_PAD_I2C2_SDA field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad I2C2_SDA is determined by functionality 1 SION_ENABLED — Force Input Path of pad I2C2_SDA
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: I2C2_SDA 000 ALT0_I2C2_SDA — Select mux mode: ALT0 mux port: SDA of instance: I2C2 001 ALT1_ENET1_1588_EVENT1_OUT — Select mux mode: ALT1 mux port: 1588_EVENT1_OUT of instance: ENET1 101 ALT5_GPIO5_IO17 — Select mux mode: ALT5 mux port: IO17 of instance: GPIO5

8.2.5.133 SW_MUX_CTL_PAD_I2C3_SCL SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C3_SCL)

SW_MUX_CTL Register

Address: 3033_0000h base + 224h offset = 3033_0224h



IOMUXC_SW_MUX_CTL_PAD_I2C3_SCL field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad I2C3_SCL is determined by functionality 1 SION_ENABLED — Force Input Path of pad I2C3_SCL
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: I2C3_SCL 000 ALT0_I2C3_SCL — Select mux mode: ALT0 mux port: SCL of instance: I2C3 001 ALT1_PWM4_OUT — Select mux mode: ALT1 mux port: OUT of instance: PWM4 010 ALT2_GPT2_CLK — Select mux mode: ALT2 mux port: CLK of instance: GPT2 101 ALT5_GPIO5_IO18 — Select mux mode: ALT5 mux port: IO18 of instance: GPIO5

8.2.5.134 SW_MUX_CTL_PAD_I2C3_SDA SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C3_SDA)

SW_MUX_CTL Register

Address: 3033_0000h base + 228h offset = 3033_0228h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION				Reserved	MUX_MODE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1

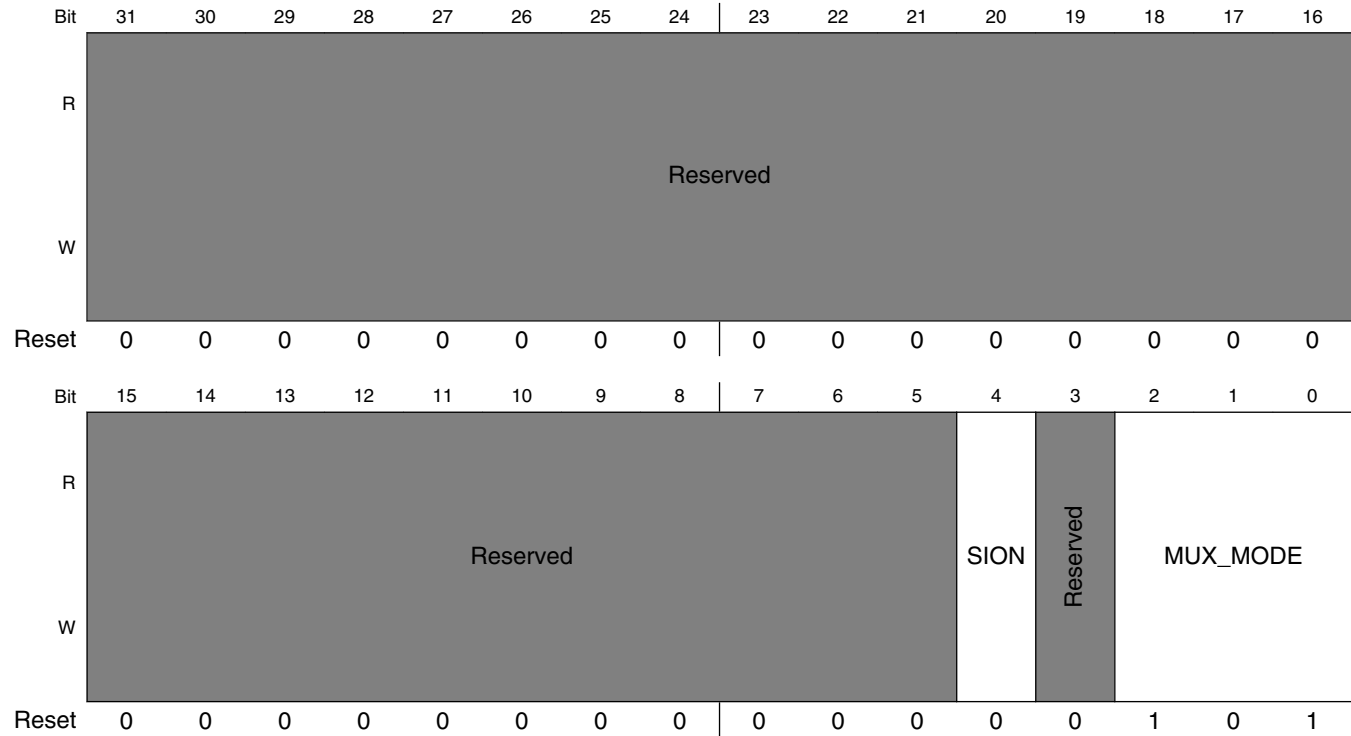
IOMUXC_SW_MUX_CTL_PAD_I2C3_SDA field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad I2C3_SDA is determined by functionality 1 SION_ENABLED — Force Input Path of pad I2C3_SDA
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: I2C3_SDA 000 ALT0_I2C3_SDA — Select mux mode: ALT0 mux port: SDA of instance: I2C3 001 ALT1_PWM3_OUT — Select mux mode: ALT1 mux port: OUT of instance: PWM3 010 ALT2_GPT3_CLK — Select mux mode: ALT2 mux port: CLK of instance: GPT3 101 ALT5_GPIO5_IO19 — Select mux mode: ALT5 mux port: IO19 of instance: GPIO5

8.2.5.135 SW_MUX_CTL_PAD_I2C4_SCL SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C4_SCL)

SW_MUX_CTL Register

Address: 3033_0000h base + 22Ch offset = 3033_022Ch



IOMUXC_SW_MUX_CTL_PAD_I2C4_SCL field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad I2C4_SCL is determined by functionality 1 SION_ENABLED — Force Input Path of pad I2C4_SCL
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: I2C4_SCL 000 ALT0_I2C4_SCL — Select mux mode: ALT0 mux port: SCL of instance: I2C4 001 ALT1_PWM2_OUT — Select mux mode: ALT1 mux port: OUT of instance: PWM2 010 ALT2_PCIE1_CLKREQ_B — Select mux mode: ALT2 mux port: CLKREQ_B of instance: PCIE1 101 ALT5_GPIO5_IO20 — Select mux mode: ALT5 mux port: IO20 of instance: GPIO5

8.2.5.136 SW_MUX_CTL_PAD_I2C4_SDA SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_I2C4_SDA)

SW_MUX_CTL Register

Address: 3033_0000h base + 230h offset = 3033_0230h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								SION		Reserved		MUX_MODE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1

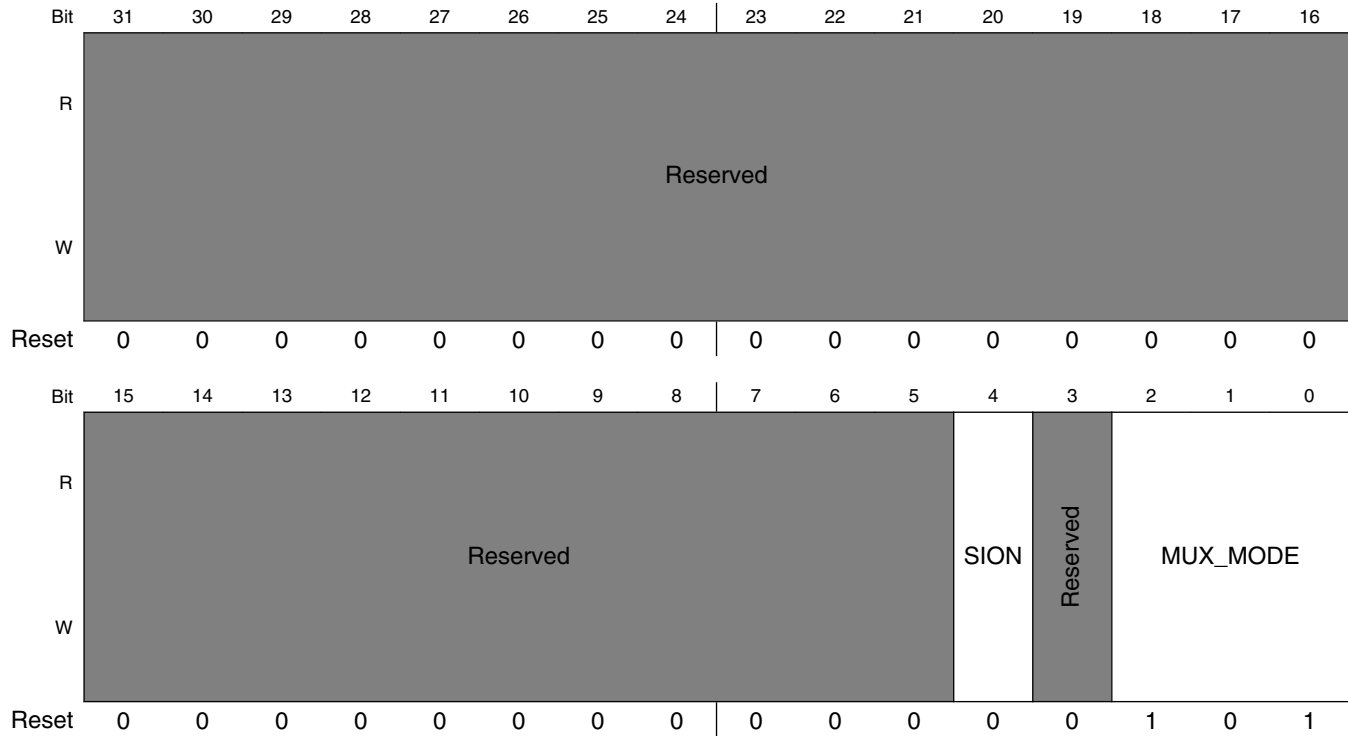
IOMUXC_SW_MUX_CTL_PAD_I2C4_SDA field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad I2C4_SDA is determined by functionality 1 SION_ENABLED — Force Input Path of pad I2C4_SDA
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: I2C4_SDA 000 ALT0_I2C4_SDA — Select mux mode: ALT0 mux port: SDA of instance: I2C4 001 ALT1_PWM1_OUT — Select mux mode: ALT1 mux port: OUT of instance: PWM1 010 ALT2_PCIE2_CLKREQ_B — Select mux mode: ALT2 mux port: CLKREQ_B of instance: PCIE2 101 ALT5_GPIO5_IO21 — Select mux mode: ALT5 mux port: IO21 of instance: GPIO5

8.2.5.137 SW_MUX_CTL_PAD_UART1_RXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART1_RXD)

SW_MUX_CTL Register

Address: 3033_0000h base + 234h offset = 3033_0234h



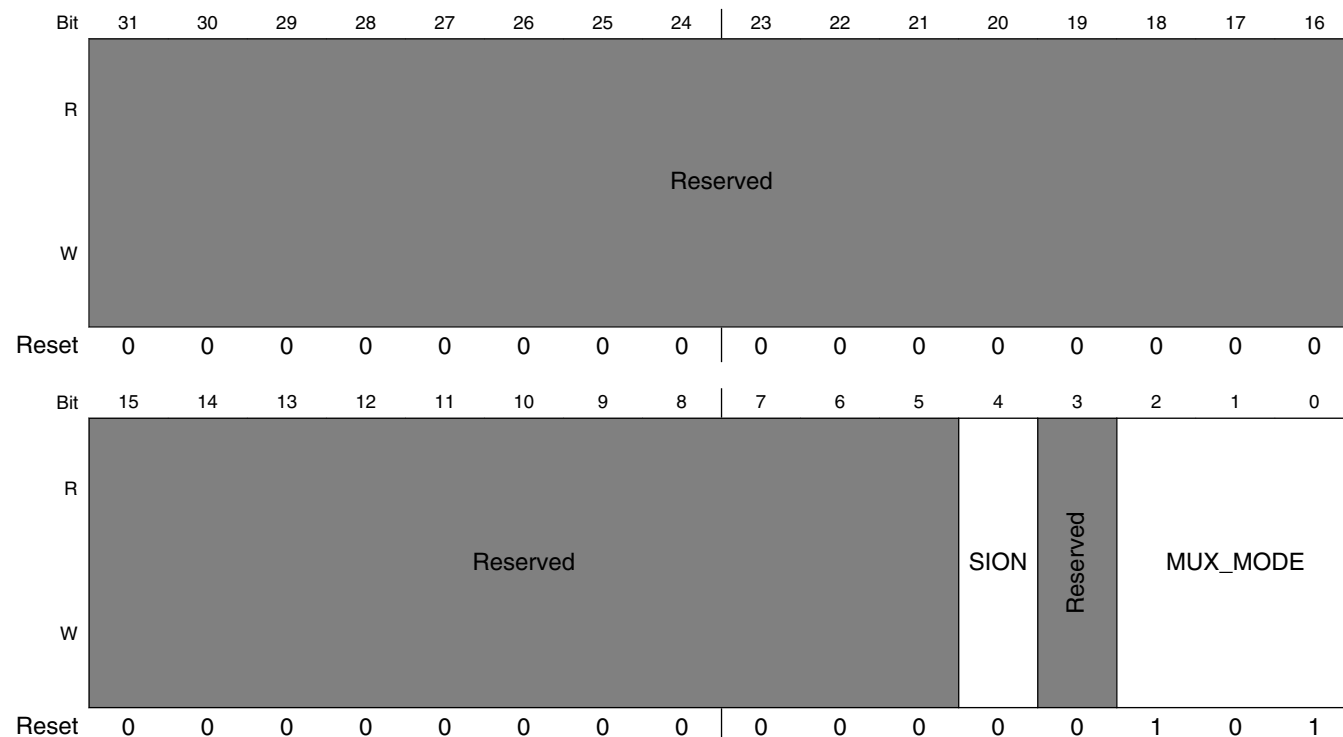
IOMUXC_SW_MUX_CTL_PAD_UART1_RXD field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad UART1_RXD is determined by functionality 1 SION_ENABLED — Force Input Path of pad UART1_RXD
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: UART1_RXD 000 ALT0_UART1_RX — Select mux mode: ALT0 mux port: RX of instance: UART1 001 ALT1_ECSPi3_SCLK — Select mux mode: ALT1 mux port: SCLK of instance: ECSPi3 101 ALT5_GPIO5_IO22 — Select mux mode: ALT5 mux port: IO22 of instance: GPIO5

8.2.5.138 SW_MUX_CTL_PAD_UART1_TXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART1_TXD)

SW_MUX_CTL Register

Address: 3033_0000h base + 238h offset = 3033_0238h



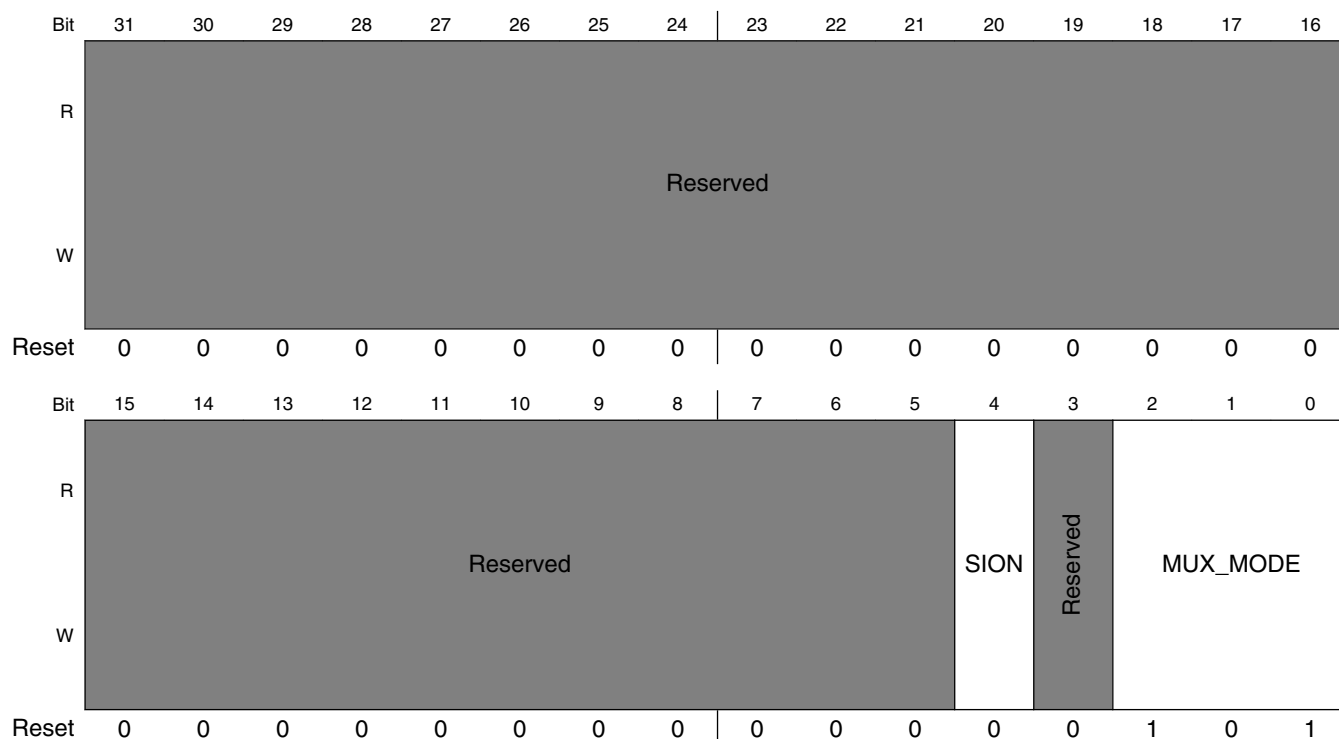
IOMUXC_SW_MUX_CTL_PAD_UART1_TXD field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad UART1_TXD is determined by functionality 1 SION_ENABLED — Force Input Path of pad UART1_TXD
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: UART1_TXD 000 ALT0_UART1_TX — Select mux mode: ALT0 mux port: TX of instance: UART1 001 ALT1_ECSPi3_MOSI — Select mux mode: ALT1 mux port: MOSI of instance: ECSPi3 101 ALT5_GPIO5_IO23 — Select mux mode: ALT5 mux port: IO23 of instance: GPIO5

8.2.5.139 SW_MUX_CTL_PAD_UART2_RXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART2_RXD)

SW_MUX_CTL Register

Address: 3033_0000h base + 23Ch offset = 3033_023Ch



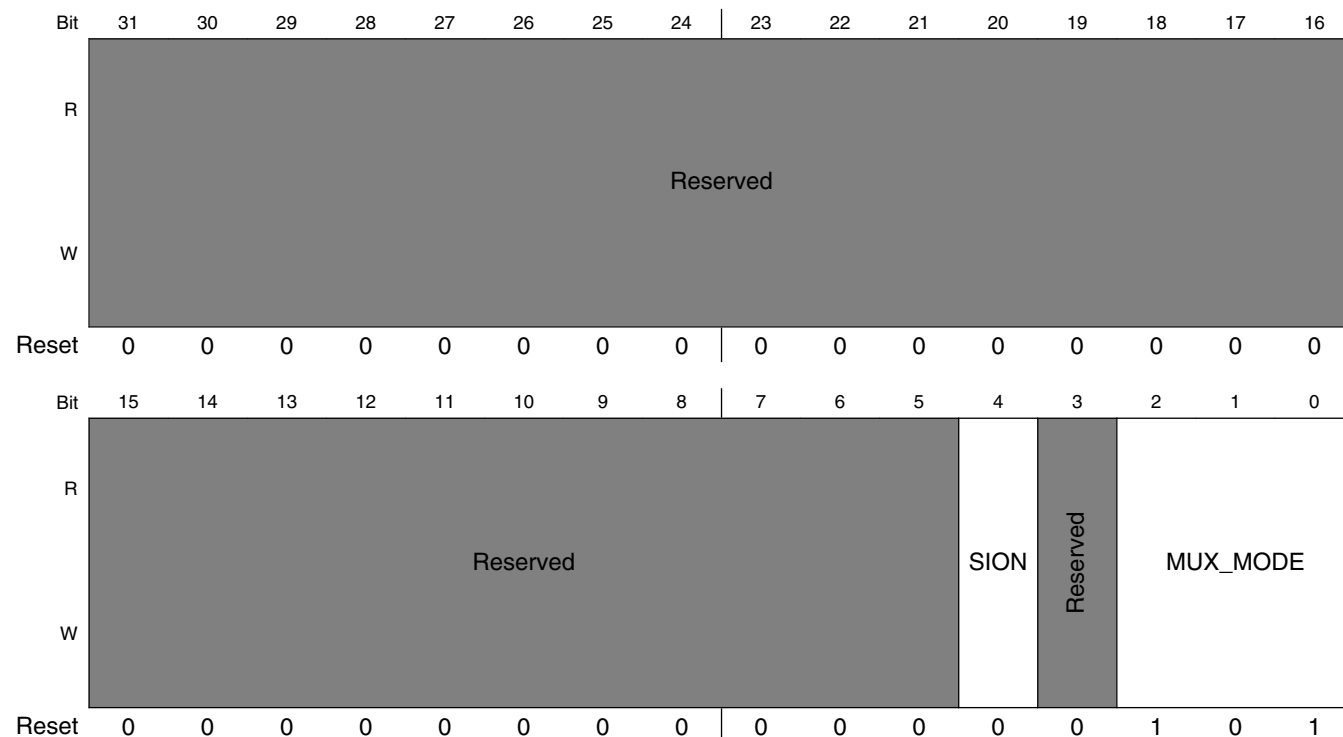
IOMUXC_SW_MUX_CTL_PAD_UART2_RXD field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad UART2_RXD is determined by functionality 1 SION_ENABLED — Force Input Path of pad UART2_RXD
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: UART2_RXD 000 ALT0_UART2_RX — Select mux mode: ALT0 mux port: RX of instance: UART2 001 ALT1_ECSPi3_MISO — Select mux mode: ALT1 mux port: MISO of instance: ECSPi3 101 ALT5_GPIO5_IO24 — Select mux mode: ALT5 mux port: IO24 of instance: GPIO5

8.2.5.140 SW_MUX_CTL_PAD_UART2_TXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART2_TXD)

SW_MUX_CTL Register

Address: 3033_0000h base + 240h offset = 3033_0240h



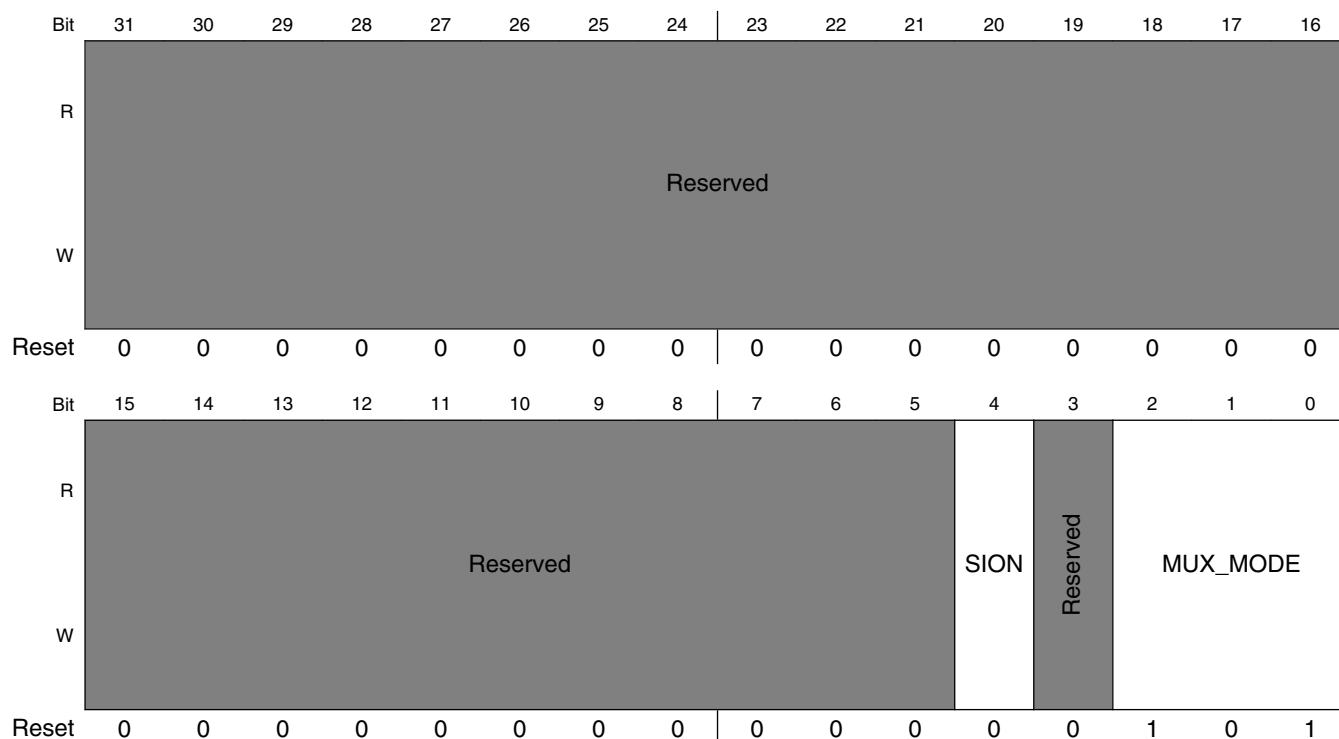
IOMUXC_SW_MUX_CTL_PAD_UART2_TXD field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad UART2_TXD is determined by functionality 1 SION_ENABLED — Force Input Path of pad UART2_TXD
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: UART2_TXD 000 ALT0_UART2_TX — Select mux mode: ALT0 mux port: TX of instance: UART2 001 ALT1_ECSPi3_SS0 — Select mux mode: ALT1 mux port: SS0 of instance: ECSPi3 101 ALT5_GPIO5_IO25 — Select mux mode: ALT5 mux port: IO25 of instance: GPIO5

8.2.5.141 SW_MUX_CTL_PAD_UART3_RXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART3_RXD)

SW_MUX_CTL Register

Address: 3033_0000h base + 244h offset = 3033_0244h



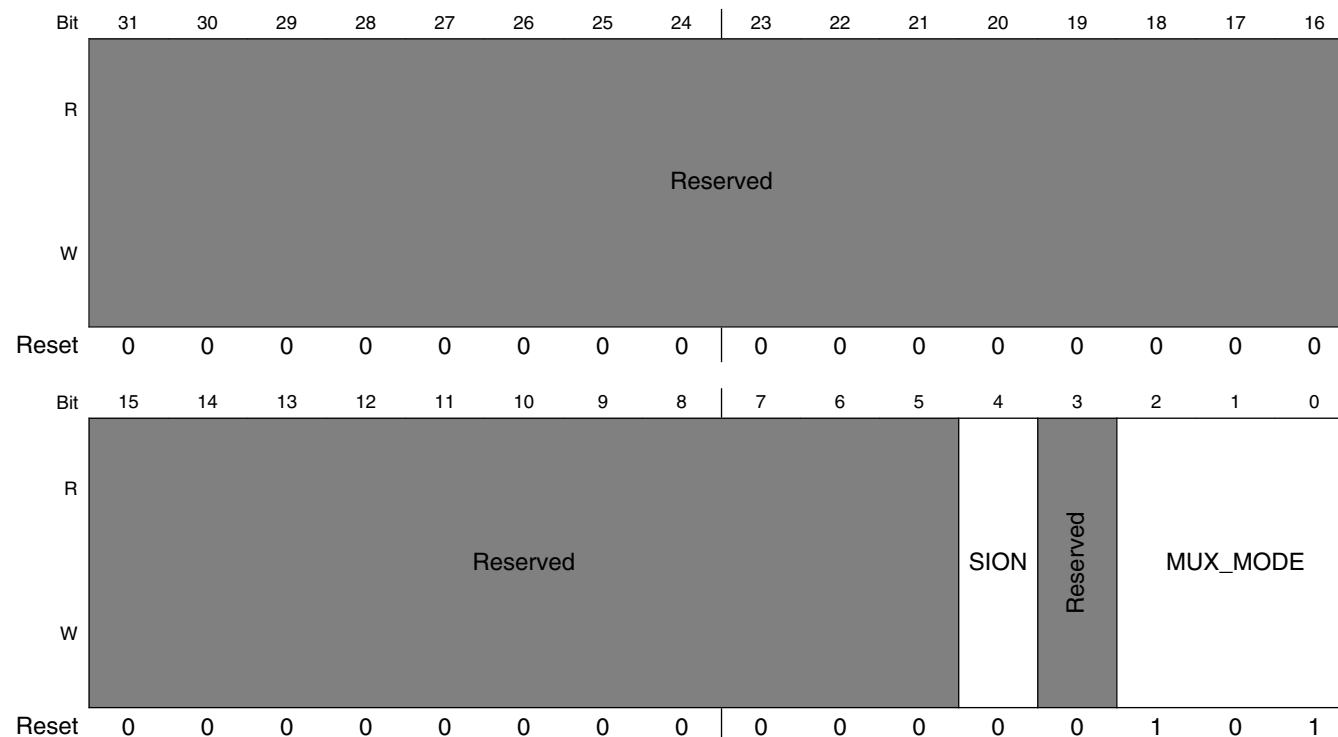
IOMUXC_SW_MUX_CTL_PAD_UART3_RXD field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad UART3_RXD is determined by functionality 1 SION_ENABLED — Force Input Path of pad UART3_RXD
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: UART3_RXD 000 ALT0_UART3_RX — Select mux mode: ALT0 mux port: RX of instance: UART3 001 ALT1_UART1_CTS_B — Select mux mode: ALT1 mux port: CTS_B of instance: UART1 101 ALT5_GPIO5_IO26 — Select mux mode: ALT5 mux port: IO26 of instance: GPIO5

8.2.5.142 SW_MUX_CTL_PAD_UART3_TXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART3_TXD)

SW_MUX_CTL Register

Address: 3033_0000h base + 248h offset = 3033_0248h



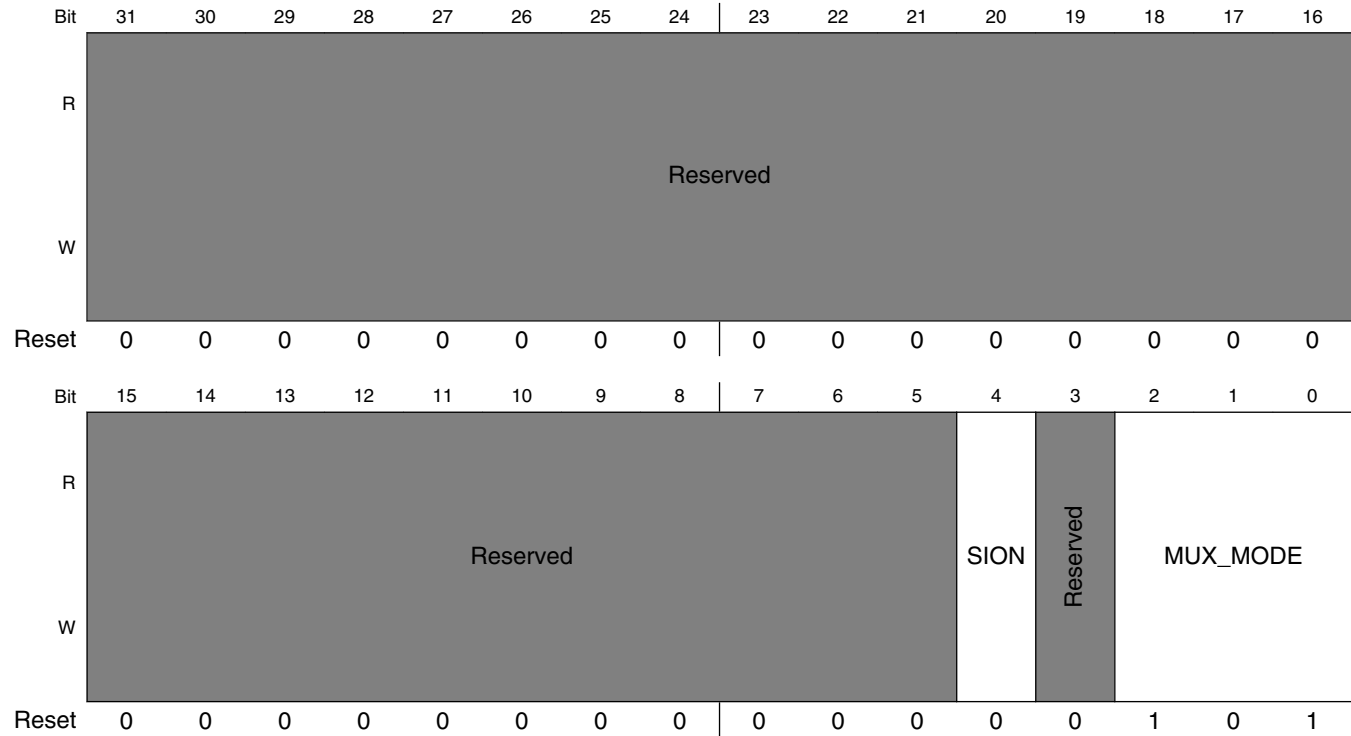
IOMUXC_SW_MUX_CTL_PAD_UART3_TXD field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad UART3_TXD is determined by functionality 1 SION_ENABLED — Force Input Path of pad UART3_TXD
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 3 iomux modes to be used for pad: UART3_TXD 000 ALT0_UART3_TX — Select mux mode: ALT0 mux port: TX of instance: UART3 001 ALT1_UART1_RTS_B — Select mux mode: ALT1 mux port: RTS_B of instance: UART1 101 ALT5_GPIO5_IO27 — Select mux mode: ALT5 mux port: IO27 of instance: GPIO5

8.2.5.143 SW_MUX_CTL_PAD_UART4_RXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART4_RXD)

SW_MUX_CTL Register

Address: 3033_0000h base + 24Ch offset = 3033_024Ch



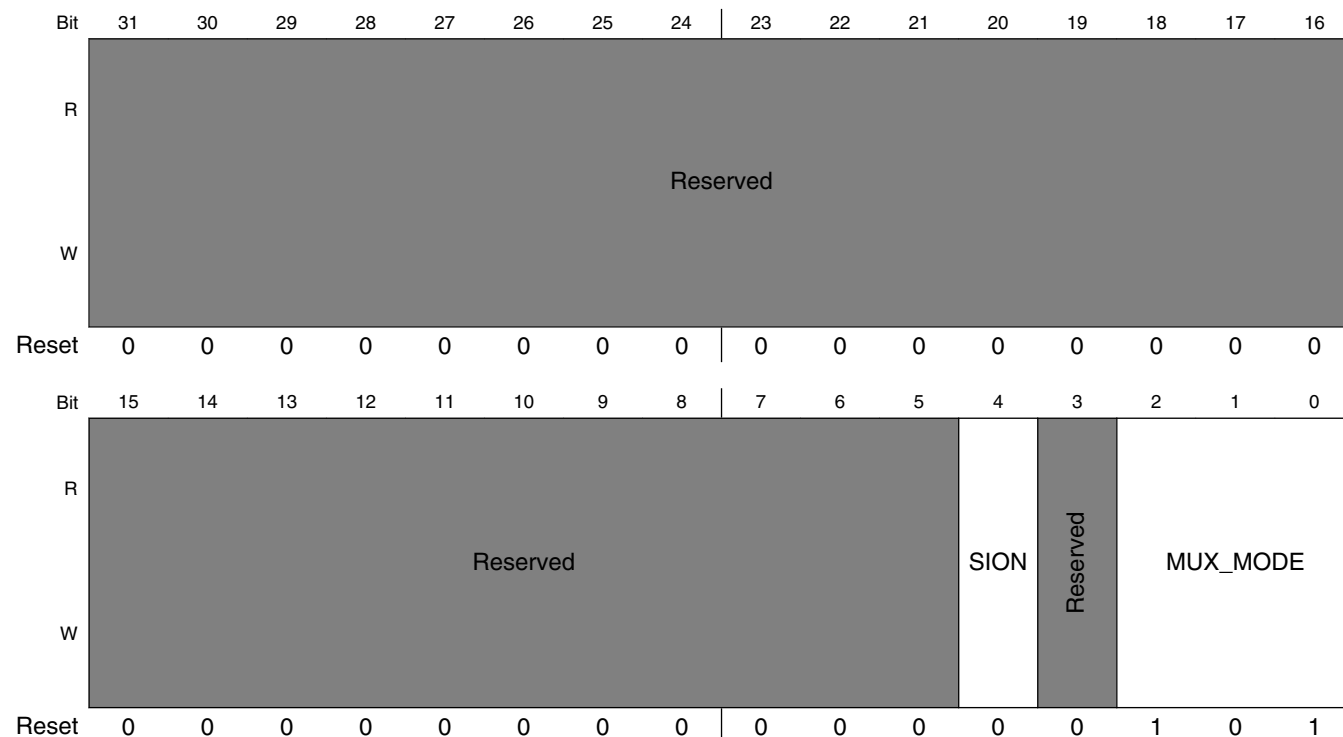
IOMUXC_SW_MUX_CTL_PAD_UART4_RXD field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad UART4_RXD is determined by functionality 1 SION_ENABLED — Force Input Path of pad UART4_RXD
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: UART4_RXD 000 ALT0_UART4_RX — Select mux mode: ALT0 mux port: RX of instance: UART4 001 ALT1_UART2_CTS_B — Select mux mode: ALT1 mux port: CTS_B of instance: UART2 010 ALT2_PCIE1_CLKREQ_B — Select mux mode: ALT2 mux port: CLKREQ_B of instance: PCIE1 101 ALT5_GPIO5_IO28 — Select mux mode: ALT5 mux port: IO28 of instance: GPIO5

8.2.5.144 SW_MUX_CTL_PAD_UART4_TXD SW MUX Control Register (IOMUXC_SW_MUX_CTL_PAD_UART4_TXD)

SW_MUX_CTL Register

Address: 3033_0000h base + 250h offset = 3033_0250h



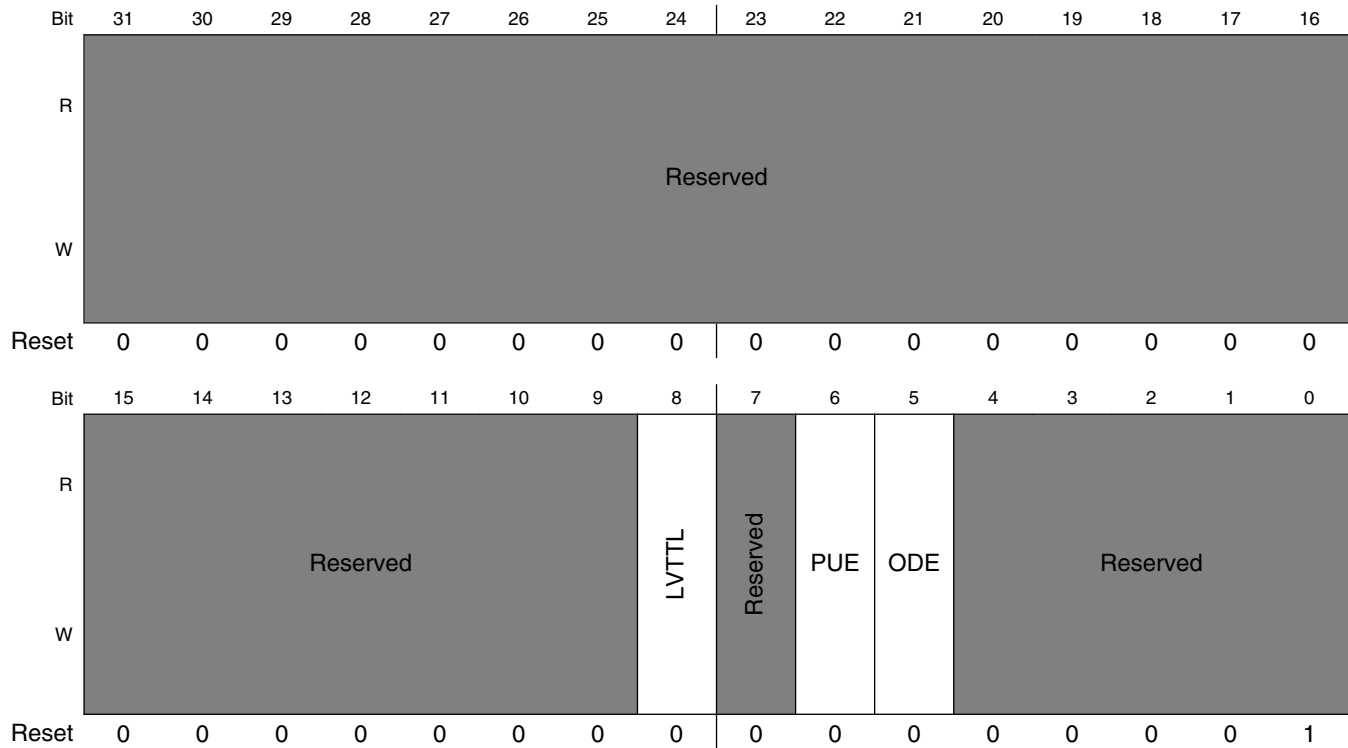
IOMUXC_SW_MUX_CTL_PAD_UART4_TXD field descriptions

Field	Description
31–5 -	This field is reserved. Reserved
4 SION	Software Input On Field Force the select mux mode Input path no matter of MUX_MODE functionality 0 SION_DISABLED — Input Path of pad UART4_TXD is determined by functionality 1 SION_ENABLED — Force Input Path of pad UART4_TXD
3 -	This field is reserved. Reserved
MUX_MODE	MUX Mode Select Field Select 1 of 4 iomux modes to be used for pad: UART4_TXD 000 ALT0_UART4_TX — Select mux mode: ALT0 mux port: TX of instance: UART4 001 ALT1_UART2_RTS_B — Select mux mode: ALT1 mux port: RTS_B of instance: UART2 010 ALT2_PCIE2_CLKREQ_B — Select mux mode: ALT2 mux port: CLKREQ_B of instance: PCIE2 101 ALT5_GPIO5_IO29 — Select mux mode: ALT5 mux port: IO29 of instance: GPIO5

8.2.5.145 SW_PAD_CTL_PAD_TEST_MODE SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_TEST_MODE)

SW_PAD_CTL Register

Address: 3033_0000h base + 254h offset = 3033_0254h



IOMUXC_SW_PAD_CTL_PAD_TEST_MODE field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 -	This field is reserved. Reserved
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_TEST_MODE field descriptions (continued)

Field	Description
5 ODE	Open Drain Enable Field Select one out of next values for pad: TEST_MODE 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
-	This field is reserved. Reserved

8.2.5.146 SW_PAD_CTL_PAD_BOOT_MODE0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_BOOT_MODE0)**SW_PAD_CTL Register**

Address: 3033_0000h base + 258h offset = 3033_0258h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTTL	Reserved	PUE	ODE	Reserved			
W																
Reset	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1

IOMUXC_SW_PAD_CTL_PAD_BOOT_MODE0 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTTL	Lvttl Enable Field

Table continues on the next page...

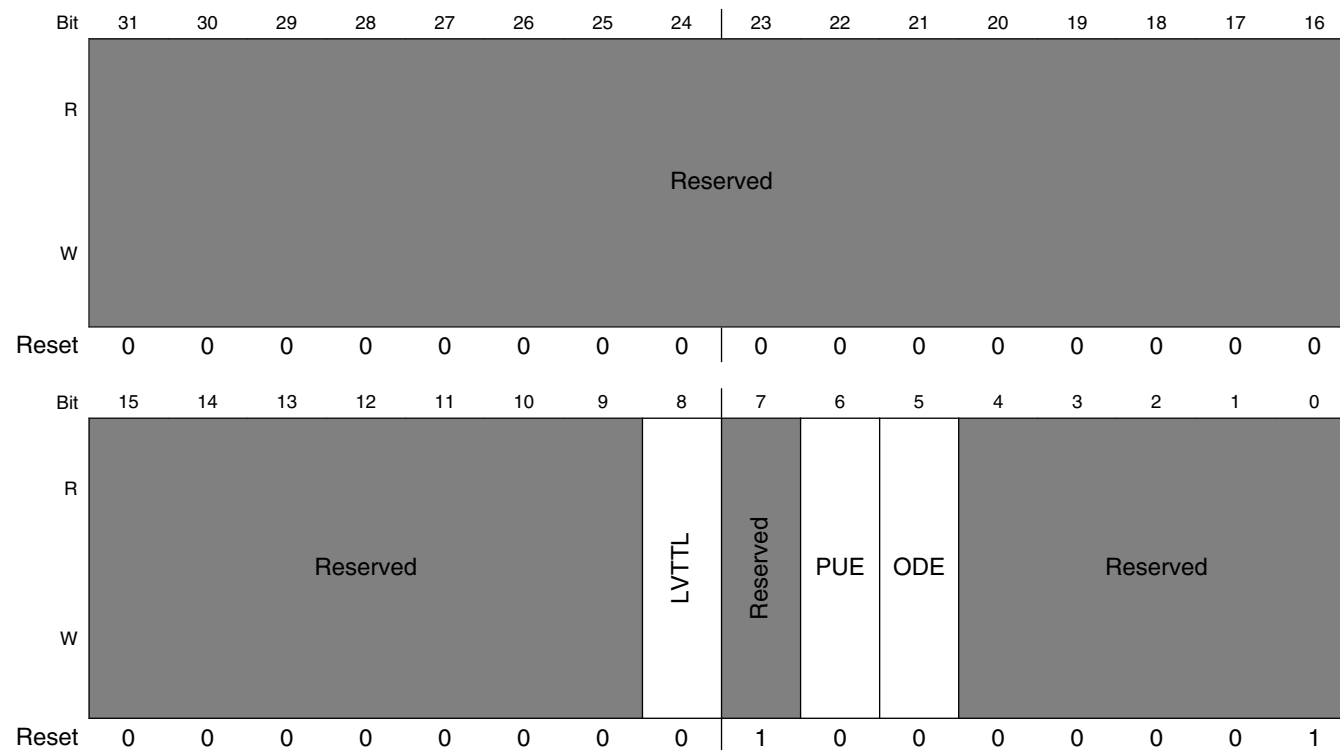
IOMUXC_SW_PAD_CTL_PAD_BOOT_MODE0 field descriptions (continued)

Field	Description
	Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 -	This field is reserved. Reserved
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: BOOT_MODE0 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
-	This field is reserved. Reserved

8.2.5.147 SW_PAD_CTL_PAD_BOOT_MODE1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_BOOT_MODE1)

SW_PAD_CTL Register

Address: 3033_0000h base + 25Ch offset = 3033_025Ch



IOMUXC_SW_PAD_CTL_PAD_BOOT_MODE1 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 -	This field is reserved. Reserved
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_BOOT_MODE1 field descriptions (continued)

Field	Description
5 ODE	Open Drain Enable Field Select one out of next values for pad: BOOT_MODE1 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
-	This field is reserved. Reserved

8.2.5.148 SW_PAD_CTL_PAD_JTAG_MOD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_JTAG_MOD)**SW_PAD_CTL Register**

Address: 3033_0000h base + 260h offset = 3033_0260h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved	VSEL				Reserved		LVTTL	HYS	PUE	ODE	Reserved				
W																
Reset	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	1

IOMUXC_SW_PAD_CTL_PAD_JTAG_MOD field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: JTAG_MOD 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_JTAG_MOD field descriptions (continued)

Field	Description
	110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schmitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: JTAG_MOD 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
-	This field is reserved. Reserved

8.2.5.149 SW_PAD_CTL_PAD_JTAG_TRST_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_JTAG_TRST_B)

SW_PAD_CTL Register

Address: 3033_0000h base + 264h offset = 3033_0264h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	Reserved			
W																
Reset	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1

IOMUXC_SW_PAD_CTL_PAD_JTAG_TRST_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: JTAG_TRST_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_JTAG_TRST_B field descriptions (continued)

Field	Description
-	This field is reserved. Reserved

8.2.5.150 SW_PAD_CTL_PAD_JTAG_TDI SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_JTAG_TDI)

SW_PAD_CTL Register

Address: 3033_0000h base + 268h offset = 3033_0268h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	Reserved			
W																
Reset	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1

IOMUXC_SW_PAD_CTL_PAD_JTAG_TDI field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTTL input 0 Disabled — LVTTTL Disabled 1 Enabled — LVTTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_JTAG_TDI field descriptions (continued)

Field	Description
5 ODE	Open Drain Enable Field Select one out of next values for pad: JTAG_TDI 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
-	This field is reserved. Reserved

8.2.5.151 SW_PAD_CTL_PAD_JTAG_TMS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_JTAG_TMS)**SW_PAD_CTL Register**

Address: 3033_0000h base + 26Ch offset = 3033_026Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	Reserved			
W																
Reset	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1

IOMUXC_SW_PAD_CTL_PAD_JTAG_TMS field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTTL input 0 Disabled — LVTTTL Disabled 1 Enabled — LVTTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_JTAG_TMS field descriptions (continued)

Field	Description
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: JTAG_TMS 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
-	This field is reserved. Reserved

8.2.5.152 SW_PAD_CTL_PAD_JTAG_TCK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_JTAG_TCK)**SW_PAD_CTL Register**

Address: 3033_0000h base + 270h offset = 3033_0270h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	Reserved			
W																
Reset	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1

IOMUXC_SW_PAD_CTL_PAD_JTAG_TCK field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_JTAG_TCK field descriptions (continued)

Field	Description
7 HYS	Schmitt trigger Enable Field Control signal to select Schmitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: JTAG_TCK 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
-	This field is reserved. Reserved

8.2.5.153 SW_PAD_CTL_PAD_JTAG_TDO SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_JTAG_TDO)**SW_PAD_CTL Register**

Address: 3033_0000h base + 274h offset = 3033_0274h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	Reserved			
W																
Reset	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1

IOMUXC_SW_PAD_CTL_PAD_JTAG_TDO field descriptions

Field	Description
31–9 -	This field is reserved. Reserved

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_JTAG_TDO field descriptions (continued)

Field	Description
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schmitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: JTAG_TDO 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
-	This field is reserved. Reserved

8.2.5.154 SW_PAD_CTL_PAD_RTC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_RTC)

SW_PAD_CTL Register

Address: 3033_0000h base + 278h offset = 3033_0278h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTTL	Reserved	PUE	ODE	Reserved			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

IOMUXC_SW_PAD_CTL_PAD_RTC field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTTL	Lvttl Enable Field Control signal to select LVTTTL input 0 Disabled — LVTTTL Disabled 1 Enabled — LVTTTL Enabled
7 -	This field is reserved. Reserved
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_RTC field descriptions (continued)

Field	Description
5 ODE	Open Drain Enable Field Select one out of next values for pad: RTC 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
-	This field is reserved. Reserved

8.2.5.155 SW_PAD_CTL_PAD_PMIC_STBY_REQ SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_PMIC_STBY_REQ)**SW_PAD_CTL Register**

Address: 3033_0000h base + 27Ch offset = 3033_027Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0

IOMUXC_SW_PAD_CTL_PAD_PMIC_STBY_REQ field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_PMIC_STBY_REQ field descriptions (continued)

Field	Description
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: PMIC_STBY_REQ 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: PMIC_STBY_REQ 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: PMIC_STBY_REQ 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.156 SW_PAD_CTL_PAD_PMIC_ON_REQ SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_PMIC_ON_REQ)

SW_PAD_CTL Register

Address: 3033_0000h base + 280h offset = 3033_0280h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		VSEL			Reserved		LVTTL	HYS	PUE	ODE	SRE		DSE		
W																
Reset	0	0	0	1	1	0	0	0	0	1	1	0	1	1	0	0

IOMUXC_SW_PAD_CTL_PAD_PMIC_ON_REQ field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: PMIC_ON_REQ 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode 110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_PMIC_ON_REQ field descriptions (continued)

Field	Description
	0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: PMIC_ON_REQ 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: PMIC_ON_REQ 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: PMIC_ON_REQ 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.157 SW_PAD_CTL_PAD_ONOFF SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ONOFF)

SW_PAD_CTL Register

Address: 3033_0000h base + 284h offset = 3033_0284h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	1	1	0	0	1	1	0	0

IOMUXC_SW_PAD_CTL_PAD_ONOFF field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTTL input 0 Disabled — LVTTTL Disabled 1 Enabled — LVTTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ONOFF 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ONOFF field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ONOFF 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ONOFF 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.158 SW_PAD_CTL_PAD_POR_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_POR_B)**SW_PAD_CTL Register**

Address: 3033_0000h base + 288h offset = 3033_0288h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	1	1	0	0	1	1	0	0

IOMUXC_SW_PAD_CTL_PAD_POR_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: POR_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: POR_B 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: POR_B 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.159 SW_PAD_CTL_PAD_RTC_RESET_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_RTC_RESET_B)

SW_PAD_CTL Register

Address: 3033_0000h base + 28Ch offset = 3033_028Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	1	1	0	0	1	1	0	0

IOMUXC_SW_PAD_CTL_PAD_RTC_RESET_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: RTC_RESET_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_RTC_RESET_B field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: RTC_RESET_B 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: RTC_RESET_B 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.160 SW_PAD_CTL_PAD_GPIO1_IO00 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO00)**SW_PAD_CTL Register**

Address: 3033_0000h base + 290h offset = 3033_0290h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO00 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO00 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO00 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO00 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.161 SW_PAD_CTL_PAD_GPIO1_IO01 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO01)

SW_PAD_CTL Register

Address: 3033_0000h base + 294h offset = 3033_0294h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO01 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO01 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO01 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO01 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO01 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.162 SW_PAD_CTL_PAD_GPIO1_IO02 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO02)**SW_PAD_CTL Register**

Address: 3033_0000h base + 298h offset = 3033_0298h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	1	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO02 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO02 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO02 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO02 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.163 SW_PAD_CTL_PAD_GPIO1_IO03 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO03)

SW_PAD_CTL Register

Address: 3033_0000h base + 29Ch offset = 3033_029Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO03 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO03 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO03 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO03 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO03 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.164 SW_PAD_CTL_PAD_GPIO1_IO04 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO04)**SW_PAD_CTL Register**

Address: 3033_0000h base + 2A0h offset = 3033_02A0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO04 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO04 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO04 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO04 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.165 SW_PAD_CTL_PAD_GPIO1_IO05 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO05)

SW_PAD_CTL Register

Address: 3033_0000h base + 2A4h offset = 3033_02A4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	1	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO05 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO05 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO05 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO05 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO05 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.166 SW_PAD_CTL_PAD_GPIO1_IO06 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO06)**SW_PAD_CTL Register**

Address: 3033_0000h base + 2A8h offset = 3033_02A8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO06 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO06 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO06 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO06 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.167 SW_PAD_CTL_PAD_GPIO1_IO07 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO07)

SW_PAD_CTL Register

Address: 3033_0000h base + 2ACh offset = 3033_02ACh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		VSEL			Reserved		LVTTL	HYS	PUE	ODE	SRE		DSE		
W																
Reset	0	0	0	1	1	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO07 field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: GPIO1_IO07 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode 110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO07 field descriptions (continued)

Field	Description
	0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO07 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO07 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO07 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.168 SW_PAD_CTL_PAD_GPIO1_IO08 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO08)

SW_PAD_CTL Register

Address: 3033_0000h base + 2B0h offset = 3033_02B0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO08 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO08 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO08 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO08 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO08 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.169 SW_PAD_CTL_PAD_GPIO1_IO09 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO09)**SW_PAD_CTL Register**

Address: 3033_0000h base + 2B4h offset = 3033_02B4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO09 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO09 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO09 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO09 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.170 SW_PAD_CTL_PAD_GPIO1_IO10 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO10)

SW_PAD_CTL Register

Address: 3033_0000h base + 2B8h offset = 3033_02B8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO10 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO10 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO10 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO10 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO10 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.171 SW_PAD_CTL_PAD_GPIO1_IO11 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO11)**SW_PAD_CTL Register**

Address: 3033_0000h base + 2BCh offset = 3033_02BCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO11 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO11 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO11 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO11 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.172 SW_PAD_CTL_PAD_GPIO1_IO12 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO12)

SW_PAD_CTL Register

Address: 3033_0000h base + 2C0h offset = 3033_02C0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO12 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO12 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO12 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO12 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO12 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.173 SW_PAD_CTL_PAD_GPIO1_IO13 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO13)**SW_PAD_CTL Register**

Address: 3033_0000h base + 2C4h offset = 3033_02C4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO13 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO13 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO13 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO13 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.174 SW_PAD_CTL_PAD_GPIO1_IO14 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO14)

SW_PAD_CTL Register

Address: 3033_0000h base + 2C8h offset = 3033_02C8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO14 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO14 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO14 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO14 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO14 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.175 SW_PAD_CTL_PAD_GPIO1_IO15 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO15)**SW_PAD_CTL Register**

Address: 3033_0000h base + 2CCCh offset = 3033_02CCCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_GPIO1_IO15 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: GPIO1_IO15 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: GPIO1_IO15 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: GPIO1_IO15 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.176 SW_PAD_CTL_PAD_ENET_MDC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_MDC)

SW_PAD_CTL Register

Address: 3033_0000h base + 2D0h offset = 3033_02D0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_ENET_MDC field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_MDC 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ENET_MDC field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_MDC 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_MDC 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.177 SW_PAD_CTL_PAD_ENET_MDIO SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_MDIO)**SW_PAD_CTL Register**

Address: 3033_0000h base + 2D4h offset = 3033_02D4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ENET_MDIO field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_MDIO 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_MDIO 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_MDIO 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.178 SW_PAD_CTL_PAD_ENET_TD3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_TD3)

SW_PAD_CTL Register

Address: 3033_0000h base + 2D8h offset = 3033_02D8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ENET_TD3 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_TD3 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ENET_TD3 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_TD3 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_TD3 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.179 SW_PAD_CTL_PAD_ENET_TD2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_TD2)**SW_PAD_CTL Register**

Address: 3033_0000h base + 2DCh offset = 3033_02DCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ENET_TD2 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_TD2 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_TD2 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_TD2 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.180 SW_PAD_CTL_PAD_ENET_TD1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_TD1)

SW_PAD_CTL Register

Address: 3033_0000h base + 2E0h offset = 3033_02E0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ENET_TD1 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_TD1 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ENET_TD1 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_TD1 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_TD1 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.181 SW_PAD_CTL_PAD_ENET_TD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_TD0)

SW_PAD_CTL Register

Address: 3033_0000h base + 2E4h offset = 3033_02E4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ENET_TD0 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_TD0 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_TD0 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_TD0 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.182 SW_PAD_CTL_PAD_ENET_TX_CTL SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_TX_CTL)

SW_PAD_CTL Register

Address: 3033_0000h base + 2E8h offset = 3033_02E8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved																
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved		VSEL				Reserved		LVTTL	HYS	PUE	ODE	SRE		DSE		
W																	
Reset	0	0	0	1	1	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ENET_TX_CTL field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: ENET_TX_CTL 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode 110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ENET_TX_CTL field descriptions (continued)

Field	Description
	0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_TX_CTL 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_TX_CTL 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_TX_CTL 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.183 SW_PAD_CTL_PAD_ENET_TXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_TXC)

SW_PAD_CTL Register

Address: 3033_0000h base + 2ECh offset = 3033_02ECh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ENET_TXC field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_TXC 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ENET_TXC field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_TXC 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_TXC 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.184 SW_PAD_CTL_PAD_ENET_RX_CTL SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_RX_CTL)

SW_PAD_CTL Register

Address: 3033_0000h base + 2F0h offset = 3033_02F0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ENET_RX_CTL field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_RX_CTL 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_RX_CTL 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_RX_CTL 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.185 SW_PAD_CTL_PAD_ENET_RXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_RXC)

SW_PAD_CTL Register

Address: 3033_0000h base + 2F4h offset = 3033_02F4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ENET_RXC field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_RXC 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ENET_RXC field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_RXC 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_RXC 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.186 SW_PAD_CTL_PAD_ENET_RD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_RD0)**SW_PAD_CTL Register**

Address: 3033_0000h base + 2F8h offset = 3033_02F8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_ENET_RD0 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_RD0 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_RD0 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_RD0 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.187 SW_PAD_CTL_PAD_ENET_RD1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_RD1)

SW_PAD_CTL Register

Address: 3033_0000h base + 2FCh offset = 3033_02FCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ENET_RD1 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_RD1 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ENET_RD1 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_RD1 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_RD1 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.188 SW_PAD_CTL_PAD_ENET_RD2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_RD2)**SW_PAD_CTL Register**

Address: 3033_0000h base + 300h offset = 3033_0300h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_ENET_RD2 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_RD2 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_RD2 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_RD2 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.189 SW_PAD_CTL_PAD_ENET_RD3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ENET_RD3)

SW_PAD_CTL Register

Address: 3033_0000h base + 304h offset = 3033_0304h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ENET_RD3 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ENET_RD3 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ENET_RD3 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ENET_RD3 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ENET_RD3 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.190 SW_PAD_CTL_PAD_SD1_CLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_CLK)

SW_PAD_CTL Register

Address: 3033_0000h base + 308h offset = 3033_0308h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD1_CLK field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD1_CLK 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD1_CLK 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD1_CLK 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.191 SW_PAD_CTL_PAD_SD1_CMD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_CMD)

SW_PAD_CTL Register

Address: 3033_0000h base + 30Ch offset = 3033_030Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD1_CMD field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD1_CMD 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SD1_CMD field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD1_CMD 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD1_CMD 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.192 SW_PAD_CTL_PAD_SD1_DATA0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA0)**SW_PAD_CTL Register**

Address: 3033_0000h base + 310h offset = 3033_0310h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD1_DATA0 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD1_DATA0 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD1_DATA0 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD1_DATA0 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.193 SW_PAD_CTL_PAD_SD1_DATA1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA1)

SW_PAD_CTL Register

Address: 3033_0000h base + 314h offset = 3033_0314h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD1_DATA1 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD1_DATA1 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SD1_DATA1 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD1_DATA1 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD1_DATA1 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.194 SW_PAD_CTL_PAD_SD1_DATA2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA2)**SW_PAD_CTL Register**

Address: 3033_0000h base + 318h offset = 3033_0318h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD1_DATA2 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD1_DATA2 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD1_DATA2 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD1_DATA2 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.195 SW_PAD_CTL_PAD_SD1_DATA3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA3)

SW_PAD_CTL Register

Address: 3033_0000h base + 31Ch offset = 3033_031Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		VSEL			Reserved		LVTTL	HYS	PUE	ODE	SRE		DSE		
W																
Reset	0	0	0	1	1	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD1_DATA3 field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: SD1_DATA3 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode 110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SD1_DATA3 field descriptions (continued)

Field	Description
	0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD1_DATA3 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD1_DATA3 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD1_DATA3 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.196 SW_PAD_CTL_PAD_SD1_DATA4 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA4)

SW_PAD_CTL Register

Address: 3033_0000h base + 320h offset = 3033_0320h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD1_DATA4 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD1_DATA4 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SD1_DATA4 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD1_DATA4 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD1_DATA4 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.197 SW_PAD_CTL_PAD_SD1_DATA5 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA5)**SW_PAD_CTL Register**

Address: 3033_0000h base + 324h offset = 3033_0324h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD1_DATA5 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD1_DATA5 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD1_DATA5 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD1_DATA5 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.198 SW_PAD_CTL_PAD_SD1_DATA6 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA6)

SW_PAD_CTL Register

Address: 3033_0000h base + 328h offset = 3033_0328h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD1_DATA6 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD1_DATA6 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SD1_DATA6 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD1_DATA6 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD1_DATA6 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.199 SW_PAD_CTL_PAD_SD1_DATA7 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_DATA7)**SW_PAD_CTL Register**

Address: 3033_0000h base + 32Ch offset = 3033_032Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD1_DATA7 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD1_DATA7 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD1_DATA7 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD1_DATA7 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.200 SW_PAD_CTL_PAD_SD1_RESET_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_RESET_B)

SW_PAD_CTL Register

Address: 3033_0000h base + 330h offset = 3033_0330h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD1_RESET_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD1_RESET_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SD1_RESET_B field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD1_RESET_B 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD1_RESET_B 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.201 SW_PAD_CTL_PAD_SD1_STROBE SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD1_STROBE)

SW_PAD_CTL Register

Address: 3033_0000h base + 334h offset = 3033_0334h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_SD1_STROBE field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD1_STROBE 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD1_STROBE 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD1_STROBE 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.202 SW_PAD_CTL_PAD_SD2_CD_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_CD_B)

SW_PAD_CTL Register

Address: 3033_0000h base + 338h offset = 3033_0338h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD2_CD_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD2_CD_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SD2_CD_B field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD2_CD_B 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD2_CD_B 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.203 SW_PAD_CTL_PAD_SD2_CLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_CLK)

SW_PAD_CTL Register

Address: 3033_0000h base + 33Ch offset = 3033_033Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD2_CLK field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD2_CLK 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD2_CLK 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD2_CLK 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.204 SW_PAD_CTL_PAD_SD2_CMD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_CMD)

SW_PAD_CTL Register

Address: 3033_0000h base + 340h offset = 3033_0340h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD2_CMD field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD2_CMD 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SD2_CMD field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD2_CMD 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD2_CMD 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.205 SW_PAD_CTL_PAD_SD2_DATA0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_DATA0)**SW_PAD_CTL Register**

Address: 3033_0000h base + 344h offset = 3033_0344h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		VSEL			Reserved		LVTTL	HYS	PUE	ODE	SRE		DSE		
W																
Reset	0	0	0	1	1	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD2_DATA0 field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: SD2_DATA0 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode 110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD2_DATA0 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD2_DATA0 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SD2_DATA0 field descriptions (continued)

Field	Description
	Select one out of next values for pad: SD2_DATA0
000	HI-Z — Output driver is disabled (Hi-Z State)
001	255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V
010	105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V
011	75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V
100	85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V
101	65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V
110	45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V
111	40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.206 SW_PAD_CTL_PAD_SD2_DATA1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_DATA1)**SW_PAD_CTL Register**

Address: 3033_0000h base + 348h offset = 3033_0348h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_SD2_DATA1 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SD2_DATA1 field descriptions (continued)

Field	Description
	0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD2_DATA1 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD2_DATA1 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD2_DATA1 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.207 SW_PAD_CTL_PAD_SD2_DATA2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_DATA2)

SW_PAD_CTL Register

Address: 3033_0000h base + 34Ch offset = 3033_034Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD2_DATA2 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD2_DATA2 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SD2_DATA2 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD2_DATA2 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD2_DATA2 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.208 SW_PAD_CTL_PAD_SD2_DATA3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_DATA3)**SW_PAD_CTL Register**

Address: 3033_0000h base + 350h offset = 3033_0350h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD2_DATA3 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD2_DATA3 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD2_DATA3 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD2_DATA3 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.209 SW_PAD_CTL_PAD_SD2_RESET_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_RESET_B)

SW_PAD_CTL Register

Address: 3033_0000h base + 354h offset = 3033_0354h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD2_RESET_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTTL	Lvttl Enable Field Control signal to select LVTTTL input 0 Disabled — LVTTTL Disabled 1 Enabled — LVTTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD2_RESET_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SD2_RESET_B field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD2_RESET_B 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD2_RESET_B 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.210 SW_PAD_CTL_PAD_SD2_WP SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SD2_WP)**SW_PAD_CTL Register**

Address: 3033_0000h base + 358h offset = 3033_0358h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SD2_WP field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SD2_WP 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SD2_WP 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SD2_WP 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.211 SW_PAD_CTL_PAD_NAND_ALE SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_ALE)

SW_PAD_CTL Register

Address: 3033_0000h base + 35Ch offset = 3033_035Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_ALE field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_ALE 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_NAND_ALE field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_ALE 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_ALE 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.212 SW_PAD_CTL_PAD_NAND_CE0_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_CE0_B)

SW_PAD_CTL Register

Address: 3033_0000h base + 360h offset = 3033_0360h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_CE0_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_CE0_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_CE0_B 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_CE0_B 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.213 SW_PAD_CTL_PAD_NAND_CE1_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_CE1_B)

SW_PAD_CTL Register

Address: 3033_0000h base + 364h offset = 3033_0364h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_CE1_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_CE1_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_NAND_CE1_B field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_CE1_B 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_CE1_B 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.214 SW_PAD_CTL_PAD_NAND_CE2_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_CE2_B)

SW_PAD_CTL Register

Address: 3033_0000h base + 368h offset = 3033_0368h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_CE2_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_CE2_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_CE2_B 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_CE2_B 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.215 SW_PAD_CTL_PAD_NAND_CE3_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_CE3_B)

SW_PAD_CTL Register

Address: 3033_0000h base + 36Ch offset = 3033_036Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_CE3_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_CE3_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_NAND_CE3_B field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_CE3_B 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_CE3_B 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.216 SW_PAD_CTL_PAD_NAND_CLE SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_CLE)**SW_PAD_CTL Register**

Address: 3033_0000h base + 370h offset = 3033_0370h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_CLE field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_CLE 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_CLE 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_CLE 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.217 SW_PAD_CTL_PAD_NAND_DATA00 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA00)

SW_PAD_CTL Register

Address: 3033_0000h base + 374h offset = 3033_0374h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA00 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_DATA00 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA00 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_DATA00 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_DATA00 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.218 SW_PAD_CTL_PAD_NAND_DATA01 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA01)

SW_PAD_CTL Register

Address: 3033_0000h base + 378h offset = 3033_0378h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA01 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_DATA01 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_DATA01 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_DATA01 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.219 SW_PAD_CTL_PAD_NAND_DATA02 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA02)

SW_PAD_CTL Register

Address: 3033_0000h base + 37Ch offset = 3033_037Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA02 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_DATA02 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA02 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_DATA02 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_DATA02 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.220 SW_PAD_CTL_PAD_NAND_DATA03 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA03)

SW_PAD_CTL Register

Address: 3033_0000h base + 380h offset = 3033_0380h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		VSEL			Reserved		LVTTL	HYS	PUE	ODE	SRE		DSE		
W																
Reset	0	0	0	1	1	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA03 field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: NAND_DATA03 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode 110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_DATA03 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_DATA03 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA03 field descriptions (continued)

Field	Description
	Select one out of next values for pad: NAND_DATA03
000	HI-Z — Output driver is disabled (Hi-Z State)
001	255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V
010	105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V
011	75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V
100	85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V
101	65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V
110	45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V
111	40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.221 SW_PAD_CTL_PAD_NAND_DATA04 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA04)**SW_PAD_CTL Register**

Address: 3033_0000h base + 384h offset = 3033_0384h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA04 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTTL input 0 Disabled — LVTTTL Disabled 1 Enabled — LVTTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA04 field descriptions (continued)

Field	Description
	0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_DATA04 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_DATA04 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_DATA04 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.222 SW_PAD_CTL_PAD_NAND_DATA05 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA05)

SW_PAD_CTL Register

Address: 3033_0000h base + 388h offset = 3033_0388h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA05 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_DATA05 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA05 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_DATA05 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_DATA05 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.223 SW_PAD_CTL_PAD_NAND_DATA06 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA06)

SW_PAD_CTL Register

Address: 3033_0000h base + 38Ch offset = 3033_038Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA06 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_DATA06 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_DATA06 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_DATA06 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.224 SW_PAD_CTL_PAD_NAND_DATA07 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DATA07)

SW_PAD_CTL Register

Address: 3033_0000h base + 390h offset = 3033_0390h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA07 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_DATA07 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_NAND_DATA07 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_DATA07 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_DATA07 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.225 SW_PAD_CTL_PAD_NAND_DQS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_DQS)**SW_PAD_CTL Register**

Address: 3033_0000h base + 394h offset = 3033_0394h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_DQS field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_DQS 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_DQS 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_DQS 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.226 SW_PAD_CTL_PAD_NAND_RE_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_RE_B)

SW_PAD_CTL Register

Address: 3033_0000h base + 398h offset = 3033_0398h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_RE_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_RE_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_NAND_RE_B field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_RE_B 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_RE_B 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.227 SW_PAD_CTL_PAD_NAND_READY_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_READY_B)**SW_PAD_CTL Register**

Address: 3033_0000h base + 39Ch offset = 3033_039Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_READY_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_READY_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_READY_B 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_READY_B 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.228 SW_PAD_CTL_PAD_NAND_WE_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_WE_B)

SW_PAD_CTL Register

Address: 3033_0000h base + 3A0h offset = 3033_03A0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_WE_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_WE_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_NAND_WE_B field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_WE_B 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_WE_B 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.229 SW_PAD_CTL_PAD_NAND_WP_B SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_NAND_WP_B)

SW_PAD_CTL Register

Address: 3033_0000h base + 3A4h offset = 3033_03A4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_NAND_WP_B field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: NAND_WP_B 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: NAND_WP_B 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: NAND_WP_B 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.230 SW_PAD_CTL_PAD_SAI5_RXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_RXFS)

SW_PAD_CTL Register

Address: 3033_0000h base + 3A8h offset = 3033_03A8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI5_RXFS field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI5_RXFS 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI5_RXFS field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI5_RXFS 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI5_RXFS 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.231 SW_PAD_CTL_PAD_SAI5_RXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_RXC)**SW_PAD_CTL Register**

Address: 3033_0000h base + 3ACh offset = 3033_03ACh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_SAI5_RXC field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI5_RXC 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI5_RXC 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI5_RXC 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.232 SW_PAD_CTL_PAD_SAI5_RXD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD0)

SW_PAD_CTL Register

Address: 3033_0000h base + 3B0h offset = 3033_03B0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD0 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI5_RXD0 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD0 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI5_RXD0 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI5_RXD0 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.233 SW_PAD_CTL_PAD_SAI5_RXD1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD1)**SW_PAD_CTL Register**

Address: 3033_0000h base + 3B4h offset = 3033_03B4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved																
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved		VSEL				Reserved		LVTTL	HYS	PUE	ODE	SRE		DSE		
W																	
Reset	0	0	0	1	1	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD1 field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: SAI5_RXD1 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode 110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schmitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI5_RXD1 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI5_RXD1 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD1 field descriptions (continued)

Field	Description
	Select one out of next values for pad: SAI5_RXD1
000	HI-Z — Output driver is disabled (Hi-Z State)
001	255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V
010	105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V
011	75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V
100	85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V
101	65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V
110	45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V
111	40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.234 SW_PAD_CTL_PAD_SAI5_RXD2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD2)**SW_PAD_CTL Register**

Address: 3033_0000h base + 3B8h offset = 3033_03B8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD2 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTTL	Lvttl Enable Field Control signal to select LVTTTL input 0 Disabled — LVTTTL Disabled 1 Enabled — LVTTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD2 field descriptions (continued)

Field	Description
	0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI5_RXD2 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI5_RXD2 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI5_RXD2 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.235 SW_PAD_CTL_PAD_SAI5_RXD3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD3)

SW_PAD_CTL Register

Address: 3033_0000h base + 3BCh offset = 3033_03BCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD3 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI5_RXD3 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI5_RXD3 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI5_RXD3 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI5_RXD3 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.236 SW_PAD_CTL_PAD_SAI5_MCLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI5_MCLK)**SW_PAD_CTL Register**

Address: 3033_0000h base + 3C0h offset = 3033_03C0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_SAI5_MCLK field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI5_MCLK 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI5_MCLK 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI5_MCLK 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.237 SW_PAD_CTL_PAD_SAI1_RXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXFS)

SW_PAD_CTL Register

Address: 3033_0000h base + 3C4h offset = 3033_03C4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXFS field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_RXFS 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXFS field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_RXFS 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_RXFS 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.238 SW_PAD_CTL_PAD_SAI1_RXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXC)**SW_PAD_CTL Register**

Address: 3033_0000h base + 3C8h offset = 3033_03C8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXC field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_RXC 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_RXC 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_RXC 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.239 SW_PAD_CTL_PAD_SAI1_RXD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD0)

SW_PAD_CTL Register

Address: 3033_0000h base + 3CCCh offset = 3033_03CCCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD0 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTTL input 0 Disabled — LVTTTL Disabled 1 Enabled — LVTTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_RXD0 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD0 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_RXD0 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_RXD0 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.240 SW_PAD_CTL_PAD_SAI1_RXD1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD1)**SW_PAD_CTL Register**

Address: 3033_0000h base + 3D0h offset = 3033_03D0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD1 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_RXD1 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_RXD1 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_RXD1 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.241 SW_PAD_CTL_PAD_SAI1_RXD2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD2)

SW_PAD_CTL Register

Address: 3033_0000h base + 3D4h offset = 3033_03D4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD2 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_RXD2 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD2 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_RXD2 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_RXD2 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.242 SW_PAD_CTL_PAD_SAI1_RXD3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD3)**SW_PAD_CTL Register**

Address: 3033_0000h base + 3D8h offset = 3033_03D8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD3 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_RXD3 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_RXD3 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_RXD3 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.243 SW_PAD_CTL_PAD_SAI1_RXD4 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD4)

SW_PAD_CTL Register

Address: 3033_0000h base + 3DCh offset = 3033_03DCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD4 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_RXD4 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD4 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_RXD4 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_RXD4 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.244 SW_PAD_CTL_PAD_SAI1_RXD5 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD5)**SW_PAD_CTL Register**

Address: 3033_0000h base + 3E0h offset = 3033_03E0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD5 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_RXD5 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_RXD5 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_RXD5 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.245 SW_PAD_CTL_PAD_SAI1_RXD6 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD6)

SW_PAD_CTL Register

Address: 3033_0000h base + 3E4h offset = 3033_03E4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD6 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_RXD6 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD6 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_RXD6 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_RXD6 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.246 SW_PAD_CTL_PAD_SAI1_RXD7 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD7)**SW_PAD_CTL Register**

Address: 3033_0000h base + 3E8h offset = 3033_03E8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_RXD7 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_RXD7 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_RXD7 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_RXD7 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.247 SW_PAD_CTL_PAD_SAI1_TXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXFS)

SW_PAD_CTL Register

Address: 3033_0000h base + 3ECh offset = 3033_03ECh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved																
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved		VSEL				Reserved		LVTTL	HYS	PUE	ODE	SRE		DSE		
W																	
Reset	0	0	0	1	1	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXFS field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: SAI1_TXFS 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode 110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXFS field descriptions (continued)

Field	Description
	0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_TXFS 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_TXFS 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_TXFS 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.248 SW_PAD_CTL_PAD_SAI1_TXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXC)

SW_PAD_CTL Register

Address: 3033_0000h base + 3F0h offset = 3033_03F0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXC field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_TXC 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXC field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_TXC 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_TXC 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.249 SW_PAD_CTL_PAD_SAI1_TXD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD0)**SW_PAD_CTL Register**

Address: 3033_0000h base + 3F4h offset = 3033_03F4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD0 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_TXD0 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_TXD0 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_TXD0 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.250 SW_PAD_CTL_PAD_SAI1_TXD1 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD1)

SW_PAD_CTL Register

Address: 3033_0000h base + 3F8h offset = 3033_03F8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD1 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_TXD1 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD1 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_TXD1 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_TXD1 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.251 SW_PAD_CTL_PAD_SAI1_TXD2 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD2)**SW_PAD_CTL Register**

Address: 3033_0000h base + 3FCCh offset = 3033_03FCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD2 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_TXD2 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_TXD2 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_TXD2 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.252 SW_PAD_CTL_PAD_SAI1_TXD3 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD3)

SW_PAD_CTL Register

Address: 3033_0000h base + 400h offset = 3033_0400h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD3 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_TXD3 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD3 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_TXD3 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_TXD3 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.253 SW_PAD_CTL_PAD_SAI1_TXD4 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD4)**SW_PAD_CTL Register**

Address: 3033_0000h base + 404h offset = 3033_0404h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD4 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_TXD4 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_TXD4 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_TXD4 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.254 SW_PAD_CTL_PAD_SAI1_TXD5 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD5)

SW_PAD_CTL Register

Address: 3033_0000h base + 408h offset = 3033_0408h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD5 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_TXD5 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD5 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_TXD5 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_TXD5 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.255 SW_PAD_CTL_PAD_SAI1_TXD6 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD6)**SW_PAD_CTL Register**

Address: 3033_0000h base + 40Ch offset = 3033_040Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD6 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_TXD6 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_TXD6 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_TXD6 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.256 SW_PAD_CTL_PAD_SAI1_TXD7 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD7)

SW_PAD_CTL Register

Address: 3033_0000h base + 410h offset = 3033_0410h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD7 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_TXD7 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI1_TXD7 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_TXD7 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_TXD7 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.257 SW_PAD_CTL_PAD_SAI1_MCLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI1_MCLK)**SW_PAD_CTL Register**

Address: 3033_0000h base + 414h offset = 3033_0414h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI1_MCLK field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI1_MCLK 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI1_MCLK 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI1_MCLK 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.258 SW_PAD_CTL_PAD_SAI2_RXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_RXFS)

SW_PAD_CTL Register

Address: 3033_0000h base + 418h offset = 3033_0418h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI2_RXFS field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI2_RXFS 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI2_RXFS field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI2_RXFS 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI2_RXFS 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.259 SW_PAD_CTL_PAD_SAI2_RXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_RXC)**SW_PAD_CTL Register**

Address: 3033_0000h base + 41Ch offset = 3033_041Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI2_RXC field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI2_RXC 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI2_RXC 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI2_RXC 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.260 SW_PAD_CTL_PAD_SAI2_RXD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_RXD0)

SW_PAD_CTL Register

Address: 3033_0000h base + 420h offset = 3033_0420h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI2_RXD0 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI2_RXD0 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI2_RXD0 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI2_RXD0 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI2_RXD0 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.261 SW_PAD_CTL_PAD_SAI2_TXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_TXFS)**SW_PAD_CTL Register**

Address: 3033_0000h base + 424h offset = 3033_0424h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved																
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved		VSEL				Reserved		LVTTL	HYS	PUE	ODE	SRE		DSE		
W																	
Reset	0	0	0	1	1	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI2_TXFS field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: SAI2_TXFS 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode 110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI2_TXFS 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI2_TXFS 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI2_TXFS field descriptions (continued)

Field	Description
	Select one out of next values for pad: SAI2_TXFS
000	HI-Z — Output driver is disabled (Hi-Z State)
001	255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V
010	105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V
011	75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V
100	85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V
101	65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V
110	45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V
111	40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.262 SW_PAD_CTL_PAD_SAI2_TXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_TXC)**SW_PAD_CTL Register**

Address: 3033_0000h base + 428h offset = 3033_0428h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_SAI2_TXC field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTTL input 0 Disabled — LVTTTL Disabled 1 Enabled — LVTTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI2_TXC field descriptions (continued)

Field	Description
	0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI2_TXC 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI2_TXC 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI2_TXC 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.263 SW_PAD_CTL_PAD_SAI2_TXD0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_TXD0)

SW_PAD_CTL Register

Address: 3033_0000h base + 42Ch offset = 3033_042Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI2_TXD0 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI2_TXD0 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI2_TXD0 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI2_TXD0 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI2_TXD0 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.264 SW_PAD_CTL_PAD_SAI2_MCLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI2_MCLK)**SW_PAD_CTL Register**

Address: 3033_0000h base + 430h offset = 3033_0430h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI2_MCLK field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI2_MCLK 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI2_MCLK 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI2_MCLK 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.265 SW_PAD_CTL_PAD_SAI3_RXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_RXFS)

SW_PAD_CTL Register

Address: 3033_0000h base + 434h offset = 3033_0434h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_SAI3_RXFS field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI3_RXFS 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI3_RXFS field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI3_RXFS 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI3_RXFS 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.266 SW_PAD_CTL_PAD_SAI3_RXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_RXC)**SW_PAD_CTL Register**

Address: 3033_0000h base + 438h offset = 3033_0438h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_SAI3_RXC field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI3_RXC 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI3_RXC 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI3_RXC 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.267 SW_PAD_CTL_PAD_SAI3_RXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_RXD)

SW_PAD_CTL Register

Address: 3033_0000h base + 43Ch offset = 3033_043Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI3_RXD field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI3_RXD 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI3_RXD field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI3_RXD 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI3_RXD 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.268 SW_PAD_CTL_PAD_SAI3_TXFS SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_TXFS)**SW_PAD_CTL Register**

Address: 3033_0000h base + 440h offset = 3033_0440h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI3_TXFS field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI3_TXFS 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI3_TXFS 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI3_TXFS 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.269 SW_PAD_CTL_PAD_SAI3_TXC SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_TXC)

SW_PAD_CTL Register

Address: 3033_0000h base + 444h offset = 3033_0444h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		VSEL				Reserved		LVTTL	HYS	PUE	ODE	SRE		DSE	
W																
Reset	0	0	0	1	1	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_SAI3_TXC field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: SAI3_TXC 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode 110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI3_TXC field descriptions (continued)

Field	Description
	0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI3_TXC 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI3_TXC 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI3_TXC 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.270 SW_PAD_CTL_PAD_SAI3_TXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_TXD)

SW_PAD_CTL Register

Address: 3033_0000h base + 448h offset = 3033_0448h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI3_TXD field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI3_TXD 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SAI3_TXD field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI3_TXD 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI3_TXD 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.271 SW_PAD_CTL_PAD_SAI3_MCLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SAI3_MCLK)**SW_PAD_CTL Register**

Address: 3033_0000h base + 44Ch offset = 3033_044Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SAI3_MCLK field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SAI3_MCLK 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SAI3_MCLK 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SAI3_MCLK 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.272 SW_PAD_CTL_PAD_SPDIF_TX SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SPDIF_TX)

SW_PAD_CTL Register

Address: 3033_0000h base + 450h offset = 3033_0450h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SPDIF_TX field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SPDIF_TX 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SPDIF_TX field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SPDIF_TX 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SPDIF_TX 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.273 SW_PAD_CTL_PAD_SPDIF_RX SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SPDIF_RX)**SW_PAD_CTL Register**

Address: 3033_0000h base + 454h offset = 3033_0454h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SPDIF_RX field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SPDIF_RX 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SPDIF_RX 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SPDIF_RX 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.274 SW_PAD_CTL_PAD_SPDIF_EXT_CLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_SPDIF_EXT_CLK)

SW_PAD_CTL Register

Address: 3033_0000h base + 458h offset = 3033_0458h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_SPDIF_EXT_CLK field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: SPDIF_EXT_CLK 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_SPDIF_EXT_CLK field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: SPDIF_EXT_CLK 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: SPDIF_EXT_CLK 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.275 SW_PAD_CTL_PAD_ECSPi1_SCLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSPi1_SCLK)

SW_PAD_CTL Register

Address: 3033_0000h base + 45Ch offset = 3033_045Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_ECSPi1_SCLK field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTTL input 0 Disabled — LVTTTL Disabled 1 Enabled — LVTTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ECSPi1_SCLK 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ECSPi1_SCLK 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ECSPi1_SCLK 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.276 SW_PAD_CTL_PAD_ECSPi1_MOSI SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSPi1_MOSI)

SW_PAD_CTL Register

Address: 3033_0000h base + 460h offset = 3033_0460h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ECSPi1_MOSI field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTTL	Lvttl Enable Field Control signal to select LVTTTL input 0 Disabled — LVTTTL Disabled 1 Enabled — LVTTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ECSPi1_MOSI 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ECSPi1_MOSI field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ECSPi1_MOSI 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ECSPi1_MOSI 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.277 SW_PAD_CTL_PAD_ECSPi1_MISO SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSPi1_MISO)

SW_PAD_CTL Register

Address: 3033_0000h base + 464h offset = 3033_0464h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ECSP11_MISO field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ECSP11_MISO 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ECSP11_MISO 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ECSP11_MISO 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.278 SW_PAD_CTL_PAD_ECSPi1_SS0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSPi1_SS0)

SW_PAD_CTL Register

Address: 3033_0000h base + 468h offset = 3033_0468h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ECSPi1_SS0 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ECSPi1_SS0 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ECSPi1_SS0 field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ECSPi1_SS0 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ECSPi1_SS0 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.279 SW_PAD_CTL_PAD_ECSPi2_SCLK SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSPi2_SCLK)

SW_PAD_CTL Register

Address: 3033_0000h base + 46Ch offset = 3033_046Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved																
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved		VSEL				Reserved		LVTTL	HYS	PUE	ODE	SRE		DSE		
W																	
Reset	0	0	0	1	1	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ECSPi2_SCLK field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: ECSPi2_SCLK 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode 110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ECSPi2_SCLK 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ECSPi2_SCLK 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ECSPi2_SCLK field descriptions (continued)

Field	Description
	Select one out of next values for pad: ECSPi2_SCLK
000	HI-Z — Output driver is disabled (Hi-Z State)
001	255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V
010	105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V
011	75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V
100	85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V
101	65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V
110	45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V
111	40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.280 SW_PAD_CTL_PAD_ECSPi2_MOSI SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSPi2_MOSI)**SW_PAD_CTL Register**

Address: 3033_0000h base + 470h offset = 3033_0470h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_ECSPi2_MOSI field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTTL	Lvttl Enable Field Control signal to select LVTTTL input 0 Disabled — LVTTTL Disabled 1 Enabled — LVTTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ECSPi2_MOSI field descriptions (continued)

Field	Description
	0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ECSPi2_MOSI 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ECSPi2_MOSI 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ECSPi2_MOSI 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.281 SW_PAD_CTL_PAD_ECSPi2_MISO SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSPi2_MISO)

SW_PAD_CTL Register

Address: 3033_0000h base + 474h offset = 3033_0474h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ECSPi2_MISO field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ECSPi2_MISO 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_ECSPi2_MISO field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ECSPi2_MISO 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ECSPi2_MISO 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.282 SW_PAD_CTL_PAD_ECSPi2_SS0 SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_ECSPi2_SS0)**SW_PAD_CTL Register**

Address: 3033_0000h base + 478h offset = 3033_0478h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_ECSPi2_SS0 field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: ECSPi2_SS0 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: ECSPi2_SS0 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: ECSPi2_SS0 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.283 SW_PAD_CTL_PAD_I2C1_SCL SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C1_SCL)

SW_PAD_CTL Register

Address: 3033_0000h base + 47Ch offset = 3033_047Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_I2C1_SCL field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTTL input 0 Disabled — LVTTTL Disabled 1 Enabled — LVTTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: I2C1_SCL 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_I2C1_SCL field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: I2C1_SCL 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: I2C1_SCL 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.284 SW_PAD_CTL_PAD_I2C1_SDA SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C1_SDA)**SW_PAD_CTL Register**

Address: 3033_0000h base + 480h offset = 3033_0480h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_I2C1_SDA field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: I2C1_SDA 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: I2C1_SDA 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: I2C1_SDA 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.285 SW_PAD_CTL_PAD_I2C2_SCL SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C2_SCL)

SW_PAD_CTL Register

Address: 3033_0000h base + 484h offset = 3033_0484h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_I2C2_SCL field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: I2C2_SCL 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_I2C2_SCL field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: I2C2_SCL 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: I2C2_SCL 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.286 SW_PAD_CTL_PAD_I2C2_SDA SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C2_SDA)**SW_PAD_CTL Register**

Address: 3033_0000h base + 488h offset = 3033_0488h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_I2C2_SDA field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: I2C2_SDA 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: I2C2_SDA 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: I2C2_SDA 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.287 SW_PAD_CTL_PAD_I2C3_SCL SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C3_SCL)

SW_PAD_CTL Register

Address: 3033_0000h base + 48Ch offset = 3033_048Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		VSEL			Reserved		LVTTL	HYS	PUE	ODE	SRE		DSE		
W																
Reset	0	0	0	1	1	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_I2C3_SCL field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: I2C3_SCL 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode 110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_I2C3_SCL field descriptions (continued)

Field	Description
	0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: I2C3_SCL 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: I2C3_SCL 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: I2C3_SCL 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.288 SW_PAD_CTL_PAD_I2C3_SDA SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C3_SDA)

SW_PAD_CTL Register

Address: 3033_0000h base + 490h offset = 3033_0490h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_I2C3_SDA field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: I2C3_SDA 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_I2C3_SDA field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: I2C3_SDA 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: I2C3_SDA 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.289 SW_PAD_CTL_PAD_I2C4_SCL SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C4_SCL)**SW_PAD_CTL Register**

Address: 3033_0000h base + 494h offset = 3033_0494h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_I2C4_SCL field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: I2C4_SCL 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: I2C4_SCL 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: I2C4_SCL 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.290 SW_PAD_CTL_PAD_I2C4_SDA SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_I2C4_SDA)

SW_PAD_CTL Register

Address: 3033_0000h base + 498h offset = 3033_0498h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_I2C4_SDA field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: I2C4_SDA 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_I2C4_SDA field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: I2C4_SDA 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: I2C4_SDA 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.291 SW_PAD_CTL_PAD_UART1_RXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART1_RXD)**SW_PAD_CTL Register**

Address: 3033_0000h base + 49Ch offset = 3033_049Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_UART1_RXD field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: UART1_RXD 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: UART1_RXD 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: UART1_RXD 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.292 SW_PAD_CTL_PAD_UART1_TXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART1_TXD)

SW_PAD_CTL Register

Address: 3033_0000h base + 4A0h offset = 3033_04A0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_UART1_TXD field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: UART1_TXD 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_UART1_TXD field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: UART1_TXD 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: UART1_TXD 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.293 SW_PAD_CTL_PAD_UART2_RXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART2_RXD)**SW_PAD_CTL Register**

Address: 3033_0000h base + 4A4h offset = 3033_04A4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_UART2_RXD field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: UART2_RXD 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: UART2_RXD 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: UART2_RXD 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.294 SW_PAD_CTL_PAD_UART2_TXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART2_TXD)

SW_PAD_CTL Register

Address: 3033_0000h base + 4A8h offset = 3033_04A8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_UART2_TXD field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: UART2_TXD 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_UART2_TXD field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: UART2_TXD 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: UART2_TXD 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.295 SW_PAD_CTL_PAD_UART3_RXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART3_RXD)**SW_PAD_CTL Register**

Address: 3033_0000h base + 4ACh offset = 3033_04ACh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved																
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved		VSEL				Reserved		LVTTL	HYS	PUE	ODE	SRE		DSE		
W																	
Reset	0	0	0	1	1	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_UART3_RXD field descriptions

Field	Description
31–14 -	This field is reserved. Reserved
13–11 VSEL	Voltage Select Field Select one out of next values for pad: UART3_RXD 000 VSEL_0_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 001 VSEL_1_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 010 VSEL_2_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 011 VSEL_3_Auto_Detct_Mode — Auto Detect 3.3/2.5/1.2/1.8 V mode 100 VSEL_4_Manual_3p3V_Mode — Manually Set 3.3V mode 101 VSEL_5_Manual_2p5V_Mode — Manually Set 2.5V mode 110 VSEL_6_Manual_2p5V_Mode — Manually Set 2.5V mode 111 VSEL_7_Manual_1p2_1p8V_Mode — Manually Set 1.2V/1.8V mode
10–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: UART3_RXD 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: UART3_RXD 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_UART3_RXD field descriptions (continued)

Field	Description
	Select one out of next values for pad: UART3_RXD
000	HI-Z — Output driver is disabled (Hi-Z State)
001	255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V
010	105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V
011	75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V
100	85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V
101	65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V
110	45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V
111	40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.296 SW_PAD_CTL_PAD_UART3_TXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART3_TXD)**SW_PAD_CTL Register**

Address: 3033_0000h base + 4B0h offset = 3033_04B0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_UART3_TXD field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schmitt trigger

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_UART3_TXD field descriptions (continued)

Field	Description
	0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: UART3_TXD 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: UART3_TXD 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: UART3_TXD 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.297 SW_PAD_CTL_PAD_UART4_RXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART4_RXD)

SW_PAD_CTL Register

Address: 3033_0000h base + 4B4h offset = 3033_04B4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

IOMUXC_SW_PAD_CTL_PAD_UART4_RXD field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: UART4_RXD 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled

Table continues on the next page...

IOMUXC_SW_PAD_CTL_PAD_UART4_RXD field descriptions (continued)

Field	Description
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: UART4_RXD 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: UART4_RXD 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.298 SW_PAD_CTL_PAD_UART4_TXD SW PAD Control Register (IOMUXC_SW_PAD_CTL_PAD_UART4_TXD)**SW_PAD_CTL Register**

Address: 3033_0000h base + 4B8h offset = 3033_04B8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								LVTTL	HYS	PUE	ODE	SRE	DSE		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0

IOMUXC_SW_PAD_CTL_PAD_UART4_TXD field descriptions

Field	Description
31–9 -	This field is reserved. Reserved
8 LVTTL	Lvttl Enable Field Control signal to select LVTTL input 0 Disabled — LVTTL Disabled 1 Enabled — LVTTL Enabled
7 HYS	Schmitt trigger Enable Field Control signal to select Schymitt trigger 0 Disabled — Schmitt Trigger Disabled 1 Enabled — Schmitt Trigger Enabled
6 PUE	Pull Up Enable Field Control signal to select internal pullup resistor 0 Disabled — Pull Up Resistor Disabled 1 Enabled — Pull Up Resistor Enabled
5 ODE	Open Drain Enable Field Select one out of next values for pad: UART4_TXD 0 Disabled — Open Drain Disabled 1 Enabled — Open Drain Enabled
4–3 SRE	Slew Rate Field 2-bit slew rate control signals to select between 50,100 and 200MHz I/O cell operation frequency range with reduced switching noise. Select one out of next values for pad: UART4_TXD 00 SLOW — Slow Frequency Slew Rate (50Mhz) 01 MEDIUM — Medium Frequency Slew Rate (100Mhz) 10 FAST — Fast Frequency Slew Rate (150Mhz) 11 MAX — Max Frequency Slew Rate (200Mhz)
DSE	Drive Strength Field Select one out of next values for pad: UART4_TXD 000 HI-Z — Output driver is disabled (Hi-Z State) 001 255_OHM — 255 Ohm @3.3V, 240 Ohm @2.5V, 230 Ohm @1.8V, 265 Ohm @1.2V 010 105_OHM — 105 Ohm @3.3V, 100 Ohm @2.5V, 85 Ohm @1.8V, 110 Ohm @1.2V 011 75_OHM — 75 Ohm @3.3V, 70 Ohm @2.5V, 60 Ohm @1.8V, 80 Ohm @1.2V 100 85_OHM — 85 Ohm @3.3V, 80 Ohm @2.5V, 75 Ohm @1.8V, 90 Ohm @1.2V 101 65_OHM — 65 Ohm @3.3V, 60 Ohm @2.5V, 55 Ohm @1.8V, 65 Ohm @1.2V 110 45_OHM — 45 Ohm @3.3V, 45 Ohm @2.5V, 40 Ohm @1.8V, 50 Ohm @1.2V 111 40_OHM — 40 Ohm @3.3V, 40 Ohm @2.5V, 33 Ohm @1.8V, 40 Ohm @1.2V

8.2.5.299 CCM_PMIC_READY_SELECT_INPUT DAISY Register (IOMUXC_CCM_PMIC_READY_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4BCh offset = 3033_04BCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_CCM_PMIC_READY_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 0 GPIO1_IO05_ALT5 — Selecting Pad: GPIO1_IO05 Mode: ALT5 for CCM_PMIC_READY 1 GPIO1_IO11_ALT5 — Selecting Pad: GPIO1_IO11 Mode: ALT5 for CCM_PMIC_READY

8.2.5.300 ENET1_MDIO_SELECT_INPUT DAISY Register (IOMUXC_ENET1_MDIO_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4C0h offset = 3033_04C0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_ENET1_MDIO_SELECT_INPUT field descriptions

Field	Description
31–2 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 GPIO01_IO07_ALT1 — Selecting Pad: GPIO01_IO07 Mode: ALT1 for ENET1_MDIO 01 ENET_MDIO_ALT0 — Selecting Pad: ENET_MDIO Mode: ALT0 for ENET1_MDIO 10 I2C1_SDA_ALT1 — Selecting Pad: I2C1_SDA Mode: ALT1 for ENET1_MDIO 11 GPIO01_IO07_ALT1 — Selecting Pad: GPIO01_IO07 Mode: ALT1 for ENET1_MDIO

8.2.5.301 SAI1_RX_SYNC_SELECT_INPUT DAISY Register (IOMUXC_SAI1_RX_SYNC_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4C4h offset = 3033_04C4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI1_RX_SYNC_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 0 SAI1_RXFS_ALT0 — Selecting Pad: SAI1_RXFS Mode: ALT0 for SAI1_RX_SYNC 1 SAI1_RXD5_ALT3 — Selecting Pad: SAI1_RXD5 Mode: ALT3 for SAI1_RX_SYNC

8.2.5.302 SAI1_TX_BCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI1_TX_BCLK_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4C8h offset = 3033_04C8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved														DAISY	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI1_TX_BCLK_SELECT_INPUT field descriptions

Field	Description
31–2 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 SAI5_MCLK_ALT1 — Selecting Pad: SAI5_MCLK Mode: ALT1 for SAI1_TX_BCLK 01 SAI1_TXC_ALT0 — Selecting Pad: SAI1_TXC Mode: ALT0 for SAI1_TX_BCLK 10 SAI1_MCLK_ALT2 — Selecting Pad: SAI1_MCLK Mode: ALT2 for SAI1_TX_BCLK 11 SAI5_MCLK_ALT1 — Selecting Pad: SAI5_MCLK Mode: ALT1 for SAI1_TX_BCLK

8.2.5.303 SAI1_TX_SYNC_SELECT_INPUT DAISY Register (IOMUXC_SAI1_TX_SYNC_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4CCh offset = 3033_04CCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

IOMUXC_SAI1_TX_SYNC_SELECT_INPUT field descriptions

Field	Description
31–3 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field

Table continues on the next page...

IOMUXC_SAI1_TX_SYNC_SELECT_INPUT field descriptions (continued)

Field	Description
	Selecting Pads Involved in Daisy Chain
000	SAI5_RXD1_ALT2 — Selecting Pad: SAI5_RXD1 Mode: ALT2 for SAI1_TX_SYNC
001	SAI5_RXD2_ALT2 — Selecting Pad: SAI5_RXD2 Mode: ALT2 for SAI1_TX_SYNC
010	SAI5_RXD3_ALT2 — Selecting Pad: SAI5_RXD3 Mode: ALT2 for SAI1_TX_SYNC
011	SAI1_TXFS_ALT0 — Selecting Pad: SAI1_TXFS Mode: ALT0 for SAI1_TX_SYNC
100	SAI1_RXD7_ALT2 — Selecting Pad: SAI1_RXD7 Mode: ALT2 for SAI1_TX_SYNC
101	SAI5_RXD1_ALT2 — Selecting Pad: SAI5_RXD1 Mode: ALT2 for SAI1_TX_SYNC
110	SAI5_RXD1_ALT2 — Selecting Pad: SAI5_RXD1 Mode: ALT2 for SAI1_TX_SYNC
111	SAI5_RXD1_ALT2 — Selecting Pad: SAI5_RXD1 Mode: ALT2 for SAI1_TX_SYNC

8.2.5.304 SAI5_RX_BCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI5_RX_BCLK_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4D0h offset = 3033_04D0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI5_RX_BCLK_SELECT_INPUT field descriptions

Field	Description
31–2 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 SAI5_RXC_ALT0 — Selecting Pad: SAI5_RXC Mode: ALT0 for SAI5_RX_BCLK 01 SAI1_RXC_ALT1 — Selecting Pad: SAI1_RXC Mode: ALT1 for SAI5_RX_BCLK 10 SAI3_RXC_ALT2 — Selecting Pad: SAI3_RXC Mode: ALT2 for SAI5_RX_BCLK 11 SAI5_RXC_ALT0 — Selecting Pad: SAI5_RXC Mode: ALT0 for SAI5_RX_BCLK

8.2.5.305 SAI5_RXD0_SELECT_INPUT DAISY Register (IOMUXC_SAI5_RXD0_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4D4h offset = 3033_04D4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved														DAISY	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI5_RXD0_SELECT_INPUT field descriptions

Field	Description
31–2 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 SAI5_RXD0_ALT0 — Selecting Pad: SAI5_RXD0 Mode: ALT0 for SAI5_RXD0 01 SAI1_RXD0_ALT1 — Selecting Pad: SAI1_RXD0 Mode: ALT1 for SAI5_RXD0 10 SAI3_RXD_ALT2 — Selecting Pad: SAI3_RXD Mode: ALT2 for SAI5_RXD0 11 SAI5_RXD0_ALT0 — Selecting Pad: SAI5_RXD0 Mode: ALT0 for SAI5_RXD0

8.2.5.306 SAI5_RXD1_SELECT_INPUT DAISY Register (IOMUXC_SAI5_RXD1_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4D8h offset = 3033_04D8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved														DAISY	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI5_RXD1_SELECT_INPUT field descriptions

Field	Description
31–2 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 SAI5_RXD1_ALT0 — Selecting Pad: SAI5_RXD1 Mode: ALT0 for SAI5_RXD1 01 SAI1_RXD1_ALT1 — Selecting Pad: SAI1_RXD1 Mode: ALT1 for SAI5_RXD1 10 SAI3_TXFS_ALT1 — Selecting Pad: SAI3_TXFS Mode: ALT1 for SAI5_RXD1 11 SAI5_RXD1_ALT0 — Selecting Pad: SAI5_RXD1 Mode: ALT0 for SAI5_RXD1

8.2.5.307 SAI5_RXD2_SELECT_INPUT DAISY Register (IOMUXC_SAI5_RXD2_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4DCh offset = 3033_04DCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI5_RXD2_SELECT_INPUT field descriptions

Field	Description
31–2 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 SAI5_RXD2_ALT0 — Selecting Pad: SAI5_RXD2 Mode: ALT0 for SAI5_RXD2 01 SAI1_RXD2_ALT1 — Selecting Pad: SAI1_RXD2 Mode: ALT1 for SAI5_RXD2 10 SAI3_TXC_ALT2 — Selecting Pad: SAI3_TXC Mode: ALT2 for SAI5_RXD2 11 SAI5_RXD2_ALT0 — Selecting Pad: SAI5_RXD2 Mode: ALT0 for SAI5_RXD2

8.2.5.308 SAI5_RXD3_SELECT_INPUT DAISY Register (IOMUXC_SAI5_RXD3_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4E0h offset = 3033_04E0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved														DAISY	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI5_RXD3_SELECT_INPUT field descriptions

Field	Description
31–2 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 SAI5_RXD3_ALT0 — Selecting Pad: SAI5_RXD3 Mode: ALT0 for SAI5_RXD3 01 SAI1_RXD3_ALT1 — Selecting Pad: SAI1_RXD3 Mode: ALT1 for SAI5_RXD3 10 SAI3_TXD_ALT2 — Selecting Pad: SAI3_TXD Mode: ALT2 for SAI5_RXD3 11 SAI5_RXD3_ALT0 — Selecting Pad: SAI5_RXD3 Mode: ALT0 for SAI5_RXD3

8.2.5.309 SAI5_RX_SYNC_SELECT_INPUT DAISY Register (IOMUXC_SAI5_RX_SYNC_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4E4h offset = 3033_04E4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved														DAISY	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI5_RX_SYNC_SELECT_INPUT field descriptions

Field	Description
31–2 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 SAI5_RXFS_ALT0 — Selecting Pad: SAI5_RXFS Mode: ALT0 for SAI5_RX_SYNC 01 SAI1_RXFS_ALT1 — Selecting Pad: SAI1_RXFS Mode: ALT1 for SAI5_RX_SYNC 10 SAI3_RXFS_ALT2 — Selecting Pad: SAI3_RXFS Mode: ALT2 for SAI5_RX_SYNC 11 SAI5_RXFS_ALT0 — Selecting Pad: SAI5_RXFS Mode: ALT0 for SAI5_RX_SYNC

8.2.5.310 SAI5_TX_BCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI5_TX_BCLK_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4E8h offset = 3033_04E8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI5_TX_BCLK_SELECT_INPUT field descriptions

Field	Description
31–2 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 SAI5_RXD2_ALT3 — Selecting Pad: SAI5_RXD2 Mode: ALT3 for SAI5_TX_BCLK 01 SAI1_TXC_ALT1 — Selecting Pad: SAI1_TXC Mode: ALT1 for SAI5_TX_BCLK 10 SAI2_RXC_ALT1 — Selecting Pad: SAI2_RXC Mode: ALT1 for SAI5_TX_BCLK 11 SAI5_RXD2_ALT3 — Selecting Pad: SAI5_RXD2 Mode: ALT3 for SAI5_TX_BCLK

8.2.5.311 SAI5_TX_SYNC_SELECT_INPUT DAISY Register (IOMUXC_SAI5_TX_SYNC_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4ECh offset = 3033_04ECh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

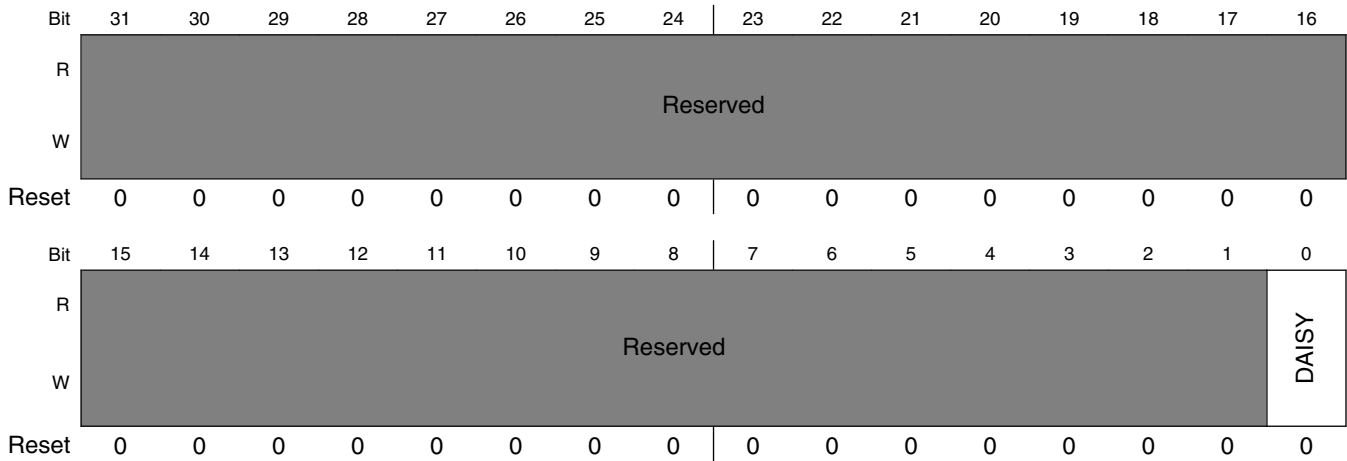
IOMUXC_SAI5_TX_SYNC_SELECT_INPUT field descriptions

Field	Description
31–2 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 SAI5_RXD1_ALT3 — Selecting Pad: SAI5_RXD1 Mode: ALT3 for SAI5_TX_SYNC 01 SAI1_TXFS_ALT1 — Selecting Pad: SAI1_TXFS Mode: ALT1 for SAI5_TX_SYNC 10 SAI2_RXFS_ALT1 — Selecting Pad: SAI2_RXFS Mode: ALT1 for SAI5_TX_SYNC 11 SAI5_RXD1_ALT3 — Selecting Pad: SAI5_RXD1 Mode: ALT3 for SAI5_TX_SYNC

8.2.5.312 UART1_RTS_B_SELECT_INPUT DAISY Register (IOMUXC_UART1_RTS_B_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4F0h offset = 3033_04F0h



IOMUXC_UART1_RTS_B_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 0 UART3_RXD_ALT1 — Selecting Pad: UART3_RXD Mode: ALT1 for UART1_RTS_B 1 UART3_TXD_ALT1 — Selecting Pad: UART3_TXD Mode: ALT1 for UART1_RTS_B

8.2.5.313 UART1_RXD_SELECT_INPUT DAISY Register (IOMUXC_UART1_RXD_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4F4h offset = 3033_04F4h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_UART1_RXD_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 0 UART1_RXD_ALT0 — Selecting Pad: UART1_RXD Mode: ALT0 for UART1_RXD 1 UART1_TXD_ALT0 — Selecting Pad: UART1_TXD Mode: ALT0 for UART1_RXD

8.2.5.314 UART2_RTS_B_SELECT_INPUT DAISY Register (IOMUXC_UART2_RTS_B_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4F8h offset = 3033_04F8h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_UART2_RTS_B_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 0 UART4_RXD_ALT1 — Selecting Pad: UART4_RXD Mode: ALT1 for UART2_RTS_B 1 UART4_TXD_ALT1 — Selecting Pad: UART4_TXD Mode: ALT1 for UART2_RTS_B

8.2.5.315 UART2_RXD_SELECT_INPUT DAISY Register (IOMUXC_UART2_RXD_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 4FCh offset = 3033_04FCh

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_UART2_RXD_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 0 UART2_RXD_ALT0 — Selecting Pad: UART2_RXD Mode: ALT0 for UART2_RXD 1 UART2_TXD_ALT0 — Selecting Pad: UART2_TXD Mode: ALT0 for UART2_RXD

8.2.5.316 UART3_RTS_B_SELECT_INPUT DAISY Register (IOMUXC_UART3_RTS_B_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 500h offset = 3033_0500h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_UART3_RTS_B_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 0 ECSP11_MISO_ALT1 — Selecting Pad: ECSP11_MISO Mode: ALT1 for UART3_RTS_B 1 ECSP11_SS0_ALT1 — Selecting Pad: ECSP11_SS0 Mode: ALT1 for UART3_RTS_B

8.2.5.317 UART3_RXD_SELECT_INPUT DAISY Register (IOMUXC_UART3_RXD_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 504h offset = 3033_0504h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_UART3_RXD_SELECT_INPUT field descriptions

Field	Description
31–2 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 ECSPI1_SCLK_ALT1 — Selecting Pad: ECSPI1_SCLK Mode: ALT1 for UART3_RXD 01 ECSPI1_MOSI_ALT1 — Selecting Pad: ECSPI1_MOSI Mode: ALT1 for UART3_RXD 10 UART3_RXD_ALT0 — Selecting Pad: UART3_RXD Mode: ALT0 for UART3_RXD 11 UART3_TXD_ALT0 — Selecting Pad: UART3_TXD Mode: ALT0 for UART3_RXD

8.2.5.318 UART4_RTS_B_SELECT_INPUT DAISY Register (IOMUXC_UART4_RTS_B_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 508h offset = 3033_0508h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_UART4_RTS_B_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 0 ECSPI2_MISO_ALT1 — Selecting Pad: ECSPI2_MISO Mode: ALT1 for UART4_RTS_B 1 ECSPI2_SS0_ALT1 — Selecting Pad: ECSPI2_SS0 Mode: ALT1 for UART4_RTS_B

8.2.5.319 UART4_RXD_SELECT_INPUT DAISY Register (IOMUXC_UART4_RXD_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 50Ch offset = 3033_050Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_UART4_RXD_SELECT_INPUT field descriptions

Field	Description
31–2 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 ECSPI2_SCLK_ALT1 — Selecting Pad: ECSPI2_SCLK Mode: ALT1 for UART4_RXD 01 ECSPI2_MOSI_ALT1 — Selecting Pad: ECSPI2_MOSI Mode: ALT1 for UART4_RXD 10 UART4_RXD_ALT0 — Selecting Pad: UART4_RXD Mode: ALT0 for UART4_RXD 11 UART4_TXD_ALT0 — Selecting Pad: UART4_RXD Mode: ALT0 for UART4_RXD

8.2.5.320 SAI6_RX_BCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI6_RX_BCLK_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 510h offset = 3033_0510h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI6_RX_BCLK_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 SAI1_RXD4_ALT2 — Selecting Pad: SAI1_RXD4 Mode: ALT2 for SAI6_RX_BCLK 01 SAI1_TXD4_ALT1 — Selecting Pad: SAI1_TXD4 Mode: ALT1 for SAI6_RX_BCLK

8.2.5.321 SAI6_RXD0_SELECT_INPUT DAISY Register (IOMUXC_SAI6_RXD0_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 514h offset = 3033_0514h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI6_RXD0_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 SAI1_RXD5_ALT2 — Selecting Pad: SAI1_RXD5 Mode: ALT2 for SAI6_RXD0 01 SAI1_TXD5_ALT1 — Selecting Pad: SAI1_TXD5 Mode: ALT1 for SAI6_RXD0

8.2.5.322 SAI6_RX_SYNC_SELECT_INPUT DAISY Register (IOMUXC_SAI6_RX_SYNC_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 518h offset = 3033_0518h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

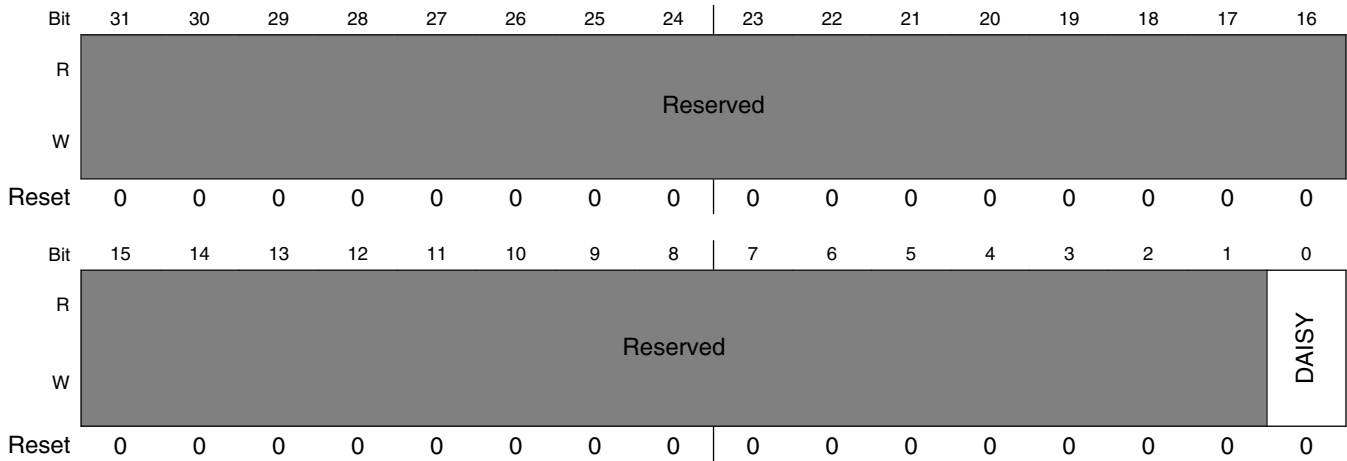
IOMUXC_SAI6_RX_SYNC_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 SAI1_RXD6_ALT2 — Selecting Pad: SAI1_RXD6 Mode: ALT2 for SAI6_RX_SYNC 01 SAI1_TXD6_ALT1 — Selecting Pad: SAI1_TXD6 Mode: ALT1 for SAI6_RX_SYNC

8.2.5.323 SAI6_TX_BCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI6_TX_BCLK_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 51Ch offset = 3033_051Ch



IOMUXC_SAI6_TX_BCLK_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 0 SAI1_RXD4_ALT1 — Selecting Pad: SAI1_RXD4 Mode: ALT1 for SAI6_TX_BCLK 1 SAI1_TXD4_ALT2 — Selecting Pad: SAI1_TXD4 Mode: ALT2 for SAI6_TX_BCLK

8.2.5.324 SAI6_TX_SYNC_SELECT_INPUT DAISY Register (IOMUXC_SAI6_TX_SYNC_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 520h offset = 3033_0520h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI6_TX_SYNC_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 0 SAI1_RXD6_ALT1 — Selecting Pad: SAI1_RXD6 Mode: ALT1 for SAI6_TX_SYNC 1 SAI1_TXD6_ALT2 — Selecting Pad: SAI1_TXD6 Mode: ALT2 for SAI6_TX_SYNC

8.2.5.325 PCIE1_CLKREQ_B_SELECT_INPUT DAISY Register (IOMUXC_PCIE1_CLKREQ_B_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 524h offset = 3033_0524h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_PCIE1_CLKREQ_B_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 0 I2C4_SCL_ALT2 — Selecting Pad: I2C4_SCL Mode: ALT2 for PCIE1_CLKREQ_B 1 UART4_RXD_ALT2 — Selecting Pad: UART4_RXD Mode: ALT2 for PCIE1_CLKREQ_B

8.2.5.326 PCIE2_CLKREQ_B_SELECT_INPUT DAISY Register (IOMUXC_PCIE2_CLKREQ_B_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 528h offset = 3033_0528h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_PCIE2_CLKREQ_B_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 0 I2C4_SDA_ALT2 — Selecting Pad: I2C4_SDA Mode: ALT2 for PCIE2_CLKREQ_B 1 UART4_TXD_ALT2 — Selecting Pad: UART4_TXD Mode: ALT2 for PCIE2_CLKREQ_B

8.2.5.327 SAI5_MCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI5_MCLK_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 52Ch offset = 3033_052Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI5_MCLK_SELECT_INPUT field descriptions

Field	Description
31–2 -	This field is reserved. Reserved
DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 00 SAI5_MCLK_ALT0 — Selecting Pad: SAI5_MCLK Mode: ALT0 for SAI5_MCLK 01 SAI1_MCLK_ALT1 — Selecting Pad: SAI1_MCLK Mode: ALT1 for SAI5_MCLK 10 SAI2_MCLK_ALT1 — Selecting Pad: SAI2_MCLK Mode: ALT1 for SAI5_MCLK 11 SAI3_MCLK_ALT2 — Selecting Pad: SAI3_MCLK Mode: ALT2 for SAI5_MCLK

8.2.5.328 SAI6_MCLK_SELECT_INPUT DAISY Register (IOMUXC_SAI6_MCLK_SELECT_INPUT)

DAISY Register

Address: 3033_0000h base + 530h offset = 3033_0530h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															DAISY
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IOMUXC_SAI6_MCLK_SELECT_INPUT field descriptions

Field	Description
31–1 -	This field is reserved. Reserved
0 DAISY	Input Select (DAISY) Field Selecting Pads Involved in Daisy Chain 0 SAI1_RXD7_ALT1 — Selecting Pad: SAI1_RXD7 Mode: ALT1 for SAI6_MCLK 1 SAI1_TXD7_ALT1 — Selecting Pad: SAI1_TXD7 Mode: ALT1 for SAI6_MCLK

8.3 General Purpose Input/Output (GPIO)

8.3.1 Overview

The GPIO general-purpose input/output peripheral provides dedicated general-purpose pins that can be configured as either inputs or outputs.

When configured as an output, it is possible to write to an internal register to control the state driven on the output pin. When configured as an input, it is possible to detect the state of the input by reading the state of an internal register. In addition, the GPIO peripheral can produce CORE interrupts.

The GPIO is one of the blocks controlling the IOMUX of the chip.

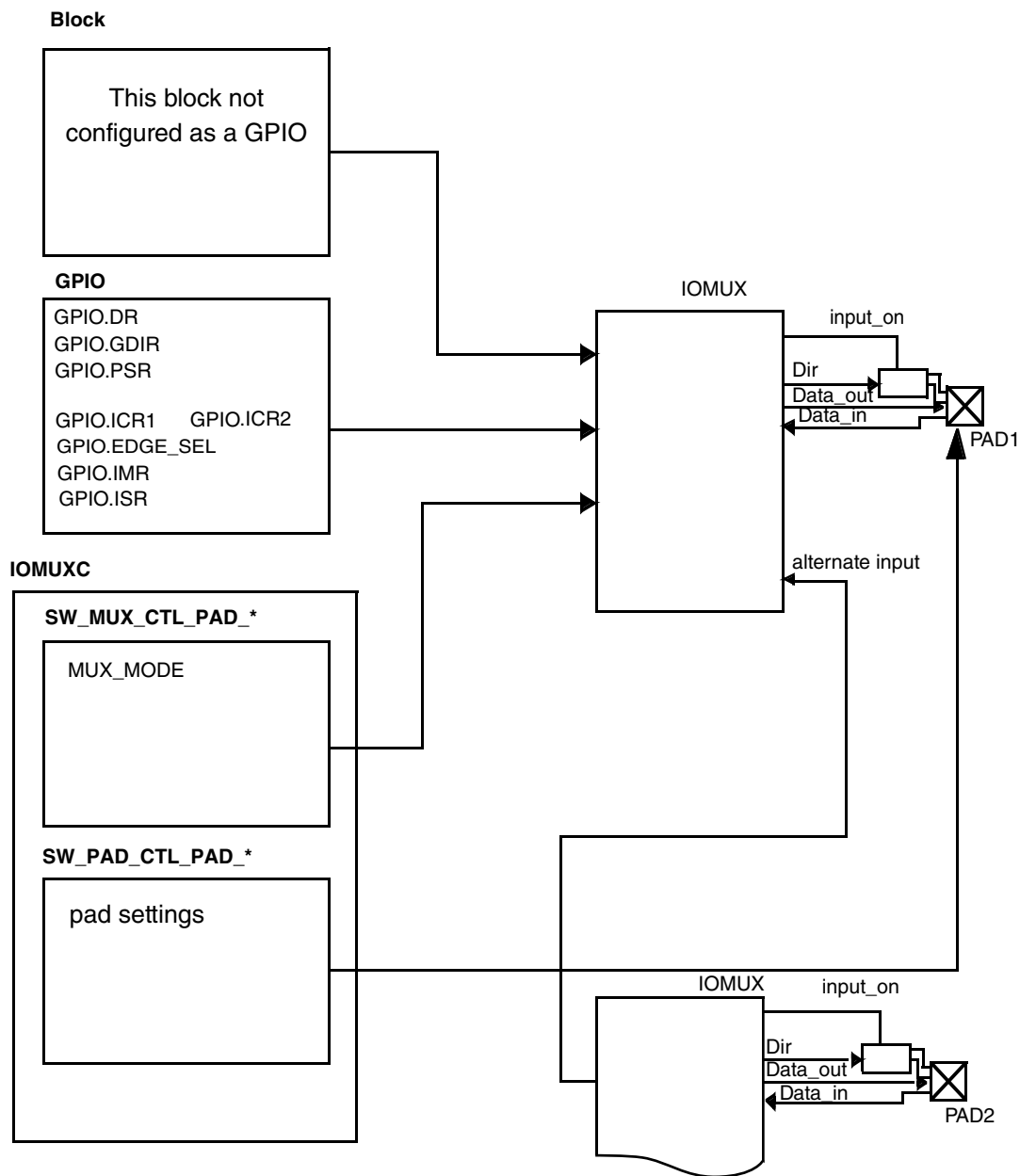


Figure 8-4. Chip IOMUX Scheme

The GPIO functionality is provided through eight registers, an edge-detect circuit, and interrupt generation logic.

The eight registers are:

- Data register (GPIO_DR)
- GPIO direction register (GPIO_GDIR)
- Pad sample register (GPIO_PSR)
- Interrupt control registers (GPIO_ICR1, GPIO_ICR2)

- Edge select register (GPIO_EDGE_SEL)
- Interrupt mask register (GPIO_IMR)
- Interrupt status register (GPIO_ISR)

These registers are described in detail in [GPIO Memory Map/Register Definition](#).

Each GPIO input has a dedicated edge-detect circuit which can be configured through software to detect rising edges, falling edges, logic low-levels or logic high-levels on the input signals. The outputs of the edge detect circuits are optionally masked by setting the corresponding bit in the interrupt mask register (GPIO_IMR). These qualified outputs are OR'ed together to generate two one-bit interrupt lines:

- Combined interrupt indication for GPIOx signals 0 - 15
- Combined interrupt indication for GPIOx signals 16 - 31

In addition, GPIO1 provides visibility to each of its 8 low order interrupt sources (i.e. GPIO1 interrupt n, for $n = 0 - 7$). However, individual interrupt indications from other GPIOx are not available.

The GPIO edge detection is described further in [Interrupt Control Unit](#).

The GPIO's overall functionality is described further in [GPIO Functional Description](#).

8.3.1.1 Block Diagram

The GPIO subsystem contains multiple GPIO blocks, which can generate and control up to 32 signals for general purpose.

A block diagram of the GPIO is shown in [Figure 8-5](#)

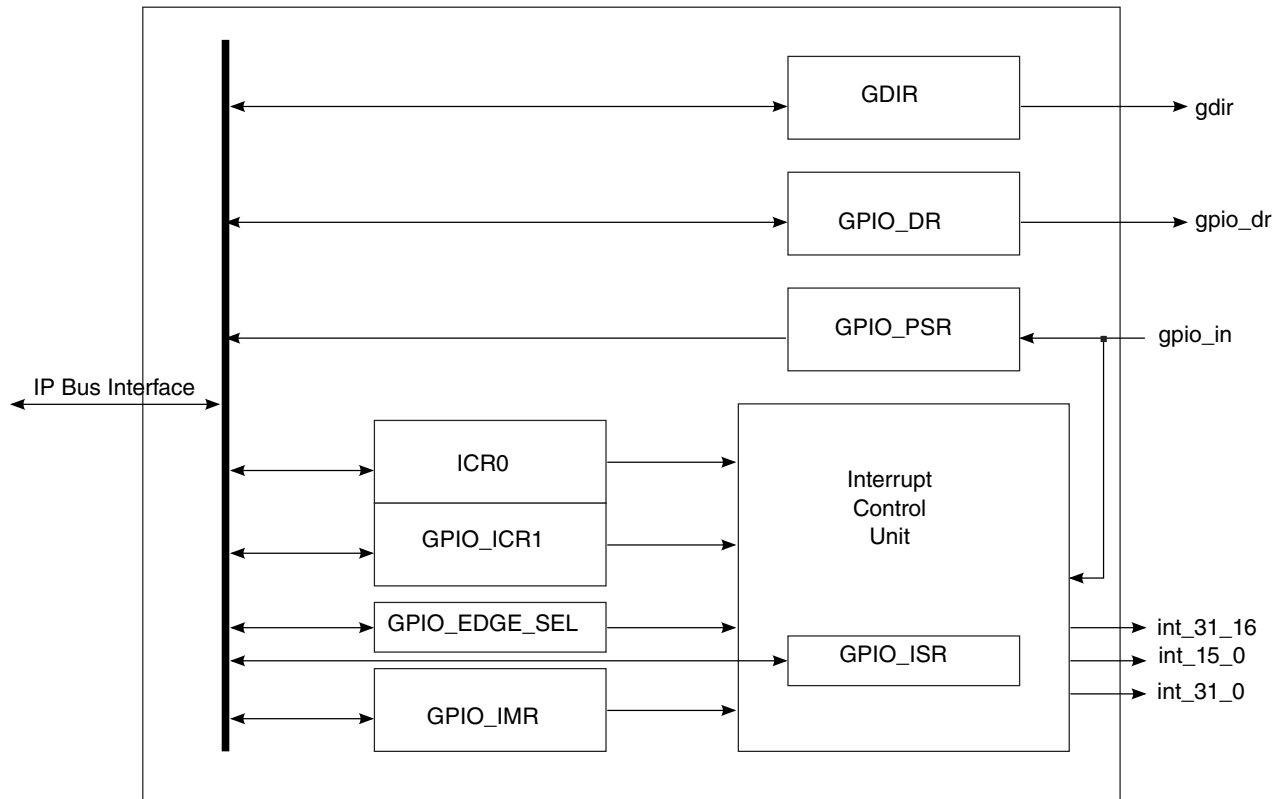


Figure 8-5. GPIO Block Diagram

8.3.1.2 Features

The GPIO includes the following features:

- General purpose input/output logic capabilities:
 - Drives specific data to output using the data register (GPIO_DR)
 - Controls the direction of the signal using the GPIO direction register (GPIO_GDIR)
 - Enables the core to sample the status of the corresponding inputs by reading the pad sample register (GPIO_PSR).
- GPIO interrupt capabilities:
 - Supports up to 32 interrupts
 - Identifies interrupt edges
 - Generates three active-high interrupts to the SoC interrupt controller

8.3.2 Clocks

The table found here describes the clock sources for GPIO.

Please see the clock controller Module for clock setting, configuration and gating information.

Table 8-3. GPIO Clocks

Clock name	Clock Root	Description
ipg_clk_s	ipg_clk_root	Peripheral access clock

8.3.3 GPIO Functional Description

This section provides a complete functional description of the block.

8.3.3.1 GPIO Function

A GPIO signal can operate as a general-purpose input/output when the IOMUX is set to GPIO mode. Each GPIO signal may be independently configured as either an input or an output using the GPIO direction register (GPIO_GDIR).

When configured as an output (GPIO_GDIR bit = 1), the value in the data bit in the GPIO data register (GPIO_DR) is driven on the corresponding GPIO line. When a signal is configured as an input (GPIO_GDIR bit = 0), the state of the input can be read from the corresponding GPIO_PSR bit.

8.3.3.2 GPIO Programming

8.3.3.2.1 GPIO Read Mode

The programming sequence for reading input signals should be as follows:

1. Configure IOMUX to select GPIO mode (Via IOMUX Controller (IOMUXC)).
2. Configure GPIO direction register to input (GPIO_GDIR[GDIR] set to 0b).
3. Read value from data register/pad status register.

A pseudocode description to read [input3:input0] values is as follows:

```
// SET INPUTS TO GPIO MODE.
write sw_mux_ctl_<input0>_<input1>_<input2>_<input3>, 32'h00000000
// SET GDIR TO INPUT.
write GDIR[31:4,input3_bit, input2_bit, input1_bit, input0_bit,] 32'hxxxxxxx0
// READ INPUT VALUE FROM DR.
read DR
// READ INPUT VALUE FROM PSR.
read PSR
```

NOTE

While the GPIO direction is set to input (GPIO_GDIR = 0), a read access to GPIO_DR does not return GPIO_DR data. Instead, it returns the GPIO_PSR data, which is the corresponding input signal value.

8.3.3.2.2 GPIO Write Mode

The programming sequence for driving output signals should be as follows:

1. Configure IOMUX to select GPIO mode (Via IOMUXC), also enable SION if need to read loopback pad value through PSR
2. Configure GPIO direction register to output (GPIO_GDIR[GDIR] set to 1b).
3. Write value to data register (GPIO_DR).

A pseudocode description to drive 4'b0101 on [output3:output0] is as follows:

```
// SET PADS TO GPIO MODE VIA IOMUX.
write sw_mux_ctl_pad<output[0-3]>.mux_mode, <GPIO_MUX_MODE>
// Enable loopback so we can capture pad value into PSR in output mode
write sw_mux_ctl_pad<output[0-3]>.sion, 1
// SET GDIR=1 TO OUTPUT BITS.
write GDIR[31:4,output3_bit,output2_bit, output1_bit, output0_bit,] 32'hxxxxxxxF
// WRITE OUTPUT VALUE=4'b0101 TO DR.
write DR, 32'hxxxxxxx5
// READ OUTPUT VALUE FROM PSR ONLY.
read_cmp PSR, 32'hxxxxxxx5
```

8.3.3.3 Interrupt Control Unit

In addition to the general-purpose input/output function, the edge-detect logic in the GPIO peripheral reflects whether a transition has occurred on a given GPIO signal that is configured as an input (GDIR bit = 0). The interrupt control registers (GPIO_ICR1 and GPIO_ICR2) may be used to independently configure the interrupt condition of each input signal (low-to-high transition, high-to-low transition, low, or high). For information about GPIO_ICR1 and GPIO_ICR2 settings, see [GPIO Memory Map/Register Definition](#).

The interrupt control unit is built of 32 interrupt control subunits, where each subunit handles a single interrupt line.

8.3.4 GPIO Memory Map/Register Definition

There are eight 32-bit GPIO registers. All registers are accessible from the IP interface. Only 32-bit access is supported.

The GPIO memory map is shown in the following table.

GPIO memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3020_0000	GPIO data register (GPIO1_DR)	32	R/W	0000_0000h	8.3.4.1/1855
3020_0004	GPIO direction register (GPIO1_GDIR)	32	R/W	0000_0000h	8.3.4.2/1856
3020_0008	GPIO pad status register (GPIO1_PSR)	32	R	0000_0000h	8.3.4.3/1856
3020_000C	GPIO interrupt configuration register1 (GPIO1_ICR1)	32	R/W	0000_0000h	8.3.4.4/1857
3020_0010	GPIO interrupt configuration register2 (GPIO1_ICR2)	32	R/W	0000_0000h	8.3.4.5/1861
3020_0014	GPIO interrupt mask register (GPIO1_IMR)	32	R/W	0000_0000h	8.3.4.6/1864
3020_0018	GPIO interrupt status register (GPIO1_ISR)	32	w1c	0000_0000h	8.3.4.7/1865
3020_001C	GPIO edge select register (GPIO1_EDGE_SEL)	32	R/W	0000_0000h	8.3.4.8/1866
3021_0000	GPIO data register (GPIO2_DR)	32	R/W	0000_0000h	8.3.4.1/1855
3021_0004	GPIO direction register (GPIO2_GDIR)	32	R/W	0000_0000h	8.3.4.2/1856
3021_0008	GPIO pad status register (GPIO2_PSR)	32	R	0000_0000h	8.3.4.3/1856
3021_000C	GPIO interrupt configuration register1 (GPIO2_ICR1)	32	R/W	0000_0000h	8.3.4.4/1857
3021_0010	GPIO interrupt configuration register2 (GPIO2_ICR2)	32	R/W	0000_0000h	8.3.4.5/1861
3021_0014	GPIO interrupt mask register (GPIO2_IMR)	32	R/W	0000_0000h	8.3.4.6/1864
3021_0018	GPIO interrupt status register (GPIO2_ISR)	32	w1c	0000_0000h	8.3.4.7/1865
3021_001C	GPIO edge select register (GPIO2_EDGE_SEL)	32	R/W	0000_0000h	8.3.4.8/1866

GPIO memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3022_0000	GPIO data register (GPIO3_DR)	32	R/W	0000_0000h	8.3.4.1/1855
3022_0004	GPIO direction register (GPIO3_GDIR)	32	R/W	0000_0000h	8.3.4.2/1856
3022_0008	GPIO pad status register (GPIO3_PSR)	32	R	0000_0000h	8.3.4.3/1856
3022_000C	GPIO interrupt configuration register1 (GPIO3_ICR1)	32	R/W	0000_0000h	8.3.4.4/1857
3022_0010	GPIO interrupt configuration register2 (GPIO3_ICR2)	32	R/W	0000_0000h	8.3.4.5/1861
3022_0014	GPIO interrupt mask register (GPIO3_IMR)	32	R/W	0000_0000h	8.3.4.6/1864
3022_0018	GPIO interrupt status register (GPIO3_ISR)	32	w1c	0000_0000h	8.3.4.7/1865
3022_001C	GPIO edge select register (GPIO3_EDGE_SEL)	32	R/W	0000_0000h	8.3.4.8/1866
3023_0000	GPIO data register (GPIO4_DR)	32	R/W	0000_0000h	8.3.4.1/1855
3023_0004	GPIO direction register (GPIO4_GDIR)	32	R/W	0000_0000h	8.3.4.2/1856
3023_0008	GPIO pad status register (GPIO4_PSR)	32	R	0000_0000h	8.3.4.3/1856
3023_000C	GPIO interrupt configuration register1 (GPIO4_ICR1)	32	R/W	0000_0000h	8.3.4.4/1857
3023_0010	GPIO interrupt configuration register2 (GPIO4_ICR2)	32	R/W	0000_0000h	8.3.4.5/1861
3023_0014	GPIO interrupt mask register (GPIO4_IMR)	32	R/W	0000_0000h	8.3.4.6/1864
3023_0018	GPIO interrupt status register (GPIO4_ISR)	32	w1c	0000_0000h	8.3.4.7/1865
3023_001C	GPIO edge select register (GPIO4_EDGE_SEL)	32	R/W	0000_0000h	8.3.4.8/1866
3024_0000	GPIO data register (GPIO5_DR)	32	R/W	0000_0000h	8.3.4.1/1855
3024_0004	GPIO direction register (GPIO5_GDIR)	32	R/W	0000_0000h	8.3.4.2/1856
3024_0008	GPIO pad status register (GPIO5_PSR)	32	R	0000_0000h	8.3.4.3/1856
3024_000C	GPIO interrupt configuration register1 (GPIO5_ICR1)	32	R/W	0000_0000h	8.3.4.4/1857
3024_0010	GPIO interrupt configuration register2 (GPIO5_ICR2)	32	R/W	0000_0000h	8.3.4.5/1861
3024_0014	GPIO interrupt mask register (GPIO5_IMR)	32	R/W	0000_0000h	8.3.4.6/1864

Table continues on the next page...

GPIO memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3024_0018	GPIO interrupt status register (GPIO5_ISR)	32	w1c	0000_0000h	8.3.4.7/1865
3024_001C	GPIO edge select register (GPIO5_EDGE_SEL)	32	R/W	0000_0000h	8.3.4.8/1866

8.3.4.1 GPIO data register (GPIOx_DR)

The 32-bit GPIO_DR register stores data that is ready to be driven to the output lines. If the IOMUXC is in GPIO mode and a given GPIO direction bit is set, then the corresponding DR bit is driven to the output. If a given GPIO direction bit is cleared, then a read of GPIO_DR reflects the value of the corresponding signal. Two wait states are required in read access for synchronization.

The results of a read of a DR bit depends on the IOMUXC input mode settings and the corresponding GDIR bit as follows:

- If GDIR[n] is set and IOMUXC input mode is GPIO, then reading DR[n] returns the contents of DR[n].
- If GDIR[n] is cleared and IOMUXC input mode is GPIO, then reading DR[n] returns the corresponding input signal's value.
- If GDIR[n] is set and IOMUXC input mode is not GPIO, then reading DR[n] returns the contents of DR[n].
- If GDIR[n] is cleared and IOMUXC input mode is not GPIO, then reading DR[n] always returns zero.

Address: Base address + 0h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DR																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPIOx_DR field descriptions

Field	Description
DR	Data bits. This register defines the value of the GPIO output when the signal is configured as an output (GDIR[n]=1). Writes to this register are stored in a register. Reading GPIO_DR returns the value stored in the register if the signal is configured as an output (GDIR[n]=1), or the input signal's value if configured as an input (GDIR[n]=0). NOTE: The I/O multiplexer must be configured to GPIO mode for the GPIO_DR value to connect with the signal. Reading the data register with the input path disabled always returns a zero value.

8.3.4.2 GPIO direction register (GPIOx_GDIR)

GPIO_GDIR functions as direction control when the IOMUXC is in GPIO mode. Each bit specifies the direction of a one-bit signal. The mapping of each DIR bit to a corresponding SoC signal is determined by the SoC's pin assignment and the IOMUX table. For more details consult the IOMUXC chapter.

Address: Base address + 4h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	GDIR																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPIOx_GDIR field descriptions

Field	Description
GDIR	GPIO direction bits. Bit n of this register defines the direction of the GPIO[n] signal. NOTE: GPIO_GDIR affects only the direction of the I/O signal when the corresponding bit in the I/O MUX is configured for GPIO. 0 INPUT — GPIO is configured as input. 1 OUTPUT — GPIO is configured as output.

8.3.4.3 GPIO pad status register (GPIOx_PSR)

GPIO_PSR is a read-only register. Each bit stores the value of the corresponding input signal (as configured in the IOMUX). This register is clocked with the ipg_clk_s clock, meaning that the input signal is sampled only when accessing this location. Two wait states are required any time this register is accessed for synchronization.

Address: Base address + 8h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PSR																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPIOx_PSR field descriptions

Field	Description
PSR	GPIO pad status bits (status bits). Reading GPIO_PSR returns the state of the corresponding input signal. Settings:

GPIOx_PSR field descriptions (continued)

Field	Description
	NOTE: The IOMUXC must be configured to GPIO mode for GPIO_PSR to reflect the state of the corresponding signal.

8.3.4.4 GPIO interrupt configuration register1 (GPIOx_ICR1)

GPIO_ICR1 contains 16 two-bit fields, where each field specifies the interrupt configuration for a different input signal.

Address: Base address + Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W																
	ICR15		ICR14		ICR13		ICR12		ICR11		ICR10		ICR9		ICR8	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
	ICR7		ICR6		ICR5		ICR4		ICR3		ICR2		ICR1		ICR0	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPIOx_ICR1 field descriptions

Field	Description
31–30 ICR15	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 15. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
29–28 ICR14	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 14. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
27–26 ICR13	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 13. Settings:

Table continues on the next page...

GPIOx_ICR1 field descriptions (continued)

Field	Description
	Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
25–24 ICR12	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 12. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
23–22 ICR11	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 11. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
21–20 ICR10	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 10. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
19–18 ICR9	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 9. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
17–16 ICR8	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 8. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows:

Table continues on the next page...

GPIOx_ICR1 field descriptions (continued)

Field	Description
	00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
15–14 ICR7	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 7. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
13–12 ICR6	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 6. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
11–10 ICR5	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 5. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
9–8 ICR4	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 4. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
7–6 ICR3	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 3. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive.

Table continues on the next page...

GPIOx_ICR1 field descriptions (continued)

Field	Description
	01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
5–4 ICR2	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 2. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
3–2 ICR1	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 1. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
ICR0	Interrupt configuration 1 fields. This register controls the active condition of the interrupt function for GPIO interrupt 0. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.

8.3.4.5 GPIO interrupt configuration register2 (GPIOx_ICR2)

GPIO_ICR2 contains 16 two-bit fields, where each field specifies the interrupt configuration for a different input signal.

Address: Base address + 10h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W																
	ICR31		ICR30		ICR29		ICR28		ICR27		ICR26		ICR25		ICR24	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
	ICR23		ICR22		ICR21		ICR20		ICR19		ICR18		ICR17		ICR16	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPIOx_ICR2 field descriptions

Field	Description
31–30 ICR31	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 31. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
29–28 ICR30	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 30. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
27–26 ICR29	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 29. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.

Table continues on the next page...

GPIOx_ICR2 field descriptions (continued)

Field	Description
25–24 ICR28	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 28. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
23–22 ICR27	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 27. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
21–20 ICR26	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 26. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
19–18 ICR25	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 25. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
17–16 ICR24	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 24. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.

Table continues on the next page...

GPIOx_ICR2 field descriptions (continued)

Field	Description
15–14 ICR23	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 23. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
13–12 ICR22	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 22. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
11–10 ICR21	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 21. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
9–8 ICR20	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 20. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
7–6 ICR19	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 19. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.

Table continues on the next page...

GPIOx_ICR2 field descriptions (continued)

Field	Description
5–4 ICR18	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 18. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
3–2 ICR17	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 17. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.
ICR16	Interrupt configuration 2 fields. This register controls the active condition of the interrupt function for GPIO interrupt 16. Settings: Bits ICRn[1:0] determine the interrupt condition for signal n as follows: 00 LOW_LEVEL — Interrupt n is low-level sensitive. 01 HIGH_LEVEL — Interrupt n is high-level sensitive. 10 RISING_EDGE — Interrupt n is rising-edge sensitive. 11 FALLING_EDGE — Interrupt n is falling-edge sensitive.

8.3.4.6 GPIO interrupt mask register (GPIOx_IMR)

GPIO_IMR contains masking bits for each interrupt line.

Address: Base address + 14h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPIOx_IMR field descriptions

Field	Description
IMR	Interrupt Mask bits. This register is used to enable or disable the interrupt function on each of the 32 GPIO signals. Settings:

GPIOx_IMR field descriptions (continued)

Field	Description
	Bit IMR[n] (n=0...31) controls interrupt n as follows:
0	MASKED — Interrupt n is disabled.
1	UNMASKED — Interrupt n is enabled.

8.3.4.7 GPIO interrupt status register (GPIOx_ISR)

The GPIO_ISR functions as an interrupt status indicator. Each bit indicates whether an interrupt condition has been met for the corresponding input signal. When an interrupt condition is met (as determined by the corresponding interrupt condition register field), the corresponding bit in this register is set. Two wait states are required in read access for synchronization. One wait state is required for reset.

Address: Base address + 18h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ISR																															
W	w1c																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

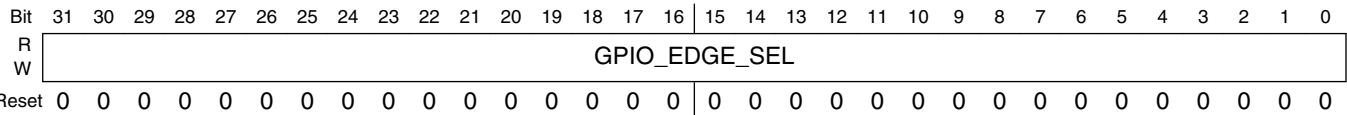
GPIOx_ISR field descriptions

Field	Description
ISR	Interrupt status bits - Bit n of this register is asserted (active high) when the active condition (as determined by the corresponding ICR bit) is detected on the GPIO input and is waiting for service. The value of this register is independent of the value in GPIO_IMR. When the active condition has been detected, the corresponding bit remains set until cleared by software. Status flags are cleared by writing a 1 to the corresponding bit position.

8.3.4.8 GPIO edge select register (GPIOx_EDGE_SEL)

GPIO_EDGE_SEL may be used to override the ICR registers' configuration. If the GPIO_EDGE_SEL bit is set, then a rising edge or falling edge in the corresponding signal generates an interrupt. This register provides backward compatibility. On reset all bits are cleared (ICR is not overridden).

Address: Base address + 1Ch offset



GPIOx_EDGE_SEL field descriptions

Field	Description
GPIO_EDGE_SEL	Edge select. When GPIO_EDGE_SEL[n] is set, the GPIO disregards the ICR[n] setting, and detects any edge on the corresponding input signal.

Chapter 9

External Memory

9.1 External Memory Overview

9.1.1 External Memory Overview

9.1.1.1 DRAM Interface

The chip has a single 16/32-bit DRAM controller.

The DRAM controller and PHY are licensed from SNPS. The key features of the DRAM controller and PHY include:

- LPDDR4-3200 in POP BGA package
- LPDDR4-3200, DDR4-2400, and DDR3L-1600 in Non-POP BGA package
- 32/16-bit DRAM interface
- 800MHz 128-bit AXI bus
- Up to 8GB of memory capacity
- Support various low power modes, clock and power gated operations, that are defined for i.MX 8M Quad. In addition, it shall be able to place the external DRAM into and out of its self-refresh mode as requested by different low-power operating modes

9.1.1.2 eSD/eMMC/SDIO Support

The chip has two Ultra Secured Digital Host Controller (uSDHC) modules for the SD/eMMC interface. It provides the interface between the host system and SD/SDIO/MMC cards. The key features include:

- Supports SD/SDIO standard, up to version 3.0
- Supports MMC standard, up to version 5.1
- Supports 3.3V and 1.8V operation, but does not support 1.2V operation

- Supports 1-bit / 4-bit SD and SDIO modes, 1-bit / 4-bit / 8-bit MMC modes
 - Up to 400 Mbps of data transfer for SDIO cards using 4 parallel data lines in SDR mode
 - Up to 800 Mbps of data transfer for SDIO card using 4 parallel data lines in DDR mode
 - Up to 400 Mbps of data transfer for SDXC cards using 4 parallel data lines in SDR mode
 - Up to 800 Mbps of data transfer for SDXC card using 4 parallel data lines in DDR mode
 - Up to 1600 Mbps of data transfer for MMC cards using 8 parallel data lines in SDR mode
 - Up to 3200 Mbps of data transfer for MMC cards using 8 parallel data lines in DDR mode
- One uSDHC controller (SD1) can support up to 8-bit interface, the other controller (SD2) can only support up to 4-bit interface.

9.1.1.3 Raw NAND Support

The Raw NAND Flash controller consists of three components:

- GPMI as the NAND controller pin interface
- APBH_DMA as the DMA engine that drives the GPMI module
- BCH as the 62-bit error correction hardware engine with an AXI bus master and a private connection to GPMI.

The Raw NAND Flash controller support following key features:

- 8-bit NAND FLASH, up to 4 devices supported by 4 chip-selects and 1 ganged ready/busy
- ONFI 2.x complaint, synchronous clock rate of up to 100 MHz with data rate of up to 200 MB/s
- Support ONFI NAND for Micron and Hynix and Toggle NAND for Toshiba and Samsung
- BCH62 for ECC, up to 200MB/s

9.1.1.4 Quad SPI Interface

There is one Quad Serial Peripheral Interface (QuadSPI) block that acts as an interface to one or two external serial flash devices, each with up to four bidirectional data lines. The key features include:

- Each channel can be configured as 1/2/4-bit operation
- Support both dual-channel or single-channel operation

- Support both SDR mode and DDR mode
- Support up to 80MHz SDR Mode and 50MHz DDR Mode

9.2 DRC Performance Monitor (DRC_PERF_MON)

9.2.1 Introduction

This section provides an introduction to the DDR performance monitor.

9.2.2 Functional description

The main functions of the DDR Performance Monitor include:

- Monitor the performance signals provided by DDR controller
- Sample the MRR read data (LPDDR4) or MPR read data (DDR4) after DDR controller issues an Mode register read command

9.2.2.1 Performance Counter

This module contain a number of counters. One of them is a special counter which counts "time". The rest of them are used for signal monitoring.

The special counter (COUNTER0) will be "cleared" when COUNTER0_CTRL[counter_0_clear] is set. It will start to increment if COUNTER0_CTRL[counter_0_enable] is set to high. This counter will overflow when the value reaches 0xFFFF_FFFF. Once this happens, all the counters will lock and an interrupt will be generated.

The counters 1-3 used for signal monitoring operates the same way as the special counter. They will be cleared when counter_n_clear is high, and become active when both counter_0 and counter_n are enabled and not overflowing. When active, counter_n will increase if the monitored signal is high.

The signal being monitored can be selected from the below list via counter_n_csv:

- WAQ_COUNT_0[3:0]: the number of used positions in the write queue FIFO
- WAQ_POP_0: one write granted by port arbiter
- RAQ_WCOUNT_0[3:0]: number of used positions in the read queue
- RAQ_POP_0: one read granted by port arbiter
- LPR_CREDIT_CNT[6:0]: number of available low priority read CAM slots free

- HPR_CREDIT_CNT[6:0]: number of available high priority read CAM slots free
- WR_CREDIT_CNT[6:0]: number of available write CAM slots free
- stat_ddrc_reg_selfref_type[1:0] self-refresh status and type:
 - 00 - not in self refresh
 - 11 - self-refresh by automatic
 - 10 - caused by SW/HW interface
- perf_hif_rd: asserts for every read command to the DDRC
- perf_hif_wr: asserts for every write command to the DDRC
- perf_hif_rmw: asserts for every read modify write to the DDRC
- perf_hif_hi_pri_rd: asserts for every high priority read
- perf_hpr_req_with_nocredit: asserts when there is a high priority read request not served due to no available credit
- perf_hpr_xact_when_critical: asserts for every high priority read transaction that is scheduled when the high priority queue is in critical state
- perf_lpr_req_with_nocredit: asserts when there is a low priority read request not served due to no available credit
- perf_lpr_xact_when_critical: asserts for every low priority read transaction that is scheduled when the low priority queue is in critical state
- perf_wr_xact_when_critical: asserts for every write transaction that is scheduled when the write queue is in critical state
- perf_dfi_rd_data_cycles: assert for every read data cycle
- perf_dfi_wr_data_cycles: assert for every write data cycle
- perf_rdwr_transitions: assert for every read->write or write->read transition
- perf_op_is_precharge: assert for every precharge command issued
- perf_op_is_activate: assert for every active command issued
- perf_op_is_load_mode: assert for every MRW or MRR command issued
- perf_op_is_mwr: assert for every masked write command issued
- perf_op_is_rd: assert for every read command issued
- perf_op_is_rd_activate: assert for every read active issued
- perf_op_is_refresh: assert for every refresh command issued
- perf_op_is_wr: assert for every write command issued
- perf_raw_hazard: asserts for every read-after-write collision that happens in the controller

9.2.2.2 Mode register read

DDR performance monitor will sample MRR read data (LPDDR4) or MPR read data (DDR4) after DDRC issues a mode register command.

Serial mode and parallel mode are supported for DDR4 MPR read and it can be selected via MRR_0_DATA[mrr_mode_sel].

9.2.3 Memory Map and register definition

This section includes the DRC_PERF_MON module memory map and detailed descriptions of all registers.

9.2.3.1 DRC_PERF_MON register descriptions

9.2.3.1.1 DRC_PERF_MON Memory map

drc_perf_monitor base address: 0h

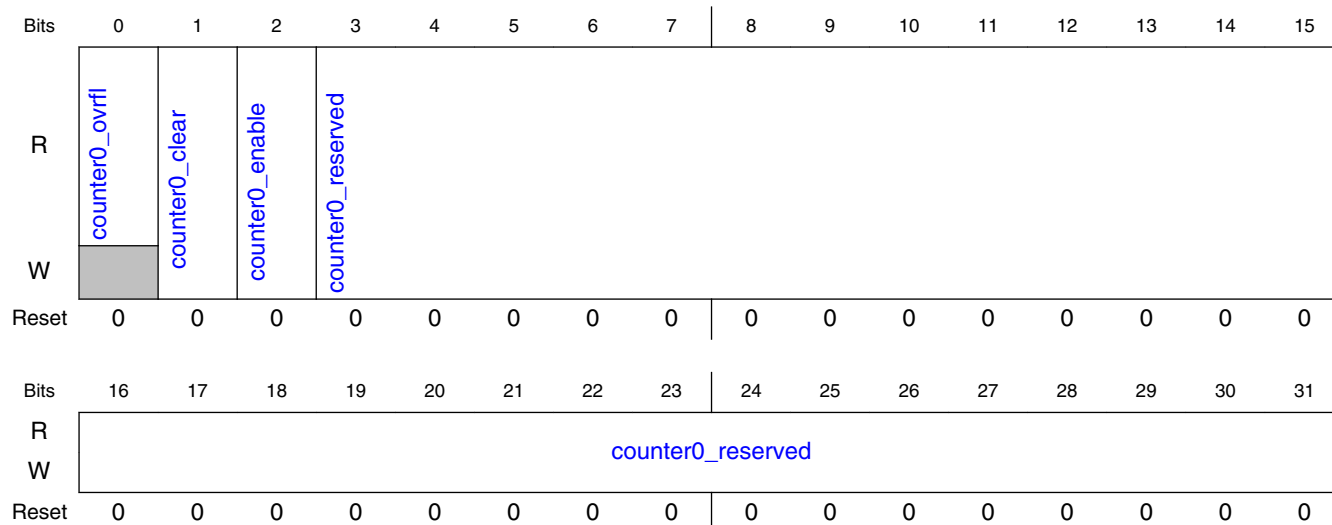
Offset	Register	Width (In bits)	Access	Reset value
0h	Counter0 control (COUNTER0_CTRL)	32	RW	0000_0000h
4h	Counter1 control (COUNTER1_CTRL)	32	RW	Table 9-
8h	Counter2 control (COUNTER2_CTRL)	32	RW	Table 9-
Ch	Counter3 control (COUNTER3_CTRL)	32	RW	Table 9-
20h	Counter0 Data (COUNTER0_DATA)	32	RO	0000_0000h
24h	Counter1 Data (COUNTER1_DATA)	32	RO	0000_0000h
28h	Counter2 Data (COUNTER2_DATA)	32	RO	0000_0000h
2Ch	Counter3 Data (COUNTER3_DATA)	32	RO	0000_0000h
40h	MRR0 data (MRR0_DATA)	32	RW	0000_0000h
44h	MRR1 data (MRR1_DATA)	32	RW	0000_0000h

9.2.3.1.2 Counter0 control (COUNTER0_CTRL)

9.2.3.1.2.1 Offset

Register	Offset
COUNTER0_CTRL	0h

9.2.3.1.2.2 Diagram



9.2.3.1.2.3 Fields

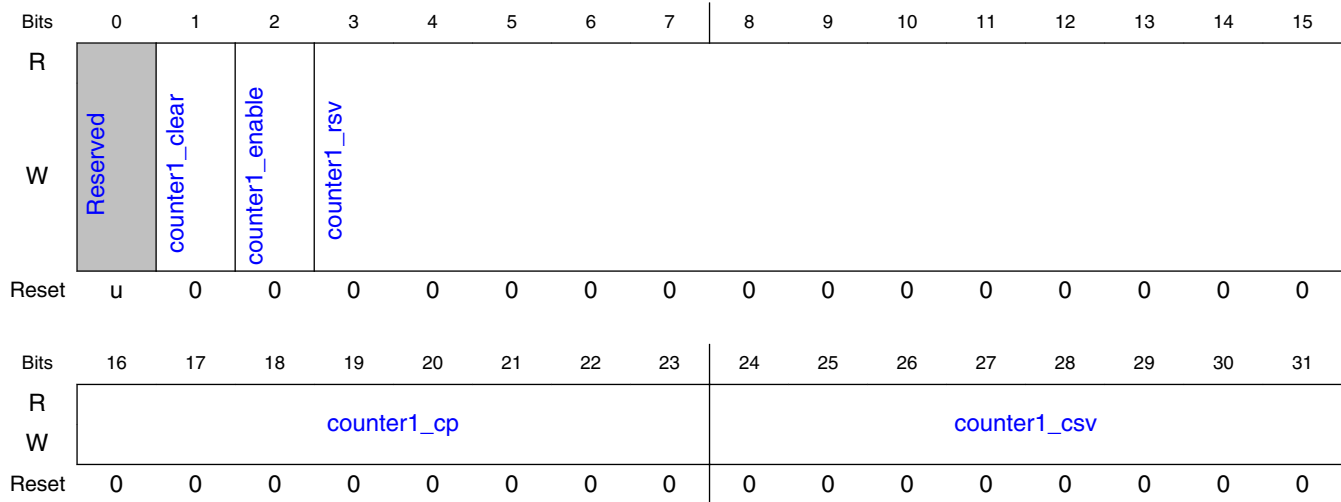
Field	Function
0 counter0_ovrfl	counter0_overflow Asserted when Counter 0 overflows
1 counter0_clear	counter0_clear Clear Counter 0 when asserted
2 counter0_enable	counter0_enable Enable Counter 0 when asserted
3-31 counter0_reserved	reserved

9.2.3.1.3 Counter1 control (COUNTER1_CTRL)

9.2.3.1.3.1 Offset

Register	Offset
COUNTER1_CTRL	4h

9.2.3.1.3.2 Diagram



9.2.3.1.3.3 Fields

Field	Function
0 —	Reserved
1 counter1_clear	counter1_clear Clear Counter 1 when asserted
2 counter1_enable	counter1_enable Enable Counter 1 when asserted
3-15 counter1_rsv	reserved
16-23 counter1_cp	counter1_conter_parameter Counter Parameter
24-31 counter1_csv	counter1_count_value_select 8'h00: use counter_1_cp for overflow calculation If configured with a non-0 value, it will select the signal to be monitored in counter 1: 8'h01: stat_ddrc_reg_selfref_type == counter_1_cp[1:0] 8'h04: raq_pop_0 8'h05: waq_pop_0 8'h06: raqb_pop_0 8'h07: raqb_wcount_0 == counter_1_cp[3:0] 8'h08: raqr_wcount_0 == counter_1_cp[3:0] 8'h09: waq_wcount_0 == counter_1_cp[3:0] 8'h10: lpr_credit_cnt == counter_1_cp[6:0] 8'h11: hpr_credit_cnt == counter_1_cp[6:0]

DRC Performance Monitor (DRC_PERF_MON)

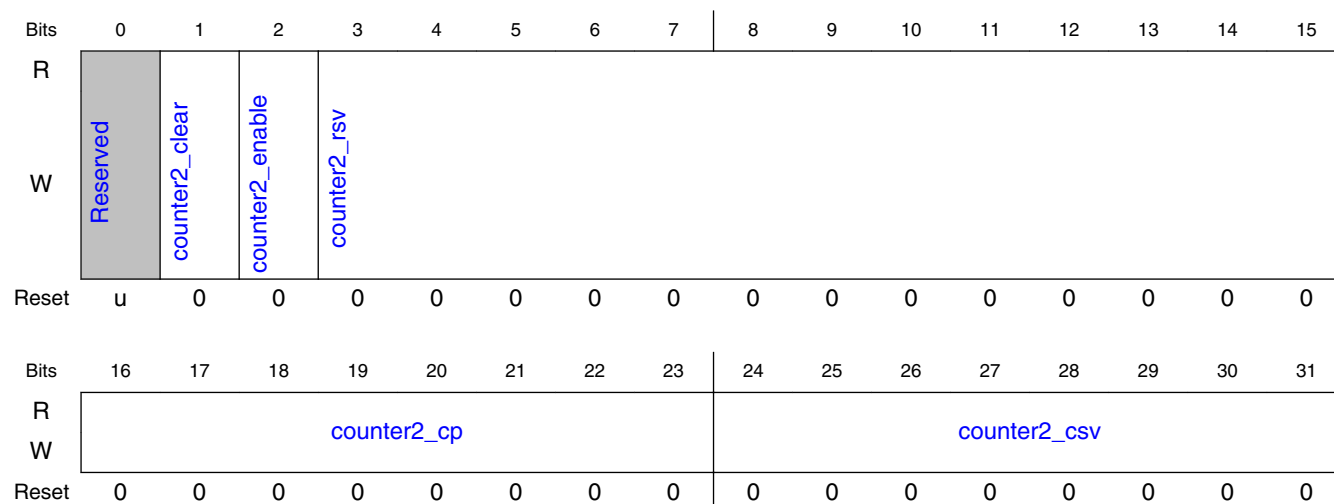
Field	Function
	8'h12: wr_credit_cnt == counter_1_cp[6:0]
	8'h20: perf_hif_rd
	8'h21: perf_hif_wr
	8'h22: perf_hif_rmw
	8'h23: perf_hif_hi_pri_rd
	8'h24: perf_hpr_req_with_nocredit
	8'h25: perf_hpr_xact_when_critical
	8'h26: perf_lpr_req_with_nocredit
	8'h27: perf_lpr_xact_when_critical
	8'h29: perf_wr_xact_when_critical
	8'h2a: perf_dfi_rd_data_cycles
	8'h2b: perf_dfi_wr_data_cycles
	8'h30: perf_rdwr_transitions
	8'h31: perf_op_is_precharge
	8'h32: perf_op_is_activate
	8'h33: perf_op_is_load_mode
	8'h34: perf_op_is_mwr
	8'h35: perf_op_is_rd
	8'h36: perf_op_is_rd_activate
	8'h37: perf_op_is_refresh
	8'h38: perf_op_is_wr
	8'h39: perf_raw_hazard

9.2.3.1.4 Counter2 control (COUNTER2_CTRL)

9.2.3.1.4.1 Offset

Register	Offset
COUNTER2_CTRL	8h

9.2.3.1.4.2 Diagram



9.2.3.1.4.3 Fields

Field	Function
0 —	Reserved
1 counter2_clear	counter2_clear Clear Counter 2 when asserted
2 counter2_enable	counter2_enable Enable Counter 2 when asserted
3-15 counter2_rsv	reserved
16-23 counter2_cp	counter2 count parameter Counter Parameter
24-31 counter2_csv	counter2_count_value_select 8'h00: use counter_2_cp for overflow calculation If configured with a non-0 value, it will select the signal to be monitored in counter 2: 8'h01: stat_ddrc_reg_selfref_type == counter_2_cp[1:0] 8'h04: raq_pop_0 8'h05: waq_pop_0 8'h06: raqb_pop_0 8'h07: raqb_wcount_0 == counter_2_cp[3:0] 8'h08: raqr_wcount_0 == counter_2_cp[3:0] 8'h09: waq_wcount_0 == counter_2_cp[3:0] 8'h10: lpr_credit_cnt == counter_2_cp[6:0] 8'h11: hpr_credit_cnt == counter_2_cp[6:0]

DRC Performance Monitor (DRC_PERF_MON)

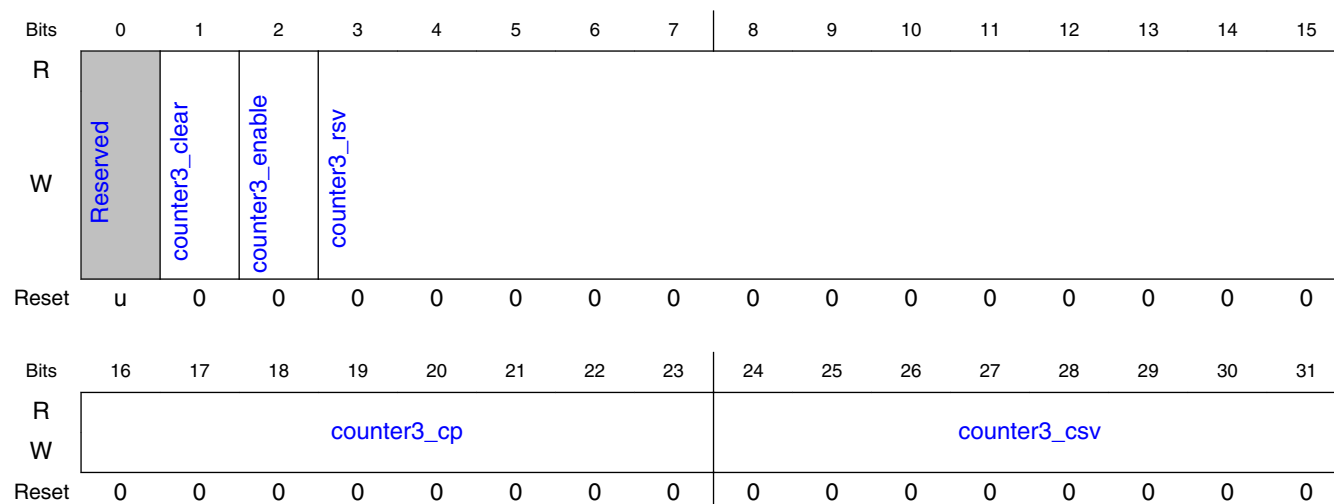
Field	Function
	8'h12: wr_credit_cnt == counter_2_cp[6:0]
	8'h20: perf_hif_rd
	8'h21: perf_hif_wr
	8'h22: perf_hif_rmw
	8'h23: perf_hif_hi_pri_rd
	8'h24: perf_hpr_req_with_nocredit
	8'h25: perf_hpr_xact_when_critical
	8'h26: perf_lpr_req_with_nocredit
	8'h27: perf_lpr_xact_when_critical
	8'h29: perf_wr_xact_when_critical
	8'h2a: perf_dfi_rd_data_cycles
	8'h2b: perf_dfi_wr_data_cycles
	8'h30: perf_rdwr_transitions
	8'h31: perf_op_is_precharge
	8'h32: perf_op_is_activate
	8'h33: perf_op_is_load_mode
	8'h34: perf_op_is_mwr
	8'h35: perf_op_is_rd
	8'h36: perf_op_is_rd_activate
	8'h37: perf_op_is_refresh
	8'h38: perf_op_is_wr
	8'h39: perf_raw_hazard

9.2.3.1.5 Counter3 control (COUNTER3_CTRL)

9.2.3.1.5.1 Offset

Register	Offset
COUNTER3_CTRL	Ch

9.2.3.1.5.2 Diagram



9.2.3.1.5.3 Fields

Field	Function
0 —	Reserved
1 counter3_clear	counter3 clear Clear Counter 3 when asserted
2 counter3_enable	counter3_enable Enable Counter 3 when asserted
3-15 counter3_rsv	reserved
16-23 counter3_cp	counter3 count parameter
24-31 counter3_csv	counter3_count_value_select 8'h00: use counter_3_cp for overflow calculation If configured with a non-0 value, it will select the signal to be monitored in counter 3: 8'h01: stat_ddrc_reg_selfref_type == counter_3_cp[1:0] 8'h04: raq_pop_0 8'h05: waq_pop_0 8'h06: raqb_pop_0 8'h07: raqb_wcount_0 == counter_3_cp[3:0] 8'h08: raqr_wcount_0 == counter_3_cp[3:0] 8'h09: waq_wcount_0 == counter_3_cp[3:0] 8'h10: lpr_credit_cnt == counter_3_cp[6:0] 8'h11: hpr_credit_cnt == counter_3_cp[6:0]

DRC Performance Monitor (DRC_PERF_MON)

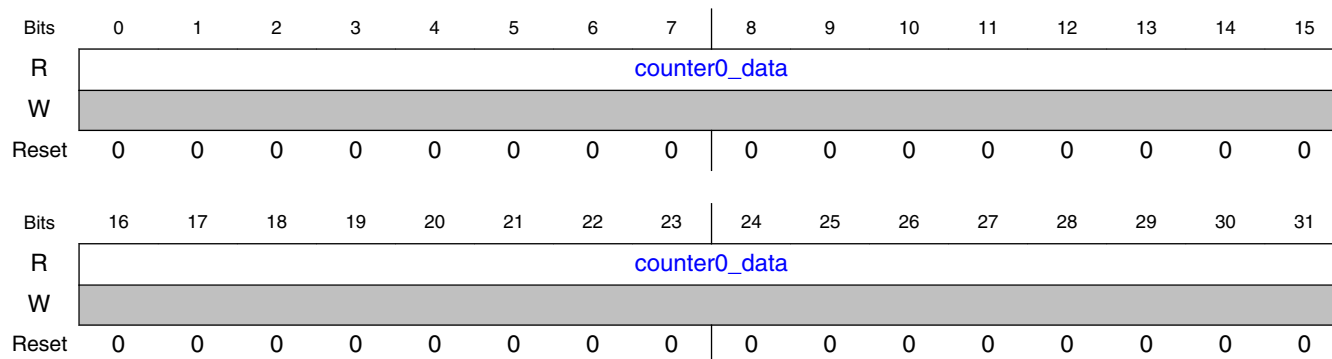
Field	Function
	8'h12: wr_credit_cnt == counter_3_cp[6:0]
	8'h20: perf_hif_rd
	8'h21: perf_hif_wr
	8'h22: perf_hif_rmw
	8'h23: perf_hif_hi_pri_rd
	8'h24: perf_hpr_req_with_nocredit
	8'h25: perf_hpr_xact_when_critical
	8'h26: perf_lpr_req_with_nocredit
	8'h27: perf_lpr_xact_when_critical
	8'h29: perf_wr_xact_when_critical
	8'h2a: perf_dfi_rd_data_cycles
	8'h2b: perf_dfi_wr_data_cycles
	8'h30: perf_rdwr_transitions
	8'h31: perf_op_is_precharge
	8'h32: perf_op_is_activate
	8'h33: perf_op_is_load_mode
	8'h34: perf_op_is_mwr
	8'h35: perf_op_is_rd
	8'h36: perf_op_is_rd_activate
	8'h37: perf_op_is_refresh
	8'h38: perf_op_is_wr
	8'h39: perf_raw_hazard

9.2.3.1.6 Counter0 Data (COUNTER0_DATA)

9.2.3.1.6.1 Offset

Register	Offset
COUNTER0_DATA	20h

9.2.3.1.6.2 Diagram



9.2.3.1.6.3 Fields

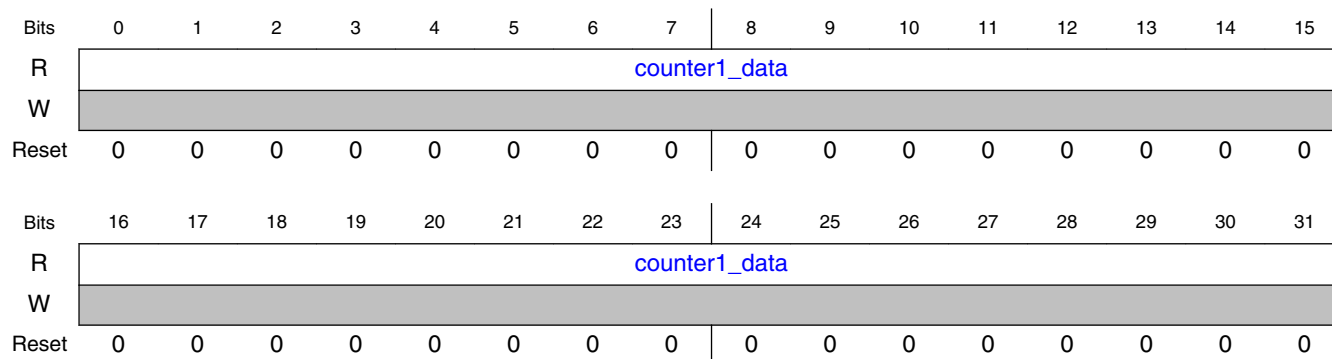
Field	Function
0-31 counter0_data	counter0 data

9.2.3.1.7 Counter1 Data (COUNTER1_DATA)

9.2.3.1.7.1 Offset

Register	Offset
COUNTER1_DATA	24h

9.2.3.1.7.2 Diagram



9.2.3.1.7.3 Fields

Field	Function
0-31 counter1_data	counter1 data

9.2.3.1.8 Counter2 Data (COUNTER2_DATA)

9.2.3.1.8.1 Offset

Register	Offset
COUNTER2_DATA	28h

9.2.3.1.8.2 Diagram

Bits	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	counter2_data															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
R	counter2_data															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.2.3.1.8.3 Fields

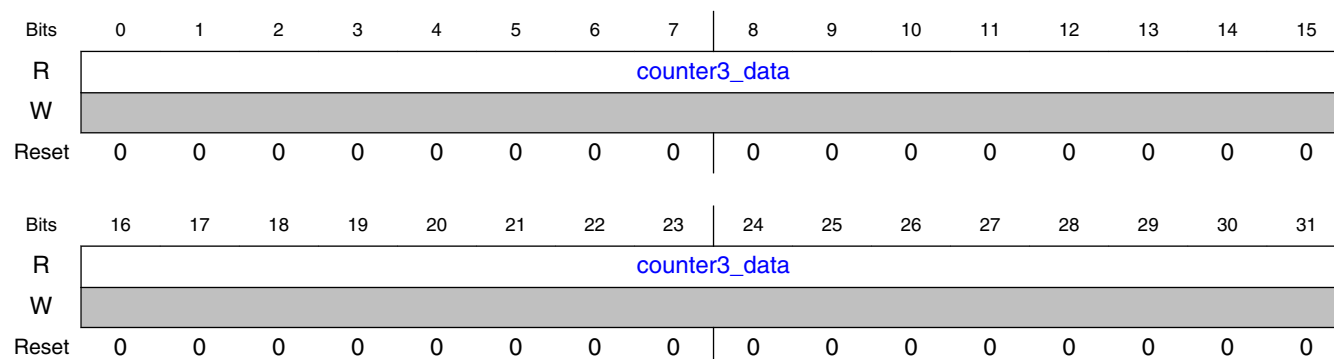
Field	Function
0-31 counter2_data	counter2 data

9.2.3.1.9 Counter3 Data (COUNTER3_DATA)

9.2.3.1.9.1 Offset

Register	Offset
COUNTER3_DATA	2Ch

9.2.3.1.9.2 Diagram



9.2.3.1.9.3 Fields

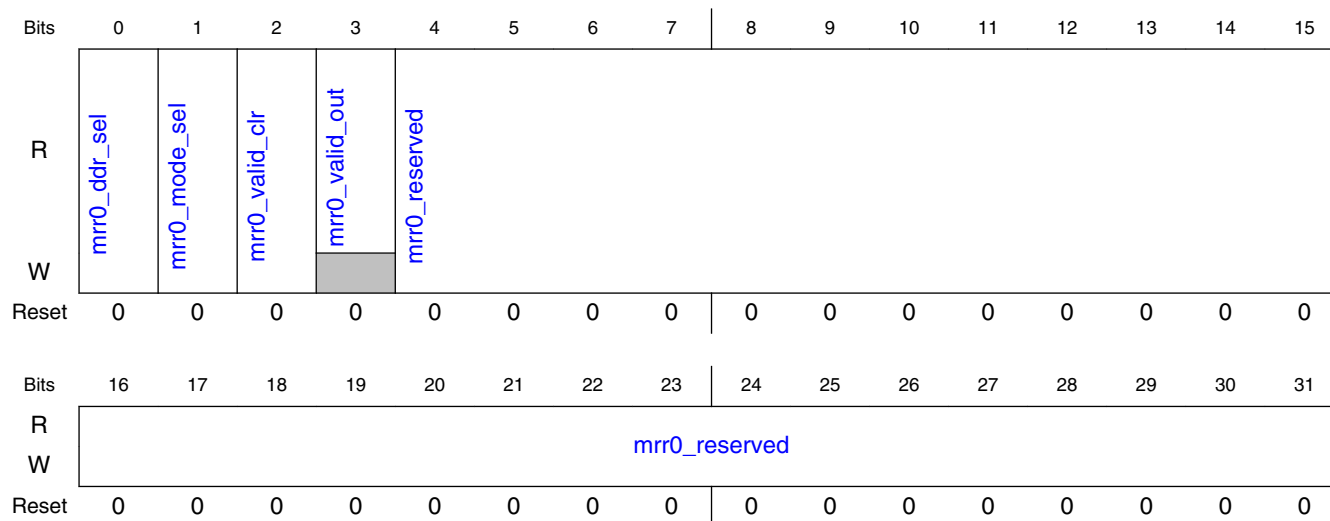
Field	Function
0-31 counter3_data	counter3 data

9.2.3.1.10 MRR0 data (MRR0_DATA)

9.2.3.1.10.1 Offset

Register	Offset
MRR0_DATA	40h

9.2.3.1.10.2 Diagram



9.2.3.1.10.3 Fields

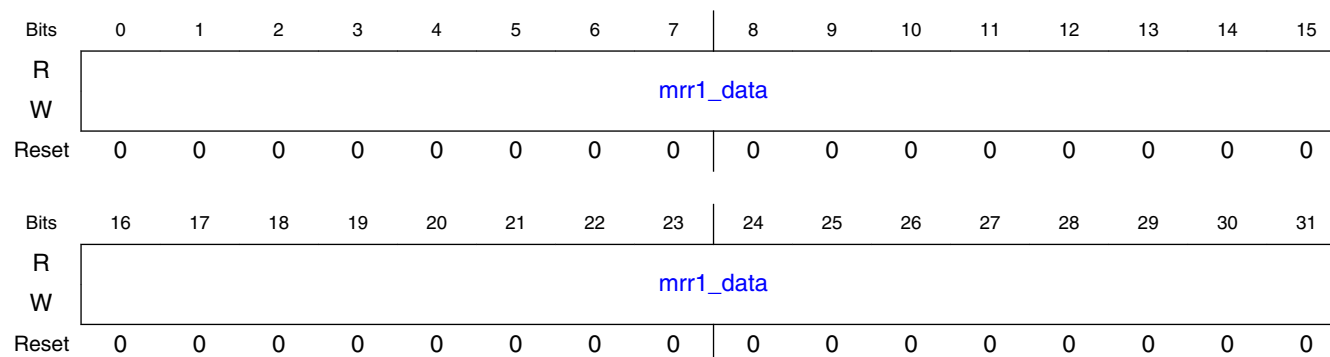
Field	Function
0 mrr0_dds_sel	mrr0 ddr selection 0b - DDR4 mode 1b - LPDDR4 mode
1 mrr0_mode_sel	mrr0 mode selection 0b - Parallel Mode (only valid on DDR4 mode) 1b - Serial Mode (only valid on LPDDR3/LPDDR4 mode)
2 mrr0_valid_clr	mrr0 valid clear 1b - Clear previous MRR result
3 mrr0_valid_out	mrr0 data valid 1b - MRR result is valid
4-31 mrr0_reserved	mrr0 reserved

9.2.3.1.11 MRR1 data (MRR1_DATA)

9.2.3.1.11.1 Offset

Register	Offset
MRR1_DATA	44h

9.2.3.1.11.2 Diagram



9.2.3.1.11.3 Fields

Field	Function
0-31 mrr1_data	mrr1 data

9.3 DDR Controller (DDRC)

9.3.1 Introduction

The DDRC (DDR Controller) is a very low power, high efficiency, low-latency and high performance memory controller for interfacing DDR based memories.

NOTE

The information within this block guide is Synopsys Proprietary. Used with permission.

The following types of SDRAMs are supported by the DDRC:

- DDR4
- LP-DDR4

9.3.1.1 Block diagram

The following shows the block diagram for the DDR Controller.

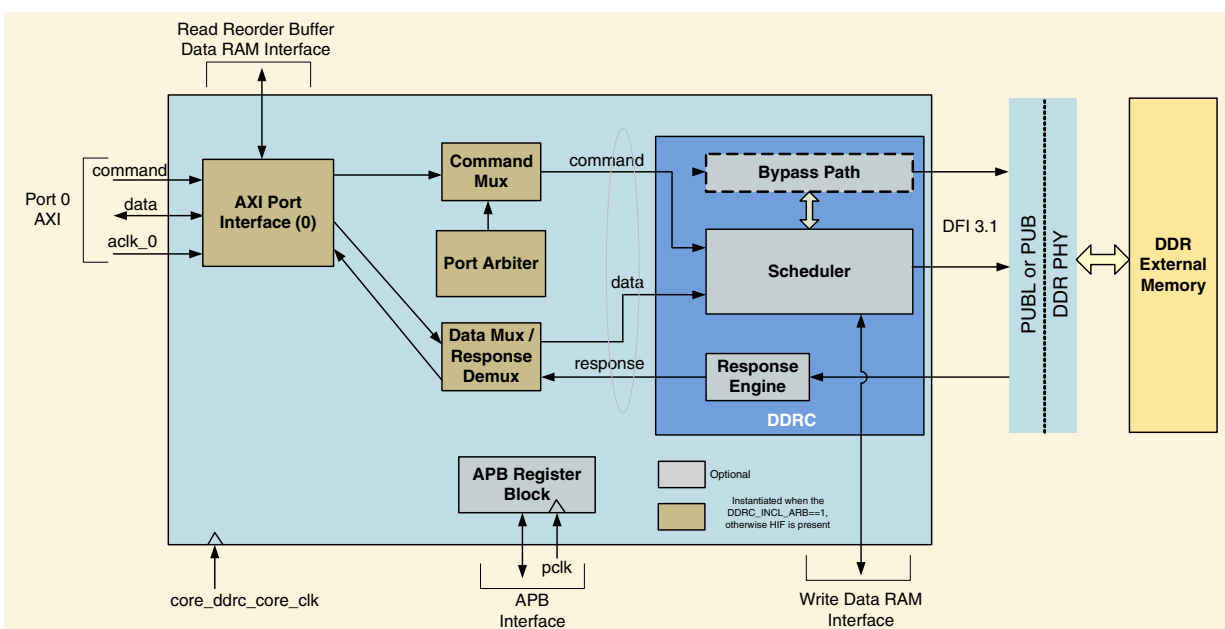


Figure 9-1. DDRC block diagram

The PUB /PUBL is the PHY Utility Block is an adaption layer between the controller and the PHY handling control function.

9.3.1.2 Features

The DDRC includes the following features:

- Direct software request control or programmable internal control for ZQ short calibration cycles
- Support for ZQ long calibration after self-refresh exit
- Direct software request control or programmable internal control for ZQ calibration cycles
- Support for ZQ Reset feature through software
- Dynamic scheduling to optimize bandwidth and latency
- Read and write buffers in fully associative CAMs, 32 each
- Delayed writes for optimum performance on SDRAM data bus
- Out of order execution of commands maximises the SDRAM efficiency
- Hardware configurable and software programmable Quality of Service (QoS) support:

- Support for three traffic classes on read commands—high priority reads, variable priority reads and low priority reads
- Support for two traffic classes on write commands—normal priority writes and variable priority writes
- Support for port urgent and port throttling control
- Programmable SDRAM parameters
- Supports BL16 burst length
- Supports 1,2 memory ranks
- Control options to avoid starvation of lower priorities
- Guaranteed coherency for write-after-read (WAR) and read-after-write (RAW) hazards
- Write combine to allow multiple writes to the same address to be combined into a single write to SDRAM; supported for same starting address
- Paging policy selectable by configuration registers as any of the following:
 - Leave pages open after accesses, or
 - Close page when there are no further accesses available in the controller for that page, or
 - Auto-precharge with each access, with an optimization for page-close mode which leaves the page open after a flush for read-write and write-read collision cases
- Supports automatic SDRAM power-down entry and exit caused by lack of transaction arrival for programmable time
- Supports automatic Clock Stop entry and exit caused by lack of transaction arrival
- Supports automatic DDRC low power mode operation caused by lack of transaction arrival for programmable time via Hardware Low Power Interface
- Supports self-refresh entry and exit as follows:
 - Supports automatic self-refresh entry and exit caused by lack of transaction arrival for programmable time
 - Support for self-refresh entry and exit under software control
 - Support for self-refresh entry and exit using dedicated DDRC hardware low power interface control (similar to the AMBA 3 AXI protocol low power control interface)
- Support for dynamically changing clock frequency while in self-refresh:
 - DDR4 DLL-off mode supported
 - Shadow timing registers provided to allow fast frequency changing
- Support for explicit SDRAM mode register updates under software control
- Flexible address mapper logic to allow application specific mapping of row, column, bank, and rank bits
- Programmable support for 1T or 2T timing
- User-selectable refresh control options:
 - Controller-generated auto-refreshes at programmable average intervals

- In multi-rank designs, an offset can be applied to each rank's refresh timer to allow rank refreshes to expire at different times (this can increase efficiency by allowing traffic to continue to other ranks while a given rank is being refreshed)
- Ability to group up to 8 controller-generated refreshes together to be issued consecutively (this reduces the frequency of page closings, increasing overall efficiency)
- When controller-generated refreshes are grouped, some refreshes can be issued speculatively when the controller is idle for a programmable period of time
- Ability to disable controller-generated auto-refreshes
- Ability to issue a refresh through direct software request
- When LP-DDR4 is used, user-selectable ability to perform per-bank refreshes rather than all-banks refreshes
- When DDR4 is used, fine granularity refresh can be selected
- DDR4 protocol up to DDR4-3200 speed grade (1:2 frequency ratio mode) and DDR4-2133 speed grade (1:1 frequency ratio mode) are supported by the DDRC.

The following features are supported:

- Data Bus Inversion (DBI)
- Geardown mode may be programmed where allowed by memory and PHY specification (not supported for 1:1 frequency ratio mode)
- Maximum power saving mode
- Multi-purpose register (MPR) reads and writes
- Per DRAM addressability
- Fine granularity refresh
- Cyclic redundancy check (CRC) with retry (not supported for 1:1 frequency ratio mode)
- Command/address latency
- Programmable Preamble (not supported for 1:1 frequency ratio mode)
- 3DS with up to 8 logical ranks/DDR4-2666
- Supports Dual Data Channel, also called "Shared AC", with a common address and command bus

9.3.2 Functional description

The DDRC converts system bus transactions into memory commands on the DFI interface that are compliant with the DDR protocols. The DDRC supports both memory components and DIMMs (Dual in-line Memory Module).

Please see the block diagram at [Block diagram](#).

The main components of the block are:

- **AXI Port Interface (XPI) block:** This block provides the interface to the application ports. It provides bus protocol handling, data buffering and reordering for read data, data bus size conversion (upsizing or downsizing), and memory burst address alignment. Read data is stored in a SRAM, read re-order buffer and returned in order, to the AXI ports. The SRAM may be instantiated as embedded memory external to the DDRC or be implemented as flops within the DDRC.
- **Port Arbiter (PA) block:** This block provides latency sensitive, priority based arbitration between the addresses issued by the XPIs (by the ports).
- **DDR Controller (DDRC) block:** This block contains a logical CAM (Content Addressable Memory), which can be synthesized using standard cells. This holds information on the commands, which is used by the scheduling algorithms to optimally schedule commands to be sent to the PHY, based on priority, bank/rank status and DDR timing constraints. A bypass path is also provided (optionally).
- **APB Register Block:** This block contains the software accessible registers.

The details on DDRC are as follows:

- Write data is stored in DDRC until its associated command is issued to the PHY.
- Read data is handled by the response engine in the DDRC and is returned in the order of scheduled read commands on the HIF.
- ECC handling is an optional function which is handled by logic modules within the DDRC in the write data path and in the response engine.

1:1 and 1:2 frequency ratios are terms used for the frequency ratio between the DDRC clock (`core_ddrc_core_clk`) and the memory clock.

When `MEMC_PROG_FREQ_RATIO = 1` the parameter `MEMC_FREQ_RATIO` is tied to 2 and the HIF data bus is four times the memory data width.

The register `MSTR.frequency_ratio` controls whether the controller operates in 1:1 frequency ratio mode or 1:2 frequency ratio mode. When `MSTR.frequency_ratio` is programmed to 1 the DDRC clock rate is the same as the memory clock rate (referred to as SDR, single data rate) i.e. 1:1 frequency ratio mode. When `MSTR.frequency_ratio` is programmed to 0 the DDRC clock rate is one half the memory clock rate (referred to as HDR, half data rate) i.e. 1:2 frequency ratio mode.

When `MEMC_PROG_FREQ_RATIO = 0` the parameter `MEMC_FREQ_RATIO` controls whether the controller operates in 1:1 frequency ratio mode or 1:2 frequency ratio mode. When `MEMC_FREQ_RATIO=1` the DDRC clock rate is the same as the memory clock rate (referred to as SDR, single data rate) i.e. 1:1 frequency ratio mode. The HIF data bus is twice the memory data width. When `MEMC_FREQ_RATIO=2` the DDRC clock rate is one half the memory clock rate (referred to as HDR, half data rate), that is, 1:2 frequency ratio mode. The HIF data bus is four times the memory data width

In the case of 1:1 frequency ratio mode, the DDRC and PHY operate at the same clock frequency and are referred to as a ‘matched frequency system’ in the DFI Specification.

For example if MEMC_PROG_FREQ_RATIO=1, in 1:1 frequency ratio mode, if the memory data width is 32 bits and the memory clock rate is 400 MHz or 800 Mbps, the HIF data bus would be 128 bits wide and only the lower half of the dfi bus is utilized. The DDRC clock rate would be 400 MHz. If the frequency ratio mode is changed to 1:2, the HIF data bus would be 128 bits wide and all of the dfi data bus is utilized. The DDRC clock rate would be 200 MHz.

For example if MEMC_PROG_FREQ_RATIO=0, in 1:1 frequency mode, if the memory data width is 32 bits and the memory clock rate is 400 MHz or 800 Mbps, the HIF data bus would be 64 bits wide and all of the dfi data bus is utilized. The DDRC clock rate would be 400 MHz. If the frequency mode is changed to 1:2, the HIF data bus would be 128 bits wide and all of the dfi data bus is utilized. The DDRC clock rate would be 200 MHz.

The terms “DFI clock” and “DFI PHY clock” are used in this document in accordance with the DFI specification:

- The DFI clock is the clock on which the DFI interface runs. It is the `core_ddrc_core_clk`.
- The DFI PHY clock is the SDRAM clock. In 1:1 frequency ratio, this has the same frequency as the DFI clock. In 1:2 frequency ratio, this has twice the frequency of the DFI clock.

9.3.2.1 AXI Port Interface (XPI)

The AXI protocol is burst-based with read and write request channels that specify the host ID for the request, start byte address, burst length, burst size, and burst type. This information is processed by the interface and is used subsequently by the DDRC.

The XPI interfaces the AXI application port to the DDRC and performs the following main functions:

- Read address generation
- Write address generation
- Read data generation
- Read data and response generation
- Write response generation

The XPI converts AXI bursts into read and write requests, which are forwarded to the Port Arbiter (PA). In the opposite direction, the XPI converts the responses from the DDRC into appropriate AXI responses.

The interface between XPI and PA is same as HIF (but with independent channels for read and write commands). Each AXI port can be independently configured to operate with a clock that is synchronous or asynchronous to the memory controller clock.

The AXI Specification requires that transactions must not cross a 4K address boundary. AXI compliant masters must meet this requirement. However, if a master does not comply with this AXI protocol requirement, the user can relax this restriction in the DDRC and set the boundary between 4K and 4G using the hardware parameter `DDRC_AXI_ADDR_BOUNDARY`.

9.3.2.1.1 Read Address Channel

The AXI read address channel has the following features:

- The read transaction can be of any length up to 256 for incremental bursts, 16 for wrapping bursts.
- The burst types supported are incremental and wrapping.
- The burst start address can be unaligned to the AXI data width boundaries
- The size of the burst can be less than the full width of the AXI data bus (also known as sub-sized transfers).

The signals on the read address channel are registered into the XPI in accordance with the AXI valid/ready handshaking protocol and are synchronous with the AXI clock (aclk).

Read requests are accepted if the request can be written into the Read Address Queue (RAQ). This is used to store all the read address requests. If `DDRC_XPI_USE2RAQ_n` is set to 0, there is single address queue on a given port. If `DDRC_XPI_USE2RAQ_n` is set to 1, there are two address queues on a given port, named Blue and Red address queues. Clock domain crossing from aclk to DDRC clock (`core_ddrc_core_clk`) is performed in the RAQ block. The depth of the RAQ can be configured to suit the system requirements of the user.

After registering the read address channel by the RAQ, the following operations are performed:

- Generation of new read requests is based on alignment (derived from AXI address and size), burst lengths (derived from AXI length and memory burst length), and

burst type (derived from AXI incremental or wrapping). Each AXI burst is divided into packets of length equal to the memory burst length (BL2, BL4, BL8, BL16).

- Generation of new HIF address in the case of unaligned burst and burst expansion. A burst is aligned to the memory burst boundary when:
 - $A(R|W)ADDR[X]=0$ if BL2
 - $A(R|W)ADDR[1+X:X]=0$ if BL4
 - $A(R|W)ADDR[2+X:X]=0$ if BL8 or
 - $A(R|W)ADDR[3+X:X]=0$ if BL16, Where, X is log2 of the number of data bytes in the SDRAM interface. Therefore,
 - If MEMC_DRAM_DATA_WIDTH = 8, X = 0
 - If MEMC_DRAM_DATA_WIDTH = 16, X = 1
 - If MEMC_DRAM_DATA_WIDTH = 32, X = 2 and so on

In case of an unaligned burst, the first read request is unaligned and the remaining read requests are aligned.

In general, realignment to a memory burst boundary potentially causes some data beats to be discarded (affecting bandwidth) and potentially introduces additional latency on the read data and response channel.

The arready output is defaulted to logic one and remains in logic one unless the RAQ becomes full or a WRAP burst is requested (arready is low on the next cycle after a WRAP burst is accepted). The read transaction is popped from the RAQ when it is not empty.

The XPI handles the generation of the token which is used by the DDRC for identifying the read command and corresponding data.

9.3.2.1.2 Write Address Channel

Write Address Queue (WAQ) is used to store all the addresses for write requests from a given port. There is a single queue for all AXI IDs from a given port. The write address channel behavior is similar to the read address channel.

The depth of the WAQ can be configured to suit the system requirements of the user. Write address and read address channels are independent and the ordering between the write and read requests may not be preserved.

To preserve the sequence, a higher-level protocol needs to wait for read/write response before sending the next transaction.

Transactions across the ports are independent and can be issued in any order.

The write command is not forwarded to the DDRC until the write data is collected and the strobes are evaluated.

9.3.2.1.3 Wrap Burst Expansion

A WRAP burst may be expanded by the XPI into multiple SDRAM transactions. This is also true for cases where WRAP bursts, which could be accommodated by a single transaction on the SDRAM interface. A WRAP burst is expanded into a single SDRAM transaction only in particular cases as highlighted in the tables below.

NOTE

SDRAM burst length 16 is not supported by XPI in this case. If the burst length is programmed to 16, XPI uses the value 8.

Table 9-1. MEMC_BURST_LENGTH 16 Wrap Expansion

Direction	Port Data Width	(SDRAM BurstLength8&&Quarter Burst) (SDRAM BurstLength4&&Half Burst) (SDRAM BurstLength4&&Quarter Burst)	(SDRAM Burst Length16&&Quarter Burst) (SDRAM Burst Length8&&Half Burst) (SDRAM Burst Length4&& Full Burst)	(SDRAM Burst Length16&&Half Burst) (SDRAM Burst Length8&&Full Burst)	SDRAM Burst Length16&&Full Burst
Read	Native-Sized	N/A	Single	Single	Single
	Up-Sized	Single(onlyif araddris alignedtotheHIF burst)	Single	Single	Single
	Down-Sized	N/A	N/A	Single	Single
Write	Native-Sized	N/A	Multiple	Multiple	Single
	Up-Sized	Multiple	Multiple	Multiple	Single(onlyif awaddrisalignedtot heHIF burst)
	Down-Sized	N/A	N/A	Multiple	Single

In the tables, full burst is when:

DDR_bytes = AXI_bytes, half burst is when DDR_bytes = 2*AXI_bytes and quarter burst is when DDR_bytes = 4*AXI_bytes.

Where, DDR_bytes refers to the number of bytes transferred in a DDR burst:

$MSTR.burst_rdwr * 2 * MEMC_DRAM_DATA_WIDTH / 8$

AXI_bytes refers to the number of bytes transferred in an AXI burst: $2(ALEN + 1) * (DDRC_PORT_DW_n / 8)$

As an example, in a natural-size port, with MEMC_FREQ_RATIO=1 and MEMC_BURST_LENGTH=16, when the SDRAM burst length is programmed to 8 (Burst length of 16), ALEN 7 results in a full burst.

9.3.2.1.4 Read Data and Response Channel

The XPI handles the common response interface to the DDRC to process the read data from the memory.

AXI read data and response channel has a single data storage queue (RDQ - Read Data Queue) with two clocks—AXI clock for reading and DDRC clock for writing.

Data from different IDs are stored in the same queue and are returned in the order of read address acceptance. You can configure the depth of the queue independently for each port.

The FSM handling the AXI handshake as well as the FIFO push and pop has the following features:

- Based on the information stored, filter the dummy beats
- Based on the signal hif_rdata_last, generate the rlast

The controller provides an OKAY response for the each read, except for exclusive read transactions. The DDRC provides an EXOKAY response.

SLVERR respinse may be returned for the read transactions (both normal and exclusive) for the following cases:

- ECC uncorrected error detected at the DFI
- On-chip parity address or data error
- InvalidLPDDR4 row address
- Transaction has been poisoned

NOTE

SLVERR has the highest priority.

9.3.2.1.4.1 Read Reorder Buffer

The read data can be returned from the DDRC in a different order from which the read commands are forwarded from the XPI. (due to the re-ordering of read commands in the DDRC to maximize SDRAM bandwidth).A read reorder buffer is implemented in each port to reorder the read data for that port to the same order as the order of the AXI read commands.

The read reorder buffer SRAM holds the same number of entries as the read CAM and each entry holds the read data corresponding to a DDR command. Storage for the reorder buffer can be implemented internally in the DDRC or implemented externally to the DDRC as embedded SRAM and accessed through an External RAM Interface. In either case, the control circuits for the read reorder SRAM are clocked by the DDRC clock (and not AXI clock). If implemented as embedded SRAM, a two-port SRAM is required.

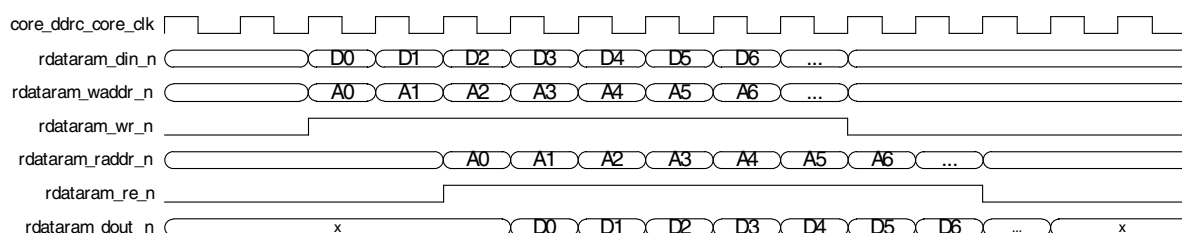


Figure 9-2. read Reorder Buffer Data RAM Interface Timing

The AXI protocol allows the read data for transactions of different IDs to be interleaved. To reduce potential delays where read data for one ID is blocked waiting for data associated with another ID, the read reorder buffer is organized as number of virtual channels (See the figure below). The Read Reorder Virtual Channel is a mechanism to allow independent read data reordering between multiple groups of AXI IDs.

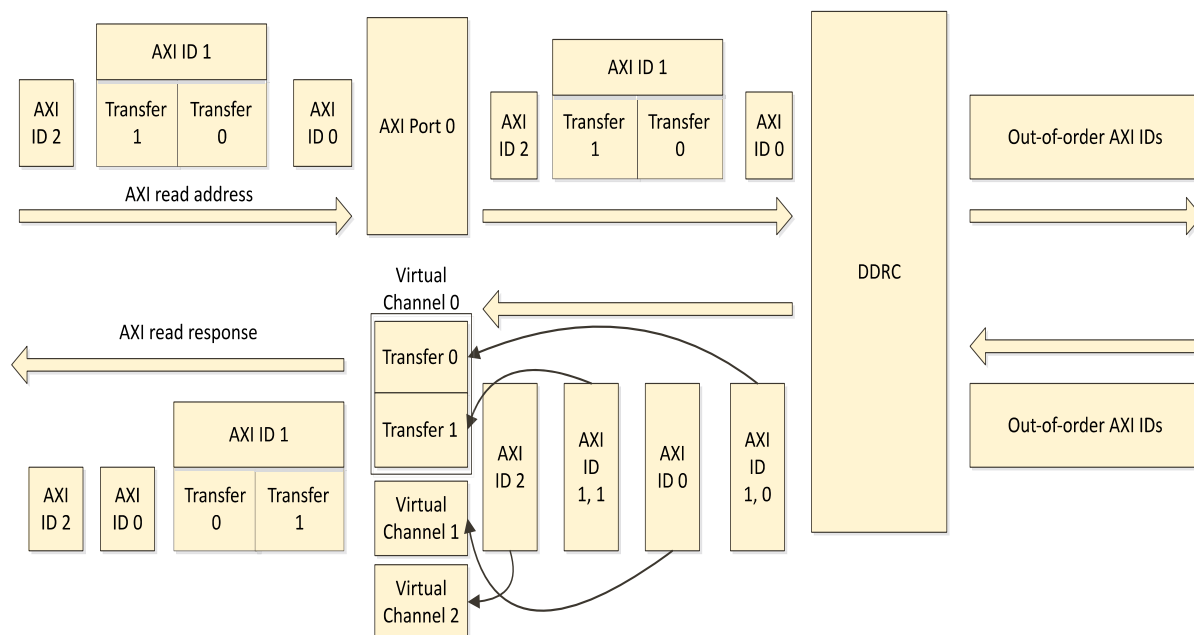


Figure 9-3. RRB Configured with Three Virtual Channels

9.3.2.1.5 Write data Channel

The AXI write data channel has a data storage queue (WDQ - Write Data Queue) with the following clocks:

- AXI clock (aclk_n) for writing and
- DDRC clock (core_ddrc_core_clk) for reading

This queue is used for the clock domain crossing between the AXI clock domain and the controller clock domain and acts as a registering layer in the case of a synchronous interface.

At the output of WDQ, some beats of a write data packet can be masked depending on alignment, burst size and burst length.

Write data from each port is forwarded to the DDRC, which forwards the data to a common data storage.

9.3.2.1.6 Data Width Conversion

The XPI performs the data width conversion between the data width at the AXI interface and internal HIF data width. The AXI data width can be set by the hardware parameter `DDRC_PORT_DW`.

Both data width down-conversion (where the AXI data bus is wider than the internal HIF data-bus width) and data width up-conversion (where the AXI data bus is narrower than the internal HIF data-bus width) are supported.

If no data width conversion is performed, port is referred as Native-sized:

$DDRC_PORT_DW_n = 2 * MEMC_DRAM_DATA_WIDTH$ if
 $MEMC_FREQ_RATIO = 1$

$DDRC_PORT_DW_n = 4 * MEMC_DRAM_DATA_WIDTH$ if
 $MEMC_FREQ_RATIO = 2$

When data width up conversion is performed, port is referred as Up-sized:

$DDRC_PORT_DW_n < 2 * MEMC_DRAM_DATA_WIDTH$ if
 $MEMC_FREQ_RATIO = 1$

$DDRC_PORT_DW_n < 4 * MEMC_DRAM_DATA_WIDTH$ if
 $MEMC_FREQ_RATIO = 2$

When data width down conversion is performed, port is referred as Down-sized:

$DDRC_PORT_DW_n > 2 * MEMC_DRAM_DATA_WIDTH$ if
 $MEMC_FREQ_RATIO = 1$

$DDRC_PORT_DW_n > 4 * MEMC_DRAM_DATA_WIDTH$ if
 $MEMC_FREQ_RATIO = 2$

The only exception is when Shared-AC with data channel interleaving is used, where Native-sized port is equivalent to a Down-sized port (1:2 ratio) in Single Channel configuration.

The width of the queues, Write Data Queue (WDQ) and Read Data Queue (RDQ) are always set to the wider data width. In the case of data width down-conversion, these queues are set to the width of the AXI data bus. In the case of data width up-conversion, these queues are set to the width of the internal HIF data-bus width

9.3.2.1.7 Write Response Channel

The write response is generated once the last beat of write data, for a given AXI burst, is accepted by the DDRC. The write response queue can be instantiated with configurable depth which is set by the hardware parameter `DDRC_AXI_WRQD_n`.

The write response generation makes use of the result of the exclusive access monitor. For a write transaction, the response is always returned as OKAY. For an exclusive write transaction, the response can be returned as OKAY or EXOKAY.

SLVERR response may be returned in the following cases:

- Invalid LPDDR4 row address
- On-chip parity address or data error
- Transaction has been poisoned

NOTE

SLVERR has the highest priority

9.3.2.1.8 Exclusive Access

The DDRC supports the AXI Exclusive Access feature. This feature is enabled using the hardware parameter DDRC_EXCL_ACCESS. This parameter defines the number of addresses the DDRC can monitor.

All exclusive read transactions have an EXOKAY response (except in the case of an ECC uncorrectable error). Successful exclusive write accesses have an EXOKAY response, unsuccessful exclusive write accesses return an OKAY response. If an exclusive write fails, the data mask for the exclusive write is forced low so the data is not written.

One address range per AXI ID is monitored for exclusivity. Therefore, if a master does not complete the write portion of an exclusive operation, a subsequent exclusive read to the same ID changes the address that is being monitored for exclusivity.

Once an exclusive access monitor for a given address is enabled, all write transactions are monitored for violation, regardless of the originating port. In other words, the violation check operates across the ports.

The exclusive access monitor compares the exclusive write transaction address, size, length, ID and port number against the exclusive read transaction address, size, length, ID and port number, and only accepts an exclusive write when these parameters match. Otherwise, the exclusive write is considered as fail.

If the EXCL_ACCESS parameter is configured to support no (0) exclusive accesses, the DDRC behaves like an AXI slave that does not support exclusive accesses. Therefore, all exclusive accesses return OKAY responses.

In some cases SLVERR response may be returned for exclusive access transactions. For more information about this, see **“Read Data and Response Channel”** and **“Write Response Channel”**.

NOTE

- The read response SLVERR is generated for an exclusive read transaction for ECC configurations, when ECC is enabled and an uncorrectable error is detected. The master, upon receiving a SLVERR to an exclusive read command should not complete the exclusive operation by sending an exclusive write command. If it does, the DDRC monitors return EXOKAY if there is no violation.
- The maximum number of bytes that are monitored as a region per address cannot exceed 128 bytes. The AXI exclusive transaction length must always be a power of two.
- When all the monitors are active, further exclusive read should not be issued. If this happens, one of the active monitors is evicted using a round robin control and new exclusive read is accepted. The selection of the monitor to be evicted is based on a pointer which rotates across all the locations when a new exclusive read is received.

9.3.2.1.8.1 Dual Channel considerations

In dual channel configurations, Shared AC and LPDDR4 dual channel, a second set of monitors is instantiated for the second channel. This means that the total number of exclusive monitors is doubled.

This has the following implications:

- Maximum number of monitored locations is equal to $DDRC_EXCL_ACCESS * 2$ ($DDRC_EXCL_ACCESS$ locations per data channel).
- If a master issues two Exclusive Read accesses to the same ARID to different channels (for example, channel 0 first, followed by channel 1, without issuing Exclusive Write access in between), channel 0 monitor for this ID is not evicted immediately and channel 1 monitor starts monitoring for the same ID.

According to AXI protocol, the maximum transfer size for exclusive accesses is 128 bytes. For correct functionality, an exclusive access transfer must not be interleaved across two data channels, that is, one EXA transfer goes to one channel only.

The following are the possible ways to achieve this requirement:

- The channel select bit of the address map ($ADDRMAP0.addrmap_dch_bit0$) can be programmed at or above the 128 byte boundary. For

MEMC_DRAM_DATA_WIDTH=16, this boundary corresponds to HIF[6] or addrmap_dch_bit0=4. For MEMC_DRAM_DATA_WIDTH=32, this boundary corresponds to HIF[5] or addrmap_dch_bit0=3.

- If the required channel interleaving boundary is less than 128 bytes, then the size of all EXA transfers must be constrained to be less or equal to the interleaving boundary.

9.3.2.1.9 Software Coherency for AXI Ports

The section “Address Collision Handling” describes how the DDRC handles in-order execution of commands to the same address.

For the commands issued at the AXI port, the logic in the DDRC protects against all types of software coherency hazards when the AXI master waits for write (read) response before sending the next same address read (same address write) or vice versa.

Some special AXI masters expect an ordering relationship between the reads and writes when the opposite direction transactions are sent without waiting for the previous response. For these special cases, additional logic should be instantiated in the XPI to enforce the order of acceptance within the reads and writes between the AXI and HIF interfaces at the DDRC. This logic is included by setting of the hardware parameter DDRC_RDWR_ORDERED_n and enabled by the register PCFGR_n.rdwr_ordered_en. When this feature is enabled and read and write transactions are presented at the same cycle, the pre-arbiter gives precedence to read transactions, incrementing the write transaction order token by 1 with respect to the read transaction order token. For example, if the last order token issued was equal to 0, then the read order token will be equal to 1 and the write order token will be equal to 2.

9.3.2.1.10 Transaction Poisoning

Sideband signals arpoison/awpoison, render an AXI transaction (read or write) invalid and the DDRC signals that AXI transaction poisoning has occurred. The input signal must be asserted coinciding with the associated AXI transaction. If a write is poisoned, all of its strobes are de-asserted, making the write effectively transparent to the memory. If a read is poisoned, the command is issued to the DDR memory and all the read data beats are overwritten and returned as zeros.

The DDRC may be programmed to signal that an AXI transaction poisoning has occurred by setting an interrupt or by driving the AXI SLVERR response for that transaction response. The AXI Poison Configuration Register POISONCFG (see DDRC_REGS Registers) is used to define whether an interrupt and/or AXI SLVERR is generated when an AXI transaction poisoning has occurred. There is separate control for read and write

transactions. The port specific AXI Poison status register is asserted whenever an AXI transaction is poisoned on that port. There is a separate status for read and write transactions. The interrupt and status register is cleared by the register `POISONCFG.rd_poison_intr_clr/POISONCFG.wr_poison_intr_clr`. Interrupt status is stored in the AXI Poison Status register `POISONSTAT` (see `DDRC_REGS` Registers on page 464). This register is a mirror in the APB clock domain of the actual interrupts, so status is updated with 2 or 3 pclk cycles of delay depending on the CDC logic.

An external block is expected to drive these new sideband signals depending on the access security requirements on transaction by transaction basis. Therefore, this feature replaces the need to implement ARM Trustzone.

9.3.2.1.11 AXI Data Channel Interleaving

A given AXI port (XPI) can access both data channels and low granularity interleaving between the two data channels is possible by mapping a suitable LSB address bit which is at the memory burst boundary. In that case, for example, a long AXI burst can be expanded to HIF commands so that back to back accesses are to alternating data channels. This setting achieves the highest efficiency possible from a single AXI port.

On the read data path, data order - based on AXI ordering rules - has to be preserved within each channel and between both channels. The read reorder buffer reorders the data coming from a given channel. The data ordered from both channels is then reordered by the data channel reorder logic. Thus two separate data SRAMs are instantiated, one for each channel, whose outputs are combined to provide a single wide AXI read data channel.

By setting the correct system mapping, one port can be set to access one data channel and another port can be set to access the other data channel. This configuration saves logic area in XPI block.

9.3.2.2 AHB Port Interface

The AHB slave port interface consists of an A2X block and XPI block as described in “AXI Port Interface (XPI)”. The A2X block converts the AHB slave port transactions into AXI equivalent transactions before forwarding them to the XPI, where the transactions are finally broken down into HIF packets and sent to the Port Arbiter (PA).

9.3.2.2.1 A2X

The A2X controls the following features of the AHB Slave port interface.

9.3.2.2.1.1 Split Mode

The split mode of the AHB slave port can be set by the parameter `DDRC_AHB_SPLIT_MODE_n`. Split response is used to control the AHB bus for efficiency and coherency on a multi-master split capable port. This is especially for AHB read transactions where the master is split once the NSEQ address is captured. When the DDRC fetches the read data for split transaction, the AHB interface slave port is free to receive a transaction for another master. The maximum number of splits (simultaneous) is equal to the number of AHB masters configured for the port. Once data is retrieved for a particular master, the master that is split is recalled.

For AHB write, the transaction may be split depending on the write response mode. For more information about this, see **“Write Response Mode”**.

In non-split mode, a single master controls the AHB interface slave port until its transaction completes, unless it is early burst terminated before completion. In the case of a read, the port output `hready_resp` is deasserted after the NSEQ phase of the transaction and asserted again during the data phase of each beat once the read data is available. When operated in this mode, every read transaction holds the bus for the amount of data beats requested plus at least the read idle latency time of the DDRC. For this reason, nonsplit AHB configuration has a negative impact on the performance of the port when compared with the split configurations.

9.3.2.2.1.2 Write Response mode

There are three AHB write response modes namely non-bufferable, bufferable, or dynamic write response modes. These modes can be programmed (per port) by the parameter `DDRC_AHB_WRITE_RESP_MODE_n`.

In non-bufferable mode, the `hready_resp` for the data phase of the last write beat for a defined length AHB burst is not asserted until the A2X logic receives confirmation from XPI (through `bresp_n`) that the last data beat is accepted by the DDRC. When the port operates in non-split mode, `hready_resp` remains low until this event occurs. When the port operates in split mode, the master whose write transaction is on the bus is split on either the last write data beat or if `hready_resp` has been driven low for `DDRC_AHB_HREADY_LOW_PERIOD_n` cycles (whichever occurs latter), and then recalled when that beat is accepted by the DDRC. For an undefined length INCRs, `hready_resp` for the data phase of each beat is not asserted until the A2X logic receives

confirmation that each beat is accepted by the DDRC. This indicates performance degradation for long undefined INCR writes when compared with the bufferable equivalent.

In bufferable mode, the logic does not wait for this last beat of write data to be accepted by the DDRC. Instead, hready_resp on the AHB port interface is asserted for the data phase of this beat once the AHB port of the DDRC can accept it. If operating in split mode, a split is generated if the hready_resp is driven low for greater than DDRC_AHB_HREADY_LOW_PERIOD_n cycles.

In dynamic mode, the transactions can be either bufferable or non-bufferable depending on the value of hprot according to the AMBA AHB Specification for the individual AHB transactions, where hprot[2]=1'b0 is non-bufferable and hprot[2]=1'b1 is bufferable.

9.3.2.2.1.3 Error Response

The AHB port only returns an error response on hresp_n for a data beat that has an uncorrectable ECC error. For this to occur, the DDRC should be configured to enable ECC logic. For more information about this, see “Read-Modify-Write (RMW) Generation”.

9.3.2.2.1.4 Early Burst Termination (EBT)

A defined length write transaction that is early burst terminated by another transaction has its remaining write data beats masked during memory access. When the early burst terminated master returns, only the remaining write data beats that are masked after the initial EBT are sent to the memory. This EBT of defined length write transactions causes multiple memory accesses. The non-bufferable response for the single defined length transaction that is split into multiple memory access is given only after all memory accesses are completed.

Any defined length read transaction must return to retrieve all its data if early burst terminated. Any undefined length INCR read transaction has all remaining data beats that are not retrieved by a master, up to the read prefetch beats set by the pin hincr_arlen_n, flushed internally in the DDRC.

9.3.2.2.1.5 AHB Lite mode

The parameter `DDRC_AHB_LITE_MODE_n` is used to set an AHB port to operate in AHB Lite mode. This means that the port can be connected only to a single master and responds to AHB read and nonbufferable write transactions by driving `hready_resp` low (as opposed to split responses). This simplifies the AHB master/arbiter system connected to an AHB Lite port.

9.3.2.2.1.6 Endian conversion

The A2X provides endian mapping from the AHB port to the XPI with respect to the transaction size. This section outlines the key features of the A2X endian mapping.

The key features of endian conversion are as follows:

- Register: The functionality is enabled by `PCFGC_n.ahb_endianness` (see `DDRC_MP Registers`)
- Endian transformations supported:
 - Little Endian (LE)
 - Big Endian-32 (BE-32) invariant
 - Big Endian-A (BE-A) address invariant

9.3.2.2.2 Write Streaming

For AHB write streaming, the write data buffer parameter `DDRC_AHB_WDQD` should be set to a size that can support enough write data beats from the first valid AHB write beat. So, the transaction on the AHB slave interface port to time the first beat is accepted by the DDRC, signified by asserting the DDRCs `hif_wdata_valid` and `hif_wdata_stall` outputs.

9.3.2.2.3 Read Streaming

For non-split configurations, it is not possible to stream different transaction due to the nature of the AHB protocol, as we cannot get the address of the next AHB transaction to access memory until the read data for the current transaction has completed. We can stream within a transaction from the time the first read beat is available until the last read beat of the same transaction.

For split capable configurations, depending on the number of masters and arbitration scheme used accessing a port and the type of AHB transaction used, it is possible to stream reads after the initial read latency delay from the first AHB transaction.

9.3.2.2.4 XPI

The XPI used in conjunction with the A2X provides features to complete the overall AHB slave port solution. These features include data width conversion from the AHB port data width to the internal data width used by the PA and DDRC, which is twice the width of the SDRAM data interface in case of 1:1 clock frequency ratio and four times the width of the SDRAM data interface in case of 1:2 clock frequency ratio.

Each AHB port can be independently configured to operate with a clock that is synchronous or asynchronous to the memory controller clock. This clock crossing logic is done within the XPI.

ECC is supported for configurations with AHB port interfaces.

9.3.2.2.5 Software Coherency For AHB Ports

Coherency of read after write (RAW) and write after read (WAR) to the same address is always guaranteed. For more information about this, see “Software Coherency for AXI Ports” and “Address Collision Handling”.

9.3.2.3 Host Interface (HIF)

HIF is an internal interface when the DDRC is configured to include a multi port arbiter (the hardware parameter DDRC_INCL_ARB is set to 1). In this case, the AXI Port Interface and/or Port Arbiter handle all signals on this interface, effectively acting as the SoC core.

Read, write, as well as write data are received through the HIF. Following the receipt and acceptance of a write the DDRC requests write data from the interface. The interface subsequently provides the independently timed write data in the requested order (which matches the order in which requests are made to the DDRC). See “Dual HIF Functionality” for further details on Dual HIF functionality. Both read and write requests can be reordered later, before being output on the DFI interface.

Requests to the DDRC are throttled in two ways:

- A credit mechanism that ensures that buffer space is available for any request the SoC core makes to the DDRC, prior to the request being made.
- An independent stall mechanism that throttles requests when address collisions take place or if entering self-refresh through the software self-refresh or hardware low power self-refresh.

Data is handled separately. The DDRC throttles data by potentially waiting after it receives a write before requesting the associated data. If there is collision, then the throttling lasts until the collision is cleared in the DDRC. Once write data is requested, it is always accepted by the DDRC.

Read data is sent to the host via a common response interface. For more information about this, see “Common Response”.

This section describes the following about the HIF:

- “Dual HIF Functionality”
- “Credit Mechanism”
- “Read Requests”
- “Write Requests and Data”
- “Read-Modify-Write Requests”
- “Common Response”

9.3.2.3.1 Dual HIF functionality

Dual HIF functionality is as follows:

- Converts HIF single command channel into separate HIF command channel for Read and Write commands.
- Read/Write Arbitration is not performed by the PA
- Dual HIF functionality is supported only if the DDRC is configured to support a multi port arbiter (DDRC_INCL_ARB=1).
- Dual HIF functionality is supported only if the DDRC is configured to include the logic to optimize timing over scheduling efficiency (MEMC_OPT_TIMING=1).
- Dual HIF functionality continues to guarantee coherency for write-after-read (WAR) and read-afterwrite (RAW) hazards. Also, if a Read and Write/ to the same address are taken at the same time (both hif_rcmd_stall=0 and hif_wcmd_stall=0), the Write/ is performed first.

The table provides the details of the signals modified by the DDRC_DUAL_HIF parameter.

Table 9-2. Comparison of Signals Modified by DDRC_DUAL_HIF Parameter

DDRC_DUAL_HIF=0 Single HIFCommand Channel	DDRC_DUAL_HIF=1 ¹ Separate HIF Command Channels One for Reads (*_rd) One for Writes/ (*_wr)
hif_cmd_valid	hif_rcmd_valid hif_wcmd_valid
hif_cmd_type	hif_rcmd_type hif_wcmd_type

Table continues on the next page...

Table 9-2. Comparison of Signals Modified by DDRC_DUAL_HIF Parameter (continued)

hif_cmd_addr	hif_rcmd_addr hif_wcmd_addr
hif_cmd_pri	hif_rcmd_pri hif_wcmd_pri
hif_cmd_token	hif_rcmd_token hif_wcmd_token
hif_cmd_token	hif_rcmd_length hif_wcmd_length
hif_cmd_wdata_ptr	hif_rcmd_wdata_ptr hif_wcmd_wdata_ptr
hif_cmd_autopre	hif_rcmd_autopre hif_wcmd_autopre
hif_cmd_stall	hif_rcmd_stall hif_wcmd_stall

NOTE

1. As DDRC_DUAL_HIF=1 is only possible if DDRC_INCL_ARB=1, the right hand column refers to internal signals within the DDRC. It is provided for debug purposes only.

NOTE

The databook uses DDRC_DUAL_HIF=0 notation.

If you are using DDRC_DUAL_HIF=1, the internal signals should be interpreted in the following way:

- hif_cmd_valid refers to hif_rcmd_valid and/or hif_wcmd_valid.
- hif_cmd_stall refers to hif_rcmd_stall and/or hif_wcmd_stall and so on.

9.3.2.3.2 Credit Mechanism

The DDRC employs a credit mechanism to ensure that buffers do not overflow. The interface making the request to the DDRC, can only request commands for which it has been granted “credits” to issue.

Credits are tracked separately for the following three command types:

- High Priority read(HPR)
- Low Priority read(LPR)
- Write

Credits are counted for each command type independently according to the following rules:

- Initially the interface has zero credits.

- The interface logic must include counters to track the number of credits granted by the DDRC and decremented due to commands issued by the interface logic to the DDRC. The interface logic must have separate credit counters for each individual command type.
- Following the de-assertion of reset to the DDRC, credits are issued to the interface for each command type. A given credit count increments every time the DDRC issues a credit, indicated by the assertion of the appropriate *_credit signal on the rising edge of core_ddrc_core_clk. The credit counting logic implemented externally to the DDRC must run on the same clock.
- When the credit count is greater than zero, the interface can issue requests of that type to the DDRC. Each time a request is issued to the DDRC, the associated credit count is decremented.
- When DDRC_VPRW_EN is set to 1, the arbiter or the host can send Variable Priority Read (VPR) and Variable Priority Write (VPW) commands.
- VPR commands do not have a pre-allocated storage resource in the Read CAM (unlike HPR commands). VPR commands share the same resource with the LPR commands. As far as the credit mechanism is concerned, the VPR commands are counted in the LPR credit bucket.
- Similarly, VPW commands do not have a pre-allocated storage resource in the Write CAM. VPW commands share the same resource with the Normal Write commands. As far as credit mechanism is concerned, the VPW commands are counted in the WR credit bucket.

The signals related to the credit request interface are:

- hif_cmd_valid
- hif_cmd_type
- hif_cmd_pri
- hif_lpr_credit
- hif_hpr_credit
- hif_wr_credit
- hif_cmd_stall

9.3.2.3.3 Clear Requests

When the LPR or HPR credit count is greater than zero, the interface can issue read requests to the DDRC accordingly.

A command is issued to the DDRC by asserting hif_cmd_valid on the rising edge of the clock. All read request fields must be driven to the appropriate values at the same time. These fields are:

- **hif_cmd_type**: specifies the request type as follows:

- '00' indicates a write request
- '01' indicates a read request
- '11' is not supported
- **hif_cmd_pri:**
 - In HIF-only configurations (DDRC_INCL_ARB=0) where DDRC_VPRW_EN is set to 0, this signal is 1-bit wide. A '1' indicates the read request is high priority. '0' indicates the read request is low priority. This field is meaningless for write commands.
 - In Arbiter configurations (DDRC_INCL_ARB=1) or in HIF-only configurations (DDRC_INCL_ARB=0) where DDRC_VPRW_EN is set to 1, this signal is 2-bit wide.

The encoding for Read is:

- 2'b00 - LPR
- 2'b01 - VPR
- 2'b10 - HPR
- 2'b11 - Reserved

VPR commands are sent only in configurations with DDRC_VPRW_EN=1

- **hif_cmd_addr:** indicates the address for the read. Each word on the HIF (equivalent to 2 DDR words for 1:1 mode configurations (MEMC_FREQ_RATIO=1), or 4 DDR words for 1:2 mode configurations (MEMC_FREQ_RATIO=2)) is uniquely addressable.
- **hif_cmd_length,** is 2 bits if MEMC_BURST_LENGTH=16, 1 bit otherwise.

Note the following:

- For MEMC_BURST_LENGTH = 16, a normal read is 16 SDRAM words

If MEMC_BURST_LENGTH=16:

- 2'b11 -> Partial (Quarter) Read
- 2'b10 -> Partial (Half) Read
- 2'b00 -> Full Read

Otherwise:

- 1'b1 -> Partial (Half) Read
- 1'b0 -> Full Read

In each case, a partial (half) read is half the length of a normal full read (where a word is the amount of data transferred on one edge of the DQS to the SDRAM in a fully configured system.) If MEMC_BURST_LENGTH=16, Partial (Quarter) Read is possible and is quarter the length of a normal full read. This field is meaningless for write or commands.

- **hif_cmd_token:** a bit field that is presented with the read command and is returned with the read response. As the responses may be presented out-of-order, the token is the identifier that indicates the read for which data is being returned on the response side. Therefore, multiple reads with the same token must never be pending in the DDRC simultaneously, as the responses would be indecipherable. Common uses for this field include identifying the requestor and serializing the response data. This field is meaningless for write or commands.

9.3.2.3.3.1 Valid And Stall

the requests by asserting hif_cmd_stall. For any cycle in which hif_cmd_stall is asserted on the rising edge of the clock, the hif_cmd_valid and all other associated request signals are ignored by the DDRC. When this happens, the interface must hold the request on the interface for another cycle and must not yet decrement the associated credit counter. When the DDRC de-asserts hif_cmd_stall on the rising edge of the clock, the request is accepted and the interface can then de-assert hif_cmd_valid or it can issue the next read, write or request (credits allowing).

The DDRC asserts hif_cmd_stall for any of the following conditions:

1. Software driven self-refresh entry requests. This is to ensure that DDRC is empty when memories are in self-refresh. For more information about this, see “Power Saving Features”.
2. Successful DDRC hardware low power entry requests to self-refresh. This is to ensure that DDRC is empty when memories are in self-refresh. For more information about this, see “Power Saving Features”.
3. Address collisions in the DDRC, where flow control is applied to prevent further commands from entering the DDRC. For more information about this, see “Address Collision Handling”.
4. In the event of the Write CAM being full and no corresponding data for any of the write command has yet arrived.
5. Setting DBG1.dis_hif=1, causes hif_cmd_stall to be asserted.

NOTE

If Dual HIF functionality is enabled (DDRC_DUAL_HIF=1), hif_rcmd_stall/hif_wcmd_stall is asserted as follows:

- Conditions (1), (2), (3), and (5) cause both `hif_rcmd_stall` and `hif_wcmd_stall` to assert at the same time.
- Condition (4) only affects `hif_wcmd_stall`.

9.3.2.3.4 Write Requests And Data

When the WR credit count is greater than zero, the interface can issue write requests to the DDRC accordingly. A command is issued to the DDRC by asserting `hif_cmd_valid` on the rising edge of the clock.

All write request fields must be driven to the appropriate values at the same time. These fields are:

- **`hif_cmd_type`:** specifies the request type as follows:
 - '00' indicates a write request
 - '01' indicates a read request
 - '11' is not supported
- **`hif_cmd_pri`:** specifies the priority of the write request
 - In HIF-only configurations (`DDRC_INCL_ARB=0`) where `DDRC_VPRW_EN` is set to 0, this signal is 1-bit wide. This field is meaningless for write commands.
 - In Arbiter configurations (`DDRC_INCL_ARB=1`) or in HIF-only configurations (`DDRC_INCL_ARB=0`) where `DDRC_VPRW_EN` is set to 1, this signal is 2-bit wide.

The encoding for Write is:

- 2'b00 - NPW (Normal Priority Writes)
- 2'b01 - VPW (Variable Priority Writes)
- 2'b10 - Reserved
- 2'b11 - Reserved

VPW commands are sent only in configurations where `DDRC_VPRW_EN=1`

- **`hif_cmd_addr`:** indicates the address for the write or . Each word on the HIF (`hif_wdata/hif_rdata_data`) is uniquely addressable. For 1:1 frequency ratio mode configurations each 2 DDR words on the DQ bus are uniquely addressable via `hif_cmd_addr`. In 1:2 frequency ratio mode configurations every 4 DDR words are uniquely addressable.
- **`hif_cmd_wdata_ptr`:** a pointer into the requestor's own buffers that can be used to subsequently retrieve the write data. Once a write or request is accepted, this pointer is returned to the interface to retrieve the associated write data. (This field is not required, as requests are always accepted in order; the interface can choose to FIFO these pointers internally and ignore this field.)

The stall mechanism for write or requests is identical to that for read requests as explained in the section “Valid and Stall”.

Writes have three phases of operation: a write command, a write data pointer return, and finally write data provided to the DDRC. The write command phase indicates to the DDRC that a write transfer is required. The write data pointer return phase indicates to the SoC core that the DDRC is ready to receive the write data. The write data phase provides the write data to the DDRC.

In all these three phases, the transactions must follow the same order. The data pointer is returned in the same order in which the command is sent. The write data must also be sent in the same order. The DDRC can perform re-ordering to the commands later, before sending them on the DFI interface, to make efficient use of the SDRAM bandwidth.

9.3.2.3.4.1 Write Data Pointer Return

Following the acceptance of a write or request, the DDRC then returns the write data pointer to the interface logic to retrieve the associated write data. This is done by asserting `hif_wdata_ptr_valid` on the rising edge of clock. In the same cycle, `hif_wdata_ptr` indicates the value of the write data pointer. This is same as a pointer that is presented to the DDRC as `hif_cmd_wdata_ptr` during the write command phase.

There is no mechanism for stalling the acceptance of the write data pointer from the DDRC; the interface logic must present write data for every cycle in which `hif_wdata_ptr_valid` is asserted by the DDRC. The mechanism to throttle `hif_wdata_ptr_valid` assertions is for the interface to throttle write or requests presented to the DDRC.

There is no required timing between a write or command presented to the DDRC, and the same pointer is returned as `hif_wdata_ptr`. Multiple pointers can also be presented to the DDRC with multiple requests before the first is returned to the interface. The order of the pointers returned matches the order of the requests presented to the DDRC.

9.3.2.3.4.2 Write Data

After a write data pointer is returned to the interface logic, the interface logic must retrieve the associated data and present it to the DDRC. Data presented to the DDRC has no required timing relationship to the transfer of the write pointer on the interface. The data, however, must be returned in the same order that the pointers are presented to the interface (same order in which the original requests are made).

The write data can be throttled using `hif_wdata_stall` signal. The write data is accepted by the DDRC when `hif_wdata_valid` is detected high and `hif_wdata_stall` is detected low at the same positive edge of the clock. The signal `hif_wdata_stall` is asserted by the DDRC during a RMW operation. When the DDRC writes the read data for the RMW operation into the write data SRAM, it blocks the SoC core interface from sending any write data during those cycles by asserting this signal.

The write data SRAM can be implemented internally as synthesized flip-flops or implemented externally to the DDRC as embedded SRAM using `DDRC_WDATA_EXTRAM` parameter.

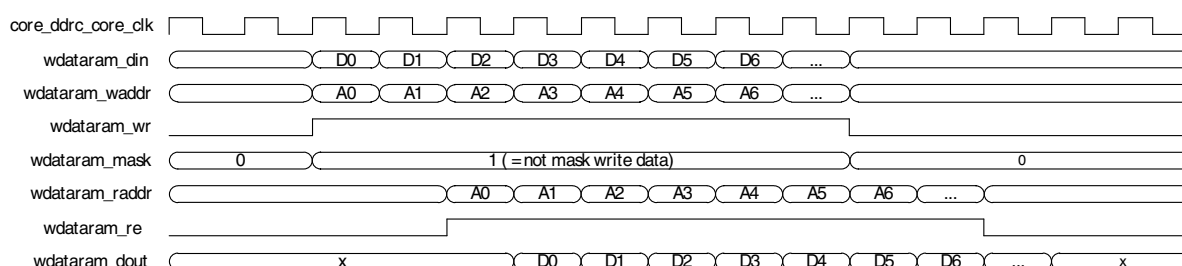


Figure 9-4. Write Data RAM Interface Timing

9.3.2.3.4.2.1 Write Data for the DDRC in BL16 Mode (`MEMC_BURST_LENGTH = 16`)

Write data is presented to the DDRC with the assertion of `hif_wdata_valid`. Each request can have one to eight data phases. `hif_wdata_valid` must be asserted for each data phase and `hif_wdata_end` must be asserted on the last data phase. `hif_wdata` is the data. Both `hif_wdata_end` and `hif_wdata` are valid only when `hif_wdata_valid` is '1'.

A normal write or RMW command has eight clocks of data associated with it. In this case, `hif_wdata_end` must be asserted on the eighth data phase. If the SoC core has less than 8 cycles of write data for a write command, it can choose to send 1, 2, 3, 4, 5, 6 or 7 cycles of data, and `hif_wdata_end` must be asserted in the appropriate last data phase - these are referred to as Partial Writes. In this case, the DDRC has all the data that it requires 7, 6, 5, 4, 3, 2 or 1 clock cycles earlier than in case of a full write. For these partial writes, the DDRC's behavior is dependent on whether `DDRC_PARTIAL_WR` is enabled or not:

- **DDRC_PARTIAL_WR=0:** DDRC still issues the number of SDRAM bursts on the DDR interface that it would issue for a normal (full) write, but masks the unused data phases.
- **DDRC_PARTIAL_WR=1:** DDRC issues the minimum number of SDRAM bursts on the DDRC interface, depending on the number of HIF write data beats and the HIF address alignment with respect to the SDRAM column address, because SDRAM writes must be sent aligned with BL. This may be smaller than that for a full write.

This analysis assumes 1:1 frequency ratio mode. For 1:2 frequency ratio mode, the number of data beats of the write data on the HIF is halved, so each request has one to four data phases. A Partial Write will have less than four data phases.

9.3.2.3.5 Common Response

The Common Response Interface returns data from the DDRC to the SoC core in response to read requests. The following signals are associated with this functionality:

- hif_rdata_valid
- hif_rdata_end
- hif_rdata_token
- hif_rdata_data

9.3.2.3.5.1 Response Data Ordering

Read data is returned on the HIF in the same order in which it is received from the PHY on the DFI interface. It is independent of whether the read address is aligned or unaligned. For information on how unaligned addresses are handled, see “Sequential/Interleaved Operations”. For data lane mapping examples where the addresses are aligned, see “Data Lane Mapping Examples”.

9.3.2.3.5.2 Common Response Interface Example

The figure below shows a pair of read requests. The first request is to address A0 with token T0 and length 0. The second request is to A1 with token T1 and length 1. The second read data is returned before the first read data. The DDRC can determine that this sequence is possible due to the state of the open pages and other actions going on in the memories. This improves overall memory bandwidth utilization. The response token indicates which read's data is being provided. Once started, read data for a given token always completes for that token before the next token's read data is returned. The

hif_rdata_valid signal indicates valid data is on the bus and the hif_rdata_end signal indicates that the last data packet is being transmitted. The hif_rdata_valid can deassert in the middle of returning read data for one or more cycles, so hif_rdata_valid must be monitored to validate each cycle of returning read data.

NOTE

Block A: If hif_cmd_length is 1'b1 for this request, 1 cycle of read data comes back and the END signal is asserted in the same cycle.

Block B: If hif_cmd_length is 1'b0 for this request, 2 cycles of read data comes back and the END signal is asserted in the second cycle.

For MEMC_BURST_LENGTH=16, DFI 1:1 configurations, the following is applicable for Block A and Block B in the above figure:

Block A: If hif_cmd_length is 2'b10 for this request, 4 cycle of read data comes back and the END signal is asserted in the fourth cycle.

Block B: If hif_cmd_length is 2'b00 for this request, 8 cycles of read data comes back and the END signal is asserted in the 8th cycle.

9.3.2.3.5.3 LPDDR4 6Gb/12Gb and hif_rdata_addr_err

For LPDDR4 configurations, there is an extra output for DDRC, named hif_rdata_addr_err. This is asserted only in cases where LPDDR4 6Gb/12Gb (ADDRMAP6.lpddr4_6gb_12gb_24gb == 2'b01 or 2'b10) device density is used and the requested address for a read is not valid (when both row address bits MSB and MSB-1 are high). In these cases, DDRC performs a dummy read by changing the physical address to a valid one (row[MSB:MSB-1] changes from 2'b11 to 2'b10). The returned data is masked to 0 and hif_rdata_addr_err is asserted along with the relevant hif_rdata_valid/hif_rdata_token.

9.3.2.4 DFI Interface

The DDRC contains a DFI MC (Memory Controller) interface, which is used to connect to a DFI-compliant PHY, and to transfer address, control and data to the PHY. The DDRC interfaces to a PUB, PUBL, or PUBM2 block which acts as the DFI PHY interface. You can get more information about the DFI interface from the following specifications:

- DDR PHY Interface (DFI) Specification, Revision 3.1 (Preliminary), 21 March 2014 and
- DDR PHY Interface (DFI) Specification, Preliminary DFI 4.0 Specification - Addendum to DFI 3.1, 2 April 2014

The DFI interface is sub-divided into the following interface groups:

- Control Interface
- Write Data Interface
- Read Data Interface
- Update Interface
- Status Interface
- Training Interface (PHY-independent mode supported by the DDRC)
- Low Power Control Interface

9.3.2.4.1 1:1 Frequency Ratio Mode Considerations

The DDRC, including its DFI interface, runs at the single data rate (SDR) clock (referred to as 1:1 frequency ratio mode in the DFI Specification), so all DFI command and address signals are equal in width to the equivalent DDR SDRAM signals. DFI data signals are twice the width of the equivalent DDR SDRAM signals. The DDRC runs with an SDR clock and the PHY operates the SDR to DDR rate conversion on the data buses. In 1:1 frequency ratio mode, `core_ddrc_core_clk` and `CK/CK#` run at the same SDR clock frequency.

9.3.2.4.2 1:2 Frequency Ratio Mode Considerations

The DDRC, including its DFI interface runs at the half data rate (HDR) clock (referred to as 1:2 frequency ratio mode in the DFI Specification), so all DFI command and address signals are twice the width of the equivalent DDR SDRAM signals. DFI data signals are four times the width of the equivalent DDR SDRAM signals. The DDRC operates with an HDR clock. The PHY handles HDR-to-SDR conversion on the address bus to memory and HDR/DDR conversion on the data buses. In 1:2 frequency ratio mode, `core_ddrc_core_clk` runs at half the frequency of `CK/CK#`.

For DFI command and address signals in DFI 1:2, the DFI protocol refers to phase 0 (`_p0`) and phase 1 (`_p1`) but there are no such `*_p0` and `*_p1` signals in the DDRC. Instead, the lower half of the bits corresponds to phase 0 and the upper half of the bits corresponds to phase 1. For example, in a 4-rank system in DFI 1:2, `dfi_cs[7:0]` is the case. `dfi_cs[3:0]` corresponds to `dfi_cs_p0` and `dfi_cs[7:4]` corresponds to `dfi_cs_p1`.

Similarly, for DFI write data signals in DFI 1:2, the DFI protocol refers to phase 0 (`_p0`) and phase 1 (`_p1`). But, there are no such `*_p0` and `*_p1` signals in the DDRC. Instead, the lower half of the bits correspond to phase 0 and the upper half of the bits correspond to phase 1.

Consider the following examples:

- In system that has 16 bits of SDRAM in DFI 1:2, `dfi_wrddata[63:0]` is the case. `dfi_wrddata[31:0]` corresponds to `dfi_wrddata_p0` and `dfi_wrddata[63:32]` corresponds to `dfi_wrddata_p1`. Similarly, `dfi_wrddata_en[3:0]` is the case. `dfi_wrddata_en[1:0]` corresponds to `dfi_wrddata_en_p0` and `dfi_wrddata_en[3:2]` corresponds to `dfi_wrddata_en_p1`. Similarly, `dfi_wrddata_mask[7:0]` is the case. `dfi_wrddata_mask[3:0]` corresponds to `dfi_wrddata_mask_p0` and `dfi_wrddata_mask[7:4]` corresponds to `dfi_wrddata_mask_p1`.
- In system that has 32 bits of SDRAM in DFI 1:2, `dfi_wrddata[127:0]` is the case. `dfi_wrddata[63:0]` corresponds to `dfi_wrddata_p0` and `dfi_wrddata[127:64]` corresponds to `dfi_wrddata_p1`. Similarly, `dfi_wrddata_en[7:0]` is the case. `dfi_wrddata_en[3:0]` corresponds to `dfi_wrddata_en_p0` and `dfi_wrddata_en[7:4]` corresponds to `dfi_wrddata_en_p1`. Similarly, `dfi_wrddata_mask[15:0]` is the case. `dfi_wrddata_mask[7:0]` corresponds to `dfi_wrddata_mask_p0` and `dfi_wrddata_mask[15:8]` corresponds to `dfi_wrddata_mask_p1`.

Similarly, for `dfi_rddata_en` in DFI 1:2, the DFI protocol refers to phase 0 (`_p0`) and phase 1 (`_p1`). But, there are no such `*_p0` and `*_p1` signals in the DDRC. Instead, the lower half of the bits correspond to phase 0 and the upper half of the bits correspond to phase 1.

Consider the following examples:

- In system that has 16 bits of SDRAM in DFI 1:2, `dfi_rddata_en[3:0]` is the case. `dfi_rddata_en[1:0]` corresponds to `dfi_rddata_en_p0` and `dfi_rddata_en[3:2]` corresponds to `dfi_rddata_en_p1`.
- In system that has 32 bits of SDRAM in DFI 1:2, `dfi_rddata_en[7:0]` is the case. `dfi_rddata_en[3:0]` corresponds to `dfi_rddata_en_p0` and `dfi_rddata_en[7:4]` corresponds to `dfi_rddata_en_p1`.

For `dfi_rddata_valid`/`dfi_rddata`/`dfi_rddata_dbi` signals in DFI 1:2, the DFI protocol refers to word 0 (`_w0`) and word 1 (`_w1`). But, there are no such `*_w0` and `*_w1` signals in the DDRC. Instead, the lower half of the bits correspond to word 0 and the upper half of the bits correspond to word 1.

Consider the following examples:

- In system that has 16 bits of SDRAM in DFI 1:2, `dfi_rddata[63:0]` is the case. `dfi_rddata[31:0]` corresponds to `dfi_rddata_w0` and `dfi_rddata[63:32]` corresponds to `dfi_rddata_w1`. Similarly, `dfi_rddata_valid[3:0]` is the case. `dfi_rddata_valid[1:0]` corresponds to `dfi_rddata_valid_w0` and `dfi_rddata_valid[3:2]` corresponds to `dfi_rddata_valid_w1`. Similarly, `dfi_rddata_dbi[7:0]` is the case. `dfi_rddata_dbi[3:0]` corresponds to `dfi_rddata_dbi_w0` and `dfi_rddata_dbi[7:4]` corresponds to `dfi_rddata_dbi_w1`.
- In system that has 32 bits of SDRAM in DFI 1:2, `dfi_rddata[127:0]` is the case. `dfi_rddata[63:0]` corresponds to `dfi_rddata_w0` and `dfi_rddata[127:64]` corresponds to `dfi_rddata_w1`. Similarly, `dfi_rddata_valid[7:0]` is the case. `dfi_rddata_valid[3:0]` corresponds to `dfi_rddata_valid_w0` and `dfi_rddata_valid[7:4]` corresponds to `dfi_rddata_valid_w1`. Similarly, `dfi_rddata_dbi[15:0]` is the case. `dfi_rddata_dbi[7:0]` corresponds to `dfi_rddata_dbi_w0` and `dfi_rddata_dbi[15:8]` corresponds to `dfi_rddata_dbi_w1`.

9.3.2.4.3 DFI Write/Read Data to SDRAM Conversion

This conversion is performed in the PHY, and therefore PHY-Specific. The following details are applicable only to Synopsys DDR PHYs; other PHYs may differ from this.

The DFI data (both read and write) should contain data for multiple byte lanes and for multiple data beats (2 in 1:1 mode, 4 in 1:2 mode). The DFI Specification does not explicitly state how this data should be packed within `dfi_rddata` and `dfi_wrdata`. The Synopsys DDR PHYs and the DDRC DFI interfaces are designed with the assumption that the data is ordered by beat and then (within each beat) by byte.

For example, for a 2-byte configuration (16 bits of SDRAM) in 1:2 mode, the DFI data order is as follows (from MSB to LSB):

[byte1-beat3, byte0-beat3, byte1-beat2, byte0-beat2, byte1-beat1, byte0-beat1, byte1-beat0, byte0-beat0]

Therefore, the PHY sends the data it receives out to the memory in the following way:

- In beat 0, the data from `dfi_wrdata[15:0]` are sent on `DQ[15:0]`
- In beat 1, the data from `dfi_wrdata[31:16]` are sent on `DQ[15:0]`

- In beat 2, the data from dfi_wrdata[47:32] are sent on DQ[15:0]
- In beat 3, the data from dfi_wrdata[63:48] are sent on DQ[15:0]

Similarly, for a 2-byte configuration (16 bits of SDRAM) in 1:1 mode, the DFI data order is as follows (from MSB to LSB):

[byte1-beat1, byte0-beat1, byte1-beat0, byte0-beat0]

Therefore, the PHY sends the data it receives out to the memory in the following way:

- In beat 0, the data from dfi_wrdata[15:0] are sent on DQ[15:0]
- In beat 1, the data from dfi_wrdata[31:16] are sent on DQ[15:0]

Similarly, for a 4-byte configuration with 32 bits of SDRAM in 1:2 mode, the DFI data order is as follows (from MSB to LSB):

[byte3-beat3, byte2-beat3, byte1-beat3, byte0-beat3, byte3-beat2, byte2-beat2, byte1-beat2, byte0-beat2, byte3-beat1, byte2-beat1, byte1-beat1, byte0-beat1, byte3-beat0, byte2-beat0, byte1-beat0, byte0-beat0]

Therefore, the PHY sends the data it receives out to the memory in the following way:

- In beat 0, the data from dfi_wrdata[31:0] are sent on DQ[31:0]
- In beat 1, the data from dfi_wrdata[63:32] are sent on DQ[31:0]
- In beat 2, the data from dfi_wrdata[95:64] are sent on DQ[31:0]
- In beat 3, the data from dfi_wrdata[127:96] are sent on DQ[31:0]

Similarly, for a 4-byte configuration (32 bits of SDRAM) in 1:1 mode, the DFI data order is as follows (from MSB to LSB):

[byte3-beat1, byte2-beat1, byte1-beat1, byte0-beat1, byte3-beat0, byte2-beat0, byte1-beat0, byte0-beat0]

Therefore, the PHY sends the data it receives out to the memory in the following way:

- In beat 0, the data from dfi_wrdata[31:0] are sent on DQ[31:0]
- In beat 1, the data from dfi_wrdata[63:32] are sent on DQ[31:0]

For more information, see the addressing example in Figure A-1. Similar ordering is true for dfi_wrdata_en/dfi_wrdata_mask and dfi_rddata_en/dfi_rddata/dfi_rddata_valid/dfi_rddata_dbi.

NOTE

- If dfi_rddata data slice independence is enabled (DDRC_DFI_RDDATA_PER_BYTE=1), the logic assumes the DDR PHY ordering on bytes/beats for dfi_rddata_valid/dfi_rddata/dfi_rddata_dbi.

9.3.2.4.4 Control Interface

The control interface is a reflection of the SDRAM control interface including address, bank, chip select, row strobe, column strobe, write enable, clock enable and ODT control, as applicable for the memory technology. This interface consists of the following signals:

- dfi_address (width of this signal depends on configuration - 20 bits if MEMC_DDR4 = 1; otherwise 16 bits)
- dfi_bank
- dfi_cas_n
- dfi_cke
- dfi_cs
- dfi_odt
- dfi_ras_n
- dfi_reset_n (this signal only exists if MEMC_DDR4 = 1)
- dfi_we_n
- dfi_bg (this signal only exists if MEMC_DDR4=1)
- dfi_act_n (this signal only exists if MEMC_DDR4 = 1)

For more information about the above signals, refer to “Signal Descriptions”.

9.3.2.4.5 Write Data Interface

The write data interface handles the transmission of write data across the DFI interface. The DFI Specification defines signals and timing relationships. This interface consists of the following signals:

- dfi_wrdata
- dfi_wrdata_en
- dfi_wrdata_mask

For more information about the above signals, refer to “Signal Descriptions”. The timing of this interface is user-configurable, to support all DFI-compliant PHYs through the DFITMG0.dfi_tphy_wrlat and DFITMG0.dfi_tphy_wrdata register (see DDRC_REGS Registers). For DFI 1:2, DFITMG0.dfi_wrdata_use_dfi_phy_clk determines whether DFITMG0.dfi_tphy_wrlat and DFITMG0.dfi_tphy_wrdata are in terms of SDR or HDR clock cycles. If HDR, timing for dfi_wrdata* generation for write commands on phase 0/phase 1 are the same and dfi_wrdata* signals are aligned to HDR clock. If SDR, timing for dfi_wrdata* generation for write commands on phase 0/phase 1 are treated separately and dfi_wrdata* signals are not guaranteed to be aligned to HDR clock. Check your PHY requirements for correct programming.

In DDR4 configurations (when MEMC_DDR4 is set) or LPDDR4 configurations (when MEMC_LPDDR4 is set), the signal dfi_wrddata_mask can act as Write DBI signal depending on the user programming of the SDRAM mode register. For more information on DBI, see “Data Bus Inversion (DBI)”.

Also, as per JEDEC Specification, the polarity of dfi_wrddata_mask is inverted when using DDR4 memories (if MSTR. ddr4 = 1). Therefore, for DDR4, if a bit of dfi_wrddata_mask is ‘0’, the corresponding byte is masked.

9.3.2.4.5.1 Support for dfi_wrddata_cs/dfi_rddata_cs Signals (DFI 3.1)

There is an optional support for dfi_wrddata_cs and dfi_rddata_cs signals by setting parameter DDRC_DFI_DATA_CS_EN=1. This is a new feature introduced in DFI 3.0, which also applies to DFI 3.1. Enable this feature only if your PHY supports it. These signals are only supported for configurations with MEMC_NUM_RANKS>1. The polarity of these signals is defined by DFIMISC.dfi_data_cs_polarity.

The signal dfi_wrddata_cs is driven as follows:

- dfi_wrddata_cs is driven according to the relevant chip select tphy_wrclat DFI PHY clock cycles after any command that generates dfi_wrddata. This includes a write, and in DDR4, a PDA (write). It is guaranteed to remain at this value for a minimum of tphy_wrcsgap plus the time for the write data (usually MSTR.burst_rdwr).

Similarly, for dfi_rddata_c:

- dfi_rddata_cs is driven according to the relevant chip select tphy_rdcslat DFI PHY clock cycles after any command that generates dfi_rddata. This includes a Read, in LPDDR4, a MRR and in DDR4, a MPR (read). It is guaranteed to remain at this value for a minimum of tphy_rdcsgap plus the time for the read data (usually MSTR.burst_rdwr).

The table outlines the DFI timing value and the equivalent register in the DDRC.

Table 9-3. DFI timing values for dfi_wrddata_cs/dfi_rddata_cs

DFI Timing Value	DDRC Register
t _{phy_wrclat}	DFITMG2.dfi_tphy_wrclat
t _{phy_wrcsgap}	RANKCTL.diff_rank_wr_gap ^{1 2}
t _{phy_rdcslat}	DFITMG2.dfi_tphy_rdcslat
t _{phy_rdcsgap}	RANKCTL.diff_rank_rd_gap ^{1 2}

NOTE

1. For configurations with MEMC_FREQ_RATIO=2, this register is in terms of DFI clocks (controller clocks), while tphy_wrsgap is in terms of DFI PHY clocks.
2. If using DDR4-LRDIMM, refer to TWRWR and TRDRD timing requirements in JEDEC DDR4 Data Buffer (DDR4DB01) Specification.

9.3.2.4.6 Read Data Interface

The read data interface handles the return of read data across the DFI interface. The DFI Specification defines signals and timing relationships. This interface consists of the following signals:

- dfi_rddata
- dfi_rddata_en
- dfi_rddata_valid
- dfi_rddata_dbi (this input signal exists only if MEMC_DDR4=1)

For more information about the above signals, refer to “Signal Descriptions”. The timing of this interface is user-configurable, to support all DFI-compliant PHYs via the DFITMG0.t_rddata_en register.

For DFI 1:2, DFITMG0.dfi_rddata_use_dfi_phy_clk determines whether DFITMG0.dfi_t_rddata_en is in terms of SDR or HDR clock cycles. If HDR, timing for dfi_rddata_en generation for read commands on phase 0/phase 1 is the same and dfi_rddata* signals are aligned to HDR clock. If SDR, timing for dfi_rddata* generation for read commands on phase 0/phase 1 are treated separately and dfi_rddata* signals are not guaranteed to be aligned to HDR clock. Check your PHY requirements for correct programming.

In DDR4 configurations (when MEMC_DDR4 is set), the signal dfi_rddata_dbi is the read DBI signal. For more information on DBI, see “Data Bus Inversion (DBI)”.

The DDRC has been verified with the Synopsys PHYs. It assumes that the DFI parameter tphy_rdlat does not exceed 48 cycles. For mDDR using BL2, it assumes that the DFI parameter tphy_rdlat does not exceed 26 cycles. For both cases, it also assumes that the DFI parameter trddata_en does not exceed the SDRAM read latency (RL).

9.3.2.4.7 Update Interface

During system operation, the system may require updates to internal settings to compensate for environmental conditions. To ensure that updates do not interfere with signals on the SDRAM interface, the DFI interface supports update modes where the DFI read, write, and control interface are suspended from normal activity. The DFI Specification defines both MC-initiated and PHY-initiated updates.

9.3.2.4.7.1 MC-initiated Updates

This interface consists of the following signals:

- dfi_ctrlupd_req
- dfi_ctrlupd_ack
- dfi_ctrlupd_ack2

For more information about the above signals, refer to “Signal Descriptions”.

MC-initiated updates can be acknowledged or ignored by the PHY. DFI MC-initiated updates are performed periodically by DDRC.

9.3.2.4.7.2 PHY-initiated Updates

This interface consists of the following signals:

- dfi_phyupd_req
- dfi_phyupd_type
- dfi_phyupd_ack

For more information about the above signals, see “Signal Descriptions”.

For more details, see “DFI PHY-initiated Update Request”.

9.3.2.4.8 Status Interface

The DFI interface requires status information for initialization and clock control to the SDRAM devices. These signals are used to convey information between the MC and PHY. This interface consists of the following signals:

- dfi_init_start
- dfi_init_complete
- dfi_frequency
- dfi_dram_clk_disable
- dfi_alert_n (this signal replaces dfi_parity_error).

The DDRC does not support the following signals (defined as optional in the DFI Specification):

- dfi_data_byte_disable
- dfi_freq_ratio

9.3.2.4.8.1 Initialization

The dfi_init_start signal is used to trigger the PHY initialization by setting it to 1. It is driven by the DDRC through the APB interface with the DFIMISC.dfi_init_start read-write register field. The signal dfi_init_complete indicates that the PHY has completed its initialization. This information can be read/pollled by the DDRC through the APB interface with the DFISTAT.dfi_init_complete read-only register field.

9.3.2.4.8.2 Clock Disabling

The dfi_dram_clk_disable signal depends on setting of PWRCTL.en_dfi_dram_clk_disable (see DDRC_REGS Registers). For more information about this, see “Assertion of dfi_dram_clk_disable”.

9.3.2.4.8.3 Frequency Change

The interface consists of three signals:

- dfi_init_start: Output from the controller to the PHY
- dfi_init_complete: Output from the PHY to the controller
- dfi_frequency: Output from the controller to the PHY

The procedure is as follows:

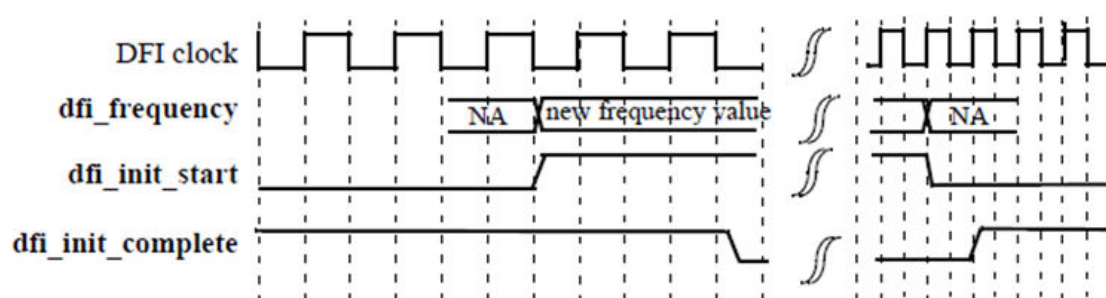


Figure 9-5. Frequency Change

`dfi_frequency` indicates the operating frequency of the system. This signal should change only at initialization, during a DFI frequency change operation, or other times that the system defines. This signal should be constant during normal operation. The number of supported frequencies and the mapping of signal values to clock frequencies are defined by the PHY, system, or both. For timing, the `dfi_frequency` signal must be set to a legal value and remain unchanged when `dfi_init_start` is asserted. The `dfi_frequency` signal can change any time when `dfi_init_start` is low and should be ignored at this time.

The behavior of the `dfi_init_start` signal is dependent on the `dfi_init_complete` signal.

If the PHY accepts the frequency change request, it must deassert the `dfi_init_complete` signal within `tinit_start` cycles of the `dfi_init_start` assertion. The MC continues to hold the `dfi_init_start` signal asserted until the clock frequency change has been completed. The de-assertion should be used by the PHY to reinitialize on the new clock frequency.

If the frequency change is not acknowledged (the `dfi_init_complete` signal remains asserted), the `dfi_init_start` signal must de-assert after `tinit_start` cycles. This requirement needs to be handled by the system as `dfi_init_start` is programmed through the software.

9.3.2.4.9 DFI Training Interface

The additional functions of SDRAM memories allow accurate alignment of critical timing signals. The DFI Specification version 2.1.1 accounts for these functions by providing a DFI training interface.

The DFI Specification defines the following operating modes for the training interface for write leveling, and read leveling:

- No Support
- MC Evaluation (not supported by the DDRC)

- PHY Evaluation
- PHY Independent

These modes define whether the MC or PHY, or neither, maintains the responsibility for managing the programming of the delay lines and evaluating the response.

The DFI training interface enables write and read leveling functions in a DFI-compliant PHY.

In DDRC, the following modes are supported:

- “No Support and PHY Independent Mode”

While interfacing to the DDR PHY, the training is initiated by the PHY Utility Block (PUB) present in the PHY, and PHY independent training mode is used.

9.3.2.4.9.1 No Support and PHY Independent Mode

To enable the No Support and PHY independent mode, the PHY must drive the following signals to “11”:

- dfi_rdlvl_mode
- dfi_rdlvl_gate_mode
- dfi_wrlvl_mode

The above signals are inputs from the DFI-compliant PHY. For these modes, any other DFI Training signals which are outputs can be ignored while any other DFI Training signals that are inputs must be tied to all zeros.

Training is then performed automatically by the PHY, without any intervention from the DDRC. The PUB or PUBL block (part of the PHY) performs all required MRS commands to move the SDRAM to the correct state for each training stage.

If you are using a non-Synopsys PHY that supports PHY independent training mode, DDRC may have to perform these MRS commands.

However, DDRC has no knowledge on when the PHY is ready to execute each training stage, so this must be controlled by the user. For more information on mode register writes, see “Mode Register Write”.

9.3.2.4.10 Low Power interface

In a DDR memory subsystem, it is advantageous to place the PHY in a low power state when the DDRC has knowledge that the memory subsystem remains idle for a period of time. Depending on the state of the system, the DDRC communicates state information to the PHY allowing the PHY to enter the appropriate power saving state. This interface consists of signals that are used to inform the PHY of a low power mode opportunity, as well as how quickly the DDRC requires the PHY to resume normal operation. This interface consists of the following signals:

- dfi_lp_req
- dfi_lp_wakeup
- dfi_lp_ack Software control is provided in DFILPCFG0/DFILPCFG1 registers (see DDRC_REGS Registers on page 464) for the following:
 - This is an optional interface for a DFI-compliant PHY. The interface signals always exist in the DDRC. But the software can enable this interface in self-refresh and/or power-down and/or deep power-down mode and/or maximum power saving mode - DFILPCFG0.dfi_lp_en_pd, DFILPCFG0.dfi_lp_en_sr, DFILPCFG0.dfi_lp_en_dpd and DFILPCFG1.dfi_lp_en_mpsm respectively.
 - Software control over what is driven on dfi_lp_wakeup signal (and its associated timing) in selfrefresh or power-down or deep power-down or maximum power saving mode - DFILPCFG0.dfi_lp_wakeup_pd, DFILPCFG0.dfi_lp_wakeup_sr, DFILPCFG0.dfi_lp_wakeup_dpd and DFILPCFG1.dfi_lp_wakeup_mpsm respectively.
 - The timing of this DFI low power interface - DFILPCFG0.dfi_tlp_resp.
- ctl_idle output pin is a sideband signal (not specified in DFI) supported by certain Synopsys PHYs to trigger the PHY's Anti-Aging feature. This is driven the same as dfi_lp_req if enabled via software (DFIMISC.ctl_idle_en=1 and DFI Low Power Interface is also enabled via DFILPCFG0/DFILPCFG1). If configured for Shared AC, dfi_lp_req signal of both data channels must be asserted for ctl_idle to be asserted.

For more information, see “Power Saving Features”.

9.3.2.4.11 PHY Master Interface

This interface consists of the following signals:

- dfi_phymstr_req
- dfi_phymstr_cs_state
- dfi_phymstr_state_sel
- dfi_phymstr_type
- dfi_phymstr_ack

For more information about the above signals, see “Signal Descriptions”.

For more information about PHY Master interface, see “PHY Master Interface”.

9.3.2.5 APB Interface

The DDRC provides a dedicated APB 3.0 bus interface that is used to access the DDRC software programmable registers. The DDRC converts APB reads and writes into accesses to the internal register file. The APB data width is fixed to 32 bits for compatibility reasons with the DDR PHYs. The APB address width is 12 bits.

All APB interface signals (p*) are synchronous to pclk. pclk can be asynchronous to the core_ddrc_core_clk if the configuration parameter DDRC_P_ASYNC_EN is set. However, pclk frequency must be the same or less than the core_ddrc_core_clk frequency.

9.3.2.5.1 Register Classes

There are three classes of registers in the DDRC:

- Dynamic Registers
- Quasi Dynamic Registers
- Static Registers

Dynamic registers can be written at any time during operation.

Static registers can be written only when the controller is in reset.

Quasi dynamic registers can be written when the controller is in reset and some specific conditions outside reset. To write them, SWCTL.sw_done (see DDRC_REGS Registers on page 464) has to be set to 0 at the beginning of the programming sequence and set back to 1 at the end. After that, SWSTAT.sw_done_ack has to be polled for acknowledge.

9.3.3 Memory Map and register definition

This section includes the DDRC module memory map and detailed descriptions of all registers.

9.3.3.1 DDRC register descriptions

9.3.3.1.1 DDRC memory map

DDRC base address: 0h

Offset	Register	Width (In bits)	Access	Reset value
0h	Master Register0 (MSTR)	32	RW	0304_0001h
4h	Operating Mode Status Register (STAT)	32	RO	0000_0000h
8h	Operating Mode Status Register (MSTR1)	32	RW	0000_0000h
Ch	Operating Mode Status Register (MRCTRL3)	32	RW	0000_0003h
10h	Mode Register Read/Write Control Register 0. (MRCTRL0)	32	RW	0000_0030h
14h	Mode Register Read/Write Control Register 1 (MRCTRL1)	32	RW	0000_0000h
18h	Mode Register Read/Write Status Register (MRSTAT)	32	RO	0000_0000h
1Ch	Mode Register Read/Write Control Register 2 (MRCTRL2)	32	RW	0000_0000h
20h	Temperature Derate Enable Register (DERATEEN)	32	RW	0000_0000h
24h	Temperature Derate Interval Register (DERATEINT)	32	RW	0080_0000h
30h	Low Power Control Register (PWRCTL)	32	RW	0000_0000h
34h	Low Power Timing Register (PWRTMG)	32	RW	0040_2010h
38h	Hardware Low Power Control Register (HWLPCTL)	32	RW	0000_0003h
50h	Refresh Control Register 0 (RFSHCTL0)	32	RW	0021_0000h
54h	Refresh Control Register 1 (RFSHCTL1)	32	RW	0000_0000h
60h	Refresh Control Register 3 (RFSHCTL3)	32	RW	0000_0000h
64h	Refresh Timing Register (RFSHTMG)	32	RW	0062_008Ch
D0h	SDRAM Initialization Register 0 (INIT0)	32	RW	0002_004Eh
D4h	SDRAM Initialization Register 1 (INIT1)	32	RW	0000_0000h
D8h	SDRAM Initialization Register 2 (INIT2)	32	RW	0000_0D05h
DCh	SDRAM Initialization Register 3 (INIT3)	32	RW	0000_0510h
E0h	SDRAM Initialization Register 4 (INIT4)	32	RW	0000_0000h
E4h	SDRAM Initialization Register 5 (INIT5)	32	RW	0010_0004h
E8h	SDRAM Initialization Register 6 (INIT6)	32	RW	0000_0000h
ECh	SDRAM Initialization Register 7 (INIT7)	32	RW	0000_0000h
F0h	DIMM Control Register (DIMMCTL)	32	RW	0000_0000h
F4h	Rank Control Register (RANKCTL)	32	RW	0000_066Fh
100h	SDRAM Timing Register 0 (DRAMTMG0)	32	RW	0F10_1B0Fh
104h	SDRAM Timing Register 1 (DRAMTMG1)	32	RW	0008_0414h
108h	SDRAM Timing Register 2 (DRAMTMG2)	32	RW	0305_060Dh
10Ch	SDRAM Timing Register 3 (DRAMTMG3)	32	RW	0050_400Ch
110h	SDRAM Timing Register 4 (DRAMTMG4)	32	RW	0504_0405h
114h	SDRAM Timing Register 5 (DRAMTMG5)	32	RW	0505_0403h
118h	SDRAM Timing Register 6 (DRAMTMG6)	32	RW	0202_0005h
11Ch	SDRAM Timing Register 7 (DRAMTMG7)	32	RW	0000_0202h

Table continues on the next page...

DDR Controller (DDRC)

Offset	Register	Width (In bits)	Access	Reset value
120h	SDRAM Timing Register 8 (DRAMTMG8)	32	RW	0303_4405h
124h	SDRAM Timing Register 9 (DRAMTMG9)	32	RW	0004_040Dh
128h	SDRAM Timing Register 10 (DRAMTMG10)	32	RW	001C_180Ah
12Ch	SDRAM Timing Register 11 (DRAMTMG11)	32	RW	440C_021Ch
130h	SDRAM Timing Register 12 (DRAMTMG12)	32	RW	0002_0610h
134h	SDRAM Timing Register 13 (DRAMTMG13)	32	RW	1C20_0004h
138h	SDRAM Timing Register 14 (DRAMTMG14)	32	RW	0000_00A0h
13Ch	SDRAM Timing Register 15 (DRAMTMG15)	32	RW	0000_0000h
180h	ZQ Control Register 0 (ZQCTL0)	32	RW	0200_0040h
184h	ZQ Control Register 1 (ZQCTL1)	32	RW	0200_0100h
188h	ZQ Control Register 2 (ZQCTL2)	32	RW	0000_0000h
18Ch	ZQ Status Register (ZQSTAT)	32	RO	0000_0000h
190h	DFI Timing Register 0 (DFITMG0)	32	RW	0702_0002h
194h	DFI Timing Register 1 (DFITMG1)	32	RW	0000_0404h
198h	DFI Low Power Configuration Register 0 (DFILPCFG0)	32	RW	0700_0000h
19Ch	DFI Low Power Configuration Register 1 (DFILPCFG1)	32	RW	0000_0000h
1A0h	DFI Update Register 0 (DFIUPD0)	32	RW	0040_0003h
1A4h	DFI Update Register 1 (DFIUPD1)	32	RW	0001_0001h
1A8h	DFI Update Register 2 (DFIUPD2)	32	RW	8000_0000h
1B0h	DFI Miscellaneous Control Register (DFIMISC)	32	RW	0000_0001h
1B4h	DFI Timing Register 2 (DFITMG2)	32	RW	0000_0202h
1B8h	DFI Timing Register 3 (DFITMG3)	32	RW	0000_0000h
1BCh	DFI Status Register (DFISTAT)	32	RO	0000_0000h
1C0h	DM/DBI Control Register (DBICTL)	32	RW	0000_0001h
200h	Address Map Register 0 (ADDRMAP0)	32	RW	0000_0000h
204h	Address Map Register 1 (ADDRMAP1)	32	RW	0000_0000h
208h	Address Map Register 2 (ADDRMAP2)	32	RW	0000_0000h
20Ch	Address Map Register 3 (ADDRMAP3)	32	RW	0000_0000h
210h	Address Map Register 4 (ADDRMAP4)	32	RW	0000_0000h
214h	Address Map Register 5 (ADDRMAP5)	32	RW	0000_0000h
218h	Address Map Register 6 (ADDRMAP6)	32	RW	0000_0000h
21Ch	Address Map Register 7 (ADDRMAP7)	32	RW	0000_0000h
220h	Address Map Register 8 (ADDRMAP8)	32	RW	0000_0000h
224h	Address Map Register 9 (ADDRMAP9)	32	RW	0000_0000h
228h	Address Map Register 10 (ADDRMAP10)	32	RW	0000_0000h
22Ch	Address Map Register 11 (ADDRMAP11)	32	RW	0000_0000h
240h	ODT Configuration Register (ODTCFG)	32	RW	0400_0400h
244h	ODT/Rank Map Register (ODTMAP)	32	RW	0000_2211h
250h	Scheduler Control Register (SCHED)	32	RW	0000_1005h
254h	Scheduler Control Register 1 (SCHED1)	32	RW	0000_0000h

Table continues on the next page...

Offset	Register	Width (In bits)	Access	Reset value
25Ch	High Priority Read CAM Register 1 (PERFHPR1)	32	RW	0F00_0001h
264h	Low Priority Read CAM Register 1 (PERFLPR1)	32	RW	0F00_007Fh
26Ch	Write CAM Register 1 (PERFWR1)	32	RW	0F00_007Fh
300h	Debug Register 0 (DBG0)	32	RW	0000_0000h
304h	Debug Register 1 (DBG1)	32	RW	0000_0000h
308h	CAM Debug Register (DBGCAM)	32	RO	0000_0000h
30Ch	Command Debug Register (DBGCMD)	32	RW	0000_0000h
310h	Status Debug Register (DBGSTAT)	32	RO	0000_0000h
320h	Software Register Programming Control Enable (SWCTL)	32	RW	0000_0001h
324h	Software Register Programming Control Status (SWSTAT)	32	RO	0000_0001h
36Ch	AXI Poison Configuration Register. (POISONCFG)	32	RW	0011_0011h
370h	AXI Poison Status Register (POISONSTAT)	32	RO	0000_0000h
3FCh	Port Status Register (PSTAT)	32	RO	0000_0000h
400h	Port Common Configuration Register (PCCFG)	32	RW	0000_0000h
404h	Port n Configuration Read Register (PCFGR_0)	32	RW	0000_0000h
408h	Port n Configuration Write Register (PCFGW_0)	32	RW	0000_4000h
490h	Port n Control Register (PCTRL_0)	32	RW	0000_0000h
494h	Port n Read QoS Configuration Register 0 (PCFGQOS0_0)	32	RW	0000_0000h
498h	Port n Read QoS Configuration Register 1 (PCFGQOS1_0)	32	RW	0000_0000h
49Ch	Port n Write QoS Configuration Register 0 (PCFGWQOS0_0)	32	RW	0000_0000h
4A0h	Port n Write QoS Configuration Register 1 (PCFGWQOS1_0)	32	RW	0000_0000h
2020h	[SHADOW] Temperature Derate Enable Register (DERATEEN_SHADOW)	32	RW	0000_0000h
2024h	[SHADOW] Temperature Derate Interval Register (DERATEINT_SHADOW)	32	RW	0080_0000h
2050h	[SHADOW] Refresh Control Register 0 (RFSHCTL0_SHADOW)	32	RW	0021_0000h
2064h	[SHADOW] Refresh Timing Register (RFSHTMG_SHADOW)	32	RW	0062_008Ch
20DCh	[SHADOW] SDRAM Initialization Register 3 (INIT3_SHADOW)	32	RW	0000_0510h
20E0h	[SHADOW] SDRAM Initialization Register 4 (INIT4_SHADOW)	32	RW	0000_0000h
20E8h	[SHADOW] SDRAM Initialization Register 6 (INIT6_SHADOW)	32	RW	0000_0000h
20ECh	[SHADOW] SDRAM Initialization Register 7 (INIT7_SHADOW)	32	RW	0000_0000h
2100h	[SHADOW] SDRAM Timing Register 0 (DRAMTMG0_SHADOW)	32	RW	0F10_1B0Fh
2104h	[SHADOW] SDRAM Timing Register 1 (DRAMTMG1_SHADOW)	32	RW	0008_0414h
2108h	[SHADOW] SDRAM Timing Register 2 (DRAMTMG2_SHADOW)	32	RW	0305_060Dh
210Ch	[SHADOW] SDRAM Timing Register 3 (DRAMTMG3_SHADOW)	32	RW	0050_400Ch
2110h	[SHADOW] SDRAM Timing Register 4 (DRAMTMG4_SHADOW)	32	RW	0504_0405h
2114h	[SHADOW] SDRAM Timing Register 5 (DRAMTMG5_SHADOW)	32	RW	0505_0403h
2118h	[SHADOW] SDRAM Timing Register 6 (DRAMTMG6_SHADOW)	32	RW	0202_0005h
211Ch	[SHADOW] SDRAM Timing Register 7 (DRAMTMG7_SHADOW)	32	RW	0000_0202h
2120h	[SHADOW] SDRAM Timing Register 8 (DRAMTMG8_SHADOW)	32	RW	0303_4405h

Table continues on the next page...

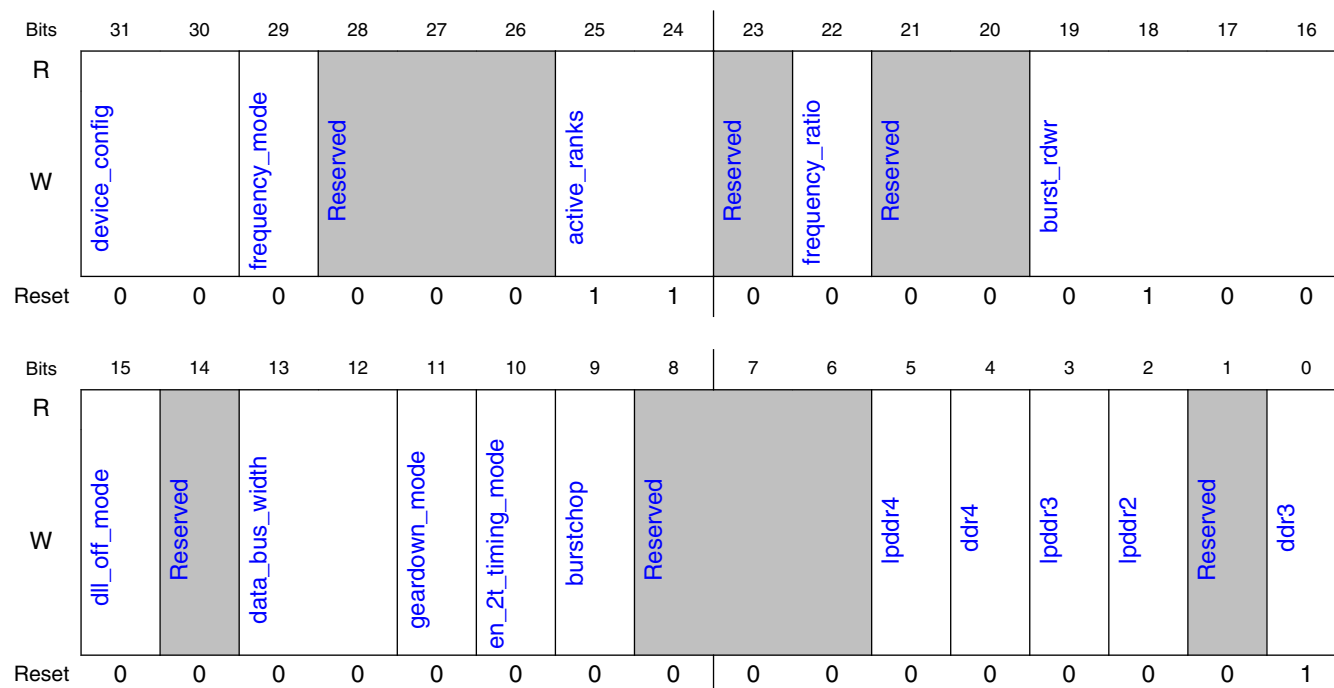
Offset	Register	Width (In bits)	Access	Reset value
2124h	[SHADOW] SDRAM Timing Register 9 (DRAMTMG9_SHADOW)	32	RW	0004_040Dh
2128h	[SHADOW] SDRAM Timing Register 10 (DRAMTMG10_SHADOW)	32	RW	001C_180Ah
212Ch	[SHADOW] SDRAM Timing Register 11 (DRAMTMG11_SHADOW)	32	RW	440C_021Ch
2130h	[SHADOW] SDRAM Timing Register 12 (DRAMTMG12_SHADOW)	32	RW	0002_0610h
2134h	[SHADOW] SDRAM Timing Register 13 (DRAMTMG13_SHADOW)	32	RW	1C20_0004h
2138h	[SHADOW] SDRAM Timing Register 14 (DRAMTMG14_SHADOW)	32	RW	0000_00A0h
213Ch	[SHADOW] SDRAM Timing Register 15 (DRAMTMG15_SHADOW)	32	RW	0000_0000h
2180h	[SHADOW] ZQ Control Register 0 (ZQCTL0_SHADOW)	32	RW	0200_0040h
2190h	[SHADOW] DFI Timing Register 0 (DFITMG0_SHADOW)	32	RW	0702_0002h
2194h	[SHADOW] DFI Timing Register 1 (DFITMG1_SHADOW)	32	RW	0000_0404h
21B4h	[SHADOW] DFI Timing Register 2 (DFITMG2_SHADOW)	32	RW	0000_0202h
21B8h	[SHADOW] DFI Timing Register 3 (DFITMG3_SHADOW)	32	RW	0000_0000h
2240h	[SHADOW] ODT Configuration Register (ODTCFG_SHADOW)	32	RW	0400_0400h

9.3.3.1.2 Master Register0 (MSTR)

9.3.3.1.2.1 Offset

Register	Offset
MSTR	0h

9.3.3.1.2.2 Diagram



9.3.3.1.2.3 Fields

Field	Function
31-30 device_config	Indicates the configuration of the device used in the system. 00b - x4 device 01b - x8 device 10b - x16 device 11b - x32 device
29 frequency_mode	Choose which registers are used. 0b - Original Registers 1b - Shadow Registers
28-26 —	Reserved
25-24 active_ranks	Only present for multi-rank configurations. Each bit represents one rank. For two-rank configurations, only bits[25:24] are present. 1 - populated 0 - unpopulated LSB is the lowest rank number. For 2 ranks following combinations are legal 01 - One rank 11 - Two ranks - Others- Reserved For 4 ranks following combinations are legal:

Table continues on the next page...

DDR Controller (DDRC)

Field	Function
	• 0001 - One rank • 0011 - Two ranks • 1111 - Four ranks •
23 —	Reserved
22 frequency_ratio	Selects the Frequency Ratio 0b - 1:2 Mode 1b - 1:1 Mode
21-20 —	Reserved
19-16 burst_rdw	SDRAM burst length used All other values are reserved. This controls the burst size used to access the SDRAM. This must match the burst length mode register setting in the SDRAM. (For BC4/8 on-the-fly mode of DDR3 and DDR4, set this field to 0x0100) Burst length of 2 is not supported with AXI ports when MEMC_BURST_LENGTH is 8. Burst length of 2 is only supported when the controller is operating in 1:1 frequency mode 0001b - Burst length of 2 (only supported for mDDR) 0010b - Burst length of 4 0100b - Burst length of 8 1000b - Burst length of 16 (only supported for mDDR, LPDDR2, and LPDDR4)
15 dll_off_mode	Set to 1 when the DDRC and DRAM has to be put in DLL-off mode for low frequency operation. Set to 0 to put DDRC and DRAM in DLL-on mode for normal frequency operation. If DDR4 CRC/parity retry is enabled (CRCPARCTL1.crc_parity_retry_enable = 1), dll_off_mode is not supported, and this bit must be set to '0'.
14 —	Reserved
13-12 data_bus_width	Selects proportion of DQ bus width that is used by the SDRAM - 00 - Full DQ bus width to SDRAM - 01 - Half DQ bus width to SDRAM - 10 - Quarter DQ bus width to SDRAM - 11 - Reserved Note that half bus width mode is only supported when the SDRAM bus width is a multiple of 16, and quarter bus width mode is only supported when the SDRAM bus width is a multiple of 32 and the configuration parameter MEMC_QBUS_SUPPORT is set. Bus width refers to DQ bus width (excluding any ECC width).
11 geardown_mode	1 indicates put the DRAM in geardown mode (2N) and 0 indicates put the DRAM in normal mode (1N). This register can be changed, only when the Controller is in self-refresh mode. This signal must be set the same value as MR3 bit A3. Note: Geardown mode is not supported if the configuration parameter MEMC_CMD_RTN2IDLE is set Note: Geardown mode is not supported if the configuration parameter DDRC_SHARED_AC is set (in Shared-AC mode) and the register value is don't care
10 en_2t_timing_m ode	If 1, then DDRC uses 2T timing. Otherwise, uses 1T timing. In 2T timing, all command signals (except chip select) are held for 2 clocks on the SDRAM bus. Chip select is asserted on the second cycle of the command Note: 2T timing is not supported in LPDDR2/LPDDR3/LPDDR4 mode Note: 2T timing is not supported if the configuration parameter MEMC_CMD_RTN2IDLE is set Note: 2T timing is not supported in DDR4 geardown mode. Note: 2T timing is not supported in Shared-AC dual channel mode and the register value is don't care.
9 burstchop	When set, enable burst-chop (BC4 or 8 on-the-fly) in DDR3/DDR4. Burst Chop for Reads is exercised only in HIF configurations (DDRC_INCL_ARB not set) and if in full bus width mode (MSTR.data_bus_width = 00) and if MEMC_BURST_LENGTH=8 or 16. Burst Chop for Writes is exercised only if Partial Writes enabled (DDRC_PARTIAL_WR=1) and if CRC is disabled (CRCPARCTL1.crc_enable = 0). If DDR4 CRC/parity retry is enabled (CRCPARCTL1.crc_parity_retry_enable = 1), burst chop is not supported, and this bit must be set to '0'. BC4 (fixed) mode is not supported.
8-6	Reserved

Table continues on the next page...

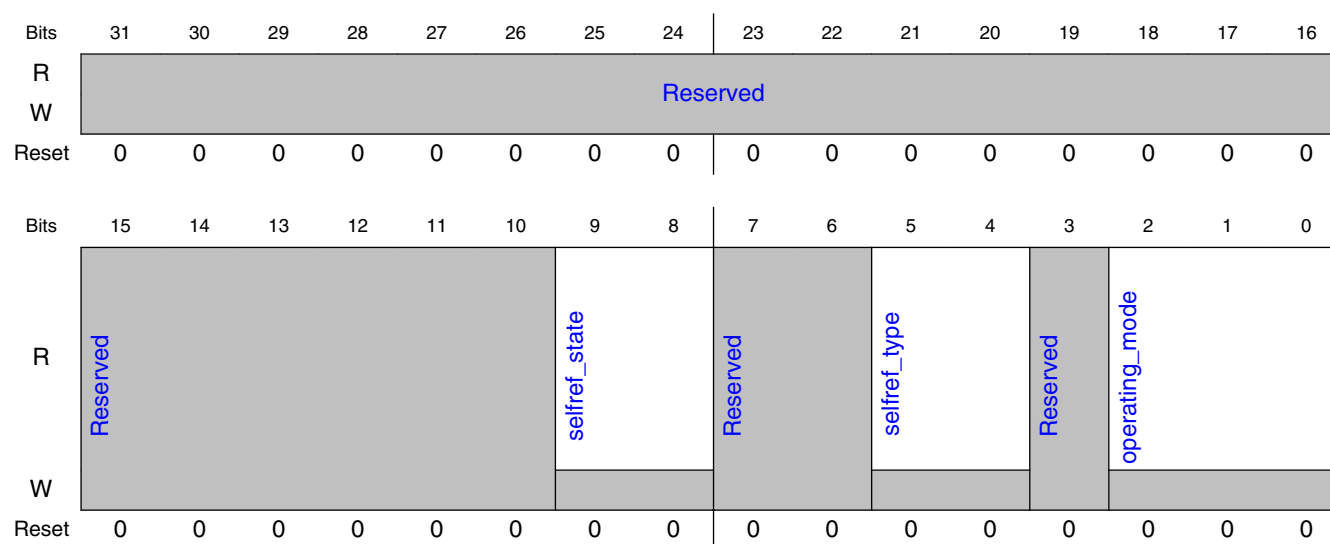
Field	Function
—	
5 lpddr4	Select LPDDR4 SDRAM - 1 - LPDDR4 SDRAM device in use. - 0 - non-LPDDR4 device in use Present only in designs configured to support LPDDR4.
4 ddr4	Select DDR4 SDRAM - 1 - DDR4 SDRAM device in use. - 0 - non-DDR4 device in use Present only in designs configured to support DDR4.
3 lpddr3	Select LPDDR3 SDRAM - 1 - LPDDR3 SDRAM device in use. - 0 - non-LPDDR3 device in use Present only in designs configured to support LPDDR3.
2 lpddr2	Select LPDDR2 SDRAM - 1 - LPDDR2 SDRAM device in use. - 0 - non-LPDDR2 device in use Present only in designs configured to support LPDDR2.
1 —	Reserved
0 ddr3	Select DDR3 SDRAM - 1 - DDR3 SDRAM device in use - 0 - non-DDR3 SDRAM device in use Only present in designs that support DDR3.

9.3.3.1.3 Operating Mode Status Register (STAT)

9.3.3.1.3.1 Offset

Register	Offset
STAT	4h

9.3.3.1.3.2 Diagram



9.3.3.1.3.3 Fields

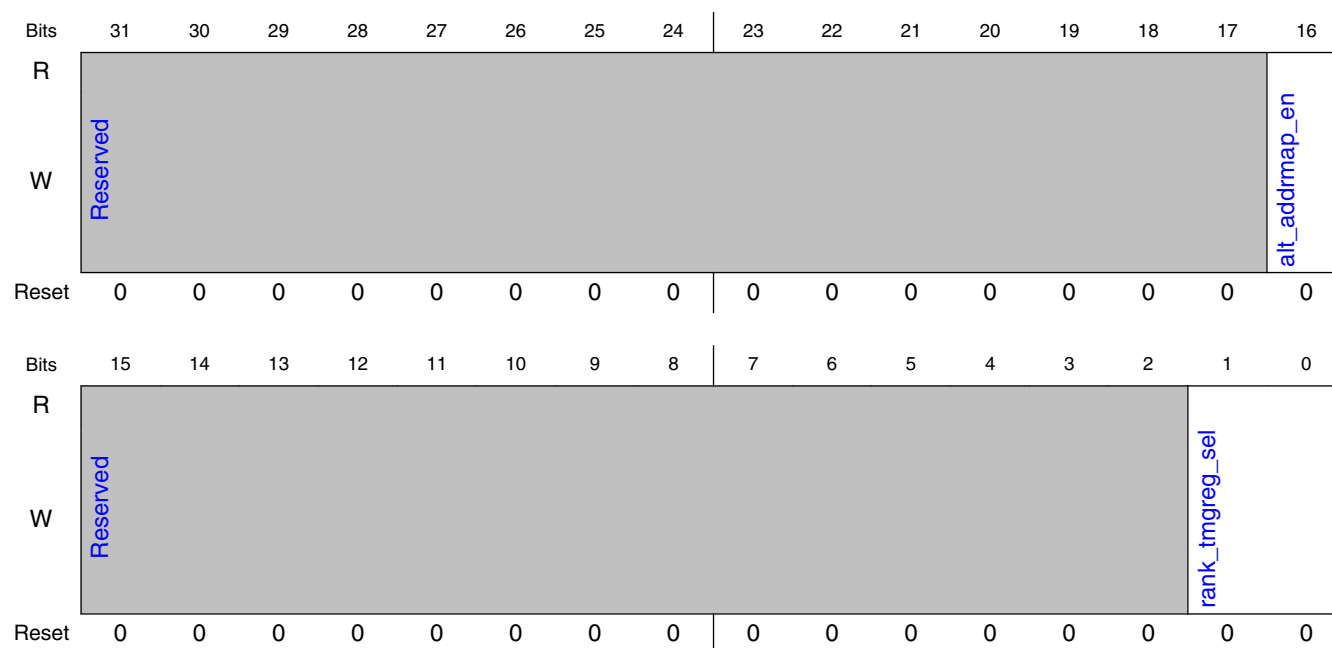
Field	Function
31-10 —	Reserved
9-8 selfref_state	Self refresh state. This indicates self refresh or self refresh power down state for LPDDR4. This register is used for frequency change and MRR/MRW access during self refresh. 00b - SDRAM is not in Self Refresh. 01b - Self refresh 1 10b - Self refresh power down 11b - Self refresh
7-6 —	Reserved
5-4 selfref_type	Flags if Self Refresh (except LPDDR4) or SR-Powerdown (LPDDR4) is entered and if it was under Automatic Self Refresh control only or not. 00b - SDRAM is not in Self Refresh (except LPDDR4) or SR-Powerdown (LPDDR4). If retry is enabled by CRCPARCTRL1.crc_parity_retry_enable, this also indicates SRE command is still in parity error window or retry is in-progress. 10b - SDRAM is in Self Refresh (except LPDDR4) or SR-Powerdown (LPDDR4), which was not caused solely under Automatic Self Refresh control. It could have been caused by Hardware Low Power Interface and/or Software (reg_ddrc_selfref_sw). If retry is enabled, this guarantees SRE command is executed correctly without parity 11b - SDRAM is in Self Refresh (except LPDDR4) or SR-Powerdown (LPDDR4), which was caused by Automatic Self Refresh only. If retry is enabled, this guarantees SRE command is executed correctly without parity error.
3 —	Reserved
2-0 operating_mode	Operating mode This is 3-bits wide in configurations with mDDR/LPDDR2/LPDDR3/LPDDR4/DDR4 support and 2-bits in all other configurations. Non-mDDR/LPDDR2/LPDDR3/LPDDR4 and non-DDR4 designs: • 00 - Init • 01 - Normal • 10 - Power-down • 11 - Self refresh mDDR/LPDDR2/LPDDR3 or DDR4 designs: • 000 - Init • 001 - Normal • 010 - Power-down • 011 - Self refresh • 1XX - Deep power-down / Maximum Power Saving Mode LPDDR4 designs: • 000 - Init • 001 - Normal • 010 - Power-down • 011 - Self refresh / Self refresh power-down

9.3.3.1.4 Operating Mode Status Register (MSTR1)

9.3.3.1.4.1 Offset

Register	Offset
MSTR1	8h

9.3.3.1.4.2 Diagram



9.3.3.1.4.3 Fields

Field	Function
31-17 —	Reserved
16 alt_addrmap_en	Enable Alternative Address Map 0b - Disable Alternative Address Map 1b - Enable Alternative Address Map
15-2 —	Reserved
1-0 rank_tmgreg_sel	rank_tmgreg_sel Indicates which register set is used for each rank. Each bit represents one rank. (LSB is the lowest rank number.)

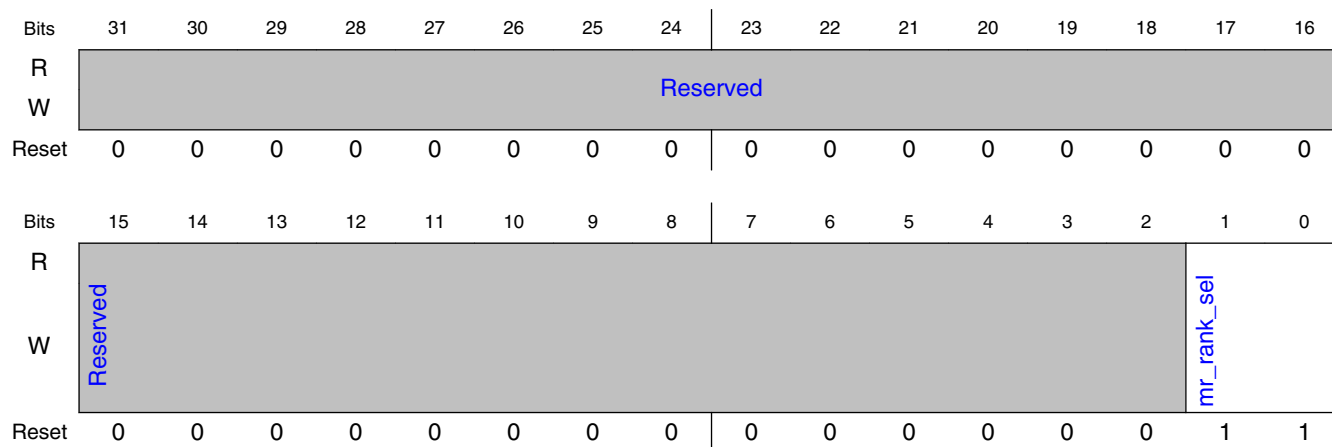
Field	Function
	NOTE: Only some of timing registers in DRAMTMGx are duplicated as MRAMTMGx for MRAM and its width is expanded for MRAM. This register can switch the duplicate registers, otherwise use DRAMTMGx. This register is only for DDR4 ST-MRAM rank, otherwise must be set to 0. This register must be set up while the Controller is in reset. 00b - USE DRAMTMGx registers for the rank 01b - USE MRAMTMGx registers for the rank

9.3.3.1.5 Operating Mode Status Register (MRCTRL3)

9.3.3.1.5.1 Offset

Register	Offset
MRCTRL3	Ch

9.3.3.1.5.2 Diagram



9.3.3.1.5.3 Fields

Field	Function
31-2	Reserved
—	
1-0	mr_rank_sel
mr_rank_sel	Controls which rank is accessed by MRCTRL0.mr_wr.

Field	Function
	Normally, it is desired to access all ranks, so all bits should be set to 1. However, for multi-rank UDIMMs/RDIMMs/LRDIMMs which implement address mirroring, it may be necessary to access ranks individually. Examples (assume uMCTL2 is configured for 8 ranks): • 0x01 - select rank 0 only • 0x02 - select rank 1 only • 0x05 - select ranks 0 and 2

9.3.3.1.6 Mode Register Read/Write Control Register 0. (MRCTRL0)

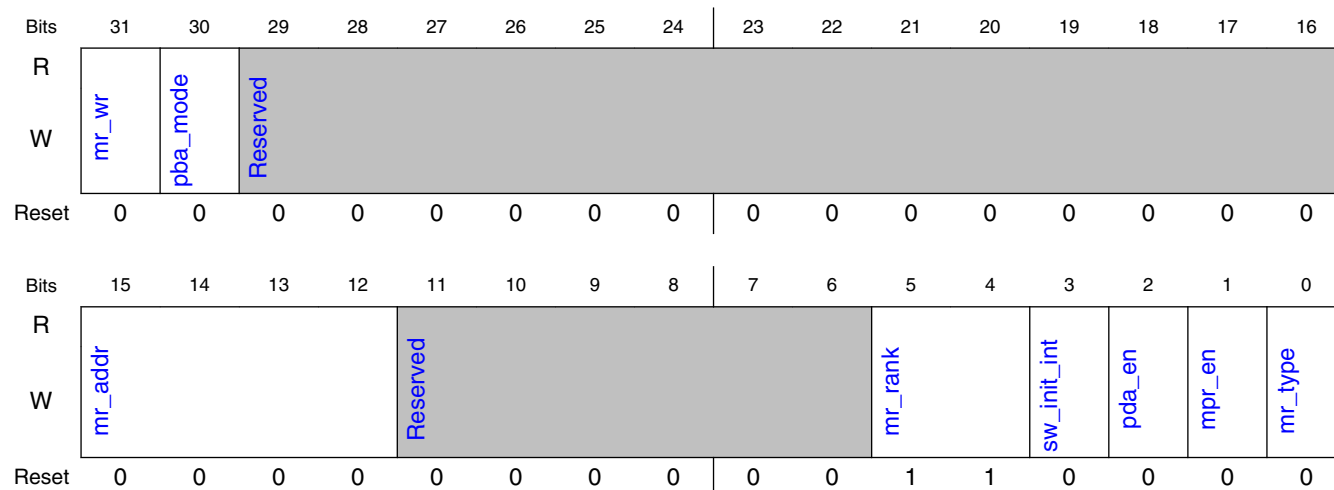
9.3.3.1.6.1 Offset

Register	Offset
MRCTRL0	10h

9.3.3.1.6.2 Function

Mode Register Read/Write Control Register 0. Note: Do not enable more than one of the following fields simultaneously: - sw_init_int - pda_en - mpr_en

9.3.3.1.6.3 Diagram



9.3.3.1.6.4 Fields

Field	Function
31 mr_wr	Setting this register bit to 1 triggers a mode register read or write operation. When the MR operation is complete, the DDRC automatically clears this bit. The other register fields of this register must be written in a separate APB transaction, before setting this mr_wr bit. It is recommended NOT to set this signal if in Init, Deep power-down or MPSM operating modes.
30 pba_mode	Indicates whether PBA access is executed. When setting this bit to 1 along with setting pda_en to 1, DDRC initiates PBA access instead of PDA access. - 0 - Per DRAM Addressability mode - 1 - Per Buffer Addressability mode The completion of PBA access is confirmed by MRSTAT.pda_done in the same way as PDA.
29-16 —	Reserved
15-12 mr_addr	Address of the mode register that is to be written to. Don't Care for LPDDR2/LPDDR3/LPDDR4 (see MRCTRL1.mr_data for mode register addressing in LPDDR2/LPDDR3/LPDDR4) This signal is also used for writing to control words of the register chip on RDIMMs/LRDIMMs. In that case, it corresponds to the bank address bits sent to the RDIMM/LRDIMM In case of DDR4, the bit[3:2] corresponds to the bank group bits. Therefore, the bit[3] as well as the bit[2:0] must be set to an appropriate value which is considered both the Address Mirroring of UDIMMs/RDIMMs/LRDIMMs and the Output Inversion of RDIMMs/LRDIMMs. 0000b - MR0 0001b - MR1 0010b - MR2 0011b - MR3 0100b - MR4 0101b - MR5 0110b - MR6 0111b - MR7
11-6 —	Reserved
5-4 mr_rank	Controls which rank is accessed by MRCTRL0.mr_wr. Normally, it is desired to access all ranks, so all bits should be set to 1. However, for multi-rank UDIMMs/RDIMMs/LRDIMMs which implement address mirroring, it may be necessary to access ranks individually. Examples (assume DDRC is configured for 4 ranks): 0x1 - select rank 0 only 0x2 - select rank 1 only 0x5 - select ranks 0 and 2 0xA - select ranks 1 and 3 0xF - select ranks 0, 1, 2 and 3
3 sw_init_int	Indicates whether Software intervention is allowed via MRCTRL0/MRCTRL1 before automatic SDRAM initialization routine or not. For DDR4, this bit can be used to initialize the DDR4 RCD (MR7) before automatic SDRAM initialization. For LPDDR4, this bit can be used to program additional mode registers before automatic SDRAM initialization if necessary. In LPDDR4 independent channel mode, note that this must be programmed to both channels beforehand. Note that this must be cleared to 0 after completing Software operation. Otherwise, SDRAM initialization routine will not re-start. 0b - Software intervention is not allowed 1b - Software intervention is allowed
2 pda_en	Indicates whether the mode register operation is MRS in PDA mode or not. Note that when pba_mode=1, PBA access is initiated instead of PDA access. 0b - MRS 1b - MRS in Per DRAM Addressability
1 mpr_en	Indicates whether the mode register operation is MRS or WR/RD for MPR (only supported for DDR4). 0b - MRS 1b - WR/RD for MPR
0 mr_type	Indicates whether the mode register operation is read or write. Only used for LPDDR2/LPDDR3/LPDDR4/DDR4.

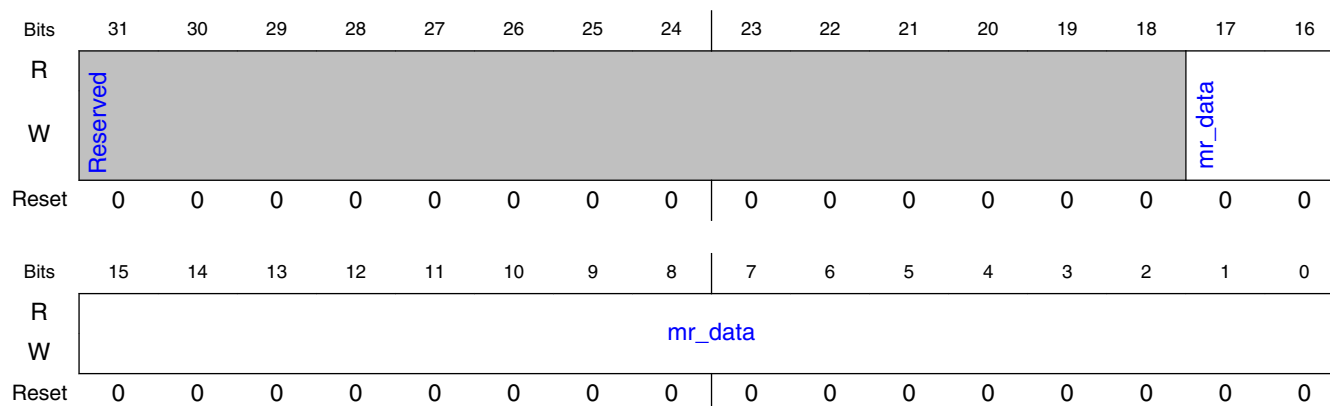
Field	Function
	0b - Write 1b - Read

9.3.3.1.7 Mode Register Read/Write Control Register 1 (MRCTRL1)

9.3.3.1.7.1 Offset

Register	Offset
MRCTRL1	14h

9.3.3.1.7.2 Diagram



9.3.3.1.7.3 Fields

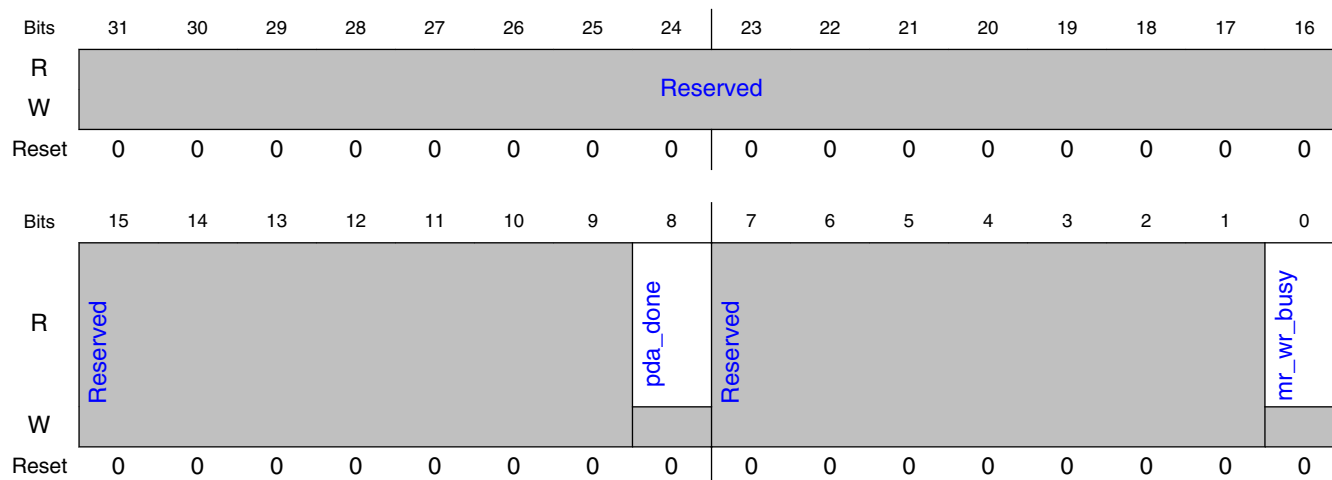
Field	Function
31-18 —	Reserved
17-0 mr_data	Mode register write data for all non-LPDDR2/non-LPDDR3/non-LPDDR4 modes. For LPDDR2/LPDDR3/LPDDR4, MRCTRL1[15:0] are interpreted as [15:8] MR Address [7:0] MR data for writes, don't care for reads. This is 18-bits wide in configurations with DDR4 support and 16-bits in all other configurations.

9.3.3.1.8 Mode Register Read/Write Status Register (MRSTAT)

9.3.3.1.8.1 Offset

Register	Offset
MRSTAT	18h

9.3.3.1.8.2 Diagram



9.3.3.1.8.3 Fields

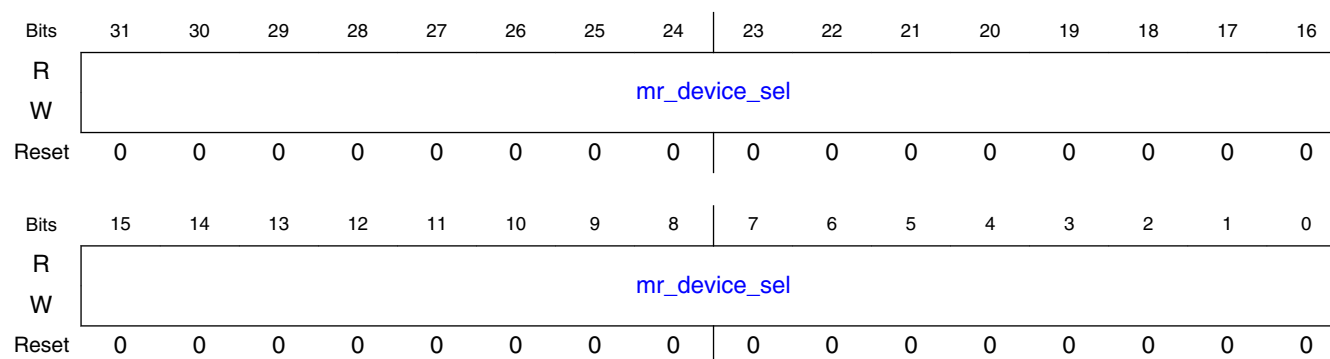
Field	Function
31-9 —	Reserved
8 pda_done	The SoC core may initiate a MR write operation in PDA/PBA mode only if this signal is low. This signal goes high when three consecutive MRS commands related to the PDA/PBA mode are issued to the SDRAM. This signal goes low when MRCTRL0.pda_en becomes 0. Therefore, it is recommended to write MRCTRL0.pda_en to 0 after this signal goes high in order to prepare to perform PDA operation next time 0b - Indicates that mode register write operation related to PDA/PBA is in progress or has not started yet. 1b - Indicates that mode register write operation related to PDA/PBA has competed.
7-1 —	Reserved
0 mr_wr_busy	The SoC core may initiate a MR write operation only if this signal is low. This signal goes high in the clock after the DDRC accepts the MRW/MRR request. It goes low when the MRW/MRR command is issued to the SDRAM. It is recommended not to perform MRW/MRR commands when 'MRSTAT.mr_wr_busy' is high. 0b - Indicates that the SoC core can initiate a mode register write operation 1b - Indicates that mode register write operation is in progress

9.3.3.1.9 Mode Register Read/Write Control Register 2 (MRCTRL2)

9.3.3.1.9.1 Offset

Register	Offset
MRCTRL2	1Ch

9.3.3.1.9.2 Diagram



9.3.3.1.9.3 Fields

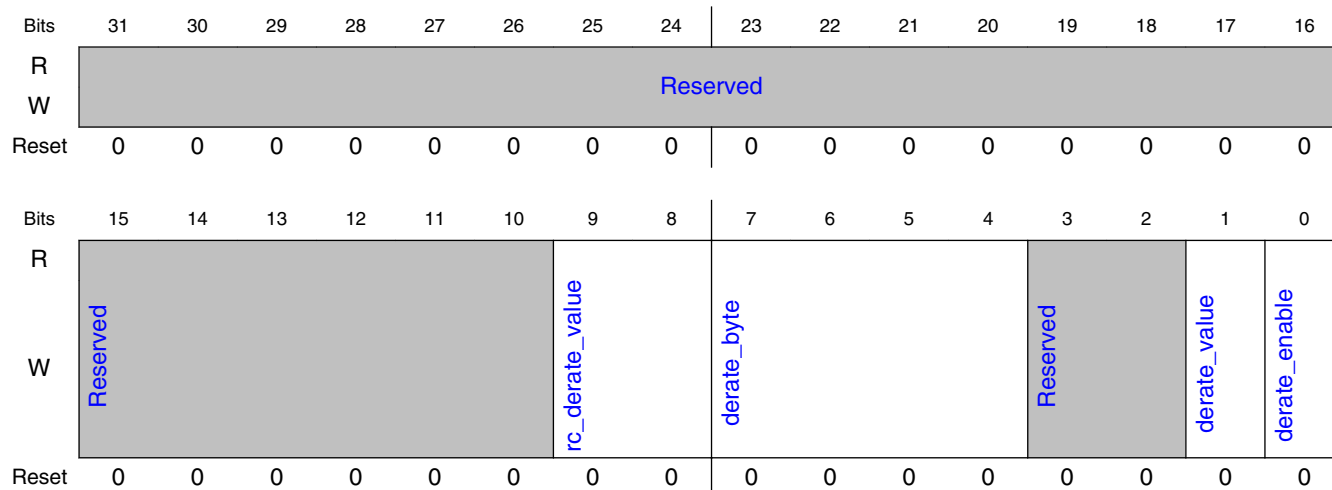
Field	Function
31-0 mr_device_sel	Indicates the device(s) to be selected during the MRS that happens in PDA mode. Each bit is associated with one device. For example, bit[0] corresponds to Device 0, bit[1] to Device 1 etc. A '1' should be programmed to indicate that the MRS command should be applied to that device. A '0' indicates that the MRS commands should be skipped for that device.

9.3.3.1.10 Temperature Derate Enable Register (DERATEEN)

9.3.3.1.10.1 Offset

Register	Offset
DERATEEN	20h

9.3.3.1.10.2 Diagram



9.3.3.1.10.3 Fields

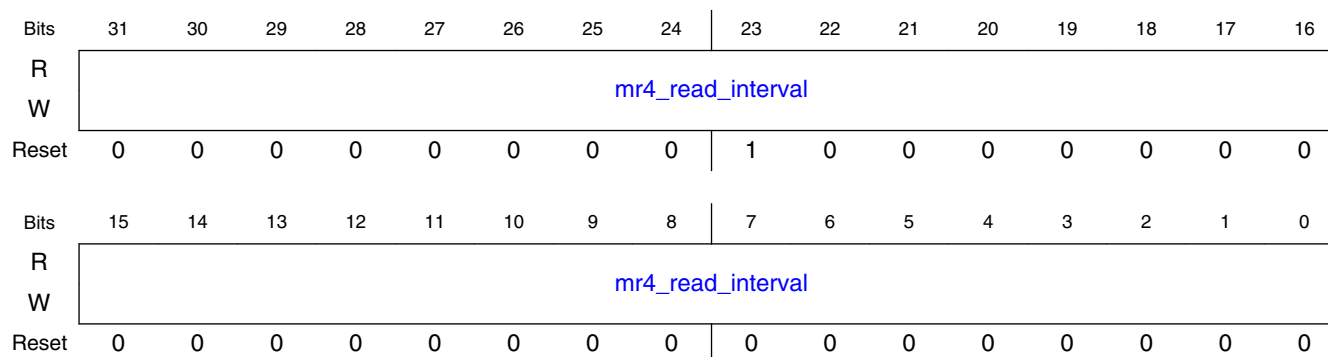
Field	Function
31-10 —	Reserved
9-8 rc_derate_value	Derate value of tRC for LPDDR4. Present only in designs configured to support LPDDR4. The required number of cycles for derating can be determined by dividing 3.75ns by the core_ddrc_core_clk period, and rounding up the next integer. 00b - Derating uses +1 01b - Derating uses +2 10b - Derating uses +3 11b - Derating uses +4
7-4 derate_byte	Derate byte Present only in designs configured to support LPDDR2/LPDDR3/LPDDR4 Indicates which byte of the MRR data is used for derating. The maximum valid value depends on MEMC_DRAM_TOTAL_DATA_WIDTH.
3-2 —	Reserved
1 derate_value	Derate value. Present only in designs configured to support LPDDR2/LPDDR3/LPDDR4 Set to 0 for all LPDDR2 speed grades as derating value of +1.875 ns is less than a core_ddrc_core_clk period. For LPDDR3/4, if the period of core_ddrc_core_clk is less than 1.875ns, this register field should be set to 1; otherwise it should be set to 0. 0b - Derating uses +1 1b - Derating uses +2
0 derate_enable	Enables derating. Present only in designs configured to support LPDDR2/LPDDR3/LPDDR4. This field must be set to '0' for non-LPDDR2/LPDDR3/LPDDR4 mode. 0b - Timing parameter derating is disabled 1b - Timing parameter derating is enabled using MR4 read value.

9.3.3.1.11 Temperature Derate Interval Register (DERATEINT)

9.3.3.1.11.1 Offset

Register	Offset
DERATEINT	24h

9.3.3.1.11.2 Diagram



9.3.3.1.11.3 Fields

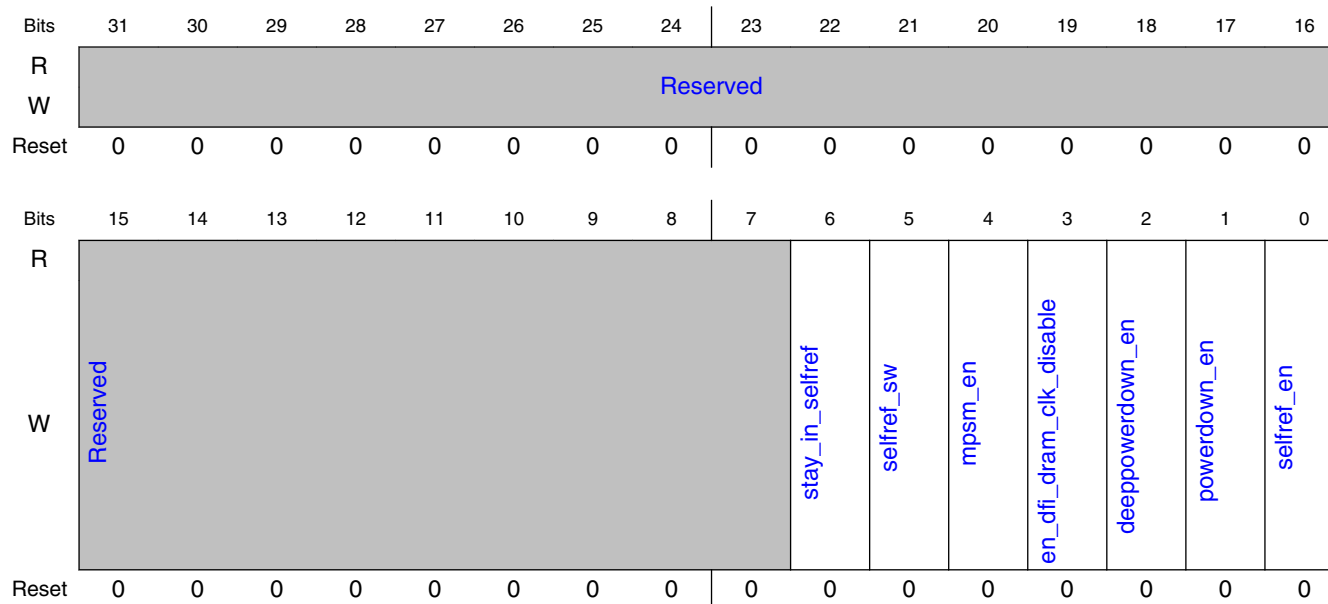
Field	Function
31-0 mr4_read_interval	Interval between two MR4 reads, used to derate the timing parameters. Present only in designs configured to support LPDDR2/LPDDR3/LPDDR4. This register must not be set to zero. Unit: DFI clock cycle.

9.3.3.1.12 Low Power Control Register (PWRCTL)

9.3.3.1.12.1 Offset

Register	Offset
PWRCTL	30h

9.3.3.1.12.2 Diagram



9.3.3.1.12.3 Fields

Field	Function
31-7 —	Reserved
6 stay_in_selfref	Self refresh state is an intermediate state to enter to Self refresh power down state or exit Self refresh power down state for LPDDR4. This register controls transition from the Self refresh state. - 1 - Prohibit transition from Self refresh state - 0 - Allow transition from Self refresh state 0b - 1b -
5 selfref_sw	A value of 1 to this register causes system to move to Self Refresh state immediately, as long as it is not in INIT or DPD/MPSM operating_mode. This is referred to as Software Entry/Exit to Self Refresh. 0b - Software Exit from Self Refresh 1b - Software Entry to Self Refresh
4 mpsm_en	When this is 1, the DDRC puts the SDRAM into maximum power saving mode when the transaction store is empty. This register must be reset to '0' to bring DDRC out of maximum power saving mode. Present only in designs configured to support DDR4. For non-DDR4, this register should not be set to 1. Note that MPSM is not supported when using a DDR PHY, if the PHY parameter DDRC_AC_CS_USE is disabled, as the MPSM exit sequence requires the chip-select signal to toggle. FOR PERFORMANCE ONLY.
3 en_dfi_dram_clk_disable	Enable the assertion of dfi_dram_clk_disable whenever a clock is not required by the SDRAM. If set to 0, dfi_dram_clk_disable is never asserted. Assertion of dfi_dram_clk_disable is as follows: In DDR2/DDR3, can only be asserted in Self Refresh. In DDR4, can be asserted in following: in Self Refresh in Maximum Power Saving Mode In mDDR/LPDDR2/LPDDR3, can be asserted in following: in Self Refresh in Power Down in Deep Power Down during Normal operation (Clock Stop) In LPDDR4, can be asserted in following: in Self Refresh Power Down in Power Down during Normal operation (Clock Stop)
2 deeppowerdown_en	When this is 1, DDRC puts the SDRAM into deep power-down mode when the transaction store is empty. This register must be reset to '0' to bring DDRC out of deep power-down mode. Controller performs automatic SDRAM initialization on deep power-down exit. Present only in designs configured to support

Table continues on the next page...

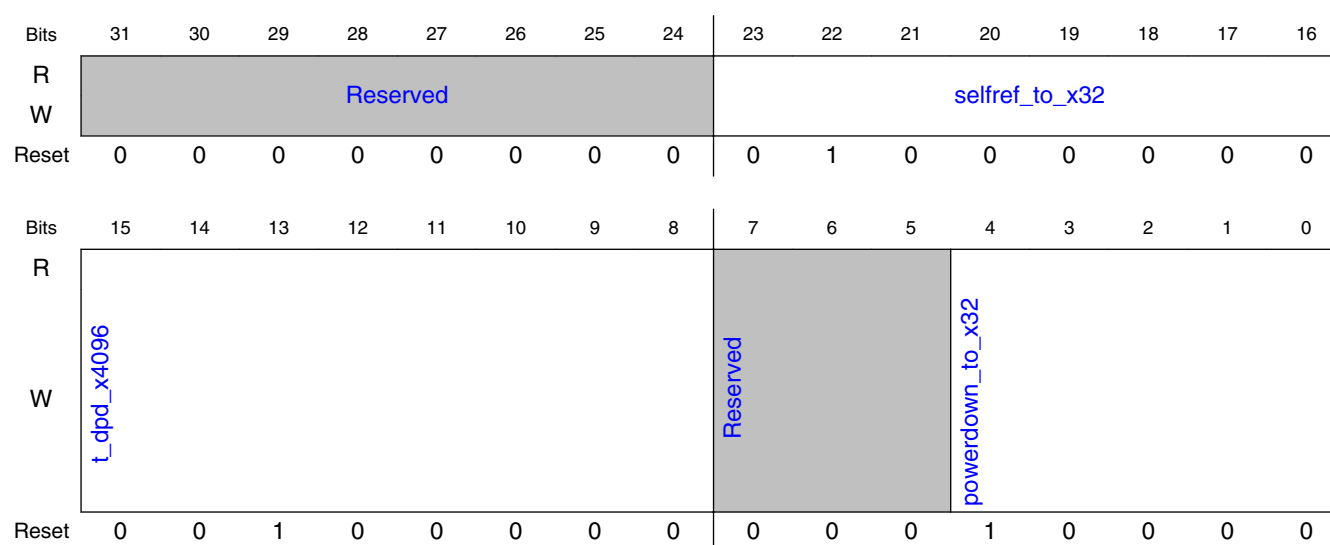
Field	Function
	mDDR or LPDDR2 or LPDDR3. For non-mDDR/non-LPDDR2/non-LPDDR3, this register should not be set to 1. FOR PERFORMANCE ONLY.
1 powerdown_en	If true then the DDRC goes into power-down after a programmable number of cycles "maximum idle clocks before power down" (PWRTMG.powerdown_to_x32). This register bit may be re-programmed during the course of normal operation.
0 selfref_en	If true then the DDRC puts the SDRAM into Self Refresh after a programmable number of cycles "maximum idle clocks before Self Refresh (PWRTMG.selfref_to_x32)". This register bit may be re-programmed during the course of normal operation.

9.3.3.1.13 Low Power Timing Register (PWRTMG)

9.3.3.1.13.1 Offset

Register	Offset
PWRTMG	34h

9.3.3.1.13.2 Diagram



9.3.3.1.13.3 Fields

Field	Function
31-24	Reserved
—	

Table continues on the next page...

DDR Controller (DDRC)

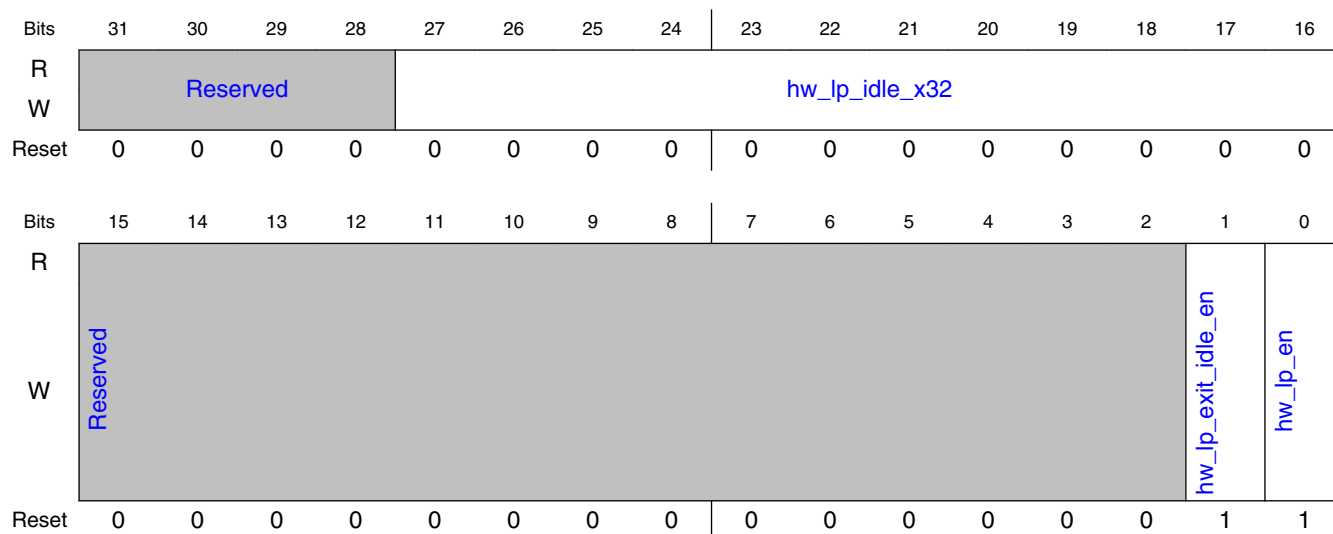
Field	Function
23-16 selfref_to_x32	After this many clocks of the DDRC command channel being idle the DDRC automatically puts the SDRAM into Self Refresh. The DDRC command channel is considered idle when there are no HIF commands outstanding. This must be enabled in the PWRCTL.selfref_en. Unit: Multiples of 32 DFI clocks. FOR PERFORMANCE ONLY.
15-8 t_dpd_x4096	Minimum deep power-down time. For mDDR, value from the JEDEC specification is 0 as mDDR exits from deep power-down mode immediately after PWRCTL.deeppowerdown_en is de-asserted. For LPDDR2/LPDDR3, value from the JEDEC specification is 500us. Unit: Multiples of 4096 DFI clocks. Present only in designs configured to support mDDR, LPDDR2 or LPDDR3. FOR PERFORMANCE ONLY.
7-5 —	Reserved
4-0 powerdown_to_x32	After this many clocks of the DDRC command channel being idle the DDRC automatically puts the SDRAM into power-down. The DDRC command channel is considered idle when there are no HIF commands outstanding. This must be enabled in the PWRCTL.powerdown_en. Unit: Multiples of 32 DFI clocks FOR PERFORMANCE ONLY.

9.3.3.1.14 Hardware Low Power Control Register (HWLPCTL)

9.3.3.1.14.1 Offset

Register	Offset
HWLPCTL	38h

9.3.3.1.14.2 Diagram



9.3.3.1.14.3 Fields

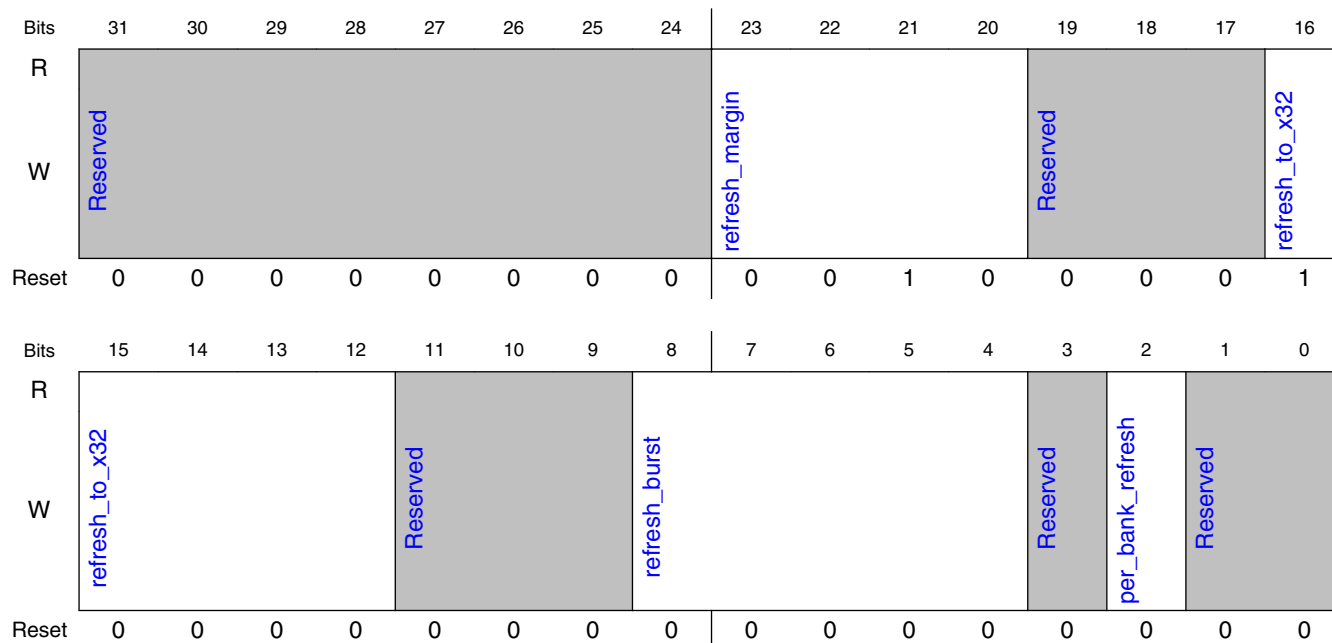
Field	Function
31-28 —	Reserved
27-16 hw_lp_idle_x32	Hardware idle period. The cactive_ddrc output is driven low if the DDRC command channel is idle for $hw_lp_idle * 32$ cycles if not in INIT or DPD/MPSM operating_mode. The DDRC command channel is considered idle when there are no HIF commands outstanding. The hardware idle function is disabled when $hw_lp_idle_x32=0$. Unit: Multiples of 32 DFI clocks. FOR PERFORMANCE ONLY.
15-2 —	Reserved
1 hw_lp_exit_idle_en	When this bit is programmed to 1 the cactive_in_ddrc pin of the DDRC can be used to exit from the automatic clock stop, automatic power down or automatic self-refresh modes. Note, it will not cause exit of Self-Refresh that was caused by Hardware Low Power Interface and/or Software (PWRCTL.selfref_sw).
0 hw_lp_en	Enable for Hardware Low Power Interface.

9.3.3.1.15 Refresh Control Register 0 (RFSHCTL0)

9.3.3.1.15.1 Offset

Register	Offset
RFSHCTL0	50h

9.3.3.1.15.2 Diagram



9.3.3.1.15.3 Fields

Field	Function
31-24 —	Reserved
23-20 refresh_margin	Threshold value in number of DFI clock cycles before the critical refresh or page timer expires. A critical refresh is to be issued before this threshold is reached. It is recommended that this not be changed from the default value, currently shown as 0x2. It must always be less than internally used t_rfc_nom_x32. Note that, in LPDDR2/LPDDR3/LPDDR4, internally used t_rfc_nom_x32 may be equal to RFSHTMG.t_rfc_nom_x32>>2 if derating is enabled (DERATEEN.derate_enable=1). Otherwise, internally used t_rfc_nom_x32 will be equal to RFSHTMG.t_rfc_nom_x32. Unit: Multiples of 32 DFI clocks.
19-17 —	Reserved
16-12 refresh_to_x32	If the refresh timer (tRFCnom, also known as tREFI) has expired at least once, but it has not expired (RFSHTMG.refresh_burst+1) times yet, then a speculative refresh may be performed. A speculative refresh is a refresh performed at a time when refresh would be useful, but before it is absolutely required. When the SDRAM bus is idle for a period of time determined by this RFSHTMG.refresh_to_x32 and the refresh timer has expired at least once since the last refresh, then a speculative refresh is performed. Speculative refreshes continues successively until there are no refreshes pending or until new reads or writes are issued to the DDRC. FOR PERFORMANCE ONLY. Unit: Multiples of 32 DFI clocks.
11-9 —	Reserved
8-4 refresh_burst	The programmed value + 1 is the number of refresh timeouts that is allowed to accumulate before traffic is blocked and the refreshes are forced to execute. Closing pages to perform a refresh is a one-time penalty that must be paid for each group of refreshes. Therefore, performing refreshes in a burst reduces

Table continues on the next page...

Field	Function
	the per-refresh penalty of these page closings. Higher numbers for RFSHCTL.refresh_burst slightly increases utilization; lower numbers decreases the worst-case latency associated with refreshes. - 0 - single refresh - 1 - burst-of-2 refresh - 7 - burst-of-8 refresh For information on burst refresh feature refer to section 3.9 of DDR2 JEDEC specification - JESD79-2F.pdf. For DDR2/3, the refresh is always per-rank and not per-bank. The rank refresh can be accumulated over 8*tREFI cycles using the burst refresh feature. In DDR4 mode, according to Fine Granularity feature, 8 refreshes can be postponed in 1X mode, 16 refreshes in 2X mode and 32 refreshes in 4X mode. If using PHY-initiated updates, care must be taken in the setting of RFSHCTL0.refresh_burst, to ensure that tRFCmax is not violated due to a PHY-initiated update occurring shortly before a refresh burst was due. In this situation, the refresh burst will be delayed until the PHY-initiated update is complete.
3 —	Reserved
2 per_bank_refresh	Per bank refresh allows traffic to flow to other banks. Per bank refresh is not supported by all LPDDR2 devices but should be supported by all LPDDR3/LPDDR4 devices. Present only in designs configured to support LPDDR2/LPDDR3/LPDDR4 0b - All bank refresh 1b - Per bank refresh
1-0 —	Reserved

9.3.3.1.16 Refresh Control Register 1 (RFSHCTL1)

9.3.3.1.16.1 Offset

Register	Offset
RFSHCTL1	54h

9.3.3.1.16.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved								refresh_timer1_start_value_x32							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								refresh_timer0_start_value_x32							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.3.3.1.16.3 Fields

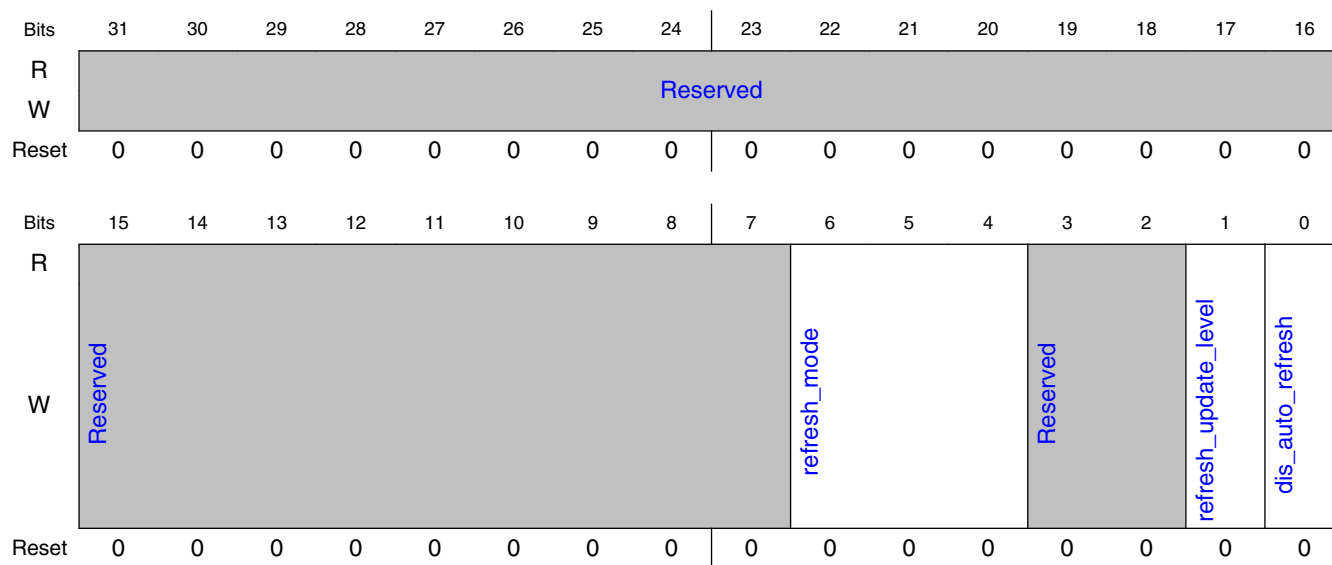
Field	Function
31-28 —	Reserved
27-16 refresh_timer1_start_value_x32	Refresh timer start for rank 1 (only present in multi-rank configurations). This is useful in staggering the refreshes to multiple ranks to help traffic to proceed. This is explained in Refresh Controls section of architecture chapter. Unit: Multiples of 32 DFI clock cycles. FOR PERFORMANCE ONLY.
15-12 —	Reserved
11-0 refresh_timer0_start_value_x32	Refresh timer start for rank 0 (only present in multi-rank configurations). This is useful in staggering the refreshes to multiple ranks to help traffic to proceed. This is explained in Refresh Controls section of architecture chapter. Unit: Multiples of 32 DFI clock cycles. FOR PERFORMANCE ONLY.

9.3.3.1.17 Refresh Control Register 3 (RFSHCTL3)

9.3.3.1.17.1 Offset

Register	Offset
RFSHCTL3	60h

9.3.3.1.17.2 Diagram



9.3.3.1.17.3 Fields

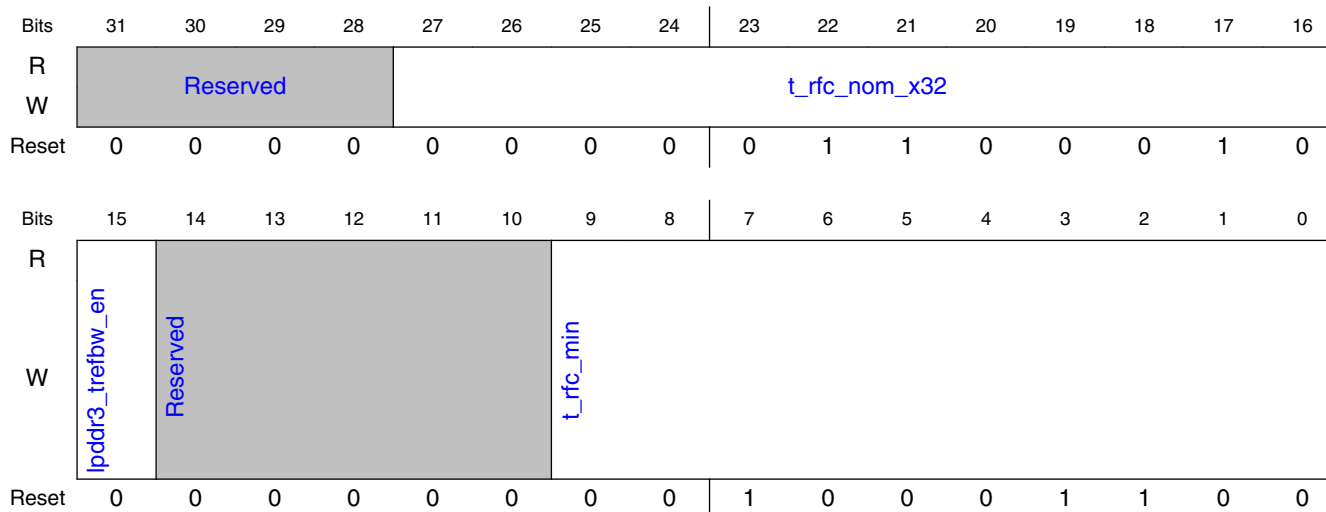
Field	Function
31-7 —	Reserved
6-4 refresh_mode	Fine Granularity Refresh Mode - 000 - Fixed 1x (Normal mode) - 001 - Fixed 2x - 010 - Fixed 4x - 101 - Enable on the fly 2x (not supported) - 110 - Enable on the fly 4x (not supported) - Everything else - reserved Note: Only Fixed 1x mode is supported if RFSHCTL3.dis_auto_refresh = 1. Note: The on-the-fly modes are not supported in this version of the DDRC. Note: This must be set up while the Controller is in reset or while the Controller is in self-refresh mode. Changing this during normal operation is not allowed. Making this a dynamic register will be supported in future version of the DDRC. Note: This register field has effect only if a DDR4 SDRAM device is in use (MSTR.ddr4 = 1).
3-2 —	Reserved
1 refresh_update_level	Toggle this signal (either from 0 to 1 or from 1 to 0) to indicate that the refresh register(s) have been updated. refresh_update_level must not be toggled when the DDRC is in reset (core_ddrc_rstn = 0). The refresh register(s) are automatically updated when exiting reset.
0 dis_auto_refresh	When '1', disable auto-refresh generated by the DDRC. When auto-refresh is disabled, the SoC core must generate refreshes using the registers reg_ddrc_rank0_refresh, reg_ddrc_rank1_refresh, reg_ddrc_rank2_refresh and reg_ddrc_rank3_refresh. When dis_auto_refresh transitions from 0 to 1, any pending refreshes are immediately scheduled by the DDRC. If DDR4 CRC/parity retry is enabled (CRCPARCTL1.crc_parity_retry_enable = 1), disable auto-refresh is not supported, and this bit must be set to '0'. (DDR4 only) If FGR mode is enabled (RFSHCTL3.refresh_mode > 0), disable auto-refresh is not supported, and this bit must be set to '0'. This register field is changeable on the fly.

9.3.3.1.18 Refresh Timing Register (RFSHTMG)

9.3.3.1.18.1 Offset

Register	Offset
RFSHTMG	64h

9.3.3.1.18.2 Diagram



9.3.3.1.18.3 Fields

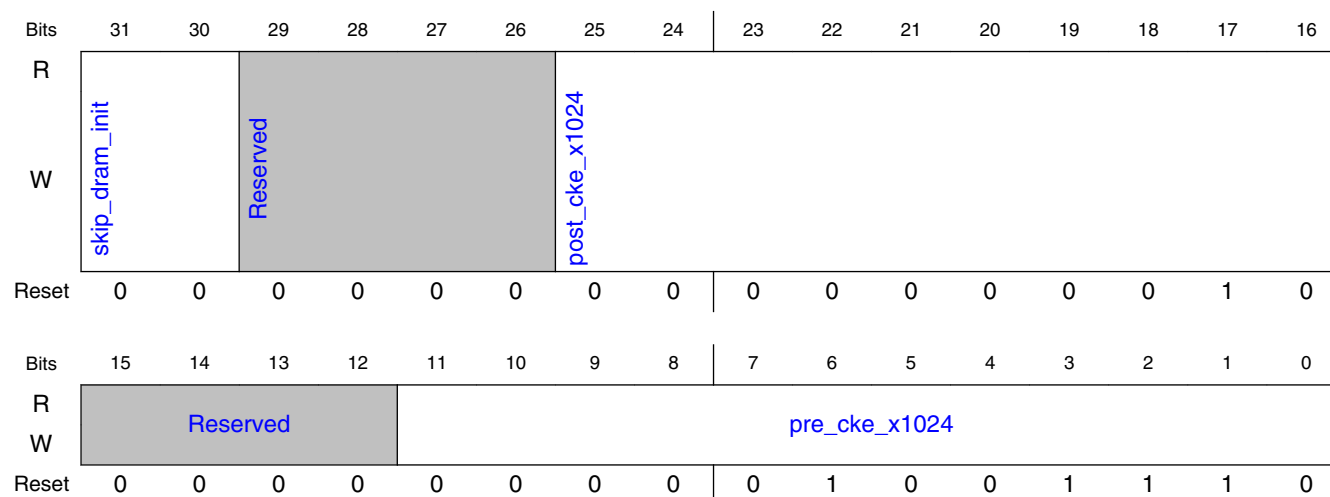
Field	Function
31-28 —	Reserved
27-16 t_rfc_nom_x32	tREFI: Average time interval between refreshes per rank (Specification: 7.8us for DDR2, DDR3 and DDR4. See JEDEC specification for mDDR, LPDDR2, LPDDR3 and LPDDR4). For LPDDR2/LPDDR3/LPDDR4: - if using all-bank refreshes (RFSHTMG.per_bank_refresh = 0), this register should be set to tREFIab - if using per-bank refreshes (RFSHTMG.per_bank_refresh = 1), this register should be set to tREFIpb When the controller is operating in 1:2 frequency ratio mode, program this to (tREFI/2), no rounding up. In DDR4 mode, tREFI value is different depending on the refresh mode. The user should program the appropriate value from the spec based on the value programmed in the refresh mode register. Note that RFSHTMG.t_rfc_nom_x32 * 32 must be greater than RFSHTMG.t_rfc_min, and RFSHTMG.t_rfc_nom_x32 must be greater than 0x1. - Non-DDR4 or DDR4 Fixed 1x mode: RFSHTMG.t_rfc_nom_x32 must be less than or equal to 0xFFE. - DDR4 Fixed 2x mode: RFSHTMG.t_rfc_nom_x32 must be less than or equal to 0x7FF. - DDR4 Fixed 4x mode: RFSHTMG.t_rfc_nom_x32 must be less than or equal to 0x3FF. Unit: Multiples of 32 clocks.
15 lpddr3_trefbw_en	Used only when LPDDR3 memory type is connected. Should only be changed when DDRC is in reset. Specifies whether to use the tREFBW parameter (required by some LPDDR3 devices which comply with earlier versions of the LPDDR3 JEDEC specification) or not: - 0 - tREFBW parameter not used - 1 - tREFBW parameter used
14-10 —	Reserved
9-0 t_rfc_min	tRFC (min): Minimum time from refresh to refresh or activate. When the controller is operating in 1:1 mode, t_rfc_min should be set to RoundUp(tRFCmin/tCK). When the controller is operating in 1:2 mode, t_rfc_min should be set to RoundUp(RoundUp(tRFCmin/tCK)/2). In LPDDR2/LPDDR3/LPDDR4 mode: - if using all-bank refreshes, the tRFCmin value in the above equations is equal to tRFCab - if using per-bank refreshes, the tRFCmin value in the above equations is equal to tRFCpb In DDR4 mode, the tRFCmin value in the above equations is different depending on the refresh mode (fixed 1X,2X,4X) and the device density. The user should program the appropriate value from the spec based on the 'refresh_mode' and the device density that is used. Unit: Clocks.

9.3.3.1.19 SDRAM Initialization Register 0 (INIT0)

9.3.3.1.19.1 Offset

Register	Offset
INIT0	D0h

9.3.3.1.19.2 Diagram



9.3.3.1.19.3 Fields

Field	Function
31-30 skip_dram_init	If lower bit is enabled the SDRAM initialization routine is skipped. The upper bit decides what state the controller starts up in when reset is removed - 00 - SDRAM Initialization routine is run after power-up - 01 - SDRAM Initialization routine is skipped after power-up. Controller starts up in Normal Mode - 11 - SDRAM Initialization routine is skipped after power-up. Controller starts up in Self-refresh Mode - 10 - SDRAM Initialization routine is run after power-up. 00b - SDRAM Initialization routine is run after power-up 01b - SDRAM Initialization routine is skipped after power-up 10b - SDRAM Initialization routine is run after power-up 11b - SDRAM Initialization routine is skipped after power-up
29-26 —	Reserved
25-16 post_cke_x1024	Cycles to wait after driving CKE high to start the SDRAM initialization sequence. Unit: 1024 DFI clock cycles. DDR2 typically requires a 400 ns delay, requiring this value to be programmed to 2 at all clock speeds. LPDDR2/LPDDR3 typically requires this to be programmed for a delay of 200 us. LPDDR4 typically requires this to be programmed for a delay of 2 us. When the controller is operating in 1:2 frequency ratio mode, program this to JEDEC spec value divided by 2, and round it up to the next integer value.

Table continues on the next page...

DDR Controller (DDRC)

Field	Function
15-12 —	Reserved
11-0 pre_cke_x1024	Cycles to wait after reset before driving CKE high to start the SDRAM initialization sequence. Unit: 1024 DFI clock cycles. DDR2 specifications typically require this to be programmed for a delay of ≥ 200 us. LPDDR2/LPDDR3: tINIT1 of 100 ns (min) LPDDR4: tINIT3 of 2 ms (min) When the controller is operating in 1:2 frequency ratio mode, program this to JEDEC spec value divided by 2, and round it up to the next integer value. For DDR3/DDR4 RDIMMs, this should include the time needed to satisfy tSTAB

9.3.3.1.20 SDRAM Initialization Register 1 (INIT1)

9.3.3.1.20.1 Offset

Register	Offset
INIT1	D4h

9.3.3.1.20.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved								dram_rstn_x1024							
W	Reserved								dram_rstn_x1024							
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												pre_ocrd_x32			
W	Reserved												pre_ocrd_x32			
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.3.3.1.20.3 Fields

Field	Function
31-25 —	Reserved
24-16 dram_rstn_x1024	Number of cycles to assert SDRAM reset signal during init sequence. This is only present for designs supporting DDR3, DDR4 or LPDDR4 devices. For use with a DDR PHY, this should be set to a minimum of 1. When the controller is operating in 1:2 frequency ratio mode, program this to JEDEC spec value divided by 2, and round it up to the next integer value. Unit: 1024 DFI clock cycles.
15-4 —	Reserved

Table continues on the next page...

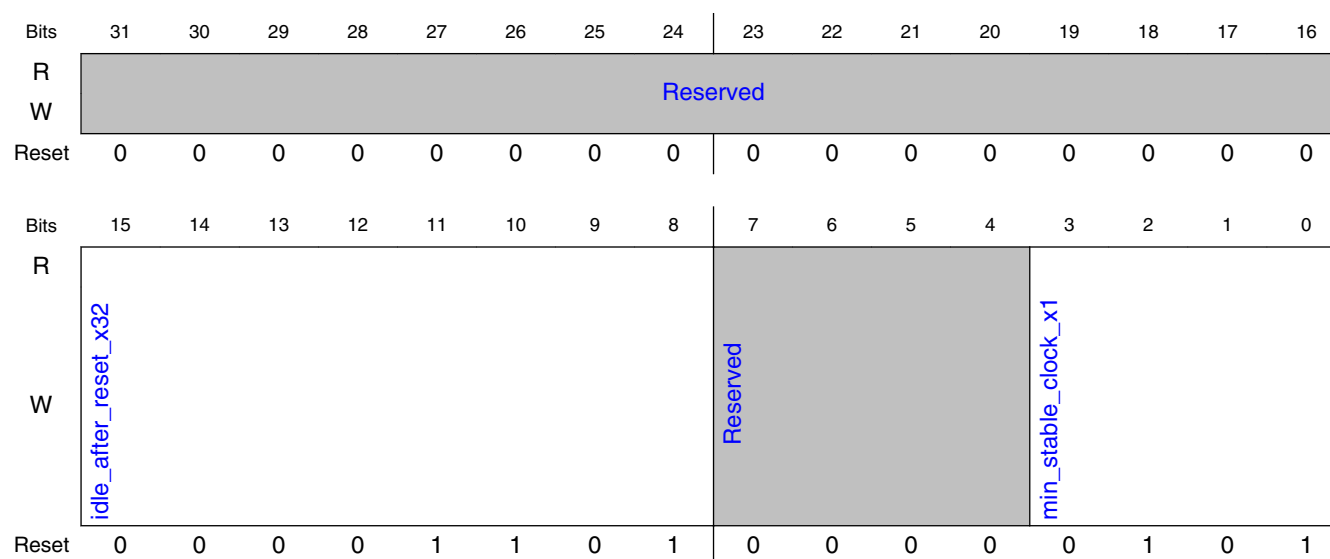
Field	Function
3-0 pre_oed_x32	Wait period before driving the OCD complete command to SDRAM. Unit: Counts of a global timer that pulses every 32 DFI clock cycles. There is no known specific requirement for this; it may be set to zero.

9.3.3.1.21 SDRAM Initialization Register 2 (INIT2)

9.3.3.1.21.1 Offset

Register	Offset
INIT2	D8h

9.3.3.1.21.2 Diagram



9.3.3.1.21.3 Fields

Field	Function
31-16 —	Reserved
15-8 idle_after_reset_x32	Idle time after the reset command, tINIT4. Present only in designs configured to support LPDDR2. When the controller is operating in 1:2 frequency ratio mode, program this to JEDEC spec value divided by 2, and round it up to the next integer value. Unit: 32 DFI clock cycles.
7-4 —	Reserved

Table continues on the next page...

DDR Controller (DDRC)

Field	Function
3-0 min_stable_clock_x1	Time to wait after the first CKE high, tINIT2. Present only in designs configured to support LPDDR2/LPDDR3. LPDDR2/LPDDR3 typically requires 5 x tCK delay. When the controller is operating in 1:2 frequency ratio mode, program this to JEDEC spec value divided by 2, and round it up to the next integer value. Unit: DFI clock cycles.

9.3.3.1.22 SDRAM Initialization Register 3 (INIT3)

9.3.3.1.22.1 Offset

Register	Offset
INIT3	DCh

9.3.3.1.22.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	mr															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	emr															
W																
Reset	0	0	0	0	0	1	0	1	0	0	0	1	0	0	0	0

9.3.3.1.22.3 Fields

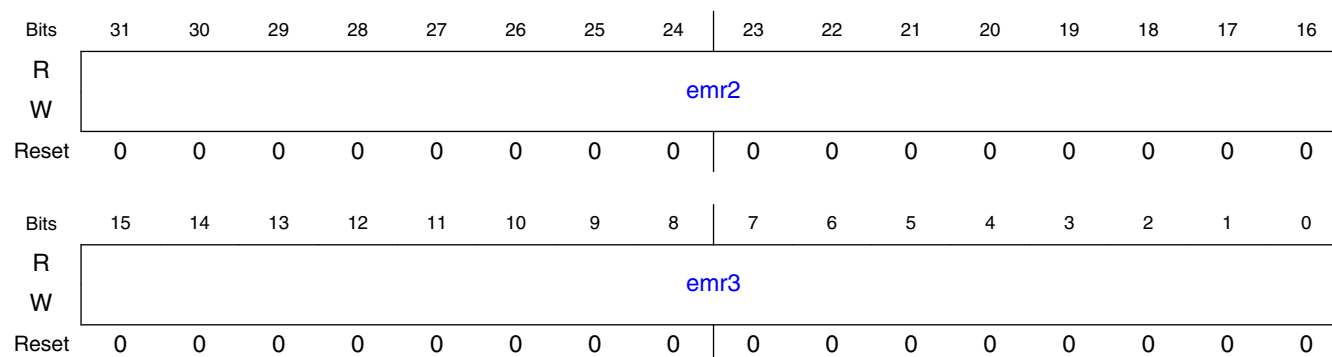
Field	Function
31-16 mr	DDR2: Value to write to MR register. Bit 8 is for DLL and the setting here is ignored. The DDRC sets this bit appropriately. DDR3/DDR4: Value loaded into MR0 register. mDDR: Value to write to MR register. LPDDR2/LPDDR3/LPDDR4 - Value to write to MR1 register
15-0 emr	DDR2: Value to write to EMR register. Bits 9:7 are for OCD and the setting in this register is ignored. The DDRC sets those bits appropriately. DDR3/DDR4: Value to write to MR1 register Set bit 7 to 0. If PHY-evaluation mode training is enabled, this bit is set appropriately by the DDRC during write leveling. mDDR: Value to write to EMR register. LPDDR2/LPDDR3/LPDDR4 - Value to write to MR2 register

9.3.3.1.23 SDRAM Initialization Register 4 (INIT4)

9.3.3.1.23.1 Offset

Register	Offset
INIT4	E0h

9.3.3.1.23.2 Diagram



9.3.3.1.23.3 Fields

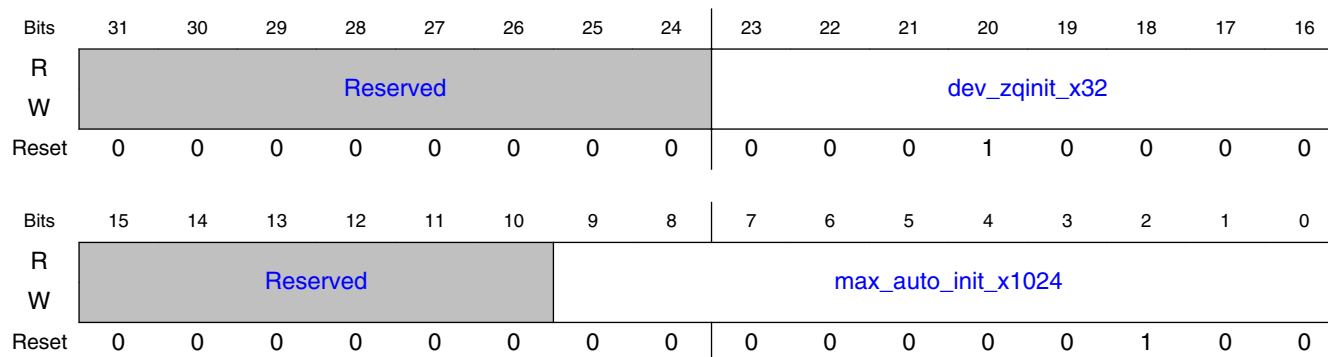
Field	Function
31-16 emr2	DDR2: Value to write to EMR2 register. DDR3/DDR4: Value to write to MR2 register LPDDR2/LPDDR3/LPDDR4: Value to write to MR3 register mDDR: Unused
15-0 emr3	DDR2: Value to write to EMR3 register. DDR3/DDR4: Value to write to MR3 register mDDR/LPDDR2/LPDDR3: Unused LPDDR4: Value to write to MR13 register

9.3.3.1.24 SDRAM Initialization Register 5 (INIT5)

9.3.3.1.24.1 Offset

Register	Offset
INIT5	E4h

9.3.3.1.24.2 Diagram



9.3.3.1.24.3 Fields

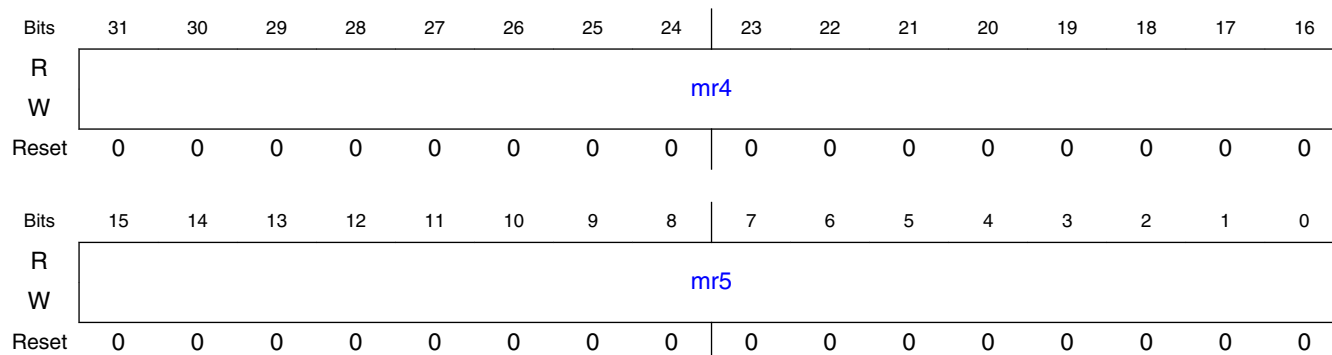
Field	Function
31-24 —	Reserved
23-16 dev_zqinit_x32	ZQ initial calibration, tZQINIT. Present only in designs configured to support DDR3 or DDR4 or LPDDR2/LPDDR3. DDR3 typically requires 512 SDRAM clock cycles. DDR4 requires 1024 SDRAM clock cycles. LPDDR2/LPDDR3 requires 1 us. When the controller is operating in 1:2 frequency ratio mode, program this to JEDEC spec value divided by 2, and round it up to the next integer value. Unit: 32 DFI clock cycles.
15-10 —	Reserved
9-0 max_auto_init_x1024	Maximum duration of the auto initialization, tINIT5. Present only in designs configured to support LPDDR2/LPDDR3. LPDDR2/LPDDR3 typically requires 10 us. Unit: 1024 DFI clock cycles.

9.3.3.1.25 SDRAM Initialization Register 6 (INIT6)

9.3.3.1.25.1 Offset

Register	Offset
INIT6	E8h

9.3.3.1.25.2 Diagram



9.3.3.1.25.3 Fields

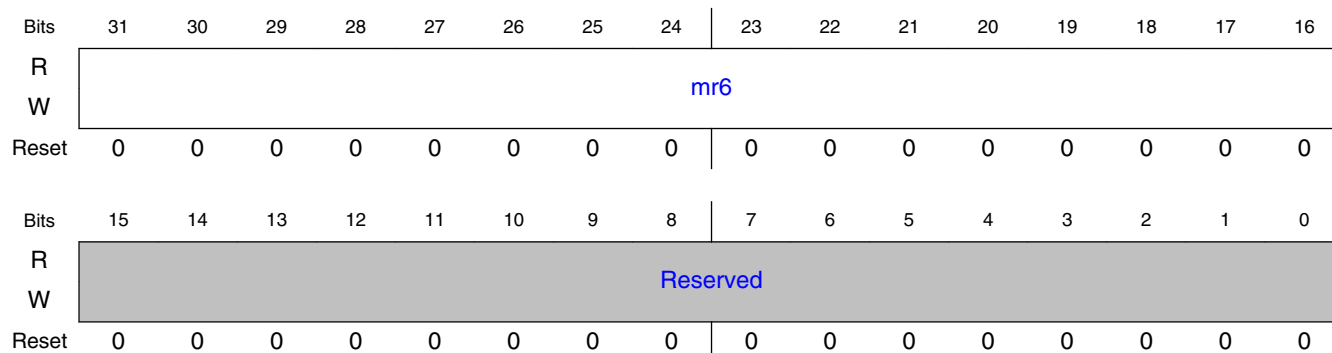
Field	Function
31-16 mr4	DDR4- Value to be loaded into SDRAM MR4 registers. Used in DDR4 designs only.
15-0 mr5	DDR4- Value to be loaded into SDRAM MR5 registers. Used in DDR4 designs only.

9.3.3.1.26 SDRAM Initialization Register 7 (INIT7)

9.3.3.1.26.1 Offset

Register	Offset
INIT7	ECh

9.3.3.1.26.2 Diagram



9.3.3.1.26.3 Fields

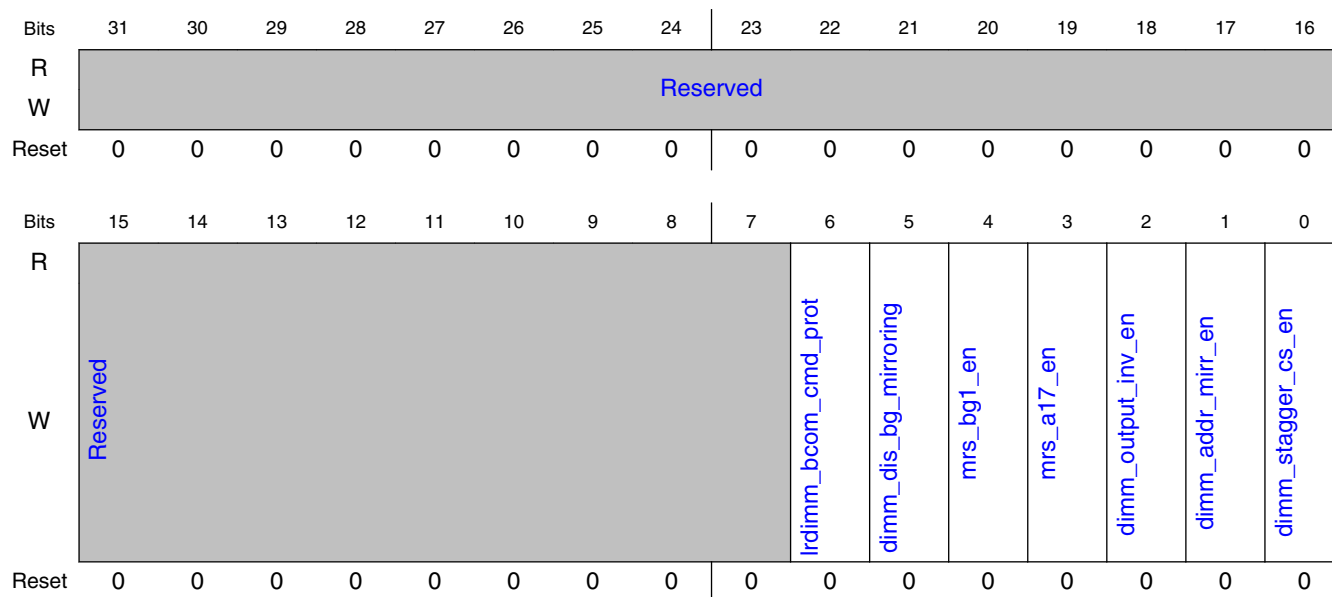
Field	Function
31-16 mr6	DDR4- Value to be loaded into SDRAM MR6 registers. Used in DDR4 designs only.
15-0 —	Reserved

9.3.3.1.27 DIMM Control Register (DIMMCTL)

9.3.3.1.27.1 Offset

Register	Offset
DIMMCTL	F0h

9.3.3.1.27.2 Diagram



9.3.3.1.27.3 Fields

Field	Function
31-7	Reserved

Table continues on the next page...

Field	Function
—	
6 lrdimm_bcom_cmd_prot	Protects the timing restrictions (tBCW/tMRC) between consecutive BCOM commands defined in the Data Buffer specification. When using DDR4 LRDIMM, this bit must be set to 1. Otherwise, this bit must be set to 0.
5 dimm_dis_bg_mirroring	Disabling Address Mirroring for BG bits. When this is set to 1, BG0 and BG1 are NOT swapped even if Address Mirroring is enabled. This will be required for DDR4 DIMMs with x16 devices. 0b - BG0 and BG1 are swapped if address mirroring is enabled. 1b - BG0 and BG1 are NOT swapped.
4 mrs_bg1_en	Enable for BG1 bit of MRS command. BG1 bit of the mode register address is specified as RFU (Reserved for Future Use) and must be programmed to 0 during MRS. In case where DRAMs which do not have BG1 are attached and both the CA parity and the Output Inversion are enabled, this must be set to 0, so that the calculation of CA parity will not include BG1 bit. Note: This has no effect on the address of any other memory accesses, or of software-driven mode register accesses. If address mirroring is enabled, this is applied to BG1 of even ranks and BG0 of odd ranks. 0b - Disabled 1b - Enabled
3 mrs_a17_en	Enable for A17 bit of MRS command. A17 bit of the mode register address is specified as RFU (Reserved for Future Use) and must be programmed to 0 during MRS. In case where DRAMs which do not have A17 are attached and the Output Inversion are enabled, this must be set to 0, so that the calculation of CA parity will not include A17 bit. Note: This has no effect on the address of any other memory accesses, or of software-driven mode register accesses. 0b - Disabled 1b - Enabled
2 dimm_output_inv_en	Output Inversion Enable (for DDR4 RDIMM/LRDIMM implementations only). DDR4 RDIMM/LRDIMM implements the Output Inversion feature by default, which means that the following address, bank address and bank group bits of B-side DRAMs are inverted: A3-A9, A11, A13, A17, BA0-BA1, BG0-BG1. Setting this bit ensures that, for mode register accesses generated by the DDRC during the automatic initialization routine and enabling of a particular DDR4 feature, separate A-side and B-side mode register accesses are generated. For B-side mode register accesses, these bits are inverted within the DDRC to compensate for this RDIMM/LRDIMM inversion. It is recommended to set this bit always, if using DDR4 RDIMMs/LRDIMMs. Note: This has no effect on the address of any other memory accesses, or of software-driven mode register accesses. 0b - Do not implement output inversion for B-side DRAMs. 1b - Implement output inversion for B-side DRAMs.
1 dimm_addr_mirroring_en	Address Mirroring Enable (for multi-rank UDIMM implementations and multi-rank DDR4 RDIMM/LRDIMM implementations). Some UDIMMs and DDR4 RDIMMs/LRDIMMs implement address mirroring for odd ranks, which means that the following address, bank address and bank group bits are swapped: (A3, A4), (A5, A6), (A7, A8), (BA0, BA1) and also (A11, A13), (BG0, BG1) for the DDR4. Setting this bit ensures that, for mode register accesses during the automatic initialization routine, these bits are swapped within the DDRC to compensate for this UDIMM/RDIMM/LRDIMM swapping. In addition to the automatic initialization routine, in case of DDR4 UDIMM/RDIMM/LRDIMM, they are swapped during the automatic MRS access to enable/disable of a particular DDR4 feature. Note: This has no effect on the address of any other memory accesses, or of software-driven mode register accesses. This is not supported for mDDR, LPDDR2, LPDDR3 or LPDDR4 SDRAMs. Note: In case of x16 DDR4 DIMMs, BG1 output of MRS for the odd ranks is same as BG0 because BG1 is invalid, hence dimm_dis_bg_mirroring register must be set to 1. 0b - Do not implement address mirroring 1b - For odd ranks, implement address mirroring for MRS commands to during initialization and for any automatic DDR4 MRS commands (to be used if UDIMM/RDIMM/LRDIMM implements address mirroring)
0 dimm_stagger_cmds_en	Staggering enable for multi-rank accesses (for multi-rank UDIMM, RDIMM and LRDIMM implementations only). This is not supported for mDDR, LPDDR2, LPDDR3 or LPDDR4 SDRAMs. Even if this bit is set it does not take care of software driven MR commands (via MRCTRL0/MRCTRL1), where software is responsible to send them to separate ranks as appropriate.

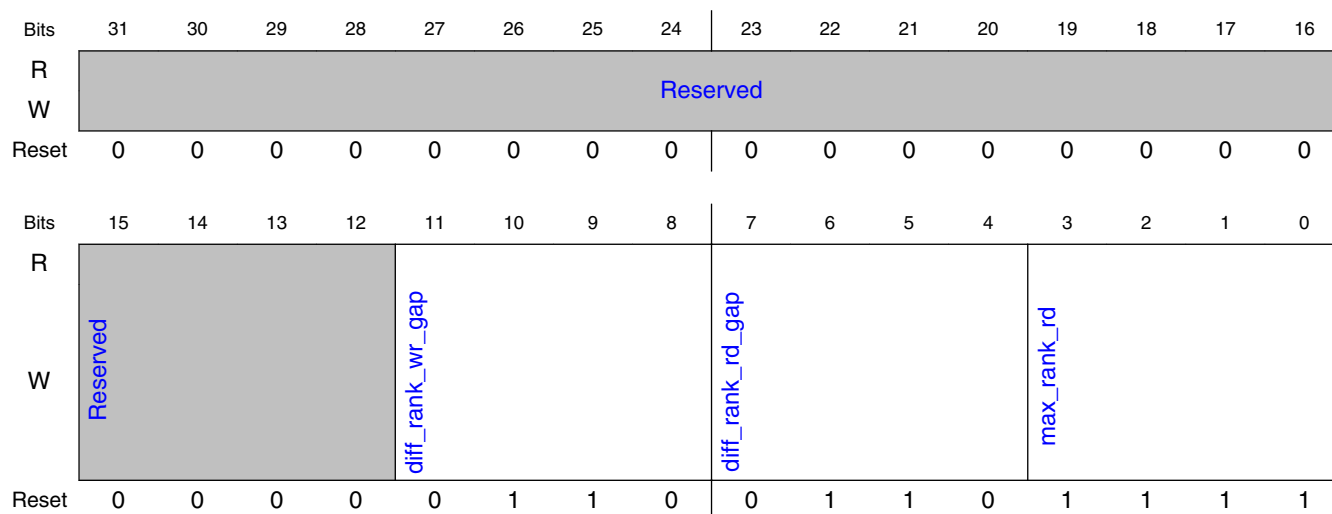
Field	Function
	0b - Do not stagger accesses 1b - (non-DDR4) Send all commands to even and odd ranks separately

9.3.3.1.28 Rank Control Register (RANKCTL)

9.3.3.1.28.1 Offset

Register	Offset
RANKCTL	F4h

9.3.3.1.28.2 Diagram



9.3.3.1.28.3 Fields

Field	Function
31-12 —	Reserved
11-8 diff_rank_wr_gap	Only present for multi-rank configurations. Indicates the number of clocks of gap in data responses when performing consecutive writes to different ranks. This is used to switch the delays in the PHY to match the rank requirements. This value should consider both PHY requirement and ODT requirement. - PHY requirement: tphy_wrcsgap + 1 (see PHY databook for value of tphy_wrcsgap) If CRC feature is enabled, should be increased by 1. If write preamble is set to 2tCK(DDR4/LPDDR4 only), should be increased by 1. If write postamble is set to 1.5tCK(LPDDR4 only), should be increased by 1. - ODT requirement: The value programmed in this register takes care of the ODT switch off timing requirement when switching ranks during writes. For LPDDR4, the requirement is ODTLoff - ODTLon - BL/2 + 1 When the controller is

Table continues on the next page...

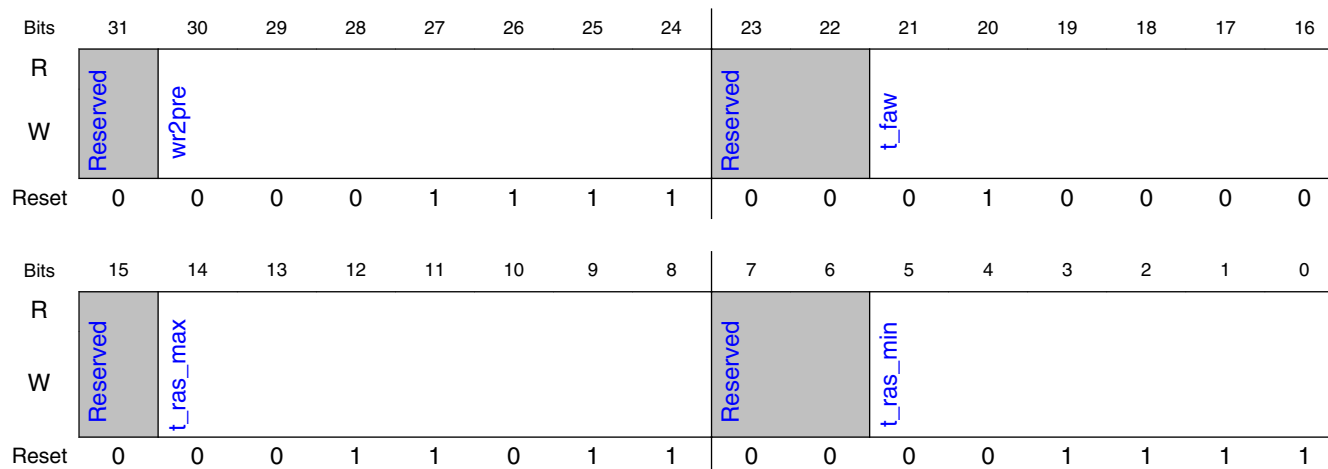
Field	Function
	operating in 1:1 mode, program this to the larger of PHY requirement or ODT requirement. When the controller is operating in 1:2 mode, program this to the larger value divided by two and round it up to the next integer. Note that, if using DDR4-LRDIMM, refer to TWRWR timing requirements in JEDEC DDR4 Data Buffer (DDR4DB01) Specification.
7-4 diff_rank_rd_gap	Only present for multi-rank configurations. Indicates the number of clocks of gap in data responses when performing consecutive reads to different ranks. This is used to switch the delays in the PHY to match the rank requirements. This value should consider both PHY requirement and ODT requirement. - PHY requirement: $t_{phy_rdcsgap} + 1$ (see PHY databook for value of $t_{phy_rdcsgap}$) If read preamble is set to $2t_{CK}$ (DDR4/LPDDR4 only), should be increased by 1. If read postamble is set to $1.5t_{CK}$ (LPDDR4 only), should be increased by 1. - ODT requirement: The value programmed in this register takes care of the ODT switch off timing requirement when switching ranks during reads. When the controller is operating in 1:1 mode, program this to the larger of PHY requirement or ODT requirement. When the controller is operating in 1:2 mode, program this to the larger value divided by two and round it up to the next integer. Note that, if using DDR4-LRDIMM, refer to TRDRD timing requirements in JEDEC DDR4 Data Buffer (DDR4DB01) Specification.
3-0 max_rank_rd	Only present for multi-rank configurations. Background: Reads to the same rank can be performed back-to-back. Reads to different ranks require additional gap dictated by the register RANKCTL.diff_rank_rd_gap. This is to avoid possible data bus contention as well as to give PHY enough time to switch the delay when changing ranks. The DDRC arbitrates for bus access on a cycle-by-cycle basis; therefore after a read is scheduled, there are few clock cycles (determined by the value on RANKCTL.diff_rank_rd_gap register) in which only reads from the same rank are eligible to be scheduled. This prevents reads from other ranks from having fair access to the data bus. This parameter represents the maximum number of reads that can be scheduled consecutively to the same rank. After this number is reached, a delay equal to RANKCTL.diff_rank_rd_gap is inserted by the scheduler to allow all ranks a fair opportunity to be scheduled. Higher numbers increase bandwidth utilization, lower numbers increase fairness. This feature can be DISABLED by setting this register to 0. When set to 0, the Controller will stay on the same rank as long as commands are available for it. Minimum programmable value is 0 (feature disabled) and maximum programmable value is 0xF. FOR PERFORMANCE ONLY.

9.3.3.1.29 SDRAM Timing Register 0 (DRAMTMG0)

9.3.3.1.29.1 Offset

Register	Offset
DRAMTMG0	100h

9.3.3.1.29.2 Diagram



9.3.3.1.29.3 Fields

Field	Function
31 —	Reserved
30-24 wr2pre	Minimum time between write and precharge to same bank. Unit: Clocks Specifications: $WL + BL/2 + tWR$ = approximately 8 cycles + 15 ns = 14 clocks @400MHz and less for lower frequencies where: - WL = write latency - BL = burst length. This must match the value programmed in the BL bit of the mode register to the SDRAM. BST (burst terminate) is not supported at present. - tWR = Write recovery time. This comes directly from the SDRAM specification. Add one extra cycle for LPDDR2/LPDDR3/LPDDR4 for this parameter. When the controller is operating in 1:2 frequency ratio mode, 1T mode, divide the above value by 2. No rounding up. When the controller is operating in 1:2 frequency ratio mode, 2T mode or LPDDR4 mode, divide the above value by 2 and round it up to the next integer value. Note that, depending on the PHY, if using LRDIMM, it may be necessary to adjust the value of this parameter to compensate for the extra cycle of latency through the LRDIMM.
23-22 —	Reserved
21-16 t_faw	tFAW Valid only when 8 or more banks(or banks x bank groups) are present. In 8-bank design, at most 4 banks must be activated in a rolling window of tFAW cycles. When the controller is operating in 1:2 frequency ratio mode, program this to (tFAW/2) and round up to next integer value. In a 4-bank design, set this register to 0x1 independent of the 1:1/1:2 frequency mode. Unit: Clocks
15 —	Reserved
14-8 t_ras_max	tRAS(max): Maximum time between activate and precharge to same bank. This is the maximum time that a page can be kept open Minimum value of this register is 1. Zero is invalid. When the controller is operating in 1:2 frequency ratio mode, program this to (tRAS(max)-1)/2. No rounding up. Unit: Multiples of 1024 clocks.
7-6 —	Reserved
5-0 t_ras_min	tRAS(min): Minimum time between activate and precharge to the same bank. When the controller is operating in 1:2 frequency mode, 1T mode, program this to tRAS(min)/2. No rounding up. When the

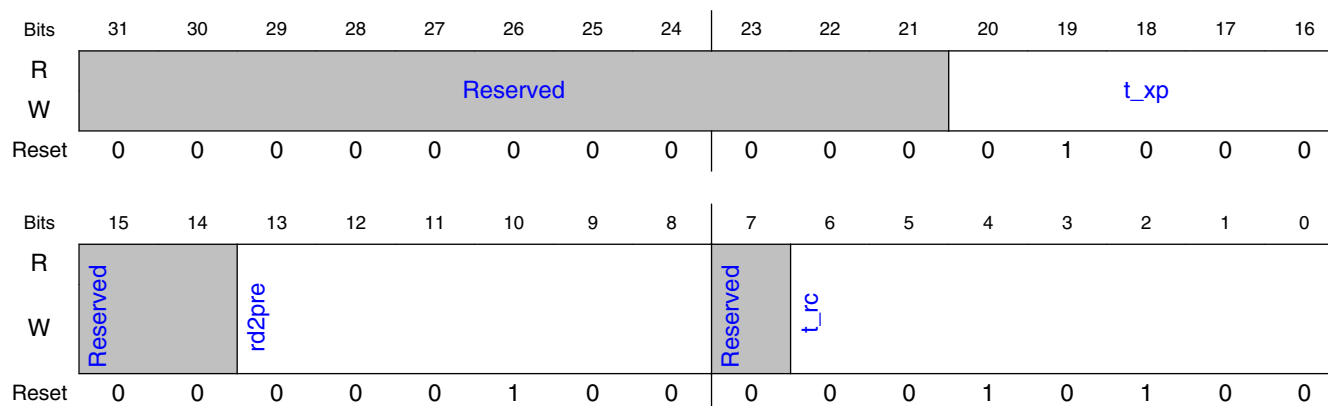
Field	Function
	controller is operating in 1:2 frequency ratio mode, 2T mode or LPDDR4 mode, program this to (tRAS(min)/2) and round it up to the next integer value. Unit: Clocks

9.3.3.1.30 SDRAM Timing Register 1 (DRAMTMG1)

9.3.3.1.30.1 Offset

Register	Offset
DRAMTMG1	104h

9.3.3.1.30.2 Diagram



9.3.3.1.30.3 Fields

Field	Function
31-21 —	Reserved
20-16 t _{xp}	tXP: Minimum time after power-down exit to any operation. For DDR3, this should be programmed to tXPDLL if slow powerdown exit is selected in MR0[12]. If C/A parity for DDR4 is used, set to (tXP+PL) instead. When the controller is operating in 1:2 frequency ratio mode, program this to (tXP/2) and round it up to the next integer value. Units: Clocks
15-14 —	Reserved
13-8 rd2pre	tRTP: Minimum time from read to precharge of same bank. - DDR2: tAL + BL/2 + max(tRTP, 2) - 2 - DDR3: tAL + max (tRTP, 4) - DDR4: Max of following two equations: tAL + max (tRTP, 4) or, RL + BL/2 - tRP (*). - mDDR: BL/2 - LPDDR2: Depends on if it's LPDDR2-S2 or LPDDR2-S4: LPDDR2-S2: BL/2 + tRTP - 1. LPDDR2-S4: BL/2 + max(tRTP,2) - 2. - LPDDR3: BL/2 + max(tRTP,4) - 4 - LPDDR4: BL/2 + max(tRTP,8) - 8 (*) When both DDR4 SDRAM and ST-MRAM are used simultaneously, use SDRAM's

Table continues on the next page...

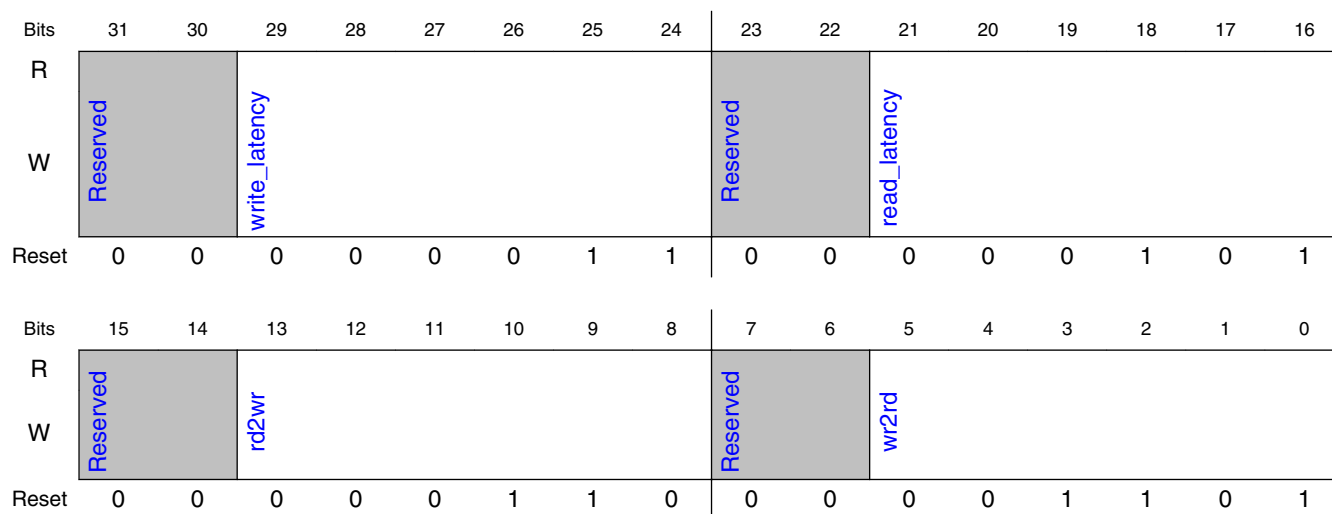
Field	Function
	tRP value for calculation. When the controller is operating in 1:2 mode, 1T mode, divide the above value by 2. No rounding up. When the controller is operating in 1:2 mode, 2T mode or LPDDR4 mode, divide the above value by 2 and round it up to the next integer value. Unit: Clocks.
7 —	Reserved
6-0 t_rc	tRC: Minimum time between activates to same bank. When the controller is operating in 1:2 frequency ratio mode, program this to (tRC/2) and round up to next integer value. Unit: Clocks.

9.3.3.1.31 SDRAM Timing Register 2 (DRAMTMG2)

9.3.3.1.31.1 Offset

Register	Offset
DRAMTMG2	108h

9.3.3.1.31.2 Diagram



9.3.3.1.31.3 Fields

Field	Function
31-30 —	Reserved

Table continues on the next page...

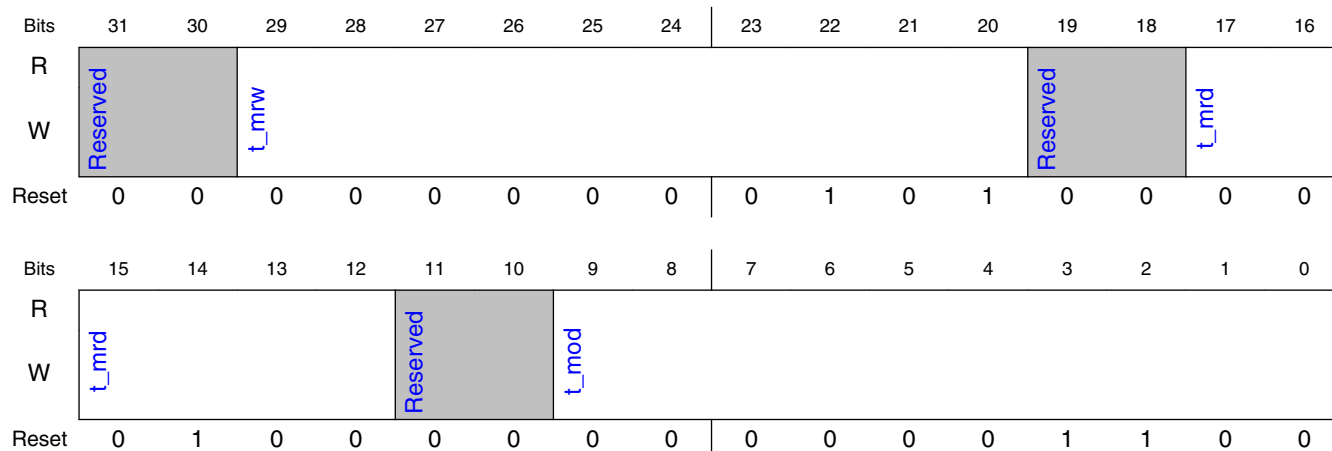
Field	Function
29-24 write_latency	Set to WL Time from write command to write data on SDRAM interface. This must be set to WL. For mDDR, it should normally be set to 1. Note that, depending on the PHY, if using RDIMM/LRDIMM, it may be necessary to adjust the value of WL to compensate for the extra cycle of latency through the RDIMM/LRDIMM. When the controller is operating in 1:2 frequency ratio mode, divide the value calculated using the above equation by 2, and round it up to next integer. This register field is not required for DDR2 and DDR3 (except if MEMC_TRAINING is set), as the DFI read and write latencies defined in DFITMG0 and DFITMG1 are sufficient for those protocols Unit: clocks
23-22 —	Reserved
21-16 read_latency	Set to RL Time from read command to read data on SDRAM interface. This must be set to RL. Note that, depending on the PHY, if using RDIMM/LRDIMM, it may be necessary to adjust the value of RL to compensate for the extra cycle of latency through the RDIMM/LRDIMM. When the controller is operating in 1:2 frequency ratio mode, divide the value calculated using the above equation by 2, and round it up to next integer. This register field is not required for DDR2 and DDR3 (except if MEMC_TRAINING is set), as the DFI read and write latencies defined in DFITMG0 and DFITMG1 are sufficient for those protocols Unit: clocks
15-14 —	Reserved
13-8 rd2wr	DDR2/3/mDDR: $RL + BL/2 + 2 - WL$ DDR4: $RL + BL/2 + 1 + WR_PREAMBLE - WL$ LPDDR2/LPDDR3: $RL + BL/2 + RU(tDQSCKmax/tCK) + 1 - WL$ LPDDR4(DQ ODT is Disabled): $RL + BL/2 + RU(tDQSCKmax/tCK) + WR_PREAMBLE + RD_POSTAMBLE - WL$ LPDDR4(DQ ODT is Enabled) : $RL + BL/2 + RU(tDQSCKmax/tCK) + RD_POSTAMBLE - ODTLon - RU(tODTon(min)/tCK)$ Minimum time from read command to write command. Include time for bus turnaround and all per-bank, per-rank, and global constraints. Please see the relevant PHY databook for details of what should be included here. Unit: Clocks. Where: - WL = write latency - BL = burst length. This must match the value programmed in the BL bit of the mode register to the SDRAM - RL = read latency = CAS latency - WR_PREAMBLE = write preamble. This is unique to DDR4 and LPDDR4. - RD_POSTAMBLE = read postamble. This is unique to LPDDR4. For LPDDR2/LPDDR3/LPDDR4, if derating is enabled (DERATEEN.derate_enable=1), derated tDQSCKmax should be used. When the controller is operating in 1:2 frequency ratio mode, divide the value calculated using the above equation by 2, and round it up to next integer. Note that, depending on the PHY, if using LRDIMM, it may be necessary to adjust the value of this parameter to compensate for the extra cycle of latency through the LRDIMM.
7-6 —	Reserved
5-0 wr2rd	DDR4: $CWL + PL + BL/2 + tWTR_L$ Others: $CWL + BL/2 + tWTR$ In DDR4, minimum time from write command to read command for same bank group. In others, minimum time from write command to read command. Includes time for bus turnaround, recovery times, and all per-bank, per-rank, and global constraints. Unit: Clocks. Where: - CWL = CAS write latency - PL = Parity latency - BL = burst length. This must match the value programmed in the BL bit of the mode register to the SDRAM - tWTR_L = internal write to read command delay for same bank group. This comes directly from the SDRAM specification. - tWTR = internal write to read command delay. This comes directly from the SDRAM specification. Add one extra cycle for LPDDR2/LPDDR3/LPDDR4 operation. When the controller is operating in 1:2 mode, divide the value calculated using the above equation by 2, and round it up to next integer.

9.3.3.1.32 SDRAM Timing Register 3 (DRAMTMG3)

9.3.3.1.32.1 Offset

Register	Offset
DRAMTMG3	10Ch

9.3.3.1.32.2 Diagram



9.3.3.1.32.3 Fields

Field	Function
31-30 —	Reserved
29-20 t_mrw	Time to wait after a mode register write or read (MRW or MRR). Present only in designs configured to support LPDDR2, LPDDR3 or LPDDR4. LPDDR2 typically requires value of 5. LPDDR3 typically requires value of 10. LPDDR4: Set this to the larger of tMRW and tMRWCKEL. For LPDDR2, this register is used for the time from a MRW/MRR to all other commands. When the controller is operating in 1:2 frequency ratio mode, program this to the above values divided by 2 and round it up to the next integer value. For LPDDR3, this register is used for the time from a MRW/MRR to a MRW/MRR.
19-18 —	Reserved
17-12 t_mrd	tMRD: Cycles to wait after a mode register write or read. Depending on the connected SDRAM, tMRD represents: DDR2/mDDR: Time from MRS to any command DDR3/4: Time from MRS to MRS command LPDDR2: not used LPDDR3/4: Time from MRS to non-MRS command. When the controller is operating in 1:2 frequency ratio mode, program this to (tMRD/2) and round it up to the next integer value. If C/A parity for DDR4 is used, set to tMRD_PAR(tMOD+PL) instead.
11-10 —	Reserved
9-0 t_mod	tMOD: Parameter used only in DDR3 and DDR4. Cycles between load mode command and following non-load mode command. If C/A parity for DDR4 is used, set to tMOD_PAR(tMOD+PL) instead. Set to tMOD if controller is operating in 1:1 frequency ratio mode, or tMOD/2 (rounded up to next integer) if controller is operating in 1:2 frequency ratio mode. Note that if using RDIMM/LRDIMM, depending on the

Field	Function
	PHY, it may be necessary to adjust the value of this parameter to compensate for the extra cycle of latency applied to mode register writes by the RDIMM/LRDIMM chip. Also note that if using LRDIMM, the minimum value of this register is tMRD_L2 if controller is operating in 1:1 frequency ratio mode, or tMRD_L2/2 (rounded up to next integer) if controller is operating in 1:2 frequency ratio mode.

9.3.3.1.33 SDRAM Timing Register 4 (DRAMTMG4)

9.3.3.1.33.1 Offset

Register	Offset
DRAMTMG4	110h

9.3.3.1.33.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved			t_rcd					Reserved				t_ccd			
W																
Reset	0	0	0	0	0	1	0	1	0	0	0	0	0	1	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				t_rrd				Reserved				t_rp			
W																
Reset	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0	1

9.3.3.1.33.3 Fields

Field	Function
31-29 —	Reserved
28-24 t_rcd	tRCD - tAL: Minimum time from activate to read or write command to same bank. When the controller is operating in 1:2 frequency ratio mode, program this to ((tRCD - tAL)/2) and round it up to the next integer value. Minimum value allowed for this register is 1, which implies minimum (tRCD - tAL) value to be 2 when the controller is operating in 1:2 frequency ratio mode. Unit: Clocks.
23-20 —	Reserved
19-16 t_ccd	DDR4: tCCD_L: This is the minimum time between two reads or two writes for same bank group. Others: tCCD: This is the minimum time between two reads or two writes. When the controller is operating in 1:2 frequency ratio mode, program this to (tCCD_L/2 or tCCD/2) and round it up to the next integer value. Unit: clocks.

Table continues on the next page...

DDR Controller (DDRC)

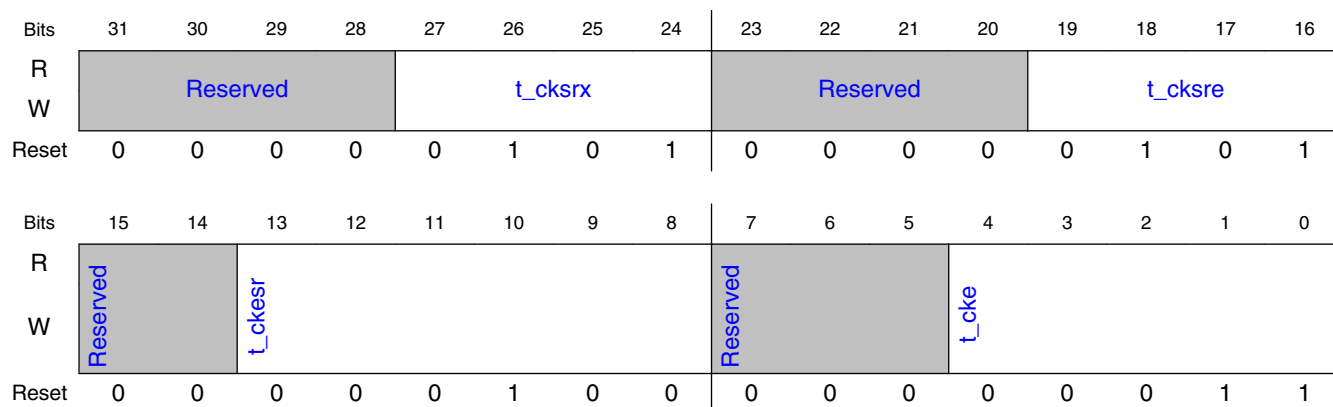
Field	Function
15-12 —	Reserved
11-8 t_rrd	DDR4: tRRD_L: Minimum time between activates from bank "a" to bank "b" for same bank group. Others: tRRD: Minimum time between activates from bank "a" to bank "b" When the controller is operating in 1:2 frequency ratio mode, program this to (tRRD_L/2 or tRRD/2) and round it up to the next integer value. Unit: Clocks.
7-5 —	Reserved
4-0 t_rp	tRP: Minimum time from precharge to activate of same bank. When the controller is operating in 1:1 frequency ratio mode, t_rp should be set to RoundUp(tRP/tCK). When the controller is operating in 1:2 frequency ratio mode, t_rp should be set to RoundDown(RoundUp(tRP/tCK)/2) + 1. When the controller is operating in 1:2 frequency ratio mode in LPDDR4, t_rp should be set to RoundUp(RoundUp(tRP/tCK)/2). Unit: Clocks.

9.3.3.1.34 SDRAM Timing Register 5 (DRAMTMG5)

9.3.3.1.34.1 Offset

Register	Offset
DRAMTMG5	114h

9.3.3.1.34.2 Diagram



9.3.3.1.34.3 Fields

Field	Function
31-28	Reserved

Table continues on the next page...

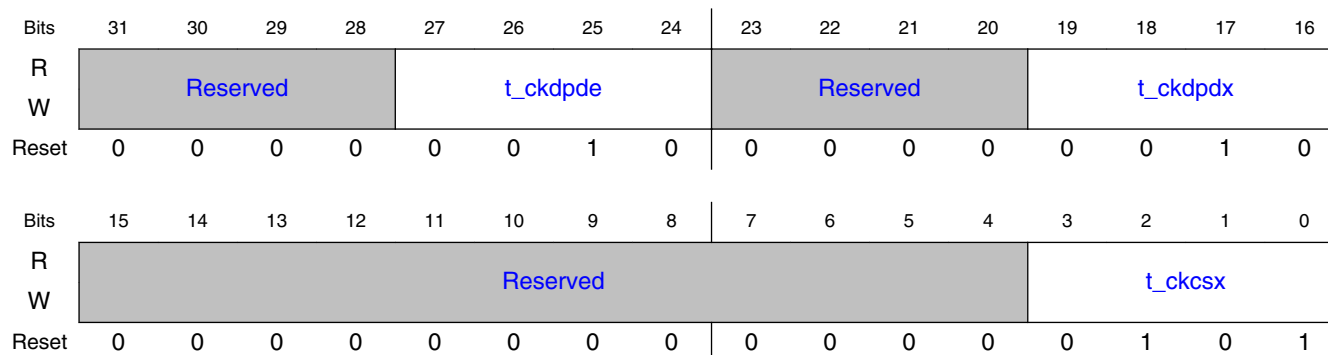
Field	Function
—	
27-24 t_cksrx	This is the time before Self Refresh Exit that CK is maintained as a valid clock before issuing SRX. Specifies the clock stable time before SRX. Recommended settings: - mDDR: 1 - LPDDR2: 2 - LPDDR3: 2 - LPDDR4: tCKCKEH - DDR2: 1 - DDR3: tCKSRX - DDR4: tCKSRX When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer.
23-20 —	Reserved
19-16 t_cksre	This is the time after Self Refresh Down Entry that CK is maintained as a valid clock. Specifies the clock disable delay after SRE. Recommended settings: - mDDR: 0 - LPDDR2: 2 - LPDDR3: 2 - LPDDR4: tCKCKEL - DDR2: 1 - DDR3: max (10 ns, 5 tCK) - DDR4: max (10 ns, 5 tCK) (+ PL(parity latency)(*)) (*)Only if CRCPARCTL1.caparity_disable_before_sr=0, this register should be increased by PL. When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer.
15-14 —	Reserved
13-8 t_ckesr	Minimum CKE low width for Self refresh or Self refresh power down entry to exit timing in memory clock cycles. Recommended settings: - mDDR: tRFC - LPDDR2: tCKESR - LPDDR3: tCKESR - LPDDR4: max(tCKELPD, tSR) - DDR2: tCKE - DDR3: tCKE + 1 - DDR4: tCKE + 1 (+ PL(parity latency)(*)) (*)Only if CRCPARCTL1.caparity_disable_before_sr=0, this register should be increased by PL. When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer.
7-5 —	Reserved
4-0 t_cke	Minimum number of cycles of CKE HIGH/LOW during power-down and self refresh. - LPDDR2/LPDDR3 mode: Set this to the larger of tCKE or tCKESR - LPDDR4 mode: Set this to the larger of tCKE, tCKELPD or tSR. - Non-LPDDR2/non-LPDDR3/non-LPDDR4 designs: Set this to tCKE value. When the controller is operating in 1:2 frequency ratio mode, program this to (value described above)/2 and round it up to the next integer value. Unit: Clocks.

9.3.3.1.35 SDRAM Timing Register 6 (DRAMTMG6)

9.3.3.1.35.1 Offset

Register	Offset
DRAMTMG6	118h

9.3.3.1.35.2 Diagram



9.3.3.1.35.3 Fields

Field	Function
31-28 —	Reserved
27-24 t_ckdpde	This is the time after Deep Power Down Entry that CK is maintained as a valid clock. Specifies the clock disable delay after DPDE. Recommended settings: - mDDR: 0 - LPDDR2: 2 - LPDDR3: 2 When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer. This is only present for designs supporting mDDR or LPDDR2/LPDDR3 devices.
23-20 —	Reserved
19-16 t_ckdpdx	This is the time before Deep Power Down Exit that CK is maintained as a valid clock before issuing DPDX. Specifies the clock stable time before DPDX. Recommended settings: - mDDR: 1 - LPDDR2: 2 - LPDDR3: 2 When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer. This is only present for designs supporting mDDR or LPDDR2 devices.
15-4 —	Reserved
3-0 t_ckcsx	This is the time before Clock Stop Exit that CK is maintained as a valid clock before issuing Clock Stop Exit. Specifies the clock stable time before next command after Clock Stop Exit. Recommended settings: - mDDR: 1 - LPDDR2: tXP + 2 - LPDDR3: tXP + 2 - LPDDR4: tXP + 2 When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer. This is only present for designs supporting mDDR or LPDDR2/LPDDR3/LPDDR4 devices.

9.3.3.1.36 SDRAM Timing Register 7 (DRAMTMG7)

9.3.3.1.36.1 Offset

Register	Offset
DRAMTMG7	11Ch

9.3.3.1.36.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				t_ckpde				Reserved				t_ckpdx			
W																
Reset	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0

9.3.3.1.36.3 Fields

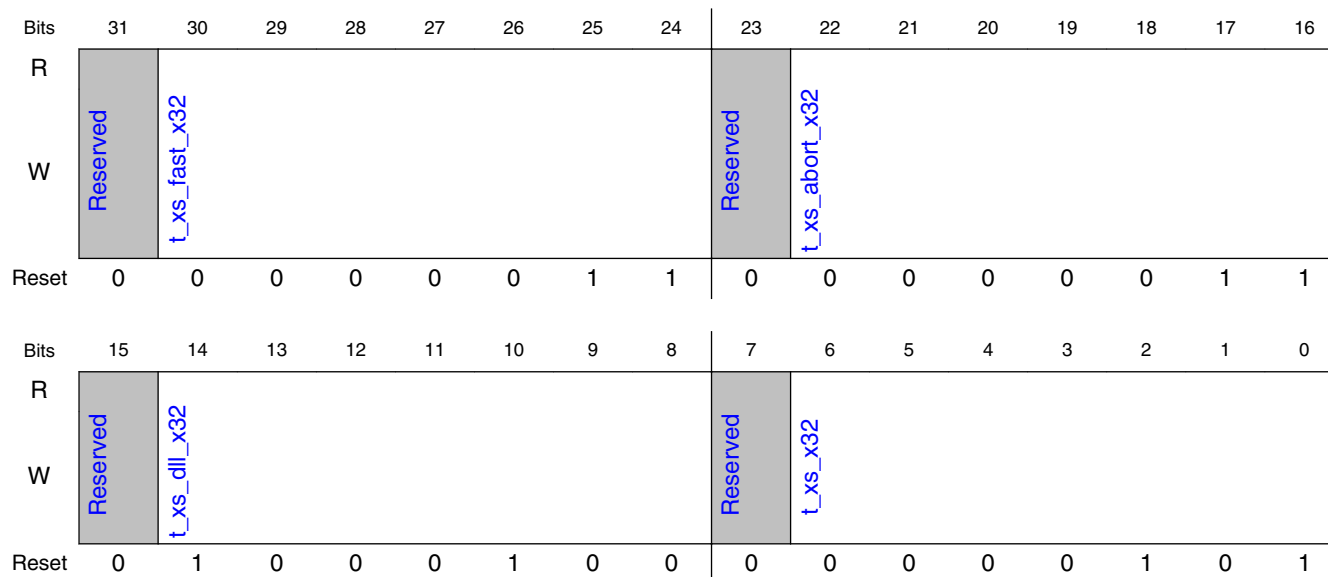
Field	Function
31-12 —	Reserved
11-8 t_ckpde	This is the time after Power Down Entry that CK is maintained as a valid clock. Specifies the clock disable delay after PDE. Recommended settings: - mDDR: 0 - LPDDR2: 2 - LPDDR3: 2 - LPDDR4: tCKCKEL When using DDR2/3/4 SDRAM, this register should be set to the same value as DRAMTMG5.t_cksre. When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer. This is only present for designs supporting mDDR or LPDDR2/LPDDR3/LPDDR4 devices.
7-4 —	Reserved
3-0 t_ckpdx	This is the time before Power Down Exit that CK is maintained as a valid clock before issuing PDX. Specifies the clock stable time before PDX. Recommended settings: - mDDR: 0 - LPDDR2: 2 - LPDDR3: 2 - LPDDR4: 2 When using DDR2/3/4 SDRAM, this register should be set to the same value as DRAMTMG5.t_cksrx. When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer. This is only present for designs supporting mDDR or LPDDR2/LPDDR3/LPDDR4 devices.

9.3.3.1.37 SDRAM Timing Register 8 (DRAMTMG8)

9.3.3.1.37.1 Offset

Register	Offset
DRAMTMG8	120h

9.3.3.1.37.2 Diagram



9.3.3.1.37.3 Fields

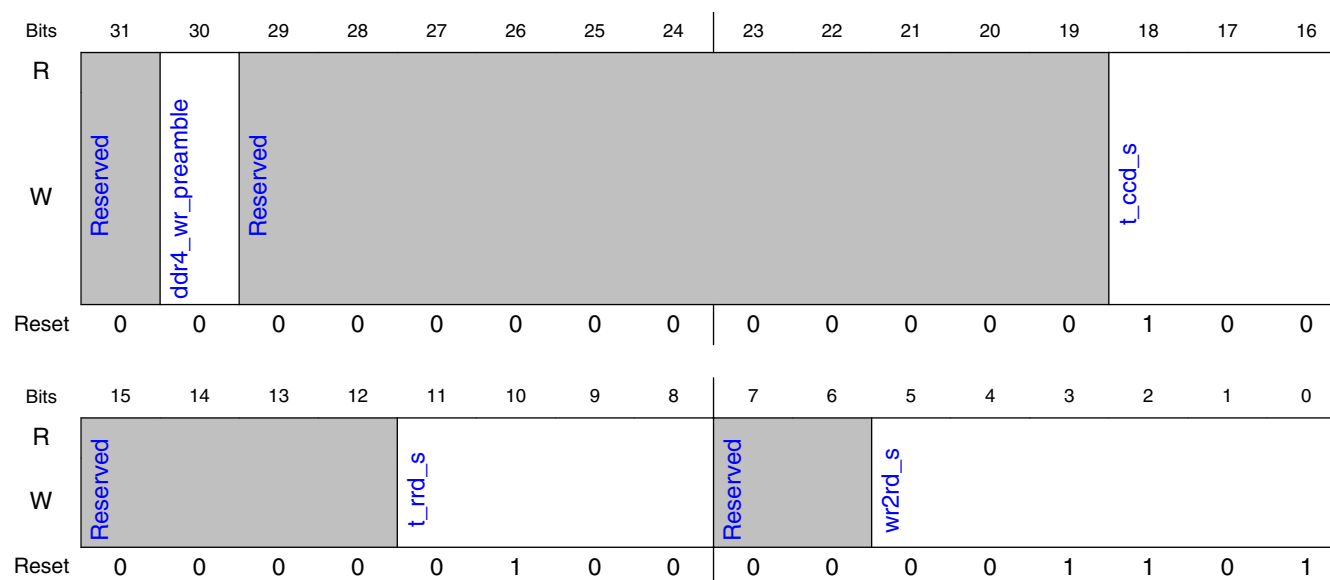
Field	Function
31 —	Reserved
30-24 t_xs_fast_x32	tXS_FAST: Exit Self Refresh to ZQCL, ZQCS and MRS (only CL, WR, RTP and Geardown mode). When the controller is operating in 1:2 frequency ratio mode, program this to the above value divided by 2 and round up to next integer value. Unit: Multiples of 32 clocks. Note: This is applicable to only ZQCL/ZQCS commands. Note: Ensure this is less than or equal to t_xs_x32.
23 —	Reserved
22-16 t_xs_abort_x32	tXS_ABORT: Exit Self Refresh to commands not requiring a locked DLL in Self Refresh Abort. When the controller is operating in 1:2 frequency ratio mode, program this to the above value divided by 2 and round up to next integer value. Unit: Multiples of 32 clocks. Note: Ensure this is less than or equal to t_xs_x32.
15 —	Reserved
14-8 t_xs_dll_x32	tXS_DLL: Exit Self Refresh to commands requiring a locked DLL. When the controller is operating in 1:2 frequency ratio mode, program this to the above value divided by 2 and round up to next integer value. Unit: Multiples of 32 clocks. Note: Used only for DDR2, DDR3 and DDR4 SDRAMs.
7 —	Reserved
6-0 t_xs_x32	tXS: Exit Self Refresh to commands not requiring a locked DLL. When the controller is operating in 1:2 frequency ratio mode, program this to the above value divided by 2 and round up to next integer value. Unit: Multiples of 32 clocks. Note: Used only for DDR2, DDR3 and DDR4 SDRAMs.

9.3.3.1.38 SDRAM Timing Register 9 (DRAMTMG9)

9.3.3.1.38.1 Offset

Register	Offset
DRAMTMG9	124h

9.3.3.1.38.2 Diagram



9.3.3.1.38.3 Fields

Field	Function
31 —	Reserved
30 ddr4_wr_preamble	DDR4 Write preamble mode - 0: 1tCK preamble - 1: 2tCK preamble Present only with MEMC_FREQ_RATIO=2
29-19 —	Reserved
18-16 t_ccd_s	tCCD_S: This is the minimum time between two reads or two writes for different bank group. For bank switching (from bank "a" to bank "b"), the minimum time is this value + 1. When the controller is operating in 1:2 frequency ratio mode, program this to (tCCD_S/2) and round it up to the next integer value. Present only in designs configured to support DDR4. Unit: clocks.
15-12 —	Reserved

Table continues on the next page...

DDR Controller (DDRC)

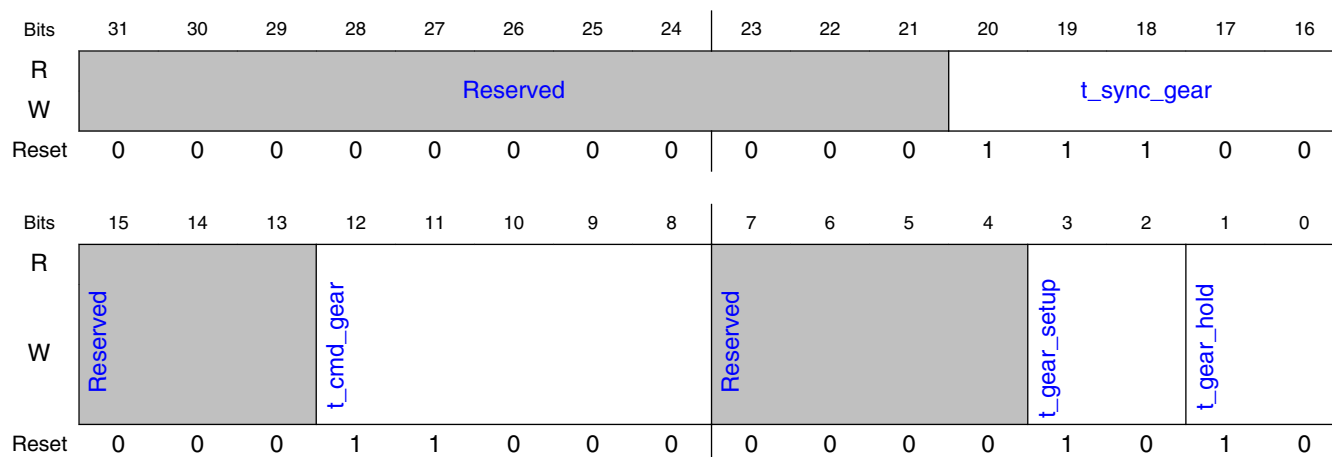
Field	Function
11-8 t_rrd_s	tRRD_S: Minimum time between activates from bank "a" to bank "b" for different bank group. When the controller is operating in 1:2 frequency ratio mode, program this to (tRRD_S/2) and round it up to the next integer value. Present only in designs configured to support DDR4. Unit: Clocks.
7-6 —	Reserved
5-0 wr2rd_s	CWL + PL + BL/2 + tWTR_S Minimum time from write command to read command for different bank group. Includes time for bus turnaround, recovery times, and all per-bank, per-rank, and global constraints. Present only in designs configured to support DDR4. Unit: Clocks. Where: - CWL = CAS write latency - PL = Parity latency - BL = burst length. This must match the value programmed in the BL bit of the mode register to the SDRAM - tWTR_S = internal write to read command delay for different bank group. This comes directly from the SDRAM specification. When the controller is operating in 1:2 mode, divide the value calculated using the above equation by 2, and round it up to next integer.

9.3.3.1.39 SDRAM Timing Register 10 (DRAMTMG10)

9.3.3.1.39.1 Offset

Register	Offset
DRAMTMG10	128h

9.3.3.1.39.2 Diagram



9.3.3.1.39.3 Fields

Field	Function
31-21	Reserved

Table continues on the next page...

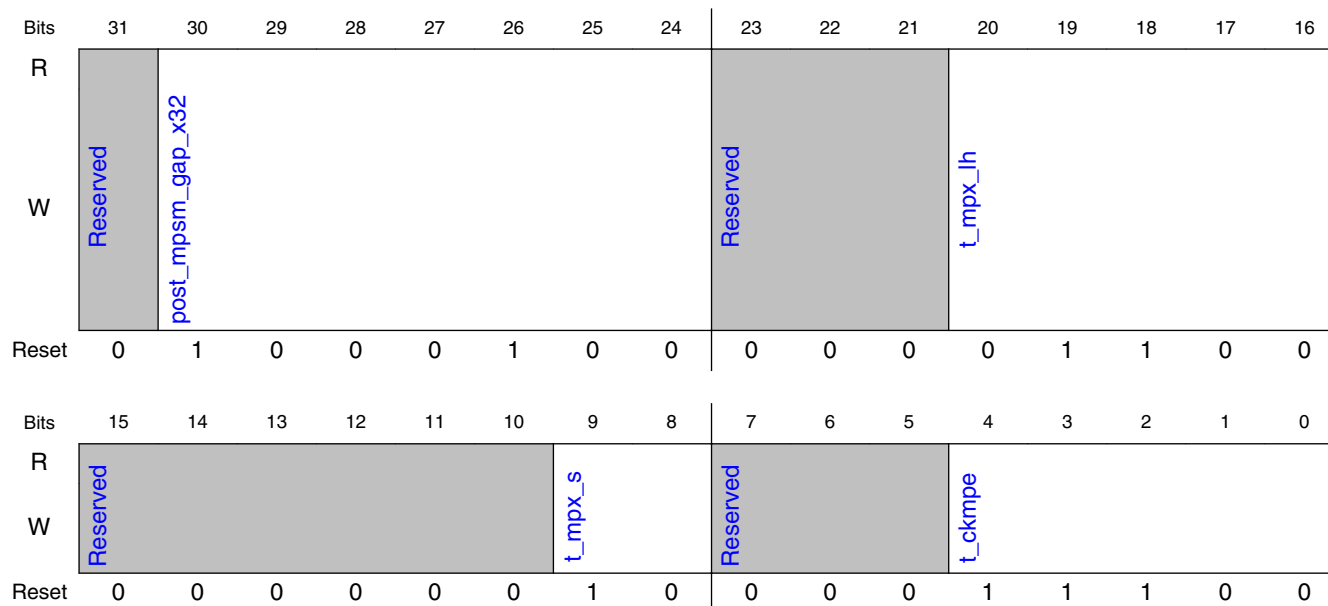
Field	Function
—	
20-16 t_sync_gear	Indicates the time between MRS command and the sync pulse time. This must be even number of clocks. For DDR4-2666 and DDR4-3200, this parameter is defined as $t_{MOD(min)} + 4nCK$ $t_{MOD(min)}$ is greater of $24nCK$ or $15ns$ $15ns / .625ns = 24$ Max value for this register is $24+4 = 28$ When the controller is operating in 1:2 mode, program this to $(t_{SYNC_GEAR}/2)$ and round it up to the next integer value. Unit: Clocks
15-13 —	Reserved
12-8 t_cmd_gear	Sync pulse to first valid command. For DDR4-2666 and DDR4-3200, this parameter is defined as $t_{MOD(min)}$ $t_{MOD(min)}$ is greater of $24nCK$ or $15ns$ $15ns / .625ns = 24$ Max value for this register is 24 When the controller is operating in 1:2 mode, program this to $(t_{CMD_GEAR}/2)$ and round it up to the next integer value. Unit: Clocks
7-4 —	Reserved
3-2 t_gear_setup	Geardown setup time. Minimum value of this register is 1. Zero is invalid. For DDR4-2666 and DDR4-3200, this parameter is defined as 2 clks When the controller is operating in 1:2 frequency ratio mode, program this to $(t_{GEAR_setup}/2)$ and round it up to the next integer value. Unit: Clocks
1-0 t_gear_hold	Geardown hold time. Minimum value of this register is 1. Zero is invalid. For DDR4-2666 and DDR4-3200, this parameter is defined as 2 clks When the controller is operating in 1:2 frequency ratio mode, program this to $(t_{GEAR_hold}/2)$ and round it up to the next integer value. Unit: Clocks

9.3.3.1.40 SDRAM Timing Register 11 (DRAMTMG11)

9.3.3.1.40.1 Offset

Register	Offset
DRAMTMG11	12Ch

9.3.3.1.40.2 Diagram



9.3.3.1.40.3 Fields

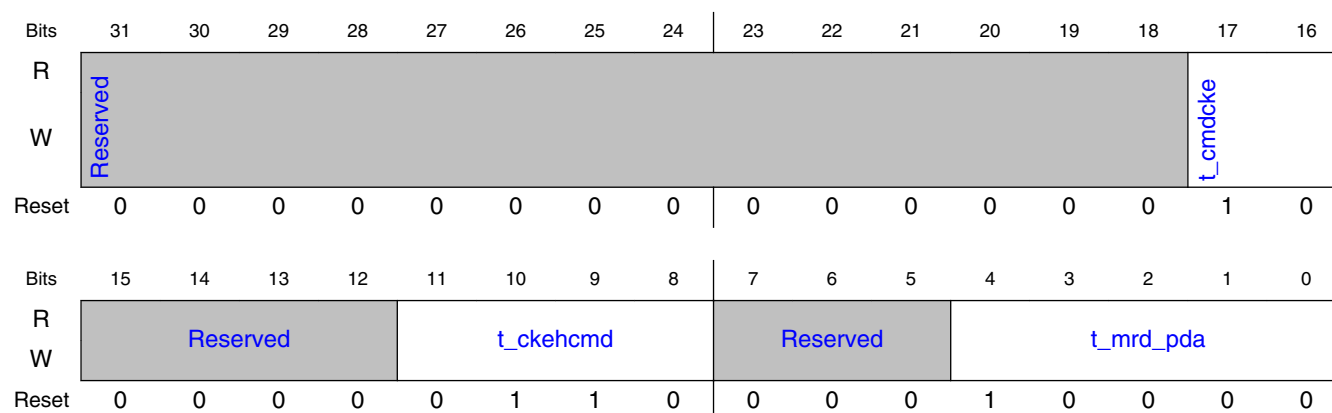
Field	Function
31 —	Reserved
30-24 post_mpsm_gap_x32	tXMPDLL: This is the minimum Exit MPSM to commands requiring a locked DLL. When the controller is operating in 1:2 frequency ratio mode, program this to (tXMPDLL/2) and round it up to the next integer value. Present only in designs configured to support DDR4. Unit: Multiples of 32 clocks.
23-21 —	Reserved
20-16 t_mpx_lh	tMPX_LH: This is the minimum CS_n Low hold time to CKE rising edge. When the controller is operating in 1:2 frequency ratio mode, program this to RoundUp(tMPX_LH/2)+1. Present only in designs configured to support DDR4. Unit: clocks.
15-10 —	Reserved
9-8 t_mpx_s	tMPX_S: Minimum time CS setup time to CKE. When the controller is operating in 1:2 frequency ratio mode, program this to (tMPX_S/2) and round it up to the next integer value. Present only in designs configured to support DDR4. Unit: Clocks.
7-5 —	Reserved
4-0 t_ckmpe	tCKMPE: Minimum valid clock requirement after MPSM entry. Present only in designs configured to support DDR4. Unit: Clocks. When the controller is operating in 1:2 frequency ratio mode, divide the value calculated using the above equation by 2, and round it up to next integer.

9.3.3.1.41 SDRAM Timing Register 12 (DRAMTMG12)

9.3.3.1.41.1 Offset

Register	Offset
DRAMTMG12	130h

9.3.3.1.41.2 Diagram



9.3.3.1.41.3 Fields

Field	Function
31-18 —	Reserved
17-16 t_cmdcke	tCMDCKE: Delay from valid command to CKE input LOW. Set this to the larger of tESCKE or tCMDCKE. When the controller is operating in 1:2 frequency ratio mode, program this to (max(tESCKE, tCMDCKE)/2) and round it up to the next integer value.
15-12 —	Reserved
11-8 t_cke/cmd	tCKEHCMD: Valid command requirement after CKE input HIGH. When the controller is operating in 1:2 frequency ratio mode, program this to (tCKEHCMD/2) and round it up to the next integer value.
7-5 —	Reserved
4-0 t_mrd_pda	tMRD_PDA: This is the Mode Register Set command cycle time in PDA mode. When the controller is operating in 1:2 frequency ratio mode, program this to (tMRD_PDA/2) and round it up to the next integer value.

9.3.3.1.42 SDRAM Timing Register 13 (DRAMTMG13)

9.3.3.1.42.1 Offset

Register	Offset
DRAMTMG13	134h

9.3.3.1.42.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved								Reserved		t_ccd_mw					
W																
Reset	0	0	0	1	1	1	0	0	0	0	1	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												t_ppd			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0

9.3.3.1.42.3 Fields

Field	Function
31 —	Reserved
30-24 odtloff	LPDDR4: tODTLoFF: This is the latency from CAS-2 command to tODTLoFF reference. When the controller is operating in 1:2 frequency ratio mode, program this to (tODTLoFF/2) and round it up to the next integer value. Unit: Clocks.
23-22 —	Reserved
21-16 t_ccd_mw	LPDDR4: tCCDMW: This is the minimum time from write or masked write to masked write command for same bank. When the controller is operating in 1:2 frequency ratio mode, program this to (tCCDMW/2) and round it up to the next integer value. Unit: Clocks.
15-3 —	Reserved
2-0 t_ppd	LPDDR4: tPPD: This is the minimum time from precharge to precharge command. When the controller is operating in 1:2 frequency ratio mode, program this to (tPPD/2) and round it up to the next integer value. Unit: Clocks.

9.3.3.1.43 SDRAM Timing Register 14 (DRAMTMG14)

9.3.3.1.43.1 Offset

Register	Offset
DRAMTMG14	138h

9.3.3.1.43.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				t_xsr											
W																
Reset	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0

9.3.3.1.43.3 Fields

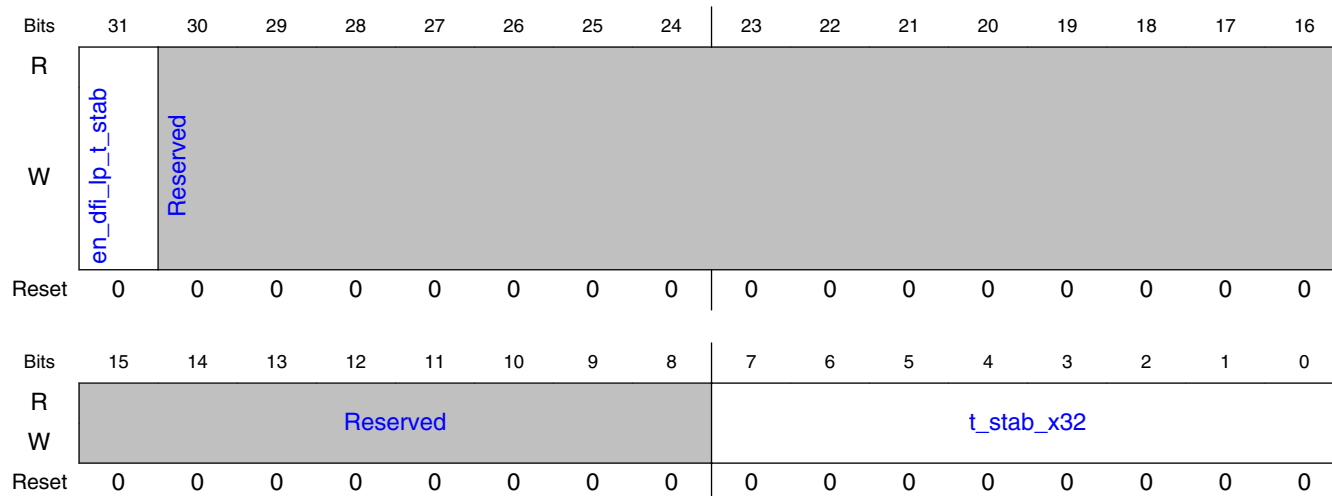
Field	Function
31-12 —	Reserved
11-0 t_xsr	tXSR: Exit Self Refresh to any command. When the controller is operating in 1:2 frequency ratio mode, program this to the above value divided by 2 and round up to next integer value. Note: Used only for mDDR/LPDDR2/LPDDR3/LPDDR4 mode.

9.3.3.1.44 SDRAM Timing Register 15 (DRAMTMG15)

9.3.3.1.44.1 Offset

Register	Offset
DRAMTMG15	13Ch

9.3.3.1.44.2 Diagram



9.3.3.1.44.3 Fields

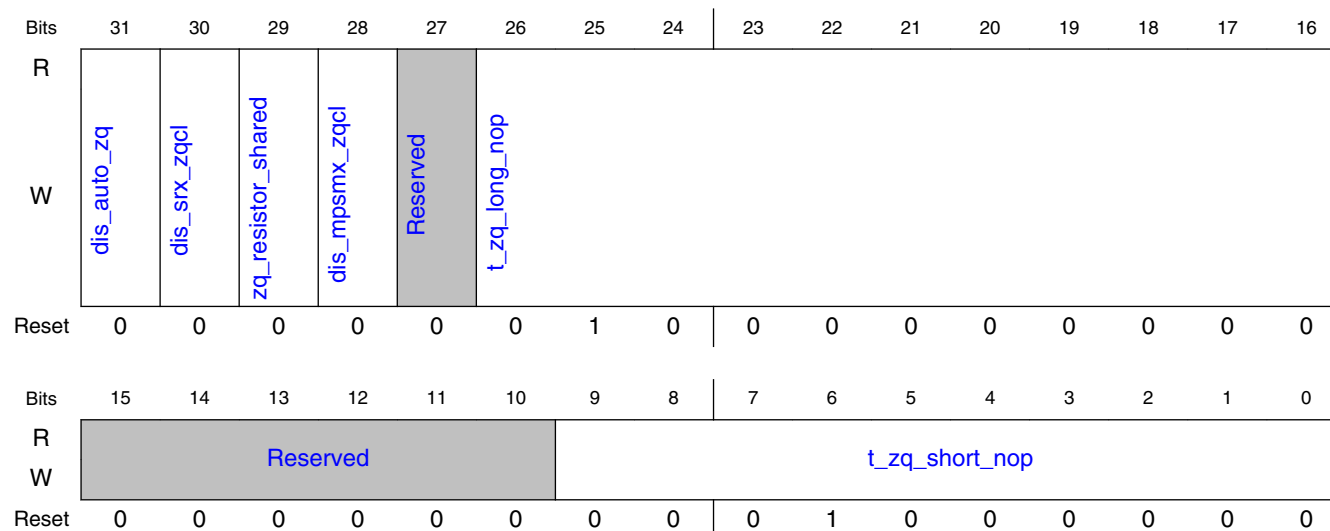
Field	Function
31 en_dfi_lp_t_stab	Enable DFI tSTAB 0b - Disable using tSTAB when exiting DFI LP 1b - Enable using tSTAB when exiting DFI LP. Needs to be set when the PHY is stopping the clock during DFI LP to save maximum power.
30-8 —	Reserved
7-0 t_stab_x32	tSTAB: Stabilization time. It is required in the following two cases for DDR3/DDR4 RDIMM : - when exiting power saving mode, if the clock was stopped, after re-enabling it the clock must be stable for a time specified by tSTAB - in the case of input clock frequency change (DDR4) - after issuing control words that refers to clock timing (Specification: 6us for DDR3, 5us for DDR4) When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer. Unit: Multiples of 32 clock cycles.

9.3.3.1.45 ZQ Control Register 0 (ZQCTL0)

9.3.3.1.45.1 Offset

Register	Offset
ZQCTL0	180h

9.3.3.1.45.2 Diagram



9.3.3.1.45.3 Fields

Field	Function
31 dis_auto_zq	Disable Auto ZQCS/MPC 0b - Internally generate ZQCS/MPC(ZQ calibration) commands based on ZQCTL1.t_zq_short_interval_x1024. 1b - Disable DDRC generation of ZQCS/MPC(ZQ calibration) command. Register DBGCMD.zq_calib_short can be used instead to issue ZQ calibration request from APB module.
30 dis_srx_zqcl	Disable ZQCL/MPC 0b - Enable issuing of ZQCL/MPC(ZQ calibration) command at Self-Refresh/SR-Powerdown exit. Only applicable when run in DDR3 or DDR4 or LPDDR2 or LPDDR3 or LPDDR4 mode. This is only present for designs supporting DDR3/DDR4 or LPDDR2/LPDDR3/LPDDR4 devices. 1b - Disable issuing of ZQCL/MPC(ZQ calibration) command at Self-Refresh/SR-Powerdown exit. Only applicable when run in DDR3 or DDR4 or LPDDR2 or LPDDR3 or LPDDR4 mode.
29 zq_resistor_shared	ZQ resistor sharing 0b - ZQ resistor is not shared. This is only present for designs supporting DDR3/DDR4 or LPDDR2/LPDDR3/LPDDR4 devices. 1b - Denotes that ZQ resistor is shared between ranks. Means ZQinit/ZQCL/tZQCS/tZQCAL/tZQLAT timing met between commands so that commands to different ranks do not overlap.
28 dis_mpsmx_zqcl	Do not issue ZQCL command at Maximum Power Save Mode exit if the DDRC_SHARED_AC configuration parameter is set. Program it to 1'b1. The software can send ZQCS after exiting MPSM mode. 0b - Enable issuing of ZQCL command at Maximum Power Saving Mode exit. Only applicable when run in DDR4 mode. This is only present for designs supporting DDR4 devices. 1b - Disable issuing of ZQCL command at Maximum Power Saving Mode exit. Only applicable when run in DDR4 mode.
27 —	Reserved
26-16 t_zq_long_nop	tZQoper for DDR3/DDR4, tZQCL for LPDDR2/LPDDR3, tZQCAL for LPDDR4: Number of DFI clock cycles of NOP required after a ZQCL (ZQ calibration long)/MPC(ZQ Start) command is issued to

Table continues on the next page...

DDR Controller (DDRC)

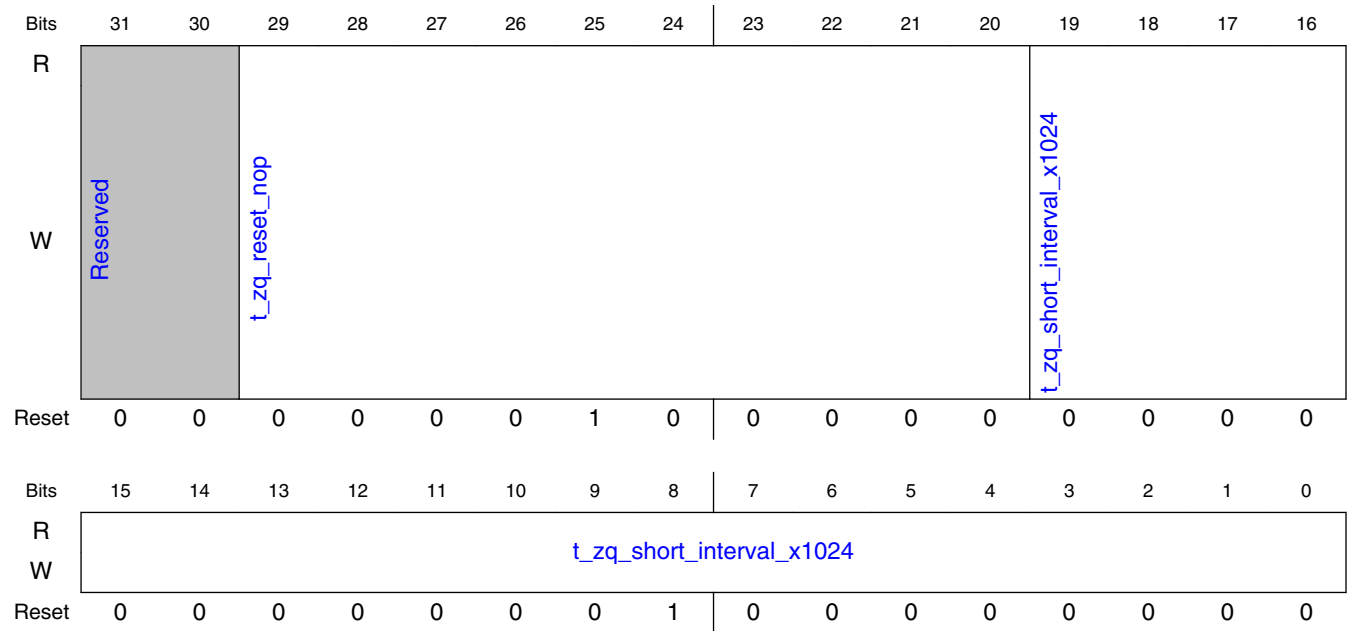
Field	Function
	SDRAM. When the controller is operating in 1:2 frequency ratio mode: DDR3/DDR4: program this to tZQoper/2 and round it up to the next integer value. LPDDR2/LPDDR3: program this to tZQCL/2 and round it up to the next integer value. LPDDR4: program this to tZQCAL/2 and round it up to the next integer value. This is only present for designs supporting DDR3/DDR4 or LPDDR2/LPDDR3/LPDDR4 devices.
15-10 —	Reserved
9-0 t_zq_short_nop	tZQCS for DDR3/DD4/LPDDR2/LPDDR3, tZQLAT for LPDDR4: Number of DFI clock cycles of NOP required after a ZQCS (ZQ calibration short)/MPC(ZQ Latch) command is issued to SDRAM. When the controller is operating in 1:2 frequency ratio mode, program this to tZQCS/2 and round it up to the next integer value. This is only present for designs supporting DDR3/DDR4 or LPDDR2/LPDDR3/LPDDR4 devices.

9.3.3.1.46 ZQ Control Register 1 (ZQCTL1)

9.3.3.1.46.1 Offset

Register	Offset
ZQCTL1	184h

9.3.3.1.46.2 Diagram



9.3.3.1.46.3 Fields

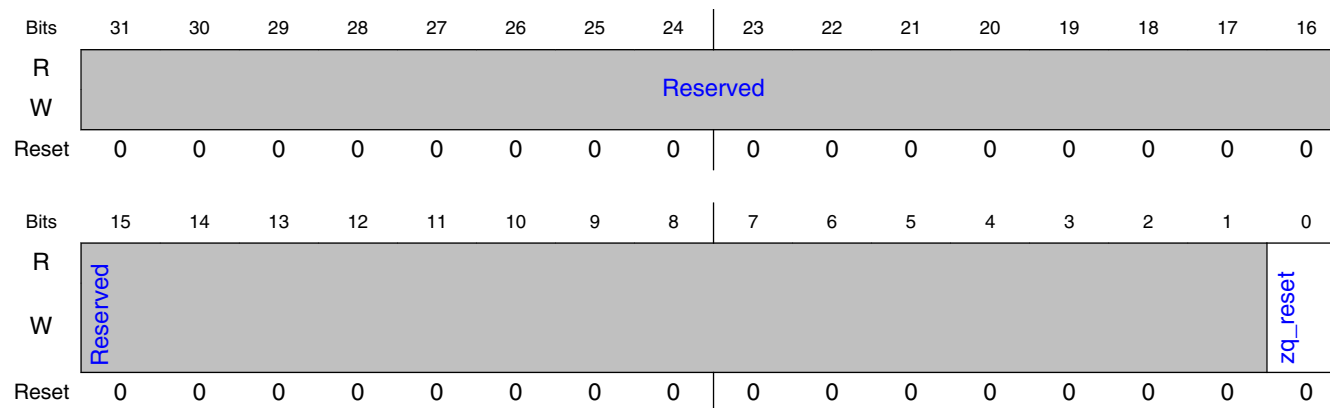
Field	Function
31-30 —	Reserved
29-20 t_zq_reset_nop	tZQReset: Number of DFI clock cycles of NOP required after a ZQReset (ZQ calibration Reset) command is issued to SDRAM. When the controller is operating in 1:2 frequency ratio mode, program this to tZQReset/2 and round it up to the next integer value. This is only present for designs supporting LPDDR2/LPDDR3/LPDDR4 devices.
19-0 t_zq_short_inter val_x1024	Average interval to wait between automatically issuing ZQCS (ZQ calibration short)/MPC(ZQ calibration) commands to DDR3/DDR4/LPDDR2/LPDDR3/LPDDR4 devices. Meaningless, if ZQCTL0.dis_auto_zq=1. Unit: 1024 DFI clock cycles. This is only present for designs supporting DDR3/DDR4 or LPDDR2/LPDDR3/LPDDR4 devices.

9.3.3.1.47 ZQ Control Register 2 (ZQCTL2)

9.3.3.1.47.1 Offset

Register	Offset
ZQCTL2	188h

9.3.3.1.47.2 Diagram



9.3.3.1.47.3 Fields

Field	Function
31-1 —	Reserved

Table continues on the next page...

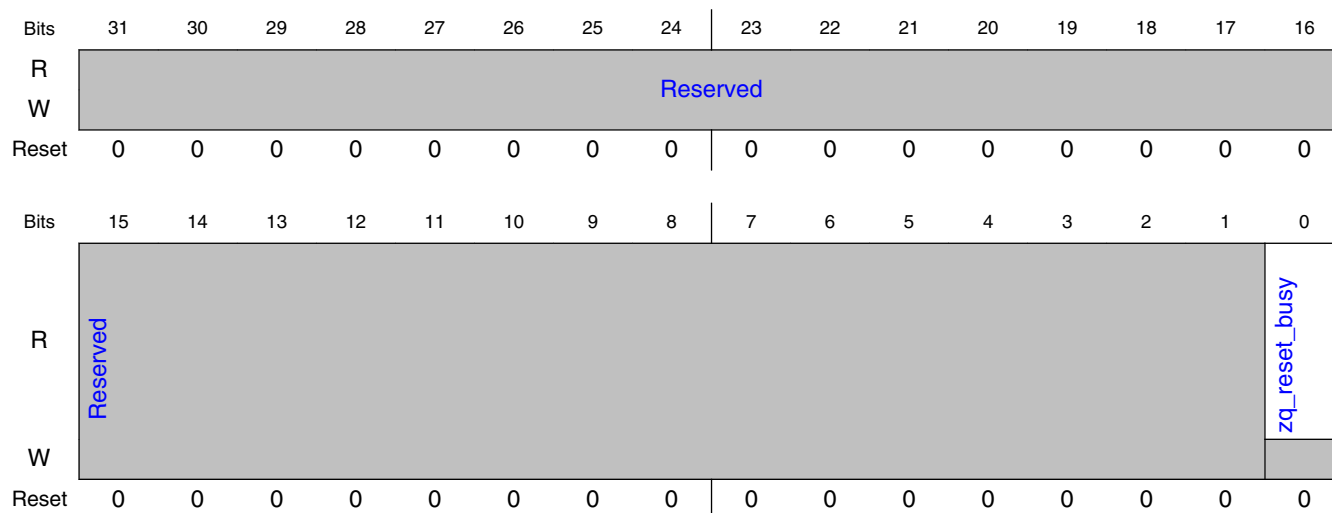
Field	Function
0 zq_reset	Setting this register bit to 1 triggers a ZQ Reset operation. When the ZQ Reset operation is complete, the DDRC automatically clears this bit. It is recommended NOT to set this signal if in Init, Self-Refresh(except LPDDR4) or SR-Powerdown(LPDDR4) or Deep power-down operating modes. This is only present for designs supporting LPDDR2/LPDDR3/LPDDR4 devices.

9.3.3.1.48 ZQ Status Register (ZQSTAT)

9.3.3.1.48.1 Offset

Register	Offset
ZQSTAT	18Ch

9.3.3.1.48.2 Diagram



9.3.3.1.48.3 Fields

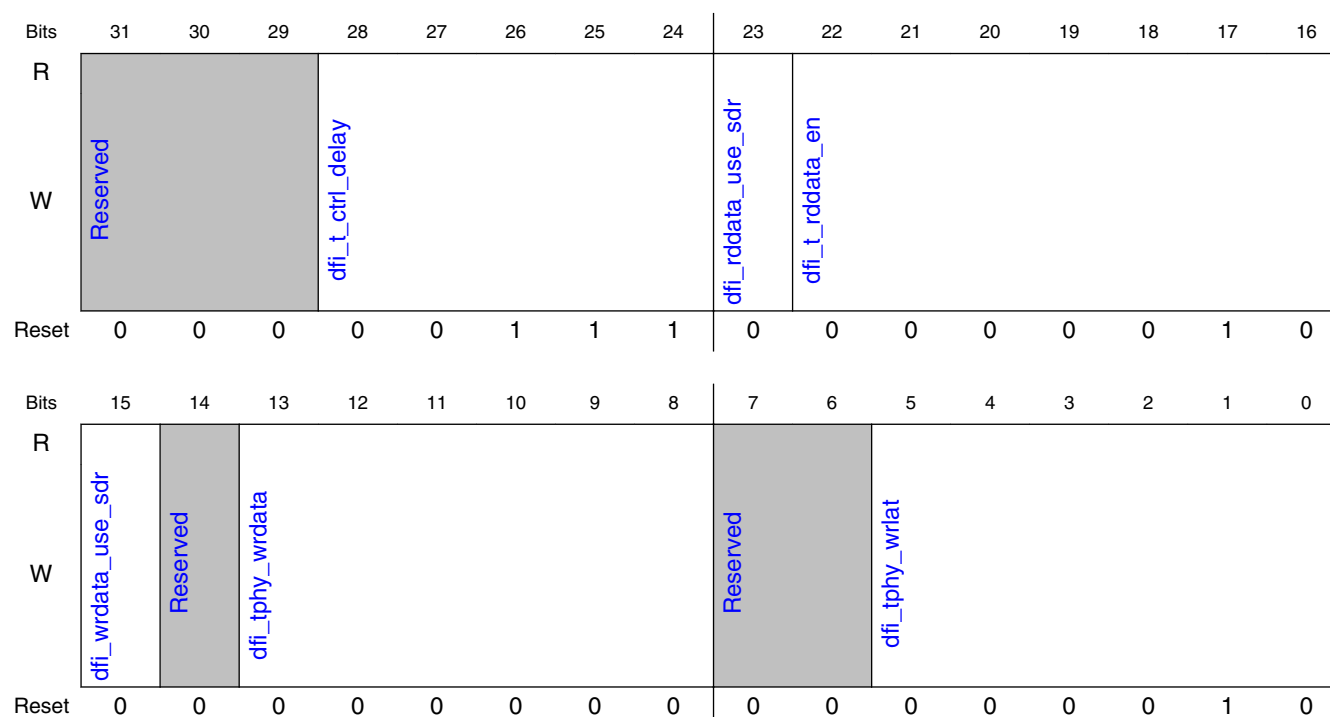
Field	Function
31-1 —	Reserved
0 zq_reset_busy	SoC core may initiate a ZQ Reset operation only if this signal is low. This signal goes high in the clock after the DDRC accepts the ZQ Reset request. It goes low when the ZQ Reset command is issued to the SDRAM and the associated NOP period is over. It is recommended not to perform ZQ Reset commands when this signal is high. 0b - Indicates that the SoC core can initiate a ZQ Reset operation 1b - Indicates that ZQ Reset operation is in progress

9.3.3.1.49 DFI Timing Register 0 (DFITMG0)

9.3.3.1.49.1 Offset

Register	Offset
DFITMG0	190h

9.3.3.1.49.2 Diagram



9.3.3.1.49.3 Fields

Field	Function
31-29 —	Reserved
28-24 <code>dfi_t_ctrl_delay</code>	Specifies the number of DFI clock cycles after an assertion or de-assertion of the DFI control signals that the control signals at the PHY-DRAM interface reflect the assertion or de-assertion. If the DFI clock and the memory clock are not phase-aligned, this timing parameter should be rounded up to the next integer value. Note that if using RDIMM/LRDIMM, it is necessary to increment this parameter by RDIMM's/LRDIMM's extra cycle of latency in terms of DFI clock.
23	Defines whether <code>dfi_rddata_en</code> / <code>dfi_rddata</code> / <code>dfi_rddata_valid</code> is generated using HDR (DFI clock) or SDR (DFI PHY clock) values. Selects whether value in <code>DFITMG0.dfi_t_rddata_en</code> is in terms of HDR (DFI

Table continues on the next page...

DDR Controller (DDRC)

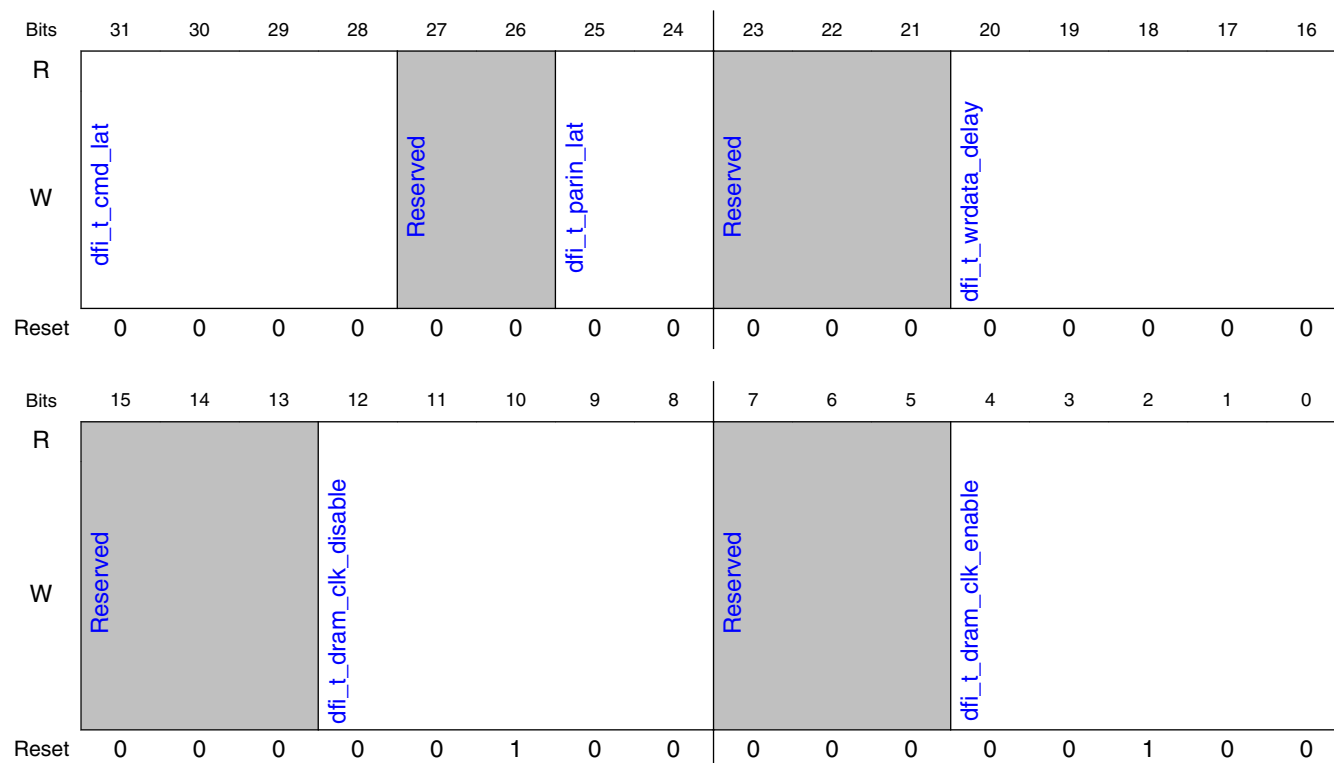
Field	Function
dfi_rddata_use_sdr	clock) or SDR (DFI PHY clock) cycles: - 0 in terms of HDR (DFI clock) cycles - 1 in terms of SDR (DFI PHY clock) cycles Refer to PHY specification for correct value.
22-16 dfi_t_rddata_en	Time from the assertion of a read command on the DFI interface to the assertion of the dfi_rddata_en signal. Refer to PHY specification for correct value. This corresponds to the DFI parameter trddata_en. Note that, depending on the PHY, if using RDIMM/LRDIMM, it may be necessary to use the adjusted value of CL in the calculation of trddata_en. This is to compensate for the extra cycle(s) of latency through the RDIMM/LRDIMM. Unit: DFI clock cycles or DFI PHY clock cycles, depending on DFITMG0.dfi_rddata_use_sdr.
15 dfi_wrdata_use_sdr	Defines whether dfi_wrdata_en/dfi_wrdata/dfi_wrdata_mask is generated using HDR (DFI clock) or SDR (DFI PHY clock) values Selects whether value in DFITMG0.dfi_tphy_wrlat is in terms of HDR (DFI clock) or SDR (DFI PHY clock) cycles Selects whether value in DFITMG0.dfi_tphy_wrdata is in terms of HDR (DFI clock) or SDR (DFI PHY clock) cycles - 0 in terms of HDR (DFI clock) cycles - 1 in terms of SDR (DFI PHY clock) cycles Refer to PHY specification for correct value.
14 —	Reserved
13-8 dfi_tphy_wrdata	Specifies the number of clock cycles between when dfi_wrdata_en is asserted to when the associated write data is driven on the dfi_wrdata signal. This corresponds to the DFI timing parameter tphy_wrdata. Refer to PHY specification for correct value. Note, max supported value is 8. Unit: DFI clock cycles or DFI PHY clock cycles, depending on DFITMG0.dfi_wrdata_use_sdr.
7-6 —	Reserved
5-0 dfi_tphy_wrlat	Write latency Number of clocks from the write command to write data enable (dfi_wrdata_en). This corresponds to the DFI timing parameter tphy_wrlat. Refer to PHY specification for correct value. Note that, depending on the PHY, if using RDIMM/LRDIMM, it may be necessary to use the adjusted value of CL in the calculation of tphy_wrlat. This is to compensate for the extra cycle(s) of latency through the RDIMM/LRDIMM. Unit: DFI clock cycles or DFI PHY clock cycles, depending on DFITMG0.dfi_wrdata_use_sdr.

9.3.3.1.50 DFI Timing Register 1 (DFITMG1)

9.3.3.1.50.1 Offset

Register	Offset
DFITMG1	194h

9.3.3.1.50.2 Diagram



9.3.3.1.50.3 Fields

Field	Function
31-28 dfi_t_cmd_lat	Specifies the number of DFI PHY clock cycles between when the dfi_cs signal is asserted and when the associated command is driven. This field is used for CAL mode, should be set to '0' or the value which matches the CAL mode register setting in the DRAM. If the PHY can add the latency for CAL mode, this should be set to '0'. Valid Range: 0, 3, 4, 5, 6, and 8
27-26 —	Reserved
25-24 dfi_t_parin_lat	Specifies the number of DFI PHY clock cycles between when the dfi_cs signal is asserted and when the associated dfi_parity_in signal is driven.
23-21 —	Reserved
20-16 dfi_t_wrdata_delay	Specifies the number of DFI clock cycles between when the dfi_wrdata_en signal is asserted and when the corresponding write data transfer is completed on the DRAM bus. This corresponds to the DFI timing parameter twrdata_delay. Refer to PHY specification for correct value. For DFI 3.0 PHY, set to twrdata_delay, a new timing parameter introduced in DFI 3.0. For DFI 2.1 PHY, set to tphy_wrdata + (delay of DFI write data to the DRAM). Value to be programmed is in terms of DFI clocks, not PHY clocks. In FREQ_RATIO=2, divide PHY's value by 2 and round up to next integer. If using DFITMG0.dfi_wrdata_use_sdr=1, add 1 to the value. Unit: Clocks
15-13 —	Reserved

Table continues on the next page...

DDR Controller (DDRC)

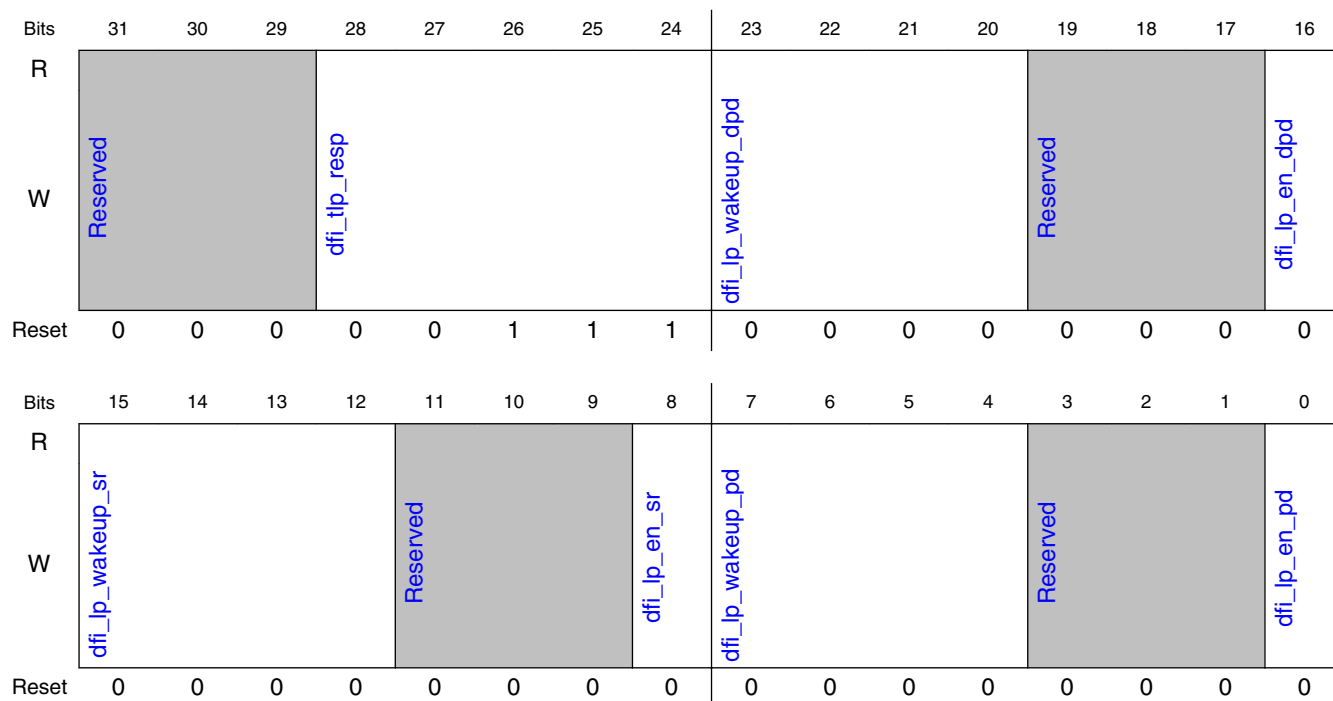
Field	Function
12-8 dfi_t_dram_clk_disable	Specifies the number of DFI clock cycles from the assertion of the dfi_dram_clk_disable signal on the DFI until the clock to the DRAM memory devices, at the PHY-DRAM boundary, maintains a low value. If the DFI clock and the memory clock are not phase aligned, this timing parameter should be rounded up to the next integer value.
7-5 —	Reserved
4-0 dfi_t_dram_clk_enable	Specifies the number of DFI clock cycles from the de-assertion of the dfi_dram_clk_disable signal on the DFI until the first valid rising edge of the clock to the DRAM memory devices, at the PHY-DRAM boundary. If the DFI clock and the memory clock are not phase aligned, this timing parameter should be rounded up to the next integer value.

9.3.3.1.51 DFI Low Power Configuration Register 0 (DFILPCFG0)

9.3.3.1.51.1 Offset

Register	Offset
DFILPCFG0	198h

9.3.3.1.51.2 Diagram



9.3.3.1.51.3 Fields

Field	Function
31-29 —	Reserved
28-24 dfi_tlp_resp	Setting in DFI clock cycles for DFI's tlp_resp time. Same value is used for both Power Down, Self Refresh, Deep Power Down and Maximum Power Saving modes. DFI 2.1 specification onwards, recommends using a fixed value of 7 always.
23-20 dfi_lp_wakeup_dpd	Value in DFI clock cycles to drive on dfi_lp_wakeup signal when Deep Power Down mode is entered. Determines the DFI's tlp_wakeup time: This is only present for designs supporting mDDR or LPDDR2/LPDDR3 devices. 0000b - 16 cycles 0001b - 32 cycles 0010b - 64 cycles 0011b - 128 cycles 0100b - 256 cycles 0101b - 512 cycles 0110b - 1024 cycles 0111b - 2048 cycles 1000b - 4096 cycles 1001b - 8192 cycles 1010b - 16384 cycles 1011b - 32768 cycles 1100b - 65536 cycles 1101b - 131072 cycles 1110b - 262144 cycles 1111b - Unlimited cycles
19-17 —	Reserved
16 dfi_lp_en_dpd	Enables DFI Low Power interface handshaking during Deep Power Down Entry/Exit. - 0 - Disabled - 1 - Enabled This is only present for designs supporting mDDR or LPDDR2/LPDDR3 devices.
15-12 dfi_lp_wakeup_sr	Value in DFI clpck cycles to drive on dfi_lp_wakeup signal when Self Refresh mode is entered. Determines the DFI's tlp_wakeup time: 0000b - 16 cycles 0001b - 32 cycles 0010b - 64 cycles 0011b - 128 cycles 0100b - 256 cycles 0101b - 512 cycles 0110b - 1024 cycles 0111b - 2048 cycles 1000b - 4096 cycles 1001b - 8192 cycles 1010b - 16384 cycles 1011b - 32768 cycles 1100b - 65536 cycles 1101b - 131072 cycles 1110b - 262144 cycles 1111b - Unlimited cycles
11-9 —	Reserved
8	Enables DFI Low Power interface handshaking during Self Refresh Entry/Exit. - 0 - Disabled - 1 - Enabled 0b - Disabled

Table continues on the next page...

DDR Controller (DDRC)

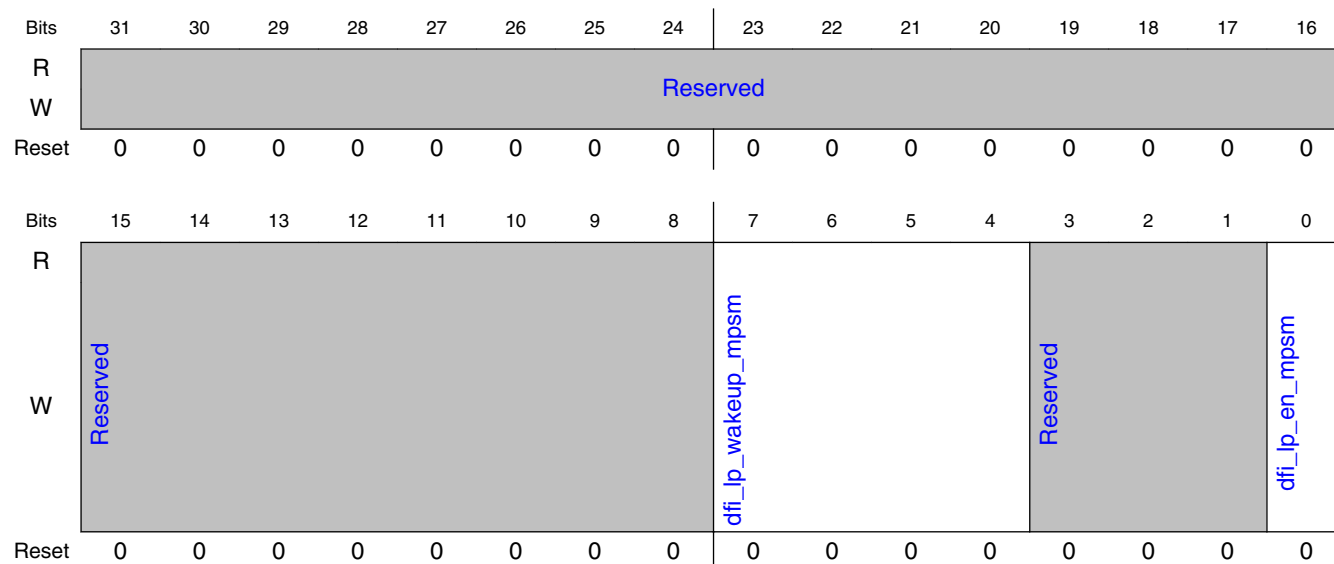
Field	Function
dfi_lp_en_sr	1b - Enabled
7-4 dfi_lp_wakeup_ pd	Value in DFI clock cycles to drive on dfi_lp_wakeup signal when Power Down mode is entered. Determines the DFI's tlp_wakeup time: 0000b - 16 cycles 0001b - 32 cycles 0010b - 64 cycles 0011b - 128 cycles 0100b - 256 cycles 0101b - 512 cycles 0110b - 1024 cycles 0111b - 2048 cycles 1000b - 4096 cycles 1001b - 8192 cycles 1010b - 16384 cycles 1011b - 32768 cycles 1100b - 65536 cycles 1101b - 131072 cycles 1110b - 262144 cycles 1111b - Unlimited cycles
3-1 —	Reserved
0 dfi_lp_en_pd	Enables DFI Low Power interface handshaking during Power Down Entry/Exit. - 0 - Disabled - 1 - Enabled

9.3.3.1.52 DFI Low Power Configuration Register 1 (DFILPCFG1)

9.3.3.1.52.1 Offset

Register	Offset
DFILPCFG1	19Ch

9.3.3.1.52.2 Diagram



9.3.3.1.52.3 Fields

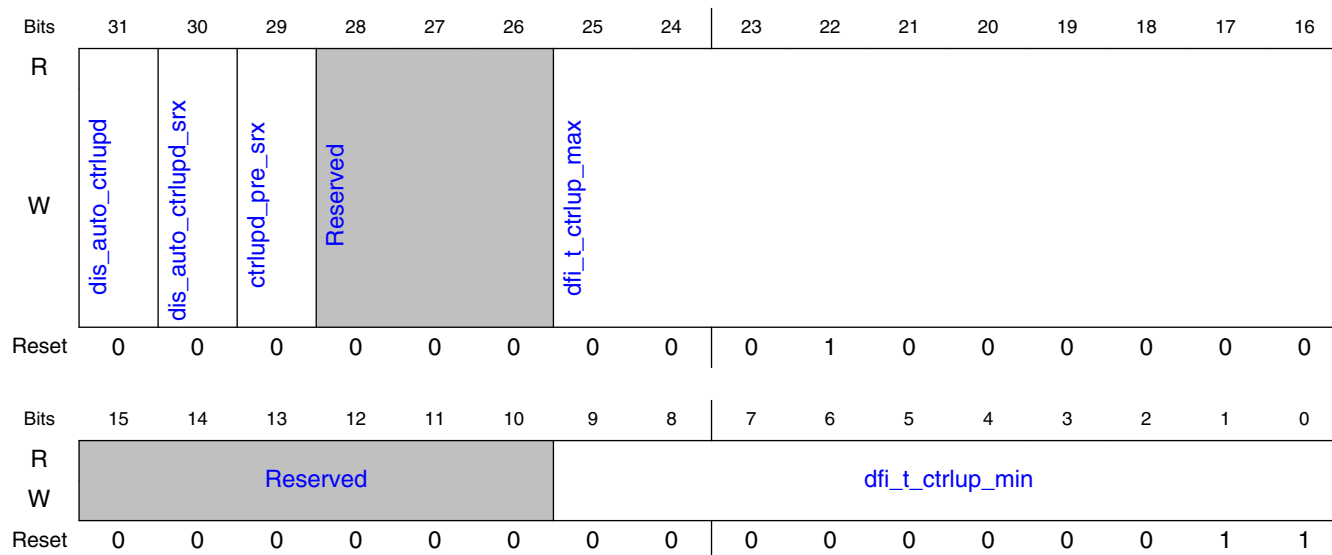
Field	Function
31-8 —	Reserved
7-4 dfi_lp_wakeup_mpsm	Value in DFI clock cycles to drive on dfi_lp_wakeup signal when Maximum Power Saving Mode is entered. Determines the DFI's tlp_wakeup time: 0000b - 16 cycles 0001b - 32 cycles 0010b - 64 cycles 0011b - 128 cycles 0100b - 256 cycles 0101b - 512 cycles 0110b - 1024 cycles 0111b - 2048 cycles 1000b - 4096 cycles 1001b - 8192 cycles 1010b - 16384 cycles 1011b - 32768 cycles 1100b - 65536 cycles 1101b - 131072 cycles 1110b - 262144 cycles 1111b - Unlimited cycles
3-1 —	Reserved
0 dfi_lp_en_mpsm	Enables DFI Low Power interface handshaking during Maximum Power Saving Mode Entry/Exit. - 0 - Disabled - 1 - Enabled This is only present for designs supporting DDR4 devices.

9.3.3.1.53 DFI Update Register 0 (DFIUPD0)

9.3.3.1.53.1 Offset

Register	Offset
DFIUPD0	1A0h

9.3.3.1.53.2 Diagram



9.3.3.1.53.3 Fields

Field	Function
31 dis_auto_ctrlupd	automatic dfi_ctrlupd_req generation by the DDRC 0b - DDRC issues dfi_ctrlupd_req periodically. 1b - disable the automatic dfi_ctrlupd_req generation by the DDRC. The core must issue the dfi_ctrlupd_req signal using register reg_ddrc_ctrlupd.
30 dis_auto_ctrlupd_srx	Auto ctrlupd request generation 0b - DDRC issues a dfi_ctrlupd_req before or after exiting self-refresh, depending on DFIUPD0.ctrlupd_pre_srx. 1b - disable the automatic dfi_ctrlupd_req generation by the DDRC at self-refresh exit.
29 ctrlupd_pre_srx	Selects dfi_ctrlupd_req requirements at SRX: - 0 : send ctrlupd after SRX - 1 : send ctrlupd before SRX If DFIUPD0.dis_auto_ctrlupd_srx=1, this register has no impact, because no dfi_ctrlupd_req will be issued when SRX. 0b - send ctrlupd after SRX 1b - send ctrlupd before SRX
28-26 —	Reserved

Table continues on the next page...

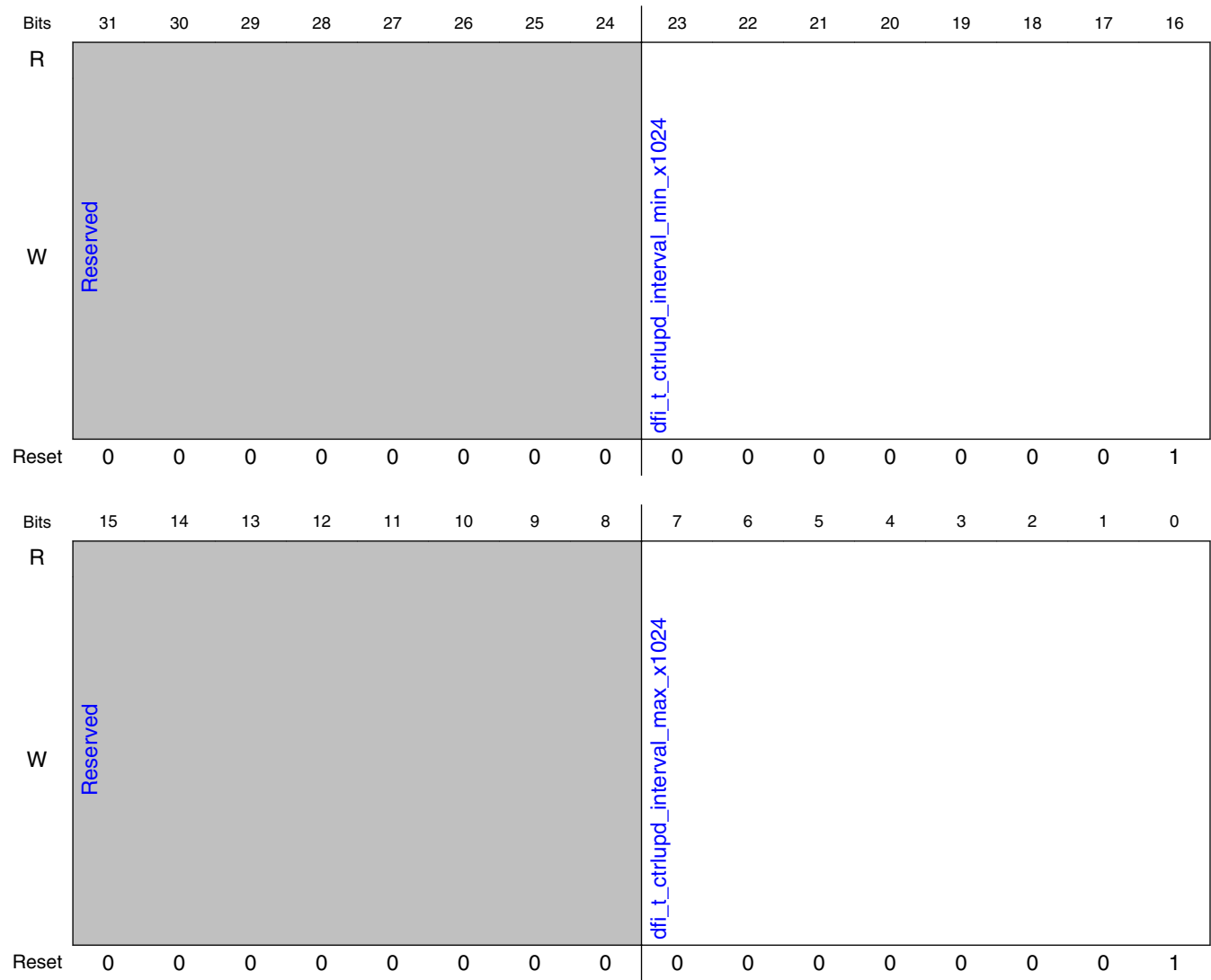
Field	Function
25-16 dfi_t_ctrlup_max	Specifies the maximum number of DFI clock cycles that the dfi_ctrlupd_req signal can assert. Lowest value to assign to this variable is 0x40.
15-10 —	Reserved
9-0 dfi_t_ctrlup_min	Specifies the minimum number of DFI clock cycles that the dfi_ctrlupd_req signal must be asserted. The DDRC expects the PHY to respond within this time. If the PHY does not respond, the DDRC will de-assert dfi_ctrlupd_req after dfi_t_ctrlup_min + 2 cycles. Lowest value to assign to this variable is 0x3.

9.3.3.1.54 DFI Update Register 1 (DFIUPD1)

9.3.3.1.54.1 Offset

Register	Offset
DFIUPD1	1A4h

9.3.3.1.54.2 Diagram



9.3.3.1.54.3 Fields

Field	Function
31-24 —	Reserved
23-16 dfi_t_ctrlupd_interval_min_x1024	This is the minimum amount of time between DDRC initiated DFI update requests (which is executed whenever the DDRC is idle). Set this number higher to reduce the frequency of update requests, which can have a small impact on the latency of the first read request when the DDRC is idle. Minimum allowed value for this field is 1. Unit: 1024 DFI clock cycles
15-8 —	Reserved

Table continues on the next page...

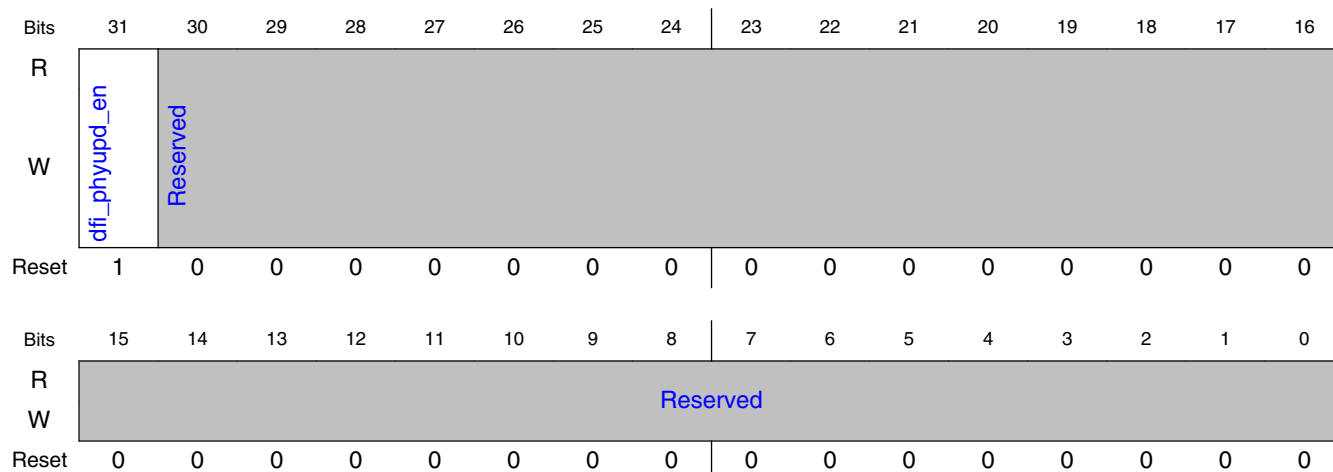
Field	Function
7-0 dfi_t_ctrlupd_interval_max_x1024	This is the maximum amount of time between DDRC initiated DFI update requests. This timer resets with each update request; when the timer expires dfi_ctrlupd_req is sent and traffic is blocked until the dfi_ctrlupd_ackx is received. PHY can use this idle time to recalibrate the delay lines to the DLLs. The DFI controller update is also used to reset PHY FIFO pointers in case of data capture errors. Updates are required to maintain calibration over PVT, but frequent updates may impact performance. Minimum allowed value for this field is 1. Note: Value programmed for DFIUPD1.dfi_t_ctrlupd_interval_max_x1024 must be greater than DFIUPD1.dfi_t_ctrlupd_interval_min_x1024. Unit: 1024 DFI clock cycles

9.3.3.1.55 DFI Update Register 2 (DFIUPD2)

9.3.3.1.55.1 Offset

Register	Offset
DFIUPD2	1A8h

9.3.3.1.55.2 Diagram



9.3.3.1.55.3 Fields

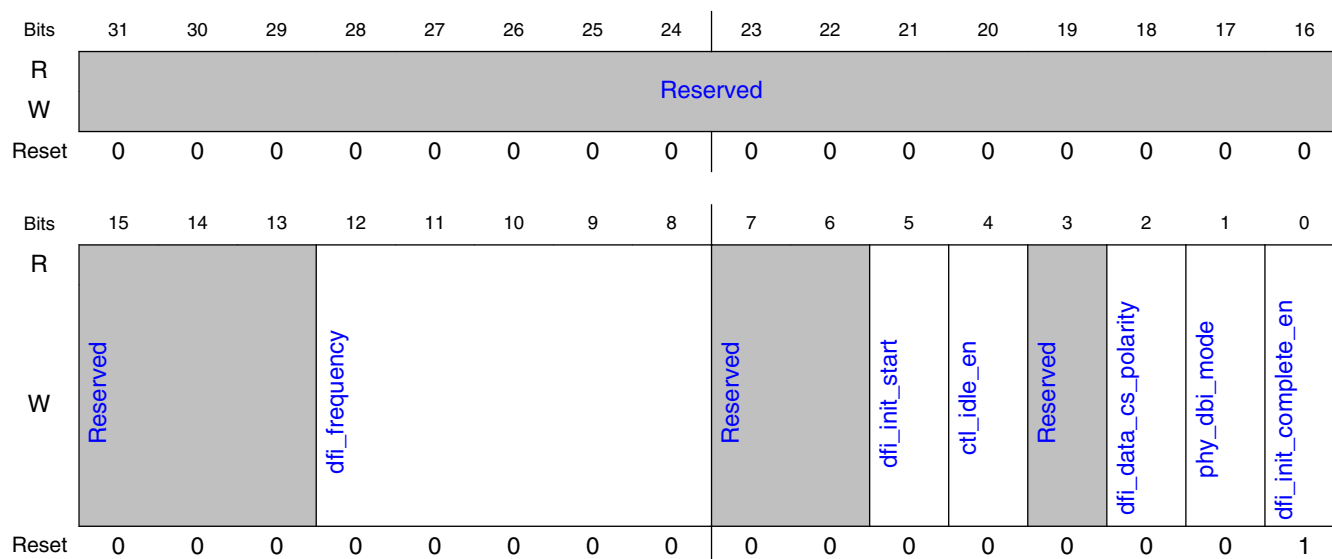
Field	Function
31 dfi_phyupd_en	Enables the support for acknowledging PHY-initiated updates: 0b - Disabled 1b - Enabled
30-0 —	Reserved

9.3.3.1.56 DFI Miscellaneous Control Register (DFIMISC)

9.3.3.1.56.1 Offset

Register	Offset
DFIMISC	1B0h

9.3.3.1.56.2 Diagram



9.3.3.1.56.3 Fields

Field	Function
31-13 —	Reserved
12-8 dfi_frequency	Indicates the operating frequency of the system. The number of supported frequencies and the mapping of signal values to clock frequencies are defined by the PHY.
7-6 —	Reserved
5 dfi_init_start	PHY init start request signal. When asserted it triggers the PHY init start request
4 ctl_idle_en	Enables support of ctl_idle signal, which is non-DFI related pin specific to certain PHYs. See signal description of ctl_idle signal for further details of ctl_idle functionality.
3	Reserved

Table continues on the next page...

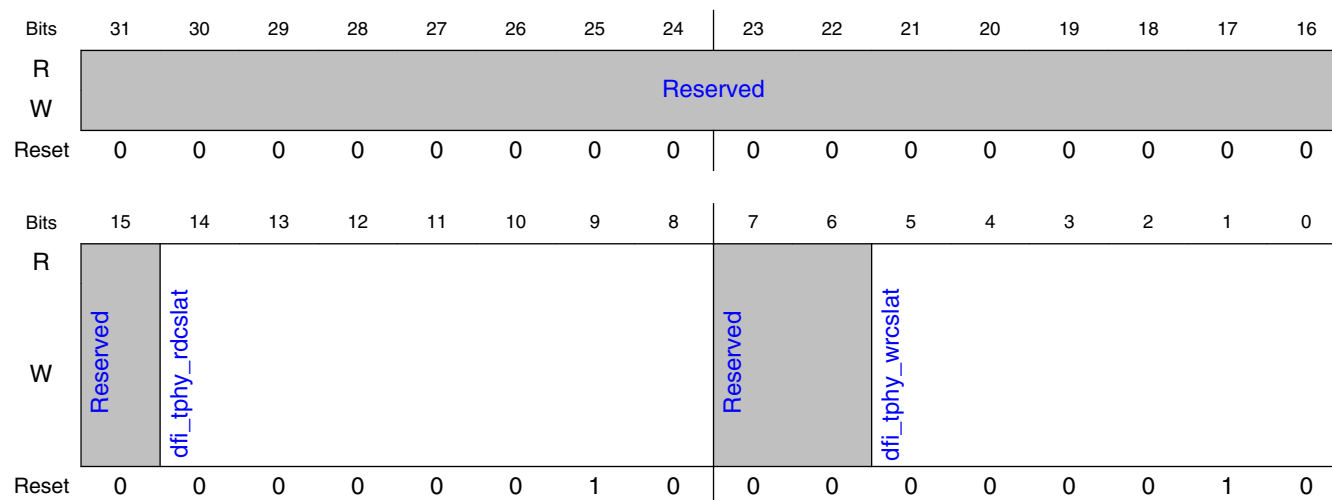
Field	Function
—	
2 dfi_data_cs_polarity	Defines polarity of dfi_wrdata_cs and dfi_rddata_cs signals. 0b - Signals are active low 1b - Signals are active high
1 phy_dbi_mode	DBI implemented in DDRC or PHY. Present only in designs configured to support DDR4 and LPDDR4. 0b - DDRC implements DBI functionality. 1b - PHY implements DBI functionality.
0 dfi_init_complete_en	PHY initialization complete enable signal. When asserted the dfi_init_complete signal can be used to trigger SDRAM initialisation

9.3.3.1.57 DFI Timing Register 2 (DFITMG2)

9.3.3.1.57.1 Offset

Register	Offset
DFITMG2	1B4h

9.3.3.1.57.2 Diagram



9.3.3.1.57.3 Fields

Field	Function
31-15	Reserved

Table continues on the next page...

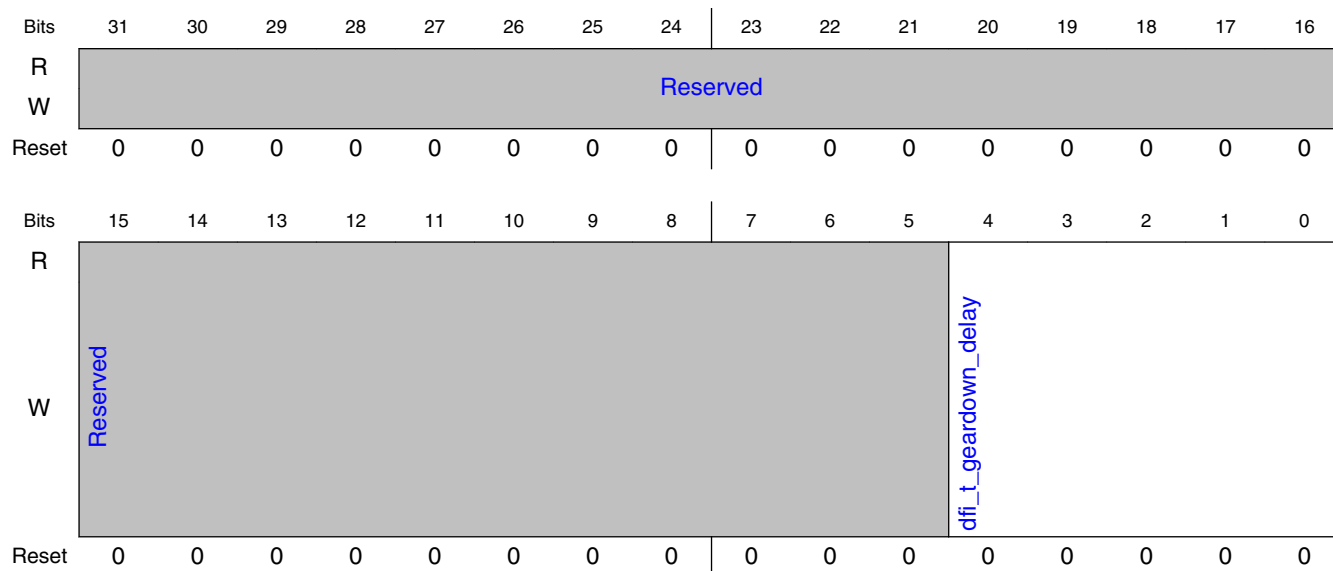
Field	Function
—	
14-8 dfi_tphy_rdcslat	Number of DFI PHY clock cycles between when a read command is sent on the DFI control interface and when the associated dfi_rddata_cs signal is asserted. This corresponds to the DFI timing parameter tphy_rdcslat. Refer to PHY specification for correct value.
7-6 —	Reserved
5-0 dfi_tphy_wrclat	Number of DFI PHY clock cycles between when a write command is sent on the DFI control interface and when the associated dfi_wrdata_cs signal is asserted. This corresponds to the DFI timing parameter tphy_wrclat. Refer to PHY specification for correct value.

9.3.3.1.58 DFI Timing Register 3 (DFITMG3)

9.3.3.1.58.1 Offset

Register	Offset
DFITMG3	1B8h

9.3.3.1.58.2 Diagram



9.3.3.1.58.3 Fields

Field	Function
31-5	Reserved

Table continues on the next page...

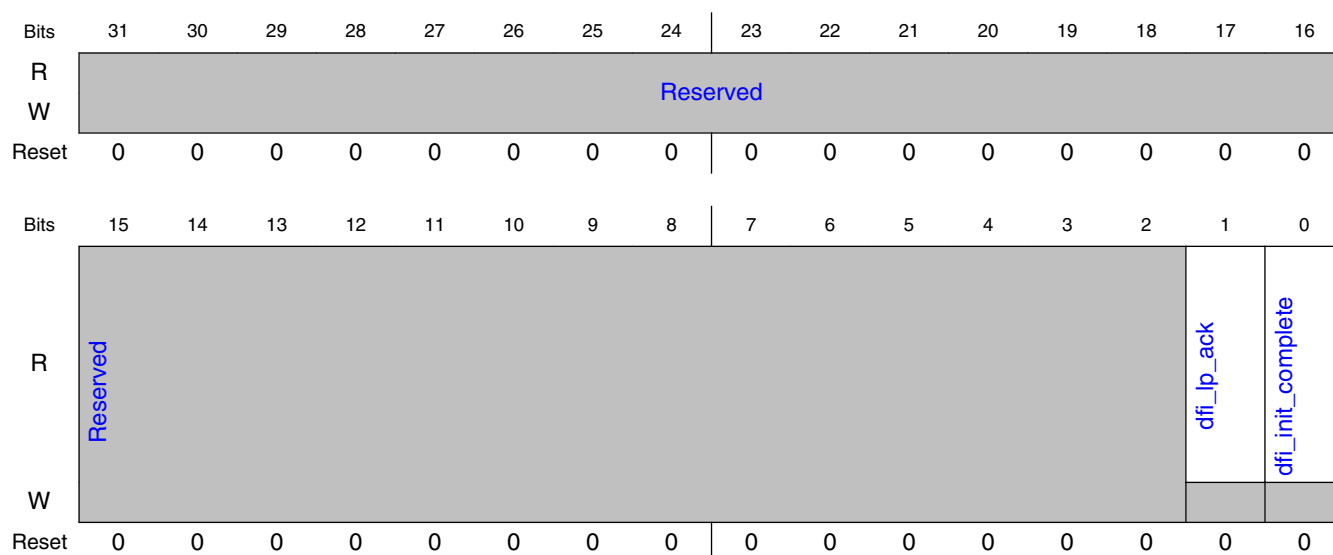
Field	Function
—	
4-0 dfi_t_geardown_delay	The delay from dfi_geardown_en assertion to the time of the PHY being ready to receive commands. Refer to PHY specification for correct value. When the controller is operating in 1:2 frequency ratio mode, program this to (tgeardown_delay/2) and round it up to the next integer value. Unit: Clocks

9.3.3.1.59 DFI Status Register (DFISTAT)

9.3.3.1.59.1 Offset

Register	Offset
DFISTAT	1BCh

9.3.3.1.59.2 Diagram



9.3.3.1.59.3 Fields

Field	Function
31-2 —	Reserved
1 dfi_lp_ack	Stores the value of the dfi_lp_ack input to the controller.

Table continues on the next page...

DDR Controller (DDRC)

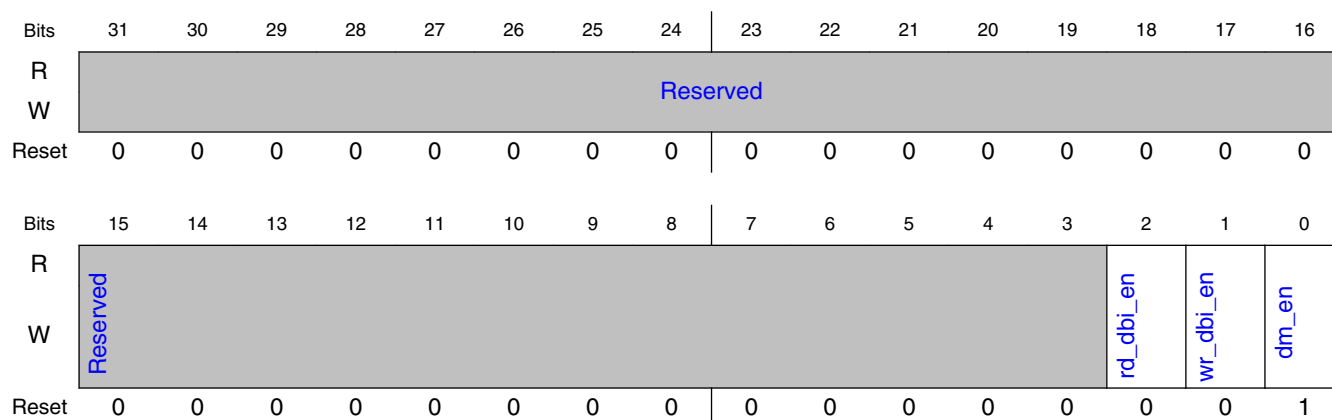
Field	Function
0 dfi_init_complete	The status flag register which announces when the DFI initialization has been completed. The DFI INIT triggered by dfi_init_start signal and then the dfi_init_complete flag is polled to know when the initialization is done.

9.3.3.1.60 DM/DBI Control Register (DBICTL)

9.3.3.1.60.1 Offset

Register	Offset
DBICTL	1C0h

9.3.3.1.60.2 Diagram



9.3.3.1.60.3 Fields

Field	Function
31-3 —	Reserved
2 rd_dbi_en	Read DBI enable signal in DDRC. - 0 - Read DBI is disabled. - 1 - Read DBI is enabled. This signal must be set the same value as DRAM's mode register. - DDR4: MR5 bit A12. When x4 devices are used, this signal must be set to 0. - LPDDR4: MR3[6]
1 wr_dbi_en	This signal must be set the same value as DRAM's mode register. - DDR4: MR5 bit A11. When x4 devices are used, this signal must be set to 0. - LPDDR4: MR3[7] 0b - Write DBI is disabled 1b - Write DBI is enabled.

Table continues on the next page...

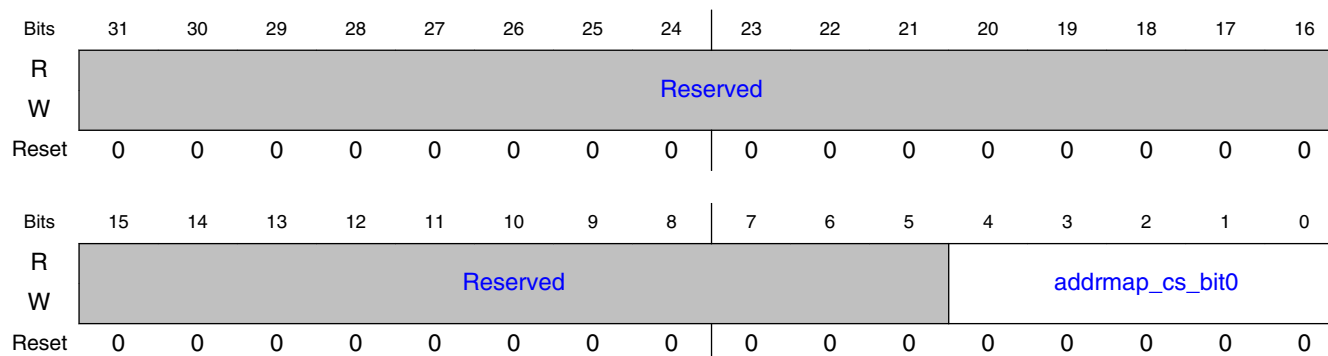
Field	Function
0 dm_en	DM enable signal in DDRC. This signal must be set the same logical value as DRAM's mode register. - DDR4: Set this to same value as MR5 bit A10. When x4 devices are used, this signal must be set to 0. - LPDDR4: Set this to inverted value of MR13[5] which is opposite polarity from this signal 0b - DM is disabled 1b - DM is enabled

9.3.3.1.61 Address Map Register 0 (ADDRMAP0)

9.3.3.1.61.1 Offset

Register	Offset
ADDRMAP0	200h

9.3.3.1.61.2 Diagram



9.3.3.1.61.3 Fields

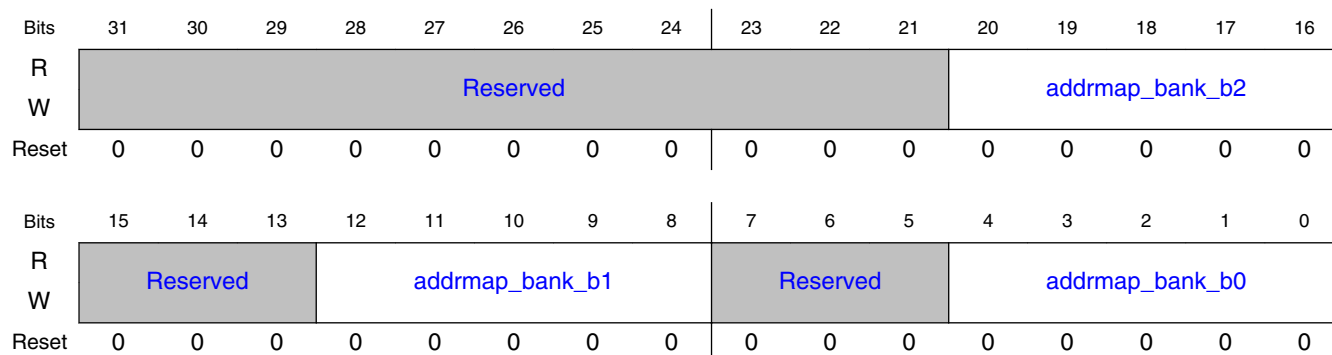
Field	Function
31-5 —	Reserved
4-0 addrmap_cs_bit0	Selects the HIF address bit used as rank address bit 0. Valid Range: 0 to 28, and 31 Internal Base: 6 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 31, rank address bit 0 is set to 0.

9.3.3.1.62 Address Map Register 1 (ADDRMAP1)

9.3.3.1.62.1 Offset

Register	Offset
ADDRMAP1	204h

9.3.3.1.62.2 Diagram



9.3.3.1.62.3 Fields

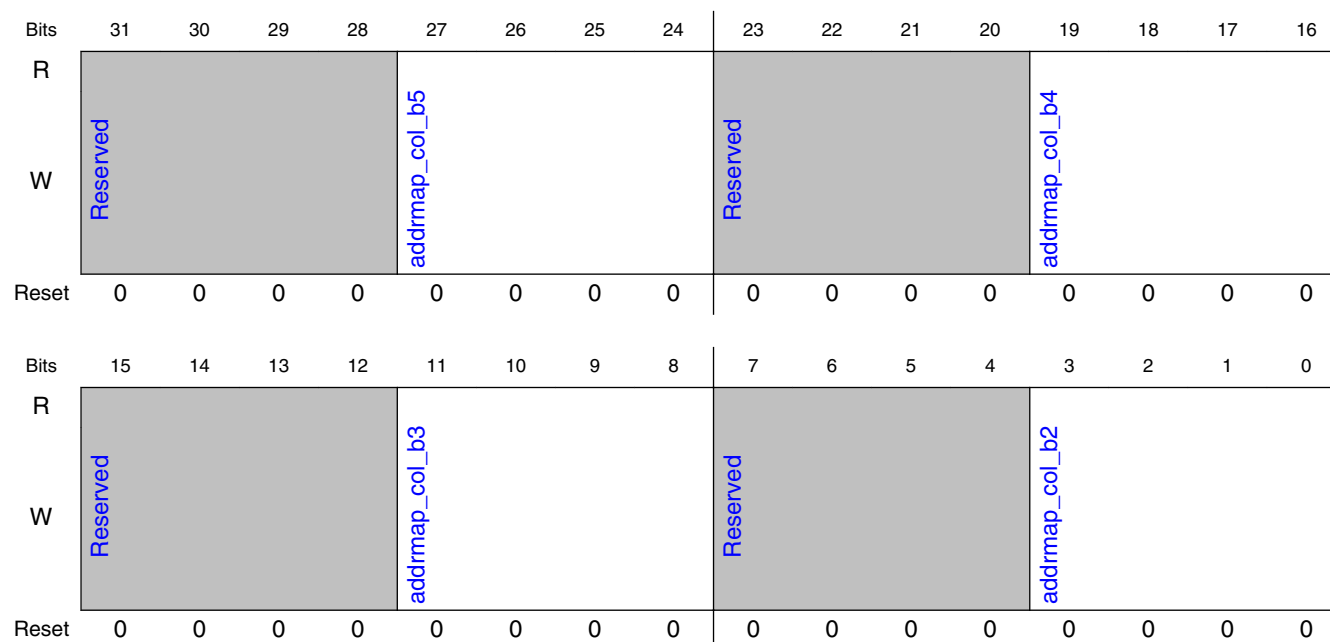
Field	Function
31-21 —	Reserved
20-16 addrmap_bank_b2	Selects the HIF address bit used as bank address bit 2. Valid Range: 0 to 30 and 31 Internal Base: 4 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 31, bank address bit 2 is set to 0.
15-13 —	Reserved
12-8 addrmap_bank_b1	Selects the HIF address bits used as bank address bit 1. Valid Range: 0 to 31 Internal Base: 3 The selected HIF address bit for each of the bank address bits is determined by adding the internal base to the value of this field.
7-5 —	Reserved
4-0 addrmap_bank_b0	Selects the HIF address bits used as bank address bit 0. Valid Range: 0 to 31 Internal Base: 2 The selected HIF address bit for each of the bank address bits is determined by adding the internal base to the value of this field.

9.3.3.1.63 Address Map Register 2 (ADDRMAP2)

9.3.3.1.63.1 Offset

Register	Offset
ADDRMAP2	208h

9.3.3.1.63.2 Diagram



9.3.3.1.63.3 Fields

Field	Function
31-28 —	Reserved
27-24 addrmap_col_b5	- Full bus width mode: Selects the HIF address bit used as column address bit 5. - Half bus width mode: Selects the HIF address bit used as column address bit 6. - Quarter bus width mode: Selects the HIF address bit used as column address bit 7. Valid Range: 0 to 7, and 15 Internal Base: 5 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, this column address bit is set to 0.
23-20 —	Reserved
19-16 addrmap_col_b4	- Full bus width mode: Selects the HIF address bit used as column address bit 4. - Half bus width mode: Selects the HIF address bit used as column address bit 5. - Quarter bus width mode: Selects the HIF address bit used as column address bit 6. Valid Range: 0 to 7, and 15 Internal Base: 4 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, this column address bit is set to 0.
15-12	Reserved

Table continues on the next page...

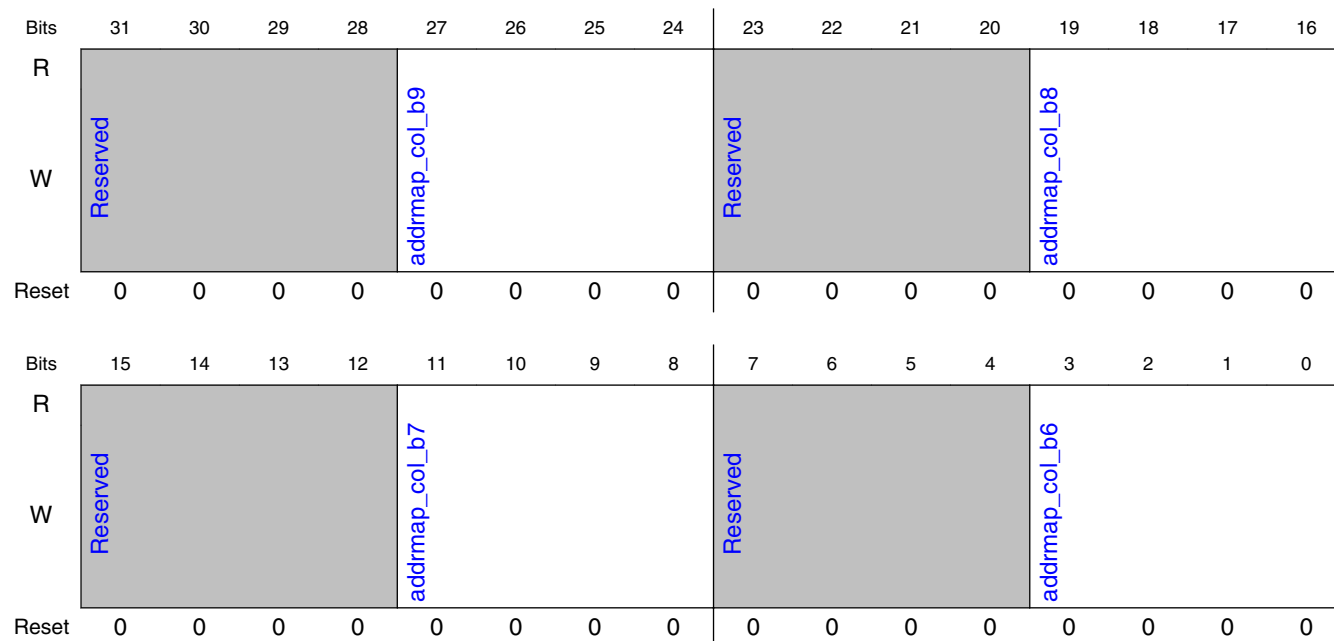
Field	Function
—	
11-8 addrmap_col_b3	- Full bus width mode: Selects the HIF address bit used as column address bit 3. - Half bus width mode: Selects the HIF address bit used as column address bit 4. - Quarter bus width mode: Selects the HIF address bit used as column address bit 5. Valid Range: 0 to 7 Internal Base: 3 The selected HIF address bit is determined by adding the internal base to the value of this field. Note, if DDRC_INCL_ARB=1, MEMC_BURST_LENGTH=16, Full bus width (MSTR.data_bus_width=00) and BL16 (MSTR.burst_rdw=1000), it is recommended to program this to 0.
7-4 —	Reserved
3-0 addrmap_col_b2	- Full bus width mode: Selects the HIF address bit used as column address bit 2. - Half bus width mode: Selects the HIF address bit used as column address bit 3. - Quarter bus width mode: Selects the HIF address bit used as column address bit 4. Valid Range: 0 to 7 Internal Base: 2 The selected HIF address bit is determined by adding the internal base to the value of this field. Note, if DDRC_INCL_ARB=1 and MEMC_BURST_LENGTH=8, it is required to program this to 0 unless: - in Half or Quarter bus width (MSTR.data_bus_width!=00) and - PCCFG.bl_exp_mode==1 and either - In DDR4 and ADDRMAP8.addrmap_bg_b0==0 or - In LPDDR4 and ADDRMAP1.addrmap_bank_b0==0 If DDRC_INCL_ARB=1 and MEMC_BURST_LENGTH=16, it is required to program this to 0 unless: - in Half or Quarter bus width (MSTR.data_bus_width!=00) and - PCCFG.bl_exp_mode==1 and - In DDR4 and ADDRMAP8.addrmap_bg_b0==0 Otherwise, if MEMC_BURST_LENGTH=8 and Full Bus Width (MSTR.data_bus_width==00), it is recommended to program this to 0 so that HIF[2] maps to column address bit 2. If MEMC_BURST_LENGTH=16 and Full Bus Width (MSTR.data_bus_width==00), it is recommended to program this to 0 so that HIF[2] maps to column address bit 2. If MEMC_BURST_LENGTH=16 and Half Bus Width (MSTR.data_bus_width==01), it is recommended to program this to 0 so that HIF[2] maps to column address bit 3.

9.3.3.1.64 Address Map Register 3 (ADDRMAP3)

9.3.3.1.64.1 Offset

Register	Offset
ADDRMAP3	20Ch

9.3.3.1.64.2 Diagram



9.3.3.1.64.3 Fields

Field	Function
31-28 —	Reserved
27-24 addrmap_col_b9	- Full bus width mode: Selects the HIF address bit used as column address bit 9. - Half bus width mode: Selects the HIF address bit used as column address bit 11 (10 in LPDDR2/LPDDR3 mode). - Quarter bus width mode: Selects the HIF address bit used as column address bit 13 (11 in LPDDR2/LPDDR3 mode). Valid Range: 0 to 7, and 15 Internal Base: 9 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, this column address bit is set to 0. Note: Per JEDEC DDR2/3/mDDR specification, column address bit 10 is reserved for indicating auto-precharge, and hence no source address bit can be mapped to column address bit 10. In LPDDR2/LPDDR3, there is a dedicated bit for auto-precharge in the CA bus and hence column bit 10 is used.
23-20 —	Reserved
19-16 addrmap_col_b8	- Full bus width mode: Selects the HIF address bit used as column address bit 8. - Half bus width mode: Selects the HIF address bit used as column address bit 9. - Quarter bus width mode: Selects the HIF address bit used as column address bit 11 (10 in LPDDR2/LPDDR3 mode). Valid Range: 0 to 7, and 15 Internal Base: 8 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, this column address bit is set to 0. Note: Per JEDEC DDR2/3/mDDR specification, column address bit 10 is reserved for indicating auto-precharge, and hence no source address bit can be mapped to column address bit 10. In LPDDR2/LPDDR3, there is a dedicated bit for auto-precharge in the CA bus and hence column bit 10 is used.
15-12 —	Reserved

Table continues on the next page...

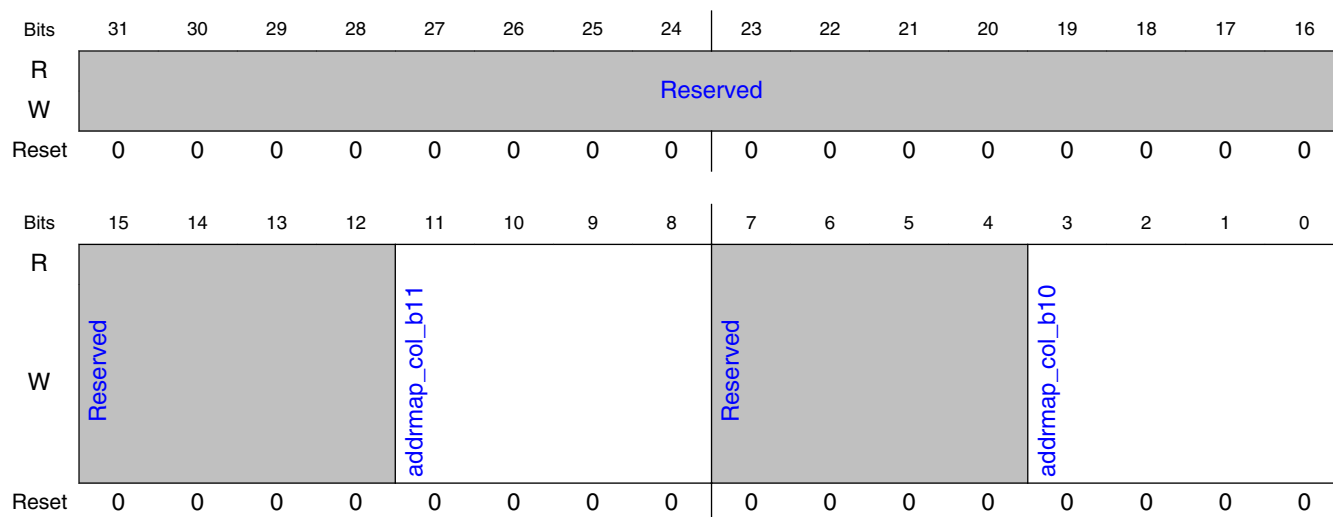
Field	Function
11-8 addrmap_col_b7	- Full bus width mode: Selects the HIF address bit used as column address bit 7. - Half bus width mode: Selects the HIF address bit used as column address bit 8. - Quarter bus width mode: Selects the HIF address bit used as column address bit 9. Valid Range: 0 to 7, and 15 Internal Base: 7 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, this column address bit is set to 0.
7-4 —	Reserved
3-0 addrmap_col_b6	- Full bus width mode: Selects the HIF address bit used as column address bit 6. - Half bus width mode: Selects the HIF address bit used as column address bit 7. - Quarter bus width mode: Selects the HIF address bit used as column address bit 8. Valid Range: 0 to 7, and 15 Internal Base: 6 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, this column address bit is set to 0.

9.3.3.1.65 Address Map Register 4 (ADDRMAP4)

9.3.3.1.65.1 Offset

Register	Offset
ADDRMAP4	210h

9.3.3.1.65.2 Diagram



9.3.3.1.65.3 Fields

Field	Function
31-12	Reserved

Table continues on the next page...

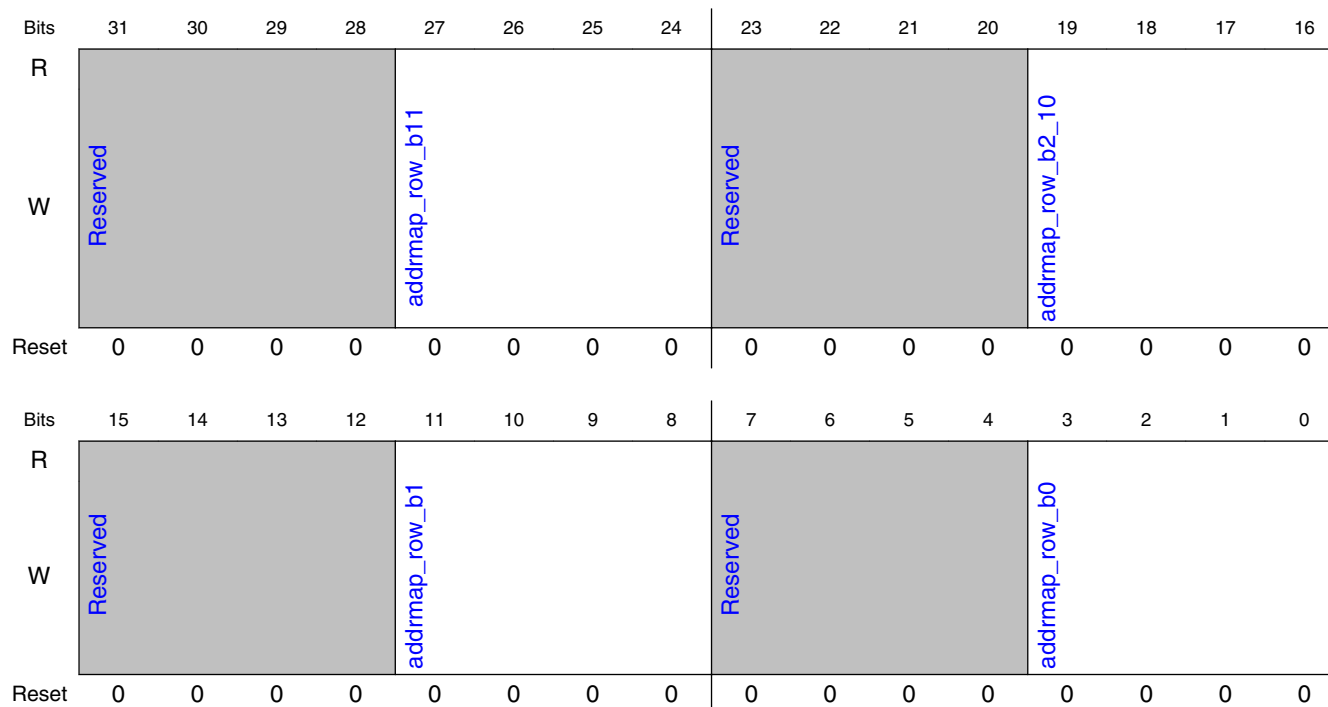
Field	Function
—	
11-8 addrmap_col_b1 1	- Full bus width mode: Selects the HIF address bit used as column address bit 13 (11 in LPDDR2/LPDDR3 mode). - Half bus width mode: Unused. To make it unused, this should be tied to 4'hF. - Quarter bus width mode: Unused. To make it unused, this must be tied to 4'hF. Valid Range: 0 to 7, and 15 Internal Base: 11 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, this column address bit is set to 0. Note: Per JEDEC DDR2/3/mDDR specification, column address bit 10 is reserved for indicating auto-precharge, and hence no source address bit can be mapped to column address bit 10. In LPDDR2/LPDDR3, there is a dedicated bit for auto-precharge in the CA bus and hence column bit 10 is used.
7-4 —	Reserved
3-0 addrmap_col_b1 0	- Full bus width mode: Selects the HIF address bit used as column address bit 11 (10 in LPDDR2/LPDDR3 mode). - Half bus width mode: Selects the HIF address bit used as column address bit 13 (11 in LPDDR2/LPDDR3 mode). - Quarter bus width mode: UNUSED. To make it unused, this must be tied to 4'hF. Valid Range: 0 to 7, and 15 Internal Base: 10 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, this column address bit is set to 0. Note: Per JEDEC DDR2/3/mDDR specification, column address bit 10 is reserved for indicating auto-precharge, and hence no source address bit can be mapped to column address bit 10. In LPDDR2/LPDDR3, there is a dedicated bit for auto-precharge in the CA bus and hence column bit 10 is used.

9.3.3.1.66 Address Map Register 5 (ADDRMAP5)

9.3.3.1.66.1 Offset

Register	Offset
ADDRMAP5	214h

9.3.3.1.66.2 Diagram



9.3.3.1.66.3 Fields

Field	Function
31-28 —	Reserved
27-24 <code>addrmap_row_b11</code>	Selects the HIF address bit used as row address bit 11. Valid Range: 0 to 11, and 15 Internal Base: 17 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, row address bit 11 is set to 0.
23-20 —	Reserved
19-16 <code>addrmap_row_b2_10</code>	Selects the HIF address bits used as row address bits 2 to 10. Valid Range: 0 to 11, and 15 Internal Base: 8 (for row address bit 2), 9 (for row address bit 3), 10 (for row address bit 4) etc increasing to 16 (for row address bit 10) The selected HIF address bit for each of the row address bits is determined by adding the internal base to the value of this field. When value 15 is used the values of row address bits 2 to 10 are defined by registers ADDRMAP9, ADDRMAP10, ADDRMAP11.
15-12 —	Reserved
11-8 <code>addrmap_row_b1</code>	Selects the HIF address bits used as row address bit 1. Valid Range: 0 to 11 Internal Base: 7 The selected HIF address bit for each of the row address bits is determined by adding the internal base to the value of this field.
7-4 —	Reserved

Table continues on the next page...

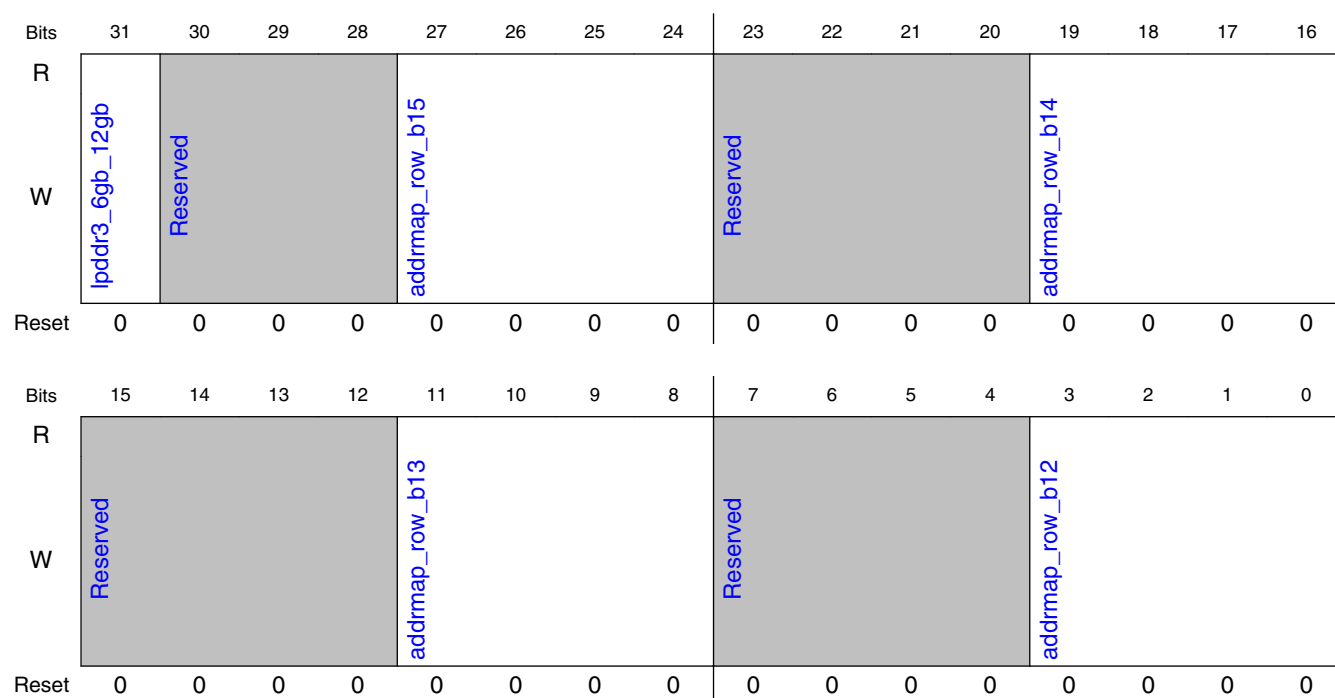
Field	Function
3-0 addrmap_row_b0	Selects the HIF address bits used as row address bit 0. Valid Range: 0 to 11 Internal Base: 6 The selected HIF address bit for each of the row address bits is determined by adding the internal base to the value of this field.

9.3.3.1.67 Address Map Register 6 (ADDRMAP6)

9.3.3.1.67.1 Offset

Register	Offset
ADDRMAP6	218h

9.3.3.1.67.2 Diagram



9.3.3.1.67.3 Fields

Field	Function
31 lpddr3_6gb_12gb	Set this to 1 if there is an LPDDR3 SDRAM 6Gb or 12Gb device in use. - 1 - LPDDR3 SDRAM 6Gb/12Gb device in use. Every address having row[14:13]==2'b11 is considered as invalid - 0 - non-LPDDR3 6Gb/12Gb device in use. All addresses are valid Present only in designs configured to support LPDDR3.

Table continues on the next page...

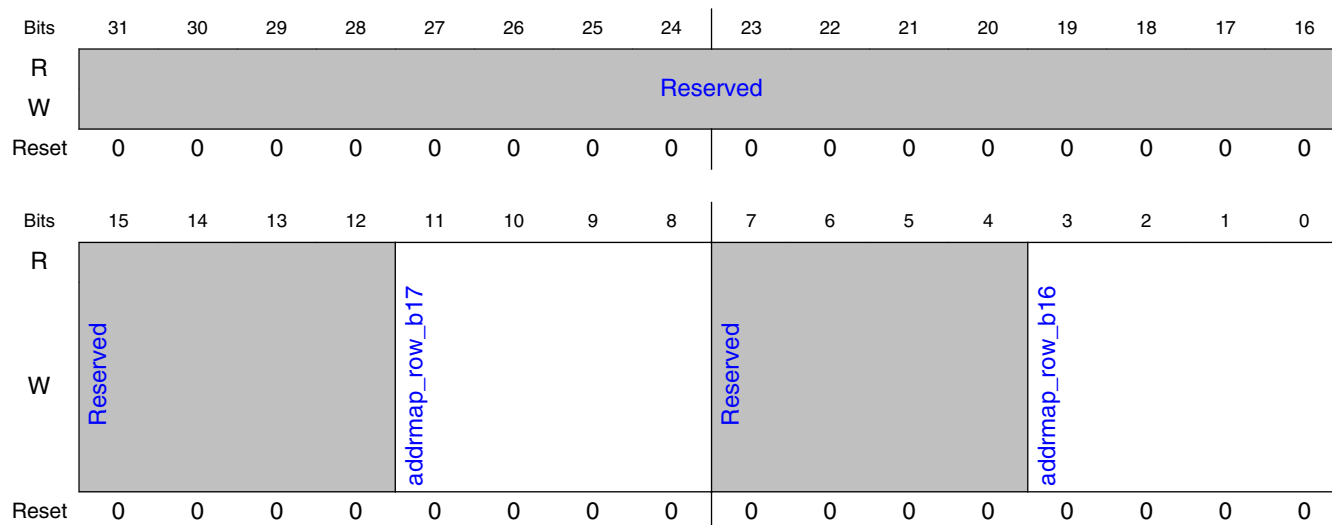
Field	Function
30-28 —	Reserved
27-24 addrmap_row_b 15	Selects the HIF address bit used as row address bit 15. Valid Range: 0 to 11, and 15 Internal Base: 21 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, row address bit 15 is set to 0.
23-20 —	Reserved
19-16 addrmap_row_b 14	Selects the HIF address bit used as row address bit 14. Valid Range: 0 to 11, and 15 Internal Base: 20 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, row address bit 14 is set to 0.
15-12 —	Reserved
11-8 addrmap_row_b 13	Selects the HIF address bit used as row address bit 13. Valid Range: 0 to 11, and 15 Internal Base: 19 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, row address bit 13 is set to 0.
7-4 —	Reserved
3-0 addrmap_row_b 12	Selects the HIF address bit used as row address bit 12. Valid Range: 0 to 11, and 15 Internal Base: 18 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, row address bit 12 is set to 0.

9.3.3.1.68 Address Map Register 7 (ADDRMAP7)

9.3.3.1.68.1 Offset

Register	Offset
ADDRMAP7	21Ch

9.3.3.1.68.2 Diagram



9.3.3.1.68.3 Fields

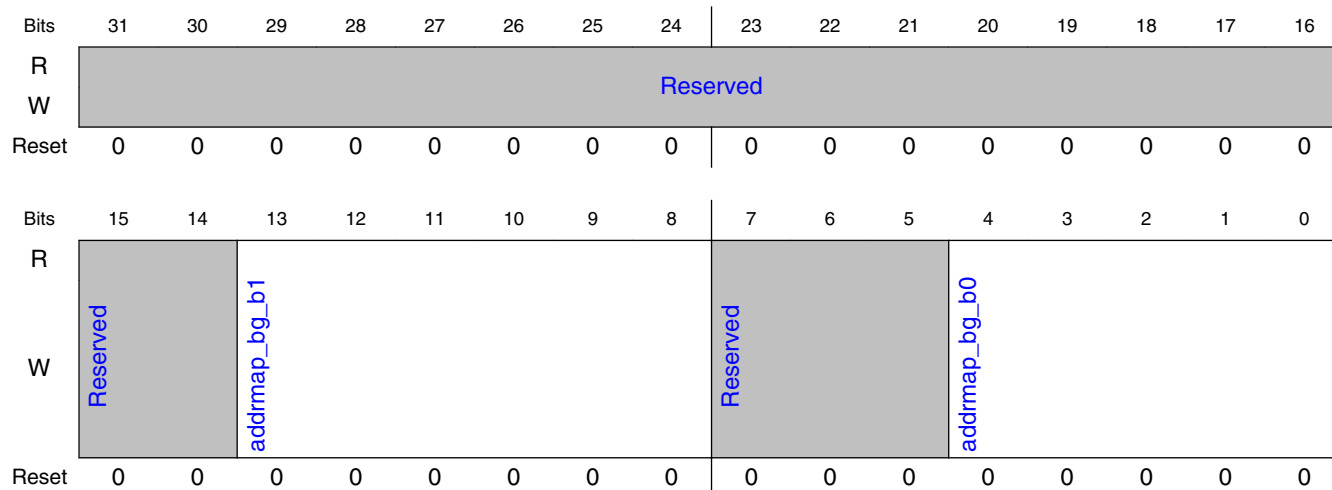
Field	Function
31-12 —	Reserved
11-8 addrmap_row_b17	Selects the HIF address bit used as row address bit 17. Valid Range: 0 to 11, and 15 Internal Base: 23 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, row address bit 17 is set to 0.
7-4 —	Reserved
3-0 addrmap_row_b16	Selects the HIF address bit used as row address bit 16. Valid Range: 0 to 11, and 15 Internal Base: 22 The selected HIF address bit is determined by adding the internal base to the value of this field. If set to 15, row address bit 16 is set to 0.

9.3.3.1.69 Address Map Register 8 (ADDRMAP8)

9.3.3.1.69.1 Offset

Register	Offset
ADDRMAP8	220h

9.3.3.1.69.2 Diagram



9.3.3.1.69.3 Fields

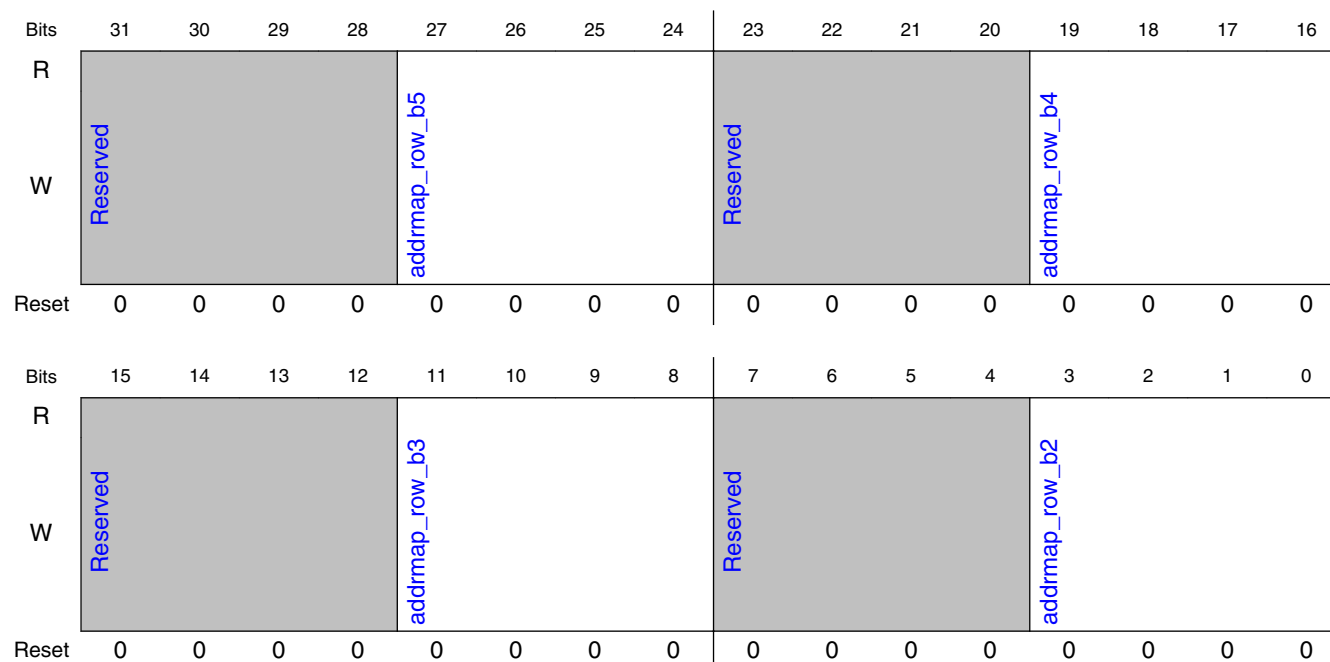
Field	Function
31-14 —	Reserved
13-8 addrmap_bg_b1	Selects the HIF address bits used as bank group address bit 1. Valid Range: 0 to 31, and 63 Internal Base: 3 The selected HIF address bit for each of the bank group address bits is determined by adding the internal base to the value of this field. If set to 63, bank group address bit 1 is set to 0.
7-5 —	Reserved
4-0 addrmap_bg_b0	Selects the HIF address bits used as bank group address bit 0. Valid Range: 0 to 31 Internal Base: 2 The selected HIF address bit for each of the bank group address bits is determined by adding the internal base to the value of this field.

9.3.3.1.70 Address Map Register 9 (ADDRMAP9)

9.3.3.1.70.1 Offset

Register	Offset
ADDRMAP9	224h

9.3.3.1.70.2 Diagram



9.3.3.1.70.3 Fields

Field	Function
31-28 —	Reserved
27-24 addrmap_row_b5	Selects the HIF address bits used as row address bit 5. Valid Range: 0 to 11 Internal Base: 11 The selected HIF address bit for each of the row address bits is determined by adding the internal base to the value of this field. This register field is used only when ADDRMAP5.addrmap_row_b2_10 is set to value 15.
23-20 —	Reserved
19-16 addrmap_row_b4	Selects the HIF address bits used as row address bit 4. Valid Range: 0 to 11 Internal Base: 10 The selected HIF address bit for each of the row address bits is determined by adding the internal base to the value of this field. This register field is used only when ADDRMAP5.addrmap_row_b2_10 is set to value 15.
15-12 —	Reserved
11-8 addrmap_row_b3	Selects the HIF address bits used as row address bit 3. Valid Range: 0 to 11 Internal Base: 9 The selected HIF address bit for each of the row address bits is determined by adding the internal base to the value of this field. This register field is used only when ADDRMAP5.addrmap_row_b2_10 is set to value 15.
7-4 —	Reserved

Table continues on the next page...

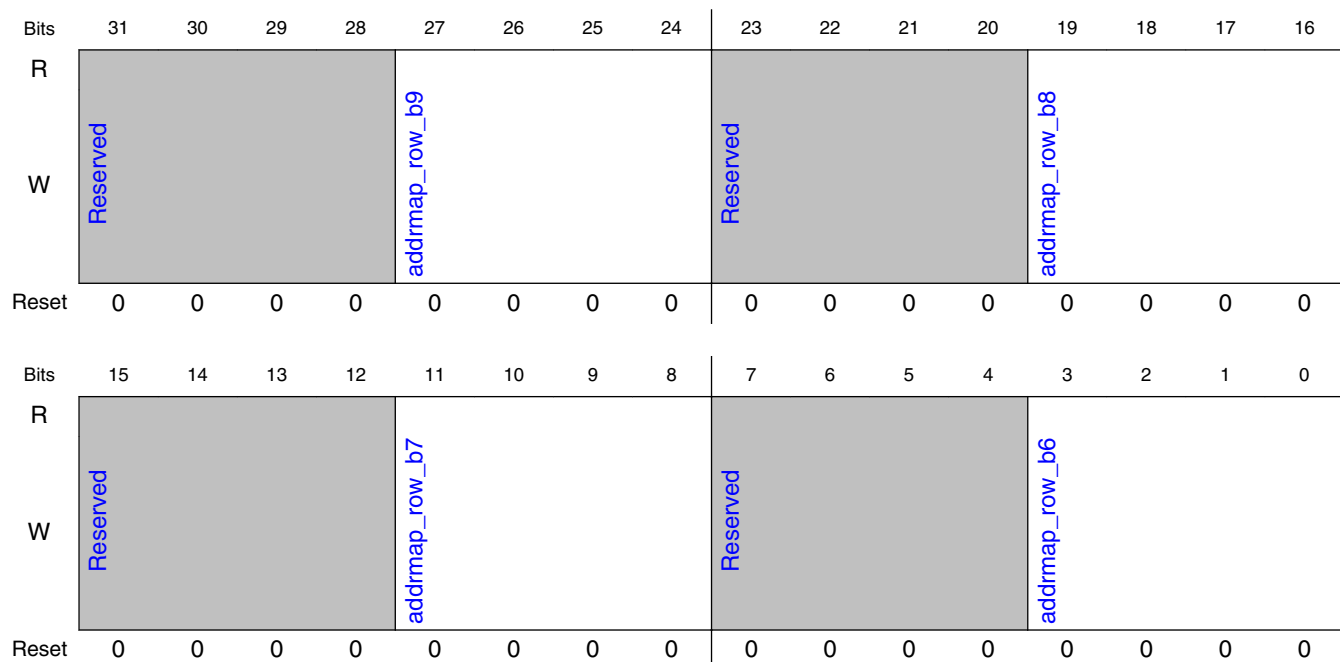
Field	Function
3-0 addrmap_row_b2	Selects the HIF address bits used as row address bit 2. Valid Range: 0 to 11 Internal Base: 8 The selected HIF address bit for each of the row address bits is determined by adding the internal base to the value of this field. This register field is used only when ADDRMAP5.addrmap_row_b2_10 is set to value 15.

9.3.3.1.71 Address Map Register 10 (ADDRMAP10)

9.3.3.1.71.1 Offset

Register	Offset
ADDRMAP10	228h

9.3.3.1.71.2 Diagram



9.3.3.1.71.3 Fields

Field	Function
31-28 —	Reserved

Table continues on the next page...

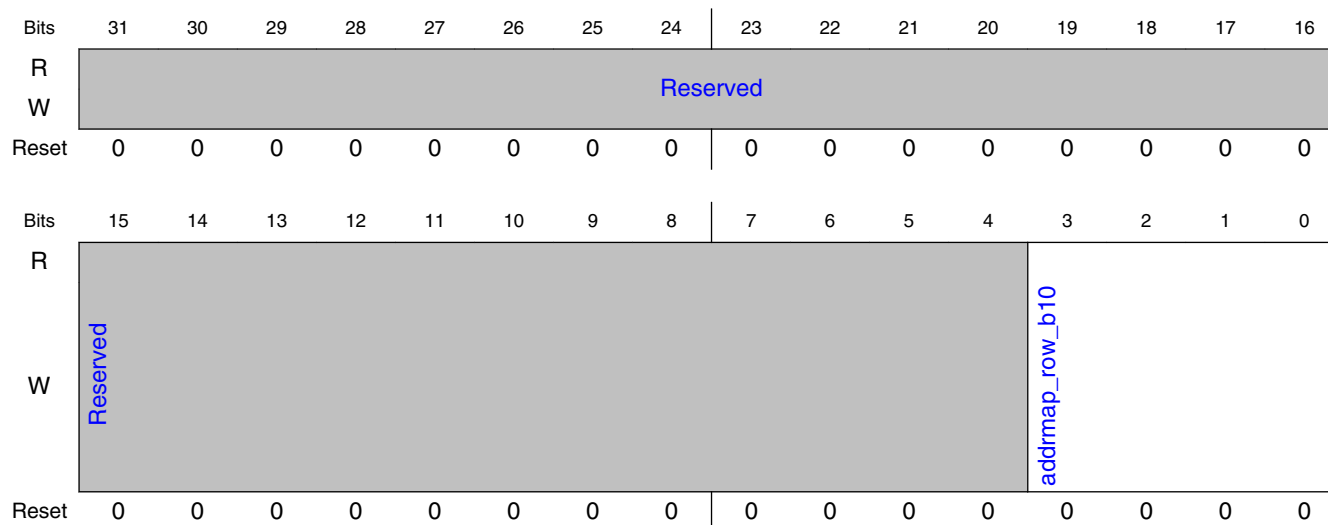
Field	Function
27-24 addrmap_row_b 9	Selects the HIF address bits used as row address bit 9. Valid Range: 0 to 11 Internal Base: 15 The selected HIF address bit for each of the row address bits is determined by adding the internal base to the value of this field. This register field is used only when ADDRMAP5.addrmap_row_b2_10 is set to value 15.
23-20 —	Reserved
19-16 addrmap_row_b 8	Selects the HIF address bits used as row address bit 8. Valid Range: 0 to 11 Internal Base: 14 The selected HIF address bit for each of the row address bits is determined by adding the internal base to the value of this field. This register field is used only when ADDRMAP5.addrmap_row_b2_10 is set to value 15.
15-12 —	Reserved
11-8 addrmap_row_b 7	Selects the HIF address bits used as row address bit 7. Valid Range: 0 to 11 Internal Base: 13 The selected HIF address bit for each of the row address bits is determined by adding the internal base to the value of this field. This register field is used only when ADDRMAP5.addrmap_row_b2_10 is set to value 15.
7-4 —	Reserved
3-0 addrmap_row_b 6	Selects the HIF address bits used as row address bit 6. Valid Range: 0 to 11 Internal Base: 12 The selected HIF address bit for each of the row address bits is determined by adding the internal base to the value of this field. This register field is used only when ADDRMAP5.addrmap_row_b2_10 is set to value 15.

9.3.3.1.72 Address Map Register 11 (ADDRMAP11)

9.3.3.1.72.1 Offset

Register	Offset
ADDRMAP11	22Ch

9.3.3.1.72.2 Diagram



9.3.3.1.72.3 Fields

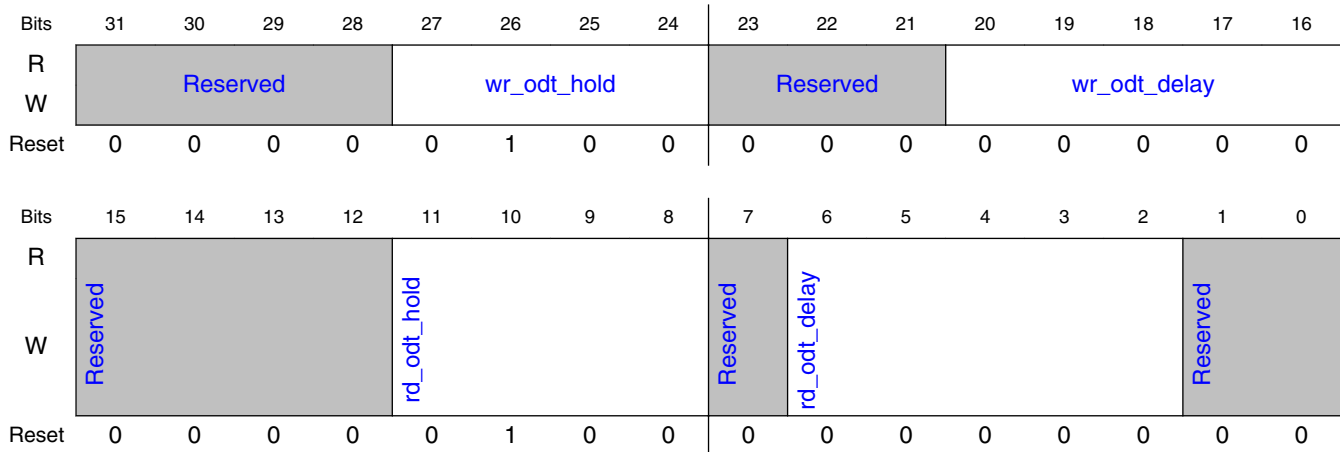
Field	Function
31-4 —	Reserved
3-0 addrmap_row_b10	Selects the HIF address bits used as row address bit 10. Valid Range: 0 to 11 Internal Base: 16 The selected HIF address bit for each of the row address bits is determined by adding the internal base to the value of this field. This register field is used only when ADDRMAP5.addrmap_row_b2_10 is set to value 15.

9.3.3.1.73 ODT Configuration Register (ODTCFG)

9.3.3.1.73.1 Offset

Register	Offset
ODTCFG	240h

9.3.3.1.73.2 Diagram



9.3.3.1.73.3 Fields

Field	Function
31-28 —	Reserved
27-24 wr_odt_hold	DFI PHY clock cycles to hold ODT for a write command. The minimum supported value is 2. Recommended values: DDR2: - BL8: 0x5 (DDR2-400/533/667), 0x6 (DDR2-800), 0x7 (DDR2-1066) - BL4: 0x3 (DDR2-400/533/667), 0x4 (DDR2-800), 0x5 (DDR2-1066) DDR3: - BL8: 0x6 DDR4: - BL8: 5 + WR_PREAMBLE + CRC_MODE WR_PREAMBLE = 1 (1tCK write preamble), 2 (2tCK write preamble) CRC_MODE = 0 (not CRC mode), 1 (CRC mode) LPDDR3: - BL8: 7 + RU(tODTOn(max)/tCK)
23-21 —	Reserved
20-16 wr_odt_delay	The delay, in DFI PHY clock cycles, from issuing a write command to setting ODT values associated with that command. ODT setting must remain constant for the entire time that DQS is driven by the DDRC. Recommended values: DDR2: - CWL + AL - 3 (DDR2-400/533/667), CWL + AL - 4 (DDR2-800), CWL + AL - 5 (DDR2-1066) If (CWL + AL - 3 < 0), DDRC does not support ODT for write operation. DDR3: - 0x0 DDR4: - DFITMG1.dfi_t_cmd_lat (to adjust for CAL mode) LPDDR3: - WL - 1 - RU(tODTOn(max)/tCK)
15-12 —	Reserved
11-8 rd_odt_hold	DFI PHY clock cycles to hold ODT for a read command. The minimum supported value is 2. Recommended values: DDR2: - BL8: 0x6 (not DDR2-1066), 0x7 (DDR2-1066) - BL4: 0x4 (not DDR2-1066), 0x5 (DDR2-1066) DDR3: - BL8 - 0x6 DDR4: - BL8: 5 + RD_PREAMBLE RD_PREAMBLE = 1 (1tCK write preamble), 2 (2tCK write preamble) LPDDR3: - BL8: 5 + RU(tDQSCK(max)/tCK) - RD(tDQSCK(min)/tCK) + RU(tODTOn(max)/tCK)
7 —	Reserved
6-2 rd_odt_delay	The delay, in DFI PHY clock cycles, from issuing a read command to setting ODT values associated with that command. ODT setting must remain constant for the entire time that DQS is driven by the DDRC. Recommended values: DDR2: - CL + AL - 4 (not DDR2-1066), CL + AL - 5 (DDR2-1066) If (CL + AL - 4 < 0), DDRC does not support ODT for read operation. DDR3: - CL - CWL DDR4: - CL - CWL - RD_PREAMBLE + WR_PREAMBLE + DFITMG1.dfi_t_cmd_lat (to adjust for CAL mode) WR_PREAMBLE = 1 (1tCK write preamble), 2 (2tCK write preamble) RD_PREAMBLE = 1 (1tCK write

Table continues on the next page...

DDR Controller (DDRC)

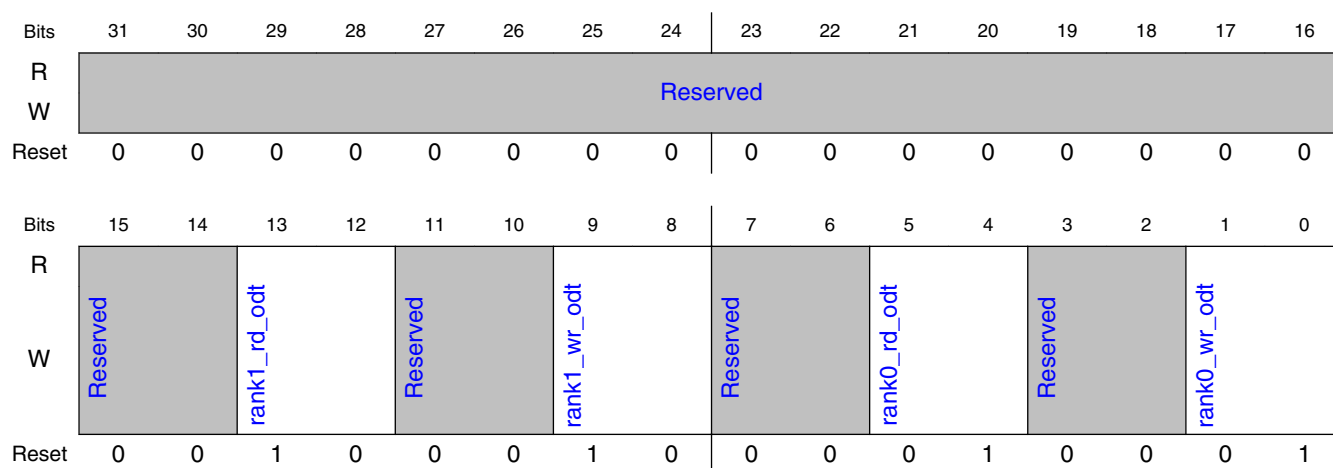
Field	Function
	preamble), 2 (2tCK write preamble) If (CL - CWL - RD_PREAMBLE + WR_PREAMBLE) < 0, DDRC does not support ODT for read operation. LPDDR3: - RL + RD(tDQSK(min)/tCK) - 1 - RU(tODTOn(max)/tCK)
1-0 —	Reserved

9.3.3.1.74 ODT/Rank Map Register (ODTMAP)

9.3.3.1.74.1 Offset

Register	Offset
ODTMAP	244h

9.3.3.1.74.2 Diagram



9.3.3.1.74.3 Fields

Field	Function
31-14 —	Reserved
13-12 rank1_rd_odt	Indicates which remote ODTs must be turned on during a read from rank 1. Each rank has a remote ODT (in the SDRAM) which can be turned on by setting the appropriate bit here. Rank 0 is controlled by the LSB; rank 1 is controlled by bit next to the LSB, etc. For each rank, set its bit to 1 to enable its ODT. Present only in configurations that have 2 or more ranks
11-10 —	Reserved

Table continues on the next page...

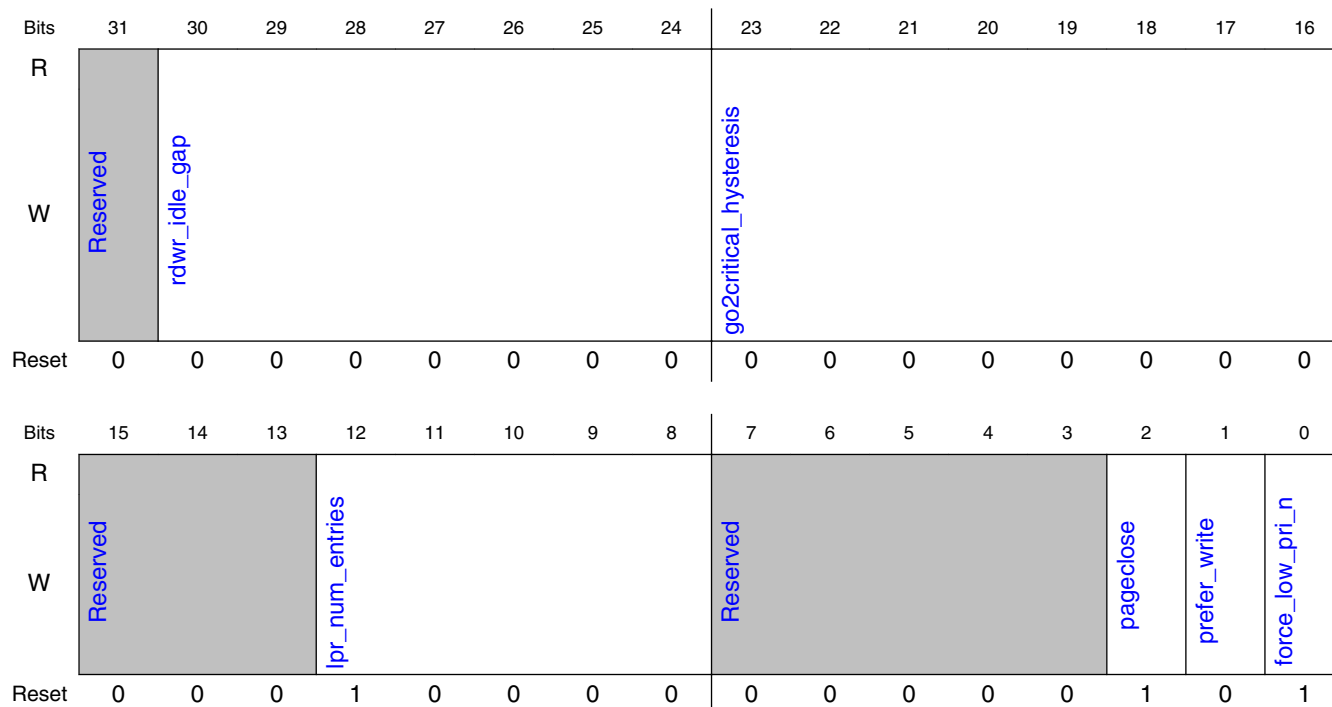
Field	Function
9-8 rank1_wr_odt	Indicates which remote ODTs must be turned on during a write to rank 1. Each rank has a remote ODT (in the SDRAM) which can be turned on by setting the appropriate bit here. Rank 0 is controlled by the LSB; rank 1 is controlled by bit next to the LSB, etc. For each rank, set its bit to 1 to enable its ODT. Present only in configurations that have 2 or more ranks
7-6 —	Reserved
5-4 rank0_rd_odt	Indicates which remote ODTs must be turned on during a read from rank 0. Each rank has a remote ODT (in the SDRAM) which can be turned on by setting the appropriate bit here. Rank 0 is controlled by the LSB; rank 1 is controlled by bit next to the LSB, etc. For each rank, set its bit to 1 to enable its ODT.
3-2 —	Reserved
1-0 rank0_wr_odt	Indicates which remote ODTs must be turned on during a write to rank 0. Each rank has a remote ODT (in the SDRAM) which can be turned on by setting the appropriate bit here. Rank 0 is controlled by the LSB; rank 1 is controlled by bit next to the LSB, etc. For each rank, set its bit to 1 to enable its ODT.

9.3.3.1.75 Scheduler Control Register (SCHED)

9.3.3.1.75.1 Offset

Register	Offset
SCHED	250h

9.3.3.1.75.2 Diagram



9.3.3.1.75.3 Fields

Field	Function
31 —	Reserved
30-24 rdwr_idle_gap	When the preferred transaction store is empty for these many clock cycles, switch to the alternate transaction store if it is non-empty. The read transaction store (both high and low priority) is the default preferred transaction store and the write transaction store is the alternative store. When prefer write over read is set this is reversed. 0x0 is a legal value for this register. When set to 0x0, the transaction store switching will happen immediately when the switching conditions become true. FOR PERFORMANCE ONLY
23-16 go2critical_hysteresis	UNUSED
15-13 —	Reserved
12-8 lpr_num_entries	Number of entries in the low priority transaction store is this value + 1. (MEMC_NO_OF_ENTRY - (SCHED.lpr_num_entries + 1)) is the number of entries available for the high priority transaction store. Setting this to maximum value allocates all entries to low priority transaction store. Setting this to 0 allocates 1 entry to low priority transaction store and the rest to high priority transaction store. Note: In ECC configurations, the numbers of write and low priority read credits issued is one less than in the non-ECC case. One entry each is reserved in the write and low-priority read CAMs for storing the RMW requests arising out of single bit error correction RMW operation.
7-3	Reserved

Table continues on the next page...

Field	Function
—	
2 pageclose	If true, bank is kept open only while there are page hit transactions available in the CAM to that bank. The last read or write command in the CAM with a bank and page hit will be executed with auto-precharge if SCHED1.pageclose_timer=0. Even if this register set to 1 and SCHED1.pageclose_timer is set to 0, explicit precharge (and not auto-precharge) may be issued in some cases where there is a mode switch between Write and Read or between LPR and HPR. The Read and Write commands that are executed as part of the ECC scrub requests are also executed without auto-precharge. If false, the bank remains open until there is a need to close it (to open a different page, or for page timeout or refresh timeout) - also known as open page policy. The open page policy can be overridden by setting the per-command-autopre bit on the HIF interface (hif_cmd_autopre). The pageclose feature provides a midway between Open and Close page policies. FOR PERFORMANCE ONLY.
1 prefer_write	If set then the bank selector prefers writes over reads. FOR DEBUG ONLY.
0 force_low_pri_n	Active low signal. When asserted ('0'), all incoming transactions are forced to low priority. This implies that all High Priority Read (HPR) and Variable Priority Read commands (VPR) will be treated as Low Priority Read (LPR) commands. On the write side, all Variable Priority Write (VPW) commands will be treated as Normal Priority Write (NPW) commands. Forcing the incoming transactions to low priority implicitly turns off Bypass path for read commands. FOR PERFORMANCE ONLY.

9.3.3.1.76 Scheduler Control Register 1 (SCHED1)

9.3.3.1.76.1 Offset

Register	Offset
SCHED1	254h

9.3.3.1.76.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								pageclose_timer							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.3.3.1.76.3 Fields

Field	Function
31-8 —	Reserved
7-0 pageclose_timer	This field works in conjunction with SCHED.pageclose. It only has meaning if SCHED.pageclose==1. If SCHED.pageclose==1 and pageclose_timer==0, then an auto-precharge may be scheduled for last read or write command in the CAM with a bank and page hit. Note, sometimes an explicit precharge is scheduled instead of the auto-precharge. See SCHED.pageclose for details of when this may happen. If SCHED.pageclose==1 and pageclose_timer>0, then an auto-precharge is not scheduled for last read or write command in the CAM with a bank and page hit. Instead, a timer is started, with pageclose_timer as the initial value. There is a timer on a per bank basis. The timer decrements unless the next read or write in the CAM to a bank is a page hit. It gets reset to pageclose_timer value if the next read or write in the CAM to a bank is a page hit. Once the timer has reached zero, an explicit precharge will be attempted to be scheduled.

9.3.3.1.77 High Priority Read CAM Register 1 (PERFHPR1)

9.3.3.1.77.1 Offset

Register	Offset
PERFHPR1	25Ch

9.3.3.1.77.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	hpr_xact_run_length								Reserved							
W																
Reset	0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	hpr_max_starve															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

9.3.3.1.77.3 Fields

Field	Function
31-24	Number of transactions that are serviced once the HPR queue goes critical is the smaller of: - (a) This number - (b) Number of transactions available. Unit: Transaction. FOR PERFORMANCE ONLY.

Table continues on the next page...

Field	Function
hpr_xact_run_length	
23-16 —	Reserved
15-0 hpr_max_starve	Number of DFI clocks that the HPR queue can be starved before it goes critical. The minimum valid functional value for this register is 0x1. Programming it to 0x0 will disable the starvation functionality; during normal operation, this function should not be disabled as it will cause excessive latencies. FOR PERFORMANCE ONLY.

9.3.3.1.78 Low Priority Read CAM Register 1 (PERFLPR1)

9.3.3.1.78.1 Offset

Register	Offset
PERFLPR1	264h

9.3.3.1.78.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	lpr_xact_run_length								Reserved							
W																
Reset	0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	lpr_max_starve															
W																
Reset	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1

9.3.3.1.78.3 Fields

Field	Function
31-24 lpr_xact_run_length	Number of transactions that are serviced once the LPR queue goes critical is the smaller of: - (a) This number - (b) Number of transactions available. Unit: Transaction. FOR PERFORMANCE ONLY.
23-16 —	Reserved
15-0 lpr_max_starve	Number of DFI clocks that the LPR queue can be starved before it goes critical. The minimum valid functional value for this register is 0x1. Programming it to 0x0 will disable the starvation functionality;

DDR Controller (DDRC)

Field	Function
	during normal operation, this function should not be disabled as it will cause excessive latencies. FOR PERFORMANCE ONLY.

9.3.3.1.79 Write CAM Register 1 (PERFWR1)

9.3.3.1.79.1 Offset

Register	Offset
PERFWR1	26Ch

9.3.3.1.79.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	w_xact_run_length								Reserved							
W																
Reset	0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	w_max_starve															
W																
Reset	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1

9.3.3.1.79.3 Fields

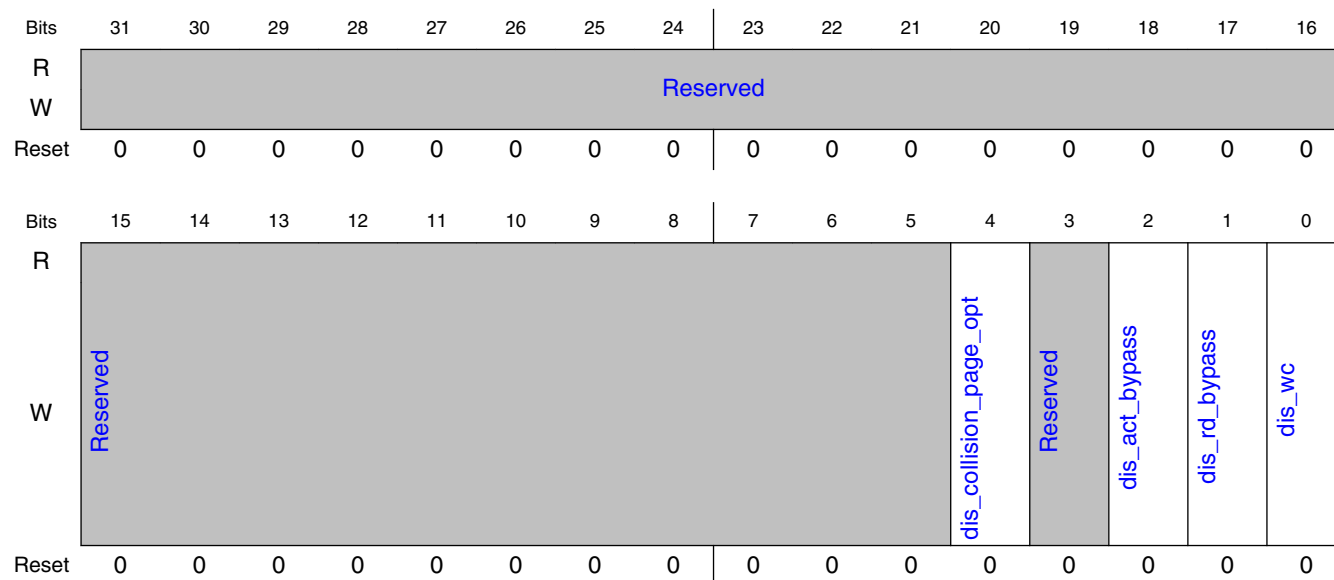
Field	Function
31-24 w_xact_run_length	Number of transactions that are serviced once the WR queue goes critical is the smaller of: - (a) This number - (b) Number of transactions available. Unit: Transaction. FOR PERFORMANCE ONLY.
23-16 —	Reserved
15-0 w_max_starve	Number of DFI clocks that the WR queue can be starved before it goes critical. The minimum valid functional value for this register is 0x1. Programming it to 0x0 will disable the starvation functionality; during normal operation, this function should not be disabled as it will cause excessive latencies. FOR PERFORMANCE ONLY.

9.3.3.1.80 Debug Register 0 (DBG0)

9.3.3.1.80.1 Offset

Register	Offset
DBG0	300h

9.3.3.1.80.2 Diagram



9.3.3.1.80.3 Fields

Field	Function
31-5 —	Reserved
4 dis_collision_page_opt	When this is set to '0', auto-precharge is disabled for the flushed command in a collision case. Collision cases are write followed by read to same address, read followed by write to same address, or write followed by write to same address with DBG0.dis_wc bit = 1 (where same address comparisons exclude the two address bits representing critical word). FOR DEBUG ONLY.
3 —	Reserved
2 dis_act_bypass	Only present in designs supporting activate bypass. When 1, disable bypass path for high priority read activates FOR DEBUG ONLY.
1 dis_rd_bypass	Only present in designs supporting read bypass. When 1, disable bypass path for high priority read page hits FOR DEBUG ONLY.
0 dis_wc	When 1, disable write combine. FOR DEBUG ONLY

9.3.3.1.81 Debug Register 1 (DBG1)

9.3.3.1.81.1 Offset

Register	Offset
DBG1	304h

9.3.3.1.81.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved														dis_hif	dis_dq
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.3.3.1.81.3 Fields

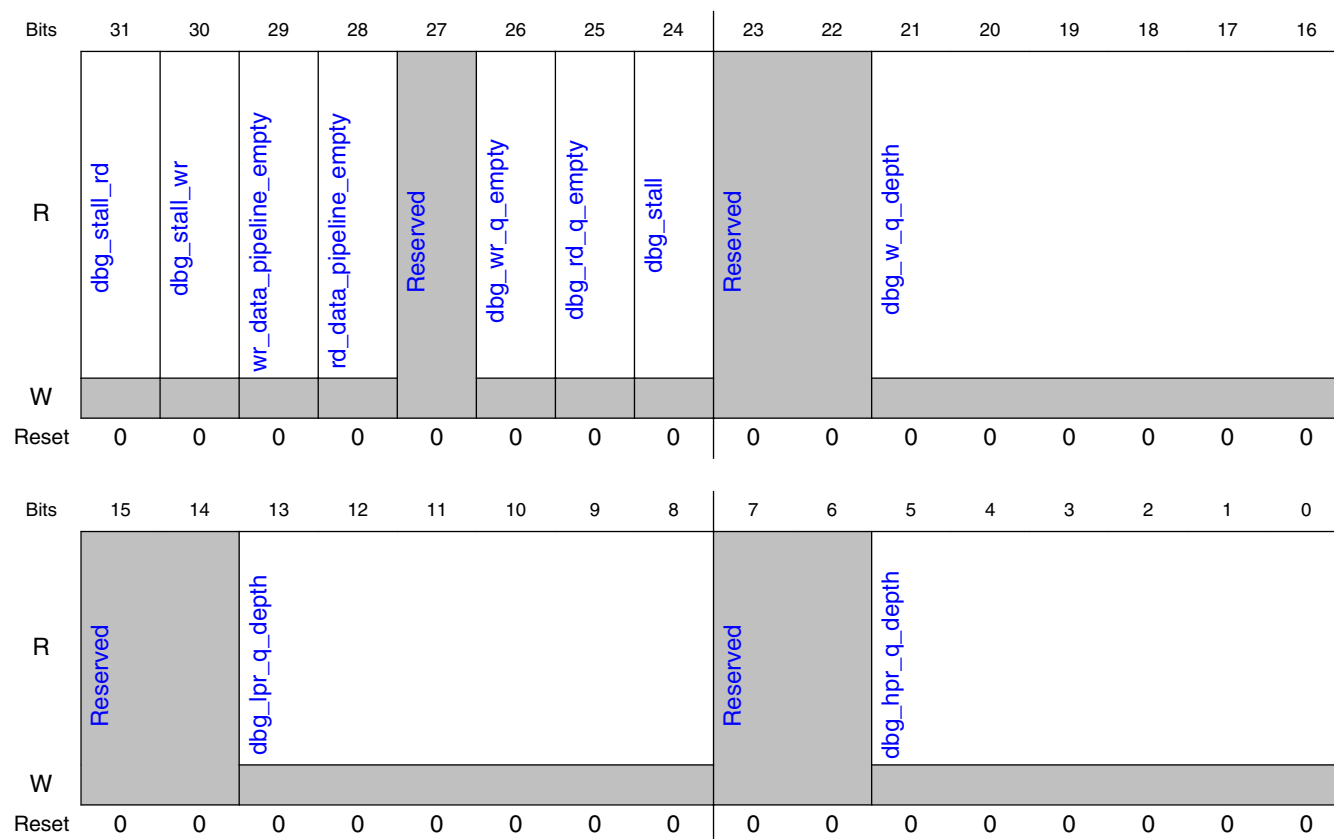
Field	Function
31-2 —	Reserved
1 dis_hif	When 1, DDRC asserts the HIF command signal hif_cmd_stall. DDRC will ignore the hif_cmd_valid and all other associated request signals. This bit is intended to be switched on-the-fly.
0 dis_dq	When 1, DDRC will not de-queue any transactions from the CAM. Bypass is also disabled. All transactions are queued in the CAM. No reads or writes are issued to SDRAM as long as this is asserted. This bit may be used to prevent reads or writes being issued by the DDRC, which makes it safe to modify certain register fields associated with reads and writes (see User Guide for details). After setting this bit, it is strongly recommended to poll DBGCAM.wr_data_pipeline_empty and DBGCAM.rd_data_pipeline_empty, before making changes to any registers which affect reads and writes. This will ensure that the relevant logic in the DDRC is idle. This bit is intended to be switched on-the-fly.

9.3.3.1.82 CAM Debug Register (DBGCAM)

9.3.3.1.82.1 Offset

Register	Offset
DBGCAM	308h

9.3.3.1.82.2 Diagram



9.3.3.1.82.3 Fields

Field	Function
31 <code>dbg_stall_rd</code>	Stall for Read channel FOR DEBUG ONLY
30 <code>dbg_stall_wr</code>	Stall for Write channel FOR DEBUG ONLY
29 <code>wr_data_pipeline_empty</code>	This bit indicates that the write data pipeline on the DFI interface is empty. This register is intended to be polled at least twice after setting <code>DBG1.dis_dq</code> , to ensure that all remaining commands/data have completed.

Table continues on the next page...

DDR Controller (DDRC)

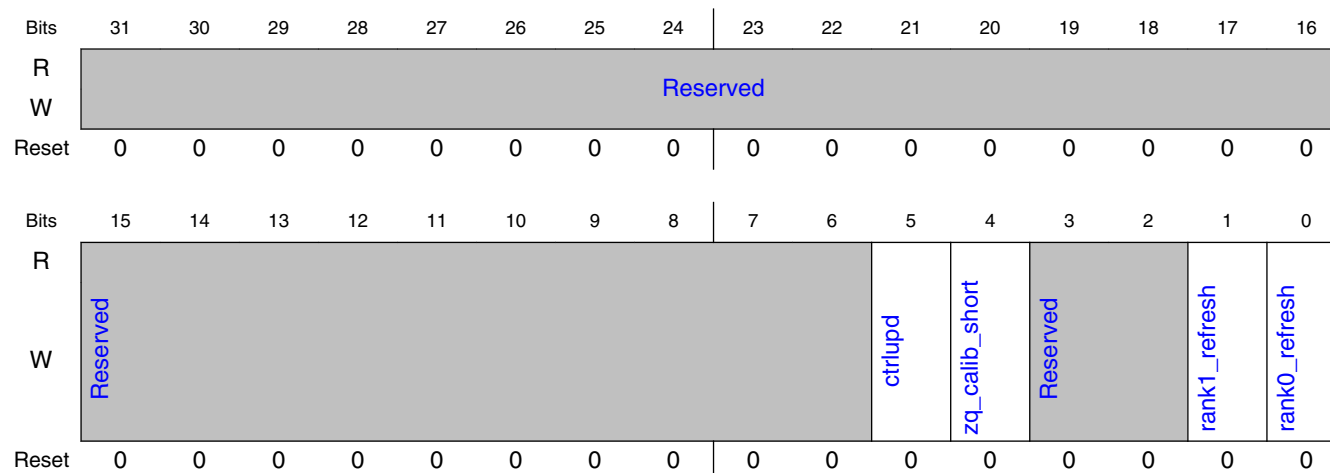
Field	Function
28 rd_data_pipeline_empty	This bit indicates that the read data pipeline on the DFI interface is empty. This register is intended to be polled at least twice after setting DBG1.dis_dq, to ensure that all remaining commands/data have completed.
27 —	Reserved
26 dbg_wr_q_empty	When 1, all the Write command queues and Write data buffers inside DDRC are empty. This register is to be used for debug purpose. An example use-case scenario: When Controller enters Self-Refresh using the Low-Power entry sequence, Controller is expected to have executed all the commands in its queues and the write and read data drained. Hence this register should be 1 at that time. FOR DEBUG ONLY
25 dbg_rd_q_empty	When 1, all the Read command queues and Read data buffers inside DDRC are empty. This register is to be used for debug purpose. An example use-case scenario: When Controller enters Self-Refresh using the Low-Power entry sequence, Controller is expected to have executed all the commands in its queues and the write and read data drained. Hence this register should be 1 at that time. FOR DEBUG ONLY
24 dbg_stall	Stall FOR DEBUG ONLY
23-22 —	Reserved
21-16 dbg_w_q_depth	Write queue depth The last entry of WR queue is reserved for ECC SCRUB operation. This entry is not included in the calculation of the queue depth. FOR DEBUG ONLY
15-14 —	Reserved
13-8 dbg_lpr_q_depth	Low priority read queue depth The last entry of Lpr queue is reserved for ECC SCRUB operation. This entry is not included in the calculation of the queue depth. FOR DEBUG ONLY
7-6 —	Reserved
5-0 dbg_hpr_q_depth	High priority read queue depth FOR DEBUG ONLY

9.3.3.1.83 Command Debug Register (DBGCMD)

9.3.3.1.83.1 Offset

Register	Offset
DBGCMD	30Ch

9.3.3.1.83.2 Diagram



9.3.3.1.83.3 Fields

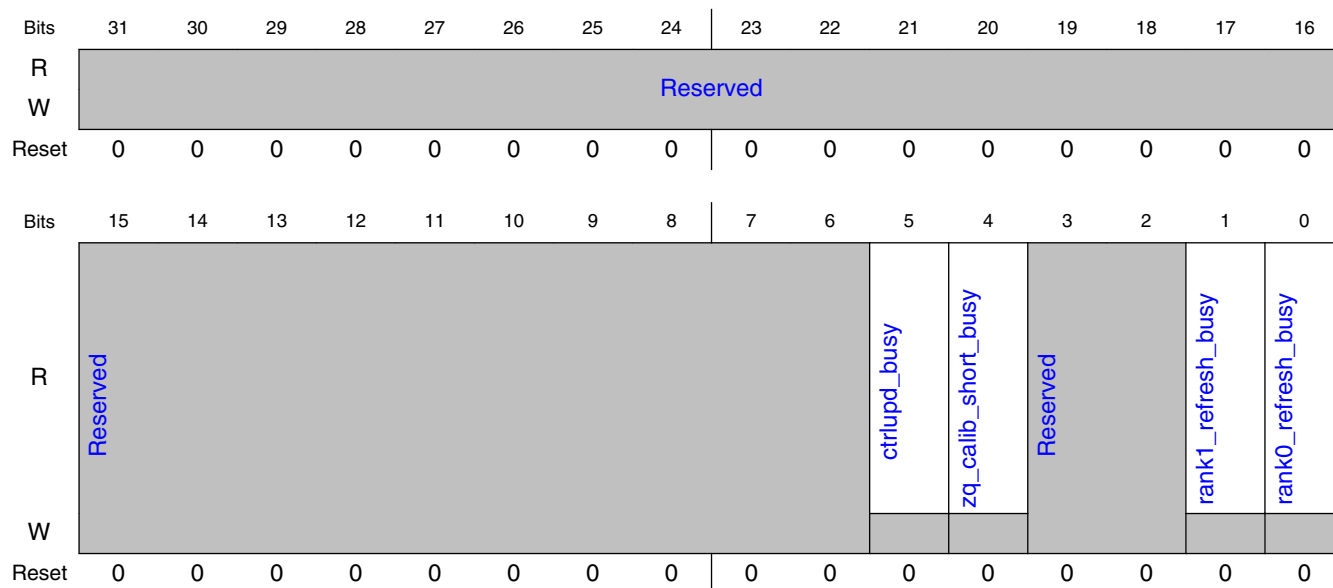
Field	Function
31-6 —	Reserved
5 ctrlupd	Setting this register bit to 1 indicates to the DDRC to issue a dfi_ctrlupd_req to the PHY. When this request is stored in the DDRC, the bit is automatically cleared. This operation must only be performed when DFIUPD0.dis_auto_ctrlupd=1.
4 zq_calib_short	Setting this register bit to 1 indicates to the DDRC to issue a ZQCS (ZQ calibration short)/MPC(ZQ calibration) command to the SDRAM. When this request is stored in the DDRC, the bit is automatically cleared. This operation can be performed only when ZQCTL0.dis_auto_zq=1. It is recommended NOT to set this register bit if in Init operating mode. This register bit is ignored when in Self-Refresh(except LPDDR4) and SR-Powerdown(LPDDR4) and Deep power-down operating modes and Maximum Power Saving Mode.
3-2 —	Reserved
1 rank1_refresh	Setting this register bit to 1 indicates to the DDRC to issue a refresh to rank 1. Writing to this bit causes DBGSTAT.rank1_refresh_busy to be set. When DBGSTAT.rank1_refresh_busy is cleared, the command has been stored in DDRC. For 3DS configuration, refresh is sent to rank index 1. This operation can be performed only when RFSHCTL3.dis_auto_refresh=1. It is recommended NOT to set this register bit if in Init or Deep power-down operating modes or Maximum Power Saving Mode.
0 rank0_refresh	Setting this register bit to 1 indicates to the DDRC to issue a refresh to rank 0. Writing to this bit causes DBGSTAT.rank0_refresh_busy to be set. When DBGSTAT.rank0_refresh_busy is cleared, the command has been stored in DDRC. For 3DS configuration, refresh is sent to rank index 0. This operation can be performed only when RFSHCTL3.dis_auto_refresh=1. It is recommended NOT to set this register bit if in Init or Deep power-down operating modes or Maximum Power Saving Mode.

9.3.3.1.84 Status Debug Register (DBGSTAT)

9.3.3.1.84.1 Offset

Register	Offset
DBGSTAT	310h

9.3.3.1.84.2 Diagram



9.3.3.1.84.3 Fields

Field	Function
31-6 —	Reserved
5 ctrlupd_busy	SoC core may initiate a ctrlupd operation only if this signal is low. This signal goes high in the clock after the DDRC accepts the ctrlupd request. It goes low when the ctrlupd operation is initiated in the DDRC. It is recommended not to perform ctrlupd operations when this signal is high. - 0 - Indicates that the SoC core can initiate a ctrlupd operation - 1 - Indicates that ctrlupd operation has not been initiated yet in the DDRC
4 zq_calib_short_busy	SoC core may initiate a ZQCS (ZQ calibration short) operation only if this signal is low. This signal goes high in the clock after the DDRC accepts the ZQCS request. It goes low when the ZQCS operation is initiated in the DDRC. It is recommended not to perform ZQCS operations when this signal is high. - 0 - Indicates that the SoC core can initiate a ZQCS operation - 1 - Indicates that ZQCS operation has not been initiated yet in the DDRC
3-2 —	Reserved
1 rank1_refresh_busy	SoC core may initiate a rank1_refresh operation (refresh operation to rank 1) only if this signal is low. This signal goes high in the clock after DBGCMD.rank1_refresh is set to one. It goes low when the rank1_refresh operation is stored in the DDRC. It is recommended not to perform rank1_refresh

Table continues on the next page...

Field	Function
	operations when this signal is high. - 0 - Indicates that the SoC core can initiate a rank1_refresh operation - 1 - Indicates that rank1_refresh operation has not been stored yet in the DDRC
0 rank0_refresh_busy	SoC core may initiate a rank0_refresh operation (refresh operation to rank 0) only if this signal is low. This signal goes high in the clock after DBGCMD.rank0_refresh is set to one. It goes low when the rank0_refresh operation is stored in the DDRC. It is recommended not to perform rank0_refresh operations when this signal is high. - 0 - Indicates that the SoC core can initiate a rank0_refresh operation - 1 - Indicates that rank0_refresh operation has not been stored yet in the DDRC

9.3.3.1.85 Software Register Programming Control Enable (SWCTL)

9.3.3.1.85.1 Offset

Register	Offset
SWCTL	320h

9.3.3.1.85.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															sw_done
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

9.3.3.1.85.3 Fields

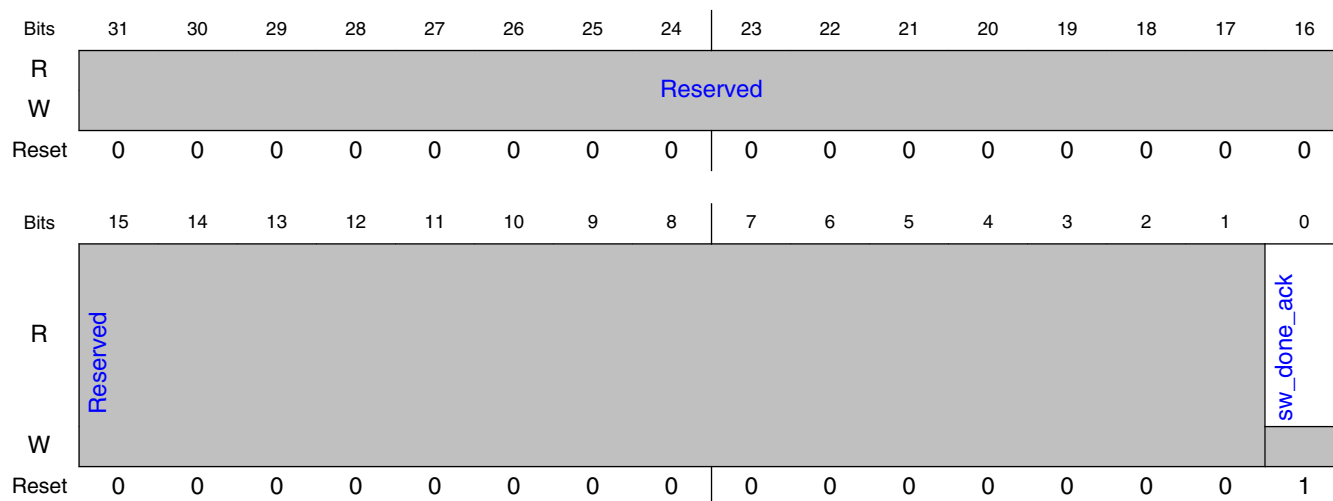
Field	Function
31-1 —	Reserved
0 sw_done	Enable quasi-dynamic register programming outside reset. Program register to 0 to enable quasi-dynamic programming. Set back register to 1 once programming is done.

9.3.3.1.86 Software Register Programming Control Status (SWSTAT)

9.3.3.1.86.1 Offset

Register	Offset
SWSTAT	324h

9.3.3.1.86.2 Diagram



9.3.3.1.86.3 Fields

Field	Function
31-1 —	Reserved
0 sw_done_ack	Register programming done. This register is the echo of SWCTL.sw_done. Wait for sw_done value 1 to propagate to sw_done_ack at the end of the programming sequence to ensure that the correct registers values are propagated to the destination clock domains.

9.3.3.1.87 AXI Poison Configuration Register. (POISONCFG)

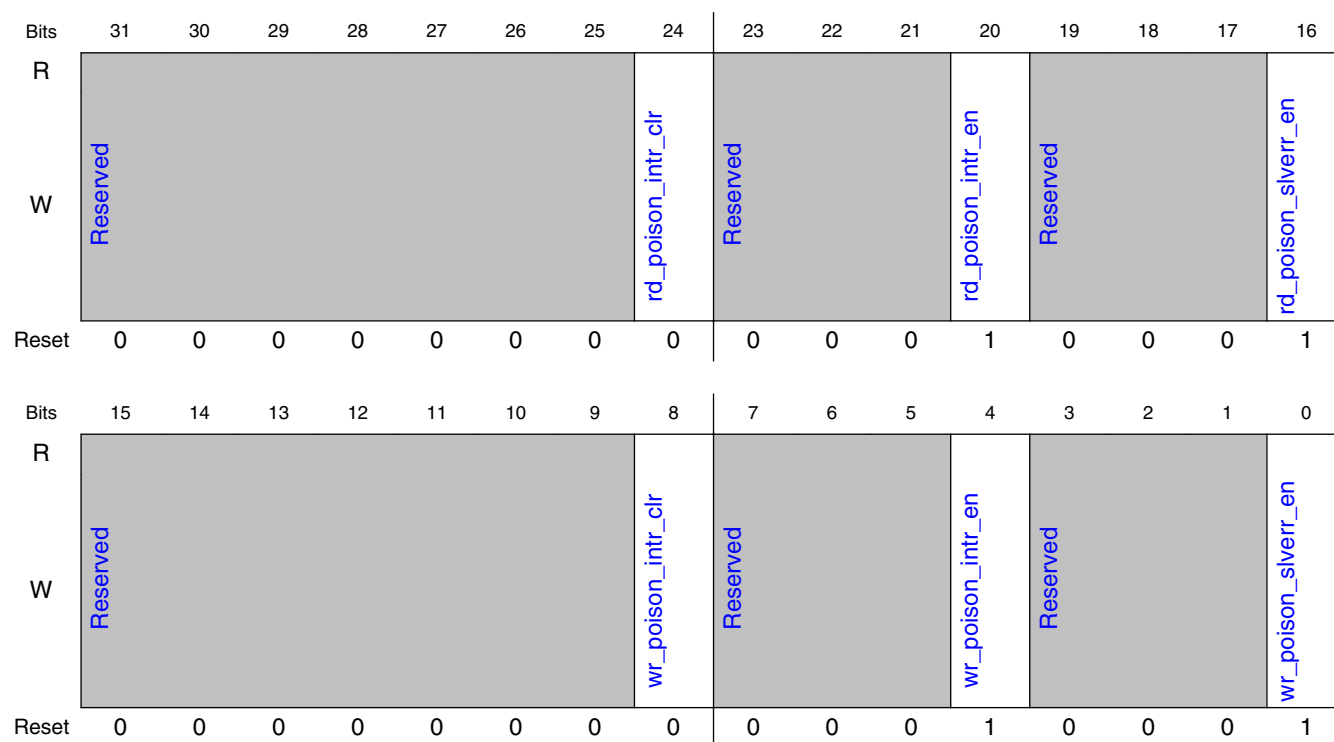
9.3.3.1.87.1 Offset

Register	Offset
POISONCFG	36Ch

9.3.3.1.87.2 Function

AXI Poison Configuration Register. Common for all AXI ports

9.3.3.1.87.3 Diagram



9.3.3.1.87.4 Fields

Field	Function
31-25 —	Reserved
24 <code>rd_poison_intr_clr</code>	Interrupt clear for read transaction poisoning. Allow 2/3 clock cycles for correct value to propagate to core logic and clear the interrupts.
23-21 —	Reserved
20 <code>rd_poison_intr_en</code>	If set to 1, enables interrupts for read transaction poisoning
19-17 —	Reserved
16	If set to 1, enables SLVERR response for read transaction poisoning

Table continues on the next page...

DDR Controller (DDRC)

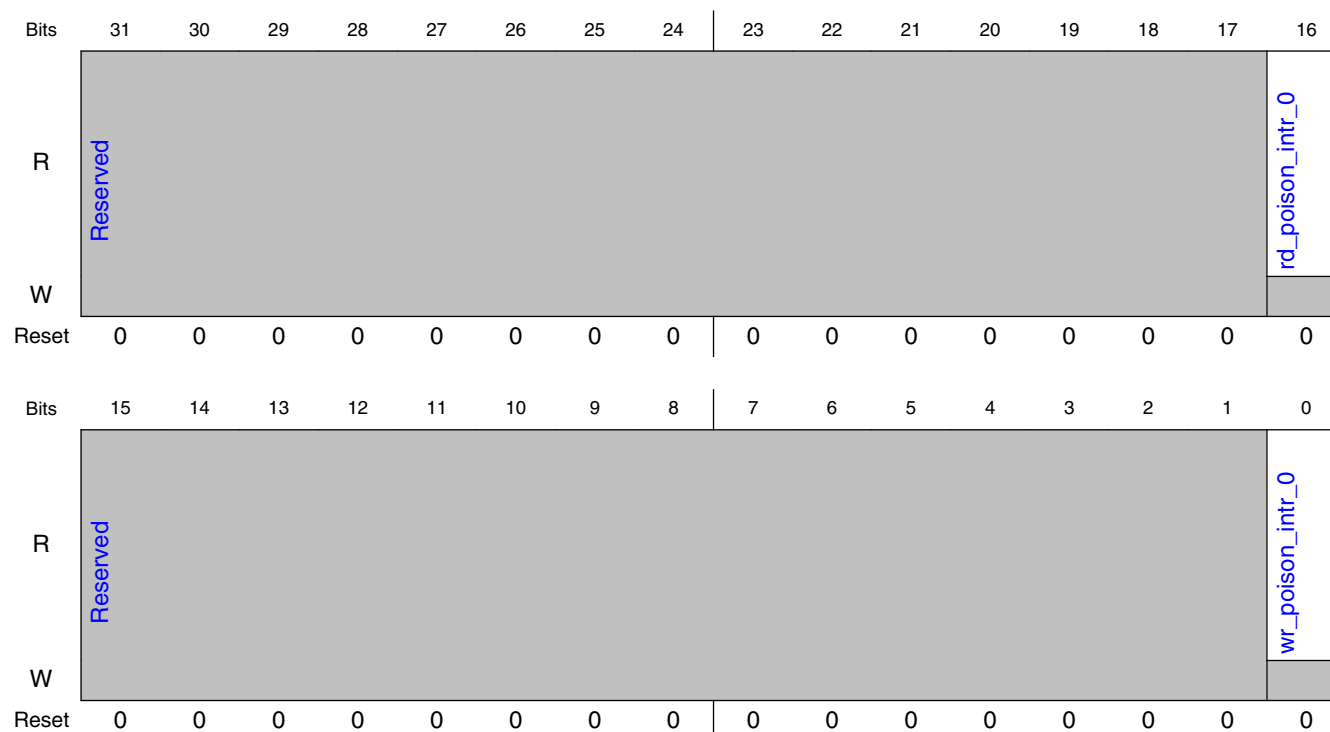
Field	Function
rd_poison_slvrr_en	
15-9 —	Reserved
8 wr_poison_intr_clr	Interrupt clear for write transaction poisoning. Allow 2/3 clock cycles for correct value to propagate to core logic and clear the interrupts.
7-5 —	Reserved
4 wr_poison_intr_en	If set to 1, enables interrupts for write transaction poisoning
3-1 —	Reserved
0 wr_poison_slvr_r_en	If set to 1, enables SLVERR response for write transaction poisoning

9.3.3.1.88 AXI Poison Status Register (POISONSTAT)

9.3.3.1.88.1 Offset

Register	Offset
POISONSTAT	370h

9.3.3.1.88.2 Diagram



9.3.3.1.88.3 Fields

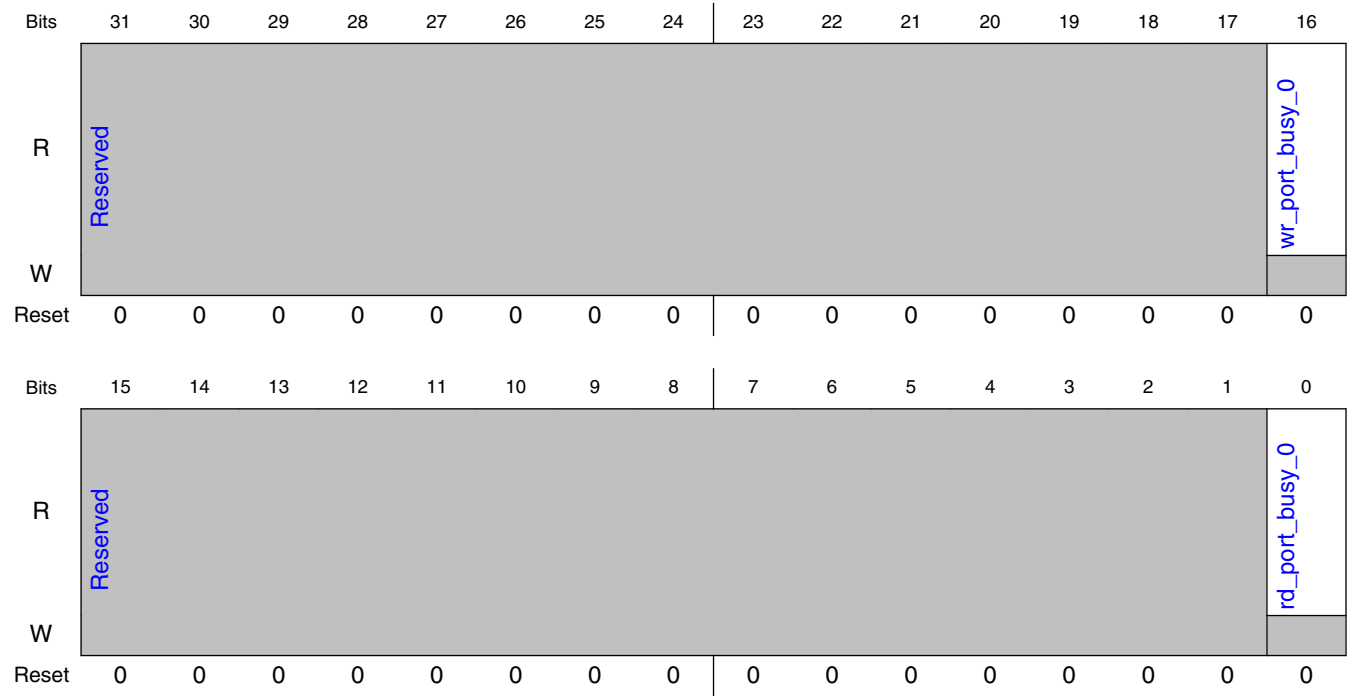
Field	Function
31-17 —	Reserved
16 <code>rd_poison_intr_0</code>	Read transaction poisoning error interrupt for port 0. This register is a APB clock copy (double register synchronizer) of the interrupt asserted when a transaction is poisoned on the corresponding AXI port's read address channel. Bit 0 corresponds to Port 0, and so on. Interrupt is cleared by register <code>rd_poison_intr_clr</code> , then value propagated to APB clock.
15-1 —	Reserved
0 <code>wr_poison_intr_0</code>	Write transaction poisoning error interrupt for port 0. This register is a APB clock copy (double register synchronizer) of the interrupt asserted when a transaction is poisoned on the corresponding AXI port's write address channel. Bit 0 corresponds to Port 0, and so on. Interrupt is cleared by register <code>wr_poison_intr_clr</code> , then value propagated to APB clock.

9.3.3.1.89 Port Status Register (PSTAT)

9.3.3.1.89.1 Offset

Register	Offset
PSTAT	3FCh

9.3.3.1.89.2 Diagram



9.3.3.1.89.3 Fields

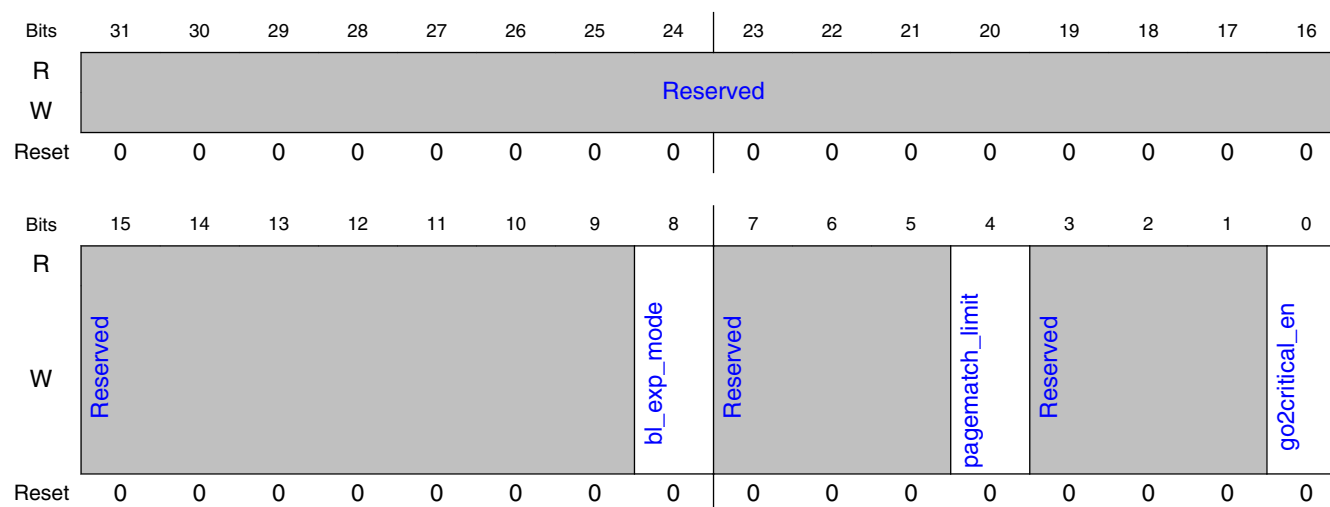
Field	Function
31-17 —	Reserved
16 wr_port_busy_0	Indicates if there are outstanding writes for AXI port 0.
15-1 —	Reserved
0 rd_port_busy_0	Indicates if there are outstanding reads for AXI port 0.

9.3.3.1.90 Port Common Configuration Register (PCCFG)

9.3.3.1.90.1 Offset

Register	Offset
PCCFG	400h

9.3.3.1.90.2 Diagram



9.3.3.1.90.3 Fields

Field	Function
31-9 —	Reserved
8 bl_exp_mode	Burst length expansion mode. By default (i.e. bl_exp_mode==0) XPI expands every AXI burst into multiple HIF commands, using the memory burst length as a unit. If set to 1, then XPI will use half of the memory burst length as a unit. This applies to both reads and writes. When MSTR.data_bus_width==00, setting bl_exp_mode to 1 has no effect. This can be used in cases where Partial Writes is enabled (DDRC_PARTIAL_WR=1), in order to avoid or minimize t_ccd_l penalty in DDR4 and t_ccd_mw penalty in LPDDR4. Hence, bl_exp_mode=1 is only recommended if DDR4 or LPDDR4. Note that if DBICTL.reg_ddrc_dm_en=0, functionality is not supported in the following cases: - DDRC_PARTIAL_WR=0 - DDRC_PARTIAL_WR=1, MSTR.data_bus_width=01, MEMC_BURST_LENGTH=8 and MSTR.burst_rdw=1000 (LPDDR4 only) - DDRC_PARTIAL_WR=1, MSTR.data_bus_width=01, MEMC_BURST_LENGTH=4 and MSTR.burst_rdw=0100 (DDR4 only), with either MSTR.reg_ddrc_burstchop=0 or CRCPARCTL1.reg_ddrc_crc_enable=1 Functionality is also not supported if Data Channel Interleave is enabled
7-5 —	Reserved

Table continues on the next page...

DDR Controller (DDRC)

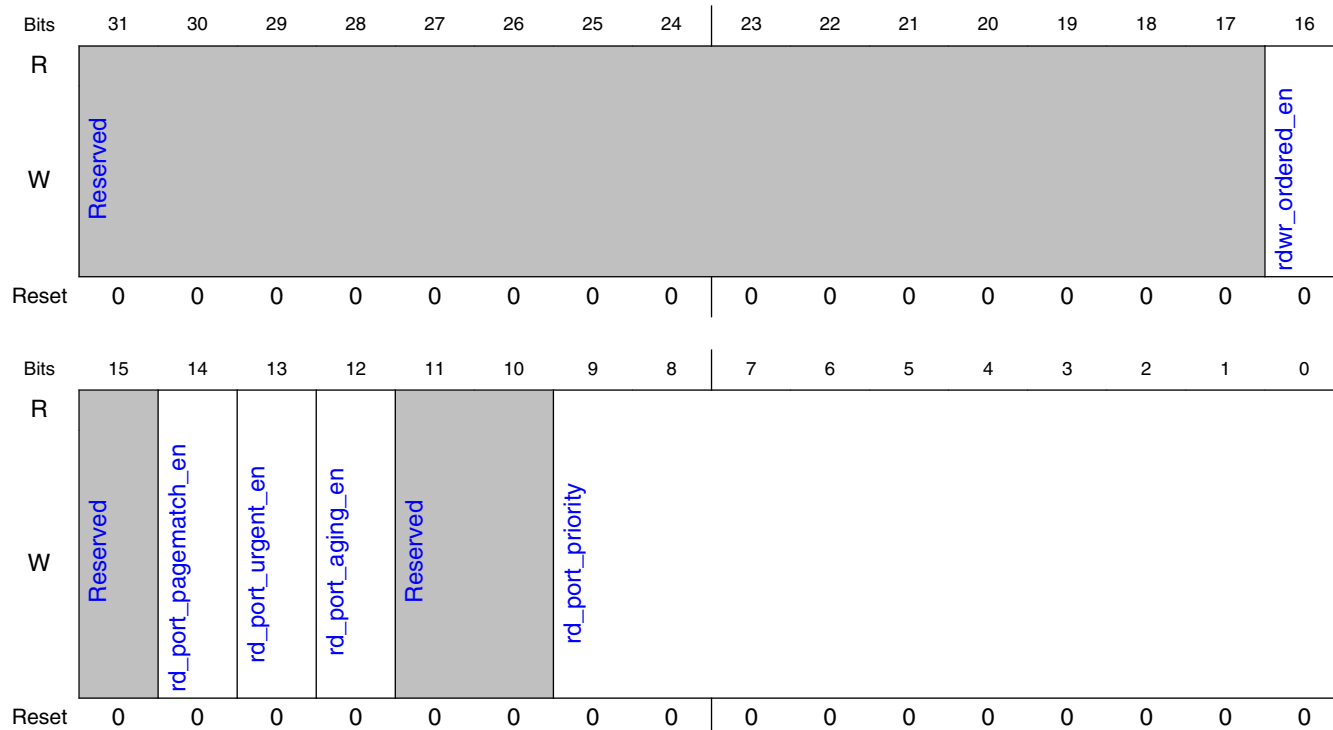
Field	Function
4 pagematch_limit	Page match four limit. If set to 1, limits the number of consecutive same page DDRC transactions that can be granted by the Port Arbiter to four when Page Match feature is enabled. If set to 0, there is no limit imposed on number of consecutive same page DDRC transactions.
3-1 —	Reserved
0 go2critical_en	If set to 1 (enabled), sets co_gs_go2critical_wr and co_gs_go2critical_lpr/co_gs_go2critical_hpr signals going to DDRC based on urgent input (awurgent, arurgent) coming from AXI master. If set to 0 (disabled), co_gs_go2critical_wr and co_gs_go2critical_lpr/co_gs_go2critical_hpr signals at DDRC are driven to 1b'0.

9.3.3.1.91 Port n Configuration Read Register (PCFGR_0)

9.3.3.1.91.1 Offset

Register	Offset
PCFGR_0	404h

9.3.3.1.91.2 Diagram



9.3.3.1.91.3 Fields

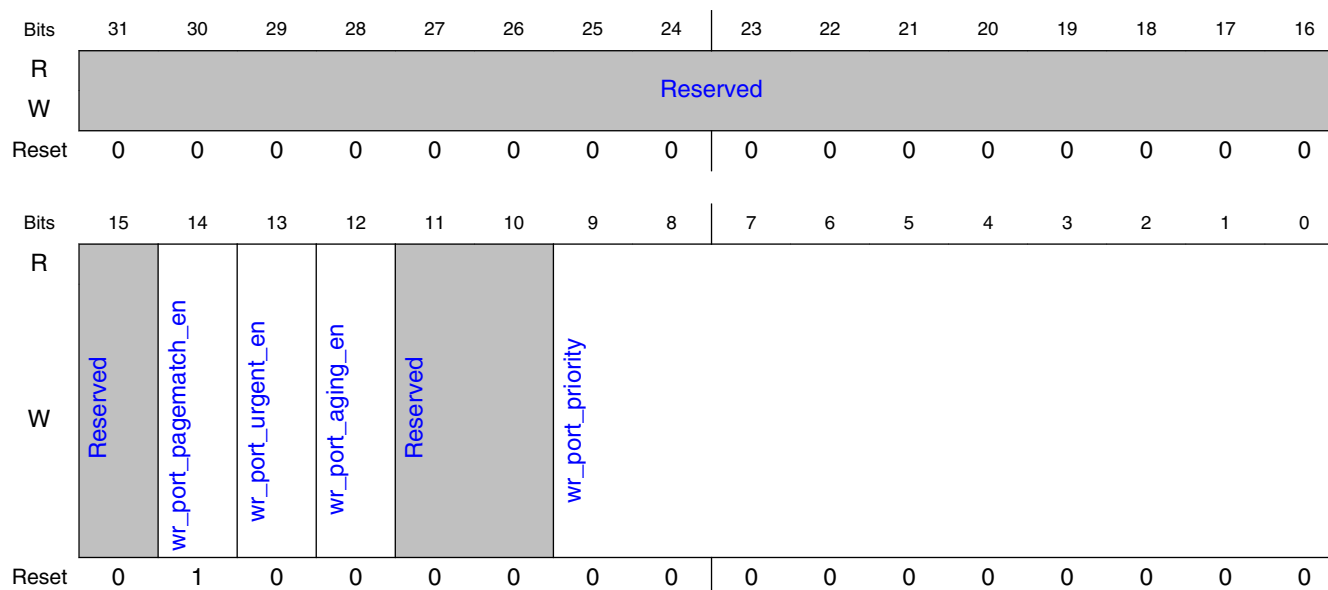
Field	Function
31-17 —	Reserved
16 rdwr_ordered_en	Enable ordered read/writes. If set to 1, preserves the ordering between read transaction and write transaction issued to the same address, on a given port. In other words, the controller ensures that all same address read and write commands from the application port interface are transported to the DFI interface in the order of acceptance. This feature is useful in cases where software coherency is desired for masters issuing back-to-back read/write transactions without waiting for write/read responses. Note that this register has an effect only if necessary logic is instantiated via the DDRC_RDWR_ORDERED_n parameter.
15 —	Reserved
14 rd_port_pagematch_en	If set to 1, enables the Page Match feature. If enabled, once a requesting port is granted, the port is continued to be granted if the following immediate commands are to the same memory page (same bank and same row). See also related PCCFG.pagematch_limit register.
13 rd_port_urgent_en	If set to 1, enables the AXI urgent sideband signal (arurgent). When enabled and arurgent is asserted by the master, that port becomes the highest priority and co_gs_go2critical_lpr/co_gs_go2critical_hpr signal to DDRC is asserted if enabled in PCCFG.go2critical_en register. Note that arurgent signal can be asserted anytime and as long as required which is independent of address handshaking (it is not associated with any particular command).
12 rd_port_aging_en	If set to 1, enables aging function for the read channel of the port.
11-10 —	Reserved
9-0 rd_port_priority	Determines the initial load value of read aging counters. These counters will be parallel loaded after reset, or after each grant to the corresponding port. The aging counters down-count every clock cycle where the port is requesting but not granted. The higher significant 5-bits of the read aging counter sets the priority of the read channel of a given port. Port's priority will increase as the higher significant 5-bits of the counter starts to decrease. When the aging counter becomes 0, the corresponding port channel will have the highest priority level (timeout condition - Priority0). For multi-port configurations, the aging counters cannot be used to set port priorities when external dynamic priority inputs (arqos) are enabled (timeout is still applicable). For single port configurations, the aging counters are only used when they timeout (become 0) to force read-write direction switching. In this case, external dynamic priority input, arqos (for reads only) can still be used to set the DDRC read priority (2 priority levels: low priority read - LPR, high priority read - HPR) on a command by command basis. Note: The two LSBs of this register field are tied internally to 2'b00.

9.3.3.1.92 Port n Configuration Write Register (PCFGW_0)

9.3.3.1.92.1 Offset

Register	Offset
PCFGW_0	408h

9.3.3.1.92.2 Diagram



9.3.3.1.92.3 Fields

Field	Function
31-15 —	Reserved
14 wr_port_pagematch_en	If set to 1, enables the Page Match feature. If enabled, once a requesting port is granted, the port is continued to be granted if the following immediate commands are to the same memory page (same bank and same row). See also related PCCFG.pagematch_limit register.
13 wr_port_urgent_en	If set to 1, enables the AXI urgent sideband signal (awurgent). When enabled and awurgent is asserted by the master, that port becomes the highest priority and co_gs_go2critical_wr signal to DDRC is asserted if enabled in PCCFG.go2critical_en register. Note that awurgent signal can be asserted anytime and as long as required which is independent of address handshaking (it is not associated with any particular command).
12 wr_port_aging_en	If set to 1, enables aging function for the write channel of the port.
11-10 —	Reserved
9-0 wr_port_priority	Determines the initial load value of write aging counters. These counters will be parallel loaded after reset, or after each grant to the corresponding port. The aging counters down-count every clock cycle where the port is requesting but not granted. The higher significant 5-bits of the write aging counter sets the initial priority of the write channel of a given port. Port's priority will increase as the higher significant 5-bits of the counter starts to decrease. When the aging counter becomes 0, the corresponding port channel will have the highest priority level. For multi-port configurations, the aging counters cannot be used to set port priorities when external dynamic priority inputs (awqos) are enabled (timeout is still applicable). For single port configurations, the aging counters are only used when they timeout (become

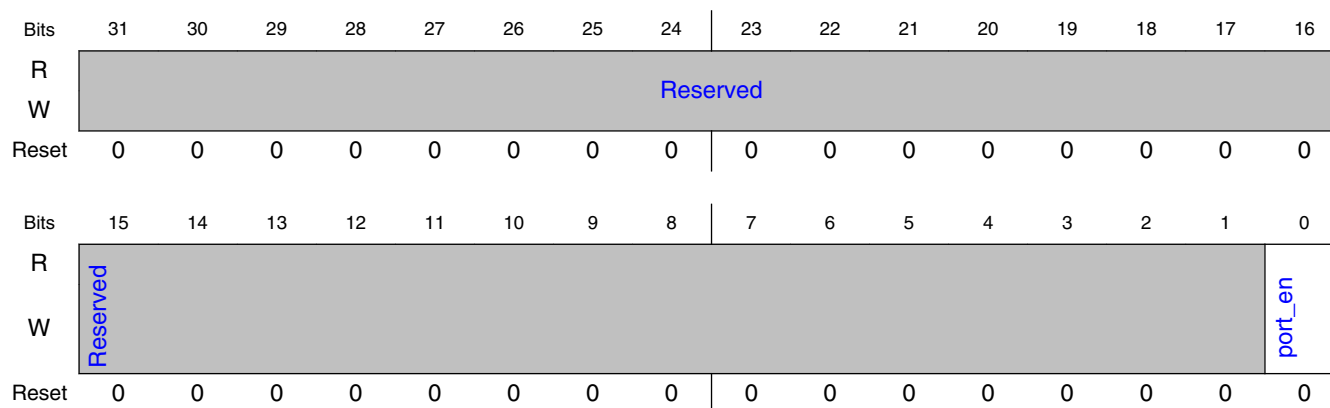
Field	Function
	0) to force read-write direction switching. Note: The two LSBs of this register field are tied internally to 2'b00.

9.3.3.1.93 Port n Control Register (PCTRL_0)

9.3.3.1.93.1 Offset

Register	Offset
PCTRL_0	490h

9.3.3.1.93.2 Diagram



9.3.3.1.93.3 Fields

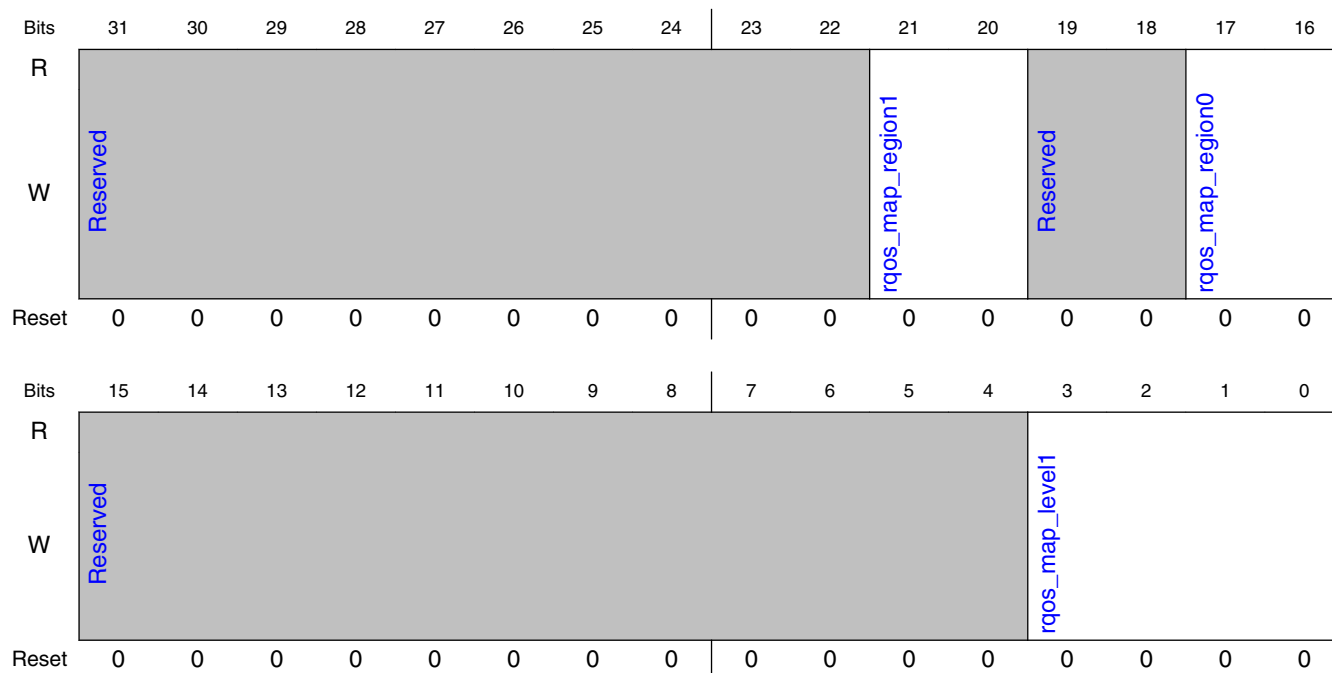
Field	Function
31-1 —	Reserved
0 port_en	Enables AXI port n.

9.3.3.1.94 Port n Read QoS Configuration Register 0 (PCFGQOS0_0)

9.3.3.1.94.1 Offset

Register	Offset
PCFGQOS0_0	494h

9.3.3.1.94.2 Diagram



9.3.3.1.94.3 Fields

Field	Function
31-22 —	Reserved
21-20 <code>rqos_map_region1</code>	This bitfield indicates the traffic class of region 1. Valid values are: 0 : LPR, 1: VPR, 2: HPR. For dual address queue configurations, region1 maps to the blue address queue. In this case, valid values are 0: LPR and 1: VPR only. When VPR support is disabled (DDRC_VPR_EN = 0) and traffic class of region 1 is set to 1 (VPR), VPR traffic is aliased to LPR traffic.
19-18 —	Reserved
17-16 <code>rqos_map_region0</code>	This bitfield indicates the traffic class of region 0. Valid values are: 0: LPR, 1: VPR, 2: HPR. For dual address queue configurations, region 0 maps to the blue address queue. In this case, valid values are 0: LPR and 1: VPR only. When VPR support is disabled (DDRC_VPR_EN = 0) and traffic class of region0 is set to 1 (VPR), VPR traffic is aliased to LPR traffic.
15-4 —	Reserved

Table continues on the next page...

Field	Function
3-0 rqos_map_level1	Separation level1 indicating the end of region0 mapping; start of region0 is 0. Possible values for level1 are 0 to 13 (for dual RAQ) or 0 to 14 (for single RAQ) which corresponds to arqos. Note that for PA, arqos values are used directly as port priorities, where the higher the value corresponds to higher port priority. All of the map_level* registers must be set to distinct values.

9.3.3.1.95 Port n Read QoS Configuration Register 1 (PCFGQOS1_0)

9.3.3.1.95.1 Offset

Register	Offset
PCFGQOS1_0	498h

9.3.3.1.95.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved					rqos_map_timeoutr										
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved					rqos_map_timeoutb										
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.3.3.1.95.3 Fields

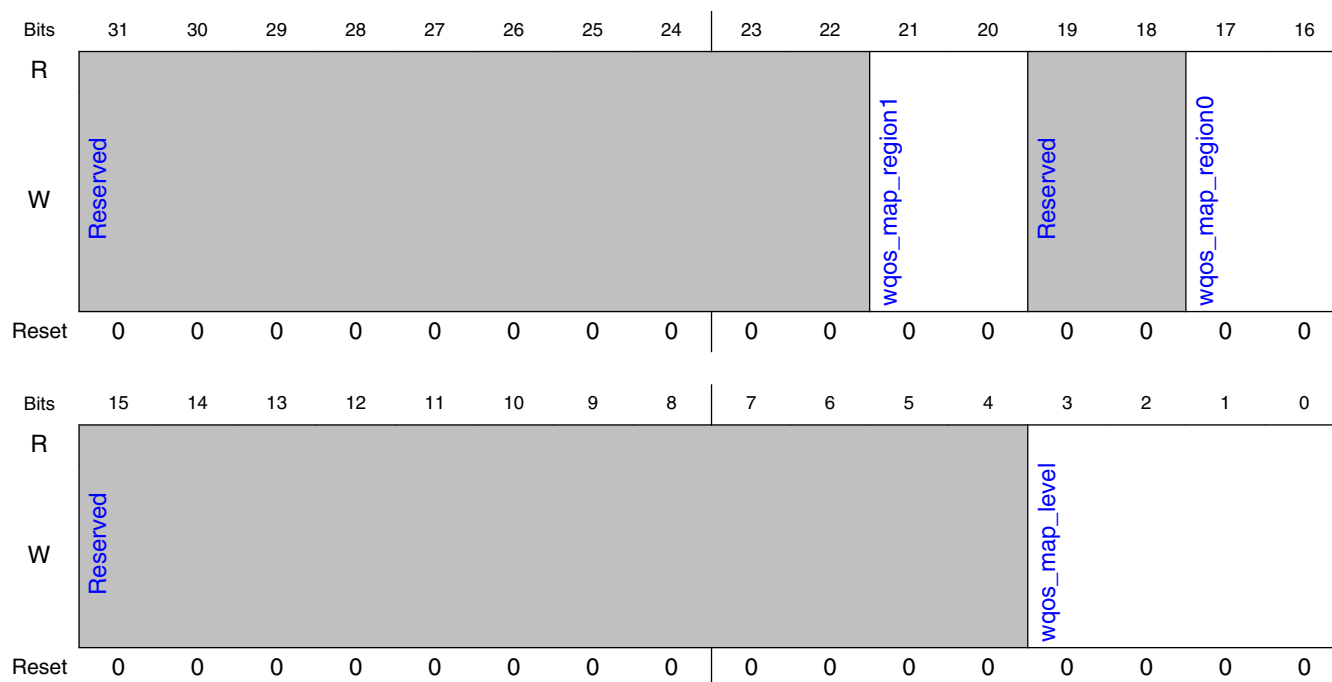
Field	Function
31-27 —	Reserved
26-16 rqos_map_timeoutr	Specifies the timeout value for transactions mapped to the red address queue.
15-11 —	Reserved
10-0 rqos_map_timeoutb	Specifies the timeout value for transactions mapped to the blue address queue.

9.3.3.1.96 Port n Write QoS Configuration Register 0 (PCFGWQOS0_0)

9.3.3.1.96.1 Offset

Register	Offset
PCFGWQOS0_0	49Ch

9.3.3.1.96.2 Diagram



9.3.3.1.96.3 Fields

Field	Function
31-22 —	Reserved
21-20 wqos_map_region1	This bitfield indicates the traffic class of region 1. Valid values are: 0: NPW, 1: VPW. When VPW support is disabled (DDRC_VPW_EN = 0) and traffic class of region 1 is set to 1 (VPW), VPW traffic is aliased to LPW traffic.
19-18 —	Reserved

Table continues on the next page...

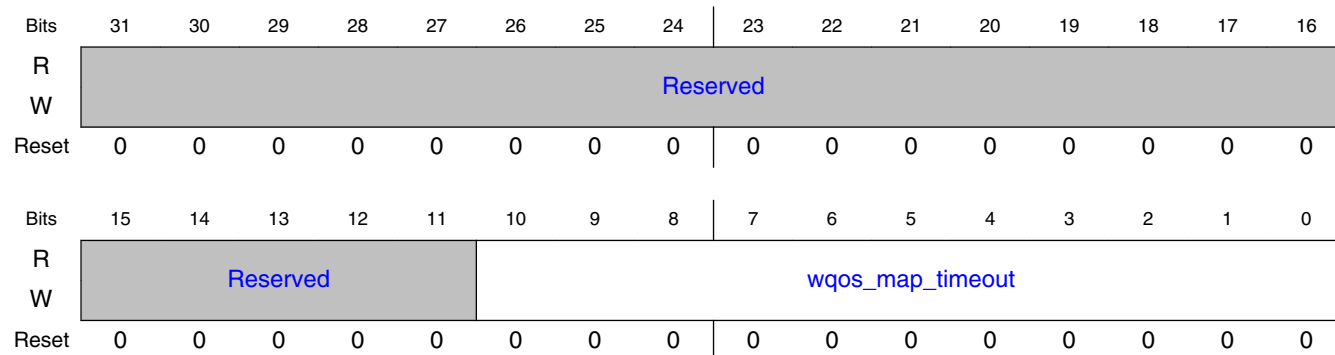
Field	Function
17-16 wqos_map_region0	This bitfield indicates the traffic class of region 0. Valid values are: 0: NPW, 1: VPW. When VPW support is disabled (DDRC_VPW_EN = 0) and traffic class of region0 is set to 1 (VPW), VPW traffic is aliased to NPW traffic.
15-4 —	Reserved
3-0 wqos_map_level	Separation level indicating the end of region0 mapping; start of region0 is 0. Possible values for level1 are 0 to 14 which corresponds to awqos. Note that for PA, awqos values are used directly as port priorities, where the higher the value corresponds to higher port priority.

9.3.3.1.97 Port n Write QoS Configuration Register 1 (PCFGWQOS1_0)

9.3.3.1.97.1 Offset

Register	Offset
PCFGWQOS1_0	4A0h

9.3.3.1.97.2 Diagram



9.3.3.1.97.3 Fields

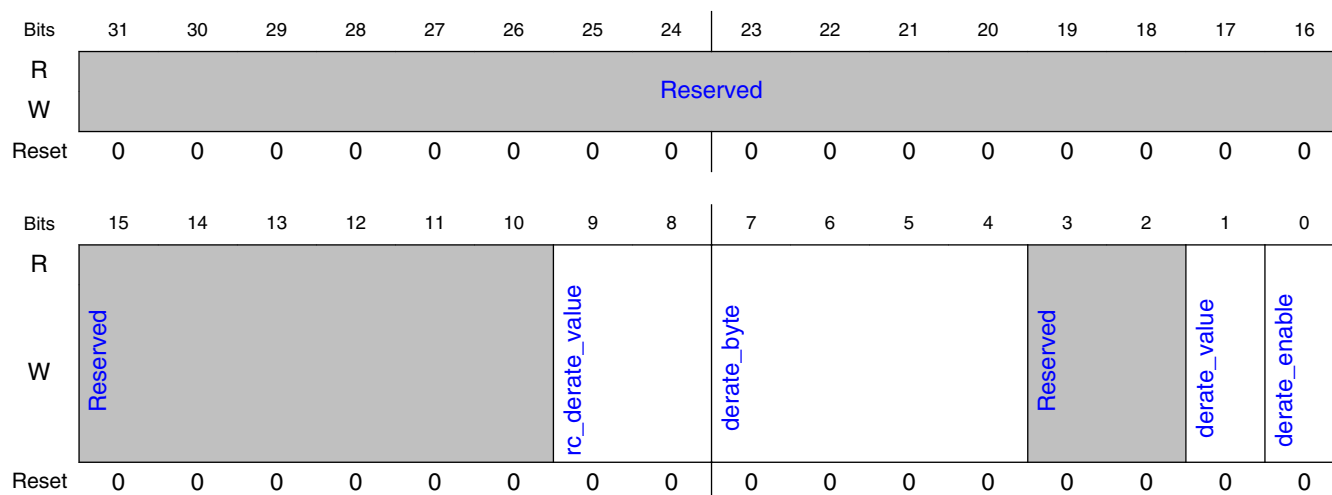
Field	Function
31-11 —	Reserved
10-0 wqos_map_timeout	Specifies the timeout value for write transactions.

9.3.3.1.98 [SHADOW] Temperature Derate Enable Register (DERATEEN_SHADOW)

9.3.3.1.98.1 Offset

Register	Offset
DERATEEN_SHADOW	2020h

9.3.3.1.98.2 Diagram



9.3.3.1.98.3 Fields

Field	Function
31-10 —	Reserved
9-8 rc_derate_value	Derate value of tRC for LPDDR4 - 0 - Derating uses +1. - 1 - Derating uses +2. - 2 - Derating uses +3. - 3 - Derating uses +4. Present only in designs configured to support LPDDR4. The required number of cycles for derating can be determined by dividing 3.75ns by the core_ddrc_core_clk period, and rounding up the next integer.
7-4 derate_byte	Derate byte Present only in designs configured to support LPDDR2/LPDDR3/LPDDR4 Indicates which byte of the MRR data is used for derating. The maximum valid value depends on MEMC_DRAM_TOTAL_DATA_WIDTH.
3-2 —	Reserved
1 derate_value	Derate value - 0 - Derating uses +1. - 1 - Derating uses +2. Present only in designs configured to support LPDDR2/LPDDR3/LPDDR4 Set to 0 for all LPDDR2 speed grades as derating value of +1.875 ns is less than a core_ddrc_core_clk period. For LPDDR3/4, if the period of core_ddrc_core_clk is less than 1.875ns, this register field should be set to 1; otherwise it should be set to 0.

Table continues on the next page...

Field	Function
0 derate_enable	Enables derating - 0 - Timing parameter derating is disabled - 1 - Timing parameter derating is enabled using MR4 read value. Present only in designs configured to support LPDDR2/LPDDR3/LPDDR4. This field must be set to '0' for non-LPDDR2/LPDDR3/LPDDR4 mode.

9.3.3.1.99 [SHADOW] Temperature Derate Interval Register (DERATEINT_SHADOW)

9.3.3.1.99.1 Offset

Register	Offset
DERATEINT_SHADOW	2024h

9.3.3.1.99.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	mr4_read_interval															
W																
Reset	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	mr4_read_interval															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.3.3.1.99.3 Fields

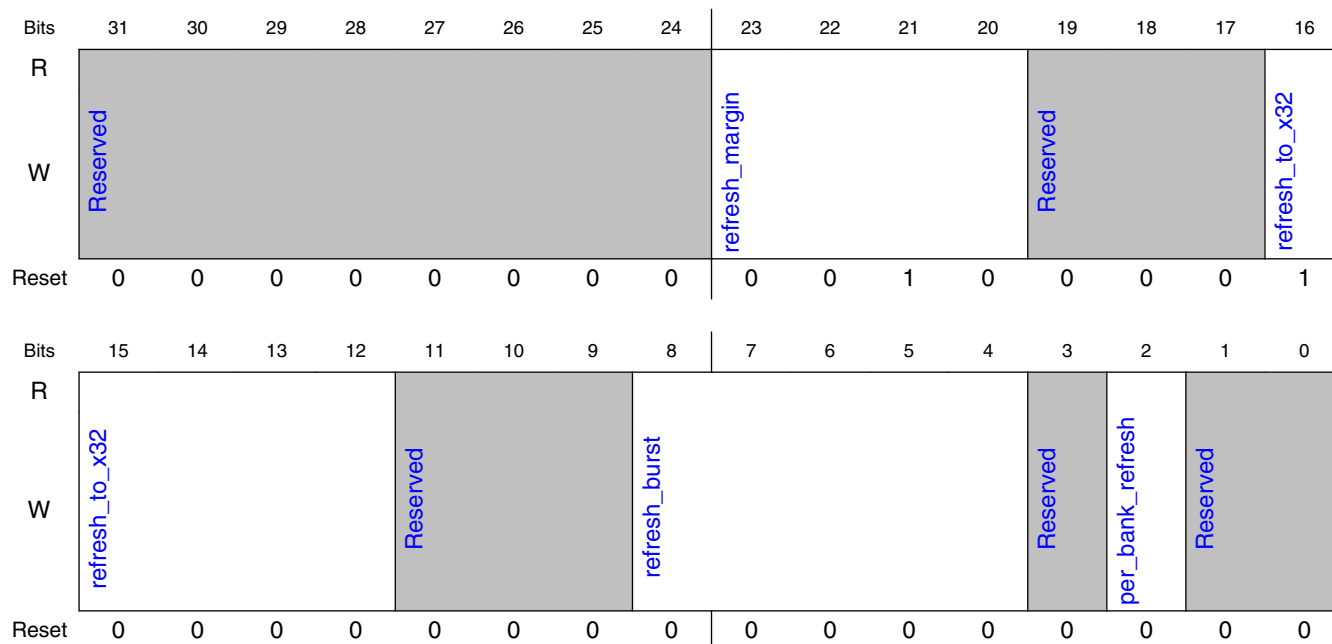
Field	Function
31-0 mr4_read_interval	Interval between two MR4 reads, used to derate the timing parameters. Present only in designs configured to support LPDDR2/LPDDR3/LPDDR4. This register must not be set to zero. Unit: DFI clock cycle.

9.3.3.1.100 [SHADOW] Refresh Control Register 0 (RFSHCTL0_SHADOW)

9.3.3.1.100.1 Offset

Register	Offset
RFSHCTL0_SHADOW	2050h

9.3.3.1.100.2 Diagram



9.3.3.1.100.3 Fields

Field	Function
31-24 —	Reserved
23-20 refresh_margin	Threshold value in number of DFI clock cycles before the critical refresh or page timer expires. A critical refresh is to be issued before this threshold is reached. It is recommended that this not be changed from the default value, currently shown as 0x2. It must always be less than internally used <code>t_rfc_nom_x32</code> . Note that, in LPDDR2/LPDDR3/LPDDR4, internally used <code>t_rfc_nom_x32</code> may be equal to <code>RFSHTMG.t_rfc_nom_x32>>2</code> if derating is enabled (<code>DERATEEN.derate_enable=1</code>). Otherwise, internally used <code>t_rfc_nom_x32</code> will be equal to <code>RFSHTMG.t_rfc_nom_x32</code> . Unit: Multiples of 32 DFI clocks.
19-17 —	Reserved
16-12 refresh_to_x32	If the refresh timer (<code>tRFCnom</code> , also known as <code>tREFI</code>) has expired at least once, but it has not expired (<code>RFSHCTL0.refresh_burst+1</code>) times yet, then a speculative refresh may be performed. A speculative refresh is a refresh performed at a time when refresh would be useful, but before it is absolutely required. When the SDRAM bus is idle for a period of time determined by this <code>RFSHCTL0.refresh_to_x32</code> and the refresh timer has expired at least once since the last refresh, then a speculative refresh is performed.

Table continues on the next page...

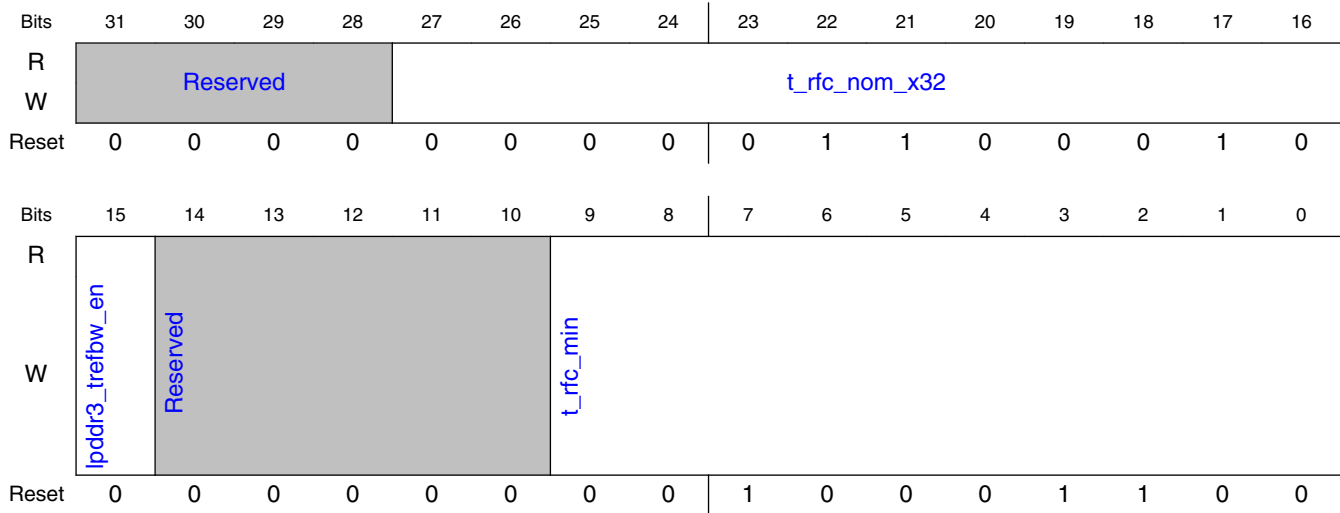
Field	Function
	Speculative refreshes continues successively until there are no refreshes pending or until new reads or writes are issued to the DDRC. FOR PERFORMANCE ONLY. Unit: Multiples of 32 DFI clocks.
11-9 —	Reserved
8-4 refresh_burst	The programmed value + 1 is the number of refresh timeouts that is allowed to accumulate before traffic is blocked and the refreshes are forced to execute. Closing pages to perform a refresh is a one-time penalty that must be paid for each group of refreshes. Therefore, performing refreshes in a burst reduces the per-refresh penalty of these page closings. Higher numbers for RFSHCTL.refresh_burst slightly increases utilization; lower numbers decreases the worst-case latency associated with refreshes. - 0 - single refresh - 1 - burst-of-2 refresh - 7 - burst-of-8 refresh For information on burst refresh feature refer to section 3.9 of DDR2 JEDEC specification - JESD79-2F.pdf. For DDR2/3, the refresh is always per-rank and not per-bank. The rank refresh can be accumulated over 8*tREFI cycles using the burst refresh feature. In DDR4 mode, according to Fine Granularity feature, 8 refreshes can be postponed in 1X mode, 16 refreshes in 2X mode and 32 refreshes in 4X mode. If using PHY-initiated updates, care must be taken in the setting of RFSHCTL0.refresh_burst, to ensure that tRFCmax is not violated due to a PHY-initiated update occurring shortly before a refresh burst was due. In this situation, the refresh burst will be delayed until the PHY-initiated update is complete.
3 —	Reserved
2 per_bank_refresh	- 1 - Per bank refresh; - 0 - All bank refresh. Per bank refresh allows traffic to flow to other banks. Per bank refresh is not supported by all LPDDR2 devices but should be supported by all LPDDR3/LPDDR4 devices. Present only in designs configured to support LPDDR2/LPDDR3/LPDDR4
1-0 —	Reserved

9.3.3.1.101 [SHADOW] Refresh Timing Register (RFSHTMG_SHADOW)

9.3.3.1.101.1 Offset

Register	Offset
RFSHTMG_SHADOW	2064h

9.3.3.1.101.2 Diagram



9.3.3.1.101.3 Fields

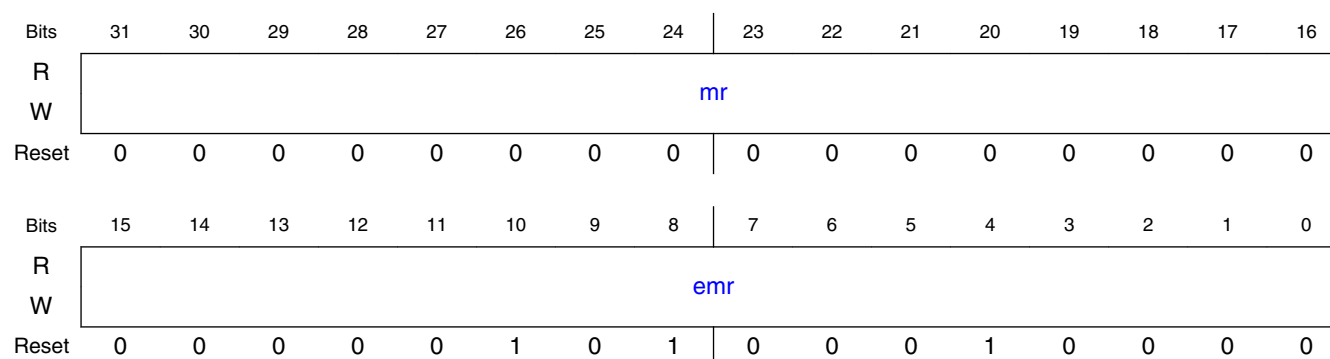
Field	Function
31-28 —	Reserved
27-16 t_rfc_nom_x32	tREFI: Average time interval between refreshes per rank (Specification: 7.8us for DDR2, DDR3 and DDR4. See JEDEC specification for mDDR, LPDDR2, LPDDR3 and LPDDR4). For LPDDR2/LPDDR3/LPDDR4: - if using all-bank refreshes (RFSHCTL0.per_bank_refresh = 0), this register should be set to tREFIab - if using per-bank refreshes (RFSHCTL0.per_bank_refresh = 1), this register should be set to tREFIpb When the controller is operating in 1:2 frequency ratio mode, program this to (tREFI/2), no rounding up. In DDR4 mode, tREFI value is different depending on the refresh mode. The user should program the appropriate value from the spec based on the value programmed in the refresh mode register. Note that RFSHTMG.t_rfc_nom_x32 * 32 must be greater than RFSHTMG.t_rfc_min, and RFSHTMG.t_rfc_nom_x32 must be greater than 0x1. - Non-DDR4 or DDR4 Fixed 1x mode: RFSHTMG.t_rfc_nom_x32 must be less than or equal to 0xFFE. - DDR4 Fixed 2x mode: RFSHTMG.t_rfc_nom_x32 must be less than or equal to 0x7FF. - DDR4 Fixed 4x mode: RFSHTMG.t_rfc_nom_x32 must be less than or equal to 0x3FF. Unit: Multiples of 32 clocks.
15 lpddr3_trefbw_en	Used only when LPDDR3 memory type is connected. Should only be changed when DDRC is in reset. Specifies whether to use the tREFBW parameter (required by some LPDDR3 devices which comply with earlier versions of the LPDDR3 JEDEC specification) or not: - 0 - tREFBW parameter not used - 1 - tREFBW parameter used
14-10 —	Reserved
9-0 t_rfc_min	tRFC (min): Minimum time from refresh to refresh or activate. When the controller is operating in 1:1 mode, t_rfc_min should be set to RoundUp(tRFCmin/tCK). When the controller is operating in 1:2 mode, t_rfc_min should be set to RoundUp(RoundUp(tRFCmin/tCK)/2). In LPDDR2/LPDDR3/LPDDR4 mode: - if using all-bank refreshes, the tRFCmin value in the above equations is equal to tRFCab - if using per-bank refreshes, the tRFCmin value in the above equations is equal to tRFCpb In DDR4 mode, the tRFCmin value in the above equations is different depending on the refresh mode (fixed 1X,2X,4X) and the device density. The user should program the appropriate value from the spec based on the 'refresh_mode' and the device density that is used. Unit: Clocks.

9.3.3.1.102 [SHADOW] SDRAM Initialization Register 3 (INIT3_SHADOW)

9.3.3.1.102.1 Offset

Register	Offset
INIT3_SHADOW	20DCh

9.3.3.1.102.2 Diagram



9.3.3.1.102.3 Fields

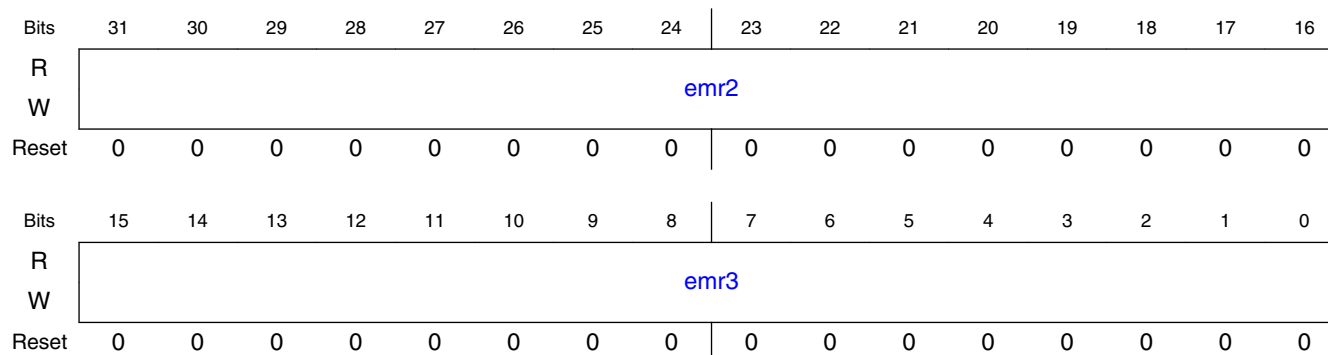
Field	Function
31-16 mr	DDR2: Value to write to MR register. Bit 8 is for DLL and the setting here is ignored. The DDRC sets this bit appropriately. DDR3/DDR4: Value loaded into MR0 register. mDDR: Value to write to MR register. LPDDR2/LPDDR3/LPDDR4 - Value to write to MR1 register
15-0 emr	DDR2: Value to write to EMR register. Bits 9:7 are for OCD and the setting in this register is ignored. The DDRC sets those bits appropriately. DDR3/DDR4: Value to write to MR1 register Set bit 7 to 0. If PHY-evaluation mode training is enabled, this bit is set appropriately by the DDRC during write leveling. mDDR: Value to write to EMR register. LPDDR2/LPDDR3/LPDDR4 - Value to write to MR2 register

9.3.3.1.103 [SHADOW] SDRAM Initialization Register 4 (INIT4_SHADOW)

9.3.3.1.103.1 Offset

Register	Offset
INIT4_SHADOW	20E0h

9.3.3.1.103.2 Diagram



9.3.3.1.103.3 Fields

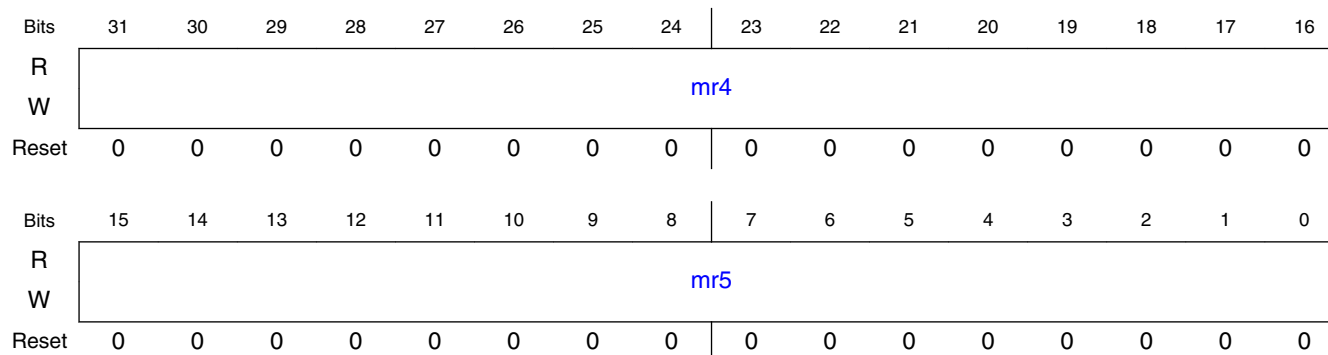
Field	Function
31-16 emr2	DDR2: Value to write to EMR2 register. DDR3/DDR4: Value to write to MR2 register LPDDR2/LPDDR3/ LPDDR4: Value to write to MR3 register mDDR: Unused
15-0 emr3	DDR2: Value to write to EMR3 register. DDR3/DDR4: Value to write to MR3 register mDDR/LPDDR2/ LPDDR3: Unused LPDDR4: Value to write to MR13 register

9.3.3.1.104 [SHADOW] SDRAM Initialization Register 6 (INIT6_SHADOW)

9.3.3.1.104.1 Offset

Register	Offset
INIT6_SHADOW	20E8h

9.3.3.1.104.2 Diagram



9.3.3.1.104.3 Fields

Field	Function
31-16 mr4	DDR4- Value to be loaded into SDRAM MR4 registers. Used in DDR4 designs only.
15-0 mr5	DDR4- Value to be loaded into SDRAM MR5 registers. Used in DDR4 designs only.

9.3.3.1.105 [SHADOW] SDRAM Initialization Register 7 (INIT7_SHADOW)

9.3.3.1.105.1 Offset

Register	Offset
INIT7_SHADOW	20ECh

9.3.3.1.105.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	mr6															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.3.3.1.105.3 Fields

Field	Function
31-16 mr6	DDR4- Value to be loaded into SDRAM MR6 registers. Used in DDR4 designs only.
15-0 —	Reserved

9.3.3.1.106 [SHADOW] SDRAM Timing Register 0 (DRAMTMG0_SHADOW)

9.3.3.1.106.1 Offset

Register	Offset
DRAMTMG0_SHADOW	2100h

9.3.3.1.106.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved	wr2pre							Reserved		t_faw					
W																
Reset	0	0	0	0	1	1	1	1	0	0	0	1	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved	t_ras_max							Reserved		t_ras_min					
W																
Reset	0	0	0	1	1	0	1	1	0	0	0	0	1	1	1	1

9.3.3.1.106.3 Fields

Field	Function
31 —	Reserved
30-24 wr2pre	Minimum time between write and precharge to same bank. Unit: Clocks Specifications: $WL + BL/2 + tWR$ = approximately 8 cycles + 15 ns = 14 clocks @400MHz and less for lower frequencies where: - WL = write latency - BL = burst length. This must match the value programmed in the BL bit of the mode register to the SDRAM. BST (burst terminate) is not supported at present. - tWR = Write recovery time. This comes directly from the SDRAM specification. Add one extra cycle for LPDDR2/LPDDR3/LPDDR4 for this parameter. When the controller is operating in 1:2 frequency ratio mode, 1T mode, divide the above value by 2. No rounding up. When the controller is operating in 1:2 frequency ratio mode, 2T mode or LPDDR4 mode, divide the above value by 2 and round it up to the next integer value. Note that, depending on the PHY, if using LRDIMM, it may be necessary to adjust the value of this parameter to compensate for the extra cycle of latency through the LRDIMM.
23-22 —	Reserved
21-16 t_faw	tFAW Valid only when 8 or more banks(or banks x bank groups) are present. In 8-bank design, at most 4 banks must be activated in a rolling window of tFAW cycles. When the controller is operating in 1:2 frequency ratio mode, program this to (tFAW/2) and round up to next integer value. In a 4-bank design, set this register to 0x1 independent of the 1:1/1:2 frequency mode. Unit: Clocks

Table continues on the next page...

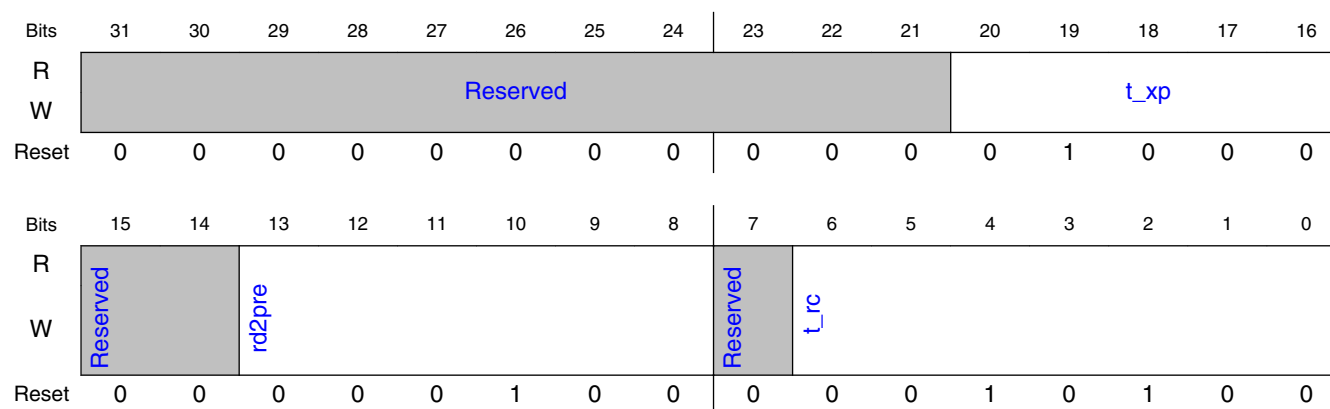
Field	Function
15 —	Reserved
14-8 t_ras_max	tRAS(max): Maximum time between activate and precharge to same bank. This is the maximum time that a page can be kept open Minimum value of this register is 1. Zero is invalid. When the controller is operating in 1:2 frequency ratio mode, program this to (tRAS(max)-1)/2. No rounding up. Unit: Multiples of 1024 clocks.
7-6 —	Reserved
5-0 t_ras_min	tRAS(min): Minimum time between activate and precharge to the same bank. When the controller is operating in 1:2 frequency mode, 1T mode, program this to tRAS(min)/2. No rounding up. When the controller is operating in 1:2 frequency ratio mode, 2T mode or LPDDR4 mode, program this to (tRAS(min)/2) and round it up to the next integer value. Unit: Clocks

9.3.3.1.107 [SHADOW] SDRAM Timing Register 1 (DRAMTMG1_SHADOW)

9.3.3.1.107.1 Offset

Register	Offset
DRAMTMG1_SHADOW	2104h

9.3.3.1.107.2 Diagram



9.3.3.1.107.3 Fields

Field	Function
31-21 —	Reserved

Table continues on the next page...

DDR Controller (DDRC)

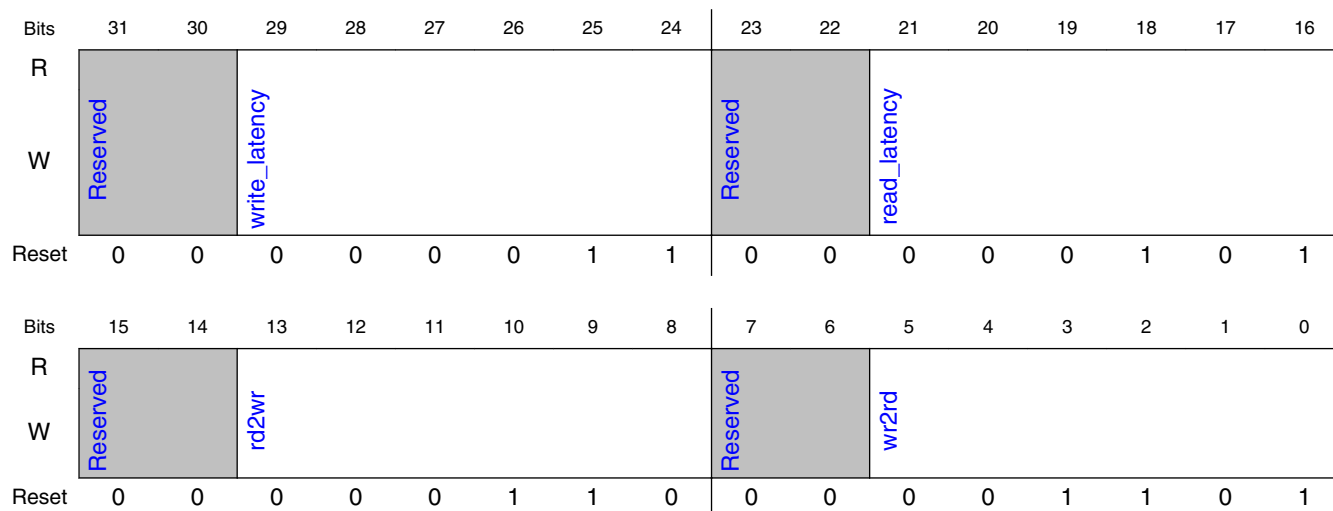
Field	Function
20-16 t_xp	tXP: Minimum time after power-down exit to any operation. For DDR3, this should be programmed to tXPDLL if slow powerdown exit is selected in MR0[12]. If C/A parity for DDR4 is used, set to (tXP+PL) instead. When the controller is operating in 1:2 frequency ratio mode, program this to (tXP/2) and round it up to the next integer value. Units: Clocks
15-14 —	Reserved
13-8 rd2pre	tRTP: Minimum time from read to precharge of same bank. - DDR2: tAL + BL/2 + max(tRTP, 2) - 2 - DDR3: tAL + max (tRTP, 4) - DDR4: Max of following two equations: tAL + max (tRTP, 4) or, RL + BL/2 - tRP (*). - mDDR: BL/2 - LPDDR2: Depends on if it's LPDDR2-S2 or LPDDR2-S4: LPDDR2-S2: BL/2 + tRTP - 1. LPDDR2-S4: BL/2 + max(tRTP,2) - 2. - LPDDR3: BL/2 + max(tRTP,4) - 4 - LPDDR4: BL/2 + max(tRTP,8) - 8 (*) When both DDR4 SDRAM and ST-MRAM are used simultaneously, use SDRAM's tRP value for calculation. When the controller is operating in 1:2 mode, 1T mode, divide the above value by 2. No rounding up. When the controller is operating in 1:2 mode, 2T mode or LPDDR4 mode, divide the above value by 2 and round it up to the next integer value. Unit: Clocks.
7 —	Reserved
6-0 t_rc	tRC: Minimum time between activates to same bank. When the controller is operating in 1:2 frequency ratio mode, program this to (tRC/2) and round up to next integer value. Unit: Clocks.

9.3.3.1.108 [SHADOW] SDRAM Timing Register 2 (DRAMTMG2_SHADOW)

9.3.3.1.108.1 Offset

Register	Offset
DRAMTMG2_SHADOW	2108h

9.3.3.1.108.2 Diagram



9.3.3.1.108.3 Fields

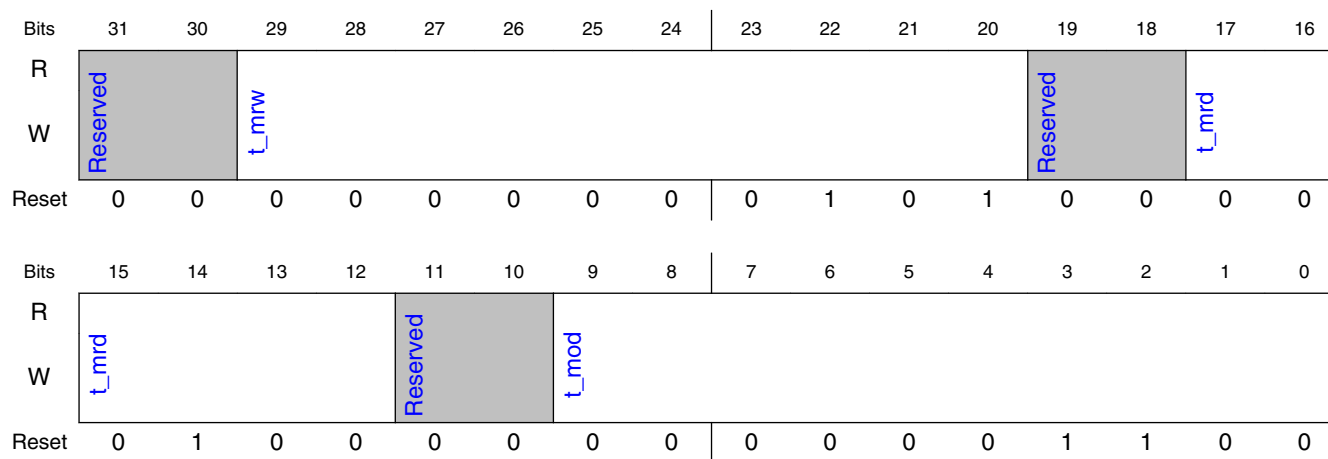
Field	Function
31-30 —	Reserved
29-24 write_latency	Set to WL Time from write command to write data on SDRAM interface. This must be set to WL. For mDDR, it should normally be set to 1. Note that, depending on the PHY, if using RDIMM/LRDIMM, it may be necessary to adjust the value of WL to compensate for the extra cycle of latency through the RDIMM/LRDIMM. When the controller is operating in 1:2 frequency ratio mode, divide the value calculated using the above equation by 2, and round it up to next integer. This register field is not required for DDR2 and DDR3 (except if MEMC_TRAINING is set), as the DFI read and write latencies defined in DFITMG0 and DFITMG1 are sufficient for those protocols Unit: clocks
23-22 —	Reserved
21-16 read_latency	Set to RL Time from read command to read data on SDRAM interface. This must be set to RL. Note that, depending on the PHY, if using RDIMM/LRDIMM, it may be necessary to adjust the value of RL to compensate for the extra cycle of latency through the RDIMM/LRDIMM. When the controller is operating in 1:2 frequency ratio mode, divide the value calculated using the above equation by 2, and round it up to next integer. This register field is not required for DDR2 and DDR3 (except if MEMC_TRAINING is set), as the DFI read and write latencies defined in DFITMG0 and DFITMG1 are sufficient for those protocols Unit: clocks
15-14 —	Reserved
13-8 rd2wr	DDR2/3/mDDR: $RL + BL/2 + 2 - WL$ DDR4: $RL + BL/2 + 1 + WR_PREAMBLE - WL$ LPDDR2/LPDDR3: $RL + BL/2 + RU(tDQSKmax/tCK) + 1 - WL$ LPDDR4(DQ ODT is Disabled): $RL + BL/2 + RU(tDQSKmax/tCK) + WR_PREAMBLE + RD_POSTAMBLE - WL$ LPDDR4(DQ ODT is Enabled) : $RL + BL/2 + RU(tDQSKmax/tCK) + RD_POSTAMBLE - ODTLon - RU(tODTon(min)/tCK)$ Minimum time from read command to write command. Include time for bus turnaround and all per-bank, per-rank, and global constraints. Please see the relevant PHY databook for details of what should be included here. Unit: Clocks. Where: - WL = write latency - BL = burst length. This must match the value programmed in the BL bit of the mode register to the SDRAM - RL = read latency = CAS latency - WR_PREAMBLE = write preamble. This is unique to DDR4 and LPDDR4. - RD_POSTAMBLE = read postamble. This is unique to LPDDR4. For LPDDR2/LPDDR3/LPDDR4, if derating is enabled (DERATEEN.derate_enable=1), derated tDQSKmax should be used. When the controller is operating in 1:2 frequency ratio mode, divide the value calculated using the above equation by 2, and round it up to next integer. Note that, depending on the PHY, if using LRDIMM, it may be necessary to adjust the value of this parameter to compensate for the extra cycle of latency through the LRDIMM.
7-6 —	Reserved
5-0 wr2rd	DDR4: $CWL + PL + BL/2 + tWTR_L$ Others: $CWL + BL/2 + tWTR$ In DDR4, minimum time from write command to read command for same bank group. In others, minimum time from write command to read command. Includes time for bus turnaround, recovery times, and all per-bank, per-rank, and global constraints. Unit: Clocks. Where: - CWL = CAS write latency - PL = Parity latency - BL = burst length. This must match the value programmed in the BL bit of the mode register to the SDRAM - tWTR_L = internal write to read command delay for same bank group. This comes directly from the SDRAM specification. - tWTR = internal write to read command delay. This comes directly from the SDRAM specification. Add one extra cycle for LPDDR2/LPDDR3/LPDDR4 operation. When the controller is operating in 1:2 mode, divide the value calculated using the above equation by 2, and round it up to next integer.

9.3.3.1.109 [SHADOW] SDRAM Timing Register 3 (DRAMTMG3_SHADOW)

9.3.3.1.109.1 Offset

Register	Offset
DRAMTMG3_SHADOW	210Ch

9.3.3.1.109.2 Diagram



9.3.3.1.109.3 Fields

Field	Function
31-30 —	Reserved
29-20 t_mrw	Time to wait after a mode register write or read (MRW or MRR). Present only in designs configured to support LPDDR2, LPDDR3 or LPDDR4. LPDDR2 typically requires value of 5. LPDDR3 typically requires value of 10. LPDDR4: Set this to the larger of tMRW and tMRWCKEL. For LPDDR2, this register is used for the time from a MRW/MRR to all other commands. When the controller is operating in 1:2 frequency ratio mode, program this to the above values divided by 2 and round it up to the next integer value. For LPDDR3, this register is used for the time from a MRW/MRR to a MRW/MRR.
19-18 —	Reserved
17-12 t_mrd	tMRD: Cycles to wait after a mode register write or read. Depending on the connected SDRAM, tMRD represents: DDR2/mDDR: Time from MRS to any command DDR3/4: Time from MRS to MRS command LPDDR2: not used LPDDR3/4: Time from MRS to non-MRS command. When the controller is operating in 1:2 frequency ratio mode, program this to (tMRD/2) and round it up to the next integer value. If C/A parity for DDR4 is used, set to tMRD_PAR(tMOD+PL) instead.
11-10 —	Reserved

Table continues on the next page...

Field	Function
9-0 t_mod	tMOD: Parameter used only in DDR3 and DDR4. Cycles between load mode command and following non-load mode command. If C/A parity for DDR4 is used, set to tMOD_PAR(tMOD+PL) instead. Set to tMOD if controller is operating in 1:1 frequency ratio mode, or tMOD/2 (rounded up to next integer) if controller is operating in 1:2 frequency ratio mode. Note that if using RDIMM/LRDIMM, depending on the PHY, it may be necessary to adjust the value of this parameter to compensate for the extra cycle of latency applied to mode register writes by the RDIMM/LRDIMM chip. Also note that if using LRDIMM, the minimum value of this register is tMRD_L2 if controller is operating in 1:1 frequency ratio mode, or tMRD_L2/2 (rounded up to next integer) if controller is operating in 1:2 frequency ratio mode.

9.3.3.1.110 [SHADOW] SDRAM Timing Register 4 (DRAMTMG4_SHADOW)

9.3.3.1.110.1 Offset

Register	Offset
DRAMTMG4_SHADOW	2110h

9.3.3.1.110.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved			t_rcd					Reserved				t_ccd			
W	Reserved			t_rcd					Reserved				t_ccd			
Reset	0	0	0	0	0	1	0	1	0	0	0	0	0	1	0	0

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				t_rrd				Reserved			t_rp				
W	Reserved				t_rrd				Reserved			t_rp				
Reset	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0	1

9.3.3.1.110.3 Fields

Field	Function
31-29 —	Reserved
28-24 t_rcd	tRCD - tAL: Minimum time from activate to read or write command to same bank. When the controller is operating in 1:2 frequency ratio mode, program this to ((tRCD - tAL)/2) and round it up to the next integer value. Minimum value allowed for this register is 1, which implies minimum (tRCD - tAL) value to be 2 when the controller is operating in 1:2 frequency ratio mode. Unit: Clocks.
23-20 —	Reserved

Table continues on the next page...

DDR Controller (DDRC)

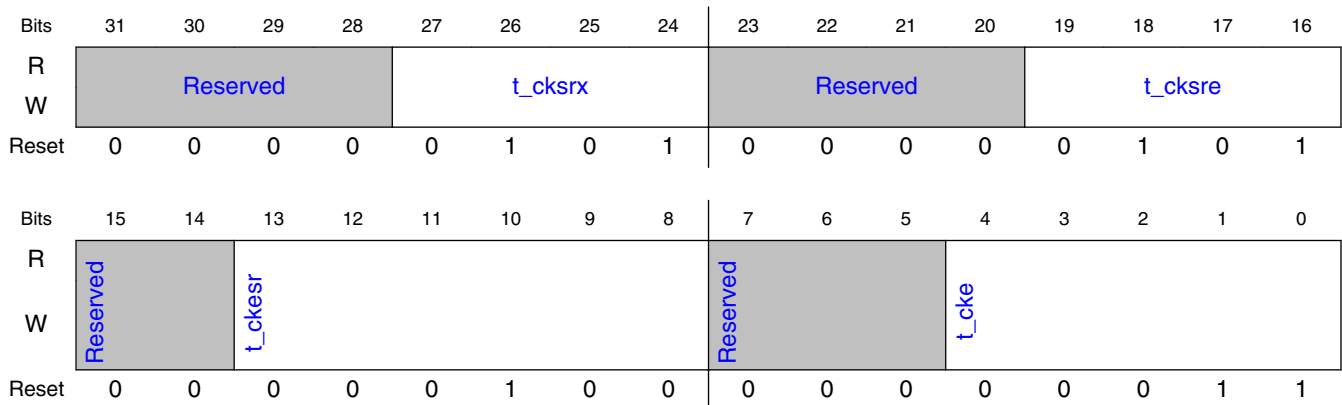
Field	Function
19-16 t_ccd	DDR4: tCCD_L: This is the minimum time between two reads or two writes for same bank group. Others: tCCD: This is the minimum time between two reads or two writes. When the controller is operating in 1:2 frequency ratio mode, program this to (tCCD_L/2 or tCCD/2) and round it up to the next integer value. Unit: clocks.
15-12 —	Reserved
11-8 t_rrd	DDR4: tRRD_L: Minimum time between activates from bank "a" to bank "b" for same bank group. Others: tRRD: Minimum time between activates from bank "a" to bank "b" When the controller is operating in 1:2 frequency ratio mode, program this to (tRRD_L/2 or tRRD/2) and round it up to the next integer value. Unit: Clocks.
7-5 —	Reserved
4-0 t_rp	tRP: Minimum time from precharge to activate of same bank. When the controller is operating in 1:1 frequency ratio mode, t_rp should be set to RoundUp(tRP/tCK). When the controller is operating in 1:2 frequency ratio mode, t_rp should be set to RoundDown(RoundUp(tRP/tCK)/2) + 1. When the controller is operating in 1:2 frequency ratio mode in LPDDR4, t_rp should be set to RoundUp(RoundUp(tRP/tCK)/2). Unit: Clocks.

9.3.3.1.111 [SHADOW] SDRAM Timing Register 5 (DRAMTMG5_SHADOW)

9.3.3.1.111.1 Offset

Register	Offset
DRAMTMG5_SHADOW	2114h

9.3.3.1.111.2 Diagram



9.3.3.1.111.3 Fields

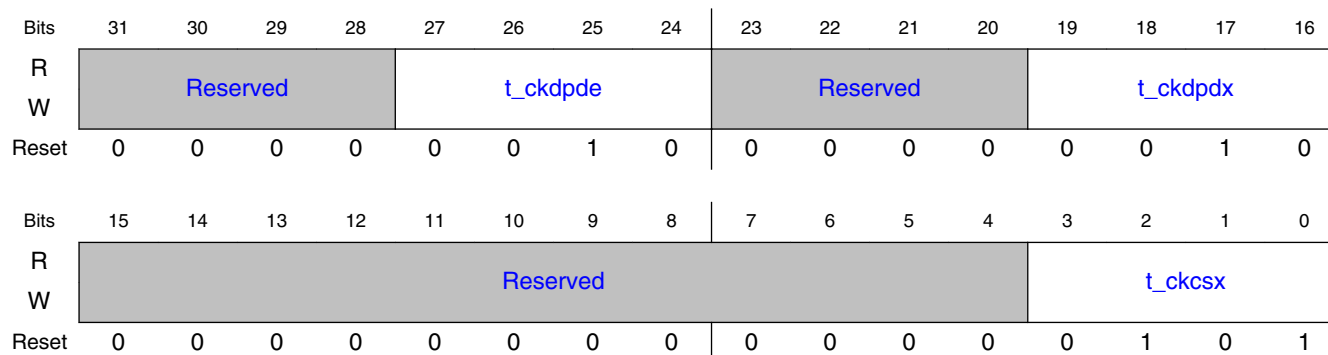
Field	Function
31-28 —	Reserved
27-24 t_cksrx	This is the time before Self Refresh Exit that CK is maintained as a valid clock before issuing SRX. Specifies the clock stable time before SRX. Recommended settings: - mDDR: 1 - LPDDR2: 2 - LPDDR3: 2 - LPDDR4: tCKCKEH - DDR2: 1 - DDR3: tCKSRX - DDR4: tCKSRX When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer.
23-20 —	Reserved
19-16 t_cksre	This is the time after Self Refresh Down Entry that CK is maintained as a valid clock. Specifies the clock disable delay after SRE. Recommended settings: - mDDR: 0 - LPDDR2: 2 - LPDDR3: 2 - LPDDR4: tCKCKEL - DDR2: 1 - DDR3: max (10 ns, 5 tCK) - DDR4: max (10 ns, 5 tCK) (+ PL(parity latency)(*)) (*)Only if CRCPARCTL1.caparity_disable_before_sr=0, this register should be increased by PL. When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer.
15-14 —	Reserved
13-8 t_ckesr	Minimum CKE low width for Self refresh or Self refresh power down entry to exit timing in memory clock cycles. Recommended settings: - mDDR: tRFC - LPDDR2: tCKESR - LPDDR3: tCKESR - LPDDR4: max(tCKELPD, tSR) - DDR2: tCKE - DDR3: tCKE + 1 - DDR4: tCKE + 1 (+ PL(parity latency)(*)) (*)Only if CRCPARCTL1.caparity_disable_before_sr=0, this register should be increased by PL. When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer.
7-5 —	Reserved
4-0 t_cke	Minimum number of cycles of CKE HIGH/LOW during power-down and self refresh. - LPDDR2/LPDDR3 mode: Set this to the larger of tCKE or tCKESR - LPDDR4 mode: Set this to the larger of tCKE, tCKELPD or tSR. - Non-LPDDR2/non-LPDDR3/non-LPDDR4 designs: Set this to tCKE value. When the controller is operating in 1:2 frequency ratio mode, program this to (value described above)/2 and round it up to the next integer value. Unit: Cycles.

9.3.3.1.112 [SHADOW] SDRAM Timing Register 6 (DRAMTMG6_SHADOW)

9.3.3.1.112.1 Offset

Register	Offset
DRAMTMG6_SHADOW	2118h

9.3.3.1.112.2 Diagram



9.3.3.1.112.3 Fields

Field	Function
31-28 —	Reserved
27-24 t_ckdpde	This is the time after Deep Power Down Entry that CK is maintained as a valid clock. Specifies the clock disable delay after DPDE. Recommended settings: - mDDR: 0 - LPDDR2: 2 - LPDDR3: 2 When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer. This is only present for designs supporting mDDR or LPDDR2/LPDDR3 devices.
23-20 —	Reserved
19-16 t_ckdpdx	This is the time before Deep Power Down Exit that CK is maintained as a valid clock before issuing DPDX. Specifies the clock stable time before DPDX. Recommended settings: - mDDR: 1 - LPDDR2: 2 - LPDDR3: 2 When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer. This is only present for designs supporting mDDR or LPDDR2 devices.
15-4 —	Reserved
3-0 t_ckcsx	This is the time before Clock Stop Exit that CK is maintained as a valid clock before issuing Clock Stop Exit. Specifies the clock stable time before next command after Clock Stop Exit. Recommended settings: - mDDR: 1 - LPDDR2: tXP + 2 - LPDDR3: tXP + 2 - LPDDR4: tXP + 2 When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer. This is only present for designs supporting mDDR or LPDDR2/LPDDR3/LPDDR4 devices.

9.3.3.1.113 [SHADOW] SDRAM Timing Register 7 (DRAMTMG7_SHADOW)

9.3.3.1.113.1 Offset

Register	Offset
DRAMTMG7_SHADOW	211Ch

9.3.3.1.113.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				t_ckpde				Reserved				t_ckpdx			
W																
Reset	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0

9.3.3.1.113.3 Fields

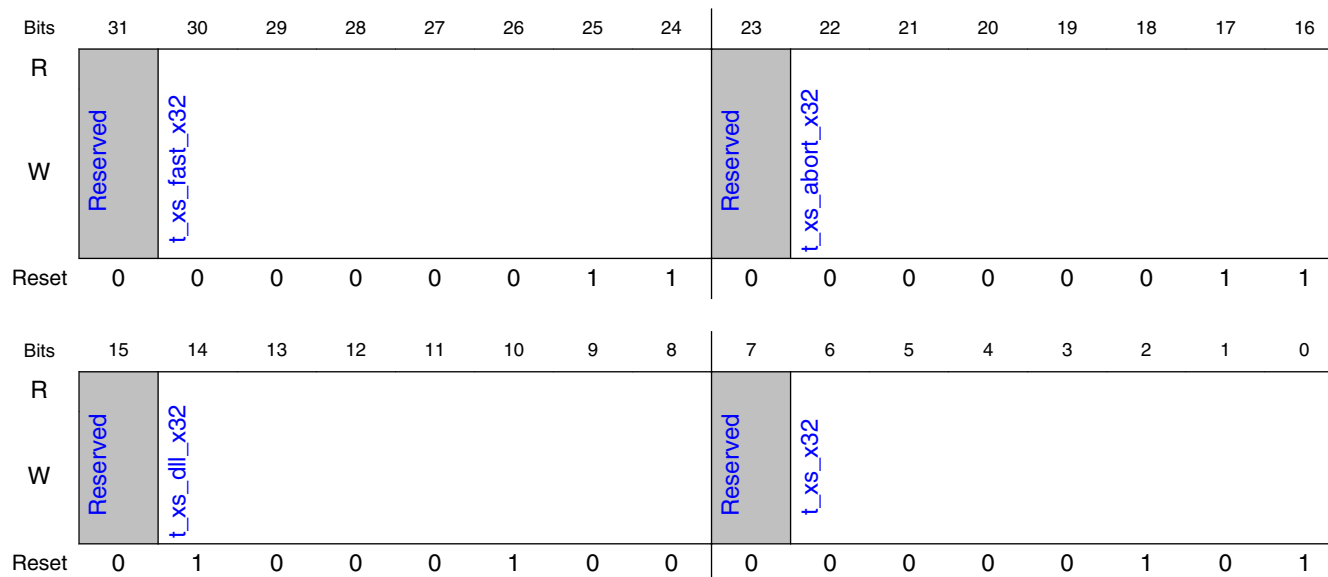
Field	Function
31-12 —	Reserved
11-8 t_ckpde	This is the time after Power Down Entry that CK is maintained as a valid clock. Specifies the clock disable delay after PDE. Recommended settings: - mDDR: 0 - LPDDR2: 2 - LPDDR3: 2 - LPDDR4: tCKCKEL When using DDR2/3/4 SDRAM, this register should be set to the same value as DRAMTMG5.t_cksre. When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer. This is only present for designs supporting mDDR or LPDDR2/LPDDR3/LPDDR4 devices.
7-4 —	Reserved
3-0 t_ckpdx	This is the time before Power Down Exit that CK is maintained as a valid clock before issuing PDX. Specifies the clock stable time before PDX. Recommended settings: - mDDR: 0 - LPDDR2: 2 - LPDDR3: 2 - LPDDR4: 2 When using DDR2/3/4 SDRAM, this register should be set to the same value as DRAMTMG5.t_cksrx. When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer. This is only present for designs supporting mDDR or LPDDR2/LPDDR3/LPDDR4 devices.

9.3.3.1.114 [SHADOW] SDRAM Timing Register 8 (DRAMTMG8_SHADOW)

9.3.3.1.114.1 Offset

Register	Offset
DRAMTMG8_SHADOW	2120h

9.3.3.1.114.2 Diagram



9.3.3.1.114.3 Fields

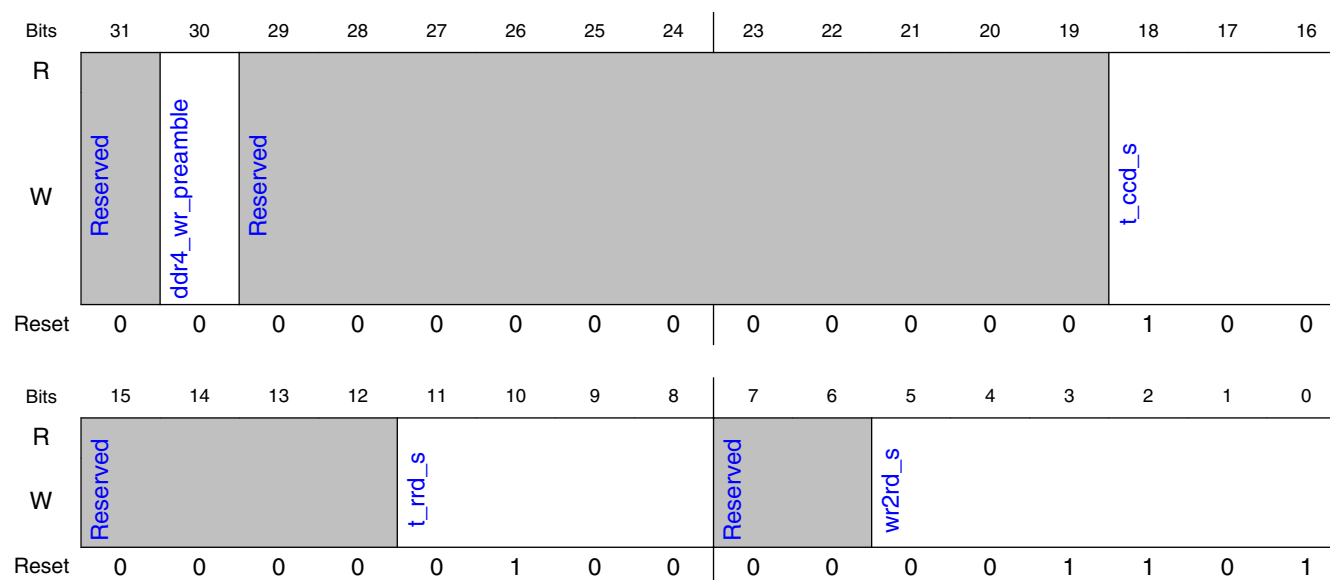
Field	Function
31 —	Reserved
30-24 t_xs_fast_x32	tXS_FAST: Exit Self Refresh to ZQCL, ZQCS and MRS (only CL, WR, RTP and Geardown mode). When the controller is operating in 1:2 frequency ratio mode, program this to the above value divided by 2 and round up to next integer value. Unit: Multiples of 32 clocks. Note: This is applicable to only ZQCL/ZQCS commands. Note: Ensure this is less than or equal to t_xs_x32.
23 —	Reserved
22-16 t_xs_abort_x32	tXS_ABORT: Exit Self Refresh to commands not requiring a locked DLL in Self Refresh Abort. When the controller is operating in 1:2 frequency ratio mode, program this to the above value divided by 2 and round up to next integer value. Unit: Multiples of 32 clocks. Note: Ensure this is less than or equal to t_xs_x32.
15 —	Reserved
14-8 t_xs_dll_x32	tXSDLL: Exit Self Refresh to commands requiring a locked DLL. When the controller is operating in 1:2 frequency ratio mode, program this to the above value divided by 2 and round up to next integer value. Unit: Multiples of 32 clocks. Note: Used only for DDR2, DDR3 and DDR4 SDRAMs.
7 —	Reserved
6-0 t_xs_x32	tXS: Exit Self Refresh to commands not requiring a locked DLL. When the controller is operating in 1:2 frequency ratio mode, program this to the above value divided by 2 and round up to next integer value. Unit: Multiples of 32 clocks. Note: Used only for DDR2, DDR3 and DDR4 SDRAMs.

9.3.3.1.115 [SHADOW] SDRAM Timing Register 9 (DRAMTMG9_SHADOW)

9.3.3.1.115.1 Offset

Register	Offset
DRAMTMG9_SHADOW	2124h

9.3.3.1.115.2 Diagram



9.3.3.1.115.3 Fields

Field	Function
31 —	Reserved
30 ddr4_wr_preamble	DDR4 Write preamble mode - 0: 1tCK preamble - 1: 2tCK preamble Present only with MEMC_FREQ_RATIO=2
29-19 —	Reserved
18-16 t_ccd_s	tCCD_S: This is the minimum time between two reads or two writes for different bank group. For bank switching (from bank "a" to bank "b"), the minimum time is this value + 1. When the controller is operating in 1:2 frequency ratio mode, program this to (tCCD_S/2) and round it up to the next integer value. Present only in designs configured to support DDR4. Unit: clocks.
15-12 —	Reserved

Table continues on the next page...

DDR Controller (DDRC)

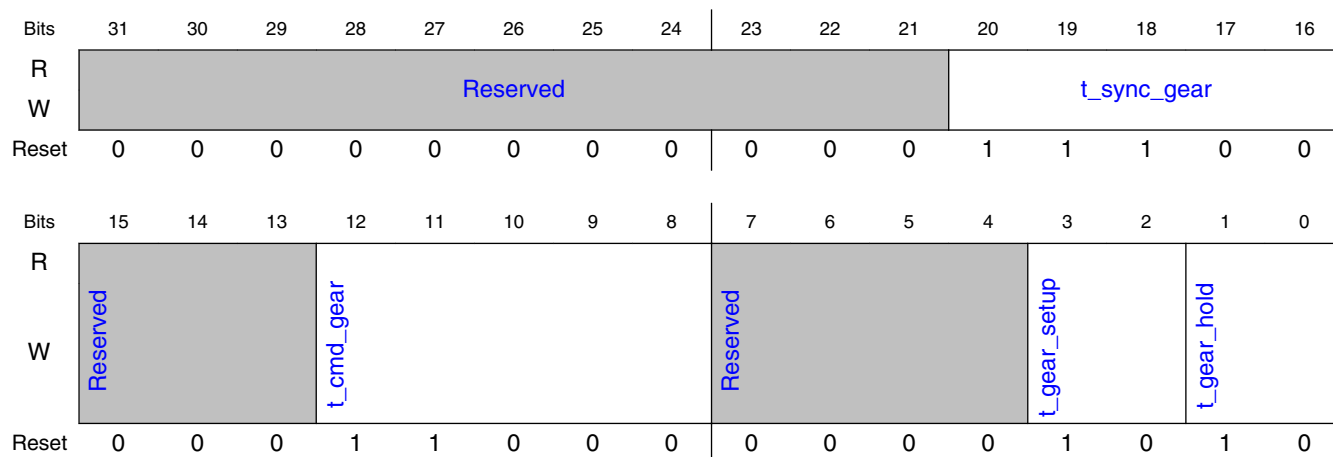
Field	Function
11-8 t_rrd_s	tRRD_S: Minimum time between activates from bank "a" to bank "b" for different bank group. When the controller is operating in 1:2 frequency ratio mode, program this to (tRRD_S/2) and round it up to the next integer value. Present only in designs configured to support DDR4. Unit: Clocks.
7-6 —	Reserved
5-0 wr2rd_s	CWL + PL + BL/2 + tWTR_S Minimum time from write command to read command for different bank group. Includes time for bus turnaround, recovery times, and all per-bank, per-rank, and global constraints. Present only in designs configured to support DDR4. Unit: Clocks. Where: - CWL = CAS write latency - PL = Parity latency - BL = burst length. This must match the value programmed in the BL bit of the mode register to the SDRAM - tWTR_S = internal write to read command delay for different bank group. This comes directly from the SDRAM specification. When the controller is operating in 1:2 mode, divide the value calculated using the above equation by 2, and round it up to next integer.

9.3.3.1.116 [SHADOW] SDRAM Timing Register 10 (DRAMTMG10_SHADOW)

9.3.3.1.116.1 Offset

Register	Offset
DRAMTMG10_SHADOW	2128h

9.3.3.1.116.2 Diagram



9.3.3.1.116.3 Fields

Field	Function
31-21	Reserved

Table continues on the next page...

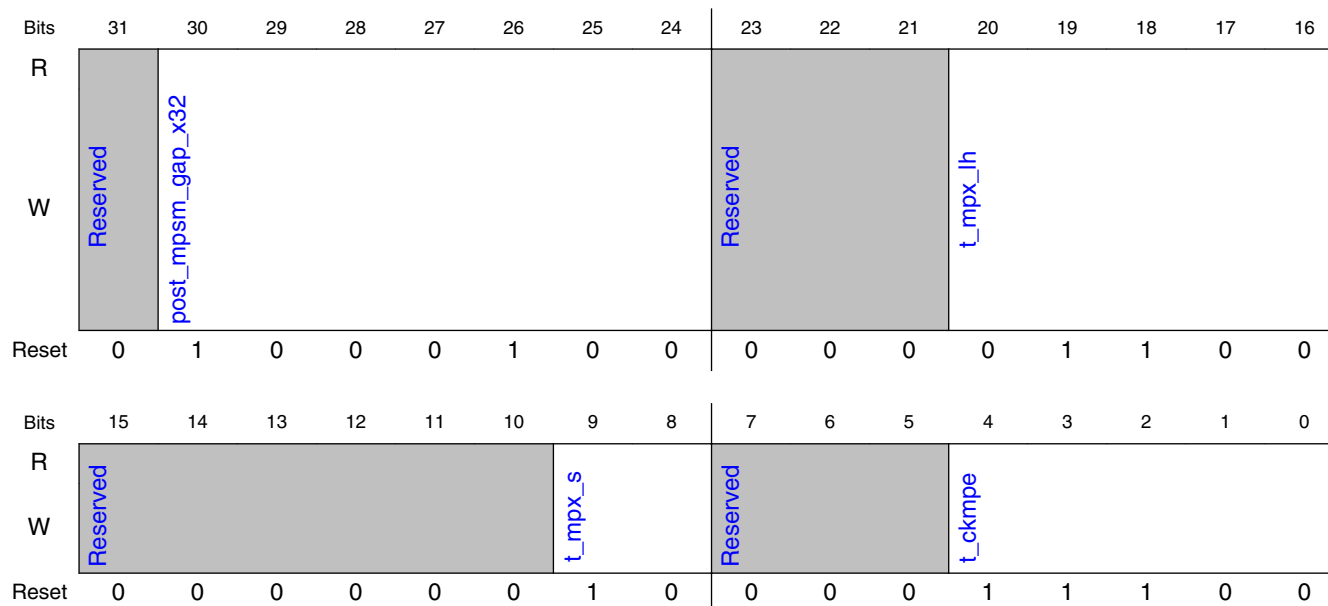
Field	Function
—	
20-16 t_sync_gear	Indicates the time between MRS command and the sync pulse time. This must be even number of clocks. For DDR4-2666 and DDR4-3200, this parameter is defined as $t_{MOD(min)} + 4nCK$ $t_{MOD(min)}$ is greater of $24nCK$ or $15ns$ $15ns / .625ns = 24$ Max value for this register is $24+4 = 28$ When the controller is operating in 1:2 mode, program this to $(t_{SYNC_GEAR}/2)$ and round it up to the next integer value. Unit: Clocks
15-13 —	Reserved
12-8 t_cmd_gear	Sync pulse to first valid command. For DDR4-2666 and DDR4-3200, this parameter is defined as $t_{MOD(min)}$ $t_{MOD(min)}$ is greater of $24nCK$ or $15ns$ $15ns / .625ns = 24$ Max value for this register is 24 When the controller is operating in 1:2 mode, program this to $(t_{CMD_GEAR}/2)$ and round it up to the next integer value. Unit: Clocks
7-4 —	Reserved
3-2 t_gear_setup	Geardown setup time. Minimum value of this register is 1. Zero is invalid. For DDR4-2666 and DDR4-3200, this parameter is defined as 2 clks When the controller is operating in 1:2 frequency ratio mode, program this to $(t_{GEAR_setup}/2)$ and round it up to the next integer value. Unit: Clocks
1-0 t_gear_hold	Geardown hold time. Minimum value of this register is 1. Zero is invalid. For DDR4-2666 and DDR4-3200, this parameter is defined as 2 clks When the controller is operating in 1:2 frequency ratio mode, program this to $(t_{GEAR_hold}/2)$ and round it up to the next integer value. Unit: Clocks

9.3.3.1.117 [SHADOW] SDRAM Timing Register 11 (DRAMTMG11_SHADOW)

9.3.3.1.117.1 Offset

Register	Offset
DRAMTMG11_SHADOW	212Ch

9.3.3.1.117.2 Diagram



9.3.3.1.117.3 Fields

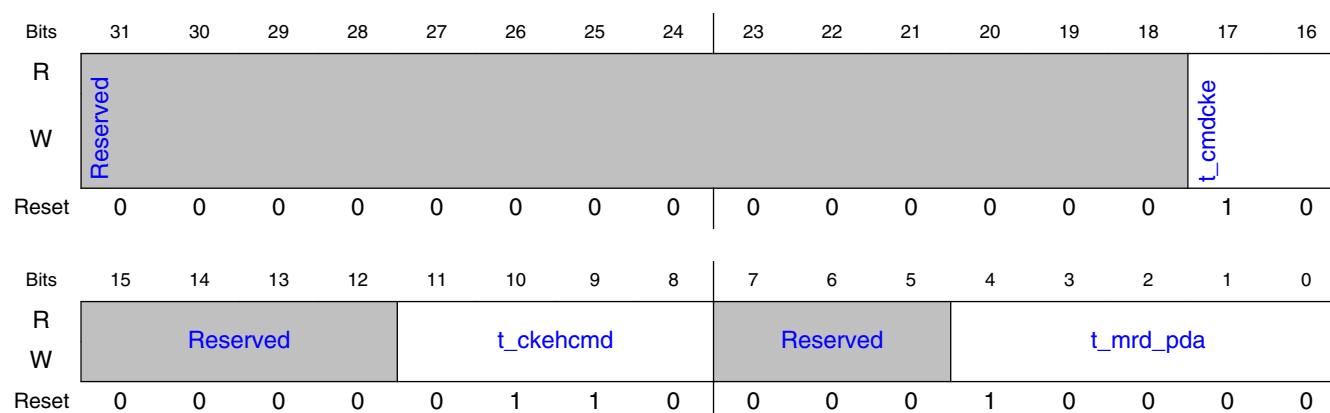
Field	Function
31 —	Reserved
30-24 post_mpsm_gap_x32	tXMPDLL: This is the minimum Exit MPSM to commands requiring a locked DLL. When the controller is operating in 1:2 frequency ratio mode, program this to (tXMPDLL/2) and round it up to the next integer value. Present only in designs configured to support DDR4. Unit: Multiples of 32 clocks.
23-21 —	Reserved
20-16 t_mpx_lh	tMPX_LH: This is the minimum CS_n Low hold time to CKE rising edge. When the controller is operating in 1:2 frequency ratio mode, program this to RoundUp(tMPX_LH/2)+1. Present only in designs configured to support DDR4. Unit: clocks.
15-10 —	Reserved
9-8 t_mpx_s	tMPX_S: Minimum time CS setup time to CKE. When the controller is operating in 1:2 frequency ratio mode, program this to (tMPX_S/2) and round it up to the next integer value. Present only in designs configured to support DDR4. Unit: Clocks.
7-5 —	Reserved
4-0 t_ckmpe	tCKMPE: Minimum valid clock requirement after MPSM entry. Present only in designs configured to support DDR4. Unit: Clocks. When the controller is operating in 1:2 frequency ratio mode, divide the value calculated using the above equation by 2, and round it up to next integer.

9.3.3.1.118 [SHADOW] SDRAM Timing Register 12 (DRAMTMG12_SHADOW)

9.3.3.1.118.1 Offset

Register	Offset
DRAMTMG12_SHADOW	2130h

9.3.3.1.118.2 Diagram



9.3.3.1.118.3 Fields

Field	Function
31-18 —	Reserved
17-16 t_cmdcke	tCMDCKE: Delay from valid command to CKE input LOW. Set this to the larger of tESCKE or tCMDCKE. When the controller is operating in 1:2 frequency ratio mode, program this to (max(tESCKE, tCMDCKE)/2) and round it up to the next integer value.
15-12 —	Reserved
11-8 t_ckehecmd	tCKEHCMDD: Valid command requirement after CKE input HIGH. When the controller is operating in 1:2 frequency ratio mode, program this to (tCKEHCMDD/2) and round it up to the next integer value.
7-5 —	Reserved
4-0 t_mrd_pda	tMRD_PDA: This is the Mode Register Set command cycle time in PDA mode. When the controller is operating in 1:2 frequency ratio mode, program this to (tMRD_PDA/2) and round it up to the next integer value.

9.3.3.1.119 [SHADOW] SDRAM Timing Register 13 (DRAMTMG13_SHADOW)

9.3.3.1.119.1 Offset

Register	Offset
DRAMTMG13_SHADOW	2134h

9.3.3.1.119.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved								Reserved				t _{ccd_mw}			
W																
Reset	0	0	0	1	1	1	0	0	0	0	1	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												t _{ppd}			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0

9.3.3.1.119.3 Fields

Field	Function
31 —	Reserved
30-24 odtloff	LPDDR4: tODTLoFF: This is the latency from CAS-2 command to tODTLoFF reference. When the controller is operating in 1:2 frequency ratio mode, program this to (tODTLoFF/2) and round it up to the next integer value. Unit: Clocks.
23-22 —	Reserved
21-16 t _{ccd_mw}	LPDDR4: tCCDMW: This is the minimum time from write or masked write to masked write command for same bank. When the controller is operating in 1:2 frequency ratio mode, program this to (tCCDMW/2) and round it up to the next integer value. Unit: Clocks.
15-3 —	Reserved
2-0 t _{ppd}	LPDDR4: tPPD: This is the minimum time from precharge to precharge command. When the controller is operating in 1:2 frequency ratio mode, program this to (tPPD/2) and round it up to the next integer value. Unit: Clocks.

9.3.3.1.120 [SHADOW] SDRAM Timing Register 14 (DRAMTMG14_SHADOW)

9.3.3.1.120.1 Offset

Register	Offset
DRAMTMG14_SHADOW	2138h

9.3.3.1.120.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				t_xsr											
W																
Reset	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0

9.3.3.1.120.3 Fields

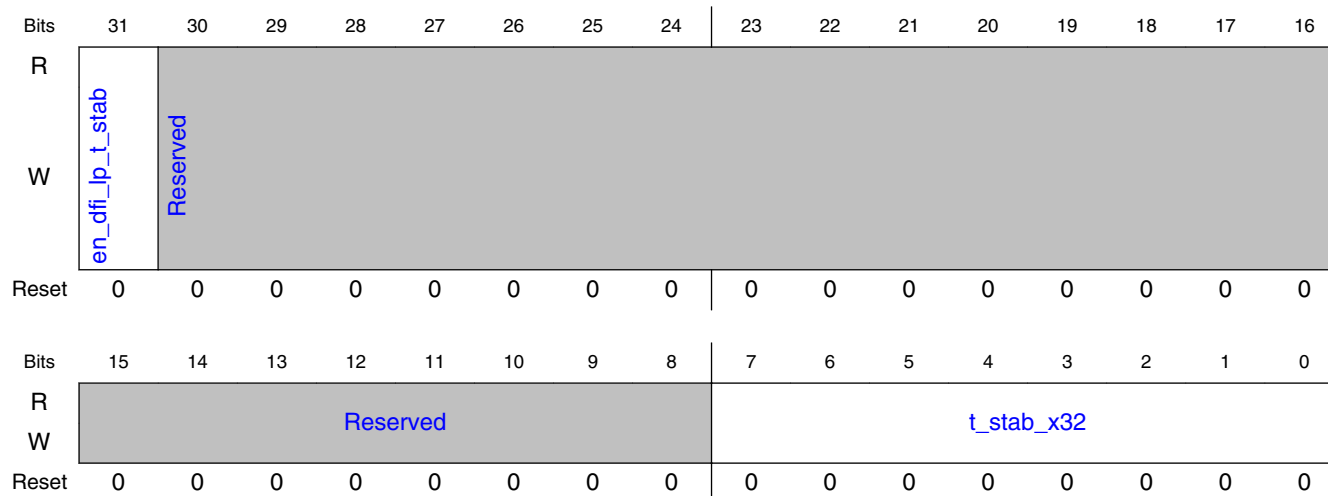
Field	Function
31-12	Reserved
—	
11-0 t_xsr	tXSR: Exit Self Refresh to any command. When the controller is operating in 1:2 frequency ratio mode, program this to the above value divided by 2 and round up to next integer value. Note: Used only for mDDR/LPDDR2/LPDDR3/LPDDR4 mode.

9.3.3.1.121 [SHADOW] SDRAM Timing Register 15 (DRAMTMG15_SHADOW)

9.3.3.1.121.1 Offset

Register	Offset
DRAMTMG15_SHADOW	213Ch

9.3.3.1.121.2 Diagram



9.3.3.1.121.3 Fields

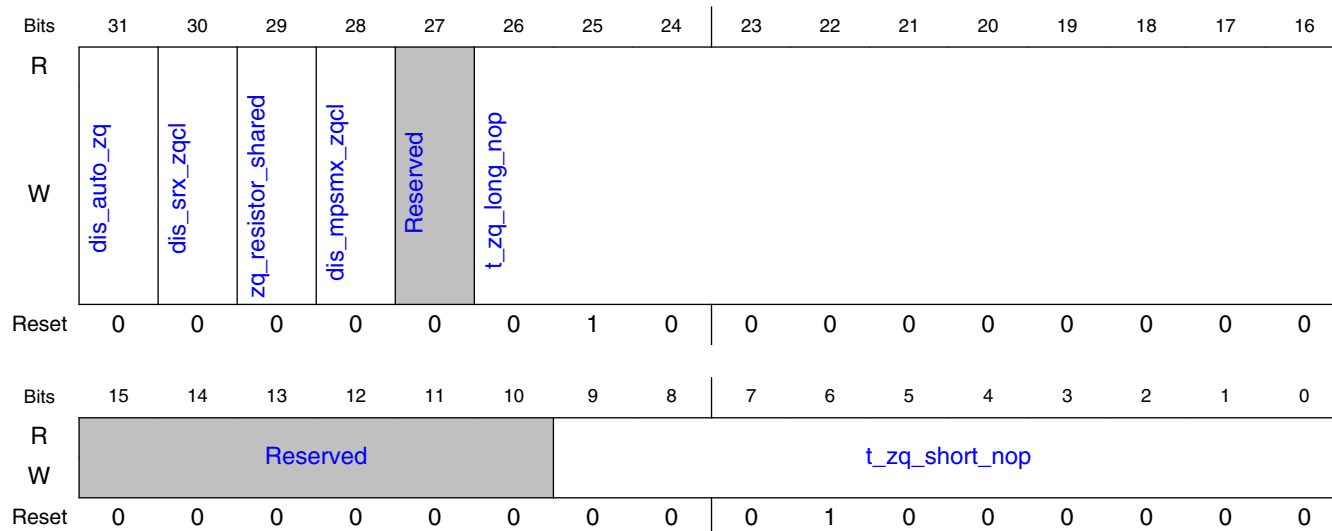
Field	Function
31 en_dfi_lp_t_stab	- 1 - Enable using tSTAB when exiting DFI LP. Needs to be set when the PHY is stopping the clock during DFI LP to save maximum power. - 0 - Disable using tSTAB when exiting DFI LP
30-8 —	Reserved
7-0 t_stab_x32	tSTAB: Stabilization time. It is required in the following two cases for DDR3/DDR4 RDIMM : - when exiting power saving mode, if the clock was stopped, after re-enabling it the clock must be stable for a time specified by tSTAB - in the case of input clock frequency change (DDR4) - after issuing control words that refers to clock timing (Specification: 6us for DDR3, 5us for DDR4) When the controller is operating in 1:2 frequency ratio mode, program this to recommended value divided by two and round it up to next integer. Unit: Multiples of 32 clock cycles.

9.3.3.1.122 [SHADOW] ZQ Control Register 0 (ZQCTL0_SHADOW)

9.3.3.1.122.1 Offset

Register	Offset
ZQCTL0_SHADOW	2180h

9.3.3.1.122.2 Diagram



9.3.3.1.122.3 Fields

Field	Function
31 dis_auto_zq	- 1 - Disable DDRC generation of ZQCS/MPC(ZQ calibration) command. Register DBGCMD.zq_calib_short can be used instead to issue ZQ calibration request from APB module. - 0 - Internally generate ZQCS/MPC(ZQ calibration) commands based on ZQCTL1.t_zq_short_interval_x1024. This is only present for designs supporting DDR3/DDR4 or LPDDR2/LPDDR3/LPDDR4 devices.
30 dis_srx_zqcl	- 1 - Disable issuing of ZQCL/MPC(ZQ calibration) command at Self-Refresh/SR-Powerdown exit. Only applicable when run in DDR3 or DDR4 or LPDDR2 or LPDDR3 or LPDDR4 mode. - 0 - Enable issuing of ZQCL/MPC(ZQ calibration) command at Self-Refresh/SR-Powerdown exit. Only applicable when run in DDR3 or DDR4 or LPDDR2 or LPDDR3 or LPDDR4 mode. This is only present for designs supporting DDR3/DDR4 or LPDDR2/LPDDR3/LPDDR4 devices.
29 zq_resistor_shared	- 1 - Denotes that ZQ resistor is shared between ranks. Means ZQinit/ZQCL/ZQCS/MPC(ZQ calibration) commands are sent to one rank at a time with tZQinit/tZQCL/tZQCS/tZQCAL/tZQLAT timing met between commands so that commands to different ranks do not overlap. - 0 - ZQ resistor is not shared. This is only present for designs supporting DDR3/DDR4 or LPDDR2/LPDDR3/LPDDR4 devices.
28 dis_mpsmx_zqcl	- 1 - Disable issuing of ZQCL command at Maximum Power Saving Mode exit. Only applicable when run in DDR4 mode. - 0 - Enable issuing of ZQCL command at Maximum Power Saving Mode exit. Only applicable when run in DDR4 mode. This is only present for designs supporting DDR4 devices. Note: Do not issue ZQCL command at Maximum Power Save Mode exit if the DDRC_SHARED_AC configuration parameter is set. Program it to 1'b1. The software can send ZQCS after exiting MPSM mode.
27 —	Reserved
26-16 t_zq_long_nop	tZQoper for DDR3/DDR4, tZQCL for LPDDR2/LPDDR3, tZQCAL for LPDDR4: Number of DFI clock cycles of NOP required after a ZQCL (ZQ calibration long)/MPC(ZQ Start) command is issued to SDRAM. When the controller is operating in 1:2 frequency ratio mode: DDR3/DDR4: program this to tZQoper/2 and round it up to the next integer value. LPDDR2/LPDDR3: program this to tZQCL/2 and round it up to the next integer value. LPDDR4: program this to tZQCAL/2 and round it up to the next integer value. This is only present for designs supporting DDR3/DDR4 or LPDDR2/LPDDR3/LPDDR4 devices.
15-10	Reserved

Table continues on the next page...

DDR Controller (DDRC)

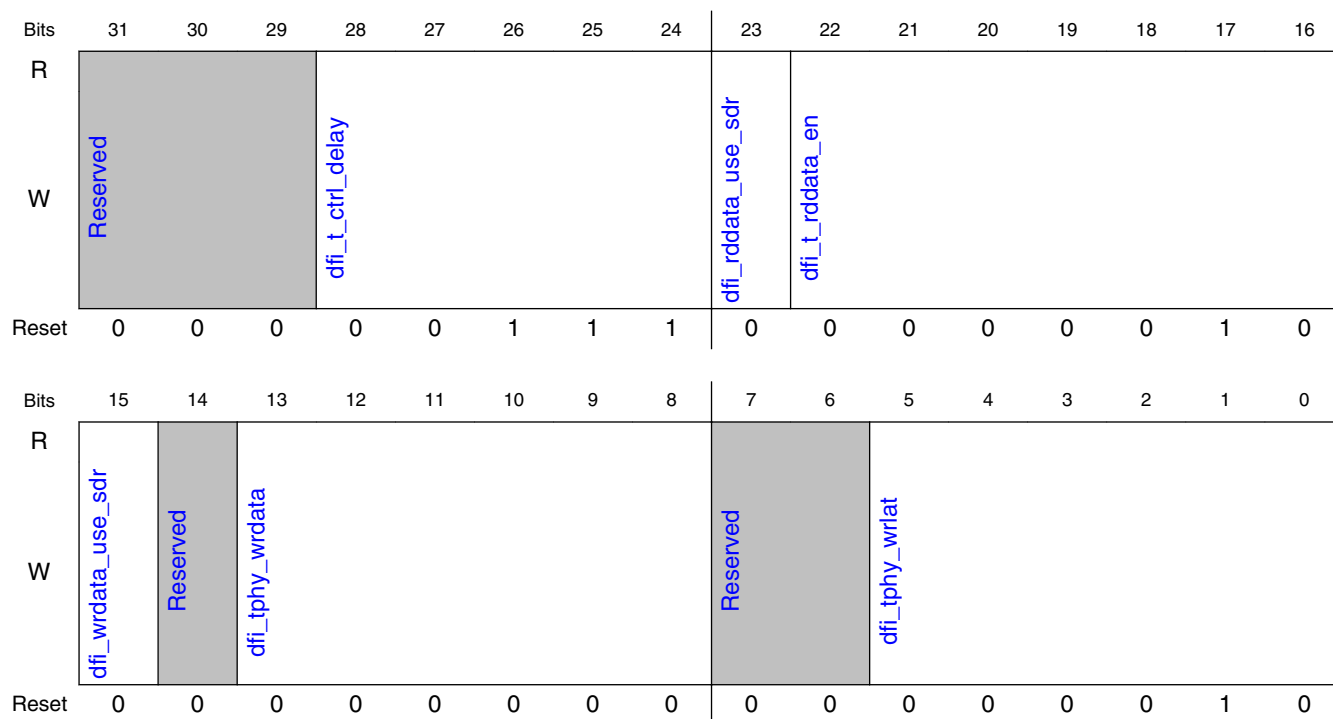
Field	Function
—	
9-0 t_zq_short_nop	tZQCS for DDR3/DD4/LPDDR2/LPDDR3, tZQLAT for LPDDR4: Number of DFI clock cycles of NOP required after a ZQCS (ZQ calibration short)/MPC(ZQ Latch) command is issued to SDRAM. When the controller is operating in 1:2 frequency ratio mode, program this to tZQCS/2 and round it up to the next integer value. This is only present for designs supporting DDR3/DDR4 or LPDDR2/LPDDR3/LPDDR4 devices.

9.3.3.1.123 [SHADOW] DFI Timing Register 0 (DFITMG0_SHADOW)

9.3.3.1.123.1 Offset

Register	Offset
DFITMG0_SHADOW	2190h

9.3.3.1.123.2 Diagram



9.3.3.1.123.3 Fields

Field	Function
31-29	Reserved

Table continues on the next page...

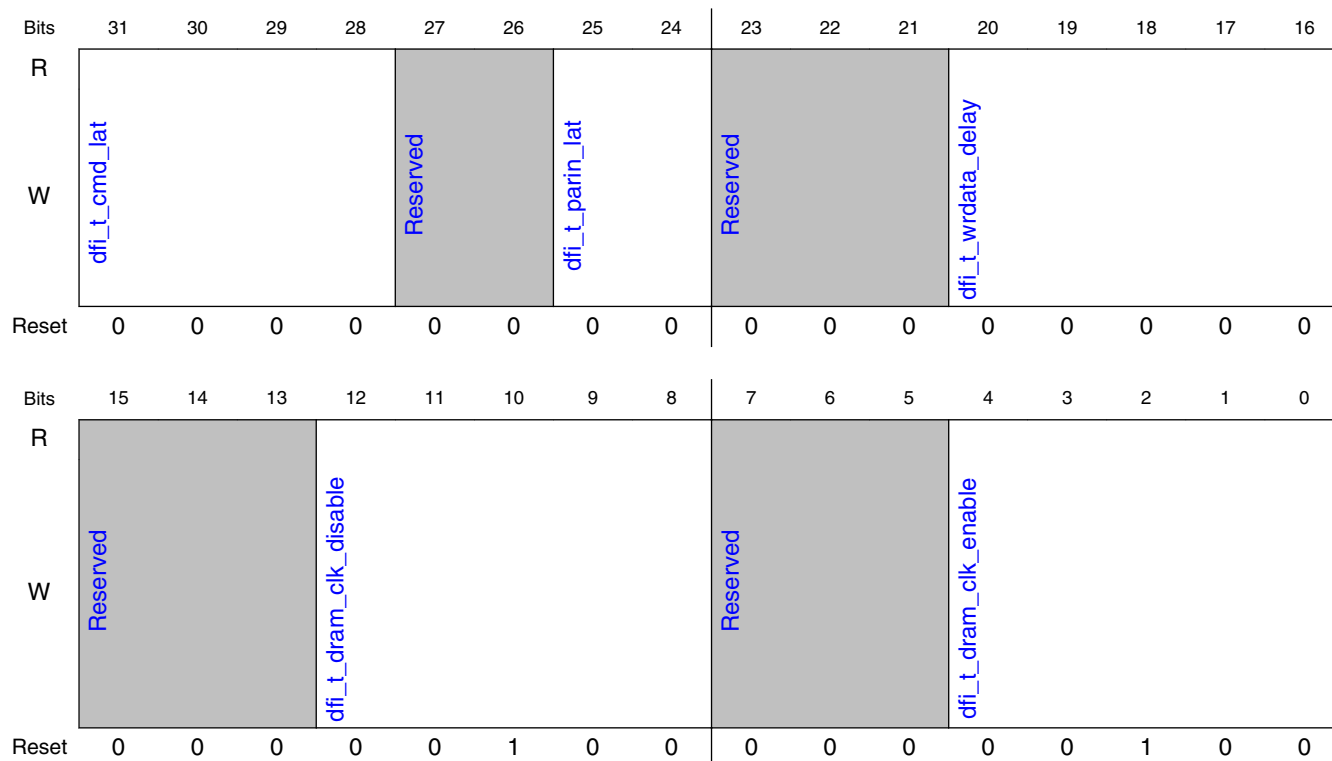
Field	Function
—	
28-24 dfi_t_ctrl_delay	Specifies the number of DFI clock cycles after an assertion or de-assertion of the DFI control signals that the control signals at the PHY-DRAM interface reflect the assertion or de-assertion. If the DFI clock and the memory clock are not phase-aligned, this timing parameter should be rounded up to the next integer value. Note that if using RDIMM/LRDIMM, it is necessary to increment this parameter by RDIMM's/LRDIMM's extra cycle of latency in terms of DFI clock.
23 dfi_rddata_use_sdr	Defines whether dfi_rddata_en/dfi_rddata/dfi_rddata_valid is generated using HDR (DFI clock) or SDR (DFI PHY clock) values. Selects whether value in DFITMG0.dfi_t_rddata_en is in terms of HDR (DFI clock) or SDR (DFI PHY clock) cycles: - 0 in terms of HDR (DFI clock) cycles - 1 in terms of SDR (DFI PHY clock) cycles Refer to PHY specification for correct value.
22-16 dfi_t_rddata_en	Time from the assertion of a read command on the DFI interface to the assertion of the dfi_rddata_en signal. Refer to PHY specification for correct value. This corresponds to the DFI parameter trddata_en. Note that, depending on the PHY, if using RDIMM/LRDIMM, it may be necessary to use the adjusted value of CL in the calculation of trddata_en. This is to compensate for the extra cycle(s) of latency through the RDIMM/LRDIMM. Unit: DFI clock cycles or DFI PHY clock cycles, depending on DFITMG0.dfi_rddata_use_sdr.
15 dfi_wrdata_use_sdr	Defines whether dfi_wrdata_en/dfi_wrdata/dfi_wrdata_mask is generated using HDR (DFI clock) or SDR (DFI PHY clock) values Selects whether value in DFITMG0.dfi_tphy_wrlat is in terms of HDR (DFI clock) or SDR (DFI PHY clock) cycles Selects whether value in DFITMG0.dfi_tphy_wrdata is in terms of HDR (DFI clock) or SDR (DFI PHY clock) cycles - 0 in terms of HDR (DFI clock) cycles - 1 in terms of SDR (DFI PHY clock) cycles Refer to PHY specification for correct value.
14 —	Reserved
13-8 dfi_tphy_wrdata	Specifies the number of clock cycles between when dfi_wrdata_en is asserted to when the associated write data is driven on the dfi_wrdata signal. This corresponds to the DFI timing parameter tphy_wrdata. Refer to PHY specification for correct value. Note, max supported value is 8. Unit: DFI clock cycles or DFI PHY clock cycles, depending on DFITMG0.dfi_wrdata_use_sdr.
7-6 —	Reserved
5-0 dfi_tphy_wrlat	Write latency Number of clocks from the write command to write data enable (dfi_wrdata_en). This corresponds to the DFI timing parameter tphy_wrlat. Refer to PHY specification for correct value. Note that, depending on the PHY, if using RDIMM/LRDIMM, it may be necessary to use the adjusted value of CL in the calculation of tphy_wrlat. This is to compensate for the extra cycle(s) of latency through the RDIMM/LRDIMM. Unit: DFI clock cycles or DFI PHY clock cycles, depending on DFITMG0.dfi_wrdata_use_sdr.

9.3.3.1.124 [SHADOW] DFI Timing Register 1 (DFITMG1_SHADOW)

9.3.3.1.124.1 Offset

Register	Offset
DFITMG1_SHADOW	2194h

9.3.3.1.124.2 Diagram



9.3.3.1.124.3 Fields

Field	Function
31-28 <code>dfi_t_cmd_lat</code>	Specifies the number of DFI PHY clock cycles between when the <code>dfi_cs</code> signal is asserted and when the associated command is driven. This field is used for CAL mode, should be set to '0' or the value which matches the CAL mode register setting in the DRAM. If the PHY can add the latency for CAL mode, this should be set to '0'. Valid Range: 0, 3, 4, 5, 6, and 8
27-26 —	Reserved
25-24 <code>dfi_t_parin_lat</code>	Specifies the number of DFI PHY clock cycles between when the <code>dfi_cs</code> signal is asserted and when the associated <code>dfi_parity_in</code> signal is driven.
23-21 —	Reserved
20-16 <code>dfi_t_wrdata_delay</code>	Specifies the number of DFI clock cycles between when the <code>dfi_wrdata_en</code> signal is asserted and when the corresponding write data transfer is completed on the DRAM bus. This corresponds to the DFI timing parameter <code>twrdata_delay</code> . Refer to PHY specification for correct value. For DFI 3.0 PHY, set to <code>twrdata_delay</code> , a new timing parameter introduced in DFI 3.0. For DFI 2.1 PHY, set to <code>tphy_wrdata</code> + (delay of DFI write data to the DRAM). Value to be programmed is in terms of DFI clocks, not PHY clocks. In <code>FREQ_RATIO=2</code> , divide PHY's value by 2 and round up to next integer. If using <code>DFITMG0.dfi_wrdata_use_sdr=1</code> , add 1 to the value. Unit: Clocks
15-13 —	Reserved

Table continues on the next page...

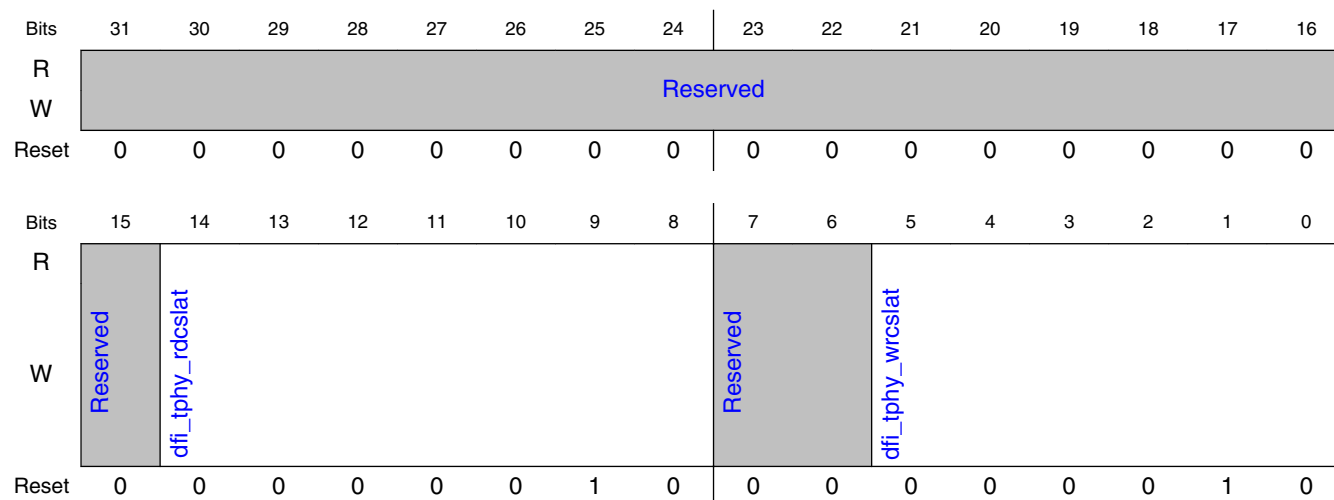
Field	Function
12-8 dfi_t_dram_clk_disable	Specifies the number of DFI clock cycles from the assertion of the dfi_dram_clk_disable signal on the DFI until the clock to the DRAM memory devices, at the PHY-DRAM boundary, maintains a low value. If the DFI clock and the memory clock are not phase aligned, this timing parameter should be rounded up to the next integer value.
7-5 —	Reserved
4-0 dfi_t_dram_clk_enable	Specifies the number of DFI clock cycles from the de-assertion of the dfi_dram_clk_disable signal on the DFI until the first valid rising edge of the clock to the DRAM memory devices, at the PHY-DRAM boundary. If the DFI clock and the memory clock are not phase aligned, this timing parameter should be rounded up to the next integer value.

9.3.3.1.125 [SHADOW] DFI Timing Register 2 (DFITMG2_SHADOW)

9.3.3.1.125.1 Offset

Register	Offset
DFITMG2_SHADOW	21B4h

9.3.3.1.125.2 Diagram



9.3.3.1.125.3 Fields

Field	Function
31-15 —	Reserved

Table continues on the next page...

DDR Controller (DDRC)

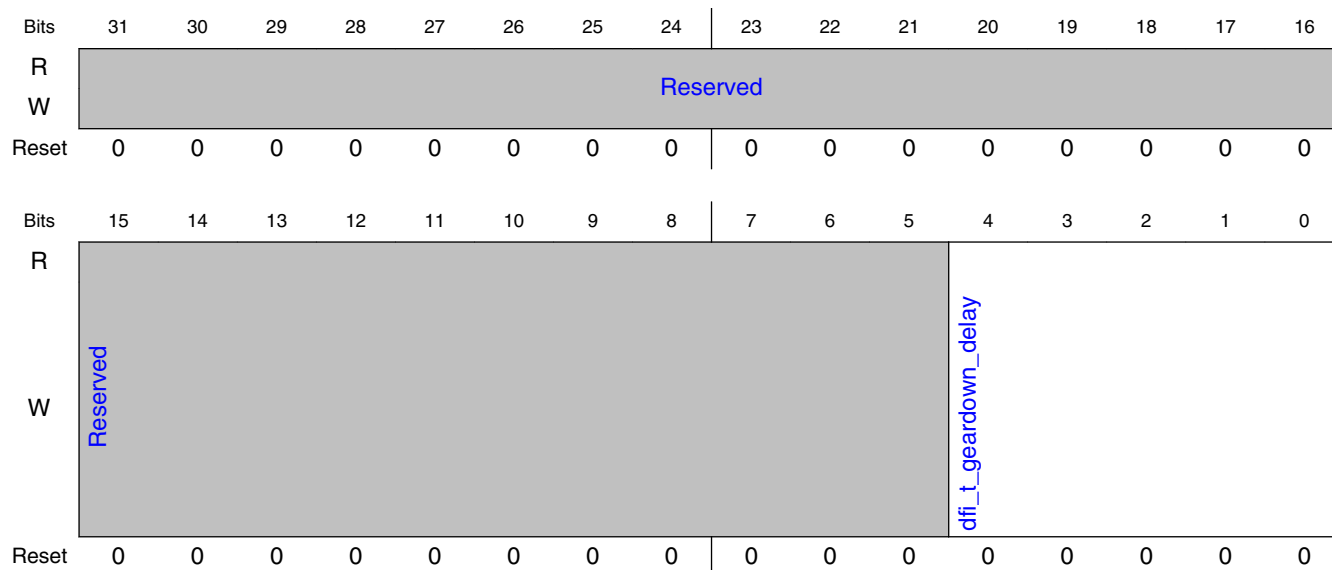
Field	Function
14-8 dfi_tphy_rdcslat	Number of DFI PHY clock cycles between when a read command is sent on the DFI control interface and when the associated dfi_rddata_cs signal is asserted. This corresponds to the DFI timing parameter tphy_rdcslat. Refer to PHY specification for correct value.
7-6 —	Reserved
5-0 dfi_tphy_wrcslat	Number of DFI PHY clock cycles between when a write command is sent on the DFI control interface and when the associated dfi_wrdata_cs signal is asserted. This corresponds to the DFI timing parameter tphy_wrcslat. Refer to PHY specification for correct value.

9.3.3.1.126 [SHADOW] DFI Timing Register 3 (DFITMG3_SHADOW)

9.3.3.1.126.1 Offset

Register	Offset
DFITMG3_SHADOW	21B8h

9.3.3.1.126.2 Diagram



9.3.3.1.126.3 Fields

Field	Function
31-5	Reserved

Table continues on the next page...

Field	Function
—	
4-0 dfi_t_geardown_delay	The delay from dfi_geardown_en assertion to the time of the PHY being ready to receive commands. Refer to PHY specification for correct value. When the controller is operating in 1:2 frequency ratio mode, program this to (tgeardown_delay/2) and round it up to the next integer value. Unit: Clocks

9.3.3.1.127 [SHADOW] ODT Configuration Register (ODTCFG_SHADOW)

9.3.3.1.127.1 Offset

Register	Offset
ODTCFG_SHADOW	2240h

9.3.3.1.127.2 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved				wr_odt_hold				Reserved				wr_odt_delay			
W																
Reset	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				rd_odt_hold				Reserved		rd_odt_delay				Reserved	
W																
Reset	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0

9.3.3.1.127.3 Fields

Field	Function
31-28 —	Reserved
27-24 wr_odt_hold	DFI PHY clock cycles to hold ODT for a write command. The minimum supported value is 2. Recommended values: DDR2: - BL8: 0x5 (DDR2-400/533/667), 0x6 (DDR2-800), 0x7 (DDR2-1066) - BL4: 0x3 (DDR2-400/533/667), 0x4 (DDR2-800), 0x5 (DDR2-1066) DDR3: - BL8: 0x6 DDR4: - BL8: 5 + WR_PREAMBLE + CRC_MODE WR_PREAMBLE = 1 (1tCK write preamble), 2 (2tCK write preamble) CRC_MODE = 0 (not CRC mode), 1 (CRC mode) LPDDR3: - BL8: 7 + RU(tODT _{on} (max)/tCK)
23-21	Reserved

Table continues on the next page...

DDR PHY (DDR_PHY)

Field	Function
—	
20-16 wr_odt_delay	The delay, in DFI PHY clock cycles, from issuing a write command to setting ODT values associated with that command. ODT setting must remain constant for the entire time that DQS is driven by the DDRC. Recommended values: DDR2: - CWL + AL - 3 (DDR2-400/533/667), CWL + AL - 4 (DDR2-800), CWL + AL - 5 (DDR2-1066) If (CWL + AL - 3 < 0), DDRC does not support ODT for write operation. DDR3: - 0x0 DDR4: - DFITMG1.dfi_t_cmd_lat (to adjust for CAL mode) LPDDR3: - WL - 1 - RU(tODTon(max)/tCK)
15-12 —	Reserved
11-8 rd_odt_hold	DFI PHY clock cycles to hold ODT for a read command. The minimum supported value is 2. Recommended values: DDR2: - BL8: 0x6 (not DDR2-1066), 0x7 (DDR2-1066) - BL4: 0x4 (not DDR2-1066), 0x5 (DDR2-1066) DDR3: - BL8 - 0x6 DDR4: - BL8: 5 + RD_PREAMBLE RD_PREAMBLE = 1 (1tCK write preamble), 2 (2tCK write preamble) LPDDR3: - BL8: 5 + RU(tDQSCK(max)/tCK) - RD(tDQSCK(min)/tCK) + RU(tODTon(max)/tCK)
7 —	Reserved
6-2 rd_odt_delay	The delay, in DFI PHY clock cycles, from issuing a read command to setting ODT values associated with that command. ODT setting must remain constant for the entire time that DQS is driven by the DDRC. Recommended values: DDR2: - CL + AL - 4 (not DDR2-1066), CL + AL - 5 (DDR2-1066) If (CL + AL - 4 < 0), DDRC does not support ODT for read operation. DDR3: - CL - CWL DDR4: - CL - CWL - RD_PREAMBLE + WR_PREAMBLE + DFITMG1.dfi_t_cmd_lat (to adjust for CAL mode) WR_PREAMBLE = 1 (1tCK write preamble), 2 (2tCK write preamble) RD_PREAMBLE = 1 (1tCK write preamble), 2 (2tCK write preamble) If (CL - CWL - RD_PREAMBLE + WR_PREAMBLE) < 0, DDRC does not support ODT for read operation. LPDDR3: - RL + RD(tDQSCK(min)/tCK) - 1 - RU(tODTon(max)/tCK)
1-0 —	Reserved

9.4 DDR PHY (DDR_PHY)

9.4.1 Overview

The Double Data Rate Physical Layer (DDR PHY), provides an interface between universal controllers and external DDR3/DDR3L/DDR4/LPDDR3/LPDDR4 SDRAM devices.

NOTE

The information within this block guide is Synopsys Proprietary. Used with permission.

The block diagram of the DDR PHY is shown below.

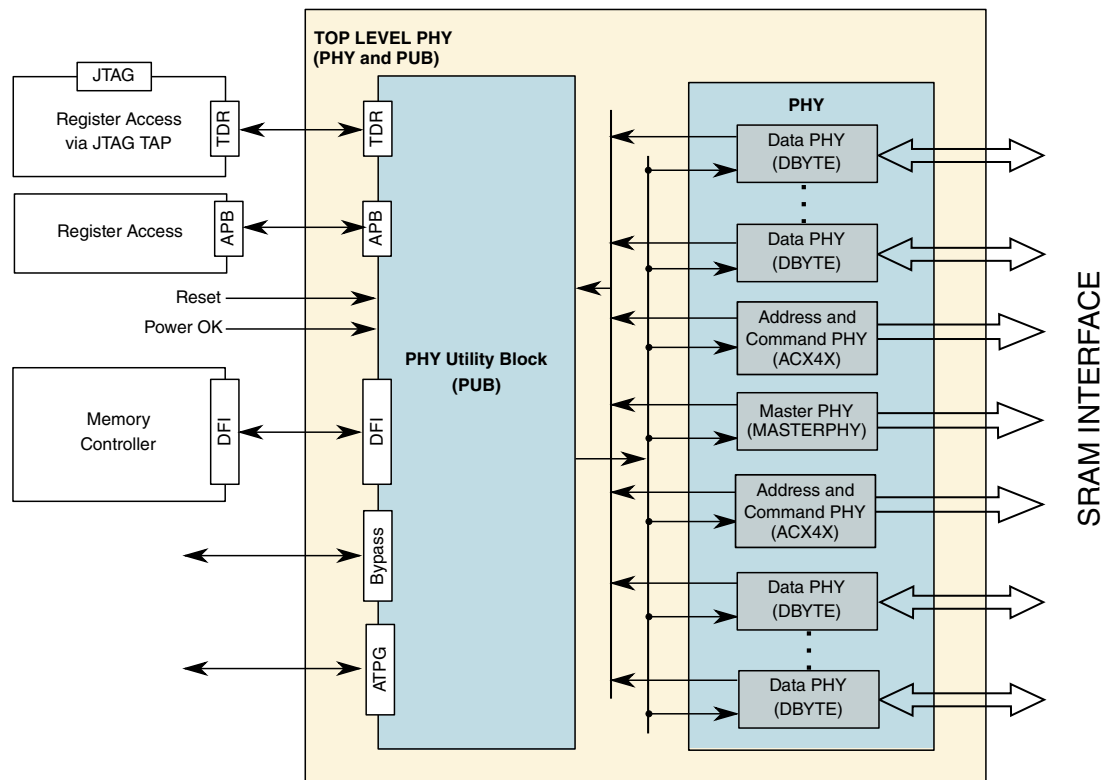


Figure 9-6. Block Diagram

9.4.1.1 Features

The following features are supported in the DDR PHY:

- LPDDR4, LPDDR3, DDR4, DDR3, and DDR3L operation
- Multiple memory rank support:
 - Four ranks supported for DDR4, DDR3, and DDR3L
 - Two ranks supported for LPDDR4 and LPDDR3
- DFI interface (DFI v4.0 Addendum 2, March 2015)
- At-speed loopback testing on both the address and data channels
- Mux-scan ATPG
- High-speed PLL used to generate the high speed clock for data transmit and high speed digital pipelines
- PHY Utility Block (PUB), which provides configuration and control
- Power on reset (POR) circuit, DRAM reset, DRAM alert and PHY debug
- A global voltage reference generation and observation circuit
- Impedance calibration circuits

9.4.2 Functional Description

The DDR PHY is composed of the following:

- DBYTE - The eight-lane data IO unit
- ACX4 - The four-lane address/command hard IP unit
- MASTERPHY - Contains the PLL, the voltage reference, thermal calibration, and reset
- PUB - PHY Utility Block

9.4.2.1 DDR Address/Command PHY (ACX4)

Info to be included at a future date.

The DDR SDRAM address/command PHY (DDRPHYACX4) provides an address and command interface to the external SDRAM memories. A memory interface typically contains a single address/command PHY and one or more data PHYs (one for each byte lane). Figure 2-1 shows the organization of the AC component.

The AC consists of multiple design slices. The address/command (AC) slice is used for all address and command and clk/cs/odt signals. Each slice contains all logic required for both the output (address/command) and input (loopback) paths of the respective signals.

All clocking inside the AC is derived from an input clock, which is generated by the PLL in the MASTERPHY. The PLL generates the higher frequency clocks that are used inside the macro and minimizes input clock jitter.

9.4.2.1.1 ACX4 Transmit Lanes

The ACX4 component contains 4 transmit only lanes which are grouped as follows:

- Clock group: {CK_t, CK_c}
- CA group: {ras_n, cas_n, we_n, addr...}
- CKE,CS,ODT group{CKE, CS_n, ODT...}

In DDR4 systems, 12 ACs are required to control one 72b/64b channel with four independent ranks.

In LPDDR3 and LPDDR4 systems, the 12 ACs are broken up into two independently controllable groups, enabling up to two independent 32b channels (each with a maximum of two ranks).

9.4.2.1.2 ACX4 Command Path

Each SDRAM address and command signal is driven from the memory controller to the SDRAM through an address/command slice. Controlled address/command pipeline logic inside the AC slice converts the DFI-rate signals from the controller to single data rate (SDR) Address/Command signals going to the SDRAM. The last launching register inside the address/command slice is clocked by a delay-shifted DDR clock. For SDR operation, as required by DDR3 DRAMs, all outputs remain constants for two cycles at a time.

The path for the SDRAM clock (CK) is through a standard AC slice which is configured to act as a dedicated clock (CK) slice. This shifted address/command output makes the CK centered into the data eye of the address/command signals.

All AC outputs going to the SDRAM have a LCDL, which can be used to align or shift the outputs of the data PHY. This delay element can also be used to provide delay adjustment during functional or loopback modes.

9.4.2.1.3 ACX4 Loopback/DFT Paths

For every lane there will be a loopback-only receive path. The loopback path is stimulated and checked by utilizing a FirmWare image.

9.4.2.2 DDR Data PHY (DBYTE)

The DDR SDRAM Data PHY (DBYTE) provides data interface to external SDRAM memory. Features of the DBYTE include:

- High speed digital logic pipeline for transmit datapaths to the SDRAM
- High speed digital logic pipeline for receive datapaths from the SDRAM
- Drivers and receivers for the DQ and DQS signals as well as all the necessary high voltage analog macros to interface off chip
- I/O component that includes PVT-compensated on-die termination (ODT) and output impedance
- Loop-back test mode and PRBS checkers to help verify the integrity of the internal datapath of all DBYTE I/O lanes

9.4.2.2.1 DBYTE Write Path

Each SDRAM data is driven from the PUB to the SDRAM through a data (DQ) slice. Controlled data pipeline logic inside the DBYTE slice converts the DFI-rate signals from the PUB to double-data rate (DDR) signals going to the SDRAM. The last launching register inside the data slice is clocked by the DDR clock.

The path for the SDRAM write strobe is through a dedicated data strobe (DQS) slice. The output register of the DQS slice is clocked by a delayed version of PclkIn. PclkIn drives the output register in the DQ slice is shifted using the DLL in training mode. This centers the write data strobe signal in the write data eye.

All DBYTE outputs going to the SDRAM, including write data, write data strobe read DQS gate, driver power down, receiver power down, on-die termination enable, and output enable have a DLL that is used to deskew the outputs in order to maximize the write data eye.

Apart from the DQS pipeline, the DQS slice also contains a similar pipeline for output enables, on-die termination enable, driver power down, and receiver power down. This matches the I/O control signals to the corresponding data and data strobe.

9.4.2.2.2 DBYTE Read and Loopback Path

Read data from the SDRAM is written into a four-deep asynchronous FIFO that is implemented inside the DQ slice.

NOTE

The FIFO is 16 deep in terms of the DRAM DQS writing into the FIFO; it is 8 deep on the core side when a half-speed DFICLK is employed and it is 8 deep on the core side when a full speed DFICLK is employed.

The FIFO is written using the DQS clock and read using the Controller clock. A per-bit delay line at the input of the DQ slice is used to optionally de-skew read data with respect to the read strobe in order to maximize the read data eye.

The read data strobe path and a read DQS gate path is implemented inside the DQS slice. A delay line at the input of the slice shifts the read DQS by nominal 90 degrees (with the actual value arrived at through training) so that the strobe is centered into the read data eye before clocking the data into the FIFO. A Bit- Wise Delay Line (BDL) in the read DQS path is used to match a corresponding bit de-skew BDL in the DQ slice.

The DQS slice also generates the write pointers for the data FIFO. These pointers, together with the strobe, are then distributed to all DQ slices. The PUB generates a trained read-valid signal to read the data from the PHY.

The DBI uses DQS Slice 1, configured to behave as a DQ slice in x8 mode only.

9.4.2.2.2.1 DBYTE Loopback/DFT Paths

High speed loopback is implemented on all DBYTE I/Os. The DBYTE read path also doubles as the loopback path. During loopback, the PUB enables the gating of the DQS signal in the same way as it would normally gate the DQS during reads. This allows the looped back data to be clocked into the read data FIFO using the looped back DQS signal.

9.4.2.2.2.2 Memory Controller based DBYTE Loopback

This high speed PHY loopback mode uses all the DBYTE control and data paths in mission mode like operation modes. In this mode, the memory controller issues simultaneous Read/Write commands to the PHY, resulting in transmitting and receiving the same data on each DQ pad.

9.4.2.3 Master PHY (MASTERPHY)

The Master block contains the PLL, the Primary Voltage Reference, thermal calibration, reset, and I/O cells for test and control.

9.4.2.3.1 PLL

The PLL in the MASTER is a simplified PLL that has DFICLK as an input and creates an internal clock by quadrupling the DFICLK frequency. The PLL in the MASTER can only multiply the input clock frequency to produce an output clock that is four times the frequency of the input clock. The internal clock is the output of the MASTER to the ACX4 and DBYTE.

9.4.2.3.1.1 Optimal PLL Settings for Mission Mode Frequency Bins

Table 9-4. Optimal PLL Settings for Mission Mode Frequency Bins (PLL Max Bandwidth = 2 MHz)

pllin min (MHz)	pllin max (MHz)	Input div. ratio	Fbk min (MHz)	Fbk max (MHz)	Fbk div. ratio	VCO min (MHz)	VCO max (MHz)	PIICtrl2 PIIFreqSel[4:0]	PLLTestMode int_half[8] prop_half[5] vco_mode[2] overflow[1]	PIICtrl1 PIICpPropCtrl[8:5]	PIICtrl1 PIICpIntCtrl[4:0]	PIICtrl4 PIICpPropGsCtrl[8:5]	PIICtrl4 PIICpIntGsCtrl[4:0]
937.5	1067	8	117.2	133.4	4	3750	4268	11000'b	1011'b	0011'b	00000'b	1011'b	11111'b
625	937.5	8	78.13	117.2	4	2500	3750	11001'b	1011'b	0100'b	00000'b	1011'b	11111'b
468.75	625	4	117.2	156.3	8	3750	5000	01010'b	1011'b	0011'b	00000'b	1011'b	11111'b
312.5	468.75	4	78.13	117.2	8	2500	3750	01011'b	1011'b	0100'b	00000'b	1011'b	11111'b
234.4	312.5	2	117.2	156.3	16	3750	5000	00110'b	1011'b	0011'b	00000'b	1011'b	11111'b
166	234.4	2	83	117.2	16	2656	3750	00111'b	1011'b	0100'b	00000'b	1011'b	11111'b

9.4.2.3.2 Voltage Reference

This primary voltage reference generates the VREF global out used by the receivers in the ACX4 and DBYTE.

9.4.2.3.3 Calibration

The MASTER contains the analog blocks to perform output drive and input termination impedance calibration. Through the use of a digital control state machine, located in the PUB, the full mixed-signal calibration loop matches the impedance of the driver and termination with that of an external precision resistor. The resulting digital calibration code is fed to the address/command, DQ, and DQS blocks to control their impedance.

9.4.2.3.4 Reset

Reset takes the asynchronous reset bump, the PwrOk bump, and dfi_reset_n signal and combines them and synchronizes to the DFI clock. This is used by the ACX4 and DBYTE.

9.4.2.4 PHY Utility Block (PUB)

The following is an overview of the different functionality of the PHY Utility Block (PUB):

- Registers required to configure, control, train and self-test the PHY. Also contains registers required to initialize the DRAM.
- Initialization - this includes locking of the PLL, delay line calibration and calibration of I/O impedances.
- Data Training- train the PHY for optimum timing margins. This includes finding the best positioning of the DQS gating window during reads, write leveling, data bit deskew, and optimizing the read and write data eye.
- Implements a DFI-compliant interface between the external controller and the PHY.

9.4.2.4.1 DDR PHY Control and Configuration

The DDR PHY's PUB registers provide the required configuration, control, training and self-testing of the PHY. Also contains registers required to initialize the DRAM.

Refer to the [DDR PHY register descriptions](#) for more information.

9.4.2.4.2 Initialization and Training Sequence

The following outlines the general initialization and training sequence. Unless otherwise stated, these functions occur within the DDR PHY.

1. Bring up VDD, VDDQ, and VAA. This occurs outside of the DDR PHY.
2. Start the clocks and reset the PHY. This also occurs outside of the PHY.
3. Initialize PHY configuration.
4. Load the PHY training firmware into instruction memory SRAM.
5. Set the PHY input clocks to the desired frequency. This occurs outside of the PHY.

6. Write the Message Block parameters for the training firmware. There are several locations in the data memory that must be written for the training firmware to have these parameters.
7. Execute the training firmware.
8. Once the training firmware completes, the results are returned to the data memory in the message block structure. This message block can be read. The result may be that another frequency needs to be trained (return to step 5).
9. Load the PHY Initialization Engine registers with the provided initialization sequence.
10. Initialize the PHY to mission mode by performing a DFI initialization sequence per the DFI specification.
11. The PHY is now in mission mode.

9.4.2.4.3 DFI Interface

The DDR PHY and the DDR Controller both require a standard interface to transfer address, data, and control between these two blocks. These interfaces are DFI compliant. The PUB contains the DFI interface for the PHY.

For more information regarding the DFI specification, refer to the following:

- DDR PHY Interface (DFI) Specification, Revision 3.1 (Preliminary), 21, March 2014
- DDR PHY Interface (DFI) Specification Preliminary DFI 4.0 Specification - Addendum to DFI 3.1, 2 April 2014

The DFI interface is sub-divided into the following interface groups:

- Control Interface
- Write Data Interface
- Read Data Interface
- Update Interface
- Status Interface
- Training Interface (PHY-independent mode supported by the DDRC)
- Low Power Control Interface

9.4.3 Memory Map and Register Definition

This section includes the DDRC PHY module memory map and detailed descriptions of all registers.

NOTE

Synopsys Proprietary. Used with permission.

9.4.3.1 DWC_DDRPHYA_ANIB register descriptions

9.4.3.1.1 DWC_DDRPHYA_ANIB Memory map

DDR4/3 PHY address block

DWC_DDRPHYA_ANIB0 base address: 0h

DWC_DDRPHYA_ANIB1 base address: 1000h

DWC_DDRPHYA_ANIB2 base address: 2000h

DWC_DDRPHYA_ANIB3 base address: 3000h

DWC_DDRPHYA_ANIB4 base address: 4000h

DWC_DDRPHYA_ANIB5 base address: 5000h

DWC_DDRPHYA_ANIB6 base address: 6000h

DWC_DDRPHYA_ANIB7 base address: 7000h

DWC_DDRPHYA_ANIB8 base address: 8000h

DWC_DDRPHYA_ANIB9 base address: 9000h

Offset	Register	Width (In bits)	Access	Reset value
34h	Digital Observation Pin control (MtestMuxSel)	16	RW	0000h
50h	Force Address/Command Tristate (Lanes A3-A0) (AForceTriCont)	16	RW	0000h
86h	Address TX impedance controls (ATxImpedance)	16	RW	03FFh
A6h	Address Loopback PRBS Error status for an entire ACX4 block (ATestPrbsErr)	16	RO	Table 9-4
AAh	Address TX slew rate and predriver controls (ATxSlewRate)	16	RW	07FFh
ACh	Address Loopback Test Result register (ATestPrbsErrCnt)	16	RO	Table 9-4
100h	Address/Command Delay, per pstate. (ATxDly_p0)	16	RW	0000h
20_0100h	Address/Command Delay, per pstate. (ATxDly_p1)	16	RW	0000h
40_0100h	Address/Command Delay, per pstate. (ATxDly_p2)	16	RW	0000h
60_0100h	Address/Command Delay, per pstate. (ATxDly_p3)	16	RW	0000h

9.4.3.1.2 Digital Observation Pin control (MtestMuxSel)

9.4.3.1.2.1 Offset

Register	Offset
MtestMuxSel	34h

9.4.3.1.2.2 Diagram



9.4.3.1.2.3 Fields

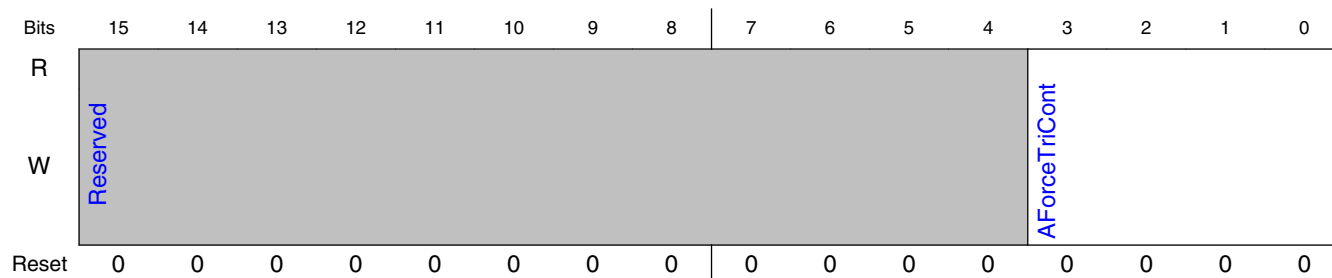
Field	Function
15-6 —	Reserved
5-0 MtestMuxSel	Controls for the 64-1 mux for asynchronous data to the Digital Observation Pin. Controls for the 64-1 mux for asynchronous data to the Digital Observation Pin. Encoding 6'h0 --> Drive 0 from this chiplet (allows flat 'OR' of pass-through information) Encoding 6'h01:1f --> Select local data from AC/DBYTE/MASTER macro Encoding 6'h20 --> Reserved (Drive 0 from macro into mux; Drive 0 from PUB synth logic path into mux) Encoding 6'h21:3f --> Select local data from PUB AC/DBYTE/MASTER synthesized logic Note: See PUB documentation for how, or if, the Digital Observation Pin is mapped to a physical bump in this configuration.

9.4.3.1.3 Force Address/Command Tristate (Lanes A3-A0) (AForceTriCont)

9.4.3.1.3.1 Offset

Register	Offset
AForceTriCont	50h

9.4.3.1.3.2 Diagram



9.4.3.1.3.3 Fields

Field	Function
15-4 —	Reserved
3-0 AForceTriCont	Force tristate control, per-lane, of the ACX4 instance controlled by this register Setting this register will cause the PHY to tristate the target lane when dfi_init_complete==1 Bit [0] = controls lane 0 of the target ACX4 block Bit [1] = controls lane 1 of the target ACX4 block Bit [2] = controls lane 2 of the target ACX4 block Bit [3] = controls lane 3 of the target ACX4 block

9.4.3.1.4 Address TX impedance controls (ATxImpedance)

9.4.3.1.4.1 Offset

Register	Offset
ATxImpedance	86h

9.4.3.1.4.2 Diagram



9.4.3.1.4.3 Fields

Field	Function
15-10	Reserved

Table continues on the next page...

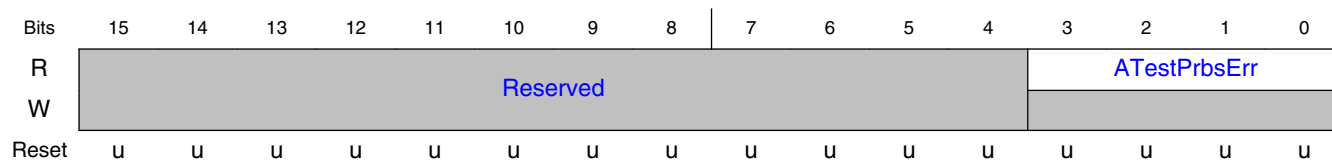
Field	Function
—	
9-5 ADrvStrenN	5 bit bus used to select the target pull down output impedance. 5 bit bus used to select the target pull down output impedance. Please Refer to Technology specific PHY DATABOOK for supported options Connects to the DrvStren pins of the driver thus: Csr_DrvStren120N[5:0] = csrADrvStrenN[4:0],1'b1 00000 = 120.0 Ohm 00001 = 60.0 Ohm 00011 = 40.0 Ohm 00111 = 30.0 Ohm 01111 = 24.0 Ohm 11111 = 20.0 Ohm
4-0 ADrvStrenP	5 bit bus used to select the target pull up output impedance. 5 bit bus used to select the target pull up output impedance. Please Refer to Technology specific PHY DATABOOK for supported options Connects to the DrvStren pins of the driver thus: Csr_DrvStren120P[5:0] = csrADrvStrenP[4:0],1'b1 00000 = 120.0 Ohm 00001 = 60.0 Ohm 00011 = 40.0 Ohm 00111 = 30.0 Ohm 01111 = 24.0 Ohm 11111 = 20.0 Ohm

9.4.3.1.5 Address Loopback PRBS Error status for an entire ACX4 block (ATestPrbsErr)

9.4.3.1.5.1 Offset

Register	Offset
ATestPrbsErr	A6h

9.4.3.1.5.2 Diagram



9.4.3.1.5.3 Fields

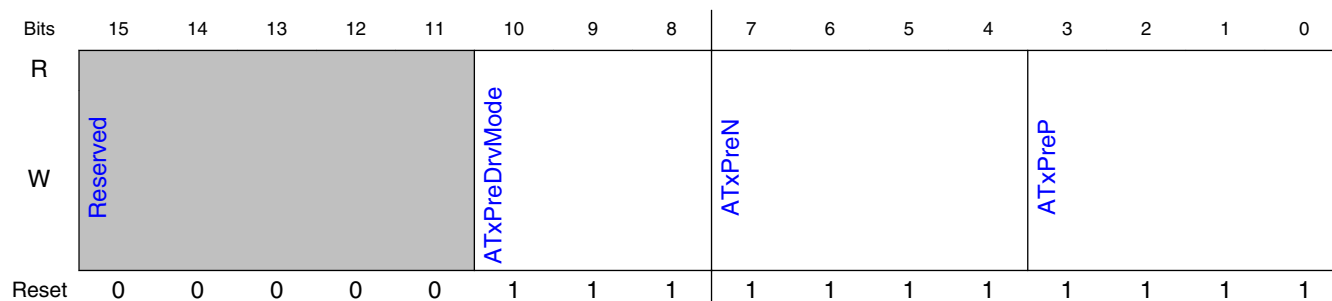
Field	Function
15-4 —	Reserved
3-0 ATestPrbsErr	Overall error indicator for each prbs bump checker. Overall error indicator for each prbs bump checker. Bit[0] = Lane0 Error Status 1 = errors found, 0 = no errors Bit[1] = Lane1 Error Status Bit[2] = Lane2 Error Status Bit[3] = Lane3 Error Status

9.4.3.1.6 Address TX slew rate and predriver controls (ATxSlewRate)

9.4.3.1.6.1 Offset

Register	Offset
ATxSlewRate	AAh

9.4.3.1.6.2 Diagram



9.4.3.1.6.3 Fields

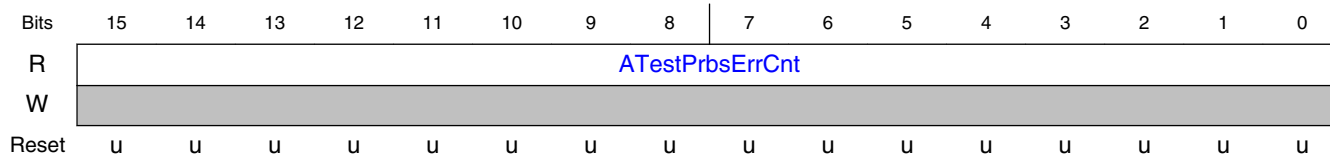
Field	Function
15-11 —	Reserved
10-8 ATxPreDrvMode	Controls predrivers to adjust timing of turn-on and turn-off of pull-up and pull-down segments.
7-4 ATxPreN	4 bit binary trim for the driver pull down slew rate. 4 bit binary trim for the driver pull down slew rate. 4'b0000 has a slower slew rate than 4'b1111
3-0 ATxPreP	4 bit binary trim for the driver pull up slew rate. 4 bit binary trim for the driver pull up slew rate. 4'b0000 has a slower slew rate than 4'b1111

9.4.3.1.7 Address Loopback Test Result register (ATestPrbsErrCnt)

9.4.3.1.7.1 Offset

Register	Offset
ATestPrbsErrCnt	ACH

9.4.3.1.7.2 Diagram



9.4.3.1.7.3 Fields

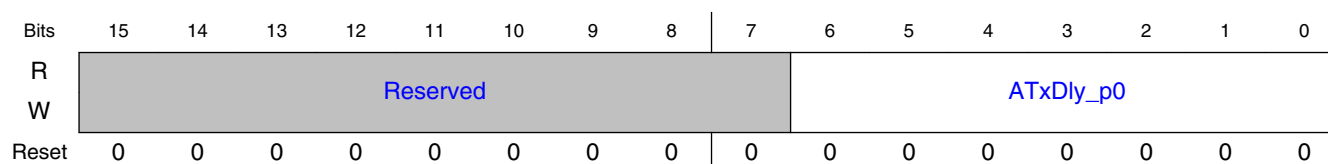
Field	Function
15-0 ATestPrbsErrCnt	Overall error indicator for each prbs bump checker. Overall error indicator for each prbs bump checker. Bit [3:0] = Lane0 Error Count Bit [7:4] = Lane1 Error Count Bit [11:8] = Lane2 Error Count Bit [15:12] = Lane3 Error Count

9.4.3.1.8 Address/Command Delay, per pstate. (ATxDly_p0)

9.4.3.1.8.1 Offset

Register	Offset
ATxDly_p0	100h

9.4.3.1.8.2 Diagram



9.4.3.1.8.3 Fields

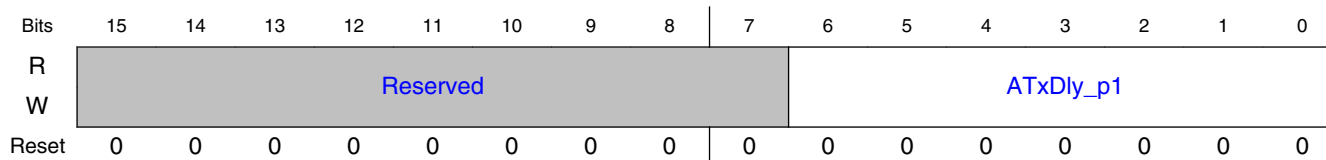
Field	Function
15-7 —	Reserved
6-0 ATxDly_p0	Trained for LPDDR3/4 to generate timed address and command signals to the DRAMs, per ACX4. Trained for LPDDR3/4 to generate timed address and command signals to the DRAMs, per ACX4. For DDR3/DDR4, it is recommended that address and command ACX4 have it ATxDly=0x00 The four signals generated by an ACX4 will have common timing. ATxDly[6] is the coarse delay, ie one unit of delay is 1 UI. ATxDly[5] is reserved. ATxDly[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. The target ACX4 used for memory clock generation, must have its ATxDly configured to 0x00.

9.4.3.1.9 Address/Command Delay, per pstate. (ATxDly_p1)

9.4.3.1.9.1 Offset

Register	Offset
ATxDly_p1	20_0100h

9.4.3.1.9.2 Diagram



9.4.3.1.9.3 Fields

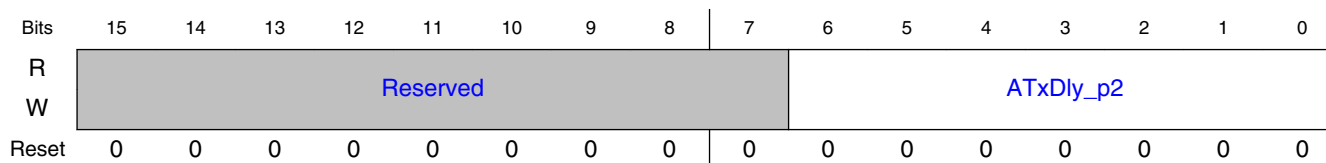
Field	Function
15-7 —	Reserved
6-0 ATxDly_p1	Trained for LPDDR3/4 to generate timed address and command signals to the DRAMs, per ACX4. Trained for LPDDR3/4 to generate timed address and command signals to the DRAMs, per ACX4. For DDR3/DDR4, it is recommended that address and command ACX4 have it ATxDly=0x00 The four signals generated by an ACX4 will have common timing. ATxDly[6] is the coarse delay, ie one unit of delay is 1 UI. ATxDly[5] is reserved. ATxDly[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. The target ACX4 used for memory clock generation, must have its ATxDly configured to 0x00.

9.4.3.1.10 Address/Command Delay, per pstate. (ATxDly_p2)

9.4.3.1.10.1 Offset

Register	Offset
ATxDly_p2	40_0100h

9.4.3.1.10.2 Diagram



9.4.3.1.10.3 Fields

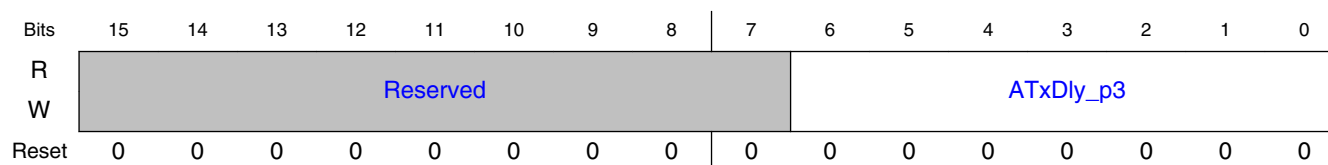
Field	Function
15-7 —	Reserved
6-0 ATxDly_p2	Trained for LPDDR3/4 to generate timed address and command signals to the DRAMs, per ACX4. Trained for LPDDR3/4 to generate timed address and command signals to the DRAMs, per ACX4. For DDR3/DDR4, it is recommended that address and command ACX4 have it ATxDly=0x00 The four signals generated by an ACX4 will have common timing. ATxDly[6] is the coarse delay, ie one unit of delay is 1 UI. ATxDly[5] is reserved. ATxDly[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. The target ACX4 used for memory clock generation, must have its ATxDly configured to 0x00.

9.4.3.1.11 Address/Command Delay, per pstate. (ATxDly_p3)

9.4.3.1.11.1 Offset

Register	Offset
ATxDly_p3	60_0100h

9.4.3.1.11.2 Diagram



9.4.3.1.11.3 Fields

Field	Function
15-7 —	Reserved
6-0 ATxDly_p3	Trained for LPDDR3/4 to generate timed address and command signals to the DRAMs, per ACX4. Trained for LPDDR3/4 to generate timed address and command signals to the DRAMs, per ACX4. For DDR3/DDR4, it is recommended that address and command ACX4 have it ATxDly=0x00 The four signals generated by an ACX4 will have common timing.

Field	Function
	ATxDly[6] is the coarse delay, ie one unit of delay is 1 UI. ATxDly[5] is reserved. ATxDly[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. The target ACX4 used for memory clock generation, must have its ATxDly configured to 0x00.

9.4.3.2 DWC_DDRPHYA_APBONLY register descriptions

9.4.3.2.1 DWC_DDRPHYA_APBONLY Memory map

DDR4/3 PHY address block

DWC_DDRPHYA_APBONLY0 base address: D_0000h

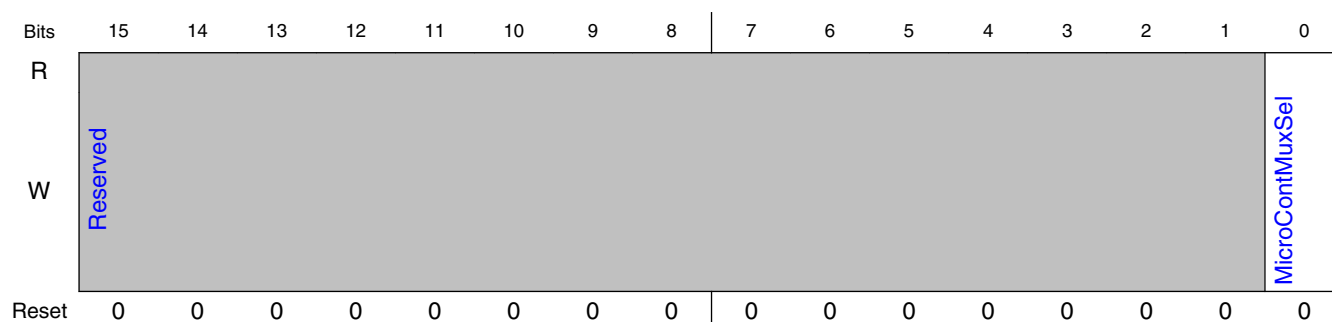
Offset	Register	Width (In bits)	Access	Reset value
0h	PMU Config Mux Select (MicroContMuxSel)	16	RW	0000h
8h	PMU/Controller Protocol - Controller Read-only Shadow (UctShadowRegs)	16	RO	Table 9-4
60h	Reserved for future use. (DctWriteOnly)	16	RW	0000h
62h	DCT downstream mailbox protocol CSR. (DctWriteProt)	16	RW	0001h
64h	Read-only view of the csr UctDatWriteOnly (UctWriteOnlyShadow)	16	RO	Table 9-4
68h	Read-only view of the csr UctDatWriteOnly (UctDatWriteOnlyShadow)	16	RO	Table 9-4
6Eh	Number of DfiClk ticks required for valid csr Rd Data. (DfiCfgRdDataValidTicks)	16	RW	0006h
132h	Controls reset and clock shutdown on the local microcontroller (MicroReset)	16	RW	0001h
1F4h	dfi_init_complete - Controller Read-only Shadow (DfiInitCompleteShadow)	16	RO	Table 9-4

9.4.3.2.2 PMU Config Mux Select (MicroContMuxSel)

9.4.3.2.2.1 Offset

Register	Offset
MicroContMuxSel	0h

9.4.3.2.2.2 Diagram



9.4.3.2.2.3 Fields

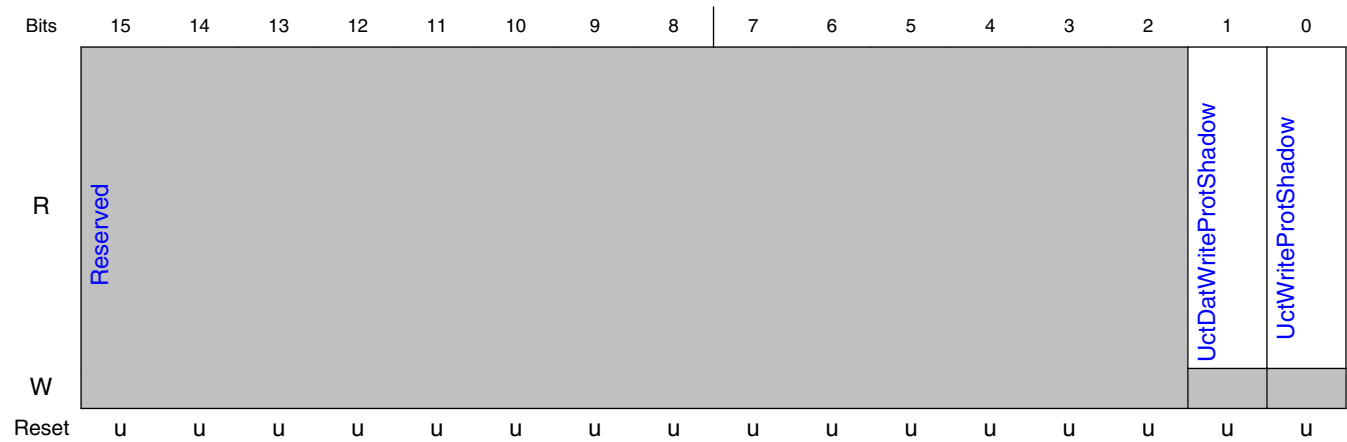
Field	Function
15-1 —	Reserved
0 MicroContMuxSel	This register controls access to the PHY configuration registers. This register controls access to the PHY configuration registers. 1 = MicroController/PHY Init Engine has control of csr bus. 0 = MicroController/PHY Init Engine csr requests are ignored.

9.4.3.2.3 PMU/Controller Protocol - Controller Read-only Shadow (UctShadowRegs)

9.4.3.2.3.1 Offset

Register	Offset
UctShadowRegs	8h

9.4.3.2.3.2 Diagram



9.4.3.2.3.3 Fields

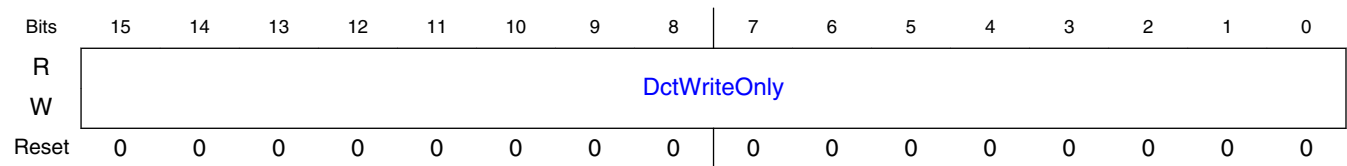
Field	Function
15-2	Reserved
1	Reserved for future use.
UctDatWriteProtShadow	
0	When set to 0, the PMU has a message for the user
UctWriteProtShadow	

9.4.3.2.4 Reserved for future use. (DctWriteOnly)

9.4.3.2.4.1 Offset

Register	Offset
DctWriteOnly	60h

9.4.3.2.4.2 Diagram



9.4.3.2.4.3 Fields

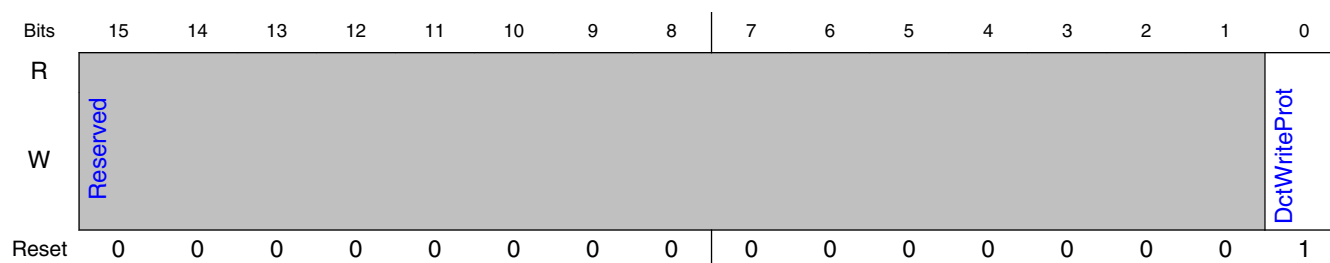
Field	Function
15-0 DctWriteOnly	Reserved for future use.

9.4.3.2.5 DCT downstream mailbox protocol CSR. (DctWriteProt)

9.4.3.2.5.1 Offset

Register	Offset
DctWriteProt	62h

9.4.3.2.5.2 Diagram



9.4.3.2.5.3 Fields

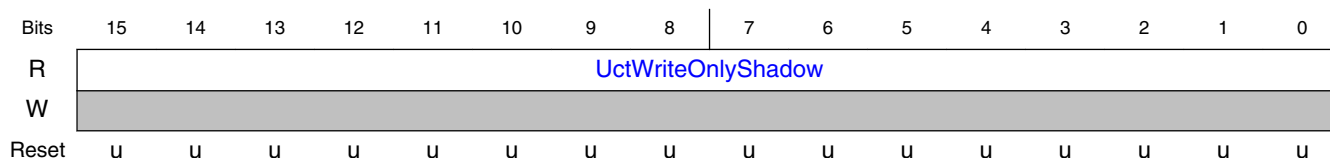
Field	Function
15-1 —	Reserved
0 DctWriteProt	By setting this register to 0, the user acknowledges the receipt of the message.

9.4.3.2.6 Read-only view of the csr UctDatWriteOnly (UctWriteOnlyShadow)

9.4.3.2.6.1 Offset

Register	Offset
UctWriteOnlyShadow	64h

9.4.3.2.6.2 Diagram



9.4.3.2.6.3 Fields

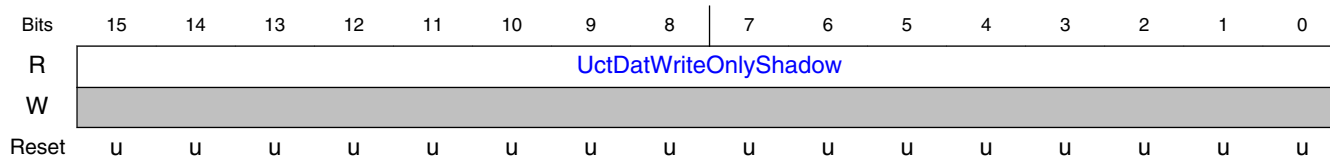
Field	Function
15-0	Used to pass the message ID for major messages.
UctWriteOnlyShadow	Used to pass the message ID for major messages. Also used to pass the lower 16 bits for streaming messages.

9.4.3.2.7 Read-only view of the csr UctDatWriteOnly (UctDatWriteOnlyShadow)

9.4.3.2.7.1 Offset

Register	Offset
UctDatWriteOnlyShadow	68h

9.4.3.2.7.2 Diagram



9.4.3.2.7.3 Fields

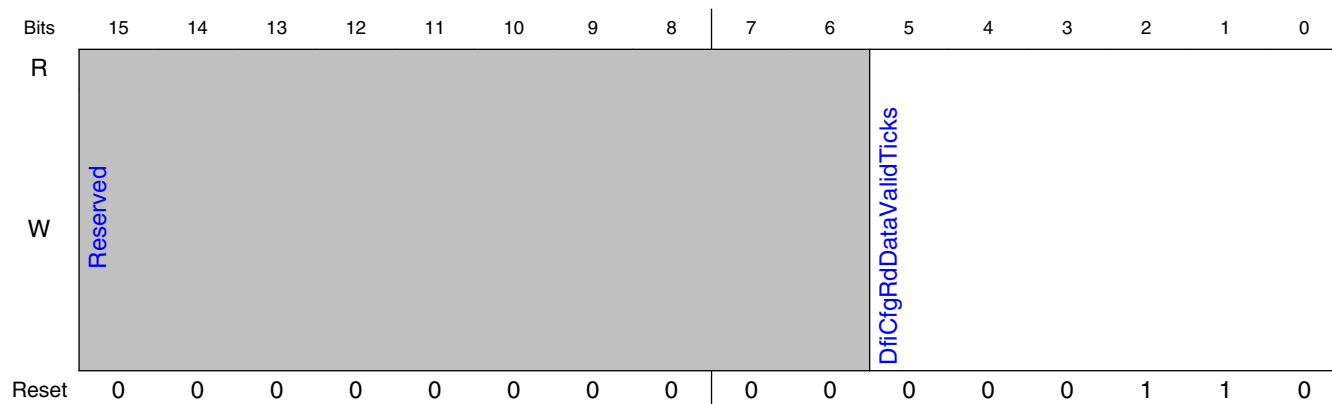
Field	Function
15-0	Used to pass the upper 16 bits for streaming messages.
UctDatWriteOnly Shadow	Used to pass the upper 16 bits for streaming messages. Not used in passing major messages.

9.4.3.2.8 Number of DfiClk ticks required for valid csr Rd Data. (DfiCfgRd DataValidTicks)

9.4.3.2.8.1 Offset

Register	Offset
DfiCfgRdDataValidTicks	6Eh

9.4.3.2.8.2 Diagram



9.4.3.2.8.3 Fields

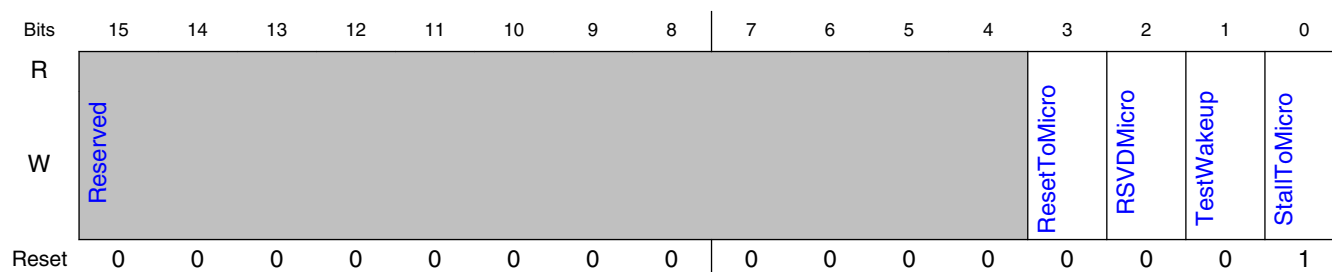
Field	Function
15-6	Reserved
—	
5-0	Roundtrip delay of a register read access.
DfiCfgRdDataValidTicks	Roundtrip delay of a register read access. This value must not be changed from its reset value.

9.4.3.2.9 Controls reset and clock shutdown on the local microcontroller (MicroReset)

9.4.3.2.9.1 Offset

Register	Offset
MicroReset	132h

9.4.3.2.9.2 Diagram



9.4.3.2.9.3 Fields

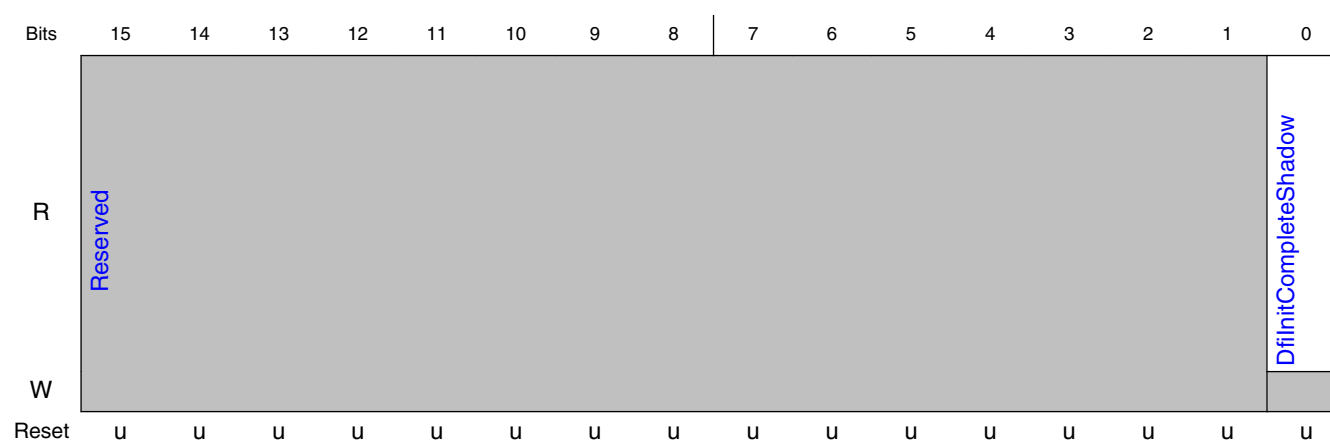
Field	Function
15-4 —	Reserved
3 ResetToMicro	Set this bit to apply synchronous reset to the microcontroller. Set this bit to apply synchronous reset to the microcontroller. Clear this bit to release reset to the microcontroller. This reset initializes the program counter to begin execution from the boot vector. This reset also clears the interrupt sticky bits.
2 RSVDMicro	RSVD
1 TestWakeup	Reserved Reserved. Must always be set to 0.
0 StallToMicro	Set this bit to stall the microcontroller by hardware. Set this bit to stall the microcontroller by hardware. Clear this bit to allow the microcontroller to continue executing from its current program counter location. Typically, this bit is used at power up to hold the program counter at the boot vector while BIOS loads the microcontroller program code. While stalled, the microcontroller clocks are gated off for power reduction.

9.4.3.2.10 dfi_init_complete - Controller Read-only Shadow (DfilnitCompleteShadow)

9.4.3.2.10.1 Offset

Register	Offset
DfilnitCompleteShadow	1F4h

9.4.3.2.10.2 Diagram



9.4.3.2.10.3 Fields

Field	Function
15-1 —	Reserved
0 DfilnitCompleteShadow	This csr presents a read-only view (a shadow) of the Register DfilnitComplete which is used by the sequencer to control the state of dfi_init_complete. This csr presents a read-only view (a shadow) of the Register DfilnitComplete which is used by the sequencer to control the state of dfi_init_complete. The value in this Register is not affected by the BlockSeq0BAck field of the SequencerOverride register. While the Register MicroContMuxSel is set, access to this Shadow register will not steal config bus bandwidth from the micro controller. That is polling won't have a performance penalty.

9.4.3.3 DWC_DDRPHYA_DBYTE register descriptions

9.4.3.3.1 DWC_DDRPHYA_DBYTE Memory map

DDR4/3 PHY address block

DWC_DDRPHYA_DBYTE0 base address: 1_0000h

DWC_DDRPHYA_DBYTE1 base address: 1_1000h

DWC_DDRPHYA_DBYTE2 base address: 1_2000h

DWC_DDRPHYA_DBYTE3 base address: 1_3000h

Offset	Register	Width (In bits)	Access	Reset value
0h	DBYTE Module Disable (DbyteMiscMode)	16	RW	Table 9-4
34h	Digital Observation Pin control (MtestMuxSel)	16	RW	0000h
40h	DFI MaxReadLatency (DFIMRL_p0)	16	RW	0006h
60h - 1060h	VrefDAC1 control for DQ Receiver (used only when DFE is enabled in DDR4) (VrefDAC1_r0 - VrefDAC1_r8)	16	RW	0000h
80h - 1080h	VrefDAC0 control for DQ Receiver (VrefDAC0_r0 - VrefDAC0_r8)	16	RW	0000h
82h - 60_0282h	Data TX impedance controls (TxImpedanceCtrl0_b0_p0 - TxImpedanceCtrl0_b1_p3)	16	RW	0FFFh
86h - 60_0286h	Dq/Dqs receiver control (DqDqsRcvCntrl_b0_p0 - DqDqsRcvCntrl_b1_p3)	16	RW	05B0h
90h - 60_0090h	Tx dq driver equalization mode controls. (TxEqualizationMode_p0 - TxEqualizationMode_p3)	16	RW	0000h
92h - 60_0292h	TX impedance controls (TxImpedanceCtrl1_b0_p0 - TxImpedanceCtrl1_b1_p3)	16	RW	0FFFh
94h	Dq/Dqs receiver control (DqDqsRcvCntrl1)	16	RW	0400h
96h - 60_0296h	TX equalization impedance controls (TxImpedanceCtrl2_b0_p0 - TxImpedanceCtrl2_b1_p3)	16	RW	0000h
98h - 60_0098h	Dq/Dqs receiver control (DqDqsRcvCntrl2_p0 - DqDqsRcvCntrl2_p3)	16	RW	0000h
9Ah - 60_029Ah	TX ODT driver strength control (TxOdtDrvStren_b0_p0 - TxOdtDrvStren_b1_p3)	16	RW	0000h
ACh	Status of RX FIFO Consistency Checks (RxFifoCheckStatus)	16	RO	Table 9-4
AEh	Contains the captured values associated with an RxFifo consistency error (RxFifoCheckErrValues)	16	RO	Table 9-4
B0h	Data Receive FIFO Pointer Values (RxFifoInfo)	16	RO	Table 9-4
B2h	RX FIFO visibility (RxFifoVisibility)	16	RW	0000h
B4h	RX FIFO contents, lane[3:0] (RxFifoContentsDQ3210)	16	RO	Table 9-4
B6h	RX FIFO contents, lane[7:4] (RxFifoContentsDQ7654)	16	RO	Table 9-4
B8h	RX FIFO contents, dbi (RxFifoContentsDBI)	16	RO	Table 9-4

Table continues on the next page...

Offset	Register	Width (In bits)	Access	Reset value
BEh - 60_02BEh	TX slew rate controls (TxSlewRate_b0_p0 - TxSlewRate_b1_p3)	16	RW	07FFh
D0h - 10D0h	Read DQ per-bit BDL delay (Timing Group 0). (RxPBDlyTg0_r0 - RxPBDlyTg0_r8)	16	RW	0000h
D2h - 10D2h	Read DQ per-bit BDL delay (Timing Group 1). (RxPBDlyTg1_r0 - RxPBDlyTg1_r8)	16	RW	0000h
D4h - 10D4h	Read DQ per-bit BDL delay (Timing Group 2). (RxPBDlyTg2_r0 - RxPBDlyTg2_r8)	16	RW	0000h
D6h - 10D6h	Read DQ per-bit BDL delay (Timing Group 3). (RxPBDlyTg3_r0 - RxPBDlyTg3_r8)	16	RW	0000h
100h - 60_0300h	Trained Receive Enable Delay (For Timing Group 0) (RxEnDlyTg0_u0_p0 - RxEnDlyTg0_u1_p3)	16	RW	0100h
102h - 60_0302h	Trained Receive Enable Delay (For Timing Group 1) (RxEnDlyTg1_u0_p0 - RxEnDlyTg1_u1_p3)	16	RW	0100h
104h - 60_0304h	Trained Receive Enable Delay (For Timing Group 2) (RxEnDlyTg2_u0_p0 - RxEnDlyTg2_u1_p3)	16	RW	0100h
106h - 60_0306h	Trained Receive Enable Delay (For Timing Group 3) (RxEnDlyTg3_u0_p0 - RxEnDlyTg3_u1_p3)	16	RW	0100h
118h - 60_0318h	Trained Read DQS to RxClk Delay (Timing Group DEST=0). (RxClkDlyTg0_u0_p0 - RxClkDlyTg0_u1_p3)	16	RW	0010h
11Ah - 60_031Ah	Trained Read DQS to RxClk Delay (Timing Group DEST=1). (RxClkDlyTg1_u0_p0 - RxClkDlyTg1_u1_p3)	16	RW	0010h
11Ch - 60_031Ch	Trained Read DQS to RxClk Delay (Timing Group DEST=2). (RxClkDlyTg2_u0_p0 - RxClkDlyTg2_u1_p3)	16	RW	0010h
11Eh - 60_031Eh	Trained Read DQS to RxClk Delay (Timing Group DEST=3). (RxClkDlyTg3_u0_p0 - RxClkDlyTg3_u1_p3)	16	RW	0010h
140h - 14Eh	Maps Phy DQ lane to memory DQ0 (Dq0LnSel - Dq7LnSel)	16	RW	0000h
180h - 60_1180h	Write DQ Delay (Timing Group 0). (TxDqDlyTg0_r0_p0 - TxDqDlyTg0_r8_p3)	16	RW	0010h
182h - 60_1182h	Write DQ Delay (Timing Group 1). (TxDqDlyTg1_r0_p0 - TxDqDlyTg1_r8_p3)	16	RW	0010h
184h - 60_1184h	Write DQ Delay (Timing Group 2). (TxDqDlyTg2_r0_p0 - TxDqDlyTg2_r8_p3)	16	RW	0010h
186h - 60_1186h	Write DQ Delay (Timing Group 3). (TxDqDlyTg3_r0_p0 - TxDqDlyTg3_r8_p3)	16	RW	0010h
1A0h - 60_03A0h	Write DQS Delay (Timing Group DEST=0). (TxDqsDlyTg0_u0_p0 - TxDqsDlyTg0_u1_p3)	16	RW	0100h
1A2h - 60_03A2h	Write DQS Delay (Timing Group DEST=1). (TxDqsDlyTg1_u0_p0 - TxDqsDlyTg1_u1_p3)	16	RW	0100h
1A4h - 60_03A4h	Write DQS Delay (Timing Group DEST=2). (TxDqsDlyTg2_u0_p0 - TxDqsDlyTg2_u1_p3)	16	RW	0100h
1A6h - 60_03A6h	Write DQS Delay (Timing Group DEST=3). (TxDqsDlyTg3_u0_p0 - TxDqsDlyTg3_u1_p3)	16	RW	0100h
1C8h	Debug status of the DBYTE LCDL (DxLcdlStatus)	16	RO	Table 9-4
20_0040h	DFI MaxReadLatency (DFIMRL_p1)	16	RW	0006h
40_0040h	DFI MaxReadLatency (DFIMRL_p2)	16	RW	0006h

Table continues on the next page...

DDR PHY (DDR_PHY)

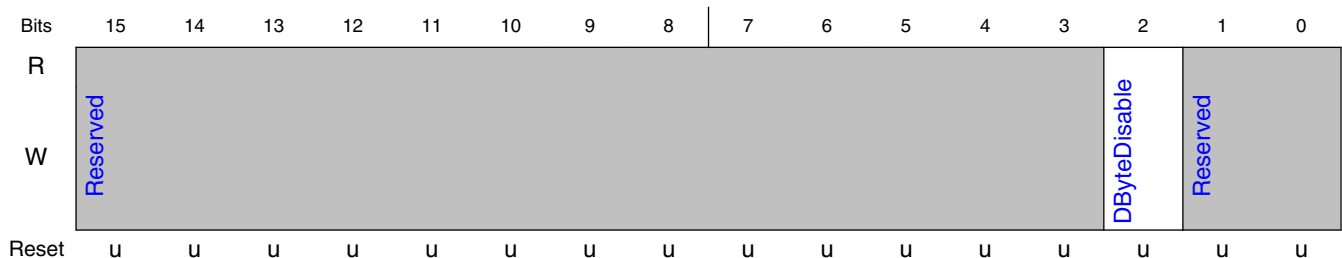
Offset	Register	Width (In bits)	Access	Reset value
40_015Ch	DQS Oscillator Count inverse at time of training in LPDDR4 drift compensation (PptDqsCntInvTrnTg0_p2)	16	RW	0000h
40_015Eh	DQS Oscillator Count inverse at time of training in LPDDR4 drift compensation (PptDqsCntInvTrnTg1_p2)	16	RW	0000h
60_0040h	DFI MaxReadLatency (DFIMRL_p3)	16	RW	0006h
60_015Ch	DQS Oscillator Count inverse at time of training in LPDDR4 drift compensation (PptDqsCntInvTrnTg0_p3)	16	RW	0000h
60_015Eh	DQS Oscillator Count inverse at time of training in LPDDR4 drift compensation (PptDqsCntInvTrnTg1_p3)	16	RW	0000h

9.4.3.3.2 DBYTE Module Disable (DbyteMiscMode)

9.4.3.3.2.1 Offset

Register	Offset
DbyteMiscMode	0h

9.4.3.3.2.2 Diagram



9.4.3.3.2.3 Fields

Field	Function
15-3 —	Reserved
2 DByteDisable	Controls whether this DBYTE module is disabled. Controls whether this DBYTE module is disabled. If this DBYTE module is not enabled, it receives no clocks and remains in reset. 0 - Enable this DBYTE module 1 - Disable this DBYTE module

Table continues on the next page...

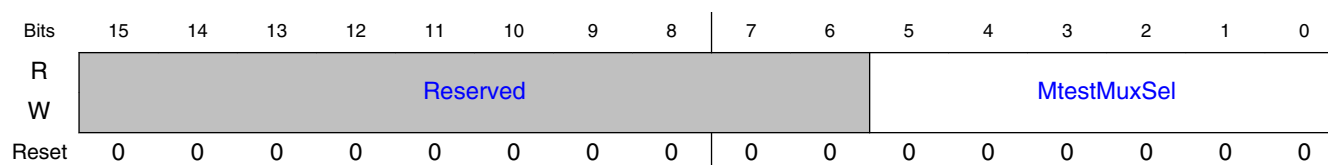
Field	Function
	This field should only be changed PHY initialization step C.
1-0 —	Reserved

9.4.3.3.3 Digital Observation Pin control (MtestMuxSel)

9.4.3.3.3.1 Offset

Register	Offset
MtestMuxSel	34h

9.4.3.3.3.2 Diagram



9.4.3.3.3.3 Fields

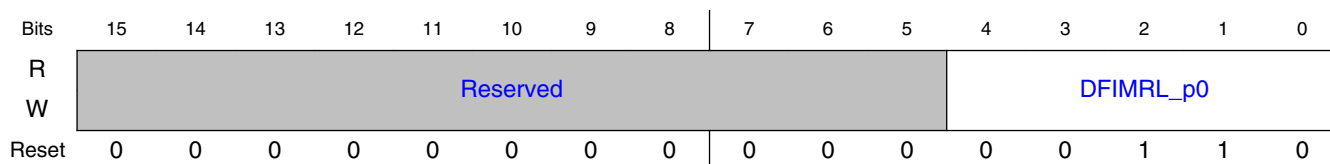
Field	Function
15-6 —	Reserved
5-0 MtestMuxSel	Controls for the 64-1 mux for asynchronous data to the Digital Observation Pin. Controls for the 64-1 mux for asynchronous data to the Digital Observation Pin. Encoding 6'h0 --> Drive 0 from this chiplet (allows flat 'OR' of pass-through information) Encoding 6'h01:1f --> Select local data from AC/DBYTE/MASTER macro Encoding 6'h20 --> Reserved (Drive 0 from macro into mux; Drive 0 from PUB synth logic path into mux) Encoding 6'h21:3f --> Select local data from PUB AC/DBYTE/MASTER synthesized logic Note: See PUB documentation for how, or if, the Digital Observation Pin is mapped to a physical bump in this configuration.

9.4.3.3.4 DFI MaxReadLatency (DFIMRL_p0)

9.4.3.3.4.1 Offset

Register	Offset
DFIMRL_p0	40h

9.4.3.3.4.2 Diagram



9.4.3.3.4.3 Fields

Field	Function
15-5 —	Reserved
4-0 DFIMRL_p0	This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. This is the value, in units of two mem clocks, between dfi_rddata_en and dfi_rddata_valid A unit change in the LSB is a change in MRL of two mem clocks.

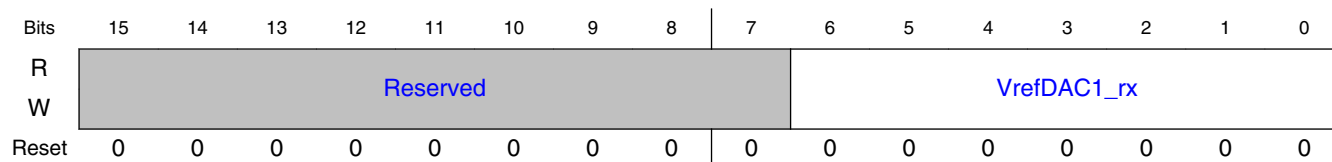
9.4.3.3.5 VrefDAC1 control for DQ Receiver (used only when DFE is enabled in DDR4) (VrefDAC1_r0 - VrefDAC1_r8)

9.4.3.3.5.1 Offset

For x = 0 to 8:

Register	Offset
VrefDAC1_rx	60h + (x × 200h)

9.4.3.3.5.2 Diagram



9.4.3.3.5.3 Fields

Field	Function
15-7 —	Reserved
6-0 VrefDAC1_rx	VrefDAC1 controls the alternate VREF setting for DFE (used only when DFE is enabled in DDR4) DAC control for rxdq cell internal VREF, trained by Firmware The VREF generators have different ranges, depending on the Mission Mode settings for DqDqsRcvCntrl::MajorMode,DqDqsRcvCntrl::ExtVrefRange 011,0 :: VREF = VDDQ*(0. VrefDAC1 controls the alternate VREF setting for DFE (used only when DFE is enabled in DDR4) DAC control for rxdq cell internal VREF, trained by Firmware The VREF generators have different ranges, depending on the Mission Mode settings for DqDqsRcvCntrl::MajorMode,DqDqsRcvCntrl::ExtVrefRange 011,0 :: VREF = VDDQ*(0.510 + VrefDAC1[6:0]*0.00345) 011,1 :: VREF = VDDQ*(0.453 + VrefDAC1[6:0]*0.00385) non-enumerated encodings are reserved Register Block Offset Address 0x030 is the VrefDAC1 for lane0. Register Block Offset Address 0x130 is the VrefDAC1 for lane1. Register Block Offset Address 0x230 is the VrefDAC1 for lane2. Register Block Offset Address 0x330 is the VrefDAC1 for lane3. Register Block Offset Address 0x430 is the VrefDAC1 for lane4. Register Block Offset Address 0x530 is the VrefDAC1 for lane5. Register Block Offset Address 0x630 is the VrefDAC1 for lane6. Register Block Offset Address 0x730 is the VrefDAC1 for lane7. Register Block Offset Address 0x830 is the VrefDAC1 for laneDBI.

9.4.3.3.6 VrefDAC0 control for DQ Receiver (VrefDAC0_r0 - VrefDAC0_r8)

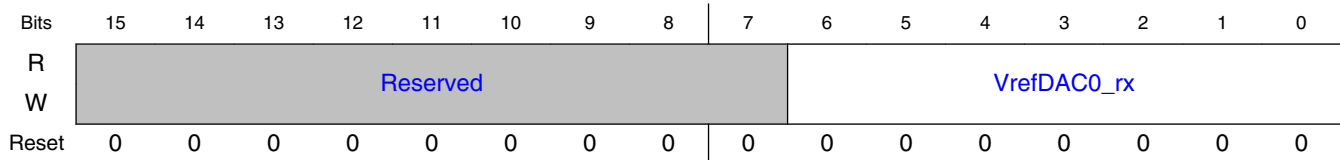
9.4.3.3.6.1 Offset

For x = 0 to 8:

DDR PHY (DDR_PHY)

Register	Offset
VrefDAC0_rx	80h + (x × 200h)

9.4.3.3.6.2 Diagram



9.4.3.3.6.3 Fields

Field	Function
15-7 —	Reserved
6-0 VrefDAC0_rx	PHY RX VREF DAC control for rxdq cell internal VREF, (used only when 2D training is enabled in LPDDR4,DDR4) DAC control for rxdq cell internal VREF, trained by Firmware The VREF generators have different ranges, depending on the Mission Mode settings for DqDqsRcvCntrl::MajorMode,DqDqsRcvCntrl::ExtVrefRange 000,0 :: VREF = VDDQ*(0. PHY RX VREF DAC control for rxdq cell internal VREF, (used only when 2D training is enabled in LPDDR4,DDR4) DAC control for rxdq cell internal VREF, trained by Firmware The VREF generators have different ranges, depending on the Mission Mode settings for DqDqsRcvCntrl::MajorMode,DqDqsRcvCntrl::ExtVrefRange 000,0 :: VREF = VDDQ*(0.287 + VrefDAC0[6:0]*0.00330) 000,1 :: VREF = VDDQ*(0.250 + VrefDAC0[6:0]*0.00385) 011,0 :: VREF = VDDQ*(0.510 + VrefDAC0[6:0]*0.00345) 011,1 :: VREF = VDDQ*(0.453 + VrefDAC0[6:0]*0.00385) 010,0 :: VREF = VDDQ*(0.047 + VrefDAC0[6:0]*0.00367) non-enumerated encodings are reserved Register Block Offset Address 0x040 is the VrefDAC0 for lane0. Register Block Offset Address 0x140 is the VrefDAC0 for lane1. Register Block Offset Address 0x240 is the VrefDAC0 for lane2. Register Block Offset Address 0x340 is the VrefDAC0 for lane3. Register Block Offset Address 0x440 is the VrefDAC0 for lane4. Register Block Offset Address 0x540 is the VrefDAC0 for lane5. Register Block Offset Address 0x640 is the VrefDAC0 for lane6. Register Block Offset Address 0x740 is the VrefDAC0 for lane7. Register Block Offset Address 0x840 is the VrefDAC0 for laneDBI.

9.4.3.3.7 Data TX impedance controls (TxImpedanceCtrl0_b0_p0 - TxImpedanceCtrl0_b1_p3)

9.4.3.3.7.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
TxImpedanceCtrl0_bn_px	82h + (x × 20_0000h) + (n × 200h)

9.4.3.3.7.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				DrvStrenDqN				DrvStrenDqP							
W																
Reset	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1

9.4.3.3.7.3 Fields

Field	Function
15-12 —	Reserved
11-6 DrvStrenDqN	Please Refer to Technology specific PHY DATABOOK for supported options 6 bit bus used to select the target pull down output impedance. Please Refer to Technology specific PHY DATABOOK for supported options 6 bit bus used to select the target pull down output impedance. Connects to the DrvStren pins of the driver thus: Csr_DrvStren120N[2:0], Csr_DrvStren240N[1:0], Csr_DrvStren480N[0:0] = csrDrvStrenDqN See table of csr DrvStrenDqP for list of possible impedances.
5-0 DrvStrenDqP	Please Refer to Technology specific PHY DATABOOK for supported options 6 bit bus used to select the target pull down output impedance. Please Refer to Technology specific PHY DATABOOK for supported options 6 bit bus used to select the target pull down output impedance. Connects to the DrvStren pins of the driver thus: Csr_DrvStren120P[2:0], Csr_DrvStren240P[1:0], Csr_DrvStren480P[0:0] = csrDrvStrenDqP 000000 - HiZ. 000001 - 480.0 000010 - 240.0

Field	Function
	000011 - 160.0
	000110 - 120.0
	000111 - 96.0
	001010 - 80.0
	001011 - 68.6
	001110 - 60.0
	001111 - 53.3
	011010 - 48.0
	011011 - 43.6
	011110 - 40.0
	011111 - 36.9
	111010 - 34.3
	111011 - 32.0
	111110 - 30.0
	111111 - 28.2

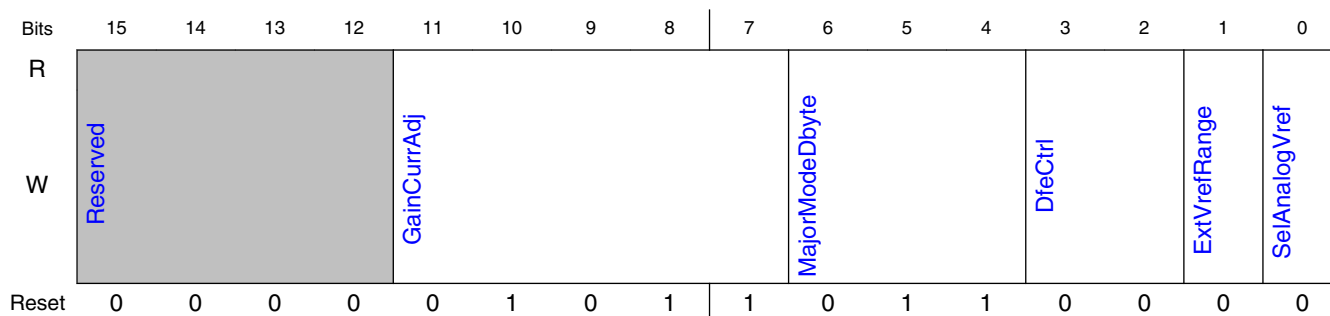
9.4.3.3.8 Dq/Dqs receiver control (DqDqsRcvCntrl_b0_p0 - DqDqsRcvCntrl_b1_p3)

9.4.3.3.8.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
DqDqsRcvCntrl_bn_px	86h + (x × 20_0000h) + (n × 200h)

9.4.3.3.8.2 Diagram



9.4.3.3.8.3 Fields

Field	Function
15-12 —	Reserved[12] unavailable - should not be set Reserved[13] for future RX circuit trim Reserved[14] Set to 1 to enable VIL/VIH testing of the DFE0 Amp Reserved[15] Set to 1 to enable VIL/VIH testing of the DFE1 Amp NOTE: Reserved[15:14] should not be asserted simultaneously
11-7 GainCurrAdj	Adjust gain current of RX amplifier stage. Adjust gain current of RX amplifier stage. It is recommended to use default values for this CSR.
6-4 MajorModeDbyte	Selects the major mode of operation for the receiver. Selects the major mode of operation for the receiver. These settings are determined by PHY Configuration based on DRAM protocol 000 - DDR3 001 - reserved (room for LPDDR3) 010 - LPDDR4 011 - DDR4 100 - DDR3 Low-power/low-speed 101 - reserved (LPDDR3 low-power/low-speed) 110 - LPDDR4 low-power/low-speed 111 - DDR4 Low-power/low-speed
3-2 DfeCtrl	DFE may be used with MajorModeDbyte=011 only 00 - DFE off 01 - DFE on 10 - Train DFE0 Amplifier 11 - Train DFE1 Amplifier These settings are determined by PHY Training FW and should not be overridden.
1 ExtVrefRange	Extends the range available in the local per-bit VREF generator.
0 SelAnalogVref	Setting this signal high will force the local per-bit VREF generator to pass the global VREFA to the samplers.

9.4.3.3.9 Tx dq driver equalization mode controls. (TxEqualizationMode_p0 - TxEqualizationMode_p3)

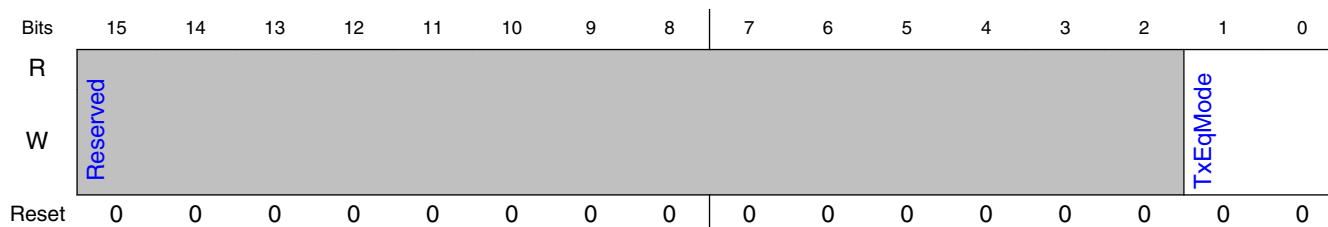
9.4.3.3.9.1 Offset

Register	Offset
TxEqualizationMode_p0	90h
TxEqualizationMode_p1	20_0090h
TxEqualizationMode_p2	40_0090h
TxEqualizationMode_p3	60_0090h

9.4.3.3.9.2 Function

Tx dq driver equalization mode controls. FFE, de-emphasis DDR43 PHY ONLY

9.4.3.3.9.3 Diagram



9.4.3.3.9.4 Fields

Field	Function
15-2 —	Reserved
1-0 TxEqMode	

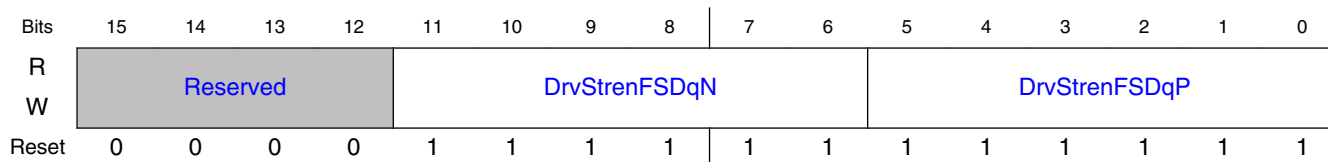
9.4.3.3.10 TX impedance controls (TxImpedanceCtrl1_b0_p0 - TxImpedanceCtrl1_b1_p3)

9.4.3.3.10.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
TxImpedanceCtrl1_bn_px	92h + (x × 20_0000h) + (n × 200h)

9.4.3.3.10.2 Diagram



9.4.3.3.10.3 Fields

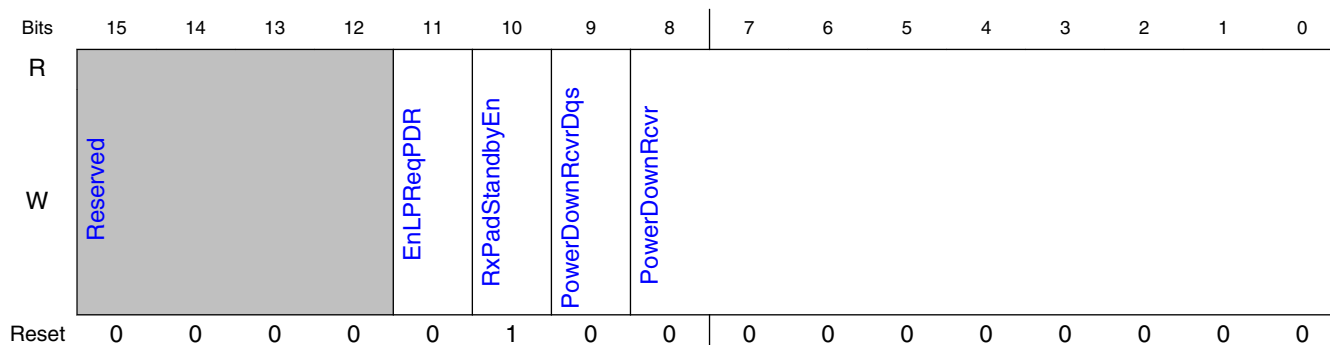
Field	Function
15-12 —	Reserved
11-6 DrvStrenFSDqN	Please Refer to Technology specific PHY DATABOOK for supported options 6 bit bus used to select the target pull up output impedance used in equalization. Please Refer to Technology specific PHY DATABOOK for supported options 6 bit bus used to select the target pull up output impedance used in equalization. Connects to the DrvStren pins of the driver thus: Csr_DrvStrenFS120N[2:0], Csr_DrvStrenFS240N[1:0], Csr_DrvStrenFS480N[0:0] = csrDrvStrenDqN See table of csr DrvStrenFSDqP for list of possible impedances.
5-0 DrvStrenFSDqP	Please Refer to Technology specific PHY DATABOOK for supported options 6 bit bus used to select the target pull up output impedance used in equalization. Please Refer to Technology specific PHY DATABOOK for supported options 6 bit bus used to select the target pull up output impedance used in equalization. Connects to the DrvStren pins of the driver thus: Csr_DrvStrenFS120P[2:0], Csr_DrvStrenFS240P[1:0], Csr_DrvStrenFS480P[0] = csrFSDrvStrenDq 000000 - HiZ. 000001 - 480.0 000010 - 240.0 000011 - 160.0 000110 - 120.0 000111 - 96.0 001010 - 80.0 001011 - 68.6 001110 - 60.0 001111 - 53.3 011010 - 48.0 011011 - 43.6 011110 - 40.0 011111 - 36.9 111010 - 34.3 111011 - 32.0 111110 - 30.0 111111 - 28.2

9.4.3.3.11 Dq/Dqs receiver control (DqDqsRcvCntrl1)

9.4.3.3.11.1 Offset

Register	Offset
DqDqsRcvCntrl1	94h

9.4.3.3.11.2 Diagram



9.4.3.3.11.3 Fields

Field	Function
15-12 —	Reserved
11 EnLPReqPDR	Reserved for future use
10 RxPadStandbyEn	Enables the rxdq/rxdqs StandBy power savings, per pad-group.
9 PowerDownRcvrDqs	Active high signal which powers down the receiver. Active high signal which powers down the receiver. After this pin is deasserted the receiver cannot be used for a minimum of 100 ns. This control powers down both the upper and lower DQS receivers. If Register X4TG[3:0]=0000, then the upper DQS receiver is powered down.
8-0 PowerDownRcvr	Active high signal which powers down the receiver. Active high signal which powers down the receiver. [7:0 -> dq[7:0], 8 -> dbi] After this pin is deasserted the receiver cannot be used for a minimum of 100 ns.

9.4.3.3.12 TX equalization impedance controls (TxImpedanceCtrl2_b0_p0 - TxImpedanceCtrl2_b1_p3)

9.4.3.3.12.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
TxImpedanceCtrl2_bn_px	96h + (x × 20_0000h) + (n × 200h)

9.4.3.3.12.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				DrvStrenEQLoDqN				DrvStrenEQHiDqP							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.4.3.3.12.3 Fields

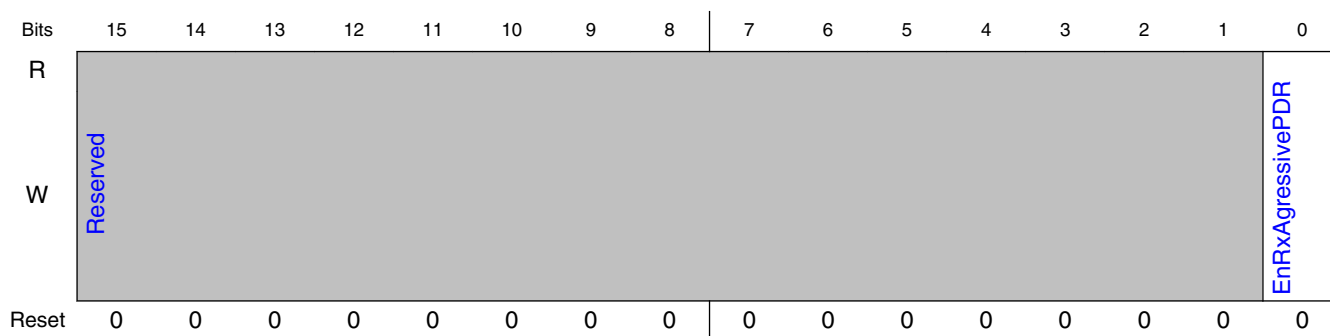
Field	Function
15-12 —	Reserved
11-6 DrvStrenEQLoDqN	Please Refer to Technology specific PHY DATABOOK for supported options 6 bit bus used to select the target pull down output impedance used in equalization. Please Refer to Technology specific PHY DATABOOK for supported options 6 bit bus used to select the target pull down output impedance used in equalization. Connects to the DrvStren pins of the driver thus: Csr_DrvStrenEQLo120N[2:0], Csr_DrvStrenEQLo240N[1:0], Csr_DrvStrenEQLo480P[0] = csrDrvStrenEQLoDq
5-0 DrvStrenEQHiDqP	Please Refer to Technology specific PHY DATABOOK for supported options 6 bit bus used to select the target pull up output impedance used in equalization. Please Refer to Technology specific PHY DATABOOK for supported options 6 bit bus used to select the target pull up output impedance used in equalization. Connects to the DrvStren pins of the driver thus: Csr_DrvStrenEQHi120P[2:0], Csr_DrvStrenEQHi240P[1:0], Csr_DrvStrenEQHi480P[0] = csrDrvStrenEQHiDq

9.4.3.3.13 Dq/Dqs receiver control (DqDqsRcvCntrl2_p0 - DqDqsRcvCntrl2_p3)

9.4.3.3.13.1 Offset

Register	Offset
DqDqsRcvCntrl2_p0	98h
DqDqsRcvCntrl2_p1	20_0098h
DqDqsRcvCntrl2_p2	40_0098h
DqDqsRcvCntrl2_p3	60_0098h

9.4.3.3.13.2 Diagram



9.4.3.3.13.3 Fields

Field	Function
15-1	Reserved
—	
0	reserved
EnRxAggressive PDR	

9.4.3.3.14 TX ODT driver strength control (TxOdtDrvStren_b0_p0 - TxOdtDrvStren_b1_p3)

9.4.3.3.14.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
TxOdtDrvStren_bn_px	9Ah + (x × 20_0000h) + (n × 200h)

9.4.3.3.14.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				ODTStrenN				ODTStrenP							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.4.3.3.14.3 Fields

Field	Function
15-12 —	Reserved
11-6 ODTStrenN	Selects the ODT pull-down impedance. Selects the ODT pull-down impedance.. Please Refer to Technology specific PHY DATABOOK for supported options ODTStren120N[2:0], ODTStren120N[1:0],ODTStren480N[0:0] 000_00_0 = High Impedance 000_00_1 = 480 ohms 000_01_0 = 240 ohms 000_01_1 = 160 ohms 001_00_0 = 120 ohms 001_00_1 = 96.0 ohms 001_01_0 = 80.0 ohms 001_01_1 = 68.6 ohms 011_00_0 = 60.0 ohms 011_00_1 = 53.3 ohms 011_01_0 = 48.0 ohms 011_01_1 = 43.6 ohms 111_00_0 = 40.0 ohms 111_00_1 = 36.9 ohms 111_01_0 = 34.3 ohms 111_01_1 = 32.0 ohms 111_11_0 = 30.0 ohms 111_11_1 = 28.2 ohms
5-0	Selects the ODT pull-up impedance.

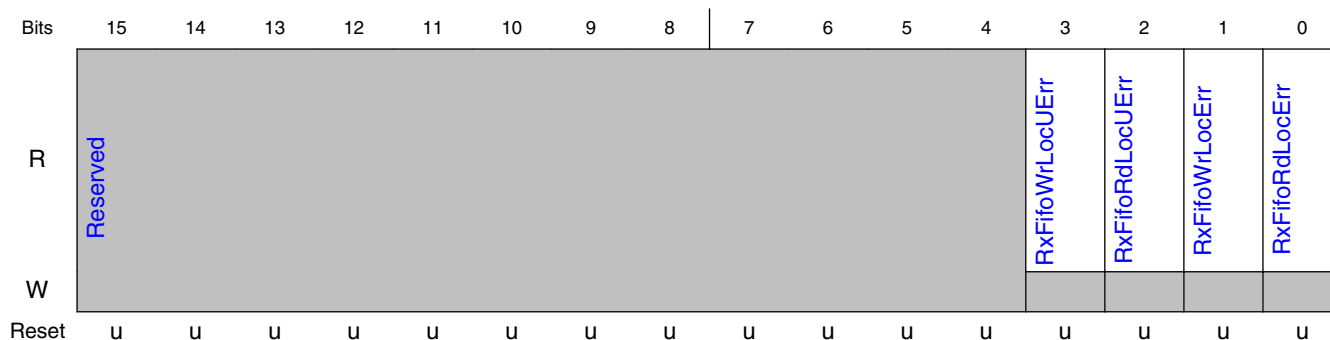
Field	Function
ODTStrenP	Selects the ODT pull-up impedance. Please Refer to Technology specific PHY DATABOOK for supported options ODTStren120P[2:0], ODTStren240P[1:0],ODTStren480P[0:0] 000_00_0 = High Impedance 000_00_1 = 480 ohms 000_01_0 = 240 ohms 000_01_1 = 160 ohms 001_00_0 = 120 ohms 001_00_1 = 96.0 ohms 001_01_0 = 80.0 ohms 001_01_1 = 68.6 ohms 011_00_0 = 60.0 ohms 011_00_1 = 53.3 ohms 011_01_0 = 48.0 ohms 011_01_1 = 43.6 ohms 111_00_0 = 40.0 ohms 111_00_1 = 36.9 ohms 111_01_0 = 34.3 ohms 111_01_1 = 32.0 ohms 111_11_0 = 30.0 ohms 111_11_1 = 28.2 ohms

9.4.3.3.15 Status of RX FIFO Consistency Checks (RxFifoCheckStatus)

9.4.3.3.15.1 Offset

Register	Offset
RxFifoCheckStatus	ACh

9.4.3.3.15.2 Diagram



9.4.3.3.15.3 Fields

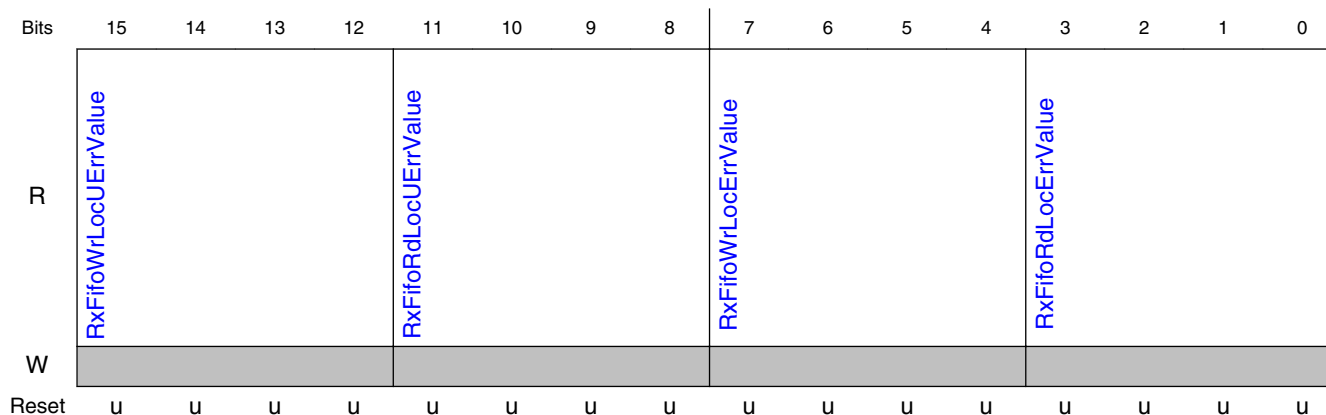
Field	Function
15-4 —	Reserved
3 RxFifoWrLocUErr	If set, the write pointer (DQS side) on the read FIFO associated with data bits [7:4] has a non-zero value at least once.
2 RxFifoRdLocUErr	If set, the read pointer (DFI side) on the read FIFO associated with data bits [7:4] has a non-zero value at least once.
1 RxFifoWrLocErr	If set, the write pointer (DQS side) on the read FIFO associated with data bits [3:0] has a non-zero value at least once.
0 RxFifoRdLocErr	If set, the read pointer (DFI side) on the read FIFO associated with data bits [3:0] had a non-zero value at least once.

9.4.3.3.16 Contains the captured values associated with an Rx Fifo consistency error (RxFifoCheckErrValues)

9.4.3.3.16.1 Offset

Register	Offset
RxFifoCheckErrValues	AEh

9.4.3.3.16.2 Diagram



9.4.3.3.16.3 Fields

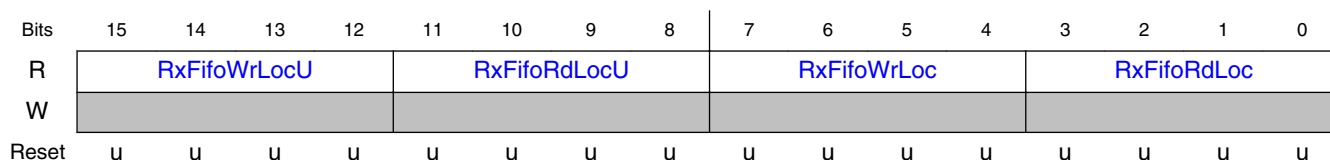
Field	Function
15-12 RxFifoWrLocUErrValue	The first error value captured for the write pointer (DQS side) on the read FIFO associated with data bits [7:4];
11-8 RxFifoRdLocUErrValue	The first error value captured for the read pointer (DFI side) on the read FIFO associated with data bits [7:4];
7-4 RxFifoWrLocErrValue	The first error value captured for the write pointer (DQS side) on the read FIFO associated with data bits [3:0];
3-0 RxFifoRdLocErrValue	The first error value captured for the read pointer (DFI side) on the read FIFO associated with data bits [3:0];

9.4.3.3.17 Data Receive FIFO Pointer Values (RxFifoInfo)

9.4.3.3.17.1 Offset

Register	Offset
RxFifoInfo	B0h

9.4.3.3.17.2 Diagram



9.4.3.3.17.3 Fields

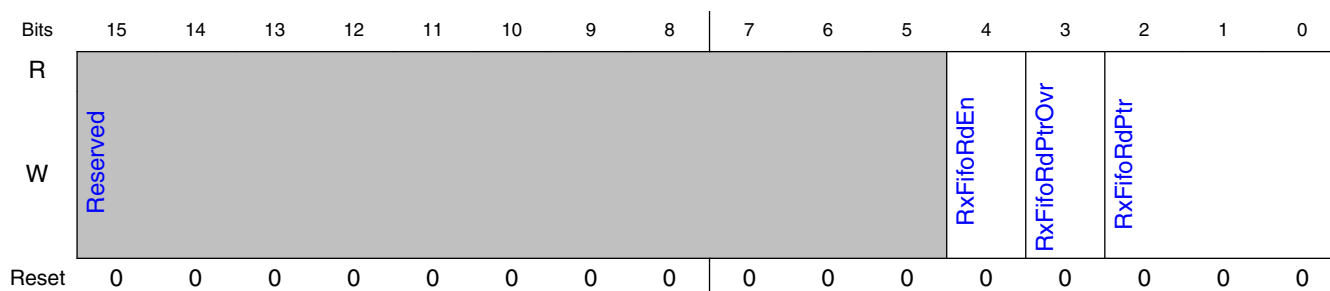
Field	Function
15-12 RxFifoWrLocU	The Mission mode write pointer of the upper-nibble Rx fifo.
11-8 RxFifoRdLocU	The Mission mode read pointer of the upper-nibble Rx fifo.
7-4 RxFifoWrLoc	The Mission mode write pointer of the lower-nibble Rx fifo.
3-0 RxFifoRdLoc	The Mission mode read pointer of the lower-nibble Rx fifo.

9.4.3.3.18 RX FIFO visibility (RxFifoVisibility)

9.4.3.3.18.1 Offset

Register	Offset
RxFifoVisibility	B2h

9.4.3.3.18.2 Diagram



9.4.3.3.18.3 Fields

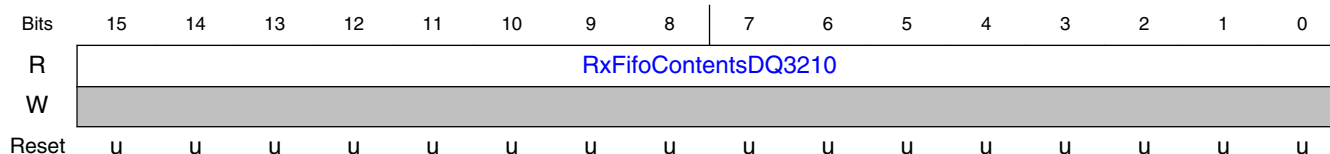
Field	Function
15-5 —	Reserved
4 RxFifoRdEn	Pulse set 0-->1-->0 this bit to capture the Fifo Contents.
3 RxFifoRdPtrOvr	0 : Normal operation - mission mode read pointer is enabled 1 : Override - Control of the rx fifo read pointer is ceded to CSR RxFifoRdPtr.
2-0 RxFifoRdPtr	If CSR RxFifoRdPtrOvr is set, then this CSR selects the rxfifo entry is visible in CSR This 3b field addresses 4b units of the 8x4b (32entry) fifo; that is, rdfifo_nibble_address[2:0]=csrRxFifoRdPtr[2:0] For example, Register RxFifoRdPtr[2:0]=2 will enable reading bit-entries 11. If CSR RxFifoRdPtrOvr is set, then this CSR selects the rxfifo entry is visible in CSR This 3b field addresses 4b units of the 8x4b (32entry) fifo; that is, rdfifo_nibble_address[2:0]=csrRxFifoRdPtr[2:0] For example, Register RxFifoRdPtr[2:0]=2 will enable reading bit-entries 11..8. The exact location of read data will depend on the prior history of reads and PHY initialization.

9.4.3.3.19 RX FIFO contents, lane[3:0] (RxFifoContentsDQ3210)

9.4.3.3.19.1 Offset

Register	Offset
RxFifoContentsDQ3210	B4h

9.4.3.3.19.2 Diagram



9.4.3.3.19.3 Fields

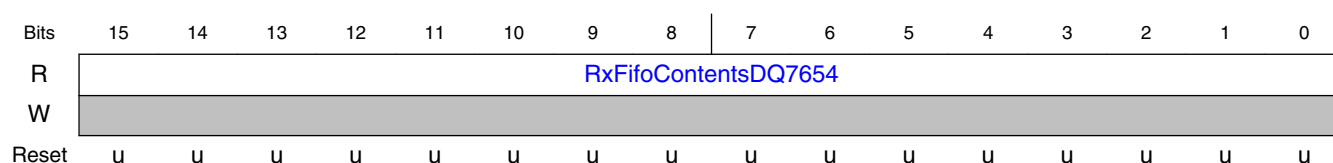
Field	Function
15-0 RxFifoContents DQ3210	A window into the contents of the RxFifo, as controlled by CSR RxFifoVisibility This register reads 4b at a time from lane0. A window into the contents of the RxFifo, as controlled by CSR RxFifoVisibility This register reads 4b at a time from lane0..3 from the four fifo entries addressed by rdfifo_nibble_address[2:0]=RxFifoRdPtr[2:0] Register [15:12] = lane3_ui3, lane3_ui2, lane3_ui1, lane3_ui0 where ui0 is the first occurring bit [11: 8] = lane2_ui3, lane2_ui2, lane2_ui1, lane2_ui0 [7: 4] = lane1_ui3, lane1_ui2, lane1_ui1, lane1_ui0 [3: 0] = lane0_ui3, lane0_ui2, lane0_ui1, lane0_ui0 Note that the DBYTE lane of a given index is not the same as a memory DQ of the same index unless the csr Dq<7..0>LnSel[2:0] have their default/reset value.

9.4.3.3.20 RX FIFO contents, lane[7:4] (RxFifoContentsDQ7654)

9.4.3.3.20.1 Offset

Register	Offset
RxFifoContentsDQ7654	B6h

9.4.3.3.20.2 Diagram



9.4.3.3.20.3 Fields

Field	Function
15-0 RxFifoContents DQ7654	A window into the contents of the RxFifo, as controlled by CSR RxFifoVisibility This register reads 4b at a time from lane4. A window into the contents of the RxFifo, as controlled by CSR RxFifoVisibility This register reads 4b at a time from lane4..7 from the four fifo entries addressed by rdfifo_nibble_address[2:0]=RxFifoRdPtr[2:0] Register [15:12] = lane7_ui3, lane7_ui2, lane7_ui1, lane7_ui0

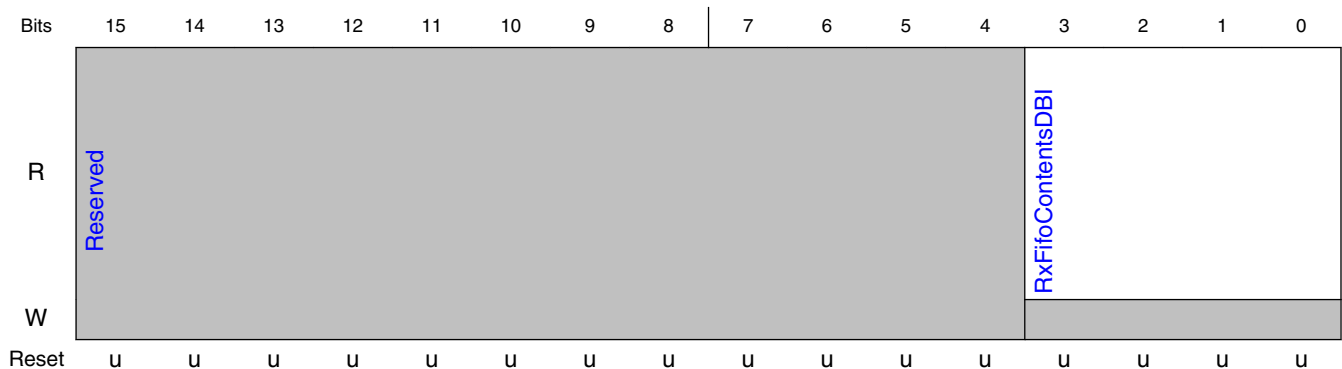
Field	Function
	[11: 8] = lane6_ui3, lane6_ui2, lane6_ui1, lane6_ui0 [7: 4] = lane5_ui3, lane5_ui2, lane5_ui1, lane5_ui0 [3: 0] = lane4_ui3, lane4_ui2, lane4_ui1, lane4_ui0 Note that the DBYTE lane of a given index is not the same as a memory DQ of the same index unless the csr Dq<7..0>LnSel[2:0] have their default/reset value.

9.4.3.3.21 RX FIFO contents, dbi (RxFifoContentsDBI)

9.4.3.3.21.1 Offset

Register	Offset
RxFifoContentsDBI	B8h

9.4.3.3.21.2 Diagram



9.4.3.3.21.3 Fields

Field	Function
15-4 —	Reserved
3-0 RxFifoContents DBI	A window into the contents of the RxFifo, as controlled by CSR RxFifoVisibility This register reads 4b at a time from DBI from the four fifo entries addressed by rdfifo_nibble_address[2:0]=RxFifoRdPtr[2:0] Register [3: 0] = dbi_ui3, dbi_ui2, dbi_ui1, dbi_ui0 Note that the DBYTE DBI lane is the same as the memory DBI; it is not subject to mapping using csr Dq<7. A window into the contents of the RxFifo, as controlled by CSR RxFifoVisibility This register reads 4b at a time from DBI

Field	Function
	from the four fifo entries addressed by <code>rdfifo_nibble_address[2:0]=RxFifoRdPtr[2:0]</code> Register [3: 0] = <code>dbi_ui3,dbi_ui2,dbi_ui1,dbi_ui0</code> Note that the DBYTE DBI lane is the same as the memory DBI; it is not subject to mapping using <code>csr Dq<7..0>LnSel[2:0]</code> as the DQ are.

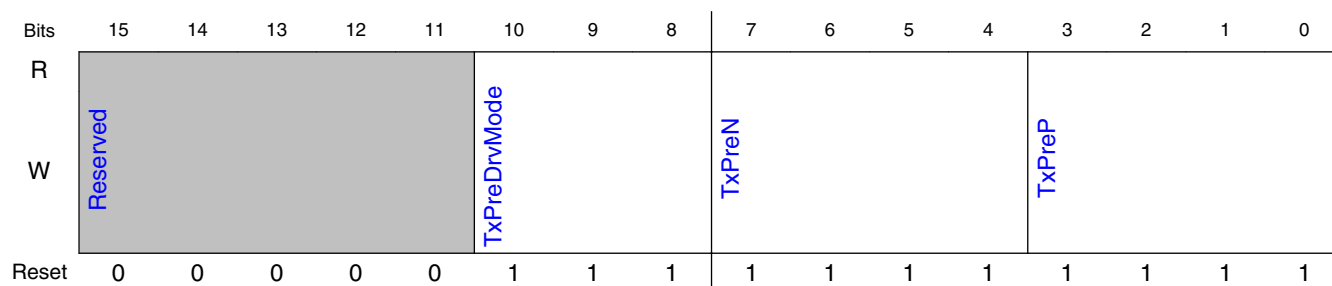
9.4.3.3.22 TX slew rate controls (TxSlewRate_b0_p0 - TxSlewRate_b1_p3)

9.4.3.3.22.1 Offset

For $n = 0$ to 1; $x = 0$ to 3:

Register	Offset
TxSlewRate_bn_px	$BEh + (x \times 20_0000h) + (n \times 200h)$

9.4.3.3.22.2 Diagram



9.4.3.3.22.3 Fields

Field	Function
15-11 —	Reserved
10-8 TxPreDrvMode	Controls predrivers to adjust timing of turn-on and turn-off of pull-up and pull-down segments.
7-4 TxPreN	4 bit binary trim for the driver pull down slew rate. 4 bit binary trim for the driver pull down slew rate. 4'b0000 has a slower slew rate than 4'b1111
3-0 TxPreP	4 bit binary trim for the driver pull up slew rate. 4 bit binary trim for the driver pull up slew rate. 4'b0000 has a slower slew rate than 4'b1111

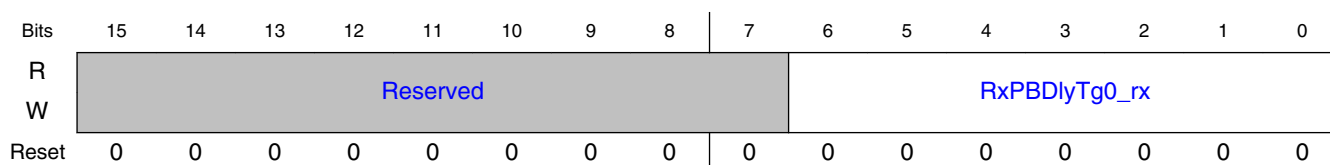
9.4.3.3.23 Read DQ per-bit BDL delay (Timing Group 0). (RxPBDlyTg0_r0 - RxPBDlyTg0_r8)

9.4.3.3.23.1 Offset

For x = 0 to 8:

Register	Offset
RxPBDlyTg0_rx	D0h + (x × 200h)

9.4.3.3.23.2 Diagram



9.4.3.3.23.3 Fields

Field	Function
15-7 —	Reserved
6-0 RxPBDlyTg0_rx	Read DQ per-bit BDL delay (Timing Group 0). Read DQ per-bit BDL delay (Timing Group 0). Trained to deskew the read DQ data that will be sampled by the incoming per-nibble read DQS, to improve the composite eye. The unit of change is a LSB in the control of the delay circuit, and is not denominated in units of UI. Register Block Offset Address 0x068 controls the lane0 read deskew timing. (For Timing Group 0) Register Block Offset Address 0x168 controls the lane1 read deskew timing. (For Timing Group 0) Register Block Offset Address 0x268 controls the lane2 read deskew timing. (For Timing Group 0) Register Block Offset Address 0x368 controls the lane3 read deskew timing. (For Timing Group 0) Register Block Offset Address 0x468 controls the lane4 read deskew timing. (For Timing Group 0) Register Block Offset Address 0x568 controls the lane5 read deskew timing. (For Timing Group 0) Register Block Offset Address 0x668 controls the lane6 read deskew timing. (For Timing Group 0) Register Block Offset Address 0x768 controls the lane7 read deskew timing. (For Timing Group 0) Register Block Offset Address 0x868 controls the laneDBI read deskew timing. (For Timing Group 0) Note that this register is not per-pstate. The same deskew (for channel-routing differences) is used for all pstates.

Field	Function
	. Note: Timing Group 0 = RANK0 for all systems except 3DS/LRDIMM.

9.4.3.3.24 Read DQ per-bit BDL delay (Timing Group 1). (RxPBDlyTg1_r0 - RxPBDlyTg1_r8)

9.4.3.3.24.1 Offset

For x = 0 to 8:

Register	Offset
RxPBDlyTg1_rx	D2h + (x × 200h)

9.4.3.3.24.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								RxPBDlyTg1_rx							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.4.3.3.24.3 Fields

Field	Function
15-7 —	Reserved
6-0 RxPBDlyTg1_rx	Read DQ per-bit BDL delay (Timing Group 1). Read DQ per-bit BDL delay (Timing Group 1). Trained to deskew the read DQ data that will be sampled by the incoming per-nibble read DQS, to improve the composite eye. The unit of change is a LSB in the control of the delay circuit, and is not denominated in units of UI. Register Block Offset Address 0x069 controls the lane0 read deskew timing. (For Timing Group 1) Register Block Offset Address 0x169 controls the lane1 read deskew timing. (For Timing Group 1) Register Block Offset Address 0x269 controls the lane2 read deskew timing. (For Timing Group 1) Register Block Offset Address 0x369 controls the lane3 read deskew timing. (For Timing Group 1) Register Block Offset Address 0x469 controls the lane4 read deskew timing. (For Timing Group 1)

Field	Function
	Register Block Offset Address 0x569 controls the lane5 read deskew timing. (For Timing Group 1) Register Block Offset Address 0x669 controls the lane6 read deskew timing. (For Timing Group 1) Register Block Offset Address 0x769 controls the lane7 read deskew timing. (For Timing Group 1) Register Block Offset Address 0x869 controls the laneDBI read deskew timing. (For Timing Group 1) Note that this register is not per-pstate. The same deskew (for channel-routing differences) is used for all pstates. . Note: Timing Group 1 = RANK1 for all systems except 3DS/LRDIMM.

9.4.3.3.25 Read DQ per-bit BDL delay (Timing Group 2). (RxPBDlyTg2_r0 - RxPBDlyTg2_r8)

9.4.3.3.25.1 Offset

For x = 0 to 8:

Register	Offset
RxPBDlyTg2_rx	D4h + (x × 200h)

9.4.3.3.25.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								RxPBDlyTg2_rx							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.4.3.3.25.3 Fields

Field	Function
15-7 —	Reserved
6-0 RxPBDlyTg2_rx	Read DQ per-bit BDL delay (Timing Group 2). Read DQ per-bit BDL delay (Timing Group 2). Trained to deskew the read DQ data that will be sampled by the incoming per-nibble read DQS, to improve the composite eye. The unit of change is a LSB in the control of the delay circuit, and is not denominated in units of UI.

Field	Function
	Register Block Offset Address 0x06a controls the lane0 read deskew timing. (For Timing Group 2) Register Block Offset Address 0x16a controls the lane1 read deskew timing. (For Timing Group 2) Register Block Offset Address 0x26a controls the lane2 read deskew timing. (For Timing Group 2) Register Block Offset Address 0x36a controls the lane3 read deskew timing. (For Timing Group 2) Register Block Offset Address 0x46a controls the lane4 read deskew timing. (For Timing Group 2) Register Block Offset Address 0x56a controls the lane5 read deskew timing. (For Timing Group 2) Register Block Offset Address 0x66a controls the lane6 read deskew timing. (For Timing Group 2) Register Block Offset Address 0x76a controls the lane7 read deskew timing. (For Timing Group 2) Register Block Offset Address 0x86a controls the laneDBI read deskew timing. (For Timing Group 2) Note that this register is not per-pstate. The same deskew (for channel-routing differences) is used for all pstates. . Note: Timing Group 2 = RANK2 for all systems except 3DS/LRDIMM.

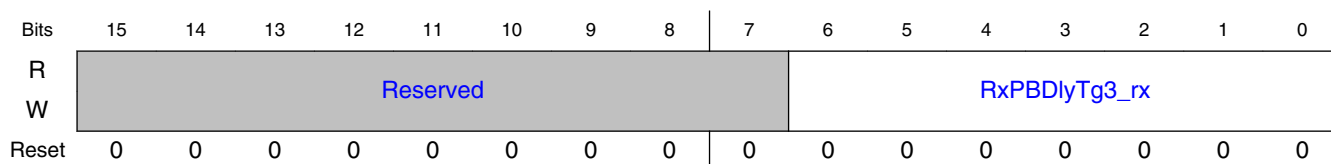
9.4.3.3.26 Read DQ per-bit BDL delay (Timing Group 3). (RxPBDlyTg3_r0 - RxPBDlyTg3_r8)

9.4.3.3.26.1 Offset

For x = 0 to 8:

Register	Offset
RxPBDlyTg3_rx	D6h + (x × 200h)

9.4.3.3.26.2 Diagram



9.4.3.3.26.3 Fields

Field	Function
15-7	Reserved
—	

Table continues on the next page...

Field	Function
6-0 RxPBDlyTg3_rx	Read DQ per-bit BDL delay (Timing Group 3). Read DQ per-bit BDL delay (Timing Group 3). Trained to deskew the read DQ data that will be sampled by the incoming per-nibble read DQS, to improve the composite eye. The unit of change is a LSB in the control of the delay circuit, and is not denominated in units of UI. Register Block Offset Address 0x06b controls the lane0 read deskew timing. (For Timing Group 3) Register Block Offset Address 0x16b controls the lane1 read deskew timing. (For Timing Group 3) Register Block Offset Address 0x26b controls the lane2 read deskew timing. (For Timing Group 3) Register Block Offset Address 0x36b controls the lane3 read deskew timing. (For Timing Group 3) Register Block Offset Address 0x46b controls the lane4 read deskew timing. (For Timing Group 3) Register Block Offset Address 0x56b controls the lane5 read deskew timing. (For Timing Group 3) Register Block Offset Address 0x66b controls the lane6 read deskew timing. (For Timing Group 3) Register Block Offset Address 0x76b controls the lane7 read deskew timing. (For Timing Group 3) Register Block Offset Address 0x86b controls the laneDBI read deskew timing. (For Timing Group 3) Note that this register is not per-pstate. The same deskew (for channel-routing differences) is used for all pstates. . Note: Timing Group 3 = RANK3 for all systems except 3DS/LRDIMM.

9.4.3.3.27 Trained Receive Enable Delay (For Timing Group 0) (RxEnDlyTg0_u0_p0 - RxEnDlyTg0_u1_p3)

9.4.3.3.27.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
RxEnDlyTg0_un_px	$100h + (x \times 20_0000h) + (n \times 200h)$

9.4.3.3.27.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved						RxEnDlyTg0_un_px									
W																
Reset	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0

9.4.3.3.27.3 Fields

Field	Function
15-11 —	Reserved
10-0 RxEnDlyTg0_un_px	Trained Receive Enable Delay (For Timing Group 0) Trained to set the delay from the memory-read command to the signal enabling the read DQS to generate read-data strobes. Trained Receive Enable Delay (For Timing Group 0) Trained to set the delay from the memory-read command to the signal enabling the read DQS to generate read-data strobes. The trained read DQS enable asserts during the read DQS preamble and deasserts at the end of the read burst. RxEnDlyTg0[10:6] is the coarse delay, ie one unit of delay is 1 UI. RxEnDlyTg0[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. RxEnDlyTg0[5] is reserved. Register Block Offset Address 0x080 controls the lower DQ nibble DqsEn. (For Timing Group 0) Register Block Offset Address 0x180 controls the upper DQ nibble DqsEn. (For Timing Group 0) These Registers are replicated per pstate. . Note: Timing Group 0 = RANK0 for all systems except 3DS/LRDIMM.

9.4.3.3.28 Trained Receive Enable Delay (For Timing Group 1) (RxEnDlyTg1_u0_p0 - RxEnDlyTg1_u1_p3)

9.4.3.3.28.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
RxEnDlyTg1_un_px	102h + (x × 20_0000h) + (n × 200h)

9.4.3.3.28.2 Diagram



9.4.3.3.28.3 Fields

Field	Function
15-11 —	Reserved
10-0 RxEnDlyTg1_un_px	Trained Receive Enable Delay (For Timing Group 1) Trained to set the delay from the memory-read command to the signal enabling the read DQS to generate read-data strobes. Trained Receive Enable Delay (For Timing Group 1) Trained to set the delay from the memory-read command to the signal enabling the read DQS to generate read-data strobes. The trained read DQS enable asserts during the read DQS preamble and deasserts at the end of the read burst. RxEnDlyTg1[10:6] is the coarse delay, ie one unit of delay is 1 UI. RxEnDlyTg1[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. RxEnDlyTg1[5] is reserved. Register Block Offset Address 0x081 controls the lower DQ nibble DqsEn. (For Timing Group 1) Register Block Offset Address 0x181 controls the upper DQ nibble DqsEn. (For Timing Group 1) These Registers are replicated per pstate. . Note: Timing Group 1 = RANK1 for all systems except 3DS/LRDIMM.

9.4.3.3.29 Trained Receive Enable Delay (For Timing Group 2) (RxEnDlyTg2_u0_p0 - RxEnDlyTg2_u1_p3)

9.4.3.3.29.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
RxEnDlyTg2_un_px	104h + (x × 20_0000h) + (n × 200h)

9.4.3.3.29.2 Diagram



9.4.3.3.29.3 Fields

Field	Function
15-11 —	Reserved
10-0 RxEnDlyTg2_un_px	Trained Receive Enable Delay (For Timing Group 2) Trained to set the delay from the memory-read command to the signal enabling the read DQS to generate read-data strobes. Trained Receive Enable Delay (For Timing Group 2) Trained to set the delay from the memory-read command to the signal enabling the read DQS to generate read-data strobes. The trained read DQS enable asserts during the read DQS preamble and deasserts at the end of the read burst. RxEnDlyTg2[10:6] is the coarse delay, ie one unit of delay is 1 UI. RxEnDlyTg2[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. RxEnDlyTg2[5] is reserved. Register Block Offset Address 0x082 controls the lower DQ nibble DqsEn. (For Timing Group 2) Register Block Offset Address 0x182 controls the upper DQ nibble DqsEn. (For Timing Group 2) These Registers are replicated per pstate. . Note: Timing Group 2 = RANK2 for all systems except 3DS/LRDIMM.

9.4.3.3.30 Trained Receive Enable Delay (For Timing Group 3) (RxEnDlyTg3_u0_p0 - RxEnDlyTg3_u1_p3)

9.4.3.3.30.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
RxEnDlyTg3_un_px	106h + (x × 20_0000h) + (n × 200h)

9.4.3.3.30.2 Diagram



9.4.3.3.30.3 Fields

Field	Function
15-11 —	Reserved
10-0 RxEnDlyTg3_un _px	Trained Receive Enable Delay (For Timing Group 3) Trained to set the delay from the memory-read command to the signal enabling the read DQS to generate read-data strobes. Trained Receive Enable Delay (For Timing Group 3) Trained to set the delay from the memory-read command to the signal enabling the read DQS to generate read-data strobes. The trained read DQS enable asserts during the read DQS preamble and deasserts at the end of the read burst. RxEnDlyTg3[10:6] is the coarse delay, ie one unit of delay is 1 UI. RxEnDlyTg3[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. RxEnDlyTg3[5] is reserved. Register Block Offset Address 0x083 controls the lower DQ nibble DqsEn. (For Timing Group 3) Register Block Offset Address 0x183 controls the upper DQ nibble DqsEn. (For Timing Group 3) These Registers are replicated per pstate. . Note: Timing Group 3 = RANK3 for all systems except 3DS/LRDIMM.

9.4.3.3.31 Trained Read DQS to RxClk Delay (Timing Group DEST=0). (RxClkDlyTg0_u0_p0 - RxClkDlyTg0_u1_p3)

9.4.3.3.31.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
RxClkDlyTg0_un_px	118h + (x × 20_0000h) + (n × 200h)

9.4.3.3.31.2 Diagram



9.4.3.3.31.3 Fields

Field	Function
15-6 —	Reserved
5-0 RxClkDlyTg0_un_px	Trained Read DQS to RxClk Delay (Timing Group DEST=0). Trained Read DQS to RxClk Delay (Timing Group DEST=0). Trained to generate read DQ receive clock from the received read DQS. RxClk Delay (6 bits, fine delay only); (For Timing Group 0) RxClkDlyTg0[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. RxClkDlyTg0[5] is reserved. There is no coarse delay field, ie the max configurable delay is 31/32 UI. Register Block Offset Address 0x08c controls the lower DQ nibble RxClk timing. (For Timing Group 0) Register Block Offset Address 0x18c controls the upper DQ nibble RxClk timing. (For Timing Group 0) These Registers are replicated per pstate. . Note: Timing Group 0 = RANK0 for all systems except 3DS/LRDIMM.

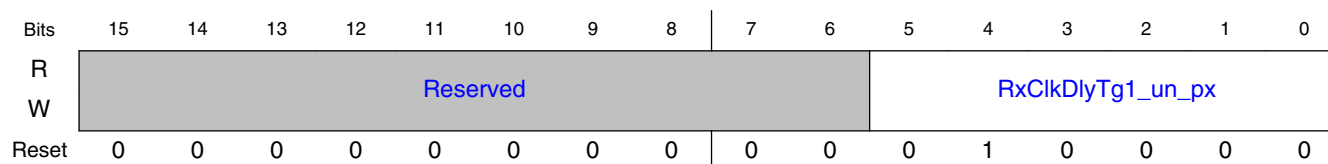
9.4.3.3.32 Trained Read DQS to RxClk Delay (Timing Group DEST=1). (RxClkDlyTg1_u0_p0 - RxClkDlyTg1_u1_p3)

9.4.3.3.32.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
RxClkDlyTg1_un_px	11Ah + (x × 20_0000h) + (n × 200h)

9.4.3.3.32.2 Diagram



9.4.3.3.32.3 Fields

Field	Function
15-6	Reserved

Table continues on the next page...

Field	Function
—	
5-0 RxClkDlyTg1_un_px	Trained Read DQS to RxClk Delay (Timing Group DEST=1). Trained Read DQS to RxClk Delay (Timing Group DEST=1). Trained to generate read DQ receive clock from the received read DQS. RxClk Delay (6 bits, fine delay only); (For Timing Group 1) RxClkDlyTg1[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. RxClkDlyTg1[5] is reserved. There is no coarse delay field, ie the max configurable delay is 31/32 UI. Register Block Offset Address 0x08d controls the lower DQ nibble RxClk timing. (For Timing Group 1) Register Block Offset Address 0x18d controls the upper DQ nibble RxClk timing. (For Timing Group 1) These Registers are replicated per pstate. . Note: Timing Group 1 = RANK1 for all systems except 3DS/LRDIMM.

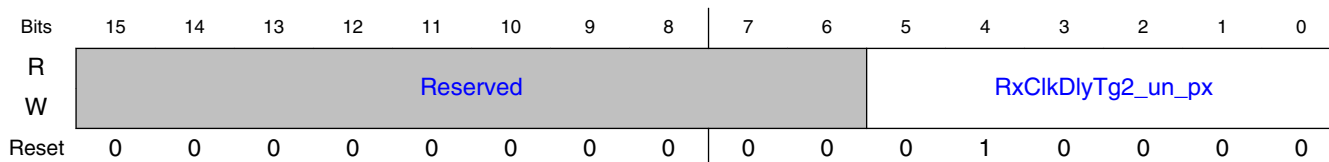
9.4.3.3.33 Trained Read DQS to RxClk Delay (Timing Group DEST=2). (RxClkDlyTg2_u0_p0 - RxClkDlyTg2_u1_p3)

9.4.3.3.33.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
RxClkDlyTg2_un_px	11Ch + (x × 20_0000h) + (n × 200h)

9.4.3.3.33.2 Diagram



9.4.3.3.33.3 Fields

Field	Function
15-6	Reserved
—	
5-0	Trained Read DQS to RxClk Delay (Timing Group DEST=2).

Field	Function
RxCIkDlyTg2_un_px	Trained Read DQS to RxClk Delay (Timing Group DEST=2). Trained to generate read DQ receive clock from the received read DQS. RxCIk Delay (6 bits, fine delay only); (For Timing Group 2) RxCIkDlyTg2[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. RxCIkDlyTg2[5] is reserved. There is no coarse delay field, ie the max configurable delay is 31/32 UI. Register Block Offset Address 0x08e controls the lower DQ nibble RxClk timing. (For Timing Group 2) Register Block Offset Address 0x18e controls the upper DQ nibble RxClk timing. (For Timing Group 2) These Registers are replicated per pstate. . Note: Timing Group 2 = RANK2 for all systems except 3DS/LRDIMM.

9.4.3.3.34 Trained Read DQS to RxClk Delay (Timing Group DEST=3). (RxCIkDlyTg3_u0_p0 - RxCIkDlyTg3_u1_p3)

9.4.3.3.34.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
RxCIkDlyTg3_un_px	11Eh + (x × 20_0000h) + (n × 200h)

9.4.3.3.34.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved										RxCIkDlyTg3_un_px					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0

9.4.3.3.34.3 Fields

Field	Function
15-6 —	Reserved
5-0 RxCIkDlyTg3_un_px	Trained Read DQS to RxClk Delay (Timing Group DEST=3). Trained Read DQS to RxClk Delay (Timing Group DEST=3). Trained to generate read DQ receive clock from the received read DQS.

Field	Function
	RxCkDelay (6 bits, fine delay only); (For Timing Group 3) RxCkDlyTg3[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. RxCkDlyTg3[5] is reserved. There is no coarse delay field, ie the max configurable delay is 31/32 UI. Register Block Offset Address 0x08f controls the lower DQ nibble RxCk timing. (For Timing Group 3) Register Block Offset Address 0x18f controls the upper DQ nibble RxCk timing. (For Timing Group 3) These Registers are replicated per pstate. . Note: Timing Group 3 = RANK3 for all systems except 3DS/LRDIMM.

9.4.3.3.35 Maps Phy DQ lane to memory DQ0 (Dq0LnSel - Dq7LnSel)

9.4.3.3.35.1 Offset

For n = 0 to 7:

Register	Offset
DqnLnSel	140h + (n × 2h)

9.4.3.3.35.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												DqLnSel			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.4.3.3.35.3 Fields

Field	Function
15-3 —	Reserved
2-0 DqLnSel	Supports mapping of PHY dq to dram dq within a byte (swizzle). Supports mapping of PHY dq to dram dq within a byte (swizzle). Intended to undo the swizzle of board-level phy-to-dram connections. such that MRR operations of binary counters may be return correct values. Note that this register is per-dbyte such that the sizzle may be different per dbyte. Each register in a byte's set of DqLnSel must have a unique value within the set.

Field	Function
	For example, if, on this dbyte, PHY lane 3 is connected to memory dq0 (on this dbyte), then Register Dq0LnSel for this dbyte should be 3.

9.4.3.3.36 Write DQ Delay (Timing Group 0). (TxDqDlyTg0_r0_p0 - TxDqDlyTg0_r8_p3)

9.4.3.3.36.1 Offset

For n = 0 to 8; x = 0 to 3:

Register	Offset
TxDqDlyTg0_rn_px	180h + (x × 20_0000h) + (n × 200h)

9.4.3.3.36.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								TxDqDlyTg0_rn_px							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

9.4.3.3.36.3 Fields

Field	Function
15-9 —	Reserved
8-0 TxDqDlyTg0_rn_px	Write DQ Delay (Timing Group 0). Write DQ Delay (Timing Group 0). Trained to center the delay between the write DQ to the write DQS. Tx DQ Delay (9 bits, includes 3b coarse and 6b fine delays); (For Timing Group 0) TxDqDlyTg0[8:6] is the coarse delay, ie one unit of delay is 1 UI. TxDqDlyTg0[4:0] is the fine (fractional UI) delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. TxDqDlyTg0[5] is reserved. Register Block Offset Address 0x0C0 controls the lane0 write DQ timing. (For Timing Group 0) Register Block Offset Address 0x1C0 controls the lane1 write DQ timing. (For Timing Group 0) Register Block Offset Address 0x2C0 controls the lane2 write DQ timing. (For Timing Group 0) Register Block Offset Address 0x3C0 controls the lane3 write DQ timing. (For Timing Group 0)

Field	Function
	Register Block Offset Address 0x4C0 controls the lane4 write DQ timing. (For Timing Group 0) Register Block Offset Address 0x5C0 controls the lane5 write DQ timing. (For Timing Group 0) Register Block Offset Address 0x6C0 controls the lane6 write DQ timing. (For Timing Group 0) Register Block Offset Address 0x7C0 controls the lane7 write DQ timing. (For Timing Group 0) Register Block Offset Address 0x8C0 controls the laneDMDBI write DQ timing. (For Timing Group 0) These Registers are replicated per pstate. . Note: Timing Group 0 = RANK0 for all systems except 3DS/LRDIMM.

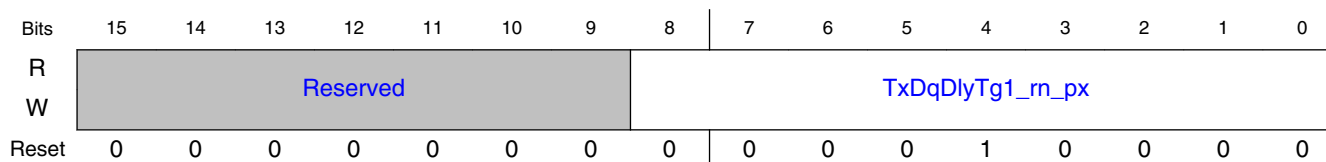
9.4.3.3.37 Write DQ Delay (Timing Group 1). (TxDqDlyTg1_r0_p0 - TxDqDlyTg1_r8_p3)

9.4.3.3.37.1 Offset

For n = 0 to 8; x = 0 to 3:

Register	Offset
TxDqDlyTg1_rn_px	182h + (x × 20_0000h) + (n × 200h)

9.4.3.3.37.2 Diagram



9.4.3.3.37.3 Fields

Field	Function
15-9 —	Reserved
8-0 TxDqDlyTg1_rn _px	Write DQ Delay (Timing Group 1). Write DQ Delay (Timing Group 1). Trained to center the delay between the write DQ to the write DQS. Tx DQ Delay (9 bits, includes 3b coarse and 6b fine delays); (For Timing Group 1) TxDqDlyTg1[8:6] is the coarse delay, ie one unit of delay is 1 UI.

Field	Function
	TxDqDlyTg1[4:0] is the fine (fractional UI) delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. TxDqDlyTg1[5] is reserved. Register Block Offset Address 0x0C1 controls the lane0 write DQ timing. (For Timing Group 1) Register Block Offset Address 0x1C1 controls the lane1 write DQ timing. (For Timing Group 1) Register Block Offset Address 0x2C1 controls the lane2 write DQ timing. (For Timing Group 1) Register Block Offset Address 0x3C1 controls the lane3 write DQ timing. (For Timing Group 1) Register Block Offset Address 0x4C1 controls the lane4 write DQ timing. (For Timing Group 1) Register Block Offset Address 0x5C1 controls the lane5 write DQ timing. (For Timing Group 1) Register Block Offset Address 0x6C1 controls the lane6 write DQ timing. (For Timing Group 1) Register Block Offset Address 0x7C1 controls the lane7 write DQ timing. (For Timing Group 1) Register Block Offset Address 0x8C1 controls the laneDMDBI write DQ timing. (For Timing Group 1) These Registers are replicated per pstate. . Note: Timing Group 1 = RANK1 for all systems except 3DS/LRDIMM.

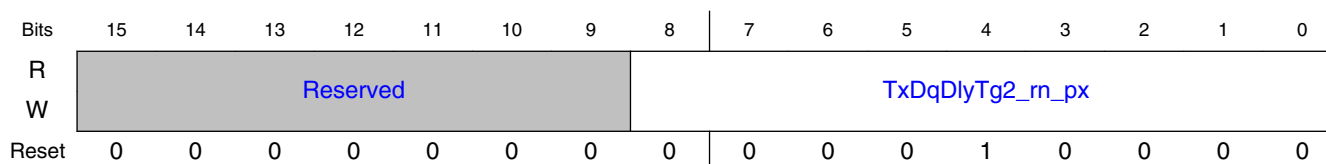
9.4.3.3.38 Write DQ Delay (Timing Group 2). (TxDqDlyTg2_r0_p0 - TxDqDlyTg2_r8_p3)

9.4.3.3.38.1 Offset

For n = 0 to 8; x = 0 to 3:

Register	Offset
TxDqDlyTg2_rn_px	$184h + (x \times 20_0000h) + (n \times 200h)$

9.4.3.3.38.2 Diagram



9.4.3.3.38.3 Fields

Field	Function
15-9	Reserved

Table continues on the next page...

Field	Function
—	
8-0 TxDqDlyTg2_rn_px	Write DQ Delay (Timing Group 2). Write DQ Delay (Timing Group 2). Trained to center the delay between the write DQ to the write DQS. Tx DQ Delay (9 bits, includes 3b coarse and 6b fine delays); (For Timing Group 2) TxDqDlyTg2[8:6] is the coarse delay, ie one unit of delay is 1 UI. TxDqDlyTg2[4:0] is the fine (fractional UI) delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. TxDqDlyTg2[5] is reserved. Register Block Offset Address 0x0C2 controls the lane0 write DQ timing. (For Timing Group 2) Register Block Offset Address 0x1C2 controls the lane1 write DQ timing. (For Timing Group 2) Register Block Offset Address 0x2C2 controls the lane2 write DQ timing. (For Timing Group 2) Register Block Offset Address 0x3C2 controls the lane3 write DQ timing. (For Timing Group 2) Register Block Offset Address 0x4C2 controls the lane4 write DQ timing. (For Timing Group 2) Register Block Offset Address 0x5C2 controls the lane5 write DQ timing. (For Timing Group 2) Register Block Offset Address 0x6C2 controls the lane6 write DQ timing. (For Timing Group 2) Register Block Offset Address 0x7C2 controls the lane7 write DQ timing. (For Timing Group 2) Register Block Offset Address 0x8C2 controls the laneDMDBI write DQ timing. (For Timing Group 2) These Registers are replicated per pstate. . Note: Timing Group 2 = RANK2 for all systems except 3DS/LRDIMM.

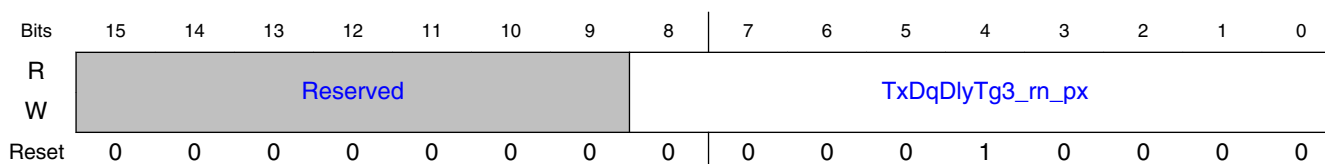
9.4.3.3.39 Write DQ Delay (Timing Group 3). (TxDqDlyTg3_r0_p0 - TxDqDlyTg3_r8_p3)

9.4.3.3.39.1 Offset

For n = 0 to 8; x = 0 to 3:

Register	Offset
TxDqDlyTg3_rn_px	$186h + (x \times 20_0000h) + (n \times 200h)$

9.4.3.3.39.2 Diagram



9.4.3.3.39.3 Fields

Field	Function
15-9 —	Reserved
8-0 TxDqDlyTg3_rn _px	Write DQ Delay (Timing Group 3). Write DQ Delay (Timing Group 3). Trained to center the delay between the write DQ to the write DQS. Tx DQ Delay (9 bits, includes 3b coarse and 6b fine delays); (For Timing Group 3) TxDqDlyTg3[8:6] is the coarse delay, ie one unit of delay is 1 UI. TxDqDlyTg3[4:0] is the fine (fractional UI) delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. TxDqDlyTg3[5] is reserved. Register Block Offset Address 0x0C3 controls the lane0 write DQ timing. (For Timing Group 3) Register Block Offset Address 0x1C3 controls the lane1 write DQ timing. (For Timing Group 3) Register Block Offset Address 0x2C3 controls the lane2 write DQ timing. (For Timing Group 3) Register Block Offset Address 0x3C3 controls the lane3 write DQ timing. (For Timing Group 3) Register Block Offset Address 0x4C3 controls the lane4 write DQ timing. (For Timing Group 3) Register Block Offset Address 0x5C3 controls the lane5 write DQ timing. (For Timing Group 3) Register Block Offset Address 0x6C3 controls the lane6 write DQ timing. (For Timing Group 3) Register Block Offset Address 0x7C3 controls the lane7 write DQ timing. (For Timing Group 3) Register Block Offset Address 0x8C3 controls the laneDMDBI write DQ timing. (For Timing Group 3) These Registers are replicated per pstate. . Note: Timing Group 3 = RANK3 for all systems except 3DS/LRDIMM.

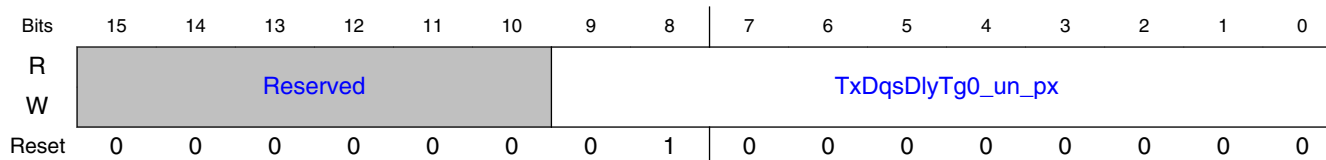
9.4.3.3.40 Write DQS Delay (Timing Group DEST=0). (TxDqsDlyTg0_u0_p0 - TxDqsDlyTg0_u1_p3)

9.4.3.3.40.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
TxDqsDlyTg0_un_px	1A0h + (x × 20_0000h) + (n × 200h)

9.4.3.3.40.2 Diagram



9.4.3.3.40.3 Fields

Field	Function
15-10 —	Reserved
9-0 TxDqsDlyTg0_un_px	Write DQS Delay (Timing Group DEST=0). Write DQS Delay (Timing Group DEST=0). Trained to set the delay from the memory-write command to the signal driving the write DQS Tx DQS Delay (10 bits, includes coarse and fine delays); (For Timing Group 0) TxDqsDlyTg0[9:6] is the coarse delay, ie one unit of delay is 1 UI. TxDqsDlyTg0[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. TxDqsDlyTg0[5] is reserved. Register is per nibble, per timing-group, per pstate. Register Block Offset Address 0x0D0 controls the lower DQ nibble write DQS timing. (For Timing Group 0) Register Block Offset Address 0x1D0 controls the upper DQ nibble write DQS timing. (For Timing Group 0) . Note: Timing Group 0 = RANK0 for all systems except 3DS/LRDIMM.

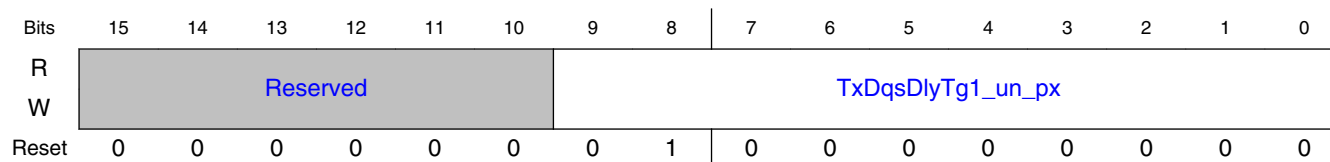
9.4.3.3.41 Write DQS Delay (Timing Group DEST=1). (TxDqsDlyTg1_u0_p0 - TxDqsDlyTg1_u1_p3)

9.4.3.3.41.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
TxDqsDlyTg1_un_px	1A2h + (x × 20_0000h) + (n × 200h)

9.4.3.3.41.2 Diagram



9.4.3.3.41.3 Fields

Field	Function
15-10 —	Reserved
9-0 TxDqsDlyTg1_un_px	Write DQS Delay (Timing Group DEST=1). Write DQS Delay (Timing Group DEST=1). Trained to set the delay from the memory-write command to the signal driving the write DQS Tx DQS Delay (10 bits, includes coarse and fine delays); (For Timing Group 1) TxDqsDlyTg1[9:6] is the coarse delay, ie one unit of delay is 1 UI. TxDqsDlyTg1[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. TxDqsDlyTg1[5] is reserved. Register is per nibble, per timing-group, per pstate. Register Block Offset Address 0x0D1 controls the lower DQ nibble write DQS timing. (For Timing Group 1) Register Block Offset Address 0x1D1 controls the upper DQ nibble write DQS timing. (For Timing Group 1) . Note: Timing Group 1 = RANK1 for all systems except 3DS/LRDIMM.

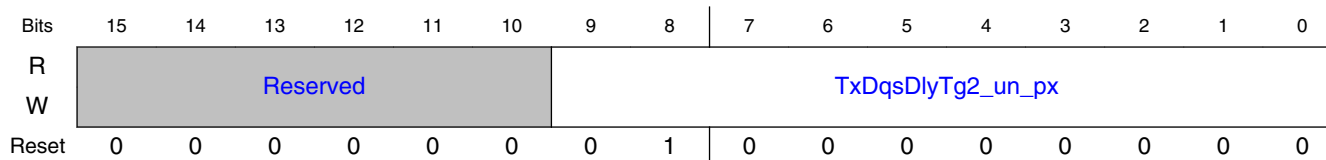
9.4.3.3.42 Write DQS Delay (Timing Group DEST=2). (TxDqsDlyTg2_u0_p0 - TxDqsDlyTg2_u1_p3)

9.4.3.3.42.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
TxDqsDlyTg2_un_px	$1A4h + (x \times 20_0000h) + (n \times 200h)$

9.4.3.3.42.2 Diagram



9.4.3.3.42.3 Fields

Field	Function
15-10 —	Reserved
9-0 TxDqsDlyTg2_un_px	Write DQS Delay (Timing Group DEST=2). Write DQS Delay (Timing Group DEST=2). Trained to set the delay from the memory-write command to the signal driving the write DQS Tx DQS Delay (10 bits, includes coarse and fine delays); (For Timing Group 2) TxDqsDlyTg2[9:6] is the coarse delay, ie one unit of delay is 1 UI. TxDqsDlyTg2[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. TxDqsDlyTg2[5] is reserved. Register is per nibble, per timing-group, per pstate. Register Block Offset Address 0x0D2 controls the lower DQ nibble write DQS timing. (For Timing Group 2) Register Block Offset Address 0x1D2 controls the upper DQ nibble write DQS timing. (For Timing Group 2) . Note: Timing Group 2 = RANK2 for all systems except 3DS/LRDIMM.

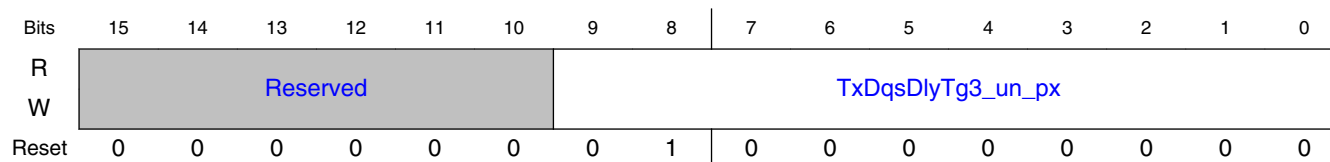
9.4.3.3.43 Write DQS Delay (Timing Group DEST=3). (TxDqsDlyTg3_u0_p0 - TxDqsDlyTg3_u1_p3)

9.4.3.3.43.1 Offset

For n = 0 to 1; x = 0 to 3:

Register	Offset
TxDqsDlyTg3_un_px	1A6h + (x × 20_0000h) + (n × 200h)

9.4.3.3.43.2 Diagram



9.4.3.3.43.3 Fields

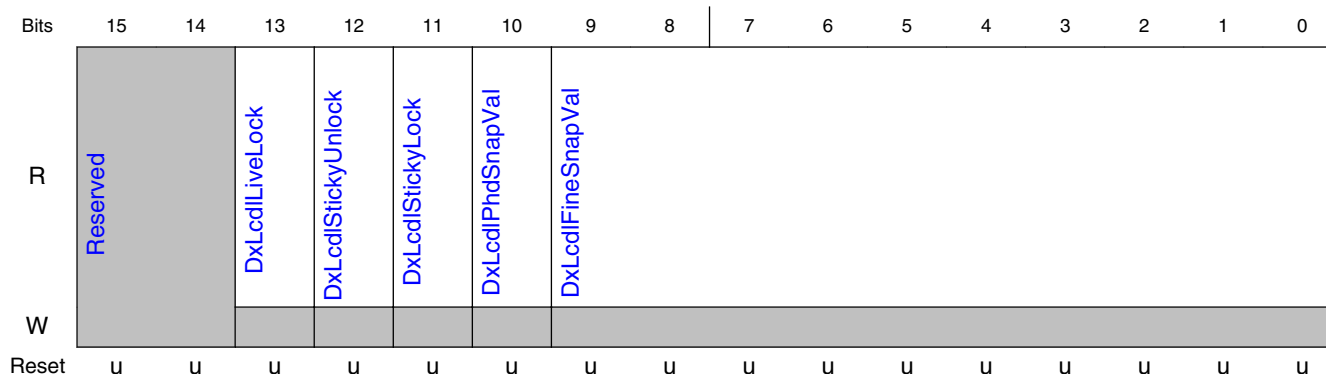
Field	Function
15-10 —	Reserved
9-0 TxDqsDlyTg3_un_px	Write DQS Delay (Timing Group DEST=3). Write DQS Delay (Timing Group DEST=3). Trained to set the delay from the memory-write command to the signal driving the write DQS Tx DQS Delay (10 bits, includes coarse and fine delays); (For Timing Group 3) TxDqsDlyTg3[9:6] is the coarse delay, ie one unit of delay is 1 UI. TxDqsDlyTg3[4:0] is the fine delay, ie one unit of delay is one-thirtysecond of a UI = UI/32. TxDqsDlyTg3[5] is reserved. Register is per nibble, per timing-group, per pstate. Register Block Offset Address 0x0D3 controls the lower DQ nibble write DQS timing. (For Timing Group 3) Register Block Offset Address 0x1D3 controls the upper DQ nibble write DQS timing. (For Timing Group 3) . Note: Timing Group 3 = RANK3 for all systems except 3DS/LRDIMM.

9.4.3.3.44 Debug status of the DBYTE LCDL (DxLcdlStatus)

9.4.3.3.44.1 Offset

Register	Offset
DxLcdlStatus	1C8h

9.4.3.3.44.2 Diagram



9.4.3.3.44.3 Fields

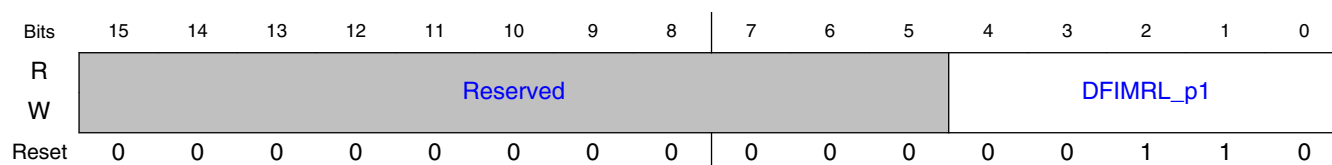
Field	Function
15-14 —	Reserved
13 DxLcdlLiveLock	present value of whether the LCDL is locked, valid when LcdITstEnable=1.
12 DxLcdlStickyUnl ock	latched value of whether the LCDL ever lost lock after the assertion of LcdITstEnable.
11 DxLcdlStickyLoc k	latched value of whether the LCDL ever achieved lock after the assertion of LcdITstEnable.
10 DxLcdlPhdSnap Val	Value of the LCDL phase-detector output, latched by pulse on csr LcdlFineSnap. Value of the LCDL phase-detector output, latched by pulse on csr LcdlFineSnap. while csr LcdITstEnable=1.
9-0 DxLcdlFineSnap Val	Value of the LCDL 1UI estimate code, latched by pulse on csr LcdlFineSnap while csr LcdITstEnable=1. Value of the LCDL 1UI estimate code, latched by pulse on csr LcdlFineSnap while csr LcdITstEnable=1. Index 9 is reserved for growth.

9.4.3.3.45 DFI MaxReadLatency (DFIMRL_p1)

9.4.3.3.45.1 Offset

Register	Offset
DFIMRL_p1	20_0040h

9.4.3.3.45.2 Diagram



9.4.3.3.45.3 Fields

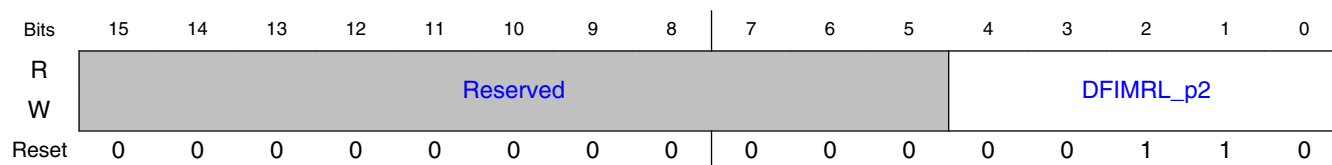
Field	Function
15-5 —	Reserved
4-0 DFIMRL_p1	This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. This is the value, in units of two mem clocks, between dfi_rddata_en and dfi_rddata_valid A unit change in the LSB is a change in MRL of two mem clocks.

9.4.3.3.46 DFI MaxReadLatency (DFIMRL_p2)

9.4.3.3.46.1 Offset

Register	Offset
DFIMRL_p2	40_0040h

9.4.3.3.46.2 Diagram



9.4.3.3.46.3 Fields

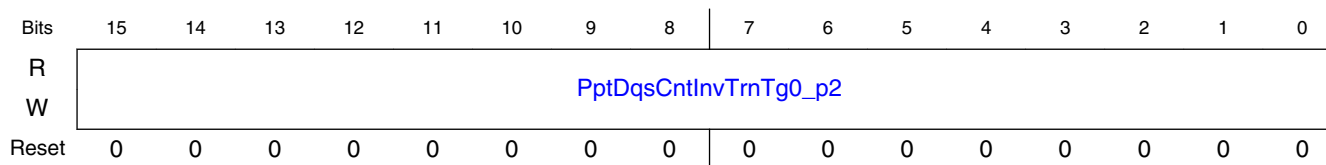
Field	Function
15-5 —	Reserved
4-0 DFIMRL_p2	This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. This is the value, in units of two mem clocks, between dfi_rddata_en and dfi_rddata_valid A unit change in the LSB is a change in MRL of two mem clocks.

9.4.3.3.47 DQS Oscillator Count inverse at time of training in LPDDR4 drift compensation (PptDqsCntInvTrnTg0_p2)

9.4.3.3.47.1 Offset

Register	Offset
PptDqsCntInvTrnTg0_p2	40_015Ch

9.4.3.3.47.2 Diagram



9.4.3.3.47.3 Fields

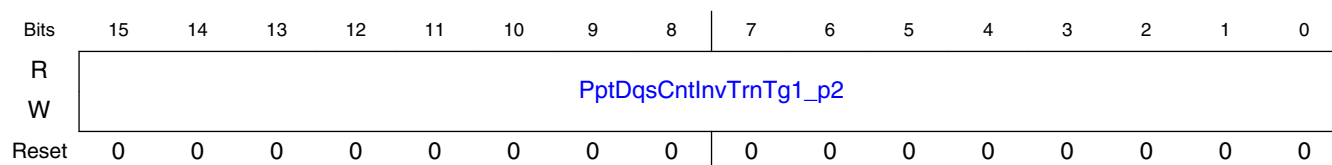
Field	Function
15-0 PptDqsCntInvTrnTg0_p2	Programmed by PHY training firmware to support LPDDR3/LPDDR4 DRAM drift compensation.

9.4.3.3.48 DQS Oscillator Count inverse at time of training in LPDDR4 drift compensation (PptDqsCntInvTrnTg1_p2)

9.4.3.3.48.1 Offset

Register	Offset
PptDqsCntInvTrnTg1_p2	40_015Eh

9.4.3.3.48.2 Diagram



9.4.3.3.48.3 Fields

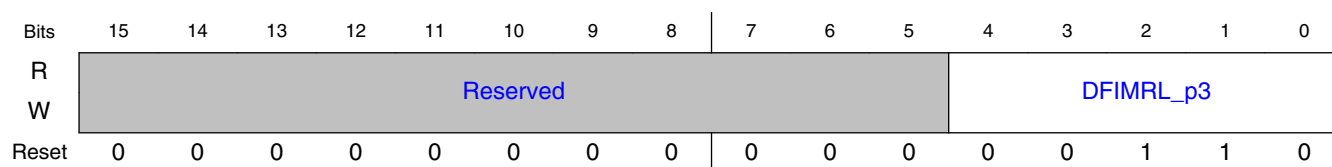
Field	Function
15-0 PptDqsCntInvTrnTg1_p2	Programmed by PHY training firmware to support LPDDR3/LPDDR4 DRAM drift compensation.

9.4.3.3.49 DFI MaxReadLatency (DFIMRL_p3)

9.4.3.3.49.1 Offset

Register	Offset
DFIMRL_p3	60_0040h

9.4.3.3.49.2 Diagram



9.4.3.3.49.3 Fields

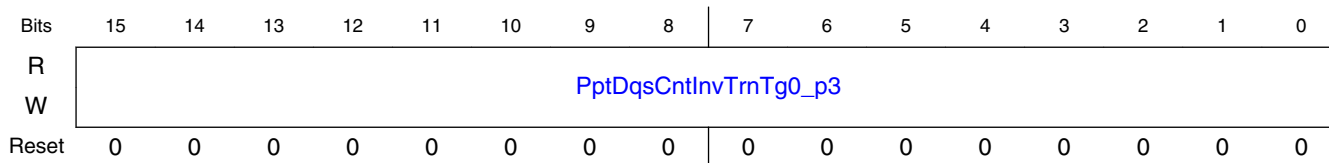
Field	Function
15-5 —	Reserved
4-0 DFIMRL_p3	This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. This is the value, in units of two mem clocks, between dfi_rddata_en and dfi_rddata_valid A unit change in the LSB is a change in MRL of two mem clocks.

9.4.3.3.50 DQS Oscillator Count inverse at time of training in LPDDR4 drift compensation (PptDqsCntInvTrnTg0_p3)

9.4.3.3.50.1 Offset

Register	Offset
PptDqsCntInvTrnTg0_p3	60_015Ch

9.4.3.3.50.2 Diagram



9.4.3.3.50.3 Fields

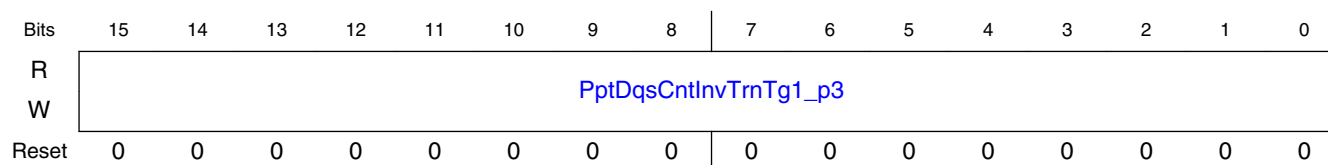
Field	Function
15-0 PptDqsCntInvTrnTg0_p3	Programmed by PHY training firmware to support LPDDR3/LPDDR4 DRAM drift compensation.

9.4.3.3.51 DQS Oscillator Count inverse at time of training in LPDDR4 drift compensation (PptDqsCntInvTrnTg1_p3)

9.4.3.3.51.1 Offset

Register	Offset
PptDqsCntInvTrnTg1_p3	60_015Eh

9.4.3.3.51.2 Diagram



9.4.3.3.51.3 Fields

Field	Function
15-0 PptDqsCntInvTrnTg1_p3	Programmed by PHY training firmware to support LPDDR3/LPDDR4 DRAM drift compensation.

9.4.3.4 DWC_DDRPHYA_DRTUB register descriptions

9.4.3.4.1 DWC_DDRPHYA_DRTUB Memory map

DDR4/3 PHY address block

DWC_DDRPHYA_DRTUB0 base address: C_0000h

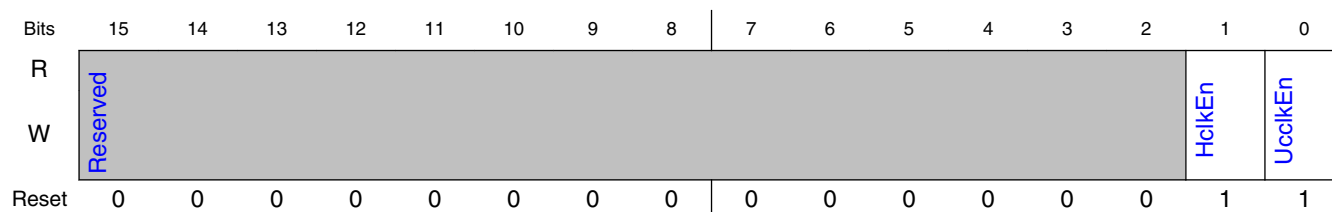
Offset	Register	Width (In bits)	Access	Reset value
100h	Ucclk and Hclk enables (UcclkHclkEnables)	16	RW	0003h
102h	PIE current Pstate value (CurPstate0b)	16	RW	0000h
1DAh	Customer settable by the customer (CUSTPUBREV)	16	RO	Table 9-4
1DCh	The hardware version of this PUB, excluding the PHY (PUBREV)	16	RO	1160h

9.4.3.4.2 Ucclk and Hclk enables (UcclkHclkEnables)

9.4.3.4.2.1 Offset

Register	Offset
UcclkHclkEnables	100h

9.4.3.4.2.2 Diagram



9.4.3.4.2.3 Fields

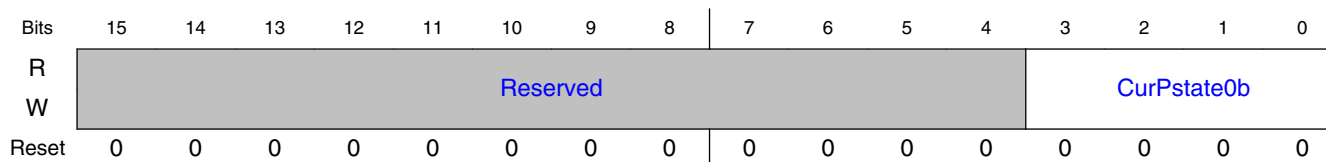
Field	Function
15-2 —	Reserved
1 HclkEn	When training has completed (and assuming no further need for the training hardware), the enable should be set to 0 to reduce power.
0 UcclkEn	When training has completed (and assuming no further need for the microcontroller), the enable should be set to 0 to reduce power.

9.4.3.4.3 PIE current Pstate value (CurPstate0b)

9.4.3.4.3.1 Offset

Register	Offset
CurPstate0b	102h

9.4.3.4.3.2 Diagram



9.4.3.4.3.3 Fields

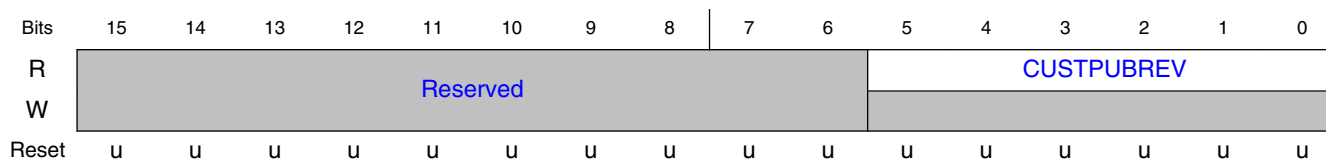
Field	Function
15-4 —	Reserved
3-0 CurPstate0b	PIE current Pstate value This register is used to select values for writing by the Pstate sequencer and is written in the beginning of the Pstate switch. PIE current Pstate value This register is used to select values for writing by the Pstate sequencer and is written in the beginning of the Pstate switch. It can also be used to reference which Pstate the sequencer thinks it's in.

9.4.3.4.4 Customer settable by the customer (CUSTPUBREV)

9.4.3.4.4.1 Offset

Register	Offset
CUSTPUBREV	1DAh

9.4.3.4.4.2 Diagram



9.4.3.4.4.3 Fields

Field	Function
15-6	Reserved

Table continues on the next page...

DDR PHY (DDR_PHY)

Field	Function
—	
5-0 CUSTPUBREV	The customer settable PUB version number. The customer settable PUB version number. The value is derived from the `define DWC_DDRPHY_CUST_PUBREV

9.4.3.4.5 The hardware version of this PUB, excluding the PHY (PUBREV)

9.4.3.4.5.1 Offset

Register	Offset
PUBREV	1DCh

9.4.3.4.5.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PUBMJR								PUBMDR				PUBMNR			
W																
Reset	0	0	0	1	0	0	0	1	0	1	1	0	0	0	0	0

9.4.3.4.5.3 Fields

Field	Function
15-8 PUBMJR	Indicates major revision of the PUB.
7-4 PUBMDR	Indicates moderate revision of the PUB.
3-0 PUBMNR	Indicates minor update of the PUB.

9.4.3.5 DWC_DDRPHYA_INITENG register descriptions

9.4.3.5.1 DWC_DDRPHYA_INITENG Memory map

DesignWare Cores DDR4/3 PHY address block

DWC_DDRPHYA_INITENG0 base address: 9_0000h

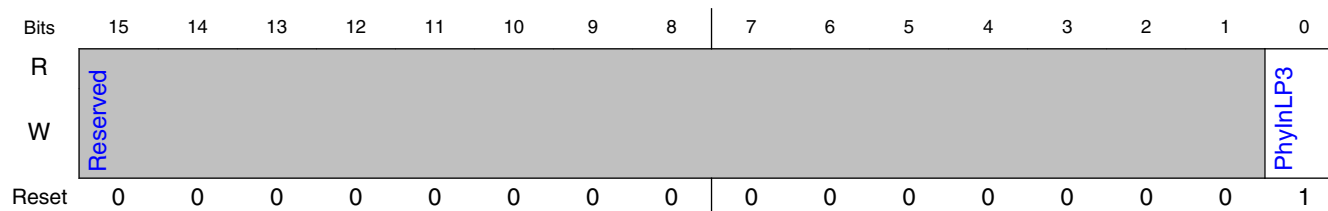
Offset	Register	Width (In bits)	Access	Reset value
50h	Indicator for PIE Lower Power 3 (LP3) Status (PhyInLP3)	16	RW	0001h

9.4.3.5.2 Indicator for PIE Lower Power 3 (LP3) Status (PhyInLP3)

9.4.3.5.2.1 Offset

Register	Offset
PhyInLP3	50h

9.4.3.5.2.2 Diagram



9.4.3.5.2.3 Fields

Field	Function
15-1 —	Reserved
0 PhyInLP3	Read Only. Read Only. Set to 1 by the PIE once completed LP3 Entry sequence; Cleared during LP3 Exit sequence. System software can read this csr for the status of PIE engine.

9.4.3.6 DWC_DDRPHYA_MASTER register descriptions

9.4.3.6.1 DWC_DDRPHYA_MASTER Memory map

DDR4/3 PHY address block

DWC_DDRPHYA_MASTER0 base address: 2_0000h

Offset	Register	Width (In bits)	Access	Reset value
0h	Rx FIFO pointer initialization control (RxFifoInit)	16	RW	0000h
2h	Clock gating control (ForceClkDisable)	16	RW	0000h
6h	This Register used by Training Firmware to force an internal PHY Update Event. (ForceInternalUpdate)	16	RW	0000h
8h	Read Only displays PHY Configuration. (PhyConfig)	16	RO	Table 9-4
Eh	Test Bump Control1 (TestBumpCntrl1)	16	RW	0B03h
10h	Impedance Calibration Clock Ratio (CalUclkInfo_p0)	16	RW	0320h
14h	Test Bump Control (TestBumpCntrl)	16	RW	0000h
16h	PHY Initialization Engine (PIE) Delay Register 0 (Seq0BDLY0_p0)	16	RW	0000h
18h	PHY Initialization Engine (PIE) Delay Register 1 (Seq0BDLY1_p0)	16	RW	0000h
1Ah	PHY Initialization Engine (PIE) Delay Register 2 (Seq0BDLY2_p0)	16	RW	0000h
1Ch	PHY Initialization Engine (PIE) Delay Register 3 (Seq0BDLY3_p0)	16	RW	0000h
1Eh	PHY Alert status bit (PhyAlertStatus)	16	RO	Table 9-4
20h	Setup Intervals for DFI PHY Master operations (PPTTrainSetup_p0)	16	RW	0000h
24h	ATestMode control (ATestMode)	16	RW	0000h
28h	TX P Impedance Calibration observation (TxCalBinP)	16	RO	Table 9-4
2Ah	TX N Impedance Calibration observation (TxCalBinN)	16	RO	Table 9-4
2Ch	TX P Impedance Calibration override (TxCalPOvr)	16	RW	0000h
2Eh	TX N Impedance Calibration override (TxCalNOvr)	16	RW	0000h
30h	Enables for update and low-power interfaces for DFI0 and DFI1 (DfiMode)	16	RW	0005h
32h	Mode select register for MEMCLK/Address/Command Tristates (TristateModeCA_p0)	16	RW	0000h
34h	Digital Observation Pin control (MtestMuxSel)	16	RW	0000h
36h	Digital Observation Pin program info for debug (MtestPgmlInfo)	16	RW	0000h
38h	Dynamic Power Up/Down control (DynPwrDnUp)	16	RW	0000h
3Ch	PHY Technology ID Register (PhyTID)	16	RW	0000h
40h	HWT MaxReadLatency. (HwtMRL_p0)	16	RW	0006h
42h	DFI PhyUpdate Request time counter (in MEMCLKs) (DFIPHYUPD)	16	RW	FF07h
44h	Controls the write DQ generation for Per-Dram-Addressing of MRS (PdaMrsWriteMode)	16	RW	0000h
46h	Controls whether dfi_geardown_en will cause CS and CKE timing to change. (DFIGEARDOWNCTL)	16	RW	0000h

Table continues on the next page...

Offset	Register	Width (In bits)	Access	Reset value
48h	Control the PHY logic related to the read and write DQS preamble (DqsPreambleControl_p0)	16	RW	0008h
4Ah	DBYTE module controls to select X4 Dram device mode (MasterX4 Config)	16	RW	0000h
4Ch	Write level feedback DQ observability select. (WrLevBits)	16	RW	0098h
4Eh	In DDR4 Mode , this controls whether CS_N[3:2] should be multicast on CID[1:0] (EnableCsMulticast)	16	RW	0000h
50h	Drives cs_n[0] onto cs_n[1] during training (HwtLpCsMultiCast)	16	RW	0000h
58h	Disable for unused ACX Nibbles (Acx4AnibDis)	16	RW	0000h
5Ah	This Register is used to enable the Read-DBI function in each DBYTE (DMIPinPresent_p0)	16	RW	0000h
5Ch	Address/Command FIFO ReadPointer Initial Value (ARdPtrInitVal_p0)	16	RW	0000h
74h	DLL Mode control CSR for DBYTEs (DbyteDllModeCntrl)	16	RW	Table 9-4
8Ah	Impedance Calibration offsets control (CalOffsets)	16	RW	0000h
8Eh	Sar Init Vals (SarInitVals)	16	RW	0127h
92h	Impedance Calibration PExt Override control (CalPExtOvr)	16	RW	0000h
94h	Impedance Calibration Cmpr 50 control (CalCmpr5Ovr)	16	RW	0000h
96h	Impedance Calibration NInt Override control (CalNIntOvr)	16	RW	0000h
A0h	Impedance Calibration driver strength control (CalDrvStr0)	16	RW	0000h
ACh	READ DATA On-Die Termination Timing Control (by PHY) (ProcOdtTimeCtl_p0)	16	RW	0006h
B6h	This Register is used to configure the MemAlert Receiver (MemAlert Control)	16	RW	4000h
B8h	This Register is used to configure the MemAlert Receiver (MemAlert Control2)	16	RW	0000h
C0h	Protection and control of BP_MemReset_L (MemResetL)	16	RW	0000h
DAh	Drive CS_N 3:0 onto CS_N 7:4 (DriveCSLowOntoHigh)	16	RW	0000h
DCh	PUBMODE - HWT Mux Select (PUBMODE)	16	RW	0000h
DEh	Misc PHY status bits (MiscPhyStatus)	16	RO	Table 9-4
E0h	Controls whether the loopback path bypasses the final PAD node. (CoreLoopbackSel)	16	RW	0000h
E2h	DLL Various Training Parameters (DllTrainParam)	16	RW	0002h
E8h	CSn Disable Bypass for LPDDR3/4 (HwtLpCsEnBypass)	16	RW	0000h
EAh	Dfi Command/Address Mode (DfiCAMode)	16	RW	0000h
F0h	DLL Lock State machine control register (DllControl)	16	RW	0003h
F2h	DLL update phase control (PulseDllUpdatePhase)	16	RW	0000h
F8h	DLL gain control (DllGainCtl_p0)	16	RW	00A1h
110h	Impedance Calibration Control (CalRate)	16	RW	0009h
112h	Impedance Calibration Zap/Reset (CalZap)	16	RW	0000h
116h	PSTATE Selection (PState)	16	RW	000Ch
11Ah	PLL Output Control (PllOutGateControl)	16	RW	0000h

Table continues on the next page...

DDR PHY (DDR_PHY)

Offset	Register	Width (In bits)	Access	Reset value
120h	PMU Power-on Reset Control (PLL/DLL Lock Done) (PorControl)	16	RW	0000h
12Eh	Impedance Calibration Busy Status (CalBusy)	16	RO	Table 9-4
130h	Miscellaneous impedance calibration controls. (CalMisc2)	16	RW	0004h
134h	Controls for disabling the impedance calibration of certain targets. (CalMisc)	16	RW	0000h
138h	Impedance Calibration Cmpr control (CalCmpr5)	16	RO	Table 9-4
13Ah	Impedance Calibration NInt control (CalNInt)	16	RO	Table 9-4
13Ch	Impedance Calibration PExt control (CalPExt)	16	RO	Table 9-4
150h	Impedance Calibration Cmp Invert control (CalCmpInvert)	16	RW	0003h
15Ch	Impedance Calibration Cmpna control (CalCmpnaCntrl)	16	RW	0112h
160h	Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiRdDataCsDestMap_p0)	16	RW	00E4h
164h	PHY Global Vref Controls (VrefInGlobal_p0)	16	RW	0200h
168h	Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiWrDataCsDestMap_p0)	16	RW	00E4h
16Ah	Counts successful PHY Master Interface Updates (PPTs) (MasUpdGoodCtr)	16	RO	Table 9-4
16Ch	Counts successful PHY-initiated DFI0 Interface Updates (PhyUpd0GoodCtr)	16	RO	Table 9-4
16Eh	Counts successful PHY-initiated DFI1 Interface Updates (PhyUpd1GoodCtr)	16	RO	Table 9-4
170h	Counts successful Memory Controller DFI0 Interface Updates (CtlUpd0GoodCtr)	16	RO	Table 9-4
172h	Counts successful Memory Controller DFI1 Interface Updates (CtlUpd1GoodCtr)	16	RO	Table 9-4
174h	Counts unsuccessful PHY Master Interface Updates (MasUpdFailCtr)	16	RO	Table 9-4
176h	Counts unsuccessful PHY-initiated DFI0 Interface Updates (PhyUpd0FailCtr)	16	RO	Table 9-4
178h	Counts unsuccessful PHY-initiated DFI1 Interface Updates (PhyUpd1FailCtr)	16	RO	Table 9-4
17Ah	Enables for Performance Counters (PhyPerfCtrEnable)	16	RW	0000h
186h	PLL Power Down (PIIPwrDn)	16	RW	0001h
188h	PLL Reset (PIIReset)	16	RW	0001h
18Ah	PState dependent PLL Control Register 2 (PIICtrl2_p0)	16	RW	0019h
18Ch	PLL Control Register 0 (PIICtrl0)	16	RW	2008h
18Eh	PState dependent PLL Control Register 1 (PIICtrl1_p0)	16	RW	0020h
190h	PLL Testing Control Register (PIITst)	16	RW	0000h
192h	PLL's pll_lock pin output (PIILockStatus)	16	RO	Table 9-4
194h	Additional controls for PLL CP/VCO modes of operation (PIITestMode_p0)	16	RW	0124h
196h	PLL Control Register 3 (PIICtrl3)	16	RW	01F0h
198h	PState dependent PLL Control Register 4 (PIICtrl4_p0)	16	RW	017Fh
19Ah	PLL's eoc (end of calibration) output (PIIEndofCal)	16	RO	Table 9-4

Table continues on the next page...

Offset	Register	Width (In bits)	Access	Reset value
19Ch	PLL's standby_eff (effective standby) output (PIIStandbyEff)	16	RO	Table 9-4
19Eh	PLL's Dacval_out output (PIIDacValOut)	16	RO	Table 9-4
1C6h	Controls for use in observing and testing the LCDLs. (LcdIDbgCntl)	16	RW	0000h
1C8h	Debug status of the DBYTE LCDL (AcLcdlStatus)	16	RO	Table 9-4
1DAh	Customer settable by the customer (CUSTPHYREV)	16	RO	Table 9-4
1DCh	The hardware version of this PHY, excluding the PUB (PHYREV)	16	RO	0100h
1DEh	Start vector value to be used for LP3-exit or Init PIE Sequence (LP3ExitSeq0BStartVector)	16	RW	00DEh
1E0h	DFI Frequency Translation Register 0 (DfiFreqXlat0)	16	RW	0000h
1E2h	DFI Frequency Translation Register 1 (DfiFreqXlat1)	16	RW	0000h
1E4h	DFI Frequency Translation Register 2 (DfiFreqXlat2)	16	RW	0000h
1E6h	DFI Frequency Translation Register 3 (DfiFreqXlat3)	16	RW	0000h
1E8h	DFI Frequency Translation Register 4 (DfiFreqXlat4)	16	RW	0000h
1EAh	DFI Frequency Translation Register 5 (DfiFreqXlat5)	16	RW	0000h
1ECh	DFI Frequency Translation Register 6 (DfiFreqXlat6)	16	RW	0000h
1EEh	DFI Frequency Translation Register 7 (DfiFreqXlat7)	16	RW	0000h
1F0h	TxRdPtrlInit control register (TxRdPtrlInit)	16	RW	0001h
1F2h	DFI Init Complete control (DfiInitComplete)	16	RW	0000h
1F4h	DFI Frequency Ratio (DfiFreqRatio_p0)	16	RW	0001h
1F6h	Enable more frequent consistency checks of the RX FIFOs (RxFifoChecks)	16	RW	0000h
200h	Maps PHY CAA lane 0 from dfi0_address of the index of the register contents (MapCAA0toDfi)	16	RW	0000h
202h	Maps PHY CAA lane 1 from dfi0_address of the index of the register contents (MapCAA1toDfi)	16	RW	0001h
204h	Maps PHY CAA lane 2 from dfi0_address of the index of the register contents (MapCAA2toDfi)	16	RW	0002h
206h	Maps PHY CAA lane 3 from dfi0_address of the index of the register contents (MapCAA3toDfi)	16	RW	0003h
208h	Maps PHY CAA lane 4 from dfi0_address of the index of the register contents (MapCAA4toDfi)	16	RW	0004h
20Ah	Maps PHY CAA lane 5 from dfi0_address of the index of the register contents (MapCAA5toDfi)	16	RW	0005h
20Ch	Maps PHY CAA lane 6 from dfi0_address of the index of the register contents (MapCAA6toDfi)	16	RW	0006h
20Eh	Maps PHY CAA lane 7 from dfi0_address of the index of the register contents (MapCAA7toDfi)	16	RW	0007h
210h	Maps PHY CAA lane 8 from dfi0_address of the index of the register contents (MapCAA8toDfi)	16	RW	0008h
212h	Maps PHY CAA lane 9 from dfi0_address of the index of the register contents (MapCAA9toDfi)	16	RW	0009h
220h	Maps PHY CAB lane 0 from dfi1_address of the index of the register contents (MapCAB0toDfi)	16	RW	0000h

Table continues on the next page...

DDR PHY (DDR_PHY)

Offset	Register	Width (In bits)	Access	Reset value
222h	Maps PHY CAB lane 1 from dfi1_address of the index of the register contents (MapCAB1toDfi)	16	RW	0001h
224h	Maps PHY CAB lane 2 from dfi1_address of the index of the register contents (MapCAB2toDfi)	16	RW	0002h
226h	Maps PHY CAB lane 3 from dfi1_address of the index of the register contents (MapCAB3toDfi)	16	RW	0003h
228h	Maps PHY CAB lane 4 from dfi1_address of the index of the register contents (MapCAB4toDfi)	16	RW	0004h
22Ah	Maps PHY CAB lane 5 from dfi1_address of the index of the register contents (MapCAB5toDfi)	16	RW	0005h
22Ch	Maps PHY CAB lane 6 from dfi1_address of the index of the register contents (MapCAB6toDfi)	16	RW	0006h
22Eh	Maps PHY CAB lane 7 from dfi1_address of the index of the register contents (MapCAB7toDfi)	16	RW	0007h
230h	Maps PHY CAB lane 8 from dfi1_address of the index of the register contents (MapCAB8toDfi)	16	RW	0008h
232h	Maps PHY CAB lane 9 from dfi1_address of the index of the register contents (MapCAB9toDfi)	16	RW	0009h
236h	Interrupt Enable Bits (PhyInterruptEnable)	16	RW	0000h
238h	Interrupt Firmware Control Bits (PhyInterruptFWControl)	16	RW	0000h
23Ah	Interrupt Mask Bits (PhyInterruptMask)	16	RW	0000h
23Ch	Interrupt Clear Bits (PhyInterruptClear)	16	RW	0000h
23Eh	Interrupt Status Bits (PhyInterruptStatus)	16	RO	Table 9-4
240h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress0)	16	RW	0000h
242h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress1)	16	RW	0000h
244h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress2)	16	RW	0000h
246h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress3)	16	RW	0000h
248h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress4)	16	RW	0000h
24Ah	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress5)	16	RW	0000h
24Ch	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress6)	16	RW	0000h
24Eh	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress7)	16	RW	0000h
250h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress8)	16	RW	0000h
252h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress9)	16	RW	0000h
254h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress10)	16	RW	0000h
256h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress11)	16	RW	0000h
258h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress12)	16	RW	0000h
25Ah	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress13)	16	RW	0000h
25Ch	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress14)	16	RW	0000h
25Eh	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress15)	16	RW	0000h
260h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress17)	16	RW	0000h
262h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtActN)	16	RW	0000h
264h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtBank0)	16	RW	0000h

Table continues on the next page...

Offset	Register	Width (In bits)	Access	Reset value
266h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtBank1)	16	RW	0000h
268h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtBank2)	16	RW	0000h
26Ah	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtBg0)	16	RW	0000h
26Ch	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtBg1)	16	RW	0000h
26Eh	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtCasN)	16	RW	0000h
270h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtRasN)	16	RW	0000h
272h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtWeN)	16	RW	0000h
274h	Signal swizzle selection for HWT swizzle (HwtSwizzleHwtParityIn)	16	RW	0000h
20_0010h	Impedance Calibration Clock Ratio (CalUclkInfo_p1)	16	RW	0320h
20_0016h	PHY Initialization Engine (PIE) Delay Register 0 (Seq0BDLY0_p1)	16	RW	0000h
20_0018h	PHY Initialization Engine (PIE) Delay Register 1 (Seq0BDLY1_p1)	16	RW	0000h
20_001Ah	PHY Initialization Engine (PIE) Delay Register 2 (Seq0BDLY2_p1)	16	RW	0000h
20_001Ch	PHY Initialization Engine (PIE) Delay Register 3 (Seq0BDLY3_p1)	16	RW	0000h
20_0020h	Setup Intervals for DFI PHY Master operations (PPTTrainSetup_p1)	16	RW	0000h
20_0032h	Mode select register for MEMCLK/Address/Command Tristates (Tris tateModeCA_p1)	16	RW	0000h
20_0040h	HWT MaxReadLatency. (HwtMRL_p1)	16	RW	0006h
20_0048h	Control the PHY logic related to the read and write DQS preamble (DqsPreambleControl_p1)	16	RW	0008h
20_005Ah	This Register is used to enable the Read-DBI function in each DBYTE (DMIPinPresent_p1)	16	RW	0000h
20_005Ch	Address/Command FIFO ReadPointer Initial Value (ARdPtrInitVal_ p1)	16	RW	0000h
20_00ACh	READ DATA On-Die Termination Timing Control (by PHY) (ProcOdtT imeCtl_p1)	16	RW	0006h
20_00F8h	DLL gain control (DllGainCtl_p1)	16	RW	00A1h
20_0160h	Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiRdDataCsDestMap_p1)	16	RW	00E4h
20_0164h	PHY Global Vref Controls (VrefInGlobal_p1)	16	RW	0200h
20_0168h	Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiWrDataCsDestMap_p1)	16	RW	00E4h
20_018Ah	PState dependent PLL Control Register 2 (PllCtrl2_p1)	16	RW	0019h
20_018Eh	PState dependent PLL Control Register 1 (PllCtrl1_p1)	16	RW	0020h
20_0194h	Additional controls for PLL CP/VCO modes of operation (PllTestM ode_p1)	16	RW	0124h
20_0198h	PState dependent PLL Control Register 4 (PllCtrl4_p1)	16	RW	017Fh
20_01F4h	DFI Frequency Ratio (DfiFreqRatio_p1)	16	RW	0001h
40_0010h	Impedance Calibration Clock Ratio (CalUclkInfo_p2)	16	RW	0320h
40_0016h	PHY Initialization Engine (PIE) Delay Register 0 (Seq0BDLY0_p2)	16	RW	0000h
40_0018h	PHY Initialization Engine (PIE) Delay Register 1 (Seq0BDLY1_p2)	16	RW	0000h
40_001Ah	PHY Initialization Engine (PIE) Delay Register 2 (Seq0BDLY2_p2)	16	RW	0000h
40_001Ch	PHY Initialization Engine (PIE) Delay Register 3 (Seq0BDLY3_p2)	16	RW	0000h

Table continues on the next page...

DDR PHY (DDR_PHY)

Offset	Register	Width (In bits)	Access	Reset value
40_0020h	Setup Intervals for DFI PHY Master operations (PPTTrainSetup_p2)	16	RW	0000h
40_0032h	Mode select register for MEMCLK/Address/Command Tristates (TristateModeCA_p2)	16	RW	0000h
40_0040h	HWT MaxReadLatency. (HwtMRL_p2)	16	RW	0006h
40_0048h	Control the PHY logic related to the read and write DQS preamble (DqsPreambleControl_p2)	16	RW	0008h
40_005Ah	This Register is used to enable the Read-DBI function in each DBYTE (DMIPinPresent_p2)	16	RW	0000h
40_005Ch	Address/Command FIFO ReadPointer Initial Value (ARdPtrInitVal_p2)	16	RW	0000h
40_00ACh	READ DATA On-Die Termination Timing Control (by PHY) (ProcOdtTimeCtl_p2)	16	RW	0006h
40_00F8h	DLL gain control (DlIGainCtl_p2)	16	RW	00A1h
40_0160h	Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiRdDataCsDestMap_p2)	16	RW	00E4h
40_0164h	PHY Global Vref Controls (VrefInGlobal_p2)	16	RW	0200h
40_0168h	Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiWrDataCsDestMap_p2)	16	RW	00E4h
40_018Ah	PState dependent PLL Control Register 2 (PllCtrl2_p2)	16	RW	0019h
40_018Eh	PState dependent PLL Control Register 1 (PllCtrl1_p2)	16	RW	0020h
40_0194h	Additional controls for PLL CP/VCO modes of operation (PllTestMode_p2)	16	RW	0124h
40_0198h	PState dependent PLL Control Register 4 (PllCtrl4_p2)	16	RW	017Fh
40_01F4h	DFI Frequency Ratio (DfiFreqRatio_p2)	16	RW	0001h
60_0010h	Impedance Calibration Clock Ratio (CalUclkInfo_p3)	16	RW	0320h
60_0016h	PHY Initialization Engine (PIE) Delay Register 0 (Seq0BDLY0_p3)	16	RW	0000h
60_0018h	PHY Initialization Engine (PIE) Delay Register 1 (Seq0BDLY1_p3)	16	RW	0000h
60_001Ah	PHY Initialization Engine (PIE) Delay Register 2 (Seq0BDLY2_p3)	16	RW	0000h
60_001Ch	PHY Initialization Engine (PIE) Delay Register 3 (Seq0BDLY3_p3)	16	RW	0000h
60_0020h	Setup Intervals for DFI PHY Master operations (PPTTrainSetup_p3)	16	RW	0000h
60_0032h	Mode select register for MEMCLK/Address/Command Tristates (TristateModeCA_p3)	16	RW	0000h
60_0040h	HWT MaxReadLatency. (HwtMRL_p3)	16	RW	0006h
60_0048h	Control the PHY logic related to the read and write DQS preamble (DqsPreambleControl_p3)	16	RW	0008h
60_005Ah	This Register is used to enable the Read-DBI function in each DBYTE (DMIPinPresent_p3)	16	RW	0000h
60_005Ch	Address/Command FIFO ReadPointer Initial Value (ARdPtrInitVal_p3)	16	RW	0000h
60_00ACh	READ DATA On-Die Termination Timing Control (by PHY) (ProcOdtTimeCtl_p3)	16	RW	0006h
60_00F8h	DLL gain control (DlIGainCtl_p3)	16	RW	00A1h
60_0160h	Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiRdDataCsDestMap_p3)	16	RW	00E4h

Table continues on the next page...

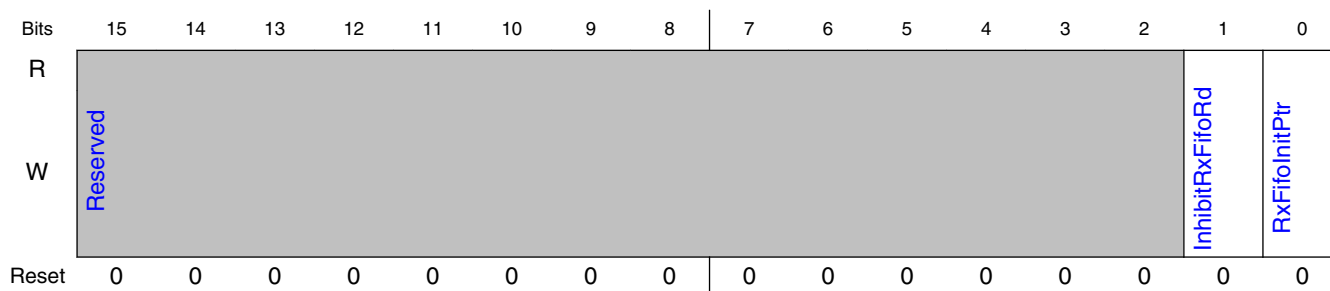
Offset	Register	Width (In bits)	Access	Reset value
60_0164h	PHY Global Vref Controls (VrefInGlobal_p3)	16	RW	0200h
60_0168h	Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiWrDataCsDestMap_p3)	16	RW	00E4h
60_018Ah	PState dependent PLL Control Register 2 (PIICtrl2_p3)	16	RW	0019h
60_018Eh	PState dependent PLL Control Register 1 (PIICtrl1_p3)	16	RW	0020h
60_0194h	Additional controls for PLL CP/VCO modes of operation (PIITestMode_p3)	16	RW	0124h
60_0198h	PState dependent PLL Control Register 4 (PIICtrl4_p3)	16	RW	017Fh
60_01F4h	DFI Frequency Ratio (DfiFreqRatio_p3)	16	RW	0001h

9.4.3.6.2 Rx FIFO pointer initialization control (RxFifoInit)

9.4.3.6.2.1 Offset

Register	Offset
RxFifoInit	0h

9.4.3.6.2.2 Diagram



9.4.3.6.2.3 Fields

Field	Function
15-2 —	Reserved
1 InhibitRxFifoRd	This field is reserved for training FW use. This field is reserved for training FW use. Setting this inhibits reads of the PHYRXDATAFIFO.
0	Setting this bit will reset the PHY RXDATAFIFO read and write pointers.

DDR PHY (DDR_PHY)

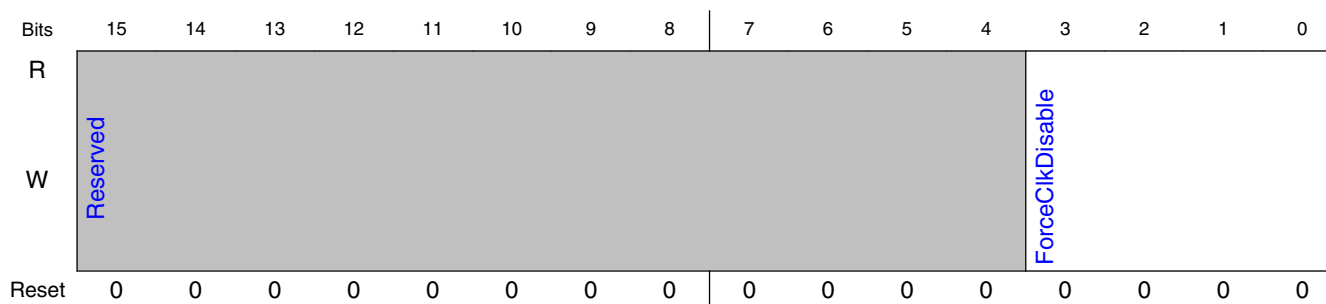
Field	Function
RxFifoInitPtr	Setting this bit will reset the PHY RXDATAFIFO read and write pointers. This bit must be Cleared to return to normal operation. This is not required in normal operation, but can be used instead of a dfi_ctrlupdate (for controllers that don't support ctrlupdate)

9.4.3.6.3 Clock gating control (ForceClkDisable)

9.4.3.6.3.1 Offset

Register	Offset
ForceClkDisable	2h

9.4.3.6.3.2 Diagram



9.4.3.6.3.3 Fields

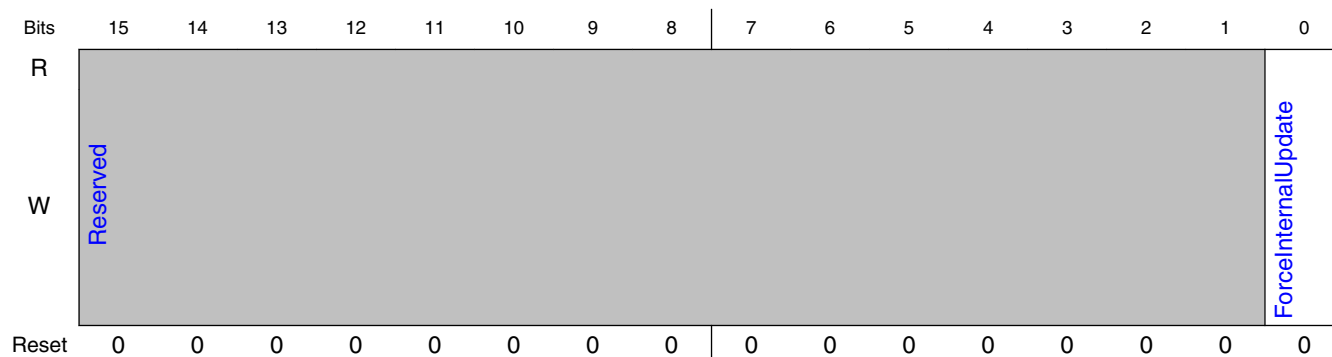
Field	Function
15-4 —	Reserved
3-0 ForceClkDisable	This CSR forces the gating of MEMCLKs driven from the PHY ForceClkDisable[0] - controls CLK_H/L0 ForceClkDisable[1] - controls CLK_H/L1 (if present) ForceClkDisable[2] - controls CLK_H/L2 (if present) ForceClkDisable[3] - controls CLK_H/L3 (if present)

9.4.3.6.4 This Register used by Training Firmware to force an internal PHY Update Event. (ForceInternalUpdate)

9.4.3.6.4.1 Offset

Register	Offset
ForceInternalUpdate	6h

9.4.3.6.4.2 Diagram



9.4.3.6.4.3 Fields

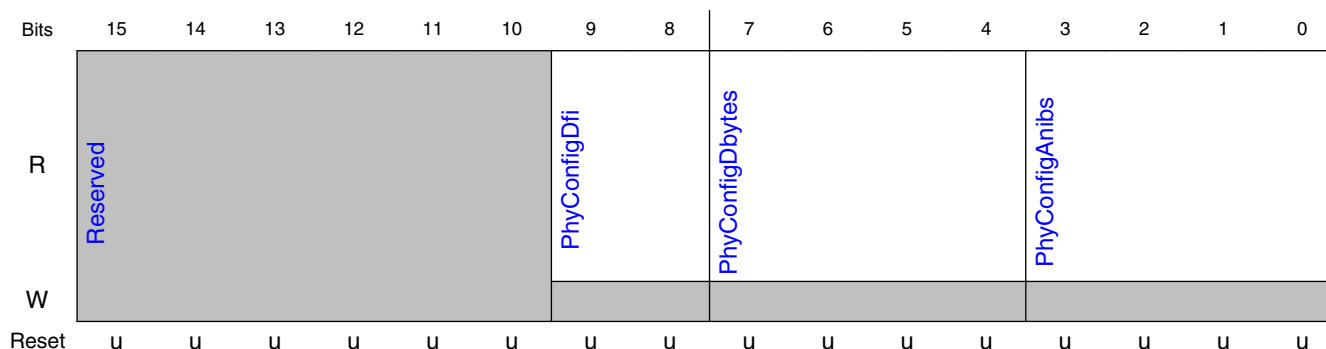
Field	Function
15-1	Reserved
0 ForceInternalUpdate	This Register is used by Training Firmware to force an internal PHY Update Event. This Register is used by Training Firmware to force an internal PHY Update Event. Optionally can be used by System Software to issue a PHY update Event. The register may be written to 1'b1 only when all of the following are true: [+] The DFI interface is offline. [+] The Training Hardware is sending only DES and all transactions have retired. Additionally, this Register must stay asserted for at least 32 DFICLKs before clearing. Prematurely clearing this register may result in an incomplete update.

9.4.3.6.5 Read Only displays PHY Configuration. (PhyConfig)

9.4.3.6.5.1 Offset

Register	Offset
PhyConfig	8h

9.4.3.6.5.2 Diagram



9.4.3.6.5.3 Fields

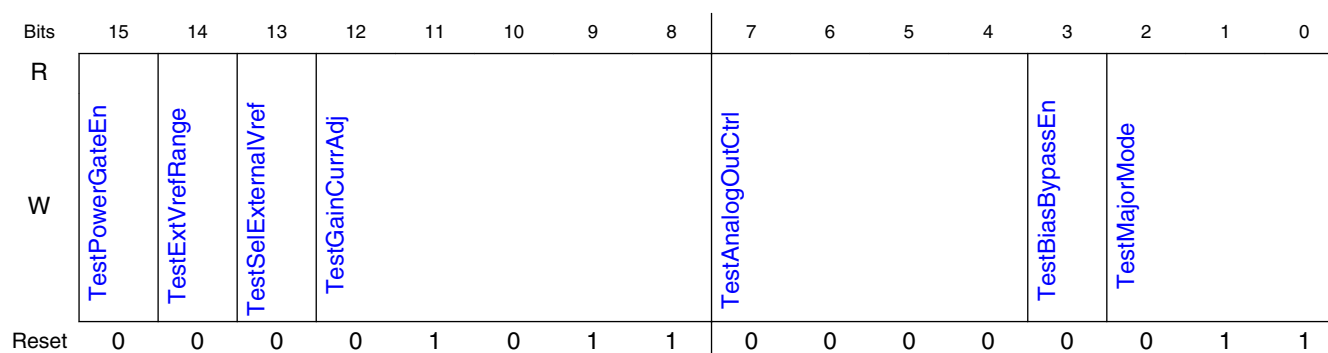
Field	Function
15-10 —	Reserved
9-8 PhyConfigDfi	Returns the following value . Returns the following value ... depending on the define "0x1" if DWC_DDRPHY_DFI1_EXISTS is not defined "0x2" if DWC_DDRPHY_DFI1_EXISTS is defined
7-4 PhyConfigDbytes	Returns the following value . Returns the following value ... depending on the chosen define "0x1" if DWC_DDRPHY_NUM_DBYTES_1 "0x2" if DWC_DDRPHY_NUM_DBYTES_2 "0x3" if DWC_DDRPHY_NUM_DBYTES_3 "0x4" if DWC_DDRPHY_NUM_DBYTES_4 "0x5" if DWC_DDRPHY_NUM_DBYTES_5 "0x6" if DWC_DDRPHY_NUM_DBYTES_6 "0x7" if DWC_DDRPHY_NUM_DBYTES_7 "0x8" if DWC_DDRPHY_NUM_DBYTES_8 "0x9" if DWC_DDRPHY_NUM_DBYTES_9 "0xA" if DWC_DDRPHY_NUM_DBYTES_10
3-0 PhyConfigAnibs	Returns the following value . Returns the following value ... depending on the chosen define "0x3" if DWC_DDRPHY_NUM_ANIBS_3 "0x6" if DWC_DDRPHY_NUM_ANIBS_6 "0xA" if DWC_DDRPHY_NUM_ANIBS_10 "0xC" if DWC_DDRPHY_NUM_ANIBS_12

9.4.3.6.6 Test Bump Control1 (TestBumpCntrl1)

9.4.3.6.6.1 Offset

Register	Offset
TestBumpCntrl1	Eh

9.4.3.6.6.2 Diagram



9.4.3.6.6.3 Fields

Field	Function
15 TestPowerGateEn	Do not use, for debug only
14 TestExtVrefRange	Setting this bit will extend the VREF DAC range for debug. Setting this bit will extend the VREF DAC range for debug. MUST BE SET TO ZERO in mission mode when MemAlert is enabled
13 TestSelExternalVref	Do not use, for debug only
12-8 TestGainCurrAdj	Adjust gain and current of analog observe RX amplifier stage at analog test point Recommended mission mode default = 5'b01011
7-4 TestAnalogOutCtrl	Select receiver internal analog signals to monitor at analog test point 0xxx: AnalogTestOut=HiZ 1000: AnalogTestOut=VSS 1001: AnalogTestOut=vref_dfe0 -- observe by sweeping MALERTVrefLevel 1010: AnalogTestOut=vref_dfe1 -- observe by sweeping MALERTVrefLevel 1011: AnalogTestOut=VSS 1100: AnalogTestOut=vstg2 1101: AnalogTestOut=vcasc_cs1 1110: AnalogTestOut=vbias_cs1 Recommended mission mode default = 4'b0000
3	Do not use, for debug only

Table continues on the next page...

DDR PHY (DDR_PHY)

Field	Function
TestBiasBypass En	
2-0	Selects the major mode of operation for the receiver.
TestMajorMode	Selects the major mode of operation for the receiver. These modes are mutually exclusive. 000 - setting for DDR3-RDIMM systems to receive ERROUT_n 001 - reserved 010 - reserved 011 - setting for DDR4 systems to receive Alert_n 100 - reserved for debug 101 - reserved 110 - reserved 111 - reserved for debug

9.4.3.6.7 Impedance Calibration Clock Ratio (CalUclkInfo_p0)

9.4.3.6.7.1 Offset

Register	Offset
CalUclkInfo_p0	10h

9.4.3.6.7.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved							CalUclkTicksPer1uS								
W	Reserved							CalUclkTicksPer1uS								
Reset	0	0	0	0	0	0	1	1	0	0	1	0	0	0	0	0

9.4.3.6.7.3 Fields

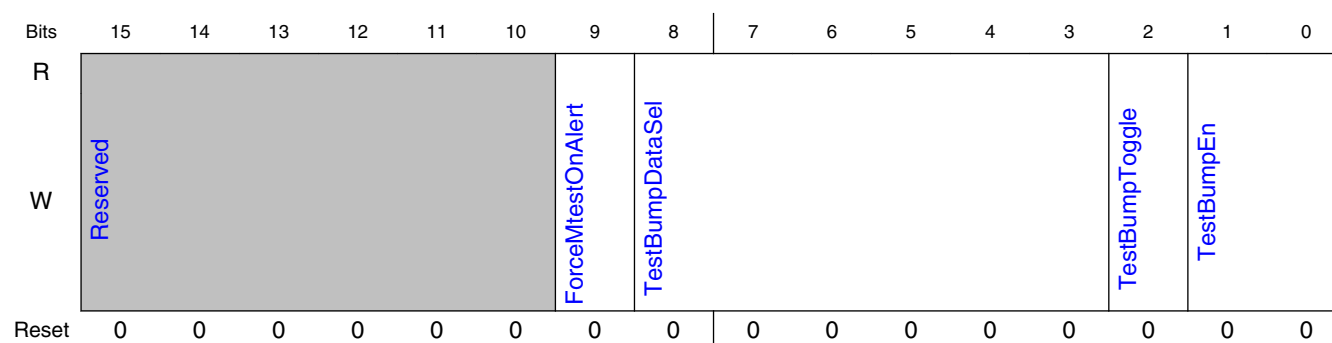
Field	Function
15-10 —	Reserved
9-0 CalUclkTicksPer1uS	Must be programmed to the number of DfiClks in 1us (rounded up), with minimum value of 24. Must be programmed to the number of DfiClks in 1us (rounded up), with minimum value of 24. if (DfiClk < 24MHz) CalUclkInfo = 24 else CalUclkInfo = (number of DfiClks in 1us)

9.4.3.6.8 Test Bump Control (TestBumpCntrl)

9.4.3.6.8.1 Offset

Register	Offset
TestBumpCntrl	14h

9.4.3.6.8.2 Diagram



9.4.3.6.8.3 Fields

Field	Function
15-10 —	Reserved
9 ForceMtestOnAlert	When set, causes the Digital Observation output pin to be driven onto BP_ALERT_N
8-3 TestBumpDataSel	RVSD.
2 TestBumpToggle	This field controls the output function of the signal Digital Observation Pin, if available in the configuration of the PHY. This field controls the output function of the signal Digital Observation Pin, if available in the configuration of the PHY. When set to 1'b0 -- the output will be tristated When set to 1'b1 -- the output will be selected by MtestMuxSel
1-0 TestBumpEn	Field TestBumpEn[1:0] controls the output function of: the signal BP_ALERT_N. Field TestBumpEn[1:0] controls the output function of: the signal BP_ALERT_N.

DDR PHY (DDR_PHY)

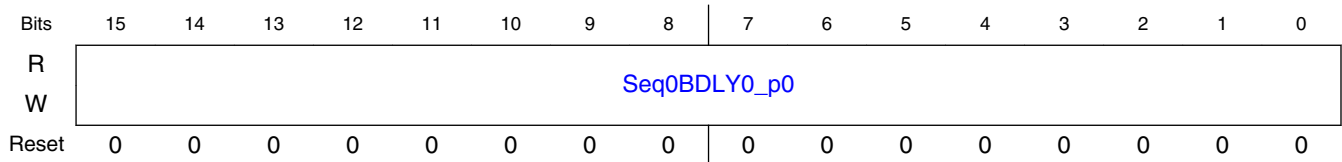
Field	Function
	TestBumpEn Value on BP_ALERT_N output ----- 2'b00 high-z, 2'b01 Pllout-divided-by-2, works up to full data rate 2'b10 PllDigTst[1], works up to 400MHz 2'b11 PllDigTst[1], works up to 400MHz

9.4.3.6.9 PHY Initialization Engine (PIE) Delay Register 0 (Seq0BDLY0_p0)

9.4.3.6.9.1 Offset

Register	Offset
Seq0BDLY0_p0	16h

9.4.3.6.9.2 Diagram



9.4.3.6.9.3 Fields

Field	Function
15-0 Seq0BDLY0_p0	PHY Initialization Engine (PIE) Delay Register 0 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 0 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddsphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfIClk Frequency. When calculating the number of cycles to be programmed for each PState, it

Field	Function
	should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.10 PHY Initialization Engine (PIE) Delay Register 1 (Seq0BDLY1_p0)

9.4.3.6.10.1 Offset

Register	Offset
Seq0BDLY1_p0	18h

9.4.3.6.10.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Seq0BDLY1_p0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.4.3.6.10.3 Fields

Field	Function
15-0 Seq0BDLY1_p0	PHY Initialization Engine (PIE) Delay Register 1 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 1 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: -----

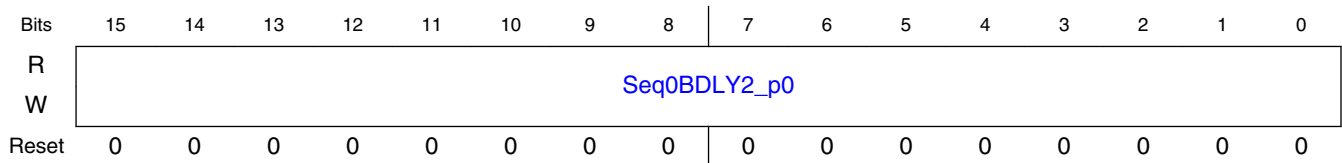
Field	Function
	These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfIClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.11 PHY Initialization Engine (PIE) Delay Register 2 (Seq0BDLY2_p0)

9.4.3.6.11.1 Offset

Register	Offset
Seq0BDLY2_p0	1Ah

9.4.3.6.11.2 Diagram



9.4.3.6.11.3 Fields

Field	Function
15-0 Seq0BDLY2_p0	PHY Initialization Engine (PIE) Delay Register 2 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 2 This register is available for selection by the NOP and WAIT instructions

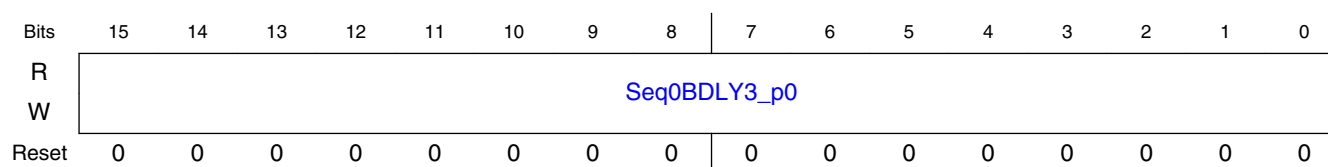
Field	Function
	in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfIClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.12 PHY Initialization Engine (PIE) Delay Register 3 (Seq0BDLY3_p0)

9.4.3.6.12.1 Offset

Register	Offset
Seq0BDLY3_p0	1Ch

9.4.3.6.12.2 Diagram



9.4.3.6.12.3 Fields

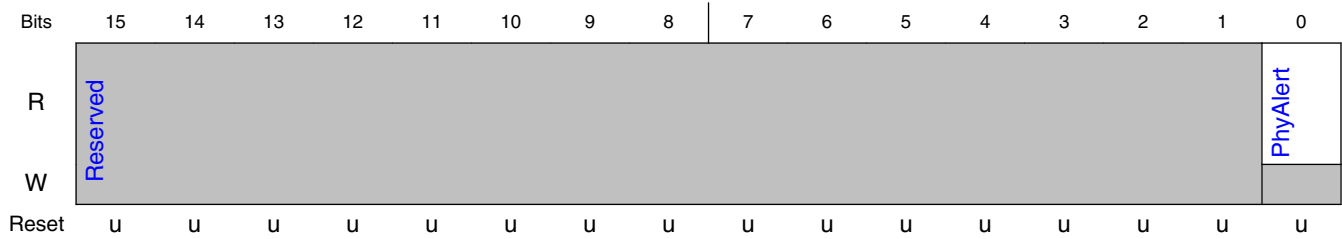
Field	Function
15-0 Seq0BDLY3_p0	PHY Initialization Engine (PIE) Delay Register 3 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 3 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfIClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.13 PHY Alert status bit (PhyAlertStatus)

9.4.3.6.13.1 Offset

Register	Offset
PhyAlertStatus	1Eh

9.4.3.6.13.2 Diagram



9.4.3.6.13.3 Fields

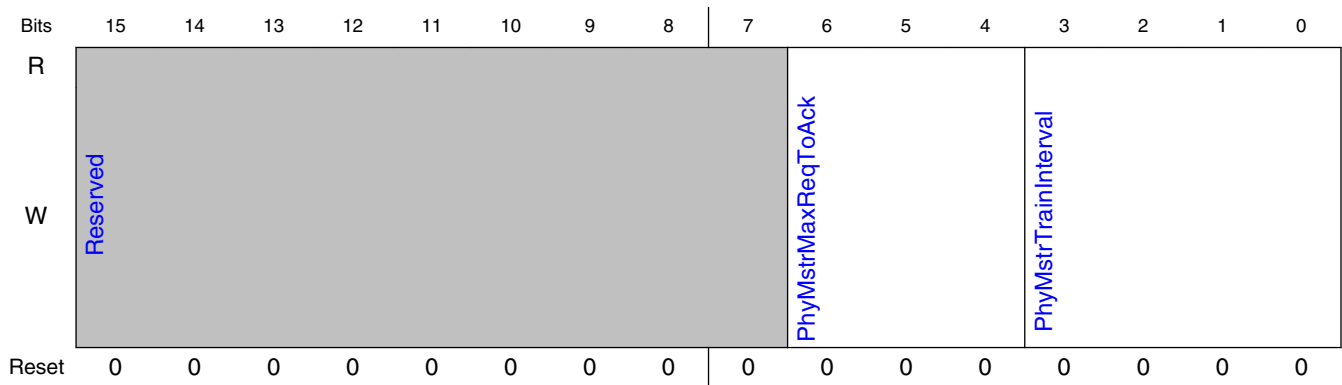
Field	Function
15-1	Reserved
0	Current state of ALERT_N.
PhyAlert	

9.4.3.6.14 Setup Intervals for DFI PHY Master operations (PPTTrainSetup_p0)

9.4.3.6.14.1 Offset

Register	Offset
PPTTrainSetup_p0	20h

9.4.3.6.14.2 Diagram



9.4.3.6.14.3 Fields

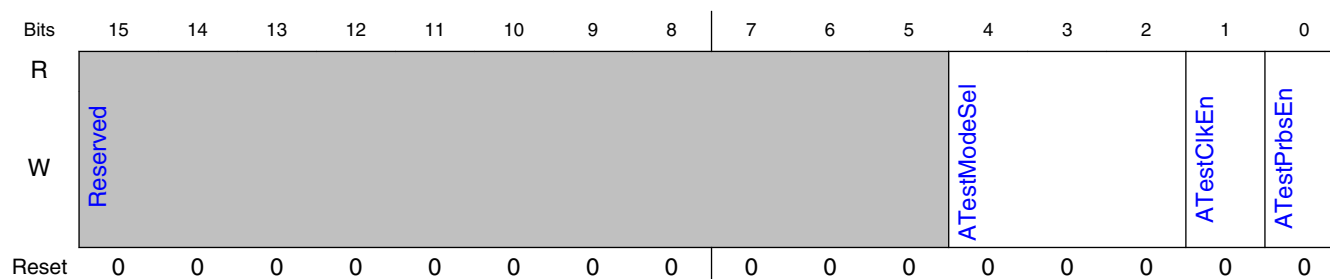
Field	Function
15-7 —	Reserved
6-4 PhyMstrMaxReq ToAck	Bits 6:4 of this register specify the max time from tdfi_phymstr_req asserted to tdfi_phymstr_ack asserted. Bits 6:4 of this register specify the max time from tdfi_phymstr_req asserted to tdfi_phymstr_ack asserted. 3'b000 Disable PHY Master Interface 3'b001 set tPHYMSTR_resp. = 512 MEMCLKs 3'b010 set tPHYMSTR_resp. = 1024 MEMCLKs 3'b011 set tPHYMSTR_resp. = 2048 MEMCLKs 3'b100 set tPHYMSTR_resp. = 4096 MEMCLKs 3'b101 set tPHYMSTR_resp. = 8192 MEMCLKs 3'b110 set tPHYMSTR_resp. = 32768 MEMCLKs 3'b111 set tPHYMSTR_resp. = undefined
3-0 PhyMstrTrainInterval	Bits 3:0 of this register specifies the time between the end of one training and the start of the next. Bits 3:0 of this register specifies the time between the end of one training and the start of the next. (As per section 11 of the Preliminary DFI 4.0 Specification, V2, it is the max expected time from dfi_init_complete asserted to tdfi_phymstr_ack asserted). 4'b0000 Disable PHY Master Interface 4'b0001 PHY MASTER Request Interval = 524288 MEMCLKs 4'b0010 PHY MASTER Request Interval = 1048576 MEMCLKs 4'b0011 PHY MASTER Request Interval = 2097152 MEMCLKs 4'b0100 PHY MASTER Request Interval = 4194304 MEMCLKs 4'b0101 PHY MASTER Request Interval = 8388608 MEMCLKs 4'b0110 PHY MASTER Request Interval = 16777216 MEMCLKs 4'b0111 PHY MASTER Request Interval = 33554432 MEMCLKs 4'b1000 PHY MASTER Request Interval = 67108864 MEMCLKs 4'b1001 PHY MASTER Request Interval = 134217728 MEMCLKs 4'b1010 PHY MASTER Request Interval = 268435456 MEMCLKs 4'b1011 - 4'b1111 PHY MASTER Request Interval = undefined

9.4.3.6.15 ATestMode control (ATestMode)

9.4.3.6.15.1 Offset

Register	Offset
ATestMode	24h

9.4.3.6.15.2 Diagram



9.4.3.6.15.3 Fields

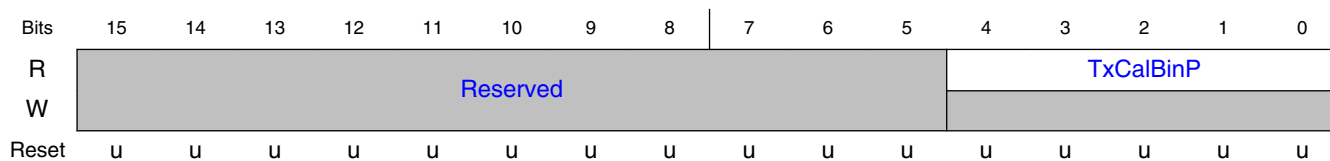
Field	Function
15-5 —	Reserved
4-2 ATestModeSel	Master Mode select for ATest (Loopback) 000 - Mission mode, all ATest disabled, loopback receivers powered down 001 - External Loopback mode [Single data rate pattern - dfi_cas sent to all lanes] 010 - Internal Loopback mode [Single data rate pattern] 011 - Internal Loopback mode [Double data rate pattern] 100 - External Loopback mode [Single data rate pattern - corresponding DFI signal sent to each lane]
1 ATestClkEn	Enables the clock for loopback PRBS7 testing for all BP_A* pins.
0 ATestPrbsEn	Enables loopback PRBS7 testing of all the DDR output pins in this chiplet. Enables loopback PRBS7 testing of all the DDR output pins in this chiplet. The asserting edge resets all the individual TestPrbsErrCnt & consequently TestPrbsErr; there is no other reset for the TestPrbsErrCnt. The ATestClkEn CSR below must be enabled first prior to enabling this bit.

9.4.3.6.16 TX P Impedance Calibration observation (TxCalBinP)

9.4.3.6.16.1 Offset

Register	Offset
TxCaIBinP	28h

9.4.3.6.16.2 Diagram



9.4.3.6.16.3 Fields

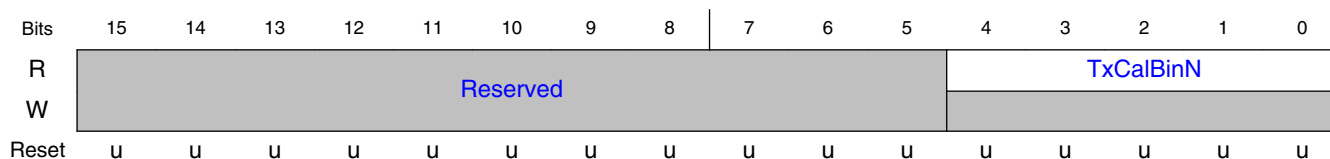
Field	Function
15-5 —	Reserved
4-0 TxCalBinP	This csr holds the binary result of the 31 bit thermometer pullup code.

9.4.3.6.17 TX N Impedance Calibration observation (TxCalBinN)

9.4.3.6.17.1 Offset

Register	Offset
TxCaIBinN	2Ah

9.4.3.6.17.2 Diagram



9.4.3.6.17.3 Fields

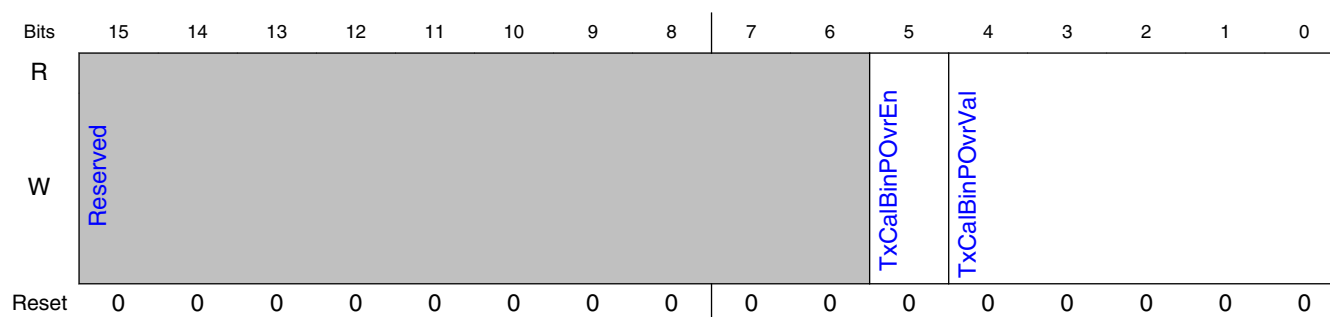
Field	Function
15-5 —	Reserved
4-0 TxCalBinN	This csr holds the binary result of the 31 bit thermometer pulldown code.

9.4.3.6.18 TX P Impedance Calibration override (TxCalPOvr)

9.4.3.6.18.1 Offset

Register	Offset
TxCalPOvr	2Ch

9.4.3.6.18.2 Diagram



9.4.3.6.18.3 Fields

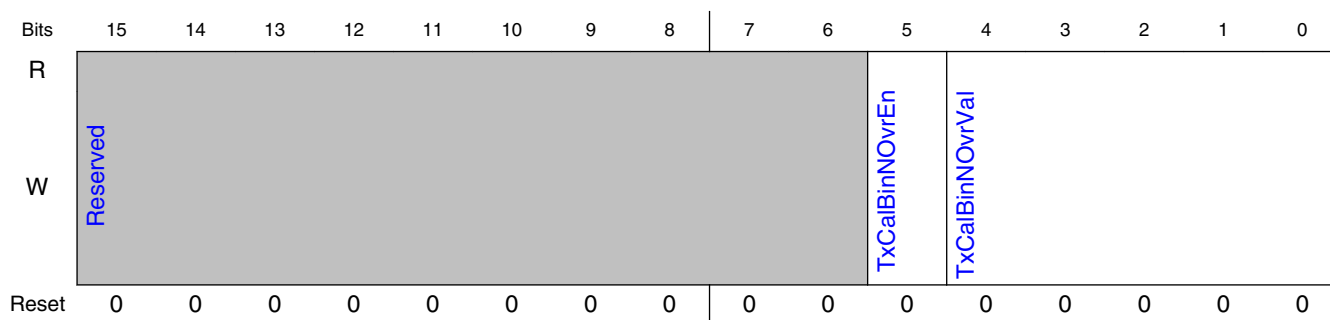
Field	Function
15-6 —	Reserved
5 TxCalBinPOvrEn	1 = use the override value present in Register TxCalBinPOvrVal 0 = don't.
4-0 TxCalBinPOvrVal	The binary value which can override the Register TxCalBinP calibrator results if Register TxCalBinPOvrEn is set.

9.4.3.6.19 TX N Impedance Calibration override (TxCalNOvr)

9.4.3.6.19.1 Offset

Register	Offset
TxCalNOvr	2Eh

9.4.3.6.19.2 Diagram



9.4.3.6.19.3 Fields

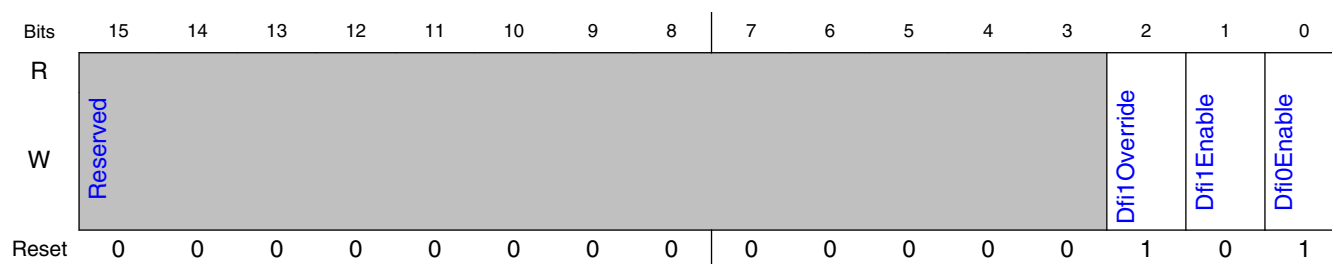
Field	Function
15-6 —	Reserved
5 TxCalBinNOvrEn	1 = use the override value present in Register TxCalBinNOvrVal 0 = don't.
4-0 TxCalBinNOvrVal	The binary value which can override the Register TxCalBinN calibrator results if Register TxCalBinPOvrEn is set.

9.4.3.6.20 Enables for update and low-power interfaces for DFI0 and DFI1 (DfiMode)

9.4.3.6.20.1 Offset

Register	Offset
DfiMode	30h

9.4.3.6.20.2 Diagram



9.4.3.6.20.3 Fields

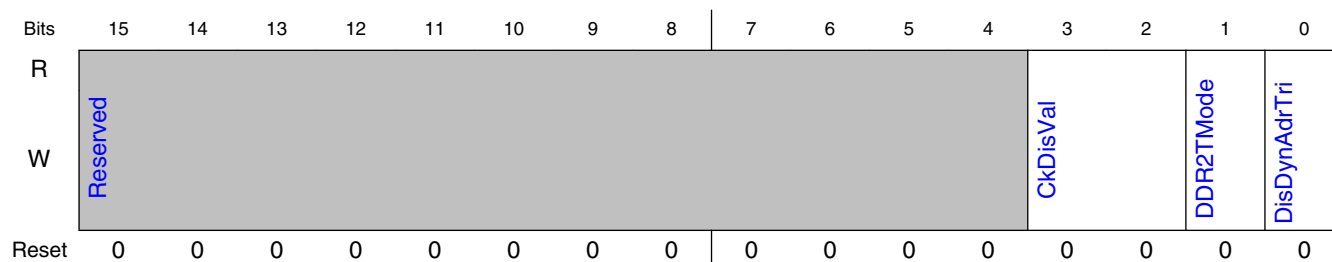
Field	Function
15-3 —	Reserved
2 Dfi1Override	DFI0 is used to control the PHY logic associated with both DFI0 and DFI1
1 Dfi1Enable	Enables operation for the PHY logic associated with DFI1
0 Dfi0Enable	Enables operation for the PHY logic associated with DFI0

9.4.3.6.21 Mode select register for MEMCLK/Address/Command Tristates (TristateModeCA_p0)

9.4.3.6.21.1 Offset

Register	Offset
TristateModeCA_p0	32h

9.4.3.6.21.2 Diagram



9.4.3.6.21.3 Fields

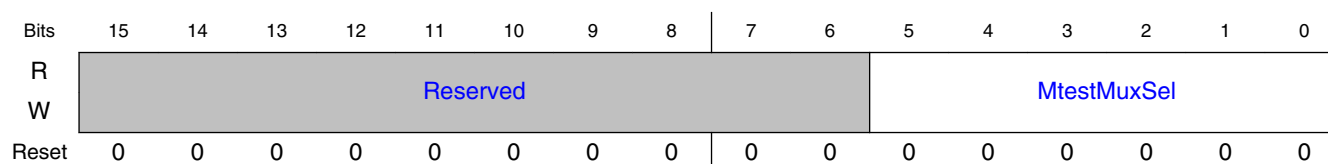
Field	Function
15-4 —	Reserved
3-2 CkDisVal	The PHY provides 4 memory clocks, n=0. The PHY provides 4 memory clocks, n=0..3. When the toggling of memory clock CK_t[n] is disabled with dfi_clk_disable[n]=1, the memory clock CK_t[n] is driven with Register CkDisVal[1]. When the toggling of memory clock CK_c[n] is disabled with dfi_clk_disable[n]=1, the memory clock CK_c[n] is driven with ~(Register CkDisVal[0]) (ie inverted). Note that the non-toggling differential memory clock CK_t[n],CK_c[n] may be driven with any combination, LL,LH,HL,HH; the default CK_t[n],CK_c[n]=LH.
1 DDR2TMode	Must be set to 1 for Dynamic Tristate to work when CA bus is 2T or Geardown mode. Must be set to 1 for Dynamic Tristate to work when CA bus is 2T or Geardown mode. This CSR bit has no effect when DisDynAdrTri==1
0 DisDynAdrTri	When DisDynAdrTri=1, Dynamic Tristating is disabled. When DisDynAdrTri=1, Dynamic Tristating is disabled. Dynamic Tristating is on by default. . Dynamic Tristating should be disabled (DisDynAdrTri=0) for these modes: 1. DDR3/2T if the controller cannot follow the 2T PHY tristate protocol. 2. DDR4/2T/2N if the controller cannot follow the 2T PHY tristate protocol. 3. LPDDR4 . When DisDynAdrTri=0 (default), In DDR3 mode, The following SDRAM pins can be dynamically tristated: A,BA,RAS_n,CAS_n,WE_n In DDR4 mode, The following SDRAM pins can be dynamically tristated: A,BA,BG,ACT_n In LPDDR3 mode, The following SDRAM pins can be dynamically tristated: CA[*] . In 1T mode, the PHY will tristate the relevant pins when all ranks are DESelected . In 2T mode (or geardown), the PHY will tristate the relevant pins when: D3 -- All ranks DESelected (all CS_L==1) AND RAS_n,CAS_n,WE_n,BA0 = 1,1,1,0 D4 -- All ranks DESelected (all CS_L==1) AND RAS_n,CAS_n,WE_n,BA0,ACT_n = 1,1,1,0,1 When in this mode, the controller should avoid sending the above patterns with any rank selected. [e.g. NOPs sent by the controller should have BA0 set to a non-zero value]

9.4.3.6.22 Digital Observation Pin control (MtestMuxSel)

9.4.3.6.22.1 Offset

Register	Offset
MtestMuxSel	34h

9.4.3.6.22.2 Diagram



9.4.3.6.22.3 Fields

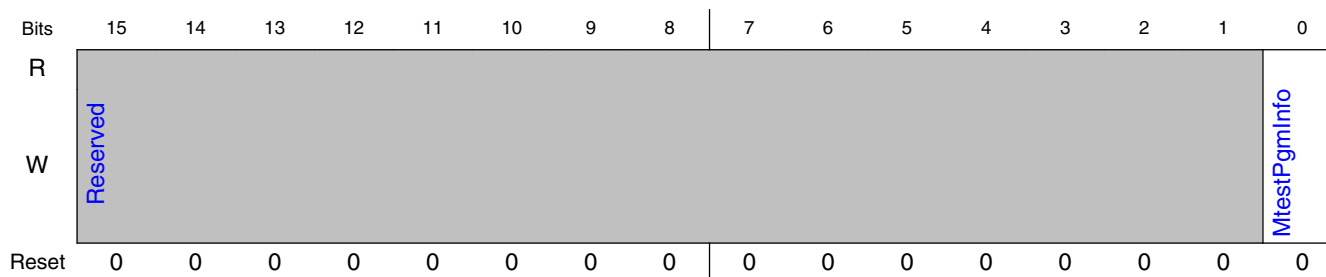
Field	Function
15-6 —	Reserved
5-0 MtestMuxSel	Controls for the 64-1 mux for asynchronous data to the Digital Observation Pin. Controls for the 64-1 mux for asynchronous data to the Digital Observation Pin. Encoding 6'h0 --> Drive 0 from this chiplet (allows flat 'OR' of pass-through information) Encoding 6'h01:1f --> Select local data from AC/DBYTE/MASTER macro Encoding 6'h20 --> Reserved (Drive 0 from macro into mux; Drive 0 from PUB synth logic path into mux) Encoding 6'h21:3f --> Select local data from PUB AC/DBYTE/MASTER synthesized logic Note: See PUB documentation for how, or if, the Digital Observation Pin is mapped to a physical bump in this configuration.

9.4.3.6.23 Digital Observation Pin program info for debug (MtestPgmlInfo)

9.4.3.6.23.1 Offset

Register	Offset
MtestPgmlInfo	36h

9.4.3.6.23.2 Diagram



9.4.3.6.23.3 Fields

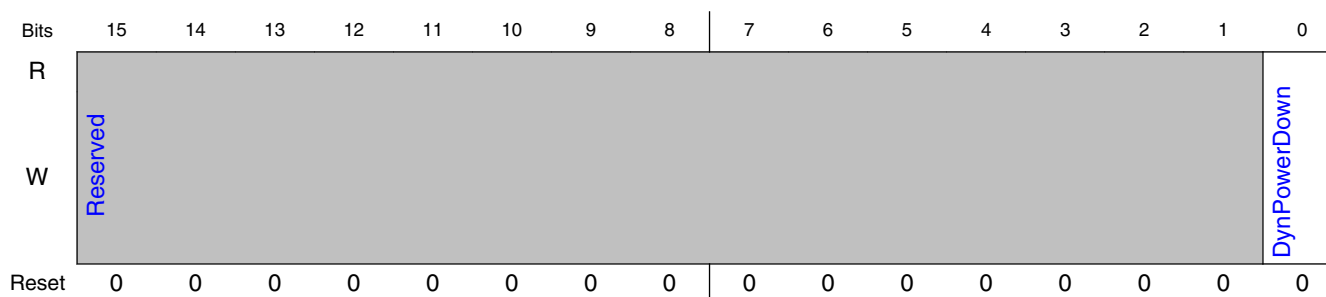
Field	Function
15-1	Reserved
0	The value of this csr may be driven onto the Digital Observation Pin.
MtestPgmInfo	The value of this csr may be driven onto the Digital Observation Pin. It has no other hardware effect.

9.4.3.6.24 Dynaimc Power Up/Down control (DynPwrDnUp)

9.4.3.6.24.1 Offset

Register	Offset
DynPwrDnUp	38h

9.4.3.6.24.2 Diagram



9.4.3.6.24.3 Fields

Field	Function
15-1 —	Reserved
0 DynPowerDown	1 - analog circuitry (voltage dacs, bias gen) is turned off. 1 - analog circuitry (voltage dacs, bias gen) is turned off. 0 - normal operation.

9.4.3.6.25 PHY Technology ID Register (PhyTID)

9.4.3.6.25.1 Offset

Register	Offset
PhyTID	3Ch

9.4.3.6.25.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PhyTID															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.4.3.6.25.3 Fields

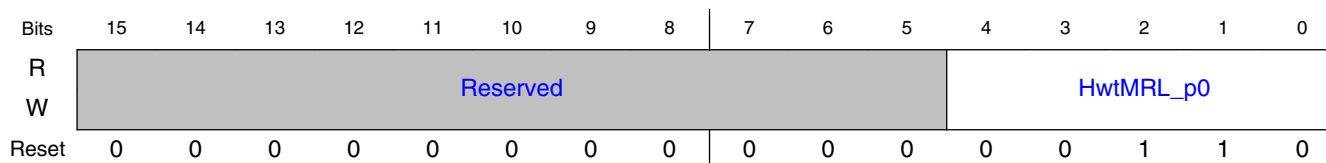
Field	Function
15-0 PhyTID	This register is a placeholder to store technology-specific information

9.4.3.6.26 HWT MaxReadLatency. (HwtMRL_p0)

9.4.3.6.26.1 Offset

Register	Offset
HwtMRL_p0	40h

9.4.3.6.26.2 Diagram



9.4.3.6.26.3 Fields

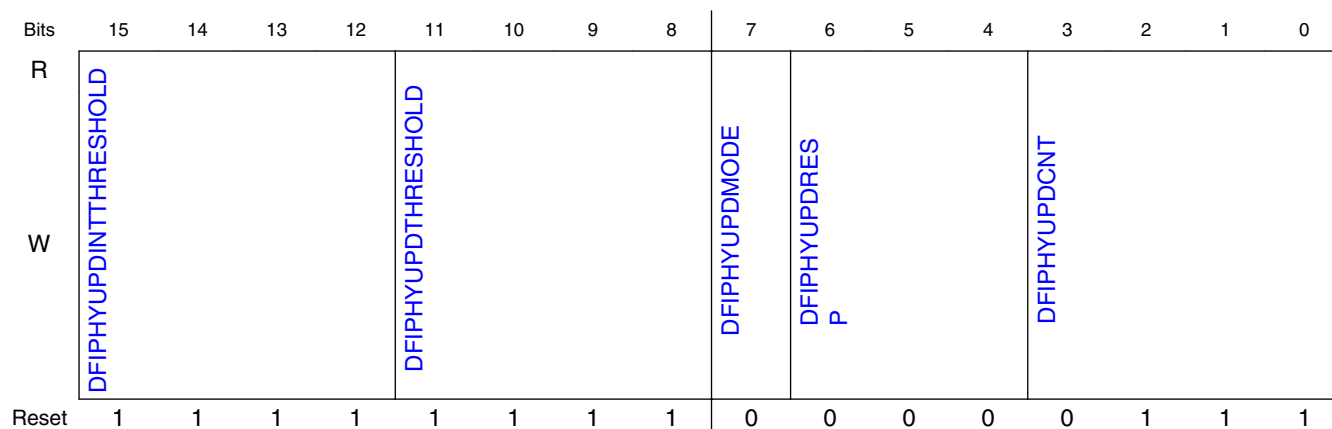
Field	Function
15-5 —	Reserved
4-0 HwtMRL_p0	This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. The CSR is in units of two mem clocks; that is, a unit change in the LSB is a change in MRL of two mem clocks. This MASTER copy of MRL is used by the PHY training hardware only.

9.4.3.6.27 DFI PhyUpdate Request time counter (in MEMCLKs) (DFIPHYUPD)

9.4.3.6.27.1 Offset

Register	Offset
DFIPHYUPD	42h

9.4.3.6.27.2 Diagram



9.4.3.6.27.3 Fields

Field	Function
15-12 DFIPHYUPDINT THRESHOLD	This subfield is similar to DFIPHYUPDTHRESHOLD except that rather than affecting the Phy Update request, it affects only the threshold used to generate the VT Drift Alarm Interrupt. This subfield is similar to DFIPHYUPDTHRESHOLD except that rather than affecting the Phy Update request, it affects only the threshold used to generate the VT Drift Alarm Interrupt. Further, the actual threshold is twice what is specified in the field.
11-8 DFIPHYUPDTH RESHOLD	4'h0 Disable Threshold-based Phy Update Requests when DFIPHYUPDMODE==1'b1 Nonzero codes are the threshold value for the change in the master LCDL 1UI phase code since the last Phy Update Request that will trigger a new Phy Update Request; If (current_1UI_phase - last_1UI_phase) > DFIPHYUPDTHRESHOLD, then a Phy Update will be requested. 4'h0 Disable Threshold-based Phy Update Requests when DFIPHYUPDMODE==1'b1 Nonzero codes are the threshold value for the change in the master LCDL 1UI phase code since the last Phy Update Request that will trigger a new Phy Update Request; If (current_1UI_phase - last_1UI_phase) > DFIPHYUPDTHRESHOLD, then a Phy Update will be requested. In units of native-control phase units, Minimum value is 1. With DFIPHYUPDTHRESHOLD=1, a master LCDL dithering between two values will not cause a Phy Update Request.
7 DFIPHYUPDMO DE	1'b0 [Default] deterministic timer-based Phy Update Requests; enables multi-channel/multi-phy lockstep operation. 1'b0 [Default] deterministic timer-based Phy Update Requests; enables multi-channel/multi-phy lockstep operation. 1'b1 Non-deterministic threshold-based Phy Update Requests; multi-channel/multi-phy operation not permitted.
6-4 DFIPHYUPDRE SP	Enforces the t_phyupd_resp time, the maximum time that is allowed to controller to respond to the request for a PHY update. Enforces the t_phyupd_resp time, the maximum time that is allowed to controller

Table continues on the next page...

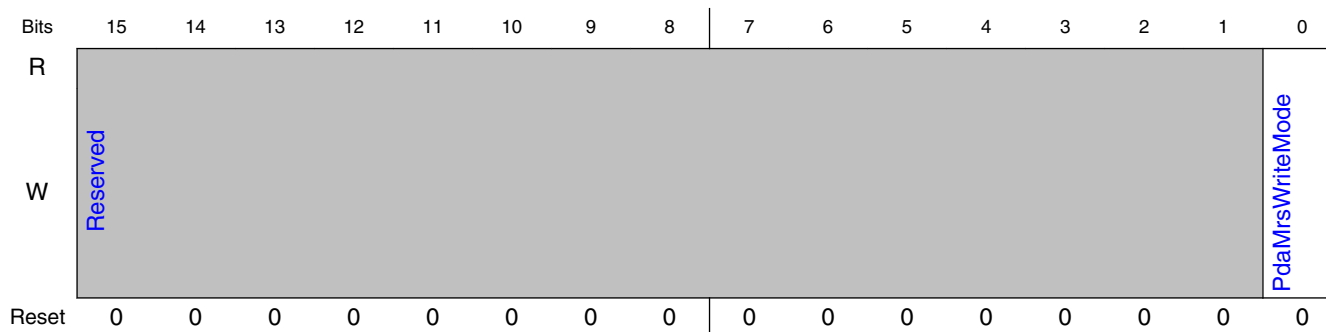
Field	Function
	to respond to the request for a PHY update. A dfi_error will be signaled if there is no acknowledgement of the update request within nMEMCLKs_phyupd_resp. 3'b000 nMEMCLKs_phyupd_resp= 1024 MEMCLKs, default value 3'b001 nMEMCLKs_phyupd_resp= 2048 MEMCLKs 3'b010 nMEMCLKs_phyupd_resp= 4096 MEMCLKs 3'b011 nMEMCLKs_phyupd_resp= 8192 MEMCLKs, [not allowed in LPDDR3, LPDDR4 modes] Any code not enumerated above is RSVD and should not be set.
3-0 DFIPHYUPDCNT	This controls the interval between the end of a phyupdate transaction and a subsequent request. This controls the interval between the end of a phyupdate transaction and a subsequent request. 4'b0000 - Disable timer-based Phy Update when DFIPHYUPDMODE==1'b0 4'b0001 - 16K MEMCLKs minus nMEMCLKs_phyupd_resp 4'b0011 - 32K MEMCLKs minus nMEMCLKs_phyupd_resp 4'b0111 - 64K MEMCLKs minus nMEMCLKs_phyupd_resp 4'b1111 - 128K MEMCLKs minus nMEMCLKs_phyupd_resp Any code not enumerated above is RSVD and should not be set.

9.4.3.6.28 Controls the write DQ generation for Per-Dram-Addressing of MRS (PdaMrsWriteMode)

9.4.3.6.28.1 Offset

Register	Offset
PdaMrsWriteMode	44h

9.4.3.6.28.2 Diagram



9.4.3.6.28.3 Fields

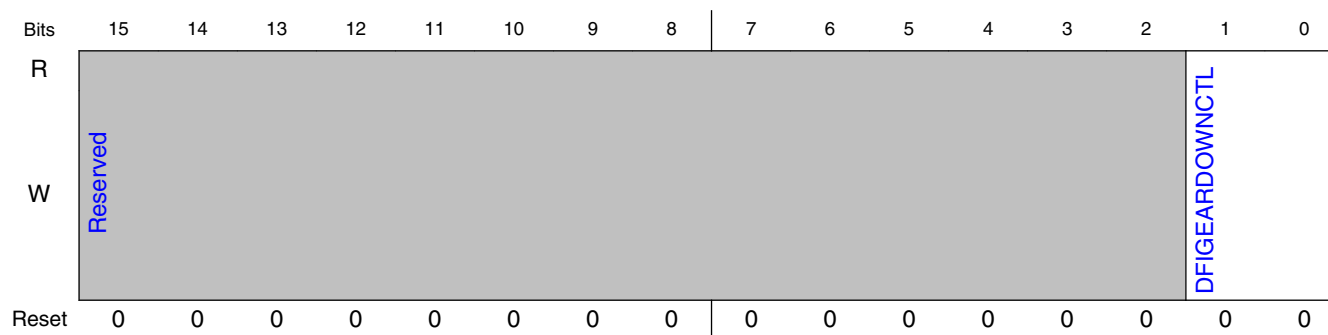
Field	Function
15-1 —	Reserved
0 PdaMrsWriteMode	Controls the write DQ generation per the timing requirements on the DQ signals used for Per-Dram-Addressing mode of MRS commands. Controls the write DQ generation per the timing requirements on the DQ signals used for Per-Dram-Addressing mode of MRS commands. When asserted, the DQ are driven for an additional UI before the logical burst and for an additional UI after the logical burst. The early UI has the value of first logical UI, and the late UI has the value of the last logical UI. The timing of the logical burst is unchanged; no modification to csr TxDqDly is required or performed. Strongly recommended to be cleared after completing the MRS that clears MR3 A4 and for mission-mode writes, else required bubbles will be larger and the power consumed will be greater. See JEDEC MR3 A4, Per DRAM Addressability.

9.4.3.6.29 Controls whether dfi_geardown_en will cause CS and CKE timing to change. (DFIGEARDOWNCTL)

9.4.3.6.29.1 Offset

Register	Offset
DFIGEARDOWNCTL	46h

9.4.3.6.29.2 Diagram



9.4.3.6.29.3 Fields

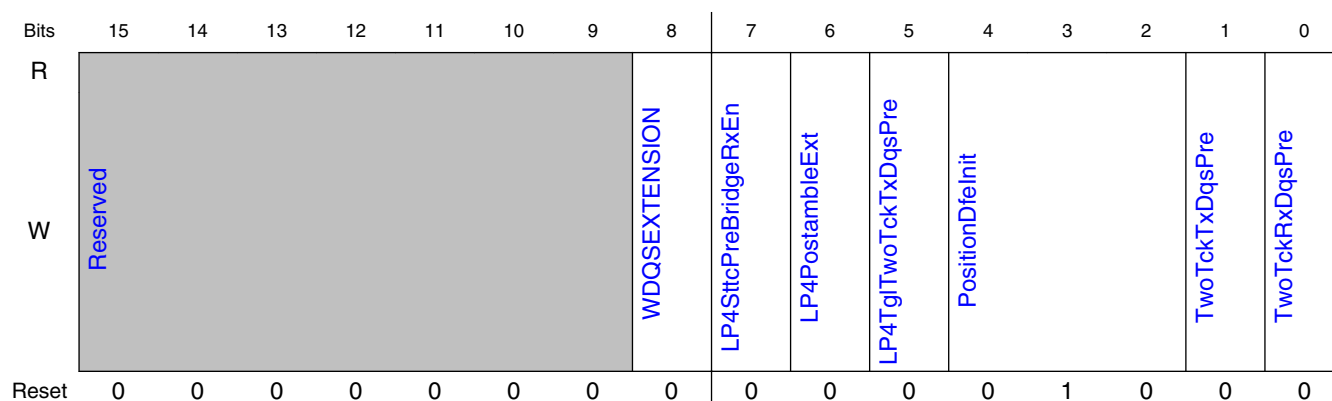
Field	Function
15-2 —	Reserved
1-0 DFIGEARDOWNCTL	DFIGEARDOWNCTL[0] controls whether dfi_geardown_en will cause chip-select (CS) timing to change. DFIGEARDOWNCTL[0] controls whether dfi_geardown_en will cause chip-select (CS) timing to change. 0 - CS timing will not be dynamically modified with dfi_geardown_en 1 - CS will be delayed by an additional 1UI dynamically when dfi_geardown_en==1 Note, this CSR must be set to zero if the PHY is not in DDR4 Mode DFIGEARDOWNCTL[1] controls whether dfi_geardown_en will cause clock-enable (CKE) timing to change. 0 - CKE timing will not be dynamically modified with dfi_geardown_en 1 - CKE will be delayed by an additional 1UI dynamically when dfi_geardown_en==1 Note, this CSR must be set to zero if the PHY is not in DDR4 Mode

9.4.3.6.30 Control the PHY logic related to the read and write DQS preamble (DqsPreambleControl_p0)

9.4.3.6.30.1 Offset

Register	Offset
DqsPreambleControl_p0	48h

9.4.3.6.30.2 Diagram



9.4.3.6.30.3 Fields

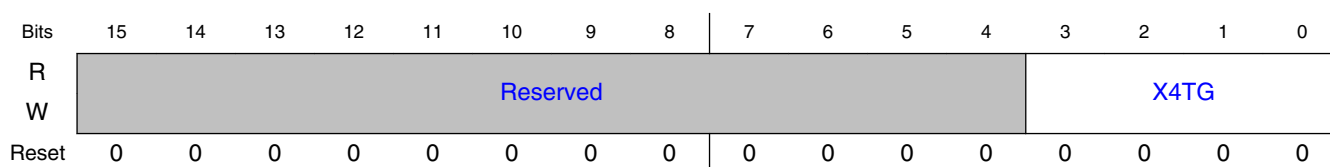
Field	Function
15-9 —	Reserved
8 WDQSEXTENSION	When set, DQS_T and DQS_C will be driven differentially to 0 and 1, respectively, before and after a write burst, except during a memory read transaction. When set, DQS_T and DQS_C will be driven differentially to 0 and 1, respectively, before and after a write burst, except during a memory read transaction. See DesignWare Cores LPDDR4 MultiPHY: WDQS Extension Application Note. Use of WDQSEXTENSION requires POdtTailWidth=3 and POdtStartDelay=00
7 LP4SttcPreBridgeRxEn	Used in LPDDR4 static-preamble mode to bridge the RxEn between two reads to the same timing group when the bubble is 1 memclk. Used in LPDDR4 static-preamble mode to bridge the RxEn between two reads to the same timing group when the bubble is 1 memclk. LPDDR4 static-preamble mode is set in the DRAMs with MR1, OP[3]=0.
6 LP4PostambleExt	In LPDDR4 mode must be set to extend the write postamble. In LPDDR4 mode must be set to extend the write postamble. 0: half-memclk postamble, as in D4 1: one-and-one-half-memclk postamble; see LPDDR4 Spec MR3, OP[1] WR PST, vendor-specific function.
5 LP4TglTwoTckTxDqsPre	Used in LPDDR4 mode to modify the early preamble when Register TwoTckTxDqsPre=1 0: level first-memclk preamble 1: toggling first-memclk preamble
4-2 PositionDfelnit	For DDR4 phy only when receive DFE is enabled. For DDR4 phy only when receive DFE is enabled. PositionDfelnit[2]=1 causes Dfelnit to be asserted late PositionDfelnit[1]=1 causes Dfelnit to be asserted at the nominal time PositionDfelnit[0]=1 causes Dfelnit to be asserted early PositionDfelnit=3'b010 is the nominal, recommended value for leading edge used by receiver.
1 TwoTckTxDqsPre	0: Standard 1tck TxDqs Preamble 1: Enable Optional D4 2tck TxDqs Preamble The DDR4 MR4 A12 is Write Preamble, 1=2nCK, 0=1nCK.
0 TwoTckRxDqsPre	Widens the RxDqsEn window to allow larger drift in the incoming read DQS to take advantage of the larger/wider preamble generated by the DRAMs when the D4 DRAMS are configured with DDR4 MR4 A11 Read Preamble=1 for causing a 2nCK read preamble. Widens the RxDqsEn window to allow larger drift in the incoming read DQS to take advantage of the larger/wider preamble generated by the DRAMs when the D4 DRAMS are configured with DDR4 MR4 A11 Read Preamble=1 for causing a 2nCK read preamble. For LPDDR4, all read operations are 2nCK such that this control must be set to 1. Note the nominal trained-to-center point will be earlier relative to the first strobing edge of DQS than when not in this mode.

9.4.3.6.31 DBYTE module controls to select X4 Dram device mode (MasterX4Config)

9.4.3.6.31.1 Offset

Register	Offset
MasterX4Config	4Ah

9.4.3.6.31.2 Diagram



9.4.3.6.31.3 Fields

Field	Function
15-4 —	Reserved
3-0 X4TG	Set to 1 if this Timing Group/Rank is x4 (as opposed to x8) memory. Set to 1 if this Timing Group/Rank is x4 (as opposed to x8) memory. XXX[1 0] : TxReqDest/RxReqDest 0 X4/X8. XX[1 0]X : TxReqDest/RxReqDest 1 X4/X8. X[1 0]XX : TxReqDest/RxReqDest 2 X4/X8. [1 0]XXX : TxReqDest/RxReqDest 3 X4/X8. This CSR must be written before dfi_init_start is asserted.

9.4.3.6.32 Write level feedback DQ observability select. (WrLevBits)

9.4.3.6.32.1 Offset

Register	Offset
WrLevBits	4Ch

9.4.3.6.32.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								WrLevForDQSU				WrLevForDQSL			
W																
Reset	0	0	0	0	0	0	0	0	1	0	0	1	1	0	0	0

9.4.3.6.32.3 Fields

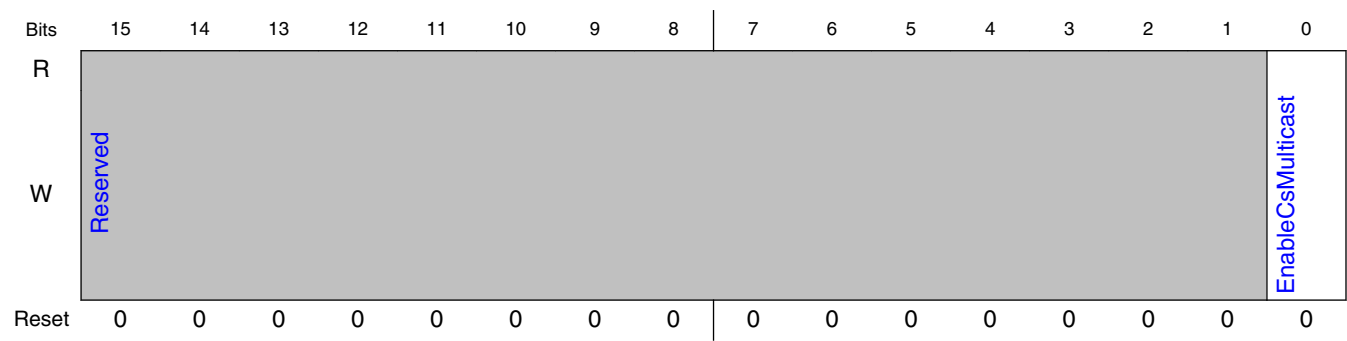
Field	Function
15-8 —	Reserved
7-4 WrLevForDQSU	Indicates which DQ bit is used for Write Levelization. Indicates which DQ bit is used for Write Levelization. 0000 - use DQ0 ... 0111 - use DQ7 1000 - use IDQ[3:0] 1001 - use IDQ[7:4] 1010 - use IDQ[7:0]
3-0 WrLevForDQSL	Indicates which DQ bit is used for Write Levelization. Indicates which DQ bit is used for Write Levelization. 0000 - use DQ0 ... 0111 - use DQ7 1000 - use IDQ[3:0] 1001 - use IDQ[7:4] 1010 - use IDQ[7:0]

9.4.3.6.33 In DDR4 Mode , this controls whether CS_N[3:2] should be multicast on CID[1:0] (EnableCsMulticast)

9.4.3.6.33.1 Offset

Register	Offset
EnableCsMulticast	4Eh

9.4.3.6.33.2 Diagram



9.4.3.6.33.3 Fields

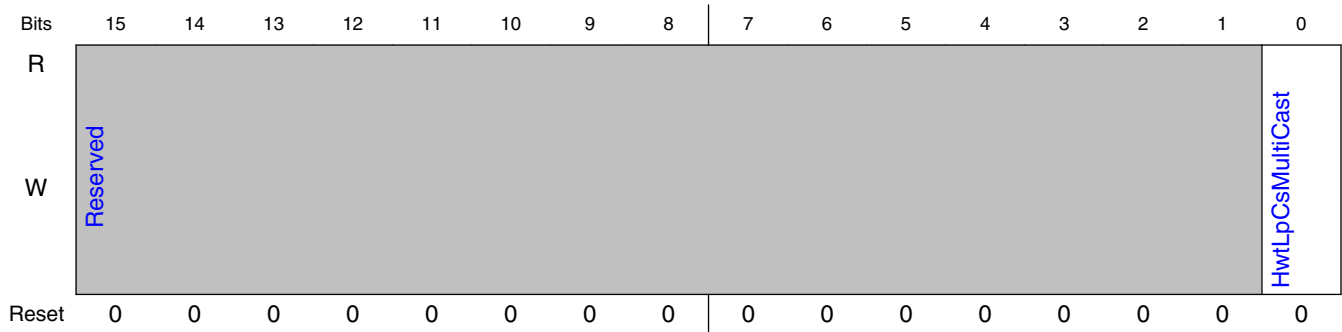
Field	Function
15-1	Reserved
0	In DDR4 Mode , this controls whether CS_N[3:2] should be multicast on CID[1:0] 0 - Do not override pins corresponding to cid[1:0] (dfi_cid[1:0] will connect to the pads) 1 - Overrirde pins corresponding to cid[1:0] with dfi_cs[3:2].
EnableCsMultiCast	In DDR4 Mode , this controls whether CS_N[3:2] should be multicast on CID[1:0] 0 - Do not override pins corresponding to cid[1:0] (dfi_cid[1:0] will connect to the pads) 1 - Overrirde pins corresponding to cid[1:0] with dfi_cs[3:2]. (cs[3:2] will be multicast on cs[3:2] and cid[1:0])

9.4.3.6.34 Drives cs_n[0] onto cs_n[1] during training (HwtLpCsMultiCast)

9.4.3.6.34.1 Offset

Register	Offset
HwtLpCsMultiCast	50h

9.4.3.6.34.2 Diagram



9.4.3.6.34.3 Fields

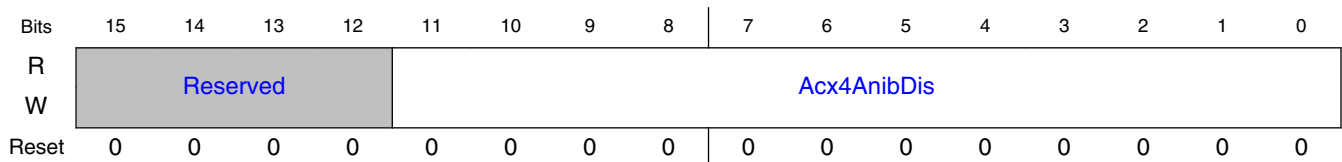
Field	Function
15-1 —	Reserved
0 HwtLpCsMultiCast	When set, drives cs_n[0] onto cs_n[1] during training

9.4.3.6.35 Disable for unused ACX Nibbles (Acx4AnibDis)

9.4.3.6.35.1 Offset

Register	Offset
Acx4AnibDis	58h

9.4.3.6.35.2 Diagram



9.4.3.6.35.3 Fields

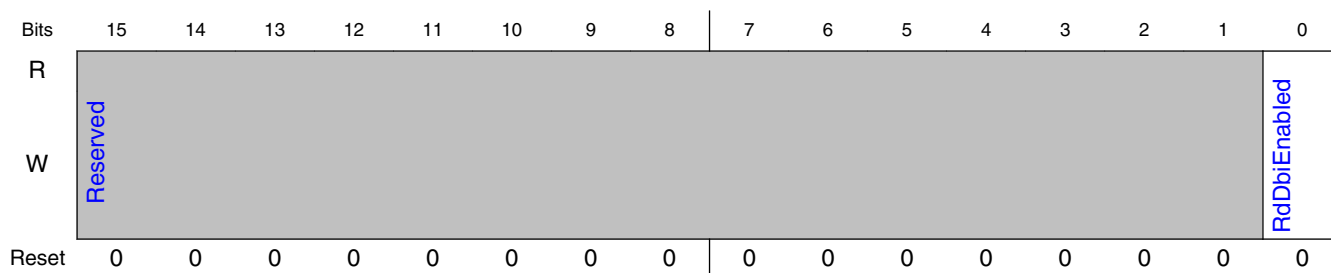
Field	Function
15-12 —	Reserved
11-0 Acx4AnibDis	When a bit is set, the corresponding ACX nibble is disabled (specifically, the I/O OE is disabled, as is the Dfi-side FIFO clock)

9.4.3.6.36 This Register is used to enable the Read-DBI function in each DBYTE (DMIPinPresent_p0)

9.4.3.6.36.1 Offset

Register	Offset
DMIPinPresent_p0	5Ah

9.4.3.6.36.2 Diagram



9.4.3.6.36.3 Fields

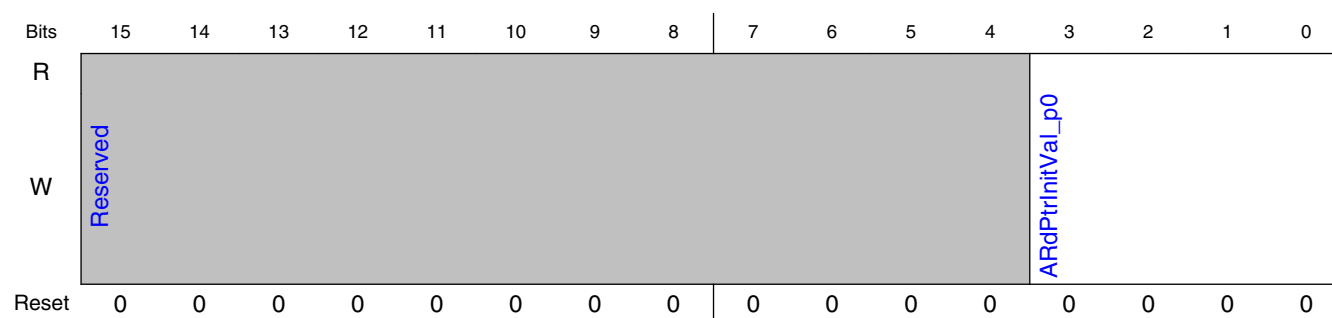
Field	Function
15-1 —	Reserved
0 RdDbiEnabled	This bit must be set to 1'b1 if Read-DBI is enabled in a connected DDR4 or LPDDR4 device. This bit must be set to 1'b1 if Read-DBI is enabled in a connected DDR4 or LPDDR4 device. If set, the following DRAM MR should also be set [DDR4.MR5.A12=1 or LPDDR4.MR3.OP[7]=1]

9.4.3.6.37 Address/Command FIFO ReadPointer Initial Value (ARdPtrInitVal_p0)

9.4.3.6.37.1 Offset

Register	Offset
ARdPtrInitVal_p0	5Ch

9.4.3.6.37.2 Diagram



9.4.3.6.37.3 Fields

Field	Function
15-4 —	Reserved
3-0 ARdPtrInitVal_p0	This is the initial Pointer Offset for the free-running FIFOs in the DBYTE and ACX4 hardips. This is the initial Pointer Offset for the free-running FIFOs in the DBYTE and ACX4 hardips. The units of this register are in UI. The pointer separation should be chosen to compensate for all sources of skew and drift of the PHY DFICLK and PCLK networks. Please see PUB databook section 8.1.1 This CSR must be programmed in Step C of the PHY Initialization sequence

9.4.3.6.38 DLL Mode control CSR for DBYTEs (DbyteDllModeCntrl)

9.4.3.6.38.1 Offset

Register	Offset
DbyteDllModeCntrl	74h

9.4.3.6.38.2 Diagram



9.4.3.6.38.3 Fields

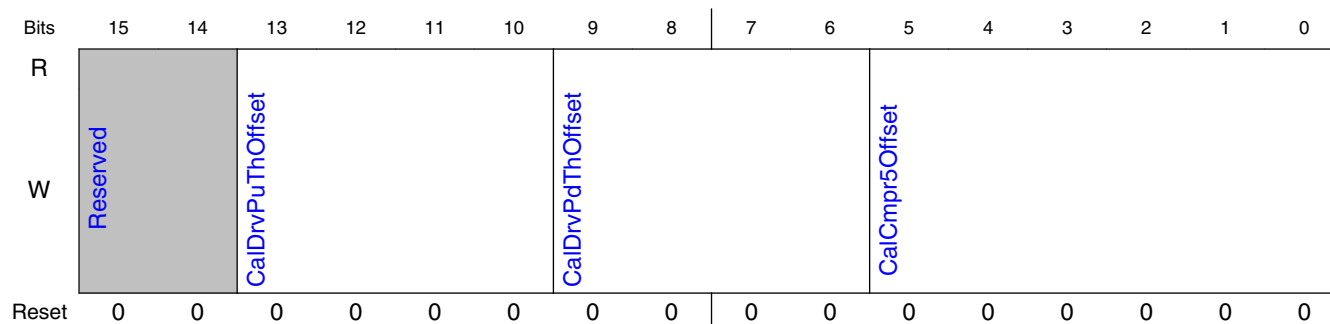
Field	Function
15-2 —	Reserved
1 DIIRxPreambleMode	Must be set to 1 if read DQS preamble contains a toggle, for example DDR4 or LPDDR4 read toggling preamble mode
0 —	Reserved

9.4.3.6.39 Impedance Calibration offsets control (CalOffsets)

9.4.3.6.39.1 Offset

Register	Offset
CalOffsets	8Ah

9.4.3.6.39.2 Diagram



9.4.3.6.39.3 Fields

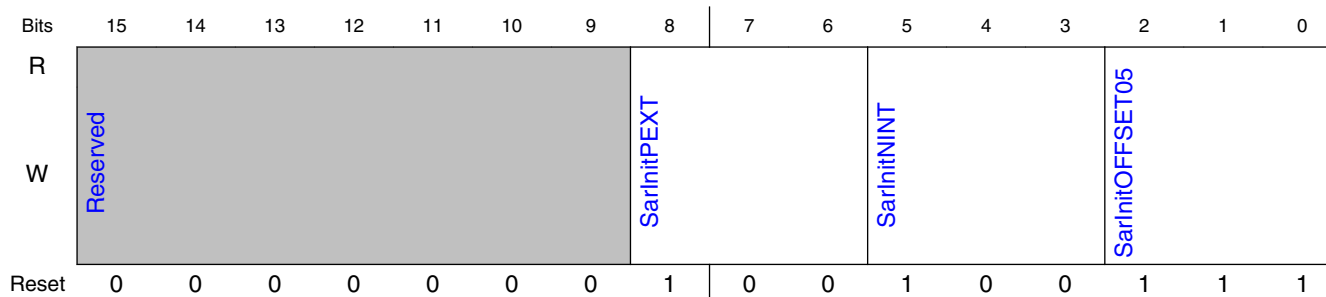
Field	Function
15-14 —	Reserved
13-10 CalDrvPuThOffset	This value adjusts the driver pullup calibration code
9-6 CalDrvPdThOffset	This value adjusts the driver pulldown calibration code
5-0 CalCmpr5Offset	This value adjusts the offset-compensated DAC code for the cmpa circuit at VRef == 0. This value adjusts the offset-compensated DAC code for the cmpa circuit at VRef == 0.5 * VDDQ.

9.4.3.6.40 Sar Init Vals (SarInitVals)

9.4.3.6.40.1 Offset

Register	Offset
SarInitVals	8Eh

9.4.3.6.40.2 Diagram



9.4.3.6.40.3 Fields

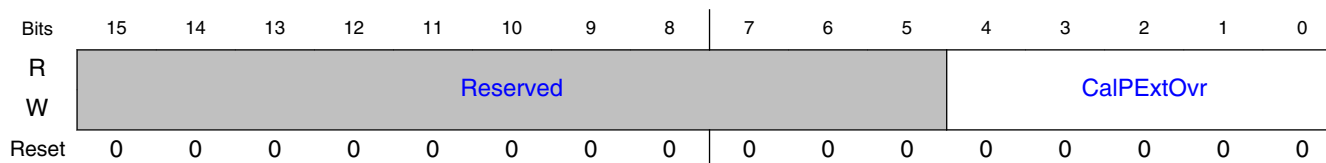
Field	Function
15-9 —	Reserved
8-6 SarInitPEXT	Specify the SAR starting value for PEXT calibration. Specify the SAR starting value for PEXT calibration. This is a 5 bit wide target.
5-3 SarInitNINT	Specify the SAR starting value for NINT calibration. Specify the SAR starting value for NINT calibration. This is a 5 bit wide target.
2-0 SarInitOFFSET05	Specify the SAR starting value for OFFSET05 calibration. Specify the SAR starting value for OFFSET05 calibration. This is an 8 bit wide target.

9.4.3.6.41 Impedance Calibration PExt Override control (CalPExtOvr)

9.4.3.6.41.1 Offset

Register	Offset
CalPExtOvr	92h

9.4.3.6.41.2 Diagram



9.4.3.6.41.3 Fields

Field	Function
15-5 —	Reserved
4-0 CalPExtOvr	If the CSR CalPExtDis is set then the value provided here by software will be used instead of the automatically generated value which is visible via CSR CalPExt This CSR may only be written when the calibrator is not running.

9.4.3.6.42 Impedance Calibration Cmpr 50 control (CalCmpr5Ovr)

9.4.3.6.42.1 Offset

Register	Offset
CalCmpr5Ovr	94h

9.4.3.6.42.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								CalCmpr5Ovr							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.4.3.6.42.3 Fields

Field	Function
15-8 —	Reserved
7-0 CalCmpr5Ovr	If the CSR CalCmpr5Dis is set then the value provided here by software will be used instead of the automatically generated value which is visible via CSR CalCmpr5 This CSR may only be written when the calibrator is not running.

9.4.3.6.43 Impedance Calibration NInt Override control (CalNIntOvr)

9.4.3.6.43.1 Offset

Register	Offset
CalNIntOvr	96h

9.4.3.6.43.2 Diagram



9.4.3.6.43.3 Fields

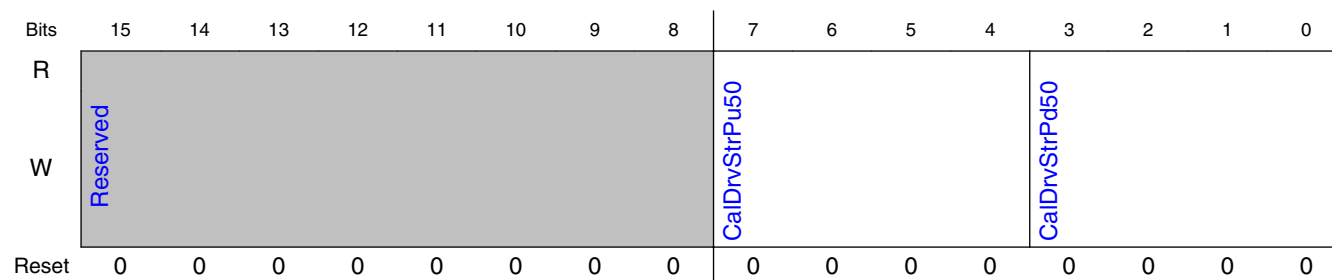
Field	Function
15-5 —	Reserved
4-0 CalNIntOvr	If the CSR CalNIntDis is set then the value provided here by software will be used instead of the automatically generated value which is visible via CSR CalNInt. If the CSR CalNIntDis is set then the value provided here by software will be used instead of the automatically generated value which is visible via CSR CalNInt. This CSR may only be written when the calibrator is not running.

9.4.3.6.44 Impedance Calibration driver strength control (CalDrvStr0)

9.4.3.6.44.1 Offset

Register	Offset
CalDrvStr0	A0h

9.4.3.6.44.2 Diagram



9.4.3.6.44.3 Fields

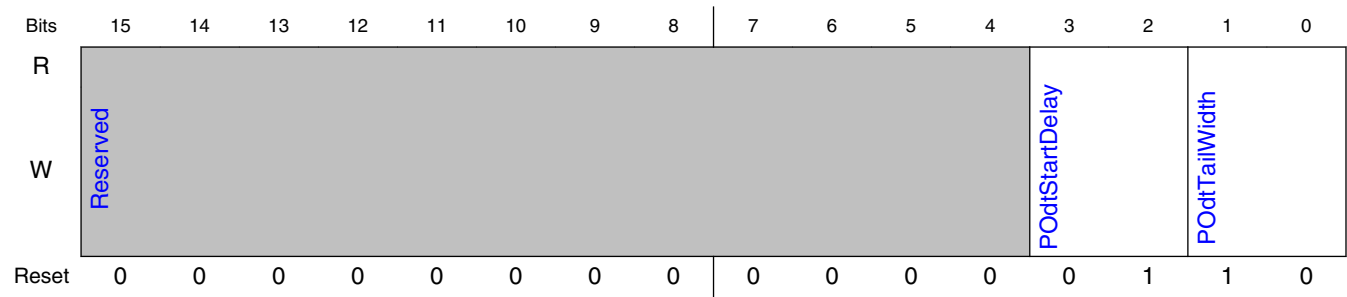
Field	Function
15-8 —	Reserved
7-4 CalDrvStrPu50	3. 3..15 = Reserved for future. 2 - 40.0R 1 - 120.0R 0 - 240.0R (Default)
3-0 CalDrvStrPd50	3. 3..15 = Reserved for future. 2 - 40.0R 1 - 120.0R 0 - 240.0R (Default)

9.4.3.6.45 READ DATA On-Die Termination Timing Control (by PHY) (ProcOdtTimeCtl_p0)

9.4.3.6.45.1 Offset

Register	Offset
ProcOdtTimeCtl_p0	ACh

9.4.3.6.45.2 Diagram



9.4.3.6.45.3 Fields

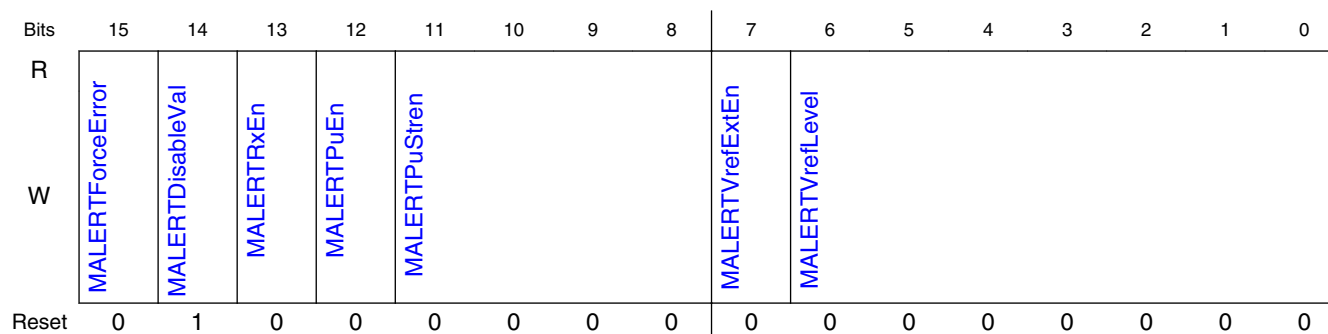
Field	Function
15-4 —	Reserved
3-2 POdtStartDelay	controls the start of ProcOdt, units of UI 3 delay start 2 UI, maximum delay of start of ProcOdt 2 delay start 1 UI, 1 delay start 0 UI, default 0 early by 1 UI, The time from ProcODT assertion to opening the window to receive DQS is (10 - POdtStartDelay) UI.
1-0 POdtTailWidth	controls the length of the tail of ProcOdt, units of UI 3 tail 3UI more than for Register POdtTailWidth=0, maximum 2 tail 2UI more than for Register POdtTailWidth=0, default 1 tail 1UI more than for Register POdtTailWidth=0 0 minimum length tail The time from ProcODT to closing the window to receive DQS to ProcODT POdtTailWidth is (2 + POdtTailWidth) UI

9.4.3.6.46 This Register is used to configure the MemAlert Receiver (MemAlertControl)

9.4.3.6.46.1 Offset

Register	Offset
MemAlertControl	B6h

9.4.3.6.46.2 Diagram



9.4.3.6.46.3 Fields

Field	Function
15 MALERTForceError	When MALERTForceError is set, this CSR state is used to force parity error to memory.
14 MALERTDisableVal	When MALERTRxEn is not set, this CSR state is used to drive dfi_alert_n.
13 MALERTRxEn	1 - Enables receiver and received data is forwarded to dfi_alert_n. 1 - Enables receiver and received data is forwarded to dfi_alert_n. 0 - Disable the receiver and send MALERTDisableVal to dfi_alert_n
12 MALERTPuEn	When set, enables the Pull-up termination on MALERT
11-8 MALERTPuStren	Controls the Pull-up termination on MALERT ===== bit[8] - controls a 240 Ohm Pull-up leg bit[9] - controls a 240 Ohm Pull-up leg bit[10] - controls a 120 Ohm Pull-up leg bit[11] - controls a 120 Ohm Pull-up leg ===== 0000 - No PullUp Strength 0001 - 240 Ohm PullUp Strength 0010 - 240 Ohm PullUp Strength 0011 - 120 Ohm PullUp Strength 0100 - 120 Ohm PullUp Strength 0101 - 80 Ohm PullUp Strength 0110 - 80 Ohm PullUp Strength 0111 - 60 Ohm PullUp Strength 1000 - 120 Ohm PullUp Strength 1001 - 80 Ohm PullUp Strength 1010 - 80 Ohm PullUp Strength 1011 - 60 Ohm PullUp Strength 1100 - 60 Ohm PullUp Strength 1101 - 48 Ohm PullUp Strength 1110 - 48 Ohm PullUp Strength 1111 - 40 Ohm PullUp Strength
7 MALERTVrefExtEn	When set for test/debug, selects external Vref source, This should not be set in mission mode.
6-0 MALERTVrefLevel	Sets the vref level of internal VREF DAC. Sets the vref level of internal VREF DAC. Assuming the following Mission mode settings: TestExtVrefRange, TestMajorMode = 0011 or 0000) The VREF level chosen by the receiver follows the equation:: $VREF = VDDQ * (0.51 + MALERTVrefLevel[6:0] * 0.00345)$

DDR PHY (DDR_PHY)

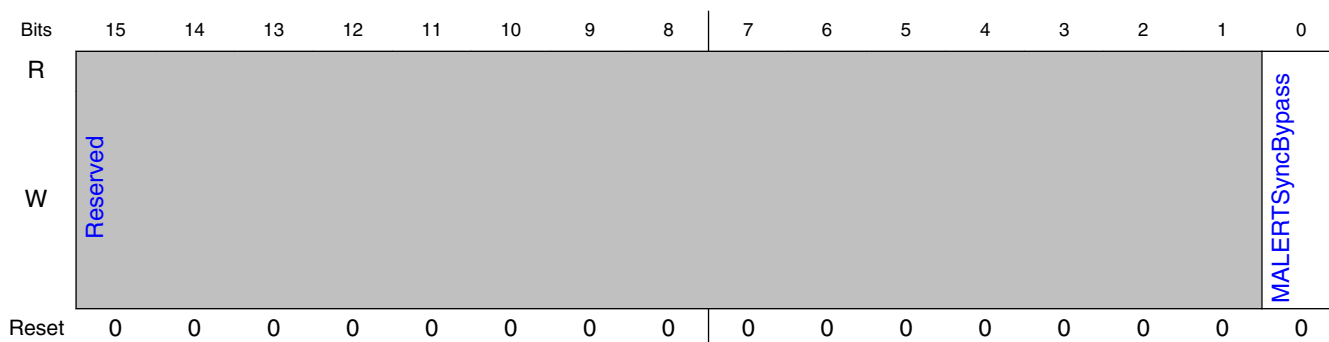
Field	Function
	The min/max values possible are: VREF_min = VDDQ*(0.51) ; VREF_max = VDDQ*(0.948)

9.4.3.6.47 This Register is used to configure the MemAlert Receiver (MemAlertControl2)

9.4.3.6.47.1 Offset

Register	Offset
MemAlertControl2	B8h

9.4.3.6.47.2 Diagram



9.4.3.6.47.3 Fields

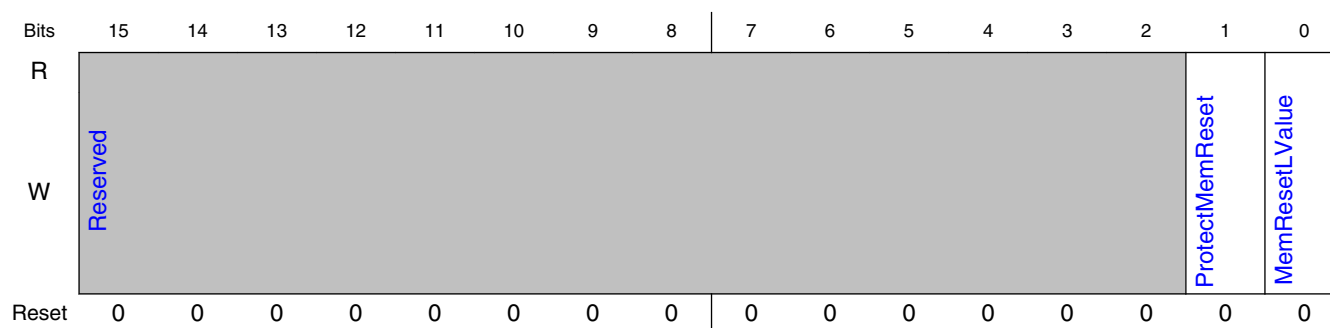
Field	Function
15-1 —	Reserved
0 MALERTSyncBypass	MALERTSyncBypass==[0], the phy will drive dfi_alert_n with a synchronized value of the ALERT_N receiver. MALERTSyncBypass==[0], the phy will drive dfi_alert_n with a synchronized value of the ALERT_N receiver. MALERTSyncBypass==[1], the phy will drive dfi_alert_n with the asynchronous value of the ALERT_N receiver.

9.4.3.6.48 Protection and control of BP_MemReset_L (MemResetL)

9.4.3.6.48.1 Offset

Register	Offset
MemResetL	C0h

9.4.3.6.48.2 Diagram



9.4.3.6.48.3 Fields

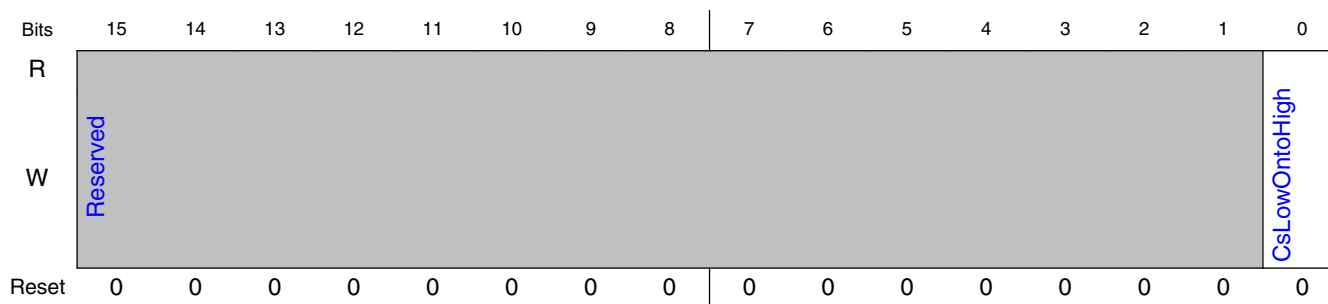
Field	Function
15-2 —	Reserved
1 ProtectMemReset	Control the MemResetL output of the PHY. Control the MemResetL output of the PHY. The state of the MemResetL field is transferred to the PHY BP_MemReset_L output when ProtectMemReset is set.
0 MemResetLValue	Control the MemResetL output of the PHY. Control the MemResetL output of the PHY. The state of this field is transferred to the PHY BP_MemReset_L output when ProtectMemReset is set. BP_MemReset_L cannot reset LPDDR3.

9.4.3.6.49 Drive CS_N 3:0 onto CS_N 7:4 (DriveCSLowOntoHigh)

9.4.3.6.49.1 Offset

Register	Offset
DriveCSLowOntoHigh	DAh

9.4.3.6.49.2 Diagram



9.4.3.6.49.3 Fields

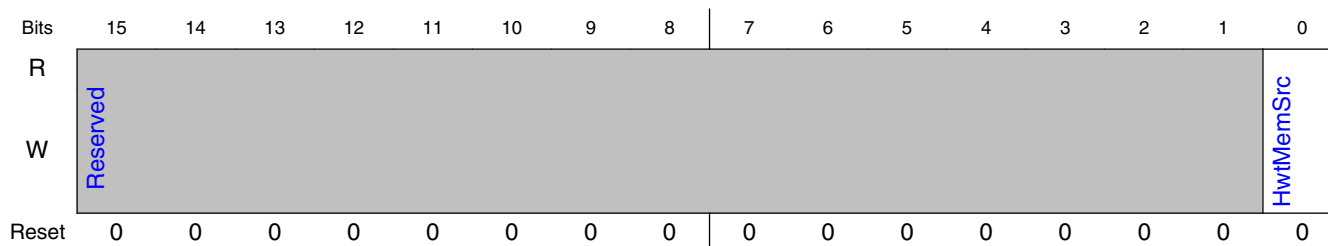
Field	Function
15-1	Reserved
0	When this is set to a 1, CS[3:0] from the ACSM are driven to CS[7:4] pins and CS[3:0] are deasserted.
CsLowOntoHigh	When this is set to a 1, CS[3:0] from the ACSM are driven to CS[7:4] pins and CS[3:0] are deasserted. When this is set to a 0, CS[3:0] from the ACSM are driven to CS[3:0] pins and CS[7:4] are deasserted. This is used only in DDR4 DirectQuadCS ACX13 mode

9.4.3.6.50 PUBMODE - HWT Mux Select (PUBMODE)

9.4.3.6.50.1 Offset

Register	Offset
PUBMODE	DCh

9.4.3.6.50.2 Diagram



9.4.3.6.50.3 Fields

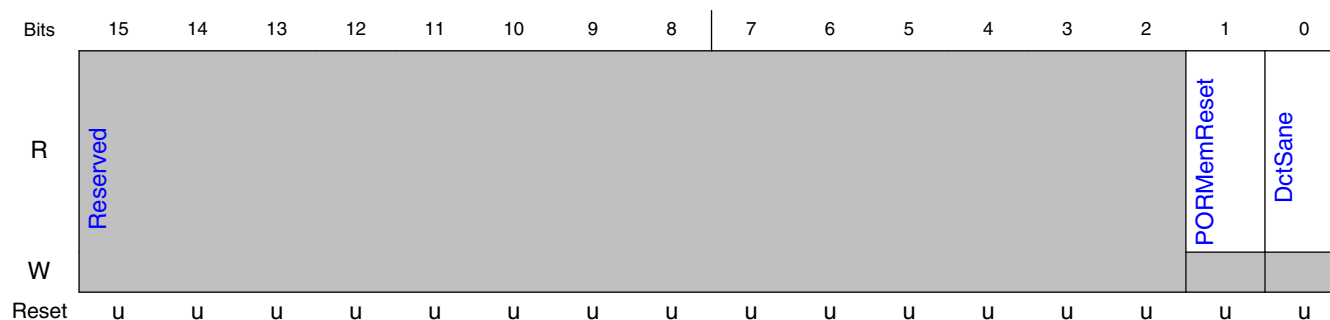
Field	Function
15-1 —	Reserved
0 HwtMemSrc	When this is set to a 1, the mux that switches between DCT and HWT for the source of memory transactions is switched to HWT. When this is set to a 1, the mux that switches between DCT and HWT for the source of memory transactions is switched to HWT. Setting this bit relinquishes control to the HWT. Clearing this bit relinquishes control to the DCT. The transition can only be made when the FIFOs are all in PtrInit (i.e. all PtrInit signals are high).

9.4.3.6.51 Misc PHY status bits (MiscPhyStatus)

9.4.3.6.51.1 Offset

Register	Offset
MiscPhyStatus	DEh

9.4.3.6.51.2 Diagram



9.4.3.6.51.3 Fields

Field	Function
15-2	Reserved

Table continues on the next page...

DDR PHY (DDR_PHY)

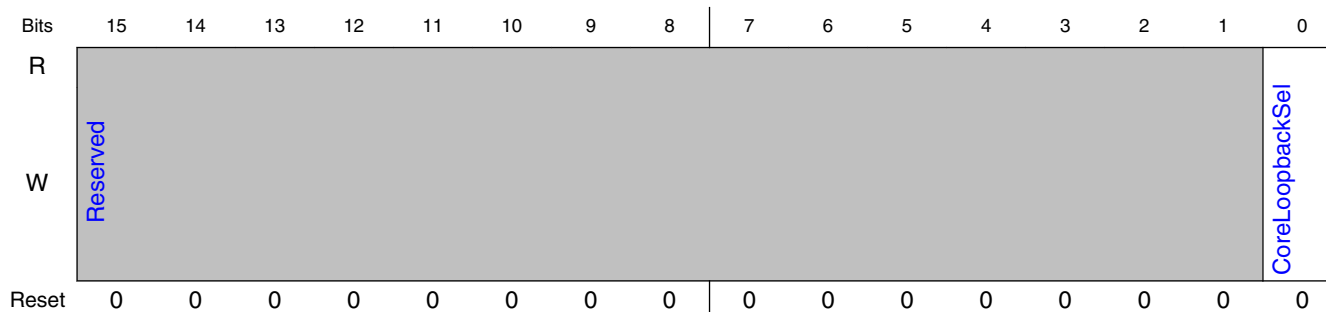
Field	Function
—	
1 PORMemReset	Returns the active-high value used by the custom circuit which drives the memory RESET signal. Returns the active-high value used by the custom circuit which drives the memory RESET signal. To comply with JEDEC specifications for memory reset, this signal powers on 1'b1. This signal is reset during the PHY Initialization sequence.
0 DctSane	Returns the status of the custom circuit which protects the MemResetL output of the PHY on initial power-on or reset. Returns the status of the custom circuit which protects the MemResetL output of the PHY on initial power-on or reset. This circuit resets to 0 meaning that hardware has control of the MemResetL output. If set to 1, memory RESET is being controlled by Registers MemResetLValue and ProtectMemReset or dfi_reset_n.

9.4.3.6.52 Controls whether the loopback path bypasses the final PAD node. (CoreLoopbackSel)

9.4.3.6.52.1 Offset

Register	Offset
CoreLoopbackSel	E0h

9.4.3.6.52.2 Diagram



9.4.3.6.52.3 Fields

Field	Function
15-1	Reserved

Table continues on the next page...

Field	Function
—	
0 CoreLoopbackSel	This register is controlled by the PHY test firmware This register enables Core-Side loopback operation of the PHY. This register is controlled by the PHY test firmware This register enables Core-Side loopback operation of the PHY. When set, the PHY PAD node will be bypassed, and pre-pad node will be selected for sampling by the PHY RX. Setting this register enables full-speed loopback operation of the PHY regardless of the loading of the pad. This register must be set to 1'b0 for mission mode operation (or pad-side loopback).

9.4.3.6.53 DLL Various Training Parameters (DllTrainParam)

9.4.3.6.53.1 Offset

Register	Offset
DllTrainParam	E2h

9.4.3.6.53.2 Diagram



9.4.3.6.53.3 Fields

Field	Function
15-2 —	Reserved
1-0 ExtendPhdTime	Used by the PHY firmware locking the LCDL delay cells. Used by the PHY firmware locking the LCDL delay cells. The number of extra DfIClk periods (= 4*UI) for the DLL phase-detector outputs to become not-metastable

DDR PHY (DDR_PHY)

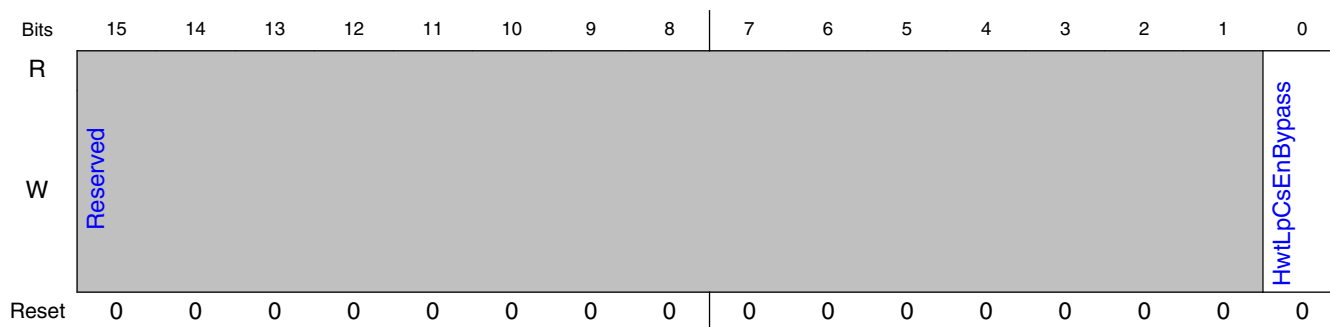
Field	Function
	Recommend to set to 3 if mem CK frequency ge 1800MHz
	Recommend to set to 2 if mem CK frequency lt 1800MHz and ge 1600MHz
	Recommend to set to 1 if mem CK frequency lt 1600MHz and ge 1000MHz
	Recommend to set to 0 if mem CK frequency lt 1000MHz

9.4.3.6.54 CSn Disable Bypass for LPDDR3/4 (HwtLpCsEnBypass)

9.4.3.6.54.1 Offset

Register	Offset
HwtLpCsEnBypass	E8h

9.4.3.6.54.2 Diagram



9.4.3.6.54.3 Fields

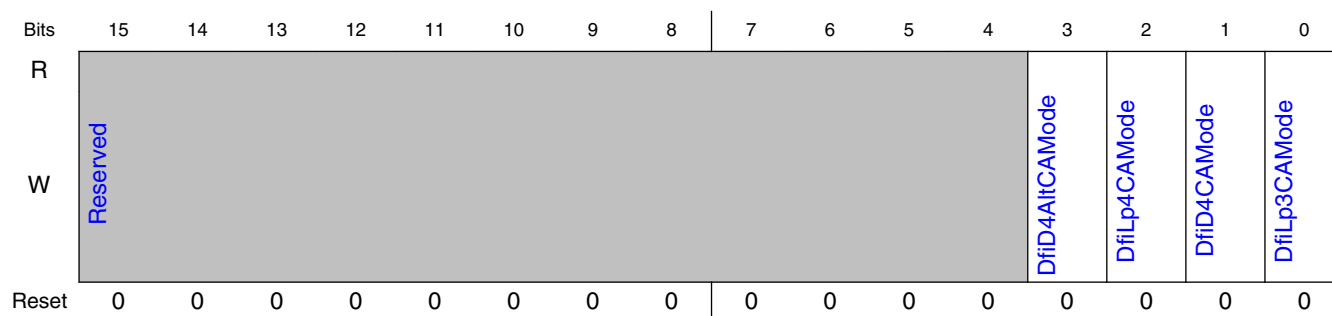
Field	Function
15-1	Reserved
—	
0	When set, these bits disable LpCsEn function for LPDDR3/4
HwtLpCsEnBypass	

9.4.3.6.55 Dfi Command/Address Mode (DfiCAMode)

9.4.3.6.55.1 Offset

Register	Offset
DfiCAMode	EAh

9.4.3.6.55.2 Diagram



9.4.3.6.55.3 Fields

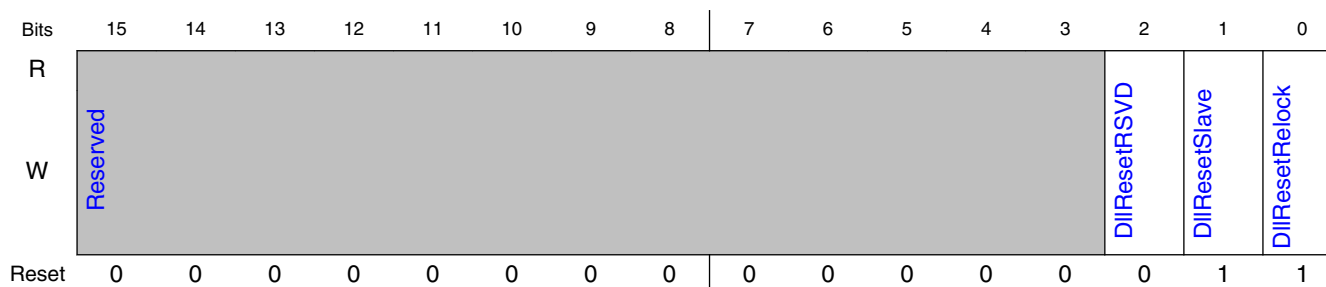
Field	Function
15-4 —	Reserved
3 DfiD4AltCAMode	Enable D4-Alt Mode 0: D4-Altmode disabled 1: D4-Altmode enabled
2 DfiLp4CAMode	Enable LP4 Mode 0: LP4 mode disabled 1: LP4 mode enabled
1 DfiD4CAMode	Enable D4 Mode 0: D4 mode disabled 1: D4 mode enabled
0 DfiLp3CAMode	Controls the output data-rate of the AC module Command/Address pins 0: LP3 DDR address mode disabled 1: LP3 DDR address mode enabled

9.4.3.6.56 DLL Lock State machine control register (DIIControl)

9.4.3.6.56.1 Offset

Register	Offset
DIIControl	F0h

9.4.3.6.56.2 Diagram



9.4.3.6.56.3 Fields

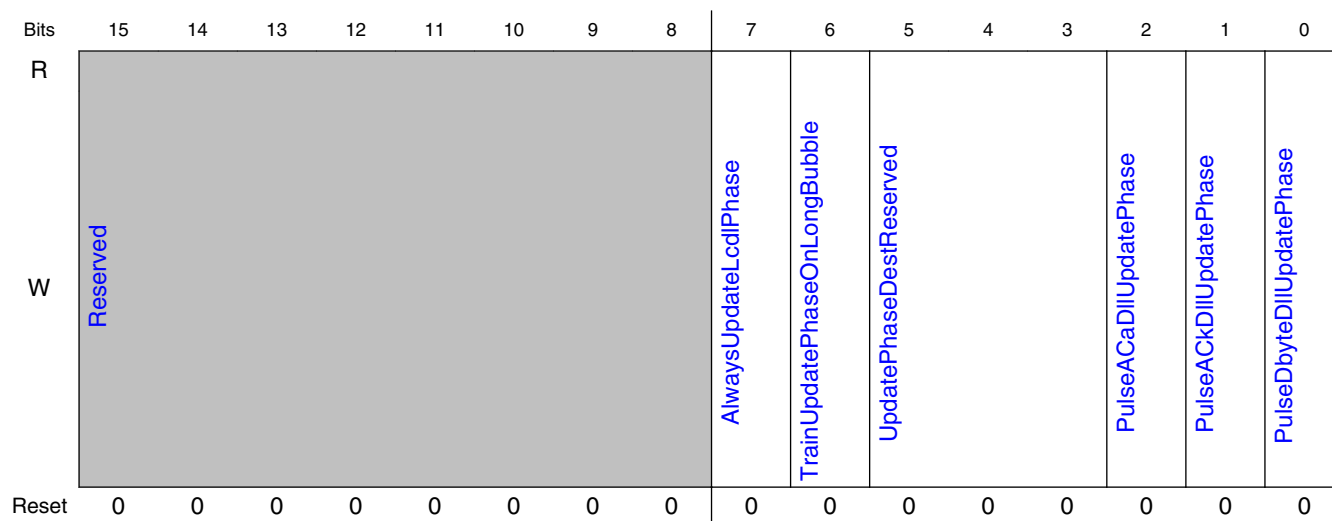
Field	Function
15-3 —	Reserved
2 DIIResetRSVD	RSVD for future use
1 DIIResetSlave	Reserved
0 DIIResetRelock	Used to reset the DDL/LCDL lock state machine Deasserting starts locking sequence. Used to reset the DDL/LCDL lock state machine Deasserting starts locking sequence. This must be done anytime the DLLs are to be relocked (e.g. frequency change).

9.4.3.6.57 DLL update phase control (PulseDIIUpdatePhase)

9.4.3.6.57.1 Offset

Register	Offset
PulseDIIUpdatePhase	F2h

9.4.3.6.57.2 Diagram



9.4.3.6.57.3 Fields

Field	Function
15-8 —	Reserved
7 AlwaysUpdateLcdIPhase	Causes each new operation to reload the LcdIPhase; will increase bubbles.
6 TrainUpdatePhaseOnLongBubble	Causes LongBubble to update the dbyte & anib LDCL Phase. Causes LongBubble to update the dbyte & anib LDCL Phase. May be used for training, but not to be used for mission mode.
5-3 UpdatePhaseDestReserved	reserved, not used
2 PulseACaDIIUpdatePhase	Causes an AC module CA (command/address/cke/odt) DLL phase update.
1 PulseACkDIIUpdatePhase	Causes an AC module CK (memck) DLL phase update. Causes an AC module CK (memck) DLL phase update. This should be written with 1 only when the memory CKE=0 or memory RST=0, or when there is no memory, as in tester mode.
0 PulseDbyteDIIUpdatePhase	Causes a LongBubble to the DBYTE modules, which causes a update of the DBYTE module DLLs (tx,rxen,rxclk). Causes a LongBubble to the DBYTE modules, which causes a update of the DBYTE module DLLs (tx,rxen,rxclk). The value of Register UpdatePhaseDest must be setup prior to this pulse.

9.4.3.6.58 DLL gain control (DlIGainCtl_p0)

9.4.3.6.58.1 Offset

Register	Offset
DlIGainCtl_p0	F8h

9.4.3.6.58.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				DlISeedSel				DlIGainTV				DlIGainIV			
W																
Reset	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	1

9.4.3.6.58.3 Fields

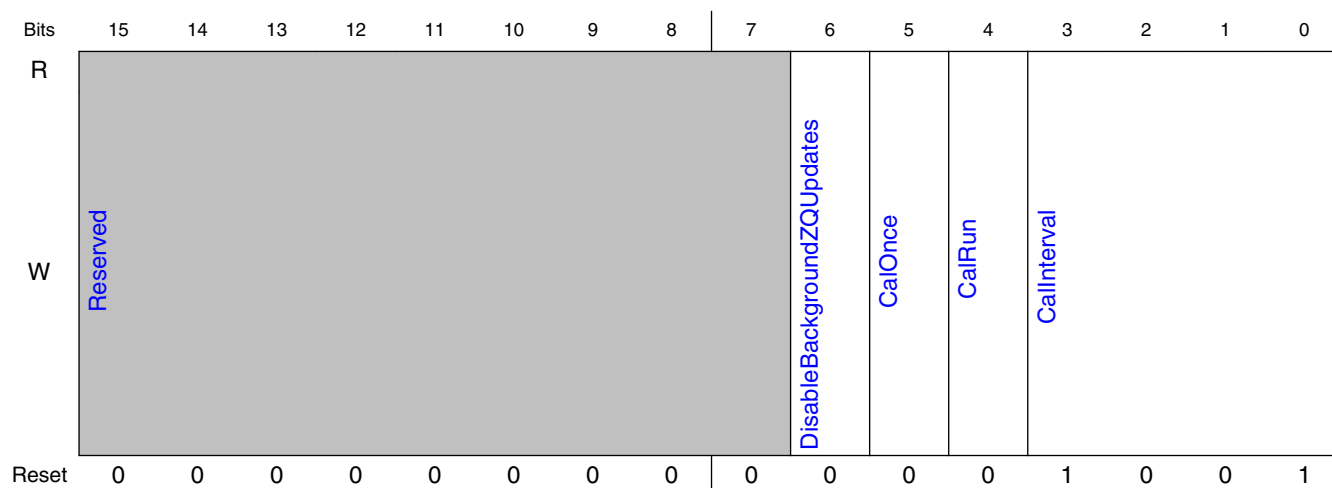
Field	Function
15-12 —	Reserved
11-8 DlISeedSel	Reserved, must be configured to be 0.
7-4 DlIGainTV	Terminal value of DlIGain, ie the value in effect when locking is done and the value used for maintaining lock, ie tracking pclk variation. Terminal value of DlIGain, ie the value in effect when locking is done and the value used for maintaining lock, ie tracking pclk variation. DlIGainTV must be greater than or equal to DlIGainIV. The maximum value is 10. The minimum value is 6.
3-0 DlIGainIV	Initial value of DlIGain.

9.4.3.6.59 Impedance Calibration Control (CalRate)

9.4.3.6.59.1 Offset

Register	Offset
CalRate	110h

9.4.3.6.59.2 Diagram



9.4.3.6.59.3 Fields

Field	Function
15-7 —	Reserved
6 DisableBackgroundZQUpdates	1: Instead of having the driver compensation codes go asynchronously out to all IO, hold until for any of PHYUPD ACK, CTRLUPD ACK, PHYMSTR ACK) 0: Calibrated ZQ Updates to IO aren't gated. 1: Instead of having the driver compensation codes go asynchronously out to all IO, hold until for any of PHYUPD ACK, CTRLUPD ACK, PHYMSTR ACK) 0: Calibrated ZQ Updates to IO aren't gated. The default value of this field is 0 because the minor Impedance corrections (Converted from binary to thermometer) is applied over multiple cycles and the impact caused is minor.
5 CalOnce	The setting of this CSR changes the behaviour of CSR CalRun. The setting of this CSR changes the behaviour of CSR CalRun. 1: The 0->1 transition of CSR CalRun causes a single iteration of the calibration sequence to occur. Once the calibration is complete, the calibration engine will remain IDLE until another CalRun Trigger. 0: Calibration will proceed at the rate determined by Register CalInterval.

Table continues on the next page...

Field	Function
	This field should only be changed while the calibrator is idle. ie before csr CalRun is set.
4 CalRun	1: A calibration sequence will be triggered by the 0->1 transition of this bit, as determined by CSR CalOnce. 1: A calibration sequence will be triggered by the 0->1 transition of this bit, as determined by CSR CalOnce. 0: Calibrator will not run. If already active then it will complete the current sequence before quiescing. The default value of this field is 0 because the calibrator needs to know whether it is in D3 mode before it starts up. This csr won't do anything while CalZap is asserted.
3-0 CalInterval	This CSR specifies the interval between successive calibrations, in mS. This CSR specifies the interval between successive calibrations, in mS. csrValue : Interval (mS) 0 : 0 (continuous) 1 : 0.013 2 : 0.10 3 : 1 4 : 2 5 : 3 6 : 4 7 : 8 8 : 10 9 : 20 10-15 : Reserved. This field should only be changed while the calibrator is idle. ie before csr CalRun is set.

9.4.3.6.60 Impedance Calibration Zap/Reset (CalZap)

9.4.3.6.60.1 Offset

Register	Offset
CalZap	112h

9.4.3.6.60.2 Diagram



9.4.3.6.60.3 Fields

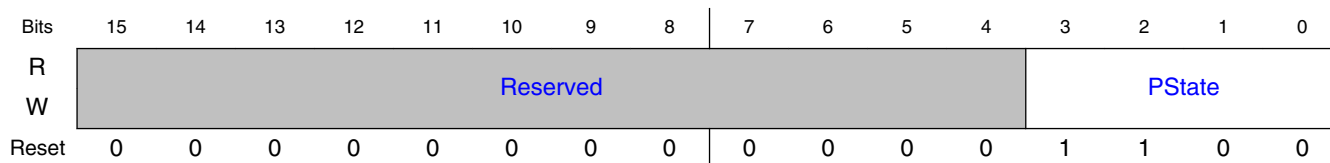
Field	Function
15-1	Reserved
0	NOTE : This CSR is written by PHY Initialization Engine (PIE) and the data in here will be overwritten.
CalZap	NOTE : This CSR is written by PHY Initialization Engine (PIE) and the data in here will be overwritten. Setting this csr resets the calibrator to its idle state. Must be cleared in order for the calibration engine to run again.

9.4.3.6.61 PSTATE Selection (PState)

9.4.3.6.61.1 Offset

Register	Offset
PState	116h

9.4.3.6.61.2 Diagram



9.4.3.6.61.3 Fields

Field	Function
15-4	Reserved

Table continues on the next page...

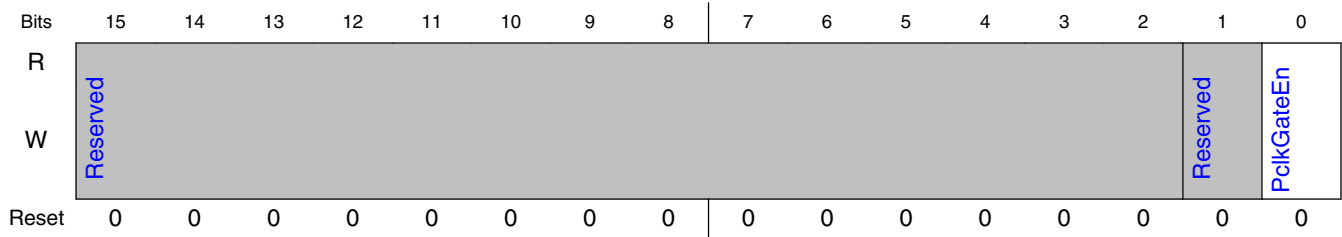
Field	Function
—	
3-0 PState	NOTE : This CSR is written by PHY Initialization Engine (PIE) and the data in here will be overwritten. NOTE : This CSR is written by PHY Initialization Engine (PIE) and the data in here will be overwritten. The current PState inferred by the PIE from handshake requests on DFI status interface. 0000 - PState P0 0001 - PState P1 0010 - PState P2 0011 - PState P3 0100 - Reserved 0101 - Reserved 0110 - Reserved 0111 - Reserved 1000 - Reserved 1001 - Reserved 1010 - Reserved 1011 - Reserved 1100 - LP2 1101 - Reserved 1110 - Reserved 1111 - LP3 The lower two bits of this CSR are used to select the PHY's current Pstate for all Pstateable registers. Note: It is a don't care what value is selected in either LP2 or LP3 because no transactions will be taking place when in those states.

9.4.3.6.62 PLL Output Control (PllOutGateControl)

9.4.3.6.62.1 Offset

Register	Offset
PllOutGateControl	11Ah

9.4.3.6.62.2 Diagram



9.4.3.6.62.3 Fields

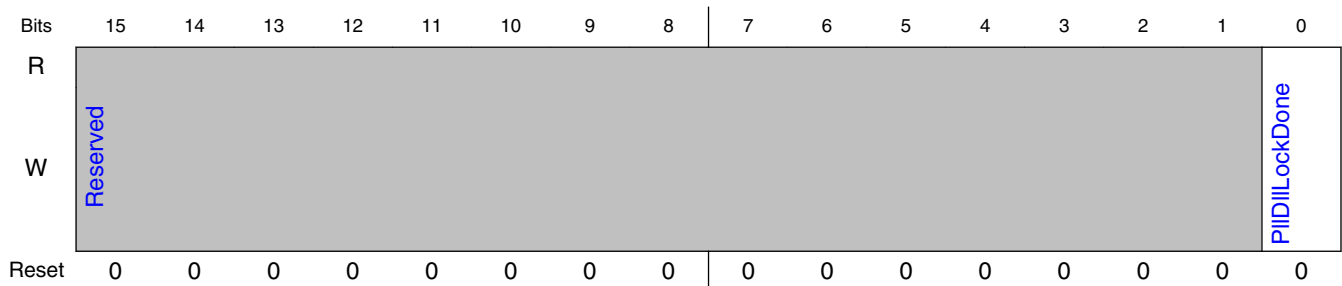
Field	Function
15-2 —	Reserved
1 —	Reserved for future.
0 PclkGateEn	Reserved

9.4.3.6.63 PMU Power-on Reset Control (PLL/DLL Lock Done) (PorControl)

9.4.3.6.63.1 Offset

Register	Offset
PorControl	120h

9.4.3.6.63.2 Diagram



9.4.3.6.63.3 Fields

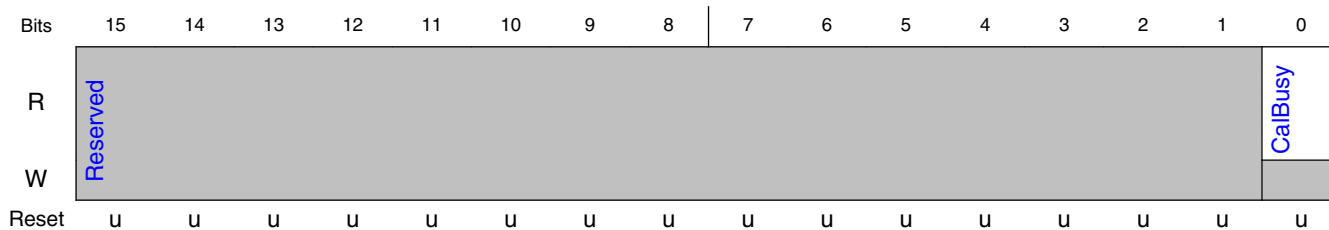
Field	Function
15-1 —	Reserved
0 PIIDILockDone	Set by the PIE to 1 after it has finished the PLL/DLL lock sequence. Set by the PIE to 1 after it has finished the PLL/DLL lock sequence. It is only cleared to 0 when PHY is Reset or exiting LP3 state.

9.4.3.6.64 Impedance Calibration Busy Status (CalBusy)

9.4.3.6.64.1 Offset

Register	Offset
CalBusy	12Eh

9.4.3.6.64.2 Diagram



9.4.3.6.64.3 Fields

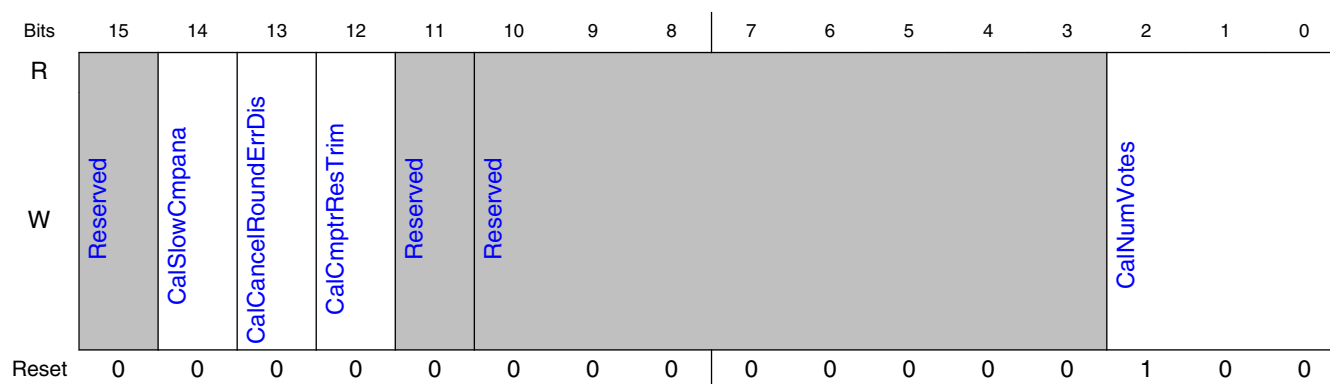
Field	Function
15-1 —	Reserved
0 CalBusy	Read 1 if the calibrator is actively calibrating. Read 1 if the calibrator is actively calibrating. Any changes to calibrator-related CSRs may only be made if the calibrator is disabled (via CSR CalRun) and this CSR reads 0.

9.4.3.6.65 Miscellaneous impedance calibration controls. (CalMisc2)

9.4.3.6.65.1 Offset

Register	Offset
CalMisc2	130h

9.4.3.6.65.2 Diagram



9.4.3.6.65.3 Fields

Field	Function
15 —	Miscellaneous impedance calibration controls.
14 CalSlowCmpna	When set, this CSR increases the time allowed for the cmpna cell to settle, by 50%. When set, this CSR increases the time allowed for the cmpna cell to settle, by 50%. It changes from 3 to 6 refClk ticks.
13 CalCancelRoundErrDis	The PEXT calibration result and NINT calibration results naturally include a rounding error which manifests as a change of impedance at the pad. The PEXT calibration result and NINT calibration results naturally include a rounding error which manifests as a change of impedance at the pad. Where both of these errors are in the same direction the PEXT calibration result will be adjusted by 1 to offset NINT error. Setting this csr to 1 prevents this adjustment.
12 CalCmptrResTrim	Reserved for future use.

Table continues on the next page...

DDR PHY (DDR_PHY)

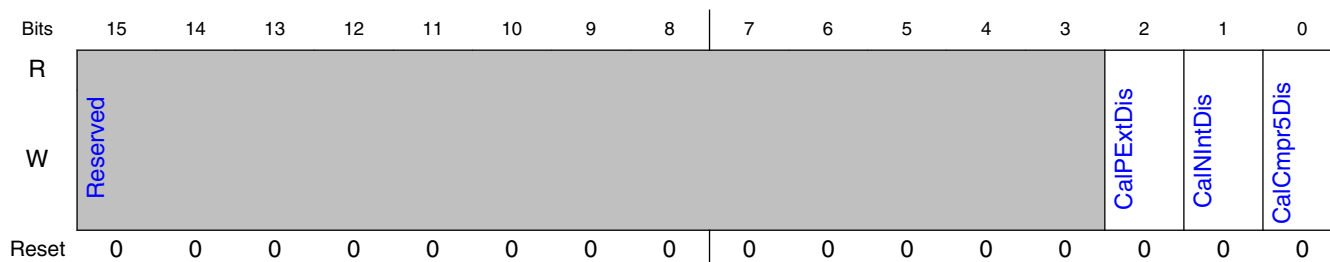
Field	Function
11 —	Reserved for future use. Reserved for future use. Formerly used to Skip ODT calibration if required.
10-3 —	Reserved for future use.
2-0 CalNumVotes	This CSR controls the number of consecutive comparator output bits over which majority voting is done. This CSR controls the number of consecutive comparator output bits over which majority voting is done. The result of the voting is fed into the SAR. The voting is initiated after the time configured via csrs CalSlowCmpana has elapsed. CSR Value : number of Votes 0 : 1 1 : 9 2 : 17 3 : 33 4 : 65 (defaults) 5 : 129 6 : 193 7 : 255

9.4.3.6.66 Controls for disabling the impedance calibration of certain targets. (CalMisc)

9.4.3.6.66.1 Offset

Register	Offset
CalMisc	134h

9.4.3.6.66.2 Diagram



9.4.3.6.66.3 Fields

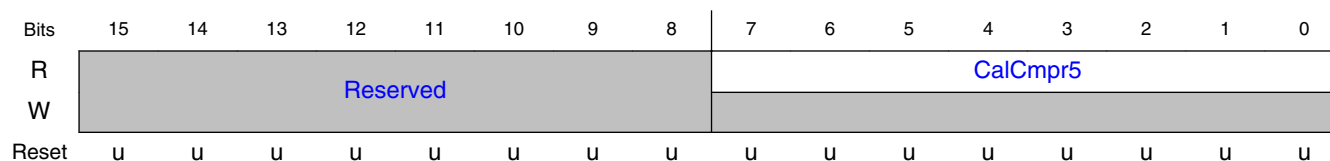
Field	Function
15-3 —	Reserved
2 CalPExtDis	Setting this CSR prevents the calibration engine from overwriting the CSRs TxCalBinP and TxCalThP with an automatically generated value, in which case a value must be supplied by software.
1 CalNIntDis	Setting this CSR prevents the calibration engine from overwriting the CSRs TxCalBinN and TxCalThN with an automatically generated value, in which case a value must be supplied by software.
0 CalCmpr5Dis	Setting this CSR prevents the calibration engine from using the result from the CalCmpr5 stage of calibration. Setting this CSR prevents the calibration engine from using the result from the CalCmpr5 stage of calibration. Software must supply a replacement value via CSR CalCmpr5Ovr.

9.4.3.6.67 Impedance Calibration Cmpr control (CalCmpr5)

9.4.3.6.67.1 Offset

Register	Offset
CalCmpr5	138h

9.4.3.6.67.2 Diagram



9.4.3.6.67.3 Fields

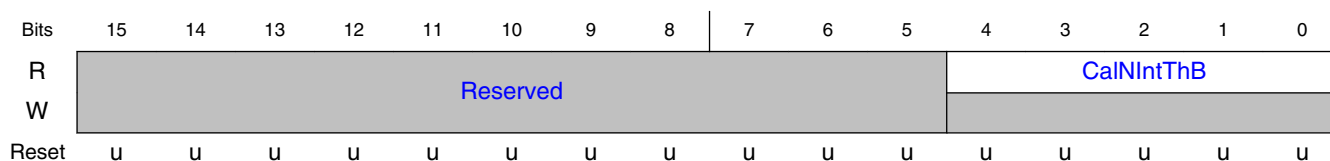
Field	Function
15-8 —	Reserved
7-0 CalCmpr5	Returns the offset-compensated DAC code for the cmpna circuit at VRef == 0. Returns the offset-compensated DAC code for the cmpna circuit at VRef == 0.5 * VDDQ.

9.4.3.6.68 Impedance Calibration NInt control (CalNInt)

9.4.3.6.68.1 Offset

Register	Offset
CalNInt	13Ah

9.4.3.6.68.2 Diagram



9.4.3.6.68.3 Fields

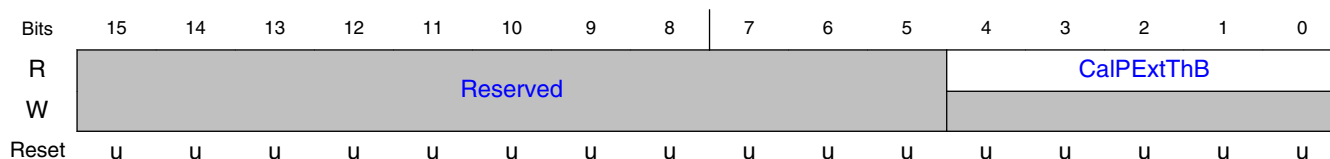
Field	Function
15-5 —	Reserved
4-0 CalNIntThB	The value here is the number of thermometer bits which are set.

9.4.3.6.69 Impedance Calibration PExt control (CalPExt)

9.4.3.6.69.1 Offset

Register	Offset
CalPExt	13Ch

9.4.3.6.69.2 Diagram



9.4.3.6.69.3 Fields

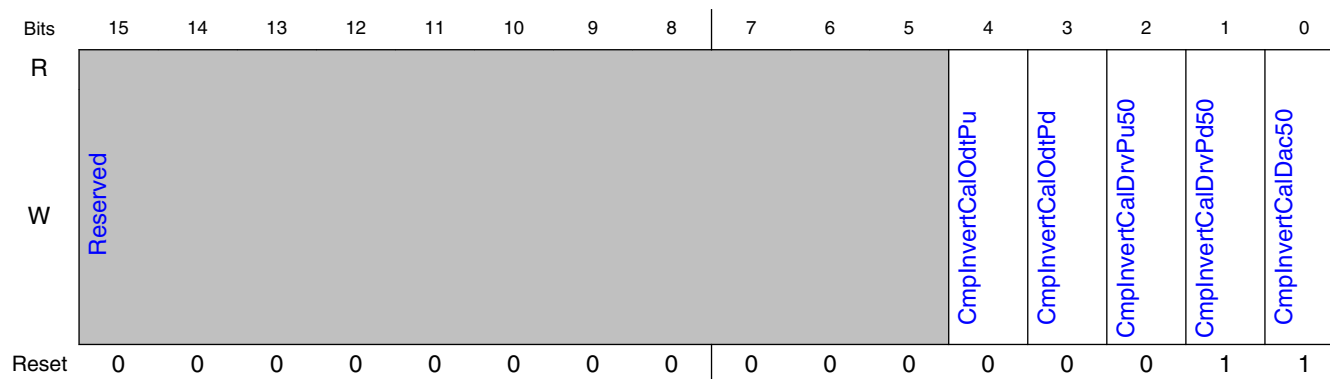
Field	Function
15-5 —	Reserved
4-0 CalPExtThB	The value here is the number of thermometer bits which are set.

9.4.3.6.70 Impedance Calibration Cmp Invert control (CalCmplInvert)

9.4.3.6.70.1 Offset

Register	Offset
CalCmplInvert	150h

9.4.3.6.70.2 Diagram



9.4.3.6.70.3 Fields

Field	Function
15-5 —	Reserved
4 CmplInvertCalOd tPu	Impedance Calibration Cmp Invert control
3	Impedance Calibration Cmp Invert control

Table continues on the next page...

DDR PHY (DDR_PHY)

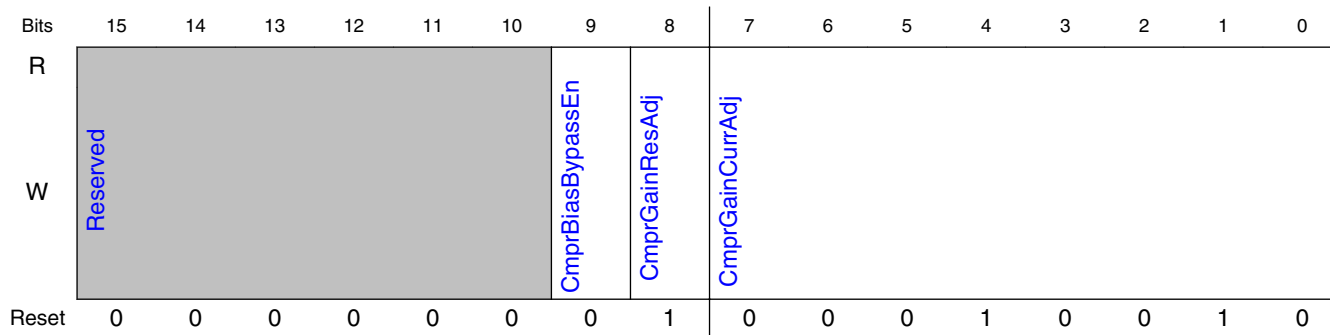
Field	Function
CmplInvertCalOd tPd	
2 CmplInvertCalDr vPu50	Impedance Calibration Cmp Invert control
1 CmplInvertCalDr vPd50	Impedance Calibration Cmp Invert control
0 CmplInvertCalDa c50	Impedance Calibration Cmp Invert control

9.4.3.6.71 Impedance Calibration Cmpana control (CalCmpanaCntrl)

9.4.3.6.71.1 Offset

Register	Offset
CalCmpanaCntrl	15Ch

9.4.3.6.71.2 Diagram



9.4.3.6.71.3 Fields

Field	Function
15-10 —	Reserved
9	Impedance Calibration Cmpana control

Table continues on the next page...

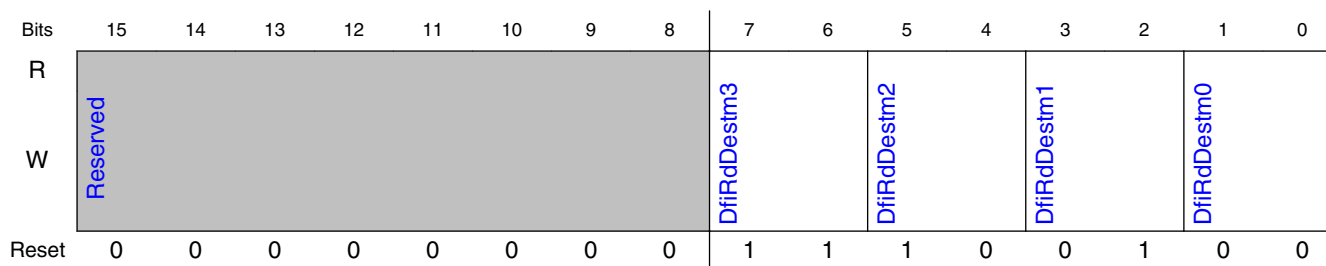
Field	Function
CmprBiasBypassEn	
8 CmprGainResAdj	Impedance Calibration Cmpana control
7-0 CmprGainCurrAdj	Impedance Calibration Cmpana control

9.4.3.6.72 Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiRdDataCsDestMap_p0)

9.4.3.6.72.1 Offset

Register	Offset
DfiRdDataCsDestMap_p0	160h

9.4.3.6.72.2 Diagram



9.4.3.6.72.3 Fields

Field	Function
15-8 —	Reserved
7-6 DfiRdDestm3	Maps dfi_rddata_cs_n_p0[3] to dest DfiRdDestm3 timing For example, if 3 dfi_rddata_cs_n_p0[3] will use Register RxEn,ClkDlyTg3 timing.
5-4 DfiRdDestm2	Maps dfi_rddata_cs_n_p0[2] to dest DfiRdDestm2 timing For example, if 2 dfi_rddata_cs_n_p0[2] will use Register RxEn,ClkDlyTg2 timing.

Table continues on the next page...

DDR PHY (DDR_PHY)

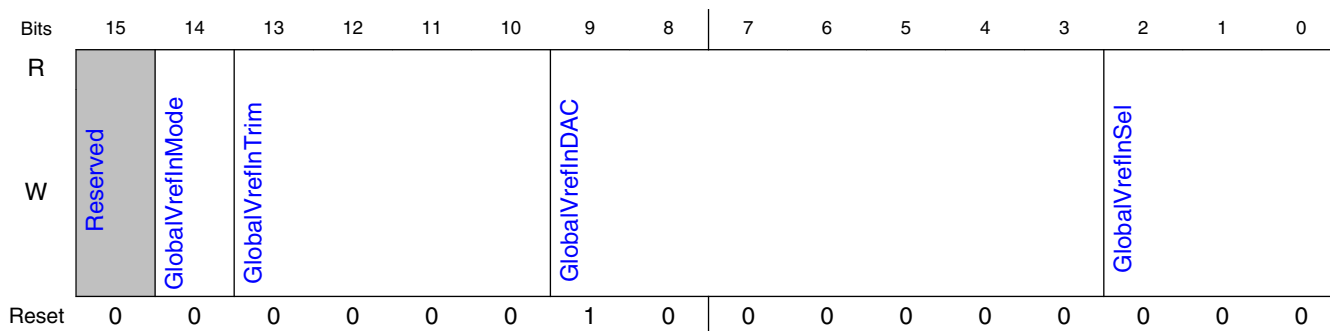
Field	Function
3-2 DfiRdDestm1	Maps dfi_rddata_cs_n_p0[1] to dest DfiRdDestm1 timing For example, if 1 dfi_rddata_cs_n[_p01] will use Register RxEn,ClkDlyTg1 timing.
1-0 DfiRdDestm0	Maps dfi_rddata_cs_n_p0[0] to dest DfiRdDestm0 timing For example, if 0 dfi_rddata_cs_n_p0[0] will use Register RxEn,ClkDlyTg0 timing.

9.4.3.6.73 PHY Global Vref Controls (VrefInGlobal_p0)

9.4.3.6.73.1 Offset

Register	Offset
VrefInGlobal_p0	164h

9.4.3.6.73.2 Diagram



9.4.3.6.73.3 Fields

Field	Function
15 —	Reserved
14 GlobalVrefInMode	RSVD
13-10 GlobalVrefInTrim	RSVD

Table continues on the next page...

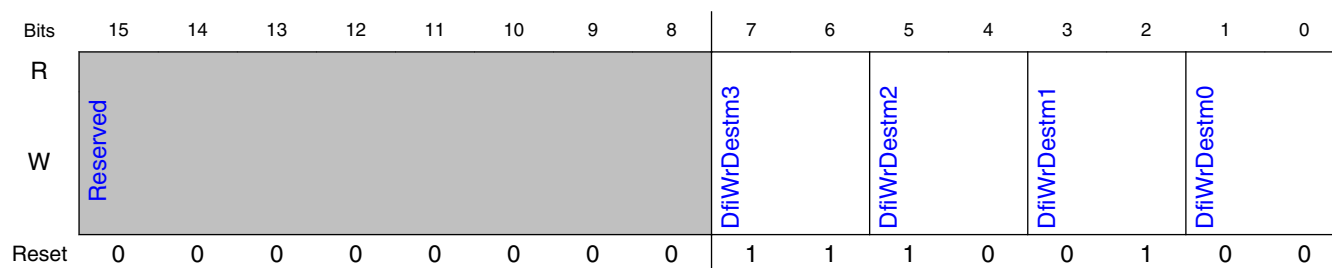
Field	Function
9-3 GlobalVrefInDAC	DAC code for internal Vref generation The DAC has two ranges; the range is set by GlobalVrefInSel[2] ===== RANGE0 : DDR3,DDR4,LPDDR3 [GlobalVrefInSel[2] = 0] DAC Output Voltage = GlobalVrefInDAC == 6'h00 ? Hi-Z : 0. DAC code for internal Vref generation The DAC has two ranges; the range is set by GlobalVrefInSel[2] ===== RANGE0 : DDR3,DDR4,LPDDR3 [GlobalVrefInSel[2] = 0] DAC Output Voltage = GlobalVrefInDAC == 6'h00 ? Hi-Z : $0.345 \cdot VDDQ + (0.005 \cdot \text{GlobalVrefInDAC}) \cdot VDDQ$ ===== RANGE1 : LPDDR4 [GlobalVrefInSel[2] = 1] DAC Output Voltage = GlobalVrefInDAC == 6'h00 ? Hi-Z : $(0.005 \cdot (\text{GlobalVrefInDAC} - 1)) \cdot VDDQ$
2-0 GlobalVrefInSel	GlobalVrefInSel[1:0] controls the mode of the PHY VREF DAC and the BP_VREF pin ===== 2'b00 - PHY Vref DAC Range0 -- BP_VREF = Hi-Z 2'b01 - Reserved Encoding 2'b10 - PHY Vref DAC Range0 -- BP_VREF connected to PLL Analog Bus 2'b11 - PHY Vref DAC Range0 -- BP_VREF connected to PHY Vref DAC ===== GlobalVrefInSel[2] shall be set according to Dram Protocol: Protocol GlobalVrefInSel[2] ----- DDR3 1'b0 DDR4 1'b0 LPDDR3 1'b0 LPDDR4 1'b1

9.4.3.6.74 Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiWrDataCsDestMap_p0)

9.4.3.6.74.1 Offset

Register	Offset
DfiWrDataCsDestMap_p0	168h

9.4.3.6.74.2 Diagram



9.4.3.6.74.3 Fields

Field	Function
15-8 —	Reserved
7-6 DfiWrDestm3	Maps dfi_wrdata_cs_n_p0[3] to dest DfiWrDestm3 timing (use Register TxDq,DqsDlyTg3) For example, if 3 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg3 timing.
5-4 DfiWrDestm2	Maps dfi_wrdata_cs_n_p0[2] to dest DfiWrDestm2 timing For example, if 2 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg2 timing.
3-2 DfiWrDestm1	Maps dfi_wrdata_cs_n_p0[1] to dest DfiWrDestm1 timing For example, if 1 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg1 timing.
1-0 DfiWrDestm0	Maps dfi_wrdata_cs_n_p0[0] to dest DfiWrDestm0 timing For example, if 0 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg0 timing.

9.4.3.6.75 Counts successful PHY Master Interface Updates (PPTs) (MasUpdGoodCtr)

9.4.3.6.75.1 Offset

Register	Offset
MasUpdGoodCtr	16Ah

9.4.3.6.75.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	MasUpdGoodCtr															
W																
Reset	u	u	u	u	u	u	u	u	u	u	u	u	u	u	u	u

9.4.3.6.75.3 Fields

Field	Function
15-0 MasUpdGoodCtr	This register increments whenever the Memory Controller acknowledges a PHY Master Interface request (i.e., a request for the PHY to take over the DFI for Periodic Phase Training (PPT)). (As per section 11 of the Preliminary DFI 4.0 Specification Rev. 2)

Field	Function
	In a 2-channel system, both MC channels must ack. The counter saturates at max value 65535.

9.4.3.6.76 Counts successful PHY-initiated DFI0 Interface Updates (PhyUpd0GoodCtr)

9.4.3.6.76.1 Offset

Register	Offset
PhyUpd0GoodCtr	16Ch

9.4.3.6.76.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PhyUpd0GoodCtr															
W																
Reset	u	u	u	u	u	u	u	u	u	u	u	u	u	u	u	u

9.4.3.6.76.3 Fields

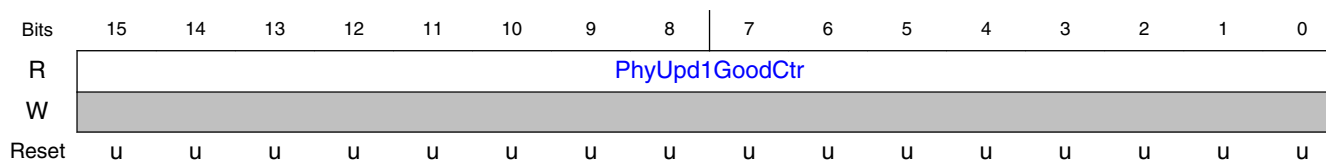
Field	Function
15-0 PhyUpd0GoodCtr	This register increments whenever the Memory Controller acknowledges a PHY-initiated DFI0 interface update request. This register increments whenever the Memory Controller acknowledges a PHY-initiated DFI0 interface update request. The counter saturates at max value 65535.

9.4.3.6.77 Counts successful PHY-initiated DFI1 Interface Updates (PhyUpd1GoodCtr)

9.4.3.6.77.1 Offset

Register	Offset
PhyUpd1GoodCtr	16Eh

9.4.3.6.77.2 Diagram



9.4.3.6.77.3 Fields

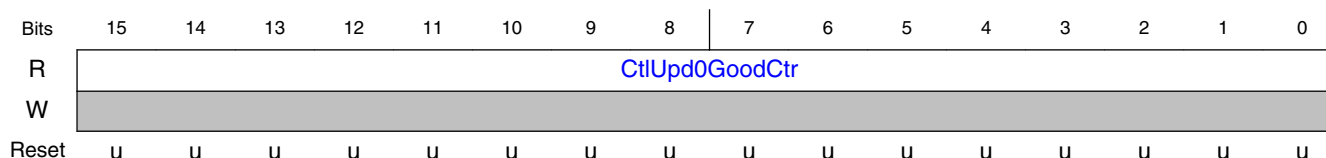
Field	Function
15-0 PhyUpd1GoodCtr	This register increments whenever the Memory Controller acknowledges a PHY-initiated DFI1 interface update request. This register increments whenever the Memory Controller acknowledges a PHY-initiated DFI1 interface update request. The counter saturates at max value 65535.

9.4.3.6.78 Counts successful Memory Controller DFI0 Interface Updates (CtlUpd0GoodCtr)

9.4.3.6.78.1 Offset

Register	Offset
CtlUpd0GoodCtr	170h

9.4.3.6.78.2 Diagram



9.4.3.6.78.3 Fields

Field	Function
15-0 CtlUpd0GoodCtr	This register increments whenever the PHY acknowledges a Memory Controller-initiated DFI0 interface update request.

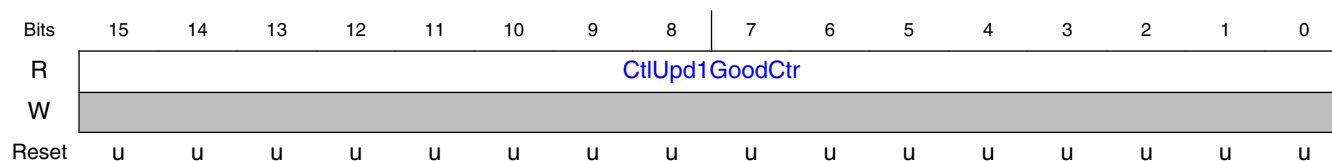
Field	Function
	This register increments whenever the PHY acknowledges a Memory Controller-initiated DFI0 interface update request. The counter saturates at max value 65535.

9.4.3.6.79 Counts successful Memory Controller DFI1 Interface Updates (CtlUpd1GoodCtr)

9.4.3.6.79.1 Offset

Register	Offset
CtlUpd1GoodCtr	172h

9.4.3.6.79.2 Diagram



9.4.3.6.79.3 Fields

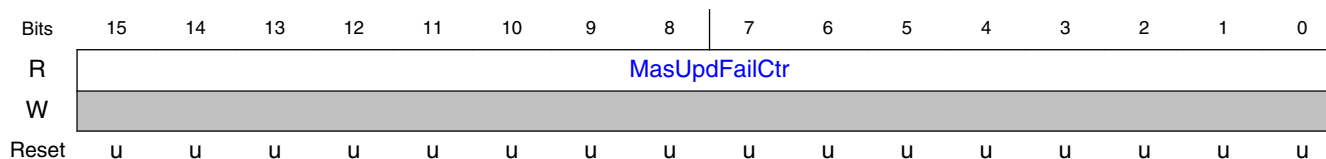
Field	Function
15-0 CtlUpd1GoodCtr	This register increments whenever the PHY acknowledges a Memory Controller-initiated DFI1 interface update request. This register increments whenever the PHY acknowledges a Memory Controller-initiated DFI1 interface update request. The counter saturates at max value 65535.

9.4.3.6.80 Counts unsuccessful PHY Master Interface Updates (MasUpdFailCtr)

9.4.3.6.80.1 Offset

Register	Offset
MasUpdFailCtr	174h

9.4.3.6.80.2 Diagram



9.4.3.6.80.3 Fields

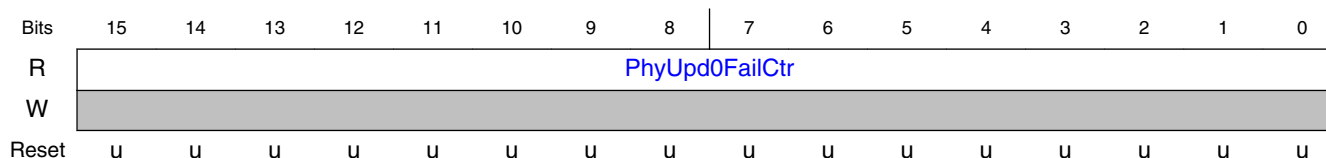
Field	Function
15-0 MasUpdFailCtr	This register increments whenever the PHY asserts a PHY Master Interface request, but the Memory Controller doesn't acknowledge the request within the allowed interval. This register increments whenever the PHY asserts a PHY Master Interface request, but the Memory Controller doesn't acknowledge the request within the allowed interval. (As per section 11 of the Preliminary DFI 4.0 Specification Rev. 2) The counter saturates at max value 65535. In a 2-channel system, a failure occurs if either MC channels fails to ack.

9.4.3.6.81 Counts unsuccessful PHY-initiated DFI0 Interface Updates (PhyUpd0FailCtr)

9.4.3.6.81.1 Offset

Register	Offset
PhyUpd0FailCtr	176h

9.4.3.6.81.2 Diagram



9.4.3.6.81.3 Fields

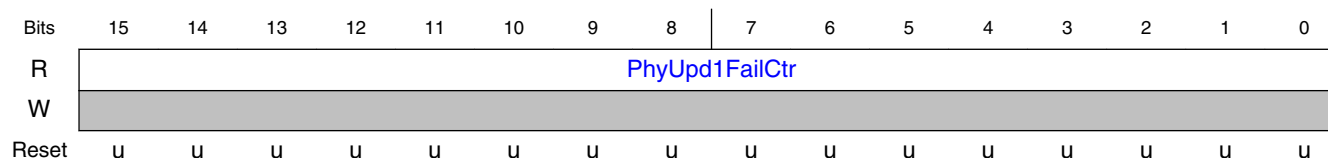
Field	Function
15-0 PhyUpd0FailCtr	This register increments whenever the PHY asserts a DFI0 Interface update request, but the Memory Controller doesn't acknowledge the request within the allowed interval. This register increments whenever the PHY asserts a DFI0 Interface update request, but the Memory Controller doesn't acknowledge the request within the allowed interval. (As per section 11 of the Preliminary DFI 4.0 Specification Rev. 2) The counter saturates at max value 65535.

9.4.3.6.82 Counts unsuccessful PHY-initiated DFI1 Interface Updates (PhyUpd1FailCtr)

9.4.3.6.82.1 Offset

Register	Offset
PhyUpd1FailCtr	178h

9.4.3.6.82.2 Diagram



9.4.3.6.82.3 Fields

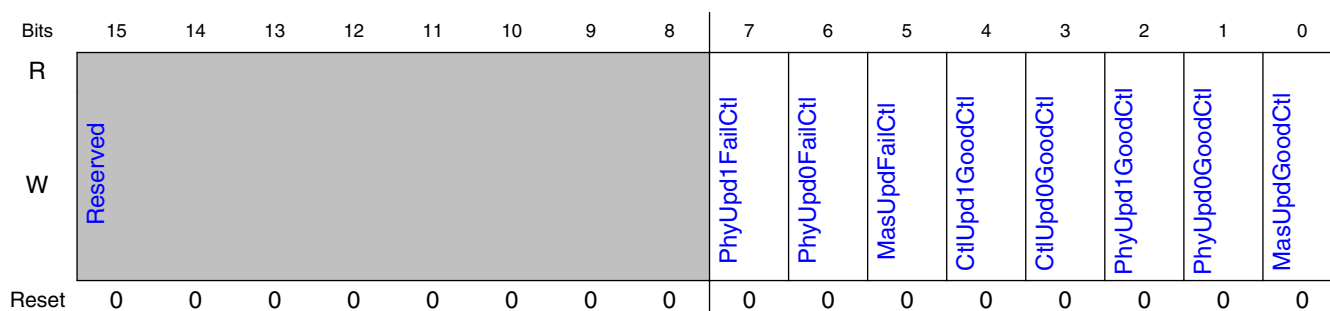
Field	Function
15-0 PhyUpd1FailCtr	This register increments whenever the PHY asserts a DFI1 Interface update request, but the Memory Controller doesn't acknowledge the request within the allowed interval. This register increments whenever the PHY asserts a DFI1 Interface update request, but the Memory Controller doesn't acknowledge the request within the allowed interval. (As per section 11 of the Preliminary DFI 4.0 Specification Rev. 2) The counter saturates at max value 65535.

9.4.3.6.83 Enables for Performance Counters (PhyPerfCtrEnable)

9.4.3.6.83.1 Offset

Register	Offset
PhyPerfCtrEnable	17Ah

9.4.3.6.83.2 Diagram



9.4.3.6.83.3 Fields

Field	Function
15-8 —	Reserved
7 PhyUpd1FailCtl	Enables PhyUpd1FailCtr
6 PhyUpd0FailCtl	Enables PhyUpd0FailCtr
5 MasUpdFailCtl	Enables MasUpdFailCtr
4 CtlUpd1GoodCtl	Enables CtlUpd1GoodCtr
3 CtlUpd0GoodCtl	Enables CtlUpd0GoodCtr
2 PhyUpd1GoodC tl	Enables PhyUpd1GoodCtr
1 PhyUpd0GoodC tl	Enables PhyUpd0GoodCtr

Table continues on the next page...

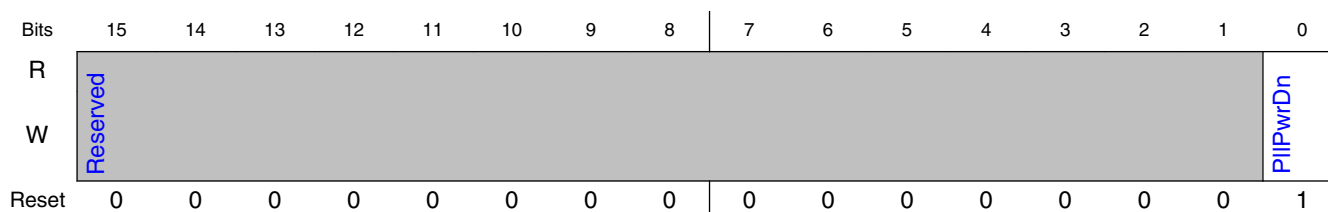
Field	Function
0 MasUpdGoodCtl	Enables MasUpdGoodCtr

9.4.3.6.84 PLL Power Down (PIIPwrDn)

9.4.3.6.84.1 Offset

Register	Offset
PIIPwrDn	186h

9.4.3.6.84.2 Diagram



9.4.3.6.84.3 Fields

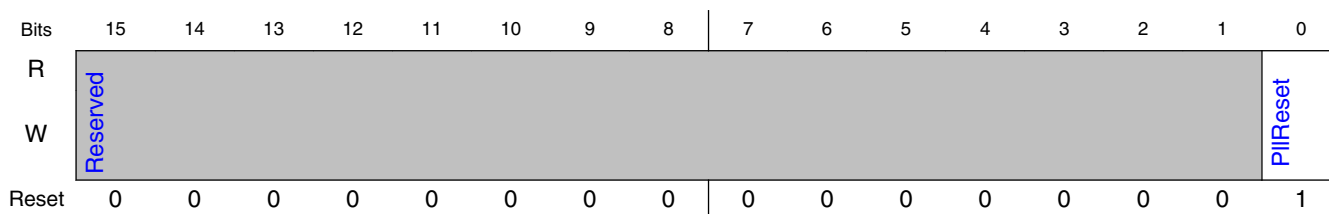
Field	Function
15-1 —	Reserved
0 PIIPwrDn	NOTE : This CSR is written by PHY Initialization Engine (PIE) and the data in here will be overwritten. NOTE : This CSR is written by PHY Initialization Engine (PIE) and the data in here will be overwritten. Places the PLL in Power Down when asserted.

9.4.3.6.85 PLL Reset (PIIReset)

9.4.3.6.85.1 Offset

Register	Offset
PIIReset	188h

9.4.3.6.85.2 Diagram



9.4.3.6.85.3 Fields

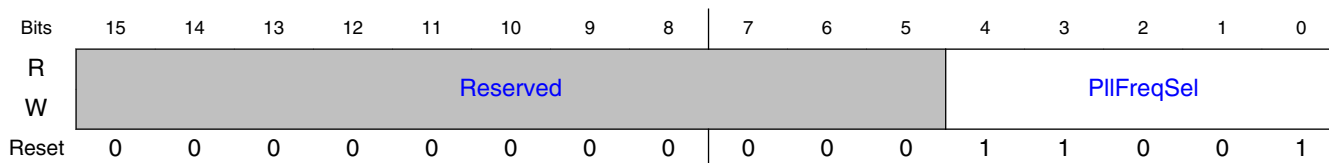
Field	Function
15-1 —	Reserved
0 PIIReset	Reserved

9.4.3.6.86 PState dependent PLL Control Register 2 (PIICtrl2_p0)

9.4.3.6.86.1 Offset

Register	Offset
PIICtrl2_p0	18Ah

9.4.3.6.86.2 Diagram



9.4.3.6.86.3 Fields

Field	Function
15-5 —	Reserved

Table continues on the next page...

Field	Function
4-0 PILFreqSel	Adjusts the loop parameters to compensate for different VCO bias points, and input/output clock division ratios. Adjusts the loop parameters to compensate for different VCO bias points, and input/output clock division ratios. Program based on reference clock frequency (DfiClk) with this table.

9.4.3.6.87 PLL Control Register 0 (PILCtrl0)

9.4.3.6.87.1 Offset

Register	Offset
PILCtrl0	18Ch

9.4.3.6.87.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
Reset	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0

9.4.3.6.87.3 Fields

Field	Function
15 PILSpareCtrl0	Spare bits for PLL control.
14-13 PILLockPhSel	Lock detect phase selection. Lock detect phase selection. 0: Test mode reduced lock window 1: Default operation phase lock 2: Test mode (mismatch check) 3: Test mode (+80ps window)
12	Lock detect counter selection.

Table continues on the next page...

DDR PHY (DDR_PHY)

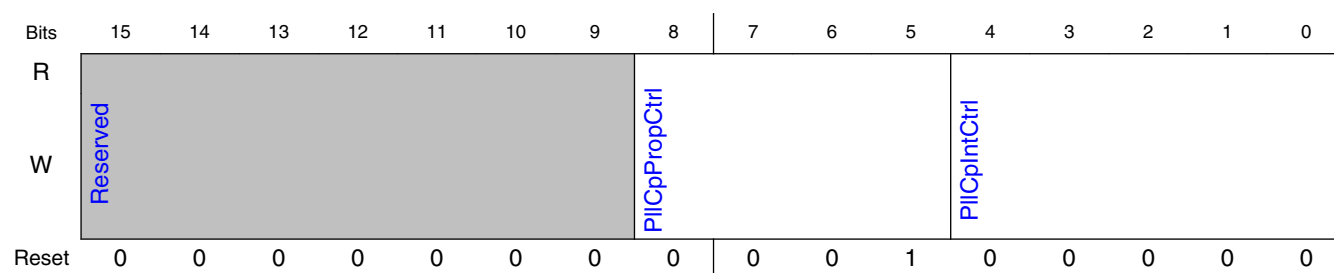
Field	Function
PIILockCntSel	Lock detect counter selection. 0: Normal lock detector behavior phase lock 1: Test mode (Shorten lock cycle count 16->8)
11 PIIGearShift	Puts PLL in fast re-locking mode 0: default, normal mode 1: fast relock gear
10-9 PIIReserved10x9	for future use.
8 PIISyncBusByp	When asserted bypasses the PII SyncPulse and uses a synchronizer of the same latency.
7 PIISyncBusFlus h	Used to flush the syncbus logic of the PLL during PHY initialization or LP3 Exit sequence. Should Only be toggled when synbus is not used and Register PIIOutBypEn is asserted.
6 PIISelDfiFreqRat io	reserved.
5 PIIBypassMode	PLL Bypass clock mux control. PLL Bypass clock mux control. If set, BypassPclk input will be used as byp_pll1x_in clock for PLL. Default 0.
4 PIIPreset	Put PLL in preset mode.
3 PIIOutBypEn	Controls the antiglitch mux on the pllout_x1x2x4 path 1: pllout_x1x2x4 = byp_pll1x_in 0: pllout_x1x2x4 = VCO (SCD) (selected by x2_mode)
2 PIIX2Mode	connects to x2_mode pins of PLL. connects to x2_mode pins of PLL. pllout_x4x2 output frequency selection. 0: PLOUT_X4X2 output = 4*pll1x_in frequency 1: PLOUT_X4X2 output = 2*pll1x_in frequency
1 PIIBypSel	Reserved
0 PIIStandby	Connects directly to standby pin of PLL. Connects directly to standby pin of PLL. Puts PLL into standbymode.

9.4.3.6.88 PState dependent PLL Control Register 1 (PIICtrl1_p0)

9.4.3.6.88.1 Offset

Register	Offset
PIICtrl1_p0	18Eh

9.4.3.6.88.2 Diagram



9.4.3.6.88.3 Fields

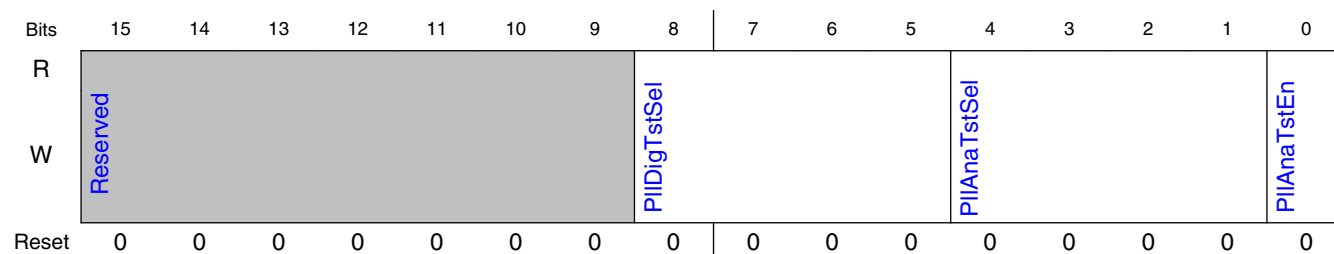
Field	Function
15-9 —	Reserved
8-5 PIICpPropCtrl	connects directly to cp_prop_cntrl<3:0> of PLL. connects directly to cp_prop_cntrl<3:0> of PLL. Charge pump proportional current control.
4-0 PIICpIntCtrl	connects directly to cp_int_cntrl<1:0> in PLL. connects directly to cp_int_cntrl<1:0> in PLL. Charge pump integrating current control.

9.4.3.6.89 PLL Testing Control Register (PIITst)

9.4.3.6.89.1 Offset

Register	Offset
PIITst	190h

9.4.3.6.89.2 Diagram



9.4.3.6.89.3 Fields

Field	Function
15-9 —	Reserved
8-5 PIIDigTstSel	Connects directly to pll_dig_test_sel<2:0> of PLL. Connects directly to pll_dig_test_sel<2:0> of PLL. Digital test mux select. Control bits are decoded to enable various digital test signals to be brought out via digital test pin, see decoder table for detail. PIIDigTstOut[1] is observable on BP_ALERT and Digital Observation Pin via MtestMuxSel logic. (if enabled in this configuration) PIIDigTstOut[0] is observable on Digital Observation Pin via MtestMuxSel logic only. During mission mode PIIDigTstSel should be set to 3'h0. Byp = PIIBypassMode CSR value Sel = PIIDigTstSel CSR value ----- PIIDigTstOut Byp Sel [1] [0] Description ===== ===== 0 000 0 0 This code should be set during normal use to minimize clock jitter 0 001 fbk_clk_ldet pllin_x1 Buffered version of fbk_clk at PFD input 0 010 ref_clk_ldet pllin_x1 Buffered version of ref_clk at PFD input 0 011 pllin_x1 pllin_x1 For calibrating delay skew between two test outputs - input reference clock 0 100 clk_fbk pllin_x1 Buffered version of pllout_x1 0 101 pllout_amux_x1x2x4 pllin_x1 Synclbus clock output (Fout<Fmax_dto only) 0 110 en_count pll_lock Lock detector state test 0 111 standby_eff eoc Calibration state test 1 000 0 0 This code should be set during normal use to minimize clock jitter 1 001 fbk_clk_ldet pllin_x1 Buffered version of fbk_clk at PFD input if PLL in mission/byp 1 modes 1 010 ref_clk_ldet pllin_x1 Buffered version of ref_clk at PFD input if PLL in mission/byp 1 modes 1 011 pllin_x1 pllin_x1 For calibrating delay skew between two test outputs - input reference clock

Table continues on the next page...

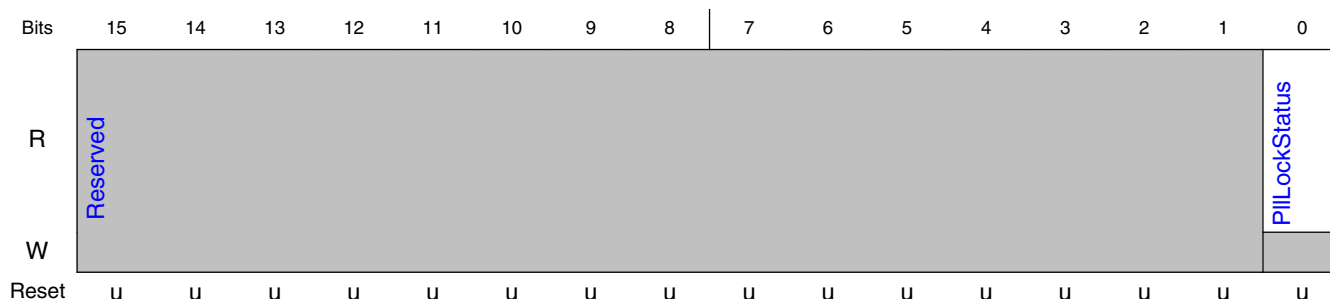
Field	Function
	1 1 1 100 clk_fbk pll_in_x1 Buffered version of pllout_x1 if PLL in mission/byp 1 modes 1 1 1 101 pllout_amux_x1x2x4 pll_in_x1 Synclbus clock output (Fout<Fmax_dto only) 1 1 1 110 en_count pll_lock Lock detector test if PLL in mission/byp 1 modes 1 1 1 111 standby_eff eoc Calibration state test if PLL in mission/byp 1 modes
4-1 PIIAAnaTstSel	Connects directly to pll_ana_test_sel<3:0> of PLL. Connects directly to pll_ana_test_sel<3:0> of PLL. Control bits are decoded to enable various analog test signals to be brought out via analog test pin (pll_analog_test). Table below provides decode detail. PIIAAnaTstSel PII Analog Description 4:1 Pin Function ===== 0000 Hi-Z This code should be set when the PLL is in normal use or when the digital test port is being used for accurate timing measurements 0001 core Override vmarg_ext with analog pin 0010 core Override regulator input with analog pin 0011 core Observe gd voltage on analog test pin 0100 core Observe vp voltage on analog test pin 0101 core Observe vph voltage on analog test pin 0110 core Observe vreg_cp voltage on analog test pin 0111 core Observe v2i current (~10uA) on analog test pin 1000 core Observe vreg_v2i voltage on analog test pin 1001 core Observe vbp voltage from current source on analog pin 1010 core Observe vbn voltage from current source on analog tpin 1011 core NC 1100 core NC 1101 VSS 0v probe ir drop test 1110 VP Core supply probe SCD IR drop test 1111 VPH Core Supply probe Refdiv IR drop test
0 PIIAAnaTstEn	Connects directly to pll_ana_test_en of PLL. Connects directly to pll_ana_test_en of PLL. Analog test port enable 0 = analog test port disabled (hi-Z) 1 = analog test port enabled

9.4.3.6.90 PLL's pll_lock pin output (PIILockStatus)

9.4.3.6.90.1 Offset

Register	Offset
PIILockStatus	192h

9.4.3.6.90.2 Diagram



9.4.3.6.90.3 Fields

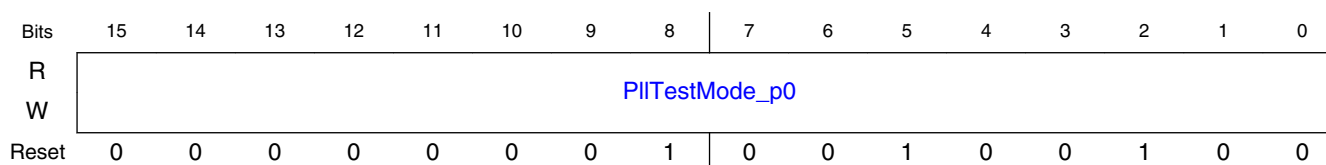
Field	Function
15-1 —	Reserved
0 PIILockStatus	Directly connected to the pll_Lock output. Directly connected to the pll_Lock output. When asserted Pll macro has asserted the Lock status. To avoid metastability issues, this register should perform a majority-vote after reading this register several times.

9.4.3.6.91 Additional controls for PLL CP/VCO modes of operation (PIITestMode_p0)

9.4.3.6.91.1 Offset

Register	Offset
PIITestMode_p0	194h

9.4.3.6.91.2 Diagram



9.4.3.6.91.3 Fields

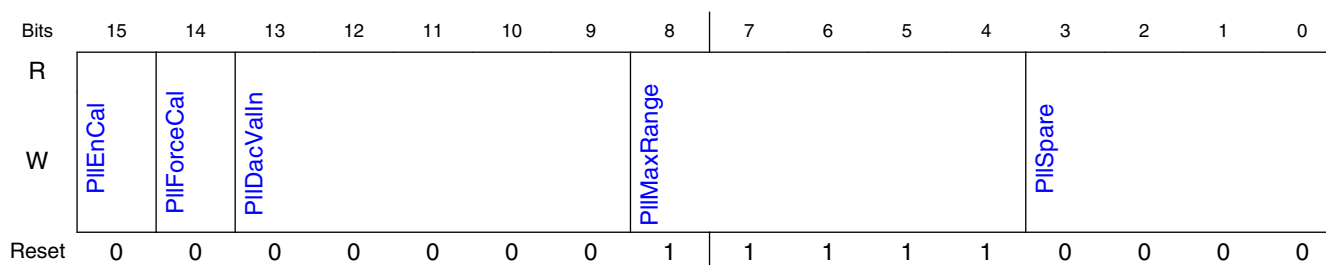
Field	Function
15-0	It is required to use default values for this CSR unless directed otherwise by Synopsys.
PIITestMode_p0	It is required to use default values for this CSR unless directed otherwise by Synopsys. Directly connected to testmode[15:0] pin of PLL

9.4.3.6.92 PLL Control Register 3 (PIICtrl3)

9.4.3.6.92.1 Offset

Register	Offset
PIICtrl3	196h

9.4.3.6.92.2 Diagram



9.4.3.6.92.3 Fields

Field	Function
15	Calibration will run at standby rising edge if en_cal=1 if en_cal=0 calibration will not run

Table continues on the next page...

DDR PHY (DDR_PHY)

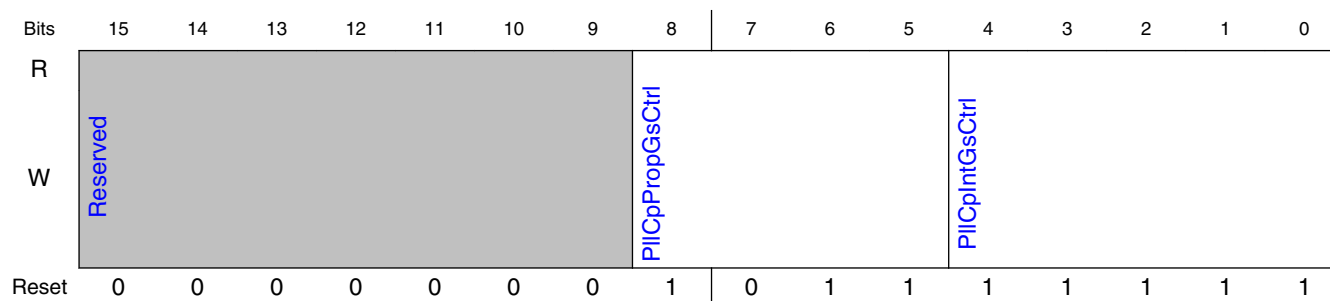
Field	Function
PIIEnCal	
14 PIIForceCal	connects directly to force_cal of PLL. connects directly to force_cal of PLL. Forces Calibration code dacval_in<4:0> to be used at preset mode if force_cal=1 if force_cal=0 previous stored value will be used
13-9 PIIDacValln	connects directly to dacval_in<4:0> of PLL. connects directly to dacval_in<4:0> of PLL. Calibration DAC input current setting, can be forced at rising edge of preset if force_cal =1
8-4 PIIMaxRange	connects directly to maxrange of PLL. connects directly to maxrange of PLL. Setting for Saturation control of PLL.
3-0 PIISpare	Spare bits for future PLL control modes

9.4.3.6.93 PState dependent PLL Control Register 4 (PIICtrl4_p0)

9.4.3.6.93.1 Offset

Register	Offset
PIICtrl4_p0	198h

9.4.3.6.93.2 Diagram



9.4.3.6.93.3 Fields

Field	Function
15-9	Reserved

Table continues on the next page...

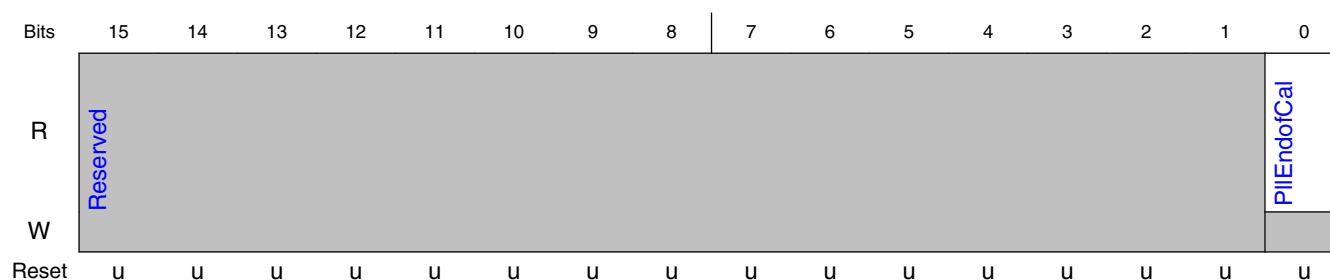
Field	Function
—	
8-5 PllCpPropGsCtrl	connects directly to cp_prop_gs_cntrl<3:0> of PLL. Charge pump proportional current control for fast relock and gearshift.
4-0 PllCpIntGsCtrl	connects directly to cp_int_gs_cntrl<4:0> in PLL. Charge pump integrating current control for fast relock and gearshift.

9.4.3.6.94 PLL's eoc (end of calibration) output (PllEndofCal)

9.4.3.6.94.1 Offset

Register	Offset
PllEndofCal	19Ah

9.4.3.6.94.2 Diagram



9.4.3.6.94.3 Fields

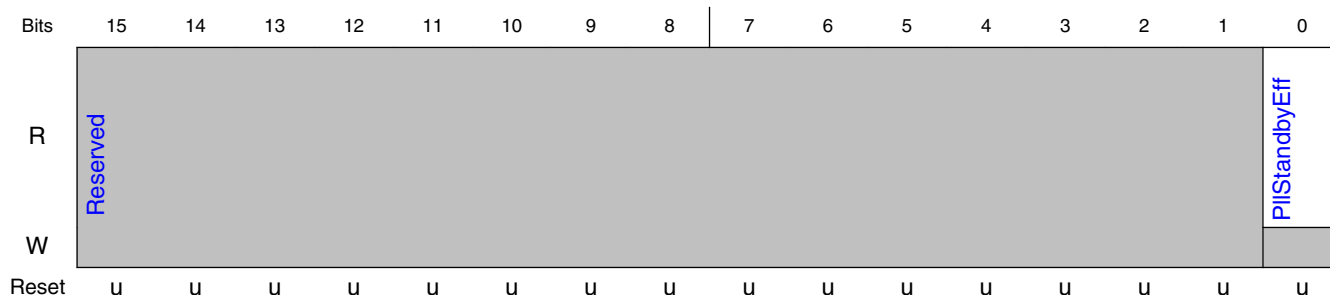
Field	Function
15-1 —	Reserved
0 PllEndofCal	Directly connected to the pll's eoc output.

9.4.3.6.95 PLL's standby_eff (effective standby) output (PllStandbyEff)

9.4.3.6.95.1 Offset

Register	Offset
PIIStandbyEff	19Ch

9.4.3.6.95.2 Diagram



9.4.3.6.95.3 Fields

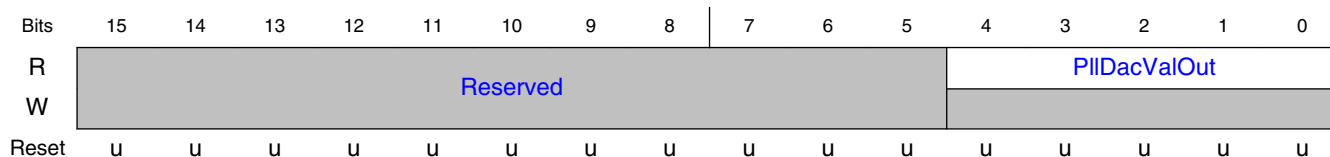
Field	Function
15-1	Reserved
—	
0	Returns state off PLL standby.
PIIStandbyEff	Returns state off PLL standby. PLL will be in standby in PHY LP2 states.

9.4.3.6.96 PLL's Dacval_out output (PIIDacValOut)

9.4.3.6.96.1 Offset

Register	Offset
PIIDacValOut	19Eh

9.4.3.6.96.2 Diagram



9.4.3.6.96.3 Fields

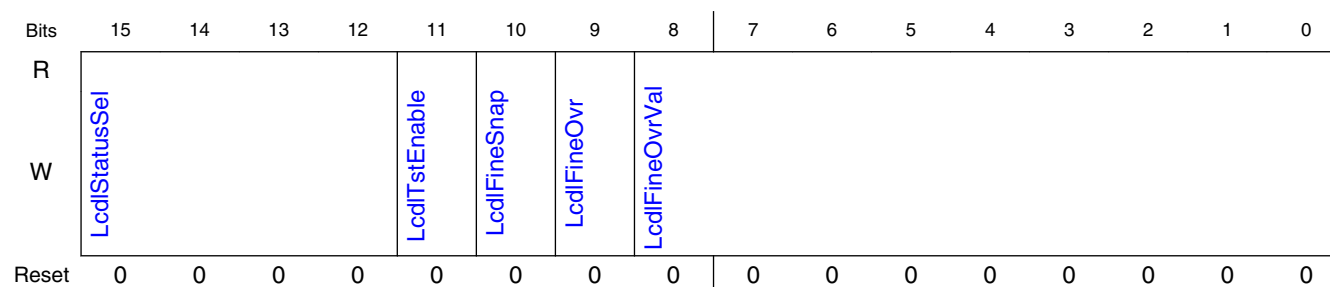
Field	Function
15-5 —	Reserved
4-0 PllDacValOut	Directly connected to the pll's dacval_out output. Directly connected to the pll's dacval_out output. Used for observation of PLL DAC code.

9.4.3.6.97 Controls for use in observing and testing the LCDLs. (LcdIDbgCntl)

9.4.3.6.97.1 Offset

Register	Offset
LcdIDbgCntl	1C6h

9.4.3.6.97.2 Diagram



9.4.3.6.97.3 Fields

Field	Function
15-12 LcdlStatusSel	Selects the LCDL status, from among the status for the 16 LCDLs in the DBYTE, for reading via Register DxLcdlStatus and an LCDL from among the LCDLs in the ANIB for reading via Register AcLcdlStatus LcdlStatusSel source for DxLcdlStatus source for AcLcdlStatus 15 lcdl_rxclk1t reserved 14 lcdl_rxclk0t reserved 13 lcdl_rxclk1c reserved 12 lcdl_rxclk0c reserved 11 lcdl_rxen1 anib11-tx 10 lcdl_rxen0 anib10-tx 9 lcdl_txln9 (dq5-lower) anib9-tx 8 lcdl_txln8 (dm/dqs-upper) anib8-tx 7 lcdl_txln7 (dq7) anib7-tx 6 lcdl_txln6 (dq6) anib6-tx 5 lcdl_txln5 (dq5) anib5-tx 4 lcdl_txln4 (dq4) anib4-tx 3 lcdl_txln3 (dq3) anib3-tx 2 lcdl_txln2 (dq2) anib2-tx 1 lcdl_txln1 (dq1) anib1-tx 0 lcdl_txln0 (dq0) anib0-tx
11 LcdlTstEnable	Enables the debug/test operations and status Ovr,Snap,StickyLock,StickyUnlock, and LiveLock.

Table continues on the next page...

DDR PHY (DDR_PHY)

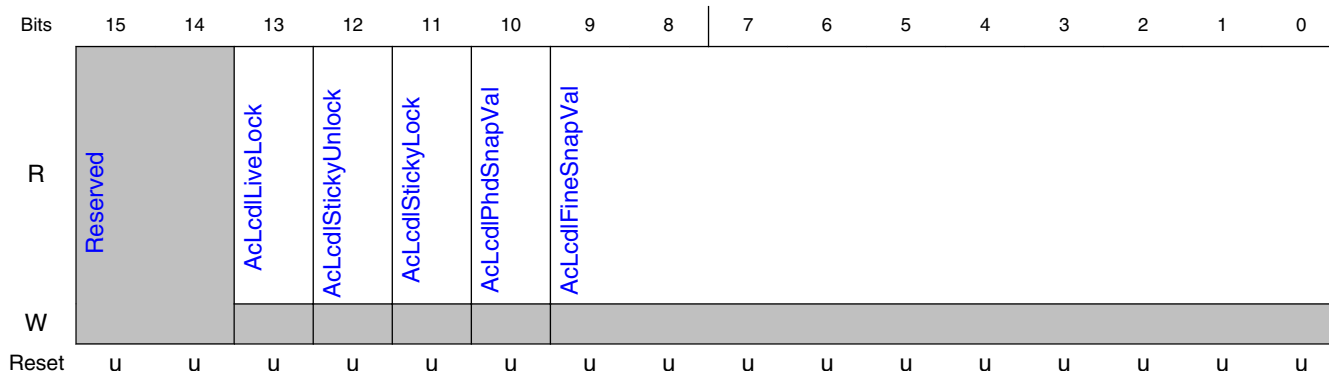
Field	Function
10 LcdIFineSnap	Latch enable for reading the present LCDL 1UI estimate code in LcdIFineSnapVal and the present phase-detector value in LcdIPhdSnapVal
9 LcdIFineOvr	Forces the value of the present LCDL 1UI estimate code to be LcdIFineOvrVal for all LCDLs.
8-0 LcdIFineOvrVal	Value forced as the initial value while LcdITstEnable=1 and LcdIFineOvr. Value forced as the initial value while LcdITstEnable=1 and LcdIFineOvr. After the deassertion of LcdIFineOvr, the locking state-machine will attempt locking.

9.4.3.6.98 Debug status of the DBYTE LCDL (AcLcdIStatus)

9.4.3.6.98.1 Offset

Register	Offset
AcLcdIStatus	1C8h

9.4.3.6.98.2 Diagram



9.4.3.6.98.3 Fields

Field	Function
15-14 —	Reserved
13 AcLcdILiveLock	present value of whether the LCDL is locked, valid when LcdITstEnable=1.
12	latched value of whether the LCDL ever lost lock after the assertion of LcdITstEnable.

Table continues on the next page...

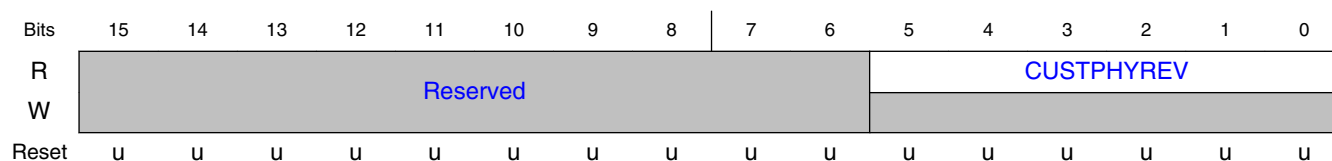
Field	Function
AcLcdlStickyUnl ock	
11 AcLcdlStickyLoc k	latched value of whether the LCDL ever achieved lock after the assertion of LcdlTstEnable.
10 AcLcdlPhdSnap Val	Value of the LCDL phase-detector output, latched by pulse on LcdlFineSnap while csr LcdlTstEnable=1.
9-0 AcLcdlFineSnap Val	Value of the LCDL 1UI estimate code, latched by pulse on csrLcdlFineSnap while csr LcdlTstEnable=1. Value of the LCDL 1UI estimate code, latched by pulse on csrLcdlFineSnap while csr LcdlTstEnable=1. Index 9 is reserved for growth.

9.4.3.6.99 Customer settable by the customer (CUSTPHYREV)

9.4.3.6.99.1 Offset

Register	Offset
CUSTPHYREV	1DAh

9.4.3.6.99.2 Diagram



9.4.3.6.99.3 Fields

Field	Function
15-6 —	Reserved
5-0 CUSTPHYREV	The customer settable PHY version number. The customer settable PHY version number. The value is derived from the `define DWC_DDRPHY_CUST_PHYREV

9.4.3.6.100 The hardware version of this PHY, excluding the PUB (PHYR EV)

9.4.3.6.100.1 Offset

Register	Offset
PHYREV	1DCh

9.4.3.6.100.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PHYMJR								PHYMDR				PHYMNR			
W																
Reset	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0

9.4.3.6.100.3 Fields

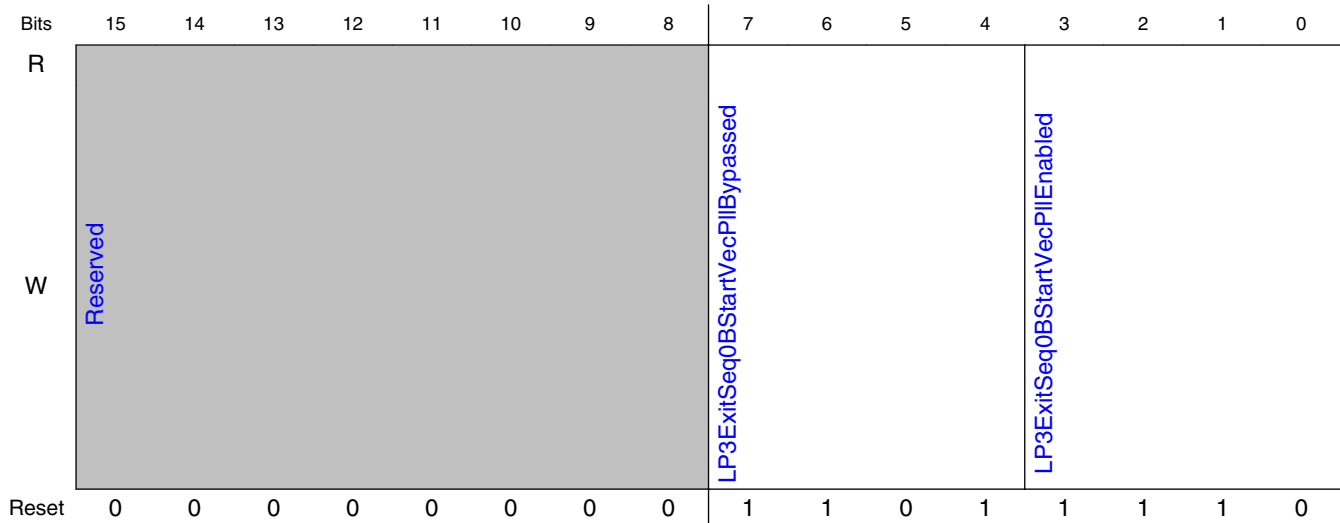
Field	Function
15-8 PHYMJR	Indicates major revision of the PHY.
7-4 PHYMDR	Indicates moderate revision of the PHY.
3-0 PHYMNR	Indicates minor update of the PHY.

9.4.3.6.101 Start vector value to be used for LP3-exit or Init PIE Sequence (LP3ExitSeq0BStartVector)

9.4.3.6.101.1 Offset

Register	Offset
LP3ExitSeq0BStartVector	1DEh

9.4.3.6.101.2 Diagram



9.4.3.6.101.3 Fields

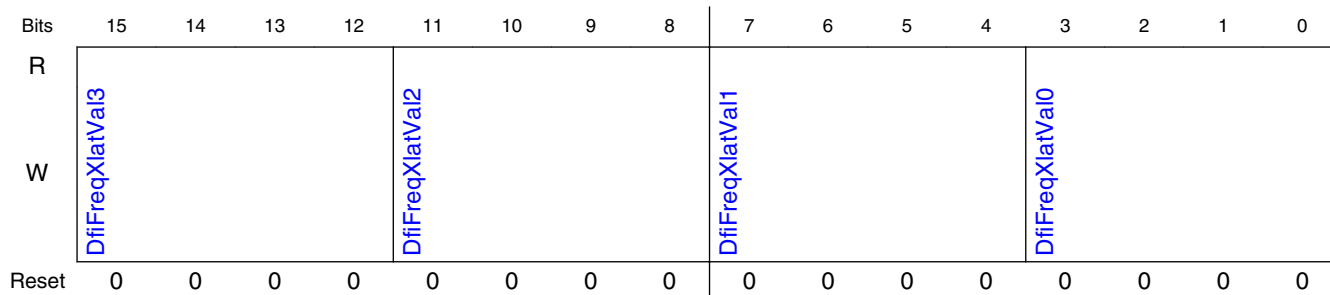
Field	Function
15-8 —	Reserved
7-4 LP3ExitSeq0BStartVecPIIBypassed	PIE Start Vector value to be used for LP3-exit or Init and target P-state has PLL bypassed
3-0 LP3ExitSeq0BStartVecPIIEnabled	PIE Start Vector value to be used for LP3-exit or Init and target P-state has PLL enabled

9.4.3.6.102 DFI Frequency Translation Register 0 (DfiFreqXlat0)

9.4.3.6.102.1 Offset

Register	Offset
DfiFreqXlat0	1E0h

9.4.3.6.102.2 Diagram



9.4.3.6.102.3 Fields

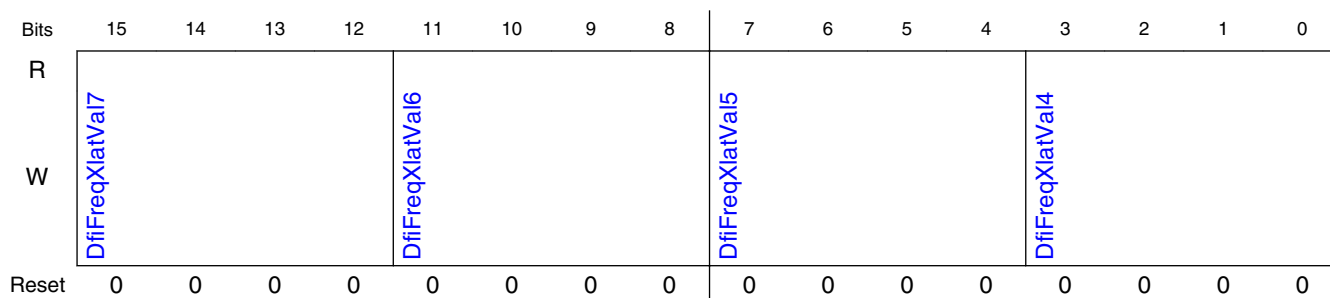
Field	Function
15-12 DfiFreqXlatVal3	The sequencer start vector used when dfi_freq value is 3.
11-8 DfiFreqXlatVal2	The sequencer start vector used when dfi_freq value is 2.
7-4 DfiFreqXlatVal1	The sequencer start vector used when dfi_freq value is 1.
3-0 DfiFreqXlatVal0	The sequencer start vector used when dfi_freq value is 0.

9.4.3.6.103 DFI Frequency Translation Register 1 (DfiFreqXlat1)

9.4.3.6.103.1 Offset

Register	Offset
DfiFreqXlat1	1E2h

9.4.3.6.103.2 Diagram



9.4.3.6.103.3 Fields

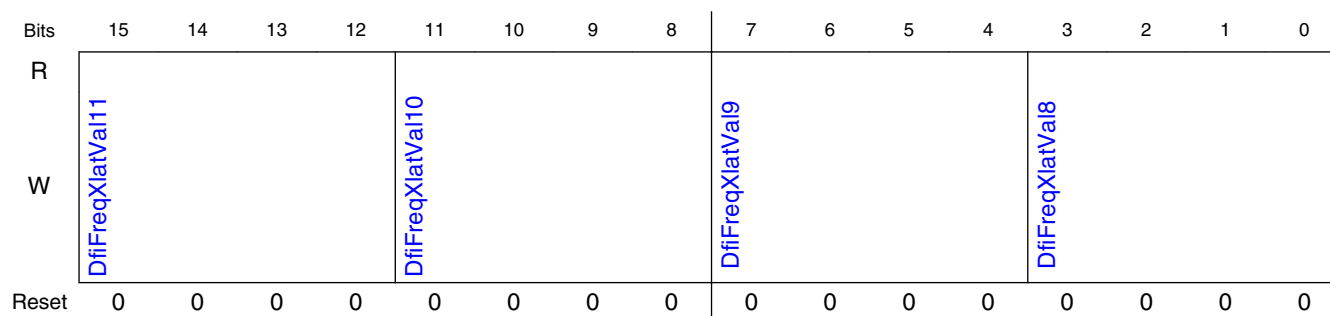
Field	Function
15-12 DfiFreqXlatVal7	The sequencer start vector used when dfi_freq value is 7.
11-8 DfiFreqXlatVal6	The sequencer start vector used when dfi_freq value is 6.
7-4 DfiFreqXlatVal5	The sequencer start vector used when dfi_freq value is 5.
3-0 DfiFreqXlatVal4	The sequencer start vector used when dfi_freq value is 4.

9.4.3.6.104 DFI Frequency Translation Register 2 (DfiFreqXlat2)

9.4.3.6.104.1 Offset

Register	Offset
DfiFreqXlat2	1E4h

9.4.3.6.104.2 Diagram



9.4.3.6.104.3 Fields

Field	Function
15-12 DfiFreqXlatVal1 1	The sequencer start vector used when dfi_freq value is 11.

Table continues on the next page...

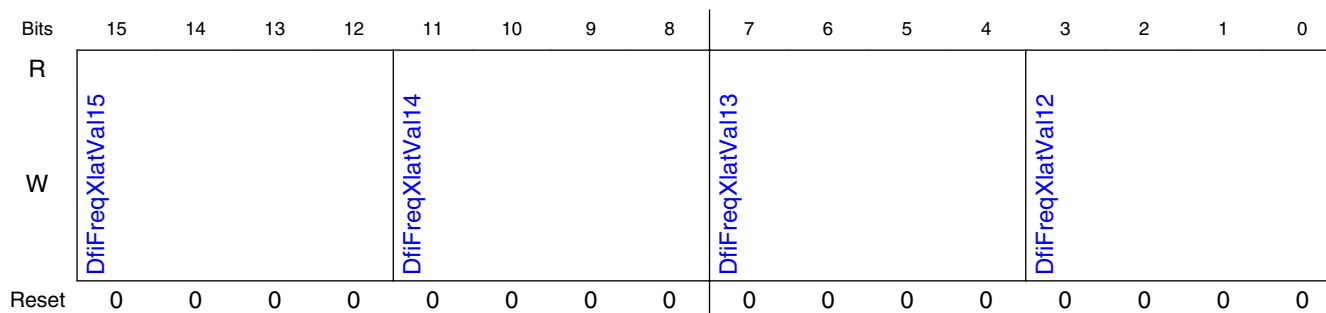
Field	Function
11-8 DfiFreqXlatVal10	The sequencer start vector used when dfi_freq value is 10.
7-4 DfiFreqXlatVal9	The sequencer start vector used when dfi_freq value is 9.
3-0 DfiFreqXlatVal8	The sequencer start vector used when dfi_freq value is 8.

9.4.3.6.105 DFI Frequency Translation Register 3 (DfiFreqXlat3)

9.4.3.6.105.1 Offset

Register	Offset
DfiFreqXlat3	1E6h

9.4.3.6.105.2 Diagram



9.4.3.6.105.3 Fields

Field	Function
15-12 DfiFreqXlatVal15	The sequencer start vector used when dfi_freq value is 15.
11-8 DfiFreqXlatVal14	The sequencer start vector used when dfi_freq value is 14.
7-4	The sequencer start vector used when dfi_freq value is 13.

Table continues on the next page...

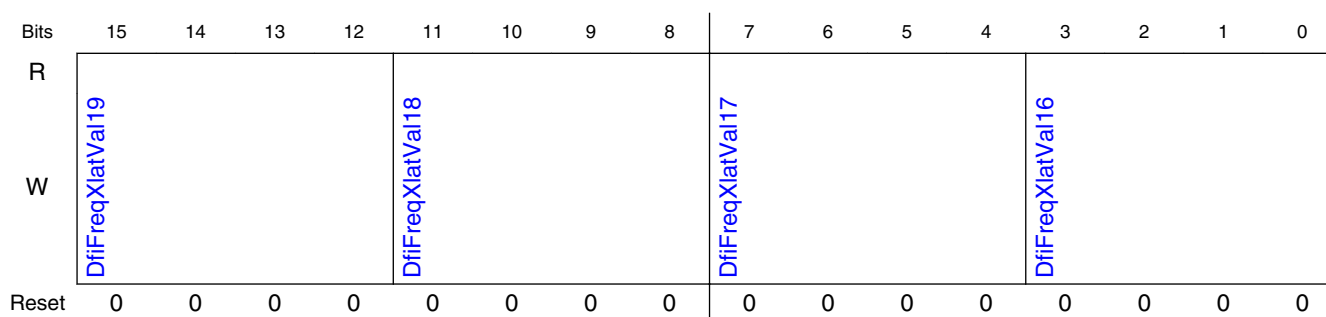
Field	Function
DfiFreqXlatVal1 3	The sequencer start vector used when dfi_freq value is 12.
3-0 DfiFreqXlatVal1 2	

9.4.3.6.106 DFI Frequency Translation Register 4 (DfiFreqXlat4)

9.4.3.6.106.1 Offset

Register	Offset
DfiFreqXlat4	1E8h

9.4.3.6.106.2 Diagram



9.4.3.6.106.3 Fields

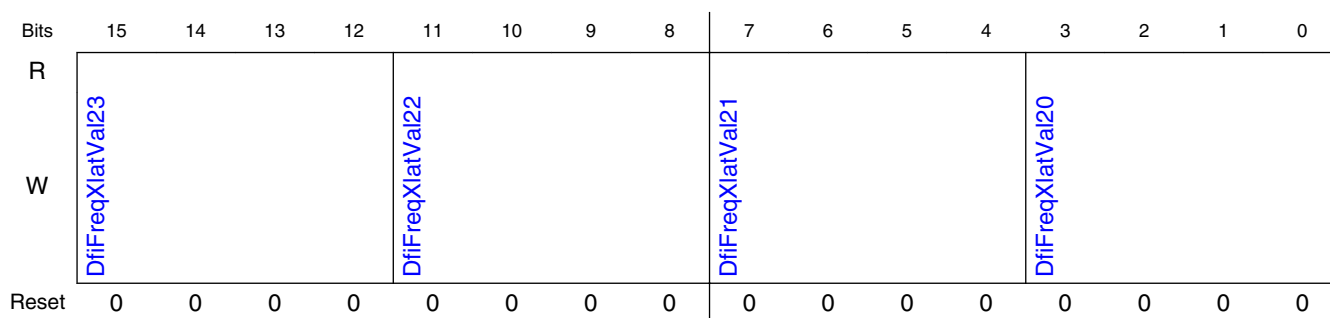
Field	Function
15-12 DfiFreqXlatVal1 9	The sequencer start vector used when dfi_freq value is 19.
11-8 DfiFreqXlatVal1 8	
7-4 DfiFreqXlatVal1 7	The sequencer start vector used when dfi_freq value is 17.
3-0 DfiFreqXlatVal1 6	

9.4.3.6.107 DFI Frequency Translation Register 5 (DfiFreqXlat5)

9.4.3.6.107.1 Offset

Register	Offset
DfiFreqXlat5	1EAh

9.4.3.6.107.2 Diagram



9.4.3.6.107.3 Fields

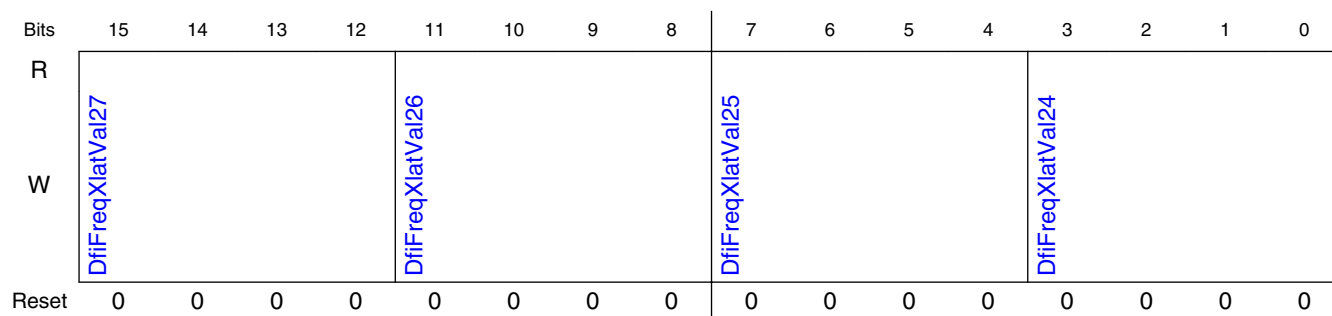
Field	Function
15-12 DfiFreqXlatVal23	The sequencer start vector used when dfi_freq value is 23.
11-8 DfiFreqXlatVal22	The sequencer start vector used when dfi_freq value is 22.
7-4 DfiFreqXlatVal21	The sequencer start vector used when dfi_freq value is 21.
3-0 DfiFreqXlatVal20	The sequencer start vector used when dfi_freq value is 20.

9.4.3.6.108 DFI Frequency Translation Register 6 (DfiFreqXlat6)

9.4.3.6.108.1 Offset

Register	Offset
DfiFreqXlat6	1ECh

9.4.3.6.108.2 Diagram



9.4.3.6.108.3 Fields

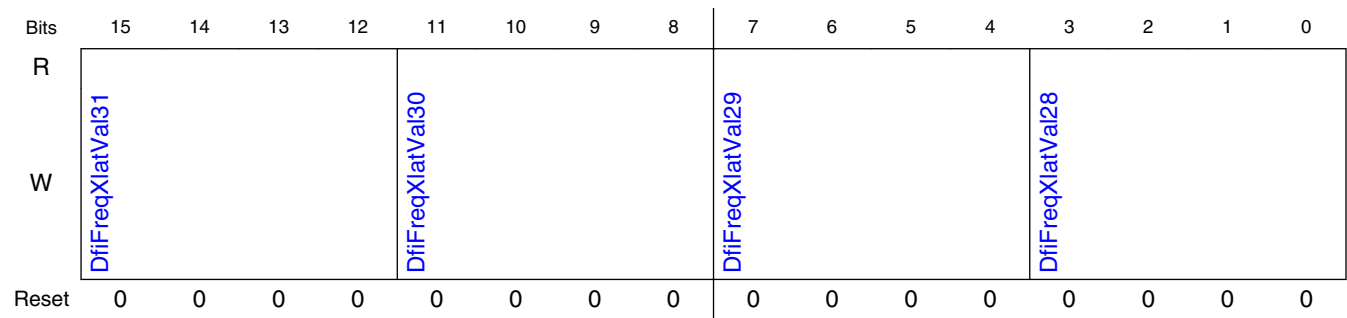
Field	Function
15-12 DfiFreqXlatVal27	The sequencer start vector used when dfi_freq value is 27.
11-8 DfiFreqXlatVal26	The sequencer start vector used when dfi_freq value is 26.
7-4 DfiFreqXlatVal25	The sequencer start vector used when dfi_freq value is 25.
3-0 DfiFreqXlatVal24	The sequencer start vector used when dfi_freq value is 24.

9.4.3.6.109 DFI Frequency Translation Register 7 (DfiFreqXlat7)

9.4.3.6.109.1 Offset

Register	Offset
DfiFreqXlat7	1EEh

9.4.3.6.109.2 Diagram



9.4.3.6.109.3 Fields

Field	Function
15-12 DfiFreqXlatVal31	The sequencer start vector used when dfi_freq value is 31.
11-8 DfiFreqXlatVal30	The sequencer start vector used when dfi_freq value is 30.
7-4 DfiFreqXlatVal29	The sequencer start vector used when dfi_freq value is 29.
3-0 DfiFreqXlatVal28	The sequencer start vector used when dfi_freq value is 28.

9.4.3.6.110 TxRdPtrInit control register (TxRdPtrInit)

9.4.3.6.110.1 Offset

Register	Offset
TxRdPtrInit	1F0h

9.4.3.6.110.2 Diagram



9.4.3.6.110.3 Fields

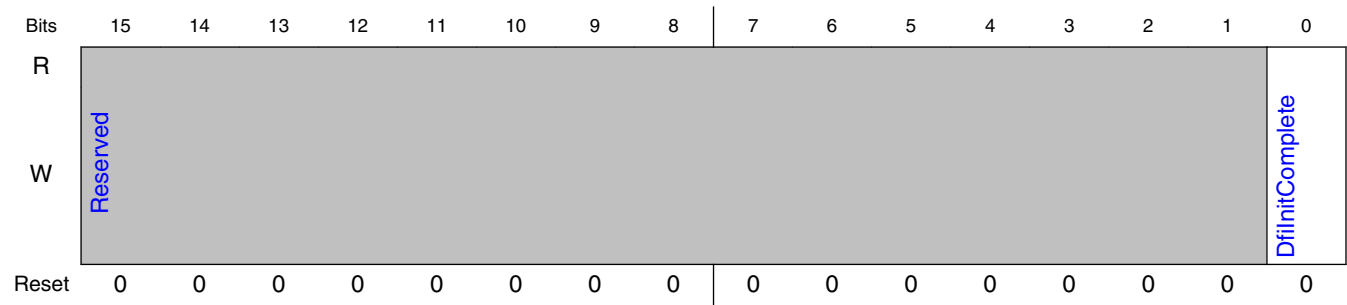
Field	Function
15-1 —	Reserved
0 TxRdPtrInit	This register directly controls TxRdPtrInit, and is meant to be written by the PState sequencer as part of the power state switching sequence. This register directly controls TxRdPtrInit, and is meant to be written by the PState sequencer as part of the power state switching sequence. It should be written to a 1 once the transition to LP2 is complete and 32 cycles before the de-asserting of init_complete. Likewise it should be written to 0 once the transition to an active PState is complete and 32 cycles before the assertion of init_complete. It should reset to a 1 on both warm and cold reset.

9.4.3.6.111 DFI Init Complete control (DfilnitComplete)

9.4.3.6.111.1 Offset

Register	Offset
DfilnitComplete	1F2h

9.4.3.6.111.2 Diagram



9.4.3.6.111.3 Fields

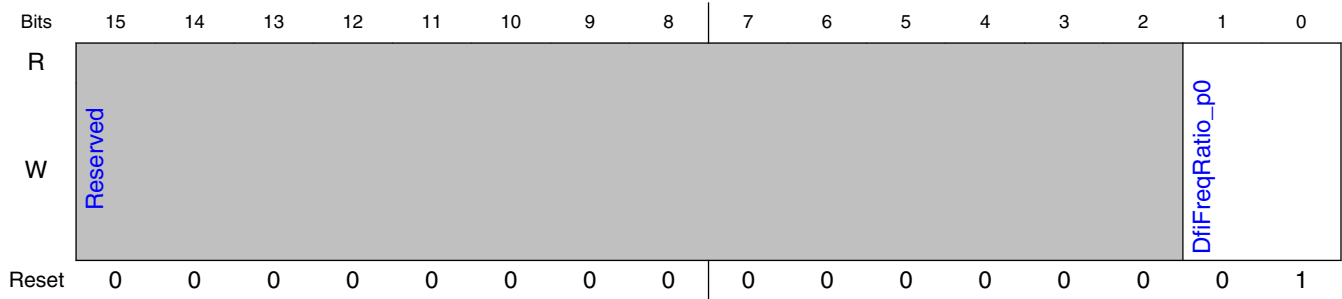
Field	Function
15-1 —	Reserved
0 DfinitComplete	This register directly controls DfinitComplete, and is meant to be written by the PState sequencer as part of the power state switching sequence. This register directly controls DfinitComplete, and is meant to be written by the PState sequencer as part of the power state switching sequence. It should be written to a 1 once the transition to LP2 is complete. Likewise it should be written to 0 once the transition to an active and PState is complete. It should reset to a 0 on both warm and cold reset.

9.4.3.6.112 DFI Frequency Ratio (DfiFreqRatio_p0)

9.4.3.6.112.1 Offset

Register	Offset
DfiFreqRatio_p0	1F4h

9.4.3.6.112.2 Diagram



9.4.3.6.112.3 Fields

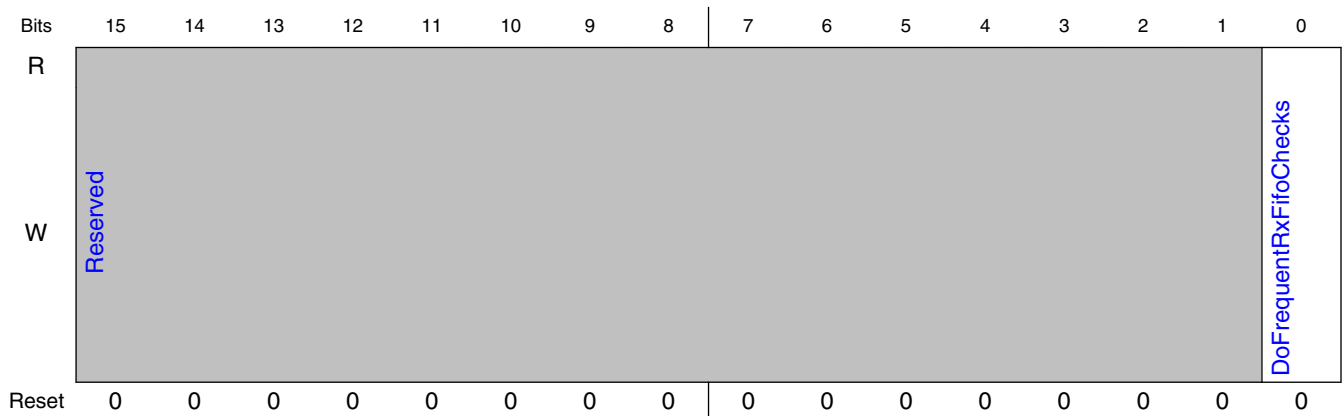
Field	Function
15-2 —	Reserved
1-0 DfiFreqRatio_p0	Used in dwc_ddrphy_pub_serdes to serialize or de-serialize DFI signals 00 = 1:1 mode 01 = 1:2 mode 1x = 1:4 mode* *Note: 1:4 is for future pub revision.

9.4.3.6.113 Enable more frequent consistency checks of the RX FIFOs (RxFifoChecks)

9.4.3.6.113.1 Offset

Register	Offset
RxFifoChecks	1F6h

9.4.3.6.113.2 Diagram



9.4.3.6.113.3 Fields

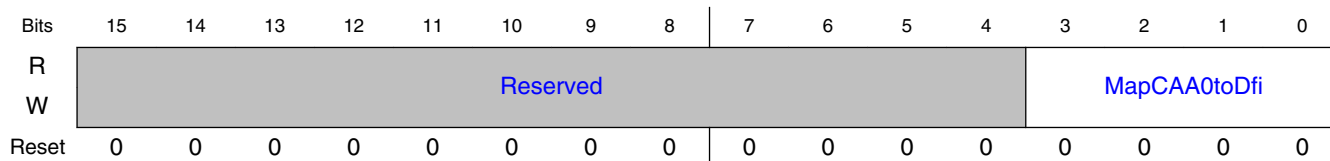
Field	Function
15-1 —	Reserved
0 DoFrequentRxFifoChecks	When 0, read data FIFO pointer consistency checks are performed only during sideband transactions (i.e., phyupd, ctrlupd, lp_data, phymstr) or when dfi_init_start is asserted. This option relies upon intentional quiescing of the DFI interface in order for the check to be performed. When 1, read data FIFO consistency checks are performed as when this bit is 0, and also at the beginning of each break in read data traffic from the DRAM. This options provides the lowest latency in reporting of a discrepancy in the read data FIFO pointer values.

9.4.3.6.114 Maps PHY CAA lane 0 from dfi0_address of the index of the register contents (MapCAA0toDfi)

9.4.3.6.114.1 Offset

Register	Offset
MapCAA0toDfi	200h

9.4.3.6.114.2 Diagram



9.4.3.6.114.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAA0toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 0.

Field	Function
	Unused for DDR3 and unused for DDR4 applications. Each register in the set of MapCAAtoDfi must have a unique value within the set. For example, if PHY CAA0 is mapped from dfi0_address[2], then Register MapCAA0toDfi should be 2. LP4 ACX3,ACX6,ACX10,ACX12 registers MapCAA0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAA6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6,ACX10,ACX12 registers MapCAA0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.115 Maps PHY CAA lane 1 from dfi0_address of the index of the register contents (MapCAA1toDfi)

9.4.3.6.115.1 Offset

Register	Offset
MapCAA1toDfi	202h

9.4.3.6.115.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								MapCAA1toDfi							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

9.4.3.6.115.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAA1toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 1. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 1. Unused for DDR3 and unused for DDR4 applications. Each register in the set of MapCAAtoDfi must have a unique value within the set. For example, if PHY CAA1 is mapped from dfi0_address[2], then Register MapCAA1toDfi should be 2.

Field	Function
	LP4 ACX3,ACX6,ACX10,ACX12 registers MapCAA0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAA6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6,ACX10,ACX12 registers MapCAA0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.116 Maps PHY CAA lane 2 from dfi0_address of the index of the register contents (MapCAA2toDfi)

9.4.3.6.116.1 Offset

Register	Offset
MapCAA2toDfi	204h

9.4.3.6.116.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												MapCAA2toDfi			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

9.4.3.6.116.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAA2toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 2. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 2. Unused for DDR3 and unused for DDR4 applications. Each register in the set of MapCAAtDfi must have a unique value within the set. For example, if PHY CAA2 is mapped from dfi0_address[2], then Register MapCAA2toDfi should be 2. LP4 ACX3,ACX6,ACX10,ACX12 registers MapCAA0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAA6..9toDfi are reserved

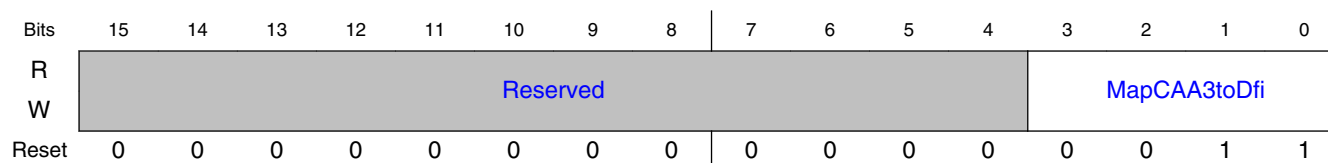
Field	Function
	LP3 ACX3 not supported LP3 ACX6,ACX10,ACX12 registers MapCAA0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.117 Maps PHY CAA lane 3 from dfi0_address of the index of the register contents (MapCAA3toDfi)

9.4.3.6.117.1 Offset

Register	Offset
MapCAA3toDfi	206h

9.4.3.6.117.2 Diagram



9.4.3.6.117.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAA3toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 3. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 3. Unused for DDR3 and unused for DDR4 applications. Each register in the set of MapCAAtoDfi must have a unique value within the set. For example, if PHY CAA3 is mapped from dfi0_address[2], then Register MapCAA3toDfi should be 2. LP4 ACX3,ACX6,ACX10,ACX12 registers MapCAA0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAA6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6,ACX10,ACX12 registers MapCAA0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.118 Maps PHY CAA lane 4 from dfi0_address of the index of the register contents (MapCAA4toDfi)

9.4.3.6.118.1 Offset

Register	Offset
MapCAA4toDfi	208h

9.4.3.6.118.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												MapCAA4toDfi			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0

9.4.3.6.118.3 Fields

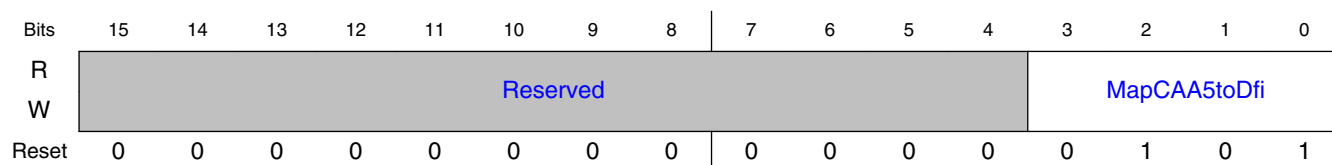
Field	Function
15-4 —	Reserved
3-0 MapCAA4toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 4. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 4. Unused for DDR3 and unused for DDR4 applications. Each register in the set of MapCAAtoDfi must have a unique value within the set. For example, if PHY CAA4 is mapped from dfi0_address[2], then Register MapCAA4toDfi should be 2. LP4 ACX3,ACX6,ACX10,ACX12 registers MapCAA0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAA6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6,ACX10,ACX12 registers MapCAA0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.119 Maps PHY CAA lane 5 from dfi0_address of the index of the register contents (MapCAA5toDfi)

9.4.3.6.119.1 Offset

Register	Offset
MapCAA5toDfi	20Ah

9.4.3.6.119.2 Diagram



9.4.3.6.119.3 Fields

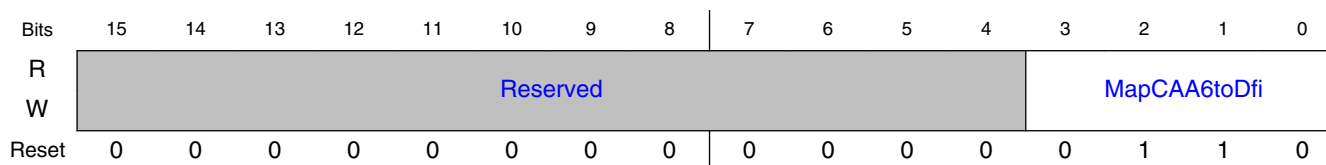
Field	Function
15-4 —	Reserved
3-0 MapCAA5toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 5. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 5. Unused for DDR3 and unused for DDR4 applications. Each register in the set of MapCAAtoDfi must have a unique value within the set. For example, if PHY CAA5 is mapped from dfi0_address[2], then Register MapCAA5toDfi should be 2. LP4 ACX3,ACX6,ACX10,ACX12 registers MapCAA0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAA6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6,ACX10,ACX12 registers MapCAA0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.120 Maps PHY CAA lane 6 from dfi0_address of the index of the register contents (MapCAA6toDfi)

9.4.3.6.120.1 Offset

Register	Offset
MapCAA6toDfi	20Ch

9.4.3.6.120.2 Diagram



9.4.3.6.120.3 Fields

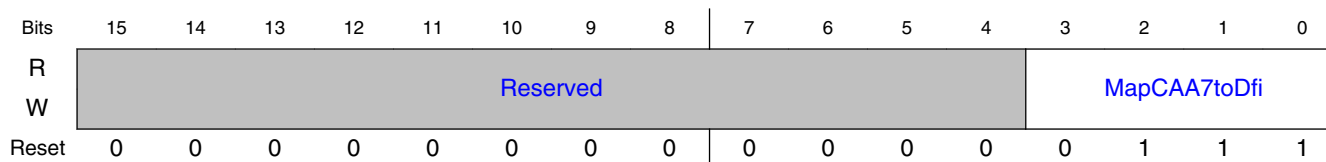
Field	Function
15-4 —	Reserved
3-0 MapCAA6toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 6. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 6. Unused for DDR3 and unused for DDR4 applications. Each register in the set of MapCAAtoDfi must have a unique value within the set. For example, if PHY CAA6 is mapped from dfi0_address[2], then Register MapCAA6toDfi should be 2. LP4 ACX3,ACX6,ACX10,ACX12 registers MapCAA0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAA6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6,ACX10,ACX12 registers MapCAA0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.121 Maps PHY CAA lane 7 from dfi0_address of the index of the register contents (MapCAA7toDfi)

9.4.3.6.121.1 Offset

Register	Offset
MapCAA7toDfi	20Eh

9.4.3.6.121.2 Diagram



9.4.3.6.121.3 Fields

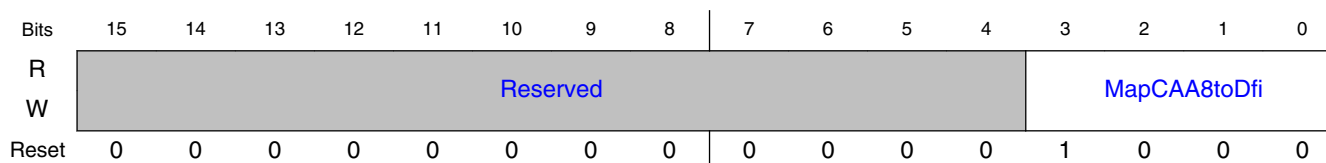
Field	Function
15-4 —	Reserved
3-0 MapCAA7toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 7. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 7. Unused for DDR3 and unused for DDR4 applications. Each register in the set of MapCAAtoDfi must have a unique value within the set. For example, if PHY CAA7 is mapped from dfi0_address[2], then Register MapCAA7toDfi should be 2. LP4 ACX3,ACX6,ACX10,ACX12 registers MapCAA0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAA6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6,ACX10,ACX12 registers MapCAA0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.122 Maps PHY CAA lane 8 from dfi0_address of the index of the register contents (MapCAA8toDfi)

9.4.3.6.122.1 Offset

Register	Offset
MapCAA8toDfi	210h

9.4.3.6.122.2 Diagram



9.4.3.6.122.3 Fields

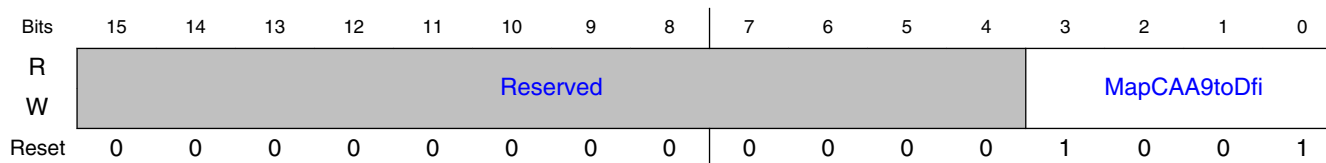
Field	Function
15-4 —	Reserved
3-0 MapCAA8toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 8. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 8. Unused for DDR3 and unused for DDR4 applications. Each register in the set of MapCAAtoDfi must have a unique value within the set. For example, if PHY CAA8 is mapped from dfi0_address[2], then Register MapCAA8toDfi should be 2. LP4 ACX3,ACX6,ACX10,ACX12 registers MapCAA0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAA6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6,ACX10,ACX12 registers MapCAA0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.123 Maps PHY CAA lane 9 from dfi0_address of the index of the register contents (MapCAA9toDfi)

9.4.3.6.123.1 Offset

Register	Offset
MapCAA9toDfi	212h

9.4.3.6.123.2 Diagram



9.4.3.6.123.3 Fields

Field	Function
15-4	Reserved

Table continues on the next page...

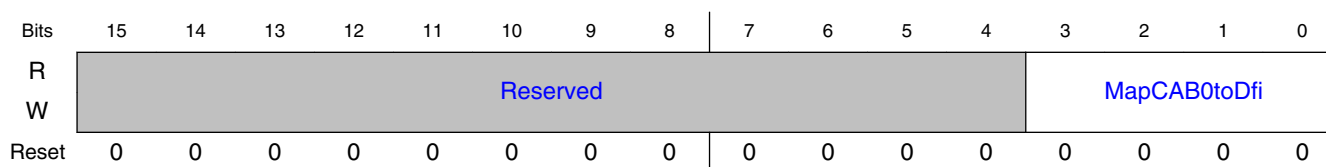
Field	Function
—	
3-0 MapCAA9toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 9. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi0_address to CAA 9. Unused for DDR3 and unused for DDR4 applications. Each register in the set of MapCAAtoDfi must have a unique value within the set. For example, if PHY CAA9 is mapped from dfi0_address[2], then Register MapCAA9toDfi should be 2. LP4 ACX3,ACX6,ACX10,ACX12 registers MapCAA0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAA6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6,ACX10,ACX12 registers MapCAA0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.124 Maps PHY CAB lane 0 from dfi1_address of the index of the register contents (MapCAB0toDfi)

9.4.3.6.124.1 Offset

Register	Offset
MapCAB0toDfi	220h

9.4.3.6.124.2 Diagram



9.4.3.6.124.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAB0toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 0. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 0.

DDR PHY (DDR_PHY)

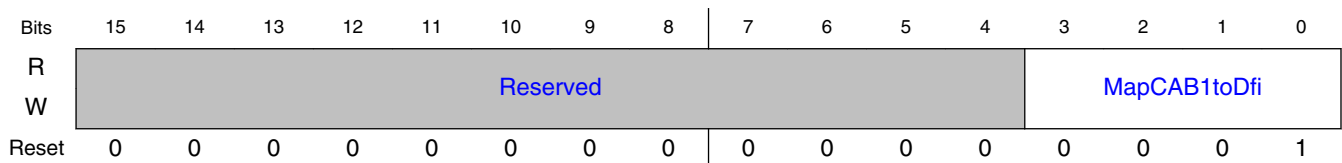
Field	Function
	Unused for DDR3 and unused for DDR4 applications. Unused for the 3-ANIB configuration. Each register in the set of MapCABtoDfi must have a unique value within the set. For example, if PHY CAB0 is mapped from dfi1_address[2], then Register MapCAB0toDfi should be 2. LP4 ACX3 registers MapCAB0..9toDfi are reserved, second channel not supported LP4 ACX6,ACX10,ACX12 registers MapCAB0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAB6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6 registers MapCAB0..9 are unused, second channel not supported LP3 ACX10,ACX12 registers MapCAB0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.125 Maps PHY CAB lane 1 from dfi1_address of the index of the register contents (MapCAB1toDfi)

9.4.3.6.125.1 Offset

Register	Offset
MapCAB1toDfi	222h

9.4.3.6.125.2 Diagram



9.4.3.6.125.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAB1toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 1. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 1.

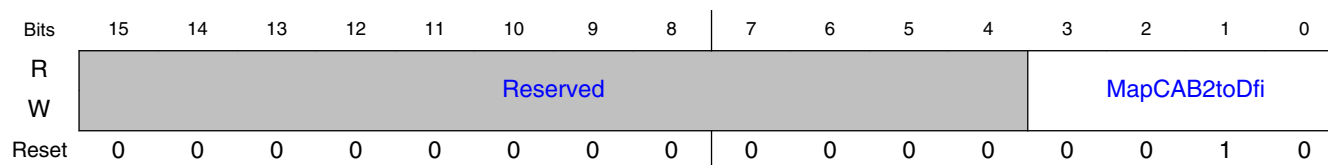
Field	Function
	Unused for DDR3 and unused for DDR4 applications. Unused for the 3-ANIB configuration. Each register in the set of MapCABtoDfi must have a unique value within the set. For example, if PHY CAB1 is mapped from dfi1_address[2], then Register MapCAB1toDfi should be 2. LP4 ACX3 registers MapCAB0..9toDfi are reserved, second channel not supported LP4 ACX6,ACX10,ACX12 registers MapCAB0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAB6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6 registers MapCAB0..9 are unused, second channel not supported LP3 ACX10,ACX12 registers MapCAB0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.126 Maps PHY CAB lane 2 from dfi1_address of the index of the register contents (MapCAB2toDfi)

9.4.3.6.126.1 Offset

Register	Offset
MapCAB2toDfi	224h

9.4.3.6.126.2 Diagram



9.4.3.6.126.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAB2toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 2. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 2.

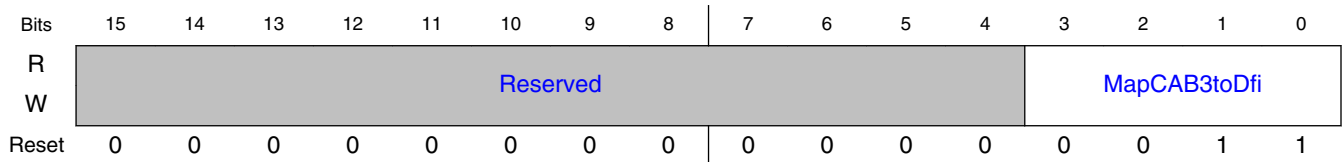
Field	Function
	Unused for DDR3 and unused for DDR4 applications. Unused for the 3-ANIB configuration. Each register in the set of MapCABtoDfi must have a unique value within the set. For example, if PHY CAB2 is mapped from dfi1_address[2], then Register MapCAB2toDfi should be 2. LP4 ACX3 registers MapCAB0..9toDfi are reserved, second channel not supportd LP4 ACX6,ACX10,ACX12 registers MapCAB0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAB6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6 registers MapCAB0..9 are unused, second channel not supported LP3 ACX10,ACX12 registers MapCAB0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.127 Maps PHY CAB lane 3 from dfi1_address of the index of the register contents (MapCAB3toDfi)

9.4.3.6.127.1 Offset

Register	Offset
MapCAB3toDfi	226h

9.4.3.6.127.2 Diagram



9.4.3.6.127.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAB3toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 3. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 3.

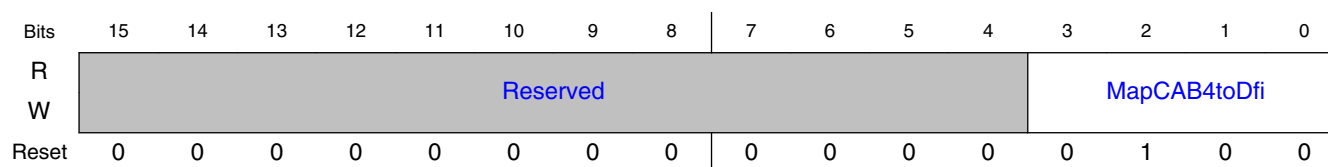
Field	Function
	Unused for DDR3 and unused for DDR4 applications. Unused for the 3-ANIB configuration. Each register in the set of MapCABtoDfi must have a unique value within the set. For example, if PHY CAB3 is mapped from dfi1_address[2], then Register MapCAB3toDfi should be 2. LP4 ACX3 registers MapCAB0..9toDfi are reserved, second channel not supported LP4 ACX6,ACX10,ACX12 registers MapCAB0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAB6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6 registers MapCAB0..9 are unused, second channel not supported LP3 ACX10,ACX12 registers MapCAB0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.128 Maps PHY CAB lane 4 from dfi1_address of the index of the register contents (MapCAB4toDfi)

9.4.3.6.128.1 Offset

Register	Offset
MapCAB4toDfi	228h

9.4.3.6.128.2 Diagram



9.4.3.6.128.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAB4toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 4. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 4.

DDR PHY (DDR_PHY)

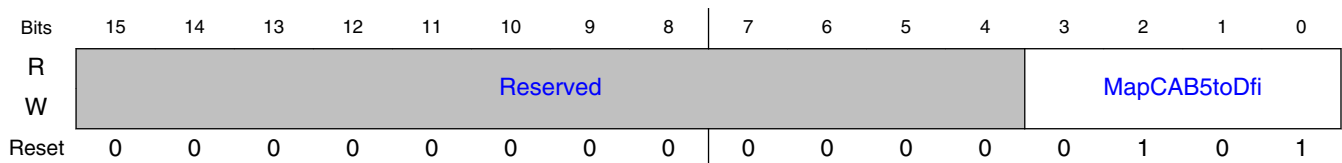
Field	Function
	Unused for DDR3 and unused for DDR4 applications. Unused for the 3-ANIB configuration. Each register in the set of MapCABtoDfi must have a unique value within the set. For example, if PHY CAB4 is mapped from dfi1_address[2], then Register MapCAB4toDfi should be 2. LP4 ACX3 registers MapCAB0..9toDfi are reserved, second channel not supported LP4 ACX6,ACX10,ACX12 registers MapCAB0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAB6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6 registers MapCAB0..9 are unused, second channel not supported LP3 ACX10,ACX12 registers MapCAB0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.129 Maps PHY CAB lane 5 from dfi1_address of the index of the register contents (MapCAB5toDfi)

9.4.3.6.129.1 Offset

Register	Offset
MapCAB5toDfi	22Ah

9.4.3.6.129.2 Diagram



9.4.3.6.129.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAB5toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 5. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 5.

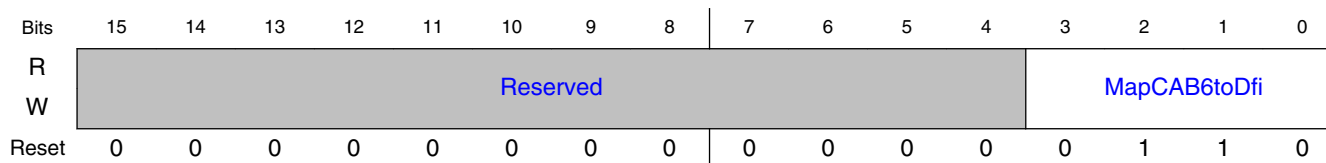
Field	Function
	Unused for DDR3 and unused for DDR4 applications. Unused for the 3-ANIB configuration. Each register in the set of MapCABtoDfi must have a unique value within the set. For example, if PHY CAB5 is mapped from dfi1_address[2], then Register MapCAB5toDfi should be 2. LP4 ACX3 registers MapCAB0..9toDfi are reserved, second channel not supported LP4 ACX6,ACX10,ACX12 registers MapCAB0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAB6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6 registers MapCAB0..9 are unused, second channel not supported LP3 ACX10,ACX12 registers MapCAB0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.130 Maps PHY CAB lane 6 from dfi1_address of the index of the register contents (MapCAB6toDfi)

9.4.3.6.130.1 Offset

Register	Offset
MapCAB6toDfi	22Ch

9.4.3.6.130.2 Diagram



9.4.3.6.130.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAB6toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 6. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 6.

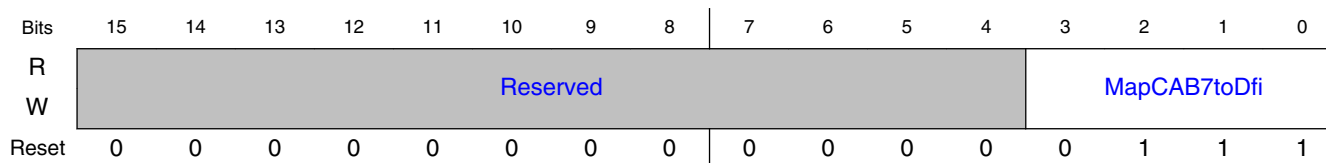
Field	Function
	Unused for DDR3 and unused for DDR4 applications. Unused for the 3-ANIB configuration. Each register in the set of MapCABtoDfi must have a unique value within the set. For example, if PHY CAB6 is mapped from dfi1_address[2], then Register MapCAB6toDfi should be 2. LP4 ACX3 registers MapCAB0..9toDfi are reserved, second channel not supported LP4 ACX6,ACX10,ACX12 registers MapCAB0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAB6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6 registers MapCAB0..9 are unused, second channel not supported LP3 ACX10,ACX12 registers MapCAB0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.131 Maps PHY CAB lane 7 from dfi1_address of the index of the register contents (MapCAB7toDfi)

9.4.3.6.131.1 Offset

Register	Offset
MapCAB7toDfi	22Eh

9.4.3.6.131.2 Diagram



9.4.3.6.131.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAB7toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 7. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 7.

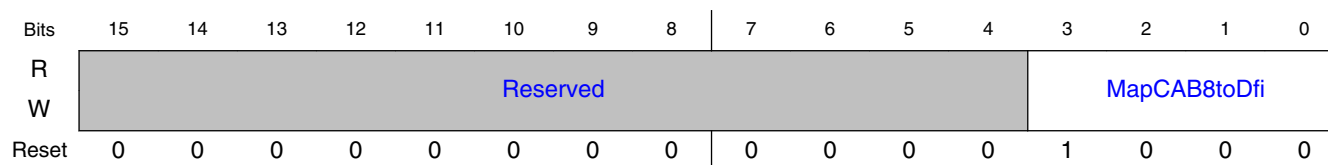
Field	Function
	Unused for DDR3 and unused for DDR4 applications. Unused for the 3-ANIB configuration. Each register in the set of MapCABtoDfi must have a unique value within the set. For example, if PHY CAB7 is mapped from dfi1_address[2], then Register MapCAB7toDfi should be 2. LP4 ACX3 registers MapCAB0..9toDfi are reserved, second channel not supported LP4 ACX6,ACX10,ACX12 registers MapCAB0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAB6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6 registers MapCAB0..9 are unused, second channel not supported LP3 ACX10,ACX12 registers MapCAB0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.132 Maps PHY CAB lane 8 from dfi1_address of the index of the register contents (MapCAB8toDfi)

9.4.3.6.132.1 Offset

Register	Offset
MapCAB8toDfi	230h

9.4.3.6.132.2 Diagram



9.4.3.6.132.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAB8toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 8. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 8.

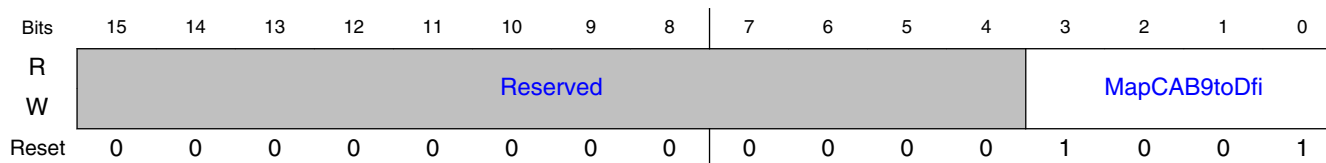
Field	Function
	Unused for DDR3 and unused for DDR4 applications. Unused for the 3-ANIB configuration. Each register in the set of MapCABtoDfi must have a unique value within the set. For example, if PHY CAB8 is mapped from dfi1_address[2], then Register MapCAB8toDfi should be 2. LP4 ACX3 registers MapCAB0..9toDfi are reserved, second channel not supported LP4 ACX6,ACX10,ACX12 registers MapCAB0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAB6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6 registers MapCAB0..9 are unused, second channel not supported LP3 ACX10,ACX12 registers MapCAB0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.133 Maps PHY CAB lane 9 from dfi1_address of the index of the register contents (MapCAB9toDfi)

9.4.3.6.133.1 Offset

Register	Offset
MapCAB9toDfi	232h

9.4.3.6.133.2 Diagram



9.4.3.6.133.3 Fields

Field	Function
15-4 —	Reserved
3-0 MapCAB9toDfi	For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 9. For LPDDR3 and LPDDR4 applications, these CSRs map a dfi1_address to CAB 9.

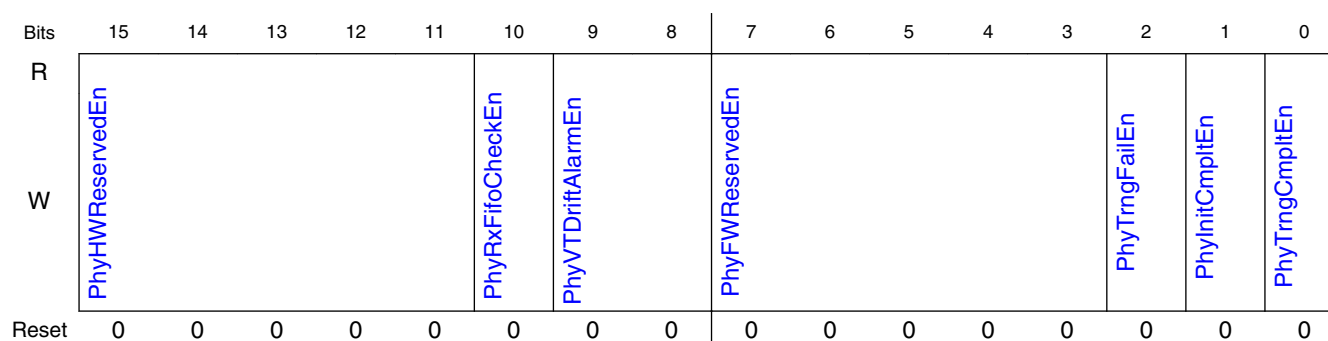
Field	Function
	Unused for DDR3 and unused for DDR4 applications. Unused for the 3-ANIB configuration. Each register in the set of MapCABtoDfi must have a unique value within the set. For example, if PHY CAB9 is mapped from dfi1_address[2], then Register MapCAB9toDfi should be 2. LP4 ACX3 registers MapCAB0..9toDfi are reserved, second channel not supported LP4 ACX6,ACX10,ACX12 registers MapCAB0..5toDfi are used, values 0-5 are valid, values 6-15 are reserved registers MapCAB6..9toDfi are reserved LP3 ACX3 not supported LP3 ACX6 registers MapCAB0..9 are unused, second channel not supported LP3 ACX10,ACX12 registers MapCAB0..9toDfi are used, values 0-9 are valid, values 10-15 are reserved

9.4.3.6.134 Interrupt Enable Bits (PhyInterruptEnable)

9.4.3.6.134.1 Offset

Register	Offset
PhyInterruptEnable	236h

9.4.3.6.134.2 Diagram



9.4.3.6.134.3 Fields

Field	Function
15-11	Reserved

Table continues on the next page...

DDR PHY (DDR_PHY)

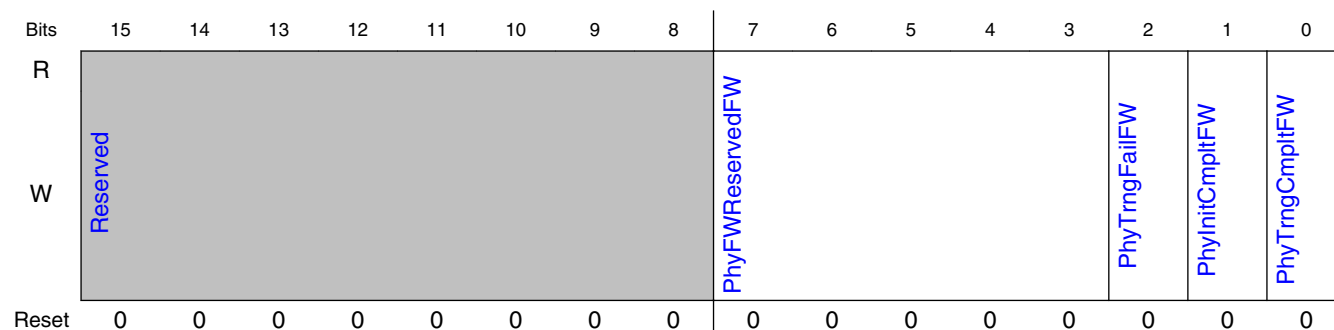
Field	Function
PhyHWReservedEn	
10 PhyRxFifoCheckEn	Enable for the RxFifo Pointers Check Interrupt 0 : Interrupt not enabled 1 : Interrupt enabled
9-8 PhyVTDriftAlarmEn	Enable for the PHY VT Drift Alarm interrupts. Enable for the PHY VT Drift Alarm interrupts. 0 : Interrupt not enabled 1 : Interrupt enabled
7-3 PhyFWReservedEn	Reserved
2 PhyTrngFailEn	Enable for the PHY Training Failure interrupt. Enable for the PHY Training Failure interrupt. 0 : Interrupt not enabled 1 : Interrupt enabled
1 PhyInitCmpltEn	Enable for the PHY Initialization Complete interrupt. Enable for the PHY Initialization Complete interrupt. 0 : Interrupt not enabled 1 : Interrupt enabled
0 PhyTrngCmpltEn	Enable for the PHY Training Complete interrupt. Enable for the PHY Training Complete interrupt. 0 : Interrupt not enabled 1 : Interrupt enabled

9.4.3.6.135 Interrupt Firmware Control Bits (PhyInterruptFWControl)

9.4.3.6.135.1 Offset

Register	Offset
PhyInterruptFWControl	238h

9.4.3.6.135.2 Diagram



9.4.3.6.135.3 Fields

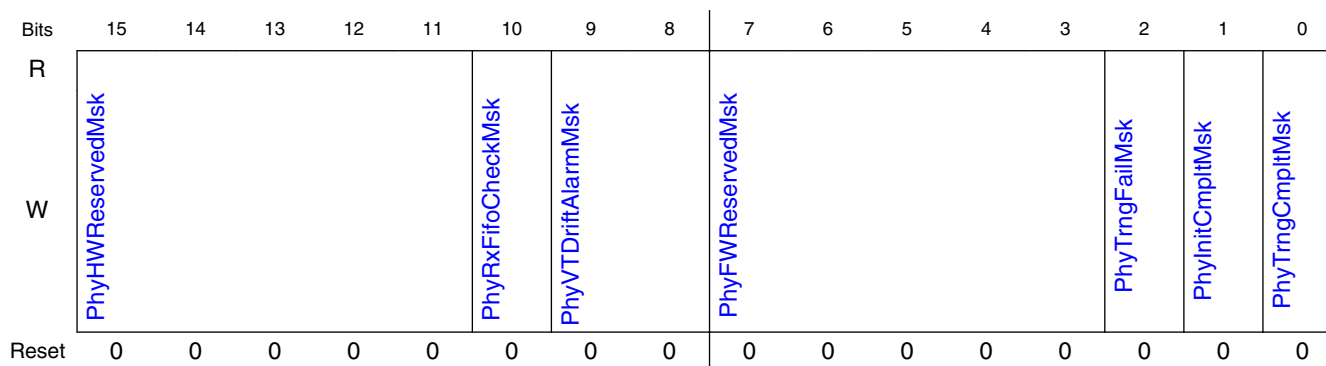
Field	Function
15-8 —	Reserved
7-3 PhyFWReserve dFW	Reserved
2 PhyTrngFailFW	PHY Training Failure Firmware interrupt. PHY Training Failure Firmware interrupt. 0 : Interrupt not asserted 1 : Interrupt asserted
1 PhyInitCmpltFW	PHY Initialization Complete Firmware interrupt. PHY Initialization Complete Firmware interrupt. 0 : Interrupt not asserted 1 : Interrupt asserted
0 PhyTrngCmpltF W	PHY Training Complete Firmware interrupt. PHY Training Complete Firmware interrupt. 0 : Interrupt not asserted 1 : Interrupt asserted

9.4.3.6.136 Interrupt Mask Bits (PhyInterruptMask)

9.4.3.6.136.1 Offset

Register	Offset
PhyInterruptMask	23Ah

9.4.3.6.136.2 Diagram



9.4.3.6.136.3 Fields

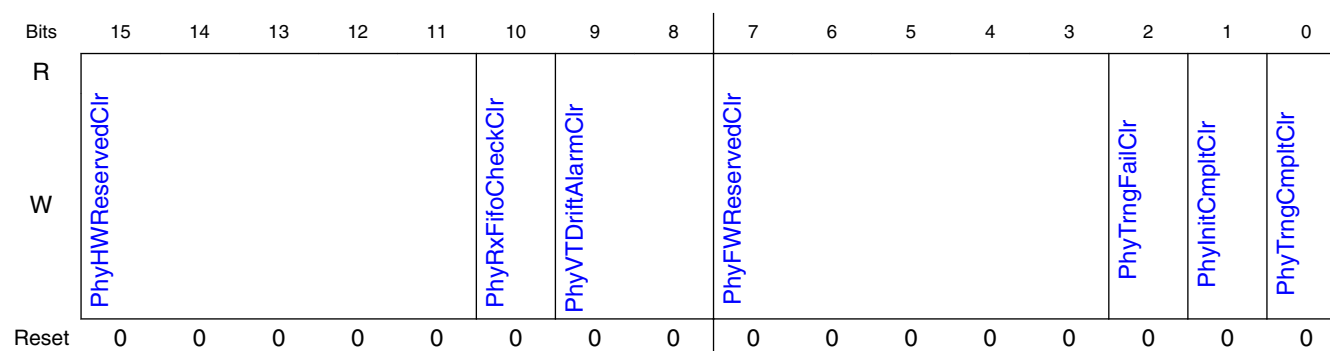
Field	Function
15-11 PhyHWReserve dMsk	Reserved
10 PhyRxFifoCheck Msk	Mask for the RxFifo Pointers Check Interrupt 0 : Interrupt not masked 1 : Interrupt masked
9-8 PhyVTDriftAlarm Msk	Mask for the PHY VT Drift Alarm interrupts. Mask for the PHY VT Drift Alarm interrupts. 0 : Interrupt not masked 1 : Interrupt masked
7-3 PhyFWReserve dMsk	Reserved
2 PhyTrngFailMsk	Mask for the PHY Training Failure interrupt. Mask for the PHY Training Failure interrupt. 0 : Interrupt not masked 1 : Interrupt masked
1 PhyInitCmplM sk	Mask for the PHY Initialization Complete interrupt. Mask for the PHY Initialization Complete interrupt. 0 : Interrupt not masked 1 : Interrupt masked
0 PhyTrngCmplM sk	Mask for the PHY Training Complete interrupt. Mask for the PHY Training Complete interrupt. 0 : Interrupt not masked 1 : Interrupt masked

9.4.3.6.137 Interrupt Clear Bits (PhyInterruptClear)

9.4.3.6.137.1 Offset

Register	Offset
PhyInterruptClear	23Ch

9.4.3.6.137.2 Diagram



9.4.3.6.137.3 Fields

Field	Function
15-11 PhyHWReservedClr	Reserved
10 PhyRxFifoCheckClr	Clear for the RxFifo Pointers Check Interrupt 0 : Interrupt not affected 1 : Interrupt cleared
9-8 PhyVTDriftAlarmClr	Clear for the PHY VT Drift Alarm interrupt. Clear for the PHY VT Drift Alarm interrupt. 0 : Interrupt not affected 1 : Interrupt cleared
7-3 PhyFWReservedClr	Reserved
2 PhyTrngFailClr	Clear for the PHY Training Failure interrupt. Clear for the PHY Training Failure interrupt. 0 : Interrupt not affected 1 : Interrupt cleared

Table continues on the next page...

DDR PHY (DDR_PHY)

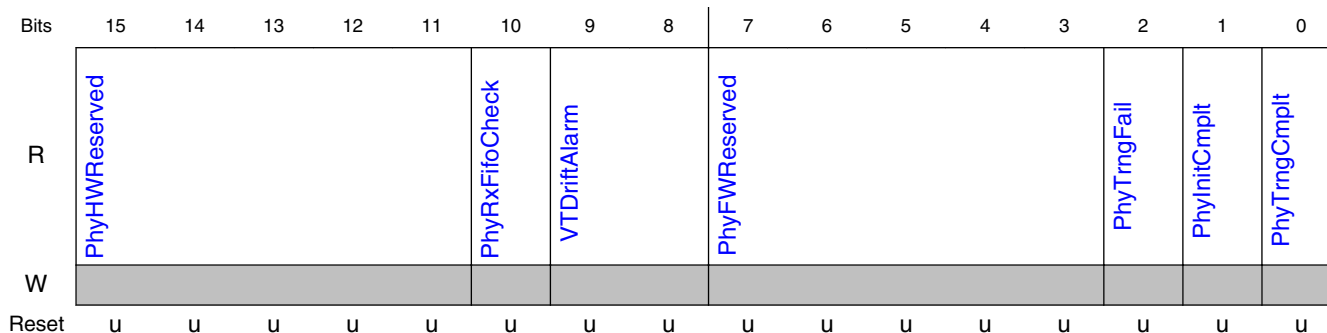
Field	Function
1 PhyInitCmpltClr	Clear for the PHY Initialization Complete interrupt. Clear for the PHY Initialization Complete interrupt. 0 : Interrupt not affected 1 : Interrupt cleared
0 PhyTrngCmpltClr	Clear for the PHY Training Complete interrupt. Clear for the PHY Training Complete interrupt. 0 : Interrupt not affected 1 : Interrupt cleared

9.4.3.6.138 Interrupt Status Bits (PhyInterruptStatus)

9.4.3.6.138.1 Offset

Register	Offset
PhyInterruptStatus	23Eh

9.4.3.6.138.2 Diagram



9.4.3.6.138.3 Fields

Field	Function
15-11 PhyHWReserved	Reserved
10 PhyRxFifoCheck	A mechanism in the PHY checks the Read Fifo pointers for consistency at times they are idle. A mechanism in the PHY checks the Read Fifo pointers for consistency at times they are idle. If a check fails, this interrupt is asserted, and detailed status is found in the RxFifoCheckStatus register.

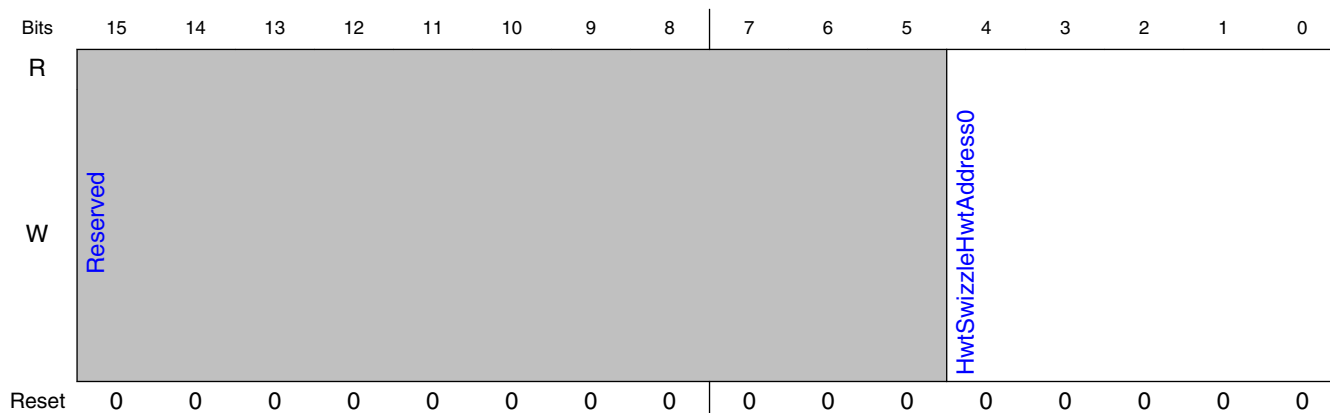
Table continues on the next page...

Field	Function
	0 : Interrupt not enabled and asserted 1 : Interrupt enabled and asserted
9-8 VTDriftAlarm	PHY VT Drift Alarm interrupt. PHY VT Drift Alarm interrupt. Assertion of this interrupt indicates the number of tap points in the master DLL has shifted by an amount greater than $2 * \text{DfiPhyUpdIntThreshold}$. There is one bit for each master DLL and they are fully independent of one another. Please see the <code>DfiPhyUpdIntThreshold</code> CSR for additional information. 0 : Interrupt not enabled and asserted 1 : Interrupt enabled and asserted
7-3 PhyFWReserved	Reserved
2 PhyTrngFail	PHY Training Failure interrupt. PHY Training Failure interrupt. Assertion of this interrupt indicates there was an issue during training which did not allow the PHY to successfully complete training. 0 : Interrupt not enabled and asserted 1 : Interrupt enabled and asserted
1 PhyInitCmpl	PHY Initialization Complete interrupt. PHY Initialization Complete interrupt. Assertion of this interrupt indicates that the PHY Initialization Engine has been fully loaded and the PHY is ready and able to assert <code>dfi_init_complete</code> following the assertion of <code>dfi_init_start</code> . It is possible this interrupt may not assert until after <code>dfi_init_complete</code> has asserted, depending on when <code>dfi_init_start</code> asserts. 0 : Interrupt not enabled and asserted 1 : Interrupt enabled and asserted
0 PhyTrngCmpl	PHY Training Complete interrupt. PHY Training Complete interrupt. Assertion of this interrupt indicates that the PHY has successfully trained the memory interface at all indicated frequencies and is ready for the PHY Initialization Engine to be loaded. 0 : Interrupt not enabled and asserted 1 : Interrupt enabled and asserted

9.4.3.6.139 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress0)

9.4.3.6.139.1 Offset

Register	Offset
HwtSwizzleHwtAddress0	240h

9.4.3.6.139.2 Diagram**9.4.3.6.139.3 Fields**

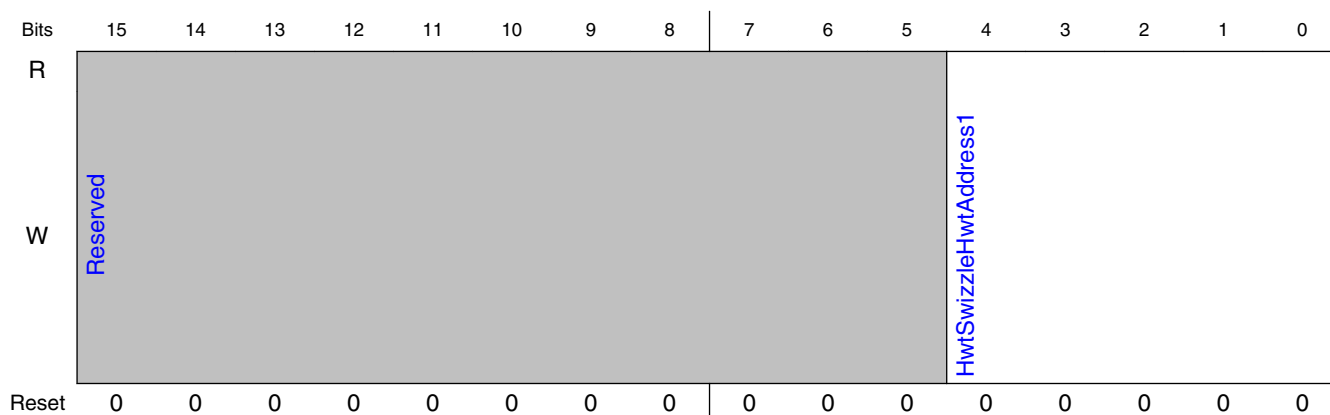
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress0	This set of registers is used in DDR3/DDR4 mode where a user has re-mapped the DFI inputs to the PHY. This set of registers is used in DDR3/DDR4 mode where a user has re-mapped the DFI inputs to the PHY. This register does not affect the mapping of those signals, or remap the tristate properties of the bumps. However, when the user remaps the DFI signals, the HWT inputs must be mapped in a similar fashion using this register. Each remappable hwt bit has a register associated with it. When the register value is zero, the bit is not re-mapped. When the register value is non-zero, the signal associated with the number in the table below is mapped to the signal associated with the register. So, for example, if you set the value for the register associated with hwt_act_n (HwtSwizzleHwtActN) to 27, then hwt_parity_in will be mapped to hwt_act_n. These registers should always be 0 for LPDDR3 and LPDDR4. Some registers are used only for DDR3 or DDR4. These are noted in the descriptions. Note: If the system uses DDR3-RDIMM or

Field	Function
	DDR4-RDIMM/LRDIMM, it is strongly recommended NOT to swizzle the PAR pin (parity). If the PAR pin is swizzled, the Dynamic Address Tristate feature must be disabled. (see the DisDynAdrTri field of the TristateMode CA Register). ----- Index: Signal 0: No-remapping 1: hwt_address[0] 2: hwt_address[1] 3: hwt_address[2] 4: hwt_address[3] 5: hwt_address[4] 6: hwt_address[5] 7: hwt_address[6] 8: hwt_address[7] 9: hwt_address[8] 10: hwt_address[9] 11: hwt_address[10] 12: hwt_address[11] 13: hwt_address[12] 14: hwt_address[13] 15: hwt_address[14] (DDR3 Only) 16: hwt_address[15] (DDR3 Only) 17: hwt_address[17] (DDR4 Only) 18: hwt_act_n (DDR4 Only) 19: hwt_bank[0] 20: hwt_bank[1] 21: hwt_bank[2] (DDR3 Only) 22: hwt_bg[0] (DDR4 Only) 23: hwt_bg[1] (DDR4 Only) 24: hwt_cas_n 25: hwt_ras_n 26: hwt_we_n 27: hwt_parity_in

9.4.3.6.140 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress1)

9.4.3.6.140.1 Offset

Register	Offset
HwtSwizzleHwtAddress1	242h

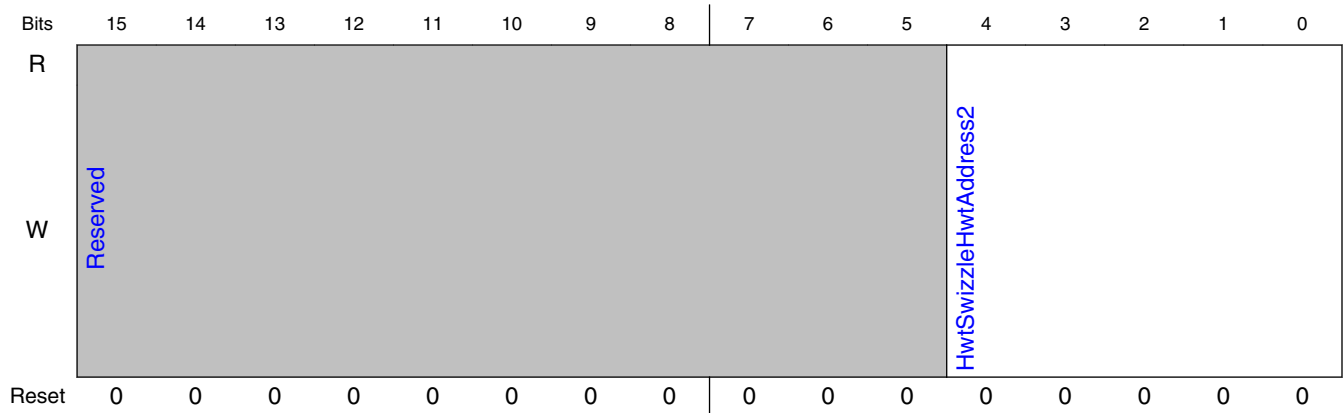
9.4.3.6.140.2 Diagram**9.4.3.6.140.3 Fields**

Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress1	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.141 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress2)**9.4.3.6.141.1 Offset**

Register	Offset
HwtSwizzleHwtAddress2	244h

9.4.3.6.141.2 Diagram



9.4.3.6.141.3 Fields

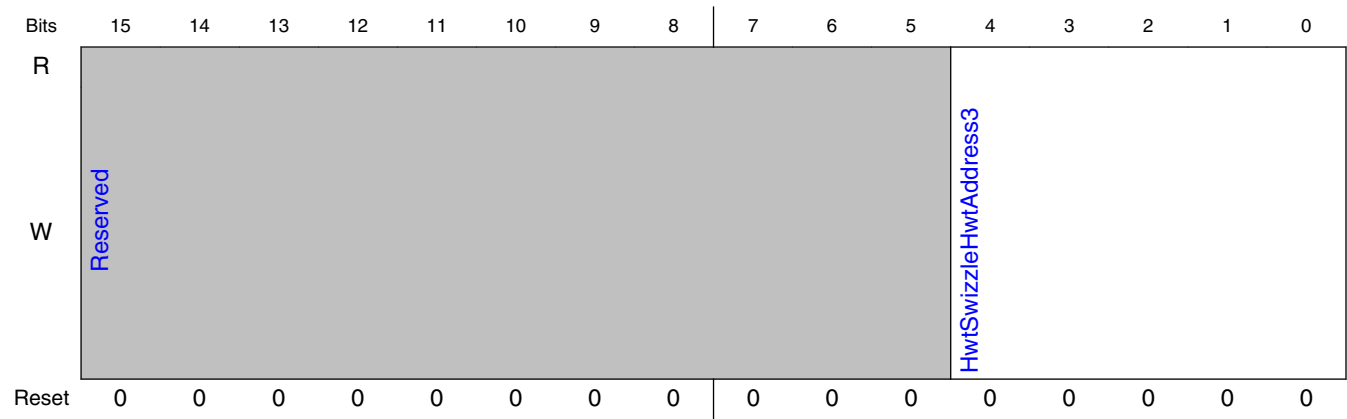
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress2	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.142 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress3)

9.4.3.6.142.1 Offset

Register	Offset
HwtSwizzleHwtAddress3	246h

9.4.3.6.142.2 Diagram



9.4.3.6.142.3 Fields

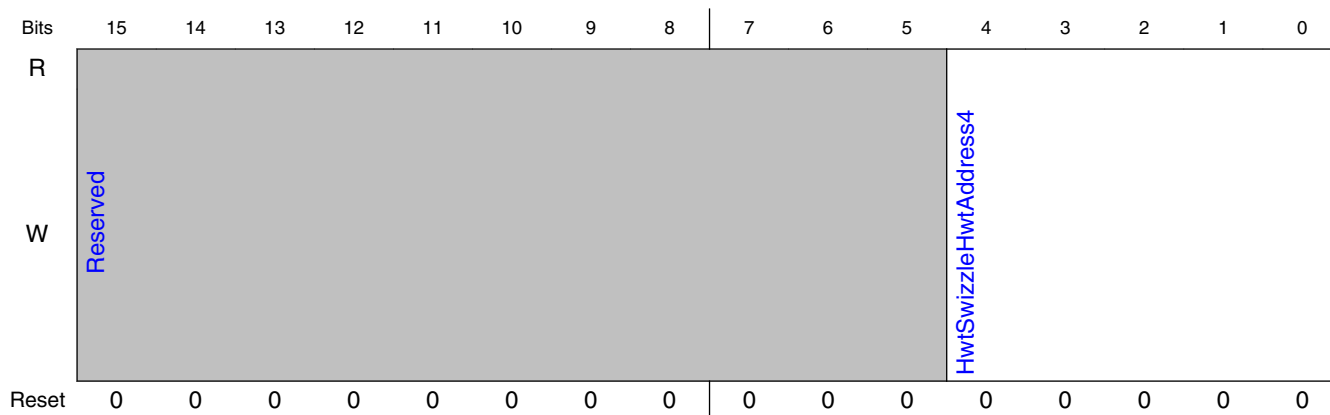
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress3	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.143 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress4)

9.4.3.6.143.1 Offset

Register	Offset
HwtSwizzleHwtAddress4	248h

9.4.3.6.143.2 Diagram



9.4.3.6.143.3 Fields

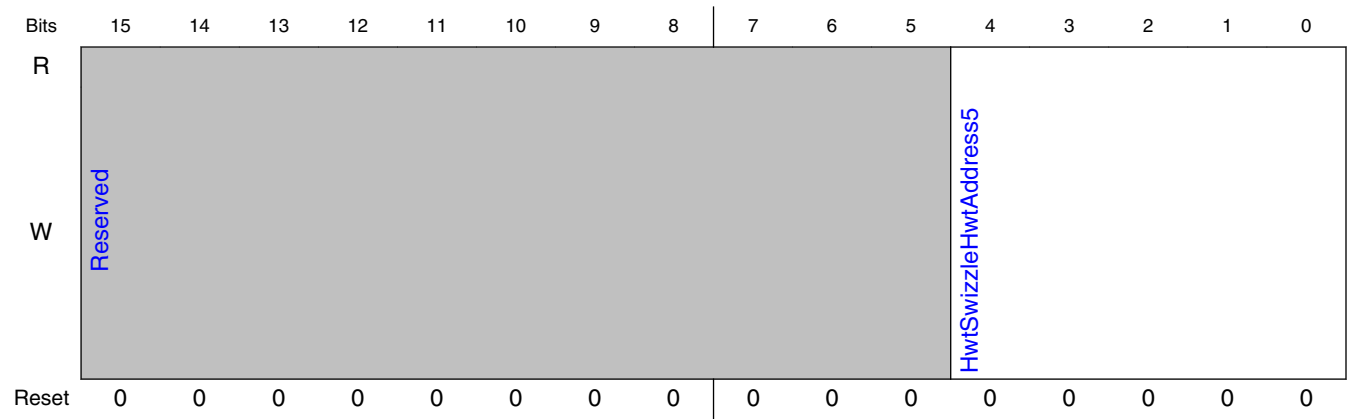
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress4	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.144 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress5)

9.4.3.6.144.1 Offset

Register	Offset
HwtSwizzleHwtAddress5	24Ah

9.4.3.6.144.2 Diagram



9.4.3.6.144.3 Fields

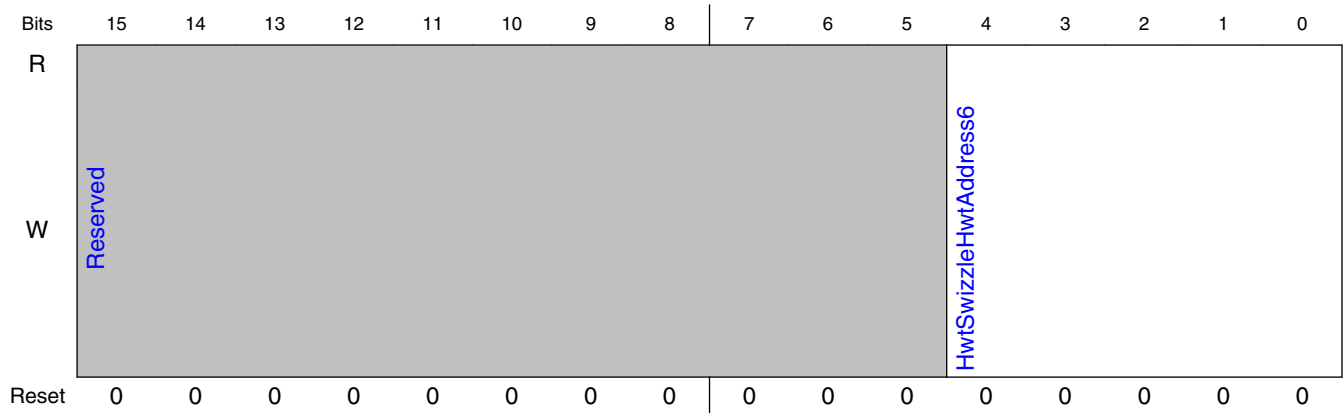
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress5	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.145 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress6)

9.4.3.6.145.1 Offset

Register	Offset
HwtSwizzleHwtAddress6	24Ch

9.4.3.6.145.2 Diagram



9.4.3.6.145.3 Fields

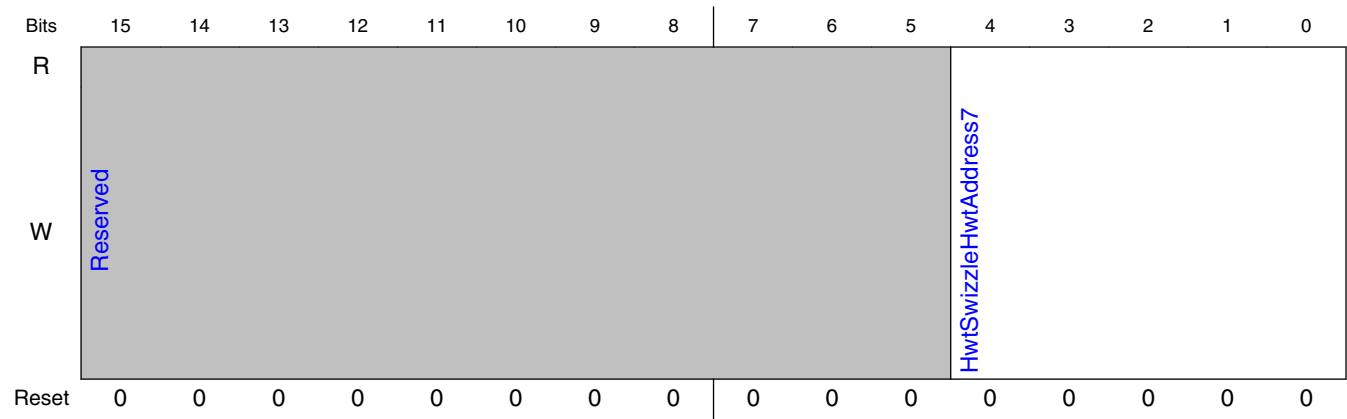
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress6	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.146 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress7)

9.4.3.6.146.1 Offset

Register	Offset
HwtSwizzleHwtAddress7	24Eh

9.4.3.6.146.2 Diagram



9.4.3.6.146.3 Fields

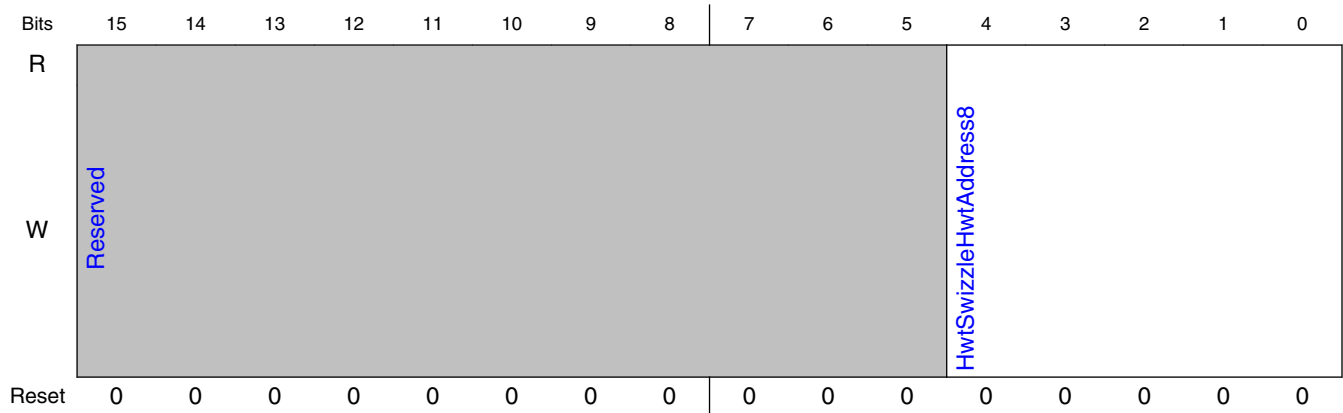
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress7	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.147 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress8)

9.4.3.6.147.1 Offset

Register	Offset
HwtSwizzleHwtAddress8	250h

9.4.3.6.147.2 Diagram



9.4.3.6.147.3 Fields

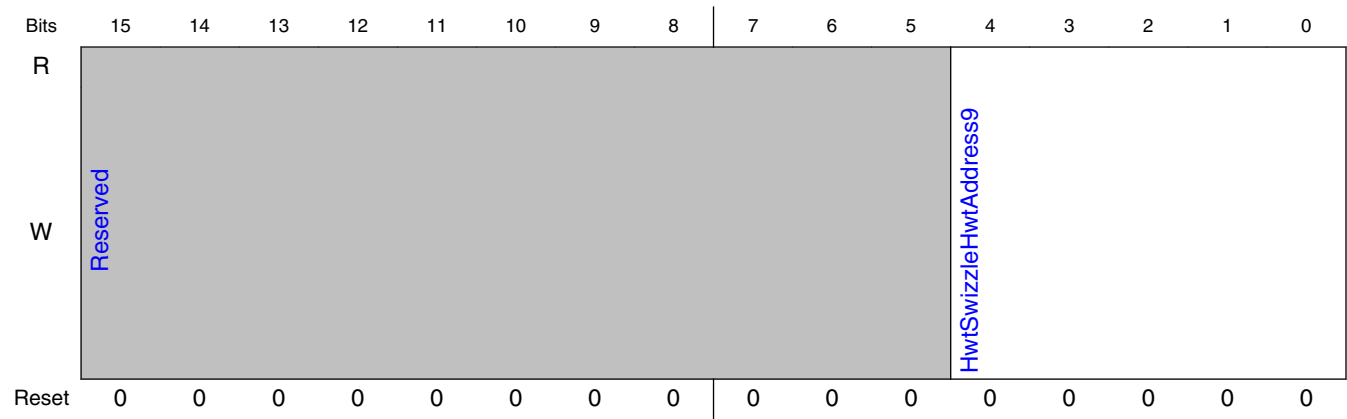
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress8	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.148 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress9)

9.4.3.6.148.1 Offset

Register	Offset
HwtSwizzleHwtAddress9	252h

9.4.3.6.148.2 Diagram



9.4.3.6.148.3 Fields

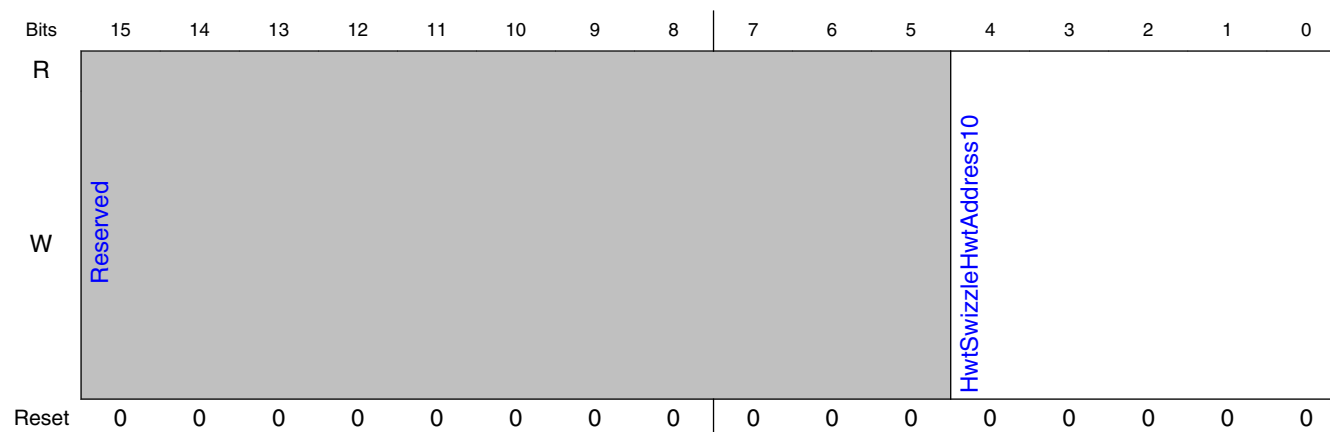
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress9	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.149 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress10)

9.4.3.6.149.1 Offset

Register	Offset
HwtSwizzleHwtAddress10	254h

9.4.3.6.149.2 Diagram



9.4.3.6.149.3 Fields

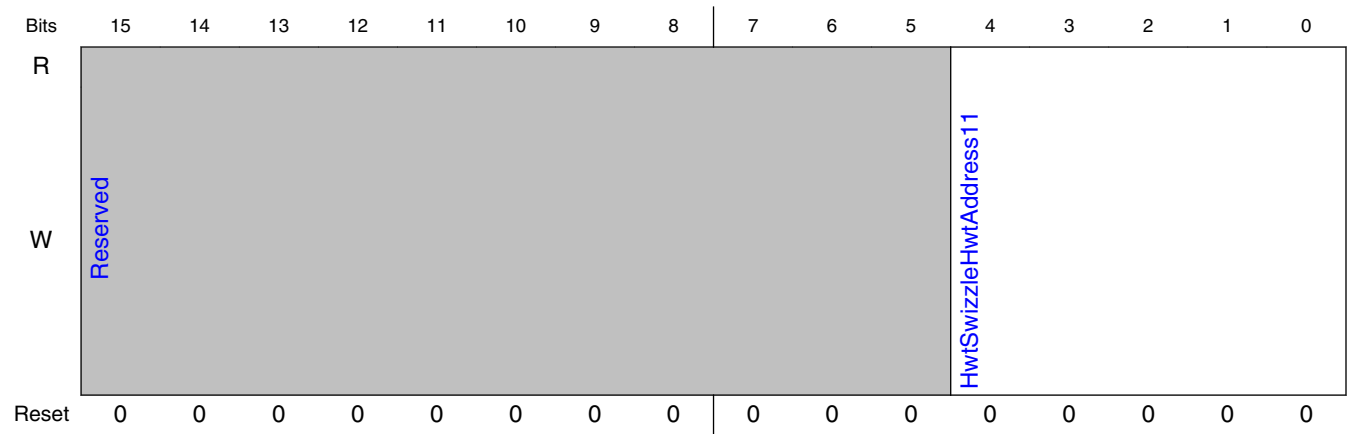
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress10	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.150 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress11)

9.4.3.6.150.1 Offset

Register	Offset
HwtSwizzleHwtAddress11	256h

9.4.3.6.150.2 Diagram



9.4.3.6.150.3 Fields

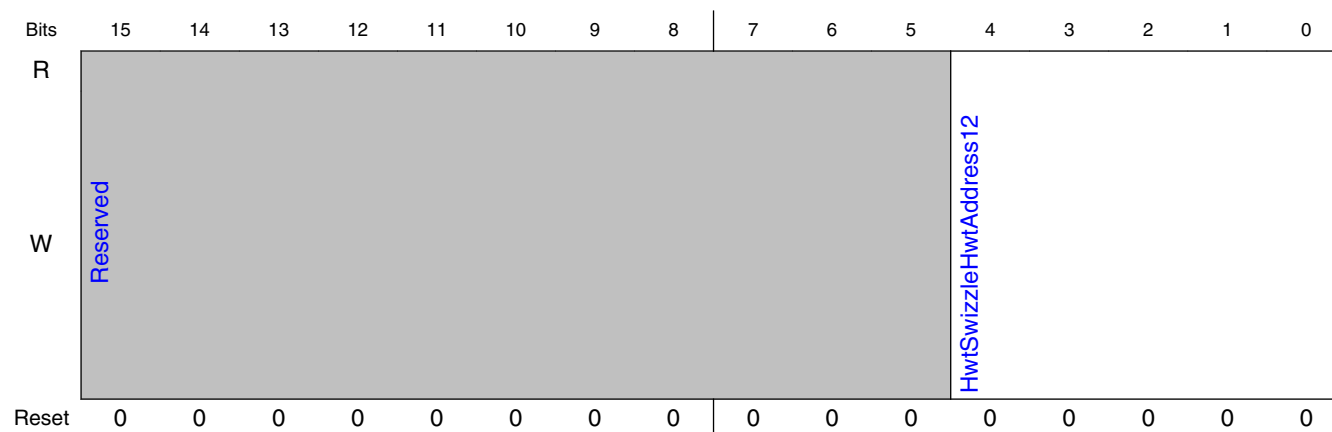
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress11	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.151 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress12)

9.4.3.6.151.1 Offset

Register	Offset
HwtSwizzleHwtAddress12	258h

9.4.3.6.151.2 Diagram



9.4.3.6.151.3 Fields

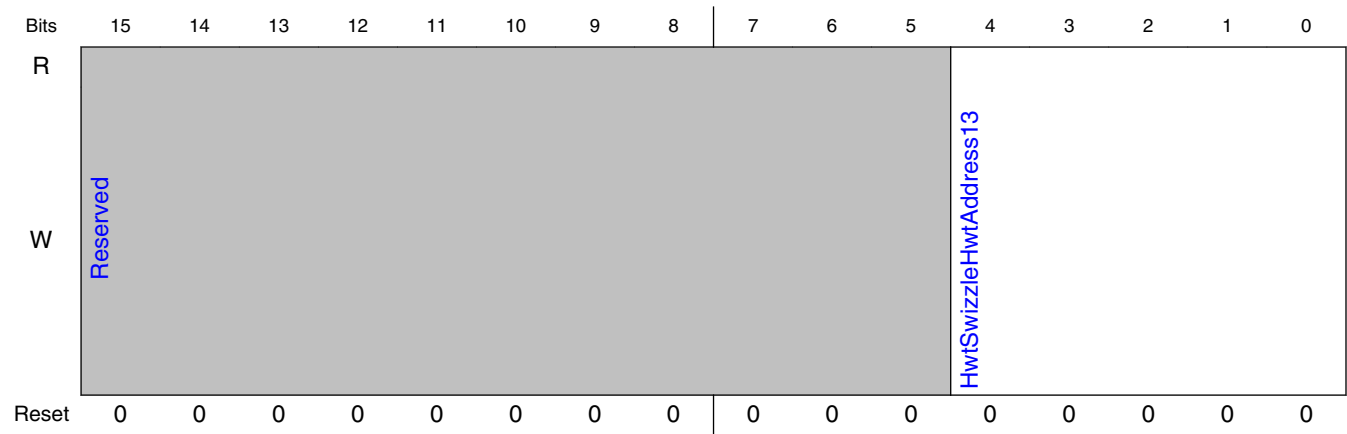
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress12	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.152 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress13)

9.4.3.6.152.1 Offset

Register	Offset
HwtSwizzleHwtAddress13	25Ah

9.4.3.6.152.2 Diagram



9.4.3.6.152.3 Fields

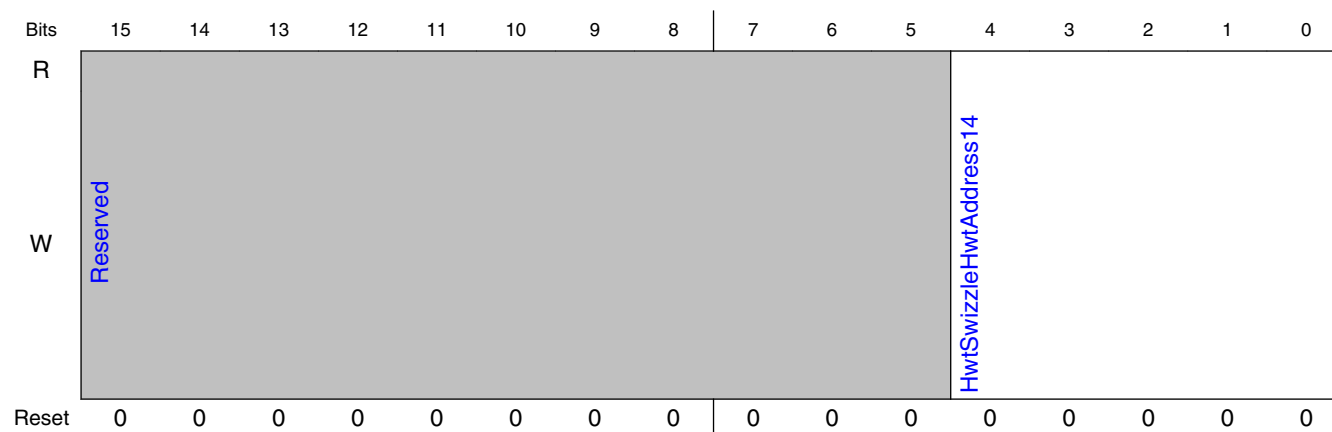
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress13	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.153 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress14)

9.4.3.6.153.1 Offset

Register	Offset
HwtSwizzleHwtAddress14	25Ch

9.4.3.6.153.2 Diagram



9.4.3.6.153.3 Fields

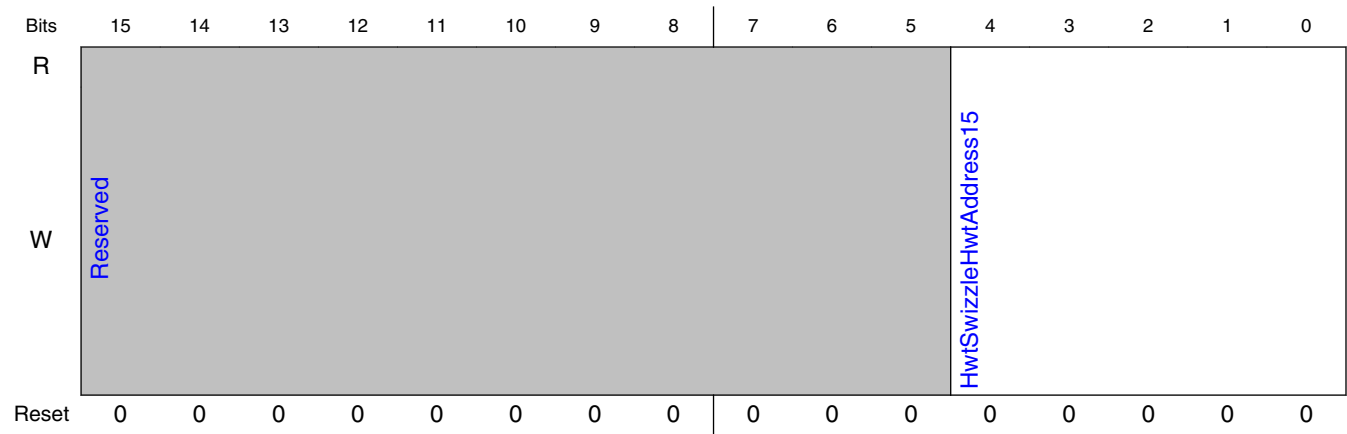
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress14	See Description of HwtSwizzleHwtAddress0 for details. See Description of HwtSwizzleHwtAddress0 for details. Used only for DDR3.

9.4.3.6.154 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress15)

9.4.3.6.154.1 Offset

Register	Offset
HwtSwizzleHwtAddress15	25Eh

9.4.3.6.154.2 Diagram



9.4.3.6.154.3 Fields

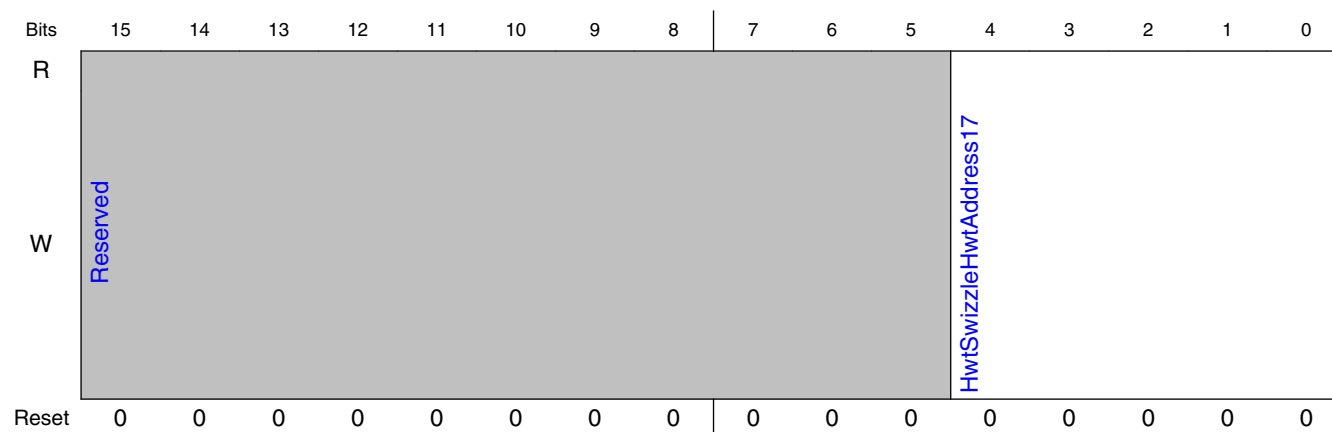
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress15	See Description of HwtSwizzleHwtAddress0 for details. See Description of HwtSwizzleHwtAddress0 for details. Used only for DDR3.

9.4.3.6.155 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtAddress17)

9.4.3.6.155.1 Offset

Register	Offset
HwtSwizzleHwtAddress17	260h

9.4.3.6.155.2 Diagram



9.4.3.6.155.3 Fields

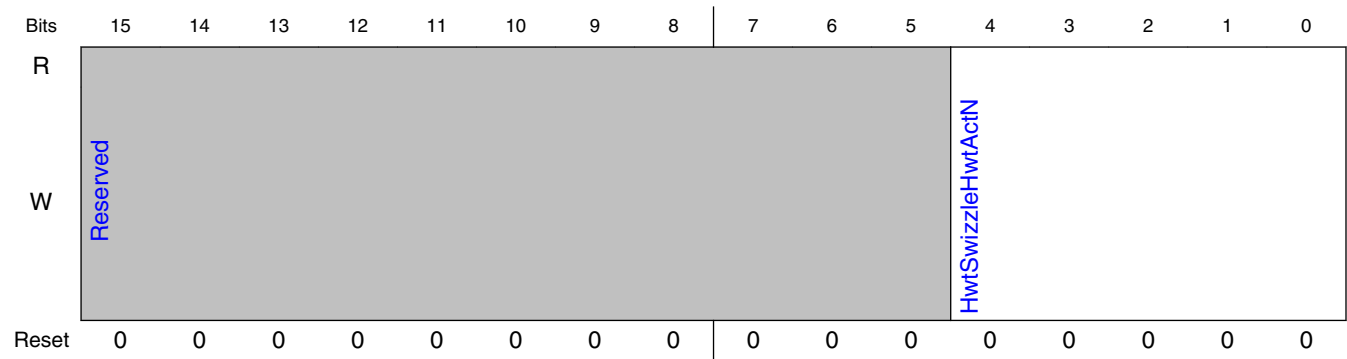
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtAddress17	See Description of HwtSwizzleHwtAddress0 for details. See Description of HwtSwizzleHwtAddress0 for details. Used only for DDR4.

9.4.3.6.156 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtActN)

9.4.3.6.156.1 Offset

Register	Offset
HwtSwizzleHwtActN	262h

9.4.3.6.156.2 Diagram



9.4.3.6.156.3 Fields

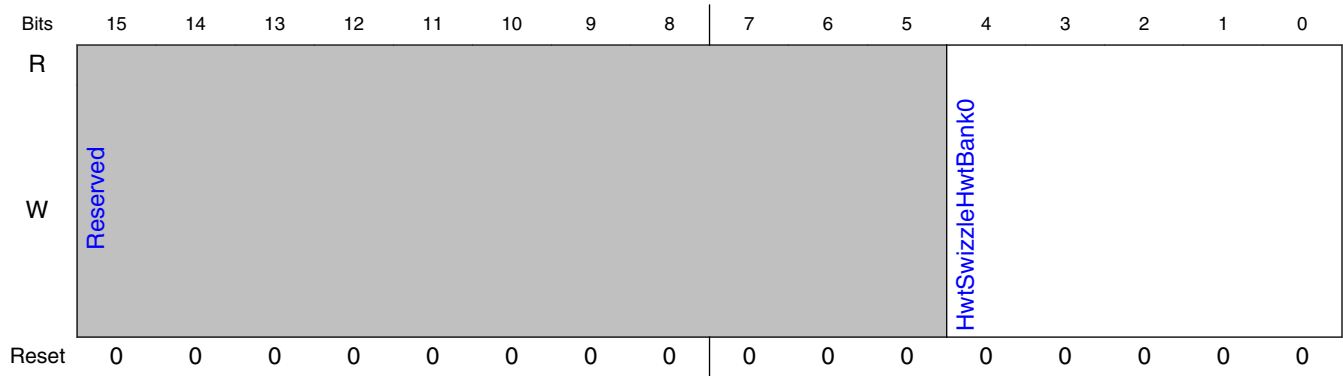
Field	Function
15-5	Reserved
—	
4-0	See Description of HwtSwizzleHwtAddress0 for details.
HwtSwizzleHwtActN	See Description of HwtSwizzleHwtAddress0 for details. Used only for DDR4.

9.4.3.6.157 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtBank0)

9.4.3.6.157.1 Offset

Register	Offset
HwtSwizzleHwtBank0	264h

9.4.3.6.157.2 Diagram



9.4.3.6.157.3 Fields

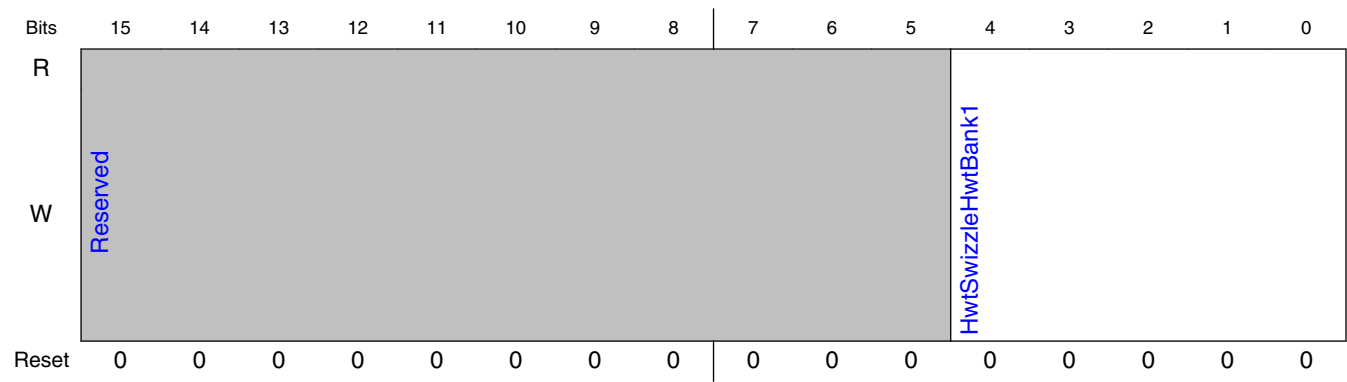
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtBank0	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.158 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtBank1)

9.4.3.6.158.1 Offset

Register	Offset
HwtSwizzleHwtBank1	266h

9.4.3.6.158.2 Diagram



9.4.3.6.158.3 Fields

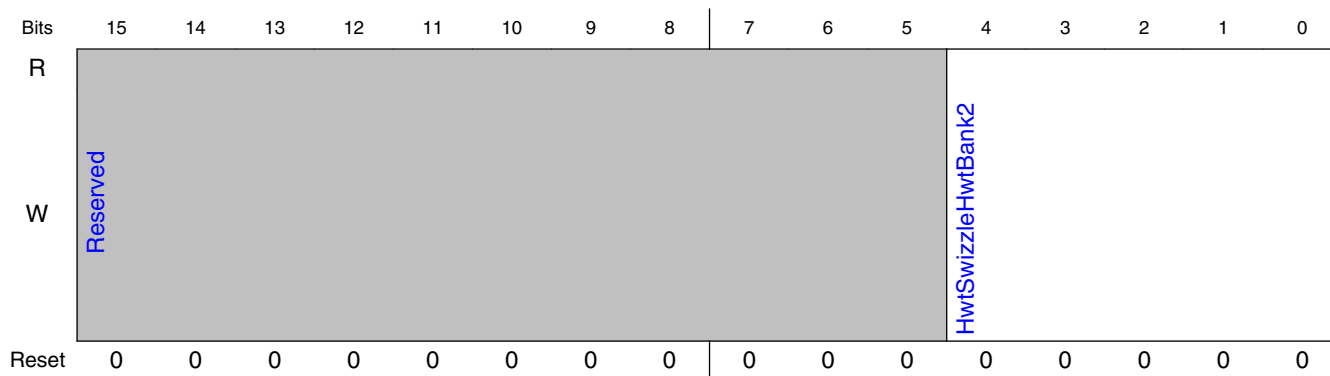
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtBank1	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.159 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtBank2)

9.4.3.6.159.1 Offset

Register	Offset
HwtSwizzleHwtBank2	268h

9.4.3.6.159.2 Diagram



9.4.3.6.159.3 Fields

Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtBank2	See Description of HwtSwizzleHwtAddress0 for details. See Description of HwtSwizzleHwtAddress0 for details. Used only for DDR3.

9.4.3.6.160 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtBg0)

9.4.3.6.160.1 Offset

Register	Offset
HwtSwizzleHwtBg0	26Ah

9.4.3.6.160.2 Diagram



9.4.3.6.160.3 Fields

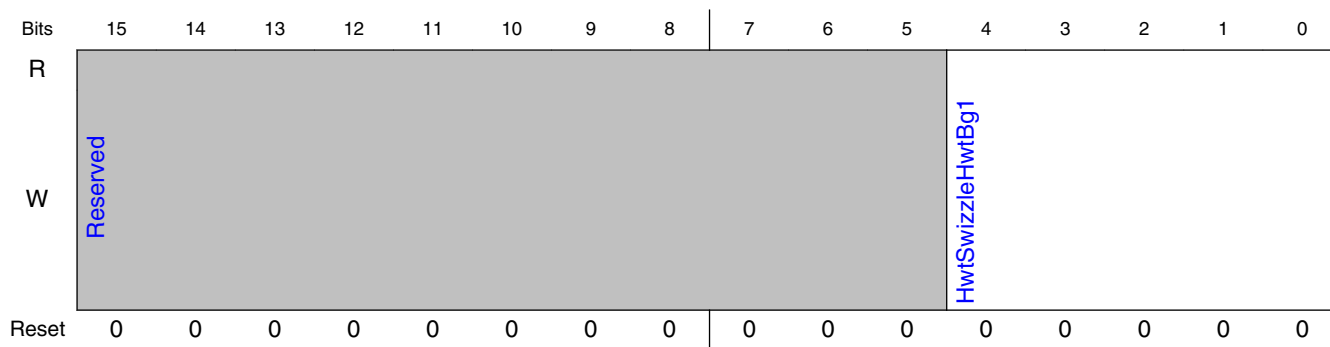
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtBg0	See Description of HwtSwizzleHwtAddress0 for details. See Description of HwtSwizzleHwtAddress0 for details. Used only for DDR4.

9.4.3.6.161 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtBg1)

9.4.3.6.161.1 Offset

Register	Offset
HwtSwizzleHwtBg1	26Ch

9.4.3.6.161.2 Diagram



9.4.3.6.161.3 Fields

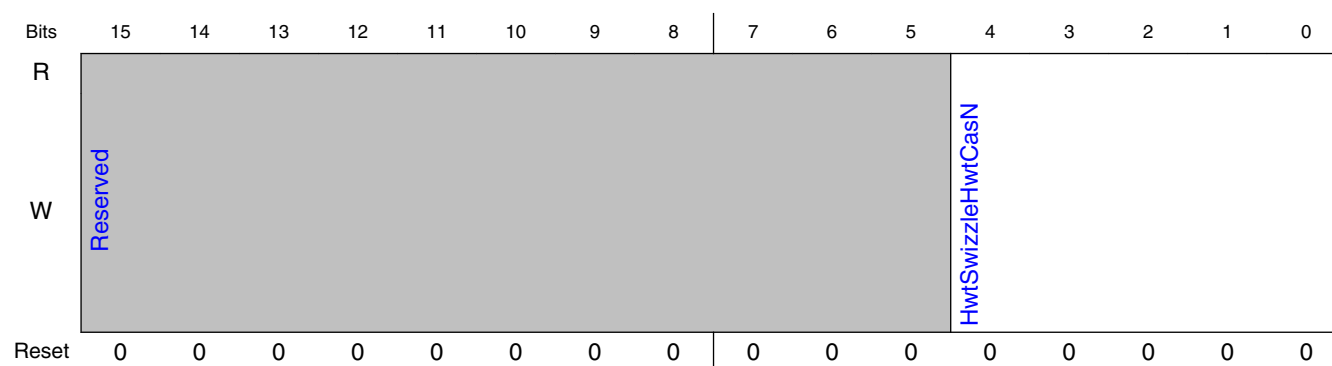
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtBg1	See Description of HwtSwizzleHwtAddress0 for details. See Description of HwtSwizzleHwtAddress0 for details. Used only for DDR4.

9.4.3.6.162 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtCasN)

9.4.3.6.162.1 Offset

Register	Offset
HwtSwizzleHwtCasN	26Eh

9.4.3.6.162.2 Diagram



9.4.3.6.162.3 Fields

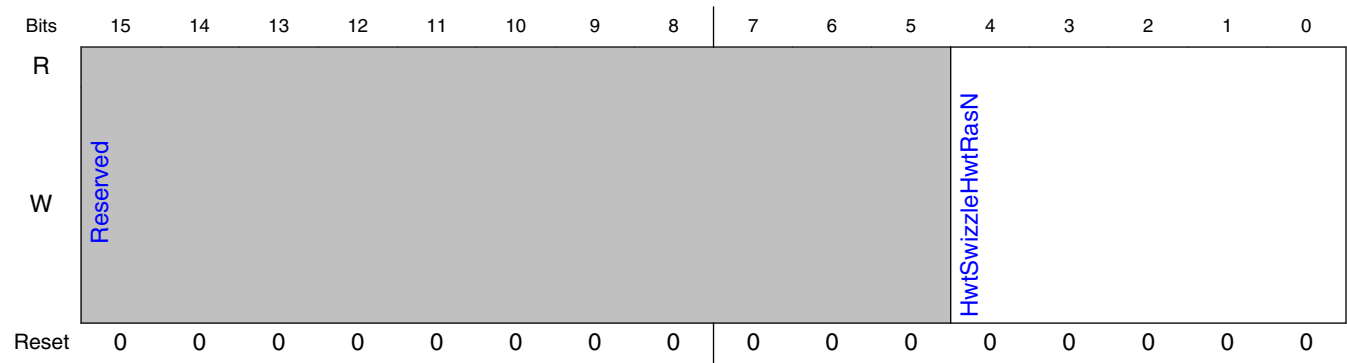
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtCasN	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.163 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtRasN)

9.4.3.6.163.1 Offset

Register	Offset
HwtSwizzleHwtRasN	270h

9.4.3.6.163.2 Diagram



9.4.3.6.163.3 Fields

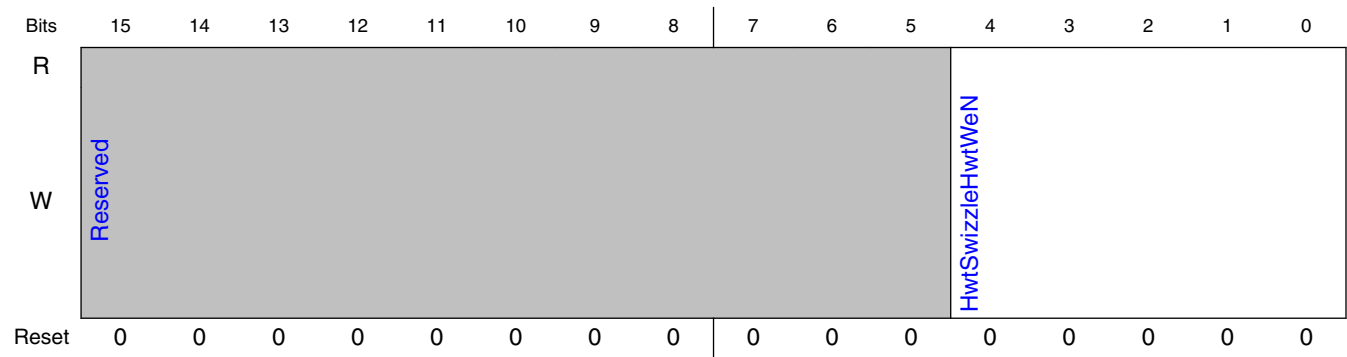
Field	Function
15-5	Reserved
—	
4-0 HwtSwizzleHwtRasN	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.164 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtWeN)

9.4.3.6.164.1 Offset

Register	Offset
HwtSwizzleHwtWeN	272h

9.4.3.6.164.2 Diagram



9.4.3.6.164.3 Fields

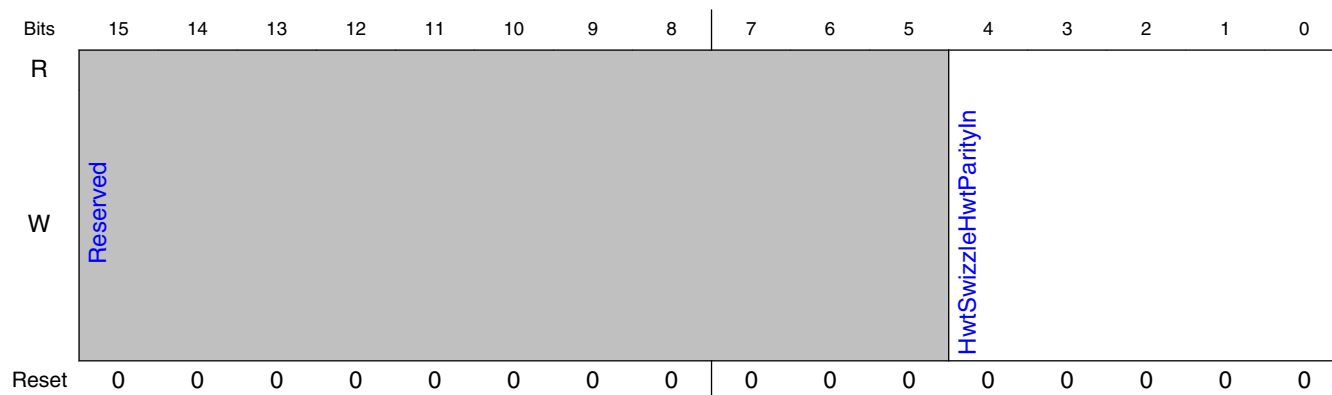
Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtWeN	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.165 Signal swizzle selection for HWT swizzle (HwtSwizzleHwtParityIn)

9.4.3.6.165.1 Offset

Register	Offset
HwtSwizzleHwtParityIn	274h

9.4.3.6.165.2 Diagram



9.4.3.6.165.3 Fields

Field	Function
15-5 —	Reserved
4-0 HwtSwizzleHwtParityIn	See Description of HwtSwizzleHwtAddress0 for details.

9.4.3.6.166 Impedance Calibration Clock Ratio (CalUclkInfo_p1)

9.4.3.6.166.1 Offset

Register	Offset
CalUclkInfo_p1	20_0010h

9.4.3.6.166.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved							CalUclkTicksPer1uS								
W																
Reset	0	0	0	0	0	0	1	1	0	0	1	0	0	0	0	0

9.4.3.6.166.3 Fields

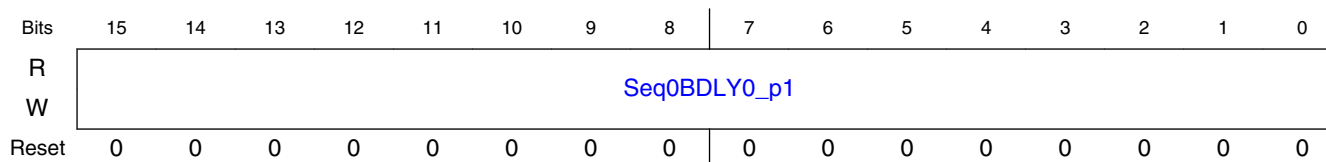
Field	Function
15-10 —	Reserved
9-0 CalUclkTicksPer1uS	Must be programmed to the number of DfiClks in 1us (rounded up), with minimum value of 24. Must be programmed to the number of DfiClks in 1us (rounded up), with minimum value of 24. if (DfiClk < 24MHz) CalUclkInfo = 24 else CalUclkInfo = (number of DfiClks in 1us)

9.4.3.6.167 PHY Initialization Engine (PIE) Delay Register 0 (Seq0BDLY0_p1)

9.4.3.6.167.1 Offset

Register	Offset
Seq0BDLY0_p1	20_0016h

9.4.3.6.167.2 Diagram



9.4.3.6.167.3 Fields

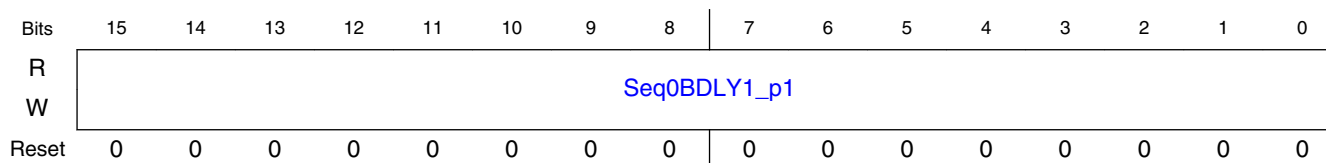
Field	Function
15-0 Seq0BDLY0_p1	PHY Initialization Engine (PIE) Delay Register 0 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 0 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.168 PHY Initialization Engine (PIE) Delay Register 1 (Seq0BDLY1_p1)

9.4.3.6.168.1 Offset

Register	Offset
Seq0BDLY1_p1	20_0018h

9.4.3.6.168.2 Diagram



9.4.3.6.168.3 Fields

Field	Function
15-0 Seq0BDLY1_p1	PHY Initialization Engine (PIE) Delay Register 1 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 1 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.169 PHY Initialization Engine (PIE) Delay Register 2 (Seq0BDLY2_p1)

9.4.3.6.169.1 Offset

Register	Offset
Seq0BDLY2_p1	20_001Ah

9.4.3.6.169.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Seq0BDLY2_p1															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.4.3.6.169.3 Fields

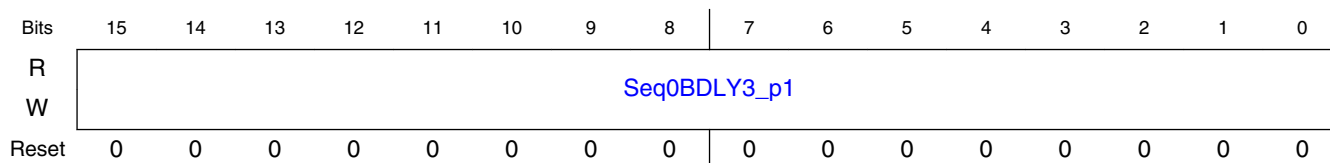
Field	Function
15-0 Seq0BDLY2_p1	PHY Initialization Engine (PIE) Delay Register 2 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 2 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.170 PHY Initialization Engine (PIE) Delay Register 3 (Seq0BDLY3_p1)

9.4.3.6.170.1 Offset

Register	Offset
Seq0BDLY3_p1	20_001Ch

9.4.3.6.170.2 Diagram



9.4.3.6.170.3 Fields

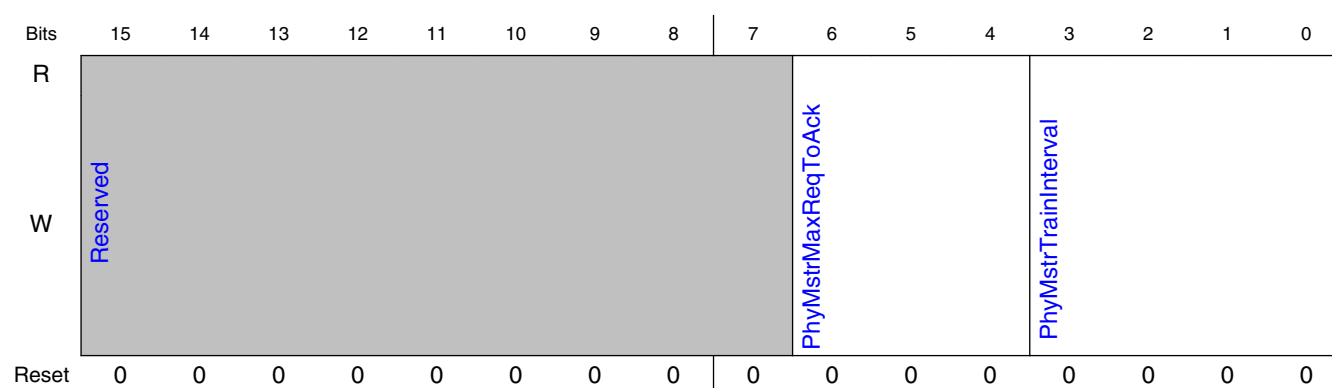
Field	Function
15-0 Seq0BDLY3_p1	PHY Initialization Engine (PIE) Delay Register 3 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 3 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfIClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.171 Setup Intervals for DFI PHY Master operations (PPTTrainSetup_p1)

9.4.3.6.171.1 Offset

Register	Offset
PPTTrainSetup_p1	20_0020h

9.4.3.6.171.2 Diagram



9.4.3.6.171.3 Fields

Field	Function
15-7 —	Reserved
6-4 PhyMstrMaxReqToAck	Bits 6:4 of this register specify the max time from tdfi_phymstr_req asserted to tdfi_phymstr_ack asserted. Bits 6:4 of this register specify the max time from tdfi_phymstr_req asserted to tdfi_phymstr_ack asserted. 3'b000 Disable PHY Master Interface 3'b001 set tPHYMSTR_resp. = 512 MEMCLKs 3'b010 set tPHYMSTR_resp. = 1024 MEMCLKs 3'b011 set tPHYMSTR_resp. = 2048 MEMCLKs 3'b100 set tPHYMSTR_resp. = 4096 MEMCLKs 3'b101 set tPHYMSTR_resp. = 8192 MEMCLKs 3'b110 set tPHYMSTR_resp. = 32768 MEMCLKs

Table continues on the next page...

DDR PHY (DDR_PHY)

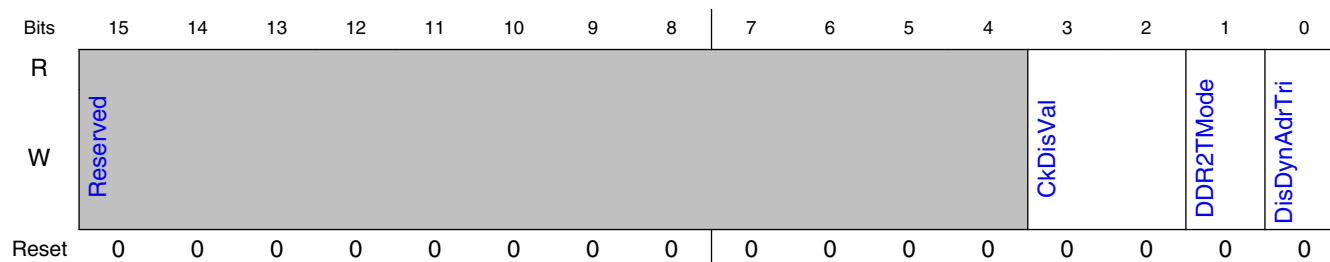
Field	Function
	3'b111 set tPHYMSTR_resp. = undefined
3-0 PhyMstrTrainInterval	Bits 3:0 of this register specifies the time between the end of one training and the start of the next. Bits 3:0 of this register specifies the time between the end of one training and the start of the next. (As per section 11 of the Preliminary DFI 4.0 Specification, V2, it is the max expected time from dfi_init_complete asserted to tdfi_phymstr_ack asserted). 4'b0000 Disable PHY Master Interface 4'b0001 PHY MASTER Request Interval = 524288 MEMCLKs 4'b0010 PHY MASTER Request Interval = 1048576 MEMCLKs 4'b0011 PHY MASTER Request Interval = 2097152 MEMCLKs 4'b0100 PHY MASTER Request Interval = 4194304 MEMCLKs 4'b0101 PHY MASTER Request Interval = 8388608 MEMCLKs 4'b0110 PHY MASTER Request Interval = 16777216 MEMCLKs 4'b0111 PHY MASTER Request Interval = 33554432 MEMCLKs 4'b1000 PHY MASTER Request Interval = 67108864 MEMCLKs 4'b1001 PHY MASTER Request Interval = 134217728 MEMCLKs 4'b1010 PHY MASTER Request Interval = 268435456 MEMCLKs 4'b1011 - 4'b1111 PHY MASTER Request Interval = undefined

9.4.3.6.172 Mode select register for MEMCLK/Address/Command Tristates (TristateModeCA_p1)

9.4.3.6.172.1 Offset

Register	Offset
TristateModeCA_p1	20_0032h

9.4.3.6.172.2 Diagram



9.4.3.6.172.3 Fields

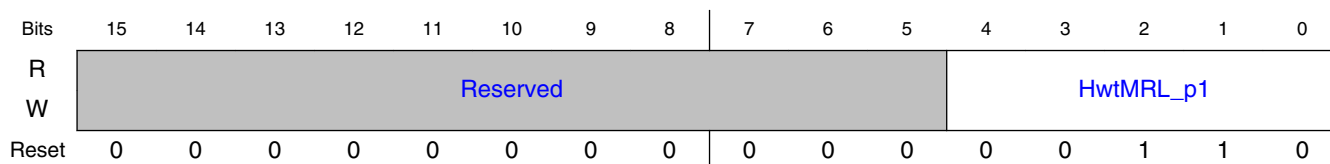
Field	Function
15-4 —	Reserved
3-2 CkDisVal	The PHY provides 4 memory clocks, n=0. The PHY provides 4 memory clocks, n=0..3. When the toggling of memory clock CK_t[n] is disabled with dfi_clk_disable[n]=1, the memory clock CK_t[n] is driven with Register CkDisVal[1]. When the toggling of memory clock CK_c[n] is disabled with dfi_clk_disable[n]=1, the memory clock CK_c[n] is driven with ~(Register CkDisVal[0]) (ie inverted). Note that the non-toggling differential memory clock CK_t[n],CK_c[n] may be driven with any combination, LL,LH,HL,HH; the default CK_t[n],CK_c[n]=LH.
1 DDR2TMode	Must be set to 1 for Dynamic Tristate to work when CA bus is 2T or Geardown mode. Must be set to 1 for Dynamic Tristate to work when CA bus is 2T or Geardown mode. This CSR bit has no effect when DisDynAdrTri==1
0 DisDynAdrTri	When DisDynAdrTri=1, Dynamic Tristating is disabled. When DisDynAdrTri=1, Dynamic Tristating is disabled. Dynamic Tristating is on by default. . Dynamic Tristating should be disabled (DisDynAdrTri=0) for these modes: 1. DDR3/2T if the controller cannot follow the 2T PHY tristate protocol. 2. DDR4/2T/2N if the controller cannot follow the 2T PHY tristate protocol. 3. LPDDR4 . When DisDynAdrTri=0 (default), In DDR3 mode, The following SDRAM pins can be dynamically tristated: A,BA,RAS_n,CAS_n,WE_n In DDR4 mode, The following SDRAM pins can be dynamically tristated: A,BA,BG,ACT_n In LPDDR3 mode, The following SDRAM pins can be dynamically tristated: CA[*] . In 1T mode, the PHY will tristate the relevant pins when all ranks are DESelected . In 2T mode (or geardown), the PHY will tristate the relevant pins when: D3 -- All ranks DESelected (all CS_L==1) AND RAS_n,CAS_n,WE_n,BA0 = 1,1,1,0 D4 -- All ranks DESelected (all CS_L==1) AND RAS_n,CAS_n,WE_n,BA0,ACT_n = 1,1,1,0,1 When in this mode, the controller should avoid sending the above patterns with any rank selected. [e.g. NOPs sent by the controller should have BA0 set to a non-zero value]

9.4.3.6.173 HWT MaxReadLatency. (HwtMRL_p1)

9.4.3.6.173.1 Offset

Register	Offset
HwtMRL_p1	20_0040h

9.4.3.6.173.2 Diagram



9.4.3.6.173.3 Fields

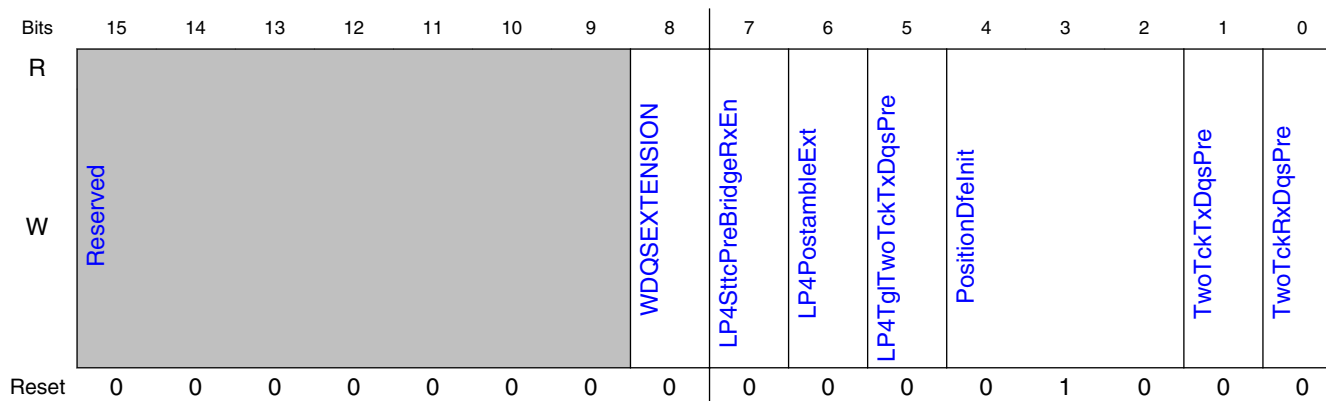
Field	Function
15-5 —	Reserved
4-0 HwtMRL_p1	This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. The CSR is in units of two mem clocks; that is, a unit change in the LSB is a change in MRL of two mem clocks. This MASTER copy of MRL is used by the PHY training hardware only.

9.4.3.6.174 Control the PHY logic related to the read and write DQS preamble (DqsPreambleControl_p1)

9.4.3.6.174.1 Offset

Register	Offset
DqsPreambleControl_p1	20_0048h

9.4.3.6.174.2 Diagram



9.4.3.6.174.3 Fields

Field	Function
15-9 —	Reserved
8 WDQSEXTENSION	When set, DQS_T and DQS_C will be driven differentially to 0 and 1, respectively, before and after a write burst, except during a memory read transaction. When set, DQS_T and DQS_C will be driven differentially to 0 and 1, respectively, before and after a write burst, except during a memory read transaction. See DesignWare Cores LPDDR4 MultiPHY: WDQS Extension Application Note. Use of WDQSEXTENSION requires POdtTailWidth=3 and POdtStartDelay=00
7 LP4SttcPreBridgeRxEn	Used in LPDDR4 static-preamble mode to bridge the RxEn between two reads to the same timing group when the bubble is 1 memclk. Used in LPDDR4 static-preamble mode to bridge the RxEn between two reads to the same timing group when the bubble is 1 memclk. LPDDR4 static-preamble mode is set in the DRAMs with MR1, OP[3]=0.
6 LP4PostambleExt	In LPDDR4 mode must be set to extend the write postamble. In LPDDR4 mode must be set to extend the write postamble. 0: half-memclk postamble, as in D4 1: one-and-one-half-memclk postamble; see LPDDR4 Spec MR3, OP[1] WR PST, vendor-specific function.
5 LP4TglTwoTckTxDqsPre	Used in LPDDR4 mode to modify the early preamble when Register TwoTckTxDqsPre=1 0: level first-memclk preamble 1: toggling first-memclk preamble
4-2 PositionDfelnit	For DDR4 phy only when receive DFE is enabled. For DDR4 phy only when receive DFE is enabled. PositionDfelnit[2]=1 causes Dfelnit to be asserted late PositionDfelnit[1]=1 causes Dfelnit to be asserted at the nominal time

Table continues on the next page...

DDR PHY (DDR_PHY)

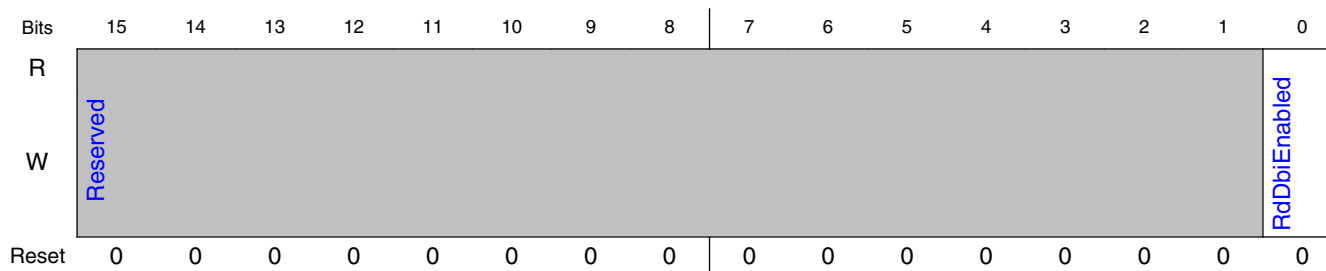
Field	Function
	PositionDfelnit[0]=1 causes Dfelnit to be asserted early PositionDfelnit=3'b010 is the nominal, recommended value for leading edge used by receiver.
1 TwoTckTxDqsPre	0: Standard 1tck TxDqs Preamble 1: Enable Optional D4 2tck TxDqs Preamble The DDR4 MR4 A12 is Write Preamble, 1=2nCK, 0=1nCK.
0 TwoTckRxDqsPre	Widens the RxDqsEn window to allow larger drift in the incoming read DQS to take advantage of the larger/wider preamble generated by the DRAMs when the D4 DRAMs are configured with DDR4 MR4 A11 Read Preamble=1 for causing a 2nCK read preamble. Widens the RxDqsEn window to allow larger drift in the incoming read DQS to take advantage of the larger/wider preamble generated by the DRAMs when the D4 DRAMs are configured with DDR4 MR4 A11 Read Preamble=1 for causing a 2nCK read preamble. For LPDDR4, all read operations are 2nCK such that this control must be set to 1. Note the nominal trained-to-center point will be earlier relative to the first strobing edge of DQS than when not in this mode.

9.4.3.6.175 This Register is used to enable the Read-DBI function in each DBYTE (DMIPinPresent_p1)

9.4.3.6.175.1 Offset

Register	Offset
DMIPinPresent_p1	20_005Ah

9.4.3.6.175.2 Diagram



9.4.3.6.175.3 Fields

Field	Function
15-1	Reserved

Table continues on the next page...

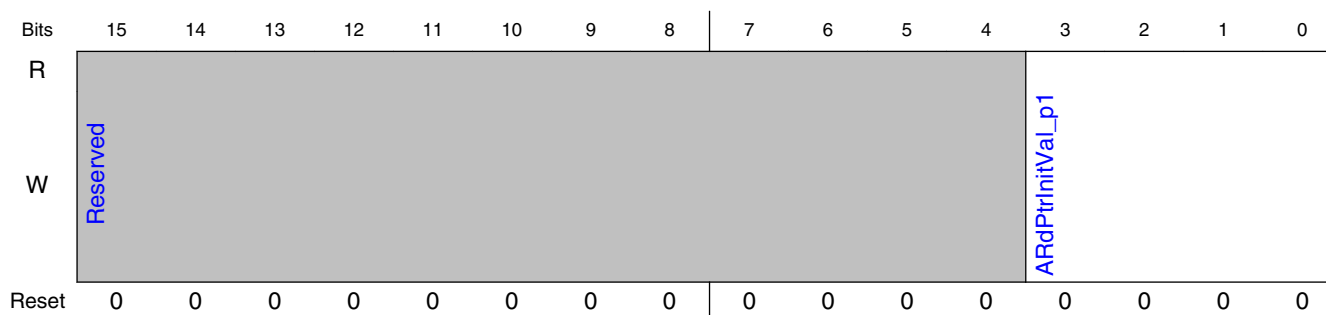
Field	Function
—	
0	This bit must be set to 1'b1 if Read-DBI is enabled in a connected DDR4 or LPDDR4 device.
RdDbiEnabled	This bit must be set to 1'b1 if Read-DBI is enabled in a connected DDR4 or LPDDR4 device. If set, the following DRAM MR should also be set [DDR4.MR5.A12=1 or LPDDR4.MR3.OP[7]=1]

9.4.3.6.176 Address/Command FIFO ReadPointer Initial Value (ARdPtrInitVal_p1)

9.4.3.6.176.1 Offset

Register	Offset
ARdPtrInitVal_p1	20_005Ch

9.4.3.6.176.2 Diagram



9.4.3.6.176.3 Fields

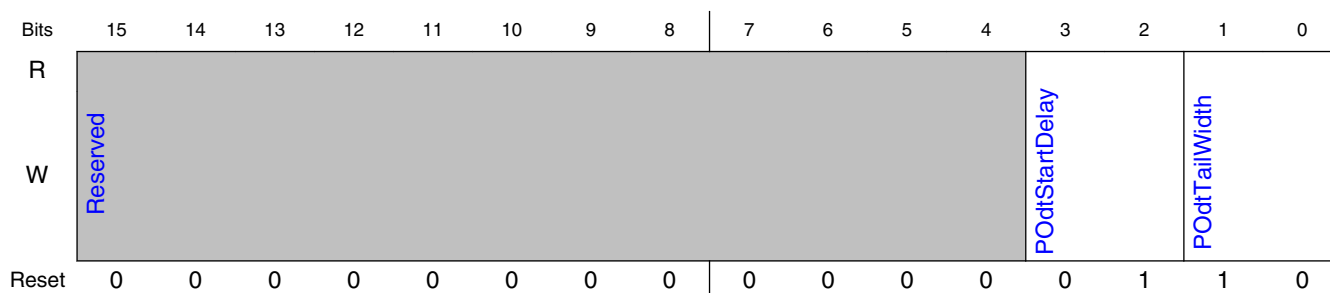
Field	Function
15-4	Reserved
—	
3-0	This is the initial Pointer Offset for the free-running FIFOs in the DBYTE and ACX4 hardips.
ARdPtrInitVal_p1	This is the initial Pointer Offset for the free-running FIFOs in the DBYTE and ACX4 hardips. The units of this register are in UI. The pointer separation should be chosen to compensate for all sources of skew and drift of the PHY DFICLK and PCLK networks. Please see PUB databook section 8.1.1 This CSR must be programmed in Step C of the PHY Initialization sequence

9.4.3.6.177 READ DATA On-Die Termination Timing Control (by PHY) (ProcOdtTimeCtl_p1)

9.4.3.6.177.1 Offset

Register	Offset
ProcOdtTimeCtl_p1	20_00ACh

9.4.3.6.177.2 Diagram



9.4.3.6.177.3 Fields

Field	Function
15-4 —	Reserved
3-2 POdtStartDelay	controls the start of ProcOdt, units of UI 3 delay start 2 UI, maximum delay of start of ProcOdt 2 delay start 1 UI, 1 delay start 0 UI, default 0 early by 1 UI, The time from ProcODT assertion to opening the window to receive DQS is (10 - POdtStartDelay) UI.
1-0 POdtTailWidth	controls the length of the tail of ProcOdt, units of UI 3 tail 3UI more than for Register POdtTailWidth=0, maximum 2 tail 2UI more than for Register POdtTailWidth=0, default 1 tail 1UI more than for Register POdtTailWidth=0 0 minimum length tail The time from ProcODT to closing the window to receive DQS to ProcODT POdtTailWidth is (2 + POdtTailWidth) UI

9.4.3.6.178 DLL gain control (DllGainCtl_p1)

9.4.3.6.178.1 Offset

Register	Offset
DllGainCtl_p1	20_00F8h

9.4.3.6.178.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				DIISeedSel				DIIGainTV				DIIGainIV			
W																
Reset	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	1

9.4.3.6.178.3 Fields

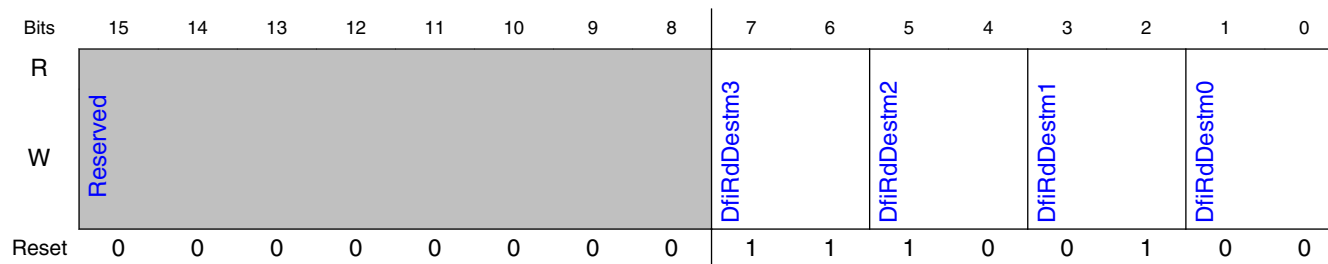
Field	Function
15-12 —	Reserved
11-8 DIISeedSel	Reserved, must be configured to be 0.
7-4 DIIGainTV	Terminal value of DIIGain, ie the value in effect when locking is done and the value used for maintaining lock, ie tracking pclk variation. Terminal value of DIIGain, ie the value in effect when locking is done and the value used for maintaining lock, ie tracking pclk variation. DIIGainTV must be greater than or equal to DIIGainIV. The maximum value is 10. The minimum value is 6.
3-0 DIIGainIV	Initial value of DIIGain.

9.4.3.6.179 Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiRdDataCsDestMap_p1)

9.4.3.6.179.1 Offset

Register	Offset
DfiRdDataCsDestMap_p1	20_0160h

9.4.3.6.179.2 Diagram



9.4.3.6.179.3 Fields

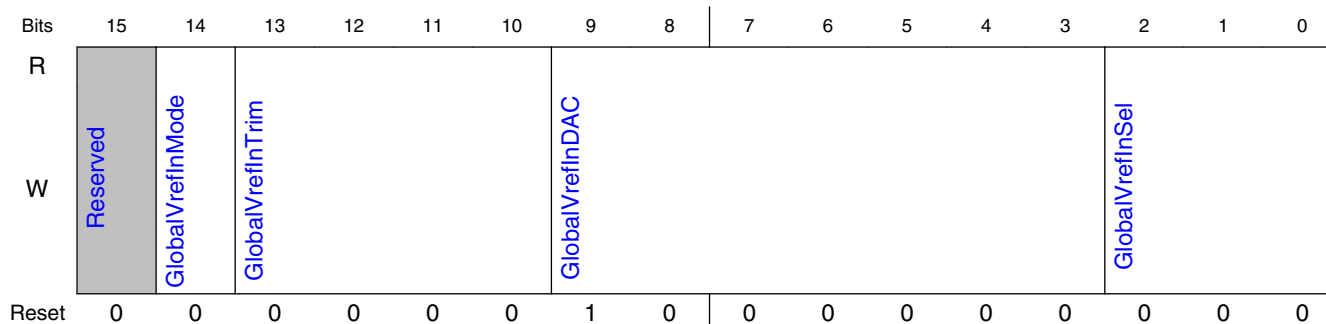
Field	Function
15-8 —	Reserved
7-6 DfiRdDestm3	Maps dfi_rddata_cs_n_p0[3] to dest DfiRdDestm3 timing For example, if 3 dfi_rddata_cs_n_p0[3] will use Register RxEn,ClkDlyTg3 timing.
5-4 DfiRdDestm2	Maps dfi_rddata_cs_n_p0[2] to dest DfiRdDestm2 timing For example, if 2 dfi_rddata_cs_n_p0[2] will use Register RxEn,ClkDlyTg2 timing.
3-2 DfiRdDestm1	Maps dfi_rddata_cs_n_p0[1] to dest DfiRdDestm1 timing For example, if 1 dfi_rddata_cs_n_p0[1] will use Register RxEn,ClkDlyTg1 timing.
1-0 DfiRdDestm0	Maps dfi_rddata_cs_n_p0[0] to dest DfiRdDestm0 timing For example, if 0 dfi_rddata_cs_n_p0[0] will use Register RxEn,ClkDlyTg0 timing.

9.4.3.6.180 PHY Global Vref Controls (VreflnGlobal_p1)

9.4.3.6.180.1 Offset

Register	Offset
VreflnGlobal_p1	20_0164h

9.4.3.6.180.2 Diagram



9.4.3.6.180.3 Fields

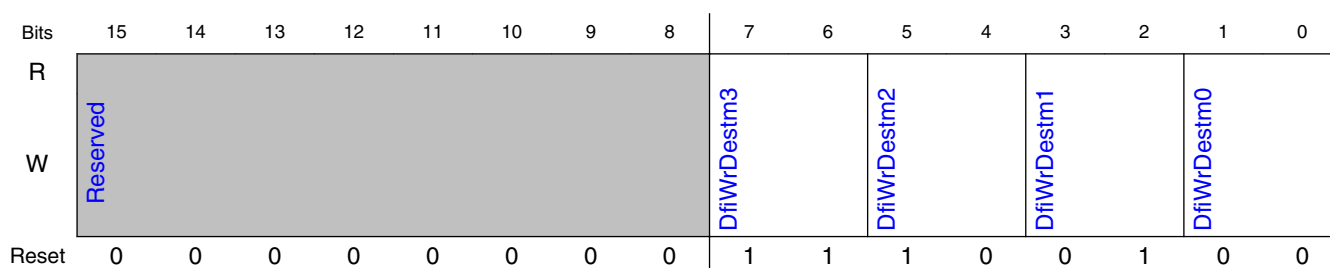
Field	Function
15 —	Reserved
14 GlobalVrefInMode	RSVD
13-10 GlobalVrefInTrim	RSVD
9-3 GlobalVrefInDAC	DAC code for internal Vref generation The DAC has two ranges; the range is set by GlobalVrefInSel[2] ===== RANGE0 : DDR3,DDR4,LPDDR3 [GlobalVrefInSel[2] = 0] DAC Output Voltage = GlobalVrefInDAC == 6'h00 ? Hi-Z : 0. DAC code for internal Vref generation The DAC has two ranges; the range is set by GlobalVrefInSel[2] ===== RANGE0 : DDR3,DDR4,LPDDR3 [GlobalVrefInSel[2] = 0] DAC Output Voltage = GlobalVrefInDAC == 6'h00 ? Hi-Z : 0.345*VDDQ + (0.005*GlobalVrefInDAC)*VDDQ ===== RANGE1 : LPDDR4 [GlobalVrefInSel[2] = 1] DAC Output Voltage = GlobalVrefInDAC == 6'h00 ? Hi-Z : (0.005*(GlobalVrefInDAC-1))*VDDQ
2-0 GlobalVrefInSel	GlobalVrefInSel[1:0] controls the mode of the PHY VREF DAC and the BP_VREF pin ===== 2'b00 - PHY Vref DAC Range0 -- BP_VREF = Hi-Z 2'b01 - Reserved Encoding 2'b10 - PHY Vref DAC Range0 -- BP_VREF connected to PLL Analog Bus 2'b11 - PHY Vref DAC Range0 -- BP_VREF connected to PHY Vref DAC ===== GlobalVrefInSel[2] shall be set according to Dram Protocol: Protocol GlobalVrefInSel[2] ----- DDR3 1'b0 DDR4 1'b0 LPDDR3 1'b0 LPDDR4 1'b1

9.4.3.6.181 Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiWrDataCsDestMap_p1)

9.4.3.6.181.1 Offset

Register	Offset
DfiWrDataCsDestMap_p1	20_0168h

9.4.3.6.181.2 Diagram



9.4.3.6.181.3 Fields

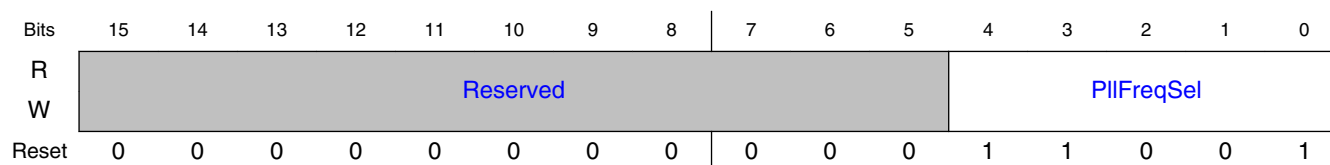
Field	Function
15-8 —	Reserved
7-6 DfiWrDestm3	Maps dfi_wrdata_cs_n_p0[3] to dest DfiWrDestm3 timing (use Register TxDq,DqsDlyTg3) For example, if 3 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg3 timing.
5-4 DfiWrDestm2	Maps dfi_wrdata_cs_n_p0[2] to dest DfiWrDestm2 timing For example, if 2 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg2 timing.
3-2 DfiWrDestm1	Maps dfi_wrdata_cs_n_p0[1] to dest DfiWrDestm1 timing For example, if 1 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg1 timing.
1-0 DfiWrDestm0	Maps dfi_wrdata_cs_n_p0[0] to dest DfiWrDestm0 timing For example, if 0 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg0 timing.

9.4.3.6.182 PState dependent PLL Control Register 2 (PlIcCtrl2_p1)

9.4.3.6.182.1 Offset

Register	Offset
PIICtrl2_p1	20_018Ah

9.4.3.6.182.2 Diagram



9.4.3.6.182.3 Fields

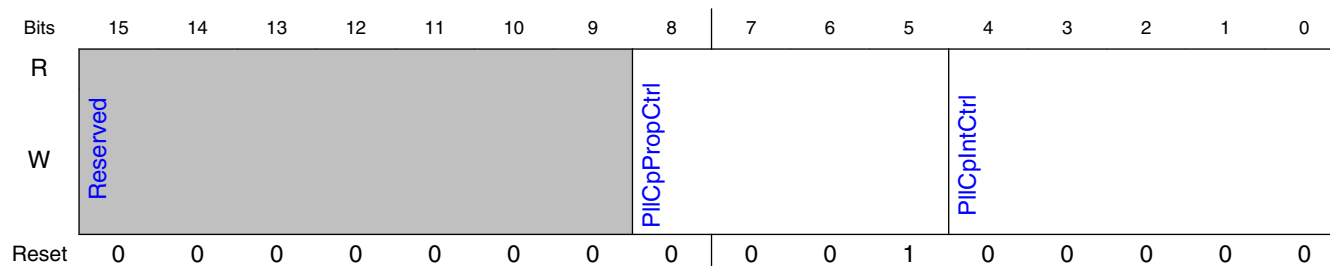
Field	Function
15-5 —	Reserved
4-0 PIIFreqSel	Adjusts the loop parameters to compensate for different VCO bias points, and input/output clock division ratios. Adjusts the loop parameters to compensate for different VCO bias points, and input/output clock division ratios. Program based on reference clock frequency (DfiClk) with this table.

9.4.3.6.183 PState dependent PLL Control Register 1 (PIICtrl1_p1)

9.4.3.6.183.1 Offset

Register	Offset
PIICtrl1_p1	20_018Eh

9.4.3.6.183.2 Diagram



9.4.3.6.183.3 Fields

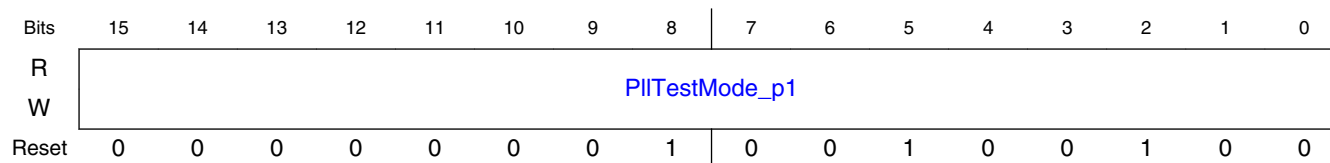
Field	Function
15-9 —	Reserved
8-5 PllCpPropCtrl	connects directly to cp_prop_cntrl<3:0> of PLL. connects directly to cp_prop_cntrl<3:0> of PLL. Charge pump proportional current control.
4-0 PllCpIntCtrl	connects directly to cp_int_cntrl<1:0> in PLL. connects directly to cp_int_cntrl<1:0> in PLL. Charge pump integrating current control.

9.4.3.6.184 Additional controls for PLL CP/VCO modes of operation (PIITestMode_p1)

9.4.3.6.184.1 Offset

Register	Offset
PIITestMode_p1	20_0194h

9.4.3.6.184.2 Diagram



9.4.3.6.184.3 Fields

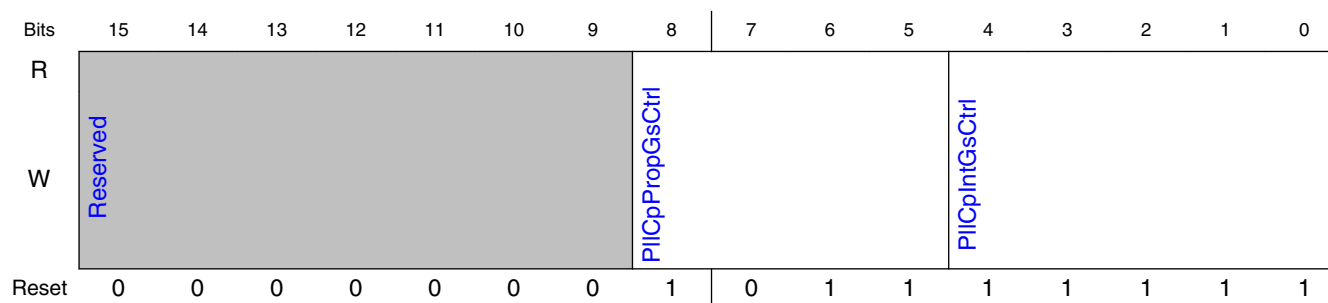
Field	Function
15-0	It is required to use default values for this CSR unless directed otherwise by Synopsys.
PIITestMode_p1	It is required to use default values for this CSR unless directed otherwise by Synopsys. Directly connected to testmode[15:0] pin of PLL

9.4.3.6.185 PState dependent PLL Control Register 4 (PIICtrl4_p1)

9.4.3.6.185.1 Offset

Register	Offset
PIICtrl4_p1	20_0198h

9.4.3.6.185.2 Diagram



9.4.3.6.185.3 Fields

Field	Function
15-9 —	Reserved
8-5 PIICpPropGsCtrl	connects directly to cp_prop_gs_cntrl<3:0> of PLL. Charge pump proportional current control for fast relock and gearshift.
4-0 PIICpIntGsCtrl	connects directly to cp_int_gs_cntrl<4:0> in PLL. Charge pump integrating current control for fast relock and gearshift.

9.4.3.6.186 DFI Frequency Ratio (DfiFreqRatio_p1)

9.4.3.6.186.1 Offset

Register	Offset
DfiFreqRatio_p1	20_01F4h

9.4.3.6.186.2 Diagram



9.4.3.6.186.3 Fields

Field	Function
15-2 —	Reserved
1-0 DfiFreqRatio_p1	Used in dwc_ddrphy_pub_serdes to serialize or de-serialize DFI signals 00 = 1:1 mode 01 = 1:2 mode 1x = 1:4 mode* *Note: 1:4 is for future pub revision.

9.4.3.6.187 Impedance Calibration Clock Ratio (CalUclkInfo_p2)

9.4.3.6.187.1 Offset

Register	Offset
CalUclkInfo_p2	40_0010h

9.4.3.6.187.2 Diagram



9.4.3.6.187.3 Fields

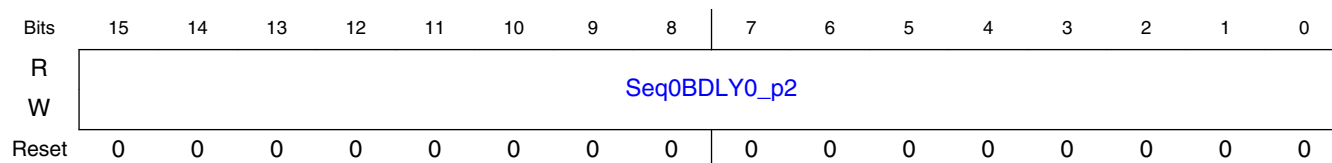
Field	Function
15-10 —	Reserved
9-0 CalUClkTicksPer1uS	Must be programmed to the number of DfIClks in 1us (rounded up), with minimum value of 24. Must be programmed to the number of DfIClks in 1us (rounded up), with minimum value of 24. if (DfIClk < 24MHz) CalUclInfo = 24 else CalUclInfo = (number of DfIClks in 1us)

9.4.3.6.188 PHY Initialization Engine (PIE) Delay Register 0 (Seq0BDLY0_p2)

9.4.3.6.188.1 Offset

Register	Offset
Seq0BDLY0_p2	40_0016h

9.4.3.6.188.2 Diagram



9.4.3.6.188.3 Fields

Field	Function
15-0 Seq0BDLY0_p2	PHY Initialization Engine (PIE) Delay Register 0 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value.

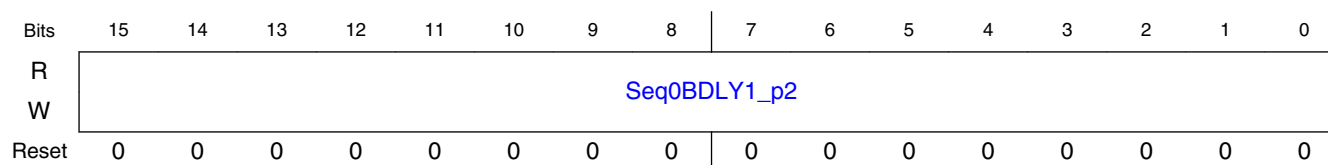
Field	Function
	PHY Initialization Engine (PIE) Delay Register 0 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.189 PHY Initialization Engine (PIE) Delay Register 1 (Seq0BDLY1_p2)

9.4.3.6.189.1 Offset

Register	Offset
Seq0BDLY1_p2	40_0018h

9.4.3.6.189.2 Diagram



9.4.3.6.189.3 Fields

Field	Function
15-0 Seq0BDLY1_p2	PHY Initialization Engine (PIE) Delay Register 1 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 1 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfiClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.190 PHY Initialization Engine (PIE) Delay Register 2 (Seq0BDLY2_p2)

9.4.3.6.190.1 Offset

Register	Offset
Seq0BDLY2_p2	40_001Ah

9.4.3.6.190.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Seq0BDLY2_p2															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.4.3.6.190.3 Fields

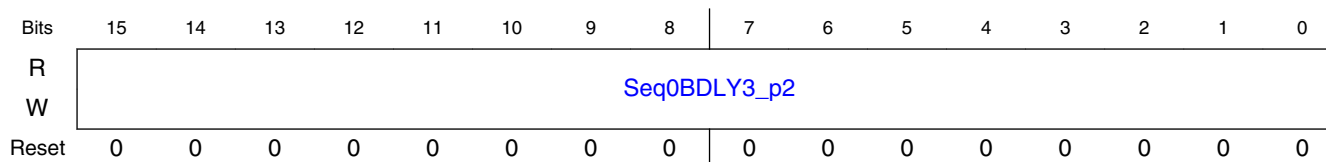
Field	Function
15-0 Seq0BDLY2_p2	PHY Initialization Engine (PIE) Delay Register 2 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 2 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.191 PHY Initialization Engine (PIE) Delay Register 3 (Seq0BDLY3_p2)

9.4.3.6.191.1 Offset

Register	Offset
Seq0BDLY3_p2	40_001Ch

9.4.3.6.191.2 Diagram



9.4.3.6.191.3 Fields

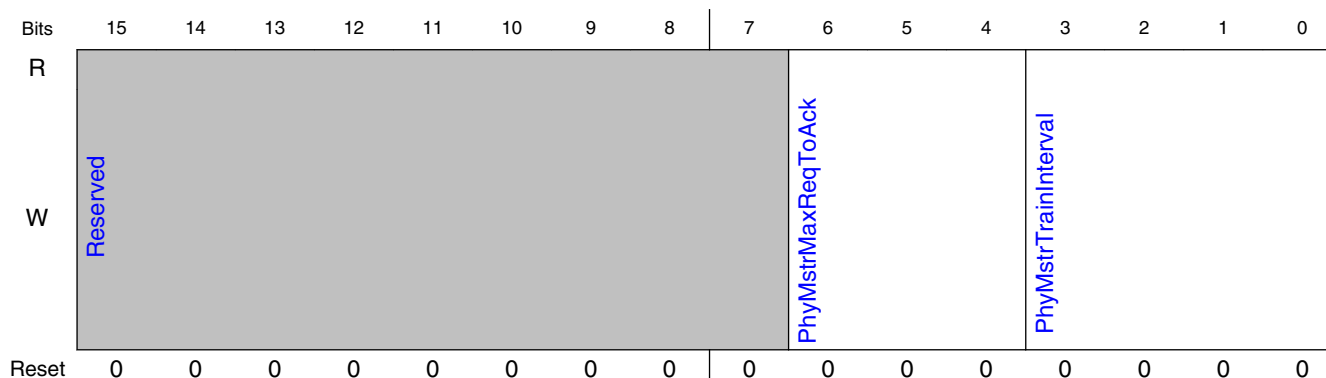
Field	Function
15-0 Seq0BDLY3_p2	PHY Initialization Engine (PIE) Delay Register 3 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 3 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfIClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.192 Setup Intervals for DFI PHY Master operations (PPTTrainSetup_p2)

9.4.3.6.192.1 Offset

Register	Offset
PPTTrainSetup_p2	40_0020h

9.4.3.6.192.2 Diagram



9.4.3.6.192.3 Fields

Field	Function
15-7 —	Reserved
6-4 PhyMstrMaxReqToAck	Bits 6:4 of this register specify the max time from tdfi_phymstr_req asserted to tdfi_phymstr_ack asserted. Bits 6:4 of this register specify the max time from tdfi_phymstr_req asserted to tdfi_phymstr_ack asserted. 3'b000 Disable PHY Master Interface 3'b001 set tPHYMSTR_resp. = 512 MEMCLKs 3'b010 set tPHYMSTR_resp. = 1024 MEMCLKs 3'b011 set tPHYMSTR_resp. = 2048 MEMCLKs 3'b100 set tPHYMSTR_resp. = 4096 MEMCLKs 3'b101 set tPHYMSTR_resp. = 8192 MEMCLKs 3'b110 set tPHYMSTR_resp. = 32768 MEMCLKs 3'b111 set tPHYMSTR_resp. = undefined
3-0 PhyMstrTrainInterval	Bits 3:0 of this register specifies the time between the end of one training and the start of the next. Bits 3:0 of this register specifies the time between the end of one training and the start of the next. (As per section 11 of the Preliminary DFI 4.0 Specification, V2, it is the max expected time from dfi_init_complete asserted to tdfi_phymstr_ack asserted). 4'b0000 Disable PHY Master Interface 4'b0001 PHY MASTER Request Interval = 524288 MEMCLKs 4'b0010 PHY MASTER Request Interval = 1048576 MEMCLKs

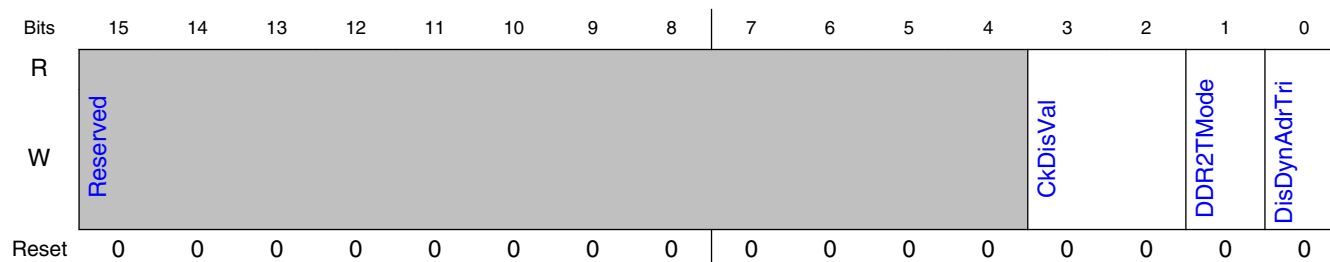
Field	Function
	4'b0011 PHY MASTER Request Interval = 2097152 MEMCLKs
	4'b0100 PHY MASTER Request Interval = 4194304 MEMCLKs
	4'b0101 PHY MASTER Request Interval = 8388608 MEMCLKs
	4'b0110 PHY MASTER Request Interval = 16777216 MEMCLKs
	4'b0111 PHY MASTER Request Interval = 33554432 MEMCLKs
	4'b1000 PHY MASTER Request Interval = 67108864 MEMCLKs
	4'b1001 PHY MASTER Request Interval = 134217728 MEMCLKs
	4'b1010 PHY MASTER Request Interval = 268435456 MEMCLKs
	4'b1011 - 4'b1111 PHY MASTER Request Interval = undefined

9.4.3.6.193 Mode select register for MEMCLK/Address/Command Tristates (TristateModeCA_p2)

9.4.3.6.193.1 Offset

Register	Offset
TristateModeCA_p2	40_0032h

9.4.3.6.193.2 Diagram



9.4.3.6.193.3 Fields

Field	Function
15-4 —	Reserved
3-2 CkDisVal	The PHY provides 4 memory clocks, n=0. The PHY provides 4 memory clocks, n=0..3. When the toggling of memory clock CK_t[n] is disabled with dfi_clk_disable[n]=1,

Table continues on the next page...

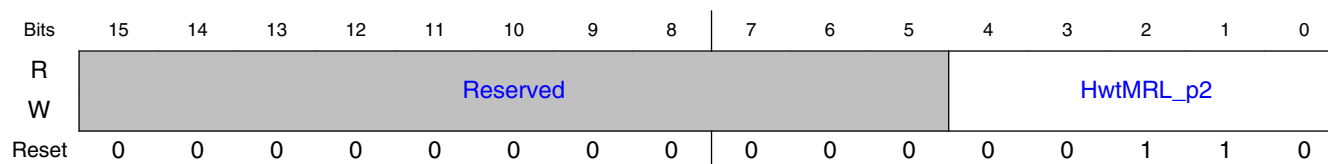
Field	Function
	the memory clock CK_t[n] is driven with Register CkDisVal[1]. When the toggling of memory clock CK_c[n] is disabled with dfi_clk_disable[n]=1, the memory clock CK_c[n] is driven with ~(Register CkDisVal[0]) (ie inverted). Note that the non-toggling differential memory clock CK_t[n],CK_c[n] may be driven with any combination, LL,LH,HL,HH; the default CK_t[n],CK_c[n]=LH.
1 DDR2TMode	Must be set to 1 for Dynamic Tristate to work when CA bus is 2T or Geardown mode. Must be set to 1 for Dynamic Tristate to work when CA bus is 2T or Geardown mode. This CSR bit has no effect when DisDynAdrTri==1
0 DisDynAdrTri	When DisDynAdrTri=1, Dynamic Tristating is disabled. When DisDynAdrTri=1, Dynamic Tristating is disabled. Dynamic Tristating is on by default. . Dynamic Tristating should be disabled (DisDynAdrTri=0) for these modes: 1. DDR3/2T if the controller cannot follow the 2T PHY tristate protocol. 2. DDR4/2T/2N if the controller cannot follow the 2T PHY tristate protocol. 3. LPDDR4 . When DisDynAdrTri=0 (default), In DDR3 mode, The following SDRAM pins can be dynamically tristated: A,BA,RAS_n,CAS_n,WE_n In DDR4 mode, The following SDRAM pins can be dynamically tristated: A,BA,BG,ACT_n In LPDDR3 mode, The following SDRAM pins can be dynamically tristated: CA[*] . In 1T mode, the PHY will tristate the relevant pins when all ranks are DESelected . In 2T mode (or geardown), the PHY will tristate the relevant pins when: D3 -- All ranks DESelected (all CS_L==1) AND RAS_n,CAS_n,WE_n,BA0 = 1,1,1,0 D4 -- All ranks DESelected (all CS_L==1) AND RAS_n,CAS_n,WE_n,BA0,ACT_n = 1,1,1,0,1 When in this mode, the controller should avoid sending the above patterns with any rank selected. [e.g. NOPs sent by the controller should have BA0 set to a non-zero value]

9.4.3.6.194 HWT MaxReadLatency. (HwtMRL_p2)

9.4.3.6.194.1 Offset

Register	Offset
HwtMRL_p2	40_0040h

9.4.3.6.194.2 Diagram



9.4.3.6.194.3 Fields

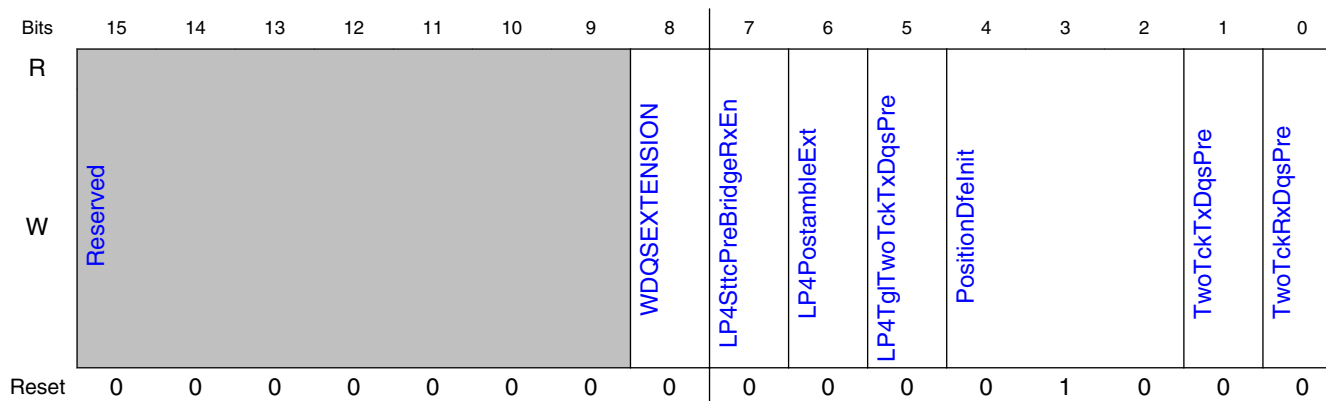
Field	Function
15-5 —	Reserved
4-0 HwtMRL_p2	This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. The CSR is in units of two mem clocks; that is, a unit change in the LSB is a change in MRL of two mem clocks. This MASTER copy of MRL is used by the PHY training hardware only.

9.4.3.6.195 Control the PHY logic related to the read and write DQS preamble (DqsPreambleControl_p2)

9.4.3.6.195.1 Offset

Register	Offset
DqsPreambleControl_p2	40_0048h

9.4.3.6.195.2 Diagram



9.4.3.6.195.3 Fields

Field	Function
15-9 —	Reserved
8 WDQSEXTENSION	When set, DQS_T and DQS_C will be driven differentially to 0 and 1, respectively, before and after a write burst, except during a memory read transaction. When set, DQS_T and DQS_C will be driven differentially to 0 and 1, respectively, before and after a write burst, except during a memory read transaction. See DesignWare Cores LPDDR4 MultiPHY: WDQS Extension Application Note. Use of WDQSEXTENSION requires POdtTailWidth=3 and POdtStartDelay=00
7 LP4SttcPreBridgeRxEn	Used in LPDDR4 static-preamble mode to bridge the RxEn between two reads to the same timing group when the bubble is 1 memclk. Used in LPDDR4 static-preamble mode to bridge the RxEn between two reads to the same timing group when the bubble is 1 memclk. LPDDR4 static-preamble mode is set in the DRAMs with MR1, OP[3]=0.
6 LP4PostambleExt	In LPDDR4 mode must be set to extend the write postamble. In LPDDR4 mode must be set to extend the write postamble. 0: half-memclk postamble, as in D4 1: one-and-one-half-memclk postamble; see LPDDR4 Spec MR3, OP[1] WR PST, vendor-specific function.
5 LP4TglTwoTckTxDqsPre	Used in LPDDR4 mode to modify the early preamble when Register TwoTckTxDqsPre=1 0: level first-memclk preamble 1: toggling first-memclk preamble
4-2 PositionDfelnit	For DDR4 phy only when receive DFE is enabled. For DDR4 phy only when receive DFE is enabled. PositionDfelnit[2]=1 causes Dfelnit to be asserted late PositionDfelnit[1]=1 causes Dfelnit to be asserted at the nominal time

Table continues on the next page...

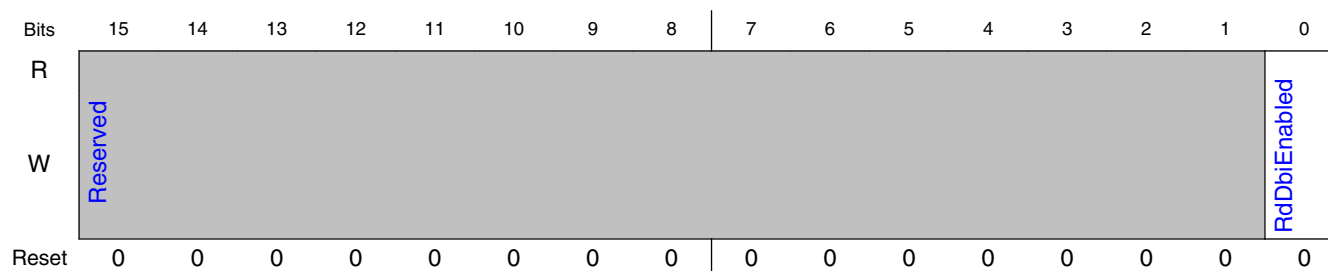
Field	Function
	PositionDfelnit[0]=1 causes Dfelnit to be asserted early PositionDfelnit=3'b010 is the nominal, recommended value for leading edge used by receiver.
1 TwoTckTxDqsPreamble	0: Standard 1tck TxDqs Preamble 1: Enable Optional D4 2tck TxDqs Preamble The DDR4 MR4 A12 is Write Preamble, 1=2nCK, 0=1nCK.
0 TwoTckRxDqsPreamble	Widens the RxDqsEn window to allow larger drift in the incoming read DQS to take advantage of the larger/wider preamble generated by the DRAMs when the D4 DRAMs are configured with DDR4 MR4 A11 Read Preamble=1 for causing a 2nCK read preamble. Widens the RxDqsEn window to allow larger drift in the incoming read DQS to take advantage of the larger/wider preamble generated by the DRAMs when the D4 DRAMs are configured with DDR4 MR4 A11 Read Preamble=1 for causing a 2nCK read preamble. For LPDDR4, all read operations are 2nCK such that this control must be set to 1. Note the nominal trained-to-center point will be earlier relative to the first strobing edge of DQS than when not in this mode.

9.4.3.6.196 This Register is used to enable the Read-DBI function in each DBYTE (DMIPinPresent_p2)

9.4.3.6.196.1 Offset

Register	Offset
DMIPinPresent_p2	40_005Ah

9.4.3.6.196.2 Diagram



9.4.3.6.196.3 Fields

Field	Function
15-1	Reserved

Table continues on the next page...

DDR PHY (DDR_PHY)

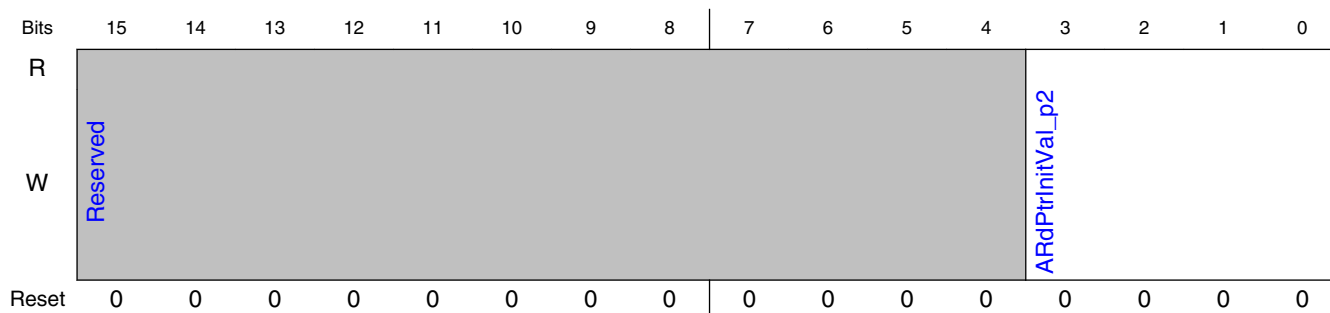
Field	Function
—	
0	This bit must be set to 1'b1 if Read-DBI is enabled in a connected DDR4 or LPDDR4 device.
RdDbiEnabled	This bit must be set to 1'b1 if Read-DBI is enabled in a connected DDR4 or LPDDR4 device. If set, the following DRAM MR should also be set [DDR4.MR5.A12=1 or LPDDR4.MR3.OP[7]=1]

9.4.3.6.197 Address/Command FIFO ReadPointer Initial Value (ARdPtrInitVal_p2)

9.4.3.6.197.1 Offset

Register	Offset
ARdPtrInitVal_p2	40_005Ch

9.4.3.6.197.2 Diagram



9.4.3.6.197.3 Fields

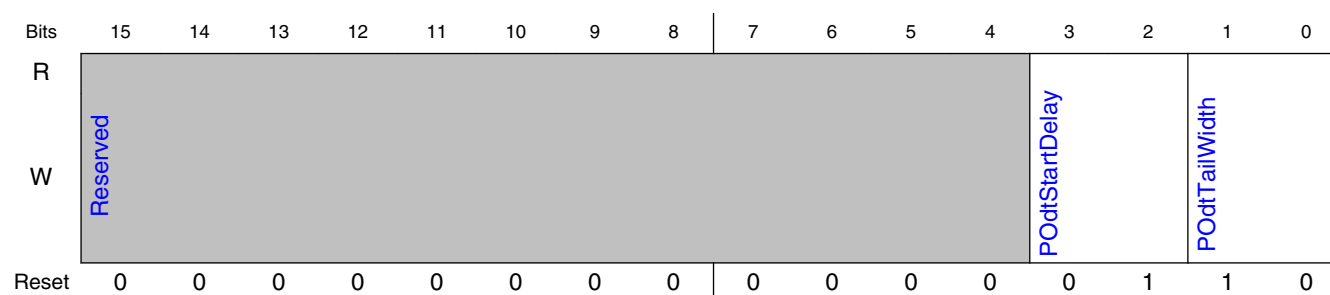
Field	Function
15-4	Reserved
—	
3-0	This is the initial Pointer Offset for the free-running FIFOs in the DBYTE and ACX4 hardips.
ARdPtrInitVal_p2	This is the initial Pointer Offset for the free-running FIFOs in the DBYTE and ACX4 hardips. The units of this register are in UI. The pointer separation should be chosen to compensate for all sources of skew and drift of the PHY DFICLK and PCLK networks. Please see PUB databook section 8.1.1 This CSR must be programmed in Step C of the PHY Initialization sequence

9.4.3.6.198 READ DATA On-Die Termination Timing Control (by PHY) (ProcOdtTimeCtl_p2)

9.4.3.6.198.1 Offset

Register	Offset
ProcOdtTimeCtl_p2	40_00ACh

9.4.3.6.198.2 Diagram



9.4.3.6.198.3 Fields

Field	Function
15-4 —	Reserved
3-2 POdtStartDelay	controls the start of ProcOdt, units of UI 3 delay start 2 UI, maximum delay of start of ProcOdt 2 delay start 1 UI, 1 delay start 0 UI, default 0 early by 1 UI, The time from ProcODT assertion to opening the window to receive DQS is (10 - POdtStartDelay) UI.
1-0 POdtTailWidth	controls the length of the tail of ProcOdt, units of UI 3 tail 3UI more than for Register POdtTailWidth=0, maximum 2 tail 2UI more than for Register POdtTailWidth=0, default 1 tail 1UI more than for Register POdtTailWidth=0 0 minimum length tail The time from ProcODT to closing the window to receive DQS to ProcODT POdtTailWidth is (2 + POdtTailWidth) UI

9.4.3.6.199 DLL gain control (DllGainCtl_p2)

9.4.3.6.199.1 Offset

Register	Offset
DllGainCtl_p2	40_00F8h

9.4.3.6.199.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				DIISeedSel				DIIGainTV				DIIGainIV			
W																
Reset	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	1

9.4.3.6.199.3 Fields

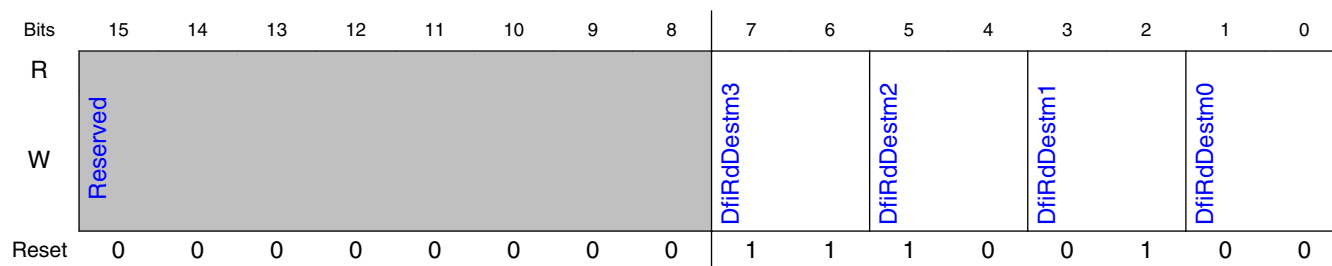
Field	Function
15-12 —	Reserved
11-8 DIISeedSel	Reserved, must be configured to be 0.
7-4 DIIGainTV	Terminal value of DIIGain, ie the value in effect when locking is done and the value used for maintaining lock, ie tracking pclk variation. Terminal value of DIIGain, ie the value in effect when locking is done and the value used for maintaining lock, ie tracking pclk variation. DIIGainTV must be greater than or equal to DIIGainIV. The maximum value is 10. The minimum value is 6.
3-0 DIIGainIV	Initial value of DIIGain.

9.4.3.6.200 Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiRdDataCsDestMap_p2)

9.4.3.6.200.1 Offset

Register	Offset
DfiRdDataCsDestMap_p2	40_0160h

9.4.3.6.200.2 Diagram



9.4.3.6.200.3 Fields

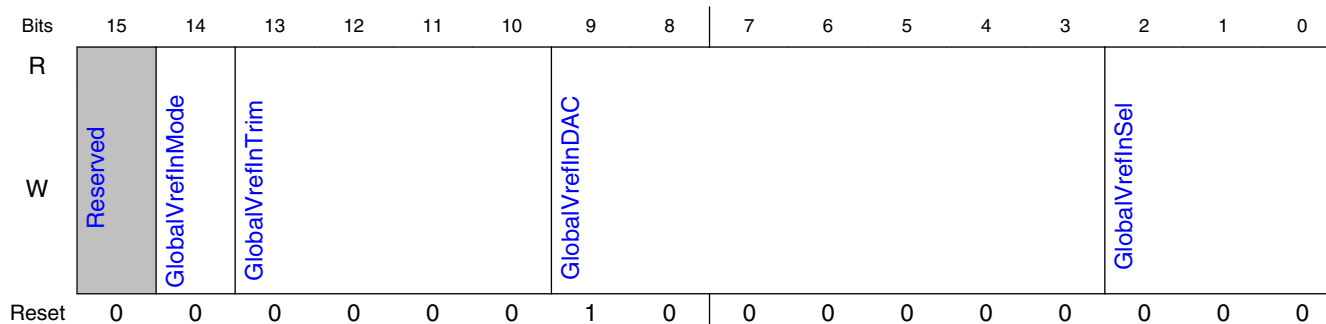
Field	Function
15-8 —	Reserved
7-6 DfiRdDestm3	Maps dfi_rddata_cs_n_p0[3] to dest DfiRdDestm3 timing For example, if 3 dfi_rddata_cs_n_p0[3] will use Register RxEn,ClkDlyTg3 timing.
5-4 DfiRdDestm2	Maps dfi_rddata_cs_n_p0[2] to dest DfiRdDestm2 timing For example, if 2 dfi_rddata_cs_n_p0[2] will use Register RxEn,ClkDlyTg2 timing.
3-2 DfiRdDestm1	Maps dfi_rddata_cs_n_p0[1] to dest DfiRdDestm1 timing For example, if 1 dfi_rddata_cs_n_p0[1] will use Register RxEn,ClkDlyTg1 timing.
1-0 DfiRdDestm0	Maps dfi_rddata_cs_n_p0[0] to dest DfiRdDestm0 timing For example, if 0 dfi_rddata_cs_n_p0[0] will use Register RxEn,ClkDlyTg0 timing.

9.4.3.6.201 PHY Global Vref Controls (VreflnGlobal_p2)

9.4.3.6.201.1 Offset

Register	Offset
VreflnGlobal_p2	40_0164h

9.4.3.6.201.2 Diagram



9.4.3.6.201.3 Fields

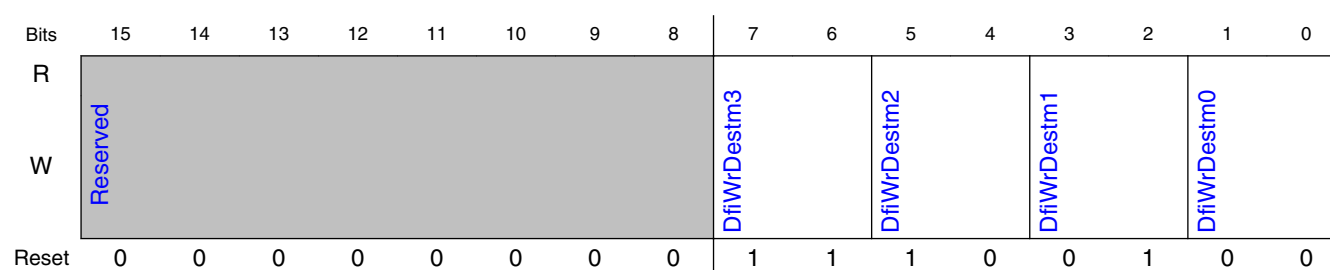
Field	Function
15 —	Reserved
14 GlobalVrefInMode	RSVD
13-10 GlobalVrefInTrim	RSVD
9-3 GlobalVrefInDAC	DAC code for internal Vref generation The DAC has two ranges; the range is set by GlobalVrefInSel[2] ===== RANGE0 : DDR3,DDR4,LPDDR3 [GlobalVrefInSel[2] = 0] DAC Output Voltage = GlobalVrefInDAC == 6'h00 ? Hi-Z : 0. DAC code for internal Vref generation The DAC has two ranges; the range is set by GlobalVrefInSel[2] ===== RANGE0 : DDR3,DDR4,LPDDR3 [GlobalVrefInSel[2] = 0] DAC Output Voltage = GlobalVrefInDAC == 6'h00 ? Hi-Z : 0.345*VDDQ + (0.005*GlobalVrefInDAC)*VDDQ ===== RANGE1 : LPDDR4 [GlobalVrefInSel[2] = 1] DAC Output Voltage = GlobalVrefInDAC == 6'h00 ? Hi-Z : (0.005*(GlobalVrefInDAC-1))*VDDQ
2-0 GlobalVrefInSel	GlobalVrefInSel[1:0] controls the mode of the PHY VREF DAC and the BP_VREF pin ===== 2'b00 - PHY Vref DAC Range0 -- BP_VREF = Hi-Z 2'b01 - Reserved Encoding 2'b10 - PHY Vref DAC Range0 -- BP_VREF connected to PLL Analog Bus 2'b11 - PHY Vref DAC Range0 -- BP_VREF connected to PHY Vref DAC ===== GlobalVrefInSel[2] shall be set according to Dram Protocol: Protocol GlobalVrefInSel[2] ----- DDR3 1'b0 DDR4 1'b0 LPDDR3 1'b0 LPDDR4 1'b1

9.4.3.6.202 Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiWrDataCsDestMap_p2)

9.4.3.6.202.1 Offset

Register	Offset
DfiWrDataCsDestMap_p2	40_0168h

9.4.3.6.202.2 Diagram



9.4.3.6.202.3 Fields

Field	Function
15-8 —	Reserved
7-6 DfiWrDestm3	Maps dfi_wrdata_cs_n_p0[3] to dest DfiWrDestm3 timing (use Register TxDq,DqsDlyTg3) For example, if 3 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg3 timing.
5-4 DfiWrDestm2	Maps dfi_wrdata_cs_n_p0[2] to dest DfiWrDestm2 timing For example, if 2 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg2 timing.
3-2 DfiWrDestm1	Maps dfi_wrdata_cs_n_p0[1] to dest DfiWrDestm1 timing For example, if 1 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg1 timing.
1-0 DfiWrDestm0	Maps dfi_wrdata_cs_n_p0[0] to dest DfiWrDestm0 timing For example, if 0 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg0 timing.

9.4.3.6.203 PState dependent PLL Control Register 2 (PlIcCtrl2_p2)

9.4.3.6.203.1 Offset

Register	Offset
PIICtrl2_p2	40_018Ah

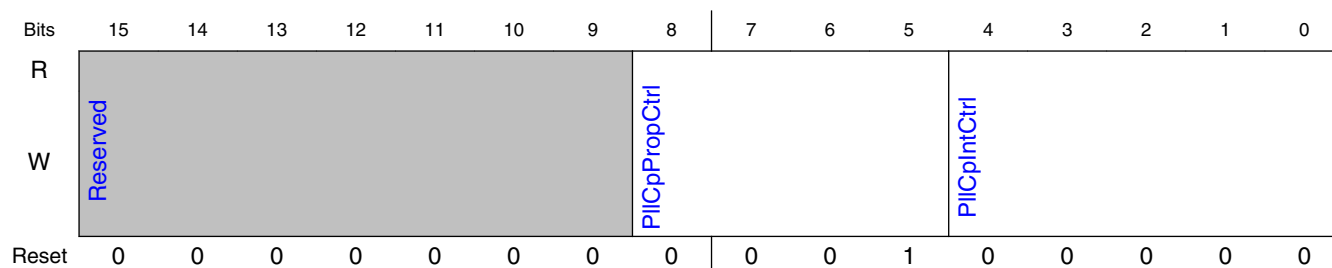
9.4.3.6.203.2 Diagram**9.4.3.6.203.3 Fields**

Field	Function
15-5 —	Reserved
4-0 PIIFreqSel	Adjusts the loop parameters to compensate for different VCO bias points, and input/output clock division ratios. Adjusts the loop parameters to compensate for different VCO bias points, and input/output clock division ratios. Program based on reference clock frequency (DfiClk) with this table.

9.4.3.6.204 PState dependent PLL Control Register 1 (PIICtrl1_p2)**9.4.3.6.204.1 Offset**

Register	Offset
PIICtrl1_p2	40_018Eh

9.4.3.6.204.2 Diagram



9.4.3.6.204.3 Fields

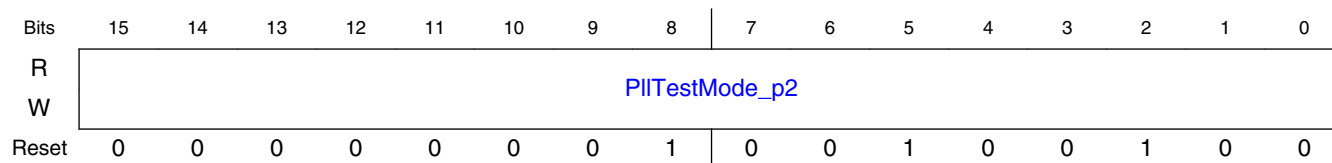
Field	Function
15-9 —	Reserved
8-5 PllCpPropCtrl	connects directly to cp_prop_cntrl<3:0> of PLL. connects directly to cp_prop_cntrl<3:0> of PLL. Charge pump proportional current control.
4-0 PllCpIntCtrl	connects directly to cp_int_cntrl<1:0> in PLL. connects directly to cp_int_cntrl<1:0> in PLL. Charge pump integrating current control.

9.4.3.6.205 Additional controls for PLL CP/VCO modes of operation (PIITestMode_p2)

9.4.3.6.205.1 Offset

Register	Offset
PIITestMode_p2	40_0194h

9.4.3.6.205.2 Diagram



9.4.3.6.205.3 Fields

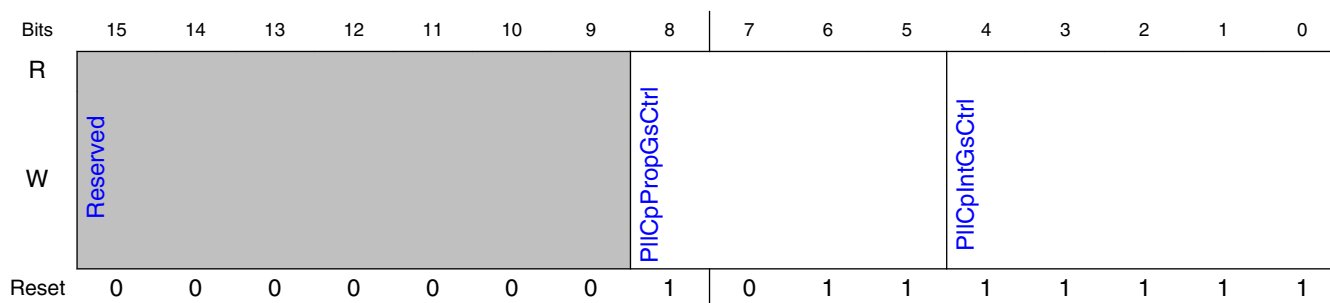
Field	Function
15-0	It is required to use default values for this CSR unless directed otherwise by Synopsys.
PIITestMode_p2	It is required to use default values for this CSR unless directed otherwise by Synopsys. Directly connected to testmode[15:0] pin of PLL

9.4.3.6.206 PState dependent PLL Control Register 4 (PIICtrl4_p2)

9.4.3.6.206.1 Offset

Register	Offset
PIICtrl4_p2	40_0198h

9.4.3.6.206.2 Diagram



9.4.3.6.206.3 Fields

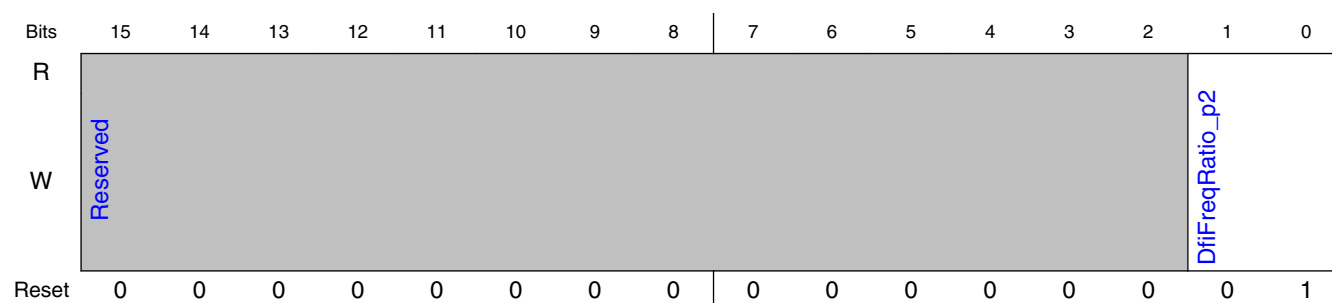
Field	Function
15-9 —	Reserved
8-5 PIICpPropGsCtrl	connects directly to cp_prop_gs_cntrl<3:0> of PLL. Charge pump proportional current control for fast relock and gearshift.
4-0 PIICpIntGsCtrl	connects directly to cp_int_gs_cntrl<4:0> in PLL. Charge pump integrating current control for fast relock and gearshift.

9.4.3.6.207 DFI Frequency Ratio (DfiFreqRatio_p2)

9.4.3.6.207.1 Offset

Register	Offset
DfiFreqRatio_p2	40_01F4h

9.4.3.6.207.2 Diagram



9.4.3.6.207.3 Fields

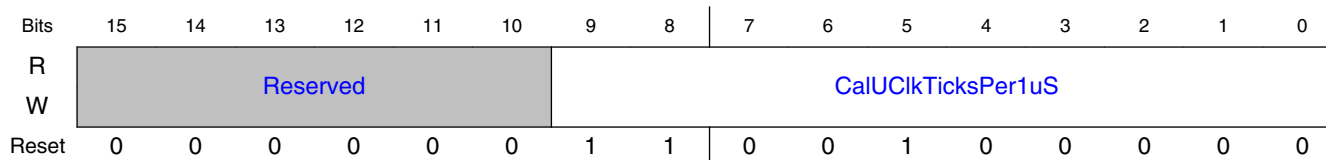
Field	Function
15-2	Reserved
—	
1-0 DfiFreqRatio_p2	Used in dwc_ddrphy_pub_serdes to serialize or de-serialize DFI signals 00 = 1:1 mode 01 = 1:2 mode 1x = 1:4 mode* *Note: 1:4 is for future pub revision.

9.4.3.6.208 Impedance Calibration Clock Ratio (CalUclkInfo_p3)

9.4.3.6.208.1 Offset

Register	Offset
CalUclkInfo_p3	60_0010h

9.4.3.6.208.2 Diagram



9.4.3.6.208.3 Fields

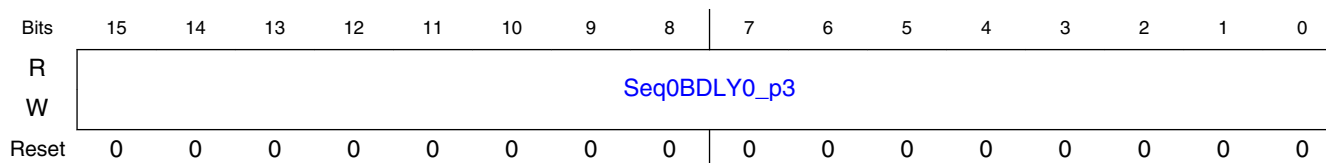
Field	Function
15-10 —	Reserved
9-0 CalUClkTicksPer1uS	Must be programmed to the number of DfIClks in 1us (rounded up), with minimum value of 24. Must be programmed to the number of DfIClks in 1us (rounded up), with minimum value of 24. if (DfIClk < 24MHz) CalUclInfo = 24 else CalUclInfo = (number of DfIClks in 1us)

9.4.3.6.209 PHY Initialization Engine (PIE) Delay Register 0 (Seq0BDLY0_p3)

9.4.3.6.209.1 Offset

Register	Offset
Seq0BDLY0_p3	60_0016h

9.4.3.6.209.2 Diagram



9.4.3.6.209.3 Fields

Field	Function
15-0 Seq0BDLY0_p3	PHY Initialization Engine (PIE) Delay Register 0 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value.

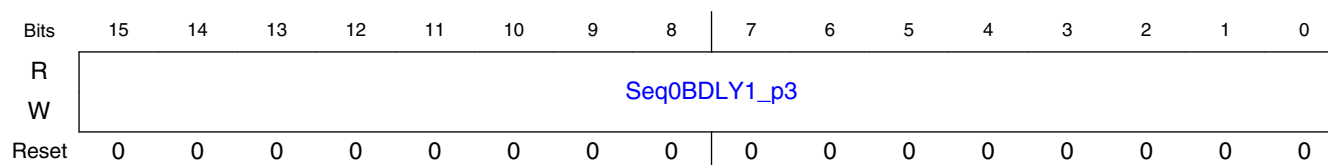
Field	Function
	PHY Initialization Engine (PIE) Delay Register 0 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.210 PHY Initialization Engine (PIE) Delay Register 1 (Seq0BDLY1_p3)

9.4.3.6.210.1 Offset

Register	Offset
Seq0BDLY1_p3	60_0018h

9.4.3.6.210.2 Diagram



9.4.3.6.210.3 Fields

Field	Function
15-0 Seq0BDLY1_p3	PHY Initialization Engine (PIE) Delay Register 1 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 1 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfiClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.211 PHY Initialization Engine (PIE) Delay Register 2 (Seq0BDLY2_p3)

9.4.3.6.211.1 Offset

Register	Offset
Seq0BDLY2_p3	60_001Ah

9.4.3.6.211.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Seq0BDLY2_p3															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.4.3.6.211.3 Fields

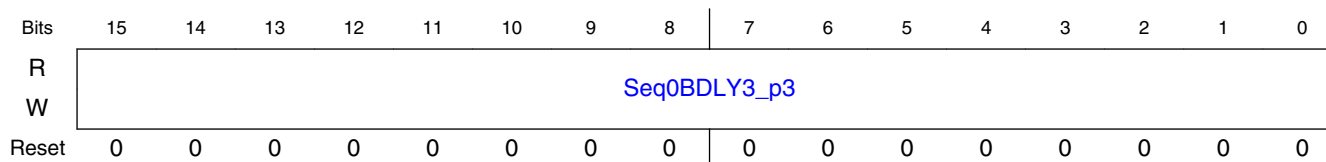
Field	Function
15-0 Seq0BDLY2_p3	PHY Initialization Engine (PIE) Delay Register 2 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 2 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfIClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.212 PHY Initialization Engine (PIE) Delay Register 3 (Seq0BDLY3_p3)

9.4.3.6.212.1 Offset

Register	Offset
Seq0BDLY3_p3	60_001Ch

9.4.3.6.212.2 Diagram



9.4.3.6.212.3 Fields

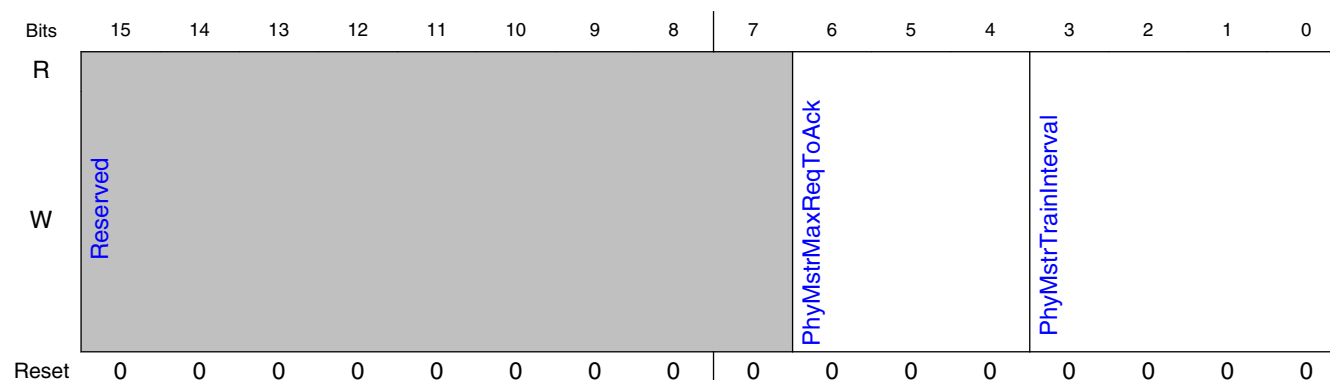
Field	Function
15-0 Seq0BDLY3_p3	PHY Initialization Engine (PIE) Delay Register 3 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. PHY Initialization Engine (PIE) Delay Register 3 This register is available for selection by the NOP and WAIT instructions in the PIE for the delay value. Programmed by dwc_ddrphy_phyinit_l_loadPIEImage() to support frequency changes. Programming Seq0BDLY[3,2,1,0] Registers: ----- These registers are used to control various delays during PLL, DLL initialization. These registers should be programmed with the corresponding number of MEMCLK cycles representing a fixed amount of delay irrespective of DfIClk Frequency. When calculating the number of cycles to be programmed for each PState, it should be noted that the sequencer multiplies programmed values by a factor of 8. Register Required Delay Programmed Value ----- Seq0BDLY0 0.5us (0.5us / MEMCLK period) / 8 Seq0BDLY1 1.0us (1.0us / MEMCLK period) / 8 Seq0BDLY2 10.0us (10.0us / MEMCLK period) / 8 64 if (MEMCLK Freq <= 400MHz) Seq0BDLY3 - 132 if (400MHz Freq < MEMCLK Freq <= 533MHz) 176 if (MEMCLK Freq > 533MHz)

9.4.3.6.213 Setup Intervals for DFI PHY Master operations (PPTTrainSetup_p3)

9.4.3.6.213.1 Offset

Register	Offset
PPTTrainSetup_p3	60_0020h

9.4.3.6.213.2 Diagram



9.4.3.6.213.3 Fields

Field	Function
15-7 —	Reserved
6-4 PhyMstrMaxReqToAck	Bits 6:4 of this register specify the max time from tdfi_phymstr_req asserted to tdfi_phymstr_ack asserted. Bits 6:4 of this register specify the max time from tdfi_phymstr_req asserted to tdfi_phymstr_ack asserted. 3'b000 Disable PHY Master Interface 3'b001 set tPHYMSTR_resp. = 512 MEMCLKs 3'b010 set tPHYMSTR_resp. = 1024 MEMCLKs 3'b011 set tPHYMSTR_resp. = 2048 MEMCLKs 3'b100 set tPHYMSTR_resp. = 4096 MEMCLKs 3'b101 set tPHYMSTR_resp. = 8192 MEMCLKs 3'b110 set tPHYMSTR_resp. = 32768 MEMCLKs 3'b111 set tPHYMSTR_resp. = undefined
3-0 PhyMstrTrainInterval	Bits 3:0 of this register specifies the time between the end of one training and the start of the next. Bits 3:0 of this register specifies the time between the end of one training and the start of the next. (As per section 11 of the Preliminary DFI 4.0 Specification, V2, it is the max expected time from dfi_init_complete asserted to tdfi_phymstr_ack asserted). 4'b0000 Disable PHY Master Interface 4'b0001 PHY MASTER Request Interval = 524288 MEMCLKs 4'b0010 PHY MASTER Request Interval = 1048576 MEMCLKs

DDR PHY (DDR_PHY)

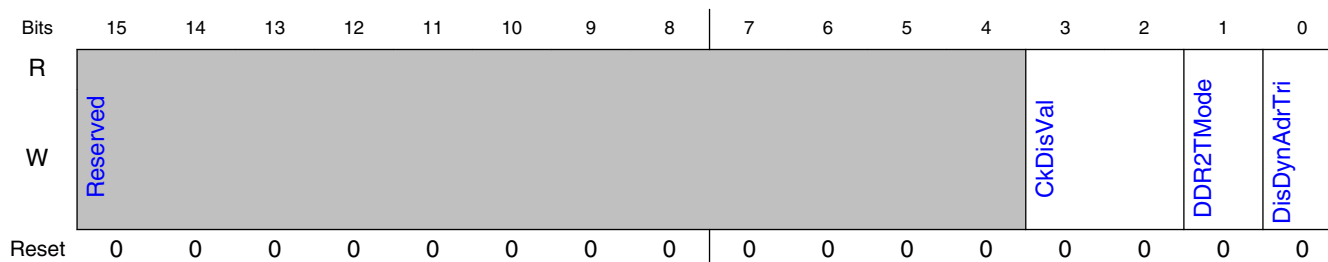
Field	Function
	4'b0011 PHY MASTER Request Interval = 2097152 MEMCLKs
	4'b0100 PHY MASTER Request Interval = 4194304 MEMCLKs
	4'b0101 PHY MASTER Request Interval = 8388608 MEMCLKs
	4'b0110 PHY MASTER Request Interval = 16777216 MEMCLKs
	4'b0111 PHY MASTER Request Interval = 33554432 MEMCLKs
	4'b1000 PHY MASTER Request Interval = 67108864 MEMCLKs
	4'b1001 PHY MASTER Request Interval = 134217728 MEMCLKs
	4'b1010 PHY MASTER Request Interval = 268435456 MEMCLKs
	4'b1011 - 4'b1111 PHY MASTER Request Interval = undefined

9.4.3.6.214 Mode select register for MEMCLK/Address/Command Tristates (TristateModeCA_p3)

9.4.3.6.214.1 Offset

Register	Offset
TristateModeCA_p3	60_0032h

9.4.3.6.214.2 Diagram



9.4.3.6.214.3 Fields

Field	Function
15-4 —	Reserved
3-2 CkDisVal	The PHY provides 4 memory clocks, n=0. The PHY provides 4 memory clocks, n=0..3. When the toggling of memory clock CK_t[n] is disabled with dfi_clk_disable[n]=1,

Table continues on the next page...

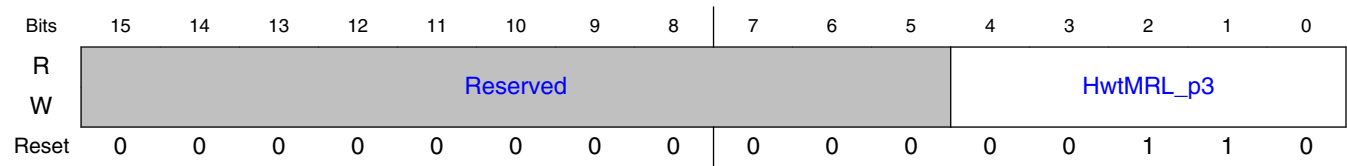
Field	Function
	the memory clock CK_t[n] is driven with Register CkDisVal[1]. When the toggling of memory clock CK_c[n] is disabled with dfi_clk_disable[n]=1, the memory clock CK_c[n] is driven with ~(Register CkDisVal[0]) (ie inverted). Note that the non-toggling differential memory clock CK_t[n],CK_c[n] may be driven with any combination, LL,LH,HL,HH; the default CK_t[n],CK_c[n]=LH.
1 DDR2TMode	Must be set to 1 for Dynamic Tristate to work when CA bus is 2T or Geardown mode. Must be set to 1 for Dynamic Tristate to work when CA bus is 2T or Geardown mode. This CSR bit has no effect when DisDynAdrTri==1
0 DisDynAdrTri	When DisDynAdrTri=1, Dynamic Tristating is disabled. When DisDynAdrTri=1, Dynamic Tristating is disabled. Dynamic Tristating is on by default. . Dynamic Tristating should be disabled (DisDynAdrTri=0) for these modes: 1. DDR3/2T if the controller cannot follow the 2T PHY tristate protocol. 2. DDR4/2T/2N if the controller cannot follow the 2T PHY tristate protocol. 3. LPDDR4 . When DisDynAdrTri=0 (default), In DDR3 mode, The following SDRAM pins can be dynamically tristated: A,BA,RAS_n,CAS_n,WE_n In DDR4 mode, The following SDRAM pins can be dynamically tristated: A,BA,BG,ACT_n In LPDDR3 mode, The following SDRAM pins can be dynamically tristated: CA[*] . In 1T mode, the PHY will tristate the relevant pins when all ranks are DESelected . In 2T mode (or geardown), the PHY will tristate the relevant pins when: D3 -- All ranks DESelected (all CS_L==1) AND RAS_n,CAS_n,WE_n,BA0 = 1,1,1,0 D4 -- All ranks DESelected (all CS_L==1) AND RAS_n,CAS_n,WE_n,BA0,ACT_n = 1,1,1,0,1 When in this mode, the controller should avoid sending the above patterns with any rank selected. [e.g. NOPs sent by the controller should have BA0 set to a non-zero value]

9.4.3.6.215 HWT MaxReadLatency. (HwtMRL_p3)

9.4.3.6.215.1 Offset

Register	Offset
HwtMRL_p3	60_0040h

9.4.3.6.215.2 Diagram



9.4.3.6.215.3 Fields

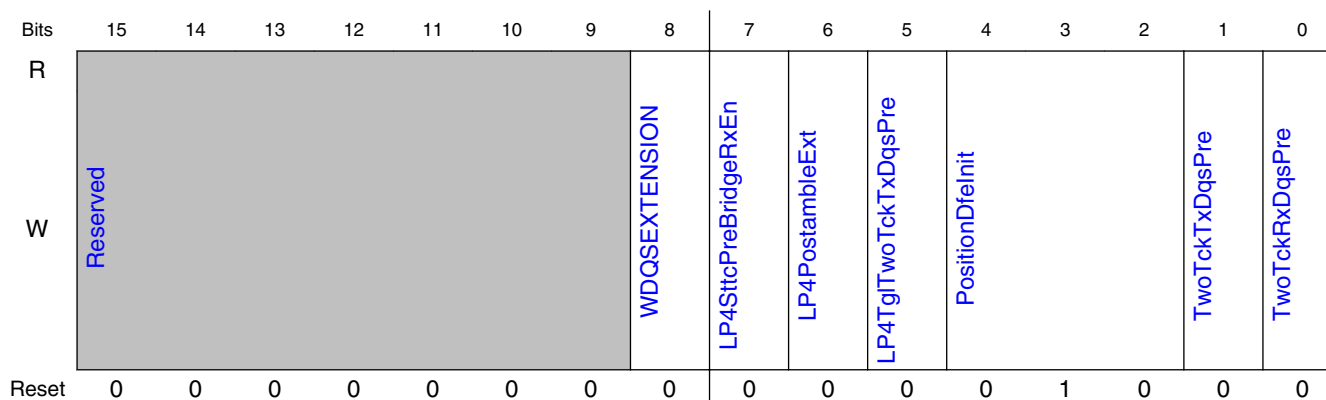
Field	Function
15-5 —	Reserved
4-0 HwtMRL_p3	This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. This Max Read Latency CSR is to be trained to ensure the rx-data fifo is not read until after all dbytes have their read data valid. The CSR is in units of two mem clocks; that is, a unit change in the LSB is a change in MRL of two mem clocks. This MASTER copy of MRL is used by the PHY training hardware only.

9.4.3.6.216 Control the PHY logic related to the read and write DQS preamble (DqsPreambleControl_p3)

9.4.3.6.216.1 Offset

Register	Offset
DqsPreambleControl_p3	60_0048h

9.4.3.6.216.2 Diagram



9.4.3.6.216.3 Fields

Field	Function
15-9 —	Reserved
8 WDQSEXTENSION	When set, DQS_T and DQS_C will be driven differentially to 0 and 1, respectively, before and after a write burst, except during a memory read transaction. When set, DQS_T and DQS_C will be driven differentially to 0 and 1, respectively, before and after a write burst, except during a memory read transaction. See DesignWare Cores LPDDR4 MultiPHY: WDQS Extension Application Note. Use of WDQSEXTENSION requires POdtTailWidth=3 and POdtStartDelay=00
7 LP4SttcPreBridgeRxEn	Used in LPDDR4 static-preamble mode to bridge the RxEn between two reads to the same timing group when the bubble is 1 memclk. Used in LPDDR4 static-preamble mode to bridge the RxEn between two reads to the same timing group when the bubble is 1 memclk. LPDDR4 static-preamble mode is set in the DRAMs with MR1, OP[3]=0.
6 LP4PostambleExt	In LPDDR4 mode must be set to extend the write postamble. In LPDDR4 mode must be set to extend the write postamble. 0: half-memclk postamble, as in D4 1: one-and-one-half-memclk postamble; see LPDDR4 Spec MR3, OP[1] WR PST, vendor-specific function.
5 LP4TglTwoTckTxDqsPre	Used in LPDDR4 mode to modify the early preamble when Register TwoTckTxDqsPre=1 0: level first-memclk preamble 1: toggling first-memclk preamble
4-2 PositionDfelnit	For DDR4 phy only when receive DFE is enabled. For DDR4 phy only when receive DFE is enabled. PositionDfelnit[2]=1 causes Dfelnit to be asserted late PositionDfelnit[1]=1 causes Dfelnit to be asserted at the nominal time

Table continues on the next page...

DDR PHY (DDR_PHY)

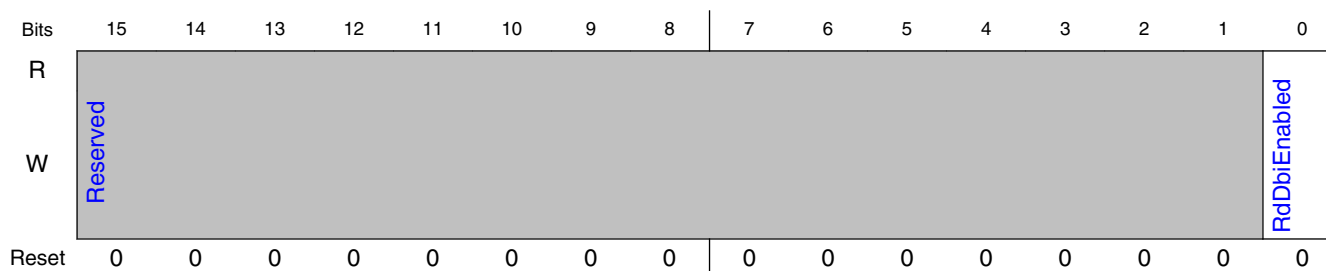
Field	Function
	PositionDfelnit[0]=1 causes Dfelnit to be asserted early PositionDfelnit=3'b010 is the nominal, recommended value for leading edge used by receiver.
1 TwoTckTxDqsPre	0: Standard 1tck TxDqs Preamble 1: Enable Optional D4 2tck TxDqs Preamble The DDR4 MR4 A12 is Write Preamble, 1=2nCK, 0=1nCK.
0 TwoTckRxDqsPre	Widens the RxDqsEn window to allow larger drift in the incoming read DQS to take advantage of the larger/wider preamble generated by the DRAMs when the D4 DRAMs are configured with DDR4 MR4 A11 Read Preamble=1 for causing a 2nCK read preamble. Widens the RxDqsEn window to allow larger drift in the incoming read DQS to take advantage of the larger/wider preamble generated by the DRAMs when the D4 DRAMs are configured with DDR4 MR4 A11 Read Preamble=1 for causing a 2nCK read preamble. For LPDDR4, all read operations are 2nCK such that this control must be set to 1. Note the nominal trained-to-center point will be earlier relative to the first strobing edge of DQS than when not in this mode.

9.4.3.6.217 This Register is used to enable the Read-DBI function in each DBYTE (DMIPinPresent_p3)

9.4.3.6.217.1 Offset

Register	Offset
DMIPinPresent_p3	60_005Ah

9.4.3.6.217.2 Diagram



9.4.3.6.217.3 Fields

Field	Function
15-1	Reserved

Table continues on the next page...

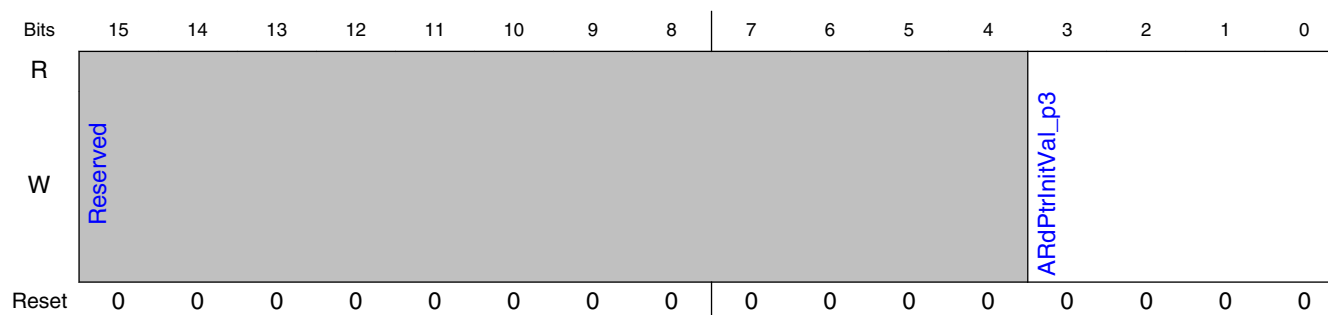
Field	Function
—	
0	This bit must be set to 1'b1 if Read-DBI is enabled in a connected DDR4 or LPDDR4 device.
RdDbiEnabled	This bit must be set to 1'b1 if Read-DBI is enabled in a connected DDR4 or LPDDR4 device. If set, the following DRAM MR should also be set [DDR4.MR5.A12=1 or LPDDR4.MR3.OP[7]=1]

9.4.3.6.218 Address/Command FIFO ReadPointer Initial Value (ARdPtrInitVal_p3)

9.4.3.6.218.1 Offset

Register	Offset
ARdPtrInitVal_p3	60_005Ch

9.4.3.6.218.2 Diagram



9.4.3.6.218.3 Fields

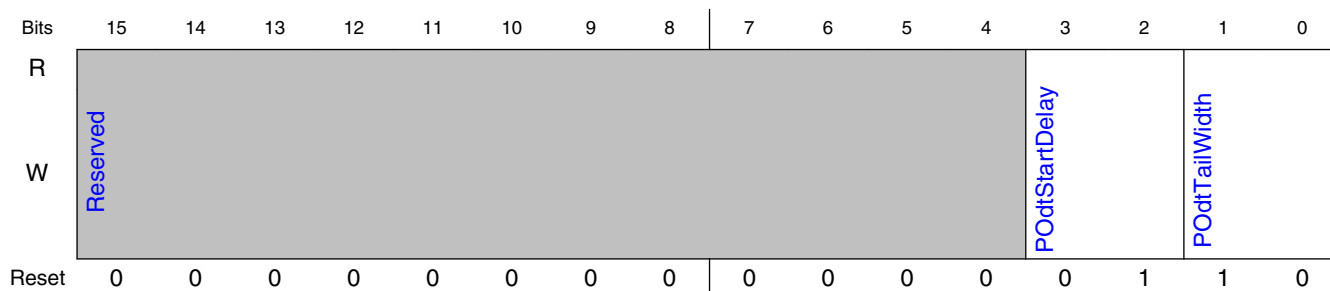
Field	Function
15-4	Reserved
—	
3-0	This is the initial Pointer Offset for the free-running FIFOs in the DBYTE and ACX4 hardips.
ARdPtrInitVal_p3	This is the initial Pointer Offset for the free-running FIFOs in the DBYTE and ACX4 hardips. The units of this register are in UI. The pointer separation should be chosen to compensate for all sources of skew and drift of the PHY DFICLK and PCLK networks. Please see PUB databook section 8.1.1 This CSR must be programmed in Step C of the PHY Initialization sequence

9.4.3.6.219 READ DATA On-Die Termination Timing Control (by PHY) (ProcOdtTimeCtl_p3)

9.4.3.6.219.1 Offset

Register	Offset
ProcOdtTimeCtl_p3	60_00ACh

9.4.3.6.219.2 Diagram



9.4.3.6.219.3 Fields

Field	Function
15-4 —	Reserved
3-2 POdtStartDelay	controls the start of ProcOdt, units of UI 3 delay start 2 UI, maximum delay of start of ProcOdt 2 delay start 1 UI, 1 delay start 0 UI, default 0 early by 1 UI, The time from ProcODT assertion to opening the window to receive DQS is (10 - POdtStartDelay) UI.
1-0 POdtTailWidth	controls the length of the tail of ProcOdt, units of UI 3 tail 3UI more than for Register POdtTailWidth=0, maximum 2 tail 2UI more than for Register POdtTailWidth=0, default 1 tail 1UI more than for Register POdtTailWidth=0 0 minimum length tail The time from ProcODT to closing the window to receive DQS to ProcODT POdtTailWidth is (2 + POdtTailWidth) UI

9.4.3.6.220 DLL gain control (DllGainCtl_p3)

9.4.3.6.220.1 Offset

Register	Offset
DllGainCtl_p3	60_00F8h

9.4.3.6.220.2 Diagram

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved				DIISeedSel				DIIGainTV				DIIGainIV			
W																
Reset	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	1

9.4.3.6.220.3 Fields

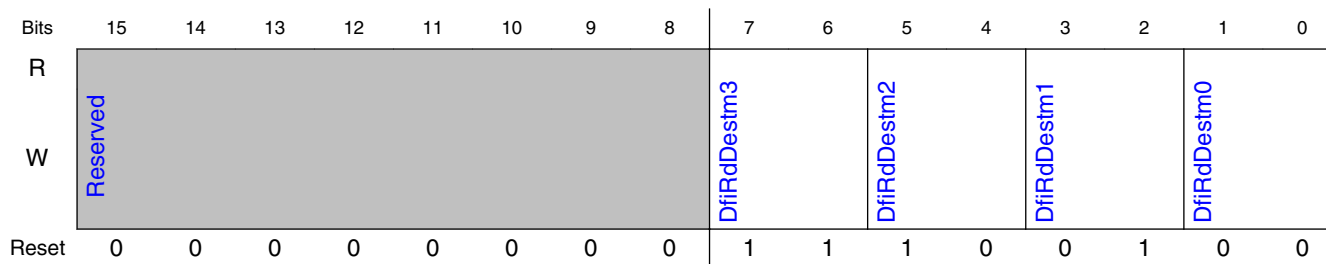
Field	Function
15-12 —	Reserved
11-8 DIISeedSel	Reserved, must be configured to be 0.
7-4 DIIGainTV	Terminal value of DIIGain, ie the value in effect when locking is done and the value used for maintaining lock, ie tracking pclk variation. Terminal value of DIIGain, ie the value in effect when locking is done and the value used for maintaining lock, ie tracking pclk variation. DIIGainTV must be greater than or equal to DIIGainIV. The maximum value is 10. The minimum value is 6.
3-0 DIIGainIV	Initial value of DIIGain.

9.4.3.6.221 Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiRdDataCsDestMap_p3)

9.4.3.6.221.1 Offset

Register	Offset
DfiRdDataCsDestMap_p3	60_0160h

9.4.3.6.221.2 Diagram



9.4.3.6.221.3 Fields

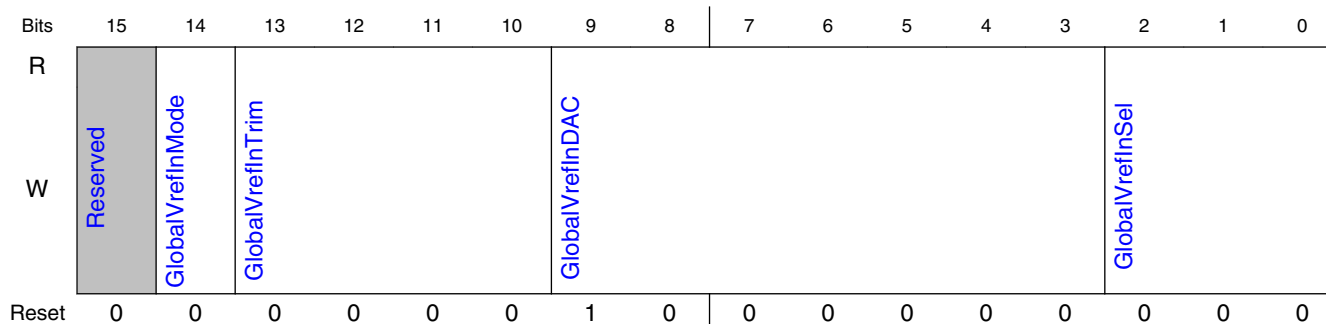
Field	Function
15-8 —	Reserved
7-6 DfiRdDestm3	Maps dfi_rddata_cs_n_p0[3] to dest DfiRdDestm3 timing For example, if 3 dfi_rddata_cs_n_p0[3] will use Register RxEn,ClkDlyTg3 timing.
5-4 DfiRdDestm2	Maps dfi_rddata_cs_n_p0[2] to dest DfiRdDestm2 timing For example, if 2 dfi_rddata_cs_n_p0[2] will use Register RxEn,ClkDlyTg2 timing.
3-2 DfiRdDestm1	Maps dfi_rddata_cs_n_p0[1] to dest DfiRdDestm1 timing For example, if 1 dfi_rddata_cs_n_p0[1] will use Register RxEn,ClkDlyTg1 timing.
1-0 DfiRdDestm0	Maps dfi_rddata_cs_n_p0[0] to dest DfiRdDestm0 timing For example, if 0 dfi_rddata_cs_n_p0[0] will use Register RxEn,ClkDlyTg0 timing.

9.4.3.6.222 PHY Global Vref Controls (VreflnGlobal_p3)

9.4.3.6.222.1 Offset

Register	Offset
VreflnGlobal_p3	60_0164h

9.4.3.6.222.2 Diagram



9.4.3.6.222.3 Fields

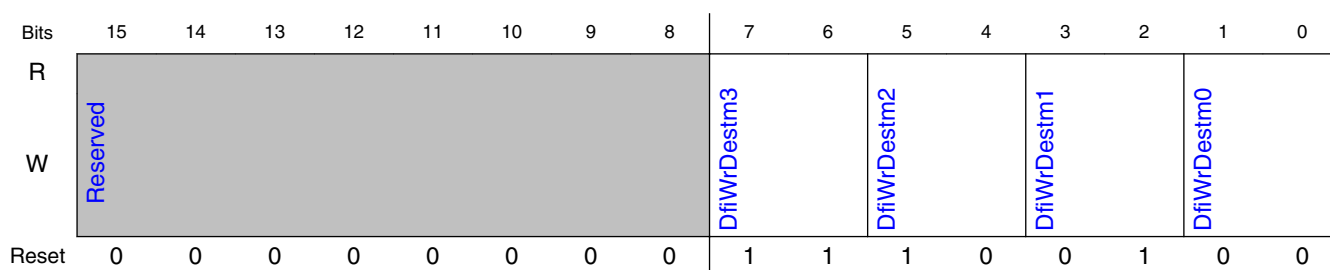
Field	Function
15 —	Reserved
14 GlobalVrefInMode	RSVD
13-10 GlobalVrefInTrim	RSVD
9-3 GlobalVrefInDAC	DAC code for internal Vref generation The DAC has two ranges; the range is set by GlobalVrefInSel[2] ===== RANGE0 : DDR3,DDR4,LPDDR3 [GlobalVrefInSel[2] = 0] DAC Output Voltage = GlobalVrefInDAC == 6'h00 ? Hi-Z : 0. DAC code for internal Vref generation The DAC has two ranges; the range is set by GlobalVrefInSel[2] ===== RANGE0 : DDR3,DDR4,LPDDR3 [GlobalVrefInSel[2] = 0] DAC Output Voltage = GlobalVrefInDAC == 6'h00 ? Hi-Z : 0.345*VDDQ + (0.005*GlobalVrefInDAC)*VDDQ ===== RANGE1 : LPDDR4 [GlobalVrefInSel[2] = 1] DAC Output Voltage = GlobalVrefInDAC == 6'h00 ? Hi-Z : (0.005*(GlobalVrefInDAC-1))*VDDQ
2-0 GlobalVrefInSel	GlobalVrefInSel[1:0] controls the mode of the PHY VREF DAC and the BP_VREF pin ===== 2'b00 - PHY Vref DAC Range0 -- BP_VREF = Hi-Z 2'b01 - Reserved Encoding 2'b10 - PHY Vref DAC Range0 -- BP_VREF connected to PLL Analog Bus 2'b11 - PHY Vref DAC Range0 -- BP_VREF connected to PHY Vref DAC ===== GlobalVrefInSel[2] shall be set according to Dram Protocol: Protocol GlobalVrefInSel[2] ----- DDR3 1'b0 DDR4 1'b0 LPDDR3 1'b0 LPDDR4 1'b1

9.4.3.6.223 Maps dfi_rddata_cs_n to destination dimm timing group for use with D4 LRDIMM (DfiWrDataCsDestMap_p3)

9.4.3.6.223.1 Offset

Register	Offset
DfiWrDataCsDestMap_p3	60_0168h

9.4.3.6.223.2 Diagram



9.4.3.6.223.3 Fields

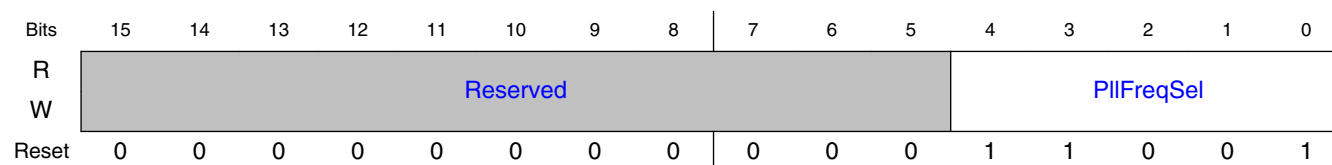
Field	Function
15-8 —	Reserved
7-6 DfiWrDestm3	Maps dfi_wrdata_cs_n_p0[3] to dest DfiWrDestm3 timing (use Register TxDq,DqsDlyTg3) For example, if 3 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg3 timing.
5-4 DfiWrDestm2	Maps dfi_wrdata_cs_n_p0[2] to dest DfiWrDestm2 timing For example, if 2 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg2 timing.
3-2 DfiWrDestm1	Maps dfi_wrdata_cs_n_p0[1] to dest DfiWrDestm1 timing For example, if 1 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg1 timing.
1-0 DfiWrDestm0	Maps dfi_wrdata_cs_n_p0[0] to dest DfiWrDestm0 timing For example, if 0 dfi_wrdata_cs_n_p0[0] will use Register TxDq,DqsDlyTg0 timing.

9.4.3.6.224 PState dependent PLL Control Register 2 (PlIcCtrl2_p3)

9.4.3.6.224.1 Offset

Register	Offset
PIICtrl2_p3	60_018Ah

9.4.3.6.224.2 Diagram



9.4.3.6.224.3 Fields

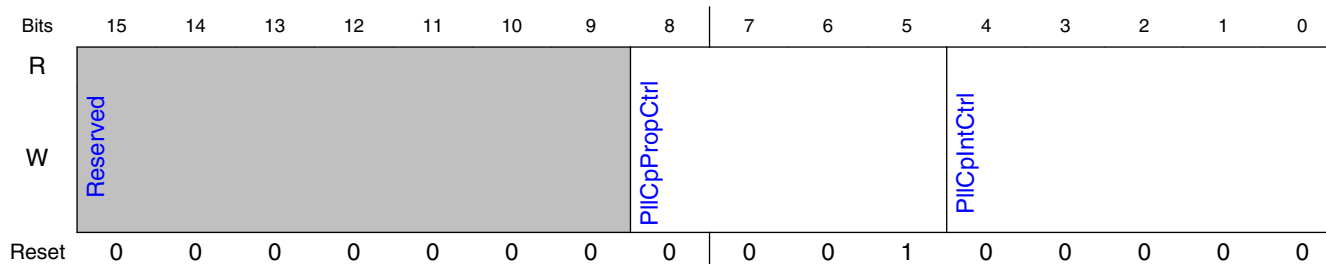
Field	Function
15-5 —	Reserved
4-0 PIIFreqSel	Adjusts the loop parameters to compensate for different VCO bias points, and input/output clock division ratios. Adjusts the loop parameters to compensate for different VCO bias points, and input/output clock division ratios. Program based on reference clock frequency (DfiClk) with this table.

9.4.3.6.225 PState dependent PLL Control Register 1 (PIICtrl1_p3)

9.4.3.6.225.1 Offset

Register	Offset
PIICtrl1_p3	60_018Eh

9.4.3.6.225.2 Diagram



9.4.3.6.225.3 Fields

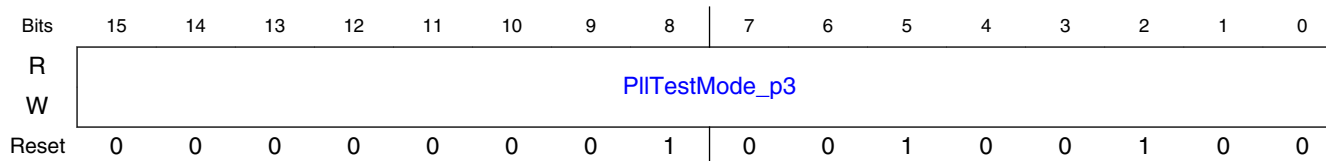
Field	Function
15-9 —	Reserved
8-5 PllCpPropCtrl	connects directly to cp_prop_cntrl<3:0> of PLL. connects directly to cp_prop_cntrl<3:0> of PLL. Charge pump proportional current control.
4-0 PllCpIntCtrl	connects directly to cp_int_cntrl<1:0> in PLL. connects directly to cp_int_cntrl<1:0> in PLL. Charge pump integrating current control.

9.4.3.6.226 Additional controls for PLL CP/VCO modes of operation (PIITestMode_p3)

9.4.3.6.226.1 Offset

Register	Offset
PIITestMode_p3	60_0194h

9.4.3.6.226.2 Diagram



9.4.3.6.226.3 Fields

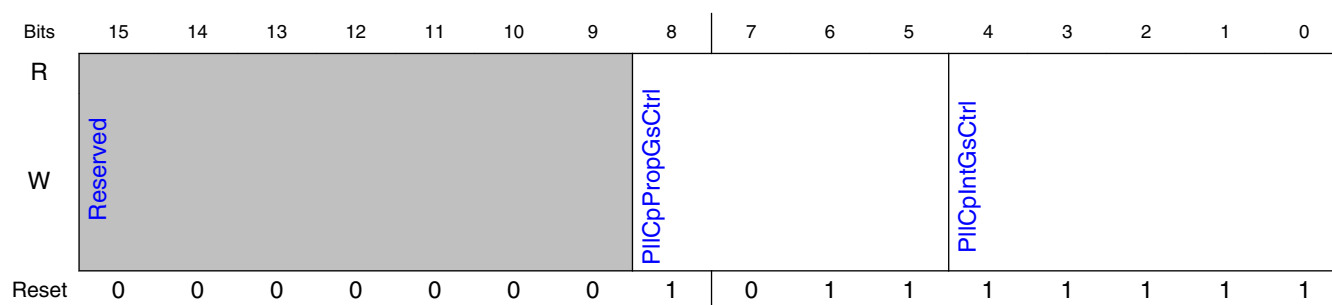
Field	Function
15-0	It is required to use default values for this CSR unless directed otherwise by Synopsys.
PIITestMode_p3	It is required to use default values for this CSR unless directed otherwise by Synopsys. Directly connected to testmode[15:0] pin of PLL

9.4.3.6.227 PState dependent PLL Control Register 4 (PIICtrl4_p3)

9.4.3.6.227.1 Offset

Register	Offset
PIICtrl4_p3	60_0198h

9.4.3.6.227.2 Diagram



9.4.3.6.227.3 Fields

Field	Function
15-9 —	Reserved
8-5 PIICpPropGsCtrl	connects directly to cp_prop_gs_cntrl<3:0> of PLL. Charge pump proportional current control for fast relock and gearshift.
4-0 PIICpIntGsCtrl	connects directly to cp_int_gs_cntrl<4:0> in PLL. Charge pump integrating current control for fast relock and gearshift.

9.4.3.6.228 DFI Frequency Ratio (DfiFreqRatio_p3)

9.4.3.6.228.1 Offset

Register	Offset
DfiFreqRatio_p3	60_01F4h

9.4.3.6.228.2 Diagram



9.4.3.6.228.3 Fields

Field	Function
15-2	Reserved
—	
1-0	Used in dwc_ddrphy_pub_serdes to serialize or de-serialize DFI signals 00 = 1:1 mode 01 = 1:2 mode 1x = 1:4 mode* *Note: 1:4 is for future pub revision.
DfiFreqRatio_p3	

9.5 AHB-to-APBH Bridge with DMA (APBH-Bridge-DMA)

9.5.1 Overview

The AHB-to-APBH bridge provides the chip with an inexpensive peripheral attachment bus running on the AHB's HCLK. (The H in APBH denotes that the APBH is synchronous to HCLK.)

As shown in the figure below, the AHB-to-APBH bridge includes the AHB-to-APB PIO bridge for a memory-mapped I/O to the APB devices, as well as a central DMA facility for devices on this bus and a vectored interrupt controller for the Arm core. Each one of the APB peripherals, including the vectored interrupt controller, is documented in their respective chapters.

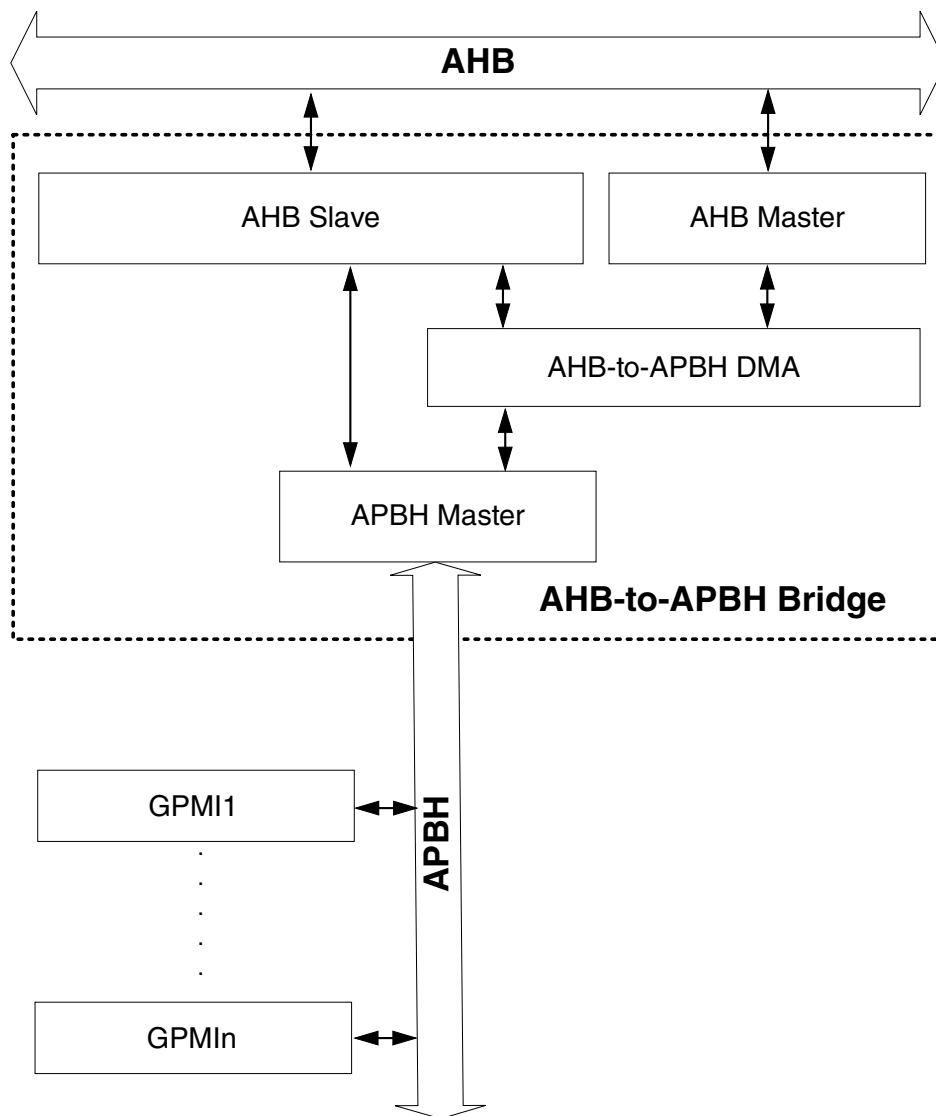


Figure 9-7. AHB-to-APBH Bridge DMA Block Diagram

The DMA controller uses the APBH bus to transfer read and write data to and from each peripheral. There is no separate DMA bus for these devices. Contention between the DMA's use of the APBH bus and the AHB-to-APB bridge functions' use of the APBH is mediated by an internal arbitration logic. For contention between these two units, the DMA is favored and the AHB slave will report "not ready" through its HREADY output until the bridge transfer can complete. The arbiter tracks repeated lockouts and inverts the priority, guaranteeing the Arm platform every fourth transfer on the APB.

9.5.2 Clocks

The table found here describes the clock sources for APBH. Please see [Clock Controller Module \(CCM\)](#) for clock setting, configuration and gating information.

Table 9-5. APBH Clocks

Clock name	Clock Root	Description
hclk		Module clock

9.5.3 APBH DMA

The DMA supports four channels of DMA services, as shown in the following table. The shared DMA resource allows each independent channel to follow a simple chained command list. Command chains are built up using the general structure, as shown in [Figure 9-8](#).

Table 9-6. APBH DMA channel assignments

APBH DMA Channel #	Usage
0	GPMI0
1	GPMI1
2	GPMI2
3	GPMI3

A single command structure or channel command word specifies a number of operations to be performed by the DMA in support of a given device. Thus, the Arm platform can set up large units of work, chaining together many DMA channel command words, pass them off to the DMA, and have no further concern for the device until the DMA completion interrupt occurs. The goal is to have enough intelligence in the DMA and the devices to keep the interrupt frequency from any device below 1 KHz (arrival intervals longer than 1 ms).

A single command structure can issue 32-bit PIO write operations to key registers in the associated device using the same APB bus and controls that it uses to write DMA data bytes to the device. For example, this allows a chain of operations to be issued to the GPMI controller to send NAND command bytes, address bytes, and data transfers where the command and the address structure is completely under software control, but the administration of that transfer is handled autonomously by the DMA. Each DMA

structure can have 0–15 PIO words appended to it. The CMDPIOWORDS field, if non-zero, instructs the DMA engine to copy these words to the APB, beginning at the first register address offset for the peripheral and incrementing the register offset each cycle.

The DMA master generates only normal read/write transfers to the APBH. It does *not* generate set, clear, or toggle (SCT) transfers.

After any requested PIO words have been transferred to the peripheral, the DMA examines the two-bit command field in the channel command structure. [Table 9-7](#) shows the four commands implemented by the DMA.

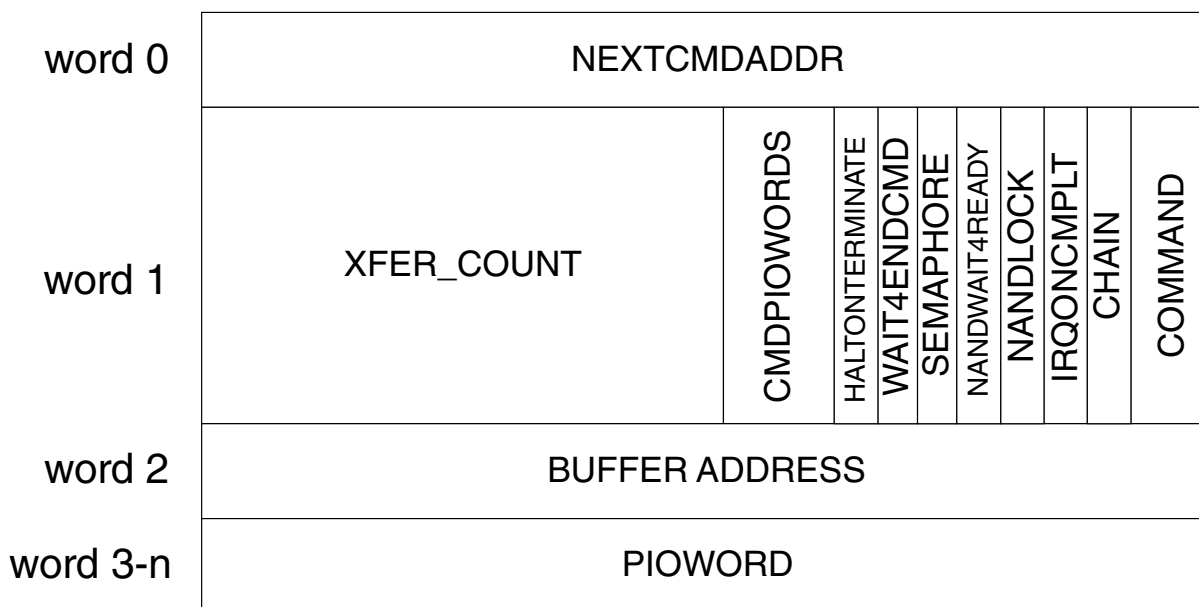


Figure 9-8. AHB-to-APBH Bridge DMA channel command structure

Table 9-7. APBH DMA commands

DMA Command	Usage
00	NO_DMA_XFER. Perform any requested PIO word transfers, but terminate the command before any DMA transfer.
01	DMA_WRITE. Perform any requested PIO word transfers, then perform a DMA transfer from the peripheral for the specified number of bytes.
10	DMA_READ. Perform any requested PIO word transfers and then perform a DMA transfer to the peripheral for the specified number of bytes.
11	DMA_SENSE. Perform any requested PIO word transfers, then perform a conditional branch to the next chained device. Follow the NEXTCMD_ADDR pointer if the peripheral sense is false. Follow the BUFFER_ADDRESS as a chain pointer if the peripheral sense line is true. This command becomes a no-operation for any channel other than a GPPI channel.

DMA_WRITE operations copy data bytes to the system memory (on-chip RAM or SDRAM) from the associated peripheral.

DMA_READ operations copy data bytes to the APB peripheral from the system memory. The DMA engine contains a shared byte aligner that aligns bytes from system memory to or from the peripherals. Peripherals always assume little-endian-aligned data arrives or departs on their 32-bit APB. The DMA_READ transfer uses the BUFFER_ADDRESS word in the command structure to point to the DMA data buffer to be read by the DMA_READ command.

The NO_DMA_XFER command is used to write PIO words to a device without performing any DMA data byte transfers. This command is useful in such applications as activating the NAND devices CHECKSTATUS operation. The check status command reads a status byte from the NAND device, performs an XOR and MASK against an expected value supplied as part of the PIO transfer. Once the read check completes (see [NAND Read Status Polling Example](#)), the NO_DMA_XFER command completes. The result in the peripheral is that its sense line is driven by the results of the comparison. The sense flip-flop is only updated by CHECKSTATUS for the device that is executed. At some future point, the chain contains a DMA command structure with the fourth and final command value, that is, the DMA_SENSE command.

As each DMA command completes, it triggers the DMA to load the next DMA command structure in the chain. The normal flow list of DMA commands is found by following the NEXTCMD_ADDR pointer in the DMA command structure. The DMA_SENSE command uses the DMA buffer pointer word of the command structure to point to an alternate DMA command structure chain or list. The DMA_SENSE command examines the sense line of the associated peripheral. If the sense line is false, then the DMA follows the standard list found whose next command is found from the pointer in the NEXTCMD_ADDR word of the command structure. If the sense line is true, then the DMA follows the alternate list whose next command is found from the pointer in the DMA Buffer Pointer word of the DMA_SENSE command structure (see [Figure 9-8](#)). The sense command ignores the CHAIN bit, so that both pointers must be valid when the DMA comes to a sense command.

If the wait-for-end-command bit (WAIT4ENDCMD) is set in a command structure, the DMA channel waits for the device to signal completion of a command by toggling the endcmd signal before proceeding to load and execute the next command structure. Then, if DECREMENT_SEMAPHORE is set, the semaphore is decremented after the end command is seen.

A detailed bit-field view of the DMA command structure is shown in the following table, which shows a field that specifies the number of bytes to be transferred by this DMA command. The transfer-count mechanism is duplicated in the associated peripheral, either as an implied or as a specified count in the peripheral.

Table 9-8. DMA channel command word in system memory

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00																
NEXT_COMMAND_ADDRESS																																															
Number DMA Bytes to Transfer																Number PIO Words to Write								HALT TERMINATE	WAIT4 ENDCMD	DECREMENT SEMAPHORE	NANDwait4 READY	NANDLOCK	IRQ_COMPLETE	CHAIN	COMMAND																
DMA Buffer or Alternate CCW																																															
Zero or More PIO Words to Write to the Associated Peripheral Starting at its Base Address on the APBH Bus																																															

Figure 9-8 also shows the CHAIN bit in bit 2 of the second word of the command structure. This bit is set to 1, if the NEXT_COMMAND_ADDRESS contains a pointer to another DMA command structure. If a null pointer (0) is loaded into the NEXT_COMMAND_ADDRESS, it is not detected by the DMA hardware. Only the CHAIN bit indicates whether a valid list exists beyond the current structure.

If the IRQ_COMPLETE bit is set in the command structure, then the last act of the DMA before loading the next command is to set the interrupt-status bit corresponding to the current channel. The sticky interrupt request bit in the DMA CSR remains set until cleared by the software. It can be used to interrupt the Arm platform.

The NAND_LOCK bit is monitored by the DMA channel arbiter. Once a NAND channel (from channel 0 to channel 3) succeeds in the arbiter with its NAND_LOCK bit set, then the arbiter ignores the other NAND channels until a command is completed in which the NAND_LOCK is not set. Notice that the semantic here is that the NAND_LOCK state is to limit scheduling of a non-locked DMA. A DMA channel can go from unlocked to locked in the arbiter at the beginning of a command when the NAND_LOCK bit is set. When the last DMA command of an atomic sequence is completed, the lock should be removed. To accomplish this, the last command does not have the NAND_LOCK bit. It is still locked in the atomic state within the arbiter when the command starts, so that it is the only NAND command that can be executed. At the end, it drops from the atomic state within the arbiter.

The NAND_WAIT4READY bit also has a special use for GPMI channels (from channel 0 to channel 3), i.e., the NAND device channels. The GPMI peripheral supplies a sample of the ready line from the NAND device. This ready value is used to hold off of a command with this bit set until the ready line is asserted to 1. Once the arbiter sees a command with a wait-for-ready set, it holds off that channel until ready is asserted.

Receiving an IRQ for HALTONTERMINATE (HOT) is a feature in the APBH DMA descriptor that allows GPMI to signal to the DMA engine that an error has occurred. If a command is stalled due to an error, a HOT signal is sent from the peripheral to the DMA engine and causes an IRQ after terminating the DMA descriptor being executed.

Therefore, it is recommended that software use this signal as follows:

- Always set HALTONTERMINATE to 1 in a DMA descriptor. That way, if a peripheral signals HOT, the transfer will end, leaving the peripheral block and the DMA engine synchronized (but at the end of a command).
- When an IRQ from an APBH channel is received, and the IRQ is determined to be due to an error (as opposed to an IRQONCOMPLETE interrupt) the software should:
 - Reset the channel.
 - Determine the error from error reporting in the peripheral block, then manage the error in the peripheral that is attached to that channel in whatever appropriate way exists for that device (software recovery, device reset, block reset, etc).

Each channel has an eight-bit counting semaphore that controls whether it is in the idle state. When the semaphore is non-zero, the channel is ready to run, process commands and perform DMA transfers. Whenever a command finishes its DMA transfer, it checks the DECREMENT_SEMAPHORE bit. If set, it decrements the counting semaphore. If the semaphore goes to 0 as a result, then the channel enters the idle state and remains there until the semaphore is incremented by the software. When the semaphore goes to non-zero and the channel is in its idle state, then it uses the value in the APBH_CHn_NXTCMDAR register (next command address register) to fetch a pointer to the next command to process.

NOTE

This is a double indirect case. This method allows the software to append to a running command list under the protection of the counting semaphore.

To start processing the first time, software creates the command list to be processed. It writes the address of the first command into the APBH_CHn_NXTCMDAR register, and then writes 1 to the counting semaphore in APBH_CHn_SEMA. The DMA channel loads APBH_CHn_CURCMDAR register and then enters the normal state machine processing for the next command. When the software writes a value to the counting semaphore, it is added to the semaphore count by hardware, protecting the case where both hardware and software are trying to change the semaphore on the same clock edge.

Software can examine the value of APBH_CHn_CURCMDAR at any time to determine the location of the command structure currently being processed.

9.5.4 NAND Read Status Polling Example

The following figure shows a more complicated scenario.

This subset of a NAND device workload shows that the first two command structures are used during the data-write phase of an NAND device write operation (CLE and ALE transfers omitted for clarity).

- After writing the data, one must wait until the NAND device status register indicates that the write charge has been transferred. This is built into the workload using a check status command in the NAND in a loop created from the next two DMA command structures.
- The NO_DMA_TRANSFER command is shown here performing the read check, followed by a DMA_SENSE command to branch the DMA command structure list, based on the status of a bit in the external NAND device.

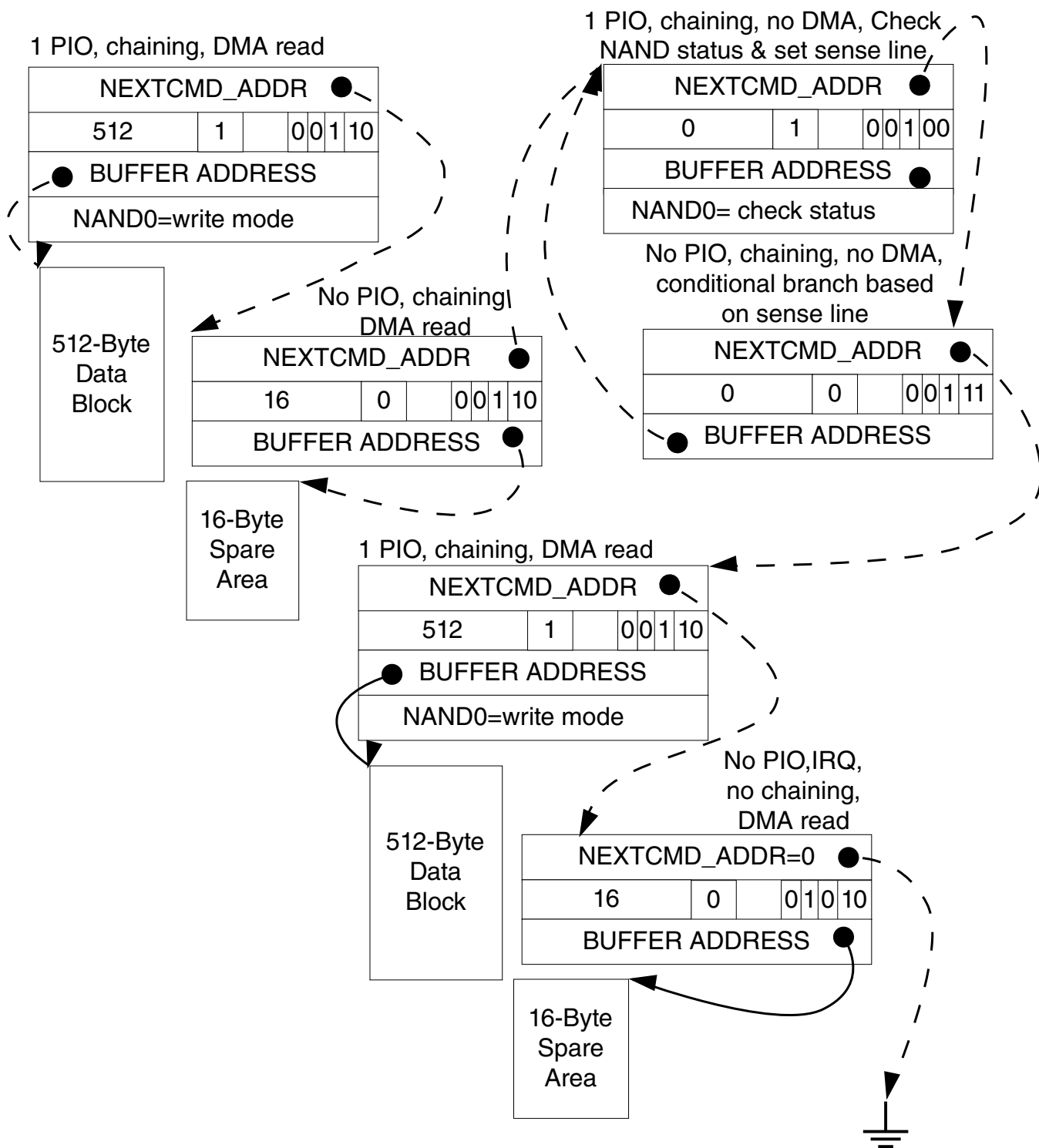


Figure 9-9. AHB-to-APBH Bridge DMA NAND Read Status Polling with DMA Sense Command

The example in the above figure shows the workload continuing immediately to the next NAND page transfer. However, one could perform a second sense operation to see if an error has occurred after the write. One could then point the sense command alternate branch at a NO_DMA_XFER command with the interrupt bit set. If the CHAIN bit is not

set on this failure branch, then the Arm platform is interrupted immediately, and the channel process is also immediately terminated in the presence of a workload-detected NAND error bit.

Note that each word of the three-word DMA command structure corresponds to a PIO register of the DMA that is accessible on the APBH bus. Normally, the DMA copies the next command structure onto these registers for processing at the start of each command by following the value of the pointer previously loaded into the NEXTCMD_ADDR register.

To start DMA processing for the first command, initialize the PIO registers of the desired channel, as follows:

- First, load the next command address register with a pointer to the first command to be loaded.
- Then, write 1 to the counting semaphore register. This causes the DMA to schedule the targeted channel for the DMA command structure load, just as if it had finished its previous command.

9.5.5 APBH Memory Map/Register Definition

APBH Hardware Register Format Summary

APBH memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3300_0000	AHB to APBH Bridge Control and Status Register 0 (APBH_CTRL0)	32	R/W	E000_0000h	9.5.5.1/2403
3300_0004	AHB to APBH Bridge Control and Status Register 0 (APBH_CTRL0_SET)	32	R/W	E000_0000h	9.5.5.1/2403
3300_0008	AHB to APBH Bridge Control and Status Register 0 (APBH_CTRL0_CLR)	32	R/W	E000_0000h	9.5.5.1/2403
3300_000C	AHB to APBH Bridge Control and Status Register 0 (APBH_CTRL0_TOG)	32	R/W	E000_0000h	9.5.5.1/2403
3300_0010	AHB to APBH Bridge Control and Status Register 1 (APBH_CTRL1)	32	R/W	0000_0000h	9.5.5.2/2405
3300_0014	AHB to APBH Bridge Control and Status Register 1 (APBH_CTRL1_SET)	32	R/W	0000_0000h	9.5.5.2/2405
3300_0018	AHB to APBH Bridge Control and Status Register 1 (APBH_CTRL1_CLR)	32	R/W	0000_0000h	9.5.5.2/2405

Table continues on the next page...

APBH memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3300_001C	AHB to APBH Bridge Control and Status Register 1 (APBH_CTRL1_TOG)	32	R/W	0000_0000h	9.5.5.2/2405
3300_0020	AHB to APBH Bridge Control and Status Register 2 (APBH_CTRL2)	32	R/W	0000_0000h	9.5.5.3/2408
3300_0024	AHB to APBH Bridge Control and Status Register 2 (APBH_CTRL2_SET)	32	R/W	0000_0000h	9.5.5.3/2408
3300_0028	AHB to APBH Bridge Control and Status Register 2 (APBH_CTRL2_CLR)	32	R/W	0000_0000h	9.5.5.3/2408
3300_002C	AHB to APBH Bridge Control and Status Register 2 (APBH_CTRL2_TOG)	32	R/W	0000_0000h	9.5.5.3/2408
3300_0030	AHB to APBH Bridge Channel Register (APBH_CHANNEL_CTRL)	32	R/W	0000_0000h	9.5.5.4/2413
3300_0034	AHB to APBH Bridge Channel Register (APBH_CHANNEL_CTRL_SET)	32	R/W	0000_0000h	9.5.5.4/2413
3300_0038	AHB to APBH Bridge Channel Register (APBH_CHANNEL_CTRL_CLR)	32	R/W	0000_0000h	9.5.5.4/2413
3300_003C	AHB to APBH Bridge Channel Register (APBH_CHANNEL_CTRL_TOG)	32	R/W	0000_0000h	9.5.5.4/2413
3300_0040	AHB to APBH DMA Device Assignment Register (APBH_DEVSEL)	32	R	0000_0000h	9.5.5.5/2414
3300_0050	AHB to APBH DMA burst size (APBH_DMA_BURST_SIZE)	32	R/W	0055_5555h	9.5.5.6/2415
3300_0060	AHB to APBH DMA Debug Register (APBH_DEBUG)	32	R/W	0000_0000h	9.5.5.7/2416
3300_0100	APBH DMA Channel n Current Command Address Register (APBH_CH0_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_0110	APBH DMA Channel n Next Command Address Register (APBH_CH0_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_0120	APBH DMA Channel n Command Register (APBH_CH0_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_0130	APBH DMA Channel n Buffer Address Register (APBH_CH0_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_0140	APBH DMA Channel n Semaphore Register (APBH_CH0_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_0150	AHB to APBH DMA Channel n Debug Information (APBH_CH0_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_0160	AHB to APBH DMA Channel n Debug Information (APBH_CH0_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_0170	APBH DMA Channel n Current Command Address Register (APBH_CH1_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_0180	APBH DMA Channel n Next Command Address Register (APBH_CH1_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_0190	APBH DMA Channel n Command Register (APBH_CH1_CMD)	32	R	0000_0000h	9.5.5.10/2418

Table continues on the next page...

APBH memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3300_01A0	APBH DMA Channel n Buffer Address Register (APBH_CH1_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_01B0	APBH DMA Channel n Semaphore Register (APBH_CH1_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_01C0	AHB to APBH DMA Channel n Debug Information (APBH_CH1_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_01D0	AHB to APBH DMA Channel n Debug Information (APBH_CH1_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_01E0	APBH DMA Channel n Current Command Address Register (APBH_CH2_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_01F0	APBH DMA Channel n Next Command Address Register (APBH_CH2_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_0200	APBH DMA Channel n Command Register (APBH_CH2_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_0210	APBH DMA Channel n Buffer Address Register (APBH_CH2_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_0220	APBH DMA Channel n Semaphore Register (APBH_CH2_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_0230	AHB to APBH DMA Channel n Debug Information (APBH_CH2_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_0240	AHB to APBH DMA Channel n Debug Information (APBH_CH2_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_0250	APBH DMA Channel n Current Command Address Register (APBH_CH3_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_0260	APBH DMA Channel n Next Command Address Register (APBH_CH3_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_0270	APBH DMA Channel n Command Register (APBH_CH3_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_0280	APBH DMA Channel n Buffer Address Register (APBH_CH3_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_0290	APBH DMA Channel n Semaphore Register (APBH_CH3_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_02A0	AHB to APBH DMA Channel n Debug Information (APBH_CH3_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_02B0	AHB to APBH DMA Channel n Debug Information (APBH_CH3_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_02C0	APBH DMA Channel n Current Command Address Register (APBH_CH4_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_02D0	APBH DMA Channel n Next Command Address Register (APBH_CH4_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_02E0	APBH DMA Channel n Command Register (APBH_CH4_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_02F0	APBH DMA Channel n Buffer Address Register (APBH_CH4_BAR)	32	R	0000_0000h	9.5.5.11/2420

Table continues on the next page...

APBH memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3300_0300	APBH DMA Channel n Semaphore Register (APBH_CH4_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_0310	AHB to APBH DMA Channel n Debug Information (APBH_CH4_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_0320	AHB to APBH DMA Channel n Debug Information (APBH_CH4_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_0330	APBH DMA Channel n Current Command Address Register (APBH_CH5_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_0340	APBH DMA Channel n Next Command Address Register (APBH_CH5_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_0350	APBH DMA Channel n Command Register (APBH_CH5_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_0360	APBH DMA Channel n Buffer Address Register (APBH_CH5_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_0370	APBH DMA Channel n Semaphore Register (APBH_CH5_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_0380	AHB to APBH DMA Channel n Debug Information (APBH_CH5_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_0390	AHB to APBH DMA Channel n Debug Information (APBH_CH5_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_03A0	APBH DMA Channel n Current Command Address Register (APBH_CH6_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_03B0	APBH DMA Channel n Next Command Address Register (APBH_CH6_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_03C0	APBH DMA Channel n Command Register (APBH_CH6_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_03D0	APBH DMA Channel n Buffer Address Register (APBH_CH6_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_03E0	APBH DMA Channel n Semaphore Register (APBH_CH6_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_03F0	AHB to APBH DMA Channel n Debug Information (APBH_CH6_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_0400	AHB to APBH DMA Channel n Debug Information (APBH_CH6_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_0410	APBH DMA Channel n Current Command Address Register (APBH_CH7_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_0420	APBH DMA Channel n Next Command Address Register (APBH_CH7_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_0430	APBH DMA Channel n Command Register (APBH_CH7_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_0440	APBH DMA Channel n Buffer Address Register (APBH_CH7_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_0450	APBH DMA Channel n Semaphore Register (APBH_CH7_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421

Table continues on the next page...

APBH memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3300_0460	AHB to APBH DMA Channel n Debug Information (APBH_CH7_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_0470	AHB to APBH DMA Channel n Debug Information (APBH_CH7_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_0480	APBH DMA Channel n Current Command Address Register (APBH_CH8_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_0490	APBH DMA Channel n Next Command Address Register (APBH_CH8_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_04A0	APBH DMA Channel n Command Register (APBH_CH8_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_04B0	APBH DMA Channel n Buffer Address Register (APBH_CH8_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_04C0	APBH DMA Channel n Semaphore Register (APBH_CH8_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_04D0	AHB to APBH DMA Channel n Debug Information (APBH_CH8_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_04E0	AHB to APBH DMA Channel n Debug Information (APBH_CH8_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_04F0	APBH DMA Channel n Current Command Address Register (APBH_CH9_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_0500	APBH DMA Channel n Next Command Address Register (APBH_CH9_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_0510	APBH DMA Channel n Command Register (APBH_CH9_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_0520	APBH DMA Channel n Buffer Address Register (APBH_CH9_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_0530	APBH DMA Channel n Semaphore Register (APBH_CH9_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_0540	AHB to APBH DMA Channel n Debug Information (APBH_CH9_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_0550	AHB to APBH DMA Channel n Debug Information (APBH_CH9_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_0560	APBH DMA Channel n Current Command Address Register (APBH_CH10_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_0570	APBH DMA Channel n Next Command Address Register (APBH_CH10_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_0580	APBH DMA Channel n Command Register (APBH_CH10_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_0590	APBH DMA Channel n Buffer Address Register (APBH_CH10_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_05A0	APBH DMA Channel n Semaphore Register (APBH_CH10_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_05B0	AHB to APBH DMA Channel n Debug Information (APBH_CH10_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422

Table continues on the next page...

APBH memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3300_05C0	AHB to APBH DMA Channel n Debug Information (APBH_CH10_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_05D0	APBH DMA Channel n Current Command Address Register (APBH_CH11_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_05E0	APBH DMA Channel n Next Command Address Register (APBH_CH11_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_05F0	APBH DMA Channel n Command Register (APBH_CH11_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_0600	APBH DMA Channel n Buffer Address Register (APBH_CH11_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_0610	APBH DMA Channel n Semaphore Register (APBH_CH11_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_0620	AHB to APBH DMA Channel n Debug Information (APBH_CH11_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_0630	AHB to APBH DMA Channel n Debug Information (APBH_CH11_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_0640	APBH DMA Channel n Current Command Address Register (APBH_CH12_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_0650	APBH DMA Channel n Next Command Address Register (APBH_CH12_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_0660	APBH DMA Channel n Command Register (APBH_CH12_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_0670	APBH DMA Channel n Buffer Address Register (APBH_CH12_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_0680	APBH DMA Channel n Semaphore Register (APBH_CH12_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_0690	AHB to APBH DMA Channel n Debug Information (APBH_CH12_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_06A0	AHB to APBH DMA Channel n Debug Information (APBH_CH12_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_06B0	APBH DMA Channel n Current Command Address Register (APBH_CH13_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_06C0	APBH DMA Channel n Next Command Address Register (APBH_CH13_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_06D0	APBH DMA Channel n Command Register (APBH_CH13_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_06E0	APBH DMA Channel n Buffer Address Register (APBH_CH13_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_06F0	APBH DMA Channel n Semaphore Register (APBH_CH13_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_0700	AHB to APBH DMA Channel n Debug Information (APBH_CH13_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_0710	AHB to APBH DMA Channel n Debug Information (APBH_CH13_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425

Table continues on the next page...

APBH memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3300_0720	APBH DMA Channel n Current Command Address Register (APBH_CH14_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_0730	APBH DMA Channel n Next Command Address Register (APBH_CH14_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_0740	APBH DMA Channel n Command Register (APBH_CH14_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_0750	APBH DMA Channel n Buffer Address Register (APBH_CH14_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_0760	APBH DMA Channel n Semaphore Register (APBH_CH14_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_0770	AHB to APBH DMA Channel n Debug Information (APBH_CH14_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_0780	AHB to APBH DMA Channel n Debug Information (APBH_CH14_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_0790	APBH DMA Channel n Current Command Address Register (APBH_CH15_CURCMDAR)	32	R	0000_0000h	9.5.5.8/2417
3300_07A0	APBH DMA Channel n Next Command Address Register (APBH_CH15_NXTCMDAR)	32	R/W	0000_0000h	9.5.5.9/2418
3300_07B0	APBH DMA Channel n Command Register (APBH_CH15_CMD)	32	R	0000_0000h	9.5.5.10/2418
3300_07C0	APBH DMA Channel n Buffer Address Register (APBH_CH15_BAR)	32	R	0000_0000h	9.5.5.11/2420
3300_07D0	APBH DMA Channel n Semaphore Register (APBH_CH15_SEMA)	32	R/W	0000_0000h	9.5.5.12/2421
3300_07E0	AHB to APBH DMA Channel n Debug Information (APBH_CH15_DEBUG1)	32	R	00A0_0000h	9.5.5.13/2422
3300_07F0	AHB to APBH DMA Channel n Debug Information (APBH_CH15_DEBUG2)	32	R	0000_0000h	9.5.5.14/2425
3300_0800	APBH Bridge Version Register (APBH_VERSION)	32	R	0301_0000h	9.5.5.15/2425

9.5.5.1 AHB to APBH Bridge Control and Status Register 0 (APBH_CTRL0n)

The APBH CTRL 0 provides overall control of the AHB to APBH bridge and DMA.

This register contains module softreset, clock gating, channel clock gating/freeze bits.

AHB-to-APBH Bridge with DMA (APBH-Bridge-DMA)

Address: 3300_0000h base + 0h offset + (4d × i), where i=0d to 3d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	SFTRST	CLKGATE	AHB_BURST8_EN	APB_BURST_EN	Reserved											
W																
Reset	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CLKGATE_CHANNEL															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

APBH_CTRL0n field descriptions

Field	Description
31 SFTRST	Set this bit to zero to enable normal APBH DMA operation. Set this bit to one (default) to disable clocking with the APBH DMA and hold it in its reset (lowest power) state. This bit can be turned on and then off to reset the APBH DMA block to its default state.
30 CLKGATE	This bit must be set to zero for normal operation. When set to one it gates off the clocks to the block.
29 AHB_BURST8_EN	Set this bit to one (default) to enable AHB 8-beat burst. Set to zero to disable 8-beat burst on AHB interface.
28 APB_BURST_EN	Set this bit to one to enable apb master do a continous transfers when a device request a burst dma. Set to zero will treat a burst dma request as 4/8 individual requests.
27–16 RSVD0	This field is reserved. Reserved, always set to zero.
CLKGATE_CHANNEL	These bits must be set to zero for normal operation of each channel. When set to one they gate off the individual clocks to the channels. 0x0001 NAND0 — 0x0002 NAND1 — 0x0004 NAND2 — 0x0008 NAND3 — 0x0010 NAND4 — 0x0020 NAND5 — 0x0040 NAND6 — 0x0080 NAND7 — 0x0100 SSP —

9.5.5.2 AHB to APBH Bridge Control and Status Register 1 (APBH_CTRL1n)

The APBH CTRL one provides overall control of the interrupts generated by the AHB to APBH DMA. This register contains the per channel interrupt status bits and the per channel interrupt enable bits. Each channel has a dedicated interrupt vector in the vectored interrupt controller.

EXAMPLE

```
BF_WR(APBH_CTRL1, CH5_CMDCMPLT_IRQ, 0); // use bitfield write macro
BF_APBH_CTRL1.CH5_CMDCMPLT_IRQ = 0;    // or, assign to register

struct's bitfield
```

Address: 3300_0000h base + 10h offset + (4d × i), where i=0d to 3d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	CH15_CMDCMPLT_IRQ_EN	CH14_CMDCMPLT_IRQ_EN	CH13_CMDCMPLT_IRQ_EN	CH12_CMDCMPLT_IRQ_EN	CH11_CMDCMPLT_IRQ_EN	CH10_CMDCMPLT_IRQ_EN	CH9_CMDCMPLT_IRQ_EN	CH8_CMDCMPLT_IRQ_EN	CH7_CMDCMPLT_IRQ_EN	CH6_CMDCMPLT_IRQ_EN	CH5_CMDCMPLT_IRQ_EN	CH4_CMDCMPLT_IRQ_EN	CH3_CMDCMPLT_IRQ_EN	CH2_CMDCMPLT_IRQ_EN	CH1_CMDCMPLT_IRQ_EN	CH0_CMDCMPLT_IRQ_EN
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CH15_CMDCMPLT_IRQ	CH14_CMDCMPLT_IRQ	CH13_CMDCMPLT_IRQ	CH12_CMDCMPLT_IRQ	CH11_CMDCMPLT_IRQ	CH10_CMDCMPLT_IRQ	CH9_CMDCMPLT_IRQ	CH8_CMDCMPLT_IRQ	CH7_CMDCMPLT_IRQ	CH6_CMDCMPLT_IRQ	CH5_CMDCMPLT_IRQ	CH4_CMDCMPLT_IRQ	CH3_CMDCMPLT_IRQ	CH2_CMDCMPLT_IRQ	CH1_CMDCMPLT_IRQ	CH0_CMDCMPLT_IRQ
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

APBH_CTRL1n field descriptions

Field	Description
31 CH15_CMDCMPLT_IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 15.
30 CH14_CMDCMPLT_IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 14.

Table continues on the next page...

APBH_CTRL1n field descriptions (continued)

Field	Description
29 CH13_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 13.
28 CH12_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 12.
27 CH11_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 11.
26 CH10_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 10.
25 CH9_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 9.
24 CH8_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 8.
23 CH7_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 7.
22 CH6_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 6.
21 CH5_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 5.
20 CH4_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 4.
19 CH3_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 3.
18 CH2_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 2.

Table continues on the next page...

APBH_CTRL1n field descriptions (continued)

Field	Description
17 CH1_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 1.
16 CH0_ CMDCMPLT_ IRQ_EN	Setting this bit enables the generation of an interrupt request for APBH DMA channel 0.
15 CH15_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA Channel 15. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
14 CH14_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA Channel 14. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
13 CH13_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA Channel 13. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
12 CH12_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA Channel 12. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
11 CH11_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA Channel 11. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
10 CH10_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA Channel 10. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
9 CH9_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA Channel 9. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
8 CH8_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA Channel 8. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
7 CH7_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA channel 7. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
6 CH6_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA channel 6. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.

Table continues on the next page...

APBH_CTRL1n field descriptions (continued)

Field	Description
5 CH5_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA channel 5. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
4 CH4_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA channel 4. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
3 CH3_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA channel 3. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
2 CH2_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA channel 2. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
1 CH1_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA channel 1. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.
0 CH0_ CMDCMPLT_ IRQ	Interrupt request status bit for APBH DMA channel 0. This sticky bit is set by DMA hardware and reset by software. It is ANDed with its corresponding enable bit to generate an interrupt.

9.5.5.3 AHB to APBH Bridge Control and Status Register 2 (APBH_CTRL2n)

The APBH CTRL 2 provides channel error interrupts generated by the AHB to APBH DMA. This register contains the per channel interrupt status bits and the per channel interrupt enable bits. Each channel has a dedicated interrupt vector in the vectored interrupt controller.

EXAMPLE

```

struct's bitfield
BF_WR(APBH_CTRL1, CH5_CMDCMPLT_IRQ, 0); // use bitfield write macro
BF_APBH_CTRL1.CH5_CMDCMPLT_IRQ = 0;      // or, assign to register

```

Address: 3300_0000h base + 20h offset + (4d × i), where i=0d to 3d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	CH15_ERROR_STATUS	CH14_ERROR_STATUS	CH13_ERROR_STATUS	CH12_ERROR_STATUS	CH11_ERROR_STATUS	CH10_ERROR_STATUS	CH9_ERROR_STATUS	CH8_ERROR_STATUS	CH7_ERROR_STATUS	CH6_ERROR_STATUS	CH5_ERROR_STATUS	CH4_ERROR_STATUS	CH3_ERROR_STATUS	CH2_ERROR_STATUS	CH1_ERROR_STATUS	CH0_ERROR_STATUS
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CH15_ERROR_IRQ	CH14_ERROR_IRQ	CH13_ERROR_IRQ	CH12_ERROR_IRQ	CH11_ERROR_IRQ	CH10_ERROR_IRQ	CH9_ERROR_IRQ	CH8_ERROR_IRQ	CH7_ERROR_IRQ	CH6_ERROR_IRQ	CH5_ERROR_IRQ	CH4_ERROR_IRQ	CH3_ERROR_IRQ	CH2_ERROR_IRQ	CH1_ERROR_IRQ	CH0_ERROR_IRQ
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

APBH_CTRL2n field descriptions

Field	Description
31 CH15_ERROR_STATUS	Error status bit for APBH DMA Channel 15. Valid when corresponding Error IRQ is set. 1 - AHB bus error

Table continues on the next page...

APBH_CTRL2n field descriptions (continued)

Field	Description
	0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
30 CH14_ERROR_STATUS	Error status bit for APBH DMA Channel 14. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
29 CH13_ERROR_STATUS	Error status bit for APBH DMA Channel 13. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
28 CH12_ERROR_STATUS	Error status bit for APBH DMA Channel 12. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
27 CH11_ERROR_STATUS	Error status bit for APBH DMA Channel 11. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
26 CH10_ERROR_STATUS	Error status bit for APBH DMA Channel 10. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
25 CH9_ERROR_STATUS	Error status bit for APBH DMA Channel 9. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
24 CH8_ERROR_STATUS	Error status bit for APBH DMA Channel 8. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination.

Table continues on the next page...

APBH_CTRL2n field descriptions (continued)

Field	Description
	0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
23 CH7_ERROR_ STATUS	Error status bit for APBX DMA Channel 7. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
22 CH6_ERROR_ STATUS	Error status bit for APBX DMA Channel 6. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
21 CH5_ERROR_ STATUS	Error status bit for APBX DMA Channel 5. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
20 CH4_ERROR_ STATUS	Error status bit for APBX DMA Channel 4. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
19 CH3_ERROR_ STATUS	Error status bit for APBX DMA Channel 3. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
18 CH2_ERROR_ STATUS	Error status bit for APBX DMA Channel 2. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
17 CH1_ERROR_ STATUS	Error status bit for APBX DMA Channel 1. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.

Table continues on the next page...

APBH_CTRL2n field descriptions (continued)

Field	Description
16 CH0_ERROR_STATUS	Error status bit for APBX DMA Channel 0. Valid when corresponding Error IRQ is set. 1 - AHB bus error 0 - channel early termination. 0x0 TERMINATION — An early termination from the device causes error IRQ. 0x1 BUS_ERROR — An AHB bus error causes error IRQ.
15 CH15_ERROR_IRQ	Error interrupt status bit for APBH DMA Channel 15. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
14 CH14_ERROR_IRQ	Error interrupt status bit for APBH DMA Channel 14. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
13 CH13_ERROR_IRQ	Error interrupt status bit for APBH DMA Channel 13. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
12 CH12_ERROR_IRQ	Error interrupt status bit for APBH DMA Channel 12. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
11 CH11_ERROR_IRQ	Error interrupt status bit for APBH DMA Channel 11. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
10 CH10_ERROR_IRQ	Error interrupt status bit for APBH DMA Channel 10. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
9 CH9_ERROR_IRQ	Error interrupt status bit for APBH DMA Channel 9. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
8 CH8_ERROR_IRQ	Error interrupt status bit for APBH DMA Channel 8. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
7 CH7_ERROR_IRQ	Error interrupt status bit for APBX DMA Channel 7. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
6 CH6_ERROR_IRQ	Error interrupt status bit for APBX DMA Channel 6. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
5 CH5_ERROR_IRQ	Error interrupt status bit for APBX DMA Channel 5. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
4 CH4_ERROR_IRQ	Error interrupt status bit for APBX DMA Channel 4. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
3 CH3_ERROR_IRQ	Error interrupt status bit for APBX DMA Channel 3. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
2 CH2_ERROR_IRQ	Error interrupt status bit for APBX DMA Channel 2. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.

Table continues on the next page...

APBH_CTRL2n field descriptions (continued)

Field	Description
1 CH1_ERROR_IRQ	Error interrupt status bit for APBX DMA Channel 1. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.
0 CH0_ERROR_IRQ	Error interrupt status bit for APBX DMA Channel 0. This sticky bit is set by DMA hardware and reset by software. It is ORed with the corresponding cmdcmplt irq to generate an irq to Arm.

9.5.5.4 AHB to APBH Bridge Channel Register (APBH_CHANNEL_CTRLn)

The APBH CHANNEL CTRL provides reset/freeze control of each DMA channel. This register contains individual channel reset/freeze bits.

Address: 3300_0000h base + 30h offset + (4d × i), where i=0d to 3d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	RESET_CHANNEL																FREEZE_CHANNEL															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

APBH_CHANNEL_CTRLn field descriptions

Field	Description
31–16 RESET_CHANNEL	Setting a bit in this field causes the DMA controller to take the corresponding channel through its reset state. The bit is reset after the channel resources are cleared. 0x0001 NAND0 — 0x0002 NAND1 — 0x0004 NAND2 — 0x0008 NAND3 — 0x0010 NAND4 — 0x0020 NAND5 — 0x0040 NAND6 — 0x0080 NAND7 — 0x0100 SSP —
FREEZE_CHANNEL	Setting a bit in this field will freeze the DMA channel associated with it. This field is a direct input to the DMA channel arbiter. When frozen, the channel is denied access to the central DMA resources. 0x0001 NAND0 — 0x0002 NAND1 — 0x0004 NAND2 — 0x0008 NAND3 — 0x0010 NAND4 — 0x0020 NAND5 — 0x0040 NAND6 —

Table continues on the next page...

APBH_CHANNEL_CTRL n field descriptions (continued)

Field	Description
0x0080 NAND7 —	
0x0100 SSP —	

9.5.5.5 AHB to APBH DMA Device Assignment Register (APBH_DEVSEL)

This register allows reassignment of the APBH device connected to the DMA Channels.

In this chip, APBH DMA channel resource is enough for high speed peripherals, so this register is of no use and reserved.

Address: 3300_0000h base + 40h offset = 3300_0040h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

APBH_DEVSEL field descriptions

Field	Description
31–30 CH15	This field is reserved. Reserved.
29–28 CH14	This field is reserved. Reserved.
27–26 CH13	This field is reserved. Reserved.
25–24 CH12	This field is reserved. Reserved.
23–22 CH11	This field is reserved. Reserved.
21–20 CH10	This field is reserved. Reserved.
19–18 CH9	This field is reserved. Reserved.
17–16 CH8	This field is reserved. Reserved.
15–14 CH7	This field is reserved. Reserved.
13–12 CH6	This field is reserved. Reserved.

Table continues on the next page...

APBH_DEVSEL field descriptions (continued)

Field	Description
11–10 CH5	This field is reserved. Reserved.
9–8 CH4	This field is reserved. Reserved.
7–6 CH3	This field is reserved. Reserved.
5–4 CH2	This field is reserved. Reserved.
3–2 CH1	This field is reserved. Reserved.
CH0	This field is reserved. Reserved.

9.5.5.6 AHB to APBH DMA burst size (APBH_DMA_BURST_SIZE)

This register programs the apbh burst size of the APBH DMA devices when a DMA burst request is issued.

This register provides a mechanism for assigning the device.

Address: 3300_0000h base + 50h offset = 3300_0050h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W																
	Reserved		Reserved		Reserved		Reserved		Reserved		Reserved		Reserved		CH8	
Reset	0	0	0	0	0	0	0	0	0	1	0	1	0	1	0	1
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
	CH7		CH6		CH5		CH4		CH3		CH2		CH1		CH0	
Reset	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

APBH_DMA_BURST_SIZE field descriptions

Field	Description
31–30 CH15	This field is reserved. Reserved.
29–28 CH14	This field is reserved. Reserved.
27–26 CH13	This field is reserved. Reserved.
25–24 CH12	This field is reserved. Reserved.
23–22 CH11	This field is reserved. Reserved.

Table continues on the next page...

APBH_DMA_BURST_SIZE field descriptions (continued)

Field	Description
21–20 CH10	This field is reserved. Reserved.
19–18 CH9	This field is reserved. Reserved.
17–16 CH8	DMA burst size for SSP. 0x0 BURST0 — 0x1 BURST4 — 0x2 BURST8 —
15–14 CH7	DMA burst size for GPPI channel 7. Do not change. GPPI only support burst size 4.
13–12 CH6	DMA burst size for GPPI channel 6. Do not change. GPPI only support burst size 4.
11–10 CH5	DMA burst size for GPPI channel 5. Do not change. GPPI only support burst size 4.
9–8 CH4	DMA burst size for GPPI channel 4. Do not change. GPPI only support burst size 4.
7–6 CH3	DMA burst size for GPPI channel 3. Do not change. GPPI only support burst size 4.
5–4 CH2	DMA burst size for GPPI channel 2. Do not change. GPPI only support burst size 4.
3–2 CH1	DMA burst size for GPPI channel 1. Do not change. GPPI only support burst size 4.
CH0	DMA burst size for GPPI channel 0. Do not change. GPPI only support burst size 4.

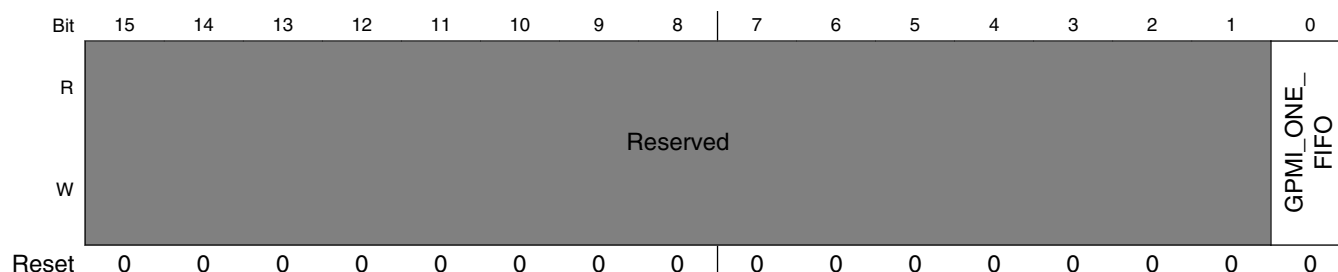
9.5.5.7 AHB to APBH DMA Debug Register (APBH_DEBUG)

This register is for debug purpose.

The debug register is for internal use only. Not recommend for customer usage.

Address: 3300_0000h base + 60h offset = 3300_0060h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

**APBH_DEBUG field descriptions**

Field	Description
31–1 -	This field is reserved. Reserved, always set to zero.
0 GPMI_ONE_FIFO	Set to One and the 8 GPMI channels will share the DMA FIFO, and when set to zero, the 8 GPMI channels will use its own DMA FIFO.

9.5.5.8 APBH DMA Channel n Current Command Address Register (APBH_CHn_CURCMDAR)

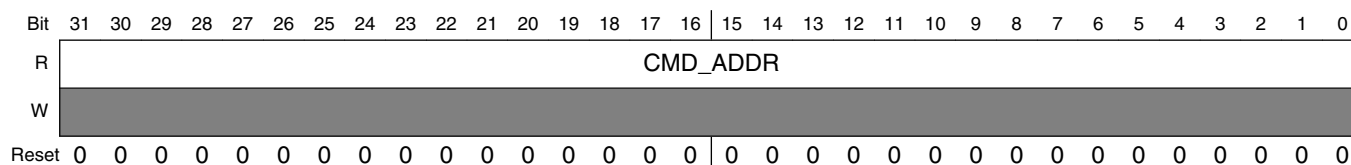
The APBH DMA channel n current command address register points to the multiword command that is currently being executed. Commands are threaded on the command address.

APBH DMA Channel n is controlled by a variable sized command structure. This register points to the command structure currently being executed.

EXAMPLE

```
pCurCmd = (apbh_chn_cmd_t *) APBH_CHn_CURCMDAR_RD(0);           // read the whole
register, since there is only one field
pCurCmd = (apbh_chn_cmd_t *) BF_RDn(APBH_CHn_CURCMDAR, 0, CMD_ADDR); // or, use multi-
register bitfield read macro
pCurCmd = (apbh_chn_cmd_t *) APBH_CHn_CURCMDAR(0).CMD_ADDR;     // or, assign from
bitfield of indexed register's struct
```

Address: 3300_0000h base + 100h offset + (112d × i), where i=0d to 15d

**APBH_CHn_CURCMDAR field descriptions**

Field	Description
CMD_ADDR	Pointer to command structure currently being processed for channel n.

9.5.5.9 APBH DMA Channel n Next Command Address Register (APBH_CHn_NXTCMDAR)

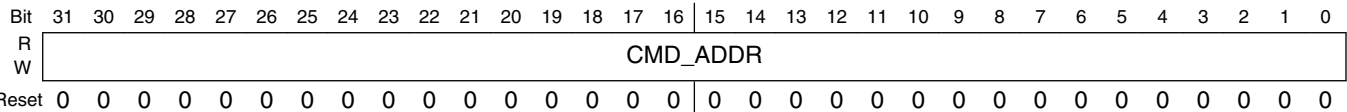
The APBH DMA Channel n Next Command Address register contains the address of the next multiword command to be executed. Commands are threaded on the command address. Set CHAIN to 1 in the DMA command word to process command lists.

APBH DMA Channel n is controlled by a variable sized command structure. Software loads this register with the address of the first command structure to process and increments the Channel n semaphore to start processing. This register points to the next command structure to be executed when the current command is completed.

EXAMPLE

```
APBH_CHn_NXTCMDAR_WR(0, (reg32_t) pCommandTwoStructure);           // write the entire
register, since there is only one field
BF_WRn(APBH_CHn_NXTCMDAR, 0, (reg32_t) pCommandTwoStructure);      // or, use multi-
register bitfield write macro
APBH_CHn_NXTCMDAR(0).CMD_ADDR = (reg32_t) pCommandTwoStructure;    // or, assign to bitfield
of indexed register's struct
```

Address: 3300_0000h base + 110h offset + (112d × i), where i=0d to 15d



APBH_CHn_NXTCMDAR field descriptions

Field	Description
CMD_ADDR	Pointer to next command structure for channel n.

9.5.5.10 APBH DMA Channel n Command Register (APBH_CHn_CMD)

The APBH DMA Channel n command register specifies the DMA transaction to perform for the current command chain item.

The command register controls the overall operation of each DMA command for this channel. It includes the number of bytes to transfer to or from the device, the number of APB PIO command words included with this command structure, whether to interrupt at command completion, whether to chain an additional command to the end of this one and whether this transfer is a read or write DMA transfer.

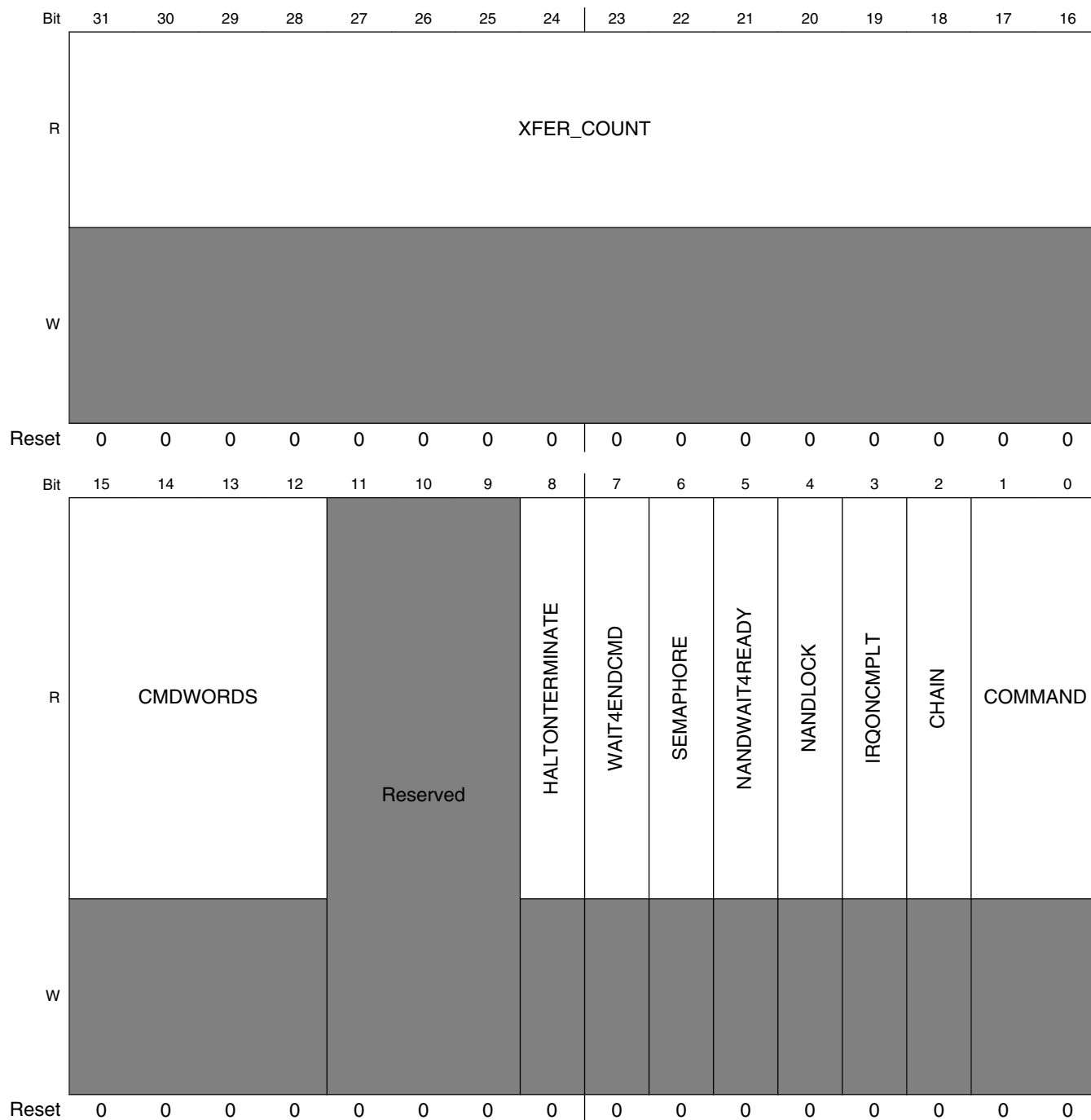
EXAMPLE

```

apbh_chn_cmd_t dma_cmd;
dma_cmd.XFER_COUNT = 512; // transfer 512 bytes
dma_cmd.COMMAND = BV_APBH_CHn_CMD_COMMAND__DMA_WRITE; // transfer to system memory from
peripheral device
dma_cmd.CHAIN = 1; // chain an additional command
structure on to the list
dma_cmd.IRQONCMPLT = 1; // generate an interrupt on
completion of this command structure

```

Address: 3300_0000h base + 120h offset + (112d × i), where i=0d to 15d



APBH_CHn_CMD field descriptions

Field	Description
31–16 XFER_COUNT	This field indicates the number of bytes to transfer to or from the appropriate PIO register in the GPMIO device. A value of 0 indicates a 64 KBytes transfer.
15–12 CMDWORDS	This field indicates the number of command words to send to the GPMIO, starting with the base PIO address of the GPMIO control register and incrementing from there. Zero means transfer NO command words
11–9 -	This field is reserved. Reserved, always set to zero.
8 HALTONTERMINATE	A value of one indicates that the channel will immediately terminate the current descriptor and halt the DMA channel if a terminate signal is set. A value of 0 will still cause an immediate terminate of the channel if the terminate signal is set, but the channel will continue as if the count had been exhausted, meaning it will honor IRQONCMPLT, CHAIN, SEMAPHORE, and WAIT4ENDCMD.
7 WAIT4ENDCMD	A value of one indicates that the channel will wait for the end of command signal to be sent from the APBH device to the DMA before starting the next DMA command.
6 SEMAPHORE	A value of one indicates that the channel will decrement its semaphore at the completion of the current command structure. If the semaphore decrements to zero, then this channel stalls until software increments it again.
5 NANDWAIT4READY	A value of one indicates that the NAND DMA channel will wait until the NAND device reports "ready" before executing the command. It is ignored for non-NAND DMA channels.
4 NANDLOCK	A value of one indicates that the NAND DMA channel will remain "locked" in the arbiter at the expense of other NAND DMA channels. It is ignored for non-NAND DMA channels.
3 IRQONCMPLT	A value of one indicates that the channel will cause the interrupt status bit to be set upon completion of the current command, i.e. after the DMA transfer is complete.
2 CHAIN	A value of one indicates that another command is chained onto the end of the current command structure. At the completion of the current command, this channel will follow the pointer in APBH_CHn_CMDAR to find the next command.
COMMAND	This bitfield indicates the type of current command: 0x0 NO_DMA_XFER — Perform any requested PIO word transfers but terminate command before any DMA transfer. 0x1 DMA_WRITE — Perform any requested PIO word transfers and then perform a DMA transfer from the peripheral for the specified number of bytes. 0x2 DMA_READ — Perform any requested PIO word transfers and then perform a DMA transfer to the peripheral for the specified number of bytes. 0x3 DMA_SENSE — Perform any requested PIO word transfers and then perform a conditional branch to the next chained device. Follow the NEXCMD_ADDR pointer if the peripheral sense is true. Follow the BUFFER_ADDRESS as a chain pointer if the peripheral sense line is false.

9.5.5.11 APBH DMA Channel n Buffer Address Register (APBH_CHn_BAR)

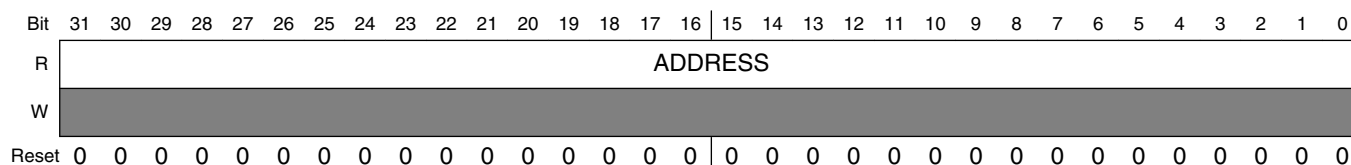
The APBH DMA Channel n buffer address register contains a pointer to the data buffer for the transfer. For immediate forms, the data is taken from this register. This is a byte address which means transfers can start on any byte boundary.

This register holds a pointer to the data buffer in system memory. After the command values have been read into the DMA controller and the device controlled by this channel, then the DMA transfer will begin, to or from the buffer pointed to by this register.

EXAMPLE

```
apbh_chn_bar_t dma_data;
dma_data.ADDRESS = (reg32_t) pDataBuffer;
```

Address: 3300_0000h base + 130h offset + (112d × i), where i=0d to 15d



APBH_CHn_BAR field descriptions

Field	Description
ADDRESS	Address of system memory buffer to be read or written over the AHB bus.

9.5.5.12 APBH DMA Channel n Semaphore Register (APBH_CHn_SEMA)

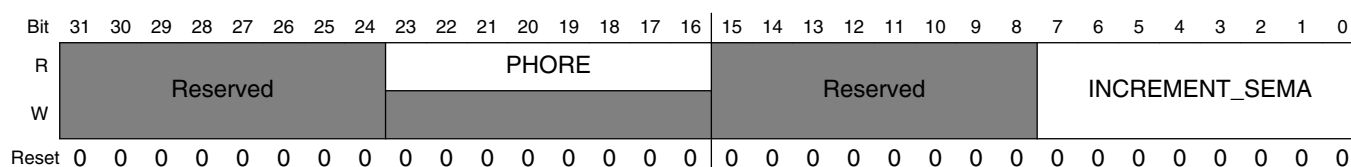
The APBH DMA Channel n semaphore register is used to synchronize the Arm platform instruction stream and the DMA chain processing state.

Each DMA channel has an 8 bit counting semaphore that is used to synchronize between the program stream and the DMA chain processing. DMA processing continues until the DMA attempts to decrement a semaphore that has already reached a value of zero. When the attempt is made, the DMA channel is stalled until software increments the semaphore count.

EXAMPLE

```
BF_WR(APBH_CHn_SEMA, 0, INCREMENT_SEMA, 2); // increment semaphore by two
current_sema = BF_RD(APBH_CHn_SEMA, 0, PHORE); // get instantaneous value
```

Address: 3300_0000h base + 140h offset + (112d × i), where i=0d to 15d



APBH_CH n _SEMA field descriptions

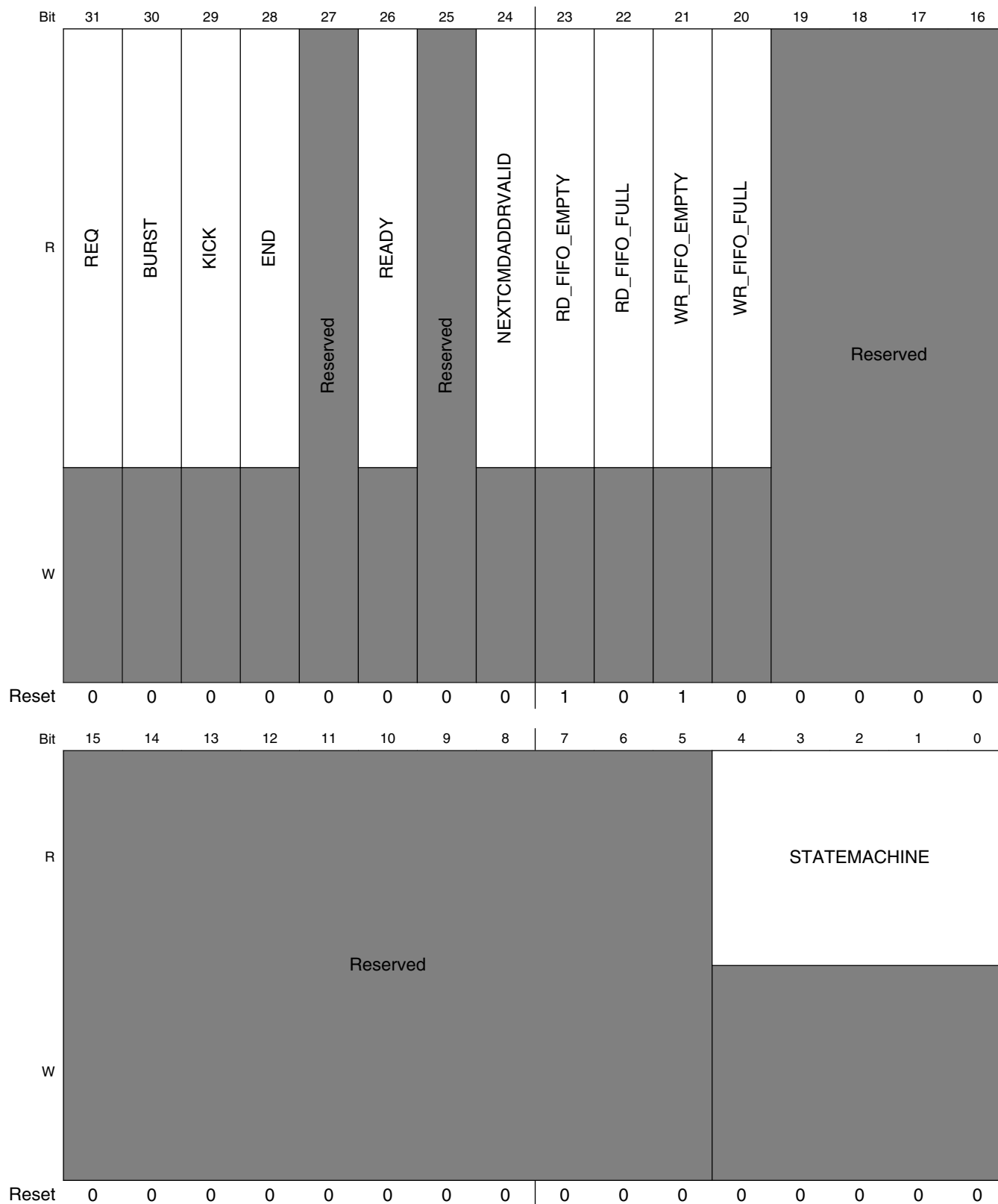
Field	Description
31–24 -	This field is reserved. Reserved, always set to zero.
23–16 PHORE	This read-only field shows the current (instantaneous) value of the semaphore counter.
15–8 -	This field is reserved. Reserved, always set to zero.
INCREMENT_ SEMA	The value written to this field is added to the semaphore count in an atomic way such that simultaneous software adds and DMA hardware subtracts happening on the same clock are protected. This bit field reads back a value of 0x00. Writing a value of 0x02 increments the semaphore count by two, unless the DMA channel decrements the count on the same clock, then the count is incremented by a net one.

9.5.5.13 AHB to APBH DMA Channel n Debug Information (APBH_CH n _DEBUG1)

This register gives debug visibility into the APBH DMA Channel n state machine and controls.

This register allows debug visibility of the APBH DMA Channel n .

Address: 3300_0000h base + 150h offset + (112d × i), where i=0d to 15d



APBH_CHn_DEBUG1 field descriptions

Field	Description
31 REQ	This bit reflects the current state of the DMA Request Signal from the APB device
30 BURST	This bit reflects the current state of the DMA Burst Signal from the APB device
29 KICK	This bit reflects the current state of the DMA Kick Signal sent to the APB Device
28 END	This bit reflects the current state of the DMA End Command Signal sent from the APB Device
27 SENSE	This field is reserved. This bit is reserved for this DMA Channel and always reads 0.
26 READY	This bit is reserved for this DMA Channel and always reads 0.
25 LOCK	This field is reserved. This bit is reserved for this Channel and always reads 0.
24 NEXTCMDADDRVALID	This bit reflects the internal bit which indicates whether the channel's next command address is valid.
23 RD_FIFO_EMPTY	This bit reflects the current state of the DMA Channel's Read FIFO Empty signal.
22 RD_FIFO_FULL	This bit reflects the current state of the DMA Channel's Read FIFO Full signal.
21 WR_FIFO_EMPTY	This bit reflects the current state of the DMA Channel's Write FIFO Empty signal.
20 WR_FIFO_FULL	This bit reflects the current state of the DMA Channel's Write FIFO Full signal.
19–5 RSVD1	This field is reserved. Reserved
STATEMACHINE	PIO Display of the DMA Channel n state machine state. 0x00 IDLE — This is the idle state of the DMA state machine. 0x01 REQ_CMD1 — State in which the DMA is waiting to receive the first word of a command. 0x02 REQ_CMD3 — State in which the DMA is waiting to receive the third word of a command. 0x03 REQ_CMD2 — State in which the DMA is waiting to receive the second word of a command. 0x04 XFER_DECODE — The state machine processes the descriptor command field in this state and branches accordingly. 0x05 REQ_WAIT — The state machine waits in this state for the PIO APB cycles to complete. 0x06 REQ_CMD4 — State in which the DMA is waiting to receive the fourth word of a command, or waiting to receive the PIO words when PIO count is greater than 1. 0x07 PIO_REQ — This state determines whether another PIO cycle needs to occur before starting DMA transfers. 0x08 READ_FLUSH — During a read transfers, the state machine enters this state waiting for the last bytes to be pushed out on the APB. 0x09 READ_WAIT — When an AHB read request occurs, the state machine waits in this state for the AHB transfer to complete. 0x0C WRITE — During DMA Write transfers, the state machine waits in this state until the AHB master arbiter accepts the request from this channel. 0x0D READ_REQ — During DMA Read transfers, the state machine waits in this state until the AHB master arbiter accepts the request from this channel.

Table continues on the next page...

APBH_CHn_DEBUG1 field descriptions (continued)

Field	Description
0x0E	CHECK_CHAIN — Upon completion of the DMA transfers, this state checks the value of the Chain bit and branches accordingly.
0x0F	XFER_COMPLETE — The state machine goes to this state after the DMA transfers are complete, and determines what step to take next.
0x14	TERMINATE — When a terminate signal is set, the state machine enters this state until the current AHB transfer is completed.
0x15	WAIT_END — When the Wait for Command End bit is set, the state machine enters this state until the DMA device indicates that the command is complete.
0x1C	WRITE_WAIT — During DMA Write transfers, the state machine waits in this state until the AHB master completes the write to the AHB memory space.
0x1D	HALT_AFTER_TERM — If HALTONTERMINATE is set and a terminate signal is set, the state machine enters this state and effectively halts. A channel reset is required to exit this state
0x1E	CHECK_WAIT — If the Chain bit is a 0, the state machine enters this state and effectively halts.
0x1F	WAIT_READY — When the NAND Wait for Ready bit is set, the state machine enters this state until the GPMI device indicates that the external device is ready.

9.5.5.14 AHB to APBH DMA Channel n Debug Information (APBH_CHn_DEBUG2)

This register gives debug visibility for the APB and AHB byte counts for DMA Channel n.

This register allows debug visibility of the APBH DMA Channel n.

Address: 3300_0000h base + 160h offset + (112d × i), where i=0d to 15d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	APB_BYTES																AHB_BYTES															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

APBH_CHn_DEBUG2 field descriptions

Field	Description
31–16 APB_BYTES	This value reflects the current number of APB bytes remaining to be transfered in the current transfer.
AHB_BYTES	This value reflects the current number of AHB bytes remaining to be transfered in the current transfer.

9.5.5.15 APBH Bridge Version Register (APBH_VERSION)

This register always returns a known read value for debug purposes it indicates the version of the block.

62BIT Correcting ECC Accelerator (BCH)

This register indicates the RTL version in use.

EXAMPLE

```
if (APBH_VERSION.B.MAJOR != 3)
    Error();
```

Address: 3300_0000h base + 800h offset = 3300_0800h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	MAJOR								MINOR								STEP															
W																																
Reset	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

APBH_VERSION field descriptions

Field	Description
31–24 MAJOR	Fixed read-only value reflecting the MAJOR field of the RTL version.
23–16 MINOR	Fixed read-only value reflecting the MINOR field of the RTL version.
STEP	Fixed read-only value reflecting the stepping of the RTL version.

9.6 62BIT Correcting ECC Accelerator (BCH)

9.6.1 Overview

The hardware ECC accelerator provides a forward error-correction function for improving the reliability of various storage media that may be attached to the device.

For example, NAND flash devices use a spare area to store ecc codes to correct some hard bit errors in data stored within the device, allowing higher device yields and, therefore, lower NAND device costs.

The Bose, Ray-Chaudhuri, Hocquenghem (BCH) Encoder and Decoder module is capable of correcting from 2 to 62 single bit errors within a block of data no larger than about 1900 bytes (512 bytes or 1024 bytes are typical) in applications such as protecting data and resources stored on modern NAND flash devices. The correction level in the BCH block is programmable to provide flexibility for varying applications and configurations of flash page size. The design can be programmed to encode protection of 2 to 62 bit errors when writing flash and to correct the corresponding number of errors on decode. The correction level when decoding **MUST** be programmed to the same correction level as was used during the encode phase.

BCH-codes are a type of block-code, which implies that all error-correction is performed over a block of N -symbols. The BCH operation will be performed over $GF(2^{13} = 8192)$ or $GF(2^{14} = 16384)$, which is the Galois Field consisting of 8191 or 16383 one-bit symbols. BCH-encoding (or encode for any block-code) can be performed by two algorithms: systematic encoding or multiplicative encoding. Systematic encoding is the process of reading all the symbols which constitute a block, dividing continuously these symbols by the generator polynomial for the $GF(8192)$ or $GF(16384)$ and appending the resulting t parity symbols to the block to create a BCH codeword (where t is the number of correctable bits).

The BCH encode process creates $t*13$ (or $t*14$)-bit parity symbols for each data block when the data is written to the flash device. The parity symbols are written to the flash device after the corresponding data block, and together these are collectively called the codeword. The codeword can be used during the decode process to correct errors that occur in either the data or parity blocks.

The BCH decoder processes code words in a 4-step fashion:

1. Syndrome Calculation (SC): This is the process of reading in all of the symbols of the codeword and continuously dividing by the generator polynomial for the field. $2*t$ syndromes must be calculated for each codeword and inspection of the syndromes determines if there are errors: a non-zero set of syndromes indicates one or more errors. This process is implemented parallel hardware to minimize processing time since it must be done every time the decode is performed.
2. Key Equation Solver (KES): The syndromes represent $2t$ -linear equations with $2t$ -unknown variables. The process of solving these equations and selecting from the numerous solutions constitutes the KES module. When the KES block completes its operations, it generates an error locator polynomial (σ) that is used in the proceeding block to determine the locations and values of the errors.
3. Chien Search (CS): This block takes input from the KES block and uses the Chien Algorithm for finding the locations of the errors based on the error locator polynomial. The method basically involves substituting all 8191 symbols from the $GF(8192)$ or 16383 symbols from the $GF(16383)$ into the locator polynomial. All evaluations that produce a zero solution indicate locations of the various errors. Since each located error corresponds to a single bit, the bit in the original data may be corrected by simply flipping the polarity of the incorrect location.
4. Correction: this block has to convert the symbol index and mask information to memory byte indexes and masks.

The BCH block, shown in the figure, was designed to operate in a pipelined fashion to maximize throughput. Aside from the initial latency to fill the pipeline stages, the BCH throughput is about 7/4 cycles/byte. Thus, the bottleneck in performing NAND reads and error corrections is the BCH rate. Current GPMI read rates are approximately 1/2 cycles/byte maximally for the current generation of NAND flash. Fortunately, BCH has a different master clock from GPMI, this gives some flexibility to match the throughput rate. The CPU is not directly involved in generating parity symbols, checking for errors, or correcting them.

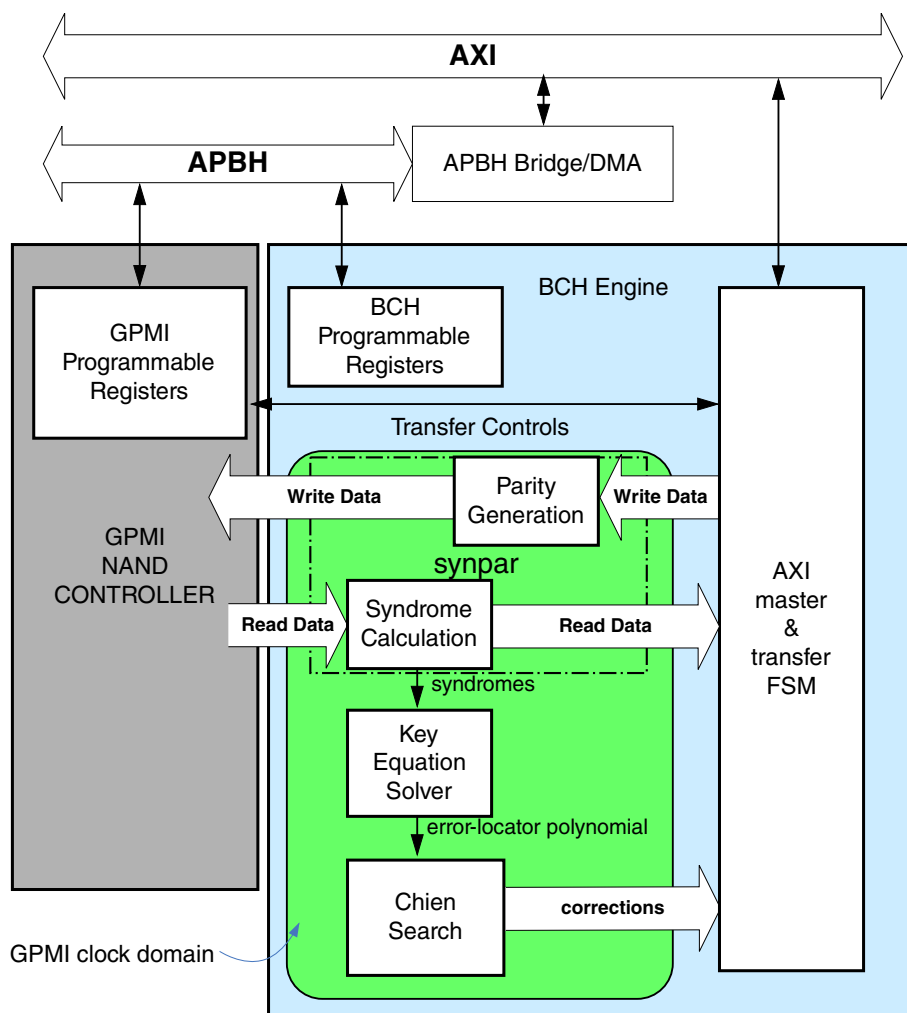


Figure 9-10. Hardware BCH Accelerator

9.6.2 Operation

Before performing any NAND flash read or write operations, software should first program the BCH's flash layout registers (see [Flash Page Layout](#)) to specify how data is to be formatted on the flash device. The BCH hardware allows full programmability over the flash page layout to enable users flexibility in balancing ECC correction levels and ever-changing flash page sizes.

To initiate a NAND Flash write, software will program a GPMI DMA operation. The DMA need only program the GPMI control registers (and handle the requisite flash addressing handshakes) since the BCH will handle all data operations using its AXI bus interface. The BCH will then send the data to the GPMI controller to be written to flash as it computes the parity symbols. At the end of each data block the BCH will insert the parity symbols into the data stream so that the GPMI sees only a continuous stream of data to be written.

NAND Flash read operations operate in a similar manner. As the GPMI controller reads the device, all data is sent to the BCH hardware for error detection/correction. The BCH controller writes all incoming read data to system memory and in parallel computes the syndromes used to detect bit errors. If errors are detected within a block, the BCH hardware activates the error correction logic to determine where bit errors have occurred and ultimately correct them in the data buffer in system memory. After an entire flash page has been read and corrected, the BCH will signal an interrupt to the CPU.

The figure below indicates how data read from the GPMI is operated on within the BCH hardware. As the BCH receives data from the GPMI (top row), it is written to memory by the BCH's Bus Interface Unit (BIU) (second row). For blocks requiring correction, the KES logic will be activated after the entire block has been received. Once the error locator polynomial has been computed, the corrections are determined by the Chien Search and fed back to the BIU, which performs a read, modify, write operation on the buffer in memory to correct the data.

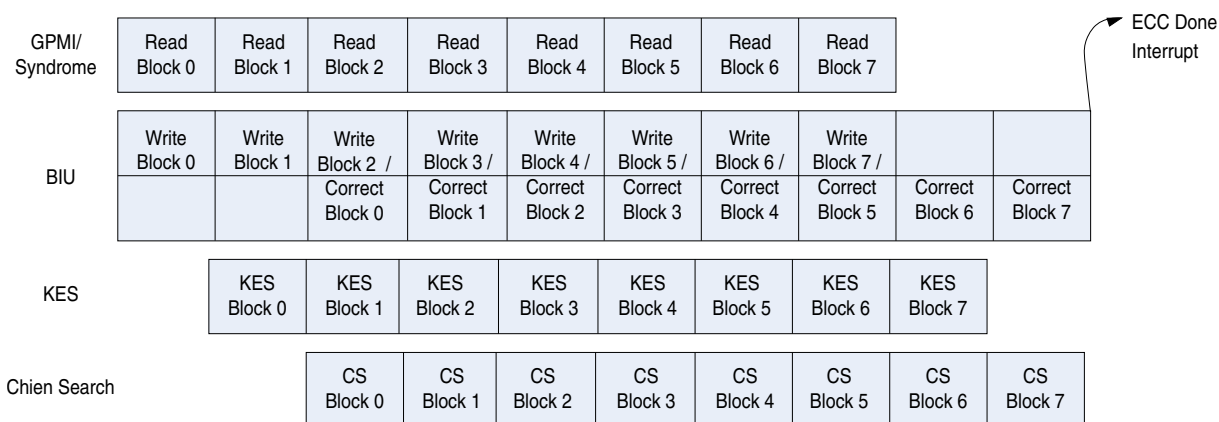


Figure 9-11. Block Pipeline while Reading Flash

9.6.2.1 BCH Limitations and Assumptions

- The BCH is programmable to support 2 to 62 bit error correction. ECC0 is supported as a pass-through, non-correcting mode.
- Data block sizes must be a multiple of 4 bytes and be aligned in system memory.
- The BCH supports a programmable number of metadata/auxiliary data bytes, from 0 to 255.
- Metadata will be written at the beginning of the flash page to facilitate fast access for filesystem operations.
- Metadata may be treated as an independent block for ECC purposes or combined with the first data block to conserve bits in the flash.
- The BCH does not support a partial page write (this can be accomplished by programming the BCH layout registers such that the BCH only sees a portion of the page).
- Flash read operations can read the entire page or the first block on the page.
- The BCH also supports a memory-to-memory mode of operation that does not require the use of DMA or the GPMI.

9.6.2.2 Flash Page Layout

The BCH supports a fully programmable flash page layout. The BCH maintains four independent layout registers that can describe four completely different NAND devices or layouts.

When the BCH initiates an operation, it selects one of the layouts by using the chip select as an index into the BCH_LAYOUTSELECT register that determines which layout should be used for the operation.

Three possible (generic) flash layout schemes are supported, as indicated in the figure below. (In each case, the metadata size may also be programmed to 0 bytes). Metadata may either be combined with the first block of data or the size of the first data block can be programmed to 0 to allow the metadata to be protected by its own ECC parity bits.

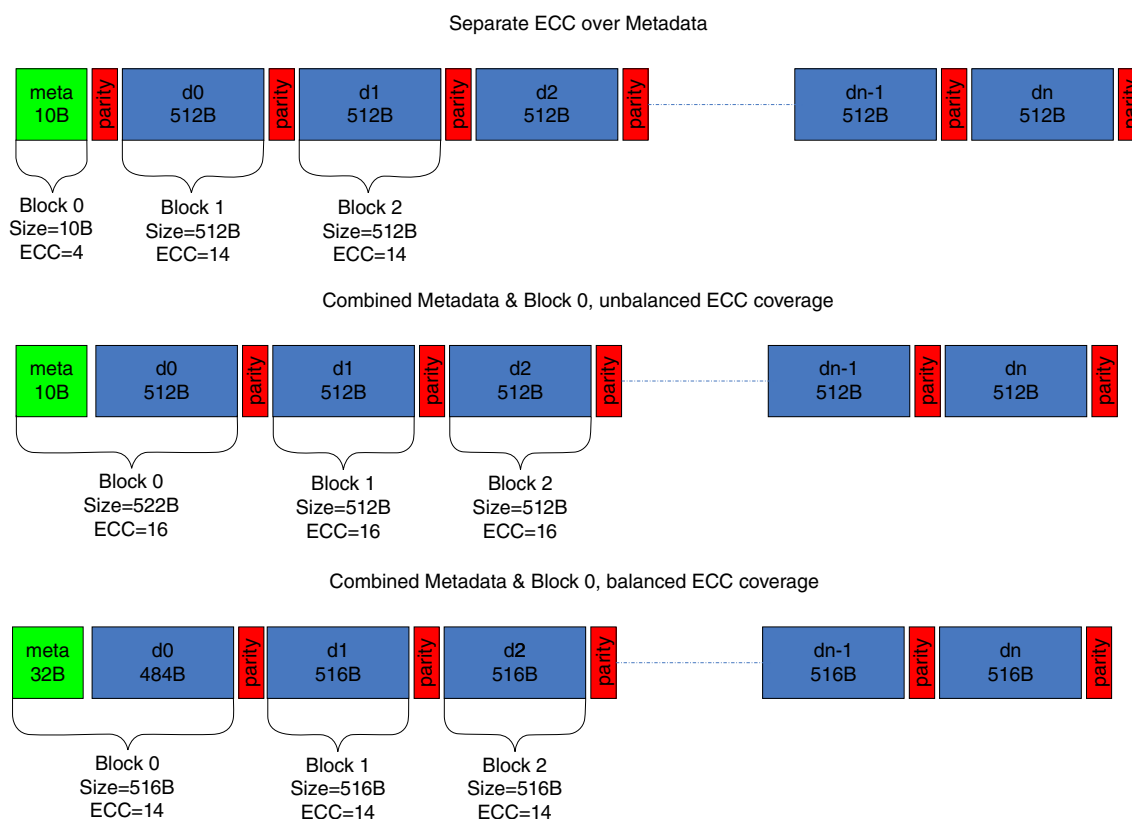


Figure 9-12. FLASH Page Layout Options

Each layout is determined by a pair of registers that define the following parameters:

- **DATA0_SIZE:** Indicates the number of data bytes in the first block on the page (this should not include parity or metadata bytes). This should be set to 0 when the metadata is to be covered separately with its own ECC. This must be a multiple of 4 bytes.
- **ECC0:** Indicates the ECC level to be used for the first block on the flash (data0+metadata).
- **META_SIZE:** Indicates the number of bytes (from 0-255) that are stored as metadata.

- **NBLOCKS:** Indicates the number of subsequent DATAN blocks on the flash, or the number of blocks following the DATA0 block.
- **DATAN_SIZE:** Indicates the number of data bytes in all subsequent data blocks. This **MUST** be a multiple of 4 bytes.
- **ECCN:** Indicates the ECC level to be used for the subsequent data blocks.
- **GF0 or GFN:** Indicates the Galois field the meta / data blocks are using
- **PAGE_SIZE:** Indicates the total number of bytes available per page on the physical flash device. This includes the spare area and is typically 4096+128, 4096+218, or 2048+64 bytes.

9.6.2.3 Determining the ECC layout for a device

Since the BCH is programmable, a system can trade off ECC levels for flash size and layout configurations.

The following examples indicate how to determine a valid layout based on the required storage space and flash size. For all cases, the size of the parity will be 13 (or 14 for GF(2¹⁴))*ECC level *bits*-- so for ECC8, 13 (or 14) bytes are required (per block).

9.6.2.3.1 4K+218 flash, 10 bytes metadata, 512 byte data blocks, separate metadata, Assuming GF(2¹³)

In this case, we have 8 data blocks each consisting of 512 bytes. Since the flash has 218 spare bytes (1744 bits), first estimate an ECC level for the data blocks by first subtracting the number of metadata bytes from the spare bytes (218 – 10 = 208 bytes = 1664 bits) then dividing the number of bits by 8 (number of blocks) and then by 13 (bits per ECC level).

$$(218 - 10) \times 8 = 1664 / 13(8) = 16$$

Therefore all the data blocks could be covered by ECC16 if the metadata had no parity. This isn't acceptable, so assume ECC14 for all the data blocks. Now calculate the number of free bits for the metadata parity as

$$1664 - (14) \times 13 \times 8 = 208$$

Therefore, 208 bits remain for metadata parity. Dividing by 13 (bits/ECC) gives 16, so the metadata can be covered with ECC16. The settings for this device would then be

Table 9-9. Settings for 4K+218 FLASH

Setting	Value
PAGE_SIZE	4096+218=4314=0x10DA
META_SIZE	10=0x0A
DATA0_SIZE	0
ECC0	16=0x10
GF0	GF(2 ¹³)
DATAN_SIZE	512=0x200 (in register interface, assigned as 0x80)
ECCN	14=0x0E
GFN	GF(2 ¹³)
NBLOCKS	8

9.6.2.3.2 4K+128 flash, 10 bytes metadata, 1024 byte data blocks, separate metadata, assuming GF(2¹³) for data and GF(2¹⁴) for metadata

This flash will have 118 bytes available for ECC (after subtracting the metadata size), therefore, 994 bits.

Dividing by 4*14 (number of blocks * ECC level) we get 17.75, therefore we can support ECC16 on the data blocks. The number of free spare bits becomes 944 - 16 * 4 * 14 = 944 - 896 = 48, divided by 13 = 3.69, therefore the metadata can be also covered by ECC2.

Table 9-10. Settings for 4K+128 FLASH

Setting	Value
PAGE_SIZE	4096+128=4224=0x1080
META_SIZE	10=0x0A
DATA0_SIZE	0
ECC0	2
GF0	GF(2 ¹³)
DATAN_SIZE	1024=0x400 (in register interface, assigned as 0x100)
ECCN	16
GFN	GF(2 ¹⁴)
NBLOCKS	4

In this case, there will be additional unused spare bits, with the BCH will pad out with zeros.

9.6.2.4 Data Buffers in System Memory

While the data on the flash is interleaved with parity symbols, the BCH assumes that the data buffers in memory are contiguous.

Metadata read from the flash will be stored to the location pointed to by the `GPMI_AUXILIARY` register and data will be written to the address specified in the `GPMI_PAYLOAD` register as is shown in the following figure where the block length is 512 bytes for example. Since the number of blocks on a flash page is programmable, the BCH also writes individual block correction status to the auxiliary pointer at the word-aligned address following the end of the metadata. Optionally, the computed syndromes may also be written to the auxiliary area if the `DEBUGSYNDROME` bit is set in the control register.

As blocks complete processing, the bus master will accumulate the status for each block and write it to the auxiliary data buffer following the metadata. The metadata area will be padded with 0's until the next word boundary and the status for blocks 0-3 will be written to the next word. The status for subsequent blocks will then be written to the buffer. The status for the first block (metadata block) is also stored in the `STATUS_BLK0` register in the `BCH_STATUS` register. The completion codes for the blocks are indicated in the [Table 9-11](#). Note that the definition of the bytes and their ordering in the auxiliary and payload storage areas are user defined. When this data is read back from the flash and put into memory, it will resemble the original buffer that was written out to the flash.

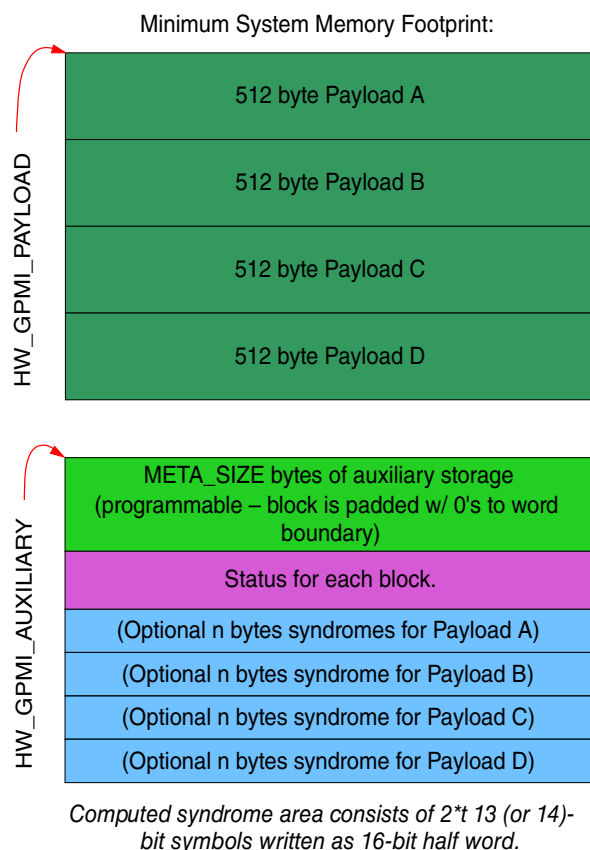
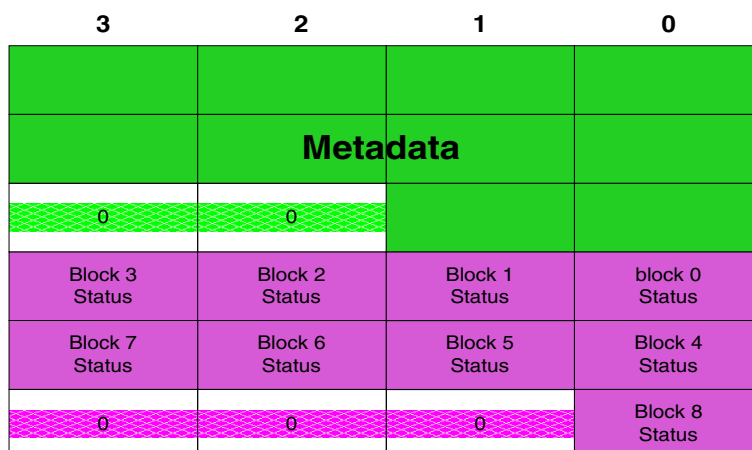


Figure 9-13. BCH Data Buffers in Memory

Table 9-11. Status Block Completion Codes

Code	Description
0xFF	Block is erased
0xFE	Block is uncorrectable
0x00	No errors found
0x01-0x3E	Number of errors corrected

The following figure shows the layout of the bytes within the status field.



Status bytes are allocated based on the NBLOCKS programmed into the flash format register. The number of status bytes are computed by NBLOCK+1. The status area will be padded with zeros to the next word boundary.

Syndrome data written for debug purposes follows the end of the status block.

Figure 9-14. Memory-to-Memory Operations

9.6.3 Memory to Memory (Loopback) Operation

The BCH supports a memory-to-memory mode of operation where both the encoded and decoded buffers reside in system memory.

This can be useful for applications where data must be protected by ECC, but the storage device does not reside on the GPMI bus.

The BCH operation in memory to memory mode is much simpler than in GPMI mode since DMAs are not required to manage the operation. Instead, software simply writes the BCH_DATAPTR and BCH_METAPTR with the addresses of the data and metadata (auxiliary) buffers and the BCH_ENCODEPTR with the address of the buffer for encoded data. To initiate the operation, software simply sets the M2M_ENCODE and M2M_ENABLE bits in the control register. The BCH can be programmed to either issue an interrupt at the end of the operation or software may poll the status bits for completion.

Memory to memory decode operations work in a similar manner. The encoded data address is written to the BCH_ENCODEPTR and the data and meta pointers are written to buffers that correspond to the desired decoded data addresses. To initiate a decode, software must set the M2M_ENCODE bit to 0 while writing the M2M_ENABLE bit. Note that the addresses written to the BCH_DATAPTR, BCH_METAPTR and BCH_ENCODEPTR registers should always be aligned on a 4 byte boundary. In other words, the 2 lower bits of the address should always be written with zeros.

9.6.4 Programming the BCH/GPMI Interfaces

Programming the BCH for NAND operations consists largely of disabling the soft reset and clock bits (SFTRST and CLKGATE) from the BCH_CTRL register and then programming the flash layout registers to correspond to the format of the attached NAND device(s).

The BCH_LAYOUTSELECT register should also be programmed to map the chip select of each attached device into one of the four layout registers.

The bulk of the programming is actually applied to the GPMI through PIO operations embedded in DMA command structures. The DMA will perform all the requisite handshaking with the GPMI interface to negotiate the address portion of the transfer, then the BCH will handle all the movement of data from memory to the GPMI (writes) or the GPMI to memory (reads). The BCH will direct all data blocks to the buffer pointed to by the PAYLOAD_BUFFER and the metadata will be written to the AUXILIARY_BUFFER. Both of these registers are located in the GPMI PIO data space and are communicated to the BCH hardware at the beginning of the transfer. Thus, the normal multi-NAND DMA based device interleaving is preserved, that is, four NANDs on four separate chip selects can be scheduled for read or write operations using the BCH. Whichever channel finishes its ready wait first and enters the DMA arbiter with its lock bit set owns the GPMI command interface and through it owns the BCH resources for the duration of its processing.

9.6.4.1 BCH Encoding for NAND Writes

The BCH encoder flowchart in [Figure 9-15](#) shows the detailed steps involved in programming and using the BCH encoder. This flowchart shows how to use the BCH block with the GPMI.

To use the BCH encoder with the GPMI's DMA, create a DMA command chain containing ten descriptor structures, as shown in [Figure 9-17](#) and detailed in the DMA structure code example that follows it in [DMA Structure Code Example](#). The ten descriptors perform the following tasks:

1. Disable the BCH block (in case it was enabled) and issue NAND write setup command byte (under CLE) and address bytes (under ALE).
2. Configure and enable the BCH and GPMI blocks to perform the NAND write.
3. Disable the BCH block and issue NAND write execute command byte (under CLE).

4. Wait for the NAND device to finish writing the data by watching the ready signal.
5. Check for NAND timeout through PSENSE.
6. Issue NAND status command byte (under CLE).
7. Read the status and compare against expected.
8. If status is incorrect or incomplete, branch to error handling descriptor chain.
9. Otherwise, write is complete and emit GPMI interrupt.

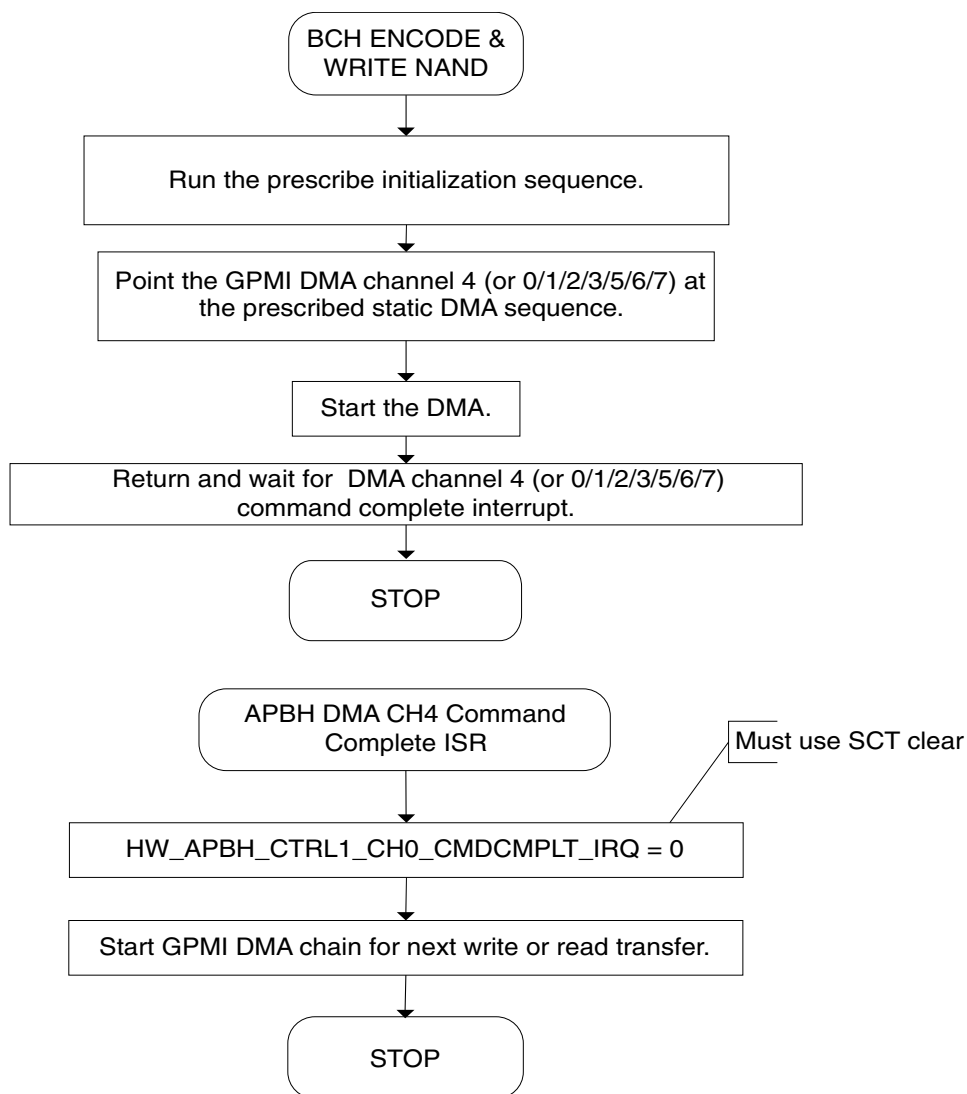


Figure 9-15. BCH Encode Flowchart

Descriptor Legend

NEXT CMD ADDR										
CMD	<=	xfer_count	cmdwords	wait4endcmd	semaphore	nandwait4ready	nandlock	irqoncmplt	chain	command
BUFFER ADDR										
HW_GPMI_CTRL0	<=	command_mode	word_length	lock_cs	CS	address	address_increment	xfer_count		
HW_GPMI_COMPARE	<=	mask				reference				
HW_GPMI_ECCCTRL	<=	ecc_cmd			enable_ecc					buffer_mask
HW_GPMI_ECCCOUNT										
HW_GPMI_PAYLOAD										
HW_GPMI_AUXILIARY										

Figure 9-16. BCH DMA Descriptor Legend

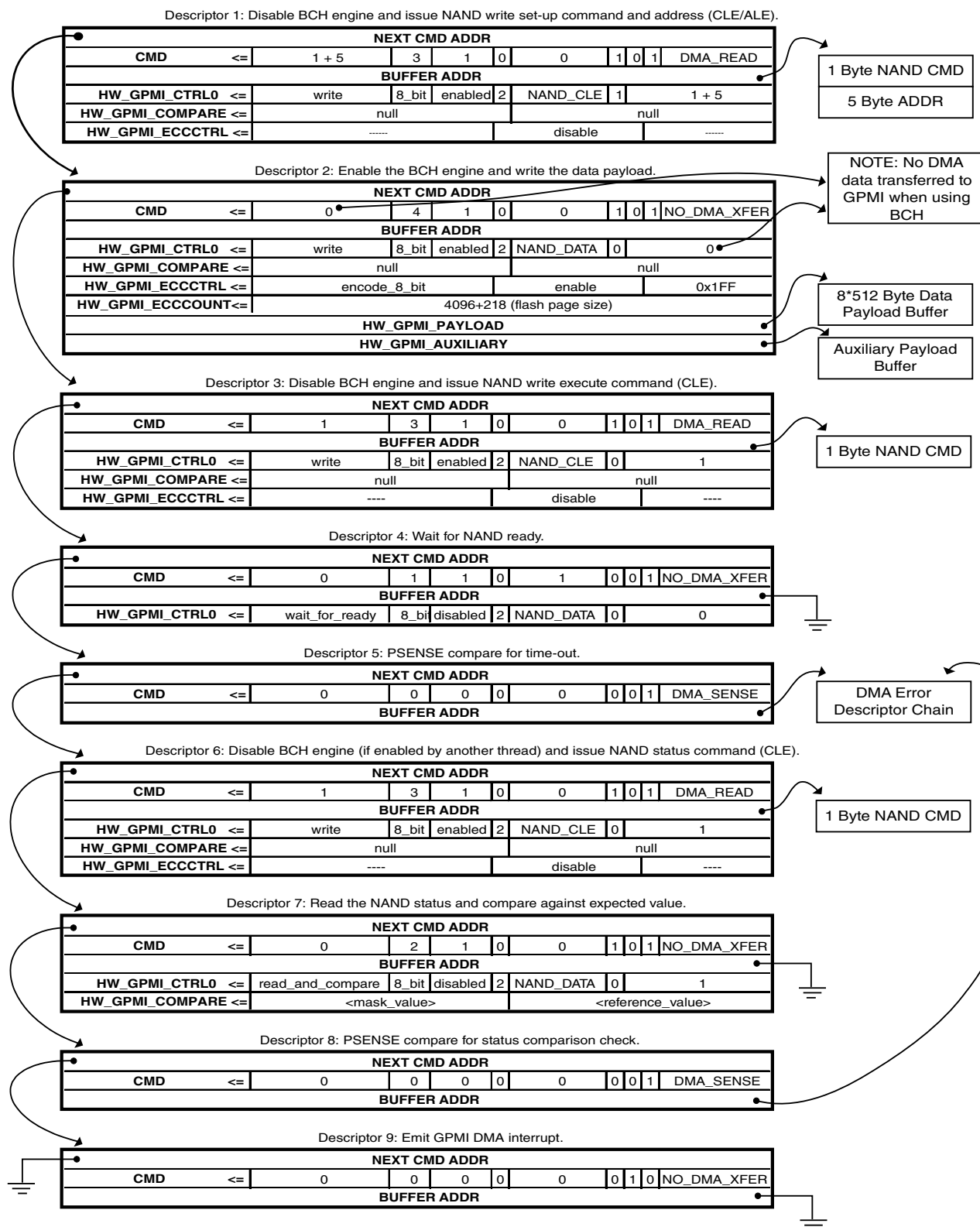


Figure 9-17. BCH Encode DMA Descriptor Chain

9.6.4.1.1 DMA Structure Code Example

The following code sample illustrates the coding for one write transaction involving 4096 bytes of data payload (eight 512-byte blocks) and 10 bytes of auxiliary payload (also referred to as metadata) to a 4K NAND page sitting on GPMI CS2.

```
//-----
// generic DMA/GPMI/ECC descriptor struct, order sensitive!
//-----
typedef struct {
    // DMA related fields
    unsigned int dma_nxtcmdar;
    unsigned int dma_cmd;
    unsigned int dma_bar;
    // GPMI related fields
    unsigned int gpmi_ctrl0;
    unsigned int gpmi_compare;
    unsigned int gpmi_eccctrl;
    unsigned int gpmi_ecccount;
    unsigned int gpmi_data_ptr;
    unsigned int gpmi_aux_ptr;
} GENERIC_DESCRIPTOR;
//-----
// allocate 10 descriptors for doing a NAND ECC Write
//-----
GENERIC_DESCRIPTOR write[10];
//-----
// DMA descriptor pointer to handle error conditions from psense checks
//-----
unsigned int * dma_error_handler;
//-----
// 8 byte NAND command and address buffer
// any alignment is ok, it is read by the GPMI DMA
// byte 0 is write setup command
// bytes 1-5 is the NAND address
// byte 6 is write execute command
// byte 7 is status command
//-----
unsigned char nand_cmd_addr_buffer[8];
//-----
// 4096 byte payload buffer used for reads or writes
// needs to be word aligned
//-----
unsigned int write_payload_buffer[(4096/4)];
//-----
// 65 byte meta-data to be written to NAND
// needs to be word aligned
//-----
unsigned int write_aux_buffer[65];
//-----
// Descriptor 1: issue NAND write setup command (CLE/ALE)
//-----
write[0].dma_nxtcmdar = &write[1]; // point to the next descriptor
write[0].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (1 + 5) | // 1 byte command, 5 byte address
                  BF_APBH_CHn_CMD_CMDWORDS (3) | // send 3 words to the GPMI
                  BF_APBH_CHn_CMD_WAIT4ENDCMD (1) | // wait for command to finish
before
                  BF_APBH_CHn_CMD_SEMAPHORE (0) | // continuing
                  BF_APBH_CHn_CMD_NANDWAIT4READY (0) |
                  BF_APBH_CHn_CMD_NANDLOCK (1) | // prevent other DMA channels
from
                  BF_APBH_CHn_CMD_IRQONCMPLT (0) | // taking over
                  BF_APBH_CHn_CMD_CHAIN (1) | // follow chain to next command
```

62BIT Correcting ECC Accelerator (BCH)

```
        BV_FLD(APBH_CHn_CMD, COMMAND, DMA_READ); // read data from DMA, write to
NAND
write[0].dma_bar = &nand_cmd_addr_buffer;          // byte 0 write setup, bytes 1 - 5 NAND
address
// 3 words sent to the GPMI
write[0].gpmi_ctrl0 = BV_FLD(GPMI_CTRL0, COMMAND_MODE, WRITE) | // write to the NAND
                    BV_FLD(GPMI_CTRL0, WORD_LENGTH, 8_BIT) |
                    BV_FLD(GPMI_CTRL0, LOCK_CS, ENABLED) |
BF_GPMI_CTRL0_CS (2) | // must correspond to NAND CS
used
                    BV_FLD(GPMI_CTRL0, ADDRESS, NAND_CLE) |
BF_GPMI_CTRL0_ADDRESS_INCREMENT (1) | // send command and
address
BF_GPMI_CTRL0_XFER_COUNT (1 + 5); // 1 byte command, 5 byte
address
write[0].gpmi_compare = NULL; // field not used but necessary to
set eccctrl
write[0].gpmi_eccctrl = BV_FLD(GPMI_ECCCTRL, ENABLE_ECC, DISABLE); // disable the ECC block
//-----
// Descriptor 2: write the data payload (DATA)
//-----
write[1].dma_nxtcmdar = &write[2]; // point to the next descriptor
write[1].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (0) | // NOTE: No DMA data transfer
                  BF_APBH_CHn_CMD_CMDWORDS (4) | // send 4 words to the GPMI
                  BF_APBH_CHn_CMD_WAIT4ENDCMD (1) | // Wait to end
                  BF_APBH_CHn_CMD_SEMAPHORE (0) |
                  BF_APBH_CHn_CMD_NANDWAIT4READY (0) |
                  BF_APBH_CHn_CMD_NANDLOCK (1) | // maintain resource lock
                  BF_APBH_CHn_CMD_IRQONCMPLT (0) |
                  BF_APBH_CHn_CMD_CHAIN (1) | // follow chain to next command
                  BV_FLD(APBH_CHn_CMD, COMMAND, DMA_NO_XFER); // No data transferred
write[1].dma_bar = &write_payload_buffer; // pointer for the 4K byte
data area
// 4 words sent to the GPMI
write[1].gpmi_ctrl0 = BV_FLD(GPMI_CTRL0, COMMAND_MODE, WRITE) | // write to the NAND
                    BV_FLD(GPMI_CTRL0, WORD_LENGTH, 8_BIT) |
                    BV_FLD(GPMI_CTRL0, LOCK_CS, ENABLED) |
BF_GPMI_CTRL0_CS (2) | // must correspond to NAND
CS used
                    BV_FLD(GPMI_CTRL0, ADDRESS, NAND_DATA) |
BF_GPMI_CTRL0_ADDRESS_INCREMENT (0) |
BF_GPMI_CTRL0_XFER_COUNT (0); // NOTE: this field
contains
// the total amount
// DMA transferred to GPMI via DMA (0)!
write[1].gpmi_compare = NULL; // field not used but necessary
to
set eccctrl
write[1].gpmi_eccctrl = BV_FLD(GPMI_ECCCTRL, ECC_CMD, ENCODE_8_BIT) | // specify t = 8
mode
                    BV_FLD(GPMI_ECCCTRL, ENABLE_ECC, ENABLE) | // enable ECC module
BF_GPMI_ECCCTRL_BUFFER_MASK (0x1FF); // write all 8 data
blocks
// and 1 aux block
write[1].gpmi_ecccount = BF_GPMI_ECCCOUNT_COUNT(4096+218); // specify number of bytes
// written to NAND
write[1].gpmi_data_pointer = &write_payload_pointer; // data buffer address
write[1].gpmi_aux_pointer = &write_aux_pointer; // metadata pointer
//-----
// Descriptor 3: issue NAND write execute command (CLE)
//-----
write[2].dma_nxtcmdar = &write[3]; // point to the next descriptor
write[2].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (1) | // 1 byte command
                  BF_APBH_CHn_CMD_CMDWORDS (3) | // send 3 words to the GPMI
                  BF_APBH_CHn_CMD_WAIT4ENDCMD (1) | // wait for command to finish
before
// continuing
                  BF_APBH_CHn_CMD_SEMAPHORE (0) |
                  BF_APBH_CHn_CMD_NANDWAIT4READY (0) |
                  BF_APBH_CHn_CMD_NANDLOCK (1) | // maintain resource lock
```

```

        BF_APBH_CHn_CMD_IRQONCMPLT    (0) |
        BF_APBH_CHn_CMD_CHAIN          (1) | // follow chain to next command
        BV_FLD(APBH_CHn_CMD, COMMAND, DMA_READ); // read data from DMA, write to
NAND
write[2].dma_bar = &nand_cmd_addr_buffer[6]; // point to byte 6, write execute
command
// 3 words sent to the GPMI
write[2].gpmi_ctrl0 = BV_FLD(GPMI_CTRL0, COMMAND_MODE, WRITE) | // write to the NAND
                    BV_FLD(GPMI_CTRL0, WORD_LENGTH, 8_BIT) |
                    BV_FLD(GPMI_CTRL0, LOCK_CS, ENABLED) |
                    BF_GPMI_CTRL0_CS (2) | // must correspond to NAND CS
used
                    BV_FLD(GPMI_CTRL0, ADDRESS, NAND_CLE) |
                    BF_GPMI_CTRL0_ADDRESS_INCREMENT (0) |
                    BF_GPMI_CTRL0_XFER_COUNT (1); // 1 byte command
write[2].gpmi_compare = NULL; // field not used but necessary to set
eccctrl
write[2].gpmi_eccctrl = BV_FLD(GPMI_ECCCTRL, ENABLE_ECC, DISABLE); // disable the ECC block
//-----
// Descriptor 4: wait for ready (CLE)
//-----
write[3].dma_nxtcmdar = &write[4]; // point to the next descriptor

write[3].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (0) | // no dma transfer
                  BF_APBH_CHn_CMD_CMDWORDS (1) | // send 1 word to the GPMI
                  BF_APBH_CHn_CMD_WAIT4ENDCMD (1) | // wait for command to finish before
                                                    // continuing
                  BF_APBH_CHn_CMD_SEMAPHORE (0) |
                  BF_APBH_CHn_CMD_NANDWAIT4READY(1) | // wait for nand to be ready
                  BF_APBH_CHn_CMD_NANDLOCK (0) | // relinquish nand lock
                  BF_APBH_CHn_CMD_IRQONCMPLT (0) |
                  BF_APBH_CHn_CMD_CHAIN (1) | // follow chain to next command
                  BV_FLD(APBH_CHn_CMD, COMMAND, NO_DMA_XFER); // no dma transfer
write[3].dma_bar = NULL; // field not used
// 1 word sent to the GPMI
write[3].gpmi_ctrl0 = BV_FLD(GPMI_CTRL0, COMMAND_MODE, WAIT_FOR_READY) | // wait for NAND
ready
                    BV_FLD(GPMI_CTRL0, WORD_LENGTH, 8_BIT) |
                    BV_FLD(GPMI_CTRL0, LOCK_CS, DISABLED) |
                    BF_GPMI_CTRL0_CS (2) | // must correspond to NAND CS
used
                    BV_FLD(GPMI_CTRL0, ADDRESS, NAND_DATA) |
                    BF_GPMI_CTRL0_ADDRESS_INCREMENT (0) |
                    BF_GPMI_CTRL0_XFER_COUNT (0);
//-----
// Descriptor 5: psense compare (time out check)
//-----
write[4].dma_nxtcmdar = &write[5]; // point to the next descriptor
write[4].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (0) | // no dma transfer
                  BF_APBH_CHn_CMD_CMDWORDS (0) | // no words sent to GPMI
                  BF_APBH_CHn_CMD_WAIT4ENDCMD (0) | // do not wait to continue
                  BF_APBH_CHn_CMD_SEMAPHORE (0) |
                  BF_APBH_CHn_CMD_NANDWAIT4READY(0) |
                  BF_APBH_CHn_CMD_NANDLOCK (0) |
                  BF_APBH_CHn_CMD_IRQONCMPLT (0) |
                  BF_APBH_CHn_CMD_CHAIN (1) | // follow chain to next command
                  BV_FLD(APBH_CHn_CMD, COMMAND, DMA_SENSE); // perform a sense check
write[4].dma_bar = dma_error_handler; // if sense check fails, branch to error
handler
//-----
// Descriptor 6: issue NAND status command (CLE)
//-----
write[5].dma_nxtcmdar = &write[6]; // point to the next descriptor
write[5].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (1) | // 1 byte command
                  BF_APBH_CHn_CMD_CMDWORDS (3) | // send 3 words to the GPMI
                  BF_APBH_CHn_CMD_WAIT4ENDCMD (1) | // wait for command to finish
before
continuing
                  BF_APBH_CHn_CMD_SEMAPHORE (0) |
                  BF_APBH_CHn_CMD_NANDWAIT4READY(0) |

```

62BIT Correcting ECC Accelerator (BCH)

```
BF_APBH_CHn_CMD_NANDLOCK      (1) | // prevent other DMA channels from
taking over
BF_APBH_CHn_CMD_IRQONCMPLT    (0) |
BF_APBH_CHn_CMD_CHAIN          (1) | // follow chain to next command
BV_FLD(APBH_CHn_CMD, COMMAND, DMA_READ); // read data from DMA, write to
NAND
write[5].dma_bar = &nand_cmd_addr_buffer[7]; // point to byte 7, status
command
write[5].gpmi_compare = NULL; // field not used but necessary to set
eccctrl
write[5].gpmi_eccctrl = BV_FLD(GPMI_ECCCTRL, ENABLE_ECC, DISABLE); // disable the ECC block
// 3 words sent to the GPMI
write[5].gpmi_ctrl0 = BV_FLD(GPMI_CTRL0, COMMAND_MODE, WRITE) | // write to the NAND
BV_FLD(GPMI_CTRL0, WORD_LENGTH, 8_BIT) |
BV_FLD(GPMI_CTRL0, LOCK_CS, ENABLED) |
BF_GPMI_CTRL0_CS (2) | // must correspond to NAND CS
used
BV_FLD(GPMI_CTRL0, ADDRESS, NAND_CLE) |
BF_GPMI_CTRL0_ADDRESS_INCREMENT (0) |
BF_GPMI_CTRL0_XFER_COUNT (1); // 1 byte command
//-----
// Descriptor 7: read status and compare (DATA)
//-----
write[6].dma_nextcmdar = &write[7]; // point to the next descriptor
write[6].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (0) | // no dma transfer
BF_APBH_CHn_CMD_CMDWORDS (2) | // send 2 words to the GPMI
BF_APBH_CHn_CMD_WAIT4ENDCMD (1) | // wait for command to finish
before
// continuing
BF_APBH_CHn_CMD_SEMAPHORE (0) |
BF_APBH_CHn_CMD_NANDWAIT4READY(0) |
BF_APBH_CHn_CMD_NANDLOCK (1) | // maintain resource lock
BF_APBH_CHn_CMD_IRQONCMPLT (0) |
BF_APBH_CHn_CMD_CHAIN (1) | // follow chain to next command
BV_FLD(APBH_CHn_CMD, COMMAND, NO_DMA_XFER); // no dma transfer
write[6].dma_bar = NULL; // field not used
// 2 word sent to the GPMI
write[6].gpmi_ctrl0 = BV_FLD(GPMI_CTRL0, COMMAND_MODE, READ_AND_COMPARE) | // read from the
// NAND and
// compare to expect
BV_FLD(GPMI_CTRL0, WORD_LENGTH, 8_BIT) |
BV_FLD(GPMI_CTRL0, LOCK_CS, DISABLED) |
BF_GPMI_CTRL0_CS (2) | // must correspond to NAND CS
used
BV_FLD(GPMI_CTRL0, ADDRESS, NAND_DATA) |
BF_GPMI_CTRL0_ADDRESS_INCREMENT (0) |
BF_GPMI_CTRL0_XFER_COUNT (1);
write[6].gpmi_compare = <MASK_AND_REFERENCE_VALUE>; // NOTE: mask and reference values are
NAND
// SPECIFIC to evaluate the NAND
status
//-----
// Descriptor 8: psense compare (time out check)
//-----
write[7].dma_nextcmdar = &write[8]; // point to the next descriptor
write[7].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (0) | // no dma transfer
BF_APBH_CHn_CMD_CMDWORDS (0) | // no words sent to GPMI
BF_APBH_CHn_CMD_WAIT4ENDCMD (0) | // do not wait to continue
BF_APBH_CHn_CMD_SEMAPHORE (0) |
BF_APBH_CHn_CMD_NANDWAIT4READY(0) |
BF_APBH_CHn_CMD_NANDLOCK (0) | // relinquish nand lock
BF_APBH_CHn_CMD_IRQONCMPLT (0) |
BF_APBH_CHn_CMD_CHAIN (1) | // follow chain to next command
BV_FLD(APBH_CHn_CMD, COMMAND, DMA_SENSE); // perform a sense check
write[7].dma_bar = dma_error_handler; // if sense check fails, branch to error
handler
//-----
// Descriptor 9: emit GPMI interrupt
//-----
write[8].dma_nextcmdar = NULL; // not used since this is
```



```

last
descriptor
write[8].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT      (0)      | // no dma transfer
                  BF_APBH_CHn_CMD_CMDWORDS        (0)      | // no words sent to GPMI
                  BF_APBH_CHn_CMD_WAIT4ENDCMD      (0)      | // do not wait to continue
                  BF_APBH_CHn_CMD_SEMAPHORE        (0)      |
                  BF_APBH_CHn_CMD_NANDWAIT4READY  (0)      |
                  BF_APBH_CHn_CMD_NANDLOCK         (0)      |
                  BF_APBH_CHn_CMD_IRQONCMPLT      (1)      | // emit GPMI interrupt
                  BF_APBH_CHn_CMD_CHAIN            (0)      | // terminate DMA chain
processing
                  BV_FLD(APBH_CHn_CMD, COMMAND, NO_DMA_XFER); // no dma transfer

```

9.6.4.1.2 Using the BCH Encoder

To use the BCH encoder, first turn off the module-wide soft reset bit in both the GPMI and BCH blocks before starting any DMA activity.

Turning off the soft reset must take place by itself, prior to programming the rest of the control registers. Turn off the BCH bus master soft reset bit. Turn off the clock gate bits.

Program the remainder of the GPMI, BCH and APBH DMA as follows:

```

// bring APBH out of reset
APBH_CTRL0_CLR(BM_APBH_CTRL0_SFTRST);
APBH_CTRL0_CLR(BM_APBH_CTRL0_CLKGATE);

// bring BCH out of reset
BCH_CTRL_CLR(BM_BCH_CTRL_SFTRST);
BCH_CTRL_CLR(BM_BCH_CTRL_CLKGATE);

// bring gpmi out of reset
GPMI_CTRL0_CLR(BM_GPMI_CTRL0_SFTRST);
GPMI_CTRL0_CLR(BM_GPMI_CTRL0_CLKGATE);
GPMI_CTRL1_SET(BM_GPMI_CTRL1_DEV_RESET | // deassert reset
               BM_GPMI_CTRL1_BCH_MODE ); // enable BCH mode

// enable pinctrl
PINCTRL_CTRL_WR(0x00000000);

// enable gpmi pins
PINCTRL_MUXSEL0_CLR(0x0000ffff); // data bits
PINCTRL_MUXSEL1_CLR(0x03ffffff); // control bits
PINCTRL_MUXSEL8_CLR(0x0003f3ff); // control bits
PINCTRL_MUXSEL8_SET(0x00015155); // control bits

```

Note that for writing NANDs (ECC encoding), only GPMI DMA command complete interrupts are used. The BCH engine is used for writing to the NAND but may optionally produce an interrupt. From the sample code in [DMA Structure Code Example](#) :

- DMA descriptor 1 prepares the NAND for data write by using the GPMI to issue a write setup command byte under CLE, then sends a 5-byte address under ALE. The BCH engine is disabled and not used for these commands.
- DMA descriptor 2 enables the BCH engine for encoding to begin the initial writing of the NAND data by specifying where the data and auxiliary payload are coming from in system memory.

- DMA descriptor 3 issues the write commit command byte under CLE to the NAND.
- DMA descriptor 4 waits for the NAND to complete the write commit/transfer by watching the NAND's ready line status. This descriptor relinquishes the NANDLOCK on the GPMI to enable the other DMA channels to initiate NAND transactions on different NAND CS lines.
- DMA descriptor 6 issues a NAND status command byte under "CLE" to check the status of the NAND device following the page write.
- DMA descriptor 7 reads back the NAND status and compares the status with an expected value. If there are differences, then the DMA processing engine follows an error-handling DMA descriptor path.
- DMA descriptor 8 disables the BCH engine and emits a GPMI interrupt to indicate that the NAND write has been completed.

9.6.4.2 BCH Decoding for NAND Reads

When a page is read from NAND flash, BCH syndromes will be computed and, if correctable errors are found, they will be corrected on a per block basis within the NAND page. This decoding process is fully overlapped with other NAND data reads and with CPU execution. The BCH decoder flowchart in the figure below shows the steps involved in programming the decoder. The hardware flow of reading and decoding a 4096-byte page is shown in [Figure 9-19](#).

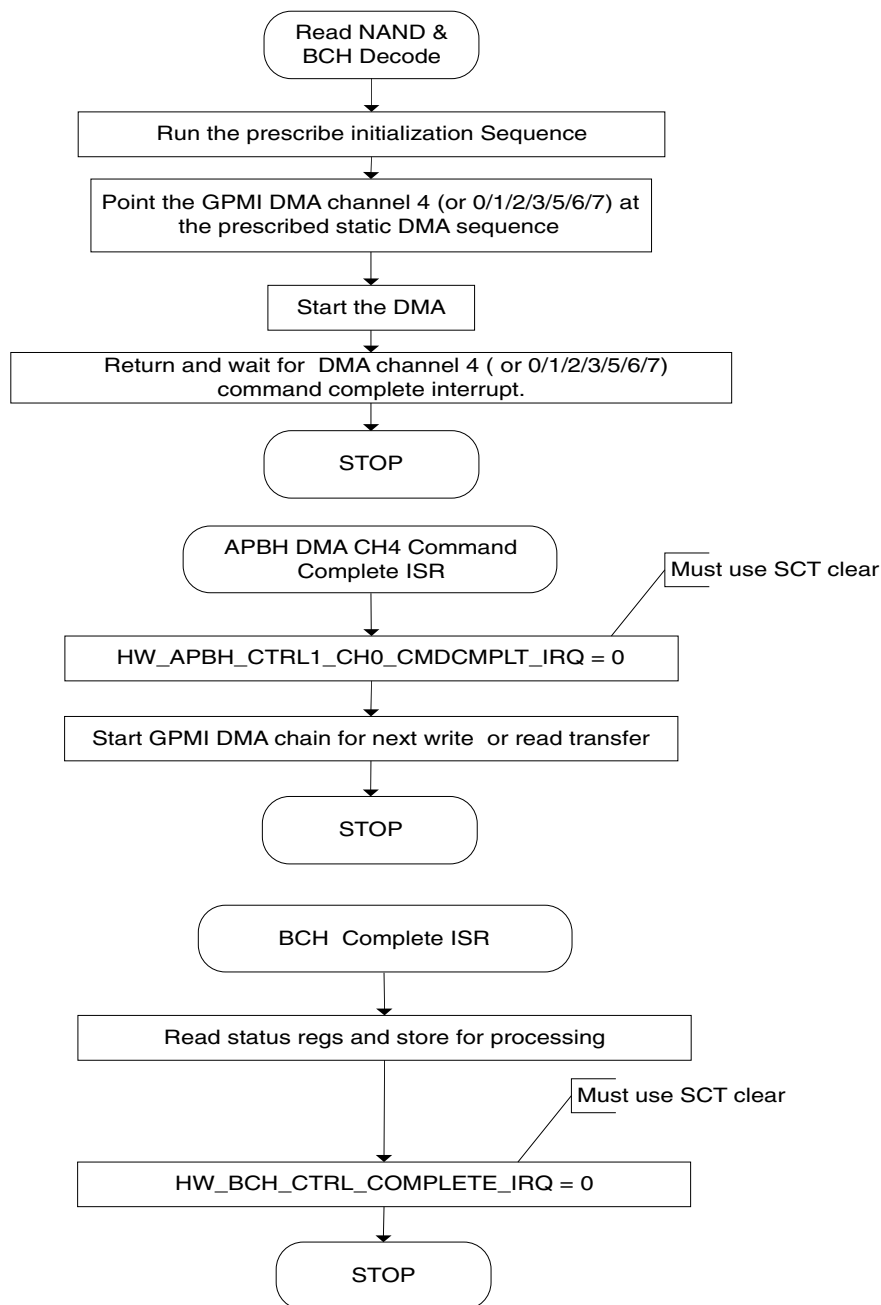


Figure 9-18. BCH Decode Flowchart

Conceptually, an APHB DMA Channel (0,1,2 or 3) command chain with seven command structures linked together is used to perform the BCH decode operation (as shown in [Figure 9-19](#)).

Note

The GPMI's DMA command structures controls the BCH decode operation.

To use the BCH decoder with the GPMI's DMA, create a DMA command chain containing seven descriptor structures, as shown in the figure below and detailed in the DMA structure code example that follows it in [DMA Structure Code Example](#). The seven DMA descriptors perform the following tasks:

1. Issue NAND read setup command byte (under "CLE") and address bytes (under "ALE").
2. Issue NAND read execute command byte (under "CLE").
3. Wait for the NAND device to complete accessing the block data by watching the ready signal.
4. Check for NAND timeout through "PSENSE".
5. Configure and enable the BCH block and read the NAND block data.
6. Disable the BCH block.
7. Descriptor NOP to allow NANDLOCK in the previous descriptor to the thread-safe.

Descriptor 1: Disable BCH engine and issue NAND read set-up command and address (CLE/ALE).

NEXT CMD ADDR										
CMD	<=	1 + 5	3	1	0	0	1	0	1	DMA_READ
BUFFER ADDR										
HW_GPMI_CTRL0	<=	write	8_bit	enabled	2	NAND_CLE	1	1 + 5		
HW_GPMI_COMPARE	<=	null				null				
HW_GPMI_ECCCTRL	<=	----				disable		----		

1 Byte NAND CMD
5 Byte ADDR

Descriptor 2: NAND read execute command (CLE).

NEXT CMD ADDR									
CMD	<=	1	1	1	0	0	1	0	1
DMA_READ									
BUFFER ADDR									
HW_GPMI_CTRL0	<=	write	8_bit	disabled	2	NAND_CLE	0	1	

1 Byte NAND CMD

Descriptor 3: Wait for NAND ready.

NEXT CMD ADDR									
CMD	<=	0	1	1	0	1	0	0	1
NO_DMA_XFER									
BUFFER ADDR									
HW_GPMI_CTRL0	<=	wait_for_ready	8_bit	disabled	2	NAND_DATA	0	0	

Descriptor 4: PSENSE compare for time-out.

NEXT CMD ADDR									
CMD	<=	0	0	0	0	0	0	0	1
DMA_SENSE									
BUFFER ADDR									

DMA Error
Descriptor Chain

Descriptor 5: Enable BCH engine and read NAND data.

NEXT CMD ADDR										
CMD	<=	0	6	1	0	0	1	0	1	NO_DMA_XFER
BUFFER ADDR										
HW_GPMI_CTRL0	<=	read	8_bit	disabled	2	NAND_DATA	0	4096+218		
HW_GPMI_COMPARE	<=	null				null				
HW_GPMI_ECCCTRL	<=	decode_8_bit			enable			0x1FF		
HW_GPMI_ECCCOUNT	<=	4096+218 (flash page size)								
HW_GPMI_PAYLOAD										
HW_GPMI_AUXILIARY										

8*512 Byte Data
Payload Buffer
412 Byte Auxiliary
Payload Buffer

Descriptor 6: Disable BCH engine (wait for ready is a NOP here).

NEXT CMD ADDR										
CMD	<=	0	3	1	0	1	1	0	1	NO_DMA_XFER
BUFFER ADDR										
HW_GPMI_CTRL0	<=	wait_for_ready	8_bit	disabled	0	NAND_DATA	0	0		
HW_GPMI_COMPARE	<=	null				null				
HW_GPMI_ECCCTRL	<=	----			disable			----		

Descriptor 7: NOP to ensure NANDLOCK in previous descriptor .

NEXT CMD ADDR									
CMD	<=	0	0	0	0	0	0	0	0
NO_DMA_XFER									
BUFFER ADDR									

Figure 9-19. BCH Decode DMA Descriptor Chain

9.6.4.2.1 DMA Structure Code Example

The following sample code illustrates the coding for one read transaction, consisting of a seven DMA command structure chain for reading all 4096 bytes of payload data (eight 512-byte blocks) and 65 bytes of metadata with the associative parity bytes ($8 * (18) + 9$) from a 4K NAND page sitting on GPMI CS2.

```
//-----
// generic DMA/GPMI/ECC descriptor struct, order sensitive!
//-----
typedef struct {
    // DMA related fields
    unsigned int dma_nxtcmdar;
    unsigned int dma_cmd;
    unsigned int dma_bar;
    // GPMI related fields
    unsigned int gpmi_ctrl0;
    unsigned int gpmi_compare;
    unsigned int gpmi_eccctrl;
    unsigned int gpmi_ecccount;
    unsigned int gpmi_data_ptr;
    unsigned int gpmi_aux_ptr;
} GENERIC_DESCRIPTOR;
//-----
// allocate 7 descriptors for doing a NAND ECC Read
//-----
GENERIC_DESCRIPTOR read[7];
//-----
// DMA descriptor pointer to handle error conditions from psense checks
//-----
unsigned int * dma_error_handler;
//-----
// 7 byte NAND command and address buffer
// any alignment is ok, it is read by the GPMI DMA
// byte 0 is read setup command
// bytes 1-5 is the NAND address
// byte 6 is read execute command
//-----
unsigned char nand_cmd_addr_buffer[7];
//-----
// 4096 byte payload buffer used for reads or writes
// needs to be word aligned
//-----
unsigned int read_payload_buffer[(4096/4)];
//-----
// 412 byte auxiliary buffer used for reads
// needs to be word aligned
//-----
unsigned int read_aux_buffer[(412/4)];
//-----
// Descriptor 1: issue NAND read setup command (CLE/ALE)
//-----
read[0].dma_nxtcmdar = &read[1];
read[0].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT      (1 + 5) | // point to the next descriptor
                  BF_APBH_CHn_CMD_CMDWORDS        (3) | // 1 byte command, 5 byte address
                  BF_APBH_CHn_CMD_WAIT4ENDCMD      (1) | // send 3 words to the GPMI
                  BF_APBH_CHn_CMD_SEMAPHORE        (0) | // wait for command to finish
                  BF_APBH_CHn_CMD_NANDWAIT4READY  (0) | // before continuing
                  BF_APBH_CHn_CMD_NANDLOCK         (1) | // prevent other DMA channels from
                  BF_APBH_CHn_CMD_IRQONCMPLT      (0) | // taking over
                  BF_APBH_CHn_CMD_CHAIN            (1) | // follow chain to next command
                  BV_FLD(APBH_CHn_CMD, COMMAND, DMA_READ); // read data from DMA, write to
```

```

NAND
read[0].dma_bar = &nand_cmd_addr_buffer;          // byte 0 read setup, bytes 1 - 5 NAND
address
// 3 words sent to the GPMI
read[0].gpmi_ctrl0 = BV_FLD(GPMI_CTRL0, COMMAND_MODE, WRITE) | // write to the NAND
                   BV_FLD(GPMI_CTRL0, WORD_LENGTH, 8_BIT) |
                   BV_FLD(GPMI_CTRL0, LOCK_CS, ENABLED) |
                   BF_GPMI_CTRL0_CS (2) | // must correspond to NAND

CS used
                   BV_FLD(GPMI_CTRL0, ADDRESS, NAND_CLE) |
                   BF_GPMI_CTRL0_ADDRESS_INCREMENT (1) | // send command and address
                   BF_GPMI_CTRL0_XFER_COUNT (1 + 5); // 1 byte command, 5 byte

address
read[0].gpmi_compare = NULL;                      // field not used but necessary to set
eccctrl
read[0].gpmi_eccctrl = BV_FLD(GPMI_ECCCTRL, ENABLE_ECC, DISABLE); // disable the ECC block
//-----
// Descriptor 2: issue NAND read execute command (CLE)
//-----
read[1].dma_nxtcmdar = &read[2];                  // point to the next descriptor
read[1].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (1) | // 1 byte read command
                  BF_APBH_CHn_CMD_CMDWORDS (1) | // send 1 word to GPMI
                  BF_APBH_CHn_CMD_WAIT4ENDCMD (1) | // wait for command to finish

before
// continuing
                  BF_APBH_CHn_CMD_SEMAPHORE (0) |
                  BF_APBH_CHn_CMD_NANDWAIT4READY(0) |
                  BF_APBH_CHn_CMD_NANDLOCK (1) | // prevent other DMA channels from
// taking over
                  BF_APBH_CHn_CMD_IRQONCMPLT (0) |
                  BF_APBH_CHn_CMD_CHAIN (1) | // follow chain to next command
                  BV_FLD(APBH_CHn_CMD, COMMAND, DMA_READ); // read data from DMA, write

to NAND
read[1].dma_bar = &nand_cmd_addr_buffer[6];        // point to byte 6, read execute
command
// 1 word sent to the GPMI
read[1].gpmi_ctrl0 = BV_FLD(GPMI_CTRL0, COMMAND_MODE, WRITE) | // write to the NAND
                   BV_FLD(GPMI_CTRL0, WORD_LENGTH, 8_BIT) |
                   BV_FLD(GPMI_CTRL0, LOCK_CS, DISABLED) |
                   BF_GPMI_CTRL0_CS (2) | // must correspond to NAND

CS used
                   BV_FLD(GPMI_CTRL0, ADDRESS, NAND_CLE) |
                   BF_GPMI_CTRL0_ADDRESS_INCREMENT (0) |
                   BF_GPMI_CTRL0_XFER_COUNT (1); // 1 byte command
//-----
// Descriptor 3: wait for ready (DATA)
//-----
read[2].dma_nxtcmdar = &read[3];                  // point to the next descriptor
read[2].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (0) | // no dma transfer
                  BF_APBH_CHn_CMD_CMDWORDS (1) | // send 1 word to GPMI
                  BF_APBH_CHn_CMD_WAIT4ENDCMD (1) | // wait for command to finish

before
// continuing
                  BF_APBH_CHn_CMD_SEMAPHORE (0) |
                  BF_APBH_CHn_CMD_NANDWAIT4READY(1) | // wait for nand to be ready
                  BF_APBH_CHn_CMD_NANDLOCK (0) | // relinquish nand lock
                  BF_APBH_CHn_CMD_IRQONCMPLT (0) |
                  BF_APBH_CHn_CMD_CHAIN (1) | // follow chain to next command
                  BV_FLD(APBH_CHn_CMD, COMMAND, NO_DMA_XFER); // no dma transfer
read[2].dma_bar = NULL;                          // field not used
// 1 word sent to the GPMI
read[2].gpmi_ctrl0 = BV_FLD(GPMI_CTRL0, COMMAND_MODE, WAIT_FOR_READY) | // wait for NAND
ready
                   BV_FLD(GPMI_CTRL0, WORD_LENGTH, 8_BIT) |
                   BV_FLD(GPMI_CTRL0, LOCK_CS, DISABLED) |
                   BF_GPMI_CTRL0_CS (2) | // must correspond

to NAND CS used
                   BV_FLD(GPMI_CTRL0, ADDRESS, NAND_DATA) |
                   BF_GPMI_CTRL0_ADDRESS_INCREMENT (0) |
                   BF_GPMI_CTRL0_XFER_COUNT (0);

```

62BIT Correcting ECC Accelerator (BCH)

```
//-----
// Descriptor 4: psense compare (time out check)
//-----
read[3].dma_nxtcmdar = &read[4]; // point to the next
descriptor
read[3].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (0) | // no dma transfer
                  BF_APBH_CHn_CMD_CMDWORDS (0) | // no words sent to GPMI
                  BF_APBH_CHn_CMD_WAIT4ENDCMD (0) | // do not wait to continue
                  BF_APBH_CHn_CMD_SEMAPHORE (0) |
                  BF_APBH_CHn_CMD_NANDWAIT4READY(0) |
                  BF_APBH_CHn_CMD_NANDLOCK (0) |
                  BF_APBH_CHn_CMD_IRQONCMPLT (0) |
                  BF_APBH_CHn_CMD_CHAIN (1) | // follow chain to next
command
                  BV_FLD(APBH_CHn_CMD, COMMAND, DMA_SENSE); // perform a sense check
read[3].dma_bar = dma_error_handler; // if sense check fails, branch to
error handler
//-----
// Descriptor 5: read 4K page plus 65 byte meta-data Nand data
// and send it to ECC block (DATA)
//-----
read[4].dma_nxtcmdar = &read[5]; // point to the next descriptor
read[4].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (0) | // no dma transfer
                  BF_APBH_CHn_CMD_CMDWORDS (6) | // send 6 words to GPMI
                  BF_APBH_CHn_CMD_WAIT4ENDCMD (1) | // wait for command to finish before
// continuing
                  BF_APBH_CHn_CMD_SEMAPHORE (0) |
                  BF_APBH_CHn_CMD_NANDWAIT4READY(0) |
                  BF_APBH_CHn_CMD_NANDLOCK (1) | // prevent other DMA channels from
taking over
                  BF_APBH_CHn_CMD_IRQONCMPLT (0) | // ECC block generates BCH interrupt
// on completion
                  BF_APBH_CHn_CMD_CHAIN (1) | // follow chain to next command
                  BV_FLD(APBH_CHn_CMD, COMMAND, NO_DMA_XFER); // no DMA transfer,
// ECC block handles
transfer
read[4].dma_bar = NULL; // field not used
// 6 words sent to the GPMI
read[4].gpmi_ctrl0 = BV_FLD(GPMI_CTRL0, COMMAND_MODE, READ) | // read from the NAND
                    BV_FLD(GPMI_CTRL0, WORD_LENGTH, 8_BIT) |
                    BV_FLD(GPMI_CTRL0, LOCK_CS, DISABLED) |
                    BF_GPMI_CTRL0_CS (2) | // must correspond to
NAND CS used
                    BV_FLD(GPMI_CTRL0, ADDRESS, NAND_DATA) |
                    BF_GPMI_CTRL0_ADDRESS_INCREMENT (0) |
                    BF_GPMI_CTRL0_XFER_COUNT (4096+218); // eight 512 byte data
blocks
// metadata, and parity

read[4].gpmi_compare = NULL; // field not used but necessary to set
eccctrl
// GPMI ECCCTRL PIO This launches the 4K byte transfer through BCH's
// bus master. Setting the ECC_ENABLE bit redirects the data flow
// within the GPMI so that read data flows to the BCH engine instead
// of flowing to the GPMI's DMA channel.
read[4].gpmi_eccctrl = BV_FLD(GPMI_ECCCTRL, ECC_CMD, DECODE_8_BIT) | // specify t = 8
mode
                    BV_FLD(GPMI_ECCCTRL, ENABLE_ECC, ENABLE) | // enable ECC
module
                    BF_GPMI_ECCCTRL_BUFFER_MASK (0X1FF); // read all 8 data blocks
and 1 aux block
read[4].gpmi_ecccount = BF_GPMI_ECCCOUNT_COUNT(4096+218); // specify number of bytes
// read from NAND
read[4].gpmi_data_ptr = &read_payload_buffer; // pointer for the 4K byte
// data area
read[4].gpmi_aux_ptr = &read_aux_buffer; // pointer for the 65 byte
aux area +
// parity and syndrome
bytes for both
// data and aux blocks.
```



```

//-----
// Descriptor 6: disable ECC block
//-----
read[5].dma_nextcmdar = &read[6]; // point to the next descriptor
read[5].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (0) | // no dma transfer
BF_APBH_CHn_CMD_CMDWORDS (3) | // send 3 words to GPMI
BF_APBH_CHn_CMD_WAIT4ENDCMD (1) | // wait for command to finish

before

// continuing
BF_APBH_CHn_CMD_SEMAPHORE (0) |
BF_APBH_CHn_CMD_NANDWAIT4READY(1) | // wait for nand to be ready
BF_APBH_CHn_CMD_NANDLOCK (1) | // need nand lock to be
// thread safe while turn-off BCH

BF_APBH_CHn_CMD_IRQONCMPLT (0) |
BF_APBH_CHn_CMD_CHAIN (1) | // follow chain to next command
BV_FLD(APBH_CHn_CMD, COMMAND, NO_DMA_XFER); // no dma transfer
read[5].dma_bar = NULL; // field not used
// 3 words sent to the GPMI
read[5].gpmi_ctrl0 = BV_FLD(GPMI_CTRL0, COMMAND_MODE, READ) |
BV_FLD(GPMI_CTRL0, WORD_LENGTH, 8_BIT) |
BV_FLD(GPMI_CTRL0, LOCK_CS, DISABLED) |
BF_GPMI_CTRL0_CS (2) | // must correspond to

NAND CS used

BV_FLD(GPMI_CTRL0, ADDRESS, NAND_DATA) |
BF_GPMI_CTRL0_ADDRESS_INCREMENT (0) |
BF_GPMI_CTRL0_XFER_COUNT (0);
read[5].gpmi_compare = NULL; // field not used but necessary to set
eccctrl
read[5].gpmi_eccctrl = BV_FLD(GPMI_ECCCTRL, ENABLE_ECC, DISABLE); // disable the ECC block
//-----
// Descriptor 7: deassert nand lock
//-----
read[6].dma_nextcmdar = NULL; // not used since this is last
descriptor
read[6].dma_cmd = BF_APBH_CHn_CMD_XFER_COUNT (0) | // no dma transfer
BF_APBH_CHn_CMD_CMDWORDS (0) | // no words sent to GPMI
BF_APBH_CHn_CMD_WAIT4ENDCMD (0) | // wait for command to finish

before

// continuing
BF_APBH_CHn_CMD_SEMAPHORE (0) |
BF_APBH_CHn_CMD_NANDWAIT4READY(0) |
BF_APBH_CHn_CMD_NANDLOCK (0) | // relinquish nand lock
BF_APBH_CHn_CMD_IRQONCMPLT (0) | // BCH engine generates interrupt
BF_APBH_CHn_CMD_CHAIN (0) | // terminate DMA chain processing
BV_FLD(APBH_CHn_CMD, COMMAND, NO_DMA_XFER); // no dma transfer
read[6].dma_bar = NULL; // field not used

```

9.6.4.2.2 Using the Decoder

As illustrated in [Figure 9-19](#) and the sample code in [DMA Structure Code Example](#) :

- DMA descriptor 1 prepares the NAND for data read by using the GPMI to issue a NAND read setup command byte under CLE, then sends a 5-byte address under ALE. The BCH engine is not used for these commands.
- DMA descriptor 2 issues a one-byte read execute command to the NAND device that triggers its read access. The NAND then goes not ready.
- DMA descriptor 3 performs a wait for ready operation allowing the DMA chain to remain dormant until the NAND device completes its read access time.

- DMA descriptor 5 handles the reading and error correction of the NAND data. This command's PIOs activate the BCH engine to write the read NAND data to system memory and to process it for any errors that need to be corrected. This DMA descriptor contains two PIO values that are system memory addresses pointing to the PAYLOAD data area and to the AUXILIARY data area. These addresses are used by the BCH engine's AHB master to move data into system memory and to correct it. While this example is reading an entire 4K page—payload plus metadata—it is equally possible to read just one 512-byte payload block or just the uniquely protected metadata block in a single 7 DMA structure transfer.
- DMA descriptor 6 disables the BCH engine with the NANDLOCK asserted. This is necessary to ensure that the GPMI resource is not arbitrated to another DMA channel when multiple DMA channels are active concurrently.
- DMA descriptor 7 de-asserts the NANDLOCK to free up the GPMI resource to another channel.

As the BCH block receives data from the GPMI:

- The decoder transforms the read NAND data block into a BCH code word and computes the codeword syndrome.
- If no errors are present, then the BCH block can immediately report back to firmware. This report is passed as the BCH_CTRL_COMPLETE_IRQ interrupt status bit and the associated status registers in BCH_STATUS0/1 registers.
- If an error is present, then the BCH block corrects the necessary data block or parity block bytes, if possible (not all errors are correctable).

As the BCH decoder reads the data and parity blocks, it records a special condition, i.e., that all of the bits of a payload data block or metadata block are one, including any associated parity bytes. The all-ones case for both parity and data indicates an erased block in the NAND device.

The BCH_STATUS0 register contains a 4-bit field that indicates the final status of the auxiliary block. A value of 0x0 indicates no errors found for a block.

- A value of 1 to 20 inclusive indicates that many correctable errors were found and fixed.
- A value of 0xFE indicates uncorrectable errors detected on the block.

- A value of 0xFF indicates that the block was in the special ALL ONES state and is therefore considered to be an ERASED block.
- All other values are disallowed by the hardware design.

Recall that up to eight NAND devices can have DMA chains in-flight at once, i.e. they can all be contending for access to the GPMI data bus. It is impossible to predict which NAND device will enter the BCH engine with a transfer first, because each chain includes a wait4ready command structure. As a result, firmware should look at the BCH_STATUS0_COMPLETED_CE bit field to determine which block is being reported in the status register. There is also a 16-bit HANDLE field in the GPMI_ECCCTRL register that is passed down the pipeline with each transaction. This handle field can be used to speed firmware's detection of which transaction is being reported.

These examples of reading and writing have focused on full page transfers of 4K page NAND devices. Other device configurations can be specified by changing the ECCOUNT field in the GPMI registers and reprogramming the BCH's FLASHnLAYOUTm registers.

The BCH and GPMI blocks are designed to be very efficient at reading single 512 (or 1024)-byte pages in one transaction. With no errors, the transaction takes less than 20 HCLKs longer than the time to read the raw data from the NAND.

To summarize, the APBH DMA command chain for a BCH decode operation is shown in [Figure 9-19](#). Seven DMA command structures must be present for each NAND read transaction decoded by the BCH. The seven DMA command structures for multiple NAND read transaction blocks can be chained together to make larger units of work for the BCH, and each will produce an appropriate error report in the BCH PIO space. Multiple NAND devices can have such multiple chains scheduled. The results can come back out of order with respect to the multiple chains.

9.6.4.3 Interrupts

There are two interrupt sources used in processing BCH protected NAND read and write transfers.

Since all BCH operations are initiated by GPMI DMA command structures, the DMA completion interrupt for the GPMI is an important ISR. Both of the flow charts of [Figure 9-15](#) and [Figure 9-18](#) show the GPMI DMA complete ISR skeleton. In both reads and writes, the GPMI DMA completion interrupt is used to schedule work *INTO* the error correction pipeline. As the front end processing completes, the DMA interrupt is

generated and additional work, such as DMA chains, are passed to the GPMI DMA to keep it *fed*. For write operations, this is the only interrupt that is generated for processing the NAND write transfer.

For reads, however, two interrupts are needed. Every read is started by a GPMI DMA command chain and the front end queue is fed as described above. The back end of the read pipeline is drained by monitoring the BCH completion interrupt found in `HW_BCH_CTRL_COMPLETE_IRQ`.

An BCH transaction consists of reading or writing all of the blocks requested in the `HW_GPMI_ECCCTRL_BUFFER_MASK` bit field. As every read transaction completes, it posts the status of all of the blocks to the `HW_BCH_STATUS0` and `HW_BCH_STATUS1` registers and sets the completion interrupt. The five stages of the BCH read pipeline completes, one in the GPMI and four in the BCH, are independently stalled as they complete and try to deliver to the next stage in the data flow. Several of these stages can be skipped if no-errors are found or once an uncorrectable error is found in a block.

In any case, the final stage will stall if the status register is busy waiting for the CPU to take status register results. The hardware monitors the state of the `HW_BCH_CTRL_COMPLETE_IRQ` bit. If it is still set when the last pipeline stage is ready to post data, then the stage will stall. It follows that the next previous stage will stall when it is ready to hand off work to the final stage, and so on up the pipeline.

CAUTION

It is important that firmware read the STATUS0/1 results and save them before clearing the interrupt request bit. Otherwise, a transaction and its results could be completely lost.

9.6.4.4 Randomizer

BCH ECC has a Randomizer module that is interfaced through the GPMI APBHDMA chain. The Randomizer can generate random data based on BCH ECC encoded/decoded data. It can be employed to reduce the disturbances caused by a neighboring cell in the NAND chip, thus reducing bit errors.

To enable the Randomizer module, set `GPMI_ECCCTRL[RANDOMIZER_ENABLE]` to 1, then set `GPMI_ECCCOUNT[RANDOMIZER_PAGE]` to select randomizer page number needed to be randomized. All these registers can be programmed by the DMA chain. The randomized data should start from the zero column address and be the size of the whole NAND page. If the randomizer function is enabled,

GPMI_ECCCTRL[ENABLE_ECC] should also be enabled. To bypass BCH error correction function, set BCH_FLASHxLAYOUT0[ECC0] and BCH_FLASHxLAYOUT1[ECCN] to 0.

9.6.5 Behavior During Reset

A soft reset (SFTRST) can take multiple clock periods to complete, so do NOT set CLKGATE when setting SFTRST.

The reset process gates the clocks automatically. The example code is shown below.

```
// A soft reset can take multiple clocks to complete, so do NOT gate the
// clock when setting soft reset. The reset process will gate the clock
// automatically. Poll until this has happened before subsequently
// preparing soft-reset and clock gate
BCH_CTRL_CLR(BM_BCH_CTRL_SFTRST);
BCH_CTRL_CLR(BM_BCH_CTRL_CLKGATE);
// asserting soft-reset
BCH_CTRL_SET(BM_BCH_CTRL_SFTRST);
// waiting for confirmation of soft-reset
while (!BCH_CTRL.B.CLKGATE)
{
// busy wait
}
// Done.
BCH_CTRL_CLR(BM_BCH_CTRL_SFTRST);
BCH_CTRL_CLR(BM_BCH_CTRL_CLKGATE);
```

9.6.6 BCH Memory Map/Register Definition

BCH Hardware Register Format Summary

BCH memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3300_4000	Hardware BCH ECC Accelerator Control Register (BCH_CTRL)	32	R/W	C000_0000h	9.6.6.1/2459
3300_4004	Hardware BCH ECC Accelerator Control Register (BCH_CTRL_SET)	32	R/W	C000_0000h	9.6.6.1/2459
3300_4008	Hardware BCH ECC Accelerator Control Register (BCH_CTRL_CLR)	32	R/W	C000_0000h	9.6.6.1/2459
3300_400C	Hardware BCH ECC Accelerator Control Register (BCH_CTRL_TOG)	32	R/W	C000_0000h	9.6.6.1/2459
3300_4010	Hardware ECC Accelerator Status Register 0 (BCH_STATUS0)	32	R	0000_0010h	9.6.6.2/2461

Table continues on the next page...

BCH memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3300_4020	Hardware ECC Accelerator Mode Register (BCH_MODE)	32	R/W	0000_0000h	9.6.6.3/ 2463
3300_4030	Hardware BCH ECC Loopback Encode Buffer Register (BCH_ENCODEPTR)	32	R/W	0000_0000h	9.6.6.4/ 2464
3300_4040	Hardware BCH ECC Loopback Data Buffer Register (BCH_DATAPTR)	32	R/W	0000_0000h	9.6.6.5/ 2464
3300_4050	Hardware BCH ECC Loopback Metadata Buffer Register (BCH_METAPTR)	32	R/W	0000_0000h	9.6.6.6/ 2465
3300_4070	Hardware ECC Accelerator Layout Select Register (BCH_LAYOUTSELECT)	32	R/W	E4E4_E4E4h	9.6.6.7/ 2465
3300_4080	Hardware BCH ECC Flash 0 Layout 0 Register (BCH_FLASH0LAYOUT0)	32	R/W	070A_4080h	9.6.6.8/ 2466
3300_4090	Hardware BCH ECC Flash 0 Layout 1 Register (BCH_FLASH0LAYOUT1)	32	R/W	10DA_4080h	9.6.6.9/ 2468
3300_40A0	Hardware BCH ECC Flash 1 Layout 0 Register (BCH_FLASH1LAYOUT0)	32	R/W	070A_4080h	9.6.6.10/ 2469
3300_40B0	Hardware BCH ECC Flash 1 Layout 1 Register (BCH_FLASH1LAYOUT1)	32	R/W	10DA_4080h	9.6.6.11/ 2471
3300_40C0	Hardware BCH ECC Flash 2 Layout 0 Register (BCH_FLASH2LAYOUT0)	32	R/W	070A_4080h	9.6.6.12/ 2472
3300_40D0	Hardware BCH ECC Flash 2 Layout 1 Register (BCH_FLASH2LAYOUT1)	32	R/W	10DA_4080h	9.6.6.13/ 2473
3300_40E0	Hardware BCH ECC Flash 3 Layout 0 Register (BCH_FLASH3LAYOUT0)	32	R/W	070A_4080h	9.6.6.14/ 2474
3300_40F0	Hardware BCH ECC Flash 3 Layout 1 Register (BCH_FLASH3LAYOUT1)	32	R/W	10DA_4080h	9.6.6.15/ 2476
3300_4100	Hardware BCH ECC Debug Register0 (BCH_DEBUG0)	32	R/W	0000_0000h	9.6.6.16/ 2477
3300_4104	Hardware BCH ECC Debug Register0 (BCH_DEBUG0_SET)	32	R/W	0000_0000h	9.6.6.16/ 2477
3300_4108	Hardware BCH ECC Debug Register0 (BCH_DEBUG0_CLR)	32	R/W	0000_0000h	9.6.6.16/ 2477
3300_410C	Hardware BCH ECC Debug Register0 (BCH_DEBUG0_TOG)	32	R/W	0000_0000h	9.6.6.16/ 2477
3300_4110	KES Debug Read Register (BCH_DBGKESREAD)	32	R	0000_0000h	9.6.6.17/ 2479
3300_4120	Chien Search Debug Read Register (BCH_DBGCSFEREAD)	32	R	0000_0000h	9.6.6.18/ 2479
3300_4130	Syndrome Generator Debug Read Register (BCH_DBGSYNDGENREAD)	32	R	0000_0000h	9.6.6.19/ 2479
3300_4140	Bus Master and ECC Controller Debug Read Register (BCH_DBGAHBMREAD)	32	R	0000_0000h	9.6.6.20/ 2480
3300_4150	Block Name Register (BCH_BLOCKNAME)	32	R	2048_4342h	9.6.6.21/ 2480

Table continues on the next page...

BCH memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3300_4160	BCH Version Register (BCH_VERSION)	32	R	0100_0000h	9.6.6.22/2481
3300_4170	Hardware BCH ECC Debug Register 1 (BCH_DEBUG1)	32	R/W	0000_0000h	9.6.6.23/2482

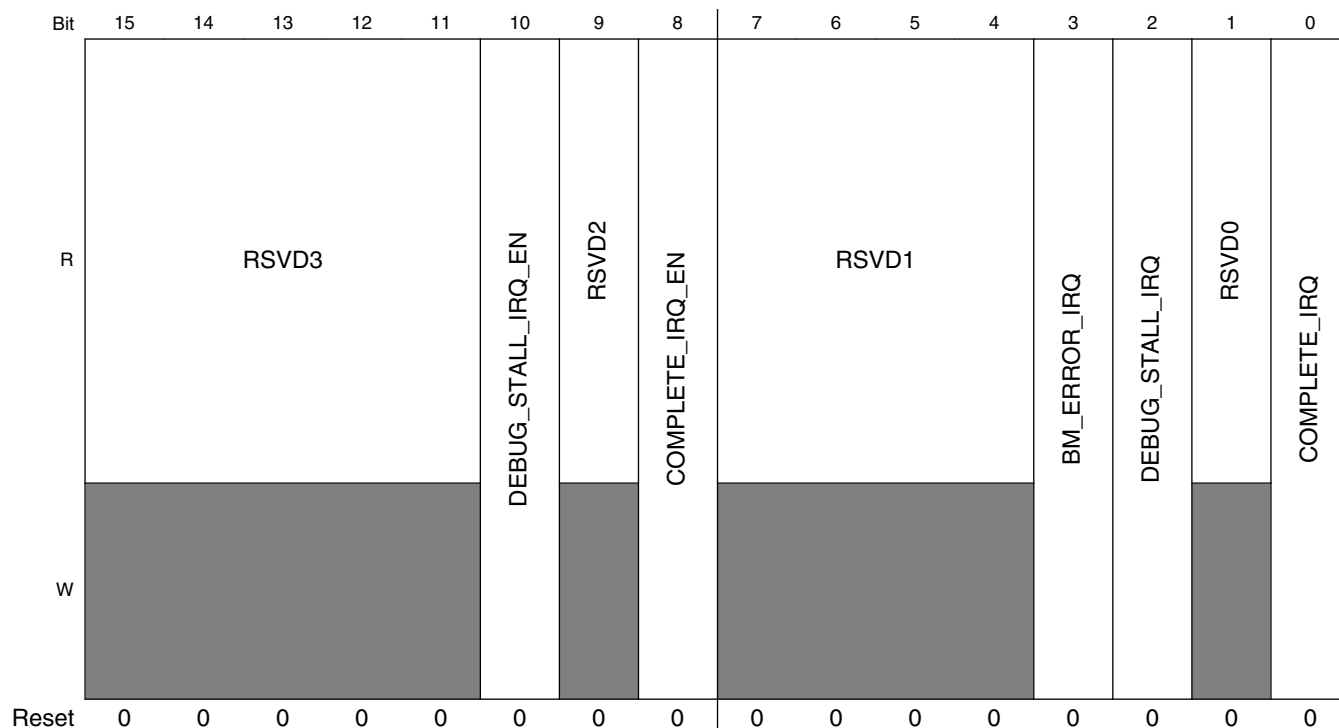
9.6.6.1 Hardware BCH ECC Accelerator Control Register (BCH_CTRL_n)

The BCH CTRL provides overall control of the hardware ECC accelerator

Address: 3300_4000h base + 0h offset + (4d × i), where i=0d to 3d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16				
R	SFTRST		CLKGATE		RSVD5								DEBUGSYNDROME		RSVD4		M2M_LAYOUT		M2M_ENCODE	M2M_ENABLE
W																				
Reset	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0				

62BIT Correcting ECC Accelerator (BCH)



BCH_CTRLn field descriptions

Field	Description
31 SFTRST	Set this bit to 0 to enable normal BCH operation. Set this bit to 1 (default) to disable clocking with the BCH and hold it in its reset (lowest power) state. This bit can be turned on and then off to reset the BCH block to its default state. This bit resets all state machines except for the AHB master state machine 0x0 RUN — Allow BCH to operate normally. 0x1 RESET — Hold BCH in reset.
30 CLKGATE	This bit must be set to 0 for normal operation. When set to 1 it gates off the clocks to the block. 0x0 RUN — Allow BCH to operate normally. 0x1 NO_CLKS — Do not clock BCH gates in order to minimize power consumption.
29–23 RSVD5	This field is reserved. This read-only field is reserved and always has the value 0.
22 DEBUGSYNDROME	(For debug purposes only). Enable write of computed syndromes to memory on BCH decode operations. Computed syndromes will be written to the auxiliary buffer after the status block. Syndromes will be written as padded 16-bit values.
21–20 RSVD4	This field is reserved. This read-only field is reserved and always has the value 0
19–18 M2M_LAYOUT	Selects the flash page format for memory-to-memory operations.
17 M2M_ENCODE	Selects encode (parity generation) or decode (correction) mode for memory-to-memory operations.
16 M2M_ENABLE	NOTE! WRITING THIS BIT INITIATES A MEMORY-TO-MEMORY OPERATION. The BCH module must be inactive (not processing data from the GPML) when this bit is set. The M2M_ENCODE and

Table continues on the next page...

BCH_CTRLn field descriptions (continued)

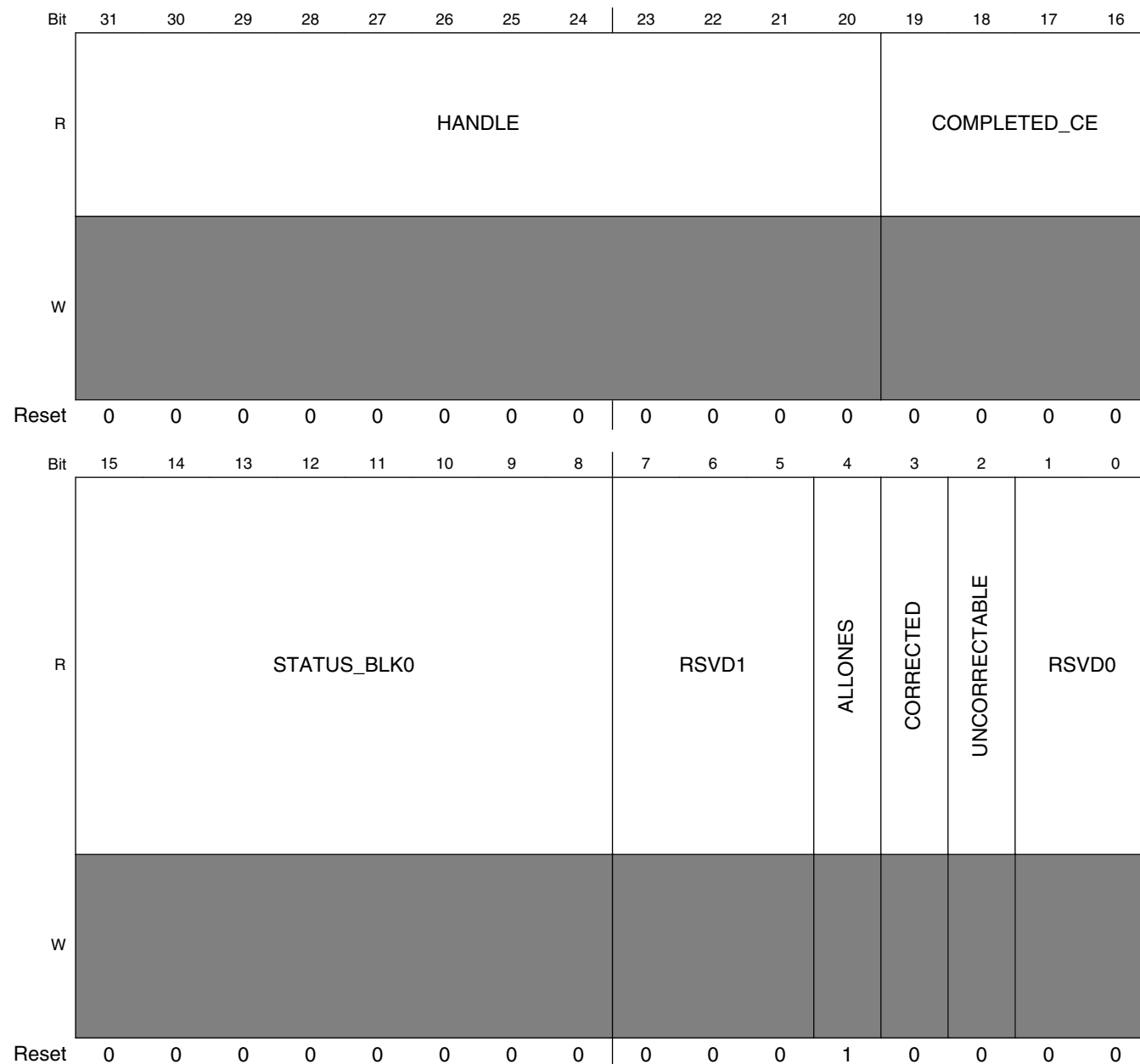
Field	Description
	M2M_LAYOUT bits as well as the ENCODEPTR, DATAPTR, and METAPTR registers are used for memory-to-memory operations and must be correctly programmed before writing this bit.
15–11 RSVD3	This field is reserved. This read-only field is reserved and always has the value 0
10 DEBUG_STALL_ IRQ_EN	1 = interrupt on debug stall mode is enabled. The IRQ is raised on every block
9 RSVD2	This field is reserved. This read-only field is reserved and always has the value 0.
8 COMPLETE_IRQ_ EN	1 = interrupt on completion of correction is enabled.
7–4 RSVD1	This field is reserved. This read-only field is reserved and always has the value 0.
3 BM_ERROR_IRQ	AHB Bus interface Error Interrupt Status. Write a 1 to the SCT clear address to clear the interrupt status bit.
2 DEBUG_STALL_IRQ	DEBUG STALL Interrupt Status. Write a 1 to the SCT clear address to clear the interrupt status bit.
1 RSVD0	This field is reserved. This read-only field is reserved and always has the value 0.
0 COMPLETE_IRQ	This bit indicates the state of the external interrupt line. Write a 1 to the SCT clear address to clear the interrupt status bit. NOTE: subsequent ECC completions will be held off as long as this bit is set. Be sure to read the data from BCH_STATUS0, 1 before clearing this interrupt bit.

9.6.6.2 Hardware ECC Accelerator Status Register 0 (BCH_STATUS0)

The BCH STAT register provides visibility into the run-time status of the BCH and status information when processing is complete. It provides overall status of the hardware ECC accelerator.

62BIT Correcting ECC Accelerator (BCH)

Address: 3300_4000h base + 10h offset = 3300_4010h



BCH_STATUS0 field descriptions

Field	Description
31–20 HANDLE	Software supplies a 12 bit handle for this transfer as part of the GPMI DMA PIO operation that started the transaction. That handle passes down the pipeline and ends up here at the time the BCH interrupt is signaled.
19–16 COMPLETED_CE	This is the chip enable number corresponding to the NAND device from which this data came.
15–8 STATUS_BLK0	Count of symbols in error during processing of first block of flash (metadata block). The number of errors reported will be in the range of 0 to the ECC correction level for block 0. 0x00 ZERO — No errors found on block.

Table continues on the next page...

BCH_STATUS0 field descriptions (continued)

Field	Description
	0x01 ERROR1 — One error found on block. 0x02 ERROR2 — One errors found on block. 0x03 ERROR3 — One errors found on block. 0x04 ERROR4 — One errors found on block. 0xFE UNCORRECTABLE — Block exhibited uncorrectable errors. 0xFF ERASED — Page is erased.
7–5 RSVD1	This field is reserved. This read-only field is reserved and always has the value 0.
4 ALLONES	1 = All data bits of this transaction are ONE.
3 CORRECTED	1 = At least one correctable error encountered during last processing cycle.
2 UNCORRECTABLE	1 = Uncorrectable error encountered during last processing cycle.
RSVD0	This field is reserved. This read-only field is reserved and always has the value 0.

9.6.6.3 Hardware ECC Accelerator Mode Register (BCH_MODE)

The BCH MODE register provides additional mode controls.

Contains additional global mode controls for the BCH engine.

Address: 3300_4000h base + 20h offset = 3300_4020h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	RSVD																ERASE_THRESHOLD															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

BCH_MODE field descriptions

Field	Description
31–8 RSVD	This field is reserved. This read-only field is reserved and always has the value 0.
ERASE_ THRESHOLD	This value indicates the maximum number of zero bits on a flash subpage for it to be considered erased. For SLC NAND devices, this value should be programmed to 0 (meaning that the entire page should consist of bytes of 0xFF. For MLC NAND devices, bit errors may occur on reads (even on blank pages), so this threshold can be used to tune the erased page checking algorithm.

9.6.6.4 Hardware BCH ECC Loopback Encode Buffer Register (BCH_ENCODEPTR)

When performing memory to memory operations, indicates the address of the encode buffer. This register should be programmed before writing a 1 to the M2M_ENABLE bit in the CTRL register.

For memory to memory operations, this register is used as the pointer to the encoded data, which is an output when encoding and an input while decoding.

Address: 3300_4000h base + 30h offset = 3300_4030h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ADDR																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

BCH_ENCODEPTR field descriptions

Field	Description
ADDR	Address pointer to encode buffer. This is the source for decode operations and the destination for encode operations. This value must be aligned on a 4 bytes boundary.

9.6.6.5 Hardware BCH ECC Loopback Data Buffer Register (BCH_DATAPTR)

When performing memory to memory operations, indicates the address of the data buffer.

For memory to memory operations, this register is used as the pointer to the data to encode or the destination buffer for decode operations.

Address: 3300_4000h base + 40h offset = 3300_4040h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ADDR																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

BCH_DATAPTR field descriptions

Field	Description
ADDR	Address pointer to data buffer. This is the source for encode operations and the destination for decode operations. This register should be programmed before writing a 1 to the M2M_ENABLE bit in the CTRL register. This value must be aligned on a 4 byte boundary.

9.6.6.6 Hardware BCH ECC Loopback Metadata Buffer Register (BCH_METAPTR)

When performing memory to memory operations, indicates the address of the metadata buffer.

For memory to memory operations, this register is used as the pointer to the metadata to encode or the extracted metadata for decode operations.

Address: 3300_4000h base + 50h offset = 3300_4050h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ADDR																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

BCH_METAPTR field descriptions

Field	Description
ADDR	Address pointer to metadata buffer. This is the source for encode metadata read operations and the destination for metadata decode operations. This register should be programmed before writing a 1 to the M2M_ENABLE bit in the CTRL register. This value must be aligned on a 4 bytes boundary.

9.6.6.7 Hardware ECC Accelerator Layout Select Register (BCH_LAYOUTSELECT)

The BCH LAYOUTSELECT register provides a mapping of chip selects to layout registers.

When the BCH engine receives a request to process a data block from the GPMI interface, it will use this register to map the incoming chip select to one of the four possible flash layout registers

Address: 3300_4000h base + 70h offset = 3300_4070h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	CS15_SELECT				CS14_SELECT				CS13_SELECT				CS12_SELECT			
W																
Reset	1	1	1	0	0	1	0	0	1	1	1	0	0	1	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CS7_SELECT				CS6_SELECT				CS5_SELECT				CS4_SELECT			
W																
Reset	1	1	1	0	0	1	0	0	1	1	1	0	0	1	0	0

BCH_LAYOUTSELECT field descriptions

Field	Description
31–30 CS15_SELECT	Selects which layout is used for chip select 15.
29–28 CS14_SELECT	Selects which layout is used for chip select 14.
27–26 CS13_SELECT	Selects which layout is used for chip select 13.
25–24 CS12_SELECT	Selects which layout is used for chip select 12.
23–22 CS11_SELECT	Selects which layout is used for chip select 11.
21–20 CS10_SELECT	Selects which layout is used for chip select 10.
19–18 CS9_SELECT	Selects which layout is used for chip select 9.
17–16 CS8_SELECT	Selects which layout is used for chip select 8.
15–14 CS7_SELECT	Selects which layout is used for chip select 7.
13–12 CS6_SELECT	Selects which layout is used for chip select 6.
11–10 CS5_SELECT	Selects which layout is used for chip select 5.
9–8 CS4_SELECT	Selects which layout is used for chip select 4.
7–6 CS3_SELECT	Selects which layout is used for chip select 3.
5–4 CS2_SELECT	Selects which layout is used for chip select 2.
3–2 CS1_SELECT	Selects which layout is used for chip select 1.
CS0_SELECT	Selects which layout is used for chip select 0.

9.6.6.8 Hardware BCH ECC Flash 0 Layout 0 Register (BCH_FLASH0LAYOUT0)

The flash format register contains a description of the logical layout of data on the flash device. This register is used in conjunction with the FLASH0LAYOUT1 register to control the format for the devices selecting layout 0 in the LAYOUTSELECT register.

Each pair of layout registers describes one of four supported flash configurations. Software should program the LAYOUTSELECT register for each supported GPMI chip select to select from one of the four layout values. Each pair of registers contains settings that are used by the BCH block while reading / writing the flash page to control data,

metadata, and flash page sizes as well as the ECC correction level. The first block written to flash can be programmed to have different ECC, metadata, and data sizes from subsequent data blocks on the device. In addition, the number of blocks stored on a page of flash is not fixed, but instead is determined by the number of bytes consumed by the initial (block 0) and subsequent data blocks.

See sections [Flash Page Layout](#) and [Determining the ECC layout for a device](#) for more detail information on setting up the flash layout registers.

Address: 3300_4000h base + 80h offset = 3300_4080h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	NBLOCKS								META_SIZE							
W																
Reset	0	0	0	0	0	1	1	1	0	0	0	0	1	0	1	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ECC0					GF13_0_GF14_1	DATA0_SIZE									
W																
Reset	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0

BCH_FLASH0LAYOUT0 field descriptions

Field	Description
31–24 NBLOCKS	Number of subsequent blocks on the flash page (excluding the data0 block). A value of 0 indicates that only the DATA0 block is present and a value of 8 indicates that 8 subsequent blocks are present for a total of 9 blocks on the flash (including the DATA0 block). Any values from 0 to 255 are supported by the hardware.
23–16 META_SIZE	Indicates the size of the metadata (in bytes) to be stored on a flash page. The BCH design support from 0 to 255 bytes for metadata—if set to 0, no metadata will be stored. Metadata is stored before the associated data in block 0. If the DATA0_SIZE field is programmed to a 0, then metadata effectively be stored with its own parity. When both the metadata and data0 fields are programmed with non-zero values, the first block will contain both portions of data and covered by a single parity block.
15–11 ECC0	Indicates the ECC level for the first block on the flash page. The first block covers metadata plus the associated data from the DATA0_SIZE field. 0x0 NONE — No ECC to be performed 0x1 ECC2 — ECC 2 to be performed 0x2 ECC4 — ECC 4 to be performed : — 0x1E ECC60 — ECC 60 to be performed 0x1F ECC62 — ECC 62 to be performed
10 GF13_0_GF14_1	Select GF13 or GF14: 0-GF13; 1-GF14

Table continues on the next page...

BCH_FLASH0LAYOUT0 field descriptions (continued)

Field	Description
DATA0_SIZE	Indicates the size of the data 0 block (in DWORDS / four bytes) to be stored on the flash page. If set to 0, the first block only contains metadata.

9.6.6.9 Hardware BCH ECC Flash 0 Layout 1 Register (BCH_FLASH0LAYOUT1)

The flash format register contains a description of the logical layout of data on the flash device. This register is used in conjunction with the FLASH0LAYOUT0 register to control the format for the device selecting layout 0 in the LAYOUTSELECT register.

Address: 3300_4000h base + 90h offset = 3300_4090h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PAGE_SIZE															
W																
Reset	0	0	0	1	0	0	0	0	1	1	0	1	1	0	1	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ECCN						GF13_0_GF14_1		DATAN_SIZE							
W																
Reset	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0

BCH_FLASH0LAYOUT1 field descriptions

Field	Description
31–16 PAGE_SIZE	Indicates the total size of the flash page (in bytes). This should be set to the page size including spare area. The page size is programmable to accomodate different flash configurations that may be available in the future.
15–11 ECCN	Indicates the ECC level for the subsequent blocks on the flash page (blocks 1-n). Subsequent blocks only contain data (no metadata). 0x0 NONE — No ECC to be performed 0x1 ECC2 — ECC 2 to be performed 0x2 ECC4 — ECC 4 to be performed : —

Table continues on the next page...

BCH_FLASH0LAYOUT1 field descriptions (continued)

Field	Description
	0x1E ECC60 — ECC 60 to be performed 0x1F ECC62 — ECC 62 to be performed
10 GF13_0_GF14_1	Select GF13 or GF14: 0-GF13; 1-GF14
DATAN_SIZE	Indicates the size of the subsequent data blocks (in DWORDS / four bytes) to be stored on the flash page. The size of subsequent data blocks does not have to match the data size for block 0, which is important when metadata is stored separately or for balancing the amount of data stored in each block.

9.6.6.10 Hardware BCH ECC Flash 1 Layout 0 Register (BCH_FLASH1LAYOUT0)

The flash format register contains a description of the logical layout of data on the flash device. This register is used in conjunction with the FLASH1LAYOUT1 register to control the format for the devices selecting layout 1 in the LAYOUTSELECT register.

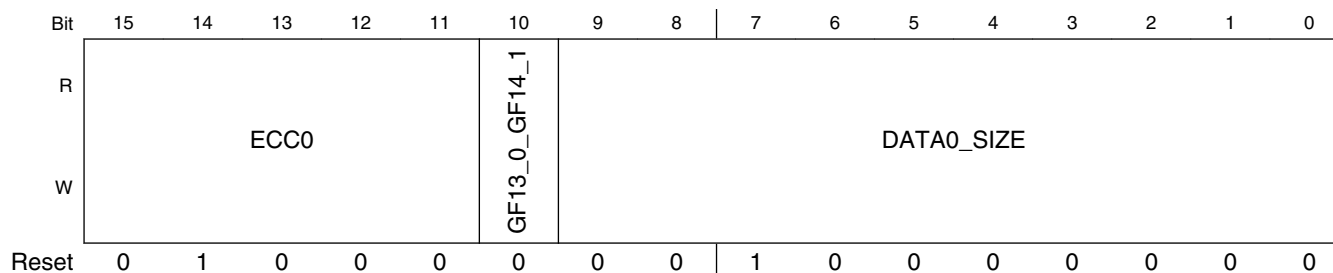
Each pair of layout registers describes one of four supported flash configurations. Software should program the LAYOUTSELECT register for each supported GPMI chip select to select from one of the four layout values. Each pair of registers contains settings that are used by the BCH block while reading / writing the flash page to control data, metadata, and flash page sizes as well as the ECC correction level. The first block written to flash can be programmed to have different ECC, metadata, and data sizes from subsequent data blocks on the device. In addition, the number of blocks stored on a page of flash is not fixed, but instead is determined by the number of bytes consumed by the initial (block 0) and subsequent data blocks.

See sections [Flash Page Layout](#) and [Determining the ECC layout for a device](#) for more detail information on setting up the flash layout registers.

Address: 3300_4000h base + A0h offset = 3300_40A0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	NBLOCKS								META_SIZE							
W																
Reset	0	0	0	0	0	1	1	1	0	0	0	0	1	0	1	0

62BIT Correcting ECC Accelerator (BCH)



BCH_FLASH1LAYOUT0 field descriptions

Field	Description
31–24 NBLOCKS	Number of subsequent blocks on the flash page (excluding the data0 block). A value of 0 indicates that only the DATA0 block is present and a value of 8 indicates that 8 subsequent blocks are present for a total of 9 blocks on the flash (including the DATA0 block). Any values from 0 to 255 are supported by the hardware.
23–16 META_SIZE	Indicates the size of the metadata (in bytes) to be stored on a flash page. The BCH design supports from 0 to 255 bytes for metadata—if set to 0, no metadata will be stored. Metadata is stored before the associated data is in block 0. If the DATA0_SIZE field is programmed to a 0, then metadata effectively be stored with its own parity. When both the metadata and data0 fields are programmed with non-zero values, the first block will contain both portions of data and covered by a single parity block.
15–11 ECC0	Indicates the ECC level for the first block on the flash page. The first block covers metadata plus the associated data from the DATA0_SIZE field. 0x0 NONE — No ECC to be performed 0x1 ECC2 — ECC 2 to be performed 0x2 ECC4 — ECC 4 to be performed : — 0x1E ECC60 — ECC 60 to be performed 0x1F ECC62 — ECC 62 to be performed
10 GF13_0_GF14_1	Select GF13 or GF14: 0-GF13; 1-GF14
DATA0_SIZE	Indicates the size of the data 0 block (in DWORDS / four bytes) to be stored on the flash page. If set to 0, the first block will contains metadata.

9.6.6.11 Hardware BCH ECC Flash 1 Layout 1 Register (BCH_FLASH1LAYOUT1)

The flash format register contains a description of the logical layout of data on the flash device. This register is used in conjunction with the FLASH1LAYOUT0 register to control the format for the device selecting layout 1 in the LAYOUTSELECT register.

Address: 3300_4000h base + B0h offset = 3300_40B0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PAGE_SIZE															
W																
Reset	0	0	0	1	0	0	0	0	1	1	0	1	1	0	1	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ECCN						GF13_0_GF14_1		DATAN_SIZE							
W																
Reset	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0

BCH_FLASH1LAYOUT1 field descriptions

Field	Description
31–16 PAGE_SIZE	Indicates the total size of the flash page (in bytes). This should be set to the page size including spare area. The page size is programmable to accomodate different flash configurations that may be available in the future.
15–11 ECCN	Indicates the ECC level for the subsequent blocks on the flash page (blocks 1-n). Subsequent blocks only contain data (no metadata). 0x0 NONE — No ECC to be performed 0x1 ECC2 — ECC 2 to be performed 0x2 ECC4 — ECC 4 to be performed : — 0x1E ECC60 — ECC 60 to be performed 0x1F ECC62 — ECC 62 to be performed
10 GF13_0_GF14_1	Select GF13 or GF14: 0-GF13; 1-GF14
DATAN_SIZE	Indicates the size of the subsequent data blocks (in DWORDS / four bytes) to be stored on the flash page. The size of subsequent data blocks does not have to match the data size for block 0, which is important when metadata is stored separately or for balancing the amount of data stored in each block.

9.6.6.12 Hardware BCH ECC Flash 2 Layout 0 Register (BCH_FLASH2LAYOUT0)

The flash format register contains a description of the logical layout of data on the flash device. This register is used in conjunction with the FLASH2LAYOUT1 register to control the format for the devices selecting layout 2 in the LAYOUTSELECT register.

Each pair of layout registers describes one of four supported flash configurations. Software should program the LAYOUTSELECT register for each supported GPMI chip select to select from one of the four layout values. Each pair of registers contains settings that are used by the BCH block while reading / writing the flash page to control data, metadata, and flash page sizes as well as the ECC correction level. The first block written to flash can be programmed to have different ECC, metadata, and data sizes from subsequent data blocks on the device. In addition, the number of blocks stored on a page of flash is not fixed, but instead is determined by the number of bytes consumed by the initial (block 0) and subsequent data blocks.

See sections [Flash Page Layout](#) and [Determining the ECC layout for a device](#) for more detail information on setting up the flash layout registers.

Address: 3300_4000h base + C0h offset = 3300_40C0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	NBLOCKS								META_SIZE							
W																
Reset	0	0	0	0	0	1	1	1	0	0	0	0	1	0	1	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ECC0						GF13_0_GF14_1		DATA0_SIZE							
W																
Reset	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0

BCH_FLASH2LAYOUT0 field descriptions

Field	Description
31–24 NBLOCKS	Number of subsequent blocks on the flash page (excluding the data0 block). A value of 0 indicates that only the DATA0 block is present and a value of 8 indicates that eight subsequent blocks are present for a total of nine blocks on the flash (including the DATA0 block). Any values from 0 to 255 are supported by the hardware.

Table continues on the next page...

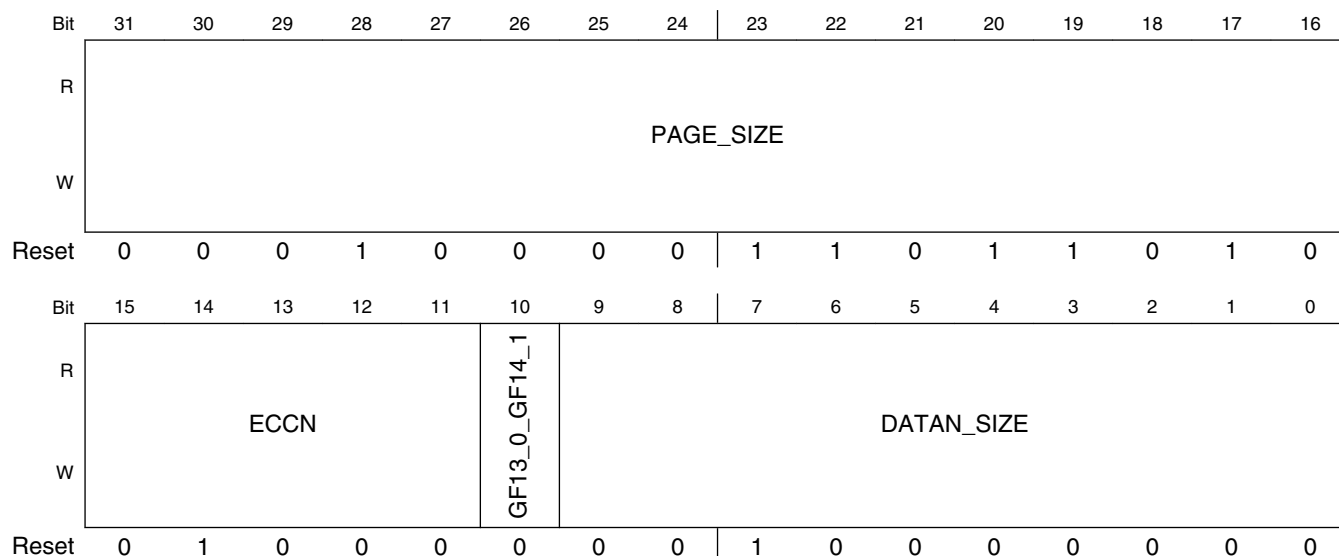
BCH_FLASH2LAYOUT0 field descriptions (continued)

Field	Description
23–16 META_SIZE	Indicates the size of the metadata (in bytes) to be stored on a flash page. The BCH design support from 0 to 255 bytes for metadata—if set to 0, no metadata will be stored. Metadata is stored before the associated data in block 0. If the DATA0_SIZE field is programmed to a 0, then metadata effectively be stored with its own parity. When both the metadata and data0 fields are programmed with non-zero values, the first block will contain both portions of data and will be covered by a single parity block.
15–11 ECC0	Indicates the ECC level for the first block on the flash page. The first block covers metadata plus the associated data from the DATA0_SIZE field. 0x0 NONE — No ECC to be performed 0x1 ECC2 — ECC 2 to be performed 0x2 ECC4 — ECC 4 to be performed : — 0x1E ECC60 — ECC 60 to be performed 0x1F ECC62 — ECC 62 to be performed
10 GF13_0_GF14_1	Select GF13 or GF14: 0-GF13; 1-GF14
DATA0_SIZE	Indicates the size of the data 0 block (in DWORDS / four bytes) to be stored on the flash page. If set to 0, the first block will only contain metadata.

9.6.6.13 Hardware BCH ECC Flash 2 Layout 1 Register (BCH_FLASH2LAYOUT1)

The flash format register contains a description of the logical layout of data on the flash device. This register is used in conjunction with the FLASH2LAYOUT0 register to control the format for the device selecting layout 2 in the LAYOUTSELECT register.

Address: 3300_4000h base + D0h offset = 3300_40D0h



BCH_FLASH2LAYOUT1 field descriptions

Field	Description
31–16 PAGE_SIZE	Indicates the total size of the flash page (in bytes). This should be set to the page size including spare area. The page size is programmable to accommodate different flash configurations that may be available in the future.
15–11 ECCN	Indicates the ECC level for the subsequent blocks on the flash page (blocks 1-n). Subsequent blocks only contain data (no metadata). 0x0 NONE — No ECC to be performed 0x1 ECC2 — ECC 2 to be performed 0x2 ECC4 — ECC 4 to be performed : — 0x1E ECC60 — ECC 60 to be performed 0x1F ECC62 — ECC 62 to be performed
10 GF13_0_GF14_1	Select GF13 or GF14: 0-GF13; 1-GF14
DATAN_SIZE	Indicates the size of the subsequent data blocks (in DWORDS / four bytes) to be stored on the flash page. The size of subsequent data blocks does not have to match the data size for block 0, which is important when metadata is stored separately or for balancing the amount of data stored in each block.

9.6.6.14 Hardware BCH ECC Flash 3 Layout 0 Register (BCH_FLASH3LAYOUT0)

The flash format register contains a description of the logical layout of data on the flash device. This register is used in conjunction with the FLASH3LAYOUT1 register to control the format for the devices selecting layout 3 in the LAYOUTSELECT register.

Each pair of layout registers describes one of four supported flash configurations. Software should program the LAYOUTSELECT register for each supported GPMI chip select to select from one of the four layout values. Each pair of registers contains settings that are used by the BCH block while reading / writing the flash page to control data, metadata, and flash page sizes as well as the ECC correction level. The first block written to flash can be programmed to have different ECC, metadata, and data sizes from subsequent data blocks on the device. In addition, the number of blocks stored on a page of flash is not fixed, but instead is determined by the number of bytes consumed by the initial (block 0) and subsequent data blocks.

See sections [Flash Page Layout](#) and [Determining the ECC layout for a device](#) for more detail information on setting up the flash layout registers.

Address: 3300_4000h base + E0h offset = 3300_40E0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	NBLOCKS								META_SIZE							
W																
Reset	0	0	0	0	0	1	1	1	0	0	0	0	1	0	1	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ECC0					GF13_0_GF14_1	DATA0_SIZE									
W																
Reset	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0

BCH_FLASH3LAYOUT0 field descriptions

Field	Description
31–24 NBLOCKS	Number of subsequent blocks on the flash page (excluding the data0 block). A value of 0 indicates that only the DATA0 block is present and a value of 8 indicates that 8 subsequent blocks are present for a total of 9 blocks on the flash (including the DATA0 block). Any values from 0 to 255 are supported by the hardware.
23–16 META_SIZE	Indicates the size of the metadata (in bytes) to be stored on a flash page. The BCH design support from 0 to 255 bytes for metadata—if set to 0, no metadata will be stored. Metadata is stored before the associated data in block 0. If the DATA0_SIZE field is programmed to a 0, then metadata effectively be stored with its own parity. When both the metadata and data0 fields are programmed with non-zero values, the first block will contain both portions of data and will be covered by a single parity block.
15–11 ECC0	Indicates the ECC level for the first block on the flash page. The first block covers metadata plus the associated data from the DATA0_SIZE field. 0x0 NONE — No ECC to be performed 0x1 ECC2 — ECC 2 to be performed 0x2 ECC4 — ECC 4 to be performed : — 0x1E ECC60 — ECC 60 to be performed 0x1F ECC62 — ECC 62 to be performed
10 GF13_0_GF14_1	Select GF13 or GF14: 0-GF13; 1-GF14
DATA0_SIZE	Indicates the size of the data 0 block (in DWORDS / four bytes) to be stored on the flash page. If set to 0, the first block will only contain metadata.

9.6.6.15 Hardware BCH ECC Flash 3 Layout 1 Register (BCH_FLASH3LAYOUT1)

The flash format register contains a description of the logical layout of data on the flash device. This register is used in conjunction with the FLASH3LAYOUT0 register to control the format for the device selecting layout 3 in the LAYOUTSELECT register.

Address: 3300_4000h base + F0h offset = 3300_40F0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	PAGE_SIZE															
W																
Reset	0	0	0	1	0	0	0	0	1	1	0	1	1	0	1	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ECCN						GF13_0_GF14_1		DATAN_SIZE							
W																
Reset	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0

BCH_FLASH3LAYOUT1 field descriptions

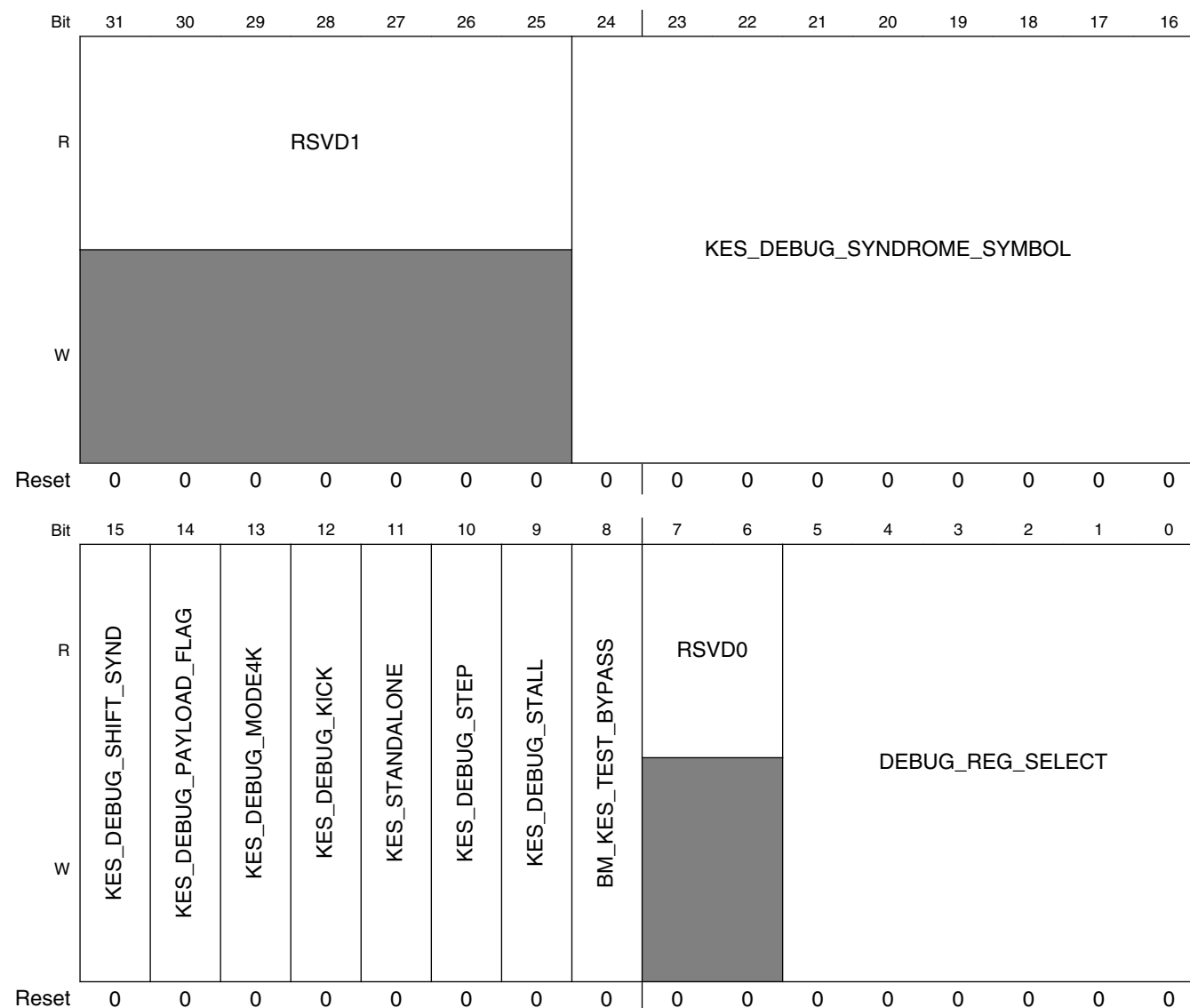
Field	Description
31–16 PAGE_SIZE	Indicates the total size of the flash page (in bytes). This should be set to the page size including spare area. The page size is programmable to accomodate different flash configurations that may be available in the future.
15–11 ECCN	Indicates the ECC level for the subsequent blocks on the flash page (blocks 1-n). Subsequent blocks only contain data (no metadata). 0x0 NONE — No ECC to be performed 0x1 ECC2 — ECC 2 to be performed 0x2 ECC4 — ECC 4 to be performed : — 0x1E ECC60 — ECC 60 to be performed 0x1F ECC62 — ECC 62 to be performed
10 GF13_0_GF14_1	Select GF13 or GF14: 0-GF13; 1-GF14
DATAN_SIZE	Indicates the size of the subsequent data blocks (in DWORDS / four bytes) to be stored on the flash page. The size of subsequent data blocks does not have to match the data size for block 0, which is important when metadata is stored separately or for balancing the amount of data stored in each block.

9.6.6.16 Hardware BCH ECC Debug Register0 (BCH_DEBUG0n)

The hardware BCH accelerator internal state machines and signals can be seen in the ECC debug register.

The BCH_DEBUG0 register provides access to various internal state information which might prove useful during hardware debug and validation.

Address: 3300_4000h base + 100h offset + (4d × i), where i=0d to 3d



BCH_DEBUG0n field descriptions

Field	Description
31–25 RSVD1	This field is reserved.

Table continues on the next page...

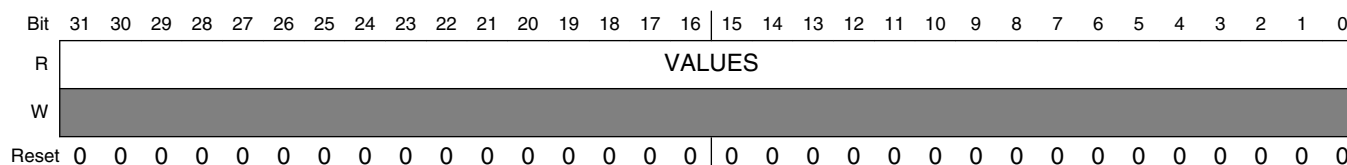
BCH_DEBUG0n field descriptions (continued)

Field	Description
	This read-only field is reserved and always has the value 0.
24–16 KES_DEBUG_ SYNDROME_ SYMBOL	The 9 bit value in this bit field shifts into the syndrome register array at the input of the KES engine whenever BCH_DEBUG0_KES_DEBUG_SHIFT_SYND is toggled. 0x0 NORMAL — Bus master address generator for SYND_GEN writes operates normally. 0x1 TEST_MODE — Bus master address generator always addresses last four bytes in Auxiliary block.
15 KES_DEBUG_ SHIFT_SYND	Toggling this bit causes the value in BCH_DEBUG0_KES_SYNDROME_SYMBOL to be shift into the syndrome register array at the input to the KES engine. After shifting in 16 symbols, one can kick off both KES and CF cycles by toggling BCH_DEBUG0_KES_DEBUG_KICK. Make sure that set KES_BCH_DEBUG0_KES_STANDALONE mode to 1 before kicking.
14 KES_DEBUG_ PAYLOAD_FLAG	When running the stand alone debug mode on the error calculator, the state of this bit is presented to the KES engine as the input payload flag. 0x1 DATA — Payload is set for 512 bytes data block. 0x1 AUX — Payload is set for 65 or 19 bytes auxiliary block.
13 KES_DEBUG_ MODE4K	When running the stand alone debug mode on the error calculator, the state of this bit is presented to the KES engine as the input mode (4K or 2K pages). 0x1 4k — Mode is set for 4K NAND pages. 0x1 2k — Mode is set for 2K NAND pages.
12 KES_DEBUG_ KICK	Toggling causes KES engine FSM to start as if kick by the Bus Master. This allows stand alone testing of the KES and Chien Search engines. Be sure to set KES_BCH_DEBUG0_KES_STANDALONE mode to 1 before kicking.
11 KES_ STANDALONE	Set to one, cause the KES engine to suppress toggling the KES_BM_DONE signal to the bus master and suppress toggling the CF_BM_DONE signal by the CF engine. 0x0 NORMAL — Bus master address generator for SYND_GEN writes operates normally. 0x1 TEST_MODE — Bus master address generator always addresses last four bytes in Auxiliary block.
10 KES_DEBUG_ STEP	Toggling this bit causes the KES FSM to skip passed the stall state if it is in DEBUG_STALL mode and completed processing a block.
9 KES_DEBUG_ STALL	Set to one to cause KES FSM to stall after notifying Chien search engine to start processing its block but before notifying the bus master that the KES computation is complete. This allows a diagnostic to stall the FSM after each blocks key equations are solved. This also has the effect of stalling the CSFE search engine so it's state can be examined after it finishes processing the KES stalled block. 0x0 NORMAL — KES FSM proceeds to next block supplied by bus master. 0x1 WAIT — KES FSM waits after current equations are solved and the search engine is started.
8 BM_KES_TEST_ BYPASS	1 = Point all SYND_GEN writes to dummy area at the end of the AUXILLIARY block so that diagnostics can preload all payload, parity bytes and computed syndrome bytes for test the KES engine. 0x0 NORMAL — Bus master address generator for SYND_GEN writes operates normally. 0x1 TEST_MODE — Bus master address generator always addresses last four bytes in Auxiliary block.
7–6 RSVD0	This field is reserved. This read-only field is reserved and always has the value 0.
DEBUG_REG_ SELECT	The value loaded in this bit field is used to select the internal register state view of KES engine or the Chien search engine.

9.6.6.17 KES Debug Read Register (BCH_DBGKESREAD)

The hardware BCH ECC accelerator key equation solver internal state machines and signals can be seen in the ECC debug registers.

Address: 3300_4000h base + 110h offset = 3300_4110h



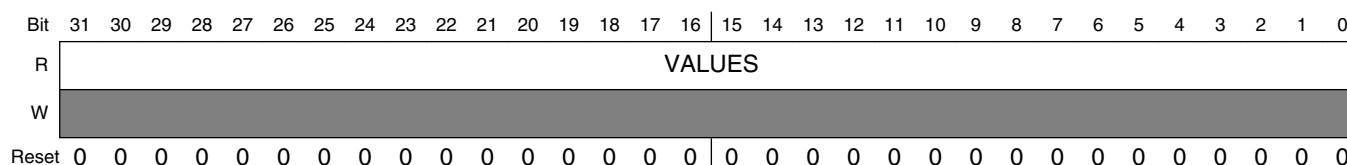
BCH_DBGKESREAD field descriptions

Field	Description
VALUES	This register returns the ROM BIST CRC value after a BIST test.

9.6.6.18 Chien Search Debug Read Register (BCH_DBGCSFEREAD)

The hardware BCH ECC accelerator Chien Search internal state machines and signals can be seen in the ECC debug registers.

Address: 3300_4000h base + 120h offset = 3300_4120h



BCH_DBGCSFEREAD field descriptions

Field	Description
VALUES	Reserved

9.6.6.19 Syndrome Generator Debug Read Register (BCH_DBGSYNDGENREAD)

The hardware BCH ECC accelerator syndrome generator internal state machines and signals can be seen in the ECC debug registers.

62BIT Correcting ECC Accelerator (BCH)

Address: 3300_4000h base + 130h offset = 3300_4130h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	VALUES																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

BCH_DBGSYNDGENREAD field descriptions

Field	Description
VALUES	Reserved

9.6.6.20 Bus Master and ECC Controller Debug Read Register (BCH_DBGAHBMREAD)

The hardware BCH ECC accelerator bus master, ECC controller internal state machines, and signals can be seen in the ECC debug registers.

Address: 3300_4000h base + 140h offset = 3300_4140h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	VALUES																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

BCH_DBGAHBMREAD field descriptions

Field	Description
VALUES	Reserved

9.6.6.21 Block Name Register (BCH_BLOCKNAME)

Read only view of the block name string BCH.

Fixed pattern read only value is for test purposes. It can be read as an ASCII string with the zero termination coming from the first byte of the BLOCKVERSION register.

Address: 3300_4000h base + 150h offset = 3300_4150h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	NAME																															
W																																
Reset	0	0	1	0	0	0	0	0	0	1	0	0	1	0	0	0	0	1	0	0	0	0	1	1	0	1	0	0	0	0	1	0

BCH_BLOCKNAME field descriptions

Field	Description
NAME	The name is in the ASCII characters BCH (0x20, H, C, B).

9.6.6.22 BCH Version Register (BCH_VERSION)

This register always returns a known read value for debug purposes and indicates the version of the block and RTL version in use.

Address: 3300_4000h base + 160h offset = 3300_4160h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	MAJOR								MINOR								STEP															
W																																
Reset	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

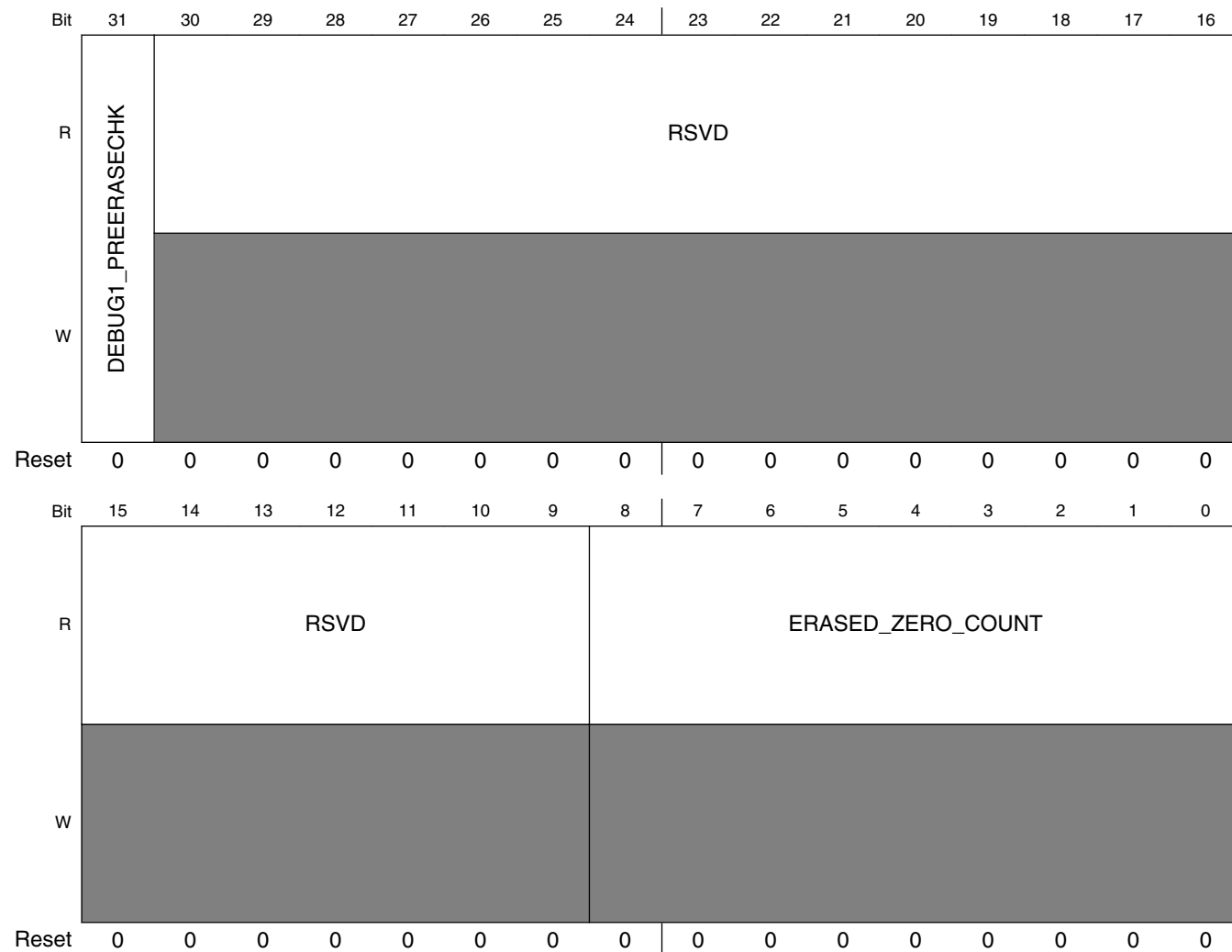
BCH_VERSION field descriptions

Field	Description
31–24 MAJOR	Fixed read-only value indicates the MAJOR field of the RTL version.
23–16 MINOR	Fixed read-only value indicates the MINOR field of the RTL version.
STEP	Fixed read-only value reflecting the stepping of the RTL version.

9.6.6.23 Hardware BCH ECC Debug Register 1 (BCH_DEBUG1)

The BCH_DEBUG1 register provides erased zero count information and pre-erase check.

Address: 3300_4000h base + 170h offset = 3300_4170h



BCH_DEBUG1 field descriptions

Field	Description
31 DEBUG1_ PREERASECHK	Blank page enables pre-erase check. 0x0 Turn off pre-erase check 0x1 Turn on pre-erase check
30–9 RSVD	This field is reserved. This read-only field is reserved and always has the value 0.

Table continues on the next page...

BCH_DEBUG1 field descriptions (continued)

Field	Description
ERASED_ZERO_COUNT	The zero counts on one page.

9.7 General Purpose Media Interface (GPMI)

9.7.1 Overview

The GPMI controller is a flexible interface to supporting up to four NAND flash chip selects.

- ONFI3.2, DDR Mode, Samsung / Toshiba Toggle NAND protocol compatible.
- Fully configurable address and command behavior, providing support for future devices not yet specified.

The GPMI resides on the APBH. The GPMI also provides an interface to the BCH module to allow direct parity processing.

Registers are clocked on the HCLK domain. The I/O and pin timing are clocked on a dedicated GPMICLK domain. GPMICLK can be set to maximize I/O performance.

The following figure shows a block diagram of the GPMI controller.

Please see [Clock Controller Module \(CCM\)](#) for clock setting, configuration and gating information.

Table 9-12. GPMI Clocks

Clock name	Clock Root	Description
bch_input_apb_clk		BCH to APBH input clock
gpmi_bch_input_bch_clk		BCH input clock
gpmi_bch_input_gpmi_io_clk		GPMI IO input clock
gpmi_input_apb_clk		GPMI to APBH clock

9.7.3 GPMI NAND Mode

The general-purpose media interface has several features to efficiently support NAND:

- Individual chip select pins and ganged ready/busy pin for up to four NANDs.
- Individual state machine and DMA channel for each chip select.
- Special command modes work with DMA controller to perform all normal NAND functions without CPU intervention.
- Configurable timing based on a dedicated clock allows optimal balance of high NAND performance and low system power.

GPMI and DMA have been designed to handle complex multi-page operations without CPU intervention. The DMA uses a linked descriptor function with branching capability to automatically handle all of the operations needed to read/write multiple pages:

- **Data/Register Read/Write**-The GPMI can be programmed to read or write multiple cycles to the NAND address, command or data registers.
- **Wait for NAND Ready**-The GPMI's Wait-for-Ready mode can monitor the ready/busy signal of a single NAND flash and signal the DMA when the device has become ready. It also has a time-out counter and can indicate to the DMA that a time-out error has occurred. The DMAs can conditionally branch to a different descriptor in the case of an error.
- **Check Status**-The Read-and-Compare mode allows the GPMI to check NAND status against a reference. If an error is found, the GPMI can instruct the DMA to branch to an alternate descriptor, which attempts to fix the problem or asserts a CPU IRQ.

9.7.3.1 Multiple NAND Support

The GPMI supports up to four NAND chip selects, with ganged ready/busy pins. Since they share a data bus and control lines, the GPMI can only actively communicate with a single NAND at a time. However, all NANDs can concurrently perform internal read, write, or erase operations. With fast NAND flash and software support for concurrent NAND operations, this architecture allows the total throughput to approach the data bus speed, which can be as high as 50 MB/s (8-bit bus running at 50 MHz single clock edge) in asynchronous mode and 200MB/s (8-bit bus running at 100MHz both clock edges) in Source Synchronous mode.

There are two options for controlling the four NAND chip selects via the DMA interface. The first option is the one to one mapping, where the each DMA channel is tied to its own NAND chip select. For example DMA channel 'n' accesses only NAND attached to chip select 'n'. The second option is the decoupled mode where a DMA channel can access any or all NAND chip selects connected to the GPMI. A DMA channel will signify the NAND chip select it wants to access by writing its chip select value in the GPMI_CTRL0[CS] field and setting the GPMI_CTRL1[DECOUPLE_CS] to 1. This option is useful if software chooses to use only one DMA channel to access all the attached NAND devices.

9.7.3.2 GPMI NAND Timing and Clocking

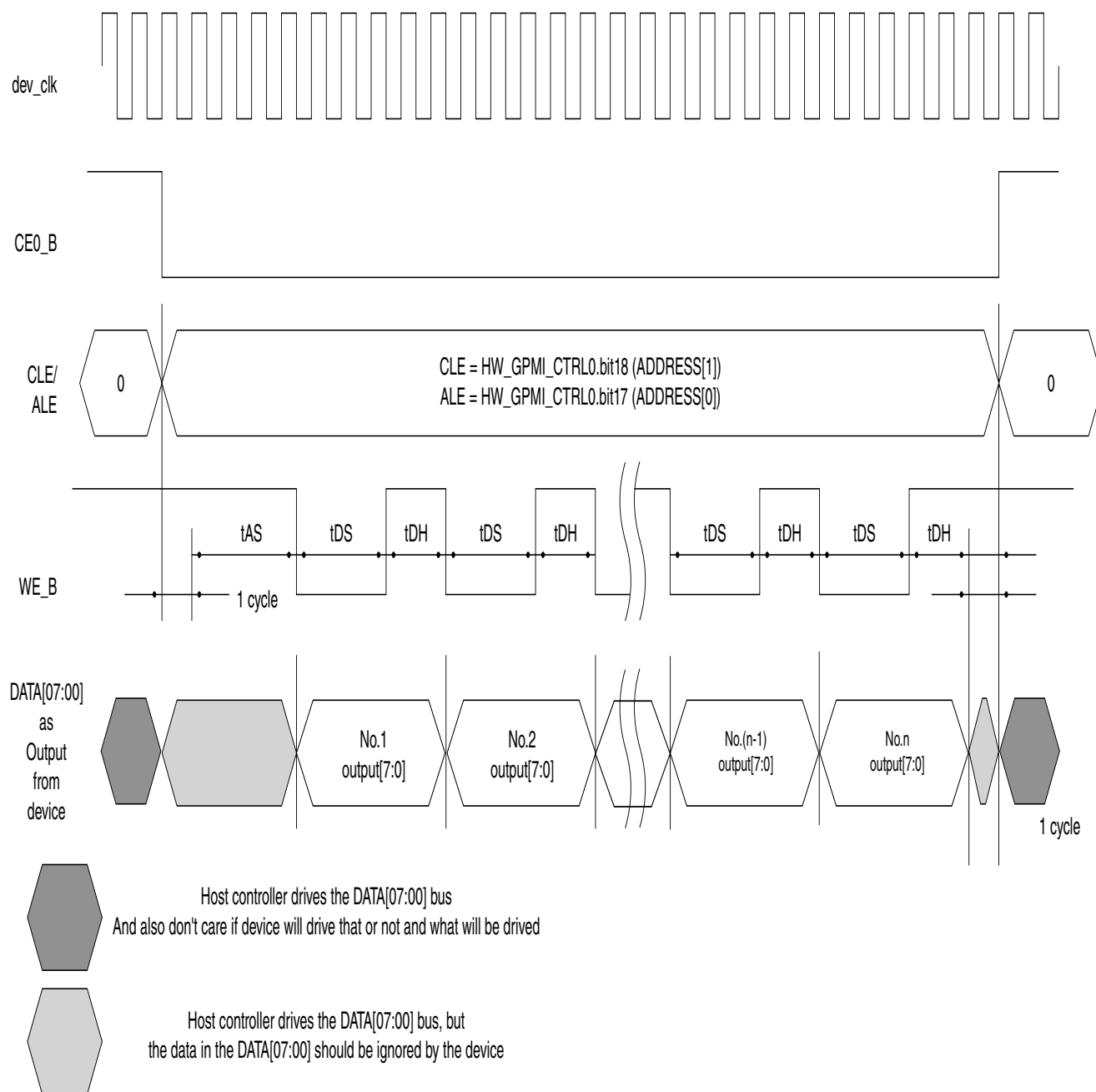
The dedicated clock, GPMICK, is used as a timing reference for NAND flash I/O. Since various NANDs have different timing requirements, GPMICK may need to be adjusted for each application.

While the actual pin timings are limited by the NAND chips used and the I/O pad configuration, the GPMI can support data bus speeds of up to 200 MHz x 8 bits. The actual read/write strobe timing parameters are adjusted as indicated in the register descriptions in Memory Map.

9.7.3.3 Basic NAND Timing

9.7.3.3.1 NAND Asynchronous Timing

[Figure 9-22](#) and illustrates the operation of the output (from host to device) timing parameters in NAND ONFI asynchronous mode.



- tAS is configurable by programming HW_GPMI_TIMING0 Address_Setup: in this example, Address_Setup = 4, tAS is equal to 4 dev_clk cycles.
- tDS is configurable by programming HW_GPMI_TIMING0 Data_Setup: in this example, Data_Setup = 3, tDS is equal to 3 dev_clk cycles
- tDH is configurable by programming HW_GPMI_TIMING0 Data_Hold: in this example, Data_Hold = 2, tDH is equal to 2 dev_clk cycles
- tAS/tDS/tDH will extend, if the output data is not ready in device fifo.

Figure 9-21. Asynchronous Mode Basic Write Timing Diagram (command write, address write, or data write)

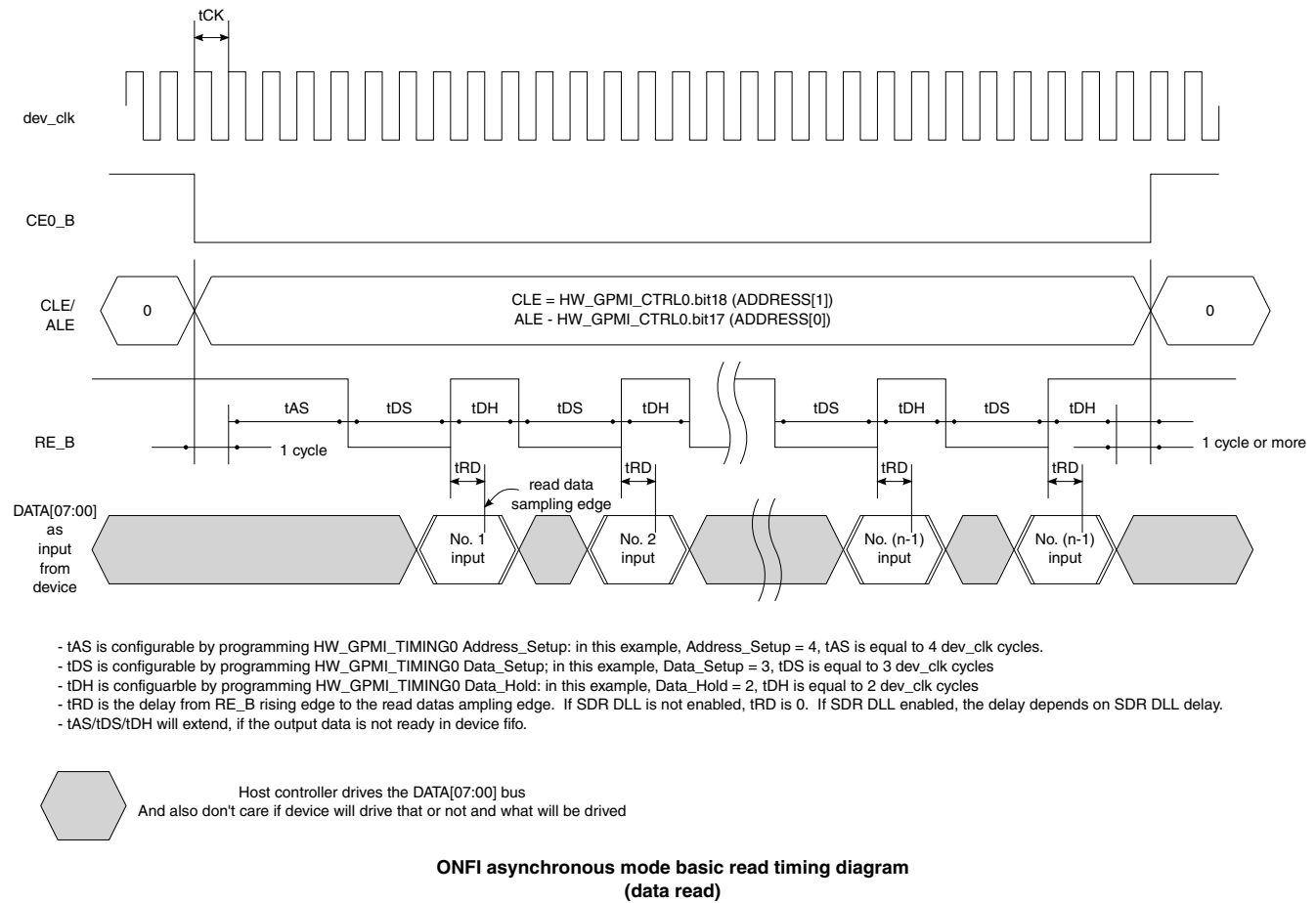


Figure 9-22. ONFI Asynchronous Mode Basic Read Timing Diagram (data read)

9.7.3.3.2 NAND Asynchronous EDO Mode Timing

In high-speed NANDS, the read data may not be valid until after the read strobe (NAND_RE_B) deasserts. This is the case when the minimum tDS is programmed to achieve higher bandwidth.

The GPML implements a feedback read strobe to sample the read data. The feedback read strobe can be delayed to support fast nand EDO (Extended Data Out) timing where the read strobe may deassert before the read data is valid, and read data is valid for some time after read strobe. Nand EDO timings is applied typically for read cycle frequency above 33 MHz. See [Figure 9-23](#).

The GPML provides control over the amount of delay applied to the feedback read strobe. This delay depends on the maximum read access time (tREA) of the nand and the read pulse width (tRP) used to access the nand. tRP is specified by GPML_TIMING0[DATA_SETUP] register. When (tREA + 4ns) is less than tRP, no

delay is required to sample to nand read data. (The 4ns provides adequate data setup time for the GPMI.) In this case set `GPMI_CTRL1[HALF_PERIOD] = 0;`
`GPMI_CTRL1[RDN_DELAY] = 0;` `GPMI_CTRL1[DLL_ENABLE] = 0.`

When $(t_{REA} + 4ns)$ is greater than or equal to t_{RP} , a delay of the feedback read strobe is required to sample to nand read data. This delay is equal to the difference between these two timings:

$$DELAY = t_{REA} + 4ns - t_{RP}.$$

Since the GPMI delay chain is limited to 6ns maximum, if $DELAY > 6ns$ then increase t_{RP} by increasing the value of `GPMI_TIMING0[DATA_SETUP]` until $DELAY$ is less than or equal to 6ns.

The GPMI programming for this $DELAY$ depends on the GPMICLK period. The GPMI DLL will not function properly if the GPMICLK period is greater than 12ns: disable the DLL if this is the case. If the GPMICLK period is greater than 6ns (and not greater than 12ns), set the `GPMI_CTRL1[HALF_PERIOD]=1;` This will cause the DLL reference period (RP) to be one-half of the GPMICLK period. If the GPMICLK period is 6ns or less then set the `GPMI_CTRL1[HALF_PERIOD]=0;` This will cause the DLL reference period (RP) to be equal to the GPMICLK period. $DELAY$ is a multiple (0 to 1.875) of RP.

The `GPMI_CTRL1[RDN_DELAY]` is encoded as a 1-bit integer and 3-bit fraction delay factor. $DELAY$ is a multiple of the delay factor and the reference period. See table below for details.

Table 9-13. RDN DELAY

HW_GPMI_CTRL1[RDN_DELAY]	Delay Factor
0	0.000
1	0.125
2	0.250
3	0.375
4	0.500
5	0.625
6	0.750
7	0.875
8	1.000
9	1.125
10	1.250
11	1.375
12	1.500
13	1.625

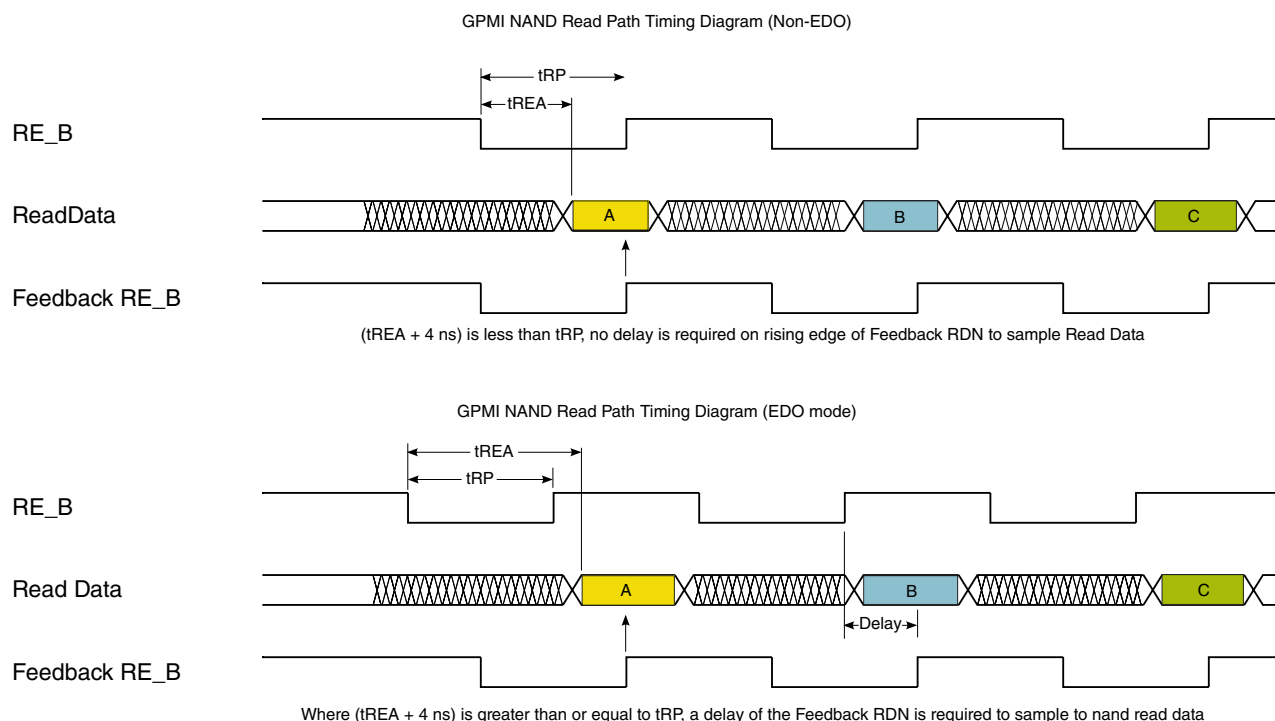
Table continues on the next page...

Table 9-13. RDN DELAY (continued)

HW_GPMI_CTRL1[RDN_DELAY]	Delay Factor
14	1.750
15	1.875

$DELAY = DelayFactor \times RP$ or $DELAY = GPMI_CTRL1[RDN_DELAY] \times 0.125 \times RP$.

Use this equation to calculate the value for GPMI_CTRL1[RDN_DELAY]. Then set GPMI_CTRL1[DLL_ENABLE]=1.

**Figure 9-23. NAND Read Path Timing**

For example, a NAND with $tREA_{max} = 20$ ns, $tRP_{min} = 12$ ns, and $tRC_{min} = 25$ ns (read cycle time) may be programmed as follows:

- GPMICLK clock frequency: Consider $480/6 = 80$ MHz which is 12.5 ns clock period. This is too close to the minimum NAND spec if we program the data setup and hold to 1 GPMICLK cycle. Consider $480/7 = 68.57$ MHz which is 14.58 ns clock period. With data setup and hold set to 1, we have a tRP of 14.58ns and a tRC of 29.16 ns (good margins).
- Since $(tREA + 4ns)$ is greater than tRP , required $DELAY = tREA + 4ns - tRP = 20 + 4 - 14.58ns = 9.42$ ns.

- $\text{GPMI_CTRL1}[\text{HALF_PERIOD}] =$, since GPMICLK period is ns. So $\text{RP} = \text{GPMICLK period} = \text{ns}$.
- $\text{DELAY} = \text{GPMI_CTRL1}[\text{RDN_DELAY}] \times 0.125 \times \text{RP}$. $9.42 \text{ ns} = \text{GPMI_CTRL1}[\text{RDN_DELAY}] \times 0.125 \times \text{ns}$. $\text{GPMI_CTRL1}[\text{RDN_DELAY}] =$

NOTE

It is recommended that the drive strength of NAND_RE_B and NAND_WE_B output pins be set to 8 mA. This will reduce the transition time under heavy loads. Low transition times will be important when NAND interface read and write cycle times are below 30 ns. The other GPMI pins may remain at 4 mA, since their frequency is only up to half that of NAND_RE_B and NAND_WE_B .

9.7.3.3.3 NAND ONFI Source Synchronous Mode Timing

NOTE

In the following figures, CLK shares the same pin as WE_B in Async Mode. And $\text{W/R\#}$ shares the same pin as RE_B in Async Mode.

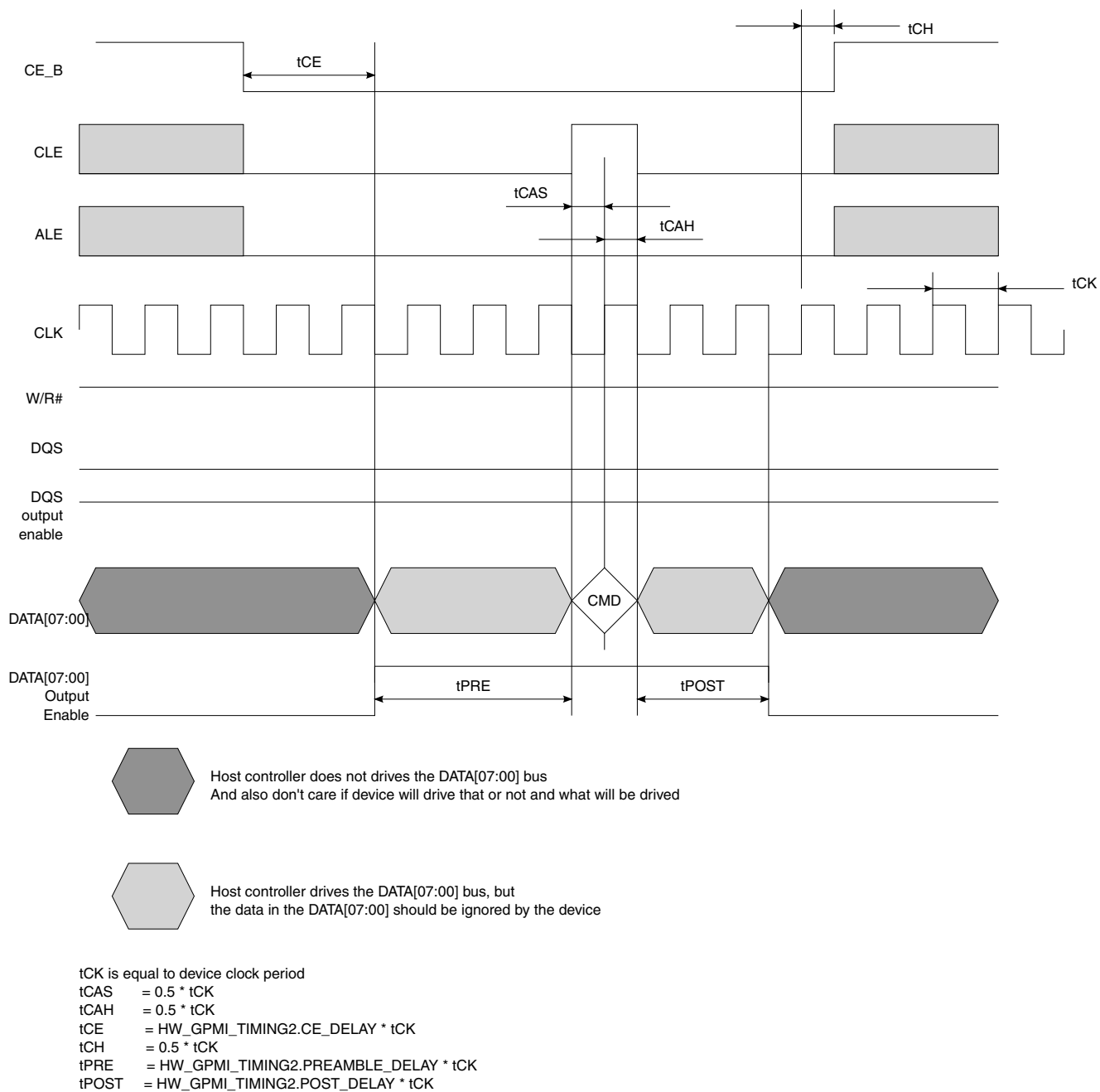


Figure 9-24. ONFI Source Synchronous Mode Basic Command Write Timing Diagram

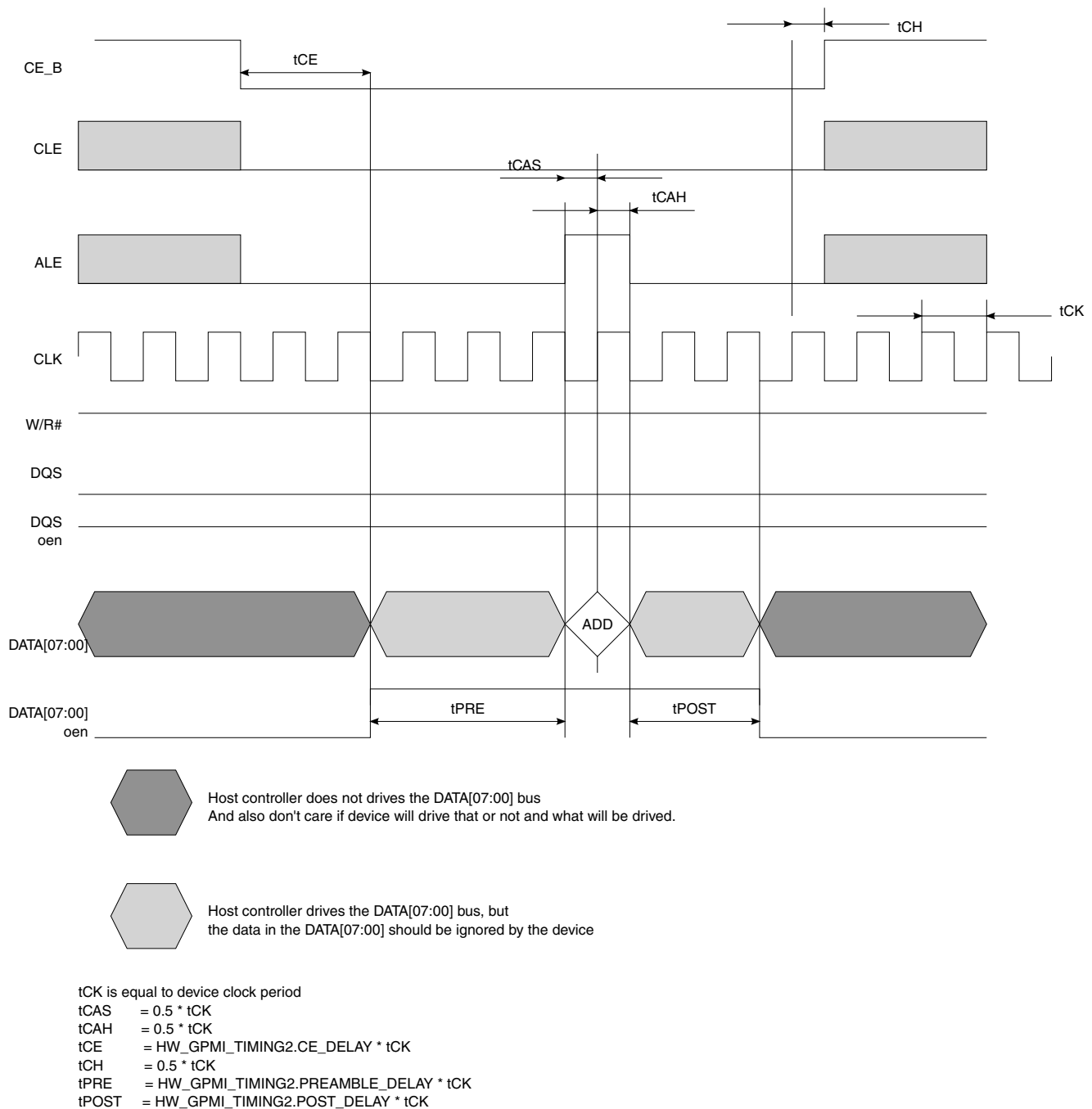
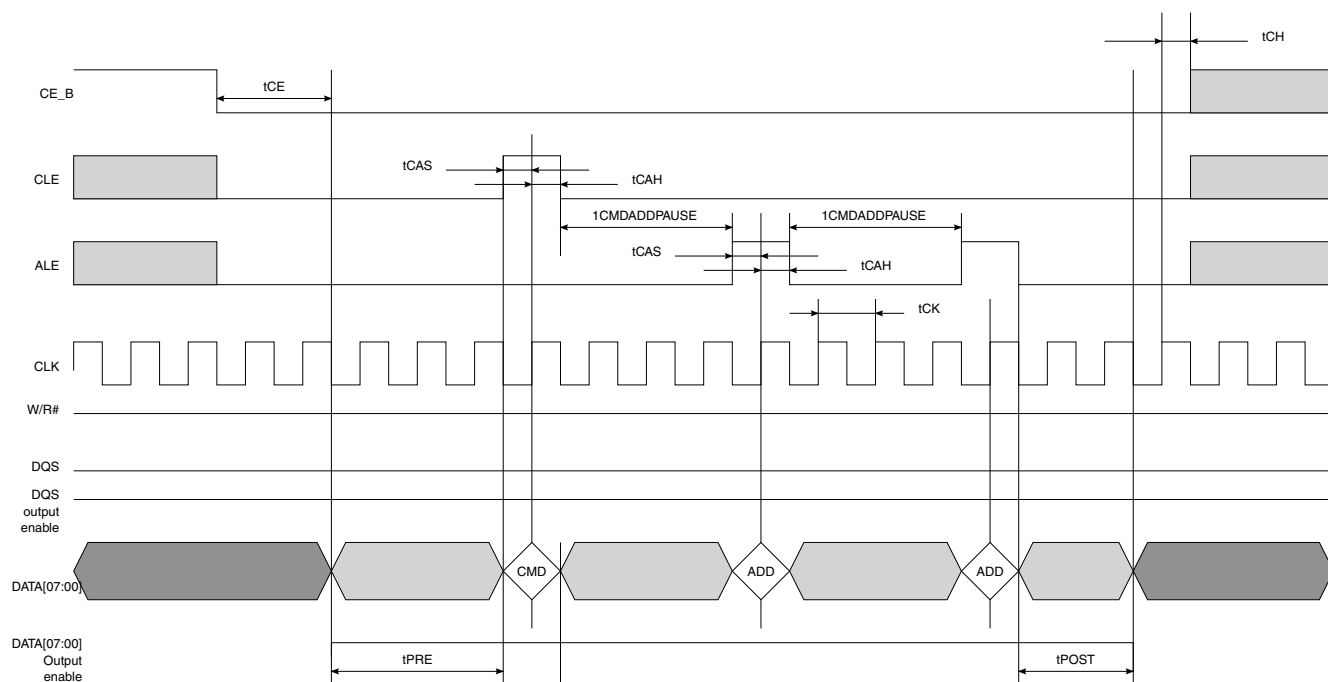


Figure 9-25. ONFI Source Synchronous Mode Basic Address Write Timing Diagram



 Host controller does not drives the DATA[07:00] bus
And also do not care if device will drive that or not and what will be driven.

 Host controller drives the DATA[07:00] bus, but
the data in the DATA[07:00] should be ignored by the device

tCK is equal to device clock period

tCAS = 0.5 * tCK

tCAH = 0.5 * tCK

tCE = HW_GP MI_TIMING2.CE_DELAY * tCK

tCH = 0.5 * tCK

tPRE = HW_GP MI_TIMING2.PREAMBLE_DELAY * tCK

tPOST = HW_GP MI_TIMING2.POST_DELAY * tCK

tCMDADDPAUSE = HW_GP MI_TIMING2.CMDADD_PAUSE * tCK

Figure 9-26. ONFI Source Synchronous Mode Command + Address Write Timing Diagram

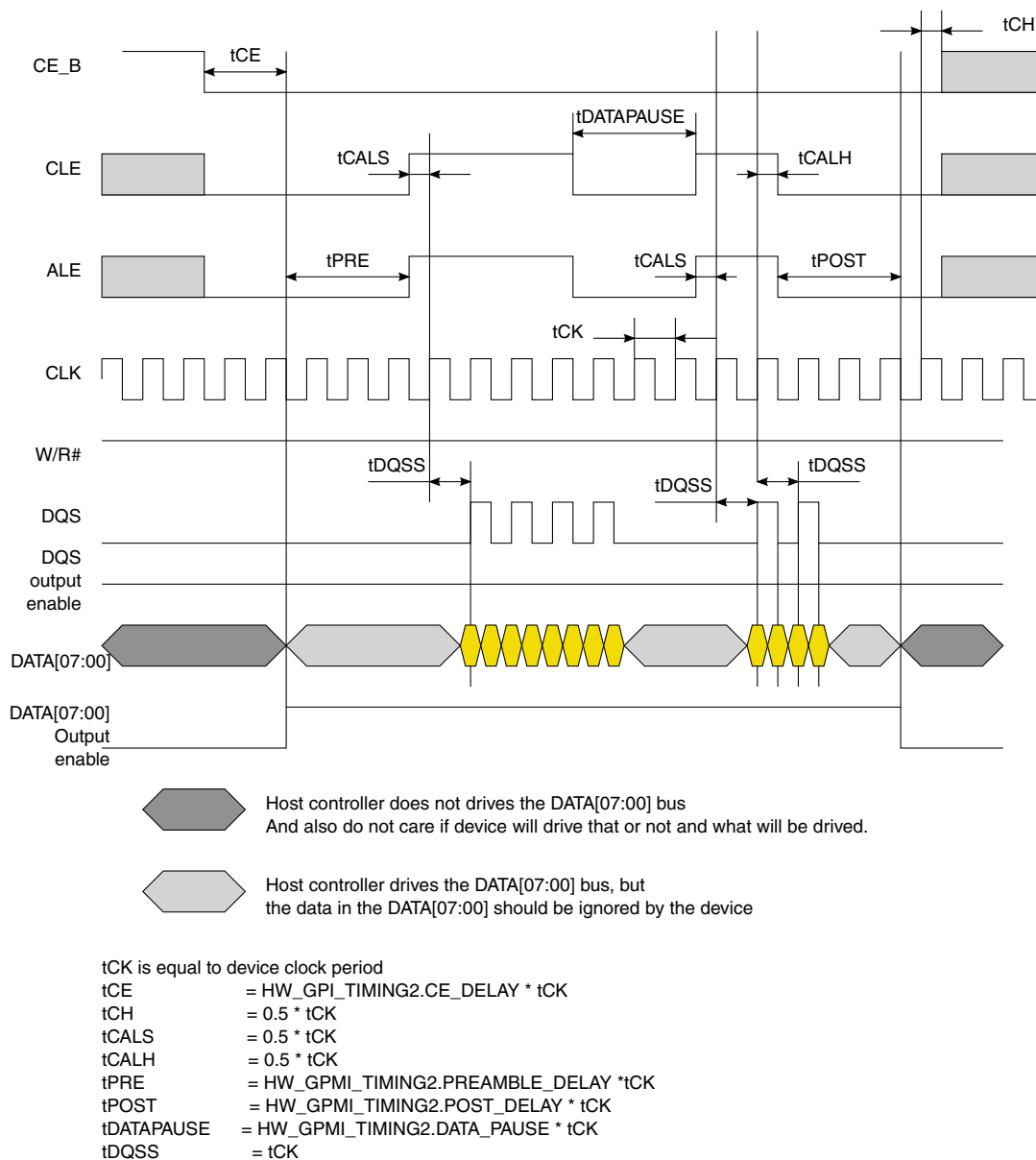
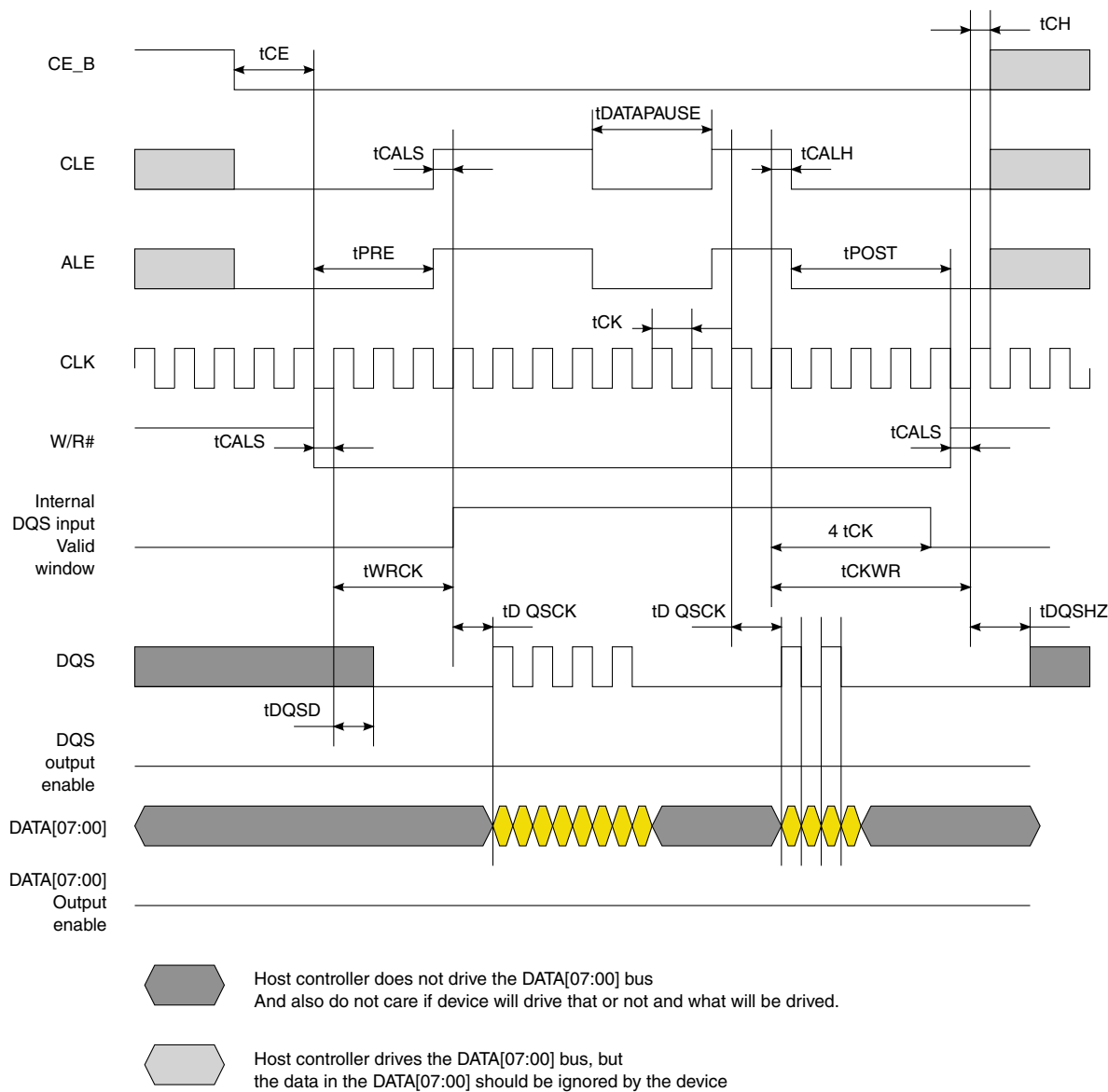


Figure 9-27. ONFI Source Synchronous Mode Data Write Timing Diagram



tCK is equal to device clock period
 $t_{CE} = HW_GPI_TIMING2.CE_DELAY * t_{CK}$
 $t_{CH} = 0.5 * t_{CK}$
 $t_{CALS} = 0.5 * t_{CK}$
 $t_{CALH} = 0.5 * t_{CK}$
 $t_{PRE} = HW_GPMI_TIMING2.PREAMBLE_DELAY * t_{CK}$
 $t_{POST} = HW_GPMI_TIMING2.POST_DELAY * t_{CK}$
 $t_{DATA\ PAUSE} = HW_GPMI_TIMING2.DATA_PAUSE * t_{CK}$
 $t_{WRCK}/t_{CKWR}/t_{DQSD}/t_{DQSK}/t_{DQSHZ}$ are device parameters

Figure 9-28. ONFI Source Synchronous Mode Data Read Timing Diagram

9.7.3.3.4 NAND Toggle Mode Timing

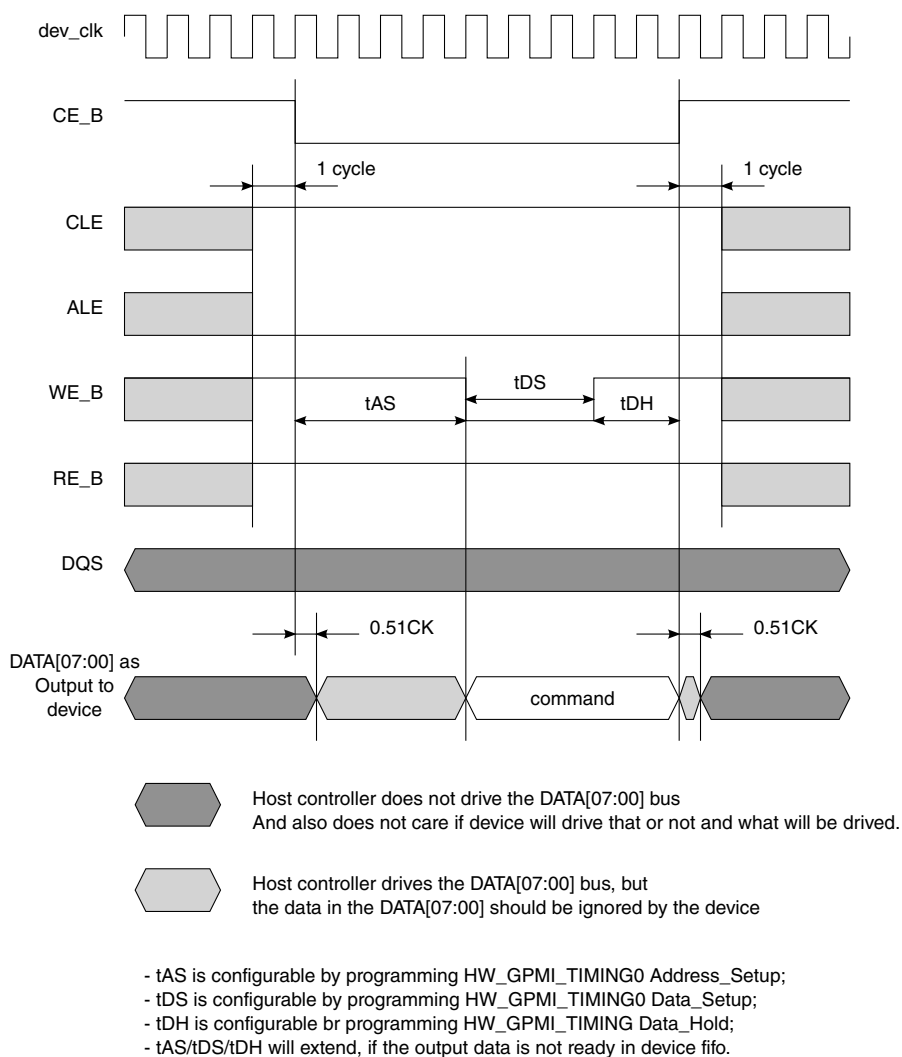


Figure 9-29. Samsung Toggle Mode Basic Command Write Timing Diagram

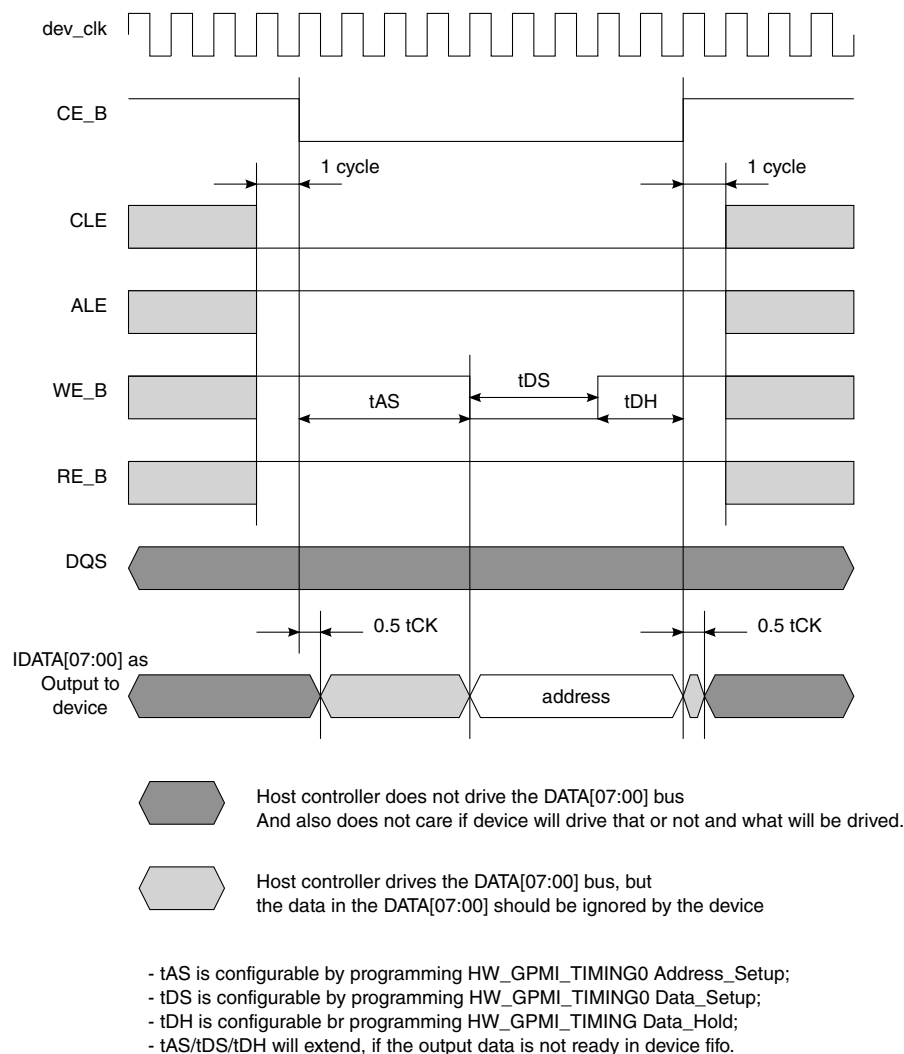


Figure 9-30. Samsung Toggle Mode Basic Address Write Timing Diagram

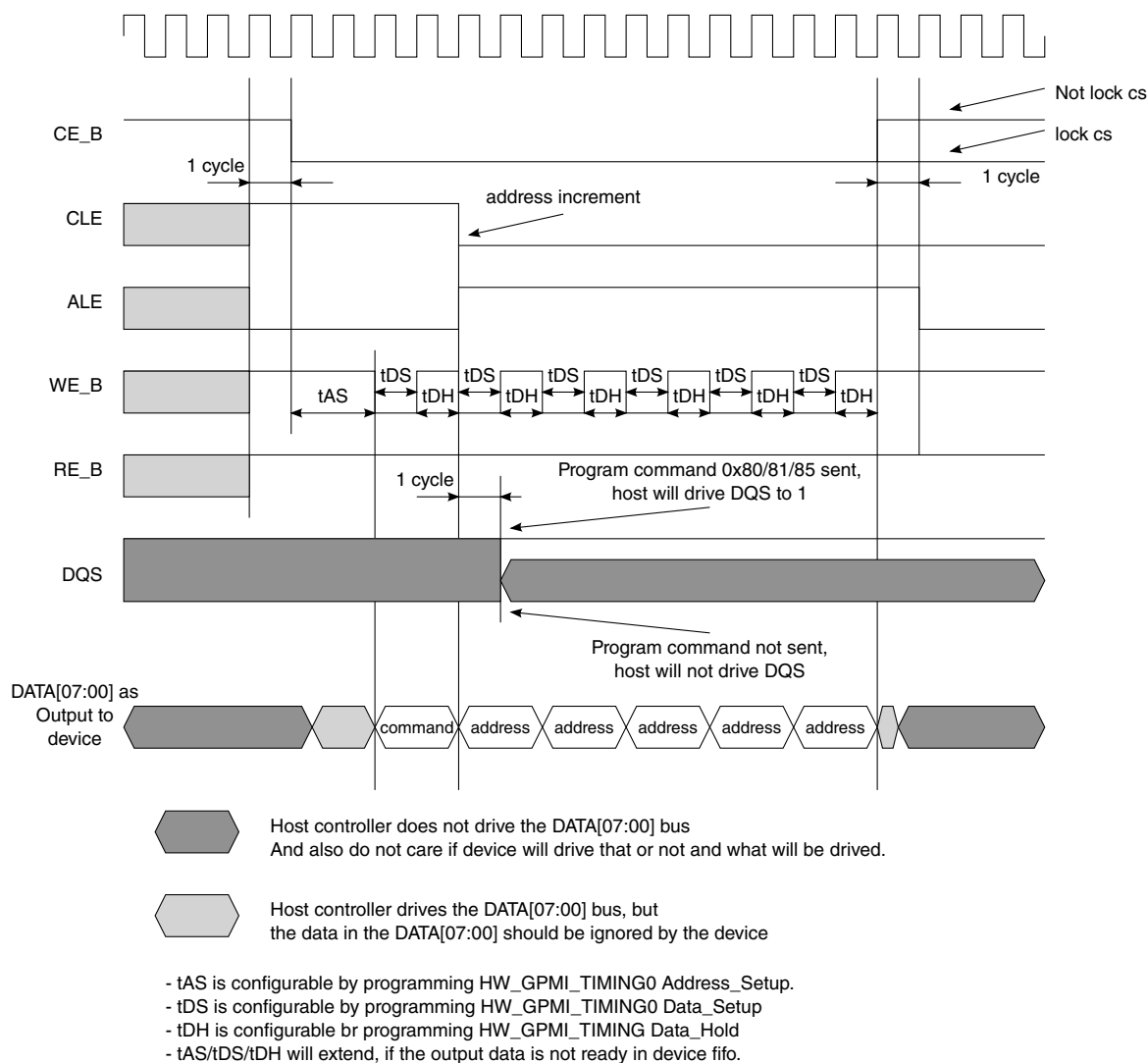


Figure 9-31. Samsung Toggle Mode Basic Command + Address Timing Diagram

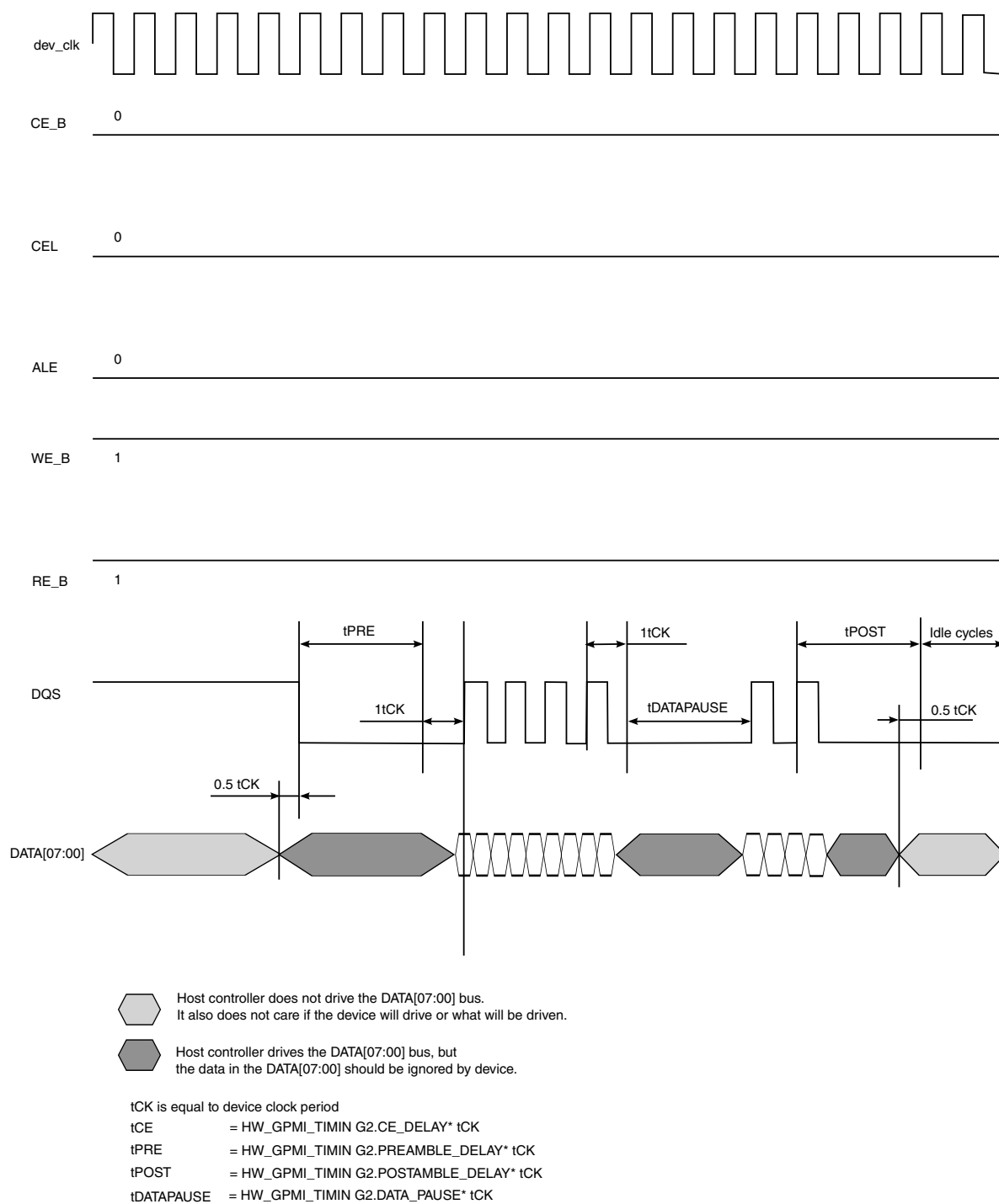


Figure 9-32. Toggle Mode Data Write Timing Diagram

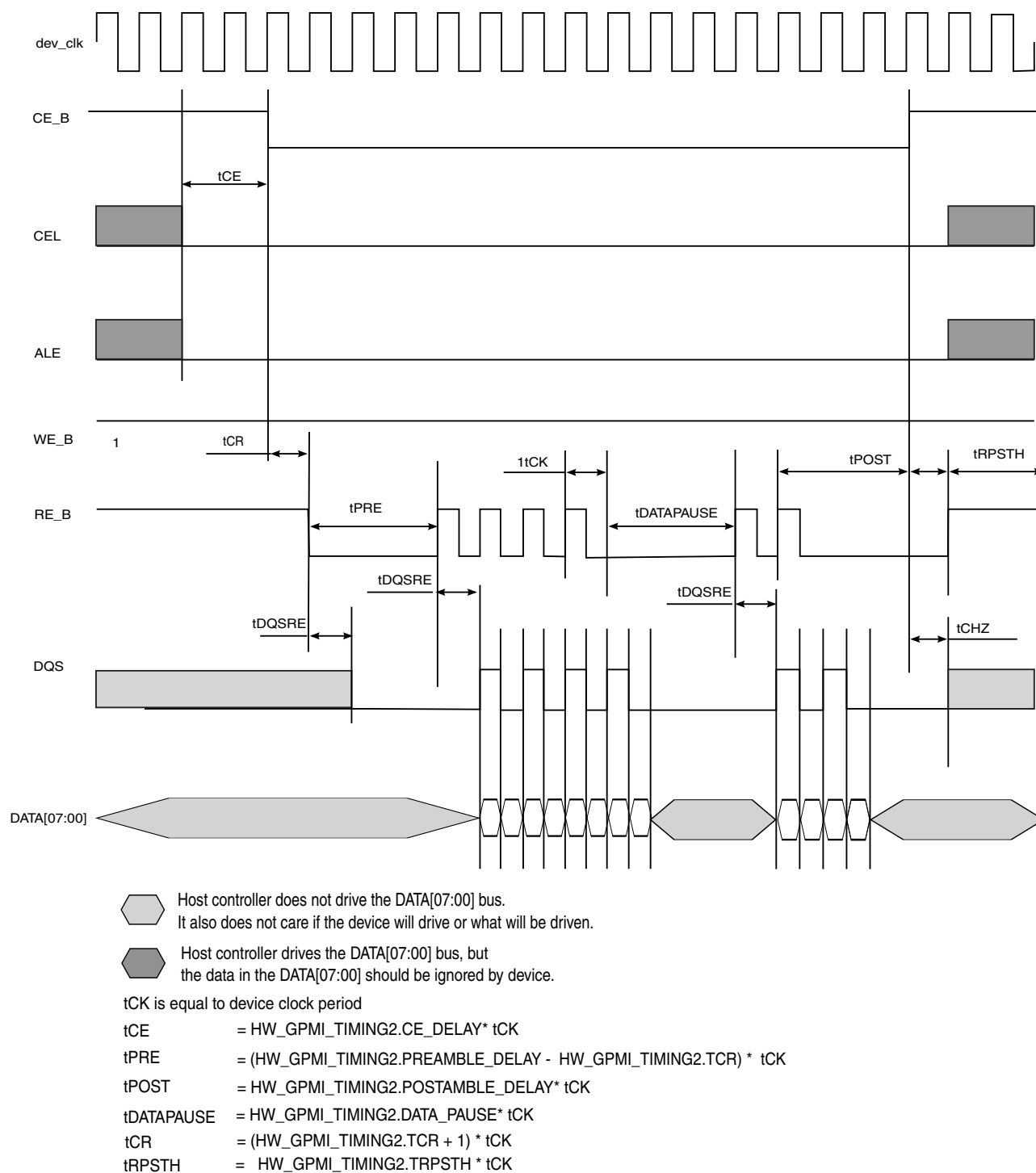


Figure 9-33. Toggle Mode Data Read Timing Diagram

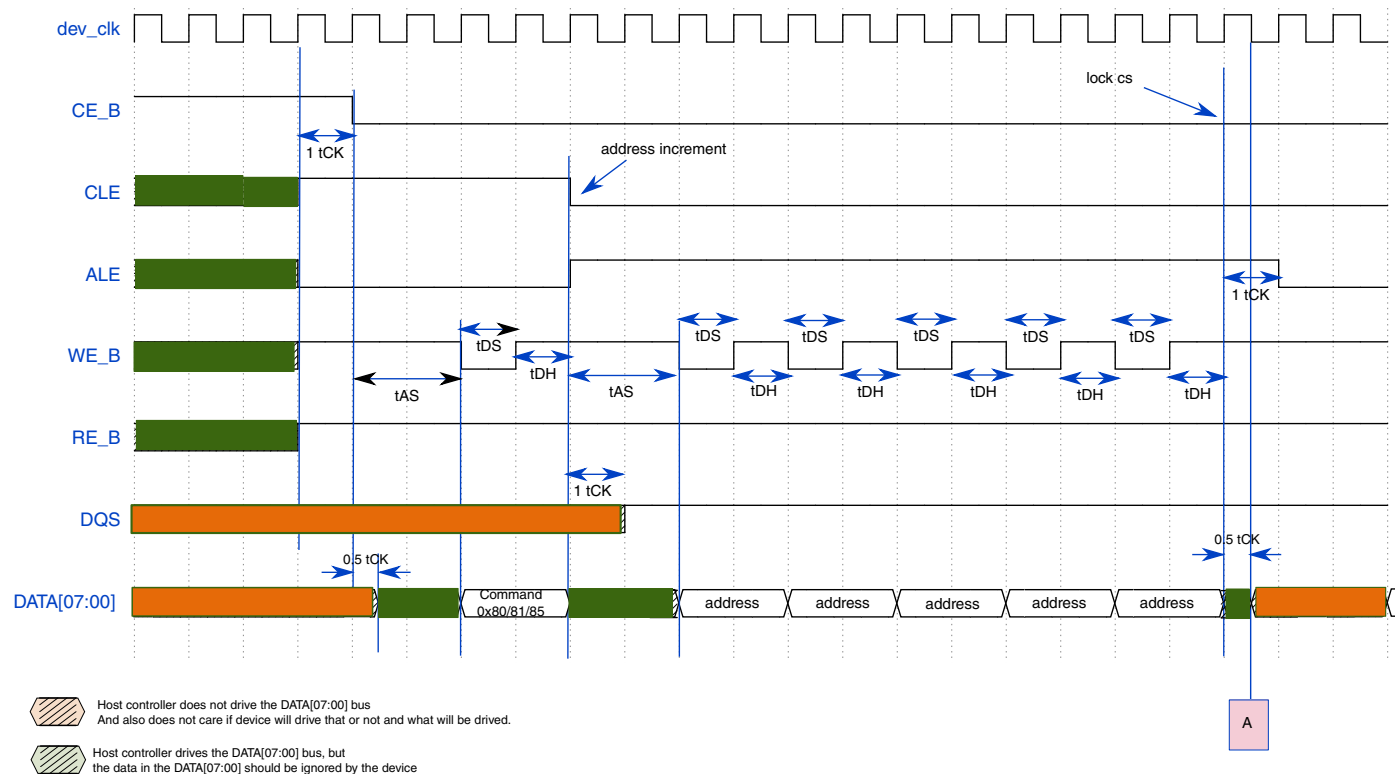


Figure 9-34. Toggle Mode Program Timing Diagram (A)

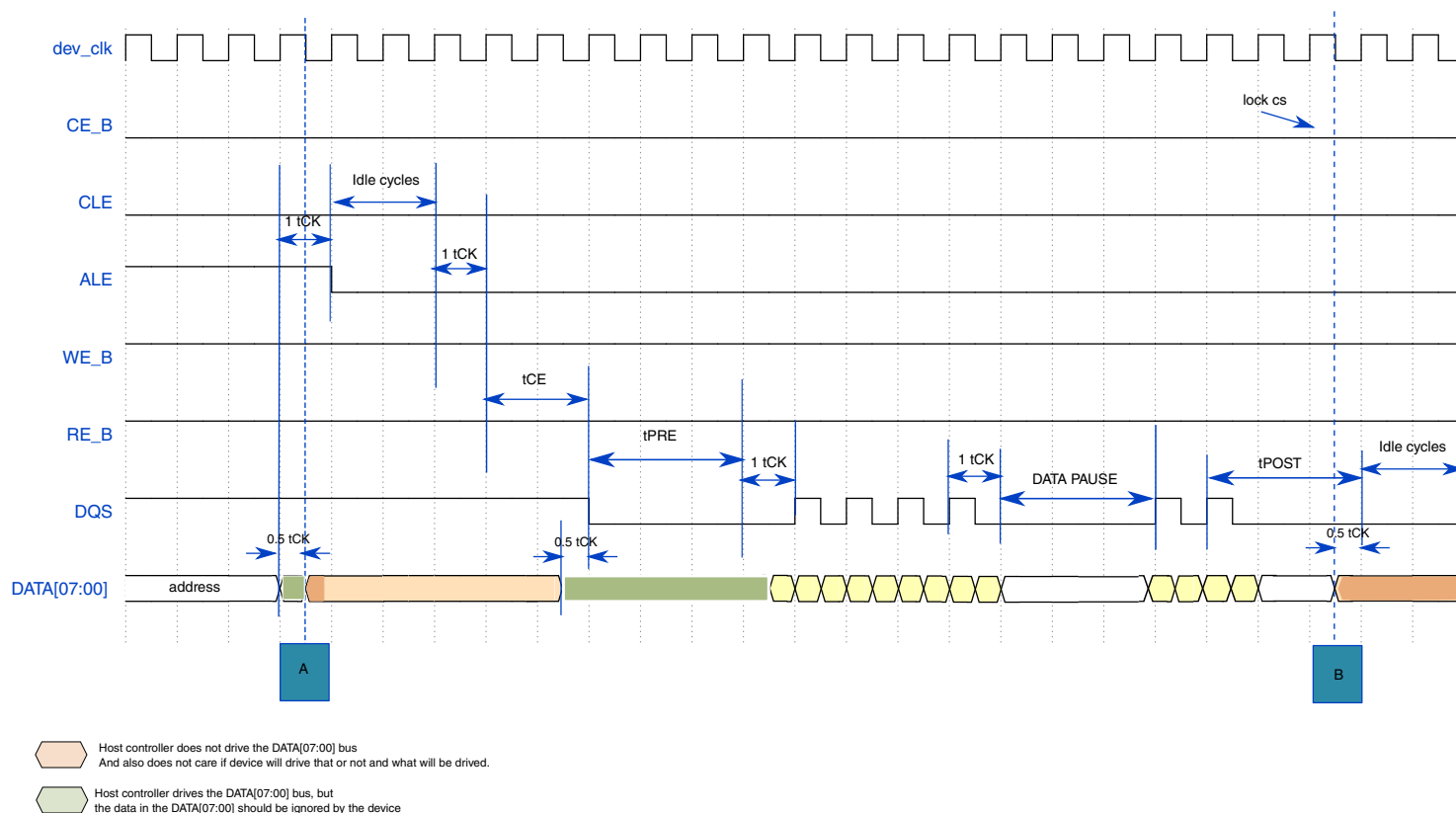


Figure 9-35. Toggle Mode Program Timing Diagram (B)

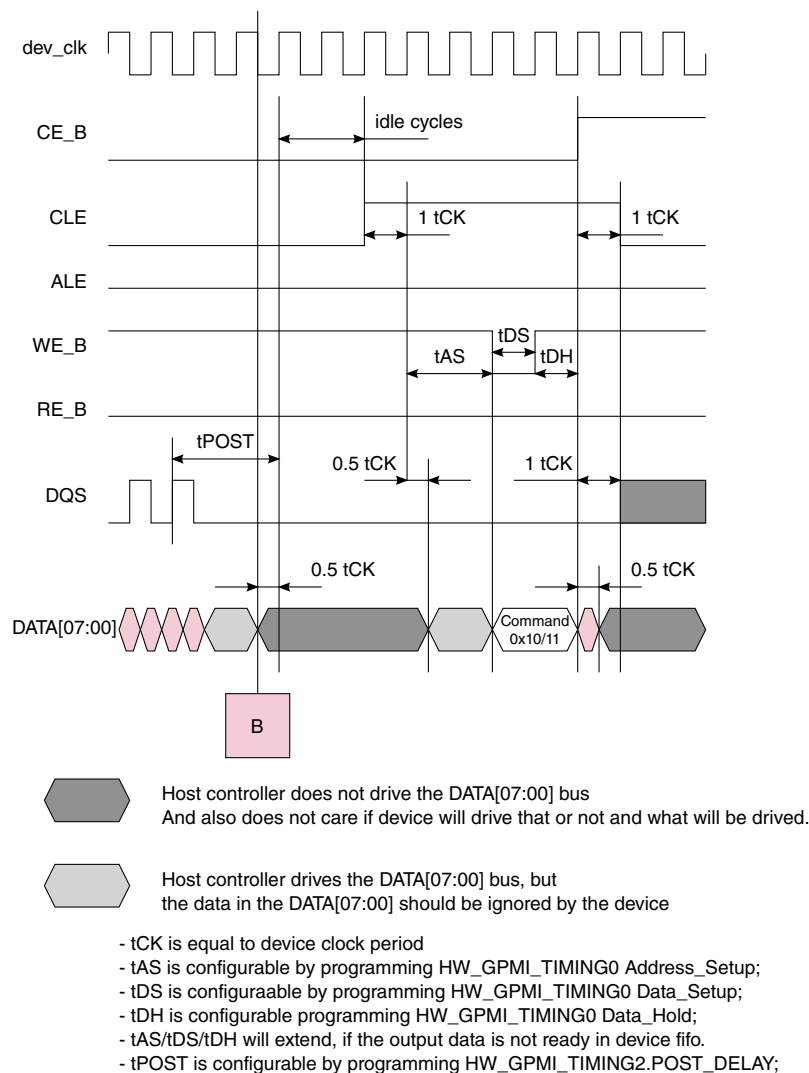


Figure 9-36. Toggle Mode Program Timing Diagram (C)

9.7.3.4 Hardware BCH Interface

The GPMI provides an interface to the BCH module. This reduces the SOC bus traffic and the software involvement.

The GPMI also provides an interface to the Randomizer module. The Randomizer can generate random data based on BCH ECC encoded / decoded data. It can be employed to reduce the disturbances caused by neighboring cells in the NAND chip, thus reducing bit errors. To enable the Randomizer module, set GPMI_ECCCTRL[RANDOMIZER_ENABLE] to 1, then set GPMI_ECCCOUNT[RANDOMIZER_PAGE] to select randomizer page number needed

to be randomized. All these registers can be programmed by the DMA chain. The randomized data should start from the zero column address and be the size of the whole NAND page. If the randomizer function is enabled, `GPMI_ECCCTRL[ENABLE_ECC]` should also be enabled. To bypass BCH error correction function, set `BCH_FLASHxLAYOUT0[ECC0]` and `BCH_FLASHxLAYOUT1[ECCN]` to 0.

When BCH ECC is enable, parity information is inserted on-the-fly during writes to 8-bit NAND devices. The BCH will supply payload and parity to the GPMI to write to the NAND. During NAND reads, parity is checked and ECC processing is performed after each read block. In this case the GPMI reads the NAND device and redirects the data and parity to the BCH module for ECC processing.

To program the BCH for NAND writes, remove the soft reset and clock gates from `BCH_CTRL[SFTRST]` and `BCH_CTRL[CLKGATE]`. The bulk of BCH programming is actually applied to the GPMI via PIO operations embedded in its DMA command structures. This has a subtle implication when writing to the GPMI ECC registers: access to these registers must be written in progressive register order. Thus, to write to the `GPMI_ECCCOUNT` register, write first (in order) to registers `GPMI_CTRL0`, `GPMI_COMPARE`, and `GPMI_ECCCTRL` before writing to `GPMI_ECCCOUNT`. These additional register writes need to be accounted for in the `CMDWORDS` field of the respective DMA channel command register.

NOTE

Note that the `GPMI_PAYLOAD` and `GPMI_AUXILIARY` pointers need to be word-aligned for proper ECC operation. If those pointers are non-word-aligned, then the BCH engine will not operate properly and could possibly corrupt system memory in the adjoining memory regions.

9.7.4 Behavior During Reset

A soft reset (`SFTRST`) can take multiple clock periods to complete, so do NOT set `CLKGATE` when setting `SFTRST`. The reset process gates the clocks automatically.

9.7.5 GPMI Memory Map/Register Definition

The following registers provide control for programmable elements of the GPMI module.

NOTE

All ATA or UDMA features are not supported for the chip.

NOTE

GPMI does not support the Set feature command in Toggle mode. The NANDF_DQS output is only enabled in program operation for Toggle mode, but the Set feature command also needs to use the NANDF_DQS signal to write data to Toggle NAND flash. So the Set feature command in Toggle mode is not supported.

GPMI memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
3300_2000	GPMI Control Register 0 Description (GPMI_CTRL0)	32	R/W	C000_0000h	9.7.5.1/2508
3300_2004	GPMI Control Register 0 Description (GPMI_CTRL0_SET)	32	R/W	C000_0000h	9.7.5.1/2508
3300_2008	GPMI Control Register 0 Description (GPMI_CTRL0_CLR)	32	R/W	C000_0000h	9.7.5.1/2508
3300_200C	GPMI Control Register 0 Description (GPMI_CTRL0_TOG)	32	R/W	C000_0000h	9.7.5.1/2508
3300_2010	GPMI Compare Register Description (GPMI_COMPARE)	32	R/W	0000_0000h	9.7.5.2/2510
3300_2020	GPMI Integrated ECC Control Register Description (GPMI_ECCCTRL)	32	R/W	0000_0000h	9.7.5.3/2511
3300_2024	GPMI Integrated ECC Control Register Description (GPMI_ECCCTRL_SET)	32	R/W	0000_0000h	9.7.5.3/2511
3300_2028	GPMI Integrated ECC Control Register Description (GPMI_ECCCTRL_CLR)	32	R/W	0000_0000h	9.7.5.3/2511
3300_202C	GPMI Integrated ECC Control Register Description (GPMI_ECCCTRL_TOG)	32	R/W	0000_0000h	9.7.5.3/2511
3300_2030	GPMI Integrated ECC Transfer Count Register Description (GPMI_ECCCOUNT)	32	R/W	0000_0000h	9.7.5.4/2512
3300_2040	GPMI Payload Address Register Description (GPMI_PAYLOAD)	32	R/W	0000_0000h	9.7.5.5/2513
3300_2050	GPMI Auxiliary Address Register Description (GPMI_AUXILIARY)	32	R/W	0000_0000h	9.7.5.6/2513

Table continues on the next page...

GPMI memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3300_2060	GPMI Control Register 1 Description (GPMI_CTRL1)	32	R/W	0004_0004h	9.7.5.7/2514
3300_2064	GPMI Control Register 1 Description (GPMI_CTRL1_SET)	32	R/W	0004_0004h	9.7.5.7/2514
3300_2068	GPMI Control Register 1 Description (GPMI_CTRL1_CLR)	32	R/W	0004_0004h	9.7.5.7/2514
3300_206C	GPMI Control Register 1 Description (GPMI_CTRL1_TOG)	32	R/W	0004_0004h	9.7.5.7/2514
3300_2070	GPMI Timing Register 0 Description (GPMI_TIMING0)	32	R/W	0001_0203h	9.7.5.8/2517
3300_2080	GPMI Timing Register 1 Description (GPMI_TIMING1)	32	R/W	0000_0000h	9.7.5.9/2517
3300_2090	GPMI Timing Register 2 Description (GPMI_TIMING2)	32	R/W	0302_3336h	9.7.5.10/2518
3300_20A0	GPMI DMA Data Transfer Register Description (GPMI_DATA)	32	R/W	0000_0000h	9.7.5.11/2519
3300_20B0	GPMI Status Register Description (GPMI_STAT)	32	R	0000_0005h	9.7.5.12/2519
3300_20C0	GPMI Debug Information Register Description (GPMI_DEBUG)	32	R	0000_0000h	9.7.5.13/2522
3300_20D0	GPMI Version Register Description (GPMI_VERSION)	32	R	0502_0000h	9.7.5.14/2523
3300_20E0	GPMI Debug2 Information Register Description (GPMI_DEBUG2)	32	R/W	0000_F100h	9.7.5.15/2523
3300_20F0	GPMI Debug3 Information Register Description (GPMI_DEBUG3)	32	R	0000_0000h	9.7.5.16/2526
3300_2100	GPMI Double Rate Read DLL Control Register Description (GPMI_READ_DDR_DLL_CTRL)	32	R/W	0000_0038h	9.7.5.17/2527
3300_2110	GPMI Double Rate Write DLL Control Register Description (GPMI_WRITE_DDR_DLL_CTRL)	32	R/W	0000_0038h	9.7.5.18/2528
3300_2120	GPMI Double Rate Read DLL Status Register Description (GPMI_READ_DDR_DLL_STS)	32	R	0000_0000h	9.7.5.19/2530
3300_2130	GPMI Double Rate Write DLL Status Register Description (GPMI_WRITE_DDR_DLL_STS)	32	R	0000_0000h	9.7.5.20/2531

9.7.5.1 GPMI Control Register 0 Description (GPMI_CTRL0n)

The GPMI control register 0 specifies the GPMI transaction to perform for the current command chain item.

Address: 3300_2000h base + 0h offset + (4d × i), where i=0d to 3d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	SFTRST	CLKGATE	RUN	DEV_IRQ_EN	LOCK_CS	UDMA	COMMAND_MODE	WORD_LENGTH	CS	ADDRESS	ADDRESS_INCREMENT					
W																
Reset	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	XFER_COUNT															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPMI_CTRL0n field descriptions

Field	Description
31 SFTRST	Set to zero for normal operation. When this bit is set to one (default), then the entire block is held in its reset state. This will not work if the CLKGATE bit is already set to '1'. CLKGATE must be cleared to '0' before issuing a soft reset. Also the GPMICLK must be running for this to work properly. RUN = 0x0 Allow GPMI to operate normally. RESET = 0x1 Hold GPMI in reset.
30 CLKGATE	Set this bit zero for normal operation. Setting this bit to one (default), gates all of the block level clocks off for minimizing AC energy consumption. RUN = 0x0 Allow GPMI to operate normally. NO_CLKS = 0x1 Do not clock GPMI gates in order to minimize power consumption.
29 RUN	The GPMI is busy running a command whenever this bit is set to '1'. The GPMI is idle whenever this bit set to zero. This can be set to one by a CPU write. In addition, the DMA sets this bit each time a DMA command has finished its PIO transfer phase. IDLE = 0x0 The GPMI is idle. BUSY = 0x1 The GPMI is busy running a command.

Table continues on the next page...

GPML_CTRL0n field descriptions (continued)

Field	Description
28 DEV_IRQ_EN	When set to '1' and ATA_IRQ pin is asserted, the GPML_IRQ output will assert.
27 LOCK_CS	For ATA/NAND mode: 0= Deassert chip select (CS) after RUN is complete. 1= Continue to assert chip select (CS) after RUN is complete. For Camera Mode: 0= Dont wait for VSYNC rising edge before capturing data. 1= Wait for VSYNC rising edge before capturing data (Camera mode only). DISABLED = 0x0 Deassert chip select (CS) after RUN is complete. ENABLED = 0x1 Continue to assert chip select (CS) after RUN is complete.
26 UDMA	DISABLED = 0x0 Use ATA-PIO mode on the external bus. ENABLED = 0x1 Use ATA-Ultra DMA mode on the external bus. 0 Use ATA-PIO mode on the external bus. 1 Use ATA-Ultra DMA mode on the external bus.
25–24 COMMAND_ MODE	WRITE = 0x0 Write mode. READ = 0x1 Read mode. READ_AND_COMPARE = 0x2 Read and Compare mode (setting sense flop). WAIT_FOR_READY = 0x3 Wait for Ready mode. For ATA WAIT_FOR_READY command set CS=01. 00 Write mode. 01 Read Mode. 10 Read and Compare Mode (setting sense flop). 11 Wait for Ready.
23 WORD_LENGTH	This bit should only be changed when RUN==0. Reserve = 0x0 Reserved. 8_BIT = 0x1 8-bit Data Bus mode. 0 Reserved. 1 8-bit Data Bus mode.
22–20 CS	Selects which chip select is active for this command. For ATA WAIT_FOR_READY command, this must be set to b01.
19–17 ADDRESS	Specifies the three address lines for ATA mode. In NAND mode, use A0 for CLE and A1 for ALE. NAND_DATA = 0x0 In NAND mode, this address is used to read and write data bytes. NAND_CLE = 0x1 In NAND mode, this address is used to write command bytes. NAND_ALE = 0x2 In NAND mode, this address is used to write address bytes.
16 ADDRESS_ INCREMENT	In ATA mode, the address will increment with each cycle. In NAND mode, the address will increment once, after the first cycle (going from CLE to ALE). DISABLED = 0x0 Address does not increment. ENABLED = 0x1 Increment address. 0 Address does not increment. 1 Increment address.
XFER_COUNT	Number of bytes to transfer for this command. A value of zero will transfer 64K bytes.

9.7.5.2 GPMI Compare Register Description (GPMI_COMPARE)

The GPMI compare register specifies the expect data and the xor mask for comparing to the status values read from the device. This register is used by the Read and Compare command.

GPMI_COMPARE 0x010

Address: 3300_2000h base + 10h offset = 3300_2010h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	MASK																REFERENCE															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

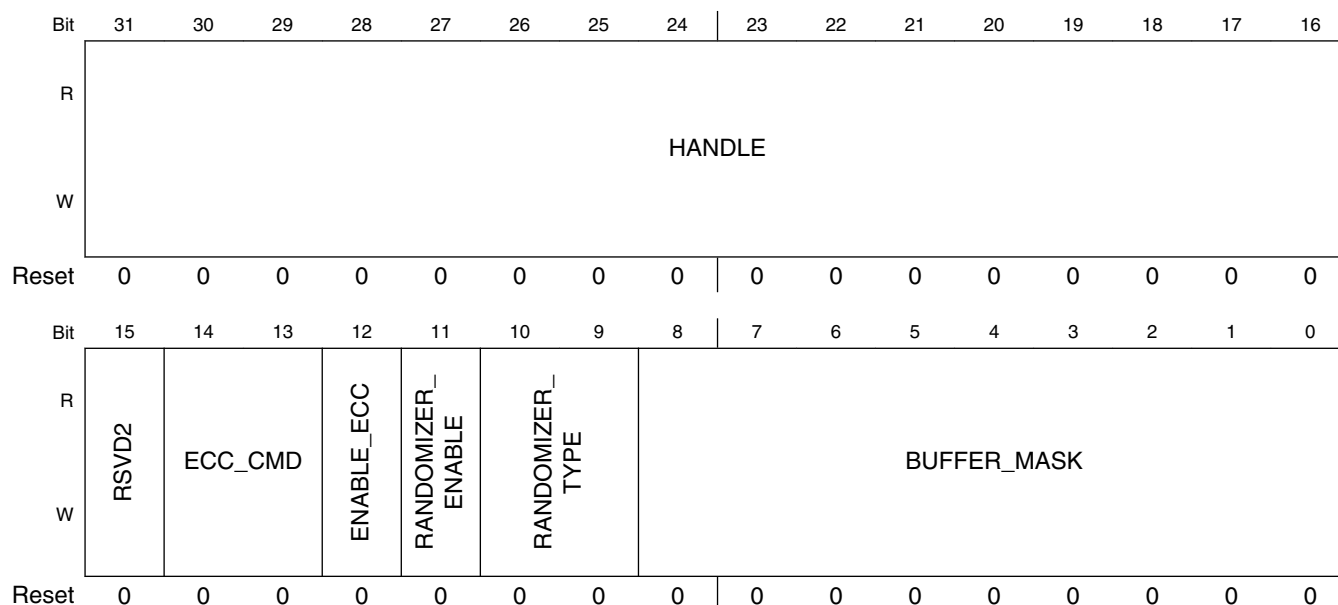
GPMI_COMPARE field descriptions

Field	Description
31–16 MASK	16-bit mask which is applied after the read data is XORed with the REFERENCE bit field.
REFERENCE	16-bit value which is XORed with data read from the NAND device.

9.7.5.3 GPMI Integrated ECC Control Register Description (GPMI_ECCCTRL_n)

The GPMI ECC control register handles configuration of the integrated ECC / Randomizer accelerator.

Address: 3300_2000h base + 20h offset + (4d × i), where i=0d to 3d



GPMI_ECCCTRL_n field descriptions

Field	Description
31–16 HANDLE	This is a register available to software to attach an identifier to a transaction in progress. This handle will be available from the ECC register space when the completion interrupt occurs.
15 RSVD2	Always write zeroes to this bit field.
14–13 ECC_CMD	ECC Command information. DECODE = 0x0 Decode. ENCODE = 0x1 Encode. RESERVE2 = 0x2 Reserved. RESERVE3 = 0x3 Reserved.
12 ENABLE_ECC	Enable ECC processing of GPMI transfers. ENABLE = 0x1 Use integrated ECC for read and write transfers. DISABLE = 0x0 Integrated ECC remains in idle.
11 RANDOMIZER_ENABLE	Enable randomizer function. If this bit is set to enable, ENABLE_ECC should be also enable.

Table continues on the next page...

GPMI_ECCCTRLn field descriptions (continued)

Field	Description
	0x0 disable 0x1 enable
10–9 RANDOMIZER_ TYPE	Set randomizer type 0x00 Type 0 0x01 Type 1 0x10 Type 2
BUFFER_MASK	ECC buffer information. The BCH error correction only allows two configurations of the buffer mask - software may either read just the first block on the flash page or the entire flash page. Write operations must be for the entire flash page. Invalid buffer mask values will cause the DMA descriptor command to be terminated. BCH_AUXONLY = 0x100 Set to request transfer from only the auxiliary buffer (block 0 on flash). BCH_PAGE = 0x1FF Set to request transfer to/from the entire page.

9.7.5.4 GPMI Integrated ECC Transfer Count Register Description (GPMI_ECCCOUNT)

The GPMI ECC Transfer Count Register contains the count of bytes that flow through the ECC / Randomizer subsystem.

GPMI_ECCCOUNT 0x030

Address: 3300_2000h base + 30h offset = 3300_2030h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R									RANDOMIZER_PAGE								COUNT															
W	0																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPMI_ECCCOUNT field descriptions

Field	Description
31–24 -	Always write zeroes to this bit field.
23–16 RANDOMIZER_ PAGE	Set NAND page number needed to be randomized. The value is between 0-255
COUNT	Number of bytes to pass through ECC. This is the GPMI transfer count plus the syndrome count that will be inserted into the stream by the ECC. In DMA2ECC_MODE this count must match the GPMI_CTRL0_XFER_COUNT. A value of zero will transfer 64K words.

9.7.5.5 GPMI Payload Address Register Description (GPMI_PAYLOAD)

The GPMI payload address register specifies the location of the data buffers in system memory. This value must be word aligned.

GPMI_PAYLOAD 0x040

Address: 3300_2000h base + 40h offset = 3300_2040h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	ADDRESS															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ADDRESS														RSVD0	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPMI_PAYLOAD field descriptions

Field	Description
31–2 ADDRESS	Pointer to an array of one or more 512 byte payload buffers.
RSVD0	Always write zeroes to this bit field.

9.7.5.6 GPMI Auxiliary Address Register Description (GPMI_AUXILIARY)

The GPMI auxiliary address register specifies the location of the auxiliary buffers in system memory. This value must be word aligned.

GPMI_AUXILIARY 0x050

Address: 3300_2000h base + 50h offset = 3300_2050h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	ADDRESS															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

General Purpose Media Interface (GPMI)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ADDRESS														RSVD0	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPMI_AUXILIARY field descriptions

Field	Description
31–2 ADDRESS	Pointer to ECC control structure and meta-data storage.
RSVD0	Always write zeroes to this bit field.

9.7.5.7 GPMI Control Register 1 Description (GPMI_CTRL1n)

The GPMI control register 1 specifies additional control fields that are not used on a per-transaction basis.

Address: 3300_2000h base + 60h offset + (4d × i), where i=0d to 3d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DEV_CLK_STOP	SSYNC_CLK_STOP	WRITE_CLK_STOP	TOGGLE_MODE	GPMI_CLK_DIV2_EN	UPDATE_CS	SSYNCMODE	DECOUPLE_CS	WRN_DLY_SEL		TEST_TRIGGER	TIMEOUT_IRQ_EN	GANGED_RDYBUSY	BCH_MODE	DLL_ENABLE	HALF_PERIOD
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	RDN_DELAY				DMA2ECC_MODE	DEV_IRQ	TIMEOUT_IRQ	BURST_EN	ABORT_WAIT_REQUEST	ABORT_WAIT_FOR_READY_CHANNEL			DEV_RESET	ATA_IRQRDY_POLARITY	CAMERA_MODE	GPMI_MODE
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0

GPMI_CTRL1n field descriptions

Field	Description
31 DEV_CLK_STOP	set this bit to 1 will stop gpmi io working clk.
30 SSYNC_CLK_STOP	set this bit to 1 will stop the source synchronous mode clk.

Table continues on the next page...

GPMI_CTRL1n field descriptions (continued)

Field	Description
29 WRITE_CLK_STOP	In onfi source synchronous mode, host may save power during the data write cycles by holding the CLK signal high (i.e. stopping the CLK). The host may only stop the CLK during data write, by setting this bit to 1, if the device supports this feature as indicated in the parameter page.
28 TOGGLE_MODE	enable samsung toggle mode.
27 GPMI_CLK_DIV2_EN	This bit should be reset to 0 in asynchronous mode. The frequency ratio of (device clock : ccm gpmi clock) will be (1 : 1). This bit should be set to 1, in source synchronous mode or toggle mode. The frequency ratio of (device clock : ccm gpmi clock) will be (1 : 2). enable the gpmi clk divider. 0x0 internal factor-2 clock divider is disabled 0x1 internal factor-2 clock divider is enabled.
26 UPDATE_CS	force the CS value is be updated to external chip select pin, even GPMI is idle.
25 SSYNCMODE	source synchronouse mode 1 or asynchronous mode 0. ASYNCR = 0x0 Asynchronous mode. SSYNCR = 0x1 Source Synchronous mode.
24 DECOUPLE_CS	Decouple Chip Select from DMA Channel. Setting this bit to 1 will allow a DMA channel to specify any value in the CTRL0_CS register field. Software can use one DMA channel to access all 8 Nand devices.
23–22 WRN_DLY_SEL	Since the GPMI write strobe (WRN) is a fast clock pin, the delay on this signal can be programmed to match the load on this pin. 0 = ~2ns; 1 = ~4ns; 2 = ~6ns; 3 = no delay.
21 TEST_TRIGGER	Test Trigger Enable 0 Disable 1 Enable
20 TIMEOUT_IRQ_EN	Setting this bit to '1' will enable timeout IRQ for transfers in ATA mode only, and for WAIT_FOR_READY commands in both ATA and Nand mode. The Device_Busy_Timeout value is used for this timeout.
19 GANGED_RDYBUSY	Set this bit to 1 will force all Nand RDY_BUSY inputs to be sourced from (tied to) RDY_BUSY0. This will free up all, except one, RDY_BUSY input pins.
18 BCH_MODE	This bit selects which error correction unit will access GPMI. This bit must always be set to '1', since only the BCH unit is available in this design.
17 DLL_ENABLE	Set this bit to 1 to enable the GPMI DLL. This is required for fast NAND reads (above 30 MHz read strobe). After setting this bit, wait 64 GPMI clock cycles for the DLL to lock before performing a NAND read.
16 HALF_PERIOD	Set this bit to 1 if the GPMI clock period is greater than 16ns for proper DLL operation. DLL_ENABLE must be zero while changing this field.
15–12 RDN_DELAY	This variable is a factor in the calculated delay to apply to the internal read strobe for correct read data sampling. The applied delay (AD) is between 0 and 1.875 times the reference period (RP). RP is one half of the GPMI clock period if HALF_PERIOD=1 otherwise it is the full GPMI clock period. The equation is: AD = RDN_DELAY x 0.125 x RP. This value must not exceed 16ns.

Table continues on the next page...

GPMI_CTRL1n field descriptions (continued)

Field	Description
	This variable is used to achieve faster NAND access. For example if the Read Strobe is asserted from time 0 to 13ns but the read access time is 20ns, then choose AD=12ns will cause the data to be sampled at time 25ns (13+12) giving a 5ns data setup time. If RP=13ns then $RDN_DELAY = 12 / (0.125 \times 13ns)$ $= 7.38$ (0111b). DLL_ENABLE must be zero while changing this field.
11 DMA2ECC_ MODE	This is mainly for testing HWECC without involving the Nand device. Setting this bit will cause DMA write data to redirected to HWECC module (instead of Nand Device) for encoding or decoding.
10 DEV_IRQ	This bit is set when an Interrupt is received from the ATA device. Write 0 to clear.
9 TIMEOUT_IRQ	This bit is set when a timeout occurs using the Device_Busy_Timeout value. Write 0 to clear.
8 BURST_EN	When set to 1 each DMA request will generate a 4-transfer burst on the APB bus.
7 ABORT_WAIT_ REQUEST	Request to abort "wait for ready" command on channel indicated by ABORT_WAIT_FOR_READY_CHANNEL. Hardware will clear this bit when abort is done.
6-4 ABORT_WAIT_ FOR_READY_ CHANNEL	Abort a wait for ready command on selected channel. Set the ABORT_WAIT_REQUEST to kick of operation.
3 DEV_RESET	ENABLED = 0x0 NANDF_WP_B(WPN) pin is held low (asserted). DISABLED = 0x1 NANDF_WP_B(WPN) pin is held high (de-asserted). 0 NANDF_WP_B pin is held low (asserted). 1 NANDF_WP_B pin is held high (de-asserted).
2 ATA_IRQRDY_ POLARITY	For ATA MODE: Note NAND_RDY_BUSY[3:2] are not affected by this bit. ACTIVELOW = 0x0 ATA IORDY and IRQ are active low, or NAND_RDY_BUSY[1:0] are active low ready. ACTIVEHIGH = 0x1 ATA IORDY and IRQ are active high, or NAND_RDY_BUSY[1:0] are active high ready. 0 External ATA IORDY and IRQ are active low. 1 External ATA IORDY and IRQ are active high. For NAND MODE: 0 External RDY_BUSY[1] and RDY_BUSY[0] pins are ready when low and busy when high. 1 External RDY_BUSY[1] and RDY_BUSY[0] pins are ready when high and busy when low.
1 CAMERA_MODE	When set to 1 and ATA UDMA is enabled the UDMA interface becomes a camera interface.
0 GPMI_MODE	ATA mode is only supported on channel zero. If ATA mode is selected, then only channel three is available for NAND use. NAND = 0x0 NAND mode. ATA = 0x1 ATA mode.

Table continues on the next page...

GPMI_CTRL1n field descriptions (continued)

Field	Description
0	NAND mode.
1	ATA mode.

9.7.5.8 GPMI Timing Register 0 Description (GPMI_TIMING0)

The GPMI timing register 0 specifies the timing parameters that are used by the cycle state machine to guarantee the various setup, hold and cycle times for the external media type.

GPMI_TIMING0 0x070

Address: 3300_2000h base + 70h offset = 3300_2070h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R									ADDRESS_SETUP								DATA_HOLD								DATA_SETUP							
W	0																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	1

GPMI_TIMING0 field descriptions

Field	Description
31–24 RSVD1	Always write zeroes to this bit field.
23–16 ADDRESS_SETUP	Number of GPMICLK cycles that the CE/ADDR signals are active before a strobe is asserted. A value of zero is interpreted as 0. For ATA PIO modes this is known in the ATA7 specification as "Address valid to DIOR-/DIOW- setup"
15–8 DATA_HOLD	Data bus hold time in GPMICLK cycles. Also the time that the data strobe is de-asserted in a cycle. A value of zero is interpreted as 256. For ATA PIO modes this is known in the ATA7 specification as "DIOR-/DIOW- recovery time"
DATA_SETUP	Data bus setup time in GPMICLK cycles. Also the time that the data strobe is asserted in a cycle. This value must be greater than 2 for ATA devices that use IORDY to extend transfer cycles. A value of zero is interpreted as 256. For ATA PIO modes this is known in the ATA7 specification as ""DIOR-/DIOW-"

9.7.5.9 GPMI Timing Register 1 Description (GPMI_TIMING1)

The GPMI timing register 1 specifies the timeouts used when monitoring the NAND READY pin or the ATA IRQ and IOWAIT signals.

GPMI_TIMING1 0x080

General Purpose Media Interface (GPMI)

Address: 3300_2000h base + 80h offset = 3300_2080h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DEVICE_BUSY_TIMEOUT																RSVD1															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPMI_TIMING1 field descriptions

Field	Description
31–16 DEVICE_BUSY_TIMEOUT	Timeout waiting for NAND Ready/Busy or ATA IRQ. Used in WAIT_FOR_READY mode. This value is the number of GPMI_CLK cycles multiplied by 4096.
RSVD1	Always write zeroes to this bit field.

9.7.5.10 GPMI Timing Register 2 Description (GPMI_TIMING2)

The GPMI timing register 2 specifies the double data rate timing parameters that are used by the cycle state machine to guarantee the various cs delay, pre-amble delay, post-amble delay, command/address delay, data delay, TCR, TRPSTH, and read latency cycle times for the external media type.

GPMI_TIMING2 0x090

Address: 3300_2000h base + 90h offset = 3300_2090h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
R	TRPSTH				TCR				READ_LATENCY				RSVD0				CE_DELAY				PREAMBLE_DELAY				POSTAMBLE_DELAY				CMDADD_PAUSE				DATA_PAUSE			
W																																				
Reset	0	0	0	0	0	0	1	1	0	0	0	0	0	0	1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	1	1	0				

GPMI_TIMING2 field descriptions

Field	Description
31–29 TRPSTH	Only for Toggle NAND timing control delay TRPSTH GPMICLK cycles for CEn_B high to RE_B high, A value of zero is interpreted as 8
28–27 TCR	Only for Toggle NAND timing control delay (TCR+1) GPMICLK cycles for CEn_B low to RE_B low, 0 is less than or equal to TCR, which is less than the PREAMBLE_DELAY
26–24 READ_LATENCY	This field is for double data rate read latency configuration. others READ LATENCY is 3 000 READ LATENCY is 0 001 READ LATENCY is 1 010 READ LATENCY is 2 011 READ LATENCY is 3 100 READ LATENCY is 4 101 READ LATENCY is 5

Table continues on the next page...

GPMI_TIMING2 field descriptions (continued)

Field	Description
23–21 RSVD0	Always write zeroes to this bit field.
20–16 CE_DELAY	GPMI dealy from CEn assert to W/Rn changing edge. value of zero is interpreted as 32.
15–12 PREAMBLE_ DELAY	GPMI pre-amble delay in GPMICLK cycles. A value of zero is interpreted as 16.
11–8 POSTAMBLE_ DELAY	GPMI post-amble delay in GPMICLK cycles. A value of zero is interpreted as 16.
7–4 CMDADD_ PAUSE	GPMI delay time from command or addres pause to command or address resume in GPMICLK cycles. A value of zero is interpreted as 16.
DATA_PAUSE	GPMI delay time from data pause to data resume in GPMICLK cycles. A value of zero is interpreted as 16.

9.7.5.11 GPMI DMA Data Transfer Register Description (GPMI_DATA)

The GPMI DMA data transfer register is used by the DMA to read or write data to or from the ATA/NAND control state machine.

GPMI_DATA 0x0A0

Address: 3300_2000h base + A0h offset = 3300_20A0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
	DATA																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPMI_DATA field descriptions

Field	Description
DATA	In 8-bit mode, one, two, three or four bytes can can be accessed to send the same number of bus cycles.

9.7.5.12 GPMI Status Register Description (GPMI_STAT)

The GPMI control and status register provides a read back path for various operational states of the GPMI controller.

GPMI_STAT 0x0B0

General Purpose Media Interface (GPMI)

Address: 3300_2000h base + B0h offset = 3300_20B0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	READY_BUSY								RDY_TIMEOUT								
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	DEV7_ERROR	DEV6_ERROR	DEV5_ERROR	DEV4_ERROR	DEV3_ERROR	DEV2_ERROR	DEV1_ERROR	DEV0_ERROR	RSVD1				ATA_IRQ	INVALID_BUFFER_MASK	FIFO_EMPTY	FIFO_FULL	PRESENT
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	

GPMI_STAT field descriptions

Field	Description
31–24 READY_BUSY	Read-only view of NAND Ready_Busy Input pins.
23–16 RDY_TIMEOUT	State of the RDY/BUSY Timeout Flags. When any bit is set to '1' in this field, it indicates that a time out has occurred while waiting for the ready state of the requested NAND device. Multiple bits may be set simultaneously. When GPMI_CTRL1_DECOUPLE_CS = 0, RDY_TIMEOUT[n] is associated with the NAND device on chip_select[n]. When GPMI_CTRL1_DECOUPLE_CS = 1, these flags become associated to a DMA channel instead of a NAND device. For example if DMA channel 6 sends a WAIT_FOR_READY command for NAND Device 2, and a timeout occurred on READY_BUSY2, then READY_TIMEOUT[6] will be set instead of READY_TIMEOUT[2].
15 DEV7_ERROR	DMA channel 7 (Timeout or compare failure, depending on COMMAND_MODE). 0 No error condition present on ATA/NAND Device accessed by DMA channel 7. 1 An Error has occurred on ATA/NAND Device accessed by
14 DEV6_ERROR	DMA channel 6 (Timeout or compare failure, depending on COMMAND_MODE). 0 No error condition present on ATA/NAND Device accessed by DMA channel 6. 1 An Error has occurred on ATA/NAND Device accessed by
13 DEV5_ERROR	DMA channel 5 (Timeout or compare failure, depending on COMMAND_MODE). 0 No error condition present on ATA/NAND Device accessed by DMA channel 5. 1 An Error has occurred on ATA/NAND Device accessed by
12 DEV4_ERROR	DMA channel 4 (Timeout or compare failure, depending on COMMAND_MODE). 0 No error condition present on ATA/NAND Device accessed by DMA channel 4. 1 An Error has occurred on ATA/NAND Device accessed by
11 DEV3_ERROR	DMA channel 3 (Timeout or compare failure, depending on COMMAND_MODE). 0 No error condition present on ATA/NAND Device accessed by DMA channel 3. 1 An Error has occurred on ATA/NAND Device accessed by
10 DEV2_ERROR	DMA channel 2 (Timeout or compare failure, depending on COMMAND_MODE). 0 No error condition present on ATA/NAND Device accessed by DMA channel 2. 1 An Error has occurred on ATA/NAND Device accessed by
9 DEV1_ERROR	DMA channel 1 (Timeout or compare failure, depending on COMMAND_MODE). 0 No error condition present on ATA/NAND Device accessed by DMA channel 1. 1 An Error has occurred on ATA/NAND Device accessed by
8 DEV0_ERROR	DMA channel 0 (Timeout or compare failure, depending on COMMAND_MODE). 0 No error condition present on ATA/NAND Device accessed by DMA channel 0. 1 An Error has occurred on ATA/NAND Device accessed by
7–5 RSVD1	Always write zeroes to this bit field.

Table continues on the next page...

GPMI_STAT field descriptions (continued)

Field	Description
4 ATA_IRQ	Status of the ATA_IRQ input pin.
3 INVALID_BUFFER_MASK	Buffer Mask Validity bit. 0 ECC Buffer Mask is not invalid. 1 ECC Buffer Mask is invalid.
2 FIFO_EMPTY	NOT_EMPTY = 0x0 FIFO is not empty. EMPTY = 0x1 FIFO is empty. 0 FIFO is not empty. 1 FIFO is empty.
1 FIFO_FULL	NOT_FULL = 0x0 FIFO is not full. FULL = 0x1 FIFO is full. 0 FIFO is not full. 1 FIFO is full.
0 PRESENT	UNAVAILABLE = 0x0 GPMI is not present in this product. AVAILABLE = 0x1 GPMI is present in this product. 0 GPMI is not present in this product. 1 GPMI is present is in this product.

9.7.5.13 GPMI Debug Information Register Description (GPMI_DEBUG)

The GPMI debug information register provides a read back path for diagnostics to determine the current operating state of the GPMI controller.

GPMI_DEBUG 0x0C0

Address: 3300_2000h base + C0h offset = 3300_20C0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	WAIT_FOR_READY_END								DMA_SENSE								DMAREQ								CMD_END							
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPMI_DEBUG field descriptions

Field	Description
31–24 WAIT_FOR_READY_END	Read Only view of the Wait_For_Ready End toggle signals to DMA. One per channel

Table continues on the next page...

GPMI_DEBUG field descriptions (continued)

Field	Description
23–16 DMA_SENSE	Read-only view of sense state of the 8 DMA channels. A value of "1" in any bit position indicates that a read and compare command failed or a timeout occurred for the corresponding channel.
15–8 DMAREQ	Read-only view of DMA request line for 8 DMA channels. A toggle on any bit position indicates a DMA request for the corresponding channel.
CMD_END	Read Only view of the Command End toggle signals to DMA. One per channel

9.7.5.14 GPMI Version Register Description (GPMI_VERSION)

This register reflects the version number for the GPMI.

GPMI_VERSION 0x0D0

Address: 3300_2000h base + D0h offset = 3300_20D0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	MAJOR								MINOR								STEP															
W																																
Reset	0	0	0	0	0	1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

GPMI_VERSION field descriptions

Field	Description
31–24 MAJOR	Fixed read-only value reflecting the MAJOR field of the RTL version.
23–16 MINOR	Fixed read-only value reflecting the MINOR field of the RTL version.
STEP	Fixed read-only value reflecting the stepping of the RTL version.

9.7.5.15 GPMI Debug2 Information Register Description (GPMI_DEBUG2)

The GPMI Debug2 information register provides a read back path for diagnostics to determine the current operating state of the GPMI controller.

GPMI_DEBUG2 0x0E0

General Purpose Media Interface (GPMI)

Address: 3300_2000h base + E0h offset = 3300_20E0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	RSVD1				UDMA_STATE				BUSY	PIN_STATE				MAIN_STATE			
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	SYND2GPMI_BE				GPMI2SYND_VALID	GPMI2SYND_READY	SYND2GPMI_VALID	SYND2GPMI_READY	VIEW_DELAYED_RDN	UPDATE_WINDOW	RDN_TAP					
W																
Reset	1	1	1	1	0	0	0	1	0	0	0	0	0	0	0	0

GPMI_DEBUG2 field descriptions

Field	Description
31–28 RSVD1	Always write zeroes to this bit field.
27–24 UDMA_STATE	USM_IDLE = 4'h0, idle USM_DMARQ = 4'h1, DMA req USM_ACK = 4'h2, DMA ACK USM_FIFO_E = 4'h3, Fifo empty USM_WPAUSE = 4'h4, WR DMA Paused by device USM_TSTRB = 4'h5, Toggle HSTROBE USM_CAPTUR = 4'h6, Capture Stage, (data sampled with DSTROBE is valid) USM_DATOUT = 4'h7, Change Burst DATAOUT USM_CRC = 4'h8, Source CRC to Device USM_WAIT_R = 4'h9, Waiting for DDMARDY- USM_END = 4'ha; Negate DMAACK (end of DMA) USM_WAIT_S = 4'hb, Waiting for DSTROBE USM_RPAUSE = 4'hc, Rd DMA Paused by Host USM_RSTOP = 4'hd, Rd DMA Stopped by Host USM_WTERM = 4'he, Wr DMA Termination State USM_RTERM = 4'hf, Rd DMA Termination state
23 BUSY	When asserted the GPMI is busy. Undefined results may occur if any registers are written when BUSY is asserted. DISABLED = 0x0 The GPMI is not busy. ENABLED = 0x1 The GPMI is busy.
22–20 PIN_STATE	parameter PSM_IDLE = 3'h0, PSM_BYTCNT = 3'h1, PSM_ADDR = 3'h2, PSM_STALL = 3'h3, PSM_STROBE = 3'h4, PSM_ATARDY = 3'h5, PSM_DHOLD = 3'h6, PSM_DONE = 3'h7. PSM_IDLE = 0x0 PSM_BYTCNT = 0x1 PSM_ADDR = 0x2 PSM_STALL = 0x3 PSM_STROBE = 0x4 PSM_ATARDY = 0x5 PSM_DHOLD = 0x6 PSM_DONE = 0x7
19–16 MAIN_STATE	parameter MSM_IDLE = 4'h0, MSM_BYTCNT = 4'h1, MSM_WAITFE = 4'h2, MSM_WAITFR = 4'h3, MSM_DMAREQ = 4'h4, MSM_DMAACK = 4'h5, MSM_WAITFF = 4'h6, MSM_LDFIFO = 4'h7, MSM_LDDMAR = 4'h8, MSM_RDCMP = 4'h9, MSM_DONE = 4'ha. MSM_IDLE = 0x0 MSM_BYTCNT = 0x1 MSM_WAITFE = 0x2 MSM_WAITFR = 0x3

Table continues on the next page...

GPMI_DEBUG2 field descriptions (continued)

Field	Description
	MSM_DMAREQ = 0x4 MSM_DMAACK = 0x5 MSM_WAITFF = 0x6 MSM_LDFIFO = 0x7 MSM_LDDMAR = 0x8 MSM_RDCMP = 0x9 MSM_DONE = 0xA
15–12 SYND2GPMI_BE	Data byte enable Input from BCH.
11 GPMI2SYND_VALID	Data handshake output to BCH.
10 GPMI2SYND_READY	Data handshake output to BCH.
9 SYND2GPMI_VALID	Data handshake Input from BCH.
8 SYND2GPMI_READY	Data handshake Input from BCH.
7 VIEW_DELAYED_RDN	Set to a 1 to select the delayed feedback RE_B to drive the GPMI_ADDR[0] (Nand CLE) pin. For debug purposes, this will allow you see if DLL is functioning properly.
6 UPDATE_WINDOW	A 1 indicates that the DLL is busy generating the required delay.
RDN_TAP	This is the DLL tap calculated by the DLL controller. The selects the amount of delay form the DLL chain.

9.7.5.16 GPMI Debug3 Information Register Description (GPMI_DEBUG3)

The GPMI Debug3 information register provides a read back path for diagnostics to determine the current operating state of the GPMI controller.

GPMI_DEBUG3 0x0F0

Address: 3300_2000h base + F0h offset = 3300_20F0h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	APB_WORD_CNTR																DEV_WORD_CNTR															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPMI_DEBUG3 field descriptions

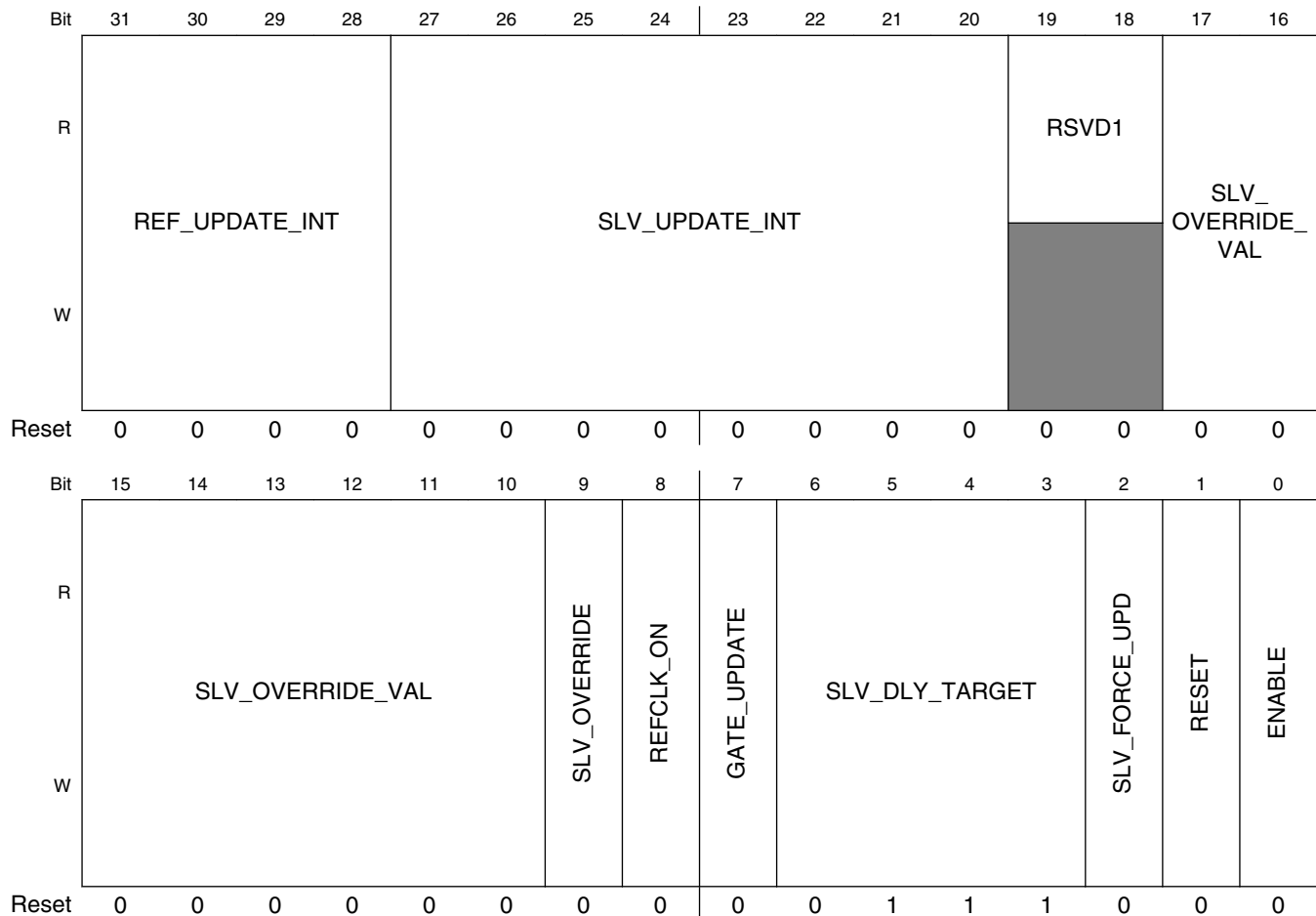
Field	Description
31–16 APB_WORD_CNTR	Reflects the number of bytes remains to be transferred on the APB bus.
DEV_WORD_CNTR	Reflects the number of bytes remains to be transferred on the ATA/Nand bus.

9.7.5.17 GPMI Double Rate Read DLL Control Register Description (GPMI_READ_DDR_DLL_CTRL)

GPMI DDR Read Delay Loop Lock Control Register. This register provides programmability in DDR mode for data input timing and data formats.

GPMI_READ_DDR_DLL_CTRL 0x100

Address: 3300_2000h base + 100h offset = 3300_2100h



GPMI_READ_DDR_DLL_CTRL field descriptions

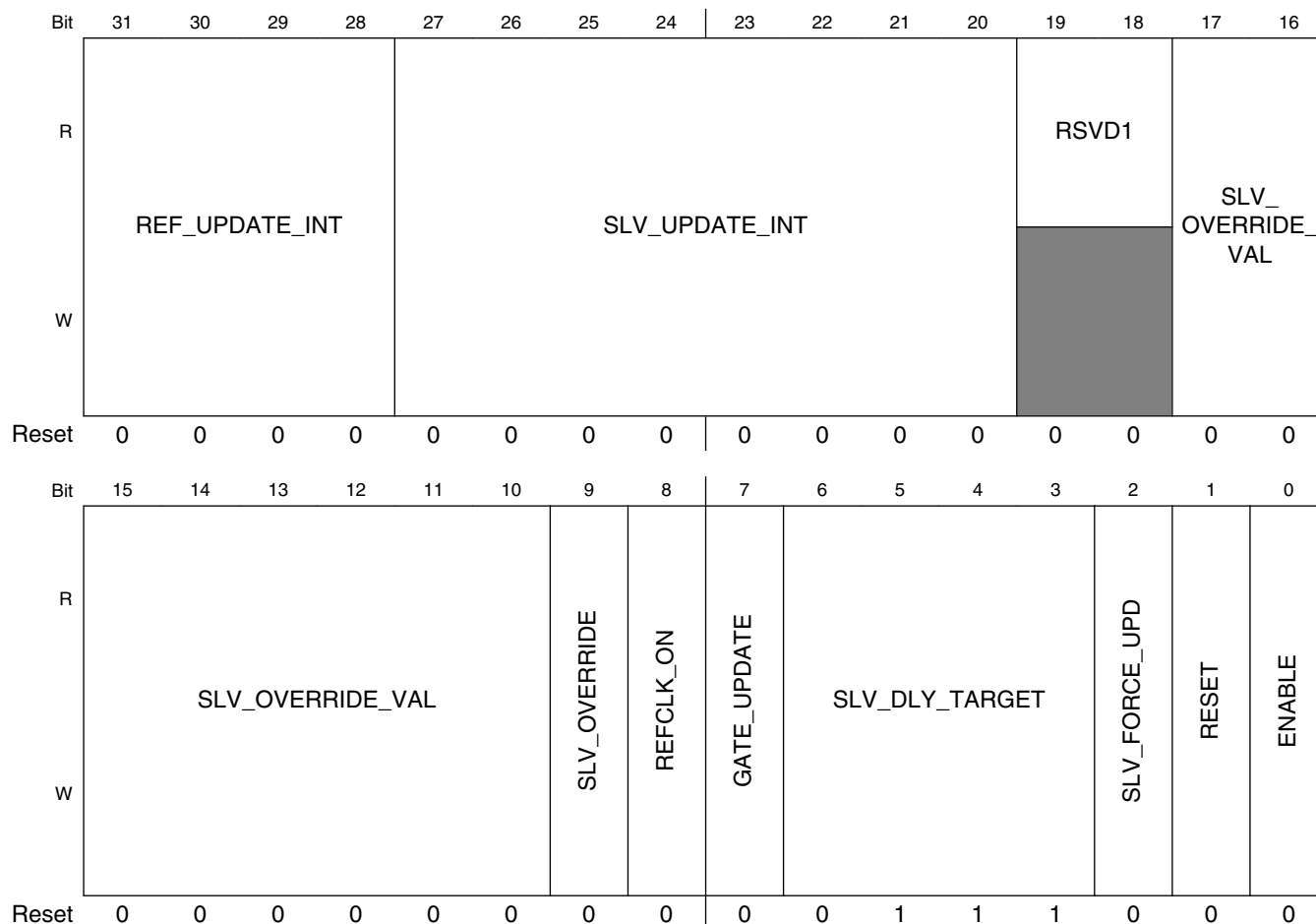
Field	Description
31–28 REF_UPDATE_INT	This field allows the user to add additional delay cycles to the DLL control loop (reference delay line control). By default, the DLL control loop shall update every two GPMICLK cycles. Programming this field results in a DLL control loop update interval of $(2 + \text{REF_UPDATE_INT}) * \text{GPMICLK}$. It should be noted that increasing the reference delay-line update interval reduces the ability of the DLL to adjust to fast changes in conditions that may effect the delay (such as voltage and temperature)
27–20 SLV_UPDATE_INT	Setting a value greater than 0 in this field, shall over-ride the default slave delay-line update interval of 256 GPMICLK cycles. A value of 0 results in an update interval of 256 GPMICLK cycles (default setting). A value of 0x0f results in 15 cycles and so on. Note that software can always cause an update of the slave-delay line using the SLV_FORCE_UPDATE register. Note that the slave delay line will also update automatically when the reference DLL transitions to a locked state (from an un-locked state).
19–18 RSVD1	Reserved
17–10 SLV_OVERRIDE_VAL	When SLV_OVERRIDE=1 This field is used to select 1 of 256 physical taps manually. A value of 0 selects tap 1, and a value of 0x7f selects tap 256.
9 SLV_OVERRIDE	Set this bit to 1 to Enable manual override for slave delay chain using SLV_OVERRIDE_VAL; to set 0 to disable manual override. This feature does not require the DLL to be enabled using the ENABLE bit. In fact to reduce power, if SLV_OVERRIDE is used, it is recommended to disable the DLL with ENABLE=0
8 REFCLK_ON	set this bit to 1 will turn on the reference clock
7 GATE_UPDATE	Setting this bit to 1, forces the slave delay line not update
6–3 SLV_DLY_TARGET	The delay target for the read clock is can be programmed in 1/16th increments of an GPMICLK half-period. So the input read-clock can be delayed relative input data from $(\text{GPMICLK}/2)/16$ to $\text{GPMICLK}/2$.
2 SLV_FORCE_UPD	Setting this bit to 1, forces the slave delay line to update to the DLL calibrated value immediately. The slave delay line shall update automatically based on the SLV_UPDATE_INT interval or when a DLL lock condition is sensed. Subsequent forcing of the slave-line update can only occur if SLV_FORCE_UP is set back to 0 and then asserted again (edge triggered).
1 RESET	Setting this bit to 1 force a reset on DLL. This will cause the DLL to lose lock and re-calibrate to detect an GPMICLK half period phase shift. This signal is used by the DLL as edge-sensitive, so in order to create a subsequent reset, RESET must be taken low and then asserted again.
0 ENABLE	Set this bit to 1 to enable the DLL and delay chain; otherwise; set to 0 to bypasses DLL. Note that using the slave delay line override feature with SLV_OVERRIDE and SLV_OVERRIDE VAL, the DLL does not need to be enabled.

9.7.5.18 GPMI Double Rate Write DLL Control Register Description (GPMI_WRITE_DDR_DLL_CTRL)

GPMI DDR Write Delay Loop Lock Control Register. This register provides programmability in DDR mode for data output timing and data formats.

GPMI_WRITE_DDR_DLL_CTRL 0x110

Address: 3300_2000h base + 110h offset = 3300_2110h

**GPMI_WRITE_DDR_DLL_CTRL field descriptions**

Field	Description
31–28 REF_UPDATE_INT	This field allows the user to add additional delay cycles to the DLL control loop (reference delay line control). By default, the DLL control loop shall update every two GPMICLK cycles. Programming this field results in a DLL control loop update interval of (2 + REF_UPDATE_INT) * GPMICLK. It should be noted that increasing the reference delay-line update interval reduces the ability of the DLL to adjust to fast changes in conditions that may effect the delay (such as voltage and temperature)
27–20 SLV_UPDATE_INT	Setting a value greater than 0 in this field, shall over-ride the default slave delay-line update interval of 256 GPMICLK cycles. A value of 0 results in an update interval of 256 GPMICLK cycles (default setting). A value of 0x0f results in 15 cycles and so on. Note that software can always cause an update of the slave-delay line using the SLV_FORCE_UPDATE register. Note that the slave delay line will also update automatically when the reference DLL transitions to a locked state (from an un-locked state).
19–18 RSVD1	Reserved
17–10 SLV_OVERRIDE_VAL	When SLV_OVERRIDE=1 This field is used to select 1 of 256 physical taps manually. A value of 0 selects tap 1, and a value of 0x7f selects tap 256.
9 SLV_OVERRIDE	Set this bit to 1 to Enable manual override for slave delay chain using SLV_OVERRIDE_VAL; to set 0 to disable manual override. This feature does not require the DLL to be enabled using the ENABLE bit. In fact to reduce power, if SLV_OVERRIDE is used, it is recommended to disable the DLL with ENABLE=0

Table continues on the next page...

GPMI_WRITE_DDR_DLL_CTRL field descriptions (continued)

Field	Description
8 REFCLK_ON	set this bit to 1 will turn on the reference clock
7 GATE_UPDATE	Setting this bit to 1, forces the slave delay line not update
6–3 SLV_DLY_TARGET	The delay target for the read clock can be programmed in 1/16th increments of an GPMICK half-period. So the input read-clock can be delayed relative input data from (GPMICK/2)/16 to GPMICK/2.
2 SLV_FORCE_UPD	Setting this bit to 1, forces the slave delay line to update to the DLL calibrated value immediately. The slave delay line shall update automatically based on the SLV_UPDATE_INT interval or when a DLL lock condition is sensed. Subsequent forcing of the slave-line update can only occur if SLV_FORCE_UP is set back to 0 and then asserted again (edge triggered).
1 RESET	Setting this bit to 1 force a reset on DLL. This will cause the DLL to lose lock and re-calibrate to detect an GPMICK half period phase shift. This signal is used by the DLL as edge-sensitive, so in order to create a subsequent reset, RESET must be taken low and then asserted again.
0 ENABLE	Set this bit to 1 to enable the DLL and delay chain; otherwise, set to 0 to bypasses DLL. Note that using the slave delay line override feature with SLV_OVERRIDE and SLV_OVERRIDE VAL, the DLL does not need to be enabled.

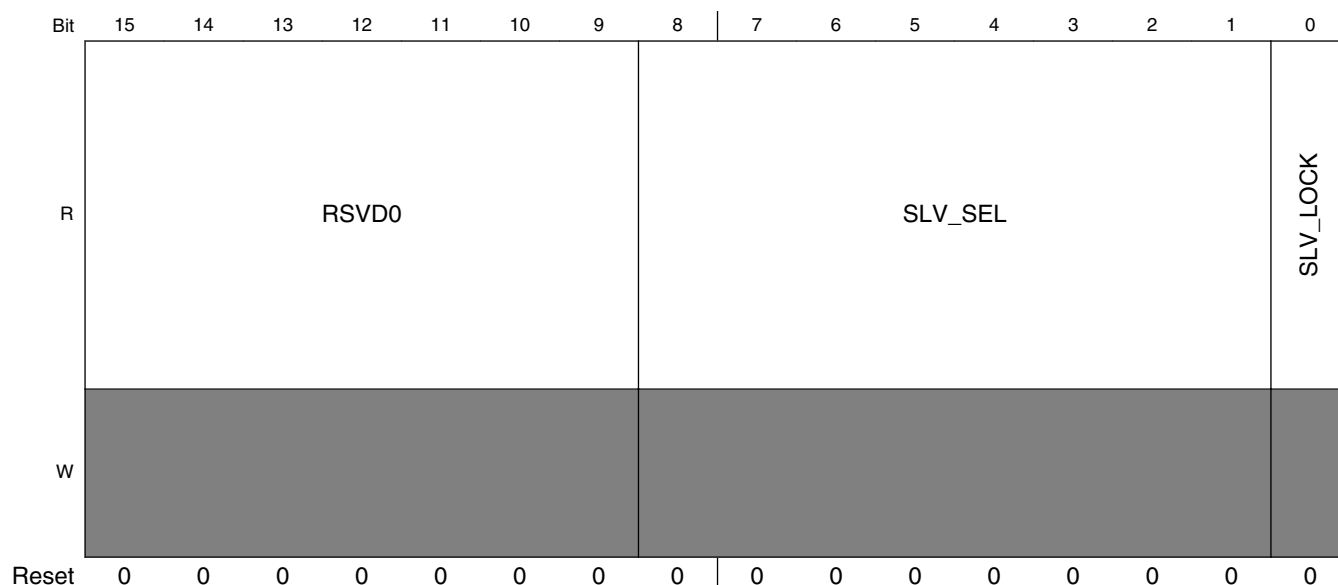
9.7.5.19 GPMI Double Rate Read DLL Status Register Description (GPMI_READ_DDR_DLL_STS)

GPMI Double Rate Read DLL Status Register, Read Only. GPMI DLL status fields are provided in this register.

GPMI_READ_DDR_DLL_STS 0x120

Address: 3300_2000h base + 120h offset = 3300_2120h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	RSVD1								REF_SEL								REF_LOCK
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

**GPMI_READ_DDR_DLL_STS field descriptions**

Field	Description
31–25 RSVD1	Reserved
24–17 REF_SEL	Reference delay line select status.
16 REF_LOCK	Reference DLL lock status. This signifies that the DLL has detected and locked to a half-phase GPMICLK shift, allowing the slave delay-line to perform programmed clock delays.
15–9 RSVD0	Reserved
8–1 SLV_SEL	Slave delay line select status
0 SLV_LOCK	Slave delay-line lock status. This signifies that a valid calibration has been set to the slave-delay line and that the slave-delay line is implementing the programmed delay value.

9.7.5.20 GPMI Double Rate Write DLL Status Register Description (GPMI_WRITE_DDR_DLL_STS)

GPMI Double Rate Write DLL Status Register, Read Only. GPMI DLL status fields are provided in this register.

GPMI_WRITE_DDR_DLL_STS 0x130

General Purpose Media Interface (GPMI)

Address: 3300_2000h base + 130h offset = 3300_2130h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	RSVD1								REF_SEL								REF_LOCK
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	RSVD0								SLV_SEL								SLV_LOCK
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

GPMI_WRITE_DDR_DLL_STS field descriptions

Field	Description
31–25 RSVD1	Reserved
24–17 REF_SEL	Reference delay line select status.
16 REF_LOCK	Reference DLL lock status. This signifies that the DLL has detected and locked to a half-phase GPMICLK shift, allowing the slave delay-line to perform programmed clock delays.
15–9 RSVD0	Reserved
8–1 SLV_SEL	Slave delay line select status

Table continues on the next page...

GPMI_WRITE_DDR_DLL_STS field descriptions (continued)

Field	Description
0 SLV_LOCK	Slave delay-line lock status. This signifies that a valid calibration has been set to the slave-delay line and that the slave-delay line is implementing the programmed delay value.

Chapter 10

Mass Storage

10.1 Enhanced Configurable SPI (ECSPI)

10.1.1 Overview

The Enhanced Configurable Serial Peripheral Interface (ECSPI) is a full-duplex, synchronous, four-wire serial communication block.

The ECSPI contains a 64 x 32 receive buffer (RXFIFO) and a 64 x 32 transmit buffer (TXFIFO). With data FIFOs, the ECSPI allows rapid data communication with fewer software interrupts. The figure below shows a block diagram of the ECSPI.

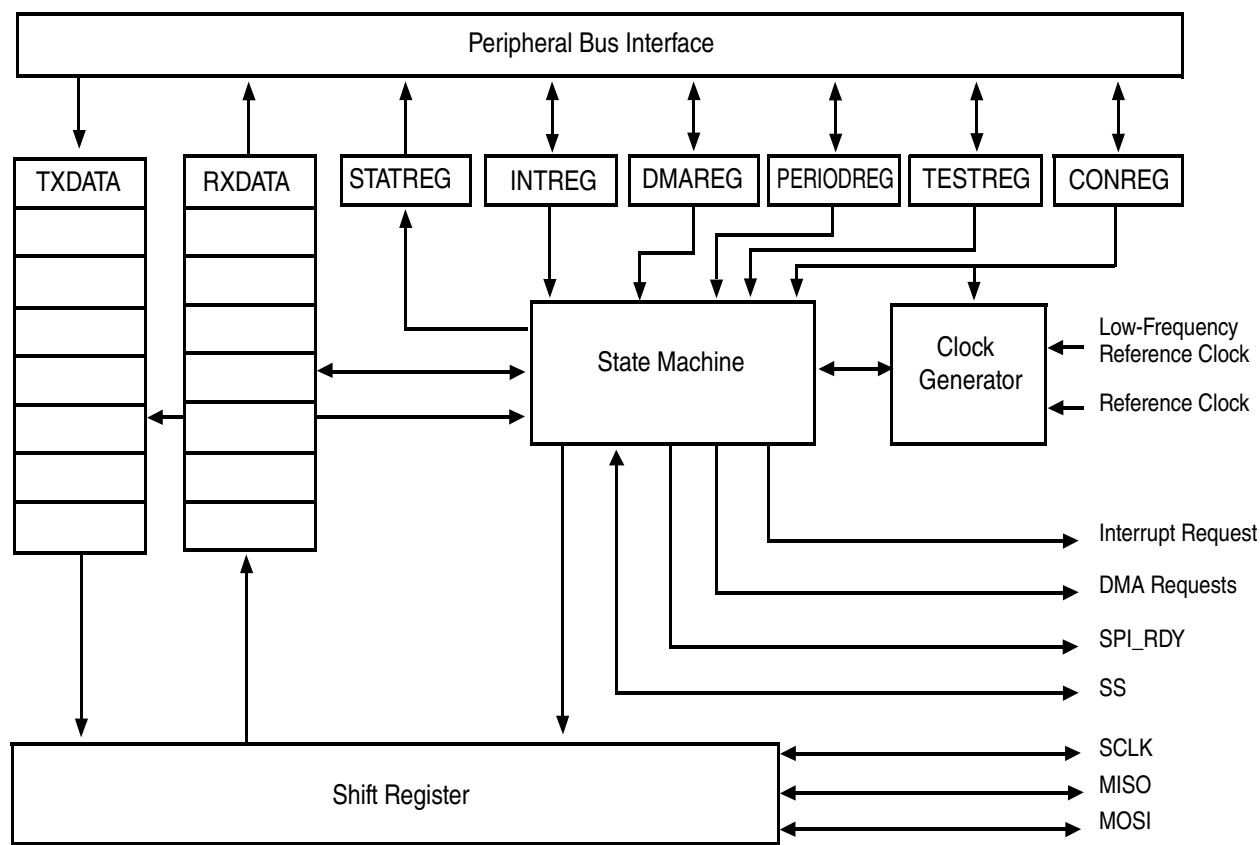


Figure 10-1. ECSPI Block Diagram

10.1.1.1 Features

Key features of the ECSPI include:

- Full-duplex synchronous serial interface
- Master/Slave configurable
- One Chip Select (SS) signal
- Transfer continuation function allows unlimited length data transfers
- 32-bit wide by 64-entry FIFO for both transmit and receive data
- Polarity and phase of the Chip Select (SS) and SPI Clock (SCLK) are configurable
- Direct Memory Access (DMA) support
- Refer to the product data sheet for the maximum operating frequency

10.1.1.2 Modes and Operations

The ECSPI supports the modes described in the indicated sections:

- [Master Mode](#)
- [Slave Mode](#)
- [Low Power Modes](#)

As described in [Operations](#), the ECSPI supports the operations described in the indicated sections:

- [Typical Master Mode](#)
 - [Master Mode with SPI_RDY](#)
 - [Master Mode with Wait States](#)
 - [Master Mode with SS_CTL\[3:0\] Control](#)
 - [Master Mode with Phase Control](#)
- [Typical Slave Mode](#)

10.1.2 Clocks

The following table describes the clock sources for eCSPI. Please see [Clock Controller Module \(CCM\)](#) for clock setting, configuration and gating information.

Table 10-1. eCSPI Clocks

Clock name	Clock Root	Description
ipg_clk	ipg_clk_root	Peripheral clock
ipg_clk_32k	ckil_sync_clk_root	Low-frequency reference clock (32kHz)
ipg_clk_per	ecspi_clk_root	eCSPI module clock
ipg_clk_s	ipg_clk_root	Peripheral access clock

10.1.3 Functional Description

This section provides a complete functional description of the ECSPI. The figure found here shows the relationship of SCLK and data lines while ECSPI has been configured with different POL and PHA settings.

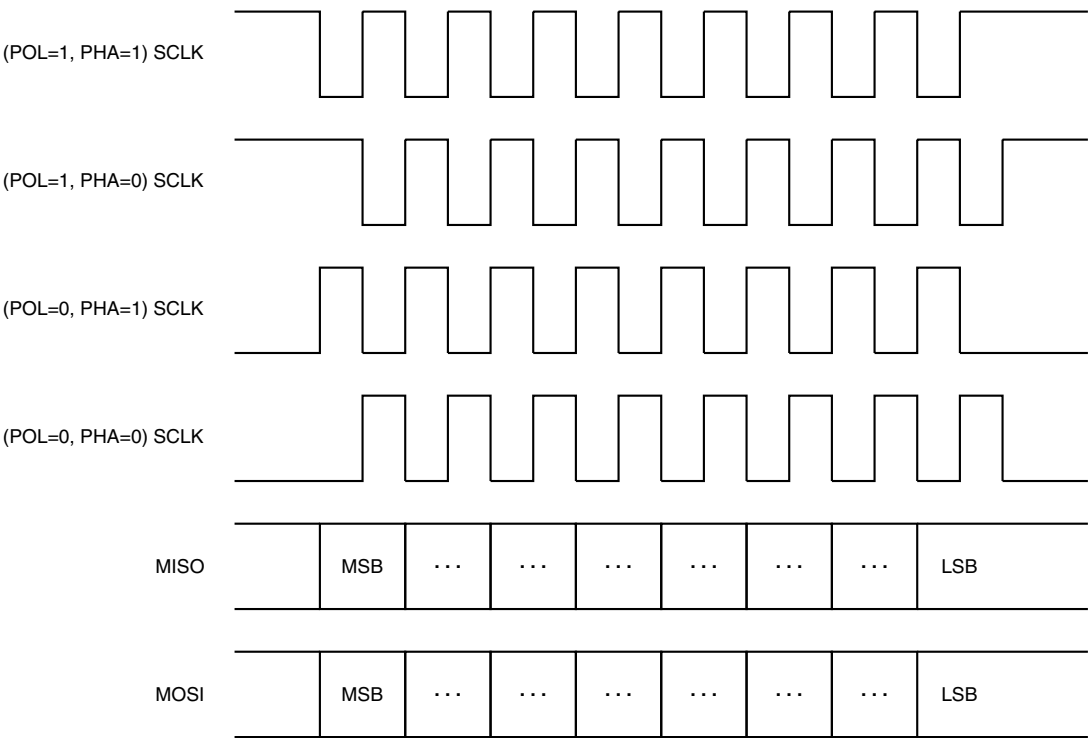


Figure 10-2. ECSPI SCLK, MISO, and MOSI Relationship

10.1.3.1 Master Mode

When the ECSPI is configured as a master, it uses a serial link to transfer data between the ECSPI and an external slave device.

The Chip Select (SS) signal and the clock signal (SCLK) are used to transfer data between two devices. If the external device is a transmit-only device, the ECSPI master's output port can be ignored and used for other purposes. In order to use the internal TXFIFO and RXFIFO, two auxiliary output signals, Chip Select (SS) and SPI_RDY, are used for data transfer rate control. Software can also configure the sample period control register to a fixed data transfer rate.

10.1.3.2 Slave Mode

When the ECSPI is configured as a slave, software can configure the ECSPI Control register to match the external SPI master's timing.

In this configuration, Chip Select (SS) becomes an input signal, and is used to control data transfers through the Shift register, as well as to load/store the data FIFO.

Slave mode only supports the case when `ECSPiX_CONFIGREG[SS_CTL]` is cleared. The accurate burst length should always be specified using the `BURST_LENGTH` parameter. `ECSPiX_CONFIGREG[SS_CTL]` set to 1 is not supported in slave mode.

10.1.3.3 Low Power Modes

The ECSPI does not operate under low power mode.

It holds its operation when its clock is gated off in master mode. In slave mode, the ECSPI does not respond when its clock is gated off.

10.1.3.4 Operations

The information found here describes the ECSPI's operations.

10.1.3.4.1 Typical Master Mode

The ECSPI master uses the Chip Select (SS) signal to enable an external SPI device, and uses the SCLK signal to transfer data in and out of the Shift register.

The `SPI_RDY` enables fast data communication with fewer software interrupts. By programming the `ECSPi_PERIODREG` register accordingly, the ECSPI can be used for a fixed data transfer rate.

When the ECSPI is in Master mode the SS, SCLK, and MOSI are output signals, and the MISO signal is an input.

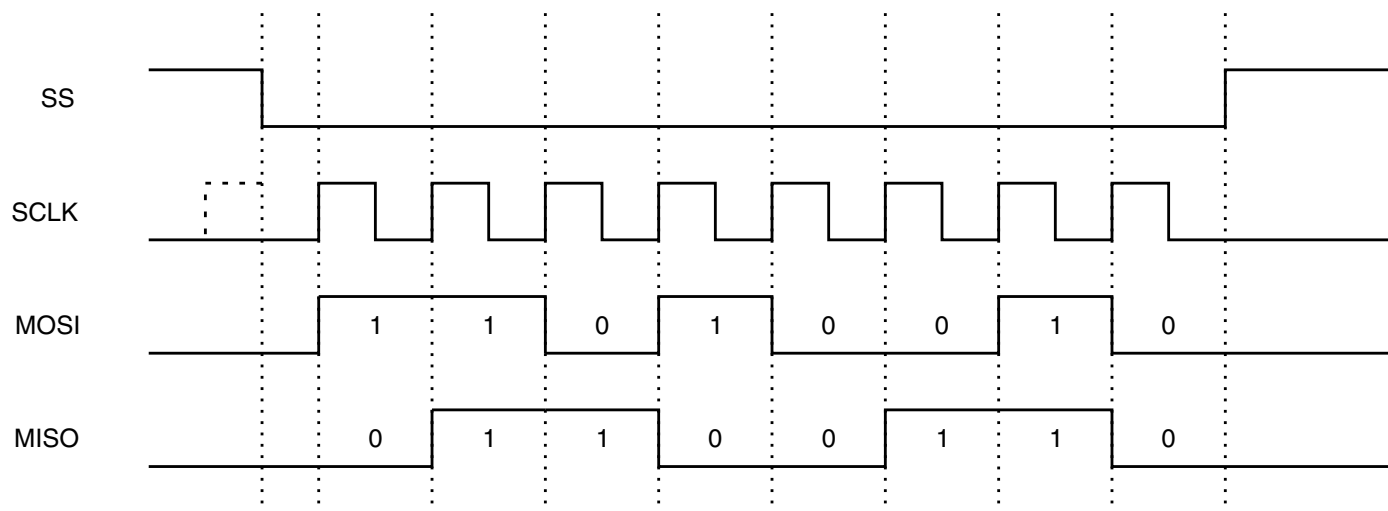


Figure 10-3. Typical SPI Burst (8-bit Transfer)

In the above figure, the Chip Select (SS) signal enables the selected external SPI device, and the SCLK synchronizes the data transfer. The MOSI and MISO signals change on rising edge of SCLK and the MISO signal is latched on the falling edge of the SCLK. The figure above shows a data of 0xD2 is shifted out, and a data of 0x66 is shifted in.

10.1.3.4.1.1 Master Mode with SPI_RDY

By default, the ECSPI does not use the SPI_RDY signal in master mode (MODE =1).

A SPI burst begins when the following events happen:

- The ECSPI is enabled, TXFIFO has data in it, and ECSPI_CONREG[XCH] bit or the ECSPI_CONREG[SMC] bit is set.
- When the SPI Data Ready Control (ECSPI_CONREG[DRCTL]) bits contains either 01 or 10, the SPI_RDY signal controls when a SPI burst starts.

A SPI burst is defined as a bus transaction that starts when the slave select is asserted and ends when the slave select is negated. The Chip Select (SS) signal will remain asserted until all the bits in a SPI burst are shifted out.

If ECSPI_CONREG[DRCTL] is set to 01, the SPI burst can be triggered only if a falling edge of the SPI_RDY signal has been detected.

The following figure shows the relationship between a SPI burst and the falling edge of SPI_RDY signal.

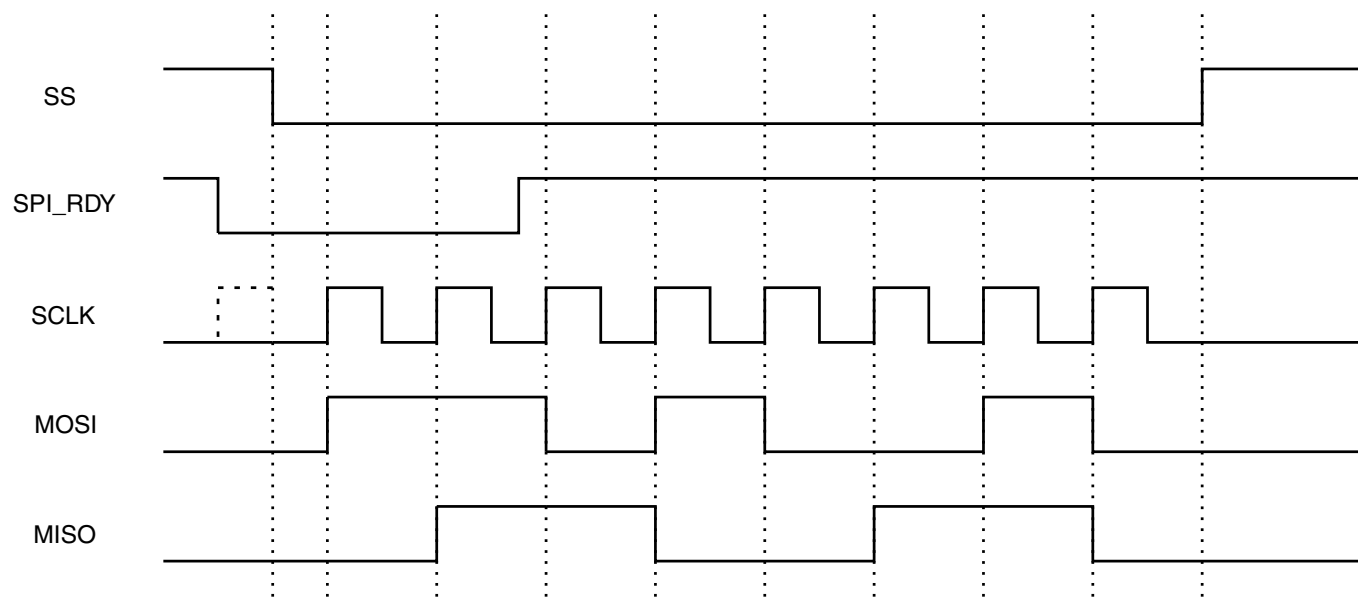


Figure 10-4. Relationship Between a SPI Burst and SPI_RDY: Falling-Edge Triggered

A SPI burst does not start until the falling edge of the SPI_RDY signal is detected. The next SPI burst starts when the next SPI_RDY falling edge is detected, after the last burst has finished.

If SPI Data Ready Control (ECSPI_CONREG[DRCTL]) is set to 10, the SPI burst can be triggered only if the SPI_RDY signal is low.

The following figure shows the relationship between a SPI burst and the SPI_RDY signal. The SPI burst does not begin until the SPI_RDY signal goes low. The ECSPI will keep transmitting SPI burst if the SPI_RDY signal remains low.

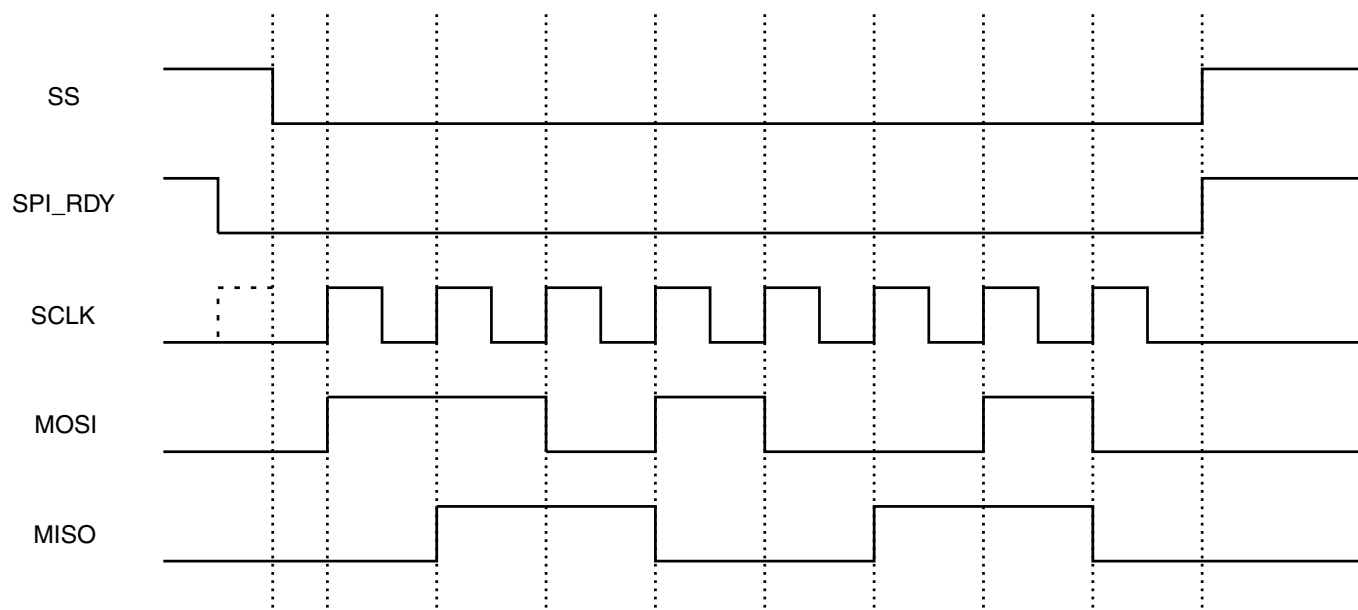


Figure 10-5. Relationship Between a SPI Burst and SPI_RDY: Low-Level Triggered

10.1.3.4.1.2 Master Mode with Wait States

Wait states can be inserted between SPI bursts. This provides a way for software to slow down the SPI burst to meet the timing requirements of a slower SPI device.

The following figure shows wait states inserted between SPI bursts.

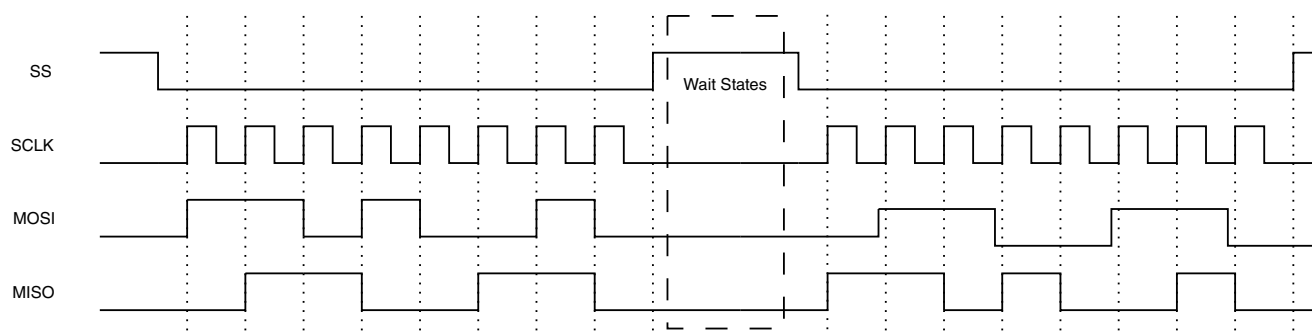


Figure 10-6. SPI Bursts with Wait States

In this case, the number of wait states is controlled by ECSPI_PERIODREG[SAMPLE PERIOD] and the wait states' clock source is selected by ECSPI_PERIODREG[CSRC].

10.1.3.4.1.3 Master Mode with SS_CTL[3:0] Control

The SPI SS Control (SS_CTL[3:0]) controls whether the current operation is single burst or multiple bursts.

When the SPI SS Wave Form Select (SS_CTL[3:0]) is set, the current operation is multiple bursts transfer. When the SPI SS Wave Form Select (SS_CTL[3:0]) bit is cleared, the current operation is single burst transfer. A SPI burst can contains multiple words as defined in the BURST LENGTH field of the ECSPI_CONREG register.

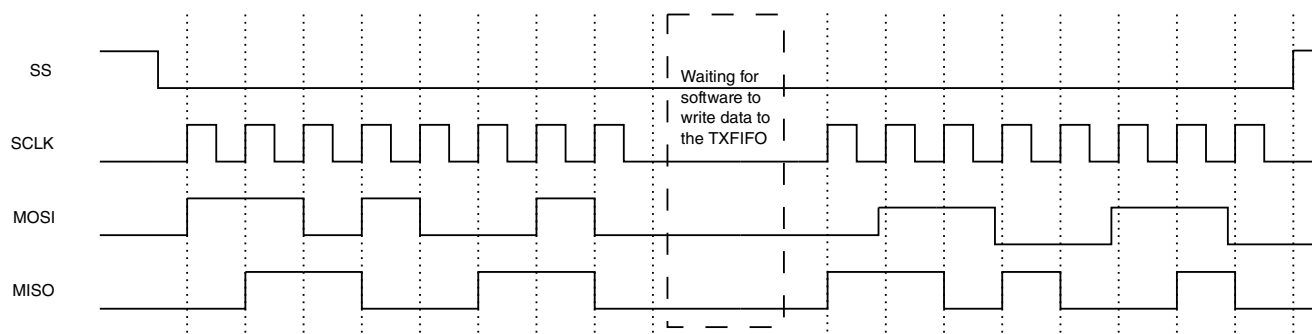


Figure 10-7. SPI Burst While SS_CTL[3:0] is Clear

In [Figure 10-7](#), two 8-bit bursts in the TXFIFO have been combined and transmitted in one SPI burst. The maximum length of a single SPI burst is defined by the BURST LENGTH and limited by the FIFO size. ([Figure 10-7](#) corresponds to a BURST LENGTH of 8.) This provides a way for transferring a longer SPI burst by writing data into TXFIFO while the ECSPI is transmitting.

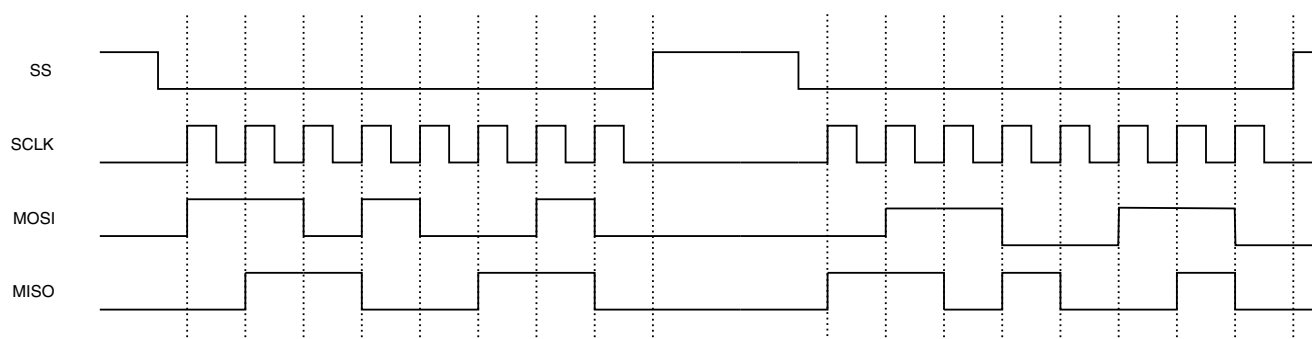


Figure 10-8. SPI Bursts While SS_CTL[3:0] is Set

In [Figure 10-8](#), two FIFO entries are transmitted, one entry with each SPI burst. The ECSPI will continue to transmit SPI bursts until the TXFIFO is empty. When wait states can be inserted between SPI bursts, the SS will negate between SPI bursts until the wait states finish.

10.1.3.4.1.4 Master Mode with Phase Control

The Phase Control (ECSPI_CONREG[PHA]) bit controls how the transmit data shifts out and the receive data shifts in.

When the Phase control (ECSPI_CONREG[PHA]) bit is set, the transmit data will shift out on the rising edge of SCLK, and the receive data is latched on the falling edge of SCLK. The most-significant bit is output on the first rising SCLK edge.

When ECSPI_CONREG[PHA] is cleared, the transmit data is shifted out on the falling edge of SCLK and the receive data is latched on the rising edge of SCLK. The MSB is output when the host processor loads the transmitted data.

Inverting the SCLK polarity does not impact the edge-triggered operations because they are internal to the serial peripheral interface master. [Figure 10-9](#) shows how SPI burst works with different POL and PHA configuration.

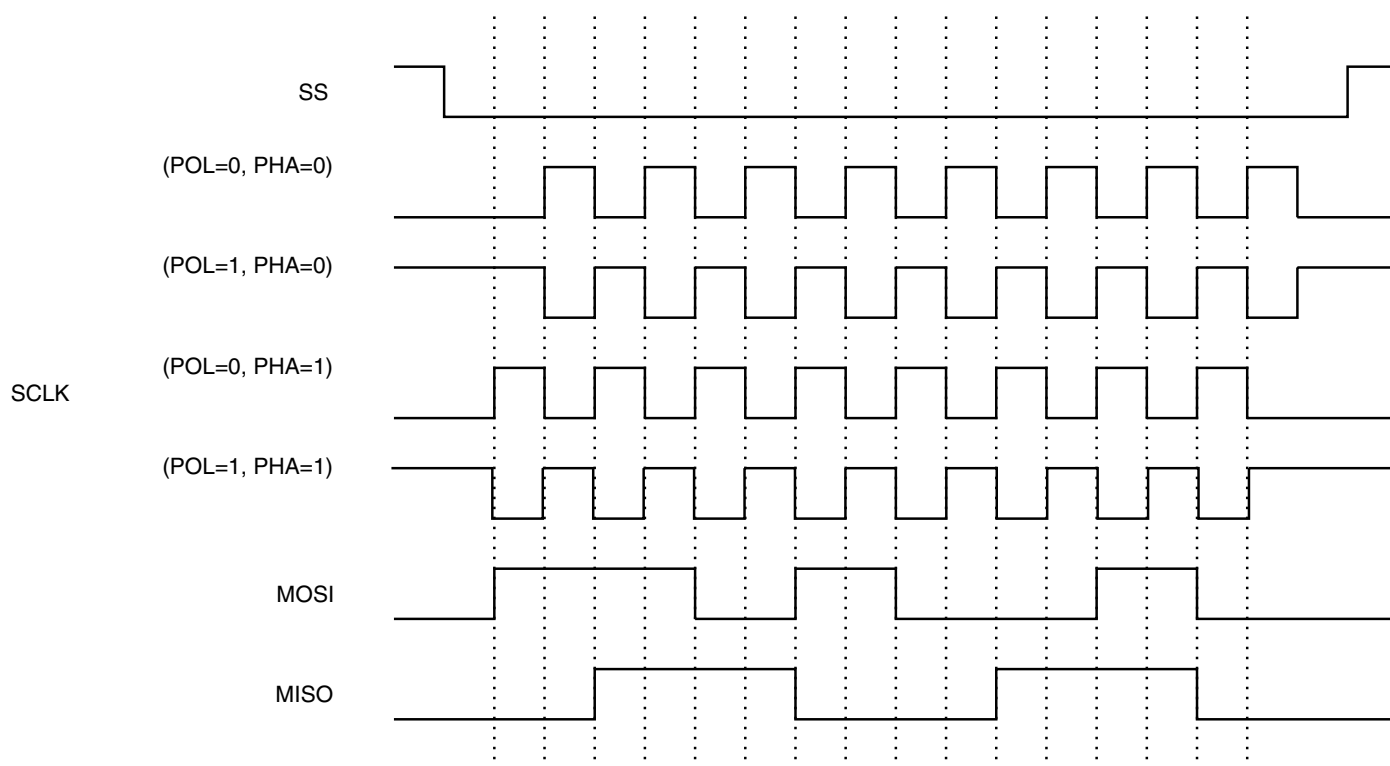


Figure 10-9. SPI Burst with Different POL and PHA Configurations

10.1.3.4.2 Typical Slave Mode

When the ECSPI is configured as a slave (Mode = 0), software can configure the ECSPI Control register to match the external SPI master's timing. In this configuration, SS becomes an input signal, and is used to latch data in and out of the internal data Shift registers, as well as to advance the data FIFO.

The SS, SCLK, and MOSI are inputs and MISO is output. Most of the timing diagrams are similar to the diagrams shown previously for the SPI in Master mode (Mode = 1), because the inputs come from a SPI master device.

However, the timing is different when SS is used to advance the data FIFO. When the SS_POL=0 is set while the ECSPI is configured in Slave mode, the data FIFO will advance on the rising edge of the SS signal. When the polarity is reversed (SS_POL = 1), the data FIFO will advance on the falling edge of the SS signal.

The figure below shows a SPI burst in which the data FIFO is advanced by the rising edge of the SS signal.

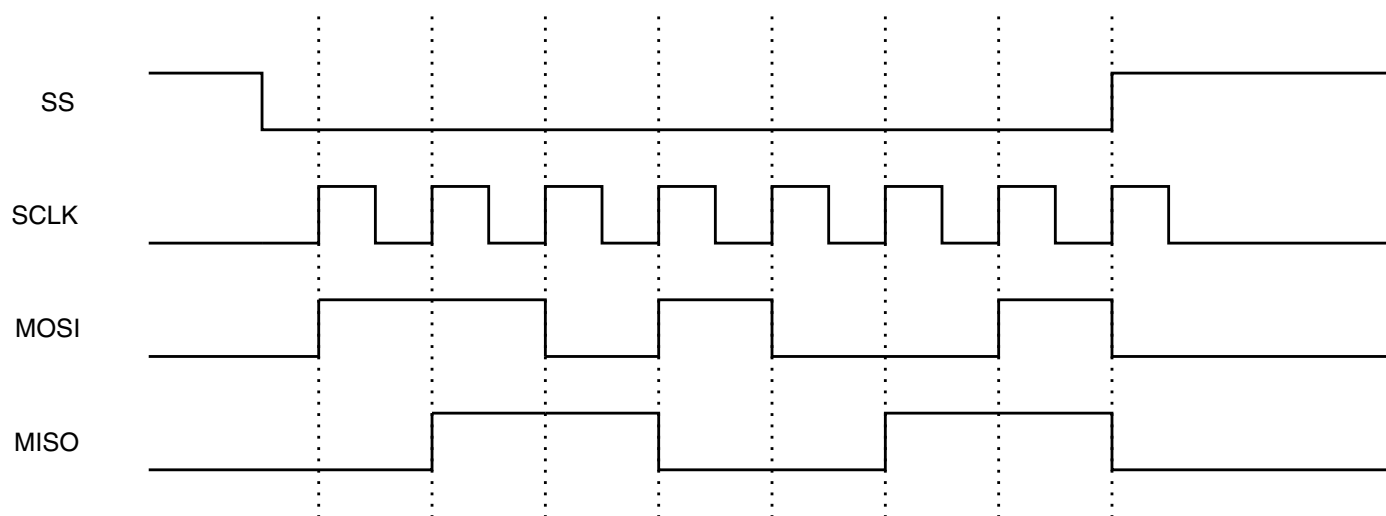


Figure 10-10. Advancing the Data FIFO on the Rising Edge of SS

In the above case, only the most significant 7 bits are loaded to the RXFIFO.

10.1.3.5 Reset

Whenever a device reset occurs, a reset is performed on the ECSPI, resetting all registers to their default values.

Software can reset the block using the CONREG[EN] bit; see [ECSPI](#).

10.1.3.6 Interrupts

Interrupt control provides a way to manage the ECSPI FIFOs:

- For transmitting data, software can enable the TXFIFO empty, TXFIFO data request, and TXFIFO full interrupts to maintain the TXFIFO using an interrupt service routine.
- For receiving data, software can enable the RXFIFO ready, RXFIFO data request, and RXFIFO full interrupts to retrieve data from the RXFIFO using an interrupt service routine.

Other interrupt sources can be used to control or debug the SPI bursts:

- The transfer-completed interrupt means that there is no data left in the TXFIFO and that the data in the Shift register has been shifted out.
- The RXFIFO overflow interrupt means that the RXFIFO received more than 64 words and will not accept any other words.

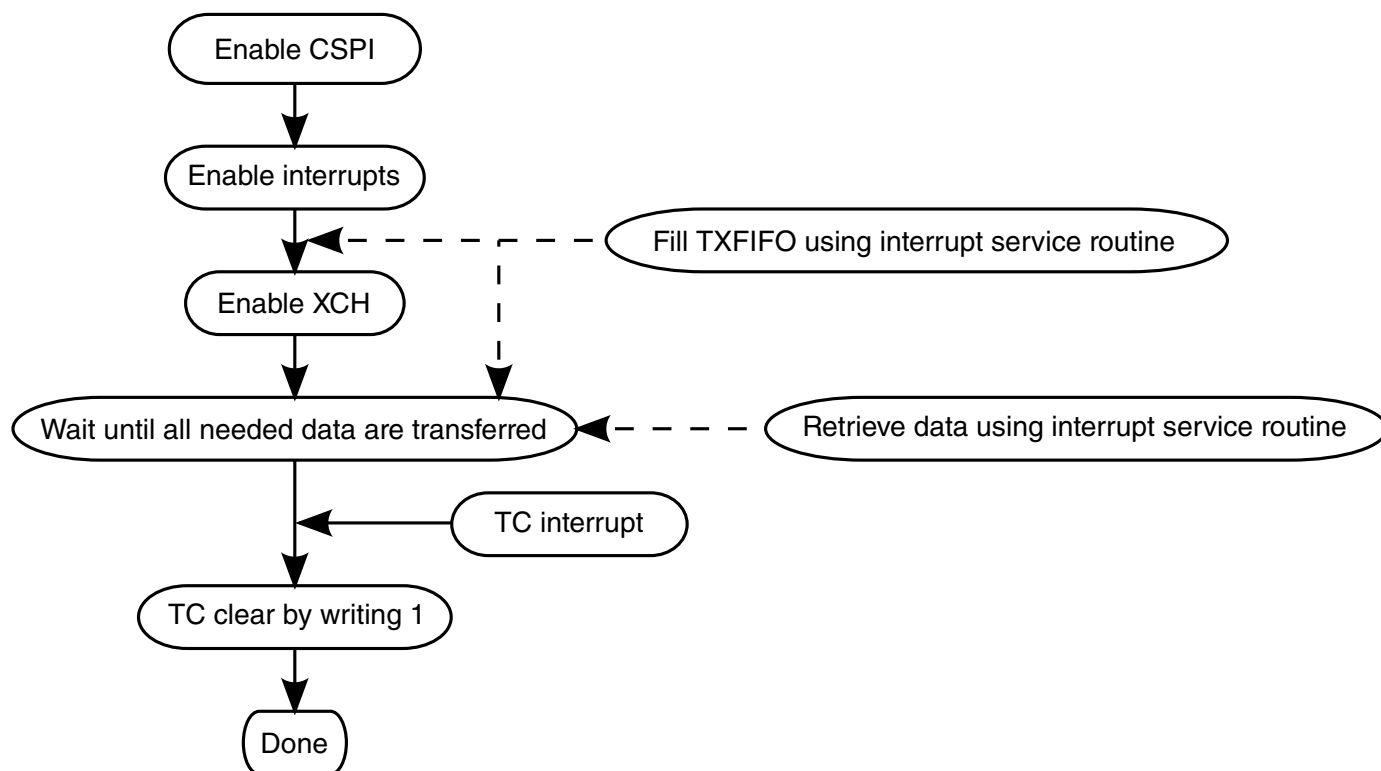


Figure 10-11. Program Sequence of SPI Burst Using Interrupt Control

NOTE

The TC bit does not provide a reliable indication that the transfer is complete. Under some conditions, the TC interrupt can occur before the transfer is completed. If the TC bit is used as an interrupt source, the XCH bit should be polled after the TC interrupt occurs to accurately confirm that the transfer is complete.

10.1.3.7 DMA

DMA control provides another method to utilize the FIFOs in the ECSPI. By using DMA request and acknowledge signals, larger amounts of data can be transferred, and will reduce interrupts and host processor loading. When the appropriate conditions are matched, the block will send out a DMA request.

The DMA can deal with the following conditions:

- TXFIFO empty
- TXFIFO data request
- RXFIFO data request
- RXFIFO full

The figure below shows a program sequence of SPI bursts using DMA control.

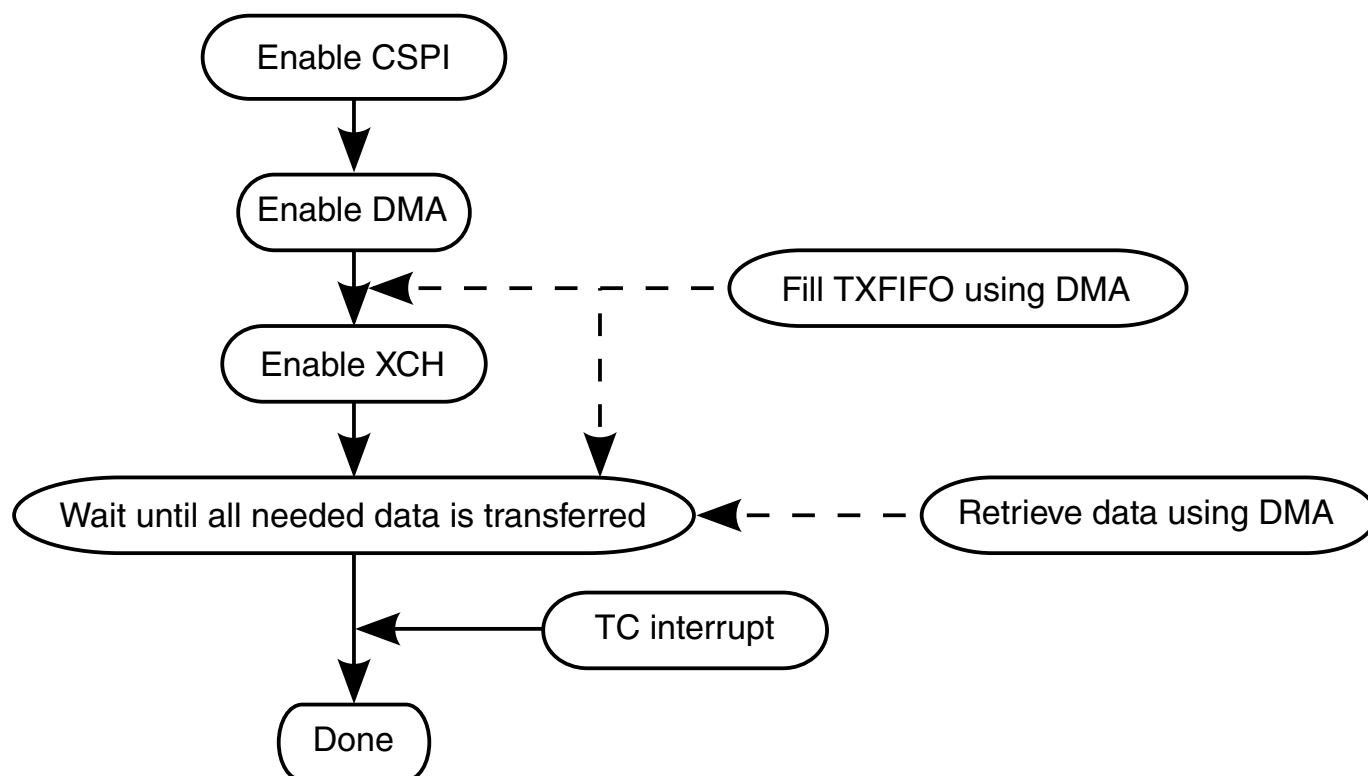


Figure 10-12. Program Sequence of SPI Burst Using DMA

NOTE

The TC bit does not provide a reliable indication that the transfer is complete. Under some conditions, the TC interrupt can occur before the transfer is completed. If the TC bit is used as an interrupt source, the XCH bit should be polled after the TC interrupt occurs to accurately confirm that the transfer is complete.

10.1.3.8 Byte Order

The ECSPI does not support byte re-ordering in hardware.

10.1.4 Initialization

This section provides initialization information for ECSPI.

To initialize the block:

1. Clear the EN bit in ECSPI_CONREG to reset the block.

2. Enable the clocks for ECSPI within the CCM.
3. Configure the Control Register and then set the EN bit in the ECSPI_CONREG to put ECSPI out of reset.
4. Configure corresponding IOMUX for ECSPI external signals.
5. Configure registers of ECSPI properly according to the specifications of the external SPI device.

10.1.5 Applications

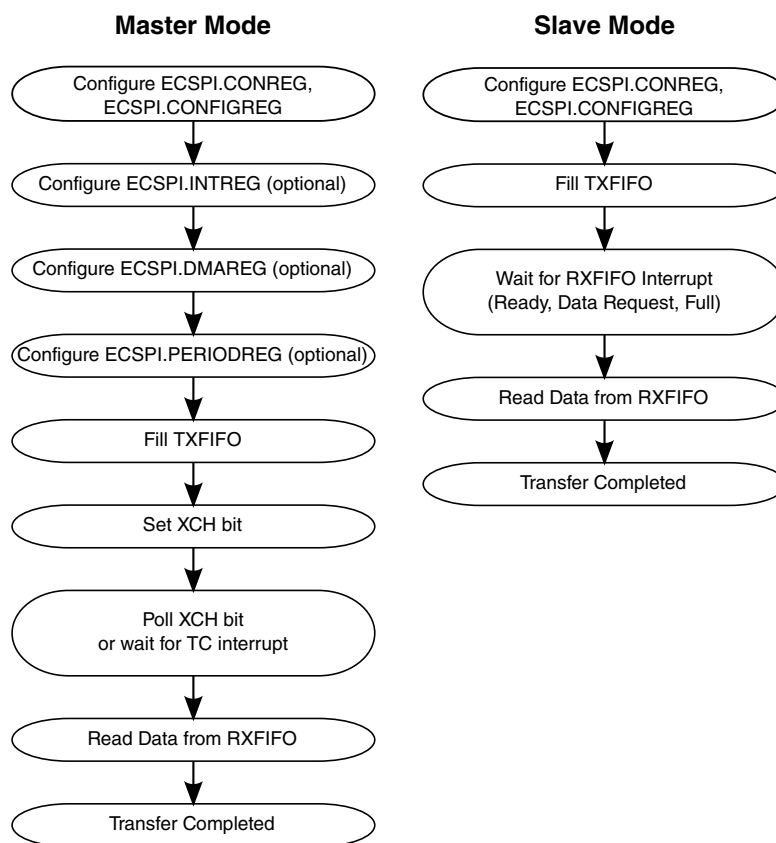


Figure 10-13. Flowchart of the ECSPI Operation

NOTE

The TC bit does not provide a reliable indication that the transfer is complete. Under some conditions, the TC interrupt can occur before the transfer is completed. If the TC bit is used as an interrupt source, the XCH bit should be polled after the TC interrupt occurs to accurately confirm that the transfer is complete.

10.1.6 ECSPI Memory Map/Register Definition

This section includes the block memory map and detailed descriptions of all registers. For the base address of a particular block instantiation, see the system memory map.

ECSPI memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3082_0000	Receive Data Register (ECSPI1_RXDATA)	32	R	0000_0000h	10.1.6.1/2551
3082_0004	Transmit Data Register (ECSPI1_TXDATA)	32	W	0000_0000h	10.1.6.2/2552
3082_0008	Control Register (ECSPI1_CONREG)	32	R/W	0000_0000h	10.1.6.3/2553
3082_000C	Config Register (ECSPI1_CONFIGREG)	32	R/W	0000_0000h	10.1.6.4/2555
3082_0010	Interrupt Control Register (ECSPI1_INTREG)	32	R/W	0000_0000h	10.1.6.5/2557
3082_0014	DMA Control Register (ECSPI1_DMAREG)	32	R/W	0000_0000h	10.1.6.6/2558
3082_0018	Status Register (ECSPI1_STATREG)	32	R/W	0000_0003h	10.1.6.7/2560
3082_001C	Sample Period Control Register (ECSPI1_PERIODREG)	32	R/W	0000_0000h	10.1.6.8/2561
3082_0020	Test Control Register (ECSPI1_TESTREG)	32	R/W	0000_0000h	10.1.6.9/2563
3082_0040	Message Data Register (ECSPI1_MSGDATA)	32	W	0000_0000h	10.1.6.10/2564
3083_0000	Receive Data Register (ECSPI2_RXDATA)	32	R	0000_0000h	10.1.6.1/2551
3083_0004	Transmit Data Register (ECSPI2_TXDATA)	32	W	0000_0000h	10.1.6.2/2552
3083_0008	Control Register (ECSPI2_CONREG)	32	R/W	0000_0000h	10.1.6.3/2553
3083_000C	Config Register (ECSPI2_CONFIGREG)	32	R/W	0000_0000h	10.1.6.4/2555
3083_0010	Interrupt Control Register (ECSPI2_INTREG)	32	R/W	0000_0000h	10.1.6.5/2557
3083_0014	DMA Control Register (ECSPI2_DMAREG)	32	R/W	0000_0000h	10.1.6.6/2558
3083_0018	Status Register (ECSPI2_STATREG)	32	R/W	0000_0003h	10.1.6.7/2560

Table continues on the next page...

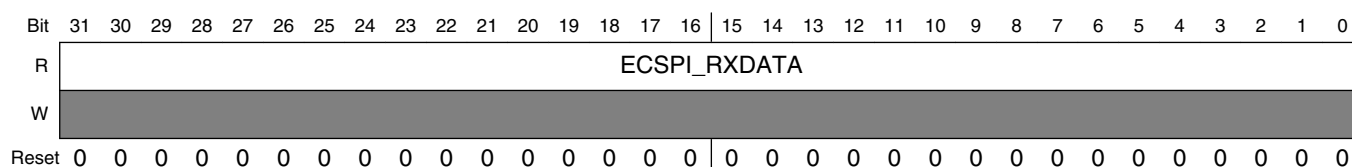
ECSPI memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
3083_001C	Sample Period Control Register (ECSPI2_PERIODREG)	32	R/W	0000_0000h	10.1.6.8/2561
3083_0020	Test Control Register (ECSPI2_TESTREG)	32	R/W	0000_0000h	10.1.6.9/2563
3083_0040	Message Data Register (ECSPI2_MSGDATA)	32	W	0000_0000h	10.1.6.10/2564
3084_0000	Receive Data Register (ECSPI3_RXDATA)	32	R	0000_0000h	10.1.6.1/2551
3084_0004	Transmit Data Register (ECSPI3_TXDATA)	32	W	0000_0000h	10.1.6.2/2552
3084_0008	Control Register (ECSPI3_CONREG)	32	R/W	0000_0000h	10.1.6.3/2553
3084_000C	Config Register (ECSPI3_CONFIGREG)	32	R/W	0000_0000h	10.1.6.4/2555
3084_0010	Interrupt Control Register (ECSPI3_INTREG)	32	R/W	0000_0000h	10.1.6.5/2557
3084_0014	DMA Control Register (ECSPI3_DMAREG)	32	R/W	0000_0000h	10.1.6.6/2558
3084_0018	Status Register (ECSPI3_STATREG)	32	R/W	0000_0003h	10.1.6.7/2560
3084_001C	Sample Period Control Register (ECSPI3_PERIODREG)	32	R/W	0000_0000h	10.1.6.8/2561
3084_0020	Test Control Register (ECSPI3_TESTREG)	32	R/W	0000_0000h	10.1.6.9/2563
3084_0040	Message Data Register (ECSPI3_MSGDATA)	32	W	0000_0000h	10.1.6.10/2564

10.1.6.1 Receive Data Register (ECSPIx_RXDATA)

The Receive Data register (ECSPI_RXDATA) is a read-only register that forms the top word of the 64 x 32 receive FIFO. This register holds the data received from an external SPI device during a data transaction. Only word-sized read operations are allowed.

Address: Base address + 0h offset



ECSPIx_RXDATA field descriptions

Field	Description
ECSPI_RXDATA	Receive Data. This register holds the top word of the receive data FIFO. The FIFO is advanced for each read of this register. The data read is undefined when the Receive Data Ready (RR) bit in the Interrupt Control/Status register is cleared. Zeros are read when ECSPI is disabled.

10.1.6.2 Transmit Data Register (ECSPIx_TXDATA)

The Transmit Data (ECSPI_TXDATA) register is a write-only data register that forms the bottom word of the 64 x 32 TXFIFO. The TXFIFO can be written to as long as it is not full, even when the SPI Exchange bit (XCH) in ECSPI_CONREG is set. This allows software to write to the TXFIFO during a SPI data exchange process. Writes to this register are ignored when the ECSPI is disabled (ECSPI_CONREG[EN] bit is cleared).

Address: Base address + 4h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W	ECSPI_TXDATA																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

ECSPIx_TXDATA field descriptions

Field	Description
ECSPI_TXDATA	Transmit Data. This register holds the top word of data loaded into the FIFO. Data written to this register must be a word operation. The number of bits actually transmitted is determined by the BURST_LENGTH field of the corresponding SPI Control register. If this field contains more bits than the number specified by BURST_LENGTH, the extra bits are ignored. For example, to transfer 10 bits of data, a 32-bit word must be written to this register. Bits 9-0 are shifted out and bits 31-10 are ignored. When the ECSPI is operating in Slave mode, zeros are shifted out when the FIFO is empty. Zeros are read when ECSPI is disabled.

10.1.6.3 Control Register (ECSPIx_CONREG)

The Control Register (ECSPI_CONREG) allows software to enable the ECSPI, configure its operating modes, specify the divider value, and SPI_RDY control signal, and define the transfer length.

Address: Base address + 8h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	BURST_LENGTH												CHANNEL_SELECT		DRCTL	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PRE_DIVIDER				POST_DIVIDER				CHANNEL_MODE				SMC	XCH	HT	EN
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

ECSPIx_CONREG field descriptions

Field	Description
31–20 BURST_LENGTH	Burst Length. This field defines the length of a SPI burst to be transferred. The Chip Select (SS) will remain asserted until all bits in a SPI burst are shifted out. A maximum of 2^{12} bits can be transferred in a single SPI burst. In master mode, it controls the number of bits per SPI burst. Since the shift register always loads 32-bit data from transmit FIFO, only the n least-significant ($n = \text{BURST_LENGTH} + 1$) will be shifted out. The remaining bits will be ignored. Number of Valid Bits in a SPI burst. 0x000 A SPI burst contains the 1 LSB in a word. 0x001 A SPI burst contains the 2 LSB in a word. 0x002 A SPI burst contains the 3 LSB in a word. ... 0x01F A SPI burst contains all 32 bits in a word. 0x020 A SPI burst contains the 1 LSB in first word and all 32 bits in second word. 0x021 A SPI burst contains the 2 LSB in first word and all 32 bits in second word. ... 0xFFE A SPI burst contains the 31 LSB in first word and $2^7 - 1$ words. 0xFFF A SPI burst contains 2^7 words.
19–18 CHANNEL_SELECT	SPI CHANNEL SELECT bits. Selects the external SPI Master/Slave Device. In master mode, these two bits select the external slave device by asserting the Chip Select (SS) output. 00 Channel 0 is selected. Chip Select (SS) will be asserted. 01 Reserved. 10 Reserved. 11 Reserved.
17–16 DRCTL	SPI Data Ready Control. This field selects the utilization of the SPI_RDY signal in master mode. ECSPI checks this field before it starts an SPI burst.

Table continues on the next page...

ECSPiX_CONREG field descriptions (continued)

Field	Description
	00 The SPI_RDY signal is a don't care. 01 Burst will be triggered by the falling edge of the SPI_RDY signal (edge-triggered). 10 Burst will be triggered by a low level of the SPI_RDY signal (level-triggered). 11 Reserved.
15–12 PRE_DIVIDER	SPI Pre Divider. ECSPI uses a two-stage divider to generate the SPI clock. This field defines the pre-divider of the reference clock. 0000 Divide by 1. 0001 Divide by 2. 0010 Divide by 3. ... 1101 Divide by 14. 1110 Divide by 15. 1111 Divide by 16.
11–8 POST_DIVIDER	SPI Post Divider. ECSPI uses a two-stage divider to generate the SPI clock. This field defines the post-divider of the reference clock using the equation: 2^n . 0000 Divide by 1. 0001 Divide by 2. 0010 Divide by 4. ... 1110 Divide by 2^{14} . 1111 Divide by 2^{15} .
7–4 CHANNEL_MODE	SPI CHANNEL MODE selects the mode for each SPI channel. CHANNEL MODE[3] is for SPI channel 3. CHANNEL MODE[2] is for SPI channel 2. CHANNEL MODE[1] is for SPI channel 1. CHANNEL MODE[0] is for SPI channel 0. 0 Slave mode. 1 Master mode.
3 SMC	Start Mode Control. This bit applies only to channels configured in Master mode (CHANNEL MODE = 1). It controls how the ECSPI starts a SPI burst, either through the SPI exchange bit, or immediately when the TXFIFO is written to. 0 SPI Exchange Bit (XCH) controls when a SPI burst can start. Setting the XCH bit will start a SPI burst or multiple bursts. This is controlled by the SPI SS Wave Form Select (SS_CTL). Refer to XCH and SS_CTL descriptions. 1 Immediately starts a SPI burst when data is written in TXFIFO.
2 XCH	SPI Exchange Bit. This bit applies only to channels configured in Master mode (CHANNEL MODE = 1). If the Start Mode Control (SMC) bit is cleared, writing a 1 to this bit starts one SPI burst or multiple SPI bursts according to the SPI SS Wave Form Select (SS_CTL). The XCH bit remains set while either the data exchange is in progress, or when the ECSPI is waiting for an active input if SPIRDY is enabled through DRCTL. This bit is cleared automatically when all data in the TXFIFO and the shift register has been shifted out. 0 Idle. 1 Initiates exchange (write) or busy (read).

Table continues on the next page...

ECSPiX_CONREG field descriptions (continued)

Field	Description
1 HT	Hardware Trigger Enable. This bit is used in master mode only. It enables hardware trigger (HT) mode. Note, HT mode is not supported by this product. 0 Disable HT mode. 1 Enable HT mode.
0 EN	SPI Block Enable Control. This bit enables the ECSPiX. This bit must be set before writing to other registers or initiating an exchange. Writing zero to this bit disables the block and resets the internal logic with the exception of the ECSPiX_CONREG. The block's internal clocks are gated off whenever the block is disabled. 0 Disable the block. 1 Enable the block.

10.1.6.4 Config Register (ECSPiX_CONFIGREG)

The Config Register (ECSPiX_CONFIGREG) allows software to configure each SPI channel, configure its operating modes, specify the phase and polarity of the clock, configure the Chip Select (SS), and define the HT transfer length. Note, HT mode is not supported by this product.

Address: Base address + Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
R	Reserved								HT_LENGTH				SCLK_CTL				DATA_CTL				SS_POL				SS_CTL				SCLK_POL				SCLK_PHA			
W	Reserved								HT_LENGTH				SCLK_CTL				DATA_CTL				SS_POL				SS_CTL				SCLK_POL				SCLK_PHA			
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				

ECSPiX_CONFIGREG field descriptions

Field	Description
31–29 -	This field is reserved. Reserved
28–24 HT_LENGTH	HT LENGTH. This field defines the message length in HT Mode. Note, HT mode is not supported by this product. The length in bits of one message is (HT LENGTH + 1).
23–20 SCLK_CTL	SCLK CTL. This field controls the inactive state of SCLK for each SPI channel. SCLK CTL[3] is for SPI channel 3. SCLK CTL[2] is for SPI channel 2. SCLK CTL[1] is for SPI channel 1. SCLK CTL[0] is for SPI channel 0. 0 Stay low. 1 Stay high.

Table continues on the next page...

ECSPIx_CONFIGREG field descriptions (continued)

Field	Description
19–16 DATA_CTL	DATA CTL. This field controls inactive state of the data line for each SPI channel. DATA CTL[3] is for SPI channel 3. DATA CTL[2] is for SPI channel 2. DATA CTL[1] is for SPI channel 1. DATA CTL[0] is for SPI channel 0. 0 Stay high. 1 Stay low.
15–12 SS_POL	SPI SS Polarity Select. In both Master and Slave modes, this field selects the polarity of the Chip Select (SS) signal. SS POL[3] is reserved. SS POL[2] is reserved. SS POL[1] is reserved. SS POL[0] is for SPI channel 0. 0 Active low. 1 Active high.
11–8 SS_CTL	SPI SS Wave Form Select. In master mode, this field controls the output wave form of the Chip Select (SS) signal when the SMC (Start Mode Control) bit is cleared. The SS_CTL bits are ignored if the SMC bit is set. SS CTL[3] is reserved. SS CTL[2] is reserved. SS CTL[1] is reserved. SS CTL[0] is for SPI channel 0. In slave mode, this bit controls when the SPI burst is completed. An SPI burst is completed by the Chip Select (SS) signal edges. (SSPOL = 0: rising edge; SSPOL = 1: falling edge) The RXFIFO is advanced whenever a Chip Select (SS) signal edge is detected or the shift register contains 32-bits of valid data. 0 In master mode - only one SPI burst will be transmitted. 1 In master mode - Negate Chip Select (SS) signal between SPI bursts. Multiple SPI bursts will be transmitted. The SPI transfer will automatically stop when the TXFIFO is empty. 0 In slave mode - an SPI burst is completed when the number of bits received in the shift register is equal to (BURST LENGTH + 1). Only the n least-significant bits (n = BURST LENGTH[4:0] + 1) of the first received word are valid. All bits subsequent to the first received word in RXFIFO are valid. 1 Reserved
7–4 SCLK_POL	SPI Clock Polarity Control. This field controls the polarity of the SCLK signal. See Figure 10-9 for more information. SCLK_POL[3] is for SPI channel 3. SCLK_POL[2] is for SPI channel 2. SCLK_POL[1] is for SPI channel 1. SCLK_POL[0] is for SPI channel 0.

Table continues on the next page...

ECSPiX_CONFIGREG field descriptions (continued)

Field	Description
	0 Active high polarity (0 = Idle). 1 Active low polarity (1 = Idle).
SCLK_PHA	SPI Clock/Data Phase Control. This field controls the clock/data phase relationship. See Figure 10-9 for more information. SCLK PHA[3] is for SPI channel 3. SCLK PHA[2] is for SPI channel 2. SCLK PHA[1] is for SPI channel 1. SCLK PHA[0] is for SPI channel 0. 0 Phase 0 operation. 1 Phase 1 operation.

10.1.6.5 Interrupt Control Register (ECSPiX_INTREG)

The Interrupt Control Register (ECSPiX_INTREG) enables the generation of interrupts to the host processor. If the ECSPi is disabled, this register reads zero.

Address: Base address + 10h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								TCEN	ROEN	RFEN	RDREN	RREN	TFEN	TDREN	TEEN
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

ECSPiX_INTREG field descriptions

Field	Description
31–8 -	This field is reserved. Reserved
7 TCEN	Transfer Completed Interrupt enable. This bit enables the Transfer Completed Interrupt. 0 Disable 1 Enable

Table continues on the next page...

ECSPIx_INTREG field descriptions (continued)

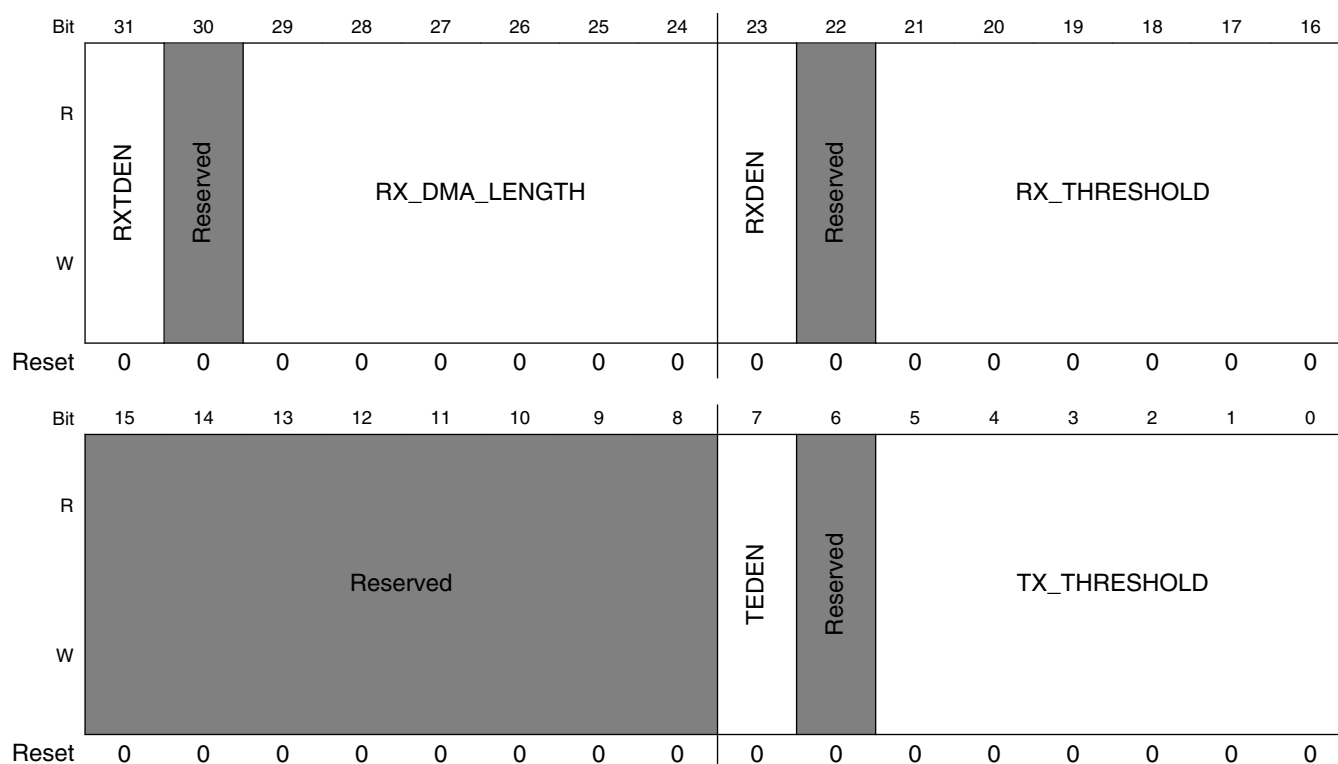
Field	Description
6 ROEN	RXFIFO Overflow Interrupt enable. This bit enables the RXFIFO Overflow Interrupt. 0 Disable 1 Enable
5 RFEN	RXFIFO Full Interrupt enable. This bit enables the RXFIFO Full Interrupt. 0 Disable 1 Enable
4 RDREN	RXFIFO Data Request Interrupt enable. This bit enables the RXFIFO Data Request Interrupt when the number of data entries in the RXFIFO is greater than RX_THRESHOLD. 0 Disable 1 Enable
3 RREN	RXFIFO Ready Interrupt enable. This bit enables the RXFIFO Ready Interrupt. 0 Disable 1 Enable
2 TFEN	TXFIFO Full Interrupt enable. This bit enables the TXFIFO Full Interrupt. 0 Disable 1 Enable
1 TDREN	TXFIFO Data Request Interrupt enable. This bit enables the TXFIFO Data Request Interrupt when the number of data entries in the TXFIFO is less than or equal to TX_THRESHOLD. 0 Disable 1 Enable
0 TEEN	TXFIFO Empty Interrupt enable. This bit enables the TXFIFO Empty Interrupt. 0 Disable 1 Enable

10.1.6.6 DMA Control Register (ECSPIx_DMAREG)

The Direct Memory Access Control Register (ECSPI_DMAREG) provides software a way to use an on-chip DMA controller for ECSPI data. Internal DMA request signals enable direct data transfers between the ECSPI FIFOs and system memory. The ECSPI sends out DMA requests when the appropriate FIFO conditions are matched.

If the ECSPI is disabled, this register is read as 0.

Address: Base address + 14h offset

**ECSPiX_DMAREG field descriptions**

Field	Description
31 RXTDEN	RXFIFO TAIL DMA Request/Interrupt Enable. This bit enables an internal counter that is increased at each read of the RXFIFO. This counter is cleared automatically when it reaches RX DMA LENGTH. If the number of words remaining in the RXFIFO is greater than or equal to RX DMA LENGTH, a DMA request/interrupt is generated even if it is less than or equal to RX_THRESHOLD. 0 Disable 1 Enable
30 -	This field is reserved. Reserved
29–24 RX_DMA_LENGTH	RX DMA LENGTH. This field defines the burst length of a DMA operation. Applies only when RXTDEN is set.
23 RXDEN	RXFIFO DMA Request Enable. This bit enables/disables the RXFIFO DMA Request. 0 Disable 1 Enable
22 -	This field is reserved. Reserved
21–16 RX_THRESHOLD	RX THRESHOLD. This field defines the FIFO threshold that triggers a RX DMA/INT request. A RX DMA/INT request is issued when the number of data entries in the RXFIFO is greater than RX_THRESHOLD.
15–8 -	This field is reserved. Reserved

Table continues on the next page...

ECSPIx_DMAREG field descriptions (continued)

Field	Description
7 TEDEN	TXFIFO Empty DMA Request Enable. This bit enables/disables the TXFIFO Empty DMA Request. 0 Disable 1 Enable
6 -	This field is reserved. Reserved
TX_ THRESHOLD	TX THRESHOLD. This field defines the FIFO threshold that triggers a TX DMA/INT request. A TX DMA/INT request is issued when the number of data entries in the TXFIFO is not greater than TX_THRESHOLD.

10.1.6.7 Status Register (ECSPIx_STATREG)

The ECSPI Status Register (ECSPI_STATREG) reflects the status of the ECSPI's operating condition. If the ECSPI is disabled, this register reads 0x0000_0003.

Address: Base address + 18h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								TC	RO	RF	RDR	RR	TF	TDR	TE
W									w1c	w1c						
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1

ECSPIx_STATREG field descriptions

Field	Description
31–8 -	This field is reserved. Reserved
7 TC	Transfer Completed Status bit. Writing 1 to this bit clears it. NOTE: The TC bit does not provide a reliable indication that the transfer is complete. Under some conditions, the TC interrupt can occur before the transfer is completed. If the TC bit is used as an interrupt source, the XCH bit should be polled after the TC interrupt occurs to accurately confirm that the transfer is complete. 0 Transfer in progress. 1 Transfer completed.
6 RO	RXFIFO Overflow. When set, this bit indicates that RXFIFO has overflowed. Writing 1 to this bit clears it.

Table continues on the next page...

ECSPiX_STATREG field descriptions (continued)

Field	Description
	0 RXFIFO has no overflow. 1 RXFIFO has overflowed.
5 RF	RXFIFO Full. This bit is set when the RXFIFO is full. 0 Not Full. 1 Full.
4 RDR	RXFIFO Data Request. 0 When RXTDE is set - Number of data entries in the RXFIFO is not greater than RX_THRESHOLD. 1 When RXTDE is set - Number of data entries in the RXFIFO is greater than RX_THRESHOLD or a DMA TAIL DMA condition exists. 0 When RXTDE is clear - Number of data entries in the RXFIFO is not greater than RX_THRESHOLD. 1 When RXTDE is clear - Number of data entries in the RXFIFO is greater than RX_THRESHOLD.
3 RR	RXFIFO Ready. This bit is set when one or more words are stored in the RXFIFO. 0 No valid data in RXFIFO. 1 More than 1 word in RXFIFO.
2 TF	TXFIFO Full. This bit is set when if the TXFIFO is full. 0 TXFIFO is not Full. 1 TXFIFO is Full.
1 TDR	TXFIFO Data Request. 0 Number of valid data slots in TXFIFO is greater than TX_THRESHOLD. 1 Number of valid data slots in TXFIFO is not greater than TX_THRESHOLD.
0 TE	TXFIFO Empty. This bit is set if the TXFIFO is empty. 0 TXFIFO contains one or more words. 1 TXFIFO is empty.

10.1.6.8 Sample Period Control Register (ECSPiX_PERIODREG)

The Sample Period Control Register (ECSPiX_PERIODREG) provides software a way to insert delays (wait states) between consecutive SPI transfers. Control bits in this register select the clock source for the sample period counter and the delay count indicating the number of wait states to be inserted between data transfers.

The delay counts apply only when the current channel is operating in Master mode (ECSPiX_CONREG[CHANNEL MODE] = 1). ECSPiX_PERIODREG also contains the CSD CTRL field used to insert a delay between the Chip Select's active edge and the first SPI Clock edge.

Enhanced Configurable SPI (ECSPI)

Address: Base address + 1Ch offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved										CSD_CTL					
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	CSRC	SAMPLE_PERIOD														
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

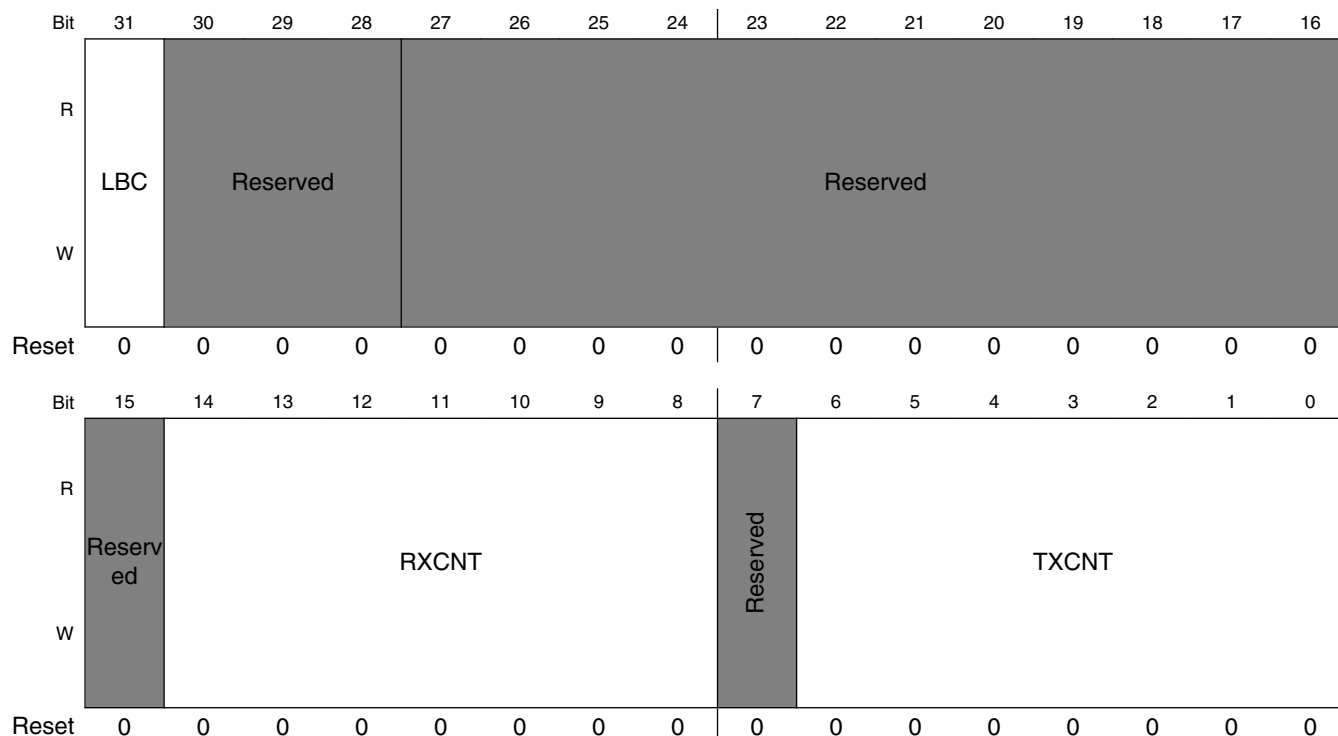
ECSPiX_PERIODREG field descriptions

Field	Description
31–22 -	This field is reserved. Reserved
21–16 CSD_CTL	Chip Select Delay Control bits. This field defines how many SPI clocks will be inserted between the chip select's active edge and the first SPI clock edge. The range is from 0 to 63.
15 CSRC	Clock Source Control. This bit selects the clock source for the sample period counter. 0 SPI Clock (SCLK) 1 Low-Frequency Reference Clock (32.768 KHz)
SAMPLE_PERIOD	Sample Period Control. These bits control the number of wait states to be inserted in data transfers. During the idle clocks, the state of the SS output will operate according to the SS_CTL control field in the ECSPi_CONREG register. 0x0000 0 wait states inserted 0x0001 1 wait state inserted 0x7FFE 32766 wait states inserted 0x7FFF 32767 wait states inserted

10.1.6.9 Test Control Register (ECSPIx_TESTREG)

The Test Control Register (ECSPI_TESTREG) provides software a mechanism to internally connect the receive and transmit devices of the ECSPI, and monitor the contents of the receive and transmit FIFOs.

Address: Base address + 20h offset



ECSPIx_TESTREG field descriptions

Field	Description
31 LBC	Loop Back Control. This bit is used in Master mode only. When this bit is set, the ECSPI connects the transmitter and receiver sections internally, and the data shifted out from the most-significant bit of the shift register is looped back into the least-significant bit of the Shift register. In this way, a self-test of the complete transmit/receive path can be made. The output pins continue to operate normally, however the input pins are ignored. 0 Not connected. 1 Transmitter and receiver sections internally connected for Loopback.
30–28 -	This field is reserved. Reserved, all bits should be ignored.
27–15 -	This field is reserved. Reserved
14–8 RXCNT	RXFIFO Counter. This field indicates the number of words in the RXFIFO.

Table continues on the next page...

ECSPiX_TESTREG field descriptions (continued)

Field	Description
7 -	This field is reserved. Reserved
TXCNT	TXFIFO Counter. This field indicates the number of words in the TXFIFO.

10.1.6.10 Message Data Register (ECSPiX_MSGDATA)

The Message Data Register (ECSPiX_MSGDATA) forms the top word of the 16 x 32 MSG Data FIFO. Only word-size accesses are allowed for this register. Reading from this register returns zero, and writing to this register stores data in the MSG Data FIFO.

Address: Base address + 40h offset

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W	ECSPI_MSGDATA																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

ECSPiX_MSGDATA field descriptions

Field	Description
ECSPiX_MSGDATA	ECSPiX_MSGDATA holds the top word of MSG Data FIFO. The MSG Data FIFO is advanced for each write of this register. The data read is zero. The data written to this register is stored in the MSG Data FIFO.

10.2 Quad Serial Peripheral Interface (QuadSPI)**10.2.1 Overview**

The Quad Serial Peripheral Interface (QuadSPI) block acts as an interface to one or two external serial flash devices, each with up to four bidirectional data lines. The following figure is a block diagram of the QuadSPI module.

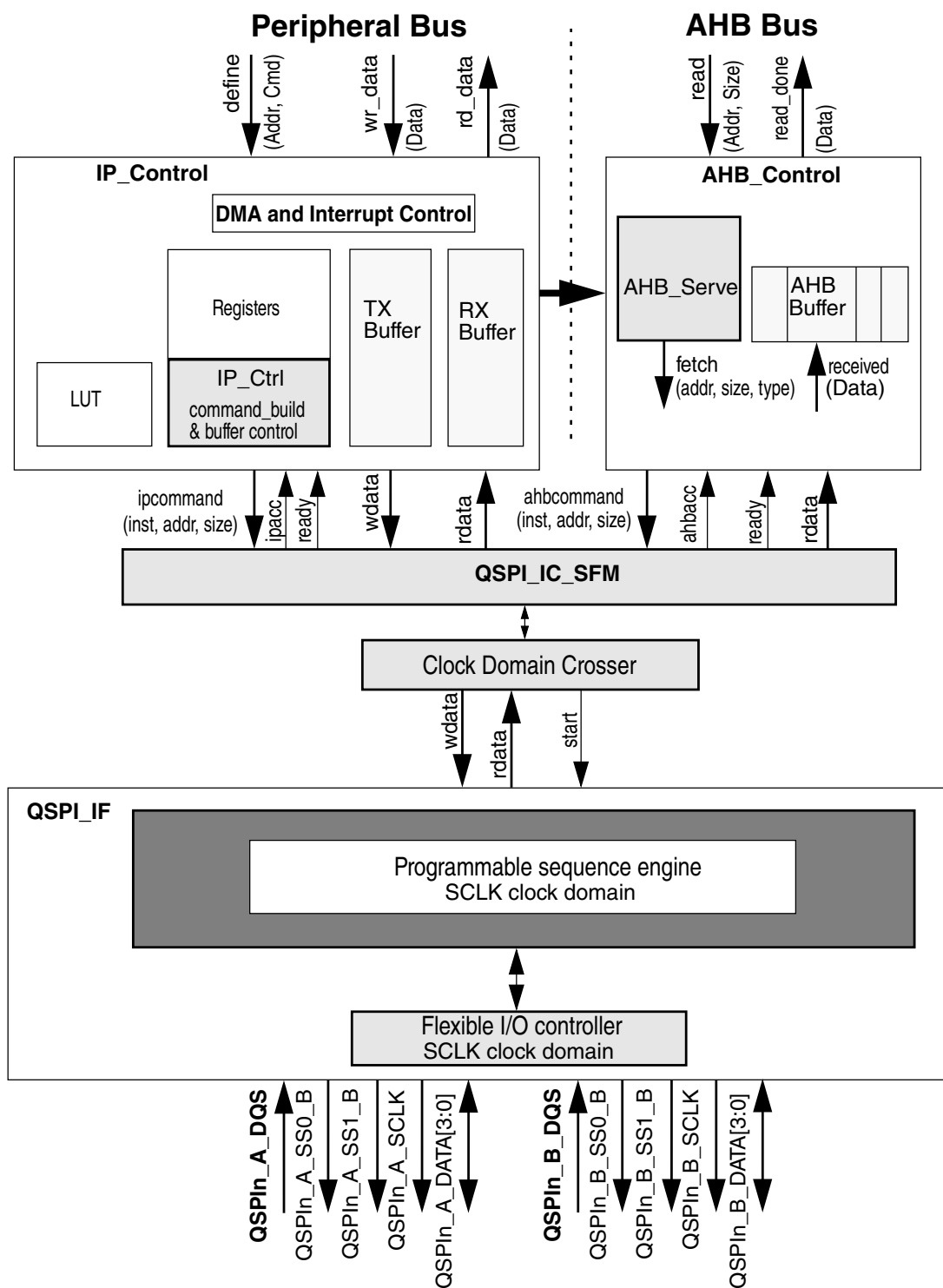


Figure 10-14. QuadSPI Block Diagram

The following figure describes the serial flash clock diagram in QuadSPI:

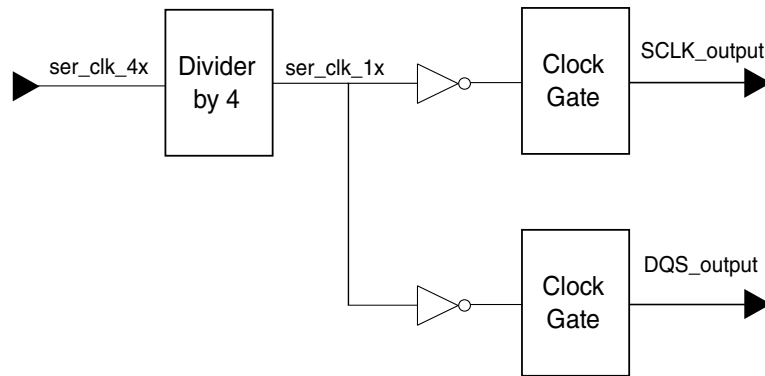


Figure 10-15. Flash Clock Diagram

ser_clk_4x is the serial clock root from CCM

ser_clk_1x' is generated clock from 'ser_clk_4x' divided by 4

SCLK_output is serial output clock generated from 'ser_clk_1x' with inverter and clock gate, it's used as Clock by external serial flash device.

DQS_output is serial flash data strobe signal generated from 'ser_clk_1x' with inverter and clock gate. This signal could be loopback from pad and used as sampling clock for serial flash data.

10.2.1.1 Features

The QuadSPI supports the following features:

- Flexible sequence engine to support various flash vendor devices.
- Single, dual, quad mode of operation.
- DDR/DTR mode wherein the data is generated on every edge of the serial flash clock.
- Support for flash data strobe signal for data sampling in DDR and SDR mode.
- Two identical serial flash devices can be connected and accessed in parallel for data read operations, forming one (virtual) flash memory with doubled readout bandwidth.
- DMA support to read RX Buffer data via AMBA AHB bus (64-bit width interface) or IP registers space (32-bit access).
 - Inner loop size of DMA access can be configured.
- Multi master accesses with priority

- Flexible and configurable buffer for each master
- Thirteen interrupt conditions (see [Table 10-10](#))
- Memory mapped read access to connected flash devices.
- Programmable sequence engine to cater to future command/protocol changes and able to support all existing vendor commands and operations.
 - Supports 3-byte and 4-byte addressing.
 - TXFIFO size is 512Byte
 - RXFIFO size is 512Byte
 - AHB BUF size is 1KByte

10.2.1.2 QuadSPI Modes of Operation

This section provides information about the modes in which the QuadSPI module can be used.

10.2.1.2.1 Normal Mode

In this mode, one or two external serial flash memory devices can be accessed. Further details about this mode of operation can be found in [Modes of Operation \(Normal Mode\)](#).

10.2.1.2.2 Module Disable Mode

This mode is used for power management of the device containing the QuadSPI module. It is controlled by signals external to the QuadSPI. The clock to the non-memory mapped logic in the QuadSPI can be stopped while in Module Disable Mode.

10.2.1.3 Acronyms and Abbreviations

The following table contains acronyms and abbreviations used in this document.

Table 10-2. Acronyms and Abbreviations

Terms	Description
AHB	Advanced High-performance Bus, version of AMBA
AMBA	Advanced Microcontroller Bus Architecture
BE	Big Endian Byte Ordering
CS	Chip Select
DMA	Direct Memory Access

Table continues on the next page...

Table 10-2. Acronyms and Abbreviations (continued)

Terms	Description
MB	Megabyte. Each MB is 1024 * 1024 bytes
IFM	Individual Flash Mode
PFM	Parallel Flash Mode
LSB	Least Significant Bit
MSB	Most Significant Bit
PCS	Peripheral Chip Select
QSPI, QuadSPI	Quad Serial Peripheral Interface
SCK	Serial Communications Clock
w1c	Write 1 to clear, writing a 1 to this field resets the flag

10.2.1.4 Glossary for QuadSPI module

Table 10-3. Glossary

Term	Definition
AHB Command	An AHB Command is a SFM Command triggered by a read access to the address range belonging to the memory mapped access defined in Table 10-7 . Refer to AHB Commands for details.
Asserted	A signal that is asserted is in its active state. An active low signal changes from logic level one to logic level zero when asserted, and an active high signal changes from logic level zero to logic level one.
Clear	To clear a bit or bits means to establish logic level zero on the bit or bits.
Clock Phase	Determines when the data should be sampled relative to the active edge of SCK
Clock Polarity	Determines the idle state of the SCK signal.
Drain	To remove entries from a FIFO by software or hardware.
Endianness	Byte Ordering scheme.
Field	Two or more register bits grouped together.
Fill	To add entries to a FIFO by software or hardware.
Host	Refers to another functional block in the device containing the QuadSPI module
Instruction Code	8 bits defining the type of command to be executed.
IP Command	An IP Command is a SFM Command triggered by writing into the QSPI_IPCR[SEQID] field.
Logic level one	The voltage that corresponds to Boolean true (1) state.
Logic level zero	The voltage that corresponds to Boolean false (0) state.
Negated	A signal that is negated is in its inactive state. An active low signal changes from logic level 0 to logic level 1 when negated, and an active high signal changes from logic level 1 to logic level 0.
QSPI_AMBA_BASE	First address of QuadSPI address space on system memory map.
QSPI_ARDB_BASE	First address of QuadSPI Rx Buffer on system memory map.
Set	To set a bit or bits means to establish logic level one on the bit or bits.

Table continues on the next page...

Table 10-3. Glossary (continued)

Term	Definition
RX Buffer PUSH Event	Addition of valid entries into the RX Buffer. In the default case each Buffer PUSH Event adds 2 entries to the RX Buffer since the interface to the serial clock domain is 64 bits in width. Depending on the number of bytes read from the serial flash device it is possible for the very last Buffer PUSH Event that only one entry is added. The QSPI_RBSR[RDBFL] field is incremented by the number of entries added to the RX Buffer.
RX Buffer POP Event	Removal of valid entries from the RX Buffer. Each Buffer POP Event removes (QSPI_RBCT[WMRK] + 1) valid entries from the buffer. The QSPI_RBSR[RDBFL] field is decremented by the same number and the QSPI_RBSR[RDCTR] field is incremented accordingly.
Individual Flash Mode	Access to a single, individual serial flash device. Refer to Serial Flash Access Schemes for details.
Parallel Flash Mode	Read access to two serial flash devices attached to the QuadSPI module in parallel. Refer to Serial Flash Access Schemes for details.
SFM Command	Serial Flash Memory Command. A SFM command consists of an instruction code and all other parameters (for example, size or mode bytes) needed for that specific instruction code. Triggering a command either initiates a transaction on the external serial flash or results in an error. Refer to Table 10-20 for details on errors.
Single/Dual/Quad Instructions	Depending on the serial flash device connected to the QuadSPI module there will be instructions using a different number of data lines. • Single: Single line I/O with one data out and one data in line to/from the serial flash device. • Dual: Dual line I/O with two bidirectional I/O lines, driven alternatively by the serial flash device or the QuadSPI module • Quad: Quad line I/O with 4 bidirectional I/O lines, driven alternatively by the serial flash device or the QuadSPI
Transaction	A transaction consists of all flags, data and signals in either direction to execute a command for an attached serial flash device. It is a combination of chip select, sclk, instruction code, address, mode- and/or dummy bytes, transmit and/or receive data.
LUT	Look-up table.

10.2.2 Memory Map and Register Definition

This section provides the memory map and register definitions of the QuadSPI module.

10.2.2.1 Register Write Access

This section describes the write access restriction terms that apply to all registers, which can be one of the following:

- **Register Write Access Restriction**

For each register bit and register field, the write access conditions are specified in the detailed register description. A description of the write access conditions is given in the following table. If, for a specific register bit or field, none of the given write access conditions is fulfilled, any write attempt to this register bit or field is ignored without any notification. The values of the bits or fields are not changed.

The condition term [A or B] indicates that the register or field can be written to if at least one of the conditions is fulfilled.

Table 10-4. Register Write Access Restrictions

Condition	Description
Anytime	No write access restriction.
Disabled Mode	Write access only if <i>QSPI_MCR[MDIS] = 1</i> .
Normal Mode	Write access only if the module is in <i>Normal Mode</i> .

• Register Write Access Requirements

All registers can be accessed with 8-bit, 16-bit, and 32-bit wide operations. For some of the registers, at least a 16/32-bit wide write access is required to ensure correct operation. This write access requirement is stated in the detailed register description for each register affected.

10.2.2.2 Serial Flash Address Assignment

The serial flash address assignment may be modified by writing into [Serial Flash A1 Top Address \(QuadSPI_SFA1AD\)](#) and [Serial Flash A2 Top Address \(QuadSPI_SFA2AD\)](#) for device A and into [Serial Flash B1 Top Address \(QuadSPI_SFB1AD\)](#) and [Serial Flash B2 Top Address \(QuadSPI_SFB2AD\)](#) for device B. The following table shows how different access modes are related to the address specified for the next SFM Command. Note that this address assignment is valid for both IP and AHB commands.

Table 10-5. Serial Flash Address Assignment

Parameter	Function	Access Mode
QSPI_AMBA_BASE ((31:10) - 22 bits)	QuadSPI AHB base address	
TOP_ADDR_MEMA1(TPADA1)	Top address for the external flash A1 (first device of the dual die flash A, or the first of the two independent flashes sharing the IOFA)	Any access to the address space between TOP_ADDR_MEMA1 and QSPI_AMBA_BASE will be routed to Serial Flash A1

Table continues on the next page...

Table 10-5. Serial Flash Address Assignment (continued)

Parameter	Function	Access Mode
TOP_ADDR_MEMA2(TPADA2)	Top address for the external flash A2 (second device of the dual die flash A, or the second of the two independent flashes sharing the IOFA).	Any access to the address space between TOP_ADDR_MEMA2 and TOP_ADDR_MEMA1 will be routed to Serial Flash A2
TOP_ADDR_MEMB1(TPADB1)	Top address for the external flash B1 (first device of the dual die flash B, or the first of the two independent flashes sharing the IOFB)	Any access to the address space between TOP_ADDR_MEMB1 and TOP_ADDR_MEMA2 will be routed to Serial Flash B1
TOP_ADDR_MEMB2(TPADB2)	Top address for the external flash B2 (second device of the dual die flash B or the second of the two independent flashes sharing the IOFB)	Any access to the address space between TOP_ADDR_MEMB2 and TOP_ADDR_MEMB1 will be routed to Serial Flash A2

10.2.2.3 AMBA Bus Register Memory Map

QSPI_AMBA_BASE defines the address to be used as start address of the serial flash device as defined by the system memory map..

Table 10-6. QuadSPI AMBA Bus Memory Map

Address	Register Name
Memory Mapped Serial Flash Data - Individual Flash Mode on Flash A	
QSPI_AMBA_BASE to (TOP_ADDR_MEMA2 - 0x01)	Memory Mapped Serial Flash Data - Individual Flash Mode on Flash A Refer to Memory Mapped Serial Flash Data - Individual Flash Mode on Flash A for details and to Table 10-14 and Table 10-18 for information about the byte ordering.
Memory Mapped Serial Flash Data - Individual Flash Mode on Flash B	
TOP_ADDR_MEMA2 to (TOP_ADDR_MEMB2 - 0x01)	Memory Mapped Serial Flash Data - Individual Flash Mode on Flash B Refer to Memory Mapped Serial Flash Data - Individual Flash Mode on Flash B for details and to Table 10-14 and Table 10-18 for information about the byte ordering.
Parallel Flash Mode	
QSPI_AMBA_BASE to (TOP_ADDR_MEMB2 - 0x01)	Parallel Flash Mode Refer to Parallel Flash Mode for details and to Table 10-17 and Table 10-18 for information about the byte ordering.
AHB RX Data Buffer (QSPI_ARDB0 to QSPI_ARDB31)	
QSPI_ARDB_BASE to... (32 * 4 Byte) QSPI_ARDB_BASE + 0x0000_01FF	AHB RX Data Buffer (QSPI_ARDB0 to QSPI_ARDB31) Refer to Table 10-14 and Table 10-16 for information about the byte ordering.

Note

Any read access to non-implemented addresses will provide undefined results.

In case single die flash devices, TOP_ADDR_MEMA2 and TOP_ADDR_MEMB2 should be initialized/programmed to TOP_ADDR_MEMA1 and TOP_ADDR_MEMB1 respectively- in effect, setting the size of these devices to 0. This would ensure that the complete memory map is assigned to only one flash device.

Parallel Flash Mode is valid only for commands related to data read from the serial flash. The first device of flash A has to be paired with the first device of flash B and the second device of flash A has to be paired with the second device of flash B in parallel mode. Parallel mode is selected via the QSPI_BFGENCR[PAR_EN] bit for all masters in AHB driven mode and via the QSPI_IPCR[PAR_EN] in IP driven mode. In parallel mode, the incoming address (SFAR address in case of IP initiated transactions and the incoming AHB address in case of AHB initiated transactions) is divided by 2 and sent to the two flashes connected in parallel.

Any IP Command other than data read in Parallel Flash Mode will result in the assertion of the QSPI_FR[IUEF] flag and any AHB Command other than data read in Parallel Flash Mode will result in the assertion of the QSPI_FR[ABSEF] flag.

In the Individual Flash Modes, the 3/4 address bytes (as programmed in the instruction/operand in the sequence) available for the flash address is determined by SFADR [23:0] or SFADR [31:0] as given in the table above.

In Parallel Flash Mode, both flashes are read with the same starting address of 3/4 (as programmed in the instruction/operand in the sequence) bytes in size. This address is derived from SFADR [24:1] or SFADR [31:1] as given in the table above. The LSB of the SFADR field is used to select the appropriate bits of both flash devices to combine the byte corresponding to the selected address.

10.2.2.4 AHB Bus Register Memory Map Descriptions

This chapter contains definitions of registers in the AMBA address space.

10.2.2.4.1 AHB Bus Access Considerations

It has to be noted that all logic in the QuadSPI module implementing the AHB Bus access is designed to read the content of an external serial flash device. Therefore the following restrictions apply to the QuadSPI module with respect to accesses to the AHB bus:

- Any write access is answered with the ERROR condition according to the AMBA AHB Specification. No write occurs.
- Any AHB Command resulting in the assertion of the QSPI_FR[ABSEF] flag is answered with the ERROR condition according to the AMBA_AHB specification. The resulting AHB Command is ignored.
- AHB Bus access types fully supported are NONSEQ and BUSY.
- AHB access type SEQ is treated in the same way like NONSEQ. Refer to the AMBA AHB Specification for further details.

10.2.2.4.2 Memory Mapped Serial Flash Data - Individual Flash Mode on Flash A

Starting with address QSPI_AMBA_BASE the content of the first external serial flash devices is mapped into the address space of the device containing the QuadSPI module. Serial flash address byte address 0x0 corresponds to bus address QSPI_AMBA_BASE with increasing order. Assuming that a dual-die flash is connected on the first set of external pads, the address space is divided into two parts, one for each device of the dual die package. Refer to the following table for the address mapping. The byte ordering for 32 bit access is given in [Table 10-14](#) and for 64 bit read access the byte ordering is given in [Table 10-18](#).

Table 10-7. Memory Mapped Individual Flash Mode - Flash A Address Scheme

Memory Mapped Address 32 Bit Access	Memory Mapped Address 64 Bit Access	Serial Flash Byte Address	Flash Device
QSPI_AMBA_BASE + 0x00	QSPI_AMBA_BASE + 0x00	0x00_0000 to 0x00_0003	A1
QSPI_AMBA_BASE + 0x04		0x00_0004 to 0x00_0007	
...	...	...	
TOP_ADDR_MEMA1 - 0x08	TOP_ADDR_MEMA1 - 0x08	(TOP_ADDR_MEMA1 - 0x08) to (TOP_ADDR_MEMA1 - 0x04 - 0x01)	
TOP_ADDR_MEMA1 - 0x04		(TOP_ADDR_MEMA1 - 0x04) to (TOP_ADDR_MEMA1 - 0x01)	

Table continues on the next page...

Table 10-7. Memory Mapped Individual Flash Mode - Flash A Address Scheme (continued)

Memory Mapped Address 32 Bit Access	Memory Mapped Address 64 Bit Access	Serial Flash Byte Address	Flash Device
TOP_ADDR_MEMA1 + 0x00	TOP_ADDR_MEMA1 + 0x00_0000	0x00_0000 to 0x00_0003	A2
TOP_ADDR_MEMA1 + 0x04		0x00_0004 to 0x00_0007	
....	...	...	
TOP_ADDR_MEMA2 - 0x08	TOP_ADDR_MEMA2 - 0x08	(TOP_ADDR_MEMA2 - 0x08) to (TOP_ADDR_MEMA2 - 0x04 - 0x01)	
TOP_ADDR_MEMA2 - 0x04		(TOP_ADDR_MEMA2 - 0x04) to (TOP_ADDR_MEMA2 - 0x01)	

The available address range depends from the size of the external serial flash device. Any access beyond the size of the external serial flash provides undefined results.

For details concerning the read process refer to [Flash Read](#).

10.2.2.4.3 Memory Mapped Serial Flash Data - Individual Flash Mode on Flash B

Starting with address TOP_ADDR_MEMA2 the content of the first external serial flash devices is mapped into the address space of the device containing the QuadSPI module. Serial flash address byte address 0x0 corresponds to bus address TOP_ADDR_MEMA2 with increasing order. Assuming that a dual-die flash is connected on the first set of external pads, the address space is divided into two parts, one for each device of the dual die package. Refer the following table for the address mapping. The byte ordering for 32 bit access is given in [Table 10-14](#) and for 64 bit read access the byte ordering is given in [Table 10-18](#).

Table 10-8. Memory Mapped Individual Flash Mode - Flash B Address Scheme

Memory Mapped Address 32 Bit Access	Memory Mapped Address 64 Bit Access	Serial Flash Byte Address	Flash Device
TOP_ADDR_MEMA2 + 0x00	TOP_ADDR_MEMA2 + 0x00	0x00_0000 to 0x00_0003	B1
TOP_ADDR_MEMA2 + 0x04		0x00_0004 to 0x00_0007	
...	...	...	
TOP_ADDR_MEMB1 - 0x08	TOP_ADDR_MEMB1 - 0x08	(TOP_ADDR_MEMB1 - TOP_ADDR_MEMA2 - 0x08) to (TOP_ADDR_MEMB1 - TOP_ADDR_MEMA2 - 0x04 - 0x01)	
TOP_ADDR_MEMB1 - 0x04		(TOP_ADDR_MEMB1 - TOP_ADDR_MEMA2 - 0x04) to (TOP_ADDR_MEMB1 - TOP_ADDR_MEMA2 - 0x01)	
TOP_ADDR_MEMB1 + 0x00	TOP_ADDR_MEMB1 + 0x00_0000	0x00_0000 to 0x00_0003	B2

Table continues on the next page...

Table 10-8. Memory Mapped Individual Flash Mode - Flash B Address Scheme (continued)

Memory Mapped Address 32 Bit Access	Memory Mapped Address 64 Bit Access	Serial Flash Byte Address	Flash Device
TOP_ADDR_MEMB1 + 0x04		0x00_0004 to 0x00_0007	
.....	...	...	
TOP_ADDR_MEMB2 - 0x08	TOP_ADDR_MEMA2 - 0x08	(TOP_ADDR_MEMB2 - TOP_ADDR_MEMB1 - 0x08) to (TOP_ADDR_MEMB2 - TOP_ADDR_MEMB1 - 0x04 - 0x01)	
TOP_ADDR_MEMB2 - 0x04		(TOP_ADDR_MEMB2 - TOP_ADDR_MEMB1 - 0x04) to (TOP_ADDR_MEMB2 - TOP_ADDR_MEMB1 - 0x01)	

The available address range depends from the size of the external serial flash device. Any access beyond the size of the external serial flash provides undefined results.

For details concerning the read process refer to [Flash Read](#).

10.2.2.4.4 Parallel Flash Mode

Any of the AHB flexible-buffers can be configured to work in parallel flash mode by programming the QSPI_BFGENCR[PAR_EN] bit to '1'. When parallel mode is set, Flash A1 is paired with Flash B1 and Flash A2 is paired with Flash B2. In parallel mode, software should ensure that the size of Flash A1(A2) is equal to the size of Flash B1(B2).

Reads from any even AHB bus address provides bits [7:4] of both serial flash devices and reads from any odd AHB bus address provides bits [3:0] of both flash devices. Refer to the following table for the address mapping. The byte ordering for 32 bit access is given in [Table 10-16](#) and for 64 bit read access the byte ordering is given in [Table 10-18](#).

Table 10-9. Memory Mapped Parallel Flash Mode Address Scheme

Memory Mapped Address 32 Bit Access	Memory Mapped Address 64 Bit Access	Serial Flash A Byte Address	Serial Flash B Byte Address
QSPI_AMBA_BASE + 0x0000_0000	QSPI_AMBA_BASE + 0x00 For details, please refer to Parallel mode and Dual Die Flashes .	0x00_0000	0x00_0000
		-	-
		0x00_0001	0x00_0001
		0x00_0002	0x00_0002
QSPI_AMBA_BASE + 0x0000_0004	QSPI_AMBA_BASE + 0x08	-	-
		0x00_0003	0x00_0003
		0x00_0004	0x00_0004
		-	-
QSPI_AMBA_BASE + 0x0000_0008	QSPI_AMBA_BASE + 0x08	0x00_0005	0x00_0005

Table continues on the next page...

Table 10-9. Memory Mapped Parallel Flash Mode Address Scheme (continued)

Memory Mapped Address 32 Bit Access	Memory Mapped Address 64 Bit Access	Serial Flash A Byte Address	Serial Flash B Byte Address
QSPI_AMBA_BASE + 0x0000_000C		0x00_0006 - 0x00_0007	0x00_0006 - 0x00_0007
...	...	...	...
TOP_ADDR_MEMB2 - 0x08	TOP_ADDR_MEMB2 - 0x08	(TOP_ADDR_MEMB2 - QSPI_AMBA_BASE - 0x08)/2	(TOP_ADDR_MEMB2 - QSPI_AMBA_BASE - 0x08)/2
TOP_ADDR_MEMB2 - 0x04		(TOP_ADDR_MEMB2 - QSPI_AMBA_BASE - 0x04)/2 + 0x01	(TOP_ADDR_MEMB2 - QSPI_AMBA_BASE - 0x04)/2 + 0x01

The available address range covers 27 address bits, corresponding to 128 MB per flash device. The usable space depends from the size of the external serial flash devices. Any access beyond the size of the external serial flash provides undefined results.

For details concerning the read process refer to [Flash Read](#).

10.2.3 Interrupt Signals

The interrupt request lines of the QuadSPI module are mapped to the internal flags according to the following table.

Table 10-10. Assignment of Interrupt Request Lines

IRQ/DMA line	QSPI_FR Flag	Interrupt Description
ipi_int_tfff	TBFF	TX Buffer Fill
ipi_int_tcf	TFF	Peripheral Command Transaction Finished
ipi_int_rfdf	RBDF	RX Buffer Drain
ipi_int_overn		Buffer Overflow/Underrun Error Logical OR from:
	RBOF	RX Buffer Overrun
	TBUF	TX Buffer Underrun
	ABOF	AHB Buffer Overflow
ipi_int_cerr		Serial Flash Command Error Logical OR from:
	IPAEF	Peripheral access while AHB busy Error
	IPIEF	Peripheral Command could not be triggered Error
	IPGEF	Peripheral access while AHB Grant Error
	IUEF	Peripheral Command Usage Error

Table continues on the next page...

Table 10-10. Assignment of Interrupt Request Lines (continued)

IRQ/DMA line	QSPI_FR Flag	Interrupt Description
ipi_int_ored	DLPFF, TBFF, TFF, ILLINE, RBDF, RBOF, TBUF, ABSEF, ABOF, IPAEF, IPIEF, IPGEF, IUFEF	Logical OR from all the QSPI_FR flags mentioned

10.2.4 Functional Description

This section provides the functional information of the QuadSPI module.

10.2.4.1 Serial Flash Access Schemes

The Quad Serial Peripheral Interface (QuadSPI) block acts as an interface to one single or two external serial flash devices, each with up to 4 bidirectional data lines. Depending from the serial flash devices attached to the QuadSPI module the following access schemes are possible:

Table 10-11. Access Schemes for Serial Flash Data Access

Access Scheme	One Flash Device on Port A	One Flash Device on Port B	Two identical Flash Devices connected on Port A and Port B
Individual Flash Mode: Access to Flash A	Yes	N/a	Yes
Individual Flash Mode: Access to Flash B	N/a	Yes	Yes
Parallel Flash Mode: Read from Flash A and Flash B	N/a	N/a	Yes

Note

If two flash devices are accessed in Parallel Flash Mode, they are accessed with identical control signals. Special alignment on per-flash basis is **not** possible. It is within the responsibility of the application to ensure that the identical signals are applicable to both flash devices.

In Parallel Flash Mode, both external serial flash devices appear logically as one single memory doubled in size with respect to one individual flash device.

If two different flash devices are attached, they can be operated only in Individual Flash Mode.

In the Parallel Flash Mode, only data read commands are supported. Any other IP Command will result in an error condition signaled by the assertion of the QSPI_FR[IUEF] flag and any other AHB Command will result in the assertion of the QSPI_FR[ABSEF] flag.

In the Individual Flash Mode, all supported commands are available.

Unless explicitly noted, all the following descriptions relate to the Individual Flash Mode.

10.2.4.2 Modes of Operation

Refer to [QuadSPI Modes of Operation](#) for an overview over the possible operational modes of the QuadSPI block.

- Normal Mode can be used for write or read accesses to an external serial flash device.
 - Serial Flash Write: Data can be programmed into the flash via the IP interface only. Refer to [Flash Programming](#) for further details.
 - Serial Flash Read: Read the contents of the serial flash device. Two separate read channels are available via RX Buffer and AHB Buffer, see [Flash Read](#).
- Stop Mode: The mode is used for power management. When a request is made to enter Stop Mode, the QuadSPI block acknowledges the request and completes the SFM Command in progress, then the system clocks to the QuadSPI block may be shut off
- Module Disable Mode: The mode is used for power management. The clock to the non-memory mapped logic in the QuadSPI can be stopped while in Module Disable Mode. The module enters the mode by setting QSPI_MCR[MDIS].

10.2.4.3 Normal Mode

This mode is used to allow communication with an external serial flash device. Compared to the standard SPI protocol, this communication method uses up to 4 bidirectional data lines operating at high data rates. The communication to the external serial flash device consists of an instruction code and optional address, mode, dummy

and data transfers. The flexible programmable core engine described below is immune to a wide variety of command/protocol differences in the serial flash devices provided by various flash vendors.

10.2.4.3.1 Programmable Sequence Engine

The core of the QuadSPI module is a programmable sequence engine that works on "instruction-operand" pairs. The core controller executes each programmed instruction sequentially. The complete list of instructions and the corresponding operands is given in the following table.

Table 10-12. Instruction set

Instruction	Instruction encoding	Pins	Operand	Action on Serial Flash(es)
CMD	6'd1	N=2'd{0,1,2} 2'd0 - One pad 2'd1 - Two pads 2'd2 - Four pads	8 bit command value	Provide the serial flash with operand on the number of pads specified
ADDR	6'd2		Number of address bits to be sent (for example, 8'd24 => 24 address bits required)	Provide the serial flash with address cycles according to the operand on the number of pads specified. The actual address to be provided will be derived from the incoming address in case of AHB initiated transactions and the value of SFAR in case of IPS initiated transactions .
DUMMY	6'd3		Number of dummy clock cycles (should be <= 64 cycles)	Provide the serial flash with dummy cycles as per the operand. The PAD information defines the number of pads in input mode. (for example, one pad implies that pad 1 is not driven, rest all are driven)
MODE	6'd4		8 bit mode value	Provide the serial flash with 8 bit operand on the number of pads specified
MODE2	6'd5	N=2'd{0,1}	2 bit mode value	Provide the serial flash with 2 bit operand on the number of pads ¹ specified
MODE4	6'd6	N=2'd{0,1,2}	4 bit mode value	Provide the serial flash with 4 bit operand on the number of pads ² specified
READ	6'd7	N=2'd{0,1,2} 2'd0 - One pad 2'd1 - Two pads 2'd2 - Four pads	Read data size in bytes (for AHB transactions, the user's application should ensure that data size is a multiple of 8 bytes)	Read data from flash on the number of pads specified. The data size may be overwritten by writing to the ADATSZ field of the QSPI_BUFxCR registers for AHB initiated transactions and IDATSZ field of IP Configuration Register (QuadSPI_IPCR) for IP initiated transactions.
WRITE	6'd8		Write data size in bytes	Write data on number of pads specified. The data size may be overwritten by writing to the IDATSZ field of IP Configuration Register (QuadSPI_IPCR) register

Table continues on the next page...

Table 10-12. Instruction set (continued)

Instruction	Instruction encoding	Pins	Operand	Action on Serial Flash(es)
JMP_ON_CS	6'd9	NA	Instruction number	Every time the CS is deasserted, jump to the instruction pointed to by the operand. This instruction allows the programmer to specify the behavior of the controller when a new read transaction is initiated following a CS deassertion.
ADDR_DDR	6'd10	N=2'd{0,1,2} 2'd0 - One pad 2'd1 - Two pads 2'd2 - Four pads	Number of address bits to be sent (for example, 8'd24 => 24 address bits required)	Provide the serial flash with address cycles according to the operand on the number of pads specified at each clock edge of serial flash clock. The actual address to be provided will be derived from the incoming address in case of AHB initiated transactions and the value of QSPI_SFAR in case of IPS initiated transactions.
MODE_DDR	6'd11		8 bit mode value	Provide the serial flash with 8 bit operand on the number of pads specified at each clock edge of serial flash.
MODE2_DDR	6'd12	N=2'd{0}	2 bit mode value	Provide the serial flash with 2 bit operand on the number of pads specified at each clock edge of serial flash ³
MODE4_DDR	6'd13	N=2'd{0,1}	4 bit mode value	Provide the serial flash with 4 bit operand on the number of pads specified at each clock edge of serial flash ⁴ .
READ_DDR	6'd14	N=2'd{0,1,2} 2'd0 - One pad 2'd1 - Two pads 2'd2 - Four pads	Read data size in bytes (for AHB transactions, the user's application should ensure that data size is in multiple of 8 bytes)	Read data from flash on the number of pads specified at each clock edge of serial flash. The data size may be overwritten by writing to the ADATSZ field of the QSPI_BUFxCR registers for AHB initiated transactions and IDATSZ field of IP Configuration Register (QuadSPI_IPCR) for IP initiated transactions
WRITE_DDR	6'd15		Write data size in bytes	Write data on the number of pads specified at each clock edge of serial flash. The data size may be overwritten by writing to the IDATSZ field of IP Configuration Register (QuadSPI_IPCR) register
DATA_LEARN ⁵	6'd16		8 bit Data learning pattern	Find the correct sampling point with the data learning pattern. When this instruction is encountered, the QSPI_SMPR[DDRSMP] values are ignored and the controller finds the correct sampling point on its own by sampling the data learning pattern. ⁶
STOP	8'd0	NA	NA	Stop execution; deassert CS

1. For a one pad instruction, MODE2 will take 2 serial flash clock cycles on the flash interface.
2. For a one pad instruction, MODE4 will take 4 serial flash clock cycles on the flash interface. For a 4 pad instruction, MODE4 will take 1 serial flash clock cycle on the flash interface.
3. For a one pad instruction, MODE2_DDR will take 1 serial flash clock cycle on the flash interface.
4. For a one pad instruction, MODE4_DDR will take 2 serial flash clock cycles on the flash interface. For a 4 pad instruction MODE4_DDR will take half a cycle on the serial flash interface.
5. Data learning is not implemented on this chip.
6. t is not recommended to have 0x00 or 0xFF as the data learning pattern.

A sequence of such instruction-operand pairs may be pre-populated in the LUT according to the device connected on board. Each instruction-operand pair is of 16 bits (2 bytes) each. Every sequence pre-programmed in the LUT is referred to by its index.

The programmable sequence engine allows the user to configure the QuadSPI module according to the serial flash connected on board. The flexible structure is easily adaptable to new command/protocol changes from different vendors.

10.2.4.3.2 Flexible AHB buffers

In order to reduce the latency of the reads for AHB masters, the data read from the serial flash is buffered in flexible AHB buffers. There are four such flexible buffers. The size of each of these buffers is configurable with the minimum size being 0 Bytes and maximum size being the size of the complete buffer instantiated. The size of buffer 0 is defined as being from 0 to QSPI_BUF0IND. The Size of buffer 1 is from QSPI_BUF0IND to QSPI_BUF1IND, buffer2 is from QSPI_BUF1IND to QSPI_BUF2IND and buffer 3 is from QSPI_BUF2IND to the size of the complete buffer, which is given in the chip-specific QuadSPI information.

Each flexible AHB buffer is associated with the following

1. An AHB master. Optionally, buffer3 may be configured as an "all master" buffer by setting the QSPI_BUF3CR[ALLMST] bit. When buffer3 is configured in such a way, any access from a master not associated with any other buffer is routed to buffer3.
2. A datasize field representing the amount of data to be fetched from the flash on every "missed" access.

The master port number of every incoming request is checked and the data is returned/fetched into the corresponding associated buffer. Every "missed" access to the buffer causes the controller to clear the buffer and fetch QSPI_BUFxCR[ADATSZ] amount of data from the serial flash. As such, there is no benefit in configuring a buffer size of greater than ADATSZ, as the locations greater than ADATSZ will never be used. For any AHB access, the sequence pointed to by the QSPI_BFGENCR[SEQID] field is used for the flash transaction initiated. The data is returned to the master as soon as the requested amount is read from the serial flash. The controller however, continues to prefetch the rest of the data in anticipation of a next consecutive request. [Figure 10-16](#) shows the flexible AHB buffers.

The QSPI_BFGENCR[SEQID] field points to an index of the LUT. Refer to [Look-up Table](#) for details.

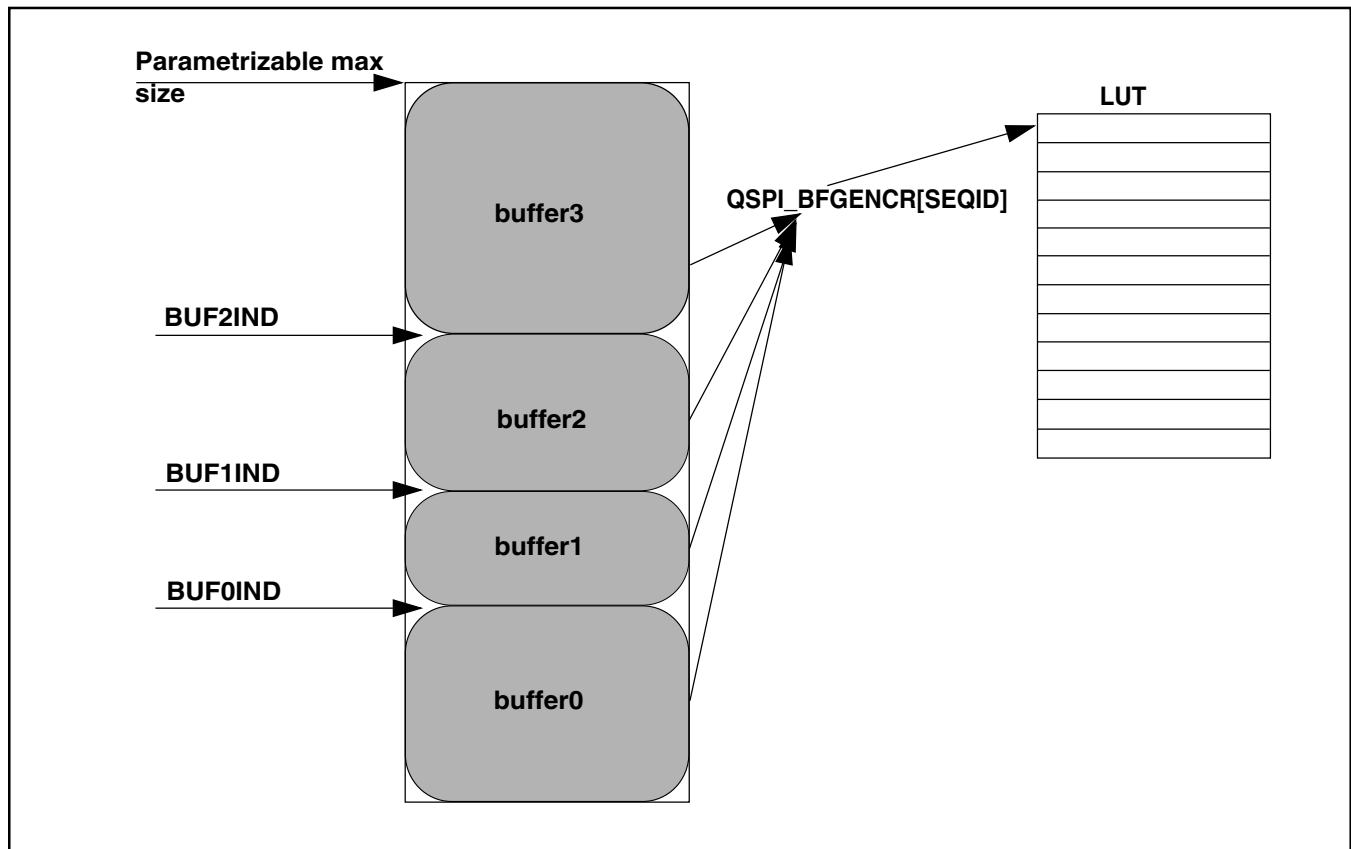


Figure 10-16. Flexible AHB Buffers

Buffer0 may optionally be configured to be associated with a high priority master by setting the QSPU_BUF0CR[HP_EN] bit. An access by a high priority master suspends any ongoing prefetch to any of the other buffers. The ongoing prefetch is suspended and the high priority master is serviced first. Once the high priority masters access completes, the suspended transaction is resumed (before any other AHB access is entertained). The status of the suspended buffer can be read from [Sequence Suspend Status Register \(QuadSPI_SPNDST\)](#).

10.2.4.3.3 Suspend-Abort Mechanism

Any low priority AHB access can be suspended by a high priority AHB master request. The ongoing transaction is suspended at 64 bit boundary. The suspended transaction is restarted after the high priority master is served and the high priority transaction including data prefetch is completed. While a transaction is in suspended state, it may be aborted if a transaction by the same suspended master is made to a location which is different from the location of the suspended transaction.

Any ongoing transaction is aborted if a request from the same master arrives for a location other than the location at which the transaction is going on. The abort can happen at any point of time.

10.2.4.3.4 Look-up Table

The Look-up-table or LUT consists of a number of pre-programmed sequences. Each sequence is basically a sequence of instruction-operand pairs which when executed sequentially generates a valid serial flash transaction. Each sequence can have a maximum of 8 instruction-operand pairs. The LUT can hold a maximum of 16 sequences. The figure below shows the basic structure of the sequence in the LUT.

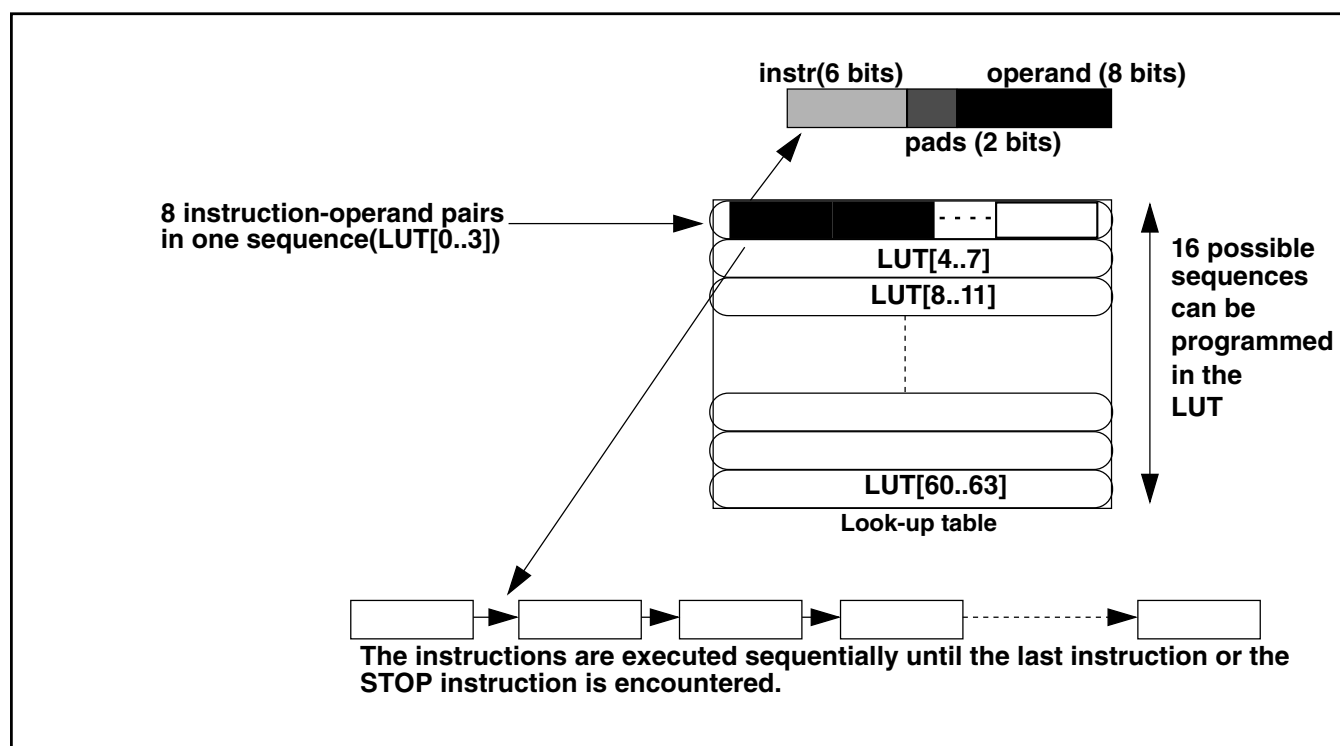


Figure 10-17. LUT and sequence structure

At reset, the index 0 of the look-up-table (LUT[0..3]) is programmed with a basic read sequence as given in [Table 10-13](#). After reset the complete LUT may be reprogrammed according to the device connected on board. In order to protect its contents during a code runover the LUT may be locked, after which a write to the LUT will not be successful until it has been unlocked again. The key for locking or unlocking the LUT is **0x5AF05AF0**. The process for locking and un-locking the LUT is as follows:

Locking the LUT

1. Write the key (**0x5AF05AF0**) in to the [LUT Key Register \(QuadSPI_LUTKEY\)](#).
2. Write 0b01 to the [LUT Lock Configuration Register \(QuadSPI_LCKCR\)](#). Note that this IPS transaction should immediately follow the above IPS transaction (no other IPS transaction can be issued in between). A successful write into this register locks the LUT.

Unlocking the LUT

1. Write the key (**0x5AF05AF0**) into the [LUT Key Register \(QuadSPI_LUTKEY\)](#)
2. Write 0b10 to the [LUT Lock Configuration Register \(QuadSPI_LCKCR\)](#). Note that this IPS transaction should immediately follow the above IPS transaction (no other IPS transaction can be issued in between). A successful write into this register unlocks the LUT.

The lock status of the LUT can be read from QSPI_LCKCR[UNLOCK] and QSPI_LCKCR[LOCK] bit.

Some example sequences are defined in [Example Sequences](#). The reset sequence at LUT index 0 is given in the following table.

Table 10-13. Reset sequence

Instruction	Pad	Operand	Comment
CMD	0x00	0x03	Read Data byte command on one pad
ADDR	0x00	0x18	24 Addr bits to be sent on one pad
READ	0x00	0x08	Read 64 bits
JMP_ON_CS	0x00	0x00	Jump to instruction 0 (CMD)

10.2.4.3.5 Issuing SFM Commands

Each access to the external device follows the same sequence:

1. The user must pre-populate the LUT with the serial flash command sequences that are required for the flash device being used.
2. The QuadSPI module starts executing the instructions in the sequence one by one. The transaction starts and the status bit QSPI_SR[BUSY] is set.
3. Communication with the external serial flash device is started and the transaction is executed.
4. When the transaction is finished (all transmit- and receive operations with the external serial flash device are finished) the status bit QSPI_SR[BUSY] is reset. In case of an IP Command the QSPI_FR[TFF] flag is asserted.

Further details are given in below in [Flash Programming](#) and [Flash Read](#).

You can trigger the processing of SFM commands in the QuadSPI module in one of the following ways:

- **Using IP commands**

For IP Commands the required components need to be written into the following registers:

- Write the serial flash address to be used by the instruction into QSPI_SFAR, refer to [Serial Flash Address Register \(QSPI_SFAR\)](#). For IP Commands not related to specific addresses, the base address of the related flash need to be programmed. For example, for an instruction which does not require an address (i.e. write enable instruction) the SFAR should be programmed with the base address of the memory the command is to be sent to.
- Write the sequence ID and data size details in the [IP Configuration Register \(QSPI_IPCR\)](#).
- Note that the write into the QSPI_IPCR[SEQID] field must be the last step of the sequence. It is possible to combine all fields of the QSPI_IPCR into one single write. Refer to [IP Configuration Register \(QSPI_IPCR\)](#) for details.

Note that there are some conditions where no IP Command is executed after writing the QSPI_IPCR[SEQID] field and the write operation itself is ignored. They are described in [Command Arbitration](#).

• Using AHB commands

Any AHB memory mapped access is routed to one of the buffers depending on the master port number of the request. If the access is a "miss", a new serial flash transaction is started. The transaction is based on the sequence pointed to by the BFGENCR[SEQID] field as described in [Flexible AHB buffers](#).

An AHB access is termed memory mapped when the access is to the memory mapped serial flashes, as described in [Memory Mapped Serial Flash Data - Individual Flash Mode on Flash A](#) and [Memory Mapped Serial Flash Data - Individual Flash Mode on Flash B](#).

Again the possible error conditions are described in [Command Arbitration](#).

10.2.4.3.6 Flash Programming

In all cases the memory sector to be written needs to be erased first. The programming sequence itself is then initiated in the following way:

1. Check that the TX Buffer is empty. If the QSPI_SR[TXEDA] bit is set then the TX Buffer must be cleared by writing 1 into the QSPI_MCR[CLR_TXF] bit.
2. Program the address related to the command in the QSPI_SFAR register.

3. Provide initial data for the program command into the circular buffer via register TX Buffer Data Register (QSPI_TBDR) . At least four word of data must be written into the TX Buffer up to a maximum of 32.
4. Program the QSPI_IPCR register to trigger the command. The QSPI_IPCR[SEQID] should point to an index of the LUT which has the flash program sequence pre-programmed. The IDATSZ field should be set to denote the size of the write.
5. Depending on the amount of data required, step 3 must be repeated until all the required data have been written into the QSPI_TBDR register. The QSPI_SR[TXFULL] can be used to check if the buffer is ready to receive more data. At any time, the QSPI_TBSR[TRCTR] field can be read to check how many words have been written actually into the TX Buffer.

Upon writing the QSPI_IPCR[SEQID] field (refer to step 4) the QuadSPI module will start to execute the programmed sequence. It is the responsibility of the software to ensure that a correct sequence is programmed into the LUT in accordance with the flash memory connected to the module. The data is fetched from the TX Buffer. It consists of **32** entries of 32-bits and is organized as a circular FIFO, whose read pointer is incremented by four after each fetch. When all data are transmitted, the QuadSPI module will return from 'busy' to 'idle'. However, this is not true for the external device since the internal programming is still ongoing. It is up to the user to monitor the relevant status information available from the serial flash device and to ensure that the programming is finished properly.

10.2.4.3.7 Flash Read

Host access to the data stored in the external serial flash device is done in two steps. First, the data must be read into the internal buffers and in the second step these internal buffers can be read by the host.

1. Reading Serial Flash Data into the QuadSPI Module Internal Buffers

A read access to the external serial flash device can be triggered in two different ways:

- **IP Command Read:** For **reading flash data into the RX Buffer** the user must provide the correct sequence ID in the QuadSPI_IPCR[SEQID] register. The sequence ID points to a sequence in the LUT. It is the responsibility of the software to ensure that a correct read sequence is programmed in the LUT in accordance with the serial flash device connected on board. The user should program the Serial Flash Address Register (QSPI_SFAR) and the IP Configuration Register (QSPI_IPCR) registers. All available read commands supported by the external serial flash are possible.

Optionally it is possible to clear the RX Buffer pointer prior to triggering the IP Command by writing a 1 into the QuadSPI_MCR[CLR_RXF] field.

From these inputs, the complete transaction is built when the QSPI_IPCR[SEQID] field is written. The transaction related to the read access starts and the requested number of bytes is fetched from the external serial flash device into the RX Buffer. Since the read access is triggered by an IP command, the IP_ACC status bit and the BUSY bit are both set (both are located in the Status Register (QSPI_SR)). A count of the number of entries currently in the Rx Buffer can be obtained from QSPI_RBCT[RXBRD].

The communication with the external serial flash is stopped when the specified number of bytes has been read (successful completion of the transaction).

- **AHB Command Read:** For **reading flash data into the AHB Buffer** the user must set up a read access by a master to the address range in the system memory map which the external serial flash devices are mapped to. The user should also program the buffer registers corresponding to the AHB master initiating the request, this is depends on the configuration of the QSPI_RBCT[RXBRD]. The user should provide the correct sequence ID into the buffer generic configuration register (QSPI_BFGENCR). It is the responsibility of the software to ensure that a correct read sequence is programmed in the LUT in accordance with the serial flash device connected on board. Flash device selection and access mode are determined by the address accessed in the AHB address space associated to the QuadSPI module (refer to [Memory Mapped Serial Flash Data - Individual Flash Mode on Flash A](#), [Memory Mapped Serial Flash Data - Individual Flash Mode on Flash B](#) and [Parallel Flash Mode](#).)

On each AHB read access to the memory mapped area the valid data in the AHB Buffer is checked against the address requested in the actual read. When the AHB read request can't be served from the content of the AHB Buffer, the buffer is flushed and the sequence pointed to by the sequence ID is executed by the controller. The requested number of buffer entries defined in the QSPI_BUFxCR[ADATSZ] field is then fetched from the external serial flash device into the internal AHB Buffer. Since the read access is triggered via the AHB bus, the QSPI_SR[AHB_ACC] status bit is set driving in turn the QSPI_SR[BUSY] bit until the transaction is finished. The communication with the external serial flash is stopped when the specified number of entries has been filled.

2. Data Transfer from the QuadSPI Module Internal Buffers

The data read out from the external serial flash device by the QuadSPI module is stored in the internal buffers. The means of accessing the data from the buffer differs depending on which buffer the data has been loaded to. Refer to [Block Diagram](#) for details about the two available buffers, the RX Buffer and the AHB Buffer, in the QuadSPI module:

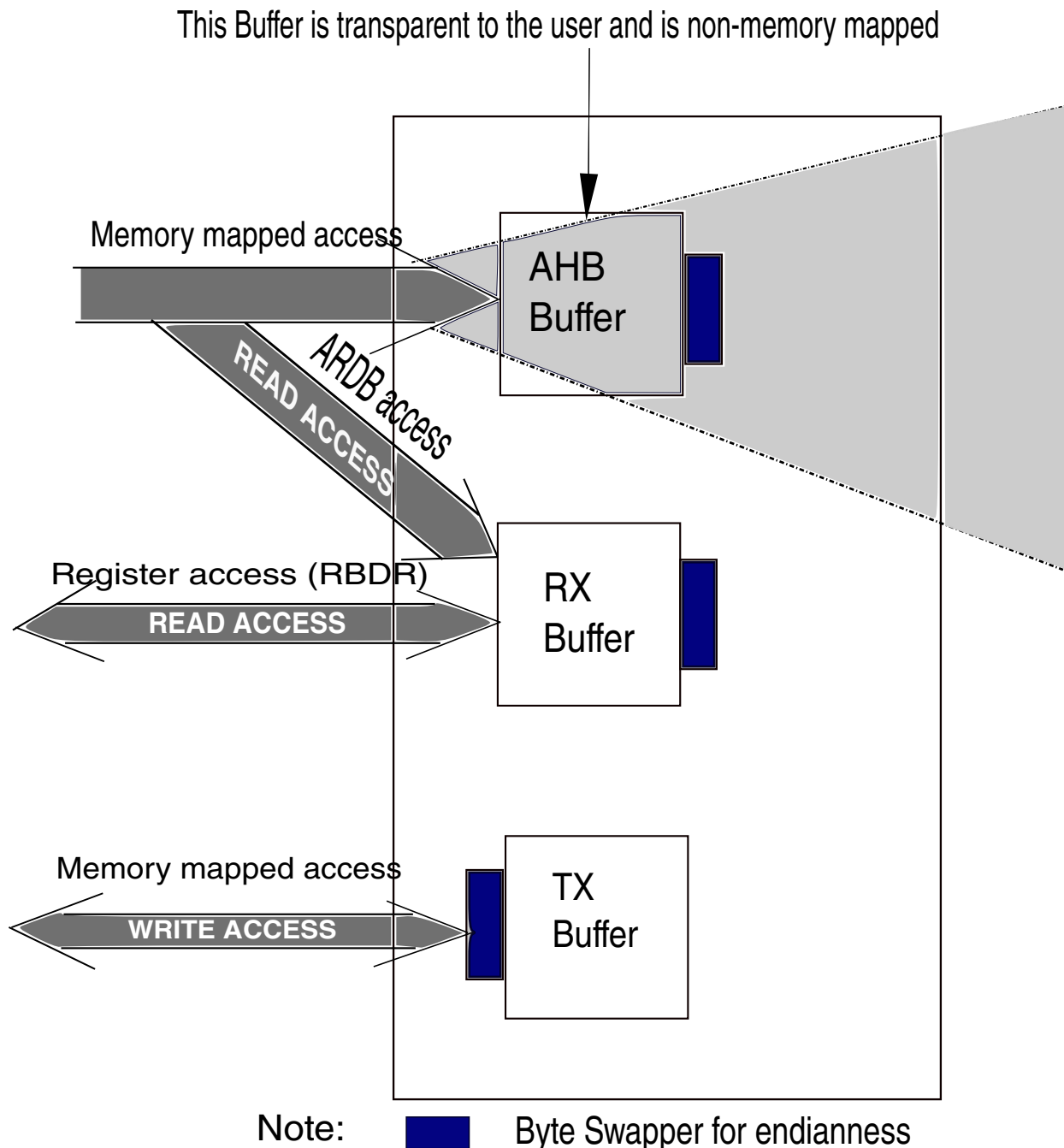


Figure 10-18. QuadSPI memory map

- The RX Buffer is implemented as FIFO of depth 32 entries of 4 bytes. Its content is accessible in two different address areas both referring to the identical data and the same physical memory.

In the IPS address space in the area associated to QSPI_RBDR0 to QSPI_RBDR31

In the AHB address space in the area associated to QSPI_ARDB0 to QSPI_ARDB31. Two successive entries are accessed with one single 64 bit AHB read operation.

RX Buffer operation can be summarized as follows: The QSPI_RBCT[WMRK] field determines at which fill level the RXWE bit is asserted and how many entries are removed from the RX Buffer on each Buffer POP operation. So the QSPI_SR[RXWE] bit indicates that the configured number of data entries is available in the RX Buffer and the QSPI_RBSR[RDBFL] field indicates how many valid entries are available in total. Note that the first entry (QSPI_RBDR0 or QSPI_ARDB0) always corresponds to the first valid entry in the RX Buffer. The software needs to manage the number of valid data bytes itself.

Further details can be found in [RX Buffer Data Register \(QuadSPI_RBDR_n\)](#) and in [AHB RX Data Buffer \(QSPI_ARDB0 to QSPI_ARDB31\)](#).

- **Flag-based Data Read of the RX Buffer** is done by polling the QSPI_SR[RXWE] bit. When it is asserted the valid entries can be read either via the IPS address space (QSPI_RBDR_n) or the AHB address space (QSPI_ARDB_n). A Buffer POP operation must be triggered by the application by writing a 1 into the QSPI_FR[RBDF] bit - this automatically updates the FIFO to point to the next entry as defined by RBCT[WMRK]. For example, if WMRK is set to 3, then the buffer will discard 16 bytes of data.
- **DMA controlled Data Read of the RX Buffer** is done by using the DMA module. The application must ensure that the DMA controller of the related device is programmed appropriately like it is described in [DMA Usage](#).

DMA controlled read out is triggered fully automatically by the assertion of the QSPI_SR[RXWE] bit. The related Buffer POP operation is also handled completely inside the QuadSPI module. Like in the case above, accessing the RX Buffer content either on QSPI_RBDR_n or QSPI_ARDB_n related addresses is equivalent.

- **AHB Buffer data read via memory mapped access:** This kind of access is done by reading one of the addresses assigned to the external serial flash device(s) within the range given in [Table 10-6](#) table *under the condition that the data requested are already present in the AHB Buffer or it is currently being read from the serial flash device by the instruction in progress*. If this is not the

case a memory mapped AHB command read is triggered as described above. If the requested data is already available in the AHB Buffer they are provided directly to the host.

When AHB access are made to the flash memory mapped address, the data will be fetched and returned to the AHB interface. Till the data is being fetched the AHB interface would be stalled. As soon as the data from the requested address has been read by the QuadSPI module the AHB read access is served. So it is possible to run sequential reads from the AHB buffer at arbitrary speed without the need to monitor any information about the availability of the data.

Nevertheless this access scheme stalls the AHB bus for the time required to read the data from the serial flash device. A better way is (when it is known the access is sequential) to have a prefetch enabled (by programming the ADATSZ field) such that before the next sequential AHB access come, the data is already fetched into the buffer.

As long as the host restricts its accesses to the data already in the buffer and the data currently fetched from the serial flash, it is possible to run the host read from the AHB Buffer in parallel to the serial flash read into the AHB Buffer.

10.2.4.3.8 Byte Ordering of Serial Flash Read Data

In this paragraph the byte ordering of the serial flash data is given. The basic scheme is that the **first** byte read out of the serial flash device - which is addressed by the QSPI_SFAR[SFADR] field - corresponds to bit position QSPI_RBDR0[31:24] register for IP Command read. In contrast to that for AHB Command read the bytes are always positioned according to the byte ordering of the AHB bus.

- **Byte Ordering in Individual Flash Mode**

The following table gives the byte ordering scheme of how the byte oriented data space of the serial flash device is mapped into one single 32 bit entry of the RX Buffer or the AHB Buffer. The table is valid within the following context:

- Flash A or Flash B in Individual Flash Mode
- All AHB data read commands with access size of 32 bit

Table 10-14. Byte Ordering in Individual Flash Mode

Serial Flash Byte Numbering	3	2	1	0
Buffer Entry Bit Position [31:0] (32 Bit data width)	[31:24]	[23:16]	[15:8]	[7:0]

Note

For IP Commands the read size can be given in number of bytes. If this number is not a multiple of 4, then the last buffer entry is not completely filled with the missing higher numbered bytes at undefined values.

For AHB Commands, reads, starting from an address not aligned to 32 bit boundaries, the requested bytes are given at the appropriate positions according to the AMBA AHB specification.

• Byte Ordering in Parallel Flash Mode

In Parallel Flash Mode each byte is combined out of 2 half bytes which are read in parallel from the two serial flash devices. The following tables shows how the flash content is separated into the half bytes and how the half bytes are assembled to the content of the QSPI_RBDR0 register.

Table 10-15. Serial Flash Device Half Byte Ordering

Serial Flash Device Byte #	Flash A Bit Position		Flash B Bit Position	
	[7:4]	[3:0]	[7:4]	[3:0]
0	fah0	fal0	fbh0	fbI0
1	fah1	fal1	fbh1	fbI1
2	fah2	fal2	fbh2	fbI2
3	fah3	fal3	fbh3	fbI3
4	fah4	fal4	fbh4	fbI4
5	fah5	fal5	fbh5	fbI5
6	fah6	fal6	fbh6	fbI6
7	fah7	fal7	fbh7	fbI7
8	fah8	fal8	fbh8	fbI8

The table entry naming reflects the half byte positioning in the serial flash devices:

- <fa>h0 means **Flash A**, <fb>h0 means Flash B.

- fa<h>0 means half byte in **high position**, fa<l>0 means half byte in low position.
- fah<0> means **physical byte address 0** in the serial flash device, fal<1> means physical byte address 1 in the serial flash device.

Table 10-16. Byte Ordering in Parallel Flash Mode - RX Buffer

QSPI_SFAR[SFADR] set to 0x000_0000								
QSPI_RBDR0 QSPI_ARDB0	fal1	fbl1	fah1	fbh1	fal0	fbl0	fah0	fbh0
QSPI_RBDR1 QSPI_ARDB1	fal3	fbl3	fah3	fbh3	fal2	fbl2	fah2	fbh2
QSPI_SFAR[SFADR] set to 0x000_0001								
QSPI_RBDR0 QSPI_ARDB0	fal2	fbl2	fah2	fbh2	fal1	fbl1	fah1	fbh1
QSPI_RBDR1 QSPI_ARDB1	fal4	fbl4	fah4	fbh4	fal3	fbl3	fah3	fbh3

Note

For IP Commands the read size can be given in number of bytes. If this number is not a multiple of 4 the last buffer entry is not completely filled with the missing higher numbered bytes at undefined values.

* Applicable only for single io mode.

Table 10-17. Byte Ordering in Parallel Flash Mode - AHB Buffer

AHB Address (32 Bit Access)	fal1	fbl1	fah1	fbh1	fal0	fbl0	fah0	fbh0
AHB Address 0x800_0004 (32 Bit Access)	fal3	fbl3	fah3	fbh3	fal2	fbl2	fah2	fbh2

Note

For AHB Command read starting from an address not aligned to 32 bit boundaries or AHB access size smaller than 32 bit the requested bytes are given at the appropriate positions according to the AMBA AHB specification.

• Buffer Entry Ordering for 64 Bit Read Access

For read access via the AHB interface 64 bit access is possible. Each 64 bit access reads 2 32 bit entries simultaneously. The ordering of these 32 bit entries within the 64 bit word is given in the following table.

Table 10-18. 64 Bit Read Access Buffer Entry Ordering

AHB Read Data Bit Position [63:0]	[63:32]	[31:0]
Buffer Entry #	Odd (1, 3, 5, ...)	Even (0, 2, 4, ...)

10.2.4.3.9 Normal Mode Interrupt and DMA Requests

The QuadSPI module has different flags that can only generate interrupt requests and one flag that can generate interrupt as well as DMA requests. The following table lists the eight conditions. Note that the flags mentioned in the table are related to the [Flag Register \(QSPI_FR\)](#).

Table 10-19. Interrupt and DMA Request Conditions

Condition	Flag(QSPI_FR)	DMA
Data Learn pattern Failure	DLFFF	-
TX Buffer Fill	TBFF	-
TX Buffer Underrun	TBUF	-
Illegal Instruction Error	ILLINE	-
RX Buffer Drain	RBDF	X
RX Buffer Overflow	RBOF	-
AHB Buffer Overflow	ABOF	-
AHB Sequence Error	ABSEF	-
IP Command Usage Error	IUEF	-
IP Command Trigger during AHB Access Error	IPAEF	-
IP Command Trigger could not be executed Error	IPIEF	-
IP Access during AHB Grant Error	IPGEF	-

Table continues on the next page...

Table 10-19. Interrupt and DMA Request Conditions (continued)

Condition	Flag(QSPI_FR)	DMA
IP Command related Transaction Finished	TFF	-

Each condition has a flag bit in the [Flag Register \(QSPI_FR\)](#) and a Request Enable bit in the [DMA Request Select and Enable Register \(QSPI_RSER\)](#). The RX Buffer Drain Flag (RBDF) has separate enable bits for generating IRQ and DMA requests. Note that not all flags have an individual IRQ line. Check the devices Interrupt Vector Table for more details.

- **Transmit Buffer Fill Interrupt Request:**

The Transmit Buffer Fill IRQ indicates that the TX Buffer can accept new data. It is asserted if the QSPI_FR[TBFF] flag is asserted and if the corresponding enable bit (QSPI_RSER[TBFIE]) is set. Refer to [TX Buffer Operation](#), for details about the assertion of the QSPI_FR[TBFF] flag.

- **Receive Buffer Drain Interrupt or DMA Request:**

The Receive Buffer Drain IRQ derived from the QSPI_FR[RBDF] flag indicates that the RX Buffer of the QuadSPI module has data available from the serial flash device to be read by the host. It remains set as long as the QSPI_RBSR[RXWE] bit is set. The QSPI_RSER[RBDIE] bit enables the related IRQ.

Aside from the IRQ it is possible to handle RX Buffer drain by DMA. If the QSPI_RSER[RBDDE] bit is set, a DMA request will be triggered when the RX Buffer contains more than QSPI_RBCT[WMRK] valid entries. The application must set the environment appropriately (for example, the DMA controller) for the DMA transfers.

- **Buffer Overflow/Underrun Interrupt Request:**

The Buffer Overflow/Underrun IRQ is a combination of the following flags (all located in the QSPI_FR register with the related enable bits in the QSPI_RSER register):

- TBUF - TX Buffer Underrun, enabled by TBUIE
- RBOF - RX Buffer Overflow, enabled by RBOIE
- ABOF - AHB Buffer Overflow, enabled by ABOIE

The Transmit Buffer Underrun indicates that an underrun condition in the TX Buffer has occurred. It is generated when a write instruction is triggered whilst the Tx Buffer is empty and the QSPI_RSER[TFUFIE] bit is set.

The Receive Buffer Overflow indicates that an overflow condition in the RX Buffer has occurred. It is generated when the RX Buffer is full, an additional read transfer attempts to write into the RX Buffer and the QSPI_RSER[RBOIE] bit is set.

The AHB Buffer Overflow indicates that an overflow condition in the AHB Buffer has occurred. It is generated when the AHB Buffer is full, an additional read transfer attempts to write into the AHB Buffer and the QSPI_RSER[ABOIE] bit is set.

The data from the transfers that generated the individual overflow conditions is ignored.

- Serial Flash Command Error Interrupt Request

If the IPAEF, IPIEF, IPGEF or IUEF flags in the QSPI_FR are set, and the related interrupt enable bits in the QSPI_RSER are also set, then an interrupt is requested.

- Transaction Finished Interrupt Request

The IP Command Transaction Finished IRQ indicates the completion of the current IP Command. It is triggered by the QSPI_FR[TFF] flag and is masked by the QSPI_RSER[TFIE] bit.

10.2.4.3.10 TX Buffer Operation

The TX Buffer provides the data used for page programming. For proper operation it is required to provide at least four entry in the TX Buffer prior to starting the execution of the page programming command. The application must ensure that the required number of data bytes is written into the TX Buffer fast enough as long as the command is executed without a TX Buffer overflow or underrun.

The QuadSPI module sets the QSPI_FR[TBFF] flag so long as the TX Buffer is not full and can accept more data.

When the QuadSPI module tries to pull data out of an empty TX Buffer the TX Buffer underrun is signaled by the QSPI_FR[TBUF] flag. The TX buffer underrun flag is also asserted when TX buffer contains less than 128 bits of data and QuadSPI module tries to pull out data from it. The current IP Command leading to the underrun condition is continued until the specified number of bytes has been sent to the serial flash device, in the underrun condition when QuadSPI module tries to pull out data of empty TX buffer, the data transferred is all F's i.e. once the underrun flag is set under this condition, it will return F's until the required number of bytes are not sent. This has been done to ensure that the software need not to erase whole sector after underrun, just reprogramming from

failure point will serve the purpose. When this Sequence Command is finished, the QSPI_FR[TBFF] flag is asserted indicating that the Tx Buffer is ready to be written again.

The TX Buffer overflow isn't signaled explicitly, but the TX Buffer fill level can be monitored by the QSPI_TBSR[TRBFL] field.

Refer to [TX Buffer Status Register \(QuadSPI_TBSR\)](#) and [Flag Register \(QuadSPI_FR\)](#) for details about the TX Buffer related registers.

10.2.4.3.11 Address scheme

Earlier serial flash memories supported only 24-bit address space hence restricting the maximum memory size of the serial flash as 16 MB. The new memory specification supports two types of 32-bit addressing mode in addition to legacy 24-bit address mode.

- **Extended Address Mode**

In this mode, the legacy 24-bit commands are converted to accept 32-bit address commands. The flash memory needs to be configured for 32-bit address mode. Also, while programming the LUT sequence in QuadSPI for 32-bit mode, the ADDR and ADDR_DDR command should be programmed with 8'd32 as the operand value. By default, the QuadSPI is in 24-bit legacy address mode. Each of the memory vendors have a different way of enabling this mode (Refer to the memory specification from memory vendors). For example, the command B7h sent to Macronix flash will enable it for 32-bit address mode.

- **Extended Address register**

In this mode, the upper 8-bit of the 32-bit address is provided by the Extended address register in the memory itself. The memory provides a specific register which is updated according to the address to be accessed. This effectively converts the legacy 24-bit address command into 32-bit address commands. The memories greater in size than 16 MB, consists of banks of 16 MB. The 8-bit written in the extended address register effectively enables a bank. For example in Spansion memory, when the extended address register is updated with a value of 0x01 with the help of the command 17h, it will open Bank1 of the memory. The consequent 24-bit address commands will lead to Bank1. The extended address register needs to be update with the respective value for access to other banks. This effectively converts the legacy 24-bit address command into 32-bit address commands.

10.2.5 Initialization/Application Information

This section provides the initialization and application information of the QuadSPI module.

10.2.5.1 Power Up and Reset

Note that the serial flash devices connected to the QuadSPI module may require special voltage characteristics of their inputs during power up or reset. It is the responsibility of the application to ensure this.

10.2.5.2 Available Status/Flag Information

This paragraph gives an overview of the different status and flag information available and their interdependencies for different use cases. Related registers are QSPI_SR and QSPI_FR. Refer to the related descriptions how to set up the QuadSPI module appropriately.

10.2.5.2.1 IP Commands

Refer to [IP Configuration Register \(QuadSPI_IPCR\)](#) for additional details not explicitly covered in this paragraph.

- **IP Commands - Normal Operation**

Writing the QSPI_IPCR[SEQID] field triggers the execution of a new IP Command. Given that this is a legal command the QSPI_SR[IPACC] and the QSPI_SR[BUSY] bits are asserted simultaneously, immediately after the execution is started.

When the instruction on the serial flash device has been finished these bits are de-asserted and the QSPI_FR[TFF] flag is set.

- **IP Commands - Error Situations**

Refer to [Table 10-20](#) below.

10.2.5.2.2 AHB Commands

Refer to Section 1, Reading Serial Flash Data into the QuadSPI Module, in [Flash Read](#) for additional details not explicitly covered in this paragraph.

• AHB Commands - Normal Operation

Memory mapped read access to a serial flash address not contained in the AHB Buffer, triggers the execution of an AHB Command. Given that this is a legal command the QSPI_SR[AHBACC] and the QSPI_SR[BUSY] bits are asserted simultaneously immediately after the execution is started. When the instruction on the serial flash device has been finished these bits are de-asserted.

• IP Commands - Error Situations

Refer to [Table 10-20](#) below.

10.2.5.2.3 Overview of Error Flags

The following table gives an overview of the different error flags in the QSPI_FR register and additional error-related details.

Table 10-20. Overview of QSPI_FR Error Flags

Error Category	Error Flag in QSPI_FR	Command Execution on Serial Flash Device TFF Behavior (in case of IP commands only)	Description
AHB Error Flag	ABSEF	Flash transaction is aborted	AHB sequence contains • WRITE instruction • WRITE_DDR instruction
AHB Error Flag	ABOF	Flash transaction continues until it finishes	Set when the module tried to push data into the AHB buffer that exceeded the size of the AHB buffer. Only occurs due to wrong programming of the QSPI_BUFxCR[ADATSZ].
Miscellaneous Error Flag	DLPPF ¹	Flash transaction continues until it finishes	Set when DATA_LEARN instruction was encountered in a sequence but no sampling point was found for the data learning pattern.
Miscellaneous Error Flag	ILLINE	Flash transaction aborted	Illegal instruction Error Flag – Set when an illegal instruction is encountered by the controller in any of the sequences.
Command Arbitration Error	IPIEF	TFF not asserted in conjunction with that command	IP Command Error - caused when IP access is currently in progress (IP_ACC set) and • write attempt to QSPI_IPCR register.

Table continues on the next page...

**Table 10-20. Overview of QSPI_FR Error Flags
(continued)**

Error Category	Error Flag in QSPI_FR	Command Execution on Serial Flash Device TFF Behavior (in case of IP commands only)	Description
			• write attempt to QSPI_SFAR register. • write attempt to QSPI_RBCT register.
Command Arbitration Error	IPAEF		• AHB Command already running, another IP Command could not be executed. • AHB Command already running, write attempt to QSPI_IPCR[SEQID] field.
Command Arbitration Error	IPGEF		• Exclusive access to the serial flash granted for AHB Commands, write attempt to QSPI_IPCR[SEQID] field.
IP Command Error	IUEF	—	• IP Command Usage Error
Buffer Related Error	RBOF	TFF is asserted on completion	• RX Buffer Overrun
Buffer Related Error	TBUF		• TX Buffer Underrun

1. Data learning is not implemented on this chip.

Note that only the buffer related errors are related to a transaction on the external serial flash. All the other errors do not trigger an actual transaction.

10.2.5.2.4 IP Bus and AHB Access Command Collisions

There are two flags related to this topic, the QSPI_FR[IPAEF] and QSPI_FR[IPIEF]. Refer to sub-section "Reading Serial Flash Data into the QuadSPI Module" of [Flash Read](#) section, for a description of the flags and [Command Arbitration](#), for details about possible command collisions.

10.2.5.3 Exclusive Access to Serial Flash for AHB Commands

It is possible that several masters need to access the serial flash device connected to the QuadSPI module separately, one master by triggering IP Commands and reading the RX Buffer (via RBDR_n register) and the other masters by triggering AHB Commands (via ARDB_n Registers). These two set of buffer (RBDR and ARDB Buffer) points to the same physical buffer. Refer to [Figure 10-18](#) To avoid command collisions resulting in

excessive latencies the QuadSPI module implements a request-handshake mechanism between the master triggering AHB Commands and the QuadSPI module allowing this specific master to request exclusive access to the serial flash device for AHB Commands. If this exclusive access is granted the execution of IP Commands is blocked. This resolves command collisions and excessive times where the AHB interface may be blocked.

If this capability is used in the device there is additional status and flag information available related to this mechanism. The QSPI_SR[AHBGNT] bit reflects the module-internal state that the exclusive access mentioned above is granted, any attempt to trigger an IP Command is rejected and results in the assertion of the QSPI_FR[IPGEF] flag. Refer to the descriptions of the related bit and flag for details.

It is within the responsibility of the application to set up the master using this mechanism appropriately, if used incorrectly no IP Commands at all can be triggered.

Two different cases can be distinguished:

10.2.5.3.1 RX Buffer Read via QSPI_ARDB Registers

In this case all masters share the AHB bus for RX Buffer as well as for AHB Buffer read. In this case the access to the AHB interface by the master triggering AHB Commands must be deferred until any pending IP Command has been finished **and** the RX Buffer readout has been finished as well. The QSPI ARDB Buffers access the Rx buffer i.e the data from the Rx Buffer is returned and no data from AHB Buffer is touched. This is the conservative use case, corresponding to the reset value 0 of the QSPI_RBCT[RXBRD] bit.

In this case the QSPI_SR[AHBGNT] bit is asserted not earlier than any running IP Command has been finished (QSPI_SR[IP_ACC] is 0), the RX Buffer has been read out completely (QSPI_RBSR[RDBFL] equal to 0) or no DMA read is pending (QSPI_SR[RXDMA] equal to 0 and Rx Buffer readout is via AHB(QSPI_RBCT[RXBRD]) equal to 1).

10.2.5.3.2 RX Buffer Read via QSPI_RBDR Registers

This is the preferred use case as an access to the AHB buffer (memory mapped flash) does not interfere with any IPS access to read the RBDR buffer. It is not possible that a pending AHB bus access triggered by an AHB Command stalls the AHB bus and blocks the RX Buffer readout since the RX Buffer is read via the IP bus based registers QSPI_RBDR0 to QSPI_RBDR31.

For this case it is recommended to program the QSPI_RBCT[RXBRD] bit to 1. The QSPI_SR[AHBGNT] bit is asserted immediately after any running IP Command has been finished (QSPI_SR[IP_ACC] is 0), the RX Buffer has been read out completely (QSPI_RBSR[RDBFL] equal to 0) or no DMA read is pending (QSPI_SR[RXDMA] equal to 0), allowing the master triggering AHB Commands to trigger AHB Commands as soon as possible without the need to wait for the RX Buffer readout to be finished.

10.2.5.4 Command Arbitration

In case of overlapping commands, the arbitration scheme is described in the following paragraphs under the assumption that the priority mechanism described in [Exclusive Access to Serial Flash for AHB Commands](#) is **not** used:

- During the execution of an IP Command, the running IP Command can't be terminated by issuing another IP Command or AHB Command. The QSPI_FR[PIEF] flag is asserted when the host tries to write into the QSPI_IPCR register. When the host triggers an AHB Command (refer to sub-section "Reading Serial Flash Data into the QuadSPI Module" of [Flash Read](#) section, for details), this command is stalled until the currently running IP Command is finished.
- During the execution of an AHB Command, the running AHB Command can't be terminated by issuing an IP Command. The command is ignored and the QSPI_FR[IPAEF] flag is asserted. Refer to [Flag Register \(QuadSPI_FR\)](#) for the description of these flags.

When another AHB Command is triggered the address of the memory mapped access is considered. If the requested address is currently read from the serial flash device, the running command is continued. If this is not the case the currently running command is terminated and another AHB Command related to the requested address is executed. Refer to sub-section "Reading Serial Flash Data into the QuadSPI Module" of [Flash Read](#) section, for further details.

In case of coinciding commands the IP Command is triggered and the AHB Command is stalled until the IP Command has been finished (QSPI_SR[IP_ACC] has been deasserted).

The IP Commands ignored in case of command collision will not result in the assertion of the QSPI_FR[TFF] flag.

10.2.5.5 Flash Device Selection

Regardless of the SFM Command (IP or AHB) the access mode is selected by specifying the 32 bit address value for the following SFM Command.

For IP Commands the access mode is selected with the address programmed into the QSPI_SFAR register. Refer to [Serial Flash Address Register \(QuadSPI_SFAR\)](#) for details.

For AHB Commands the access mode is determined by the memory mapped address which is accessed Refer to [AMBA Bus Register Memory Map](#) for details.

10.2.5.6 DMA Usage

For the complete description of the DMA module refer to the related DMA Controller chapter. In this paragraph only the details specific to the DMA usage related to the QuadSPI module are given.

10.2.5.6.1 DMA Usage in Normal Mode

10.2.5.6.1.1 Bandwidth considerations

Careful consideration of the throughput rate of the entire chain (serial flash -> AHB bus / IP Bus -> DMA controller) involved in the read data process is essential for proper operation. Such analysis must take into account not only the data rate provided by the serial flash but also the data rate of the AHB bus and the performance of the DMA controller in reading data from the RX buffer.

Two figures must match for proper operation, that means that the data rate provided by the serial flash device must not exceed the average RX Buffer readout data rate. Otherwise, the longer this state persists, a RX Buffer overflow will result.

AHB Bus Side (data read):

The total number of bus cycles for each DMA Minor Loop completion is added from the following components:

- Overhead for each minor loop, given by DMA controller: Assume 10 cycles
- Overhead due to clock domain crossing: Assume 2 cycles
- Number of bus clock cycles required for 8 bytes (64 bit read size): Assume 2 cycles (read/write sequence of DMA controller)

Note that the size of the minor loop is determined by the size of the QSPI_RBCT[WMRK] field, therefore the overhead given above distributes among $(\text{QSPI_RBCT[WMRK]}+1)/2$ read accesses of 64 bit each.

The following table gives some examples for typical use cases:

Table 10-21. Access Duration Examples - Bus Clock Side

QSPI_RBCT[WMRK]	Number of Bytes per DMA Loop ¹	Number of Bus Clock cycles for DMA Minor Loop	Time Duration of DMA Minor Loop for 120Mhz Bus clock Frequency
0	4	$12+2 = 14$	~117ns
1	8	$12+2 = 14$	~117ns
3	16	$12+4 = 16$	~133ns
7	32	$12+8 = 20$	~167ns
11	48	$12+12 = 24$	~200ns

1. DMA Loop means one Minor Loop Completion which is equivalent to one.

NOTE

The table figure represents ideal scenario, actual performance will depend on how the system is integrated.

Serial Flash Device Side (data read):

The number of serial flash cycles can be determined in the following way:

- Number of serial flash clock cycles required to read 4 bytes, corresponding to one RX Buffer entry (setup of command and address not considered): 2 cycles for Quad DDR mode instructions in Parallel Flash Mode, 4 cycles for Quad (SDR) mode instruction in parallel flash mode or Dual IO DDR mode instruction in parallel flash mode, 8 cycles for Quad Mode (SDR) instructions in Individual Flash Mode etc.
- Overhead due to clock domain crossing : 1 cycle.

The following table lists the number of clock cycles required to read the data from the serial flash corresponding to the different settings of the QSPI_RBCT[WMRK] field:

Table 10-22. Access Duration Examples - Serial Flash side

QSPI_RBCT[WMRK] setting	Num Bytes per DMA Loop ¹	Num SCKFx for 60MHz SCKFx			Time duration of Flash data readout for 60MHz SCKFx (~16.6ns period)		
		IFM ² Quad	IFM Quad DDR	PFM ³ Quad DDR	IFM Quad	IFM Quad DDR	PFM Quad DDR
0	4	9	5	3	~150ns	~83ns	~50ns
1	8	17	9	5	~282ns	~150ns	~83ns
3	16	33	17	9	~548ns	~282ns	~150ns
7	32	65	33	17	~1079ns	~548ns	~282ns

Table continues on the next page...

Table 10-22. Access Duration Examples - Serial Flash side (continued)

QSPI_RBCT[WMRK] setting	Num Bytes per DMA Loop ¹	Num SCKF _x for 60MHz SCKF _x			Time duration of Flash data readout for 60MHz SCKF _x (~16.6ns period)		
		IFM ² Quad	IFM Quad DDR	PFM ³ Quad DDR	IFM Quad	IFM Quad DDR	PFM Quad DDR
11	48	97	49	25	~1610ns	~813ns	~415ns

1. DMA Loop means one Minor loop completion which is equivalent to one Major Loop iteration.

2. Individual flash mode.

3. Parallel flash mode.

From the examples given in the two tables above, it can be seen that depending on the relationship between the Bus clock and Serial flash clock frequencies, there are settings possible where the serial flash provides the read data faster than the AHB bus can read out the RX buffer. In the above tables, it is the case of PFM Quad DDR mode with Watermark up to 3 and other cases. In these cases, the RX buffer data keeps accumulating over time and will eventually overflow. To avoid RX Buffer overflow, the data transaction size should be small enough.

A complementary example would be when the watermark is set to be too high. In such a case, the time taken by the DMA to read out the RX buffer entries should be smaller than the time taken by the controller to push in the remaining entries in the buffer.

NOTE

The tables mentioned above are only examples which must be correlated with the DMA in the system.

10.2.5.7 Parallel mode

QuadSPI can access two flashes in parallel. This increases the throughput of the QuadSPI by two times. Both write and read operations are supported in parallel mode. When dual die flashes are accessed in parallel mode, it is mandatory for flash A1 to be of the same size as B1 and A2 to be of the same size as B2. The following figure shows how QuadSPI maps the incoming addresses to the different flashes connected on board.

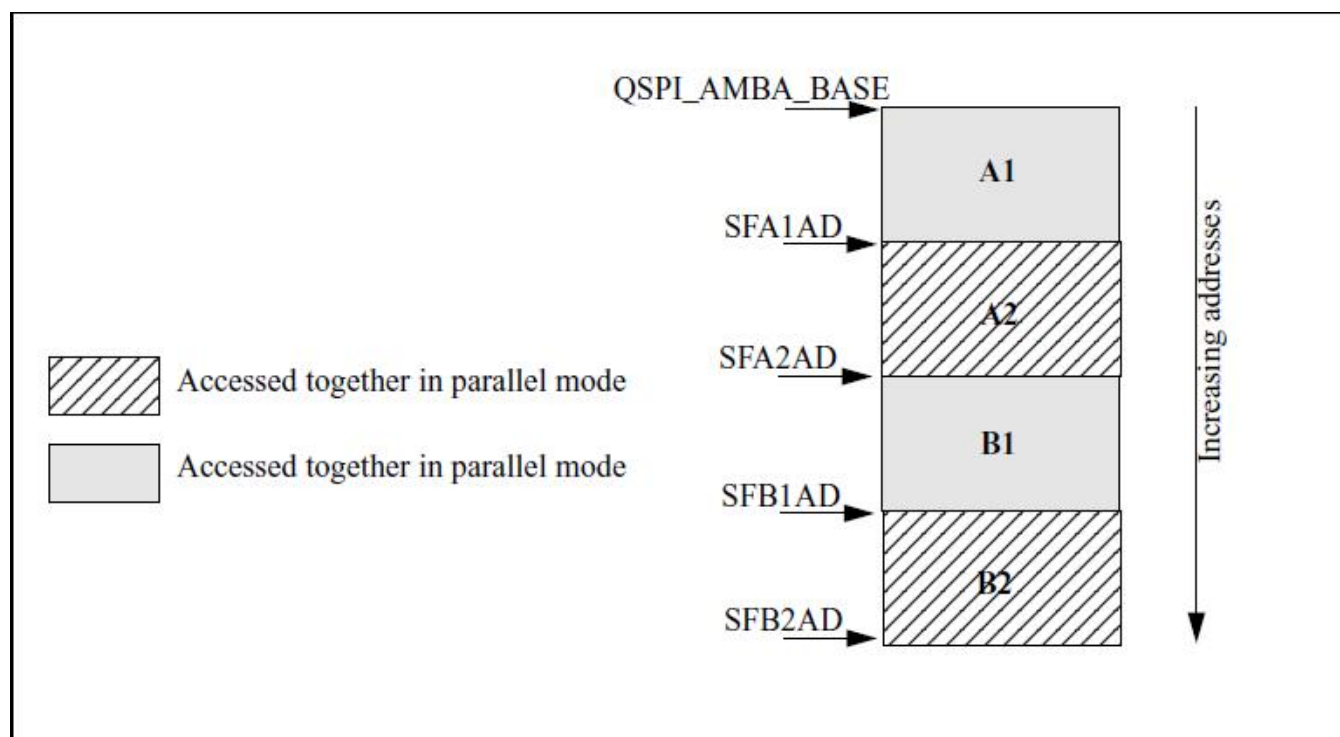


Figure 10-19. Flash addressing

An example programming for parallel mode access is given below (flash sizes are assumed to be 256MB):

- QSPI_AMBA_BASE - 0x10000000
- QSPI_SFA1AD[TPADA1] - 0x20000000
- QSPI_SFA2AD[TPADA2] - 0x30000000
- QSPI_SFB1AD[TPADB1] - 0x40000000
- QSPI_SFB2AD[TPADB2] - 0x50000000

In order to access the first location of A1/B1 pair, the incoming address should be 0x10000000. QSPI_AMBA_BASE is subtracted from this address and the result is divided by two. Therefore, address provided to flash A1 and B1

$$\text{Flash Address} = (\text{Memory mapped address} - \text{QSPI_AMBA_BASE})/2$$

For Memory Mapped address:

- 0x10000000, flash address: 0x0 (Or, the first address of flash A1 and B1)
- 0x10000004, flash address: 0x2
- 0x10000008, flash address: 0x4 etc.

Similarly, in order to access the first location of A2/B2 pair, the incoming address should be 0x30000000.

$$\text{Flash Address} = (\text{Memory mapped address} - \text{SFA2AD})/2$$

For Memory Mapped address:

- 0x30000000, flash address: 0x0 (Or, the first address of flash A2 and B2)
- 0x30000004, flash address: 0x2
- 0x30000008, flash address: 0x4 etc.

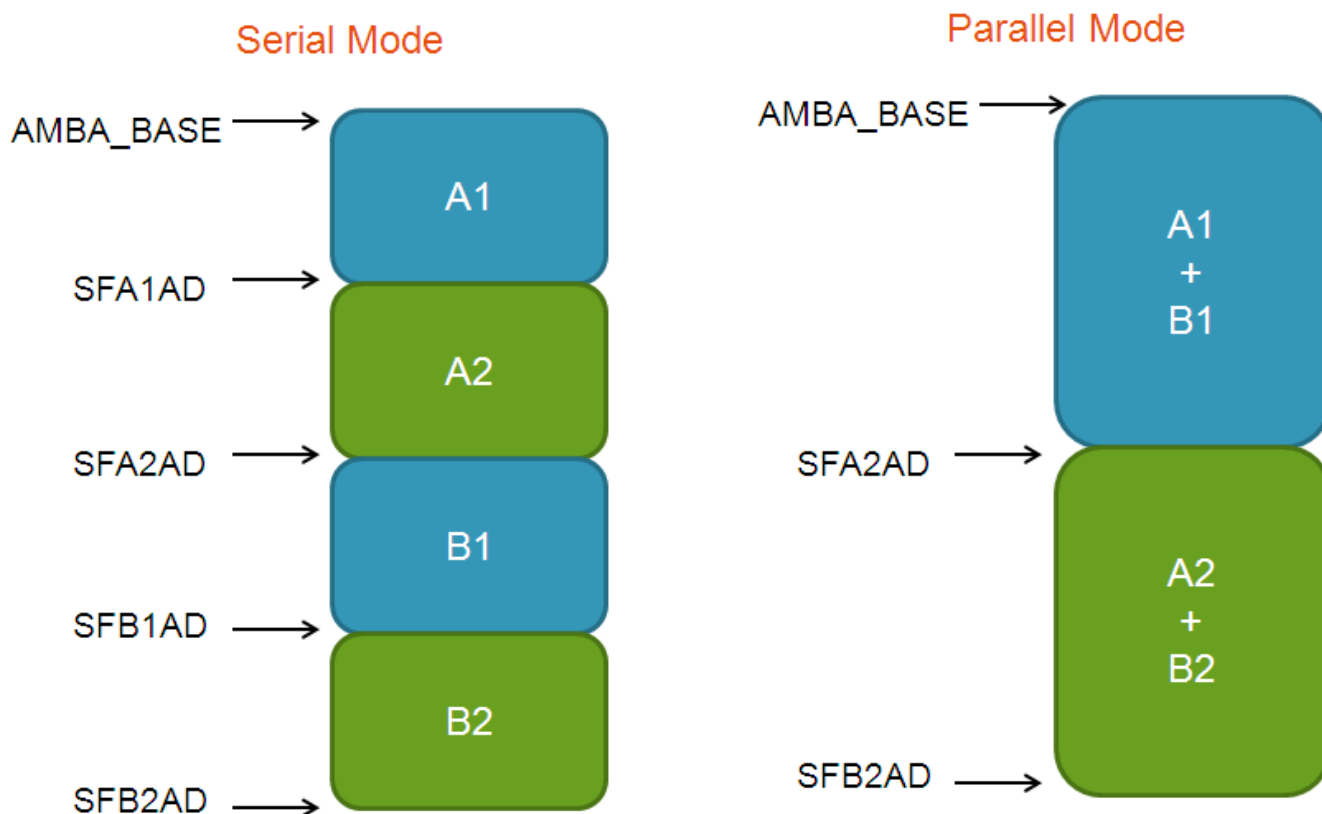


Figure 10-20. Memory map - Serial and Parallel

10.2.6 Byte Ordering - Endianness

QuadSPI provides support for swapping the flash read/write data based on the configuration of the QSPI_MCR[END_CFG]. By default the data is always returned in 64 bit LE format on the AHB bus and 32 bit LE format on the IPS interface when read via the RX buffer and written in 32 bit LE format when written via the TX buffer.

The table(QSPI_MCR[END_CFG]) below shows the complete bit ordering. BE signifies Big Endian which means the high order bits of the associated data vectors are associated with low order address positions. LE signifies Little Endian which means the lower order bits of the associated data vectors are associated with low order address positions. Refer to figure [Figure 10-18](#)

Table 10-23. QSPI_MCR[END_CFG]

00	64 bit BE
01	32 bit LE
10	32 bit BE
11	64 bit LE

The tables below (Byte ordering configuration in AHB) and (Byte ordering configuration in IPS) show how this configuration is implemented in QSPI AHB and IPS interfaces respectively. B in the table signifies Byte and the index 1-8 refers to the byte position i.e. 1 refer to bits[7:0], 8 refer to bits[63:56] and so on.

Table 10-24. Byte ordering configuration in AHB

64 bit BE	B1	B2	B3	B4	B5	B6	B7	B8
64 bit LE	B8	B7	B6	B5	B4	B3	B2	B1
32 bit BE	B5	B6	B7	B8	B1	B2	B3	B4
32 bit LE	B4	B3	B2	B1	B8	B7	B6	B5

Table 10-25. Byte ordering configuration in IPS

32BE	B1	B2	B3	B4
32LE	B4	B3	B2	B1

The examples below show the byte ordering in 64 bit BE configuration for AHB Buffer and 32 bit BE for TX/RX Buffer:

10.2.6.1 Programming Flash Data

CPU write instructions to the QSPI_TBDR register like

- Write QSPI_TBDR -> 0x01_02_03_04
- Write QSPI_TBDR -> 0x05_06_07_08

result in the following content of the TX Buffer:

Table 10-26. Example of QuadSPI TX Buffer

TX Buffer Entry	Content
0	32'h01_02_03_04
1	32'h05_06_07_08

Programming the TX Buffer into the external serial flash device results in the following byte order to be sent to the serial flash:

- 01...02...03...04...05...06...07...08

10.2.6.2 Reading Flash Data into the RX Buffer

Reading the content from the same address provides the following sequence of bytes, identical to the write case:

- 01...02...03...04...05...06...07...08

This results in the RX Buffer filled with:

Table 10-27. Resulting RX Buffer Content

RX Buffer Entry	Content
0	32'h01_02_03_04
1	32'h05_06_07_08

10.2.6.2.1 Readout of the RX Buffer via QSPI_RBDRn

The RX Buffer content appears at CPU read access via the Peripheral bus interface in the following order:

- Read QSPI_RBDR0 <- 0x01_02_03_04
- Read QSPI_RBDR1 <- 0x05_06_07_08

10.2.6.2.2 Readout of the RX Buffer via ARDBn

The RX Buffer content appears at read access on the AMBA AHB interface at the QuadSPI module boundary:

- (1a): 32 Bit Access: Read QSPI_ARDB0 <- 0x01_02_03_04

- (2a): 32 Bit Access: Read QSPI_ARDB1 <- 0x05_06_07_08
- (1b/2b): 64 Bit Access: Read QSPI_ARDB0 <- 0x01_02_03_04_05_06_07_08

10.2.6.3 Reading Flash Data into the AHB Buffer

Reading the content from the same address as it was written to provides the following sequence of bytes, identical to the write case:

- 01...02...03...04...05...06...07...08

This results in the AHB Buffer filled with:

Table 10-28. Resulting AHB Buffer Content

AHB Buffer Entry	Content
0	64'h01_02_03_04_05_06_07_08

10.2.6.3.1 Readout of the AHB Buffer via Memory Mapped Read

The AHB Buffer content appears at read access on the AMBA AHB interface at the QuadSPI module boundary:

- (1a): 32 Bit Read Access: <- 0x01_02_03_04
- (2a): 32 Bit Read Access: <- 0x05_06_07_08
- (1/2): 64 Bit Read Access: <- 0x01_02_03_04_05_06_07_08

10.2.7 Serial Flash Devices

Several different vendors make flash devices with a QuadSPI interface. At present there is no set standard for the QuadSPI instruction set. Most common commands currently have the same instruction code for all vendors, however some commands are unique to specific vendors. Some example sequences are provided below.

10.2.7.1 Example Sequences

This section provides the example sequences of the QuadSPI module.

Table 10-29. Exit 4 x I/O Read Enhance Performance Mode (XIP) (Macronix) and Read Status

INSTR	PAD	OPERAND	COMMENT
CMD	0x0	0xEB	4xIO Read Command
ADDR	0x2	0x18	24 Bit address to be send on 4 pads
MODE	0x2	0x00	2 mode cycles (exit XIP)
DUMMY	0x0	0x04	4 dummy cycles
READ	0x2	0x08	Read 64 bits
CMD	0x0	0x05	Read Status register
READ	0x0	0x01	Status register data
STOP	0x0	0x00	STOP, Instruction over

10.2.7.1.1 Fast Read Sequence (Macronix/Numonyx/Spansion/Winbond)

The following table shows the fast read sequence for Macronix/Numonyx/Spansion/Winbond flashes.

Table 10-30. Fast Read sequence

Instruction	Pad	Operand	Comment
CMD	0x0	0x0B	Fast Read command = 0x0B
ADDR	0x0	0x18	24 Addr bits to be sent on one pad
DUMMY	0x0	0x08	8 Dummy cycles
READ	0x0	0x04	Read 32 Bits on one pad
JMP_ON_CS	0x0	0x00	Jump to instruction 0 (CMD)

10.2.7.1.2 Fast Dual I/O DT Read Sequence (Macronix)

The following table shows the Fast Dual I/O DT read sequence for Macronix flashes.

Table 10-31. Fast Dual I/O DT Read sequence

Instruction	Pad	Operand	Comment
CMD	0x0	0xBD	Fast Dual I/O DT read command = 0xBD
ADDR_DDR	0x1	0x18	24 Addr bits to be sent on 2 pads in DDR mode
MODE4_DDR	0x1	0x00	P2=P0 or P3=P1 is necessary. Refer to Macronix datasheet for details. One clock cycle for mode.
DUMMY	0x0	0x06	6 Dummy cycles

Table continues on the next page...

Table 10-31. Fast Dual I/O DT Read sequence (continued)

Instruction	Pad	Operand	Comment
READ_DDR	0x1	0x04	Read 32 Bits on 2 pads in DDR mode
JMP_ON_CS	0x0	0x00	Jump to instruction 0 (CMD)

10.2.7.1.3 Fast Read Quad Output (Winbond)

The following table shows the Fast read quad output sequence for Winbond memories

Table 10-32. Fast Read Quad output sequence

Instruction	Pad	Operand	Comment
CMD	0x0	0x6B	Fast read quad output command = 0x6B
ADDR	0x0	0x18	24 Addr bits to be sent on 1 pad
DUMMY	0x0	0x08	8 Dummy cycles
READ	0x2	0x04	Read 32 Bits on 4 pads
JMP_ON_CS	0x0	0x00	Jump to instruction 0 (CMD)

10.2.7.1.4 4 x I/O Read Enhance Performance Mode (XIP) (Macronix)

The following table shows the 4 x I/O Read Enhance Performance Mode for Macronix flashes. The enhanced performance mode is also known as XIP mode.

Table 10-33. Fast Read Quad output sequence

Instruction	Pad	Operand	Comment
CMD	0x0	0xEB	4xI/O Read command = 0xEB
ADDR	0x2	0x18	24 Addr bits to be sent on 4 pads
MODE	0x2	0xA5	2 mode cycles
DUMMY	0x0	0x04	4 Dummy cycles
READ	0x2	0x04	Read 32 Bits on 4 pads
JMP_ON_CS	0x0	0x01	Jump to instruction 1 (ADDR)

When in XIP mode the software should ensure that all the flashes connected to the controller are in XIP mode. As a part of initializing the controller, all the flashes may be enabled with XIP by carrying out dummy reads.

10.2.7.1.5 Dual Command Page Program (Numonyx)

The following table shows the Dual command page program sequence for Numonyx flashes.

Table 10-34. Dual Command Page Program sequence

Instruction	Pad	Operand	Comment
CMD	0x1	0x02	Dual command page program = 0x02 on 2 pads
ADDR	0x1	0x18	24 Addr bits to be sent on 2 pads
WRITE	0x1	0x20	Write 32 Bytes on 2 pads
STOP	0x0	0x00	STOP, Instruction over

10.2.7.1.6 Sector Erase (Macronix/Spansion/Numonyx)

The following table shows the Sector erase sequence for Macronix/Spansion/Numonyx flashes

Table 10-35. Sector Erase sequence

Instruction	Pad	Operand	Comment
CMD	0x0	0xD8	Sector erase command = 0xD8
ADDR	0x0	0x18	24 Addr bits to be sent on 1 pad
STOP	0x0	0x00	STOP, Instruction over

10.2.7.1.7 Read Status Register (Macronix/Spansion/Numonyx/Winbond)

The following table shows the Read status register sequence for Macronix/Spansion/Numonyx/Winbond flashes.

Table 10-36. Read Status Register Sequence

Instruction	Pad	Operand	Comment
CMD	0x0	0x05	Read status register command = 0x05
READ	0x0	0x01	Read status register data
STOP	0x0	0x00	STOP, Instruction over

10.2.7.2 Dual Die Flashes

Certain serial flash vendors provide dual-die packages which are essentially two devices (dies) stacked within the same package to increase the memory capacity of a single package. These two devices within a package share the same data and clock pins, but have individual Chip Selects. QuadSPI controller provides support for two dual-die packages to be connected simultaneously. The figure below shows the two dual-die packages and the naming conventions used in this document. For simplicity, the data pins are shown to be unidirectional.

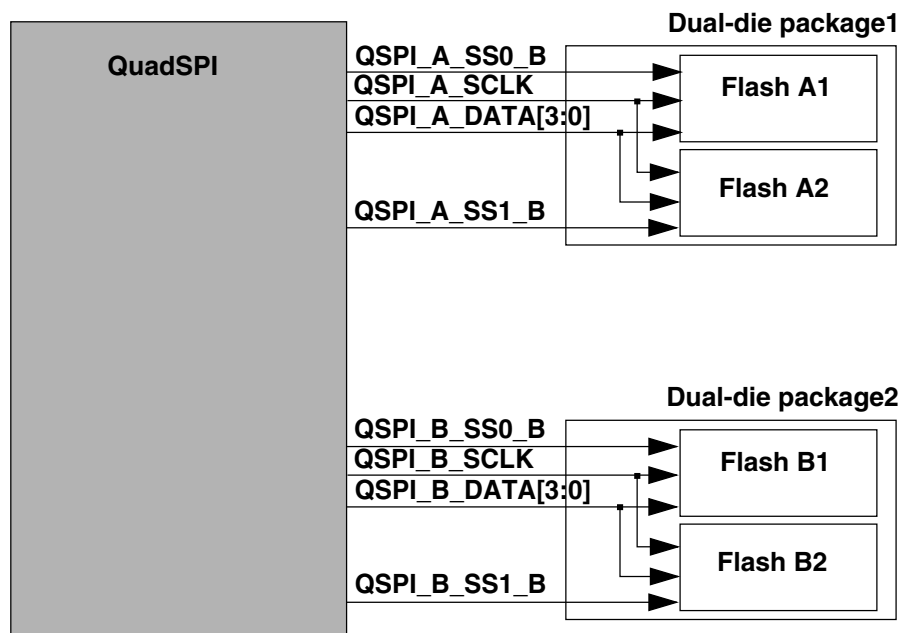


Figure 10-21. Dual-die support

Since the two devices within one package share the same i/o pads, they cannot function in parallel mode. Software should ensure that when QuadSPI is configured in parallel mode the two selected flash devices are from different dual-die packages.

10.2.7.3 Boot initialization sequence

The following are the recommended sequence of steps for booting from QuadSPI:

- System out of reset and flash available (300us)
- Clocks still at very low frequency. Clock tree configured, I/O pins configured. First request sent to QuadSPI for address 0x0 of flash.
- The reset command sequence in QuadSPI has 0x03 (basic read command) which is applicable to all flashes at < 50MHz serial flash clock
- The first few bytes of data is read from the flash which contains the following information:
 - The total sizes of all the flashes connected on board
 - Whether DDR mode supported
 - Frequency of DDR operation

- Continuous mode entry sequence
- 24bit or 32bit addressing (assuming 24bit for first accesses)
- All the serial flashes are configured
 - Quad Mode enabled
 - Dummy reads to enter into XIP
- QuadSPI is configured
 - Parallel enable set
 - LUT configured for highest performance reads
 - DDR mode enabled (if applicable)
 - Buffers configured
- Serial flash clock frequency increased.
- Boot reads happen in parallel, DDR enabled, quad output mode.

10.2.8 Sampling of Serial Flash Input Data

10.2.8.1 Internal Sampling of Serial Flash Input Data

Depending from the actual implementation there is a delay between the internal clocking in the QuadSPI module and the external serial flash device. Refer to the following figure for an overview of this scheme.

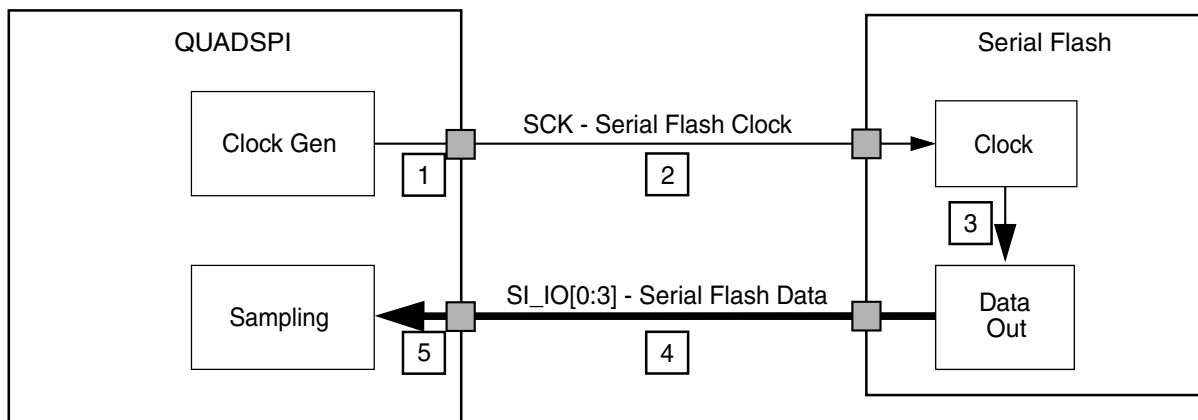


Figure 10-22. Serial Flash Sampling Clock Overview

Note

The arrival of the serial flash data in the sampling stage of the QuadSPI module are given in the following figure. Note that the amount of the total delay $t_{\text{Del,total}}$ is very specific to the characteristics of the actual implementation.

Note also that the serial flash device clock SCK is inverted with respect to the QuadSPI internal reference clock.

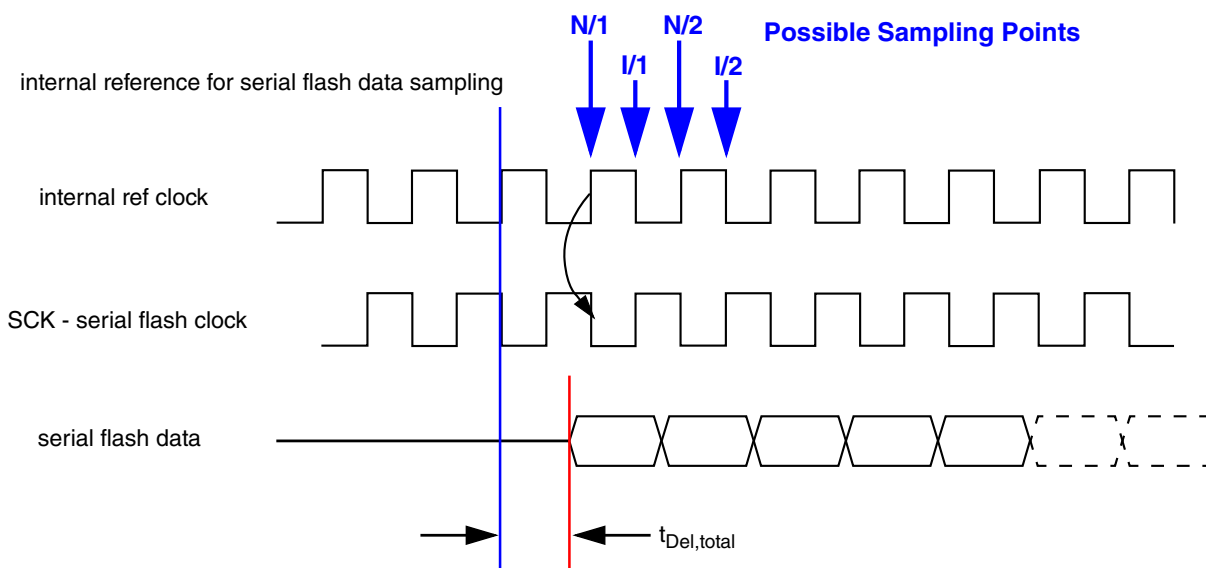


Figure 10-23. Serial Flash Sampling Clock Timing

The rising edge of the internal reference clock is taken as timing reference for the data output of the serial flash. After a time of $t_{\text{Del,total}}$ the data arrive at the internal sampling stage of the QuadSPI module.

According to the Serial Flash Sampling Clock Overview figure, the following parts of the delay chain contribute to $t_{\text{Del,total}}$:

1. Output delay of the serial flash clock output of the device containing the QuadSPI module
2. Wire delay of application/PCB from the device containing the QuadSPI module to the external serial flash device
3. Clock to data out delay of the external serial flash device, including input and output delays
4. Wire delay of application/PCB from the external serial flash device to the device containing the QuadSPI module
5. Input delay belonging to the data in input

The possible points in time for the sampling of the incoming data are denoted as N/1, I/1, N/2 and I/2 above. The sampling point relevant for the internal sampling is configured in the QSPI_SMPR register, refer to [Sampling Register \(QuadSPI_SMPR\)](#) for details. Note that the falling edges of the reference clock are not actually used, instead the inverted clock is used for sampling at these positions. The following table gives an overview of the available configurations for the commands running at regular (full) speed:

Table 10-37. Sampling Configuration

Sampling Point	Description	Delay [FSDLY] [HSDLY]	Phase [FSPHS] [HSPHS]	QSPI_SMPR for Full Speed Setting ¹
N/1	sampling with non-inverted clock, 1 sample delay	0	0	0x0000000x
I/1	sampling with inverted clock, 1 sample delay	0	1	0x0000002x
N/2	sampling with non-inverted clock, 2 samples delay	1	0	0x0000004x
I/2	sampling with inverted clock, 2 samples delay	1	1	0x0000006x

1. 'x' is not considered here

Depending from the actual delay and the serial flash clock frequency the appropriate sampling point can be chosen. The following remarks should be considered when selecting the appropriate setting:

- Theoretically there should be 2 settings possible to capture the correct data since the serial flash output is valid for 1 clock cycle, disregarding rise and fall times and timing uncertainties.
- Depending from the timing uncertainties it may turn out in actual applications that only one possible sample positions remains. This is subject to careful consideration depending from the actual implementation.
- The delay $t_{Del,total}$ is an absolute size to shift the point in time when the serial flash data get valid at the QuadSPI input.
- For decreasing frequency of the serial flash clock the distance between the edges increases. So for large differences in the frequency the required setting may change.
- For commands running at half of the regular serial flash clock (QSPI_SMPR[HSENA] bit set) the sampling point must be figured separately to allow for the compensation of the absolute shift in time with respect to the sample-relative setting in the QSPI_SMPR register.

10.2.8.2 DDR Mode

The increasing requirement of improved throughput has introduced the double data rate (DDR) mode. In DDR mode, the data is transferred on both the rising and falling edges of the serial flash clock. The DDR serial flashes sample as well as drive the data on both rising and falling edges of serial flash clock.

10.2.9 Serial Flash Data Input Timing

There are sampling modes for input flash data:

- Internal sampling - Input serial flash data is captured by internal serial clock. In SDR mode, serial data is sampled by serial clock 1x (ser_clk_1x) rise edge. In DDR mode, serial data is sampled by serial clock 4x (ser_clk_4x) rise edge.
- Loopback DQS sampling - Soc will output serial data strobe with internal serial clock. This serial data strobe would be loopback from pad and used to sample input serial data. In SDR mode, serial data is sampled by loopback DQS rise edge. In DDR mode, serial data is sampled by loopback DQS both edge

NOTE

DQS pad need to be set to force input for loopback.

- Flash DQS sampling - Some serial Flash device provide the data strobe output together with serial data. This strobe signal is used to sample input serial data directly. In SDR mode, serial data is sampled by Flash DQS rise edge. In DDR mode, serial data is sampled by Flash DQS both edge

Following table shows selection for sampling mode:

Sampling mode	DQS_EN	DQS_LOOPBACK_EN
Internal sampling	0	doesn't matter
Loopback DQS sampling	1	1
Flash DQS sampling	1	0

Serial flash data and clock path for input timing is shown in the following figure.

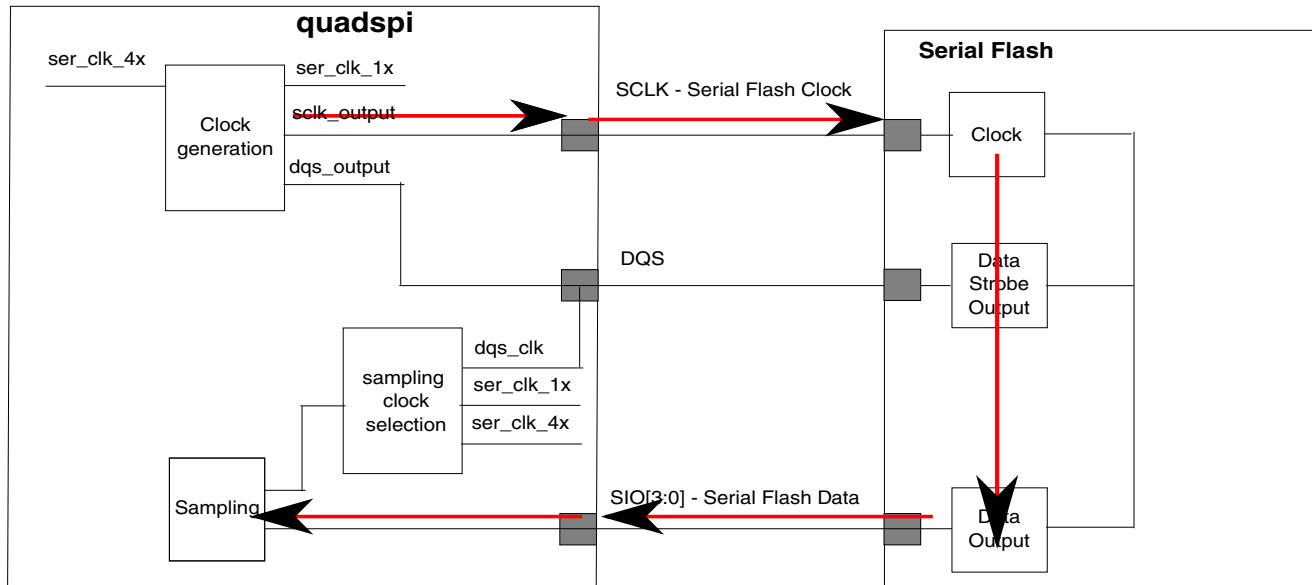


Figure 10-24. Serial flash data and clock path

NOTE

The red line is for data input path, the blue line is for DQS loopback path, and the purple line is for DQS input path from Serial Flash.

Total delay ($T_{total_delay_data}$) for serial flash data input is the sum of following delay:

1. Output delay of serial flash clock from internal serial clock to SCLK pad inside SOC
2. Wire delay of serial flash clock (SCLK) from SOC to external Serial Flash Device
3. Clock to Output Valid time of external Serial Flash Device
4. Wire delay of serial flash data (SIO) from external serial Flash Device to SOC
5. Input delay of serial flash data from SIO pad to internal sample register

Total delay ($T_{total_delay_loopback_dqs}$) for loopback DQS clock is the sum of following delay:

1. Output delay of DQS clock from internal serial clock to DQS pad inside SOC
2. Input delay of DQS clock from DQS pad to internal sample register

Total delay ($T_{total_delay_flash_dqs}$) for Flash DQS clock is the sum of following delay:

1. Output delay of serial flash clock from internal serial clock to SCLK pad inside SOC
2. Wire delay of serial flash clock (SCLK) from SOC to external Serial Flash Device
3. Clock to DQS Output time of external Serial Flash Device
4. Wire delay of serial flash data strobe (DQS) from external serial Flash Device to SOC
5. Input delay of DQS clock from DQS pad to internal sample register

10.2.9.1 Input timing in SDR mode with internal sampling

Input Timing diagram in SDR mode with internal sampling is show in the figure below.

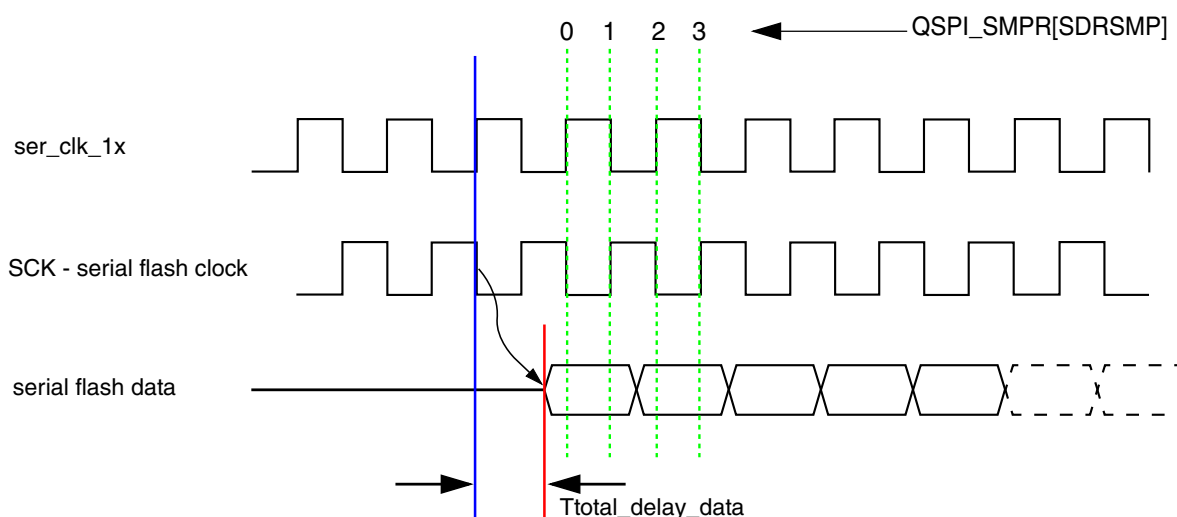


Figure 10-25. Internal sample SDR

There are four sample points for this sampling mode, which is determined by register field QSPI_SMPR.SDRSMP.

Sampling point need to be select correctly to meet both Setup and Hold timing for internal sample registers:

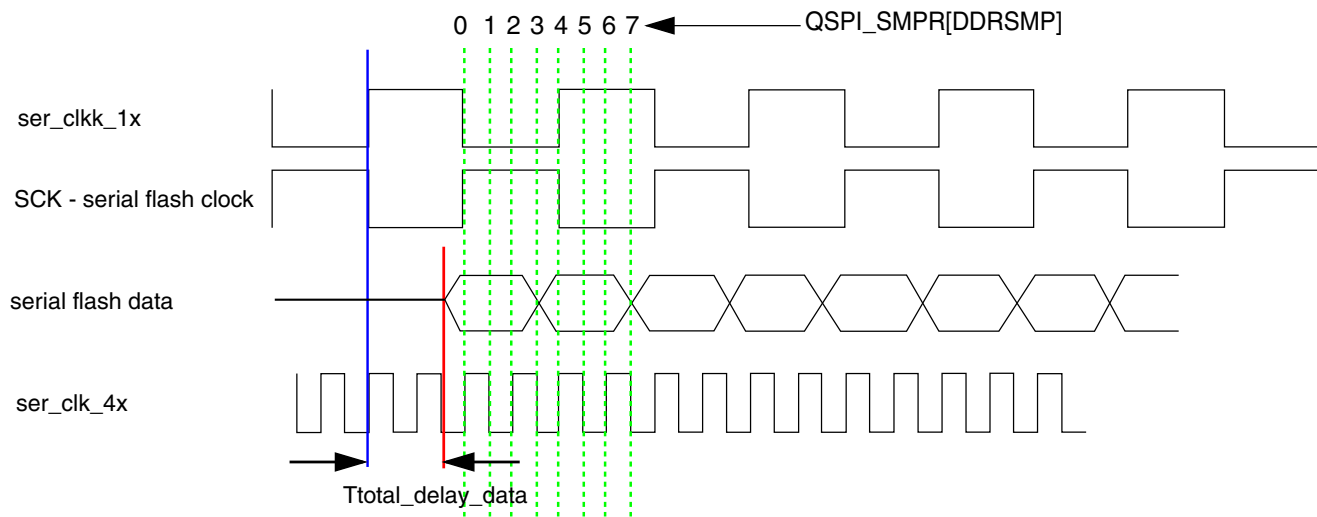
- For sample point N, Setup requirement is: $T_{\text{cycle}} * (N+2)/2 > T_{\text{total_delay_data,max}}$
- For sample point N, Hold requirement is: $T_{\text{total_delay_data,min}} > T_{\text{cycle}} * N/2$

NOTE

T_{cycle} is the cycle of `ser_clk_1x`. $T_{\text{total_delay_data,max}}$ is maximum delay of serial data input path. $T_{\text{total_delay_data,min}}$ is the minimum delay of serial data input path. $N=0,1,2,3$

10.2.9.2 Input timing in DDR mode with internal sampling

Input Timing diagram in SDR mode with internal sampling is show in the following figure.



There are 8 sample points for this sampling mode, which is determined by register field DDRSMP.

Sampling point need to be select correctly to meet both Setup and Hold timing for internal sample registers:

- For sample point N, Setup requirement is: $T_{\text{cycle}} * (N+2)/8 > T_{\text{total_delay_data,max}}$
- For sample point N, Hold requirement is: $T_{\text{total_delay_data,min}} > T_{\text{cycle}} * N/8$

10.2.9.3 Input timing in SDR mode with loopback DQS sampling

Input Timing diagram in SDR mode with internal sampling is show in the following figure.

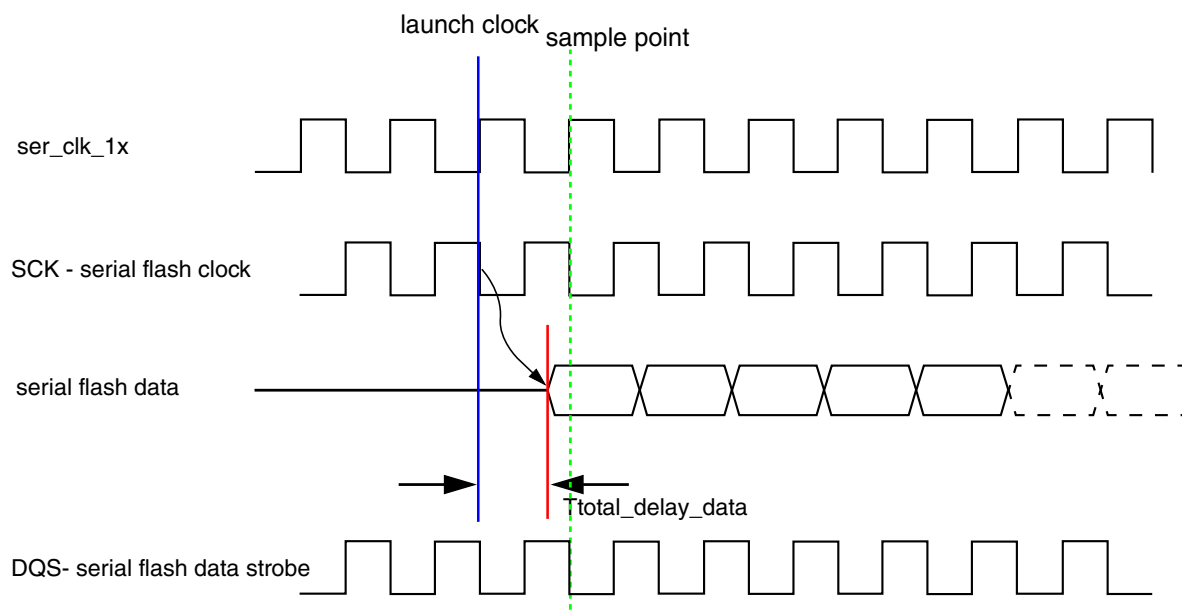


Figure 10-26. Input timing in SDR mode with loopback DQS sampling

In SDR mode and loopback sampling mode, DQS_PHASE_EN should be set 1.

For this sample point, the Setup requirement is: $T_{\text{cycle}} > \max(T_{\text{total_delay_data}} - T_{\text{total_delay_loopback_dqs}})$. The Hold requirement is: $\min(T_{\text{total_delay_data}} - T_{\text{total_delay_loopback_dqs}}) > 0$.

10.2.9.4 Input timing in DDR mode with loopback DQS sampling

Input Timing diagram in DDR mode with internal sampling is shown in the following figure.

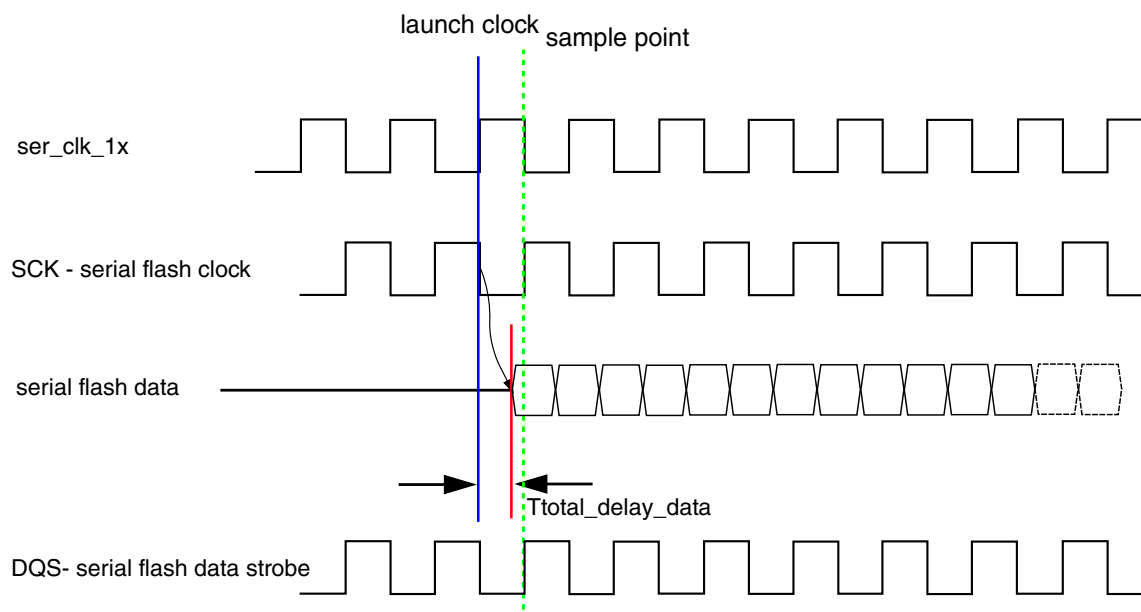


Figure 10-27. Input timing in DDR mode with loopback DQS sampling

In DDR mode and loopback sampling mode, DQS_PHASE_EN should be set 0.

For this sample point, the Setup requirement is: $T_{cycle} > \max(T_{total_delay_data} - T_{total_delay_loopback_dqs})$. The Hold requirement is: $\min(T_{total_delay_data} - T_{total_delay_loopback_dqs}) > 0$.

10.2.9.5 Input timing in SDR mode with flash DQS sampling

Input Timing diagram in SDR mode with internal sampling is show in the following figure.

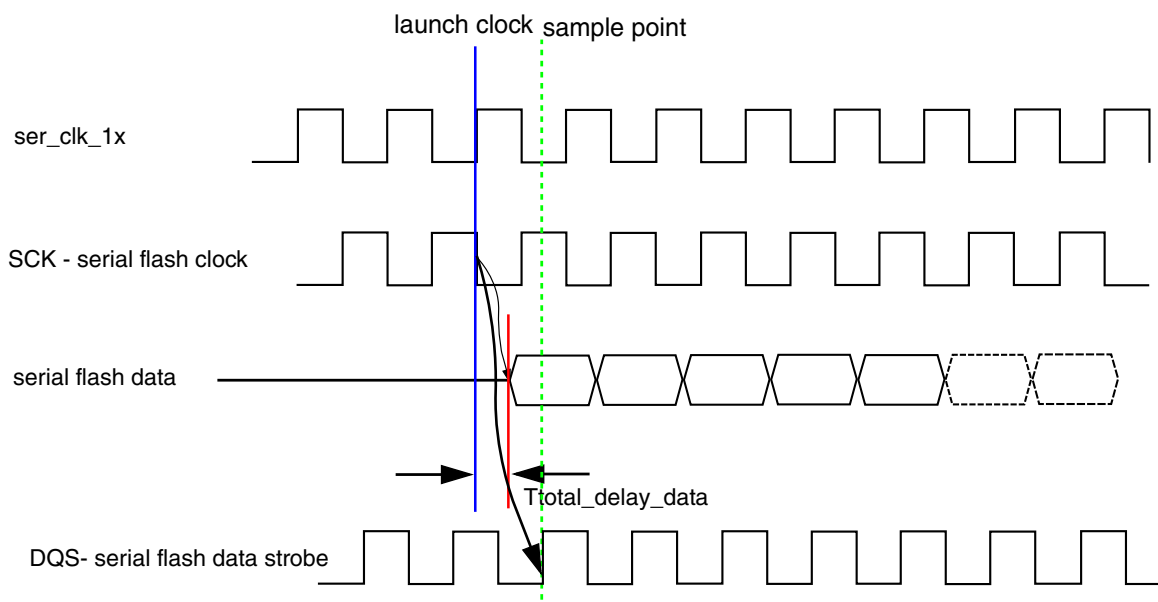


Figure 10-28. Input timing in SDR mode with flash DQS sampling

In this sampling mode, there will be a setup/hold requirement on Flash serial Data and Flash serial Data Strobe. This value will be specified in the data sheet.

10.2.9.6 Input timing in DDR mode with flash DQS sampling

Input Timing diagram in DDR mode with internal sampling is shown in the following figure.

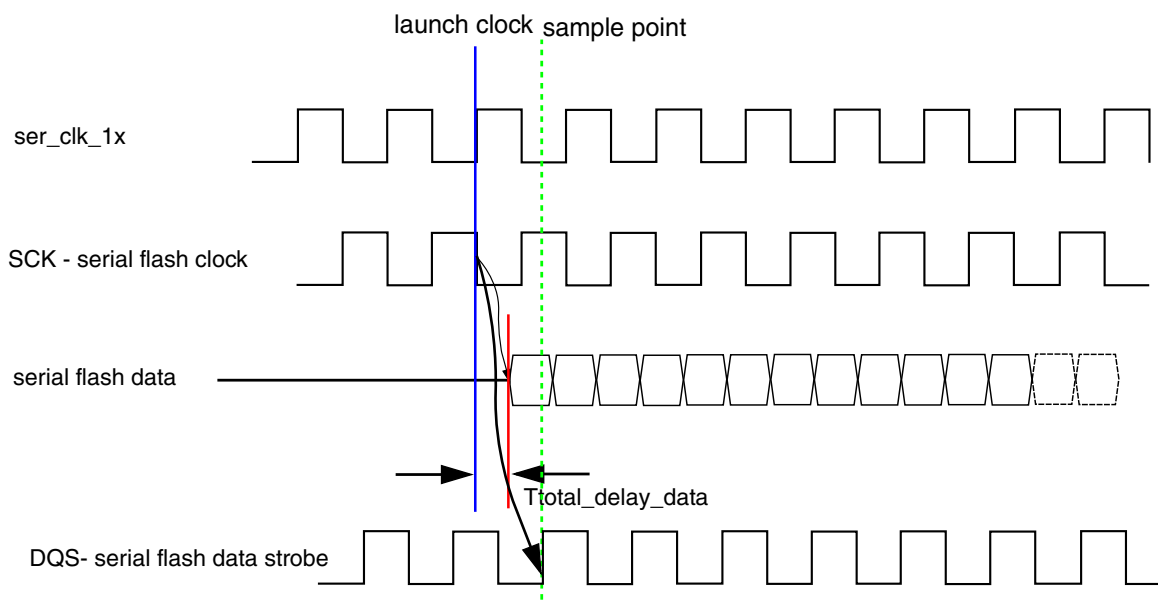


Figure 10-29. Input timing in DDR mode with flash DQS sampling

In this sampling mode, there will be a setup/hold requirement on Flash serial Data and Flash serial Data Strobe. This value will be specified in the data sheet.

10.2.9.7 Data Strobe Signal functionality

Some external serial flashes provide the data strobe (DQS/RDS) output which is fed directly to the QuadSPI module. The strobe (DQS/RDS) signal needs to be delayed to have the edges aligned to the data valid period. QuadSPI internally samples the incoming data at posedge of the strobe signal for SDR and on both the edges of the strobe signal for DDR. Refer to the figure for more detail.

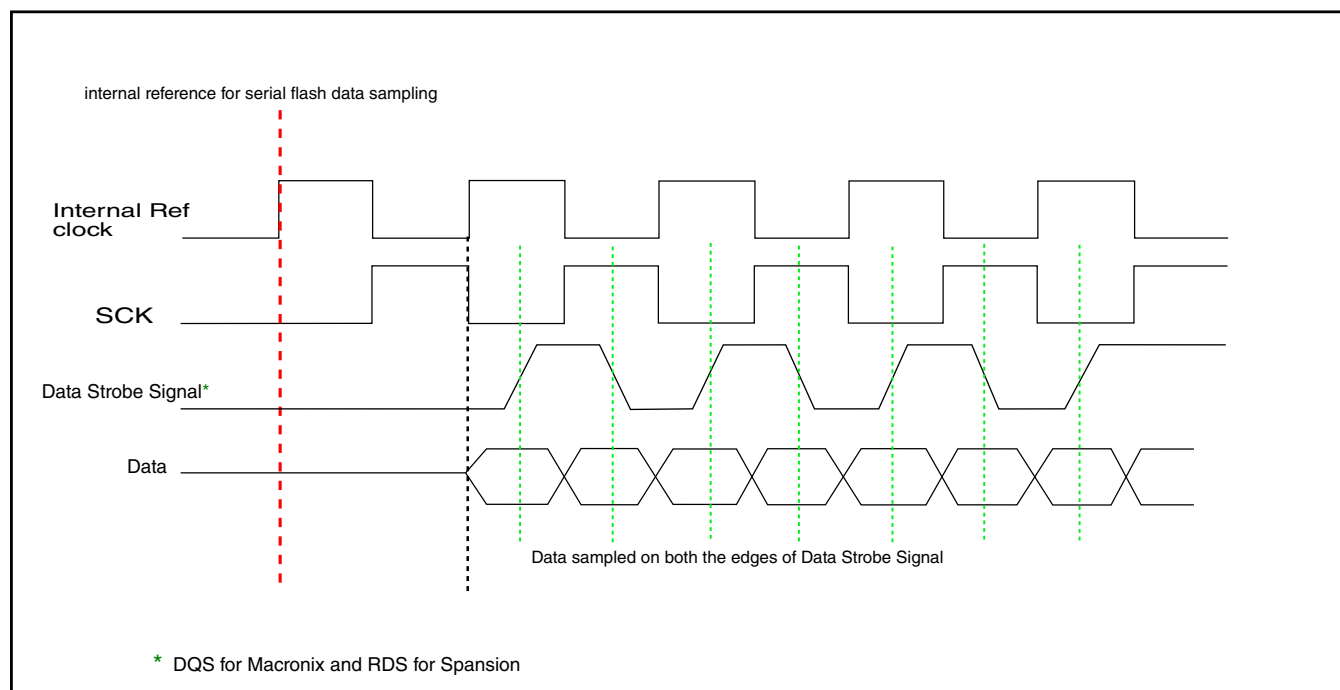


Figure 10-30. Data strobe signal functionality

10.2.10 Output timing in SDR mode

Output timing diagram in SDR mode is show in the following figure.

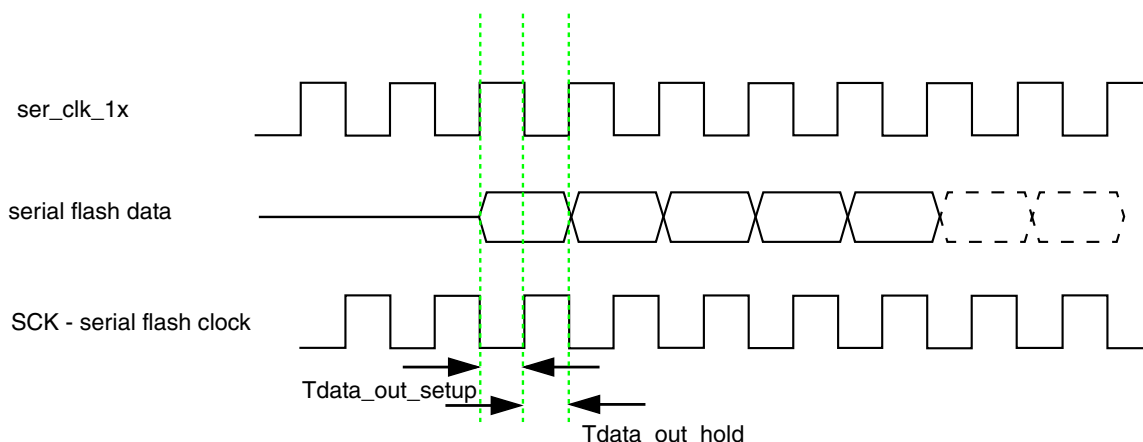


Figure 10-31. Output timing in SDR mode

In SDR mode, quadspi output serial data with internal serial clock rise edge 1x (`ser_clk_1x`). Flash Device has requirement on Data and Clock setup and hold timing.

10.2.11 Output timing in DDR mode

Output Timing diagram in DDR mode is show in the following figure.

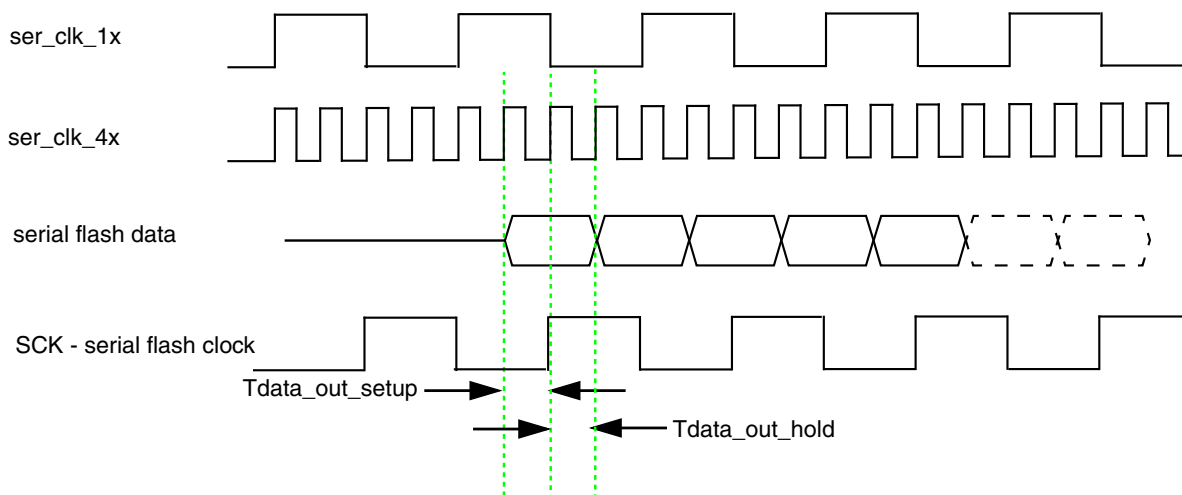


Figure 10-32. Output timing in DDR mode

In DDR mode, quadspi output serial data with internal serial clock both edge 1x (`ser_clk_1x`) and then delay one `ser_clk_4x` cycle for hold timing.

NOTE

TX_DDR_DELAY_EN should be set to 1 for DDR mode.

10.2.12 AHB RX Data Buffer (QSPI_ARDB0 to QSPI_ARDB31)

10.2.12.1 AHB RX Data Buffer (QSPI_ARDB0 to QSPI_ARDB31)

NOTE

See the System Memory map in this document for the base address of the QSPI AHB RX Data Buffer.

memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
0	AHB RX Data Buffer register (ARDB0)	32	R/W	0000_0000h	10.2.12.1.1/2626
4	AHB RX Data Buffer register (ARDB1)	32	R/W	0000_0000h	10.2.12.1.1/2626
8	AHB RX Data Buffer register (ARDB2)	32	R/W	0000_0000h	10.2.12.1.1/2626
C	AHB RX Data Buffer register (ARDB3)	32	R/W	0000_0000h	10.2.12.1.1/2626
10	AHB RX Data Buffer register (ARDB4)	32	R/W	0000_0000h	10.2.12.1.1/2626
14	AHB RX Data Buffer register (ARDB5)	32	R/W	0000_0000h	10.2.12.1.1/2626
18	AHB RX Data Buffer register (ARDB6)	32	R/W	0000_0000h	10.2.12.1.1/2626
1C	AHB RX Data Buffer register (ARDB7)	32	R/W	0000_0000h	10.2.12.1.1/2626
20	AHB RX Data Buffer register (ARDB8)	32	R/W	0000_0000h	10.2.12.1.1/2626
24	AHB RX Data Buffer register (ARDB9)	32	R/W	0000_0000h	10.2.12.1.1/2626
28	AHB RX Data Buffer register (ARDB10)	32	R/W	0000_0000h	10.2.12.1.1/2626
2C	AHB RX Data Buffer register (ARDB11)	32	R/W	0000_0000h	10.2.12.1.1/2626
30	AHB RX Data Buffer register (ARDB12)	32	R/W	0000_0000h	10.2.12.1.1/2626
34	AHB RX Data Buffer register (ARDB13)	32	R/W	0000_0000h	10.2.12.1.1/2626

Table continues on the next page...

memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
38	AHB RX Data Buffer register (ARDB14)	32	R/W	0000_0000h	10.2.12.1.1/2626
3C	AHB RX Data Buffer register (ARDB15)	32	R/W	0000_0000h	10.2.12.1.1/2626
40	AHB RX Data Buffer register (ARDB16)	32	R/W	0000_0000h	10.2.12.1.1/2626
44	AHB RX Data Buffer register (ARDB17)	32	R/W	0000_0000h	10.2.12.1.1/2626
48	AHB RX Data Buffer register (ARDB18)	32	R/W	0000_0000h	10.2.12.1.1/2626
4C	AHB RX Data Buffer register (ARDB19)	32	R/W	0000_0000h	10.2.12.1.1/2626
50	AHB RX Data Buffer register (ARDB20)	32	R/W	0000_0000h	10.2.12.1.1/2626
54	AHB RX Data Buffer register (ARDB21)	32	R/W	0000_0000h	10.2.12.1.1/2626
58	AHB RX Data Buffer register (ARDB22)	32	R/W	0000_0000h	10.2.12.1.1/2626
5C	AHB RX Data Buffer register (ARDB23)	32	R/W	0000_0000h	10.2.12.1.1/2626
60	AHB RX Data Buffer register (ARDB24)	32	R/W	0000_0000h	10.2.12.1.1/2626
64	AHB RX Data Buffer register (ARDB25)	32	R/W	0000_0000h	10.2.12.1.1/2626
68	AHB RX Data Buffer register (ARDB26)	32	R/W	0000_0000h	10.2.12.1.1/2626
6C	AHB RX Data Buffer register (ARDB27)	32	R/W	0000_0000h	10.2.12.1.1/2626
70	AHB RX Data Buffer register (ARDB28)	32	R/W	0000_0000h	10.2.12.1.1/2626
74	AHB RX Data Buffer register (ARDB29)	32	R/W	0000_0000h	10.2.12.1.1/2626
78	AHB RX Data Buffer register (ARDB30)	32	R/W	0000_0000h	10.2.12.1.1/2626
7C	AHB RX Data Buffer register (ARDB31)	32	R/W	0000_0000h	10.2.12.1.1/2626

10.2.12.1.1 AHB RX Data Buffer register (ARDB_n)

The AHB RX Data Buffer register 0 to 31 can be used to read the buffer content of the RX Buffer from successive addresses. QSPI_ARDB0 corresponds to the RX Buffer register entry corresponding to the current value of the read pointer with increasing order.

The increment of the read pointer depends from the access scheme (DMA or flag-driven). Refer to "Data Transfer from the QuadSPI Module Internal Buffers" section in [Flash Read](#) section, RX Buffer, data read via register interface and AHB read, for the description of successive accesses to the RX Buffer content. Refer also to [Byte Ordering of Serial Flash Read Data](#) for the byte ordering scheme.

Valid address range accessible in the QSPI_ARDBn range depends from the number of RX Buffer entries implemented and from the number of valid buffer entries available in the RX Buffer.

- Example 1, RX Buffer filled completely with 32 words: In this case the address range for valid read access extends from QSPI_ARDB0 to QSPI_ARDB31.
- Example 2, RX Buffer filled with 5 valid words, RX Buffer fill level QSPI_RBSR[RDBFL] is 5. In this case an access to QSPI_ARDB4 provides the last valid entry.

Address: 0h base + 0h offset + (4d × i), where i=0d to 31d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ARXD																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

ARDBn field descriptions

Field	Description
ARXD	ARDB provided RX Buffer Data. Byte order (endianness) is identical to the RX Buffer Data Registers.

10.2.13 Peripheral Bus Register Descriptions

This section provides the peripheral bus register information of the QuadSPI module.

This section provides the memory map and register definitions of the QuadSPI module.

QuadSPI memory map

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
30BB_0000	Module Configuration Register (QuadSPI_MCR)	32	R/W	000F_4000h	10.2.13.1/2634
30BB_0008	IP Configuration Register (QuadSPI_IPCR)	32	R/W	0000_0000h	10.2.13.2/2637

Table continues on the next page...

QuadSPI memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/page
30BB_000C	Flash Configuration Register (QuadSPI_FLSHCR)	32	R/W	0000_0303h	10.2.13.3/2637
30BB_0010	Buffer0 Configuration Register (QuadSPI_BUF0CR)	32	R/W	0000_0000h	10.2.13.4/2638
30BB_0014	Buffer1 Configuration Register (QuadSPI_BUF1CR)	32	R/W	0000_0000h	10.2.13.5/2639
30BB_0018	Buffer2 Configuration Register (QuadSPI_BUF2CR)	32	R/W	0000_0000h	10.2.13.6/2640
30BB_001C	Buffer3 Configuration Register (QuadSPI_BUF3CR)	32	R/W	See section	10.2.13.7/2641
30BB_0020	Buffer Generic Configuration Register (QuadSPI_BFGENCR)	32	R/W	0000_0000h	10.2.13.8/2642
30BB_0030	Buffer0 Top Index Register (QuadSPI_BUF0IND)	32	R/W	0000_0000h	10.2.13.9/2643
30BB_0034	Buffer1 Top Index Register (QuadSPI_BUF1IND)	32	R/W	0000_0000h	10.2.13.10/2643
30BB_0038	Buffer2 Top Index Register (QuadSPI_BUF2IND)	32	R/W	0000_0000h	10.2.13.11/2644
30BB_0100	Serial Flash Address Register (QuadSPI_SFAR)	32	R/W	0000_0000h	10.2.13.12/2645
30BB_0108	Sampling Register (QuadSPI_SMPR)	32	R/W	0000_0000h	10.2.13.13/2645
30BB_010C	RX Buffer Status Register (QuadSPI_RBSR)	32	R	0000_0000h	10.2.13.14/2646
30BB_0110	RX Buffer Control Register (QuadSPI_RBCT)	32	R/W	0000_0000h	10.2.13.15/2647
30BB_0150	TX Buffer Status Register (QuadSPI_TBSR)	32	R	0000_0000h	10.2.13.16/2648
30BB_0154	TX Buffer Data Register (QuadSPI_TBDR)	32	R/W	0000_0000h	10.2.13.17/2648
30BB_015C	Status Register (QuadSPI_SR)	32	R	0000_3800h	10.2.13.18/2650
30BB_0160	Flag Register (QuadSPI_FR)	32	w1c	0800_0000h	10.2.13.19/2653
30BB_0164	Interrupt and DMA Request Select and Enable Register (QuadSPI_RSER)	32	R/W	0000_0000h	10.2.13.20/2656
30BB_0168	Sequence Suspend Status Register (QuadSPI_SPNDST)	32	R	0000_0000h	10.2.13.21/2659
30BB_016C	Sequence Pointer Clear Register (QuadSPI_SPTRCLR)	32	R/W	0000_0000h	10.2.13.22/2661
30BB_0180	Serial Flash A1 Top Address (QuadSPI_SFA1AD)	32	R/W	0000_0000h	10.2.13.23/2661
30BB_0184	Serial Flash A2 Top Address (QuadSPI_SFA2AD)	32	R/W	0000_0000h	10.2.13.24/2662

Table continues on the next page...

QuadSPI memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
30BB_0188	Serial Flash B1Top Address (QuadSPI_SFB1AD)	32	R/W	0000_0000h	10.2.13.25/ 2662
30BB_018C	Serial Flash B2Top Address (QuadSPI_SFB2AD)	32	R/W	0000_0000h	10.2.13.26/ 2663
30BB_0200	RX Buffer Data Register (QuadSPI_RBDR0)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0204	RX Buffer Data Register (QuadSPI_RBDR1)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0208	RX Buffer Data Register (QuadSPI_RBDR2)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_020C	RX Buffer Data Register (QuadSPI_RBDR3)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0210	RX Buffer Data Register (QuadSPI_RBDR4)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0214	RX Buffer Data Register (QuadSPI_RBDR5)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0218	RX Buffer Data Register (QuadSPI_RBDR6)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_021C	RX Buffer Data Register (QuadSPI_RBDR7)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0220	RX Buffer Data Register (QuadSPI_RBDR8)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0224	RX Buffer Data Register (QuadSPI_RBDR9)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0228	RX Buffer Data Register (QuadSPI_RBDR10)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_022C	RX Buffer Data Register (QuadSPI_RBDR11)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0230	RX Buffer Data Register (QuadSPI_RBDR12)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0234	RX Buffer Data Register (QuadSPI_RBDR13)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0238	RX Buffer Data Register (QuadSPI_RBDR14)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_023C	RX Buffer Data Register (QuadSPI_RBDR15)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0240	RX Buffer Data Register (QuadSPI_RBDR16)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0244	RX Buffer Data Register (QuadSPI_RBDR17)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0248	RX Buffer Data Register (QuadSPI_RBDR18)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_024C	RX Buffer Data Register (QuadSPI_RBDR19)	32	R/W	0000_0000h	10.2.13.27/ 2663

Table continues on the next page...

QuadSPI memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
30BB_0250	RX Buffer Data Register (QuadSPI_RBDR20)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0254	RX Buffer Data Register (QuadSPI_RBDR21)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0258	RX Buffer Data Register (QuadSPI_RBDR22)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_025C	RX Buffer Data Register (QuadSPI_RBDR23)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0260	RX Buffer Data Register (QuadSPI_RBDR24)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0264	RX Buffer Data Register (QuadSPI_RBDR25)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0268	RX Buffer Data Register (QuadSPI_RBDR26)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_026C	RX Buffer Data Register (QuadSPI_RBDR27)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0270	RX Buffer Data Register (QuadSPI_RBDR28)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0274	RX Buffer Data Register (QuadSPI_RBDR29)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0278	RX Buffer Data Register (QuadSPI_RBDR30)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_027C	RX Buffer Data Register (QuadSPI_RBDR31)	32	R/W	0000_0000h	10.2.13.27/ 2663
30BB_0300	LUT Key Register (QuadSPI_LUTKEY)	32	R/W	5AF0_5AF0h	10.2.13.28/ 2664
30BB_0304	LUT Lock Configuration Register (QuadSPI_LCKCR)	32	R/W	0000_0002h	10.2.13.29/ 2665
30BB_0310	Look-up Table register (QuadSPI_LUT0)	32	R/W	0818_0403h	10.2.13.30/ 2666
30BB_0314	Look-up Table register (QuadSPI_LUT1)	32	R/W	2400_1C08h	10.2.13.31/ 2667
30BB_0318	Look-up Table register (QuadSPI_LUT2)	32	R/W	0000_0000h	10.2.13.32/ 2668
30BB_031C	Look-up Table register (QuadSPI_LUT3)	32	R/W	0000_0000h	10.2.13.32/ 2668
30BB_0320	Look-up Table register (QuadSPI_LUT4)	32	R/W	0000_0000h	10.2.13.32/ 2668
30BB_0324	Look-up Table register (QuadSPI_LUT5)	32	R/W	0000_0000h	10.2.13.32/ 2668
30BB_0328	Look-up Table register (QuadSPI_LUT6)	32	R/W	0000_0000h	10.2.13.32/ 2668
30BB_032C	Look-up Table register (QuadSPI_LUT7)	32	R/W	0000_0000h	10.2.13.32/ 2668

Table continues on the next page...

QuadSPI memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
30BB_0330	Look-up Table register (QuadSPI_LUT8)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0334	Look-up Table register (QuadSPI_LUT9)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0338	Look-up Table register (QuadSPI_LUT10)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_033C	Look-up Table register (QuadSPI_LUT11)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0340	Look-up Table register (QuadSPI_LUT12)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0344	Look-up Table register (QuadSPI_LUT13)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0348	Look-up Table register (QuadSPI_LUT14)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_034C	Look-up Table register (QuadSPI_LUT15)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0350	Look-up Table register (QuadSPI_LUT16)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0354	Look-up Table register (QuadSPI_LUT17)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0358	Look-up Table register (QuadSPI_LUT18)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_035C	Look-up Table register (QuadSPI_LUT19)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0360	Look-up Table register (QuadSPI_LUT20)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0364	Look-up Table register (QuadSPI_LUT21)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0368	Look-up Table register (QuadSPI_LUT22)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_036C	Look-up Table register (QuadSPI_LUT23)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0370	Look-up Table register (QuadSPI_LUT24)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0374	Look-up Table register (QuadSPI_LUT25)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0378	Look-up Table register (QuadSPI_LUT26)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_037C	Look-up Table register (QuadSPI_LUT27)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0380	Look-up Table register (QuadSPI_LUT28)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0384	Look-up Table register (QuadSPI_LUT29)	32	R/W	0000_0000h	10.2.13.32/2668

Table continues on the next page...

QuadSPI memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
30BB_0388	Look-up Table register (QuadSPI_LUT30)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_038C	Look-up Table register (QuadSPI_LUT31)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0390	Look-up Table register (QuadSPI_LUT32)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0394	Look-up Table register (QuadSPI_LUT33)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0398	Look-up Table register (QuadSPI_LUT34)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_039C	Look-up Table register (QuadSPI_LUT35)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03A0	Look-up Table register (QuadSPI_LUT36)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03A4	Look-up Table register (QuadSPI_LUT37)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03A8	Look-up Table register (QuadSPI_LUT38)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03AC	Look-up Table register (QuadSPI_LUT39)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03B0	Look-up Table register (QuadSPI_LUT40)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03B4	Look-up Table register (QuadSPI_LUT41)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03B8	Look-up Table register (QuadSPI_LUT42)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03BC	Look-up Table register (QuadSPI_LUT43)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03C0	Look-up Table register (QuadSPI_LUT44)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03C4	Look-up Table register (QuadSPI_LUT45)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03C8	Look-up Table register (QuadSPI_LUT46)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03CC	Look-up Table register (QuadSPI_LUT47)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03D0	Look-up Table register (QuadSPI_LUT48)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03D4	Look-up Table register (QuadSPI_LUT49)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03D8	Look-up Table register (QuadSPI_LUT50)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03DC	Look-up Table register (QuadSPI_LUT51)	32	R/W	0000_0000h	10.2.13.32/2668

Table continues on the next page...

QuadSPI memory map (continued)

Absolute address (hex)	Register name	Width (in bits)	Access	Reset value	Section/ page
30BB_03E0	Look-up Table register (QuadSPI_LUT52)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03E4	Look-up Table register (QuadSPI_LUT53)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03E8	Look-up Table register (QuadSPI_LUT54)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03EC	Look-up Table register (QuadSPI_LUT55)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03F0	Look-up Table register (QuadSPI_LUT56)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03F4	Look-up Table register (QuadSPI_LUT57)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03F8	Look-up Table register (QuadSPI_LUT58)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_03FC	Look-up Table register (QuadSPI_LUT59)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0400	Look-up Table register (QuadSPI_LUT60)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0404	Look-up Table register (QuadSPI_LUT61)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_0408	Look-up Table register (QuadSPI_LUT62)	32	R/W	0000_0000h	10.2.13.32/2668
30BB_040C	Look-up Table register (QuadSPI_LUT63)	32	R/W	0000_0000h	10.2.13.32/2668

10.2.13.1 Module Configuration Register (QuadSPI_MCR)

The QuadSPI_MCR holds configuration data associated with QuadSPI operation.

Write:

- *All other fields: Anytime*

Address: 30BB_0000h base + 0h offset = 30BB_0000h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved					DQS_PHASE_EN	DQS_LOOPBACK_EN	DQS_LOOPBACK_FROM_PAD	Reserved				Reserved			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
Reset	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	Reserved	MDIS	Reserved	Reserved	CLR_TXF	CLR_RXF	Reserved	Reserved	DDR_EN	DQS_EN	Reserved	Reserved	END_CFG	Reserved	SWRSTHD	SWRSTSD

QuadSPI_MCR field descriptions

Field	Description
31–27 Reserved	This field is reserved.
26 DQS_PHASE_EN	This bit controls internal DQS output phase. If DQS_EN and DQS_LOOPBACK_EN are both set to 1, this bit should be set in SDR mode, and cleared in DDR mode. If either DQS_EN or DQS_LOOPBACK_EN is set to 0, this bit is ignored.
25 DQS_LOOPBACK_EN	Quadspi will output serial data strobe signal which will be loopback from pad to sample input flash serial data. Please note pad should be force input for loopback. Quadspi will output serial data strobe signal which will be loopback from pad to sample input flash serial data. Please note pad should be force input for loopback. This bit is a don't care when DQS_EN is set to 0. 1'b1 DQS loopback sampling enabled 1'b0 DQS loopback sampling disabled
24 DQS_LOOPBACK_FROM_PAD	This bit should always be set to '1' when DQS_LOOPBACK_EN is set to '1'. When DQS_LOOPBACK_EN is set to '1', this bit is ignored.
23–20 Reserved	This field is reserved.
19–16 Reserved	This field is reserved. This field is reserved and should always be set to 0xF.
15 Reserved	This field is reserved. This field is reserved.
14 MDIS	Module Disable. The MDIS bit allows the clock to the non-memory mapped logic in the QuadSPI to be stopped, putting the QuadSPI in a software controlled power-saving state. Please refer to , for more information. 0 Enable QuadSPI clocks. 1 Allow external logic to disable QuadSPI clocks.
13–12 Reserved	This field is reserved.
11 CLR_TXF	Clear TX FIFO/Buffer. Invalidate the TX Buffer content. 0 No action. 1 Read and write pointers of the TX Buffer are reset to 0. QSPI_TBSR[TRCTR] is reset to 0.

Table continues on the next page...

QuadSPI_MCR field descriptions (continued)

Field	Description
10 CLR_RXF	Clear RX FIFO. Invalidate the RX Buffer. 0 No action. 1 Read and write pointers of the RX Buffer are reset to 0. QSPI_RBSR[RDBFL] is reset to 0.
9–8 Reserved	This field is reserved.
7 DDR_EN	DDR mode enable: 0 2x and 4x clocks are disabled for SDR instructions only 1 2x and 4x clocks are enabled supports both SDR and DDR instruction.
6 DQS_EN	DQS enable: This field is valid for both SDR and DDR mode. For more details Refer Data Strobe Signal Functionality 0 DQS disabled. 1 DQS enabled- When enabled, the incoming data is sampled on both the edges of DQS input when QSPI_MCR[DDR_EN] is set, else, on only one edge when QSPI_MCR[DDR_EN] is 0. The QSPI_SMPR[DDR_SMP] values are ignored.
5–4 Reserved	This field is reserved.
3–2 END_CFG	Defines the endianness of the QSPI module. For more details refer to Byte Ordering Endianness
1 SWRSTHD	Software reset for AHB domain NOTE: Please keep other fields value when write to SWRSTHD and SWRSTSD NOTE: These software reset don't reset register setting but only reset internal flip-flops in quadspi controller NOTE: To remove the reset, need to write 0 to SWRSTHD and SWRSTSD 0 No action 1 AHB domain flops are reset. Does not reset configuration registers. It is advisable to reset both the serial flash domain and AHB domain at the same time. Resetting only one domain might lead to side effects.
0 SWRSTSD	Software reset for Serial Flash domain NOTE: Please keep other fields value when write to SWRSTHD and SWRSTSD NOTE: These software reset don't reset register setting but only reset internal flip-flops in quadspi controller NOTE: To remove the reset, need to write 0 to SWRSTHD and SWRSTSD 0 No action 1 Serial Flash domain flops are reset. Does not reset configuration registers. It is advisable to reset both the serial flash domain and AHB domain at the same time. Resetting only one domain might lead to side effects.

10.2.13.2 IP Configuration Register (QuadSPI_IPCR)

The IP configuration register provides all the configuration required for an IP initiated command. An IP command can be triggered by writing in the SEQID field of this register. If the SEQID field is written successfully, a new command to the external serial flash is started as per the sequence pointed to by the SEQID field. Refer to [Normal Mode](#), for details about the command triggering and command execution.

Write:

- $QSPI_SR[IP_ACC]=0$

Address: 30BB_0000h base + 8h offset = 30BB_0008h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R	Reserved				SEQID				Reserved								PAR_EN
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	IDATSZ																
W																	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

QuadSPI_IPCR field descriptions

Field	Description
31–28 Reserved	This field is reserved.
27–24 SEQID	Points to a sequence in the Look-up-table. The SEQID defines the bits [6:2] of the LUT index. The bits [1:0] are always assumed to be 0. Refer to Look-up Table for more details. A write to this bit -field triggers a transaction on the serial flash interface.
23–17 Reserved	This field is reserved.
16 PAR_EN	When set, a transaction to two serial flash devices is triggered in parallel mode. Refer to Parallel Flash Mode for more details.
IDATSZ	IP data transfer size: Defines the data transfer size in bytes of the IP command.

10.2.13.3 Flash Configuration Register (QuadSPI_FLSHCR)

The Flash configuration register contains the flash device specific timings that must be met by the QuadSPI controller for the device to function correctly.

Write:

- $QSPI_SR[AHB_ACC] = 0$
- $QSPI_SR[IP_ACC] = 0$

Address: 30BB_0000h base + Ch offset = 30BB_000Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
R	Reserved															TDH		Reserved				TCSH				Reserved				TCSS			
W	Reserved															TDH		Reserved				TCSH				Reserved				TCSS			
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	1	1	

QuadSPI_FLSHCR field descriptions

Field	Description
31–18 Reserved	This field is reserved.
17–16 TDH	Serial flash Data In hold time: This helps in meeting the Data In Hold time requirement of a Flash. This is valid only in DDR mode. Refer to Data input hold requirement of Flash for details. NOTE: This field should be set to 01 in DDR mode (DDR_EN=1) and set to 00 in SDR mode (DDR_EN=0). Other value are reserved. 00 Data aligned with the posedge of Internal reference clock of QuadSPI 01 Data aligned with 2x serial flash half clock 10 Reserved 11 Reserved
15–12 Reserved	This field is reserved.
11–8 TCSH	Serial flash CS hold time in terms of serial flash clock cycles. NOTE: The actual delay between chip select assertion and clock fall edge is defined as: • 1 SCK cycle if TCSH is set 0 or 1 • N SCK cycle if TCSH is set N (N>1)
7–4 Reserved	This field is reserved. Reserved.
TCSS	Serial flash CS setup time in terms of serial flash clock cycles. NOTE: 1. The actual delay between chip select assertion and clock fall edge is defined as: • 0.5 SCK cycle if TCSS is set 0 or 1 • N+0.5 SCK cycle if TCSS is set N (N>1) 2. Any update to TCSS register bits is visible on the flash interface only from the second transaction following the update.

10.2.13.4 Buffer0 Configuration Register (QuadSPI_BUF0CR)

This register provides the configuration for any access to buffer0. An access is routed to buffer0 when the master port number of the incoming AHB request matches the MSTRID field of the BUF0CR. Any buffer "miss" leads to a serial flash transaction being triggered as per the sequence pointed to the SEQID field. Buffer0 may also be configured as a high priority buffer by setting the HP_EN field of this register.

Write:

- $QSPI_SR[AHB_ACC] = 0$

Address: 30BB_0000h base + 10h offset = 30BB_0010h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	HP_EN	Reserved														
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ADATSZ								Reserved				MSTRID			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

QuadSPI_BUF0CR field descriptions

Field	Description
31 HP_EN	High Priority Enable: When set, the master associated with this buffer is assigned a priority higher than the rest of the masters. An access by a high priority master will suspend any ongoing prefetch by another AHB master and will be serviced on high priority. Refer to Flexible AHB Buffers for details.
30–16 Reserved	This field is reserved.
15–8 ADATSZ	AHB data transfer size: Defines the data transfer size in 8 bytes of an AHB triggered access to serial flash. For example, a value of 0x2 will set transfer size to 16bytes. When ADATSZ = 0, the data size mentioned the sequence pointed to by the SEQID field overrides this value. SW should ensure that this transfer size is not greater than the size of this buffer.
7–4 Reserved	This field is reserved. Reserved.
MSTRID	Master ID: The ID of the AHB master associated with BUFFER0. Any AHB access with this master port number is routed to this buffer.

10.2.13.5 Buffer1 Configuration Register (QuadSPI_BUF1CR)

This register provides the configuration for any access to buffer1. An access is routed to buffer1 when the master port number of the incoming AHB request matches the MSTRID field of the BUF1CR. Any buffer "miss" leads to the buffer being flushed and a serial flash transaction being triggered as per the sequence pointed to by the SEQID field.

Write:

- $QSPI_SR[AHB_ACC] = 0$

Address: 30BB_0000h base + 14h offset = 30BB_0014h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved																ADATSZ								Reserved				MSTRID			
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

QuadSPI_BUF1CR field descriptions

Field	Description
31–16 Reserved	This field is reserved.
15–8 ADATSZ	AHB data transfer size: Defines the data transfer size in 8 bytes of an AHB triggered access to serial flash. For example, a value of 0x2 will set transfer size to 16bytes. When ADATSZ = 0, the data size mentioned the sequence pointed to by the SEQID field overrides this value. SW should ensure that this transfer size is not greater than the size of this buffer.
7–4 Reserved	This field is reserved.
MSTRID	Master ID: The ID of the AHB master associated with BUFFER1. Any AHB access with this master port number is routed to this buffer.

10.2.13.6 Buffer2 Configuration Register (QuadSPI_BUF2CR)

This register provides the configuration for any access to buffer2. An access is routed to buffer2 when the master port number of the incoming AHB request matches the MSTRID field of the BUF2CR. Any buffer "miss" leads to the buffer being flushed and a serial flash transaction being triggered as per the sequence pointed to by the SEQID field.

Write:

- $QSPI_SR[AHB_ACC] = 0$

Address: 30BB_0000h base + 18h offset = 30BB_0018h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved																ADATSZ								Reserved				MSTRID			
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

QuadSPI_BUF2CR field descriptions

Field	Description
31–16 Reserved	This field is reserved. Reserved.
15–8 ADATSZ	AHB data transfer size: Defines the data transfer size in 8 Bytes of an AHB triggered access to serial flash. For example, a value of 0x2 will set transfer size to 16bytes. When ADATSZ = 0, the data size mentioned the sequence pointed to by the SEQID field overrides this value. SW should ensure that this transfer size is not greater than the size of this buffer.
7–4 Reserved	This field is reserved. Reserved.
MSTRID	Master ID: The ID of the AHB master associated with BUFFER2. Any AHB access with this master port number is routed to this buffer.

10.2.13.7 Buffer3 Configuration Register (QuadSPI_BUF3CR)

This register provides the configuration for any access to buffer3. An access is routed to buffer3 when the master port number of the incoming AHB request matches the MSTRID field of the BUF3CR. Any buffer "miss" leads to the buffer being flushed a serial flash transaction being triggered as per the sequence pointed to by the SEQID field. If the ALLMST field is set, any transaction where the master port number does not match any of the buffer MSTRID fields will be routed to this buffer. In the case that the ALLMST field is not set, any such transaction (where master port number does not match any of the MSTRID fields) will be returned an ERROR response.

Write:

- $QSPI_SR[AHB_ACC] = 0$

Address: 30BB_0000h base + 1Ch offset = 30BB_001Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	ALLMST	Reserved														
W																
Reset	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ADATSZ								Reserved				MSTRID			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

QuadSPI_BUF3CR field descriptions

Field	Description
31 ALLMST	All master enable: When set, buffer3 acts as an all-master buffer. Any AHB access with a master port number not matching with the master ID of buffer0 or buffer1 or buffer2 is routed to buffer3. When set, the MSTRID field of this register is ignored.
30–16 Reserved	This field is reserved. Reserved.
15–8 ADATSZ	AHB data transfer size: Defines the data transfer size in 8 Bytes of an AHB triggered access to serial flash. When ADATSZ = 0, the data size mentioned the sequence pointed to by the SEQID field overrides this value. SW should ensure that this transfer size is not greater than the size of this buffer.
7–4 Reserved	This field is reserved.
MSTRID	Master ID: The ID of the AHB master associated with BUFFER3. Any AHB access with this master port number is routed to this buffer.

10.2.13.8 Buffer Generic Configuration Register (QuadSPI_BFGENCR)

This register provides the generic configuration to any of the buffer accesses. Any buffer "miss" leads to the buffer being flushed and a serial flash transaction being triggered as per the sequence pointed to by the SEQID field. If the PAR_EN field is set, all the buffer accesses result in parallel accesses to the flashes.

Write:

- $QSPI_SR[AHB_ACC] = 0$

Address: 30BB_0000h base + 20h offset = 30BB_0020h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															PAR_EN
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	SEQID				Reserved											
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

QuadSPI_BFGENCR field descriptions

Field	Description
31–17 Reserved	This field is reserved.
16 PAR_EN	When set, a transaction to two serial flash devices is triggered in parallel mode. Refer to Parallel Flash Mode for more details.
15–12 SEQID	Points to a sequence in the Look-up-table. The SEQID defines the bits [6:2] of the LUT index. The bits [1:0] are always assumed to be 0. Refer to Look-up Table . NOTE: If the sequence pointer differs between the new and previous sequence then the user should reset this. See QSPI_SPTRCLR for more information.
Reserved	This field is reserved.

10.2.13.9 Buffer0 Top Index Register (QuadSPI_BUF0IND)

This register specifies the top index of buffer0, which defines its size. Note that the 3 LSBs of this register are set to zero - this ensures that the buffer is 64bit aligned, as each buffer entry is 64bits long.

The register value should be set to the desired number of bytes less 8. For example, setting BUF0IND to 0 gives 8 bytes, 1 give 16bytes etc.

The size of buffer0 is the difference between the BUF0IND+8 and 0.

It is the responsibility of the software to ensure that BUF0IND value is not greater than the overall size of the buffer. The hardware does not provide any protection against illegal programming.

Write:

- $QSPI_SR[AHB_ACC] = 0$

Address: 30BB_0000h base + 30h offset = 30BB_0030h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	TPINDX0																Reserved															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

QuadSPI_BUF0IND field descriptions

Field	Description
31–3 TPINDX0	Top index of buffer 0.
Reserved	This field is reserved. Reserved.

10.2.13.10 Buffer1 Top Index Register (QuadSPI_BUF1IND)

This register specifies the top index of buffer1, which defines its size. Note that the 3 LSBs of this register are set to zero - this ensures that the buffer is 64bit aligned as each buffer entry is 64bits long.

The register value should be set to the desired number of bytes less. The size of buffer1 is the difference between the BUF1IND and BUF0IND.

It is the responsibility of the software to ensure that BUF1IND value is not greater than the overall size of the buffer. The hardware does not provide any protection against illegal programming.

Write:

- $QSPI_SR[AHB_ACC] = 0$

Address: 30BB_0000h base + 34h offset = 30BB_0034h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	TPINDX1																Reserved															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

QuadSPI_BUF1IND field descriptions

Field	Description
31–3 TPINDX1	Top index of buffer 1.
Reserved	This field is reserved.

10.2.13.11 Buffer2 Top Index Register (QuadSPI_BUF2IND)

This register specifies the top index of buffer2, which defines its size. Note that the 3 LSBs of this register are set to zero - this ensures that the buffer is 64bit aligned as each buffer entry is 64bits long.

The register value should be set to the desired number of bytes less 8. The size of buffer2 is the difference between the BUF2IND and BUF1IND.

It is the responsibility of the software to ensure that BUF2IND value is not greater than the overall size of the buffer. The hardware does not provide any protection against illegal programming.

Write:

- $QSPI_SR[AHB_ACC] = 0$

Address: 30BB_0000h base + 38h offset = 30BB_0038h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	TPINDX2																Reserved															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

QuadSPI_BUF2IND field descriptions

Field	Description
31–3 TPINDX2	Top index of buffer 2.
Reserved	This field is reserved.

10.2.13.12 Serial Flash Address Register (QuadSPI_SFAR)

The module automatically translates this address on the memory map to the address on the flash itself. When operating in 24bit mode, only bits 23-0 are sent to the flash, in 32bit mode, bits 27-0 are used with bits 31-28 driven to 0. Refer to [Table 10-5](#) for the mapping between the access mode and the QSPI_SFAR content and to [Normal Mode](#) for details about the command triggering and command execution. The software should ensure that the serial flash address provided in the QSPI_SFAR register lies in the valid flash address range as defined in [Table 10-5](#).

Write:

- $QSPI_SR[IP_ACC] = 0$

Address: 30BB_0000h base + 100h offset = 30BB_0100h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	SFADR																															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

QuadSPI_SFAR field descriptions

Field	Description
SFADR	Serial Flash Address. The register content is used as byte address for all following IP Commands.

10.2.13.13 Sampling Register (QuadSPI_SMPR)

The Sampling Register allows configuration of how the incoming data from the external serial flash devices are sampled in the QuadSPI module.

Write: Disabled Mode

Address: 30BB_0000h base + 108h offset = 30BB_0108h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0													DDRSMP		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0								SDRSMP			0		0		
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

QuadSPI_SMPR field descriptions

Field	Description
31–19 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
18–16 DDRSMP	DDR Sampling point. Select the sampling point for incoming data when serial flash is executing a DDR instruction. Refer to Input timing in DDR mode with internal sampling , for details on the sampling points.
15–7 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
6–5 SDRSMP	SDR sampling point.
4–3 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
Reserved	This field is reserved. This read-only field is reserved and always has the value 0.

10.2.13.14 RX Buffer Status Register (QuadSPI_RBSR)

This register contains information related to the receive data buffer.

Address: 30BB_0000h base + 10Ch offset = 30BB_010Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	RDCTR															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved		RDBFL						Reserved							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

QuadSPI_RBSR field descriptions

Field	Description
31–16 RDCTR	Read Counter, indicates how many entries of 4 bytes have been removed from the RX Buffer. For example a value of 0x2 would indicate 8bytes have been removed It is incremented by the number (QSPI_RBCT[WMRK] + 1) on RX Buffer POP event. The RX Buffer can be popped using DMA or pop flag QSPI_FR[RBDP]. The QSPI_RSER[RBDDE] defines which pop has to be done. For further details please refer to AHB RX Data Buffer (QSPI_ARDB0 to QSPI_ARDB31) and "Data Transfer from the QuadSPI Module Internal Buffers" section in Flash Read section.
15–14 Reserved	This field is reserved.
13–8 RDBFL	RX Buffer Fill Level, indicates how many entries of 4 bytes are still available in the RX Buffer. For example a value of 0x2 would indicate 8bytes are available.
Reserved	This field is reserved.

10.2.13.15 RX Buffer Control Register (QuadSPI_RBCT)

This register contains control data related to the receive data buffer.

Write:

- $QSPI_SR[IP_ACC] = 0$

Address: 30BB_0000h base + 110h offset = 30BB_0110h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								RXBRD			Reserved			WMRK	
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

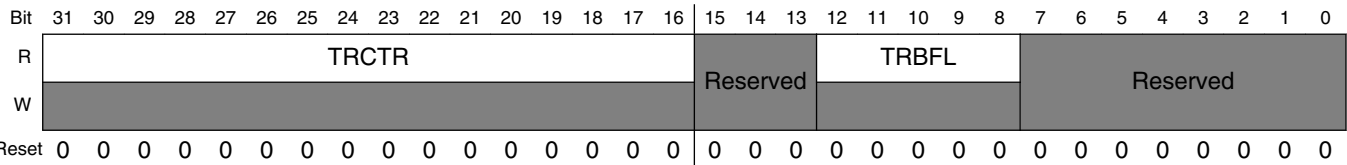
QuadSPI_RBCT field descriptions

Field	Description
31–9 Reserved	This field is reserved.
8 RXBRD	RX Buffer Readout: This bit specifies the access scheme for the RX Buffer readout. 0 RX Buffer content is read using the AHB Bus registers QSPI_ARDB0 to QSPI_ARDB31. For details, refer to Exclusive Access to Serial Flash for AHB Commands. 1 RX Buffer content is read using the IP Bus registers QSPI_RBDR0 to QSPI_RBDR31.
7–5 Reserved	This field is reserved.
WMRK	RX Buffer Watermark: This field determines when the readout action of the RX Buffer is triggered. When the number of valid entries in the RX Buffer is equal to or greater than the number given by (WMRK+1) the QSPI_SR[RXWE] flag is asserted. The value should be entered as the number of 4byte entries minus 1. For example a value of 0x0 would set the watermark to 4bytes, 1 to 8bytes, 2 to 12bytes etc. For details, refer to DMA Usage.

10.2.13.16 TX Buffer Status Register (QuadSPI_TBSR)

This register contains information related to the transmit data buffer.

Address: 30BB_0000h base + 150h offset = 30BB_0150h



QuadSPI_TBSR field descriptions

Field	Description
31–16 TRCTR	Transmit Counter. This field indicates how many entries of 4 bytes have been written into the TX Buffer by host accesses. It is reset to 0 when a 1 is written into the QSPI_MCR[CLR_TXF] bit. It is incremented on each write access to the QSPI_TBDR register when another word has been pushed onto the TX Buffer. When it is not cleared the TRCTR field wraps around to 0. Refer to TX Buffer Data Register (QuadSPI_TBDR) for details.
15–13 Reserved	This field is reserved.
12–8 TRBFL	TX Buffer Fill Level. The TRBFL field contains the number of entries of 4 bytes each available in the TX Buffer for the QuadSPI module to transmit to the serial flash device.
Reserved	This field is reserved.

10.2.13.17 TX Buffer Data Register (QuadSPI_TBDR)

The QSPI_TBDR register provides access to the circular TX Buffer of depth 128 bytes. This buffer provides the data written into it as write data for the page programming commands to the serial flash device. Refer to [Table 10-14](#) for the byte ordering scheme. A write transaction on the flash with data size of less than 32 bits will lead to the removal of four data entry from the TX buffer. The valid bits will be used and the rest of the bits will be discarded.

Write:

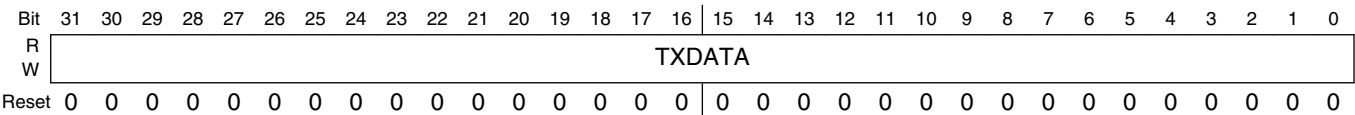
- *QSPI_SR[TXFULL] = 0*

32-bit write access required

NOTE

There is no limitation on Flash programming size. But the data size written to TX Buffer should be 64 Byte aligned. If flash programming size is not 64 bytes aligned, please write redundant data to TX Buffers. The redundant data will be ignored by QuadSPI controller.

Address: 30BB_0000h base + 154h offset = 30BB_0154h



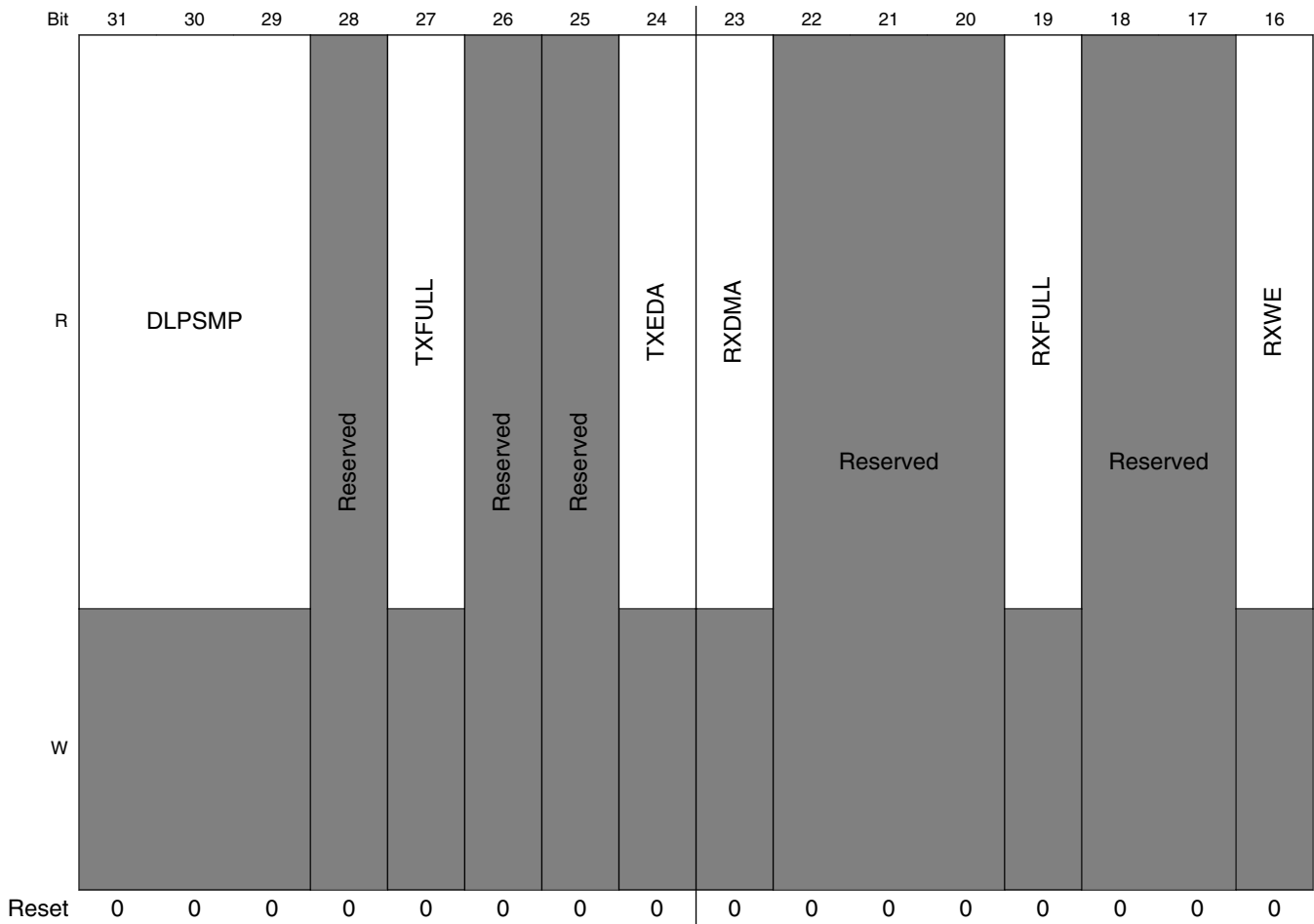
QuadSPI_TBDR field descriptions

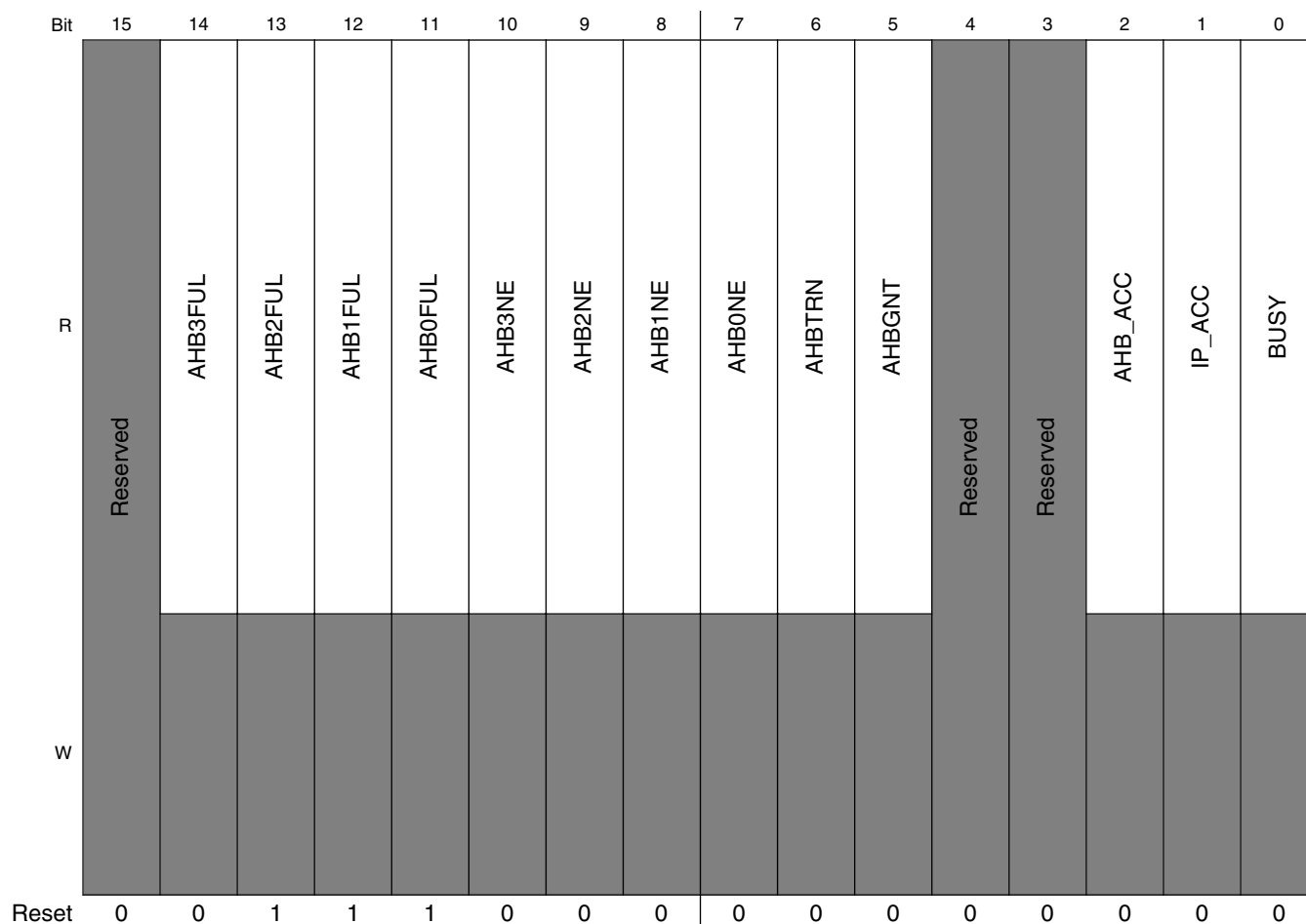
Field	Description
TXDATA	TX Data On write access the data is written into the next available entry of the TX Buffer and the QPSI_TBSR[TRBFL] field is updated accordingly. On a read access, the last data written to the register is returned.

10.2.13.18 Status Register (QuadSPI_SR)

The QSPI_SR register provides all available status information about SFM command execution and arbitration, the RX Buffer and TX Buffer and the AHB Buffer.

Address: 30BB_0000h base + 15Ch offset = 30BB_015Ch





QuadSPI_SR field descriptions

Field	Description
31–29 DLPSMP	NOTE: Data learning is not implemented on this chip. Data learning pattern sampling point: The sampling point found by the controller with the data learning pattern. - This is used for DDR only. - If the learning fails, this field will return garbage and DLPFF bit will be set.
28 Reserved	This field is reserved.
27 TXFULL	TX Buffer Full: Asserted when no more data can be stored.
26 Reserved	This field is reserved.
25 Reserved	This field is reserved.
24 TXEDA	Tx Buffer Enough Data Available Asserted when TX Buffer contains enough data for any pop operation to take place. There must be atleast 128bit data available in TX FIFO for any pop operation otherwise QSPI_FR[TBUF] will be set.

Table continues on the next page...

QuadSPI_SR field descriptions (continued)

Field	Description
23 RXDMA	RX Buffer DMA: Asserted when RX Buffer read out via DMA is active i.e DMA is requested or running.
22–20 Reserved	This field is reserved.
19 RXFULL	RX Buffer Full: Asserted when the RX Buffer is full, i.e. that QSPI_RBSR[RDBFL] field is equal to 32.
18–17 Reserved	This field is reserved.
16 RXWE	RX Buffer Watermark Exceeded: Asserted when the number of valid entries in the RX Buffer exceeds the number given in the QSPI_RBCT[WMRK] field.
15 Reserved	This field is reserved.
14 AHB3FUL	AHB 3 Buffer Full: Asserted when AHB 3 buffer is full.
13 AHB2FUL	AHB 2 Buffer Full: Asserted when AHB 2 buffer is full.
12 AHB1FUL	AHB 1 Buffer Full: Asserted when AHB 1 buffer is full.
11 AHB0FUL	AHB 0 Buffer Full: Asserted when AHB 0 buffer is full.
10 AHB3NE	AHB 3 Buffer Not Empty: Asserted when AHB 3 buffer contains data.
9 AHB2NE	AHB 2 Buffer Not Empty: Asserted when AHB 2 buffer contains data.
8 AHB1NE	AHB 1 Buffer Not Empty: Asserted when AHB 1 buffer contains data.
7 AHB0NE	AHB 0 Buffer Not Empty: Asserted when AHB 0 buffer contains data.
6 AHBTRN	AHB Access Transaction pending: Asserted when there is a pending request on the AHB interface. Refer to the AMBA specification for details.
5 AHBGNT	AHB Command priority Granted: Asserted when another module has been granted priority of AHB Commands against IP Commands. For details refer to Command Arbitration .
4 Reserved	This field is reserved.
3 RESERVED	This field is reserved.
2 AHB_ACC	AHB Access: Asserted when the transaction currently executed was initiated by AHB bus.
1 IP_ACC	IP Access: Asserted when transaction currently executed was initiated by IP bus.
0 BUSY	Module Busy: Asserted when module is currently busy handling a transaction to an external flash device.

10.2.13.19 Flag Register (QuadSPI_FR)

The QSPI_FR register provides all available flags about SFM command execution and arbitration which may serve as source for the generation of interrupt service requests. Note that the error flags in this register do not relate directly to the execution of the transaction in the serial flash device itself but only to the behavior and conditions visible in the QuadSPI module.

Write: Enabled Mode

Address: 30BB_0000h base + 160h offset = 30BB_0160h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16		
R	DLFFF	Reserved		Reserved		TBFF	TBUF	Reserved		ILLINE	Reserved						RBOF	RBDF
W	w1c					w1c	w1c			w1c							w1c	w1c
Reset	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0		

AHB RX Data Buffer (QSPI_ARDB0 to QSPI_ARDB31)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	ABSEF	Reserved	Reserved	ABOF	IUEF	Reserved			IPAEF	IPIEF	Reserved	IPGEF	Reserved			TFF
W	w1c			w1c	w1c				w1c	w1c		w1c				w1c
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

QuadSPI_FR field descriptions

Field	Description
31 DLPFF	NOTE: Data learning is not implemented on this chip. Data Learning Pattern Failure Flag: Set when DATA_LEARN instruction was encountered in a sequence but no sampling point was found for the data learning pattern The controller automatically starts sampling using the value in QSPI_SMPR[DDRSMP].
30 Reserved	This field is reserved.
29–28 Reserved	This field is reserved.
27 TBFF	TX Buffer Fill Flag: Before writing to the TX buffer, this bit should be cleared. Then this bit has to be read back. If the bit is set, the TX Buffer can take more data. If the bit remains cleared, the TX buffer is full. Refer to Tx Buffer Operation for details.
26 TBUF	TX Buffer Underrun Flag: Set when the module tried to pull data although TX Buffer was empty or the buffer contains less than 128bits of data. The application must ensure that the buffer never goes empty during a transaction expect for the last data fetch. The IP Command leading to the TX Buffer underrun is continued (data sent to the serial flash device is all F in case of valid tx underrun. The application must clear the TX Buffer in response to this event by writing a 1 into the QSPI_MCR[CLR_TXF] bit.
25–24 Reserved	This field is reserved.
23 ILLINE	Illegal Instruction Error Flag: Set when an illegal instruction is encountered by the controller in any of the sequences. Refer to Table 10-12 for a list of legal instructions.
22–18 Reserved	This field is reserved.
17 RBOF	RX Buffer Overflow Flag: Set when not all the data read from the serial flash device could be pushed into the RX Buffer. The IP Command leading to this condition is continued until the number of bytes according to the QSPI_IPCR[IDATSZ] field has been read from the serial flash device.

Table continues on the next page...

QuadSPI_FR field descriptions (continued)

Field	Description
	The content of the RX Buffer is not changed.
16 RBDF	RX Buffer Drain Flag: Will be set if the QuadSPI_SR[RXWE] status bit is asserted. Writing 1 into this bit triggers one of the following actions: • If the RX Buffer has up to QuadSPI_RBCT[WMRK] valid entries then the flag is cleared. • If the RX Buffer has more than QuadSPI_RBCT[WMRK] valid entries and the QuadSPI_RSER[RBDDE] bit is not set (flag driven mode) a RX Buffer POP event is triggered. The flag remains set if the RX Buffer contains more than QuadSPI_RBCT[WMRK] valid entries after the RX Buffer POP event is finished. The flag is cleared if the RX Buffer contains less than or equal to QuadSPI_RBCT[WMRK] valid entries after the RX Buffer POP event is finished. Refer to "Receive Buffer Drain Interrupt or DMA Request" section in Normal Mode Interrupt and DMA Requests, for details.
15 ABSEF	AHB Sequence Error Flag: Set when the execution of an AHB Command is started with an WRITE or WRITE_DDR Command in the sequence pointed to by the QSPI_BUFxCR register Communication with the serial flash device is terminated before the execution of WRITE/WRITE_DDR command by the QuadSPI module. The AHB bus request which triggered this command is answered with an ERROR response.
14 Reserved	Reserved This field is reserved.
13 Reserved	Reserved This field is reserved.
12 ABOF	AHB Buffer Overflow Flag: Set when the size of the AHB access exceeds the size of the AHB buffer. This condition can occur only if the QSPI_BUFxCR[ADATSZ] field is programmed incorrectly. The AHB Command leading to this condition is continued until the number of entries according to the QSPI_BUFxCR[ADATSZ] field has been read from the serial flash device. The content of the AHB Buffer is not changed.
11 IUEF	IP Command Usage Error Flag: Set when in parallel flash mode the execution of an IP Command is started and the sequence pointed to by the sequence ID contains a WRITE or a WRITE_DDR command. Refer to Table 10-12 table for the related commands. Communication with the serial flash device is terminated before the execution of WRITE/WRITE_DDR command by the QuadSPI module.
10–8 Reserved	This field is reserved.
7 IPAEF	IP Command Trigger during AHB Access Error Flag. Set when the following condition occurs: • A write access occurs to the QSPI_IPCR[SEQID] field and the QSPI_SR[AHB_ACC] bit is set. Any command leading to the assertion of the IPAEF flag is ignored.
6 IPIEF	IP Command Trigger could not be executed Error Flag. Set when the QSPI_SR[IP_ACC] bit is set (i.e. an IP triggered command is currently executing) and any of the following conditions occurs: • Write access to the QSPI_IPCR register. Any command leading to the assertion of the IPIEF flag is ignored • Write access to the QSPI_SFAR register. • Write access to the QSPI_RBCT register.
5 Reserved	This field is reserved.

Table continues on the next page...

QuadSPI_FR field descriptions (continued)

Field	Description
4 IPGEF	IP Command Trigger during AHB Grant Error Flag: Set when the following condition occurs: A write access occurs to the QSPI_IPCR[SEQID] field and the QSPI_SR[AHBGNT] bit is set. Any command leading to the assertion of the IPGEF flag is ignored.
3–1 Reserved	This field is reserved.
0 TFF	IP Command Transaction Finished Flag: Set when the QuadSPI module has finished a running IP Command. If an error occurred the related error flags are valid, at the latest, in the same clock cycle when the TFF flag is asserted.

1. QSPI_BUFxCR implies anyone of QSPI_BUF0CR/QSPI_BUF1CR/QSPI_BUF2CR/QSPI_BUF3CR

10.2.13.20 Interrupt and DMA Request Select and Enable Register (QuadSPI_RSER)

The QuadSPI_RSER register provides enables and selectors for the interrupts in the QuadSPI module.

NOTE

Each flag of the QuadSPI_FR register enabled as source for an interrupt prevents the QuadSPI module from entering Stop Mode or Module Disable Mode when this flag is set.

Write: Anytime

Address: 30BB_0000h base + 164h offset = 30BB_0164h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

QuadSPI_RSER field descriptions

Field	Description
31 DLPFIE	NOTE: Data learning is not implemented on this chip. Data Learning Pattern Failure Interrupt enable. Triggered by DLPFF flag in QSPI_FR register 0 No DLPFF interrupt will be generated 1 DLPFF interrupt will be generated
30 Reserved	This field is reserved.
29–28 RESERVED	This field is reserved.
27 TBFIE	TX Buffer Fill Interrupt Enable 0 No TBFF interrupt will be generated 1 TBFF interrupt will be generated
26 TBUIE	TX Buffer Underrun Interrupt Enable 0 No TBUF interrupt will be generated 1 TBUF interrupt will be generated
25 Reserved	This field is reserved.
24 Reserved	This field is reserved.
23 ILLINIE	Illegal Instruction Error Interrupt Enable. Triggered by ILLINE flag in QSPI_FR 0 No ILLINE interrupt will be generated 1 ILLINE interrupt will be generated
22 Reserved	This field is reserved.
21 RBDDE	RX Buffer Drain DMA Enable: Enables generation of DMA requests for RX Buffer Drain. When this bit is set DMA requests are generated as long as the QSPI_SR[RXWE] status bit is set. 0 No DMA request will be generated 1 DMA request will be generated
20–18 Reserved	This field is reserved.

Table continues on the next page...

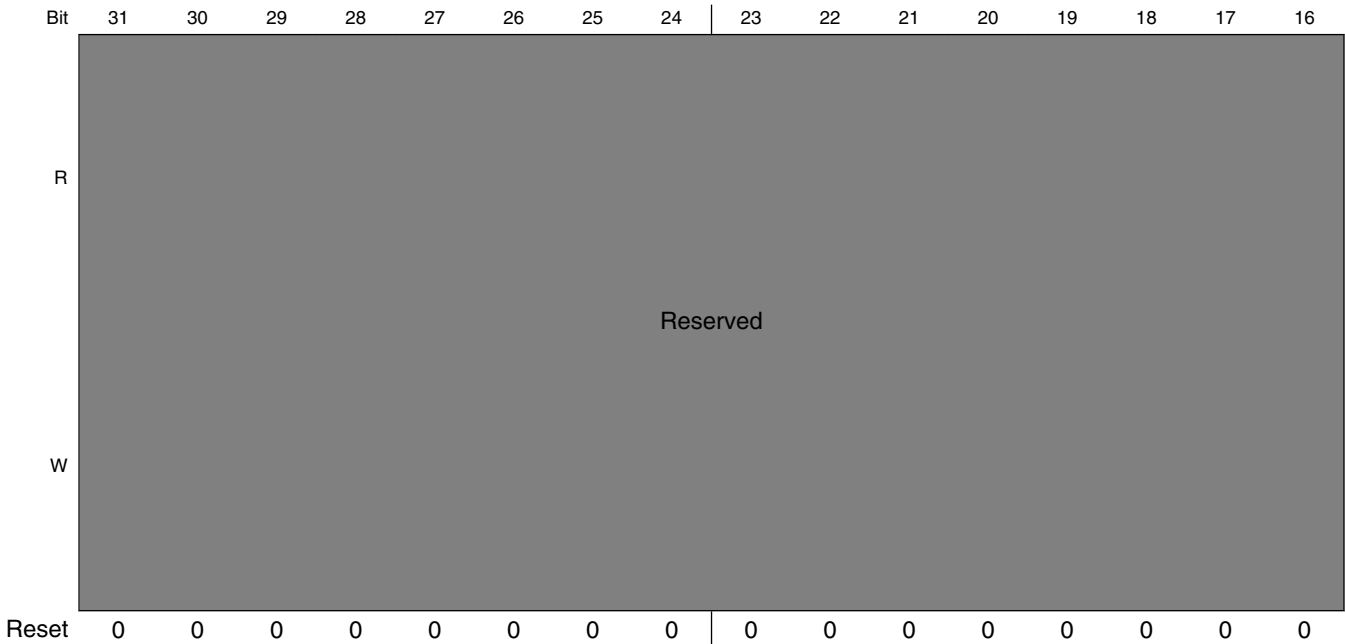
QuadSPI_RSER field descriptions (continued)

Field	Description
17 RBOIE	RX Buffer Overflow Interrupt Enable 0 No RBOF interrupt will be generated 1 RBOF interrupt will be generated
16 RBDIE	RX Buffer Drain Interrupt Enable: Enables generation of IRQ requests for RX Buffer Drain. When this bit is set the interrupt is asserted as long as the QuadSPI_SR[RBDF] flag is set. 0 No RBDF interrupt will be generated 1 RBDF Interrupt will be generated
15 ABSEIE	AHB Sequence Error Interrupt Enable: Triggered by ABSEF flags of QSPI_FR 0 No ABSEF interrupt will be generated 1 ABSEF interrupt will be generated
14 Reserved	Reserved This field is reserved.
13 Reserved	Reserved This field is reserved.
12 ABOIE	AHB Buffer Overflow Interrupt Enable 0 No ABOF interrupt will be generated 1 ABOF interrupt will be generated
11 IUEIE	IP Command Usage Error Interrupt Enable 0 No IUEF interrupt will be generated 1 IUEF interrupt will be generated
10–8 Reserved	This field is reserved.
7 IPAEIE	IP Command Trigger during AHB Access Error Interrupt Enable 0 No IPAEF interrupt will be generated 1 IPAEF interrupt will be generated
6 IPIEIE	IP Command Trigger during IP Access Error Interrupt Enable 0 No IPIEF interrupt will be generated 0 IPIEF interrupt will be generated
5 Reserved	This field is reserved.
4 IPGEIE	IP Command Trigger during AHB Grant Error Interrupt Enable 0 No IPGEF interrupt will be generated 1 IPGEF interrupt will be generated
3–1 Reserved	This field is reserved. Reserved.
0 TFIE	Transaction Finished Interrupt Enable 0 No TFF interrupt will be generated 1 TFF interrupt will be generated

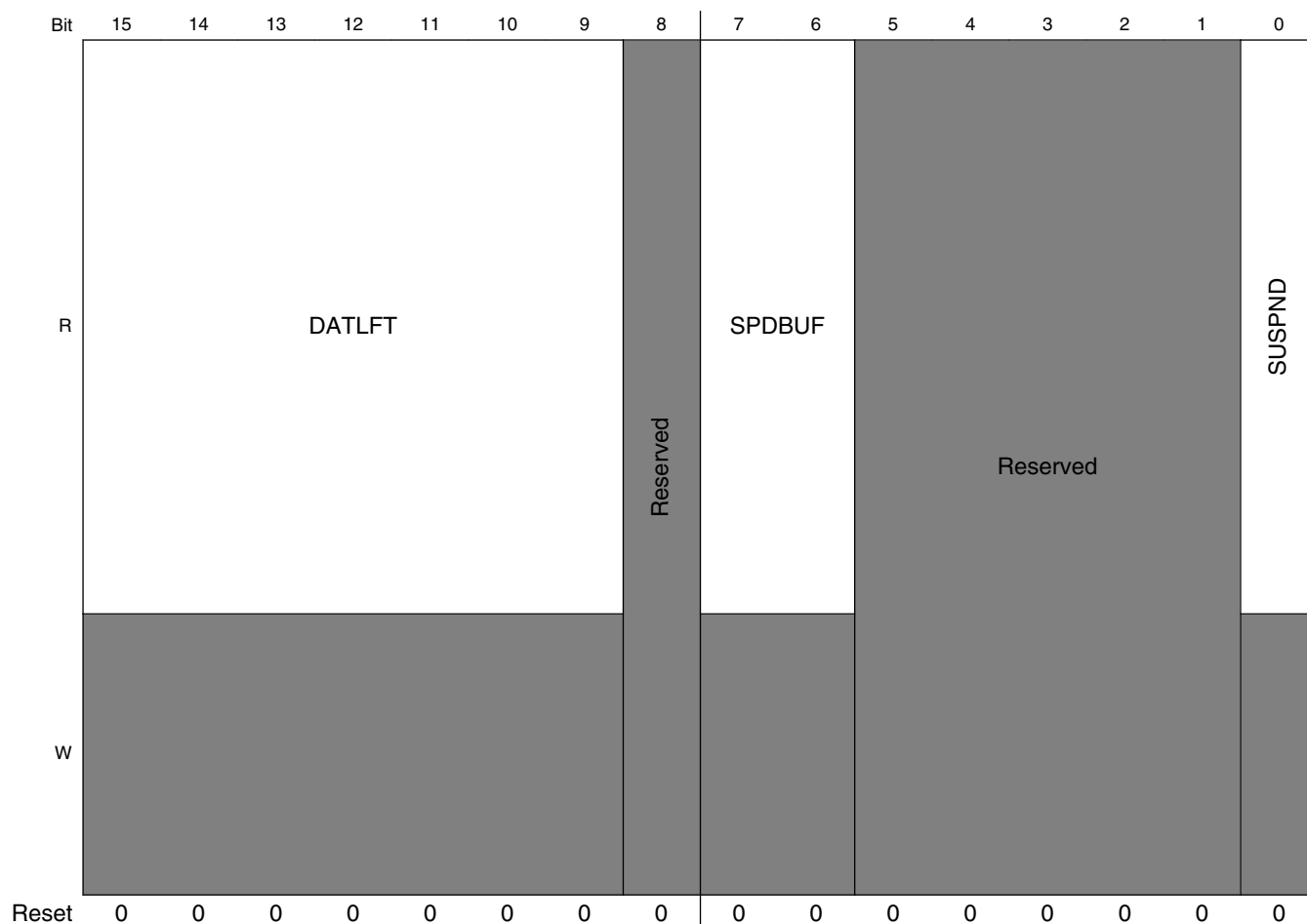
10.2.13.21 Sequence Suspend Status Register (QuadSPI_SPNDST)

The sequence suspend status register provides information specific to any suspended sequence. An AHB sequence may be suspended when a high priority AHB master makes an access before the AHB sequence completes the data transfer requested.

Address: 30BB_0000h base + 168h offset = 30BB_0168h



AHB RX Data Buffer (QSPI_ARDB0 to QSPI_ARDB31)



QuadSPI_SPNDST field descriptions

Field	Description
31–16 Reserved	This field is reserved.
15–9 DATLFT	Data left: Provides information about the amount of data left to be read in the suspended sequence. Valid only when SUSPND is set to 1'b1. Value in terms of 64 bits or 8 bytes
8 Reserved	This field is reserved.
7–6 SPDBUF	Suspended Buffer: Provides the suspended buffer number. Valid only when SUSPND is set to 1'b1
5–1 Reserved	This field is reserved.
0 SUSPND	When set, it signifies that a sequence is in suspended state

10.2.13.22 Sequence Pointer Clear Register (QuadSPI_SPTRCLR)

The sequence pointer clear register provides bits to reset the IP and Buffer sequence pointers. The sequence pointer contains the index of which instruction within the LUT entry is to be executed next. For example, if the LUT entry ends on a JMP_ON_CS value of 2, the index will be stored as 2.

The software should reset the sequence pointers whenever the sequence ID is changed by updating the SEQID field in QSPI_IPCR or QSPI_BFGENCR.

Address: 30BB_0000h base + 16Ch offset = 30BB_016Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								Reserved							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

QuadSPI_SPTRCLR field descriptions

Field	Description
31–9 Reserved	This field is reserved.
8 IPPTRC	IP Pointer Clear: 1: Clears the sequence pointer for IP accesses as defined in QuadSPI_IPCR
7–1 Reserved	This field is reserved. Reserved.
0 BFPTRC	Buffer Pointer Clear: 1: Clears the sequence pointer for AHB accesses as defined in QuadSPI_BFGENCR.

10.2.13.23 Serial Flash A1 Top Address (QuadSPI_SFA1AD)

The QSPI_SFA1AD register provides the address mapping for the serial flash A1. The difference between QSPI_SFA1AD[TPADA1] and QSPI_AMBA_BASE defines the size of the memory map for serial flash A1.

Write:

AHB RX Data Buffer (QSPI_ARDB0 to QSPI_ARDB31)

- $QSPI_SR[IP_ACC] = 0$
- $QSPI_SR[AHB_ACC] = 0$

Address: 30BB_0000h base + 180h offset = 30BB_0180h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	TPADA1																Reserved															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

QuadSPI_SFA1AD field descriptions

Field	Description
31–10 TPADA1	Top address for Serial Flash A1. In effect, TPADxx is the first location of the next memory.
Reserved	This field is reserved.

10.2.13.24 Serial Flash A2 Top Address (QuadSPI_SFA2AD)

The QSPI_SFA2AD register provides the address mapping for the serial flash A2. The difference between QSPI_SFA2AD[TPADA2] and QSPI_SFA1AD[TPADA1] defines the size of the memory map for serial flash A2.

Write:

- $QSPI_SR[IP_ACC] = 0$
- $QSPI_SR[AHB_ACC] = 0$

Address: 30BB_0000h base + 184h offset = 30BB_0184h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	TPADA2																Reserved															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

QuadSPI_SFA2AD field descriptions

Field	Description
31–10 TPADA2	Top address for Serial Flash A2. In effect, TPxxAD is the first location of the next memory.
Reserved	This field is reserved.

10.2.13.25 Serial Flash B1Top Address (QuadSPI_SFB1AD)

The QSPI_SFB1AD register provides the address mapping for the serial flash B1. The difference between QSPI_SFB1AD[TPADB1] and QSPI_SFA2AD[TPADA2] defines the size of the memory map for serial flash B1.

Write:

- $QSPI_SR[IP_ACC] = 0$
- $QSPI_SR[AHB_ACC] = 0$

Address: 30BB_0000h base + 188h offset = 30BB_0188h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	TPADB1																Reserved															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

QuadSPI_SFB1AD field descriptions

Field	Description
31–10 TPADB1	Top address for Serial Flash B1. In effect, TPxxAD is the first location of the next memory.
Reserved	This field is reserved.

10.2.13.26 Serial Flash B2Top Address (QuadSPI_SFB2AD)

The QSPI_SFB2AD register provides the address mapping for the serial flash B2. The difference between QSPI_SFB2AD[TPADB2] and QSPI_SFB1AD[TPADB1] defines the size of the memory map for serial flash B2.

Write:

- $QSPI_SR[IP_ACC] = 0$
- $QSPI_SR[AHB_ACC] = 0$

Address: 30BB_0000h base + 18Ch offset = 30BB_018Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	TPADB2																Reserved															
W																																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

QuadSPI_SFB2AD field descriptions

Field	Description
31–10 TPADB2	Top address for Serial Flash B2. In effect, TPxxAD is the first location of the next memory.
Reserved	This field is reserved.

10.2.13.27 RX Buffer Data Register (QuadSPI_RBDRn)

The QuadSPI_RBDR registers provide access to the individual entries in the RX Buffer. Refer to [Table 10-14](#) for the byte ordering scheme.

QuadSPI_RBDR0 corresponds to the actual position of the read pointer within the RX Buffer. The number of valid entries available depends from the number of RX Buffer entries implemented and from the number of valid buffer entries available in the RX Buffer.

Example 1, RX Buffer filled completely with 32 words: In this case the address range for valid read access extends from QuadSPI_RBDR0 to QuadSPI_RBDR31.

Example 2, RX Buffer filled with 5 valid words: RX Buffer fill level QuadSPI_RBSR[RDBFL] is 5. In this case an access to QuadSPI_RBDR4 provides the last valid entry.

Any access beyond the range of valid RX Buffer entries provides undefined results.

Address: 30BB_0000h base + 200h offset + (4d × i), where i=0d to 31d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
	RXDATA																															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

QuadSPI_RBDRn field descriptions

Field	Description
RXDATA	RX Data. The RXDATA field contains the data associated with the related RX Buffer entry. Data format and byte ordering is given in Byte Ordering of Serial Flash Read Data .

10.2.13.28 LUT Key Register (QuadSPI_LUTKEY)

The LUT Key register contains the key to lock and unlock the Look-up-table. Refer to [Look-up Table](#) for details.

Write:Anytime

Address: 30BB_0000h base + 300h offset = 30BB_0300h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R																																
W																																
	KEY																															
Reset	0	1	0	1	1	0	1	0	1	1	1	1	0	0	0	0	0	1	0	1	1	0	1	0	1	1	1	1	0	0	0	0

QuadSPI_LUTKEY field descriptions

Field	Description
KEY	The key to lock or unlock the LUT. The KEY is 0x5AF05AF0. The read value is always 0x5AF05AF0

10.2.13.29 LUT Lock Configuration Register (QuadSPI_LCKCR)

The LUT lock configuration register is used along with QSPI_LUTKEY register to lock or unlock the LUT. This register has to be written immediately after QSPI_LUTKEY register for the lock or unlock operation to be successful. Refer to [Look-up Table](#) for details. Setting both the LOCK and UNLOCK bits as "00" or "11" is not allowed.

Write: Just after writing the LUT Key Register

(QSPI_LUTKEY)

Address: 30BB_0000h base + 304h offset = 30BB_0304h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved														UNLOCK	LOCK
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

QuadSPI_LCKCR field descriptions

Field	Description
31–2 Reserved	This field is reserved.
1 UNLOCK	Unlocks the LUT when the following two conditions are met: 1. This register is written just after the LUT Key Register (QuadSPI_LUTKEY) 2. The LUT key register was written with 0x5AF05AF0 key
0 LOCK	Locks the LUT when the following condition is met: 1. This register is written just after the LUT Key Register (QuadSPI_LUTKEY) 2. The LUT key register was written with 0x5AF05AF0 key

10.2.13.30 Look-up Table register (QuadSPI_LUT0)

The LUT registers are a look-up-table for sequences of instructions. The programmable sequence engine executes the instructions in these sequences to generate a valid serial flash transaction. There are a total of 64 LUT registers. These 64 registers are divided into groups of 4 registers that make a valid sequence. Therefore, QSPI_LUT[0], QSPI_LUT[4], QSPI_LUT[8] QSPI_LUT[60] are the starting registers of a valid sequence. Each of these sets of 4 registers can have a maximum of 8 instructions. A maximum of 16 sequences can be defined at one time. [Look-up Table](#) describes the LUT registers in detail.

Write: Once the LUT is unlocked

Address: 30BB_0000h base + 310h offset = 30BB_0310h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	INSTR1								OPRND1							
W																
Reset	0	0	0	0	1	0	0	0	0	0	0	1	1	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	INSTR0								OPRND0							
W																
Reset	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	1

QuadSPI_LUT0 field descriptions

Field	Description
31–26 INSTR1	Instruction 1
25–24 PAD1	Pad information for INSTR1. 00 1 Pad 01 2 Pads 10 4 Pads 11 NA
23–16 OPRND1	Operand for INSTR1.
15–10 INSTR0	Instruction 0
9–8 PAD0	Pad information for INSTR0. 00 1 Pad 01 2 Pads 10 4 Pads 11 NA
OPRND0	Operand for INSTR0.

10.2.13.31 Look-up Table register (QuadSPI_LUT1)

The LUT registers are a look-up-table for sequences of instructions. The programmable sequence engine executes the instructions in these sequences to generate a valid serial flash transaction. There are a total of 64 LUT registers. These 64 registers are divided into groups of 4 registers that make a valid sequence. Therefore, QSPI_LUT[0], QSPI_LUT[4], QSPI_LUT[8] QSPI_LUT[60] are the starting registers of a valid sequence. Each of these sets of 4 registers can have a maximum of 8 instructions. A maximum of 16 sequences can be defined at one time. [Look-up Table](#) describes the LUT registers in detail.

Write: Once the LUT is unlocked

Address: 30BB_0000h base + 314h offset = 30BB_0314h

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	INSTR1								OPRND1							
W																
Reset	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	INSTR0								OPRND0							
W																
Reset	0	0	0	1	1	1	0	0	0	0	0	0	1	0	0	0

QuadSPI_LUT1 field descriptions

Field	Description
31–26 INSTR1	Instruction 1
25–24 PAD1	Pad information for INSTR1. 00 1 Pad 01 2 Pads 10 4 Pads 11 NA
23–16 OPRND1	Operand for INSTR1.
15–10 INSTR0	Instruction 0
9–8 PAD0	Pad information for INSTR0. 00 1 Pad 01 2 Pads 10 4 Pads 11 NA
OPRND0	Operand for INSTR0.

10.2.13.32 Look-up Table register (QuadSPI_LUTn)

The LUT registers are a look-up-table for sequences of instructions. The programmable sequence engine executes the instructions in these sequences to generate a valid serial flash transaction. There are a total of 64 LUT registers. These 64 registers are divided into groups of 4 registers that make a valid sequence. Therefore, QSPI_LUT[0], QSPI_LUT[4], QSPI_LUT[8] QSPI_LUT[60] are the starting registers of a valid sequence. Each of these sets of 4 registers can have a maximum of 8 instructions. A maximum of 16 sequences can be defined at one time. [Look-up Table](#) describes the LUT registers in detail.

Write: Once the LUT is unlocked

Address: 30BB_0000h base + 318h offset + (4d × i), where i=0d to 61d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	INSTR1								OPRND1							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	INSTR0								OPRND0							
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

QuadSPI_LUTn field descriptions

Field	Description
31–26 INSTR1	Instruction 1
25–24 PAD1	Pad information for INSTR1. 00 1 Pad 01 2 Pads 10 4 Pads 11 NA
23–16 OPRND1	Operand for INSTR1.
15–10 INSTR0	Instruction 0
9–8 PAD0	Pad information for INSTR0. 00 1 Pad 01 2 Pads 10 4 Pads 11 NA
OPRND0	Operand for INSTR0.

10.3 Ultra Secured Digital Host Controller (uSDHC)

10.3.1 Introduction

The uSDHC provides the interface between the host system and the SD/SDIO/MMC cards, as depicted in [Figure 10-33](#). It acts as a bridge, passing host bus transactions to the SD/SDIO/MMC cards by sending commands and performing data accesses to/from the cards. It handles the SD/SDIO/MMC protocols at the transmission level. The following are brief descriptions of the cards supported by uSDHC:

The Multi Media Card (MMC) is a universal low-cost data storage and communication media designed to cover a wide array of applications including mobile video and gaming. Previous MMC cards were based on a 7-pin serial bus with a single data pin, while the new high speed MMC communication is based on an advanced 11-pin serial bus designed to operate in the low-voltage range.

The Secure Digital Card (SD) is an evolution of the old MMC technology. It is specifically designed to meet the security, capacity, performance, and environmental requirements inherent in newly emerging audio and video consumer electronic devices. The physical form factor, pin assignment, and data transfer protocol are forward compatible with the old MMC (with some additions).

Under the SD protocol, it can be categorized into memory card, I/O card, and combo card, which have both memory and I/O functions. The memory card invokes a copyright protection mechanism that complies with the security of the SDMI standard. The I/O card, which is also known as SDIO card, provides high-speed data I/O with low-power consumption for mobile electronic devices. For the sake of simplicity, the next figure does not show cards with reduced size or mini cards.

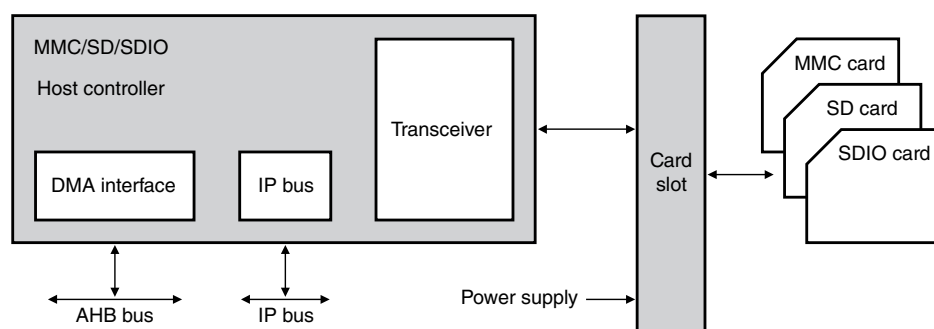


Figure 10-33. System connection of uSDHC

The following figure illustrates the block diagram of uSDHC.

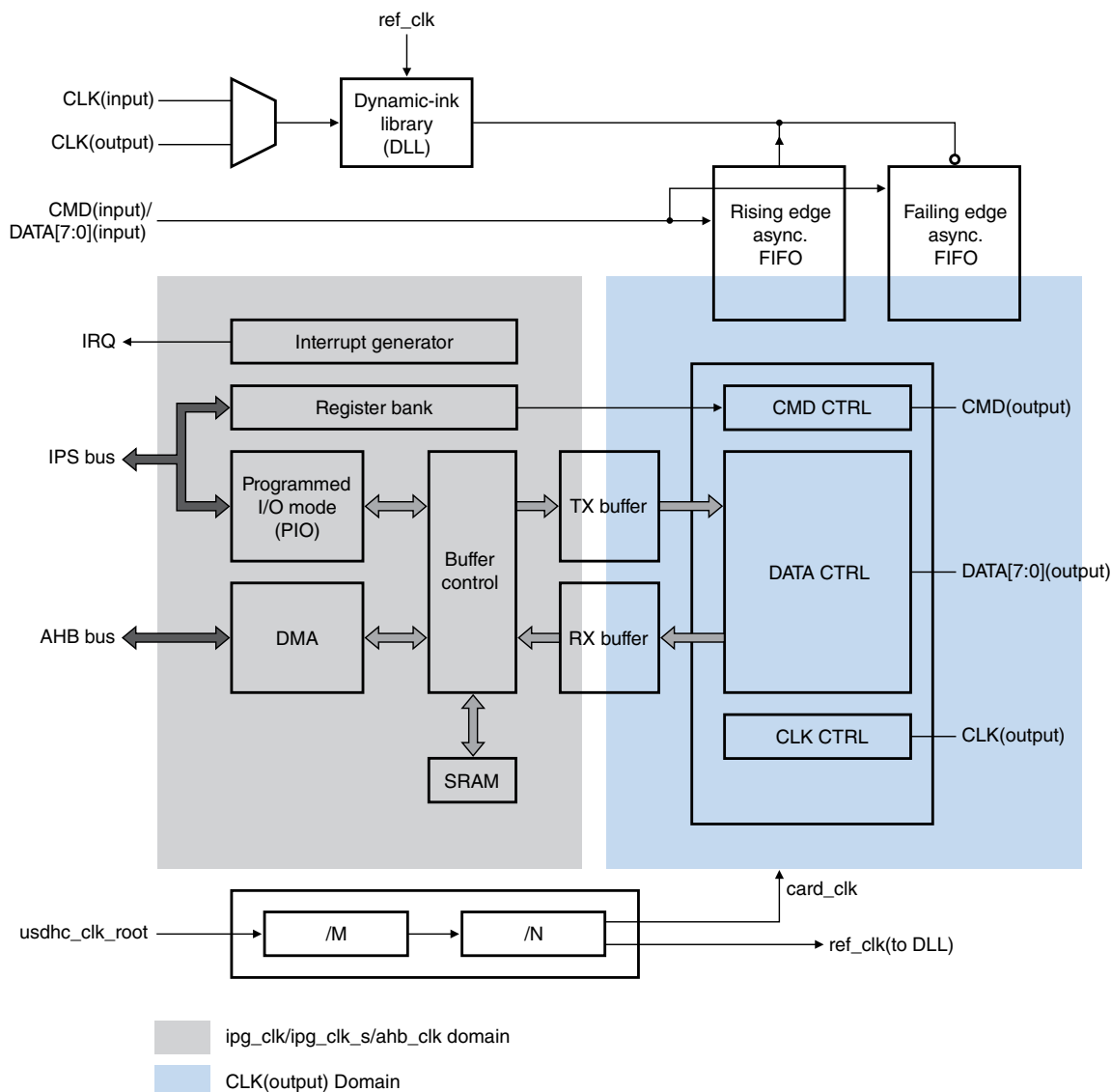


Figure 10-34. uSDHC block diagram

10.3.1.1 Features

The features of the uSDHC module include the following:

- Conforms to the SD Host Controller Standard Specification version 2.0/3.0
- Compatible with the MMC System Specification version 4.2/4.3/4.4/4.41/5.0/5.1
- Compatible with the SD Memory Card Specification version 3.0 and supports the Extended Capacity SD Memory Card
- Compatible with the SDIO Card Specification version 2.0/3.0
- Designed to work with SD Memory, miniSD Memory, SDIO, miniSDIO, SD Combo, MMC, MMC plus, and MMC RS cards
- Card bus clock frequency up to 208 MHz

- Supports 1-bit/4-bit SD and SDIO modes, and 1-bit/4-bit/8-bit MMC modes
 - Up to 832 Mbps of data transfer for SDIO cards using four parallel data lines in the Single Data Rate (SDR) mode
 - Up to 400 Mbps of data transfer for SDIO card using four parallel data lines in the Dual Data Rate (DDR) mode
 - Up to 832 Mbps of data transfer for SDXC cards using four parallel data lines in the Single Data Rate (SDR) mode
 - Up to 400 Mbps of data transfer for SDXC card using four parallel data lines in the Dual Data Rate (DDR) mode
 - Up to 416 Mbps of data transfer for MMC cards using eight parallel data lines in the Single Data Rate (SDR) mode
 - Up to 3200 Mbps of data transfer for MMC cards using eight parallel data lines in the Dual Data Rate (DDR) mode
- Supports a single block/multi-block read and write
- Supports block sizes of 1 ~ 4096 bytes
- Supports the write protection switch for write operations
- Supports both synchronous and asynchronous abort
- Supports pause during the data transfer at block gap
- Supports SDIO Read Wait and Suspend Resume operations
- Supports Auto CMD12 for multi-block transfer
- Host can initiate non-data transfer command while data transfer is in progress
- Allows cards to interrupt the host in 1-bit and 4-bit SDIO modes; also supports interrupt period
- Embodies two fully configurable 128x32-bit FIFO for read/write data
- Supports internal and external DMA capabilities
- Support voltage selection by configuring vendor-specific register bit
- Supports Advanced DMA to perform linked memory access
- support Command queue mechanism

10.3.1.2 Modes and operations

10.3.1.2.1 Data transfer modes

The uSDHC module can select the following modes for data transfer:

- SD 1-bit
- SD 4-bit
- MMC 1-bit
- MMC 4-bit
- MMC 8-bit
- Identification mode (up to 400 kHz)

- MMC full-speed mode (up to 26 MHz)
- MMC high-speed mode (up to 52 MHz)
- MMC HS400 mode (200 MHz both edges)
- MMC DDR mode (52 MHz both edges)
- SD/SDIO full-speed mode (up to 25 MHz)
- SD/SDIO high-speed mode (up to 50 MHz)
- SD/SDIO UHS-I mode (up to 208 MHz in SDR mode, up to 50 MHz in the DDR mode)

10.3.2 External signals

The following table describes the external signals of uSDHC:

Table 10-38. uSDHC external signals

Signal	Description	Direction
CLK	Clock for MMC/SD/SDIO card	O
CMD	CMD line connect to card	I/O
DATA7	DATA7 line in the 8-bit mode — Not used in other modes	I/O
DATA6	DATA6 line in the 8-bit mode — Not used in other modes	I/O
DATA5	DATA5 line in the 8-bit mode — Not used in other modes	I/O
DATA4	DATA4 line in the 8-bit mode — Not used in other modes	I/O
DATA3	DATA3 line in the 4/8-bit mode or configured as card detection pin. The bit may be configured as card detection pin in the 1-bit mode.	I/O
DATA2	DATA2 line or Read Wait in the 4-bit mode Read Wait in 1-bit mode	I/O
DATA1	DATA1 line in the 4/8-bit mode Also, used to detect interrupt in 1/4-bit mode	I/O
DATA0	DATA0 line in all the modes Also, used to detect busy state	I/O
CD_B	Card detection pin If not used (for the embedded memory), tie low to indicate that there is a card attached.	I
WP	Card write protect detect If not used (for the embedded memory), tie low to indicate that it is not write protected.	I

Table continues on the next page...

Table 10-38. uSDHC external signals (continued)

Signal	Description	Direction
LCTL	LED control used to drive an external LED active high, fully controlled by the driver Optional output	O
RESET_B	Card hardware reset signal, active low	O
VSELECT	IO power voltage selection signal	O
STROBE	Input clock for eMMC HS400 mode	I

10.3.2.1 Signals overview

uSDHC has 15 associated I/O signals.

- The CLK is an internally generated clock used to drive the MMC, SD, and SDIO cards.
- The CMD I/O is used to send commands and receive responses to and from the card. Eight data lines (DATA7~DATA0) are used to perform data transfers between the uSDHC module and the card.
- The CD_B and WP are card detection and write protection signals directly routed from the socket. These two signals are active low (0). A low on CD_B means that a card is inserted, and a high on WP means that the write protect switch is active.
- LCTL is an output signal used to drive an external LED to indicate that the SD interface is busy.
- RESET_B is an output signal used to reset the MMC card.
- VSELECT is an output signal used to change the voltage of the external power supplier.
- STROBE is input clock signal for eMMC HS400 mode.

CD_B, WP, LCTL, RESET_B, and VSELECT are all optional for system implementation. If uSDHC needs to support a 4-bit data transfer, DATA7~DATA4 can also be optional and tied to high. If the uSDHC does not support the HS400 mode, STROBE can also be optional and tied to low.

10.3.3 Functional description

The following sections provide a brief functional description of the major system blocks, including the data buffer, DMA AHB interface, register bank as well as IP Bus interface, dual-port memory wrapper, data/command controller, clock and reset manager, and clock generator.

10.3.3.1 Data buffer

The uSDHC module uses one configurable data buffer to transfer data between the system bus (IP bus or advanced high-performance bus (AHB) bus) and the SD card in an optimized manner, maximizing throughput between the two clock domains (IP peripheral clock and the master clock).

The buffer is used as a temporary storage for transferring data between the host system and the card. The watermark levels for read and write are both configurable and can range between 1 to 128 words. The burst lengths for read and write are also configurable and can range between 1 to 31 words. The next figure provides the uSDHC buffer scheme.

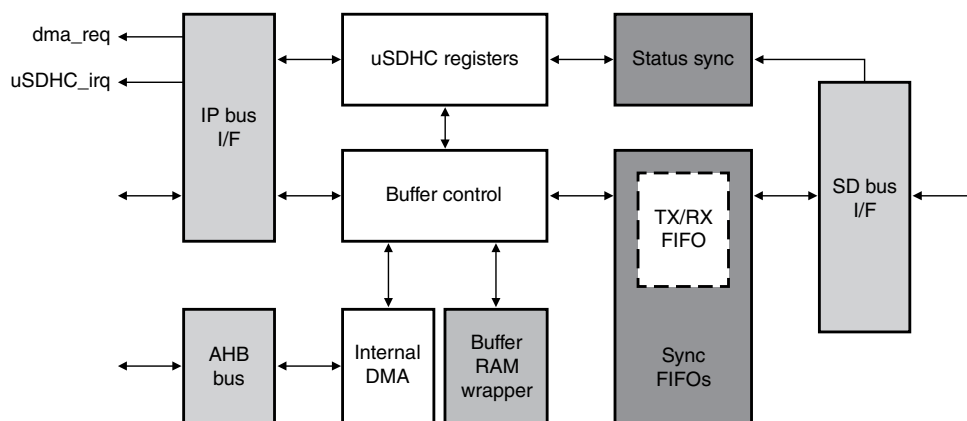


Figure 10-35. uSDHC buffer scheme

Here are 3 the two transfer modes to access the data buffer:

- CPU polling mode:
 - For a host-read operation, when the number of words received in the buffer meets or exceeds the RD_WML watermark value, by polling the BRR bit, the host driver can read the Buffer Data Port register to fetch the amount of words set in the RD_WML register from the buffer. The write operation is similar. For more information on the process of writing operation, see [Write operation sequence](#).
- External DMA mode:
 - For a read operation, when there are more words received in the buffer than the amount set in the RD_WML register, a DMA request is sent out to inform the external DMA to fetch the data. The request will be immediately de-asserted when there is an access on the Buffer Data Port register. If the number of words in the buffer after the current burst meets or exceeds RD_WML value, the DMA request is asserted again. For instance, if there are twice as many words in the

buffer as there are in the RD_WML value, there are two successive DMA requests with only one cycle of de-assertion between. The write operation is similar. Note the accesses CPU polling mode and external DMA mode both use the IP bus, and if the external DMA is enabled, in both modes an external DMA request is sent when the buffer is ready.

- Internal DMA mode (includes simple and advanced DMA accesses):
 - The internal DMA access, either by simple or advanced DMA, is over the AHB bus. For internal DMA access mode, the external DMA request will never be sent out.

For a read operation, when there are more words in the buffer than the amount set in the RD_WML register, the internal DMA starts fetching data over the AHB bus. Except for INCR4 and INCR8, the burst type is always the INCR mode and the burst length depends on the shortest of the following factors:

- Burst length configured in the burst length field of the Watermark Level register
- Watermark level boundary
- Block size boundary
- Data boundary configured in the current descriptor (if the ADMA is active)
- 1 KB address boundary defined in the AHB protocol

The Write operation functions in a similar manner—sequential and contiguous access is necessary to ensure that the pointer address value is correct. Random or skipped access is not possible. The byte order, by reset, is little endian mode. The actual byte order is swapped inside the buffer, according to the endian mode configured by software (see the following figures). For a host write operation, the byte order is swapped after the data is fetched from the buffer and ready to send to the SD Bus. For a host read operation, the byte order is swapped before the data is stored in the buffer.

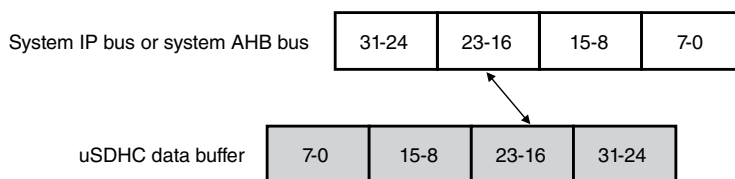


Figure 10-36. Data swap between system bus and uSDHC data buffer in the byte little endian mode

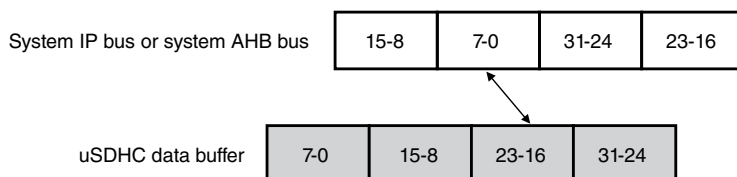


Figure 10-37. Data swap between system bus and uSDHC data buffer in half word big endian mode

10.3.3.1.1 Write operation sequence

There are 3 ways to write data into the buffer when the user transfers data to the card:

- External DMA through the uSDHC DMA request signal
- Processor core polling through the BWR bit in the Interrupt Status register (interrupt or polling)
- Internal DMA

When the internal DMA is not used, the DMAEN bit in the Transfer Type register is not set when the command is sent, uSDHC asserts a DMA request when the amount of buffer space exceeds the value set in the WR_WML register and is ready for receiving new data. At the same time, uSDHC sets the BWR bit. The buffer write ready interrupt is generated if it is enabled by software.

When internal DMA is used, uSDHC does not inform the system before all the required number of bytes are transferred (if no error is encountered). When an error occurs during the data transfer, uSDHC aborts the data transfer and abandons the current block. The host driver should read the contents of the DMA System Address register to obtain the starting address of the abandoned data block. If the current data transfer is in multi-block mode, uSDHC does not automatically send CMD12, even though the AC12EN bit in the Transfer Type register is set. The host driver sends CMD12 in this scenario and restarts the write operation from that address. It is recommended that a software reset for Data be applied before the transfer is restarted.

The uSDHC module does not start data transmission until the number of words set in the WR_WML register can be held in the buffer. If the buffer is empty and the host system does not write data in time, uSDHC stops the CLK to avoid the data buffer underrun situation.

10.3.3.1.2 Read operation sequence

There are 3 ways to read data from the buffer when the user transfers data to the card:

- External DMA through uSDHC DMA request signal

- Processor core polling through the BRR bit in the Interrupt Status register (interrupt or polling)
- Internal DMA

When internal DMA is not used (DMAEN bit in Transfer Type register is not set when the command is sent), uSDHC asserts a DMA request when the amount of data exceeds the value set in the RD_WML register, which is available and ready for system fetching data. At the same time, uSDHC sets the BRR bit. The buffer read ready interrupt is generated if it is enabled by the software.

When internal DMA is used, uSDHC does not inform the system before all the required number of bytes are transferred (if no error is encountered). When an error occurs during the data transfer, uSDHC aborts the data transfer and abandons the current block. The host driver should read the content of the DMA System Address register to get the starting address of the abandoned data block. If the current data transfer is in multi-block mode, uSDHC does not automatically send CMD12, even though the AC12EN bit in the Transfer Type register is set. The host driver sends CMD12 in this scenario and restarts the read operation from that address. It is recommended that a software reset for data be applied before the transfer is restarted.

For any write transfer mode, uSDHC does not start data transmission until the number of words set in the RD_WML register are in buffer. If the buffer is full and the host system does not read data in time, uSDHC stops the CLK to avoid the data buffer overrun situation.

10.3.3.1.3 Data buffer and block size

The user needs to know the buffer size for the buffer operation during a data transfer to utilize it in the most optimized way. In the uSDHC module, the only data buffer can hold up to 128 words (32-bit) and the watermark levels for write and read can be configured accordingly.

For both read and write, the watermark level can range between 1 to 128 words. For both read and write, the burst length can range between from 1 to 31 words. The host driver may configure the value according to the system situation and requirement.

During a multi-block data transfer, the block length can be set to any value between 1 and 4096 bytes, satisfying the requirements of the external card. The only restriction is from the external card, which can be limited in size or support of a partial block access (which is not the integer times of 512 bytes).

As uSDHC treats each block individually, for block sizes which are not multiples of four (not word-aligned), stuffed bytes are required at the end of each block. For example, if the block size is 7 bytes and there are 12 blocks to write, the system side must write twice

for each block. For each block, the ending byte is abandoned by uSDHC because it only sends 7 bytes to the card and picks data from the following system write, resulting in 24 beats of write access in total.

10.3.3.1.4 Dividing large data transfer

This SDIO command CMD53 definition limits the maximum data size of data transfers according to the following formula:

Max data size = Block size x Block count

The length of a multiple block transfer needs to be in block size units. If the total data length cannot be divided evenly into a multiple of the block size, then there are two ways to transfer the data. These two ways depend on the function and the card design. Option 1 is for the host driver to split the transaction. The remainder of the block size data is then transferred by using a single block command at the end. Option 2 is to add dummy data in the last block to fill the block size. For option 2, the card must manage the removal of the dummy data.

See the figure below for an example showing the division of large data transfers, assuming a kind of WLAN SDIO card that only supports a block size of up to 64 bytes. Although uSDHC supports a block size of up to 4096 bytes, the SDIO can only accept a block size of less than 64 bytes, so the data must be divided (see the example below).

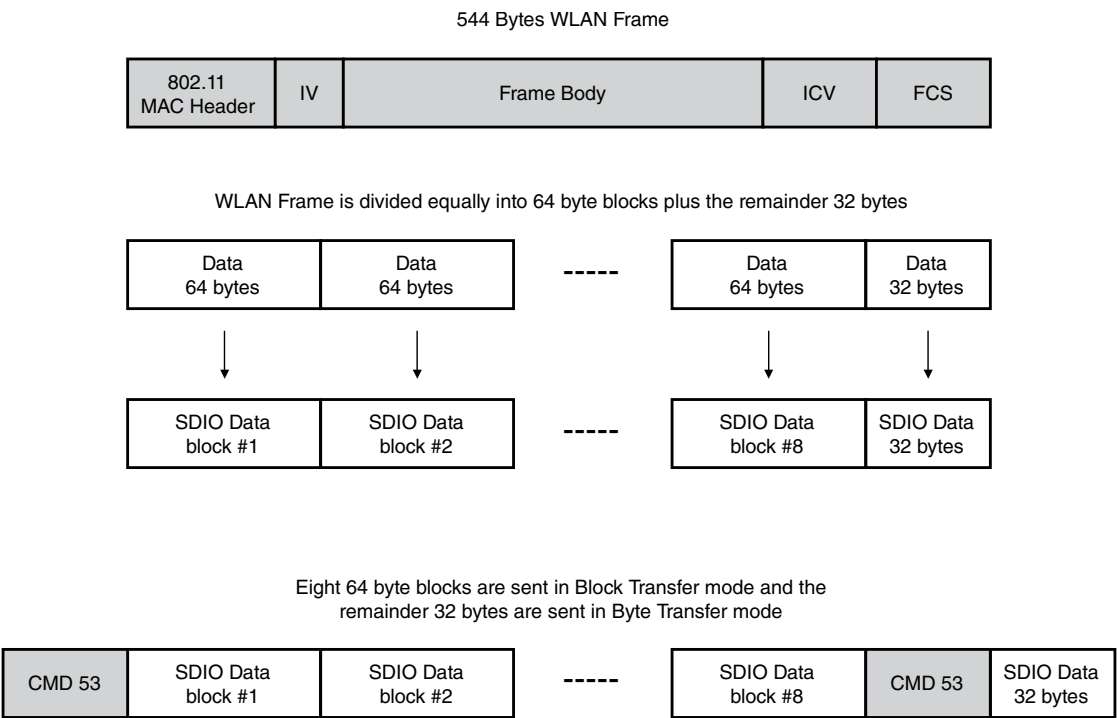


Figure 10-38. Example for dividing large data transfers

10.3.3.1.5 External DMA Request

When the internal DMA is not in use and external DMA is enabled, the Data Buffer will generate a DMA request to the system. During a write operation, when the number of WR_WML words can be held in the buffer free space, the signal uSDHC_dreq_b is asserted to 0, informing the Host System of a DMA write.

The BWR bit in the Interrupt Status register is also set, as long as the BWRSEN bit in the Interrupt Status Enable register is set. The DMA request is de-asserted after several accesses to the Data Port register are made while the buffer's free space can't meet the watermark condition (free space > write watermark level).

On read operation, when the number of RD_WML words are already in the buffer, the signal uSDHC_dreq_b is asserted to 0, informing the Host System for a DMA read. The BRR bit in the Interrupt Status register is also set, as long as the BRRSEN bit in the Interrupt Status Enable register is set. The DMA request is de-asserted after several accesses to the Data Port register are made while the buffer's data can't meet the watermark condition (the number of data in buffer > read watermark level).

If the DMA burst length can't change during a data transfer for an external DMA transfer, the watermark level (read or write) must be a divisor of the block size. If it is not, transferring the block may cause buffer under-run (read operation) or over-run (write operation). For example, if the block size is 512 bytes, the watermark level of read (or write) must be a power of two between 1 and 128. For processor core polling access there is no such issue, as the last access in the block transfer can be controlled by software. The watermark level can be any value, even larger than the block size (but no greater than 128 words) because the actual number of bytes transferred by the software can be controlled and does not exceed the block size in each transfer.

The uSDHC also supports non-word aligned block size, as long as the card supports that block size. In this case, the watermark level should be set as the number of words. For example, if the block size is 31 bytes, the watermark level can be set to any number of words. For this case, the BLKSIZE bits of the Block Attribute register will be set as 1fh. For the CPU polling access, the burst length can be 1 to 128 words, without restriction. This is because the software will transfer 8 words, and the uSDHC will also set the BWR or BRR bits when the remaining data does not violate data buffer. See [DMA burst length](#) for more details about the dynamic watermark level of the data buffer.

For the above example, even though 8 words are transferred via the Data Port register, the uSDHC will transfer only 31 bytes over the SD Bus, as required by the BLKSIZE bits. In this data transfer, with non-word aligned block size, the endian mode should be set cautiously or invalid data will be transferred to and from the card.

10.3.3.2 DMA AHB interface

The internal DMA implements a DMA engine and the AHB master. When the internal DMA is enabled, the `uSDHC_dreq_b` is not asserted during the transfer, but the BWR and BRR bits are set if the BWRSEN and BRRSEN bits have been set in the Interrupt Status Enable register.

See the figure below for an illustration of the DMA AHB interface block.

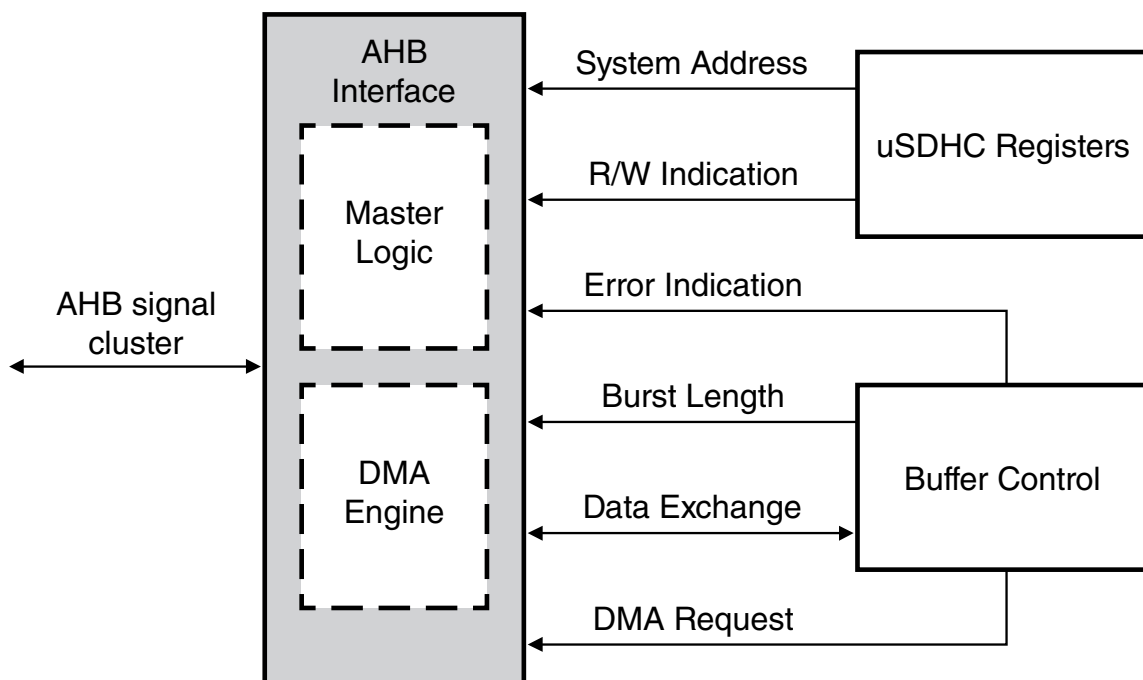


Figure 10-39. DMA AHB interface block

10.3.3.2.1 Internal DMA request

If the watermark level requirement is met in data transfer or if the last data of current block is ready in the data buffer, and the Internal DMA is enabled, the data buffer block sends a DMA request to AHB interface. Meanwhile, the external DMA request signal (`uSDHC_dreq_b`) is disabled.

The delay in response from the internal DMA engine depends on the system AHB bus loading and the priority assigned to uSDHC. The DMA engine does not respond to the request during its burst transfer, but is ready to serve as soon as the burst is over. The data buffer de-asserts the request if the data buffer space (for write) or bytes in data buffer is

smaller than the watermark level. Upon access to the buffer by internal DMA, the data buffer updates its internal buffer pointer, and when the watermark level is satisfied or the last data of the current block is ready in the data buffer, another DMA request is sent.

The data transfer is in the block unit, and the subsequent watermark level is always set as the remaining number of words. For instance, for a multi block data read with each block size of 31 bytes, and the burst length set to six words. After the first burst transfer, if there are more than two words in the buffer, which might contain some data of the next block), another DMA request is sent. This is because the remaining number of words to send for the current block is $(31 - 6 * 4) / 4 = 2$. The uSDHC module reads two words out of the buffer, with seven valid bytes and one stuffed byte.

10.3.3.2.2 DMA burst length

Just like a CPU polling access, the DMA burst length for the internal DMA engine can range between 1 to 16 words. The actual burst length for the DMA depends on the lesser of the configured burst length or the remaining words of the current block. See the example in [Internal DMA request](#). After six words are read, the burst length is two words, then the next burst length is six words. This is because the next block starts, which is 31 bytes, more than six words. The host driver may take this variable burst length into account. It is also acceptable to configure the burst length as the divisor of the block size, so that each time the burst length is the same.

10.3.3.2.3 AHB master interface

It is possible that the internal AHB DMA engine could fail during the data transfer. Upon detection of an AHB bus error during DMA transfer, the DMA engine stops the transfer and goes to the idle state. At that point, the internal data buffer stops receiving incoming data and sending out data. The DMAE bit in the Interrupt Status register is generated to host CPU to report a bus error condition.

After the DMAE interrupt is received, the software sends a CMD12 to abort the current transfer and read the DS_ADDR bits of the DMA System Address register to get the starting address of the corrupted block. After the DMA error is fixed, the software should apply a data reset and restart the transfer from this address to recover the corrupted block. DMA operation resumes when the interrupt is serviced by the software.

10.3.3.2.4 ADMA engine

In the SD host controller standard, a new DMA transfer algorithm called the Advanced DMA (ADMA) is defined. For simple DMA, after the page boundary is reached, a DMA interrupt is generated and the new system address is programmed by the host driver.

The ADMA defines the programmable descriptor table in the system memory. The host driver can calculate the system address at the page boundary and program the descriptor table before executing ADMA. It reduces the frequency of interrupts to the host system. Therefore, higher speed DMA transfers could be realized because the host MCU intervention is not needed during long DMA-based data transfers.

There are two types of ADMA in host controller: ADMA1 and ADMA2. ADMA1 can support data transfer of 4KB aligned data in system memory, and ADMA2 improves the restriction so that the data of any location and any size can be transferred in system memory. Their formats of Descriptor Table are different.

ADMA can recognize all kinds of descriptors defined in SD host controller Standard, and if the "End" flag is detected in the descriptor, ADMA stops after this descriptor is processed.

10.3.3.2.4.1 ADMA concept and descriptor format

ADMA1 includes the following descriptors:

- Valid/invalid descriptor
- Nop descriptor
- Set data length descriptor
- Set data address descriptor
- Link descriptor
- Interrupt flag and end flag in descriptor

ADMA2 includes the following descriptors:

- Valid/invalid descriptor
- Nop descriptor
- Rsv descriptor
- Set data length and address descriptor
- Link descriptor
- Interrupt flag and end flag in descriptor

ADMA starts read/write operation after it reaches the tran state, using the data length and data address analyzed from most recent descriptor(s).

For ADMA1, the valid data length descriptor is the last set type descriptor before the tran type descriptor. Every tran type triggers a transfer, and the transfer data length is extracted from the most recent set type descriptor. If there is no set type descriptor after the previous Trans descriptor, the data length is the value for previous transfer, or 0 if no set descriptor is ever met.

For ADMA2, the tran type descriptor contains both data length and transfer data address, so only a tran type descriptor can start a data transfer.

See the figure below for the format of the descriptor table for ADMA1.

Address/ Page Field		Address/ Page Field		Attribute Field					
31	12	11	6	5	4	3	2	1	0
Address or Data Length		000000		Act 2	Act 1	0	Int	End	Valid

Act 2	Act1	Symbol	Comment	31- 28	27- 12
0	0	Nop	No Operation	Don't Care	
0	1	Set	Set Data Length	0000	Data Length
1	0	Tran	Transfer Data	Data Address	
1	1	Link	Link Descriptor	Descriptor Address	

Valid	Valid = 1 indicates this line of descriptor is effective. If Valid = 0 generate ADMA Error Interrupt and stop ADMA.
End	End = 1 indicates current descriptor is the ending one.
Int	Int = 1 generates DMA Interrupt when this descriptor is processed.

Figure 10-40. Format of the ADMA1 descriptor table

See the figure below for the concept and access method of the descriptor table for ADMA1.

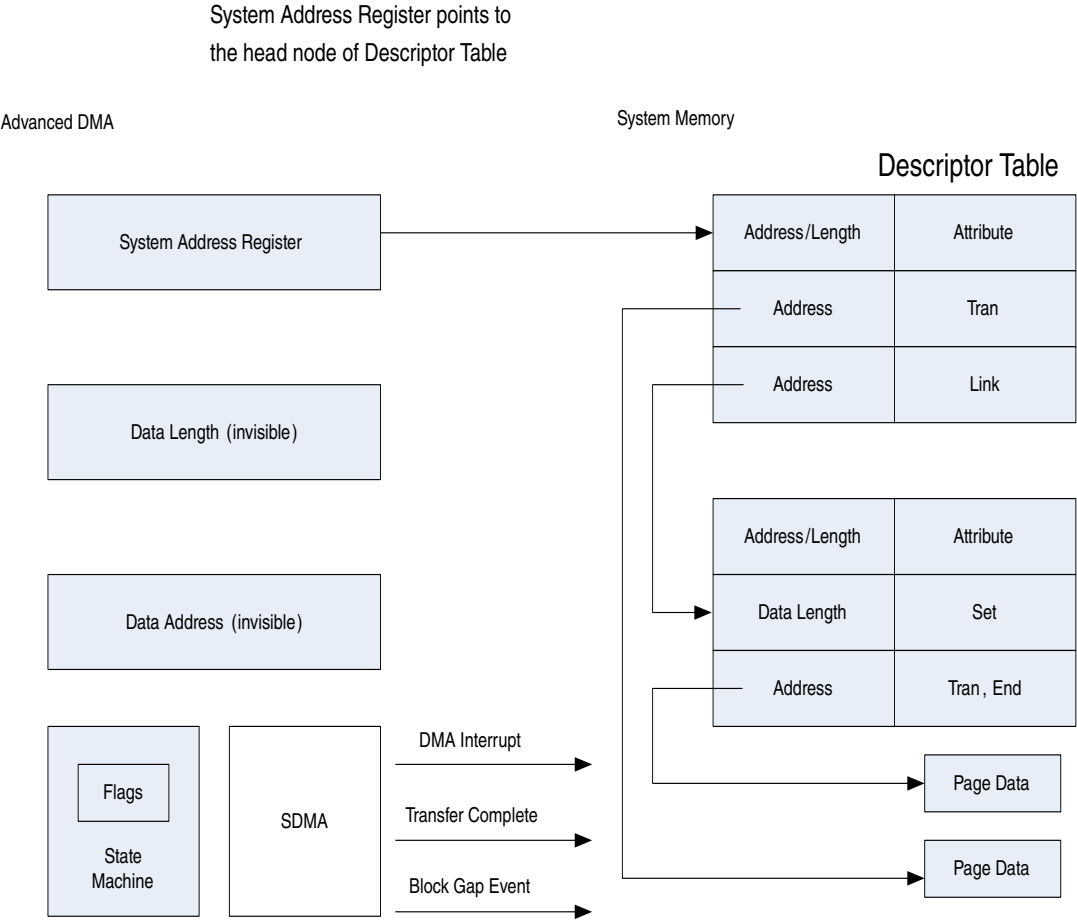


Figure 10-41. Concept and access method of the ADMA1 descriptor table

The figure below explains the ADMA2 format. ADMA2 deals with the lower 32-bit first, and then the higher 32-bit. If the 'Valid' flag of descriptor is 0, it ignores the higher 32-bit. The Address field should be set to word aligned (lower 2-bit is always set to 0). Data length is in byte unit.

Address Field		Length		Reserved		Attribute Field					
63	32	31	16	15	06	05	04	03	02	01	00
32-bit Address		16-bit length		0000000000		Act 2	Act 1	0	Int	End	Valid

Act 2	Act1	Symbol	Comment	Operation
0	0	Nop	No Operation	Don't Care
0	1	Rsv	Reserved	Same as Nop. Read this line and go to next one
1	0	Tran	Transfer Data	Transfer data with address and length set in this descriptor line
1	1	Link	Link Descriptor	Link to another descriptor

Valid	Valid = 1 indicates this line of descriptor is effective. If Valid = 0 generate ADMA Error Interrupt and stop ADMA.
End	End = 1 indicates current descriptor is the ending one.
Int	Int = 1 generates DMA Interrupt when this descriptor is processed.

Figure 10-42. Format of the ADMA2 descriptor table

See the figure below for the concept and access method of the descriptor table for ADMA2.

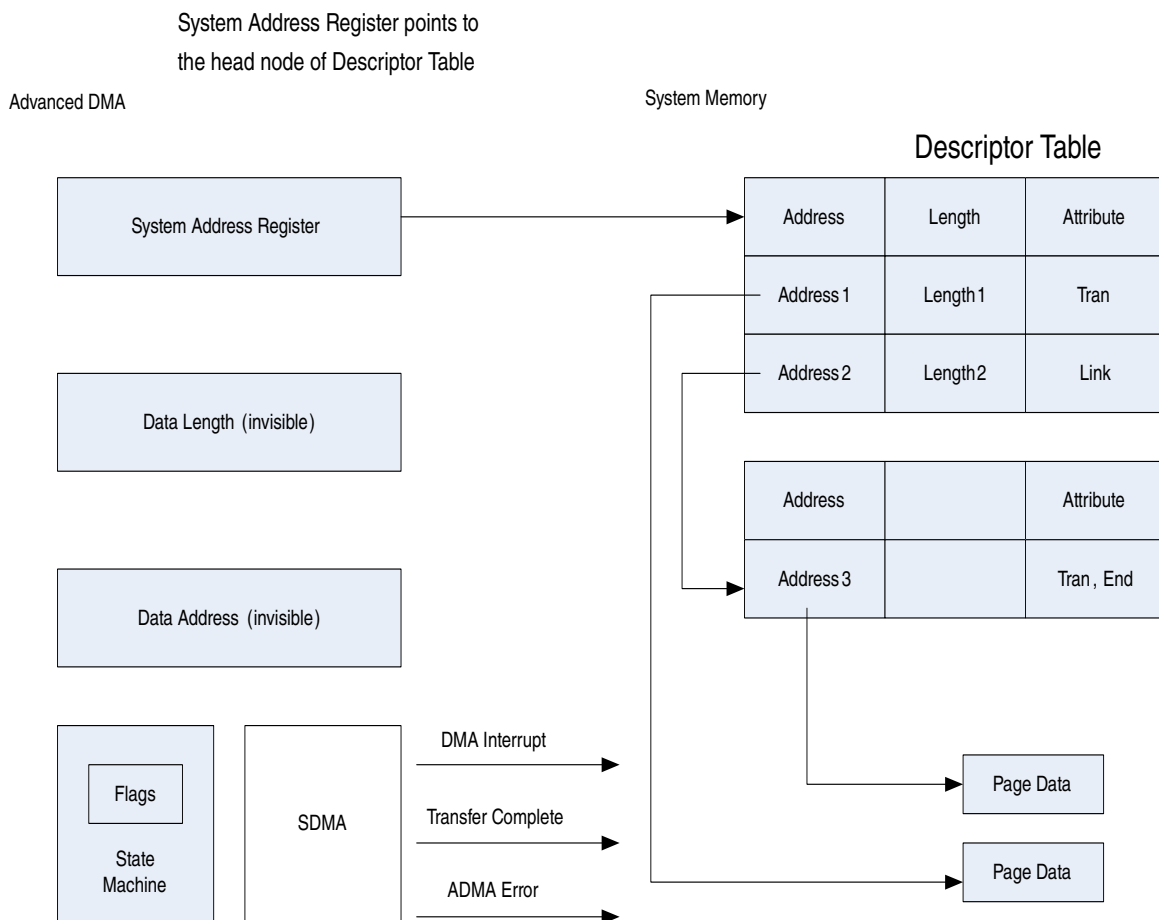


Figure 10-43. Concept and access method of the ADMA2 descriptor table

10.3.3.2.4.2 ADMA interrupt

If the interrupt flag descriptor is set, ADMA generates an interrupt according to various types of descriptors.

For ADMA1:

- Set type of descriptor: interrupt is generated when data length is set.
- Tran type descriptor: interrupt is generated when this transfer is complete.
- Link type of descriptor: interrupt is generated when new descriptor address is set.
- Nop type of descriptor: interrupt is generated just after this descriptor is fetched.

For ADMA2:

- Tran type of descriptor: interrupt is generated when this transfer is complete.
- Link type of descriptor: interrupt is generated when new descriptor address is set.
- Nop/Rsv type of descriptor: interrupt is generated just after this descriptor is fetched.

10.3.3.2.4.3 ADMA error

The ADMA stops whenever an error is encountered. These errors include:

- Fetching descriptor error
- AHB response error
- Data length mismatch error

An ADMA descriptor error is generated when it fails to detect a "valid" flag in the descriptor. If an ADMA descriptor error occurs, the interrupt is not generated even if the "Interrupt" flag of this descriptor is set.

When BLKCNTEN bit is set, data length set in buffer must be equal to the whole data length set in descriptor nodes, otherwise data length mismatch error is generated.

When BLKCNTEN bit is not set, then the whole data length set in descriptor is a multiple of block lengths; otherwise, when data set in the descriptor nodes are not performed at block boundaries, then data mismatch errors occur.

10.3.3.3 Register bank with IP bus interface

Register accesses through the IP bus interface are on the register bank.

See the figure below for the block diagram.

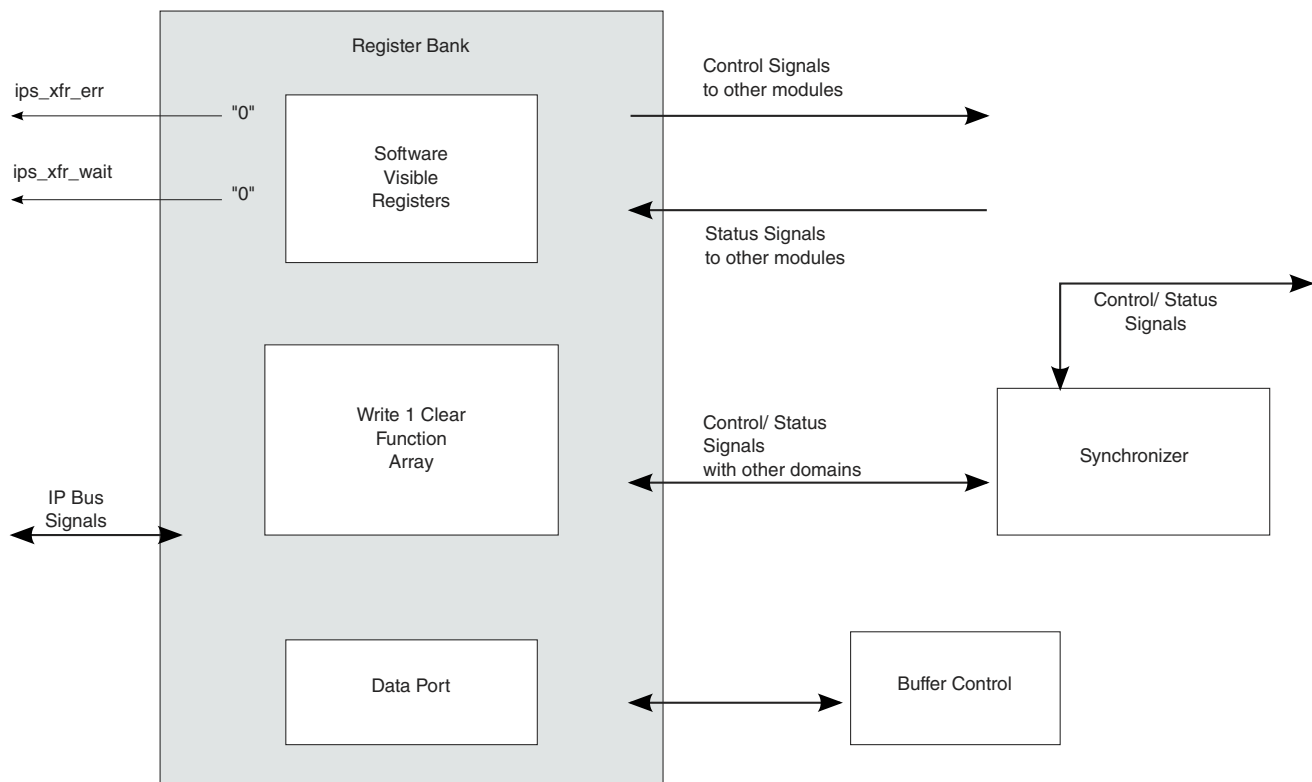


Figure 10-44. Register bank diagram

Only a 32-bit access is allowed, and no partial read/write is supported; therefore, all accesses are word aligned.

10.3.3.3.1 SD protocol unit

The SD protocol unit deals with all SD protocol affairs.

The SD protocol unit performs the following functions:

- Acts as the bridge between the internal buffer and the SD bus
- Sends the command data as well as its argument serially
- Stores the serial response bit stream into corresponding registers
- Detects the bus state on the CMD/DATA lines
- Monitors the interrupt from the SDIO card
- Asserts the read wait signal
- Gates off the SD clock when buffer is announcing danger status
- Detects the write protect state

The SD protocol unit consists of four submodules:

- SD control misc
- Command control

- Data control
- Clock control

10.3.3.3.2 SD control misc

In the SD control misc unit, the card detection (including the CD_B and DATA3 used for card detection), write protection, and card interrupt are implemented.

This module monitors the signal level on all the eight data lines and the command lines. It directly routes the level values into the register bank.

The module also detects the Write Protect (WP) line. If WP is active, writes to the register bank are ignored.

This module also drives the LCTL output signal when the LCTL bit is set by the driver.

10.3.3.3.3 SD clock control

If the internal data buffer is near full (for read) or near empty (for write), the SD clock must be gated off to avoid buffer over/under-run. This module asserts the gate of the output SD clock to shut the clock off. After the buffer has space (for read) or has data (for write), the clock gate of this module opens, and the SD clock is active again.

10.3.3.3.4 Command control

The Command Control module deals with the transactions on the CMD line.

See the figure below for an illustration of the structure for the Command CRC shift register.

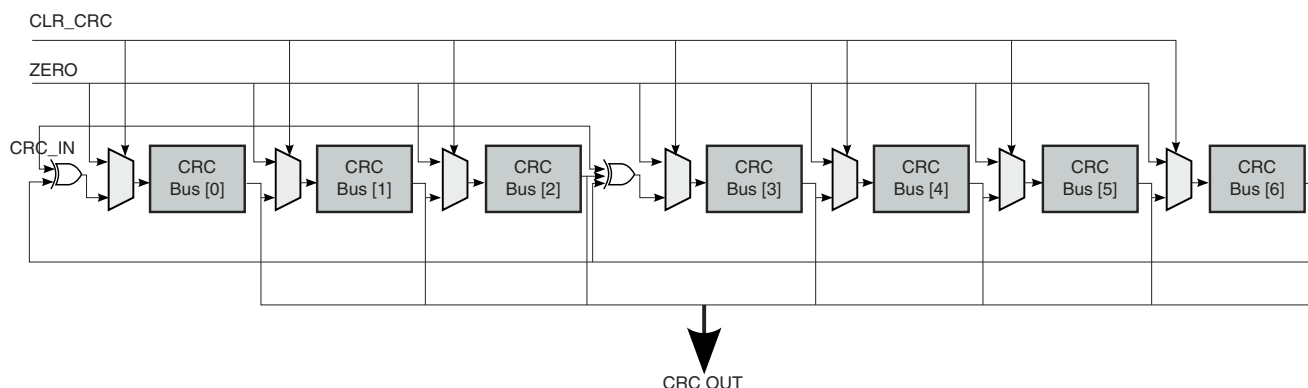


Figure 10-45. Command CRC shift register

The CRC polynomials for the CMD are as follows:

Generator polynomial: $G(x) = x^7 + x^3 + 1$
 $M(x) = (\text{first bit}) * x^n + (\text{second bit}) * x^{n-1} + \dots + (\text{last bit}) * x^0$
 $\text{CRC}[6:0] = \text{Remainder} [(M(x) * x^7) / G(x)]$

10.3.3.3.5 Data control

The data agent deals with the transactions on the eight data lines. Moreover, this module also detects the busy state on the DATA0 line, and generates the read wait state by the request from the transceiver.

The CRC polynomials for the data are as follows:

Generator polynomial: $G(x) = x^{16} + x^{12} + x^5 + 1$
 $M(x) = (\text{first bit}) * x^n + (\text{second bit}) * x^{n-1} + \dots + (\text{last bit}) * x^0$
 $\text{CRC}[15:0] = \text{Remainder} [(M(x) * x^{16}) / G(x)]$

10.3.3.4 Clock and reset manager

This module controls all four kinds reset signals within uSDHC:

- Hardware reset
- Software reset for all logic
- Software reset for the data logic
- Software reset for the command logic

All these signals are fed into this module and stable signals are generated inside the module to reset all other modules. The module also gates off all the inside signals.

10.3.3.5 Clock generator

The clock generator generates the card CLK by peripheral source clock in two stages. The clock divisor can be configured through register [SYS_CTRL \[SDCLKFS\]](#) is for prescaler configuration while the [\[DVS\]](#) is for divisor configuration. Details can be found in the register function description. See the following figure for the structure of the divider. The term "Base" represents the frequency of peripheral source clock.

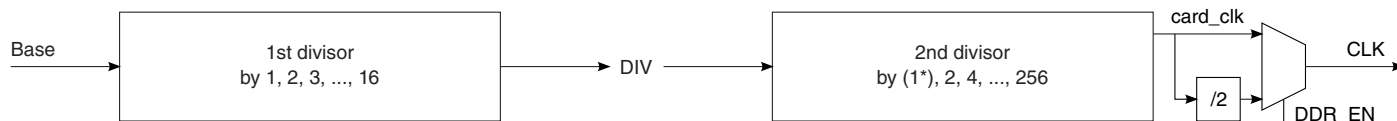


Figure 10-46. Two stages of the clock divider

The first stage outputs an intermediate clock (DIV) that can be Base, Base/2, Base/3, ..., or Base/16.

The second stage is a prescaler and outputs the actual internal working clock (card_clk). This clock is the driving clock for all the sub modules of the SD protocol unit, and helps in syncing FIFOs (see [Figure 10-35](#)) to synchronize with the data rate from the internal data buffer. The frequency of the clock output from this stage can be DIV, DIV/2, DIV/4, ..., or DIV/256. Therefore, the highest frequency of the card_clk is base, and the next highest is Base/2, while the lowest frequency is Base/4096. If the duty cycle of the base clock is 50%, the duty cycle of card_clk is also 50%, even when the compound divisor is an odd value.

NOTE

CLK is different for the SDR and DDR modes.

- In the SDR mode, CLK is equal to the internal working clock (card_clk).
- In the DDR mode, CLK is equal to card_clk/2.

10.3.3.6 SDIO card interrupt

Information on interrupts in 1-bit mode, interrupts in 4-bit mode, and card interrupt handling are detailed in the sections below.

10.3.3.6.1 Interrupts in 1-bit mode

In this case, the DATA1 pin provides the interrupt function. An interrupt is asserted by pulling the DATA1 low from the SDIO card, until the interrupt service is finished to clear the interrupt.

10.3.3.6.2 Interrupt in 4-bit mode

As the interrupt and data line 1 share pin 8 in a 4-bit mode, an interrupt is only sent by the card and recognized by the host during a specific time. This is known as the interrupt period. The uSDHC module only provides sample the level on pin 8 during the interrupt period. At all other times, the host ignores the level on pin 8 and treats it as the data signal. The definition of the interrupt period is different for operations with single block and multiple block data transfers.

In the case of normal single data block transmissions, the interrupt period becomes active two clock cycles after the completion of a data packet. This interrupt period lasts until after the card receives the end bit of the next command that has a data block transfer associated with it.

For multiple block data transfers in a 4-bit mode, there is only a limited period of time that the interrupt period can be active because of the limited period of data line availability between the multiple blocks of data. This requires stricter definition of the interrupt period. For this case, the interrupt period is limited to two clock cycles. This begins two clocks after the end bit of the previous data block. During this 2-clock cycle interrupt period, if an interrupt is pending, the DATA1 line holds low for one clock cycle with the last clock cycle pulling DATA1 high. On completion of the interrupt period, the card releases the DATA1 line into the high Z state. The uSDHC module provides sample of the DATA1 during the interrupt period when the IABG bit in the Protocol Control register is set.

Refer to SDIO Card Specification v1.10 for further information about the SDIO card interrupt.

10.3.3.6.3 Card interrupt handling

When the CINTIEN bit in the Interrupt Signal Enable Register is set to 0, uSDHC clears the interrupt request to the host system. The host driver should clear this bit before servicing the SDIO interrupt and should set this bit again after all interrupt requests from the card are cleared to prevent inadvertent interrupts.

The SDIO Card Interrupt Status can be cleared by writing 1 to this bit. But as the interrupt source from the SDIO card does not clear, this bit is set again. To clear this bit, it is required to reset the interrupt source from the external card followed by writing 1 to this bit. In a 1-bit mode, uSDHC detects the SDIO interrupt with or without the SD clock (to support wakeup). In a 4-bit mode, the interrupt signal is sampled during the interrupt period, so there are some sample delays between the interrupt signal from the SDIO card and the interrupt to the Host System Interrupt Controller. When the SDIO status is set and the host driver needs to service this interrupt, the SDIO bit in the Interrupt Control Register of SDIO card is cleared. This is required to clear the SDIO interrupt status latched in uSDHC and to stop driving the interrupt signal to the System Interrupt Controller. The host driver must issue a CMD52 to clear the card interrupt. After completion of the card interrupt service, the SDIO Interrupt Status Enable bit is set to 1 and uSDHC starts sampling the interrupt signal again.

See the figure below for an illustration of the SDIO card interrupt scheme and for the sequences of software and hardware events that take place during a card interrupt handling procedure.

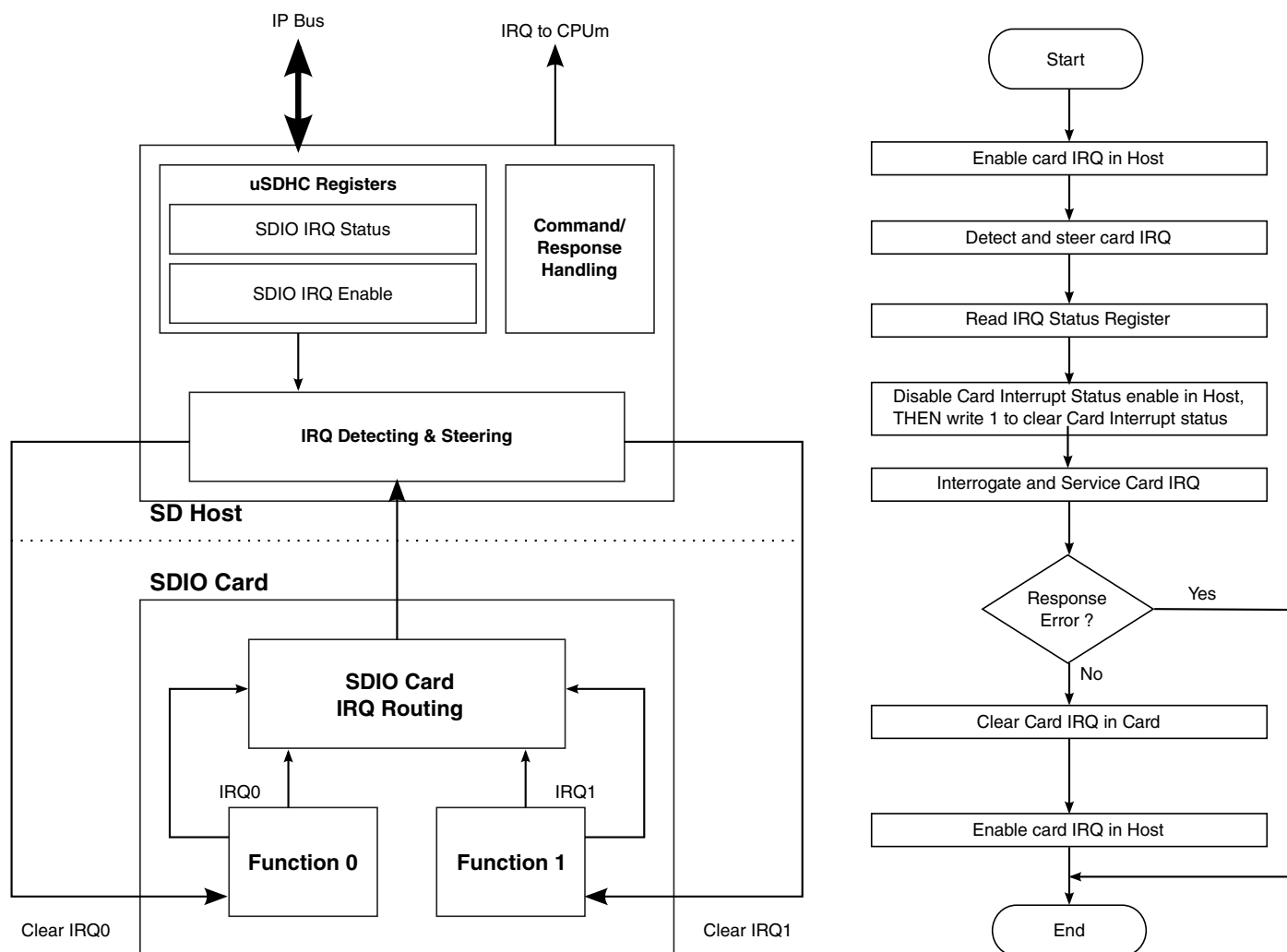


Figure 10-47. Card interrupt scheme, card interrupt detection, and handling procedure

10.3.3.7 Card insertion and removal detection

The uSDHC module uses either the DATA3 pin or the CD_B pin to detect card insertion or removal. When there is no card on the MMC/SD bus, the DATA3 is pulled to a low voltage level by default.

When any card is inserted to or removed from the socket, uSDHC detects the logic value changes on the DATA3 pin and generates an interrupt. When the DATA3 pin is not used for card detection (for example, it is implemented in GPIO), the CD_B pin must be connected for card detection. Whether DATA3 is configured for card detection or not, the CD_B pin is always a reference for card detection. Whether the DATA3 pin or the CD_B pin is used to detect card insertion, uSDHC sends an interrupt (if enabled) to inform the Host system that a card is inserted.

10.3.3.8 Power management and wakeup events

When there is no operation between uSDHC and the card through the SD bus, the user can completely disable the peripheral clock and base clock in the chip-level clock control module to save power. When the user needs to use uSDHC to communicate with the card, it can enable the clock and start the operation.

In some circumstances, when the clocks to uSDHC are disabled, for instance, when the system is in low-power mode, there are some events for which the user needs to enable the clock and handle the event. These events are called wakeup interrupts. The uSDHC module can generate these interrupts even when there are no clocks enabled. The three interrupts that can be used as wakeup events are these:

- Card removal interrupt
- Card insertion interrupt
- Interrupt from SDIO card

The uSDHC module offers a power management feature. By clearing the clock enabled bits in the System Control register, the clocks are gated in the low position to uSDHC. For maximum power saving, the user can disable all the clocks to uSDHC when there is no operation in progress.

These three wakeup events (or wakeup interrupts) can also be used to wakeup the system from low-power modes.

NOTE

To make the interrupt a wakeup event, when all the clocks to uSDHC are disabled or when the entire system is in low power mode, the corresponding wakeup enabled bit needs to be set. Refer to [Protocol Control \(PROT_CTRL\)](#) for more information on the uSDHC Protocol Control register.

10.3.3.8.1 Setting wakeup events

For uSDHC to respond to a wakeup event, the software must set the respective wakeup enable bit before the CPU enters the sleep mode.

Before the software disables the host clock, it should ensure that all the following conditions have been met:

- No read or write transfer is active
- Data and command lines are not active
- No interrupts are pending
- Internal data buffer is empty

10.3.3.9 MMC fast boot

The Embedded MultiMediaCard (eMMC4.3) specification adds a fast boot feature that requires hardware support. There are two types of fast boot modes in the eMMC4.3 specification: boot operation and alternative boot operation in the eMMC4.3 specification. Each type also has with-acknowledge and without-acknowledge modes.

In the boot operation mode, the master (MultiMediaCard host) can read boot data from the slave (MMC device) by keeping CMD line low after power-on, or sending CMD0 with argument + 0xFFFFFFFF (optional for slave), before issuing CMD1.

NOTE

For the eMMC4.3 card setting, please see the eMMC4.3 specification.

10.3.3.9.1 Boot operation

If the CMD line is held low for 74 clock cycles and more after power-up before the first command is issued, the slave recognizes that boot mode is being initiated and starts preparing boot data internally.

Within one second after the CMD line goes low, the slave starts to send the first boot data to the master on the DATA line(s). The master must keep the CMD line low to read all of the boot data.

NOTE

For the purposes of this documentation, fast boot is called "normal fast boot mode".

If boot acknowledge is enabled, the slave has to send acknowledge pattern '010' to the master within 50ms after the CMD line goes low. If boot acknowledge is disabled, the slave does not send out acknowledge pattern '010'.

The master can terminate the boot mode with the CMD line high.

The boot operation is terminated when all the contents of the enabled boot data are sent to the master. After the boot operation is executed, the slave gets ready for the CMD1 operation and the master needs to start a normal MMC initialization sequence by sending CMD1.

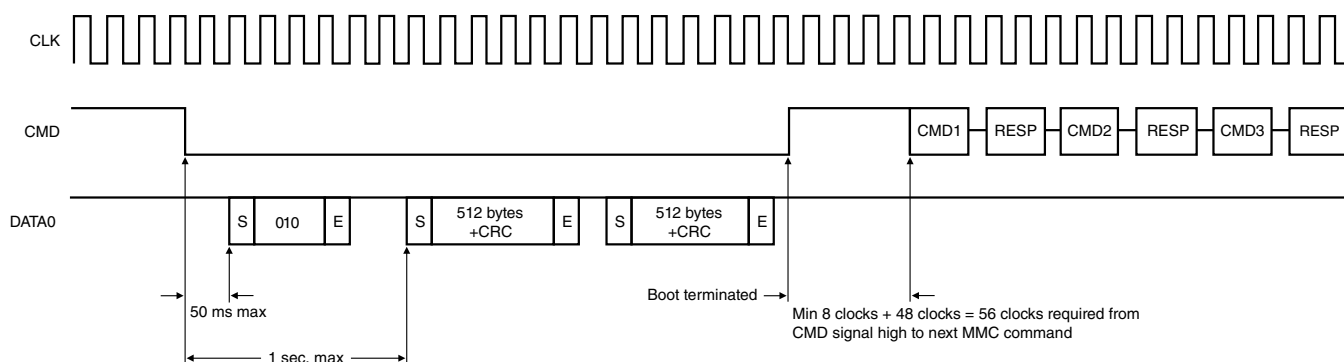


Figure 10-48. MultiMediaCard state diagram (normal boot mode)

10.3.3.9.2 Alternative boot operation

This boot function is optional for the device. If bit 0 in the extended CSD byte[228] is set to '1', the device supports the alternative boot operation.

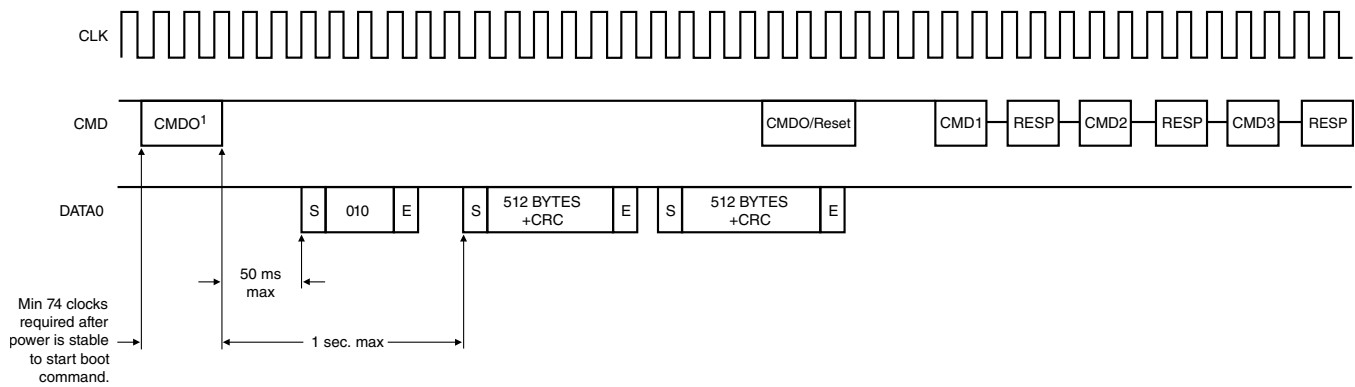
After power-up, if the host issues CMD0 with the argument of 0xFFFFFFFFFA after 74 clock cycles, before CMD1 is issued or the CMD line goes low, the slave recognizes that boot mode is being initiated and starts preparing boot data internally.

Within one second after CMD0 with the argument of 0xFFFFFFFFFA is issued, the slave starts to send the first boot data to the master on the DATA line(s).

If boot acknowledge is enabled, the slave has to send the acknowledge pattern '010' to the master within 50ms after the CMD0 with the argument of 0xFFFFFFFFFA is received. If boot acknowledge is disabled, the slave does not send out acknowledge pattern '010'.

The master can terminate the boot mode by issuing CMD0 (Reset).

Boot operation is terminated when all the contents of the enabled boot data are sent to the master. After the boot operation is executed, the slave gets ready for the CMD1 operation and the master needs to start a normal MMC initialization sequence by sending CMD1.



NOTE 1: CMD0 with argument 0xFFFFFFFF

Figure 10-49. MultiMediaCard state diagram (alternative boot mode)

10.3.4 Initialization/application of uSDHC

All communication between the system and cards are controlled by the host. The host sends commands of two types: broadcast and addressed (point-to-point).

Broadcast commands are intended for all cards, such as GO_IDLE_STATE, SEND_OP_COND, and ALL_SEND_CID. In the Broadcast mode, all cards are in the open-drain mode to avoid bus contention. Refer to [Commands for MMC/SD/SDIO](#) for the commands of bc and bcr categories.

After the broadcast command CMD3 is issued, the cards enter the standby mode. Addressed type commands are used from this point. In this mode, the CMD/DATA I/O pads turn to the push-pull mode to have the driving capability for maximum frequency operation. Refer to [Commands for MMC/SD/SDIO](#) for the commands of ac and adtc categories.

10.3.4.1 Command send & response receive basic operation

Assuming that the data type WORD is an unsigned 32-bit integer, the flow indicated below presents a guideline for sending a command to the card(s):

```
send_command(cmd_index, cmd_arg, other requirements)
{
    WORD wCmd; // 32-bit integer to make up the data to write into Transfer Type register, it is
    recommended to implement in a bit-field manner
    wCmd = (<cmd_index> & 0x3f) >> 24; // set the first 8 bits as '00'+<cmd_index>
    set CMDTYP, DPSEL, CICEN, CCCEN, RSTYP, DTDSEL according to the command index;
    if (internal DMA is used) wCmd |= 0x1;
    if (multi-block transfer) {
        set MSBSEL bit;
        if (finite block number) {
            set BCEN bit;
            if (auto12 command is to use) set AC12EN bit;
        }
    }
}
```

```
}  
write_reg(CMDARG, <cmd_arg>); // configure the command argument  
write_reg(XFERTYP, wCmd); // set Transfer Type register as wCmd value to issue the command  
}  
wait_for_response(cmd_index)  
{  
while (CC bit in IRQ Status register is not set); // wait until Command Complete bit is set  
read IRQ Status register and check if any error bits about Command are set  
if (any error bits are set) report error;  
write 1 to clear CC bit and all Command Error bits;  
}
```

For the sake of simplicity, the function `wait_for_response` is implemented here by means of polling. For an effective and formal way, the response is usually checked after the Command Complete Interrupt is received. When doing this, make sure that the corresponding interrupt status bits are enabled.

For some scenarios, the response time-out is expected. For instance, after all cards respond to CMD3 and move to the Standby state no response to the Host when CMD2 is sent. The host driver deals with "fake" errors like this with caution.

10.3.4.2 Card identification mode

When a card is inserted to the socket or the card is reset by the host, the host needs to validate the operation voltage range, identify the cards, request the cards to publish the Relative Card Address (RCA) or to set the RCA for MMC cards.

10.3.4.2.1 Card detect

See the figure below for a flow diagram showing the detection of MMC, SD, and SDIO cards using uSDHC.

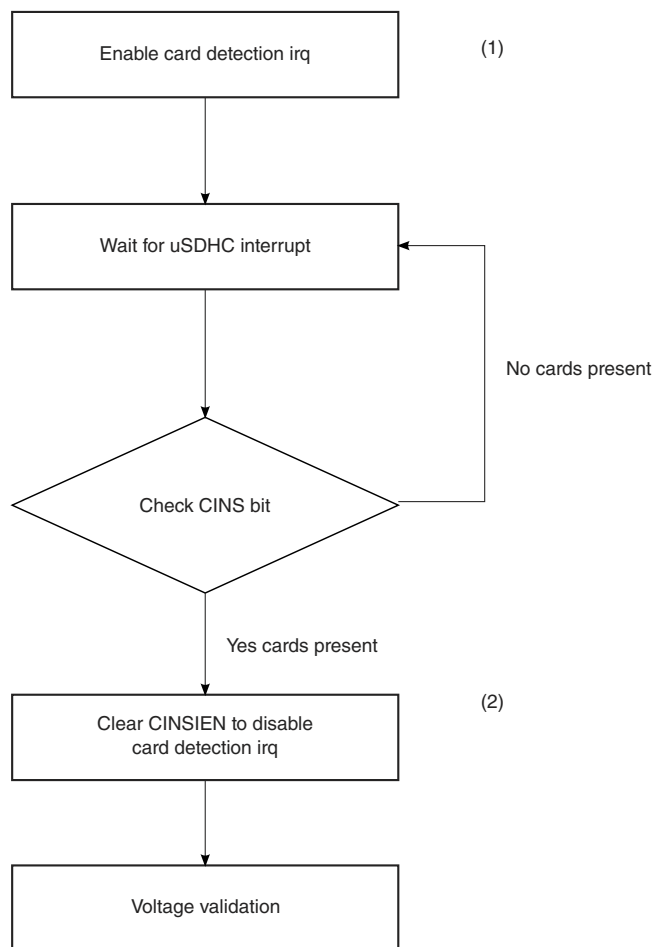


Figure 10-50. Flow diagram for card detection

Here is the card detect sequence:

- Set the CINSIEN bit to enable card detection interrupt.
- When an interrupt from uSDHC is received, check the CINS bit in the Interrupt Status register to see if it was caused by card insertion.
- Clear the CINSIEN bit to disable the card detection interrupt and ignore all card insertion interrupts afterwards.

10.3.4.2.2 Reset

The host consists of three types of resets:

- Hardware reset (Card and Host) that is driven by Power On Reset (POR).

- Software reset (Host only) is initiated by the write operation on the RSTD, RSTC, or RSTA bits of the System Control register to reset the data part, command part, or all parts of the host controller, respectively.
- Card reset (Card only): The command, "Go_Idle_State" (CMD0), is the software reset command for all types of MMC cards and SD memory cards. This command sets each card into the idle state regardless of the current card state. For an SD I/O card, CMD52 is used to write an I/O reset in CCCR. The cards are initialized with a default relative card address (RCA=0x0000) and with a default driver stage register setting (lowest speed, highest driving current capability).

After the card is reset, the host needs to validate the voltage range of the card. See the figure below for the software flow to reset both uSDHC and the card.

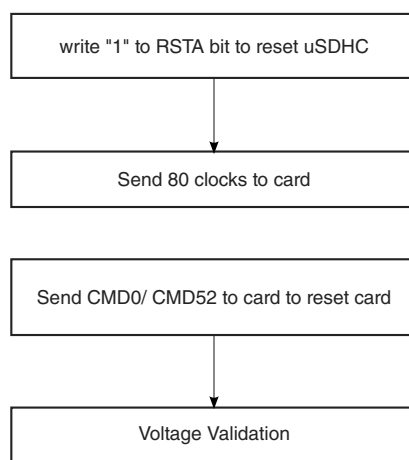


Figure 10-51. Flow chart for resetting uSDHC and SD I/O card

```

software_reset()
{
  set_bit(SYSCTRL, RSTA); // software reset the Host
  set DTOCV and SDCLKFS bit fields to get the CLK of frequency around 400kHz
  configure IO pad to set the power voltage of external card to around 3.0V
  poll bits CIHB and CDIHB bits of PRSSTAT to wait both bits are cleared
  set_bit(SYSCTRL, INTIA); // send 80 clock ticks for card to power up
  send_command(CMD_GO_IDLE_STATE, <other parameters>); // reset the card with CMD0
  or send_command(CMD_IO_RW_DIRECT, <other parameters>);
}
  
```

10.3.4.2.3 Voltage validation

All cards should be able to establish communication with the host using any operation voltage in the maximum allowed voltage range specified in the card specification. However, the supported minimum and maximum values for VDD are defined in the Operation Conditions Register (OCR) and may not cover the whole range.

Cards that store the CID and CSD data in the preload memory are only able to communicate this information under data transfer VDD conditions. This means that if the host and card have non-common VDD ranges, the card is neither able to complete the identification cycle nor able to send CSD data.

Therefore, special commands `Send_Op_Cont` (CMD1 for MMC), `SD_Send_Op_Cont` (ACMD41 for SD Memory), and `IO_Send_Op_Cont` (CMD5 for SD I/O) are used. The voltage validation procedure is designed to provide a mechanism to identify and reject cards that do not match the VDD range(s) desired by the host. This is accomplished when the host sends the desired VDD voltage window as the operand of this command. Cards that cannot perform the data transfer in the specified range must discard themselves from further bus operations and go into the Inactive state. By omitting the voltage range in the command, the host can query each card and determine the common voltage range before sending out-of-range cards into the Inactive state. This query should be used if the host is able to select a common voltage range or if a notification is sent to the system when a non-usable card in the stack is detected.

The following steps show how to perform voltage validation when a card is inserted:

```

voltage_validation(voltage_range_arguement)
{
    label the card as UNKNOWN;
    send_command(IO_SEND_OP_COND, 0x0, <other parameters are omitted>); // CMD5, check SDIO
    operation voltage, command argument is zero
    if (RESP_TIMEOUT != wait_for_response(IO_SEND_OP_COND)) { // SDIO command is accepted
        if (0 < number of IO functions) {
            label the card as SDIO;
            IORDY = 0;
            while (!(IORDY in IO OCR response)) { // set voltage range for each IO
                function
                    send_command(IO_SEND_OP_COND, <voltage range>, <other
parameter>);
                    wait_for_response(IO_SEND_OP_COND);
            } // end of while ...
        } // end of if (0 < ...
        if (memory part is present inside SDIO card) label the card as SDCombo; // this is
an
SD-Combo card
    } // end of if (RESP_TIMEOUT ...
    if (the card is labelled as SDIO card) return; // card type is identified and voltage range
is
set, so exit the function;
    send_command(APP_CMD, 0x0, <other parameters are omitted>); // CMD55, Application specific
CMD
prefix
    if (no error calling wait_for_response(APP_CMD, <...>)) { // CMD55 is accepted
        send_command(SD_APP_OP_COND, <voltage range>, <...>); // ACMD41, to set voltage
range
        for memory part or SD card
            wait_for_response(SD_APP_OP_COND); // voltage range is set
            if (card type is UNKNOWN) label the card as SD;
            return; //
    } // end of if (no error ...
    else if (errors other than time-out occur) { // command/response pair is corrupted
        deal with it by program specific manner;
    } // of else if (response time-out
    else { // CMD55 is refuse, it must be MMC card if (card is already labelled as SDCombo)
    { //
change label

```

```

        re-label the card as SDIO;
        ignore the error or report it;
        return; // card is identified as SDIO card
    } // of if (card is ...
    send_command(SEND_OP_COND, <voltage range>, <...>);
    if (RESP_TIMEOUT == wait_for_response(SEND_OP_COND)) { // CMD1 is not accepted,
either
        label the card as UNKNOWN;
        return;
    } // of if (RESP_TIMEOUT ...
} // of else
}

```

10.3.4.2.4 Card registry

Card registry for the MMC and SD/SDIO/SD combo cards are different. For the SD card, the identification process starts at a clock rate lower than 400 kHz and the power voltage higher than 2.7 V (as defined by the card spec). Currently, the CMD line output drives are push-pull drivers instead of open-drain. After the bus is activated, the host requests the card to send their valid operation conditions.

The response to ACMD41 is the operation condition register of the card. The same command is sent to all the new cards in the system. Incompatible cards are put into the Inactive state. The host then issues the command, All_Send_CID (CMD2), to each card to get its unique card identification (CID) number. Cards that are currently unidentified (in the Ready state), send their CID number as the response. After the CID is sent by the card, the card goes into the Identification state.

The host then issues Send_Relative_Addr (CMD3), requesting the card to publish a new relative card address (RCA) that is shorter than the CID. This RCA is used to address the card for future data transfer operations. After the RCA is received, the card changes its state to the Standby state. At this point, if the host wants the card to have an alternative RCA number, it may ask the card to publish a new number by sending another Send_Relative_Addr command to the card. The last published RCA is the actual RCA of the card.

The host repeats the identification process with CMD2 and CMD3 for each card in the system until the last CMD2 gets no response from any of the cards in the system.

For MMC operation, the host starts the card identification process in the open-drain mode with the identification clock rate lower than 400 kHz and the power voltage higher than 2.7 V. The open drain driver stages on the CMD line to allow parallel card operation during card identification. After the bus is activated, the host requests the cards to send their valid operation conditions (CMD1). The response to CMD1 is the "wired OR" operation on the condition restrictions of all cards in the system. Incompatible cards are sent into the Inactive state. The host then issues the broadcast command All_Send_CID (CMD2), asking all cards for their unique card identification (CID) number. All unidentified cards (the cards in the Ready state) simultaneously start sending their CID

numbers serially, while bit-wise monitoring their outgoing bit stream. Those cards, whose outgoing CID bits do not match the corresponding bits on the command line in any one of the bit periods, stop sending their CID immediately and must wait for the next identification cycle. As the CID is unique for each card, only one card can be successfully sent its full CID to the host. This card then goes into the Identification state. Thereafter, the host issues Set_Relative_Addr (CMD3) to assign a relative card address (RCA) to the card. After the RCA is received, the state of the card changes to standby, and the card does not react in further identification cycles. Also, its output driver switches from open-drain to push-pull. The host repeats the process, mainly CMD2 and CMD3, until the host receives a time-out condition to recognize the completion of the identification process.

The following steps show how to perform an operation using MMC cards:

```
card_registry()
{
do { // decide RCA for each card until response time-out
    if(card is labelled as SDCombo or SDIO) { // for SDIO card like device
        send_command(SET_RELATIVE_ADDR, 0x00, <...>); // ask SDIO card to
publish its
RCA
        retrieve RCA from response;
    } // end if (card is labelled as SDCombo ...
else if (card is labelled as SD) { // for SD card
    send_command(ALL_SEND_CID, <...>);
    if (RESP_TIMEOUT == wait_for_response(ALL_SEND_CID)) break;
    send_command(SET_RELATIVE_ADDR, <...>);
    retrieve RCA from response;
} // else if (card is labelled as SD ...
else if (card is labelled as MMC) {
    send_command(ALL_SEND_CID, <...>);
    rca = 0x1; // arbitrarily set RCA, 1 here for example
    send_command(SET_RELATIVE_ADDR, 0x1 << 16, <...>); // send RCA at upper
16
bits
    } // end of else if (card is labelled as MMC ...
} while (response is not time-out);
}
```

10.3.4.3 Card access

Information about Block Write, Block Read, Suspend Resume, ADMA Usage, Transfer Error, and Card Interrupt are detailed in the sections below.

10.3.4.3.1 Block write

Information on Normal Write, DDR Write, and Write with Pause are detailed in the sections below.

10.3.4.3.1.1 Normal Write

During a block write (CMD24 - 27, CMD60, CMD61), one or more blocks of data are transferred from the host to the card with a CRC appended to the end of each block by the host. If the CRC fails, the card indicates that the failure on the DATA line. The transferred data is discarded and not written, and all further transmitted blocks (in multiple block write mode) are ignored.

If the host uses partial blocks whose accumulated length is not block aligned and block misalignment is not allowed (CSD parameter WRITE_BLK_MISALIGN is not set), the card detects the block misalignment error and aborts the programming before the beginning of the first misaligned block. The card sets the ADDRESS_ERROR error bit in the status register, and while ignoring all further data transfer, waits in the Receive-data-State for a stop command. The write operation is also aborted if the host tries to write over a write-protected area.

For MMC and SD cards, programming of the CID and CSD registers does not require a previous block length setting. The transferred data is also CRC protected. If a part of the CSD or CID register is stored in ROM, then this unchangeable part must match the corresponding part of the receive buffer. If this match fails, then the card reports an error and does not change any register contents.

For all types of cards, some may require long and unpredictable periods of time to write a block of data. After receiving a block of data and completing the CRC check, the card begins writing and holds the DATA line low if its write buffer is full and unable to accept new data from a new WRITE_BLOCK command. The host may poll the status of the card with a SEND_STATUS command (CMD13) or other means for SDIO cards at any time, and the card responds with its status. The responded status indicates whether the card can accept new data or whether the write process is still in progress. The host may deselect the card by issuing a CMD7 (to select a different card) to place the card into the Standby state and release the DATA line without interrupting the write operation. When re-selecting the card, it reactivates the busy indication by pulling data to low if the programming is still in progress and the write buffer is unavailable.

The software flow to write to a card incorporates the internal DMA and the write operation is a multi-block write with the Auto CMD12 enabled. For the other two methods (by means of external DMA or CPU polling status) with different transfer methods, the internal DMA parts should be removed and the alternative steps should be straightforward.

The software flow to write to a card is described below:

1. Check the card status, wait until the card is ready for data.
2. Set the card block length/size:

- For SD/MMC cards, use SET_BLOCKLEN (CMD16).
 - For SDIO cards or the I/O portion of SDCombo cards, use IO_RW_DIRECT (CMD52) to set the I/O Block Size bit field in the CCCR register (for function 0) or FBR register (for functions 1~7).
3. Set the uSDHC block length register to be the same as the block length set for the card in step 2.
 4. Set the uSDHC number block register (NOB), where nob is 5, for instance.
 5. Disable the buffer write ready interrupt, configure the DMA settings and enable the uSDHC DMA when sending the command with data transfer. The AC12EN bit should also be set.
 6. Wait for the Transfer Complete interrupt.
 7. Check the status bit to see if a write CRC error occurred, or another error that occurred during the auto12 command sending and response receiving.

10.3.4.3.1.2 DDR write

uSDHC supports the dual data rate mode.

The software flow to write to a card in the DDR mode is described as below:

1. Check the card status and wait until the card is ready for data.

NOTE

For eMMC4.4 card, block length only can be set to 512byte.

2. Set the uSDHC number block register (NOB), where nob is 5, for instance.
3. Set the eMMC4.4 card to high-speed mode and use SWITCH(CMD6).
4. Set the eMMC4.4 card bus with 4-bit/8-bit DDR mode and use SWITCH(CMD6).
5. Disable the buffer write ready interrupt, configure the DMA settings and enable the uSDHC DMA when sending the command with data transfer. The DDR_EN and AC12EN bits should be set.
6. Wait for the Transfer Complete interrupt.
7. Check the status bit to see if a write CRC error occurred or another error that occurred during the auto12 command sending and response receiving.

10.3.4.3.1.3 Write with Pause

The write operation can be paused during the transfer. Instead of stopping the CLK at any time to pause all the operations, which is also inaccessible to the host driver, the driver can set the Stop At Block Gap Request (SABGREQ) bit in the Protocol Control register to pause the transfer between the data blocks. As there is no time-out condition in a write

operation during the data blocks, a write to all types of cards can be paused in this way, and if the DATA0 line is not required to de-assert to release the busy state, no suspend command is needed.

Similar to the flow described in [Normal Write](#), the write with pause is shown with the same kind of write operation:

1. Check the card status and wait until the card is ready for data.
2. Set the card block length/size:
 - For SD/MMC, use SET_BLOCKLEN (CMD16).
 - For SDIO cards or the I/O portion of SDCombo cards, use IO_RW_DIRECT(CMD52) to set the I/O Block Size bit field in the CCCR register (for function 0) or FBR register (for functions 1~7).
3. Set the uSDHC block length register to be the same as the block length set for the card in step 2.
4. Set the uSDHC number block register (NOB), where nob is 5, for instance.
5. Disable the buffer write ready interrupt, configure the DMA settings and enable the uSDHC DMA when sending the command with data transfer. The AC12EN bit should also be set.
6. Set the SABGREQ bit.
7. Wait for the Transfer Complete interrupt.
8. Clear the SABGREQ bit.
9. Check the status bit to see if a write CRC error occurred.
10. Set the CREQ bit to continue the write operation.
11. Wait for the Transfer Complete interrupt.
12. Check the status bit to see if a write CRC error occurred or some another error that occurred during the auto12 command sending and response receiving.

The number of blocks left during the data transfer is accessible by reading the contents of the BLKCNT field in the Block Attribute register. As the data transfer and the setting of the SABGREQ bit are concurrent, and the delay of register read and the register setting, the actual number of blocks left may not be exactly the value read earlier. The host driver reads the value of BLKCNT after the transfer is paused and the Transfer Complete interrupt is received.

It is also possible that the last block has begun when the Stop At Block Gap Request is sent to the buffer. In this case, the next block gap is the end of the transfer. These types of requests are ignored, the driver should treat these as a non-pause transfer, and deal with it as a common write operation.

When the write operation is paused, the data transfer inside the host system is not stopped, and the transfer is active until the data buffer is full. Because of this, it is recommended to avoid using the Suspend command for the SDIO card. This is because when such a command is sent, uSDHC interprets the system that switches to another

function on the SDIO card and flush the data buffer. uSDHC takes the Resume command as a normal command with data transfer, and it is left for the host driver to set all the relevant registers before the transfer is resumed. If there is only one block to send when the transfer is resumed, the MSBSEL and BCEN bits of the Transfer Type register are set as well as the AC12EN bit. However, the uSDHC module automatically sends a CMD12 to mark the end of the multi-block transfer.

10.3.4.3.2 Block read

Information about Normal read, DDR read, Read with Pause, and Delay Line (DLL) in Read Path are detailed in the sections below.

10.3.4.3.2.1 Normal read

For block reads, the basic unit of data transfer is a block whose maximum size is stored in areas defined by the corresponding card specification. A CRC is appended to the end of each block, ensuring data transfer integrity. The CMD17, CMD18, CMD53, CMD60, CMD61, and so on, can initiate a block read. After completing the transfer, the card returns to the Transfer state. For multi blocks read, data blocks are continuously transferred until a stop command is issued.

The software flow to read from a card incorporates the internal DMA and the read operation is a multi-block read with the Auto CMD12 enabled. For the other two methods (by means of external DMA or CPU polling status) with different transfer methods, the internal DMA parts should be removed and the alternative steps should be straightforward.

The software flow to read from a card is described below:

1. Check the card status and wait until card is ready for data.
2. Set the card block length/size:
 - For SD/MMC, use SET_BLOCKLEN (CMD16).
 - For SDIO cards or the I/O portion of SDCombo cards, use IO_RW_DIRECT(CMD52) to set the I/O Block Size bit field in the CCCR register (for function 0) or FBR register (for functions 1~7).
3. Set the uSDHC block length register to be the same as the block length set for the card in step 2.
4. Set the uSDHC number block register (NOB), where nob is 5, for instance.
5. Disable the buffer read ready interrupt, configure the DMA settings, and enable the uSDHC DMA when sending the command with data transfer. The AC12EN bit should also be set.
6. Wait for the Transfer Complete interrupt.

7. Check the status bit to see if a read CRC error occurred, or another error occurred during the auto12 command sending and response receiving.

10.3.4.3.2.2 DDR read

The uSDHC module supports dual data rate mode.

The software flow to write to a card in the DDR mode is described below:

1. Check the card status and wait until the card is ready for data.
2. For eMMC4.4 card, block length can be set to only 512 bytes.
3. Set the uSDHC number block register (NOB) where nob is 5, for instance.
4. Set the eMMC4.4 card to high-speed mode and use SWITCH (CMD6).
5. Set the eMMC4.4 card bus with 4-bit /8-bit DDR mode and use SWITCH (CMD6).
6. Disable the buffer write ready interrupt, configure the DMA settings, and enable the uSDHC DMA when sending the command with data transfer. The DDR_EN and AC12EN bits should be set.
7. Wait for the Transfer Complete interrupt.
8. Check the status bit to see if a write CRC error occurred, or another error that occurred during the auto12 command sending and response receiving.

10.3.4.3.2.3 Read with Pause

The read operation is not generally able to pause. Only the SDIO card (and SDCombo card working under I/O mode) supporting the Read Wait feature can pause during the read operation. If the SDIO card supports Read Wait (SRW bit in CCCR register is 1), the host driver can set the SABGREQ bit in the Protocol Control register to pause the transfer between the data blocks.

Before setting the SABGREQ bit, ensure that the RWCTL bit in the Protocol Control register is set, otherwise uSDHC does not assert the Read Wait signal during the block gap and data corruption occurs. It is recommended to set the RWCTL bit after the Read Wait capability of the SDIO card is recognized.

Similar to the flow described in [Normal read](#), the read with pause is shown with the same kind of read operation:

1. Check the SRW bit in the CCR register on the SDIO card to confirm that the card supports the Read Wait mode.
2. Set the RWCTL bit.
3. Check the card status and wait until the card is ready for data.
4. Set the card block length/size:

- For SD/MMC, use SET_BLOCKLEN (CMD16).
 - For SDIO cards or the I/O portion of SDCombo cards, use IO_RW_DIRECT(CMD52) to set the I/O Block Size bit field in the CCCR register (for function 0) or FBR register (for functions 1~7).
5. Set the uSDHC block length register to be the same as the block length set for the card in step 2.
 6. Set the uSDHC number block register (NOB), where nob is 5, for instance.
 7. Disable the buffer read ready interrupt, configure the DMA setting, and enable the uSDHC DMA when sending the command with data transfer. The AC12EN bit should also be set.
 8. Set the SABGREQ bit.
 9. Wait for the Transfer Complete interrupt.
 10. Clear the SABGREQ bit.
 11. Check the status bit to see if read CRC error occurred.
 12. Set the CREQ bit to continue the read operation.
 13. Wait for the Transfer Complete interrupt.
 14. Check the status bit to see if a read CRC error occurred, or another error occurred during the auto12 command sending and response receiving.

Similar to the Write operation, it is possible to meet the ending block of the transfer when paused. In this case, uSDHC ignores the Stop At Block Gap Request and treats it as a command read operation.

Unlike the write operation, there is no remaining data inside the buffer when the transfer is paused. All data received before the pause is transferred to the host system. No matter if the Suspend command is sent or not, the internal data buffer is not flushed.

If the Suspend command is sent and the transfer is later resumed by means of a Resume command, uSDHC takes the command as a normal one accompanied with data transfer. It is left for the host driver to set all the relevant registers before the transfer is resumed. If there is only one block to send when the transfer is resumed, the MSBSEL and BCEN bits of the Transfer Type register are set, as well as the AC12EN bit. However, the uSDHC automatically sends CMD12 to mark the end of multi-block transfer.

10.3.4.3.2.4 Delay Line (DLL) in Read Path

The DLL is newly added to assist in sampling read data. The DLL provides the ability to programmatically select a quantized delay (in fractions of the clock period) regardless of on-chip variations such as process, voltage, and temperature (PVT).

The reasons why DLL is needed for uSDHC are these:

- The path of read data traveling from card to host varies.
- In the SD/MMC DDR mode, the minimum input setup and hold time are both at 2.5 ns.

The data sampling window is so small that the delay of loopback clock needs to be accurate and consistent regardless of PVT. The DLL takes the divided card_clk as the reference clock and loopback clock as the input clock. It then generates a delayed version of the input clock according to the programmed target delay.

The DLL can be disabled or bypassed, and it can also be manually set for a fixed delay in the override mode. The override value set is the number of delay cells. In the override mode, there is no need to set the DLL_enable. Another DLL mode is target value mode. In this mode, the DLL automatically adjusts the number of delay cells according to the target value set by the user and PVT changes. Be aware that the target value is in units of 1/32 of the clock reference period. If the card_clk is 100Mhz, then the reference clock period is 10ns; setting target value of 16 means $5\text{ns} = (16/32) * 10\text{ns}$. The software can disable automatic update by the setting dll_gate_update bit.

As the user may change the frequency of card_clk from time to time by changing SDCLKFS[7:0]/DVS[3:0], the software must adjust the delay value to ensure it works correctly when the reference clock (card_clk) is changed. The following is the correct flow, which should be ignored if DLL_CTRL_ENABLE is not set.

Step 1: Set the DLL_CTRL_RESET bit

Step 2: Configure the SDCLKFS[7:0] and DVS[3:0]

Step 3: Wait until SDSTB is asserted

Step 4: Clear the DLL_CTRL_RESET bit

Step 5: Wait until both the DLL_STS_SLV_LOCK and DLL_STS_REF_LOCK are asserted

Step 6: Set the DLL_CTRL_SLV_FORCE_UPD

Step 7: Clear the DLL_CTRL_SLV_FORCE_UPD

NOTE

The software should make sure that the DLL_CTRL_SLV_FORCE_UPD lasts for at least one card_clk. So, the software may need to add some delay between step 6 and step 7.

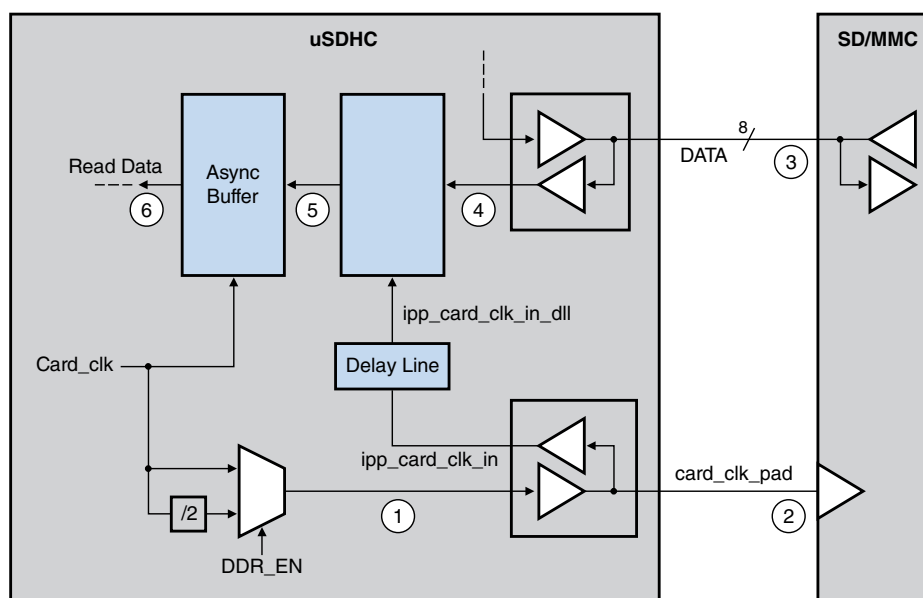


Figure 10-52. DLL in read path

10.3.4.3.3 Suspend Resume

The uSDHC module supports the Suspend Resume operations of SDIO cards, although slightly different than the suggested implementation of Suspend in the SDIO card specification.

10.3.4.3.3.1 Suspend

After setting the SABGREQ bit, the host driver may send a Suspend command to switch to another function of the SDIO card. The uSDHC module does not monitor the content of the response, so it does not know if the Suspend command succeeded or not. Accordingly, it does not de-assert Read Wait for read pause. To solve this problem, the host driver does not mark the Suspend command as "Suspend", (that is, setting the CMDTYP bits to 01). Instead, the driver sends this command as if it were a normal command, and only when the command succeeds, and the BS bit is set in the response, can the Driver send another command marked as "Suspend" to inform uSDHC that the current transfer is suspended. Here is the sequence for the Suspend operation:

1. Set the SABREQ bit to pause the current data transfer at block gap.
2. After the BGE bit is set, send the Suspend command to suspend the active function. The CMDTYP bit field must be 2'b00.
3. Check the BS bit of the CCCR in the response. If it is 1, repeat this step until the BS bit is cleared or abandon the suspend operation according to the Driver strategy.

4. Send another normal I/O command to the suspended function. The CMDTYP of this command must be 2'b01, so uSDHC can detect this special setting and be informed that the paused operation has successfully suspended. If the paused transfer is a read operation, uSDHC stops driving DATA2 and goes to the idle state.
5. Save the context registers in the system memory for later use, including the DMA System Address Register (for internal DMA operation), and the Block Attribute Register.
6. Begin operation for another function on the SDIO card.

10.3.4.3.2 Resume

To resume the data transfer, a Resume command is issued:

1. To resume the suspended function, restore the context register with the saved value in step #5 of the Suspend operation.
2. Send the Resume command: In the Transfer Type register, all the bit fields are set to the value as if this were another ordinary data transfer instead of a transfer resume (except the CMDTYP is set to 2'b10).
3. If the Resume command has responded, the data transfer is resumed.

10.3.4.3.4 ADMA usage

To use the ADMA in a data transfer, the host driver must prepare the correct descriptor chain prior to sending the read/write command.

The steps to prepare the correct descriptor chain are these:

1. Create a descriptor to set the data length that the current descriptor group is about to transfer. The data length should be even numbers of the block size.
2. Create another descriptor to transfer the data from the address setting in this descriptor. The data address must be at a page boundary (4KB address aligned).
3. If necessary, create a Link descriptor containing the address of the next descriptor. The descriptor group is created in steps 1 ~ 3.
4. Repeat steps 1 ~ 3 until all descriptors are created.
5. In the last descriptor, set the End flag to 1 and make sure the total length of all descriptors match the product of the block size and block number configured in the Block Attribute Register.
6. Set the ADMA System Address Register to the address of the first descriptor and set the DMAS field in the Protocol Control Register to 01 to select the ADMA.
7. Issue a write or read command with the DMAEN bit set to 1 in the Transfer Type Register.

Steps 1 ~ 5 are independent of step 6, so step 6 can finish before steps 1 ~ 5. Regarding the descriptor configuration, it is recommended not to use the Link descriptor as it requires extra system memory access.

10.3.4.3.5 Transfer error

Information about CRC, Internal DMA, Transfer ADMA, and Auto CMD12 errors are detailed in the sections below.

10.3.4.3.5.1 CRC error

It is possible at the end of a block transfer that a write CRC status error or read CRC error occurs. For this type of error, the latest block received is discarded. This is because the integrity of the data block is not guaranteed. It is recommended to discard the following data blocks and re-transfer the block from the corrupted one.

For a multi-block transfer, the host driver issues a CMD12 to abort the current process and start the transfer by a new data command. In this scenario, even when the AC12EN and BCEND bits are set, uSDHC does not automatically send a CMD12 because the last block is not transferred. On the other hand, if it is within the last block that the CRC error occurs, an Auto CMD12 is sent by uSDHC. In this case, the host driver re-sends or re-obtains the last block with a single block transfer.

10.3.4.3.5.2 Internal DMA error

During the data transfer with internal Simple DMA, if the DMA engine encounters an error on the AHB bus, the DMA operation is aborted, and the DMA error interrupt is sent to the host system. When acknowledged by such an interrupt, the host driver calculates the start address of data block in which the error occurs.

The start address can be calculated by either:

- Reading the DMA System Address register: The error occurs during the previous burst. Considering the block size, the previous burst length and the start address of the next burst transfer, it is straight forward to obtain the start address of the corrupted block.
- Reading the BLKCNT field of the Block Attribute register: Considering the number of blocks left, the total number to transfer, the start address of transfer, and the size of each block, the start address of corrupted block can be determined. When the BCEN bit is not set, the contents of the Block Attribute register do not change, so this method does not work.

When a DMA error occurs, it is recommended to abort the current transfer by means of a CMD12 (for multi block transfer), apply a reset for data, and restart the transfer from the corrupted block to recover from the error.

10.3.4.3.5.3 Transfer ADMA error

There are three kinds of possible ADMA errors: The AHB transfer, invalid descriptor, and data-length mismatch. Whenever these errors occur, the DMA transfer stops and the corresponding error status bit is set.

For acknowledging the status, the host driver should recover the error as shown below and re-transfer from the place of interruption.

- AHB transfer error: Such errors may occur during data transfer or descriptor fetch. For either scenario, it is recommended to retrieve the transfer context, reset for the data part and re-transfer the block that was corrupted, or to the next block if no block is corrupted.
- Invalid descriptor error: For such errors, it is recommended to retrieve the transfer context, reset for the data part and recreate the descriptor chain from the invalid descriptor and issue a new transfer. As the data to transfer now may be less than the previous setting, the data length configured in the new descriptor chain should match the new value.
- Data-length mismatch error: It is similar to recover from this error. The Host Driver polls relating registers to retrieve the transfer context, apply a reset for the data part, configure a new descriptor chain and make another transfer if there is data left. Like the previous scenario of the invalid descriptor error, the data length must match the new transfer.

10.3.4.3.5.4 Auto CMD12 error

After the last block of the multi-block transfer is sent or received, and the AC12EN bit is set when the data transfer is initiated by the data command, uSDHC automatically sends a CMD12 to the card to stop the transfer.

When errors with this command occur, it is recommended to the host driver to deal with the situations in the following manner:

- Auto CMD12 response time-out: It is not certain whether the command is accepted by the card or not. The host driver clears the auto CMD12 error status bits and re-send CMD12 until it is accepted by the card.

- Auto CMD12 response CRC error: As the card responds to CMD12, it aborts the transfer. The host driver may ignore the error and clear the error status bit.
- Auto CMD12 conflict error or not sent: The command is not sent; therefore, the host driver sends a CMD12 manually.

10.3.4.3.6 Card interrupt

The external cards can inform the host controller by means of some special signals. For the SDIO card, it can be the low-level on the DATA1 line during some special period. The uSDHC module only monitors the DATA1 line and supports the SDIO interrupt.

When the SDIO interrupt is captured by uSDHC, and the host system is informed by uSDHC asserting the uSDHC interrupt line, the interrupt service from the host driver is called.

As the interrupt source is controlled by the external card, the interrupt from the SDIO card must be serviced before the CINT bit is cleared by written. Refer to [Card interrupt handling](#) for the card interrupt handling flow.

10.3.4.4 Switch function

A switch command is issued by the host driver to enable new features added to the SD/MMC spec. The SD/MMC cards can transfer data at bus widths other than 1-bit. Different speed modes are also defined. To enable these features, a switch command is issued by the host driver.

For SDIO cards, the high-speed mode/DDR50/SDR50/SDR104 are enabled by writing the EHS bit in the CCCR register after the SHS bit is confirmed. For SD cards, the high-speed mode/DDR50/SDR50/SDR104 are queried and enabled by a CMD6 (with the mnemonic symbol as SWITCH_FUNC). For MMC cards, the high-speed mode/HS200/HS400 are queried by a CMD8 and enabled by a CMD6 (with the mnemonic symbol as SWITCH).

The SDR4-bit, SDR8-bit, DDR4-bit, and DDR8-bit width of MMC is also enabled by the SWITCH command, but with a different argument.

These new functions can also be disabled by a software reset. For SDIO cards, it can be done by setting the RES bit in the CCCR register. For other cards, it can be accomplished by issuing a CMD0. This method of restoring to the normal mode is not recommended because a complete identification process is needed before the card is ready for data transfer.

For the sake of simplicity, the following pseudocode examples do not show current capability check, which is recommended in the function switch process.

10.3.4.4.1 Query, enable, and disable SDIO high-speed mode

```
enable_sdio_high_speed_mode(void)
{
    send CMD52 to query bit SHS at address 0x13;
    if (SHS bit is '0') report the SDIO card does not support high speed mode and return;
    send CMD52 to set bit EHS at address 0x13 and read after write to confirm EHS bit is set;
    change clock divisor value or configure the system clock feeding into uSDHC to generate the
    card_clk of around 50MHz;
    (data transactions like normal peers)
}
disable_sdio_high_speed_mode(void)
{
    send CMD52 to clear bit EHS at address 0x13 and read after write to confirm EHS bit is
    cleared;
    change clock divisor value or configure the system clock feeding into uSDHC to generate the
    card_clk of the desired value below 25MHz;
    (data transactions like normal peers)
}
```

10.3.4.4.2 Query, enable, and disable SD high-speed mode/DDR50/SDR50/SDR104

```
enable_sd_speed_mode(void)
{
    set BLKCNT field to 1 (block), set BLKSIZE field to 64 (bytes);
    send CMD6, with argument 0xFFFFFx and read 64 bytes of data accompanying the R1 response;
    (high speed mode,x=1; SDR50,x=2; SDR104,x=3; DDR50,x=4;)
    wait data transfer done bit is set;
    check if the bit x of received 512 bits is set;
    if (bit 401 is '0') report the SD card does not support high speed mode and return;
    if (bit 402 is '0') report the SD card does not support SDR50 mode and return;
    if (bit 403 is '0') report the SD card does not support SDR104 mode and return;
    if (bit 404 is '0') report the SD card does not support DDR50 mode and return;
    send CMD6, with argument 0x80FFFFFx and read 64 bytes of data accompanying the R1 response;
    (high speed mode,x=1; SDR50,x=2; SDR104 x=3; DDR50 x=4;)
    check if the bit field 379~376 is 0xF;
    if (the bit field is 0xF) report the function switch failed and return;
    change clock divisor value or configure the system clock feeding into uSDHC to generate the
    card_clk of around 50MHz for high speed mode, 100MHz for SDR50, 200MHz for SDR104, 50MHz for
    DDR50;
    (data transactions like normal peers)
}
disable_sd_speed_mode(void)
{
    set BLKCNT field to 1 (block), set BLKSIZE field to 64 (bytes);
    send CMD6, with argument 0x80FFFFF0 and read 64 bytes of data accompanying the R1 response;
    check if the bit field 379~376 is 0xF;
    if (the bit field is 0xF) report the function switch failed and return;
    change clock divisor value or configure the system clock feeding into uSDHC to generate the
    card_clk of the desired value below 25MHz;
    (data transactions like normal peers)
}
```

10.3.4.4.3 Query, enable, and disable MMC high-speed mode

```
enable_mmc_high_speed_mode(void)
{
    send CMD9 to get CSD value of MMC;
```



```

check if the value of SPEC_VER field is 4 or above;
if (SPEC_VER value is less than 4) report the MMC does not support high speed mode and
return;
set BLKCNT field to 1 (block), set BLKSIZE field to 512 (bytes);
send CMD8 to get EXT_CSD value of MMC;
extract the value of CARD_TYPE field to check the 'high speed mode' in this MMC is 26MHz or
52MHz;
send CMD6 with argument 0x1B90100;
send CMD13 to wait card ready (busy line released);
send CMD8 to get EXT_CSD value of MMC;
check if HS_TIMING byte (byte number 185) is 1;
if (HS_TIMING is not 1) report MMC switching to high speed mode failed and return;
change clock divisor value or configure the system clock feeding into uSDHC to generate the
card_clk of around 26MHz or 52MHz according to the CARD_TYPE;
(data transactions like normal peers)
}
disable_mmc_high_speed_mode(void)
{
send CMD6 with argument 0x2B90100;
set BLKCNT field to 1 (block), set BLKSIZE field to 512 (bytes);
send CMD8 to get EXT_CSD value of MMC;
check if HS_TIMING byte (byte number 185) is 0;
if (HS_TIMING is not 0) report the function switch failed and return;
change clock divisor value or configure the system clock feeding into uSDHC to generate the
card_clk of the desired value below 20MHz;
(data transactions like normal peers)
}

```

10.3.4.4.4 Set MMC bus width

```

change_mmc_bus_width(void)
{
send CMD9 to get CSD value of MMC;
check if the value of SPEC_VER field is 4 or above;
if (SPEC_VER value is less than 4) report the MMC does not support multiple bit width and
return;
send CMD6 with argument 0x3B70x00; (8-bit(dual data rate), x=6; 4-bit(dual data rate), x=5; 8-
bit, x=2; 4-bit, x=1; 1-bit, x=0)
send CMD13 to wait card ready (busy line released);
(data transactions like normal peers)
}

```

10.3.4.5 ADMA operation

Here are the codes for the ADMA1 and ADMA2 operations.

10.3.4.5.1 ADMA1 operation

```

Set_adma1_descriptor
{
if (to start data transfer) {
// Make sure the address is 4KB align.
Set 'Set' type descriptor;
{
Set Act bits to 01;
Set [31:12] bits data length (byte unit);
}
Set 'Tran' type descriptor;
{
Set Act bits to 10;
Set [31:12] bits address (4KB align);
}
}

```

```

}
else if (to fetch descriptor at non-continuous address) {
Set Act bits to 11;
Set [31:12] bits the next descriptor address (4KB aligned);
}
else { // other types of descriptor
Set Act bits accordingly
}
if (this descriptor is the last one) {
Set End bit to 1;
}
if (to generate interrupt for this descriptor) {
Set Int bit to 1;
}
Set Valid bit to 1;
}

```

10.3.4.5.2 ADMA2 operation

```

Set_adma2_descriptor
{
if (to start data transfer) {
// Make sure the address is a 32-bit boundary (lower 2-bit are always '00').
Set higher 32-bit of descriptor for this data transfer initial address;
Set [31:16] bits data length (byte unit);
Set Act bits to '10';
}
else if (to fetch descriptor at non-continuous address) {
Set Act bits to '11';
// Make sure the address is 32-bit boundary (lower 2-bit are always set to '00').
Set higher 32-bit of descriptor for the next descriptor address;
}
else { // other types of descriptor
Set Act bits accordingly
}
if (this descriptor is the last one) {
Set 'End' bit '1';
}
if (to generate interrupt for this descriptor) {
Set 'Int' bit '1';
}
Set the 'Valid' bit to '1';
}

```

10.3.4.6 Fast boot operation

10.3.4.6.1 Normal fast boot flow

Here are the steps of normal fast boot flow:

1. Software must configure the init_active bit (system control register bit 27) to make sure that 74 card clocks are finished.
2. Software must configure the MMC Boot Register (offset 0xc4) bit 6 to 1 (enable boot), and bit 5 to 0 (normal fast boot), and bit 4 to select the ack mode. If the data is sent through the DMA mode, the software should configure bit 7 to enable the automatic stop at block gap feature, and configure bit 3-bit 0 to select the ack timeout value according to the SD CLK frequency.

3. Software then needs to configure the Block Attributes Register to set the block size and count. If in DDR fast boot mode, the block size only can be configured to 512 bytes.
4. Software must configure the Protocol control register to set Data Transfer Width (DTW). If in the DDR fast boot mode, DTW only can be configured to 4-bit/8-bit dataline mode.
5. Software needs to configure the Command Argument Register to set argument if needed (no need in normal fast boot).
6. Software must configure the Transfer Type Register to start the boot process. In normal boot mode, CMDINX, CMDTYP, RSPTYP, CICEN, CCCEN, AC12EN, BCEN and DMAEN retain the default value, where DPSEL bit is set to 1, DTDSEL is set to 1 and MSBSEL is set to 1.
7. DMAEN should be configured as 0 in the polling mode and if BCEN is configured as 1, it is recommended to configure the number of blocks in the Block Attributes Register to the maximum value. If in DDR fast boot mode, DDR_EN needs to be set to 1.
8. When step 6 is configured, the boot process begins. The software needs to poll the data buffer ready status to read the data from the buffer in time. If a boot timeout happens (ack times out or the first data read times out), an interrupt is triggered, and the software must configure MMC Boot Register to bit 6 to 0 to disable boot. This makes CMD high, then after at least 56 clocks, it is ready to begin a normal initialization process.
9. If there is no timeout, software needs to determine when the data read is finished and then configure MMC Boot Register bit 6 to 0 to disable boot. This render CMD line high and command completed asserted. After at least 56 clocks, it is ready to begin the normal initialization process.
10. Reset the host and then can begin the normal process.

10.3.4.6.2 Alternative fast boot flow

Here are the steps of alternative fast boot flow:

1. Software needs to configure init_active bit (system control register bit 27) to make sure 74 card clocks are finished.
2. Software needs to configure MMC Boot Register (offset 0xc4) bit 6 to 1 (enable boot), and bit 5 to 1 (alternative boot), and bit 4 to select the ack mode or not. If data needs to be sent through the DMA mode, then configure bit 7 to enable the automatic stop at block gap feature. Software should also configure bit 3-bit 0 to select the ack timeout value according to the SD clock frequency.
3. Software then needs to configure the Block Attributes Register to set the block size and count. If in the DDR fast boot mode, the block size only can be configured to 512 bytes.

4. Software needs to configure the Protocol control register to set the data transfer width (DTW). If in the DDR fast boot mode, DTW only can be configured to 4-bit/8-bit dataline mode.
5. Software needs to configure Command Argument Register to set argument to 0xFFFFFFFFFA.
6. Software needs to configure the Transfer Type Register to start the boot process by CMD0 with the 0xFFFFFFFFFA argument. In alternative boot, CMDINX, CMDTYP, RSPTYP, CICEN, CCCEN, AC12EN, BCEN, and DMAEN retain the default value. DPSEL bit is set to 1, DTDSEL is set to 1, and MSBSEL is set to 1. Note DMAEN should be configured as 0 in the polling mode, and if BCEN is configured as 1 in polling mode, it is recommended to configure the block count in the Block Attributes Register to the maximum value. If in the DDR fast boot mode, DDR_EN needs to be set to 1.
7. When step 6 is configured, the boot process begins. Software needs to poll the data buffer ready status to read the data from the buffer in time. If there is a boot timeout (ack data timeout in 50ms or data timeout in 1s), the host sends out the interrupt and the software needs to send CMD0 with reset and then configure the boot enable bit to 0 to stop this process.
8. If there is no time out, the software needs to decide when to stop the boot process, and send out the CMD0 with reset and then after the command is completed, configure the MMC Boot Register bit 6 to stop the process. After 8 clocks from the command completion, the slave (card) is ready for the identification step.
9. Reset the host and then begin the normal process.

10.3.4.6.3 Fast boot application case (in DMA mode)

In the boot application case, because the image destination and the image size are contained in the beginning of the image, it is necessary to switch DMA parameters on the fly during MMC fast boot.

In fast boot, the host can use Advanced DMA2 (ADMA2) with two destinations.

The detailed flow is described below:

1. The software needs to configure INIT_ACTIVE bit (system control register bit 27) to make sure that 74 card clocks are finished.
2. The software needs to configure the MMC Boot Register (offset 0xc4) bit 6 to 1 (enable boot); and bit 5 to 0 (normal fast boot) or 1 (alternative boot); and bit 4 to select the ack mode. In DMA mode, configure bit 7 to 1 to enable the automatic stop at block gap feature. Also configure bits[31-16] to set the (BLK_CNT - VALUE1). Here VALUE1 is the value of the block count that needs to transfer the first time, so that the host stops at the block gap when the uSDHC controller gets VALUE1 blocks

from the device. Also, configure bits[3-0] to select the ack timeout value according to the SD clock frequency.

3. The software then needs to configure the Block Attributes Register to set block size and count. If in DDR fast boot mode, the block size only can be configured to 512 bytes. In DMA mode, it is recommended to set the block count (BLK_CNT) to the max value (16'hffff).
4. The software needs to configure Protocol Control Register to set DTW (data transfer width). If in DDR fast boot mode, the DTW only can be configured to 4-bit/8-bit dataline mode.
5. Software enable ADMA2 by configuring Protocol Control Register bits [9-8].
6. The software needs to set at least three pairs of ADMA2 descriptor in boot memory (that is, in IRAM, at least six words). The first pair descriptor defines the start address (that is, IRAM) and data length (that is, 512byte*VALUE1) of the first part boot code. The software also needs to set the second pair descriptor, the second start address (any value that is writable), and data length is suggested to set 1~2word (record as VALUE2). Note that the second couple desc also transfers useful data even at lease 1 word, because our ADMA2 cannot support 0 data_length data transfer descriptor.
7. The software needs to configure Command Argument Register to set argument to 0xFFFFFFFF in alternative fast boot and do not need to be set in normal fast boot.
8. The software needs to configure Transfer Type Register to start the boot process. CMDINX, CMDTYP, RSPTYP, CICEN, CCCEN, AC12EN, BCEN, and DMAEN retain the default value. DPSEL bit is set to 1, DTDSEL is set to 1, and MSBSEL is set to 1. DMAEN is configured as 1 in the DMA mode. And, if BCEN is configured as 1, then configure blk no in Block Attributes Register to the max value. And, if in the DDR fast boot mode, DDR_EN needs to be set to 1.
9. When step 8 is configured, boot process begins, the first VALUE1 block number data gets transferred. The software needs to poll the TC bit (bit1 in Interrupt Status Register) to determine first transfer is ended. Also, the software needs to polling the BGE bit (bit2 in Interrupt Status Register) to determine if the first transfer stops at the block gap.
10. When TC and BGE bits are set to 1, the software can analyze the first code of VALUE1 block, initializes the new memory device, if required, and sets the third pair of descriptors to define the start address and length of the remaining part of the boot code (VALUE3, the remain boot code block). Remember to set the last descriptor with END.
11. The software needs to configure the MMC Boot Register (offset 0xc4) again. Set bit 6 to 1 (enable boot); and bit 5 to 0 (normal fast boot), to 1 (alternative boot); and bit 4 to select the ack mode or not. In the DMA mode, configure bit 7 to 1 for enabling the automatically stop at block gap feature. Also, configure bit31-bit16 to set the (BLK_CNT - (VALUE1+1+VALUE3)), that host stops at block gap when the

uSDHC controller gets (VALUE1+1+VALUE3)) blocks from device totally include the blocks received in step 9. And need to configure bit 3-bit0 to select the ack timeout value according to the sd clk frequency. Note that the software does not need to configure the BLK_CNT again, because it is counted down automatically by the uSDHC controller.

12. The software needs to clear the TC and BGE bits and the software needs to clear SABGREQ (bit 16 in the Protocol control register) and set CREQ (bit17 in the Protocol control register) to 1 to resume the data transfer. Host transfers the VALUE2 and VALUE3 data to the destination that is set by descriptor.
13. The software needs to do poll BGE bit to determine if the fast boot is over.

Note:

1. When ADMA boot flow starts, for uSDHC, it is like a normal ADMA read operation. So, set ADMA2 descriptor as the normal ADMA2 transfer.
2. Need a few words length memory to keep descriptor.
3. For the 1~2-word data in second descriptor setting, it is a useful data, so the software needs to deal the data because of the application case.

10.3.5 Commands for MMC/SD/SDIO

A table containing the list of commands for the MMC/SD/SDIO cards is provided here.

Refer to the corresponding specifications for more details about the command information.

There are four kinds of commands defined to control the MultiMediaCard:

1. Broadcast commands (bc), no response
2. Broadcast commands with response (bcr), response from all cards simultaneously
3. Addressed (point-to-point) commands (ac), no data transfer on the DATA
4. Addressed (point-to-point) data transfer commands (adtc)

Response: A response is a token that is sent from the card to the host as an answer to a previously received command. A response is transferred serially on the CMD line.

Table 10-39. Commands for MMC/SD/SDIO cards

CMD INDEX	Type	Argument	Response type	Abbreviation	Description
CMD0	bc	[31:0] stuff bits	-	GO_IDLE_STATE	Resets all MMC and SD memory cards to idle state.

Table continues on the next page...

Table 10-39. Commands for MMC/SD/SDIO cards (continued)

CMD INDEX	Type	Argument	Response type	Abbreviation	Description
CMD1	bcr	[31:0] OCR without busy	R3	SEND_OP_COND	Asks all MMC and SD Memory cards in idle state to send their operation conditions register contents in the response on the CMD line.
CMD2	bcr	[31:0] stuff bits	R2	ALL_SEND_CID	Asks all cards to send their CID numbers on the CMD line.
CMD3 ¹	ac	[31:6] RCA [15:0] stuff bits	R1 R6 (SDIO)	SET/ SEND_RELATIVE_ADDR	Assigns relative address to the card.
CMD4	bc	[31:0] DSR [15:0] stuff bits	-	SET_DSR	Programs the DSR of all cards.
CMD5	bc	[31:0] OCR without busy	R4	IO_SEND_OP_COND	Asks all SDIO cards in idle state to send them operation conditions register contents in the response on the CMD line.
CMD6 ²	adtc	[31] Mode 0: Check function 1: Switch function [30:8] Reserved for function groups 6 ~ 3 (All 0 or 0xFFFF) [7:4] Function group1 for command system [3:0] Function group2 for access mode	R1	SWITCH_FUNC	Checks switch ability (mode 0) and switch card function (mode 1). Refer to "SD Physical Specification V1.1" for more details.
CMD6 ³	ac	[31:26] Set to 0 [25:24] Access [23:16] Index [15:8] Value [7:3] Set to 0 [2:0] Cmd Set	R1b	SWITCH	Switches the mode of operation of the selected card or modifies the EXT_CSD registers. Refer to "The MultiMediaCard System Specification Version 4.0 Final draft 2" for more details.
CMD7	ac	[31:6] RCA [15:0] stuff bits	R1b	SELECT/ DESELECT_CARD	Toggles a card between the stand-by and transfer states or between the programming and disconnect states. In both cases, the card is selected by its own relative address and gets deselected by any other address. Address 0 deselects all.
CMD8	adtc	[31:0] stuff bits	R1	SEND_EXT_CSD	The card sends its EXT_CSD register as a block of data, with a block size of 512 bytes.
CMD9	ac	[31:6] RCA [15:0] stuff bits	R2	SEND_CSD	Addressed card sends its card-specific data (CSD) on the CMD line.

Table continues on the next page...

Table 10-39. Commands for MMC/SD/SDIO cards (continued)

CMD INDEX	Type	Argument	Response type	Abbreviation	Description
CMD10	ac	[31:6] RCA [15:0] stuff bits	R2	SEND_CID	Addressed card sends its card-identification (CID) on the CMD line.
CMD11	adtc	[31:0] data address	R1	READ_DAT_UNTIL_STOP	Reads data stream from the card, starting at the given address, until a STOP_TRANSMISSION follows.
CMD12	ac	[31:0] stuff bits	R1b	STOP_TRANSMISSION	Forces the card to stop transmission.
CMD13	ac	[31:6] RCA [15:0] stuff bits	R1	SEND_STATUS	Addressed card sends its status register.
CMD14	Reserved				
CMD15	ac	[31:6] RCA [15:0] stuff bits	-	GO_INACTIVE_STATE	Sets the card to inactive state in order to protect the card stack against communication breakdowns.
CMD16	ac	[31:0] block length	R1	SET_BLOCKLEN	Sets the block length (in bytes) for all following block commands (read and write). Default block length is specified in the CSD.
CMD17	adtc	[31:0] data address	R1	READ_SINGLE_BLOCK	Reads a block of the size selected by the SET_BLOCKLEN command.
CMD18	adtc	[31:0] data address	R1	READ_MULTIPLE_BLOCK	Continuously transfers data blocks from card to host until interrupted by a stop command.
CMD19	adtc	[31:0] reserved bits(all 0)	R1	SEND_TUNING_BLOCK	64 bytes tuning pattern is sent for SDR50 and SDR104.
CMD20	adtc	[31:0] data address	R1	WRITE_DAT_UNTIL_STOP	Writes data stream from the host, starting at the given address, until a STOP_TRANSMISSION follows.
CMD21	Reserved				
CMD22-23	Reserved				
CMD24	adtc	[31:0] data address	R1	WRITE_BLOCK	Writes a block of the size selected by the SET_BLOCKLEN command.
CMD25	adtc	[31:0] data address	R1	WRITE_MULTIPLE_BLOCK	Continuously writes blocks of data until a STOP_TRANSMISSION follows.
CMD26	adtc	[31:0] stuff bits	R1	PROGRAM_CID	Programming of the card identification register. This command is issued only once per card. The card contains hardware to prevent this operation after the first programming. Normally this command is reserved for the manufacturer.
CMD27	adtc	[31:0] stuff bits	R1	PROGRAM_CSD	Programming of the programmable bits of the CSD.

Table continues on the next page...

Table 10-39. Commands for MMC/SD/SDIO cards (continued)

CMD INDEX	Type	Argument	Response type	Abbreviation	Description
CMD28	ac	[31:0] data address	R1b	SET_WRITE_PROT	If the card has write protection features, this command sets the write protection bit of the addressed group. The properties of write protection are coded in the card specific data (WP_GRP_SIZE).
CMD29	ac	[31:0] data address	R1b	CLR_WRITE_PROT	If the card provides write protection features, this command clears the write protection bit of the addressed group.
CMD30	adtc	[31:0] write protect data address	R1	SEND_WRITE_PROT	If the card provides write protection features, this command asks the card to send the status of the write protection bits.
CMD31	Reserved				
CMD32	ac	[31:0] data address	R1	TAG_SECTOR_START	Sets the address of the first sector of the erase group.
CMD33	ac	[31:0] data address	R1	TAG_SECTOR_END	Sets the address of the last sector in a continuous range within the selection of a single sector to be selected for erase.
CMD34	ac	[31:0] data address	R1	UNTAG_SECTOR	Removes one previously selected sector from the erase selection.
CMD35	ac	[31:0] data address	R1	TAG_ERASE_GROUP_START	Sets the address of the first erase group within a range to be selected for erase.
CMD36	ac	[31:0] data address	R1	TAG_ERASE_GROUP_END	Sets the address of the last erase group within a continuous range to be selected for erase.
CMD37	ac	[31:0] data address	R1	UNTAG_ERASE_GROUP	Removes one previously selected erase group from the erase selection.
CMD38	ac	[31:0] stuff bits	R1b	ERASE	Erase all previously selected sectors.
CMD39	ac	[31:0] RCA [15] register write flag [14:8] register address [7:0] register data	R4	FAST_IO	Used to write and read 8-bit (register) data fields. The command addresses a card, and a register, and provides the data for writing if the write flag is set. The R4 response contains data read from the address register. This command accesses application dependent registers which are not defined in the MMC standard.
CMD40	bcr	[31:0] stuff bits	R5	GO_IRQ_STATE	Sets the system into interrupt mode.
CMD41	Reserved				

Table continues on the next page...

Table 10-39. Commands for MMC/SD/SDIO cards (continued)

CMD INDEX	Type	Argument	Response type	Abbreviation	Description
CDM42	adtc	[31:0] stuff bits	R1b	LOCK_UNLOCK	Used to set/reset the password or lock/unlock the card. The size of the data block is set by the SET_BLOCK_LEN command.
CMD43~51	Reserved				
CMD52	ac	[31:0] stuff bits	R5	IO_RW_DIRECT	Access a single register within the total 128k of register space in any I/O function.
CMD53	ac	[31:0] stuff bits	R5	IO_RW_EXTENDED	Accesses a multiple I/O register with a single command. Allows the reading or writing of a large number of I/O registers.
CMD54	Reserved				
CMD55	ac	[31:16] RCA [15:0] stuff bits	R1	APP_CMD	Indicates to the card that the next command is an application specific command rather than a standard command.
CMD56	adtc	[31:1] stuff bits [0]: RD/WR	R1b	GEN_CMD	Used either to transfer a data block to the card or to get a data block from the card for general purpose / application specific commands. The size of the data block is set by the SET_BLOCK_LEN command.
CMD57-59	Reserved				
CMD60	adtc	[31] WR [30:24] stuff bits [23:16] address [15:8] stuff bits [7:0] byte count	R1b	RW_MULTIPLE_REGISTER	These registers are used to control the behavior of the device and to retrieve status information regarding the operation of the device. All Status and Control registers are WORD (32-bit) in size and are WORD aligned. CMD60 is used to read and write these registers.
CMD61	adtc	[31] WR [30:16] stuff bits [15:0] data unit count	R1b	RW_MULTIPLE_BLOCK	The host issues a RW_MULTIPLE_BLOCK (CMD61) to begin the data transfer.
CMD62-63	Reserved				
ACMD6 ⁴	ac	[31:2] stuff bits [1:0] bus width	R1	SET_BUS_WIDTH	Defines the data bus width ('00'=1bit or '10'=4bit bus) to be used for data transfer. The allowed data bus widths are given in SCR register.
ACMD13 ⁴	adtc	[31:0] stuff bits	R1	SD_STATUS	Send the SD Memory Card status.
ACMD22 ⁴	adtc	[31:0] stuff bits	R1	SEND_NUM_WR_SECTORS	Send the number of the written sectors (without errors). Responds with 32-bit plus the CRC data block.

Table continues on the next page...

Table 10-39. Commands for MMC/SD/SDIO cards (continued)

CMD INDEX	Type	Argument	Response type	Abbreviation	Description
ACMD23 ⁴	ac	[31:23] stuff bits [22:0] Number of blocks	R1	SET_WR_BLK_ERASE_COUNT	Set the number of write blocks to be pre-erased before writing (to be used for fast Multiple Block WR command). "1"=default(one write block).
ACMD41 ⁴	bcr	[31:0] OCR	R3	SD_APP_OP_COND	Asks the accessed card to send its operating condition register (OCR) contents in the response on the CMD line.
ACMD42 ⁴	ac	[31:1] stuff bits [0] set_cd	R1	SET_CLR_CARD_DETECT	Connect(1)/Disconnect(0) the 50KOhm pull-up resistor on CD_B/ DATA3 of the card.
ACMD51 ⁴	adtc	[31:0] stuff bits	R1	SEND_SCR	Reads the SD Configuration Register (SCR).

1. CMD3 differs for MMC and SD cards. For MMC cards, it is referred to as SET_RELATIVE_ADDR, with a response type of R1. For SD cards, it is referred to as SEND_RELATIVE_ADDR, with a response type of R6 (with RCA inside).
2. CMD6 differs completely between high speed MMC cards and high-speed SD cards. Command SWITCH_FUNC is for high-speed SD cards.
3. Command SWITCH is for high speed MMC cards. The Index field can contain any value from 0-255, but only values 0-191 are valid. If the Index value is in the 192-255 range the card does not perform any modification and the SWITCH_ERROR status bit in the EXT_CSD register is set. The Access Bits are shown in [Table 2](#).
4. ACMDs is preceded with the APP_CMD command. Commands listed are used for SD only, other SD commands not listed are not supported on this module.

The access bits for the EXT_CSD access modes are shown below.

Table 10-40. EXT_CSD access modes

Bits	Access name	Operation
00	Command set	The command set is changed according to the Cmd Set field of the argument.
01	Set bits	The bits in the pointed byte are set, according to the bits set to 1 in the Value field.
10	Clear bits	The bits in the pointed byte are cleared, according to the bits set to 1 in the Value field.
11	Write byte	The Value field is written into the pointed byte.

10.3.6 Software restrictions

10.3.6.1 Initialization active

The driver cannot set INITA bit in System Control register when any of the command line or data lines are active, so the driver must ensure both CDIHB and CIHB bits are cleared.

10.3.6.2 Software polling procedure

For polling read or write, after the software begins a buffer read or write, it must access exactly the number of times as the values set in the Watermark Level Register; moreover, if the block size is not a multiple of the value in the Watermark Level Register (read and write respectively), the software must access exactly the remaining number of words at the end of each block.

For example, for a read operation, if the RD_WML is 4, indicating the watermark level is 16 bytes, block size is 40 bytes, and the block number is 2, then the access times for the burst sequence in the whole transfer process must be 4, 4, 2, 4, 4, 2.

10.3.6.3 Suspend operation

To suspend the data transfer, the software must inform uSDHC that the suspend command is successfully accepted. To achieve this, after the Suspend command is accepted by the SDIO card, software must send another normal command marked as suspend command (CMDTYP bits set as '01') to inform uSDHC that the transfer is suspended.

If software needs to resume the suspended transfer, it should read the value in BLKCNT register to save the remaining number of blocks before sending the normal command marked as suspend, otherwise on sending such a 'suspend' command, uSDHC treats the current transfer is aborted and change the BLKCNT register to its original value, instead of retaining the remaining number of blocks.

10.3.6.4 Data length setting

For either ADMA (ADMA1 or ADMA2) transfer, the data in the data buffer must be word aligned, so the data length set in the descriptor must be a multiple of 4.

10.3.6.5 (A)DMA address setting

To configure the ADMA1/ADMA2/DMA address register, when TC bit is set, the register always updates itself with the internal address value to support dynamic address synchronization, so the software must ensure that the TC bit is cleared prior to configuring the ADMA1/ADMA2/DMA address register.

10.3.6.6 Data port access

Data port does not support parallel access. For example, during an internal DMA access, it is not allowed to write any data to the data port by CPU; or during a CPU read operation, it is also prohibited to write any data to the data port, by either CPU or internal DMA. Otherwise the data is corrupted inside the uSDHC buffer.

10.3.6.7 Change clock frequency

The uSDHC module does not automatically gate off the card clock when the host driver changes the clock frequency. To prevent possible glitch on the card clock, clear the `FRC_SDCLK_ON` bit when changing the clock divisor value (`SDCLKFS` or `DVS` in System Control Register) or setting the `RSTA` bit.

Also, before changing the clock divisor value, the host driver should make sure that the `SDSTB` bit is high.

10.3.6.8 Multi-block read

For pre-defined multi-block read operation, that is, the number of blocks to read has been defined by previous `CMD23` for MMC, or pre-defined number of blocks in `CMD53` for SDIO/SDCombo, or whatever multi-block read without abort command at card side, an abort command, either automatic or manual `CMD12/CMD52`, is still required by uSDHC after the pre-defined number of blocks are done, to drive the internal state machine to idle mode.

In this case, the card may not respond to this extra abort command and uSDHC gets response timeout. It is recommended to manually send an abort command with `RSPTYP[1:0]` both bits cleared.

10.3.7 uSDHC memory map/register definition

10.3.7.1 uSDHC register descriptions

This section includes the module memory map and detailed descriptions of all registers.

See the table below for the register memory map of uSDHC. All these registers only support 32-bit accesses.

NOTE

The uSDHC registers are 32-bit wide and only support 32-bit access.

10.3.7.1.1 uSDHC memory map

uSDHC1 base address: 30B4_0000h

uSDHC2 base address: 30B5_0000h

uSDHC3 base address: 30B6_0000h

Offset	Register	Width (In bits)	Access	Reset value
0h	DMA System Address (DS_ADDR)	32	RW	0000_0000h
4h	Block Attributes (BLK_ATT)	32	RW	0001_0000h
8h	Command Argument (CMD_ARG)	32	RW	0000_0000h
Ch	Command Transfer Type (CMD_XFR_TYP)	32	RW	0000_0000h
10h	Command Response0 (CMD_RSP0)	32	RO	0000_0000h
14h	Command Response1 (CMD_RSP1)	32	RO	0000_0000h
18h	Command Response2 (CMD_RSP2)	32	RO	0000_0000h
1Ch	Command Response3 (CMD_RSP3)	32	RO	0000_0000h
20h	Data Buffer Access Port (DATA_BUFF_ACC_PORT)	32	RW	0000_0000h
24h	Present State (PRES_STATE)	32	RO	Table 10-248
28h	Protocol Control (PROT_CTRL)	32	RW	0880_0020h
2Ch	System Control (SYS_CTRL)	32	RW	0080_800Fh
30h	Interrupt Status (INT_STATUS)	32	W1C	0000_0000h
34h	Interrupt Status Enable (INT_STATUS_EN)	32	RW	0000_0000h
38h	Interrupt Signal Enable (INT_SIGNAL_EN)	32	RW	0000_0000h
3Ch	Auto CMD12 Error Status (AUTOCMD12_ERR_STATUS)	32	RW	0000_0000h
40h	Host Controller Capabilities (HOST_CTRL_CAP)	32	RW	07F3_B407h
44h	Watermark Level (WTMK_LVL)	32	RW	0810_0810h
48h	Mixer Control (MIX_CTRL)	32	RW	8000_0000h
50h	Force Event (FORCE_EVENT)	32	WORZ	0000_0000h
54h	ADMA Error Status (ADMA_ERR_STATUS)	32	RO	0000_0000h
58h	ADMA System Address (ADMA_SYS_ADDR)	32	RW	0000_0000h
60h	DLL (Delay Line) Control (DLL_CTRL)	32	RW	0000_0000h
64h	DLL Status (DLL_STATUS)	32	RO	0000_0200h
68h	CLK Tuning Control and Status (CLK_TUNE_CTRL_STATUS)	32	RW	0000_0000h
70h	Strobe DLL control (STROBE_DLL_CTRL)	32	RW	0000_0000h

Table continues on the next page...

Offset	Register	Width (In bits)	Access	Reset value
74h	Strobe DLL status (STROBE_DLL_STATUS)	32	RO	0000_0200h
C0h	Vendor Specific Register (VEND_SPEC)	32	RW	3000_7809h
C4h	MMC Boot (MMC_BOOT)	32	RW	0000_0000h
C8h	Vendor Specific 2 Register (VEND_SPEC2)	32	RW	0001_9006h
CCh	Tuning Control (TUNING_CTRL)	32	RW	0021_2800h
100h	Command Queue (CQE)	32	ROZ	0000_0510h

10.3.7.1.2 DMA System Address (DS_ADDR)

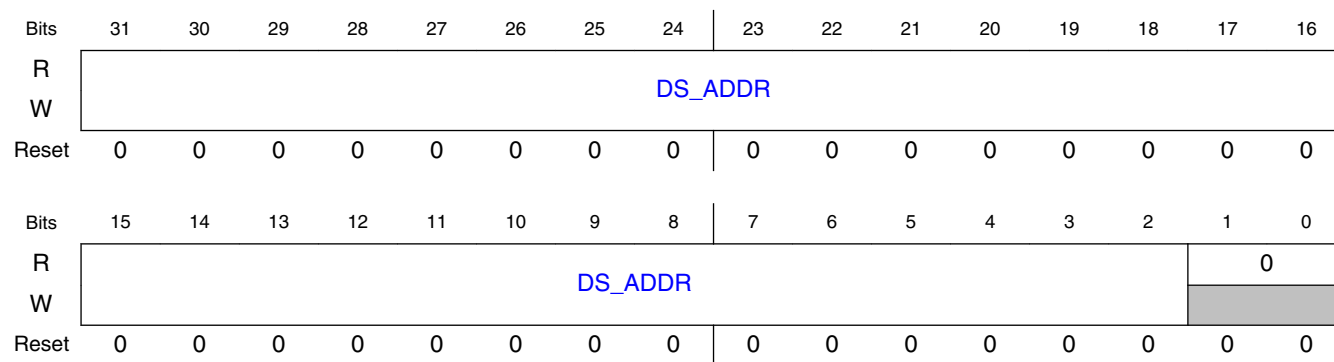
10.3.7.1.2.1 Offset

Register	Offset
DS_ADDR	0h

10.3.7.1.2.2 Function

This register contains the physical system memory address used for DMA transfers.

10.3.7.1.2.3 Diagram



10.3.7.1.2.4 Fields

Field	Function
31-2	System address
DS_ADDR	DMA system address / argument 2

Table continues on the next page...

Field	Function
	When ACMD23_ARGU2_EN is set to 0, SDMA uses this register as system address and supports only 32-bit addressing mode. Auto CMD23 cannot be used with SDMA. When ACMD23_ARGU2_EN is set to 1, SDMA uses ADMA System Address register (05Fh – 058h) instead of this register to support both 32-bit and 64-bit addressing. This register is used only for Argument2 and SDMA may use Auto CMD23. 1. SDMA system address Because the address must be word (4 bytes) aligned, the least 2 bits are reserved, always 0. When uSDHC stops a DMA transfer, this register points out the system address of the next contiguous data position. It can be accessed only when no transaction is executing (that is, after a transaction has stopped). Read operation during transfers may return an invalid value. The host driver initializes this register before starting a DMA transaction. After DMA has stopped, the system address of the next contiguous data position can be read from this register. This register is protected during a data transfer. When data lines are active, write to this register is ignored. The host driver waits until the DLA bit in the Present State register is cleared, before writing to this register. The uSDHC internal DMA does not support a virtual memory system. It only supports continuous physical memory access. And due to AHB burst limitations, if the burst must cross the 1 KB boundary, uSDHC automatically changes SEQ burst type to NSEQ. Because this register supports dynamic address reflecting, when TC bit is set, it automatically alters the value of internal address counter, so the software cannot change this register when TC bit is set. Such restriction is also listed in Software restrictions. 2. Argument 2 This register is used with the Auto CMD23 to set a 32-bit block count value to the argument of the CMD23 while executing Auto CMD23. If Auto CMD23 is used with ADMA, the full 32-bit block count value can be used. If Auto CMD23 is used without ADMA, the available block count value is limited by the Block Count register. 65535 blocks is the maximum value in this case.
1-0 —	Reserved

10.3.7.1.3 Block Attributes (BLK_ATT)

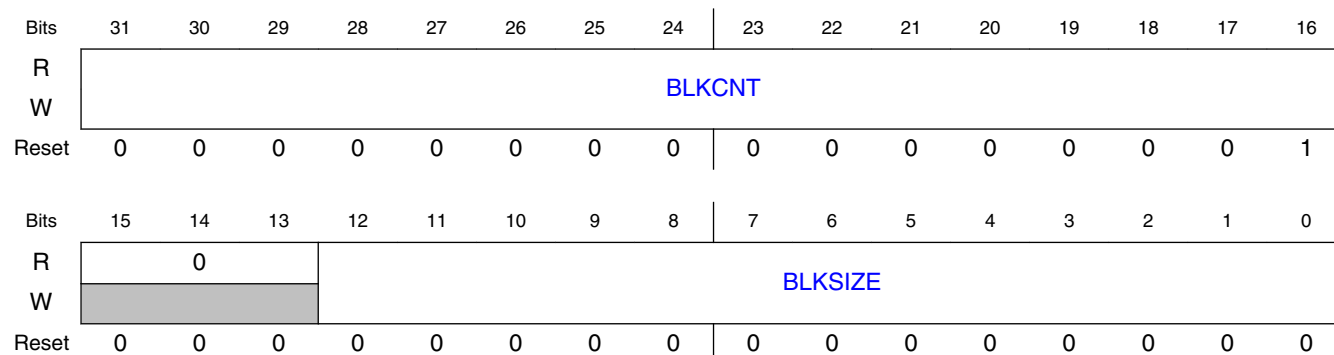
10.3.7.1.3.1 Offset

Register	Offset
BLK_ATT	4h

10.3.7.1.3.2 Function

This register is used to configure the number of data blocks and the number of bytes in each block.

10.3.7.1.3.3 Diagram



10.3.7.1.3.4 Fields

Field	Function
31-16 BLKCNT	Blocks count for current transfer This register is enabled when the Block Count Enable field in the Transfer Mode register is set to 1 and is valid only for multiple block transfers. For single block transfer, this register always reads as 1. The host driver sets this register to a value between 1 and the maximum block count. The uSDHC module decrements the block count after each block transfer and stops when the count reaches zero. Setting the block count to zero results in no data blocks being transferred. This register should be accessed only when no transaction is executing (that is, after transactions are stopped). During data transfer, read operations on this register may return an invalid value and write operations are ignored. When saving transfer content because of a Suspend command, the number of blocks yet to be transferred can be determined by reading this register. The reading of this register should be applied after transfer is paused by stop at block gap operation and before sending the command marked as suspend. This is because when the Suspend command is sent out, uSDHC treats the current transfer as aborted and change the BLKCNT register back to its original value instead of keeping the dynamical indicator of the remaining block count. When restoring transfer content prior to issuing a Resume command, the host driver restores the previously saved block count. NOTE: Although the BLKCNT field is 0 after reset, the read of reset value is 0x1. This is because when MSBSEL field is indicating a single block transfer, the read value of BLKCNT is always 1. 0000000000000000b - Stop count 0000000000000001b - 1 block 0000000000000010b - 2 blocks 1111111111111111b - 65535 blocks
15-13 —	Reserved
12-0 BLKSIZE	Transfer block size This register specifies the block size for block data transfers. Values ranging from 1 byte up to the maximum buffer size can be set. It can be accessed only when no transaction is executing (that is, after a transaction has stopped). Read operations during transfers may return an invalid value, and write operations are ignored. 00000000000000b - No data transfer 00000000000001b - 1 byte

Field	Function
	0000000000010b - 2 bytes 0000000000011b - 3 bytes 0000000000100b - 4 bytes 0000111111111b - 511 bytes 0001000000000b - 512 bytes 0100000000000b - 2048 bytes 1000000000000b - 4096 bytes

10.3.7.1.4 Command Argument (CMD_ARG)

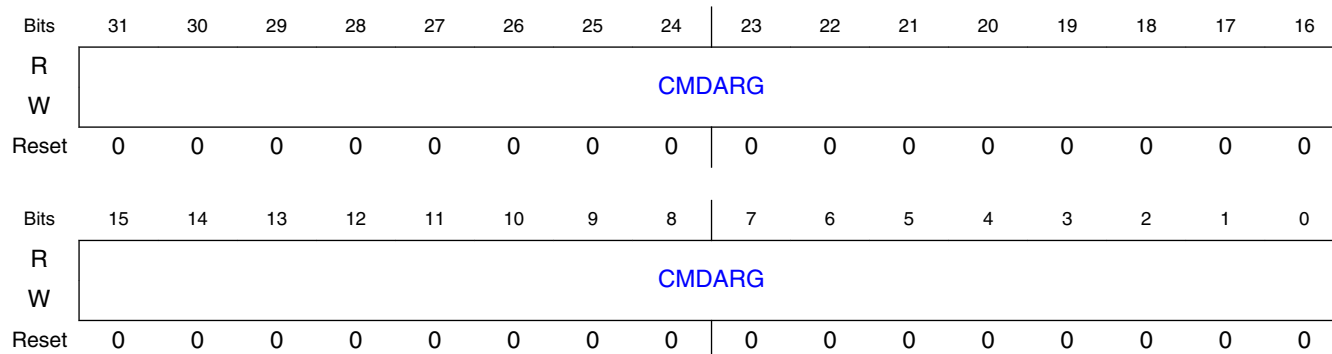
10.3.7.1.4.1 Offset

Register	Offset
CMD_ARG	8h

10.3.7.1.4.2 Function

This register contains the SD/MMC command argument.

10.3.7.1.4.3 Diagram



10.3.7.1.4.4 Fields

Field	Function
31-0	Command argument
CMDARG	The SD/MMC command argument is specified as bits 39-8 of the command format in the SD or MMC specification. This register is write protected when the Command Inhibit (CMD) field in the Present State register is set.

10.3.7.1.5 Command Transfer Type (CMD_XFR_TYP)

10.3.7.1.5.1 Offset

Register	Offset
CMD_XFR_TYP	Ch

10.3.7.1.5.2 Function

This register is used to control the operation of data transfers. The host driver sets this register before issuing a command followed by a data transfer or before issuing a Resume command. To prevent data loss, uSDHC prevents writing to the bits, which are involved in the data transfer of this register, when data transfer is active. These bits are DPSEL, MBSEL, DTDSEL, AC12EN, BCEN, and DMAEN.

The host driver checks the Command Inhibit DAT field ([PRES_STATE\[CDIHB\]](#)) and the Command Inhibit CMD field ([PRES_STATE\[CIHB\]](#)) in the Present State register before writing to this register. When the CDIHB field in the Present State register is set, any attempt to send a command with data by writing to this register is ignored; when the CIHB field is set, any write to this register is ignored.

On sending commands with data transfer involved, it is mandatory that the block size is non-zero. Block count must also be non-zero, or indicated as a single block transfer (bit 5 of this register is '0' when written), or block count is disabled (bit 1 of this register is '0' when written), otherwise uSDHC ignores the sending of this command and do nothing. For write command, with all above restrictions, it is also mandatory that the write protect switch is not active ([PRES_STATE\[WPSPL\]](#) field of Present State register is '1'); otherwise, uSDHC also ignores the command.

If the commands with data transfer do not receive the response in 64 clock cycles, that is, if response time-out happens, uSDHC treats the external device, does not accept the command, and aborts the data transfer. In this scenario, the driver should issue the command again to retry the transfer. It is also possible that for some reason the card responds to the command but uSDHC does not receive the response, and if it is internal DMA (either simple DMA or ADMA) read operation, the external system memory is over-written by the internal DMA with data sent back from the card.

The table below shows the summary of how register settings determine the type of data transfer.

Table 10-41. Transfer type register setting for various transfer types

Multi/single block select	Block count enable	Block count	Function
0	Do not care	Do not care	Single transfer
1	0	Do not care	Infinite transfer
1	1	Positive number	Multiple transfer
1	1	Zero	No data transfer

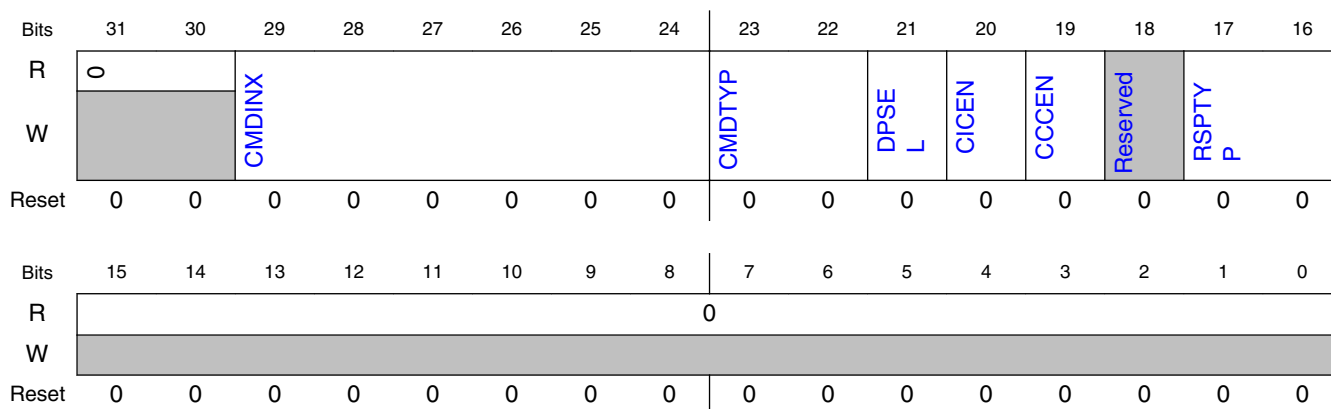
The table below shows the relationship between the Command Index Check Enable and the Command CRC Check Enable, regarding the Response Type bits as well as the name of the response type.

Table 10-42. Relationship between parameters and the name of the response type

Response type	Index check enable	CRC check enable	Name of response type
00	0	0	No response
01	0	1	R2
10	0	0	R3, R4
10	1	1	R1, R5, R6
11	1	1	R1b,R5b

- In the SDIO specification, response type notation for R5b is not defined. R5 includes R5b in the SDIO specification, but R5b is defined in this specification to specify that uSDHC checks the busy status after receiving a response. For example, usually CMD52 is used with R5, but the I/O abort command is used with R5b.
- The CRC fields for R3 and R4 are expected to be all 1 bits. The CRC check is disabled for these response types.

10.3.7.1.5.3 Diagram



10.3.7.1.5.4 Fields

Field	Function
31-30 —	Reserved
29-24 CMDINX	Command index These bits are set to the command number that is specified in bits 45-40 of the command-format in the SD Memory Card Physical Layer Specification and SDIO Card Specification.
23-22 CMDTYP	Command type There are three types of special commands: Suspend, Resume, and Abort. These bits are set to 00b for all other commands. • Suspend command: If the Suspend command succeeds, uSDHC assumes that the card bus has been released and that it is possible to issue the next command that uses the DATA line. Because uSDHC does not monitor the content of command response, it does not know if the Suspend command succeeded or not. It is the host driver's responsibility to check the status of the Suspend command and send another command marked as Suspend to inform uSDHC that a Suspend command was successfully issued. See Suspend Resume for more details. After the end bit of command is sent, uSDHC deasserts Read Wait for read transactions and stops checking busy for write transactions. In a 4-bit mode, the interrupt cycle starts. If the Suspend command fails, uSDHC maintains its current state, and the host driver restarts the transfer by setting the Continue Request field in the Protocol Control register. • Resume command: The host driver re-starts the data transfer by restoring the registers saved before sending the Suspend command and then sends the Resume command. The uSDHC module checks for a pending busy state before starting write transfers. • Abort command: If this command is set when executing a read transfer, uSDHC stops reads to the buffer. If this command is set when executing a write transfer, uSDHC stops driving the DATA line. After issuing the Abort command, the host driver should issue a software reset (Abort Transaction). 00b - Normal other commands 01b - Suspend CMD52 for writing bus suspend in CCCR 10b - Resume CMD52 for writing function select in CCCR 11b - Abort CMD12, CMD52 for writing I/O Abort in CCCR
21 DPSEL	Data present select This field is set to 1 to indicate that data is present and is transferred using the DATA line. It is set to 0 for the following: • Commands using only the CMD line (for example, CMD52) • Commands with no data transfer, but using the busy signal on DATA0 line (R1b or R5b (for example, CMD38)) NOTE: In resume command, this field is set, and other bits in this register is set the same as when the transfer was initially launched. When the write protect switch is on, (that is, the WPSPL field is active as '0'), any command with a write operation ignored. When this field is set, while the DTDSEL field is 0, writes to the register Transfer Type are ignored. 0b - No data present 1b - Data present
20 CICEN	Command index check enable If this field is set to 1, uSDHC checks the Index field in the response to see if it has the same value as the command index. If it is not, it is reported as a Command Index Error. If this field is set to 0, the Index field is not checked. 0b - Disable command index check 1b - Enables command index check

Table continues on the next page...

Field	Function
19 CCCEEN	Command CRC check enable If this field is set to 1, uSDHC checks the CRC field in the response. If an error is detected, it is reported as a Command CRC Error. If this field is set to 0, the CRC field is not checked. The number of bits checked by the CRC field value changes according to the length of the response. See RSPTYP[1:0] and Command Transfer Type (CMD_XFR_TYP) . 0b - Disables command CRC check 1b - Enables command CRC check
18 —	Reserved
17-16 RSPTYP	Response type select 00b - No response 01b - Response length 136 10b - Response length 48 11b - Response length 48, check busy after response
15-0 —	Reserved

10.3.7.1.6 Command Response0 (CMD_RSP0)

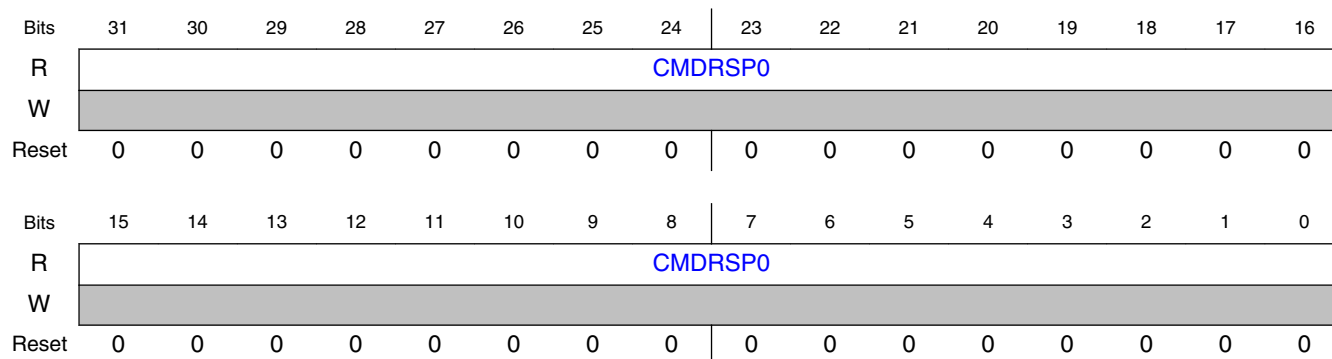
10.3.7.1.6.1 Offset

Register	Offset
CMD_RSP0	10h

10.3.7.1.6.2 Function

This register is used to store part 0 of the response bits from the card.

10.3.7.1.6.3 Diagram



10.3.7.1.6.4 Fields

Field	Function
31-0 CMDRSP0	Command response 0 See Command Response3 (CMD_RSP3) for the mapping of command responses from the SD bus to this register for each response type.

10.3.7.1.7 Command Response1 (CMD_RSP1)

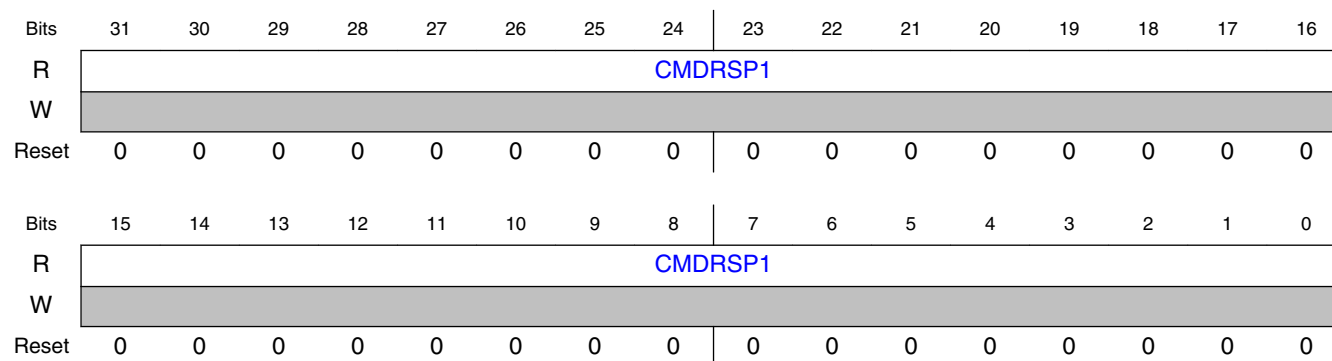
10.3.7.1.7.1 Offset

Register	Offset
CMD_RSP1	14h

10.3.7.1.7.2 Function

This register is used to store part 1 of the response bits from the card.

10.3.7.1.7.3 Diagram



10.3.7.1.7.4 Fields

Field	Function
31-0 CMDRSP1	Command response 1 See Command Response3 (CMD_RSP3) for the mapping of command responses from the SD bus to this register for each response type.

10.3.7.1.8 Command Response2 (CMD_RSP2)

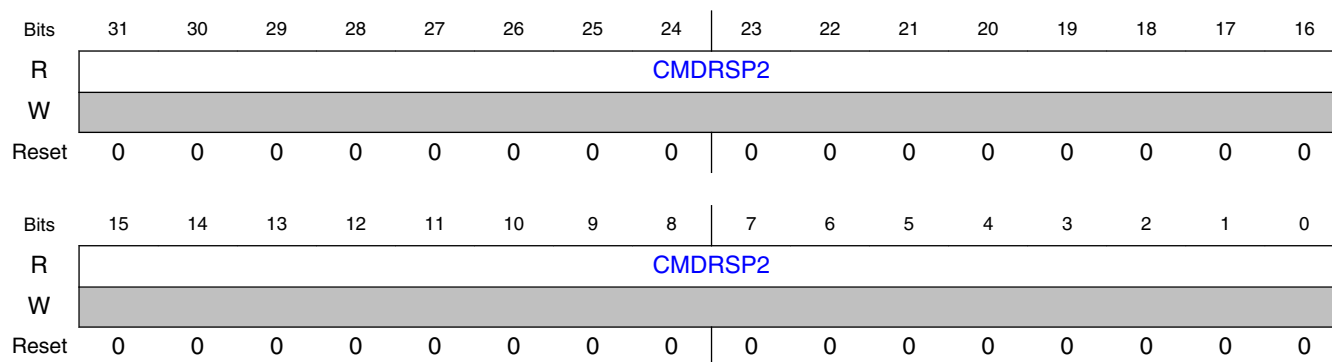
10.3.7.1.8.1 Offset

Register	Offset
CMD_RSP2	18h

10.3.7.1.8.2 Function

This register is used to store part 2 of the response bits from the card.

10.3.7.1.8.3 Diagram



10.3.7.1.8.4 Fields

Field	Function
31-0	Command response 2
CMDRSP2	See Command Response3 (CMD_RSP3) for the mapping of command responses from the SD bus to this register for each response type.

10.3.7.1.9 Command Response3 (CMD_RSP3)

10.3.7.1.9.1 Offset

Register	Offset
CMD_RSP3	1Ch

10.3.7.1.9.2 Function

This register is used to store part 3 of the response bits from the card.

The table below describes the mapping of command responses from the SD bus to Command Response registers for each response type. In this table, R[] refers to a bit range within the response data as transmitted on the SD bus.

Table 10-43. Response bit definition for each response type

Response type	Meaning of response	Response field	Response register
R1,R1b (normal response)	Card status	R[39:8]	CMDRSP0
R1b (auto CMD12 response)	Card status for auto CMD12	R[39:8]	CMDRSP3
R2 (CID, CSD register)	CID/CSD register [127:8]	R[127:8]	{CMDRSP3[23:0], CMDRSP2, CMDRSP1, CMDRSP0}
R3 (OCR register)	OCR register for memory	R[39:8]	CMDRSP0
R4 (OCR register)	OCR register for I/O etc.	R[39:8]	CMDRSP0
R5, R5b	SDIO response	R[39:8]	CMDRSP0
R6 (publish RCA)	New published RCA[31:16] and card status[15:0]	R[39:9]	CMDRSP0

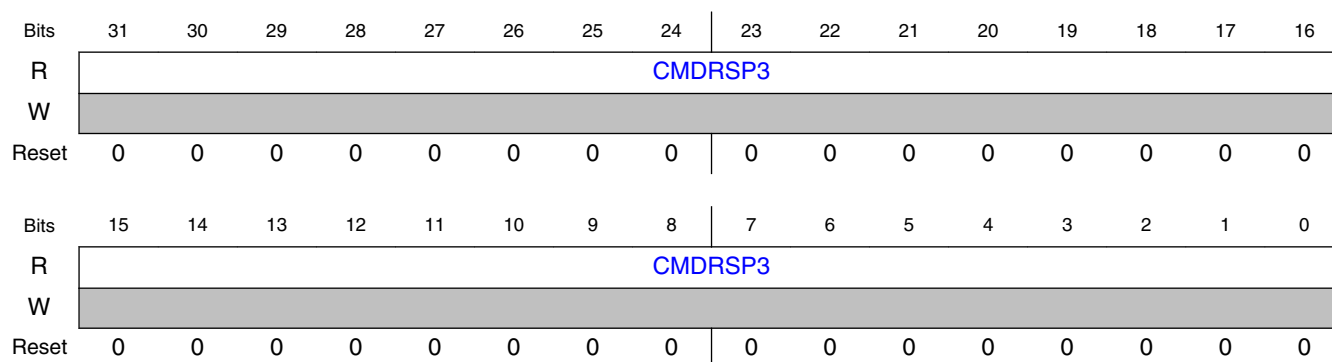
This table shows that most responses with a length of 48 (R[47:0]) have 32-bits of the response data (R[39:8]) stored in the CMDRSP0 register. Responses of type R1b (Auto CMD12 responses) have response data bits (R[39:8]) stored in the CMDRSP3 register. Responses with length 136 (R[135:0]) have 120-bits of the response data (R[127:8]) stored in the CMDRSP0, 1, 2, and 3 registers.

To be able to read the response status efficiently, uSDHC only stores part of the response data in the Command Response registers. This enables the host driver to efficiently read 32-bits of response data in one read cycle on a 32-bit bus system. Parts of the response, the Index field and the CRC, are checked by uSDHC (as specified by the Command Index Check Enable and the Command CRC Check Enable bits in the Transfer Type register) and generate an error interrupt if any error is detected. The bit range for the CRC check depends on the response length. If the response length is 48, uSDHC checks R[47:1], and if the response length is 136 the uSDHC checks R[119:1].

Because uSDHC may have a multiple block data transfer executing concurrently with a CMD_wo_DAT command, uSDHC stores the Auto CMD12 response in the CMDRSP3 register. The CMD_wo_DAT response is stored in CMDRSP0. This allows uSDHC to

avoid overwriting the Auto CMD12 response with the CMD_wo_DAT and vice versa. When uSDHC modifies part of the Command Response registers, as shown in the table above, it preserves the unmodified bits.

10.3.7.1.9.3 Diagram



10.3.7.1.9.4 Fields

Field	Function
31-0 CMDRSP3	Command response 3 See Command Response3 (CMD_RSP3) for the mapping of command responses from the SD bus to this register for each response type.

10.3.7.1.10 Data Buffer Access Port (DATA_BUFF_ACC_PORT)

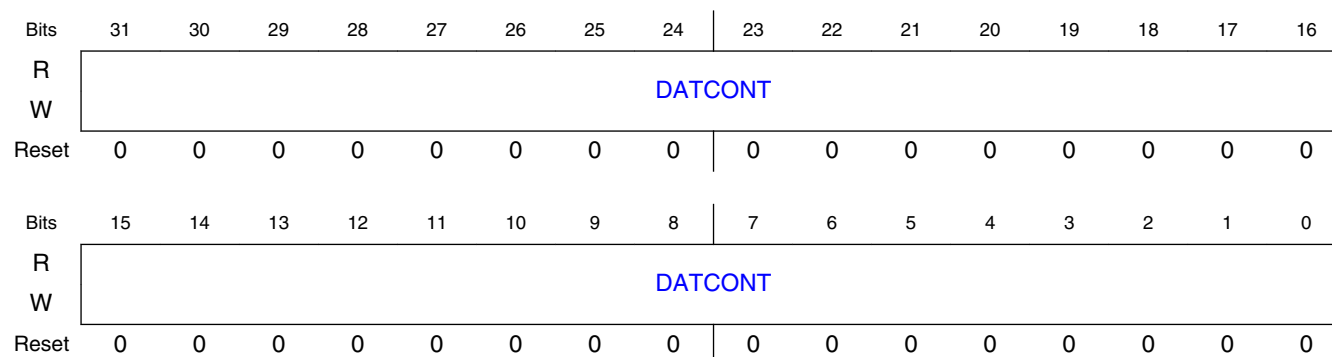
10.3.7.1.10.1 Offset

Register	Offset
DATA_BUFF_ACC_PORT	20h

10.3.7.1.10.2 Function

This is a 32-bit data port register used to access the internal buffer.

10.3.7.1.10.3 Diagram



10.3.7.1.10.4 Fields

Field	Function
31-0	Data content
DATCONT	The Buffer Data Port register is for 32-bit data access by the Arm platform or the external DMA. When the internal DMA is enabled, any write to this register is ignored, and any read from this register always yields 0s.

10.3.7.1.11 Present State (PRES_STATE)

10.3.7.1.11.1 Offset

Register	Offset
PRES_STATE	24h

10.3.7.1.11.2 Function

The host driver can get status of uSDHC from this 32-bit read only register.

The host driver can issue CMD0, CMD12, CMD13 (for memory) and CMD52 (for SDIO) when the DATA lines are busy during a data transfer. These commands can be issued when Command Inhibit (CMD) is set to zero. Other commands are issued when Command Inhibit (DATA) is set to zero. Possible changes to the SD Physical Specification may add other commands to this list in the future.

NOTE

The reset value of Present State register depends on board connectivity.

10.3.7.1.11.3 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	DLSL								CLSL	0			WPSP	CDPL	0	CINST
W																
Reset	u	u	u	u	u	u	u	u	u	0	0	0	u	u	0	u

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	TSCD	0		RT	BRE	BWEN	RTA	WTA	SDOF	PEROF	HCKOFF	IPGOFF	SDST	DLA	CDIHB	CIHB
W																
Reset	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0

10.3.7.1.11.4 Fields

Field	Function
31-24 DLSL	DATA[7:0] line signal level This status is used to check the DATA line level to recover from errors, and for debugging. This is especially useful in detecting the busy signal level from DATA0. The reset value is affected by the external pull-up / pull-down resistors. By default, the read value of this field after reset is 8'b11110111, when DATA3 is pulled down and the other lines are pulled up. 00000000b - Data 0 line signal level 00000001b - Data 1 line signal level 00000010b - Data 2 line signal level 00000011b - Data 3 line signal level 00000100b - Data 4 line signal level 00000101b - Data 5 line signal level 00000110b - Data 6 line signal level 00000111b - Data 7 line signal level
23 CLSL	CMD line signal level This status is used to check the CMD line level to recover from errors, and for debugging. The reset value is affected by the external pull-up / pull-down resistor, by default, the read value of this field after reset is 1'b1, when the command line is pulled up.
22-20 —	Reserved
19 WPSP	Write protect switch pin level The Write Protect switch is supported for memory and combo cards. This field reflects the inverted value of the WP pin of the card socket. A software reset does not affect this field. The reset value is affected by the external write protect switch. If the WP pin is not used, it should be tied low, so that the reset value of this field is high and write is enabled. 0b - Write protected (WP = 1) 1b - Write enabled (WP = 0)

Table continues on the next page...

Field	Function
18 CDPL	Card detect pin level This field reflects the inverse value of the CD_B pin for the card socket. Debouncing is not performed on this field. This field may be valid, but is not guaranteed, because of propagation delay. Use of this field is limited to testing because it must be debounced by software. A software reset does not affect this field. A write to the Force Event Register does not affect this field. The reset value is affected by the external card detection pin. This field shows the value on the CD_B pin (that is, when a card is inserted in the socket, it is 0 on the CD_B input, and consequently, the CDPL reads 1. 0b - No card present (CD_B = 1) 1b - Card present (CD_B = 0)
17 —	Reserved
16 CINST	Card inserted This field indicates whether a card has been inserted. The uSDHC module debounces this signal so that the host driver does not need to wait for it to stabilize. Changing from a 0 to 1 generates a Card Insertion interrupt in the Interrupt Status register. Changing from a 1 to 0 generates a Card Removal interrupt in the Interrupt Status register. A write to the Force Event Register does not affect this field. The Software Reset For All in the System Control register does not affect this field. A software reset does not affect this field. 0b - Power on reset or no card 1b - Card inserted
15 TSCD	Tape select change done This field indicates the delay setting is effective after write CLK_TUNE_CTRL_STATUS register. 0b - Delay cell select change is not finished. 1b - Delay cell select change is finished.
14-13 —	Reserved
12 RTR	Re-Tuning Request (only for SD3.0 SDR104 mode and EMMC HS200 mode) Host controller may request host driver to execute re-tuning sequence by setting this field when the data window is shifted by temperature drift and a tuned sampling point does not have a good margin to receive correct data. This field is cleared when a command is issued with setting Execute Tuning field in MIXER_CTRL register. Changing of this field from 0 to 1 generates Re-Tuning Event. See Interrupt status registers for more detail. This field isn't set to 1 if Sampling Clock Select in the MIXER_CTRL register is set to 0 (using fixed sampling clock). 0b - Fixed or well tuned sampling clock 1b - Sampling clock needs re-tuning
11 BREN	Buffer read enable This status field is used for non-DMA read transfers. The uSDHC module implements an internal buffer to transfer data efficiently. This read only flag indicates that valid data exists in the host side buffer. If this field is high, valid data greater than the watermark level exist in the buffer. A change of this field from 1 to 0 occurs when some reads from the buffer (read DATPORT (Base + 0x20)) are made and the buffer hasn't valid data greater than the watermark level. A change of this field from 0 to 1 occurs when there is enough valid data ready in the buffer and the Buffer Read Ready interrupt has been generated and enabled. 0b - Read disable

Table continues on the next page...

Field	Function
	1b - Read enable
10 BWEN	Buffer write enable This status field is used for non-DMA write transfers. The uSDHC module implements an internal buffer to transfer data efficiently. This read only flag indicates if space is available for write data. If this field is 1, valid data greater than the watermark level can be written to the buffer. A change of this field from 1 to 0 occurs when some writes to the buffer (write DATPORT(Base + 0x20)) are made and the buffer hasn't valid space greater than the watermark level. A change of this field from 0 to 1 occurs when the buffer can hold valid data greater than the write watermark level and the Buffer Write Ready interrupt is generated and enabled. 0b - Write disable 1b - Write enable
9 RTA	Read transfer active This status field is used for detecting completion of a read transfer. This field is set for either of the following conditions: • After the end field of the read command • When writing a 1 to the Continue Request field in the Protocol Control register to restart a read transfer A transfer complete interrupt is generated when this field changes to 0. This field is cleared for either of the following conditions: • When the last data block as specified by block length is transferred to the System, that is, all data are read away from uSDHC internal buffer. • When all valid data blocks have been transferred from uSDHC internal buffer to the System and no current block transfers are being sent because of the Stop At Block Gap Request being set to 1. 0b - No valid data 1b - Transferring data
8 WTA	Write transfer active This status field indicates a write transfer is active. If this field is 0, it means no valid write data exists in uSDHC. This field is set in either of the following cases: • After the end field of the write command • When writing 1 to the Continue Request field in the Protocol Control register to restart a write transfer This field is cleared in either of the following cases: • After getting the CRC status of the last data block as specified by the transfer count (Single and Multiple) • After getting the CRC status of any block where data transmission is about to be stopped by a Stop At Block Gap Request During a write transaction, a Block Gap Event interrupt is generated when this field is changed to 0, as result of the Stop At Block Gap Request being set. This status is useful for the host driver in determining when to issue commands during Write Busy state. 0b - No valid data 1b - Transferring data
7 SDOFF	SD clock gated off internally This status field indicates that the SD clock is internally gated off, because of buffer over / under-run or read pause without read wait assertion, or the driver set FRC_SDCLK_ON field is 0 to stop the SD clock in idle status. This field is for the host driver to debug data transaction on the SD bus.

Table continues on the next page...

Field	Function
	0b - SD clock is active. 1b - SD clock is gated off.
6 PEROFF	IPG_PERCLK gated off internally This status field indicates that the IPG_PERCLK is internally gated off. This field is for the host driver to debug transaction on the SD bus. When IPG_CLK_SOFT_EN is cleared, IPG_PERCLK is gated off, otherwise IPG_PERCLK is always active. 0b - IPG_PERCLK is active. 1b - IPG_PERCLK is gated off.
5 HCKOFF	HCLK gated off internally This status field indicates that the HCLK is internally gated off. This field is for the host driver to debug during a data transfer. 0b - HCLK is active. 1b - HCLK is gated off.
4 IPGOFF	Peripheral clock gated off internally This status field indicates that the peripheral clock is internally gated off. This field is for the host driver to debug. 0b - Peripheral clock is active. 1b - Peripheral clock is gated off.
3 SDSTB	SD clock stable This status field indicates that the internal card clock is stable. This field is for the host driver to poll clock status when changing the clock frequency. It is recommended to clear FRC_SDCLK_ON field in System Control register to remove glitches on the card clock when the frequency is changing. Before changing clock divisor value (SDCLKFS or DVS), host driver should make sure the SDSTB field is high. 0b - Clock is changing frequency and not stable. 1b - Clock is stable.
2 DLA	Data line active This status field indicates whether one of the DATA lines on the SD bus is in use. In the case of read transactions: This status indicates if a read transfer is executing on the SD bus. Changes in this value from 1 to 0, between data blocks, generates a Block Gap Event interrupt in the Interrupt Status register. This field is set in either of the following cases: • After the end field of the read command • When writing a 1 to the Continue Request field in the Protocol Control register to restart a read transfer This field is cleared in either of the following cases: • When the end field of the last data block is sent from the SD bus to uSDHC. • When the Read Wait state is stopped by a Suspend command and the DATA2 line is released. The uSDHC module waits at the next block gap by driving Read Wait at the start of the interrupt cycle. If the Read Wait signal is already driven (data buffer cannot receive data), uSDHC can wait for a current block gap by continuing to drive the Read Wait signal. It is necessary to support Read Wait to use the suspend / resume function. This field remains 1 during Read Wait. In the case of write transactions: This status indicates that a write transfer is executing on the SD bus. Changes in this value from 1 to 0 generate a Transfer Complete interrupt in the Interrupt Status register.

Table continues on the next page...

Field	Function
	This field is set in either of the following cases: • After the end field of the write command • When writing to 1 to the Continue Request field in the Protocol Control register to continue a write transfer This field is cleared in either of the following cases: • When the SD card releases Write Busy of the last data block, uSDHC also detects if the output is not busy. If the SD card does not drive the busy signal after the CRC status is received, uSDHC assumes the card drive "Not Busy". • When the SD card releases write busy, prior to waiting for write transfer, and because of a Stop At Block Gap Request. In the case of command with busy pending: This status indicates that a busy state follows the command and the data line is in use. This field is cleared when the DATA0 line is released. 0b - DATA line inactive 1b - DATA line active
1 CDIHB	Command inhibit (DATA) This status field is generated if either the DAT Line Active or the Read Transfer Active is set to 1. If this field is 0, it indicates that uSDHC can issue the next SD / MMC Command. Commands with a busy signal belong to Command Inhibit (DATA) (for example. R1b, R5b type). Changing from 1 to 0 generates a Transfer Complete interrupt in the Interrupt Status register. NOTE: The SD host driver can save registers for a suspend transaction after this field has changed from 1 to 0. 0b - Can issue command that uses the DATA line 1b - Cannot issue command that uses the DATA line
0 CIHB	Command inhibit (CMD) If this status bit is 0, it indicates that the CMD line is not in use and uSDHC can issue a SD / MMC command using the CMD line. This field is set also immediately after the Transfer Type register is written. This field is cleared when the command response is received. Even if the Command Inhibit (DATA) is set to 1, commands using only the CMD line can be issued if this field is 0. Changing from 1 to 0 generates a Command Complete interrupt in the Interrupt Status register. If uSDHC cannot issue the command because of a command conflict error (see Command CRC Error) or because of a Command Not Issued By Auto CMD12 Error, this field remains 1 and the Command Complete is not set. The Status of issuing an auto CMD12 does not show on this field. 0b - Can issue command using only CMD line 1b - Cannot issue command

10.3.7.1.12 Protocol Control (PROT_CTRL)

10.3.7.1.12.1 Offset

Register	Offset
PROT_CTRL	28h

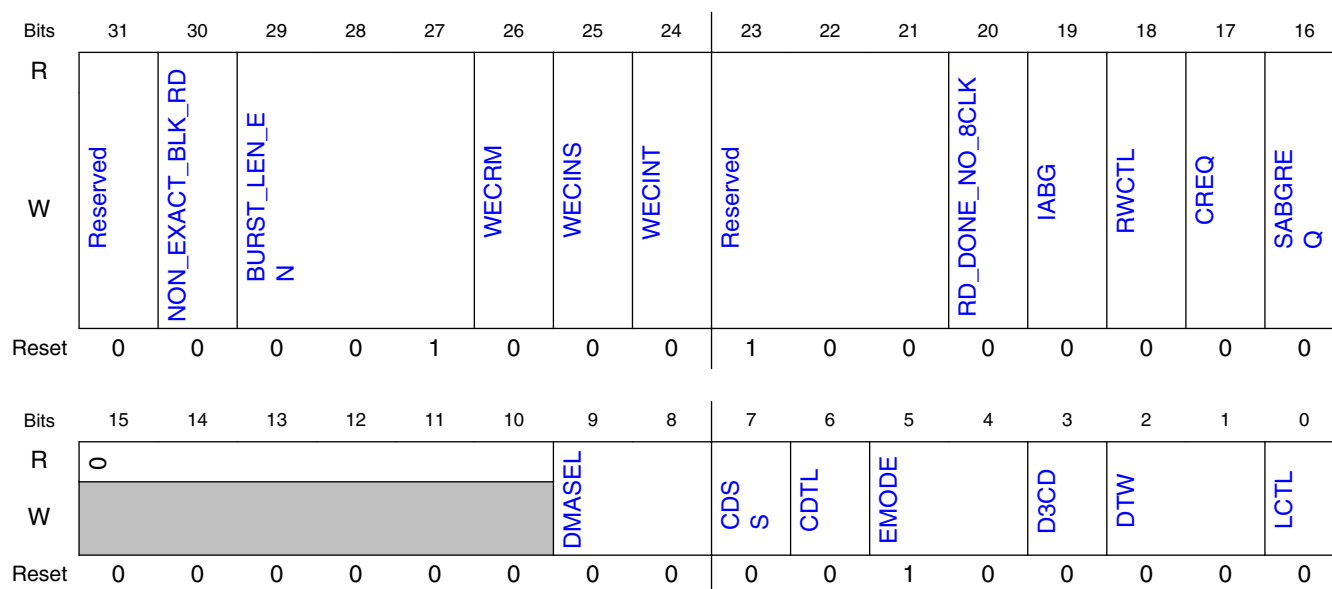
10.3.7.1.12.2 Function

There are three cases to restart the transfer after stop at the block gap. Which case is appropriate depends on whether uSDHC issues a Suspend command or the SD card accepts the Suspend command.

- If the host driver does not issue a Suspend command, the Continue request is used to restart the transfer.
- If the host driver issues a Suspend command and the SD card accepts it, a Resume command is used to restart the transfer.
- If the host driver issues a Suspend command and the SD card does not accept it, the Continue request is used to restart the transfer.

Any time stop at block gap request stops the data transfer, the host driver waits for a Transfer Complete (in the Interrupt Status register), before attempting to restart the transfer. When restarting the data transfer by Continue Request, the host driver clears the Stop At Block Gap Request before or simultaneously.

10.3.7.1.12.3 Diagram



10.3.7.1.12.4 Fields

Field	Function
31	Reserved
—	Always write as 0

Table continues on the next page...

Field	Function
30 NON_EXACT_BLOCK_READ	Non-exact block read Current block read is non-exact block read. It is only used for SDIO. 0b - The block read is exact block read. Host driver does not need to issue abort command to terminate this multi-block read. 1b - The block read is non-exact block read. Host driver needs to issue abort command to terminate this multi-block read.
29-27 BURST_LENGTH_ENABLE	BURST length enable for INCR, INCR4 / INCR8 / INCR16, INCR4-WRAP / INCR8-WRAP / INCR16-WRAP This is used to enable / disable the burst length for the external AHB2AXI bridge. It is useful especially for INCR transfer because without burst length indicator, the AHB2AXI bridge does not know the burst length in advance. Without burst length indicator, AHB INCR transfers can only be converted to SINGLES on the AXI side. 1xxb - Burst length is enabled for INCR4-WRAP / INCR8-WRAP / INCR16-WRAP. x1xb - Burst length is enabled for INCR4 / INCR8 / INCR16. xx1b - Burst length is enabled for INCR.
26 WECRM	Wakeup event enable on SD card removal This field enables a wakeup event, via a card removal, in the Interrupt Status register. FN_WUS (Wake Up Support) in CIS does not affect this field. When this field is set, the Card Removal Status and uSDHC interrupt can be asserted without CLK toggling. When the wakeup feature is not enabled, the CLK must be active to assert the Card Removal Status and uSDHC interrupt. 0b - Disables wakeup event enable on SD card removal 1b - Enables wakeup event enable on SD card removal
25 WECINS	Wakeup event enable on SD card insertion This field enables a wakeup event, via a card insertion, in the Interrupt Status register. FN_WUS (Wake Up Support) in CIS does not affect this field. When this field is set, the Card Insertion Status and uSDHC interrupt can be asserted without CLK toggling. When the wakeup feature is not enabled, the CLK must be active to assert the Card Insertion Status and uSDHC interrupt. 0b - Disable wakeup event enable on SD card insertion 1b - Enable wakeup event enable on SD card insertion
24 WECINT	Wakeup event enable on card interrupt This field enables a wakeup event, via a card interrupt, in the Interrupt Status register. This field can be set to 1 if FN_WUS (Wake Up Support) in CIS is set to 1. When this field is set, the Card Interrupt Status and uSDHC interrupt can be asserted without CLK toggling. When the wakeup feature is not enabled, the CLK must be active to assert the Card Interrupt Status and uSDHC interrupt. 0b - Disables wakeup event enable on card interrupt 1b - Enables wakeup event enable on card interrupt
23-21 —	Reserved Always write as 3'b100
20 RD_DONE_NO_8CLK	Read performed number 8 clock According to the SD/MMC spec, for read data transaction, 8 clocks are needed after the end field of the last data block. So, by default(RD_DONE_NO_8CLK=0), 8 clocks are active after the end field of the last read data transaction. However, these 8 clocks should not be active if user wants to use stop at block gap (include the auto stop at block gap in boot mode) feature for read and the RWCTL field (bit18) is not enabled. In this case, software should set RD_DONE_NO_8CLK to avoid these 8 clocks. Otherwise, the device might send extra data to uSDHC while uSDHC ignores these data. In a summary, this field should be set only if the use case needs to use stop at block gap feature while the device can't support the read wait feature.

Table continues on the next page...

Field	Function
19 IABG	Interrupt at block gap This field is valid only in 4-bit mode, of the SDIO card, and selects a sample point in the interrupt cycle. Setting to 1 enables interrupt detection at the block gap for a multiple block transfer. Setting to 0 disables interrupt detection during a multiple block transfer. If the SDIO card cannot signal an interrupt during a multiple block transfer, this field should be set to 0 to avoid an inadvertent interrupt. When the host driver detects an SDIO card insertion, it sets this field according to the CCCR of the card. 0b - Disables interrupt at block gap 1b - Enables interrupt at block gap
18 RWCTL	Read wait control The read wait function is optional for SDIO cards. If the card supports read wait, set this field to enable use of the read wait protocol to stop read data using the DATA2 line. Otherwise, uSDHC has to stop the SD clock to hold read data, which restricts commands generation. When the host driver detects an SDIO card insertion, it sets this field according to the CCCR of the card. If the card does not support read wait, this field should never be set to 1; otherwise, DATA line conflicts might occur. If this field is set to 0, stop at block gap during read operation is also supported, but uSDHC stops the SD clock to pause reading operation. 0b - Disables read wait control and stop SD clock at block gap when SABGREQ field is set 1b - Enables read wait control and assert read wait without stopping SD clock at block gap when SABGREQ field is set
17 CREQ	Continue request This field is used to restart a transaction which was stopped using the stop at block gap request. When a suspend operation is not accepted by the card, it is also by setting this field to restart the paused transfer. To cancel stop at the block gap, set stop at block gap request to 0 and set this field to 1 to restart the transfer. The uSDHC module automatically clears this field, therefore it is not necessary for the host driver to set this field to 0. If both stop at block gap request and this field are 1, the continue request is ignored. 0b - No effect 1b - Restart
16 SABGREQ	Stop at block gap request This field is used to stop executing a transaction at the next block gap for both DMA and non-DMA transfers. Until the transfer complete is set to 1, indicating a transfer completion, the host driver leaves this field set to 1. Clearing both the stop at block gap request and continue request does not cause the transaction to restart. Read Wait is used to stop the read transaction at the block gap. The uSDHC module supports the stop at block gap request for write transfers, but for read transfers it requires that the SDIO card support read wait. Therefore, the host driver does not set this field during read transfers unless the SDIO card supports Read Wait and has set the read wait control to 1; otherwise, uSDHC stops the SD bus clock to pause the read operation during block gap. In the case of write transfers in which the host driver writes data to the Data Port register, the host driver sets this field after all block data is written. If this field is set to 1, the host driver does not write data to the Data Port register after a block is sent. Once this field is set, the host driver does not clear this field before the Transfer Complete field in Interrupt Status register is set, otherwise uSDHC's behavior is undefined. This field effects read transfer active, write transfer active, DATA Line Active and Command Inhibit (DATA) in the Present State register. 0b - Transfer 1b - Stop
15-10 —	Reserved
9-8 DMASEL	DMA select This field is valid while DMA (SDMA or ADMA) is enabled and selects the DMA operation.

Table continues on the next page...

Field	Function
	00b - No DMA or simple DMA is selected. 01b - ADMA1 is selected. 10b - ADMA2 is selected. 11b - Reserved
7 CDSS	Card detect signal selection This field selects the source for the card detection. 0b - Card detection level is selected (for normal purpose). 1b - Card detection test level is selected (for test purpose).
6 CDTL	Card detect test level This bit is enabled while the card detection signal selection is set to 1 and it indicates card insertion. 0b - Card detect test level is 0, no card inserted 1b - Card detect test level is 1, card inserted
5-4 EMODE	Endian mode The uSDHC module supports all three endian modes in data transfer. See Data buffer for more details. 00b - Big endian mode 01b - Half word big endian mode 10b - Little endian mode 11b - Reserved
3 D3CD	DATA3 as card detection pin If this field is set, DATA3 should be pulled down to act as a card detection pin. Be cautious when using this feature, because DATA3 is also a chip-select for the SPI mode. A pull-down on this pin and CMD0 might set the card into the SPI mode, which uSDHC does not support. 0b - DATA3 does not monitor card insertion 1b - DATA3 as card detection pin
2-1 DTW	Data transfer width This field selects the data width of the SD bus for a data transfer. The host driver sets it to match the data width of the card. Possible data transfer width is 1-bit, 4-bits or 8-bits. 00b - 1-bit mode 01b - 4-bit mode 10b - 8-bit mode 11b - Reserved
0 LCTL	LED control This field, fully controlled by the host driver, is used to caution the user not to remove the card while the card is being accessed. If the software is going to issue multiple SD commands, this field can be set during all these transactions. It is not necessary to change for each transaction. When the software issues multiple SD commands, setting the field once before the first command is sufficient: it is not necessary to reset the bit between commands. 0b - LED off 1b - LED on

10.3.7.1.13 System Control (SYS_CTRL)

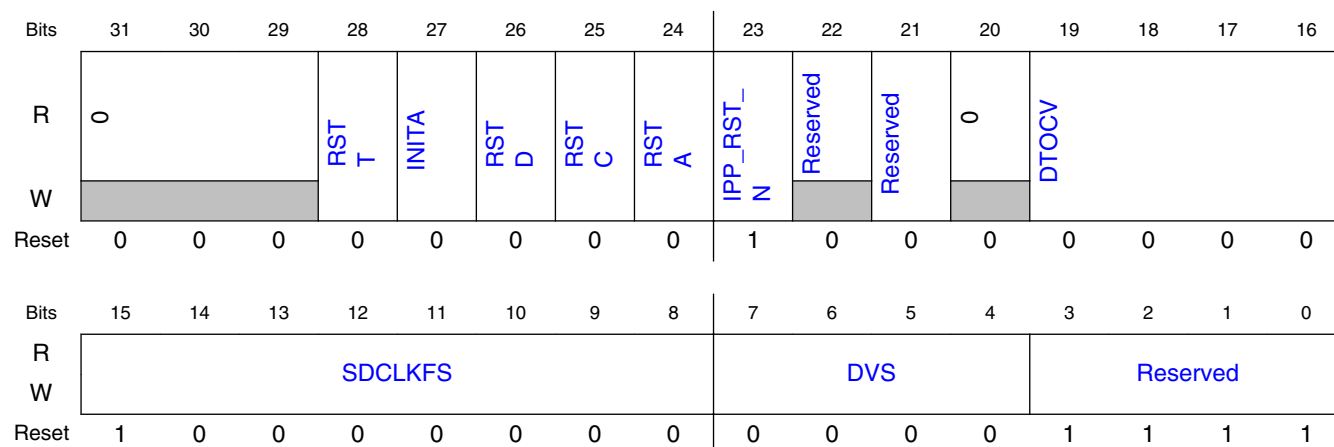
10.3.7.1.13.1 Offset

Register	Offset
SYS_CTRL	2Ch

10.3.7.1.13.2 Function

This register provides control of the system. See detail in the field description.

10.3.7.1.13.3 Diagram



10.3.7.1.13.4 Fields

Field	Function
31-29 —	Reserved
28 RSTT	Reset tuning When set this field to 1, it resets tuning circuit. After tuning circuits are reset, bit value is 0. Clearing execute_tuning field in AUTOCMD12_ERR_STATUS also sets this field to 1 to reset tuning circuit
27 INITA	Initialization active When this field is set, 80 SD-clocks are sent to the card. After the 80 clocks are sent, this field is self cleared. This field is very useful during the card power-up period when 74 SD-clocks are needed and the clock auto gating feature is enabled. Writing 1 to this bit when this field is already 1 has no effect. Writing 0 to this field at any time has no effect. When either of the CIHB and CDIHB fields in the Present State register are set, writing 1 to this field is ignored (that is, when command line or data lines are active, write to this field is not allowed). On the other-hand, when this field is set, that is, during initialization active period, it is allowed to issue command, and the command bit stream appears on the CMD pad after all 80 clock cycles are done. So, when this command ends, the driver can make sure the 80 clock cycles are sent out. This is very useful when the driver needs to send 80 cycles to the card and does not want to wait till this field is self cleared.
26	Software reset for data line

Table continues on the next page...

Field	Function
RSTD	Only part of the data circuit is reset. DMA circuit is also reset. After this field is set, the software waits for self-clear. The following registers and bits are cleared by this field: • Data Port register • Buffer is cleared and initialized • Present State register • Buffer read enable • Buffer write enable • Read transfer active • Write transfer active • DATA line active • Command Inhibit (DATA) Protocol Control register • Continue request • Stop At Block Gap Request Interrupt Status register • Buffer read ready • Buffer write ready • DMA interrupt • Block gap event • Transfer complete NOTE: When reset, the software must make sure there is no incomplete data transferring. If there is data transfer going on, the software needs to wait TC or DC INT_STATUS register is set. 0b - No reset 1b - Reset
25 RSTC	Software reset for CMD line Only part of the command circuit is reset. After this field is set, the software waits for self-clear. The following registers and bits are cleared by this field: • Present State Register Command Inhibit (CMD) • Interrupt Status register Command Complete 0b - No reset 1b - Reset
24 RSTA	Software reset for all This reset effects the entire host controller except for the card detection circuit. Register bits of type ROC, RW, RW1C, RWAC are cleared. During its initialization, the host driver is set this field to 1 to reset uSDHC. The uSDHC module resets this field to 0 when the capabilities registers are valid and the host driver can read them. Additional use of Software Reset For All does not affect the value of the capabilities registers. After this field is set, it is recommended that the host driver reset the external card and re-initialize it. After this field is set, the software should wait for self-clear. In tuning process, after every CMD19 is finished, this field is set to retest uSDHC. NOTE: When reset, the software must make sure there is no incomplete data transferring. If there is data transfer going on, the software needs to wait TC or DC INT_STATUS register is set. 0b - No reset 1b - Reset
23 IPP_RST_N	Hardware reset This register's value is output to card through pad directly to hardware reset pin of the card if the card supports this feature.
22 —	Reserved
21	Reserved

Table continues on the next page...

Field	Function
—	
20	Reserved
—	
19-16 DTCV	Data timeout counter value This value determines the interval by which DAT line timeouts are detected. See the Data Timeout Error field in the Interrupt Status register for information on factors that dictate time-out generation. Time-out clock frequency is generated by dividing the base clock SDCLK value by this value. The host driver can clear the Data Timeout Error Status Enable (in the Interrupt Status Enable register) to prevent inadvertent time-out events. 0000b - SDCLK x 2¹⁴ 0001b - SDCLK x 2¹⁵ 0010b - SDCLK x 2¹⁶ 0011b - SDCLK x 2¹⁷ 0100b - SDCLK x 2¹⁸ 0101b - SDCLK x 2¹⁹ 0110b - SDCLK x 2²⁰ 0111b - SDCLK x 2²¹ 1000b - SDCLK x 2²² 1001b - SDCLK x 2²³ 1010b - SDCLK x 2²⁴ 1011b - SDCLK x 2²⁵ 1100b - SDCLK x 2²⁶ 1101b - SDCLK x 2²⁷ 1110b - SDCLK x 2²⁸ 1111b - SDCLK x 2²⁹
15-8 SDCLKFS	SDCLK frequency select This register is used to select the frequency of the SDCLK pin. The frequency is not programmed directly, rather this register holds the prescaler (this register) and divisor (next register) of the Base Clock Frequency register. In Single Data Rate mode (DDR_EN field of MIXERCTRL is '0') Only the following settings are allowed: 80h) Base clock divided by 256 40h) Base clock divided by 128 20h) Base clock divided by 64 10h) Base clock divided by 32 08h) Base clock divided by 16 04h) Base clock divided by 8 02h) Base clock divided by 4 01h) Base clock divided by 2 00h) Base clock divided by 1 While in Dual Data Rate mode (DDR_EN field of MIXERCTRL is '1') Only the following settings are allowed: 80h) Base clock divided by 512 40h) Base clock divided by 256 20h) Base clock divided by 128

Table continues on the next page...

Field	Function
	10h) Base clock divided by 64 08h) Base clock divided by 32 04h) Base clock divided by 16 02h) Base clock divided by 8 01h) Base clock divided by 4 00h) Base clock divided by 2 When the software changes the DDR_EN field, SDCLKFS might need to be changed also. In Single Data Rate mode, setting 00h bypasses the frequency prescaler of the SD clock. Multiple bits must not be set, or the behavior of this prescaler is undefined. The two default divider values can be calculated by the frequency of ipg_perclk and the following Divisor bits. The frequency of SDCLK is set by the following formula: Clock Frequency = (Base Clock) / (prescaler x divisor) For example, in Single Data Rate mode, if the Base Clock Frequency is 96 MHz, and the target frequency is 25 MHz, then choosing the prescaler value of 01h and divisor value of 1h yields 24 MHz, which is the nearest frequency less than or equal to the target. Similarly, to approach a clock value of 400 kHz, the prescaler value of 08h and divisor value of eh yields the exact clock value of 400 kHz. The reset value of this field is 80h, so if the input Base Clock (ipg_perclk) is about 96 MHz, the default SD clock after reset is 375 kHz. According to the SD Physical Specification Version 1.1 and the SDIO Card Specification Version 1.2, the maximum SD clock frequency is 50 MHz and is never exceed this limit. Before changing clock divisor value (SDCLKFS or DVS), host driver should make sure the SDSTB field is high. If setting SDCLKFS and DVS can generate the same clock frequency,(for example, in SDR mode, SDCLKFS = 01h is same as DVS = 01h.) SDCLKFS is highly recommended.
7-4 DVS	Divisor This register is used to provide a more exact divisor to generate the desired SD clock frequency. Note the divider can even support odd divisors without deterioration of duty cycle. Before changing clock divisor value (SDCLKFS or DVS), Host driver should make sure the SDSTB field is high. The settings are as follows: 0000b - Divide-by-1 0001b - Divide-by-2 1110b - Divide-by-15 1111b - Divide-by-16
3-0 —	Reserved Always write as 1.

10.3.7.1.14 Interrupt Status (INT_STATUS)

10.3.7.1.14.1 Offset

Register	Offset
INT_STATUS	30h

10.3.7.1.14.2 Function

An interrupt is generated when the normal interrupt signal enable is enabled and at least one of the status fields is set to 1. For all fields, writing 1 to a bit clears it; writing to 0 keeps the bit unchanged. More than one status can be cleared with a single register write. For card interrupt, before writing 1 to clear, it is required that the card stops asserting the interrupt, meaning that when the card driver services the interrupt condition; otherwise, the CINT field is asserted again.

The table below shows the relationship between the command timeout error and the command complete.

Table 10-44. uSDHC status for command timeout error/command complete bit combinations

Command CRC Error	Command timeout error	Meaning of the status
0	0	X
X	1	Response not received within 64 SDCLK cycles
1	0	Response received

The table below shows the relationship between the transfer complete and the data timeout error.

Table 10-45. uSDHC status for data timeout error/transfer complete bit combinations

Transfer complete	Data timeout error	Meaning of the status
0	0	X
0	1	Timeout occurred during transfer
1	X	Data transfer complete

The table below shows the relationship between the command CRC error and command timeout error.

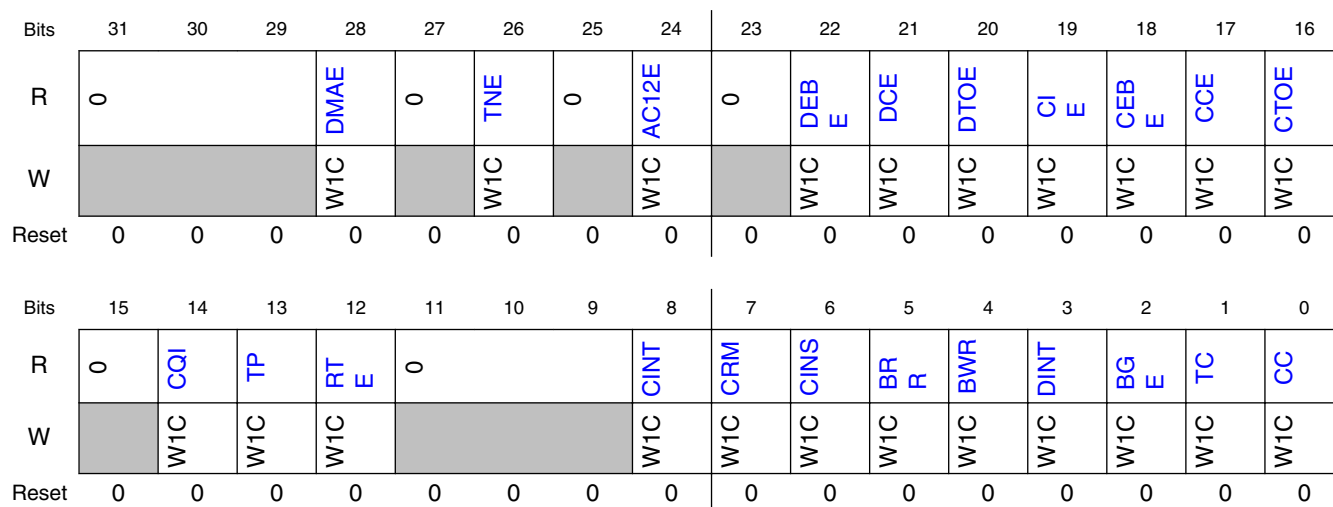
Table 10-46. uSDHC status for command CRC error/command timeout error bit combinations

Command complete	Command timeout error	Meaning of the status
0	0	No error

Table continues on the next page...

Table 10-46. uSDHC status for command CRC error/command timeout error bit combinations (continued)

Command complete	Command timeout error	Meaning of the status
0	1	Response timeout error
1	0	Response CRC error
1	1	CMD line conflict

10.3.7.1.14.3 Diagram**10.3.7.1.14.4 Fields**

Field	Function
31-29 —	Reserved
28 DMAE	DMA error Occurs when an Internal DMA transfer has failed. This field is set to 1, when some error occurs in the data transfer. This error can be caused by either simple DMA or ADMA, depending on which DMA is in use. The value in DMA System Address register is the next fetch address where the error occurs. Because any error corrupts the whole data block, the host driver restarts the transfer from the corrupted block boundary. The address of the block boundary can be calculated either from the current DS_ADDR value or from the remaining number of blocks and the block size. 0b - No error 1b - Error
27 —	Reserved
26 TNE	Tuning error: (only for SD3.0 SDR104 mode and EMMC HS200 mode)

Table continues on the next page...

Field	Function
	This field is set when an unrecoverable error is detected in a tuning circuit. By detecting Tuning Error, host driver needs to abort a command executing and perform tuning.
25 —	Reserved
24 AC12E	Auto CMD12 error Occurs when detecting that one of the fields in the Auto CMD12 Error Status register has changed from 0 to 1. This field is set to 1, not only when the errors in Auto CMD12 occur, but also, when the Auto CMD12 is not executed due to the previous command error. 0b - No error 1b - Error
23 —	Reserved
22 DEBE	Data end bit error Occurs either when detecting 0 at the end field position of read data that uses the DATA line, or at the end field position of the CRC. This field is not asserted in tuning process. 0b - No error 1b - Error
21 DCE	Data CRC error Occurs when detecting a CRC error when transferring read data that uses the DATA line, or when detecting the Write CRC status having a value other than 010. This field is not asserted in tuning process. 0b - No error 1b - Error
20 DTOE	Data timeout error Occurs when detecting one of following time-out conditions. • Busy time-out for R1b, R5b type • Busy time-out after Write CRC status • Read Data time-out. This field is not asserted in tuning process. 0b - No error 1b - Time out
19 CIE	Command index error Occurs if a command index error occurs in the command response. This field is not asserted in tuning process. 0b - No error 1b - Error
18 CEBE	Command end bit error Occurs when detecting that the end field of a command response is 0. This field is not asserted in tuning process. 0b - No error 1b - End bit error generated
17	Command CRC error

Table continues on the next page...

Field	Function
CCE	Command CRC Error is generated in two cases. • If a response is returned and the Command Timeout Error is set to 0 (indicating no time-out), this field is set when detecting a CRC error in the command response. • The uSDHC module detects a CMD line conflict by monitoring the CMD line when a command is issued. If uSDHC drives the CMD line to 1, but detects 0 on the CMD line at the next SDCLK edge, then uSDHC aborts the command (Stop driving CMD line) and set this bit to 1. The Command Timeout Error should also be set to 1 to distinguish CMD line conflict. This field is not asserted in tuning process. 0b - No error 1b - CRC error generated
16 CTOE	Command timeout error Occurs only if no response is returned within 64 SDCLK cycles from the end field of the command. If uSDHC detects a CMD line conflict, in which case a Command CRC Error is also be set (as shown in Interrupt Status (INT_STATUS)), this field is set without waiting for 64 SDCLK cycles. This is because the command is aborted by uSDHC. This field is not asserted in tuning process. 0b - No error 1b - Time out
15 —	Reserved
14 CQI	Command queuing interrupt This interrupt is asserted when at least one of the bits in CQIS register is set. This interrupt is cleared only by clearing the source interrupt in CQIS register.
13 TP	Tuning pass:(only for SD3.0 SDR104 mode and EMMC HS200 mode) Current CMD19 transfer is done successfully. That is, current sampling point is correct.
12 RTE	Re-tuning event: (only for SD3.0 SDR104 mode and EMMC HS200 mode) This status is set if Re-Tuning Request in the Present State register changes from 0 to 1. host controller requests host driver to perform re-tuning for next data transfer. Current data transfer (not large block count) can be completed without re-tuning. 0b - Re-tuning is not required. 1b - Re-tuning should be performed.
11-9 —	Reserved
8 CINT	Card interrupt This status field is set when an interrupt signal is detected from the external card. In 1-bit mode, uSDHC detects the Card Interrupt without the SD clock to support wakeup. In 4-bit mode, the card interrupt signal is sampled during the interrupt cycle, so the interrupt from card can only be sampled during interrupt cycle, introducing some delay between the interrupt signal from the SDIO card and the interrupt to the host system. Writing this field to 1 can clear this field, but as the interrupt source from the SDIO card does not clear, this field is set again. to clear this field, it is required to reset the interrupt source from the external card followed by a writing 1 to this field. When this status has been set, and the host driver needs to service this interrupt, the Card Interrupt Signal Enable in the Interrupt Signal Enable register should be 0 to stop driving the interrupt signal to the host system. After completion of the card interrupt service (It should reset the interrupt sources in the SDIO card and the interrupt signal might not be asserted), write 1 to clear this field, set the Card Interrupt Signal Enable to 1, and start sampling the interrupt signal again. 0b - No card interrupt

Table continues on the next page...

Field	Function
	1b - Generate card interrupt
7 CRM	Card removal This status field is set if the Card Inserted field in the Present State register changes from 1 to 0. When the host driver writes this field to 1 to clear this status, the status of the Card Inserted in the Present State register should be confirmed. Because the card state might possibly be changed when the host driver clears this field and the interrupt event might not be generated. When this field is cleared, it is set again if no card is inserted. to leave it cleared, clear the Card Removal Status Enable field in Interrupt Status Enable register. 0b - Card state unstable or inserted 1b - Card removed
6 CINS	Card insertion This status field is set if the Card Inserted field in the Present State register changes from 0 to 1. When the host driver writes this field to 1 to clear this status, the status of the Card Inserted in the Present State register should be confirmed. Because the card state might possibly be changed when the host driver clears this field and the interrupt event might not be generated. When this field is cleared, it is set again if a card is inserted. to leave it cleared, clear the Card Inserted Status Enable field in Interrupt Status Enable register. 0b - Card state unstable or removed 1b - Card inserted
5 BRR	Buffer read ready This status field is set if the Buffer Read Enable field, in the Present State register, changes from 0 to 1. See the Buffer Read Enable field in the Present State register for additional information. This field indicates that cmd19 is finished in tuning process. 0b - Not ready to read buffer 1b - Ready to read buffer
4 BWR	Buffer write ready This status field is set if the Buffer Write Enable field, in the Present State register, changes from 0 to 1. See the Buffer Write Enable field in the Present State register for additional information. 0b - Not ready to write buffer 1b - Ready to write buffer
3 DINT	DMA interrupt Occurs only when the internal DMA finishes the data transfer successfully. Whenever errors occur during data transfer, this field does not be set. Instead, the DMAE field is set. Either Simple DMA or ADMA finishes data transferring, this field is set. 0b - No DMA interrupt 1b - DMA interrupt is generated.
2 BGE	Block gap event If the Stop At Block Gap Request field in the Protocol Control register is set, this field is set when a read or write transaction is stopped at a block gap. If Stop At Block Gap Request is not set to 1, this field is not set to 1. In the case of a Read Transaction: This field is set at the falling edge of the DATA Line Active Status (When the transaction is stopped at SD bus timing). The Read Wait must be supported to use this function. In the case of Write Transaction: This field is set at the falling edge of Write Transfer Active Status (After getting CRC status at SD Bus timing). 0b - No block gap event 1b - Transaction stopped at block gap

Table continues on the next page...

Field	Function
1 TC	Transfer complete This field is set when a read or write transfer is completed. In the case of a Read Transaction: This field is set at the falling edge of the Read Transfer Active Status. There are two cases in which this interrupt is generated. The first is when a data transfer is completed as specified by the data length (after the last data has been read to the host system). The second is when data has stopped at the block gap and completed the data transfer by setting the Stop At Block Gap Request field in the Protocol Control register (after valid data has been read to the host system). In the case of a Write Transaction: This field is set at the falling edge of the DATA Line Active Status. There are two cases in which this interrupt is generated. The first is when the last data is written to the SD card as specified by the data length and the busy signal is released. The second is when data transfers are stopped at the block gap, by setting the Stop At Block Gap Request field in the Protocol Control register, and the data transfers are completed. (after valid data is written to the SD card and the busy signal released). In the case of a command with busy, this field is set when busy is deasserted. This field is not asserted in tuning process. 0b - Transfer does not complete 1b - Transfer complete
0 CC	Command complete This field is set when you receive the end field of the command response (except auto CMD12). See the Command Inhibit (CMD) in the Present State register. This field is not asserted in tuning process. 0b - Command not complete 1b - Command complete

10.3.7.1.15 Interrupt Status Enable (INT_STATUS_EN)

10.3.7.1.15.1 Offset

Register	Offset
INT_STATUS_EN	34h

10.3.7.1.15.2 Function

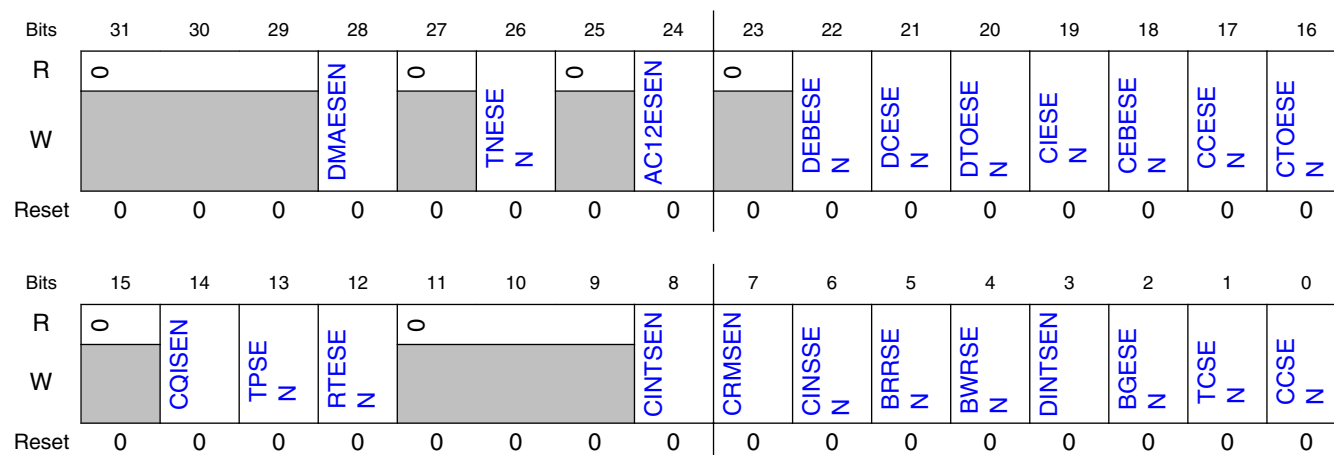
Setting the fields in this register to 1 enables the corresponding Interrupt Status to be set by the specified event. If any field is cleared, the corresponding Interrupt Status field is also cleared (that is, when the field in this register is cleared, the corresponding field in Interrupt Status register is always 0).

- Depending on IABG field setting, uSDHC might be programmed to sample the card interrupt signal during the interrupt period and hold its value in the flip-flop. There

are some delays on the Card Interrupt, asserted from the card, to the time the host system is informed.

- To detect a CMD line conflict, the host driver must set both Command Timeout Error Status Enable and Command CRC Error Status Enable to 1.

10.3.7.1.15.3 Diagram



10.3.7.1.15.4 Fields

Field	Function
31-29 —	Reserved
28 DMAESEN	DMA error status enable 0b - Masked 1b - Enabled
27 —	Reserved
26 TNESEN	Tuning error status enable 0b - Masked 1b - Enabled
25 —	Reserved
24 AC12ESEN	Auto CMD12 error status enable 0b - Masked 1b - Enabled
23 —	Reserved
22 DEBESEN	Data end bit error status enable 0b - Masked 1b - Enabled

Table continues on the next page...

Field	Function
21 DCESEN	Data CRC error status enable 0b - Masked 1b - Enabled
20 DTESEN	Data timeout error status enable 0b - Masked 1b - Enabled
19 CIESEN	Command index error status enable 0b - Masked 1b - Enabled
18 CEBESEN	Command end bit error status enable 0b - Masked 1b - Enabled
17 CCESEN	Command CRC error status enable 0b - Masked 1b - Enabled
16 CTESEN	Command timeout error status enable 0b - Masked 1b - Enabled
15 —	Reserved
14 CQISEN	Command queuing status enable 0b - Masked 1b - Enabled
13 TPSEN	Tuning pass status enable 0b - Masked 1b - Enabled
12 RTESEN	Re-tuning event status enable 0b - Masked 1b - Enabled
11-9 —	Reserved
8 CINTSEN	Card interrupt status enable If this field is set to 0, uSDHC clears the interrupt request to the system. The Card Interrupt detection is stopped when this field is cleared and restarted when this field is set to 1. The host driver should clear the Card Interrupt Status Enable before servicing the Card Interrupt and should set this field again after all interrupt requests from the card are cleared to prevent inadvertent interrupts. 0b - Masked 1b - Enabled
7 CRMSSEN	Card removal status enable 0b - Masked 1b - Enabled
6 CINSSSEN	Card insertion status enable 0b - Masked 1b - Enabled
5 BRRSEN	Buffer read ready status enable 0b - Masked 1b - Enabled
4	Buffer write ready status enable 0b - Masked

Table continues on the next page...

Field	Function
BWRSEN	1b - Enabled
3 DINTSEN	DMA interrupt status enable 0b - Masked 1b - Enabled
2 BGESEN	Block gap event status enable 0b - Masked 1b - Enabled
1 TCSEN	Transfer complete status enable 0b - Masked 1b - Enabled
0 CCSEN	Command complete status enable 0b - Masked 1b - Enabled

10.3.7.1.16 Interrupt Signal Enable (INT_SIGNAL_EN)

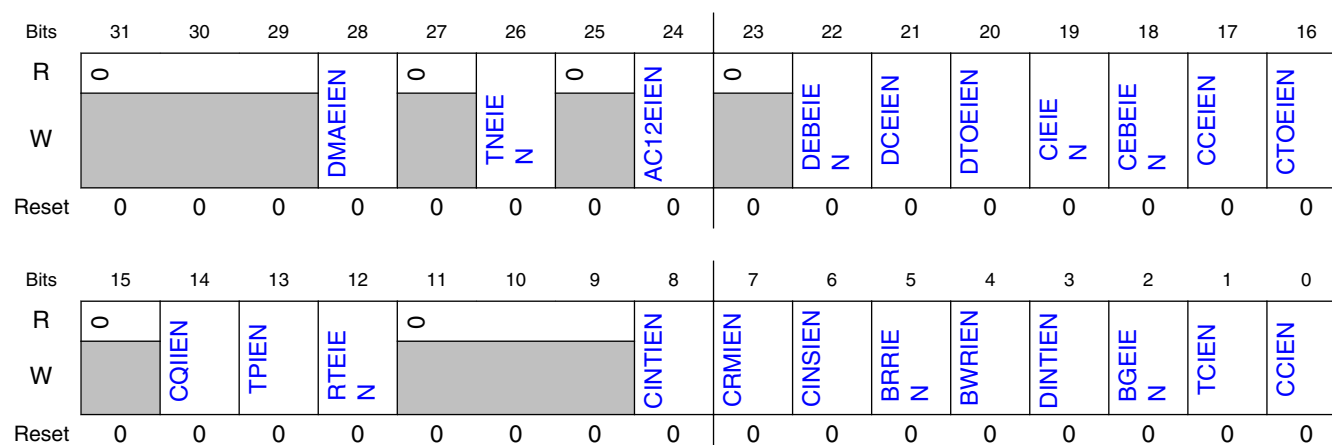
10.3.7.1.16.1 Offset

Register	Offset
INT_SIGNAL_EN	38h

10.3.7.1.16.2 Function

This register is used to select which interrupt status is indicated to the host system as the interrupt. These status fields all share the same interrupt line. Setting any of these fields to 1 enables interrupt generation. The corresponding Status register field generates an interrupt when the corresponding interrupt signal enable field is set.

10.3.7.1.16.3 Diagram



10.3.7.1.16.4 Fields

Field	Function
31-29 —	Reserved
28 DMAEIEN	DMA error interrupt enable 0b - Masked 1b - Enable
27 —	Reserved
26 TNEIEN	Tuning error interrupt enable 0b - Masked 1b - Enabled
25 —	Reserved
24 AC12EIEN	Auto CMD12 error interrupt enable 0b - Masked 1b - Enabled
23 —	Reserved
22 DEBEIEN	Data end bit error interrupt enable 0b - Masked 1b - Enabled
21 DCEIEN	Data CRC error interrupt enable 0b - Masked 1b - Enabled
20 DTOEIEN	Data timeout error interrupt enable 0b - Masked 1b - Enabled
19 CIEIEN	Command index error interrupt enable 0b - Masked 1b - Enabled
18 CEBEIEN	Command end bit error interrupt enable 0b - Masked 1b - Enabled
17 CCEIEN	Command CRC error interrupt enable 0b - Masked 1b - Enabled
16 CTOEIEN	Command timeout error interrupt enable 0b - Masked 1b - Enabled
15 —	Reserved
14 CQIEN	Command queuing signal enable 0b - Masked 1b - Enabled

Table continues on the next page...

Field	Function
13 TPIEN	Tuning pass interrupt enable 0b - Masked 1b - Enabled
12 RTEIEN	Re-tuning event interrupt enable 0b - Masked 1b - Enabled
11-9 —	Reserved
8 CINTIEN	Card interrupt enable 0b - Masked 1b - Enabled
7 CRMIEN	Card removal interrupt enable 0b - Masked 1b - Enabled
6 CINSIEN	Card insertion interrupt enable 0b - Masked 1b - Enabled
5 BRRIEN	Buffer read ready interrupt enable 0b - Masked 1b - Enabled
4 BWRIEN	Buffer write ready interrupt enable 0b - Masked 1b - Enabled
3 DINTIEN	DMA interrupt enable 0b - Masked 1b - Enabled
2 BGEIEN	Block gap event interrupt enable 0b - Masked 1b - Enabled
1 TCIEN	Transfer complete interrupt enable 0b - Masked 1b - Enabled
0 CCIEN	Command complete interrupt enable 0b - Masked 1b - Enabled

10.3.7.1.17 Auto CMD12 Error Status (AUTOCMD12_ERR_STATUS)

10.3.7.1.17.1 Offset

Register	Offset
AUTOCMD12_ERR_ST ATUS	3Ch

10.3.7.1.17.2 Function

When the Auto CMD12 Error Status field in the Status register is set, the host driver checks this register to identify what kind of error the Auto CMD12 / CMD 23 indicated. Auto CMD23 errors are indicated in field 04-01. This register is valid only when the Auto CMD12 Error status field is set.

The table below shows the relationship between the Auto CMGD12 CRC Error and the Auto CMD12 Command Timeout Error.

Table 10-47. Relationship between command CRC error and command timeout error for auto CMD12

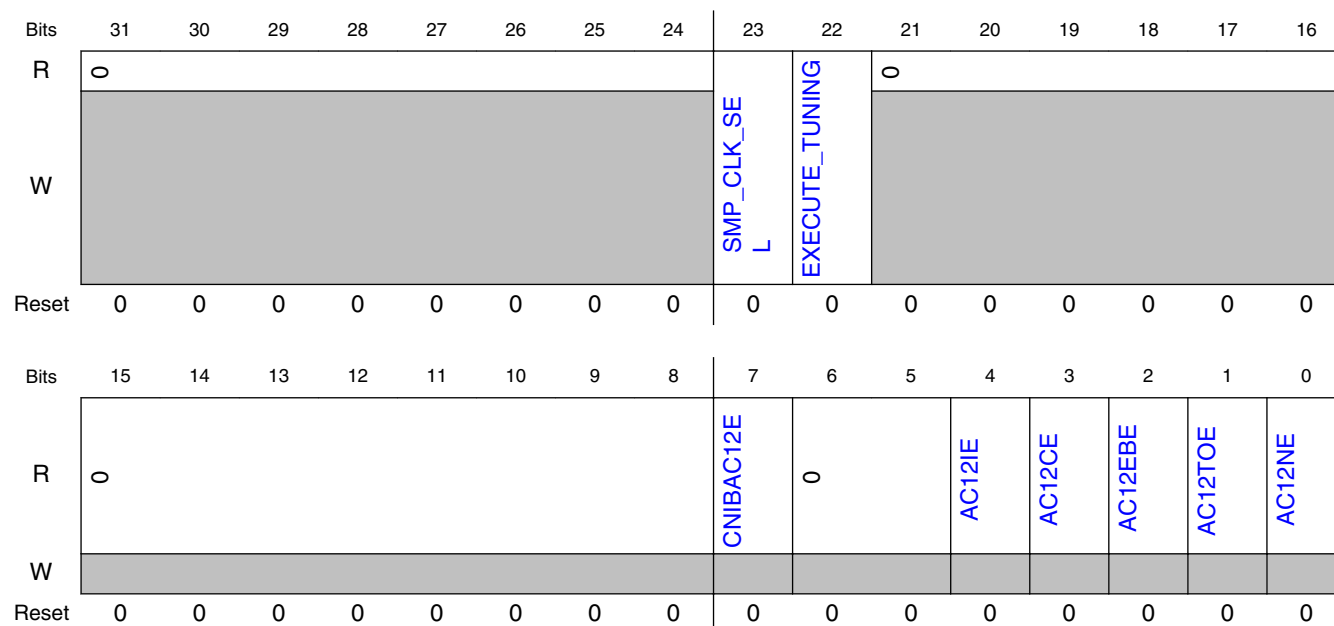
Auto CMD12 CRC error	Auto CMD12 timeout error	Type of error
0	0	No error
0	1	Response timeout error
1	0	Response CRC error
1	1	CMD line conflict

Changes in Auto CMD12 Error Status register can be classified in three scenarios:

- When uSDHC is going to issue an Auto CMD12
 - Set field 0 to 1 if the Auto CMD12 cannot be issued due to an error in the previous command
 - Set field 0 to 0 if the Auto CMD12 is issued
- At the end field of an Auto CMD12 response
 - Check errors correspond to fields 1-4
 - Set fields 1-4 corresponding to detected errors
 - Clear fields 1-4 corresponding to detected errors
- Before reading the Auto CMD12 Error Status field 7
 - Set field 7 to 1 if there is a command that can't be issued
 - Clear field 7 if there is no command to issue

The timing for generating the Auto CMD12 Error and writing to the Command register are asynchronous. After that, field 7 is sampled when the driver is not writing to the Command register. So, it is suggested to read this register only when the AC12E field in Interrupt Status register is set. An Auto CMD12 Error Interrupt is generated when one of the error fields (0-4) is set to 1. The Command Not Issued By Auto CMD12 Error does not generate an interrupt.

10.3.7.1.17.3 Diagram



10.3.7.1.17.4 Fields

Field	Function
31-24 —	Reserved
23 SMP_CLK_SEL	Sample clock select When std_tuning_en field is set, this field is used to select sampling clock to receive CMD and DATA. Otherwise, this field is reserved. This field is set by ty tuning procedure and valid after the completion of tuning(When Execute Tuning is cleared). Setting 1 means that tuning is completed successfully and setting 0 means that tuning is failed. Writing 1 to this field is meaningless and ignored. A tuning circuit is reset by writing to 0. This field can be cleared with setting Execute Tuning. Once the tuning circuit is reset, it takes time to complete tuning sequence. Therefore, host driver should keep this field to 1 to perform re-tuning sequence to complete re-tuning sequence in a short time. Change of this field is not allowed while the Host controller us receiving response or a read data block. 0b - Fixed clock is used to sample data 1b - Tuned clock is used to sample data
22 EXECUTE_TUNING	Execute tuning When std_tuning_en field is set, this field is used to start tuning procedure. Otherwise, this field is reserved. This field is set to start tuning procedure and automatically cleared when runing procedure is completed. The result of tuning is indicated to sam_clk_sel field. Tuning procedure is aborted by writing 0.
21-8 —	Reserved
7 CNIBAC12E	Command not issued by Auto CMD12 error Setting this field to 1 means CMD_wo_DAT is not executed due to an Auto CMD12 error (D04-D01) in this register.

Table continues on the next page...

Field	Function
	0b - No error 1b - Not issued
6-5 —	Reserved
4 AC12IE	Auto CMD12 / 23 index error Occurs if the command index error occurs in response to a command. 0b - No error 1b - Error, the CMD index in response is not CMD12/23
3 AC12CE	Auto CMD12 / 23 CRC error Occurs when detecting a CRC error in the command response. 0b - No CRC error 1b - CRC error met in Auto CMD12/23 response
2 AC12EBE	Auto CMD12 / 23 end bit error Occurs when detecting that the end field of command response is 0 which should be 1. 0b - No error 1b - End bit error generated
1 AC12TOE	Auto CMD12 / 23 timeout error Occurs if no response is returned within 64 SDCLK cycles from the end field of the command. If this field is set to 1, the other error status fields (2-4) have no meaning. 0b - No error 1b - Time out
0 AC12NE	Auto CMD12 not executed If memory multiple block data transfer is not started, due to a command error, this field is not set because it is not necessary to issue an Auto CMD12. Setting this field to 1 means uSDHC cannot issue the Auto CMD12 to stop a memory multiple block data transfer due to some error. If this field is set to 1, other error status fields (1-4) have no meaning. 0b - Executed 1b - Not executed

10.3.7.1.18 Host Controller Capabilities (HOST_CTRL_CAP)

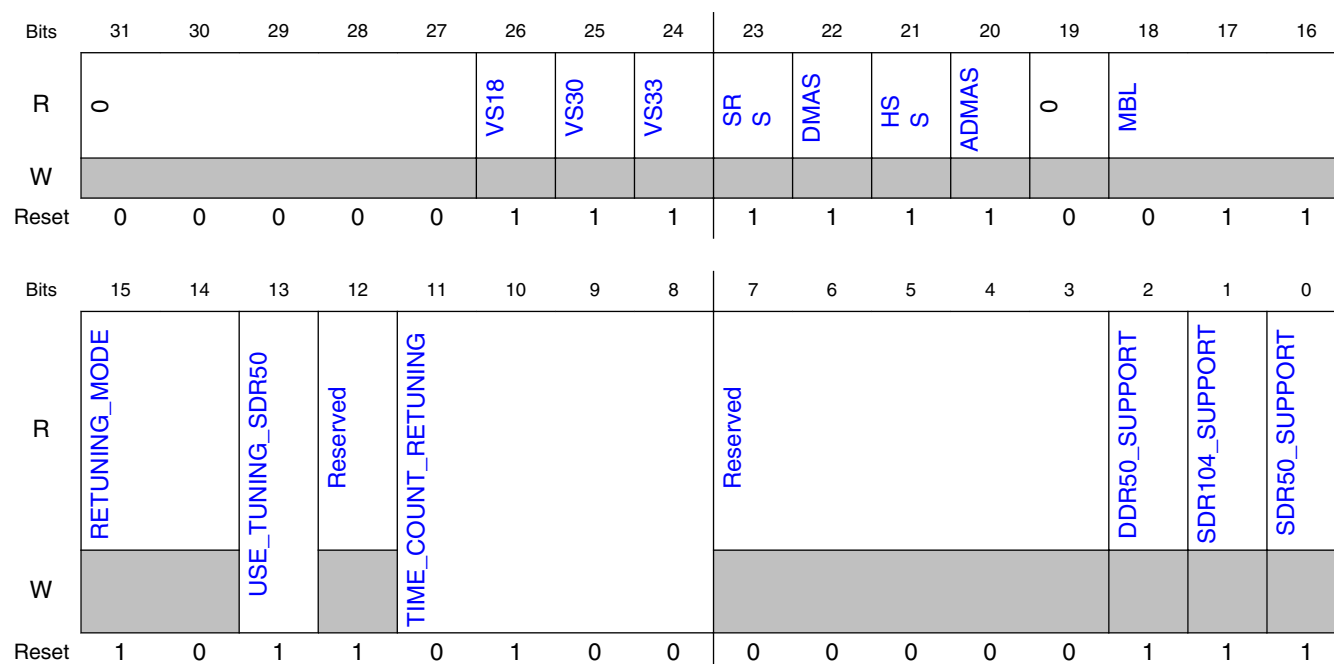
10.3.7.1.18.1 Offset

Register	Offset
HOST_CTRL_CAP	40h

10.3.7.1.18.2 Function

This register provides the host driver with information specific to uSDHC implementation. The value in this register is the power-on-reset value and does not change with a software reset.

10.3.7.1.18.3 Diagram



10.3.7.1.18.4 Fields

Field	Function
31-27 —	Reserved
26 VS18	Voltage support 1.8 V This field depends on the host system ability. 0b - 1.8 V not supported 1b - 1.8 V supported
25 VS30	Voltage support 3.0 V This field depends on the host system ability. 0b - 3.0 V not supported 1b - 3.0 V supported
24 VS33	Voltage support 3.3 V This field depends on the host system ability. 0b - 3.3 V not supported 1b - 3.3 V supported
23 SRS	Suspend / resume support This field indicates whether uSDHC supports Suspend / Resume functionality. If this field is 0, the Suspend and Resume mechanism, as well as the read wait, are not supported, and the host driver does not issue either Suspend or Resume commands. 0b - Not supported

Table continues on the next page...

Ultra Secured Digital Host Controller (uSDHC)

Field	Function
	1b - Supported
22 DMAS	DMA support This field indicates whether uSDHC is capable of using the internal DMA to transfer data between system memory and the data buffer directly. 0b - DMA not supported 1b - DMA supported
21 HSS	High speed support This field indicates whether uSDHC supports High Speed mode and the host system can supply a SD clock frequency from 25 MHz to 50 MHz. 0b - High speed not supported 1b - High speed supported
20 ADMAS	ADMA support This field indicates whether uSDHC supports the ADMA feature. 0b - Advanced DMA not supported 1b - Advanced DMA supported
19 —	Reserved
18-16 MBL	Max block length This value indicates the maximum block size that the host driver can read and write to the buffer in uSDHC. The buffer transfers block size without wait cycles. 000b - 512 bytes 001b - 1024 bytes 010b - 2048 bytes 011b - 4096 bytes
15-14 RETUNING_MODE	Retuning Mode This field selects retuning method. 00b - Mode 1 01b - Mode 2 10b - Mode 3 11b - Reserved
13 USE_TUNING_SDR50	Use Tuning for SDR50 This field is set to 1. Host controller requires tuning to operate SDR50. 0b - SDR does not require tuning. 1b - SDR50 requires tuning.
12 —	Reserved
11-8 TIME_COUNT_RETUNING	Time counter for retuning This field indicates an initial value of the Retuning Timer for Re-Tuning Mode1 and 3. Setting to 0 disables Retuning Timer.
7-3 —	Reserved
2 DDR50_SUPPORT	DDR50 support This field indicates support of DDR50 mode.

Table continues on the next page...

Field	Function
1 SDR104_SUPPORT	SDR104 support This field indicates support of SDR104 mode.
0 SDR50_SUPPORT	SDR50 support This field indicates support of SDR50 mode.

10.3.7.1.19 Watermark Level (WTMK_LVL)

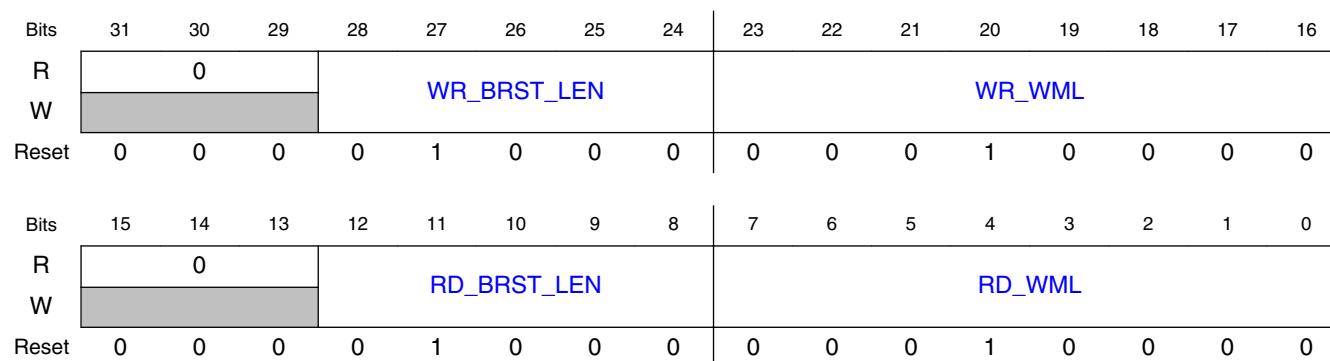
10.3.7.1.19.1 Offset

Register	Offset
WTMK_LVL	44h

10.3.7.1.19.2 Function

Both write and read watermark levels (FIFO threshold) are configurable. Their value can range from 1 to 128 words. Both write and read burst lengths are also Configurable. Their value can range from 1 to 31 words.

10.3.7.1.19.3 Diagram



10.3.7.1.19.4 Fields

Field	Function
31-29 —	Reserved

Table continues on the next page...

Field	Function
28-24 WR_BRST_LEN	Write burst length due to system restriction, the actual burst length might not exceed 16 The number of words uSDHC writes in a single burst. The write burst length must be less than or equal to the write watermark level, and all bursts within a watermark level transfer is in back-to-back mode. On reset, this field is 8. Writing 0 to this field results in '01000' (that is, it is not able to clear this field).
23-16 WR_WML	Write watermark level The number of words used as the watermark level (FIFO threshold) in a DMA write operation. Also, the number of words as a sequence of write bursts in back-to-back mode. The maximum legal value for the write watermark level is 128.
15-13 —	Reserved
12-8 RD_BRST_LEN	Read burst length due to system restriction, the actual burst length might not exceed 16 The number of words uSDHC reads in a single burst. The read burst length must be less than or equal to the read watermark level, and all bursts within a watermark level transfer is in back-to-back mode. On reset, this field is 8. Writing 0 to this field results in '01000' (that is, it is not able to clear this field).
7-0 RD_WML	Read watermark level The number of words used as the watermark level (FIFO threshold) in a DMA read operation. Also, the number of words as a sequence of read bursts in back-to-back mode. The maximum legal value for the read water mark level is 128.

10.3.7.1.20 Mixer Control (MIX_CTRL)

10.3.7.1.20.1 Offset

Register	Offset
MIX_CTRL	48h

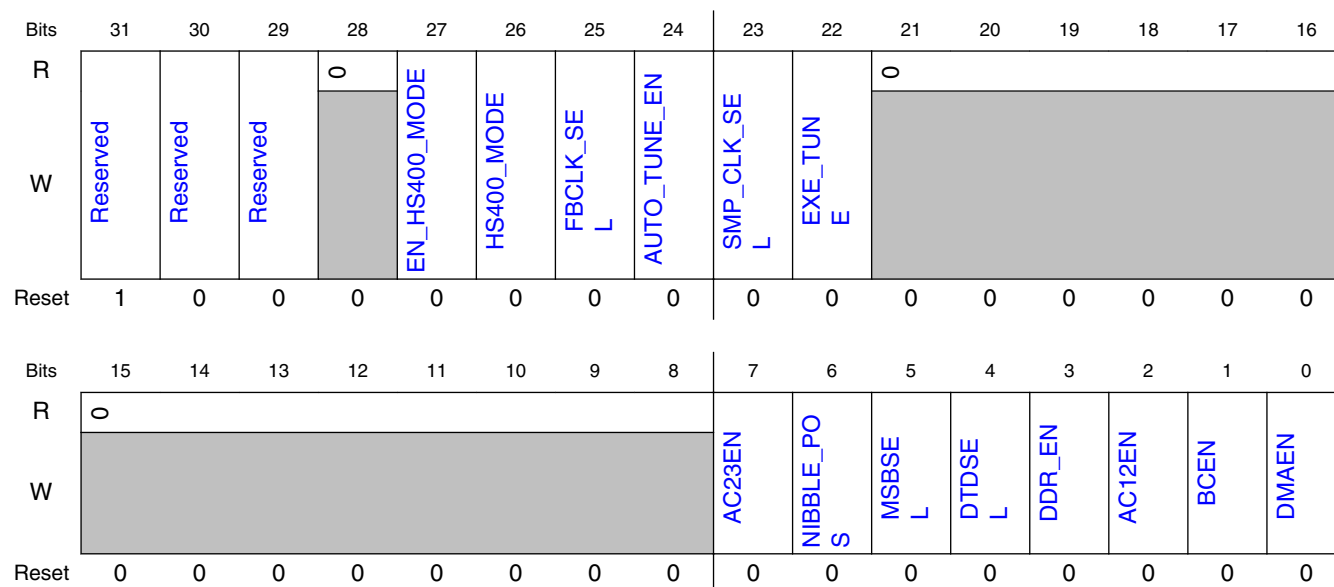
10.3.7.1.20.2 Function

This register is used to DMA and data transfer. To prevent data loss, The software should check if data transfer is active before writing this register. These fields are DPSEL, MBSEL, DTDSEL, AC12EN, BCEN, and DMAEN.

Table 10-48. Transfer type register setting for various transfer types

Multi/single block select	Block count enable	Block count	Function
0	Do not care	Do not care	Single transfer
1	0	Do not care	Infinite transfer
1	1	Positive number	Multiple transfer
1	1	Zero	No data transfer

10.3.7.1.20.3 Diagram



10.3.7.1.20.4 Fields

Field	Function
31 —	Reserved Always write as 1.
30 —	Reserved Always write as 0.
29 —	Reserved Always write as 0.
28 —	Reserved
27 EN_HS400_MODE	Enable enhance HS400 mode Enhance HS400 enable
26 HS400_MODE	Enable HS400 mode HS400 Enable
25 FBCLK_SEL	Feedback clock source selection (Only used for SD3.0, SDR104 mode and EMMC HS200 mode) 0b - Feedback clock comes from the loopback CLK 1b - Feedback clock comes from the ipp_card_clk_out
24 AUTO_TUNE_EN	Auto tuning enable (Only used for SD3.0, SDR104 mode and EMMC HS200 mode) 0b - Disable auto tuning 1b - Enable auto tuning
23 SMP_CLK_SEL	Clock selection When STD_TUNING_EN is 0, this field is used to select Tuned clock or Fixed clock to sample data / cmd (Only used for SD3.0, SDR104 mode and EMMC HS200 mode)

Table continues on the next page...

Ultra Secured Digital Host Controller (uSDHC)

Field	Function
	0b - Fixed clock is used to sample data / cmd 1b - Tuned clock is used to sample data / cmd
22 EXE_TUNE	Execute tuning: (Only used for SD3.0, SDR104 mode and EMMC HS200 mode) When STD_TUNING_EN is 0, this field is set to 1 to indicate the host driver is starting tuning procedure. Tuning procedure is aborted by writing 0. 0b - Not tuned or tuning completed 1b - Execute tuning
21-8 —	Reserved
7 AC23EN	Auto CMD23 enable When this field is set to 1, the host controller issues a CMD23 automatically before issuing a command specified in the Command Register.
6 NIBBLE_POS	Nibble position indication In DDR 4-bit mode nibble position indication. 0- the sequence is 'odd high nibble -> even high nibble -> odd low nibble -> even low nibble'; 1- the sequence is 'odd high nibble -> odd low nibble -> even high nibble -> even low nibble'.
5 MSBSEL	Multi / Single block select This field enables multiple block DATA line data transfers. For any other commands, this field can be set to 0. If this field is 0, it is not necessary to set the Block Count register. See Command Transfer Type (CMD_XFR_TYP) . 0b - Single block 1b - Multiple blocks
4 DTDSEL	Data transfer direction select This field defines the direction of DATA line data transfers. The field is set to 1 by the host driver to transfer data from the SD card to uSDHC and is set to 0 for all other commands. 0b - Write (Host to card) 1b - Read (Card to host)
3 DDR_EN	Dual data rate mode selection
2 AC12EN	Auto CMD12 enable Multiple block transfers for memory require a CMD12 to stop the transaction. When this field is set to 1, uSDHC issues a CMD12 automatically when the last block transfer has completed. The host driver is not set this field to issue commands that do not require CMD12 to stop a multiple block data transfer. In particular, secure commands defined in File Security Specification (see reference list) do not require CMD12. In single block transfer, uSDHC ignores this field no matter it is set or not. 0b - Disable 1b - Enable
1 BCEN	Block count enable This field is used to enable the Block Count register, which is only relevant for multiple block transfers. When this field is 0, the internal counter for block is disabled, which is useful in executing an infinite transfer. 0b - Disable 1b - Enable
0 DMAEN	DMA enable

Field	Function
	This field enables DMA functionality. If this field is set to 1, a DMA operation begins when the host driver sets the DPSEL field of this register. Whether the simple DMA or the advanced DMA is active depends on the DMA Select field of the Protocol Control register. 0b - Disable 1b - Enable

10.3.7.1.21 Force Event (FORCE_EVENT)

10.3.7.1.21.1 Offset

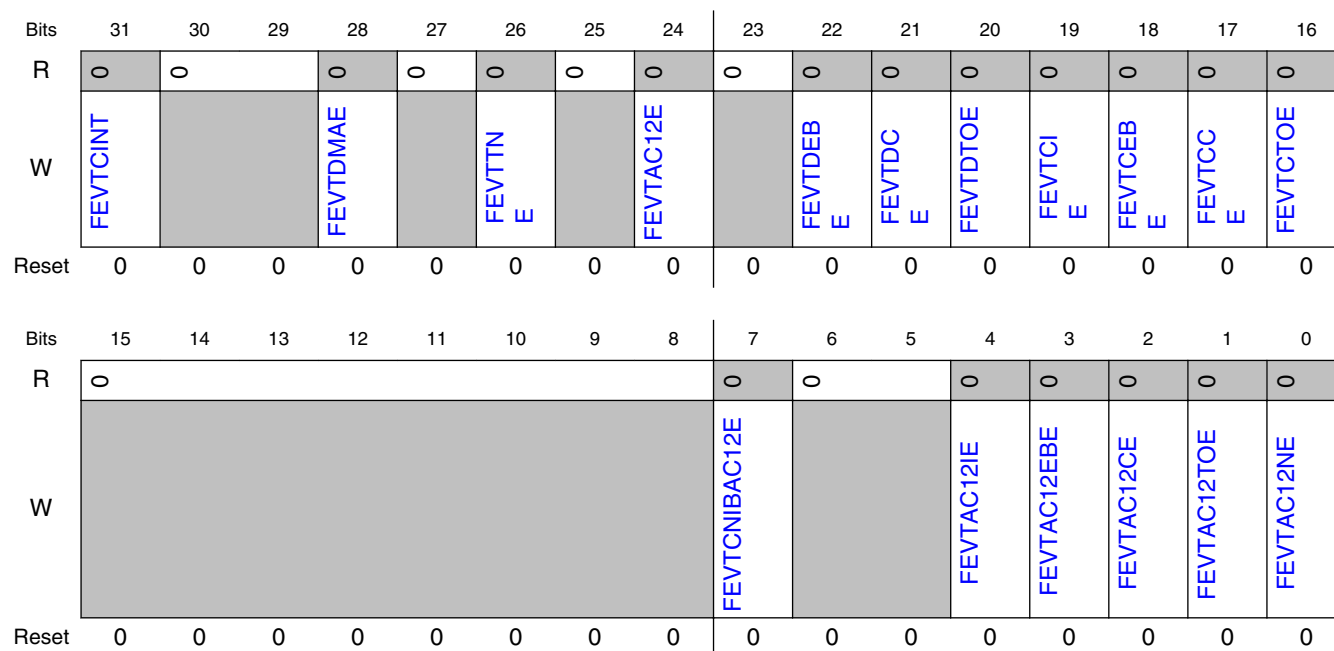
Register	Offset
FORCE_EVENT	50h

10.3.7.1.21.2 Function

The Force Event Register is not a physically implemented register. Rather, it is an address at which the Interrupt Status register can be written if the corresponding field of the Interrupt Status Enable Register is set. This register is a write only register and writing 0 to it has no effect. Writing 1 to this register actually sets the corresponding field of Interrupt Status register. A read from this register always results in 0's. to change the corresponding status fields in the Interrupt Status register, make sure to set IPGEN field in System Control Register so that peripheral clock is always active.

Forcing a card interrupt generates a short pulse on the DATA1 line, and the driver might treat this interrupt as a normal interrupt. The interrupt service routine might skip polling the card interrupt factor as the interrupt is self cleared.

10.3.7.1.21.3 Diagram



10.3.7.1.21.4 Fields

Field	Function
31 FEVTCINT	Force event card interrupt Writing 1 to this field generates a short low-level pulse on the internal DATA1 line, as if a self clearing interrupt was received from the external card. If enabled, the CINT field is set and the interrupt service routine might treat this interrupt as a normal interrupt from the external card.
30-29 —	Reserved
28 FEVTDMAE	Force event DMA error Forces the DMAE field of Interrupt Status register to be set.
27 —	Reserved
26 FEVTTNE	Force tuning error Forces the TNE field of Interrupt Status register to be set.
25 —	Reserved
24 FEVTAC12E	Force event Auto Command 12 error Forces the AC12E field of Interrupt Status register to be set.
23 —	Reserved
22	Force event data end bit error

Table continues on the next page...

Field	Function
FEVTDEBE	Forces the DEBE field of Interrupt Status register to be set.
21 FEVTDCE	Force event data CRC error Forces the DCE field of Interrupt Status register to be set.
20 FEVTDTOE	Force event data time out error Force the DTOE field of Interrupt Status register to be set.
19 FEVTCIE	Force event command index error Forces the CCE field of Interrupt Status register to be set.
18 FEVTCEBE	Force event command end bit error Forces the CEBE field of Interrupt Status register to be set.
17 FEVTCCE	Force event command CRC error Forces the CCE field of Interrupt Status register to be set.
16 FEVTCTOE	Force event command time out error Forces the CTOE field of Interrupt Status register to be set.
15-8 —	Reserved
7 FEVTCNIBAC12E	Force event command not executed by Auto Command 12 error Forces the CNIBAC12E field in the Auto Command12 Error Status register to be set.
6-5 —	Reserved
4 FEVTAC12IE	Force event Auto Command 12 index error Forces the AC12IE field in the Auto Command12 Error Status register to be set.
3 FEVTAC12EBE	Force event Auto Command 12 end bit error Forces the AC12EBE field in the Auto Command12 Error Status register to be set.
2 FEVTAC12CE	Force event auto command 12 CRC error Forces the AC12CE field in the Auto Command12 Error Status register to be set.
1 FEVTAC12TOE	Force event auto command 12 time out error Forces the AC12TOE field in the Auto Command12 Error Status register to be set.
0 FEVTAC12NE	Force event auto command 12 not executed Forces the AC12NE field in the Auto Command12 Error Status register to be set.

10.3.7.1.22 ADMA Error Status (ADMA_ERR_STATUS)

10.3.7.1.22.1 Offset

Register	Offset
ADMA_ERR_STATUS	54h

10.3.7.1.22.2 Function

When an ADMA Error Interrupt has occurred, the ADMA Error States field in this register holds the ADMA state and the ADMA System Address register holds the address around the error descriptor.

For recovering from this error, the host driver requires the ADMA state to identify the error descriptor address as follows:

- **ST_STOP:** Previous location set in the ADMA System Address register is the error descriptor address.
- **ST_FDS:** Current location set in the ADMA System Address register is the error descriptor address.
- **ST_CADR:** This state is never set because it only increments the descriptor pointer and does not generate an ADMA error.
- **ST_TFR:** Previous location set in the ADMA System Address register is the error descriptor address.

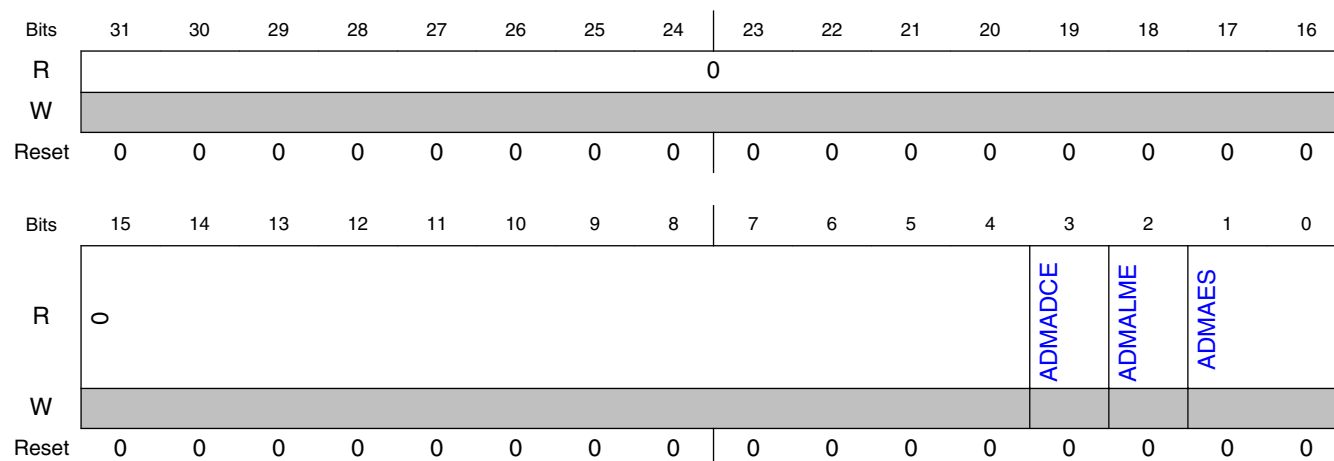
In case of a write operation, the host driver should use the ACMD22 to get the number of the written block, rather than using this information, because unwritten data might exist in the host controller.

The host controller generates the ADMA Error Interrupt when it detects invalid descriptor data (Valid=0) in the ST_FDS state. The host driver can distinguish this error by reading the Valid field of the error descriptor.

Table 10-49. ADMA error state coding

D01-D00	ADMA error state (when error has occurred)	Contents of ADMA System Address register
00	ST_STOP (Stop DMA)	Holds the address of the next executable descriptor command
01	ST_FDS (Fetch descriptor)	Holds the valid descriptor address
10	ST_CADR (Change address)	No ADMA error is generated
11	ST_TFR (Transfer data)	Holds the address of the next executable descriptor command

10.3.7.1.22.3 Diagram



10.3.7.1.22.4 Fields

Field	Function
31-4 —	Reserved
3 ADMADCE	ADMA descriptor error This error occurs when invalid descriptor fetched by ADMA. 0b - No error 1b - Error
2 ADMALME	ADMA length mismatch error This error occurs in the following two cases: - While the block count enable is being set, the total data length specified by the descriptor table is different from that specified by the block count and block length. - Total data length cannot be divided by the block length. 0b - No error 1b - Error
1-0 ADMAES	ADMA error state (when ADMA error is occurred) This field indicates the state of the ADMA when an error has occurred during an ADMA data transfer. See ADMA Error Status (ADMA_ERR_STATUS) for more details.

10.3.7.1.23 ADMA System Address (ADMA_SYS_ADDR)

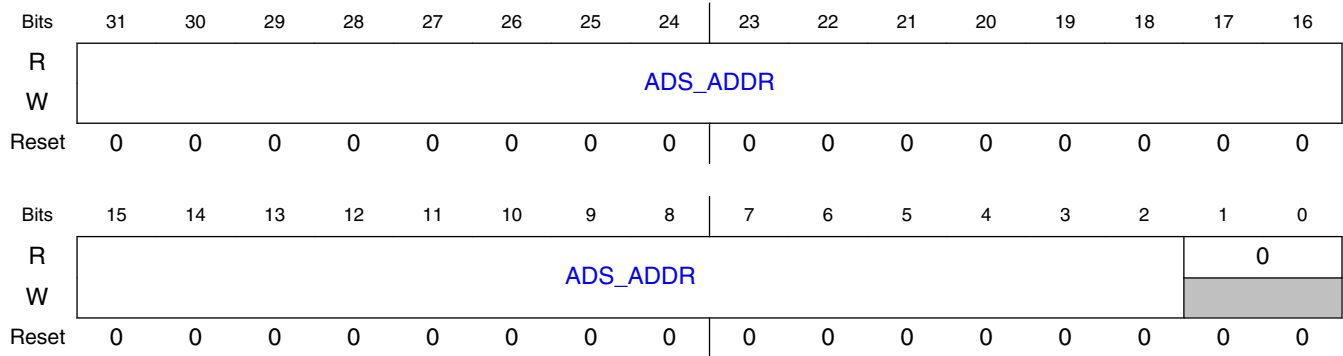
10.3.7.1.23.1 Offset

Register	Offset
ADMA_SYS_ADDR	58h

10.3.7.1.23.2 Function

This register contains the physical system memory address used for ADMA transfers.

10.3.7.1.23.3 Diagram



10.3.7.1.23.4 Fields

Field	Function
31-2 ADS_ADDR	ADMA system address This register holds the word address of the executing command in the Descriptor table. At the start of ADMA, the host driver sets the start address of the Descriptor table. The ADMA engine increments this register address whenever fetching a Descriptor command. When the ADMA is stopped at the Block Gap, this register indicates the address of the next executable Descriptor command. When the ADMA Error Interrupt is generated, this register holds the valid Descriptor address depending on the ADMA state. The lower 2 bits of this register is tied to '0' so the ADMA address is always word aligned. Because this register supports dynamic address reflecting, when TC field is set, it automatically alters the value of internal address counter, so the software cannot change this register when TC field is set. Such restriction is also listed in Software restrictions.
1-0 —	Reserved

10.3.7.1.24 DLL (Delay Line) Control (DLL_CTRL)

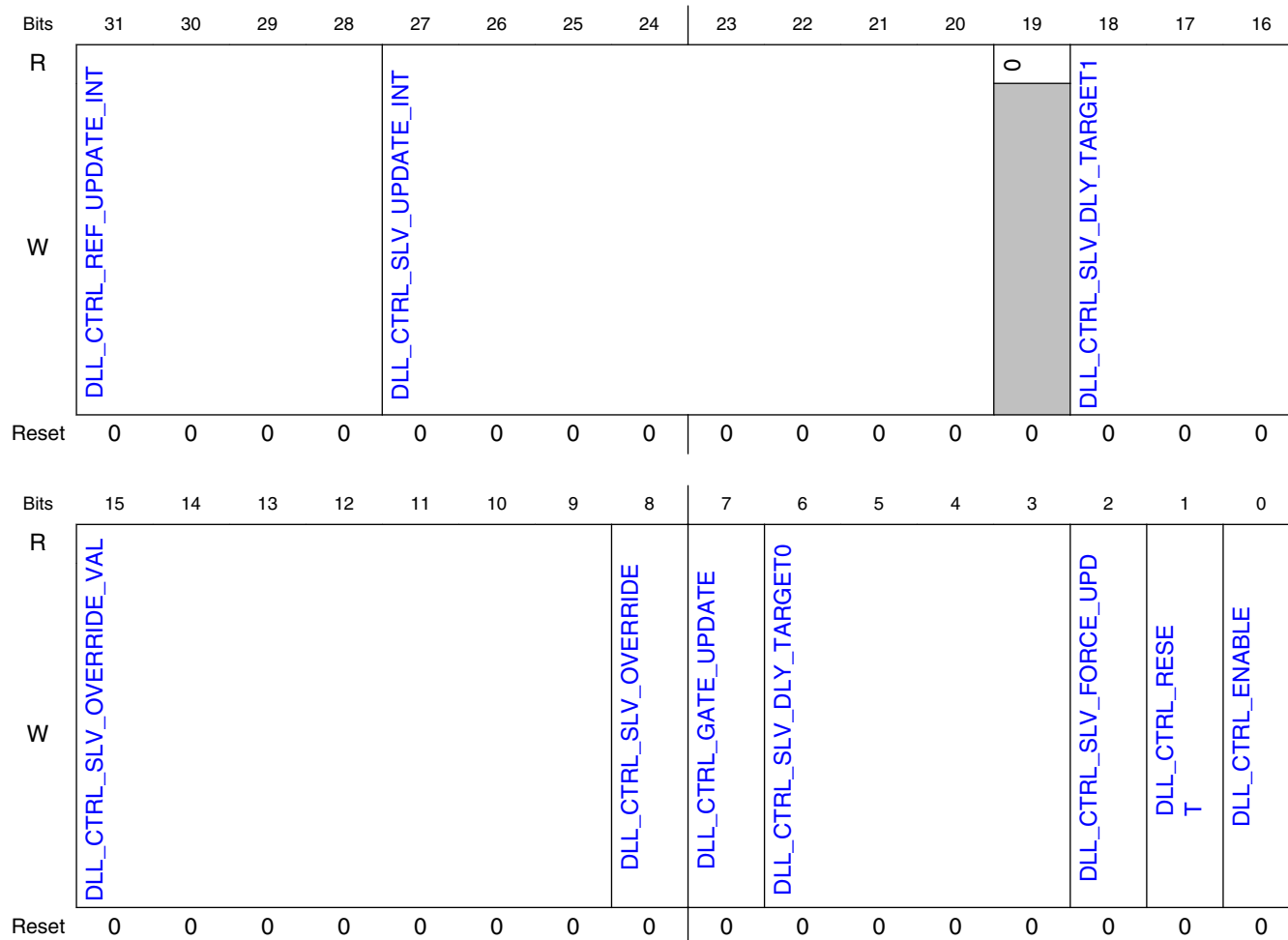
10.3.7.1.24.1 Offset

Register	Offset
DLL_CTRL	60h

10.3.7.1.24.2 Function

This register contains control fields for DLL.

10.3.7.1.24.3 Diagram



10.3.7.1.24.4 Fields

Field	Function
31-28 DLL_CTRL_REF_UPDATE_INT	DLL control loop update interval The interval cycle is $(2 + \text{REF_UPDATE_INT}) * \text{REF_CLOCK}$. By default, the DLL control loop updates every two REF_CLOCK cycles. It should be noted that increasing the reference delay-line update interval reduces the ability of the DLL to adjust to fast changes in conditions that might effect the delay (such as voltage and temperature)
27-20	Slave delay line update interval

Table continues on the next page...

Field	Function
DLL_CTRL_SLV_UPDATE_INT	If default 0 is used, it means 256 cycles of REF_CLOCK. A value of 0x0f results in 15 cycles and so on. Note that software can always cause an update of the slave-delay line using the SLV_FORCE_UPDATE register. Note that the slave delay line also updates automatically when the reference DLL transitions to a locked state (from an un-locked state).
19 —	Reserved
18-16 DLL_CTRL_SLV_DLY_TARGET1	DLL slave delay target1 See DLL_CTRL_SLV_DLY_TARGET0 below.
15-9 DLL_CTRL_SLV_OVERRIDE_VAL	DLL slave override val When SLV_OVERRIDE = 1, this field is used to select 1 of 128 physical taps manually. A value of 0 selects tap 1, and a value of 0x7f selects tap 128.
8 DLL_CTRL_SLV_OVERRIDE	DLL slave override Set this field to 1 to Enable manual override for slave delay chain using SLV_OVERRIDE_VAL; to set 0 to disable manual override. This feature does not require the DLL to be enabled using the ENABLE field. In fact to reduce power, if SLV_OVERRIDE is used, it is recommended to disable the DLL with ENABLE = 0
7 DLL_CTRL_GATE_UPDATE	DLL gate update Set this field to 1 to prevent the DLL from updating (because when clock_in exists, glitches might appear during DLL updates). This field might be used by software if such a condition occurs. Clear the bit to 0 to allow the DLL to update automatically.
6-3 DLL_CTRL_SLV_DLY_TARGET0	DLL slave delay target0 The delay target for uSDHC loopback read clock can be programmed in 1/16th increments of an ref_clock half-period. The delay is $((\{DLL_CTRL_SLV_DLY_TARGET1, DLL_CTRL_SLV_DLY_TARGET0\} + 1) * REF_CLOCK / 2) / 16$ So the input read-clock can be delayed relative input data from $(REF_CLOCK / 2) / 16$ to $REF_CLOCK * 4$.
2 DLL_CTRL_SLV_FORCE_UPD	DLL slave delay line Setting this field to 1, forces the slave delay line to update to the DLL calibrated value immediately. The slave delay line updates automatically based on the SLV_UPDATE_INT interval or when a DLL lock condition is sensed. Subsequent forcing of the slave-line update can only occur if SLV_FORCE_UP is set back to 0 and then asserted again (edge triggered). Be sure to use it when uSDHC is idle. This function might not work when uSDHC is working on data / cmd / response.
1 DLL_CTRL_RESET	DLL reset Setting this field to 1 force a reset on DLL. This causes the DLL to lose lock and re-calibrate to detect an REF_CLOCK half period phase shift. This signal is used by the DLL as edge-sensitive, so to create a subsequent reset, RESET must be taken low and then asserted again.
0 DLL_CTRL_ENABLE	DLL and delay chain Set this field to 1 to enable the DLL and delay chain; otherwise; set to 0 to bypasses DLL. Note that using the slave delay line override feature with SLV_OVERRIDE and SLV_OVERRIDE VAL, the DLL does not need to be enabled.

10.3.7.1.25 DLL Status (DLL_STATUS)

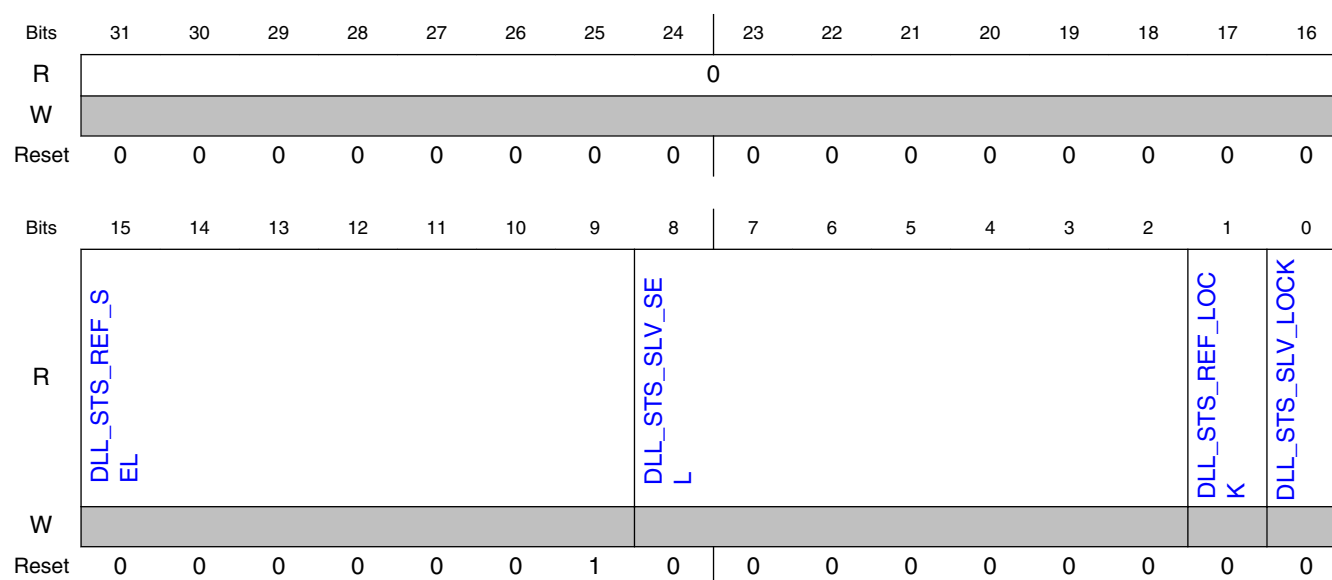
10.3.7.1.25.1 Offset

Register	Offset
DLL_STATUS	64h

10.3.7.1.25.2 Function

This register contains the DLL status information. All fields are read only and reads the same as the power-reset value.

10.3.7.1.25.3 Diagram



10.3.7.1.25.4 Fields

Field	Function
31-16 —	Reserved
15-9 DLL_STS_REF_SEL	Reference delay line select taps This is encoded by 7 fields for 127 taps.
8-2 DLL_STS_SLV_SEL	Slave delay line select status This is the instant value generated from reference chain. Because the reference chain can only be updated when REF_CLOCK is detected, this value should be the right value to be updated when the reference is locked.
1	Reference DLL lock status

Table continues on the next page...

Field	Function
DLL_STS_REF_LOCK	This signifies that the DLL has detected and locked to a half-phase ref_clock shift, allowing the slave delay-line to perform programmed clock delays
0	Slave delay-line lock status
DLL_STS_SLV_LOCK	This signifies that a valid calibration has been set to the slave-delay line and that the slave-delay line is implementing the programmed delay value

10.3.7.1.26 CLK Tuning Control and Status (CLK_TUNE_CTRL_STATUS)

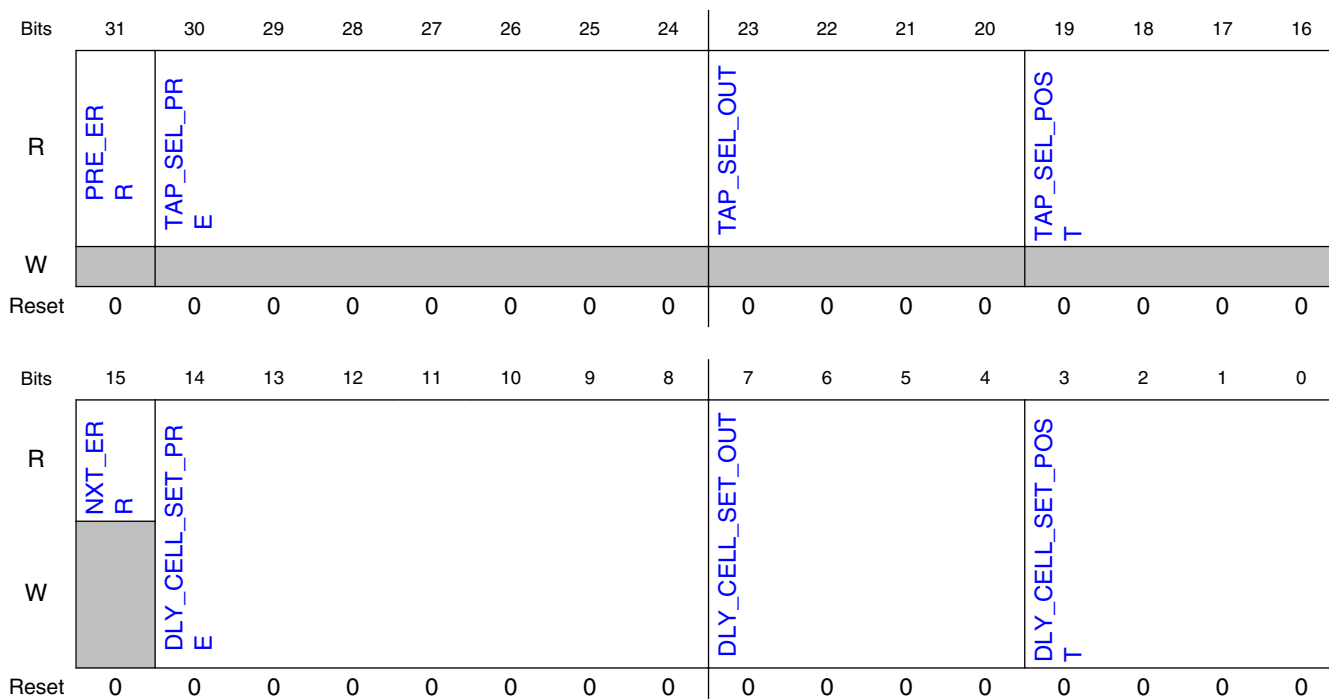
10.3.7.1.26.1 Offset

Register	Offset
CLK_TUNE_CTRL_STATUS	68h

10.3.7.1.26.2 Function

This register contains the Clock Tuning Control status information. All fields are read only and reads the same as the power-reset value. This register is added to support SD3.0 UHS-I SDR104 mode and EMMC HS200 mode.

10.3.7.1.26.3 Diagram



10.3.7.1.26.4 Fields

Field	Function
31 PRE_ERR	PRE error PRE error which means the number of delay cells added on the feedback clock is too small. It is valid only when SMP_CLK_SEL of Mix control register (bit23 of 0x48) is enabled.
30-24 TAP_SEL_PRE	TAP_SEL_PRE Reflects the number of delay cells added on the feedback clock between the feedback clock and CLK_PRE. When AUTO_TUNE_EN (bit24 of 0x48) is disabled, TAP_SEL_PRE is always equal to DLY_CELL_SET_PRE. When AUTO_TUNE_EN (bit24 of 0x48) is enabled, TAP_SEL_PRE is updated automatically according to the status of the auto tuning circuit to adjust the sample clock phase.
23-20 TAP_SEL_OUT	Delay cells added on the feedback clock between CLK_PRE and CLK_OUT Reflects the number of delay cells added on the feedback clock between CLK_PRE and CLK_OUT.
19-16 TAP_SEL_POST	Delay cells added on the feedback clock between CLK_OUT and CLK_POST Reflects the number of delay cells added on the feedback clock between CLK_OUT and CLK_POST.
15 NXT_ERR	NXT error NXT error which means the number of delay cells added on the feedback clock is too large. It's valid only when SMP_CLK_SEL of Mix control register (bit23 of 0x48) is enabled.
14-8 DLY_CELL_SET_PRE	delay cells on the feedback clock between the feedback clock and CLK_PRE Set the number of delay cells on the feedback clock between the feedback clock and CLK_PRE.
7-4 DLY_CELL_SET_OUT	Delay cells on the feedback clock between CLK_PRE and CLK_OUT Set the number of delay cells on the feedback clock between CLK_PRE and CLK_OUT.
3-0 DLY_CELL_SET_POST	Delay cells on the feedback clock between CLK_OUT and CLK_POST Set the number of delay cells on the feedback clock between CLK_OUT and CLK_POST.

10.3.7.1.27 Strobe DLL control (STROBE_DLL_CTRL)

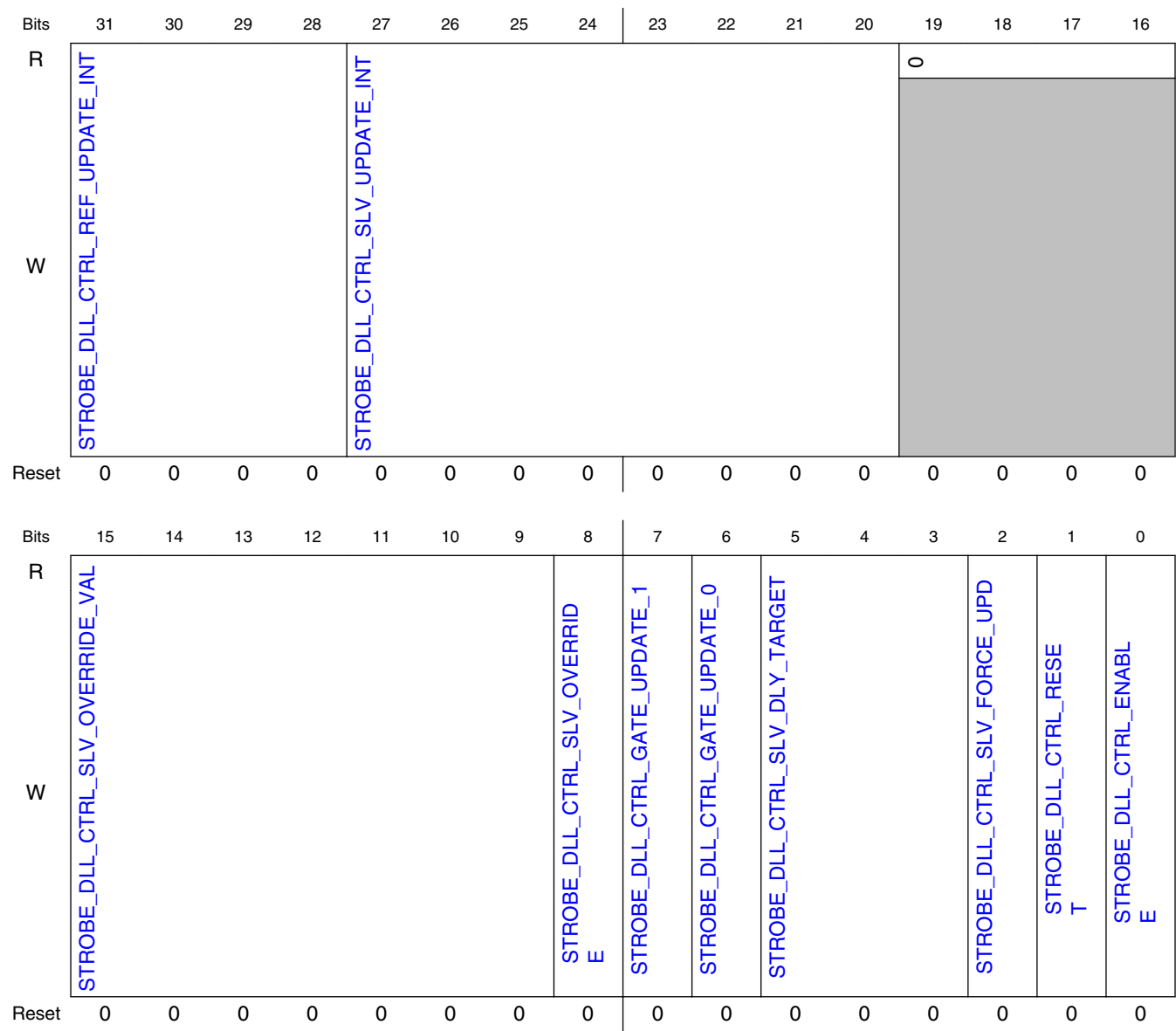
10.3.7.1.27.1 Offset

Register	Offset
STROBE_DLL_CTRL	70h

10.3.7.1.27.2 Function

This register contains the strobe DLL control information.

10.3.7.1.27.3 Diagram



10.3.7.1.27.4 Fields

Field	Function
31-28	Strobe DLL control reference update interval
STROBE_DLL_CTRL_REF_UPDATE_INT	The interval cycle is (2 + STROBE_REF_UPDATE_INT) * STROBE_REF_CLOCK. By default, the DLL control loop updates every two STROBE_REF_CLOCK cycles.

Table continues on the next page...

Field	Function
	NOTE: Increasing the reference delay-line update interval reduces the ability of the DLL to adjust to fast changes in conditions that might effect the delay (such as voltage and temperature)
27-20 STROBE_DLL_CTRL_SLV_UPDATE_INT	Strobe DLL control slave update interval Slave delay line update interval. If default 0 is used, it means 256 cycles of STROBE_REF_CLOCK. A value of 0x0F results in 15 cycles and so on. NOTE: software can always cause an update of the slave-delay line using the STROBE_SLV_FORCE_UPDATE register. The slave delay line also updates automatically when the reference DLL transitions to a locked state (from an un-locked state).
19-16 —	This field is reserved. This read-only field is reserved and always has the value 0.
15-9 STROBE_DLL_CTRL_SLV_OVERRIDE_VAL	Strobe DLL control slave Override value When STROBE_SLV_OVERRIDE = 1, this field is used to manually select one of 128 physical taps. A value of 0 selects tap 1, and a value of 0x7F selects tap 128.
8 STROBE_DLL_CTRL_SLV_OVERRIDE	Strobe DLL control slave override Set this field to 1 to Enable manual override for slave delay chain using STROBE_SLV_OVERRIDE_VAL; set this field to 0 to disable manual override. This feature does not require the DLL to be enabled using the ENABLE field. In fact to reduce power, if STROBE_SLV_OVERRIDE is used, it is recommended to disable the DLL with ENABLE = 0.
7 STROBE_DLL_CTRL_GATE_UPDATE_1	Strobe DLL control gate update Set this field to 1 to prevent the DLL from updating (because when STROBE_CLOCK_IN exists, glitches might appear during DLL updates). This field can be used by software if such a condition occurs. Clear the field to 0 to allow the DLL to update automatically.
6 STROBE_DLL_CTRL_GATE_UPDATE_0	Strobe DLL control gate update Set this field to 1 to prevent the DLL from updating (because when STROBE_CLOCK_IN exists, glitches might appear during DLL updates). This field can be used by software if such a condition occurs. Clear the field to 0 to allow the DLL to update automatically.
5-3 STROBE_DLL_CTRL_SLV_DELAY_TARGET	Strobe DLL Control Slave Delay Target The delay target for uSDHC loopback read clock can be programmed in 1/16th increments of an STROBE_REF_CLOCK half-period. The delay is $((\text{STROBE_DLL_CTRL_SLV_DLY_TARGET} + 1) * \text{STROBE_REF_CLOCK} / 2) / 16$, So the input read-clock can be delayed relative input data from $(\text{STROBE_REF_CLOCK} / 2) / 16$ to $(\text{STROBE_REF_CLOCK} * 4) / 16$.
2 STROBE_DLL_CTRL_SLV_FORCE_UPDATE	Strobe DLL control slave force updated Setting this field to 1, forces the slave delay line to update to the DLL calibrated value immediately. The slave delay line should automatically update the STROBE_SLV_UPDATE_INT interval or when a DLL lock condition is sensed. Subsequent forcing of the slave-line update can only occur if STROBE_SLV_FORCE_UP is set back to 0 and then asserted again (edge triggered). Be sure to use it when uSDHC is idle. This function might not work when uSDHC is working on data / cmd / response.
1 STROBE_DLL_CTRL_RESET	Strobe DLL control reset Setting this field to 1 to force a reset on DLL. This causes the DLL to lose lock and re-calibrate to detect an REF_CLOCK half period phase shift. This signal is used by the DLL as edge-sensitive, to create a subsequent reset, RESET must be taken low and then asserted again.
0 STROBE_DLL_CTRL_ENABLE	Strobe DLL control enable Set this field to 1 to enable the DLL and delay chain; otherwise, set to 0 to bypasses DLL. NOTE: Using the slave delay line override feature with STROBE_SLV_OVERRIDE and STROBE_SLV_OVERRIDE VAL, the DLL does not need to be enabled.

10.3.7.1.28 Strobe DLL status (STROBE_DLL_STATUS)

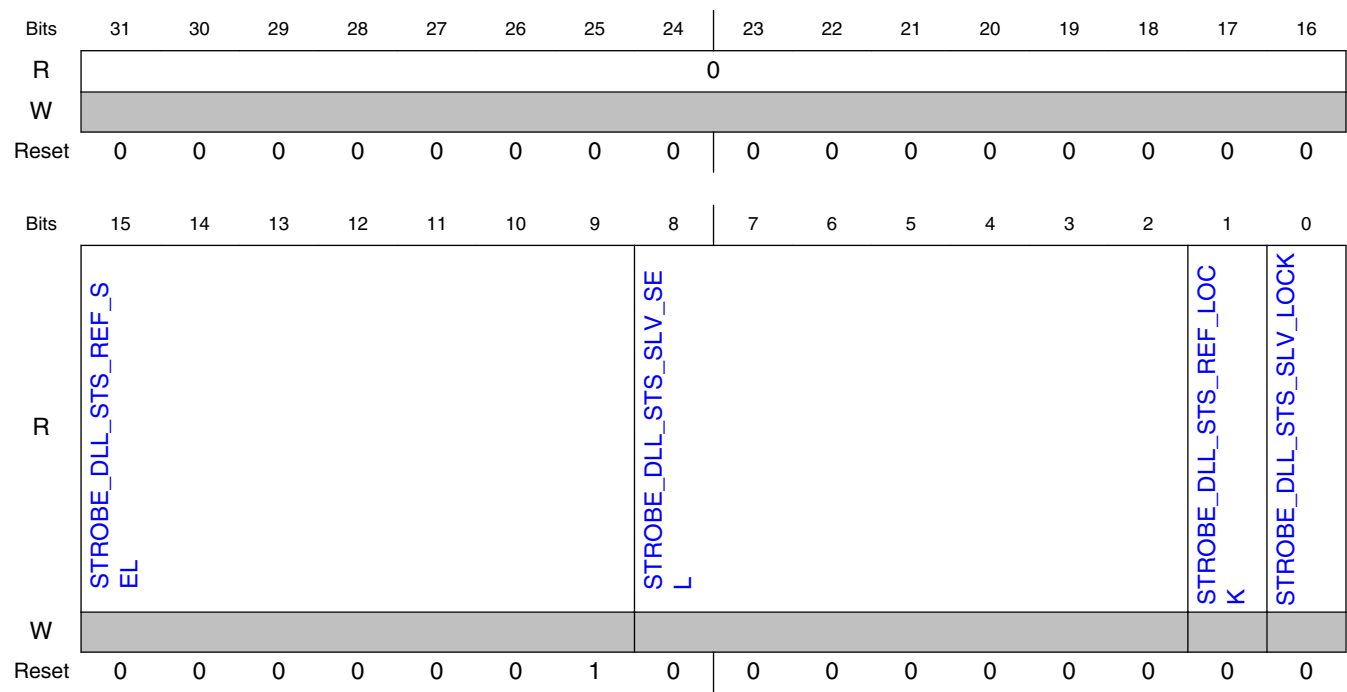
10.3.7.1.28.1 Offset

Register	Offset
STROBE_DLL_STATUS	74h

10.3.7.1.28.2 Function

This register contains the strobe DLL status information. All fields are read only and read the same as the power-reset value.

10.3.7.1.28.3 Diagram



10.3.7.1.28.4 Fields

Field	Function
31-16	This field is reserved.
—	This read-only field is reserved and always has the value 0.
15-9	Strobe DLL status reference select

Table continues on the next page...

Field	Function
STROBE_DLL_STS_REF_SEL	Reference delay line select taps. This is encoded by 7 fields for 127 taps.
8-2 STROBE_DLL_STS_SLV_SEL	Strobe DLL status slave select Slave delay line select status. This is the instant value generated from reference chain. Because the reference chain can only be updated when STROBE_REF_CLOCK is detected, this value can be updated with the right value when the reference is locked.
1 STROBE_DLL_STS_REF_LOCK	Strobe DLL status reference lock This signifies that the DLL has detected and locked to a half-phase REF_CLOCK shift, it allows the slave delay-line to perform programmed clock delays.
0 STROBE_DLL_STS_SLV_LOCK	Strobe DLL status slave lock Slave delay-line lock status. This signifies that a valid calibration has been set to the slave-delay line, and the slave-delay line is implementing the programmed delay value.

10.3.7.1.29 Vendor Specific Register (VEND_SPEC)

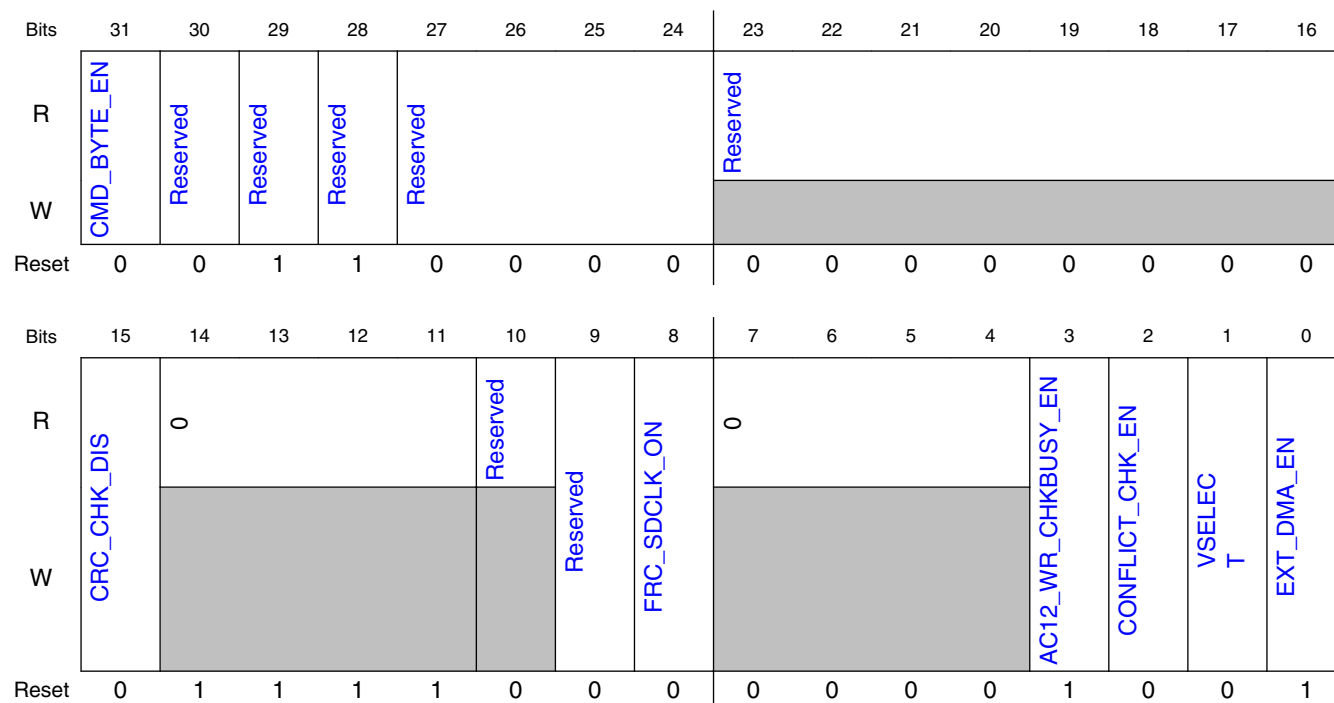
10.3.7.1.29.1 Offset

Register	Offset
VEND_SPEC	C0h

10.3.7.1.29.2 Function

This register contains the vendor specific control / status register.

10.3.7.1.29.3 Diagram



10.3.7.1.29.4 Fields

Field	Function
31 CMD_BYTE_EN	Byte access This bit controls the byte access. 0b - Disable 1b - Enable
30 —	Reserved Always write as 0.
29 —	Reserved Always write as 1
28 —	Reserved Always write as 0.
27-24 —	Reserved Always write as 4'b0000.
23-16 —	Reserved Always write as 8'h00.
15 CRC_CHK_DIS	CRC Check Disable 0b - Check CRC16 for every read data packet and check CRC fields for every write data packet 1b - Ignore CRC16 check for every read data packet and ignore CRC fields check for every write data packet
14-11	Reserved

Table continues on the next page...

Field	Function
—	Reserved
10	Reserved
—	Always write as 0.
9	Reserved
—	Always write as 0.
8	Force CLK
FRC_SDCLK_ON	Force CLK output active 0b - CLK active or inactive is fully controlled by the hardware. 1b - Force CLK active
7-4	Reserved
—	
3	Check busy enable
AC12_WR_CHK_BUSY_EN	Check busy enable after auto CMD12 for write data packet 0b - Do not check busy after auto CMD12 for write data packet 1b - Check busy after auto CMD12 for write data packet
2	Conflict check enable
CONFLICT_CHK_EN	It is not implemented in uSDHC IP. 0b - Conflict check disable 1b - Conflict check enable
1	Voltage selection
VSELECT	Change the value of output signal VSELECT, to control the voltage on pads for external card. There must be a control circuit out of uSDHC to change the voltage on pads. 0b - Change the voltage to high voltage range, around 3.0 V 1b - Change the voltage to low voltage range, around 1.8 V
0	External DMA request enable
EXT_DMA_EN	Enable the request to external DMA. When the internal DMA (either Simple DMA or Advanced DMA) is not in use, and this field is set, uSDHC sends out DMA request when the internal buffer is ready. This field is particularly useful when transferring data by Arm platform polling mode, and it is not allowed to send out the external DMA request. By default, this field is set. 0b - In any scenario, uSDHC does not send out external DMA request. 1b - When internal DMA is not active, the external DMA request is sent out.

10.3.7.1.30 MMC Boot (MMC_BOOT)

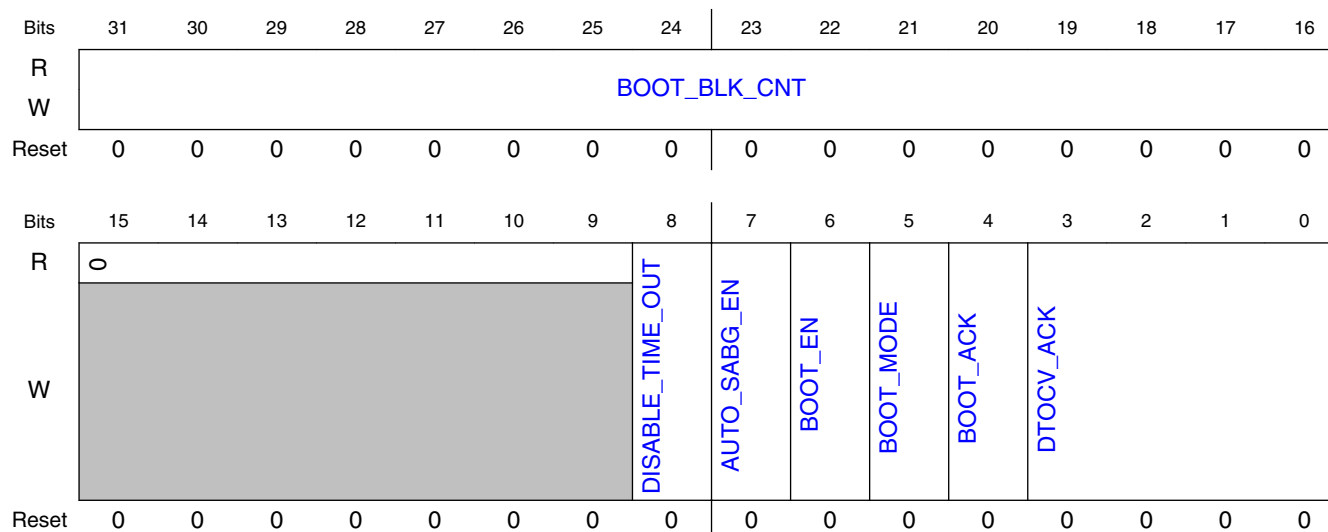
10.3.7.1.30.1 Offset

Register	Offset
MMC_BOOT	C4h

10.3.7.1.30.2 Function

This register contains the MMC Fast Boot control register.

10.3.7.1.30.3 Diagram



10.3.7.1.30.4 Fields

Field	Function
31-16 BOOT_BLK_CNT	Stop At Block Gap value of automatic mode The value defines the Stop At Block Gap value of automatic mode. When received, card block cnt is equal to (BLK_CNT - BOOT_BLK_CNT) and AUTO_SABG_EN is 1, then Stop At Block Gap. Here, BLK_CNT is defined in the Block Attributes Register, field 31 - 16 of 0x04.
15-9 —	Reserved
8 DISABLE_TIME_OUT	Time out NOTE: When this field is set, there is no timeout check no matter whether BOOT_EN is set or not. 0b - Enable time out 1b - Disable time out
7 AUTO_SABG_EN	Auto stop at block gap During boot, enable auto stop at block gap function. This function is triggered, and host stops at block gap when received card block cnt is equal to (BLK_CNT - BOOT_BLK_CNT).
6 BOOT_EN	Boot enable Boot mode enable 0b - Fast boot disable 1b - Fast boot enable
5 BOOT_MODE	Boot mode Boot mode select

Table continues on the next page...

Field	Function
	0b - Normal boot 1b - Alternative boot
4 BOOT_ACK	BOOT ACK Boot ACK mode select 0b - No ack 1b - Ack
3-0 DTCV_ACK	DTCV_ACK Boot ACK time out counter value 0000b - SDCLK x 2 ³² 0001b - SDCLK x 2 ³³ 0010b - SDCLK x 2 ¹⁸ 0011b - SDCLK x 2 ¹⁹ 0100b - SDCLK x 2 ²⁰ 0101b - SDCLK x 2 ²¹ 0110b - SDCLK x 2 ²² 0111b - SDCLK x 2 ²³ 1110b - SDCLK x 2 ³⁰ 1111b - SDCLK x 2 ³¹

10.3.7.1.31 Vendor Specific 2 Register (VEND_SPEC2)

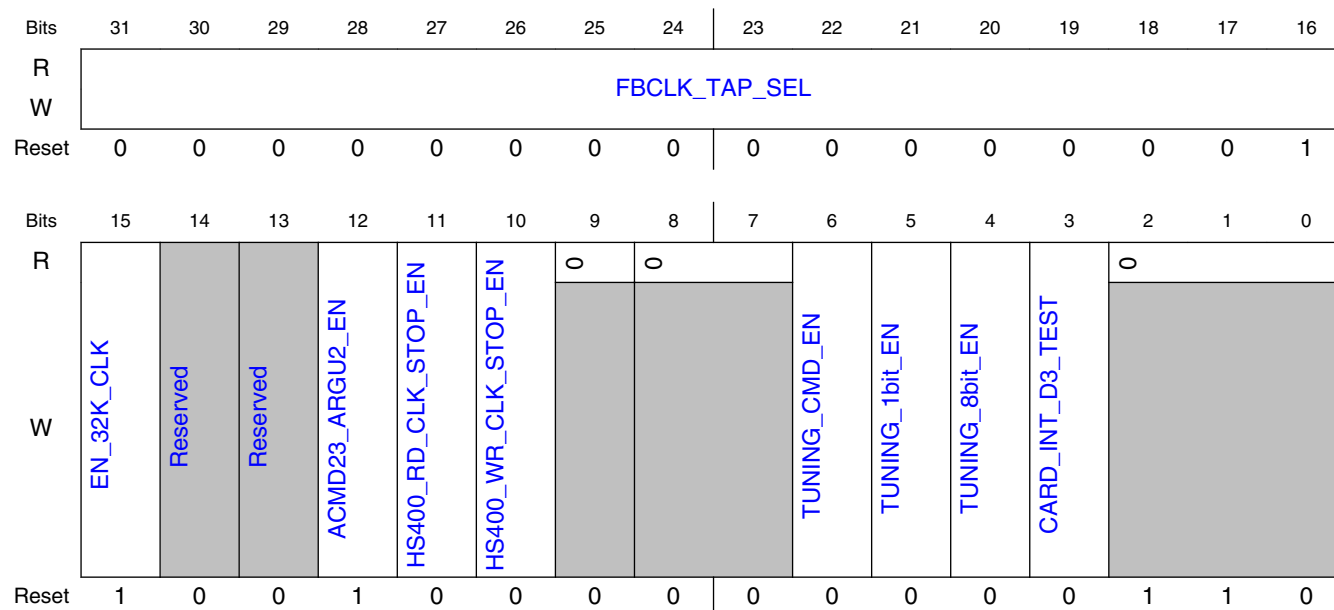
10.3.7.1.31.1 Offset

Register	Offset
VEND_SPEC2	C8h

10.3.7.1.31.2 Function

This register contains the vendor specific control 2 register.

10.3.7.1.31.3 Diagram



10.3.7.1.31.4 Fields

Field	Function
31-16 FBCLK_TAP_SEL	Enable extra delay on internal feedback clock
15 EN_32K_CLK	Enable 32khz clock for card detection Use 32khz clock for card detection
14 —	Reserved
13 —	Reserved
12 ACMD23_ARGU2_EN	Argument2 register enable for ACMD23 0b - Disable 1b - Argument2 register enable for ACMD23 sharing with SDMA system address register. Default is enabled.
11 HS400_RD_CLK_STOP_EN	HS400 read clock stop enable Only stop clock at read block gap.
10 HS400_WR_CLK_STOP_EN	HS400 write clock stop enable Only stop clock at write block gap.
9 —	Reserved

Table continues on the next page...

Field	Function
8-7 —	Reserved
6 TUNING_CMD_EN	Tuning command enable Enable the auto tuning circuit to check the CMD line. 0b - Auto tuning circuit does not check the CMD line. 1b - Auto tuning circuit checks the CMD line.
5 TUNING_1bit_EN	Tuning 1bit enable Enable the auto tuning circuit to check the DATA0 only. It is used with the TUNING_8bit_EN together.
4 TUNING_8bit_EN	Tuning 8bit enable Enable the auto tuning circuit to check the DATA[7:0]. It is used with the TUNING_1bit_EN together. 0b00 - Tuning circuit only checks the DATA[3:0] 0b01 - Tuning circuit only checks the DATA0 0b10 - Tuning circuit checks the whole DATA[7:0] 0b11 - Invalid NOTE: The format of these two fields are [TUNNING_8bit_EN:TUNNING_1bit_EN].
3 CARD_INT_D3_TEST	Card interrupt detection test This field only uses for debugging. 0b - Check the card interrupt only when DATA3 is high. 1b - Check the card interrupt by ignoring the status of DATA3.
2-0 —	Reserved

10.3.7.1.32 Tuning Control (TUNING_CTRL)

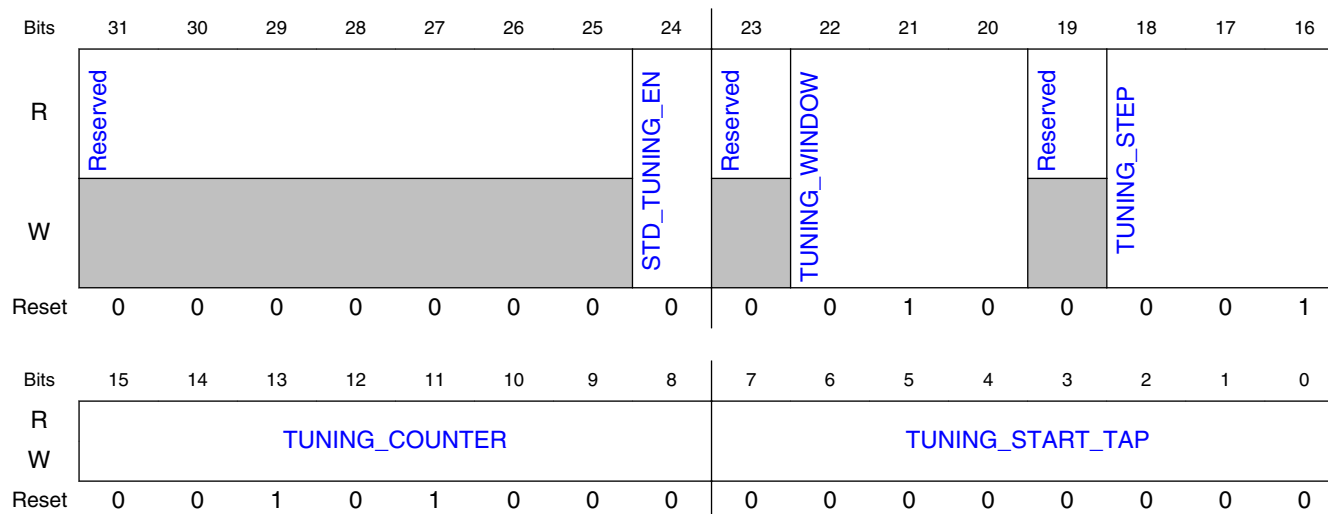
10.3.7.1.32.1 Offset

Register	Offset
TUNING_CTRL	CCh

10.3.7.1.32.2 Function

The register contains configuration of tuning circuit.

10.3.7.1.32.3 Diagram



10.3.7.1.32.4 Fields

Field	Function
31-25 —	Reserved
24 STD_TUNING_EN	Standard tuning circuit and procedure enable This field is used to enable standard tuning circuit and procedure.
23 —	Reserved
22-20 TUNING_WINDOW	Data window Select data window value for auto tuning
19 —	Reserved
18-16 TUNING_STEP	TUNING_STEP The increasing delay cell steps in tuning procedure.
15-8 TUNING_COUNTER	Tuning counter The MAX repeat CMD19 times in tuning procedure.
7-0 TUNING_START_TAP	Tuning start The start delay cell point when send first CMD19 in tuning procedure.

10.3.7.1.33 Command Queue (CQE)

10.3.7.1.33.1 Offset

Register	Offset
CQE	100h

10.3.7.1.33.2 Function

All the CQE registers are defined in JESD84-B51.pdf. See the B.4 section of JESD84-B51.pdf

10.3.7.1.33.3 Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0															
W																
Reset	0	0	0	0	0	1	0	1	0	0	0	1	0	0	0	0

10.3.7.1.33.4 Fields

Field	Function
31-0	Reserved
—	

